

ARRAKIS

Harvester Down

Complete Code and Functionality Explanation

Plain-language walkthroughs, mathematics, tests, build details,
and complete numbered source references

CSE 4102: Computer Graphics and Image Processing Laboratory

Documented source snapshot: 2026-10-09 UTC

Contents

1. Project, Architecture, and Build Configuration	6
What This Project Actually Does	6
How to Read the Detailed Explanations	6
Repository Organization and Cleanup Decisions	6
C++ Foundations Used Throughout the Code	7
CMakeLists.txt: Lines 1-18, Project and Compilation Units	8
CMakeLists.txt: Lines 20-30, Dependency Discovery and Include Paths	8
CMakeLists.txt: Lines 32-43, Definitions and Linking	8
CMakeLists.txt: Lines 45-56, Runtime Files and Tests	9
The Alternative Visual Studio Project	9
Shared Editor and Dependency Configuration	10
Every .gitignore Rule and Why It Exists	10
Verified Cleanup Behavior and Remaining Limits	11
 Chapter 2: Terrain, Particles, Procedural Meshes, and the Renderer	 12
Reading This Chapter	12
File Setup and Shared State	13
Terrain Height and Normal Queries	15
Wind and Particle Simulation	16
Mesh Records and GPU Upload	19
Procedural Primitive Builders	21
Surface Shader Interface and Vertex Processing	26
Surface Shadow Filtering	28
Surface Material and Lighting Calculations	29
Shadow and Particle Shader Programs	31
Procedural Sky	32
Shader Compilation and GPU Targets	33
Draw Helpers and Materials	36
Shared Color Palette	37
Implementation Boundaries	38
 Scene Assembly and Application Runtime	 40
Reading Transforms and Materials	40
Ornithopter Construction	41
Harvester Construction	45
Spice Tank Construction	47
Desert Rock Construction	49
Rescue Objects and People	49
Master Scene Traversal	52
GLFW Input Callbacks	53

Screenshot File Encoding	55
Application Startup	56
Frame Timing and Simulation	59
Camera System	60
Spatial Audio Handoff	62
Shadow Rendering Pass	63
Main Rendering Pass	64
HUD State Assembly	66
Render Smoke Tests and Frame Presentation	67
Resource Teardown and Exit	68
Coverage Ledger	69
Chapter 4: Mission Rules, Flight Controls, and Tests	72
Reading the Mission	72
Game Types and Storage	73
World and Input Helpers	74
Starting and Finishing Attempts	76
The Active Update	78
Test Harness and Fixtures	82
Reset, Population, and Pickup Cases	84
Failure, Retry, and Completion Cases	86
Input-Only Rescue Scenario	88
Analytic Route and Hazard Cases	91
Cursor and Steering Cases	93
Camera Cases and Test Result	95
Important Limits	96
Coverage Reference	96
Chapter 5: The Rescue HUD and the Procedural Sandworm	98
Reading This Chapter	98
The HUD Header and Its Input Contract	98
HUD Data, Colors, and Formatting	99
Preserving OpenGL State	100
Turning HUD Shapes Into Triangles	101
The Complete Bitmap Font	102
The North-Up Radar	106
Title, Pause, and Results Screens	107
Composing the In-Game HUD	108
HUD Initialization, Drawing, and Cleanup	110
The Sandworm's Procedural Geometry	112
Shared Surface Construction	114
The Thousand Curved Teeth	115

Worm Pose and Attack Phases	117
Deforming the Flexible Body	118
Creating and Releasing All Worm Meshes	120
Drawing, Cache Reuse, and Materials	121
The Moving Sand Wake	123
Numerical and Design Checks	124
Complete Source Coverage Index	125
6. Spatial Audio and Vendor Implementations	129
6.1 Scope, Versions, and Reading Conventions	129
6.2 arrakis_audio.h: Includes, File Lookup, and State	130
6.3 Initialization, Lambda, Candidates, and Optional Assets	132
6.4 update: Parameters, Math, Every Branch, and Event Semantics	135
6.5 Mute, Mission Reset, Shutdown, and API-Lifetime Details	140
6.6 miniaudio_impl.cpp and Compile-Time Selection	142
6.7 miniaudio: Complete Coverage Map and Functional Architecture	143
6.8 GLAD: Generated Loader, Every Version, and Platform Branch	155
6.9 Historical Vorbis Study: Removed, Never Integrated	160
6.10 API Reference and Annotated-Appendix Strategy	164
7. The Documentation Generator, Line by Line	167
Purpose and Boundaries	167
Lines 1-23: Module Setup and Imports	167
Lines 25-39: Exact Input Lists	167
Lines 42-54: inline, Safe Inline Formatting	167
Lines 57-68: source_rows, Preserve Every Source Line	168
Lines 71-109: ExplanationDocument and PDF Navigation	168
Lines 112-131: make_styles, Typography	168
Lines 134-138: heading, Document Hierarchy	169
Lines 141-206: markdown_flowables, Each Parsing Branch	169
Lines 209-226: Arguments and Input Validation	169
Lines 228-247: Cover, Contents, and Chapter Text	170
Lines 249-267: Snapshot Manifest and Source-Reference Rules	170
Lines 269-286: Complete Source Appendices and PDF Verification	170
How to Update and Validate the Manual	170
Appendix A: Source Snapshot and Reading Rules	172
Source Reference: src/arrakis.cpp	173
Source Reference: src/arrakis_game.h	199
Source Reference: src/arrakis_hud.h	204

Source Reference: <code>src/arrakis_worm.h</code>	212
Source Reference: <code>src/arrakis_audio.h</code>	217
Source Reference: <code>tests/arrakis_tests.h</code>	221
Source Reference: <code>CMakeLists.txt</code>	237
Source Reference: <code>Arrakis_001.vcxproj</code>	238
Source Reference: <code>Arrakis_001.vcxproj.filters</code>	241
Source Reference: <code>.gitignore</code>	242
Source Reference: <code>.vscode/c_cpp_properties.json</code>	243
Source Reference: <code>.vscode/launch.json</code>	244
Source Reference: <code>vcpkg.json</code>	245
Source Reference: <code>tools/generate_docs.py</code>	246
Source Reference: <code>tools/requirements.txt</code>	250
Source Reference: <code>third_party/miniaudio/miniaudio_impl.cpp</code>	251
Source Reference: <code>third_party/glad/glad.c</code>	252
Source Reference: <code>third_party/miniaudio/miniaudio.h</code>	278

1. Project, Architecture, and Build Configuration

What This Project Actually Does

Arrakis: Harvester Down is an interactive, real-time three-dimensional rescue game. The player pilots an ornithopter above a mathematical desert. A moving spice harvester periodically releases workers while a sandworm pursues it. The player must fly to each group, slow down, descend, hold the rescue control, carry a limited number of workers, and deliver them to a landing pad. A worker merely collected aboard is not yet a scored survivor. The game contains a title screen, timed mission, pause overlay, debrief, retry, escalating later missions, and a best score retained in memory.

The program is not a 3D-model viewer and does not parse OBJ files. Its cubes, cylinders, spheres, aerofoil blades, desert grid, rings, eroded rocks, people, worm body, mouth, and teeth are generated in code. Colored materials and procedural shaders replace image textures for world objects. Sound files are the actual disk-loaded runtime assets. The original screenshots, worm photograph, and proposal in docs/reference/ are human references, not inputs used to draw a frame.

The implementation is C++17 with an OpenGL 3.3 core graphics context. GLFW creates the window, receives events, reports time, and provides driver procedure addresses. GLAD turns those addresses into callable OpenGL functions. GLM supplies vectors, matrices, angle conversion, interpolation, and camera transforms. Miniaudio decodes and spatializes sounds, mixes them, and feeds an operating-system audio device. Python and ReportLab are documentation-generation dependencies; neither is required to play the game.

How to Read the Detailed Explanations

Chapters 2 and 3 explain `src/arrakis.cpp`, including the shader programs embedded inside C++ strings. Chapter 4 explains every mission field, state, helper, update path, and deterministic test in `src/arrakis_game.h` and `tests/arrakis_tests.h`. Chapter 5 explains the HUD, bitmap font, radar, and sandworm mesh/animation code. Chapter 6 explains the audio integration in detail and the retained vendor libraries by functional family. Chapter 7 explains the PDF generation tool itself. The numbered appendices include complete retained code and configuration, including miniaudio's large vendor implementation.

Code-range headings refer to physical source-file line numbers. Their purpose is to make small implementation details locatable, not to turn comments or braces into imaginary features. A range can contain several repeated draw calls or assertions; the explanation accounts for what their individual parameters or conditions do. Blank lines provide visual separation and execute nothing. A comment documents an intention but does not run; when a comment and executable behavior disagree, this explanation prioritizes the behavior. The complete source snapshot, file hashes, and source line counts are supplied in the appendix and `docs/documentation_manifest.json`.

Application code is explained much more finely than the third-party libraries: the latter are grouped by algorithms and API families, with their existing comments and full source retained. A library line listing is a reference, not a claim that every branch in every operating-system backend has been rewritten as a unique prose explanation. This distinction matters because miniaudio alone contains approximately 96,000 lines and many backends that this Windows game never executes. External installed GLFW, GLM, and GLAD headers are not project-authored files and are not bundled by this cleanup.

Repository Organization and Cleanup Decisions

`src/` holds five application source/header files. `src/arrakis.cpp` remains the executable's main translation unit: it introduces shared globals and drawing types, then includes project headers where their dependent names are available. The headers are not independently compiled C++ libraries. In particular, the game header assumes terrain and flight state have already been declared, and the worm header assumes rendering helpers exist. This arrangement is preserved instead of introducing a risky architecture rewrite during folder cleanup.

`tests/arrakis_tests.h` contains deterministic logic tests. The main program includes it through a dedicated compiler include directory. Running `--test-game` returns before GLFW window or audio initialization, so tests do not need a desktop. The test code still links into the same executable. The cleanup does not claim to have extracted a

separate test library.

`third_party/glad/` retains the generated loader implementation; `third_party/miniaudio/` retains the audio header and its two-line implementation translation unit. These files are runtime dependencies, not unnecessary downloads. Their license statements and embedded documentation remain intact. `assets/audio/` holds the eight primary sounds requested by the application plus the retained Kenney license notice. Retaining that notice does not establish the provenance or licensing of every unrelated MP3 file; the repository contains no complete per-file sound attribution manifest.

`docs/chapters/` contains this editable explanation. `docs/reference/` preserves the original proposal PDF, `arrakis.png`, and `sandworm.jpg`. `tools/` contains the PDF generator and Python dependency constraints. CMake, the `vcpkg` manifest, README, alternative Visual Studio project, and shared editor settings stay at the root because they are entry points to building or understanding the repository.

The removed folders were `dune`, `spice harvester/`, `low_poly_rocks/`, `ornithopter/`, `spice tank/`, and `thumper/`. They contained unused imported geometry, materials, and textures. Nothing in the renderer loads them; retaining them would imply a nonexistent model-loading pipeline. The old `build/`, `x64/`, and `vcpkg_installed/` folders were generated caches/output rather than source. A new ignored `build/rescue/` is produced when verifying the organized project. It is intentional that verification leaves a runnable build rather than pretending builds never create files.

The unused `stb_vorbis.c`, redundant Ogg delivery sound, and downloaded Kenney fallback pack were removed. The miniaudio implementation never enabled its optional stb Vorbis adapter; the retained selected sounds are WAV or MP3. The audio candidate lists still allow alternate filenames, including Ogg names, but such fallbacks are not included and the default build has no enabled Vorbis decoder. The previous `CODE_EXPLANATION.txt` and historical README sections described an earlier renderer and contradictory controls; the current detailed chapters replace those notes rather than preserving two conflicting manuals. The machine-specific Visual Studio `.user` file and unrelated C/C++ Runner editor settings were removed; the debugger launch was corrected to the actual MSVC executable.

C++ Foundations Used Throughout the Code

An `int` stores integer counts or selections. A `float` stores a single-precision real value such as elapsed seconds, speed, height, or color. A `bool` stores true or false. A structure groups related fields, for example a particle's position, velocity, lifetime, size, and opacity. `enum class` gives names to state choices and prevents accidentally confusing one enumeration with another. `std::vector` is a resizable contiguous container; the code uses it for vertices, indices, workers, groups, and particles. `std::array` is fixed-size. A `std::string` owns text, such as a sound filename or HUD message.

The `g_` naming convention marks global application state. Globals are shared across functions and included headers; they are not automatically thread-safe. Most game/rendering code executes on the main thread. Miniaudio also runs its own processing threads internally. A reference parameter such as `const Mesh&` avoids copying while disallowing changes through that parameter. A non-const reference allows a helper to modify the caller's object. A pointer represents an address and can be null. OpenGL object IDs are integer handles into driver-managed resources, not pointers to their data.

Static variables inside functions retain their values across calls. That is why a seeded random generator produces a deterministic sequence rather than restarting every frame, and why cached uniform locations avoid repeated lookups. At namespace scope, `static` gives internal linkage. `inline` permits definitions in a header to satisfy C++'s multiple-definition rules; it is not a guarantee that the compiler literally substitutes the function's body at every call. `#pragma once` or a matching `#ifndef/#define` header guard prevents the same header from being included twice into one translation unit.

Floating-point literals ending in `f`, such as `0.35f`, specify float precision. Vector arithmetic applies component-wise or according to GLM's defined operators. `std::sin` and `std::cos` expect radians; aircraft headings are stored in degrees and converted before trigonometry or rotations. A `glm::vec2` often represents horizontal coordinates, a `glm::vec3` a world point/direction or RGB color, a `glm::vec4` homogeneous coordinates or RGBA. Normalizing changes a nonzero vector to unit length without changing direction. Clamping restricts a value to a permitted interval.

Linear interpolation `mix(a, b, t)` computes $a * (1 - t) + b * t$; exponential interpolation factors such as `1 - exp(-rate*dt)` implement a time-based approach to a target.

Triangles are the basic filled rendering primitive. A vertex contains position and other per-vertex attributes. An index selects a vertex; indexed triangles reuse data. A shader is a GPU program; GLSL vertex shaders transform points, and fragment shaders decide the color for a covered pixel/sample. A model matrix converts local object coordinates to world space, a view matrix converts the world into camera space, and a projection matrix produces clip-space coordinates. Matrix multiplication order matters. An inverse-transpose normal matrix preserves normal direction under nonuniform scaling. A framebuffer directs rendering to either the screen or off-screen attachments such as the shadow depth texture.

CMakeLists.txt: Lines 1-18, Project and Compilation Units

Line 1 requires CMake 3.20 or newer; an older installation stops configuration rather than attempting unsupported syntax. Line 3 names the project Arrakis and, without a `LANGUAGES` restriction, enables the normal C and C++ compiler discovery needed by the C loader and C++ application. Lines 5-6 request C++17 and require it instead of silently falling back to an older language mode.

Lines 8-18 create an executable target called Arrakis. `src/arrakis.cpp` is compiled as C++; `third_party/glad/glad.c` is compiled as C; `third_party/miniaudio/miniaudio_impl.cpp` is compiled as C++ and instantiates miniaudio's implementation exactly once. The four application headers and the test header are listed as target sources primarily for build dependency tracking and IDE visibility. A header is not an additional independently linked object just because it appears in this list.

CMakeLists.txt: Lines 20-30, Dependency Discovery and Include Paths

Line 20 requires OpenGL and provides the imported `OpenGL::GL` target. On this Windows installation it links the system OpenGL library. Lines 22-24 search for `glad/glad.h`, `GLFW/glfw3.h`, and `glm/glm.hpp`. The specified E-drive directories are hints; `REQUIRED` makes configuration fail when a dependency cannot be found. GLM additionally searches the optional manifest-installed `vcpkg_installed/x64-windows/include` directory. Removing the installed cache does not remove `vcpkg.json`, so it can be regenerated. The successfully verified setup has GLM available at `E:/glm` independently of that cache.

Line 25 searches for a GLFW binary library named `glfw3` or `glfw`. Headers tell the compiler what functions exist, but a library supplies their linked machine code; both are necessary. Lines 26-30 apply discovered include directories only to this executable, using `PRIVATE` rather than publishing them to consumers. The explicit tests include directory resolves `arrakis_tests.h` after its move. The miniaudio include directory resolves `miniaudio.h` from the application audio header; the implementation file also finds it in its own directory. Project headers alongside `src/arrakis.cpp` continue to resolve through normal quoted-include search rules.

Cache overrides such as `-DGLFW_LIBRARY=C:/deps/lib/glfw3.lib` let another machine supply its installation. An include-directory override must point at the parent of `GLFW/`, `glad/`, or `glm/`, not at the individual header file. The retained GLAD C loader was generated for core GL 4.6 even though the application requests a 3.3 context. Its matching declarations are required for compilation; the context's supported version determines which function groups actually load. A randomly different GLAD generation can therefore fail to compile this loader even when the application only uses 3.3 features.

CMakeLists.txt: Lines 32-43, Definitions and Linking

`target_compile_definitions` passes `GLFW_INCLUDE_NONE` to compilation. It asks GLFW not to include a competing system OpenGL header, because GLAD must provide the function declarations and pointers. `PRIVATE` restricts the definition to this executable. `target_link_libraries` links GLFW and `OpenGL::GL`. Miniaudio's platform backend implementation comes from the vendored implementation unit; it is not a separately downloaded miniaudio library. GLM is header-only and does not need a GLM binary here.

The verified Release build emits linker warning LNK4098 about `MSVCRTD` conflicting with other runtime libraries. This is a property of the existing local library/runtime combination, not a missing source file introduced by the reorganization.

The executable and tests succeed, but rebuilding GLFW with the matching release runtime is preferable to hiding the diagnostic. The project does not add `/NODEFAULTLIB` to conceal it.

CMakeLists.txt: Lines 45-56, Runtime Files and Tests

Lines 46-50 add a post-build command to the actual executable target. CMake invokes its own executable using the cross-platform `-E copy_directory` helper, copying the complete `assets/audio` directory under the chosen executable directory. `$(TARGET_FILE_DIR:Arrakis)` is a generator expression evaluated for the selected configuration, so a Visual Studio Release build receives files beneath `Release/assets/audio/`, and Debug receives its own copy. It avoids a hardcoded configuration or user path.

Line 51 sets the generated Visual Studio debugger's working directory to the executable directory. The audio lookup first checks `assets/audio/` relative to its current working directory, then four ancestor directories. This supports root launches, normal build-directory launches, and packaged launches from the executable directory. The implementation still uses relative working-directory lookup, not an absolute path derived from the executable; arbitrary unrelated working directories are not guaranteed to resolve sounds. Failure to resolve a sound does not terminate gameplay.

Line 53 loads CTest integration and the standard `BUILD_TESTING` option. Lines 54-56 register `RescueMissionLogic` only when testing is enabled. The command references the executable target and passes `--test-game`; CMake resolves the right built executable for the selected configuration. `ctest --test-dir build/rescue -C Release --output-on-failure` executes it and reports details on failure. No graphical context is requested for this command.

The Alternative Visual Studio Project

`Arrakis_001.vcxproj` is MSBuild XML. The XML declaration identifies encoding. `<Project DefaultTargets="Build">` selects the default target and the Visual C++ MSBuild namespace. The `ProjectConfigurations` item group defines four pairs: `Debug/Win32`, `Release/Win32`, `Debug/x64`, and `Release/x64`. Each pair records both configuration and platform. The globals record Visual C++ project version 17, project identity GUID, `Win32Proj` keyword, root namespace, and the Windows 10 SDK family. The GUID identifies this project in the IDE; it does not change gameplay.

The first imported Microsoft defaults file supplies standard properties. Four conditional configuration groups choose an Application executable, debug-library selection, toolset v143, Unicode character set, and Release whole-program optimization. Unicode here selects the Windows character-set build property; it does not automatically make every game string Unicode. `Microsoft.Cpp.props` brings in further toolchain settings. Empty extension/shared import groups are standard places for optional tool integration. The four property-sheet import groups allow per-user platform properties if their files exist. The empty user-macros group is a conventional customization point.

The x64 property groups retain E-drive `include/library` paths. `Release/x64` also allows optional `vcpkg`-installed GLM headers; `Debug/x64` uses the laboratory GLM path. These are local-installation conveniences, not portable dependency provisioning. Although Win32 configurations are listed, corresponding 32-bit dependency paths are not configured here; the verified build is x64 through CMake, not a claim that all four standalone configurations are independently tested.

The common item-definition group sets `C++17`, `GLFW_INCLUDE_NONE`, and the relocated `tests/miniaudio` include directories. Its post-build event uses `xcopy /E /I /Y: /E` includes subdirectories, `/I` treats the destination as a directory, and `/Y` overwrites without prompting. The command quotes directories to survive spaces in the project path and places audio below `$(OutDir)`. This event is Windows-specific; the primary CMake copy helper is cross-platform.

Each configuration's compiler group sets warning level 3, SDL checks, relevant console/debug/release macros, and conformance mode. Release additionally enables function-level linking and intrinsic functions. Link settings select a console executable and generate debug information. The x64 linker groups add `glfw3.lib` and `opengl32.lib`, preserving inherited dependencies with `%(AdditionalDependencies)`. `$(...)` is an MSBuild property substitution; `%(...)` refers to inherited item metadata. The source item group compiles the three relocated C/C++ units. The `None` item displays the `vcpkg` manifest without compiling it. Header items display the five relocated application/test headers. The final Microsoft C++ targets import provides actual compile/link targets; the empty extension-target group is a standard trailing integration hook.

`Arrakis_001.vcxproj.filters` is only IDE presentation. Its Source Files, Header Files, and Resource Files filters have stable identifiers and extension lists. Compile entries place the three actual source units under Source Files. Header entries place audio, game, worm, HUD, and tests under Header Files. The manifest is a None item. Filters do not move files, compile resources, or alter output; changing only the filter file never repairs a broken source path in the project itself.

Shared Editor and Dependency Configuration

`.vscode/c_cpp_properties.json` contains one IntelliSense configuration named `Win32-MSVC`. Its recursive workspace include path sees relocated headers; its additional E-drive paths expose installed GLFW/GLAD and GLM. `compilerPath` selects `cl.exe`, C language mode is `C17`, C++ language mode is `C++17`, and IntelliSense uses the Windows MSVC x64 model. The final version value 4 identifies the extension's settings schema, not the compiler version. These settings assist editing and do not replace CMake's real compilation flags.

`.vscode/launch.json` uses the debugger schema version 0.2.0. Its single configuration is named `Arrakis (CMake Release)`, has `type: cppvsdbg` for the Visual Studio Windows debugger, and `request: launch` to start rather than attach. The empty argument array launches normal gameplay. `stopAtEntry: false` avoids an automatic initial breakpoint. The integrated terminal displays console messages, the working directory is `${workspaceFolder}`, and the program path selects `build/rescue/Release/Arrakis.exe`. This configuration assumes that Release has already been built; it does not silently build or select a Debug executable.

`vcpkg.json` is a JSON manifest. Its `dependencies` array contains only `glm`; it does not install GLFW, GLAD, or miniaudio. Its `builtin-baseline` records a specific `vcpkg` registry revision for reproducible version selection. A manifest's existence alone does not run installation without a configured `vcpkg` workflow/toolchain. The existing build prints a notice that manifest mode is disabled; this is not a build failure because independently discovered GLM satisfies the dependency.

Every .gitignore Rule and Why It Exists

Git ignores are repository hygiene, not deletion instructions and not security boundaries. A pattern never removes a tracked file or prevents the program from using an ignored file already on disk. A leading `/` anchors the pattern to the repository root. A trailing `/` targets directories. `*` matches arbitrary text. A comment beginning with `#` has no matching effect.

`/build/`, `/build-*/`, `/out/`, `/x64/`, and `/x86/` cover root build/output folders. `Debug/`, `Release/`, `RelWithDebInfo/`, and `MinSizeRel/` exclude generated configuration directories at any depth. `CMakeFiles/`, `CMakeCache.txt`, `cmake_install.cmake`, `install_manifest.txt`, `CTestTestfile.cmake`, `Testing/`, and `_deps/` cover CMake discovery state, generated scripts, test output, and fetched dependencies.

`CMakeUserPresets.json` is a personal configuration file; a shared `CMakePresets.json`, if later added, remains trackable. `/vcpkg_installed/` excludes reproducible installed-package copies, while `vcpkg.json` remains included.

`*.o` and `*.a` are common Unix object/static-library outputs. `*.exe`, `*.dll`, `*.so`, `*.so.*`, `*.dylib`, and `*.lib` cover executables/shared/static libraries on Windows, Linux, and macOS. `*.pdb` and `*.idb` are debug databases; `*.ilk` is incremental-link state; `*.exp` is exported-symbol linker output; `*.pch` is a precompiled header; `*.tlog` is MSBuild tracking; `*.obj.enc` is Edit-and-Continue object state; `*.exe.recipe` is a generated executable recipe; `*.log` is log output. The broad old `*.obj` rule is deliberately absent because Wavefront 3D model files use the same extension as Windows object files. MSVC object files belong in the already ignored build directories rather than forcing all future model assets invisible to Git.

`.vs/`, `*.vcxproj.user`, `*.suo`, `*.user`, `*.VC.db`, `*.VC.VC.opendb`, and `.vscode/ipch/` exclude user-specific IDE caches and preferences. Shared `.vcxproj`, `.vcxproj.filters`, `.sln`, `launch`, and IntelliSense files are not globally hidden. `__pycache__/`, `*.py[cod]`, and `.venv/` exclude Python bytecode or a local virtual environment. `/docs/.cache/` provides an ignored place for generated documentation intermediates; the requested PDF is intentionally not ignored. `Thumbs.db`, `ehthumbs.db`, `Desktop.ini`, and `.DS_Store` cover operating-system folder metadata. `*~`, `*.swp`, and `*.swo` cover common editor backups/swap files. No blanket PDF, PNG, JPG, WAV, or MP3

pattern hides required documentation or assets.

Verified Cleanup Behavior and Remaining Limits

The organized project was configured from a clean build directory and built in MSVC Release/x64. The alternative standalone Visual Studio project was also built successfully in Release/x64, with its output and intermediates explicitly directed beneath `build/native/` and `vcpkg` integration disabled for that check. Its post-build event copied all eight sounds and the retained license. `RescueMissionLogic` passed. The normal render, breach, mouse-steering/camera, and pursuit smoke tests each rendered 120 frames and reported success on an Intel Iris Xe OpenGL 3.3 driver. An additional smoke run from the executable directory also loaded every selected sound. These checks exercise actual relocated runtime assets rather than only searching source text for filenames. The standalone build additionally reports a vendor warning C4244 at `miniaudio` line 80532 about narrowing an unsigned 64-bit value to 32 bits; the vendored code is retained unchanged rather than modifying library internals during cleanup.

The post-build asset copy is not an installer. It is triggered by a build of the executable target, and does not continuously watch asset edits. Rebuild the target or recopy assets if only sound files change after a previous build. Tests validate deterministic mechanics and finite scripted rendering windows, not every possible human flight path. No network, save file, persistent leaderboard, imported-model loader, cloud sky renderer, or image-based worm texture is implemented. Those distinctions are documented so the explanation remains a description of existing functionality rather than an aspirational feature list.

Chapter 2: Terrain, Particles, Procedural Meshes, and the Renderer

Reading This Chapter

This chapter explains **original arrakis.cpp lines 1-967**, captured before the source file was reorganized into `src/arrakis.cpp`. All numbered line-range headings refer to that original snapshot, not to whichever line numbers a later editor displays. The final covered line, 967, is blank; the last actual declarations are the rock colors on lines 965-966. Source listings are supplied separately, so the explanations below describe the code rather than replacing explanation with long source quotations.

The ranges follow source order. Every nonblank line in the requested interval belongs to a numbered subsection, including file comments, preprocessor directives, declarations, shader-string delimiters, and closing braces. Blank lines between ranges provide visual separation and do not execute. A brace starts or ends a scope: the body of a function, loop, conditional, structure, initializer, or shader. A semicolon finishes a declaration or statement. Where several closing braces occur together, they close the previously described nested operations rather than introduce hidden work.

For a beginner, it helps to separate three kinds of activity:

- **CPU-side C++** constructs terrain samples, particle states, vertex arrays, and rendering commands. Its ordinary functions execute on the processor.
- **OpenGL API calls** ask the graphics driver to create objects, upload bytes, configure state, or draw. An OpenGL object identifier is an integer handle, not the actual vertex data or texture pixels.
- **GPU-side GLSL** is source text embedded in C++ strings. The driver compiles that text into shader programs. Vertex shaders run on vertices; fragment shaders compute results for rasterized fragments, which are candidates for image pixels.

The terrain uses a Y-up convention: X and Z run across the ground, while Y is height. A `glm::vec2` contains two numbers, `vec3` three, and `vec4` four. A vector can represent a position, direction, color, or packed record depending on context. `glm::mat4` is a four-by-four transformation matrix. Coordinates with an extra `w` component are **homogeneous coordinates**: using `w = 1` allows a matrix to translate a point, while a direction normally uses `w = 0` so translation has no effect.

`float` is a floating-point number, `int` an integer, and `bool` a true-or-false value. The suffix `f` marks a C++ numeric literal as a `float`, such as `0.4f`; GLSL does not need that suffix. `const` prevents ordinary reassignment. A reference, written `&`, refers to an existing object instead of making a copy. `const T&` lets a function inspect an object without modifying it. `auto` asks the compiler to infer a type. `std::vector<T>` is a resizable, contiguous collection of values of type `T`.

The mathematical helpers recur throughout the renderer. `normalize(v)` divides a nonzero vector by its length, producing a direction of length one. `dot(a, b)` adds componentwise products; for normalized directions it equals the cosine of their angle. `cross(a, b)` produces a perpendicular vector whose orientation depends on argument order. `mix(a, b, t)` is linear interpolation, $(1-t)a + tb$. `clamp(x, a, b)` confines a value to the interval from `a` to `b`. `smoothstep(a, b, x)` first clamps $t = (x-a)/(b-a)$ to $[0, 1]$, then returns $t*t*(3-2*t)$, a smooth transition with zero endpoint slopes. `sin` and `cos` take radians; one full turn is 2π , approximately 6.2831853 radians.

Scope caution: these lines define reusable renderer machinery. They do not yet contain the main frame loop, complete aircraft or harvester assembly, input callbacks, all uniform assignments, resource destruction, or the actual particle draw call. A declaration here does not prove that a later feature is enabled, nor does a shader uniform declaration provide its runtime value. The discussion distinguishes exact values visible in this interval from values that must be supplied elsewhere.

File Setup and Shared State

1. Original Lines 1-43: File Banner, Feature Claims, and Control Notes

Every line in this range is a C++ `//` comment. The compiler ignores the banner and its descriptive text. Lines 1 and 43 are decorative separators. Lines 2-3 name the project, its rescue-game theme, and its laboratory course. Lines 5-7 summarize its intended presentation and claim that geometry is generated from primitives rather than loaded from `.OBJ` files. The primitive builders explained later support the procedural-geometry description; these comments are not a runtime prohibition against adding other assets.

Lines 9-28 list claimed visual features. The aircraft description names eight wings, a faceted cockpit, turbine intakes, thruster coloration, and banking. The harvester description names four crawler units, a front drum, an upper refinery structure, floodlights, and funnels. The tank description names pressure vessels, domed ends, supporting trusses, gauges, and pipes. These assemblies are outside the present line interval; the meshes and materials here are building blocks for them.

The terrain comment says "140x140 vertex heightfield" and "analytical normals." The actual default terrain has **140 by 140 cells and 141 by 141 vertices**, because both vertex loops include their final endpoints. Its normals are calculated using four nearby height samples, a finite-difference method, not exact symbolic differentiation. The atmospheric comment says more than 2,000 particles; the actual pool constant is 2,400. The lighting description is broadly recognizable in the later shaders, but its sand diffuse equation is a particular 0.35-wrap expression rather than the canonical squared half-Lambert formula. The shadow filter is specifically a 5-by-5 manual PCF filter.

Lines 30-42 list user-facing controls: Enter starts or advances missions; mouse hover steers; W/S and A/D move; Q/E change altitude; Left Shift boosts; Space hovers or operates rescue actions; P/R pause or retry; arrow keys steer or adjust camera elevation; C switches camera mode; F requests wireframe; F11 requests fullscreen; Escape quits. This comment does not implement those actions. Within this chapter, only the key-state storage and the particle branch that checks Space are present. Keeping descriptive comments separate from executable logic prevents assuming that the banner itself enforces behavior.

2. Original Lines 45-59: Graphics, Mathematics, and Standard-Library Includes

`#include` is a preprocessor instruction that makes declarations from another header available before C++ compilation. Angle brackets normally select library headers through configured include paths. Line 45 imports GLAD's OpenGL declarations and function-loading interface. Modern OpenGL functions need to be loaded for the active context; merely including this header does not load them. Line 46 imports GLFW, which supplies window/context management, key codes, and the `glfwGetTime()` clock used below.

Line 48 imports GLM vector and matrix types. Line 49 adds transformation helpers such as translation, scaling, rotation, and projection-related matrix operations. Line 50 supplies `glm::value_ptr`, which exposes the contiguous numerical storage needed when passing GLM objects to OpenGL uniform functions. Whether all mathematical constants used later are made available transitively depends on the project's GLM configuration and headers; these lines do not contain an explicit constants-header include.

Lines 52-59 include eight standard C++ facilities. `<iostream>` provides `std::cerr` and stream output for shader/framebuffer errors. `<vector>` supplies dynamic collections. `<cmath>` supplies trigonometry, absolute values, and other math operations. `<random>` supplies the Mersenne Twister generator and uniform real distributions. `<algorithm>` supplies operations such as `std::swap`, used to correct rock triangle orientation. `<string>` provides strings, `<fstream>` file streams, and `<stdint>` fixed-width integer definitions. Some of these facilities serve later parts of the same translation unit; being included here does not imply that every header is directly used before line 967.

These directives have no visible rendering effect by themselves. They tell the compiler what names and types mean. Actual context creation, function loading, and initialization must happen before the later OpenGL calls execute.

3. Original Lines 61-67: Window and Display Globals

Lines 61-63 are a section-label comment. `g_scrW = 1280` and `g_scrH = 720` establish initial display dimensions. Their ratio is $1280/720 = 16/9$, approximately 1.7778. These values can later be changed by window or framebuffer callbacks; the declarations do not guarantee that the physical framebuffer permanently has this resolution.

`g_isFullscreen = false` records that startup is not fullscreen. `GLFWwindow* g_window = nullptr` declares a pointer to a GLFW window object and initializes it to the null pointer, meaning that no valid window object is referenced yet. A pointer stores an address-like reference; it is not a copied window. A later successful window-creation call must assign it before window operations can safely use it.

The `g_` prefix is a naming convention for shared globals, not a C++ keyword. These variables are defined at file scope, so functions later in the translation unit can access them. Shared state is convenient in a compact teaching project but couples callbacks, simulation, and rendering: an incorrect dimension update can affect projection or viewport behavior elsewhere. This range stores configuration only and neither creates a window nor sets an OpenGL viewport.

4. Original Lines 69-87: Keyboard, Camera, Flight, and Wing State

Lines 69-71 introduce camera and flight state. `g_keys[1024] = {}` is a fixed array of 1,024 Boolean entries, all initialized to false through value initialization. A later input callback can mark a key as pressed or released; this array does not itself poll the keyboard. The Space particle branch indexes it using GLFW's Space key constant. General input code must still avoid out-of-range key indices.

`g_wireframe = false` records a filled-rendering preference. `g_camMode = 0` selects the comment's cinematic chase mode; values 1 and 2 represent cockpit/close and overhead/tactical modes. These integers are a convention, not a type-safe enumeration or validation mechanism. `g_camOrbitPitch = 0.0f` stores an initial camera-orbit pitch offset.

The flight comment at line 77 introduces `g_ornPos(0, 18, 0)`, placing the aircraft initially at $X=0, Y=18, Z=0$. This is a world-space position, not automatically a terrain-relative altitude. `g_ornVel(0.0f)` uses GLM's scalar constructor to set all three velocity components to zero. `g_ornYaw`, `g_ornPitch`, and `g_ornRoll` start at zero degrees. Yaw is heading, pitch is nose tilt, and roll is banking around the forward axis. The exact rotation order of a model is determined by later transformation code rather than these names alone.

`g_ornSpeed = 0` stores a scalar cruise-speed state separate from the velocity vector. `g_isBoosting = false` records that boost is inactive. None of these declarations integrates acceleration or computes flight dynamics; they supply the initial state used by later logic. Line 86's wing comment precedes `g_wingPhase = 0`, an animation-phase accumulator. Its declaration alone specifies neither frequency nor amplitude and does not enforce the "movie-accurate" claim. The visible implementation should be explained as animated mathematical geometry rather than physical aerodynamics.

5. Original Lines 89-99: Harvester Coordinates and Tank Instances

The harvester comment introduces `g_harvX = -80` and `g_harvZ = 15`. These are initial horizontal coordinates. No harvester Y coordinate appears here because later code can derive elevation from terrain. The exact horizontal distance of this point from the origin is $\sqrt{80^2 + 15^2}$, approximately 81.39 world units.

Lines 93-98 define `SpiceTankInstance`, a small C++ record grouping three related properties. `pos` is a three-component placement vector, `yaw` is a scalar heading, and `scale` is a scalar size multiplier. Their declarations have no in-class default initializers. Code constructing a tank must provide meaningful values before using them; one should not read these members as automatically containing a ready-to-render placement. The opening and closing braces define the structure's member list, and the trailing semicolon finishes the type declaration.

`std::vector<SpiceTankInstance> g_tanks` defines a initially empty collection of tank instances. A structure defines what one record looks like, whereas a vector stores zero or more actual records. The tank geometry is not duplicated here; later code can reuse the same primitive meshes with different transformations and material settings. This separates an object's placement data from the GPU objects used to draw it.

Terrain Height and Normal Queries

6. Original Lines 101-118: `rawDuneHeight` and the Undisturbed Height Equation

Lines 101-104 label the continuous dune function. `rawDuneHeight(float x, float z)` takes horizontal coordinates by value and returns one floating-point elevation. It does not allocate a mesh; any subsystem can ask for a height at any coordinate, including coordinates beyond the finite rendered terrain. The code sums deterministic trigonometric patterns, not random terrain noise or a physical wind-erosion simulation.

The primary swells are $h1 = 14 \cdot \sin(0.009 \cdot x + 0.004 \cdot z)$ and $h2 = 9.5 \cdot \cos(0.004 \cdot x - 0.012 \cdot z)$. Their amplitudes are 14 and 9.5 units. The arguments combine X and Z, so each pattern runs diagonally rather than along only one axis. For the first wave, the horizontal wave-vector length is $\sqrt{0.009^2 + 0.004^2}$, approximately 0.0098489 radians per unit, producing a perpendicular wavelength of about $2 \cdot \pi / 0.0098489 = 637.95$ units. The second wavelength is about 496.73 units. These are broad landscape undulations.

The ridge starts with $s = \sin(0.028 \cdot x + 0.016 \cdot z)$, transforms it to $r = 1 - \text{abs}(s)$, and produces $5 \cdot r \cdot r$. Since $\text{abs}(s)$ lies between zero and one, the ridge contribution lies between zero and five. Squaring concentrates its elevation around locations where the original sine is zero. Taking an absolute value halves the repetition period in the phase and introduces a derivative cusp at those zero crossings. Despite the "asymmetric windward & leeward" comment, this expression does not explicitly distinguish a windward side from a leeward side; it creates a symmetric folded ridge pattern.

$h3 = 1.8 \cdot \sin(0.065 \cdot x + 0.038 \cdot z)$ adds smaller, shorter undulations with a wavelength of approximately 83.45 units. The return statement sums $h1 + h2 + \text{ridge} + h3 - 4$. The minus-four term shifts the overall height down. Independent term bounds give a conservative range from -29.3 to 26.3, though the waves do not necessarily reach all extreme values at the same point. At $(0, 0)$, the sine terms vanish, $h2 = 9.5$, and $\text{ridge} = 5$, so the returned height is 10.5. The closing brace ends the query; there is no state update or temporal animation in this function.

7. Original Lines 120-130: `getDuneHeight` and the Flattened Landing Terrace

`getDuneHeight(x, z)` begins with the raw height, then modifies it near a landing pad. The local static `const` array is initialized only once, when this function first reaches its declaration, and remains available on later calls. It contains one `glm::vec3`: `{55, 55, rawDuneHeight(55, 55)}`. Here the first two components are **horizontal X and Z**, and the third is the target height. This is a special packed convention, not the normal world-position convention (x, y, z) .

For this one center, `center.x = 55`, `center.y = 55`, and `center.z` is approximately 13.3066. Converting `center` to `glm::vec2` keeps its first two components, correctly obtaining the horizontal pad center $(55, 55)$. Conversely, the height blend deliberately uses `center.z`, the stored target elevation. Misreading that component as a horizontal coordinate would make the function seem incorrect when it is actually using a compact record layout.

The range-based loop uses `const auto& center`, so each iteration reads an existing array element without copying or modifying it. The ternary `center.x == 55 ? 15 : 10` chooses radius 15 for the present entry. The 10-unit alternative is dormant with this one-element array; it is not a second existing terrace. The equality test works for the exactly initialized value 55 but is an ad hoc rule tied to position rather than an explicit radius field.

Let d be the horizontal distance from (x, z) to $(55, 55)$. The blend weight is $b = 1 - \text{smoothstep}(15, 25, d)$. Inside radius 15, $b=1$ and the height becomes exactly the stored plateau height. Beyond radius 25, $b=0$ and raw terrain is unchanged. In between, the cubic transition blends the two. At radius 20, $b=0.5$, so the result is their average. `glm::mix(height, center.z, blend)` implements that interpolation. The return statement supplies the final height after the loop. Evacuation sites are not flattened by this array, matching the comment's distinction from the landing pad.

8. Original Lines 132-139: `getDuneNormal` and Central Finite Differences

`getDuneNormal(x, z)` estimates which direction the surface faces. A surface normal is perpendicular to the local ground; lighting uses it to decide how strongly the surface faces the sun. The sampling offset is `eps = 0.4`, giving a full difference baseline of $2 \cdot \text{eps} = 0.8$ units.

The function evaluates the final, terraced height at four points. h_L samples X minus 0.4, h_R X plus 0.4, h_D Z minus 0.4, and h_U Z plus 0.4. Calling `getDuneHeight` rather than `rawDuneHeight` means the normal includes the landing-pad blend. Approximate derivatives are $(h_R - h_L)/0.8$ and $(h_U - h_D)/0.8$.

For a heightfield $y=h(x, z)$, an upward-facing unnormalized normal is $(-dh/dx, 1, -dh/dz)$. The code multiplies all components by 0.8 to avoid two explicit divisions, constructing $(-(h_R - h_L), 0.8, -(h_U - h_D))$, then normalizing. Multiplying a nonzero vector by a positive scalar does not change its normalized direction. On a perfectly flat surface, both differences vanish and the result is $(0, 1, 0)$.

This is **finite-difference estimation**, not an analytical derivative. It samples across sharp ridge features and the terrace transition, producing a smoothed estimate over the baseline. Four height queries are needed per normal. The positive Y component ensures a nonzero input even on flat ground, so this particular normalization does not encounter a zero vector. The method also differs from computing a triangle's geometric face normal: the rendered mesh samples terrain every five units by default, while this normal samples a much smaller 0.8-unit baseline. Later interpolation gives smooth shading that need not exactly match each triangle plane.

9. Original Lines 141-144: Local Audio, Game, and Test Headers

Lines 141-143 include `arrakis_audio.h`, `arrakis_game.h`, and `arrakis_tests.h` using quotes, which select project-local headers through the configured search rules. They appear after the shared flight globals and terrain-query functions. That placement makes earlier declarations visible to code introduced by these headers, an important dependency in this single-translation-unit organization.

The later particle update uses names such as `wormAttackTime()`, `wormPosition()`, `harvesterForward(...)`, and `g_mission.harvesterYaw`, supplied through the game's header-level code rather than declared in the immediate particle section. This chapter explains how those values are consumed, not the internal mission rules that produce them. Audio and test internals likewise belong to their own chapters. Line 144 is a blank separator.

An include is textual preprocessing, not a call that executes the entire game at that moment. Definitions brought in by a header can still create globals or functions as part of the resulting translation unit, but runtime actions occur only when the resulting code is initialized or invoked. This distinction matters when following source order: declaration visibility follows includes, whereas frame-by-frame control flow follows function calls.

Wind and Particle Simulation

10. Original Lines 145-162: Wind Constants, SandParticle, and the Pool

Lines 145-147 label the particle system. `WIND_DIR` normalizes $(-0.92, -0.04, 0.38)$. The original vector length is $\sqrt{0.9924}$, approximately 0.99619, so the normalized result is approximately $(-0.92352, -0.04015, 0.38145)$. Most wind therefore travels toward negative X , somewhat toward positive Z , and slightly downward. `WIND_BASE_SPEED = 18` supplies the nominal speed multiplier in world units per second, assuming the caller passes `dt` in seconds.

`SandParticle` groups six simulation/rendering properties. `pos` is world position; `vel` is velocity; `life` is remaining lifetime; `maxLife` records a reference lifetime; `size` is a scalar billboard size; and `alpha` is an opacity multiplier. These members have no explicit defaults in the structure. The initialization and respawn functions assign the fields they need. `life` is not a normalized progress percentage, and `maxLife` does not automatically affect opacity; later draw preparation must decide whether to use their ratio.

`MAX_PARTICLES = 2400` fixes the intended pool size. `g_particles` starts as an empty vector and is resized by `initParticles`. A fixed pool lets updates recycle records instead of continuously allocate and remove individual particles. These particles model visible dust, not conserved physical sand grains: their positions and velocities are manually reassigned, they have no mass, and they do not collide with one another. The section's closing structure brace and semicolon complete the record type rather than performing initialization.

11. Original Lines 163-179: `initParticles` and Reproducible Initial Values

`initParticles()` resizes the vector to 2,400 entries. It then constructs a local `std::mt19937` generator with seed 1337. This is a deterministic pseudorandom engine: repeated initialization starts the same engine sequence. Exact samples from `std::uniform_real_distribution` can vary across standard-library implementations, so reproducibility should not be overclaimed across every platform. The generator exists only for this initialization call.

Four distributions describe desired ranges: horizontal position from -120 to 120, Y from 0.5 to 25, lifetime from 1 to 4, and wind-speed factor from 0.8 to 1.4. Uniform real distributions conventionally produce values in a half-open range, although floating-point endpoint behavior should not be used as a safety guarantee. The range-based loop obtains each particle as a mutable reference, `auto& p`, so assignments change the vector entry directly.

The position constructor draws two horizontal values and one vertical value. It does **not** add the dune height; an initial particle may therefore start under a hill and be corrected during update. The velocity is the normalized wind multiplied by `18*speedFactor`, giving nominal speeds from 14.4 to 25.2. `maxLife` and `life` are drawn independently. Consequently a newly initialized particle can have `life > maxLife`; these are not initially guaranteed to form a ratio in $[0, 1]$.

The engine's raw integer output modulo 100 gives values 0-99. Size becomes $0.18 + 0.003 * n$, ranging from 0.18 to 0.477. Alpha becomes $0.25 + 0.0035 * n$, ranging from 0.25 to 0.5965. Modulo sampling has a small distribution bias unless the engine range divides evenly by 100. These are deliberate visual variations, not real-distribution draws. The closing loop brace finishes all particle assignments, and the function brace finishes initialization; no GPU instance buffer is uploaded here.

12. Original Lines 181-190: `updateParticles`, the Gust Clock, and Integration

`updateParticles(float dt, glm::vec3 centerPos, glm::vec3 harvPos)` advances the pool. The elapsed step `dt` is copied as a scalar; both positions are copied as small vectors. `centerPos` is the focus/player position used for distance tests and respawning, while `harvPos` anchors crawler dust. The function assumes useful, nonnegative time steps; it does not clamp or validate `dt` here.

The gust multiplier is $1 + 0.35 * \sin(\text{glfwGetTime}() * 1.8)$. It ranges from 0.65 to 1.35 and repeats every $2 * \pi / 1.8$, approximately 3.491 seconds. The clock is GLFW's absolute elapsed time, cast from its wider floating type to `float`. Gust timing therefore follows that external clock rather than an accumulator local to the particle function. If simulation updates are paused elsewhere while the clock continues, the gust phase can change before updates resume.

The local static `std::mt19937 rng(1984)` is initialized once and retains its state across calls, unlike the initialization engine. Horizontal respawn offsets use -100 to 100, and vertical offsets use 0.2 to 32. The indexed loop visits every vector entry, casting its size to `int`; with a 2,400-entry pool this conversion is safe in practice, though it is not a generic solution for arbitrarily enormous vectors.

`p.life -= dt` decreases remaining lifetime. `p.pos += p.vel * (gust * dt)` performs a forward-Euler position update with gust-scaled displacement. Gust does not modify the stored velocity; it scales movement during this step. This multiplier affects every particle class, including worm spray and downwash, not only ordinary wind dust. There is no acceleration integration, drag calculation, or gravity term in this opening part of the update.

13. Original Lines 192-204: Ground Response and Respawn Eligibility

The ground-collision comment introduces a height query at the particle's current X/Z. If Y is below `groundY + 0.2`, the code places it exactly 0.2 units above the sampled terrain and changes vertical velocity to $\text{abs}(\text{oldV}_y) * 0.5 + 0.8$. A downward velocity of -2 becomes +1.8; an upward velocity of +2 also becomes +1.8 if the particle still lies below the threshold. This is a visual rebound-and-lift rule, not an energy-conserving collision. There is no later gravity here to pull the particle down again.

The distance test converts both positions to horizontal two-component coordinates and uses Euclidean distance, ignoring vertical separation. Thus a particle high above the aircraft can still be horizontally near it. Respawning occurs when `life <= 0`, or when the particle is beyond 160 units **and** its index is divisible by neither 4 nor 3. Operator

precedence makes the nested parentheses important: the far-distance exemptions do not protect a particle whose lifetime has expired.

The `i % 4 != 0 && i % 3 != 0` condition exempts indices reserved for possible crawler or worm effects from distance-only recycling, even if those effects are currently inactive. Among 2,400 indices starting at zero, 1,200 are divisible by 3 or 4, by inclusion-exclusion: 800 plus 600 minus 200. These 1,200 still respawn on lifetime expiry. The remaining 1,200 are subject to both expiry and distance recycling.

Inside the respawn block, lifetime is $2 + 0.02 * n$ for integer n from 0 to 99, giving 2.00-3.98 seconds. `maxLife` is then assigned that same value, unlike the independent initial samples. The particle's old state has already been moved and ground-corrected before it is recycled. Subsequent branches replace its position, velocity, size, and opacity based on effect priority.

14. Original Lines 205-214: Crawler-Plume Respawning

The comment says "Half particles spawn upwind" and mentions crawler dust, but exact proportions depend on the ordered branches and their time conditions. The first branch accepts indices divisible by four when `wormAttackTime() < 12`. It has highest priority. A particle qualifying here cannot also enter the worm or Space branches on this respawn. The comparison includes negative attack-time values as well as positive ones below 12; the meaning of those values is defined by mission logic elsewhere.

`forward = harvesterForward(g_mission.harvesterYaw)` obtains the harvester's heading direction. `right = cross(forward, (0, 1, 0))` obtains a perpendicular lateral vector. Its sign follows the heading convention in `harvesterForward`; the variable's name should not replace checking that convention. The code does not normalize `right` here, so a unit lateral displacement assumes the supplied forward vector is appropriately normalized and horizontal.

The new position is the harvester position, minus four units along forward, plus a lateral offset, plus a vertical offset. The lateral value is $-4 + 0.1 * n$ with n from 0 to 79, so it ranges from -4 to 3.9. The vertical offset is $1 + 0.1 * n$ with n from 0 to 29, ranging from 1 to 3.9. This creates a rectangular region behind and above the harvester rather than positions tied to individual tread geometry.

Velocity combines half-strength wind, `WIND_DIR*9`, a two-unit-per-second backward component, $-\text{forward} * 2$, and +2.5 vertical lift. It is not renormalized, so its total speed depends on the vector sum. Alpha is fixed at 0.55 and size at 0.45. The branch does not resample terrain after spawning; its placement is based on the supplied harvester position, and the next update's ground check will correct a below-ground result.

15. Original Lines 215-223: Worm Dust, Angular Samples, and Attack-Dependent Spray

The `else if` selects indices divisible by three when `wormAttackTime() < 18`, provided the preceding crawler branch did not run. A multiple of both three and four will therefore use crawler dust while the earlier time condition is active; after that condition fails, it can use worm dust. These are index-based visual allocations, not independent emitter objects.

`wormPosition()` supplies the effect center. The angle $a = (\text{rng}() \% 10000) * 0.0006283185$ spans approximately one full turn in 10,000 discrete steps, because the multiplier is $2 * \pi / 10000$. Radius is $10 + 0.01 * n$, where n is 0-1499, giving 10-24.99. Uniformly sampling radius is not uniform sampling of disk area; moreover, this distribution is an annular region, not a filled disk.

The position offset is $(\cos(a) * \text{radius} - 0.01 * m, 0, \sin(a) * \text{radius})$ with m from 0 to 2199. The extra X-only subtraction ranges from 0 to 21.99, creating a negative-X trail rather than a symmetric ring. After constructing X/Z, the code overwrites Y with local terrain height plus 0.8, so worm dust begins near the undisturbed CPU terrain even if the vertex shader later visually depresses that area.

Velocity uses radial horizontal speed 7, vertical speed 10 when attack time is greater than zero or 2 otherwise, then adds `WIND_DIR*4`. Size similarly switches between 5.5 for positive attack time and 2.2 otherwise. Alpha is fixed at 0.22. These large translucent billboards depict disturbed sand, not individual grains. Repeated `wormAttackTime()` calls are separate function evaluations; the code does not cache a single time for all comparisons in this update.

16. Original Lines 224-228: Space-Key Downwash Respawning

The next `else if` requires an index divisible by five and `g_keys[GLFW_KEY_SPACE]` to be true, while all earlier branches must have failed. It responds to the stored key-state flag rather than querying GLFW directly. Because this code is inside the respawn block, pressing Space does not instantly reposition all qualifying particles; the effect appears as particles expire or satisfy their distance-recycling condition.

The angle is `(rng()%1000)*0.006283185`, approximately $2\pi/1000$ times a 0-999 integer. The new position is centered on `centerPos` with a four-unit horizontal radius and a vertical offset of -4. It is therefore a circular emitter four units below the focus position, not a surface sample under each rotor or a simulation of actual wing airflow.

Velocity is `(11*cos(a), 1.5, 11*sin(a))`. Its horizontal magnitude is exactly 11 in ideal arithmetic; including vertical lift gives a speed of `sqrt(121+2.25)`, approximately 11.102. It has no wind term at assignment, but the shared gust multiplier still scales its motion in later updates. Size is 1.8 and alpha 0.18, both assigned on the same source line. The low opacity and large size create a broad dust appearance.

No local height adjustment occurs in this branch. If the aircraft is low enough, the new particle can be below the ground until the next call applies the ground correction. The branch also does not check mission hover eligibility, winch state, or actual altitude beyond the supplied position: its visible trigger in this interval is simply Space plus index priority.

17. Original Lines 229-240: Ordinary Upwind Respawning and Update Closure

The final `else` handles every respawn not claimed by crawler, worm, or downwash effects. `upwind = -WIND_DIR*90` offsets the emitter opposite the nominal wind. Using the normalized direction, this is approximately (83.117, 3.614, -34.331). Because the wind has a slight downward component, going upwind also moves the starting point upward.

The new position is `centerPos + upwind + (randomX, randomY, randomZ)`, with horizontal offsets from -100 to 100 and a vertical offset from 0.2 to 32. The vertical value is relative to the center position, not directly relative to terrain. The following height query checks `if (p.pos.y < gy)` and, only then, sets `Y` to `gy+1`. A particle already slightly above the surface is not raised to a full unit; this differs from the earlier unconditional collision threshold of terrain plus 0.2.

Velocity is wind times $18*(0.85+0.01*n)$ with `n` from 0 to 39. The stored speed therefore ranges from 15.3 to 22.32. After gust scaling, its instantaneous displacement speed can range from about 9.945 to 30.132 across those independent extremes. Alpha is 0.28 and size is 0.22. Recycling overwrites any earlier plume or spray settings so the record becomes an ordinary wind particle again.

Line 237 closes the ordinary branch, line 238 closes the respawn conditional, line 239 closes the particle loop, and line 240 closes the function. No particle sorting, mesh upload, alpha blending, collision with objects, or resource allocation occurs in this update. Its responsibility is CPU simulation state only.

Mesh Records and GPU Upload

18. Original Lines 242-258: Vertex, Mesh, and Indexed Drawing

Lines 242-245 introduce procedural primitives and reiterate that shapes are mathematically built. `Vertex` contains `pos`, `normal`, and `uv`. Position places a vertex in a mesh's local coordinates. A normal provides shading orientation and may differ from a geometric face normal when smoothing is desired. UV is a two-component surface coordinate used by shaders; it does not require an image texture to exist. In this renderer, particle UVs define circular opacity, while the surface shader's patterns chiefly use world coordinates instead.

Mesh stores three OpenGL handles initialized to zero: `VAO`, `VBO`, and `EBO`. A **vertex buffer object** stores vertex bytes. An **element buffer object** stores indices that select vertices to form primitives. A **vertex array object** remembers attribute layout and associated buffer state. It is not a second copy of the geometry. Handle zero means no generated object has yet been assigned to that field.

`GLsizei indexCount = 0` holds the number of indices to draw, not the number of vertices or triangles. `GLenum drawMode = GL_TRIANGLES` selects independent groups of three indices by default. For example, 36 indices

represent 12 triangles when this mode is used. `GLsizei` and `GLenum` are OpenGL-defined types, communicating the expected integer category to the API.

The record has no destructor and no ownership-enforcing copy rule. Copying a `Mesh` copies its handles rather than duplicating GPU storage. Returning one from a builder is convenient, but automatic scope exit will not call `glDeleteBuffers` or `glDeleteVertexArrays`. Any resource cleanup must be handled explicitly elsewhere. This is a lightweight handle container, not an RAII resource wrapper.

19. Original Lines 260-274: `uploadMesh`, Buffer Creation, and Storage

`uploadMesh` accepts constant references to vertex and unsigned-index vectors. It can inspect their contiguous data without copying whole CPU arrays or changing them. A local `Mesh m` applies the structure's default member initializers. The index count is obtained by casting `indices.size()` to `GLsizei`; extremely large input would need overflow checks, but these procedural meshes are small enough for the intended type.

The three `glGen...` calls request one vertex-array handle, one vertex-buffer handle, and one element-buffer handle, writing each generated name through the address of its member. Generation is not a geometry upload. Binding `m.VAO` makes it the active vertex-array state record. Subsequent element-buffer and attribute configuration will belong to that VAO.

`glBindBuffer(GL_ARRAY_BUFFER, m.VBO)` selects the vertex buffer as the target for the next upload. `glBufferData` allocates/copies `verts.size()*sizeof(Vertex)` bytes from `verts.data()`, with `GL_STATIC_DRAW` as a driver usage hint. That hint suggests infrequent replacement and frequent drawing; it is not an immutable-storage guarantee. `sizeof(Vertex)` includes any padding chosen for the structure, so the later stride must use that same size.

The element-buffer upload similarly copies `indices.size()*sizeof(unsigned int)` bytes from `indices.data()` after binding `GL_ELEMENT_ARRAY_BUFFER`. Unlike the generic array-buffer binding, this element-buffer binding is stored in the currently bound VAO. The CPU vectors can later be destroyed after upload because the driver has its own buffer storage. This function does not check OpenGL error codes or validate that indices are within the vertex range, so correct input and a valid current graphics context are prerequisites.

20. Original Lines 276-288: Attribute Locations, Byte Offsets, and the Returned Mesh

The attribute-layout comment fixes the contract between uploaded `Vertex` records and shader inputs: location 0 is position, 1 is normal, and 2 is UV. `glVertexAttribPointer` does not copy one attribute at a time; it records how the GPU should find that attribute in the currently bound array buffer.

Position uses three `GL_FLOAT` components, normal also three, and UV two. `GL_FALSE` means OpenGL should not apply integer-to-normalized-value conversion. Each attribute has stride `sizeof(Vertex)`, so advancing to the next vertex advances one complete record. `offsetof(Vertex, pos)`, `offsetof(Vertex, normal)`, and `offsetof(Vertex, uv)` compute member byte offsets, preserving correctness if the type contains padding rather than assuming tightly packed offsets of 0, 12, and 24.

The `(void*)` casts encode byte offsets for the buffer-backed attribute API. They do not mean that OpenGL will dereference arbitrary CPU addresses while drawing. Each `glEnableVertexAttribArray` enables the corresponding attribute; defining its pointer alone would not enable array fetching. All these attributes are per-vertex by default because this function does not set instancing divisors.

`glBindVertexArray(0)` unbinds the newly configured VAO, reducing accidental subsequent edits to its state. It does not unbind every global buffer target or delete any object. `return m` passes the resulting handle record back to the primitive builder's caller. The closing brace finishes upload. Draw mode remains the default triangles because this function never changes it. Uploading a mesh also does not choose a shader, assign materials, or issue a draw call; those are separate responsibilities handled later.

Procedural Primitive Builders

21. Original Lines 290-314: createCube, Its Face Lambda, and Six Faces

The comment describes a cube with separate normals per face. `createCube()` creates empty vertex and index vectors, then defines a local lambda named `addFace`. A lambda is a small unnamed function stored in a variable. Its `[&]` capture gives it reference access to the surrounding vectors so its `push_back` calls modify the builder's data. The parameters are four corner positions and one shared face normal.

Inside the lambda, `base` records the current vertex count before appending. Four new vertices receive the same normal and the four UV corners $(0, 0)$, $(1, 0)$, $(1, 1)$, and $(0, 1)$. The six appended indices select triangles $(base, base+1, base+2)$ and $(base, base+2, base+3)$, splitting the face along the diagonal from its first to third corner. A triangle's index order determines its winding, which matters if face culling is enabled elsewhere. The lambda's closing `}`; finishes its body and local declaration.

Six calls supply faces at $Z=+0.5$, $Z=-0.5$, $X=-0.5$, $X=+0.5$, $Y=+0.5$, and $Y=-0.5$. Their normals are respectively $(0, 0, 1)$, $(0, 0, -1)$, $(-1, 0, 0)$, $(1, 0, 0)$, $(0, 1, 0)$, and $(0, -1, 0)$. Every coordinate magnitude is 0.5, so the cube is centered at the origin with side length one. The supplied corner orders give outward-facing triangles under the usual counterclockwise convention.

The cube has 24 vertex records, not just eight unique spatial corners, because a shared corner needs three different normals for three sharply separated faces. It has 36 indices and 12 triangles. This duplication produces a crisp box rather than a rounded-looking cube with averaged corner normals. `return uploadMesh(v, idx)` uploads those arrays and returns the GPU handle record; the final brace finishes the builder. Changing an object's apparent box dimensions happens through a later model scale, not by rebuilding this unit cube.

22. Original Lines 316-334: createCylinder and Smooth Side Vertices

`createCylinder(int slices = 24)` takes an optional circumferential segment count. Calling it without an argument uses 24. A default argument selects a value at the call site; it does not stop callers from passing another value. The function creates local vertex and index vectors, then sets radius $r=0.5$ and height $h=1$. The cylinder is aligned with Y and centered at the origin.

The body loop uses `i <= slices`, generating 25 angular samples at the default rather than 24. The first and final sample occupy the same seam direction but have different U coordinates. $\theta = i/slices * 2\pi$ covers a full revolution; explicit floating-point casts prevent integer division from collapsing most fractions to zero. $c=\cos(\theta)$ and $s=\sin(\theta)$ locate the circle direction. $norm=(c, 0, s)$ is the outward radial side normal, of unit length in ideal arithmetic.

At each angle, the function appends a lower vertex $(r*c, -0.5*h, r*s)$ and an upper vertex $(r*c, +0.5*h, r*s)$. Their UVs are $(u, 0)$ and $(u, 1)$, with $u=i/slices$. The result is a strip of vertical pairs, suitable for two triangles between neighboring pairs. Shared normals around the circumference allow interpolation to make the side look smooth despite its polygonal silhouette.

The duplicated seam supports UV values zero and one without forcing interpolation across a discontinuity. This is normal for textured cylinder construction even though the scene shader does not currently use its UV coordinates for surface color. The input is not validated: zero slices creates division by zero, and too few slices give a degenerate or very coarse object. Intended calls therefore need a sensible positive slice count. The body loop closes at line 334, but the function continues by generating its indices and separate caps.

23. Original Lines 336-354: Cylinder Side Indices and Top Cap

The side-index loop runs from zero to `slices-1`. $b=i*2$ is the lower vertex in the current angular pair. The upper current vertex is $b+1$; the lower and upper next vertices are $b+2$ and $b+3$. Triangles $(b, b+1, b+3)$ and $(b, b+3, b+2)$ fill that rectangular panel. With 24 slices, the side contributes 48 triangles and 144 indices.

The top-cap section starts by storing `topCenter=v.size()`, the future center vertex index. It appends $(0, +0.5*h, 0)$ with upward normal $(0, 1, 0)$ and UV $(0.5, 0.5)$. The rim loop again includes the endpoint, creating

`slices+1` cap rim vertices. Each lies at $(0.5 \cdot \cos(\theta), +0.5, 0.5 \cdot \sin(\theta))$ and receives the same upward normal as the center, not the side's radial normal.

Cap UVs map the circle into a unit square: `U` is $0.5 + 0.5 \cdot \cos(\theta)$ and `V` is $0.5 + 0.5 \cdot \sin(\theta)$. The `+0.5` moves the circle center to the square center, and the `0.5` factor fits its coordinates into zero through one. The cap uses its own rim records even where positions coincide with side vertices, allowing the hard normal transition between side and lid.

The top-index loop makes a triangle fan using the center, next rim vertex, then current rim vertex. That reversed rim order is deliberate: when seen from above, it produces an upward-facing cap. Each slice contributes one top triangle. The closing braces finish the rim and fan loops; no cap thickness or bevel is generated. This is a flat closed disk, not a domed pressure-vessel head. A later sphere primitive can be used for a dome in an assembled model.

24. Original Lines 355-369: Cylinder Bottom Cap and Final Counts

The bottom cap mirrors the top but lies at `Y=-0.5`. Its center is assigned the index `btmCenter`, position $(0, -0.5 \cdot h, 0)$, downward normal $(0, -1, 0)$, and center UV $(0.5, 0.5)$. The inclusive rim loop generates the same angular samples and circular UV mapping at the lower height.

Its triangles use the center, current rim, then next rim. This order is opposite the top fan because the outward-facing direction is now downward. Simply copying the top index order would make the bottom's geometric face orientation disagree with its assigned normal. The separate rim and center vertices keep the lower cap flat-shaded with its own normal.

For `s` slices, the body has $2 \cdot (s+1)$ vertices, and each cap has one center plus `s+1` rim vertices. Total vertices are therefore $4 \cdot s + 6$: 102 at the default 24. Total triangles are $2 \cdot s$ on the side plus `s` on each cap, or $4 \cdot s$: 96 by default. The index count is $12 \cdot s$, or 288. These counts include duplicated seams and duplicated side/cap positions, which are necessary for the chosen shading and UV layout.

`return uploadMesh(v, idx)` sends the entire closed primitive to the GPU, and the closing function brace ends construction. No dimensions are exposed beyond the fixed unit geometry and slice count; arbitrary radius and height are obtained later through scaling. Nonuniform scaling requires a correctly transformed normal, which the scene vertex shader supplies through an inverse-transpose matrix.

25. Original Lines 371-392: createSphere and Latitude-Longitude Vertices

`createSphere(int lats=16, int lons=24)` produces a radius-0.5 UV sphere. The two defaults mean 16 latitude bands and 24 longitude sectors, not 16 times 24 total vertices. Local arrays hold the generated records and indices.

The outer loop includes all latitude endpoints, from `i=0` to 16. Its angle $\theta = i / \text{lats} \cdot \pi$ spans zero at the north pole to π at the south pole. `sinT` is the horizontal circle-radius factor and `cosT` the vertical component. The inner loop includes `j=0` through 24, with $\phi = j / \text{lons} \cdot 2 \cdot \pi$, so it also duplicates a seam.

The normal is $(\sin(\theta) \cdot \cos(\phi), \cos(\theta), \sin(\theta) \cdot \sin(\phi))$. Squaring and summing its components yields one in ideal arithmetic, so this is a unit outward direction. Multiplying by 0.5 gives a point on a sphere of radius 0.5 centered at the origin. UV is $(j / \text{lons}, i / \text{lats})$: `U` moves around the sphere, `V` moves from pole to pole. The vertex append stores those three values together.

The two closing braces end the nested vertex loops. With defaults, there are $(16+1) \cdot (24+1) = 425$ records. At each pole, many longitude records coincide spatially, but their UV values differ. Such duplication permits a rectangular parameterization while creating some degenerate pole triangles in the later indexing. As with the cylinder, zero divisors or unsuitable negative counts are not checked here. The function expects valid positive subdivisions; it neither adaptively refines the sphere nor loads an existing model.

26. Original Lines 394-410: Sphere Cell Indices and Degenerate Pole Triangles

The indexing loops visit 16 latitude bands and 24 longitude cells per band. Because a complete vertex row contains `lons+1` records, `first=i*(lons+1)+j` addresses the current row and longitude. `second=first+lons+1` addresses the same longitude on the next latitude row.

The first triangle is (first, first+1, second) and the second is (second, first+1, second+1). Together they fill a latitude-longitude cell. The ordering gives outward orientation away from the poles for this position parameterization. Each cell always appends six indices; there is no branch to remove collapsed pole faces.

With defaults, the total is $16 * 24 * 2 = 768$ submitted triangles and $16 * 24 * 6 = 2304$ indices. One triangle in each cell adjoining each pole has coincident pole vertices in exact mathematics, so 48 of the submitted triangles are degenerate. The remaining 720 represent the actual sphere surface. Floating-point sine near pi may produce tiny coordinate differences at the south pole, but the intended shape still has that pole collapse.

The nested closing braces finish the index loops, and `uploadMesh` returns the sphere's GPU record. Smooth normal interpolation gives a round-looking surface, while the silhouette remains limited by the chosen mesh resolution. UV poles compress many longitudinal samples into one spatial point; this familiar distortion is a limitation of UV spheres rather than an OpenGL error. The shader cannot make the 24-sector silhouette arbitrarily smooth merely by smoothing lighting.

27. Original Lines 412-436: createWingBlade and the Tapered Four-Point Profile

The wing builder creates a slender primitive along positive X, from root $X=0$ to tip $X=1$. The comment gives approximate tip values of 0.25 chord and 0.02 thickness, but the actual expressions use **0.22 chord and 0.015 thickness**. The code, not those rounded comments, defines the mesh.

`spanSegs=8` yields nine cross-sections through an inclusive loop. $t=i/8$ is a normalized span coordinate and $x=t$. Chord is $\text{mix}(1, 0.22, t) = 1 - 0.78 * t$; thickness is $\text{mix}(0.08, 0.015, t) = 0.08 - 0.065 * t$. At the root, thickness is 0.08; at the tip it is 0.015. The primitive has straight linear taper, not a physically derived airfoil equation.

Each section contains four points: leading edge ($x, 0, -0.45 * \text{chord}$), top crest ($x, +0.5 * \text{thick}, -0.1 * \text{chord}$), trailing edge ($x, 0, +0.55 * \text{chord}$), and bottom crest ($x, -0.5 * \text{thick}, -0.1 * \text{chord}$). The chord extends 45 percent forward and 55 percent backward relative to $Z=0$. The crests lie near the leading portion, making a thin diamond-like cross-section rather than an anatomically accurate insect-wing membrane.

Assigned normals are the coordinate directions $(0, 0, -1)$, $(0, 1, 0)$, $(0, 0, 1)$, and $(0, -1, 0)$. Normalizing these already-unit vectors changes nothing. Their UVs are $(t, 0)$, $(t, 0.33)$, $(t, 0.66)$, and $(t, 1)$. These normals are artistic approximations and ignore taper-induced X components; they are not computed from the actual sloped faces. Thirty-six vertex records result. The loop closes at line 436, and the mesh still needs connectivity along its span.

28. Original Lines 438-449: Wing Connectivity, Wraparound, and Open Ends

For each of eight span intervals, $b1=i * 4$ addresses the current section and $b2=(i+1) * 4$ the next. The inner loop visits all four profile edges. $kNext=(k+1)\%4$ advances to the next point and wraps index 3 back to 0, closing the cross-section around the leading edge.

Each edge receives triangles $(b1+k, b2+k, b2+kNext)$ and $(b1+k, b2+kNext, b1+kNext)$. Across eight intervals and four edges, this produces 64 triangles and 192 indices. Closing the profile around each section does not close the root and tip: no end-cap triangles are added. The blade is therefore an open-ended tapered shell. Its integration into a hub can hide the root opening, but that is an assembly decision outside this builder.

There is also a distinction between assigned shading normals and winding. For the upper-leading panel, the index order gives a geometric normal generally opposite the intended outward upper-leading direction, while vertex normals are assigned outward-looking axis directions. Thus the index order and the artistic normals should not be assumed to agree. With face culling enabled, this matters for which side is visible; with culling disabled, it can still yield lighting that does not match the geometric face orientation. The function does not correct winding as the rock builder later does.

The inner and outer braces close the profile and span loops. The return uploads the blade, and the final brace finishes construction. Wing count, flapping phase, and aerodynamic banking are not implemented in this primitive; repeated transformed draws can turn one blade mesh into an eight-wing aircraft.

29. Original Lines 451-471: createDuneTerrain, Reservations, and Vertex Sampling

The terrain builder defaults to `gridW=140`, `gridH=140`, and `totalSize=700`. It creates vectors and reserves space for $(\text{gridW}+1) * (\text{gridH}+1)$ vertices and $\text{gridW} * \text{gridH} * 6$ indices. `reserve` increases capacity without creating those elements; subsequent `push_back` calls still populate the vectors. This avoids repeated reallocations during predictable construction.

`halfSize=350` and `step=700/140=5` define the default horizontal spacing. The Z loop runs from zero through `gridH`, and the X loop from zero through `gridW`. World coordinates are `worldX=-350+5*x` and `worldZ=-350+5*z`, covering -350 through +350 on both axes. These are generated directly as terrain coordinates; a later model matrix can still transform the whole mesh.

`worldY=getDuneHeight(worldX, worldZ)` includes the landing terrace. `norm=getDuneNormal(...)` calculates the finite-difference shading normal. Together, one vertex requires its own height query plus four height samples for the normal. Default construction therefore involves 19,881 direct height evaluations and 79,524 normal-related height evaluations, or 99,405 calls to `getDuneHeight`, apart from the terrace array's one-time initialization.

UV is $(\text{worldX} * 0.05, \text{worldZ} * 0.05)$, so one UV unit corresponds to 20 world units. Its default range is -17.5 to +17.5; UV coordinates need not stay in $[0, 1]$. The scene's sand pattern currently uses world positions rather than this UV. The vertex append stores all data, and the braces finish both sampling loops.

A subtle limitation is that both axes use a step derived only from `gridW`. If `gridH` differs from `gridW`, the Z span is $\text{gridH} * \text{totalSize} / \text{gridW}$, not necessarily `totalSize`, and its positive endpoint may not be `+halfSize`. The defaults are square and avoid that mismatch, but the function's parameters suggest more flexibility than its step calculation actually provides.

30. Original Lines 473-489: Terrain Triangles and Finite Rendered Extent

The cell loops use strict less-than conditions, so they visit 140 by 140 cells instead of the inclusive 141 by 141 vertex positions. `row1=z*(gridW+1)` and `row2=(z+1)*(gridW+1)` locate neighboring vertex rows. Each cell is split into triangles $(\text{row1}+x, \text{row2}+x, \text{row1}+x+1)$ and $(\text{row1}+x+1, \text{row2}+x, \text{row2}+x+1)$.

On a flat Y-up grid, the first triangle's Z-forward edge crossed with its X-forward edge points upward, so this index order matches upward-facing terrain. Both triangles share the diagonal between the next-row current-X vertex and the current-row next-X vertex. Shared vertices allow adjacent cells to meet without cracks when their transformation is the same.

Defaults produce $140 * 140 = 19,600$ cells, 39,200 triangles, 117,600 indices, and 19,881 vertex records. The upload returns one static mesh. The function does not regenerate vertices around the camera, create terrain chunks, or implement level of detail. The height query is mathematically continuous over the horizontal plane, but the rendered mesh is a **finite sampled patch** covering a 700-unit square.

Within each triangle, rasterization interpolates its planar geometry. It does not evaluate the CPU sine height function at every interior pixel. Collision/height queries at intermediate coordinates can therefore disagree slightly with the triangle surface, particularly near sharper ridges or terrace boundaries. Shader ripples later improve apparent detail without adding geometric vertices. The two loop-closing braces, `return`, and function-closing brace finish the terrain builder and do not change any simulation state beyond GPU-object creation.

31. Original Lines 491-501: createQuad and the Particle Unit Square

The quad comment identifies its billboard use. `createQuad()` initializes four Vertex records directly: bottom-left $(-0.5, -0.5, 0)$, bottom-right $(0.5, -0.5, 0)$, top-right $(0.5, 0.5, 0)$, and top-left $(-0.5, 0.5, 0)$. All have normal $(0, 0, 1)$. Their UVs are the corresponding corners $(0, 0)$, $(1, 0)$, $(1, 1)$, and $(0, 1)$.

This is a unit square in the local XY plane, centered on the origin. The initializer's braces group the vector, each vertex, and each member's coordinate components. They are data initialization, not nested loops. Indices $\{0, 1, 2, 0, 2, 3\}$ make two counterclockwise triangles facing positive Z.

The return uploads four vertices and six indices. The primitive is not inherently camera-facing; **billboarding is implemented by the particle vertex shader**, which combines local X/Y with camera-right and camera-up vectors. The stored normal is unused by that shader, which does not declare location 1. Keeping the common Vertex format lets this quad use the generic upload routine anyway.

The particle fragment shader will discard fragments outside a UV-space circle, so a square mesh can visually appear as a soft circular dust patch. The geometry itself remains two square-covering triangles. This separation between simple geometry and shaped opacity is an efficient rendering technique, not the creation of a true circular polygon or a volumetric cloud.

32. Original Lines 503-517: createRing and a Thin Torus

`createRing()` creates a torus: a circular tube swept around a larger circle. The compact nested loop has $i=0..80$ around the major circle and $j=0..8$ around the tube cross-section, both inclusive. Angles are $a=i*2\pi/80$ and $b=j*2\pi/8$.

The normal is $(\cos(a)*\cos(b), \sin(b), \sin(a)*\cos(b))$, of unit length in exact arithmetic. The major-circle centerline is $(\cos(a), 0, \sin(a))$, with radius one in the XZ plane. Adding $0.025*n$ supplies tube radius 0.025. Consequently the ring's outer horizontal radius is 1.025, its inner radius 0.975, and its vertical range is -0.025 to +0.025. UV is $(i/80, j/8)$, with duplicated endpoint records for both seams.

The second nested loop visits 80 by 8 cells. $a=i*9+j$ is an index here, reusing the earlier name in a separate scope; it is no longer an angle. $b=a+9$ addresses the next major-circle row because each row has nine tube samples. `indices.insert(indices.end(), {...})` appends six indices at once: two triangles $(a, a+1, b)$ and $(b, a+1, b+1)$. This is functionally similar to six `push_back` calls.

The mesh has $81*9=729$ vertices, 1,280 triangles, and 3,840 indices. The eight-sided tube gives it a polygonal cross-section despite smooth normals. The return uploads it and closes the builder. The shape is not a flat annulus: it has a thin three-dimensional circular tube, which can be scaled or placed as a visual marker later.

33. Original Lines 519-526: createErodedRock and the Point-Generating Lambda

The rock builder creates empty arrays and a captureless lambda point. Its `[]` capture means it uses only its arguments and accessible global/static functions, not local builder variables. Parameters `lat` and `lon` are integer grid indices. Angle $a=lat*\pi/8$ follows eight latitude bands, while $b=lon*2\pi/13$ follows thirteen longitude sectors.

The erosion factor is $0.83 + 0.13*\sin(7.1*lon+2.3*lat) + 0.08*\cos(5.4*lat)$. Independent bounds place it between 0.62 and 1.04. This name describes an artistic irregularity; there is no iterative erosion process, water simulation, or random seed. The perturbation is deterministic and depends on integer indices rather than on a continuous world-space noise field.

The returned point is $0.5*(\sin(a)*\cos(b)*erosion, \cos(a), \sin(a)*\sin(b)*erosion)$. The erosion factor changes X/Z radius but not Y. Y still ranges approximately -0.5 to +0.5, while maximum horizontal radius is bounded by 0.52. The result resembles a vertically parameterized irregular rock rather than a uniformly perturbed sphere.

There is a seam limitation: `point(lat, 0)` and `point(lat, 13)` have matching angular directions but generally **different erosion factors**, because the sine expression in `lon` is not periodic over thirteen samples. The longitude strip closes in angle without necessarily closing in position, leaving a geometric mismatch along the seam away from the poles. The lambda closes at line 526; later code consumes its points to create flat-shaded triangles and handles degenerate pole faces.

34. Original Lines 527-541: Rock Triangle Lambda, Winding Repair, and Flat Shading

The local `tri` lambda captures the vectors by reference and accepts three positions by value. `cross(b-a, c-a)` computes an area-weighted triangle normal. Its length equals twice the triangle area. If that length is below 0.00001, the lambda returns early, avoiding normalization of an almost-zero vector and excluding collapsed pole triangles.

The orientation test is `dot(n, a+b+c) < 0`. Since the geometry is centered around the origin, the sum of its three vertex positions points roughly toward the triangle's exterior location. A negative dot suggests that its normal points

inward. The branch swaps B and C to reverse winding and negates n to match the new order. This is an origin-relative heuristic appropriate to the intended roughly convex rock, not a universal normal repair for arbitrary concave meshes.

After normalization, `start` records the vertex count using an explicit `static_cast<unsigned int>`. Three vertices are appended with the same normal and UVs $(0, 0)$, $(1, 0)$, $(1, 1)$, followed by indices `start`, `start+1`, and `start+2`. Each triangle gets separate vertex records, even where corners coincide with another face. That duplication prevents interpolation across different face normals and creates a deliberately faceted appearance.

The nested loops visit eight latitude bands and thirteen sectors. Each cell attempts two triangles using the point `lambda`. There are 208 attempts; in ideal geometry, 26 pole-degenerate triangles are skipped, leaving 182 nondegenerate triangles, 546 independent vertex records, and 546 indices. The threshold also rejects tiny numerical pole artifacts. There is no global UV unwrap: every triangle gets the same small UV pattern. The return uploads the resulting rock. Winding repair does not weld its seam or turn this artistic perturbation into a physically eroded geological model.

35. Original Lines 543-557: Shared Mesh Handles and Packed Particle Instances

The mesh-assets comment introduces eight global Mesh records: `g_meshCube`, `g_meshCylinder`, `g_meshSphere`, `g_meshWing`, `g_meshTerrain`, `g_meshQuad`, `g_meshRing`, and `g_meshRock`. Their names connect them to the builders just explained. Each begins with the structure's zero handles, zero index count, and triangle draw mode. Merely declaring them does not invoke the corresponding builder; later initialization must assign uploaded meshes.

`g_particleInstanceVB0 = 0` stores the future buffer handle for per-particle instance data. `ParticleInstance` is a compact record with two `glm::vec4` members. `posSize` packs position in X/Y/Z and size in W. `colorAlpha` packs RGB color and alpha in W. This packing matches particle shader inputs at locations 3 and 4. It is distinct from `SandParticle`: simulation stores velocity and lifetime, while the instance record stores only what drawing needs.

`g_particleInstances` is an initially empty vector of those packed draw records. Later code can rebuild it from the simulation pool and upload many instances for one instanced draw. These lines do not establish attribute divisors or perform the draw; declaring attributes in a shader does not automatically make a buffer advance once per instance. Proper VAO/buffer configuration must be supplied elsewhere.

The one-line structure's braces and semicolon define its member grouping. The storage format assumes that the upload and attribute layout agree on stride and offsets; using `sizeof` and `offsetof` is preferable to assuming a particular GLM packing configuration. Separating simulation and rendering records avoids sending unused velocity/lifetime values to the GPU for this simple billboard shader.

Surface Shader Interface and Vertex Processing

36. Original Lines 558-580: SCENE_VERT, Attributes, Uniforms, and Varyings

Lines 558-560 label shader data. The surface-shader comment names intended lighting features; the vertex shader itself handles geometry and passes data to a later fragment shader. `const char* SCENE_VERT = R"(` starts a C++ raw string. Its contents are GLSL source, not directly executed C++. Raw-string syntax lets embedded newlines and quotation marks appear without ordinary escape sequences. A later compiler call sends this text to OpenGL.

`#version 330` core requests GLSL 3.30 in the core language profile. Attributes at locations 0, 1, and 2 are `aPos`, `aNorm`, and `aUV`, matching the generic mesh upload. These are per-vertex values. **Uniforms** are values set by C++ that remain constant for a draw rather than vary for each vertex:

- `model` maps mesh-local positions into world coordinates. It also determines the normal transformation.
- `view` maps world coordinates into camera coordinates.
- `projection` maps camera coordinates into clip coordinates for perspective or another chosen projection.
- `lightSpaceMatrix` maps world coordinates into the sun/shadow camera's clip space.
- `wormCenter` supplies the world-space center for terrain depression; only its X/Z components are used here.

- `collapse` supplies the scalar amount of depression. This range declares it but supplies no default or runtime maximum.

- `isSand` selects a material/geometry mode. Value 1 enables terrain depression; other values do not in this shader.

Outputs `FragPos`, `Normal`, `TexCoord`, and `FragPosLight` carry world position, transformed normal, UV, and light clip-space position to the fragment stage. Default smooth interpolation blends vertex outputs across triangles, normally with perspective correction. Passing a vector named `Normal` does not automatically keep it unit length after interpolation; the fragment shader normalizes it again. The light output is a four-component homogeneous coordinate and must later be divided by `W` before texture-coordinate comparison.

37. Original Lines 582-594: Scene Vertex `main`, Collapse, and Coordinate Transforms

GLSL's `main` is the entry point executed for each vertex. `worldPos=model*vec4(aPos,1)` places a local vertex in world space. The `if(isSand==1)` branch limits terrain collapse to the designated sand mode.

`d=length(worldPos.xz-wormCenter.xz)` is horizontal radial distance from the worm; `Y` differences do not matter.

The depression is `collapse*(1-smoothstep(5,48,d))*13`, subtracted from `Y`. Within five units, the radial weight is one; beyond 48 it is zero. In the transition band, a smooth cubic reduces the displacement. If `collapse=1`, the central depression is 13 units; if `collapse=0.5`, it is 6.5. At distance 26.5, halfway between the two radii, the weight is 0.5. Nothing here clamps `collapse`, so a negative value would raise terrain and a value above one would deepen it beyond thirteen units.

`FragPos=worldPos.xyz` passes the displaced world position.

`Normal=mat3(transpose(inverse(model)))*aNorm` applies the inverse-transpose normal matrix and discards translation through conversion to `mat3`. The inverse transpose corrects orientation under nonuniform scaling. It assumes `model` is invertible; a zero scale can make inversion undefined. The fragment shader later renormalizes the result.

Importantly, the normal does **not** incorporate the derivative of the collapse deformation. The vertex height moves but its supplied terrain normal remains based on the undepressed CPU heightfield. CPU `getDuneHeight` queries also remain unchanged. This can cause lighting and object/particle placement to disagree with the visible crater. The deformation is sampled only at existing terrain vertices, so its smooth radial formula is represented by a finite triangle mesh.

`TexCoord=aUV` passes UV unchanged. `FragPosLight=lightSpaceMatrix*worldPos` uses the displaced position so shadows refer to the same geometry. `gl_Position=projection*view*worldPos` supplies the camera's clip-space position, with matrices applied right to left. The closing GLSL brace ends `main; )`; closes the raw string and its C++ declaration, not another rendering pass.

38. Original Lines 596-620: SCENE_FRAG Inputs and Every Surface Uniform

`SCENE_FRAG` begins a second GLSL raw string at version 330 core. Inputs `FragPos`, `Normal`, `TexCoord`, and `FragPosLight` match the vertex outputs by name and type. `FragColor` is the four-component color result. Although UV is declared, the visible surface calculations below do not use `TexCoord`; a compiler may optimize that varying and its input attribute away for this program.

The material uniforms are `objectColor`, the RGB base/diffuse color; `specularColor`, an RGB multiplier for highlights; `shininess`, the specular exponent; `emissiveColor`, an added self-lit RGB term; and `isSand`, the integer mode selector. Value zero uses ordinary material behavior. Value one receives sand ripples, wrap lighting, and the vertex collapse. Value two receives rock-like grain and layering but no sand collapse or wrap branch. Despite its name, `isSand` is therefore more than a Boolean.

Lighting uniforms are `lightDir`, documented as the direction **toward** the sun; `sunColor`, the RGB direct-light multiplier; `ambientColor`, the RGB ambient-light multiplier; and `viewPos`, the world-space camera position.

Directional light uses one common direction for the entire scene, with no distance attenuation from a sun position. The fragment shader normalizes `lightDir` for its local lighting vector, but the shadow bias later uses the uniform directly, so the caller should honor the normalized-direction contract.

`sampler2D shadowMap` identifies a texture unit from which ordinary depth samples are read. It is not a `sampler2DShadow`, so the shader performs the depth comparison itself. `fogColor` is the RGB endpoint for distant haze, and `fogDensity` controls the exponential-squared distance response. No initial values are assigned in this string; C++ must supply intended scene values. The fog comment describes an artistic warm-desert effect, not a measured atmospheric scattering model. The shader has no uniform for a diffuse texture, roughness map, metalness, point lights, or normal-map image.

Surface Shadow Filtering

39. Original Lines 621-635: `calcShadow`, Projection, Receiver Slope, and Bias

`calcShadow(vec4 fragLight, vec3 norm)` returns a scalar occlusion fraction. It first divides light clip-space XYZ by W to obtain normalized device coordinates. Multiplying by 0.5 and adding 0.5 changes the usual $[-1, 1]$ coordinate range into $[0, 1]$, suitable for texture coordinates and depth comparison. The code assumes a valid nonzero W; there is no guard against division by zero.

`dFdx(proj)` and `dFdy(proj)` estimate how projected coordinates vary over neighboring screen fragments. Here `dx` and `dy` are not world-space terrain samples: they are GPU fragment derivatives. Their X/Y components describe changes in shadow-texture coordinates and their Z components describe changes in receiver depth. The determinant `dx.x*dy.y - dx.y*dy.x` measures whether those two projected screen directions define an invertible two-dimensional mapping.

The desired receiver-depth slope (`sx, sy`) satisfies $dx.z = sx * dx.x + sy * dx.y$ and $dy.z = sx * dy.x + sy * dy.y$. Solving these two equations gives exactly the code's numerator pair divided by the determinant. The initial slope is $(0, 0)$. Only if `abs(determinant) > 0.000000001` does the shader apply the division; near-degenerate mappings retain zero correction rather than amplify numerical noise through an unstable inverse.

The following one-line conditional returns zero shadow if Z is below zero or above one, or if either texture coordinate is below zero or above one. `lessThan` and `greaterThan` return Boolean component vectors; any tests whether at least one component qualifies. A fragment outside the light's map is treated as unshadowed rather than clamped into the map. Equal endpoints are accepted. Derivatives are computed before this early return.

Bias is `max(0.0035*(1-dot(norm, lightDir)), 0.0008)`. A surface facing the sun receives the minimum 0.0008; perpendicular directions give 0.0035; perfectly opposite directions would give 0.007 if both vectors are unit length. Bias shifts comparison toward "lit" to reduce self-shadow acne, at the risk of detached or weakened shadows. These are **normalized depth units**, not world distances. `shadow=0` initializes the accumulator. `texelSize=1/textureSize(shadowMap, 0)` derives one base-level texel step; for a 2048-square texture, each component is $1/2048 = 0.00048828125$.

40. Original Lines 636-645: The Exact 5-by-5 PCF Loop

The comment specifies a 5-by-5 filter. The outer loop uses integer X offsets -2,-1,0,1,2, and the inner loop uses the same Y offsets. There are exactly 25 iterations, not a 3-by-3 filter or an adaptive sample count. Each integer pair multiplied by `texelSize` becomes a shadow-texture offset, spanning two texels in either direction from the receiver.

`texture(shadowMap, proj.xy+offset).r` reads the stored depth sample's first component. The texture is configured later with nearest-neighbor filtering, so each fetch selects a depth texel instead of averaging neighboring depths. The shader does the comparison manually: the sample counts as shadowed when `proj.z + dot(depthSlope, offset) - bias > pcfDepth`.

The dot product estimates how the receiver's own depth changes between the central projected coordinate and this tap. Without it, a sloping surface might compare every tap against one center depth and generate striped self-shadow artifacts at low sun angles. Subtracting bias makes the test less likely to label the receiver as behind itself. The ternary adds 1 for a failed light-visibility test and 0 otherwise. Strict greater-than means equal biased depth is treated as lit.

The two loop braces close the 25-sample traversal. `return shadow/25` gives values from zero to one in increments of 0.04. Ten occluded taps return 0.4; all 25 return one. This is **percentage-closer filtering**: it averages comparison results, not raw depths, to soften shadow edges. It does not estimate a real sun's angular size, sample a physical

penumbra, or provide contact-hardening soft shadows. Its fixed texel footprint also changes apparent world-space softness if the light's covered region changes.

The earlier bounds test checks the center only. Edge taps can extend beyond the texture, where the later white border-depth setting supplies depth one. Such taps generally count as lit and soften shadows near map boundaries. The final brace closes `calcShadow`; no color has yet been produced.

Surface Material and Lighting Calculations

41. Original Lines 647-658: Fragment Directions and Rock-Mode Color Variation

Surface fragment main begins by normalizing `Normal`, the interpolated world-space normal, to obtain `N`. `L=normalize(lightDir)` points toward the sun. `V=normalize(viewPos-FragPos)` points from the surface toward the camera. `H=normalize(L+V)` is the halfway direction between light and viewer, used by Blinn-Phong highlights. If the inputs yield a zero vector, such as exact opposing `L` and `V`, normalization is not guarded; normal operation assumes useful scene directions.

`baseCol=objectColor` starts with the material's RGB color. When `isSand==2`, the shader creates rock-style variation. `grain=fRACT(sin(dot(FragPos,(12.3,42.8,27.1)))*43758.5453)` reduces a world-position-dependent sine value to a fractional part. `fRACT(x)=x-floor(x)`, so the result is in $[0, 1]$ even for negative inputs. These constants are a common inexpensive hash-like visual pattern, not a rigorous random generator, and floating-point implementation differences can affect its precise appearance.

`layers=sin(5*FragPos.y + sin(0.8*FragPos.x))` creates bands chiefly controlled by height, with an X-dependent phase wobble. The nested sine shifts those layers rather than modifying actual geometry. At fixed X, the vertical repeat interval is $2\pi/5$, approximately 1.2566 units. The X phase wobble repeats every $2\pi/0.8$, approximately 7.854 units.

The color is multiplied by $0.88+0.12*layers+0.07*grain$. Its theoretical range is from 0.76 to just under 1.07. Thus this mode mostly darkens the supplied color but can slightly brighten it. It does not alter normals, shininess, or shadow depth, and it is not a texture image. The branch closes at line 658. Ordinary mode zero and sand mode one skip this rock variation entirely.

42. Original Lines 660-679: Sand Ripples, Slope Tint, Distance Fade, and Sparkles

The sand branch runs only for `isSand==1`. `ripple1=0.5*sin(2.2*x+1.4*z)+0.5` and `ripple2=0.5*sin(0.8*x-1.9*z)+0.5` create two wave patterns remapped into $[0, 1]$. Their 50-percent mix is their arithmetic average. Their perpendicular wavelengths are approximately 2.410 and 3.048 world units. The comment mentions wind alignment, but these numerical directions are hard-coded; they do not read the CPU `WIND_DIR` constant or adapt if wind changes.

`slope=clamp(dot(N,(0,1,0)),0,1)` measures upward orientation. The color mix between $0.82*baseCol$ and $1.15*baseCol$ is equivalent to multiplying by $0.82+0.33*slope$. Horizontal upward ground is brightest; steep or downward-facing orientations receive the darker factor. This is not a direct crest-versus-valley detector. A flat valley and a flat crest can both have upward normals and receive the same multiplier.

The fade is `1-smoothstep(25,120,length(viewPos-FragPos))`. Ripples and sparkle retain full strength within 25 units and vanish at 120 units or beyond. At 72.5 units, fade is 0.5. The color addition $(0.035,0.02,0.008)*(ripple-0.5)*fade$ shifts warm tint around zero, with component bounds of plus/minus 0.0175, 0.01, and 0.004 at full fade.

The normal is perturbed by $(0.055*\cos(q),0,0.035*\cos(q))*fade$, where $q=2.2*x+1.4*z$, then renormalized. This uses only the first ripple's phase and no Y perturbation. Its maximum added-vector magnitude is about 0.06519. It is a small artistic bump approximation, not the derivative of a displaced sand surface. Geometry and CPU heights stay unchanged. The split source lines 672-673 form one expression, not two separate updates.

Sparkle is `pow(max(dot(N,H),0),120)`, a narrow halfway-direction highlight. A second world-XZ hash uses coefficients (12.9898, 78.233) and the same large sine multiplier. Only `noise>0.88` adds

$(0.35, 0.3, 0.22) * \text{sparkle} * \text{fade}$ to base color. Roughly twelve percent of a well-distributed hash field meets that threshold, but it is not an exact particle count or probabilistic guarantee. Because this is added to base color before all lighting, it can influence ambient lighting too; "under direct sun" is an artistic description rather than an explicit sunlight-only gate. The closing brace ends sand-specific processing.

43. Original Lines 681-697: Ambient, Wrapped Diffuse, Specular, and Shadow Combination

Ambient lighting is the componentwise product $\text{ambientColor} * \text{baseCol}$. It is a constant-direction-independent approximation to environmental bounce light, not calculated interreflection or ambient occlusion. It remains present in full shadow because the later shadow multiplier does not affect this term.

$\text{NdotL} = \text{dot}(\text{N}, \text{L})$ measures sun-facing orientation. Ordinary materials and rock mode use $\text{diff} = \max(\text{NdotL}, 0)$, Lambert-style diffuse intensity. Sand instead uses $\max((\text{NdotL} + 0.35) / 1.35, 0)$. At $\text{NdotL} = 1$, the result is one; at zero it is about 0.25926; at -0.35 it becomes zero. Slightly sun-away surfaces therefore retain diffuse contribution, producing a soft wrapped appearance. This differs from a canonical $(0.5 * \text{NdotL} + 0.5)^2$ half-Lambert equation despite the comment's shorthand.

$\text{diffuse} = \text{diff} * \text{sunColor} * \text{baseCol}$ combines scalar illumination with colored sunlight and material color. The products of two RGB vectors are componentwise. Specular uses $\text{spec} = \text{pow}(\max(\text{dot}(\text{N}, \text{H}), 0), \text{shininess})$ and $\text{specular} = \text{spec} * \text{specularColor} * \text{sunColor}$. Larger positive exponents narrow highlights. For the default exponent 32, a halfway dot of 0.9 gives about 0.03434, while 0.99 gives about 0.72498. Material defaults are declared later; individual models can supply other exponents.

$\text{shadow} = \text{calcShadow}(\text{FragPosLight}, \text{N})$ uses the possibly ripple-perturbed normal for bias while receiver-plane slope comes from the projected position derivatives.

$\text{litColor} = \text{ambient} + (1 - \text{shadow}) * (\text{diffuse} + \text{specular}) + \text{emissiveColor}$ applies one visibility factor to both direct-light terms. A shadow of 0.4 retains 60 percent of diffuse and specular; a full shadow removes those terms but not ambient or emission.

The specular term is not explicitly gated on $\text{NdotL} > 0$, so a suitably oriented halfway vector can produce a highlight even when the light is behind the normal. emissiveColor brightens this fragment but does not cast light onto surrounding objects. There is no energy-conserving material model, multiple-scattering calculation, or transparency here. Comments separating the terms are explanatory labels, not extra execution.

44. Original Lines 698-706: Exponential-Squared Fog, Tone Mapping, and Opaque Output

$\text{dist} = \text{length}(\text{viewPos} - \text{FragPos})$ is three-dimensional camera-to-fragment distance, not just ground-plane distance or projected depth. Fog weight is $f = 1 - \exp(-(\text{dist} * \text{fogDensity})^2)$, clamped to $[0, 1]$. If the product of distance and density is 0.5, f is about 0.22120; at one it is about 0.63212; at two it is about 0.98168. The clear-image contribution is $1 - f = \exp(-(\text{dist} * \text{density})^2)$.

Density zero disables this fog for finite distances. Because density is squared as part of the product, a negative value would produce the same weight as its positive magnitude; the shader does not validate that a physically sensible nonnegative value was supplied. The clamp limits rounding or unusual results but is not a substitute for valid uniform data.

Before fog, each color channel is mapped as $\text{mapped} = (\text{lit} / (\text{lit} + 0.8))^{(1/1.8)}$. The rational fraction compresses positive bright values toward one, and the exponent is approximately 0.55556. For lit values 0, 0.8, and 1.6, the pre-exponent fractions are 0, 0.5, and 2/3. After the exponent, the latter two are approximately 0.68040 and 0.79833. This is an artistic tone/gamma-like transform, not a specified standard sRGB conversion or an HDR framebuffer guarantee.

The expression assumes nonnegative useful lighting values. A negative color can produce a negative fractional-power input or an unsuitable denominator; no explicit nonnegative clamp is present. $\text{mix}(\text{mapped}, \text{fogColor}, \text{fogFactor})$ then interpolates toward the supplied fog color. Fog color is mixed **after** the surface tone mapping and is not passed through that same expression in this shader.

`FragColor=vec4(...,1)` writes alpha one, making every surviving surface fragment opaque. Dark canopy colors therefore do not imply transparent glass. The GLSL brace closes `main`, and the raw-string terminator closes the C++ fragment-source declaration. Depth testing and blending state remain responsibilities of the draw pipeline outside this string.

Shadow and Particle Shader Programs

45. Original Lines 708-727: SHADOW_VERT, Matching Collapse, and Empty Fragment `main`

The shadow-pass comment introduces `SHADOW_VERT`, another GLSL 330 core raw string. It needs only position at attribute location zero. Its uniforms are `lightSpaceMatrix` for light clip coordinates, `model` for local-to-world placement, `wormCenter` for collapse center, `collapse` for displacement amount, and `isSand` for mode selection. It does not need UVs, normal attributes, material color, or camera matrices because it is constructing a depth map rather than a shaded image.

The vertex `main` starts with `p=model*vec4(aPos,1)`. The one-line sand conditional subtracts exactly the same `collapse*(1-smoothstep(5,48,length(p.xz-wormCenter.xz)))*13` depression as the scene vertex shader. Matching these deformations is essential: if the shadow pass rendered undeformed terrain while the visible pass rendered a crater, stored depths would disagree with visible receivers and casters.

`gl_Position=lightSpaceMatrix*p` projects into the light camera. The vertex function and its raw string then close. `SHADOW_FRAG` starts another version-330 string with `void main(){}`. The body is deliberately empty: no color output is required for a depth-only framebuffer. Rasterization and the fixed depth pipeline can still update its depth attachment using interpolated projected depth, subject to the OpenGL depth-state configuration established elsewhere.

An empty fragment shader is not the same as discarding all fragments. It simply omits a color computation. This shader does not explicitly write `gl_FragDepth`, pack depth into RGB, or store linear world-space distance. The framebuffer uses an actual depth texture, and the resulting values follow the chosen projection's depth mapping. The final string terminator and blank line end this pair of shader declarations, not the execution of a shadow frame.

46. Original Lines 728-750: PARTICLE_VERT and Instanced Camera-Facing Geometry

The particle comment calls these soft billboards. `PARTICLE_VERT` uses GLSL 330 core. Location zero supplies quad position; location two supplies UV. Location three supplies `instancePosSize`, whose XYZ are the particle center and W is size. Location four supplies `instanceColorAlpha`, whose RGB are color and W is opacity. These names match the earlier packed `ParticleInstance` record concept, though buffer association and per-instance divisors are established outside this range.

Uniforms `view` and `projection` are the camera transforms. `camRight` and `camUp` are world-space basis directions for the billboard plane. The caller should supply appropriately normalized, perpendicular directions if it wants a square of the requested size. Otherwise the vectors themselves can stretch, skew, or rotate the sprite. There is no per-particle model matrix, normal, or lighting direction in this shader.

Outputs `TexCoord` and `ColorAlpha` pass UV and packed color to the fragment stage. `main` first copies those values, then computes `worldPos=center+camRight*(aPos.x*size)+camUp*(aPos.y*size)`. For the unit quad, local X/Y span -0.5 to +0.5, so the full billboard width and height equal `size` when the basis is orthonormal. Local Z is ignored. Every vertex is built directly around the particle center rather than rotating an already placed mesh.

`gl_Position=projection*view*vec4(worldPos,1)` projects the billboard. The function and raw-string declaration close. With proper instancing setup, many particles share one quad mesh and this program while fetching different packed centers and colors. This shader alone does not sort translucent sprites, determine lifetime fade, implement world collisions, or sample a texture. "Soft" appearance comes from the next fragment shader's opacity profile, not from soft intersection against scene depth.

47. Original Lines 752-765: PARTICLE_FRAG, Circular Discard, and Opacity Falloff

The particle fragment string declares GLSL 330 core, input `TexCoord`, output `FragColor`, and input `ColorAlpha`. There are no uniforms in this fragment stage. The interpolated UV square is used as a mathematical opacity mask rather than coordinates into a sprite-image sampler.

`dist=length(TexCoord-vec2(0.5))` measures UV-space distance from the square center $(0.5, 0.5)$. The scalar constructor `vec2(0.5)` sets both components to 0.5. `if(dist>0.5) discard` removes fragments outside the inscribed circle; a corner has distance $\sqrt{0.5^2+0.5^2}$, about 0.7071, and is discarded. An exact point at radius 0.5 is not discarded, though its computed alpha becomes zero.

`soft=1-smoothstep(0,0.5,dist)` makes opacity one at the center and zero at the circle edge. At radius 0.25, the normalized smoothstep parameter is 0.5, so `soft` is 0.5. The output is

`vec4(ColorAlpha.rgb, soft*ColorAlpha.a)`: RGB is unchanged, while alpha falls smoothly to the boundary. This is ordinary, non-premultiplied color output because RGB is not multiplied by the resulting alpha.

The softness is not a Gaussian density profile, a volumetric integration, or depth-based soft-particle fading. Proper blending state must combine this alpha with the existing image, and appropriate depth testing/depth-write policy must prevent unwanted occlusion. Those states are not declared here. Unsorted overlapping translucent sprites can still exhibit order artifacts. Closing `main` and the raw string completes the shader source without issuing a draw.

Procedural Sky

48. Original Lines 767-780: SKY_VERT and Reconstructing World-Space Rays

The sky comment introduces a procedural clear desert sky rather than a cubemap texture. The vertex source declares GLSL 330 core, position at location zero, output `RayDir`, and uniform `invViewProj`. The uniform is the inverse of the camera's combined view-projection transform, conventionally `inverse(projection*view)`; the CPU must calculate and supply it.

In `main`, `farPoint=invViewProj*vec4(aPos.xy,1,1)` transforms a clip-space point at far-plane normalized $Z=+1$. `nearPoint=invViewProj*vec4(aPos.xy,-1,1)` transforms the matching near-plane point. Their XYZ components are divided by their own W values to recover world-space positions. Subtracting near from far gives a world-space ray direction through that screen coordinate. Removing the two points also cancels camera translation, so sky direction depends on orientation rather than absolute camera location.

The code assumes valid invertible camera matrices and nonzero resulting W components. An infinite-far projection would require careful treatment of the far homogeneous point; this shader does not guard for $W=0$. `RayDir` is not normalized yet; the fragment shader does that after interpolation.

`gl_Position=vec4(aPos.xy,0.9999,1)` places the fullscreen geometry close to the far depth boundary. Under the default OpenGL normalized-depth mapping, 0.9999 corresponds to depth approximately 0.99995. The input Z is ignored. This positioning helps a background sky coexist with scene depth, but actual results still depend on depth state and draw order outside these lines.

The later setup supplies three vertices, so this is a **fullscreen triangle**, not a geometric skybox or fullscreen quad. Its interpolation supplies a direction at each covered fragment. The brace and string terminator finish the source declaration.

49. Original Lines 782-800: SKY_FRAG and the Blue-Gold Elevation Gradient

The sky fragment source declares version 330, input `RayDir`, output `FragColor`, and uniforms `lightDir` and `sunColor`. Here the same conceptual sun direction locates the visible sun, while the sun color determines its additive RGB contribution. There are no cloud textures, cubemap samplers, fog-density uniform, or time-dependent noise inputs.

`rd=normalize(RayDir)` converts the reconstructed vector into a unit viewing direction. The three constant colors are `horizonDust=(0.90,0.74,0.47)`, a warm yellow-brown haze; `lowBlue=(0.48,0.69,0.88)`, a pale lower-sky blue; and `zenithBlue=(0.10,0.34,0.72)`, a deeper overhead blue. RGB values are numerical artistic choices, not samples from a physical atmospheric model.

`elevation=clamp(rd.y, 0, 1)` takes the vertical component of the ray. Since `rd` is unit length, this equals the sine of elevation angle above the horizontal, not the angle itself. Directions at or below the horizon are clamped to zero and receive the same initial warm haze color rather than an independently modeled lower hemisphere.

The first mix uses `smoothstep(0, 0.13, elevation)` to blend haze into low blue near the horizon. An elevation component of 0.13 corresponds to an angle of approximately $\text{asin}(0.13) = 7.47$ degrees. The second mix blends that result toward zenith blue with `elevation^0.65`. Because this exponent is below one, the weight grows comparatively quickly at low positive elevations. At component 0.5, the weight is about 0.63728; at one it is one, giving exact zenith blue before sun addition.

The two blends are sequential, so the final low-sky color is not merely a piecewise selection among three fixed values. The comment explicitly notes the absence of cloud noise, which matches the actual calculations. Line 800 is a blank separating gradient construction from the sun terms.

50. Original Lines 801-810: Sun Disk, Corona, Haze, and Untonemapped Sky Output

`sunDot=max(dot(rd, normalize(lightDir)), 0)` is the cosine of the angle to the sun, clipped at zero for the opposite hemisphere. Both participating directions are normalized here. The disk uses `smoothstep(cos(radians(0.80)), cos(radians(0.58)), sunDot)*1.9`. The `radians` calls convert degrees to the units expected by `cos`.

Cosine decreases as angle increases over this small range, so the first threshold, at 0.80 degrees, is smaller than the second, at 0.58 degrees. The `smoothstep` therefore transitions from zero beyond the outer angular radius toward full 1.9 strength inside the inner radius. The inner disk diameter is 1.16 degrees; the outer soft boundary diameter is 1.60 degrees. These are chosen visual sizes, not an inferred physical size from a sun distance and radius.

`sunCorona=pow(sunDot, 180)*0.30` creates a concentrated halo around the same direction.

`sunHaze=pow(sunDot, 12)*0.10` creates a much broader, weaker glow. At exact alignment, the three coefficients add to $1.9+0.3+0.1=2.3$. `finalSky=skyBase+(sunDisk+sunCorona+sunHaze)*sunColor` adds that colored intensity to the elevation gradient.

`FragColor=vec4(finalSky, 1)` outputs opaque sky. Unlike the surface shader, this shader does not apply the rational tone map or the $1/1.8$ exponent. Values can exceed one near the sun; whether those values are clipped or retained depends on the actual render-target format and later pipeline. There is no bloom calculation despite the bright disk, no clouds, and no physical wavelength-dependent scattering. The GLSL and C++ string close at lines 809-810, finishing the final shader-source pair in this interval.

Shader Compilation and GPU Targets

51. Original Lines 812-828: `g_shaderOkay` and `compileShaderModule`

The management comment introduces CPU-side shader creation. `g_shaderOkay=true` is a shared success flag. The functions below turn it false when they encounter a reported shader, program, or framebuffer failure. They do not set it true again after a later success, so it accumulates failure unless some other code explicitly resets it.

`compileShaderModule(GLenum type, const char* src)` accepts a shader-stage identifier and a pointer to its source text. `glCreateShader(type)` obtains the shader object handle `s`; typical callers supply `GL_VERTEX_SHADER` or `GL_FRAGMENT_SHADER`. `glShaderSource(s, 1, &src, nullptr)` supplies one source string. Passing the address of `src` meets the API's array-of-string-pointers interface, while the null length pointer tells OpenGL to use null-terminated text. The call copies source into the shader object; it is not a draw.

`glCompileShader(s)` asks the driver to compile it. `int ok; char log[512];` declares space for the result and a 512-byte diagnostic buffer. Although `ok` has no initial value, the next

`glGetShaderiv(s, GL_COMPILE_STATUS, &ok)` is intended to write it before testing. `if(!ok)` means "if compilation did not report success." Only that branch retrieves a compiler log, emits `[Shader Error]:` plus the message to `std::cerr`, and sets the shared flag false.

`std::endl` adds a newline and flushes the output stream, which helps immediate visibility of an error but is not necessary for every normal print. The log size is limited; lengthy driver messages can be truncated. There is no special handling here for a zero handle or a failed API call outside the compile-status check.

The function returns `s` even if compilation failed. It does not throw, abort, or delete the failed shader. The caller can proceed to linking and observe another failure. The closing conditional brace and function brace finish those scopes. This design favors gathering diagnostic state and letting later application startup decide what to do with the failure flag.

52. Original Lines 830-847: `buildProgram`, Linking, and Shader-Object Cleanup

`buildProgram(const char* vs, const char* fs)` combines vertex and fragment source. It calls `compileShaderModule` twice, with stage constants identifying each source. The two resulting handles, `v` and `f`, are attached to a newly created program `p` through `glAttachShader`. A shader object represents one compiled stage; a program combines compatible stages into something `glUseProgram` can activate for drawing.

`glLinkProgram(p)` checks and connects the stages, including their varying interfaces. Linking is a different validation step from compilation: two individually valid shader modules can still disagree on interfaces or exceed implementation limits. The local `ok` and 512-byte `log` play the same roles as in the module compiler.

`glGetProgramiv(p, GL_LINK_STATUS, &ok)` obtains link success; the failure branch fetches a program log, prints `[Program Error]:`, and sets `g_shaderOkay=false`.

The code links even if one earlier compile failed. This can provide both stage and link diagnostics but does not make the resulting program usable. Like module compilation, this helper returns a handle regardless of success. Caller logic must inspect the shared flag or program status before relying on rendering.

`glDeleteShader(v)` and `glDeleteShader(f)` request deletion of the temporary shader objects. Because the objects are attached, OpenGL may retain them until attachment references are removed; deleting a shader handle does not destroy the linked program's executable. The helper does not explicitly detach the shaders, delete a failed program, or activate `p`. `return p` gives the caller the program handle, and the final brace finishes construction. Program deletion at application shutdown is outside this range. Uniform locations queried later belong to this linked program and must not be assumed valid across relinking or replacement.

53. Original Lines 849-858: Four Program Handles and Shadow-Map Resolution

The four unsigned globals are `g_progScene`, `g_progShadow`, `g_progParticle`, and `g_progSky`. They store handles for the four shader-program roles just explained. Although their declarations do not explicitly write `=0`, file-scope objects with static storage duration are zero-initialized before ordinary runtime initialization. They still need assignments from successful program construction before use.

The shadow-framebuffer comment introduces `SHADOW_RES=2048`. Both texture dimensions later use it, giving $2048 \times 2048 = 4,194,304$ depth texels. This is a resolution, not a world-space coverage size or guaranteed precision per texel. The light's projection, defined elsewhere, determines how many world units each texel covers. A 5-by-5 PCF filter therefore has no fixed world-space softness from this constant alone.

`g_shadowFBO=0` and `g_shadowTex=0` are the future framebuffer and texture handles. A framebuffer selects image attachments as rendering destinations; a texture stores the sampled image itself. The framebuffer does not substitute for or contain a duplicated texture allocation. Keeping separate handles lets the same texture be written as an attachment during the shadow pass and sampled during a later visible pass.

If storage were 32 bits per texel, this depth image would occupy about 16 MiB before implementation overhead. The actual allocation below uses an unsized internal format, so that memory size and 32-bit depth precision are **not guaranteed** by these declarations. There are no cascaded maps, per-object maps, mip chains, or multiple depth layers declared here.

54. Original Lines 859-870: setupShadowFramebuffer and Depth-Texture Parameters

The setup function generates one framebuffer and one texture, then binds the texture to `GL_TEXTURE_2D`.

`glTexImage2D` allocates level zero using target 2D, unsized internal format `GL_DEPTH_COMPONENT`, width and height 2048, border-width argument zero, external format `GL_DEPTH_COMPONENT`, external data type `GL_FLOAT`, and null pixel data. The null pointer means there is no initial uploaded image; depth contents must be cleared or rendered before reliable sampling.

`GL_FLOAT` describes the input transfer type, not a promise that the GPU stores `GL_DEPTH_COMPONENT32F`. Because no initial pixel data is supplied and the internal format is unsized, the driver chooses a supported depth storage representation. The implementation should not be documented as fixed 16-bit, 24-bit, 32-bit, or RGB-encoded depth based only on this call.

Both minimum and magnification filters are `GL_NEAREST`. Minimum filtering without a mipmap mode permits using only level zero; no mipmap generation appears. Nearest filtering matters because the scene shader explicitly reads one depth value and compares it. Automatic shadow comparison mode is not enabled, and the GLSL sampler is an ordinary `sampler2D`.

S and T wrapping are `GL_CLAMP_TO_BORDER`, not clamp-to-edge. The four-value border array is all ones and is supplied with `glTexParameterfv`. For depth sampling, the relevant border depth is one, representing the far end of normalized depth. This makes out-of-image PCF taps ordinarily behave as unoccluded. The texture-border-width argument zero in the allocation is unrelated to this sampling border color; setting a virtual sampling border does not allocate an extra ring of pixels.

At this point the image and parameters exist, but the texture has not yet been attached to the framebuffer. No depth clear, viewport change, depth-test enable, or render occurs during these allocation lines.

55. Original Lines 871-880: Depth Attachment, No Color Buffers, and Completeness Check

`glBindFramebuffer(GL_FRAMEBUFFER, g_shadowFBO)` selects the generated framebuffer for both drawing and reading through the combined target. `glFramebufferTexture2D` attaches texture level zero to `GL_DEPTH_ATTACHMENT`; the final zero is the mip level, not a texture handle. Rendering depth while this framebuffer is active writes into the same image later sampled as `shadowMap`.

`glDrawBuffer(GL_NONE)` and `glReadBuffer(GL_NONE)` state that this framebuffer has no color draw/read target. This matches its depth-only attachment and the empty shadow fragment shader. These calls do not disable depth writes. They configure color-buffer selection for the framebuffer rather than make all rendering disappear.

`glCheckFramebufferStatus(GL_FRAMEBUFFER)` is compared with `GL_FRAMEBUFFER_COMPLETE`. If another status is returned, the branch prints `Shadow framebuffer is incomplete` followed by a newline and sets `g_shaderOkay=false`. Reusing a shader-named flag for framebuffer failure is a broad graphics-initialization success convention. The code does not print the exact failing status, clean up the partial objects, or stop inside this function.

The conditional closes, then `glBindFramebuffer(GL_FRAMEBUFFER, 0)` returns to the default framebuffer. It does not restore an arbitrary framebuffer that may have been bound before this function. Likewise, the shadow texture remains bound on the active texture unit. The function assumes a startup-oriented context where these state changes are acceptable. Its final brace ends setup; later shadow rendering must choose a 2048-square viewport, clear depth, bind the appropriate program, and supply light matrices. None of those operations is hidden in the completeness check.

56. Original Lines 882-899: setupSkyVAO and the Oversized Fullscreen Triangle

The comment says "Skybox fullscreen quad VAO," but the actual array defines **three vertices**, not four. `g_skyVAO` and `g_skyVBO` start at zero. `setupSkyVAO()` initializes a nine-float array with positions `(-1, -1, 0)`, `(3, -1, 0)`, and `(-1, 3, 0)`.

In clip-space XY, this triangle extends beyond the visible square `[-1, 1]` in both directions. Its diagonal edge is `x+y=2`, which passes through the visible square's upper-right corner. It therefore covers the full screen with one oversized triangle, avoiding a two-triangle quad's interior diagonal. Clipping trims its outside portions; input vertices at coordinate 3 are intentional, not evidence of an invalid screen mesh.

The function generates a VAO and VBO, binds them, and uploads the entire array with `sizeof(verts)` bytes and `GL_STATIC_DRAW`. Location zero receives three float components with stride `3*sizeof(float)` and offset zero, then is enabled. There is no EBO because the intended primitive can be drawn using nonindexed vertex order. The draw call itself is outside this setup function.

`glBindVertexArray(0)` leaves the configured sky VAO stored but inactive. The VBO's array-buffer binding is not explicitly restored. The shader only uses `aPos.xy`; the supplied zero Z values keep a simple three-float attribute layout consistent with its `vec3` input. Sky depth is supplied explicitly by `SKY_VERT` as 0.9999 rather than using these zeros. The initializer brace, function brace, and semicolons finish data and setup scopes. No cubemap faces, sky textures, or actual blue-gold pixels are generated by this CPU helper; the fragment shader supplies them at draw time.

Draw Helpers and Materials

57. Original Lines 901-912: Pass Selection and Material Defaults

The rendering-pipeline comment introduces `g_isShadowPass=false`. This global tells the generic primitive draw helper which set of uniforms to update. It does not itself bind a framebuffer or activate a shader. Later pass orchestration must keep the Boolean synchronized with actual OpenGL state.

`Material` is a record with in-class defaults. `diffuse=vec3(0.6)` sets RGB to `(0.6, 0.6, 0.6)`, a medium gray base. `specular=vec3(0.3)` sets all highlight channels to 0.3. `shininess=32` selects a moderately concentrated Blinn-Phong exponent. `emissive=vec3(0)` adds no self-lit contribution by default. `isSand=0` chooses ordinary material behavior, leaving both specialized sand and rock branches disabled.

These defaults are applied when a material is default-constructed, unlike the earlier tank structure with no in-class defaults. Models can then change only selected members. The `diffuse` member is uploaded to the shader's `objectColor`, `specular` to `specularColor`, and `emissive` to `emissiveColor`; different names in the C++ record and shader do not imply different values.

The structure contains no alpha, image texture handle, roughness, metalness, refractive index, or shadow-casting flag. Its integer mode can select more than just sand, and its emission does not illuminate neighboring geometry. The final brace and semicolon complete the type. A material is CPU parameter data, not an OpenGL program or buffer object, so creating one does not allocate GPU resources. Applying it requires the uniform updates in the next helper.

58. Original Lines 914-922: drawMeshPrimitive and Cached Uniform Locations

`drawMeshPrimitive(const Mesh& mesh, const glm::mat4& model, const Material& mat)` receives mesh handles, transformation, and material by constant reference. It does not copy full records or change its arguments. Its first eight statements query shader uniform locations and store them in local static `const GLint` variables. A uniform location is a program-specific integer index used for later updates, not the uniform's current numerical value.

The two shadow queries obtain `model` and `isSand` from `g_progShadow`. The six scene queries obtain `model`, `objectColor`, `specularColor`, `shininess`, `emissiveColor`, and `isSand` from `g_progScene`. All eight are initialized when execution first reaches their declarations, even if the first draw is only a shadow pass. Both programs must therefore have been successfully created before the first call.

Local static caching avoids repeated string-based queries on every object draw. However, `const` makes these locations permanent for the process's remaining calls. Rebuilding, relinking, or replacing the programs can invalidate the cached locations; the helper offers no refresh mechanism. Calling it before valid programs exist can likewise cache unsuitable values and will not automatically recover after later initialization.

`glGetUniformLocation` returns -1 for an inactive or absent uniform. Passing -1 to a uniform setter is ordinarily ignored, so a miss can be silent. The helper does not validate any of these results. It also does not cache view, projection, sun, fog, worm, or collapse locations because those are per-pass or shared values assigned by other code. These lines only prepare access to per-primitive uniforms; no drawing has occurred yet.

59. Original Lines 923-936: Pass-Specific Uniform Uploads and Indexed Drawing

When `g_isShadowPass` is true, the helper uploads one model matrix and the material's integer mode to the shadow uniform locations. `glUniformMatrix4fv(..., 1, GL_FALSE, glm::value_ptr(model))` means one four-by-four float matrix, no transposition requested, using GLM's data pointer. `glUniform1i` sends one integer. Colors and highlight parameters are unnecessary for the depth-only shader.

The `else` branch uploads that model matrix to the scene program, then sends diffuse RGB, specular RGB, shininess, emissive RGB, and mode. Each `glUniform3fv` has count one, meaning one three-component vector, not three separate vector records. `glUniform1f` sends a single float. The material's default shininess is 32, but this helper accepts whatever the caller supplies and does not clamp invalid exponents or colors.

These `glUniform` calls update the **currently active program**. They do not automatically activate the program used for querying a location. The helper contains no `glUseProgram`, so the caller must already have selected the correct program for the current pass. Likewise, it assumes appropriate framebuffer, viewport, depth state, culling state, and shared uniform values have been set externally. A Boolean/program mismatch can cause errors or incorrect rendering.

After the conditional, `glBindVertexArray(mesh.VAO)` activates the stored vertex/index configuration. `glDrawElements(mesh.drawMode, mesh.indexCount, GL_UNSIGNED_INT, 0)` draws the requested mode using `indexCount` unsigned indices starting at byte offset zero in the VAO's element buffer. The last zero is an offset, not an immediate CPU index array, because an EBO is bound through the VAO.

There is no automatic material batching, instancing, visibility culling, or mesh validity check here. Each call submits one indexed primitive draw. The helper leaves that VAO bound afterward and returns no value. Its final brace ends the function; it does not unbind or delete the mesh.

60. Original Lines 938-947: drawPart and Exact Transformation Order

The helper comment introduces a convenient way to place a primitive. `drawPart` takes a constant mesh reference, a root matrix by value, translation vector `t`, rotation vector `rDeg`, scale vector `s`, and a constant material reference. Copying `root` means the caller's matrix is not changed. `m=root` creates the local working transformation.

`glm::translate(m, t)` postmultiplies the translation under GLM's usual column-vector convention. Three conditionals then apply rotations only when their respective degree values are nonzero. Y rotation is appended first, then X, then Z. Each angle is converted from degrees through `glm::radians`, and axis vectors are $(0, 1, 0)$, $(1, 0, 0)$, and $(0, 0, 1)$. Skipping zero rotations avoids unnecessary matrix operations but does not introduce tolerance-based snapping for tiny angles.

The final scale is appended with `glm::scale(m, s)`. The combined matrix is `root * T * Ry * Rx * Rz * S`. Since matrices act on column vectors from right to left, a local mesh vertex is scaled first, then rotated around Z, then X, then Y, then translated, then transformed by the root. The **written call order** and the **order experienced by a vertex** are therefore opposite. Rotation order matters because three-dimensional rotations generally do not commute.

Translation here is expressed in the root's coordinate system. A root rotation can rotate the translated offset, making the helper suitable for assembling attached parts. Negative scales can reverse winding; zero scales can make the scene shader's inverse normal matrix undefined. The helper does not prevent those cases or automatically adjust culling.

`drawMeshPrimitive(mesh, m, mat)` then sends the assembled matrix and material to the active pass and draws. There is no separate object stored for this part: it is an immediate transformed reuse of a primitive mesh. The final brace ends the placement-and-draw helper.

Shared Color Palette

61. Original Lines 949-967: Every Palette Constant and Its Actual Meaning

Lines 949-951 introduce the cinematic palette. Each declaration defines a constant three-component RGB vector. The values are color inputs, not final screen pixels: lighting, shadowing, tone mapping, fog, and render-target behavior can change their visible appearance. All fifteen declarations are immutable through ordinary assignment. Their exact values and intended roles are:

- Line 952, COL_DUNE_SAND=(0.76, 0.49, 0.23): warm ochre-brown sand, with red strongest and blue weakest.
- Line 953, COL_DUNE_SHADOW=(0.55, 0.36, 0.18): a darker warm companion color. Its name does not perform shadow testing; only actual uses elsewhere decide where it appears.
- Line 954, COL_ATREIDES_HULL=(0.22, 0.23, 0.22): dark nearly neutral hull gray with a slight green emphasis. The "matte carbon" comment describes artistic intent; shininess and specular settings determine actual highlight behavior.
- Line 955, COL_ATREIDES_DARK=(0.13, 0.14, 0.13): a darker, similarly green-tinted gray for contrasting aircraft parts.
- Line 956, COL_CANOPY_GLASS=(0.06, 0.08, 0.11): very dark blue-gray canopy color. The "armored glass" comment does not enable transparency, refraction, or Fresnel effects.
- Line 957, COL_JET_NOZZLE=(0.08, 0.08, 0.08): equal-channel dark gray for jet nozzle geometry.
- Line 958, COL_AFTERBURNER=(1.00, 0.55, 0.12): bright orange-yellow reheat coloration. Brightness or glow requires appropriate emissive/material use; this vector alone does not create light or bloom.
- Line 959, COL_SPICE_ORANGE=(0.96, 0.42, 0.04): strongly saturated spice orange, with very little blue.
- Line 960, COL_SPICE_GLOW=(1.00, 0.50, 0.08): a somewhat brighter orange intended for spice-related glow elements. The name is not an emissive assignment by itself.
- Line 961, COL_CRAWLER_TRACK=(0.11, 0.11, 0.10): near-neutral very dark tread color, slightly warmer than exact gray.
- Line 962, COL_HARV_CHASSIS=(0.42, 0.34, 0.25): muted warm brown for industrial chassis geometry.
- Line 963, COL_HARV_RUST=(0.48, 0.24, 0.12): red-brown rust accent, with each successive channel roughly half the preceding one.
- Line 964, COL_TANK_STEEL=(0.52, 0.46, 0.38): warm light-gray/brown steel coloration. There is no physically metallic material classification in this palette.
- Line 965, COL_ROCK_STRATA1=(0.42, 0.28, 0.17): a warmer, lighter rock-stratum color.
- Line 966, COL_ROCK_STRATA2=(0.32, 0.20, 0.11): a darker companion rock-stratum color.

The semicolon on each line completes its declaration. Line 967 is blank, ending this chapter's original-source interval before the aircraft assembly begins. No palette declaration uploads a uniform until later drawing selects a color and applies a material. Constants provide consistent art direction without claiming measured spectral reflectance or standard color-space calibration.

Implementation Boundaries

The source-order walkthrough covers every nonblank original line from 1 through 967. The following points connect those details into the renderer's actual architecture without adding claims not established by the code:

- The CPU terrain function is continuous over X/Z, but the GPU terrain is one finite 700-unit patch sampled at five-unit intervals by default. Mathematical continuity does not mean infinite geometry, camera-centered regeneration, or an exact per-pixel height surface.
- Terrain normals are central finite differences, and rasterization smooths/interpolates them. Collapse deforms the two vertex passes consistently but does not update normals, collision heights, landing-pad queries, or particle ground queries.
- The CPU dust pool has 2,400 records with deterministic pseudorandom initialization and recycled lifetimes. Branch priority selects crawler, worm, downwash, or ordinary dust; comments should not be substituted for their exact modulo and time conditions.
- Particle motion is velocity-based visual integration with gust scaling and a terrain lift rule. There is no sand-grain mass, gravity integration, mutual collision, fluid solver, or conservation model in these functions.

- Procedural primitives are reusable GPU meshes. Hard edges use duplicated normals; sphere poles submit collapsed faces; wings have open ends and simplified normals; the rock perturbation does not guarantee a welded longitudinal seam.
- Surface lighting is ambient plus directional wrapped/Lambert diffuse plus Blinn-Phong specular plus emission. It is not a physically based renderer, and opaque alpha is used even for glass-colored surfaces.
- Shadows use a single 2048-square depth image, nearest depth fetches, receiver-slope adjustment, bias, and 25 manual comparisons per shadowed surface fragment. Unsized depth allocation leaves actual depth precision to the driver; fixed PCF is not a physically variable penumbra.
- The sky is a ray-shaded fullscreen triangle with an elevation gradient and analytic sun terms, not a cubemap, cloud simulation, or scattering solver. It does not share the surface shader's final tone-mapping expression.
- OpenGL handles and cached uniform locations are lightweight shared state. Correct operation depends on successful context/program setup, matching active pass state, valid transformations, and explicit lifecycle work outside this chapter.

These limitations do not make the implementation unsuitable for a real-time graphics laboratory. They identify the deliberate approximations and concrete edge cases a reader must understand before extending or diagnosing it. The key learning sequence is: compute simulation and geometry on the CPU, upload reusable data, transform vertices in shaders, shade fragments with explicit mathematical terms, then orchestrate the passes with the correct OpenGL state.

Scene Assembly and Application Runtime

This chapter explains the scene-building and application-lifecycle portion of `src/arrakis.cpp`. Its source references use the **original, pre-relocation line numbers 968-1892** from the root-level `arrakis.cpp` inspected for this documentation. The relocated `src/arrakis.cpp` was also checked: the covered range still has the same line numbers and statements. The original ranges below identify the inspected statements, comments, and function boundaries. The complete source listings supplied elsewhere in the documentation remain the authority for the final physical file layout.

The covered code turns previously generated meshes into recognizable vehicles, machinery, landscape objects, and rescue markers, then connects them to GLFW input, simulation updates, spatial audio, OpenGL rendering, screenshots, automated render checks, and cleanup. It does not load vehicle models from external model files. The geometric pieces are reused meshes with different transformations and materials. The supporting application headers live alongside the application source in `src/`; mission tests are documented under `tests/arrakis_tests.h`. Third-party implementation files and audio assets belong under `third_party/` and `assets/audio/`, respectively. Those layout names do not imply additional runtime behavior in the original code explained here.

Reading Transforms and Materials

Shared Conventions Used Throughout This Chapter

Context for original lines 968-1354: the actual transform helper and default material definitions are immediately before this chapter's assigned range, at original lines 906-947. Understanding them is necessary to read every scene draw call accurately.

`drawPart(mesh, root, t, rDeg, s, material)` starts with the supplied parent matrix `root`, appends translation `t`, appends any nonzero Y rotation, then X rotation, then Z rotation, and finally appends scale `s`. In matrix notation the result is $root * T * R_y * R_x * R_z * S$. The rotation tuple is still written as (X, Y, Z) in the calls; it is not a description of the multiplication order. Degrees are converted to radians by the helper before calling GLM. With column-vector positions, the scale is applied to a mesh point first, then the local Z, X, and Y rotations, then the local translation, then all inherited parent transformations. A translated part therefore rotates about its own local origin before being placed at its attachment location. When a parent is tilted or scaled, the child's apparent world-space position and orientation inherit that parent change.

The notation in the tables below is literal: **T** is the translation tuple, **R** is the (X, Y, Z) degree tuple, and **S** is the scale tuple. All coordinates are in the parent's local space unless a section explicitly constructs a world-space root. Vehicle construction uses X for left/right, Y for up/down, and Z for front/back; the ornithopter's nose is toward negative Z. Numbers are scene units, not a declared real-world measurement system.

The generated cube is centered at the origin and has unit side length. The sphere has radius 0.5, so its scale tuple gives its full axis diameters before other transforms. The generated cylinder has radius 0.5, height 1, and its long axis along local Y. A cylinder with $S(2.2, 3.2, 2.2)$ therefore has diameter 2.2 and length 3.2; rotating it 90 degrees around X places that length along Z. Nonuniform X/Z scaling changes a round cylinder into an elliptical one. A mesh called a cylinder remains the same capped-cylinder geometry even when a source comment calls the result a probe, funnel, nozzle, or drum. Likewise, an entire sphere is drawn for each nominal domed tank head; overlapping spheres and a cylinder produce the visible rounded ends rather than explicit hemisphere meshes.

Every `Material` initially has diffuse $(0.6, 0.6, 0.6)$, specular $(0.3, 0.3, 0.3)$, shininess 32, emissive $(0, 0, 0)$, and `isSand = 0`. A construction function only overrides the fields shown in its body. In material tables, `default` means exactly these unchanged values, not a missing value or a random shader setting. A scalar `glm::vec3(v)` repeats `v` in all three color channels. Diffuse controls the surface color, specular and shininess control the highlight contribution, emissive adds self-colored brightness in the scene shader, and `isSand` selects special shader treatment. Emission does not create an additional OpenGL light source and does not, by itself, illuminate neighboring geometry.

`drawPart` ultimately calls `drawMeshPrimitive`. During the shadow pass, that helper sends the model matrix and material surface classification to the shadow program. During the normal scene pass, it also sends the color, specular, shininess, and emission values to the scene program. The same assembled object can therefore participate in both passes without duplicating its construction logic.

Ornithopter Construction

Function Inputs and Six Surface Materials

Original source: lines 968-1001. The section banner describes an *Atreides*-style ornithopter with eight tandem dragonfly blades, an insect-like hull, and glowing jets. These are design descriptions; the implementation that follows is a collection of scaled primitive meshes and a dedicated wing mesh.

`drawMovieOrnithopter(root, wingPhase, roll, pitch, boosting)` receives the assembled aircraft-to-world transform, the animation phase for the wings, roll and pitch values, and the boost state. In this function, roll and pitch are not read. Aircraft attitude has already been applied to `root` by `drawFullScene`; the redundant arguments do not add a second banking or pitching transform. `wingPhase` drives the flap and twist formulas. `boosting` chooses between two exhaust lengths and emission values.

The six local material objects are rebuilt on each invocation. They do not represent six texture files or persistent GPU objects. Each supplies the fields used by the primitive helper for that draw.

Material	Diffuse RGB	Specular RGB	Shininess	Emissive RGB	Role
<code>matHull</code>	(0.22, 0.23, 0.22)	(0.4, 0.4, 0.4)	64	default zero	Main dark gray body and selected structural parts.
<code>matDark</code>	(0.13, 0.14, 0.13)	(0.3, 0.3, 0.3)	32	default zero	Darker probe, canopy frame, gimbals, tail pieces, and skids.
<code>matCanopy</code>	(0.06, 0.08, 0.11)	(0.9, 0.9, 0.9)	128	default zero	Very dark blue-gray cockpit surface with a sharp highlight.
<code>matFlame</code>	(1, 0.55, 0.12)	default (0.3, 0.3, 0.3)	default 32	Boosting: (1.2, 0.6, 0.15); otherwise (0.6, 0.25, 0.05)	Orange exhaust geometry with stronger self-brightness under boost.
<code>matJetNozzle</code>	(0.08, 0.08, 0.08)	(0.5, 0.5, 0.5)	64	default zero	Dark terminal engine nozzles.
<code>matBlade</code>	(0.18, 0.19, 0.18)	(0.7, 0.7, 0.7)	90	default zero	Thin dark metallic-looking wing blades.

All six keep `isSand = 0`. The canopy is not rendered with an alpha-bearing glass material here. Its name and high specular response give it a glass-like appearance, but the call supplies no transparency, refraction, or view through the canopy. The flame's emission can exceed one in some channels because the assigned float values are not clamped in this construction function.

Fuselage, Cockpit, and Dorsal Intakes

Original source: lines 1002-1018. These comments and seven draw calls build the central pod, front details, canopy, and paired intakes. Every call uses the incoming aircraft `root` directly, so none introduces a second articulated parent.

Original line	Mesh and visual component	T	R	S	Material
1004	Sphere: central elongated body pod	(0, 0, -0.4)	(0, 0, 0)	(1.3, 0.85, 3.2)	matHull
1006	Cylinder: forward nose segment	(0, -0.05, -2.1)	(85, 0, 0)	(0.7, 1.4, 0.45)	matHull
1008	Cylinder: narrow forward sensor probe	(0, -0.1, -3.1)	(90, 0, 0)	(0.06, 1.2, 0.06)	matDark
1011	Sphere: tinted cockpit canopy	(0, 0.32, -1.2)	(-10, 0, 0)	(0.85, 0.65, 1.6)	matCanopy
1013	Cube: slim canopy spine	(0, 0.58, -1.1)	(-10, 0, 0)	(0.12, 0.18, 1.5)	matDark
1016	Cylinder: positive-X dorsal intake	(0.42, 0.35, -0.2)	(82, 0, 0)	(0.35, 1.2, 0.28)	matHull
1017	Cylinder: negative-X dorsal intake	(-0.42, 0.35, -0.2)	(82, 0, 0)	(0.35, 1.2, 0.28)	matHull

The central sphere's nonuniform dimensions turn it into a long, low ellipsoid. The nose cylinder is nearly horizontal, with a slight departure from the probe's exact 90-degree X rotation. Its name says tapered, but this particular draw does not change the cylinder's radius along its length. The probe is long compared with its 0.06 diameter, producing the needle-like detail at the front. The canopy lies above the forward body, and the thin cube spine is higher still, making a dark structural line over the shiny surface.

The comment about multiple canopy facets does not correspond to multiple separate canopy calls in this range: one tessellated sphere is drawn. Its generated triangles and its ellipsoidal scale define its actual surface. The two intake cylinders differ only in the sign of their X positions, placing them symmetrically on the upper body. They are solid capped meshes; no separate open interior or airflow simulation is added by these calls.

Wing Gimbals, Turbines, Nozzles, and Boost Exhaust

Original source: lines 1019-1035. Four shoulder cylinders suggest front and aft wing mounting hardware. Two larger cylinders create engine bodies, two shorter cylinders create the nozzles, and two final cylinders create visible exhaust. Their exact transforms are below.

Original line	Mesh and component	T	R	S	Material
1020	Cylinder: front positive-X wing gimbal	(0.72, 0.1, -0.3)	(0, 0, 90)	(0.45, 0.65, 0.45)	matDark
1021	Cylinder: front negative-X wing gimbal	(-0.72, 0.1, -0.3)	(0, 0, 90)	(0.45, 0.65, 0.45)	matDark
1022	Cylinder: aft positive-X wing gimbal	(0.68, 0.05, 0.5)	(0, 0, 90)	(0.42, 0.62, 0.42)	matDark
1023	Cylinder: aft negative-X wing gimbal	(-0.68, 0.05, 0.5)	(0, 0, 90)	(0.42, 0.62, 0.42)	matDark
1026	Cylinder: positive-X turbine body	(0.52, -0.15, 0.9)	(90, 0, 0)	(0.42, 1.6, 0.42)	matHull

Original line	Mesh and component	T	R	S	Material
1027	Cylinder: negative-X turbine body	(-0.52, -0.15, 0.9)	(90, 0, 0)	(0.42, 1.6, 0.42)	matHull
1029	Cylinder: positive-X exhaust nozzle	(0.52, -0.15, 1.72)	(90, 0, 0)	(0.36, 0.25, 0.36)	matJetNozzle
1030	Cylinder: negative-X exhaust nozzle	(-0.52, -0.15, 1.72)	(90, 0, 0)	(0.36, 0.25, 0.36)	matJetNozzle
1033	Cylinder: positive-X luminous exhaust	(0.52, -0.15, 1.8 + flameLen * 0.5)	(90, 0, 0)	(0.24, flameLen, 0.24)	matFlame
1034	Cylinder: negative-X luminous exhaust	(-0.52, -0.15, 1.8 + flameLen * 0.5)	(90, 0, 0)	(0.24, flameLen, 0.24)	matFlame

A 90-degree Z rotation puts each gimbal's long cylinder axis across the aircraft rather than upright. The front pair is slightly larger and higher than the aft pair. The visible mounting cylinders do not individually rotate with wing strokes; the wing matrices constructed later are separate transforms sharing the aircraft parent.

Both turbine bodies run along Z beneath the fuselage. Their nozzle centers are farther toward positive Z, the rear of the aircraft. Line 1032 sets `flameLen` to 1.4 under boost and 0.6 otherwise. Consequently the exhaust centers are at Z 2.5 or 2.1, while the scaled cylinders extend between Z 1.8 and Z 3.2 or 2.4. Moving the center by half the chosen length keeps the forward end at the same location. The glowing shapes exist even when the aircraft is not boosting. They become longer and use the stronger emission values when the boolean is true; no branch removes them, animates their noise, or makes a tapered flame mesh.

Tail Boom, Stabilizers, and Landing Skids

Original source: lines 1036-1048. The remaining fixed aircraft pieces extend behind the engines and below the body.

Original line	Mesh and component	T	R	S	Material
1037	Cylinder: main tail boom	(0, 0, 2.4)	(90, 0, 0)	(0.32, 2.6, 0.28)	matHull
1038	Cylinder: narrower aft boom	(0, 0.05, 4.4)	(90, 0, 0)	(0.18, 2.2, 0.16)	matDark
1040	Cylinder: terminal stinger	(0, 0.05, 5.7)	(90, 0, 0)	(0.08, 1.1, 0.08)	matDark
1043	Cube: positive-X canted tail fin	(0.5, 0.55, 4.2)	(0, 0, 38)	(0.08, 0.9, 0.65)	matHull
1044	Cube: negative-X canted tail fin	(-0.5, 0.55, 4.2)	(0, 0, -38)	(0.08, 0.9, 0.65)	matHull
1046	Cube: positive-X ventral skid	(0.45, -0.6, -0.4)	(0, 0, 0)	(0.08, 0.45, 2.2)	matDark
1047	Cube: negative-X ventral skid	(-0.45, -0.6, -0.4)	(0, 0, 0)	(0.08, 0.45, 2.2)	matDark

The tail is assembled from progressively narrower, overlapping cylinders instead of one continuously tapered extrusion. The final stinger is a cylinder too. The two fins are thin cubes tilted in opposite Z directions to make a V-like arrangement. They have no separate steering animation in this function. The skids are also fixed cubes, long in Z and

thin in X, positioned below the pod. No extension or retraction branch is present. In total, the fixed portion of this aircraft function issues 24 primitive draws before the wings.

Flap Waveforms and Eight Wing Definitions

Original source: lines 1049-1081. The wing section begins with its own descriptive banner and defines a 34-degree flap amplitude. $\text{flap1} = \sin(\text{wingPhase}) * 34$ and $\text{flap2} = \sin(\text{wingPhase} + \pi * 0.75) * 34$. A phase difference of $0.75 * \pi$ is 135 degrees, not an exact 180-degree opposition. The comment's term anti-phase is therefore an informal description of staggered strokes. The aft definitions introduce an additional sign reversal of selected flap values.

The local `WingDef` structure stores five pieces of information: `side` is +1 for the right/positive-X blade and -1 for the left/negative-X blade; `pivot` is the attachment point; `flapAng` is the current waveform sample; `sweepBack` is a fixed backward-sweep angle in degrees; and `dihedral` is the resting vertical angle, also in degrees. The eight-element array is recreated from the current phase at each draw.

Pair and side	side	Pivot	flapAng	sweepBack	dihedral
Forward upper, right	1	(0.95, 0.22, -0.35)	flap1	14	12
Forward upper, left	-1	(-0.95, 0.22, -0.35)	flap1	14	12
Forward lower, right	1	(0.95, -0.05, -0.25)	flap2	18	-10
Forward lower, left	-1	(-0.95, -0.05, -0.25)	flap2	18	-10
Aft upper, right	1	(0.90, 0.18, 0.50)	-flap2	28	8
Aft upper, left	-1	(-0.90, 0.18, 0.50)	-flap2	28	8
Aft lower, right	1	(0.90, -0.08, 0.60)	-flap1	32	-12
Aft lower, left	-1	(-0.90, -0.08, 0.60)	-flap1	32	-12

The upper and lower attachment points are separated vertically and slightly longitudinally. The aft pivots are closer to the centerline and farther back. Greater sweep values for the aft blades keep their resting directions distinct from the forward blades. A mirrored pair shares its scalar phase and dihedral settings; the side multiplier in the matrix logic is what turns the two sides into corresponding opposite-space movements.

Wing Matrix Order, Mirroring, and Culling

Original source: lines 1082-1101. The range-based loop visits all eight array entries. It copies the aircraft matrix to `wm`, translates to the current wing's pivot, and appends a Z rotation of `side * (dihedral + flapAng)` degrees. That is the flap motion about the aircraft's longitudinal axis. It then appends a Y rotation of `side * sweepBack` degrees. Finally it computes $\text{pitchTwist} = \cos(\text{wingPhase}) * 8 * \text{side}$ and appends that X rotation.

All blades use the same cosine phase for twist; the twist is not computed from the wing's own `flapAng`, and the aft blades do not receive an additional twist-phase shift. The twist amplitude is eight degrees. The sign flips left versus right. The resulting wing matrix is $\text{root} * T(\text{pivot}) * R_z(\text{flap and dihedral}) * R_y(\text{sweep}) * R_x(\text{twist}) * S(\text{side} * 5.2, 1, 1)$.

The final scale stretches the generated wing's original X span from one to 5.2 units. On the left, negative X scale both extends the blade to the opposite side and reverses the winding of its triangles. Instead of selecting a different front-face convention for those blades, the code disables face culling before drawing each wing, draws `g_meshWing` using

matBlade, and then enables face culling again. Both sides of the thin blades are therefore drawable. This is an explicit state setting, not a saved-state restoration: the loop always leaves culling enabled. It does not change the configured `glCullFace(GL_BACK)` choice.

The wing draw calls use `drawMeshPrimitive` directly because their composite transform is already built. Together with the 24 fixed draws, the aircraft has 32 mesh draws per invocation. Since the full scene is invoked for both depth and color rendering, a visible aircraft is assembled and submitted in each of those passes with the same frame phase. The closing braces at lines 1100-1101 end the wing loop and aircraft function; they introduce no additional update of animation state.

Harvester Construction

Materials, Main Hopper, and Refinery Deck

Original source: lines 1103-1119. The harvester banner identifies four crawler units, a rotary scoop, and a multi-level industrial body. `drawSpiceHarvester(root, crawlerAnim)` accepts the already positioned and slope-adjusted vehicle matrix plus an animation amount. The caller supplies accumulated harvester travel, not wall-clock time. The visible rotation formulas multiply this distance-like value by their own constants.

Material	Diffuse RGB	Specular RGB	Shininess	Emissive RGB
matChassis	(0.42, 0.34, 0.25)	(0.2, 0.2, 0.2)	20	default zero
matTread	(0.11, 0.11, 0.10)	(0.15, 0.15, 0.15)	10	default zero
matRust	(0.48, 0.24, 0.12)	(0.2, 0.2, 0.2)	16	default zero
matSpice	(0.96, 0.42, 0.04)	default (0.3, 0.3, 0.3)	default 32	(0.8, 0.35, 0.05)
matGlass	(0.1, 0.2, 0.25)	(0.8, 0.8, 0.8)	90	default zero
matLight	(1, 0.95, 0.7)	default (0.3, 0.3, 0.3)	default 32	(1.2, 1.1, 0.8)

All retain `isSand = 0`. The chassis is warm brown, the tread almost black, and the rust reddish brown. Bright orange spice details and pale floodlights use emission rather than additional point-light uniforms. The bridge glass, like the aircraft canopy, is shiny colored opaque geometry in this construction path.

Line 1116 draws the main body using a cube with T (0, 2.6, 0), R (0, 0, 0), S (9.5, 3.8, 6.2), and `matChassis`. The large width and length establish a broad rectangular hopper. Line 1118 draws another chassis cube at T (0, 4.8, -0.4), R zero, S (8.2, 1.2, 5.0) for the upper deck. It is narrower, shorter, elevated, and slightly forward of the main body's center. These meshes are closed cubes, not hollow storage compartments. Their names do not add an interior refinery simulation.

Four Track Units, Wheels, and Traveling Pads

Original source: lines 1120-1139. The track loops enumerate X offsets -4.6 and 4.6 and Z offsets -2.4 and 2.4. Every combination creates one track parent `tm = translate(root, (ox, 0.95, oz))`, yielding four units. Their left/right placement is inherited from `ox`, while their front/rear placement is inherited from `oz`.

Each unit first draws a tread-colored housing cube with local T zero, R zero, and S (1.8, 1.7, 3.8). It is centered at the track parent, making the same housing usable at all four corners. A wheel loop starts `wz` at -1.4, continues while `wz <= 1.4`, and adds 0.93 each iteration. It produces four wheel locations approximately -1.4, -0.47, 0.46, and 1.39; the next value exceeds the bound. The endpoint is not exactly 1.4 because the spacing and floating-point accumulation determine the actual sample locations.

At each wheel location line 1130 draws a chassis cylinder with $T(ox > 0 ? 0.95 : -0.95, -0.2, wz)$, $R(0, 0, 90)$, and $S(1.2, 0.35, 1.2)$. The sign test depends on the enclosing track's X location. It places each wheel on that track's outward-facing side. The Z rotation lays the 0.35 cylinder thickness along X, while the other scale values establish a 1.2 diameter. The full world X position includes the track parent offset as well as this local side offset.

Line 1131 adds a narrow rusty cube on the outer face: $T(ox > 0 ? 1.15 : -1.15, -0.2, wz)$, $R(crawlerAnim * 80, 0, 0)$, $S(0.04, 0.95, 0.12)$. This is the changing decorative wheel-face detail. The 80 coefficient supplies degrees per unit of `crawlerAnim` through `drawPart`; no conversion to radians is done at this line because the helper does it. Its explicit rotation axis is local X before the track parent transforms, aligned with the cylinder axle after that cylinder's 90-degree Z rotation. The wheel cylinder itself does not receive an animation angle; the bar's changing orientation supplies the visible spinning detail while the smooth wheel body remains unchanged.

Ten pad cubes are drawn per track. For integer pad values zero through nine, line 1134 computes $a = pad * two_pi / 10 + crawlerAnim * 0.7$. Here a is in radians and feeds sine and cosine directly. The pad center is $(0, 0.85 * \sin(a), 1.8 * \cos(a))$, an ellipse in the unit's Y/Z plane. The angle is $-\text{degrees}(\text{atan2}(0.85 * \cos(a), -1.8 * \sin(a)))$. The two `atan2` arguments are the Y and Z components of the derivative of that elliptical path. Negating the resulting angle sets the cube's local X rotation so its long Z direction follows the path tangent.

Line 1136 therefore draws each pad with that computed $T, R(\text{angle}, 0, 0), S(1.9, 0.12, 0.38)$, and `matRust`. Pads span the track's width but are thin in Y and short in Z. Their centers revolve around the housing as `crawlerAnim` grows. The code does not translate a continuous tread texture or construct a linked belt mesh; it moves ten separate cubes around each ellipse. One unit submits one housing, four cylinders, four bars, and ten pads, or 19 draws. The nested loops submit 76 track-related draws in total. The closing braces finish the pad loop, front/rear loop, and left/right loop without changing the harvester root.

Cutter, Exit Door, Ramp, and Side Separators

Original source: lines 1141-1153. This section constructs the low forward harvesting equipment and several side details. Although the cutter comment mentions teeth, no separate tooth draw calls occur in this range: the primary cutter is the reused cylinder.

Original line	Mesh and component	T	R	S	Material
1143	Cylinder: wide front cutter drum	$(0, 1.1, -4.2)$	$(crawlerAnim * 70, 0, 90)$	$(2.2, 8.8, 2.2)$	<code>matRust</code>
1145	Cube: dark side door	$(4.78, 1.8, 1.2)$	$(0, 0, 0)$	$(0.12, 2.4, 1.5)$	<code>matDoor</code>
1146	Cube: outward side ramp	$(5.5, 0.45, 1.2)$	$(0, 0, -16)$	$(2, 0.15, 1.7)$	<code>matChassis</code>
1148	Cube: luminous intake scoop	$(0, 0.9, -4.8)$	$(15, 0, 0)$	$(8.2, 0.6, 1.4)$	<code>matSpice</code>
1151	Cylinder: positive-X separator	$(4.8, 2.2, -1.8)$	$(0, 0, -22)$	$(1.2, 2.4, 1.2)$	<code>matRust</code>
1152	Cylinder: negative-X separator	$(-4.8, 2.2, -1.8)$	$(0, 0, 22)$	$(1.2, 2.4, 1.2)$	<code>matRust</code>

Line 1144 constructs `matDoor` with diffuse $(0.06, 0.07, 0.06)$ and leaves every other field at its default. The door is an almost black thin rectangle on the positive-X side; the ramp extends farther out and slopes through its negative Z rotation. Neither has an opening animation in this function. The scoop is a thin orange slab angled about X at the front and lower than the drum's center.

The drum call appends X rotation before Z rotation in the helper's matrix multiplication. At zero `crawlerAnim`, the 90-degree Z rotation turns its original Y axis across the vehicle along X; subsequent changes to the X angle rotate

around that resulting axis. The 70 coefficient is in degrees per supplied animation unit. The two separators are straight cylinders tilted oppositely around Z. Their comment calls them funnels, but there is no tapered or hollow funnel primitive in these draw calls.

Bridge, Viewport, Floodlights, and Exhaust Stacks

Original source: lines 1154-1168. The elevated details complete the harvester and close its draw function.

Original line	Mesh and component	T	R	S	Material
1155	Cube: observation bridge body	(2.8, 5.8, -2.2)	zero	(2.6, 1.4, 2)	matChassis
1157	Cube: forward bridge viewport	(2.8, 5.9, -3.25)	zero	(2.4, 0.65, 0.2)	matGlass
1159	Sphere: first forward floodlight	(1.8, 5.2, -3.3)	zero	(0.35, 0.35, 0.35)	matLight
1160	Sphere: second forward floodlight	(3.8, 5.2, -3.3)	zero	(0.35, 0.35, 0.35)	matLight
1163	Cylinder: positive-X exhaust stack	(1.6, 6.2, 1.8)	zero	(0.75, 2.6, 0.75)	matChassis
1164	Cylinder: negative-X exhaust stack	(-1.6, 6.2, 1.8)	zero	(0.75, 2.6, 0.75)	matChassis
1166	Cylinder: positive-X glowing vent rim	(1.6, 7.55, 1.8)	zero	(0.85, 0.2, 0.85)	matSpice
1167	Cylinder: negative-X glowing vent rim	(-1.6, 7.55, 1.8)	zero	(0.85, 0.2, 0.85)	matSpice

The bridge is offset to the positive-X front of the refinery deck. Its viewport is thin in Z and just forward of the bridge. The two small emissive spheres sit on either side below that viewport. The comment says they illuminate the desert, but the local code submits no spotlight parameters; their actual local effect is visibly bright geometry. Overall scene lighting remains the directional sun and ambient contribution described later.

The vertical stacks sit at the rear of the top deck. Because no rotation is supplied, their 2.6 Y scales make them upright. The slightly wider, much shorter orange cylinders sit at their tops as luminous rims. This function does not emit particles from those exact stack positions. Particle placement is handled separately by the particle simulation. With two body cubes, 76 track draws, six cutter/door/ramp/separator draws, and eight elevated-detail draws, the harvester function submits 92 meshes per invocation.

Spice Tank Construction

Four Materials and the Transport Cradle

Original source: lines 1170-1192. The tank banner describes a heavy horizontal vessel with rounded heads, supporting members, and an illuminated spice indicator. `drawMovieSpiceTank(root)` has no animation parameter. Any placement, yaw, or instance scale must already be present in `root`.

`matCradle` has diffuse (0.24, 0.23, 0.22), specular (0.3, 0.3, 0.3), and shininess 32. `matVessel` uses tank steel diffuse (0.52, 0.46, 0.38), specular (0.6, 0.6, 0.6), and shininess 64. `matSeam` is darker, with diffuse (0.18, 0.17, 0.16), specular (0.2, 0.2, 0.2), and shininess 20. `matSpice` uses diffuse (0.96, 0.42, 0.04) and emission (1.2, 0.55, 0.08), keeping default specular and shininess. None changes `isSand`, and only the spice material changes default zero emission.

Original line	Mesh and cradle component	T	R	S	Material
1182	Cube: positive-X ground skid	(1.35, 0.18, 0)	zero	(0.35, 0.35, 4.4)	matCradle
1183	Cube: negative-X ground skid	(-1.35, 0.18, 0)	zero	(0.35, 0.35, 4.4)	matCradle
1185	Cube: rear cross support	(0, 0.25, 1.6)	zero	(3, 0.25, 0.35)	matCradle
1186	Cube: forward cross support	(0, 0.25, -1.6)	zero	(3, 0.25, 0.35)	matCradle
1189	Cube: positive-X inclined stanchion, at each Z	(1.3, 1.1, z)	(0, 0, -22)	(0.28, 1.8, 0.35)	matCradle
1190	Cube: negative-X inclined stanchion, at each Z	(-1.3, 1.1, z)	(0, 0, 22)	(0.28, 1.8, 0.35)	matCradle

The stanchion loop uses exactly two Z values, -1.3 and 1.3, so its two draw statements produce four inclined support members. The long skids run along Z; the cross supports run along X; and the upright members lean through opposite Z rotations. These are independent solid boxes that visually overlap where necessary. There is no separate welding, constraint, or load-bearing solver associated with the comments. The cradle contains eight primitive draws.

Vessel Body, Domed Ends, Weld Bands, and Indicator

Original source: lines 1193-1210. The rest of the tank uses eight draws to form its pressure vessel and visible fittings.

Original line	Mesh and vessel component	T	R	S	Material
1195	Cylinder: horizontal central vessel	(0, 1.55, 0)	(90, 0, 0)	(2.2, 3.2, 2.2)	matVessel
1197	Sphere: positive-Z rounded head	(0, 1.55, 1.6)	zero	(2.2, 2.2, 1.3)	matVessel
1198	Sphere: negative-Z rounded head	(0, 1.55, -1.6)	zero	(2.2, 2.2, 1.3)	matVessel
1201	Cylinder: positive-Z reinforcing band	(0, 1.55, 0.8)	(90, 0, 0)	(2.28, 0.18, 2.28)	matSeam
1202	Cylinder: negative-Z reinforcing band	(0, 1.55, -0.8)	(90, 0, 0)	(2.28, 0.18, 2.28)	matSeam
1205	Cylinder: top loading hatch	(0, 2.72, 0)	zero	(0.65, 0.25, 0.65)	matSeam
1206	Sphere: bright relief-valve detail	(0, 2.92, 0)	zero	(0.4, 0.25, 0.4)	matSpice
1209	Cube: vertical side sight indicator	(1.15, 1.55, 0)	zero	(0.08, 1.5, 0.18)	matSpice

There are eight rows in this table and eight draws; together with the eight cradle draws, the complete tank submits 16 meshes. The central vessel extends from local Z -1.6 to 1.6. The full ellipsoidal head meshes are centered at those endpoints, overlapping the cylinder and projecting out by 0.65 along Z. They keep the same X/Y diameter as the body. The narrow band cylinders have a slightly larger 2.28 diameter and only 0.18 axial length, causing their outer circumferences to show as dark reinforcing bands around the shell.

The top hatch is a short upright cylinder. The flattened emissive sphere above it supplies the bright valve-like shape. The side sight indicator is a thin, tall cube at positive X. Its material remains constant; no fill-level calculation, changing length, pulsing brightness, or animated spice level appears in this function. The comments identify the intended industrial interpretation of the visible detail, while the draw calls determine its actual implementation. Line 1210 closes the function.

Desert Rock Construction

Strata Materials and Four Eroded Mesh Pieces

Original source: lines 1212-1225. `drawDesertRockFormation(root, variant)` reuses the dedicated eroded-rock mesh four times to suggest one larger wind-carved formation. `matStrata1` has diffuse (0.42, 0.28, 0.17), specular (0.15, 0.15, 0.15), and shininess 16. `matStrata2` has darker diffuse (0.32, 0.20, 0.11), specular (0.12, 0.12, 0.12), and shininess 12. Both retain zero emission.

Line 1220 performs a chained assignment, setting both materials' `isSand` fields to 2. The integer is a surface category consumed by the rendering shaders, not a C++ boolean assertion that the rocks are sand. It distinguishes their treatment from ordinary metal or cloth (0) and the main dune terrain (1). The shader chapter explains the category's detailed lighting and procedural color effect.

Original line	Visual component	T	R	S	Material
1221	Dominant base crag	(0, 2.2, 0)	(8, float(variant * 19), -5)	(8, 7, 6)	matStrata1
1222	Upper darker crag	(1, 5, -1)	(0, 55, 10)	(5, 5, 4)	matStrata2
1223	Lower negative-X outcrop	(-3, 0.8, 2)	(5, -40, 12)	(4, 3, 3)	matStrata2
1224	Small positive-X rubble piece	(3.2, 0.3, 2.2)	(15, 70, 20)	(2.2, 1.6, 2.1)	matStrata1

Every row draws `g_meshRock`, not a cube or sphere. The variant only changes the first piece's Y rotation by 19 degrees per integer variant. It does not select another mesh or alter the other three pieces. A separate per-site rotation in the master scene rotates the whole formation as well. Large nonuniform scales make the repeated mesh less obviously identical, and the different relative positions create a layered cluster. The function does not animate erosion or generate a fresh random rock on each frame; it transforms the mesh already generated during startup.

Rescue Objects and People

Header Integration, Materials, and Landing Base

Original source: lines 1227-1245. The two include directives pull in `arrakis_worm.h` and `arrakis_hud.h` before the rescue-world function and later runtime references. In the organized project these are application headers under `src/`. Including them makes their definitions available to this translation unit; it does not itself draw a worm or show the HUD.

`drawRescueWorld(time)` constructs five materials. `cyan` uses diffuse (0.05, 0.5, 0.55) and emission (0.03, 0.6, 0.65). `amber` uses diffuse (0.9, 0.5, 0.1) and emission (0.6, 0.25, 0.03). `dark` changes diffuse to (0.16, 0.19, 0.19). `cloth` changes diffuse to (0.30, 0.24, 0.17) and specular to (0.03, 0.03, 0.03), leaving shininess 32. `visor` has diffuse (0.08, 0.20, 0.27), specular (0.8, 0.8, 0.8), and shininess 90. Other fields stay at the shared defaults, including `isSand = 0` for all five.

The base parent is an identity matrix translated to `g_mission.base`. Mission reset has already put that position at the intended terrain elevation. This function does not sample the base height again. It draws the following local parts.

Original line	Mesh and base component	T	R	S	Material
1237	Cylinder: broad landing-pad disk	(0, 0.25, 0)	zero	(24, 0.5, 24)	dark
1238	Ring: illuminated landing perimeter	(0, 0.55, 0)	zero	(11, 1, 11)	cyan
1239	Cube: long Z-oriented cross stripe	(0, 0.56, 0)	zero	(1.2, 0.05, 10)	cyan
1240	Cube: long X-oriented cross stripe	(0, 0.56, 0)	zero	(10, 0.05, 1.2)	cyan
1243	Cylinder: perimeter beacon pole	(cos(a) * 13, 2, sin(a) * 13)	zero	(0.2, 4, 0.2)	dark
1244	Sphere: beacon light	(cos(a) * 13, 4.1, sin(a) * 13)	zero	(0.65, 0.65, 0.65)	cyan

The ring scale is literal; it multiplies the ring generator's existing radii rather than specifying a cylinder diameter. The pad disk extends from Y zero to 0.5 relative to the base. The ring and cross are slightly above its top, avoiding exactly coincident surfaces. The pole loop uses integer *i* from zero through three and sets $a = i * \text{half_pi}$, so four beacon pairs occupy the cardinal directions at radius 13. Their cylinder centers are two units high and their heights are four, placing their bottoms at base level; the spheres are just above their tops. Four fixed pad draws plus eight beacon draws give 12 base meshes per invocation. None blinks as a function of *time* in this code.

Released Group Markers and Terrain Alignment

Original source: lines 1246-1261. The group loop uses integer indices from zero up to `g_mission.groups.size() - 1`, with a cast so the container size can be compared to an `int`. It takes a `const` reference to each group. An unreleased group immediately executes `continue`, so there is no ground marker for workers still associated with an unreleased evacuation group.

For a released group, `waiting` starts at zero. An inner crew loop increments it only for a crew member whose group equals the current index and whose state is `Running` or `Waiting`. Other groups and states do not count. If the final count is zero, the group is skipped as well. Thus a released marker remains visible while at least one member is actively running or waiting outside, and disappears after that condition no longer holds. The code bases this visibility on crew states, not on proximity to the camera or aircraft.

The marker position `p` copies `group.center` but replaces its Y value with `getDuneHeight(p.x, p.z) + 0.2`. A translation to that position forms `ring`. The surface normal is sampled at the same X/Z. The code computes `axis = cross((0, 1, 0), normal)` and rotates the ring only if `length(axis) > 0.001`. The angle is `acos(clamp(normal.y, -1, 1))`, and the rotation axis is normalized. The clamp prevents small numeric excursions beyond the legal inverse-cosine range. This rotates the ring's local upward direction toward the terrain normal when there is a usable axis.

Line 1258 draws `g_meshRing` using the tilted root, local T and R zero, S (4.5, 1, 4.5), and `amber`. Lines 1259-1260 deliberately use a fresh translation-only root at `p`, rather than the tilted ring matrix. They draw a dark cylinder at T (4, 2, 0), R zero, S (0.12, 4, 0.12), then an amber sphere at T (4, 4.1, 0), R zero, S (0.5, 0.5, 0.5). The pole and light therefore stay vertical even when the ring follows a slope.

The pole's height is not independently adjusted for the terrain four units to the side: both use the sampled center height through `p`. On sloped ground its bottom can consequently sit differently relative to the nearby surface. If the cross-product axis is too small, the ring remains unrotated; there is no explicit opposite-normal fallback in this group-marker branch. The function uses the actual upward-facing dune normals produced by the terrain helper rather

than adding a general orientation solver.

Visible Crew Bodies, Strides, and Pickup Lift

Original source: lines 1262-1278. The crew loop examines every crew vector entry using its integer index *i*. It only renders Waiting and Running members. Inside, aboard, rescued, or lost crew states do not receive body geometry through this branch. The const reference *c* remains unchanged; rendering works with a copied *crewPosition*.

If *i* equals *g_mission.target* and *g_mission.pickup* > 0, line 1267 raises the copied Y coordinate by $(g_ornPos.y - crewPosition.y - 1.8) * smoothstep(0, 1, pickup)$. At zero eased progress the original height is retained; at full eased progress the crew root reaches *g_ornPos.y* - 1.8. Because the change is made to a copy, it does not overwrite the gameplay crew position. X and Z stay at the member's stored location; this rendering formula only interpolates height.

The crew root is identity translated to that resulting position. *stride* is a degree-valued animation amount: running uses $\sin(time * 11 + i) * 24$, while waiting uses $\sin(time * 2 + i) * 4$. Index *i* adds a phase offset so all people do not swing together. Running uses a faster, wider movement than the small waiting sway. The input *time* comes from the paused-aware simulation clock in the main loop, not directly from the latest GLFW wall clock after that clock is reassigned.

Original line	Mesh and person component	T	R	S	Material
1270	Sphere: torso	(0, 1.05, 0)	zero	(0.55, 0.85, 0.4)	cloth
1271	Sphere: head/helmet	(0, 1.65, 0)	zero	(0.43, 0.46, 0.43)	cloth
1272	Sphere: front visor	(0, 1.67, 0.17)	zero	(0.32, 0.15, 0.10)	visor
1274	Cylinder: leg for each side	(side * 0.14, 0.4, 0)	(side * stride, 0, 0)	(0.17, 0.8, 0.17)	cloth
1275	Cylinder: arm for each side	(side * 0.34, 1.1, 0)	(-side * stride, 0, side * 14)	(0.15, 0.7, 0.15)	cloth
1277	Sphere: overhead locator light	(0, 2.25, 0)	zero	(0.18, 0.18, 0.18)	cyan

The side loop runs exactly twice, for -1 and 1. Leg X rotations have opposite signs on the two sides. Arm X rotations oppose the corresponding leg rotations, while fixed *side * 14* Z rotations splay the arms. These cylinders rotate about their own centers because there is no additional hip or shoulder pivot matrix. There are no separate elbows, knees, hands, feet, or skeletal joints in this function. One visible crew member generates eight draws: torso, head, visor, two legs, two arms, and locator sphere.

No heading rotation is applied to the person root. Regardless of the member's walking direction, the locally positive-Z visor remains oriented according to the same world-axis convention after translation. This is an implementation detail of the primitive figure, not an undocumented facing system. The overhead cyan sphere is attached to the same root, so it rises with the pickup animation.

Winch Rope Orientation and Sandworm Call

Original source: lines 1279-1293. A rope is drawn only when *g_mission.pickup* > 0 and *g_mission.target* >= 0. The branch then indexes *g_mission.crew[target]* directly; it does not independently check that the target is less than the vector size. Valid target management is the mission logic's responsibility.

The initial lower endpoint *end* is the target's stored position plus (0, 1, 0). Its Y is eased by $(g_ornPos.y - end.y - 0.8) * smoothstep(0, 1, pickup)$. This is consistent with the lifted person's root: the person root

tends toward aircraft Y minus 1.8, and adding one puts the rope endpoint at aircraft Y minus 0.8. The upper endpoint start is `g_ornPos - (0, 0.7, 0)`. Aircraft roll and pitch are not used to rotate this attachment offset; it is a world-vertical subtraction from the aircraft position.

`dir = end - start` gives the endpoint displacement, `length = length(dir)` gives the desired rope length, and a translation to `(start + end) * 0.5` puts the cylinder center halfway between them. To align the cylinder's original positive-Y axis, the code crosses world up with `dir / length`. A usable cross product causes a rotation by `acos(clamp(dir.y / length, -1, 1))` about the normalized cross-product axis. If that axis is too small and `dir.y < 0`, it instead rotates by π around X, handling a downward parallel rope. If the displacement points upward with an effectively zero cross product, no rotation is necessary.

Line 1289 draws a cyan cylinder with the assembled rope parent, local T and R zero, S (0.055, length, 0.055). Its thickness is fixed while its axial length tracks the endpoints. The code divides by length before testing the cross-product length and supplies no zero-length guard. It therefore assumes distinct endpoints; this chapter describes the implementation rather than adding a hypothetical correction. The model is a straight cylinder, not a simulated flexible cable or a segmented rope.

After the optional rope, lines 1291-1292 always call `drawSandworm(wormPosition(), wormAttackTime(), time, g_mission.wormYaw, clamp(elapsed / breachAt, 0, 1))`. These arguments supply current terrain-following worm position, time relative to breach, animation time, worm heading, and normalized pursuit progress. The worm renderer itself decides what to show for those phases; the caller does not suppress the call before breach or after mission results. The separate worm chapter explains that renderer's internal meshes. Line 1293 closes `drawRescueWorld`.

Master Scene Traversal

Terrain and Aircraft Visibility

Original source: lines 1295-1316. The banner states that `drawFullScene(curTime)` is used twice per frame, for shadow depth and then the main view. It does not advance gameplay or the wing phase; it constructs the scene from already updated state.

The first object is the terrain. A fresh `matSand` gets diffuse (0.76, 0.49, 0.23), specular (0.25, 0.25, 0.25), shininess 16, and `isSand = 1`, with default zero emission. `drawMeshPrimitive(g_meshTerrain, identity, matSand)` submits the terrain with no extra model translation, rotation, or scaling. The terrain generator has already placed its vertices in scene space.

The aircraft branch runs only if `!g_mission.planeLost`. Its matrix starts as identity translated to `g_ornPos`, then appends Y rotation by `g_ornYaw`, X rotation by `g_ornPitch`, and Z rotation by `g_ornRoll`, each converted from degrees to radians. The composed matrix is passed to the aircraft function along with current phase, roll, pitch, and boost state. The latter two numeric inputs are redundant inside the aircraft function, as noted earlier; the actual bank and pitch are already encoded in this parent matrix. If `planeLost` is true, none of the aircraft's hull, wings, or exhaust draws is submitted here.

Harvester Terrain Slope and Swallowing Transform

Original source: lines 1317-1333. The harvester is rendered only while `wormAttackTime() < 12`. Because attack time is mission elapsed time minus breach time, this includes the entire negative pre-breach interval as well as the first twelve seconds of attack. At exactly 12, this branch stops drawing it.

Its base height `hy` is the dune height at `(g_harvX, g_harvZ)`. `sink = smoothstep(6, 12, attackTime)` stays zero until attack second six, eases upward between six and twelve, and reaches one at twelve. The root translation uses `height hy - sink * 20`, sinking the machinery by as much as twenty units while it is still being drawn. It then receives the harvester's Y heading in degrees.

Slope alignment is expressed in the vehicle's heading-relative axes. `hnorm` is the sampled dune normal, `forward` is the horizontal forward vector from `harvesterForward(harvesterYaw)`, and `right = cross(forward, (0, 1, 0))`. The X rotation is `atan2(-dot(hnorm, forward), hnorm.y)` and the Z rotation is `atan2(-dot(hnorm,`

`right), hnorm.y)`. Unlike the heading, these inverse-trigonometric angles are already radians and are passed directly to GLM. Using the projections onto forward and right makes the apparent uphill/downhill tilt depend on the crawler's heading instead of blindly applying world-X/world-Z slopes.

An additional X rotation of $\text{sink} * 0.8$ radians supplies the swallowing tilt. It is not 0.8 degrees: GLM receives it directly. The final harvester matrix includes translation, yaw, terrain pitch, terrain roll, and this extra pitch in that append order. `drawSpiceHarvester` receives it together with `g_mission.harvesterTravel`, which animates tracks and drum according to accumulated travel. The master draw does not itself advance that travel or move the crawler's route.

Tank Instances, Eight Rock Sites, and Rescue World

Original source: lines 1334-1354. Every tank in `g_tanks` is considered separately. Its branch uses `if (wormAttackTime() > 12) continue`, which differs from the harvester's `strict < 12` draw condition. A tank still draws at exactly attack second twelve; the harvester does not. At later times every tank iteration skips the remaining body. This skips rendering, not deletion of tank entries from the vector.

For a drawn tank, terrain height is recomputed from the tank's X/Z coordinates. The stored `tank.pos.y` is not used. The root becomes translation to `(tank.pos.x, terrainHeight, tank.pos.z)`, followed by Y rotation from `tank.yaw` degrees and uniform scale `(tank.scale, tank.scale, tank.scale)`. The vessel function then supplies all its local geometry. No dune-normal tilt is applied to the tanks here; their roots remain upright even on a slope. The comment's description of tanks deployed behind the harvester does not make this loop continuously relocate them with the crawler. Their positions come from the instance data maintained elsewhere.

The rocks use a fixed eight-iteration loop. Each `g_rockSites[i]` is read as a two-dimensional X/Z site: its `.x` becomes `rx` and its `.y` becomes `rz`. The world Y value is the dune height minus 0.5, partially embedding each formation's root. A site root is translated there and rotated around Y by $i * 45$ degrees. The same integer `i` is passed as the formation variant, giving its first rock an additional local $i * 19$ Y rotation. Eight sites times four rock meshes yields 32 rock draws per scene traversal.

Finally the function calls `drawRescueWorld(curTime)`, which adds the base, active group markers, visible people, optional rope, and worm. Rescue markers are not rendered as a separate screen overlay here; they are world-space meshes and therefore participate in both full-scene passes. The HUD is drawn later, outside `drawFullScene`, only in the main framebuffer sequence. The closing brace completes the shared scene traversal.

GLFW Input Callbacks

Framebuffer Resize and Mouse Recentering

Original source: lines 1356-1374. The callbacks section begins with a framebuffer-size function taking an unnamed window pointer and new framebuffer width/height. Only when both `w > 0` and `h > 0` does it update `g_scrW`, `g_scrH`, and the viewport to `(0, 0, w, h)`. Zero-size dimensions, such as can occur while minimizing, are ignored rather than stored. This keeps the last positive dimensions available to projection and screenshot code. Framebuffer dimensions are physical rendering pixels and can differ from the logical window dimensions used for cursor movement.

`centerMouse(win)` obtains logical window width and height using `glfwGetWindowSize`. It immediately zeros `g_mouseHover` and `g_mouseTurnRate`, then places the cursor at `(width * 0.5, height * 0.5)` using floating-point halves. This resets both the instantaneous hover displacement and the smoothed turning state; it is more than a visual mouse reposition. It does not resize the framebuffer, clear held keyboard keys, or alter the aircraft heading.

Cursor Motion Eligibility and Coordinate Space

Original source: lines 1375-1387. Cursor motion is accepted only when the mission is `Flying`, is not paused, and the window reports focus. If any of those conditions fails, both mouse-hover and turn-rate globals are set to zero and the callback returns immediately. Thus moving the mouse at the title screen, during results, while paused, or without focus cannot keep injecting steering through this callback.

For accepted motion, the code queries the current logical window dimensions and calls `setMouseHover(x, y, width, height)`. It deliberately does not use `g_scrW/g_scrH`, since cursor coordinates are logical window pixels. The helper validates dimensions and coordinates, converts them to centered hover values, and applies its dead zone; that helper is documented with the mission code. This callback itself neither recenters the mouse after every movement nor hides or locks the pointer. Its model is cursor position relative to the window center, not accumulated relative mouse displacement.

Key State and Immediate Actions

Original source: lines 1388-1427. `key_callback` names the window, key, and action arguments but leaves scan code and modifier flags unnamed because it does not inspect them. For valid integer key indices $0 \leq \text{key} < 1024$, press sets `g_keys[key]` true and release sets it false. Repeat is not treated as another press in those statements, so it neither toggles features nor changes the held state. A held key remains true from its original press until release or the focus-loss clearing callback.

The rest of the function uses independent `if` statements for the following immediate key actions. Every action below requires `GLFW_PRESS`; holding a key does not repeatedly toggle it through GLFW's repeat action.

Key	Exact callback behavior	Phase restriction
Escape	Sets the window should-close flag to true.	None.
F	Flips <code>g_wireframe</code> ; selects <code>GL_LINE</code> or <code>GL_FILL</code> for front and back polygon faces.	None.
C	Advances <code>g_camMode</code> with <code>(g_camMode + 1) % 3</code> .	None.
Enter	From Title calls <code>resetMission(true)</code> ; from Debrief calls <code>resetMission(true, true)</code> ; then calls <code>centerMouse</code> regardless.	Reset depends on phase; recentring does not.
R	Calls <code>resetMission(true)</code> and <code>centerMouse</code> .	None.
M	Calls <code>g_audio.toggleMute()</code> .	None.
P	Flips <code>g_mission.paused</code> and recenters the mouse.	Only Flying.
F11	Toggles the fullscreen flag, switches window monitor mode, then recenters.	None.

Escape requests termination of the loop but does not immediately delete resources inside the callback. F changes rasterization mode for subsequent geometry, while the HUD later explicitly forces filled polygons for its own draw. C's three values are chase, close view, and overhead in the helper; the word cockpit in the source label does not insert an interior cockpit mesh. Enter while already Flying only centers the mouse, because neither reset condition matches. Enter from Debrief passes the second `true` argument to advance the mission wave, while R retries using the reset function's default second argument. The callback does not apply modifier requirements such as Ctrl or Alt.

F11 always fetches the primary monitor and its current video mode. When entering fullscreen, it attaches the existing window to that monitor at position `(0, 0)`, using the mode's width, height, and refresh rate. When leaving fullscreen, it detaches from any monitor and requests a window at `(100, 100)` with size `1280 x 720` and refresh-rate argument zero. It does not save and restore the prior arbitrary window position or size. This branch assumes that primary-monitor and video-mode queries return usable values; it includes no null check. Resizing and cursor-centering can also trigger the separately registered GLFW callbacks. The function's final brace ends input handling without any mission update call.

Screenshot File Encoding

Readback Buffer and Little-Endian Writer

Original source: lines 1429-1439. The main-entry section banner precedes `captureScreenshot(path)`. The function allocates `g_scrW * g_scrH * 4` unsigned bytes. The width is converted to `size_t` before multiplication, avoiding a purely signed intermediate for the full allocation expression under the normal positive-dimension invariant.

`glReadPixels(0, 0, width, height, GL_BGRA, GL_UNSIGNED_BYTE, pixels.data())` reads the entire current framebuffer region starting at its lower-left corner. Each pixel contributes blue, green, red, and alpha bytes. The function does not choose a new framebuffer or call `glReadBuffer`; it relies on the rendering state at the call site, where the main framebuffer has been rendered and buffer swapping has not yet occurred. It also does not explicitly set pixel pack state. At four bytes per pixel, every row length is a multiple of four, compatible with the usual default pack alignment.

It opens the requested path as a binary `std::ofstream`. If opening fails, it returns false immediately. Readback has already taken place before this check. The function does not create missing parent directories or infer an image format from the filename extension. Its local `word(value, bytes)` lambda emits bytes least-significant bytes of a `uint32_t` in order, using right shifts by `i * 8`, a mask of 255, and `out.put`. That implements little-endian integer serialization independently of directly writing a C++ header struct with platform padding.

BMP Header, Row Direction, and Result

Original source: lines 1440-1446. The output is a 32-bit uncompressed BMP. The first two bytes are literal B and M. The following fields are written in this exact order.

Header field	Bytes	Assigned value and meaning
File signature	2	BM, identifying a Windows bitmap file.
Total file size	4	<code>54 + pixels.size()</code> , cast to <code>uint32_t</code> .
Reserved fields	4 combined	Zero for both reserved 16-bit fields.
Pixel-data offset	4	54, immediately after the two headers.
DIB header size	4	40, the <code>BITMAPINFOHEADER</code> layout.
Width	4	<code>g_scrW</code> .
Height	4	Positive <code>g_scrH</code> , specifying bottom-up storage.
Planes	2	1.
Bits per pixel	2	32.
Compression	4	0, uncompressed <code>BI_RGB</code> .
Pixel-data byte count	4	<code>pixels.size()</code> , cast to <code>uint32_t</code> .
Horizontal resolution	4	Zero.
Vertical resolution	4	Zero.
Palette colors used	4	Zero.
Important palette colors	4	Zero.

OpenGL supplies the bottom framebuffer row first, and positive BMP height also expects bottom-up rows, so no explicit vertical flip is needed. Four bytes per pixel make rows naturally aligned for BMP storage without added row padding. The BGRA byte order matches the pixel layout written here. `BI_RGB` at 32 bits stores the fourth byte, but this routine

does not add alpha bit masks or promise that every bitmap viewer treats that byte as usable transparency.

Line 1444 writes the entire byte vector as raw data after the header. It casts the pointer to `const char*` and the length to `std::streamsize` to match the stream API. `return out.good()` reports the stream's status after these writes; the local stream closes on scope exit. The function does not test for an OpenGL readback error itself or perform image-content validation. Sizes are serialized into 32-bit fields without a separate oversized-image guard. It is a screenshot of what was rendered, including the HUD when called from the frame loop, not a scene-only render or a PDF export.

Application Startup

Command-Line Dispatch and GLFW Initialization

Original source: lines 1448-1459. `main(argc, argv)` first checks whether `argv[1]` is exactly `--test-game`. If so, it directly returns `runMissionTests()`, bypassing GLFW initialization, window creation, GLAD loading, audio initialization, and the render loop. The tests are implemented in `tests/arrakis_tests.h` in the organized project. This chapter does not duplicate their individual assertions.

Otherwise it sets four booleans by comparing only the first argument. `smokeTest` corresponds to `--smoke-test`, `breachTest` to `--smoke-breach`, `mouseTest` to `--smoke-mouse`, and `pursuitTest` to `--smoke-pursuit`. `renderTest` is their logical OR. Since each equality tests the same single string, normally only one mode can be selected. There is no general command-line parser, combined-flag scan, help option, or validation of unrecognized arguments. With an unrecognized first argument, the normal interactive path is followed. An optional second argument is considered later only as the screenshot path for render-test modes; additional arguments are not used in this code.

First argument	Startup/runtime specialization
<code>--test-game</code>	Runs mission tests and returns their result without graphics startup.
<code>--smoke-test</code>	Starts a flying mission and renders with fixed simulation steps for the bounded test run.
<code>--smoke-breach</code>	Additionally advances mission state to seven seconds after its breach time and uses the breach camera.
<code>--smoke-mouse</code>	Supplies synthetic horizontal mouse positions, holds W, cycles camera modes, and checks camera heading.
<code>--smoke-pursuit</code>	Advances mission state to 110 seconds and uses a wider harvester/worm pursuit camera.
No recognized first argument	Starts in the ordinary title-screen path.

If `glfwInit()` fails, the program writes `Failed to initialize GLFW` to standard error and returns `-1`. No GLFW window or OpenGL resources have been created by the following startup sequence yet. This failure is separate from the later render-test failure status, which uses exit code one.

Context Hints, Monitor Information, and Window Fallback

Original source: lines 1460-1485. GLFW hints request OpenGL version 3.3 with a core profile. The program gets the primary monitor and its current video mode, then copies that mode's red, green, and blue bit depths and refresh rate into window hints. It also requests four multisample samples. These are requested capabilities; GLFW and the underlying platform determine the actual context/window characteristics. No explicit alpha-bit, depth-bit, stencil-bit, or hidden-window hint is set in this range.

Although the monitor's properties are queried, the first `glfwCreateWindow` uses `nullptr` for its monitor argument, so startup is windowed. It uses the current global dimensions, initially 1280 x 720, with the title `ARRAKIS | Harvester Down - Desert Rescue`, and `nullptr` for the shared-context argument. The comment explicitly favors desktop accessibility and leaves fullscreen switching to F11. Render smoke tests also use this windowed creation path rather

than a special headless or hidden window.

If creation returns null, the program prints `Window creation failed, falling back to windowed mode...`, clears `g_isFullscreen`, sets the dimensions to 1280 x 720, and attempts another window titled `ARRAKIS`, again with no monitor and no shared context. The fallback message does not mean the first attempt was fullscreen: both calls are windowed in the inspected implementation. The second attempt changes the title and resets the dimensions but does not clear the previously requested context, color, refresh, or sample hints. If it also fails, GLFW is terminated and `main` returns -1.

Primary-monitor and mode pointers are dereferenced without a null guard in this startup range, just as they are in F11 handling. This is an assumption of the actual code, not a fallback for a machine with no available monitor.

Context Activation, Callback Registration, and Focus Loss

Original source: lines 1486-1505. The successful window's OpenGL context is made current. Its framebuffer size is read into `g_scrW` and `g_scrH`, so rendering dimensions reflect actual framebuffer pixels rather than only the requested logical window size. The named framebuffer-size, key, and cursor-position callbacks described above are registered next.

Three additional callbacks are inline lambdas. The cursor-enter callback ignores both its window pointer and the entered/left flag and always zeros `g_mouseHover` and `g_mouseTurnRate`. It therefore clears steering on either entry or exit. The window-size callback ignores the supplied new size arguments and calls `centerMouse(win)`, which queries the current logical size itself. The focus callback only acts when `focused` is false: it clears every element of the 1024-entry key array using `std::fill`, zeros the two mouse steering values, and sets mission pause true if the mission is Flying.

On focus regain, that callback has no corresponding resume branch. A flying mission remains paused until another action, such as P, changes the flag; a render test later forces it false on each normal frame. Clearing held keys avoids a key remaining logically down after a release event occurs outside focus. The focus callback does not close the window, reset the mission, mute the audio flag, or save a special focus-loss result state.

`glfwSwapInterval(1)` requests vertical synchronization against the current context. It governs swap timing, not the physics step size or the number of mission substeps. GLAD entry points are loaded only afterward. GLFW callback registration does not itself invoke the OpenGL viewport callback here; event processing is deferred to the later loop.

GLAD, Audio Initialization, Diagnostic Banner, and GL State

Original source: lines 1506-1526. The GLAD loader is called with GLFW's procedure-address function cast to `GLADloadproc`. If it fails, the program prints `Failed to initialize GLAD`, calls `glfwTerminate`, and returns -1. This path does not reach the final normal resource cleanup block. GLFW termination handles GLFW-owned window lifetime, but no application geometry or shader resources have yet been created by the later lines.

`g_audio.init()` runs after GLAD succeeds. Its boolean return value is not checked by `main`, so an audio initialization failure is not itself fatal to graphics startup. The audio subsystem records whether it initialized and supports its silent-mode path; its implementation and asset lookup are covered separately. Organizing the audio files under `assets/audio/` concerns that subsystem's file locations, not extra arguments passed here.

Six output statements print separator lines, the game title, renderer string from `glGetString(GL_RENDERER)`, OpenGL version from `glGetString(GL_VERSION)`, and current framebuffer resolution. These diagnostics describe the active driver and framebuffer, not a benchmark. The program does not inspect these strings to select a different renderer or shader variant.

Initial OpenGL state enables depth testing and multisampling, chooses `GL_LEQUAL` for depth comparison, enables face culling, and selects back-face culling. Less-or-equal allows a fragment at exactly the stored depth to pass. Individual render phases temporarily change selected state later; they do not reinitialize all possible OpenGL settings every frame. Blend remains unenabled by this initialization block until the particle stage explicitly enables it.

Procedural Meshes and Shader Programs

Original source: lines 1527-1543. Startup generates and uploads the reusable scene meshes in a fixed order. `createCube()` creates the box used for structural details; `createCylinder(24)` requests 24 radial slices; `createSphere(16, 24)` uses 16 latitude and 24 longitude subdivisions; and `createWingBlade()` creates the tapered aerofoil reused by all aircraft wings. `createDuneTerrain(260, 260, 900.0)` builds the broad terrain using those grid and extent arguments. The actual startup values here take precedence over looser descriptions elsewhere in source comments.

`createQuad()` supplies the billboard mesh for particle rendering. `createRing()` supplies landing and evacuation markers, and `createErodedRock()` supplies rock formations. `initSandwormMeshes()` initializes the worm renderer's own resources. Each result is stored in a corresponding global mesh so multiple objects and frames can reuse it. These calls happen once during startup rather than regenerating primitives inside every `drawPart`.

Four `buildProgram` calls compile/link the scene, shadow, particle, and sky programs from their respective vertex and fragment shader strings: `SCENE_VERT/SCENE_FRAG`, `SHADOW_VERT/SHADOW_FRAG`, `PARTICLE_VERT/PARTICLE_FRAG`, and `SKY_VERT/SKY_FRAG`. The resulting IDs are stored in `g_progScene`, `g_progShadow`, `g_progParticle`, and `g_progSky`. The preceding rendering chapter explains the individual shader expressions and mesh-generation loops; this range connects those resources to the application's lifecycle.

Shadow/Sky/HUD Resources and Particle Instance Attributes

Original source: lines 1544-1563. `setupShadowFramebuffer()` creates the depth-rendering target, `setupSkyVAO()` prepares the sky's screen-filling triangle resources, and `initRescueHud()` prepares HUD resources. Immediately afterward, startup checks both `g_shaderOkay` and the HUD resource program ID. If any earlier shader build reported failure or the HUD program is zero, it destroys the window, terminates GLFW, and returns 1.

This early failure path does not run the comprehensive cleanup at the bottom of `main`: it has no explicit calls here to delete the already created meshes/programs or shut down audio. Context destruction and process termination end their usable lifetime, but the code's explicit successful-path deletion sequence is not executed. The condition is also not a full audit of every OpenGL object or framebuffer; it checks the displayed shader status and HUD program specifically.

`initParticles()` initializes the CPU particle collection. The program generates `g_particleInstanceVBO`, binds the quad VAO, and binds that new buffer as `GL_ARRAY_BUFFER`. It initially allocates `MAX_PARTICLES * sizeof(ParticleInstance)` bytes with null data and `GL_STREAM_DRAW`. The earlier constant is `MAX_PARTICLES = 2400`. Each `ParticleInstance` contains two `glm::vec4` fields, `posSize` and `colorAlpha`.

The two-iteration loop enables vertex attribute locations 3 and 4. Both attributes contain four `GL_FLOAT` values, are not normalized, use `sizeof(ParticleInstance)` as stride, and have offsets 0 and `sizeof(glm::vec4)` respectively. The offset is passed using a `reinterpret_cast<void*>`, which OpenGL interprets as a byte offset into the currently bound VBO rather than a CPU pointer to read. Setting each divisor to one makes its value advance once per drawn instance rather than once per quad vertex. The quad's existing per-vertex attributes remain attached to the same VAO.

After unbinding the VAO, `g_particleInstances.reserve(MAX_PARTICLES)` reserves CPU vector capacity without filling it. `resetMission(renderTest)` initializes the world: normal interactive startup receives false and opens at Title; any recognized render-test mode receives true and starts Flying. `centerMouse(g_window)` then clears steering and places the pointer at the logical center. This order ensures reset mission state exists before the camera and timing state are initialized.

Render-Test Fast-Forward and Environment Constants

Original source: lines 1564-1578. Breach and pursuit modes advance the mission before ordinary frames begin. `targetTime` is `g_mission.breachAt + 7` for breach mode and `110.0` for pursuit mode. `steps = int(ceil(targetTime * 120))`, and the loop calls `updateMission(targetTime / steps)` exactly that many times. This supplies roughly 120 simulation updates per target second instead of passing the entire interval as one unstable update.

It then places the aircraft at `harvesterPosition() + (35, 16, 40)` and sets `g_mission.altitude = 16`. The direct position is a world offset from the harvester; the altitude field records a separate intended terrain-following height. This range does not reset velocity, yaw, or all remaining mission fields after fast-forward. It also does not explicitly assign `elapsed = targetTime`: elapsed state is whatever the repeated float-valued mission updates produced, subject to their phase checks and floating-point accumulation. Other render-test modes do not execute this pre-advance block.

Lighting constants are local to main: `sunDir` is the normalized vector `(-0.35, 0.12, -1.0)`, `sunColor` is `(1.15, 1.04, 0.85)`, `ambientColor` is `(0.34, 0.38, 0.44)`, `fogColor` is `(0.90, 0.74, 0.47)`, and `fogDensity` is `0.0022`. Normalizing the direction keeps subsequent dot products and light-camera placement independent of the vector's original magnitude. Sun color values above one allow a bright warm contribution; these constants are not clamped by main. Fog supplies both the framebuffer clear color and the scene shader's atmospheric color. No day/night clock or runtime light-color change appears here.

Initial Clocks, Camera Damping State, and Test Counters

Original source: lines 1579-1593. `prevTime` records the current GLFW time as a float. `camPosSmoothed` starts at aircraft position plus `(0, 26, 42)`, and `camTargetSmoothed` starts at the aircraft itself. This initial position is a world-space offset, not yet the helper's heading-relative chase placement. The first normal camera update below replaces X/Z placement with the actual current offset while smoothing height and target.

`simulationTime` starts at `g_mission.elapsed`, including any pre-advanced test state. It is a separate local animation clock, not an alias of the mission timer. `fps` starts at 60, `frameCount` at zero, and `cameraRevision` copies `g_missionRevision`. `renderFailed` starts false. `smokeStarted` records a fresh double-precision GLFW timestamp after resource setup and any fast-forward work; therefore the printed render-test duration later excludes those earlier initialization costs.

The banner at lines 1591-1593 introduces the main loop. There is no second thread or scheduler in this section: event handling, simulation stepping, audio parameter updates, both geometry passes, particles, and the HUD are driven sequentially by the loop.

Frame Timing and Simulation

Window Lifetime, Minimize Wait, and Delta Clamp

Original source: lines 1594-1606. The loop condition continues while GLFW's should-close flag is false. On an iconified/minimized window, it calls `glfwWaitEvents()`, updates `prevTime` to the current time after that wait, and immediately continues. That iteration performs no ordinary polling, mission update, audio update, rendering, buffer swap, screenshot, or render-test frame-count increment. Resetting `prevTime` avoids counting the entire minimized wait as the next physics interval.

On a normal iteration, `curTime` initially means GLFW wall time, `dt = curTime - prevTime`, and `prevTime` is updated before events are polled. If `dt > 0.1`, it is reduced to `0.1`; there is no corresponding lower clamp in these lines. `realDt` copies this already capped value. Thus the FPS estimate later uses the capped interval, not the uncapped duration of an unusually slow frame. The clamp bounds simulation work and jump size at low rendering rates but deliberately discards excess elapsed wall time rather than accumulating a catch-up backlog.

`glfwPollEvents()` delivers callbacks near the beginning of a normal frame. A callback can reset the mission, pause it, change rendering mode, or request close before subsequent operations. Setting should-close during polling does not instantly jump out of this body: the existing frame still continues unless other control flow changes it, and the next loop-condition check ends the run.

Fixed Render-Test Input and FPS Smoothing

Original source: lines 1607-1618. Every render-test frame replaces `dt` with `1 / 60` and forces `g_mission.paused = false`. It clears `g_mouseHover` but does not clear all keys or explicitly zero `g_mouseTurnRate` at that point. The synthetic test time is independent of actual wall time and VSync timing; `realDt` still retains the earlier measured, capped interval for FPS display.

Mouse mode queries logical window dimensions and calls `setMouseHover` at 85 percent of the width for `frameCount < 60`, or 15 percent otherwise, always halfway down the window. Frames zero through 59 therefore steer from the right side; frames 60 through 119 use the left side. It sets the `W` key state true and selects `g_camMode = (frameCount / 40) % 3`, testing mode zero for the first 40 frames, one for the next 40, and two for the last 40. No synthetic release of `W` is needed after completion because the test closes the application; the code does not restore prior input state mid-run.

The FPS value is updated with `mix(fps, 1 / max(realDt, 0.0001), 0.03)`. Only three percent of the new instantaneous estimate is incorporated each frame, giving a damped display. The minimum denominator protects against division by zero and caps the instantaneous estimate at 10000. Because `realDt` has already been capped above at `0.1`, an extremely long frame supplies an estimate of ten rather than its truly lower uncapped FPS. This statistic is display telemetry, not a pass/fail performance threshold.

Bounded Mission Substeps, Debrief Time, and Particle Updates

Original source: lines 1619-1630. Simulation work runs only when the mission is not paused. It computes `steps = max(1, int(ceil(dt / (1 / 120))))`, then invokes `updateMission(dt / steps)` repeatedly. At the normal render-test `dt = 1 / 60`, this is approximately two substeps per rendered frame. At the maximum `0.1` interval it is approximately twelve. The `max(1, . . .)` prevents a zero-iteration loop for a zero interval. Every substep receives an equal fraction of the frame interval.

The mission helper itself rejects updates outside Flying or while paused, so entering this outer block does not force title or debrief gameplay to advance. After the substep loop, however, if the phase is Debrief, the code adds the full `dt` to `g_mission.elapsed`. This allows breach/swallow/retreat visuals to continue behind the results UI. If a substep changes the phase to Debrief during the frame, the same post-loop addition still occurs; it is not prorated to only the leftover fraction of that frame.

The separate `simulationTime` always advances by `dt` inside the unpaused block, even on Title or Debrief. On Title, `g_wingPhase` additionally advances by `18 * dt`, keeping the preview aircraft's wings moving despite the mission helper not running flight gameplay. During normal flight the mission helper manages the wing phase. `updateParticles(dt, g_orbPos, harvesterPosition())` runs once per frame, not once per mission substep, with current aircraft and crawler positions.

Finally `curTime = simulationTime` repurposes the local variable. All later rendering in that frame uses the simulation animation clock, not the initial GLFW timestamp. Pausing freezes this clock, wing/title advancement, and particles, while the later render and audio call paths still execute. Resetting the mission does not reassign this local clock after its startup initialization, so visual animation phase can continue across retries even as mission elapsed resets. Minimization skips the block entirely rather than merely setting its `dt` to zero.

Camera System

Mission Revision and Keyboard Elevation

Original source: lines 1631-1643. The camera section compares its saved revision with `g_missionRevision`. Mission resets increment that global. On a mismatch, it updates the saved value, puts `camPosSmoothed` at aircraft position plus `(0, 14, 38)`, and sets `camTargetSmoothed` directly to the aircraft. This prevents the new mission camera target from easing all the way from an obsolete world location. It is a different initial height/offset from the first startup `(0, 26, 42)` values.

If not paused, Up raises `g_camOrbitPitch` by `40 * dt` degrees but caps it at 45; Down lowers it by the same rate but floors it at -10. These are separate ordered statements, not an `else if`. If both keys are held, both adjustments

run, and the bounds can affect the outcome near the limits. This block changes the camera's elevation-control variable, not the aircraft's physical pitch. While paused, held arrow keys do not advance that variable here.

Normal and Test Camera Targets

Original source: lines 1644-1654. The default target is aircraft position. `flightCameraOffset(g_camMode)` computes the current heading-relative offset, and the default desired position is aircraft plus that offset. The helper's actual conventions are relevant because source labels call mode one cockpit even though its implementation is a close third-person offset.

The helper forms behind = `-flightForward().Mode two is behind * 26 + (0, 100, 0)`. Other modes use a pitch angle derived from `clamp(12 - mouseHover.y * 22 + g_camOrbitPitch, 6, 65)` degrees. Mode one uses distance 20 and an additional 2.2 vertical offset; mode zero uses distance 40 with no such addition. The horizontal distance is `distance * cos(pitch)`, and the height is `distance * sin(pitch)` plus the mode-one addition. All three remain behind the aircraft in X/Z rather than becoming an actual first-person eye inside its hull.

Breach mode overrides the target to `harvesterPosition() + (0, 14, 0)` and desired position to `harvesterPosition() + (65, 48, 80)`. Pursuit mode, in an `else if`, aims at the midpoint of harvester and worm plus `(0, 6, 0)` and places its desired eye at that target plus `(65, 30, 155)`. If no specialized test camera is active, the helper's ordinary aircraft target and offset remain. The test overrides change desired position and target but do not overwrite the separately computed `camOffset`; the next stage handles their X/Z values explicitly.

Terrain Clearance and Heading-Preserving Smoothing

Original source: lines 1655-1669. First, desired camera Y is raised if necessary to at least `getDuneHeight(desiredX, desiredZ) + 1.5`. The target is not terrain-clamped by this statement. Camera height then eases toward desired height with coefficient `1 - exp(-9 * dt)`. Target position eases toward `camTarget` with the faster coefficient `1 - exp(-12 * dt)`.

Camera X and Z are not independently eased from their previous world positions. They are assigned `camTargetSmoothed.x + camOffset.x` and `camTargetSmoothed.z + camOffset.z`. Because the horizontal offset is exactly behind the aircraft's current heading, this preserves the view direction's horizontal alignment even when the target itself lags slightly in translation. Independently smoothing old world X/Z positions could leave the aircraft visibly side-on during turns; the source comments explain why this alternative is not used.

For breach or pursuit modes, X and Z are then assigned directly from `camDesiredPos` instead. Their height still uses the easing calculation, and their target still eases. A second terrain safety check clamps the final smoothed eye Y to at least terrain height at its actual X/Z plus 2. This second threshold is higher than the earlier 1.5 desired-position threshold and applies after horizontal reassignment, so it protects the actual rendered eye rather than only the requested one.

The smoothing calculations occur even while paused because the outer pause check only surrounds the keyboard elevation change, not this matrix preparation. With a positive measured `dt`, a paused camera can finish settling toward a stationary target. No terrain-based camera-target obstruction test, wall collision, or automatic zoom is implemented in this range.

Mouse-Test Heading Check and View/Projection Matrices

Original source: lines 1670-1682. In mouse smoke mode, `viewForward = camTargetSmoothed - camPosSmoothed` has its Y component set to zero. The code normalizes that horizontal direction and compares its dot product with `flightForward()` against 0.9999. A smaller value prints `Camera heading diverged from aircraft`, sets `renderFailed`, and requests window close. It does not immediately break the frame body, so rendering and final accounting can still continue for that iteration. There is no zero-vector normalization guard in this test, although the ordinary camera helper supplies a nonzero behind offset for its three modes.

`glm::lookAt(camPosSmoothed, camTargetSmoothed, (0, 1, 0))` creates the viewing matrix with world Y as up. The aspect ratio is `framebuffer width divided by framebuffer height as floats`. The projection is perspective with a 58-degree vertical field of view, that aspect ratio, near distance 0.4, and far distance 1200. These values are fixed in

the frame loop; none is changed by C or fullscreen except that a framebuffer resize changes aspect. The positive-size framebuffer callback ordinarily preserves a usable denominator during minimization, which is also skipped earlier in the loop.

Spatial Audio Handoff

All Eighteen Update Arguments

Original source: lines 1683-1704. The runtime first asks `nearestCrewPosition()` for a sound-interest position, then measures its horizontal distance from `g_ornPos`. The mission helper considers Running and Waiting crew and falls back to harvester position when no suitable closer candidate is found. That fallback means the returned point need not be a real crew member, and the caller sends it anyway. Distance is horizontal X/Z distance rather than a full camera-to-source or three-dimensional aircraft-to-person distance.

The single `g_audio.update` call occurs after camera calculation and before shadow rendering. Its arguments appear in the following exact order.

Position	Supplied expression	Meaning at the call boundary
1	<code>dt</code>	Current frame update interval; fixed to 1/60 for render tests.
2	<code>camPosSmoothed</code>	Listener eye position from the final terrain-safe camera.
3	<code>camTargetSmoothed</code>	Listener look target.
4	<code>g_ornPos</code>	Aircraft sound-source position.
5	<code>g_ornSpeed</code>	Current flight speed input.
6	<code>g_isBoosting</code>	Current boost boolean.
7	<code>harvesterPosition()</code>	Current harvester sound-source position.
8	<code>wormPosition()</code>	Current worm sound-source position.
9	<code>wormGap()</code>	Harvester-to-worm pursuit gap supplied by mission logic.
10	<code>wormAttackTime()</code>	Elapsed time relative to breach, negative before breach.
11	<code>g_mission.rescued</code>	Delivered-rescue count.
12	<code>g_mission.aboard</code>	Current onboard crew count.
13	<code>g_mission.pickup</code>	Pickup/winch progress.
14	<code>nearestCrewAudio</code>	Nearest eligible crew point or helper fallback.
15	<code>nearestCrewDistAudio</code>	Horizontal aircraft distance to that point.
16	<code>g_mission.base</code>	Base position for delivery-related audio.
17	<code>phase == Flying</code>	Whether gameplay is currently Flying.
18	<code>g_mission.paused</code>	Pause state, passed rather than suppressing the call.

Camera position and target make listener orientation match what the user sees. Mission counters and progress let the subsystem recognize transitions rather than merely play a uniform ambient loop. `wormGap()` is not the listener's distance to the worm; the separate worm position enables spatial positioning. The caller does not pass mute because the audio object already owns its muted field, which M toggles.

This call still happens on Title, Debrief, and paused rendered frames; `isFlying` and `isPaused` convey the correct phase. The audio implementation can return early when uninitialized or silence its master engine while paused/muted. Minimized frames do not reach it because the iconified branch continues earlier. The main source does not manually start every sound here or check an audio-update return status. The audio chapter describes volumes, pitch changes, one-shot detection, and asset loading in `src/arrakis_audio.h` rather than attributing those internal actions to this call site.

Shadow Rendering Pass

Depth Target and Player-Centered Light Camera

Original source: lines 1705-1721. The first render pass sets `g_isShadowPass = true`, changes the viewport to `SHADOW_RES x SHADOW_RES` (2048 x 2048 from the earlier constant), binds `g_shadowFBO`, and clears its depth buffer. It does not clear a color buffer because this target is used for shadow depth. The pass flag makes the shared mesh helper choose shadow-specific model/surface uniforms.

The shadow camera follows the aircraft's X/Z area, even in breach or pursuit smoke modes where the view camera may look elsewhere. `shadowCenter` starts at aircraft position but replaces Y with the dune height at the aircraft X/Z, so the light aims at ground under the player rather than the aircraft's altitude. `shadowBoxSize` is 90. The orthographic light projection spans `[-90, 90]` in both X and Y, with near/far distances 2 and 320, giving a 180-unit-wide square coverage area in light-view coordinates.

The light eye is `shadowCenter + sunDir * 120`. A look-at matrix points it at `shadowCenter` using world Y up. `lightSpaceMatrix = lightProj * lightView` transforms a world-space position into the light's projected space. There is no perspective narrowing or per-object light camera, and no second cascade for distant terrain in this range. Following the player prioritizes shadow detail near flight and rescue action rather than uniformly covering the entire 900-unit terrain extent.

Shadow Uniforms, Collapse Synchronization, and Bias

Original source: lines 1722-1736. The shadow shader becomes current. Its `lightSpaceMatrix` uniform receives one matrix, without transpose, using `glm::value_ptr`. The program then stores `wormCenter = wormPosition()` and `collapse = smoothstep(7, 14, wormAttackTime())`, sending those to shadow uniforms `wormCenter` and `collapse`.

The collapse scalar is zero through attack second seven, eases to one between seven and fourteen, and remains one afterward. It differs from the harvester sink interval of six to twelve. The preceding shader implementation uses these values for the affected terrain treatment. Computing them once here and reusing them in the scene pass keeps the shadow geometry's deformation parameters synchronized with visible terrain. `wormCenter` is a saved frame value used later for HUD coordinates as well.

The shader's terrain lowering applies only to `isSand == 1`. Rock materials marked 2 do not receive that terrain displacement. The scene shader lowers affected terrain vertices by `collapse * (1 - smoothstep(5, 48, distanceFromWormXZ)) * 13`, but the CPU `getDuneHeight` queries used for object placement, camera clearance, and HUD altitude still describe the original dune surface. The shader also retains the uploaded normal rather than recomputing a crater normal. Consequently the synchronized color/depth crater is a visual deformation, not a replacement CPU terrain or a general physical ground-collapse model. The separately sinking harvester gets its own explicit root transform.

The code enables `GL_POLYGON_OFFSET_FILL` and sets `glPolygonOffset(2.5, 4.0)` before drawing the scene. Polygon offset combines a slope-dependent contribution with a fixed depth-unit contribution, biasing filled depth polygons to reduce self-shadow acne. It does not change each model matrix or shift geometry in world coordinates. The mode is specifically fill-offset; the user-selected line rasterization mode is not forcibly replaced for this pass by these statements.

`drawFullScene(curTime)` submits terrain, any visible aircraft/harvester/tanks, rocks, rescue world, and worm to this shadow target using the current simulation time. Particles and the screen HUD are not included because they are

outside that traversal. The code disables polygon-offset fill afterward and binds framebuffer zero to return to the default window framebuffer. It does not delete or copy the shadow texture; the next pass samples the existing texture written by this pass.

Main Rendering Pass

Framebuffer Clear and Atmospheric Sky

Original source: lines 1737-1758. The main-pass banner begins by setting `g_isShadowPass = false` and restoring the viewport to the actual framebuffer dimensions. `glClearColor(fogColor.r, fogColor.g, fogColor.b, 1)` chooses the warm fog color with alpha one, and `glClear` clears both color and depth. This resets the main render surface independently of the shadow map's depth buffer.

For the sky, face culling is disabled and depth writes are disabled with `glDepthMask(GL_FALSE)`. Depth testing itself remains enabled from startup. The sky program receives `lightDir = sunDir`, `sunColor`, and `invViewProj = inverse(proj * view)`. The inverse transforms projected sample directions into the view/world representation used by the sky shader; no sky cube, external panorama texture, or vehicle model is loaded by this stage.

The runtime binds `g_skyVAO` and issues `glDrawArrays(GL_TRIANGLES, 0, 3)`, a three-vertex screen-filling triangle. The shader generates the desert atmosphere described in the shader chapter. Afterward, depth writes are reenabled and face culling is enabled again. Those are explicit known-state settings, not restoration of queried prior values. The sky is only drawn here, not into the shadow map. If global wireframe mode is active, this range does not force filled sky polygons; filled-mode forcing is reserved for the HUD later.

Scene Matrices, Lighting/Fog Uniforms, and Shadow Texture

Original source: lines 1759-1780. The scene program becomes current. All frame-wide uniforms are assigned before the shared object traversal supplies each model/material.

Uniform	Upload value	Type and count
<code>view</code>	Camera view matrix	One mat4, no transpose.
<code>projection</code>	Perspective proj matrix	One mat4, no transpose.
<code>lightSpaceMatrix</code>	Current shadow projection/view product	One mat4, no transpose.
<code>lightDir</code>	Normalized $(-0.35, 0.12, -1)$	One vec3.
<code>sunColor</code>	$(1.15, 1.04, 0.85)$	One vec3.
<code>ambientColor</code>	$(0.34, 0.38, 0.44)$	One vec3.
<code>viewPos</code>	Final <code>camPosSmoothed</code>	One vec3.
<code>fogColor</code>	$(0.90, 0.74, 0.47)$	One vec3.
<code>fogDensity</code>	0.0022	One float.
<code>wormCenter</code>	Position saved during the shadow pass	One vec3.
<code>collapse</code>	The same attack-time smoothstep scalar	One float.
<code>shadowMap</code>	Texture unit index 0	One integer sampler selection.

Each location is looked up from `g_progScene` at the call site using `glGetUniformLocation`; these frame-wide locations are not cached by this block. Matrix and vector pointers come from `glm::value_ptr`. The usual OpenGL behavior for location -1 is an ignored uniform write; this code does not separately assert the presence of each named uniform. The material helper adds model transform and surface-specific uniforms for every mesh.

Before `drawFullScene`, the runtime activates `GL_TEXTURE0`, binds `g_shadowTex` as `GL_TEXTURE_2D`, and assigns sampler `shadowMap` to integer zero. The sampler receives a texture-unit number, not the texture object's numeric ID.

The shared scene draw then reuses the same visible-object conditions and transforms as the depth pass, now shaded with light, ambient color, shadows, material properties, and fog. The block does not update mission state between passes, so drawing twice does not move the harvester or advance rescue progress twice.

Particle Blend State and Billboard Camera Basis

Original source: lines 1781-1796. The particle stage enables blending, chooses `GL_SRC_ALPHA` and `GL_ONE_MINUS_SRC_ALPHA`, disables depth writes, and disables culling. Depth testing remains on, so dust can be hidden behind already drawn solid geometry while avoiding opaque depth marks that would incorrectly block later transparent particles.

The particle program receives view and projection. Camera-right is extracted as `(view[0][0], view[1][0], view[2][0])`, and camera-up as `(view[0][1], view[1][1], view[2][1])`. GLM indexes a matrix by column and then row, so these expressions gather the desired view-basis rows instead of merely taking one entire stored column. They are uploaded to uniforms `camRight` and `camUp` to orient quad billboards toward the camera.

This stage uses one mesh and per-instance data rather than invoking `drawPart` for each grain of dust. It does not sample the shadow texture through a scene-material draw, create new CPU particle motion, or emit additional particles here. Particle updates already occurred once in the earlier unpaused simulation block.

Particle Fades, Sorting, Buffer Upload, and Instanced Draw

Original source: lines 1797-1814. The CPU instance vector is cleared every frame without discarding its reserved capacity. For every `p` in `g_particles`, it computes `fade = clamp(p.life / 0.5, 0, 1) * clamp((p.maxLife - p.life) / 0.3, 0, 1)`. Since `life` is remaining life, the first factor fades out during the last half-second. The second uses elapsed age, fading in during the first 0.3 seconds. The product limits both ends of the lifespan smoothly through linear clamped factors.

Each push stores `posSize = vec4(p.pos, p.size)` and `colorAlpha = vec4(COL_DUNE_SAND, p.alpha * fade)`. All particles share the dune-sand RGB (0.76, 0.49, 0.23) at this call site; their category-dependent differences are position, size, base alpha, life, and velocity from the particle simulation. The loop pushes every entry, including one whose resulting alpha is zero, rather than culling fully transparent entries here.

The vector is sorted with a comparator that subtracts camera position from each instance position and compares squared distances via `dot(delta, delta)`. A larger distance sorts before a smaller one, giving far-to-near order for alpha compositing. Squared distance avoids a square root without changing distance ordering. It is radial world-space distance, not exact projected eye-depth ordering, and `std::sort` does not promise stable order for equal-distance entries.

The program binds the particle instance VBO and replaces its data store with exactly `instances.size() * sizeof(ParticleInstance)` bytes using `GL_STREAM_DRAW`. This is a new `glBufferData` upload rather than a per-particle `glBufferSubData` series; the driver receives the whole sorted frame's instance array. The quad VAO is then bound and `glDrawElementsInstanced` uses triangle topology, the quad's index count, unsigned-int indices at offset null, and the vector size converted to `GLsizei` as instance count. Attribute divisors configured at startup provide one position/size and color/alpha record per quad instance.

After drawing, the runtime restores depth writes, disables blending, and enables culling. Those changes prepare the known scene state for HUD handling and the next frame. No glow accumulation buffer or post-processing smoke compositor is present. Here smoke-like plumes are ordinary sand-colored blended billboards; the term smoke test later refers to automated render execution and is not this dust particle system.

HUD State Assembly

Crew Counts, Markers, and Basic Metadata

Original source: lines 1815-1827. A new `RescueHudState` `hud` is constructed every frame, using its header-defined default member values. Width and height become current framebuffer dimensions. Rescued, aboard, lost, best, and wave are copied from the mission, and muted is copied from `g_audio.muted`. The latter displays the mute toggle state, not whether an audio device initialized successfully.

`hud.inside` is explicitly reset to zero because its default in the HUD structure is 36. The crew loop increments it for each `Inside` member. For `Running` or `Waiting`, it increments `hud.waiting` and appends (`c.pos.x`, `c.pos.z`) to `crewMarkers`. The new HUD object's waiting counter already starts at zero and marker vector starts empty, so no explicit clearing statement is needed. Other crew states are not placed on this marker list.

These markers use each crew member's stored gameplay position, not the temporary lifted Y position in `drawRescueWorld`; they are two-dimensional map points, so their Y would not be used anyway. The HUD struct's total-crew value remains its default 36 because the main loop does not assign it in this range. The loop counts actual states each frame instead of deriving all numbers by subtracting rescued/lost/aboard from that total.

Timers, Flight Values, Phase Flags, and Map Coordinates

Original source: lines 1828-1846. Every following assignment connects an explicit gameplay or runtime value to the display state.

HUD field	Supplied value	Exact interpretation
<code>missionDuration</code>	<code>g_mission.breachAt</code>	Current wave's planned pre-breach duration.
<code>pursuit</code>	<code>clamp(elapsed / breachAt, 0, 1)</code>	Normalized chase progress, saturated after breach.
<code>wormDistance</code>	<code>wormGap()</code>	The pursuit gap, not camera distance.
<code>extractionRemaining</code>	<code>max(0, 60 - wormAttackTime())</code>	Time relative to attack second sixty; before breach it can exceed sixty.
<code>releasedGroups</code>	Count of all groups with <code>released true</code>	Historical release count, regardless of whether their marker remains visible.
<code>remaining</code>	<code>max(0, breachAt - elapsed)</code>	Nonnegative pre-breach countdown.
<code>altitude</code>	<code>aircraftY - getDuneHeight(aircraftX, aircraftZ)</code>	Actual aircraft terrain clearance, not merely the mission's target-altitude field.
<code>speed</code>	<code>g_ornSpeed</code>	Current flight speed.
<code>boost</code>	<code>g_mission.boost</code>	Current boost reserve.
<code>pickupProgress</code>	<code>g_mission.pickup</code>	Winch progress.
<code>unloadProgress</code>	<code>g_mission.unload</code>	Base unloading progress.
<code>fps</code>	Smoothed local fps	Runtime's damped frame-rate estimate.
<code>prompt</code>	<code>g_mission.prompt</code>	Mission-generated contextual text.
<code>heading</code>	<code>g_ornYaw</code>	Aircraft yaw in the game's degree convention.
<code>title</code>	<code>phase == Title</code>	Enables the HUD's title presentation.
<code>ended</code>	<code>phase == Debrief</code>	Enables results presentation.

HUD field	Supplied value	Exact interpretation
paused	<code>g_mission.paused</code>	Current pause display flag.
player	<code>(g_ornPos.x, g_ornPos.z)</code>	Aircraft map coordinates.
base	<code>(g_mission.base.x, g_mission.base.z)</code>	Base map coordinates.
harvester	<code>(g_harvX, g_harvZ)</code>	Current crawler map coordinates, even if hidden after swallowing.
worm	<code>(wormCenter.x, wormCenter.z)</code>	Position cached earlier in this frame.
pickup	X/Z of a fresh <code>nearestCrewPosition()</code> result	Nearest eligible crew or the helper's harvester fallback.
attackTime	<code>wormAttackTime()</code>	Signed elapsed time relative to breach.

`releasedGroups` is explicitly initialized to zero before counting. The loop does not test whether a released group still has waiting survivors; that filtering is specific to world marker visibility. Before breach, attack time is negative, so `60 - attackTime` exceeds sixty. This caller applies only a lower clamp to extraction remaining, not an upper clamp to sixty. The HUD renderer may format or selectively display values, but the passed field is exactly this expression.

There is no division-by-zero check around `eLapsed / breachAt` in the caller. Normal mission reset supplies a positive duration. Altitude also receives no lower clamp here, so if aircraft Y were below the mathematical terrain height, the raw clearance would be negative. The display renderer's formatting rules are separate from these assignments.

Filled HUD and Restored Wireframe Selection

Original source: lines 1847-1850. Immediately before `drawRescueHud(hud)`, the main loop sets both front and back polygon modes to `GL_FILL`. This ensures text, panels, and HUD geometry stay solid even when F selected wireframe for world geometry. It then restores front/back mode to `GL_LINE` or `GL_FILL` according to `g_wireframe`.

The HUD renderer internally manages its required OpenGL state as described in its own chapter. These three caller statements add a specific rasterization guarantee around that draw. They do not switch the scene back to normal mode permanently or alter `g_wireframe` itself. The HUD is last in the normal rendering order, after sky, solids, and dust, so the later screenshot call includes its final visible result.

Render Smoke Tests and Frame Presentation

Error Sampling, Last-Frame Screenshot, and Swap

Original source: lines 1851-1866. The section banner mentions buffer swap and event polling, but the actual polling operation already occurred near the beginning of the frame. This tail block performs test checks and swaps; it does not contain another `glfwPollEvents()`.

Only render-test modes call `glGetError()` here. If the returned enum is not `GL_NO_ERROR`, standard error receives OpenGL error in frame `<frameCount>: <error>`, `renderFailed` becomes true, and the window is marked for closing. A single `glGetError` call retrieves one error value; this block does not drain a queue of multiple errors or turn each OpenGL operation into a checked call. It also does not compare screenshot pixels, validate object visibility, or impose a frame-time limit. An error can cause an early exit rather than completing all 120 planned frames.

If `frameCount == 119` and `argc > 2`, it calls `captureScreenshot(argv[2])`. Thanks to left-to-right short-circuit evaluation, screenshot capture is not attempted before the final planned frame, without a second argument, or in normal interactive mode. Since the counter begins at zero, frame index 119 is the 120th rendered test frame. A failed capture prints `Screenshot write failed` and sets `renderFailed`; it does not add a separate immediate close call, because the frame limit closes the test afterward anyway. The screenshot is taken before swapping, after the HUD draw, while the default framebuffer is still the current render target.

`glfwSwapBuffers(g_window)` presents the completed frame in both normal and test modes. In render-test mode, `++frameCount` is then evaluated and a count at least 120 sets `should-close` true. In non-test mode, short-circuit evaluation of `renderTest && ...` means `frameCount` is not incremented by this statement at all. A heading or GL error that already requested closure still allows this swap and test increment for the current body. If the user closes a smoke-test window early, the loop exits with fewer frames; no separate assertion here changes an otherwise false `renderFailed` solely because the count is less than 120.

With an uninterrupted test, the fixed 1/60 update interval gives approximately two simulated seconds over 120 rendered frames, in addition to breach/pursuit pre-advance where applicable. The actual wall-clock duration depends on rendering, swap synchronization, driver behavior, and any event waits. No timer-based retry or unattended restart is present.

What the Smoke Test Result Does and Does Not Mean

Original source context: lines 1449-1454, 1564-1570, 1607-1617, 1670-1677, and 1854-1865. The smoke modes exercise real graphics initialization and rendering rather than merely unit-testing state functions. All run the ordinary geometry, both render passes, particles, HUD assembly, and audio calls. Mouse mode additionally checks horizontal view alignment against the aircraft. Breach/pursuit modes select their requested mission times and viewpoints so those scenes are rendered during the bounded run.

A reported pass means `renderFailed` remained false under the checks this program actually performs. It is not proof that every pixel is correct, all requested assets played, no earlier unchecked startup assumption failed, or a user-closed test reached exactly 120 frames. Screenshot success means the file stream accepted the bitmap writes, not that the bitmap contents were compared with a reference image. These distinctions preserve the existing implementation's actual test semantics without claiming hypothetical visual-regression coverage.

Resource Teardown and Exit

Programs, Targets, Renderer Resources, and Mesh Buffers

Original source: lines 1868-1885. After the loop ends normally, cleanup occurs while the window's context still exists. The four main shader programs are deleted in scene, shadow, particle, and sky order. The shadow framebuffer and depth texture are deleted. The particle instance VBO, sky VBO, and sky VAO are deleted next.

`cleanupSandwormMeshes()` releases the worm renderer's resources, and `cleanupRescueHud()` releases HUD resources. Those helpers own resources distinct from the eight basic globals in the following loop. No draw is attempted after these cleanup calls.

The range-based loop then iterates pointers to `g_meshCube`, `g_meshCylinder`, `g_meshSphere`, `g_meshWing`, `g_meshTerrain`, `g_meshQuad`, `g_meshRing`, and `g_meshRock`. For each, it deletes the VAO, VBO, and EBO using that object's stored IDs. A VAO describes attribute/index bindings, while the VBO and EBO hold vertex and index data; deleting only the VAO would not release those buffer objects, so all three deletions are explicit. The order is VAO first, vertex buffer second, element buffer third.

The cleanup does not manually clear the CPU particle vectors, tank vector, or mission crew containers. Their ordinary C++ lifetime ends with program termination. It also does not reset every deleted global ID to zero in this loop because no new render cycle follows. Startup failures that returned before reaching the loop do not run this bottom-of-main sequence, as documented with those branches.

Audio Shutdown, Window Destruction, Timing Output, and Return

Original source: lines 1886-1892. `g_audio.shutdown()` runs after the graphics-resource deletions. The window is then explicitly destroyed. In render-test mode, standard output prints `Render smoke test:` followed by `FAILED` if `renderFailed` is true or `PASS` otherwise, then the frame count and `(glfwGetTime() - smokeStarted)` seconds.

The time is queried after window destruction but before `glfwTerminate`, while GLFW timing remains available. Because `smokeStarted` was taken after setup and fast-forward, this duration measures the render-loop interval plus its normal resource cleanup, audio shutdown, and window destruction, not full process startup. The frame count is the

number incremented at the end of rendered test frames; iconified waits do not increment it.

Finally `glfwTerminate()` shuts down GLFW, and `main` returns 1 for a recorded render failure or 0 otherwise. Normal interactive execution ordinarily leaves `renderFailed` false because the failure-setting render checks are guarded by test modes, so its normal exit returns zero. The earlier `--test-game` return, GLFW/window/GLAD -1 failures, and shader/HUD startup failure 1 remain separate exit paths. The final brace at original line 1892 ends `main` and the inspected source file.

Coverage Ledger

Original Lines Accounted For

This ledger is an index to the detailed explanations above. Blank separator lines have no executable behavior. Every nonblank line in the requested original interval is covered, including descriptive banners, includes, declarations, assignments, branches, loops, calls, lambda bodies, return statements, and closing braces. Line references are deliberately original-source references rather than promises about numbering after relocation.

Original range	Covered source responsibility
968-1001	Ornithopter banner, signature, inputs, six materials, boost emission selection.
1002-1018	Central pod, nose, probe, canopy/frame, paired intakes.
1019-1035	Four gimbals, turbine bodies/nozzles, conditional exhaust length and both exhaust draws.
1036-1048	Three tail sections, paired canted fins, paired landing skids.
1049-1081	Wing banner, waveforms, five-field definition, all eight blade entries.
1082-1101	Wing iteration, ordered matrices, twist, mirrored span, culling, draw, function end.
1103-1119	Harvester banner/signature, six materials, hopper and upper deck.
1120-1139	Track offsets, nested unit loops, housing, wheel/bar loop, ten-pad ellipse and rotations.
1141-1153	Cutter, door material, door, ramp, luminous scoop, two separators.
1154-1168	Bridge, viewport, floodlights, exhaust stacks/rims, harvester function end.
1170-1192	Tank banner/signature, four materials, skids/crosspieces, two-station support loop.
1193-1210	Vessel cylinder, two full spherical heads, bands, hatch, valve, sight indicator, function end.
1212-1225	Rock banner/signature, strata materials/category, four transformed rock meshes, function end.
1227-1245	Worm/HUD includes, rescue signature/materials, base root, pad/ring/cross, four beacons.
1246-1261	Group eligibility, member counting, terrain-normal alignment, amber ring/pole/light.
1262-1278	Crew eligibility, visual pickup lift, stride selection, complete body and locator draws.
1279-1293	Rope eligibility/endpoints/length/orientation, cyan cable draw, worm arguments, function end.

Original range	Covered source responsibility
1295-1316	Master banner/signature, terrain material/draw, aircraft loss branch and attitude matrix.
1317-1333	Harvester visibility boundary, sinking, heading-relative slope rotations, swallowing tilt/draw.
1334-1354	Tank boundary and roots, eight terrain-embedded rock sites, rescue traversal, function end.
1356-1374	Callback banner, positive framebuffer resize, logical cursor recentering/reset.
1375-1387	Cursor eligibility, early reset/return, logical dimensions and hover helper.
1388-1427	Key bounds/press/release, Escape/F/C/Enter/R/M/P/F11, fullscreen branch, function end.
1429-1439	Entry banner, screenshot allocation/readback/open check, little-endian lambda.
1440-1446	Complete BMP header, raw pixels, stream status and screenshot function end.
1448-1459	Main signature, mission-test early return, four render flags, GLFW failure.
1460-1485	GL version/profile and monitor hints, multisampling, first window and fallback/error branch.
1486-1505	Current context/framebuffer size, named callbacks, three lambda callbacks, VSync.
1506-1526	GLAD failure path, audio startup, diagnostic output, initial depth/MSAA/cull state.
1527-1543	Eight reusable meshes, worm mesh initialization, four shader-program builds.
1544-1563	Shadow/sky/HUD setup, startup shader failure, particles/VBO/attributes/divisors/reserve/reset.
1564-1578	Breach/pursuit fixed-substep pre-advance, aircraft placement, sun/ambient/fog constants.
1579-1593	Initial timing, smoothed camera/animation/FPS state, revision/failure/count/timer, loop banner.
1594-1606	Loop condition, iconified wait/continue, wall-time delta/cap/copy, event polling.
1607-1618	Render-test fixed dt/unpause, synthetic mouse/W/camera modes, FPS filtering.
1619-1630	Bounded mission updates, debrief clock, animation time, title wings, particles, clock handoff.
1631-1643	Camera banner, revision reset, pause-gated Up/Down elevation bounds.
1644-1654	Aircraft target/helper offset, breach and pursuit override branches.
1655-1669	Desired terrain safety, height/target smoothing, heading-locked X/Z, test X/Z, final clearance.
1670-1682	Mouse heading invariant/error/close, view, framebuffer aspect, perspective projection.

Original range	Covered source responsibility
1683-1704	Nearest crew/distance calculation, all eighteen spatial-audio arguments.
1705-1721	Shadow banner/flag/viewport/FBO/depth clear, player-ground center, ortho and light matrices.
1722-1736	Shadow program/uniforms, worm/collapse, polygon bias, complete scene, target unbind.
1737-1758	Main banner/flag/viewport/clear, sky states/uniforms/inverse matrix/triangle/state restoration.
1759-1780	Scene program and all frame uniforms, unit-zero shadow texture, complete scene traversal.
1781-1796	Particle blend/depth/cull state, matrices, camera-right/up uniforms.
1797-1814	Instance clearing/fade/packing/sorting, stream upload/instanced quad draw, state reset.
1815-1827	Fresh HUD, size/counters/mute, inside and outside crew counting/map markers.
1828-1846	HUD timers/progress/gap/groups/flight data/text/flags/map coordinates/attack time.
1847-1850	Filled HUD draw and user wireframe mode restoration.
1851-1866	Presentation banner, render GL error, optional last-frame BMP, swap/count/close, loop end.
1868-1885	Four programs, shadow objects, particle/sky resources, worm/HUD cleanup, eight mesh deletions.
1886-1892	Audio shutdown, window destruction, test result/time output, GLFW termination, return/file end.

The requested interval ends at the end of the original file; there is no further scene or runtime code after original line 1892. This chapter explains that interval without modifying source behavior, adding features, or duplicating the final full-source appendix.

Chapter 4: Mission Rules, Flight Controls, and Tests

This chapter explains the complete gameplay header and the complete mission-test header in beginner-friendly terms. The original files were `arrakis_game.h` and `arrakis_tests.h` at the project root when this chapter was researched. In the reorganized project their locations are `src/arrakis_game.h` and `tests/arrakis_tests.h`. Every source range in a heading below refers to the original header, not to this Markdown file. The numbered source appendices provide the exact text; the explanations here describe what that text does rather than replacing explanations with a copied listing.

The gameplay header has 307 original lines. The test header has 1,041 original lines. The ranges below collectively account for every nonblank line in both. Opening and closing braces belong to the constructs explained in their ranges; empty separator lines have no runtime behavior. Both headers depend on types, variables, libraries, and functions already introduced by the translation unit that includes them. They are not independently compilable programs.

Reading the Mission

The player controls an ornithopter, a fictional aircraft with flapping wings. A moving harvester carries 36 people. Those people leave in timed groups, run to fixed gathering points, and wait for pickup. Holding Space near people, while low and slow enough, winches them into an eight-person cabin. Holding Space near the base unloads the whole cabin after a separate dwell interval. Only delivered people contribute to the rescue score.

A pursuing worm approaches the moving harvester. Its danger rules depend on elapsed mission time, the time of breach, horizontal distance, and, for the aircraft, height. Crew aboard the aircraft are protected from ordinary local crew hazards but not from an aircraft collision or the final timeout. Crew already delivered remain safe even if the aircraft is subsequently lost.

Here, a `glm::vec3` is a three-component vector with *x*, *y*, and *z*. The world uses *y* for height and the *x*, *z* plane for ground positions. A `glm::vec2` holds two components. For the mouse these are horizontal and vertical normalized input; for distance checks they usually stand for world *x*, *z*. Distances are expressed in world units, treated as meters by the test messages. Times are seconds, yaw and camera pitch are stored in degrees, and trigonometric functions receive radians.

The notation *dt* means the amount of simulated time represented by one update. A frame at 120 updates per second has a *dt* near 1/120. Multiplying a speed by *dt* gives the movement for that update. This is essential to avoid rules that change simply because the program draws more frames, although not every integration in this file is exactly frame-rate independent.

Several small mathematical operations recur throughout the chapter. `clamp(value, minimum, maximum)` returns the minimum when *value* is below it, the maximum when above it, and *value* otherwise. `min` chooses the lesser of two inputs and `max` the greater. `abs` measures magnitude without sign. A vector's length is its Euclidean magnitude; normalizing a nonzero vector divides it by that length to preserve direction while making its length one. The code checks for zero before normalizing movement input.

`mix(a, b, w)` means $a * (1 - w) + b * w$. With weight zero it returns *a*; with weight one it returns *b*. The same operation works on each component of a vector. Here most smoothing weights are $1 - \exp(-k * dt)$, where `exp` is the exponential function and *k* is a positive response rate. Over a constant-target interval *t*, the fraction of the original error remaining is $\exp(-k * t)$. The characteristic time is 1/*k*: approximately 0.125 seconds for rate eight, 0.167 for six, 0.182 for 5.5, 0.083 for 12, and 0.25 for four. A smaller characteristic time means faster response, not a fixed delay after which the value suddenly snaps to its target.

A dot product multiplies corresponding vector components and sums them. For unit directions it equals the cosine of the angle between them, so a value near one means nearly identical directions and a value near minus one means opposite directions. A cross product returns a perpendicular vector with an order-dependent sign. Taking forward cross world-up gives aircraft-right in this project's coordinate convention. Converting degrees to radians multiplies by $\pi/180$; converting radians to degrees multiplies by $180/\pi$.

In C++, `if` chooses whether a statement or block runs, `else` selects an alternative, `continue` skips the remainder of the current loop iteration, and `return` leaves a function. Conditions joined by `&&` require both operands, while `||` requires either; they short-circuit, so the second operand need not be evaluated once the result is already known. Prefix `++` and `--` change a counter by one. The references used by range-based loops operate on the original objects, while constant references allow reading without changing them. These details explain why the update can mutate every person's state in place and why comparison loops can stop safely when a mismatch is found.

Game Types and Storage

Game Lines 1-4: Inclusion and State Enumerations

Line 1 uses `#pragma once` to avoid processing this header more than once within a translation unit. This does not make its globals or ordinary function definitions safe to duplicate across many independently compiled translation units. The project includes them as part of its existing compilation structure.

`MissionPhase` is a scoped enumeration with `Title`, `Flying`, and `Debrief`. With no explicit assigned values, their underlying values begin at zero in declaration order, but the code uses their names rather than relying on numeric values. `Title` is the pre-mission state, `Flying` enables simulation, and `Debrief` is the finished state. A pause is not a fourth phase: it is a separate Boolean within a mission.

`CrewState` has six scoped values, likewise in declaration order: `Inside`, `Running`, `Waiting`, `Aboard`, `Safe`, and `Lost`. `Inside` means still attached to the harvester's evacuation exit. `Running` means moving to a scheduled destination. `Waiting` means standing at that destination. `Aboard` means counted in the aircraft cabin. `Safe` means unloaded and scored. `Lost` means no longer rescuable. These are mutually exclusive because each person stores one enumeration value, not several independent flags.

The usual successful path is `Inside -> Running -> Waiting -> Aboard -> Safe`, but pickup also accepts `Running`, so waiting is not compulsory. Local hazards can turn `Running` or `Waiting` into `Lost`; swallowing can turn `Inside` into `Lost`; aircraft loss or timeout converts every state other than `Safe` and already `Lost` into `Lost`. A mission reset creates an entirely new population rather than reversing those transitions on existing people.

Game Lines 5-11: Every Evacuee Field

`Evacuee` represents one crew member. `pos{0}` initializes all three position components to zero, and `destination{0}` does the same for the target position. These are world-space values, not coordinates relative to a moving harvester. `state` initially equals `CrewState::Inside`.

`release` is a floating-point scheduled release time, initially zero. `group` is an integer index identifying the person's evacuation group, initially zero. `slot` is an integer identifying their position within that group, also initially zero. The declaration places `group` and `slot` on one line but initializes each separately. A freshly constructed person therefore has a valid default state, but mission setup must assign their real route position, release schedule, group, and slot before gameplay.

The person has no individual health, score, velocity, animation timer, or inventory. Walking speed is supplied by the update function. The group index selects which release loop will activate the person, while the slot determines the spacing within the gathering cluster. The duplicate release value on the person is used to calculate how much walking time is available when a frame crosses the release threshold.

Game Lines 12-17: Every Evacuation Group Field

`EvacuationGroup` stores `count`, initially zero, for the number of people assigned to the group. `release`, initially zero, records the group's scheduled release time. `center{0}` is a three-component world-space cluster center, initially the origin. `released`, initially `false`, prevents the release operation from being repeated in later updates.

Before release, `reset` stores the initial harvester position in `center`; this is temporary. At release, the code replaces it with the actual scheduled gathering point. The group has no list of member pointers. Instead, the update scans the mission's crew vector and selects people whose group index matches the group being released.

Game Lines 18-32: Every Mission Field and Default

RescueMission contains the state of one attempt. `phase` initially equals `Title`. `paused`, `planeLost`, and `boostLocked` are initially `false`. `paused` stops the update before any simulation work. `planeLost` records a worm collision with the aircraft. `boostLocked` prevents held Shift from immediately reactivating boost after energy runs low.

The integer fields are `wave = 1`, `rescued = 0`, `aboard = 0`, `lost = 0`, and `best = 0`. `wave` affects the breach timer, not crew count or cabin capacity. `rescued` counts delivered people. `aboard` counts current cabin occupants. `lost` counts people marked lost. `best` is the highest delivered total retained across resets in the current process. This header does not save that record to disk or calculate a separate cumulative score across waves.

The floating-point fields are `elapsed = 0`, `breachAt = 210`, `altitude = 8`, and `boost = 1`. `elapsed` is the attempt's simulation clock. `breachAt` is an absolute elapsed-time threshold. `altitude` is desired terrain-relative height, not necessarily the aircraft's instantaneous measured height. `boost` is a normalized resource from zero to one.

`harvesterYaw = 0` and `harvesterTravel = 0` describe orientation and accumulated horizontal travel. `wormYaw = 90` is a stored fixed orientation; no function in this header changes it. `pickup = 0` and `unload = 0` are normalized dwell progress. Their completion threshold is one. `target = -1` means no current pickup person; nonnegative values are indices into crew.

`prompt` starts as an empty `std::string` and is replaced during active updates with instruction text. `base{55, 0, 55}` gives the base's horizontal location; reset later fills in its terrain height. `crew` and `groups` are initially empty standard-library vectors. Their sizes and contents are populated by reset.

These defaults are significant because `reset` replaces the entire mission object with `RescueMission{}`. Any field that is not explicitly restored afterward returns to exactly these values. For example, pause, plane loss, boost lock, prompts, targets, travel, and old populations disappear, while `best` and `wave` are specially preserved or advanced.

Game Lines 33-39: Every Global and Rock Site

`g_mission` is the global mission object and begins with the defaults above. `g_missionRevision` starts at zero and increments whenever `resetMission` is called. It can let other systems recognize a new attempt; this file does not otherwise read it. `g_mouseHover(0)` begins with both mouse-input axes zero. `g_mouseTurnRate = 0` begins with no stored angular inertia.

`RESCUE_CAPACITY` is a compile-time integer constant of eight. It is the cabin limit used during boarding and by several tests. It is not an adjustable per-wave mission field.

`g_rockSites` is a fixed array of eight horizontal obstacle centers: (45, -65), (-55, 40), (90, 30), (-90, -70), (15, 95), (-25, -115), (120, -25), and (-130, 85). The pairs represent x, z, not x, y. The flight update uses these centers to increase terrain clearance nearby. It does not calculate collisions from rendered rock triangles or assign each rock a separate height.

Globals such as `g_ornPos`, `g_ornVel`, `g_ornYaw`, `g_keys`, `g_harvX`, `g_harvZ`, `g_audio`, and `g_tanks` are used but not declared here. Their definitions and, where relevant, their callbacks and rendering uses belong to other chapters. The distinction matters: this header supplies the rules, not the entire window/input/audio implementation.

World and Input Helpers

Game Lines 41-46: Horizontal Distance and Current Harvester Position

`horizontalDistance(a, b)` subtracts the two points' x and z components, forms a two-component vector, and returns its length. Mathematically this is $\sqrt{(a.x - b.x)^2 + (a.z - b.z)^2}$. Height differences are deliberately ignored. Pickup, base arrival, and worm proximity therefore have separate horizontal and vertical requirements rather than a single spherical distance check.

`harvesterPosition()` constructs a position from the external `g_harvX` and `g_harvZ`. Its height is freshly queried from `getDuneHeight` at those coordinates. The height is not separately stored in a harvester global. If the terrain function changes, this helper reflects that change on its next call.

Game Lines 47-60: Analytic Route, Tangent Yaw, and Forward Direction

`harvesterRoute(time)` calculates an absolute route position directly from a time value. The horizontal formulas are $x = -40 + 0.62 \cdot \text{time} + 15 \cdot \sin(0.045 \cdot \text{time})$ and $z = -55 + 22 \cdot \sin(0.032 \cdot \text{time}) + 9 \cdot \sin(0.071 \cdot \text{time})$. At zero time the harvester is at (-40, -55) horizontally. The linear term generally carries it east, while three sine terms create the wandering path. Height is the terrain height at the calculated position.

These are analytic coordinates: the function does not move a previous position incrementally and does not modify mission state. Asking for time six always gives the same result, regardless of the frame that noticed a release. The helper itself does not clamp time; clamping happens in `updateHarvester`. Scheduled release code deliberately calls this raw route at the release's own time.

`harvesterRouteYaw(time)` derives the horizontal tangent. Differentiating the x formula gives $dx = 0.62 + 0.675 \cdot \cos(0.045 \cdot \text{time})$ because $15 \cdot 0.045 = 0.675$. Differentiating the z formula gives $dz = 0.704 \cdot \cos(0.032 \cdot \text{time}) + 0.639 \cdot \cos(0.071 \cdot \text{time})$ because $22 \cdot 0.032 = 0.704$ and $9 \cdot 0.071 = 0.639$.

The returned yaw is the degree conversion of $\text{atan2}(-dx, -dz)$. This sign and argument order align a model whose local forward direction is negative Z with the tangent. `atan2` accounts for quadrants rather than dividing one derivative by the other. This is a horizontal heading; it does not tilt the harvester to match the vertical terrain slope.

`harvesterForward(yaw)` converts degrees to radians and returns $(-\sin(\text{angle}), 0, -\cos(\text{angle}))$. It is a horizontal unit direction. At zero yaw it points toward negative Z. This same convention appears in aircraft flight, although `wormYaw` is a separate stored model-orientation value and should not be interpreted as an aircraft heading calculation.

Game Lines 61-78: Harvester Updates and Worm Geometry

`updateHarvester(elapsed)` first clamps the supplied elapsed time into $[0, \text{breachAt}]$. Negative times select the beginning; times after breach select the breach pose. It reads the previous current position, evaluates the next route position, and adds the horizontal distance between them to `harvesterTravel`. It then writes the new external x/z globals and the route tangent yaw.

Accumulated travel is a sum of straight chords between sampled positions, not an exact integral of curved-route arc length. Its value can differ slightly with update frequency. Calling the helper with nonmonotonic times can also add backward travel; this helper does not reject such calls. Once repeated calls clamp to the same breach pose, the added distance is zero and the pose and accumulated travel stop changing.

`wormGap()` calculates $\text{progress} = \text{clamp}(\text{elapsed}/\text{breachAt}, 0, 1)$ and linearly interpolates from 220 to 18. Equivalently, the gap is $220 - 202 \cdot \text{progress}$. During a normal first wave it closes by $202/210$, approximately 0.962 world units per second. Later waves reach the same endpoint sooner. After breach the gap remains 18. The implementation assumes a positive nonzero breach time, which normal reset guarantees.

`wormPosition()` begins with the current harvester position, subtracts the gap from x, leaves z unchanged, and recomputes terrain height at the worm's new x/z. Thus the worm is always west of the harvester, not necessarily behind the harvester's current tangent direction. It does not independently steer toward a person or aircraft. Its motion follows this geometric relationship.

`wormAttackTime()` returns $\text{elapsed} - \text{breachAt}$. A negative answer means before breach, zero means the exact breach instant, and a positive answer measures seconds since breach. Most hazard windows below are expressed in this relative time, so shortening a wave automatically shifts them earlier in absolute mission time.

Game Lines 80-89: Nearest Crew Position

`nearestCrewPosition()` initializes a search radius of 10,000 and a fallback point equal to the current harvester position. It loops over every person by constant reference, skipping states other than `Waiting` and `Running`. For each eligible person it measures horizontal distance from the aircraft, and a strictly smaller distance replaces the current best position.

The helper returns a position, not a crew index, and has no 12-unit pickup radius. If no eligible person is closer than the initial 10,000-unit bound, it returns the harvester fallback. Equal-distance ties retain the first previously selected person because the comparison is strict. It does not change the pickup target or board anyone.

Game Lines 91-100: Cursor Validation, Normalization, and Deadzone

`setMouseHover(x, y, width, height)` first clears both hover axes. It returns immediately if width or height is nonpositive, either coordinate is nonfinite, or either coordinate lies outside the window. Coordinates exactly zero or exactly width/height are accepted because the boundary comparisons use `<` and `>`, not `<=` and `>=`.

For valid input, horizontal position becomes $2 \cdot x / \text{width} - 1$ and vertical position becomes $2 \cdot y / \text{height} - 1$. A left or top edge is -1, the center is zero, and a right or bottom edge is +1. The intermediate expressions use the double cursor coordinates before conversion to float for the deadzone function.

The local lambda maps one normalized value to `copysign(max(0, (abs(value) - 0.12) / 0.88), value)`. The central magnitude of 0.12 produces zero, and the rest is rescaled so magnitude one remains one. The deadzone occupies six percent of window width or height on either side of center, because the normalization doubles pixel displacement. In a 1,000-pixel-wide window, horizontal coordinates 440 through 560 are in that neutral region.

`copysign` keeps the original sign, including a possible negative zero; comparisons and vector lengths still treat that zero as neutral. Each axis is treated separately. There is no radial deadzone. A corner can therefore produce $(1, 1)$, whose two-component length is greater than one. Invalid input clears both axes even if only one coordinate was wrong. This function only stores arithmetic input: it does not itself steer, move, advance time, or update turn inertia.

Game Lines 102-118: Flight Direction and Camera Helpers

`flightForward()` uses the aircraft's current `g_ornYaw` and the same $(-\sin(\text{yaw}), 0, -\cos(\text{yaw}))$ convention as the harvester helper. It does not inspect velocity. Reversing or strafing therefore does not automatically aim the nose in the movement direction.

`flightCameraPitch()` returns `clamp(12 - g_mouseHover.y*22 + g_camOrbitPitch, 6, 65)`. With a centered mouse and zero orbit adjustment, pitch is 12 degrees. Top-edge hover has $y = -1$ and gives 34 degrees. Bottom-edge hover has $y = +1$ and would give -10 before clamping, so its result is six degrees. The separately supplied orbit adjustment can raise the result to the overall 65-degree maximum or lower it to six. This helper calculates a value; it does not implement the callback that changes orbit adjustment.

`flightCameraOffset(mode)` starts with `behind = -flightForward()`. Mode 2 returns 26 units behind and 100 units upward, independent of mouse pitch. Otherwise, mode 1 uses distance 20 and every other integer uses distance 40. Thus unknown modes fall through to the ordinary follow calculation rather than producing an error.

For those non-overview modes, horizontal distance behind is $\text{distance} \cdot \cos(\text{pitch})$ and height is $\text{distance} \cdot \sin(\text{pitch})$. Mode 1 adds another 2.2 vertical units. The ordinary follow offset has length 40, and the close offset has length 20 after subtracting the extra height. All offsets remain horizontally behind the nose, which keeps camera screen-right consistent with aircraft-right. The helper returns an offset, not an absolute eye position or a view matrix; the runtime uses it to place the camera.

Starting and Finishing Attempts

Game Lines 120-140: Reset Ordering and Flight Restoration

`resetMission(start, next=false)` is the attempt initializer. `start` chooses active play versus title, while `next` determines whether to increment the current wave. The default second argument means a one-argument call retries the same wave.

It first increments the mission revision and calls `g_audio.resetMission()`. Audio internals are explained separately. It saves the previous best and either preserves wave or calculates $\text{wave} + 1$. Then it replaces the mission with a default-constructed object and restores those two saved values.

The new breach threshold is $\max(150, 210 - (\text{wave} - 1) * 10)$. Wave one gets 210 seconds, wave two 200, wave three 190, wave four 180, wave five 170, wave six 160, and wave seven and later 150. Nothing here prevents callers from changing wave to an unusual number outside normal play; the documented progression describes ordinary positive waves.

The phase becomes `Flying` when `start` is true and `Title` otherwise. The harvester is restored to `harvesterRoute(0)`, its x/z globals are replaced, and its yaw is recomputed from the route tangent. The base's y is replaced with terrain height. The aircraft is placed eight units above that base and receives zero velocity and yaw.

Both hover axes, stored turn rate, aircraft speed, pitch, roll, and camera orbit pitch are zeroed. The chained assignment to speed, pitch, and roll resets all three values. Importantly, the ordinary reset does not clear `g_keys`, does not explicitly clear `g_isBoosting`, and does not reset `g_wingPhase`. The testing helper `fresh` handles those additional globals. A reset should not be described as clearing every application-global variable.

Game Lines 141-154: Exact Population and Release Schedule

Population construction begins with `remaining = 36`. The while loop continues until none remain. For each iteration, the group index is the current number of groups. Desired size is $1 + \text{group} \% 4$, capped by remaining people. Release time is $6 + \text{group} * 7$ seconds.

The group is appended with its count, scheduled release, initial harvester position, and `released=false`. A nested for loop creates slots from zero through `count - 1`. Each new person's default state remains `Inside`, while release, group index, and slot are assigned explicitly. Both position and destination initially become the starting harvester position. Each person is appended, then the outer loop subtracts that group's count from remaining.

The exact result is 15 groups. Their sizes are 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, totaling 36. The release schedule is 6, 13, 20, 27, 34, 41, 48, 55, 62, 69, 76, 83, 90, 97, 104. The last group's three people complete the population without truncating its nominal size. No random generator determines these numbers. The schedule is unchanged on later waves, so even the 150-second breach floor remains after the final scheduled release.

The loops assign identity by vector position and group/slot fields, not by a separately allocated identifier. Vector insertion order later matters for nearest-person ties. All people in a group share a release time but receive different destinations when their group is activated.

Game Lines 155-159: Tank Reset

The external tank vector is cleared, then three aggregate records are inserted. Their positions and accompanying values are $(-12, 0, -26)$, $(20, 1)$; $(-19, 0, -32)$, $(-15, 1)$; and $(-26, 0, -23)$, $(10, 1)$. The game header supplies the literal records but does not declare the tank structure; the scene chapter identifies how its orientation and scale fields are rendered.

These y coordinates are explicitly zero in the stored records. This routine does not query terrain for them, and this header does not give tanks an independent collision, crew population, or mission counter. Clearing the vector means a normal mission reset replaces any previous tank entries with these three.

Game Lines 161-166: Finishing Without a Separate Win Flag

`finishMission()` changes phase to `Debrief`, sets `best` to the larger of its existing value and the current delivered count, clears both dwell-progress values, and zeros aircraft velocity. It does not itself mark unresolved crew lost, empty the cabin, change the prompt, or decide whether the result is a success. The calling update branches do their own loss reconciliation before invoking it.

There is no won Boolean or `Win` phase. Delivering all 36 without losses is the full-success scenario, but delivering the final survivors after other crew are lost also ends naturally in `Debrief`. Direct callers can invoke `finishMission()` on an unresolved population, as the next-wave test intentionally does; the function alone does not enforce a finished-population invariant.

It also does not reset `g_orbSpeed`, pitch, roll, wing phase, boost status, or turn rate. Because later mission updates return immediately in `Debrief`, those values can remain as the last-frame snapshot even though velocity is now zero.

The tests correctly check frozen simulation rather than assuming all display or animation values become their defaults.

The Active Update

Game Lines 168-174: Guard, Clock, Harvester, and Winch Input

`updateMission(dt)` takes a reference named `m` to the global mission, avoiding repeated long names and ensuring writes affect the real object. If `phase` is anything other than `Flying`, or if `pause` is true, it immediately returns. `Title`, `debrief`, and `pause` therefore freeze all work within this function, including the clock, routes, resources, crew, hazards, and flight animation.

For active play it first adds `dt` to elapsed time, then updates the harvester at the new elapsed time, then calculates attack time. Thresholds in the frame therefore see the end-of-step clock. `winch` is a constant Boolean snapshot of the Space key for this update. No mouse click is required to steer or winch; mouse position and a keyboard Space state are separate inputs.

The function does not validate that `dt` is finite, nonnegative, or reasonably small. Normal application timing and test fixtures provide suitable values. A zero `dt` still evaluates release and hazard conditions and can finish a mission already placed at its timeout. That behavior is deliberately used in tests. Negative or nonfinite `dt` is not covered by these tests and should not be advertised as supported.

Game Lines 175-182: Steering and Exact Angular Smoothing

Steering combines normalized horizontal hover, a +1 contribution from Right, and a -1 contribution from Left, then clamps the result into `[-1, 1]`. Both arrows together cancel. An arrow can reinforce or oppose mouse hover, but the clamp limits total strength. Desired angular rate is `turnRate = -steer * 80`, so full right gives -80 degrees per second and full left gives +80.

The previous actual rate is `g_mouseTurnRate`. With damping constant eight per second, `turnDecay = exp(-8*dt)`. The new stored rate is `mix(turnRate, oldRate, turnDecay)`, or `turnRate + (oldRate - turnRate) * exp(-8*dt)`. It approaches the requested rate exponentially rather than switching instantly.

Yaw uses the exact integral of that exponential over the step: its change is `turnRate*dt + (oldRate - turnRate) * (1 - exp(-8*dt)) / 8`. This is more accurate than first computing the new rate and multiplying that end-of-step rate by `dt`. With a constant steering command it agrees with the continuous solution across different step sizes, apart from floating-point error.

`std::remainder(..., 360)` wraps yaw using the nearest multiple of 360. Its result is around the signed half-turn range rather than the `[0, 360)` interval returned by some wrap conventions. Equivalent headings such as 359 and -1 therefore remain physically identical even if their stored numbers differ.

Roll moves exponentially toward `g_mouseTurnRate * 0.28` with rate six per second. At a settled full turn, the target magnitude is 22.4 degrees. This is a visual bank based on turn rate, not a cause of sideways acceleration. Because the roll target itself evolves during the step, the same exact-yaw guarantee should not automatically be claimed for the entire visual roll trajectory.

Game Lines 183-195: Translation, Boost Lock, and Velocity

The update computes forward from the current yaw and right as `cross(forward, (0, 1, 0))`. At zero yaw these are negative Z and positive X. D contributes +right, A contributes -right, W contributes +forward, and S contributes -forward. Opposite held keys cancel on their respective axes.

If the resulting input vector has positive length, it is normalized. This prevents diagonal W+D movement from being $\sqrt{2}$ times faster than a single direction. A zero vector stays zero, avoiding division by zero. Translation remains relative to nose orientation; neither desired movement nor velocity realigns yaw.

Boost lock is evaluated in order. Releasing left Shift clears the lock first. A resource level at or below 0.04 then sets it again. Actual boosting requires held left Shift, an unlocked resource, no winch, and nonzero movement input. Holding Shift alone while stationary is not boosting; holding Space suppresses boost even when Shift and movement are also

held.

Resource then changes by $dt * (-0.22)$ while boosting or $dt * (0.14)$ otherwise, clamped into $[0, 1]$. Full-resource continuous use reaches the 0.04 lock threshold in approximately $(1 - 0.04) / 0.22 = 4.36$ seconds under small steps. Recharging from zero to one takes approximately $1 / 0.14 = 7.14$ seconds while not boosting. A held Shift cannot automatically unlock after recharge: release is needed, and if that release happens while energy is still at or below 0.04 the next condition relocks immediately. Threshold decisions use the resource before this frame's resource change.

Desired velocity is normalized input times 48 while actually boosting and times 28 otherwise. If Space is held, desired velocity is reduced to ten percent, giving at most 2.8 with ordinary translation. Holding Space therefore permits slow movement rather than forcing a stationary aircraft.

Velocity approaches desired velocity using $1 - \exp(-dt * k)$, with $k=12$ while winching and $k=5.5$ otherwise. With a constant target this smoothing matches exponential damping. The aircraft position then advances by the resulting velocity times dt . That positional integration uses the new sampled velocity, not the analytic integral used for yaw, so complete movement is not exactly independent of step size. Changing heading and changing boost targets introduce additional per-step effects.

Game Lines 196-214: Bounds, Height, Obstacle Clearance, and Animation

The aircraft's x and z are each clamped into $[-160, 160]$. This is an axis-aligned square, not a circular map boundary. Velocity is not zeroed or clipped on hitting an edge, so a craft pressing against the boundary can still have a nonzero speed value. g_or_nSpeed is the length of its velocity after the update, not displacement divided by dt .

Altitude input is $(Q - E) * 15 * dt$, clamped between four and 55. Q raises desired altitude; E lowers it; both cancel. With Space held, the already clamped desired altitude is then exponentially mixed toward six at rate four. The order means altitude keys can still influence the value while winching, but the winch's pull toward six acts afterward.

ground is terrain height under the aircraft's newly updated horizontal position. $clearance$ starts as desired altitude. For each of eight rock sites, the code measures horizontal distance and computes $avoid = 1 - smoothstep(8, 17, distance)$. Specifically, let $u = clamp((distance - 8) / 9, 0, 1)$; $smoothstep$ is $u * u * (3 - 2 * u)$, and $avoid$ is one minus that result. At distance eight or less, $avoid$ is one. At 17 or more, it is zero. Between them it changes smoothly with the $smoothstep$ cubic rather than linearly. $Clearance$ becomes the maximum of its current value and $13 * avoid$. Thus close rocks request at least 13 units, and the outer region fades that extra request.

There is also harvester avoidance. While attack time is less than 11, and the craft is within strictly eight x units and strictly seven z units of the harvester, $clearance$ becomes at least 12. This is a rectangle, not a radius. The condition includes all pre-breach times because negative attack times are less than 11. It stops at attack 11, one second before the Inside-crew swallowing threshold of 12. It is a height-assistance rule, not a harvester impact/failure test.

Aircraft y approaches $ground + clearance$ exponentially at rate eight. A subsequent hard maximum guarantees y is at least $ground + 3$ immediately. The hard floor protects against abrupt terrain changes; the 12/13-unit assistance is only a smoothed target and does not guarantee instantaneous clearance above every rendered obstacle.

Aircraft pitch approaches $-speed * 0.3$ at rate six. Faster horizontal motion therefore tilts the display downward more strongly. Wing phase increases by $36 * dt$ during actual boost or $23 * dt$ otherwise, including stationary active flight. The phase is not wrapped here. Q/E height changes and y smoothing do not add a vertical component to the translational velocity vector; this is terrain-following arcade flight, not full rigid-body aerodynamics.

Game Lines 215-233: Releasing Groups at the Scheduled Pose

The update loops over all groups by integer index. It skips a group if already released or if elapsed time is still below its release time. A group can therefore activate exactly at its schedule, and one coarse update can activate several due groups.

For an activating group it evaluates the route and tangent yaw at $group.release$, not the current elapsed time.

Forward and right are constructed at that scheduled pose. The exit is $origin + right * 6.1 - forward * 1.2$. The

gathering center is another 14 units along right from the exit, and its y is terrain height. `released` becomes true before processing members, preventing future destination recomputation.

The nested crew scan selects only members of this group still in `Inside`. Those people become `Running` and start at the exit. Their destination is center plus a forward spread of $1.7 * (\text{slot} - (\text{count} - 1) * 0.5)$ and an additional right spread of $1.3 * (\text{slot} \% 2)$. Centering the forward offsets around zero distributes slots symmetrically along forward; alternating even/odd slots creates a second small row sideways. Destination y is terrain height at each person's own destination.

For a one-person group, slot zero has neither spread and its destination equals the center. For larger groups, not everyone stands exactly at the center. The exit's initial y comes from the route's origin because horizontal vectors have zero y, but the later crew loop sets actual position height to local terrain before hazards and pickup.

The release loop's work is proportional to groups times crew because each released group scans the whole population. With 15 groups and 36 people this is small. More importantly, setting a group's flag while none of its people remain `Inside` is permitted; the flag records scheduled activation, not a successful evacuation count.

Game Lines 234-250: Walking and Terrain Following

`waiting` starts at zero. Despite its name, it will count all unresolved ground/harvester states, including `Inside` and `Running`, not just `Waiting`. The update calculates the current worm position and harvester forward/right before walking through all crew.

An `Inside` person's current position follows the current moving harvester exit. Their original destination is not updated until release. A `Running` person forms a displacement to destination and removes its vertical component, so movement is horizontal. Available walking time is $\min(dt, \max(0, \text{elapsed} - \text{released}))$. This prevents a newly released person from walking the entire `dt` when only a small final part of the step occurred after release.

At walking speed 3.8, a person reaches their destination when remaining distance is at most $\text{walkDt} * 3.8 + 0.00001$. The tiny extra tolerance avoids floating-point near-arrival jitter. Arrival snaps position to destination and changes the state to `Waiting`. Otherwise movement is normalized horizontal displacement times walking distance. Zero-distance people take the arrival branch rather than dividing by zero.

Every person then has their y set to terrain height at their current x/z, regardless of state. That includes `Aboard`, `Safe`, and `Lost` people. This header does not physically attach an `Aboard` person's stored position to the aircraft or move a `Safe` person's stored position to base. Their states and counters carry those meanings; their stored positions can remain at old pickup points, which the renderer can choose not to draw for those states.

Game Lines 251-259: Crew Hazard Windows and Unresolved Count

`outside` means the state is `Running` or `Waiting`. Danger radius is 16 before breach, 32 from breach until attack time 12, and 44 at attack time 12 and later. Separately, `activeWorm` is true before breach only after elapsed time is strictly greater than eight, or after breach only when attack time is strictly greater than three and strictly less than 18.

An `Inside` person is lost when attack time is at least 12, independent of their distance and independent of `activeWorm`. An outside person is lost when the worm is active and their horizontal distance is strictly less than the current radius. At the exact radius they are not lost by that test. The state changes to `Lost` and the loss counter increments once. On future updates that person is no longer `Inside` or `outside`, so the same local loss is not counted again.

The actual outdoor windows are therefore: no pursuit danger through exactly elapsed eight; radius 16 after eight and before breach; no outside danger from breach through exactly attack three; radius 32 after attack three and before 12; radius 44 from attack 12 until just before 18; and no local outside danger at 18 or later. `Inside` swallowing remains active after 12. The interval between breach and attack three is a real rule gap, not an omitted continuous hazard.

After those possible transitions, the update counts every remaining `Inside`, `Running`, or `Waiting` person into `waiting`. `Aboard`, `Safe`, and `Lost` do not contribute. This local count is used by the natural-completion branch and is decremented immediately if someone boards later in the same update.

Game Lines 261-277: Prompts and Unloading

The initial prompt is `HARVESTER FLEEING / RESCUE AMBER GROUPS` before breach or `WORM BREACH / CLEAR THE HARVESTER` at and after breach. Actual low-flight status is `g_ornPos.y-ground <= 11`; the desired altitude alone is not enough. Steady means speed strictly below six. At base means horizontal distance strictly below 14.

A full cabin overwrites the prompt with `CABIN FULL / RETURN TO CYAN BASE`. At attack 12 or later, that is overwritten with either `HARVESTER LOST / DELIVER SURVIVORS TO BASE` if anyone is aboard, or `HARVESTER LOST / RESCUE REMAINING GROUPS` otherwise. Prompt order matters: these are assignments, not independent UI messages.

If at base with at least one occupant, the prompt becomes `HOLD SPACE / UNLOAD SURVIVORS`. Only simultaneous Space, low status, and steady status increase unload progress, by $dt/1.2$. Completion at progress one means a continuous eligible interval of about 1.2 seconds. Every Aboard person's state changes to Safe, rescued increases by the entire cabin count, aboard becomes zero, unload resets to zero, and best updates to the maximum banked score.

If any unloading requirement fails, progress resets, rather than pausing and preserving its partial value. Being outside base or having no cabin occupants also resets it. No fractional score is banked before completion. A large step that exceeds the interval unloads once and discards extra progress; it does not calculate repeated unload transactions.

Unloading happens before nearest-person pickup. In an unusual fixture with waiting people at base, an update may unload a previous occupant and then start or complete a new pickup. The code does not declare the base a pickup-free area. Also, later pickup prompts can overwrite the unloading instruction even while unload progress is being processed, so prompt text is not an exclusive state-machine representation.

Game Lines 278-297: Target Selection and Pickup Dwell

The pickup search starts with index -1 and radius 12, scans every crew index, and skips everyone not Running or Waiting. A candidate replaces the nearest only if its horizontal distance is strictly smaller than the current best. A person exactly 12 units away is not selected, and equal-distance ties favor the first encountered index.

Changing target resets pickup progress immediately. The current target field then becomes the selected index. A valid target and an aboard count below eight allow the pickup block. If measured clearance is too high, the prompt says `T00 HIGH / HOLD SPACE TO LOWER WINCH`; otherwise it says `HOLD SPACE / HOVER TO WINCH CREW`. This text does not separately report excessive speed, although speed still controls eligibility.

With Space, clearance at most 11, and speed below six, pickup adds $dt/0.42$. At one or more, that single person becomes Aboard, aboard increases, unresolved waiting decreases, progress resets, and target resets to -1. The next frame must search again. Losing eligibility, having no target, or having a full cabin resets progress.

The 0.42-second requirement is per person. It is not enough to spend that total near multiple changing targets, since each target switch loses partial progress. Boarding does not increase rescued or best. The cabin limit is checked before boarding, so a correct counter never goes above eight.

Only one person can board per call, even with a very large dt. Extra progress is discarded rather than carried to the next person. Thus dwell behavior under coarse steps is quantized, and exact angular smoothing should not be confused with a guarantee that every gameplay outcome is identical under arbitrary dt. The test suite mainly uses fine updates and separately tests the scheduled-release boundary logic.

Game Lines 298-307: Aircraft Collision, Natural Completion, and Timeout

Aircraft approach collision requires pre-breach attack time, elapsed strictly greater than eight, horizontal distance to the worm strictly below 18, and aircraft y strictly below `worm.y+22`. Breach collision requires attack strictly greater than three and strictly less than 15, horizontal distance strictly below 24, and y strictly below `worm.y+40`.

These heights are relative to terrain at the worm, not necessarily terrain under the aircraft. At coincident x/z they are the same terrain height, which the height-envelope tests use. At separated points on a dune they can differ. Aircraft and crew therefore have different radii and different post-breach end times: aircraft local breach danger stops at 15, outside crew danger stops at 18.

If either aircraft-hit condition is true, every person other than Safe or already Lost becomes Lost and increments the loss total. This includes remote people and cabin occupants, not just those near the crash. Aboard is set to zero, `planeLost` becomes true, the prompt becomes ORNITHOPTER LOST IN THE WORM'S MAW, and `finishMission()` enters debrief. Banked Safe people survive and retain their score.

The alternate finishing branch runs only if there was no aircraft hit. It triggers when unresolved waiting is zero and aboard is zero, or when attack time is at least 60. Any remaining nonsafe/nonlost people are then marked Lost, aboard is cleared, and finish is called. The natural condition does not require rescued to equal 36: a mixture of delivered and lost people can satisfy it.

Timeout is 60 seconds after breach, making a first-wave total deadline of 270 seconds and a floor-wave deadline of 210 seconds. The function does not set `planeLost` for timeout. Because unloading and pickup precede this branch, a delivery that completes on the update reaching the timeout can bank its occupants before unresolved people are lost. Conversely, any person newly picked up on that same frame without a completed delivery is still unbanked and is lost.

No separate penalties are applied for rock contact, leaving the map, or hitting the harvester in this header. Map edges clamp movement, obstacle proximity requests clearance, and the worm alone supplies explicit aircraft loss. Pausing freezes elapsed time, so deadlines are simulation-time limits rather than unavoidable wall-clock limits.

Test Harness and Fixtures

Test Lines 1-13: Dependencies, Namespace, and Result Fields

The test header also uses `#pragma once`. Its comment says to include it after the game header and that these tests never initialize GLFW or call OpenGL. They still use GLFW key constants and the globals supplied by the surrounding program. This is a mission-logic test suite, not an independent replacement for all application initialization.

`<iostream>` supplies console streams and `<limits>` supplies infinity and NaN values. The namespace `arrakis_tests_detail` contains the small harness and reusable setup helpers without putting their short names directly into the global namespace.

`Results.current` is a borrowed `const char*` case-name pointer, initially the empty string. `cases`, `failedCases`, `checks`, and `failures` are integers initially zero. A case is an entire named scenario; a check is one Boolean assertion. Several checks can fail within one case. Consequently `failedCases` and `failures` need not be equal.

Test Lines 15-32: Every Results Method

`check(condition, message)` always increments the check count. If condition is false, it increments failure count and writes `[FAIL]`, the current case name, a colon, the supplied explanation, and a newline to standard error. It does not throw, abort, or return early, allowing a case to collect additional failures.

`run(name, test)` is a function template accepting a callable of any suitable type. In this file those callables are lambdas. It stores the case name, increments `cases`, snapshots the existing failure count, and calls the callable. If the count has changed afterward it increments `failedCases`. It prints `[PASS]` or `[FAIL]` followed by the name to standard output according to whether that case added failures.

An earlier failure does not automatically fail every later case, because comparison is against the per-case snapshot. The harness does not catch exceptions, measure test durations, isolate process state, or stop after the first failure. Helpers reset globals between scenarios. A serious setup failure could still lead to later invalid indexing because `check` is not a protective assertion that terminates execution.

Test Lines 34-40: Near Comparison and State Counter

`near(a, b)` checks whether absolute difference is strictly less than 0.001. It uses absolute tolerance, not a percentage or a relative error. Exactly 0.001 is outside the accepted interval. NaN differences do not satisfy the comparison.

`count(state)` starts at zero, loops over all mission crew by constant reference, adds one for each exact state match, and returns the count. It derives population information from individual states rather than trusting score/cabin/loss counters, which makes later reconciliation meaningful.

Test Lines 42-55: Fresh State and Fine-Grained Advancement

`fresh(start=true)` clears every Boolean key in the key array, centers hover, zeros stored turn rate, replaces the mission with defaults, clears actual boost status, and resets wing phase. It then calls `resetMission(start)`. Unlike an ordinary retry, this fixture deliberately discards a previous best and wave because the default mission was installed first. The revision counter and audio state are affected by reset in the normal way.

`advance(seconds)` chooses `max(1, ceil(seconds*120))` update steps and calls `updateMission(seconds/steps)` for each. For positive durations this produces steps no longer than approximately 1/120 second. It always calls the updater at least once, even for zero seconds. Tests sometimes directly call the updater instead when an exact coarse interval or boundary is the subject of the scenario.

The helper is not a real-time wait and does not sleep. Advancing 100 seconds in title or pause simply executes many immediately returning simulation calls. Negative or nonfinite input is not its intended use; its integer conversion and update semantics do not provide general validation.

Test Lines 57-70: Hover Teleport and Staging Point

`hover(position, altitude=6)` is explicitly a teleport fixture, not autopilot. It replaces the passed position's y with terrain height plus the requested altitude, writes aircraft position, clears velocity and speed, centers hover and turn rate, and sets desired altitude. It does not clear keys, reset mission time, or change crew states.

`staging()` returns the first crew member's destination when crew exist, otherwise the current harvester position. It does not find the nearest unresolved member. In the compact fixture the first destination is a useful common pickup area even after that first person becomes Aboard or Safe, since their stored destination remains there.

These helpers isolate state transitions from route navigation. A test that uses hover demonstrates requirements at a chosen location but does not prove the player can fly there quickly enough. The later input-only case addresses that distinct limitation.

Test Lines 72-99: Compact Ready Crew, Collection, and Delivery

`readyCrew()` begins with `fresh()`. It chooses a terrain-height center at x zero and z -10. Every group is marked released and receives that same center. For each crew index *i*, the person becomes Waiting and is placed at center plus $(0.6 * (i \% 6), 0, 0.6 * (i / 6))$; integer division creates six rows and modulo creates six columns. The person's y is recalculated and destination becomes position.

All 36 therefore form a compact six-by-six grid. Original group identities, slots, and release values remain, but the release flags prevent normal scheduled spawning. This is a deliberate synthetic fixture. It must not be described as the actual spatial layout generated by the harvester's route.

`collect(occupants)` teleports to staging, holds Space, and advances `occupants*0.45` seconds. The 0.45 interval gives a small margin over each 0.42-second pickup. It does not directly set aboard or crew states, nor does it release Space afterward. Collection can still be limited by capacity or remaining eligible crew.

`deliver()` teleports to base, holds Space, and advances 1.3 seconds, a margin over the 1.2-second unload dwell. It likewise relies on real mission logic and leaves Space held. Tests that require Space released must explicitly change the key.

Test Lines 101-113: Every Reconciliation Assertion

`reconciled(r)` computes unresolved people by adding actual Inside, Running, and Waiting state counts. Its first check requires the vector to contain exactly 36 people. Its second requires actual Safe count to equal `rescued`; its third requires actual Aboard count to equal `aboard`; its fourth requires actual Lost count to equal `lost`.

The fifth check requires `rescued + aboard + lost + unresolved == 36`. In combination with the individual state-counter comparisons, this detects missing or double-accounted people. The sixth check requires the cabin count to lie inclusively between zero and eight. If phase is Debrief, a seventh conditional check requires unresolved to be zero and aboard to be zero.

The helper does not require every debrief to have 36 rescues, require best to equal rescued, inspect group identities, or compare stored cabin positions with the aircraft. It tests accounting invariants appropriate to both success and failure. Calls made while Flying produce six assertions; calls during Debrief produce seven.

Reset, Population, and Pickup Cases

Test Lines 115-140: Case 1, Reset and Title Do Not Advance

`runMissionTests()` imports the helper namespace locally and constructs a fresh `Results`. Each case is registered with `r.run` and a lambda capturing surrounding variables by reference. The first case calls `fresh(false)` to enter Title rather than active flight.

The checks at lines 121-126 require Title, wave one with breach time near 210, all four score-related counters at zero, all 36 people Inside, altitude near eight, and full boost near one. The combined counter check specifically includes best because `fresh` discards old records before reset.

The case snapshots aircraft position/yaw and harvester/worm positions. It sets the cursor to the right edge at midheight and holds D plus Space, then advances 100 seconds. Assertions at lines 133-138 require elapsed time still zero; aircraft position unchanged within 0.001 and wing phase near zero; unchanged yaw and zero smoothed turn rate; all crew still Inside with zero pickup; and both moving-world actors unchanged within 0.001.

These checks prove the update guard freezes title simulation even under nonneutral steering, movement, and winch input. They do not require `g_mouseHover` itself to stay zero, because cursor arithmetic is allowed to store hover while the simulation is inactive. The case closes with the six active/title reconciliation checks.

Test Lines 142-178: Case 2, Pause Freezes Mission and Resumes

After `fresh`, the case applies right-edge hover and advances seven seconds. This gives an already moving harvester and a first group in the evacuation process, making pause more meaningful than freezing an untouched initial state. It copies the entire mission, snapshots flight position, wing phase, yaw, stored turn rate, harvester/worm positions, and worm gap, then sets pause and holds W, Shift, and Space.

After 100 simulated seconds of attempted advancement, lines 155-158 separately require an unchanged elapsed clock; unchanged aircraft position and wings; unchanged yaw and stored turn rate; and unchanged boost plus pickup progress. Because pause returns before boost logic, contradictory boost/winch keys do not alter resources.

The first comparison loop initializes `crewUnchanged` from vector-size equality, then proceeds only while that flag remains true and the index is within the copied population. Each person must retain state and position within 0.001. The assertion at line 163 checks that accumulated Boolean. It does not compare every Evacuee field, such as destination, individually.

Lines 164-166 require unchanged harvester and worm positions, yaw, accumulated travel, and gap. A second short-circuit comparison loop checks group vector-size equality, then released flags and centers within 0.001. Line 171 checks its result. The case clears all keys, centers the mouse, unpauses, advances 0.1 second, and requires elapsed time to increase. Reconciliation then confirms the still-active population.

Test Lines 180-246: Case 3, Scheduled Groups and Fixed Clusters

The initial assertion requires exactly 15 groups. For each group, lines 186-187 check the repeating one-through-four size, release $6+7*i$, and an initially false release flag. The nested crew loop counts members with this group index. Each member must have a slot from zero through `count - 1` and the group's release time within near tolerance.

A bitmask records slots by setting `1u << slot` for slots zero through three. The group assertion requires both the correct member count and the mask `(1u << count) - 1`. Together these verify every expected slot is represented exactly once: a duplicate would consume one of the fixed number of members and leave a required slot absent. The accumulated count must total 36 and the last group must contain three.

The case records the last Inside person's initial position and the initial harvester position, then advances 5.99 seconds. It requires all 36 still Inside. A combined movement assertion requires that last person to have moved more than two

units, remain less than 15 horizontally from the current harvester, and the harvester itself to have moved more than two. This checks following rather than a fixed spawn position.

Advancing another 0.02 crosses the first release. The first person must be Running and exactly 35 remain Inside. The case records their current position, destination, and group center, then independently constructs the tangent, right vector, and exit at exactly time six. Lines 219-221 require center to equal scheduled exit plus 14 rightward units within horizontal tolerance 0.001, and the one-person destination to equal center horizontally.

Another 0.25 seconds must move that person by approximately $3.8 * 0.25 = 0.95$, with absolute movement error below 0.01. Another 5.74 seconds leaves elapsed near 12, before the second release at 13, and all 35 other people must still be Inside. Another 1.02 crosses that release: group one must be released and only 33 remain Inside.

Advancing 106 more seconds reaches a time at which all default groups have released and walked, but still before breach. The assertion requires 36 Waiting. Stored first destination and center must remain unchanged in full three-dimensional distance below 0.001. A loop then requires every person's horizontal distance to destination below 0.15 and their y near terrain height. The first and final group centers must differ by more than 20 horizontal units.

Finally, the first waiting position is snapshotted, five more seconds are advanced, and it must stay fixed within 0.001 while the harvester and worm continue. Reconciliation verifies all 36 remain unresolved but correctly counted. This case exercises actual scheduled evacuation rather than the compact fixture.

Test Lines 248-267: Case 4, Space Dwell and Eight-Person Capacity

The case uses readyCrew and teleports to staging. One second without Space must leave aboard zero. It then holds Space for 0.3 seconds, less than the pickup dwell, and requires aboard still zero but pickup progress positive.

Releasing Space for 0.05 seconds must reset progress to near zero. Holding it for 0.5 seconds must produce exactly one occupant and exactly one person in Aboard. Rescued and best must still be zero, proving boarding is not scoring.

After five additional seconds of eligible hovering, the capacity constant must equal eight and the cabin count must equal eight. The remaining 28 people must still be Waiting. This simultaneously checks repeated pickup, capacity enforcement, and preservation of ground crew once full. Reconciliation verifies cabin and population totals.

Test Lines 269-278: Case 5, Excessive Speed Blocks Pickup

The compact fixture is prepared, the craft is placed at staging, and Space is held. The case deliberately gives the aircraft a velocity of $(20, 0, 0)$ and runs one 0.05-second update. With Space damping at rate 12, it remains fast enough to exceed the six-unit threshold despite slowing.

The first assertion explicitly checks that setup still has speed greater than six. The second requires aboard zero and pickup progress near zero. Without the first assertion, a test could accidentally pass or fail after its injected velocity had decayed below the intended boundary. Reconciliation checks population consistency. This is not a sustained high-speed flight test; it isolates one invalid pickup frame.

Test Lines 280-294: Case 6, High Hover Must Lower Before Pickup

The aircraft begins at staging 35 units above local terrain. After 0.75 seconds without Space, aboard must be zero and desired altitude still near 35. The case then holds Space for 0.5 seconds.

Actual clearance must still exceed 11 after that initial lowering period, and aboard plus pickup progress must both remain zero. This distinguishes desired-altitude smoothing from actual y smoothing; merely pressing Space does not instantly satisfy clearance.

After two further seconds, measured clearance must be at most 11. At least one person must be aboard, while rescued remains zero. Reconciliation confirms no accounting change was mistaken for scoring. The scenario demonstrates eventual winch lowering, not a direct Q/E altitude-key test.

Test Lines 296-318: Case 7, Delivery Banks Only at Base

`collect(3)` must create exactly three unbanked occupants. The craft then hovers at base plus 15 x units and advances 1.4 seconds with Space still held by collection. Aboard must remain three, rescued zero, and unload near zero. This tests a point outside the strict 14-unit base radius, not the exact radius boundary.

At base, Space is released and 1.3 seconds passes; rescued must still be zero. Space is held for 0.6 seconds; rescued remains zero but unload becomes positive. Releasing it for 0.02 seconds resets unload to zero.

`deliver()` then supplies a fresh eligible dwell. The next combined check requires rescued three, best three, and no occupants. Exactly three people must be Safe and the phase must remain Flying because 33 unresolved people remain. Reconciliation checks that their states match all counters. This case separately tests location, key requirement, dwell, interruption, score banking, and continuation after a partial delivery.

Failure, Retry, and Completion Cases

Test Lines 320-347: Case 8, Swallowing Spares Remote Ground Crew

The case delays every group and every person's release to 1,000 seconds. It sets elapsed to breach plus 11.5 and updates the harvester, then makes person one Running exactly 40 x units from the worm with destination equal to position. People two and three become Waiting at base and base plus two x units. The other 33 remain Inside.

One 0.25 update reaches attack 11.75. No one must be lost and phase must remain Flying: the local person is beyond the 32-unit radius and Inside swallowing has not started. Because destination equals position, the Running local person can become Waiting during the crew loop; the scenario's key property is being outside at 40 units, not continued walking.

Another 0.25 reaches attack exactly 12. The assertion requires attack time near 12, losses 34, and no Inside people. Those losses are the 33 Inside people plus the outside person within the now-44-unit radius. Exactly two Waiting people must survive, phase remain Flying, and rescued stay zero.

After eight seconds, attack is 20 and local outdoor hazards have stopped. The craft is teleported to base and Space is held for one second. It must carry both survivors with losses still 34. `deliver()` must bank two and produce Debrief because nobody else remains unresolved. The seven debrief reconciliation checks prove all 36 are accounted for. The test explicitly disproves the assumption that swallowing automatically kills every waiting person anywhere in the world.

Test Lines 349-367: Case 9, Carried Crew Survive Swallowing

The compact fixture collects eight, and an assertion confirms that setup. Every non-Aboard person is changed to Inside and given a personal release time of 1,000. Group release flags remain true from the fixture, so these manually Inside people are not rereleased by the scheduled loop.

Space is released; elapsed is placed at breach plus 11.75; one 0.25 update reaches attack 12. Exactly 28 must be lost while eight remain aboard. Phase must still be Flying and rescued zero, demonstrating that cabin survival is not yet a banked rescue.

Reconciliation runs before delivery. `deliver()` then must produce Debrief, rescued eight, and best eight. A second reconciliation runs in the finished state. This separates ordinary crew swallowing from aircraft collision: the aircraft remains remote, so carried people are not automatically consumed with the harvester.

Test Lines 369-387: Case 10, Collision Preserves Banked Score

The fixture collects eight, delivers them, then collects eight more. An assertion requires rescued eight and aboard eight. Space is released, elapsed becomes breach plus four, and the harvester is updated. The craft is teleported to the worm and a 0.01 update triggers the active breach aircraft envelope.

Phase must become Debrief and cabin count zero. Losses must equal 28, while rescued and best both remain eight. The 28 are every unbanked person, including eight occupants and 20 ground people. The state reconciliation confirms the eight already Safe were not included in losses.

The case snapshots elapsed, advances ten more seconds, and requires elapsed unchanged and losses still 28. That tests both the debrief guard and protection against double-counting finished losses. This particular case does not separately assert `pPlaneLost`; the dedicated envelope case does.

Test Lines 389-409: Case 11, Extraction Timeout

After banking eight and carrying another eight, the aircraft is placed 20 x units from base, outside unloading radius, and Space is released. Every remaining Waiting person is placed at base plus (40, 0, 40), remote from the stopped worm. Elapsed becomes breach plus 59.75.

A 0.125 update reaches attack 59.875. Assertions require Flying, eight occupants, zero losses, and 20 Waiting people. Reconciliation verifies that the setup is valid just before timeout.

A second 0.125 reaches exactly 60. The next assertion requires attack time near 60 and Debrief. The final totals must be rescued eight, best eight, losses 28, and cabin zero. Timeout consumes both undelivered cabin occupants and unresolved remote ground crew, but not the banked eight. Final reconciliation verifies the complete accounting. The aircraft need not collide or set `pPlaneLost` for this ending.

Test Lines 411-440: Case 12, Retry Preserves Best and Clears Attempt State

The fixture collects and delivers eight, releases Space, places elapsed at breach plus 60, and calls `updateMission(0)`. The setup assertion requires Debrief and best eight. Zero dt is enough because the ending condition evaluates the already-set clock.

The case deliberately contaminates pause, boost, pickup, unload, target, velocity, hover, and stored turn rate. Values include boost 0.2, pickup 0.7, unload 0.4, target five, velocity (10, 0, 0), corner hover, and turn rate -50. It then calls ordinary `resetMission(true)`, not `fresh`, so preserving best is actually exercised.

The six subsequent checks require Flying, unpaused, wave one; best eight with current rescued/aboard/lost zero; elapsed and both dwell values near zero with target -1; full boost, altitude eight, and velocity length below 0.001; neutral hover, zero turn rate, and camera pitch 12; and aircraft position within 0.001 of base plus eight upward units with all 36 Inside.

Reconciliation checks the new population. The scenario does not claim retry clears keys or wing phase. Its assertions are restricted to state the reset actually restores and the camera default derived from those resets.

Test Lines 442-457: Case 13, Next-Wave Timer Floor

After `fresh`, the test writes best 17 and directly calls `finishMission`, then resets with both arguments true. Wave must become two with breach time near 200. Best must remain 17, phase Flying, and all 36 Inside.

An eight-iteration loop snapshots the current breach time and starts another next wave. Each iteration checks that the new threshold is no greater than the previous one and no less than 150. Afterward wave must equal ten and time be near 150. Reconciliation validates the final `fresh` population.

This directly exercises `reset`'s progression formula without playing every intermediate wave. It does not prove later waves remain fully rescuable before their shorter breach thresholds; the detailed input-only playability scenario uses wave one only.

Test Lines 459-490: Case 14, All 36 Through Real Crew State Flow

This case uses normal `fresh` and advances 120 seconds so all real scheduled groups release and walk. It first requires 36 Waiting. It does not use `readyCrew`, but it still uses teleport assistance for navigation.

banked starts at zero. The five-flight loop chooses `batch = min(8, 36 - banked)`, producing batches 8,8,8,8,4. For each person in the batch, a nested search starts with no index and infinite distance, scans actual Waiting crew, and picks the nearest to current aircraft position. If no target is found it breaks that inner collection loop rather than dereferencing -1.

Each selected position is reached with hover, Space is held, and 0.45 seconds are advanced. At the end of each batch, cabin must equal expected batch while rescued still equals the previous banked total. Reconciliation checks the load. Delivery then runs, banked increases by the expected batch, and rescued must equal that value with an empty cabin.

The phase assertion expects Flying for every banked total below 36 and Debrief exactly at 36. Reconciliation runs after each base visit. Final assertions require rescued and best 36, zero losses, 36 Safe, and elapsed before breach. This proves full successful state transitions are possible without altering the timer, but its instantaneous relocation does not by itself prove a physically flown route meets the deadline.

Input-Only Rescue Scenario

Case 15 is substantially larger than the other cases because it constructs a small test-side controller. The controller reads the world and emits Boolean key states. It is not player-facing autopilot shipped in the gameplay header. After its initial fresh, it does not teleport the aircraft, directly change velocity, set crew states, edit the clock, or shorten the deadline. All subsequent game-state transitions come from the actual updater.

Test Lines 492-504: Case 15 Setup and Finite-Vector Helper

The case begins with fresh and an assertion that wave is one and the breach threshold is near 210. The comments explicitly rule out hover, collect, and deliver assistance after reset. The mouse remains neutral, so aircraft yaw stays at its initial zero and world-aligned cardinal movement is available through the aircraft-relative keys.

The controller snapshots base and uses a constant dt of 1/120. steps, travelSteps, boostSteps, directionsUsed, and directionReversals start at zero. Steps count every update; travelSteps count updates performed inside flight-leg attempts; boostSteps count actual boost frames; directionsUsed is a bitmask; directionReversals count changes between positive and negative commands within a flight leg.

finiteState, clearOfGround, and neutralHeading start true and are accumulated as permanent success flags. Once one becomes false, later good frames cannot restore it. minimumClearance starts with actual initial aircraft y minus local terrain height. The finiteVector lambda checks all three components with std::isfinite, detecting NaN and either infinity.

The lambda objects capture the surrounding test variables as needed. A capture by reference means updates inside a lambda affect those original counters and flags, rather than independent copies. The later seconds lambda instead captures constant dt by value, since it only converts a tick count to display time.

Test Lines 505-529: Step Instrumentation and Every Observed Quantity

Before each game update, step records W as bit one, A as bit two, S as bit four, and D as bit eight whenever held. Bitwise OR preserves every direction ever used. It then calls updateMission(dt), increments total steps, and increments boostSteps only if the updater reports actual g_isBoosting.

Finite-state checks include every component of aircraft position, velocity, and base; aircraft yaw, pitch, roll, speed, and wing phase; mission elapsed time, breach time, desired altitude, boost, pickup progress, and unload progress; both hover axes; and stored mouse turn rate. A loop also checks every person's position, destination, and release time.

These checks are extensive but not literally every floating-point field in the mission. They do not test finite group centers/release values, harvesterTravel, harvesterYaw, or wormYaw in this lambda. The final assertion's wording says all flight and crew float state, and should be read according to this actual list rather than generalized to every object in the application.

Neutral-heading monitoring requires yaw near zero, stored turn rate near zero, and hover vector length below 0.001 on every substep. This checks that WASD translation never secretly aligns the nose with its movement direction. Ground monitoring computes actual clearance each time, requires it finite and at least 3 - 0.001, and updates the minimum encountered clearance.

Because these are cumulative checks performed after every update, they can catch a transient invalid frame even if final state later looks correct. They still sample at update boundaries; they do not test a continuous path between those

samples or compare the craft with rendered rock geometry.

Test Lines 530-550: Active Predicate, Flight-Leg State, and Braking

`active()` requires `finiteState`, phase `Flying`, and `elapsed` strictly before `breach`. The controller voluntarily stops at `breach` even though gameplay permits further extraction for 60 seconds. Thus a successful result is a pre-breach playability result, not merely a rescue before the final timeout.

`fly(target)` snapshots the current step count. It starts previous `X/Z` command signs at zero, per-axis pulse accumulators at zero, and `braking` false. Its while loop continues only while `active` and fewer than `12*120` updates have elapsed for this leg, limiting a leg to 12 seconds.

Each iteration calculates a target delta and horizontal distance. Braking becomes permanently true for the rest of the leg once distance is below `4 + speed/12`. The `speed/12` term approximates coasting distance under Space's rate-12 damping; the four-unit margin starts slowing early. Once braking is set it is not unset if the next distance grows slightly.

All keys are cleared before new commands are chosen. Space is held exactly when braking. Left Shift is requested only while not braking, distance exceeds 20, and boost is above 0.08. This gives a controller-specific energy margin above the game's 0.04 lock threshold.

The leg succeeds when horizontal distance is at most two and actual speed is below 0.5. It adds its elapsed updates to `travelSteps` and returns true. These tolerances are more restrictive than the actual pickup/base radii and speed-six dwell requirement, helping stabilize arrival. Because the return happens before this iteration's step, no extra update is needed solely to acknowledge arrival.

Test Lines 551-569: Boolean-Key Pulse Controller and Axis Projection

If still more than two units away, the controller estimates available speed as 2.8 in braking mode, otherwise 48 if Shift is requested or 28 if not. It selects damping 12 while braking or 5.5 otherwise. The estimated speed can differ from actual boost behavior if the game's lock rejects Shift, but the game remains the authority over motion.

It constructs `backward = -flightForward()` and aircraft right from the cross product. These form the horizontal axes for commands. Dot products project world-space position error and actual velocity onto each axis, so the controller still uses the declared aircraft coordinate system rather than writing `x/z` velocity directly.

The nested `axis lambda` takes error, velocity, a pulse accumulator by reference, and the negative/positive key constants. It predicts remaining error after coasting as `error - velocity/damping`. If the current absolute error is below one, or predicted error multiplied by current error is at most zero, it clears the accumulator and emits no key. The latter means estimated coast movement is sufficient to reach or cross the target on that axis.

Otherwise, it adds `min(1, abs(predicted)*(braking?2:4)/speed)` to the accumulator. When that accumulator reaches one, it presses the key corresponding to the sign of error and subtracts one. Smaller desired commands therefore become occasional one-frame key pulses. This expresses proportional control through Boolean keys, not through fractional keyboard values, position assignments, or velocity overrides.

For the right axis, negative means A and positive D. For the backward axis, negative means W and positive S. The reversed-looking W/S arguments are correct because the axis is backward, not forward. Diagonal pulses are still normalized by actual game logic. Pulse duty cycle and feedback influence the route, but all acceleration, braking, altitude lowering, boost consumption, and pickup remain ordinary updates.

Test Lines 570-590: Reversal Accounting, Leg Failure, and Space-Only Steps

Command X is D minus A; command Z is S minus W. Only nonzero commands are compared with their previous nonzero sign. If a previous sign exists and differs, `directionReversals` increments. Zero-input coasting frames do not erase the remembered sign. Each `fly` call initializes these memories anew, so an outbound D and a later return A are not counted as a reversal within one leg.

The controller executes `step()` after commands and reversal checks. If `active` becomes false or the leg budget expires before arrival, it adds that leg's steps to `travelSteps` and returns false. This ensures unsuccessful movement attempts

still contribute to reported travel time.

`holdSpace()` clears all keys, enables only `Space`, and performs one instrumented step. It is used for waiting, boarding, and unloading. It does not directly complete a dwell. The `seconds(ticks)` helper multiplies ticks by `dt` and rounds to the nearest millisecond using `round(value*1000)/1000`; that affects printed summaries, not simulated time.

Test Lines 591-617: Return Decisions, Banking Assertions, and Trip Output

The controller initializes completed flights, cluster visits, outbound/boarding timing counters, and a 15-group visit bitmask. Departure is the current total step count. In the main while loop, `outside` counts Running plus Waiting, and `allReleased` means no Inside people remain.

Return is chosen with a full cabin, or with a nonempty partial cabin when all crew have released and no outside people remain. The assertion at line 598 requires either a batch of eight or that final-survivor condition. It prevents intentionally inefficient early partial returns while people still await pickup.

The controller snapshots batch, banked score, and boarding-completion step, then flies to the captured base. A check requires successful arrival; if arrival failed it breaks out instead of entering an endless unloading loop. While still active and occupants remain, it calls `holdSpace`. The next assertion requires `rescued` to increase by exactly the original carried batch and `aboard` to become zero. Reconciliation verifies all individual states afterward.

Each completed trip prints its sequential flight number, outbound duration, boarding/waiting duration, return duration, unloading duration, batch size, total rescued, elapsed mission time, and total trip duration. It increments flights as part of that print expression. Departure then becomes current steps, outbound and boarding counters reset to zero, and `continue` begins the next main iteration.

These durations classify test-controller phases. They are not guaranteed to match exclusive physical activities: `Space` during braking can already begin pickup or unloading before `fly` reports settled arrival. The printed unloading interval after settled arrival may therefore be less than 1.2 seconds because some progress happened during the return approach. No assertion requires a fixed wall-clock duration from these print labels.

Test Lines 618-646: Selecting Released Crew and Visiting Every Cluster

If no return is due, a nearest-person search starts with index -1 and infinite distance. It scans only Running or Waiting people and selects strictly smaller horizontal distances. If none exist yet, `holdSpace` performs a waiting step, `boardingSteps` increases, and the controller retries. Early scheduled gaps therefore consume legitimate mission time.

For a selected person, the controller records group and chooses a fixed pickup point: destination for Running crew, actual position for Waiting crew. It does not chase the Running person's changing position every frame. It flies to that point, adds the leg to `outboundSteps`, asserts successful arrival, and breaks on failure.

Successful arrival increments `clustersVisited`. A group index between zero and 14 sets its bit in `visitedGroups`. Repeated visits increase `clustersVisited` repeatedly but do not create new bits, so the final bitmask is necessary to prove coverage of every group rather than merely many visits to a few groups.

After arrival, a loop holds `Space` while active, cabin is below capacity, fewer than 3×120 updates have passed at the cluster, and the originally selected person's state remains Running or Waiting. It stops when that person is boarded/lost, the cabin fills, the local three-second budget ends, or active fails. Other nearby people can also be picked up during these ordinary updates. Boarding timing accumulates after the loop, and the outer controller reevaluates the world.

This loop does not forcibly wait until an entire group is collected in one visit. A group can require repeated outer-loop selections. The complete-group coverage and full-load return conditions are validated by final assertions rather than assumed from the controller's intent.

Test Lines 647-666: Every Final Input-Only Assertion and Summary

All keys are cleared at the end of the controller. The first final check requires the accumulated finiteState flag, and the second requires neutralHeading throughout the route. The third requires the cumulative ground-clearance flag. These validate numerical safety, no unintended yaw, and the terrain floor across all sampled substeps.

The fourth requires directionsUsed == 15, meaning every W/A/S/D bit was exercised, plus actual boostSteps greater than zero. The fifth requires directionReversals zero. It means no individual fly leg changed its nonzero cardinal command sign on an axis; it does not forbid opposite directions on different outbound/return legs.

The sixth requires rescued 36, zero losses, and 36 Safe. The seventh requires Debrief, best 36, and no occupants. The eighth requires elapsed strictly less than 210 and the unchanged breach threshold near 210. These prove a completely banked, lossless first-wave finish before breach rather than a partially filled cabin at the deadline.

The ninth requires exactly five completed flights, at least 15 cluster arrivals, and visitedGroups equal (1u<<15) - 1. Those conditions establish visits to all 15 released groups and the four full eight-person loads plus final four-person load implied by capacity and total 36. Reconciliation supplies the seven debrief accounting checks.

The final print reports travel seconds, elapsed seconds, rescued out of 36, aboard, lost, flights, clusters, minimum clearance, and approach reversals. The case then closes. Output reports are diagnostics, not extra Boolean assertions. Exact numeric travel durations are determined by execution and should not be invented from this source alone.

Analytic Route and Hazard Cases

Test Lines 668-705: Case 16, Route, Heading, Breach Freeze, and Worm Gap

The first part begins with fresh and samples route times 0,6,25,60,104,150,210. For each, it independently constructs x and z from the documented sinusoidal formulas and terrain height. Its first assertion requires the route helper to match that expected three-dimensional position within 0.001.

It converts route yaw to radians, constructs negative-Z-model forward, and independently calculates the derivative tangent. The second assertion requires finite yaw and a dot product greater than 0.99999 between heading and normalized tangent. A unit-vector dot product near one means the directions nearly coincide, rather than merely having similar angle numbers modulo 360.

It calls updateHarvester(t). The third assertion requires current position to match expected within 0.001 and yaw difference, wrapped with remainder, to have magnitude below 0.001. That checks both the pure helpers and the state-mutating helper. Three checks are executed at each of the seven sampled times.

After the final sample, the case snapshots breach position, yaw, and travel, then calls updateHarvester at breach plus 50. A combined assertion requires all three unchanged, checking the clamp and zero additional travel beyond the endpoint. It does not compare accumulated travel with an exact analytic arc length.

The second part calls fresh again and sets previousGap to 221, larger than the initial expected 220. It samples 0,1,30,75,104,150,209,210,230 by assigning elapsed and updating the harvester. Expected gap is independently calculated as $220 - 202 * \text{clamp}(t/\text{breachAt}, 0, 1)$.

At each of nine times, one check requires gap near expected and no greater than the previous sampled gap. Another requires worm x/z to match current harvester minus expected gap in x within 0.001. A third requires stored wormYaw near 90 and worm y near its own terrain height. It then records the gap for the next monotonic comparison.

This is a geometry test using deliberate clock assignments. It proves the worm relation, monotonic sampled closure, terrain following, and fixed stored yaw. It does not move an aircraft through a route or test whether the worm model's independently animated pose matches every geometric point.

Test Lines 707-748: Case 17, Frame-Rate-Stable Scheduled Release

Empty reference vectors are created for crew and groups. The outer loop runs at 30,60,120 updates per second. Each frequency starts with fresh; an inner loop performs 30 simulated seconds of updates. Before each update, elapsed is explicitly set to (tick-1)/frequency, then the updater receives 1/frequency.

These clock assignments intentionally remove accumulated-float drift from this fixture. This test is about release pose and walking time at equal absolute endpoints, not a proof that a long real clock accumulates identically across frame rates. The source comment contrasts it with the previous controller, which never edits the real clock.

The first, 30-Hz run is saved as reference. At later frequencies, every crew member must have the same state, horizontal position within 0.01, and full destination within 0.001. Every group's released flag must match and center must be within 0.001. At time 30, groups scheduled at six,13,20,27 have released; later groups remain Inside. Comparing all 36 therefore covers both released and unreleased populations.

The next part tests a release-straddling coarse frame. Starting from elapsed 5.99, it updates the harvester and runs dt 0.02, ending near 6.01. It saves first-person position/destination and first group center. A second fresh setup starts at 5.75, updates the harvester, and runs dt 0.26, also ending near 6.01.

The final combined assertion requires the two first-person horizontal positions within 0.001 and destination plus center within 0.001 in full distance. Both frames allow only approximately 0.01 second of post-release walking, even though their total frame durations differ drastically. This directly validates `min(dt, elapsed-release)` and use of the scheduled route pose. It does not compare coarse pickup throughput, aircraft displacement, or the other crew hazards across arbitrary frame sizes.

Test Lines 750-774: Case 18, Pursuit Crew and Aircraft Envelopes

Case 18 contains four distinct setups inside one named run. The first delays every group and person release to 1,000, sets elapsed to 100 before the default breach, and updates the harvester. Person zero becomes Running at worm plus 15 x units; person one becomes Waiting at worm plus 17. Destinations equal their positions.

After dt 0.01, the first assertion requires person zero Lost, person one Waiting, and loss count one. That distinguishes the 16-unit pursuit radius. The second requires 34 still Inside and phase Flying, showing ordinary pre-breach pursuit does not swallow Inside people or end a remote aircraft's mission.

The craft is then teleported at the worm with altitude 30. After another 0.01, phase must remain Flying and planeLost false, proving it is above the 22-unit aircraft height envelope. Teleporting at altitude 21 and updating 0.01 must instead produce Debrief, planeLost true, and all 36 Lost. Reconciliation verifies that the earlier local loss was not counted twice when the crash lost everyone else.

The locations are set relative to the worm at setup, and the worm moves slightly in the following update. The chosen distances 15/17 and heights 30/21 have margins, so this is not an exact equality test for 16, 18, or 22. Aircraft y also passes through normal smoothing before the hit test.

Test Lines 776-795: Case 18, Early Breach Crew and Aircraft Envelopes

A new fresh setup again delays release and places elapsed at breach plus four. People zero and one become Waiting at worm plus 31 and 33 x units. The craft is placed at worm altitude 45, then dt 0.01 runs.

The crew assertion requires the 31-unit person Lost and the 33-unit person still Waiting, distinguishing the active 32-unit outdoor breach radius before swallowing. The phase assertion requires Flying, because the aircraft remains above the 40-unit height envelope.

The aircraft is then placed at worm altitude 39 and another dt 0.01 runs. A combined assertion requires planeLost and Debrief, demonstrating the active breach aircraft envelope at horizontal distance near zero and height below worm terrain plus 40. Reconciliation follows. This segment does not independently assert a specific final loss counter; the reconciliation checks actual Lost states against the counter.

Test Lines 797-822: Case 18, Exact Start and Retreat Boundaries

The third fresh setup delays release and sets elapsed exactly eight. The first person is manually Waiting at the current worm position. `updateMission(0)` must leave that person Waiting because pursuit activation uses elapsed greater than eight, not greater than or equal. A subsequent 0.01 update must mark them Lost and set loss count one.

The fourth setup resets again, delays releases, and places elapsed at breach plus 18. The first person is Waiting exactly at the worm; the aircraft is remote at base. A zero-dt update must yield 35 losses, one Waiting person, and phase Flying. The 35 Inside people are swallowed because attack is already at least 12, but outside danger has ceased because its upper bound is strictly less than 18.

Reconciliation confirms one unresolved survivor remains. This scenario highlights that the retreat boundary does not revive prior Lost people and does not stop the unconditional Inside-crew loss rule. The closing lambda brace ends the whole four-part case.

Cursor and Steering Cases

Test Lines 824-856: Case 19, Cursor Arithmetic and Resize

The case snapshots aircraft position, yaw, and elapsed after fresh reset. Center cursor (500, 250) in a 1000x500 window must produce hover length below 0.001.

The horizontal sample loop uses x values 440, 460, 500, 540, 560 with centered y. Each must give x hover near zero, including the two exact nominal deadzone edges. The vertical sample loop uses y values 220, 230, 250, 270, 280 with centered x; each y hover must be near zero. These are five assertions per loop, not a single check of one representative center.

At $x = 570$, horizontal hover must lie strictly between zero and one; at $x = 430$ it must lie strictly between -1 and zero. At the left window edge, hover must be $x = -1$ and $y = 0$; at the right edge, $x = 1$ and $y = 0$.

At (800, 100) the raw normalized coordinates are (0.6, -0.6). After deadzone, the assertion expects (0.48/0.88, -0.48/0.88) within near tolerance. Doubling coordinates and dimensions to (1600, 200) in 2000x1000 must preserve that vector within 0.001. The new center (1000, 500) must return neutral hover.

The final combined assertion requires aircraft position unchanged, yaw unchanged, elapsed unchanged, and stored turn rate still zero. Repeated arithmetic input changes do not simulate a frame. This case validates normalized behavior after resize, but it does not create or resize an actual GLFW window or test event callback coordinate scaling.

Test Lines 858-878: Case 20, All Invalid Cursor Scenarios

The case obtains quiet NaN and positive infinity from `std::numeric_limits<double>`. A local `Cursor` record stores two double coordinates and integer width/height. Its array contains 14 invalid inputs, each tested independently after reseeding active hover.

The four outside-window scenarios are $x = -0.01$, $x = 1000.01$, $y = -0.01$, and $y = 500.01$ in the original 1000x500 dimensions. Two NaN scenarios put NaN in x or y. Four infinity scenarios put positive or negative infinity in either coordinate. Four invalid-size scenarios use width zero, height zero, width -1, or height -1 with zero cursor coordinates.

For each array entry, the case first supplies valid right/top corner hover and requires its vector length above 0.5. It then submits the invalid input and requires both components finite and total length below 0.001. Thus there are two assertions for each of the 14 scenarios: active setup and safe clearing.

The reseeding is important. Without it, an implementation that merely ignored invalid input could appear correct after the first zeroed result. These checks ensure invalid input actively clears previous steering on both axes, without producing NaN. Accepted exact window edges are covered separately by Case 19, so slightly outside values here intentionally test strict rejection.

Test Lines 880-908: Case 21, Hover Turns Without Clicks and W Follows It

The outer loop runs side -1 and side +1. Each begins fresh, records position, sets left edge for negative side or right edge for positive side, and advances 0.5 second with no keyboard movement or mouse-click input.

The first assertion requires yaw times side below -1 and stored rate times side below -1: right produces negative yaw/rate, left positive. The second requires rate near $-side * 80 * (1 - \exp(-8 * 0.5))$, validating the exponential approach at the half-second endpoint. The third requires horizontal position unchanged within 0.001 and speed near

zero. The fourth requires forward.x times side positive and forward.z negative, proving the new nose points toward the hovered side while still generally facing negative Z.

The cursor is then centered and two seconds pass to damp residual turning. The case records settled yaw, forward, and departure position, holds W, and advances 0.5. The next assertion requires displacement length above one, normalized displacement dot settled forward above 0.999, and displacement.x times side positive. The final assertion requires yaw change from the recorded settled value below 0.001.

Both left and right scenarios therefore prove stationary yaw steering, the correct sign convention, no unintended translation from hover, forward movement in the newly chosen heading, and heading retention during centered forward flight. The method does not require any button press for mouse steering.

Test Lines 910-921: Case 22, Arrow-Key Stationary Backup

For side -1 and +1, each fresh setup records aircraft position and holds the corresponding Left or Right arrow for 0.5 second. A sign assertion requires yaw*side and $\text{stored turnRate*side}$ below -1, matching hover steering's sign.

A second assertion requires horizontal position unchanged within 0.001 and speed near zero. This confirms arrows rotate a stationary craft without injecting translational input. It does not separately test simultaneous mouse-plus-arrow saturation, both arrows held together, or Up/Down keys; those behaviors must be inferred only where the game expressions actually define them.

Test Lines 923-946: Case 23, Every Translation Key at Wrapped Headings

The key array is W, S, A, D. The heading array is 0, 90, -90, 179, -181, 359, 361. The outer loop independently constructs forward $(-\sin, 0, -\cos)$ and right $(\cos, 0, -\sin)$ for each heading. The inner loop performs all four keys, giving 28 combinations.

Each combination resets, assigns the chosen yaw, and first asserts `flightForward()` matches the independently calculated vector within 0.001. Expected motion is forward for W, reverse forward for S, right for D, and reverse right for A. After recording position and holding the chosen key for 0.5 second, it forms horizontal displacement.

The second assertion requires velocity length above one and normalized velocity aligned with expected by dot product above 0.999, plus displacement length above one and its normalized direction likewise aligned. The third requires wrapped yaw difference from the assigned heading below 0.001 and stored turn rate near zero.

There are three assertion calls per combination, or 84 total within this case. The unusual headings check equivalent angles around wrap boundaries: 179 and -181 represent the same direction; 359 and -1 would as well; 361 is equivalent to one. The yaw storage can wrap during an update without failing because comparison uses remainder. Reverse and strafe are explicitly verified not to turn the nose toward velocity.

Test Lines 948-962: Case 24, Centering Damps Inertia Rather Than Recentering Nose

The case steers right for 0.5 second, snapshots yaw and turn rate, centers the mouse, then advances 0.25. The first assertion requires the new stored rate near $\text{oldRate} * \exp(-8 * 0.25)$. The second requires yaw smaller than its previous value and rate still negative, demonstrating continued turn while inertia decays rather than an instant snap.

Two more seconds must reduce rate magnitude below 0.001. After recording that settled yaw, another 0.25 must change it by less than 0.001. A centered mouse thus means stop requesting turn, not aim the nose back to zero degrees. These four checks cover the release transient, eventual settling, and retained heading.

Test Lines 964-980: Case 25, Exact One-Second Turning at Three Frequencies

Two three-element arrays store yaw and rate. The frequency array is 30, 60, 120. Each run resets, applies right-edge hover, and calls the updater exactly frequency times with $\text{dt } 1/\text{frequency}$, simulating one second without the advance helper.

The rate assertion requires absolute error below 0.002 from $-80 * (1 - \exp(-8))$. The yaw assertion uses the exact one-second integral $-80 * (1 - (1 - \exp(-8))/8)$ and also requires error below 0.002. Two checks run at each

frequency.

After the loop, a final combined assertion requires 30-Hz yaw and 60-Hz yaw each within 0.002 of 120-Hz yaw. This is a seven-check case. It is strong evidence for the analytic angular integration under constant steering, but it does not show equivalent translational trajectories, roll animations, target switching, pickup totals under very coarse steps, or arbitrary changing-input schedules.

Camera Cases and Test Result

Test Lines 982-1006: Case 26, Every Camera Behind Heading

After fresh reset, headings 0, 90, -90, 179, -181, 359, 361, 720 are tested. The addition of 720 checks multiple full rotations, because trigonometric direction should be unchanged. For each heading, the case computes flight forward and aircraft right. It then tests vertical cursor positions 0, 250, 500 in a 1000x500 window, and camera modes 0, 1, 2.

That gives eight headings times three vertical inputs times three modes, or 72 combinations. For each combination, the offset and its horizontal projection are calculated. The first check requires all offset components finite and y positive. The second requires nonzero horizontal length and normalized horizontal offset dot forward below -0.999, proving the camera is directly behind rather than merely somewhere elevated.

The case computes `viewRight = normalize(cross(-offset, up))`, treating the direction from camera toward target as `-offset`. The third check requires its dot with aircraft right above 0.999. This ensures D/right movement is screen-right in the corresponding look-at orientation rather than visually reversed.

Exactly one mode-specific fourth check runs per combination. Mode 0 requires offset length near 40. Mode 1 subtracts $(0, 2.2, 0)$ before requiring length near 20. Mode 2 requires y near 100 and horizontal length near 26. The branch structure means each combination receives four checks, for 288 assertion executions.

This verifies helper geometry and screen-right basis, not an OpenGL image, projection matrix, camera-target offset used elsewhere, or a real mouse callback. Mode 2 is checked at all three vertical cursor positions even though its offset deliberately ignores camera pitch.

Test Lines 1008-1036: Case 27, Vertical Pitch Mapping, Orbit Clamp, and Both Reset Modes

The case begins centered and requires pitch 12. At the top edge, it records pitch and checks hover magnitude near one plus exact mapping to the clamped expression `12-hover.y*22`. At the bottom edge, it performs the corresponding check again. A fourth check requires the lesser of these pitches near 6 and the greater near 34.

With bottom-edge hover still stored, an orbit loop uses -100, 0, 15, 100. For each, it writes `g_camOrbitPitch` and checks pitch against `clamp(12-hover.y*22+orbit, 6, 65)`. Those inputs exercise extreme low/high clamping and intermediate adjustment. They demonstrate that the full 6-to-65 range includes orbit contribution, while mouse extremes alone span 6-to-34 under zero orbit.

The final loop runs `start=false` and `start=true`. Before each reset it installs right/top hover, stored turn rate -60, and orbit 35. Calling `resetMission(start)` must center both hover axes, clear turn inertia, and restore yaw zero. A second check requires orbit zero and derived camera pitch 12. These are two checks per reset mode.

The scenario proves camera and steering state restoration for both title and playing resets. It does not explicitly check their phase values here because title and flying phase selection are already checked elsewhere. There are 12 assertion executions in this case: four initial mapping checks, four orbit checks, and four reset checks.

Test Lines 1038-1041: Console Summary and Process Result

After all 27 named cases, the function prints a summary containing successful cases over total cases, total checks, and total failures. Successful cases are calculated as `cases - failedCases`, not as checks minus failures. The checks total depends on repeated loops and, for the input controller, the route's visited clusters and return legs.

It returns zero when failures is zero and one otherwise. A caller can use that result as an executable exit code for automation. The code does not return the number of failures or use different exit codes for different cases.

Standard-output case labels and diagnostics plus standard-error assertion messages provide the human-readable detail.

This chapter explains what the suite checks; it is not an execution log or a claim that a build was run while writing the documentation. The build-and-run documentation describes the actual test invocation and any independently observed run results.

Important Limits

The implementation and tests are deliberately small and deterministic. Understanding their actual boundaries is as important as understanding the successful scenarios.

- Best score is process-local mission memory here. Reset preserves it, but this header provides no persistent save file.
- Aboard and Safe are logical states. Stored crew positions are not updated to aircraft or base coordinates when those states change.
- The unresolved counter called `waiting` includes `Inside` and `Running`, not merely `Waiting`. Natural completion requires no unresolved people and no cabin occupants, not a special victory flag.
- Harvester movement and release destinations use analytic route formulas. Harvester accumulated travel is sampled chord length, and translational position integration is not an exact continuous integral.
- Angular rate and yaw have an explicit exponential solution under a constant steering command. This does not make all controls, animation values, dwell transactions, or hazards invariant under arbitrarily large `dt`.
- Pickup and unload progress reset on interruption. Pickup additionally resets when nearest target changes. One update boards at most one person, and excess progress is discarded.
- Local crew hazard, local aircraft hazard, `Inside` swallowing, and final timeout are four distinct rules with different radii and time intervals.
- Clearance assistance is not a collision model for rendered rocks or the harvester. The only explicit aircraft-loss checks in this header are worm envelopes.
- Mouse invalid-input tests cover finite and nonfinite cursor data, but gameplay does not similarly validate every externally supplied `dt` or every externally mutated global.
- Most transition tests use deliberate fixture mutations or teleportation. The input-only controller is the specific case proving an actual key-driven complete first-wave rescue before the unchanged breach timer.
- The controller's no-reversal assertion applies within each flight leg. Opposite commands on separate return legs are expected and are exercised.
- Headless mission tests do not verify rendered pixels, model orientation in a screenshot, graphics driver behavior, real-time audio playback, window callbacks, or asset loading.
- Later waves have shorter timers, but the exhaustive input-only rescue scenario establishes full pre-breach completion for wave one, not every difficulty wave.

Coverage Reference

The game explanation covers original lines 1-307, including both enumerations, all three structures, six crew states, three mission phases, every default field, six game globals/constants, all eight rock sites, every helper, both reset loops, tank records, finish behavior, and every update branch. The headings divide the file into source-order ranges so a reader can match each nonblank line to its explanation.

The test explanation covers original lines 1-1041. It explains the namespace and dependencies; all five `Results` fields; both `Results` methods; `near`, `count`, `fresh`, `advance`, `hover`, `staging`, `readyCrew`, `collect`, `deliver`, and `reconciled`; all 27 named cases; every test-side loop and helper `lambda`; the individual scenario assertions and repeated assertion families; and the final summary and return value.

For the long input-only case, source ranges 492-504, 505-529, 530-550, 551-569, 570-590, 591-617, 618-646, 647-666 separate setup, instrumentation, controller decisions, keyboard pulse conversion, movement accounting, delivery, crew visits, and final assertions. For the combined worm-envelope case, ranges 750-774, 776-795, 797-822 distinguish pursuit, early breach, and exact activation/retreat boundaries. These subdivisions preserve complete coverage without hiding complex test logic inside a single unexplained case title.

Chapter 5: The Rescue HUD and the Procedural Sandworm

Reading This Chapter

This chapter explains all of `arrakis_hud.h` and `arrakis_worm.h`, including their declarations, constants, helper functions, loops, conditions, rendering calls, failure paths, and cleanup. The HUD is the two-dimensional information layer drawn over the game. The sandworm is a three-dimensional object built from mathematical surfaces rather than loaded from a model file. Their purposes are different, but both turn game state into visible geometry.

All source references below use the original root-level files read for this chapter: `arrakis_hud.h`, lines 1-552, and `arrakis_worm.h`, lines 1-360. Their final project paths are `src/arrakis_hud.h` and `src/arrakis_worm.h`; the original references identify the same code after relocation. The explanations are not substitutes for the source appendix: they describe what the code means and why its operations are arranged this way without reproducing the complete source listings.

For a beginner, three distinctions are important. A value in a C++ structure is ordinary CPU data. A VBO is an OpenGL buffer containing data for the GPU. A shader is a small program that runs as part of drawing on the GPU. Generating a triangle in a CPU vector does not itself put a pixel on the screen; the program must eventually upload the vector and issue a draw call. Likewise, moving a mathematical centerline does not automatically update a worm mesh already stored on the GPU; the changed vertices must be sent again.

The HUD Header and Its Input Contract

Header Guard, Includes, and State Structure

****Source:** `arrakis_hud.h`, lines 1-28.**

Lines 1-2 start an include guard named `ARRAKIS_HUD_H`. The first inclusion defines the name and processes the contents. Later inclusions in the same translation unit skip the protected contents, avoiding duplicate declarations. The matching end directive is at line 552. The guard is a compile-time device; it is unrelated to whether OpenGL resources have been initialized at runtime.

The includes in lines 4-12 supply the facilities used throughout the header. GLAD supplies OpenGL types, constants, and loaded function entry points. GLM supplies vectors. `<algorithm>` provides clamping, minimum, and maximum. `<array>` provides fixed-size arrays for glyph rows and saved capability state. `<cmath>` supplies finiteness tests, rounding, and trigonometry. `<cstdint>` provides `std::size_t` and `offsetof`, which are important for buffer byte sizes and vertex attribute offsets. `<cstdio>` provides bounded formatted output and diagnostic printing. `<string>` provides owned strings and conversions. `<vector>` provides resizable arrays of geometry and markers.

`RescueHudState`, opened at line 14 and closed at line 28, is a snapshot of information to display. It does not run the mission, advance the timer, move the aircraft, or change crew states. The caller fills it from the game and passes it to the HUD. Its member initializers make an unmodified snapshot internally usable, but they are defaults rather than a second source of gameplay rules.

width and height default to 1280 and 720. These are framebuffer pixels, not necessarily the logical window dimensions reported by an operating system. On a high-density display, a window can have more framebuffer pixels than screen-coordinate units. The HUD uses these values for its viewport and derives a separate layout canvas from them.

Line 16 specifically explains `waiting`: it includes crew in both Running and Waiting states who are outside the harvester, not workers who are still inside. This matters because the UI later labels that count SAND; it is not merely the number of stationary people. Line 17 declares seven integers. `rescued` is the count secured by delivery to base and initially zero. `aboard` is the count in the aircraft, initially zero, and is visually compared with eight seats. `waiting` is the outside count just described. `lost` is the number no longer rescueable. `total` defaults to 36, the mission population displayed beside the secured count. `best` stores the record used on the results screen. `wave` starts at one and

identifies the mission or wave. All seven are numbers supplied by the caller; the HUD does not reconcile contradictory counts.

Line 18 declares `remaining`, initially 210 seconds; `altitude`, initially zero and displayed in meters; `speed`, initially zero; and `boost`, initially one. `Boost` is drawn as a normalized meter, so one means a full displayed bar. Line 19 adds `missionDuration`, initially 210, as the denominator for the pre-breach timer bar; `pursuit`, initially zero, as a normalized threat bar; `wormDistance`, initially 220, as the displayed range; and `extractionRemaining`, initially 60, as the post-breach countdown. A changing countdown and its total duration are different quantities: without the denominator, a progress meter could not tell what fraction remains.

Line 20 supplies `inside`, initially 36, and `releasedGroups`, initially zero. The first means crew still in the harvester; the second means groups released so far. Line 21 supplies `pickupProgress` and `unloadProgress`, both initially zero, and `fps`, initially 60. The two progress values drive the action message and shared action bar. FPS is diagnostic frame-rate information. Line 22 supplies a heading in degrees, initially zero, used only to rotate the player marker on the north-up radar.

Line 23 contains four booleans. `title` initially true requests the introduction overlay. `paused` requests the pause overlay. `ended` requests the result overlay and takes precedence over the title flag there. `muted` changes the telemetry text and color; it does not itself mute audio. The default false values for the last three mean that a default state shows the title screen rather than a paused, ended, or muted mission.

`prompt` in line 24 is an initially empty string. It can override the normal action instruction, and one exact prompt is also recognized when choosing the failure text on the result screen. Line 25 initializes five two-dimensional positions to zero: `player`, `harvester`, `base`, `worm`, and `pickup`. The radar interprets each used vector as ground-plane coordinates. `pickup` is declared but is not referenced by any drawing function in this header; describing it as a visible marker would be inaccurate. `crewMarkers`, in line 26, is a variable-length list of ground-plane crew markers. The header draws every supplied entry without deciding which crew qualify. Finally, `attackTime`, initially -1 at line 27, distinguishes approach from an attack that has begun. The HUD uses only whether it is at least zero; it does not implement the worm's detailed attack phases.

Rendering Preconditions and Internal Namespace

****Source: `arrakis_hud.h`, lines 30-34 and 454.****

The comments in lines 30-33 define the intended environment: a current OpenGL 3.3 core context, loaded GLAD entry points, framebuffer-pixel dimensions, and one owning context until cleanup. Creating or deleting OpenGL objects without the correct current context is outside that contract. A static CPU resource holder does not make GPU object names portable between arbitrary contexts.

The comments also describe the drawing function's state isolation. It saves and restores the program, VAO, array-buffer binding, viewport, polygon mode, write mask, blend setup, and selected enable flags. It does not switch the caller's framebuffer or texture bindings. This means the HUD is drawn into whichever framebuffer the caller has already bound. It is not a routine that automatically finds the screen framebuffer. Nor is the comment a promise to snapshot every possible OpenGL setting: only the state actually saved by `GLState` receives explicit restoration.

Line 34 opens `arrakis_hud_detail`, and line 454 closes it. Keeping implementation helpers in a named internal namespace prevents common names such as `text`, `bar`, and `Color` from colliding with unrelated names in the surrounding program. The three public lifecycle functions after the namespace form the intended entry points.

HUD Data, Colors, and Formatting

Color Palette, Vertex Layout, and Shared Resources

****Source: `arrakis_hud.h`, lines 36-52.****

`Color` is four floating-point components: red, green, blue, and alpha. RGB controls the tint; alpha controls how strongly the fragment contributes when blended over the existing scene. All palette components are expressed on a zero-to-one scale. The constants are `inline constexpr`, so they are compile-time values that can be defined in a header without

ordinary multiple-definition problems.

bone is (0.91, 0.89, 0.82, 1.0), a warm nearly white main text color. muted is (0.59, 0.61, 0.58, 1.0), used for secondary text. cyan is (0.36, 0.85, 0.86, 1.0), connecting the aircraft, safe base, and rescue progress. amber is (0.98, 0.65, 0.27, 1.0), used for crew and warnings. white is pure opaque white. threat is (0.98, 0.36, 0.27, 1.0), the warmer red-orange worm and pursuit color. black is (0.018, 0.025, 0.028, 0.84): it is a very dark panel background that still permits some scene contribution. edge is (0.42, 0.47, 0.46, 0.25), a faint translucent border and empty-bar color.

The HUD's Vertex at line 45 is not the scene mesh's position-normal-UV vertex. It has exactly six floats: pixel-space x and y, followed by r, g, b, and a. Every triangle vertex carries its own color. The simple HUD shader needs no normals, texture coordinates, light calculation, or font atlas.

Resources in lines 46-51 owns program, vao, and vbo, initially zero to mean no object has been created. canvas, initially -1, is the uniform location for the two-dimensional layout size; a uniform location is not an OpenGL object name. capacity, initially zero, counts allocated GPU buffer bytes, not vertices. vertices is the CPU-side batch of all triangles for the current composition. Its vector capacity is distinct from the GPU byte capacity.

resources() in line 52 returns a reference to a function-local static Resources. The object is created once and shared by these inline helpers. Returning a reference avoids copying its object names and vector. This is convenient for a single-context HUD, but it is not a collection of per-window or per-context resources. The static object's ordinary destruction also does not replace explicit OpenGL cleanup.

Finite Values, Counts, Rounded Measurements, and Clocks

****Source: arrakis_hud.h, lines 53-66.****

finite(v, fallback) returns v when std::isfinite accepts it; otherwise it returns the fallback, zero unless specified. It screens out both infinities and NaN. This prevents malformed measurements from turning into invalid positions or unreadable timer calculations. It does not clamp large finite values by itself.

unit(v) first replaces a non-finite value with zero, then clamps the result between zero and one. Bars can therefore receive values below zero or above one without drawing negative widths or filling beyond their background. number(v) converts the greater of zero and the integer argument to a string. Negative counts become 0; positive counts are not otherwise limited. The parenthesized (std::max) syntax also prevents an unfortunate function-like max macro from matching the name on platforms where such macros are present.

decimal(v) creates a 48-byte local character array, clamps a finite-or-zero value to the interval -99999 through 99999, and formats it with %.0f. Despite the function name, the displayed result has zero fractional digits. It rounds according to formatted floating-point output rather than truncating with an integer cast. The clamp limits the text width, and the bounded snprintf receives the full buffer size. Returning out constructs an owned std::string, so no pointer to a dead local array escapes.

clock(v) uses a different policy. It clamps to 0-5999, rounds upward with ceil, and casts the result to an integer number of seconds. Thus a positive fraction of a remaining second continues to display as one rather than immediately becoming zero. Integer division by 60 obtains minutes; remainder modulo 60 obtains seconds. %02d:%02d prints each part with at least two digits, producing values from 00:00 through 99:59. The 16-byte array comfortably holds this representation. A NaN or infinity becomes 00:00, because the default finite fallback is zero. These helpers sanitize display, not game state: they never modify the original snapshot.

Preserving OpenGL State

The Saved-State Structure and Constructor

****Source: arrakis_hud.h, lines 68-90.****

GLState is an automatic scope guard. Its constructor reads the caller's settings before the HUD changes them; its destructor restores those settings when the drawing function leaves scope. This pattern is often called resource acquisition is initialization, or RAII, even though here the managed resource is a restoration obligation rather than a

newly allocated buffer.

The integer fields in line 69 hold the active shader program, bound vertex array, bound array buffer, four viewport numbers, and two polygon-mode values. The viewport entries are origin x, origin y, width, and height. Line 70 stores four blend factors, separately for RGB and alpha, and two blend equations. Line 71 stores the four color-write booleans, one each for red, green, blue, and alpha. Preserving the mask matters because a previous render pass might intentionally have disabled color writes.

The fixed capability array in lines 72-74 names ten switches. Blending combines HUD fragments with the scene. Depth testing could reject overlay pixels based on three-dimensional depth. Face culling could discard triangles because of their winding. Scissoring could clip the HUD to an old rectangle. Stencil testing could mask it according to a previous pass. Rasterizer discard could prevent all fragments. Color logic operations could replace normal blending with bitwise operations. Sample alpha-to-coverage, sample coverage, and sample alpha-to-one could alter multisample behavior. The corresponding ten-element enabled array starts value-initialized and then stores each switch's actual incoming state.

Lines 77-88 query each scalar or array into its matching member using `glGetIntegerv` or `glGetBooleanv`. These queries do not intentionally change the state they inspect. The loop at line 89 visits every capability index from zero to one less than `caps.size()` and records `glIsEnabled(caps[i])`. The shared index ties each enum to the boolean that will later restore it; no capability is inferred from another one.

Destructor Restoration and Non-Copyability

****Source: `arrakis_hud.h`, lines 91-107.****

The destructor rebinds the old program, VAO, and array buffer in lines 92-94. The explicit casts convert the queried integer object names back to OpenGL's unsigned object type. It then restores all four viewport values. OpenGL core contexts use one polygon mode for front and back faces, so line 97 restores `polygon[0]` through `GL_FRONT_AND_BACK`; the comment at line 96 explains why it does not separately restore two independent face modes.

Lines 98-100 restore the separate RGB and alpha factors, separate equations, and per-channel write mask. The loop in lines 101-103 checks each saved boolean. A true value calls `glEnable` for that capability; a false value calls `glDisable`. This is stronger than blindly disabling blend after the HUD: if the caller had blending enabled, blending remains enabled afterward with the original factors.

The deleted copy constructor and assignment operator in lines 105-106 prevent accidentally copying a guard and creating two restoration obligations for the same old state. The closing brace and semicolon complete the type at line 107. This guard does not save buffer contents, uniforms inside programs, framebuffer bindings, textures, or every obscure rendering option. The HUD does not need to change the framebuffer or textures, and its canvas uniform belongs to its own program. Its buffer uploads likewise target its own VBO. Restoring bindings is not the same operation as undoing those intentional resource updates.

Turning HUD Shapes Into Triangles

Shader Compilation With Failure Cleanup

****Source: `arrakis_hud.h`, lines 109-124.****

`shader(type, source)` receives the shader stage, such as vertex or fragment, and a pointer to its source text. It asks OpenGL for a shader object. If the returned name is zero, line 111 returns zero immediately; there is no object to compile or delete. With a valid name, `glShaderSource` supplies one string and a null length pointer, meaning the string is null-terminated. Compilation is then requested.

The local `ok` starts as `GL_FALSE`, and `glGetShaderiv` asks for the compile status. If compilation fails, a zero-initialized 2048-byte log buffer receives the compiler message. `fprintf` sends the `Arrakis HUD shader` diagnostic to standard error, including a newline. The failed object is deleted before returning zero. If compilation succeeds, line 123 returns its object name to the caller, transferring responsibility for later deletion. The function reports

failure but does not throw, exit the application, or continue with an unusable shader.

Triangle, Rectangle, Outline, Card, and Bar Helpers

****Source: `arrakis_hud.h`, lines 126-147.****

`triangle` takes three two-dimensional positions and a color. It obtains the shared CPU vector by reference and appends exactly three vertices, each with the same RGBA values. These pushes preserve order. With the final `GL_TRIANGLES` draw call, each consecutive triple is one triangle. There is no index buffer and no attempt to share corner vertices between HUD shapes.

`rect` treats (x, y) as the upper-left corner and w, h as extents in the downward-y canvas. Non-positive width or height makes it return without appending anything. Otherwise it constructs the four corners and covers them using two triangles, split along the diagonal from the upper-left to the lower-right corner. The first triangle runs through top-left, top-right, and bottom-right; the second uses top-left, bottom-right, and bottom-left. Six vertices are appended, even though the mathematical rectangle has only four distinct corners.

`outline` builds a one-unit top edge at y , bottom edge at $y+h-1$, left edge at x , and right edge at $x+w-1$. This uses rectangle geometry rather than OpenGL line primitives, making thickness explicit in canvas units. The corner areas overlap, so this is not a single stroked polygon. `card` draws its dark background first, its faint outline second, and a two-unit accent strip at the left last. The default accent is edge; warning or action cards can supply a stronger color.

`bar` first draws a full-width edge background, then draws a foreground with width $w * \text{unit}(\text{amount})$. Its default height is four units, although callers often supply three or five. The amount is sanitized and bounded; no explicit condition is needed for zero because a zero-width foreground rectangle is ignored by `rect`. The order of appending determines visual layering when blending later runs. In this batching scheme, creating a card, adding its text, and later drawing an overlay naturally puts later triangles over earlier ones without multiple draw calls.

The Complete Bitmap Font

Five-Bit Rows and Every Glyph Entry

****Source: `arrakis_hud.h`, lines 149-204.****

The font is a small hand-coded bitmap font, five columns wide and seven rows high. It needs no image file or texture. `glyph(c)` returns seven unsigned bytes, but only the low five bits of each byte are meaningful. Row zero is the top row. Bit 16 is the leftmost column, followed by 8, 4, 2, and 1 toward the right. A set bit means a filled square. The comment at line 149 anticipates the optimization in the text renderer: adjacent set bits in one row will become one wider rectangle.

Here is the complete numeric vocabulary needed to decode every entry. Zero is 00000; 1 is 00001; 2 is 00010; 4 is 00100; 6 is 00110; 7 is 00111; 8 is 01000; 10 is 01010; 12 is 01100; 13 is 01101; 14 is 01110; 15 is 01111; 16 is 10000; 17 is 10001; 18 is 10010; 19 is 10011; 20 is 10100; 21 is 10101; 23 is 10111; 24 is 11000; 25 is 11001; 27 is 11011; 30 is 11110; and 31 is 11111. The row lists below therefore identify the exact shape of every supported character, not just its name.

Source Line	Character	Rows From Top to Bottom	How the Shape Is Constructed
152	A	14, 17, 17, 31, 17, 17, 17	A centered three-pixel top, two vertical sides, and a full-width middle crossbar.
153	B	30, 17, 17, 30, 17, 17, 30	A left stem with three nearly full horizontal bars and right edges between them.
154	C	14, 17, 16, 16, 16, 17, 14	Rounded top and bottom; the three middle rows retain only the left edge.

Source Line	Character	Rows From Top to Bottom	How the Shape Is Constructed
155	D	30, 17, 17, 17, 17, 17, 30	A straight left stem and a right edge, with the outer upper and lower right corners omitted.
156	E	31, 16, 16, 30, 16, 16, 31	Full top and bottom bars, a shorter middle bar, and a left stem.
157	F	31, 16, 16, 30, 16, 16, 16	The E pattern without its bottom bar.
158	G	14, 17, 16, 23, 17, 17, 15	A C-like outline with right-side interior pixels and a fuller lower-right closure.
159	H	17, 17, 17, 31, 17, 17, 17	Two vertical stems joined by a complete middle bar.
160	I	31, 4, 4, 4, 4, 4, 31	Full-width top and bottom bars with a centered vertical stem.
161	J	7, 2, 2, 2, 18, 18, 12	A top-right cap, a near-right stem, and a hook returning toward the lower left.
162	K	17, 18, 20, 24, 20, 18, 17	The left stem stays on while the second stroke approaches it and moves away diagonally.
163	L	16, 16, 16, 16, 16, 16, 31	A left stem ending in a complete bottom bar.
164	M	17, 27, 21, 21, 17, 17, 17	Two sides with a pair of inward upper diagonals and centered upper pixels.
165	N	17, 25, 21, 19, 17, 17, 17	Two sides with an upper-left to lower-right internal diagonal in the early rows.
166	O	14, 17, 17, 17, 17, 17, 14	A closed outline with trimmed corners.
167	P	30, 17, 17, 30, 16, 16, 16	A closed upper bowl over a left-only lower stem.
168	Q	14, 17, 17, 17, 21, 18, 13	An O-like bowl altered at its bottom to introduce a diagonal tail.
169	R	30, 17, 17, 30, 20, 18, 17	The P upper bowl followed by a diagonal leg to the right.
170	S	15, 16, 16, 14, 1, 1, 30	An upper right-reaching bar, left upper stroke, middle bar, right lower stroke, and lower left-reaching bar.
171	T	31, 4, 4, 4, 4, 4, 4	A full-width top and centered stem.
172	U	17, 17, 17, 17, 17, 17, 14	Two sides ending in a centered three-pixel base.
173	V	17, 17, 17, 17, 17, 10, 4	Two sides narrow to paired inner pixels and then a centered point.

Source Line	Character	Rows From Top to Bottom	How the Shape Is Constructed
174	W	17, 17, 17, 21, 21, 21, 10	Two outer strokes with a lower central stroke and a paired inner base.
175	X	17, 17, 10, 4, 10, 17, 17	Two diagonals meet at the center and diverge again.
176	Y	17, 17, 10, 4, 4, 4, 4	Diagonals merge into a centered lower stem.
177	Z	31, 1, 2, 4, 8, 16, 31	Full top and bottom joined by a descending right-to-left diagonal.
178	0	14, 17, 19, 21, 25, 17, 14	The closed O outline plus an internal diagonal distinguishing the digit.
179	1	4, 12, 4, 4, 4, 4, 14	A central stem, a small upper-left flag, and a three-pixel base.
180	2	14, 17, 1, 2, 4, 8, 31	A curved top, a diagonal down toward the left, and a full bottom bar.
181	3	30, 1, 1, 14, 1, 1, 30	Three horizontal regions joined by right-side strokes.
182	4	2, 6, 10, 18, 31, 2, 2	A diagonal into a full crossbar with a near-right stem continuing below.
183	5	31, 16, 16, 30, 1, 1, 30	A full top, left upper stroke, and rounded lower-right bowl.
184	6	14, 16, 16, 30, 17, 17, 14	A left-connected upper curve and a closed lower bowl.
185	7	31, 1, 2, 4, 8, 8, 8	A full top and diagonal into a near-left lower stem.
186	8	14, 17, 17, 14, 17, 17, 14	Two closed bowls meeting in a centered middle bar.
187	9	14, 17, 17, 15, 1, 1, 14	A closed upper bowl with a right lower stroke and curved base.
188	/	1, 1, 2, 4, 8, 16, 16	A diagonal from the upper right to the lower left.
189	+	0, 4, 4, 31, 4, 4, 0	A central vertical segment crossing a full-width middle row.
190	-	0, 0, 0, 31, 0, 0, 0	A single full-width middle row.
191	:	0, 4, 4, 0, 4, 4, 0	Two two-pixel-tall centered dots.
192	.	0, 0, 0, 0, 0, 6, 6	A two-by-two lower dot.
193	,	0, 0, 0, 0, 0, 4, 8	A small lower dot with a leftward descending tail.
194	%	25, 25, 2, 4, 8, 19, 19	Paired corner marks separated by a diagonal.

Source Line	Character	Rows From Top to Bottom	How the Shape Is Constructed
195	!	4, 4, 4, 4, 4, 0, 4	A centered stem, gap, and terminal dot.
196	?	14, 17, 1, 2, 4, 0, 4	A curved hook, descending center, gap, and dot.
197	(	2, 4, 8, 8, 8, 4, 2	A parenthesis bowing left in its middle.
198	)	8, 4, 2, 2, 2, 4, 8	The horizontally mirrored parenthesis.
199	=	0, 0, 31, 0, 31, 0, 0	Two complete horizontal bars with a blank row between.
200	_	0, 0, 0, 0, 0, 0, 31	A complete bottom row.
201	space	0, 0, 0, 0, 0, 0, 0	No visible pixels, but the text renderer still advances the cursor.
202	any unsupported character	14, 17, 1, 2, 4, 0, 4	The exact question-mark shape, making unsupported input visibly identifiable.

The switch begins at line 151, returns immediately from each case, and closes at line 203 before the function closes at 204. There is no fall-through and no need for `break` after a returned array. There are 26 uppercase letters, ten digits, 14 supported non-letter entries including space, and one default branch. Lowercase letters are handled elsewhere by conversion to uppercase. Apostrophes and square brackets have no case. Consequently the failure sentence's apostrophe in `WORM'S` and the telemetry string `[MUTED]` do not render literally: each unsupported punctuation character appears as `?`. That is actual behavior, not an extra font feature.

Text Width and Rectangle-Based Rendering

****Source: `arrakis_hud.h`, lines 205-222.****

`textWidth` returns zero for an empty string. For a nonempty string of `n` bytes, it returns $(6*n-1)*size$. Each character has five columns and a one-column gap; the final character does not need a trailing gap. The explicit floating-point conversion avoids performing the full width expression in an unsigned string-size type. Width counts all characters, including spaces and characters whose painted pixels do not touch both outer columns.

`text` walks the string as unsigned bytes. That avoids negative character values when testing ASCII ranges. A byte between `a` and `z` is shifted by subtracting `a` and adding `A`; other bytes retain their value before the cast to `char`. The resulting character selects the seven glyph rows.

The outer row loop runs from zero through six. The inner column loop deliberately has no increment expression in its header because the body advances by either one unlit position or a whole lit run. `16 >> column` creates the mask for the current left-to-right column. If bitwise AND with the row is zero, the negated test is true; the code increments the column and continues without drawing. If the bit is set, `first=column++` remembers the beginning and advances past the first lit pixel. The `while` loop then advances through consecutive set bits, stopping before column five or before an unlit bit.

One rectangle is emitted at `x+first*size, y+row*size`. Its width is the number of pixels in the run multiplied by `size`; its height is one pixel multiplied by `size`. A five-pixel horizontal font stroke therefore requires six triangle vertices, not thirty. After all seven rows, the character origin advances by `6*size`, preserving the one-column inter-character gap. The renderer is proportional only in scale, not in character width: `I`, `M`, and space all advance equally. It also has no newline interpretation; multiline behavior comes from wrapping and the separate `lines` helper.

Word Wrapping, Alignment, and Line Limits

****Source: `arrakis_hud.h`, lines 223-255.****

`wrap` estimates the maximum number of characters that fit with the same width formula used by the font. Solving $(6*n-1)*size \leq width$ gives $n \leq (width+size)/(6*size)$. Line 224 floors that value and takes at least one before converting to `std::size_t`. Actual callers supply positive font sizes; the function is not a general validation routine for zero or negative size.

The function creates an output vector, a pending line, and a pending word. The lambda `append`, capturing these by reference, is a local operation for transferring a completed word into the output. An empty word returns immediately. If a nonempty existing line plus one separating space plus the word would exceed the column count, the existing line is emitted and cleared. A word longer than the available column count is split repeatedly: each leading substring of exactly `columns` characters is emitted, then erased from the pending word. Any remainder is added to the line, with one space first only if the line already contains text. The word is cleared so it cannot be appended twice.

The byte loop treats any byte at or below ASCII space as whitespace. It first flushes the pending word. For a newline specifically, it then emits the current line, even if empty, and clears it. Ordinary spaces, tabs, and other low control bytes do not explicitly force new lines; repeated whitespace is collapsed when words are joined. A non-whitespace byte below 127 is kept, while a byte at least 127 becomes `?`. This is byte-oriented ASCII handling, not UTF-8 decoding, so a multibyte character can become multiple question marks. Unsupported printable ASCII such as an apostrophe survives wrapping but later reaches the glyph fallback.

After the loop, `append()` handles the final word, and a nonempty final line is added. The vector is returned by value. Very long exact-multiple words can leave no remainder, and an explicit newline can add an empty line after such processing; this follows from the separate splitting and newline branches. Normal HUD strings contain manageable words and use explicit newlines mainly to separate introductory instructions.

`centered` subtracts `textWidth` from the supplied region width, takes half of the difference, and offsets the text origin by that amount. It does not clip or shrink text if the difference is negative. This helper is defined but not called elsewhere in this header. `lines` takes the smaller of the content length and a maximum count, whose default is 100, then draws each selected line at $y+i*size*10$. A glyph is seven pixels high, so the ten-pixel step leaves three pixels of vertical separation at the selected scale. Limiting count drops later lines rather than rewriting their contents.

The North-Up Radar

Grid, World Mapping, and Marker Shapes

****Source: `arrakis_hud.h`, lines 257-294.****

`radar` first emits a square card. A loop with `i` equal to one, two, and three draws vertical and horizontal lines at one quarter, one half, and three quarters of side. Each grid line stays one unit inside the outer border and spans `side-2`. The result is a four-by-four reference grid, not an automatically scaled terrain map.

The local point lambda receives a position, color, and integer marker type. For each position component, it replaces non-finite input with zero, clamps to -300 through +300, adds 300, divides by 600, and scales to `side-12`. Adding `x+6` or `y+6` places the useful map inside a six-unit inset. The negative world limit lands at the inner left or top boundary; the positive limit lands at the inner right or bottom boundary. Out-of-range objects stick to the boundary instead of disappearing. The comment in line 265 emphasizes that this map stays north-up even when the heading-locked chase camera turns.

Type zero is the player triangle. Heading is sanitized and multiplied by 0.01745329252, the degrees-to-radians conversion factor. `forward=(-sin(yaw), -cos(yaw))` points up the radar at zero degrees; `right=(-forward.y, forward.x)` is its perpendicular. The triangle's tip is four units forward from the marker center, while the tail center is three units backward. The two tail corners lie four units to either side of that tail center. Passing these three points to `triangle` turns heading into an unmistakable directional arrow. The negative sine and cosine belong to this screen-map convention; they should not be replaced by a guessed three-dimensional forward vector.

Type one is an eight-by-eight outline centered on the position, used for the base. Type two is a filled six-by-six square, used for the harvester. Type four is a filled three-by-three square, used for crew. Every other type uses two triangles meeting along a vertical diagonal to produce a diamond with tips four units above, below, left, and right. The worm is explicitly passed type three and therefore gets that diamond branch.

Drawing order is base, all crew entries, harvester, worm, and player. Later markers can cover earlier ones if they coincide. Crew iteration uses const references to avoid copying each GLM vector. `pickup` has no corresponding call. Labels are optional: the `if( labels )` block emits `SECTOR / +/-300` in muted color 18 units above the map; `YOU / BASE` in cyan nine units below; and `CREW, HARV, and WORM` 26 units below in amber, white, and threat color. The last two begin 47 and 92 units to the right. These are legends, not labels individually positioned beside each object.

Title, Pause, and Results Screens

Overlay Choice and Responsive Measurements

****Source: `arrakis_hud.h`, lines 296-322.****

`over lay` begins by adding a full-canvas dark rectangle with `( 0.01, 0.018, 0.022, 0.78 )`. It darkens the already composed HUD and underlying scene. `result=s.ended` takes priority; the local `title` is true only when the mission is not ended and the input title flag is true. Otherwise the overlay is the pause design. This local precedence also resolves snapshots with more than one flag set.

The horizontal margin is 12 below width 480, otherwise 28. Panel width is the smaller of 680 and the available width after both margins. Interior padding is 14 when the panel is narrower than 420 or the canvas shorter than 420, otherwise 28. Content width subtracts that padding twice. Text pixel scale becomes one below height 300; otherwise it becomes 1.5 for a panel below width 380 or a height below 420, and two for roomier screens. Heading scale is two below height 300, 2.5 for a panel below width 420 or height below 420, and four otherwise. These choices reduce spacing and type size selectively instead of uniformly squeezing a desktop design.

The heading is exactly `MISSION CLOSED, HARVESTER FLIGHT, or FLIGHT PAUSED` according to the selected mode. For a result, the code compares the prompt with the exact string `ORNITHOPTER LOST IN THE WORM'S MAW`. A match chooses `ORNITHOPTER LOST IN THE WORM'S MAW. DELIVERED CREW REMAIN SAFE.`; otherwise it chooses `RESCUE COMPLETE. ONLY CREW DELIVERED TO BASE COUNT. EVERY LIFE COUNTS..` This is a display-message test, not collision detection. The apostrophe survives the string comparison correctly even though its font glyph is unsupported.

For the title mode, four explicit instruction lines state `FLEEING HARVESTER RELEASES 1-4 CREW AT A TIME., AMBER GROUPS WAIT ON SAND. 8 RESCUE SEATS., MOUSE STEERS. FLY LOW. HOLD SPACE TO WINCH., and RETURN TO CYAN BASE. HOLD SPACE TO BANK RESCUED..` Adjacent C++ string literals concatenate into one string containing the specified newline separators. Pause mode uses `THE RESCUE IS ON HOLD. YOUR CREW IS COUNTING ON YOU..` All three possibilities pass through `wrap`, so long lines still break at the current content width.

The key instruction is independently wrapped: `ENTER NEXT MISSION / R RETRY, ENTER START, or P RESUME`. `lineH=size*10` matches the `lines` helper. Results reserve `lineH*3+12` for their three statistical rows and a gap; other modes reserve zero. Panel height adds twice the inset, 23 for the branding step, seven heading pixels at heading scale, 18 after the heading, the result reserve, all wrapped body and key lines at `lineH`, and 25 for the final separator spacing. Horizontal position centers the panel. Vertical position centers it unless that would be less than eight, in which case the top is eight. The card accent is amber for results and cyan otherwise.

These calculations are responsive but not an absolute overflow guarantee. If a framebuffer is extremely tiny or unusually short, the computed panel can extend below it. There is no final panel-height clamp, scroll area, or scissor-based text clipping in this function. It prioritizes readable sizes and lets framebuffer bounds provide the ultimate clipping.

Overlay Content and Cursor Advancement

****Source: `arrakis_hud.h`, lines 323-341.****

The mutable cursor begins at the panel's top inset. The header `ARRAKIS / SEARCH + RESCUE` is drawn in muted color at scale 1.5, after which the cursor advances 23. The selected heading is then drawn in bone color at `headingSize`, and the cursor advances its seven-row glyph height plus 18.

For results only, the first line reads `SECURED <rescued> / <total>` in cyan. The second reads `BEST <max(best, rescued)> / MISSION <wave>` in bone. Taking the maximum means a just-achieved score is visible as the best even if the caller has not yet synchronized its stored record. The third reads `ABOARD <aboard> / LOST <lost>` in muted color. Each count passes through the nonnegative number helper. Cursor increments match `lineH` between rows, then add 12 after the final row.

The wrapped body is drawn in muted color, and the cursor advances by its entire line count times `lineH` plus 12. A one-unit horizontal separator spans `contentW`. Another advance of 13 puts the key instructions below it; these are cyan. This cursor scheme accounts for all the components used in the panel-height expression. The function closes after adding geometry; no rendering or input handling happens inside it.

Composing the In-Game HUD

Global Layout Decisions and the Countdown

****Source: `arrakis_hud.h`, lines 343-355.****

`compose(w, h, s)` is the central geometry builder. Its `w, h` are layout-canvas dimensions, not necessarily raw framebuffer dimensions. Margin `m` is ten when width is below 600 or height below 400, otherwise 22. `inner=w-2*m` is the content width between margins. `compact` is true below width 760 or height 440. `shortView` is true below height 360. These are independent tests: a wide but short screen still enters compact mode.

`extraction` checks `attackTime>=0`. It selects `extractionRemaining` instead of ordinary `remaining`, and a fixed duration of 60 instead of `missionDuration`. Before extraction, the duration is sanitized with a fallback of 210 and constrained to at least 0.001, preventing division by zero in the time bar. A non-finite remaining value is treated as zero in the danger test. Danger is true when remaining time is at most 15 or when extraction is active. Thus the entire extraction phase uses warning styling, not only its last fifteen seconds.

The label precedence is `EXTRACTION WINDOW` for extraction, then `BREACH IMMINENT` for low pre-breach time, otherwise `TIME TO BREACH`. Danger uses amber; ordinary time uses bone. The top card height is 78 for compact short view, 112 for other compact views, and 106 for noncompact views. The initial card spans the full inner width. Label choice, clock formatting, and bar clamping are separate: even inconsistent caller data does not make the displayed fraction overfill the bar.

Full-Width Desktop Header

****Source: `arrakis_hud.h`, lines 356-375.****

The noncompact branch arranges the top card as branding, crew status, and a timer. Branding begins 18 units from the left and 16 from the top with `ARRAKIS` at scale three. `FLEEING HARVESTER` appears at top offset 47, scale 1.5, muted. `SEARCH + RESCUE / <wave>` appears at offset 76, scale 1.5, cyan. The positions are fixed relative to the panel, not inferred from text height.

`scoreX=m+inner*0.30` starts the central status cluster at thirty percent of the inner width. `SECURED <rescued>/<total>` is cyan at scale 2.5 and y offset 15. `ABOARD <aboard>/8` appears at offset 44; `INSIDE <inside>` shares that row 126 units to the right in muted color. A loop draws eight seat slots at x spacing 14, each ten units wide and five high, at y offset 64. Slot `i` is cyan when `i<s.aboard`, otherwise edge. The comparison uses the original count; a negative count fills none and a count above eight fills all, but the code never creates more than eight slots.

`GROUPS <releasedGroups>` is amber, positioned 126 units to the right of `scoreX` on the seat row. `SAND <waiting> / LOST <lost>` is muted at y offset 82. `SAND` follows the documented Running-plus-Waiting outside count rather than the inside count.

`timerX=m+inner-230` anchors the timer cluster near the right. The label is at y offset 15, the clock at 39 and scale three, and a 210-by-five time bar at 74. Its amount is sanitized remaining time divided by the selected duration. At y offset 87, `SECURE CREW AT CYAN BASE` reminds the player that boarding alone is not scoring. The branch ends at line 375 before the alternate compact branch.

Compact and Very Short Headers

****Source: `arrakis_hud.h`, lines 375-392.****

The compact branch uses a combined `ARRAKIS / FLEEING HARVESTER` title at x offset 12 and y offset ten. Its scale is 1.5 when the inner width is below 330, otherwise two. `row` is y offset 26 for short views, 32 otherwise. The timer label sits on that row; the clock is two units higher and right-aligned using its measured width and a twelve-unit inset.

The timer bar is only 74 units wide and three high, ending at that same right inset. Its y offset is 56 for short views but 48 for other compact views. This intentional rearrangement puts it below the seat row when space is especially short. `statsY` is offset 43 for short views and 54 otherwise. At that position, `SAVED <rescued>/<total>` is cyan on the left and `SEATS <aboard>/8` is right-aligned on the right. The change from `SECURED` to `SAVED` shortens the label; it does not change which numeric field is displayed.

The next left-aligned line combines `SAND <waiting> INSIDE <inside> GROUPS <releasedGroups>`, at offset 61 in short view or 76 otherwise. If the view is not short, an extra line at offset 96 shows `LOST <lost> / WAVE <wave>`. Only in that non-short branch, and only if inner width is at least 360, does the code draw a compact eight-seat diagram. Each slot is eight by four, separated by twelve units, beginning 104 units from the right and sitting at offset 99. These omissions reduce clutter without changing any mission values.

Wrapped Controls and the Action Card

****Source: `arrakis_hud.h`, lines 394-422.****

The comment in lines 394-395 explains the design principle: controls wrap at readable pixel sizes instead of shrinking an entire 1280-pixel design. The control font scale is always 1.5. The complete instruction string is `MOUSE STEER / W S THRUST / A D STRAFE / Q E ALT / SPACE WINCH / SHIFT BOOST / C CAMERA / M MUTE / P PAUSE / R RETRY`. It identifies mouse steering, forward/back thrust, lateral strafe, altitude controls, winching, boosting, camera choice, mute, pause, and retry. The HUD merely displays those mappings; actual key handling belongs to the game.

Wrapping uses `inner-20`, allowing ten units on both sides. Controls-card height is fifteen units per wrapped line plus 14. `controlsY=h-m-controlsH` anchors its bottom above the bottom margin. The card spans the inner width, and text starts ten units from its left and eight from its top. A narrow screen makes more lines and a taller bottom card, rather than smaller letters.

The action message initially copies `s.prompt`. An empty prompt chooses one of three defaults in descending priority: with at least eight aboard, `SEATS FULL / RETURN TO CYAN BASE`; with a positive smaller aboard count, `RETURN TO BASE / HOLD SPACE TO BANK CREW`; otherwise `FLY LOW TO CREW / HOLD SPACE TO WINCH`. Active unloading overrides either a prompt or a default with `BANKING CREW / HOLD SPACE`. Only when unloading is not positive does positive pickup progress override with `WINCH ACTIVE / HOLD SPACE`. The `if` followed by `else if` gives unloading explicit precedence if both progress values are positive.

`sideRadar` requires all three conditions: short view, inner width at least 280, and enough room that a 64-unit radar beginning at `controlsY-72` is at least six units below the top card. If true, action width reserves 76 units for that radar; otherwise it uses the full inner width. In either case width is capped at 620. Action text scale is 1.5 for compact mode and two for desktop mode. The message wraps within `actionW-28`, reserving fourteen-unit side insets.

If wrapping produces more than two lines, the vector is resized to two. When the retained last line contains at least three characters, its final three characters are replaced by `...`. This communicates truncation without adding width. A shorter retained line receives no ellipsis. Action height is its retained line count times the line step plus 23. Its bottom sits eight units above the controls card. With a side radar, it is right-aligned to the margin; otherwise it is centered across the whole canvas.

The card gets a cyan accent, and its text begins at x inset 14 and y inset ten. The progress selection uses unloading if it is positive, otherwise pickup. The bar is three units high, fourteen units inset from both sides, and nine units above the card's bottom. Its unit sanitization handles out-of-range values. The message-selection conditions themselves do not call `finite`; for example, a NaN progress fails the positive comparison, while a positive infinity selects the active message but produces a zero displayed bar through `unit`. This is the precise split between display formatting and raw conditional state.

Radar Placement, Telemetry, and Room-Dependent Omission

****Source: `arrakis_hud.h`, lines 424-442.****

The normal radar begins below the top card by ten units in compact mode or 32 on desktop. `available` measures space from that radar top to the action card, subtracting six units in compact mode or 52 on desktop. The larger desktop reserve accommodates legends and breathing room. Requested side length is capped at 92 for compact mode and 156 otherwise, but cannot exceed the available vertical space.

If `sideRadar` is true, the special radar is drawn at the left margin, 72 units above the controls top, with side 64 and no labels. Otherwise, a normal radar is drawn only when the computed side is at least 40, at the right margin. Labels are enabled only for a noncompact radar at least 128 units wide. The `else if` means the two radar placements never both execute.

Telemetry appears only when there is no side radar, at least 88 units of vertical room, and either the view is noncompact or inner width is at least 420. Its width is 184 in compact mode and 222 otherwise. `pursuitRoom` checks available height at least 112 and chooses a 112-unit card instead of an 88-unit card. This gate prevents a pursuit row from being added when only the basic telemetry fits.

The first row is ALT <rounded altitude> M at y offset twelve. The second is SPD <rounded speed> / FPS <rounded fps>, at offset 31. When muted, it appends [MUTED] and uses amber rather than muted color. The square brackets fall back to question-mark glyphs, as described in the font section. BOOST appears at offset 52; its five-high cyan bar begins x offset 72 and y offset 55 and spans `tw-86`, leaving a right inset of fourteen.

WORM RANGE <rounded distance> M appears at offset 72 in amber. Its distance is made finite and clamped to a minimum of zero before passing through `decimal`, unlike altitude and speed, which may legitimately format negative values. If pursuit room exists, PURSUIT appears at offset 92 in amber and a five-high threat-colored bar begins x offset 86 and y offset 95, with width `tw-100`. Both normalized bars use the generic bar helper and therefore bound their foreground to the background.

Flight Guides and Overlay Order

****Source: `arrakis_hud.h`, lines 443-452.****

Guides are emitted only when the mission is not ended, not on the title screen, and not paused. A local copy of cyan has alpha reduced to 0.35. Nested range loops choose dx and dy from -1 and +1, producing all four combinations. Guide centers lie at 44 or 56 percent of canvas width and height: `0.5+sign*0.06`.

At each position, one seven-by-one rectangle and one one-by-seven rectangle create an L-shaped corner. Positive dx subtracts seven from the horizontal bar's starting x, so it extends inward toward the screen center; negative dx does not. Positive dy similarly subtracts seven from the vertical bar's y. Thus the four marks outline a quiet central aiming or flight region rather than drawing a full crosshair over the scene.

Finally, if any of ended, title, or paused is true, `overlay` is called. Because it is called after all other composition, its dimmer and panel triangles are appended last and cover the normal interface. The underlying HUD is still composed rather than omitted; the overlay darkens it. Closing line 452 ends `compose`, and line 454 ends the internal namespace.

HUD Initialization, Drawing, and Cleanup

Shader Sources and Program Linking

****Source: `arrakis_hud.h`, lines 456-491.****

`initRescueHud` introduces a short namespace alias `hud`, retrieves the shared resources, and immediately returns if the program already exists. This makes normal repeated initialization calls cheap. It also means the `program` field is the initialization sentinel; the function assumes a nonzero existing program belongs to a valid resource set in the current owning context.

The vertex shader consists of adjacent C++ string literals with explicit newline escapes and requests GLSL `#version 330 core`. Attribute `location zero` is a two-component position; `location one` is a four-component color. `canvas` is a two-component uniform and `tint` is a four-component output. The main function assigns the input color to `tint` and converts pixel-style layout coordinates to clip coordinates.

For `x`, `position.x/canvas.x*2-1` maps zero to -1 and the right edge to +1. For `y`, `1-position.y/canvas.y*2` maps the top to +1 and the bottom to -1, reversing the down-growing canvas convention into OpenGL's up-growing clip convention. The resulting `z` is zero and `w` is one. Because no perspective division changes this `w`, the HUD is a flat overlay. The fragment shader receives interpolated `tint` and writes it directly to its output fragment; it does not sample a texture, apply lighting, or discard pixels.

The helper compiles both stages. If either is zero, the `if` in lines 473-477 deletes whichever stage did succeed and returns. Compiling both before checking means a fragment stage can exist even when the vertex stage failed, and this branch handles that case. Program creation has its own zero check; failure deletes both compiled shader objects and returns.

With a program, the code attaches both stages, links, and deletes the stage objects. OpenGL retains what the linked program needs; the application no longer needs the temporary shader handles. The link-status query starts from false. Failure obtains a bounded 2048-byte program log, prints it with the `Arrakis HUD Link:` prefix, deletes the unsuccessful program, and returns. No failed program is installed in `Resources`.

Vertex Array Setup and Binding Preservation

****Source: `arrakis_hud.h`, lines 492-510.****

Before configuring vertex storage, the function queries the old VAO and array-buffer bindings. It generates one HUD VAO and one VBO. If either name is zero, it deletes any object successfully generated, resets both fields to zero, deletes the otherwise linked program, and returns. Since this failure occurs before the new objects are bound, there are no new binding changes to undo in that branch.

After successful object creation, `r.program` receives the linked program and `r.canvas` receives the location of its canvas uniform. The VAO and VBO are bound. Both attributes are enabled. Location zero reads two `GL_FLOAT` values starting at byte offset zero, with no normalization and stride `sizeof(hud::Vertex)`. Location one reads four floats starting at `offsetof(hud::Vertex, r)`, the red field, with the same stride. The cast to a pointer type supplies a buffer byte offset to OpenGL; it is not dereferencing an arbitrary CPU pointer.

The old VAO and array-buffer bindings are restored explicitly. Initialization has not called `glUseProgram`, so it does not need to restore an active program. Finally, the CPU vector reserves room for 65536 vertices. `reserve` changes capacity without adding elements; the first composed frame still begins from an empty vector. It is a starting allocation, not a maximum geometry limit. No GPU vertex storage is allocated here; the drawing function manages that dynamically.

Frame Composition, Render State, Buffer Orphaning, and Draw Call

****Source: `arrakis_hud.h`, lines 512-540.****

`drawRescueHud` first returns for nonpositive framebuffer width or height. This avoids division by zero and is appropriate for a minimized or otherwise undrawable framebuffer. It calls initialization, retrieves resources, and returns if initialization left no program. Only then does it construct `GLState saved`, making restoration automatic for the subsequent draw operations.

The scale is the greater of one and the smaller of `width/1280` and `height/720`. This chooses the limiting dimension for proportional enlargement but forbids scaling below one. A 640-by-360 framebuffer therefore uses a 640-by-360 layout canvas rather than a shrunken 1280-by-720 one. A 2560-by-1440 framebuffer uses scale two and a 1280-by-720

canvas; one canvas unit then covers two framebuffer pixels. An unusually wide screen with only 720 pixels of height stays scale one because height is limiting. $w = \text{width}/\text{scale}$ and $h = \text{height}/\text{scale}$ define the dimensions passed to layout and to the vertex shader.

The vertex vector is cleared without normally releasing its capacity, then `compose` appends the entire frame. All ten capabilities named by the saved guard are disabled. Blend is then reenabled, with both equations set to addition. RGB factors are source alpha and one minus source alpha, giving the familiar translucent overlay. Alpha factors are one and one minus source alpha, so output alpha accumulates as source alpha plus destination alpha times its remaining fraction. This separate alpha setup is not identical to using source alpha for both RGB and alpha.

All color channels are made writable, polygon mode is forced to filled triangles, and the viewport is the entire framebuffer starting at zero. Disabling depth test prevents the HUD from being hidden by scene depth. Disabling the other named capabilities prevents leftover scissor, stencil, wireframe, discard, and similar settings from corrupting the overlay. The function does not clear color or depth buffers. The caller's framebuffer remains bound, and its existing image becomes the background for blending.

The HUD program is selected and its canvas uniform receives layout width and height. Its VAO and VBO are bound. `bytes` is the vector length multiplied by the actual vertex structure size. If bytes exceed GPU capacity, capacity becomes the greater of the required byte size and twice the old capacity. First use therefore grows from zero to at least the required amount; later growth tends to be geometric, reducing repeated reallocations. If the frame uses fewer bytes, capacity is retained.

`glBufferData` is called every frame with that retained capacity, a null source pointer, and `GL_DYNAMIC_DRAW`. The null pointer requests new storage without copying old data. This is called orphaning: the driver can let a previous draw finish reading the old storage while providing new storage for the current frame. It is a performance technique, not a guarantee that every driver will avoid every stall. `glBufferSubData` then copies exactly `bytes` from the CPU vector to offset zero of the new storage.

`glDrawArrays` uses `GL_TRIANGLES`, starts at vertex zero, and consumes the whole vector after conversion to `GLsizei`. All card, glyph, radar, and overlay triangles are rendered in this one call. When the function closes at line 540, `saved` is destroyed and restores the previously described state. CPU vertices remain in the shared vector until next frame's clear or final cleanup; there is no per-shape OpenGL call hidden in the geometry helpers.

Explicit Cleanup and End of Header

****Source: `arrakis_hud.h`, lines 542-552.****

`cleanupRescueHud` retrieves resources and conditionally deletes the VBO, VAO, and shader program, in that order. Each zero check avoids treating an uncreated resource as a live one. The chained assignment resets all three object fields to zero. The uniform location returns to -1 and the GPU byte capacity to zero. Swapping the CPU vector with a freshly constructed empty vector releases its allocation, unlike `clear`, which normally retains capacity.

After cleanup, a future valid call can initialize a fresh HUD because the program sentinel is zero. Cleanup should run while the owning OpenGL context is current and before that context is destroyed. It does not use a state-restoration guard; deleting an object that is currently bound has OpenGL-defined effects on bindings, so this is a lifecycle operation rather than a promised transparent render operation. Line 550 closes cleanup, and line 552 closes the include guard begun at lines 1-2.

The Sandworm's Procedural Geometry

The remaining sections explain `arrakis_worm.h` completely. Unlike the HUD, the worm uses the scene's existing mesh upload and material drawing helpers. It has a changing body mesh, separately constructed mouth surfaces, many individual curved teeth combined into one mesh, and an animation pose shared between scene render passes.

Header Guard, Dependencies, Constants, and Cache Fields

****Source: `arrakis_worm.h`, lines 1-20 and 258.****

Lines 1-2 begin the `ARRAKIS_WORM_H` include guard, closed at line 360. The comments in lines 4-5 describe integration: include the header after the rock-formation drawing definitions and before `drawFullScene`, initialize and clean up while the GL context is current, and draw in both scene passes. This header is not independently self-contained. It includes `<cmath>` and `<vector>` in lines 6-7, but expects GLM, OpenGL declarations, scene vertex and mesh types, materials, terrain height queries, upload routines, drawing helpers, and the shared sphere mesh to have been declared already by the including translation unit.

In the surrounding renderer, `Vertex` contains `pos`, `normal`, and `uv`: a three-dimensional position, a three-dimensional lighting normal, and two texture coordinates. `Mesh` contains `VAO`, `VBO`, `EBO`, index count, and draw mode. The `VAO` describes vertex attributes; the `VBO` contains vertex data; the `EBO` contains the unsigned indices selecting vertices for triangles. `uploadMesh` creates these GPU objects and returns a `Mesh`. `drawMeshPrimitive` supplies the current pass's uniforms and draws the indexed mesh. A `Material` has diffuse and specular colors, shininess, emissive color, and an `isSand` flag. The worm uses these existing types instead of defining replacements.

Line 9 opens `arrakis_worm_detail`, and line 258 closes it. `pi` at line 11 is the single-precision constant 3.14159265358979323846, used in all angular construction. `bodyRows=192` means 192 longitudinal intervals, not 192 total vertex rings. `bodySlices=96` means 96 angular intervals around each ring, not 96 stored vertices per ring. Both axes need an extra endpoint, so the full body grid has 193 rings of 97 vertices each.

Line 14 declares five static `Mesh` objects. `body` is the flexible skin; `lip` is the thick exterior rim; `funnel` is the narrowing visible mouth interior; `throat` is the deep dark continuation and recessed closure; `teeth` is the aggregate tooth geometry. `bodyVertices` at line 15 is the CPU copy retained for deformation and uploading. Other geometry is built once and does not require a persistent CPU vector here.

`ready`, initially false, indicates whether initialization has been performed. It is not an OpenGL error-status query. `poseValid`, also false, says whether the remembered input tuple corresponds to the currently uploaded body pose. `lastCenter` starts at the zero vector. `lastAttack` and `lastTime` start at zero; `lastHeading` and `lastProgress` also start at zero. Those values are meaningful as a cache only after `poseValid` becomes true. A deliberately separate validity flag avoids guessing that a zero-valued input tuple must mean an upload has already happened.

The static namespace variables have internal linkage, so separate translation units including this header would have separate data instances. The actual project includes this implementation into its scene translation unit rather than treating it as a general multi-context library. No cache member contains a shader, framebuffer, or separate shadow pose.

Smooth Transitions and Mesh Destruction

****Source: `arrakis_worm.h`, lines 22-32.****

`smooth(a, b, t)` converts a quantity into a smooth progress value across the interval from `a` to `b`. It first computes $(t-a)/(b-a)$ and clamps it to zero through one. It then returns $u*u*(3-2*u)$, the cubic smoothstep polynomial. The polynomial is zero at `u` zero, one at `u` one, and has zero first derivative at both ends. This makes staged rises, lunges, retreats, and tapers start and stop gently rather than introducing abrupt velocity changes.

The helper assumes distinct interval endpoints. It has no branch for `a==b`, so such a call would divide by zero. Every interval used in this header has distinct endpoints. It also does not itself screen out non-finite `t`; the public drawing function checks its animation inputs before calling the pose logic.

`destroy(mesh)` conditionally deletes the `EBO`, `VBO`, and `VAO`, in that order. Each test checks the object name, allowing cleanup of a default or partially populated mesh without issuing an intentional delete for an absent resource. `mesh=Mesh{}` then resets not only the three handles but also its index count and default draw mode according to the scene's `Mesh` member initializers. This is a complete logical reset of the record, not merely the deletion of GPU storage. As with all OpenGL deletion, the correct current context is required.

Shared Surface Construction

Rectangular Grid Indices and Triangle Winding

****Source: arrakis_worm.h, lines 34-43.****

`gridIndices(indices, rows, slices)` builds triangle connectivity for a rectangular grid whose stored ring width is `slices+1`. It reserves `rows*slices*6` index slots. There are `rows*slices` cells, each split into two triangles, each triangle containing three indices. `reserve` preallocates capacity without inserting values. This routine appends rather than clearing its argument; its callers pass newly created empty vectors.

The `j` loop visits row intervals zero through `rows-1`. The nested `i` loop visits angular intervals zero through `slices-1`. `a=j*(slices+1)+i` locates the current cell's first corner. `b=a+1` is the next vertex around the same ring. `c=a+slices+1` is the corresponding vertex in the next ring. `d=c+1` is the next-ring next-angle corner. The inserted sequence `a, c, d, a, d, b` creates two triangles sharing the `a-to-d` diagonal.

This consistent winding controls which side of each surface is considered the front and the sign of cross-product normals. The formula uses every adjacent pair of stored angles, including the final pair ending at the duplicated seam vertex. It does not wrap the index modulo `slices`, because the extra vertex explicitly represents the angular endpoint. The same connectivity supports both a straight revolved profile and the much more complicated bent body, provided their vertices use this ring-major layout.

For the body, 192 by 96 produces 18432 cells, 36864 triangles, and 110592 indices. These connectivity numbers never change during animation. Deformation changes positions and normals, not the number or order of triangles. The closing braces in lines 41-43 finish the inner loop, outer loop, and function respectively.

Triangle-Derived Surface Normals and the Duplicated Seam

****Source: arrakis_worm.h, lines 45-61.****

`surfaceNormals` begins by setting every vertex normal to zero. It then visits the index vector in steps of three, matching the triangle groups created by `gridIndices`. References `a`, `b`, and `c` select the actual vertex records for those indices. The vector `cross(b.pos-a.pos, c.pos-a.pos)` is perpendicular to the triangle and has length proportional to twice its area. Its direction depends on the index winding. The same unnormalized vector is added to the normal accumulator at all three corners.

Because the face vector is not normalized before accumulation, larger triangles contribute more strongly than smaller ones. This is area-weighted averaging. When the accumulators are eventually normalized, neighboring faces produce smooth vertex lighting rather than isolated flat face normals. It is useful for the gently roughened mouth surfaces.

The UV seam has two stored vertices at the same nominal angular position: the first angle is zero and the last angle is one full turn. Their separate UV values permit a continuous parameter interval from zero to one, but separate accumulation would otherwise give each side only its own adjoining faces. The `j` loop at line 56 advances by `slices+1`, jumping to the beginning of each ring. It adds the first and final vertex's accumulated normals and assigns that sum to both. This welds their shading, not their indices or positions.

The comment in line 55 specifies why this should not be used indiscriminately on tooth surfaces: teeth are separate objects, and averaging unrelated surfaces together would blur their lighting. In fact, the tooth builder supplies its own normals and does not call this function. A final range loop normalizes every accumulated vector to unit length. There is no zero-length fallback here; callers are expected to supply usable nondegenerate surfaces. The later throat floor is added after this computation specifically to give it independent upward normals.

Revolving a Profile Into a Lathe Surface

****Source: arrakis_worm.h, lines 63-84.****

The comments in lines 63-64 explain the geometric ordering: the profile follows the exterior upward toward the lip, then proceeds down the inner funnel. With the same triangle winding, an increasing-height exterior has normals pointing away from the axis, while a descending interior has normals pointing into the mouth cavity. The normal direction comes

from traversal direction and the cross product, not from a separate branch that labels a surface as inside or outside.

`lathe(profile, roughness, deepFloor)` receives radius-height points in `glm::vec2` records. The x component is radius and the y component is vertical position in the local mouth coordinate system. Revolving those points around the y axis creates a surface of revolution, analogous to shaping clay on a spinning wheel. `deepFloor` defaults to false. Each lathe uses 160 angular slices, a higher angular resolution than the body's 96 slices.

The function allocates local vertex and index vectors and reserves `profile.size()*161` vertices. It assumes a profile with at least two points; both the interpolation denominator `profile.size()-1` and the longitudinal cell count depend on that. The supplied lip, funnel, and throat profiles satisfy the assumption. The outer loop visits every profile point; the inner loop visits angles `i` zero through 160 inclusive, creating the duplicate angular seam.

`u=float(i)/slices` is normalized angular position. `a=2*pi*u` maps it to one complete revolution. The roughness formula multiplies three things: the supplied roughness amplitude, a longitudinal sine envelope, and a weighted angular wave sum. The envelope `sin(pi*j/(profile.size()-1))` is nominally zero at each end of the profile and positive in the middle, so neighboring surfaces retain their intended radius at attachment rings while interior rings gain irregularity. Floating-point trigonometry can leave tiny endpoint residuals rather than mathematically exact zero.

The angular wave sum is `0.55*sin(11*a+0.7*j) + 0.30*sin(23*a-0.9*j) + 0.15*sin(47*a+1.1*j)`. Eleven, 23, and 47 are distinct integer frequencies around a full turn. Their weights total one, and their `j`-dependent phases prevent identical bumps in every profile row. The signs and phase rates vary the directions in which the patterns shift. For amplitude 0.16, the magnitude is bounded by approximately 0.16 before the envelope; for 0.08, by approximately 0.08. This is deterministic procedural roughness, not random noise that changes every frame.

`r=profile[j].x+rough` perturbs radius only. Each vertex position is `(r*cos(a), profile[j].y, r*sin(a))`; its initial normal is zero; its UV is `(u, j/(profile.size()-1))`. The UV's first coordinate tracks the revolution and the second tracks profile row fraction, not physical arclength. After building all rings, `gridIndices` connects `profile.size()-1` longitudinal intervals, and `surfaceNormals` computes normals including the actual roughened shape.

The Recessed Floor, Its Extra Vertices, and Upload

****Source: `arrakis_worm.h`, lines 85-97.****

The optional `deepFloor` block closes only the deepest throat, not the large mouth opening. It records the current vertex count as the center index and appends a center at x and z zero and the final profile point's y. That center has normal `(0, 1, 0)` and UV `(0.5, 0.5)`.

The angular loop again includes zero through 160. Each iteration copies a vertex from the final lathe ring, overrides its normal to point upward, and appends the copy. Copying rather than reusing the existing ring preserves a hard normal transition between the narrowing throat wall and its floor. The copied UV remains the wall-ring UV; this is not a fully remapped radial disk texture layout. Here the material is mainly dark color, so geometric closure is its main purpose.

For `i` below 160, the code inserts a triangle from the center to the next copied ring vertex and then the current copied ring vertex. The indices use `center+i+2` and `center+i+1` because the center itself occupies the first newly appended slot. At the time some indices are inserted, the next ring copy will be appended in a later iteration; this is valid because all referenced vertices exist before final upload. The final `i` equals 160 adds the duplicate endpoint but no extra triangle. This makes exactly 160 floor triangles with upward winding.

The function returns `uploadMesh(v, indices)`. That creates GPU buffers while the temporary vectors still exist; they can be destroyed after their data has been uploaded. There is no stored CPU copy of these lathe vertices in the worm namespace. Both the false-floor path and true-floor path converge on the same upload return.

The Thousand Curved Teeth

Row Counts, Radii, Depths, and Deterministic Variation

****Source: `arrakis_worm.h`, lines 99-120.****

makeTeeth constructs all teeth in one mesh. There are six concentric rows, five cross-section sides per tooth, and six sampled cross-section rings along each tooth. The row counts are 192, 184, 176, 168, 152, and 128, totaling exactly 1000 teeth. The attachment radii are 13.1, 11.3, 9.4, 7.7, 6.0, and 4.5. The corresponding depths are -0.8, -2.6, -4.5, -6.3, -8.2, and -10.1. These pairs match explicit points in the funnel profile, so each row starts on an appropriate narrowing section of the inner mouth.

The vertex reserve is $1000 * (\text{steps} * \text{sides} + 1)$: 30 ring vertices and one apex per tooth, giving 31000 vertices. The index reserve is $1000 * ((\text{steps} - 1) * \text{sides} * 6 + \text{sides} * 3)$: five bridges of five quadrilateral sides, each split into two triangles, plus five final triangles at the apex. This is 165 indices per tooth and 165000 indices overall. Reserving the exact expected quantities avoids repeated vector growth during construction.

The outer loop selects a row and the inner loop visits each tooth number below that row's count.

`seed=float(tooth*17+row*131)` produces a deterministic number from its identifiers.

`variation=0.5+0.5*sin(seed*1.713)` maps a sine into zero through one. There is no random-number engine, random seed file, or per-frame regeneration: the same row and tooth produce the same result each time meshes are initialized.

The tooth angle is $2 * \pi * (\text{tooth} + 0.37 * \text{row} + 0.18 * \sin(\text{seed})) / \text{counts}[\text{row}]$. The tooth number spaces teeth around the row. The 0.37-row term offsets consecutive rows so their teeth do not form perfectly aligned radial columns. The final sine term supplies a small angular jitter measured in fractions of nominal tooth spacing.

`radial=(cos(a), 0, sin(a))` points out from the axis toward the attachment. `sideways=(-sin(a), 0, cos(a))` is a perpendicular horizontal direction that allows a tooth to curl sideways independently of inward reach.

`reach=(radii[row]-1.8)*(0.55+0.17*variation)` sets inward extent. The multiplier ranges from 0.55 to 0.72; subtracting 1.8 preserves a central region instead of forcing every tip to the axis. The possible reach ranges by row are 6.215-8.136, 5.225-6.840, 4.180-5.472, 3.245-4.248, 2.310-3.024, and 1.485-1.944. As the mouth narrows, the deeper teeth become shorter in horizontal reach.

`drop=2.1+1.1*variation` ranges from 2.1 to 3.2 downward units. `curl=0.20+0.30*sin(seed*0.91)` ranges from -0.10 to +0.50, so some teeth bend slightly the opposite way but most have a positive-side tendency.

`width=(0.075+0.035*variation)*(1-0.06*row)` sets the starting cross-section radius. Its base range is 0.075-0.110, multiplied by 1.00, 0.94, 0.88, 0.82, 0.76, and 0.70 as rows deepen. The local variable base remembers where this tooth's vertices begin in the shared vector, ensuring later indices refer to this tooth rather than a previous one.

Curved Centerline and a Moving Cross-Section Frame

****Source: arrakis_worm.h, lines 121-137.****

The k loop samples six rings with $t=k/\text{steps}$. Since k runs zero through five and the denominator is six, the sampled values are 0, 1/6, 2/6, 3/6, 4/6, and 5/6. The endpoint at one is deliberately not a small ring; it will be a single pointed apex later.

The centerline position p combines three independent directions. Its radial component is `radii[row]-reach*t`, which moves inward linearly. Its sideways component is `curl*t*t`, which starts with zero lateral slope and increasingly bends to one side. Its vertical component is `depths[row]+0.65*sin(pi*t)-drop*t*t`. The sine briefly lifts the tooth above a purely downward parabola, while the quadratic term bends it down toward the final tip. At t zero the center is exactly at its attachment radius and depth. At t one the sine is zero, the inward displacement is reach, sideways displacement is curl, and vertical displacement is negative drop.

The tangent formula is the derivative of this centerline with respect to t, normalized to unit length. The radial derivative is `-reach*radial`; the sideways derivative is $2 * \text{curl} * t * \text{sideways}$; and the vertical derivative is $0.65 * \pi * \cos(\pi * t) - 2 * \text{drop} * t$. This makes the local cross-section track the changing tooth direction rather than remaining parallel to a fixed plane.

The vector b is the normalized cross product of tangent with sideways. It is perpendicular to the tangent. The vector n is `cross(b, tangent)`, completing a perpendicular pair for the cross-section. Here n is a local frame direction, not yet a particular vertex normal. Because the inward radial derivative is nonzero for all supplied rows, the tangent does not

collapse into the sideways direction and this construction has a usable frame for the generated teeth.

Cross-section radius is $\text{width} * \text{pow}(1 - t, 0.85)$. At the attachment it is width, and it decreases toward zero. The exponent 0.85 controls how the narrowing develops along the shaft; it is not a linear cone. The five-side loop chooses angles $2 * \pi * s / \text{sides}$, then creates a normal direction $n * \cos(\text{angle}) + b * \sin(\text{angle})$. Multiplying that direction by the radius and adding p gives the surface vertex. The same direction is stored as its lighting normal. UV is $(s / \text{sides}, t)$.

Only five angular vertices are stored per ring, so there is no sixth duplicated seam vertex. Connectivity later wraps the fifth side back to the first with modulo. The supplied normals are moving-frame radial normals, an efficient approximation for a tapered curved tooth; they do not include a derivative correction for every taper slope in the way the flexible body's finite-difference normals do. The UV seam is also a wrapped five-vertex polygon rather than the explicitly duplicated zero/one seam of the lathe.

Shaft Triangles and One True Apex

****Source: arrakis_worm.h, lines 138-160.****

The next nested loops connect adjacent sampled rings. k ranges zero through four, covering five bridges; s ranges zero through four, covering the five pentagonal sides. $a0 = \text{base} + k * \text{sides} + s$ is the current corner, while $b0 = \text{base} + k * \text{sides} + (s + 1) \% \text{sides}$ is the next corner around the ring. Modulo returns to side zero after side four. Adding sides selects corresponding corners in the next ring. The six inserted indices form two triangles covering the side quadrilateral.

The comment at line 145 emphasizes the apex design. tip is the next vertex index. tipPos evaluates the same centerline at t one: radial radius becomes $\text{radii}[\text{row}] - \text{reach}$, sideways displacement becomes curl, and depth becomes $\text{depths}[\text{row}] - \text{drop}$. tipNormal uses the normalized endpoint tangent. Its vertical term is $-0.65 * \pi - 2 * \text{drop}$, because cosine at pi is -1. The tip UV is $(0.5, 1)$.

The final side loop takes each adjacent pair from the sixth sampled ring and joins them to the single apex. It contributes five triangles. There is no flat tiny cap at the tip and no set of five coincident endpoint vertices. There is also no cap at the attachment base; that open end starts against the funnel surface. Each tooth therefore has 31 vertices, 50 shaft triangles, and five tip triangles. Every tooth's base and tip keep its connectivity independent of the other 999 teeth.

The nested row and tooth loops close at lines 157-158. uploadMesh receives the aggregate vectors once and returns the teeth mesh at line 159; line 160 closes the function. This is one GPU mesh with 55000 triangles, not a thousand separate draw calls. The teeth do not deform individually after construction: the common head matrix moves, tilts, and radially scales all of them together.

Worm Pose and Attack Phases

Pose Fields and Terrain-Aware Head Position

****Source: arrakis_worm.h, lines 162-189.****

Pose contains head, a three-dimensional translation for the local mouth origin; tilt, an angle in radians used to tip the mouth and bend the body; and aperture, a radial scale for the mouth. There is no yaw field because heading is applied separately by a root matrix. The structure is an animation result, not persistent game state.

pose(center, attack, time, headingDegrees, approachProgress) accepts a worm center, an attack-phase time, the general animation time, and optional heading and approach progress defaulting to zero. The first three smooth transitions are rise over attack seconds 0-6, lunge over 6-12, and retreat over 13.5-18. These overlap only at endpoints and create distinct stages. Negative attack time gives zero for all three and represents approach.

ground=getDuneHeight(center.x, center.z) samples terrain beneath the center. The center's y value is not used as the head's terrain reference. approachHeight=10+4*smooth(0,1,approachProgress) raises the approaching head from ten to fourteen units above local terrain as progress advances. For negative attack time, this is the chosen height. For nonnegative attack time, the alternative is $14 + 22 * \text{rise} - 27 * \text{lunge}$: the head begins fourteen above ground, rises to 36, and lunges down to nine.

The next expression adds a 0.25-amplitude sine bob using $\text{time} * 1.6$, multiplied by rise and by $1 - \text{lunge}$. Bobbing fades in as the rise develops and fades out during the lunge. It does not oscillate the approach height because rise is zero before attack. `glm::mix(height, -24, retreat)` then pulls the head below terrain during retreat. The final target is 24 units below the sampled ground, not absolute world y -24.

Forward reach is $12 * \text{lunge} * (1 - \text{retreat})$. It grows to twelve during the lunge and disappears during retreat. Head is initially constructed at $(\text{center.x}, \text{ground} + \text{height}, \text{center.z} + \text{reach})$ in the unyawed deformation coordinate system. The later root matrix rotates that local +z reach toward the requested heading. This separation lets body and mouth geometry share a simple local bending plane.

Heading is converted from degrees to radians. A terrain target is sampled eighteen units from the center in the direction $(\sin(\text{heading}), \cos(\text{heading}))$ on the xz plane, and one unit is added to that height.

`aim = atan2(18 - reach, targetY - head.y)` is intentionally written with horizontal remaining distance as the first argument and vertical difference as the second. An ordinary yaw calculation would use a different interpretation; here the angle is measured away from the upward y direction into local +z. A target below the head therefore can produce a tilt greater than ninety degrees, tipping the mouth into a downward bite.

Tilt is $\text{radians}(70) * (1 - \text{rise}) + \text{aim} * \text{lunge} * (1 - \text{retreat})$. During approach it is seventy degrees. The rise removes that initial tilt, leaving an upright pose at attack six. The lunge adds the terrain-aware aim. Retreat fades the aim out. `swallow = smooth(12, 13.5, attack) * (1 - retreat)` narrows the mouth after the lunge and releases that narrowing during retreat. Aperture is $1 - 0.18 * \text{swallow}$, so the minimum intended radial scale is 0.82. Returning `{head, tilt, aperture}` uses aggregate initialization of the three fields.

At attack zero, height is fourteen and tilt seventy degrees. At six, height is 36 plus possible bob, reach is zero, and tilt is zero. At twelve, height is nine, reach is twelve, and tilt is the full aim, with swallow just starting. At 13.5, radial aperture is 0.82 and retreat has just begun. At eighteen, the formulas reach a submerged, untilted, reopened pose, but the public draw function rejects attack times at least eighteen, so that exact endpoint is not drawn. These are animation values rather than a separate collision model.

The Head Matrix and Order of Operations

****Source: `arrakis_worm.h`, lines 191-195.****

`headMatrix` begins with an identity matrix translated to `p.head`, then applies an x-axis rotation by tilt, then an xz radial scale $(\text{aperture}, 1, \text{aperture})$. With the renderer's usual column-vector convention, the resulting product is translation times rotation times scale. A local mouth vertex is radially scaled first, rotated second, and placed at the head third.

Scaling x and z but not y narrows the mouth and teeth arrangement without directly shortening its throat depths. An x-axis positive rotation transforms local up $(0, 1, 0)$ into $(0, \cos(\text{tilt}), \sin(\text{tilt}))$, exactly the axis used by body deformation. This agreement is what lets the body end meet the tilted lip. The function returns a matrix and does not bind a shader, change OpenGL state, or modify mesh vertices.

Deforming the Flexible Body

Neck Axis, Circular-Arc Centerline, and Buried Bezier Segment

****Source: `arrakis_worm.h`, lines 197-213.****

`deformBody(center, p)` changes the retained CPU vertex vector. It does not upload data by itself. It starts with the mouth's unit axis $(0, \cos(\text{tilt}), \sin(\text{tilt}))$. Neck length is 64 and join is 0.4. The body meets the rim at `end = p.head - 1.4 * axis`, placing its final centerline ring at the local mouth y of -1.4. That matches the first lip-profile point's height.

The `neckPoint(s)` lambda describes a constant-curvature neck rather than joining the head to the ground with a sharp corner. $\text{half} = 0.5 * \text{tilt} * (1 - s)$ is half the remaining angular sweep, and $\text{mid} = 0.5 * \text{tilt} * (1 + s)$ is the corresponding midpoint angle. `sinc` is $\sin(\text{half}) / \text{half}$ when half exceeds 0.0001 and one otherwise. The one branch avoids division by a nearly zero value and matches the limiting value of the expression for the normal

nonnegative tilt range used here.

The comparison is specifically $\text{half} > 0.0001$, not an absolute-value test. A negative tilt passed to this helper would take the one branch rather than evaluate its nonzero sinc, so it should not be described as a universally robust signed-arc implementation. The current approach and bite poses produce the nonnegative bending behavior the helper is designed for. This is a concrete implementation condition, not a source correction.

$\text{span} = 64 * (1 - s) * \text{sinc}$ is the straight chord distance associated with the remaining arc length. Subtracting $(0, \text{span} * \cos(\text{mid}), \text{span} * \sin(\text{mid}))$ from end gives the point on the circular arc. At s one, span is zero and the point is end. At s zero, it is the neck's lower endpoint. As tilt tends to zero, sinc tends to one and this becomes a straight upright 64-unit segment. The comment about a bend radius over twenty meters expresses the intent to keep a large smooth bend even during a downward bite; the geometry is not a thin hinged cylinder.

The lower body begins at $\text{p0} = (\text{center.x}, \text{terrainHeight} - 140, \text{center.z})$, deeply underground. $\text{p3} = \text{neckPoint}(0)$ is the connection to the visible curved neck. $\text{buriedLength} = (\text{p3.y} - \text{p0.y}) / 3$ defines vertical control offsets. $\text{p1} = \text{p0} + (0, \text{buriedLength}, 0)$ and $\text{p2} = \text{p3} - (0, \text{buriedLength}, 0)$ give a cubic Bezier segment vertical tangents at its ends while bridging any z displacement between the buried base and the curved neck's lower point. Only the upper part needs to be visually convincing; the deep base keeps the object from looking like a floating short tube when terrain or pose changes.

Ring Centers, Tangents, Radius, Ridges, and Surface Grain

****Source: arrakis_worm.h, lines 214-241.****

The j loop visits all 193 body rings. $t = j / \text{bodyRows}$ is the normalized longitudinal grid parameter. The code declares a center c and tangent for that ring and chooses one of two centerline formulas.

When t is below the join value 0.4, $s = t / \text{join}$ stretches the buried part onto a zero-to-one Bezier parameter, and $u = 1 - s$. Center c is the cubic combination $u^3 * \text{p0} + 3 * u^2 * s * \text{p1} + 3 * u * s^2 * \text{p2} + s^3 * \text{p3}$. Those coefficients form the familiar cubic Bezier curve. The tangent uses its derivative with the common factor of three removed: $u^2 * (\text{p1} - \text{p0}) + 2 * u * s * (\text{p2} - \text{p1}) + s^2 * (\text{p3} - \text{p2})$, then normalizes it. Removing that common positive factor does not change direction.

When t is at least 0.4, $s = (t - \text{join}) / (1 - \text{join})$ maps the rest of the body onto the neck arc. Center c comes from $\text{neckPoint}(s)$. Tangent is $(0, \cos(\text{tilt} * s), \sin(\text{tilt} * s))$, turning continuously from upward at the lower neck to the mouth's full tilt at the top. At the join, both constructions meet p3 and have an upward tangent, giving positional and directional continuity. Parameter speeds and sampling densities need not be identical; this is not a proof of equal arclength sampling for the whole body.

$\text{across} = (0, -\text{tangent.z}, \text{tangent.y})$ gives a perpendicular direction in the yz plane. The other cross-section direction is the x axis. Because the centerline remains in a fixed- x plane and tangent is normalized, these form a usable radial frame without constructing a full arbitrary three-dimensional transport frame.

$\text{taper} = 1 - \text{smooth}(0.85, 1, t)$ stays one through most of the body and falls to zero over the final fifteen percent of its parameter range. $\text{ridge} = 0.70 * \text{pow}(0.5 + 0.5 * \cos(2 * \pi * 32 * t), 3)$ creates 32 repeated ring ridges along t . The shifted cosine is in zero through one, cubing sharpens its peaks, and the coefficient limits ridge height to 0.70. The base body radius is consequently between 13.9 and 14.6 before the mouth blend and grain.

$\text{radius} = \text{mix}(13.9 + \text{ridge}, 15 * \text{aperture}, 1 - \text{taper})$ transitions from ridged skin to the mouth's 15-unit outer attachment radius, scaled by aperture. Far from the head, the mix weight is zero. At the final ring it is one, giving exactly the intended mouth-join radius in the mathematical formula. During swallowing, that final radius becomes 12.3 because 15 times 0.82 is 12.3. The skin therefore follows the narrowing mouth rather than remaining wide behind a shrunken lip.

For every ring, the i loop visits 97 angular positions including the duplicate seam. $a = 2 * \pi * i / \text{bodySlices}$ is the angle. $\text{radial} = (\cos(a), 0, 0) + \sin(a) * \text{across}$ combines the x direction and the transverse yz direction to generate a circle perpendicular to the current tangent. $\text{grain} = \text{taper} * (0.11 * \sin(13 * a + 9 * t) + 0.07 * \sin(29 * a - 17 * t))$ adds smaller irregularities with two angular frequencies and longitudinal phase changes. Its possible magnitude is at most about 0.18 when taper is one, and it fades away at the lip so the surfaces meet cleanly.

The vector index $j * (\text{bodySlices} + 1) + i$ accesses the existing body vertex rather than appending a new one. Position becomes $c + (\text{radius} + \text{grain}) * \text{radial}$. UV becomes $(i / \text{bodySlices}, t * 32)$, giving one wrap around the circumference and 32 repetitions along the body. The repeated longitudinal coordinate works with the renderer's procedural appearance; it is not a statement that the header loads a worm texture. Normals are deliberately left for a second pass because they depend on neighboring updated positions. Lines 240-241 finish the angular and longitudinal loops.

Finite-Difference Normals and Seam Copying

****Source: `arrakis_worm.h`, lines 242-256.****

The comment at line 242 explains the motivation for another normal method: a radial direction alone cannot account for changing ridges, taper, and bend. The next pair of loops recomputes normals from positional differences after every vertex position has been updated. The outer loop includes all body rings. The inner loop covers only the 96 unique angles, leaving the duplicated endpoint for an explicit copy afterward.

`prev=(i+bodySlices-1)%bodySlices` and `next=(i+1)%bodySlices` choose wrapped angular neighbors. At angle zero, the previous neighbor is 95 rather than an invalid -1; at 95, the next neighbor is zero. This uses the unique-angle set and avoids differentiating between the duplicated endpoint and its same-position counterpart.

The lower row neighbor is zero at the first row, otherwise $j-1$. The upper row neighbor is j at the final row, otherwise $j+1$. Interior rings therefore use differences across both sides, while end rings use one-sided differences. `ds` is the positional difference between higher and lower rows at the same angle; `da` is the positional difference between next and previous angles in the same row. Their cross product, `cross(ds, da)`, is normalized and assigned as the lighting normal.

The order matters. For a straight upward tube at its positive- x point, `ds` points up and `da` points roughly positive z , so `ds` crossed with `da` points outward positive x . Reversing the order would reverse the surface normal. Since these differences come from the full deformed positions, their directions include local grain, ring-ridge slope, radius change, and centerline bending. Dividing by the small parameter steps is unnecessary for the final direction because positive scalar factors vanish under normalization.

After each row's unique normals are finished, the last stored seam vertex receives the first vertex's normal exactly. This guarantees matching seam shading even if tiny trigonometric roundoff makes the nominally equal endpoint positions slightly different. This method is distinct from the lathe's accumulated triangle normals. There is no zero-vector check here either; it relies on the configured mesh and deformation producing valid neighbor directions. The final brace closes `deformBody` at line 256, and the internal namespace closes at 258.

Creating and Releasing All Worm Meshes

Cleanup and What It Resets

****Source: `arrakis_worm.h`, lines 260-265.****

`cleanupSandwormMeshes` brings the detail names into scope and calls `destroy` for body, lip, funnel, throat, and teeth. This releases fifteen possible GPU objects, three per mesh, and resets every mesh record. It then clears `bodyVertices` and sets both `ready` and `poseValid` false with a chained assignment.

Clearing the vector removes its active elements but normally retains its allocated capacity. Unlike the HUD's empty-vector swap, this cleanup does not explicitly shrink the retained CPU storage. The remembered center and animation floats are also not reset, but their cache is invalidated by `poseValid=false`, so stale values cannot suppress the next required upload. The routine can be called again on default mesh records because `destroy` tests each handle. It should run before destroying the context, and it makes no promise to restore all current draw bindings.

Body Allocation, Initial Pose, and Dynamic Storage

****Source: `arrakis_worm.h`, lines 267-279.****

`initSandwormMeshes` returns immediately when `ready` is true, avoiding duplicate creation. It resizes the CPU body vector to $(192+1) * (96+1) = 18721$ elements. `resize`, unlike `reserve`, creates the elements that the deformation

routine indexes. It then deforms this initial body at center zero using attack six and animation time zero. In that pose, rise is one, lunge and retreat are zero, tilt is zero, and the mouth is fully open. Terrain still contributes to the initial head's y; this is not necessarily a body centered vertically at world zero.

An empty local index vector receives the fixed grid connectivity, and `uploadMesh` creates the body VAO, VBO, and EBO. The scene upload helper initially supplies static buffer storage for ordinary meshes. The following block specifically replaces the body VBO's storage with `GL_DYNAMIC_DRAW`, using the full current CPU vector as data, because the body will change over time.

The code queries the current `GL_ARRAY_BUFFER_BINDING`, binds the body VBO, allocates dynamic storage using `bodyVertices.size()*sizeof(Vertex)`, and restores the queried binding. This is a narrow preservation block around the reallocation. It does not make the preceding `uploadMesh` state-transparent: that shared helper has its own binding behavior, including ending with VAO zero and leaving its array buffer bound. A reader should not assume this initialization has the HUD's broad `GLState` contract. Byte size is determined by `sizeof(Vertex)` so it remains correct if compiler alignment changes the record layout.

Exact Lip, Funnel, and Throat Profiles

****Source: `arrakis_worm.h`, lines 281-292.****

The lip has eight radius-height points and roughness 0.16. In profile order these are (15.0, -1.4), (15.6, -0.9), (15.9, -0.2), (15.5, 0.5), (14.8, 0.8), (14.0, 0.5), (13.4, -0.1), and (13.1, -0.8). The first attaches to the final body ring. Radius increases to a nominal maximum 15.9 around the outside, height peaks at 0.8, and the profile curls inward and down to the funnel's first point. This makes a thick rounded lip rather than a paper-thin circular edge. Interior roughness can shift radii slightly from these nominal values.

The funnel has fourteen points and roughness 0.08. Its exact ordered profile is (13.1, -0.8), (12.8, -1.3), (12.0, -2.0), (11.3, -2.6), (10.5, -3.5), (9.4, -4.5), (8.6, -5.4), (7.7, -6.3), (6.8, -7.3), (6.0, -8.2), (5.2, -9.2), (4.5, -10.1), (3.6, -11.4), and (2.9, -13.0). Both radius and height decrease, creating a deep narrowing cavity with inward-facing wall normals. The six tooth rows use the funnel points at radii 13.1, 11.3, 9.4, 7.7, 6.0, and 4.5 exactly. The lip-to-funnel and funnel-to-throat endpoints therefore agree by design.

The throat has five points, zero roughness, and the true-floor option enabled. Its profile is (2.9, -13.0), (2.6, -15.0), (2.2, -19.0), (1.7, -24.0), and (0.8, -27.0). It continues down another fourteen local units from the funnel's end and closes only at the narrow 0.8-radius bottom. There is no disc spanning the 13.1-radius mouth entrance. A dark material and genuine depth together make the opening look hollow rather than merely painted black.

At 160 angular slices, the lip's eight rings contain 1288 vertices and 2240 triangles. The funnel's fourteen rings contain 2254 vertices and 4160 triangles. The throat wall's five rings contain 805 vertices and 1280 triangles; adding a center and 161 copied ring vertices yields 967 vertices and 1440 triangles including the floor. These are exact topology counts for the current constants.

`teeth=makeTeeth()` completes the five meshes. `ready=true` marks initialization complete, while `poseValid=false` ensures the first real drawing input will trigger an upload even if it happens to match some remembered defaults. The function does not test individual upload results or recover from allocation failure; `ready` means the setup code finished, not that every GL operation was independently verified successful. The closed function at line 292 is the end of resource construction.

Together, the five meshes contain 54230 stored vertices and 99704 triangles: 36864 body, 2240 lip, 4160 funnel, 1440 throat, and 55000 tooth triangles. Duplicate seams and the independent floor rim count as stored vertices even when some positions coincide. The sand wake uses the renderer's existing sphere mesh and is not included in these totals.

Drawing, Cache Reuse, and Materials

Input Validation and Exact Pose-Cache Comparison

****Source: `arrakis_worm.h`, lines 294-316.****

`drawSandworm` accepts the center, attack time, animation time, optional heading degrees, and optional approach progress. It brings the detail namespace into local scope. The initial compound condition returns when resources are not ready; attack time or animation time is non-finite; any x, y, or z center component is non-finite; heading or approach progress is non-finite; or attack time is at least eighteen. Short-circuit evaluation stops at the first true disqualifier. Negative attack times are deliberately allowed because they display approach and its wake.

Although center y does not determine the terrain-based head height, the function still validates it and includes it in the cache. That makes the entire public position tuple consistently checked. It rejects non-finite progress before clamping it to zero through one. Pose calculation therefore sees a bounded approach progress. The pose is computed on every accepted draw, even when the body upload is cached, because the head matrix and wake still need current animation results.

The cache condition is true if the stored pose is invalid or any center component, attack time, animation time, heading, or clamped approach progress differs exactly from its remembered value. There is no epsilon threshold. A very small finite input change can cause a deformation and upload, while two passes using identical floats share the uploaded skin. Because time is included even when its bob contribution is currently zero, the cache may recompute geometry for some time changes that do not materially alter the body. It favors straightforward correctness over a phase-specific minimal key.

When the condition is true, `deformBody` updates all CPU positions and normals. The block saves the array-buffer binding, binds the body's VBO, and uses `glBufferSubData` to overwrite the full fixed-size vertex storage starting at byte zero. It does not orphan the buffer here, unlike the HUD's per-frame `glBufferData` strategy. The index buffer remains unchanged. The previous array-buffer binding is restored, then all input values are remembered and `poseValid` becomes true.

The comment at line 303 explains the render-pass motivation. A shadow pass and a normal scene pass should display the same pose. Only the flexible skin requires streaming; lip, funnel, throat, and teeth keep static vertex data and animate through model matrices. A second pass still makes the draw calls, but skips repeated CPU deformation and GPU data transfer when the input tuple matches.

The cache does not include a terrain version or terrain sample results. If `getDuneHeight` changed while all input floats stayed identical, the body cache could remain valid despite a changed height field, whereas the freshly computed head pose would reflect the query. In the current use, these terrain queries form a stable underlying height function; the tuple cache is not a general dependency-tracking system. Heading is included because it affects the terrain-aware target height, even though the final yaw is also performed by a matrix.

Five Material Records and Their Inheritance

****Source: `arrakis_worm.h`, lines 317-332.****

`skin` begins with the scene's default `Material`, then receives diffuse $(0.43, 0.27, 0.12)$, specular $(0.055, 0.04, 0.025)$, and shininess eight. These values describe a subdued brown exterior with weak, broad highlights. Default emissive color remains zero and default `isSand` remains zero. Brown coloration alone does not activate the renderer's terrain-sand shader branch.

`rim` copies `skin`, retaining its specular color, shininess, emissive setting, and non-sand flag, but raises diffuse to $(0.49, 0.31, 0.14)$. This makes the lip somewhat lighter without a wholly different reflection model. `inside` also copies `skin`, then uses darker diffuse $(0.15, 0.075, 0.029)$ and equal specular channels 0.018. The one-argument GLM vector constructor fills all three channels, so this is $(0.018, 0.018, 0.018)$, not only a red value.

`dark` copies `inside` and changes diffuse to $(0.009, 0.004, 0.002)$ with specular zero. It still inherits shininess eight, but zero specular makes that exponent irrelevant to a specular contribution. This nearly black throat material creates depth while keeping the actual geometry. `enamel` copies `skin`, uses diffuse $(0.58, 0.43, 0.24)$, equal specular channels 0.07, and shininess eighteen. The teeth are brighter and have more concentrated highlights than `skin` but are not pure white.

These material records are reconstructed on each accepted draw; they are cheap local descriptions rather than persistent GPU material objects. Their colors influence the normal scene pass through the shared renderer. In the

shadow pass the relevant geometry and transforms still matter, while the pass primarily writes depth and uses the renderer's shadow uniforms rather than displaying these RGB colors directly.

Rotation About the Worm Center and Five Draw Calls

****Source: `arrakis_worm.h`, lines 333-341.****

The root matrix starts with translation to center, then a y-axis rotation by the heading in radians, then translation by negative center. Its product is $T(\text{center}) * R_y(\text{heading}) * T(-\text{center})$. This rotates geometry about the worm's center rather than about world origin. It is necessary because the flexible body vertices have already been generated in coordinates positioned around center; applying an ordinary translation again would move them twice.

Positive heading rotates local +z toward positive x under GLM's usual right-handed convention. The pose target samples terrain along that same $(\sin(\text{heading}), \cos(\text{heading}))$ ground direction, so aiming and rendering agree. `head=root*headMatrix(p)` applies the local mouth aperture, tilt, and placement first, then the overall heading about center. This gives one shared mouth transform for all four static mouth pieces.

The five calls draw body with root and skin, lip with head and rim, funnel with head and inside, throat with head and dark, and teeth with head and enamel. The order is explicit and the mesh-material pairings are exact. They use the caller's active scene or shadow program through `drawMeshPrimitive`; `drawSandworm` does not create or choose its own shader. There is no separate translucent mouth pass, no tooth-by-tooth loop during drawing, and no dynamic mouth VBO upload.

Unlike `drawRescueHud`, this function has no broad OpenGL state guard. The common drawing helper binds each mesh VAO and leaves its normal renderer-side state behavior in place. The narrow array-buffer restoration inside the cache-update block is the only explicit binding preservation performed here. Calling the worm as part of both normal scene passes follows the renderer's existing conventions rather than treating it as an isolated overlay library.

The Moving Sand Wake

Wake Timing, Material, Positions, and Animated Scales

****Source: `arrakis_worm.h`, lines 342-358.****

`wake=1-smooth(0,6,attackTime)` is one throughout negative-time approach and fades to zero during the initial rise. The wake block executes only when wake exceeds 0.001. This suppresses tiny nearly vanished spheres as the rising worm emerges, and no wake is drawn once the rise has completed. It is not an additional mesh allocation or a particle system in this header.

sand copies skin, changes diffuse to $(0.68, 0.46, 0.22)$, and sets equal specular channels 0.015. It retains shininess eight, emissive zero, and `isSand=0`; the name describes the intended visual mound, not a request to apply the terrain's special sand flag.

The loop draws four mounds for `i` zero through three. An initial local position is $(\text{center.x}, \text{center.y}, \text{center.z}-14-14*i)$. Thus the centers are fourteen, 28, 42, and 56 units behind local +z movement. The code transforms this point with root and homogeneous w one to find its yawed world x and z. A point uses w one so translation participates; a direction would use w zero. This temporary world position is used for a terrain query, not directly as the final draw position.

`local.y` becomes terrain height at the yawed world xz minus 0.8. Because root's only net rotation is about y, world y equals local y after the paired center translations; replacing local y with this ground value therefore places the mound correctly even though the original center y was present in the first point. The -0.8 sinks each sphere into the dune so its visible top looks like disturbed sand rather than a sphere resting on the surface.

`mound=translate(root, local)` composes the root with the mound translation. Scaling then uses x extent $(12-i)*\text{wake}$, y extent $(3+0.35*\sin(\text{time}*2-i))*\text{wake}$, and z extent $10*\text{wake}$. The four x scales are twelve, eleven, ten, and nine times wake. Vertical scale oscillates between 2.65 and 3.35 times wake, with each mound phase offset by `i`. The z scale is uniformly ten times wake. When wake approaches zero, all three dimensions shrink together.

The final call draws the already-existing `g_meshSphere` with that mound matrix and sand material. Four transformed spheres create a stylized raised wake using shared geometry. Their actual geometric radii depend on the renderer's sphere primitive definition; these numbers are model scale factors, not an independently asserted sphere radius. The positions rotate with heading, stay terrain-aware, and pulse with animation time. Lines 356-358 close the loop, conditional wake block, and drawing function.

End of Header and Lifecycle Summary

****Source: `arrakis_worm.h`, line 360.****

The final preprocessor directive closes the guard begun at lines 1-2. The complete lifecycle is initialization while the context is current, repeated validated calls in both render passes, and cleanup before context destruction. CPU construction happens once for static mouth geometry; CPU skin deformation and VBO updates happen only when the cached pose inputs change. All five worm meshes are explicitly destroyed, while the shared sphere belongs to the surrounding renderer and is not deleted by worm cleanup.

Numerical and Design Checks

Following One HUD Frame From State to Pixels

The input snapshot remains `const` throughout composition. On a 1280-by-720 framebuffer, `scale` is one and the noncompact desktop layout applies. Its margin is 22, inner width is 1236, top card height is 106, and normal radar top is 160. The control string wraps according to its 1.5-unit pixel font, and the bottom cards' positions depend on the resulting line counts. These dependencies explain why the radar is measured after the action card rather than assigned an unconditional fixed y-to-bottom region.

On a 640-by-360 framebuffer, `scale` is still one, margin is ten, and `compact` is true. Because `shortView` tests strictly less than 360, height exactly 360 is compact but not short. Its top card is therefore 112 high rather than 78. On height 359, `shortView` becomes true and the special side-radar conditions become eligible. Strict threshold details matter when checking screenshots at boundary sizes; `w<760`, `h<440`, and `h<360` do not include their equality values.

On a 2560-by-1440 framebuffer, `scale two` produces the same 1280-by-720 layout dimensions as the desktop example while increasing physical glyph and line thickness. This is why the width and height uniform must use `w` and `h` rather than the raw framebuffer dimensions: the viewport already maps the clip-space result to the actual framebuffer. Using raw framebuffer dimensions in the shader after scaled composition would incorrectly compress the HUD into part of the image.

The HUD also keeps display roles separate. `number` handles negative integer counts, `clock` handles bounded upward-rounded time, `decimal` handles bounded zero-decimal measurements, `unit` handles bar fractions, and `finite` handles individual floating-point values. Not all conditions sanitize inputs before comparing them. The explicit finite checks and fallback policies described above should therefore not be generalized into a claim that every imaginable corrupted snapshot is completely layout-safe.

Following One Worm Pose Through Both Passes

Suppose the caller draws a finite, initialized worm at attack six with time zero. Rise is one, lunge zero, retreat zero, aperture one, and tilt zero. The head lies 36 units above terrain. The body final ring is 1.4 units below that head along the upward axis and has radius fifteen, matching the lip's first profile ring. The neck is a straight 64-unit centerline segment in this pose because the arc sweep is zero. The buried Bezier segment joins it to a base 140 units below terrain.

If `poseValid` is false, the first pass deforms 18721 vertices, computes their normals, and uploads the complete body VBO. The other four meshes are already static. Five mesh draw calls render the worm; the wake is zero at attack six, so no mound spheres are drawn. If the second pass receives exactly the same center, attack, time, heading, and clamped progress, it calculates the pose again but skips body deformation and upload. It still issues the same five mesh draws with the appropriate caller-selected shader pass.

On a later attack value, changing the body skin and multiplying static mouth vertices by the current head matrix produce coordinated motion. The root rotates the full assembly toward heading. The body normals are recomputed from its bent surface, while the static mouth and tooth normals are transformed by the scene shader's normal-transform machinery. When attack reaches eighteen, the validation branch returns before any material setup or draw calls. Visibility termination is therefore explicit, not merely the result of the worm becoming dark or moving beneath dunes.

Limitations That Should Not Be Mistaken for Features

The HUD has a fixed ASCII bitmap vocabulary rather than Unicode typography, no mouse-click buttons, no input event handling, no texture atlas, and no automatic count reconciliation. The layouts adapt by wrapping, reducing sizes, changing placements, and omitting secondary panels, but they do not implement scrolling for impossibly small framebuffers. The unsupported apostrophe and brackets are real fallback behavior. The declared pickup position and centered-text helper are unused in this header.

The worm does not load `sandworm.jpg`, import a model, regenerate a random set of teeth every frame, or animate each tooth separately. It has five owned meshes and reuses a scene sphere for the wake. It uses a fixed topology, deterministic sinusoidal irregularity, a circular-arc visible neck, a buried cubic Bezier transition, and a tuple cache. There is no general OpenGL state guard, no upload error recovery, no terrain-version cache key, and no automatic initialization inside `drawSandworm`. These boundaries help distinguish what the code really guarantees from what a reader might infer from the visual result.

Complete Source Coverage Index

The intervals below are an audit of the explanation, not an additional source listing. Every nonblank line in the original two headers is included in an explained region. Structural lines such as opening and closing braces, namespace boundaries, return statements, comments, and end guards belong to the cited region containing the construction they complete. Blank separators do not carry operations.

Original Header	Line Range	Explained Region
<code>arrakis_hud.h</code>	1-28	Include guard, all includes, every <code>RescueHudState</code> member, defaults, and the outside-crew comment.
<code>arrakis_hud.h</code>	30-34	Current-context contract, framebuffer coordinates, preservation contract, and internal namespace opening.
<code>arrakis_hud.h</code>	36-52	Every palette constant, both internal structs, and the static resource accessor.
<code>arrakis_hud.h</code>	53-66	Finiteness, normalized clamping, count strings, rounded values, and countdown formatting.
<code>arrakis_hud.h</code>	68-90	Every saved-state field, capability entry, GL query, and constructor loop.
<code>arrakis_hud.h</code>	91-107	All restoration calls, branches, destructor behavior, and deleted copy operations.
<code>arrakis_hud.h</code>	109-124	Shader creation, compilation, status, diagnostics, failure deletion, and successful return.
<code>arrakis_hud.h</code>	126-147	Triangle, rectangle, border, card, and progress-bar construction.
<code>arrakis_hud.h</code>	149-204	Font comment, all 50 explicit glyph cases, default case, and five-bit encoding.

Original Header	Line Range	Explained Region
arrakis_hud.h	205-222	Text width, case conversion, each font scan branch, run merging, and character advance.
arrakis_hud.h	223-247	Width-to-column expression, lambda, word splitting, whitespace/newline behavior, and returned lines.
arrakis_hud.h	248-255	Centered placement, line-count cap, and line-loop spacing.
arrakis_hud.h	257-294	Radar card, grid, coordinate clamp/map, all marker branches, marker order, and every legend.
arrakis_hud.h	296-322	Overlay dimming, precedence, responsive dimensions, every body/key string, and panel placement.
arrakis_hud.h	323-341	Branding, heading, all result rows, cursor increments, body, separator, and keys.
arrakis_hud.h	343-355	Margin, modes, extraction timer, duration defense, warning decisions, and top-card size.
arrakis_hud.h	356-375	All desktop header labels, coordinates, seat loop, counts, countdown, and bar.
arrakis_hud.h	375-392	Compact branch, right alignment, short-view placements, omitted rows, and compact seats.
arrakis_hud.h	394-422	Controls string and wrap, action precedence, side-radar test, truncation, card, and shared progress.
arrakis_hud.h	424-442	Radar sizing and labels, telemetry gates, every measurement/string, boost, and pursuit.
arrakis_hud.h	443-452	Flight-only guide condition, four corner loops, offsets, and final overlay call.
arrakis_hud.h	454	Internal namespace closure.
arrakis_hud.h	456-491	Public initializer, GLSL expressions, shader and program creation, all compilation/link failure paths.
arrakis_hud.h	492-510	Binding queries, VAO/VBO allocation failure, uniform location, vertex attributes, restore, and CPU reserve.
arrakis_hud.h	512-540	Dimension checks, lazy init, scope guard, scale, composition, drawing state, buffer growth/orphaning, upload, draw.
arrakis_hud.h	542-552	Conditional resource deletion, resets, CPU capacity release, function end, and guard end.
arrakis_worm.h	1-20	Guard, comments, dependencies, namespace, all constants, meshes, CPU vector, flags, and cached values.
arrakis_worm.h	22-32	Smoothstep expression and conditional mesh deletion/reset.

Original Header	Line Range	Explained Region
arrakis_worm.h	34-43	Grid capacity, both loops, all corner-index expressions, triangle order, and closure.
arrakis_worm.h	45-61	Normal reset, triangle accumulation, seam loop, normalization, and assumptions.
arrakis_worm.h	63-84	Winding comments, lathe parameters, sampling loops, complete roughness formula, positions, UVs, connectivity, normals.
arrakis_worm.h	85-97	Deep-floor branch, center, copied rim, triangle fan, normals, upload, and function closure.
arrakis_worm.h	99-120	All tooth constants and arrays, exact capacity expressions, row/tooth loops, seed, angles, variation, dimensions, base.
arrakis_worm.h	121-137	Ring loop, centerline, derivative, frame vectors, taper exponent, side loop, vertex data, and loop closure.
arrakis_worm.h	138-160	Side connections, modulo, apex comment, exact tip position/normal/UV, final triangles, loops, and upload return.
arrakis_worm.h	162-189	Every pose field, all timed transitions, terrain samples, heights, bobbing, reach, aim, tilt, swallowing, and return.
arrakis_worm.h	191-195	Head translation, rotation, aperture scale, returned product, and closure.
arrakis_worm.h	197-213	Mouth axis, neck constants, end, arc lambda and small-angle branch, buried endpoints and control points.
arrakis_worm.h	214-241	All body rings, Bezier/arc branch, tangent, radial frame, taper, ridge, blend, grain, position and UV expressions.
arrakis_worm.h	242-256	Normal comment, both derivative loops, wrapped neighbors, endpoint branches, differences, cross product, seam, closure.
arrakis_worm.h	258	Internal namespace closure.
arrakis_worm.h	260-265	Public cleanup, all five destroys, CPU clear, and flag invalidation.
arrakis_worm.h	267-279	Idempotence, body size, initial pose, grid/upload, dynamic reallocation, and narrow binding preservation.
arrakis_worm.h	281-292	Every exact profile point, roughness, throat floor, teeth creation, ready/cache flags, and closure.
arrakis_worm.h	294-316	Public parameters, every validation term, clamp, pose, exact cache condition, deformation, upload, restore, and cache updates.
arrakis_worm.h	317-332	All five materials, every diffuse/specular/shininess number, and inherited defaults.

Original Header	Line Range	Explained Region
arrakis_worm.h	333-341	Root pivot transforms, heading conversion, head product, and all five mesh-material draws.
arrakis_worm.h	342-358	Wake transition and threshold, sand material, four-mound loop, terrain alignment, all scale factors, sphere draws, and closures.
arrakis_worm.h	360	Guard end.

The chapter covers 552 original HUD lines and 360 original worm lines, or 912 total source lines including blank separators. The HUD contains 533 nonblank lines and the worm contains 341, giving 874 nonblank lines in total. A mechanical comparison of the relocated headers against this coverage index found no uncovered nonblank lines in either file. The coverage index assigns every nonblank source line to explanatory prose or the exact glyph table. No complete code listing is embedded here; the complete source appendix can be appended separately without replacing these explanations.

6. Spatial Audio and Vendor Implementations

6.1 Scope, Versions, and Reading Conventions

This chapter explains the complete Arrakis audio wrapper and the architecture of the supplied audio and OpenGL vendor implementations. The gameplay code chooses events, positions, pitch, and volume. miniaudio performs decoding, channel conversion, resampling, spatialization, mixing, and delivery to an operating-system audio device. GLAD performs a completely different task: discovering OpenGL capabilities and resolving graphics-driver entry points. The formerly bundled stb_vorbis source was unused and has been removed during cleanup; its historical architecture is discussed separately to explain the unintegrated Ogg fallback, not as remaining executable or appendix code.

The source was reorganized while this documentation was prepared. Audio references give the final organized line numbers as well as explicitly marked original numbers for historical comparison. The source appendix contains current remaining code, not the removed Vorbis file. Vendor implementation line numbers are unchanged except for GLAD's final blank line. Audio lines after the old search-directory list moved upward by two lines. Function names and the tables below are the primary navigation keys if later edits change line numbers.

Source	Organized location	Original line total	Inspected organized line total	Build role
arrakis_audio.h	src/arrakis_audio.h	295	293	Header-only gameplay audio wrapper; one C++17 inline global
miniaudio_impl.cpp	third_party/miniaudio/miniaudio_impl.cpp	2	2	Emits the miniaudio implementation in exactly one translation unit
miniaudio.h	third_party/miniaudio/miniaudio.h	95,864	95,864	miniaudio 0.11.25, dated 2026-03-04; includes embedded decoder implementations
glad.c	third_party/glad/glad.c	1,833	1,832	GLAD 0.1.36-generated OpenGL 4.6 core loader, generated 2026-09-11
stb_vorbis.c	Removed unused root-level source, inspected before cleanup	5,584	Removed	Historical stb_vorbis 1.22; never integrated into this executable

The chapter supplies complete, contiguous line-coverage maps and explanatory sections for the vendor functional families. It does **not** pretend that an architectural explanation is a separate semantic annotation of every one of miniaudio's 95,864 lines, or a security audit of all platform branches. The numbered source appendix is the exact-code companion for remaining project code. Use the family explanations here, the declaration/definition index strategy in Section 6.10, and the current source listings together when reading an individual vendor function. An unused backend, declaration, table, generated assignment, comment, or license is still accounted for in the coverage maps; being accounted for does not mean it executes in this Windows game. Historical Vorbis ranges are separate from current-source coverage.

6.2 arrakis_audio.h: Includes, File Lookup, and State

6.2.1 Includes and Linkage

Original lines 1-9, also organized lines 1-9, contain `#pragma once` and the dependency includes. The `pragma` prevents repeated definitions within one translation unit. `miniaudio.h` provides the engine, sound objects, result types, and C-compatible audio API. GLM supplies `glm::vec3`, length, normalization, clamp, and mix operations. `<string>` stores paths; `<vector>` stores ordered search directories and candidates; `<fstream>` probes readability; `<iostream>` prints status messages; `<algorithm>` is included but has no directly named algorithm invocation in this header.

The wrapper uses `std::exp` and `rand()` without directly including `<cmath>` and `<cstdlib>` here. Their availability currently depends on the surrounding include environment. This is an include-hygiene observation, not a claim that this documentation changed the header. No vendor or wrapper implementation is edited by this chapter.

The free function `findSoundFile` is `inline`, and methods defined inside `ArrakisAudio` are implicitly `inline`. Original line 295, now line 293, declares `inline ArrakisAudio g_audio;`. Under C++17 this is one program-wide global object rather than a separate audio engine per including translation unit. Its aggregate/default member initializers prepare storage and flags; they do not initialize an operating-system device.

Final lines 291-293 close the `ArrakisAudio` structure with a brace and semicolon, then define that global instance. The semicolon finishes the type declaration; the following object definition creates the shared owner used by input, mission, and runtime code. The intervening blank line only separates those declarations visually.

6.2.2 findSoundFile(filename)

Final lines 11-25; original lines 11-27.

The sole parameter is `const std::string& filename`: a read-only filename reference, normally without a directory component. The return value is either the first readable relative path or an empty string. The function neither decodes the file nor establishes that its contents are valid audio.

The **current organized search order** is exact:

1. `assets/audio/`
2. `../assets/audio/`
3. `../../assets/audio/`
4. `../../../assets/audio/`
5. `../../../../assets/audio/`

The **original search order**, recorded only for older laboratory launches and the cleanup change, was:

1. `sounds/`
2. `../sounds/`
3. `../../sounds/`
4. `../../../sounds/`
5. `sounds/kenney_interface-sounds/Audio/`
6. `../sounds/kenney_interface-sounds/Audio/`
7. `"",` meaning the current working directory itself.

The organized implementation no longer searches a Kenney subdirectory or the bare current working directory. A relative directory is resolved against the **process working directory**, not the header's location or necessarily the executable's directory. The revised CMake post-build command copies `assets/audio` under `$(TARGET_FILE_DIR:Arrakis)/assets/audio` and sets the Visual Studio debugger working directory to the

executable directory; this makes the first candidate usable for normal configured IDE launches. Independently launching the executable from an unrelated directory can still make all five probes fail.

`static const std::vector<std::string> searchDirs` is initialized once and reused. The range-for binds `dir` as `const auto&`, avoiding string copies. `path = dir + filename` constructs one candidate. `std::ifstream f(path.c_str())` opens the path for input; `f.good()` checks the stream state. Success returns immediately, with the local stream closed by its destructor. On failure the loop moves to the next directory; after exhaustion it returns `""`.

There is no cache, filesystem canonicalization, absolute-path asset root, extension validation, binary content test, recursive search, or diagnostic for individual lookup failures. Each call can perform five filesystem probes in the organized layout. Importantly, it returns the first readable instance of a **particular filename**. If that file later fails audio decoding, `loadSound` advances to a different candidate filename; it does not ask `findSoundFile` to try a second directory containing the same filename. A corrupt nearer copy can therefore shadow a valid farther copy.

6.2.3 Every ArrakisAudio Field

Final lines 27-60; original lines 29-62.

Field	Initial value	Meaning and lifetime
<code>ma_engine engine</code>	<code>{}</code>	Stable, zero-initialized storage for the high-level engine, graph, listeners, and pointers to device/resource-manager state. Must be initialized before engine API use and uninitialized after dependent sounds.
<code>initialized</code>	<code>false</code>	Records successful engine initialization. Guards update and shutdown; does not mean every optional sound loaded.
<code>muted</code>	<code>false</code>	User's requested mute state. Persists across mission resets and is displayed by the HUD.
<code>masterVolume</code>	<code>1.0f</code>	Linear engine gain restored on active, unmuted updates. 1 is unity, not a percentage in decibels. No local clamp or UI adjustment is implemented.
<code>sndFlight</code>	<code>{}</code>	Flight-loop sound instance, located at the ornithopter; pitch changes with flight speed.
<code>sndBoost</code>	<code>{}</code>	Continuously started boost loop, made audible by smoothed volume.
<code>sndWormRumble</code>	<code>{}</code>	Worm-position rumble loop; controlled by mission phase, attack time, and proximity.
<code>sndWormBreach</code>	<code>{}</code>	Single reusable eruption/roar sound instance. A mission latch permits one trigger.
<code>sndRescueWinch</code>	<code>{}</code>	Single reusable pickup/winch one-shot located at the ornithopter.
<code>sndRescueSafe</code>	<code>{}</code>	Single reusable successful-delivery one-shot located at the base.
<code>sndHarvester</code>	<code>{}</code>	Harvester movement loop located at the crawler.
<code>sndCrewHelp</code>	<code>{}</code>	Single reusable periodic crew-help one-shot located at the nearest selected crew member.

Field	Initial value	Meaning and lifetime
hasFlight	false	sndFlight successfully initialized; permits its updates and uninitialization.
hasBoost	false	Equivalent validity flag for sndBoost.
hasWormRumble	false	Equivalent validity flag for sndWormRumble.
hasWormBreach	false	Equivalent validity flag for sndWormBreach.
hasRescueWinch	false	Equivalent validity flag for sndRescueWinch.
hasRescueSafe	false	Equivalent validity flag for sndRescueSafe.
hasHarvester	false	Equivalent validity flag for sndHarvester.
hasCrewHelp	false	Equivalent validity flag for sndCrewHelp.
boostVolume	0.0f	Application-side smoothed boost gain. Changes only when a boost sound exists and update passes its early guards.
flightPitch	1.0f	Application-side smoothed playback-rate/pitch multiplier. Changes only for a loaded flight sound.
crewHelpTimer	3.0f	Seconds until the next proximity check/call. Decrement only while flying, loaded, unpaused, and unmuted.
breachTriggered	false	Once-per-mission breach latch. Reset by resetMission, not by the sound ending.
lastRescued	0	Previous processed rescued count, used for positive delivery edges.
lastAboard	0	Previous processed aboard count, used for positive pickup edges.
lastPickup	0.0f	Previous processed pickup-progress value, used for a threshold crossing.

Each sound has exactly one `ma_sound`, so retriggering the same cue restarts or seeks that voice; it does not allocate overlapping copies. The structs own internal pointers and participate in a live graph. Do not copy, move, or `memset` initialized engine/sound objects. The global's stable address satisfies miniaudio's object-address lifetime requirement.

6.3 Initialization, Lambda, Candidates, and Optional Assets

6.3.1 `init()` and the Silent-Mode Branch

Final lines 62-129; original lines 64-131. No parameters. The boolean return reports engine creation, not complete asset availability.

`ma_engine_config config = ma_engine_config_init()` uses vendor defaults instead of manually zeroing a configuration. `ma_engine_init(&config, &engine)` returns a `ma_result`. If it is not `MA_SUCCESS`, the wrapper writes exactly the warning prefix `[AUDIO] Warning: Failed to initialize audio engine (code: , the numeric result, and )`. Running in silent mode., sets `initialized = false`, and returns `false`. The game calls `g_audio.init()` without aborting on its return, making hardware failure nonfatal to rendering and mission logic. Later update calls immediately return in this mode.

Success sets `initialized = true`. `ma_engine_listener_set_world_up(&engine, 0, 0, 1, 0)` means listener **index 0**, then world-up components **(0, 1, 0)**. The two initial zeros have different meanings; this does not set an up vector of `(0, 0, 1)`. The game uses a Y-up world.

Default engine configuration creates one listener, a playback device unless explicitly disabled, and an internal resource manager unless supplied by the caller. The vendor engine normally auto-starts its device; individual sounds remain stopped until their own start calls. Channels and sample rate are negotiated with the device rather than hardcoded by Arrakis. The library's generic defaults include f32, two channels, 48,000 Hz, three periods, and 10 ms low-latency/100 ms conservative period sizes, but those are fallback/configuration defaults, not proof of the actual runtime endpoint format or latency.

`init()` has no already-initialized guard. Calling it twice without shutdown is not a supported wrapper lifecycle: it can overwrite live vendor objects. It also does not apply a preexisting muted state at initialization, so the normal frame update is responsible for enforcing it.

6.3.2 The `loadSound Lambda`

Final lines 75-91; original lines 77-93.

[this] captures the owning `ArrakisAudio*`, allowing the lambda to use engine. It is local to `init`, invoked immediately for each sound, and never escapes into an asynchronous callback. The trailing `-> bool` declares its explicit result type. The initializer-list arguments construct temporary candidate vectors valid for each call.

Parameter	Exact type	Purpose
<code>snd</code>	<code>ma_sound*</code>	Address of the destination sound storage.
<code>candidates</code>	<code>const std::vector<std::string>&</code>	Ordered filename alternatives; first successfully decoded/initialized candidate wins.
<code>loop</code>	<code>bool</code>	Converted to miniaudio's <code>MA_TRUE/MA_FALSE</code> loop setting.
<code>minDist</code>	<code>float</code>	Reference/minimum attenuation distance in the game's coordinate units.
<code>maxDist</code>	<code>float</code>	Maximum distance used by the clamped attenuation model, not an automatic cutoff radius.
<code>model</code>	<code>ma_attenuation_model</code>	Defaults to <code>ma_attenuation_model_inverse</code> ; some calls repeat this explicitly.

For each `const auto& name`, it calls `findSoundFile(name)`. An empty result skips initialization. A nonempty path is passed to `ma_sound_init_from_file(&engine, path.c_str(), 0, nullptr, nullptr, snd)`. Its six arguments mean engine owner, narrow-character path, **flags 0**, no initial sound group, no completion fence, and output sound storage.

Flags 0 do not request streaming, asynchronous loading, predecoded PCM, disabled pitch, disabled spatialization, or detached graph placement. The resource manager's normal non-streaming path loads encoded data into memory and makes a decoder-backed data source; decoding/conversion can occur during reads. Vendor sound initialization internally adds a wait-for-initialization requirement so the sound's data format is known. This is not the same as setting `MA_SOUND_FLAG_DECODE` to predecode an entire MP3 file.

If `r == MA_SUCCESS`, the lambda sets looping, minimum distance, maximum distance, and attenuation model in that order, prints `[AUDIO] Loaded:` followed by the selected path, and returns `true`. If initialization fails, it silently continues to the next candidate. There is no per-candidate decode error report and no assertion that filename existence implies codec support. Exhaustion returns `false`, which becomes the relevant `has*` flag. Vendor initialization handles its internal failure cleanup; the wrapper never uninitialized a sound that did not report successful creation.

6.3.3 Complete Sound-Candidate and Constant Table

All filename orderings are literal and significant. Every sound uses inverse-distance attenuation, whether supplied explicitly or by the lambda default.

Cue; final lines	Candidates in exact order	Loop	Minimum / maximum distance	Initial sound gain	Initial playback
Flight; 93-95	ornithopter_flight.mp3, ornithopter_flight.wav	Yes	12.0f / 150.0f	0.75f	Stopped; first active update starts it
Boost; 97-102	ornithopter_boost.mp3, ornithopter_boost.wav	Yes	12.0f / 150.0f	0.0f	Started immediately, initially inaudible
Worm rumble; 104-106	worm_rumble.mp3, worm_rumble.wav	Yes	30.0f / 320.0f	0.9f	Stopped
Worm breach; 108-110	worm_breach.mp3, worm_breach.wav	No	45.0f / 380.0f	1.0f	Stopped
Harvester; 112-114	harvester_engine.wav, harvester_engine.mp3, harvester.wav	Yes	18.0f / 200.0f	0.75f	Stopped
Winch; 116-118	rescue_winch.wav, rescue_winch.mp3	No	15.0f / 120.0f	0.85f	Stopped
Safe delivery; 120-122	rescue_safe.wav, rescue_safe.ogg, rescue_safe.mp3, confirmation_004.ogg	No	20.0f / 140.0f	0.95f	Stopped
Crew help; 124-126	crew_help.wav, crew_help.mp3, question_001.ogg	No	16.0f / 85.0f	0.85f	Stopped

Original lines for these calls are two greater than the final numbers. `if (has . . .)` guards every initial volume assignment and the boost start. The final `return true` occurs even if **all eight files fail** after the engine succeeded. That separation intentionally permits a partly or wholly silent asset configuration.

6.3.4 What Is Present and What Is Optional or Absent

The organized active asset folder `assets/audio/` contains all eight preferred files:

- `ornithopter_flight.mp3`
- `ornithopter_boost.mp3`
- `worm_rumble.mp3`
- `worm_breach.mp3`
- `harvester_engine.wav`
- `rescue_winch.wav`
- `rescue_safe.wav`

- crew_help.wav

The following alternative candidate names were **not present in the inspected active folder**:

ornithopter_flight.wav, ornithopter_boost.wav, worm_rumble.wav, worm_breach.wav, harvester_engine.mp3, harvester.wav, rescue_winch.mp3, rescue_safe.ogg, rescue_safe.mp3, confirmation_004.ogg, crew_help.mp3, and question_001.ogg. They are optional fallback names, not mandatory missing dependencies. Adding one is useful only if its format is supported and it is placed in a searched directory.

Before reorganization the wider sounds/ tree did include rescue_safe.ogg and Kenney Audio/confirmation_004.ogg and Audio/question_001.ogg, along with many unused Kenney interface clips. Those duplicate/unused Ogg assets were removed during cleanup while the eight active files and license were retained. A separately supplied old/external pack would not be discovered unless its files were placed in the organized searched asset locations. More importantly, the current miniaudio implementation does not enable Vorbis decoding, so a discoverable .ogg fallback still would not become playable just because its file exists. The primary WAV/MP3 cues avoid this issue.

assets/audio/Kenney-License.txt is attribution/license material, not a decoded cue. Asset presence was established by filename inspection, not by listening to every file, proving its binary encoding, or validating the semantic content implied by names such as crew_help.wav. Runtime load success is a separate check.

6.4 update: Parameters, Math, Every Branch, and Event Semantics

6.4.1 Complete Parameter Contract

Final lines 137-154; original lines 139-156. The function returns void; it consumes a snapshot rather than querying mission globals itself.

Parameter	Type	Meaning and actual caller input
dt	float	Frame time in seconds. Used by exponential smoothing and crew timer; not a count of audio samples.
camPos	const glm::vec3&	Smoothed camera eye position; becomes listener position.
camTarget	const glm::vec3&	Smoothed camera target; determines listener forward vector.
ornPos	const glm::vec3&	Ornithopter position, used by flight, boost, and pickup sounds.
ornSpeed	float	Ornithopter speed scalar; drives artificial flight pitch rather than velocity-based Doppler.
isBoosting	bool	Boost-input/gameplay state; reduces flight-loop gain and can raise boost-loop gain.
harvPos	const glm::vec3&	harvesterPosition(); harvester sound origin.
wormPos	const glm::vec3&	wormPosition(); rumble origin and breach origin at trigger time.
wormDist	float	wormGap(); a gameplay separation measure driving extra rumble modulation. It is not the listener-source distance calculated by miniaudio.

Parameter	Type	Meaning and actual caller input
attackTime	float	wormAttackTime(), defined as mission elapsed time minus breachAt; negative before breach and nonnegative afterward.
rescuedCount	int	Cumulative delivered crew; an increase causes a safe-delivery cue.
aboardCount	int	Current onboard crew; an increase causes a pickup cue.
pickupProgress	float	Current winch/pickup progress; the upward crossing of 0.08f causes a pickup cue.
nearestCrewPos	const glm::vec3&	nearestCrewPosition(); selected help-call origin.
nearestCrewDist	float	Caller computes horizontalDistance(g_ornPos, nearestCrewAudio); this is horizontal ornithopter-to-crew distance, not camera distance.
basePos	const glm::vec3&	Mission base position; safe-delivery sound origin.
isFlying	bool	Exact test g_mission.phase == MissionPhase::Flying; not inferred from speed or altitude.
isPaused	bool	Mission pause flag; suppresses master output and further wrapper processing.

The original call is in `arrakis.cpp` lines 1683-1703. Position references are borrowed for this call only. `miniaudio` setters copy scalar/vector values into its objects; the audio callback does not retain references to these temporary snapshot arguments.

6.4.2 Engine Guard, Pause, and Mute

Final lines 155-164; original lines 157-166. The opening brace begins the update body; its matching close is final line 268.

If `!initialized`, return without touching vendor state. Otherwise, if `isPaused || muted`, set engine gain to `0.0f` and return. The else path restores `masterVolume` every active frame.

This is **output muting, not transport pausing**. The engine/device is not stopped, sound objects are not stopped, decoder cursors can continue, and a playing one-shot can finish inaudibly. The early return also freezes the wrapper's listener updates, pitch/boost interpolation, crew-help timer, breach latch processing, and `last*` snapshots. When unmuting, count differences accumulated during silence may cause delayed winch or delivery triggers. A breach occurring entirely outside the `0 ≤ attackTime < 2.5` interval while muted can be missed permanently. Pause normally also pauses mission progression elsewhere, whereas mute alone does not.

6.4.3 Listener Direction

Final lines 166-172; original lines 168-174.

`look = camTarget - camPos` constructs the camera forward displacement. If `glm::length(look) > 0.001f`, it is normalized to unit length. Otherwise `look = glm::vec3(0, 0, -1)` provides a valid negative-Z forward direction rather than dividing an almost-zero vector by its length. Equality at exactly `0.001f` takes the fallback branch.

`ma_engine_listener_set_position(&engine, 0, camPos.x, camPos.y, camPos.z)` and `ma_engine_listener_set_direction(&engine, 0, look.x, look.y, look.z)` both target listener zero.

Camera movement changes perceived panning/attenuation even when gameplay objects do not move. No listener velocity or source velocity is supplied, so the library's Doppler capability is not driven by the game's actual velocities.

6.4.4 Exponential Smoothing, with Exact Interpretation

The flight and boost branches use `glm::mix(current, target, 1 - exp(-k*dt))`. For scalars, mix is $(1-a)*current + a*target$. Therefore the update is:

```
a = 1 - exp(-k * dt)
next = target + (current - target) * exp(-k * dt)
```

For constant targets and nonnegative finite `dt`, the remaining error decays exponentially, and successive updates compose by elapsed time. This is more frame-rate-independent than using `k*dt` directly as an unclamped lerp weight. `dt = 0` changes nothing; large positive `dt` approaches the target without overshoot. Negative `dt` yields a negative interpolation weight and moves away from the target; the wrapper does not validate it. NaNs/infinities likewise are not rejected here.

The flight rate is `k = 6.0f`, time constant $1/6$ second and half-life $\ln(2)/6$, approximately 0.1155 seconds. The boost rate is `k = 8.0f`, time constant 0.125 seconds and half-life about 0.0866 seconds. At 60 Hz, the weights are approximately 0.09516 and 0.12483 . These application-side states are distinct from miniaudio's own short per-channel gain smoothing.

6.4.5 Flight Loop

Final lines 174-187; original lines 176-189.

If `hasFlight` is false, the entire branch is skipped. Otherwise source position follows `ornPos` every processed frame. The target pitch is exactly $0.92f + \text{glm::clamp}(\text{ornSpeed} / 42.0f, 0.0f, 0.38f)$. It is bounded from 0.92 to 1.30 ; the upper clamp is reached at speed $42*0.38 = 15.96$, **not at speed 42**. Nonpositive speed yields 0.92 ; speed 8.4 yields 1.12 . The divisor is a gain scale, not a normalized full-speed endpoint.

The exponential mix updates `flightPitch`, then `ma_sound_set_pitch` applies it as a playback-rate/pitch factor. Changing pitch also changes the rate at which the sound's audio content is consumed; it is not time-preserving spectral pitch shifting.

`targetVol = isBoosting ? 0.40f : 0.78f` switches flight gain immediately, with no wrapper volume smoothing. The initial $0.75f$ is replaced by one of these values on the first update. Boosting lowers the ordinary flight voice so the boost voice can dominate.

If `!ma_sound_is_playing(&sndFlight)`, start it. There is **no isFlying condition** and no flight-stop branch. A loaded flight loop therefore runs in menu/end phases as long as updates pass pause/mute guards, and its location continues to follow the ornithopter. Describing this as strictly flight-phase-only would be incorrect.

6.4.6 Boost Loop

Final lines 189-195; original lines 191-197.

The `hasBoost` guard protects the sound. Position follows `ornPos`. The target is `(isFlying && isBoosting) ? 0.90f : 0.0f`; both flags must be true. `boostVolume` is smoothed with rate $8.0f$, then applied to the sound. Initialization already started this looping sound, so update does not repeatedly start/stop it and does not restart its waveform when boost begins. After release or leaving the Flying phase, the volume decays toward zero rather than instantly reaching it. Muted/paused frames freeze the application-side decay even though the loop's playback cursor continues.

6.4.7 Harvester Loop

Final lines 197-205; original lines 199-207.

If loaded, always update its position to `harvPos`. If `isFlying && attackTime < 11.0f`, start only if not already playing. Otherwise stop only if currently playing. Because `attackTime` is breach-relative, the `< 11` condition includes

all negative pre-breach times and permits playback for the first eleven seconds after breach. At exactly 11.0f, it takes the stop branch. A stop does not seek to frame zero; restarting generally resumes the source rather than rewinding unless it was at the end. The initial gain remains 0.75f because update never changes it.

6.4.8 Worm Rumble and Two Different Distances

Final lines 207-218; original lines 209-220.

If loaded, update position to wormPos. `isFlying && attackTime < 16.0f` enables the loop, again including all pre-breach times. In this branch:

```
t = clamp(wormDist / 200.0, 0.0, 1.0)
proximityBoost = mix(1.2, 0.8, t) = 1.2 - 0.4*t
```

The gain is 1.2 for distance zero or less, 1.0 at distance 100, and 0.8 at distance 200 or more. Despite its name, this value is not restricted below unity. It replaces the initial 0.9f sound gain. This gameplay modulation uses `wormGap()`; the spatializer separately applies source-to-**camera-listener** attenuation from the source's 3D position. Both factors contribute to the final signal.

Start only when needed; if the phase/time predicate fails, stop only when playing. At `attackTime == 16.0f`, stop. No local fade is requested in the stop branch and no new position is assigned for an absent cue.

6.4.9 Breach One-Shot

Final lines 220-228; original lines 222-230.

The loaded-sound branch contains `attackTime >= 0.0f && attackTime < 2.5f && !breachTriggered`. Zero is inclusive; 2.5 is exclusive. Once eligible, set `breachTriggered = true`, set position to the worm's **current trigger-time** position, seek to PCM frame 0, and start. Position is not updated again after the event, so a moving worm does not drag an already-triggered roar through space.

The latch is set before the ignored seek/start result codes. A failed start still consumes the once-per-mission event in this wrapper. The predicate does not include `isFlying`; time and latch determine eligibility if the engine update is active. If frames skip the entire 2.5-second window, no catch-up branch exists. `resetMission` re-arms it.

6.4.10 Winch/Pickup One-Shot

Final lines 230-238; original lines 232-240.

Trigger when `aboardCount > lastAboard` or `(pickupProgress > 0.08f && lastPickup <= 0.08f)`. The first condition detects any positive onboard-count change, not one sound per person. The second detects an upward threshold crossing from below-or-equal to strictly above 0.08. Remaining above the threshold produces no new threshold event. Falling below it and later crossing again can trigger another. A simultaneous count increase and threshold crossing still runs the body only once.

The body sets the source to `ornPos`, requests seek frame zero, and starts. This cue was initialized with `loop = false`. The source comment saying "Trigger or loop while picking up" is not an implementation of continuous looping. Long pickups do not continuously retrigger unless a new edge appears. Retriggering while the single sound is playing schedules a seek on the existing voice rather than mixing a second voice.

6.4.11 Safe Delivery One-Shot

Final lines 240-247; original lines 242-249.

If `hasRescueSafe` and `rescuedCount > lastRescued`, locate the cue at `basePos`, seek to zero, and start. A batch delivery with multiple rescued people emits one trigger for that processed frame. Equal or smaller totals do not trigger. The cue is spatialized at the base, not played as an unconditional nonspatial UI sound, so camera distance can make confirmation quiet. The safe-delivery branch has no separate phase restriction.

6.4.12 Periodic Crew Help

Final lines 249-262; original lines 251-264.

Proceed only if `hasCrewHelp && isFlying`. Subtract `dt` from `crewHelpTimer`. When `crewHelpTimer <= 0.0f`, check the **strict** interval `nearestCrewDist > 0.0f && nearestCrewDist < 75.0f`. Exactly zero and exactly 75 are excluded. An eligible crew member triggers position assignment, seek to zero, and start, then:

```
crewHelpTimer = 5.5f + (rand() % 30) * 0.1f
```

`rand() % 30` gives an integer from 0 through 29. The timer is therefore one of 30 nominal values 5.5, 5.6, ..., 8.4 seconds, not a continuous uniform float and not a range ending at 8.5. Modulo reduction can have small distribution bias; this code does not create a dedicated random generator or seed it locally. It consumes the program's shared C random sequence.

If the distance is ineligible, set the timer to 2.0f seconds and retry later. A large `dt` still produces at most one call/check because there is no catch-up while-loop. Negative overshoot is discarded when resetting the timer. No decrement occurs outside Flying, during pause/mute, or when the sound is absent. The first normal mission call is eligible after approximately three processed seconds, provided a crew member is within the strict distance interval. The selected source position is captured only at trigger time.

6.4.13 End-of-Frame State

Final lines 264-268; original lines 266-270. After all sound branches, unconditionally assign `lastRescued = rescuedCount`, `lastAboard = aboardCount`, and `lastPickup = pickupProgress`. "Unconditionally" here means only after getting past the engine/pause/mute early returns. The snapshots advance even if the corresponding cues are absent, preventing missing assets from continually generating the same logical edge. They are not updated when a paused/muted call returns early.

6.4.14 Spatial Attenuation Math from the Actual Vendor Code

`miniaudio.h` lines 51664-51688 implement inverse, linear, and exponential distance gains. The active inverse formula is:

```
if minDistance >= maxDistance: gain = 1
otherwise:
    d = clamp(distance, minDistance, maxDistance)
    gain = minDistance / (minDistance + rolloff*(d - minDistance))
```

Default rolloff is 1; then gain simplifies to `minDistance/d`. The distance is the listener/source relative vector's 3D length, with gain limits and directional/channel processing applied by the spatializer afterward. At or below the minimum distance, inverse attenuation is unity. Above the maximum distance, the distance is clamped and attenuation stops decreasing. There is no implicit hard culling at `maxDist`.

Cue	Inverse gain at its maximum distance, with default rolloff 1
Flight and boost	$12/150 = 0.08$
Worm rumble	$30/320 = 0.09375$
Worm breach	$45/380$, approximately 0.11842
Harvester	$18/200 = 0.09$
Winch	$15/120 = 0.125$
Safe delivery	$20/140$, approximately 0.14286
Crew help	$16/85$, approximately 0.18824

The unused linear alternative is $1 - \text{rolloff} * (d - \text{min}) / (\text{max} - \text{min})$. The unused exponential alternative is $(d / \text{min})^{(-\text{rolloff})}$. All three return unity when `min >= max` to avoid invalid division. In the default spatializer

configuration, source positioning is absolute, handedness is right-handed, forward is negative Z, min/max gain are 0/1, cones are full-circle 6.283185f, Doppler and directional factors are 1, and the minimum spatialization channel gain is 0.2f. Standalone spatializer smoothing defaults to 360 frames; the engine derives its own default 8 ms smoothing duration. The wrapper overrides attenuation distances/model, not the whole spatializer configuration.

A useful schematic, not a complete implementation equation for multichannel spatialization, is `audible sample = decoded sample * cue gain * spatial/channel gain * master gain`. The engine sums voices and clips its output; rumble gain above one and multiple overlapping voices can therefore increase clipping risk. No occlusion, environmental reverb, custom filter node, underwater/subterranean propagation model, or velocity-driven Doppler is configured by Arrakis.

6.5 Mute, Mission Reset, Shutdown, and API-Lifetime Details

6.5.1 `toggleMute()`

Final lines 131-135; original lines 133-137. No parameters or return. First flip `muted = !muted`; then, if the engine is uninitialized, return while preserving the new user state. Otherwise immediately set engine gain to 0.0f when muted or `masterVolume` when unmuted. The keyboard M handler invokes this method, and the HUD reads `g_audio.muted`.

The method does not accept or examine `isPaused`. Consequently unmuting during pause can momentarily restore engine gain until the next update mutes it again. This follows from the source; it is not an intentional pause-state integration supplied by `miniaudio`.

6.5.2 `resetMission()`

Final lines 270-276; original lines 272-278. It sets `breachTriggered = false`, `crewHelpTimer = 3.0f`, `lastRescued = 0`, `lastAboard = 0`, and `lastPickup = 0.0f`. Game-level mission reset calls this routine before rebuilding mission state.

It does not need a device and is safe before initialization because it changes only wrapper scalars. It does **not** reset `boostVolume`, `flightPitch`, `muted`, `masterVolume`, any `has*` flag, sound position, playback cursor, or playing state. Existing one-shots can continue into a restarted mission; boost/pitch smoothing history can carry across resets. The method re-arms events, not the whole audio engine.

6.5.3 `shutdown()`

Final lines 278-290; original lines 280-292. If uninitialized, return. Otherwise uninitialize every successfully created sound in this exact order: flight, boost, rumble, breach, winch, safe delivery, harvester, crew help. Then call `ma_engine_uninit(&engine)` and set `initialized = false`.

Destroying sounds first keeps their engine/resource manager alive during sound teardown. `ma_sound_uninit` detaches the sound's engine node before freeing caches and owned resource-manager data sources. `ma_engine_uninit` stops/uninitializes the owned device before tearing down the graph so the device's callback cannot read freed nodes; it then cleans up owned resources and listener state. The outer initialized guard makes a second wrapper shutdown a no-op even though the `has*` flags are not cleared.

There is no custom destructor or RAII owner in `ArrakisAudio`; normal application exit explicitly calls `g_audio.shutdown()`. Failure paths that terminate before this call must be considered separately. A future owner should not rely on aggregate destruction of `ma_engine` to free vendor allocations.

6.5.4 Exact Vendor Calls Used by the Wrapper

API	Inputs in Arrakis	What actually happens
ma_engine_config_init	None	Produces logical defaults; does not create a device. Definition at vendor line 77455.
ma_engine_init	Config pointer, stable engine address	Negotiates/creates device, graph, listeners, and internal resource manager; can fail with a ma_result. Definition 77521.
ma_engine_listener_set_world_up	Engine, index 0, (0, 1, 0)	Establishes listener orientation's up axis.
ma_sound_init_from_file	Engine, path, flags 0, group null, fence null, destination	Wraps sound config and creates a resource-managed data source and engine node. Definition 78567.
ma_sound_set_looping	Sound, MA_TRUE/MA_FALSE	Changes data-source loop behavior; not a call to start playback.
ma_sound_set_min_distance, ma_sound_set_max_distance, ma_sound_set_attenuation_model	Sound and configured scalar/enum	Update spatializer attenuation configuration.
ma_sound_set_volume	Sound and linear gain	Delegates to engine-node volume state. Definition 78832.
ma_engine_set_volume	Engine, zero or masterVolume	Controls endpoint/master output gain; does not pause source transport.
ma_engine_listener_set_position, ma_engine_listener_set_direction	Engine, listener 0, three components	Set listener spatial values; positions/directions use vendor synchronized vector storage.
ma_sound_set_position	Sound and three world components	Sets source spatial position; no pointer to the caller's GLM vector is retained.
ma_sound_set_pitch	Sound and flightPitch	Changes engine-node resampling/pitch ratio.
ma_sound_is_playing	Sound pointer	Queries node/playback state to avoid unnecessary starts/stops.
ma_sound_start	Sound pointer	Returns success if already playing; if at end, seeks/re-arms; sets node state started. Definition 78746.
ma_sound_stop	Sound pointer	Sets stopped node state; does not destroy or automatically rewind. Definition 78774.
ma_sound_seek_to_pcm_frame	Sound, frame 0	Schedules an atomic seek target for the mixing thread; actual seek is not performed synchronously by this wrapper call. Definition 79425.
ma_sound_uninit	Each valid sound pointer	Detaches node, releases processing cache and owned source; definition 78699.
ma_engine_uninit	Engine pointer	Stops callback access before releasing graph/device/manager/listener resources; definition 77795.

The wrapper checks engine/sound **initialization** results but ignores later start, stop, seek, and master-volume results. A gameplay event can be consumed even if the device or underlying source later fails. Documentation should distinguish intended cue dispatch from confirmed audibility.

6.6 miniaudio_impl.cpp and Compile-Time Selection

6.6.1 Both Lines of the Implementation Unit

Line 1 is `#define MINIAUDIO_IMPLEMENTATION`; line 2 is `#include "miniaudio.h"`. That macro selects the definitions below the public header region. It must be used in **one** compiled translation unit to avoid multiple external definitions. Other game headers include `miniaudio.h` without it and see declarations/types only.

The public header guard is `miniaudio_h`; the implementation has a separate `miniaudio_c` guard and is selected by either `MINIAUDIO_IMPLEMENTATION` or `MA_IMPLEMENTATION` at line 11552. Separate guards permit the common pattern of including declarations and later including an implementation in the same translation unit. Editor-analysis macros around lines 11538-11543 can enable implementation visibility for IntelliSense/Qt Creator/CDT; that is not the normal executable build's source-selection mechanism.

CMake and the inspected Visual Studio project compile the main application, GLAD C unit, and this miniaudio C++ unit. Merely retaining another .c file on disk does not compile it. Include directories make `miniaudio.h` resolvable from its organized third-party location; physical relocation by itself does not enable a codec.

6.6.2 Is stb_vorbis Compiled or Necessary?

The answer for the inspected project is **no, it was never compiled or included and has now been removed; no, it is not necessary for the eight active preferred WAV/MP3 files**. The two-line implementation unit does not include it. Neither executable source list contains it. The miniaudio Vorbis adapter begins at line 65330 under `#ifdef STB_VORBIS_INCLUDE_STB_VORBIS_H`; that macro is defined by `stb_vorbis`'s declaration section, but no project source makes those declarations visible before the miniaudio implementation.

To use this particular optional adapter, `stb_vorbis` declarations must be visible when miniaudio's implementation is compiled and `stb_vorbis` definitions must be linked exactly once. A declaration-only include using `STB_VORBIS_HEADER_ONLY`, followed by miniaudio implementation and one separately compiled `stb_vorbis` implementation, is one possible integration design. A single-translation-unit inclusion arrangement is another. These are explanations of requirements, **not changes made to the build**. Adding `stb_vorbis.c` alone to CMake without exposing its declarations to miniaudio does not activate the adapter. Defining its header guard alone without declarations is also not valid integration.

An alternate custom Vorbis decoder could satisfy Ogg support instead. Vorbis would become necessary only if the game relies on Ogg assets and chooses this codec implementation. The filename suffix .ogg denotes a container; arbitrary Ogg-contained codecs such as Opus are not decoded by a Vorbis-only decoder.

6.6.3 Compile-Switch Families and Dependencies

Build-switch documentation is at lines 547-718, platform/type detection starts at 3860, backend feature selection is at 6654-6699, and implementation selection is at 11552.

Switch family	Effects and cautions
MA_NO_WASAPI, MA_NO_DSOUND, MA_NO_WINMM, MA_NO_ALSA, MA_NO_PULSEAUDIO, MA_NO_JACK, MA_NO_COREAUDIO, MA_NO_SNDIO, MA_NO_AUDIO4, MA_NO_OSS, MA_NO_AAUDIO, MA_NO_OPENSL, MA_NO_WEBAUDIO, MA_NO_CUSTOM, MA_NO_NULL	Remove individual device backends. They do not automatically remove the engine's platform-neutral DSP.
MA_ENABLE_ONLY_SPECIFIC_BACKENDS and matching MA_ENABLE_* macros	Change backend selection to an explicit allowlist. Platform support is still required; enabling ALSA on Windows does not manufacture Linux APIs.
MA_NO_DEVICE_IO	Removes context/device playback/capture APIs; allows decode/conversion-only uses, or a no-device engine with explicit channels/sample rate. Not selected in Arrakis.

Switch family	Effects and cautions
MA_NO_THREADING	Removes thread/mutex/semaphore/event support. Device I/O must also be disabled; worker behavior and resource-manager configuration change.
MA_NO_DECODING, MA_NO_ENCODING	Remove decoder or encoder API sections. The resource manager requires decoding; disabling decoding forces the related dependency change.
MA_NO_WAV, MA_NO_FLAC, MA_NO_MP3; optional MA_NO_VORBIS	Remove individual built-in/adapted codecs. WAV also supplies the built-in encoder. None of these are defined in the two-line implementation unit.
MA_NO_RESOURCE_MANAGER	Removes resource-managed file loading and file-based engine helpers, including the exact <code>ma_sound_init_from_file</code> API the game needs. External data sources remain possible.
MA_NO_NODE_GRAPH, MA_NO_ENGINE	Remove graph or high-level engine APIs. The engine depends on the graph; Arrakis needs both.
MA_NO_GENERATION	Removes waveform/noise generator APIs; the game does not use them.
MA_NO_SSE2, MA_NO_AVX2, MA_NO_NEON	Remove CPU-specific optimization paths while retaining scalar implementations where available.
MA_NO_RUNTIME_LINKING	Replaces dynamic system-library loading requirements with directly linked backend symbols; may require additional linker configuration.
MA_USE_STDINT, MA_API, allocation/assert/log customization macros	Control sized types, symbol decoration, allocation hooks, diagnostics, and embedding policy rather than cue semantics.
MA_COINIT_VALUE, MA_FORCE_UWP	Windows COM initialization and Windows-app backend behavior. Default COM model is multithreaded.
MA_ON_THREAD_ENTRY, MA_ON_THREAD_EXIT, MA_THREAD_DEFAULT_STACK_SIZE	Hooks/stack size for internally created threads; must not be confused with gameplay frame callbacks.
MA_ENABLE_AUDIO_WORKLETS	Emscripten worklet path; requires matching toolchain options such as <code>-SAUDIO_WORKLET=1</code> <code>-sWASM_WORKERS=1</code> <code>-sASYNCIFY</code> . Not the Windows path.

Declarations, implementation, and consumers must agree on configuration macros and struct layouts. miniaudio explicitly does not guarantee ABI compatibility even across patch versions. The safest use here is the supplied header and implementation compiled together, not arbitrary interchange with another binary miniaudio version.

6.7 miniaudio: Complete Coverage Map and Functional Architecture

6.7.1 Complete Contiguous Source Map

All numbers in this subsection refer to the inspected 95,864-line `miniaudio.h`, unchanged by relocation. Boundaries include separator comments/blank lines so there are no uncovered source regions. A broad bin can contain shared helpers belonging to the adjacent family; the declaration and definition names provide the exact semantic entry points.

Lines	Functional contents
1-3737	Version/provenance; complete in-header manual, examples, build switches, low/high-level APIs, resource manager, graph, decoding/encoding, conversion, filters, generation, buffering, backends, and optimization notes
3738-4515	Public guard/linkage, fixed-width/platform types, result/error enums, formats/channels, allocation callbacks, atomic/thread type declarations, version APIs

Lines	Functional contents
4516-4590	Logging callback/configuration APIs
4591-5003	Biquad, low-pass, high-pass, band-pass, notch, peaking EQ, low/high shelf declarations
5004-5317	Delay, gainer, stereo panner, fader, vector/spatializer/listener declarations
5318-5830	Linear/general resampling, channel conversion, data conversion, format conversion, channel-map helpers
5831-6111	Data-source base/vtable APIs; borrowed/owned/paged audio buffers; ring-buffer declarations
6112-6593	Results/allocations/miscellaneous helpers, spinlocks and OS synchronization, fences, notifications, slot allocation, job queue
6594-9751	Backend selection, context/device IDs/configs/callbacks/native structs; enumeration, lifecycle, volume, buffer-size and device helper APIs
9752-9865	PCM copying, silencing, clipping, volume/mix/interleave utilities
9866-9944	Virtual filesystem and file-I/O abstraction declarations
9945-10110	Generic decoder and custom decoding-backend APIs
10111-10169	Encoder configuration/callback/API declarations
10170-10296	Waveform, pulsewave, and white/pink/brownian noise declarations
10297-10614	Resource-manager configuration, flags, buffer/stream/source objects, registration, notifications, job processing
10615-11107	Node graph, input/output buses, node/vtable/state flags, endpoint/source/splitter and filter/delay node APIs
11108-11535	Engine/engine-node/sound/group configuration, listener/control/scheduling APIs and public guard closure
11536-13569	Implementation gate/includes; architecture/SIMD/compiler detection; platform linkage, memory/string and portability groundwork
13570-18578	Logging definitions; internal math/random/vector/string/platform helpers, atomics, thread creation and synchronization, fences/notifications
18579-19470	Slot allocator and bounded job queue implementation
19471-20933	Common device-I/O definitions, backend platform types/constants, conversions, descriptors, and shared setup
20934-21707	Null backend: virtual timing/buffering device without physical output
21708-25216	WASAPI backend, COM/interface shims, endpoint enumeration, format negotiation, event/client processing, notifications/routing
25217-27001	DirectSound backend and playback/capture/notification interface/buffer management
27002-28101	WinMM waveform input/output backend
28102-30262	ALSA PCM backend, including enabled body through 30254 and boundary/shared setup

Lines	Functional contents
30263-32799	PulseAudio backend and server/mainloop/stream interactions
32800-33476	JACK client, port, sample-rate and callback backend
33477-36716	Core Audio/AudioUnit backend and Apple device/routing handling
36717-37563	sndio backend
37564-38460	BSD audio(4) backend
38461-39094	OSS backend
39095-40230	AAudio backend, error/reroute jobs, and absent-backend no-op job glue
40231-41494	OpenSL ES backend and Android object/queue/channel mapping
41495-42450	Web Audio JavaScript/Emscripten bridges and optional worklet processing
42451-44574	Shared context/backend dispatch, enumeration, device lifecycle, device jobs, state transitions, period/buffer selection
44575-45089	Buffer-duration calculations; PCM copy/silence/clip/volume/mix helpers
45090-47163	Sample-format conversion, scalar/SIMD conversion paths, normalization/dithering/interleaving support
47164-50374	Biquad and all filter-family implementations
50375-53026	Delay, gainer, panner, fader, listener/spatializer, attenuation, cone, Doppler and directional gains
53027-54276	Linear and generic/custom-vtable resampling implementations
54277-55901	Channel conversion, mono expansion, rearrangement, weighted mixing
55902-57072	Composite data converter and frame-count/latency processing
57073-58096	Channel maps and one-shot conversion helper implementations
58097-58922	Byte/PCM ring buffer operations and duplex buffering support
58923-60903	Result descriptions, allocation wrappers, sample-size helpers, data-source operations, borrowed/owned/paged audio-buffer implementations
60904-61933	VFS callback dispatch and default filesystem implementation; generated-decoder-header preamble
61934-62368	Embedded dr_wav 0.14.5 declarations, structs, metadata, read/write/seek/format conversion APIs
62369-62662	Embedded dr_flac 0.13.3 declarations, bitstream/metadata/seek/read APIs
62663-62839	Embedded dr_mp3 0.7.3 declarations, frame/decoder state, read/seek/metadata APIs
62840-65329	Generic decoding groundwork and WAV/FLAC/MP3 data-source/backend adapters
65330-66159	Optional stb_vorbis adapter, excluded unless stb_vorbis declarations are visible

Lines	Functional contents
66160-67830	Generic decoder initialization/probing, callback/file/memory paths, output conversion, reading/seeking/length/cursor helpers
67831-68147	Built-in WAV encoder adapter, encoder initialization/writing/teardown
68148-69308	Waveform/pulsewave and noise generator implementations
69309-73426	Resource manager: path/hash registry, cached buffers, streamed pages, job dispatch, synchronization, source operations
73427-76536	Node graph: graph traversal/mixing, attachment lists, state/scheduling, source/splitter/filter/delay nodes
76537-79893	Engine, listeners, engine-node processing, sound/group loading/control/seeking/fades and inlined sounds
79894-79907	Embedded codec-implementation preamble and warning/integration setup
79908-84837	Embedded dr_wav implementation
84838-92696	Embedded dr_flac implementation
92697-95804	Embedded dr_mp3/minimp3-derived implementation
95805-95864	Warning-state restoration, implementation guards, public-domain/MIT No Attribution license alternatives

6.7.2 Object Model, Results, Memory, and PCM Units

The main recurring pattern is `*_config_init`, `*_init` or `*_init_preallocated`, `*_process_pcm_frames/read/control` calls, and `*_uninit`. Configs are temporary descriptions; initialized objects are long-lived, noncopyable-in-practice structures containing caches, locks, pointers, and ownership bits. Some components expose `*_get_heap_size` plus preallocated initialization to let the application provide one suitably aligned backing block; ordinary initialization allocates using `ma_allocation_callbacks`.

`ma_result` is a signed result enum: `MA_SUCCESS = 0`, with negative error/status values. General argument/memory/I/O errors, no-data/at-end states, unsupported-format errors, device/backend failures, and cancellation are distinct. Device errors include `MA_NO_BACKEND = -203`, `MA_BACKEND_NOT_ENABLED = -208`, and initialization/open/start/stop backend errors `-400` through `-403`. `ma_result_description` at line 58926 translates known result values to readable text; Arrakis prints numeric initialization results instead. Success from sound start means the request/state transition was accepted, not that the operating-system speakers audibly emitted the cue.

`ma_malloc`, `ma_realloc`, `ma_free`, aligned allocation, and copied allocation callbacks keep allocation/free policy paired. Borrowed data must outlive its reader; owned data is released by the corresponding `uninit`. Resource-manager buffers can be shared/refcounted, while independent decoder cursors belong to their consumers. Caller-provided memory and an external manager/device must not be freed as if owned; ownership flags decide teardown.

A PCM **frame** contains one sample for each channel. For interleaved audio, bytes required are `frameCount * channels * bytesPerSample`; stereo f32 uses 8 bytes per frame. A stereo frame count of 480 is 10 ms at 48 kHz, not 960 ms or 960 frames. f32 is conventionally `[-1, 1]`; s16 is `[-32768, 32767]`; packed s24 uses three bytes; s32 uses four; unsigned u8 silence is **128**, while signed/float silence is zero. The source's `ma_silence_pcm_frames` explicitly handles that distinction at lines 44606-44616.

Conversion and decode functions often return both a result and an actual frame count, or update consumed/produced count pointers. The caller must respect partial output and buffer capacities instead of assuming a requested count was produced. Frame-index APIs can use 64-bit types even when an underlying optional decoder has a narrower seek limitation.

6.7.3 Logs, Atomics, Threads, Notifications, and Jobs

Logging (`ma_log_*`) stores registered callbacks, severity, user data, and synchronization around callback registration/posting. Version/math/string/vector and platform helpers underpin all later subsystems; they are not game logic. SIMD dispatch selects suitable implementations after compiler/architecture detection rather than assuming every CPU supports all enabled instruction sets.

Atomic helpers implement integer/pointer/float access and synchronized vector snapshots using supported compiler intrinsics or fallback locking. They let the game publish spatial/control state while the mixer consumes it. This does **not** imply that arbitrary direct edits to transparent struct members are safe, or that every public function is callback-safe.

`ma_thread_*`, `ma_mutex_*`, `ma_event_*`, `ma_semaphore_*`, and spinlocks abstract Windows and pthread/platform primitives. Events are wakeup mechanisms; a fence tracks outstanding work and permits waiting until completion; asynchronous notifications can use callbacks, event/fence adapters, or their combinations.

`ma_sound_seek_to_pcm_frame` illustrates deferred ownership: the game thread atomically publishes a target; the thread reading audio performs the actual decoder seek, avoiding concurrent decoder mutation.

The slot allocator reserves a fixed collection of job storage slots with generation/counter bookkeeping, reducing allocation churn and avoiding reuse confusion. The bounded job queue posts and retrieves typed jobs, optionally waits or reports `MA_NO_DATA_AVAILABLE`, and uses quit/cancellation semantics. Jobs include resource load/free/page/seek work and AAudio rerouting; a queue-capacity failure is different from decoding failure. The resource manager defaults to **one** managed job thread, as seen at line 69977, not one thread per sound.

The device's realtime callback must produce exactly the requested frame region without blocking filesystem work, expensive unbounded allocation, or waiting on device lifecycle operations. The manual explicitly forbids device `init/uninit/start/stop` from within its own callback because that can deadlock. Arrakis performs these lifecycle actions from application setup/shutdown, not from a custom device callback.

6.7.4 Device/Context Abstraction and Backend Families

`ma_context` selects a usable backend, holds global backend state/function tables, manages logs/allocation callbacks, and enumerates playback/capture devices. `ma_device_id` is a backend-specific identifier; display names are not a universal persistent device handle. `ma_device_config` expresses playback/capture format, channels/maps, rate, share mode, periods, callbacks, user data, and backend options. `ma_device` owns negotiated native state, conversion paths, intermediate buffers, synchronization, and start/stop state.

Device types are playback, capture, duplex, and supported loopback modes. Playback callbacks write output; capture callbacks read input; duplex can do both. Native device format can differ from callback format, so conversion occurs at the boundary. Period sizes, native sample rates, backend buffering, and callbacks determine actual latency; the game's graphics dt does not drive sample delivery.

Default backend priority from the supplied manual is WASAPI, DirectSound, WinMM, Core Audio, sndio, audio(4), OSS, PulseAudio, ALSA, JACK, AAudio, OpenSL ES, Web Audio, Custom, Null. Unavailable/uncompiled backends are skipped; initialization can fall through until a usable backend is found. A Windows compilation excludes Linux/BSD/Apple/Android implementations through feature guards even though their source remains in the header.

Backend	Native mechanism, threading/buffering, and limits
WASAPI; 21708-25216	Windows COM MMDevice/AudioClient interfaces, endpoint discovery, shared/exclusive format negotiation, event/client buffering, render/capture services, loopback, endpoint notifications and rerouting. COM startup and some commands need carefully chosen threads. Low-latency shared mode has sample-rate/autoconversion caveats; UWP has default-device and capability restrictions.
DirectSound; 25217-27001	Windows playback/capture buffer interfaces and notifications. Ring/circular-buffer positions and notifications drive chunk transfer. Interface release and DLL ownership are separate from sound data ownership. This is a fallback backend, not OpenGL's Direct State Access.

Backend	Native mechanism, threading/buffering, and limits
WinMM; 27002-28101	Older Windows waveOut/waveIn style devices and queued waveform headers/buffers. Completion/requeue logic feeds output or gathers input; format support and latency differ from WASAPI.
ALSA; 28102-30262	Linux PCM devices, hardware/software parameter negotiation, read/write or supported mapped transfer, polling/wakeup and recovery for underruns/overruns. Dynamic libasound resolution avoids requiring its development headers in the normal embedded build.
PulseAudio; 30263-32799	Server-based playback/capture streams, context/mainloop synchronization, server/device enumeration and stream buffering/latency requests. Server policy can route devices automatically; the game does not issue PulseAudio calls directly.
JACK; 32800-33476	JACK client/ports and server realtime process callback. Server rate/channel/period constraints and port connection define operation; callback processing must fit realtime deadlines.
Core Audio; 33477-36716	Apple device/AudioUnit machinery, render/capture callbacks, stream properties and route changes. macOS/iOS platform branches have different device/default-route behavior and Apple linkage paths.
sndio; 36717-37563	OpenBSD libsndio handle/options and PCM transfer with polling/drain/volume support as provided by the backend. Normally enabled on OpenBSD only.
audio(4); 37564-38460	NetBSD/OpenBSD device files and audio ioctl/configuration structures with read/write transfer. On OpenBSD, an active sndiod can affect availability.
OSS; 38461-39094	FreeBSD/OSS-style device file and ioctl format/rate/channel negotiation, read/write/polling; platform-specific capability/device constraints.
AAudio; 39095-40230	Android stream builder/stream APIs, realtime data/error callbacks, usage/content/input presets, disconnect handling and reroute jobs. Android 8+; native enumeration is limited to default devices unless the app supplies Java-discovered IDs.
OpenSL ES; 40231-41494	Android engine/mix/player/recorder objects, PCM channel-mask translation and buffer-queue callbacks. Single application-level engine/context constraints, queued buffer lifetime and API 16+ requirements matter.
Web Audio; 41495-42450	Emscripten JavaScript audio context/device bridges, script processing and optional AudioWorklet path. Browser user-gesture/autoplay restrictions and no-pthread builds change startup/job processing.
Custom	Application-supplied backend callbacks/vtable enter the common context/device dispatch; there is no separate magic hardware driver implementation to describe. Correct lifecycle, enumeration and data transfer are the application's responsibility.
Null; 20934-21707	Software clock/buffer/callback device without physical playback. Useful for testing or fallback; a successfully initialized null endpoint does not prove audible output.

Backend structs and shims include ABI-compatible native declarations/function pointers, library opening/closing, native enumeration/configuration, format/rate conversion, device init/uninit/start/stop, processing or callback entry, and backend-specific error translation. The common dispatch at lines 42451-44574 wires these into one public API. Unsupported branches return result codes rather than selecting an unrelated graphics API. Runtime linking is platform-specific; MA_NO_RUNTIME_LINKING changes linker obligations. On Linux the supplied manual calls for `-ldl`

- lpthread - lm, possibly - latomic for 32-bit ARM; the Windows-specific project link configuration should not be described as already implementing every portability requirement.

6.7.5 PCM Utilities and DSP Filter Families

Utilities copy/silence PCM, compute offsets, clip values, apply gains, mix streams and interleave/deinterleave channel storage. They distinguish frames from samples and signed from unsigned silence. Format converters cover u8/s16/s24/s32/f32 with scalar/SIMD paths, scaling, clipping and requested dithering behavior. A conversion's direction determines destination capacity and precision loss; changing byte representation is not resampling.

The biquad family stores coefficients plus per-channel delay/register state. Its mathematical prototype is $y[n] = (b_0 \cdot x[n] + b_1 \cdot x[n-1] + b_2 \cdot x[n-2] - a_1 \cdot y[n-1] - a_2 \cdot y[n-2]) / a_0$; implementations normalize coefficients and choose supported processing formats. `reinit` can replace configuration while retaining history, unlike tearing down/reinitializing the object. Separate channels need separate state.

DSP family	Declaration lines; definition entry	Explanation
<code>ma_biquad_*</code>	4593-4642; config at 47166	General second-order filter with caller coefficients, heap/preallocated state and in-place/buffer processing.
<code>ma_lpf1_*</code> , <code>ma_lpf2_*</code> , <code>ma_lpf_*</code>	4645-4731; 47487 onward	First/second/higher-order low-pass filters, cutoff and sample rate; cascaded sections for higher-order Butterworth behavior. Removes high frequencies and supplies resampler antialias smoothing.
<code>ma_hpf1_*</code> , <code>ma_hpf2_*</code> , <code>ma_hpf_*</code>	4734-4817; 48376 onward	High-pass counterparts, retaining high frequencies and rejecting low/DC components.
<code>ma_bpf2_*</code> , <code>ma_bpf_*</code>	4820-4879; 49200 onward	Band-pass sections controlled by center/cutoff frequency and Q; higher-order family cascades sections.
<code>ma_notch2_*</code>	4882-4909; 49686	Rejects a narrow band while retaining frequencies outside it.
<code>ma_peak2_*</code>	4912-4940; 49859	Peaking equalization with frequency/Q/gain in dB; boosts or cuts around a center.
<code>ma_loshelf2_*</code>	4943-4971; 50034	Low-frequency shelf, gain in dB and slope.
<code>ma_hishelf2_*</code>	4974-5003; 50207	High-frequency shelf counterpart.
<code>ma_delay_*</code>	5005-5039; 50379	Circular delayed sample history, delay frame count and decay/dry-wet controls; stateful echo, not decoder seeking.
<code>ma_gainer_*</code>	5040-5074; 50533	Per-channel gains and gradual transition over configured frames; smooths control changes.
<code>ma_panner_*</code>	5075-5109; 51078	Stereo balance/pan modes modify left/right gains; not full 3D geometry.
<code>ma_fader_*</code>	5110-5137; 51286	Time/frame-position-dependent fades and gain endpoints; seconds/milliseconds must be converted using sample rate.
<code>ma_spatializer_listener_*</code> , <code>ma_spatializer_*</code>	5138-5317; spatial config at 52051	Source/listener coordinate transform, distance/cone/directional attenuation, speaker-channel weighting, gain interpolation, and Doppler ratio.

Arrakis invokes gain, pitch and spatialization through the engine; it does not construct custom low/high-pass, shelf, notch, delay or noise objects. Their vendor code is still included by the unspecialized implementation configuration, though unused functions may be removed by link-time/dead-code optimization depending on the build.

6.7.6 Resampling, Channel Conversion, and Composite Conversion

`ma_linear_resampler_*` config begins at 53032. It maintains fractional source position, previous samples/filter state and an input/output rate relationship. Linear interpolation estimates between adjacent samples; low-pass filtering limits aliasing when rates change. Required-input/expected-output/latency helpers describe how much input is needed and how delay affects frames. Dynamic rate/ratio setters support pitch or device-rate changes without rebuilding every object.

`ma_resampler_*` dispatches either the built-in linear resampler or a custom resampler vtable, handling init/heap/process/reset/rate/count/latency operations. An external implementation must obey frame-count and lifetime contracts. Pitch is usually expressed by modifying the consumption ratio, which changes duration as well as frequency. The default engine pitch-resampling config uses f32 linear resampling with low-pass order 0 at lines 77464-77465; the vendor comment explicitly cites instability concerns for some pitch-filter cases. Do not infer a separate studio-quality time-stretch algorithm.

`ma_channel_converter_*` creates mono expansion, positional channel mapping, reordering and weighted mixing. It can use passthrough/copy paths when layouts match; adding or removing channels is a different operation from changing sample rate. Channel maps encode speaker positions, not merely ordinal indexes; surround-to-stereo and mono-to-multichannel policies affect spatial perception.

`ma_data_converter_*` composes format conversion, channel conversion and resampling, chooses efficient processing order/caches, calculates heap requirements, and updates consumed/produced frame counts. Unknown/default fields are resolved before processing. PCM format changes can lose precision; resampling adds state and latency; channel conversion may mix information. Single-shot `ma_convert_*` helpers build or drive these operations for one buffer, while long-running converters retain state across blocks.

6.7.7 Data Sources, Buffers, Rings, and Filesystem

`ma_data_source` is a common read/seek/format/cursor/length/loop interface implemented through a vtable and base state. It also supports source ranges, loop points and chaining. The base operations enforce ranges and looping around underlying readers. Not all sources implement seeking or reliable length; callers must inspect results rather than assuming a finite file-backed stream.

`ma_audio_buffer_ref` borrows a PCM array and maintains a cursor without owning the array. `ma_audio_buffer` can allocate/copy/own storage according to its config, and its uninit releases owned storage through matching callbacks. Paged buffers link PCM pages and let producers expose growing decoded data; page ownership and lifetime must not race readers. Data-source initialization entry is 59138, borrowed-buffer init 60044, owned-buffer init 60361, paged-buffer init 60733.

`ma_rb` is a byte ring; `ma_pcm_rb` adds format/channel/frame knowledge. Acquire/commit read and write APIs expose contiguous regions, handle wraparound, and advance cursors only on commit. Ring buffers are documented for **one consumer and one producer**, not arbitrary multi-reader/multi-writer access. Available read/write counts prevent overrunning unread data or reading unwritten data. Duplex helpers buffer between capture/playback clock domains; differing clocks can drift even when nominal sample rates match.

The VFS is a callback table for open/close/read/write/seek/tell/info. Default VFS uses OS/CRT filesystem operations with narrow/wide path variants and 64-bit-position support where available. Decoders and resource manager can use a custom archive/memory/package filesystem without hardcoding game paths. Arrakis does not supply one: it performs a standard `ifstream` pre-probe and then lets miniaudio's default VFS open the selected path. That double-open pattern allows a file to change/disappear between lookup and decode.

6.7.8 Decoders, Probing, Adapters, and Encoder

`ma_decoder_config` describes requested output format/channels/rate, maps/mixing/resampling, allocation callbacks and optional custom backend vtables. `ma_decoder` owns one selected backend plus conversion/cursor/cache state. Its callback, VFS-file, wide-file and memory initializers attempt supported backends, establish native audio properties and configure output conversion. Built-in and custom backend attempts can fail and be cleaned up before the next attempt; an encoding hint/extension can guide selection but existence or suffix is not authoritative content validation.

The WAV adapter, FLAC adapter, MP3 adapter and optional Vorbis adapter expose codec-specific states as `ma_data_sources`. Their vtables translate read/seek/get-format/get-cursor/get-length into codec functions and normalize result reporting. Definitions and declarations inside the implementation region are internal integration scaffolding, not evidence the wrapper calls those functions directly.

`ma_decoder_read_pcm_frames` reads native decoded PCM and converts it to configured output while honoring partial counts/EOF. Seek operations reset or reposition dependent state; cursor and length are in PCM frames, not encoded bytes. Memory decoders borrow encoded memory for their decoder lifetime unless the surrounding manager owns a loaded copy. File decoders must close file state during teardown. Whole-file decode helpers allocate output PCM that the caller must free with the corresponding allocator.

The optional `ma_stbvorbis` state stores a base data source, read/seek/tell callbacks and user pointer, allocation callbacks, f32 format, channels/rate, 64-bit reported cursor, the stb decoder pointer, push/pull mode and push-buffer/cache state. `MA_VORBIS_DATA_CHUNK_SIZE` is **4096 bytes**. Callback mode uses pushdata because stb_vorbis's pull API lacks generic read callbacks; file/memory modes can use stb's native pull APIs. Push mode accumulates encoded bytes, tracks consumed bytes and remaining planar samples, then copies/interleaves output. Buffer growth and stb's `int` length APIs impose checked size limits. Push seeking can rebuild/redecode from the beginning because flush-and-arbitrary-seek is not robust enough for this adapter. Push-mode length is reported as zero where reliable length cannot be obtained; native pull mode can query stream length. Adapter allocation callbacks do not automatically control stb_vorbis's own malloc/alloca use.

The encoder family supports the built-in WAV encoder, not MP3/Vorbis/FLAC encoding merely because those decoders exist. It initializes a write/seek callback or file target with an explicit encoding format and PCM format/rate/channels, writes PCM frames, and finalizes/patches container bookkeeping during uninit where supported. Sequential writers need total-size knowledge when seeking is unavailable. Arrakis does not record or encode audio.

6.7.9 Embedded WAV Implementation: All Subfamilies

The embedded copy is `dr_wav 0.14.5`, renamed with `ma_dr_wav_*` symbols to coexist with independent `dr_wav` users. Declarations occupy 61934-62368; implementation occupies 79908-84837. This is actual codec/container code included in `miniaudio`, not a reference to a separately installed DLL.

Implementation lines	Functional explanation
79908-81736	Version/platform/memory/endian helpers; RIFF/WAVE, Wave64 and RF64 container identification and chunk iteration; fmt/data discovery; padding/size accounting; format/subformat validation; metadata parsing/allocation and setup/read callbacks. Unknown chunks are skipped subject to lengths; truncated/invalid headers fail instead of becoming arbitrary PCM.
81737-82850	Public callback/memory/file initialization, sequential and seekable writer setup, metadata-aware writer variants, file wrapper callbacks, uninitialization/finalization, raw reads/writes, cursor/length/seek and common PCM transfer. Ownership flags distinguish library-opened files from borrowed callback data.
82851-83967	Native PCM reads and endian variants, format-specific s16 and f32 conversion helpers, A-law/mu-law expansion, Microsoft ADPCM and IMA/DVI ADPCM block decoding with predictor/history and cached decoded frames. Encoded bytes and PCM frames have different ratios for compressed WAV.

Implementation lines	Functional explanation
83968-84837	f32/s32 normalized read variants, remaining scalar conversion families, callback/file/memory whole-file decoding helpers, allocation/free/error handling and embedded implementation closure. Whole-file helpers allocate a buffer, repeatedly decode and return actual frame count; they do not leak ownership to the engine without an explicit handoff.

RIFF chunk sizes/padding, extensible format tags and channel/sample metadata are validated before data transfer. Supported WAV encodings include integer PCM, IEEE float and the listed telephony/ADPCM formats; .wav is a container, not a guarantee of uncompressed 16-bit PCM. Writes use supported uncompressed/container formats and header bookkeeping; having an ADPCM decoder does not imply the library writes every compressed WAV format. The game uses ordinary WAV reads for harvester, winch, safe and crew cues, without directly invoking metadata-aware authoring APIs.

6.7.10 Embedded FLAC Implementation: All Subfamilies

The embedded copy is `dr_flac 0.13.3`, namespaced as `ma_dr_flac_*`. It is a lossless decoder; no FLAC assets are selected by the wrapper's candidate lists.

Implementation lines	Functional explanation
84838-86342	Portability/SIMD/byte-order helpers, allocation callbacks, CRC tables/checks, buffered bitstream reads, signed/unsigned extraction and FLAC frame/header number decoding. Efficient cache/bit reads are bounded by available data and the decoded field width.
86343-87624	Rice-coded residual reading; reference/scalar/SSE4.1/NEON predictor paths; escape/unencoded residual partitions and residual seeking. Unary quotient plus Rice remainder reconstruct signed residuals, which are added to a prediction rather than treated as finished audio.
87625-88555	Constant/verbatim/fixed-order/LPC subframe decoding, warm-up samples, coefficients and shift, wasted-bit restoration, frame/subframe headers, channel decorrelation, frame validation, skipping and seek-forward helpers. Fixed predictors and LPC reconstruct original integer samples losslessly.
88556-90162	Metadata block parsing, STREAMINFO, seektable and optional metadata callbacks, decoder allocation/init preparation, native FLAC/Ogg mapping infrastructure and callback/memory/file setup groundwork. Container metadata is distinct from PCM.
90163-92265	Public open/close variants, file/memory adapters, seek strategies and output-read paths, conversion/interleaving to s32/s16/f32, optimized channel decorrelation and buffering. Seek tables accelerate location but absent tables do not necessarily make decoding impossible.
92266-92696	Public f32 frame-reading entry, remaining whole-file/memory convenience and cleanup/implementation closure. Allocated output and decoder-object ownership are documented separately.

For predictive subframes, conceptual reconstruction is `sample[n] = prediction(previous samples, coefficients) + residual[n]`, with shifts and bit widths dictated by the stream. Rice residual decoding maps a quotient/remainder code back to a signed integer. Stereo channel assignments include independent, left-side, right-side and mid-side; decoding reverses the selected decorrelation. CRC and validated bounds are part of handling malformed data but do not establish that every vendor branch was audited here. FLAC's Ogg-container facilities do not make it a Vorbis decoder.

6.7.11 Embedded MP3 Implementation: All Subfamilies

The embedded copy is `dr_mp3 0.7.3`, namespaced as `ma_dr_mp3_*`; low-level decode machinery derives from `minimp3`. This decoder is directly relevant to flight, boost and worm cues.

Implementation lines	Functional explanation
92697-94591	Portability/SIMD setup, callback/file I/O, seek/metadata helper groundwork and minimp3-derived core: MPEG frame synchronization/header interpretation, bit readers, Layer I/II scale-factor/dequantization processing, Layer III side information/scalefactors/Huffman decode, bit reservoir, stereo processing, requantization, short-block reorder, antialiasing, inverse transforms and polyphase synthesis. Tables encode codec constants; SIMD accelerates equivalent PCM reconstruction.
94592-94979	Higher-level next-frame decoding for callback and memory inputs, encoded-buffer refill/compaction, metadata/frame-start and stream setup, native format properties and decoded-frame cache management. A frame can need reservoir bytes retained from earlier compressed frames.
94980-95264	Public callback/file/memory init variants, metadata callbacks, teardown and common read preparation; ownership/allocation errors and first-frame initialization are handled here.
95265-95804	f32/s16 frame reads, PCM cursor/length, seeking and seek-point tables, frame/PCM count helpers, whole-file convenience decoding, free functions, and implementation closure. Seeking generally locates an encoded frame then decodes/discards as necessary to reach a requested PCM position.

Constants in declarations include maximum **1152 PCM frames per MP3 frame**, maximum **2304 samples** for two channels, maximum **511 reservoir bytes**, and **2304** free-format frame-size constant. These are codec-buffer limits, not the game's audio-callback frame size. Layer III uses a compressed bit reservoir, so a sound cannot always begin decoding correctly from an arbitrary encoded byte without reconstructing prior state.

MP3 reconstruction is lossy: compressed spectral values are entropy decoded, requantized/scaled, possibly jointly stereo-reconstructed, transformed and synthesized into PCM. `dr_mp3` provides format decoding, not device output. The generic miniaudio converter/engine subsequently adapts rate/channels and spatial gain. A `.mp3` filename is checked by decoding/probing; an invalid or truncated file causes failure or a short read, not an automatic successful sound.

6.7.12 Generators

`ma_waveform` synthesizes sine/square/triangle/sawtooth-style periodic waves using configured amplitude/frequency/rate and persistent phase/time. `ma_pulsewave` adds pulse width/duty behavior. `ma_noise` supplies white, pink and brownian noise with seed/random state and per-channel history/filtering where required. Their data-source interfaces permit reads without a finite file. Setters change parameters; seek/cursor semantics are different from reopening a recorded asset. White noise is uncorrelated random output; pink and brownian variants apply frequency/history shaping. These families are compiled under generation support but Arrakis creates none of them: rumble is an asset, not synthesized brown noise here.

6.7.13 Resource Manager: Registry, Caches, Streaming, and Lifetime

The resource manager lives at lines 69309-73426, with declarations at 10297-10614. Config initialization at 69969 defaults to unknown output format, zero channels/rate (defer resolution), one worker thread, and a bounded job queue. The engine's internal manager overrides decoded format to f32 and decoded rate to engine rate while leaving decoded channels at zero to preserve useful native-channel/spatialization behavior.

The registry hashes paths (default hash seed **42**) and stores reusable buffer nodes in a search-tree structure. Loading the same nonstreamed path can share encoded or decoded backing data with reference counting; sound instances still

have independent playback state/cursors. Registering preloaded encoded or decoded data avoids repeated file I/O, but caller-supplied registered data has its own lifetime contract.

Flag families are `STREAM = 0x1`, `DECODE = 0x2`, `ASYNC = 0x4`, `WAIT_INIT = 0x8`, `UNKNOWN_LENGTH = 0x10`, and `LOOPING = 0x20`. Streaming uses paged/double-buffered decoded data so worker threads can read/decode/refill without a full file-sized allocation. Nonstreaming without `DECODE` keeps compressed/encoded bytes in memory; `DECODE` trades larger memory for less mixer decoding work. `ASYNC` queues loading/decoding instead of completing all work on the caller. `WAIT_INIT` establishes initial usable format/data even when later completion is asynchronous.

Typed jobs load/free a shared buffer node, load/free an instance buffer, page decoded data, load/free/page a stream, or seek a stream. Order/execution counters, notification/fence state and result fields coordinate initialization, future pages and destruction. A stream cannot free its decoder/page memory while a queued job still accesses it. Free/uninit must wait or otherwise honor those outstanding jobs. Job-thread shutdown posts quit/cancellation and joins threads before freeing queue/manager storage.

The manager API exposes register/unregister, buffer/stream/source init/uninit/copy, read/seek/format/cursor/length/loop controls and explicit process-next-job for application-managed execution. Custom VFS and custom decoder backends plug in at this level. On single-threaded Emscripten the engine has special job-processing behavior in its callback; that is not the usual Windows resource-manager thread model.

6.7.14 Node Graph and Every Built-In Node Family

Declarations 10615-11107; implementation 73427-76536. A graph mixes audio by pulling frames from its endpoint. Nodes have input and output buses, compatible channel counts, process vtables, state, times, gains and caches. Each input bus can receive several output buses; those sources are mixed before the destination process. A splitter's multiple outputs can feed distinct downstream branches. Channels must match at attachments; graph connections are not arbitrary byte-stream joins.

The implementation traverses upstream attachments, caches/mixes blocks, handles nodes with different consumed/produced rates, accounts for node start/stop scheduling and per-bus volume, and produces silence where required. Output-bus attachment lists are doubly linked with synchronized iteration/refcounts so detachment can wait for readers before destruction. This is why `ma_sound_uninit` first detaches its node. Feedback cycles should not be treated as a supported general-purpose cyclic graph merely because nodes can be connected.

The default node-cache capacity is **480 frames per bus** and default premix stack size is **524288 bytes per channel**, as named near 73492-73496. These are implementation/configuration defaults; engine device processing size can influence graph batching. Node caches are not the application's graphics frame buffers.

Node family	Role
Graph endpoint/base node	Common state, scheduling, bus attachments, pulling/mixing and endpoint output.
Data-source node	Reads a <code>ma_data_source</code> , emits PCM, optionally loops; source must remain valid until detached/uninitialized.
Splitter node	Copies/routes an input to multiple output buses for parallel processing.
Biquad node	Wraps a general coefficient filter in graph processing.
Low-pass node	Wraps the low-pass DSP family.
High-pass node	Wraps high-pass DSP.
Band-pass node	Wraps band-pass DSP.
Notch node	Wraps narrow rejection filtering.
Peaking node	Wraps parametric/peaking equalization.
Low-shelf node	Wraps low-frequency shelf equalization.

Node family	Role
High-shelf node	Wraps high-frequency shelf equalization.
Delay node	Wraps stateful delayed feedback/output mixing.

Node flags express passthrough (0x1), continuous processing (0x2), allow-null input (0x4), differing processing rates (0x8) and silent output (0x10). They guide graph traversal/caching and callback contracts; they do not enable a physical device backend. Stateful effect nodes need their own DSP heap/register cleanup when removed.

6.7.15 Engine, Engine Nodes, Sounds, and Groups

Engine definitions 76537-79893 compose the graph and resource manager with device output. Config initialization at 77455 chooses one listener, default mono expansion, and linear resource/pitch resampling. Initialization at 77521 zeroes output storage, copies config/callbacks, optionally allocates and initializes an f32 playback device, resolves channel count/rate, initializes graph, validates listener count against `MA_ENGINE_MAX_LISTENERS = 4`, creates listeners, selects gain-smoothing time, creates/configures an internal resource manager when needed, and starts the engine last unless `noAutoStart` is set. Error labels unwind the portions reached; exact ownership/error behavior should be checked in source when changing this vendor version.

The internal device callback at 77472 reads `ma_engine*` from device user data and calls `ma_engine_read_pcm_frames`. That pulls the graph and writes the device's requested frame block independently of the render loop. The engine manages clipping itself and disables redundant device presilencing/clipping when it creates its device.

An engine node supplies gain, pitch resampling, spatialization, fader state, listener choice, mono expansion and caches around node processing. A `ma_sound` adds a data source, seek/end/loop state, processing cache and optional end callback. File initialization allocates a resource-manager source, initializes it, sets ownership, and creates/attaches the sound node. A group is essentially a sound-like engine node without a data-source voice, accepting other voices and applying shared controls. Group controls mirror sound controls for volume, pitch, pan, spatial configuration and timed fades.

APIs cover explicit device/manager/log retrieval, engine read/start/stop/time/volume, listener position/direction/up/velocity/cone/enabled state, sound/group init/uninit/control, data-source access, scheduled starts/stops in frames/milliseconds, fades, looping, seeking, cursor/length, and play-once inlined sounds. `ma_engine_play_sound` internally manages a temporary/inlined sound list and reclamation; Arrakis instead owns eight explicit sounds. Seeking is deferred via an atomic target, and end-state handling can rewind on restart. Setting master gain to zero does not remove nodes or stop consumption.

Owned device teardown precedes graph teardown to stop callback access. Sound teardown precedes backing-source release to stop graph readers. External device/resource-manager ownership differs from internal ownership: a caller must retain external objects while engine/sounds use them and release them in an order consistent with the docs. The wrapper's explicit stable global storage and sound-before-engine shutdown follow the core required order, without implementing a general lifetime-safe reusable audio-owner class.

6.8 GLAD: Generated Loader, Every Version, and Platform Branch

6.8.1 What This File Implements and What It Does Not

`third_party/glad/glad.c` is generated with API `gl=4.6`, core profile, loader enabled, and **an empty selected-extension list**. Its provenance records `Local files: False`, `Omit khrrplatform: False`, and `Reproducible: False`. It includes `<glad/glad.h>` supplied by the project's configured external include directory. This `.c` file is not the OpenGL renderer implementation and not the driver: its function pointers lead to driver-provided implementations in the current context.

Arrakis explicitly uses `gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)` after GLFW creates and makes an OpenGL context current. That uses GLFW's resolver, not GLAD's optional direct-system-library

`gladLoadGL()` entry. Both paths are nevertheless present and documented because the request includes the entire supplied file.

6.8.2 Complete GLAD Source Coverage

Original lines	Contents
1-26	Generator provenance/options and C/header includes
27-157	Procedure resolver declaration; Windows/UWP or dlopen platform loader; resolver fallback; <code>gladLoadGL</code>
158-171	Version struct/state, max loaded version, extension storage
172-205	Extension enumeration/storage
206-216	Extension-storage cleanup
217-257	Exact extension-name membership helper
258-276	Nineteen <code>GLAD_GL_VERSION_*</code> flags, all initially zero
277-975	Alphabetically arranged typed <code>glad_gl*</code> function-pointer globals, initially null
976-1735	Nineteen version-specific loading functions, individually mapped below
1736-1742	Selected-extension discovery wrapper; this generated selection is empty
1743-1803	Current-context version parsing and all version-flag tests
1804-1832	Public caller-resolver entry, core loads, extension handling and boolean success
1833	Original trailing blank line; absent after relocation

6.8.3 System Library Opening, Resolution, and Closing

Windows/Cygwin uses an `HMODULE libGL` and a typed pointer to `wglGetProcAddress`. The preprocessor manages `APIENTRY` and includes `<windows.h>`. MSVC SDK detection conditionally includes `winapifamily.h`; a non-desktop app partition defines `IS_UWP`. On desktop, `open_gl` calls `LoadLibraryW(L"opengl32.dll")`, obtains `wglGetProcAddress` using `GetProcAddress`, and succeeds only if both the library and resolver are available. UWP's disabled desktop opening path returns zero. `close_gl` calls `FreeLibrary` if non-null and clears `libGL`.

Non-Windows uses `<dlfcn.h>`, a `void* libGL`, and on non-Apple/non-Haiku platforms a `glXGetProcAddressARB` pointer. Apple attempts these paths in exact order:

1. `../Frameworks/OpenGL.framework/OpenGL`
2. `/Library/Frameworks/OpenGL.framework/OpenGL`
3. `/System/Library/Frameworks/OpenGL.framework/OpenGL`
4. `/System/Library/Frameworks/OpenGL.framework/Versions/Current/OpenGL`

Other non-Windows platforms attempt `libGL.so.1`, then `libGL.so`. Each is opened with `RTLD_NOW` | `RTLD_GLOBAL`. Apple/Haiku return success as soon as a library opens; other platforms require `dlsym(libGL, "glXGetProcAddressARB")` to succeed. `close_gl` uses `dlopen` and clears the pointer. The loop returns immediately after the first opened library's resolver check; an opened library without the required resolver is not automatically followed by testing the next name. The failure path's exact cleanup is worth reading before embedding this generated direct-library helper in another host.

`get_proc(const char* namez)` initializes result null, rejects null `libGL`, tries the platform context resolver where compiled, then falls back to Windows `GetProcAddress` or POSIX `dlsym` only when the first result is null. It returns the raw procedure address. The supplied Windows helper does not explicitly reject the unusual non-null sentinel values some `wglGetProcAddress` implementations can return. Arrakis relies on GLFW's resolver instead.

`gladLoadGL()` calls `open_gl`; on success calls `gladLoadGLLoader(&get_proc)`, then `close_gl`; returns the loader status. It does not create an OpenGL context. Resolution remains context dependent, and closing the helper's temporary module handle should not be interpreted as destroying GLFW's context/driver lifetime. The application must keep the context alive for graphics calls and must not assume pointer validity after context/provider changes.

6.8.4 Function Pointer Storage and Assignment Semantics

Every global declaration has the form `PFNGL...PROC glad_glName = NULL;`. The typedef from `glad.h` encodes the driver function's return type, parameters and platform calling convention. The header normally aliases familiar `glName` spellings to these globals. The declaration does not implement that OpenGL operation; it reserves a call target.

Every generated loading assignment has the form `glad_glName = (PFNGL...PROC) load("glName")`. Its one parameter is `GLADloadproc load`, the caller's resolver taking a symbol name. Its only branch is the version guard `if (!GLAD_GL_VERSION_M_N) return;`, followed by a fixed sequence of lookups. There is no per-assignment validation, wrapper around arguments, shader compilation, mathematical rendering work, GPU object allocation, or error checking here. Two appearances of a symbol in successive version blocks mean it can be assigned again, not that two GPU implementations are compiled.

The globals are process-global and this loader is not a multi-context dispatch table with automatic context switching. Reloading for a new context updates shared targets. Version success alone is not a proof that every pointer needed by the game is non-null. Unloaded/unsupported pointers must not be called.

6.8.5 Every Generated Core-Version Loading Family

These ranges are exact boundaries of the generated functions, not a generic list of OpenGL release headlines. Every assignment in a range resolves a driver function; the described operation is performed **later by the driver when called**, not by GLAD.

Loader and lines	Loaded functional families and meaning
<code>load_GL_VERSION_1_0</code> , 976-1026	Rasterization state (culling/front face/hints/line/point/polygon/scissor), texture parameters and 1D/2D images, draw/read buffers, clear values/masks, enable/disable, finish/flush, blending/logic/stencil/depth, pixel packing/readback, scalar/vector state queries, errors/string queries, texture inspection, depth range and viewport. This generated core subset does not supply classic immediate-mode <code>glBegin</code> rendering.
<code>load_GL_VERSION_1_1</code> , 1027-1042	Array/indexed draws, polygon offset, texture copy/subimage updates, bind/generate/delete/is-texture. These provide object-oriented texture lifetime and bulk geometry submission.
<code>load_GL_VERSION_1_2</code> , 1043-1049	Draw-range indexed draw and 3D texture image/subimage/copy operations.
<code>load_GL_VERSION_1_3</code> , 1050-1061	Active texture unit, sample coverage, compressed texture image/subimage/get for 1D/2D/3D. Texture-unit selection is separate from binding/creating texture objects.
<code>load_GL_VERSION_1_4</code> , 1062-1073	Separate blend factors, multidraw arrays/elements, float/integer point parameters, blend color/equation. Multidraw combines submission descriptions; GLAD merely resolves the entry.
<code>load_GL_VERSION_1_5</code> , 1074-1095	Query object create/delete/begin/end/results and buffer object bind/create/delete/data/subdata/readback/map/unmap/parameter/pointer queries. Buffer data calls later allocate/update GPU-visible storage.

Loader and lines	Loaded functional families and meaning
load_GL_VERSION_2_0, 1096-1191	Separate blend equations, multiple render targets, per-face stencil; shader/program create/source/compile/attach/link/validate/use/delete/reflection/logs; float/integer scalar/vector and square-matrix uniforms; vertex attribute values/pointers/enabling/query. Shader failure handling belongs to application calls and logs, not loader initialization.
load_GL_VERSION_2_1, 1192-1200	Six rectangular float uniform matrix forms: 2x3, 3x2, 2x4, 4x2, 3x4, 4x3.
load_GL_VERSION_3_0, 1201-1287	Indexed state/masks, transform feedback and buffer-base/range binding, color clamp, conditional rendering, integer vertex attributes, unsigned uniforms, fragment-output binding, integer texture parameters, typed clears, indexed strings, renderbuffers/framebuffers and attachments/status/blit/multisample/layer, range mapping/flush and vertex array objects. This contains Arrakis's framebuffer/VAO-era infrastructure.
load_GL_VERSION_3_1, 1288-1305	Instanced draws, buffer textures, primitive restart, buffer copies, uniform index/block reflection/binding and repeated base/range/indexed integer lookups. Uniform blocks share packed state across draw programs.
load_GL_VERSION_3_2, 1306-1327	Base-vertex indexed draw variants, provoking vertex, sync object fence/waits/delete/query, 64-bit state/buffer queries, layered framebuffer texture, multisample textures and sample masks. Sync functions govern CPU/GPU completion, not audio thread synchronization.
load_GL_VERSION_3_3, 1328-1388	Indexed fragment outputs, sampler-object lifetime/parameters, timestamp/query-counter and 64-bit results, vertex attribute divisors for instancing, packed vertex/texture/normal/color attribute variants. The supplied file contains some packed legacy-named targets despite its core provenance; presence is not a guarantee they are valid in every core context.
load_GL_VERSION_4_0, 1389-1437	Sample shading, per-draw-buffer blending, indirect draws, double scalar/vector/matrix uniforms and queries, shader subroutine reflection/selection, tessellation patch parameters, transform feedback objects/stream draws, indexed queries.
load_GL_VERSION_4_1, 1438-1529	Shader/compiler binary/precision controls, float depth range/clear, program binaries/parameters, separate program pipelines/stage selection/validation/logs, all direct program-uniform forms (float/int/uint/double and matrices), double vertex attributes/pointers, viewport/scissor/depth-range arrays/indexed forms and indexed float/double queries.
load_GL_VERSION_4_2, 1530-1544	Base-instance instanced draws, internal-format queries, atomic counter buffer reflection, image texture binding, memory barriers, immutable texture storage, instanced transform feedback/stream drawing. Barriers control visibility of GPU memory operations.
load_GL_VERSION_4_3, 1545-1591	Clear buffer data/subdata, compute dispatch/direct-indirect, image copies, framebuffer parameter configuration, 64-bit format queries, invalidate texture/buffer/framebuffer storage, multidraw indirect, program-interface/resource reflection and shader storage binding, ranged buffer textures, multisample storage, texture views, separate vertex-binding/format APIs and debug message/group/object-label APIs.
load_GL_VERSION_4_4, 1592-1603	Immutable/persistent-capable buffer storage, texture clears, multibind buffer base/range, textures, samplers, images and vertex buffers. Capability flags supplied by later callers decide persistence/coherency, not loader assignment.

Loader and lines	Loaded functional families and meaning
load_GL_VERSION_4_5, 1604-1728	Clip control; direct-state-access transform feedback/buffer/framebuffer/renderbuffer/texture/vertex-array creation and named operations; texture unit/image/storage/subimage/mipmap/readback APIs; named object/map/query APIs; sampler/pipeline/query creation; query-buffer results; regional barriers; robustness reset/bounded readback/uniform calls, several legacy-named bounded calls, texture barrier. Direct-state access operates on explicitly named objects rather than implicit bind state.
load_GL_VERSION_4_6, 1729-1735	Exactly four resolutions: glSpecializeShader, glMultiDrawArraysIndirectCount, glMultiDrawElementsIndirectCount, glPolygonOffsetClamp. Shader specialization supplies SPIR-V entry/specialization data to the driver; indirect-count draws read draw counts; polygon offset clamp adds a bound.

GLAD includes versions through 4.6 so the application can resolve newer operations where available. That does not mean Arrakis requires or requests a 4.6 context, or that it uses compute, tessellation, SPIR-V, persistent mapping and every other loaded family. Shader language versions, requested GLFW context and actual driver version are separate facts.

6.8.6 Extension Storage, Matching, and Empty Selection

GLVersion starts {0,0}. max_loaded_major/minor, exts, num_exts_i, and exts_i hold discovery state. _GLAD_IS_SOME_NEW_VERSION is compiled when suitable GL/ES 3.0 declarations exist.

get_exts takes the legacy path for loaded major below 3: glGetString(GL_EXTENSIONS) supplies one driver-owned space-separated string. Otherwise it obtains GL_NUM_EXTENSIONS, allocates a char** array, queries each glGetStringi(GL_EXTENSIONS, index), allocates len+1 bytes and copies a null-terminated name. Those copied strings are GLAD-owned; the legacy string is not. Array allocation failure returns zero. Individual string allocation failures leave a null element. The source also returns failure when a modern context reports zero extensions and exts_i remains null; this is a generated-loader edge case, not proof the GL context is absent.

free_exts frees every copied element (null is safe), frees the array, and clears exts_i. has_ext(ext) uses strstr plus boundaries in legacy mode: a match must begin at the string start or after a space and end at space or terminator. This avoids accepting GL_F00 inside GL_F00BAR. Modern mode iterates copied names with exact strcmp, skipping null entries. Null requested/legacy strings reject membership.

find_extensionsGL calls get_exts, returns zero on failure, references has_ext only to silence unused-function concerns, frees extensions, then returns one. **No selected extension flags and no extension loader blocks are generated** because the selected extension list was empty. The code enumerates extensions but does not expose a complete extension registry to Arrakis. There is therefore no missing list of hundreds of extension-loading functions to explain in this particular file; they are genuinely not present.

6.8.7 Version Parsing and Public Loader Success

find_coreGL obtains glGetString(GL_VERSION); null returns immediately. It strips the first matching prefix among "OpenGL ES-CM ", "OpenGL ES-CL ", and "OpenGL ES ", then scans %d.%d using sscanf_s on MSVC or sscanf otherwise. Major/minor are stored in GLVersion and maximum-loaded state. The source does not check the scan's return count before using locals; valid driver version strings are assumed.

For every supported version M.N, the exact flag test is (major == M && minor >= N) || major > M. Thus a 3.3 context sets flags 1.0 through 3.3 true and 4.x false. A reported version above 4.6 is capped in **max-loaded discovery state** to 4.6, while GLVersion records the reported version. That cap reflects generator coverage, not a request to downgrade a context.

gladLoadGLLoader(load) resets GLVersion, resolves glGetString first, fails if that pointer is null or its GL_VERSION query is null, calls version discovery, invokes all nineteen loaders in ascending version order (each

self-guards), invokes extension discovery, and returns `GLVersion.major != 0 || GLVersion.minor != 0`. It does not validate the resolver pointer before its first call, confirm every required graphics symbol, compile shaders, or check framebuffer completeness. On a failed/repeated load, not every old global target is proactively cleared; callers should not keep using stale state just because some pointers remain non-null.

The game checks the returned boolean and terminates graphics setup on failure. A current context must exist before loading. A successful 4.6-capable loader build running against a 3.3 context is normal: unsupported later blocks simply do nothing. The renderer must limit calls to the supported/requested API subset.

6.9 Historical Vorbis Study: Removed, Never Integrated

6.9.1 Complete Source Coverage Map

These are historical original `stb_vorbis.c` line numbers recorded from inspection before cleanup. The unused 5,584-line file has been removed; it is not remaining project code, not compiled, and not part of the current-source appendix. This optional decoder study explains the previously supplied file and the Ogg integration requirements. Restoring it would be a future codec feature, not a missing requirement of the current WAV/MP3 game.

Lines	Contents and purpose
1-68	Decoder version 1.22, authors/history, explicit codec/seek limitations
69-171	Header guard/linkage, allocator/info/comment structs, shared handle/query/close APIs and thread-safety rules
172-246	Pushdata declarations and detailed consumed-byte/output/flush contracts
247-371	Pull/file/memory/seek/length/frame/sample/whole-file declarations, buffer sizing and channel coercion
372-419	Error enum, header closure, implementation boundary
420-662	Compile configuration, defaults, includes, endian/inlining/alloca/CRT portability and implementation setup
663-937	Internal codebook/floor/residue/mapping/mode/page/decoder-state types, packet/sample/buffer state, error/allocation macro groundwork
938-1232	Block arrays and permanent/temporary allocator helpers, CRC32, bit reversal/integer log/float unpacking, canonical/sorted/accelerated Huffman construction
1233-1336	Header validation, vector-quantization lookup size, twiddle/window/bit-reverse tables, blocksize setup, floor-neighbor/sorting helpers
1337-1633	File/memory byte access/offsets, Ogg capture/page parsing, packet/lacing/segment handling and bit accumulator
1634-1934	Huffman scalar fast/slow decoding and vector/codebook expansion into output/residue arrays
1935-2082	Floor prediction and inverse-dB lookup table, line rasterization into spectral gains
2083-2407	Residue types and partition/codebook decoding, spectral vectors, reference/conditional transform groundwork
2408-3058	Optimized inverse-MDCT stages, butterflies, rotations, permutations and transform reconstruction

Lines	Contents and purpose
3059-3518	Window lookup, floor application, packet-mode/short-long block decoding, channel coupling inversion, residue/floor combination, inverse transforms, overlap/add and initial-frame priming
3519-4207	Push whole-packet availability and complete decoder-start/setup parsing: identification/comments/codebooks/time/floor/residue/mapping/modes, validation/allocation
4208-4340	Deep state cleanup, close/init/allocate and info/comment/error/sample-offset queries
4341-4557	Push flush, CRC page resynchronization, push-frame decoding/open and file-offset query
4558-5027	Pull seeking: Ogg page search/probes, coarse/binary-guided location, frame/sample-exact refinement, seek start and length queries
5028-5120	Pull float-frame decode and file-section/file/filename/memory opening
5121-5425	Channel assignment/conversion tables, float-to-short clipping/mixing, frame/sample short APIs and whole-file filename/memory decode
5426-5479	Caller-buffer float sample/interleaved APIs, buffered remainder handling, pull implementation closure
5480-5543	Detailed historical changes and implementation guard closure
5544-5584	MIT/public-domain alternative license text

6.9.2 Public Structures, Modes, and Ownership

`stb_vorbis*` is an opaque decoder handle. Unlike miniaudio's transparent outer engine structs, application code does not allocate its internal decoder struct by ordinary public field initialization. `stb_vorbis_alloc` contains `char* alloc_buffer` and `int alloc_buffer_length_in_bytes`; a caller-provided block must remain alive until close and be large enough for setup and temporary decode memory. Without it, startup uses heap allocation and frame temporaries commonly use stack `alloca`.

`stb_vorbis_info` reports sample rate, channel count, permanent setup memory, setup temporary memory, frame temporary memory and maximum frame size. `stb_vorbis_comment` reports a vendor string, comment-list length and borrowed comment pointers. `get_info` copies scalar information; comments and frame outputs point into decoder-owned storage. `get_error` obtains and clears the last stored error. `close` releases owned internal allocations and, when configured, the opened file. An individual handle is not safe for simultaneous decode calls on multiple threads; separate handles can operate independently.

Push mode accepts successive encoded-byte windows. `open_pushdata` parses complete initial headers or returns null with an error such as `VORBIS_need_more_data`; on success it reports bytes consumed.

`decode_frame_pushdata` reports consumed **encoded bytes** separately from output **samples per channel**, and returns planar `float**` output through a `float***` output parameter. Zero consumed/zero output means more bytes are needed; consumed bytes with zero samples can mean resynchronization or initial-frame discard; consumed bytes with samples means a decoded frame. The first audio frame after open is discarded as part of Vorbis overlap priming. Internal output is transient and overwritten by later decoding; do not free or modify it.

`flush_pushdata` says the next encoded block is noncontiguous and resets the scanner/overlap continuity so the decoder searches for a valid page. The caller owns buffering and seeks its own source; flush alone does not select an exact PCM frame. Pull mode instead owns traversal of a complete memory buffer or a `FILE*` section. Open-file parameters specify close-on-free behavior and optional section length; external repositioning of an active owned stream confuses the decoder.

Frame APIs return a codec-frame-sized planar/interleaved output; sample APIs fill caller buffers across frame boundaries and cache unused samples. Interleaved capacity arguments are counts of scalar floats/shorts, while return values are samples **per channel**. Whole-file short decoders allocate a new interleaved buffer and return sample count or failure; the caller frees that buffer. Float sample APIs and integer coercion APIs do not all apply identical channel-mixing rules.

6.9.3 Container, Entropy Decode, Floor, Residue, and Mapping

Ogg is the container. Page parsing recognizes the capture pattern, version/flags, granule position, stream serial/sequence, checksum and lacing/segment table. A lacing byte of 255 continues a packet; packet boundaries can span pages. Byte/bit readers distinguish encoded EOF from an incomplete push buffer. CRC32 tables/updates validate candidate pages during push resynchronization, where several apparent headers may overlap in corrupted/unrelated data.

Vorbis setup packets define codebooks, floor, residue, mapping and modes. Codebooks contain symbol code lengths, Huffman decode tables and optional vector-quantization lookup values. Initialization computes canonical codes with Vorbis bit ordering, a short-code accelerated lookup, and sorted slow-path tables. Scalar decode consumes a codeword to produce a symbol; vector decode converts a symbol/index into values added or interleaved into spectral residue. Fast lookup is an optimization of entropy decoding, not an alternate audio codec.

The supported floor-1 algorithm decodes a sparse spectral envelope. Neighbor prediction reconstructs control-point heights; integer line drawing and an inverse-dB table build frequency-bin gains. Residue decodes the remaining spectral detail with partition classes/codebooks and type-specific arrangement. Mapping selects submaps and channel coupling; reverse coupling reconstructs channel spectral coefficients. Applying the floor to residue yields the spectrum supplied to the inverse transform.

Header setup validates dimensions/counts/types before allocating permanent tables and working buffers. It rejects unsupported floor 0 and invalid mappings/types rather than treating them as a supported generalized format. Comments/vendor strings are metadata, not synthesis data. The code has many bounds/overflow/truncation checks, but reading them for this chapter is not an independent claim of vulnerability freedom.

6.9.4 Inverse MDCT, Windowing, and Overlap

Compressed Vorbis packets reconstruct frequency-domain coefficients. `inverse_mdct` converts a block's spectrum into time-domain audio using precomputed trigonometric twiddles, staged butterfly/rotation loops, bit-reversed ordering and final rearrangement. The reference/conditional transform paths and optimized helper loops compute the same codec transform; they are not unrelated game-specific math.

Short and long blocks use different window regions, selected by packet mode and previous/next-window flags. The Vorbis window shape is built using the codec's nested-sine taper, conceptually $\sin((\pi/2) * \sin(\pi * (i+0.5)/n)^2)$ over the rising half. Windowed adjacent blocks overlap/add to avoid discontinuities while permitting long-block compression and short-block transient response. `vorbis_finish_frame` combines retained previous-window samples with the current block, establishes valid output bounds, updates sample location and retains the next overlap. The first packet primes history; the final granule position can trim valid final output.

The decoder therefore requires retained state across frames. Reopening or seeking may need preroll/neighbor-frame decoding; simply inverse-transforming a random packet does not produce sample-exact continuous output. The declared absolute maximum frame output is 4096 samples per channel under the Vorbis I block-size constraints, but callers should use reported info/capacities and the actual returned count.

6.9.5 Push Resynchronization, Pull Seeking, and Conversion

Push resynchronization scans for candidate Ogg pages, incrementally checks CRC and tracks a limited number of overlapping candidates. Once it finds a valid stream/page/frame boundary it restores continuity and sample-position knowledge where possible. Consumed bytes can be less than the supplied block so a capture pattern split across calls is not lost. More input may be required even when some garbage was consumed.

Pull seeking searches/probes pages using byte position and granule/sample information to narrow an interval, then decodes/refines around the target. `seek_frame` permits a returned frame that contains the target; seek positions buffered sample output so sample APIs begin exactly at the requested sample; `seek_start` resets to the beginning. Length queries inspect stream/page information and return sample or seconds forms. The source documents 32-bit sample-position limits despite Ogg supporting 64-bit granule positions.

Integer conversion scales/clamps float values to signed short output and implements simple channel coercion. Requested mono can sum all source channels; requested stereo combines left/center and right/center assignments; requesting more than present fills extra channels with zero; other cases select available channels. The vendor explicitly warns that this is not high-quality general surround mixing. Buffer-based float readers manage channel copying and cached frame remainders separately, and should not be assumed to follow all short-output coercion rules.

6.9.6 Compile Switches, Exact Defaults, and Explicit Limits

Switch or default	Effect
<code>STB_VORBIS_HEADER_ONLY</code>	Emits declarations/header section only; used when implementations are compiled separately.
<code>STB_VORBIS_NO_PUSHDATA_API</code>	Removes push-data APIs and supporting scanner behavior.
<code>STB_VORBIS_NO_PULLDATA_API</code>	Removes pull APIs and implies no stdio/integer conversion.
<code>STB_VORBIS_NO_STDIO</code>	Removes file APIs; memory/push paths can remain.
<code>STB_VORBIS_NO_INTEGER_CONVERSION</code>	Removes short conversion/whole-file short APIs while float decode can remain.
<code>STB_VORBIS_NO_CRT</code>	Changes allocator/CRT assumptions and implies no stdio; caller-provided memory becomes essential where malloc is unavailable.
<code>STB_VORBIS_MAX_CHANNELS = 16</code>	Default compile-time decoder channel cap; container field's theoretical 255 limit does not override this build default.
<code>STB_VORBIS_PUSHDATA_CRC_COUNT = 4</code>	Four overlapping page candidates during push resynchronization.
<code>STB_VORBIS_FAST_HUFFMAN_LENGTH = 10</code>	Fast lookup uses a 10-bit prefix, nominally 1024 table positions; larger permitted values trade memory/cache behavior for fewer slow decodes.
<code>STB_VORBIS_FAST_HUFFMAN_INT</code> / default short table	Changes accelerated table entry width; default chooses <code>STB_VORBIS_FAST_HUFFMAN_SHORT</code> .
<code>STB_VORBIS_NO_HUFFMAN_BINARY_SEARCH</code>	Uses smaller/slower fallback lookup choices instead of the usual sorted-code search where applicable.
<code>STB_VORBIS_DIVIDES_IN_RESIDUE</code> , <code>STB_VORBIS_DIVIDES_IN_CODEBOOK</code>	Trade precomputed storage for runtime divide/index reconstruction.
<code>STB_VORBIS_DIVIDE_TABLE</code>	Optional floor integer-division lookup tradeoff; documented as disabled by default for small benefit.
<code>STB_VORBIS_NO_INLINE_DECODE</code>	Disables scalar fast-decoder inlining; useful for size/debugging tradeoffs.
<code>STB_VORBIS_NO_DEFER_FLOOR</code>	Synthesizes floor earlier rather than deferring, changing working-memory/processing behavior.
<code>STB_VORBIS_NO_FAST_SCALED_FLOAT</code> , <code>STB_VORBIS_BIG_ENDIAN</code>	Select safe/fast float-to-int conversion and endian handling.
<code>STB_VORBIS_CODEBOOK_SHORTS</code>	Explicitly rejected with <code>#error</code> because prior behavior could produce incorrect results.

The top-of-file limitations are literal: no floor 0 (older pre-2004 streams), ignored lossless sample truncation at the beginning, no concatenated multiple Vorbis streams, and 32-bit sample positions limiting high-rate long seeks (approximately six hours at 192 kHz). It is a Vorbis decoder, not an encoder, Opus decoder, playback backend, resampler, game mixer or replacement for miniaudio's engine. None of these compile switches are currently selected by the game because the file is not part of its compiled path.

6.10 API Reference and Annotated-Appendix Strategy

6.10.1 Index All Code Without Replacing Explanation with Symbol Lists

The prose above explains behavior by family; the exact source appendix supplies the full declarations, constants, generated assignments, tables and branches. A maintainable API reference should index a symbol by **source path, original line, family, declaration, definition, conditional guards, ownership, callback/thread restrictions, parameters and result semantics**. A bare alphabetical dump is helpful for lookup but cannot explain interactions such as a sound holding a resource-manager source or a graph detach waiting for callback readers.

For miniaudio, search MA_API declarations in the public region and matching definitions in the implementation. Many symbols occur twice; retaining both locations avoids mistaking a prototype for an implementation. Also index internal static helpers by family, structs/enums/macros and embedded `ma_dr_*` functions. Preprocessor guards should be recorded because source inclusion, compile inclusion and runtime selection are three different things. For GLAD, pair each `glad_gl*` global declaration with every `load("glName")` assignment and its enclosing core-version guard; the actual OpenGL signature is in the external `glad/glad.h`. For `stb_vorbis`, public declarations and matching implementation definitions plus static decode helpers need separate entries.

6.10.2 Family-Level API Lookup Index

Symbol prefix / group	Public/source entry region	Why consult it
ma_version*, ma_result_description, allocation helpers	4508; 6112-6182; definitions 12298, 58926 onward	Version/diagnostics, allocator/free matching and PCM byte sizing
ma_log_*	4516-4590; definitions around 13572	Callback logging and registration lifetime
ma_biquad_*, ma_lpf*, ma_hpf*, ma_bpf*, ma_notch*, ma_peak*, ma_loshelf*, ma_hishelf*	4591-5003; 47164-50374	Filter coefficients/configs, per-channel state, reinit/processing/preallocation
ma_delay_*, ma_gainer_*, ma_panner_*, ma_fader_*, ma_spatializer*	5004-5317; 50375-53026	Time/gain/pan/spatial behavior and channel-state memory
ma_linear_resampler_*, ma_resampler_*	5331-5523; 53027-54276	Ratio/rate, custom vtables, latency and consumed/produced frame counts
ma_channel_converter_*, ma_data_converter_*, ma_convert_*, channel-map helpers	5524-5830; 54277-58096	Format/rate/channel composition and buffer capacities
ma_data_source_*, ma_audio_buffer*, ma_paged_audio_buffer*	5831-6025; 59138 onward	Borrowed/owned data, cursor/seek/loop/range and paging lifetime
ma_rb_*, ma_pcm_rb_*, duplex buffer	6026-6111; 58097-58922	Producer/consumer buffering and acquire/commit rules
ma_spinlock_*, ma_mutex_*, ma_event_*, ma_semaphore_*, ma_fence_*, notification APIs	6183-6335; 17544 onward	Cross-thread synchronization and callback-safety requirements
ma_slot_allocator_*, ma_job_*, ma_job_queue_*	6336-6593; 18579-19470	Bounded work storage, worker dispatch and shutdown
ma_context_*, ma_device_*, backend IDs/configs	6594-9751; 19471-44574	Device enumeration/init/start/stop/native negotiation and platform guards
PCM copy/silence/clip/volume/mix/interleave helpers	9752-9865; 44575-47163	Samples/frames, signed silence, clipping and conversions
ma_vfs_*, ma_default_vfs_*	9866-9944; 60904-61933	Custom/default file streams and file ownership
ma_decoder_*, ma_decoding_backend_*	9945-10110; 62840-67830	Codec probing/output conversion, custom decoding, read/seek/length
ma_encoder_*	10111-10169; 67831-68147	WAV write/finalization contract
ma_waveform_*, ma_pulsewave_*, ma_noise_*	10170-10296; 68148-69308	Infinite/generated data sources, phase/random state
ma_resource_manager_*	10297-10614; 69309-73426	Encoded/decoded cache, stream pages, worker jobs/refcounts
ma_node_*, ma_node_graph_*, source/splitter/filter/delay nodes	10615-11107; 73427-76536	Graph pull/attachment/state/bus contracts
ma_engine_*, ma_engine_node_*, ma_sound_*, ma_sound_group_*	11108-11535; 76537-79893	High-level playback, listener, pitch/gain/event/lifetime APIs used by Arrakis
ma_dr_wav_*	61934-62368; 79908-84837	WAV container metadata, PCM/ADPCM reads, conversion/write helpers

Symbol prefix / group	Public/source entry region	Why consult it
ma_dr_flac_*	62369-62662; 84838-92696	Lossless subframes/residuals/metadata/channel decorrelation/seek
ma_dr_mp3*	62663-62839; 92697-95804	MPEG decode/reservoir/synthesis and MP3 wrapper APIs
ma_stbvorbis_*	65330-66159, conditional	Optional callback/push/file/memory Vorbis bridge
gladLoadGL, gladLoadGLLoader, version/ext helpers	GLAD 27-257, 1736-1832	Loading success/context/platform/extension behavior
glad_gl*, load_GL_VERSION_*	GLAD 277-1735	Every resolved graphics symbol and version guard
Historical stb_vorbis_* and decode helpers	Removed source, former lines 69-5479	Historical codec study only; absent from current code/appendix/build

6.10.3 Auditable Limits and Practical Recommendations

The source listings and line maps establish coverage of the supplied files, including unused vendor branches. They do not establish audible playback on every machine, validity of every sound's binary contents, support for absent optional assets, correctness of all native API drivers, or a separate explanation of every generated graphics signature supplied by the external GLAD header. External GLAD/GLFW/GLM headers and actual driver/backend libraries are not reproduced by pretending they are contained in these four vendor files.

For verification, keep three results separate: engine initialization, per-file [AUDIO] Loaded messages, and actual audible/spatial behavior. Useful checks include running with no device; removing/renaming an optional primary asset in a controlled test; launching from a supported and unsupported working directory; comparing camera movement with fixed source position; muting across rescue/breach edges; restarting a mission during active one-shots; and observing boost gain/pitch thresholds. Those are recommended tests, not tests claimed to have been executed for a documentation-only chapter.

If Ogg fallbacks are retained as an advertised feature, integrate and test an actual Vorbis backend, or replace/remove those fallback expectations. The organized package currently retains the eight WAV/MP3 primaries and removed the unused stb source; it is not needed in the executable. Archive/library-license handling is separate from compilation necessity. Keep the current line-numbered appendix with its vendor version identifiers so future library updates do not silently invalidate this chapter's references. Historical references here are not a promise that removed files remain available.

7. The Documentation Generator, Line by Line

Purpose and Boundaries

`tools/generate_docs.py` produces the PDF requested for this project. It renders authored Markdown explanation chapters, generates page numbers and navigation, and appends complete numbered source/configuration listings. It also writes a machine-readable snapshot manifest. It does not infer the meaning of the game through artificial intelligence, execute the game, prove the prose correct, or automatically update descriptions after gameplay changes. The written chapters must be maintained when behavior changes; source hashes make a stale snapshot detectable.

The generator deliberately uses Python plus ReportLab instead of requiring a browser, LaTeX distribution, Node.js, or an online conversion service. The PDF is created locally. `tools/requirements.txt` contains two constraint lines: `reportlab>=4.2, <5` allows compatible 4.x renderer versions, and `pypdf>=5, <7` allows supported PDF-inspection versions. The lower bounds request available functionality; the upper bounds avoid silently adopting a major-version API change. Pillow is brought in as a transitive ReportLab dependency and is not a game dependency.

Lines 1-23: Module Setup and Imports

The module docstring describes the file's purpose and later supplies command-line help. `from __future__ import annotations` postpones annotation evaluation; types such as `List[str]` communicate expected values without affecting game execution. `argparse` defines command-line options and user-facing errors. `hashlib` computes SHA-256 fingerprints. `json` serializes the snapshot manifest. `re` matches the supported Markdown syntax. `textwrap` splits long text without truncating it. `datetime` and `timezone` produce a UTC generation date/time. `Path` joins and resolves paths independently of the current working directory. XML escaping protects Paragraph text from treating source characters such as `<` or `&` as formatting markup.

The ReportLab imports provide RGB/hex colors, centered alignment, the A4 page size, paragraph styles, millimeter conversion, a base multi-pass document, text frames, page templates, page breaks, formatted paragraphs, literal preformatted blocks, spacing, tables, table styling, and a table of contents. These classes divide responsibilities: Flowables are pieces of page content, a Frame defines where pieces can fit, and a PageTemplate defines recurring page furniture.

Lines 25-39: Exact Input Lists

`ROOT` resolves this script's actual location, takes its parent `tools` directory, then its parent repository directory. This makes normal input lookup independent of the shell's current directory. `CHAPTERS` is an ordered tuple of all seven required chapter filenames. A tuple fixes the desired reading sequence rather than relying on operating-system directory enumeration. `SOURCE_PATHS` explicitly lists application, test, build, editor, dependency, generator, and vendor files. No generated build files or removed model packs are included. Miniaudio's implementation unit is listed before GLAD and the large miniaudio header, so the most oversized source reference appears at the end instead of interrupting explanations.

These lists are intentionally explicit. Adding a new source file requires adding it to the listing and writing its explanation; the script does not pretend it can determine which newly added folder is meaningful. Before rendering, every listed input is checked. A missing chapter or source causes a clear error rather than an incomplete PDF that silently omits the file.

Lines 42-54: inline, Safe Inline Formatting

`inline(text)` receives ordinary text and returns the limited markup understood by ReportLab's Paragraph renderer. `re.split` separates backtick-delimited inline-code fragments while retaining the matched fragments. Each fragment is examined in order. A backtick pair is removed and its literal content escaped, then wrapped in a Courier font tag. This makes identifiers/paths visually distinct and ensures a code expression containing XML characters is not interpreted as markup.

Non-code fragments are escaped first. The bold substitution turns `**text**` into a `<b>` span. The Markdown-link substitution renders the readable label followed by the destination in parentheses; it does not fetch that destination or embed remote content. Appending results to a list and joining once avoids repeatedly growing a string for every fragment. The function deliberately does not implement the complete CommonMark standard, nested Markdown, or arbitrary raw HTML. Unrecognized markup remains text instead of enabling unexpected processing.

Lines 57-68: `source_rows`, Preserve Every Source Line

`source_rows(content, width=100)` converts source text into printable lines. `splitlines()` obtains physical source lines; enumeration begins at one to match editor line numbers. Tabs expand to four spaces to give a consistent printed indentation. `textwrap.wrap` caps code fragments at 100 characters, preserves whitespace, permits splitting a very long uninterrupted identifier/string, and avoids additional hyphen-based breaks. An empty original line becomes one empty chunk rather than disappearing.

The first fragment receives a six-character right-aligned line number and `|` separator. Every continuation receives spaces and `>` instead. The `>` means this is still the same original source line, not a new line number. Each fragment is appended to the result. No character-limit truncation or first-N-lines selection is applied. Line endings and tabs are normalized for presentation, but the manifest hash is calculated from original file bytes rather than this display transformation.

Lines 71-109: `ExplanationDocument` and PDF Navigation

`ExplanationDocument` inherits ReportLab's `BaseDocTemplate`. Its constructor passes the output filename, A4 size, 19 mm left/right margins, 20 mm top margin, 19 mm bottom margin, descriptive PDF title/author, and compression setting to the base class. A body `Frame` fills the allowed document width/height. A default `PageTemplate` uses that frame and calls `decorate` on each page. The style dictionary is retained on the object and the heading counter starts at zero.

`beforeDocument` resets the counter each time a rendering pass begins. The table of contents requires multiple passes: the first discovers heading page numbers, and later passes lay out the resulting TOC and adjust pagination. Resetting yields stable bookmark identifiers across passes rather than continually increasing them.

`decorate` saves the canvas's graphics state before drawing. It sets a pale sand-colored stroke and draws a horizontal line 14 mm below the top. It sets muted text color and an 8-point Helvetica font, adds the reference title above the line, adds course/reference text at the bottom, and right-aligns the current page number. The saved state is restored so these choices cannot accidentally change a paragraph, diagram, or source listing drawn afterward. Dimensions are points internally; multiplying by mm converts the intended physical dimensions.

`afterFlowable` runs after a `Flowable` has been placed. A `Flowable` without `toc_level` is ordinary text and returns immediately. A numbered heading obtains a unique `section-N` key and a plain-text title. `bookmarkPage` binds that key to the current page. `addOutlineEntry` inserts a clickable PDF sidebar entry at chapter or subsection level; subsections start collapsed. `notify("TOCEntry", ...)` tells the TOC its level, title, page number, and bookmark destination. The counter is navigation state, not a source-code line counter.

Lines 112-131: `make_styles`, Typography

The renderer begins with ReportLab's standard sample stylesheet, then builds named styles rather than scattering font settings throughout parsing code. Body text is 9-point Helvetica with 13.5-point line spacing and six points after a paragraph; long words can split to avoid overflowing margins. A loop defines heading levels 1-4 with sizes 20, 13, 10.5, and 10. Each is bold, uses proportional line spacing, has space before/after, uses warm brown text, and remains with the next `Flowable` so headings do not sit alone at a page bottom.

Code samples use 6.5-point Courier with 8-point leading. Source appendices use 7.2-point Courier with 9-point leading, balancing very large retained libraries against readable monospaced references. Table cells inherit body behavior but use 7.8-point text and 10-point leading. The cover title uses 30-point size, 36-point leading, and centered alignment. A separate centered body style supports cover subtitles. The function returns the complete dictionary, making later style selection deterministic.

Lines 134-138: heading, Document Hierarchy

`heading(text, level, styles)` creates a Paragraph from escaped/formatted text. Levels deeper than four use the fourth visual style. Only levels one and two receive `toc_level`: level one becomes TOC/outline level zero, and level two becomes level one. This includes navigable chapter/subsection entries while keeping hundreds of small line-range headings out of the contents. The returned object is still a normal Paragraph with one additional navigation attribute.

Lines 141-206: markdown_flowables, Each Parsing Branch

The parser creates an output list, splits the chapter into lines, creates a pending prose buffer, and tracks a current index. Its nested `flush` function turns accumulated prose lines into a single escaped Paragraph separated by spaces, then clears the buffer. This preserves ordinary paragraphs spread across multiple editor lines. Flushing before structural blocks preserves their intended order.

The main loop strips whitespace only for syntax recognition; code blocks later use the original unstripped line. A line beginning with three backticks starts a literal code block. Pending prose is flushed, the opening marker skipped, and original code lines collected until a closing marker or end of file. They expand tabs and wrap at 109 characters with whitespace preserved. Chunks of at most 45 printed lines become Preformatted objects, reducing oversized unsplitable blocks. A small Spacer follows the code block. The language label after the opening marker is not used for syntax highlighting; this is a plain source-reference renderer.

A heading matched by one through six `#` characters followed by a space is split into marker and title. Its visual depth is capped at four and passed through `heading`. A Markdown table is recognized when a stripped line starts and ends with `|`. Consecutive pipe rows are collected; cells split on the separator and have surrounding whitespace removed. Separator rows consisting only of dashes and optional alignment colons are skipped, while real cells become formatted Paragraphs. This is a simple pipe-table subset: escaped literal pipes inside a cell are not a supported feature and should be written in prose/code blocks instead.

The table branch determines the largest cell count and pads shorter rows with empty cells so the table is rectangular. Available page width is divided evenly among columns. Header row repetition keeps column context on later pages. The header gets a pale sand background, all cells align at the top, a thin grid separates values, and top/bottom padding prevents cramped text. A Spacer separates the table from following content. The parser decreases its index once because the outer loop also advances it; without that correction the first non-table line would be skipped.

Lines beginning with `-`, `*`, or `+`, and lines matching a numbered-list marker, flush pending prose and become their own paragraphs. The markers remain visible text; the parser does not implement nested list indentation. Empty lines and simple horizontal separators flush prose. A `>` line becomes its own paragraph without the quotation marker; no special shaded quote box is generated. Everything else joins the pending buffer. After the loop, a final flush emits any trailing paragraph. The function returns the Flowables in original chapter order.

Lines 209-226: Arguments and Input Validation

`main` creates an `ArgumentParser` using the module description. `--output` accepts a Path and defaults to `docs/Arrakis_Code_Explanation.pdf`. Chapter and source Paths are joined beneath the known repository root. A list comprehension identifies inputs that are not files. If any are missing, `parser.error` prints the complete missing-input list and exits before generating a misleading partial document. A nonexistent output parent is also rejected; the tool does not create arbitrary external directory trees. An explicitly supplied relative output path is interpreted relative to the invoking shell, while default inputs and output remain repository-root based.

Lines 228-247: Cover, Contents, and Chapter Text

The generator creates styles and a Story list with an initial vertical spacer. The cover title uses explicit line breaks between Arrakis and Harvester Down. Subtitle paragraphs state the document's purpose, source-reference scope, and course. A UTC date records the documented snapshot's generation day. A page break starts Contents. The TOC has a bold 9-point chapter style and an 8-point indented subsection style. The TOC object joins the Story before any explanation chapters.

Each chapter then starts on a fresh page. Its UTF-8 text is parsed into Flowables and extended onto the Story. This makes chapter ordering explicit and does not require combining the editable Markdown files into another duplicate manual on disk. The Story is a sequence of rendering objects held in memory until the multi-pass build.

Lines 249-267: Snapshot Manifest and Source-Reference Rules

The manifest begins with UTC timestamp, empty source list, and empty chapter list. An appendix heading and introductory paragraph explain the exact line-number and continuation convention. They also distinguish authored explanations from third-party source reference and exclude installed external headers from the claimed repository scope.

For each source/configuration file, the generator reads its bytes, decodes UTF-8 while tolerating an initial byte-order mark, derives a repository-relative forward-slash path, counts physical text lines, and computes SHA-256 on the unchanged bytes. A dictionary records the path, count, and digest; a corresponding human-readable paragraph is added to the appendix. Hashing bytes detects changes in content and line endings, unlike merely checking file length. The SHA-256 value is an identity fingerprint, not evidence that the implementation is secure or correct.

Lines 269-286: Complete Source Appendices and PDF Verification

The source loop pairs each input Path with its manifest entry, begins a new page, and adds a navigable source-reference heading. `source_rows` formats every line of that file. Blocks of 72 printed rows become Preformatted Flowables; ReportLab can split them at page boundaries if necessary. The complete 95,000-plus-line miniaudio header is therefore retained, not cut down to the handful of calls the game makes.

Each explanation chapter is separately hashed and its whitespace-separated word count recorded. The custom document performs `multiBuild` with up to six layout passes so TOC page numbers stabilize. If layout or parsing fails, the exception remains visible instead of recording fabricated success. After successful PDF creation, `pypdf` reads the actual output and counts its pages. The manifest's PDF entry records the path, physical page count, and byte size. Outputs inside the repository use relative paths; an external requested destination is kept as an external path.

The manifest is serialized with two-space JSON indentation, UTF-8 encoding, and a terminal newline to `docs/documentation_manifest.json`. That generated JSON is not a source input to its own PDF, which would otherwise create a circular hash/size dependency. Two console messages report the actual PDF page/byte count and number of included source/configuration files and chapters. These counts confirm generation, not semantic completeness of the prose. The final `if __name__ == "__main__"` guard runs `main()` only when executing the script, allowing validation tools to import formatting helpers without regenerating the PDF.

How to Update and Validate the Manual

First update the explanation chapter corresponding to a behavior change, including code-range references if line numbers move. Add new project files to `SOURCE_PATHS` and new chapters to `CHAPTERS`. Then install the constrained dependencies and run the generator. Its strict input checks prevent accidental omission of already-listed files. Compare the manifest digests against repository files to verify that the PDF source snapshot is current. Use the PDF contents/sidebar to navigate explanation sections and source files rather than scrolling through thousands of vendor-reference pages.

The generator's subset parser handles the chapter conventions used here, not arbitrary Markdown extensions, Mermaid diagrams, remote image downloads, HTML execution, or syntax-highlighted code. Standard built-in fonts prioritize portability; unusual non-ASCII symbols should be written as words or ASCII formulas if a glyph is unavailable. Keeping

these limitations explicit avoids confusing presentation features with implemented game features.

Appendix A: Source Snapshot and Reading Rules

The following appendices contain every line of the retained application, tests, build/editor configuration, documentation generator, and vendored C/C++ sources. The left number is the physical source line, not a PDF line. A > continuation is the rest of that same source line; long lines are wrapped, never omitted. SHA-256 fingerprints identify this exact snapshot. Application semantics are explained in the preceding code-range chapters. The libraries are explained by functional families; their full source is included for reference, not presented as original game logic. Installed external GLFW, GLM and GLAD headers are dependencies, not files implemented in this repository.

src/arrakis.cpp: 1892 lines; SHA-256

0088edeac81c14658147e8381ef57be5f422ef7421c85b3eb02d0248926b7d31

src/arrakis_game.h: 307 lines; SHA-256

7829e212a9f8665a718c3a464eea30a07f7aaff75e83c550cfa3806285f6c090

src/arrakis_hud.h: 552 lines; SHA-256

2bb9f101426b22f56f37b7a426eff93020e6fb60d3ea2dd61fcb1bc92dd1ee31

src/arrakis_worm.h: 360 lines; SHA-256

09496272eb925afbfe2290398fbde90f6eec632f21defaa20b755c80701c6f04

src/arrakis_audio.h: 293 lines; SHA-256

aec39668979caaef357e93dfe8db1ca9a2eafeb2ab3e44c4c7225dc61e53a4d5

tests/arrakis_tests.h: 1041 lines; SHA-256

6ea0ae56accadd1f5292aa8bf40c4a40a5d138e9287a3157df69bda5ecc51274

CMakeLists.txt: 56 lines; SHA-256

83da39cf976b4ccc36ac211802fa85687a92d267bb5f8aa15c7cb57c6bdf70ec

Arrakis_001.vcxproj: 163 lines; SHA-256

398c0c6b9b62a5a1dec1720d970b9ddba307f6529af96398ba7a4ef788d8d47d

Arrakis_001.vcxproj.filters: 36 lines; SHA-256

9ad2728aacab7ce948d446e9004cc950bdc0e82910f5ab2d9a1aad79dd571021

.gitignore: 64 lines; SHA-256 e32c3f1818b3dd89f09f80d43e14a1d9302dbec810d8bd7664b53bb7ac6fa170

.vscode/c_cpp_properties.json: 17 lines; SHA-256

3874782a0b4f8f2e692ed4bf9db0500b607f9467777feb3d0ba16b26ab01dd97

.vscode/launch.json: 15 lines; SHA-256

06547d7be3bc42251d0a0afd3c5848d7c224c2af009d2ea517ec4372b9f60599

vcpkg.json: 6 lines; SHA-256 c5f248b214efa9d6d231ed581d5f5c4520c452a3e31b5aab82f6024f472ff105

tools/generate_docs.py: 289 lines; SHA-256

053c396dbd11474820b4f6e952a6ab8a750285a5671d5aafadcbfb705c0566a4

tools/requirements.txt: 2 lines; SHA-256

44457c2e516ac79360a956538fd67972988805d2219c048ca573a0b5ec9d20c8

third_party/miniaudio/miniaudio_impl.cpp: 2 lines; SHA-256

ab1984bb9804ffd7b0303813595d0b345a8a86c34da1dafc353a14b34102a65

third_party/glad/glad.c: 1832 lines; SHA-256

95728ccb0552c8bf5cb01de5b6d48a116159d5a71a6ee2dec1bdb45ba294d8bc

third_party/miniaudio/miniaudio.h: 95864 lines; SHA-256

ac7af4de748b7e26b777f37e01cee313a308a7296a3eb080e2906b320cc55c89

Source Reference: src/arrakis.cpp

```

1 | // =====
2 | //   ARRAKIS - Harvester Down: a timed desert rescue game
3 | //   CSE 4102: Computer Graphics and Image Processing Laboratory
4 | //
5 | //   A high-fidelity real-time 3D simulation inspired by the DUNE universe.
6 | //   ALL GEOMETRY IS PROCEDURALLY GENERATED FROM MATHEMATICAL PRIMITIVES.
7 | //   NO EXTERNAL .OBJ FILES ARE LOADED.
8 | //
9 | //   FEATURING:
10 | //       - Movie-Accurate Atreides Ornithopter:
11 | //           8-blade dragonfly tandem flapping wings, faceted stealth cockpit canopy,
12 | //           turbine jet intakes, glowing heat reheat thrusters, aerodynamic banking.
13 | //       - Massive Industrial Spice Harvester:
14 | //           Quad crawler tread units, front rotary harvesting drum, refinery superstructure,
15 | //           bridge floodlights, exhaust funnels venting burning spice fumes.
16 | //       - Movie-Realistic Spice Pressure Tanks:
17 | //           Cylindrical pressure vessels with hemispherical domed heads, structural
18 | //           cradles with diagonal trusses, pulsing illuminated spice level gauges, pipes.
19 | //       - Continuous Sweeping Desert Sand Dunes:
20 | //           Seamless 140x140 vertex heightfield terrain with barchan ridges, procedural
21 | //           wind ripples, slope-dependent color stratification, and analytical normals.
22 | //       - Atmospheric Wind & Blowing Sand Particle System:
23 | //           Over 2,000 blowing sand particles streaming across dunes with wind gusts,
24 | //           harvester crawler dust plumes, and ornithopter wing downwash dust swirls.
25 | //       - Professional Lighting & Atmosphere:
26 | //           Blinn-Phong + Half-Lambert sand wrap lighting, directional desert sun,
27 | //           specular micro-sparkles, soft PCF shadow mapping, and warm desert distance fog.
28 | //       - Cinematic Third-Person Chase Camera with smooth flight kinematics.
29 | //
30 | //   CONTROLS:
31 | //       Enter           - Start / next mission
32 | //       Mouse hover     - Steer aircraft and camera together (center stops turning)
33 | //       W / S, A / D    - Forward / reverse, strafe left / right
34 | //       Q / E           - Raise / lower terrain-following altitude
35 | //       Left Shift      - Rechargeable boost
36 | //       Space           - Hover, lower winch, rescue / unload at base
37 | //       P / R           - Pause / retry mission
38 | //       Arrow Keys      - Keyboard steering / camera elevation
39 | //       C               - Toggle Camera View (Cinematic Chase / Cockpit / Overhead)
40 | //       F               - Toggle Wireframe
41 | //       F11            - Toggle Fullscreen
42 | //       ESC            - Quit
43 | // =====
44 |
45 | #include <glad/glad.h>
46 | #include <GLFW/glfw3.h>
47 |
48 | #include <glm/glm.hpp>
49 | #include <glm/gtc/matrix_transform.hpp>
50 | #include <glm/gtc/type_ptr.hpp>
51 |
52 | #include <iostream>
53 | #include <vector>
54 | #include <cmath>
55 | #include <random>
56 | #include <algorithm>
57 | #include <string>
58 | #include <fstream>
59 | #include <stdint>
60 |
61 | // =====
62 | //   WINDOW & DISPLAY CONFIGURATION
63 | // =====
64 | int  g_scrW = 1280;
65 | int  g_scrH = 720;
66 | bool g_isFullscreen = false;
67 | GLFWwindow* g_window = nullptr;
68 |
69 | // =====
70 | //   CAMERA & FLIGHT STATE
71 | // =====
72 | bool g_keys[1024] = {};
73 | bool g_wireframe = false;
74 | int  g_camMode = 0; // 0: Cinematic Chase, 1: Cockpit/Close, 2: Overhead Tactical
75 | float g_camOrbitPitch = 0.0f;
76 |

```

```

77 | // Flight kinematics
78 | glm::vec3 g_ornPos(0.0f, 18.0f, 0.0f);
79 | glm::vec3 g_ornVel(0.0f);
80 | float     g_ornYaw = 0.0f;      // Heading in degrees
81 | float     g_ornPitch = 0.0f;    // Pitch tilt in degrees
82 | float     g_ornRoll = 0.0f;     // Aerodynamic banking roll in degrees
83 | float     g_ornSpeed = 0.0f;    // Current cruise velocity
84 | bool      g_isBoosting = false;
85 |
86 | // Wing kinematics (movie-accurate 8-wing tandem flutter)
87 | float g_wingPhase = 0.0f;
88 |
89 | // Harvester state
90 | float g_harvX = -80.0f;
91 | float g_harvZ = 15.0f;
92 |
93 | // Dynamic Spice Tanks
94 | struct SpiceTankInstance {
95 |     glm::vec3 pos;
96 |     float yaw;
97 |     float scale;
98 | };
99 | std::vector<SpiceTankInstance> g_tanks;
100 |
101 | // =====
102 | // DUNE TERRAIN HEIGHTFIELD FUNCTION
103 | // Continuous procedural analytical surface for seamless rolling dunes
104 | // =====
105 | float rawDuneHeight(float x, float z) {
106 |     // Primary sweeping barchan dune swells
107 |     float h1 = std::sin(x * 0.009f + z * 0.004f) * 14.0f;
108 |     float h2 = std::cos(x * 0.004f - z * 0.012f) * 9.5f;
109 |
110 |     // Asymmetric windward & leeward slipface ridges
111 |     float r = std::sin(x * 0.028f + z * 0.016f);
112 |     r = 1.0f - std::abs(r);
113 |     float ridge = r * r * 5.0f;
114 |
115 |     // Fine dunes & desert undulation
116 |     float h3 = std::sin(x * 0.065f + z * 0.038f) * 1.8f;
117 |     return h1 + h2 + ridge + h3 - 4.0f;
118 | }
119 |
120 | float getDuneHeight(float x, float z) {
121 |     float height = rawDuneHeight(x, z);
122 |     // Level the landing pad into the dunes; evacuation sites stay natural.
123 |     static const glm::vec3 terraces[] = {{55, 55, rawDuneHeight(55, 55)}};
124 |     for (const auto& center : terraces) {
125 |         float radius = center.x == 55 ? 15.0f : 10.0f;
126 |         float blend = 1.0f - glm::smoothstep(radius, radius + 10, glm::distance(glm::vec2(x, z),
> glm::vec2(center)));
127 |         height = glm::mix(height, center.z, blend);
128 |     }
129 |     return height;
130 | }
131 |
132 | glm::vec3 getDuneNormal(float x, float z) {
133 |     float eps = 0.4f;
134 |     float hL = getDuneHeight(x - eps, z);
135 |     float hR = getDuneHeight(x + eps, z);
136 |     float hD = getDuneHeight(x, z - eps);
137 |     float hU = getDuneHeight(x, z + eps);
138 |     return glm::normalize(glm::vec3(-(hR - hL), 2.0f * eps, -(hU - hD)));
139 | }
140 |
141 | #include "arrakis_audio.h"
142 | #include "arrakis_game.h"
143 | #include "arrakis_tests.h"
144 |
145 | // =====
146 | // WIND & PARTICLE SYSTEM
147 | // =====
148 | const glm::vec3 WIND_DIR = glm::normalize(glm::vec3(-0.92f, -0.04f, 0.38f));
149 | const float     WIND_BASE_SPEED = 18.0f;
150 |
151 | struct SandParticle {
152 |     glm::vec3 pos;
153 |     glm::vec3 vel;
154 |     float     life;

```



```

155 |     float    maxLife;
156 |     float    size;
157 |     float    alpha;
158 | };
159 |
160 | const int MAX_PARTICLES = 2400;
161 | std::vector<SandParticle> g_particles;
162 |
163 | void initParticles() {
164 |     g_particles.resize(MAX_PARTICLES);
165 |     std::mt19937 rng(1337);
166 |     std::uniform_real_distribution<float> distPos(-120.0f, 120.0f);
167 |     std::uniform_real_distribution<float> distY(0.5f, 25.0f);
168 |     std::uniform_real_distribution<float> distLife(1.0f, 4.0f);
169 |     std::uniform_real_distribution<float> distSpeed(0.8f, 1.4f);
170 |
171 |     for (auto& p : g_particles) {
172 |         p.pos = glm::vec3(distPos(rng), distY(rng), distPos(rng));
173 |         p.vel = WIND_DIR * (WIND_BASE_SPEED * distSpeed(rng));
174 |         p.maxLife = distLife(rng);
175 |         p.life = distLife(rng);
176 |         p.size = 0.18f + (rng() % 100) * 0.003f;
177 |         p.alpha = 0.25f + (rng() % 100) * 0.0035f;
178 |     }
179 | }
180 |
181 | void updateParticles(float dt, glm::vec3 centerPos, glm::vec3 harvPos) {
182 |     float gust = 1.0f + 0.35f * std::sin((float)glfwGetTime() * 1.8f);
183 |     static std::mt19937 rng(1984);
184 |     std::uniform_real_distribution<float> distOffset(-100.0f, 100.0f);
185 |     std::uniform_real_distribution<float> distY(0.2f, 32.0f);
186 |
187 |     for (int i = 0; i < (int)g_particles.size(); ++i) {
188 |         auto& p = g_particles[i];
189 |         p.life -= dt;
190 |         p.pos += p.vel * (gust * dt);
191 |
192 |         // Ground collision & terrain hugging
193 |         float groundY = getDuneHeight(p.pos.x, p.pos.z);
194 |         if (p.pos.y < groundY + 0.2f) {
195 |             p.pos.y = groundY + 0.2f;
196 |             p.vel.y = std::abs(p.vel.y) * 0.5f + 0.8f;
197 |         }
198 |
199 |         // Respawn if life expired or drifted too far from camera/player
200 |         float distToPlayer = glm::distance(glm::vec2(p.pos.x, p.pos.z), glm::vec2(centerPos.x,
> centerPos.z));
201 |         if (p.life <= 0.0f || (distToPlayer > 160.0f && i % 4 != 0 && i % 3 != 0)) {
202 |             p.life = 2.0f + (rng() % 100) * 0.02f;
203 |             p.maxLife = p.life;
204 |
205 |             // Half particles spawn upwind from player, some spawn behind harvester crawler treads
206 |             if (i % 4 == 0 && wormAttackTime() < 12) {
207 |                 // Crawler dust plume
208 |                 glm::vec3 forward = harvesterForward(g_mission.harvesterYaw);
209 |                 glm::vec3 right = glm::cross(forward, glm::vec3(0, 1, 0));
210 |                 p.pos = harvPos - forward * 4.0f + right * (-4.0f + (rng() % 80) * 0.1f)
211 |                     + glm::vec3(0, 1.0f + (rng() % 30) * 0.1f, 0);
212 |                 p.vel = WIND_DIR * (WIND_BASE_SPEED * 0.5f) - forward * 2.0f + glm::vec3(0, 2.5f,
> 0);
213 |                 p.alpha = 0.55f;
214 |                 p.size = 0.45f;
215 |             } else if (i % 3 == 0 && wormAttackTime() < 18) {
216 |                 glm::vec3 worm = wormPosition();
217 |                 float a = (rng() % 10000) * 0.0006283185f;
218 |                 float radius = 10 + (rng() % 1500) * 0.01f;
219 |                 p.pos = worm + glm::vec3(std::cos(a) * radius - (rng() % 2200) * 0.01f, 0,
> std::sin(a) * radius);
220 |                 p.pos.y = getDuneHeight(p.pos.x, p.pos.z) + 0.8f;
221 |                 p.vel = glm::vec3(std::cos(a) * 7, wormAttackTime() > 0 ? 10 : 2, std::sin(a) * 7) +
> WIND_DIR * 4.0f;
222 |                 p.size = wormAttackTime() > 0 ? 5.5f : 2.2f;
223 |                 p.alpha = 0.22f;
224 |             } else if (i % 5 == 0 && g_keys[GLFW_KEY_SPACE]) {
225 |                 float a = (rng() % 1000) * 0.006283185f;
226 |                 p.pos = centerPos + glm::vec3(std::cos(a) * 4, -4, std::sin(a) * 4);
227 |                 p.vel = glm::vec3(std::cos(a) * 11, 1.5f, std::sin(a) * 11);
228 |                 p.size = 1.8f; p.alpha = 0.18f;
229 |             } else {

```

```

230 |         glm::vec3 upwind = -WIND_DIR * 90.0f;
231 |         p.pos = centerPos + upwind + glm::vec3(distOffset(rng), distY(rng),
> distOffset(rng));
232 |         float gy = getDuneHeight(p.pos.x, p.pos.z);
233 |         if (p.pos.y < gy) p.pos.y = gy + 1.0f;
234 |         p.vel = WIND_DIR * (WIND_BASE_SPEED * (0.85f + (rng() % 40) * 0.01f));
235 |         p.alpha = 0.28f;
236 |         p.size = 0.22f;
237 |     }
238 | }
239 | }
240 | }
241 |
242 | // =====
243 | // PROCEDURAL MESH DATA STRUCTURES & PRIMITIVES
244 | // All shapes created mathematically without external asset files!
245 | // =====
246 | struct Vertex {
247 |     glm::vec3 pos;
248 |     glm::vec3 normal;
249 |     glm::vec2 uv;
250 | };
251 |
252 | struct Mesh {
253 |     unsigned int VAO = 0;
254 |     unsigned int VBO = 0;
255 |     unsigned int EBO = 0;
256 |     GLsizei      indexCount = 0;
257 |     GLenum       drawMode = GL_TRIANGLES;
258 | };
259 |
260 | Mesh uploadMesh(const std::vector<Vertex>& verts, const std::vector<unsigned int>& indices) {
261 |     Mesh m;
262 |     m.indexCount = (GLsizei)indices.size();
263 |
264 |     glGenVertexArrays(1, &m.VAO);
265 |     glGenBuffers(1, &m.VBO);
266 |     glGenBuffers(1, &m.EBO);
267 |
268 |     glBindVertexArray(m.VAO);
269 |
270 |     glBindBuffer(GL_ARRAY_BUFFER, m.VBO);
271 |     glBufferData(GL_ARRAY_BUFFER, verts.size() * sizeof(Vertex), verts.data(), GL_STATIC_DRAW);
272 |
273 |     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, m.EBO);
274 |     glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(unsigned int), indices.data(),
> GL_STATIC_DRAW);
275 |
276 |     // layout 0: pos, 1: normal, 2: uv
277 |     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)offsetof(Vertex, pos));
278 |     glEnableVertexAttribArray(0);
279 |
280 |     glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)offsetof(Vertex,
> normal));
281 |     glEnableVertexAttribArray(1);
282 |
283 |     glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)offsetof(Vertex, uv));
284 |     glEnableVertexAttribArray(2);
285 |
286 |     glBindVertexArray(0);
287 |     return m;
288 | }
289 |
290 | // 1. Primitive Cube (with separate normals per face)
291 | Mesh createCube() {
292 |     std::vector<Vertex> v;
293 |     std::vector<unsigned int> idx;
294 |
295 |     auto addFace = [&](glm::vec3 p0, glm::vec3 p1, glm::vec3 p2, glm::vec3 p3, glm::vec3 norm) {
296 |         unsigned int base = (unsigned int)v.size();
297 |         v.push_back({ p0, norm, {0.0f, 0.0f} });
298 |         v.push_back({ p1, norm, {1.0f, 0.0f} });
299 |         v.push_back({ p2, norm, {1.0f, 1.0f} });
300 |         v.push_back({ p3, norm, {0.0f, 1.0f} });
301 |         idx.push_back(base + 0); idx.push_back(base + 1); idx.push_back(base + 2);
302 |         idx.push_back(base + 0); idx.push_back(base + 2); idx.push_back(base + 3);
303 |     };
304 |
305 |     // Front (+Z), Back (-Z), Left (-X), Right (+X), Top (+Y), Bottom (-Y)

```

```

306 |     addFace({-0.5,-0.5, 0.5}, { 0.5,-0.5, 0.5}, { 0.5, 0.5, 0.5}, {-0.5, 0.5, 0.5}, { 0, 0, 1});
307 |     addFace({ 0.5,-0.5,-0.5}, {-0.5,-0.5,-0.5}, {-0.5, 0.5,-0.5}, { 0.5, 0.5,-0.5}, { 0, 0,-1});
308 |     addFace({-0.5,-0.5,-0.5}, {-0.5,-0.5, 0.5}, {-0.5, 0.5, 0.5}, {-0.5, 0.5,-0.5}, {-1, 0, 0});
309 |     addFace({ 0.5,-0.5, 0.5}, { 0.5,-0.5,-0.5}, { 0.5, 0.5,-0.5}, { 0.5, 0.5, 0.5}, { 1, 0, 0});
310 |     addFace({-0.5, 0.5, 0.5}, { 0.5, 0.5, 0.5}, { 0.5, 0.5,-0.5}, {-0.5, 0.5,-0.5}, { 0, 1, 0});
311 |     addFace({-0.5,-0.5,-0.5}, { 0.5,-0.5,-0.5}, { 0.5,-0.5, 0.5}, {-0.5,-0.5, 0.5}, { 0,-1, 0});
312 |
313 |     return uploadMesh(v, idx);
314 | }
315 |
316 | // 2. Primitive Cylinder (smooth body + capped ends)
317 | Mesh createCylinder(int slices = 24) {
318 |     std::vector<Vertex> v;
319 |     std::vector<unsigned int> idx;
320 |
321 |     float r = 0.5f;
322 |     float h = 1.0f;
323 |
324 |     // Body
325 |     for (int i = 0; i <= slices; ++i) {
326 |         float theta = (float)i / (float)slices * glm::two_pi<float>();
327 |         float c = std::cos(theta);
328 |         float s = std::sin(theta);
329 |         glm::vec3 norm(c, 0.0f, s);
330 |         float u = (float)i / (float)slices;
331 |
332 |         v.push_back({ glm::vec3(r * c, -0.5f * h, r * s), norm, glm::vec2(u, 0.0f) });
333 |         v.push_back({ glm::vec3(r * c, 0.5f * h, r * s), norm, glm::vec2(u, 1.0f) });
334 |     }
335 |
336 |     for (int i = 0; i < slices; ++i) {
337 |         unsigned int b = i * 2;
338 |         idx.push_back(b); idx.push_back(b + 1); idx.push_back(b + 3);
339 |         idx.push_back(b); idx.push_back(b + 3); idx.push_back(b + 2);
340 |     }
341 |
342 |     // Top cap (+Y)
343 |     unsigned int topCenter = (unsigned int)v.size();
344 |     v.push_back({ glm::vec3(0.0f, 0.5f * h, 0.0f), glm::vec3(0, 1, 0), glm::vec2(0.5f, 0.5f) });
345 |     for (int i = 0; i <= slices; ++i) {
346 |         float theta = (float)i / (float)slices * glm::two_pi<float>();
347 |         v.push_back({ glm::vec3(r * std::cos(theta), 0.5f * h, r * std::sin(theta)), glm::vec3(0, 1,
> 0), glm::vec2(0.5f + 0.5f * std::cos(theta), 0.5f + 0.5f * std::sin(theta)) });
348 |     }
349 |     for (int i = 0; i < slices; ++i) {
350 |         idx.push_back(topCenter);
351 |         idx.push_back(topCenter + 1 + i + 1);
352 |         idx.push_back(topCenter + 1 + i);
353 |     }
354 |
355 |     // Bottom cap (-Y)
356 |     unsigned int btmCenter = (unsigned int)v.size();
357 |     v.push_back({ glm::vec3(0.0f, -0.5f * h, 0.0f), glm::vec3(0, -1, 0), glm::vec2(0.5f, 0.5f) });
358 |     for (int i = 0; i <= slices; ++i) {
359 |         float theta = (float)i / (float)slices * glm::two_pi<float>();
360 |         v.push_back({ glm::vec3(r * std::cos(theta), -0.5f * h, r * std::sin(theta)), glm::vec3(0,
> -1, 0), glm::vec2(0.5f + 0.5f * std::cos(theta), 0.5f + 0.5f * std::sin(theta)) });
361 |     }
362 |     for (int i = 0; i < slices; ++i) {
363 |         idx.push_back(btmCenter);
364 |         idx.push_back(btmCenter + 1 + i);
365 |         idx.push_back(btmCenter + 1 + i + 1);
366 |     }
367 |
368 |     return uploadMesh(v, idx);
369 | }
370 |
371 | // 3. Primitive Sphere (smooth UV sphere)
372 | Mesh createSphere(int lats = 16, int lons = 24) {
373 |     std::vector<Vertex> v;
374 |     std::vector<unsigned int> idx;
375 |
376 |     for (int i = 0; i <= lats; ++i) {
377 |         float theta = (float)i / (float)lats * glm::pi<float>();
378 |         float sinT = std::sin(theta);
379 |         float cosT = std::cos(theta);
380 |
381 |         for (int j = 0; j <= lons; ++j) {
382 |             float phi = (float)j / (float)lons * glm::two_pi<float>();

```

```

383 |         float sinP = std::sin(phi);
384 |         float cosP = std::cos(phi);
385 |
386 |         glm::vec3 norm(sinT * cosP, cosT, sinT * sinP);
387 |         glm::vec3 pos = norm * 0.5f;
388 |         glm::vec2 uv((float)j / (float)lons, (float)i / (float)lats);
389 |
390 |         v.push_back({ pos, norm, uv });
391 |     }
392 | }
393 |
394 | for (int i = 0; i < lats; ++i) {
395 |     for (int j = 0; j < lons; ++j) {
396 |         unsigned int first = (i * (lons + 1)) + j;
397 |         unsigned int second = first + lons + 1;
398 |
399 |         idx.push_back(first);
400 |         idx.push_back(first + 1);
401 |         idx.push_back(second);
402 |
403 |         idx.push_back(second);
404 |         idx.push_back(first + 1);
405 |         idx.push_back(second + 1);
406 |     }
407 | }
408 |
409 | return uploadMesh(v, idx);
410 | }
411 |
412 | // 4. Primitive Aerofoil Blade / Wedge (tapered insectoid wing)
413 | Mesh createWingBlade() {
414 |     std::vector<Vertex> v;
415 |     std::vector<unsigned int> idx;
416 |
417 |     // A slender tapered blade cross-section along X (span: 0 to 1), with chord along Z (-0.5 to
> 0.5)
418 |     // Root chord = 1.0, Tip chord = 0.25, Root thickness = 0.08, Tip thickness = 0.02
419 |     int spanSegs = 8;
420 |     for (int i = 0; i <= spanSegs; ++i) {
421 |         float t = (float)i / (float)spanSegs; // 0 (root) to 1 (tip)
422 |         float x = t;
423 |         float chord = glm::mix(1.0f, 0.25f, t);
424 |         float thick = glm::mix(0.08f, 0.015f, t);
425 |
426 |         // 4 points around aerofoil profile: leading edge, top crest, trailing edge, bottom crest
427 |         glm::vec3 pLead(x, 0.0f, -chord * 0.45f);
428 |         glm::vec3 pTop(x, thick * 0.5f, -chord * 0.1f);
429 |         glm::vec3 pTrail(x, 0.0f, chord * 0.55f);
430 |         glm::vec3 pBtm(x, -thick * 0.5f, -chord * 0.1f);
431 |
432 |         v.push_back({ pLead, glm::normalize(glm::vec3(0, 0, -1)), {t, 0.0f} });
433 |         v.push_back({ pTop, glm::normalize(glm::vec3(0, 1, 0)), {t, 0.33f} });
434 |         v.push_back({ pTrail, glm::normalize(glm::vec3(0, 0, 1)), {t, 0.66f} });
435 |         v.push_back({ pBtm, glm::normalize(glm::vec3(0, -1, 0)), {t, 1.0f} });
436 |     }
437 |
438 |     for (int i = 0; i < spanSegs; ++i) {
439 |         unsigned int b1 = i * 4;
440 |         unsigned int b2 = (i + 1) * 4;
441 |         for (int k = 0; k < 4; ++k) {
442 |             int kNext = (k + 1) % 4;
443 |             idx.push_back(b1 + k); idx.push_back(b2 + k); idx.push_back(b2 + kNext);
444 |             idx.push_back(b1 + k); idx.push_back(b2 + kNext); idx.push_back(b1 + kNext);
445 |         }
446 |     }
447 |
448 |     return uploadMesh(v, idx);
449 | }
450 |
451 | // 5. Continuous Rolling Sand Dune Terrain Heightfield Mesh
452 | Mesh createDuneTerrain(int gridW = 140, int gridH = 140, float totalSize = 700.0f) {
453 |     std::vector<Vertex> v;
454 |     std::vector<unsigned int> idx;
455 |     v.reserve((gridW + 1) * (gridH + 1));
456 |     idx.reserve(gridW * gridH * 6);
457 |
458 |     float halfSize = totalSize * 0.5f;
459 |     float step = totalSize / (float)gridW;
460 |

```

```

461 |     for (int z = 0; z <= gridH; ++z) {
462 |         float worldZ = -halfSize + z * step;
463 |         for (int x = 0; x <= gridW; ++x) {
464 |             float worldX = -halfSize + x * step;
465 |             float worldY = getDuneHeight(worldX, worldZ);
466 |             glm::vec3 norm = getDuneNormal(worldX, worldZ);
467 |             glm::vec2 uv(worldX * 0.05f, worldZ * 0.05f);
468 |
469 |             v.push_back({ glm::vec3(worldX, worldY, worldZ), norm, uv });
470 |         }
471 |     }
472 |
473 |     for (int z = 0; z < gridH; ++z) {
474 |         for (int x = 0; x < gridW; ++x) {
475 |             unsigned int row1 = z * (gridW + 1);
476 |             unsigned int row2 = (z + 1) * (gridW + 1);
477 |
478 |             idx.push_back(row1 + x);
479 |             idx.push_back(row2 + x);
480 |             idx.push_back(row1 + x + 1);
481 |
482 |             idx.push_back(row1 + x + 1);
483 |             idx.push_back(row2 + x);
484 |             idx.push_back(row2 + x + 1);
485 |         }
486 |     }
487 |
488 |     return uploadMesh(v, idx);
489 | }
490 |
491 | // 6. Billboard Quad for blowing sand & particles
492 | Mesh createQuad() {
493 |     std::vector<Vertex> v = {
494 |         { {-0.5f, -0.5f, 0.0f}, {0, 0, 1}, {0, 0} },
495 |         { { 0.5f, -0.5f, 0.0f}, {0, 0, 1}, {1, 0} },
496 |         { { 0.5f,  0.5f, 0.0f}, {0, 0, 1}, {1, 1} },
497 |         { {-0.5f,  0.5f, 0.0f}, {0, 0, 1}, {0, 1} }
498 |     };
499 |     std::vector<unsigned int> idx = { 0, 1, 2, 0, 2, 3 };
500 |     return uploadMesh(v, idx);
501 | }
502 |
503 | Mesh createRing() {
504 |     std::vector<Vertex> vertices;
505 |     std::vector<unsigned int> indices;
506 |     for (int i = 0; i <= 80; ++i) for (int j = 0; j <= 8; ++j) {
507 |         float a = i * glm::two_pi<float>() / 80;
508 |         float b = j * glm::two_pi<float>() / 8;
509 |         glm::vec3 n(std::cos(a) * std::cos(b), std::sin(b), std::sin(a) * std::cos(b));
510 |         vertices.push_back({glm::vec3(std::cos(a), 0, std::sin(a)) + n * 0.025f, n, {i / 80.0f, j /
> 8.0f}}});
511 |     }
512 |     for (int i = 0; i < 80; ++i) for (int j = 0; j < 8; ++j) {
513 |         unsigned int a = i * 9 + j, b = a + 9;
514 |         indices.insert(indices.end(), {a, a + 1, b, b, a + 1, b + 1});
515 |     }
516 |     return uploadMesh(vertices, indices);
517 | }
518 |
519 | Mesh createErodedRock() {
520 |     std::vector<Vertex> v;
521 |     std::vector<unsigned int> idx;
522 |     auto point = [](int lat, int lon) {
523 |         float a = lat * glm::pi<float>() / 8, b = lon * glm::two_pi<float>() / 13;
524 |         float erosion = 0.83f + 0.13f * std::sin(lon * 7.1f + lat * 2.3f) + 0.08f * std::cos(lat *
> 5.4f);
525 |         return glm::vec3(std::sin(a) * std::cos(b) * erosion, std::cos(a), std::sin(a) * std::sin(b)
> * erosion) * 0.5f;
526 |     };
527 |     auto tri = [&](glm::vec3 a, glm::vec3 b, glm::vec3 c) {
528 |         glm::vec3 n = glm::cross(b - a, c - a);
529 |         if (glm::length(n) < 0.00001f) return;
530 |         if (glm::dot(n, a + b + c) < 0) { std::swap(b, c); n = -n; }
531 |         n = glm::normalize(n);
532 |         unsigned int start = static_cast<unsigned int>(v.size());
533 |         v.insert(v.end(), {{a, n, {0, 0}}, {b, n, {1, 0}}, {c, n, {1, 1}}});
534 |         idx.insert(idx.end(), {start, start + 1, start + 2});
535 |     };
536 |     for (int i = 0; i < 8; ++i) for (int j = 0; j < 13; ++j) {

```

```

537 |         tri(point(i, j), point(i + 1, j), point(i, j + 1));
538 |         tri(point(i, j + 1), point(i + 1, j), point(i + 1, j + 1));
539 |     }
540 |     return uploadMesh(v, idx);
541 | }
542 |
543 | // =====
544 | // GLOBAL MESH ASSETS
545 | // =====
546 | Mesh g_meshCube;
547 | Mesh g_meshCylinder;
548 | Mesh g_meshSphere;
549 | Mesh g_meshWing;
550 | Mesh g_meshTerrain;
551 | Mesh g_meshQuad;
552 | Mesh g_meshRing;
553 | Mesh g_meshRock;
554 | unsigned int g_particleInstanceVBO = 0;
555 | struct ParticleInstance { glm::vec4 posSize; glm::vec4 colorAlpha; };
556 | std::vector<ParticleInstance> g_particleInstances;
557 |
558 | // =====
559 | // SHADERS & UNIFORMS
560 | // =====
561 |
562 | // 1. Comprehensive Cinematic Surface Shader (Blinn-Phong + Sand Wrap + Shadows + Fog)
563 | const char* SCENE_VERT = R"(
564 | #version 330 core
565 | layout(location=0) in vec3 aPos;
566 | layout(location=1) in vec3 aNorm;
567 | layout(location=2) in vec2 aUV;
568 |
569 | uniform mat4 model;
570 | uniform mat4 view;
571 | uniform mat4 projection;
572 | uniform mat4 lightSpaceMatrix;
573 | uniform vec3 wormCenter;
574 | uniform float collapse;
575 | uniform int isSand;
576 |
577 | out vec3 FragPos;
578 | out vec3 Normal;
579 | out vec2 TexCoord;
580 | out vec4 FragPosLight;
581 |
582 | void main(){
583 |     vec4 worldPos = model * vec4(aPos, 1.0);
584 |     if(isSand == 1) {
585 |         float d = length(worldPos.xz - wormCenter.xz);
586 |         worldPos.y -= collapse * (1.0 - smoothstep(5.0, 48.0, d)) * 13.0;
587 |     }
588 |     FragPos = worldPos.xyz;
589 |     Normal = mat3(transpose(inverse(model))) * aNorm;
590 |     TexCoord = aUV;
591 |     FragPosLight = lightSpaceMatrix * worldPos;
592 |     gl_Position = projection * view * worldPos;
593 | }
594 | )";
595 |
596 | const char* SCENE_FRAG = R"(
597 | #version 330 core
598 | in vec3 FragPos;
599 | in vec3 Normal;
600 | in vec2 TexCoord;
601 | in vec4 FragPosLight;
602 |
603 | out vec4 FragColor;
604 |
605 | uniform vec3 objectColor;
606 | uniform vec3 specularColor;
607 | uniform float shininess;
608 | uniform vec3 emissiveColor;
609 | uniform int isSand;
610 |
611 | uniform vec3 lightDir;        // Normalized direction toward sun
612 | uniform vec3 sunColor;
613 | uniform vec3 ambientColor;
614 | uniform vec3 viewPos;
615 | uniform sampler2D shadowMap;

```



```

616 |
617 | // Warm Desert Atmospheric Fog
618 | uniform vec3 fogColor;
619 | uniform float fogDensity;
620 |
621 | float calcShadow(vec4 fragLight, vec3 norm){
622 |     vec3 proj = fragLight.xyz / fragLight.w;
623 |     proj = proj * 0.5 + 0.5;
624 |     // Receiver-plane depth adjustment prevents PCF striping at grazing sunlight.
625 |     vec3 dx = dFdx(proj), dy = dFdy(proj);
626 |     float determinant = dx.x * dy.y - dx.y * dy.x;
627 |     vec2 depthSlope = vec2(0);
628 |     if(abs(determinant) > 0.000000001)
629 |         depthSlope = vec2(dx.z * dy.y - dy.z * dx.y, dx.x * dy.z - dy.x * dx.z) / determinant;
630 |     if(proj.z > 1.0 || proj.z < 0.0 || any(lessThan(proj.xy, vec2(0))) || any(greaterThan(proj.xy,
> vec2(1)))) return 0.0;
631 |
632 |     float bias = max(0.0035 * (1.0 - dot(norm, lightDir)), 0.0008);
633 |     float shadow = 0.0;
634 |     vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
635 |
636 |     // 5x5 PCF with a slope-corrected receiver depth at each sample.
637 |     for(int x = -2; x <= 2; ++x){
638 |         for(int y = -2; y <= 2; ++y){
639 |             vec2 offset = vec2(x,y) * texelSize;
640 |             float pcfDepth = texture(shadowMap, proj.xy + offset).r;
641 |             shadow += (proj.z + dot(depthSlope, offset) - bias > pcfDepth) ? 1.0 : 0.0;
642 |         }
643 |     }
644 |     return shadow / 25.0;
645 | }
646 |
647 | void main(){
648 |     vec3 N = normalize(Normal);
649 |     vec3 L = normalize(lightDir);
650 |     vec3 V = normalize(viewPos - FragPos);
651 |     vec3 H = normalize(L + V);
652 |
653 |     vec3 baseCol = objectColor;
654 |     if(isSand == 2) {
655 |         float grain = fract(sin(dot(FragPos, vec3(12.3, 42.8, 27.1))) * 43758.5453);
656 |         float layers = sin(FragPos.y * 5.0 + sin(FragPos.x * 0.8));
657 |         baseCol *= 0.88 + 0.12 * layers + 0.07 * grain;
658 |     }
659 |
660 |     // Procedural Dune Sand Micro-Shading
661 |     if(isSand == 1){
662 |         // Fine wind ripples aligned with wind angle
663 |         float ripple1 = sin(FragPos.x * 2.2 + FragPos.z * 1.4) * 0.5 + 0.5;
664 |         float ripple2 = sin(FragPos.x * 0.8 - FragPos.z * 1.9) * 0.5 + 0.5;
665 |         float ripple = mix(ripple1, ripple2, 0.5);
666 |
667 |         // Slope shading (crests are sun-bleached golden, valleys are deeper ochre)
668 |         float slope = clamp(dot(N, vec3(0, 1, 0)), 0.0, 1.0);
669 |         baseCol = mix(baseCol * 0.82, baseCol * 1.15, slope);
670 |         float fade = 1.0 - smoothstep(25.0, 120.0, length(viewPos - FragPos));
671 |         baseCol += vec3(0.035, 0.02, 0.008) * (ripple - 0.5) * fade;
672 |         N = normalize(N + vec3(cos(FragPos.x * 2.2 + FragPos.z * 1.4) * 0.055, 0,
673 |             cos(FragPos.x * 2.2 + FragPos.z * 1.4) * 0.035) * fade);
674 |
675 |         // Subtle quartz sand sparkle under direct sun
676 |         float sparkle = pow(max(dot(N, H), 0.0), 120.0);
677 |         float noise = fract(sin(dot(FragPos.xz, vec2(12.9898, 78.233))) * 43758.5453);
678 |         if(noise > 0.88) baseCol += vec3(0.35, 0.3, 0.22) * sparkle * fade;
679 |     }
680 |
681 |     // Ambient Term (warm bounce light from vast desert sand)
682 |     vec3 ambient = ambientColor * baseCol;
683 |
684 |     // Diffuse Term (Half-Lambert wrap lighting for velvety sand diffusion)
685 |     float NdotL = dot(N, L);
686 |     float diff = isSand == 1 ? max((NdotL + 0.35) / 1.35, 0.0) : max(NdotL, 0.0);
687 |     vec3 diffuse = diff * sunColor * baseCol;
688 |
689 |     // Specular Term (Blinn-Phong)
690 |     float spec = pow(max(dot(N, H), 0.0), shininess);
691 |     vec3 specular = spec * specularColor * sunColor;
692 |
693 |     // Shadow Calculation

```

```

694 |     float shadow = calcShadow(FragPosLight, N);
695 |
696 |     vec3 litColor = ambient + (1.0 - shadow) * (diffuse + specular) + emissiveColor;
697 |
698 |     // Exponential Squared Desert Distance Fog
699 |     float dist = length(viewPos - FragPos);
700 |     float fogFactor = 1.0 - exp(-pow(dist * fogDensity, 2.0));
701 |     fogFactor = clamp(fogFactor, 0.0, 1.0);
702 |
703 |     vec3 mapped = pow(litColor / (litColor + vec3(0.8)), vec3(1.0 / 1.8));
704 |     FragColor = vec4(mix(mapped, fogColor, fogFactor), 1.0);
705 | }
706 | )";
707 |
708 | // 2. Shadow Depth Pass Shader
709 | const char* SHADOW_VERT = R"(
710 | #version 330 core
711 | layout(location=0) in vec3 aPos;
712 | uniform mat4 lightSpaceMatrix;
713 | uniform mat4 model;
714 | uniform vec3 wormCenter;
715 | uniform float collapse;
716 | uniform int isSand;
717 | void main(){
718 |     vec4 p = model * vec4(aPos, 1.0);
719 |     if(isSand == 1) p.y -= collapse * (1.0 - smoothstep(5.0, 48.0, length(p.xz - wormCenter.xz))) *
> 13.0;
720 |     gl_Position = lightSpaceMatrix * p;
721 | }
722 | )";
723 | const char* SHADOW_FRAG = R"(
724 | #version 330 core
725 | void main(){}
726 | )";
727 |
728 | // 3. Sand Particles Shader (Soft Billboards)
729 | const char* PARTICLE_VERT = R"(
730 | #version 330 core
731 | layout(location=0) in vec3 aPos;
732 | layout(location=2) in vec2 aUV;
733 | layout(location=3) in vec4 instancePosSize;
734 | layout(location=4) in vec4 instanceColorAlpha;
735 |
736 | uniform mat4 view;
737 | uniform mat4 projection;
738 | uniform vec3 camRight;
739 | uniform vec3 camUp;
740 |
741 | out vec2 TexCoord;
742 | out vec4 ColorAlpha;
743 |
744 | void main(){
745 |     TexCoord = aUV;
746 |     ColorAlpha = instanceColorAlpha;
747 |     vec3 worldPos = instancePosSize.xyz + camRight * (aPos.x * instancePosSize.w) + camUp * (aPos.y
> * instancePosSize.w);
748 |     gl_Position = projection * view * vec4(worldPos, 1.0);
749 | }
750 | )";
751 |
752 | const char* PARTICLE_FRAG = R"(
753 | #version 330 core
754 | in vec2 TexCoord;
755 | out vec4 FragColor;
756 |
757 | in vec4 ColorAlpha;
758 |
759 | void main(){
760 |     float dist = length(TexCoord - vec2(0.5));
761 |     if(dist > 0.5) discard;
762 |     float soft = 1.0 - smoothstep(0.0, 0.5, dist);
763 |     FragColor = vec4(ColorAlpha.rgb, soft * ColorAlpha.a);
764 | }
765 | )";
766 |
767 | // 4. Clear Blue-Gold Desert Sky and Sun
768 | const char* SKY_VERT = R"(
769 | #version 330 core
770 | layout(location=0) in vec3 aPos;

```

```

771 | out vec3 RayDir;
772 | uniform mat4 invViewProj;
773 |
774 | void main(){
775 |     vec4 farPoint = invViewProj * vec4(aPos.xy, 1.0, 1.0);
776 |     vec4 nearPoint = invViewProj * vec4(aPos.xy, -1.0, 1.0);
777 |     RayDir = farPoint.xyz / farPoint.w - nearPoint.xyz / nearPoint.w;
778 |     gl_Position = vec4(aPos.xy, 0.9999, 1.0);
779 | }
780 | }";
781 |
782 | const char* SKY_FRAG = R"(
783 | #version 330 core
784 | in vec3 RayDir;
785 | out vec4 FragColor;
786 |
787 | uniform vec3 lightDir;
788 | uniform vec3 sunColor;
789 |
790 | void main(){
791 |     vec3 rd = normalize(RayDir);
792 |
793 |     // No cloud noise: golden sand haze fades into a clean blue atmosphere.
794 |     vec3 horizonDust = vec3(0.90, 0.74, 0.47);
795 |     vec3 lowBlue = vec3(0.48, 0.69, 0.88);
796 |     vec3 zenithBlue = vec3(0.10, 0.34, 0.72);
797 |     float elevation = clamp(rd.y, 0.0, 1.0);
798 |     vec3 skyBase = mix(horizonDust, lowBlue, smoothstep(0.0, 0.13, elevation));
799 |     skyBase = mix(skyBase, zenithBlue, pow(elevation, 0.65));
800 |
801 |     // Crisp warm sun disk surrounded by a restrained atmospheric halo.
802 |     float sunDot = max(dot(rd, normalize(lightDir)), 0.0);
803 |     float sunDisk = smoothstep(cos(radians(0.80)), cos(radians(0.58)), sunDot) * 1.9;
804 |     float sunCorona = pow(sunDot, 180.0) * 0.30;
805 |     float sunHaze = pow(sunDot, 12.0) * 0.10;
806 |
807 |     vec3 finalSky = skyBase + (sunDisk + sunCorona + sunHaze) * sunColor;
808 |     FragColor = vec4(finalSky, 1.0);
809 | }
810 | }";
811 |
812 | // =====
813 | // SHADER COMPILATION & MANAGEMENT
814 | // =====
815 | bool g_shaderOkay = true;
816 | unsigned int compileShaderModule(GLenum type, const char* src) {
817 |     unsigned int s = glCreateShader(type);
818 |     glShaderSource(s, 1, &src, nullptr);
819 |     glCompileShader(s);
820 |     int ok; char log[512];
821 |     glGetShaderiv(s, GL_COMPILE_STATUS, &ok);
822 |     if (!ok) {
823 |         glGetShaderInfoLog(s, 512, nullptr, log);
824 |         std::cerr << "[Shader Error]: " << log << std::endl;
825 |         g_shaderOkay = false;
826 |     }
827 |     return s;
828 | }
829 |
830 | unsigned int buildProgram(const char* vs, const char* fs) {
831 |     unsigned int v = compileShaderModule(GL_VERTEX_SHADER, vs);
832 |     unsigned int f = compileShaderModule(GL_FRAGMENT_SHADER, fs);
833 |     unsigned int p = glCreateProgram();
834 |     glAttachShader(p, v);
835 |     glAttachShader(p, f);
836 |     glLinkProgram(p);
837 |     int ok; char log[512];
838 |     glGetProgramiv(p, GL_LINK_STATUS, &ok);
839 |     if (!ok) {
840 |         glGetProgramInfoLog(p, 512, nullptr, log);
841 |         std::cerr << "[Program Error]: " << log << std::endl;
842 |         g_shaderOkay = false;
843 |     }
844 |     glDeleteShader(v);
845 |     glDeleteShader(f);
846 |     return p;
847 | }
848 |
849 | unsigned int g_progScene;

```

```

850 | unsigned int g_progShadow;
851 | unsigned int g_progParticle;
852 | unsigned int g_progSky;
853 |
854 | // Shadow Map Framebuffer
855 | const int SHADOW_RES = 2048;
856 | unsigned int g_shadowFBO = 0;
857 | unsigned int g_shadowTex = 0;
858 |
859 | void setupShadowFramebuffer() {
860 |     glGenFramebuffers(1, &g_shadowFBO);
861 |     glGenTextures(1, &g_shadowTex);
862 |     glBindTexture(GL_TEXTURE_2D, g_shadowTex);
863 |     glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT, SHADOW_RES, SHADOW_RES, 0,
> GL_DEPTH_COMPONENT, GL_FLOAT, nullptr);
864 |     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
865 |     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
866 |     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);
867 |     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_BORDER);
868 |     float border[] = { 1.0f, 1.0f, 1.0f, 1.0f };
869 |     glTexParameterfv(GL_TEXTURE_2D, GL_TEXTURE_BORDER_COLOR, border);
870 |
871 |     glBindFramebuffer(GL_FRAMEBUFFER, g_shadowFBO);
872 |     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, g_shadowTex, 0);
873 |     glDrawBuffer(GL_NONE);
874 |     glReadBuffer(GL_NONE);
875 |     if (glCheckFramebufferStatus(GL_FRAMEBUFFER) != GL_FRAMEBUFFER_COMPLETE) {
876 |         std::cerr << "Shadow framebuffer is incomplete\n";
877 |         g_shaderOkay = false;
878 |     }
879 |     glBindFramebuffer(GL_FRAMEBUFFER, 0);
880 | }
881 |
882 | // Skybox fullscreen quad VAO
883 | unsigned int g_skyVAO = 0;
884 | unsigned int g_skyVBO = 0;
885 | void setupSkyVAO() {
886 |     float verts[] = {
887 |         -1.0f, -1.0f, 0.0f,
888 |         3.0f, -1.0f, 0.0f,
889 |         -1.0f, 3.0f, 0.0f
890 |     };
891 |     glGenVertexArrays(1, &g_skyVAO);
892 |     glGenBuffers(1, &g_skyVBO);
893 |     glBindVertexArray(g_skyVAO);
894 |     glBindBuffer(GL_ARRAY_BUFFER, g_skyVBO);
895 |     glBufferData(GL_ARRAY_BUFFER, sizeof(verts), verts, GL_STATIC_DRAW);
896 |     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
897 |     glEnableVertexAttribArray(0);
898 |     glBindVertexArray(0);
899 | }
900 |
901 | // =====
902 | // RENDERING PIPELINE & DRAW CALL HELPERS
903 | // =====
904 | bool g_isShadowPass = false;
905 |
906 | struct Material {
907 |     glm::vec3 diffuse = glm::vec3(0.6f);
908 |     glm::vec3 specular = glm::vec3(0.3f);
909 |     float shininess = 32.0f;
910 |     glm::vec3 emissive = glm::vec3(0.0f);
911 |     int isSand = 0;
912 | };
913 |
914 | void drawMeshPrimitive(const Mesh& mesh, const glm::mat4& model, const Material& mat) {
915 |     static const GLint shadowModel = glGetUniformLocation(g_progShadow, "model");
916 |     static const GLint shadowSand = glGetUniformLocation(g_progShadow, "isSand");
917 |     static const GLint modelLoc = glGetUniformLocation(g_progScene, "model");
918 |     static const GLint colorLoc = glGetUniformLocation(g_progScene, "objectColor");
919 |     static const GLint specLoc = glGetUniformLocation(g_progScene, "specularColor");
920 |     static const GLint shineLoc = glGetUniformLocation(g_progScene, "shininess");
921 |     static const GLint emitLoc = glGetUniformLocation(g_progScene, "emissiveColor");
922 |     static const GLint sandLoc = glGetUniformLocation(g_progScene, "isSand");
923 |     if (g_isShadowPass) {
924 |         glUniformMatrix4fv(shadowModel, 1, GL_FALSE, glm::value_ptr(model));
925 |         glUniform1i(shadowSand, mat.isSand);
926 |     } else {
927 |         glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

```

```

928 |         glUniform3fv(colorLoc, 1, glm::value_ptr(mat.diffuse));
929 |         glUniform3fv(specLoc, 1, glm::value_ptr(mat.specular));
930 |         glUniform1f(shineLoc, mat.shininess);
931 |         glUniform3fv(emitLoc, 1, glm::value_ptr(mat.emissive));
932 |         glUniform1i(sandLoc, mat.isSand);
933 |     }
934 |     glBindVertexArray(mesh.VAO);
935 |     glDrawElements(mesh.drawMode, mesh.indexCount, GL_UNSIGNED_INT, 0);
936 | }
937 |
938 | // Helper: transform primitive easily
939 | void drawPart(const Mesh& mesh, glm::mat4 root, glm::vec3 t, glm::vec3 rDeg, glm::vec3 s, const
> Material& mat) {
940 |     glm::mat4 m = root;
941 |     m = glm::translate(m, t);
942 |     if (rDeg.y != 0.0f) m = glm::rotate(m, glm::radians(rDeg.y), glm::vec3(0, 1, 0));
943 |     if (rDeg.x != 0.0f) m = glm::rotate(m, glm::radians(rDeg.x), glm::vec3(1, 0, 0));
944 |     if (rDeg.z != 0.0f) m = glm::rotate(m, glm::radians(rDeg.z), glm::vec3(0, 0, 1));
945 |     m = glm::scale(m, s);
946 |     drawMeshPrimitive(mesh, m, mat);
947 | }
948 |
949 | // =====
950 | // COLOR PALETTE (CINEMATIC DUNE)
951 | // =====
952 | const glm::vec3 COL_DUNE_SAND      (0.76f, 0.49f, 0.23f);
953 | const glm::vec3 COL_DUNE_SHADOW    (0.55f, 0.36f, 0.18f);
954 | const glm::vec3 COL_ATREIDES_HULL  (0.22f, 0.23f, 0.22f); // Matte carbon stealth grey
955 | const glm::vec3 COL_ATREIDES_DARK  (0.13f, 0.14f, 0.13f);
956 | const glm::vec3 COL_CANOPY_GLASS   (0.06f, 0.08f, 0.11f); // Dark tinted armored glass
957 | const glm::vec3 COL_JET_NOZZLE     (0.08f, 0.08f, 0.08f);
958 | const glm::vec3 COL_AFTERBURNER     (1.00f, 0.55f, 0.12f); // Fiery jet reheat
959 | const glm::vec3 COL_SPICE_ORANGE    (0.96f, 0.42f, 0.04f); // Melange spice orange
960 | const glm::vec3 COL_SPICE_GLOW      (1.00f, 0.50f, 0.08f);
961 | const glm::vec3 COL_CRAWLER_TRACK   (0.11f, 0.11f, 0.10f);
962 | const glm::vec3 COL_HARV_CHASSIS    (0.42f, 0.34f, 0.25f);
963 | const glm::vec3 COL_HARV_RUST       (0.48f, 0.24f, 0.12f);
964 | const glm::vec3 COL_TANK_STEEL      (0.52f, 0.46f, 0.38f);
965 | const glm::vec3 COL_ROCK_STRATA1    (0.42f, 0.28f, 0.17f);
966 | const glm::vec3 COL_ROCK_STRATA2    (0.32f, 0.20f, 0.11f);
967 |
968 | // =====
969 | // 1. MOVIE-ACCURATE ATREIDES ORNITHOPTER
970 | // 8-blade tandem dragonfly kinematics, faceted insectoid hull, glowing jets
971 | // =====
972 | void drawMovieOrnithopter(glm::mat4 root, float wingPhase, float roll, float pitch, bool boosting) {
973 |     Material matHull;
974 |     matHull.diffuse = COL_ATREIDES_HULL;
975 |     matHull.specular = glm::vec3(0.4f);
976 |     matHull.shininess = 64.0f;
977 |
978 |     Material matDark;
979 |     matDark.diffuse = COL_ATREIDES_DARK;
980 |     matDark.specular = glm::vec3(0.3f);
981 |     matDark.shininess = 32.0f;
982 |
983 |     Material matCanopy;
984 |     matCanopy.diffuse = COL_CANOPY_GLASS;
985 |     matCanopy.specular = glm::vec3(0.9f);
986 |     matCanopy.shininess = 128.0f;
987 |
988 |     Material matFlame;
989 |     matFlame.diffuse = COL_AFTERBURNER;
990 |     matFlame.emissive = boosting ? glm::vec3(1.2f, 0.6f, 0.15f) : glm::vec3(0.6f, 0.25f, 0.05f);
991 |
992 |     Material matJetNozzle;
993 |     matJetNozzle.diffuse = COL_JET_NOZZLE;
994 |     matJetNozzle.specular = glm::vec3(0.5f);
995 |     matJetNozzle.shininess = 64.0f;
996 |
997 |     Material matBlade;
998 |     matBlade.diffuse = glm::vec3(0.18f, 0.19f, 0.18f);
999 |     matBlade.specular = glm::vec3(0.7f);
1000 |     matBlade.shininess = 90.0f;
1001 |
1002 |     // --- Fuselage Hull: Aerodynamic insectoid/dragonfly pod ---
1003 |     // Central pod
1004 |     drawPart(g_meshSphere, root, {0.0f, 0.0f, -0.4f}, {0, 0, 0}, {1.3f, 0.85f, 3.2f}, matHull);
1005 |     // Tapered forward nose

```

```

1006 |     drawPart(g_meshCylinder, root, {0.0f, -0.05f, -2.1f}, {85, 0, 0}, {0.7f, 1.4f, 0.45f}, matHull);
1007 |     // Forward sensor probe needle
1008 |     drawPart(g_meshCylinder, root, {0.0f, -0.1f, -3.1f}, {90, 0, 0}, {0.06f, 1.2f, 0.06f}, matDark);
1009 |
1010 |     // Faceted stealth cockpit canopy (multiple tinted glass facets)
1011 |     drawPart(g_meshSphere, root, {0.0f, 0.32f, -1.2f}, {-10, 0, 0}, {0.85f, 0.65f, 1.6f},
> matCanopy);
1012 |     // Canopy structural spine frame
1013 |     drawPart(g_meshCube, root, {0.0f, 0.58f, -1.1f}, {-10, 0, 0}, {0.12f, 0.18f, 1.5f}, matDark);
1014 |
1015 |     // Twin dorsal intake cowls
1016 |     drawPart(g_meshCylinder, root, { 0.42f, 0.35f, -0.2f}, {82, 0, 0}, {0.35f, 1.2f, 0.28f},
> matHull);
1017 |     drawPart(g_meshCylinder, root, {-0.42f, 0.35f, -0.2f}, {82, 0, 0}, {0.35f, 1.2f, 0.28f},
> matHull);
1018 |
1019 |     // --- Heavy Shoulder Wing-Gimbals (mounting nacelles) ---
1020 |     drawPart(g_meshCylinder, root, { 0.72f, 0.1f, -0.3f}, {0, 0, 90}, {0.45f, 0.65f, 0.45f},
> matDark);
1021 |     drawPart(g_meshCylinder, root, {-0.72f, 0.1f, -0.3f}, {0, 0, 90}, {0.45f, 0.65f, 0.45f},
> matDark);
1022 |     drawPart(g_meshCylinder, root, { 0.68f, 0.05f, 0.5f}, {0, 0, 90}, {0.42f, 0.62f, 0.42f},
> matDark);
1023 |     drawPart(g_meshCylinder, root, {-0.68f, 0.05f, 0.5f}, {0, 0, 90}, {0.42f, 0.62f, 0.42f},
> matDark);
1024 |
1025 |     // --- Twin Jet Turbines & Glowing Heat Reheat Exhaust ---
1026 |     drawPart(g_meshCylinder, root, { 0.52f, -0.15f, 0.9f}, {90, 0, 0}, {0.42f, 1.6f, 0.42f},
> matHull);
1027 |     drawPart(g_meshCylinder, root, {-0.52f, -0.15f, 0.9f}, {90, 0, 0}, {0.42f, 1.6f, 0.42f},
> matHull);
1028 |     // Exhaust nozzles
1029 |     drawPart(g_meshCylinder, root, { 0.52f, -0.15f, 1.72f}, {90, 0, 0}, {0.36f, 0.25f, 0.36f},
> matJetNozzle);
1030 |     drawPart(g_meshCylinder, root, {-0.52f, -0.15f, 1.72f}, {90, 0, 0}, {0.36f, 0.25f, 0.36f},
> matJetNozzle);
1031 |     // Reheat thrust flame
1032 |     float flameLen = boosting ? 1.4f : 0.6f;
1033 |     drawPart(g_meshCylinder, root, { 0.52f, -0.15f, 1.8f + flameLen * 0.5f}, {90, 0, 0}, {0.24f,
> flameLen, 0.24f}, matFlame);
1034 |     drawPart(g_meshCylinder, root, {-0.52f, -0.15f, 1.8f + flameLen * 0.5f}, {90, 0, 0}, {0.24f,
> flameLen, 0.24f}, matFlame);
1035 |
1036 |     // --- Elongated Dragonfly Tail Boom & Canted Stabilizers ---
1037 |     drawPart(g_meshCylinder, root, {0.0f, 0.0f, 2.4f}, {90, 0, 0}, {0.32f, 2.6f, 0.28f}, matHull);
1038 |     drawPart(g_meshCylinder, root, {0.0f, 0.05f, 4.4f}, {90, 0, 0}, {0.18f, 2.2f, 0.16f}, matDark);
1039 |     // Stinger tip
1040 |     drawPart(g_meshCylinder, root, {0.0f, 0.05f, 5.7f}, {90, 0, 0}, {0.08f, 1.1f, 0.08f}, matDark);
1041 |
1042 |     // Canted twin V-Tail rudders
1043 |     drawPart(g_meshCube, root, { 0.5f, 0.55f, 4.2f}, {0, 0, 38}, {0.08f, 0.9f, 0.65f}, matHull);
1044 |     drawPart(g_meshCube, root, {-0.5f, 0.55f, 4.2f}, {0, 0, -38}, {0.08f, 0.9f, 0.65f}, matHull);
1045 |     // Ventral landing skids
1046 |     drawPart(g_meshCube, root, { 0.45f, -0.6f, -0.4f}, {0, 0, 0}, {0.08f, 0.45f, 2.2f}, matDark);
1047 |     drawPart(g_meshCube, root, {-0.45f, -0.6f, -0.4f}, {0, 0, 0}, {0.08f, 0.45f, 2.2f}, matDark);
1048 |
1049 |     // =====
1050 |     // TANDEM DRAGONFLY WINGS (8 BLADES TOTAL - 4 PER SIDE)
1051 |     // Dynamic out-of-phase movie oscillation:
1052 |     // Upper pair & lower pair flutter in anti-phase with realistic pitch twist!
1053 |     // =====
1054 |     float flapAmp = 34.0f; // degrees of flapping
1055 |     float flap1 = std::sin(wingPhase) * flapAmp;
1056 |     float flap2 = std::sin(wingPhase + glm::pi<float>()) * 0.75f) * flapAmp; // anti-phase flutter
1057 |
1058 |     // 4 wings on starboard (+X) and 4 wings on port (-X)
1059 |     struct WingDef {
1060 |         int side;           // +1: right, -1: left
1061 |         glm::vec3 pivot;    // gimbal attachment position
1062 |         float flapAng;      // current flap sweep
1063 |         float sweepBack;    // wing sweep angle in degrees
1064 |         float dihedral;     // resting vertical tilt
1065 |     };
1066 |
1067 |     WingDef wings[8] = {
1068 |         // Forward Upper Pair
1069 |         { 1, { 0.95f, 0.22f, -0.35f }, flap1, 14.0f, 12.0f },
1070 |         { -1, { -0.95f, 0.22f, -0.35f }, flap1, 14.0f, 12.0f },
1071 |         // Forward Lower Pair (fluttering anti-phase)

```



```

1072 |         { 1, { 0.95f, -0.05f, -0.25f }, flap2, 18.0f, -10.0f },
1073 |         { -1, { -0.95f, -0.05f, -0.25f }, flap2, 18.0f, -10.0f },
1074 |         // Aft Upper Pair
1075 |         { 1, { 0.90f, 0.18f, 0.50f }, -flap2, 28.0f, 8.0f },
1076 |         { -1, { -0.90f, 0.18f, 0.50f }, -flap2, 28.0f, 8.0f },
1077 |         // Aft Lower Pair
1078 |         { 1, { 0.90f, -0.08f, 0.60f }, -flap1, 32.0f, -12.0f },
1079 |         { -1, { -0.90f, -0.08f, 0.60f }, -flap1, 32.0f, -12.0f },
1080 |     };
1081 |
1082 |     for (const auto& w : wings) {
1083 |         glm::mat4 wm = root;
1084 |         wm = glm::translate(wm, w.pivot);
1085 |
1086 |         // Flapping rotation around longitudinal/roll axis
1087 |         wm = glm::rotate(wm, glm::radians(w.side * (w.dihedral + w.flapAng)), glm::vec3(0, 0, 1));
1088 |         // Backward sweep
1089 |         wm = glm::rotate(wm, glm::radians(w.side * w.sweepBack), glm::vec3(0, 1, 0));
1090 |         // Pitch twist during stroke
1091 |         float pitchTwist = std::cos(wingPhase) * 8.0f * (float)w.side;
1092 |         wm = glm::rotate(wm, glm::radians(pitchTwist), glm::vec3(1, 0, 0));
1093 |
1094 |         // Mirroring reverses winding; render both sides of thin aerofoils.
1095 |         wm = glm::scale(wm, glm::vec3(w.side * 5.2f, 1.0f, 1.0f));
1096 |
1097 |         glDisable(GL_CULL_FACE);
1098 |         drawMeshPrimitive(g_meshWing, wm, matBlade);
1099 |         glEnable(GL_CULL_FACE);
1100 |     }
1101 | }
1102 |
1103 | // =====
1104 | // 2. MASSIVE INDUSTRIAL SPICE HARVESTER
1105 | // Quad caterpillar treads, rotary crusher scoop, multi-deck refinery
1106 | // =====
1107 | void drawSpiceHarvester(glm::mat4 root, float crawlerAnim) {
1108 |     Material matChassis; matChassis.diffuse = COL_HARV_CHASSIS; matChassis.specular =
1109 | > glm::vec3(0.2f); matChassis.shininess = 20.0f;
1110 |     Material matTread; matTread.diffuse = COL_CRAWLER_TRACK; matTread.specular =
1111 | > glm::vec3(0.15f); matTread.shininess = 10.0f;
1112 |     Material matRust; matRust.diffuse = COL_HARV_RUST; matRust.specular = glm::vec3(0.2f);
1113 | > matRust.shininess = 16.0f;
1114 |     Material matSpice; matSpice.diffuse = COL_SPICE_ORANGE; matSpice.emissive = glm::vec3(0.8f,
1115 | > 0.35f, 0.05f);
1116 |     Material matGlass; matGlass.diffuse = glm::vec3(0.1f, 0.2f, 0.25f); matGlass.specular =
1117 | > glm::vec3(0.8f); matGlass.shininess = 90.0f;
1118 |     Material matLight; matLight.diffuse = glm::vec3(1.0f, 0.95f, 0.7f); matLight.emissive =
1119 | > glm::vec3(1.2f, 1.1f, 0.8f);
1120 |
1121 |     // Main heavy crawler body / hopper
1122 |     drawPart(g_meshCube, root, {0.0f, 2.6f, 0.0f}, {0, 0, 0}, {9.5f, 3.8f, 6.2f}, matChassis);
1123 |     // Upper refinery deck
1124 |     drawPart(g_meshCube, root, {0.0f, 4.8f, -0.4f}, {0, 0, 0}, {8.2f, 1.2f, 5.0f}, matChassis);
1125 |
1126 |     // --- Quad Giant Caterpillar Track Units (Front-L, Front-R, Rear-L, Rear-R) ---
1127 |     float trackOffsetsX[2] = { -4.6f, 4.6f };
1128 |     float trackOffsetsZ[2] = { -2.4f, 2.4f };
1129 |     for (float ox : trackOffsetsX) {
1130 |         for (float oz : trackOffsetsZ) {
1131 |             glm::mat4 tm = glm::translate(root, glm::vec3(ox, 0.95f, oz));
1132 |             // Track pontoon housing
1133 |             drawPart(g_meshCube, tm, {0, 0, 0}, {0, 0, 0}, {1.8f, 1.7f, 3.8f}, matTread);
1134 |             // Drive sprockets & road wheels
1135 |             for (float wz = -1.4f; wz <= 1.4f; wz += 0.93f) {
1136 |                 drawPart(g_meshCylinder, tm, {ox > 0 ? 0.95f : -0.95f, -0.2f, wz}, {0, 0, 90},
1137 | > {1.2f, 0.35f, 1.2f}, matChassis);
1138 |                 drawPart(g_meshCube, tm, {ox > 0 ? 1.15f : -1.15f, -0.2f, wz}, {crawlerAnim * 80, 0,
1139 | > 0}, {0.04f, 0.95f, 0.12f}, matRust);
1140 |             }
1141 |             for (int pad = 0; pad < 10; ++pad) {
1142 |                 float a = pad * glm::two_pi<float>() / 10 + crawlerAnim * 0.7f;
1143 |                 float angle = -glm::degrees(std::atan2(0.85f * std::cos(a), -1.8f * std::sin(a)));
1144 |                 drawPart(g_meshCube, tm, {0, 0.85f * std::sin(a), 1.8f * std::cos(a)}, {angle, 0,
1145 | > 0}, {1.9f, 0.12f, 0.38f}, matRust);
1146 |             }
1147 |         }
1148 |     }
1149 |
1150 |     // --- Forward Harvesting Cutter Drum / Scoop ---

```

```

1142 | // Massive rotating cylinder with teeth
1143 | drawPart(g_meshCylinder, root, {0.0f, 1.1f, -4.2f}, {crawlerAnim * 70, 0, 90}, {2.2f, 8.8f,
> 2.2f}, matRust);
1144 | Material matDoor; matDoor.diffuse = {0.06f, 0.07f, 0.06f};
1145 | drawPart(g_meshCube, root, {4.78f, 1.8f, 1.2f}, {0, 0, 0}, {0.12f, 2.4f, 1.5f}, matDoor);
1146 | drawPart(g_meshCube, root, {5.5f, 0.45f, 1.2f}, {0, 0, -16}, {2, 0.15f, 1.7f}, matChassis);
1147 | // Glowing spice intake suction scoop inside drum
1148 | drawPart(g_meshCube, root, {0.0f, 0.9f, -4.8f}, {15, 0, 0}, {8.2f, 0.6f, 1.4f}, matSpice);
1149 |
1150 | // Side intake separator funnels
1151 | drawPart(g_meshCylinder, root, {4.8f, 2.2f, -1.8f}, {0, 0, -22}, {1.2f, 2.4f, 1.2f}, matRust);
1152 | drawPart(g_meshCylinder, root, {-4.8f, 2.2f, -1.8f}, {0, 0, 22}, {1.2f, 2.4f, 1.2f}, matRust);
1153 |
1154 | // --- Overhead Bridge Observation Deck ---
1155 | drawPart(g_meshCube, root, {2.8f, 5.8f, -2.2f}, {0, 0, 0}, {2.6f, 1.4f, 2.0f}, matChassis);
1156 | // Command bridge panoramic viewports
1157 | drawPart(g_meshCube, root, {2.8f, 5.9f, -3.25f}, {0, 0, 0}, {2.4f, 0.65f, 0.2f}, matGlass);
1158 | // Forward floodlights illuminating desert sand
1159 | drawPart(g_meshSphere, root, {1.8f, 5.2f, -3.3f}, {0, 0, 0}, {0.35f, 0.35f, 0.35f}, matLight);
1160 | drawPart(g_meshSphere, root, {3.8f, 5.2f, -3.3f}, {0, 0, 0}, {0.35f, 0.35f, 0.35f}, matLight);
1161 |
1162 | // --- Industrial Exhaust Stacks & Spice Smoke Vents ---
1163 | drawPart(g_meshCylinder, root, {1.6f, 6.2f, 1.8f}, {0, 0, 0}, {0.75f, 2.6f, 0.75f},
> matChassis);
1164 | drawPart(g_meshCylinder, root, {-1.6f, 6.2f, 1.8f}, {0, 0, 0}, {0.75f, 2.6f, 0.75f},
> matChassis);
1165 | // Stack glowing spice vapor vent rims
1166 | drawPart(g_meshCylinder, root, {1.6f, 7.55f, 1.8f}, {0, 0, 0}, {0.85f, 0.2f, 0.85f}, matSpice);
1167 | drawPart(g_meshCylinder, root, {-1.6f, 7.55f, 1.8f}, {0, 0, 0}, {0.85f, 0.2f, 0.85f}, matSpice);
1168 | }
1169 |
1170 | // =====
1171 | // 3. MOVIE-REALISTIC SPICE PRESSURE TANKS
1172 | // Heavy pressure cylinder, hemispherical heads, diagonal transport cradle, glowing spice level
1173 | // =====
1174 | void drawMovieSpiceTank(glm::mat4 root) {
1175 |     Material matCradle; matCradle.diffuse = glm::vec3(0.24f, 0.23f, 0.22f); matCradle.specular =
> glm::vec3(0.3f); matCradle.shininess = 32.0f;
1176 |     Material matVessel; matVessel.diffuse = COL_TANK_STEEL; matVessel.specular = glm::vec3(0.6f);
> matVessel.shininess = 64.0f;
1177 |     Material matSeam; matSeam.diffuse = glm::vec3(0.18f, 0.17f, 0.16f); matSeam.specular =
> glm::vec3(0.2f); matSeam.shininess = 20.0f;
1178 |     Material matSpice; matSpice.diffuse = COL_SPICE_ORANGE; matSpice.emissive = glm::vec3(1.2f,
> 0.55f, 0.08f);
1179 |
1180 | // --- Heavy Structural Transport Cradle with Diagonal Trusses ---
1181 | // Ground skid beams
1182 | drawPart(g_meshCube, root, {1.35f, 0.18f, 0.0f}, {0, 0, 0}, {0.35f, 0.35f, 4.4f}, matCradle);
1183 | drawPart(g_meshCube, root, {-1.35f, 0.18f, 0.0f}, {0, 0, 0}, {0.35f, 0.35f, 4.4f}, matCradle);
1184 | // Cross supports
1185 | drawPart(g_meshCube, root, {0.0f, 0.25f, 1.6f}, {0, 0, 0}, {3.0f, 0.25f, 0.35f}, matCradle);
1186 | drawPart(g_meshCube, root, {0.0f, 0.25f, -1.6f}, {0, 0, 0}, {3.0f, 0.25f, 0.35f}, matCradle);
1187 | // Cradle vertical stanchions & saddles
1188 | for (float z : { -1.3f, 1.3f }) {
1189 |     drawPart(g_meshCube, root, {1.3f, 1.1f, z}, {0, 0, -22}, {0.28f, 1.8f, 0.35f}, matCradle);
1190 |     drawPart(g_meshCube, root, {-1.3f, 1.1f, z}, {0, 0, 22}, {0.28f, 1.8f, 0.35f}, matCradle);
1191 | }
1192 |
1193 | // --- Horizontal Cylindrical Pressure Vessel ---
1194 | // Central cylinder (diameter: 2.2, length: 3.2 along Z)
1195 | drawPart(g_meshCylinder, root, {0.0f, 1.55f, 0.0f}, {90, 0, 0}, {2.2f, 3.2f, 2.2f}, matVessel);
1196 | // Hemispherical domed heads (front and rear caps)
1197 | drawPart(g_meshSphere, root, {0.0f, 1.55f, 1.6f}, {0, 0, 0}, {2.2f, 2.2f, 1.3f}, matVessel);
1198 | drawPart(g_meshSphere, root, {0.0f, 1.55f, -1.6f}, {0, 0, 0}, {2.2f, 2.2f, 1.3f}, matVessel);
1199 |
1200 | // Reinforcing stiffener weld bands
1201 | drawPart(g_meshCylinder, root, {0.0f, 1.55f, 0.8f}, {90, 0, 0}, {2.28f, 0.18f, 2.28f},
> matSeam);
1202 | drawPart(g_meshCylinder, root, {0.0f, 1.55f, -0.8f}, {90, 0, 0}, {2.28f, 0.18f, 2.28f},
> matSeam);
1203 |
1204 | // Top sealed loading hatch & relief valve
1205 | drawPart(g_meshCylinder, root, {0.0f, 2.72f, 0.0f}, {0, 0, 0}, {0.65f, 0.25f, 0.65f}, matSeam);
1206 | drawPart(g_meshSphere, root, {0.0f, 2.92f, 0.0f}, {0, 0, 0}, {0.4f, 0.25f, 0.4f}, matSpice);
1207 |
1208 | // Glowing Vertical Spice Level Sight Tube (illuminated Melange spice indicator)
1209 | drawPart(g_meshCube, root, {1.15f, 1.55f, 0.0f}, {0, 0, 0}, {0.08f, 1.5f, 0.18f}, matSpice);
1210 | }
1211 |

```

```

1212 | // =====
1213 | // 4. REALISTIC DESERT ROCK YARDANG FORMATIONS
1214 | // Wind-carved stratified sandstone crags with sediment bands
1215 | // =====
1216 | void drawDesertRockFormation(glm::mat4 root, int variant) {
1217 |     Material matStrata1; matStrata1.diffuse = COL_ROCK_STRATA1; matStrata1.specular =
> glm::vec3(0.15f); matStrata1.shininess = 16.0f;
1218 |     Material matStrata2; matStrata2.diffuse = COL_ROCK_STRATA2; matStrata2.specular =
> glm::vec3(0.12f); matStrata2.shininess = 12.0f;
1219 |
1220 |     matStrata1.isSand = matStrata2.isSand = 2;
1221 |     drawPart(g_meshRock, root, {0, 2.2f, 0}, {8, float(variant * 19), -5}, {8, 7, 6}, matStrata1);
1222 |     drawPart(g_meshRock, root, {1, 5, -1}, {0, 55, 10}, {5, 5, 4}, matStrata2);
1223 |     drawPart(g_meshRock, root, {-3, 0.8f, 2}, {5, -40, 12}, {4, 3, 3}, matStrata2);
1224 |     drawPart(g_meshRock, root, {3.2f, 0.3f, 2.2f}, {15, 70, 20}, {2.2f, 1.6f, 2.1f}, matStrata1);
1225 | }
1226 |
1227 | #include "arrakis_worm.h"
1228 | #include "arrakis_hud.h"
1229 |
1230 | void drawRescueWorld(float time) {
1231 |     Material cyan; cyan.diffuse = {0.05f, 0.5f, 0.55f}; cyan.emissive = {0.03f, 0.6f, 0.65f};
1232 |     Material amber; amber.diffuse = {0.9f, 0.5f, 0.1f}; amber.emissive = {0.6f, 0.25f, 0.03f};
1233 |     Material dark; dark.diffuse = {0.16f, 0.19f, 0.19f};
1234 |     Material cloth; cloth.diffuse = {0.30f, 0.24f, 0.17f}; cloth.specular = {0.03f, 0.03f, 0.03f};
1235 |     Material visor; visor.diffuse = {0.08f, 0.20f, 0.27f}; visor.specular = {0.8f, 0.8f, 0.8f};
> visor.shininess = 90;
1236 |     glm::mat4 base = glm::translate(glm::mat4(1), g_mission.base);
1237 |     drawPart(g_meshCylinder, base, {0, 0.25f, 0}, {0, 0, 0}, {24, 0.5f, 24}, dark);
1238 |     drawPart(g_meshRing, base, {0, 0.55f, 0}, {0, 0, 0}, {11, 1, 11}, cyan);
1239 |     drawPart(g_meshCube, base, {0, 0.56f, 0}, {0, 0, 0}, {1.2f, 0.05f, 10}, cyan);
1240 |     drawPart(g_meshCube, base, {0, 0.56f, 0}, {0, 0, 0}, {10, 0.05f, 1.2f}, cyan);
1241 |     for (int i = 0; i < 4; ++i) {
1242 |         float a = i * glm::half_pi<float>();
1243 |         drawPart(g_meshCylinder, base, {std::cos(a) * 13, 2, std::sin(a) * 13}, {0, 0, 0}, {0.2f, 4,
> 0.2f}, dark);
1244 |         drawPart(g_meshSphere, base, {std::cos(a) * 13, 4.1f, std::sin(a) * 13}, {0, 0, 0}, {0.65f,
> 0.65f, 0.65f}, cyan);
1245 |     }
1246 |     for (int groupIndex = 0; groupIndex < static_cast<int>(g_mission.groups.size()); ++groupIndex) {
1247 |         const auto& group = g_mission.groups[groupIndex];
1248 |         if (!group.released) continue;
1249 |         int waiting = 0;
1250 |         for (const auto& c : g_mission.crew)
1251 |             if (c.group == groupIndex && (c.state == CrewState::Running || c.state ==
> CrewState::Waiting)) ++waiting;
1252 |         if (waiting == 0) continue;
1253 |         glm::vec3 p = group.center;
1254 |         p.y = getDuneHeight(p.x, p.z) + 0.2f;
1255 |         glm::mat4 ring = glm::translate(glm::mat4(1), p);
1256 |         glm::vec3 normal = getDuneNormal(p.x, p.z), axis = glm::cross(glm::vec3(0, 1, 0), normal);
1257 |         if (glm::length(axis) > 0.001f) ring = glm::rotate(ring, std::acos(glm::clamp(normal.y,
> -1.0f, 1.0f)), glm::normalize(axis));
1258 |         drawPart(g_meshRing, ring, {0, 0, 0}, {0, 0, 0}, {4.5f, 1, 4.5f}, amber);
1259 |         drawPart(g_meshCylinder, glm::translate(glm::mat4(1), p), {4, 2, 0}, {0, 0, 0}, {0.12f, 4,
> 0.12f}, dark);
1260 |         drawPart(g_meshSphere, glm::translate(glm::mat4(1), p), {4, 4.1f, 0}, {0, 0, 0}, {0.5f,
> 0.5f, 0.5f}, amber);
1261 |     }
1262 |     for (int i = 0; i < int(g_mission.crew.size()); ++i) {
1263 |         const auto& c = g_mission.crew[i];
1264 |         if (c.state != CrewState::Waiting && c.state != CrewState::Running) continue;
1265 |         glm::vec3 crewPosition = c.pos;
1266 |         if (i == g_mission.target && g_mission.pickup > 0)
1267 |             crewPosition.y += (g_ornPos.y - crewPosition.y - 1.8f) * glm::smoothstep(0.0f, 1.0f,
> g_mission.pickup);
1268 |         glm::mat4 root = glm::translate(glm::mat4(1), crewPosition);
1269 |         float stride = c.state == CrewState::Running ? std::sin(time * 11 + i) * 24 : std::sin(time
> * 2 + i) * 4;
1270 |         drawPart(g_meshSphere, root, {0, 1.05f, 0}, {0, 0, 0}, {0.55f, 0.85f, 0.4f}, cloth);
1271 |         drawPart(g_meshSphere, root, {0, 1.65f, 0}, {0, 0, 0}, {0.43f, 0.46f, 0.43f}, cloth);
1272 |         drawPart(g_meshSphere, root, {0, 1.67f, 0.17f}, {0, 0, 0}, {0.32f, 0.15f, 0.10f}, visor);
1273 |         for (int side : {-1, 1}) {
1274 |             drawPart(g_meshCylinder, root, {side * 0.14f, 0.4f, 0}, {side * stride, 0, 0}, {0.17f,
> 0.8f, 0.17f}, cloth);
1275 |             drawPart(g_meshCylinder, root, {side * 0.34f, 1.1f, 0}, {-side * stride, 0, side *
> 14.0f}, {0.15f, 0.7f, 0.15f}, cloth);
1276 |         }
1277 |         drawPart(g_meshSphere, root, {0, 2.25f, 0}, {0, 0, 0}, {0.18f, 0.18f, 0.18f}, cyan);

```

```

1278 |     }
1279 |     if (g_mission.pickup > 0 && g_mission.target >= 0) {
1280 |         glm::vec3 end = g_mission.crew[g_mission.target].pos + glm::vec3(0, 1, 0);
1281 |         end.y += (g_ornPos.y - end.y - 0.8f) * glm::smoothstep(0.0f, 1.0f, g_mission.pickup);
1282 |         glm::vec3 start = g_ornPos - glm::vec3(0, 0.7f, 0);
1283 |         glm::vec3 dir = end - start;
1284 |         float length = glm::length(dir);
1285 |         glm::mat4 rope = glm::translate(glm::mat4(1), (start + end) * 0.5f);
1286 |         glm::vec3 axis = glm::cross(glm::vec3(0, 1, 0), dir / length);
1287 |         if (glm::length(axis) > 0.001f) rope = glm::rotate(rope, std::acos(glm::clamp(dir.y /
> length, -1.0f, 1.0f)), glm::normalize(axis));
1288 |         else if (dir.y < 0) rope = glm::rotate(rope, glm::pi<float>(), glm::vec3(1, 0, 0));
1289 |         drawPart(g_meshCylinder, rope, {0, 0, 0}, {0, 0, 0}, {0.055f, length, 0.055f}, cyan);
1290 |     }
1291 |     drawSandworm(wormPosition(), wormAttackTime(), time, g_mission.wormYaw,
1292 |                 glm::clamp(g_mission.elapsed / g_mission.breachAt, 0.0f, 1.0f));
1293 | }
1294 |
1295 | // =====
1296 | // MASTER SCENE DRAW CALL
1297 | // Executed twice per frame: once for the shadow map, once for the main view
1298 | // =====
1299 | void drawFullScene(float curTime) {
1300 |     // 1. Vast Continuous Sand Dune Landscape
1301 |     Material matSand;
1302 |     matSand.diffuse = COL_DUNE_SAND;
1303 |     matSand.specular = glm::vec3(0.25f);
1304 |     matSand.shininess = 16.0f;
1305 |     matSand.isSand = 1;
1306 |     drawMeshPrimitive(g_meshTerrain, glm::mat4(1.0f), matSand);
1307 |
1308 |     // 2. Movie-Accurate Atreides Ornithopter
1309 |     if (!g_mission.planeLost) {
1310 |         glm::mat4 om = glm::translate(glm::mat4(1.0f), g_ornPos);
1311 |         om = glm::rotate(om, glm::radians(g_ornYaw), glm::vec3(0, 1, 0));
1312 |         om = glm::rotate(om, glm::radians(g_ornPitch), glm::vec3(1, 0, 0));
1313 |         om = glm::rotate(om, glm::radians(g_ornRoll), glm::vec3(0, 0, 1));
1314 |         drawMovieOrnithopter(om, g_wingPhase, g_ornRoll, g_ornPitch, g_isBoosting);
1315 |     }
1316 |
1317 |     // 3. Massive Industrial Spice Harvester (hugging the dune surface!)
1318 |     if (wormAttackTime() < 12) {
1319 |         float hy = getDuneHeight(g_harvX, g_harvZ);
1320 |         float sink = glm::smoothstep(6.0f, 12.0f, wormAttackTime());
1321 |         hy -= sink * 20;
1322 |         glm::mat4 hm = glm::translate(glm::mat4(1.0f), glm::vec3(g_harvX, hy, g_harvZ));
1323 |         hm = glm::rotate(hm, glm::radians(g_mission.harvesterYaw), glm::vec3(0, 1, 0));
1324 |         // Apply dune slope in the crawler's own forward/right axes.
1325 |         glm::vec3 hnrm = getDuneNormal(g_harvX, g_harvZ);
1326 |         glm::vec3 forward = harvesterForward(g_mission.harvesterYaw);
1327 |         glm::vec3 right = glm::cross(forward, glm::vec3(0, 1, 0));
1328 |         hm = glm::rotate(hm, std::atan2(-glm::dot(hnrm, forward), hnrm.y), glm::vec3(1, 0, 0));
1329 |         hm = glm::rotate(hm, std::atan2(-glm::dot(hnrm, right), hnrm.y), glm::vec3(0, 0, 1));
1330 |         hm = glm::rotate(hm, sink * 0.8f, glm::vec3(1, 0, 0));
1331 |         drawSpiceHarvester(hm, g_mission.harvesterTravel);
1332 |     }
1333 |
1334 |     // 4. Dynamic Spice Tanks (deployed behind Harvester onto dune surface)
1335 |     for (const auto& tank : g_tanks) {
1336 |         if (wormAttackTime() > 12) continue;
1337 |         float ty = getDuneHeight(tank.pos.x, tank.pos.z);
1338 |         glm::mat4 tm = glm::translate(glm::mat4(1.0f), glm::vec3(tank.pos.x, ty, tank.pos.z));
1339 |         tm = glm::rotate(tm, glm::radians(tank.yaw), glm::vec3(0, 1, 0));
1340 |         tm = glm::scale(tm, glm::vec3(tank.scale));
1341 |         drawMovieSpiceTank(tm);
1342 |     }
1343 |
1344 |     // 5. Desert Rock Yardang Crags (at natural elevated points across the desert)
1345 |     for (int i = 0; i < 8; ++i) {
1346 |         float rx = g_rockSites[i].x;
1347 |         float rz = g_rockSites[i].y;
1348 |         float ry = getDuneHeight(rx, rz) - 0.5f;
1349 |         glm::mat4 rm = glm::translate(glm::mat4(1.0f), glm::vec3(rx, ry, rz));
1350 |         rm = glm::rotate(rm, glm::radians((float)(i * 45)), glm::vec3(0, 1, 0));
1351 |         drawDesertRockFormation(rm, i);
1352 |     }
1353 |     drawRescueWorld(curTime);
1354 | }
1355 |

```

```

1356 | // =====
1357 | // GLFW CALLBACKS
1358 | // =====
1359 | void framebuffer_size_callback(GLFWwindow* win, int w, int h) {
1360 |     if (w > 0 && h > 0) {
1361 |         g_scrW = w;
1362 |         g_scrH = h;
1363 |         glViewport(0, 0, w, h);
1364 |     }
1365 | }
1366 |
1367 | void centerMouse(GLFWwindow* win) {
1368 |     int width, height;
1369 |     glfwGetWindowSize(win, &width, &height);
1370 |     g_mouseHover = glm::vec2(0);
1371 |     g_mouseTurnRate = 0;
1372 |     glfwSetCursorPos(win, width * 0.5, height * 0.5);
1373 | }
1374 |
1375 | void cursor_position_callback(GLFWwindow* win, double x, double y) {
1376 |     if (g_mission.phase != MissionPhase::Flying || g_mission.paused
1377 |         || !glfwGetWindowAttrib(win, GLFW_FOCUSED)) {
1378 |         g_mouseHover = glm::vec2(0);
1379 |         g_mouseTurnRate = 0;
1380 |         return;
1381 |     }
1382 |     // Cursor coordinates are logical window pixels, not framebuffer pixels.
1383 |     int width, height;
1384 |     glfwGetWindowSize(win, &width, &height);
1385 |     setMouseHover(x, y, width, height);
1386 | }
1387 |
1388 | void key_callback(GLFWwindow* win, int key, int, int action, int) {
1389 |     if (key >= 0 && key < 1024) {
1390 |         if (action == GLFW_PRESS) g_keys[key] = true;
1391 |         if (action == GLFW_RELEASE) g_keys[key] = false;
1392 |     }
1393 |     if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS) {
1394 |         glfwSetWindowShouldClose(win, true);
1395 |     }
1396 |     if (key == GLFW_KEY_F && action == GLFW_PRESS) {
1397 |         g_wireframe = !g_wireframe;
1398 |         glPolygonMode(GL_FRONT_AND_BACK, g_wireframe ? GL_LINE : GL_FILL);
1399 |     }
1400 |     if (key == GLFW_KEY_C && action == GLFW_PRESS) {
1401 |         g_camMode = (g_camMode + 1) % 3;
1402 |     }
1403 |     if (key == GLFW_KEY_ENTER && action == GLFW_PRESS) {
1404 |         if (g_mission.phase == MissionPhase::Title) resetMission(true);
1405 |         else if (g_mission.phase == MissionPhase::Debrief) resetMission(true, true);
1406 |         centerMouse(win);
1407 |     }
1408 |     if (key == GLFW_KEY_R && action == GLFW_PRESS) { resetMission(true); centerMouse(win); }
1409 |     if (key == GLFW_KEY_M && action == GLFW_PRESS) {
1410 |         g_audio.toggleMute();
1411 |     }
1412 |     if (key == GLFW_KEY_P && action == GLFW_PRESS && g_mission.phase == MissionPhase::Flying) {
1413 |         g_mission.paused = !g_mission.paused;
1414 |         centerMouse(win);
1415 |     }
1416 |     if (key == GLFW_KEY_F11 && action == GLFW_PRESS) {
1417 |         g_isFullscreen = !g_isFullscreen;
1418 |         GLFWmonitor* primary = glfwGetPrimaryMonitor();
1419 |         const GLFWvidmode* mode = glfwGetVideoMode(primary);
1420 |         if (g_isFullscreen) {
1421 |             glfwSetWindowMonitor(win, primary, 0, 0, mode->width, mode->height, mode->refreshRate);
1422 |         } else {
1423 |             glfwSetWindowMonitor(win, nullptr, 100, 100, 1280, 720, 0);
1424 |         }
1425 |         centerMouse(win);
1426 |     }
1427 | }
1428 |
1429 | // =====
1430 | // MAIN ENTRY POINT
1431 | // =====
1432 | bool captureScreenshot(const std::string& path) {
1433 |     std::vector<unsigned char> pixels(static_cast<size_t>(g_scrW) * g_scrH * 4);
1434 |     glReadPixels(0, 0, g_scrW, g_scrH, GL_BGRA, GL_UNSIGNED_BYTE, pixels.data());

```

```

1435 |     std::ofstream out(path, std::ios::binary);
1436 |     if (!out) return false;
1437 |     auto word = [&](uint32_t value, int bytes) {
1438 |         for (int i = 0; i < bytes; ++i) out.put(static_cast<char>((value >> (i * 8)) & 255));
1439 |     };
1440 |     out.put('B'); out.put('M'); word(static_cast<uint32_t>(54 + pixels.size()), 4);
1441 |     word(0, 4); word(54, 4); word(40, 4); word(g_scrW, 4); word(g_scrH, 4);
1442 |     word(1, 2); word(32, 2); word(0, 4); word(static_cast<uint32_t>(pixels.size()), 4);
1443 |     word(0, 4); word(0, 4); word(0, 4); word(0, 4);
1444 |     out.write(reinterpret_cast<const char*>(pixels.data()),
> static_cast<std::streamsize>(pixels.size()));
1445 |     return out.good();
1446 | }
1447 |
1448 | int main(int argc, char** argv) {
1449 |     if (argc > 1 && std::string(argv[1]) == "--test-game") return runMissionTests();
1450 |     bool smokeTest = argc > 1 && std::string(argv[1]) == "--smoke-test";
1451 |     bool breachTest = argc > 1 && std::string(argv[1]) == "--smoke-breach";
1452 |     bool mouseTest = argc > 1 && std::string(argv[1]) == "--smoke-mouse";
1453 |     bool pursuitTest = argc > 1 && std::string(argv[1]) == "--smoke-pursuit";
1454 |     bool renderTest = smokeTest || breachTest || mouseTest || pursuitTest;
1455 |     if (!glfwInit()) {
1456 |         std::cerr << "Failed to initialize GLFW" << std::endl;
1457 |         return -1;
1458 |     }
1459 |
1460 |     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
1461 |     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
1462 |     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
1463 |
1464 |     GLFWmonitor* monitor = glfwGetPrimaryMonitor();
1465 |     const GLFWvidmode* mode = glfwGetVideoMode(monitor);
1466 |
1467 |     glfwWindowHint(GLFW_RED_BITS, mode->redBits);
1468 |     glfwWindowHint(GLFW_GREEN_BITS, mode->greenBits);
1469 |     glfwWindowHint(GLFW_BLUE_BITS, mode->blueBits);
1470 |     glfwWindowHint(GLFW_REFRESH_RATE, mode->refreshRate);
1471 |
1472 |     glfwWindowHint(GLFW_SAMPLES, 4);
1473 |     // Windowed startup leaves the desktop accessible; F11 enters fullscreen.
1474 |     g_window = glfwCreateWindow(g_scrW, g_scrH,
1475 |         "ARRAKIS | Harvester Down - Desert Rescue",
1476 |         nullptr, nullptr);
1477 |
1478 |     if (!g_window) {
1479 |         std::cerr << "Window creation failed, falling back to windowed mode..." << std::endl;
1480 |         g_isFullscreen = false;
1481 |         g_scrW = 1280; g_scrH = 720;
1482 |         g_window = glfwCreateWindow(g_scrW, g_scrH, "ARRAKIS", nullptr, nullptr);
1483 |         if (!g_window) { glfwTerminate(); return -1; }
1484 |     }
1485 |
1486 |     glfwMakeContextCurrent(g_window);
1487 |     glfwGetFramebufferSize(g_window, &g_scrW, &g_scrH);
1488 |     glfwSetFramebufferSizeCallback(g_window, framebuffer_size_callback);
1489 |     glfwSetKeyCallback(g_window, key_callback);
1490 |     glfwSetCursorPosCallback(g_window, cursor_position_callback);
1491 |     glfwSetCursorEnterCallback(g_window, [](GLFWwindow*, int) {
1492 |         g_mouseHover = glm::vec2(0);
1493 |         g_mouseTurnRate = 0;
1494 |     });
1495 |     glfwSetWindowSizeCallback(g_window, [](GLFWwindow* win, int, int) { centerMouse(win); });
1496 |     glfwSetWindowFocusCallback(g_window, [](GLFWwindow*, int focused) {
1497 |         if (!focused) {
1498 |             std::fill(std::begin(g_keys), std::end(g_keys), false);
1499 |             g_mouseHover = glm::vec2(0);
1500 |             g_mouseTurnRate = 0;
1501 |             if (g_mission.phase == MissionPhase::Flying) g_mission.paused = true;
1502 |         }
1503 |     });
1504 |     glfwSwapInterval(1); // Enable VSync for buttery smooth frames
1505 |
1506 |     if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
1507 |         std::cerr << "Failed to initialize GLAD" << std::endl;
1508 |         glfwTerminate();
1509 |         return -1;
1510 |     }
1511 |
1512 |     g_audio.init();

```



```

1513 |
1514 |     std::cout << "=====\n";
1515 |     std::cout << " ARRAKIS / Harvester Down / Desert Rescue\n";
1516 |     std::cout << " GPU: " << glGetString(GL_RENDERER) << "\n";
1517 |     std::cout << " OpenGL Version: " << glGetString(GL_VERSION) << "\n";
1518 |     std::cout << " Resolution: " << g_scrW << "x" << g_scrH << "\n";
1519 |     std::cout << "=====\n";
1520 |
1521 |     glEnable(GL_DEPTH_TEST);
1522 |     glEnable(GL_MULTISAMPLE);
1523 |     glDepthFunc(GL_LEQUAL);
1524 |     glEnable(GL_CULL_FACE);
1525 |     glCullFace(GL_BACK);
1526 |
1527 |     // Build procedural geometry
1528 |     g_meshCube = createCube();
1529 |     g_meshCylinder = createCylinder(24);
1530 |     g_meshSphere = createSphere(16, 24);
1531 |     g_meshWing = createWingBlade();
1532 |     g_meshTerrain = createDuneTerrain(260, 260, 900.0f);
1533 |     g_meshQuad = createQuad();
1534 |     g_meshRing = createRing();
1535 |     g_meshRock = createErodedRock();
1536 |     initSandwormMeshes();
1537 |
1538 |     // Compile shader programs
1539 |     g_progScene = buildProgram(SCENE_VERT, SCENE_FRAG);
1540 |     g_progShadow = buildProgram(SHADOW_VERT, SHADOW_FRAG);
1541 |     g_progParticle = buildProgram(PARTICLE_VERT, PARTICLE_FRAG);
1542 |     g_progSky = buildProgram(SKY_VERT, SKY_FRAG);
1543 |
1544 |     setupShadowFramebuffer();
1545 |     setupSkyVAO();
1546 |     initRescueHud();
1547 |     if (!g_shaderOkay || !arrakis_hud_detail::resources().program) {
1548 |         glfwDestroyWindow(g_window); glfwTerminate(); return 1;
1549 |     }
1550 |     initParticles();
1551 |     glGenBuffers(1, &g_particleInstanceVBO);
1552 |     glBindVertexArray(g_meshQuad.VAO);
1553 |     glBindBuffer(GL_ARRAY_BUFFER, g_particleInstanceVBO);
1554 |     glBufferData(GL_ARRAY_BUFFER, MAX_PARTICLES * sizeof(ParticleInstance), nullptr,
> GL_STREAM_DRAW);
1555 |     for (int i = 0; i < 2; ++i) {
1556 |         glEnableVertexAttribArray(3 + i);
1557 |         glVertexAttribPointer(3 + i, 4, GL_FLOAT, GL_FALSE, sizeof(ParticleInstance),
> reinterpret_cast<void*>(i * sizeof(glm::vec4)));
1558 |         glVertexAttribDivisor(3 + i, 1);
1559 |     }
1560 |     glBindVertexArray(0);
1561 |     g_particleInstances.reserve(MAX_PARTICLES);
1562 |     resetMission(renderTest);
1563 |     centerMouse(g_window);
1564 |     if (breachTest || pursuitTest) {
1565 |         float targetTime = breachTest ? g_mission.breachAt + 7 : 110.0f;
1566 |         int steps = static_cast<int>(std::ceil(targetTime * 120));
1567 |         for (int i = 0; i < steps; ++i) updateMission(targetTime / steps);
1568 |         g_ornPos = harvesterPosition() + glm::vec3(35, 16, 40);
1569 |         g_mission.altitude = 16;
1570 |     }
1571 |
1572 |     // Environment Lighting Constants
1573 |     glm::vec3 sunDir = glm::normalize(glm::vec3(-0.35f, 0.12f, -1.0f));
1574 |     glm::vec3 sunColor(1.15f, 1.04f, 0.85f);
1575 |     glm::vec3 ambientColor(0.34f, 0.38f, 0.44f);
1576 |     glm::vec3 fogColor(0.90f, 0.74f, 0.47f);
1577 |     float fogDensity = 0.0022f;
1578 |
1579 |     float prevTime = (float)glfwGetTime();
1580 |
1581 |     // Camera damping state
1582 |     glm::vec3 camPosSmoothed = g_ornPos + glm::vec3(0, 26, 42);
1583 |     glm::vec3 camTargetSmoothed = g_ornPos;
1584 |     float simulationTime = g_mission.elapsed;
1585 |     float fps = 60;
1586 |     int frameCount = 0;
1587 |     int cameraRevision = g_missionRevision;
1588 |     bool renderFailed = false;
1589 |     double smokeStarted = glfwGetTime();

```



```

1590 |
1591 | // =====
1592 | // MAIN INTERACTIVE RENDER LOOP
1593 | // =====
1594 | while (!glfwWindowShouldClose(g_window)) {
1595 |     if (glfwGetWindowAttrib(g_window, GLFW_ICONIFIED)) {
1596 |         glfwWaitEvents();
1597 |         prevTime = static_cast<float>(glfwGetTime());
1598 |         continue;
1599 |     }
1600 |     float curTime = (float)glfwGetTime();
1601 |     float dt = curTime - prevTime;
1602 |     prevTime = curTime;
1603 |     if (dt > 0.1f) dt = 0.1f; // Clamp delta time to avoid physics explosion
1604 |     float realDt = dt;
1605 |
1606 |     glfwPollEvents();
1607 |     if (renderTest) {
1608 |         dt = 1.0f / 60; g_mission.paused = false;
1609 |         g_mouseHover = glm::vec2(0);
1610 |         if (mouseTest) {
1611 |             int width, height;
1612 |             glfwGetWindowSize(g_window, &width, &height);
1613 |             setMouseHover(width * (frameCount < 60 ? 0.85 : 0.15), height * 0.5, width, height);
1614 |             g_keys[GLFW_KEY_W] = true;
1615 |             g_camMode = (frameCount / 40) % 3;
1616 |         }
1617 |     }
1618 |     fps = glm::mix(fps, 1.0f / std::max(realDt, 0.0001f), 0.03f);
1619 |     // Bounded substeps keep winching and flight stable at low frame rates.
1620 |     if (!g_mission.paused) {
1621 |         int steps = std::max(1, static_cast<int>(std::ceil(dt / (1.0f / 120))));
1622 |         for (int i = 0; i < steps; ++i) updateMission(dt / steps);
1623 |         // Let the swallowing/retreat animation finish behind the results UI.
1624 |         if (g_mission.phase == MissionPhase::Debrief) g_mission.elapsed += dt;
1625 |         simulationTime += dt;
1626 |         if (g_mission.phase == MissionPhase::Title) g_wingPhase += 18 * dt;
1627 |         updateParticles(dt, g_ornPos, harvesterPosition());
1628 |     }
1629 |     curTime = simulationTime;
1630 |
1631 |     // -----
1632 |     // 4. CAMERA SYSTEM (CINEMATIC CHASE / COCKPIT / OVERHEAD)
1633 |     // -----
1634 |     if (cameraRevision != g_missionRevision) {
1635 |         cameraRevision = g_missionRevision;
1636 |         camPosSmoothed = g_ornPos + glm::vec3(0, 14, 38);
1637 |         camTargetSmoothed = g_ornPos;
1638 |     }
1639 |     if (!g_mission.paused) {
1640 |         if (g_keys[GLFW_KEY_UP]) g_camOrbitPitch = std::min(g_camOrbitPitch + 40.0f * dt,
> 45.0f);
1641 |         if (g_keys[GLFW_KEY_DOWN]) g_camOrbitPitch = std::max(g_camOrbitPitch - 40.0f * dt,
> -10.0f);
1642 |     }
1643 |
1644 |     glm::vec3 camTarget = g_ornPos;
1645 |     glm::vec3 camOffset = flightCameraOffset(g_camMode);
1646 |     glm::vec3 camDesiredPos = g_ornPos + camOffset;
1647 |     if (breachTest) {
1648 |         camTarget = harvesterPosition() + glm::vec3(0, 14, 0);
1649 |         camDesiredPos = harvesterPosition() + glm::vec3(65, 48, 80);
1650 |     } else if (pursuitTest) {
1651 |         camTarget = (harvesterPosition() + wormPosition()) * 0.5f + glm::vec3(0, 6, 0);
1652 |         camDesiredPos = camTarget + glm::vec3(65, 30, 155);
1653 |     }
1654 |
1655 |     // Camera terrain collision safety
1656 |     float camGroundY = getDuneHeight(camDesiredPos.x, camDesiredPos.z) + 1.5f;
1657 |     if (camDesiredPos.y < camGroundY) camDesiredPos.y = camGroundY;
1658 |
1659 |     // Smooth translation/elevation, but share the aircraft's exact heading.
1660 |     // Smoothing world-space camera X/Z independently would expose its side.
1661 |     camPosSmoothed.y = glm::mix(camPosSmoothed.y, camDesiredPos.y, 1.0f - std::exp(-9.0f * dt));
1662 |     camTargetSmoothed = glm::mix(camTargetSmoothed, camTarget, 1.0f - std::exp(-12.0f * dt));
1663 |     camPosSmoothed.x = camTargetSmoothed.x + camOffset.x;
1664 |     camPosSmoothed.z = camTargetSmoothed.z + camOffset.z;
1665 |     if (breachTest || pursuitTest) {
1666 |         camPosSmoothed.x = camDesiredPos.x;

```

```

1667 |         camPosSmoothed.z = camDesiredPos.z;
1668 |     }
1669 |     camPosSmoothed.y = std::max(camPosSmoothed.y, getDuneHeight(camPosSmoothed.x,
> camPosSmoothed.z) + 2);
1670 |     if (mouseTest) {
1671 |         glm::vec3 viewForward = camTargetSmoothed - camPosSmoothed;
1672 |         viewForward.y = 0;
1673 |         if (glm::dot(glm::normalize(viewForward), flightForward()) < 0.9999f) {
1674 |             std::cerr << "Camera heading diverged from aircraft\n";
1675 |             renderFailed = true; glfwSetWindowShouldClose(g_window, true);
1676 |         }
1677 |     }
1678 |
1679 |     glm::mat4 view = glm::lookAt(camPosSmoothed, camTargetSmoothed, glm::vec3(0, 1, 0));
1680 |     float aspect = (float)g_scrW / (float)g_scrH;
1681 |     glm::mat4 proj = glm::perspective(glm::radians(58.0f), aspect, 0.4f, 1200.0f);
1682 |
1683 |     // 3D Spatial Audio Update
1684 |     glm::vec3 nearestCrewAudio = nearestCrewPosition();
1685 |     float nearestCrewDistAudio = horizontalDistance(g_ornPos, nearestCrewAudio);
1686 |     g_audio.update(dt,
1687 |         camPosSmoothed,
1688 |         camTargetSmoothed,
1689 |         g_ornPos,
1690 |         g_ornSpeed,
1691 |         g_isBoosting,
1692 |         harvesterPosition(),
1693 |         wormPosition(),
1694 |         wormGap(),
1695 |         wormAttackTime(),
1696 |         g_mission.rescued,
1697 |         g_mission.aboard,
1698 |         g_mission.pickup,
1699 |         nearestCrewAudio,
1700 |         nearestCrewDistAudio,
1701 |         g_mission.base,
1702 |         g_mission.phase == MissionPhase::Flying,
1703 |         g_mission.paused);
1704 |
1705 |     // -----
1706 |     // 5. SHADOW MAP PASS (PASS 1)
1707 |     // -----
1708 |     g_isShadowPass = true;
1709 |     glViewport(0, 0, SHADOW_RES, SHADOW_RES);
1710 |     glBindFramebuffer(GL_FRAMEBUFFER, g_shadowFBO);
1711 |     glClear(GL_DEPTH_BUFFER_BIT);
1712 |
1713 |     // Follow player area with orthographic shadow frustum
1714 |     glm::vec3 shadowCenter = g_ornPos;
1715 |     shadowCenter.y = getDuneHeight(shadowCenter.x, shadowCenter.z);
1716 |     float shadowBoxSize = 90.0f;
1717 |     glm::mat4 lightProj = glm::ortho(-shadowBoxSize, shadowBoxSize, -shadowBoxSize,
> shadowBoxSize, 2.0f, 320.0f);
1718 |     glm::vec3 lightCamPos = shadowCenter + sunDir * 120.0f;
1719 |     glm::mat4 lightView = glm::lookAt(lightCamPos, shadowCenter, glm::vec3(0, 1, 0));
1720 |     glm::mat4 lightSpaceMatrix = lightProj * lightView;
1721 |
1722 |     glUseProgram(g_progShadow);
1723 |     glUniformMatrix4fv(glGetUniformLocation(g_progShadow, "lightSpaceMatrix"), 1, GL_FALSE,
> glm::value_ptr(lightSpaceMatrix));
1724 |     glm::vec3 wormCenter = wormPosition();
1725 |     float collapse = glm::smoothstep(7.0f, 14.0f, wormAttackTime());
1726 |     glUniform3fv(glGetUniformLocation(g_progShadow, "wormCenter"), 1,
> glm::value_ptr(wormCenter));
1727 |     glUniform1f(glGetUniformLocation(g_progShadow, "collapse"), collapse);
1728 |
1729 |     // Slope-scale bias via polygon offset to eliminate shadow acne
1730 |     glEnable(GL_POLYGON_OFFSET_FILL);
1731 |     glPolygonOffset(2.5f, 4.0f);
1732 |     drawFullScene(curTime);
1733 |     glDisable(GL_POLYGON_OFFSET_FILL);
1734 |
1735 |     glBindFramebuffer(GL_FRAMEBUFFER, 0);
1736 |
1737 |     // -----
1738 |     // 6. MAIN SCENE RENDER PASS (PASS 2)
1739 |     // -----
1740 |     g_isShadowPass = false;
1741 |     glViewport(0, 0, g_scrW, g_scrH);

```

```

1742 |         glClearColor(fogColor.r, fogColor.g, fogColor.b, 1.0f);
1743 |         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
1744 |
1745 |         // --- Step A: Atmospheric Desert Skybox ---
1746 |         glDisable(GL_CULL_FACE);
1747 |         glDepthMask(GL_FALSE);
1748 |         glUseProgram(g_progSky);
1749 |         glUniform3fv(glGetUniformLocation(g_progSky, "lightDir"), 1, glm::value_ptr(sunDir));
1750 |         glUniform3fv(glGetUniformLocation(g_progSky, "sunColor"), 1, glm::value_ptr(sunColor));
1751 |         glm::mat4 invViewProj = glm::inverse(proj * view);
1752 |         glUniformMatrix4fv(glGetUniformLocation(g_progSky, "invViewProj"), 1, GL_FALSE,
> glm::value_ptr(invViewProj));
1753 |
1754 |         glBindVertexArray(g_skyVAO);
1755 |         glDrawArrays(GL_TRIANGLES, 0, 3);
1756 |         glDepthMask(GL_TRUE);
1757 |         glEnable(GL_CULL_FACE);
1758 |
1759 |         // --- Step B: 3D Scene Geometry with Blinn-Phong + Shadows + Fog ---
1760 |         glUseProgram(g_progScene);
1761 |         glUniformMatrix4fv(glGetUniformLocation(g_progScene, "view"), 1, GL_FALSE,
> glm::value_ptr(view));
1762 |         glUniformMatrix4fv(glGetUniformLocation(g_progScene, "projection"), 1, GL_FALSE,
> glm::value_ptr(proj));
1763 |         glUniformMatrix4fv(glGetUniformLocation(g_progScene, "lightSpaceMatrix"), 1, GL_FALSE,
> glm::value_ptr(lightSpaceMatrix));
1764 |
1765 |         glUniform3fv(glGetUniformLocation(g_progScene, "lightDir"), 1, glm::value_ptr(sunDir));
1766 |         glUniform3fv(glGetUniformLocation(g_progScene, "sunColor"), 1, glm::value_ptr(sunColor));
1767 |         glUniform3fv(glGetUniformLocation(g_progScene, "ambientColor"), 1,
> glm::value_ptr(ambientColor));
1768 |         glUniform3fv(glGetUniformLocation(g_progScene, "viewPos"), 1,
> glm::value_ptr(camPosSmoothed));
1769 |         glUniform3fv(glGetUniformLocation(g_progScene, "fogColor"), 1, glm::value_ptr(fogColor));
1770 |         glUniform1f(glGetUniformLocation(g_progScene, "fogDensity"), fogDensity);
1771 |         glUniform3fv(glGetUniformLocation(g_progScene, "wormCenter"), 1,
> glm::value_ptr(wormCenter));
1772 |         glUniform1f(glGetUniformLocation(g_progScene, "collapse"), collapse);
1773 |
1774 |         // Bind shadow depth texture to texture slot 0
1775 |         glActiveTexture(GL_TEXTURE0);
1776 |         glBindTexture(GL_TEXTURE_2D, g_shadowTex);
1777 |         glUniform1i(glGetUniformLocation(g_progScene, "shadowMap"), 0);
1778 |
1779 |         drawFullScene(curTime);
1780 |
1781 |         // --- Step C: Wind & Blowing Sand Particle System ---
1782 |         glEnable(GL_BLEND);
1783 |         glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
1784 |         glDepthMask(GL_FALSE); // Don't write depth for soft dust billboards
1785 |         glDisable(GL_CULL_FACE);
1786 |
1787 |         glUseProgram(g_progParticle);
1788 |         glUniformMatrix4fv(glGetUniformLocation(g_progParticle, "view"), 1, GL_FALSE,
> glm::value_ptr(view));
1789 |         glUniformMatrix4fv(glGetUniformLocation(g_progParticle, "projection"), 1, GL_FALSE,
> glm::value_ptr(proj));
1790 |
1791 |         // Extract camera right & up vectors for billboard orientation
1792 |         glm::vec3 camRight(view[0][0], view[1][0], view[2][0]);
1793 |         glm::vec3 camUp(view[0][1], view[1][1], view[2][1]);
1794 |         glUniform3fv(glGetUniformLocation(g_progParticle, "camRight"), 1, glm::value_ptr(camRight));
1795 |         glUniform3fv(glGetUniformLocation(g_progParticle, "camUp"), 1, glm::value_ptr(camUp));
1796 |
1797 |         g_particleInstances.clear();
1798 |         for (const auto& p : g_particles) {
1799 |             float fade = glm::clamp(p.life / 0.5f, 0.0f, 1.0f) * glm::clamp((p.maxLife - p.life) /
> 0.3f, 0.0f, 1.0f);
1800 |             g_particleInstances.push_back({glm::vec4(p.pos, p.size), glm::vec4(COL_DUNE_SAND,
> p.alpha * fade)});
1801 |         }
1802 |         std::sort(g_particleInstances.begin(), g_particleInstances.end(), [&](const
> ParticleInstance& a, const ParticleInstance& b) {
1803 |             glm::vec3 da = glm::vec3(a.posSize) - camPosSmoothed, db = glm::vec3(b.posSize) -
> camPosSmoothed;
1804 |             return glm::dot(da, da) > glm::dot(db, db);
1805 |         });
1806 |         glBindBuffer(GL_ARRAY_BUFFER, g_particleInstanceVBO);
1807 |         glBufferData(GL_ARRAY_BUFFER, g_particleInstances.size() * sizeof(ParticleInstance),

```

```

> g_particleInstances.data(), GL_STREAM_DRAW);
1808 |         glBindVertexArray(g_meshQuad.VAO);
1809 |         glDrawElementsInstanced(GL_TRIANGLES, g_meshQuad.indexCount, GL_UNSIGNED_INT, nullptr,
> static_cast<GLsizei>(g_particleInstances.size()));
1810 |
1811 |         glDepthMask(GL_TRUE);
1812 |         glDisable(GL_BLEND);
1813 |         glEnable(GL_CULL_FACE);
1814 |
1815 |         RescueHudState hud;
1816 |         hud.width = g_scrW; hud.height = g_scrH;
1817 |         hud.rescued = g_mission.rescued; hud.aboard = g_mission.aboard;
1818 |         hud.lost = g_mission.lost; hud.best = g_mission.best; hud.wave = g_mission.wave;
1819 |         hud.muted = g_audio.muted;
1820 |         hud.inside = 0;
1821 |         for (const auto& c : g_mission.crew) {
1822 |             if (c.state == CrewState::Inside) ++hud.inside;
1823 |             if (c.state == CrewState::Running || c.state == CrewState::Waiting) {
1824 |                 ++hud.waiting;
1825 |                 hud.crewMarkers.push_back({c.pos.x, c.pos.z});
1826 |             }
1827 |         }
1828 |         hud.missionDuration = g_mission.breachAt;
1829 |         hud.pursuit = glm::clamp(g_mission.elapsed / g_mission.breachAt, 0.0f, 1.0f);
1830 |         hud.wormDistance = wormGap();
1831 |         hud.extractionRemaining = std::max(0.0f, 60 - wormAttackTime());
1832 |         hud.releasedGroups = 0;
1833 |         for (const auto& group : g_mission.groups) if (group.released) ++hud.releasedGroups;
1834 |         hud.remaining = std::max(0.0f, g_mission.breachAt - g_mission.elapsed);
1835 |         hud.altitude = g_ornPos.y - getDuneHeight(g_ornPos.x, g_ornPos.z);
1836 |         hud.speed = g_ornSpeed; hud.boost = g_mission.boost;
1837 |         hud.pickupProgress = g_mission.pickup; hud.unloadProgress = g_mission.unload;
1838 |         hud.fps = fps; hud.prompt = g_mission.prompt;
1839 |         hud.heading = g_ornYaw;
1840 |         hud.title = g_mission.phase == MissionPhase::Title;
1841 |         hud.ended = g_mission.phase == MissionPhase::Debrief; hud.paused = g_mission.paused;
1842 |         hud.player = {g_ornPos.x, g_ornPos.z}; hud.base = {g_mission.base.x, g_mission.base.z};
1843 |         hud.harvester = {g_harvX, g_harvZ}; hud.worm = {wormCenter.x, wormCenter.z};
1844 |         glm::vec3 pickup = nearestCrewPosition();
1845 |         hud.pickup = {pickup.x, pickup.z};
1846 |         hud.attackTime = wormAttackTime();
1847 |         glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
1848 |         drawRescueHud(hud);
1849 |         glPolygonMode(GL_FRONT_AND_BACK, g_wireframe ? GL_LINE : GL_FILL);
1850 |
1851 |         // -----
1852 |         // 7. BUFFER SWAP & EVENT POLLING
1853 |         // -----
1854 |         if (renderTest) {
1855 |             GLenum error = glGetError();
1856 |             if (error != GL_NO_ERROR) {
1857 |                 std::cerr << "OpenGL error in frame " << frameCount << ": " << error << "\n";
1858 |                 renderFailed = true; glfwSetWindowShouldClose(g_window, true);
1859 |             }
1860 |             if (frameCount == 119 && argc > 2 && !captureScreenshot(argv[2])) {
1861 |                 std::cerr << "Screenshot write failed\n"; renderFailed = true;
1862 |             }
1863 |         }
1864 |         glfwSwapBuffers(g_window);
1865 |         if (renderTest && ++frameCount >= 120) glfwSetWindowShouldClose(g_window, true);
1866 |     }
1867 |
1868 |     // Cleanup resources
1869 |     glDeleteProgram(g_progScene);
1870 |     glDeleteProgram(g_progShadow);
1871 |     glDeleteProgram(g_progParticle);
1872 |     glDeleteProgram(g_progSky);
1873 |     glDeleteFramebuffers(1, &g_shadowFBO);
1874 |     glDeleteTextures(1, &g_shadowTex);
1875 |     glDeleteBuffers(1, &g_particleInstanceVBO);
1876 |     glDeleteBuffers(1, &g_skyVBO);
1877 |     glDeleteVertexArrays(1, &g_skyVAO);
1878 |     cleanupSandwormMeshes();
1879 |     cleanupRescueHud();
1880 |     for (Mesh* m : {&g_meshCube, &g_meshCylinder, &g_meshSphere, &g_meshWing, &g_meshTerrain,
> &g_meshQuad, &g_meshRing, &g_meshRock}) {
1881 |         glDeleteVertexArrays(1, &m->VAO);
1882 |         glDeleteBuffers(1, &m->VBO);
1883 |         glDeleteBuffers(1, &m->EBO);

```

```
1884 |     }
1885 |
1886 |     g_audio.shutdown();
1887 |     glfwDestroyWindow(g_window);
1888 |     if (renderTest) std::cout << "Render smoke test: " << (renderFailed ? "FAILED" : "PASS")
1889 |         << " / " << frameCount << " frames / " << (glfwGetTime() - smokeStarted) << " seconds\n";
1890 |     glfwTerminate();
1891 |     return renderFailed ? 1 : 0;
1892 | }
```

Source Reference: src/arrakis_game.h

```

1 | #pragma once
2 |
3 | enum class MissionPhase { Title, Flying, Debrief };
4 | enum class CrewState { Inside, Running, Waiting, Aboard, Safe, Lost };
5 | struct Evacuee {
6 |     glm::vec3 pos{0};
7 |     glm::vec3 destination{0};
8 |     CrewState state = CrewState::Inside;
9 |     float release = 0;
10 |    int group = 0, slot = 0;
11 | };
12 | struct EvacuationGroup {
13 |     int count = 0;
14 |     float release = 0;
15 |     glm::vec3 center{0};
16 |     bool released = false;
17 | };
18 | struct RescueMission {
19 |     MissionPhase phase = MissionPhase::Title;
20 |     bool paused = false;
21 |     bool planeLost = false;
22 |     bool boostLocked = false;
23 |     int wave = 1, rescued = 0, aboard = 0, lost = 0, best = 0;
24 |     float elapsed = 0, breachAt = 210, altitude = 8, boost = 1;
25 |     float harvesterYaw = 0, harvesterTravel = 0, wormYaw = 90;
26 |     float pickup = 0, unload = 0;
27 |     int target = -1;
28 |     std::string prompt = "";
29 |     glm::vec3 base{55, 0, 55};
30 |     std::vector<Evacuee> crew;
31 |     std::vector<EvacuationGroup> groups;
32 | };
33 | RescueMission g_mission;
34 | int g_missionRevision = 0;
35 | glm::vec2 g_mouseHover{0};
36 | float g_mouseTurnRate = 0;
37 | constexpr int RESCUE_CAPACITY = 8;
38 | const glm::vec2 g_rockSites[] = {{45, -65}, {-55, 40}, {90, 30}, {-90, -70},
39 |                                   {15, 95}, {-25, -115}, {120, -25}, {-130, 85}};
40 |
41 | float horizontalDistance(glm::vec3 a, glm::vec3 b) {
42 |     return glm::length(glm::vec2(a.x - b.x, a.z - b.z));
43 | }
44 | glm::vec3 harvesterPosition() {
45 |     return {g_harvX, getDuneHeight(g_harvX, g_harvZ), g_harvZ};
46 | }
47 | glm::vec3 harvesterRoute(float time) {
48 |     float x = -40 + 0.62f * time + 15 * std::sin(time * 0.045f);
49 |     float z = -55 + 22 * std::sin(time * 0.032f) + 9 * std::sin(time * 0.071f);
50 |     return {x, getDuneHeight(x, z), z};
51 | }
52 | float harvesterRouteYaw(float time) {
53 |     float dx = 0.62f + 0.675f * std::cos(time * 0.045f);
54 |     float dz = 0.704f * std::cos(time * 0.032f) + 0.639f * std::cos(time * 0.071f);
55 |     return glm::degrees(std::atan2(-dx, -dz));
56 | }
57 | glm::vec3 harvesterForward(float yaw) {
58 |     float angle = glm::radians(yaw);
59 |     return {-std::sin(angle), 0, -std::cos(angle)};
60 | }
61 | void updateHarvester(float elapsed) {
62 |     float time = glm::clamp(elapsed, 0.0f, g_mission.breachAt);
63 |     glm::vec3 previous = harvesterPosition(), next = harvesterRoute(time);
64 |     g_mission.harvesterTravel += horizontalDistance(previous, next);
65 |     g_harvX = next.x; g_harvZ = next.z;
66 |     g_mission.harvesterYaw = harvesterRouteYaw(time);
67 | }
68 | float wormGap() {
69 |     float progress = glm::clamp(g_mission.elapsed / g_mission.breachAt, 0.0f, 1.0f);
70 |     return glm::mix(220.0f, 18.0f, progress);
71 | }
72 | glm::vec3 wormPosition() {
73 |     glm::vec3 p = harvesterPosition();
74 |     p.x -= wormGap();
75 |     p.y = getDuneHeight(p.x, p.z);
76 |     return p;

```



```

77 | }
78 | float wormAttackTime() { return g_mission.elapsed - g_mission.breachAt; }
79 |
80 | glm::vec3 nearestCrewPosition() {
81 |     float nearest = 10000;
82 |     glm::vec3 point = harvesterPosition();
83 |     for (const auto& c : g_mission.crew) {
84 |         if (c.state != CrewState::Waiting && c.state != CrewState::Running) continue;
85 |         float distance = horizontalDistance(g_ornPos, c.pos);
86 |         if (distance < nearest) { nearest = distance; point = c.pos; }
87 |     }
88 |     return point;
89 | }
90 |
91 | void setMouseHover(double x, double y, int width, int height) {
92 |     g_mouseHover = glm::vec2(0);
93 |     if (width <= 0 || height <= 0 || !std::isfinite(x) || !std::isfinite(y)
94 |         || x < 0 || y < 0 || x > width || y > height) return;
95 |     auto deadzone = [](float value) {
96 |         return std::copysign(std::max(0.0f, (std::abs(value) - 0.12f) / 0.88f), value);
97 |     };
98 |     g_mouseHover = {deadzone(static_cast<float>(2 * x / width - 1)),
99 |                     deadzone(static_cast<float>(2 * y / height - 1))};
100 | }
101 |
102 | glm::vec3 flightForward() {
103 |     float yaw = glm::radians(g_ornYaw);
104 |     return {-std::sin(yaw), 0, -std::cos(yaw)};
105 | }
106 |
107 | float flightCameraPitch() {
108 |     return glm::clamp(12 - g_mouseHover.y * 22 + g_camOrbitPitch, 6.0f, 65.0f);
109 | }
110 |
111 | glm::vec3 flightCameraOffset(int mode) {
112 |     glm::vec3 behind = -flightForward();
113 |     if (mode == 2) return behind * 26.0f + glm::vec3(0, 100, 0);
114 |     float distance = mode == 1 ? 20.0f : 40.0f;
115 |     float pitch = glm::radians(flightCameraPitch());
116 |     return behind * (distance * std::cos(pitch))
117 |         + glm::vec3(0, distance * std::sin(pitch) + (mode == 1 ? 2.2f : 0.0f), 0);
118 | }
119 |
120 | void resetMission(bool start, bool next = false) {
121 |     ++g_missionRevision;
122 |     g_audio.resetMission();
123 |     int best = g_mission.best;
124 |     int wave = next ? g_mission.wave + 1 : g_mission.wave;
125 |     g_mission = RescueMission{};
126 |     g_mission.best = best;
127 |     g_mission.wave = wave;
128 |     g_mission.breachAt = std::max(150.0f, 210.0f - (wave - 1) * 10.0f);
129 |     g_mission.phase = start ? MissionPhase::Flying : MissionPhase::Title;
130 |     glm::vec3 harvester = harvesterRoute(0);
131 |     g_harvX = harvester.x; g_harvZ = harvester.z;
132 |     g_mission.harvesterYaw = harvesterRouteYaw(0);
133 |     g_mission.base.y = getDuneHeight(g_mission.base.x, g_mission.base.z);
134 |     g_ornPos = g_mission.base + glm::vec3(0, 8, 0);
135 |     g_ornVel = glm::vec3(0);
136 |     g_ornYaw = 0;
137 |     g_mouseHover = glm::vec2(0);
138 |     g_mouseTurnRate = 0;
139 |     g_ornSpeed = g_ornPitch = g_ornRoll = 0;
140 |     g_camOrbitPitch = 0;
141 |     int remaining = 36;
142 |     while (remaining > 0) {
143 |         int group = static_cast<int>(g_mission.groups.size());
144 |         int count = std::min(remaining, 1 + group % 4);
145 |         float release = 6.0f + group * 7.0f;
146 |         g_mission.groups.push_back({count, release, harvester, false});
147 |         for (int slot = 0; slot < count; ++slot) {
148 |             Evacuee c;
149 |             c.release = release; c.group = group; c.slot = slot;
150 |             c.pos = c.destination = harvester;
151 |             g_mission.crew.push_back(c);
152 |         }
153 |         remaining -= count;
154 |     }
155 |     g_tanks.clear();

```

```

156 |     g_tanks.push_back({{-12, 0, -26}, 20, 1});
157 |     g_tanks.push_back({{-19, 0, -32}, -15, 1});
158 |     g_tanks.push_back({{-26, 0, -23}, 10, 1});
159 | }
160 |
161 | void finishMission() {
162 |     g_mission.phase = MissionPhase::Debrief;
163 |     g_mission.best = std::max(g_mission.best, g_mission.rescued);
164 |     g_mission.pickup = g_mission.unload = 0;
165 |     g_ornVel = glm::vec3(0);
166 | }
167 |
168 | void updateMission(float dt) {
169 |     auto& m = g_mission;
170 |     if (m.phase != MissionPhase::Flying || m.paused) return;
171 |     m.elapsed += dt;
172 |     updateHarvester(m.elapsed);
173 |     float attack = wormAttackTime();
174 |     const bool winch = g_keys[GLFW_KEY_SPACE];
175 |     float steer = glm::clamp(g_mouseHover.x + float(g_keys[GLFW_KEY_RIGHT]) -
> float(g_keys[GLFW_KEY_LEFT]), -1.0f, 1.0f);
176 |     float turnRate = -steer * 80;
177 |     float turnDecay = std::exp(-dt * 8);
178 |     // Integrate the smoothed turn analytically so steering does not depend on FPS.
179 |     g_ornYaw = std::remainder(g_ornYaw + turnRate * dt
180 |         + (g_mouseTurnRate - turnRate) * (1 - turnDecay) / 8, 360.0f);
181 |     g_mouseTurnRate = glm::mix(turnRate, g_mouseTurnRate, turnDecay);
182 |     g_ornRoll = glm::mix(g_ornRoll, g_mouseTurnRate * 0.28f, 1 - std::exp(-dt * 6));
183 |     glm::vec3 forward = flightForward();
184 |     glm::vec3 right = glm::cross(forward, glm::vec3(0, 1, 0));
185 |     glm::vec3 input = right * (float(g_keys[GLFW_KEY_D]) - float(g_keys[GLFW_KEY_A]));
186 |     + forward * (float(g_keys[GLFW_KEY_W]) - float(g_keys[GLFW_KEY_S]));
187 |     if (glm::length(input) > 0) input = glm::normalize(input);
188 |     if (!g_keys[GLFW_KEY_LEFT_SHIFT]) m.boostLocked = false;
189 |     if (m.boost <= 0.04f) m.boostLocked = true;
190 |     g_isBoosting = g_keys[GLFW_KEY_LEFT_SHIFT] && !m.boostLocked && !winch && glm::length(input) >
> 0;
191 |     m.boost = glm::clamp(m.boost + dt * (g_isBoosting ? -0.22f : 0.14f), 0.0f, 1.0f);
192 |     glm::vec3 desired = input * (g_isBoosting ? 48.0f : 28.0f);
193 |     if (winch) desired *= 0.10f;
194 |     g_ornVel = glm::mix(g_ornVel, desired, 1.0f - std::exp(-dt * (winch ? 12.0f : 5.5f)));
195 |     g_ornPos += g_ornVel * dt;
196 |     g_ornPos.x = glm::clamp(g_ornPos.x, -160.0f, 160.0f);
197 |     g_ornPos.z = glm::clamp(g_ornPos.z, -160.0f, 160.0f);
198 |     g_ornSpeed = glm::length(g_ornVel);
199 |     m.altitude = glm::clamp(m.altitude + (float(g_keys[GLFW_KEY_Q]) - float(g_keys[GLFW_KEY_E])) *
> 15 * dt, 4.0f, 55.0f);
200 |     if (winch) m.altitude = glm::mix(m.altitude, 6.0f, 1.0f - std::exp(-dt * 4));
201 |     float ground = getDuneHeight(g_ornPos.x, g_ornPos.z);
202 |     float clearance = m.altitude;
203 |     for (const auto& rock : g_rockSites) {
204 |         float distance = glm::distance(glm::vec2(g_ornPos.x, g_ornPos.z), rock);
205 |         float avoid = 1 - glm::smoothstep(8.0f, 17.0f, distance);
206 |         clearance = std::max(clearance, 13 * avoid);
207 |     }
208 |     if (attack < 11 && std::abs(g_ornPos.x - g_harvX) < 8 && std::abs(g_ornPos.z - g_harvZ) < 7)
209 |         clearance = std::max(clearance, 12.0f);
210 |     g_ornPos.y = glm::mix(g_ornPos.y, ground + clearance, 1.0f - std::exp(-dt * 8));
211 |     g_ornPos.y = std::max(g_ornPos.y, ground + 3.0f);
212 |     g_ornPitch = glm::mix(g_ornPitch, -g_ornSpeed * 0.3f, 1.0f - std::exp(-dt * 6));
213 |     g_wingPhase += (g_isBoosting ? 36 : 23) * dt;
214 |
215 |     // Drop each batch at its scheduled world-space point, independent of FPS.
216 |     for (int i = 0; i < static_cast<int>(m.groups.size()); ++i) {
217 |         auto& group = m.groups[i];
218 |         if (group.released || m.elapsed < group.release) continue;
219 |         glm::vec3 origin = harvesterRoute(group.release);
220 |         glm::vec3 forward = harvesterForward(harvesterRouteYaw(group.release));
221 |         glm::vec3 right = glm::cross(forward, glm::vec3(0, 1, 0));
222 |         glm::vec3 exit = origin + right * 6.1f - forward * 1.2f;
223 |         group.center = exit + right * 14.0f;
224 |         group.center.y = getDuneHeight(group.center.x, group.center.z);
225 |         group.released = true;
226 |         for (auto& c : m.crew) if (c.group == i && c.state == CrewState::Inside) {
227 |             c.state = CrewState::Running;
228 |             c.pos = exit;
229 |             c.destination = group.center + forward * (1.7f * (c.slot - (group.count - 1) * 0.5f))
230 |                 + right * (1.3f * (c.slot % 2));
231 |             c.destination.y = getDuneHeight(c.destination.x, c.destination.z);

```

```

232 |     }
233 | }
234 | int waiting = 0;
235 | glm::vec3 worm = wormPosition();
236 | glm::vec3 harvForward = harvesterForward(m.harvesterYaw);
237 | glm::vec3 harvRight = glm::cross(harvForward, glm::vec3(0, 1, 0));
238 | for (auto& c : m.crew) {
239 |     if (c.state == CrewState::Inside)
240 |         c.pos = harvesterPosition() + harvRight * 6.1f - harvForward * 1.2f;
241 |     if (c.state == CrewState::Running) {
242 |         glm::vec3 d = c.destination - c.pos; d.y = 0;
243 |         float distance = glm::length(d);
244 |         float walkDt = std::min(dt, std::max(0.0f, m.elapsed - c.release));
245 |         if (distance <= walkDt * 3.8f + 0.00001f) {
246 |             c.pos = c.destination;
247 |             c.state = CrewState::Waiting;
248 |         } else c.pos += d / distance * (walkDt * 3.8f);
249 |     }
250 |     c.pos.y = getDuneHeight(c.pos.x, c.pos.z);
251 |     bool outside = c.state == CrewState::Running || c.state == CrewState::Waiting;
252 |     float dangerRadius = attack < 0 ? 16.0f : (attack >= 12 ? 44.0f : 32.0f);
253 |     bool activeWorm = (attack < 0 && m.elapsed > 8) || (attack > 3 && attack < 18);
254 |     if ((c.state == CrewState::Inside && attack >= 12)
255 |         || (outside && activeWorm && horizontalDistance(c.pos, worm) < dangerRadius)) {
256 |         c.state = CrewState::Lost; ++m.lost;
257 |     }
258 |     if (c.state == CrewState::Waiting || c.state == CrewState::Running || c.state ==
> CrewState::Inside) ++waiting;
259 | }
260 |
261 | m.prompt = attack < 0 ? "HARVESTER FLEEING / RESCUE AMBER GROUPS" : "WORM BREACH / CLEAR THE
> HARVESTER";
262 | bool low = g_ornPos.y - ground <= 11.0f;
263 | bool steady = g_ornSpeed < 6;
264 | bool atBase = horizontalDistance(g_ornPos, m.base) < 14;
265 | if (m.aboard == RESCUE_CAPACITY) m.prompt = "CABIN FULL / RETURN TO CYAN BASE";
266 | if (attack >= 12) m.prompt = m.aboard > 0 ? "HARVESTER LOST / DELIVER SURVIVORS TO BASE" :
> "HARVESTER LOST / RESCUE REMAINING GROUPS";
267 | if (atBase && m.aboard > 0) {
268 |     m.prompt = "HOLD SPACE / UNLOAD SURVIVORS";
269 |     if (winch && low && steady) {
270 |         m.unload += dt / 1.2f;
271 |         if (m.unload >= 1) {
272 |             for (auto& c : m.crew) if (c.state == CrewState::Aboard) c.state = CrewState::Safe;
273 |             m.rescued += m.aboard; m.aboard = 0; m.unload = 0;
274 |             m.best = std::max(m.best, m.rescued);
275 |         }
276 |     } else m.unload = 0;
277 | } else m.unload = 0;
278 | int nearest = -1;
279 | float nearestDistance = 12;
280 | for (int i = 0; i < int(m.crew.size()); ++i) {
281 |     auto& c = m.crew[i];
282 |     if (c.state != CrewState::Running && c.state != CrewState::Waiting) continue;
283 |     float d = horizontalDistance(g_ornPos, c.pos);
284 |     if (d < nearestDistance) { nearestDistance = d; nearest = i; }
285 | }
286 | if (nearest != m.target) m.pickup = 0;
287 | m.target = nearest;
288 | if (nearest >= 0 && m.aboard < RESCUE_CAPACITY) {
289 |     m.prompt = !low ? "TOO HIGH / HOLD SPACE TO LOWER WINCH" : "HOLD SPACE / HOVER TO WINCH
> CREW";
290 |     if (winch && low && steady) {
291 |         m.pickup += dt / 0.42f;
292 |         if (m.pickup >= 1) {
293 |             m.crew[nearest].state = CrewState::Aboard;
294 |             ++m.aboard; --waiting; m.pickup = 0; m.target = -1;
295 |         }
296 |     } else m.pickup = 0;
297 | } else m.pickup = 0;
298 | bool approachHit = attack < 0 && m.elapsed > 8 && horizontalDistance(g_ornPos, worm) < 18 &&
> g_ornPos.y < worm.y + 22;
299 | bool breachHit = attack > 3 && attack < 15 && horizontalDistance(g_ornPos, worm) < 24 &&
> g_ornPos.y < worm.y + 40;
300 | if (approachHit || breachHit) {
301 |     for (auto& c : m.crew) if (c.state != CrewState::Safe && c.state != CrewState::Lost) {
> c.state = CrewState::Lost; ++m.lost; }
302 |     m.aboard = 0; m.planeLost = true; m.prompt = "ORNITHOPTER LOST IN THE WORM'S MAW";
> finishMission();

```

```
303 |     } else if ((waiting == 0 && m.aboard == 0) || attack >= 60) {  
304 |         for (auto& c : m.crew) if (c.state != CrewState::Safe && c.state != CrewState::Lost) {  
    > c.state = CrewState::Lost; ++m.lost; }  
305 |         m.aboard = 0; finishMission();  
306 |     }  
307 | }
```

Source Reference: src/arrakis_hud.h

```

1 | #ifndef ARRAKIS_HUD_H
2 | #define ARRAKIS_HUD_H
3 |
4 | #include <glad/glad.h>
5 | #include <glm/glm.hpp>
6 | #include <algorithm>
7 | #include <array>
8 | #include <cmath>
9 | #include <cstdint>
10 | #include <cstdio>
11 | #include <string>
12 | #include <vector>
13 |
14 | struct RescueHudState {
15 |     int width = 1280, height = 720;
16 |     // waiting counts Running + Waiting crew outside, not workers still inside.
17 |     int rescued = 0, aboard = 0, waiting = 0, lost = 0, total = 36, best = 0, wave = 1;
18 |     float remaining = 210, altitude = 0, speed = 0, boost = 1;
19 |     float missionDuration = 210, pursuit = 0, wormDistance = 220, extractionRemaining = 60;
20 |     int inside = 36, releasedGroups = 0;
21 |     float pickupProgress = 0, unloadProgress = 0, fps = 60;
22 |     float heading = 0;
23 |     bool title = true, paused = false, ended = false, muted = false;
24 |     std::string prompt;
25 |     glm::vec2 player{0}, harvester{0}, base{0}, worm{0}, pickup{0};
26 |     std::vector<glm::vec2> crewMarkers;
27 |     float attackTime = -1;
28 | };
29 |
30 | // Call with a current OpenGL 3.3 core context and loaded GLAD entry points.
31 | // width/height are framebuffer pixels. One context owns the HUD until cleanup.
32 | // Drawing preserves program, VAO/VBO, viewport, raster, color and blend state;
33 | // the caller's framebuffer and textures are not changed.
34 | namespace arrakis_hud_detail {
35 |
36 | struct Color { float r, g, b, a; };
37 | inline constexpr Color bone{0.91f, 0.89f, 0.82f, 1.0f};
38 | inline constexpr Color muted{0.59f, 0.61f, 0.58f, 1.0f};
39 | inline constexpr Color cyan{0.36f, 0.85f, 0.86f, 1.0f};
40 | inline constexpr Color amber{0.98f, 0.65f, 0.27f, 1.0f};
41 | inline constexpr Color white{1.0f, 1.0f, 1.0f, 1.0f};
42 | inline constexpr Color threat{0.98f, 0.36f, 0.27f, 1.0f};
43 | inline constexpr Color black{0.018f, 0.025f, 0.028f, 0.84f};
44 | inline constexpr Color edge{0.42f, 0.47f, 0.46f, 0.25f};
45 | struct Vertex { float x, y, r, g, b, a; };
46 | struct Resources {
47 |     GLuint program = 0, vao = 0, vbo = 0;
48 |     GLint canvas = -1;
49 |     std::size_t capacity = 0;
50 |     std::vector<Vertex> vertices;
51 | };
52 | inline Resources& resources() { static Resources r; return r; }
53 | inline float finite(float v, float fallback = 0.0f) { return std::isfinite(v) ? v : fallback; }
54 | inline float unit(float v) { return std::clamp(finite(v), 0.0f, 1.0f); }
55 | inline std::string number(int v) { return std::to_string((std::max)(0, v)); }
56 | inline std::string decimal(float v) {
57 |     char out[48];
58 |     std::snprintf(out, sizeof(out), "%.0f", std::clamp(finite(v), -99999.0f, 99999.0f));
59 |     return out;
60 | }
61 | inline std::string clock(float v) {
62 |     const int seconds = static_cast<int>(std::ceil(std::clamp(finite(v), 0.0f, 5999.0f)));
63 |     char out[16];
64 |     std::snprintf(out, sizeof(out), "%02d:%02d", seconds / 60, seconds % 60);
65 |     return out;
66 | }
67 |
68 | struct GlState {
69 |     GLint program, vao, buffer, viewport[4], polygon[2];
70 |     GLint srcRgb, dstRgb, srcAlpha, dstAlpha, equationRgb, equationAlpha;
71 |     GLboolean colorMask[4];
72 |     const std::array<GLenum, 10> caps{{GL_BLEND, GL_DEPTH_TEST, GL_CULL_FACE,
73 |         GL_SCISSOR_TEST, GL_STENCIL_TEST, GL_RASTERIZER_DISCARD, GL_COLOR_LOGIC_OP,
74 |         GL_SAMPLE_ALPHA_TO_COVERAGE, GL_SAMPLE_COVERAGE, GL_SAMPLE_ALPHA_TO_ONE}};
75 |     std::array<GLboolean, 10> enabled{};
76 |     GlState() {

```

```

77 |         glGetIntegerv(GL_CURRENT_PROGRAM, &program);
78 |         glGetIntegerv(GL_VERTEX_ARRAY_BINDING, &vao);
79 |         glGetIntegerv(GL_ARRAY_BUFFER_BINDING, &buffer);
80 |         glGetIntegerv(GL_VIEWPORT, viewport);
81 |         glGetIntegerv(GL_POLYGON_MODE, polygon);
82 |         glGetIntegerv(GL_BLEND_SRC_RGB, &srcRgb);
83 |         glGetIntegerv(GL_BLEND_DST_RGB, &dstRgb);
84 |         glGetIntegerv(GL_BLEND_SRC_ALPHA, &srcAlpha);
85 |         glGetIntegerv(GL_BLEND_DST_ALPHA, &dstAlpha);
86 |         glGetIntegerv(GL_BLEND_EQUATION_RGB, &equationRgb);
87 |         glGetIntegerv(GL_BLEND_EQUATION_ALPHA, &equationAlpha);
88 |         glGetBooleanv(GL_COLOR_WRITEMASK, colorMask);
89 |         for (std::size_t i = 0; i < caps.size(); ++i) enabled[i] = glIsEnabled(caps[i]);
90 |     }
91 |     ~GlState() {
92 |         glUseProgram(static_cast<GLuint>(program));
93 |         glBindVertexArray(static_cast<GLuint>(vao));
94 |         glBindBuffer(GL_ARRAY_BUFFER, static_cast<GLuint>(buffer));
95 |         glViewport(viewport[0], viewport[1], viewport[2], viewport[3]);
96 |         // Core contexts have one polygon mode for both faces.
97 |         glPolygonMode(GL_FRONT_AND_BACK, static_cast<GLenum>(polygon[0]));
98 |         glBlendFuncSeparate(srcRgb, dstRgb, srcAlpha, dstAlpha);
99 |         glBlendEquationSeparate(equationRgb, equationAlpha);
100 |         glColorMask(colorMask[0], colorMask[1], colorMask[2], colorMask[3]);
101 |         for (std::size_t i = 0; i < caps.size(); ++i) {
102 |             if (enabled[i]) glEnable(caps[i]); else glDisable(caps[i]);
103 |         }
104 |     }
105 |     GlState(const GlState&) = delete;
106 |     GlState& operator=(const GlState&) = delete;
107 | };
108 |
109 | inline GLuint shader(GLenum type, const char* source) {
110 |     const GLuint id = glCreateShader(type);
111 |     if (!id) return 0;
112 |     glShaderSource(id, 1, &source, nullptr);
113 |     glCompileShader(id);
114 |     GLint ok = GL_FALSE;
115 |     glGetShaderiv(id, GL_COMPILE_STATUS, &ok);
116 |     if (!ok) {
117 |         char log[2048]{};
118 |         glGetShaderInfoLog(id, sizeof(log), nullptr, log);
119 |         std::fprintf(stderr, "Arrakis HUD shader: %s\n", log);
120 |         glDeleteShader(id);
121 |         return 0;
122 |     }
123 |     return id;
124 | }
125 |
126 | inline void triangle(float ax, float ay, float bx, float by, float cx, float cy, Color c) {
127 |     auto& v = resources().vertices;
128 |     v.push_back({ax, ay, c.r, c.g, c.b, c.a});
129 |     v.push_back({bx, by, c.r, c.g, c.b, c.a});
130 |     v.push_back({cx, cy, c.r, c.g, c.b, c.a});
131 | }
132 | inline void rect(float x, float y, float w, float h, Color c) {
133 |     if (w <= 0 || h <= 0) return;
134 |     triangle(x, y, x + w, y, x + w, y + h, c);
135 |     triangle(x, y, x + w, y + h, x, y + h, c);
136 | }
137 | inline void outline(float x, float y, float w, float h, Color c) {
138 |     rect(x, y, w, 1, c); rect(x, y + h - 1, w, 1, c);
139 |     rect(x, y, 1, h, c); rect(x + w - 1, y, 1, h, c);
140 | }
141 | inline void card(float x, float y, float w, float h, Color accent = edge) {
142 |     rect(x, y, w, h, black); outline(x, y, w, h, edge);
143 |     rect(x, y, 2, h, accent);
144 | }
145 | inline void bar(float x, float y, float w, float amount, Color c, float h = 4) {
146 |     rect(x, y, w, h, edge); rect(x, y, w * unit(amount), h, c);
147 | }
148 |
149 | // Seven rows of five bits. Adjacent lit pixels are emitted as one rectangle.
150 | inline std::array<unsigned char, 7> glyph(char c) {
151 |     switch (c) {
152 |         case 'A': return {{14,17,17,31,17,17,17}};
153 |         case 'B': return {{30,17,17,30,17,17,30}};
154 |         case 'C': return {{14,17,16,16,16,17,14}};
155 |         case 'D': return {{30,17,17,17,17,17,30}};

```

```

156 | case 'E': return {{31,16,16,30,16,16,31}};
157 | case 'F': return {{31,16,16,30,16,16,16}};
158 | case 'G': return {{14,17,16,23,17,17,15}};
159 | case 'H': return {{17,17,17,31,17,17,17}};
160 | case 'I': return {{31,4,4,4,4,4,31}};
161 | case 'J': return {{7,2,2,2,18,18,12}};
162 | case 'K': return {{17,18,20,24,20,18,17}};
163 | case 'L': return {{16,16,16,16,16,16,31}};
164 | case 'M': return {{17,27,21,21,17,17,17}};
165 | case 'N': return {{17,25,21,19,17,17,17}};
166 | case 'O': return {{14,17,17,17,17,17,14}};
167 | case 'P': return {{30,17,17,30,16,16,16}};
168 | case 'Q': return {{14,17,17,17,21,18,13}};
169 | case 'R': return {{30,17,17,30,20,18,17}};
170 | case 'S': return {{15,16,16,14,1,1,30}};
171 | case 'T': return {{31,4,4,4,4,4,4}};
172 | case 'U': return {{17,17,17,17,17,17,14}};
173 | case 'V': return {{17,17,17,17,17,10,4}};
174 | case 'W': return {{17,17,17,21,21,21,10}};
175 | case 'X': return {{17,17,10,4,10,17,17}};
176 | case 'Y': return {{17,17,10,4,4,4,4}};
177 | case 'Z': return {{31,1,2,4,8,16,31}};
178 | case '0': return {{14,17,19,21,25,17,14}};
179 | case '1': return {{4,12,4,4,4,4,14}};
180 | case '2': return {{14,17,1,2,4,8,31}};
181 | case '3': return {{30,1,1,14,1,1,30}};
182 | case '4': return {{2,6,10,18,31,2,2}};
183 | case '5': return {{31,16,16,30,1,1,30}};
184 | case '6': return {{14,16,16,30,17,17,14}};
185 | case '7': return {{31,1,2,4,8,8,8}};
186 | case '8': return {{14,17,17,14,17,17,14}};
187 | case '9': return {{14,17,17,15,1,1,14}};
188 | case '/': return {{1,1,2,4,8,16,16}};
189 | case '+': return {{0,4,4,31,4,4,0}};
190 | case '-': return {{0,0,0,31,0,0,0}};
191 | case ':': return {{0,4,4,0,4,4,0}};
192 | case '.': return {{0,0,0,0,0,6,6}};
193 | case ',': return {{0,0,0,0,0,4,8}};
194 | case '%': return {{25,25,2,4,8,19,19}};
195 | case '!': return {{4,4,4,4,4,0,4}};
196 | case '?': return {{14,17,1,2,4,0,4}};
197 | case '(': return {{2,4,8,8,8,4,2}};
198 | case ')': return {{8,4,2,2,2,4,8}};
199 | case '=': return {{0,0,31,0,31,0,0}};
200 | case '_': return {{0,0,0,0,0,0,31}};
201 | case ' ': return {{0,0,0,0,0,0,0}};
202 | default: return {{14,17,1,2,4,0,4}};
203 | }
204 | }
205 | inline float textWidth(const std::string& s, float size) {
206 |     return s.empty() ? 0.0f : (static_cast<float>(s.size()) * 6 - 1) * size;
207 | }
208 | inline void text(float x, float y, const std::string& s, float size, Color c = bone) {
209 |     for (unsigned char ch : s) {
210 |         const char upper = static_cast<char>(ch >= 'a' && ch <= 'z' ? ch - 'a' + 'A' : ch);
211 |         const auto rows = glyph(upper);
212 |         for (int row = 0; row < 7; ++row) {
213 |             for (int column = 0; column < 5; ) {
214 |                 if (!(rows[row] & (16 >> column))) { ++column; continue; }
215 |                 const int first = column++;
216 |                 while (column < 5 && (rows[row] & (16 >> column))) ++column;
217 |                 rect(x + first * size, y + row * size, (column - first) * size, size, c);
218 |             }
219 |         }
220 |         x += 6 * size;
221 |     }
222 | }
223 | inline std::vector<std::string> wrap(const std::string& s, float width, float size) {
224 |     const std::size_t columns = static_cast<std::size_t>((std::max)(1.0f, std::floor((width + size)
> / (6 * size))));
225 |     std::vector<std::string> lines;
226 |     std::string line, word;
227 |     auto append = [&]() {
228 |         if (word.empty()) return;
229 |         if (!line.empty() && line.size() + 1 + word.size() > columns) {
230 |             lines.push_back(line); line.clear();
231 |         }
232 |         while (word.size() > columns) {
233 |             lines.push_back(word.substr(0, columns)); word.erase(0, columns);

```



```

234 |     }
235 |     if (!word.empty()) { if (!line.empty()) line += ' '; line += word; }
236 |     word.clear();
237 | };
238 | for (unsigned char c : s) {
239 |     if (c <= ' ') {
240 |         append();
241 |         if (c == '\n') { lines.push_back(line); line.clear(); }
242 |     } else { word += static_cast<char>(c < 127 ? c : '?'); }
243 |     }
244 |     append();
245 |     if (!line.empty()) lines.push_back(line);
246 |     return lines;
247 | }
248 | inline void centered(float x, float y, float width, const std::string& s, float size, Color c =
> bone) {
249 |     text(x + (width - textWidth(s, size)) * 0.5f, y, s, size, c);
250 | }
251 | inline void lines(float x, float y, const std::vector<std::string>& content, float size,
252 |     Color c = bone, std::size_t count = 100) {
253 |     const std::size_t n = (std::min)(content.size(), count);
254 |     for (std::size_t i = 0; i < n; ++i) text(x, y + i * size * 10, content[i], size, c);
255 | }
256 |
257 | inline void radar(float x, float y, float side, const RescueHudState& s, bool labels) {
258 |     card(x, y, side, side);
259 |     for (int i = 1; i < 4; ++i) {
260 |         rect(x + side * i / 4, y + 1, 1, side - 2, edge);
261 |         rect(x + 1, y + side * i / 4, side - 2, 1, edge);
262 |     }
263 |     auto point = [&](glm::vec2 p, Color c, int type) {
264 |         const float px = x + 6 + (std::clamp(finite(p.x), -300.0f, 300.0f) + 300) / 600 * (side -
> 12);
265 |         // Radar stays north-up even when the heading-locked chase camera turns.
266 |         const float py = y + 6 + (std::clamp(finite(p.y), -300.0f, 300.0f) + 300) / 600 * (side -
> 12);
267 |         if (type == 0) {
268 |             float yaw = finite(s.heading) * 0.01745329252f;
269 |             glm::vec2 forward(-std::sin(yaw), -std::cos(yaw)), right(-forward.y, forward.x);
270 |             glm::vec2 tip = glm::vec2(px, py) + forward * 4.0f;
271 |             glm::vec2 tail = glm::vec2(px, py) - forward * 3.0f;
272 |             triangle(tip.x, tip.y, tail.x - right.x * 4, tail.y - right.y * 4,
273 |                 tail.x + right.x * 4, tail.y + right.y * 4, c);
274 |         }
275 |         else if (type == 1) outline(px - 4, py - 4, 8, 8, c);
276 |         else if (type == 2) { rect(px - 3, py - 3, 6, 6, c); }
277 |         else if (type == 4) { rect(px - 1.5f, py - 1.5f, 3, 3, c); }
278 |         else {
279 |             triangle(px, py - 4, px - 4, py, px, py + 4, c);
280 |             triangle(px, py - 4, px, py + 4, px + 4, py, c);
281 |         }
282 |     };
283 |     point(s.base, cyan, 1);
284 |     for (const glm::vec2& crew : s.crewMarkers) point(crew, amber, 4);
285 |     point(s.harvester, white, 2);
286 |     point(s.worm, threat, 3); point(s.player, cyan, 0);
287 |     if (labels) {
288 |         text(x, y - 18, "SECTOR / +/-300", 1.5f, muted);
289 |         text(x, y + side + 9, "YOU / BASE", 1.5f, cyan);
290 |         text(x, y + side + 26, "CREW", 1.5f, amber);
291 |         text(x + 47, y + side + 26, "HARV", 1.5f, white);
292 |         text(x + 92, y + side + 26, "WORM", 1.5f, threat);
293 |     }
294 | }
295 |
296 | inline void overlay(float w, float h, const RescueHudState& s) {
297 |     rect(0, 0, w, h, {0.01f, 0.018f, 0.022f, 0.78f});
298 |     const bool result = s.ended;
299 |     const bool title = !result && s.title;
300 |     const float margin = w < 480 ? 12.0f : 28.0f;
301 |     const float panelW = (std::min)(680.0f, w - margin * 2);
302 |     const float inset = panelW < 420 || h < 420 ? 14.0f : 28.0f;
303 |     const float contentW = panelW - inset * 2;
304 |     const float size = h < 300 ? 1.0f : panelW < 380 || h < 420 ? 1.5f : 2.0f;
305 |     const float headingSize = h < 300 ? 2.0f : panelW < 420 || h < 420 ? 2.5f : 4.0f;
306 |     const std::string heading = result ? "MISSION CLOSED" : title ? "HARVESTER FLIGHT" : "FLIGHT
> PAUSED";
307 |     const auto body = wrap(result ? (s.prompt == "ORNITHOPTER LOST IN THE WORM'S MAW" ?
308 |         "ORNITHOPTER LOST IN THE WORM'S MAW. DELIVERED CREW REMAIN SAFE." :

```

```

309 |         "RESCUE COMPLETE. ONLY CREW DELIVERED TO BASE COUNT. EVERY LIFE COUNTS.") :
310 |         title ? "FLEEING HARVESTER RELEASES 1-4 CREW AT A TIME.\n"
311 |         "AMBER GROUPS WAIT ON SAND. 8 RESCUE SEATS.\n"
312 |         "MOUSE STEERS. FLY LOW. HOLD SPACE TO WINCH.\n"
313 |         "RETURN TO CYAN BASE. HOLD SPACE TO BANK RESCUED." :
314 |         "THE RESCUE IS ON HOLD. YOUR CREW IS COUNTING ON YOU.", contentW, size);
315 |     const auto keys = wrap(result ? "ENTER NEXT MISSION / R RETRY" : title ? "ENTER START" : "P
> RESUME", contentW, size);
316 |     const float lineH = size * 10;
317 |     const float statsH = result ? lineH * 3 + 12 : 0;
318 |     const float panelH = inset * 2 + 23 + headingSize * 7 + 18 + statsH +
319 |         static_cast<float>(body.size() + keys.size()) * lineH + 25;
320 |     const float x = (w - panelW) * 0.5f;
321 |     const float y = (std::max)(8.0f, (h - panelH) * 0.5f);
322 |     card(x, y, panelW, panelH, result ? amber : cyan);
323 |     float cursor = y + inset;
324 |     text(x + inset, cursor, "ARRAKIS / SEARCH + RESCUE", 1.5f, muted);
325 |     cursor += 23;
326 |     text(x + inset, cursor, heading, headingSize, bone);
327 |     cursor += headingSize * 7 + 18;
328 |     if (result) {
329 |         text(x + inset, cursor, "SECURED " + number(s.rescued) + " / " + number(s.total), size,
> cyan);
330 |         cursor += lineH;
331 |         text(x + inset, cursor, "BEST " + number((std::max)(s.best, s.rescued)) + " / MISSION " +
> number(s.wave), size, bone);
332 |         cursor += lineH;
333 |         text(x + inset, cursor, "ABOARD " + number(s.aboard) + " / LOST " + number(s.lost), size,
> muted);
334 |         cursor += lineH + 12;
335 |     }
336 |     lines(x + inset, cursor, body, size, muted);
337 |     cursor += static_cast<float>(body.size()) * lineH + 12;
338 |     rect(x + inset, cursor, contentW, 1, edge);
339 |     cursor += 13;
340 |     lines(x + inset, cursor, keys, size, cyan);
341 | }
342 |
343 | inline void compose(float w, float h, const RescueHudState& s) {
344 |     const float m = w < 600 || h < 400 ? 10.0f : 22.0f;
345 |     const float inner = w - 2 * m;
346 |     const bool compact = w < 760 || h < 440;
347 |     const bool shortView = h < 360;
348 |     const bool extraction = s.attackTime >= 0;
349 |     const float remaining = extraction ? s.extractionRemaining : s.remaining;
350 |     const float duration = extraction ? 60.0f : (std::max)(0.001f, finite(s.missionDuration,
> 210.0f));
351 |     const bool danger = finite(remaining) <= 15 || extraction;
352 |     const std::string timerLabel = extraction ? "EXTRACTION WINDOW" : danger ? "BREACH IMMINENT" :
> "TIME TO BREACH";
353 |     const Color timerColor = danger ? amber : bone;
354 |     const float topH = compact ? (shortView ? 78.0f : 112.0f) : 106.0f;
355 |     card(m, m, inner, topH);
356 |     if (!compact) {
357 |         text(m + 18, m + 16, "ARRAKIS", 3.0f);
358 |         text(m + 18, m + 47, "FLEEING HARVESTER", 1.5f, muted);
359 |         text(m + 18, m + 76, "SEARCH + RESCUE / " + number(s.wave), 1.5f, cyan);
360 |         const float scoreX = m + inner * 0.30f;
361 |         text(scoreX, m + 14, "SECURED " + number(s.rescued) + "/" + number(s.total), 2.5f, cyan);
362 |         text(scoreX, m + 44, "ABOARD " + number(s.aboard) + "/8", 1.5f);
363 |         text(scoreX + 126, m + 44, "INSIDE " + number(s.inside), 1.5f, muted);
364 |         for (int i = 0; i < 8; ++i) {
365 |             const Color c = i < s.aboard ? cyan : edge;
366 |             rect(scoreX + i * 14, m + 64, 10, 5, c);
367 |         }
368 |         text(scoreX + 126, m + 64, "GROUPS " + number(s.releasedGroups), 1.5f, amber);
369 |         text(scoreX, m + 82, "SAND " + number(s.waiting) + " / LOST " + number(s.lost), 1.5f,
> muted);
370 |         const float timerX = m + inner - 230;
371 |         text(timerX, m + 15, timerLabel, 1.5f, timerColor);
372 |         text(timerX, m + 39, clock(remaining), 3.0f, timerColor);
373 |         bar(timerX, m + 74, 210, finite(remaining) / duration, timerColor, 5);
374 |         text(timerX, m + 87, "SECURE CREW AT CYAN BASE", 1.5f, muted);
375 |     } else {
376 |         text(m + 12, m + 10, "ARRAKIS / FLEEING HARVESTER", inner < 330 ? 1.5f : 2.0f);
377 |         const float row = m + (shortView ? 26 : 32);
378 |         text(m + 12, row, timerLabel, 1.5f, timerColor);
379 |         text(m + inner - 12 - textWidth(clock(remaining), 2), row - 2, clock(remaining), 2,
> timerColor);

```

```

380 |         bar(m + inner - 86, m + (shortView ? 56 : 48), 74, finite(remaining) / duration, timerColor,
> 3);
381 |         const float statsY = m + (shortView ? 43 : 54);
382 |         text(m + 12, statsY, "SAVED " + number(s.rescued) + "/" + number(s.total), 1.5f, cyan);
383 |         const std::string seats = "SEATS " + number(s.aboard) + "/" + 8;
384 |         text(m + inner - 12 - textWidth(seats, 1.5f), statsY, seats, 1.5f);
385 |         text(m + 12, m + (shortView ? 61 : 76), "SAND " + number(s.waiting) + " INSIDE " +
> number(s.inside) +
386 |             " GROUPS " + number(s.releasedGroups), 1.5f, muted);
387 |         if (!shortView) {
388 |             text(m + 12, m + 96, "LOST " + number(s.lost) + " / WAVE " + number(s.wave), 1.5f,
> muted);
389 |             if (inner >= 360) for (int i = 0; i < 8; ++i)
390 |                 rect(m + inner - 104 + i * 12, m + 99, 8, 4, i < s.aboard ? cyan : edge);
391 |         }
392 |     }
393 |
394 |     // Controls wrap at readable pixel sizes instead of scaling a 1280px screen down.
395 |     const float controlSize = 1.5f;
396 |     const auto controls = wrap("MOUSE STEER / W S THRUST / A D STRAFE / Q E ALT / SPACE WINCH /
> SHIFT BOOST / C CAMERA / M MUTE / P PAUSE / R RETRY", inner - 20, controlSize);
397 |     const float controlsH = static_cast<float>(controls.size()) * controlSize * 10 + 14;
398 |     const float controlsY = h - m - controlsH;
399 |     card(m, controlsY, inner, controlsH);
400 |     lines(m + 10, controlsY + 8, controls, controlSize, muted);
401 |
402 |     std::string action = s.prompt;
403 |     if (action.empty()) action = s.aboard >= 8 ? "SEATS FULL / RETURN TO CYAN BASE" :
404 |         s.aboard > 0 ? "RETURN TO BASE / HOLD SPACE TO BANK CREW" : "FLY LOW TO CREW / HOLD SPACE TO
> WINCH";
405 |     if (s.unloadProgress > 0) action = "BANKING CREW / HOLD SPACE";
406 |     else if (s.pickupProgress > 0) action = "WINCH ACTIVE / HOLD SPACE";
407 |     const bool sideRadar = shortView && inner >= 280 && controlsY - 72 >= m + topH + 6;
408 |     const float actionW = (std::min)(inner - (sideRadar ? 76.0f : 0.0f), 620.0f);
409 |     const float actionSize = compact ? 1.5f : 2.0f;
410 |     auto actionLines = wrap(action, actionW - 28, actionSize);
411 |     if (actionLines.size() > 2) {
412 |         actionLines.resize(2);
413 |         std::string& last = actionLines.back();
414 |         if (last.size() >= 3) last.replace(last.size() - 3, 3, "...");
415 |     }
416 |     const float actionH = static_cast<float>(actionLines.size()) * actionSize * 10 + 23;
417 |     const float actionY = controlsY - actionH - 8;
418 |     const float actionX = sideRadar ? w - m - actionW : (w - actionW) * 0.5f;
419 |     card(actionX, actionY, actionW, actionH, cyan);
420 |     lines(actionX + 14, actionY + 10, actionLines, actionSize);
421 |     const float progress = s.unloadProgress > 0 ? s.unloadProgress : s.pickupProgress;
422 |     bar(actionX + 14, actionY + actionH - 9, actionW - 28, progress, cyan, 3);
423 |
424 |     const float radarY = m + topH + (compact ? 10 : 32);
425 |     const float available = actionY - radarY - (compact ? 6 : 52);
426 |     const float side = (std::min)(compact ? 92.0f : 156.0f, available);
427 |     if (sideRadar) radar(m, controlsY - 72, 64, s, false);
428 |     else if (side >= 40) radar(w - m - side, radarY, side, s, !compact && side >= 128);
429 |     if (!sideRadar && available >= 88 && (!compact || inner >= 420)) {
430 |         const float tw = compact ? 184.0f : 222.0f;
431 |         const bool pursuitRoom = available >= 112;
432 |         card(m, radarY, tw, pursuitRoom ? 112.0f : 88.0f);
433 |         text(m + 12, radarY + 12, "ALT " + decimal(s.altitude) + " M", 1.5f);
434 |         text(m + 12, radarY + 31, "SPD " + decimal(s.speed) + " / FPS " + decimal(s.fps) + (s.muted
> ? " [MUTED]" : ""), 1.5f, s.muted ? amber : muted);
435 |         text(m + 12, radarY + 52, "BOOST", 1.5f, muted);
436 |         bar(m + 72, radarY + 55, tw - 86, s.boost, cyan, 5);
437 |         text(m + 12, radarY + 72, "WORM RANGE " + decimal((std::max)(0.0f, finite(s.wormDistance)))
> + " M", 1.5f, amber);
438 |         if (pursuitRoom) {
439 |             text(m + 12, radarY + 92, "PURSUIT", 1.5f, amber);
440 |             bar(m + 86, radarY + 95, tw - 100, s.pursuit, threat, 5);
441 |         }
442 |     }
443 |     if (!s.ended && !s.title && !s.paused) {
444 |         Color guide = cyan; guide.a = 0.35f;
445 |         for (int dx : {-1, 1}) for (int dy : {-1, 1}) {
446 |             float x = w * (0.5f + dx * 0.06f), y = h * (0.5f + dy * 0.06f);
447 |             rect(x - (dx > 0 ? 7 : 0), y, 7, 1, guide);
448 |             rect(x, y - (dy > 0 ? 7 : 0), 1, 7, guide);
449 |         }
450 |     }
451 |     if (s.ended || s.title || s.paused) overlay(w, h, s);

```

```

452 | }
453 |
454 | } // namespace arrakis_hud_detail
455 |
456 | inline void initRescueHud() {
457 |     namespace hud = arrakis_hud_detail;
458 |     auto& r = hud::resources();
459 |     if (r.program) return;
460 |     const char* vs =
461 |         "#version 330 core\n"
462 |         "layout(location=0) in vec2 position;\n"
463 |         "layout(location=1) in vec4 color;\n"
464 |         "uniform vec2 canvas; out vec4 tint;\n"
465 |         "void main(){ tint=color; gl_Position=vec4(position.x/canvas.x*2.0-1.0,"
466 |         "1.0-position.y/canvas.y*2.0,0.0,1.0); }\n";
467 |     const char* fs =
468 |         "#version 330 core\n"
469 |         "in vec4 tint; out vec4 fragment;\n"
470 |         "void main(){ fragment=tint; }\n";
471 |     const GLuint vertexShader = hud::shader(GL_VERTEX_SHADER, vs);
472 |     const GLuint fragmentShader = hud::shader(GL_FRAGMENT_SHADER, fs);
473 |     if (!vertexShader || !fragmentShader) {
474 |         if (vertexShader) glDeleteShader(vertexShader);
475 |         if (fragmentShader) glDeleteShader(fragmentShader);
476 |         return;
477 |     }
478 |     const GLuint program = glCreateProgram();
479 |     if (!program) { glDeleteShader(vertexShader); glDeleteShader(fragmentShader); return; }
480 |     glAttachShader(program, vertexShader); glAttachShader(program, fragmentShader);
481 |     glLinkProgram(program);
482 |     glDeleteShader(vertexShader); glDeleteShader(fragmentShader);
483 |     GLint ok = GL_FALSE;
484 |     glGetProgramiv(program, GL_LINK_STATUS, &ok);
485 |     if (!ok) {
486 |         char log[2048]{};
487 |         glGetProgramInfoLog(program, sizeof(log), nullptr, log);
488 |         std::fprintf(stderr, "Arrakis HUD link: %s\n", log);
489 |         glDeleteProgram(program);
490 |         return;
491 |     }
492 |     GLint oldVao = 0, oldBuffer = 0;
493 |     glGetIntegerv(GL_VERTEX_ARRAY_BINDING, &oldVao);
494 |     glGetIntegerv(GL_ARRAY_BUFFER_BINDING, &oldBuffer);
495 |     glGenVertexArrays(1, &r.vao); glGenBuffers(1, &r.vbo);
496 |     if (!r.vao || !r.vbo) {
497 |         if (r.vao) glDeleteVertexArrays(1, &r.vao);
498 |         if (r.vbo) glDeleteBuffers(1, &r.vbo);
499 |         r.vao = r.vbo = 0; glDeleteProgram(program); return;
500 |     }
501 |     r.program = program;
502 |     r.canvas = glGetUniformLocation(r.program, "canvas");
503 |     glBindVertexArray(r.vao); glBindBuffer(GL_ARRAY_BUFFER, r.vbo);
504 |     glEnableVertexAttribArray(0); glEnableVertexAttribArray(1);
505 |     glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(hud::Vertex), nullptr);
506 |     glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, sizeof(hud::Vertex), reinterpret_cast<const
> void*>(offsetof(hud::Vertex, r)));
507 |     glBindVertexArray(static_cast<GLuint>(oldVao));
508 |     glBindBuffer(GL_ARRAY_BUFFER, static_cast<GLuint>(oldBuffer));
509 |     r.vertices.reserve(65536);
510 | }
511 |
512 | inline void drawRescueHud(const RescueHudState& state) {
513 |     namespace hud = arrakis_hud_detail;
514 |     if (state.width <= 0 || state.height <= 0) return;
515 |     initRescueHud();
516 |     auto& r = hud::resources();
517 |     if (!r.program) return;
518 |     hud::GLState saved;
519 |     // Keep small-window text legible; only large framebuffers scale the canvas up.
520 |     const float scale = (std::max)(1.0f, (std::min)(state.width / 1280.0f, state.height / 720.0f));
521 |     const float w = state.width / scale, h = state.height / scale;
522 |     r.vertices.clear();
523 |     hud::compose(w, h, state);
524 |     for (GLenum cap : saved.caps) glDisable(cap);
525 |     glEnable(GL_BLEND);
526 |     glBlendEquationSeparate(GL_FUNC_ADD, GL_FUNC_ADD);
527 |     glBlendFuncSeparate(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA, GL_ONE, GL_ONE_MINUS_SRC_ALPHA);
528 |     glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE);
529 |     glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);

```

```
530 |     glViewport(0, 0, state.width, state.height);
531 |     glUseProgram(r.program);
532 |     glUniform2f(r.canvas, w, h);
533 |     glBindVertexArray(r.vao); glBindBuffer(GL_ARRAY_BUFFER, r.vbo);
534 |     const std::size_t bytes = r.vertices.size() * sizeof(hud::Vertex);
535 |     if (bytes > r.capacity) r.capacity = (std::max)(bytes, r.capacity * 2);
536 |     // Orphan the dynamic buffer so the previous frame can finish without a stall.
537 |     glBufferData(GL_ARRAY_BUFFER, static_cast<GLsizeiptr>(r.capacity), nullptr, GL_DYNAMIC_DRAW);
538 |     glBufferSubData(GL_ARRAY_BUFFER, 0, static_cast<GLsizeiptr>(bytes), r.vertices.data());
539 |     glDrawArrays(GL_TRIANGLES, 0, static_cast<GLsizei>(r.vertices.size()));
540 | }
541 |
542 | inline void cleanupRescueHud() {
543 |     auto& r = arrakis_hud_detail::resources();
544 |     if (r.vbo) glDeleteBuffers(1, &r.vbo);
545 |     if (r.vao) glDeleteVertexArrays(1, &r.vao);
546 |     if (r.program) glDeleteProgram(r.program);
547 |     r.vbo = r.vao = r.program = 0;
548 |     r.canvas = -1; r.capacity = 0;
549 |     std::vector<arrakis_hud_detail::Vertex>().swap(r.vertices);
550 | }
551 |
552 | #endif // ARRAKIS_HUD_H
```

Source Reference: src/arrakis_worm.h

```

1 | #ifndef ARRAKIS_WORM_H
2 | #define ARRAKIS_WORM_H
3 |
4 | // Include after drawDesertRockFormation and before drawFullScene in arrakis.cpp.
5 | // Initialize/clean up with the GL context current; draw in both scene passes.
6 | #include <cmath>
7 | #include <vector>
8 |
9 | namespace arrakis_worm_detail {
10 |
11 | static const float pi = 3.14159265358979323846f;
12 | static const int bodyRows = 192;
13 | static const int bodySlices = 96;
14 | static Mesh body, lip, funnel, throat, teeth;
15 | static std::vector<Vertex> bodyVertices;
16 | static bool ready = false;
17 | static bool poseValid = false;
18 | static glm::vec3 lastCenter(0.0f);
19 | static float lastAttack = 0.0f, lastTime = 0.0f;
20 | static float lastHeading = 0.0f, lastProgress = 0.0f;
21 |
22 | inline float smooth(float a, float b, float t) {
23 |     float u = glm::clamp((t - a) / (b - a), 0.0f, 1.0f);
24 |     return u * u * (3.0f - 2.0f * u);
25 | }
26 |
27 | inline void destroy(Mesh& mesh) {
28 |     if (mesh.EBO) glDeleteBuffers(1, &mesh.EBO);
29 |     if (mesh.VBO) glDeleteBuffers(1, &mesh.VBO);
30 |     if (mesh.VAO) glDeleteVertexArrays(1, &mesh.VAO);
31 |     mesh = Mesh{};
32 | }
33 |
34 | inline void gridIndices(std::vector<unsigned int>& indices, int rows, int slices) {
35 |     indices.reserve(rows * slices * 6);
36 |     for (int j = 0; j < rows; ++j) {
37 |         for (int i = 0; i < slices; ++i) {
38 |             unsigned int a = j * (slices + 1) + i;
39 |             unsigned int b = a + 1, c = a + slices + 1, d = c + 1;
40 |             indices.insert(indices.end(), {a, c, d, a, d, b});
41 |         }
42 |     }
43 | }
44 |
45 | inline void surfaceNormals(std::vector<Vertex>& v,
46 |                             const std::vector<unsigned int>& indices, int slices) {
47 |     for (Vertex& vertex : v) vertex.normal = glm::vec3(0.0f);
48 |     for (size_t i = 0; i < indices.size(); i += 3) {
49 |         Vertex& a = v[indices[i]];
50 |         Vertex& b = v[indices[i + 1]];
51 |         Vertex& c = v[indices[i + 2]];
52 |         glm::vec3 n = glm::cross(b.pos - a.pos, c.pos - a.pos);
53 |         a.normal += n; b.normal += n; c.normal += n;
54 |     }
55 |     // Weld the duplicated UV seam without welding separate tooth surfaces.
56 |     for (size_t j = 0; j < v.size(); j += slices + 1) {
57 |         glm::vec3 n = v[j].normal + v[j + slices].normal;
58 |         v[j].normal = v[j + slices].normal = n;
59 |     }
60 |     for (Vertex& vertex : v) vertex.normal = glm::normalize(vertex.normal);
61 | }
62 |
63 | // Profile order follows the exterior up to the lip, then down the inner funnel.
64 | // The same winding therefore gives outward skin and inward-facing mouth normals.
65 | inline Mesh lathe(const std::vector<glm::vec2>& profile, float roughness, bool deepFloor = false) {
66 |     const int slices = 160;
67 |     std::vector<Vertex> v;
68 |     std::vector<unsigned int> indices;
69 |     v.reserve(profile.size() * (slices + 1));
70 |     for (size_t j = 0; j < profile.size(); ++j) {
71 |         for (int i = 0; i <= slices; ++i) {
72 |             float u = float(i) / slices;
73 |             float a = 2.0f * pi * u;
74 |             float rough = roughness * std::sin(pi * float(j) / (profile.size() - 1))
75 |                 * (0.55f * std::sin(11.0f * a + 0.7f * j)
76 |                 + 0.30f * std::sin(23.0f * a - 0.9f * j))

```

```

77 |         + 0.15f * std::sin(47.0f * a + 1.1f * j));
78 |         float r = profile[j].x + rough;
79 |         v.push_back(glm::vec3(r * std::cos(a), profile[j].y, r * std::sin(a)),
80 |             glm::vec3(0.0f), glm::vec2(u, float(j) / (profile.size() - 1)));
81 |     }
82 | }
83 | gridIndices(indices, int(profile.size()) - 1, slices);
84 | surfaceNormals(v, indices, slices);
85 | if (deepFloor) {
86 |     // Recessed throat closure, never a cap across the mouth opening.
87 |     unsigned int center = static_cast<unsigned int>(v.size());
88 |     v.push_back({0.0f, profile.back().y, 0.0f}, {0.0f, 1.0f, 0.0f}, {0.5f, 0.5f});
89 |     for (int i = 0; i <= slices; ++i) {
90 |         Vertex vertex = v[(profile.size() - 1) * (slices + 1) + i];
91 |         vertex.normal = glm::vec3(0.0f, 1.0f, 0.0f);
92 |         v.push_back(vertex);
93 |         if (i < slices) indices.insert(indices.end(), {center, center + i + 2, center + i + 1});
94 |     }
95 | }
96 | return uploadMesh(v, indices);
97 | }
98 |
99 | inline Mesh makeTeeth() {
100 |     const int rows = 6, sides = 5, steps = 6;
101 |     const int counts[rows] = {192, 184, 176, 168, 152, 128};
102 |     const float radii[rows] = {13.1f, 11.3f, 9.4f, 7.7f, 6.0f, 4.5f};
103 |     const float depths[rows] = {-0.8f, -2.6f, -4.5f, -6.3f, -8.2f, -10.1f};
104 |     std::vector<Vertex> v;
105 |     std::vector<unsigned int> indices;
106 |     v.reserve(1000 * (steps * sides + 1));
107 |     indices.reserve(1000 * ((steps - 1) * sides * 6 + sides * 3));
108 |     for (int row = 0; row < rows; ++row) {
109 |         for (int tooth = 0; tooth < counts[row]; ++tooth) {
110 |             float seed = float(tooth * 17 + row * 131);
111 |             float variation = 0.5f + 0.5f * std::sin(seed * 1.713f);
112 |             float a = 2.0f * pi * (tooth + 0.37f * row
113 |                 + 0.18f * std::sin(seed)) / counts[row];
114 |             glm::vec3 radial(std::cos(a), 0.0f, std::sin(a));
115 |             glm::vec3 sideways(-std::sin(a), 0.0f, std::cos(a));
116 |             float reach = (radii[row] - 1.8f) * (0.55f + 0.17f * variation);
117 |             float drop = 2.1f + 1.1f * variation;
118 |             float curl = 0.20f + 0.30f * std::sin(seed * 0.91f);
119 |             float width = (0.075f + 0.035f * variation) * (1.0f - 0.06f * row);
120 |             unsigned int base = static_cast<unsigned int>(v.size());
121 |             for (int k = 0; k < steps; ++k) {
122 |                 float t = float(k) / steps;
123 |                 glm::vec3 p = radial * (radii[row] - reach * t)
124 |                     + sideways * (curl * t * t)
125 |                     + glm::vec3(0.0f, depths[row] + 0.65f * std::sin(pi * t)
126 |                         - drop * t * t, 0.0f);
127 |                 glm::vec3 tangent = glm::normalize(-reach * radial + 2.0f * curl * t * sideways
128 |                     + glm::vec3(0.0f, 0.65f * pi * std::cos(pi * t) - 2.0f * drop * t, 0.0f));
129 |                 glm::vec3 b = glm::normalize(glm::cross(tangent, sideways));
130 |                 glm::vec3 n = glm::cross(b, tangent);
131 |                 float radius = width * std::pow(1.0f - t, 0.85f);
132 |                 for (int s = 0; s < sides; ++s) {
133 |                     float angle = 2.0f * pi * s / sides;
134 |                     glm::vec3 normal = n * std::cos(angle) + b * std::sin(angle);
135 |                     v.push_back({p + radius * normal, normal, glm::vec2(float(s) / sides, t)});
136 |                 }
137 |             }
138 |             for (int k = 0; k < steps - 1; ++k) {
139 |                 for (int s = 0; s < sides; ++s) {
140 |                     unsigned int a0 = base + k * sides + s;
141 |                     unsigned int b0 = base + k * sides + (s + 1) % sides;
142 |                     indices.insert(indices.end(), {a0, b0, b0 + sides, a0, b0 + sides, a0 + sides});
143 |                 }
144 |             }
145 |             // One true apex, not a blunt, tiny capped cylinder.
146 |             unsigned int tip = static_cast<unsigned int>(v.size());
147 |             glm::vec3 tipPos = radial * (radii[row] - reach) + sideways * curl
148 |                 + glm::vec3(0.0f, depths[row] - drop, 0.0f);
149 |             glm::vec3 tipNormal = glm::normalize(-reach * radial + 2.0f * curl * sideways
150 |                 + glm::vec3(0.0f, -0.65f * pi - 2.0f * drop, 0.0f));
151 |             v.push_back({tipPos, tipNormal, glm::vec2(0.5f, 1.0f)});
152 |             for (int s = 0; s < sides; ++s) {
153 |                 unsigned int a0 = base + (steps - 1) * sides + s;
154 |                 unsigned int b0 = base + (steps - 1) * sides + (s + 1) % sides;
155 |                 indices.insert(indices.end(), {a0, b0, tip});

```



```

156 |     }
157 | }
158 | }
159 | return uploadMesh(v, indices);
160 | }
161 |
162 | struct Pose {
163 |     glm::vec3 head;
164 |     float tilt;
165 |     float aperture;
166 | };
167 |
168 | inline Pose pose(glm::vec3 center, float attack, float time,
169 |                 float headingDegrees = 0.0f, float approachProgress = 0.0f) {
170 |     float rise = smooth(0.0f, 6.0f, attack);
171 |     float lunge = smooth(6.0f, 12.0f, attack);
172 |     float retreat = smooth(13.5f, 18.0f, attack);
173 |     float ground = getDuneHeight(center.x, center.z);
174 |     float approachHeight = 10.0f + 4.0f * smooth(0.0f, 1.0f, approachProgress);
175 |     float height = attack < 0.0f ? approachHeight : 14.0f + 22.0f * rise - 27.0f * lunge;
176 |     height += 0.25f * std::sin(time * 1.6f) * rise * (1.0f - lunge);
177 |     height = glm::mix(height, -24.0f, retreat);
178 |     float reach = 12.0f * lunge * (1.0f - retreat);
179 |     glm::vec3 head(center.x, ground + height, center.z + reach);
180 |     float heading = glm::radians(headingDegrees);
181 |     float targetY = getDuneHeight(center.x + 18.0f * std::sin(heading),
182 |                                   center.z + 18.0f * std::cos(heading)) + 1.0f;
183 |     // Deform in the local YZ plane; the draw root rotates local +Z into the heading.
184 |     float aim = std::atan2(18.0f - reach, targetY - head.y);
185 |     float tilt = glm::radians(70.0f) * (1.0f - rise)
186 |               + aim * lunge * (1.0f - retreat);
187 |     float swallow = smooth(12.0f, 13.5f, attack) * (1.0f - retreat);
188 |     return {head, tilt, 1.0f - 0.18f * swallow};
189 | }
190 |
191 | inline glm::mat4 headMatrix(const Pose& p) {
192 |     glm::mat4 m = glm::translate(glm::mat4(1.0f), p.head);
193 |     m = glm::rotate(m, p.tilt, glm::vec3(1.0f, 0.0f, 0.0f));
194 |     return glm::scale(m, glm::vec3(p.aperture, 1.0f, p.aperture));
195 | }
196 |
197 | inline void deformBody(glm::vec3 center, const Pose& p) {
198 |     glm::vec3 axis(0.0f, std::cos(p.tilt), std::sin(p.tilt));
199 |     const float neckLength = 64.0f, join = 0.4f;
200 |     glm::vec3 end = p.head - 1.4f * axis;
201 |     // Constant-curvature neck: even a downward bite has a bend radius over 20 m.
202 |     auto neckPoint = [&](float s) {
203 |         float half = 0.5f * p.tilt * (1.0f - s);
204 |         float mid = 0.5f * p.tilt * (1.0f + s);
205 |         float sinc = half > 0.0001f ? std::sin(half) / half : 1.0f;
206 |         float span = neckLength * (1.0f - s) * sinc;
207 |         return end - glm::vec3(0.0f, span * std::cos(mid), span * std::sin(mid));
208 |     };
209 |     glm::vec3 p0(center.x, getDuneHeight(center.x, center.z) - 140.0f, center.z);
210 |     glm::vec3 p3 = neckPoint(0.0f);
211 |     float buriedLength = (p3.y - p0.y) / 3.0f;
212 |     glm::vec3 p1 = p0 + glm::vec3(0.0f, buriedLength, 0.0f);
213 |     glm::vec3 p2 = p3 - glm::vec3(0.0f, buriedLength, 0.0f);
214 |     for (int j = 0; j <= bodyRows; ++j) {
215 |         float t = float(j) / bodyRows;
216 |         glm::vec3 c, tangent;
217 |         if (t < join) {
218 |             float s = t / join, u = 1.0f - s;
219 |             c = u * u * u * p0 + 3.0f * u * u * s * p1
220 |               + 3.0f * u * s * s * p2 + s * s * s * p3;
221 |             tangent = glm::normalize(u * u * (p1 - p0)
222 |                                     + 2.0f * u * s * (p2 - p1) + s * s * (p3 - p2));
223 |         } else {
224 |             float s = (t - join) / (1.0f - join);
225 |             c = neckPoint(s);
226 |             tangent = glm::vec3(0.0f, std::cos(p.tilt * s), std::sin(p.tilt * s));
227 |         }
228 |         glm::vec3 across(0.0f, -tangent.z, tangent.y);
229 |         float taper = 1.0f - smooth(0.85f, 1.0f, t);
230 |         float ridge = 0.70f * std::pow(0.5f + 0.5f * std::cos(2.0f * pi * 32.0f * t), 3.0f);
231 |         float radius = glm::mix(13.9f + ridge, 15.0f * p.aperture, 1.0f - taper);
232 |         for (int i = 0; i <= bodySlices; ++i) {
233 |             float a = 2.0f * pi * i / bodySlices;
234 |             glm::vec3 radial = glm::vec3(std::cos(a), 0.0f, 0.0f) + std::sin(a) * across;

```

```

235 |         float grain = taper * (0.11f * std::sin(13.0f * a + 9.0f * t)
236 |             + 0.07f * std::sin(29.0f * a - 17.0f * t));
237 |         Vertex& vertex = bodyVertices[j * (bodySlices + 1) + i];
238 |         vertex.pos = c + (radius + grain) * radial;
239 |         vertex.uv = glm::vec2(float(i) / bodySlices, t * 32.0f);
240 |     }
241 | }
242 | // Derivatives include the bend, taper and ridge slopes, not just radial normals.
243 | for (int j = 0; j <= bodyRows; ++j) {
244 |     for (int i = 0; i < bodySlices; ++i) {
245 |         int prev = (i + bodySlices - 1) % bodySlices;
246 |         int next = (i + 1) % bodySlices;
247 |         int lo = j == 0 ? 0 : j - 1, hi = j == bodyRows ? j : j + 1;
248 |         glm::vec3 ds = bodyVertices[hi * (bodySlices + 1) + i].pos
249 |             - bodyVertices[lo * (bodySlices + 1) + i].pos;
250 |         glm::vec3 da = bodyVertices[j * (bodySlices + 1) + next].pos
251 |             - bodyVertices[j * (bodySlices + 1) + prev].pos;
252 |         bodyVertices[j * (bodySlices + 1) + i].normal = glm::normalize(glm::cross(ds, da));
253 |     }
254 |     bodyVertices[j * (bodySlices + 1) + bodySlices].normal = bodyVertices[j * (bodySlices +
> 1)].normal;
255 | }
256 | }
257 |
258 | } // namespace arrakis_worm_detail
259 |
260 | inline void cleanupSandwormMeshes() {
261 |     using namespace arrakis_worm_detail;
262 |     destroy(body); destroy(lip); destroy(funnel); destroy(throat); destroy(teeth);
263 |     bodyVertices.clear();
264 |     ready = poseValid = false;
265 | }
266 |
267 | inline void initSandwormMeshes() {
268 |     using namespace arrakis_worm_detail;
269 |     if (ready) return;
270 |     bodyVertices.resize((bodyRows + 1) * (bodySlices + 1));
271 |     deformBody(glm::vec3(0.0f), pose(glm::vec3(0.0f), 6.0f, 0.0f));
272 |     std::vector<unsigned int> indices;
273 |     gridIndices(indices, bodyRows, bodySlices);
274 |     body = uploadMesh(bodyVertices, indices);
275 |     GLint previousBuffer = 0;
276 |     glGetIntegerv(GL_ARRAY_BUFFER_BINDING, &previousBuffer);
277 |     glBindBuffer(GL_ARRAY_BUFFER, body.VBO);
278 |     glBufferData(GL_ARRAY_BUFFER, bodyVertices.size() * sizeof(Vertex), bodyVertices.data(),
> GL_DYNAMIC_DRAW);
279 |     glBindBuffer(GL_ARRAY_BUFFER, static_cast<GLuint>(previousBuffer));
280 |
281 |     lip = lathe({{15.0f, -1.4f}, {15.6f, -0.9f}, {15.9f, -0.2f}, {15.5f, 0.5f},
282 |         {14.8f, 0.8f}, {14.0f, 0.5f}, {13.4f, -0.1f}, {13.1f, -0.8f}}, 0.16f);
283 |     funnel = lathe({{13.1f, -0.8f}, {12.8f, -1.3f}, {12.0f, -2.0f}, {11.3f, -2.6f},
284 |         {10.5f, -3.5f}, {9.4f, -4.5f}, {8.6f, -5.4f}, {7.7f, -6.3f},
285 |         {6.8f, -7.3f}, {6.0f, -8.2f}, {5.2f, -9.2f}, {4.5f, -10.1f},
286 |         {3.6f, -11.4f}, {2.9f, -13.0f}}, 0.08f);
287 |     throat = lathe({{2.9f, -13.0f}, {2.6f, -15.0f}, {2.2f, -19.0f},
288 |         {1.7f, -24.0f}, {0.8f, -27.0f}}, 0.0f, true);
289 |     teeth = makeTeeth();
290 |     ready = true;
291 |     poseValid = false;
292 | }
293 |
294 | inline void drawSandworm(glm::vec3 center, float attackTime, float time,
295 |     float headingDegrees = 0.0f, float approachProgress = 0.0f) {
296 |     using namespace arrakis_worm_detail;
297 |     if (!ready || !std::isfinite(attackTime) || !std::isfinite(time)
298 |         || !std::isfinite(center.x) || !std::isfinite(center.y) || !std::isfinite(center.z)
299 |         || !std::isfinite(headingDegrees) || !std::isfinite(approachProgress)
300 |         || attackTime >= 18.0f) return;
301 |     approachProgress = glm::clamp(approachProgress, 0.0f, 1.0f);
302 |     Pose p = pose(center, attackTime, time, headingDegrees, approachProgress);
303 |     // Both render passes use the same pose. Only the flexible skin needs streaming.
304 |     if (!poseValid || center.x != lastCenter.x || center.y != lastCenter.y || center.z !=
> lastCenter.z
305 |         || attackTime != lastAttack || time != lastTime
306 |         || headingDegrees != lastHeading || approachProgress != lastProgress) {
307 |         deformBody(center, p);
308 |         GLint previousBuffer = 0;
309 |         glGetIntegerv(GL_ARRAY_BUFFER_BINDING, &previousBuffer);
310 |         glBindBuffer(GL_ARRAY_BUFFER, body.VBO);

```

```

311 |         glBufferSubData(GL_ARRAY_BUFFER, 0, bodyVertices.size() * sizeof(Vertex),
> bodyVertices.data());
312 |         glBindBuffer(GL_ARRAY_BUFFER, static_cast<GLuint>(previousBuffer));
313 |         lastCenter = center; lastAttack = attackTime; lastTime = time;
314 |         lastHeading = headingDegrees; lastProgress = approachProgress;
315 |         poseValid = true;
316 |     }
317 |     Material skin;
318 |     skin.diffuse = glm::vec3(0.43f, 0.27f, 0.12f);
319 |     skin.specular = glm::vec3(0.055f, 0.04f, 0.025f);
320 |     skin.shininess = 8.0f;
321 |     Material rim = skin;
322 |     rim.diffuse = glm::vec3(0.49f, 0.31f, 0.14f);
323 |     Material inside = skin;
324 |     inside.diffuse = glm::vec3(0.15f, 0.075f, 0.029f);
325 |     inside.specular = glm::vec3(0.018f);
326 |     Material dark = inside;
327 |     dark.diffuse = glm::vec3(0.009f, 0.004f, 0.002f);
328 |     dark.specular = glm::vec3(0.0f);
329 |     Material enamel = skin;
330 |     enamel.diffuse = glm::vec3(0.58f, 0.43f, 0.24f);
331 |     enamel.specular = glm::vec3(0.07f);
332 |     enamel.shininess = 18.0f;
333 |     glm::mat4 root = glm::translate(glm::mat4(1.0f), center);
334 |     root = glm::rotate(root, glm::radians(headingDegrees), glm::vec3(0.0f, 1.0f, 0.0f));
335 |     root = glm::translate(root, -center);
336 |     glm::mat4 head = root * headMatrix(p);
337 |     drawMeshPrimitive(body, root, skin);
338 |     drawMeshPrimitive(lip, head, rim);
339 |     drawMeshPrimitive(funnel, head, inside);
340 |     drawMeshPrimitive(throat, head, dark);
341 |     drawMeshPrimitive(teeth, head, enamel);
342 |     float wake = 1.0f - smooth(0.0f, 6.0f, attackTime);
343 |     if (wake > 0.001f) {
344 |         Material sand = skin;
345 |         sand.diffuse = glm::vec3(0.68f, 0.46f, 0.22f);
346 |         sand.specular = glm::vec3(0.015f);
347 |         for (int i = 0; i < 4; ++i) {
348 |             glm::vec3 local(center.x, center.y, center.z - 14.0f - 14.0f * i);
349 |             glm::vec3 world(root * glm::vec4(local, 1.0f));
350 |             local.y = getDuneHeight(world.x, world.z) - 0.8f;
351 |             glm::mat4 mound = glm::translate(root, local);
352 |             mound = glm::scale(mound, glm::vec3((12.0f - i) * wake,
353 |             (3.0f + 0.35f * std::sin(time * 2.0f - i)) * wake,
354 |             10.0f * wake));
355 |             drawMeshPrimitive(g_meshSphere, mound, sand);
356 |         }
357 |     }
358 | }
359 |
360 | #endif // ARRAKIS_WORM_H

```

Source Reference: src/arrakis_audio.h

```

1 | #pragma once
2 |
3 | #include "miniaudio.h"
4 | #include <glm/glm.hpp>
5 | #include <string>
6 | #include <vector>
7 | #include <fstream>
8 | #include <iostream>
9 | #include <algorithm>
10 |
11 | inline std::string findSoundFile(const std::string& filename) {
12 |     static const std::vector<std::string> searchDirs = {
13 |         "assets/audio/",
14 |         "../assets/audio/",
15 |         "../../assets/audio/",
16 |         "../../../assets/audio/",
17 |         "../../../../assets/audio/"
18 |     };
19 |     for (const auto& dir : searchDirs) {
20 |         std::string path = dir + filename;
21 |         std::ifstream f(path.c_str());
22 |         if (f.good()) return path;
23 |     }
24 |     return "";
25 | }
26 |
27 | struct ArrakisAudio {
28 |     ma_engine engine{};
29 |     bool initialized = false;
30 |     bool muted = false;
31 |     float masterVolume = 1.0f;
32 |
33 |     // Sound objects
34 |     ma_sound sndFlight{};
35 |     ma_sound sndBoost{};
36 |     ma_sound sndWormRumble{};
37 |     ma_sound sndWormBreach{};
38 |     ma_sound sndRescueWinch{};
39 |     ma_sound sndRescueSafe{};
40 |     ma_sound sndHarvester{};
41 |     ma_sound sndCrewHelp{};
42 |
43 |     bool hasFlight = false;
44 |     bool hasBoost = false;
45 |     bool hasWormRumble = false;
46 |     bool hasWormBreach = false;
47 |     bool hasRescueWinch = false;
48 |     bool hasRescueSafe = false;
49 |     bool hasHarvester = false;
50 |     bool hasCrewHelp = false;
51 |
52 |     // Dynamic smoothing
53 |     float boostVolume = 0.0f;
54 |     float flightPitch = 1.0f;
55 |     float crewHelpTimer = 3.0f;
56 |     bool breachTriggered = false;
57 |
58 |     int lastRescued = 0;
59 |     int lastAboard = 0;
60 |     float lastPickup = 0.0f;
61 |
62 |     bool init() {
63 |         ma_engine_config config = ma_engine_config_init();
64 |         ma_result res = ma_engine_init(&config, &engine);
65 |         if (res != MA_SUCCESS) {
66 |             std::cerr << "[AUDIO] Warning: Failed to initialize audio engine (code: " << res << ").
> Running in silent mode.\n";
67 |             initialized = false;
68 |             return false;
69 |         }
70 |         initialized = true;
71 |
72 |         // Configure listener
73 |         ma_engine_listener_set_world_up(&engine, 0, 0, 1, 0);
74 |
75 |         auto loadSound = [this](ma_sound* snd, const std::vector<std::string>& candidates, bool

```

```

> loop, float minDist, float maxDist, ma_attenuation_model model = ma_attenuation_model_inverse) ->
> bool {
76 |         for (const auto& name : candidates) {
77 |             std::string path = findSoundFile(name);
78 |             if (!path.empty()) {
79 |                 ma_result r = ma_sound_init_from_file(&engine, path.c_str(), 0, nullptr,
> nullptr, snd);
80 |                 if (r == MA_SUCCESS) {
81 |                     ma_sound_set_looping(snd, loop ? MA_TRUE : MA_FALSE);
82 |                     ma_sound_set_min_distance(snd, minDist);
83 |                     ma_sound_set_max_distance(snd, maxDist);
84 |                     ma_sound_set_attenuation_model(snd, model);
85 |                     std::cout << "[AUDIO] Loaded: " << path << "\n";
86 |                     return true;
87 |                 }
88 |             }
89 |         }
90 |         return false;
91 |     };
92 |
93 |     // 1. Ornithopter flight (looping engine drone & wing flutter)
94 |     hasFlight = loadSound(&sndFlight, {"ornithopter_flight.mp3", "ornithopter_flight.wav"},
> true, 12.0f, 150.0f);
95 |     if (hasFlight) ma_sound_set_volume(&sndFlight, 0.75f);
96 |
97 |     // 2. Ornithopter boost (looping afterburner)
98 |     hasBoost = loadSound(&sndBoost, {"ornithopter_boost.mp3", "ornithopter_boost.wav"}, true,
> 12.0f, 150.0f);
99 |     if (hasBoost) {
100 |         ma_sound_set_volume(&sndBoost, 0.0f);
101 |         ma_sound_start(&sndBoost);
102 |     }
103 |
104 |     // 3. Sandworm rumble (sub-bass approach tremor)
105 |     hasWormRumble = loadSound(&sndWormRumble, {"worm_rumble.mp3", "worm_rumble.wav"}, true,
> 30.0f, 320.0f, ma_attenuation_model_inverse);
106 |     if (hasWormRumble) ma_sound_set_volume(&sndWormRumble, 0.9f);
107 |
108 |     // 4. Sandworm breach (one-shot eruption & roar)
109 |     hasWormBreach = loadSound(&sndWormBreach, {"worm_breach.mp3", "worm_breach.wav"}, false,
> 45.0f, 380.0f, ma_attenuation_model_inverse);
110 |     if (hasWormBreach) ma_sound_set_volume(&sndWormBreach, 1.0f);
111 |
112 |     // 5. Harvester movement (crawler treads & engine)
113 |     hasHarvester = loadSound(&sndHarvester, {"harvester_engine.wav", "harvester_engine.mp3",
> "harvester.wav"}, true, 18.0f, 200.0f, ma_attenuation_model_inverse);
114 |     if (hasHarvester) ma_sound_set_volume(&sndHarvester, 0.75f);
115 |
116 |     // 6. Rescuing winch / pickup
117 |     hasRescueWinch = loadSound(&sndRescueWinch, {"rescue_winch.wav", "rescue_winch.mp3"}, false,
> 15.0f, 120.0f);
118 |     if (hasRescueWinch) ma_sound_set_volume(&sndRescueWinch, 0.85f);
119 |
120 |     // 7. Safe rescue delivery at base
121 |     hasRescueSafe = loadSound(&sndRescueSafe, {"rescue_safe.wav", "rescue_safe.ogg",
> "rescue_safe.mp3", "confirmation_004.ogg"}, false, 20.0f, 140.0f);
122 |     if (hasRescueSafe) ma_sound_set_volume(&sndRescueSafe, 0.95f);
123 |
124 |     // 8. Stranded crew calling for help
125 |     hasCrewHelp = loadSound(&sndCrewHelp, {"crew_help.wav", "crew_help.mp3",
> "question_001.ogg"}, false, 16.0f, 85.0f);
126 |     if (hasCrewHelp) ma_sound_set_volume(&sndCrewHelp, 0.85f);
127 |
128 |     return true;
129 | }
130 |
131 | void toggleMute() {
132 |     muted = !muted;
133 |     if (!initialized) return;
134 |     ma_engine_set_volume(&engine, muted ? 0.0f : masterVolume);
135 | }
136 |
137 | void update(float dt,
138 |             const glm::vec3& camPos,
139 |             const glm::vec3& camTarget,
140 |             const glm::vec3& ornPos,
141 |             float ornSpeed,
142 |             bool isBoosting,
143 |             const glm::vec3& harvPos,

```

```

144 |         const glm::vec3& wormPos,
145 |         float wormDist,
146 |         float attackTime,
147 |         int rescuedCount,
148 |         int aboardCount,
149 |         float pickupProgress,
150 |         const glm::vec3& nearestCrewPos,
151 |         float nearestCrewDist,
152 |         const glm::vec3& basePos,
153 |         bool isFlying,
154 |         bool isPaused)
155 | {
156 |     if (!initialized) return;
157 |
158 |     // Master pause / mute handling
159 |     if (isPaused || muted) {
160 |         ma_engine_set_volume(&engine, 0.0f);
161 |         return;
162 |     } else {
163 |         ma_engine_set_volume(&engine, masterVolume);
164 |     }
165 |
166 |     // 1. Update 3D Audio Listener (Camera eye & look direction)
167 |     glm::vec3 look = camTarget - camPos;
168 |     if (glm::length(look) > 0.001f) look = glm::normalize(look);
169 |     else look = glm::vec3(0, 0, -1);
170 |
171 |     ma_engine_listener_set_position(&engine, 0, camPos.x, camPos.y, camPos.z);
172 |     ma_engine_listener_set_direction(&engine, 0, look.x, look.y, look.z);
173 |
174 |     // 2. Ornithopter flight sound
175 |     if (hasFlight) {
176 |         ma_sound_set_position(&sndFlight, ornPos.x, ornPos.y, ornPos.z);
177 |         float targetPitch = 0.92f + glm::clamp(ornSpeed / 42.0f, 0.0f, 0.38f);
178 |         flightPitch = glm::mix(flightPitch, targetPitch, 1.0f - std::exp(-dt * 6.0f));
179 |         ma_sound_set_pitch(&sndFlight, flightPitch);
180 |
181 |         float targetVol = isBoosting ? 0.40f : 0.78f;
182 |         ma_sound_set_volume(&sndFlight, targetVol);
183 |
184 |         if (!ma_sound_is_playing(&sndFlight)) {
185 |             ma_sound_start(&sndFlight);
186 |         }
187 |     }
188 |
189 |     // 3. Ornithopter boost sound
190 |     if (hasBoost) {
191 |         ma_sound_set_position(&sndBoost, ornPos.x, ornPos.y, ornPos.z);
192 |         float targetBoostVol = (isFlying && isBoosting) ? 0.90f : 0.0f;
193 |         boostVolume = glm::mix(boostVolume, targetBoostVol, 1.0f - std::exp(-dt * 8.0f));
194 |         ma_sound_set_volume(&sndBoost, boostVolume);
195 |     }
196 |
197 |     // 4. Harvester moving sound (3D spatialized at crawler)
198 |     if (hasHarvester) {
199 |         ma_sound_set_position(&sndHarvester, harvPos.x, harvPos.y, harvPos.z);
200 |         if (isFlying && attackTime < 11.0f) {
201 |             if (!ma_sound_is_playing(&sndHarvester)) ma_sound_start(&sndHarvester);
202 |         } else {
203 |             if (ma_sound_is_playing(&sndHarvester)) ma_sound_stop(&sndHarvester);
204 |         }
205 |     }
206 |
207 |     // 5. Sandworm rumble (distance relative subterranean tremor)
208 |     if (hasWormRumble) {
209 |         ma_sound_set_position(&sndWormRumble, wormPos.x, wormPos.y, wormPos.z);
210 |         if (isFlying && attackTime < 16.0f) {
211 |             // Modulate volume slightly as worm gets closer
212 |             float proximityBoost = glm::mix(1.2f, 0.8f, glm::clamp(wormDist / 200.0f, 0.0f,
> 1.0f));
213 |             ma_sound_set_volume(&sndWormRumble, proximityBoost);
214 |             if (!ma_sound_is_playing(&sndWormRumble)) ma_sound_start(&sndWormRumble);
215 |         } else {
216 |             if (ma_sound_is_playing(&sndWormRumble)) ma_sound_stop(&sndWormRumble);
217 |         }
218 |     }
219 |
220 |     // 6. Sandworm breach eruption (triggered once on breach)
221 |     if (hasWormBreach) {

```

```

222 |         if (attackTime >= 0.0f && attackTime < 2.5f && !breachTriggered) {
223 |             breachTriggered = true;
224 |             ma_sound_set_position(&sndWormBreach, wormPos.x, wormPos.y, wormPos.z);
225 |             ma_sound_seek_to_pcm_frame(&sndWormBreach, 0);
226 |             ma_sound_start(&sndWormBreach);
227 |         }
228 |     }
229 |
230 |     // 7. Winch / Rescuing people
231 |     if (hasRescueWinch) {
232 |         // Trigger or loop while picking up or when a person is secured aboard
233 |         if (aboardCount > lastAboard || (pickupProgress > 0.08f && lastPickup <= 0.08f)) {
234 |             ma_sound_set_position(&sndRescueWinch, ornPos.x, ornPos.y, ornPos.z);
235 |             ma_sound_seek_to_pcm_frame(&sndRescueWinch, 0);
236 |             ma_sound_start(&sndRescueWinch);
237 |         }
238 |     }
239 |
240 |     // 8. Safe delivery at cyan base
241 |     if (hasRescueSafe) {
242 |         if (rescuedCount > lastRescued) {
243 |             ma_sound_set_position(&sndRescueSafe, basePos.x, basePos.y, basePos.z);
244 |             ma_sound_seek_to_pcm_frame(&sndRescueSafe, 0);
245 |             ma_sound_start(&sndRescueSafe);
246 |         }
247 |     }
248 |
249 |     // 9. Stranded crew asking for help (periodic 3D calls when near)
250 |     if (hasCrewHelp && isFlying) {
251 |         crewHelpTimer -= dt;
252 |         if (crewHelpTimer <= 0.0f) {
253 |             if (nearestCrewDist > 0.0f && nearestCrewDist < 75.0f) {
254 |                 ma_sound_set_position(&sndCrewHelp, nearestCrewPos.x, nearestCrewPos.y,
> nearestCrewPos.z);
255 |                 ma_sound_seek_to_pcm_frame(&sndCrewHelp, 0);
256 |                 ma_sound_start(&sndCrewHelp);
257 |                 crewHelpTimer = 5.5f + (rand() % 30) * 0.1f;
258 |             } else {
259 |                 crewHelpTimer = 2.0f;
260 |             }
261 |         }
262 |     }
263 |
264 |     // State update for next frame
265 |     lastRescued = rescuedCount;
266 |     lastAboard = aboardCount;
267 |     lastPickup = pickupProgress;
268 | }
269 |
270 | void resetMission() {
271 |     breachTriggered = false;
272 |     crewHelpTimer = 3.0f;
273 |     lastRescued = 0;
274 |     lastAboard = 0;
275 |     lastPickup = 0.0f;
276 | }
277 |
278 | void shutdown() {
279 |     if (!initialized) return;
280 |     if (hasFlight) ma_sound_uninit(&sndFlight);
281 |     if (hasBoost) ma_sound_uninit(&sndBoost);
282 |     if (hasWormRumble) ma_sound_uninit(&sndWormRumble);
283 |     if (hasWormBreach) ma_sound_uninit(&sndWormBreach);
284 |     if (hasRescueWinch) ma_sound_uninit(&sndRescueWinch);
285 |     if (hasRescueSafe) ma_sound_uninit(&sndRescueSafe);
286 |     if (hasHarvester) ma_sound_uninit(&sndHarvester);
287 |     if (hasCrewHelp) ma_sound_uninit(&sndCrewHelp);
288 |     ma_engine_uninit(&engine);
289 |     initialized = false;
290 | }
291 | };
292 |
293 | inline ArrakisAudio g_audio;

```


Source Reference: tests/arrakis_tests.h

```

1 | #pragma once
2 |
3 | // Include after arrakis_game.h. These tests never initialize GLFW or call GL.
4 | #include <iostream>
5 | #include <limits>
6 |
7 | namespace arrakis_tests_detail {
8 | struct Results {
9 |     const char* current = "";
10 |     int cases = 0;
11 |     int failedCases = 0;
12 |     int checks = 0;
13 |     int failures = 0;
14 |
15 |     void check(bool condition, const char* message) {
16 |         ++checks;
17 |         if (!condition) {
18 |             ++failures;
19 |             std::cerr << "[FAIL] " << current << ": " << message << '\n';
20 |         }
21 |     }
22 |
23 |     template <typename Test>
24 |     void run(const char* name, Test test) {
25 |         current = name;
26 |         ++cases;
27 |         const int before = failures;
28 |         test();
29 |         if (failures != before) ++failedCases;
30 |         std::cout << (failures == before ? "[PASS] " : "[FAIL] ") << name << '\n';
31 |     }
32 | };
33 |
34 | inline bool near(float a, float b) { return std::abs(a - b) < 0.001f; }
35 |
36 | inline int count(CrewState state) {
37 |     int total = 0;
38 |     for (const auto& c : g_mission.crew) if (c.state == state) ++total;
39 |     return total;
40 | }
41 |
42 | inline void fresh(bool start = true) {
43 |     for (bool& key : g_keys) key = false;
44 |     g_mouseHover = glm::vec2(0);
45 |     g_mouseTurnRate = 0;
46 |     g_mission = RescueMission{};
47 |     g_isBoosting = false;
48 |     g_wingPhase = 0;
49 |     resetMission(start);
50 | }
51 |
52 | inline void advance(float seconds) {
53 |     const int steps = std::max(1, static_cast<int>(std::ceil(seconds * 120.0f)));
54 |     for (int i = 0; i < steps; ++i) updateMission(seconds / steps);
55 | }
56 |
57 | inline void hover(glm::vec3 position, float altitude = 6.0f) {
58 |     // Teleports exercise mission state transitions, not flight-route assistance.
59 |     position.y = getDuneHeight(position.x, position.z) + altitude;
60 |     g_ornPos = position;
61 |     g_ornVel = glm::vec3(0);
62 |     g_ornSpeed = 0;
63 |     g_mouseHover = glm::vec2(0);
64 |     g_mouseTurnRate = 0;
65 |     g_mission.altitude = altitude;
66 | }
67 |
68 | inline glm::vec3 staging() {
69 |     return g_mission.crew.empty() ? harvesterPosition() : g_mission.crew.front().destination;
70 | }
71 |
72 | inline void readyCrew() {
73 |     // Compact explicit-state fixture isolates winch/capacity rules from routing.
74 |     fresh();
75 |     const glm::vec3 center(0, getDuneHeight(0, -10), -10);
76 |     for (auto& group : g_mission.groups) {

```

```

77 |         group.released = true;
78 |         group.center = center;
79 |     }
80 |     for (std::size_t i = 0; i < g_mission.crew.size(); ++i) {
81 |         auto& c = g_mission.crew[i];
82 |         c.state = CrewState::Waiting;
83 |         c.pos = center + glm::vec3(float(i % 6) * 0.6f, 0, float(i / 6) * 0.6f);
84 |         c.pos.y = getDuneHeight(c.pos.x, c.pos.z);
85 |         c.destination = c.pos;
86 |     }
87 | }
88 |
89 | inline void collect(int occupants) {
90 |     hover(staging());
91 |     g_keys[GLFW_KEY_SPACE] = true;
92 |     advance(occupants * 0.45f);
93 | }
94 |
95 | inline void deliver() {
96 |     hover(g_mission.base);
97 |     g_keys[GLFW_KEY_SPACE] = true;
98 |     advance(1.3f);
99 | }
100 |
101 | inline void reconciled(Results& r) {
102 |     const int unrescued = count(CrewState::Inside) + count(CrewState::Running) +
> count(CrewState::Waiting);
103 |     r.check(g_mission.crew.size() == 36, "mission must contain 36 crew");
104 |     r.check(count(CrewState::Safe) == g_mission.rescued, "Safe count must equal banked rescues");
105 |     r.check(count(CrewState::Aboard) == g_mission.aboard, "Aboard count must equal cabin
> occupants");
106 |     r.check(count(CrewState::Lost) == g_mission.lost, "Lost count must equal losses");
107 |     r.check(g_mission.rescued + g_mission.aboard + g_mission.lost + unrescued == 36,
108 |         "all 36 crew must be accounted for exactly once");
109 |     r.check(g_mission.aboard >= 0 && g_mission.aboard <= RESCUE_CAPACITY, "cabin count must stay
> within capacity");
110 |     if (g_mission.phase == MissionPhase::Debrief)
111 |         r.check(unrescued == 0 && g_mission.aboard == 0, "debrief must leave no unresolved crew");
112 | }
113 | } // namespace arrakis_tests_detail
114 |
115 | inline int runMissionTests() {
116 |     using namespace arrakis_tests_detail;
117 |     Results r;
118 |
119 |     r.run("Reset and title do not advance", [&] {
120 |         fresh(false);
121 |         r.check(g_mission.phase == MissionPhase::Title, "reset(false) must enter title");
122 |         r.check(g_mission.wave == 1 && near(g_mission.breachAt, 210), "first wave must have a
> 210-second timer");
123 |         r.check(g_mission.rescued == 0 && g_mission.aboard == 0 && g_mission.lost == 0 &&
> g_mission.best == 0,
124 |             "fresh reset must clear all score counters");
125 |         r.check(count(CrewState::Inside) == 36, "all crew must initially be inside");
126 |         r.check(near(g_mission.altitude, 8) && near(g_mission.boost, 1), "reset must restore
> altitude and boost");
127 |         const glm::vec3 position = g_ornPos;
128 |         const float yaw = g_ornYaw;
129 |         const glm::vec3 harvester = harvesterPosition(), worm = wormPosition();
130 |         setMouseHover(1000, 250, 1000, 500);
131 |         g_keys[GLFW_KEY_D] = g_keys[GLFW_KEY_SPACE] = true;
132 |         advance(100.0f);
133 |         r.check(near(g_mission.elapsed, 0), "title must not consume mission time");
134 |         r.check(glm::distance(g_ornPos, position) < 0.001f && near(g_wingPhase, 0), "title update
> must not advance flight");
135 |         r.check(near(g_ornYaw, yaw) && near(g_mouseTurnRate, 0), "title must not steer even with
> off-center mouse hover");
136 |         r.check(count(CrewState::Inside) == 36 && near(g_mission.pickup, 0), "title must not release
> or pick up crew");
137 |         r.check(glm::distance(harvesterPosition(), harvester) < 0.001f &&
> glm::distance(wormPosition(), worm) < 0.001f,
138 |             "title must freeze harvester and pursuing worm");
139 |         reconciled(r);
140 |     });
141 |
142 |     r.run("Pause freezes mission and resumes", [&] {
143 |         fresh();
144 |         setMouseHover(1000, 250, 1000, 500);
145 |         advance(7.0f);

```

```

146 |         const RescueMission before = g_mission;
147 |         const glm::vec3 position = g_ornPos;
148 |         const float wings = g_wingPhase;
149 |         const float yaw = g_ornYaw, turnRate = g_mouseTurnRate;
150 |         const glm::vec3 harvester = harvesterPosition(), worm = wormPosition();
151 |         const float gap = wormGap();
152 |         g_mission.paused = true;
153 |         g_keys[GLFW_KEY_W] = g_keys[GLFW_KEY_LEFT_SHIFT] = g_keys[GLFW_KEY_SPACE] = true;
154 |         advance(100.0f);
155 |         r.check(near(g_mission.elapsed, before.elapsed), "pause must freeze the breach clock");
156 |         r.check(glm::distance(g_ornPos, position) < 0.001f && near(g_wingPhase, wings), "pause must
> freeze flight");
157 |         r.check(near(g_ornYaw, yaw) && near(g_mouseTurnRate, turnRate), "pause must freeze yaw and
> smoothed turn rate");
158 |         r.check(near(g_mission.boost, before.boost) && near(g_mission.pickup, before.pickup), "pause
> must freeze resources and winch progress");
159 |         bool crewUnchanged = g_mission.crew.size() == before.crew.size();
160 |         for (std::size_t i = 0; crewUnchanged && i < before.crew.size(); ++i)
161 |             crewUnchanged = g_mission.crew[i].state == before.crew[i].state &&
162 |                 glm::distance(g_mission.crew[i].pos, before.crew[i].pos) < 0.001f;
163 |         r.check(crewUnchanged, "pause must freeze crew release and walking");
164 |         r.check(glm::distance(harvesterPosition(), harvester) < 0.001f &&
> glm::distance(wormPosition(), worm) < 0.001f &&
165 |             near(g_mission.harvesterYaw, before.harvesterYaw) && near(g_mission.harvesterTravel,
> before.harvesterTravel) && near(wormGap(), gap),
166 |             "pause must freeze moving harvester, travel, yaw, and pursuing worm");
167 |         bool groupsUnchanged = g_mission.groups.size() == before.groups.size();
168 |         for (std::size_t i = 0; groupsUnchanged && i < before.groups.size(); ++i)
169 |             groupsUnchanged = g_mission.groups[i].released == before.groups[i].released &&
170 |                 glm::distance(g_mission.groups[i].center, before.groups[i].center) <
> 0.001f;
171 |         r.check(groupsUnchanged, "pause must freeze every evacuation group");
172 |         for (bool& key : g_keys) key = false;
173 |         setMouseHover(500, 250, 1000, 500);
174 |         g_mission.paused = false;
175 |         advance(0.1f);
176 |         r.check(g_mission.elapsed > before.elapsed, "unpausing must resume mission time");
177 |         reconciled(r);
178 |     });
179 |
180 |     r.run("Crew groups release on schedule and walk to fixed clusters", [&] {
181 |         fresh();
182 |         r.check(g_mission.groups.size() == 15, "exactly fifteen groups must contain thirty-six
> crew");
183 |         int total = 0;
184 |         for (std::size_t i = 0; i < g_mission.groups.size(); ++i) {
185 |             const auto& group = g_mission.groups[i];
186 |             r.check(group.count == int(i % 4) + 1 && near(group.release, 6.0f + 7.0f * float(i)) &&
> !group.released,
187 |                 "group sizes must repeat one through four, with first release six and cadence
> seven");
188 |             int members = 0;
189 |             unsigned slots = 0;
190 |             for (const auto& c : g_mission.crew) if (c.group == int(i)) {
191 |                 ++members;
192 |                 r.check(c.slot >= 0 && c.slot < group.count && near(c.release, group.release),
193 |                     "each crew member must have its group's scheduled release and a valid
> slot");
194 |                 if (c.slot >= 0 && c.slot < 4) slots |= 1u << c.slot;
195 |             }
196 |             r.check(members == group.count && slots == (1u << group.count) - 1,
197 |                 "group membership and slots must cover each member exactly once");
198 |             total += group.count;
199 |         }
200 |         r.check(total == 36 && g_mission.groups.back().count == 3, "group counts must total
> thirty-six with final group three");
201 |         const glm::vec3 initialInside = g_mission.crew.back().pos;
202 |         const glm::vec3 initialHarvester = harvesterPosition();
203 |         advance(5.99f);
204 |         r.check(count(CrewState::Inside) == 36, "no crew may release before six seconds");
205 |         r.check(horizontalDistance(g_mission.crew.back().pos, initialInside) > 2 &&
206 |             horizontalDistance(g_mission.crew.back().pos, harvesterPosition()) < 15 &&
207 |             horizontalDistance(initialHarvester, harvesterPosition()) > 2,
208 |             "Inside crew must follow the moving harvester, not remain at its initial location");
209 |         advance(0.02f);
210 |         r.check(g_mission.crew.front().state == CrewState::Running && count(CrewState::Inside) ==
> 35,
211 |             "the first one-person group must release while the others remain inside");
212 |         const glm::vec3 start = g_mission.crew.front().pos;

```

```

213 |         const glm::vec3 destination = g_mission.crew.front().destination;
214 |         const glm::vec3 center = g_mission.groups.front().center;
215 |         const float scheduledYaw = glm::radians(harvesterRouteYaw(6));
216 |         const glm::vec3 scheduledForward(-std::sin(scheduledYaw), 0, -std::cos(scheduledYaw));
217 |         const glm::vec3 scheduledRight = glm::cross(scheduledForward, glm::vec3(0, 1, 0));
218 |         const glm::vec3 scheduledExit = harvesterRoute(6) + scheduledRight * 6.1f - scheduledForward
> * 1.2f;
219 |         r.check(horizontalDistance(center, scheduledExit + scheduledRight * 14.0f) < 0.001f &&
220 |             horizontalDistance(destination, center) < 0.001f,
221 |             "release exit and first group center must use the route and tangent at exactly
> scheduled time");
222 |         advance(0.25f);
223 |         r.check(std::abs(horizontalDistance(g_mission.crew.front().pos, start) - 3.8f * 0.25f) <
> 0.01f,
224 |             "released crew must run outward at 3.8 meters per second");
225 |         advance(5.74f);
226 |         r.check(count(CrewState::Inside) == 35, "second group must remain inside before its
> thirteen-second release");
227 |         advance(1.02f);
228 |         r.check(g_mission.groups[1].released && count(CrewState::Inside) == 33, "second release must
> evacuate two crew together");
229 |         advance(106.0f);
230 |         r.check(count(CrewState::Waiting) == 36, "all real groups must eventually wait at separate
> fixed clusters");
231 |         r.check(glm::distance(g_mission.crew.front().destination, destination) < 0.001f &&
232 |             glm::distance(g_mission.groups.front().center, center) < 0.001f,
233 |             "released destinations and group centers must never follow the harvester");
234 |         bool atDestinations = true;
235 |         for (const auto& c : g_mission.crew)
236 |             atDestinations = atDestinations && horizontalDistance(c.pos, c.destination) < 0.15f &&
237 |                 near(c.pos.y, getDuneHeight(c.pos.x, c.pos.z));
238 |         r.check(atDestinations, "crew must arrive at their destinations and follow terrain height");
239 |         r.check(horizontalDistance(g_mission.groups.front().center, g_mission.groups.back().center)
> > 20,
240 |             "scheduled releases must produce different fixed evacuation clusters");
241 |         const glm::vec3 waitingPosition = g_mission.crew.front().pos;
242 |         advance(5);
243 |         r.check(glm::distance(waitingPosition, g_mission.crew.front().pos) < 0.001f,
244 |             "waiting crew must stay fixed while harvester and worm keep moving");
245 |         reconciled(r);
246 |     });
247 |
248 |     r.run("Hover pickup requires Space and caps at eight", [&] {
249 |         readyCrew();
250 |         hover(staging());
251 |         advance(1.0f);
252 |         r.check(g_mission.aboard == 0, "hovering without Space must not pick up crew");
253 |         g_keys[GLFW_KEY_SPACE] = true;
254 |         advance(0.3f);
255 |         r.check(g_mission.aboard == 0 && g_mission.pickup > 0, "pickup must require a dwell
> interval");
256 |         g_keys[GLFW_KEY_SPACE] = false;
257 |         advance(0.05f);
258 |         r.check(near(g_mission.pickup, 0), "releasing Space must reset pickup progress");
259 |         g_keys[GLFW_KEY_SPACE] = true;
260 |         advance(0.5f);
261 |         r.check(g_mission.aboard == 1 && count(CrewState::Aboard) == 1, "a steady low hover must
> winch a crew member");
262 |         r.check(g_mission.rescued == 0 && g_mission.best == 0, "pickup must not bank score");
263 |         advance(5.0f);
264 |         r.check(RESCUE_CAPACITY == 8 && g_mission.aboard == 8, "cabin capacity must be exactly
> eight");
265 |         r.check(count(CrewState::Waiting) == 28, "a full cabin must leave remaining crew on the
> ground");
266 |         reconciled(r);
267 |     });
268 |
269 |     r.run("Moving too fast disallows pickup", [&] {
270 |         readyCrew();
271 |         hover(staging());
272 |         g_keys[GLFW_KEY_SPACE] = true;
273 |         g_ornVel = glm::vec3(20, 0, 0);
274 |         updateMission(0.05f);
275 |         r.check(g_ornSpeed > 6, "test setup must remain above pickup speed limit");
276 |         r.check(g_mission.aboard == 0 && near(g_mission.pickup, 0), "moving aircraft must not
> accumulate winch progress");
277 |         reconciled(r);
278 |     });
279 |

```

```

280 |     r.run("High altitude blocks pickup until Space lowers", [&] {
281 |         readyCrew();
282 |         hover(staging(), 35.0f);
283 |         advance(0.75f);
284 |         r.check(g_mission.aboard == 0 && near(g_mission.altitude, 35), "high hover must remain too
> high without Space");
285 |         g_keys[GLFW_KEY_SPACE] = true;
286 |         advance(0.5f);
287 |         r.check(g_ornPos.y - getDuneHeight(g_ornPos.x, g_ornPos.z) > 11,
288 |             "initial lowering interval must still exceed pickup clearance");
289 |         r.check(g_mission.aboard == 0 && near(g_mission.pickup, 0), "Space must not pick up crew
> while clearance is too high");
290 |         advance(2.0f);
291 |         r.check(g_ornPos.y - getDuneHeight(g_ornPos.x, g_ornPos.z) <= 11, "Space must lower the
> aircraft into pickup range");
292 |         r.check(g_mission.aboard > 0 && g_mission.rescued == 0, "lowered hover must allow pickup
> without banking score");
293 |         reconciled(r);
294 |     });
295 |
296 |     r.run("Delivery banks score only at base", [&] {
297 |         readyCrew();
298 |         collect(3);
299 |         r.check(g_mission.aboard == 3 && g_mission.rescued == 0, "three collected crew must remain
> unbanked");
300 |         hover(g_mission.base + glm::vec3(15, 0, 0));
301 |         advance(1.4f);
302 |         r.check(g_mission.aboard == 3 && g_mission.rescued == 0 && near(g_mission.unload, 0),
303 |             "holding Space just outside base radius must not deliver");
304 |         hover(g_mission.base);
305 |         g_keys[GLFW_KEY_SPACE] = false;
306 |         advance(1.3f);
307 |         r.check(g_mission.rescued == 0, "delivery at base still requires Space");
308 |         g_keys[GLFW_KEY_SPACE] = true;
309 |         advance(0.6f);
310 |         r.check(g_mission.rescued == 0 && g_mission.unload > 0, "delivery must require unloading
> dwell time");
311 |         g_keys[GLFW_KEY_SPACE] = false;
312 |         advance(0.02f);
313 |         r.check(near(g_mission.unload, 0), "interrupting unloading must reset its progress");
314 |         deliver();
315 |         r.check(g_mission.rescued == 3 && g_mission.best == 3 && g_mission.aboard == 0, "base must
> bank delivered crew and update best");
316 |         r.check(count(CrewState::Safe) == 3 && g_mission.phase == MissionPhase::Flying, "delivered
> crew become Safe while rescue continues");
317 |         reconciled(r);
318 |     });
319 |
320 |     r.run("Swallowing loses Inside and local crew but spares remote waiting crew", [&] {
321 |         fresh();
322 |         for (auto& group : g_mission.groups) group.release = 1000;
323 |         for (auto& c : g_mission.crew) c.release = 1000;
324 |         g_mission.elapsed = g_mission.breachAt + 11.5f;
325 |         updateHarvester(g_mission.elapsed);
326 |         g_mission.crew[1].state = CrewState::Running;
327 |         g_mission.crew[1].pos = g_mission.crew[1].destination = wormPosition() + glm::vec3(40, 0,
> 0);
328 |         g_mission.crew[2].state = CrewState::Waiting;
329 |         g_mission.crew[2].pos = g_mission.crew[2].destination = g_mission.base;
330 |         g_mission.crew[3].state = CrewState::Waiting;
331 |         g_mission.crew[3].pos = g_mission.crew[3].destination = g_mission.base + glm::vec3(2, 0, 0);
332 |         updateMission(0.25f);
333 |         r.check(g_mission.lost == 0 && g_mission.phase == MissionPhase::Flying, "Inside and
> forty-meter crew must survive before swallow radius expands at twelve");
334 |         updateMission(0.25f);
335 |         r.check(near(wormAttackTime(), 12) && g_mission.lost == 34 && count(CrewState::Inside) == 0,
336 |             "swallowing must lose Inside crew and local outside crew within forty-four meters");
337 |         r.check(count(CrewState::Waiting) == 2 && g_mission.phase == MissionPhase::Flying &&
> g_mission.rescued == 0,
338 |             "remote waiting crew must survive harvester swallowing and keep extraction active");
339 |         advance(8);
340 |         hover(g_mission.base);
341 |         g_keys[GLFW_KEY_SPACE] = true;
342 |         advance(1.0f);
343 |         r.check(g_mission.aboard == 2 && g_mission.lost == 34, "remote survivors must remain
> rescuable after worm retreat");
344 |         deliver();
345 |         r.check(g_mission.rescued == 2 && g_mission.phase == MissionPhase::Debrief, "remote
> survivors must be bankable after swallowing");

```

```

346 |         reconciled(r);
347 |     });
348 |
349 |     r.run("Carried crew survive breach and can deliver", [&] {
350 |         readyCrew();
351 |         collect(8);
352 |         r.check(g_mission.aboard == 8, "test setup must carry eight crew");
353 |         for (auto& c : g_mission.crew) if (c.state != CrewState::Aboard) {
354 |             c.state = CrewState::Inside;
355 |             c.release = 1000;
356 |         }
357 |         g_keys[GLFW_KEY_SPACE] = false;
358 |         g_mission.elapsed = g_mission.breachAt + 11.75f;
359 |         updateMission(0.25f);
360 |         r.check(g_mission.lost == 28 && g_mission.aboard == 8, "breach must spare carried crew but
> consume those left behind");
361 |         r.check(g_mission.phase == MissionPhase::Flying && g_mission.rescued == 0, "survivors must
> still need extraction");
362 |         reconciled(r);
363 |         deliver();
364 |         r.check(g_mission.phase == MissionPhase::Debrief && g_mission.rescued == 8 && g_mission.best
> == 8,
365 |             "post-breach delivery must bank survivors and finish the mission");
366 |         reconciled(r);
367 |     });
368 |
369 |     r.run("Worm collision loses occupants but preserves banked score", [&] {
370 |         readyCrew();
371 |         collect(8);
372 |         deliver();
373 |         collect(8);
374 |         r.check(g_mission.rescued == 8 && g_mission.aboard == 8, "collision setup must include
> banked and carried crew");
375 |         g_keys[GLFW_KEY_SPACE] = false;
376 |         g_mission.elapsed = g_mission.breachAt + 4;
377 |         updateHarvester(g_mission.elapsed);
378 |         hover(wormPosition());
379 |         updateMission(0.01f);
380 |         r.check(g_mission.phase == MissionPhase::Debrief && g_mission.aboard == 0, "worm collision
> must end flight and empty cabin");
381 |         r.check(g_mission.lost == 28 && g_mission.rescued == 8 && g_mission.best == 8,
382 |             "collision must lose all unbanked crew and preserve Safe crew and best score");
383 |         reconciled(r);
384 |         const float elapsed = g_mission.elapsed;
385 |         advance(10.0f);
386 |         r.check(near(g_mission.elapsed, elapsed) && g_mission.lost == 28, "debrief must freeze time
> and avoid counting losses twice");
387 |     });
388 |
389 |     r.run("Extraction timeout reconciles all totals", [&] {
390 |         readyCrew();
391 |         collect(8);
392 |         deliver();
393 |         collect(8);
394 |         hover(g_mission.base + glm::vec3(20, 0, 0));
395 |         g_keys[GLFW_KEY_SPACE] = false;
396 |         for (auto& c : g_mission.crew) if (c.state == CrewState::Waiting) {
397 |             c.pos = c.destination = g_mission.base + glm::vec3(40, 0, 40);
398 |         }
399 |         g_mission.elapsed = g_mission.breachAt + 59.75f;
400 |         updateMission(0.125f);
401 |         r.check(g_mission.phase == MissionPhase::Flying && g_mission.aboard == 8 && g_mission.lost
> == 0 && count(CrewState::Waiting) == 20,
402 |             "carried and remote ground survivors must remain extractable before timeout");
403 |         reconciled(r);
404 |         updateMission(0.125f);
405 |         r.check(near(wormAttackTime(), 60) && g_mission.phase == MissionPhase::Debrief, "extraction
> must end at attack time sixty");
406 |         r.check(g_mission.rescued == 8 && g_mission.best == 8 && g_mission.lost == 28 &&
> g_mission.aboard == 0,
407 |             "timeout must lose both undelivered occupants and unresolved remote crew without
> removing banked score");
408 |         reconciled(r);
409 |     });
410 |
411 |     r.run("Retry resets counters and preserves best", [&] {
412 |         readyCrew();
413 |         collect(8);
414 |         deliver();

```



```

415 |         g_keys[GLFW_KEY_SPACE] = false;
416 |         g_mission.elapsed = g_mission.breachAt + 60;
417 |         updateMission(0);
418 |         r.check(g_mission.phase == MissionPhase::Debrief && g_mission.best == 8, "retry setup must
> finish with a banked best score");
419 |         g_mission.paused = true;
420 |         g_mission.boost = 0.2f;
421 |         g_mission.pickup = 0.7f;
422 |         g_mission.unload = 0.4f;
423 |         g_mission.target = 5;
424 |         g_ornVel = glm::vec3(10, 0, 0);
425 |         setMouseHover(1000, 0, 1000, 500);
426 |         g_mouseTurnRate = -50;
427 |         resetMission(true);
428 |         r.check(g_mission.phase == MissionPhase::Flying && !g_mission.paused && g_mission.wave == 1,
> "retry must restart the same wave unpause");
429 |         r.check(g_mission.best == 8 && g_mission.rescued == 0 && g_mission.aboard == 0 &&
> g_mission.lost == 0,
430 |             "retry must clear current counters but retain best");
431 |         r.check(near(g_mission.elapsed, 0) && near(g_mission.pickup, 0) && near(g_mission.unload, 0)
> && g_mission.target == -1,
432 |             "retry must reset clock and winch state");
433 |         r.check(near(g_mission.boost, 1) && near(g_mission.altitude, 8) && glm::length(g_ornVel) <
> 0.001f,
434 |             "retry must reset flight resources and velocity");
435 |         r.check(glm::length(g_mouseHover) < 0.001f && near(g_mouseTurnRate, 0) &&
> near(flightCameraPitch(), 12),
436 |             "retry must center mouse steering, clear turn inertia, and restore camera pitch");
437 |         r.check(glm::distance(g_ornPos, g_mission.base + glm::vec3(0, 8, 0)) < 0.001f &&
> count(CrewState::Inside) == 36,
438 |             "retry must return aircraft to base and replenish crew");
439 |         reconciled(r);
440 |     });
441 |
442 |     r.run("Next wave shortens timer with a floor", [&] {
443 |         fresh();
444 |         g_mission.best = 17;
445 |         finishMission();
446 |         resetMission(true, true);
447 |         r.check(g_mission.wave == 2 && near(g_mission.breachAt, 200), "next wave must increment wave
> and shorten timer by ten seconds");
448 |         r.check(g_mission.best == 17 && g_mission.phase == MissionPhase::Flying &&
> count(CrewState::Inside) == 36,
449 |             "next wave must retain best and initialize fresh crew");
450 |         for (int i = 0; i < 8; ++i) {
451 |             const float previous = g_mission.breachAt;
452 |             resetMission(true, true);
453 |             r.check(g_mission.breachAt <= previous && g_mission.breachAt >= 150, "later waves must
> never lengthen timer or fall below one hundred fifty");
454 |         }
455 |         r.check(g_mission.wave == 10 && near(g_mission.breachAt, 150), "high waves must clamp timer
> to one hundred fifty seconds");
456 |         reconciled(r);
457 |     });
458 |
459 |     r.run("Complete all thirty-six rescues through real state flow", [&] {
460 |         fresh();
461 |         advance(120);
462 |         r.check(count(CrewState::Waiting) == 36, "complete-rescue setup must use released and walked
> crew");
463 |         int banked = 0;
464 |         for (int flight = 0; flight < 5; ++flight) {
465 |             const int batch = std::min(8, 36 - banked);
466 |             for (int person = 0; person < batch; ++person) {
467 |                 int nearest = -1;
468 |                 float distance = std::numeric_limits<float>::infinity();
469 |                 for (int i = 0; i < int(g_mission.crew.size()); ++i) if (g_mission.crew[i].state ==
> CrewState::Waiting) {
470 |                     const float d = horizontalDistance(g_ornPos, g_mission.crew[i].pos);
471 |                     if (d < distance) { distance = d; nearest = i; }
472 |                 }
473 |                 if (nearest < 0) break;
474 |                 hover(g_mission.crew[nearest].pos);
475 |                 g_keys[GLFW_KEY_SPACE] = true;
476 |                 advance(0.45f);
477 |             }
478 |             r.check(g_mission.aboard == batch && g_mission.rescued == banked, "each staging visit
> must load the expected batch without banking it");
479 |             reconciled(r);

```



```

480 |         deliver();
481 |         banked += batch;
482 |         r.check(g_mission.rescued == banked && g_mission.aboard == 0, "each base visit must bank
> exactly its carried batch");
483 |         r.check(g_mission.phase == (banked == 36 ? MissionPhase::Debrief :
> MissionPhase::Flying),
484 |             "mission must remain active until the final batch is delivered");
485 |         reconciled(r);
486 |     }
487 |     r.check(g_mission.rescued == 36 && g_mission.best == 36 && g_mission.lost == 0 &&
> count(CrewState::Safe) == 36,
488 |         "all thirty-six crew must finish Safe with full banked score and no losses");
489 |     r.check(g_mission.elapsed < g_mission.breachAt, "state-flow simulation must finish before
> breach without changing its timer");
490 | });
491 |
492 |     r.run("Input-only round trips rescue all thirty-six before breach", [&] {
493 |         fresh();
494 |         r.check(g_mission.wave == 1 && near(g_mission.breachAt, 210), "playability must use the
> unmodified first mission timer");
495 |         // Neutral mouse keeps yaw zero. After reset, only keys and updateMission
496 |         // change game state; no hover(), collect(), or deliver() assistance.
497 |         const glm::vec3 base = g_mission.base;
498 |         constexpr float dt = 1.0f / 120.0f;
499 |         int steps = 0, travelSteps = 0, boostSteps = 0, directionsUsed = 0, directionReversals = 0;
500 |         bool finiteState = true, clearOfGround = true, neutralHeading = true;
501 |         float minimumClearance = g_ornPos.y - getDuneHeight(g_ornPos.x, g_ornPos.z);
502 |         auto finiteVector = [&](glm::vec3 v) {
503 |             return std::isfinite(v.x) && std::isfinite(v.y) && std::isfinite(v.z);
504 |         };
505 |         auto step = [&] {
506 |             if (g_keys[GLFW_KEY_W]) directionsUsed |= 1;
507 |             if (g_keys[GLFW_KEY_A]) directionsUsed |= 2;
508 |             if (g_keys[GLFW_KEY_S]) directionsUsed |= 4;
509 |             if (g_keys[GLFW_KEY_D]) directionsUsed |= 8;
510 |             updateMission(dt);
511 |             ++steps;
512 |             if (g_isBoosting) ++boostSteps;
513 |             finiteState = finiteState && finiteVector(g_ornPos) && finiteVector(g_ornVel) &&
514 |                 finiteVector(g_mission.base) && std::isfinite(g_ornYaw) &&
515 |                 std::isfinite(g_ornPitch) && std::isfinite(g_ornRoll) &&
516 |                 std::isfinite(g_ornSpeed) && std::isfinite(g_wingPhase) &&
517 |                 std::isfinite(g_mission.elapsed) && std::isfinite(g_mission.breachAt) &&
518 |                 std::isfinite(g_mission.altitude) && std::isfinite(g_mission.boost) &&
519 |                 std::isfinite(g_mission.pickup) && std::isfinite(g_mission.unload);
520 |             finiteState = finiteState && std::isfinite(g_mouseHover.x) &&
> std::isfinite(g_mouseHover.y) &&
521 |                 std::isfinite(g_mouseTurnRate);
522 |             neutralHeading = neutralHeading && near(g_ornYaw, 0) && near(g_mouseTurnRate, 0) &&
523 |                 glm::length(g_mouseHover) < 0.001f;
524 |             for (const auto& c : g_mission.crew)
525 |                 finiteState = finiteState && finiteVector(c.pos) && finiteVector(c.destination) &&
> std::isfinite(c.release);
526 |             const float clearance = g_ornPos.y - getDuneHeight(g_ornPos.x, g_ornPos.z);
527 |             clearOfGround = clearOfGround && std::isfinite(clearance) && clearance >= 3.0f - 0.001f;
528 |             minimumClearance = std::min(minimumClearance, clearance);
529 |         };
530 |         auto active = [&] {
531 |             return finiteState && g_mission.phase == MissionPhase::Flying && g_mission.elapsed <
> g_mission.breachAt;
532 |         };
533 |         auto fly = [&](glm::vec3 target) {
534 |             const int started = steps;
535 |             int previousX = 0, previousZ = 0;
536 |             float pulseX = 0, pulseZ = 0;
537 |             bool braking = false;
538 |             while (active() && steps - started < 12 * 120) {
539 |                 const glm::vec3 delta = target - g_ornPos;
540 |                 const float distance = horizontalDistance(target, g_ornPos);
541 |                 // SPACE has damping 12/s. Begin braking before the remaining
542 |                 // distance falls below the coast distance plus a 4-unit margin.
543 |                 braking = braking || distance < 4.0f + g_ornSpeed / 12.0f;
544 |                 for (bool& key : g_keys) key = false;
545 |                 g_keys[GLFW_KEY_SPACE] = braking;
546 |                 g_keys[GLFW_KEY_LEFT_SHIFT] = !braking && distance > 20 && g_mission.boost > 0.08f;
547 |                 if (distance <= 2.0f && g_ornSpeed < 0.5f) {
548 |                     travelSteps += steps - started;
549 |                     return true;
550 |                 }

```

```

551 |         if (distance > 2.0f) {
552 |             const float speed = braking ? 2.8f : (g_keys[GLFW_KEY_LEFT_SHIFT] ? 48.0f :
> 28.0f);
553 |             const float damping = braking ? 12.0f : 5.5f;
554 |             // Project into the aircraft's right/back axes, then express
555 |             // proportional commands through bool keys, not velocity edits.
556 |             const glm::vec3 backward = -flightForward();
557 |             const glm::vec3 right = glm::cross(flightForward(), glm::vec3(0, 1, 0));
558 |             auto axis = [&](float error, float velocity, float& pulse, int negative, int
> positive) {
559 |                 const float predicted = error - velocity / damping;
560 |                 if (std::abs(error) < 1.0f || predicted * error <= 0) { pulse = 0; return; }
561 |                 pulse += std::min(1.0f, std::abs(predicted) * (braking ? 2.0f : 4.0f) /
> speed);
562 |                 if (pulse >= 1) {
563 |                     g_keys[error < 0 ? negative : positive] = true;
564 |                     pulse -= 1;
565 |                 }
566 |             };
567 |             axis(glm::dot(delta, right), glm::dot(g_ornVel, right), pulseX, GLFW_KEY_A,
> GLFW_KEY_D);
568 |             axis(glm::dot(delta, backward), glm::dot(g_ornVel, backward), pulseZ,
> GLFW_KEY_W, GLFW_KEY_S);
569 |             }
570 |             const int commandX = int(g_keys[GLFW_KEY_D]) - int(g_keys[GLFW_KEY_A]);
571 |             const int commandZ = int(g_keys[GLFW_KEY_S]) - int(g_keys[GLFW_KEY_W]);
572 |             if (commandX) {
573 |                 if (previousX && previousX != commandX) ++directionReversals;
574 |                 previousX = commandX;
575 |             }
576 |             if (commandZ) {
577 |                 if (previousZ && previousZ != commandZ) ++directionReversals;
578 |                 previousZ = commandZ;
579 |             }
580 |             step();
581 |         }
582 |         travelSteps += steps - started;
583 |         return false;
584 |     };
585 |     auto holdSpace = [&] {
586 |         for (bool& key : g_keys) key = false;
587 |         g_keys[GLFW_KEY_SPACE] = true;
588 |         step();
589 |     };
590 |     auto seconds = [dt](int ticks) { return std::round(ticks * dt * 1000.0f) / 1000.0f; };
591 |     int flights = 0, clustersVisited = 0, departure = steps, outboundSteps = 0, boardingSteps =
> 0;
592 |     unsigned visitedGroups = 0;
593 |     while (active()) {
594 |         const int outside = count(CrewState::Running) + count(CrewState::Waiting);
595 |         const bool allReleased = count(CrewState::Inside) == 0;
596 |         if (g_mission.aboard == RESCUE_CAPACITY || (allReleased && outside == 0 &&
> g_mission.aboard > 0)) {
597 |             const int batch = g_mission.aboard, banked = g_mission.rescued, boarded = steps;
598 |             r.check(batch == 8 || (allReleased && outside == 0), "return only with a full cabin
> or the final released survivors");
599 |             const bool reachedBase = fly(base);
600 |             const int arrivedBase = steps;
601 |             r.check(reachedBase, "input-driven return leg must reach base without timing out");
602 |             if (!reachedBase) break;
603 |             while (active() && g_mission.aboard > 0) holdSpace();
604 |             r.check(g_mission.rescued == banked + batch && g_mission.aboard == 0,
605 |                 "Space at base must bank exactly the actual carried load");
606 |             reconciled(r);
607 |             std::cout << "Input flight " << ++flights
608 |                 << ": outbound=" << seconds(outboundSteps)
609 |                 << "s boarding/waiting=" << seconds(boardingSteps)
610 |                 << "s return=" << seconds(arrivedBase - boarded)
611 |                 << "s unloading=" << seconds(steps - arrivedBase)
612 |                 << "s batch=" << batch << " rescued=" << g_mission.rescued
613 |                 << " elapsed=" << g_mission.elapsed << "s trip=" << seconds(steps -
> departure) << "s\n";
614 |             departure = steps;
615 |             outboundSteps = boardingSteps = 0;
616 |             continue;
617 |         }
618 |         int nearest = -1;
619 |         float nearestDistance = std::numeric_limits<float>::infinity();
620 |         for (int i = 0; i < int(g_mission.crew.size()); ++i) {

```

```

621 |         const auto& c = g_mission.crew[i];
622 |         if (c.state != CrewState::Running && c.state != CrewState::Waiting) continue;
623 |         const float distance = horizontalDistance(g_ornPos, c.pos);
624 |         if (distance < nearestDistance) { nearestDistance = distance; nearest = i; }
625 |     }
626 |     if (nearest < 0) {
627 |         holdSpace();
628 |         ++boardingSteps;
629 |         continue;
630 |     }
631 |     const auto& crew = g_mission.crew[nearest];
632 |     const int group = crew.group;
633 |     const glm::vec3 pickupPoint = crew.state == CrewState::Running ? crew.destination :
> crew.pos;
634 |     const int started = steps;
635 |     const bool reachedCrew = fly(pickupPoint);
636 |     outboundSteps += steps - started;
637 |     r.check(reachedCrew, "input-driven outbound leg must reach the nearest released crew's
> fixed cluster");
638 |     if (!reachedCrew) break;
639 |     ++clustersVisited;
640 |     if (group >= 0 && group < 15) visitedGroups |= 1u << group;
641 |     const int arrivedCrew = steps;
642 |     while (active() && g_mission.aboard < RESCUE_CAPACITY && steps - arrivedCrew < 3 * 120
> &&
643 |         (g_mission.crew[nearest].state == CrewState::Running ||
> g_mission.crew[nearest].state == CrewState::Waiting))
644 |         holdSpace();
645 |         boardingSteps += steps - arrivedCrew;
646 |     }
647 |     for (bool& key : g_keys) key = false;
648 |     r.check(finiteState, "all flight and crew float state must remain finite at every substep");
649 |     r.check(neutralHeading, "neutral-hover WASD rescue route must never auto-align the nose to
> its velocity");
650 |     r.check(clearOfGround, "aircraft must maintain at least three units of terrain clearance at
> every substep");
651 |     r.check(directionsUsed == 15 && boostSteps > 0, "round trips must exercise all WASD
> directions and actual SHIFT boosting");
652 |     r.check(directionReversals == 0, "each flight leg must settle without reversing cardinal
> input or oscillating about its target");
653 |     r.check(g_mission.rescued == 36 && g_mission.lost == 0 && count(CrewState::Safe) == 36,
654 |         "real input simulation must rescue all thirty-six crew without losses");
655 |     r.check(g_mission.phase == MissionPhase::Debrief && g_mission.best == 36 && g_mission.aboard
> == 0,
656 |         "final partial delivery must end the mission with a fully banked best score");
657 |     r.check(g_mission.elapsed < 210 && near(g_mission.breachAt, 210), "input-only rescue must
> finish before the unchanged 210-second breach");
658 |     r.check(flights == 5 && clustersVisited >= 15 && visitedGroups == (1u << 15) - 1,
659 |         "real input route must visit the fifteen released groups and bank four full loads
> plus a final partial load");
660 |     reconciled(r);
661 |     std::cout << "Input-only rescue: travel=" << seconds(travelSteps)
662 |         << "s elapsed=" << g_mission.elapsed << "s rescued=" << g_mission.rescued
663 |         << "/36 aboard=" << g_mission.aboard << "lost=" << g_mission.lost
664 |         << "flights=" << flights << "clusters=" << clustersVisited
665 |         << "min-clearance=" << minimumClearance << "approach-reversals=" <<
> directionReversals << '\n';
666 |     });
667 |
668 |     r.run("Harvester follows analytic terrain route and freezes at breach", [&] {
669 |         fresh();
670 |         for (float t : {0.0f, 6.0f, 25.0f, 60.0f, 104.0f, 150.0f, 210.0f}) {
671 |             const float x = -40.0f + 0.62f * t + 15.0f * std::sin(0.045f * t);
672 |             const float z = -55.0f + 22.0f * std::sin(0.032f * t) + 9.0f * std::sin(0.071f * t);
673 |             const glm::vec3 expected(x, getDuneHeight(x, z), z);
674 |             r.check(glm::distance(harvesterRoute(t), expected) < 0.001f, "route must match both
> sinusoidal coordinates and terrain height");
675 |             const float yaw = glm::radians(harvesterRouteYaw(t));
676 |             const glm::vec3 heading(-std::sin(yaw), 0, -std::cos(yaw));
677 |             const glm::vec3 tangent(0.62f + 0.675f * std::cos(0.045f * t), 0,
678 |                 0.704f * std::cos(0.032f * t) + 0.639f * std::cos(0.071f * t));
679 |             r.check(std::isfinite(yaw) && glm::dot(heading, glm::normalize(tangent)) > 0.99999f,
680 |                 "harvester yaw must align model negative-Z forward with the analytic tangent");
681 |             updateHarvester(t);
682 |             r.check(glm::distance(harvesterPosition(), expected) < 0.001f &&
683 |                 std::abs(std::remainder(g_mission.harvesterYaw - harvesterRouteYaw(t), 360.0f))
> < 0.001f,
684 |                 "elapsed-time harvester helper must update world position and tangent yaw");
685 |         }

```

```

686 |         const glm::vec3 atBreach = harvesterPosition();
687 |         const float yawAtBreach = g_mission.harvesterYaw, travelAtBreach =
> g_mission.harvesterTravel;
688 |         updateHarvester(g_mission.breachAt + 50);
689 |         r.check(glm::distance(harvesterPosition(), atBreach) < 0.001f &&
> near(g_mission.harvesterYaw, yawAtBreach) &&
690 |         near(g_mission.harvesterTravel, travelAtBreach), "harvester pose and travel must
> freeze at breach");
691 |         fresh();
692 |         float previousGap = 221;
693 |         for (float t : {0.0f, 1.0f, 30.0f, 75.0f, 104.0f, 150.0f, 209.0f, 210.0f, 230.0f}) {
694 |             g_mission.elapsed = t;
695 |             updateHarvester(t);
696 |             const float expected = 220.0f - 202.0f * glm::clamp(t / g_mission.breachAt, 0.0f, 1.0f);
697 |             r.check(near(wormGap(), expected) && wormGap() <= previousGap,
698 |             "worm pursuit gap must shrink monotonically and linearly from 220 to 18, then
> clamp");
699 |             r.check(horizontalDistance(wormPosition(), harvesterPosition() - glm::vec3(expected, 0,
> 0)) < 0.001f,
700 |             "pursuing worm must remain the current gap west of the moving harvester");
701 |             r.check(near(g_mission.wormYaw, 90) && near(wormPosition().y,
> getDuneHeight(wormPosition().x, wormPosition().z)),
702 |             "pursuing worm must face east at ninety degrees and follow terrain");
703 |             previousGap = wormGap();
704 |         }
705 |     });
706 |
707 |     r.run("Scheduled releases have no frame-spawn or destination drift", [&] {
708 |         std::vector<Evacuee> reference;
709 |         std::vector<EvacuationGroup> groups;
710 |         for (int frequency : {30, 60, 120}) {
711 |             fresh();
712 |             // Absolute ticks avoid elapsed accumulation differences in the fixture.
713 |             // The real controller above never edits time or any world state.
714 |             for (int tick = 1; tick <= 30 * frequency; ++tick) {
715 |                 g_mission.elapsed = float(tick - 1) / float(frequency);
716 |                 updateMission(1.0f / float(frequency));
717 |             }
718 |             if (reference.empty()) {
719 |                 reference = g_mission.crew;
720 |                 groups = g_mission.groups;
721 |             } else {
722 |                 for (std::size_t i = 0; i < reference.size(); ++i)
723 |                     r.check(g_mission.crew[i].state == reference[i].state &&
724 |                     horizontalDistance(g_mission.crew[i].pos, reference[i].pos) < 0.01f &&
725 |                     glm::distance(g_mission.crew[i].destination, reference[i].destination) <
> 0.001f,
726 |                     "equal absolute time must give identical release, walk position, and
> scheduled destination across frame rates");
727 |                 for (std::size_t i = 0; i < groups.size(); ++i)
728 |                     r.check(g_mission.groups[i].released == groups[i].released &&
729 |                     glm::distance(g_mission.groups[i].center, groups[i].center) < 0.001f,
730 |                     "group center must be computed at scheduled route pose, not at the frame
> that notices release");
731 |             }
732 |         }
733 |         fresh();
734 |         g_mission.elapsed = 5.99f;
735 |         updateHarvester(g_mission.elapsed);
736 |         updateMission(0.02f);
737 |         const glm::vec3 shortlyAfterRelease = g_mission.crew.front().pos;
738 |         const glm::vec3 fixedDestination = g_mission.crew.front().destination;
739 |         const glm::vec3 fixedCenter = g_mission.groups.front().center;
740 |         fresh();
741 |         g_mission.elapsed = 5.75f;
742 |         updateHarvester(g_mission.elapsed);
743 |         updateMission(0.26f);
744 |         r.check(horizontalDistance(g_mission.crew.front().pos, shortlyAfterRelease) < 0.001f &&
745 |         glm::distance(g_mission.crew.front().destination, fixedDestination) < 0.001f &&
746 |         glm::distance(g_mission.groups.front().center, fixedCenter) < 0.001f,
747 |         "crossing six-second release in a coarse frame must only walk the time since
> scheduled release");
748 |     });
749 |
750 |     r.run("Pursuing worm applies local crew and aircraft collision envelopes", [&] {
751 |         fresh();
752 |         for (auto& group : g_mission.groups) group.release = 1000;
753 |         for (auto& c : g_mission.crew) c.release = 1000;
754 |         g_mission.elapsed = 100;

```

```

755 |         updateHarvester(g_mission.elapsed);
756 |         for (int i = 0; i < 2; ++i) {
757 |             auto& c = g_mission.crew[i];
758 |             c.state = i == 0 ? CrewState::Running : CrewState::Waiting;
759 |             c.pos = c.destination = wormPosition() + glm::vec3(i == 0 ? 15.0f : 17.0f, 0, 0);
760 |         }
761 |         updateMission(0.01f);
762 |         r.check(g_mission.crew[0].state == CrewState::Lost && g_mission.crew[1].state ==
> CrewState::Waiting && g_mission.lost == 1,
763 |             "approaching worm must kill outside crew within16 meters but spare those beyond16");
764 |         r.check(count(CrewState::Inside) == 34 && g_mission.phase == MissionPhase::Flying,
765 |             "pursuit must not swallow Inside crew or end a remote aircraft's mission");
766 |         hover(wormPosition(), 30);
767 |         updateMission(0.01f);
768 |         r.check(g_mission.phase == MissionPhase::Flying && !g_mission.planeLost,
769 |             "approaching worm must spare an aircraft above22 meters terrain clearance");
770 |         hover(wormPosition(), 21);
771 |         updateMission(0.01f);
772 |         r.check(g_mission.phase == MissionPhase::Debrief && g_mission.planeLost && g_mission.lost ==
> 36,
773 |             "approaching worm must collide with an aircraft within18 meters and below22 terrain
> clearance");
774 |         reconciled(r);
775 |
776 |         fresh();
777 |         for (auto& group : g_mission.groups) group.release = 1000;
778 |         for (auto& c : g_mission.crew) c.release = 1000;
779 |         g_mission.elapsed = g_mission.breachAt + 4;
780 |         updateHarvester(g_mission.elapsed);
781 |         for (int i = 0; i < 2; ++i) {
782 |             auto& c = g_mission.crew[i];
783 |             c.state = CrewState::Waiting;
784 |             c.pos = c.destination = wormPosition() + glm::vec3(i == 0 ? 31.0f : 33.0f, 0, 0);
785 |         }
786 |         hover(wormPosition(), 45);
787 |         updateMission(0.01f);
788 |         r.check(g_mission.crew[0].state == CrewState::Lost && g_mission.crew[1].state ==
> CrewState::Waiting,
789 |             "breach must kill local outside crew within32 meters but spare crew beyond32 before
> swallowing");
790 |         r.check(g_mission.phase == MissionPhase::Flying, "breach must spare aircraft above forty
> meters");
791 |         hover(wormPosition(), 39);
792 |         updateMission(0.01f);
793 |         r.check(g_mission.planeLost && g_mission.phase == MissionPhase::Debrief,
794 |             "active breach must collide with aircraft within24 meters below ground plus40");
795 |         reconciled(r);
796 |
797 |         fresh();
798 |         for (auto& group : g_mission.groups) group.release = 1000;
799 |         for (auto& c : g_mission.crew) c.release = 1000;
800 |         g_mission.elapsed = 8;
801 |         updateHarvester(g_mission.elapsed);
802 |         auto& remote = g_mission.crew.front();
803 |         remote.state = CrewState::Waiting;
804 |         remote.pos = remote.destination = wormPosition();
805 |         updateMission(0);
806 |         r.check(remote.state == CrewState::Waiting, "pursuit crew hazards must remain inactive at
> exactly eight seconds");
807 |         updateMission(0.01f);
808 |         r.check(remote.state == CrewState::Lost && g_mission.lost == 1, "pursuit crew hazard must
> activate only after eight seconds");
809 |
810 |         fresh();
811 |         for (auto& group : g_mission.groups) group.release = 1000;
812 |         for (auto& c : g_mission.crew) c.release = 1000;
813 |         g_mission.elapsed = g_mission.breachAt + 18;
814 |         updateHarvester(g_mission.elapsed);
815 |         g_mission.crew.front().state = CrewState::Waiting;
816 |         g_mission.crew.front().pos = g_mission.crew.front().destination = wormPosition();
817 |         hover(g_mission.base);
818 |         updateMission(0);
819 |         r.check(g_mission.lost == 35 && count(CrewState::Waiting) == 1 && g_mission.phase ==
> MissionPhase::Flying,
820 |             "outside crew must survive within the former swallow radius once attack reaches
> eighteen");
821 |         reconciled(r);
822 |     });
823 |

```

```

824 |     r.run("Cursor normalization, deadzone, and resize are arithmetic only", [&] {
825 |         fresh();
826 |         const glm::vec3 position = g_ornPos;
827 |         const float yaw = g_ornYaw, elapsed = g_mission.elapsed;
828 |         setMouseHover(500, 250, 1000, 500);
829 |         r.check(glm::length(g_mouseHover) < 0.001f, "window center must produce neutral hover");
830 |         for (double x : {440.0, 460.0, 500.0, 540.0, 560.0}) {
831 |             setMouseHover(x, 250, 1000, 500);
832 |             r.check(near(g_mouseHover.x, 0), "horizontal +/-0.12 deadzone must not steer");
833 |         }
834 |         for (double y : {220.0, 230.0, 250.0, 270.0, 280.0}) {
835 |             setMouseHover(500, y, 1000, 500);
836 |             r.check(near(g_mouseHover.y, 0), "vertical +/-0.12 deadzone must remain centered");
837 |         }
838 |         setMouseHover(570, 250, 1000, 500);
839 |         r.check(g_mouseHover.x > 0 && g_mouseHover.x < 1, "just beyond right deadzone must gently
> steer right");
840 |         setMouseHover(430, 250, 1000, 500);
841 |         r.check(g_mouseHover.x < 0 && g_mouseHover.x > -1, "just beyond left deadzone must gently
> steer left");
842 |         setMouseHover(0, 250, 1000, 500);
843 |         r.check(near(g_mouseHover.x, -1) && near(g_mouseHover.y, 0), "left window edge must produce
> full left hover");
844 |         setMouseHover(1000, 250, 1000, 500);
845 |         r.check(near(g_mouseHover.x, 1) && near(g_mouseHover.y, 0), "right window edge must produce
> full right hover");
846 |         setMouseHover(800, 100, 1000, 500);
847 |         const glm::vec2 original = g_mouseHover;
848 |         r.check(near(original.x, 0.48f / 0.88f) && near(original.y, -0.48f / 0.88f),
849 |             "cursor values beyond the deadzone must rescale the remaining range to full
> normalized steering");
850 |         setMouseHover(1600, 200, 2000, 1000);
851 |         r.check(glm::distance(g_mouseHover, original) < 0.001f, "resizing must preserve normalized
> hover at equivalent coordinates");
852 |         setMouseHover(1000, 500, 2000, 1000);
853 |         r.check(glm::length(g_mouseHover) < 0.001f, "new window center must remain neutral after
> resize");
854 |         r.check(glm::distance(g_ornPos, position) < 0.001f && near(g_ornYaw, yaw) &&
> near(g_mission.elapsed, elapsed) && near(g_mouseTurnRate, 0),
855 |             "cursor arithmetic alone must not update flight, mission time, or turn inertia");
856 |     });
857 |
858 |     r.run("Outside and invalid cursor coordinates clear both hover axes", [&] {
859 |         fresh();
860 |         const double nan = std::numeric_limits<double>::quiet_NaN();
861 |         const double infinity = std::numeric_limits<double>::infinity();
862 |         struct Cursor { double x, y; int width, height; };
863 |         const Cursor invalid[] = {
864 |             {-0.01, 250, 1000, 500}, {1000.01, 250, 1000, 500},
865 |             {500, -0.01, 1000, 500}, {500, 500.01, 1000, 500},
866 |             {nan, 250, 1000, 500}, {500, nan, 1000, 500},
867 |             {infinity, 250, 1000, 500}, {-infinity, 250, 1000, 500},
868 |             {500, infinity, 1000, 500}, {500, -infinity, 1000, 500},
869 |             {0, 0, 0, 500}, {0, 0, 1000, 0}, {0, 0, -1, 500}, {0, 0, 1000, -1}
870 |         };
871 |         for (const auto& cursor : invalid) {
872 |             setMouseHover(1000, 0, 1000, 500);
873 |             r.check(glm::length(g_mouseHover) > 0.5f, "invalid-input setup must first have active
> hover");
874 |             setMouseHover(cursor.x, cursor.y, cursor.width, cursor.height);
875 |             r.check(std::isfinite(g_mouseHover.x) && std::isfinite(g_mouseHover.y) &&
> glm::length(g_mouseHover) < 0.001f,
876 |                 "outside, nonfinite, or invalid-size cursor input must clear both axes without
> NaNs");
877 |         }
878 |     });
879 |
880 |     r.run("Mouse hover turns stationary aircraft without clicks", [&] {
881 |         for (int side : {-1, 1}) {
882 |             fresh();
883 |             const glm::vec3 position = g_ornPos;
884 |             setMouseHover(side < 0 ? 0 : 1000, 250, 1000, 500);
885 |             advance(0.5f);
886 |             r.check(g_ornYaw * side < -1 && g_mouseTurnRate * side < -1,
887 |                 "right hover must decrease yaw and left hover must increase it without any key
> or click");
888 |             r.check(near(g_mouseTurnRate, -side * 80.0f * (1.0f - std::exp(-8.0f * 0.5f))),
889 |                 "hover yaw rate must exponentially approach eighty degrees per second at rate
> eight");

```



```

890 |         r.check(horizontalDistance(g_ornPos, position) < 0.001f && near(g_ornSpeed, 0),
891 |             "stationary hover steering must rotate without creating translation");
892 |         r.check(flightForward().x * side > 0 && flightForward().z < 0,
893 |             "right hover must face positive X and left hover negative X from initial
> negative-Z heading");
894 |         setMouseHover(500, 250, 1000, 500);
895 |         advance(2.0f);
896 |         const float steeredYaw = g_ornYaw;
897 |         const glm::vec3 steeredForward = flightForward();
898 |         const glm::vec3 departure = g_ornPos;
899 |         g_keys[GLFW_KEY_W] = true;
900 |         advance(0.5f);
901 |         const glm::vec3 displacement(g_ornPos.x - departure.x, 0, g_ornPos.z - departure.z);
902 |         r.check(glm::length(displacement) > 1 && glm::dot(glm::normalize(displacement),
> steeredForward) > 0.999f &&
903 |             displacement.x * side > 0,
904 |             "forward input after mouse steering must fly toward the hovered side in the new
> heading");
905 |         r.check(std::abs(g_ornYaw - steeredYaw) < 0.001f,
906 |             "centered forward flight must retain the mouse-selected nose heading");
907 |     }
908 | });
909 |
910 |     r.run("Arrow keys remain a stationary yaw backup", [&] {
911 |         for (int side : {-1, 1}) {
912 |             fresh();
913 |             const glm::vec3 position = g_ornPos;
914 |             g_keys[side < 0 ? GLFW_KEY_LEFT : GLFW_KEY_RIGHT] = true;
915 |             advance(0.5f);
916 |             r.check(g_ornYaw * side < -1 && g_mouseTurnRate * side < -1,
917 |                 "left/right arrow fallback must use the same yaw sign as mouse hover");
918 |             r.check(horizontalDistance(g_ornPos, position) < 0.001f && near(g_ornSpeed, 0),
919 |                 "arrow yaw fallback must work while stationary");
920 |         }
921 |     });
922 |
923 |     r.run("Forward, reverse, and strafe stay relative to yaw without auto-align", [&] {
924 |         const int keys[] = {GLFW_KEY_W, GLFW_KEY_S, GLFW_KEY_A, GLFW_KEY_D};
925 |         const float headings[] = {0, 90, -90, 179, -181, 359, 361};
926 |         for (float heading : headings) {
927 |             const float radians = glm::radians(heading);
928 |             const glm::vec3 forward(-std::sin(radians), 0, -std::cos(radians));
929 |             const glm::vec3 right(std::cos(radians), 0, -std::sin(radians));
930 |             for (int key : keys) {
931 |                 fresh();
932 |                 g_ornYaw = heading;
933 |                 r.check(glm::distance(flightForward(), forward) < 0.001f, "flightForward must match
> the yaw convention including wraparound");
934 |                 const glm::vec3 expected = key == GLFW_KEY_W ? forward : key == GLFW_KEY_S ?
> -forward : key == GLFW_KEY_D ? right : -right;
935 |                 const glm::vec3 position = g_ornPos;
936 |                 g_keys[key] = true;
937 |                 advance(0.5f);
938 |                 const glm::vec3 displacement(g_ornPos.x - position.x, 0, g_ornPos.z - position.z);
939 |                 r.check(glm::length(g_ornVel) > 1 && glm::dot(glm::normalize(g_ornVel), expected) >
> 0.999f &&
940 |                     glm::length(displacement) > 1 && glm::dot(glm::normalize(displacement),
> expected) > 0.999f,
941 |                     "W/S and A/D must translate along aircraft forward/back and left/right
> axes");
942 |                 r.check(std::abs(std::remainder(g_ornYaw - heading, 360.0f)) < 0.001f &&
> near(g_mouseTurnRate, 0),
943 |                     "translation, especially reverse and strafe, must never turn the nose toward
> velocity");
944 |             }
945 |         }
946 |     });
947 |
948 |     r.run("Centered hover damps turn rate instead of snapping heading", [&] {
949 |         fresh();
950 |         setMouseHover(1000, 250, 1000, 500);
951 |         advance(0.5f);
952 |         const float yaw = g_ornYaw, rate = g_mouseTurnRate;
953 |         setMouseHover(500, 250, 1000, 500);
954 |         advance(0.25f);
955 |         r.check(near(g_mouseTurnRate, rate * std::exp(-8.0f * 0.25f)), "returning mouse to deadzone
> must exponentially damp yaw rate");
956 |         r.check(g_ornYaw < yaw && g_mouseTurnRate < 0, "turn inertia must decay smoothly without
> snapping nose back to center");

```



```

957 |         advance(2.0f);
958 |         r.check(std::abs(g_mouseTurnRate) < 0.001f, "centered hover must settle to stationary yaw");
959 |         const float settled = g_ornYaw;
960 |         advance(0.25f);
961 |         r.check(std::abs(g_ornYaw - settled) < 0.001f, "settled neutral hover must retain the last
> heading");
962 |     });
963 |
964 |     r.run("Hover turn smoothing is stable across frame rates", [&] {
965 |         float yaw[3], rate[3];
966 |         const int frequencies[] = {30, 60, 120};
967 |         for (int i = 0; i < 3; ++i) {
968 |             fresh();
969 |             setMouseHover(1000, 250, 1000, 500);
970 |             for (int step = 0; step < frequencies[i]; ++step) updateMission(1.0f / frequencies[i]);
971 |             yaw[i] = g_ornYaw;
972 |             rate[i] = g_mouseTurnRate;
973 |             r.check(std::abs(rate[i] + 80.0f * (1.0f - std::exp(-8.0f))) < 0.002f,
974 |                 "exponential target yaw rate must be independent of thirty, sixty, or
> one-hundred-twenty Hz updates");
975 |             const float integral = -80.0f * (1.0f - (1.0f - std::exp(-8.0f)) / 8.0f);
976 |             r.check(std::abs(yaw[i] - integral) < 0.002f, "integrated one-second yaw must match the
> continuous smoothed-turn solution");
977 |         }
978 |         r.check(std::abs(yaw[0] - yaw[2]) < 0.002f && std::abs(yaw[1] - yaw[2]) < 0.002f,
979 |             "yaw trajectories must agree across frame rates without a frame-count-dependent
> integration error");
980 |     });
981 |
982 |     r.run("Every camera stays behind heading with matching screen-right", [&] {
983 |         fresh();
984 |         const float headings[] = {0, 90, -90, 179, -181, 359, 361, 720};
985 |         for (float heading : headings) {
986 |             g_ornYaw = heading;
987 |             const glm::vec3 forward = flightForward();
988 |             const glm::vec3 right = glm::cross(forward, glm::vec3(0, 1, 0));
989 |             for (double mouseY : {0.0, 250.0, 500.0}) {
990 |                 setMouseHover(500, mouseY, 1000, 500);
991 |                 for (int mode = 0; mode < 3; ++mode) {
992 |                     const glm::vec3 offset = flightCameraOffset(mode);
993 |                     const glm::vec3 horizontal(offset.x, 0, offset.z);
994 |                     r.check(std::isfinite(offset.x) && std::isfinite(offset.y) &&
> std::isfinite(offset.z) && offset.y > 0,
995 |                         "all camera modes and vertical hover extremes must return a finite
> elevated offset");
996 |                     r.check(glm::length(horizontal) > 0.001f && glm::dot(glm::normalize(horizontal),
> forward) < -0.999f,
997 |                         "camera XZ offset must stay directly behind forward at every heading
> including wraparound");
998 |                     const glm::vec3 viewRight = glm::normalize(glm::cross(-offset, glm::vec3(0, 1,
> 0)));
999 |                     r.check(glm::dot(viewRight, right) > 0.999f, "camera view-right must match
> aircraft-right so right input moves right on screen");
1000 |                     if (mode == 0) r.check(near(glm::length(offset), 40), "follow camera distance
> must be forty units");
1001 |                     if (mode == 1) r.check(near(glm::length(offset - glm::vec3(0, 2.2f, 0)), 20),
> "close camera must use twenty units plus 2.2 units of height");
1002 |                     if (mode == 2) r.check(near(offset.y, 100) && near(glm::length(horizontal), 26),
> "overview camera must be one hundred units high and twenty-six behind");
1003 |                 }
1004 |             }
1005 |         }
1006 |     });
1007 |
1008 |     r.run("Vertical hover controls clamped pitch and reset centers it", [&] {
1009 |         fresh();
1010 |         setMouseHover(500, 250, 1000, 500);
1011 |         r.check(near(flightCameraPitch(), 12), "centered mouse must give default twelve-degree
> camera pitch");
1012 |         setMouseHover(500, 0, 1000, 500);
1013 |         const float topPitch = flightCameraPitch();
1014 |         r.check(near(std::abs(g_mouseHover.y), 1) && near(topPitch, glm::clamp(12.0f -
> g_mouseHover.y * 22.0f, 6.0f, 65.0f)),
1015 |             "top window edge must apply the specified vertical pitch mapping");
1016 |         setMouseHover(500, 500, 1000, 500);
1017 |         const float bottomPitch = flightCameraPitch();
1018 |         r.check(near(std::abs(g_mouseHover.y), 1) && near(bottomPitch, glm::clamp(12.0f -
> g_mouseHover.y * 22.0f, 6.0f, 65.0f)),
1019 |             "bottom window edge must apply the specified vertical pitch mapping");

```

```
1020 |         r.check(near(std::min(topPitch, bottomPitch), 6) && near(std::max(topPitch, bottomPitch),
> 34),
1021 |             "vertical mouse extremes must span the clamped six-to-thirty-four-degree pitch
> range");
1022 |         for (float orbit : {-100.0f, 0.0f, 15.0f, 100.0f}) {
1023 |             g_camOrbitPitch = orbit;
1024 |             r.check(near(flightCameraPitch(), glm::clamp(12.0f - g_mouseHover.y * 22.0f + orbit,
> 6.0f, 65.0f)),
1025 |                 "camera orbit pitch must combine with mouse pitch and clamp between six and
> sixty-five degrees");
1026 |         }
1027 |         for (bool start : {false, true}) {
1028 |             setMouseHover(1000, 0, 1000, 500);
1029 |             g_mouseTurnRate = -60;
1030 |             g_camOrbitPitch = 35;
1031 |             resetMission(start);
1032 |             r.check(glm::length(g_mouseHover) < 0.001f && near(g_mouseTurnRate, 0) && near(g_ornYaw,
> 0),
1033 |                 "title and playing resets must center both hover axes and clear yaw inertia");
1034 |             r.check(near(g_camOrbitPitch, 0) && near(flightCameraPitch(), 12), "reset must restore
> centered camera pitch without stale vertical hover");
1035 |         }
1036 |     });
1037 |
1038 |     std::cout << "Mission tests: " << (r.cases - r.failedCases) << '/' << r.cases << " cases passed,
> "
1039 |         << r.checks << " checks, " << r.failures << " failures\n";
1040 |     return r.failures == 0 ? 0 : 1;
1041 | }
```

Source Reference: CMakeLists.txt

```

1 | cmake_minimum_required(VERSION 3.20)
2 |
3 | project(Arrakis)
4 |
5 | set(CMAKE_CXX_STANDARD 17)
6 | set(CMAKE_CXX_STANDARD_REQUIRED ON)
7 |
8 | add_executable(
9 |     Arrakis
10 |     src/arrakis.cpp
11 |     third_party/glad/glad.c
12 |     third_party/miniaudio/miniaudio_impl.cpp
13 |     src/arrakis_audio.h
14 |     src/arrakis_game.h
15 |     src/arrakis_worm.h
16 |     src/arrakis_hud.h
17 |     tests/arrakis_tests.h
18 | )
19 |
20 | find_package(OpenGL REQUIRED)
21 | # Existing laboratory installs remain hints; every path can be overridden in CMake.
22 | find_path(GLAD_INCLUDE_DIR glad/glad.h HINTS "E:/glfw_nec/Include/include" REQUIRED)
23 | find_path(GLFW_INCLUDE_DIR GLFW/glfw3.h HINTS "E:/glfw_nec/Include/include" REQUIRED)
24 | find_path(GLM_INCLUDE_DIR glm/glm.hpp HINTS "E:/glm"
25 | > "${CMAKE_CURRENT_SOURCE_DIR}/vcpkg_installed/x64-windows/include" REQUIRED)
26 | find_library(GLFW_LIBRARY NAMES glfw3 glfw HINTS "E:/glfw_nec/Libs" REQUIRED)
27 | target_include_directories(Arrakis PRIVATE
28 |     ${GLAD_INCLUDE_DIR} ${GLFW_INCLUDE_DIR} ${GLM_INCLUDE_DIR}
29 |     "${CMAKE_CURRENT_SOURCE_DIR}/tests"
30 |     "${CMAKE_CURRENT_SOURCE_DIR}/third_party/miniaudio"
31 | )
32 | target_compile_definitions(
33 |     Arrakis
34 |     PRIVATE
35 |     GLFW_INCLUDE_NONE
36 | )
37 |
38 | target_link_libraries(
39 |     Arrakis
40 |     PRIVATE
41 |     ${GLFW_LIBRARY}
42 |     OpenGL::GL
43 | )
44 |
45 | # Keep runtime audio beside the executable for IDE, CTest, and standalone launches.
46 | add_custom_command(TARGET Arrakis POST_BUILD
47 |     COMMAND ${CMAKE_COMMAND} -E copy_directory
48 |         "${CMAKE_CURRENT_SOURCE_DIR}/assets/audio"
49 |         "${TARGET_FILE_DIR:Arrakis}/assets/audio"
50 | )
51 | set_property(TARGET Arrakis PROPERTY VS_DEBUGGER_WORKING_DIRECTORY "${TARGET_FILE_DIR:Arrakis}")
52 |
53 | include(CTest)
54 | if(BUILD_TESTING)
55 |     add_test(NAME RescueMissionLogic COMMAND Arrakis --test-game)
56 | endif()

```

Source Reference: Arrakis_001.vcxproj

```

1 | <?xml version="1.0" encoding="utf-8"?>
2 | <Project DefaultTargets="Build" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
3 |   <ItemGroup Label="ProjectConfigurations">
4 |     <ProjectConfiguration Include="Debug|Win32">
5 |       <Configuration>Debug</Configuration>
6 |       <Platform>Win32</Platform>
7 |     </ProjectConfiguration>
8 |     <ProjectConfiguration Include="Release|Win32">
9 |       <Configuration>Release</Configuration>
10 |      <Platform>Win32</Platform>
11 |    </ProjectConfiguration>
12 |    <ProjectConfiguration Include="Debug|x64">
13 |      <Configuration>Debug</Configuration>
14 |      <Platform>x64</Platform>
15 |    </ProjectConfiguration>
16 |    <ProjectConfiguration Include="Release|x64">
17 |      <Configuration>Release</Configuration>
18 |      <Platform>x64</Platform>
19 |    </ProjectConfiguration>
20 |  </ItemGroup>
21 |  <PropertyGroup Label="Globals">
22 |    <VCPProjectVersion>17.0</VCPProjectVersion>
23 |    <Keyword>Win32Proj</Keyword>
24 |    <ProjectGuid>{b763452b-a6ad-4665-b537-92667c2af24c}</ProjectGuid>
25 |    <RootNamespace>Arrakis001</RootNamespace>
26 |    <WindowsTargetPlatformVersion>10.0</WindowsTargetPlatformVersion>
27 |  </PropertyGroup>
28 |  <Import Project="$(VCTargetsPath)\Microsoft.Cpp.Default.props" />
29 |  <PropertyGroup Condition=" '$(Configuration)|$(Platform)'=='Debug|Win32' " Label="Configuration">
30 |    <ConfigurationType>Application</ConfigurationType>
31 |    <UseDebugLibraries>true</UseDebugLibraries>
32 |    <PlatformToolset>v143</PlatformToolset>
33 |    <CharacterSet>Unicode</CharacterSet>
34 |  </PropertyGroup>
35 |  <PropertyGroup Condition=" '$(Configuration)|$(Platform)'=='Release|Win32' " Label="Configuration">
36 |    <ConfigurationType>Application</ConfigurationType>
37 |    <UseDebugLibraries>>false</UseDebugLibraries>
38 |    <PlatformToolset>v143</PlatformToolset>
39 |    <WholeProgramOptimization>true</WholeProgramOptimization>
40 |    <CharacterSet>Unicode</CharacterSet>
41 |  </PropertyGroup>
42 |  <PropertyGroup Condition=" '$(Configuration)|$(Platform)'=='Debug|x64' " Label="Configuration">
43 |    <ConfigurationType>Application</ConfigurationType>
44 |    <UseDebugLibraries>true</UseDebugLibraries>
45 |    <PlatformToolset>v143</PlatformToolset>
46 |    <CharacterSet>Unicode</CharacterSet>
47 |  </PropertyGroup>
48 |  <PropertyGroup Condition=" '$(Configuration)|$(Platform)'=='Release|x64' " Label="Configuration">
49 |    <ConfigurationType>Application</ConfigurationType>
50 |    <UseDebugLibraries>>false</UseDebugLibraries>
51 |    <PlatformToolset>v143</PlatformToolset>
52 |    <WholeProgramOptimization>true</WholeProgramOptimization>
53 |    <CharacterSet>Unicode</CharacterSet>
54 |  </PropertyGroup>
55 |  <Import Project="$(VCTargetsPath)\Microsoft.Cpp.props" />
56 |  <ImportGroup Label="ExtensionSettings">
57 |  </ImportGroup>
58 |  <ImportGroup Label="Shared">
59 |  </ImportGroup>
60 |  <ImportGroup Label="PropertySheets" Condition=" '$(Configuration)|$(Platform)'=='Debug|Win32' ">
61 |    <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
62 |    > Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props') "
63 |    > Label="LocalAppDataPlatform" />
64 |  </ImportGroup>
65 |  <ImportGroup Label="PropertySheets" Condition=" '$(Configuration)|$(Platform)'=='Release|Win32' ">
66 |    <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
67 |    > Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props') "
68 |    > Label="LocalAppDataPlatform" />
69 |  </ImportGroup>
70 |  <ImportGroup Label="PropertySheets" Condition=" '$(Configuration)|$(Platform)'=='Debug|x64' ">
71 |    <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
72 |    > Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props') "
73 |    > Label="LocalAppDataPlatform" />
74 |  </ImportGroup>
75 |  <ImportGroup Label="PropertySheets" Condition=" '$(Configuration)|$(Platform)'=='Release|x64' ">
76 |    <Import Project="$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props"
77 |    > Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props') "
78 |    > Label="LocalAppDataPlatform" />
79 |  </ImportGroup>
80 | </Project>

```

```

> Condition="exists('$(UserRootDir)\Microsoft.Cpp.$(Platform).user.props')"
> Label="LocalAppDataPlatform" />
71 | </ImportGroup>
72 | <PropertyGroup Label="UserMacros" />
73 | <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Release|x64'">
74 | <IncludePath>E:\glfw_nec\Include\include;E:\glm;$(ProjectDir)\vcpkg_installed\x64-windows\include
> ;$(IncludePath)</IncludePath>
75 | <LibraryPath>E:\glfw_nec\Libs;$(LibraryPath)</LibraryPath>
76 | </PropertyGroup>
77 | <PropertyGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
78 | <IncludePath>E:\glfw_nec\Include\include;E:\glm;$(IncludePath)</IncludePath>
79 | <LibraryPath>E:\glfw_nec\Libs;$(LibraryPath)</LibraryPath>
80 | </PropertyGroup>
81 | <ItemDefinitionGroup>
82 | <ClCompile>
83 | <LanguageStandard>stdcpp17</LanguageStandard>
84 |
> <PreprocessorDefinitions>GLFW_INCLUDE_NONE;%(PreprocessorDefinitions)</PreprocessorDefinitions>
85 | <AdditionalIncludeDirectories>$(ProjectDir)\tests;$(ProjectDir)\third_party\miniaudio;%(Addition
> alIncludeDirectories)</AdditionalIncludeDirectories>
86 | </ClCompile>
87 | <PostBuildEvent>
88 | <Command>xcopy /E /I /Y "$(ProjectDir)\assets\audio" "$(OutDir)\assets\audio"</Command>
89 | </PostBuildEvent>
90 | </ItemDefinitionGroup>
91 | <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">
92 | <ClCompile>
93 | <WarningLevel>Level3</WarningLevel>
94 | <SDLCheck>true</SDLCheck>
95 |
> <PreprocessorDefinitions>WIN32;_DEBUG;_CONSOLE;%(PreprocessorDefinitions)</PreprocessorDefinitions>
96 | <ConformanceMode>true</ConformanceMode>
97 | </ClCompile>
98 | <Link>
99 | <SubSystem>Console</SubSystem>
100 | <GenerateDebugInformation>true</GenerateDebugInformation>
101 | </Link>
102 | </ItemDefinitionGroup>
103 | <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">
104 | <ClCompile>
105 | <WarningLevel>Level3</WarningLevel>
106 | <FunctionLevelLinking>true</FunctionLevelLinking>
107 | <IntrinsicFunctions>true</IntrinsicFunctions>
108 | <SDLCheck>true</SDLCheck>
109 |
> <PreprocessorDefinitions>WIN32;NDEBUG;_CONSOLE;%(PreprocessorDefinitions)</PreprocessorDefinitions>
110 | <ConformanceMode>true</ConformanceMode>
111 | </ClCompile>
112 | <Link>
113 | <SubSystem>Console</SubSystem>
114 | <GenerateDebugInformation>true</GenerateDebugInformation>
115 | </Link>
116 | </ItemDefinitionGroup>
117 | <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">
118 | <ClCompile>
119 | <WarningLevel>Level3</WarningLevel>
120 | <SDLCheck>true</SDLCheck>
121 | <PreprocessorDefinitions>_DEBUG;_CONSOLE;%(PreprocessorDefinitions)</PreprocessorDefinitions>
122 | <ConformanceMode>true</ConformanceMode>
123 | </ClCompile>
124 | <Link>
125 | <SubSystem>Console</SubSystem>
126 | <GenerateDebugInformation>true</GenerateDebugInformation>
127 |
> <AdditionalDependencies>glfw3.lib;opengl32.lib;%(AdditionalDependencies)</AdditionalDependencies>
128 | </Link>
129 | </ItemDefinitionGroup>
130 | <ItemDefinitionGroup Condition="'$(Configuration)|$(Platform)'=='Release|x64'">
131 | <ClCompile>
132 | <WarningLevel>Level3</WarningLevel>
133 | <FunctionLevelLinking>true</FunctionLevelLinking>
134 | <IntrinsicFunctions>true</IntrinsicFunctions>
135 | <SDLCheck>true</SDLCheck>
136 | <PreprocessorDefinitions>NDEBUG;_CONSOLE;%(PreprocessorDefinitions)</PreprocessorDefinitions>
137 | <ConformanceMode>true</ConformanceMode>
138 | </ClCompile>
139 | <Link>
140 | <SubSystem>Console</SubSystem>
141 | <GenerateDebugInformation>true</GenerateDebugInformation>

```

```
142 |  
    > <AdditionalDependencies>glfw3.lib;opengl32.lib;%{AdditionalDependencies}</AdditionalDependencies>  
143 |     </Link>  
144 | </ItemDefinitionGroup>  
145 | <ItemGroup>  
146 |     <ClCompile Include="src\arrakis.cpp" />  
147 |     <ClCompile Include="third_party\glad\glad.c" />  
148 |     <ClCompile Include="third_party\miniaudio\miniaudio_impl.cpp" />  
149 | </ItemGroup>  
150 | <ItemGroup>  
151 |     <None Include="vcpkg.json" />  
152 | </ItemGroup>  
153 | <ItemGroup>  
154 |     <ClInclude Include="src\arrakis_audio.h" />  
155 |     <ClInclude Include="src\arrakis_game.h" />  
156 |     <ClInclude Include="src\arrakis_worm.h" />  
157 |     <ClInclude Include="src\arrakis_hud.h" />  
158 |     <ClInclude Include="tests\arrakis_tests.h" />  
159 | </ItemGroup>  
160 | <Import Project="$(VCTargetsPath)\Microsoft.Cpp.targets" />  
161 | <ImportGroup Label="ExtensionTargets">  
162 | </ImportGroup>  
163 | </Project>
```

Source Reference: Arrakis_001.vcxproj.filters

```

1 | <?xml version="1.0" encoding="utf-8"?>
2 | <Project ToolsVersion="4.0" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
3 |   <ItemGroup>
4 |     <Filter Include="Source Files">
5 |       <UniqueIdentifier>{4FC737F1-C7A5-4376-A066-2A32D752A2FF}</UniqueIdentifier>
6 |       <Extensions>cpp;c;cc;cxx;c++;cppm;ixx;def;odl;idl;hpj;bat;asm;asmx</Extensions>
7 |     </Filter>
8 |     <Filter Include="Header Files">
9 |       <UniqueIdentifier>{93995380-89BD-4b04-88EB-625FBE52EBFB}</UniqueIdentifier>
10 |      <Extensions>h;hh;hpp;hxx;h++;hm;inl;inc;ipp;xsd</Extensions>
11 |    </Filter>
12 |    <Filter Include="Resource Files">
13 |      <UniqueIdentifier>{67DA6AB6-F800-4c08-8B7A-83BB121AAD01}</UniqueIdentifier>
14 |      <Extensions>rc;ico;cur;bmp;dlg;rc2;rct;bin;rgs;gif;jpg;jpeg;jpe;resx;tiff;tif;png;wav;mfcribbo
> n-ms</Extensions>
15 |    </Filter>
16 |  </ItemGroup>
17 |  <ItemGroup>
18 |    <ClCompile Include="src\arrakis.cpp">
19 |      <Filter>Source Files</Filter>
20 |    </ClCompile>
21 |    <ClCompile Include="third_party\glad\glad.c">
22 |      <Filter>Source Files</Filter>
23 |    </ClCompile>
24 |    <ClCompile Include="third_party\miniaudio\miniaudio_impl.cpp"><Filter>Source
> Files</Filter></ClCompile>
25 |  </ItemGroup>
26 |  <ItemGroup>
27 |    <None Include="vcpkg.json" />
28 |  </ItemGroup>
29 |  <ItemGroup>
30 |    <ClInclude Include="src\arrakis_audio.h"><Filter>Header Files</Filter></ClInclude>
31 |    <ClInclude Include="src\arrakis_game.h"><Filter>Header Files</Filter></ClInclude>
32 |    <ClInclude Include="src\arrakis_worm.h"><Filter>Header Files</Filter></ClInclude>
33 |    <ClInclude Include="src\arrakis_hud.h"><Filter>Header Files</Filter></ClInclude>
34 |    <ClInclude Include="tests\arrakis_tests.h"><Filter>Header Files</Filter></ClInclude>
35 |  </ItemGroup>
36 | </Project>

```


Source Reference: .gitignore

```
1 | # Out-of-source builds and generated configuration directories.
2 | /build/
3 | /build-*/
4 | /out/
5 | /x64/
6 | /x86/
7 | Debug/
8 | Release/
9 | RelWithDebInfo/
10 | MinSizeRel/
11 | CMakeFiles/
12 | CMakeCache.txt
13 | CMakeUserPresets.json
14 | cmake_install.cmake
15 | install_manifest.txt
16 | CTestTestfile.cmake
17 | Testing/
18 | _deps/
19 |
20 | # Installed dependency copies are reproducible from vcpkg.json.
21 | /vcpkg_installed/
22 |
23 | # Compiler/linker output. Do NOT globally ignore *.obj: it can be a 3D asset.
24 | *.o
25 | *.a
26 | *.exe
27 | *.dll
28 | *.so
29 | *.so.*
30 | *.dylib
31 | *.lib
32 | *.pdb
33 | *.idb
34 | *.ilk
35 | *.exp
36 | *.pch
37 | *.tlog
38 | *.obj.enc
39 | *.exe.recipe
40 | *.log
41 |
42 | # Personal IDE state; shared projects and editor configuration stay tracked.
43 | .vs/
44 | *.vcxproj.user
45 | *.suo
46 | *.user
47 | *.VC.db
48 | *.VC.VC.opendb
49 | .vscode/ipch/
50 |
51 | # Python/documentation caches; the requested PDF itself stays tracked.
52 | __pycache__/
53 | *.py[cod]
54 | .env/
55 | /docs/.cache/
56 |
57 | # OS metadata and temporary editor files.
58 | Thumbs.db
59 | ehthumbs.db
60 | Desktop.ini
61 | .DS_Store
62 | *~
63 | *.swp
64 | *.swo
```

Source Reference: .vscode/c_cpp_properties.json

```
1 | {
2 |   "configurations": [
3 |     {
4 |       "name": "Win32-MSVC",
5 |       "includePath": [
6 |         "${workspaceFolder}/**",
7 |         "E:/glfw_nec/Include/include",
8 |         "E:/glm"
9 |       ],
10 |      "compilerPath": "cl.exe",
11 |      "cStandard": "c17",
12 |      "cppStandard": "c++17",
13 |      "intelliSenseMode": "windows-msvc-x64"
14 |    }
15 |  ],
16 |   "version": 4
17 | }
```

Source Reference: .vscode/launch.json

```
1 | {  
2 |   "version": "0.2.0",  
3 |   "configurations": [  
4 |     {  
5 |       "name": "Arrakis (CMake Release)",  
6 |       "type": "cppvsdbg",  
7 |       "request": "launch",  
8 |       "args": [],  
9 |       "stopAtEntry": false,  
10 |      "console": "integratedTerminal",  
11 |      "cwd": "${workspaceFolder}",  
12 |      "program": "${workspaceFolder}/build/rescue/Release/Arrakis.exe"  
13 |    }  
14 |  ]  
15 | }
```

Source Reference: vcpkg.json

```
1 | {  
2 |   "dependencies": [  
3 |     "glm"  
4 |   ],  
5 |   "builtin-baseline": "b8b8df2201ad8509b81a830fe0957bcb98e06c27"  
6 | }
```

Source Reference: tools/generate_docs.py

```

1 | """Render project explanation chapters and complete numbered sources as a PDF."""
2 |
3 | from __future__ import annotations
4 |
5 | import argparse
6 | import hashlib
7 | import json
8 | import re
9 | import textwrap
10 | from datetime import datetime, timezone
11 | from pathlib import Path
12 | from xml.sax.saxutils import escape
13 |
14 | from reportlab.lib import colors
15 | from reportlab.lib.enums import TA_CENTER
16 | from reportlab.lib.pagesizes import A4
17 | from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
18 | from reportlab.lib.units import mm
19 | from reportlab.platypus import (
20 |     BaseDocTemplate, Frame, PageBreak, PageTemplate, Paragraph,
21 |     Preformatted, Spacer, Table, TableStyle,
22 | )
23 | from reportlab.platypus.tableofcontents import TableOfContents
24 |
25 | ROOT = Path(__file__).resolve().parents[1]
26 | CHAPTERS = (
27 |     "01-project-and-build.md", "02-renderer.md", "03-scene-and-runtime.md",
28 |     "04-game-and-tests.md", "05-hud-and-sandworm.md", "06-audio-and-vendor.md",
29 |     "07-documentation-tools.md",
30 | )
31 | SOURCE_PATHS = (
32 |     "src/arrakis.cpp", "src/arrakis_game.h", "src/arrakis_hud.h",
33 |     "src/arrakis_worm.h", "src/arrakis_audio.h", "tests/arrakis_tests.h",
34 |     "CMakeLists.txt", "Arrakis_001.vcxproj", "Arrakis_001.vcxproj.filters",
35 |     ".gitignore", ".vscode/c_cpp_properties.json", ".vscode/launch.json",
36 |     "vcpkg.json", "tools/generate_docs.py", "tools/requirements.txt",
37 |     "third_party/miniaudio/miniaudio_impl.cpp", "third_party/glad/glad.c",
38 |     "third_party/miniaudio/miniaudio.h",
39 | )
40 |
41 |
42 | def inline(text: str) -> str:
43 |     """Escape literal text before adding the supported Markdown inline tags."""
44 |     parts = re.split(r"(`^`+)", text)
45 |     rendered = []
46 |     for part in parts:
47 |         if part.startswith("`") and part.endswith("`"):
48 |             rendered.append('<font name="Courier">' + escape(part[1:-1]) + '</font>')
49 |         else:
50 |             safe = escape(part)
51 |             safe = re.sub(r"\*(.?)\*\*", r"<b>\1</b>", safe)
52 |             safe = re.sub(r"\[([^\]]+)\]\([([^\)]+)\)", r"\1 (\2)", safe)
53 |             rendered.append(safe)
54 |     return "".join(rendered)
55 |
56 |
57 | def source_rows(content: str, width: int = 100) -> list[str]:
58 |     """Wrap rather than truncate source; continuations retain their line number."""
59 |     rows = []
60 |     for number, line in enumerate(content.splitlines(), 1):
61 |         chunks = textwrap.wrap(
62 |             line.expandtabs(4), width=width, replace_whitespace=False,
63 |             drop_whitespace=False, break_long_words=True, break_on_hyphens=False,
64 |         ) or [""]
65 |         for index, chunk in enumerate(chunks):
66 |             prefix = f"{number:6d} | " if index == 0 else "          > "
67 |             rows.append(prefix + chunk)
68 |     return rows
69 |
70 |
71 | class ExplanationDocument(BaseDocTemplate):
72 |     """Provide page furniture, PDF bookmarks, and a paginated table of contents."""
73 |
74 |     def __init__(self, filename: str, styles: dict):
75 |         super().__init__(
76 |             filename, pagesize=A4, leftMargin=19 * mm, rightMargin=19 * mm,

```

```

77 |         topMargin=20 * mm, bottomMargin=19 * mm,
78 |         title="Arrakis: Complete Code and Functionality Explanation",
79 |         author="Arrakis Project Documentation", pageCompression=1,
80 |     )
81 |     frame = Frame(self.leftMargin, self.bottomMargin, self.width, self.height, id="body")
82 |     self.addPageTemplates(PageTemplate(id="default", frames=[frame], onPage=self.decorate))
83 |     self.styles = styles
84 |     self.heading_number = 0
85 |
86 |     def beforeDocument(self):
87 |         self.heading_number = 0
88 |
89 |     def decorate(self, canvas, document):
90 |         canvas.saveState()
91 |         canvas.setStrokeColor(colors.HexColor("#c9b99e"))
92 |         canvas.line(self.leftMargin, A4[1] - 14 * mm, A4[0] - self.rightMargin, A4[1] - 14 * mm)
93 |         canvas.setFillColor(colors.HexColor("#665b4d"))
94 |         canvas.setFont("Helvetica", 8)
95 |         canvas.drawString(self.leftMargin, A4[1] - 11 * mm, "ARRAKIS / CODE AND FUNCTIONALITY
> REFERENCE")
96 |         canvas.drawString(self.leftMargin, 11 * mm, "CSE 4102 | Application, tests, configuration,
> and library source")
97 |         canvas.drawRightString(A4[0] - self.rightMargin, 11 * mm, str(document.page))
98 |         canvas.restoreState()
99 |
100 |     def afterFlowable(self, flowable):
101 |         level = getattr(flowable, "toc_level", None)
102 |         if level is None:
103 |             return
104 |         self.heading_number += 1
105 |         key = f"section-{self.heading_number}"
106 |         title = flowable.getPlainText()
107 |         self.canv.bookmarkPage(key)
108 |         self.canv.addOutlineEntry(title, key, level=level, closed=level > 0)
109 |         self.notify("TOCEntry", (level, title, self.page, key))
110 |
111 |
112 |     def make_styles() -> dict:
113 |         base = getSampleStyleSheet()
114 |         styles = {"body": ParagraphStyle(
115 |             "Body", parent=base["BodyText"], fontName="Helvetica", fontSize=9,
116 |             leading=13.5, spaceAfter=6, splitLongWords=True,
117 |         )}
118 |         for level, size in ((1, 20), (2, 13), (3, 10.5), (4, 10)):
119 |             styles[f"h{level}"] = ParagraphStyle(
120 |                 f"H{level}", parent=base["Heading1"], fontName="Helvetica-Bold",
121 |                 fontSize=size, leading=size * 1.25, spaceBefore=12, spaceAfter=8,
122 |                 textColor=colors.HexColor("#77532d"), keepWithNext=True,
123 |             )
124 |             styles["code"] = ParagraphStyle("Code", fontName="Courier", fontSize=6.5, leading=8)
125 |             styles["source"] = ParagraphStyle("Source", fontName="Courier", fontSize=7.2, leading=9)
126 |             styles["table"] = ParagraphStyle("TableText", parent=styles["body"], fontSize=7.8, leading=10)
127 |             styles["title"] = ParagraphStyle(
128 |                 "CoverTitle", parent=styles["h1"], fontSize=30, leading=36, alignment=TA_CENTER,
129 |             )
130 |             styles["center"] = ParagraphStyle("Center", parent=styles["body"], alignment=TA_CENTER)
131 |         return styles
132 |
133 |
134 |     def heading(text: str, level: int, styles: dict) -> Paragraph:
135 |         paragraph = Paragraph(inline(text), styles[f"h{min(level, 4)}"])
136 |         if level <= 2:
137 |             paragraph.toc_level = level - 1
138 |         return paragraph
139 |
140 |
141 |     def markdown_flowables(content: str, styles: dict) -> list:
142 |         """Convert the chapter's Markdown subset without allowing raw HTML execution."""
143 |         output = []
144 |         lines = content.splitlines()
145 |         paragraph = []
146 |         index = 0
147 |
148 |         def flush():
149 |             if paragraph:
150 |                 output.append(Paragraph(inline(" ".join(paragraph)), styles["body"]))
151 |                 paragraph.clear()
152 |
153 |         while index < len(lines):

```

```

154 |         line = lines[index].strip()
155 |         if line.startswith("```"):
156 |             flush()
157 |             index += 1
158 |             code = []
159 |             while index < len(lines) and not lines[index].strip().startswith("```"):
160 |                 code.extend(textwrap.wrap(
161 |                     lines[index].expandtabs(4), width=109, replace_whitespace=False,
162 |                     drop_whitespace=False, break_on_hyphens=False,
163 |                 ) or [""])
164 |                 index += 1
165 |             for offset in range(0, len(code), 45):
166 |                 output.append(Preformatted("\n".join(code[offset:offset + 45]), styles["code"]))
167 |             output.append(Spacer(1, 6))
168 |         elif re.match(r"^[1,6] ", line):
169 |             flush()
170 |             marker, title = line.split(" ", 1)
171 |             output.append(heading(title, min(len(marker), 4), styles))
172 |         elif line.startswith("|") and line.endswith("|"):
173 |             flush()
174 |             rows = []
175 |             while index < len(lines) and lines[index].strip().startswith("|"):
176 |                 cells = [cell.strip() for cell in lines[index].strip().strip("|").split("|")]
177 |                 if not all(re.fullmatch(r"?-+:?", cell.replace(" ", "")) for cell in cells):
178 |                     rows.append([Paragraph(inline(cell), styles["table"]) for cell in cells])
179 |                 index += 1
180 |             index -= 1
181 |             if rows:
182 |                 columns = max(len(row) for row in rows)
183 |                 for row in rows:
184 |                     row.extend([""] * (columns - len(row)))
185 |                 table = Table(rows, colWidths=[(A4[0] - 38 * mm - 12) / columns] * columns,
> repeatRows=1)
186 |                 table.setStyle(TableStyle([
187 |                     ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#eee5d6")),
188 |                     ("VALIGN", (0, 0), (-1, -1), "TOP"),
189 |                     ("GRID", (0, 0), (-1, -1), 0.3, colors.HexColor("#d8cbbb")),
190 |                     ("TOPPADDING", (0, 0), (-1, -1), 5),
191 |                     ("BOTTPADDING", (0, 0), (-1, -1), 5),
192 |                 ]))
193 |                 output.extend([table, Spacer(1, 8)])
194 |         elif re.match(r"^[~*+]", line) or re.match(r"^\d+[\.]", line):
195 |             flush()
196 |             output.append(Paragraph(inline(line), styles["body"]))
197 |         elif not line or line in ("---", "****", "___"):
198 |             flush()
199 |         elif line.startswith("> "):
200 |             flush()
201 |             output.append(Paragraph(inline(line[2:]), styles["body"]))
202 |         else:
203 |             paragraph.append(line)
204 |             index += 1
205 |     flush()
206 |     return output
207 |
208 |
209 | def main() -> None:
210 |     parser = argparse.ArgumentParser(description=__doc__)
211 |     parser.add_argument("--output", type=Path, default=ROOT / "docs/Arrakis_Code_Explanation.pdf")
212 |     arguments = parser.parse_args()
213 |     chapters = [ROOT / "docs/chapters" / name for name in CHAPTERS]
214 |     sources = [ROOT / name for name in SOURCE_PATHS]
215 |     missing = [str(path) for path in chapters + sources if not path.is_file()]
216 |     if missing:
217 |         parser.error("Required documentation inputs are missing: " + ", ".join(missing))
218 |     if not arguments.output.parent.is_dir():
219 |         parser.error("Output parent directory must already exist")
220 |
221 |     styles = make_styles()
222 |     story = [Spacer(1, 42 * mm)]
223 |     story.append(Paragraph("ARRAKIS<br/>Harvester Down", styles["title"]))
224 |     story.append(Spacer(1, 10 * mm))
225 |     story.append(Paragraph("Complete Code and Functionality Explanation", styles["center"]))
226 |     story.append(Paragraph("Plain-language walkthroughs, mathematics, tests, build details,<br/>and
> complete numbered source references", styles["center"]))
227 |     story.append(Spacer(1, 12 * mm))
228 |     story.append(Paragraph("CSE 4102: Computer Graphics and Image Processing Laboratory",
> styles["center"]))
229 |     story.append(Paragraph("Documented source snapshot: " +

```



```

> datetime.now(timezone.utc).strftime("%Y-%m-%d UTC"), styles["center"]))
230 |     story.append(PageBreak())
231 |     story.append(Paragraph("Contents", styles["h1"]))
232 |     toc = TableOfContents()
233 |     toc.levelStyles = [
234 |         ParagraphStyle("TOCChapter", fontName="Helvetica-Bold", fontSize=9, leading=13,
> spaceBefore=5),
235 |         ParagraphStyle("TOCSection", fontName="Helvetica", fontSize=8, leading=11, leftIndent=12),
236 |     ]
237 |     story.append(toc)
238 |     for chapter in chapters:
239 |         story.append(PageBreak())
240 |         story.extend(markdown_flowables(chapter.read_text(encoding="utf-8"), styles))
241 |
242 |     manifest = {"generated_at": datetime.now(timezone.utc).isoformat(), "sources": [], "chapters":
> []}
243 |     story.append(PageBreak())
244 |     story.append(heading("Appendix A: Source Snapshot and Reading Rules", 1, styles))
245 |     story.append(Paragraph(
246 |         "The following appendices contain every line of the retained application, tests,
> build/editor "
247 |         "configuration, documentation generator, and vendored C/C++ sources. The left number is the
> "
248 |         "physical source line, not a PDF line. A &gt; continuation is the rest of that same source
> line; "
249 |         "long lines are wrapped, never omitted. SHA-256 fingerprints identify this exact snapshot. "
250 |         "Application semantics are explained in the preceding code-range chapters. The libraries are
> "
251 |         "explained by functional families; their full source is included for reference, not
> presented "
252 |         "as original game logic. Installed external GLFW, GLM and GLAD headers are dependencies, "
253 |         "not files implemented in this repository.", styles["body"],
254 |     ))
255 |     for path in sources:
256 |         data = path.read_bytes()
257 |         content = data.decode("utf-8-sig")
258 |         relative = path.relative_to(ROOT).as_posix()
259 |         entry = {"path": relative, "lines": len(content.splitlines()), "sha256":
> hashlib.sha256(data).hexdigest()}
260 |         manifest["sources"].append(entry)
261 |         story.append(Paragraph(inline(f"{relative}: {entry['lines']} lines; SHA-256
> {entry['sha256']}", styles["body"])))
262 |
263 |         for path, entry in zip(sources, manifest["sources"]):
264 |             story.append(PageBreak())
265 |             story.append(heading("Source Reference: " + entry["path"], 1, styles))
266 |             rows = source_rows(path.read_text(encoding="utf-8-sig"))
267 |             for offset in range(0, len(rows), 72):
268 |                 story.append(Preformatted("\n".join(rows[offset:offset + 72]), styles["source"]))
269 |
270 |         for chapter in chapters:
271 |             data = chapter.read_bytes()
272 |             manifest["chapters"].append({
273 |                 "path": chapter.relative_to(ROOT).as_posix(), "sha256":
> hashlib.sha256(data).hexdigest(),
274 |                 "words": len(data.decode("utf-8").split()),
275 |             })
276 |             document = ExplanationDocument(str(arguments.output), styles)
277 |             document.multiBuild(story, maxPasses=6)
278 |             from pypdf import PdfReader
279 |             manifest["pdf"] = {
280 |                 "path": str(arguments.output.relative_to(ROOT)) if arguments.output.is_relative_to(ROOT)
> else str(arguments.output),
281 |                 "pages": len(PdfReader(arguments.output).pages), "bytes": arguments.output.stat().st_size,
282 |             }
283 |             (ROOT / "docs/documentation_manifest.json").write_text(json.dumps(manifest, indent=2) + "\n",
> encoding="utf-8")
284 |             print(f"Created {arguments.output}: {manifest['pdf']['pages']} pages,
> {manifest['pdf']['bytes']:,} bytes")
285 |             print(f"Included {len(sources)} source/config files and {len(chapters)} explanation chapters")
286 |
287 |
288 | if __name__ == "__main__":
289 |     main()

```

Source Reference: tools/requirements.txt

```
1 | reportlab>=4.2,<5  
2 | pypdf>=5,<7
```

Source Reference:

third_party/miniaudio/miniaudio_impl.cpp

```
1 | #define MINIAUDIO_IMPLEMENTATION
2 | #include "miniaudio.h"
```

Source Reference: third_party/glad/glad.c

```

1 | /*
2 |
3 |     OpenGL loader generated by glad 0.1.36 on Fri Sep 11 11:31:17 2026.
4 |
5 |     Language/Generator: C/C++
6 |     Specification: gl
7 |     APIs: gl=4.6
8 |     Profile: core
9 |     Extensions:
10 |
11 |     Loader: True
12 |     Local files: False
13 |     Omit khrplatform: False
14 |     Reproducible: False
15 |
16 |     Commandline:
17 |         --profile="core" --api="gl=4.6" --generator="c" --spec="gl" --extensions=""
18 |     Online:
19 |         https://glad.dav1d.de/#profile=core&language=c&specification=gl&loader=on&api=gl%3D4.6
20 | */
21 |
22 | #include <stdio.h>
23 | #include <stdlib.h>
24 | #include <string.h>
25 | #include <glad/glad.h>
26 |
27 | static void* get_proc(const char *namez);
28 |
29 | #if defined(_WIN32) || defined(__CYGWIN__)
30 | #ifndef _WINDOWS_
31 | #undef APIENTRY
32 | #endif
33 | #include <windows.h>
34 | static HMODULE libGL;
35 |
36 | typedef void* (APIENTRY PFNWGLGETPROCADDRESSPROC_PRIVATE)(const char*);
37 | static PFNWGLGETPROCADDRESSPROC_PRIVATE gladGetProcAddressPtr;
38 |
39 | #ifdef _MSC_VER
40 | #ifdef __has_include
41 |     #if __has_include(<winapifamily.h>)
42 |         #define HAVE_WINAPIFAMILY 1
43 |     #endif
44 | #elif _MSC_VER >= 1700 && !_USING_V110_SDK71_
45 |     #define HAVE_WINAPIFAMILY 1
46 | #endif
47 | #endif
48 |
49 | #ifdef HAVE_WINAPIFAMILY
50 |     #include <winapifamily.h>
51 |     #if !WINAPI_FAMILY_PARTITION(WINAPI_PARTITION_DESKTOP) &&
> WINAPI_FAMILY_PARTITION(WINAPI_PARTITION_APP)
52 |         #define IS_UWP 1
53 |     #endif
54 | #endif
55 |
56 | static
57 | int open_gl(void) {
58 |     #ifndef IS_UWP
59 |         libGL = LoadLibraryW(L"opengl32.dll");
60 |         if(libGL != NULL) {
61 |             void (* tmp)(void);
62 |             tmp = (void(*) (void)) GetProcAddress(libGL, "wglGetProcAddress");
63 |             gladGetProcAddressPtr = (PFNWGLGETPROCADDRESSPROC_PRIVATE) tmp;
64 |             return gladGetProcAddressPtr != NULL;
65 |         }
66 |     #endif
67 |
68 |     return 0;
69 | }
70 |
71 | static
72 | void close_gl(void) {
73 |     if(libGL != NULL) {
74 |         FreeLibrary((HMODULE) libGL);
75 |         libGL = NULL;

```

```

76 |     }
77 | }
78 | #else
79 | #include <dlfcn.h>
80 | static void* libGL;
81 |
82 | #if !defined(__APPLE__) && !defined(__HAIKU__)
83 | typedef void* (APIENTRY PFNGLXGETPROCADDRESSPROC_PRIVATE)(const char*);
84 | static PFNGLXGETPROCADDRESSPROC_PRIVATE gladGetProcAddressPtr;
85 | #endif
86 |
87 | static
88 | int open_gl(void) {
89 | #ifdef __APPLE__
90 |     static const char *NAMES[] = {
91 |         "../Frameworks/OpenGL.framework/OpenGL",
92 |         "/Library/Frameworks/OpenGL.framework/OpenGL",
93 |         "/System/Library/Frameworks/OpenGL.framework/OpenGL",
94 |         "/System/Library/Frameworks/OpenGL.framework/Versions/Current/OpenGL"
95 |     };
96 | #else
97 |     static const char *NAMES[] = {"libGL.so.1", "libGL.so"};
98 | #endif
99 |
100 |     unsigned int index = 0;
101 |     for(index = 0; index < (sizeof(NAMES) / sizeof(NAMES[0])); index++) {
102 |         libGL = dlopen(NAMES[index], RTLD_NOW | RTLD_GLOBAL);
103 |
104 |         if(libGL != NULL) {
105 | #if defined(__APPLE__) || defined(__HAIKU__)
106 |             return 1;
107 | #else
108 |             gladGetProcAddressPtr = (PFNGLXGETPROCADDRESSPROC_PRIVATE)dlsym(libGL,
109 |                                     "glXGetProcAddressARB");
110 |             return gladGetProcAddressPtr != NULL;
111 | #endif
112 |         }
113 |     }
114 |
115 |     return 0;
116 | }
117 |
118 | static
119 | void close_gl(void) {
120 |     if(libGL != NULL) {
121 |         dlclose(libGL);
122 |         libGL = NULL;
123 |     }
124 | }
125 | #endif
126 |
127 | static
128 | void* get_proc(const char *namez) {
129 |     void* result = NULL;
130 |     if(libGL == NULL) return NULL;
131 |
132 | #if !defined(__APPLE__) && !defined(__HAIKU__)
133 |     if(gladGetProcAddressPtr != NULL) {
134 |         result = gladGetProcAddressPtr(namez);
135 |     }
136 | #endif
137 |     if(result == NULL) {
138 | #if defined(_WIN32) || defined(__CYGWIN__)
139 |         result = (void*)GetProcAddress((HMODULE) libGL, namez);
140 | #else
141 |         result = dlsym(libGL, namez);
142 | #endif
143 |     }
144 |
145 |     return result;
146 | }
147 |
148 | int gladLoadGL(void) {
149 |     int status = 0;
150 |
151 |     if(open_gl()) {
152 |         status = gladLoadGLLoader(&get_proc);
153 |         close_gl();
154 |     }

```

```

155 |
156 |     return status;
157 | }
158 |
159 | struct gladGLVersionStruct GLVersion = { 0, 0 };
160 |
161 | #if defined(GL_ES_VERSION_3_0) || defined(GL_VERSION_3_0)
162 | #define _GLAD_IS_SOME_NEW_VERSION 1
163 | #endif
164 |
165 | static int max_loaded_major;
166 | static int max_loaded_minor;
167 |
168 | static const char *exts = NULL;
169 | static int num_exts_i = 0;
170 | static char **exts_i = NULL;
171 |
172 | static int get_exts(void) {
173 | #ifdef _GLAD_IS_SOME_NEW_VERSION
174 |     if(max_loaded_major < 3) {
175 | #endif
176 |         exts = (const char *)glGetString(GL_EXTENSIONS);
177 | #ifdef _GLAD_IS_SOME_NEW_VERSION
178 |     } else {
179 |         int index;
180 |
181 |         num_exts_i = 0;
182 |         glGetIntegerv(GL_NUM_EXTENSIONS, &num_exts_i);
183 |         if (num_exts_i > 0) {
184 |             exts_i = (char **)malloc((size_t)num_exts_i * (sizeof *exts_i));
185 |         }
186 |
187 |         if (exts_i == NULL) {
188 |             return 0;
189 |         }
190 |
191 |         for(index = 0; index < num_exts_i; index++) {
192 |             const char *gl_str_tmp = (const char*)glGetStringi(GL_EXTENSIONS, index);
193 |             size_t len = strlen(gl_str_tmp);
194 |
195 |             char *local_str = (char*)malloc((len+1) * sizeof(char));
196 |             if(local_str != NULL) {
197 |                 memcpy(local_str, gl_str_tmp, (len+1) * sizeof(char));
198 |             }
199 |             exts_i[index] = local_str;
200 |         }
201 |     }
202 | #endif
203 |     return 1;
204 | }
205 |
206 | static void free_exts(void) {
207 |     if (exts_i != NULL) {
208 |         int index;
209 |         for(index = 0; index < num_exts_i; index++) {
210 |             free((char *)exts_i[index]);
211 |         }
212 |         free((void *)exts_i);
213 |         exts_i = NULL;
214 |     }
215 | }
216 |
217 | static int has_ext(const char *ext) {
218 | #ifdef _GLAD_IS_SOME_NEW_VERSION
219 |     if(max_loaded_major < 3) {
220 | #endif
221 |         const char *extensions;
222 |         const char *loc;
223 |         const char *terminator;
224 |         extensions = exts;
225 |         if(extensions == NULL || ext == NULL) {
226 |             return 0;
227 |         }
228 |
229 |         while(1) {
230 |             loc = strstr(extensions, ext);
231 |             if(loc == NULL) {
232 |                 return 0;
233 |             }

```

```

234 |
235 |         terminator = loc + strlen(ext);
236 |         if((loc == extensions || *(loc - 1) == ' ') &&
237 |            (*terminator == ' ' || *terminator == '\\0')) {
238 |             return 1;
239 |         }
240 |         extensions = terminator;
241 |     }
242 | #ifdef _GLAD_IS_SOME_NEW_VERSION
243 |     } else {
244 |         int index;
245 |         if(exts_i == NULL) return 0;
246 |         for(index = 0; index < num_exts_i; index++) {
247 |             const char *e = exts_i[index];
248 |
249 |             if(exts_i[index] != NULL && strcmp(e, ext) == 0) {
250 |                 return 1;
251 |             }
252 |         }
253 |     }
254 | #endif
255 |
256 |     return 0;
257 | }
258 | int GLAD_GL_VERSION_1_0 = 0;
259 | int GLAD_GL_VERSION_1_1 = 0;
260 | int GLAD_GL_VERSION_1_2 = 0;
261 | int GLAD_GL_VERSION_1_3 = 0;
262 | int GLAD_GL_VERSION_1_4 = 0;
263 | int GLAD_GL_VERSION_1_5 = 0;
264 | int GLAD_GL_VERSION_2_0 = 0;
265 | int GLAD_GL_VERSION_2_1 = 0;
266 | int GLAD_GL_VERSION_3_0 = 0;
267 | int GLAD_GL_VERSION_3_1 = 0;
268 | int GLAD_GL_VERSION_3_2 = 0;
269 | int GLAD_GL_VERSION_3_3 = 0;
270 | int GLAD_GL_VERSION_4_0 = 0;
271 | int GLAD_GL_VERSION_4_1 = 0;
272 | int GLAD_GL_VERSION_4_2 = 0;
273 | int GLAD_GL_VERSION_4_3 = 0;
274 | int GLAD_GL_VERSION_4_4 = 0;
275 | int GLAD_GL_VERSION_4_5 = 0;
276 | int GLAD_GL_VERSION_4_6 = 0;
277 | PFNGLACTIVESHADERPROGRAMPROC glad_glActiveShaderProgram = NULL;
278 | PFNGLACTIVETEXTUREPROC glad_glActiveTexture = NULL;
279 | PFNGLATTACHSHADERPROC glad_glAttachShader = NULL;
280 | PFNGLBEGINCONDITIONALRENDERPROC glad_glBeginConditionalRender = NULL;
281 | PFNGLBEGINQUERYPROC glad_glBeginQuery = NULL;
282 | PFNGLBEGINQUERYINDEXEDPROC glad_glBeginQueryIndexed = NULL;
283 | PFNGLBEGINTRANSFORMFEEDBACKPROC glad_glBeginTransformFeedback = NULL;
284 | PFNGLBINDATTRIBLOCATIONPROC glad_glBindAttribLocation = NULL;
285 | PFNGLBINDBUFFERPROC glad_glBindBuffer = NULL;
286 | PFNGLBINDBUFFERBASEPROC glad_glBindBufferBase = NULL;
287 | PFNGLBINDBUFFERRANGEPROC glad_glBindBufferRange = NULL;
288 | PFNGLBINDBUFFERSBASEPROC glad_glBindBuffersBase = NULL;
289 | PFNGLBINDBUFFERSRANGEPROC glad_glBindBuffersRange = NULL;
290 | PFNGLBINDFRAGDATALOCATIONPROC glad_glBindFragDataLocation = NULL;
291 | PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glad_glBindFragDataLocationIndexed = NULL;
292 | PFNGLBINDFRAMEBUFFERPROC glad_glBindFramebuffer = NULL;
293 | PFNGLBINDIMAGETEXTUREPROC glad_glBindImageTexture = NULL;
294 | PFNGLBINDIMAGETEXTURESPROC glad_glBindImageTextures = NULL;
295 | PFNGLBINDPROGRAMPIPELINEPROC glad_glBindProgramPipeline = NULL;
296 | PFNGLBINDRENDERBUFFERPROC glad_glBindRenderbuffer = NULL;
297 | PFNGLBINDSAMPLERPROC glad_glBindSampler = NULL;
298 | PFNGLBINDSAMPLERSPROC glad_glBindSamplers = NULL;
299 | PFNGLBINDTEXTUREPROC glad_glBindTexture = NULL;
300 | PFNGLBINDTEXTUREUNITPROC glad_glBindTextureUnit = NULL;
301 | PFNGLBINDTEXTURESPROC glad_glBindTextures = NULL;
302 | PFNGLBINDTRANSFORMFEEDBACKPROC glad_glBindTransformFeedback = NULL;
303 | PFNGLBINDVERTEXARRAYPROC glad_glBindVertexArray = NULL;
304 | PFNGLBINDVERTEXBUFFERPROC glad_glBindVertexBuffer = NULL;
305 | PFNGLBINDVERTEXBUFFERSPROC glad_glBindVertexBuffers = NULL;
306 | PFNGLBLENDPROC glad_glBlend = NULL;
307 | PFNGLBLENDSEPARATEPROC glad_glBlendEquation = NULL;
308 | PFNGLBLENDSEPARATEIPROC glad_glBlendEquationSeparate = NULL;
309 | PFNGLBLENDSEPARATEIIPROC glad_glBlendEquationSeparatei = NULL;
310 | PFNGLBLENDSEPARATEIIPROC glad_glBlendEquationi = NULL;
311 | PFNGLBLENDFUNCPROC glad_glBlendFunc = NULL;
312 | PFNGLBLENDFUNCSEPARATEPROC glad_glBlendFuncSeparate = NULL;

```



```
313 | PFNGLBLENDFUNCSEPARATEIPROC glad_glBlendFuncSeparatei = NULL;
314 | PFNGLBLENDFUNCIPROC glad_glBlendFunci = NULL;
315 | PFNGLBLITFRAMEBUFFERPROC glad_glBlitFramebuffer = NULL;
316 | PFNGLBLITNAMEDFRAMEBUFFERPROC glad_glBlitNamedFramebuffer = NULL;
317 | PFNGLBUFFERDATAPROC glad_glBufferData = NULL;
318 | PFNGLBUFFERSTORAGEPROC glad_glBufferStorage = NULL;
319 | PFNGLBUFFERSUBDATAPROC glad_glBufferSubData = NULL;
320 | PFNGLCHECKFRAMEBUFFERSTATUSPROC glad_glCheckFramebufferStatus = NULL;
321 | PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glad_glCheckNamedFramebufferStatus = NULL;
322 | PFNGLCLAMPColorPROC glad_glClampColor = NULL;
323 | PFNGLCLEARPROC glad_glClear = NULL;
324 | PFNGLCLEARBUFFERDATAPROC glad_glClearBufferData = NULL;
325 | PFNGLCLEARBUFFERSUBDATAPROC glad_glClearBufferSubData = NULL;
326 | PFNGLCLEARBUFFERFIPROC glad_glClearBufferfi = NULL;
327 | PFNGLCLEARBUFFERFVPROC glad_glClearBufferfv = NULL;
328 | PFNGLCLEARBUFFERIVPROC glad_glClearBufferiv = NULL;
329 | PFNGLCLEARBUFFERUIVPROC glad_glClearBufferuiv = NULL;
330 | PFNGLCLEARColorPROC glad_glClearColor = NULL;
331 | PFNGLCLEARDEPTHPROC glad_glClearDepth = NULL;
332 | PFNGLCLEARDEPTHFPROC glad_glClearDepthf = NULL;
333 | PFNGLCLEARNAMEDBUFFERDATAPROC glad_glClearNamedBufferData = NULL;
334 | PFNGLCLEARNAMEDBUFFERSUBDATAPROC glad_glClearNamedBufferSubData = NULL;
335 | PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glad_glClearNamedFramebufferfi = NULL;
336 | PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glad_glClearNamedFramebufferfv = NULL;
337 | PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glad_glClearNamedFramebufferiv = NULL;
338 | PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glad_glClearNamedFramebufferuiv = NULL;
339 | PFNGLCLEARSTENCILPROC glad_glClearStencil = NULL;
340 | PFNGLCLEARTEXIMAGEPROC glad_glClearTexImage = NULL;
341 | PFNGLCLEARTEXSUBIMAGEPROC glad_glClearTexSubImage = NULL;
342 | PFNGLCLIENTWAITSYNCPROC glad_glClientWaitSync = NULL;
343 | PFNGLCLIPCONTROLPROC glad_glClipControl = NULL;
344 | PFNGLCOLORMASKPROC glad_glColorMask = NULL;
345 | PFNGLCOLORMASKIPROC glad_glColorMaski = NULL;
346 | PFNGLCOLORP3UIPROC glad_glColorP3ui = NULL;
347 | PFNGLCOLORP3UIVPROC glad_glColorP3uiv = NULL;
348 | PFNGLCOLORP4UIPROC glad_glColorP4ui = NULL;
349 | PFNGLCOLORP4UIVPROC glad_glColorP4uiv = NULL;
350 | PFNGLCOMPILESHADERPROC glad_glCompileShader = NULL;
351 | PFNGLCOMPRESSEDTEXIMAGE1DPROC glad_glCompressedTexImage1D = NULL;
352 | PFNGLCOMPRESSEDTEXIMAGE2DPROC glad_glCompressedTexImage2D = NULL;
353 | PFNGLCOMPRESSEDTEXIMAGE3DPROC glad_glCompressedTexImage3D = NULL;
354 | PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glad_glCompressedTexSubImage1D = NULL;
355 | PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glad_glCompressedTexSubImage2D = NULL;
356 | PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glad_glCompressedTexSubImage3D = NULL;
357 | PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glad_glCompressedTextureSubImage1D = NULL;
358 | PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glad_glCompressedTextureSubImage2D = NULL;
359 | PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glad_glCompressedTextureSubImage3D = NULL;
360 | PFNGLCOPYBUFFERSUBDATAPROC glad_glCopyBufferSubData = NULL;
361 | PFNGLCOPYIMAGESUBDATAPROC glad_glCopyImageSubData = NULL;
362 | PFNGLCOPYNAMEDBUFFERSUBDATAPROC glad_glCopyNamedBufferSubData = NULL;
363 | PFNGLCOPYTEXIMAGE1DPROC glad_glCopyTexImage1D = NULL;
364 | PFNGLCOPYTEXIMAGE2DPROC glad_glCopyTexImage2D = NULL;
365 | PFNGLCOPYTEXSUBIMAGE1DPROC glad_glCopyTexSubImage1D = NULL;
366 | PFNGLCOPYTEXSUBIMAGE2DPROC glad_glCopyTexSubImage2D = NULL;
367 | PFNGLCOPYTEXSUBIMAGE3DPROC glad_glCopyTexSubImage3D = NULL;
368 | PFNGLCOPYTEXTURESUBIMAGE1DPROC glad_glCopyTextureSubImage1D = NULL;
369 | PFNGLCOPYTEXTURESUBIMAGE2DPROC glad_glCopyTextureSubImage2D = NULL;
370 | PFNGLCOPYTEXTURESUBIMAGE3DPROC glad_glCopyTextureSubImage3D = NULL;
371 | PFNGLCREATEBUFFERSPROC glad_glCreateBuffers = NULL;
372 | PFNGLCREATEFRAMEBUFFERSPROC glad_glCreateFramebuffers = NULL;
373 | PFNGLCREATEPROGRAMPROC glad_glCreateProgram = NULL;
374 | PFNGLCREATEPROGRAMPIPELINESPROC glad_glCreateProgramPipelines = NULL;
375 | PFNGLCREATEQUERIESPROC glad_glCreateQueries = NULL;
376 | PFNGLCREATERENDERBUFFERSPROC glad_glCreateRenderbuffers = NULL;
377 | PFNGLCREATESAMPLERSPROC glad_glCreateSamplers = NULL;
378 | PFNGLCREATESHADERPROC glad_glCreateShader = NULL;
379 | PFNGLCREATESHADERPROGRAMVPROC glad_glCreateShaderProgramv = NULL;
380 | PFNGLCREATETEXTURESPROC glad_glCreateTextures = NULL;
381 | PFNGLCREATETRANSFORMFEEDBACKSPROC glad_glCreateTransformFeedbacks = NULL;
382 | PFNGLCREATEVERTEXARRAYSPROC glad_glCreateVertexArrays = NULL;
383 | PFNGLCULLFACEPROC glad_glCullFace = NULL;
384 | PFNGLDEBUGMESSAGECALLBACKPROC glad_glDebugMessageCallback = NULL;
385 | PFNGLDEBUGMESSAGECONTROLPROC glad_glDebugMessageControl = NULL;
386 | PFNGLDEBUGMESSAGEINSERTPROC glad_glDebugMessageInsert = NULL;
387 | PFNGLDELETEBUFFERSPROC glad_glDeleteBuffers = NULL;
388 | PFNGLDELETEFRAMEBUFFERSPROC glad_glDeleteFramebuffers = NULL;
389 | PFNGLDELETEPROGRAMPROC glad_glDeleteProgram = NULL;
390 | PFNGLDELETEPROGRAMPIPELINESPROC glad_glDeleteProgramPipelines = NULL;
391 | PFNGLDELETEQUERIESPROC glad_glDeleteQueries = NULL;
```

```
392 | PFNGLDELETERENDERBUFFERSPROC glad_glDeleteRenderbuffers = NULL;
393 | PFNGLDELETESAMPLERSPROC glad_glDeleteSamplers = NULL;
394 | PFNGLDELETESHADERPROC glad_glDeleteShader = NULL;
395 | PFNGLDELETESYNCPROC glad_glDeleteSync = NULL;
396 | PFNGLDELETETEXTURESPROC glad_glDeleteTextures = NULL;
397 | PFNGLDELETETRANSFORMFEEDBACKSPROC glad_glDeleteTransformFeedbacks = NULL;
398 | PFNGLDELETEVERTEXARRAYSPROC glad_glDeleteVertexArrays = NULL;
399 | PFNGLDEPTHFUNCPCPROC glad_glDepthFunc = NULL;
400 | PFNGLDEPTHMASKPROC glad_glDepthMask = NULL;
401 | PFNGLDEPTHRANGEPROC glad_glDepthRange = NULL;
402 | PFNGLDEPTHRANGEARRAYVPROC glad_glDepthRangeArrayv = NULL;
403 | PFNGLDEPTHRANGEINDEXEDPROC glad_glDepthRangeIndexed = NULL;
404 | PFNGLDEPTHRANGEFPROC glad_glDepthRangef = NULL;
405 | PFNGLDETACHSHADERPROC glad_glDetachShader = NULL;
406 | PFNGLDISABLEPROC glad_glDisable = NULL;
407 | PFNGLDISABLEVERTEXARRAYATTRIBPROC glad_glDisableVertexArrayAttrib = NULL;
408 | PFNGLDISABLEVERTEXATTRIBARRAYPROC glad_glDisableVertexAttribArray = NULL;
409 | PFNGLDISABLEIPROC glad_glDisablei = NULL;
410 | PFNGLDISPATCHCOMPUTEPROC glad_glDispatchCompute = NULL;
411 | PFNGLDISPATCHCOMPUTEINDIRECTPROC glad_glDispatchComputeIndirect = NULL;
412 | PFNGLDRAWARRAYSPROC glad_glDrawArrays = NULL;
413 | PFNGLDRAWARRAYSINDIRECTPROC glad_glDrawArraysIndirect = NULL;
414 | PFNGLDRAWARRAYSINSTANCEDPROC glad_glDrawArraysInstanced = NULL;
415 | PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glad_glDrawArraysInstancedBaseInstance = NULL;
416 | PFNGLDRAWBUFFERPROC glad_glDrawBuffer = NULL;
417 | PFNGLDRAWBUFFERSPROC glad_glDrawBuffers = NULL;
418 | PFNGLDRAWELEMENTSPROC glad_glDrawElements = NULL;
419 | PFNGLDRAWELEMENTSBASEVERTEXPROC glad_glDrawElementsBaseVertex = NULL;
420 | PFNGLDRAWELEMENTSINDIRECTPROC glad_glDrawElementsIndirect = NULL;
421 | PFNGLDRAWELEMENTSINSTANCEDPROC glad_glDrawElementsInstanced = NULL;
422 | PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glad_glDrawElementsInstancedBaseInstance = NULL;
423 | PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glad_glDrawElementsInstancedBaseVertex = NULL;
424 | PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC
> glad_glDrawElementsInstancedBaseVertexBaseInstance = NULL;
425 | PFNGLDRAWRANGEELEMENTSPROC glad_glDrawRangeElements = NULL;
426 | PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glad_glDrawRangeElementsBaseVertex = NULL;
427 | PFNGLDRAWTRANSFORMFEEDBACKPROC glad_glDrawTransformFeedback = NULL;
428 | PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glad_glDrawTransformFeedbackInstanced = NULL;
429 | PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glad_glDrawTransformFeedbackStream = NULL;
430 | PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glad_glDrawTransformFeedbackStreamInstanced = NULL;
431 | PFNGLENABLEPROC glad_glEnable = NULL;
432 | PFNGLENABLEVERTEXARRAYATTRIBPROC glad_glEnableVertexArrayAttrib = NULL;
433 | PFNGLENABLEVERTEXATTRIBARRAYPROC glad_glEnableVertexAttribArray = NULL;
434 | PFNGLENABLEIPROC glad_glEnablei = NULL;
435 | PFNGLENDCONDITIONALRENDERPROC glad_glEndConditionalRender = NULL;
436 | PFNGLENDQUERYPROC glad_glEndQuery = NULL;
437 | PFNGLENDQUERYINDEXEDPROC glad_glEndQueryIndexed = NULL;
438 | PFNGLENDTRANSFORMFEEDBACKPROC glad_glEndTransformFeedback = NULL;
439 | PFNGLFENCESYNCPROC glad_glFenceSync = NULL;
440 | PFNGLFINISHPROC glad_glFinish = NULL;
441 | PFNGLFLUSHPROC glad_glFlush = NULL;
442 | PFNGLFLUSHMAPPEDBUFFERRANGEPROC glad_glFlushMappedBufferRange = NULL;
443 | PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glad_glFlushMappedNamedBufferRange = NULL;
444 | PFNGLFRAMEBUFFERPARAMETERIPROC glad_glFramebufferParameteri = NULL;
445 | PFNGLFRAMEBUFFERRENDERBUFFERPROC glad_glFramebufferRenderbuffer = NULL;
446 | PFNGLFRAMEBUFFERTEXTUREPROC glad_glFramebufferTexture = NULL;
447 | PFNGLFRAMEBUFFERTEXTURE1DPROC glad_glFramebufferTexture1D = NULL;
448 | PFNGLFRAMEBUFFERTEXTURE2DPROC glad_glFramebufferTexture2D = NULL;
449 | PFNGLFRAMEBUFFERTEXTURE3DPROC glad_glFramebufferTexture3D = NULL;
450 | PFNGLFRAMEBUFFERTEXTURELAYERPROC glad_glFramebufferTextureLayer = NULL;
451 | PFNGLFRONTFACEPROC glad_glFrontFace = NULL;
452 | PFNGLGENBUFFERSPROC glad_glGenBuffers = NULL;
453 | PFNGLGENFRAMEBUFFERSPROC glad_glGenFramebuffers = NULL;
454 | PFNGLGENPROGRAMPIPELINESPROC glad_glGenProgramPipelines = NULL;
455 | PFNGLGENQUERIESPROC glad_glGenQueries = NULL;
456 | PFNGLGENRENDERBUFFERSPROC glad_glGenRenderbuffers = NULL;
457 | PFNGLGENSAMPLERSPROC glad_glGenSamplers = NULL;
458 | PFNGLGENTEXTURESPROC glad_glGenTextures = NULL;
459 | PFNGLGENTRANSFORMFEEDBACKSPROC glad_glGenTransformFeedbacks = NULL;
460 | PFNGLGENVERTEXARRAYSPROC glad_glGenVertexArrays = NULL;
461 | PFNGLGENERATEMIPMAPPROC glad_glGenerateMipmap = NULL;
462 | PFNGLGENERATETEXTUREMIPMAPPROC glad_glGenerateTextureMipmap = NULL;
463 | PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glad_glGetActiveAtomicCounterBufferiv = NULL;
464 | PFNGLGETACTIVEATTRIBPROC glad_glGetActiveAttrib = NULL;
465 | PFNGLGETACTIVESUBROUTINENAMEPROC glad_glGetActiveSubroutineName = NULL;
466 | PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glad_glGetActiveSubroutineUniformName = NULL;
467 | PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glad_glGetActiveSubroutineUniformiv = NULL;
468 | PFNGLGETACTIVEUNIFORMPROC glad_glGetActiveUniform = NULL;
469 | PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glad_glGetActiveUniformBlockName = NULL;
```

```
470 | PFNGLGETACTIVEUNIFORMBLOCKIVPROC glad_glGetActiveUniformBlockiv = NULL;
471 | PFNGLGETACTIVEUNIFORMNAMEPROC glad_glGetActiveUniformName = NULL;
472 | PFNGLGETACTIVEUNIFORMSIVPROC glad_glGetActiveUniformsiv = NULL;
473 | PFNGLGETATTACHEDSHADERSPROC glad_glGetAttachedShaders = NULL;
474 | PFNGLGETATTRIBLOCATIONPROC glad_glGetAttribLocation = NULL;
475 | PFNGLGETBOOLEANI_VPROC glad_glGetBooleani_v = NULL;
476 | PFNGLGETBOOLEANVPROC glad_glGetBooleany_v = NULL;
477 | PFNGLGETBUFFERPARAMETERI64VPROC glad_glGetBufferParameteri64v = NULL;
478 | PFNGLGETBUFFERPARAMETERIVPROC glad_glGetBufferParameteriv = NULL;
479 | PFNGLGETBUFFERPOINTERVPROC glad_glGetBufferPointerv = NULL;
480 | PFNGLGETBUFFERSUBDATAPROC glad_glGetBufferSubData = NULL;
481 | PFNGLGETCOMPRESSEDTEXIMAGEPROC glad_glGetCompressedTexImage = NULL;
482 | PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glad_glGetCompressedTextureImage = NULL;
483 | PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glad_glGetCompressedTextureSubImage = NULL;
484 | PFNGLGETDEBUGMESSAGELOGPROC glad_glGetDebugMessageLog = NULL;
485 | PFNGLGETDOUBLEI_VPROC glad_glGetDoublei_v = NULL;
486 | PFNGLGETDOUBLEVPROC glad_glGetDoublev = NULL;
487 | PFNGLGETERRORPROC glad_glGetError = NULL;
488 | PFNGLGETFLOATI_VPROC glad_glGetFloati_v = NULL;
489 | PFNGLGETFLOATVPROC glad_glGetFloatv = NULL;
490 | PFNGLGETFRAGDATAINDEXPROC glad_glGetFragDataIndex = NULL;
491 | PFNGLGETFRAGDATALOCATIONPROC glad_glGetFragDataLocation = NULL;
492 | PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glad_glGetFramebufferAttachmentParameteriv = NULL;
493 | PFNGLGETFRAMEBUFFERPARAMETERIVPROC glad_glGetFramebufferParameteriv = NULL;
494 | PFNGLGETGRAPHICSRESETSTATUSPROC glad_glGetGraphicsResetStatus = NULL;
495 | PFNGLGETINTEGER64I_VPROC glad_glGetInteger64i_v = NULL;
496 | PFNGLGETINTEGER64VPROC glad_glGetInteger64v = NULL;
497 | PFNGLGETINTEGERI_VPROC glad_glGetIntegeri_v = NULL;
498 | PFNGLGETINTEGERVPROC glad_glGetIntegeriv = NULL;
499 | PFNGLGETINTERNALFORMATI64VPROC glad_glGetInternalformati64v = NULL;
500 | PFNGLGETINTERNALFORMATIVPROC glad_glGetInternalformativ = NULL;
501 | PFNGLGETMULTISAMPLEFVPROC glad_glGetMultisamplefv = NULL;
502 | PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glad_glGetNamedBufferParameteri64v = NULL;
503 | PFNGLGETNAMEDBUFFERPARAMETERIVPROC glad_glGetNamedBufferParameteriv = NULL;
504 | PFNGLGETNAMEDBUFFERPOINTERVPROC glad_glGetNamedBufferPointerv = NULL;
505 | PFNGLGETNAMEDBUFFERSUBDATAPROC glad_glGetNamedBufferSubData = NULL;
506 | PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glad_glGetNamedFramebufferAttachmentParameteriv =
> NULL;
507 | PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glad_glGetNamedFramebufferParameteriv = NULL;
508 | PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glad_glGetNamedRenderbufferParameteriv = NULL;
509 | PFNGLGETOBJECTLABELPROC glad_glGetObjectLabel = NULL;
510 | PFNGLGETOBJECTPTRLABELPROC glad_glGetObjectPtrLabel = NULL;
511 | PFNGLGETPOINTERVPROC glad_glGetPointerv = NULL;
512 | PFNGLGETPROGRAMBINARYPROC glad_glGetProgramBinary = NULL;
513 | PFNGLGETPROGRAMINFOLOGPROC glad_glGetProgramInfoLog = NULL;
514 | PFNGLGETPROGRAMINTERFACEIVPROC glad_glGetProgramInterfaceiv = NULL;
515 | PFNGLGETPROGRAMPIPELINEINFOLOGPROC glad_glGetProgramPipelineInfoLog = NULL;
516 | PFNGLGETPROGRAMPIPELINEIVPROC glad_glGetProgramPipelineiv = NULL;
517 | PFNGLGETPROGRAMRESOURCEINDEXPROC glad_glGetProgramResourceIndex = NULL;
518 | PFNGLGETPROGRAMRESOURCELOCATIONPROC glad_glGetProgramResourceLocation = NULL;
519 | PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glad_glGetProgramResourceLocationIndex = NULL;
520 | PFNGLGETPROGRAMRESOURCECENAMEPROC glad_glGetProgramResourceName = NULL;
521 | PFNGLGETPROGRAMRESOURCEIVPROC glad_glGetProgramResourceiv = NULL;
522 | PFNGLGETPROGRAMSTAGEIVPROC glad_glGetProgramStageiv = NULL;
523 | PFNGLGETPROGRAMIVPROC glad_glGetProgramiv = NULL;
524 | PFNGLGETQUERYBUFFEROBJECTI64VPROC glad_glGetQueryBufferObjecti64v = NULL;
525 | PFNGLGETQUERYBUFFEROBJECTIVPROC glad_glGetQueryBufferObjectiv = NULL;
526 | PFNGLGETQUERYBUFFEROBJECTUI64VPROC glad_glGetQueryBufferObjectui64v = NULL;
527 | PFNGLGETQUERYBUFFEROBJECTUIVPROC glad_glGetQueryBufferObjectuiv = NULL;
528 | PFNGLGETQUERYINDEXEDIVPROC glad_glGetQueryIndexediv = NULL;
529 | PFNGLGETQUERYOBJECTI64VPROC glad_glGetQueryObjecti64v = NULL;
530 | PFNGLGETQUERYOBJECTIVPROC glad_glGetQueryObjectiv = NULL;
531 | PFNGLGETQUERYOBJECTUI64VPROC glad_glGetQueryObjectui64v = NULL;
532 | PFNGLGETQUERYOBJECTUIVPROC glad_glGetQueryObjectuiv = NULL;
533 | PFNGLGETQUERYIVPROC glad_glGetQueryiv = NULL;
534 | PFNGLGETRENDERBUFFERPARAMETERIVPROC glad_glGetRenderbufferParameteriv = NULL;
535 | PFNGLGETSAMPLERPARAMETERIIVPROC glad_glGetSamplerParameterIiv = NULL;
536 | PFNGLGETSAMPLERPARAMETERIUIVPROC glad_glGetSamplerParameterIuiv = NULL;
537 | PFNGLGETSAMPLERPARAMETERFVPROC glad_glGetSamplerParameterfv = NULL;
538 | PFNGLGETSAMPLERPARAMETERIVPROC glad_glGetSamplerParameteriv = NULL;
539 | PFNGLGETSHADERINFOLOGPROC glad_glGetShaderInfoLog = NULL;
540 | PFNGLGETSHADERPRECISIONFORMATPROC glad_glGetShaderPrecisionFormat = NULL;
541 | PFNGLGETSHADERSOURCEPROC glad_glGetShaderSource = NULL;
542 | PFNGLGETSHADERIVPROC glad_glGetShaderiv = NULL;
543 | PFNGLGETSTRINGPROC glad_glGetString = NULL;
544 | PFNGLGETSTRINGIPROC glad_glGetStringi = NULL;
545 | PFNGLGETSUBROUTINEINDEXPROC glad_glGetSubroutineIndex = NULL;
546 | PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glad_glGetSubroutineUniformLocation = NULL;
547 | PFNGLGETSYNCPROC glad_glGetSynciv = NULL;
```

```
548 | PFNGLGETTEXIMAGEPROC glad_glGetTexImage = NULL;
549 | PFNGLGETTEXLEVELPARAMETERFVPROC glad_glGetTexLevelParameterfv = NULL;
550 | PFNGLGETTEXLEVELPARAMETERIVPROC glad_glGetTexLevelParameteriv = NULL;
551 | PFNGLGETTEXPARAMETERIIVPROC glad_glGetTexParameterIiv = NULL;
552 | PFNGLGETTEXPARAMETERIUIVPROC glad_glGetTexParameterIuiv = NULL;
553 | PFNGLGETTEXPARAMETERFVPROC glad_glGetTexParameterfv = NULL;
554 | PFNGLGETTEXPARAMETERIVPROC glad_glGetTexParameteriv = NULL;
555 | PFNGLGETTEXTUREIMAGEPROC glad_glGetTextureImage = NULL;
556 | PFNGLGETTEXTURELEVELPARAMETERFVPROC glad_glGetTextureLevelParameterfv = NULL;
557 | PFNGLGETTEXTURELEVELPARAMETERIVPROC glad_glGetTextureLevelParameteriv = NULL;
558 | PFNGLGETTEXTUREPARAMETERIIVPROC glad_glGetTextureParameterIiv = NULL;
559 | PFNGLGETTEXTUREPARAMETERIUIVPROC glad_glGetTextureParameterIuiv = NULL;
560 | PFNGLGETTEXTUREPARAMETERFVPROC glad_glGetTextureParameterfv = NULL;
561 | PFNGLGETTEXTUREPARAMETERIVPROC glad_glGetTextureParameteriv = NULL;
562 | PFNGLGETTEXTURESUBIMAGEPROC glad_glGetTextureSubImage = NULL;
563 | PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glad_glGetTransformFeedbackVarying = NULL;
564 | PFNGLGETTRANSFORMFEEDBACKI64_VPROC glad_glGetTransformFeedbacki64_v = NULL;
565 | PFNGLGETTRANSFORMFEEDBACKI_VPROC glad_glGetTransformFeedbacki_v = NULL;
566 | PFNGLGETTRANSFORMFEEDBACKIVPROC glad_glGetTransformFeedbackiv = NULL;
567 | PFNGLGETUNIFORMBLOCKINDEXPROC glad_glGetUniformBlockIndex = NULL;
568 | PFNGLGETUNIFORMINDICESPROC glad_glGetUniformIndices = NULL;
569 | PFNGLGETUNIFORMLOCATIONPROC glad_glGetUniformLocation = NULL;
570 | PFNGLGETUNIFORMSUBROUTINEUIVPROC glad_glGetUniformSubroutineuiv = NULL;
571 | PFNGLGETUNIFORMDVPROC glad_glGetUniformdv = NULL;
572 | PFNGLGETUNIFORMFVPROC glad_glGetUniformfv = NULL;
573 | PFNGLGETUNIFORMIVPROC glad_glGetUniformiv = NULL;
574 | PFNGLGETUNIFORMUIVPROC glad_glGetUniformuiv = NULL;
575 | PFNGLGETVERTEXARRAYINDEXED64IVPROC glad_glGetVertexArrayIndexed64iv = NULL;
576 | PFNGLGETVERTEXARRAYINDEXEDIVPROC glad_glGetVertexArrayIndexediv = NULL;
577 | PFNGLGETVERTEXARRAYIVPROC glad_glGetVertexArrayiv = NULL;
578 | PFNGLGETVERTEXATTRIBIIVPROC glad_glGetVertexAttribIiv = NULL;
579 | PFNGLGETVERTEXATTRIBIUIVPROC glad_glGetVertexAttribIuiv = NULL;
580 | PFNGLGETVERTEXATTRIBLDVPROC glad_glGetVertexAttribLdv = NULL;
581 | PFNGLGETVERTEXATTRIBPOINTERVPROC glad_glGetVertexAttribPointerv = NULL;
582 | PFNGLGETVERTEXATTRIBDVPROC glad_glGetVertexAttribdv = NULL;
583 | PFNGLGETVERTEXATTRIBFVPROC glad_glGetVertexAttribfv = NULL;
584 | PFNGLGETVERTEXATTRIBIVPROC glad_glGetVertexAttribiv = NULL;
585 | PFNGLGETNCOLORTABLEPROC glad_glGetnColorTable = NULL;
586 | PFNGLGETNCOMPRESSEDTEXIMAGEPROC glad_glGetnCompressedTexImage = NULL;
587 | PFNGLGETNCONVOLUTIONFILTERPROC glad_glGetnConvolutionFilter = NULL;
588 | PFNGLGETNHISTOGRAMPROC glad_glGetnHistogram = NULL;
589 | PFNGLGETNMAPDVPROC glad_glGetnMapdv = NULL;
590 | PFNGLGETNMAPFVPROC glad_glGetnMapfv = NULL;
591 | PFNGLGETNMAPIVPROC glad_glGetnMapiv = NULL;
592 | PFNGLGETNMINMAXPROC glad_glGetnMinmax = NULL;
593 | PFNGLGETNPIXELMAPFVPROC glad_glGetnPixelMapfv = NULL;
594 | PFNGLGETNPIXELMAPIVPROC glad_glGetnPixelMapuiv = NULL;
595 | PFNGLGETNPIXELMAPUSVPROC glad_glGetnPixelMapusv = NULL;
596 | PFNGLGETNPOLYGONSTIPPLEPROC glad_glGetnPolygonStipple = NULL;
597 | PFNGLGETNSEPARABLEFILTERPROC glad_glGetnSeparableFilter = NULL;
598 | PFNGLGETNTEXIMAGEPROC glad_glGetnTexImage = NULL;
599 | PFNGLGETNUNIFORMDVPROC glad_glGetnUniformdv = NULL;
600 | PFNGLGETNUNIFORMFVPROC glad_glGetnUniformfv = NULL;
601 | PFNGLGETNUNIFORMIVPROC glad_glGetnUniformiv = NULL;
602 | PFNGLGETNUNIFORMUIVPROC glad_glGetnUniformuiv = NULL;
603 | PFNGLHINTPROC glad_glHint = NULL;
604 | PFNGLINVALIDATEBUFFERDATAPROC glad_glInvalidateBufferData = NULL;
605 | PFNGLINVALIDATEBUFFERSUBDATAPROC glad_glInvalidateBufferSubData = NULL;
606 | PFNGLINVALIDATEFRAMEBUFFERPROC glad_glInvalidateFramebuffer = NULL;
607 | PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glad_glInvalidateNamedFramebufferData = NULL;
608 | PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glad_glInvalidateNamedFramebufferSubData = NULL;
609 | PFNGLINVALIDATESUBFRAMEBUFFERPROC glad_glInvalidateSubFramebuffer = NULL;
610 | PFNGLINVALIDATETEXIMAGEPROC glad_glInvalidateTexImage = NULL;
611 | PFNGLINVALIDATETEXSUBIMAGEPROC glad_glInvalidateTexSubImage = NULL;
612 | PFNGLISBUFFERPROC glad_glIsBuffer = NULL;
613 | PFNGLISENABLEDPROC glad_glIsEnabled = NULL;
614 | PFNGLISENABLEDIPROC glad_glIsEnabledi = NULL;
615 | PFNGLISFRAMEBUFFERPROC glad_glIsFramebuffer = NULL;
616 | PFNGLISPROGRAMPROC glad_glIsProgram = NULL;
617 | PFNGLISPROGRAMPIPELINEPROC glad_glIsProgramPipeline = NULL;
618 | PFNGLISQUERYPROC glad_glIsQuery = NULL;
619 | PFNGLISRENDERBUFFERPROC glad_glIsRenderbuffer = NULL;
620 | PFNGLISSAMPLERPROC glad_glIsSampler = NULL;
621 | PFNGLISSHADERPROC glad_glIsShader = NULL;
622 | PFNGLISSYNCPROC glad_glIsSync = NULL;
623 | PFNGLISTEXTUREPROC glad_glIsTexture = NULL;
624 | PFNGLISTRANSFORMFEEDBACKPROC glad_glIsTransformFeedback = NULL;
625 | PFNGLISVERTEXARRAYPROC glad_glIsVertexArray = NULL;
626 | PFNGLLINEWIDTHPROC glad_glLineWidth = NULL;
```



```
627 | PFNGLLINKPROGRAMPROC glad_glLinkProgram = NULL;
628 | PFNGLLOGICOPPROC glad_glLogicOp = NULL;
629 | PFNGLMAPBUFFERPROC glad_glMapBuffer = NULL;
630 | PFNGLMAPBUFFERRANGEPROC glad_glMapBufferRange = NULL;
631 | PFNGLMAPNAMEDBUFFERPROC glad_glMapNamedBuffer = NULL;
632 | PFNGLMAPNAMEDBUFFERRANGEPROC glad_glMapNamedBufferRange = NULL;
633 | PFNGLMEMORYBARRIERPROC glad_glMemoryBarrier = NULL;
634 | PFNGLMEMORYBARRIERBYREGIONPROC glad_glMemoryBarrierByRegion = NULL;
635 | PFNGLMINSAMPLESHADINGPROC glad_glMinSampleShading = NULL;
636 | PFNGLMULTIDRAWARRAYSPROC glad_glMultiDrawArrays = NULL;
637 | PFNGLMULTIDRAWARRAYSINDIRECTPROC glad_glMultiDrawArraysIndirect = NULL;
638 | PFNGLMULTIDRAWARRAYSINDIRECTCOUNTPROC glad_glMultiDrawArraysIndirectCount = NULL;
639 | PFNGLMULTIDRAWELEMENTSPROC glad_glMultiDrawElements = NULL;
640 | PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glad_glMultiDrawElementsBaseVertex = NULL;
641 | PFNGLMULTIDRAWELEMENTSINDIRECTPROC glad_glMultiDrawElementsIndirect = NULL;
642 | PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTPROC glad_glMultiDrawElementsIndirectCount = NULL;
643 | PFNGLMULTITEXCOORDP1UIPROC glad_glMultiTexCoordP1ui = NULL;
644 | PFNGLMULTITEXCOORDP1UIVPROC glad_glMultiTexCoordP1uiv = NULL;
645 | PFNGLMULTITEXCOORDP2UIPROC glad_glMultiTexCoordP2ui = NULL;
646 | PFNGLMULTITEXCOORDP2UIVPROC glad_glMultiTexCoordP2uiv = NULL;
647 | PFNGLMULTITEXCOORDP3UIPROC glad_glMultiTexCoordP3ui = NULL;
648 | PFNGLMULTITEXCOORDP3UIVPROC glad_glMultiTexCoordP3uiv = NULL;
649 | PFNGLMULTITEXCOORDP4UIPROC glad_glMultiTexCoordP4ui = NULL;
650 | PFNGLMULTITEXCOORDP4UIVPROC glad_glMultiTexCoordP4uiv = NULL;
651 | PFNGLNAMEDBUFFERDATAPROC glad_glNamedBufferData = NULL;
652 | PFNGLNAMEDBUFFERSTORAGEPROC glad_glNamedBufferStorage = NULL;
653 | PFNGLNAMEDBUFFERSUBDATAPROC glad_glNamedBufferSubData = NULL;
654 | PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glad_glNamedFramebufferDrawBuffer = NULL;
655 | PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glad_glNamedFramebufferDrawBuffers = NULL;
656 | PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glad_glNamedFramebufferParameteri = NULL;
657 | PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glad_glNamedFramebufferReadBuffer = NULL;
658 | PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glad_glNamedFramebufferRenderbuffer = NULL;
659 | PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glad_glNamedFramebufferTexture = NULL;
660 | PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glad_glNamedFramebufferTextureLayer = NULL;
661 | PFNGLNAMEDRENDERBUFFERSTORAGEPROC glad_glNamedRenderbufferStorage = NULL;
662 | PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC glad_glNamedRenderbufferStorageMultisample = NULL;
663 | PFNGLNORMALP3UIPROC glad_glNormalP3ui = NULL;
664 | PFNGLNORMALP3UIVPROC glad_glNormalP3uiv = NULL;
665 | PFNGLOBJECTLABELPROC glad_glObjectLabel = NULL;
666 | PFNGLOBJECTPTRLABELPROC glad_glObjectPtrLabel = NULL;
667 | PFNGLPATCHPARAMETERFVPROC glad_glPatchParameterfv = NULL;
668 | PFNGLPATCHPARAMETERIPROC glad_glPatchParameteri = NULL;
669 | PFNGLPAUSETRANSFORMFEEDBACKPROC glad_glPauseTransformFeedback = NULL;
670 | PFNGLPIXELSTOREFPROC glad_glPixelStoref = NULL;
671 | PFNGLPIXELSTOREIPROC glad_glPixelStorei = NULL;
672 | PFNGLPOINTPARAMETERFPROC glad_glPointParameterf = NULL;
673 | PFNGLPOINTPARAMETERFVPROC glad_glPointParameterfv = NULL;
674 | PFNGLPOINTPARAMETERIPROC glad_glPointParameteri = NULL;
675 | PFNGLPOINTPARAMETERIVPROC glad_glPointParameteriv = NULL;
676 | PFNGLPOINTSIZEPROC glad_glPointSize = NULL;
677 | PFNGLPOLYGONMODEPROC glad_glPolygonMode = NULL;
678 | PFNGLPOLYGONOFFSETPROC glad_glPolygonOffset = NULL;
679 | PFNGLPOLYGONOFFSETCLAMPPROC glad_glPolygonOffsetClamp = NULL;
680 | PFNGLPOPEDEBUGGROUPPROC glad_glPopDebugGroup = NULL;
681 | PFNGLPRIMITIVERESTARTINDEXPROC glad_glPrimitiveRestartIndex = NULL;
682 | PFNGLPROGRAMBINARYPROC glad_glProgramBinary = NULL;
683 | PFNGLPROGRAMPARAMETERIPROC glad_glProgramParameteri = NULL;
684 | PFNGLPROGRAMUNIFORM1DPROC glad_glProgramUniform1d = NULL;
685 | PFNGLPROGRAMUNIFORM1DVPROC glad_glProgramUniform1dv = NULL;
686 | PFNGLPROGRAMUNIFORM1FPROC glad_glProgramUniform1f = NULL;
687 | PFNGLPROGRAMUNIFORM1FVPROC glad_glProgramUniform1fv = NULL;
688 | PFNGLPROGRAMUNIFORM1IPROC glad_glProgramUniform1i = NULL;
689 | PFNGLPROGRAMUNIFORM1IVPROC glad_glProgramUniform1iv = NULL;
690 | PFNGLPROGRAMUNIFORM1UIPROC glad_glProgramUniform1ui = NULL;
691 | PFNGLPROGRAMUNIFORM1UIVPROC glad_glProgramUniform1uiv = NULL;
692 | PFNGLPROGRAMUNIFORM2DPROC glad_glProgramUniform2d = NULL;
693 | PFNGLPROGRAMUNIFORM2DVPROC glad_glProgramUniform2dv = NULL;
694 | PFNGLPROGRAMUNIFORM2FPROC glad_glProgramUniform2f = NULL;
695 | PFNGLPROGRAMUNIFORM2FVPROC glad_glProgramUniform2fv = NULL;
696 | PFNGLPROGRAMUNIFORM2IPROC glad_glProgramUniform2i = NULL;
697 | PFNGLPROGRAMUNIFORM2IVPROC glad_glProgramUniform2iv = NULL;
698 | PFNGLPROGRAMUNIFORM2UIPROC glad_glProgramUniform2ui = NULL;
699 | PFNGLPROGRAMUNIFORM2UIVPROC glad_glProgramUniform2uiv = NULL;
700 | PFNGLPROGRAMUNIFORM3DPROC glad_glProgramUniform3d = NULL;
701 | PFNGLPROGRAMUNIFORM3DVPROC glad_glProgramUniform3dv = NULL;
702 | PFNGLPROGRAMUNIFORM3FPROC glad_glProgramUniform3f = NULL;
703 | PFNGLPROGRAMUNIFORM3FVPROC glad_glProgramUniform3fv = NULL;
704 | PFNGLPROGRAMUNIFORM3IPROC glad_glProgramUniform3i = NULL;
705 | PFNGLPROGRAMUNIFORM3IVPROC glad_glProgramUniform3iv = NULL;
```

```
706 | PFNGLPROGRAMUNIFORM3UIPROC glad_glProgramUniform3ui = NULL;
707 | PFNGLPROGRAMUNIFORM3UIVPROC glad_glProgramUniform3uiv = NULL;
708 | PFNGLPROGRAMUNIFORM4DPROC glad_glProgramUniform4d = NULL;
709 | PFNGLPROGRAMUNIFORM4DVPROC glad_glProgramUniform4dv = NULL;
710 | PFNGLPROGRAMUNIFORM4FPROC glad_glProgramUniform4f = NULL;
711 | PFNGLPROGRAMUNIFORM4FVPROC glad_glProgramUniform4fv = NULL;
712 | PFNGLPROGRAMUNIFORM4IPROC glad_glProgramUniform4i = NULL;
713 | PFNGLPROGRAMUNIFORM4IVPROC glad_glProgramUniform4iv = NULL;
714 | PFNGLPROGRAMUNIFORM4UIPROC glad_glProgramUniform4ui = NULL;
715 | PFNGLPROGRAMUNIFORM4UIVPROC glad_glProgramUniform4uiv = NULL;
716 | PFNGLPROGRAMUNIFORMMATRIX2DVPROC glad_glProgramUniformMatrix2dv = NULL;
717 | PFNGLPROGRAMUNIFORMMATRIX2FVPROC glad_glProgramUniformMatrix2fv = NULL;
718 | PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glad_glProgramUniformMatrix2x3dv = NULL;
719 | PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glad_glProgramUniformMatrix2x3fv = NULL;
720 | PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glad_glProgramUniformMatrix2x4dv = NULL;
721 | PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glad_glProgramUniformMatrix2x4fv = NULL;
722 | PFNGLPROGRAMUNIFORMMATRIX3DVPROC glad_glProgramUniformMatrix3dv = NULL;
723 | PFNGLPROGRAMUNIFORMMATRIX3FVPROC glad_glProgramUniformMatrix3fv = NULL;
724 | PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glad_glProgramUniformMatrix3x2dv = NULL;
725 | PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glad_glProgramUniformMatrix3x2fv = NULL;
726 | PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glad_glProgramUniformMatrix3x4dv = NULL;
727 | PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glad_glProgramUniformMatrix3x4fv = NULL;
728 | PFNGLPROGRAMUNIFORMMATRIX4DVPROC glad_glProgramUniformMatrix4dv = NULL;
729 | PFNGLPROGRAMUNIFORMMATRIX4FVPROC glad_glProgramUniformMatrix4fv = NULL;
730 | PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glad_glProgramUniformMatrix4x2dv = NULL;
731 | PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glad_glProgramUniformMatrix4x2fv = NULL;
732 | PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glad_glProgramUniformMatrix4x3dv = NULL;
733 | PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glad_glProgramUniformMatrix4x3fv = NULL;
734 | PFNGLPROVOKINGVERTEXPROC glad_glProvokingVertex = NULL;
735 | PFNGLPUSHDEBUGGROUPPROC glad_glPushDebugGroup = NULL;
736 | PFNGLQUERYCOUNTERPROC glad_glQueryCounter = NULL;
737 | PFNGLREADBUFFERPROC glad_glReadBuffer = NULL;
738 | PFNGLREADPIXELSPROC glad_glReadPixels = NULL;
739 | PFNGLREADNPIXELSPROC glad_glReadnPixels = NULL;
740 | PFNGLRELEASESHADERCOMPILERPROC glad_glReleaseShaderCompiler = NULL;
741 | PFNGLRENDERBUFFERSTORAGEPROC glad_glRenderbufferStorage = NULL;
742 | PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glad_glRenderbufferStorageMultisample = NULL;
743 | PFNGLRESUMETRANSFORMFEEDBACKPROC glad_glResumeTransformFeedback = NULL;
744 | PFNGLSAMPLECOVERAGEPROC glad_glSampleCoverage = NULL;
745 | PFNGLSAMPLERMASKIPROC glad_glSamplerMaski = NULL;
746 | PFNGLSAMPLERPARAMETERIIVPROC glad_glSamplerParameterIiv = NULL;
747 | PFNGLSAMPLERPARAMETERIUIVPROC glad_glSamplerParameterIuiv = NULL;
748 | PFNGLSAMPLERPARAMETERFPROC glad_glSamplerParameterf = NULL;
749 | PFNGLSAMPLERPARAMETERFVPROC glad_glSamplerParameterfv = NULL;
750 | PFNGLSAMPLERPARAMETERIPROC glad_glSamplerParameteri = NULL;
751 | PFNGLSAMPLERPARAMETERIVPROC glad_glSamplerParameteriv = NULL;
752 | PFNGLSCISSORPROC glad_glScissor = NULL;
753 | PFNGLSCISSORARRAYVPROC glad_glScissorArrayv = NULL;
754 | PFNGLSCISSORINDEXEDPROC glad_glScissorIndexed = NULL;
755 | PFNGLSCISSORINDEXEDVPROC glad_glScissorIndexedv = NULL;
756 | PFNGLSECONDARYCOLORP3UIPROC glad_glSecondaryColorP3ui = NULL;
757 | PFNGLSECONDARYCOLORP3UIVPROC glad_glSecondaryColorP3uiv = NULL;
758 | PFNGLSHADERBINARYPROC glad_glShaderBinary = NULL;
759 | PFNGLSHADERSOURCEPROC glad_glShaderSource = NULL;
760 | PFNGLSHADERSTORAGEBLOCKBINDINGPROC glad_glShaderStorageBlockBinding = NULL;
761 | PFNGLSPECIALIZESHADERPROC glad_glSpecializeShader = NULL;
762 | PFNGLSTENCILFUNCPROC glad_glStencilFunc = NULL;
763 | PFNGLSTENCILFUNCSEPARATEPROC glad_glStencilFuncSeparate = NULL;
764 | PFNGLSTENCILMASKPROC glad_glStencilMask = NULL;
765 | PFNGLSTENCILMASKSEPARATEPROC glad_glStencilMaskSeparate = NULL;
766 | PFNGLSTENCILOPPROC glad_glStencilOp = NULL;
767 | PFNGLSTENCILOPSEPARATEPROC glad_glStencilOpSeparate = NULL;
768 | PFNGLTEXBUFFERPROC glad_glTexBuffer = NULL;
769 | PFNGLTEXBUFFERRANGEPROC glad_glTexBufferRange = NULL;
770 | PFNGLTEXCOORDP1UIPROC glad_glTexCoordP1ui = NULL;
771 | PFNGLTEXCOORDP1UIVPROC glad_glTexCoordP1uiv = NULL;
772 | PFNGLTEXCOORDP2UIPROC glad_glTexCoordP2ui = NULL;
773 | PFNGLTEXCOORDP2UIVPROC glad_glTexCoordP2uiv = NULL;
774 | PFNGLTEXCOORDP3UIPROC glad_glTexCoordP3ui = NULL;
775 | PFNGLTEXCOORDP3UIVPROC glad_glTexCoordP3uiv = NULL;
776 | PFNGLTEXCOORDP4UIPROC glad_glTexCoordP4ui = NULL;
777 | PFNGLTEXCOORDP4UIVPROC glad_glTexCoordP4uiv = NULL;
778 | PFNGLTEXIMAGE1DPROC glad_glTexImage1D = NULL;
779 | PFNGLTEXIMAGE2DPROC glad_glTexImage2D = NULL;
780 | PFNGLTEXIMAGE2DMULTISAMPLEPROC glad_glTexImage2DMultisample = NULL;
781 | PFNGLTEXIMAGE3DPROC glad_glTexImage3D = NULL;
782 | PFNGLTEXIMAGE3DMULTISAMPLEPROC glad_glTexImage3DMultisample = NULL;
783 | PFNGLTEXPARAMETERIIVPROC glad_glTexParameterIiv = NULL;
784 | PFNGLTEXPARAMETERIUIVPROC glad_glTexParameterIuiv = NULL;
```

```
785 | PFNGLTEXPARAMETERFPROC glad_glTexParameterf = NULL;
786 | PFNGLTEXPARAMETERFVPROC glad_glTexParameterfv = NULL;
787 | PFNGLTEXPARAMETERIPROC glad_glTexParameteri = NULL;
788 | PFNGLTEXPARAMETERIVPROC glad_glTexParameteriv = NULL;
789 | PFNGLTEXTURESTORAGE1DPROC glad_glTexStorage1D = NULL;
790 | PFNGLTEXTURESTORAGE2DPROC glad_glTexStorage2D = NULL;
791 | PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glad_glTexStorage2DMultisample = NULL;
792 | PFNGLTEXTURESTORAGE3DPROC glad_glTexStorage3D = NULL;
793 | PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glad_glTexStorage3DMultisample = NULL;
794 | PFNGLTEXTURESUBIMAGE1DPROC glad_glTexSubImage1D = NULL;
795 | PFNGLTEXTURESUBIMAGE2DPROC glad_glTexSubImage2D = NULL;
796 | PFNGLTEXTURESUBIMAGE3DPROC glad_glTexSubImage3D = NULL;
797 | PFNGLTEXTUREBARRIERPROC glad_glTextureBarrier = NULL;
798 | PFNGLTEXTUREBUFFERPROC glad_glTextureBuffer = NULL;
799 | PFNGLTEXTUREBUFFERRANGEPROC glad_glTextureBufferRange = NULL;
800 | PFNGLTEXTUREPARAMETERIIVPROC glad_glTextureParameterIiv = NULL;
801 | PFNGLTEXTUREPARAMETERIUIVPROC glad_glTextureParameterIuiv = NULL;
802 | PFNGLTEXTUREPARAMETERFPROC glad_glTextureParameterf = NULL;
803 | PFNGLTEXTUREPARAMETERFVPROC glad_glTextureParameterfv = NULL;
804 | PFNGLTEXTUREPARAMETERIPROC glad_glTextureParameteri = NULL;
805 | PFNGLTEXTUREPARAMETERIVPROC glad_glTextureParameteriv = NULL;
806 | PFNGLTEXTURESTORAGE1DPROC glad_glTextureStorage1D = NULL;
807 | PFNGLTEXTURESTORAGE2DPROC glad_glTextureStorage2D = NULL;
808 | PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glad_glTextureStorage2DMultisample = NULL;
809 | PFNGLTEXTURESTORAGE3DPROC glad_glTextureStorage3D = NULL;
810 | PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glad_glTextureStorage3DMultisample = NULL;
811 | PFNGLTEXTURESUBIMAGE1DPROC glad_glTextureSubImage1D = NULL;
812 | PFNGLTEXTURESUBIMAGE2DPROC glad_glTextureSubImage2D = NULL;
813 | PFNGLTEXTURESUBIMAGE3DPROC glad_glTextureSubImage3D = NULL;
814 | PFNGLTEXTUREVIEWPROC glad_glTextureView = NULL;
815 | PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glad_glTransformFeedbackBufferBase = NULL;
816 | PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glad_glTransformFeedbackBufferRange = NULL;
817 | PFNGLTRANSFORMFEEDBACKVARYINGSPROC glad_glTransformFeedbackVaryings = NULL;
818 | PFNGLUNIFORM1DPROC glad_glUniform1d = NULL;
819 | PFNGLUNIFORM1DVPROC glad_glUniform1dv = NULL;
820 | PFNGLUNIFORM1FPROC glad_glUniform1f = NULL;
821 | PFNGLUNIFORM1FVPROC glad_glUniform1fv = NULL;
822 | PFNGLUNIFORM1IPROC glad_glUniform1i = NULL;
823 | PFNGLUNIFORM1IVPROC glad_glUniform1iv = NULL;
824 | PFNGLUNIFORM1UIPROC glad_glUniform1ui = NULL;
825 | PFNGLUNIFORM1UIVPROC glad_glUniform1uiv = NULL;
826 | PFNGLUNIFORM2DPROC glad_glUniform2d = NULL;
827 | PFNGLUNIFORM2DVPROC glad_glUniform2dv = NULL;
828 | PFNGLUNIFORM2FPROC glad_glUniform2f = NULL;
829 | PFNGLUNIFORM2FVPROC glad_glUniform2fv = NULL;
830 | PFNGLUNIFORM2IPROC glad_glUniform2i = NULL;
831 | PFNGLUNIFORM2IVPROC glad_glUniform2iv = NULL;
832 | PFNGLUNIFORM2UIPROC glad_glUniform2ui = NULL;
833 | PFNGLUNIFORM2UIVPROC glad_glUniform2uiv = NULL;
834 | PFNGLUNIFORM3DPROC glad_glUniform3d = NULL;
835 | PFNGLUNIFORM3DVPROC glad_glUniform3dv = NULL;
836 | PFNGLUNIFORM3FPROC glad_glUniform3f = NULL;
837 | PFNGLUNIFORM3FVPROC glad_glUniform3fv = NULL;
838 | PFNGLUNIFORM3IPROC glad_glUniform3i = NULL;
839 | PFNGLUNIFORM3IVPROC glad_glUniform3iv = NULL;
840 | PFNGLUNIFORM3UIPROC glad_glUniform3ui = NULL;
841 | PFNGLUNIFORM3UIVPROC glad_glUniform3uiv = NULL;
842 | PFNGLUNIFORM4DPROC glad_glUniform4d = NULL;
843 | PFNGLUNIFORM4DVPROC glad_glUniform4dv = NULL;
844 | PFNGLUNIFORM4FPROC glad_glUniform4f = NULL;
845 | PFNGLUNIFORM4FVPROC glad_glUniform4fv = NULL;
846 | PFNGLUNIFORM4IPROC glad_glUniform4i = NULL;
847 | PFNGLUNIFORM4IVPROC glad_glUniform4iv = NULL;
848 | PFNGLUNIFORM4UIPROC glad_glUniform4ui = NULL;
849 | PFNGLUNIFORM4UIVPROC glad_glUniform4uiv = NULL;
850 | PFNGLUNIFORMBLOCKBINDINGPROC glad_glUniformBlockBinding = NULL;
851 | PFNGLUNIFORMMATRIX2DVPROC glad_glUniformMatrix2dv = NULL;
852 | PFNGLUNIFORMMATRIX2FVPROC glad_glUniformMatrix2fv = NULL;
853 | PFNGLUNIFORMMATRIX2X3DVPROC glad_glUniformMatrix2x3dv = NULL;
854 | PFNGLUNIFORMMATRIX2X3FVPROC glad_glUniformMatrix2x3fv = NULL;
855 | PFNGLUNIFORMMATRIX2X4DVPROC glad_glUniformMatrix2x4dv = NULL;
856 | PFNGLUNIFORMMATRIX2X4FVPROC glad_glUniformMatrix2x4fv = NULL;
857 | PFNGLUNIFORMMATRIX3DVPROC glad_glUniformMatrix3dv = NULL;
858 | PFNGLUNIFORMMATRIX3FVPROC glad_glUniformMatrix3fv = NULL;
859 | PFNGLUNIFORMMATRIX3X2DVPROC glad_glUniformMatrix3x2dv = NULL;
860 | PFNGLUNIFORMMATRIX3X2FVPROC glad_glUniformMatrix3x2fv = NULL;
861 | PFNGLUNIFORMMATRIX3X4DVPROC glad_glUniformMatrix3x4dv = NULL;
862 | PFNGLUNIFORMMATRIX3X4FVPROC glad_glUniformMatrix3x4fv = NULL;
863 | PFNGLUNIFORMMATRIX4DVPROC glad_glUniformMatrix4dv = NULL;
```



```
864 | PFNGLUNIFORMMATRIX4FVPROC glad_glUniformMatrix4fv = NULL;
865 | PFNGLUNIFORMMATRIX4X2DVPROC glad_glUniformMatrix4x2dv = NULL;
866 | PFNGLUNIFORMMATRIX4X2FVPROC glad_glUniformMatrix4x2fv = NULL;
867 | PFNGLUNIFORMMATRIX4X3DVPROC glad_glUniformMatrix4x3dv = NULL;
868 | PFNGLUNIFORMMATRIX4X3FVPROC glad_glUniformMatrix4x3fv = NULL;
869 | PFNGLUNIFORMSUBROUTINESUIVPROC glad_glUniformSubroutinesuiv = NULL;
870 | PFNGLUNMAPBUFFERPROC glad_glUnmapBuffer = NULL;
871 | PFNGLUNMAPNAMEDBUFFERPROC glad_glUnmapNamedBuffer = NULL;
872 | PFNGLUSEPROGRAMPROC glad_glUseProgram = NULL;
873 | PFNGLUSEPROGRAMSTAGESPROC glad_glUseProgramStages = NULL;
874 | PFNGLVALIDATEPROGRAMPROC glad_glValidateProgram = NULL;
875 | PFNGLVALIDATEPROGRAMPIPELINEPROC glad_glValidateProgramPipeline = NULL;
876 | PFNGLVERTEXARRAYATTRIBBINDINGPROC glad_glVertexArrayAttribBinding = NULL;
877 | PFNGLVERTEXARRAYATTRIBFORMATPROC glad_glVertexArrayAttribFormat = NULL;
878 | PFNGLVERTEXARRAYATTRIBIFORMATPROC glad_glVertexArrayAttribIFormat = NULL;
879 | PFNGLVERTEXARRAYATTRIBIFORMATPROC glad_glVertexArrayAttribIFormat = NULL;
880 | PFNGLVERTEXARRAYBINDINGDIVISORPROC glad_glVertexArrayBindingDivisor = NULL;
881 | PFNGLVERTEXARRAYELEMENTBUFFERPROC glad_glVertexArrayElementBuffer = NULL;
882 | PFNGLVERTEXARRAYVERTEXBUFFERPROC glad_glVertexArrayVertexBuffer = NULL;
883 | PFNGLVERTEXARRAYVERTEXBUFFERSPROC glad_glVertexArrayVertexBuffers = NULL;
884 | PFNGLVERTEXATTRIB1DPROC glad_glVertexAttrib1d = NULL;
885 | PFNGLVERTEXATTRIB1DVPROC glad_glVertexAttrib1dv = NULL;
886 | PFNGLVERTEXATTRIB1FPROC glad_glVertexAttrib1f = NULL;
887 | PFNGLVERTEXATTRIB1FVPROC glad_glVertexAttrib1fv = NULL;
888 | PFNGLVERTEXATTRIB1SPROC glad_glVertexAttrib1s = NULL;
889 | PFNGLVERTEXATTRIB1SVPROC glad_glVertexAttrib1sv = NULL;
890 | PFNGLVERTEXATTRIB2DPROC glad_glVertexAttrib2d = NULL;
891 | PFNGLVERTEXATTRIB2DVPROC glad_glVertexAttrib2dv = NULL;
892 | PFNGLVERTEXATTRIB2FPROC glad_glVertexAttrib2f = NULL;
893 | PFNGLVERTEXATTRIB2FVPROC glad_glVertexAttrib2fv = NULL;
894 | PFNGLVERTEXATTRIB2SPROC glad_glVertexAttrib2s = NULL;
895 | PFNGLVERTEXATTRIB2SVPROC glad_glVertexAttrib2sv = NULL;
896 | PFNGLVERTEXATTRIB3DPROC glad_glVertexAttrib3d = NULL;
897 | PFNGLVERTEXATTRIB3DVPROC glad_glVertexAttrib3dv = NULL;
898 | PFNGLVERTEXATTRIB3FPROC glad_glVertexAttrib3f = NULL;
899 | PFNGLVERTEXATTRIB3FVPROC glad_glVertexAttrib3fv = NULL;
900 | PFNGLVERTEXATTRIB3SPROC glad_glVertexAttrib3s = NULL;
901 | PFNGLVERTEXATTRIB3SVPROC glad_glVertexAttrib3sv = NULL;
902 | PFNGLVERTEXATTRIB4NBVPROC glad_glVertexAttrib4Nbv = NULL;
903 | PFNGLVERTEXATTRIB4NIVPROC glad_glVertexAttrib4Niv = NULL;
904 | PFNGLVERTEXATTRIB4NSVPROC glad_glVertexAttrib4Nsv = NULL;
905 | PFNGLVERTEXATTRIB4NUBPROC glad_glVertexAttrib4Nub = NULL;
906 | PFNGLVERTEXATTRIB4NUBPROC glad_glVertexAttrib4Nubv = NULL;
907 | PFNGLVERTEXATTRIB4NUIVPROC glad_glVertexAttrib4Nuiv = NULL;
908 | PFNGLVERTEXATTRIB4NUSVPROC glad_glVertexAttrib4Nusv = NULL;
909 | PFNGLVERTEXATTRIB4BVPROC glad_glVertexAttrib4bv = NULL;
910 | PFNGLVERTEXATTRIB4DPROC glad_glVertexAttrib4d = NULL;
911 | PFNGLVERTEXATTRIB4DVPROC glad_glVertexAttrib4dv = NULL;
912 | PFNGLVERTEXATTRIB4FPROC glad_glVertexAttrib4f = NULL;
913 | PFNGLVERTEXATTRIB4FVPROC glad_glVertexAttrib4fv = NULL;
914 | PFNGLVERTEXATTRIB4IVPROC glad_glVertexAttrib4iv = NULL;
915 | PFNGLVERTEXATTRIB4SPROC glad_glVertexAttrib4s = NULL;
916 | PFNGLVERTEXATTRIB4SVPROC glad_glVertexAttrib4sv = NULL;
917 | PFNGLVERTEXATTRIB4UBVPROC glad_glVertexAttrib4ubv = NULL;
918 | PFNGLVERTEXATTRIB4UIVPROC glad_glVertexAttrib4uiv = NULL;
919 | PFNGLVERTEXATTRIB4USVPROC glad_glVertexAttrib4usv = NULL;
920 | PFNGLVERTEXATTRIBBINDINGPROC glad_glVertexAttribBinding = NULL;
921 | PFNGLVERTEXATTRIBDIVISORPROC glad_glVertexAttribDivisor = NULL;
922 | PFNGLVERTEXATTRIBFORMATPROC glad_glVertexAttribFormat = NULL;
923 | PFNGLVERTEXATTRIBI1IPROC glad_glVertexAttribI1i = NULL;
924 | PFNGLVERTEXATTRIBI1IVPROC glad_glVertexAttribI1iv = NULL;
925 | PFNGLVERTEXATTRIBI1UIPROC glad_glVertexAttribI1ui = NULL;
926 | PFNGLVERTEXATTRIBI1UIVPROC glad_glVertexAttribI1uiv = NULL;
927 | PFNGLVERTEXATTRIBI2IPROC glad_glVertexAttribI2i = NULL;
928 | PFNGLVERTEXATTRIBI2IVPROC glad_glVertexAttribI2iv = NULL;
929 | PFNGLVERTEXATTRIBI2UIPROC glad_glVertexAttribI2ui = NULL;
930 | PFNGLVERTEXATTRIBI2UIVPROC glad_glVertexAttribI2uiv = NULL;
931 | PFNGLVERTEXATTRIBI3IPROC glad_glVertexAttribI3i = NULL;
932 | PFNGLVERTEXATTRIBI3IVPROC glad_glVertexAttribI3iv = NULL;
933 | PFNGLVERTEXATTRIBI3UIPROC glad_glVertexAttribI3ui = NULL;
934 | PFNGLVERTEXATTRIBI3UIVPROC glad_glVertexAttribI3uiv = NULL;
935 | PFNGLVERTEXATTRIBI4BVPROC glad_glVertexAttribI4bv = NULL;
936 | PFNGLVERTEXATTRIBI4IPROC glad_glVertexAttribI4i = NULL;
937 | PFNGLVERTEXATTRIBI4IVPROC glad_glVertexAttribI4iv = NULL;
938 | PFNGLVERTEXATTRIBI4SVPROC glad_glVertexAttribI4sv = NULL;
939 | PFNGLVERTEXATTRIBI4UBVPROC glad_glVertexAttribI4ubv = NULL;
940 | PFNGLVERTEXATTRIBI4UIPROC glad_glVertexAttribI4ui = NULL;
941 | PFNGLVERTEXATTRIBI4UIVPROC glad_glVertexAttribI4uiv = NULL;
942 | PFNGLVERTEXATTRIBI4USVPROC glad_glVertexAttribI4usv = NULL;
```

```

943 | PFNGLVERTEXATTRIBIFORMATPROC glad_glVertexAttribIFormat = NULL;
944 | PFNGLVERTEXATTRIBIPOINTERPROC glad_glVertexAttribIPointer = NULL;
945 | PFNGLVERTEXATTRIBL1DPROC glad_glVertexAttribL1d = NULL;
946 | PFNGLVERTEXATTRIBL1DVPROC glad_glVertexAttribL1dv = NULL;
947 | PFNGLVERTEXATTRIBL2DPROC glad_glVertexAttribL2d = NULL;
948 | PFNGLVERTEXATTRIBL2DVPROC glad_glVertexAttribL2dv = NULL;
949 | PFNGLVERTEXATTRIBL3DPROC glad_glVertexAttribL3d = NULL;
950 | PFNGLVERTEXATTRIBL3DVPROC glad_glVertexAttribL3dv = NULL;
951 | PFNGLVERTEXATTRIBL4DPROC glad_glVertexAttribL4d = NULL;
952 | PFNGLVERTEXATTRIBL4DVPROC glad_glVertexAttribL4dv = NULL;
953 | PFNGLVERTEXATTRIBIFORMATPROC glad_glVertexAttribIFormat = NULL;
954 | PFNGLVERTEXATTRIBIPOINTERPROC glad_glVertexAttribIPointer = NULL;
955 | PFNGLVERTEXATTRIBP1UIPROC glad_glVertexAttribP1ui = NULL;
956 | PFNGLVERTEXATTRIBP1UIVPROC glad_glVertexAttribP1uiv = NULL;
957 | PFNGLVERTEXATTRIBP2UIPROC glad_glVertexAttribP2ui = NULL;
958 | PFNGLVERTEXATTRIBP2UIVPROC glad_glVertexAttribP2uiv = NULL;
959 | PFNGLVERTEXATTRIBP3UIPROC glad_glVertexAttribP3ui = NULL;
960 | PFNGLVERTEXATTRIBP3UIVPROC glad_glVertexAttribP3uiv = NULL;
961 | PFNGLVERTEXATTRIBP4UIPROC glad_glVertexAttribP4ui = NULL;
962 | PFNGLVERTEXATTRIBP4UIVPROC glad_glVertexAttribP4uiv = NULL;
963 | PFNGLVERTEXATTRIBIPOINTERPROC glad_glVertexAttribPointer = NULL;
964 | PFNGLVERTEXBINDINGDIVISORPROC glad_glVertexBindingDivisor = NULL;
965 | PFNGLVERTEXP2UIPROC glad_glVertexP2ui = NULL;
966 | PFNGLVERTEXP2UIVPROC glad_glVertexP2uiv = NULL;
967 | PFNGLVERTEXP3UIPROC glad_glVertexP3ui = NULL;
968 | PFNGLVERTEXP3UIVPROC glad_glVertexP3uiv = NULL;
969 | PFNGLVERTEXP4UIPROC glad_glVertexP4ui = NULL;
970 | PFNGLVERTEXP4UIVPROC glad_glVertexP4uiv = NULL;
971 | PFNGLVIEWPORTPROC glad_glViewport = NULL;
972 | PFNGLVIEWPORTARRAYVPROC glad_glViewportArrayv = NULL;
973 | PFNGLVIEWPORTINDEXEDFPROC glad_glViewportIndexedf = NULL;
974 | PFNGLVIEWPORTINDEXEDFVPROC glad_glViewportIndexedfv = NULL;
975 | PFNGLWAITSYNCPROC glad_glWaitSync = NULL;
976 | static void load_GL_VERSION_1_0(GLADloadproc load) {
977 |     if(!GLAD_GL_VERSION_1_0) return;
978 |     glad_glCullFace = (PFNGLCULLFACEPROC)load("glCullFace");
979 |     glad_glFrontFace = (PFNGLFRONTFACEPROC)load("glFrontFace");
980 |     glad_glHint = (PFNGLHINTPROC)load("glHint");
981 |     glad_glLineWidth = (PFNGLLINEWIDTHPROC)load("glLineWidth");
982 |     glad_glPointSize = (PFNGLPOINTSIZEPROC)load("glPointSize");
983 |     glad_glPolygonMode = (PFNGLPOLYGONMODEPROC)load("glPolygonMode");
984 |     glad_glScissor = (PFNGLSCISSORPROC)load("glScissor");
985 |     glad_glTexParameterf = (PFNGLTEXPARAMETERFPROC)load("glTexParameterf");
986 |     glad_glTexParameterfv = (PFNGLTEXPARAMETERFVPROC)load("glTexParameterfv");
987 |     glad_glTexParameteri = (PFNGLTEXPARAMETERIPROC)load("glTexParameteri");
988 |     glad_glTexParameteriv = (PFNGLTEXPARAMETERIVPROC)load("glTexParameteriv");
989 |     glad_glTexImage1D = (PFNGLTEXIMAGE1DPROC)load("glTexImage1D");
990 |     glad_glTexImage2D = (PFNGLTEXIMAGE2DPROC)load("glTexImage2D");
991 |     glad_glDrawBuffer = (PFNGLDRAWBUFFERPROC)load("glDrawBuffer");
992 |     glad_glClear = (PFNGLCLEARPROC)load("glClear");
993 |     glad_glClearColor = (PFNGLCLEARCOLORPROC)load("glClearColor");
994 |     glad_glClearStencil = (PFNGLCLEARSTENCILPROC)load("glClearStencil");
995 |     glad_glClearDepth = (PFNGLCLEARDEPTHPROC)load("glClearDepth");
996 |     glad_glStencilMask = (PFNGLSTENCILMASKPROC)load("glStencilMask");
997 |     glad_glColorMask = (PFNGLCOLORMASKPROC)load("glColorMask");
998 |     glad_glDepthMask = (PFNGLDEPTHMASKPROC)load("glDepthMask");
999 |     glad_glDisable = (PFNGLDISABLEPROC)load("glDisable");
1000 |     glad_glEnable = (PFNGLENABLEPROC)load("glEnable");
1001 |     glad_glFinish = (PFNGLFINISHPROC)load("glFinish");
1002 |     glad_glFlush = (PFNGLFLUSHPROC)load("glFlush");
1003 |     glad_glBlendFunc = (PFNGLBLENDFUNCPROC)load("glBlendFunc");
1004 |     glad_glLogicOp = (PFNGLLOGICOPPROC)load("glLogicOp");
1005 |     glad_glStencilFunc = (PFNGLSTENCILFUNCPROC)load("glStencilFunc");
1006 |     glad_glStencilOp = (PFNGLSTENCILOPPROC)load("glStencilOp");
1007 |     glad_glDepthFunc = (PFNGLDEPTHFUNCPROC)load("glDepthFunc");
1008 |     glad_glPixelStoref = (PFNGLPIXELSTOREFPROC)load("glPixelStoref");
1009 |     glad_glPixelStorei = (PFNGLPIXELSTOREIPROC)load("glPixelStorei");
1010 |     glad_glReadBuffer = (PFNGLREADBUFFERPROC)load("glReadBuffer");
1011 |     glad_glReadPixels = (PFNGLREADPIXELSPROC)load("glReadPixels");
1012 |     glad_glGetBooleanv = (PFNGLGETBOOLEANVPROC)load("glGetBooleanv");
1013 |     glad_glGetDoublev = (PFNGLGETDOUBLEVPROC)load("glGetDoublev");
1014 |     glad_glGetError = (PFNGLGETERRORPROC)load("glGetError");
1015 |     glad_glGetFloatv = (PFNGLGETFLOATVPROC)load("glGetFloatv");
1016 |     glad_glGetIntegerv = (PFNGLGETINTEGERVPROC)load("glGetIntegerv");
1017 |     glad_glGetString = (PFNGLGETSTRINGPROC)load("glGetString");
1018 |     glad_glGetTexImage = (PFNGLGETTEXIMAGEPROC)load("glGetTexImage");
1019 |     glad_glGetTexParameterfv = (PFNGLGETTEXPARAMETERFVPROC)load("glGetTexParameterfv");
1020 |     glad_glGetTexParameteriv = (PFNGLGETTEXPARAMETERIVPROC)load("glGetTexParameteriv");
1021 |     glad_glGetTexLevelParameterfv =

```

```

> (PFNGLGETTEXLEVELPARAMETERFVPROC) load("glGetTexLevelParameterfv");
1022 |   glad_glGetTexLevelParameteriv =
> (PFNGLGETTEXLEVELPARAMETERIVPROC) load("glGetTexLevelParameteriv");
1023 |   glad_glIsEnabled = (PFNGLISENABLEDPROC) load("glIsEnabled");
1024 |   glad_glDepthRange = (PFNGLDEPTHRANGEPROC) load("glDepthRange");
1025 |   glad_glViewport = (PFNGLVIEWPORTPROC) load("glViewport");
1026 | }
1027 | static void load_GL_VERSION_1_1(GLADloadproc load) {
1028 |   if(!GLAD_GL_VERSION_1_1) return;
1029 |   glad_glDrawArrays = (PFNGLDRAWARRAYSPROC) load("glDrawArrays");
1030 |   glad_glDrawElements = (PFNGLDRAWELEMENTSPROC) load("glDrawElements");
1031 |   glad_glPolygonOffset = (PFNGLPOLYGONOFFSETPROC) load("glPolygonOffset");
1032 |   glad_glCopyTexImage1D = (PFNGLCOPYTEXIMAGE1DPROC) load("glCopyTexImage1D");
1033 |   glad_glCopyTexImage2D = (PFNGLCOPYTEXIMAGE2DPROC) load("glCopyTexImage2D");
1034 |   glad_glCopyTexSubImage1D = (PFNGLCOPYTEXSUBIMAGE1DPROC) load("glCopyTexSubImage1D");
1035 |   glad_glCopyTexSubImage2D = (PFNGLCOPYTEXSUBIMAGE2DPROC) load("glCopyTexSubImage2D");
1036 |   glad_glTexSubImage1D = (PFNGLTEXSUBIMAGE1DPROC) load("glTexSubImage1D");
1037 |   glad_glTexSubImage2D = (PFNGLTEXSUBIMAGE2DPROC) load("glTexSubImage2D");
1038 |   glad_glBindTexture = (PFNGLBINDTEXTUREPROC) load("glBindTexture");
1039 |   glad_glDeleteTextures = (PFNGLDELETETEXTURESPROC) load("glDeleteTextures");
1040 |   glad_glGenTextures = (PFNGLGENTEXTURESPROC) load("glGenTextures");
1041 |   glad_glIsTexture = (PFNGLISTEXTUREPROC) load("glIsTexture");
1042 | }
1043 | static void load_GL_VERSION_1_2(GLADloadproc load) {
1044 |   if(!GLAD_GL_VERSION_1_2) return;
1045 |   glad_glDrawRangeElements = (PFNGLDRAWRANGELEMENTSPROC) load("glDrawRangeElements");
1046 |   glad_glTexImage3D = (PFNGLTEXIMAGE3DPROC) load("glTexImage3D");
1047 |   glad_glTexSubImage3D = (PFNGLTEXSUBIMAGE3DPROC) load("glTexSubImage3D");
1048 |   glad_glCopyTexSubImage3D = (PFNGLCOPYTEXSUBIMAGE3DPROC) load("glCopyTexSubImage3D");
1049 | }
1050 | static void load_GL_VERSION_1_3(GLADloadproc load) {
1051 |   if(!GLAD_GL_VERSION_1_3) return;
1052 |   glad_glActiveTexture = (PFNGLACTIVETEXTUREPROC) load("glActiveTexture");
1053 |   glad_glSampleCoverage = (PFNGLSAMPLECOVERAGEPROC) load("glSampleCoverage");
1054 |   glad_glCompressedTexImage3D = (PFNGLCOMPRESSEDTEXIMAGE3DPROC) load("glCompressedTexImage3D");
1055 |   glad_glCompressedTexSubImage2D = (PFNGLCOMPRESSEDTEXIMAGE2DPROC) load("glCompressedTexImage2D");
1056 |   glad_glCompressedTexImage1D = (PFNGLCOMPRESSEDTEXIMAGE1DPROC) load("glCompressedTexImage1D");
1057 |   glad_glCompressedTexSubImage3D =
> (PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC) load("glCompressedTexSubImage3D");
1058 |   glad_glCompressedTexSubImage2D =
> (PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC) load("glCompressedTexSubImage2D");
1059 |   glad_glCompressedTexSubImage1D =
> (PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC) load("glCompressedTexSubImage1D");
1060 |   glad_glGetCompressedTexImage = (PFNGLGETCOMPRESSEDTEXIMAGEPROC) load("glGetCompressedTexImage");
1061 | }
1062 | static void load_GL_VERSION_1_4(GLADloadproc load) {
1063 |   if(!GLAD_GL_VERSION_1_4) return;
1064 |   glad_glBlendFuncSeparate = (PFNGLBLENDFUNCSEPARATEPROC) load("glBlendFuncSeparate");
1065 |   glad_glMultiDrawArrays = (PFNGLMULTIDRAWARRAYSPROC) load("glMultiDrawArrays");
1066 |   glad_glMultiDrawElements = (PFNGLMULTIDRAWELEMENTSPROC) load("glMultiDrawElements");
1067 |   glad_glPointParameterf = (PFNGLPOINTPARAMETERFPROC) load("glPointParameterf");
1068 |   glad_glPointParameterfv = (PFNGLPOINTPARAMETERFVPROC) load("glPointParameterfv");
1069 |   glad_glPointParameteri = (PFNGLPOINTPARAMETERIPROC) load("glPointParameteri");
1070 |   glad_glPointParameteriv = (PFNGLPOINTPARAMETERIVPROC) load("glPointParameteriv");
1071 |   glad_glBlendColor = (PFNGLBLENDCOLORPROC) load("glBlendColor");
1072 |   glad_glBlendEquation = (PFNGLBLENDEQUATIONPROC) load("glBlendEquation");
1073 | }
1074 | static void load_GL_VERSION_1_5(GLADloadproc load) {
1075 |   if(!GLAD_GL_VERSION_1_5) return;
1076 |   glad_glGenQueries = (PFNGLGENQUERIESPROC) load("glGenQueries");
1077 |   glad_glDeleteQueries = (PFNGLDELETEQUERIESPROC) load("glDeleteQueries");
1078 |   glad_glIsQuery = (PFNGLISQUERYPROC) load("glIsQuery");
1079 |   glad_glBeginQuery = (PFNGLBEGINQUERYPROC) load("glBeginQuery");
1080 |   glad_glEndQuery = (PFNGLENDQUERYPROC) load("glEndQuery");
1081 |   glad_glGetQueryiv = (PFNGLGETQUERYIVPROC) load("glGetQueryiv");
1082 |   glad_glGetQueryObjectiv = (PFNGLGETQUERYOBJECTIVPROC) load("glGetQueryObjectiv");
1083 |   glad_glGetQueryObjectiui = (PFNGLGETQUERYOBJECTUIVPROC) load("glGetQueryObjectiui");
1084 |   glad_glBindBuffer = (PFNGLBINDBUFFERPROC) load("glBindBuffer");
1085 |   glad_glDeleteBuffers = (PFNGLDELETEBUFFERSPROC) load("glDeleteBuffers");
1086 |   glad_glGenBuffers = (PFNGLGENBUFFERSPROC) load("glGenBuffers");
1087 |   glad_glIsBuffer = (PFNGLISBUFFERPROC) load("glIsBuffer");
1088 |   glad_glBufferData = (PFNGLBUFFERDATAPROC) load("glBufferData");
1089 |   glad_glBufferSubData = (PFNGLBUFFERSUBDATAPROC) load("glBufferSubData");
1090 |   glad_glGetBufferSubData = (PFNGLGETBUFFERSUBDATAPROC) load("glGetBufferSubData");
1091 |   glad_glMapBuffer = (PFNGLMAPBUFFERPROC) load("glMapBuffer");
1092 |   glad_glUnmapBuffer = (PFNGLUNMAPBUFFERPROC) load("glUnmapBuffer");
1093 |   glad_glGetBufferParameteriv = (PFNGLGETBUFFERPARAMETERIVPROC) load("glGetBufferParameteriv");
1094 |   glad_glGetBufferPointerv = (PFNGLGETBUFFERPOINTERVPROC) load("glGetBufferPointerv");
1095 | }

```

```

1096 | static void load_GL_VERSION_2_0(GLADloadproc load) {
1097 |     if(!GLAD_GL_VERSION_2_0) return;
1098 |     glad_glBlendEquationSeparate = (PFNGLBLENDEQUATIONSEPARATEPROC)load("glBlendEquationSeparate");
1099 |     glad_glDrawBuffers = (PFNGLDRAWBUFFERSPROC)load("glDrawBuffers");
1100 |     glad_glStencilOpSeparate = (PFNGLSTENCILOPSEPARATEPROC)load("glStencilOpSeparate");
1101 |     glad_glStencilFuncSeparate = (PFNGLSTENCILFUNCSEPARATEPROC)load("glStencilFuncSeparate");
1102 |     glad_glStencilMaskSeparate = (PFNGLSTENCILMASKSEPARATEPROC)load("glStencilMaskSeparate");
1103 |     glad_glAttachShader = (PFNGLATTACHSHADERPROC)load("glAttachShader");
1104 |     glad_glBindAttribLocation = (PFNGLBINDATTRIBLOCATIONPROC)load("glBindAttribLocation");
1105 |     glad_glCompileShader = (PFNGLCOMPILESHADERPROC)load("glCompileShader");
1106 |     glad_glCreateProgram = (PFNGLCREATEPROGRAMPROC)load("glCreateProgram");
1107 |     glad_glCreateShader = (PFNGLCREATESHADERPROC)load("glCreateShader");
1108 |     glad_glDeleteProgram = (PFNGLDELETEPROGRAMPROC)load("glDeleteProgram");
1109 |     glad_glDeleteShader = (PFNGLDELETESHADERPROC)load("glDeleteShader");
1110 |     glad_glDetachShader = (PFNGLDETACHSHADERPROC)load("glDetachShader");
1111 |     glad_glDisableVertexAttribArray =
> (PFNGLDISABLEVERTEXATTRIBARRAYPROC)load("glDisableVertexAttribArray");
1112 |     glad_glEnableVertexAttribArray =
> (PFNGLENABLEVERTEXATTRIBARRAYPROC)load("glEnableVertexAttribArray");
1113 |     glad_glGetActiveAttrib = (PFNGLGETACTIVEATTRIBPROC)load("glGetActiveAttrib");
1114 |     glad_glGetActiveUniform = (PFNGLGETACTIVEUNIFORMPROC)load("glGetActiveUniform");
1115 |     glad_glGetAttachedShaders = (PFNGLGETATTACHEDSHADERSPROC)load("glGetAttachedShaders");
1116 |     glad_glGetAttribLocation = (PFNGLGETATTRIBLOCATIONPROC)load("glGetAttribLocation");
1117 |     glad_glGetProgramiv = (PFNGLGETPROGRAMIVPROC)load("glGetProgramiv");
1118 |     glad_glGetProgramInfoLog = (PFNGLGETPROGRAMINFOLOGPROC)load("glGetProgramInfoLog");
1119 |     glad_glGetShaderiv = (PFNGLGETSHADERIVPROC)load("glGetShaderiv");
1120 |     glad_glGetShaderInfoLog = (PFNGLGETSHADERINFOLOGPROC)load("glGetShaderInfoLog");
1121 |     glad_glGetShaderSource = (PFNGLGETSHADERSOURCEPROC)load("glGetShaderSource");
1122 |     glad_glGetUniformLocation = (PFNGLGETUNIFORMLOCATIONPROC)load("glGetUniformLocation");
1123 |     glad_glGetUniformfv = (PFNGLGETUNIFORMFVPROC)load("glGetUniformfv");
1124 |     glad_glGetUniformiv = (PFNGLGETUNIFORMIVPROC)load("glGetUniformiv");
1125 |     glad_glGetVertexAttribdv = (PFNGLGETVERTEXATTRIBDVPROC)load("glGetVertexAttribdv");
1126 |     glad_glGetVertexAttribfv = (PFNGLGETVERTEXATTRIBFVPROC)load("glGetVertexAttribfv");
1127 |     glad_glGetVertexAttribiv = (PFNGLGETVERTEXATTRIBIVPROC)load("glGetVertexAttribiv");
1128 |     glad_glGetVertexAttribPointerv =
> (PFNGLGETVERTEXATTRIBPOINTERVPROC)load("glGetVertexAttribPointerv");
1129 |     glad_glIsProgram = (PFNGLISPROGRAMPROC)load("glIsProgram");
1130 |     glad_glIsShader = (PFNGLISSHADERPROC)load("glIsShader");
1131 |     glad_glLinkProgram = (PFNGLLINKPROGRAMPROC)load("glLinkProgram");
1132 |     glad_glShaderSource = (PFNGLSHADERSOURCEPROC)load("glShaderSource");
1133 |     glad_glUseProgram = (PFNGLUSEPROGRAMPROC)load("glUseProgram");
1134 |     glad_glUniform1f = (PFNGLUNIFORM1FPROC)load("glUniform1f");
1135 |     glad_glUniform2f = (PFNGLUNIFORM2FPROC)load("glUniform2f");
1136 |     glad_glUniform3f = (PFNGLUNIFORM3FPROC)load("glUniform3f");
1137 |     glad_glUniform4f = (PFNGLUNIFORM4FPROC)load("glUniform4f");
1138 |     glad_glUniform1i = (PFNGLUNIFORM1IPROC)load("glUniform1i");
1139 |     glad_glUniform2i = (PFNGLUNIFORM2IPROC)load("glUniform2i");
1140 |     glad_glUniform3i = (PFNGLUNIFORM3IPROC)load("glUniform3i");
1141 |     glad_glUniform4i = (PFNGLUNIFORM4IPROC)load("glUniform4i");
1142 |     glad_glUniform1fv = (PFNGLUNIFORM1FVPROC)load("glUniform1fv");
1143 |     glad_glUniform2fv = (PFNGLUNIFORM2FVPROC)load("glUniform2fv");
1144 |     glad_glUniform3fv = (PFNGLUNIFORM3FVPROC)load("glUniform3fv");
1145 |     glad_glUniform4fv = (PFNGLUNIFORM4FVPROC)load("glUniform4fv");
1146 |     glad_glUniform1iv = (PFNGLUNIFORM1IVPROC)load("glUniform1iv");
1147 |     glad_glUniform2iv = (PFNGLUNIFORM2IVPROC)load("glUniform2iv");
1148 |     glad_glUniform3iv = (PFNGLUNIFORM3IVPROC)load("glUniform3iv");
1149 |     glad_glUniform4iv = (PFNGLUNIFORM4IVPROC)load("glUniform4iv");
1150 |     glad_glUniformMatrix2fv = (PFNGLUNIFORMMATRIX2FVPROC)load("glUniformMatrix2fv");
1151 |     glad_glUniformMatrix3fv = (PFNGLUNIFORMMATRIX3FVPROC)load("glUniformMatrix3fv");
1152 |     glad_glUniformMatrix4fv = (PFNGLUNIFORMMATRIX4FVPROC)load("glUniformMatrix4fv");
1153 |     glad_glValidateProgram = (PFNGLVALIDATEPROGRAMPROC)load("glValidateProgram");
1154 |     glad_glVertexAttrib1d = (PFNGLVERTEXATTRIB1DPROC)load("glVertexAttrib1d");
1155 |     glad_glVertexAttrib1dv = (PFNGLVERTEXATTRIB1DVPROC)load("glVertexAttrib1dv");
1156 |     glad_glVertexAttrib1f = (PFNGLVERTEXATTRIB1FPROC)load("glVertexAttrib1f");
1157 |     glad_glVertexAttrib1fv = (PFNGLVERTEXATTRIB1FVPROC)load("glVertexAttrib1fv");
1158 |     glad_glVertexAttrib1s = (PFNGLVERTEXATTRIB1SPROC)load("glVertexAttrib1s");
1159 |     glad_glVertexAttrib1sv = (PFNGLVERTEXATTRIB1SVPROC)load("glVertexAttrib1sv");
1160 |     glad_glVertexAttrib2d = (PFNGLVERTEXATTRIB2DPROC)load("glVertexAttrib2d");
1161 |     glad_glVertexAttrib2dv = (PFNGLVERTEXATTRIB2DVPROC)load("glVertexAttrib2dv");
1162 |     glad_glVertexAttrib2f = (PFNGLVERTEXATTRIB2FPROC)load("glVertexAttrib2f");
1163 |     glad_glVertexAttrib2fv = (PFNGLVERTEXATTRIB2FVPROC)load("glVertexAttrib2fv");
1164 |     glad_glVertexAttrib2s = (PFNGLVERTEXATTRIB2SPROC)load("glVertexAttrib2s");
1165 |     glad_glVertexAttrib2sv = (PFNGLVERTEXATTRIB2SVPROC)load("glVertexAttrib2sv");
1166 |     glad_glVertexAttrib3d = (PFNGLVERTEXATTRIB3DPROC)load("glVertexAttrib3d");
1167 |     glad_glVertexAttrib3dv = (PFNGLVERTEXATTRIB3DVPROC)load("glVertexAttrib3dv");
1168 |     glad_glVertexAttrib3f = (PFNGLVERTEXATTRIB3FPROC)load("glVertexAttrib3f");
1169 |     glad_glVertexAttrib3fv = (PFNGLVERTEXATTRIB3FVPROC)load("glVertexAttrib3fv");
1170 |     glad_glVertexAttrib3s = (PFNGLVERTEXATTRIB3SPROC)load("glVertexAttrib3s");
1171 |     glad_glVertexAttrib3sv = (PFNGLVERTEXATTRIB3SVPROC)load("glVertexAttrib3sv");

```



```

1172 |   glad_glVertexAttrib4Nbv = (PFNGLVERTEXATTRIB4NBVPROC)load("glVertexAttrib4Nbv");
1173 |   glad_glVertexAttrib4Niv = (PFNGLVERTEXATTRIB4NIVPROC)load("glVertexAttrib4Niv");
1174 |   glad_glVertexAttrib4Nsv = (PFNGLVERTEXATTRIB4NSVPROC)load("glVertexAttrib4Nsv");
1175 |   glad_glVertexAttrib4Nub = (PFNGLVERTEXATTRIB4NUBPROC)load("glVertexAttrib4Nub");
1176 |   glad_glVertexAttrib4Nubv = (PFNGLVERTEXATTRIB4NUBVPROC)load("glVertexAttrib4Nubv");
1177 |   glad_glVertexAttrib4Nuiv = (PFNGLVERTEXATTRIB4NUIVPROC)load("glVertexAttrib4Nuiv");
1178 |   glad_glVertexAttrib4Nusv = (PFNGLVERTEXATTRIB4NUSVPROC)load("glVertexAttrib4Nusv");
1179 |   glad_glVertexAttrib4bv = (PFNGLVERTEXATTRIB4BVPROC)load("glVertexAttrib4bv");
1180 |   glad_glVertexAttrib4d = (PFNGLVERTEXATTRIB4DPROC)load("glVertexAttrib4d");
1181 |   glad_glVertexAttrib4dv = (PFNGLVERTEXATTRIB4DVPROC)load("glVertexAttrib4dv");
1182 |   glad_glVertexAttrib4f = (PFNGLVERTEXATTRIB4FPROC)load("glVertexAttrib4f");
1183 |   glad_glVertexAttrib4fv = (PFNGLVERTEXATTRIB4FVPROC)load("glVertexAttrib4fv");
1184 |   glad_glVertexAttrib4iv = (PFNGLVERTEXATTRIB4IVPROC)load("glVertexAttrib4iv");
1185 |   glad_glVertexAttrib4s = (PFNGLVERTEXATTRIB4SPROC)load("glVertexAttrib4s");
1186 |   glad_glVertexAttrib4sv = (PFNGLVERTEXATTRIB4SVPROC)load("glVertexAttrib4sv");
1187 |   glad_glVertexAttrib4ubv = (PFNGLVERTEXATTRIB4UBVPROC)load("glVertexAttrib4ubv");
1188 |   glad_glVertexAttrib4uiv = (PFNGLVERTEXATTRIB4UIVPROC)load("glVertexAttrib4uiv");
1189 |   glad_glVertexAttrib4usv = (PFNGLVERTEXATTRIB4USVPROC)load("glVertexAttrib4usv");
1190 |   glad_glVertexAttribPointer = (PFNGLVERTEXATTRIBPOINTERPROC)load("glVertexAttribPointer");
1191 | }
1192 | static void load_GL_VERSION_2_1(GLADloadproc load) {
1193 |     if(!GLAD_GL_VERSION_2_1) return;
1194 |     glad_glUniformMatrix2x3fv = (PFNGLUNIFORMMATRIX2X3FVPROC)load("glUniformMatrix2x3fv");
1195 |     glad_glUniformMatrix3x2fv = (PFNGLUNIFORMMATRIX3X2FVPROC)load("glUniformMatrix3x2fv");
1196 |     glad_glUniformMatrix2x4fv = (PFNGLUNIFORMMATRIX2X4FVPROC)load("glUniformMatrix2x4fv");
1197 |     glad_glUniformMatrix4x2fv = (PFNGLUNIFORMMATRIX4X2FVPROC)load("glUniformMatrix4x2fv");
1198 |     glad_glUniformMatrix3x4fv = (PFNGLUNIFORMMATRIX3X4FVPROC)load("glUniformMatrix3x4fv");
1199 |     glad_glUniformMatrix4x3fv = (PFNGLUNIFORMMATRIX4X3FVPROC)load("glUniformMatrix4x3fv");
1200 | }
1201 | static void load_GL_VERSION_3_0(GLADloadproc load) {
1202 |     if(!GLAD_GL_VERSION_3_0) return;
1203 |     glad_glColorMaski = (PFNGLCOLORMASKIPROC)load("glColorMaski");
1204 |     glad_glGetBooleani_v = (PFNGLGETBOOLEANI_VPROC)load("glGetBooleani_v");
1205 |     glad_glGetIntegeri_v = (PFNGLGETINTEGERI_VPROC)load("glGetIntegeri_v");
1206 |     glad_glEnablei = (PFNGLENABLEIPROC)load("glEnablei");
1207 |     glad_glDisablei = (PFNGLDISABLEIPROC)load("glDisablei");
1208 |     glad_glIsEnabledi = (PFNGLISENABLEDIPROC)load("glIsEnabledi");
1209 |     glad_glBeginTransformFeedback =
1210 | > (PFNGLBEGINTRANSFORMFEEDBACKPROC)load("glBeginTransformFeedback");
1211 |     glad_glEndTransformFeedback = (PFNGLENDTRANSFORMFEEDBACKPROC)load("glEndTransformFeedback");
1212 |     glad_glBindBufferRange = (PFNGLBINDBUFFERRANGEPROC)load("glBindBufferRange");
1213 |     glad_glBindBufferBase = (PFNGLBINDBUFFERBASEPROC)load("glBindBufferBase");
1214 |     glad_glTransformFeedbackVaryings =
1215 | > (PFNGLTRANSFORMFEEDBACKVARYINGSPROC)load("glTransformFeedbackVaryings");
1216 |     glad_glGetTransformFeedbackVarying =
1217 | > (PFNGLGETTRANSFORMFEEDBACKVARYINGPROC)load("glGetTransformFeedbackVarying");
1218 |     glad_glClampColor = (PFNGLCLAMPCOLORPROC)load("glClampColor");
1219 |     glad_glBeginConditionalRender =
1220 | > (PFNGLBEGINCONDITIONALRENDERPROC)load("glBeginConditionalRender");
1221 |     glad_glEndConditionalRender = (PFNGLENDCONDITIONALRENDERPROC)load("glEndConditionalRender");
1222 |     glad_glVertexAttribIPointer = (PFNGLVERTEXATTRIBIPOINTERPROC)load("glVertexAttribIPointer");
1223 |     glad_glGetVertexAttribIiv = (PFNGLGETVERTEXATTRIBIIVPROC)load("glGetVertexAttribIiv");
1224 |     glad_glGetVertexAttribIuiv = (PFNGLGETVERTEXATTRIBUIVPROC)load("glGetVertexAttribIuiv");
1225 |     glad_glVertexAttribI1i = (PFNGLVERTEXATTRIBI1IPROC)load("glVertexAttribI1i");
1226 |     glad_glVertexAttribI2i = (PFNGLVERTEXATTRIBI2IPROC)load("glVertexAttribI2i");
1227 |     glad_glVertexAttribI3i = (PFNGLVERTEXATTRIBI3IPROC)load("glVertexAttribI3i");
1228 |     glad_glVertexAttribI4i = (PFNGLVERTEXATTRIBI4IPROC)load("glVertexAttribI4i");
1229 |     glad_glVertexAttribI1ui = (PFNGLVERTEXATTRIBI1UIPROC)load("glVertexAttribI1ui");
1230 |     glad_glVertexAttribI2ui = (PFNGLVERTEXATTRIBI2UIPROC)load("glVertexAttribI2ui");
1231 |     glad_glVertexAttribI3ui = (PFNGLVERTEXATTRIBI3UIPROC)load("glVertexAttribI3ui");
1232 |     glad_glVertexAttribI4ui = (PFNGLVERTEXATTRIBI4UIPROC)load("glVertexAttribI4ui");
1233 |     glad_glVertexAttribI1iv = (PFNGLVERTEXATTRIBI1IVPROC)load("glVertexAttribI1iv");
1234 |     glad_glVertexAttribI2iv = (PFNGLVERTEXATTRIBI2IVPROC)load("glVertexAttribI2iv");
1235 |     glad_glVertexAttribI3iv = (PFNGLVERTEXATTRIBI3IVPROC)load("glVertexAttribI3iv");
1236 |     glad_glVertexAttribI4iv = (PFNGLVERTEXATTRIBI4IVPROC)load("glVertexAttribI4iv");
1237 |     glad_glVertexAttribI1uiv = (PFNGLVERTEXATTRIBI1UIVPROC)load("glVertexAttribI1uiv");
1238 |     glad_glVertexAttribI2uiv = (PFNGLVERTEXATTRIBI2UIVPROC)load("glVertexAttribI2uiv");
1239 |     glad_glVertexAttribI3uiv = (PFNGLVERTEXATTRIBI3UIVPROC)load("glVertexAttribI3uiv");
1240 |     glad_glVertexAttribI4uiv = (PFNGLVERTEXATTRIBI4UIVPROC)load("glVertexAttribI4uiv");
1241 |     glad_glVertexAttribI4bv = (PFNGLVERTEXATTRIBI4BVPROC)load("glVertexAttribI4bv");
1242 |     glad_glVertexAttribI4sv = (PFNGLVERTEXATTRIBI4SVPROC)load("glVertexAttribI4sv");
1243 |     glad_glVertexAttribI4ubv = (PFNGLVERTEXATTRIBI4UBVPROC)load("glVertexAttribI4ubv");
1244 |     glad_glVertexAttribI4usv = (PFNGLVERTEXATTRIBI4USVPROC)load("glVertexAttribI4usv");
1245 |     glad_glGetUniformiiv = (PFNGLGETUNIFORMIIVPROC)load("glGetUniformiiv");
1246 |     glad_glBindFragDataLocation = (PFNGLBINDFRAGDATALOCATIONPROC)load("glBindFragDataLocation");
1247 |     glad_glGetFragDataLocation = (PFNGLGETFRAGDATALOCATIONPROC)load("glGetFragDataLocation");
1248 |     glad_glUniform1ui = (PFNGLUNIFORM1UIPROC)load("glUniform1ui");
1249 |     glad_glUniform2ui = (PFNGLUNIFORM2UIPROC)load("glUniform2ui");
1250 |     glad_glUniform3ui = (PFNGLUNIFORM3UIPROC)load("glUniform3ui");

```

```

1247 |   glad_glUniform4ui = (PFNGLUNIFORM4UIPROC)load("glUniform4ui");
1248 |   glad_glUniform1ui = (PFNGLUNIFORM1UIVPROC)load("glUniform1ui");
1249 |   glad_glUniform2ui = (PFNGLUNIFORM2UIVPROC)load("glUniform2ui");
1250 |   glad_glUniform3ui = (PFNGLUNIFORM3UIVPROC)load("glUniform3ui");
1251 |   glad_glUniform4ui = (PFNGLUNIFORM4UIVPROC)load("glUniform4ui");
1252 |   glad_glTexParameterIiv = (PFNGLTEXPARAMETERIIVPROC)load("glTexParameterIiv");
1253 |   glad_glTexParameterIui = (PFNGLTEXPARAMETERIUIVPROC)load("glTexParameterIui");
1254 |   glad_glGetTexParameterIiv = (PFNGLGETTEXPARAMETERIIVPROC)load("glGetTexParameterIiv");
1255 |   glad_glGetTexParameterIui = (PFNGLGETTEXPARAMETERIUIVPROC)load("glGetTexParameterIui");
1256 |   glad_glClearBufferiv = (PFNGLCLEARBUFFERIVPROC)load("glClearBufferiv");
1257 |   glad_glClearBufferiui = (PFNGLCLEARBUFFERUIVPROC)load("glClearBufferiui");
1258 |   glad_glClearBufferfv = (PFNGLCLEARBUFFERFVPROC)load("glClearBufferfv");
1259 |   glad_glClearBufferfi = (PFNGLCLEARBUFFERFIPROC)load("glClearBufferfi");
1260 |   glad_glGetStringi = (PFNGLGETSTRINGIPROC)load("glGetStringi");
1261 |   glad_glIsRenderbuffer = (PFNGLISRENDERBUFFERPROC)load("glIsRenderbuffer");
1262 |   glad_glBindRenderbuffer = (PFNGLBINDRENDERBUFFERPROC)load("glBindRenderbuffer");
1263 |   glad_glDeleteRenderbuffers = (PFNGLDELETERENDERBUFFERSPROC)load("glDeleteRenderbuffers");
1264 |   glad_glGenRenderbuffers = (PFNGLGENRENDERBUFFERSPROC)load("glGenRenderbuffers");
1265 |   glad_glRenderbufferStorage = (PFNGLRENDERBUFFERSTORAGEPROC)load("glRenderbufferStorage");
1266 |   glad_glGetRenderbufferParameteriv =
> (PFNGLGETRENDERBUFFERPARAMETERIVPROC)load("glGetRenderbufferParameteriv");
1267 |   glad_glIsFramebuffer = (PFNGLISFRAMEBUFFERPROC)load("glIsFramebuffer");
1268 |   glad_glBindFramebuffer = (PFNGLBINDFRAMEBUFFERPROC)load("glBindFramebuffer");
1269 |   glad_glDeleteFramebuffers = (PFNGLDELETEFRAMEBUFFERSPROC)load("glDeleteFramebuffers");
1270 |   glad_glGenFramebuffers = (PFNGLGENFRAMEBUFFERSPROC)load("glGenFramebuffers");
1271 |   glad_glCheckFramebufferStatus =
> (PFNGLCHECKFRAMEBUFFERSTATUSPROC)load("glCheckFramebufferStatus");
1272 |   glad_glFramebufferTexture1D = (PFNGLFRAMEBUFFERTEXTURE1DPROC)load("glFramebufferTexture1D");
1273 |   glad_glFramebufferTexture2D = (PFNGLFRAMEBUFFERTEXTURE2DPROC)load("glFramebufferTexture2D");
1274 |   glad_glFramebufferTexture3D = (PFNGLFRAMEBUFFERTEXTURE3DPROC)load("glFramebufferTexture3D");
1275 |   glad_glFramebufferRenderbuffer =
> (PFNGLFRAMEBUFFERRENDERBUFFERPROC)load("glFramebufferRenderbuffer");
1276 |   glad_glGetFramebufferAttachmentParameteriv =
> (PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC)load("glGetFramebufferAttachmentParameteriv");
1277 |   glad_glGenerateMipmap = (PFNGLGENERATEMIPMAPPROC)load("glGenerateMipmap");
1278 |   glad_glBlitFramebuffer = (PFNGLBLITFRAMEBUFFERPROC)load("glBlitFramebuffer");
1279 |   glad_glRenderbufferStorageMultisample =
> (PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC)load("glRenderbufferStorageMultisample");
1280 |   glad_glFramebufferTextureLayer =
> (PFNGLFRAMEBUFFERTEXTURELAYERPROC)load("glFramebufferTextureLayer");
1281 |   glad_glMapBufferRange = (PFNGLMAPBUFFERRANGEPROC)load("glMapBufferRange");
1282 |   glad_glFlushMappedBufferRange =
> (PFNGLFLUSHMAPPEDBUFFERRANGEPROC)load("glFlushMappedBufferRange");
1283 |   glad_glBindVertexArray = (PFNGLBINDVERTEXARRAYPROC)load("glBindVertexArray");
1284 |   glad_glDeleteVertexArrays = (PFNGLDELETEVERTEXARRAYSPROC)load("glDeleteVertexArrays");
1285 |   glad_glGenVertexArrays = (PFNGLGENVERTEXARRAYSPROC)load("glGenVertexArrays");
1286 |   glad_glIsVertexArray = (PFNGLISVERTEXARRAYPROC)load("glIsVertexArray");
1287 | }
1288 | static void load_GL_VERSION_3_1(GLADloadproc load) {
1289 |   if(!GLAD_GL_VERSION_3_1) return;
1290 |   glad_glDrawArraysInstanced = (PFNGLDRAWARRAYSINSTANCEDPROC)load("glDrawArraysInstanced");
1291 |   glad_glDrawElementsInstanced = (PFNGLDRAWELEMENTSINSTANCEDPROC)load("glDrawElementsInstanced");
1292 |   glad_glTexBuffer = (PFNGLTEXBUFFERPROC)load("glTexBuffer");
1293 |   glad_glPrimitiveRestartIndex = (PFNGLPRIMITIVERESTARTINDEXPROC)load("glPrimitiveRestartIndex");
1294 |   glad_glCopyBufferSubData = (PFNGLCOPYBUFFERSUBDATAPROC)load("glCopyBufferSubData");
1295 |   glad_glGetUniformIndices = (PFNGLGETUNIFORMINDICESPROC)load("glGetUniformIndices");
1296 |   glad_glGetActiveUniformsiv = (PFNGLGETACTIVEUNIFORMSIVPROC)load("glGetActiveUniformsiv");
1297 |   glad_glGetActiveUniformName = (PFNGLGETACTIVEUNIFORMNAMEPROC)load("glGetActiveUniformName");
1298 |   glad_glGetUniformBlockIndex = (PFNGLGETUNIFORMBLOCKINDEXPROC)load("glGetUniformBlockIndex");
1299 |   glad_glGetActiveUniformBlockiv =
> (PFNGLGETACTIVEUNIFORMBLOCKIVPROC)load("glGetActiveUniformBlockiv");
1300 |   glad_glGetActiveUniformBlockName =
> (PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC)load("glGetActiveUniformBlockName");
1301 |   glad_glUniformBlockBinding = (PFNGLUNIFORMBLOCKBINDINGPROC)load("glUniformBlockBinding");
1302 |   glad_glBindBufferRange = (PFNGLBINDBUFFERRANGEPROC)load("glBindBufferRange");
1303 |   glad_glBindBufferBase = (PFNGLBINDBUFFERBASEPROC)load("glBindBufferBase");
1304 |   glad_glGetIntegeri_v = (PFNGLGETINTEGERI_VPROC)load("glGetIntegeri_v");
1305 | }
1306 | static void load_GL_VERSION_3_2(GLADloadproc load) {
1307 |   if(!GLAD_GL_VERSION_3_2) return;
1308 |   glad_glDrawElementsBaseVertex =
> (PFNGLDRAWELEMENTSBASEVERTEXPROC)load("glDrawElementsBaseVertex");
1309 |   glad_glDrawRangeElementsBaseVertex =
> (PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC)load("glDrawRangeElementsBaseVertex");
1310 |   glad_glDrawElementsInstancedBaseVertex =
> (PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC)load("glDrawElementsInstancedBaseVertex");
1311 |   glad_glMultiDrawElementsBaseVertex =
> (PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC)load("glMultiDrawElementsBaseVertex");
1312 |   glad_glProvokingVertex = (PFNGLPROVOKINGVERTEXPROC)load("glProvokingVertex");

```

```
1313 |     glad_glFenceSync = (PFNGLFENCESYNCPROC)load("glFenceSync");
1314 |     glad_glIsSync = (PFNGLISSYNCPROC)load("glIsSync");
1315 |     glad_glDeleteSync = (PFNGLDELETESYNCPROC)load("glDeleteSync");
1316 |     glad_glClientWaitSync = (PFNGLCLIENTWAITSYNCPROC)load("glClientWaitSync");
1317 |     glad_glWaitSync = (PFNGLWAITSYNCPROC)load("glWaitSync");
1318 |     glad_glGetInteger64v = (PFNGLGETINTEGER64VPROC)load("glGetInteger64v");
1319 |     glad_glGetSynciv = (PFNGLGETSYNCAIVPROC)load("glGetSynciv");
1320 |     glad_glGetInteger64i_v = (PFNGLGETINTEGER64I_VPROC)load("glGetInteger64i_v");
1321 |     glad_glGetBufferParameteri64v =
> (PFNGLGETBUFFERPARAMETERI64VPROC)load("glGetBufferParameteri64v");
1322 |     glad_glFramebufferTexture = (PFNGLFRAMEBUFFERTEXTUREPROC)load("glFramebufferTexture");
1323 |     glad_glTexImage2DMultisample = (PFNGLTEXIMAGE2DMULTISAMPLEPROC)load("glTexImage2DMultisample");
1324 |     glad_glTexImage3DMultisample = (PFNGLTEXIMAGE3DMULTISAMPLEPROC)load("glTexImage3DMultisample");
1325 |     glad_glGetMultisamplefv = (PFNGLGETMULTISAMPLEFVPROC)load("glGetMultisamplefv");
1326 |     glad_glSampleMaski = (PFNGLSAMPLEMASKIPROC)load("glSampleMaski");
1327 | }
1328 | static void load_GL_VERSION_3_3(GLADloadproc load) {
1329 |     if(!GLAD_GL_VERSION_3_3) return;
1330 |     glad_glBindFragDataLocationIndexed =
> (PFNGLBINDFRAGDATALOCATIONINDEXEDPROC)load("glBindFragDataLocationIndexed");
1331 |     glad_glGetFragDataIndex = (PFNGLGETFRAGDATAINDEXPROC)load("glGetFragDataIndex");
1332 |     glad_glGenSamplers = (PFNGLGENSAMPLERSPROC)load("glGenSamplers");
1333 |     glad_glDeleteSamplers = (PFNGLDELETESAMPLERSPROC)load("glDeleteSamplers");
1334 |     glad_glIsSampler = (PFNGLISSAMPLERPROC)load("glIsSampler");
1335 |     glad_glBindSampler = (PFNGLBINDSAMPLERPROC)load("glBindSampler");
1336 |     glad_glSamplerParameteri = (PFNGLSAMPLERPARAMETERIPROC)load("glSamplerParameteri");
1337 |     glad_glSamplerParameteriv = (PFNGLSAMPLERPARAMETERIVPROC)load("glSamplerParameteriv");
1338 |     glad_glSamplerParameterf = (PFNGLSAMPLERPARAMETERFPROC)load("glSamplerParameterf");
1339 |     glad_glSamplerParameterfv = (PFNGLSAMPLERPARAMETERFVPROC)load("glSamplerParameterfv");
1340 |     glad_glSamplerParameteriiv = (PFNGLSAMPLERPARAMETERIIVPROC)load("glSamplerParameteriiv");
1341 |     glad_glSamplerParameteriui = (PFNGLSAMPLERPARAMETERIUIVPROC)load("glSamplerParameteriui");
1342 |     glad_glGetSamplerParameteriv = (PFNGLGETSAMPLERPARAMETERIVPROC)load("glGetSamplerParameteriv");
1343 |     glad_glGetSamplerParameteriiv =
> (PFNGLGETSAMPLERPARAMETERIIVPROC)load("glGetSamplerParameteriiv");
1344 |     glad_glGetSamplerParameterfv = (PFNGLGETSAMPLERPARAMETERFVPROC)load("glGetSamplerParameterfv");
1345 |     glad_glGetSamplerParameteriui =
> (PFNGLGETSAMPLERPARAMETERIUIVPROC)load("glGetSamplerParameteriui");
1346 |     glad_glQueryCounter = (PFNGLQUERYCOUNTERPROC)load("glQueryCounter");
1347 |     glad_glGetQueryObjecti64v = (PFNGLGETQUERYOBJECTI64VPROC)load("glGetQueryObjecti64v");
1348 |     glad_glGetQueryObjectui64v = (PFNGLGETQUERYOBJECTUI64VPROC)load("glGetQueryObjectui64v");
1349 |     glad_glVertexAttribDivisor = (PFNGLVERTEXATTRIBDIVISORPROC)load("glVertexAttribDivisor");
1350 |     glad_glVertexAttribP1ui = (PFNGLVERTEXATTRIBP1UIPROC)load("glVertexAttribP1ui");
1351 |     glad_glVertexAttribP1uiv = (PFNGLVERTEXATTRIBP1UIVPROC)load("glVertexAttribP1uiv");
1352 |     glad_glVertexAttribP2ui = (PFNGLVERTEXATTRIBP2UIPROC)load("glVertexAttribP2ui");
1353 |     glad_glVertexAttribP2uiv = (PFNGLVERTEXATTRIBP2UIVPROC)load("glVertexAttribP2uiv");
1354 |     glad_glVertexAttribP3ui = (PFNGLVERTEXATTRIBP3UIPROC)load("glVertexAttribP3ui");
1355 |     glad_glVertexAttribP3uiv = (PFNGLVERTEXATTRIBP3UIVPROC)load("glVertexAttribP3uiv");
1356 |     glad_glVertexAttribP4ui = (PFNGLVERTEXATTRIBP4UIPROC)load("glVertexAttribP4ui");
1357 |     glad_glVertexAttribP4uiv = (PFNGLVERTEXATTRIBP4UIVPROC)load("glVertexAttribP4uiv");
1358 |     glad_glVertexP2ui = (PFNGLVERTEXP2UIPROC)load("glVertexP2ui");
1359 |     glad_glVertexP2uiv = (PFNGLVERTEXP2UIVPROC)load("glVertexP2uiv");
1360 |     glad_glVertexP3ui = (PFNGLVERTEXP3UIPROC)load("glVertexP3ui");
1361 |     glad_glVertexP3uiv = (PFNGLVERTEXP3UIVPROC)load("glVertexP3uiv");
1362 |     glad_glVertexP4ui = (PFNGLVERTEXP4UIPROC)load("glVertexP4ui");
1363 |     glad_glVertexP4uiv = (PFNGLVERTEXP4UIVPROC)load("glVertexP4uiv");
1364 |     glad_glTexCoordP1ui = (PFNGLTEXCOORDP1UIPROC)load("glTexCoordP1ui");
1365 |     glad_glTexCoordP1uiv = (PFNGLTEXCOORDP1UIVPROC)load("glTexCoordP1uiv");
1366 |     glad_glTexCoordP2ui = (PFNGLTEXCOORDP2UIPROC)load("glTexCoordP2ui");
1367 |     glad_glTexCoordP2uiv = (PFNGLTEXCOORDP2UIVPROC)load("glTexCoordP2uiv");
1368 |     glad_glTexCoordP3ui = (PFNGLTEXCOORDP3UIPROC)load("glTexCoordP3ui");
1369 |     glad_glTexCoordP3uiv = (PFNGLTEXCOORDP3UIVPROC)load("glTexCoordP3uiv");
1370 |     glad_glTexCoordP4ui = (PFNGLTEXCOORDP4UIPROC)load("glTexCoordP4ui");
1371 |     glad_glTexCoordP4uiv = (PFNGLTEXCOORDP4UIVPROC)load("glTexCoordP4uiv");
1372 |     glad_glMultiTexCoordP1ui = (PFNGLMULTITEXCOORDP1UIPROC)load("glMultiTexCoordP1ui");
1373 |     glad_glMultiTexCoordP1uiv = (PFNGLMULTITEXCOORDP1UIVPROC)load("glMultiTexCoordP1uiv");
1374 |     glad_glMultiTexCoordP2ui = (PFNGLMULTITEXCOORDP2UIPROC)load("glMultiTexCoordP2ui");
1375 |     glad_glMultiTexCoordP2uiv = (PFNGLMULTITEXCOORDP2UIVPROC)load("glMultiTexCoordP2uiv");
1376 |     glad_glMultiTexCoordP3ui = (PFNGLMULTITEXCOORDP3UIPROC)load("glMultiTexCoordP3ui");
1377 |     glad_glMultiTexCoordP3uiv = (PFNGLMULTITEXCOORDP3UIVPROC)load("glMultiTexCoordP3uiv");
1378 |     glad_glMultiTexCoordP4ui = (PFNGLMULTITEXCOORDP4UIPROC)load("glMultiTexCoordP4ui");
1379 |     glad_glMultiTexCoordP4uiv = (PFNGLMULTITEXCOORDP4UIVPROC)load("glMultiTexCoordP4uiv");
1380 |     glad_glNormalP3ui = (PFNGLNORMALP3UIPROC)load("glNormalP3ui");
1381 |     glad_glNormalP3uiv = (PFNGLNORMALP3UIVPROC)load("glNormalP3uiv");
1382 |     glad_glColorP3ui = (PFNGLCOLORP3UIPROC)load("glColorP3ui");
1383 |     glad_glColorP3uiv = (PFNGLCOLORP3UIVPROC)load("glColorP3uiv");
1384 |     glad_glColorP4ui = (PFNGLCOLORP4UIPROC)load("glColorP4ui");
1385 |     glad_glColorP4uiv = (PFNGLCOLORP4UIVPROC)load("glColorP4uiv");
1386 |     glad_glSecondaryColorP3ui = (PFNGLSECONDARYCOLORP3UIPROC)load("glSecondaryColorP3ui");
1387 |     glad_glSecondaryColorP3uiv = (PFNGLSECONDARYCOLORP3UIVPROC)load("glSecondaryColorP3uiv");
```



```

1388 | }
1389 | static void load_GL_VERSION_4_0(GLADloadproc load) {
1390 |     if(!GLAD_GL_VERSION_4_0) return;
1391 |     glad_glMinSampleShading = (PFNGLMINSAMPLESHADINGPROC)load("glMinSampleShading");
1392 |     glad_glBlendEquationi = (PFNGLBLENDEQUATIONIPROC)load("glBlendEquationi");
1393 |     glad_glBlendEquationSeparatei =
> (PFNGLBLENDEQUATIONSEPARATEIPROC)load("glBlendEquationSeparatei");
1394 |     glad_glBlendFunci = (PFNGLBLENDFUNCIPROC)load("glBlendFunci");
1395 |     glad_glBlendFuncSeparatei = (PFNGLBLENDFUNCSEPARATEIPROC)load("glBlendFuncSeparatei");
1396 |     glad_glDrawArraysIndirect = (PFNGLDRAWARRAYSINDIRECTPROC)load("glDrawArraysIndirect");
1397 |     glad_glDrawElementsIndirect = (PFNGLDRAWELEMENTSINDIRECTPROC)load("glDrawElementsIndirect");
1398 |     glad_glUniform1d = (PFNGLUNIFORM1DPROC)load("glUniform1d");
1399 |     glad_glUniform2d = (PFNGLUNIFORM2DPROC)load("glUniform2d");
1400 |     glad_glUniform3d = (PFNGLUNIFORM3DPROC)load("glUniform3d");
1401 |     glad_glUniform4d = (PFNGLUNIFORM4DPROC)load("glUniform4d");
1402 |     glad_glUniform1dv = (PFNGLUNIFORM1DVPROC)load("glUniform1dv");
1403 |     glad_glUniform2dv = (PFNGLUNIFORM2DVPROC)load("glUniform2dv");
1404 |     glad_glUniform3dv = (PFNGLUNIFORM3DVPROC)load("glUniform3dv");
1405 |     glad_glUniform4dv = (PFNGLUNIFORM4DVPROC)load("glUniform4dv");
1406 |     glad_glUniformMatrix2dv = (PFNGLUNIFORMMATRIX2DVPROC)load("glUniformMatrix2dv");
1407 |     glad_glUniformMatrix3dv = (PFNGLUNIFORMMATRIX3DVPROC)load("glUniformMatrix3dv");
1408 |     glad_glUniformMatrix4dv = (PFNGLUNIFORMMATRIX4DVPROC)load("glUniformMatrix4dv");
1409 |     glad_glUniformMatrix2x3dv = (PFNGLUNIFORMMATRIX2X3DVPROC)load("glUniformMatrix2x3dv");
1410 |     glad_glUniformMatrix2x4dv = (PFNGLUNIFORMMATRIX2X4DVPROC)load("glUniformMatrix2x4dv");
1411 |     glad_glUniformMatrix3x2dv = (PFNGLUNIFORMMATRIX3X2DVPROC)load("glUniformMatrix3x2dv");
1412 |     glad_glUniformMatrix3x4dv = (PFNGLUNIFORMMATRIX3X4DVPROC)load("glUniformMatrix3x4dv");
1413 |     glad_glUniformMatrix4x2dv = (PFNGLUNIFORMMATRIX4X2DVPROC)load("glUniformMatrix4x2dv");
1414 |     glad_glUniformMatrix4x3dv = (PFNGLUNIFORMMATRIX4X3DVPROC)load("glUniformMatrix4x3dv");
1415 |     glad_glGetUniformdv = (PFNGLGETUNIFORMDVPROC)load("glGetUniformdv");
1416 |     glad_glGetSubroutineUniformLocation =
> (PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC)load("glGetSubroutineUniformLocation");
1417 |     glad_glGetSubroutineIndex = (PFNGLGETSUBROUTINEINDEXPROC)load("glGetSubroutineIndex");
1418 |     glad_glGetActiveSubroutineUniformiv =
> (PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC)load("glGetActiveSubroutineUniformiv");
1419 |     glad_glGetActiveSubroutineUniformName =
> (PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC)load("glGetActiveSubroutineUniformName");
1420 |     glad_glGetActiveSubroutineName =
> (PFNGLGETACTIVESUBROUTINENAMEPROC)load("glGetActiveSubroutineName");
1421 |     glad_glUniformSubroutinesuiv = (PFNGLUNIFORMSUBROUTINESUIVPROC)load("glUniformSubroutinesuiv");
1422 |     glad_glGetUniformSubroutineuiv =
> (PFNGLGETUNIFORMSUBROUTINEUIVPROC)load("glGetUniformSubroutineuiv");
1423 |     glad_glGetProgramStageiv = (PFNGLGETPROGRAMSTAGEIVPROC)load("glGetProgramStageiv");
1424 |     glad_glPatchParameteri = (PFNGLPATCHPARAMETERIPROC)load("glPatchParameteri");
1425 |     glad_glPatchParameterfv = (PFNGLPATCHPARAMETERFVPROC)load("glPatchParameterfv");
1426 |     glad_glBindTransformFeedback = (PFNGLBINDTRANSFORMFEEDBACKPROC)load("glBindTransformFeedback");
1427 |     glad_glDeleteTransformFeedbacks =
> (PFNGLDELETETRANSFORMFEEDBACKSPROC)load("glDeleteTransformFeedbacks");
1428 |     glad_glGenTransformFeedbacks = (PFNGLGENTRANSFORMFEEDBACKSPROC)load("glGenTransformFeedbacks");
1429 |     glad_glIsTransformFeedback = (PFNGLISTRANSFORMFEEDBACKPROC)load("glIsTransformFeedback");
1430 |     glad_glPauseTransformFeedback =
> (PFNGLPAUSETRANSFORMFEEDBACKPROC)load("glPauseTransformFeedback");
1431 |     glad_glResumeTransformFeedback =
> (PFNGLRESUMETRANSFORMFEEDBACKPROC)load("glResumeTransformFeedback");
1432 |     glad_glDrawTransformFeedback = (PFNGLDRAWTRANSFORMFEEDBACKPROC)load("glDrawTransformFeedback");
1433 |     glad_glDrawTransformFeedbackStream =
> (PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC)load("glDrawTransformFeedbackStream");
1434 |     glad_glBeginQueryIndexed = (PFNGLBEGINQUERYINDEXEDPROC)load("glBeginQueryIndexed");
1435 |     glad_glEndQueryIndexed = (PFNGLENDQUERYINDEXEDPROC)load("glEndQueryIndexed");
1436 |     glad_glGetQueryIndexediv = (PFNGLGETQUERYINDEXEDIVPROC)load("glGetQueryIndexediv");
1437 | }
1438 | static void load_GL_VERSION_4_1(GLADloadproc load) {
1439 |     if(!GLAD_GL_VERSION_4_1) return;
1440 |     glad_glReleaseShaderCompiler = (PFNGLRELEASESHADERCOMPILERPROC)load("glReleaseShaderCompiler");
1441 |     glad_glShaderBinary = (PFNGLSHADERBINARYPROC)load("glShaderBinary");
1442 |     glad_glGetShaderPrecisionFormat =
> (PFNGLGETSHADERPRECISIONFORMATPROC)load("glGetShaderPrecisionFormat");
1443 |     glad_glDepthRangef = (PFNGLDEPTHRANGEFPROC)load("glDepthRangef");
1444 |     glad_glClearDepthf = (PFNGLCLEARDEPTHFPROC)load("glClearDepthf");
1445 |     glad_glGetProgramBinary = (PFNGLGETPROGRAMBINARYPROC)load("glGetProgramBinary");
1446 |     glad_glProgramBinary = (PFNGLPROGRAMBINARYPROC)load("glProgramBinary");
1447 |     glad_glProgramParameteri = (PFNGLPROGRAMPARAMETERIPROC)load("glProgramParameteri");
1448 |     glad_glUseProgramStages = (PFNGLUSEPROGRAMSTAGESPROC)load("glUseProgramStages");
1449 |     glad_glActiveShaderProgram = (PFNGLACTIVESHADERPROGRAMPROC)load("glActiveShaderProgram");
1450 |     glad_glCreateShaderProgramv = (PFNGLCREATESHADERPROGRAMVPROC)load("glCreateShaderProgramv");
1451 |     glad_glBindProgramPipeline = (PFNGLBINDPROGRAMPIPELINEPROC)load("glBindProgramPipeline");
1452 |     glad_glDeleteProgramPipelines =
> (PFNGLDELETEPROGRAMPIPELINESPROC)load("glDeleteProgramPipelines");
1453 |     glad_glGenProgramPipelines = (PFNGLGENPROGRAMPIPELINESPROC)load("glGenProgramPipelines");
1454 |     glad_glIsProgramPipeline = (PFNGLISPROGRAMPIPELINEPROC)load("glIsProgramPipeline");

```

```
1455 |   glad_glGetProgramPipelineiv = (PFNGLGETPROGRAMPIPELINEIVPROC)load("glGetProgramPipelineiv");
1456 |   glad_glProgramParameteri = (PFNGLPROGRAMPARAMETERIPROC)load("glProgramParameteri");
1457 |   glad_glProgramUniform1i = (PFNGLPROGRAMUNIFORM1IPROC)load("glProgramUniform1i");
1458 |   glad_glProgramUniform1iv = (PFNGLPROGRAMUNIFORM1IVPROC)load("glProgramUniform1iv");
1459 |   glad_glProgramUniform1f = (PFNGLPROGRAMUNIFORM1FPROC)load("glProgramUniform1f");
1460 |   glad_glProgramUniform1fv = (PFNGLPROGRAMUNIFORM1FVPROC)load("glProgramUniform1fv");
1461 |   glad_glProgramUniform1d = (PFNGLPROGRAMUNIFORM1DPROC)load("glProgramUniform1d");
1462 |   glad_glProgramUniform1dv = (PFNGLPROGRAMUNIFORM1DVPROC)load("glProgramUniform1dv");
1463 |   glad_glProgramUniform1ui = (PFNGLPROGRAMUNIFORM1UIPROC)load("glProgramUniform1ui");
1464 |   glad_glProgramUniform1uiv = (PFNGLPROGRAMUNIFORM1UIVPROC)load("glProgramUniform1uiv");
1465 |   glad_glProgramUniform2i = (PFNGLPROGRAMUNIFORM2IPROC)load("glProgramUniform2i");
1466 |   glad_glProgramUniform2iv = (PFNGLPROGRAMUNIFORM2IVPROC)load("glProgramUniform2iv");
1467 |   glad_glProgramUniform2f = (PFNGLPROGRAMUNIFORM2FPROC)load("glProgramUniform2f");
1468 |   glad_glProgramUniform2fv = (PFNGLPROGRAMUNIFORM2FVPROC)load("glProgramUniform2fv");
1469 |   glad_glProgramUniform2d = (PFNGLPROGRAMUNIFORM2DPROC)load("glProgramUniform2d");
1470 |   glad_glProgramUniform2dv = (PFNGLPROGRAMUNIFORM2DVPROC)load("glProgramUniform2dv");
1471 |   glad_glProgramUniform2ui = (PFNGLPROGRAMUNIFORM2UIPROC)load("glProgramUniform2ui");
1472 |   glad_glProgramUniform2uiv = (PFNGLPROGRAMUNIFORM2UIVPROC)load("glProgramUniform2uiv");
1473 |   glad_glProgramUniform3i = (PFNGLPROGRAMUNIFORM3IPROC)load("glProgramUniform3i");
1474 |   glad_glProgramUniform3iv = (PFNGLPROGRAMUNIFORM3IVPROC)load("glProgramUniform3iv");
1475 |   glad_glProgramUniform3f = (PFNGLPROGRAMUNIFORM3FPROC)load("glProgramUniform3f");
1476 |   glad_glProgramUniform3fv = (PFNGLPROGRAMUNIFORM3FVPROC)load("glProgramUniform3fv");
1477 |   glad_glProgramUniform3d = (PFNGLPROGRAMUNIFORM3DPROC)load("glProgramUniform3d");
1478 |   glad_glProgramUniform3dv = (PFNGLPROGRAMUNIFORM3DVPROC)load("glProgramUniform3dv");
1479 |   glad_glProgramUniform3ui = (PFNGLPROGRAMUNIFORM3UIPROC)load("glProgramUniform3ui");
1480 |   glad_glProgramUniform3uiv = (PFNGLPROGRAMUNIFORM3UIVPROC)load("glProgramUniform3uiv");
1481 |   glad_glProgramUniform4i = (PFNGLPROGRAMUNIFORM4IPROC)load("glProgramUniform4i");
1482 |   glad_glProgramUniform4iv = (PFNGLPROGRAMUNIFORM4IVPROC)load("glProgramUniform4iv");
1483 |   glad_glProgramUniform4f = (PFNGLPROGRAMUNIFORM4FPROC)load("glProgramUniform4f");
1484 |   glad_glProgramUniform4fv = (PFNGLPROGRAMUNIFORM4FVPROC)load("glProgramUniform4fv");
1485 |   glad_glProgramUniform4d = (PFNGLPROGRAMUNIFORM4DPROC)load("glProgramUniform4d");
1486 |   glad_glProgramUniform4dv = (PFNGLPROGRAMUNIFORM4DVPROC)load("glProgramUniform4dv");
1487 |   glad_glProgramUniform4ui = (PFNGLPROGRAMUNIFORM4UIPROC)load("glProgramUniform4ui");
1488 |   glad_glProgramUniform4uiv = (PFNGLPROGRAMUNIFORM4UIVPROC)load("glProgramUniform4uiv");
1489 |   glad_glProgramUniformMatrix2fv =
> (PFNGLPROGRAMUNIFORMMATRIX2FVPROC)load("glProgramUniformMatrix2fv");
1490 |   glad_glProgramUniformMatrix3fv =
> (PFNGLPROGRAMUNIFORMMATRIX3FVPROC)load("glProgramUniformMatrix3fv");
1491 |   glad_glProgramUniformMatrix4fv =
> (PFNGLPROGRAMUNIFORMMATRIX4FVPROC)load("glProgramUniformMatrix4fv");
1492 |   glad_glProgramUniformMatrix2dv =
> (PFNGLPROGRAMUNIFORMMATRIX2DVPROC)load("glProgramUniformMatrix2dv");
1493 |   glad_glProgramUniformMatrix3dv =
> (PFNGLPROGRAMUNIFORMMATRIX3DVPROC)load("glProgramUniformMatrix3dv");
1494 |   glad_glProgramUniformMatrix4dv =
> (PFNGLPROGRAMUNIFORMMATRIX4DVPROC)load("glProgramUniformMatrix4dv");
1495 |   glad_glProgramUniformMatrix2x3fv =
> (PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC)load("glProgramUniformMatrix2x3fv");
1496 |   glad_glProgramUniformMatrix3x2fv =
> (PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC)load("glProgramUniformMatrix3x2fv");
1497 |   glad_glProgramUniformMatrix2x4fv =
> (PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC)load("glProgramUniformMatrix2x4fv");
1498 |   glad_glProgramUniformMatrix4x2fv =
> (PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC)load("glProgramUniformMatrix4x2fv");
1499 |   glad_glProgramUniformMatrix3x4fv =
> (PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC)load("glProgramUniformMatrix3x4fv");
1500 |   glad_glProgramUniformMatrix4x3fv =
> (PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC)load("glProgramUniformMatrix4x3fv");
1501 |   glad_glProgramUniformMatrix2x3dv =
> (PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC)load("glProgramUniformMatrix2x3dv");
1502 |   glad_glProgramUniformMatrix3x2dv =
> (PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC)load("glProgramUniformMatrix3x2dv");
1503 |   glad_glProgramUniformMatrix2x4dv =
> (PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC)load("glProgramUniformMatrix2x4dv");
1504 |   glad_glProgramUniformMatrix4x2dv =
> (PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC)load("glProgramUniformMatrix4x2dv");
1505 |   glad_glProgramUniformMatrix3x4dv =
> (PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC)load("glProgramUniformMatrix3x4dv");
1506 |   glad_glProgramUniformMatrix4x3dv =
> (PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC)load("glProgramUniformMatrix4x3dv");
1507 |   glad_glValidateProgramPipeline =
> (PFNGLVALIDATEPROGRAMPIPELINEPROC)load("glValidateProgramPipeline");
1508 |   glad_glGetProgramPipelineInfoLog =
> (PFNGLGETPROGRAMPIPELINEINFOLOGPROC)load("glGetProgramPipelineInfoLog");
1509 |   glad_glVertexAttribL1d = (PFNGLVERTEXATTRIBL1DPROC)load("glVertexAttribL1d");
1510 |   glad_glVertexAttribL2d = (PFNGLVERTEXATTRIBL2DPROC)load("glVertexAttribL2d");
1511 |   glad_glVertexAttribL3d = (PFNGLVERTEXATTRIBL3DPROC)load("glVertexAttribL3d");
1512 |   glad_glVertexAttribL4d = (PFNGLVERTEXATTRIBL4DPROC)load("glVertexAttribL4d");
1513 |   glad_glVertexAttribL1dv = (PFNGLVERTEXATTRIBL1DVPROC)load("glVertexAttribL1dv");
```

```

1514 |     glad_glVertexAttribL2dv = (PFNGLVERTEXATTRIBL2DVPROC)load("glVertexAttribL2dv");
1515 |     glad_glVertexAttribL3dv = (PFNGLVERTEXATTRIBL3DVPROC)load("glVertexAttribL3dv");
1516 |     glad_glVertexAttribL4dv = (PFNGLVERTEXATTRIBL4DVPROC)load("glVertexAttribL4dv");
1517 |     glad_glVertexAttribLPointer = (PFNGLVERTEXATTRIBLPOINTERPROC)load("glVertexAttribLPointer");
1518 |     glad_glGetVertexAttribLdv = (PFNGLGETVERTEXATTRIBLDVPROC)load("glGetVertexAttribLdv");
1519 |     glad_glViewportArrayv = (PFNGLVIEWPORTARRAYVPROC)load("glViewportArrayv");
1520 |     glad_glViewportIndexedf = (PFNGLVIEWPORTINDEXEDFPROC)load("glViewportIndexedf");
1521 |     glad_glViewportIndexedfv = (PFNGLVIEWPORTINDEXEDFVPROC)load("glViewportIndexedfv");
1522 |     glad_glScissorArrayv = (PFNGLSCISSORARRAYVPROC)load("glScissorArrayv");
1523 |     glad_glScissorIndexed = (PFNGLSCISSORINDEXEDPROC)load("glScissorIndexed");
1524 |     glad_glScissorIndexedv = (PFNGLSCISSORINDEXEDVPROC)load("glScissorIndexedv");
1525 |     glad_glDepthRangeArrayv = (PFNGLDEPTHRANGEARRAYVPROC)load("glDepthRangeArrayv");
1526 |     glad_glDepthRangeIndexed = (PFNGLDEPTHRANGEINDEXEDPROC)load("glDepthRangeIndexed");
1527 |     glad_glGetFloati_v = (PFNGLGETFLOATI_VPROC)load("glGetFloati_v");
1528 |     glad_glGetDoublei_v = (PFNGLGETDOUBLEI_VPROC)load("glGetDoublei_v");
1529 | }
1530 | static void load_GL_VERSION_4_2(GLADloadproc load) {
1531 |     if(!GLAD_GL_VERSION_4_2) return;
1532 |     glad_glDrawArraysInstancedBaseInstance =
1533 | > (PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC)load("glDrawArraysInstancedBaseInstance");
1534 |     glad_glDrawElementsInstancedBaseInstance =
1535 | > (PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC)load("glDrawElementsInstancedBaseInstance");
1536 |     glad_glDrawElementsInstancedBaseVertexBaseInstance =
1537 | > (PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC)load("glDrawElementsInstancedBaseVertexBaseInstance");
1538 |     glad_glGetInternalformativ = (PFNGLGETINTERNALFORMATIVPROC)load("glGetInternalformativ");
1539 |     glad_glGetActiveAtomicCounterBufferiv =
1540 | > (PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC)load("glGetActiveAtomicCounterBufferiv");
1541 |     glad_glBindImageTexture = (PFNGLBINDIMAGETEXTUREPROC)load("glBindImageTexture");
1542 |     glad_glMemoryBarrier = (PFNGLMEMORYBARRIERPROC)load("glMemoryBarrier");
1543 |     glad_glTexStorage1D = (PFNGLTEXSTORAGE1DPROC)load("glTexStorage1D");
1544 |     glad_glTexStorage2D = (PFNGLTEXSTORAGE2DPROC)load("glTexStorage2D");
1545 |     glad_glTexStorage3D = (PFNGLTEXSTORAGE3DPROC)load("glTexStorage3D");
1546 |     glad_glDrawTransformFeedbackInstanced =
1547 | > (PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC)load("glDrawTransformFeedbackInstanced");
1548 |     glad_glDrawTransformFeedbackStreamInstanced =
1549 | > (PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC)load("glDrawTransformFeedbackStreamInstanced");
1550 | }
1551 | static void load_GL_VERSION_4_3(GLADloadproc load) {
1552 |     if(!GLAD_GL_VERSION_4_3) return;
1553 |     glad_glClearBufferData = (PFNGLCLEARBUFFERDATAPROC)load("glClearBufferData");
1554 |     glad_glClearBufferSubData = (PFNGLCLEARBUFFERSUBDATAPROC)load("glClearBufferSubData");
1555 |     glad_glDispatchCompute = (PFNGLDISPATCHCOMPUTEPROC)load("glDispatchCompute");
1556 |     glad_glDispatchComputeIndirect =
1557 | > (PFNGLDISPATCHCOMPUTEINDIRECTPROC)load("glDispatchComputeIndirect");
1558 |     glad_glCopyImageSubData = (PFNGLCOPYIMAGESUBDATAPROC)load("glCopyImageSubData");
1559 |     glad_glFramebufferParameteri = (PFNGLFRAMEBUFFERPARAMETERIPROC)load("glFramebufferParameteri");
1560 |     glad_glGetFramebufferParameteriv =
1561 | > (PFNGLGETFRAMEBUFFERPARAMETERIVPROC)load("glGetFramebufferParameteriv");
1562 |     glad_glGetInternalformati64v = (PFNGLGETINTERNALFORMATI64VPROC)load("glGetInternalformati64v");
1563 |     glad_glInvalidateTexSubImage = (PFNGLINVALIDATETEXSUBIMAGEPROC)load("glInvalidateTexSubImage");
1564 |     glad_glInvalidateTexImage = (PFNGLINVALIDATETEXIMAGEPROC)load("glInvalidateTexImage");
1565 |     glad_glInvalidateBufferSubData =
1566 | > (PFNGLINVALIDATEBUFFERSUBDATAPROC)load("glInvalidateBufferSubData");
1567 |     glad_glInvalidateBufferData = (PFNGLINVALIDATEBUFFERDATAPROC)load("glInvalidateBufferData");
1568 |     glad_glInvalidateFramebuffer = (PFNGLINVALIDATEFRAMEBUFFERPROC)load("glInvalidateFramebuffer");
1569 |     glad_glInvalidateSubFramebuffer =
1570 | > (PFNGLINVALIDATESUBFRAMEBUFFERPROC)load("glInvalidateSubFramebuffer");
1571 |     glad_glMultiDrawArraysIndirect =
1572 | > (PFNGLMULTIDRAWARRAYSINDIRECTPROC)load("glMultiDrawArraysIndirect");
1573 |     glad_glMultiDrawElementsIndirect =
1574 | > (PFNGLMULTIDRAWELEMENTSINDIRECTPROC)load("glMultiDrawElementsIndirect");
1575 |     glad_glGetProgramInterfaceiv = (PFNGLGETPROGRAMINTERFACEIVPROC)load("glGetProgramInterfaceiv");
1576 |     glad_glGetProgramResourceIndex =
1577 | > (PFNGLGETPROGRAMRESOURCEINDEXPROC)load("glGetProgramResourceIndex");
1578 |     glad_glGetProgramResourceName =
1579 | > (PFNGLGETPROGRAMRESOURCENAMEPROC)load("glGetProgramResourceName");
1580 |     glad_glGetProgramResourceiv = (PFNGLGETPROGRAMRESOURCEIVPROC)load("glGetProgramResourceiv");
1581 |     glad_glGetProgramResourceLocation =
1582 | > (PFNGLGETPROGRAMRESOURCELOCATIONPROC)load("glGetProgramResourceLocation");
1583 |     glad_glGetProgramResourceLocationIndex =
1584 | > (PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC)load("glGetProgramResourceLocationIndex");
1585 |     glad_glShaderStorageBlockBinding =
1586 | > (PFNGLSHADERSTORAGEBLOCKBINDINGPROC)load("glShaderStorageBlockBinding");
1587 |     glad_glTexBufferRange = (PFNGLTEXBUFFERRANGEPROC)load("glTexBufferRange");
1588 |     glad_glTexStorage2DMultisample =
1589 | > (PFNGLTEXSTORAGE2DMULTISAMPLEPROC)load("glTexStorage2DMultisample");
1590 |     glad_glTexStorage3DMultisample =
1591 | > (PFNGLTEXSTORAGE3DMULTISAMPLEPROC)load("glTexStorage3DMultisample");
1592 |     glad_glTextureView = (PFNGLTEXTUREVIEWPROC)load("glTextureView");

```

```

1574 |     glad_glBindVertexBuffer = (PFNGLBINDVERTEXBUFFERPROC)load("glBindVertexBuffer");
1575 |     glad_glVertexAttribFormat = (PFNGLVERTEXATTRIBFORMATPROC)load("glVertexAttribFormat");
1576 |     glad_glVertexAttribIFormat = (PFNGLVERTEXATTRIBIFORMATPROC)load("glVertexAttribIFormat");
1577 |     glad_glVertexAttribLFormat = (PFNGLVERTEXATTRIBLFORMATPROC)load("glVertexAttribLFormat");
1578 |     glad_glVertexAttribBinding = (PFNGLVERTEXATTRIBBINDINGPROC)load("glVertexAttribBinding");
1579 |     glad_glVertexBindingDivisor = (PFNGLVERTEXBINDINGDIVISORPROC)load("glVertexBindingDivisor");
1580 |     glad_glDebugMessageControl = (PFNGLDEBUGMESSAGECONTROLPROC)load("glDebugMessageControl");
1581 |     glad_glDebugMessageInsert = (PFNGLDEBUGMESSAGEINSERTPROC)load("glDebugMessageInsert");
1582 |     glad_glDebugMessageCallback = (PFNGLDEBUGMESSAGECALLBACKPROC)load("glDebugMessageCallback");
1583 |     glad_glGetDebugMessageLog = (PFNGLGETDEBUGMESSAGELOGPROC)load("glGetDebugMessageLog");
1584 |     glad_glPushDebugGroup = (PFNGLPUSHDEBUGGROUPPROC)load("glPushDebugGroup");
1585 |     glad_glPopDebugGroup = (PFNGLPOPDEBUGGROUPPROC)load("glPopDebugGroup");
1586 |     glad_glObjectLabel = (PFNGLOBJECTLABELPROC)load("glObjectLabel");
1587 |     glad_glGetObjectLabel = (PFNGLGETOBJECTLABELPROC)load("glGetObjectLabel");
1588 |     glad_glObjectPtrLabel = (PFNGLOBJECTPTRLABELPROC)load("glObjectPtrLabel");
1589 |     glad_glGetObjectPtrLabel = (PFNGLGETOBJECTPTRLABELPROC)load("glGetObjectPtrLabel");
1590 |     glad_glGetPointerv = (PFNGLGETPOINTERVPROC)load("glGetPointerv");
1591 | }
1592 | static void load_GL_VERSION_4_4(GLADloadproc load) {
1593 |     if(!GLAD_GL_VERSION_4_4) return;
1594 |     glad_glBufferStorage = (PFNGLBUFFERSTORAGEPROC)load("glBufferStorage");
1595 |     glad_glClearTexImage = (PFNGLCLEARTEXIMAGEPROC)load("glClearTexImage");
1596 |     glad_glClearTexSubImage = (PFNGLCLEARTEXSUBIMAGEPROC)load("glClearTexSubImage");
1597 |     glad_glBindBuffersBase = (PFNGLBINDBUFFERSBASEPROC)load("glBindBuffersBase");
1598 |     glad_glBindBuffersRange = (PFNGLBINDBUFFERSRANGEPROC)load("glBindBuffersRange");
1599 |     glad_glBindTextures = (PFNGLBINDTEXTURESPROC)load("glBindTextures");
1600 |     glad_glBindSamplers = (PFNGLBINDSAMPLERSPROC)load("glBindSamplers");
1601 |     glad_glBindImageTextures = (PFNGLBINDIMAGETEXTURESPROC)load("glBindImageTextures");
1602 |     glad_glBindVertexBuffers = (PFNGLBINDVERTEXBUFFERSPROC)load("glBindVertexBuffers");
1603 | }
1604 | static void load_GL_VERSION_4_5(GLADloadproc load) {
1605 |     if(!GLAD_GL_VERSION_4_5) return;
1606 |     glad_glClipControl = (PFNGLCLIPCONTROLPROC)load("glClipControl");
1607 |     glad_glCreateTransformFeedbacks =
1608 | > (PFNGLCREATETRANSFORMFEEDBACKSPROC)load("glCreateTransformFeedbacks");
1609 |     glad_glTransformFeedbackBufferBase =
1610 | > (PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC)load("glTransformFeedbackBufferBase");
1611 |     glad_glTransformFeedbackBufferRange =
1612 | > (PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC)load("glTransformFeedbackBufferRange");
1613 |     glad_glGetTransformFeedbackiv =
1614 | > (PFNGLGETTRANSFORMFEEDBACKIVPROC)load("glGetTransformFeedbackiv");
1615 |     glad_glGetTransformFeedbacki_v =
1616 | > (PFNGLGETTRANSFORMFEEDBACKI_VPROC)load("glGetTransformFeedbacki_v");
1617 |     glad_glGetTransformFeedbacki64_v =
1618 | > (PFNGLGETTRANSFORMFEEDBACKI64_VPROC)load("glGetTransformFeedbacki64_v");
1619 |     glad_glCreateBuffers = (PFNGLCREATEBUFFERSPROC)load("glCreateBuffers");
1620 |     glad_glNamedBufferStorage = (PFNGLNAMEDBUFFERSTORAGEPROC)load("glNamedBufferStorage");
1621 |     glad_glNamedBufferData = (PFNGLNAMEDBUFFERDATAPROC)load("glNamedBufferData");
1622 |     glad_glNamedBufferSubData = (PFNGLNAMEDBUFFERSUBDATAPROC)load("glNamedBufferSubData");
1623 |     glad_glCopyNamedBufferSubData =
1624 | > (PFNGLCOPYNAMEDBUFFERSUBDATAPROC)load("glCopyNamedBufferSubData");
1625 |     glad_glClearNamedBufferData = (PFNGLCLEARNAMEDBUFFERDATAPROC)load("glClearNamedBufferData");
1626 |     glad_glClearNamedBufferSubData =
1627 | > (PFNGLCLEARNAMEDBUFFERSUBDATAPROC)load("glClearNamedBufferSubData");
1628 |     glad_glMapNamedBuffer = (PFNGLMAPNAMEDBUFFERPROC)load("glMapNamedBuffer");
1629 |     glad_glMapNamedBufferRange = (PFNGLMAPNAMEDBUFFERRANGEPROC)load("glMapNamedBufferRange");
1630 |     glad_glUnmapNamedBuffer = (PFNGLUNMAPNAMEDBUFFERPROC)load("glUnmapNamedBuffer");
1631 |     glad_glFlushMappedNamedBufferRange =
1632 | > (PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC)load("glFlushMappedNamedBufferRange");
1633 |     glad_glGetNamedBufferParameteriv =
1634 | > (PFNGLGETNAMEDBUFFERPARAMETERIVPROC)load("glGetNamedBufferParameteriv");
1635 |     glad_glGetNamedBufferParameteri64v =
1636 | > (PFNGLGETNAMEDBUFFERPARAMETERI64VPROC)load("glGetNamedBufferParameteri64v");
1637 |     glad_glGetNamedBufferPointerv =
1638 | > (PFNGLGETNAMEDBUFFERPOINTERVPROC)load("glGetNamedBufferPointerv");
1639 |     glad_glGetNamedBufferSubData = (PFNGLGETNAMEDBUFFERSUBDATAPROC)load("glGetNamedBufferSubData");
1640 |     glad_glCreateFramebuffers = (PFNGLCREATEFRAMEBUFFERSPROC)load("glCreateFramebuffers");
1641 |     glad_glNamedFramebufferRenderbuffer =
1642 | > (PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC)load("glNamedFramebufferRenderbuffer");
1643 |     glad_glNamedFramebufferParameteri =
1644 | > (PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC)load("glNamedFramebufferParameteri");
1645 |     glad_glNamedFramebufferTexture =
1646 | > (PFNGLNAMEDFRAMEBUFFERTEXTUREPROC)load("glNamedFramebufferTexture");
1647 |     glad_glNamedFramebufferTextureLayer =
1648 | > (PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC)load("glNamedFramebufferTextureLayer");
1649 |     glad_glNamedFramebufferDrawBuffer =
1650 | > (PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC)load("glNamedFramebufferDrawBuffer");
1651 |     glad_glNamedFramebufferDrawBuffers =
1652 | > (PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC)load("glNamedFramebufferDrawBuffers");

```



```

1635 |     glad_glNamedFramebufferReadBuffer =
> (PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC)load("glNamedFramebufferReadBuffer");
1636 |     glad_glInvalidateNamedFramebufferData =
> (PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC)load("glInvalidateNamedFramebufferData");
1637 |     glad_glInvalidateNamedFramebufferSubData =
> (PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC)load("glInvalidateNamedFramebufferSubData");
1638 |     glad_glClearNamedFramebufferiv =
> (PFNGLCLEARNAMEDFRAMEBUFFERIVPROC)load("glClearNamedFramebufferiv");
1639 |     glad_glClearNamedFramebufferuiiv =
> (PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC)load("glClearNamedFramebufferuiiv");
1640 |     glad_glClearNamedFramebufferfv =
> (PFNGLCLEARNAMEDFRAMEBUFFERFVPROC)load("glClearNamedFramebufferfv");
1641 |     glad_glClearNamedFramebufferfi =
> (PFNGLCLEARNAMEDFRAMEBUFFERFIPROC)load("glClearNamedFramebufferfi");
1642 |     glad_glBlitNamedFramebuffer = (PFNGLBLITNAMEDFRAMEBUFFERPROC)load("glBlitNamedFramebuffer");
1643 |     glad_glCheckNamedFramebufferStatus =
> (PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC)load("glCheckNamedFramebufferStatus");
1644 |     glad_glGetNamedFramebufferParameteriv =
> (PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC)load("glGetNamedFramebufferParameteriv");
1645 |     glad_glGetNamedFramebufferAttachmentParameteriv = (PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIV
> PROC)load("glGetNamedFramebufferAttachmentParameteriv");
1646 |     glad_glCreateRenderbuffers = (PFNGLCREATERENDERBUFFERSPROC)load("glCreateRenderbuffers");
1647 |     glad_glNamedRenderbufferStorage =
> (PFNGLNAMEDRENDERBUFFERSTORAGEPROC)load("glNamedRenderbufferStorage");
1648 |     glad_glNamedRenderbufferStorageMultisample =
> (PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC)load("glNamedRenderbufferStorageMultisample");
1649 |     glad_glGetNamedRenderbufferParameteriv =
> (PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC)load("glGetNamedRenderbufferParameteriv");
1650 |     glad_glCreateTextures = (PFNGLCREATETEXTURESPROC)load("glCreateTextures");
1651 |     glad_glTextureBuffer = (PFNGLTEXTUREBUFFERPROC)load("glTextureBuffer");
1652 |     glad_glTextureBufferRange = (PFNGLTEXTUREBUFFERRANGEPROC)load("glTextureBufferRange");
1653 |     glad_glTextureStorage1D = (PFNGLTEXTURESTORAGE1DPROC)load("glTextureStorage1D");
1654 |     glad_glTextureStorage2D = (PFNGLTEXTURESTORAGE2DPROC)load("glTextureStorage2D");
1655 |     glad_glTextureStorage3D = (PFNGLTEXTURESTORAGE3DPROC)load("glTextureStorage3D");
1656 |     glad_glTextureStorage2DMultisample =
> (PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC)load("glTextureStorage2DMultisample");
1657 |     glad_glTextureStorage3DMultisample =
> (PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC)load("glTextureStorage3DMultisample");
1658 |     glad_glTextureSubImage1D = (PFNGLTEXTURESUBIMAGE1DPROC)load("glTextureSubImage1D");
1659 |     glad_glTextureSubImage2D = (PFNGLTEXTURESUBIMAGE2DPROC)load("glTextureSubImage2D");
1660 |     glad_glTextureSubImage3D = (PFNGLTEXTURESUBIMAGE3DPROC)load("glTextureSubImage3D");
1661 |     glad_glCompressedTextureSubImage1D =
> (PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC)load("glCompressedTextureSubImage1D");
1662 |     glad_glCompressedTextureSubImage2D =
> (PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC)load("glCompressedTextureSubImage2D");
1663 |     glad_glCompressedTextureSubImage3D =
> (PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC)load("glCompressedTextureSubImage3D");
1664 |     glad_glCopyTextureSubImage1D = (PFNGLCOPYTEXTURESUBIMAGE1DPROC)load("glCopyTextureSubImage1D");
1665 |     glad_glCopyTextureSubImage2D = (PFNGLCOPYTEXTURESUBIMAGE2DPROC)load("glCopyTextureSubImage2D");
1666 |     glad_glCopyTextureSubImage3D = (PFNGLCOPYTEXTURESUBIMAGE3DPROC)load("glCopyTextureSubImage3D");
1667 |     glad_glTextureParameterf = (PFNGLTEXTUREPARAMETERFPROC)load("glTextureParameterf");
1668 |     glad_glTextureParameterfv = (PFNGLTEXTUREPARAMETERFVPROC)load("glTextureParameterfv");
1669 |     glad_glTextureParameteri = (PFNGLTEXTUREPARAMETERIPROC)load("glTextureParameteri");
1670 |     glad_glTextureParameteriiv = (PFNGLTEXTUREPARAMETERIIVPROC)load("glTextureParameteriiv");
1671 |     glad_glTextureParameteriuiiv = (PFNGLTEXTUREPARAMETERIUIVPROC)load("glTextureParameteriuiiv");
1672 |     glad_glTextureParameteriv = (PFNGLTEXTUREPARAMETERIVPROC)load("glTextureParameteriv");
1673 |     glad_glGenerateTextureMipmap = (PFNGLGENERATETEXTUREMIPMAPPROC)load("glGenerateTextureMipmap");
1674 |     glad_glBindTextureUnit = (PFNGLBINDTEXTUREUNITPROC)load("glBindTextureUnit");
1675 |     glad_glGetTextureImage = (PFNGLGETTEXTUREIMAGEPROC)load("glGetTextureImage");
1676 |     glad_glGetCompressedTextureImage =
> (PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC)load("glGetCompressedTextureImage");
1677 |     glad_glGetTextureLevelParameterfv =
> (PFNGLGETTEXTURELEVELPARAMETERFVPROC)load("glGetTextureLevelParameterfv");
1678 |     glad_glGetTextureLevelParameteriv =
> (PFNGLGETTEXTURELEVELPARAMETERIVPROC)load("glGetTextureLevelParameteriv");
1679 |     glad_glGetTextureParameterfv = (PFNGLGETTEXTUREPARAMETERFVPROC)load("glGetTextureParameterfv");
1680 |     glad_glGetTextureParameteriiv =
> (PFNGLGETTEXTUREPARAMETERIIVPROC)load("glGetTextureParameteriiv");
1681 |     glad_glGetTextureParameteriuiiv =
> (PFNGLGETTEXTUREPARAMETERIUIVPROC)load("glGetTextureParameteriuiiv");
1682 |     glad_glGetTextureParameteriv = (PFNGLGETTEXTUREPARAMETERIVPROC)load("glGetTextureParameteriv");
1683 |     glad_glCreateVertexArrays = (PFNGLCREATEVERTEXARRAYSPROC)load("glCreateVertexArrays");
1684 |     glad_glDisableVertexArrayAttrib =
> (PFNGLDISABLEVERTEXARRAYATTRIBPROC)load("glDisableVertexArrayAttrib");
1685 |     glad_glEnableVertexArrayAttrib =
> (PFNGLENABLEVERTEXARRAYATTRIBPROC)load("glEnableVertexArrayAttrib");
1686 |     glad_glVertexArrayElementBuffer =
> (PFNGLVERTEXARRAYELEMENTBUFFERPROC)load("glVertexArrayElementBuffer");
1687 |     glad_glVertexArrayVertexBuffer =

```

```

    > (PFNGLVERTEXARRAYVERTEXBUFFERPROC)load("glVertexArrayVertexBuffer");
1688 |     glad_glVertexArrayVertexBuffers =
    > (PFNGLVERTEXARRAYVERTEXBUFFERSPROC)load("glVertexArrayVertexBuffers");
1689 |     glad_glVertexArrayAttribBinding =
    > (PFNGLVERTEXARRAYATTRIBBINDINGPROC)load("glVertexArrayAttribBinding");
1690 |     glad_glVertexArrayAttribFormat =
    > (PFNGLVERTEXARRAYATTRIBFORMATPROC)load("glVertexArrayAttribFormat");
1691 |     glad_glVertexArrayAttribIFormat =
    > (PFNGLVERTEXARRAYATTRIBIFORMATPROC)load("glVertexArrayAttribIFormat");
1692 |     glad_glVertexArrayAttribLFormat =
    > (PFNGLVERTEXARRAYATTRIBLFORMATPROC)load("glVertexArrayAttribLFormat");
1693 |     glad_glVertexArrayBindingDivisor =
    > (PFNGLVERTEXARRAYBINDINGDIVISORPROC)load("glVertexArrayBindingDivisor");
1694 |     glad_glGetVertexArrayiv = (PFNGLGETVERTEXARRAYIVPROC)load("glGetVertexArrayiv");
1695 |     glad_glGetVertexArrayIndexediv =
    > (PFNGLGETVERTEXARRAYINDEXEDIVPROC)load("glGetVertexArrayIndexediv");
1696 |     glad_glGetVertexArrayIndexed64iv =
    > (PFNGLGETVERTEXARRAYINDEXED64IVPROC)load("glGetVertexArrayIndexed64iv");
1697 |     glad_glCreateSamplers = (PFNGLCREATESAMPLERSPROC)load("glCreateSamplers");
1698 |     glad_glCreateProgramPipelines =
    > (PFNGLCREATEPROGRAMPIPELINESPROC)load("glCreateProgramPipelines");
1699 |     glad_glCreateQueries = (PFNGLCREATEQUERIESPROC)load("glCreateQueries");
1700 |     glad_glGetQueryBufferObjecti64v =
    > (PFNGLGETQUERYBUFFEROBJECTI64VPROC)load("glGetQueryBufferObjecti64v");
1701 |     glad_glGetQueryBufferObjectiv =
    > (PFNGLGETQUERYBUFFEROBJECTIVPROC)load("glGetQueryBufferObjectiv");
1702 |     glad_glGetQueryBufferObjectui64v =
    > (PFNGLGETQUERYBUFFEROBJECTUI64VPROC)load("glGetQueryBufferObjectui64v");
1703 |     glad_glGetQueryBufferObjectuiv =
    > (PFNGLGETQUERYBUFFEROBJECTUIVPROC)load("glGetQueryBufferObjectuiv");
1704 |     glad_glMemoryBarrierByRegion = (PFNGLMEMORYBARRIERBYREGIONPROC)load("glMemoryBarrierByRegion");
1705 |     glad_glGetTextureSubImage = (PFNGLGETTEXTURESUBIMAGEPROC)load("glGetTextureSubImage");
1706 |     glad_glGetCompressedTextureSubImage =
    > (PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC)load("glGetCompressedTextureSubImage");
1707 |     glad_glGetGraphicsResetStatus =
    > (PFNGLGETGRAPHICSRESETSTATUSPROC)load("glGetGraphicsResetStatus");
1708 |     glad_glGetnCompressedTexImage =
    > (PFNGLGETNCOMPRESSEDTEXIMAGEPROC)load("glGetnCompressedTexImage");
1709 |     glad_glGetnTexImage = (PFNGLGETNTEXIMAGEPROC)load("glGetnTexImage");
1710 |     glad_glGetnUniformdv = (PFNGLGETNUNIFORMDVPROC)load("glGetnUniformdv");
1711 |     glad_glGetnUniformfv = (PFNGLGETNUNIFORMFVPROC)load("glGetnUniformfv");
1712 |     glad_glGetnUniformiv = (PFNGLGETNUNIFORMIVPROC)load("glGetnUniformiv");
1713 |     glad_glGetnUniformuiv = (PFNGLGETNUNIFORMUIVPROC)load("glGetnUniformuiv");
1714 |     glad_glReadnPixels = (PFNGLREADNPIXELSPROC)load("glReadnPixels");
1715 |     glad_glGetnMapdv = (PFNGLGETNMAPDVPROC)load("glGetnMapdv");
1716 |     glad_glGetnMapfv = (PFNGLGETNMAPFVPROC)load("glGetnMapfv");
1717 |     glad_glGetnMapiv = (PFNGLGETNMAPIVPROC)load("glGetnMapiv");
1718 |     glad_glGetnPixelMapfv = (PFNGLGETNPIXELMAPFVPROC)load("glGetnPixelMapfv");
1719 |     glad_glGetnPixelMapuiv = (PFNGLGETNPIXELMAPUIVPROC)load("glGetnPixelMapuiv");
1720 |     glad_glGetnPixelMapusv = (PFNGLGETNPIXELMAPUSVPROC)load("glGetnPixelMapusv");
1721 |     glad_glGetnPolygonStipple = (PFNGLGETNPOLYGONSTIPPLEPROC)load("glGetnPolygonStipple");
1722 |     glad_glGetnColorTable = (PFNGLGETNCOLORTABLEPROC)load("glGetnColorTable");
1723 |     glad_glGetnConvolutionFilter = (PFNGLGETNCONVOLUTIONFILTERPROC)load("glGetnConvolutionFilter");
1724 |     glad_glGetnSeparableFilter = (PFNGLGETNSEPARABLEFILTERPROC)load("glGetnSeparableFilter");
1725 |     glad_glGetnHistogram = (PFNGLGETNHISTOGRAMPROC)load("glGetnHistogram");
1726 |     glad_glGetnMinmax = (PFNGLGETNMINMAXPROC)load("glGetnMinmax");
1727 |     glad_glTextureBarrier = (PFNGLTEXTUREBARRIERPROC)load("glTextureBarrier");
1728 | }
1729 | static void load_GL_VERSION_4_6(GLADloadproc load) {
1730 |     if(!GLAD_GL_VERSION_4_6) return;
1731 |     glad_glSpecializeShader = (PFNGLSPECIALIZESHADERPROC)load("glSpecializeShader");
1732 |     glad_glMultiDrawArraysIndirectCount =
    > (PFNGLMULTIDRAWARRAYSINDIRECTCOUNTPROC)load("glMultiDrawArraysIndirectCount");
1733 |     glad_glMultiDrawElementsIndirectCount =
    > (PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTPROC)load("glMultiDrawElementsIndirectCount");
1734 |     glad_glPolygonOffsetClamp = (PFNGLPOLYGONOFFSETCLAMPPROC)load("glPolygonOffsetClamp");
1735 | }
1736 | static int find_extensionsGL(void) {
1737 |     if (!get_exts()) return 0;
1738 |     (void)&has_ext;
1739 |     free_exts();
1740 |     return 1;
1741 | }
1742 |
1743 | static void find_coreGL(void) {
1744 |
1745 |     /* Thank you @elmindreda
1746 |      * https://github.com/elmindreda/greg/blob/master/templates/greg.c.in#L176
1747 |      * https://github.com/glfw/glfw/blob/master/src/context.c#L36

```

```

1748 |     */
1749 |     int i, major, minor;
1750 |
1751 |     const char* version;
1752 |     const char* prefixes[] = {
1753 |         "OpenGL ES-CM ",
1754 |         "OpenGL ES-CL ",
1755 |         "OpenGL ES ",
1756 |         NULL
1757 |     };
1758 |
1759 |     version = (const char*) glGetString(GL_VERSION);
1760 |     if (!version) return;
1761 |
1762 |     for (i = 0; prefixes[i]; i++) {
1763 |         const size_t length = strlen(prefixes[i]);
1764 |         if (strncmp(version, prefixes[i], length) == 0) {
1765 |             version += length;
1766 |             break;
1767 |         }
1768 |     }
1769 |
1770 |     /* PR #18 */
1771 |     #ifdef _MSC_VER
1772 |         sscanf_s(version, "%d.%d", &major, &minor);
1773 |     #else
1774 |         sscanf(version, "%d.%d", &major, &minor);
1775 |     #endif
1776 |
1777 |     GLVersion.major = major; GLVersion.minor = minor;
1778 |     max_loaded_major = major; max_loaded_minor = minor;
1779 |     GLAD_GL_VERSION_1_0 = (major == 1 && minor >= 0) || major > 1;
1780 |     GLAD_GL_VERSION_1_1 = (major == 1 && minor >= 1) || major > 1;
1781 |     GLAD_GL_VERSION_1_2 = (major == 1 && minor >= 2) || major > 1;
1782 |     GLAD_GL_VERSION_1_3 = (major == 1 && minor >= 3) || major > 1;
1783 |     GLAD_GL_VERSION_1_4 = (major == 1 && minor >= 4) || major > 1;
1784 |     GLAD_GL_VERSION_1_5 = (major == 1 && minor >= 5) || major > 1;
1785 |     GLAD_GL_VERSION_2_0 = (major == 2 && minor >= 0) || major > 2;
1786 |     GLAD_GL_VERSION_2_1 = (major == 2 && minor >= 1) || major > 2;
1787 |     GLAD_GL_VERSION_3_0 = (major == 3 && minor >= 0) || major > 3;
1788 |     GLAD_GL_VERSION_3_1 = (major == 3 && minor >= 1) || major > 3;
1789 |     GLAD_GL_VERSION_3_2 = (major == 3 && minor >= 2) || major > 3;
1790 |     GLAD_GL_VERSION_3_3 = (major == 3 && minor >= 3) || major > 3;
1791 |     GLAD_GL_VERSION_4_0 = (major == 4 && minor >= 0) || major > 4;
1792 |     GLAD_GL_VERSION_4_1 = (major == 4 && minor >= 1) || major > 4;
1793 |     GLAD_GL_VERSION_4_2 = (major == 4 && minor >= 2) || major > 4;
1794 |     GLAD_GL_VERSION_4_3 = (major == 4 && minor >= 3) || major > 4;
1795 |     GLAD_GL_VERSION_4_4 = (major == 4 && minor >= 4) || major > 4;
1796 |     GLAD_GL_VERSION_4_5 = (major == 4 && minor >= 5) || major > 4;
1797 |     GLAD_GL_VERSION_4_6 = (major == 4 && minor >= 6) || major > 4;
1798 |     if (GLVersion.major > 4 || (GLVersion.major >= 4 && GLVersion.minor >= 6)) {
1799 |         max_loaded_major = 4;
1800 |         max_loaded_minor = 6;
1801 |     }
1802 | }
1803 |
1804 | int gladLoadGLLoader(GLADloadproc load) {
1805 |     GLVersion.major = 0; GLVersion.minor = 0;
1806 |     glGetString = (PFNGLGETSTRINGPROC)load("glGetString");
1807 |     if(glGetString == NULL) return 0;
1808 |     if(glGetString(GL_VERSION) == NULL) return 0;
1809 |     find_coreGL();
1810 |     load_GL_VERSION_1_0(load);
1811 |     load_GL_VERSION_1_1(load);
1812 |     load_GL_VERSION_1_2(load);
1813 |     load_GL_VERSION_1_3(load);
1814 |     load_GL_VERSION_1_4(load);
1815 |     load_GL_VERSION_1_5(load);
1816 |     load_GL_VERSION_2_0(load);
1817 |     load_GL_VERSION_2_1(load);
1818 |     load_GL_VERSION_3_0(load);
1819 |     load_GL_VERSION_3_1(load);
1820 |     load_GL_VERSION_3_2(load);
1821 |     load_GL_VERSION_3_3(load);
1822 |     load_GL_VERSION_4_0(load);
1823 |     load_GL_VERSION_4_1(load);
1824 |     load_GL_VERSION_4_2(load);
1825 |     load_GL_VERSION_4_3(load);
1826 |     load_GL_VERSION_4_4(load);

```



```
1827 |     load_GL_VERSION_4_5(load);  
1828 |     load_GL_VERSION_4_6(load);  
1829 |  
1830 |     if (!find_extensionsGL()) return 0;  
1831 |     return GLVersion.major != 0 || GLVersion.minor != 0;  
1832 | }
```

Source Reference:

third_party/miniaudio/miniaudio.h

```

1 | /*
2 | Audio playback and capture library. Choice of public domain or MIT-0. See license statements at the
  > end of this file.
3 | miniaudio - v0.11.25 - 2026-03-04
4 |
5 | David Reid - mackron@gmail.com
6 |
7 | Website:      https://miniaud.io
8 | Documentation: https://miniaud.io/docs
9 | GitHub:      https://github.com/mackron/miniaudio
10 | */
11 |
12 | /*
13 | 1. Introduction
14 | =====
15 | To use miniaudio, just include "miniaudio.h" like any other header and add "miniaudio.c" to your
16 | source tree. If you don't want to add it to your source tree you can compile and link to it like
17 | any other library. Note that ABI compatibility is not guaranteed between versions, even with bug
18 | fix releases, so take care if compiling as a shared object.
19 |
20 | miniaudio includes both low level and high level APIs. The low level API is good for those who want
21 | to do all of their mixing themselves and only require a light weight interface to the underlying
22 | audio device. The high level API is good for those who have complex mixing and effect requirements.
23 |
24 | In miniaudio, objects are transparent structures. Unlike many other libraries, there are no handles
25 | to opaque objects which means you need to allocate memory for objects yourself. In the examples
26 | presented in this documentation you will often see objects declared on the stack. You need to be
27 | careful when translating these examples to your own code so that you don't accidentally declare
28 | your objects on the stack and then cause them to become invalid once the function returns. In
29 | addition, you must ensure the memory address of your objects remain the same throughout their
30 | lifetime. You therefore cannot be making copies of your objects.
31 |
32 | A config/init pattern is used throughout the entire library. The idea is that you set up a config
33 | object and pass that into the initialization routine. The advantage to this system is that the
34 | config object can be initialized with logical defaults and new properties added to it without
35 | breaking the API. The config object can be allocated on the stack and does not need to be
36 | maintained after initialization of the corresponding object.
37 |
38 |
39 | 1.1. Low Level API
40 | -----
41 | The low level API gives you access to the raw audio data of an audio device. It supports playback,
42 | capture, full-duplex and loopback (WASAPI only). You can enumerate over devices to determine which
43 | physical device(s) you want to connect to.
44 |
45 | The low level API uses the concept of a "device" as the abstraction for physical devices. The idea
46 | is that you choose a physical device to emit or capture audio from, and then move data to/from the
47 | device when miniaudio tells you to. Data is delivered to and from devices asynchronously via a
48 | callback which you specify when initializing the device.
49 |
50 | When initializing the device you first need to configure it. The device configuration allows you to
51 | specify things like the format of the data delivered via the callback, the size of the internal
52 | buffer and the ID of the device you want to emit or capture audio from.
53 |
54 | Once you have the device configuration set up you can initialize the device. When initializing a
55 | device you need to allocate memory for the device object beforehand. This gives the application
56 | complete control over how the memory is allocated. In the example below we initialize a playback
57 | device on the stack, but you could allocate it on the heap if that suits your situation better.
58 |
59 |     ``c
60 |     void data_callback(ma_device* pDevice, void* pOutput, const void* pInput, ma_uint32 frameCount)
61 |     {
62 |         // In playback mode copy data to pOutput. In capture mode read data from pInput. In
  > full-duplex mode, both
63 |         // pOutput and pInput will be valid and you can move data from pInput into pOutput. Never
  > process more than
64 |         // frameCount frames.
65 |     }
66 |
67 |     int main()
68 |     {
69 |         ma_device_config config = ma_device_config_init(ma_device_type_playback);
70 |         config.playback.format = ma_format_f32; // Set to ma_format_unknown to use the device's

```

```

> native format.
71 |         config.playback.channels = 2;                // Set to 0 to use the device's native channel
> count.
72 |         config.sampleRate        = 48000;            // Set to 0 to use the device's native sample
> rate.
73 |         config.dataCallback       = data_callback;    // This function will be called when miniaudio
> needs more data.
74 |         config.pUserData          = pMyCustomData;    // Can be accessed from the device object
> (device.pUserData).
75 |
76 |         ma_device device;
77 |         if (ma_device_init(NULL, &config, &device) != MA_SUCCESS) {
78 |             return -1; // Failed to initialize the device.
79 |         }
80 |
81 |         ma_device_start(&device);    // The device is sleeping by default so you'll need to start
> it manually.
82 |
83 |         // Do something here. Probably your program's main loop.
84 |
85 |         ma_device_uninit(&device);
86 |         return 0;
87 |     }
88 |     ...
89 |
90 | In the example above, `data_callback()` is where audio data is written and read from the device.
91 | The idea is in playback mode you cause sound to be emitted from the speakers by writing audio data
92 | to the output buffer (`pOutput` in the example). In capture mode you read data from the input
93 | buffer (`pInput`) to extract sound captured by the microphone. The `frameCount` parameter tells you
94 | how many frames can be written to the output buffer and read from the input buffer. A "frame" is
95 | one sample for each channel. For example, in a stereo stream (2 channels), one frame is 2
96 | samples: one for the left, one for the right. The channel count is defined by the device config.
97 | The size in bytes of an individual sample is defined by the sample format which is also specified
98 | in the device config. Multi-channel audio data is always interleaved, which means the samples for
99 | each frame are stored next to each other in memory. For example, in a stereo stream the first pair
100 | of samples will be the left and right samples for the first frame, the second pair of samples will
101 | be the left and right samples for the second frame, etc.
102 |
103 | The configuration of the device is defined by the `ma_device_config` structure. The config object
104 | is always initialized with `ma_device_config_init()`. It's important to always initialize the
105 | config with this function as it initializes it with logical defaults and ensures your program
106 | doesn't break when new members are added to the `ma_device_config` structure. The example above
107 | uses a fairly simple and standard device configuration. The call to `ma_device_config_init()` takes
108 | a single parameter, which is whether or not the device is a playback, capture, duplex or loopback
109 | device (loopback devices are not supported on all backends). The `config.playback.format` member
110 | sets the sample format which can be one of the following (all formats are native-endian):
111 |
112 | +-----+-----+-----+-----+
113 | | Symbol      | Description                               | Range                |
114 | +-----+-----+-----+-----+
115 | | ma_format_f32 | 32-bit floating point                     | [-1, 1]              |
116 | | ma_format_s16 | 16-bit signed integer                     | [-32768, 32767]      |
117 | | ma_format_s24 | 24-bit signed integer (tightly packed)    | [-8388608, 8388607]  |
118 | | ma_format_s32 | 32-bit signed integer                     | [-2147483648, 2147483647] |
119 | | ma_format_u8  | 8-bit unsigned integer                    | [0, 255]             |
120 | +-----+-----+-----+-----+
121 |
122 | The `config.playback.channels` member sets the number of channels to use with the device. The
123 | channel count cannot exceed MA_MAX_CHANNELS. The `config.sampleRate` member sets the sample rate
124 | (which must be the same for both playback and capture in full-duplex configurations). This is
125 | usually set to 44100 or 48000, but can be set to anything. It's recommended to keep this between
126 | 8000 and 384000, however.
127 |
128 | Note that leaving the format, channel count and/or sample rate at their default values will result
129 | in the internal device's native configuration being used which is useful if you want to avoid the
130 | overhead of miniaudio's automatic data conversion.
131 |
132 | In addition to the sample format, channel count and sample rate, the data callback and user data
133 | pointer are also set via the config. The user data pointer is not passed into the callback as a
134 | parameter, but is instead set to the `pUserData` member of `ma_device` which you can access
135 | directly since all miniaudio structures are transparent.
136 |
137 | Initializing the device is done with `ma_device_init()`. This will return a result code telling you
138 | what went wrong, if anything. On success it will return `MA_SUCCESS`. After initialization is
139 | complete the device will be in a stopped state. To start it, use `ma_device_start()`.
140 | Uninitializing the device will stop it, which is what the example above does, but you can also stop
141 | the device with `ma_device_stop()`. To resume the device simply call `ma_device_start()` again.
142 | Note that it's important to never stop or start the device from inside the callback. This will
143 | result in a deadlock. Instead you set a variable or signal an event indicating that the device

```

```

144 | needs to stop and handle it in a different thread. The following APIs must never be called inside
145 | the callback:
146 |
147 |     ``c
148 |     ma_device_init()
149 |     ma_device_init_ex()
150 |     ma_device_uninit()
151 |     ma_device_start()
152 |     ma_device_stop()
153 |     ``
154 |
155 | You must never try uninitialized and reinitializing a device inside the callback. You must also
156 | never try to stop and start it from inside the callback. There are a few other things you shouldn't
157 | do in the callback depending on your requirements, however this isn't so much a thread-safety
158 | thing, but rather a real-time processing thing which is beyond the scope of this introduction.
159 |
160 | The example above demonstrates the initialization of a playback device, but it works exactly the
161 | same for capture. All you need to do is change the device type from `ma_device_type_playback` to
162 | `ma_device_type_capture` when setting up the config, like so:
163 |
164 |     ``c
165 |     ma_device_config config = ma_device_config_init(ma_device_type_capture);
166 |     config.capture.format    = MY_FORMAT;
167 |     config.capture.channels  = MY_CHANNEL_COUNT;
168 |     ``
169 |
170 | In the data callback you just read from the input buffer (`pInput` in the example above) and leave
171 | the output buffer alone (it will be set to NULL when the device type is set to
172 | `ma_device_type_capture`).
173 |
174 | These are the available device types and how you should handle the buffers in the callback:
175 |
176 | +-----+-----+-----+
177 | | Device Type          | | Callback Behavior          | |
178 | +-----+-----+-----+
179 | | ma_device_type_playback | | Write to output buffer, leave input buffer untouched. | |
180 | | ma_device_type_capture  | | Read from input buffer, leave output buffer untouched. | |
181 | | ma_device_type_duplex   | | Read from input buffer, write to output buffer.         | |
182 | | ma_device_type_loopback | | Read from input buffer, leave output buffer untouched. | |
183 | +-----+-----+-----+
184 |
185 | You will notice in the example above that the sample format and channel count is specified
186 | separately for playback and capture. This is to support different data formats between the playback
187 | and capture devices in a full-duplex system. An example may be that you want to capture audio data
188 | as a monaural stream (one channel), but output sound to a stereo speaker system. Note that if you
189 | use different formats between playback and capture in a full-duplex configuration you will need to
190 | convert the data yourself. There are functions available to help you do this which will be
191 | explained later.
192 |
193 | The example above did not specify a physical device to connect to which means it will use the
194 | operating system's default device. If you have multiple physical devices connected and you want to
195 | use a specific one you will need to specify the device ID in the configuration, like so:
196 |
197 |     ``c
198 |     config.playback.pDeviceID = pMyPlaybackDeviceID;    // Only if requesting a playback or duplex
199 |     config.capture.pDeviceID  = pMyCaptureDeviceID;     // Only if requesting a capture, duplex or
200 |     ``
201 |
202 | To retrieve the device ID you will need to perform device enumeration, however this requires the
203 | use of a new concept called the "context". Conceptually speaking the context sits above the device.
204 | There is one context to many devices. The purpose of the context is to represent the backend at a
205 | more global level and to perform operations outside the scope of an individual device. Mainly it is
206 | used for performing run-time linking against backend libraries, initializing backends and
207 | enumerating devices. The example below shows how to enumerate devices.
208 |
209 |     ``c
210 |     ma_context context;
211 |     if (ma_context_init(NULL, 0, NULL, &context) != MA_SUCCESS) {
212 |         // Error.
213 |     }
214 |
215 |     ma_device_info* pPlaybackInfos;
216 |     ma_uint32 playbackCount;
217 |     ma_device_info* pCaptureInfos;
218 |     ma_uint32 captureCount;
219 |     if (ma_context_get_devices(&context, &pPlaybackInfos, &playbackCount, &pCaptureInfos,
220 | > &captureCount) != MA_SUCCESS) {

```

```

220 |         // Error.
221 |     }
222 |
223 |     // Loop over each device info and do something with it. Here we just print the name with their
> index. You may want
224 |     // to give the user the opportunity to choose which device they'd prefer.
225 |     for (ma_uint32 iDevice = 0; iDevice < playbackCount; iDevice += 1) {
226 |         printf("%d - %s\n", iDevice, pPlaybackInfos[iDevice].name);
227 |     }
228 |
229 |     ma_device_config config = ma_device_config_init(ma_device_type_playback);
230 |     config.playback.pDeviceID = &pPlaybackInfos[chosenPlaybackDeviceIndex].id;
231 |     config.playback.format      = MY_FORMAT;
232 |     config.playback.channels    = MY_CHANNEL_COUNT;
233 |     config.sampleRate          = MY_SAMPLE_RATE;
234 |     config.dataCallback         = data_callback;
235 |     config.pUserData            = pMyCustomData;
236 |
237 |     ma_device device;
238 |     if (ma_device_init(&context, &config, &device) != MA_SUCCESS) {
239 |         // Error
240 |     }
241 |
242 |     ...
243 |
244 |     ma_device_uninit(&device);
245 |     ma_context_uninit(&context);
246 |     ```
247 |
248 | The first thing we do in this example is initialize a `ma_context` object with `ma_context_init()`.
249 | The first parameter is a pointer to a list of `ma_backend` values which are used to override the
250 | default backend priorities. When this is NULL, as in this example, miniaudio's default priorities
251 | are used. The second parameter is the number of backends listed in the array pointed to by the
252 | first parameter. The third parameter is a pointer to a `ma_context_config` object which can be
253 | NULL, in which case defaults are used. The context configuration is used for setting the logging
254 | callback, custom memory allocation callbacks, user-defined data and some backend-specific
255 | configurations.
256 |
257 | Once the context has been initialized you can enumerate devices. In the example above we use the
258 | simpler `ma_context_get_devices()`, however you can also use a callback for handling devices by
259 | using `ma_context_enumerate_devices()`. When using `ma_context_get_devices()` you provide a pointer
260 | to a pointer that will, upon output, be set to a pointer to a buffer containing a list of
261 | `ma_device_info` structures. You also provide a pointer to an unsigned integer that will receive
262 | the number of items in the returned buffer. Do not free the returned buffers as their memory is
263 | managed internally by miniaudio.
264 |
265 | The `ma_device_info` structure contains an `id` member which is the ID you pass to the device
266 | config. It also contains the name of the device which is useful for presenting a list of devices
267 | to the user via the UI.
268 |
269 | When creating your own context you will want to pass it to `ma_device_init()` when initializing the
270 | device. Passing in NULL, like we do in the first example, will result in miniaudio creating the
271 | context for you, which you don't want to do since you've already created a context. Note that
272 | internally the context is only tracked by it's pointer which means you must not change the location
273 | of the `ma_context` object. If this is an issue, consider using `malloc()` to allocate memory for
274 | the context.
275 |
276 |
277 | 1.2. High Level API
278 | -----
279 | The high level API consists of three main parts:
280 |
281 | * Resource management for loading and streaming sounds.
282 | * A node graph for advanced mixing and effect processing.
283 | * A high level "engine" that wraps around the resource manager and node graph.
284 |
285 | The resource manager (`ma_resource_manager`) is used for loading sounds. It supports loading sounds
286 | fully into memory and also streaming. It will also deal with reference counting for you which
287 | avoids the same sound being loaded multiple times.
288 |
289 | The node graph is used for mixing and effect processing. The idea is that you connect a number of
290 | nodes into the graph by connecting each node's outputs to another node's inputs. Each node can
291 | implement its own effect. By chaining nodes together, advanced mixing and effect processing can
292 | be achieved.
293 |
294 | The engine encapsulates both the resource manager and the node graph to create a simple, easy to
295 | use high level API. The resource manager and node graph APIs are covered in more later sections of
296 | this manual.
297 |

```

```

298 | The code below shows how you can initialize an engine using its default configuration.
299 |
300 |     ``c
301 |     ma_result result;
302 |     ma_engine engine;
303 |
304 |     result = ma_engine_init(NULL, &engine);
305 |     if (result != MA_SUCCESS) {
306 |         return result; // Failed to initialize the engine.
307 |     }
308 |     ...
309 |
310 | This creates an engine instance which will initialize a device internally which you can access with
311 | `ma_engine_get_device()`. It will also initialize a resource manager for you which can be accessed
312 | with `ma_engine_get_resource_manager()`. The engine itself is a node graph (`ma_node_graph`) which
313 | means you can pass a pointer to the engine object into any of the `ma_node_graph` APIs (with a
314 | cast). Alternatively, you can use `ma_engine_get_node_graph()` instead of a cast.
315 |
316 | Note that all objects in miniaudio, including the `ma_engine` object in the example above, are
317 | transparent structures. There are no handles to opaque structures in miniaudio which means you need
318 | to be mindful of how you declare them. In the example above we are declaring it on the stack, but
319 | this will result in the struct being invalidated once the function encapsulating it returns. If
320 | allocating the engine on the heap is more appropriate, you can easily do so with a standard call
321 | to `malloc()` or whatever heap allocation routine you like:
322 |
323 |     ``c
324 |     ma_engine* pEngine = malloc(sizeof(*pEngine));
325 |     ...
326 |
327 | The `ma_engine` API uses the same config/init pattern used all throughout miniaudio. To configure
328 | an engine, you can fill out a `ma_engine_config` object and pass it into the first parameter of
329 | `ma_engine_init()`:
330 |
331 |     ``c
332 |     ma_result result;
333 |     ma_engine engine;
334 |     ma_engine_config engineConfig;
335 |
336 |     engineConfig = ma_engine_config_init();
337 |     engineConfig.pResourceManager = &myCustomResourceManager; // <-- Initialized as some earlier
> stage.
338 |
339 |     result = ma_engine_init(&engineConfig, &engine);
340 |     if (result != MA_SUCCESS) {
341 |         return result;
342 |     }
343 |     ...
344 |
345 | This creates an engine instance using a custom config. In this particular example it's showing how
346 | you can specify a custom resource manager rather than having the engine initialize one internally.
347 | This is particularly useful if you want to have multiple engine's share the same resource manager.
348 |
349 | The engine must be uninitialized with `ma_engine_uninit()` when it's no longer needed.
350 |
351 | By default the engine will be started, but nothing will be playing because no sounds have been
352 | initialized. The easiest but least flexible way of playing a sound is like so:
353 |
354 |     ``c
355 |     ma_engine_play_sound(&engine, "my_sound.wav", NULL);
356 |     ...
357 |
358 | This plays what miniaudio calls an "inline" sound. It plays the sound once, and then puts the
359 | internal sound up for recycling. The last parameter is used to specify which sound group the sound
360 | should be associated with which will be explained later. This particular way of playing a sound is
361 | simple, but lacks flexibility and features. A more flexible way of playing a sound is to first
362 | initialize a sound:
363 |
364 |     ``c
365 |     ma_result result;
366 |     ma_sound sound;
367 |
368 |     result = ma_sound_init_from_file(&engine, "my_sound.wav", 0, NULL, NULL, &sound);
369 |     if (result != MA_SUCCESS) {
370 |         return result;
371 |     }
372 |
373 |     ma_sound_start(&sound);
374 |     ...
375 |

```



```

376 | This returns a `ma_sound` object which represents a single instance of the specified sound file. If
377 | you want to play the same file multiple times simultaneously, you need to create one sound for each
378 | instance.
379 |
380 | Sounds should be uninitialized with `ma_sound_uninit()`.
381 |
382 | Sounds are not started by default. Start a sound with `ma_sound_start()` and stop it with
383 | `ma_sound_stop()`. When a sound is stopped, it is not rewound to the start. Use
384 | `ma_sound_seek_to_pcm_frame(&sound, 0)` to seek back to the start of a sound. By default, starting
385 | and stopping sounds happens immediately, but sometimes it might be convenient to schedule the sound
386 | to be started and/or stopped at a specific time. This can be done with the following functions:
387 |
388 |     ```c
389 |     ma_sound_set_start_time_in_pcm_frames()
390 |     ma_sound_set_start_time_in_milliseconds()
391 |     ma_sound_set_stop_time_in_pcm_frames()
392 |     ma_sound_set_stop_time_in_milliseconds()
393 |     ```
394 |
395 | The start/stop time needs to be specified based on the absolute timer which is controlled by the
396 | engine. The current global time in PCM frames can be retrieved with
397 | `ma_engine_get_time_in_pcm_frames()`. The engine's global time can be changed with
398 | `ma_engine_set_time_in_pcm_frames()` for synchronization purposes if required. Note that scheduling
399 | a start time still requires an explicit call to `ma_sound_start()` before anything will play:
400 |
401 |     ```c
402 |     ma_sound_set_start_time_in_pcm_frames(&sound, ma_engine_get_time_in_pcm_frames(&engine) +
> (ma_engine_get_sample_rate(&engine) * 2);
403 |     ma_sound_start(&sound);
404 |     ```
405 |
406 | The third parameter of `ma_sound_init_from_file()` is a set of flags that control how the sound be
407 | loaded and a few options on which features should be enabled for that sound. By default, the sound
408 | is synchronously loaded fully into memory straight from the file system without any kind of
409 | decoding. If you want to decode the sound before storing it in memory, you need to specify the
410 | `MA_SOUND_FLAG_DECODE` flag. This is useful if you want to incur the cost of decoding at an earlier
411 | stage, such as a loading stage. Without this option, decoding will happen dynamically at mixing
412 | time which might be too expensive on the audio thread.
413 |
414 | If you want to load the sound asynchronously, you can specify the `MA_SOUND_FLAG_ASYNC` flag. This
415 | will result in `ma_sound_init_from_file()` returning quickly, but the sound will not start playing
416 | until the sound has had some audio decoded.
417 |
418 | The fourth parameter is a pointer to sound group. A sound group is used as a mechanism to organise
419 | sounds into groups which have their own effect processing and volume control. An example is a game
420 | which might have separate groups for sfx, voice and music. Each of these groups have their own
421 | independent volume control. Use `ma_sound_group_init()` or `ma_sound_group_init_ex()` to initialize
422 | a sound group.
423 |
424 | Sounds and sound groups are nodes in the engine's node graph and can be plugged into any `ma_node`
425 | API. This makes it possible to connect sounds and sound groups to effect nodes to produce complex
426 | effect chains.
427 |
428 | A sound can have its volume changed with `ma_sound_set_volume()`. If you prefer decibel volume
429 | control you can use `ma_volume_db_to_linear()` to convert from decibel representation to linear.
430 |
431 | Panning and pitching is supported with `ma_sound_set_pan()` and `ma_sound_set_pitch()`. If you know
432 | a sound will never have its pitch changed with `ma_sound_set_pitch()` or via the doppler effect,
433 | you can specify the `MA_SOUND_FLAG_NO_PITCH` flag when initializing the sound for an optimization.
434 |
435 | By default, sounds and sound groups have spatialization enabled. If you don't ever want to
436 | spatialize your sounds, initialize the sound with the `MA_SOUND_FLAG_NO_SPATIALIZATION` flag. The
437 | spatialization model is fairly simple and is roughly on feature parity with OpenAL. HRTF and
438 | environmental occlusion are not currently supported, but planned for the future. The supported
439 | features include:
440 |
441 |     * Sound and listener positioning and orientation with cones
442 |     * Attenuation models: none, inverse, linear and exponential
443 |     * Doppler effect
444 |
445 | Sounds can be faded in and out with `ma_sound_set_fade_in_pcm_frames()`.
446 |
447 | To check if a sound is currently playing, you can use `ma_sound_is_playing()`. To check if a sound
448 | is at the end, use `ma_sound_at_end()`. Looping of a sound can be controlled with
449 | `ma_sound_set_looping()`. Use `ma_sound_is_looping()` to check whether or not the sound is looping.
450 |
451 |
452 |
453 | 2. Building

```

```

454 | =====
455 | miniaudio should work cleanly out of the box without the need to download or install any
456 | dependencies. See below for platform-specific details.
457 |
458 | This library has been designed to be added directly to your source tree which is the preferred way
459 | of using it, but you can compile it as a normal library if that's your preference. Be careful if
460 | compiling as a shared object because miniaudio is not ABI compatible between any release, including
461 | bug fix releases. It's recommended you link statically.
462 |
463 | Note that GCC and Clang require `-msse2`, `-mavx2`, etc. for SIMD optimizations.
464 |
465 | If you get errors about undefined references to `__sync_val_compare_and_swap_8`, `__atomic_load_8`,
466 | etc. you need to link with `-latomic`.
467 |
468 |
469 | 2.1. Windows
470 | -----
471 | The Windows build should compile cleanly on all popular compilers without the need to configure any
472 | include paths nor link to any libraries.
473 |
474 | The UWP build may require linking to mmdevapi.lib if you get errors about an unresolved external
475 | symbol for `ActivateAudioInterfaceAsync()`.
476 |
477 |
478 | 2.2. macOS and iOS
479 | -----
480 | The macOS build should compile cleanly without the need to download any dependencies nor link to
481 | any libraries or frameworks. The iOS build needs to be compiled as Objective-C and will need to
482 | link the relevant frameworks but should compile cleanly out of the box with Xcode. Compiling
483 | through the command line requires linking to `-lpthread` and `-lm`.
484 |
485 | Due to the way miniaudio links to frameworks at runtime, your application may not pass Apple's
486 | notarization process. To fix this there are two options. The first is to compile with
487 | `-DMA_NO_RUNTIME_LINKING` which in turn will require linking with
488 | `-framework CoreFoundation -framework CoreAudio -framework AudioToolbox`. If you get errors about
489 | AudioToolbox, try with `-framework AudioUnit` instead. You may get this when using older versions
490 | of iOS. Alternatively, if you would rather keep using runtime linking you can add the following to
491 | your entitlements.xcent file:
492 |
493 |     <key>com.apple.security.cs.allow-dyld-environment-variables</key>
494 |     <true/>
495 |     <key>com.apple.security.cs.allow-unsigned-executable-memory</key>
496 |     <true/>
497 |
498 |
499 | See this discussion for more info: https://github.com/mackron/miniaudio/issues/203.
500 |
501 |
502 |
503 | 2.3. Linux
504 | -----
505 | The Linux build only requires linking to `-ldl`, `-lpthread` and `-lm`. You do not need any
506 | development packages. You may need to link with `-latomic` if you're compiling for 32-bit ARM.
507 |
508 |
509 | 2.4. BSD
510 | -----
511 | The BSD build only requires linking to `-lpthread` and `-lm`. NetBSD uses audio(4), OpenBSD uses
512 | sndio and FreeBSD uses OSS. You may need to link with `-latomic` if you're compiling for 32-bit
513 | ARM.
514 |
515 |
516 | 2.5. Android
517 | -----
518 | AAudio is the highest priority backend on Android. This should work out of the box without needing
519 | any kind of compiler configuration. Support for AAudio starts with Android 8 which means older
520 | versions will fall back to OpenSL|ES which requires API level 16+.
521 |
522 | There have been reports that the OpenSL|ES backend fails to initialize on some Android based
523 | devices due to `dlopen()` failing to open "libOpenSLES.so". If this happens on your platform
524 | you'll need to disable run-time linking with `MA_NO_RUNTIME_LINKING` and link with -lOpenSLES.
525 |
526 |
527 | 2.6. Emscripten
528 | -----
529 | The Emscripten build emits Web Audio JavaScript directly and should compile cleanly out of the box.
530 | You cannot use `-std=c*` compiler flags, nor `-ansi`.
531 |
532 | You can enable the use of AudioWorklets by defining `MA_ENABLE_AUDIO_WORKLETS` and then compiling

```

```

533 | with the following options:
534 |
535 |     -sAUDIO_WORKLET=1 -sWASM_WORKERS=1 -sASYNCIFY
536 |
537 | An example for compiling with AudioWorklet support might look like this:
538 |
539 |     emcc program.c -o bin/program.html -DMA_ENABLE_AUDIO_WORKLETS -sAUDIO_WORKLET=1 -sWASM_WORKERS=1
540 | > -sASYNCIFY
541 |
542 | To run locally, you'll need to use emrun:
543 |
544 |     emrun bin/program.html
545 |
546 |
547 | 2.7. Build Options
548 | -----
549 | `#define` these options before including miniaudio.c, or pass them as compiler flags:
550 |
551 | +-----+-----+-----+
552 | | Option                                | Description |
553 | +-----+-----+-----+
554 | | MA_NO_WASAPI                         | Disables the WASAPI backend. |
555 | +-----+-----+-----+
556 | | MA_NO_DSOUND                         | Disables the DirectSound backend. |
557 | +-----+-----+-----+
558 | | MA_NO_WINMM                         | Disables the WinMM backend. |
559 | +-----+-----+-----+
560 | | MA_NO_ALSA                         | Disables the ALSA backend. |
561 | +-----+-----+-----+
562 | | MA_NO_PULSEAUDIO                   | Disables the PulseAudio backend. |
563 | +-----+-----+-----+
564 | | MA_NO_JACK                         | Disables the JACK backend. |
565 | +-----+-----+-----+
566 | | MA_NO_COREAUDIO                   | Disables the Core Audio backend. |
567 | +-----+-----+-----+
568 | | MA_NO_SNDIO                       | Disables the sndio backend. |
569 | +-----+-----+-----+
570 | | MA_NO_AUDIO4                       | Disables the audio(4) backend. |
571 | +-----+-----+-----+
572 | | MA_NO_OSS                         | Disables the OSS backend. |
573 | +-----+-----+-----+
574 | | MA_NO_AAUDIO                       | Disables the AAudio backend. |
575 | +-----+-----+-----+
576 | | MA_NO_OPENSLE                      | Disables the OpenSLES backend. |
577 | +-----+-----+-----+
578 | | MA_NO_WEBAUDIO                     | Disables the Web Audio backend. |
579 | +-----+-----+-----+
580 | | MA_NO_CUSTOM                       | Disables support for custom backends. |
581 | +-----+-----+-----+

```

```

581 | +-----+-----+-----+
> |-----+
582 | | MA_NO_NULL | Disables the null backend.
> | |
583 | +-----+-----+-----+
> |-----+
584 | | MA_ENABLE_ONLY_SPECIFIC_BACKENDS | Disables all backends by default and requires `MA_ENABLE_*`
> to |
585 | | | enable specific backends.
> | |
586 | +-----+-----+-----+
> |-----+
587 | | MA_ENABLE_WASAPI | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
588 | | | enable the WASAPI backend.
> | |
589 | +-----+-----+-----+
> |-----+
590 | | MA_ENABLE_DSOUND | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
591 | | | enable the DirectSound backend.
> | |
592 | +-----+-----+-----+
> |-----+
593 | | MA_ENABLE_WINMM | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
594 | | | enable the WinMM backend.
> | |
595 | +-----+-----+-----+
> |-----+
596 | | MA_ENABLE_ALSA | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
597 | | | enable the ALSA backend.
> | |
598 | +-----+-----+-----+
> |-----+
599 | | MA_ENABLE_PULSEAUDIO | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
600 | | | enable the PulseAudio backend.
> | |
601 | +-----+-----+-----+
> |-----+
602 | | MA_ENABLE_JACK | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
603 | | | enable the JACK backend.
> | |
604 | +-----+-----+-----+
> |-----+
605 | | MA_ENABLE_COREAUDIO | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
606 | | | enable the Core Audio backend.
> | |
607 | +-----+-----+-----+
> |-----+
608 | | MA_ENABLE_SNDIO | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
609 | | | enable the sndio backend.
> | |
610 | +-----+-----+-----+
> |-----+
611 | | MA_ENABLE_AUDIO4 | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
612 | | | enable the audio(4) backend.
> | |
613 | +-----+-----+-----+
> |-----+
614 | | MA_ENABLE_OSS | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
615 | | | enable the OSS backend.
> | |
616 | +-----+-----+-----+
> |-----+
617 | | MA_ENABLE_AAUDIO | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
618 | | | enable the AAudio backend.
> | |
619 | +-----+-----+-----+
> |-----+
620 | | MA_ENABLE_OPENSL | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS

```

```

> to |
621 | | | enable the OpenSL|ES backend.
> |
622 | +-----+
> -----+
623 | | MA_ENABLE_WEBAUDIO | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
624 | | | enable the Web Audio backend.
> |
625 | +-----+
> -----+
626 | | MA_ENABLE_CUSTOM | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
627 | | | enable custom backends.
> |
628 | +-----+
> -----+
629 | | MA_ENABLE_NULL | Used in conjunction with MA_ENABLE_ONLY_SPECIFIC_BACKENDS
> to |
630 | | | enable the null backend.
> |
631 | +-----+
> -----+
632 | | MA_NO_DECODING | Disables decoding APIs.
> |
633 | +-----+
> -----+
634 | | MA_NO_ENCODING | Disables encoding APIs.
> |
635 | +-----+
> -----+
636 | | MA_NO_WAV | Disables the built-in WAV decoder and encoder.
> |
637 | +-----+
> -----+
638 | | MA_NO_FLAC | Disables the built-in FLAC decoder.
> |
639 | +-----+
> -----+
640 | | MA_NO_MP3 | Disables the built-in MP3 decoder.
> |
641 | +-----+
> -----+
642 | | MA_NO_DEVICE_IO | Disables playback and recording. This will disable
> `ma_context` |
643 | | | and `ma_device` APIs. This is useful if you only want to
> use |
644 | | | miniaudio's data conversion and/or decoding APIs.
> |
645 | +-----+
> -----+
646 | | MA_NO_RESOURCE_MANAGER | Disables the resource manager. When using the engine this
> will |
647 | | | also disable the following functions:
> |
648 | | |
> |
649 | | |
> |
650 | | | ma_sound_init_from_file()
> |
651 | | | ma_sound_init_from_file_w()
> |
652 | | | ma_sound_init_copy()
> |
653 | | | ma_engine_play_sound_ex()
> |
654 | | | ma_engine_play_sound()
> |
655 | | |
> |
656 | | |
> |
657 | | | The only way to initialize a `ma_sound` object is to
> initialize it |
658 | | | from a data source.
> |
659 | +-----+
> -----+

```

```

660 |      | MA_NO_NODE_GRAPH          | Disables the node graph API. This will also disable the
> engine API |
661 |      |                          | because it depends on the node graph.
>
662 |      +-----+
> -----+
663 |      | MA_NO_ENGINE              | Disables the engine API.
>      |                          |
664 |      +-----+
> -----+
665 |      | MA_NO_THREADING           | Disables the `ma_thread`, `ma_mutex`, `ma_semaphore` and
>      |                          | `ma_event` APIs. This option is useful if you only need to
666 |      |                          | use
> use      |                          | miniaudio for data conversion, decoding and/or encoding.
667 |      |                          | Some
> Some      |                          | families of APIs require threading which means the
668 |      |                          | following
> following  |                          | options must also be set:
669 |      |                          |
>      |                          |
670 |      |                          |
>      |                          |
671 |      |                          |
>      |                          |
672 |      |                          | MA_NO_DEVICE_IO
>      |                          |
673 |      |                          |
>      |                          |
674 |      +-----+
> -----+
675 |      | MA_NO_GENERATION           | Disables generation APIs such a `ma_waveform` and
> `ma_noise`. |
676 |      +-----+
> -----+
677 |      | MA_NO_SSE2                | Disables SSE2 optimizations.
>      |                          |
678 |      +-----+
> -----+
679 |      | MA_NO_AVX2                | Disables AVX2 optimizations.
>      |                          |
680 |      +-----+
> -----+
681 |      | MA_NO_NEON                | Disables NEON optimizations.
>      |                          |
682 |      +-----+
> -----+
683 |      | MA_NO_RUNTIME_LINKING     | Disables runtime linking. This is useful for passing
> Apple's      |                          | notarization process. When enabling this, you may need to
684 |      |                          | avoid
> avoid      |                          | using `-std=c89` or `-std=c99` on Linux builds or else you
685 |      |                          | may end |
> may end |                          | up with compilation errors due to conflicts with `timespec`
686 |      |                          | and
> and      |                          | `timeval` data types.
687 |      |                          |
>      |                          |
688 |      |                          |
>      |                          |
689 |      |                          | You may need to enable this if your target platform does
> not allow |                          | runtime linking via `dlopen()`.
690 |      |                          |
>      |                          |
691 |      +-----+
> -----+
692 |      | MA_USE_STDINT              | (Pass this in as a compiler flag. Do not `#define` this
> before      |                          | miniaudio.c) Forces the use of stdint.h for sized types.
693 |      |                          |
>      |                          |
694 |      +-----+
> -----+
695 |      | MA_DEBUG_OUTPUT            | Enable `printf()` output of debug logs
> (`MA_LOG_LEVEL_DEBUG`). |
696 |      +-----+
> -----+
697 |      | MA_COINIT_VALUE            | Windows only. The value to pass to internal calls to
>      |                          | `CoInitializeEx()`. Defaults to `COINIT_MULTITHREADED`.
698 |      |                          |
>      |                          |
699 |      +-----+

```



```

> -----+
700 |     | MA_FORCE_UWP           | Windows only. Affects only the WASAPI backend. Will force
> the |                           |
701 |     |                           | WASAPI backend to use the UWP code path instead of the
> regular |                       |
702 |     |                           | desktop path. This is normally auto-detected and should
> rarely be |                   |
703 |     |                           | needed to be used explicitly, but can be useful for
> debugging. |
704 |     +-----+-----+-----+-----+-----+-----+
> -----+
705 |     | MA_ON_THREAD_ENTRY      | Defines some code that will be executed as soon as an
> internal |                       |
706 |     |                           | miniaudio-managed thread is created. This will be the first
> thing |                           |
707 |     |                           | to be executed by the thread entry point.
>
708 |     +-----+-----+-----+-----+-----+-----+
> -----+
709 |     | MA_ON_THREAD_EXIT      | Defines some code that will be executed from the entry
> point of an |                       |
710 |     |                           | internal miniaudio-managed thread upon exit. This will be
> the last |                           |
711 |     |                           | thing to be executed before the thread's entry point exits.
>
712 |     +-----+-----+-----+-----+-----+-----+
> -----+
713 |     | MA_THREAD_DEFAULT_STACK_SIZE | If set, specifies the default stack size used by
> miniaudio-managed |                       |
714 |     |                           | threads.
>
715 |     +-----+-----+-----+-----+-----+-----+
> -----+
716 |     | MA_API                 | Controls how public APIs should be decorated. Default is
> `extern`. |
717 |     +-----+-----+-----+-----+-----+-----+
> -----+
718 |
719 |
720 | 3. Definitions
721 | =====
722 | This section defines common terms used throughout miniaudio. Unfortunately there is often ambiguity
723 | in the use of terms throughout the audio space, so this section is intended to clarify how miniaudio
724 | uses each term.
725 |
726 | 3.1. Sample
727 | -----
728 | A sample is a single unit of audio data. If the sample format is f32, then one sample is one 32-bit
729 | floating point number.
730 |
731 | 3.2. Frame / PCM Frame
732 | -----
733 | A frame is a group of samples equal to the number of channels. For a stereo stream a frame is 2
734 | samples, a mono frame is 1 sample, a 5.1 surround sound frame is 6 samples, etc. The terms "frame"
735 | and "PCM frame" are the same thing in miniaudio. Note that this is different to a compressed frame.
736 | If ever miniaudio needs to refer to a compressed frame, such as a FLAC frame, it will always
737 | clarify what it's referring to with something like "FLAC frame".
738 |
739 | 3.3. Channel
740 | -----
741 | A stream of monaural audio that is emitted from an individual speaker in a speaker system, or
742 | received from an individual microphone in a microphone system. A stereo stream has two channels (a
743 | left channel, and a right channel), a 5.1 surround sound system has 6 channels, etc. Some audio
744 | systems refer to a channel as a complex audio stream that's mixed with other channels to produce
745 | the final mix - this is completely different to miniaudio's use of the term "channel" and should
746 | not be confused.
747 |
748 | 3.4. Sample Rate
749 | -----
750 | The sample rate in miniaudio is always expressed in Hz, such as 44100, 48000, etc. It's the number
751 | of PCM frames that are processed per second.
752 |
753 | 3.5. Formats
754 | -----
755 | Throughout miniaudio you will see references to different sample formats:
756 |
757 | +-----+-----+-----+-----+-----+-----+
758 | | Symbol          | Description                               | Range                               |
759 | +-----+-----+-----+-----+-----+-----+

```

```

760 | | ma_format_f32 | 32-bit floating point | [-1, 1] |
761 | | ma_format_s16 | 16-bit signed integer | [-32768, 32767] |
762 | | ma_format_s24 | 24-bit signed integer (tightly packed) | [-8388608, 8388607] |
763 | | ma_format_s32 | 32-bit signed integer | [-2147483648, 2147483647] |
764 | | ma_format_u8 | 8-bit unsigned integer | [0, 255] |
765 | +-----+-----+-----+-----+-----+-----+
766 |
767 | All formats are native-endian.
768 |
769 |
770 |
771 | 4. Data Sources
772 | =====
773 | The data source abstraction in miniaudio is used for retrieving audio data from some source. A few
774 | examples include `ma_decoder`, `ma_noise` and `ma_waveform`. You will need to be familiar with data
775 | sources in order to make sense of some of the higher level concepts in miniaudio.
776 |
777 | The `ma_data_source` API is a generic interface for reading from a data source. Any object that
778 | implements the data source interface can be plugged into any `ma_data_source` function.
779 |
780 | To read data from a data source:
781 |
782 | ```c
783 |     ma_result result;
784 |     ma_uint64 framesRead;
785 |
786 |     result = ma_data_source_read_pcm_frames(pDataSource, pFramesOut, frameCount, &framesRead);
787 |     if (result != MA_SUCCESS) {
788 |         return result; // Failed to read data from the data source.
789 |     }
790 |     ...
791 |
792 | If you don't need the number of frames that were successfully read you can pass in `NULL` to the
793 | `pFramesRead` parameter. If this returns a value less than the number of frames requested it means
794 | the end of the file has been reached. `MA_AT_END` will be returned only when the number of frames
795 | read is 0.
796 |
797 | When calling any data source function, with the exception of `ma_data_source_init()` and
798 | `ma_data_source_uninit()`, you can pass in any object that implements a data source. For example,
799 | you could plug in a decoder like so:
800 |
801 | ```c
802 |     ma_result result;
803 |     ma_uint64 framesRead;
804 |     ma_decoder decoder; // <-- This would be initialized with `ma_decoder_init_*()``.
805 |
806 |     result = ma_data_source_read_pcm_frames(&decoder, pFramesOut, frameCount, &framesRead);
807 |     if (result != MA_SUCCESS) {
808 |         return result; // Failed to read data from the decoder.
809 |     }
810 |     ...
811 |
812 | If you want to seek forward you can pass in `NULL` to the `pFramesOut` parameter. Alternatively you
813 | can use `ma_data_source_seek_pcm_frames()`.
814 |
815 | To seek to a specific PCM frame:
816 |
817 | ```c
818 |     result = ma_data_source_seek_to_pcm_frame(pDataSource, frameIndex);
819 |     if (result != MA_SUCCESS) {
820 |         return result; // Failed to seek to PCM frame.
821 |     }
822 |     ...
823 |
824 | You can retrieve the total length of a data source in PCM frames, but note that some data sources
825 | may not have the notion of a length, such as noise and waveforms, and others may just not have a
826 | way of determining the length such as some decoders. To retrieve the length:
827 |
828 | ```c
829 |     ma_uint64 length;
830 |
831 |     result = ma_data_source_get_length_in_pcm_frames(pDataSource, &length);
832 |     if (result != MA_SUCCESS) {
833 |         return result; // Failed to retrieve the length.
834 |     }
835 |     ...
836 |
837 | Care should be taken when retrieving the length of a data source where the underlying decoder is
838 | pulling data from a data stream with an undefined length, such as internet radio or some kind of

```

```

839 | broadcast. If you do this, `ma_data_source_get_length_in_pcm_frames()` may never return.
840 |
841 | The current position of the cursor in PCM frames can also be retrieved:
842 |
843 |     ```c
844 |     ma_uint64 cursor;
845 |
846 |     result = ma_data_source_get_cursor_in_pcm_frames(pDataSource, &cursor);
847 |     if (result != MA_SUCCESS) {
848 |         return result; // Failed to retrieve the cursor.
849 |     }
850 |     ...
851 |
852 | You will often need to know the data format that will be returned after reading. This can be
853 | retrieved like so:
854 |
855 |     ```c
856 |     ma_format format;
857 |     ma_uint32 channels;
858 |     ma_uint32 sampleRate;
859 |     ma_channel channelMap[MA_MAX_CHANNELS];
860 |
861 |     result = ma_data_source_get_data_format(pDataSource, &format, &channels, &sampleRate,
> channelMap, MA_MAX_CHANNELS);
862 |     if (result != MA_SUCCESS) {
863 |         return result; // Failed to retrieve data format.
864 |     }
865 |     ...
866 |
867 | If you do not need a specific data format property, just pass in NULL to the respective parameter.
868 |
869 | There may be cases where you want to implement something like a sound bank where you only want to
870 | read data within a certain range of the underlying data. To do this you can use a range:
871 |
872 |     ```c
873 |     result = ma_data_source_set_range_in_pcm_frames(pDataSource, rangeBegInFrames,
> rangeEndInFrames);
874 |     if (result != MA_SUCCESS) {
875 |         return result; // Failed to set the range.
876 |     }
877 |     ...
878 |
879 | This is useful if you have a sound bank where many sounds are stored in the same file and you want
880 | the data source to only play one of those sub-sounds. Note that once the range is set, everything
881 | that takes a position, such as cursors and loop points, should always be relative to the start of
882 | the range. When the range is set, any previously defined loop point will be reset.
883 |
884 | Custom loop points can also be used with data sources. By default, data sources will loop after
885 | they reach the end of the data source, but if you need to loop at a specific location, you can do
886 | the following:
887 |
888 |     ```c
889 |     result = ma_data_source_set_loop_point_in_pcm_frames(pDataSource, loopBegInFrames,
> loopEndInFrames);
890 |     if (result != MA_SUCCESS) {
891 |         return result; // Failed to set the loop point.
892 |     }
893 |     ...
894 |
895 | The loop point is relative to the current range.
896 |
897 | It's sometimes useful to chain data sources together so that a seamless transition can be achieved.
898 | To do this, you can use chaining:
899 |
900 |     ```c
901 |     ma_decoder decoder1;
902 |     ma_decoder decoder2;
903 |
904 |     // ... initialize decoders with ma_decoder_init_*() ...
905 |
906 |     result = ma_data_source_set_next(&decoder1, &decoder2);
907 |     if (result != MA_SUCCESS) {
908 |         return result; // Failed to set the next data source.
909 |     }
910 |
911 |     result = ma_data_source_read_pcm_frames(&decoder1, pFramesOut, frameCount, pFramesRead);
912 |     if (result != MA_SUCCESS) {
913 |         return result; // Failed to read from the decoder.
914 |     }

```

```

915 |     ...
916 |
917 | In the example above we're using decoders. When reading from a chain, you always want to read from
918 | the top level data source in the chain. In the example above, `decoder1` is the top level data
919 | source in the chain. When `decoder1` reaches the end, `decoder2` will start seamlessly without any
920 | gaps.
921 |
922 | Note that when looping is enabled, only the current data source will be looped. You can loop the
923 | entire chain by linking in a loop like so:
924 |
925 |     ...c
926 |     ma_data_source_set_next(&decoder1, &decoder2); // decoder1 -> decoder2
927 |     ma_data_source_set_next(&decoder2, &decoder1); // decoder2 -> decoder1 (loop back to the
> start).
928 |     ...
929 |
930 | Note that setting up chaining is not thread safe, so care needs to be taken if you're dynamically
931 | changing links while the audio thread is in the middle of reading.
932 |
933 | Do not use `ma_decoder_seek_to_pcm_frame()` as a means to reuse a data source to play multiple
934 | instances of the same sound simultaneously. This can be extremely inefficient depending on the type
935 | of data source and can result in glitching due to subtle changes to the state of internal filters.
936 | Instead, initialize multiple data sources for each instance.
937 |
938 |
939 | 4.1. Custom Data Sources
940 | -----
941 | You can implement a custom data source by implementing the functions in `ma_data_source_vtable`.
942 | Your custom object must have `ma_data_source_base` as it's first member:
943 |
944 |     ...c
945 |     struct my_data_source
946 |     {
947 |         ma_data_source_base base;
948 |         ...
949 |     };
950 |     ...
951 |
952 | In your initialization routine, you need to call `ma_data_source_init()` in order to set up the
953 | base object (`ma_data_source_base`):
954 |
955 |     ...c
956 |     static ma_result my_data_source_read(ma_data_source* pDataSource, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
957 |     {
958 |         // Read data here. Output in the same format returned by my_data_source_get_data_format().
959 |     }
960 |
961 |     static ma_result my_data_source_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
962 |     {
963 |         // Seek to a specific PCM frame here. Return MA_NOT_IMPLEMENTED if seeking is not supported.
964 |     }
965 |
966 |     static ma_result my_data_source_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
967 |     {
968 |         // Return the format of the data here.
969 |     }
970 |
971 |     static ma_result my_data_source_get_cursor(ma_data_source* pDataSource, ma_uint64* pCursor)
972 |     {
973 |         // Retrieve the current position of the cursor here. Return MA_NOT_IMPLEMENTED and set
> *pCursor to 0 if there is no notion of a cursor.
974 |     }
975 |
976 |     static ma_result my_data_source_get_length(ma_data_source* pDataSource, ma_uint64* pLength)
977 |     {
978 |         // Retrieve the length in PCM frames here. Return MA_NOT_IMPLEMENTED and set *pLength to 0
> if there is no notion of a length or if the length is unknown.
979 |     }
980 |
981 |     static ma_data_source_vtable g_my_data_source_vtable =
982 |     {
983 |         my_data_source_read,
984 |         my_data_source_seek,
985 |         my_data_source_get_data_format,
986 |         my_data_source_get_cursor,
987 |         my_data_source_get_length
988 |     };

```

```

989 |
990 |     ma_result my_data_source_init(my_data_source* pMyDataSource)
991 |     {
992 |         ma_result result;
993 |         ma_data_source_config baseConfig;
994 |
995 |         baseConfig = ma_data_source_config_init();
996 |         baseConfig.vtable = &g_my_data_source_vtable;
997 |
998 |         result = ma_data_source_init(&baseConfig, &pMyDataSource->base);
999 |         if (result != MA_SUCCESS) {
1000 |             return result;
1001 |         }
1002 |
1003 |         // ... do the initialization of your custom data source here ...
1004 |
1005 |         return MA_SUCCESS;
1006 |     }
1007 |
1008 |     void my_data_source_uninit(my_data_source* pMyDataSource)
1009 |     {
1010 |         // ... do the uninitialization of your custom data source here ...
1011 |
1012 |         // You must uninitialize the base data source.
1013 |         ma_data_source_uninit(&pMyDataSource->base);
1014 |     }
1015 |     ...
1016 |
1017 | Note that `ma_data_source_init()` and `ma_data_source_uninit()` are never called directly outside
1018 | of the custom data source. It's up to the custom data source itself to call these within their own
1019 | init/uninit functions.
1020 |
1021 |
1022 |
1023 | 5. Engine
1024 | =====
1025 | The `ma_engine` API is a high level API for managing and mixing sounds and effect processing. The
1026 | `ma_engine` object encapsulates a resource manager and a node graph, both of which will be
1027 | explained in more detail later.
1028 |
1029 | Sounds are called `ma_sound` and are created from an engine. Sounds can be associated with a mixing
1030 | group called `ma_sound_group` which are also created from the engine. Both `ma_sound` and
1031 | `ma_sound_group` objects are nodes within the engine's node graph.
1032 |
1033 | When the engine is initialized, it will normally create a device internally. If you would rather
1034 | manage the device yourself, you can do so and just pass a pointer to it via the engine config when
1035 | you initialize the engine. You can also just use the engine without a device, which again can be
1036 | configured via the engine config.
1037 |
1038 | The most basic way to initialize the engine is with a default config, like so:
1039 |
1040 |     ```c
1041 |     ma_result result;
1042 |     ma_engine engine;
1043 |
1044 |     result = ma_engine_init(NULL, &engine);
1045 |     if (result != MA_SUCCESS) {
1046 |         return result; // Failed to initialize the engine.
1047 |     }
1048 |     ...
1049 |
1050 | This will result in the engine initializing a playback device using the operating system's default
1051 | device. This will be sufficient for many use cases, but if you need more flexibility you'll want to
1052 | configure the engine with an engine config:
1053 |
1054 |     ```c
1055 |     ma_result result;
1056 |     ma_engine engine;
1057 |     ma_engine_config engineConfig;
1058 |
1059 |     engineConfig = ma_engine_config_init();
1060 |     engineConfig.pDevice = &myDevice;
1061 |
1062 |     result = ma_engine_init(&engineConfig, &engine);
1063 |     if (result != MA_SUCCESS) {
1064 |         return result; // Failed to initialize the engine.
1065 |     }
1066 |     ...
1067 |

```

```

1068 | In the example above we're passing in a pre-initialized device. Since the caller is the one in
1069 | control of the device's data callback, it's their responsibility to manually call
1070 | `ma_engine_read_pcm_frames()` from inside their data callback:
1071 |
1072 |     ```c
1073 |     void playback_data_callback(ma_device* pDevice, void* pOutput, const void* pInput, ma_uint32
> frameCount)
1074 |     {
1075 |         ma_engine_read_pcm_frames(&g_Engine, pOutput, frameCount, NULL);
1076 |     }
1077 |     ```
1078 |
1079 | You can also use the engine independent of a device entirely:
1080 |
1081 |     ```c
1082 |     ma_result result;
1083 |     ma_engine engine;
1084 |     ma_engine_config engineConfig;
1085 |
1086 |     engineConfig = ma_engine_config_init();
1087 |     engineConfig.noDevice = MA_TRUE;
1088 |     engineConfig.channels = 2;           // Must be set when not using a device.
1089 |     engineConfig.sampleRate = 48000;    // Must be set when not using a device.
1090 |
1091 |     result = ma_engine_init(&engineConfig, &engine);
1092 |     if (result != MA_SUCCESS) {
1093 |         return result; // Failed to initialize the engine.
1094 |     }
1095 |     ```
1096 |
1097 | Note that when you're not using a device, you must set the channel count and sample rate in the
1098 | config or else miniaudio won't know what to use (miniaudio will use the device to determine this
1099 | normally). When not using a device, you need to use `ma_engine_read_pcm_frames()` to process audio
1100 | data from the engine. This kind of setup is useful if you want to do something like offline
1101 | processing or want to use a different audio system for playback such as SDL.
1102 |
1103 | When a sound is loaded it goes through a resource manager. By default the engine will initialize a
1104 | resource manager internally, but you can also specify a pre-initialized resource manager:
1105 |
1106 |     ```c
1107 |     ma_result result;
1108 |     ma_engine engine1;
1109 |     ma_engine engine2;
1110 |     ma_engine_config engineConfig;
1111 |
1112 |     engineConfig = ma_engine_config_init();
1113 |     engineConfig.pResourceManager = &myResourceManager;
1114 |
1115 |     ma_engine_init(&engineConfig, &engine1);
1116 |     ma_engine_init(&engineConfig, &engine2);
1117 |     ```
1118 |
1119 | In this example we are initializing two engines, both of which are sharing the same resource
1120 | manager. This is especially useful for saving memory when loading the same file across multiple
1121 | engines. If you were not to use a shared resource manager, each engine instance would use their own
1122 | which would result in any sounds that are used between both engine's being loaded twice. By using
1123 | a shared resource manager, it would only be loaded once. Using multiple engine's is useful when you
1124 | need to output to multiple playback devices, such as in a local multiplayer game where each player
1125 | is using their own set of headphones.
1126 |
1127 | By default an engine will be in a started state. To make it so the engine is not automatically
1128 | started you can configure it as such:
1129 |
1130 |     ```c
1131 |     engineConfig.noAutoStart = MA_TRUE;
1132 |
1133 |     // The engine will need to be started manually.
1134 |     ma_engine_start(&engine);
1135 |
1136 |     // Later on the engine can be stopped with ma_engine_stop().
1137 |     ma_engine_stop(&engine);
1138 |     ```
1139 |
1140 | The concept of starting or stopping an engine is only relevant when using the engine with a
1141 | device. Attempting to start or stop an engine that is not associated with a device will result in
1142 | `MA_INVALID_OPERATION`.
1143 |
1144 | The master volume of the engine can be controlled with `ma_engine_set_volume()` which takes a
1145 | linear scale, with 0 resulting in silence and anything above 1 resulting in amplification. If you

```



```

1146 | prefer decibel based volume control, use `ma_volume_db_to_linear()` to convert from dB to linear.
1147 |
1148 | When a sound is spatialized, it is done so relative to a listener. An engine can be configured to
1149 | have multiple listeners which can be configured via the config:
1150 |
1151 |     ```c
1152 |     engineConfig.listenerCount = 2;
1153 |     ```
1154 |
1155 | The maximum number of listeners is restricted to `MA_ENGINE_MAX_LISTENERS`. By default, when a
1156 | sound is spatialized, it will be done so relative to the closest listener. You can also pin a sound
1157 | to a specific listener which will be explained later. Listener's have a position, direction, cone,
1158 | and velocity (for doppler effect). A listener is referenced by an index, the meaning of which is up
1159 | to the caller (the index is 0 based and cannot go beyond the listener count, minus 1). The
1160 | position, direction and velocity are all specified in absolute terms:
1161 |
1162 |     ```c
1163 |     ma_engine_listener_set_position(&engine, listenerIndex, worldPosX, worldPosY, worldPosZ);
1164 |     ```
1165 |
1166 | The direction of the listener represents it's forward vector. The listener's up vector can also be
1167 | specified and defaults to +1 on the Y axis.
1168 |
1169 |     ```c
1170 |     ma_engine_listener_set_direction(&engine, listenerIndex, forwardX, forwardY, forwardZ);
1171 |     ma_engine_listener_set_world_up(&engine, listenerIndex, 0, 1, 0);
1172 |     ```
1173 |
1174 | The engine supports directional attenuation. The listener can have a cone the controls how sound is
1175 | attenuated based on the listener's direction. When a sound is between the inner and outer cones, it
1176 | will be attenuated between 1 and the cone's outer gain:
1177 |
1178 |     ```c
1179 |     ma_engine_listener_set_cone(&engine, listenerIndex, innerAngleInRadians, outerAngleInRadians,
> outerGain);
1180 |     ```
1181 |
1182 | When a sound is inside the inner cone, no directional attenuation is applied. When the sound is
1183 | outside of the outer cone, the attenuation will be set to `outerGain` in the example above. When
1184 | the sound is in between the inner and outer cones, the attenuation will be interpolated between 1
1185 | and the outer gain.
1186 |
1187 | The engine's coordinate system follows the OpenGL coordinate system where positive X points right,
1188 | positive Y points up and negative Z points forward.
1189 |
1190 | The simplest and least flexible way to play a sound is like so:
1191 |
1192 |     ```c
1193 |     ma_engine_play_sound(&engine, "my_sound.wav", pGroup);
1194 |     ```
1195 |
1196 | This is a "fire and forget" style of function. The engine will manage the `ma_sound` object
1197 | internally. When the sound finishes playing, it'll be put up for recycling. For more flexibility
1198 | you'll want to initialize a sound object:
1199 |
1200 |     ```c
1201 |     ma_sound sound;
1202 |
1203 |     result = ma_sound_init_from_file(&engine, "my_sound.wav", flags, pGroup, NULL, &sound);
1204 |     if (result != MA_SUCCESS) {
1205 |         return result; // Failed to load sound.
1206 |     }
1207 |     ```
1208 |
1209 | Sounds need to be uninitialized with `ma_sound_uninit()`.
1210 |
1211 | The example above loads a sound from a file. If the resource manager has been disabled you will not
1212 | be able to use this function and instead you'll need to initialize a sound directly from a data
1213 | source:
1214 |
1215 |     ```c
1216 |     ma_sound sound;
1217 |
1218 |     result = ma_sound_init_from_data_source(&engine, &dataSource, flags, pGroup, &sound);
1219 |     if (result != MA_SUCCESS) {
1220 |         return result;
1221 |     }
1222 |     ```
1223 |

```

```

1224 | Each `ma_sound` object represents a single instance of the sound. If you want to play the same
1225 | sound multiple times at the same time, you need to initialize a separate `ma_sound` object.
1226 |
1227 | For the most flexibility when initializing sounds, use `ma_sound_init_ex()`. This uses miniaudio's
1228 | standard config/init pattern:
1229 |
1230 |     ```c
1231 |     ma_sound sound;
1232 |     ma_sound_config soundConfig;
1233 |
1234 |     soundConfig = ma_sound_config_init();
1235 |     soundConfig.pFilePath = NULL; // Set this to load from a file path.
1236 |     soundConfig.pDataSource = NULL; // Set this to initialize from an existing data source.
1237 |     soundConfig.pInitialAttachment = &someNodeInTheNodeGraph;
1238 |     soundConfig.initialAttachmentInputBusIndex = 0;
1239 |     soundConfig.channelsIn = 1;
1240 |     soundConfig.channelsOut = 0; // Set to 0 to use the engine's native channel count.
1241 |
1242 |     result = ma_sound_init_ex(&soundConfig, &sound);
1243 |     if (result != MA_SUCCESS) {
1244 |         return result;
1245 |     }
1246 |     ```
1247 |
1248 | In the example above, the sound is being initialized without a file nor a data source. This is
1249 | valid, in which case the sound acts as a node in the middle of the node graph. This means you can
1250 | connect other sounds to this sound and allow it to act like a sound group. Indeed, this is exactly
1251 | what a `ma_sound_group` is.
1252 |
1253 | When loading a sound, you specify a set of flags that control how the sound is loaded and what
1254 | features are enabled for that sound. When no flags are set, the sound will be fully loaded into
1255 | memory in exactly the same format as how it's stored on the file system. The resource manager will
1256 | allocate a block of memory and then load the file directly into it. When reading audio data, it
1257 | will be decoded dynamically on the fly. In order to save processing time on the audio thread, it
1258 | might be beneficial to pre-decode the sound. You can do this with the `MA_SOUND_FLAG_DECODE` flag:
1259 |
1260 |     ```c
1261 |     ma_sound_init_from_file(&engine, "my_sound.wav", MA_SOUND_FLAG_DECODE, pGroup, NULL, &sound);
1262 |     ```
1263 |
1264 | By default, sounds will be loaded synchronously, meaning `ma_sound_init_*()` will not return until
1265 | the sound has been fully loaded. If this is prohibitive you can instead load sounds asynchronously
1266 | by specifying the `MA_SOUND_FLAG_ASYNC` flag:
1267 |
1268 |     ```c
1269 |     ma_sound_init_from_file(&engine, "my_sound.wav", MA_SOUND_FLAG_DECODE | MA_SOUND_FLAG_ASYNC,
> pGroup, NULL, &sound);
1270 |     ```
1271 |
1272 | This will result in `ma_sound_init_*()` returning quickly, but the sound won't yet have been fully
1273 | loaded. When you start the sound, it won't output anything until some sound is available. The sound
1274 | will start outputting audio before the sound has been fully decoded when the `MA_SOUND_FLAG_DECODE`
1275 | is specified.
1276 |
1277 | If you need to wait for an asynchronously loaded sound to be fully loaded, you can use a fence. A
1278 | fence in miniaudio is a simple synchronization mechanism which simply blocks until it's internal
1279 | counter hit's zero. You can specify a fence like so:
1280 |
1281 |     ```c
1282 |     ma_result result;
1283 |     ma_fence fence;
1284 |     ma_sound sounds[4];
1285 |
1286 |     result = ma_fence_init(&fence);
1287 |     if (result != MA_SUCCESS) {
1288 |         return result;
1289 |     }
1290 |
1291 |     // Load some sounds asynchronously.
1292 |     for (int iSound = 0; iSound < 4; iSound += 1) {
1293 |         ma_sound_init_from_file(&engine, mySoundFilesPaths[iSound], MA_SOUND_FLAG_DECODE |
> MA_SOUND_FLAG_ASYNC, pGroup, &fence, &sounds[iSound]);
1294 |     }
1295 |
1296 |     // ... do some other stuff here in the mean time ...
1297 |
1298 |     // Wait for all sounds to finish loading.
1299 |     ma_fence_wait(&fence);
1300 |     ```

```

```

1301 |
1302 | If loading the entire sound into memory is prohibitive, you can also configure the engine to stream
1303 | the audio data:
1304 |
1305 |     ```c
1306 |     ma_sound_init_from_file(&engine, "my_sound.wav", MA_SOUND_FLAG_STREAM, pGroup, NULL, &sound);
1307 |     ```
1308 |
1309 | When streaming sounds, 2 seconds worth of audio data is stored in memory. Although it should work
1310 | fine, it's inefficient to use streaming for short sounds. Streaming is useful for things like music
1311 | tracks in games.
1312 |
1313 | When loading a sound from a file path, the engine will reference count the file to prevent it from
1314 | being loaded if it's already in memory. When you uninitialize a sound, the reference count will be
1315 | decremented, and if it hits zero, the sound will be unloaded from memory. This reference counting
1316 | system is not used for streams. The engine will use a 64-bit hash of the file name when comparing
1317 | file paths which means there's a small chance you might encounter a name collision. If this is an
1318 | issue, you'll need to use a different name for one of the colliding file paths, or just not load
1319 | from files and instead load from a data source.
1320 |
1321 | You can use `ma_sound_init_copy()` to initialize a copy of another sound. Note, however, that this
1322 | only works for sounds that were initialized with `ma_sound_init_from_file()` and without the
1323 | `MA_SOUND_FLAG_STREAM` flag.
1324 |
1325 | When you initialize a sound, if you specify a sound group the sound will be attached to that group
1326 | automatically. If you set it to NULL, it will be automatically attached to the engine's endpoint.
1327 | If you would instead rather leave the sound unattached by default, you can specify the
1328 | `MA_SOUND_FLAG_NO_DEFAULT_ATTACHMENT` flag. This is useful if you want to set up a complex node
1329 | graph.
1330 |
1331 | Sounds are not started by default. To start a sound, use `ma_sound_start()`. Stop a sound with
1332 | `ma_sound_stop()`.
1333 |
1334 | Sounds can have their volume controlled with `ma_sound_set_volume()` in the same way as the
1335 | engine's master volume.
1336 |
1337 | Sounds support stereo panning and pitching. Set the pan with `ma_sound_set_pan()`. Setting the pan
1338 | to 0 will result in an unpanned sound. Setting it to -1 will shift everything to the left, whereas
1339 | +1 will shift it to the right. The pitch can be controlled with `ma_sound_set_pitch()`. A larger
1340 | value will result in a higher pitch. The pitch must be greater than 0.
1341 |
1342 | The engine supports 3D spatialization of sounds. By default sounds will have spatialization
1343 | enabled, but if a sound does not need to be spatialized it's best to disable it. There are two ways
1344 | to disable spatialization of a sound:
1345 |
1346 |     ```c
1347 |     // Disable spatialization at initialization time via a flag:
1348 |     ma_sound_init_from_file(&engine, "my_sound.wav", MA_SOUND_FLAG_NO_SPATIALIZATION, NULL, NULL,
1349 | > &sound);
1350 |
1351 |     // Dynamically disable or enable spatialization post-initialization:
1352 |     ma_sound_set_spatialization_enabled(&sound, isSpatializationEnabled);
1353 |     ```
1354 |
1355 | By default sounds will be spatialized based on the closest listener. If a sound should always be
1356 | spatialized relative to a specific listener it can be pinned to one:
1357 |
1358 |     ```c
1359 |     ma_sound_set_pinned_listener_index(&sound, listenerIndex);
1360 |     ```
1361 |
1362 | Like listeners, sounds have a position. By default, the position of a sound is in absolute space,
1363 | but it can be changed to be relative to a listener:
1364 |
1365 |     ```c
1366 |     ma_sound_set_positioning(&sound, ma_positioning_relative);
1367 |     ```
1368 |
1369 | Note that relative positioning of a sound only makes sense if there is either only one listener, or
1370 | the sound is pinned to a specific listener. To set the position of a sound:
1371 |
1372 |     ```c
1373 |     ma_sound_set_position(&sound, posX, posY, posZ);
1374 |     ```
1375 |
1376 | The direction works the same way as a listener and represents the sound's forward direction:
1377 |
1378 |     ```c
1379 |     ma_sound_set_direction(&sound, forwardX, forwardY, forwardZ);

```

```

1379 |     ``
1380 |
1381 | Sound's also have a cone for controlling directional attenuation. This works exactly the same as
1382 | listeners:
1383 |
1384 |     ``c
1385 |     ma_sound_set_cone(&sound, innerAngleInRadians, outerAngleInRadians, outerGain);
1386 |     ``
1387 |
1388 | The velocity of a sound is used for doppler effect and can be set as such:
1389 |
1390 |     ``c
1391 |     ma_sound_set_velocity(&sound, velocityX, velocityY, velocityZ);
1392 |     ``
1393 |
1394 | The engine supports different attenuation models which can be configured on a per-sound basis. By
1395 | default the attenuation model is set to `ma_attenuation_model_inverse` which is the equivalent to
1396 | OpenAL's `AL_INVERSE_DISTANCE_CLAMPED`. Configure the attenuation model like so:
1397 |
1398 |     ``c
1399 |     ma_sound_set_attenuation_model(&sound, ma_attenuation_model_inverse);
1400 |     ``
1401 |
1402 | The supported attenuation models include the following:
1403 |
1404 | +-----+-----+-----+
1405 | | ma_attenuation_model_none          | No distance attenuation.          |
1406 | +-----+-----+-----+
1407 | | ma_attenuation_model_inverse       | Equivalent to `AL_INVERSE_DISTANCE_CLAMPED`. |
1408 | +-----+-----+-----+
1409 | | ma_attenuation_model_linear        | Linear attenuation.                |
1410 | +-----+-----+-----+
1411 | | ma_attenuation_model_exponential  | Exponential attenuation.           |
1412 | +-----+-----+-----+
1413 |
1414 | To control how quickly a sound rolls off as it moves away from the listener, you need to configure
1415 | the rolloff:
1416 |
1417 |     ``c
1418 |     ma_sound_set_rolloff(&sound, rolloff);
1419 |     ``
1420 |
1421 | You can control the minimum and maximum gain to apply from spatialization:
1422 |
1423 |     ``c
1424 |     ma_sound_set_min_gain(&sound, minGain);
1425 |     ma_sound_set_max_gain(&sound, maxGain);
1426 |     ``
1427 |
1428 | Likewise, in the calculation of attenuation, you can control the minimum and maximum distances for
1429 | the attenuation calculation. This is useful if you want to ensure sounds don't drop below a certain
1430 | volume after the listener moves further away and to have sounds play a maximum volume when the
1431 | listener is within a certain distance:
1432 |
1433 |     ``c
1434 |     ma_sound_set_min_distance(&sound, minDistance);
1435 |     ma_sound_set_max_distance(&sound, maxDistance);
1436 |     ``
1437 |
1438 | The engine's spatialization system supports doppler effect. The doppler factor can be configure on
1439 | a per-sound basis like so:
1440 |
1441 |     ``c
1442 |     ma_sound_set_doppler_factor(&sound, dopplerFactor);
1443 |     ``
1444 |
1445 | You can fade sounds in and out with `ma_sound_set_fade_in_pcm_frames()` and
1446 | `ma_sound_set_fade_in_milliseconds()`. Set the volume to -1 to use the current volume as the
1447 | starting volume:
1448 |
1449 |     ``c
1450 |     // Fade in over 1 second.
1451 |     ma_sound_set_fade_in_milliseconds(&sound, 0, 1, 1000);
1452 |
1453 |     // ... sometime later ...
1454 |
1455 |     // Fade out over 1 second, starting from the current volume.
1456 |     ma_sound_set_fade_in_milliseconds(&sound, -1, 0, 1000);
1457 |     ``

```

```

1458 |
1459 | By default sounds will start immediately, but sometimes for timing and synchronization purposes it
1460 | can be useful to schedule a sound to start or stop:
1461 |
1462 |     ``c
1463 |     // Start the sound in 1 second from now.
1464 |     ma_sound_set_start_time_in_pcm_frames(&sound, ma_engine_get_time_in_pcm_frames(&engine) +
> (ma_engine_get_sample_rate(&engine) * 1));
1465 |
1466 |     // Stop the sound in 2 seconds from now.
1467 |     ma_sound_set_stop_time_in_pcm_frames(&sound, ma_engine_get_time_in_pcm_frames(&engine) +
> (ma_engine_get_sample_rate(&engine) * 2));
1468 |     ``c
1469 |
1470 | Note that scheduling a start time still requires an explicit call to `ma_sound_start()` before
1471 | anything will play.
1472 |
1473 | The time is specified in global time which is controlled by the engine. You can get the engine's
1474 | current time with `ma_engine_get_time_in_pcm_frames()`. The engine's global time is incremented
1475 | automatically as audio data is read, but it can be reset with `ma_engine_set_time_in_pcm_frames()`
1476 | in case it needs to be resynchronized for some reason.
1477 |
1478 | To determine whether or not a sound is currently playing, use `ma_sound_is_playing()`. This will
1479 | take the scheduled start and stop times into account.
1480 |
1481 | Whether or not a sound should loop can be controlled with `ma_sound_set_looping()`. Sounds will not
1482 | be looping by default. Use `ma_sound_is_looping()` to determine whether or not a sound is looping.
1483 |
1484 | Use `ma_sound_at_end()` to determine whether or not a sound is currently at the end. For a looping
1485 | sound this should never return true. Alternatively, you can configure a callback that will be fired
1486 | when the sound reaches the end. Note that the callback is fired from the audio thread which means
1487 | you cannot be uninitializing sound from the callback. To set the callback you can use
1488 | `ma_sound_set_end_callback()`. Alternatively, if you're using `ma_sound_init_ex()`, you can pass it
1489 | into the config like so:
1490 |
1491 |     ``c
1492 |     soundConfig.endCallback = my_end_callback;
1493 |     soundConfig.pEndCallbackUserData = pMyEndCallbackUserData;
1494 |     ``c
1495 |
1496 | The end callback is declared like so:
1497 |
1498 |     ``c
1499 |     void my_end_callback(void* pUserData, ma_sound* pSound)
1500 |     {
1501 |         ...
1502 |     }
1503 |     ``c
1504 |
1505 | Internally a sound wraps around a data source. Some APIs exist to control the underlying data
1506 | source, mainly for convenience:
1507 |
1508 |     ``c
1509 |     ma_sound_seek_to_pcm_frame(&sound, frameIndex);
1510 |     ma_sound_get_data_format(&sound, &format, &channels, &sampleRate, pChannelMap,
> channelMapCapacity);
1511 |     ma_sound_get_cursor_in_pcm_frames(&sound, &cursor);
1512 |     ma_sound_get_length_in_pcm_frames(&sound, &length);
1513 |     ``c
1514 |
1515 | Sound groups have the same API as sounds, only they are called `ma_sound_group`, and since they do
1516 | not have any notion of a data source, anything relating to a data source is unavailable.
1517 |
1518 | Internally, sound data is loaded via the `ma_decoder` API which means by default it only supports
1519 | file formats that have built-in support in miniaudio. You can extend this to support any kind of
1520 | file format through the use of custom decoders. To do this you'll need to use a self-managed
1521 | resource manager and configure it appropriately. See the "Resource Management" section below for
1522 | details on how to set this up.
1523 |
1524 |
1525 | 6. Resource Management
1526 | =====
1527 | Many programs will want to manage sound resources for things such as reference counting and
1528 | streaming. This is supported by miniaudio via the `ma_resource_manager` API.
1529 |
1530 | The resource manager is mainly responsible for the following:
1531 |
1532 | * Loading of sound files into memory with reference counting.
1533 | * Streaming of sound data.

```

```

1534 |
1535 | When loading a sound file, the resource manager will give you back a `ma_data_source` compatible
1536 | object called `ma_resource_manager_data_source`. This object can be passed into any
1537 | `ma_data_source` API which is how you can read and seek audio data. When loading a sound file, you
1538 | specify whether or not you want the sound to be fully loaded into memory (and optionally
1539 | pre-decoded) or streamed. When loading into memory, you can also specify whether or not you want
1540 | the data to be loaded asynchronously.
1541 |
1542 | The example below is how you can initialize a resource manager using it's default configuration:
1543 |
1544 | ```c
1545 |     ma_resource_manager_config config;
1546 |     ma_resource_manager resourceManager;
1547 |
1548 |     config = ma_resource_manager_config_init();
1549 |     result = ma_resource_manager_init(&config, &resourceManager);
1550 |     if (result != MA_SUCCESS) {
1551 |         ma_device_uninit(&device);
1552 |         printf("Failed to initialize the resource manager.");
1553 |         return -1;
1554 |     }
1555 | ```
1556 |
1557 | You can configure the format, channels and sample rate of the decoded audio data. By default it
1558 | will use the file's native data format, but you can configure it to use a consistent format. This
1559 | is useful for offloading the cost of data conversion to load time rather than dynamically
1560 | converting at mixing time. To do this, you configure the decoded format, channels and sample rate
1561 | like the code below:
1562 |
1563 | ```c
1564 |     config = ma_resource_manager_config_init();
1565 |     config.decodedFormat = device.playback.format;
1566 |     config.decodedChannels = device.playback.channels;
1567 |     config.decodedSampleRate = device.sampleRate;
1568 | ```
1569 |
1570 | In the code above, the resource manager will be configured so that any decoded audio data will be
1571 | pre-converted at load time to the device's native data format. If instead you used defaults and
1572 | the data format of the file did not match the device's data format, you would need to convert the
1573 | data at mixing time which may be prohibitive in high-performance and large scale scenarios like
1574 | games.
1575 |
1576 | Internally the resource manager uses the `ma_decoder` API to load sounds. This means by default it
1577 | only supports decoders that are built into miniaudio. It's possible to support additional encoding
1578 | formats through the use of custom decoders. To do so, pass in your `ma_decoding_backend_vtable`
1579 | vtables into the resource manager config:
1580 |
1581 | ```c
1582 |     ma_decoding_backend_vtable* pCustomBackendVTables[] =
1583 |     {
1584 |         &g_ma_decoding_backend_vtable_libvorbis,
1585 |         &g_ma_decoding_backend_vtable_libopus
1586 |     };
1587 |
1588 |     ...
1589 |
1590 |     resourceManagerConfig.ppCustomDecodingBackendVTables = pCustomBackendVTables;
1591 |     resourceManagerConfig.customDecodingBackendCount = sizeof(pCustomBackendVTables) /
1592 | > sizeof(pCustomBackendVTables[0]);
1593 |     resourceManagerConfig.pCustomDecodingBackendUserData = NULL;
1594 | ```
1595 |
1596 | This system can allow you to support any kind of file format. See the "Decoding" section for
1597 | details on how to implement custom decoders. The miniaudio repository includes examples for Opus
1598 | via libopus and libopusfile and Vorbis via libvorbis and libvorbisfile.
1599 |
1600 | Asynchronicity is achieved via a job system. When an operation needs to be performed, such as the
1601 | decoding of a page, a job will be posted to a queue which will then be processed by a job thread.
1602 | By default there will be only one job thread running, but this can be configured, like so:
1603 |
1604 | ```c
1605 |     config = ma_resource_manager_config_init();
1606 |     config.jobThreadCount = MY_JOB_THREAD_COUNT;
1607 | ```
1608 |
1609 | By default job threads are managed internally by the resource manager, however you can also self
1610 | manage your job threads if, for example, you want to integrate the job processing into your
1611 | existing job infrastructure, or if you simply don't like the way the resource manager does it. To
    do this, just set the job thread count to 0 and process jobs manually. To process jobs, you first

```

```

1612 | need to retrieve a job using `ma_resource_manager_next_job()` and then process it using
1613 | `ma_job_process()`:
1614 |
1615 |     ``c
1616 |         config = ma_resource_manager_config_init();
1617 |         config.jobThreadCount = 0;                                // Don't manage any job threads
> internally.
1618 |         config.flags = MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING; // Optional. Makes
> `ma_resource_manager_next_job()` non-blocking.
1619 |
1620 |         // ... Initialize your custom job threads ...
1621 |
1622 |         void my_custom_job_thread(...)
1623 |         {
1624 |             for (;;) {
1625 |                 ma_job job;
1626 |                 ma_result result = ma_resource_manager_next_job(pMyResourceManager, &job);
1627 |                 if (result != MA_SUCCESS) {
1628 |                     if (result == MA_NO_DATA_AVAILABLE) {
1629 |                         // No jobs are available. Keep going. Will only get this if the resource manager
> was initialized
1630 |                             // with MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING.
1631 |                             continue;
1632 |                         } else if (result == MA_CANCELLED) {
1633 |                             // MA_JOB_TYPE_QUIT was posted. Exit.
1634 |                             break;
1635 |                         } else {
1636 |                             // Some other error occurred.
1637 |                             break;
1638 |                         }
1639 |                     }
1640 |
1641 |                     ma_job_process(&job);
1642 |                 }
1643 |             }
1644 |         }
1645 |
1646 | In the example above, the `MA_JOB_TYPE_QUIT` event is the used as the termination
1647 | indicator, but you can use whatever you would like to terminate the thread. The call to
1648 | `ma_resource_manager_next_job()` is blocking by default, but can be configured to be non-blocking
1649 | by initializing the resource manager with the `MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING` configuration
1650 | flag. Note that the `MA_JOB_TYPE_QUIT` will never be removed from the job queue. This
1651 | is to give every thread the opportunity to catch the event and terminate naturally.
1652 |
1653 | When loading a file, it's sometimes convenient to be able to customize how files are opened and
1654 | read instead of using standard `fopen()`, `fclose()`, etc. which is what miniaudio will use by
1655 | default. This can be done by setting `pVFS` member of the resource manager's config:
1656 |
1657 |     ``c
1658 |         // Initialize your custom VFS object. See documentation for VFS for information on how to do
> this.
1659 |         my_custom_vfs vfs = my_custom_vfs_init();
1660 |
1661 |         config = ma_resource_manager_config_init();
1662 |         config.pVFS = &vfs;
1663 |     ``c
1664 |
1665 | This is particularly useful in programs like games where you want to read straight from an archive
1666 | rather than the normal file system. If you do not specify a custom VFS, the resource manager will
1667 | use the operating system's normal file operations.
1668 |
1669 | To load a sound file and create a data source, call `ma_resource_manager_data_source_init()`. When
1670 | loading a sound you need to specify the file path and options for how the sounds should be loaded.
1671 | By default a sound will be loaded synchronously. The returned data source is owned by the caller
1672 | which means the caller is responsible for the allocation and freeing of the data source. Below is
1673 | an example for initializing a data source:
1674 |
1675 |     ``c
1676 |         ma_resource_manager_data_source dataSource;
1677 |         ma_result result = ma_resource_manager_data_source_init(pResourceManager, pFilePath, flags,
> &dataSource);
1678 |         if (result != MA_SUCCESS) {
1679 |             // Error.
1680 |         }
1681 |
1682 |         // ...
1683 |
1684 |         // A ma_resource_manager_data_source object is compatible with the `ma_data_source` API. To read
> data, just call

```



```

1685 | // the `ma_data_source_read_pcm_frames()` like you would with any normal data source.
1686 | result = ma_data_source_read_pcm_frames(&dataSource, pDecodedData, frameCount, &framesRead);
1687 | if (result != MA_SUCCESS) {
1688 |     // Failed to read PCM frames.
1689 | }
1690 |
1691 | // ...
1692 |
1693 | ma_resource_manager_data_source_uninit(&dataSource);
1694 | ```
1695 |
1696 | The `flags` parameter specifies how you want to perform loading of the sound file. It can be a
1697 | combination of the following flags:
1698 |
1699 | ```
1700 | MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM
1701 | MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE
1702 | MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC
1703 | MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT
1704 | MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING
1705 | ```
1706 |
1707 | When no flags are specified (set to 0), the sound will be fully loaded into memory, but not
1708 | decoded, meaning the raw file data will be stored in memory, and then dynamically decoded when
1709 | `ma_data_source_read_pcm_frames()` is called. To instead decode the audio data before storing it in
1710 | memory, use the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE` flag. By default, the sound file will
1711 | be loaded synchronously, meaning `ma_resource_manager_data_source_init()` will only return after
1712 | the entire file has been loaded. This is good for simplicity, but can be prohibitively slow. You
1713 | can instead load the sound asynchronously using the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC` flag.
1714 | This will result in `ma_resource_manager_data_source_init()` returning quickly, but no data will be
1715 | returned by `ma_data_source_read_pcm_frames()` until some data is available. When no data is
1716 | available because the asynchronous decoding hasn't caught up, `MA_BUSY` will be returned by
1717 | `ma_data_source_read_pcm_frames()`.
1718 |
1719 | For large sounds, it's often prohibitive to store the entire file in memory. To mitigate this, you
1720 | can instead stream audio data which you can do by specifying the
1721 | `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM` flag. When streaming, data will be decoded in 1
1722 | second pages. When a new page needs to be decoded, a job will be posted to the job queue and then
1723 | subsequently processed in a job thread.
1724 |
1725 | The `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING` flag can be used so that the sound will loop
1726 | when it reaches the end by default. It's recommended you use this flag when you want to have a
1727 | looping streaming sound. If you try loading a very short sound as a stream, you will get a glitch.
1728 | This is because the resource manager needs to pre-fill the initial buffer at initialization time,
1729 | and if you don't specify the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING` flag, the resource
1730 | manager will assume the sound is not looping and will stop filling the buffer when it reaches the
1731 | end, therefore resulting in a discontinuous buffer.
1732 |
1733 | For in-memory sounds, reference counting is used to ensure the data is loaded only once. This means
1734 | multiple calls to `ma_resource_manager_data_source_init()` with the same file path will result in
1735 | the file data only being loaded once. Each call to `ma_resource_manager_data_source_init()` must be
1736 | matched up with a call to `ma_resource_manager_data_source_uninit()`. Sometimes it can be useful
1737 | for a program to register self-managed raw audio data and associate it with a file path. Use the
1738 | `ma_resource_manager_register_*(())` and `ma_resource_manager_unregister_*(())` APIs to do this.
1739 | `ma_resource_manager_register_decoded_data()` is used to associate a pointer to raw, self-managed
1740 | decoded audio data in the specified data format with the specified name. Likewise,
1741 | `ma_resource_manager_register_encoded_data()` is used to associate a pointer to raw self-managed
1742 | encoded audio data (the raw file data) with the specified name. Note that these names need not be
1743 | actual file paths. When `ma_resource_manager_data_source_init()` is called (without the
1744 | `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM` flag), the resource manager will look for these
1745 | explicitly registered data buffers and, if found, will use it as the backing data for the data
1746 | source. Note that the resource manager does *not* make a copy of this data so it is up to the
1747 | caller to ensure the pointer stays valid for its lifetime. Use
1748 | `ma_resource_manager_unregister_data()` to unregister the self-managed data. You can also use
1749 | `ma_resource_manager_register_file()` and `ma_resource_manager_unregister_file()` to register and
1750 | unregister a file. It does not make sense to use the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM`
1751 | flag with a self-managed data pointer.
1752 |
1753 |
1754 | 6.1. Asynchronous Loading and Synchronization
1755 | -----
1756 | When loading asynchronously, it can be useful to poll whether or not loading has finished. Use
1757 | `ma_resource_manager_data_source_result()` to determine this. For in-memory sounds, this will
1758 | return `MA_SUCCESS` when the file has been *entirely* decoded. If the sound is still being decoded,
1759 | `MA_BUSY` will be returned. Otherwise, some other error code will be returned if the sound failed
1760 | to load. For streaming data sources, `MA_SUCCESS` will be returned when the first page has been
1761 | decoded and the sound is ready to be played. If the first page is still being decoded, `MA_BUSY`
1762 | will be returned. Otherwise, some other error code will be returned if the sound failed to load.
1763 |

```

1764 | In addition to polling, you can also use a simple synchronization object called a "fence" to wait
 1765 | for asynchronously loaded sounds to finish. This is called `ma_fence`. The advantage to using a
 1766 | fence is that it can be used to wait for a group of sounds to finish loading rather than waiting
 1767 | for sounds on an individual basis. There are two stages to loading a sound:

- 1768 |
- 1769 | * Initialization of the internal decoder; and
- 1770 | * Completion of decoding of the file (the file is fully decoded)
- 1771 |

1772 | You can specify separate fences for each of the different stages. Waiting for the initialization
 1773 | of the internal decoder is important for when you need to know the sample format, channels and
 1774 | sample rate of the file.

1775 |

1776 | The example below shows how you could use a fence when loading a number of sounds:

```
1777 |     ``c
1778 |     // This fence will be released when all sounds are finished loading entirely.
1779 |     ma_fence fence;
1780 |     ma_fence_init(&fence);
1781 |
1782 |
1783 |     // This will be passed into the initialization routine for each sound.
1784 |     ma_resource_manager_pipeline_notifications notifications =
> ma_resource_manager_pipeline_notifications_init();
1785 |     notifications.done.pFence = &fence;
1786 |
1787 |     // Now load a bunch of sounds:
1788 |     for (iSound = 0; iSound < soundCount; iSound += 1) {
1789 |         ma_resource_manager_data_source_init(pResourceManager, pSoundFilePaths[iSound], flags,
> &notifications, &pSoundSources[iSound]);
1790 |     }
1791 |
1792 |     // ... DO SOMETHING ELSE WHILE SOUNDS ARE LOADING ...
1793 |
1794 |     // Wait for loading of sounds to finish.
1795 |     ma_fence_wait(&fence);
1796 |     ``
```

1797 |

1798 | In the example above we used a fence for waiting until the entire file has been fully decoded. If
 1799 | you only need to wait for the initialization of the internal decoder to complete, you can use the
 1800 | `init` member of the `ma_resource_manager_pipeline_notifications` object:

```
1801 |     ``c
1802 |     notifications.init.pFence = &fence;
1803 |     ``
```

1804 |

1805 |

1806 | If a fence is not appropriate for your situation, you can instead use a callback that is fired on
 1807 | an individual sound basis. This is done in a very similar way to fences:

```
1808 |     ``c
1809 |     typedef struct
1810 |     {
1811 |         ma_async_notification_callbacks cb;
1812 |         void* pMyData;
1813 |     } my_notification;
1814 |
1815 |
1816 |     void my_notification_callback(ma_async_notification* pNotification)
1817 |     {
1818 |         my_notification* pMyNotification = (my_notification*)pNotification;
1819 |
1820 |         // Do something in response to the sound finishing loading.
1821 |     }
1822 |
1823 |     ...
1824 |
1825 |     my_notification myCallback;
1826 |     myCallback.cb.onSignal = my_notification_callback;
1827 |     myCallback.pMyData = pMyData;
1828 |
1829 |     ma_resource_manager_pipeline_notifications notifications =
> ma_resource_manager_pipeline_notifications_init();
1830 |     notifications.done.pNotification = &myCallback;
1831 |
1832 |     ma_resource_manager_data_source_init(pResourceManager, "my_sound.wav", flags, &notifications,
> &mySound);
1833 |     ``
```

1834 |

1835 | In the example above we just extend the `ma_async_notification_callbacks` object and pass an
 1836 | instantiation into the `ma_resource_manager_pipeline_notifications` in the same way as we did with
 1837 | the fence, only we set `pNotification` instead of `pFence`. You can set both of these at the same
 1838 | time and they should both work as expected. If using the `pNotification` system, you need to ensure

```

1839 | your `ma_async_notification_callbacks` object stays valid.
1840 |
1841 |
1842 |
1843 | 6.2. Resource Manager Implementation Details
1844 | -----
1845 | Resources are managed in two main ways:
1846 |
1847 | * By storing the entire sound inside an in-memory buffer (referred to as a data buffer)
1848 | * By streaming audio data on the fly (referred to as a data stream)
1849 |
1850 | A resource managed data source (`ma_resource_manager_data_source`) encapsulates a data buffer or
1851 | data stream, depending on whether or not the data source was initialized with the
1852 | `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM` flag. If so, it will make use of a
1853 | `ma_resource_manager_data_stream` object. Otherwise it will use a `ma_resource_manager_data_buffer`
1854 | object. Both of these objects are data sources which means they can be used with any
1855 | `ma_data_source_*()` API.
1856 |
1857 | Another major feature of the resource manager is the ability to asynchronously decode audio files.
1858 | This relieves the audio thread of time-consuming decoding which can negatively affect scalability
1859 | due to the audio thread needing to complete it's work extremely quickly to avoid glitching.
1860 | Asynchronous decoding is achieved through a job system. There is a central multi-producer,
1861 | multi-consumer, fixed-capacity job queue. When some asynchronous work needs to be done, a job is
1862 | posted to the queue which is then read by a job thread. The number of job threads can be
1863 | configured for improved scalability, and job threads can all run in parallel without needing to
1864 | worry about the order of execution (how this is achieved is explained below).
1865 |
1866 | When a sound is being loaded asynchronously, playback can begin before the sound has been fully
1867 | decoded. This enables the application to start playback of the sound quickly, while at the same
1868 | time allowing the resource manager to keep loading in the background. Since there may be less
1869 | threads than the number of sounds being loaded at a given time, a simple scheduling system is used
1870 | to keep decoding time balanced and fair. The resource manager solves this by splitting decoding
1871 | into chunks called pages. By default, each page is 1 second long. When a page has been decoded, a
1872 | new job will be posted to start decoding the next page. By dividing up decoding into pages, an
1873 | individual sound shouldn't ever delay every other sound from having their first page decoded. Of
1874 | course, when loading many sounds at the same time, there will always be an amount of time required
1875 | to process jobs in the queue so in heavy load situations there will still be some delay. To
1876 | determine if a data source is ready to have some frames read, use
1877 | `ma_resource_manager_data_source_get_available_frames()`. This will return the number of frames
1878 | available starting from the current position.
1879 |
1880 |
1881 | 6.2.1. Job Queue
1882 | -----
1883 | The resource manager uses a job queue which is multi-producer, multi-consumer, and fixed-capacity.
1884 | This job queue is not currently lock-free, and instead uses a spinlock to achieve thread-safety.
1885 | Only a fixed number of jobs can be allocated and inserted into the queue which is done through a
1886 | lock-free data structure for allocating an index into a fixed sized array, with reference counting
1887 | for mitigation of the ABA problem. The reference count is 32-bit.
1888 |
1889 | For many types of jobs it's important that they execute in a specific order. In these cases, jobs
1890 | are executed serially. For the resource manager, serial execution of jobs is only required on a
1891 | per-object basis (per data buffer or per data stream). Each of these objects stores an execution
1892 | counter. When a job is posted it is associated with an execution counter. When the job is
1893 | processed, it checks if the execution counter of the job equals the execution counter of the
1894 | owning object and if so, processes the job. If the counters are not equal, the job will be posted
1895 | back onto the job queue for later processing. When the job finishes processing the execution order
1896 | of the main object is incremented. This system means the no matter how many job threads are
1897 | executing, decoding of an individual sound will always get processed serially. The advantage to
1898 | having multiple threads comes into play when loading multiple sounds at the same time.
1899 |
1900 | The resource manager's job queue is not 100% lock-free and will use a spinlock to achieve
1901 | thread-safety for a very small section of code. This is only relevant when the resource manager
1902 | uses more than one job thread. If only using a single job thread, which is the default, the
1903 | lock should never actually wait in practice. The amount of time spent locking should be quite
1904 | short, but it's something to be aware of for those who have pedantic lock-free requirements and
1905 | need to use more than one job thread. There are plans to remove this lock in a future version.
1906 |
1907 | In addition, posting a job will release a semaphore, which on Win32 is implemented with
1908 | `ReleaseSemaphore` and on POSIX platforms via a condition variable:
1909 |
1910 | ```c
1911 | pthread_mutex_lock(&pSemaphore->lock);
1912 | {
1913 |     pSemaphore->value += 1;
1914 |     pthread_cond_signal(&pSemaphore->cond);
1915 | }
1916 | pthread_mutex_unlock(&pSemaphore->lock);
1917 | ```

```

```

1918 |
1919 | Again, this is relevant for those with strict lock-free requirements in the audio thread. To avoid
1920 | this, you can use non-blocking mode (via the `MA_JOB_QUEUE_FLAG_NON_BLOCKING`
1921 | flag) and implement your own job processing routine (see the "Resource Manager" section above for
1922 | details on how to do this).
1923 |
1924 |
1925 |
1926 | 6.2.2. Data Buffers
1927 | -----
1928 | When the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM` flag is excluded at initialization time, the
1929 | resource manager will try to load the data into an in-memory data buffer. Before doing so, however,
1930 | it will first check if the specified file is already loaded. If so, it will increment a reference
1931 | counter and just use the already loaded data. This saves both time and memory. When the data buffer
1932 | is uninitialized, the reference counter will be decremented. If the counter hits zero, the file
1933 | will be unloaded. This is a detail to keep in mind because it could result in excessive loading and
1934 | unloading of a sound. For example, the following sequence will result in a file be loaded twice,
1935 | once after the other:
1936 |
1937 |     ```c
1938 |     ma_resource_manager_data_source_init(pResourceManager, "my_file", ..., &myDataBuffer0); //
> Refcount = 1. Initial load.
1939 |     ma_resource_manager_data_source_uninit(&myDataBuffer0);                                //
> Refcount = 0. Unloaded.
1940 |
1941 |     ma_resource_manager_data_source_init(pResourceManager, "my_file", ..., &myDataBuffer1); //
> Refcount = 1. Reloaded because previous uninit() unloaded it.
1942 |     ma_resource_manager_data_source_uninit(&myDataBuffer1);                                //
> Refcount = 0. Unloaded.
1943 |     ```
1944 |
1945 | A binary search tree (BST) is used for storing data buffers as it has good balance between
1946 | efficiency and simplicity. The key of the BST is a 64-bit hash of the file path that was passed
1947 | into `ma_resource_manager_data_source_init()`. The advantage of using a hash is that it saves
1948 | memory over storing the entire path, has faster comparisons, and results in a mostly balanced BST
1949 | due to the random nature of the hash. The disadvantages are that file names are case-sensitive and
1950 | there's a small chance of name collisions. If case-sensitivity is an issue, you should normalize
1951 | your file names to upper- or lower-case before initializing your data sources. If name collisions
1952 | become an issue, you'll need to change the name of one of the colliding names or just not use the
1953 | resource manager.
1954 |
1955 | When a sound file has not already been loaded and the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC`
1956 | flag is excluded, the file will be decoded synchronously by the calling thread. There are two
1957 | options for controlling how the audio is stored in the data buffer - encoded or decoded. When the
1958 | `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE` option is excluded, the raw file data will be stored
1959 | in memory. Otherwise the sound will be decoded before storing it in memory. Synchronous loading is
1960 | a very simple and standard process of simply adding an item to the BST, allocating a block of
1961 | memory and then decoding (if `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE` is specified).
1962 |
1963 | When the `MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC` flag is specified, loading of the data buffer
1964 | is done asynchronously. In this case, a job is posted to the queue to start loading and then the
1965 | function immediately returns, setting an internal result code to `MA_BUSY`. This result code is
1966 | returned when the program calls `ma_resource_manager_data_source_result()`. When decoding has fully
1967 | completed `MA_SUCCESS` will be returned. This can be used to know if loading has fully completed.
1968 |
1969 | When loading asynchronously, a single job is posted to the queue of the type
1970 | `MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER_NODE`. This involves making a copy of the file path
> and
1971 | associating it with job. When the job is processed by the job thread, it will first load the file
1972 | using the VFS associated with the resource manager. When using a custom VFS, it's important that it
1973 | be completely thread-safe because it will be used from one or more job threads at the same time.
1974 | Individual files should only ever be accessed by one thread at a time, however. After opening the
1975 | file via the VFS, the job will determine whether or not the file is being decoded. If not, it
1976 | simply allocates a block of memory and loads the raw file contents into it and returns. On the
1977 | other hand, when the file is being decoded, it will first allocate a decoder on the heap and
1978 | initialize it. Then it will check if the length of the file is known. If so it will allocate a
1979 | block of memory to store the decoded output and initialize it to silence. If the size is unknown,
1980 | it will allocate room for one page. After memory has been allocated, the first page will be
1981 | decoded. If the sound is shorter than a page, the result code will be set to `MA_SUCCESS` and the
1982 | completion event will be signalled and loading is now complete. If, however, there is more to
1983 | decode, a job with the code `MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE` is posted. This job
1984 | will decode the next page and perform the same process if it reaches the end. If there is more to
1985 | decode, the job will post another `MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE` job which
> will
1986 | keep on happening until the sound has been fully decoded. For sounds of an unknown length, each
1987 | page will be linked together as a linked list. Internally this is implemented via the
1988 | `ma_paged_audio_buffer` object.
1989 |
1990 |

```



```

2070 | container around the entire graph. The `ma_node_graph` object is required for some thread-safety
2071 | issues which will be explained later. A `ma_node_graph` object is initialized using miniaudio's
2072 | standard config/init system:
2073 |
2074 |     ```c
2075 |     ma_node_graph_config nodeGraphConfig = ma_node_graph_config_init(myChannelCount);
2076 |
2077 |     result = ma_node_graph_init(&nodeGraphConfig, NULL, &nodeGraph);    // Second parameter is a
> pointer to allocation callbacks.
2078 |     if (result != MA_SUCCESS) {
2079 |         // Failed to initialize node graph.
2080 |     }
2081 |     ```
2082 |
2083 | When you initialize the node graph, you're specifying the channel count of the endpoint. The
2084 | endpoint is a special node which has one input bus and one output bus, both of which have the
2085 | same channel count, which is specified in the config. Any nodes that connect directly to the
2086 | endpoint must be configured such that their output buses have the same channel count. When you read
2087 | audio data from the node graph, it'll have the channel count you specified in the config. To read
2088 | data from the graph:
2089 |
2090 |     ```c
2091 |     ma_uint32 framesRead;
2092 |     result = ma_node_graph_read_pcm_frames(&nodeGraph, pFramesOut, frameCount, &framesRead);
2093 |     if (result != MA_SUCCESS) {
2094 |         // Failed to read data from the node graph.
2095 |     }
2096 |     ```
2097 |
2098 | When you read audio data, miniaudio starts at the node graph's endpoint node which then pulls in
2099 | data from its input attachments, which in turn recursively pull in data from their inputs, and so
2100 | on. At the start of the graph there will be some kind of data source node which will have zero
2101 | inputs and will instead read directly from a data source. The base nodes don't literally need to
2102 | read from a `ma_data_source` object, but they will always have some kind of underlying object that
2103 | sources some kind of audio. The `ma_data_source_node` node can be used to read from a
2104 | `ma_data_source`. Data is always in floating-point format and in the number of channels you
2105 | specified when the graph was initialized. The sample rate is defined by the underlying data sources.
2106 | It's up to you to ensure they use a consistent and appropriate sample rate.
2107 |
2108 | The `ma_node` API is designed to allow custom nodes to be implemented with relative ease, but
2109 | miniaudio includes a few stock nodes for common functionality. This is how you would initialize a
2110 | node which reads directly from a data source (`ma_data_source_node`) which is an example of one
2111 | of the stock nodes that comes with miniaudio:
2112 |
2113 |     ```c
2114 |     ma_data_source_node_config config = ma_data_source_node_config_init(pMyDataSource);
2115 |
2116 |     ma_data_source_node dataSourceNode;
2117 |     result = ma_data_source_node_init(&nodeGraph, &config, NULL, &dataSourceNode);
2118 |     if (result != MA_SUCCESS) {
2119 |         // Failed to create data source node.
2120 |     }
2121 |     ```
2122 |
2123 | The data source node will use the output channel count to determine the channel count of the output
2124 | bus. There will be 1 output bus and 0 input buses (data will be drawn directly from the data
2125 | source). The data source must output to floating-point (`ma_format_f32`) or else an error will be
2126 | returned from `ma_data_source_node_init()`.
2127 |
2128 | By default the node will not be attached to the graph. To do so, use `ma_node_attach_output_bus()`:
2129 |
2130 |     ```c
2131 |     result = ma_node_attach_output_bus(&dataSourceNode, 0, ma_node_graph_get_endpoint(&nodeGraph),
> 0);
2132 |     if (result != MA_SUCCESS) {
2133 |         // Failed to attach node.
2134 |     }
2135 |     ```
2136 |
2137 | The code above connects the data source node directly to the endpoint. Since the data source node
2138 | has only a single output bus, the index will always be 0. Likewise, the endpoint only has a single
2139 | input bus which means the input bus index will also always be 0.
2140 |
2141 | To detach a specific output bus, use `ma_node_detach_output_bus()`. To detach all output buses, use
2142 | `ma_node_detach_all_output_buses()`. If you want to just move the output bus from one attachment to
2143 | another, you do not need to detach first. You can just call `ma_node_attach_output_bus()` and it'll
2144 | deal with it for you.
2145 |
2146 | Less frequently you may want to create a specialized node. This will be a node where you implement

```



```

2147 | your own processing callback to apply a custom effect of some kind. This is similar to initializing
2148 | one of the stock node types, only this time you need to specify a pointer to a vtable containing a
2149 | pointer to the processing function and the number of input and output buses. Example:
2150 |
2151 |     ``c
2152 |     static void my_custom_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn,
> ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
2153 |     {
2154 |         // Do some processing of ppFramesIn (one stream of audio data per input bus)
2155 |         const float* pFramesIn_0 = ppFramesIn[0]; // Input bus @ index 0.
2156 |         const float* pFramesIn_1 = ppFramesIn[1]; // Input bus @ index 1.
2157 |         float* pFramesOut_0 = ppFramesOut[0];      // Output bus @ index 0.
2158 |
2159 |         // Do some processing. On input, `pFrameCountIn` will be the number of input frames in each
2160 |         // buffer in `ppFramesIn` and `pFrameCountOut` will be the capacity of each of the buffers
2161 |         // in `ppFramesOut`. On output, `pFrameCountIn` should be set to the number of input frames
2162 |         // your node consumed and `pFrameCountOut` should be set the number of output frames that
2163 |         // were produced.
2164 |         //
2165 |         // You should process as many frames as you can. If your effect consumes input frames at the
2166 |         // same rate as output frames (always the case, unless you're doing resampling), you need
2167 |         // only look at `ppFramesOut` and process that exact number of frames. If you're doing
2168 |         // resampling, you'll need to be sure to set both `pFrameCountIn` and `pFrameCountOut`
2169 |         // properly.
2170 |     }
2171 |
2172 |     static ma_node_vtable my_custom_node_vtable =
2173 |     {
2174 |         my_custom_node_process_pcm_frames, // The function that will be called to process your
> custom node. This is where you'd implement your effect processing.
2175 |         NULL, // Optional. A callback for calculating the number of input frames that are required
> to process a specified number of output frames.
2176 |         2,    // 2 input buses.
2177 |         1,    // 1 output bus.
2178 |         0     // Default flags.
2179 |     };
2180 |
2181 |     ...
2182 |
2183 |     // Each bus needs to have a channel count specified. To do this you need to specify the channel
2184 |     // counts in an array and then pass that into the node config.
2185 |     ma_uint32 inputChannels[2]; // Equal in size to the number of input channels specified in
> the vtable.
2186 |     ma_uint32 outputChannels[1]; // Equal in size to the number of output channels specified in
> the vtable.
2187 |
2188 |     inputChannels[0] = channelsIn;
2189 |     inputChannels[1] = channelsIn;
2190 |     outputChannels[0] = channelsOut;
2191 |
2192 |     ma_node_config nodeConfig = ma_node_config_init();
2193 |     nodeConfig.vtable = &my_custom_node_vtable;
2194 |     nodeConfig.pInputChannels = inputChannels;
2195 |     nodeConfig.pOutputChannels = outputChannels;
2196 |
2197 |     ma_node_base node;
2198 |     result = ma_node_init(&nodeGraph, &nodeConfig, NULL, &node);
2199 |     if (result != MA_SUCCESS) {
2200 |         // Failed to initialize node.
2201 |     }
2202 |     ...
2203 |
2204 | When initializing a custom node, as in the code above, you'll normally just place your vtable in
2205 | static space. The number of input and output buses are specified as part of the vtable. If you need
2206 | a variable number of buses on a per-node bases, the vtable should have the relevant bus count set
2207 | to `MA_NODE_BUS_COUNT_UNKNOWN`. In this case, the bus count should be set in the node config:
2208 |
2209 |     ``c
2210 |     static ma_node_vtable my_custom_node_vtable =
2211 |     {
2212 |         my_custom_node_process_pcm_frames, // The function that will be called process your custom
> node. This is where you'd implement your effect processing.
2213 |         NULL, // Optional. A callback for calculating the number of input frames that are required
> to process a specified number of output frames.
2214 |         MA_NODE_BUS_COUNT_UNKNOWN, // The number of input buses is determined on a per-node basis.
2215 |         1,    // 1 output bus.
2216 |         0     // Default flags.
2217 |     };
2218 |

```



```

2219 | ...
2220 |
2221 |     ma_node_config nodeConfig = ma_node_config_init();
2222 |     nodeConfig.vtable         = &my_custom_node_vtable;
2223 |     nodeConfig.inputBusCount   = myBusCount;           // <-- Since the vtable specifies
> MA_NODE_BUS_COUNT_UNKNOWN, the input bus count should be set here.
2224 |     nodeConfig.pInputChannels = inputChannels;        // <-- Make sure there are
> nodeConfig.inputBusCount elements in this array.
2225 |     nodeConfig.pOutputChannels = outputChannels;      // <-- The vtable specifies 1 output bus, so
> there must be 1 element in this array.
2226 |     ``
2227 |
2228 | In the above example it's important to never set the `inputBusCount` and `outputBusCount` members
2229 | to anything other than their defaults if the vtable specifies an explicit count. They can only be
2230 | set if the vtable specifies MA_NODE_BUS_COUNT_UNKNOWN in the relevant bus count.
2231 |
2232 | Most often you'll want to create a structure to encapsulate your node with some extra data. You
2233 | need to make sure the `ma_node_base` object is your first member of the structure:
2234 |
2235 |     ``c
2236 |     typedef struct
2237 |     {
2238 |         ma_node_base base; // <-- Make sure this is always the first member.
2239 |         float someCustomData;
2240 |     } my_custom_node;
2241 |     ``
2242 |
2243 | By doing this, your object will be compatible with all `ma_node` APIs and you can attach it to the
2244 | graph just like any other node.
2245 |
2246 | In the custom processing callback (`my_custom_node_process_pcm_frames()` in the example above), the
2247 | number of channels for each bus is what was specified by the config when the node was initialized
2248 | with `ma_node_init()`. In addition, all attachments to each of the input buses will have been
2249 | pre-mixed by miniaudio. The config allows you to specify different channel counts for each
2250 | individual input and output bus. It's up to the effect to handle it appropriately, and if it can't,
2251 | return an error in it's initialization routine.
2252 |
2253 | Custom nodes can be assigned some flags to describe their behaviour. These are set via the vtable
2254 | and include the following:
2255 |
2256 | +-----+-----+
2257 | | Flag Name | Description |
2258 | +-----+-----+
2259 | | MA_NODE_FLAG_PASSTHROUGH | Useful for nodes that do not do any kind of audio |
2260 | | | processing, but are instead used for tracking |
2261 | | | time, handling events, etc. Also used by the |
2262 | | | internal endpoint node. It reads directly from |
2263 | | | the input bus to the output bus. Nodes with this |
2264 | | | flag must have exactly 1 input bus and 1 output |
2265 | | | bus, and both buses must have the same channel |
2266 | | | counts. |
2267 | +-----+-----+
2268 | | MA_NODE_FLAG_CONTINUOUS_PROCESSING | Causes the processing callback to be called even |
2269 | | | when no data is available to be read from input |
2270 | | | attachments. When a node has at least one input |
2271 | | | bus, but there are no inputs attached or the |
2272 | | | inputs do not deliver any data, the node's |
2273 | | | processing callback will not get fired. This flag |
2274 | | | will make it so the callback is always fired |
2275 | | | regardless of whether or not any input data is |
2276 | | | received. This is useful for effects like |
2277 | | | echos where there will be a tail of audio data |
2278 | | | that still needs to be processed even when the |
2279 | | | original data sources have reached their ends. It |
2280 | | | may also be useful for nodes that must always |
2281 | | | have their processing callback fired when there |
2282 | | | are no inputs attached. |
2283 | +-----+-----+
2284 | | MA_NODE_FLAG_ALLOW_NULL_INPUT | Used in conjunction with |
2285 | | | `MA_NODE_FLAG_CONTINUOUS_PROCESSING`. When this |
2286 | | | is set, the `ppFramesIn` parameter of the |
2287 | | | processing callback will be set to NULL when |
2288 | | | there are no input frames are available. When |
2289 | | | this is unset, silence will be posted to the |
2290 | | | processing callback. |
2291 | +-----+-----+
2292 | | MA_NODE_FLAG_DIFFERENT_PROCESSING_RATES | Used to tell miniaudio that input and output |
2293 | | | frames are processed at different rates. You |
2294 | | | should set this for any nodes that perform

```

```

2295 | | resampling. |
2296 | +-----+-----+
2297 | | MA_NODE_FLAG_SILENT_OUTPUT | Used to tell miniaudio that a node produces only |
2298 | | | silent output. This is useful for nodes where you |
2299 | | | don't want the output to contribute to the final |
2300 | | | mix. An example might be if you want split your |
2301 | | | stream and have one branch be output to a file. |
2302 | | | When using this flag, you should avoid writing to |
2303 | | | the output buffer of the node's processing |
2304 | | | callback because miniaudio will ignore it anyway. |
2305 | +-----+-----+
2306 |
2307 |
2308 | If you need to make a copy of an audio stream for effect processing you can use a splitter node
2309 | called `ma_splitter_node`. This takes has 1 input bus and splits the stream into 2 output buses.
2310 | You can use it like this:
2311 |
2312 | ```c
2313 |     ma_splitter_node_config splitterNodeConfig = ma_splitter_node_config_init(channels);
2314 |
2315 |     ma_splitter_node splitterNode;
2316 |     result = ma_splitter_node_init(&nodeGraph, &splitterNodeConfig, NULL, &splitterNode);
2317 |     if (result != MA_SUCCESS) {
2318 |         // Failed to create node.
2319 |     }
2320 |
2321 |     // Attach your output buses to two different input buses (can be on two different nodes).
2322 |     ma_node_attach_output_bus(&splitterNode, 0, ma_node_graph_get_endpoint(&nodeGraph), 0); //
> Attach directly to the endpoint.
2323 |     ma_node_attach_output_bus(&splitterNode, 1, &myEffectNode, 0); //
> Attach to input bus 0 of some effect node.
2324 |     ```
2325 |
2326 | The volume of an output bus can be configured on a per-bus basis:
2327 |
2328 | ```c
2329 |     ma_node_set_output_bus_volume(&splitterNode, 0, 0.5f);
2330 |     ma_node_set_output_bus_volume(&splitterNode, 1, 0.5f);
2331 |     ```
2332 |
2333 | In the code above we're using the splitter node from before and changing the volume of each of the
2334 | copied streams.
2335 |
2336 | You can start and stop a node with the following:
2337 |
2338 | ```c
2339 |     ma_node_set_state(&splitterNode, ma_node_state_started); // The default state.
2340 |     ma_node_set_state(&splitterNode, ma_node_state_stopped);
2341 |     ```
2342 |
2343 | By default the node is in a started state, but since it won't be connected to anything won't
2344 | actually be invoked by the node graph until it's connected. When you stop a node, data will not be
2345 | read from any of its input connections. You can use this property to stop a group of sounds
2346 | atomically.
2347 |
2348 | You can configure the initial state of a node in it's config:
2349 |
2350 | ```c
2351 |     nodeConfig.initialState = ma_node_state_stopped;
2352 |     ```
2353 |
2354 | Note that for the stock specialized nodes, all of their configs will have a `nodeConfig` member
2355 | which is the config to use with the base node. This is where the initial state can be configured
2356 | for specialized nodes:
2357 |
2358 | ```c
2359 |     dataSourceNodeConfig.nodeConfig.initialState = ma_node_state_stopped;
2360 |     ```
2361 |
2362 | When using a specialized node like `ma_data_source_node` or `ma_splitter_node`, be sure to not
2363 | modify the `vtable` member of the `nodeConfig` object.
2364 |
2365 |
2366 | 7.1. Timing
2367 | -----
2368 | The node graph supports starting and stopping nodes at scheduled times. This is especially useful
2369 | for data source nodes where you want to get the node set up, but only start playback at a specific
2370 | time. There are two clocks: local and global.
2371 |

```

```

2372 | A local clock is per-node, whereas the global clock is per graph. Scheduling starts and stops can
2373 | only be done based on the global clock because the local clock will not be running while the node
2374 | is stopped. The global clocks advances whenever `ma_node_graph_read_pcm_frames()` is called. On the
2375 | other hand, the local clock only advances when the node's processing callback is fired, and is
2376 | advanced based on the output frame count.
2377 |
2378 | To retrieve the global time, use `ma_node_graph_get_time()`. The global time can be set with
2379 | `ma_node_graph_set_time()` which might be useful if you want to do seeking on a global timeline.
2380 | Getting and setting the local time is similar. Use `ma_node_get_time()` to retrieve the local time,
2381 | and `ma_node_set_time()` to set the local time. The global and local times will be advanced by the
2382 | audio thread, so care should be taken to avoid data races. Ideally you should avoid calling these
2383 | outside of the node processing callbacks which are always run on the audio thread.
2384 |
2385 | There is basic support for scheduling the starting and stopping of nodes. You can only schedule one
2386 | start and one stop at a time. This is mainly intended for putting nodes into a started or stopped
2387 | state in a frame-exact manner. Without this mechanism, starting and stopping of a node is limited
2388 | to the resolution of a call to `ma_node_graph_read_pcm_frames()` which would typically be in blocks
2389 | of several milliseconds. The following APIs can be used for scheduling node states:
2390 |
2391 |     ```c
2392 |     ma_node_set_state_time()
2393 |     ma_node_get_state_time()
2394 |     ```
2395 |
2396 | The time is absolute and must be based on the global clock. An example is below:
2397 |
2398 |     ```c
2399 |     ma_node_set_state_time(&myNode, ma_node_state_started, sampleRate*1);    // Delay starting to 1
> second.
2400 |     ma_node_set_state_time(&myNode, ma_node_state_stopped, sampleRate*5);    // Delay stopping to 5
> seconds.
2401 |     ```
2402 |
2403 | An example for changing the state using a relative time.
2404 |
2405 |     ```c
2406 |     ma_node_set_state_time(&myNode, ma_node_state_started, sampleRate*1 +
> ma_node_graph_get_time(&myNodeGraph));
2407 |     ma_node_set_state_time(&myNode, ma_node_state_stopped, sampleRate*5 +
> ma_node_graph_get_time(&myNodeGraph));
2408 |     ```
2409 |
2410 | Note that due to the nature of multi-threading the times may not be 100% exact. If this is an
2411 | issue, consider scheduling state changes from within a processing callback. An idea might be to
2412 | have some kind of passthrough trigger node that is used specifically for tracking time and handling
2413 | events.
2414 |
2415 |
2416 |
2417 | 7.2. Thread Safety and Locking
2418 | -----
2419 | When processing audio, it's ideal not to have any kind of locking in the audio thread. Since it's
2420 | expected that `ma_node_graph_read_pcm_frames()` would be run on the audio thread, it does so
2421 | without the use of any locks. This section discusses the implementation used by miniaudio and goes
2422 | over some of the compromises employed by miniaudio to achieve this goal. Note that the current
2423 | implementation may not be ideal - feedback and critiques are most welcome.
2424 |
2425 | The node graph API is not *entirely* lock-free. Only `ma_node_graph_read_pcm_frames()` is expected
2426 | to be lock-free. Attachment, detachment and uninitialization of nodes use locks to simplify the
2427 | implementation, but are crafted in a way such that such locking is not required when reading audio
2428 | data from the graph. Locking in these areas are achieved by means of spinlocks.
2429 |
2430 | The main complication with keeping `ma_node_graph_read_pcm_frames()` lock-free stems from the fact
2431 | that a node can be uninitialized, and it's memory potentially freed, while in the middle of being
2432 | processed on the audio thread. There are times when the audio thread will be referencing a node,
2433 | which means the uninitialization process of a node needs to make sure it delays returning until the
2434 | audio thread is finished so that control is not handed back to the caller thereby giving them a
2435 | chance to free the node's memory.
2436 |
2437 | When the audio thread is processing a node, it does so by reading from each of the output buses of
2438 | the node. In order for a node to process data for one of its output buses, it needs to read from
2439 | each of its input buses, and so on and so forth. It follows that once all output buses of a node
2440 | are detached, the node as a whole will be disconnected and no further processing will occur unless
2441 | it's output buses are reattached, which won't be happening when the node is being uninitialized.
2442 | By having `ma_node_detach_output_bus()` wait until the audio thread is finished with it, we can
2443 | simplify a few things, at the expense of making `ma_node_detach_output_bus()` a bit slower. By
2444 | doing this, the implementation of `ma_node_uninit()` becomes trivial - just detach all output
2445 | nodes, followed by each of the attachments to each of its input nodes, and then do any final clean
2446 | up.

```

```

2447 |
2448 | With the above design, the worst-case scenario is `ma_node_detach_output_bus()` taking as long as
2449 | it takes to process the output bus being detached. This will happen if it's called at just the
2450 | wrong moment where the audio thread has just iterated it and has just started processing. The
2451 | caller of `ma_node_detach_output_bus()` will stall until the audio thread is finished, which
2452 | includes the cost of recursively processing its inputs. This is the biggest compromise made with
2453 | the approach taken by miniaudio for its lock-free processing system. The cost of detaching nodes
2454 | earlier in the pipeline (data sources, for example) will be cheaper than the cost of detaching
2455 | higher level nodes, such as some kind of final post-processing endpoint. If you need to do mass
2456 | detachments, detach starting from the lowest level nodes and work your way towards the final
2457 | endpoint node (but don't try detaching the node graph's endpoint). If the audio thread is not
2458 | running, detachment will be fast and detachment in any order will be the same. The reason nodes
2459 | need to wait for their input attachments to complete is due to the potential for desyncs between
2460 | data sources. If the node was to terminate processing mid way through processing its inputs,
2461 | there's a chance that some of the underlying data sources will have been read, but then others not.
2462 | That will then result in a potential desynchronization when detaching and reattaching higher-level
2463 | nodes. A possible solution to this is to have an option when detaching to terminate processing
2464 | before processing all input attachments which should be fairly simple.
2465 |
2466 | Another compromise, albeit less significant, is locking when attaching and detaching nodes. This
2467 | locking is achieved by means of a spinlock in order to reduce memory overhead. A lock is present
2468 | for each input bus and output bus. When an output bus is connected to an input bus, both the output
2469 | bus and input bus is locked. This locking is specifically for attaching and detaching across
2470 | different threads and does not affect `ma_node_graph_read_pcm_frames()` in any way. The locking and
2471 | unlocking is mostly self-explanatory, but a slightly less intuitive aspect comes into it when
2472 | considering that iterating over attachments must not break as a result of attaching or detaching a
2473 | node while iteration is occurring.
2474 |
2475 | Attaching and detaching are both quite simple. When an output bus of a node is attached to an input
2476 | bus of another node, it's added to a linked list. Basically, an input bus is a linked list, where
2477 | each item in the list is an output bus. We have some intentional (and convenient) restrictions on
2478 | what can be done with the linked list in order to simplify the implementation. First of all, whenever
2479 | something needs to iterate over the list, it must do so in a forward direction. Backwards iteration
2480 | is not supported. Also, items can only be added to the start of the list.
2481 |
2482 | The linked list is a doubly-linked list where each item in the list (an output bus) holds a pointer
2483 | to the next item in the list, and another to the previous item. A pointer to the previous item is
2484 | only required for fast detachment of the node - it is never used in iteration. This is an
2485 | important property because it means from the perspective of iteration, attaching and detaching of
2486 | an item can be done with a single atomic assignment. This is exploited by both the attachment and
2487 | detachment process. When attaching the node, the first thing that is done is the setting of the
2488 | local "next" and "previous" pointers of the node. After that, the item is "attached" to the list
2489 | by simply performing an atomic exchange with the head pointer. After that, the node is "attached"
2490 | to the list from the perspective of iteration. Even though the "previous" pointer of the next item
2491 | hasn't yet been set, from the perspective of iteration it's been attached because iteration will
2492 | only be happening in a forward direction which means the "previous" pointer won't actually ever get
2493 | used. The same general process applies to detachment. See `ma_node_attach_output_bus()` and
2494 | `ma_node_detach_output_bus()` for the implementation of this mechanism.
2495 |
2496 |
2497 |
2498 | 8. Decoding
2499 | =====
2500 | The `ma_decoder` API is used for reading audio files. Decoders are completely decoupled from
2501 | devices and can be used independently. Built-in support is included for the following formats:
2502 |
2503 | +-----+
2504 | | Format |
2505 | +-----+
2506 | | WAV   |
2507 | | MP3   |
2508 | | FLAC  |
2509 | +-----+
2510 |
2511 | You can disable the built-in decoders by specifying one or more of the following options before the
2512 | miniaudio implementation:
2513 |
2514 | ```c
2515 | #define MA_NO_WAV
2516 | #define MA_NO_MP3
2517 | #define MA_NO_FLAC
2518 | ```
2519 |
2520 | miniaudio supports the ability to plug in custom decoders. See the section below for details on how
2521 | to use custom decoders.
2522 |
2523 | A decoder can be initialized from a file with `ma_decoder_init_file()`, a block of memory with
2524 | `ma_decoder_init_memory()`, or from data delivered via callbacks with `ma_decoder_init()`. Here is
2525 | an example for loading a decoder from a file:

```

```

2526 |
2527 |     ``c
2528 |     ma_decoder decoder;
2529 |     ma_result result = ma_decoder_init_file("MySong.mp3", NULL, &decoder);
2530 |     if (result != MA_SUCCESS) {
2531 |         return false;    // An error occurred.
2532 |     }
2533 |
2534 |     ...
2535 |
2536 |     ma_decoder_uninit(&decoder);
2537 |     ``c
2538 |
2539 | When initializing a decoder, you can optionally pass in a pointer to a `ma_decoder_config` object
2540 | (the `NULL` argument in the example above) which allows you to configure the output format, channel
2541 | count, sample rate and channel map:
2542 |
2543 |     ``c
2544 |     ma_decoder_config config = ma_decoder_config_init(ma_format_f32, 2, 48000);
2545 |     ``c
2546 |
2547 | When passing in `NULL` for decoder config in `ma_decoder_init*()`, the output format will be the
2548 | same as that defined by the decoding backend.
2549 |
2550 | Data is read from the decoder as PCM frames. This will output the number of PCM frames actually
2551 | read. If this is less than the requested number of PCM frames it means you've reached the end. The
2552 | return value will be `MA_AT_END` if no samples have been read and the end has been reached.
2553 |
2554 |     ``c
2555 |     ma_result result = ma_decoder_read_pcm_frames(pDecoder, pFrames, framesToRead, &framesRead);
2556 |     if (framesRead < framesToRead) {
2557 |         // Reached the end.
2558 |     }
2559 |     ``c
2560 |
2561 | You can also seek to a specific frame like so:
2562 |
2563 |     ``c
2564 |     ma_result result = ma_decoder_seek_to_pcm_frame(pDecoder, targetFrame);
2565 |     if (result != MA_SUCCESS) {
2566 |         return false;    // An error occurred.
2567 |     }
2568 |     ``c
2569 |
2570 | If you want to loop back to the start, you can simply seek back to the first PCM frame:
2571 |
2572 |     ``c
2573 |     ma_decoder_seek_to_pcm_frame(pDecoder, 0);
2574 |     ``c
2575 |
2576 | When loading a decoder, miniaudio uses a trial and error technique to find the appropriate decoding
2577 | backend. This can be unnecessarily inefficient if the type is already known. In this case you can
2578 | use `encodingFormat` variable in the device config to specify a specific encoding format you want
2579 | to decode:
2580 |
2581 |     ``c
2582 |     decoderConfig.encodingFormat = ma_encoding_format_wav;
2583 |     ``c
2584 |
2585 | See the `ma_encoding_format` enum for possible encoding formats.
2586 |
2587 | The `ma_decoder_init_file()` API will try using the file extension to determine which decoding
2588 | backend to prefer.
2589 |
2590 |
2591 | 8.1. Custom Decoders
2592 | -----
2593 | It's possible to implement a custom decoder and plug it into miniaudio. This is extremely useful
2594 | when you want to use the `ma_decoder` API, but need to support an encoding format that's not one of
2595 | the stock formats supported by miniaudio. This can be put to particularly good use when using the
2596 | `ma_engine` and/or `ma_resource_manager` APIs because they use `ma_decoder` internally. If, for
2597 | example, you wanted to support Opus, you can do so with a custom decoder (there is a reference
2598 | Opus decoder in the "extras" folder of the miniaudio repository which uses libopus + libopusfile).
2599 |
2600 | A custom decoder must implement a data source. A vtable called `ma_decoding_backend_vtable` needs
2601 | to be implemented which is then passed into the decoder config:
2602 |
2603 |     ``c
2604 |     ma_decoding_backend_vtable* pCustomBackendVTables[] =

```

```

2605 | {
2606 |     &g_ma_decoding_backend_vtable_libvorbis,
2607 |     &g_ma_decoding_backend_vtable_libopus
2608 | };
2609 |
2610 | ...
2611 |
2612 | decoderConfig = ma_decoder_config_init_default();
2613 | decoderConfig.pCustomBackendUserData = NULL;
2614 | decoderConfig.ppCustomBackendVTables = pCustomBackendVTables;
2615 | decoderConfig.customBackendCount = sizeof(pCustomBackendVTables) /
> sizeof(pCustomBackendVTables[0]);
2616 | ```
2617 |
2618 | The `ma_decoding_backend_vtable` vtable has the following functions:
2619 |
2620 | ```
2621 |     onInit
2622 |     onInitFile
2623 |     onInitFileW
2624 |     onInitMemory
2625 |     onUninit
2626 | ```
2627 |
2628 | There are only two functions that must be implemented - `onInit` and `onUninit`. The other
2629 | functions can be implemented for a small optimization for loading from a file path or memory. If
2630 | these are not specified, miniaudio will deal with it for you via a generic implementation.
2631 |
2632 | When you initialize a custom data source (by implementing the `onInit` function in the vtable) you
2633 | will need to output a pointer to a `ma_data_source` which implements your custom decoder. See the
2634 | section about data sources for details on how to implement this. Alternatively, see the
2635 | "custom_decoders" example in the miniaudio repository.
2636 |
2637 | The `onInit` function takes a pointer to some callbacks for the purpose of reading raw audio data
2638 | from some arbitrary source. You'll use these functions to read from the raw data and perform the
2639 | decoding. When you call them, you will pass in the `pReadSeekTellUserData` pointer to the relevant
2640 | parameter.
2641 |
2642 | The `pConfig` parameter in `onInit` can be used to configure the backend if appropriate. It's only
2643 | used as a hint and can be ignored. However, if any of the properties are relevant to your decoder,
2644 | an optimal implementation will handle the relevant properties appropriately.
2645 |
2646 | If memory allocation is required, it should be done so via the specified allocation callbacks if
2647 | possible (the `pAllocationCallbacks` parameter).
2648 |
2649 | If an error occurs when initializing the decoder, you should leave `ppBackend` unset, or set to
2650 | NULL, and make sure everything is cleaned up appropriately and an appropriate result code returned.
2651 | When multiple custom backends are specified, miniaudio will cycle through the vtables in the order
2652 | they're listed in the array that's passed into the decoder config so it's important that your
2653 | initialization routine is clean.
2654 |
2655 | When a decoder is uninitialized, the `onUninit` callback will be fired which will give you an
2656 | opportunity to clean up and internal data.
2657 |
2658 |
2659 |
2660 | 9. Encoding
2661 | =====
2662 | The `ma_encoding` API is used for writing audio files. The only supported output format is WAV.
2663 | This can be disabled by specifying the following option before the implementation of miniaudio:
2664 |
2665 | ```c
2666 | #define MA_NO_WAV
2667 | ```
2668 |
2669 | An encoder can be initialized to write to a file with `ma_encoder_init_file()` or from data
2670 | delivered via callbacks with `ma_encoder_init()`. Below is an example for initializing an encoder
2671 | to output to a file.
2672 |
2673 | ```c
2674 | ma_encoder_config config = ma_encoder_config_init(ma_encoding_format_wav, FORMAT, CHANNELS,
> SAMPLE_RATE);
2675 | ma_encoder encoder;
2676 | ma_result result = ma_encoder_init_file("my_file.wav", &config, &encoder);
2677 | if (result != MA_SUCCESS) {
2678 |     // Error
2679 | }
2680 |
2681 | ...

```

```

2682 |
2683 |     ma_encoder_uninit(&encoder);
2684 |     ```
2685 |
2686 | When initializing an encoder you must specify a config which is initialized with
2687 | `ma_encoder_config_init()`. Here you must specify the file type, the output sample format, output
2688 | channel count and output sample rate. The following file types are supported:
2689 |
2690 | +-----+-----+
2691 | | Enum           | Description |
2692 | +-----+-----+
2693 | | ma_encoding_format_wav | WAV         |
2694 | +-----+-----+
2695 |
2696 | If the format, channel count or sample rate is not supported by the output file type an error will
2697 | be returned. The encoder will not perform data conversion so you will need to convert it before
2698 | outputting any audio data. To output audio data, use `ma_encoder_write_pcm_frames()`, like in the
2699 | example below:
2700 |
2701 |     ```c
2702 |     ma_uint64 framesWritten;
2703 |     result = ma_encoder_write_pcm_frames(&encoder, pCMFramesToWrite, framesToWrite,
> &framesWritten);
2704 |     if (result != MA_SUCCESS) {
2705 |         ... handle error ...
2706 |     }
2707 |     ```
2708 |
2709 | The `framesWritten` variable will contain the number of PCM frames that were actually written. This
2710 | is optionally and you can pass in `NULL` if you need this.
2711 |
2712 | Encoders must be uninitialized with `ma_encoder_uninit()`.
2713 |
2714 |
2715 |
2716 | 10. Data Conversion
2717 | =====
2718 | A data conversion API is included with miniaudio which supports the majority of data conversion
2719 | requirements. This supports conversion between sample formats, channel counts (with channel
2720 | mapping) and sample rates.
2721 |
2722 |
2723 | 10.1. Sample Format Conversion
2724 | -----
2725 | Conversion between sample formats is achieved with the `ma_pcm*_to_*()`, `ma_pcm_convert()` and
2726 | `ma_convert_pcm_frames_format()` APIs. Use `ma_pcm*_to_*()` to convert between two specific
2727 | formats. Use `ma_pcm_convert()` to convert based on a `ma_format` variable. Use
2728 | `ma_convert_pcm_frames_format()` to convert PCM frames where you want to specify the frame count
2729 | and channel count as a variable instead of the total sample count.
2730 |
2731 |
2732 | 10.1.1. Dithering
2733 | -----
2734 | Dithering can be set using the ditherMode parameter.
2735 |
2736 | The different dithering modes include the following, in order of efficiency:
2737 |
2738 | +-----+-----+
2739 | | Type          | Enum Token   |
2740 | +-----+-----+
2741 | | None          | ma_dither_mode_none |
2742 | | Rectangle     | ma_dither_mode_rectangle |
2743 | | Triangle      | ma_dither_mode_triangle |
2744 | +-----+-----+
2745 |
2746 | Note that even if the dither mode is set to something other than `ma_dither_mode_none`, it will be
2747 | ignored for conversions where dithering is not needed. Dithering is available for the following
2748 | conversions:
2749 |
2750 |     ```
2751 |     s16 -> u8
2752 |     s24 -> u8
2753 |     s32 -> u8
2754 |     f32 -> u8
2755 |     s24 -> s16
2756 |     s32 -> s16
2757 |     f32 -> s16
2758 |     ```
2759 |

```



```

2760 | Note that it is not an error to pass something other than ma_dither_mode_none for conversions where
2761 | dither is not used. It will just be ignored.
2762 |
2763 |
2764 |
2765 | 10.2. Channel Conversion
2766 | -----
2767 | Channel conversion is used for channel rearrangement and conversion from one channel count to
2768 | another. The `ma_channel_converter` API is used for channel conversion. Below is an example of
2769 | initializing a simple channel converter which converts from mono to stereo.
2770 |
2771 |     ```c
2772 |     ma_channel_converter_config config = ma_channel_converter_config_init(
2773 |         ma_format,           // Sample format
2774 |         1,                   // Input channels
2775 |         NULL,                 // Input channel map
2776 |         2,                   // Output channels
2777 |         NULL,                 // Output channel map
2778 |         ma_channel_mix_mode_default); // The mixing algorithm to use when combining channels.
2779 |
2780 |     result = ma_channel_converter_init(&config, NULL, &converter);
2781 |     if (result != MA_SUCCESS) {
2782 |         // Error.
2783 |     }
2784 |     ```
2785 |
2786 | To perform the conversion simply call `ma_channel_converter_process_pcm_frames()` like so:
2787 |
2788 |     ```c
2789 |     ma_result result = ma_channel_converter_process_pcm_frames(&converter, pFramesOut, pFramesIn,
> frameCount);
2790 |     if (result != MA_SUCCESS) {
2791 |         // Error.
2792 |     }
2793 |     ```
2794 |
2795 | It is up to the caller to ensure the output buffer is large enough to accommodate the new PCM
2796 | frames.
2797 |
2798 | Input and output PCM frames are always interleaved. Deinterleaved layouts are not supported.
2799 |
2800 |
2801 | 10.2.1. Channel Mapping
2802 | -----
2803 | In addition to converting from one channel count to another, like the example above, the channel
2804 | converter can also be used to rearrange channels. When initializing the channel converter, you can
2805 | optionally pass in channel maps for both the input and output frames. If the channel counts are the
2806 | same, and each channel map contains the same channel positions with the exception that they're in
2807 | a different order, a simple shuffling of the channels will be performed. If, however, there is not
2808 | a 1:1 mapping of channel positions, or the channel counts differ, the input channels will be mixed
2809 | based on a mixing mode which is specified when initializing the `ma_channel_converter_config`
2810 | object.
2811 |
2812 | When converting from mono to multi-channel, the mono channel is simply copied to each output
2813 | channel. When going the other way around, the audio of each output channel is simply averaged and
2814 | copied to the mono channel.
2815 |
2816 | In more complicated cases blending is used. The `ma_channel_mix_mode_simple` mode will drop excess
2817 | channels and silence extra channels. For example, converting from 4 to 2 channels, the 3rd and 4th
2818 | channels will be dropped, whereas converting from 2 to 4 channels will put silence into the 3rd and
2819 | 4th channels.
2820 |
2821 | The `ma_channel_mix_mode_rectangle` mode uses spacial locality based on a rectangle to compute a
2822 | simple distribution between input and output. Imagine sitting in the middle of a room, with
2823 | speakers on the walls representing channel positions. The `MA_CHANNEL_FRONT_LEFT` position can be
2824 | thought of as being in the corner of the front and left walls.
2825 |
2826 | Finally, the `ma_channel_mix_mode_custom_weights` mode can be used to use custom user-defined
2827 | weights. Custom weights can be passed in as the last parameter of
2828 | `ma_channel_converter_config_init()`.
2829 |
2830 | Predefined channel maps can be retrieved with `ma_channel_map_init_standard()`. This takes a
2831 | `ma_standard_channel_map` enum as its first parameter, which can be one of the following:
2832 |
2833 |
> +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2834 | | Name                                     | Description
> |
2835 |

```

```

> +-----+
2836 | | ma_standard_channel_map_default | Default channel map used by miniaudio. See below.
> |
2837 | | ma_standard_channel_map_microsoft | Channel map used by Microsoft's bitfield channel maps.
> |
2838 | | ma_standard_channel_map_alsa | Default ALSA channel map.
> |
2839 | | ma_standard_channel_map_rfc3551 | RFC 3551. Based on AIFF.
> |
2840 | | ma_standard_channel_map_flac | FLAC channel map.
> |
2841 | | ma_standard_channel_map_vorbis | Vorbis channel map.
> |
2842 | | ma_standard_channel_map_sound4 | FreeBSD's sound(4).
> |
2843 | | ma_standard_channel_map_sndio | sndio channel map. http://www.sndio.org/tips.html.
> |
2844 | | ma_standard_channel_map_webaudio | https://webaudio.github.io/web-audio-api/#ChannelOrdering
> |
2845 |
> +-----+
2846 |
2847 | Below are the channel maps used by default in miniaudio (`ma_standard_channel_map_default`):
2848 |
2849 | +-----+
2850 | | Channel Count | Mapping |
2851 | +-----+
2852 | | 1 (Mono) | 0: MA_CHANNEL_MONO |
2853 | +-----+
2854 | | 2 (Stereo) | 0: MA_CHANNEL_FRONT_LEFT <br> |
2855 | | | 1: MA_CHANNEL_FRONT_RIGHT |
2856 | +-----+
2857 | | 3 | 0: MA_CHANNEL_FRONT_LEFT <br> |
2858 | | | 1: MA_CHANNEL_FRONT_RIGHT <br> |
2859 | | | 2: MA_CHANNEL_FRONT_CENTER |
2860 | +-----+
2861 | | 4 (Surround) | 0: MA_CHANNEL_FRONT_LEFT <br> |
2862 | | | 1: MA_CHANNEL_FRONT_RIGHT <br> |
2863 | | | 2: MA_CHANNEL_FRONT_CENTER <br> |
2864 | | | 3: MA_CHANNEL_BACK_CENTER |
2865 | +-----+
2866 | | 5 | 0: MA_CHANNEL_FRONT_LEFT <br> |
2867 | | | 1: MA_CHANNEL_FRONT_RIGHT <br> |
2868 | | | 2: MA_CHANNEL_FRONT_CENTER <br> |
2869 | | | 3: MA_CHANNEL_BACK_LEFT <br> |
2870 | | | 4: MA_CHANNEL_BACK_RIGHT |
2871 | +-----+
2872 | | 6 (5.1) | 0: MA_CHANNEL_FRONT_LEFT <br> |
2873 | | | 1: MA_CHANNEL_FRONT_RIGHT <br> |
2874 | | | 2: MA_CHANNEL_FRONT_CENTER <br> |
2875 | | | 3: MA_CHANNEL_LFE <br> |
2876 | | | 4: MA_CHANNEL_SIDE_LEFT <br> |
2877 | | | 5: MA_CHANNEL_SIDE_RIGHT |
2878 | +-----+
2879 | | 7 | 0: MA_CHANNEL_FRONT_LEFT <br> |
2880 | | | 1: MA_CHANNEL_FRONT_RIGHT <br> |
2881 | | | 2: MA_CHANNEL_FRONT_CENTER <br> |
2882 | | | 3: MA_CHANNEL_LFE <br> |
2883 | | | 4: MA_CHANNEL_BACK_CENTER <br> |
2884 | | | 4: MA_CHANNEL_SIDE_LEFT <br> |
2885 | | | 5: MA_CHANNEL_SIDE_RIGHT |
2886 | +-----+
2887 | | 8 (7.1) | 0: MA_CHANNEL_FRONT_LEFT <br> |
2888 | | | 1: MA_CHANNEL_FRONT_RIGHT <br> |
2889 | | | 2: MA_CHANNEL_FRONT_CENTER <br> |
2890 | | | 3: MA_CHANNEL_LFE <br> |
2891 | | | 4: MA_CHANNEL_BACK_LEFT <br> |
2892 | | | 5: MA_CHANNEL_BACK_RIGHT <br> |
2893 | | | 6: MA_CHANNEL_SIDE_LEFT <br> |
2894 | | | 7: MA_CHANNEL_SIDE_RIGHT |
2895 | +-----+
2896 | | Other | All channels set to 0. This |
2897 | | | is equivalent to the same |
2898 | | | mapping as the device. |
2899 | +-----+
2900 |
2901 |
2902 |
2903 | 10.3. Resampling

```

```

2904 | -----
2905 | Resampling is achieved with the `ma_resampler` object. To create a resampler object, do something
2906 | like the following:
2907 |
2908 |     ```c
2909 |     ma_resampler_config config = ma_resampler_config_init(
2910 |         ma_format_s16,
2911 |         channels,
2912 |         sampleRateIn,
2913 |         sampleRateOut,
2914 |         ma_resample_algorithm_linear);
2915 |
2916 |     ma_resampler resampler;
2917 |     ma_result result = ma_resampler_init(&config, NULL, &resampler);
2918 |     if (result != MA_SUCCESS) {
2919 |         // An error occurred...
2920 |     }
2921 |     ```
2922 |
2923 | Do the following to uninitialized the resampler:
2924 |
2925 |     ```c
2926 |     ma_resampler_uninit(&resampler);
2927 |     ```
2928 |
2929 | The following example shows how data can be processed
2930 |
2931 |     ```c
2932 |     ma_uint64 frameCountIn = 1000;
2933 |     ma_uint64 frameCountOut = 2000;
2934 |     ma_result result = ma_resampler_process_pcm_frames(&resampler, pFramesIn, &frameCountIn,
> pFramesOut, &frameCountOut);
2935 |     if (result != MA_SUCCESS) {
2936 |         // An error occurred...
2937 |     }
2938 |
2939 |     // At this point, frameCountIn contains the number of input frames that were consumed and
> frameCountOut contains the
2940 |     // number of output frames written.
2941 |     ```
2942 |
2943 | To initialize the resampler you first need to set up a config (`ma_resampler_config`) with
2944 | `ma_resampler_config_init()`. You need to specify the sample format you want to use, the number of
2945 | channels, the input and output sample rate, and the algorithm.
2946 |
2947 | The sample format can be either `ma_format_s16` or `ma_format_f32`. If you need a different format
2948 | you will need to perform pre- and post-conversions yourself where necessary. Note that the format
2949 | is the same for both input and output. The format cannot be changed after initialization.
2950 |
2951 | The resampler supports multiple channels and is always interleaved (both input and output). The
2952 | channel count cannot be changed after initialization.
2953 |
2954 | The sample rates can be anything other than zero, and are always specified in hertz. They should be
2955 | set to something like 44100, etc. The sample rate is the only configuration property that can be
2956 | changed after initialization.
2957 |
2958 | The miniaudio resampler has built-in support for the following algorithms:
2959 |
2960 |     +-----+-----+
2961 |     | Algorithm | Enum Token          |
2962 |     +-----+-----+
2963 |     | Linear    | ma_resample_algorithm_linear |
2964 |     | Custom    | ma_resample_algorithm_custom |
2965 |     +-----+-----+
2966 |
2967 | The algorithm cannot be changed after initialization.
2968 |
2969 | Processing always happens on a per PCM frame basis and always assumes interleaved input and output.
2970 | De-interleaved processing is not supported. To process frames, use
2971 | `ma_resampler_process_pcm_frames()`. On input, this function takes the number of output frames you
2972 | can fit in the output buffer and the number of input frames contained in the input buffer. On
2973 | output these variables contain the number of output frames that were written to the output buffer
2974 | and the number of input frames that were consumed in the process. You can pass in NULL for the
2975 | input buffer in which case it will be treated as an infinitely large buffer of zeros. The output
2976 | buffer can also be NULL, in which case the processing will be treated as seek.
2977 |
2978 | The sample rate can be changed dynamically on the fly. You can change this with explicit sample
2979 | rates with `ma_resampler_set_rate()` and also with a decimal ratio with
2980 | `ma_resampler_set_rate_ratio()`. The ratio is in/out.

```

```

2981 |
2982 | Sometimes it's useful to know exactly how many input frames will be required to output a specific
2983 | number of frames. You can calculate this with `ma_resampler_get_required_input_frame_count()`.
2984 | Likewise, it's sometimes useful to know exactly how many frames would be output given a certain
2985 | number of input frames. You can do this with `ma_resampler_get_expected_output_frame_count()`.
2986 |
2987 | Due to the nature of how resampling works, the resampler introduces some latency. This can be
2988 | retrieved in terms of both the input rate and the output rate with
2989 | `ma_resampler_get_input_latency()` and `ma_resampler_get_output_latency()`.
2990 |
2991 |
2992 | 10.3.1. Resampling Algorithms
2993 | -----
2994 | The choice of resampling algorithm depends on your situation and requirements.
2995 |
2996 |
2997 | 10.3.1.1. Linear Resampling
2998 | -----
2999 | The linear resampler is the fastest, but comes at the expense of poorer quality. There is, however,
3000 | some control over the quality of the linear resampler which may make it a suitable option depending
3001 | on your requirements.
3002 |
3003 | The linear resampler performs low-pass filtering before or after downsampling or upsampling,
3004 | depending on the sample rates you're converting between. When decreasing the sample rate, the
3005 | low-pass filter will be applied before downsampling. When increasing the rate it will be performed
3006 | after upsampling. By default a fourth order low-pass filter will be applied. This can be configured
3007 | via the `lpfOrder` configuration variable. Setting this to 0 will disable filtering.
3008 |
3009 | The low-pass filter has a cutoff frequency which defaults to half the sample rate of the lowest of
3010 | the input and output sample rates (Nyquist Frequency).
3011 |
3012 | The API for the linear resampler is the same as the main resampler API, only it's called
3013 | `ma_linear_resampler`.
3014 |
3015 |
3016 | 10.3.2. Custom Resamplers
3017 | -----
3018 | You can implement a custom resampler by using the `ma_resample_algorithm_custom` resampling
3019 | algorithm and setting a vtable in the resampler config:
3020 |
3021 |     ```c
3022 |     ma_resampler_config config = ma_resampler_config_init(..., ma_resample_algorithm_custom);
3023 |     config.pBackendVTable = &g_customResamplerVTable;
3024 |     ```
3025 |
3026 | Custom resamplers are useful if the stock algorithms are not appropriate for your use case. You
3027 | need to implement the required functions in `ma_resampling_backend_vtable`. Note that not all
3028 | functions in the vtable need to be implemented, but if it's possible to implement, they should be.
3029 |
3030 | You can use the `ma_linear_resampler` object for an example on how to implement the vtable. The
3031 | `onGetHeapSize` callback is used to calculate the size of any internal heap allocation the custom
3032 | resampler will need to make given the supplied config. When you initialize the resampler via the
3033 | `onInit` callback, you'll be given a pointer to a heap allocation which is where you should store
3034 | the heap allocated data. You should not free this data in `onUninit` because miniaudio will manage
3035 | it for you.
3036 |
3037 | The `onProcess` callback is where the actual resampling takes place. On input, `pFrameCountIn`
3038 | points to a variable containing the number of frames in the `pFramesIn` buffer and
3039 | `pFrameCountOut` points to a variable containing the capacity in frames of the `pFramesOut` buffer.
3040 | On output, `pFrameCountIn` should be set to the number of input frames that were fully consumed,
3041 | whereas `pFrameCountOut` should be set to the number of frames that were written to `pFramesOut`.
3042 |
3043 | The `onSetRate` callback is optional and is used for dynamically changing the sample rate. If
3044 | dynamic rate changes are not supported, you can set this callback to NULL.
3045 |
3046 | The `onGetInputLatency` and `onGetOutputLatency` functions are used for retrieving the latency in
3047 | input and output rates respectively. These can be NULL in which case latency calculations will be
3048 | assumed to be NULL.
3049 |
3050 | The `onGetRequiredInputFrameCount` callback is used to give miniaudio a hint as to how many input
3051 | frames are required to be available to produce the given number of output frames. Likewise, the
3052 | `onGetExpectedOutputFrameCount` callback is used to determine how many output frames will be
3053 | produced given the specified number of input frames. miniaudio will use these as a hint, but they
3054 | are optional and can be set to NULL if you're unable to implement them.
3055 |
3056 |
3057 |
3058 | 10.4. General Data Conversion
3059 | -----

```

```

3060 | The `ma_data_converter` API can be used to wrap sample format conversion, channel conversion and
3061 | resampling into one operation. This is what miniaudio uses internally to convert between the format
3062 | requested when the device was initialized and the format of the backend's native device. The API
3063 | for general data conversion is very similar to the resampling API. Create a `ma_data_converter`
3064 | object like this:
3065 |
3066 |     ```c
3067 |     ma_data_converter_config config = ma_data_converter_config_init(
3068 |         inputFormat,
3069 |         outputFormat,
3070 |         inputChannels,
3071 |         outputChannels,
3072 |         inputSampleRate,
3073 |         outputSampleRate
3074 |     );
3075 |
3076 |     ma_data_converter converter;
3077 |     ma_result result = ma_data_converter_init(&config, NULL, &converter);
3078 |     if (result != MA_SUCCESS) {
3079 |         // An error occurred...
3080 |     }
3081 |     ```
3082 |
3083 | In the example above we use `ma_data_converter_config_init()` to initialize the config, however
3084 | there's many more properties that can be configured, such as channel maps and resampling quality.
3085 | Something like the following may be more suitable depending on your requirements:
3086 |
3087 |     ```c
3088 |     ma_data_converter_config config = ma_data_converter_config_init_default();
3089 |     config.formatIn = inputFormat;
3090 |     config.formatOut = outputFormat;
3091 |     config.channelsIn = inputChannels;
3092 |     config.channelsOut = outputChannels;
3093 |     config.sampleRateIn = inputSampleRate;
3094 |     config.sampleRateOut = outputSampleRate;
3095 |     ma_channel_map_init_standard(ma_standard_channel_map_flac, config.channelMapIn,
> sizeof(config.channelMapIn)/sizeof(config.channelMapIn[0]), config.channelCountIn);
3096 |     config.resampling.linear.lpfOrder = MA_MAX_FILTER_ORDER;
3097 |     ```
3098 |
3099 | Do the following to uninitialized the data converter:
3100 |
3101 |     ```c
3102 |     ma_data_converter_uninit(&converter, NULL);
3103 |     ```
3104 |
3105 | The following example shows how data can be processed
3106 |
3107 |     ```c
3108 |     ma_uint64 frameCountIn = 1000;
3109 |     ma_uint64 frameCountOut = 2000;
3110 |     ma_result result = ma_data_converter_process_pcm_frames(&converter, pFramesIn, &frameCountIn,
> pFramesOut, &frameCountOut);
3111 |     if (result != MA_SUCCESS) {
3112 |         // An error occurred...
3113 |     }
3114 |
3115 |     // At this point, frameCountIn contains the number of input frames that were consumed and
> frameCountOut contains the number
3116 |     // of output frames written.
3117 |     ```
3118 |
3119 | The data converter supports multiple channels and is always interleaved (both input and output).
3120 | The channel count cannot be changed after initialization.
3121 |
3122 | Sample rates can be anything other than zero, and are always specified in hertz. They should be set
3123 | to something like 44100, etc. The sample rate is the only configuration property that can be
3124 | changed after initialization, but only if the `resampling.allowDynamicSampleRate` member of
3125 | `ma_data_converter_config` is set to `MA_TRUE`. To change the sample rate, use
3126 | `ma_data_converter_set_rate()` or `ma_data_converter_set_rate_ratio()`. The ratio must be in/out.
3127 | The resampling algorithm cannot be changed after initialization.
3128 |
3129 | Processing always happens on a per PCM frame basis and always assumes interleaved input and output.
3130 | De-interleaved processing is not supported. To process frames, use
3131 | `ma_data_converter_process_pcm_frames()`. On input, this function takes the number of output frames
3132 | you can fit in the output buffer and the number of input frames contained in the input buffer. On
3133 | output these variables contain the number of output frames that were written to the output buffer
3134 | and the number of input frames that were consumed in the process. You can pass in NULL for the
3135 | input buffer in which case it will be treated as an infinitely large

```

```

3136 | buffer of zeros. The output buffer can also be NULL, in which case the processing will be treated
3137 | as seek.
3138 |
3139 | Sometimes it's useful to know exactly how many input frames will be required to output a specific
3140 | number of frames. You can calculate this with `ma_data_converter_get_required_input_frame_count()`.
3141 | Likewise, it's sometimes useful to know exactly how many frames would be output given a certain
3142 | number of input frames. You can do this with `ma_data_converter_get_expected_output_frame_count()`.
3143 |
3144 | Due to the nature of how resampling works, the data converter introduces some latency if resampling
3145 | is required. This can be retrieved in terms of both the input rate and the output rate with
3146 | `ma_data_converter_get_input_latency()` and `ma_data_converter_get_output_latency()`.
3147 |
3148 |
3149 |
3150 | 11. Filtering
3151 | =====
3152 |
3153 | 11.1. Biquad Filtering
3154 | -----
3155 | Biquad filtering is achieved with the `ma_biquad` API. Example:
3156 |
3157 | ```c
3158 | ma_biquad_config config = ma_biquad_config_init(ma_format_f32, channels, b0, b1, b2, a0, a1,
> a2);
3159 | ma_result result = ma_biquad_init(&config, NULL, &biquad);
3160 | if (result != MA_SUCCESS) {
3161 |     // Error.
3162 | }
3163 |
3164 | ...
3165 |
3166 | ma_biquad_process_pcm_frames(&biquad, pFramesOut, pFramesIn, frameCount);
3167 | ```
3168 |
3169 | Biquad filtering is implemented using transposed direct form 2. The numerator coefficients are b0,
3170 | b1 and b2, and the denominator coefficients are a0, a1 and a2. The a0 coefficient is required and
3171 | coefficients must not be pre-normalized.
3172 |
3173 | Supported formats are `ma_format_s16` and `ma_format_f32`. If you need to use a different format
3174 | you need to convert it yourself beforehand. When using `ma_format_s16` the biquad filter will use
3175 | fixed point arithmetic. When using `ma_format_f32`, floating point arithmetic will be used.
3176 |
3177 | Input and output frames are always interleaved.
3178 |
3179 | Filtering can be applied in-place by passing in the same pointer for both the input and output
3180 | buffers, like so:
3181 |
3182 | ```c
3183 | ma_biquad_process_pcm_frames(&biquad, pMyData, pMyData, frameCount);
3184 | ```
3185 |
3186 | If you need to change the values of the coefficients, but maintain the values in the registers you
3187 | can do so with `ma_biquad_reinit()`. This is useful if you need to change the properties of the
3188 | filter while keeping the values of registers valid to avoid glitching. Do not use
3189 | `ma_biquad_init()` for this as it will do a full initialization which involves clearing the
3190 | registers to 0. Note that changing the format or channel count after initialization is invalid and
3191 | will result in an error.
3192 |
3193 |
3194 | 11.2. Low-Pass Filtering
3195 | -----
3196 | Low-pass filtering is achieved with the following APIs:
3197 |
3198 | +-----+-----+
3199 | | API      | Description |
3200 | +-----+-----+
3201 | | ma_lpf1 | First order low-pass filter |
3202 | | ma_lpf2 | Second order low-pass filter |
3203 | | ma_lpf  | High order low-pass filter (Butterworth) |
3204 | +-----+-----+
3205 |
3206 | Low-pass filter example:
3207 |
3208 | ```c
3209 | ma_lpf_config config = ma_lpf_config_init(ma_format_f32, channels, sampleRate, cutoffFrequency,
> order);
3210 | ma_result result = ma_lpf_init(&config, &lpf);
3211 | if (result != MA_SUCCESS) {
3212 |     // Error.

```

```

3213 |     }
3214 |
3215 |     ...
3216 |
3217 |     ma_lpf_process_pcm_frames(&lpf, pFramesOut, pFramesIn, frameCount);
3218 |     ...
3219 |
3220 | Supported formats are `ma_format_s16` and `ma_format_f32`. If you need to use a different format
3221 | you need to convert it yourself beforehand. Input and output frames are always interleaved.
3222 |
3223 | Filtering can be applied in-place by passing in the same pointer for both the input and output
3224 | buffers, like so:
3225 |
3226 |     ```c
3227 |     ma_lpf_process_pcm_frames(&lpf, pMyData, pMyData, frameCount);
3228 |     ...
3229 |
3230 | The maximum filter order is limited to `MA_MAX_FILTER_ORDER` which is set to 8. If you need more,
3231 | you can chain first and second order filters together.
3232 |
3233 |     ```c
3234 |     for (iFilter = 0; iFilter < filterCount; iFilter += 1) {
3235 |         ma_lpf2_process_pcm_frames(&lpf2[iFilter], pMyData, pMyData, frameCount);
3236 |     }
3237 |     ...
3238 |
3239 | If you need to change the configuration of the filter, but need to maintain the state of internal
3240 | registers you can do so with `ma_lpf_reinit()`. This may be useful if you need to change the sample
3241 | rate and/or cutoff frequency dynamically while maintaining smooth transitions. Note that changing
3242 | > the
3243 | format or channel count after initialization is invalid and will result in an error.
3244 |
3245 | The `ma_lpf` object supports a configurable order, but if you only need a first order filter you
3246 | may want to consider using `ma_lpf1`. Likewise, if you only need a second order filter you can use
3247 | `ma_lpf2`. The advantage of this is that they're lighter weight and a bit more efficient.
3248 |
3249 | If an even filter order is specified, a series of second order filters will be processed in a
3250 | chain. If an odd filter order is specified, a first order filter will be applied, followed by a
3251 | series of second order filters in a chain.
3252 |
3253 | 11.3. High-Pass Filtering
3254 | -----
3255 | High-pass filtering is achieved with the following APIs:
3256 |
3257 | +-----+-----+
3258 | | API      | Description |
3259 | +-----+-----+
3260 | | ma_hpf1  | First order high-pass filter |
3261 | | ma_hpf2  | Second order high-pass filter |
3262 | | ma_hpf   | High order high-pass filter (Butterworth) |
3263 | +-----+-----+
3264 |
3265 | High-pass filters work exactly the same as low-pass filters, only the APIs are called `ma_hpf1`,
3266 | `ma_hpf2` and `ma_hpf`. See example code for low-pass filters for example usage.
3267 |
3268 |
3269 | 11.4. Band-Pass Filtering
3270 | -----
3271 | Band-pass filtering is achieved with the following APIs:
3272 |
3273 | +-----+-----+
3274 | | API      | Description |
3275 | +-----+-----+
3276 | | ma_bpf2  | Second order band-pass filter |
3277 | | ma_bpf   | High order band-pass filter   |
3278 | +-----+-----+
3279 |
3280 | Band-pass filters work exactly the same as low-pass filters, only the APIs are called `ma_bpf2` and
3281 | `ma_bpf`. See example code for low-pass filters for example usage. Note that the order for
3282 | band-pass filters must be an even number which means there is no first order band-pass filter,
3283 | unlike low-pass and high-pass filters.
3284 |
3285 |
3286 | 11.5. Notch Filtering
3287 | -----
3288 | Notch filtering is achieved with the following APIs:
3289 |
3290 | +-----+-----+

```



```

3291 | | API      | Description |
3292 | +-----+-----+
3293 | | ma_notch2 | Second order notching filter |
3294 | +-----+-----+
3295 |
3296 |
3297 | 11.6. Peaking EQ Filtering
3298 | -----
3299 | Peaking filtering is achieved with the following APIs:
3300 |
3301 | +-----+-----+
3302 | | API      | Description |
3303 | +-----+-----+
3304 | | ma_peak2  | Second order peaking filter |
3305 | +-----+-----+
3306 |
3307 |
3308 | 11.7. Low Shelf Filtering
3309 | -----
3310 | Low shelf filtering is achieved with the following APIs:
3311 |
3312 | +-----+-----+
3313 | | API      | Description |
3314 | +-----+-----+
3315 | | ma_loshelf2 | Second order low shelf filter |
3316 | +-----+-----+
3317 |
3318 | Where a high-pass filter is used to eliminate lower frequencies, a low shelf filter can be used to
3319 | just turn them down rather than eliminate them entirely.
3320 |
3321 |
3322 | 11.8. High Shelf Filtering
3323 | -----
3324 | High shelf filtering is achieved with the following APIs:
3325 |
3326 | +-----+-----+
3327 | | API      | Description |
3328 | +-----+-----+
3329 | | ma_hishelf2 | Second order high shelf filter |
3330 | +-----+-----+
3331 |
3332 | The high shelf filter has the same API as the low shelf filter, only you would use `ma_hishelf`
3333 | instead of `ma_loshelf`. Where a low shelf filter is used to adjust the volume of low frequencies,
3334 | the high shelf filter does the same thing for high frequencies.
3335 |
3336 |
3337 |
3338 |
3339 | 12. Waveform and Noise Generation
3340 | =====
3341 |
3342 | 12.1. Waveforms
3343 | -----
3344 | miniaudio supports generation of sine, square, triangle and sawtooth waveforms. This is achieved
3345 | with the `ma_waveform` API. Example:
3346 |
3347 | ```c
3348 | ma_waveform_config config = ma_waveform_config_init(
3349 |     FORMAT,
3350 |     CHANNELS,
3351 |     SAMPLE_RATE,
3352 |     ma_waveform_type_sine,
3353 |     amplitude,
3354 |     frequency);
3355 |
3356 | ma_waveform waveform;
3357 | ma_result result = ma_waveform_init(&config, &waveform);
3358 | if (result != MA_SUCCESS) {
3359 |     // Error.
3360 | }
3361 |
3362 | ...
3363 |
3364 | ma_waveform_read_pcm_frames(&waveform, pOutput, frameCount);
3365 | ```
3366 |
3367 | The amplitude, frequency, type, and sample rate can be changed dynamically with
3368 | `ma_waveform_set_amplitude()`, `ma_waveform_set_frequency()`, `ma_waveform_set_type()`, and
3369 | `ma_waveform_set_sample_rate()` respectively.

```

```

3370 |
3371 | You can invert the waveform by setting the amplitude to a negative value. You can use this to
3372 | control whether or not a sawtooth has a positive or negative ramp, for example.
3373 |
3374 | Below are the supported waveform types:
3375 |
3376 | +-----+
3377 | | Enum Name |
3378 | +-----+
3379 | | ma_waveform_type_sine |
3380 | | ma_waveform_type_square |
3381 | | ma_waveform_type_triangle |
3382 | | ma_waveform_type_sawtooth |
3383 | +-----+
3384 |
3385 |
3386 |
3387 | 12.2. Noise
3388 | -----
3389 | miniaudio supports generation of white, pink and Brownian noise via the `ma_noise` API. Example:
3390 |
3391 | ```c
3392 |     ma_noise_config config = ma_noise_config_init(
3393 |         FORMAT,
3394 |         CHANNELS,
3395 |         ma_noise_type_white,
3396 |         SEED,
3397 |         amplitude);
3398 |
3399 |     ma_noise noise;
3400 |     ma_result result = ma_noise_init(&config, &noise);
3401 |     if (result != MA_SUCCESS) {
3402 |         // Error.
3403 |     }
3404 |
3405 |     ...
3406 |
3407 |     ma_noise_read_pcm_frames(&noise, pOutput, frameCount);
3408 | ```
3409 |
3410 | The noise API uses simple LCG random number generation. It supports a custom seed which is useful
3411 | for things like automated testing requiring reproducibility. Setting the seed to zero will default
3412 | to `MA_DEFAULT_LCG_SEED`.
3413 |
3414 | The amplitude and seed can be changed dynamically with `ma_noise_set_amplitude()` and
3415 | `ma_noise_set_seed()` respectively.
3416 |
3417 | By default, the noise API will use different values for different channels. So, for example, the
3418 | left side in a stereo stream will be different to the right side. To instead have each channel use
3419 | the same random value, set the `duplicateChannels` member of the noise config to true, like so:
3420 |
3421 | ```c
3422 |     config.duplicateChannels = MA_TRUE;
3423 | ```
3424 |
3425 | Below are the supported noise types.
3426 |
3427 | +-----+
3428 | | Enum Name |
3429 | +-----+
3430 | | ma_noise_type_white |
3431 | | ma_noise_type_pink |
3432 | | ma_noise_type_brownian |
3433 | +-----+
3434 |
3435 |
3436 |
3437 | 13. Audio Buffers
3438 | =====
3439 | miniaudio supports reading from a buffer of raw audio data via the `ma_audio_buffer` API. This can
3440 | read from memory that's managed by the application, but can also handle the memory management for
3441 | you internally. Memory management is flexible and should support most use cases.
3442 |
3443 | Audio buffers are initialized using the standard configuration system used everywhere in miniaudio:
3444 |
3445 | ```c
3446 |     ma_audio_buffer_config config = ma_audio_buffer_config_init(
3447 |         format,
3448 |         channels,

```

```

3449 |         sizeInFrames,
3450 |         pExistingData,
3451 |         &allocationCallbacks);
3452 |
3453 |     ma_audio_buffer buffer;
3454 |     result = ma_audio_buffer_init(&config, &buffer);
3455 |     if (result != MA_SUCCESS) {
3456 |         // Error.
3457 |     }
3458 |
3459 |     ...
3460 |
3461 |     ma_audio_buffer_uninit(&buffer);
3462 |     ...
3463 |
3464 | In the example above, the memory pointed to by `pExistingData` will *not* be copied and is how an
3465 | application can do self-managed memory allocation. If you would rather make a copy of the data, use
3466 | `ma_audio_buffer_init_copy()`. To uninitialized the buffer, use `ma_audio_buffer_uninit()`.
3467 |
3468 | Sometimes it can be convenient to allocate the memory for the `ma_audio_buffer` structure and the
3469 | raw audio data in a contiguous block of memory. That is, the raw audio data will be located
3470 | immediately after the `ma_audio_buffer` structure. To do this, use
3471 | `ma_audio_buffer_alloc_and_init()`:
3472 |
3473 |     ```c
3474 |     ma_audio_buffer_config config = ma_audio_buffer_config_init(
3475 |         format,
3476 |         channels,
3477 |         sizeInFrames,
3478 |         pExistingData,
3479 |         &allocationCallbacks);
3480 |
3481 |     ma_audio_buffer* pBuffer
3482 |     result = ma_audio_buffer_alloc_and_init(&config, &pBuffer);
3483 |     if (result != MA_SUCCESS) {
3484 |         // Error
3485 |     }
3486 |
3487 |     ...
3488 |
3489 |     ma_audio_buffer_uninit_and_free(&buffer);
3490 |     ...
3491 |
3492 | If you initialize the buffer with `ma_audio_buffer_alloc_and_init()` you should uninitialized it
3493 | with `ma_audio_buffer_uninit_and_free()`. In the example above, the memory pointed to by
3494 | `pExistingData` will be copied into the buffer, which is contrary to the behavior of
3495 | `ma_audio_buffer_init()`.
3496 |
3497 | An audio buffer has a playback cursor just like a decoder. As you read frames from the buffer, the
3498 | cursor moves forward. The last parameter (`loop`) can be used to determine if the buffer should
3499 | loop. The return value is the number of frames actually read. If this is less than the number of
3500 | frames requested it means the end has been reached. This should never happen if the `loop`
3501 | parameter is set to true. If you want to manually loop back to the start, you can do so with with
3502 | `ma_audio_buffer_seek_to_pcm_frame(pAudioBuffer, 0)`. Below is an example for reading data from an
3503 | audio buffer.
3504 |
3505 |     ```c
3506 |     ma_uint64 framesRead = ma_audio_buffer_read_pcm_frames(pAudioBuffer, pFramesOut,
3507 | > desiredFrameCount, isLooping);
3508 |     if (framesRead < desiredFrameCount) {
3509 |         // If not looping, this means the end has been reached. This should never happen in looping
3510 | > mode with valid input.
3511 |     }
3512 |     ...
3513 |
3514 | Sometimes you may want to avoid the cost of data movement between the internal buffer and the
3515 | output buffer. Instead you can use memory mapping to retrieve a pointer to a segment of data:
3516 |
3517 |     ```c
3518 |     void* pMappedFrames;
3519 |     ma_uint64 frameCount = frameCountToTryMapping;
3520 |     ma_result result = ma_audio_buffer_map(pAudioBuffer, &pMappedFrames, &frameCount);
3521 |     if (result == MA_SUCCESS) {
3522 |         // Map was successful. The value in frameCount will be how many frames were _actually_
3523 | > mapped, which may be
3524 |         // less due to the end of the buffer being reached.
3525 |         ma_copy_pcm_frames(pFramesOut, pMappedFrames, frameCount, pAudioBuffer->format,
3526 | > pAudioBuffer->channels);
3527 |     }
3528 |     ...

```

```

3524 |         // You must unmap the buffer.
3525 |         ma_audio_buffer_unmap(pAudioBuffer, frameCount);
3526 |     }
3527 |     ...
3528 |
3529 | When you use memory mapping, the read cursor is increment by the frame count passed in to
3530 | `ma_audio_buffer_unmap()`. If you decide not to process every frame you can pass in a value smaller
3531 | than the value returned by `ma_audio_buffer_map()`. The disadvantage to using memory mapping is
3532 | that it does not handle looping for you. You can determine if the buffer is at the end for the
3533 | purpose of looping with `ma_audio_buffer_at_end()` or by inspecting the return value of
3534 | `ma_audio_buffer_unmap()` and checking if it equals `MA_AT_END`. You should not treat `MA_AT_END`
3535 | as an error when returned by `ma_audio_buffer_unmap()`.
3536 |
3537 |
3538 |
3539 | 14. Ring Buffers
3540 | =====
3541 | miniaudio supports lock free (single producer, single consumer) ring buffers which are exposed via
3542 | the `ma_rb` and `ma_pcm_rb` APIs. The `ma_rb` API operates on bytes, whereas the `ma_pcm_rb`
3543 | operates on PCM frames. They are otherwise identical as `ma_pcm_rb` is just a wrapper around
3544 | `ma_rb`.
3545 |
3546 | Unlike most other APIs in miniaudio, ring buffers support both interleaved and deinterleaved
3547 | streams. The caller can also allocate their own backing memory for the ring buffer to use
3548 | internally for added flexibility. Otherwise the ring buffer will manage it's internal memory for
3549 | you.
3550 |
3551 | The examples below use the PCM frame variant of the ring buffer since that's most likely the one
3552 | you will want to use. To initialize a ring buffer, do something like the following:
3553 |
3554 |     ```c
3555 |     ma_pcm_rb rb;
3556 |     ma_result result = ma_pcm_rb_init(FORMAT, CHANNELS, BUFFER_SIZE_IN_FRAMES, NULL, NULL, &rb);
3557 |     if (result != MA_SUCCESS) {
3558 |         // Error
3559 |     }
3560 |     ...
3561 |
3562 | The `ma_pcm_rb_init()` function takes the sample format and channel count as parameters because
3563 | it's the PCM variant of the ring buffer API. For the regular ring buffer that operates on bytes you
3564 | would call `ma_rb_init()` which leaves these out and just takes the size of the buffer in bytes
3565 | instead of frames. The fourth parameter is an optional pre-allocated buffer and the fifth parameter
3566 | is a pointer to a `ma_allocation_callbacks` structure for custom memory allocation routines.
3567 | Passing in `NULL` for this results in `MA_MALLOC()` and `MA_FREE()` being used.
3568 |
3569 | Use `ma_pcm_rb_init_ex()` if you need a deinterleaved buffer. The data for each sub-buffer is
3570 | offset from each other based on the stride. To manage your sub-buffers you can use
3571 | `ma_pcm_rb_get_subbuffer_stride()`, `ma_pcm_rb_get_subbuffer_offset()` and
3572 | `ma_pcm_rb_get_subbuffer_ptr()`.
3573 |
3574 | Use `ma_pcm_rb_acquire_read()` and `ma_pcm_rb_acquire_write()` to retrieve a pointer to a section
3575 | of the ring buffer. You specify the number of frames you need, and on output it will set to what
3576 | was actually acquired. If the read or write pointer is positioned such that the number of frames
3577 | requested will require a loop, it will be clamped to the end of the buffer. Therefore, the number
3578 | of frames you're given may be less than the number you requested.
3579 |
3580 | After calling `ma_pcm_rb_acquire_read()` or `ma_pcm_rb_acquire_write()`, you do your work on the
3581 | buffer and then "commit" it with `ma_pcm_rb_commit_read()` or `ma_pcm_rb_commit_write()`. This is
3582 | where the read/write pointers are updated. When you commit you need to pass in the buffer that was
3583 | returned by the earlier call to `ma_pcm_rb_acquire_read()` or `ma_pcm_rb_acquire_write()` and is
3584 | only used for validation. The number of frames passed to `ma_pcm_rb_commit_read()` and
3585 | `ma_pcm_rb_commit_write()` is what's used to increment the pointers, and can be less than what was
3586 | originally requested.
3587 |
3588 | If you want to correct for drift between the write pointer and the read pointer you can use a
3589 | combination of `ma_pcm_rb_pointer_distance()`, `ma_pcm_rb_seek_read()` and
3590 | `ma_pcm_rb_seek_write()`. Note that you can only move the pointers forward, and you should only
3591 | move the read pointer forward via the consumer thread, and the write pointer forward by the
3592 | producer thread. If there is too much space between the pointers, move the read pointer forward. If
3593 | there is too little space between the pointers, move the write pointer forward.
3594 |
3595 | You can use a ring buffer at the byte level instead of the PCM frame level by using the `ma_rb`
3596 | API. This is exactly the same, only you will use the `ma_rb` functions instead of `ma_pcm_rb` and
3597 | instead of frame counts you will pass around byte counts.
3598 |
3599 | The maximum size of the buffer in bytes is `0x7FFFFFFF-(MA_SIMD_ALIGNMENT-1)` due to the most
3600 | significant bit being used to encode a loop flag and the internally managed buffers always being
3601 | aligned to `MA_SIMD_ALIGNMENT`.
3602 |

```

```

3603 | Note that the ring buffer is only thread safe when used by a single consumer thread and single
3604 | producer thread.
3605 |
3606 |
3607 |
3608 | 15. Backends
3609 | =====
3610 | The following backends are supported by miniaudio. These are listed in order of default priority.
3611 | When no backend is specified when initializing a context or device, miniaudio will attempt to use
3612 | each of these backends in the order listed in the table below.
3613 |
3614 | Note that backends that are not usable by the build target will not be included in the build. For
3615 | example, ALSA, which is specific to Linux, will not be included in the Windows build.
3616 |
3617 | +-----+-----+-----+
3618 | | Name | Enum Name | Supported Operating Systems |
3619 | +-----+-----+-----+
3620 | | WASAPI | ma_backend_wasapi | Windows Vista+ |
3621 | | DirectSound | ma_backend_dsound | Windows XP+ |
3622 | | WinMM | ma_backend_winmm | Windows 95+ |
3623 | | Core Audio | ma_backend_coreaudio | macOS, iOS |
3624 | | sndio | ma_backend_sndio | OpenBSD |
3625 | | audio(4) | ma_backend_audio4 | NetBSD, OpenBSD |
3626 | | OSS | ma_backend_oss | FreeBSD |
3627 | | PulseAudio | ma_backend_pulseaudio | Cross Platform (disabled on Windows, BSD and Android) |
3628 | | ALSA | ma_backend_alsa | Linux |
3629 | | JACK | ma_backend_jack | Cross Platform (disabled on BSD and Android) |
3630 | | AAudio | ma_backend_aaudio | Android 8+ |
3631 | | OpenSL ES | ma_backend_opensl | Android (API level 16+) |
3632 | | Web Audio | ma_backend_webaudio | Web (via Emscripten) |
3633 | | Custom | ma_backend_custom | Cross Platform |
3634 | | Null | ma_backend_null | Cross Platform (not used on Web) |
3635 | +-----+-----+-----+
3636 |
3637 | Some backends have some nuance details you may want to be aware of.
3638 |
3639 | 15.1. WASAPI
3640 | -----
3641 | - Low-latency shared mode will be disabled when using an application-defined sample rate which is
3642 | different to the device's native sample rate. To work around this, set `wasapi.noAutoConvertSRC`
3643 | to true in the device config. This is due to IAudioClient3_InitializeSharedAudioStream() failing
3644 | when the AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM flag is specified. Setting wasapi.noAutoConvertSRC
3645 | will result in miniaudio's internal resampler being used instead which will in turn enable the
3646 | use of low-latency shared mode.
3647 |
3648 | 15.2. PulseAudio
3649 | -----
3650 | - If you experience bad glitching/noise on Arch Linux, consider this fix from the Arch wiki:
3651 | https://wiki.archlinux.org/index.php/PulseAudio/Troubleshooting#Glitches,\_skips\_or\_crackling.
3652 | Alternatively, consider using a different backend such as ALSA.
3653 |
3654 | 15.3. Android
3655 | -----
3656 | - To capture audio on Android, remember to add the RECORD_AUDIO permission to your manifest:
3657 | <uses-permission android:name="android.permission.RECORD_AUDIO" />
3658 | - With OpenSL|ES, only a single ma_context can be active at any given time. This is due to a
3659 | limitation with OpenSL|ES.
3660 | - With AAudio, only default devices are enumerated. This is due to AAudio not having an enumeration
3661 | API (devices are enumerated through Java). You can however perform your own device enumeration
3662 | through Java and then set the ID in the ma_device_id structure (ma_device_id.aaudio) and pass it
3663 | to ma_device_init().
3664 | - The backend API will perform resampling where possible. The reason for this as opposed to using
3665 | miniaudio's built-in resampler is to take advantage of any potential device-specific
3666 | optimizations the driver may implement.
3667 |
3668 | BSD
3669 | ---
3670 | - The sndio backend is currently only enabled on OpenBSD builds.
3671 | - The audio(4) backend is supported on OpenBSD, but you may need to disable sndiod before you can
3672 | use it.
3673 |
3674 | 15.4. UWP
3675 | -----
3676 | - UWP only supports default playback and capture devices.
3677 | - UWP requires the Microphone capability to be enabled in the application's manifest
3678 | > (Package.appxmanifest):
3679 | <<<
3680 | <Package ...>

```

```

3681 |     ...
3682 |     <Capabilities>
3683 |         <DeviceCapability Name="microphone" />
3684 |     </Capabilities>
3685 | </Package>
3686 |     ...
3687 |
3688 | 15.5. Web Audio / Emscripten
3689 | -----
3690 | - You cannot use `std=c` compiler flags, nor `ansi`. This only applies to the Emscripten build.
3691 | - The first time a context is initialized it will create a global object called "miniaudio" whose
3692 |   primary purpose is to act as a factory for device objects.
3693 | - Currently the Web Audio backend uses ScriptProcessorNode's, but this may need to change later as
3694 |   they've been deprecated.
3695 | - Google has implemented a policy in their browsers that prevent automatic media output without
3696 |   first receiving some kind of user input. The following web page has additional details:
3697 |   https://developers.google.com/web/updates/2017/09/autoplay-policy-changes. Starting the device
3698 |   may fail if you try to start playback without first handling some kind of user input.
3699 |
3700 |
3701 |
3702 | 16. Optimization Tips
3703 | =====
3704 | See below for some tips on improving performance.
3705 |
3706 | 16.1. Low Level API
3707 | -----
3708 | - In the data callback, if your data is already clipped prior to copying it into the output buffer,
3709 |   set the `noClip` config option in the device config to true. This will disable miniaudio's built
3710 |   in clipping function.
3711 | - By default, miniaudio will pre-silence the data callback's output buffer. If you know that you
3712 |   will always write valid data to the output buffer you can disable pre-silencing by setting the
3713 |   `noPreSilence` config option in the device config to true.
3714 |
3715 | 16.2. High Level API
3716 | -----
3717 | - If a sound does not require doppler or pitch shifting, consider disabling pitching by
3718 |   initializing the sound with the `MA_SOUND_FLAG_NO_PITCH` flag.
3719 | - If a sound does not require spatialization, disable it by initializing the sound with the
3720 |   `MA_SOUND_FLAG_NO_SPATIALIZATION` flag. It can be re-enabled again post-initialization with
3721 |   `ma_sound_set_spatialization_enabled()`.
3722 | - If you know all of your sounds will always be the same sample rate, set the engine's sample
3723 |   rate to match that of the sounds. Likewise, if you're using a self-managed resource manager,
3724 |   consider setting the decoded sample rate to match your sounds. By configuring everything to
3725 |   use a consistent sample rate, sample rate conversion can be avoided.
3726 |
3727 |
3728 |
3729 | 17. Miscellaneous Notes
3730 | =====
3731 | - Automatic stream routing is enabled on a per-backend basis. Support is explicitly enabled for
3732 |   WASAPI and Core Audio, however other backends such as PulseAudio may naturally support it, though
3733 |   not all have been tested.
3734 | - When compiling with VC6 and earlier, decoding is restricted to files less than 2GB in size. This
3735 |   is due to 64-bit file APIs not being available.
3736 | */
3737 |
3738 | #ifndef miniaudio_h
3739 | #define miniaudio_h
3740 |
3741 | #ifdef __cplusplus
3742 | extern "C" {
3743 | #endif
3744 |
3745 | #define MA_STRINGIFY(x)      #x
3746 | #define MA_XSTRINGIFY(x)    MA_STRINGIFY(x)
3747 |
3748 | #define MA_VERSION_MAJOR    0
3749 | #define MA_VERSION_MINOR    11
3750 | #define MA_VERSION_REVISION 25
3751 | #define MA_VERSION_STRING    MA_XSTRINGIFY(MA_VERSION_MAJOR) "." MA_XSTRINGIFY(MA_VERSION_MINOR) "."
3752 | > MA_XSTRINGIFY(MA_VERSION_REVISION)
3753 |
3754 | #if defined(_MSC_VER) && !defined(__clang__)
3755 |     #pragma warning(push)
3756 |     #pragma warning(disable:4201) /* nonstandard extension used: nameless struct/union */
3757 |     #pragma warning(disable:4214) /* nonstandard extension used: bit field types other than int */
3758 |     #pragma warning(disable:4324) /* structure was padded due to alignment specifier */
3759 | #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__

```

```

> >= 8)))
3759 |     #pragma GCC diagnostic push
3760 |     #pragma GCC diagnostic ignored "-Wpedantic" /* For ISO C99 doesn't support unnamed
> structs/unions [-Wpedantic] */
3761 |     #if defined(__clang__)
3762 |         #pragma GCC diagnostic ignored "-Wc11-extensions" /* anonymous unions are a C11 extension
> */
3763 |     #endif
3764 | #endif
3765 |
3766 |
3767 | #if defined(__LP64__) || defined(_WIN64) || (defined(__x86_64__) && !defined(__ILP32__)) ||
> defined(_M_X64) || defined(__ia64) || defined(_M_IA64) || defined(__aarch64__) || defined(_M_ARM64)
> || defined(__powerpc64__) || defined(__ppc64__)
3768 |     #define MA_SIZEOF_PTR 8
3769 | #else
3770 |     #define MA_SIZEOF_PTR 4
3771 | #endif
3772 |
3773 | #include <stddef.h> /* For size_t. */
3774 |
3775 | /* Sized types. */
3776 | #if defined(MA_USE_STDINT)
3777 |     #include <stdint.h>
3778 |     typedef int8_t    ma_int8;
3779 |     typedef uint8_t   ma_uint8;
3780 |     typedef int16_t   ma_int16;
3781 |     typedef uint16_t  ma_uint16;
3782 |     typedef int32_t   ma_int32;
3783 |     typedef uint32_t  ma_uint32;
3784 |     typedef int64_t   ma_int64;
3785 |     typedef uint64_t  ma_uint64;
3786 | #else
3787 |     typedef signed char    ma_int8;
3788 |     typedef unsigned char  ma_uint8;
3789 |     typedef signed short   ma_int16;
3790 |     typedef unsigned short ma_uint16;
3791 |     typedef signed int     ma_int32;
3792 |     typedef unsigned int   ma_uint32;
3793 |     #if defined(_MSC_VER) && !defined(__clang__)
3794 |         typedef signed __int64    ma_int64;
3795 |         typedef unsigned __int64   ma_uint64;
3796 |     #else
3797 |         #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 &&
> __GNUC_MINOR__ >= 6)))
3798 |             #pragma GCC diagnostic push
3799 |             #pragma GCC diagnostic ignored "-Wlong-long"
3800 |             #if defined(__clang__)
3801 |                 #pragma GCC diagnostic ignored "-Wc++11-long-long"
3802 |             #endif
3803 |             #endif
3804 |             typedef signed long long ma_int64;
3805 |             typedef unsigned long long ma_uint64;
3806 |             #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 &&
> __GNUC_MINOR__ >= 6)))
3807 |                 #pragma GCC diagnostic pop
3808 |             #endif
3809 |         #endif
3810 |     #endif /* MA_USE_STDINT */
3811 |
3812 | #if MA_SIZEOF_PTR == 8
3813 |     typedef ma_uint64      ma_uintptr;
3814 | #else
3815 |     typedef ma_uint32      ma_uintptr;
3816 | #endif
3817 |
3818 | typedef ma_uint8    ma_bool8;
3819 | typedef ma_uint32   ma_bool32;
3820 | #define MA_TRUE     1
3821 | #define MA_FALSE    0
3822 |
3823 | /* These float types are not used universally by miniaudio. It's to simplify some macro expansion
> for atomic types. */
3824 | typedef float      ma_float;
3825 | typedef double     ma_double;
3826 |
3827 | typedef void* ma_handle;
3828 | typedef void* ma_ptr;
3829 |

```



```

3830 | /*
3831 | ma_proc is annoying because when compiling with GCC we get pedantic warnings about converting
3832 | between `void*` and `void (*)()`. We can't use `void (*)()` with MSVC however, because we'll get
3833 | warning C4191 about "type cast between incompatible function types". To work around this I'm going
3834 | to use a different data type depending on the compiler.
3835 | */
3836 | #if defined(__GNUC__)
3837 | typedef void (*ma_proc)(void);
3838 | #else
3839 | typedef void* ma_proc;
3840 | #endif
3841 |
3842 | #if defined(_MSC_VER) && !defined(_WCHAR_T_DEFINED)
3843 | typedef ma_uint16 wchar_t;
3844 | #endif
3845 |
3846 | /* Define NULL for some compilers. */
3847 | #ifndef NULL
3848 | #define NULL 0
3849 | #endif
3850 |
3851 | #if defined(SIZE_MAX)
3852 | #define MA_SIZE_MAX    SIZE_MAX
3853 | #else
3854 | #define MA_SIZE_MAX    0xFFFFFFFF /* When SIZE_MAX is not defined by the standard library just
> default to the maximum 32-bit unsigned integer. */
3855 | #endif
3856 |
3857 | #define MA_UINT64_MAX    (((ma_uint64)0xFFFFFFFF << 32) | (ma_uint64)0xFFFFFFFF) /* Weird
> shifting syntax is for VC6 compatibility. */
3858 |
3859 |
3860 | /* Platform/backend detection. */
3861 | #if defined(_WIN32)
3862 | #define MA_WIN32
3863 | #if defined(MA_FORCE_UWP) || (defined(WINAPI_FAMILY) && ((defined(WINAPI_FAMILY_PC_APP) &&
> WINAPI_FAMILY == WINAPI_FAMILY_PC_APP) || (defined(WINAPI_FAMILY_PHONE_APP) && WINAPI_FAMILY ==
> WINAPI_FAMILY_PHONE_APP)))
3864 | #define MA_WIN32_UWP
3865 | #elif defined(WINAPI_FAMILY) && (defined(WINAPI_FAMILY_GAMES) && WINAPI_FAMILY ==
> WINAPI_FAMILY_GAMES)
3866 | #define MA_WIN32_GDK
3867 | #elif defined(NXDK)
3868 | #define MA_WIN32_NXDK
3869 | #else
3870 | #define MA_WIN32_DESKTOP
3871 | #endif
3872 |
3873 | /* The original Xbox. */
3874 | #if defined(NXDK) /* <-- Add other Xbox compiler toolchains here, and then add a
> toolchain-specific define in case we need to discriminate between them later. */
3875 | #define MA_XBOX
3876 |
3877 | #if defined(NXDK)
3878 | #define MA_XBOX_NXDK
3879 | #endif
3880 | #endif
3881 | #endif
3882 | #if defined(__MSDOS__) || defined(MSDOS) || defined(_MSDOS) || defined(__DOS__)
3883 | #define MA_DOS
3884 |
3885 | /* No threading allowed on DOS. */
3886 | #ifndef MA_NO_THREADING
3887 | #define MA_NO_THREADING
3888 | #endif
3889 |
3890 | /* No runtime linking allowed on DOS. */
3891 | #ifndef MA_NO_RUNTIME_LINKING
3892 | #define MA_NO_RUNTIME_LINKING
3893 | #endif
3894 | #endif
3895 | #if !defined(MA_WIN32) && !defined(MA_DOS) /* If it's not Win32, assume POSIX. */
3896 | #define MA_POSIX
3897 |
3898 | #if !defined(MA_NO_THREADING)
3899 | /*
3900 | Use the MA_NO_PTHREAD_IN_HEADER option at your own risk. This is intentionally undocumented.
3901 | You can use this to avoid including pthread.h in the header section. The downside is that it
3902 | results in some fixed sized structures being declared for the various types that are used in

```

```

3903 |         miniaudio. The risk here is that these types might be too small for a given platform. This
3904 |         risk is yours to take and no support will be offered if you enable this option.
3905 |         */
3906 |         #ifndef MA_NO_PTHREAD_IN_HEADER
3907 |             #include <pthread.h> /* Unfortunate #include, but needed for pthread_t,
> pthread_mutex_t and pthread_cond_t types. */
3908 |             typedef pthread_t      ma_pthread_t;
3909 |             typedef pthread_mutex_t ma_pthread_mutex_t;
3910 |             typedef pthread_cond_t  ma_pthread_cond_t;
3911 |         #else
3912 |             typedef ma_uintptr      ma_pthread_t;
3913 |             typedef union            ma_pthread_mutex_t { char __data[40]; ma_uint64 __alignment; }
> ma_pthread_mutex_t;
3914 |             typedef union            ma_pthread_cond_t  { char __data[48]; ma_uint64 __alignment; }
> ma_pthread_cond_t;
3915 |         #endif
3916 |     #endif
3917 |
3918 |     #if defined(__unix__)
3919 |         #define MA_UNIX
3920 |     #endif
3921 |     #if defined(__linux__)
3922 |         #define MA_LINUX
3923 |     #endif
3924 |     #if defined(__APPLE__)
3925 |         #define MA_APPLE
3926 |     #endif
3927 |     #if defined(__DragonFly__) || defined(__FreeBSD__) || defined(__NetBSD__) ||
> defined(__OpenBSD__)
3928 |         #define MA_BSD
3929 |     #endif
3930 |     #if defined(__ANDROID__)
3931 |         #define MA_ANDROID
3932 |     #endif
3933 |     #if defined(__EMSCRIPTEN__)
3934 |         #define MA_EMSCRIPTEN
3935 |     #endif
3936 |     #if defined(__ORBIS__)
3937 |         #define MA_ORBIS
3938 |     #endif
3939 |     #if defined(__PROSPERO__)
3940 |         #define MA_PROSPERO
3941 |     #endif
3942 |     #if defined(__3DS__)
3943 |         #define MA_3DS
3944 |     #endif
3945 |     #if defined(__SWITCH__) || defined(__NX__)
3946 |         #define MA_SWITCH
3947 |     #endif
3948 |     #if defined(__BEOS__) || defined(__HAIKU__)
3949 |         #define MA_BEOS
3950 |     #endif
3951 |     #if defined(__HAIKU__)
3952 |         #define MA_HAIKU
3953 |     #endif
3954 | #endif
3955 |
3956 | #if !defined(MA_FALLTHROUGH) && defined(__cplusplus) && __cplusplus >= 201703L
3957 |     #define MA_FALLTHROUGH [[fallthrough]]
3958 | #endif
3959 | #if !defined(MA_FALLTHROUGH) && defined(__STDC_VERSION__) && __STDC_VERSION__ >= 202000L
3960 |     #define MA_FALLTHROUGH [[fallthrough]]
3961 | #endif
3962 | #if !defined(MA_FALLTHROUGH) && defined(__has_attribute)
3963 |     #if __has_attribute(fallthrough)
3964 |         #define MA_FALLTHROUGH __attribute__((fallthrough))
3965 |     #endif
3966 | #endif
3967 | #if !defined(MA_FALLTHROUGH)
3968 |     #define MA_FALLTHROUGH ((void)0)
3969 | #endif
3970 |
3971 | #ifdef _MSC_VER
3972 |     #define MA_INLINE __forceinline
3973 |
3974 |     /* noline was introduced in Visual Studio 2005. */
3975 |     #if _MSC_VER >= 1400
3976 |         #define MA_NO_INLINE __declspec(noinline)
3977 |     #else

```

```

3978 |         #define MA_NO_INLINE
3979 |     #endif
3980 | #elif defined(__GNUC__)
3981 |     /*
3982 |     I've had a bug report where GCC is emitting warnings about functions possibly not being
> inlineable. This warning happens when
3983 |     the __attribute__((always_inline)) attribute is defined without an "inline" statement. I think
> therefore there must be some
3984 |     case where "__inline__" is not always defined, thus the compiler emitting these warnings. When
> using -std=c89 or -ansi on the
3985 |     command line, we cannot use the "inline" keyword and instead need to use "__inline__". In an
> attempt to work around this issue
3986 |     I am using "__inline__" only when we're compiling in strict ANSI mode.
3987 |     */
3988 |     #if defined(__STRICT_ANSI__)
3989 |         #define MA_GNUC_INLINE_HINT __inline__
3990 |     #else
3991 |         #define MA_GNUC_INLINE_HINT inline
3992 |     #endif
3993 |
3994 |     #if (__GNUC__ > 3 || (__GNUC__ == 3 && __GNUC_MINOR__ >= 2)) || defined(__clang__)
3995 |         #define MA_INLINE MA_GNUC_INLINE_HINT __attribute__((always_inline))
3996 |         #define MA_NO_INLINE __attribute__((noinline))
3997 |     #else
3998 |         #define MA_INLINE MA_GNUC_INLINE_HINT
3999 |         #define MA_NO_INLINE
4000 |     #endif
4001 | #elif defined(__WATCOMC__)
4002 |     #define MA_INLINE __inline
4003 |     #define MA_NO_INLINE
4004 | #else
4005 |     #define MA_INLINE
4006 |     #define MA_NO_INLINE
4007 | #endif
4008 |
4009 | /* MA_DLL is not officially supported. You're on your own if you want to use this. */
4010 | #if defined(MA_DLL)
4011 |     #if defined(_WIN32)
4012 |         #define MA_DLL_IMPORT __declspec(dllimport)
4013 |         #define MA_DLL_EXPORT __declspec(dllexport)
4014 |         #define MA_DLL_PRIVATE static
4015 |     #else
4016 |         #if defined(__GNUC__) && __GNUC__ >= 4
4017 |             #define MA_DLL_IMPORT __attribute__((visibility("default")))
4018 |             #define MA_DLL_EXPORT __attribute__((visibility("default")))
4019 |             #define MA_DLL_PRIVATE __attribute__((visibility("hidden")))
4020 |         #else
4021 |             #define MA_DLL_IMPORT
4022 |             #define MA_DLL_EXPORT
4023 |             #define MA_DLL_PRIVATE static
4024 |         #endif
4025 |     #endif
4026 | #endif
4027 |
4028 | #if !defined(MA_API)
4029 |     #if defined(MA_DLL)
4030 |         #if defined(MINIAUDIO_IMPLEMENTATION) || defined(MA_IMPLEMENTATION)
4031 |             #define MA_API MA_DLL_EXPORT
4032 |         #else
4033 |             #define MA_API MA_DLL_IMPORT
4034 |         #endif
4035 |     #else
4036 |         #define MA_API extern
4037 |     #endif
4038 | #endif
4039 |
4040 | #if !defined(MA_STATIC)
4041 |     #if defined(MA_DLL)
4042 |         #define MA_PRIVATE MA_DLL_PRIVATE
4043 |     #else
4044 |         #define MA_PRIVATE static
4045 |     #endif
4046 | #endif
4047 |
4048 |
4049 | /* SIMD alignment in bytes. Currently set to 32 bytes in preparation for future AVX optimizations.
> */
4050 | #define MA_SIMD_ALIGNMENT 32
4051 |

```

```

4052 | /*
4053 | Special wchar_t type to ensure any structures in the public sections that reference it have a
4054 | consistent size across all platforms.
4055 |
4056 | On Windows, wchar_t is 2 bytes, whereas everywhere else it's 4 bytes. Since Windows likes to use
4057 | wchar_t for its IDs, we need a special explicitly sized wchar type that is always 2 bytes on all
4058 | platforms.
4059 | */
4060 | #if !defined(MA_POSIX) && defined(MA_WIN32)
4061 | typedef wchar_t      ma_wchar_win32;
4062 | #else
4063 | typedef ma_uint16    ma_wchar_win32;
4064 | #endif
4065 |
4066 |
4067 |
4068 | /*
4069 | Logging Levels
4070 | =====
4071 | Log levels are only used to give logging callbacks some context as to the severity of a log message
4072 | so they can do filtering. All log levels will be posted to registered logging callbacks. If you
4073 | don't want to output a certain log level you can discriminate against the log level in the callback.
4074 |
4075 | MA_LOG_LEVEL_DEBUG
4076 |     Used for debugging. Useful for debug and test builds, but should be disabled in release builds.
4077 |
4078 | MA_LOG_LEVEL_INFO
4079 |     Informational logging. Useful for debugging. This will never be called from within the data
4080 |     callback.
4081 |
4082 | MA_LOG_LEVEL_WARNING
4083 |     Warnings. You should enable this in you development builds and action them when encountered.
> These
4084 |     logs usually indicate a potential problem or misconfiguration, but still allow you to keep
4085 |     running. This will never be called from within the data callback.
4086 |
4087 | MA_LOG_LEVEL_ERROR
4088 |     Error logging. This will be fired when an operation fails and is subsequently aborted. This can
4089 |     be fired from within the data callback, in which case the device will be stopped. You should
4090 |     always have this log level enabled.
4091 | */
4092 | typedef enum
4093 | {
4094 |     MA_LOG_LEVEL_DEBUG    = 4,
4095 |     MA_LOG_LEVEL_INFO     = 3,
4096 |     MA_LOG_LEVEL_WARNING  = 2,
4097 |     MA_LOG_LEVEL_ERROR    = 1
4098 | } ma_log_level;
4099 |
4100 | /*
4101 | Variables needing to be accessed atomically should be declared with this macro for two reasons:
4102 |
4103 |     1) It allows people who read the code to identify a variable as such; and
4104 |     2) It forces alignment on platforms where it's required or optimal.
4105 |
4106 | Note that for x86/64, alignment is not strictly necessary, but does have some performance
4107 | implications. Where supported by the compiler, alignment will be used, but otherwise if the CPU
4108 | architecture does not require it, it will simply leave it unaligned. This is the case with old
4109 | versions of Visual Studio, which I've confirmed with at least VC6.
4110 | */
4111 | #if !defined(_MSC_VER) && defined (__STDC_VERSION__) && (__STDC_VERSION__ >= 201112L)
4112 |     #include <stdalign.h>
4113 |     #define MA_ATOMIC(alignment, type)                _Alignas(alignment) type
4114 | #else
4115 |     #if defined(__GNUC__)
4116 |         /* GCC-style compilers. */
4117 |         #define MA_ATOMIC(alignment, type)            type __attribute__((aligned(alignment)))
4118 |     #elif defined(_MSC_VER) && _MSC_VER > 1200 /* 1200 = VC6. Alignment not supported, but not
> necessary because x86 is the only supported target. */
4119 |         /* MSVC. */
4120 |         #define MA_ATOMIC(alignment, type)            __declspec(aligned(alignment)) type
4121 |     #else
4122 |         /* Other compilers. */
4123 |         #define MA_ATOMIC(alignment, type)            type
4124 |     #endif
4125 | #endif
4126 |
4127 | typedef struct ma_context ma_context;
4128 | typedef struct ma_device ma_device;

```

```

4129 |
4130 | typedef ma_uint8 ma_channel;
4131 | typedef enum
4132 | {
4133 |     MA_CHANNEL_NONE           = 0,
4134 |     MA_CHANNEL_MONO           = 1,
4135 |     MA_CHANNEL_FRONT_LEFT     = 2,
4136 |     MA_CHANNEL_FRONT_RIGHT    = 3,
4137 |     MA_CHANNEL_FRONT_CENTER   = 4,
4138 |     MA_CHANNEL_LFE            = 5,
4139 |     MA_CHANNEL_BACK_LEFT      = 6,
4140 |     MA_CHANNEL_BACK_RIGHT     = 7,
4141 |     MA_CHANNEL_FRONT_LEFT_CENTER = 8,
4142 |     MA_CHANNEL_FRONT_RIGHT_CENTER = 9,
4143 |     MA_CHANNEL_BACK_CENTER     = 10,
4144 |     MA_CHANNEL_SIDE_LEFT      = 11,
4145 |     MA_CHANNEL_SIDE_RIGHT     = 12,
4146 |     MA_CHANNEL_TOP_CENTER      = 13,
4147 |     MA_CHANNEL_TOP_FRONT_LEFT = 14,
4148 |     MA_CHANNEL_TOP_FRONT_CENTER = 15,
4149 |     MA_CHANNEL_TOP_FRONT_RIGHT = 16,
4150 |     MA_CHANNEL_TOP_BACK_LEFT  = 17,
4151 |     MA_CHANNEL_TOP_BACK_CENTER = 18,
4152 |     MA_CHANNEL_TOP_BACK_RIGHT = 19,
4153 |     MA_CHANNEL_AUX_0          = 20,
4154 |     MA_CHANNEL_AUX_1          = 21,
4155 |     MA_CHANNEL_AUX_2          = 22,
4156 |     MA_CHANNEL_AUX_3          = 23,
4157 |     MA_CHANNEL_AUX_4          = 24,
4158 |     MA_CHANNEL_AUX_5          = 25,
4159 |     MA_CHANNEL_AUX_6          = 26,
4160 |     MA_CHANNEL_AUX_7          = 27,
4161 |     MA_CHANNEL_AUX_8          = 28,
4162 |     MA_CHANNEL_AUX_9          = 29,
4163 |     MA_CHANNEL_AUX_10         = 30,
4164 |     MA_CHANNEL_AUX_11         = 31,
4165 |     MA_CHANNEL_AUX_12         = 32,
4166 |     MA_CHANNEL_AUX_13         = 33,
4167 |     MA_CHANNEL_AUX_14         = 34,
4168 |     MA_CHANNEL_AUX_15         = 35,
4169 |     MA_CHANNEL_AUX_16         = 36,
4170 |     MA_CHANNEL_AUX_17         = 37,
4171 |     MA_CHANNEL_AUX_18         = 38,
4172 |     MA_CHANNEL_AUX_19         = 39,
4173 |     MA_CHANNEL_AUX_20         = 40,
4174 |     MA_CHANNEL_AUX_21         = 41,
4175 |     MA_CHANNEL_AUX_22         = 42,
4176 |     MA_CHANNEL_AUX_23         = 43,
4177 |     MA_CHANNEL_AUX_24         = 44,
4178 |     MA_CHANNEL_AUX_25         = 45,
4179 |     MA_CHANNEL_AUX_26         = 46,
4180 |     MA_CHANNEL_AUX_27         = 47,
4181 |     MA_CHANNEL_AUX_28         = 48,
4182 |     MA_CHANNEL_AUX_29         = 49,
4183 |     MA_CHANNEL_AUX_30         = 50,
4184 |     MA_CHANNEL_AUX_31         = 51,
4185 |
4186 |     /* Count. */
4187 |     MA_CHANNEL_POSITION_COUNT,
4188 |
4189 |     /* Aliases. */
4190 |     MA_CHANNEL_LEFT           = MA_CHANNEL_FRONT_LEFT,
4191 |     MA_CHANNEL_RIGHT          = MA_CHANNEL_FRONT_RIGHT,
4192 | } _ma_channel_position; /* Do not use `_ma_channel_position` directly. Use `ma_channel` instead. */
4193 |
4194 | typedef enum
4195 | {
4196 |     MA_SUCCESS                 = 0,
4197 |     MA_ERROR                   = -1, /* A generic error. */
4198 |     MA_INVALID_ARGS           = -2,
4199 |     MA_INVALID_OPERATION      = -3,
4200 |     MA_OUT_OF_MEMORY          = -4,
4201 |     MA_OUT_OF_RANGE           = -5,
4202 |     MA_ACCESS_DENIED          = -6,
4203 |     MA_DOES_NOT_EXIST         = -7,
4204 |     MA_ALREADY_EXISTS         = -8,
4205 |     MA_TOO_MANY_OPEN_FILES    = -9,
4206 |     MA_INVALID_FILE           = -10,
4207 |     MA_TOO_BIG                = -11,

```

```

4208 | MA_PATH_TOO_LONG           = -12,
4209 | MA_NAME_TOO_LONG           = -13,
4210 | MA_NOT_DIRECTORY           = -14,
4211 | MA_IS_DIRECTORY            = -15,
4212 | MA_DIRECTORY_NOT_EMPTY     = -16,
4213 | MA_AT_END                   = -17,
4214 | MA_NO_SPACE                 = -18,
4215 | MA_BUSY                     = -19,
4216 | MA_IO_ERROR                 = -20,
4217 | MA_INTERRUPT                = -21,
4218 | MA_UNAVAILABLE              = -22,
4219 | MA_ALREADY_IN_USE          = -23,
4220 | MA_BAD_ADDRESS              = -24,
4221 | MA_BAD_SEEK                 = -25,
4222 | MA_BAD_PIPE                 = -26,
4223 | MA_DEADLOCK                 = -27,
4224 | MA_TOO_MANY_LINKS          = -28,
4225 | MA_NOT_IMPLEMENTED          = -29,
4226 | MA_NO_MESSAGE               = -30,
4227 | MA_BAD_MESSAGE              = -31,
4228 | MA_NO_DATA_AVAILABLE       = -32,
4229 | MA_INVALID_DATA             = -33,
4230 | MA_TIMEOUT                  = -34,
4231 | MA_NO_NETWORK               = -35,
4232 | MA_NOT_UNIQUE               = -36,
4233 | MA_NOT_SOCKET               = -37,
4234 | MA_NO_ADDRESS               = -38,
4235 | MA_BAD_PROTOCOL             = -39,
4236 | MA_PROTOCOL_UNAVAILABLE     = -40,
4237 | MA_PROTOCOL_NOT_SUPPORTED   = -41,
4238 | MA_PROTOCOL_FAMILY_NOT_SUPPORTED = -42,
4239 | MA_ADDRESS_FAMILY_NOT_SUPPORTED = -43,
4240 | MA_SOCKET_NOT_SUPPORTED     = -44,
4241 | MA_CONNECTION_RESET         = -45,
4242 | MA_ALREADY_CONNECTED        = -46,
4243 | MA_NOT_CONNECTED            = -47,
4244 | MA_CONNECTION_REFUSED       = -48,
4245 | MA_NO_HOST                   = -49,
4246 | MA_IN_PROGRESS              = -50,
4247 | MA_CANCELLED                 = -51,
4248 | MA_MEMORY_ALREADY_MAPPED    = -52,
4249 |
4250 | /* General non-standard errors. */
4251 | MA_CRC_MISMATCH              = -100,
4252 |
4253 | /* General miniaudio-specific errors. */
4254 | MA_FORMAT_NOT_SUPPORTED      = -200,
4255 | MA_DEVICE_TYPE_NOT_SUPPORTED = -201,
4256 | MA_SHARE_MODE_NOT_SUPPORTED  = -202,
4257 | MA_NO_BACKEND                = -203,
4258 | MA_NO_DEVICE                 = -204,
4259 | MA_API_NOT_FOUND             = -205,
4260 | MA_INVALID_DEVICE_CONFIG     = -206,
4261 | MA_LOOP                      = -207,
4262 | MA_BACKEND_NOT_ENABLED       = -208,
4263 |
4264 | /* State errors. */
4265 | MA_DEVICE_NOT_INITIALIZED    = -300,
4266 | MA_DEVICE_ALREADY_INITIALIZED = -301,
4267 | MA_DEVICE_NOT_STARTED        = -302,
4268 | MA_DEVICE_NOT_STOPPED        = -303,
4269 |
4270 | /* Operation errors. */
4271 | MA_FAILED_TO_INIT_BACKEND    = -400,
4272 | MA_FAILED_TO_OPEN_BACKEND_DEVICE = -401,
4273 | MA_FAILED_TO_START_BACKEND_DEVICE = -402,
4274 | MA_FAILED_TO_STOP_BACKEND_DEVICE = -403
4275 | } ma_result;
4276 |
4277 |
4278 | #define MA_MIN_CHANNELS      1
4279 | #ifndef MA_MAX_CHANNELS
4280 | #define MA_MAX_CHANNELS      254
4281 | #endif
4282 |
4283 | #ifndef MA_MAX_FILTER_ORDER
4284 | #define MA_MAX_FILTER_ORDER  8
4285 | #endif
4286 |

```

```

4287 | typedef enum
4288 | {
4289 |     ma_stream_format_pcm = 0
4290 | } ma_stream_format;
4291 |
4292 | typedef enum
4293 | {
4294 |     ma_stream_layout_interleaved = 0,
4295 |     ma_stream_layout_deinterleaved
4296 | } ma_stream_layout;
4297 |
4298 | typedef enum
4299 | {
4300 |     ma_dither_mode_none = 0,
4301 |     ma_dither_mode_rectangle,
4302 |     ma_dither_mode_triangle
4303 | } ma_dither_mode;
4304 |
4305 | typedef enum
4306 | {
4307 |     /*
4308 |     I like to keep these explicitly defined because they're used as a key into a lookup table. When
4309 |     > items are added to this, make sure there are no gaps and that they're added to the lookup table in
4310 |     > ma_get_bytes_per_sample().
4311 |     */
4312 |     ma_format_unknown = 0, /* Mainly used for indicating an error, but also used as the default
4313 |     > for the output format for decoders. */
4314 |     ma_format_u8 = 1,
4315 |     ma_format_s16 = 2, /* Seems to be the most widely supported format. */
4316 |     ma_format_s24 = 3, /* Tightly packed. 3 bytes per sample. */
4317 |     ma_format_s32 = 4,
4318 |     ma_format_f32 = 5,
4319 |     ma_format_count
4320 | } ma_format;
4321 |
4322 | typedef enum
4323 | {
4324 |     /* Standard rates need to be in priority order. */
4325 |     ma_standard_sample_rate_48000 = 48000, /* Most common */
4326 |     ma_standard_sample_rate_44100 = 44100,
4327 |
4328 |     ma_standard_sample_rate_32000 = 32000, /* Lows */
4329 |     ma_standard_sample_rate_24000 = 24000,
4330 |     ma_standard_sample_rate_22050 = 22050,
4331 |
4332 |     ma_standard_sample_rate_88200 = 88200, /* Highs */
4333 |     ma_standard_sample_rate_96000 = 96000,
4334 |     ma_standard_sample_rate_176400 = 176400,
4335 |     ma_standard_sample_rate_192000 = 192000,
4336 |
4337 |     ma_standard_sample_rate_16000 = 16000, /* Extreme lows */
4338 |     ma_standard_sample_rate_11025 = 11025,
4339 |     ma_standard_sample_rate_8000 = 8000,
4340 |
4341 |     ma_standard_sample_rate_352800 = 352800, /* Extreme highs */
4342 |     ma_standard_sample_rate_384000 = 384000,
4343 |
4344 |     ma_standard_sample_rate_min = ma_standard_sample_rate_8000,
4345 |     ma_standard_sample_rate_max = ma_standard_sample_rate_384000,
4346 |     ma_standard_sample_rate_count = 14 /* Need to maintain the count manually. Make sure
4347 |     > this is updated if items are added to enum. */
4348 | } ma_standard_sample_rate;
4349 |
4350 | typedef enum
4351 | {
4352 |     ma_channel_mix_mode_rectangular = 0, /* Simple averaging based on the plane(s) the channel is
4353 |     > sitting on. */
4354 |     ma_channel_mix_mode_simple, /* Drop excess channels; zeroed out extra channels. */
4355 |     ma_channel_mix_mode_custom_weights, /* Use custom weights specified in
4356 |     > ma_channel_converter_config. */
4357 |     ma_channel_mix_mode_default = ma_channel_mix_mode_rectangular
4358 | } ma_channel_mix_mode;
4359 |
4360 | typedef enum
4361 | {
4362 |     ma_standard_channel_map_microsoft,
4363 |     ma_standard_channel_map_alsa,

```



```

4360 |     ma_standard_channel_map_rfc3551,    /* Based off AIFF. */
4361 |     ma_standard_channel_map_flac,
4362 |     ma_standard_channel_map_vorbis,
4363 |     ma_standard_channel_map_sound4,    /* FreeBSD's sound(4). */
4364 |     ma_standard_channel_map_sndio,    /* www.sndio.org/tips.html */
4365 |     ma_standard_channel_map_webaudio = ma_standard_channel_map_flac, /*
> https://webaudio.github.io/web-audio-api/#ChannelOrdering. Only 1, 2, 4 and 6 channels are defined,
> but can fill in the gaps with logical assumptions. */
4366 |     ma_standard_channel_map_default = ma_standard_channel_map_microsoft
4367 | } ma_standard_channel_map;
4368 |
4369 | typedef enum
4370 | {
4371 |     ma_performance_profile_low_latency = 0,
4372 |     ma_performance_profile_conservative
4373 | } ma_performance_profile;
4374 |
4375 |
4376 | typedef struct
4377 | {
4378 |     void* pUserData;
4379 |     void* (* onMalloc)(size_t sz, void* pUserData);
4380 |     void* (* onRealloc)(void* p, size_t sz, void* pUserData);
4381 |     void (* onFree)(void* p, void* pUserData);
4382 | } ma_allocation_callbacks;
4383 |
4384 | typedef struct
4385 | {
4386 |     ma_uint32 state;
4387 | } ma_lcg;
4388 |
4389 |
4390 | /*
4391 | Atomics.
4392 |
4393 | These are typesafe structures to prevent errors as a result of forgetting to reference variables
> atomically. It's too
4394 | easy to introduce subtle bugs where you accidentally do a regular assignment instead of an atomic
> load/store, etc. By
4395 | using a struct we can enforce the use of atomics at compile time.
4396 |
4397 | These types are declared in the header section because we need to reference them in structs below,
> but functions for
4398 | using them are only exposed in the implementation section. I do not want these to be part of the
> public API.
4399 |
4400 | There's a few downsides to this system. The first is that you need to declare a new struct for each
> type. Below are
4401 | some macros to help with the declarations. They will be named like so:
4402 |
4403 |     ma_atomic_uint32 - atomic ma_uint32
4404 |     ma_atomic_int32  - atomic ma_int32
4405 |     ma_atomic_uint64 - atomic ma_uint64
4406 |     ma_atomic_float  - atomic float
4407 |     ma_atomic_bool32 - atomic ma_bool32
4408 |
4409 | The other downside is that atomic pointers are extremely messy. You need to declare a new struct for
> each specific
4410 | type of pointer you need to make atomic. For example, an atomic ma_node* will look like this:
4411 |
4412 |     MA_ATOMIC_SAFE_TYPE_IMPL_PTR(node)
4413 |
4414 | Which will declare a type struct that's named like so:
4415 |
4416 |     ma_atomic_ptr_node
4417 |
4418 | Functions to use the atomic types are declared in the implementation section. All atomic functions
> are prefixed with
4419 | the name of the struct. For example:
4420 |
4421 |     ma_atomic_uint32_set() - Atomic store of ma_uint32
4422 |     ma_atomic_uint32_get() - Atomic load of ma_uint32
4423 |     etc.
4424 |
4425 | For pointer types it's the same, which makes them a bit messy to use due to the length of each
> function name, but in
4426 | return you get type safety and enforcement of atomic operations.
4427 | */
4428 | #define MA_ATOMIC_SAFE_TYPE_DECL(c89TypeExtension, typeSize, type) \

```

```

4429 |     typedef struct \
4430 |     { \
4431 |         MA_ATOMIC(typeSize, ma_##type) value; \
4432 |     } ma_atomic_##type; \
4433 |
4434 | #define MA_ATOMIC_SAFE_TYPE_DECL_PTR(type) \
4435 |     typedef struct \
4436 |     { \
4437 |         MA_ATOMIC(MA_SIZEOF_PTR, ma_##type*) value; \
4438 |     } ma_atomic_ptr_##type; \
4439 |
4440 | MA_ATOMIC_SAFE_TYPE_DECL(32, 4, uint32)
4441 | MA_ATOMIC_SAFE_TYPE_DECL(i32, 4, int32)
4442 | MA_ATOMIC_SAFE_TYPE_DECL(64, 8, uint64)
4443 | MA_ATOMIC_SAFE_TYPE_DECL(f32, 4, float)
4444 | MA_ATOMIC_SAFE_TYPE_DECL(32, 4, bool32)
4445 |
4446 |
4447 | /* Spinlocks are 32-bit for compatibility reasons. */
4448 | typedef ma_uint32 ma_spinlock;
4449 |
4450 | #ifndef MA_NO_THREADING
4451 | /* Thread priorities should be ordered such that the default priority of the worker thread is 0.
> */
4452 |     typedef enum
4453 |     {
4454 |         ma_thread_priority_idle     = -5,
4455 |         ma_thread_priority_lowest   = -4,
4456 |         ma_thread_priority_low      = -3,
4457 |         ma_thread_priority_normal   = -2,
4458 |         ma_thread_priority_high     = -1,
4459 |         ma_thread_priority_highest  = 0,
4460 |         ma_thread_priority_realtime = 1,
4461 |         ma_thread_priority_default  = 0
4462 |     } ma_thread_priority;
4463 |
4464 |     #if defined(MA_POSIX)
4465 |         typedef ma_pthread_t ma_thread;
4466 |     #elif defined(MA_WIN32)
4467 |         typedef ma_handle ma_thread;
4468 |     #endif
4469 |
4470 |     #if defined(MA_POSIX)
4471 |         typedef ma_pthread_mutex_t ma_mutex;
4472 |     #elif defined(MA_WIN32)
4473 |         typedef ma_handle ma_mutex;
4474 |     #endif
4475 |
4476 |     #if defined(MA_POSIX)
4477 |         typedef struct
4478 |         {
4479 |             ma_uint32 value;
4480 |             ma_pthread_mutex_t lock;
4481 |             ma_pthread_cond_t cond;
4482 |         } ma_event;
4483 |     #elif defined(MA_WIN32)
4484 |         typedef ma_handle ma_event;
4485 |     #endif
4486 |
4487 |     #if defined(MA_POSIX)
4488 |         typedef struct
4489 |         {
4490 |             int value;
4491 |             ma_pthread_mutex_t lock;
4492 |             ma_pthread_cond_t cond;
4493 |         } ma_semaphore;
4494 |     #elif defined(MA_WIN32)
4495 |         typedef ma_handle ma_semaphore;
4496 |     #endif
4497 | #else
4498 | /* MA_NO_THREADING is set which means threading is disabled. Threading is required by some API
> families. If any of these are enabled we need to throw an error. */
4499 |     #ifndef MA_NO_DEVICE_IO
4500 |         #error "MA_NO_THREADING cannot be used without MA_NO_DEVICE_IO";
4501 |     #endif
4502 | #endif /* MA_NO_THREADING */
4503 |
4504 |
4505 | /*

```

```

4506 | Retrieves the version of miniaudio as separated integers. Each component can be NULL if it's not
> required.
4507 | */
4508 | MA_API void ma_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision);
4509 |
4510 | /*
4511 | Retrieves the version of miniaudio as a string which can be useful for logging purposes.
4512 | */
4513 | MA_API const char* ma_version_string(void);
4514 |
4515 |
4516 | /*****
> *****/
4517 |
4518 | Logging
4519 |
4520 | *****/
> *****/
4521 | #include <stdarg.h> /* For va_list. */
4522 |
4523 | #if defined(__has_attribute)
4524 |     #if __has_attribute(format)
4525 |         #define MA_ATTRIBUTE_FORMAT(fmt, va) __attribute__((format(printf, fmt, va)))
4526 |     #endif
4527 | #endif
4528 | #ifndef MA_ATTRIBUTE_FORMAT
4529 |     #define MA_ATTRIBUTE_FORMAT(fmt, va)
4530 | #endif
4531 |
4532 | #ifndef MA_MAX_LOG_CALLBACKS
4533 |     #define MA_MAX_LOG_CALLBACKS 4
4534 | #endif
4535 |
4536 |
4537 | /*
4538 | The callback for handling log messages.
4539 |
4540 |
4541 | Parameters
4542 | -----
4543 | pUserData (in)
4544 |     The user data pointer that was passed into ma_log_register_callback().
4545 |
4546 | logLevel (in)
4547 |     The log level. This can be one of the following:
4548 |
4549 |     +-----+
4550 |     | Log Level          |
4551 |     +-----+
4552 |     | MA_LOG_LEVEL_DEBUG |
4553 |     | MA_LOG_LEVEL_INFO  |
4554 |     | MA_LOG_LEVEL_WARNING |
4555 |     | MA_LOG_LEVEL_ERROR |
4556 |     +-----+
4557 |
4558 | pMessage (in)
4559 |     The log message.
4560 | */
4561 | typedef void (* ma_log_callback_proc)(void* pUserData, ma_uint32 level, const char* pMessage);
4562 |
4563 | typedef struct
4564 | {
4565 |     ma_log_callback_proc onLog;
4566 |     void* pUserData;
4567 | } ma_log_callback;
4568 |
4569 | MA_API ma_log_callback ma_log_callback_init(ma_log_callback_proc onLog, void* pUserData);
4570 |
4571 |
4572 | typedef struct
4573 | {
4574 |     ma_log_callback callbacks[MA_MAX_LOG_CALLBACKS];
4575 |     ma_uint32 callbackCount;
4576 |     ma_allocation_callbacks allocationCallbacks; /* Need to store these persistently because
> ma_log_postv() might need to allocate a buffer on the heap. */
4577 | #ifndef MA_NO_THREADING
4578 |     ma_mutex lock; /* For thread safety just to make it easier and safer for the logging
> implementation. */
4579 | #endif

```

```

4580 | } ma_log;
4581 |
4582 | MA_API ma_result ma_log_init(const ma_allocation_callbacks* pAllocationCallbacks, ma_log* pLog);
4583 | MA_API void ma_log_uninit(ma_log* pLog);
4584 | MA_API ma_result ma_log_register_callback(ma_log* pLog, ma_log_callback callback);
4585 | MA_API ma_result ma_log_unregister_callback(ma_log* pLog, ma_log_callback callback);
4586 | MA_API ma_result ma_log_post(ma_log* pLog, ma_uint32 level, const char* pMessage);
4587 | MA_API ma_result ma_log_postv(ma_log* pLog, ma_uint32 level, const char* pFormat, va_list args);
4588 | MA_API ma_result ma_log_postf(ma_log* pLog, ma_uint32 level, const char* pFormat, ...)
> MA_ATTRIBUTE_FORMAT(3, 4);
4589 |
4590 |
4591 | /*****
> *****/
4592 |
4593 | Biquad Filtering
4594 |
4595 | *****/
> *****/
4596 | typedef union
4597 | {
4598 |     float    f32;
4599 |     ma_int32 s32;
4600 | } ma_biquad_coefficient;
4601 |
4602 | typedef struct
4603 | {
4604 |     ma_format format;
4605 |     ma_uint32 channels;
4606 |     double b0;
4607 |     double b1;
4608 |     double b2;
4609 |     double a0;
4610 |     double a1;
4611 |     double a2;
4612 | } ma_biquad_config;
4613 |
4614 | MA_API ma_biquad_config ma_biquad_config_init(ma_format format, ma_uint32 channels, double b0,
> double b1, double b2, double a0, double a1, double a2);
4615 |
4616 | typedef struct
4617 | {
4618 |     ma_format format;
4619 |     ma_uint32 channels;
4620 |     ma_biquad_coefficient b0;
4621 |     ma_biquad_coefficient b1;
4622 |     ma_biquad_coefficient b2;
4623 |     ma_biquad_coefficient a1;
4624 |     ma_biquad_coefficient a2;
4625 |     ma_biquad_coefficient* pR1;
4626 |     ma_biquad_coefficient* pR2;
4627 |
4628 |     /* Memory management. */
4629 |     void* _pHeap;
4630 |     ma_bool32 _ownsHeap;
4631 | } ma_biquad;
4632 |
4633 | MA_API ma_result ma_biquad_get_heap_size(const ma_biquad_config* pConfig, size_t* pHeapSizeInBytes);
4634 | MA_API ma_result ma_biquad_init_preallocated(const ma_biquad_config* pConfig, void* pHeap,
> ma_biquad* pBQ);
4635 | MA_API ma_result ma_biquad_init(const ma_biquad_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_biquad* pBQ);
4636 | MA_API void ma_biquad_uninit(ma_biquad* pBQ, const ma_allocation_callbacks* pAllocationCallbacks);
4637 | MA_API ma_result ma_biquad_reinit(const ma_biquad_config* pConfig, ma_biquad* pBQ);
4638 | MA_API ma_result ma_biquad_clear_cache(ma_biquad* pBQ);
4639 | MA_API ma_result ma_biquad_process_pcm_frames(ma_biquad* pBQ, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
4640 | MA_API ma_uint32 ma_biquad_get_latency(const ma_biquad* pBQ);
4641 |
4642 |
4643 | /*****
> *****/
4644 |
4645 | Low-Pass Filtering
4646 |
4647 | *****/
> *****/
4648 | typedef struct
4649 | {

```

```

4650 |     ma_format format;
4651 |     ma_uint32 channels;
4652 |     ma_uint32 sampleRate;
4653 |     double cutoffFrequency;
4654 |     double q;
4655 | } ma_lpf1_config, ma_lpf2_config;
4656 |
4657 | MA_API ma_lpf1_config ma_lpf1_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency);
4658 | MA_API ma_lpf2_config ma_lpf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency, double q);
4659 |
4660 | typedef struct
4661 | {
4662 |     ma_format format;
4663 |     ma_uint32 channels;
4664 |     ma_biquad_coefficient a;
4665 |     ma_biquad_coefficient* pR1;
4666 |
4667 |     /* Memory management. */
4668 |     void* _pHeap;
4669 |     ma_bool32 _ownsHeap;
4670 | } ma_lpf1;
4671 |
4672 | MA_API ma_result ma_lpf1_get_heap_size(const ma_lpf1_config* pConfig, size_t* pHeapSizeInBytes);
4673 | MA_API ma_result ma_lpf1_init_preallocated(const ma_lpf1_config* pConfig, void* pHeap, ma_lpf1*
> pLPF);
4674 | MA_API ma_result ma_lpf1_init(const ma_lpf1_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_lpf1* pLPF);
4675 | MA_API void ma_lpf1_uninit(ma_lpf1* pLPF, const ma_allocation_callbacks* pAllocationCallbacks);
4676 | MA_API ma_result ma_lpf1_reinit(const ma_lpf1_config* pConfig, ma_lpf1* pLPF);
4677 | MA_API ma_result ma_lpf1_clear_cache(ma_lpf1* pLPF);
4678 | MA_API ma_result ma_lpf1_process_pcm_frames(ma_lpf1* pLPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4679 | MA_API ma_uint32 ma_lpf1_get_latency(const ma_lpf1* pLPF);
4680 |
4681 | typedef struct
4682 | {
4683 |     ma_biquad bq; /* The second order low-pass filter is implemented as a biquad filter. */
4684 | } ma_lpf2;
4685 |
4686 | MA_API ma_result ma_lpf2_get_heap_size(const ma_lpf2_config* pConfig, size_t* pHeapSizeInBytes);
4687 | MA_API ma_result ma_lpf2_init_preallocated(const ma_lpf2_config* pConfig, void* pHeap, ma_lpf2*
> pHPF);
4688 | MA_API ma_result ma_lpf2_init(const ma_lpf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_lpf2* pLPF);
4689 | MA_API void ma_lpf2_uninit(ma_lpf2* pLPF, const ma_allocation_callbacks* pAllocationCallbacks);
4690 | MA_API ma_result ma_lpf2_reinit(const ma_lpf2_config* pConfig, ma_lpf2* pLPF);
4691 | MA_API ma_result ma_lpf2_clear_cache(ma_lpf2* pLPF);
4692 | MA_API ma_result ma_lpf2_process_pcm_frames(ma_lpf2* pLPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4693 | MA_API ma_uint32 ma_lpf2_get_latency(const ma_lpf2* pLPF);
4694 |
4695 |
4696 | typedef struct
4697 | {
4698 |     ma_format format;
4699 |     ma_uint32 channels;
4700 |     ma_uint32 sampleRate;
4701 |     double cutoffFrequency;
4702 |     ma_uint32 order; /* If set to 0, will be treated as a passthrough (no filtering will be
> applied). */
4703 | } ma_lpf_config;
4704 |
4705 | MA_API ma_lpf_config ma_lpf_config_init(ma_format format, ma_uint32 channels, ma_uint32 sampleRate,
> double cutoffFrequency, ma_uint32 order);
4706 |
4707 | typedef struct
4708 | {
4709 |     ma_format format;
4710 |     ma_uint32 channels;
4711 |     ma_uint32 sampleRate;
4712 |     ma_uint32 lpf1Count;
4713 |     ma_uint32 lpf2Count;
4714 |     ma_lpf1* pLPF1;
4715 |     ma_lpf2* pLPF2;
4716 |
4717 |     /* Memory management. */
4718 |     void* _pHeap;

```

```

4719 |     ma_bool32 _ownsHeap;
4720 | } ma_lpf;
4721 |
4722 | MA_API ma_result ma_lpf_get_heap_size(const ma_lpf_config* pConfig, size_t* pHeapSizeInBytes);
4723 | MA_API ma_result ma_lpf_init_preallocated(const ma_lpf_config* pConfig, void* pHeap, ma_lpf* pLPF);
4724 | MA_API ma_result ma_lpf_init(const ma_lpf_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_lpf* pLPF);
4725 | MA_API void ma_lpf_uninit(ma_lpf* pLPF, const ma_allocation_callbacks* pAllocationCallbacks);
4726 | MA_API ma_result ma_lpf_reinit(const ma_lpf_config* pConfig, ma_lpf* pLPF);
4727 | MA_API ma_result ma_lpf_clear_cache(ma_lpf* pLPF);
4728 | MA_API ma_result ma_lpf_process_pcm_frames(ma_lpf* pLPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4729 | MA_API ma_uint32 ma_lpf_get_latency(const ma_lpf* pLPF);
4730 |
4731 |
4732 | /*****
> *****/
4733 |
4734 | High-Pass Filtering
4735 |
4736 | *****/
> *****/
4737 | typedef struct
4738 | {
4739 |     ma_format format;
4740 |     ma_uint32 channels;
4741 |     ma_uint32 sampleRate;
4742 |     double cutoffFrequency;
4743 |     double q;
4744 | } ma_hpf1_config, ma_hpf2_config;
4745 |
4746 | MA_API ma_hpf1_config ma_hpf1_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency);
4747 | MA_API ma_hpf2_config ma_hpf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency, double q);
4748 |
4749 | typedef struct
4750 | {
4751 |     ma_format format;
4752 |     ma_uint32 channels;
4753 |     ma_biquad_coefficient a;
4754 |     ma_biquad_coefficient* pR1;
4755 |
4756 |     /* Memory management. */
4757 |     void* _pHeap;
4758 |     ma_bool32 _ownsHeap;
4759 | } ma_hpf1;
4760 |
4761 | MA_API ma_result ma_hpf1_get_heap_size(const ma_hpf1_config* pConfig, size_t* pHeapSizeInBytes);
4762 | MA_API ma_result ma_hpf1_init_preallocated(const ma_hpf1_config* pConfig, void* pHeap, ma_hpf1*
> pLPF);
4763 | MA_API ma_result ma_hpf1_init(const ma_hpf1_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hpf1* pHPF);
4764 | MA_API void ma_hpf1_uninit(ma_hpf1* pHPF, const ma_allocation_callbacks* pAllocationCallbacks);
4765 | MA_API ma_result ma_hpf1_reinit(const ma_hpf1_config* pConfig, ma_hpf1* pHPF);
4766 | MA_API ma_result ma_hpf1_process_pcm_frames(ma_hpf1* pHPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4767 | MA_API ma_uint32 ma_hpf1_get_latency(const ma_hpf1* pHPF);
4768 |
4769 | typedef struct
4770 | {
4771 |     ma_biquad bq; /* The second order high-pass filter is implemented as a biquad filter. */
4772 | } ma_hpf2;
4773 |
4774 | MA_API ma_result ma_hpf2_get_heap_size(const ma_hpf2_config* pConfig, size_t* pHeapSizeInBytes);
4775 | MA_API ma_result ma_hpf2_init_preallocated(const ma_hpf2_config* pConfig, void* pHeap, ma_hpf2*
> pHPF);
4776 | MA_API ma_result ma_hpf2_init(const ma_hpf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hpf2* pHPF);
4777 | MA_API void ma_hpf2_uninit(ma_hpf2* pHPF, const ma_allocation_callbacks* pAllocationCallbacks);
4778 | MA_API ma_result ma_hpf2_reinit(const ma_hpf2_config* pConfig, ma_hpf2* pHPF);
4779 | MA_API ma_result ma_hpf2_process_pcm_frames(ma_hpf2* pHPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4780 | MA_API ma_uint32 ma_hpf2_get_latency(const ma_hpf2* pHPF);
4781 |
4782 |
4783 | typedef struct
4784 | {
4785 |     ma_format format;

```

```

4786 |     ma_uint32 channels;
4787 |     ma_uint32 sampleRate;
4788 |     double cutoffFrequency;
4789 |     ma_uint32 order;    /* If set to 0, will be treated as a passthrough (no filtering will be
> applied). */
4790 | } ma_hpf_config;
4791 |
4792 | MA_API ma_hpf_config ma_hpf_config_init(ma_format format, ma_uint32 channels, ma_uint32 sampleRate,
> double cutoffFrequency, ma_uint32 order);
4793 |
4794 | typedef struct
4795 | {
4796 |     ma_format format;
4797 |     ma_uint32 channels;
4798 |     ma_uint32 sampleRate;
4799 |     ma_uint32 hpf1Count;
4800 |     ma_uint32 hpf2Count;
4801 |     ma_hpf1* pHPF1;
4802 |     ma_hpf2* pHPF2;
4803 |
4804 |     /* Memory management. */
4805 |     void* _pHeap;
4806 |     ma_bool32 _ownsHeap;
4807 | } ma_hpf;
4808 |
4809 | MA_API ma_result ma_hpf_get_heap_size(const ma_hpf_config* pConfig, size_t* pHeapSizeInBytes);
4810 | MA_API ma_result ma_hpf_init_preallocated(const ma_hpf_config* pConfig, void* pHeap, ma_hpf* pLPF);
4811 | MA_API ma_result ma_hpf_init(const ma_hpf_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hpf* pHPF);
4812 | MA_API void ma_hpf_uninit(ma_hpf* pHPF, const ma_allocation_callbacks* pAllocationCallbacks);
4813 | MA_API ma_result ma_hpf_reinit(const ma_hpf_config* pConfig, ma_hpf* pHPF);
4814 | MA_API ma_result ma_hpf_process_pcm_frames(ma_hpf* pHPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4815 | MA_API ma_uint32 ma_hpf_get_latency(const ma_hpf* pHPF);
4816 |
4817 |
4818 | /*****
> *****/
4819 |
4820 | Band-Pass Filtering
4821 |
4822 | *****/
> *****/
4823 | typedef struct
4824 | {
4825 |     ma_format format;
4826 |     ma_uint32 channels;
4827 |     ma_uint32 sampleRate;
4828 |     double cutoffFrequency;
4829 |     double q;
4830 | } ma_bpf2_config;
4831 |
4832 | MA_API ma_bpf2_config ma_bpf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency, double q);
4833 |
4834 | typedef struct
4835 | {
4836 |     ma_biquad bq;    /* The second order band-pass filter is implemented as a biquad filter. */
4837 | } ma_bpf2;
4838 |
4839 | MA_API ma_result ma_bpf2_get_heap_size(const ma_bpf2_config* pConfig, size_t* pHeapSizeInBytes);
4840 | MA_API ma_result ma_bpf2_init_preallocated(const ma_bpf2_config* pConfig, void* pHeap, ma_bpf2*
> pBPF);
4841 | MA_API ma_result ma_bpf2_init(const ma_bpf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_bpf2* pBPF);
4842 | MA_API void ma_bpf2_uninit(ma_bpf2* pBPF, const ma_allocation_callbacks* pAllocationCallbacks);
4843 | MA_API ma_result ma_bpf2_reinit(const ma_bpf2_config* pConfig, ma_bpf2* pBPF);
4844 | MA_API ma_result ma_bpf2_process_pcm_frames(ma_bpf2* pBPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4845 | MA_API ma_uint32 ma_bpf2_get_latency(const ma_bpf2* pBPF);
4846 |
4847 |
4848 | typedef struct
4849 | {
4850 |     ma_format format;
4851 |     ma_uint32 channels;
4852 |     ma_uint32 sampleRate;
4853 |     double cutoffFrequency;
4854 |     ma_uint32 order;    /* If set to 0, will be treated as a passthrough (no filtering will be

```



```

> applied). */
4855 | } ma_bpf_config;
4856 |
4857 | MA_API ma_bpf_config ma_bpf_config_init(ma_format format, ma_uint32 channels, ma_uint32 sampleRate,
> double cutoffFrequency, ma_uint32 order);
4858 |
4859 | typedef struct
4860 | {
4861 |     ma_format format;
4862 |     ma_uint32 channels;
4863 |     ma_uint32 bpf2Count;
4864 |     ma_bpf2* pBPF2;
4865 |
4866 |     /* Memory management. */
4867 |     void* _pHeap;
4868 |     ma_bool32 _ownsHeap;
4869 | } ma_bpf;
4870 |
4871 | MA_API ma_result ma_bpf_get_heap_size(const ma_bpf_config* pConfig, size_t* pHeapSizeInBytes);
4872 | MA_API ma_result ma_bpf_init_preallocated(const ma_bpf_config* pConfig, void* pHeap, ma_bpf* pBPF);
4873 | MA_API ma_result ma_bpf_init(const ma_bpf_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_bpf* pBPF);
4874 | MA_API void ma_bpf_uninit(ma_bpf* pBPF, const ma_allocation_callbacks* pAllocationCallbacks);
4875 | MA_API ma_result ma_bpf_reinit(const ma_bpf_config* pConfig, ma_bpf* pBPF);
4876 | MA_API ma_result ma_bpf_process_pcm_frames(ma_bpf* pBPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount);
4877 | MA_API ma_uint32 ma_bpf_get_latency(const ma_bpf* pBPF);
4878 |
4879 |
4880 | /*****
> *****/
4881 |
4882 | Notching Filter
4883 |
4884 | *****/
> *****/
4885 | typedef struct
4886 | {
4887 |     ma_format format;
4888 |     ma_uint32 channels;
4889 |     ma_uint32 sampleRate;
4890 |     double q;
4891 |     double frequency;
4892 | } ma_notch2_config, ma_notch_config;
4893 |
4894 | MA_API ma_notch2_config ma_notch2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double q, double frequency);
4895 |
4896 | typedef struct
4897 | {
4898 |     ma_biquad bq;
4899 | } ma_notch2;
4900 |
4901 | MA_API ma_result ma_notch2_get_heap_size(const ma_notch2_config* pConfig, size_t* pHeapSizeInBytes);
4902 | MA_API ma_result ma_notch2_init_preallocated(const ma_notch2_config* pConfig, void* pHeap,
> ma_notch2* pFilter);
4903 | MA_API ma_result ma_notch2_init(const ma_notch2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_notch2* pFilter);
4904 | MA_API void ma_notch2_uninit(ma_notch2* pFilter, const ma_allocation_callbacks*
> pAllocationCallbacks);
4905 | MA_API ma_result ma_notch2_reinit(const ma_notch2_config* pConfig, ma_notch2* pFilter);
4906 | MA_API ma_result ma_notch2_process_pcm_frames(ma_notch2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
4907 | MA_API ma_uint32 ma_notch2_get_latency(const ma_notch2* pFilter);
4908 |
4909 |
4910 | /*****
> *****/
4911 |
4912 | Peaking EQ Filter
4913 |
4914 | *****/
> *****/
4915 | typedef struct
4916 | {
4917 |     ma_format format;
4918 |     ma_uint32 channels;
4919 |     ma_uint32 sampleRate;
4920 |     double gainDB;

```

```

4921 |     double q;
4922 |     double frequency;
4923 | } ma_peak2_config, ma_peak_config;
4924 |
4925 | MA_API ma_peak2_config ma_peak2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double gainDB, double q, double frequency);
4926 |
4927 | typedef struct
4928 | {
4929 |     ma_biquad bq;
4930 | } ma_peak2;
4931 |
4932 | MA_API ma_result ma_peak2_get_heap_size(const ma_peak2_config* pConfig, size_t* pHeapSizeInBytes);
4933 | MA_API ma_result ma_peak2_init_preallocated(const ma_peak2_config* pConfig, void* pHeap, ma_peak2*
> pFilter);
4934 | MA_API ma_result ma_peak2_init(const ma_peak2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_peak2* pFilter);
4935 | MA_API void ma_peak2_uninit(ma_peak2* pFilter, const ma_allocation_callbacks* pAllocationCallbacks);
4936 | MA_API ma_result ma_peak2_reinit(const ma_peak2_config* pConfig, ma_peak2* pFilter);
4937 | MA_API ma_result ma_peak2_process_pcm_frames(ma_peak2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
4938 | MA_API ma_uint32 ma_peak2_get_latency(const ma_peak2* pFilter);
4939 |
4940 |
4941 | /*****
> *****/
4942 |
4943 | Low Shelf Filter
4944 |
4945 | *****/
> *****/
4946 | typedef struct
4947 | {
4948 |     ma_format format;
4949 |     ma_uint32 channels;
4950 |     ma_uint32 sampleRate;
4951 |     double gainDB;
4952 |     double shelfSlope;
4953 |     double frequency;
4954 | } ma_loshelf2_config, ma_loshelf_config;
4955 |
4956 | MA_API ma_loshelf2_config ma_loshelf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double gainDB, double shelfSlope, double frequency);
4957 |
4958 | typedef struct
4959 | {
4960 |     ma_biquad bq;
4961 | } ma_loshelf2;
4962 |
4963 | MA_API ma_result ma_loshelf2_get_heap_size(const ma_loshelf2_config* pConfig, size_t*
> pHeapSizeInBytes);
4964 | MA_API ma_result ma_loshelf2_init_preallocated(const ma_loshelf2_config* pConfig, void* pHeap,
> ma_loshelf2* pFilter);
4965 | MA_API ma_result ma_loshelf2_init(const ma_loshelf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_loshelf2* pFilter);
4966 | MA_API void ma_loshelf2_uninit(ma_loshelf2* pFilter, const ma_allocation_callbacks*
> pAllocationCallbacks);
4967 | MA_API ma_result ma_loshelf2_reinit(const ma_loshelf2_config* pConfig, ma_loshelf2* pFilter);
4968 | MA_API ma_result ma_loshelf2_process_pcm_frames(ma_loshelf2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
4969 | MA_API ma_uint32 ma_loshelf2_get_latency(const ma_loshelf2* pFilter);
4970 |
4971 |
4972 | /*****
> *****/
4973 |
4974 | High Shelf Filter
4975 |
4976 | *****/
> *****/
4977 | typedef struct
4978 | {
4979 |     ma_format format;
4980 |     ma_uint32 channels;
4981 |     ma_uint32 sampleRate;
4982 |     double gainDB;
4983 |     double shelfSlope;
4984 |     double frequency;
4985 | } ma_hishelf2_config, ma_hishelf_config;

```

```

4986 |
4987 | MA_API ma_hishef2_config ma_hishef2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double gainDB, double shelfSlope, double frequency);
4988 |
4989 | typedef struct
4990 | {
4991 |     ma_biquad bq;
4992 | } ma_hishef2;
4993 |
4994 | MA_API ma_result ma_hishef2_get_heap_size(const ma_hishef2_config* pConfig, size_t*
> pHeapSizeInBytes);
4995 | MA_API ma_result ma_hishef2_init_preallocated(const ma_hishef2_config* pConfig, void* pHeap,
> ma_hishef2* pFilter);
4996 | MA_API ma_result ma_hishef2_init(const ma_hishef2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hishef2* pFilter);
4997 | MA_API void ma_hishef2_uninit(ma_hishef2* pFilter, const ma_allocation_callbacks*
> pAllocationCallbacks);
4998 | MA_API ma_result ma_hishef2_reinit(const ma_hishef2_config* pConfig, ma_hishef2* pFilter);
4999 | MA_API ma_result ma_hishef2_process_pcm_frames(ma_hishef2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
5000 | MA_API ma_uint32 ma_hishef2_get_latency(const ma_hishef2* pFilter);
5001 |
5002 |
5003 |
5004 | /*
5005 | Delay
5006 | */
5007 | typedef struct
5008 | {
5009 |     ma_uint32 channels;
5010 |     ma_uint32 sampleRate;
5011 |     ma_uint32 delayInFrames;
5012 |     ma_bool32 delayStart;          /* Set to true to delay the start of the output; false otherwise. */
5013 |     float wet;                    /* 0..1. Default = 1. */
5014 |     float dry;                    /* 0..1. Default = 1. */
5015 |     float decay;                  /* 0..1. Default = 0 (no feedback). Feedback decay. Use this for
> echo. */
5016 | } ma_delay_config;
5017 |
5018 | MA_API ma_delay_config ma_delay_config_init(ma_uint32 channels, ma_uint32 sampleRate, ma_uint32
> delayInFrames, float decay);
5019 |
5020 |
5021 | typedef struct
5022 | {
5023 |     ma_delay_config config;
5024 |     ma_uint32 cursor;              /* Feedback is written to this cursor. Always equal or in front
> of the read cursor. */
5025 |     ma_uint32 bufferSizeInFrames;
5026 |     float* pBuffer;
5027 | } ma_delay;
5028 |
5029 | MA_API ma_result ma_delay_init(const ma_delay_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_delay* pDelay);
5030 | MA_API void ma_delay_uninit(ma_delay* pDelay, const ma_allocation_callbacks* pAllocationCallbacks);
5031 | MA_API ma_result ma_delay_process_pcm_frames(ma_delay* pDelay, void* pFramesOut, const void*
> pFramesIn, ma_uint32 frameCount);
5032 | MA_API void ma_delay_set_wet(ma_delay* pDelay, float value);
5033 | MA_API float ma_delay_get_wet(const ma_delay* pDelay);
5034 | MA_API void ma_delay_set_dry(ma_delay* pDelay, float value);
5035 | MA_API float ma_delay_get_dry(const ma_delay* pDelay);
5036 | MA_API void ma_delay_set_decay(ma_delay* pDelay, float value);
5037 | MA_API float ma_delay_get_decay(const ma_delay* pDelay);
5038 |
5039 |
5040 | /* Gainer for smooth volume changes. */
5041 | typedef struct
5042 | {
5043 |     ma_uint32 channels;
5044 |     ma_uint32 smoothTimeInFrames;
5045 | } ma_gainer_config;
5046 |
5047 | MA_API ma_gainer_config ma_gainer_config_init(ma_uint32 channels, ma_uint32 smoothTimeInFrames);
5048 |
5049 |
5050 | typedef struct
5051 | {
5052 |     ma_gainer_config config;
5053 |     ma_uint32 t;

```

```

5054 | float masterVolume;
5055 | float* pOldGains;
5056 | float* pNewGains;
5057 |
5058 | /* Memory management. */
5059 | void* _pHeap;
5060 | ma_bool32 _ownsHeap;
5061 | } ma_gainer;
5062 |
5063 | MA_API ma_result ma_gainer_get_heap_size(const ma_gainer_config* pConfig, size_t* pHeapSizeInBytes);
5064 | MA_API ma_result ma_gainer_init_preallocated(const ma_gainer_config* pConfig, void* pHeap,
> ma_gainer* pGainer);
5065 | MA_API ma_result ma_gainer_init(const ma_gainer_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_gainer* pGainer);
5066 | MA_API void ma_gainer_uninit(ma_gainer* pGainer, const ma_allocation_callbacks*
> pAllocationCallbacks);
5067 | MA_API ma_result ma_gainer_process_pcm_frames(ma_gainer* pGainer, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
5068 | MA_API ma_result ma_gainer_set_gain(ma_gainer* pGainer, float newGain);
5069 | MA_API ma_result ma_gainer_set_gains(ma_gainer* pGainer, float* pNewGains);
5070 | MA_API ma_result ma_gainer_set_master_volume(ma_gainer* pGainer, float volume);
5071 | MA_API ma_result ma_gainer_get_master_volume(const ma_gainer* pGainer, float* pVolume);
5072 |
5073 |
5074 |
5075 | /* Stereo panner. */
5076 | typedef enum
5077 | {
5078 |     ma_pan_mode_balance = 0, /* Does not blend one side with the other. Technically just a
> balance. Compatible with other popular audio engines and therefore the default. */
5079 |     ma_pan_mode_pan /* A true pan. The sound from one side will "move" to the other side
> and blend with it. */
5080 | } ma_pan_mode;
5081 |
5082 | typedef struct
5083 | {
5084 |     ma_format format;
5085 |     ma_uint32 channels;
5086 |     ma_pan_mode mode;
5087 |     float pan;
5088 | } ma_panner_config;
5089 |
5090 | MA_API ma_panner_config ma_panner_config_init(ma_format format, ma_uint32 channels);
5091 |
5092 |
5093 | typedef struct
5094 | {
5095 |     ma_format format;
5096 |     ma_uint32 channels;
5097 |     ma_pan_mode mode;
5098 |     float pan; /* -1..1 where 0 is no pan, -1 is left side, +1 is right side. Defaults to 0. */
5099 | } ma_panner;
5100 |
5101 | MA_API ma_result ma_panner_init(const ma_panner_config* pConfig, ma_panner* pPanner);
5102 | MA_API ma_result ma_panner_process_pcm_frames(ma_panner* pPanner, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
5103 | MA_API void ma_panner_set_mode(ma_panner* pPanner, ma_pan_mode mode);
5104 | MA_API ma_pan_mode ma_panner_get_mode(const ma_panner* pPanner);
5105 | MA_API void ma_panner_set_pan(ma_panner* pPanner, float pan);
5106 | MA_API float ma_panner_get_pan(const ma_panner* pPanner);
5107 |
5108 |
5109 |
5110 | /* Fader. */
5111 | typedef struct
5112 | {
5113 |     ma_format format;
5114 |     ma_uint32 channels;
5115 |     ma_uint32 sampleRate;
5116 | } ma_fader_config;
5117 |
5118 | MA_API ma_fader_config ma_fader_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate);
5119 |
5120 | typedef struct
5121 | {
5122 |     ma_fader_config config;
5123 |     float volumeBeg; /* If volumeBeg and volumeEnd is equal to 1, no fading happens
> (ma_fader_process_pcm_frames()) will run as a passthrough). */

```

```

5124 |     float volumeEnd;
5125 |     ma_uint64 lengthInFrames; /* The total length of the fade. */
5126 |     ma_int64 cursorInFrames; /* The current time in frames. Incremented by
> ma_fader_process_pcm_frames(). Signed because it'll be offset by startOffsetInFrames in
> set_fade_ex(). */
5127 | } ma_fader;
5128 |
5129 | MA_API ma_result ma_fader_init(const ma_fader_config* pConfig, ma_fader* pFader);
5130 | MA_API ma_result ma_fader_process_pcm_frames(ma_fader* pFader, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount);
5131 | MA_API void ma_fader_get_data_format(const ma_fader* pFader, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate);
5132 | MA_API void ma_fader_set_fade(ma_fader* pFader, float volumeBeg, float volumeEnd, ma_uint64
> lengthInFrames);
5133 | MA_API void ma_fader_set_fade_ex(ma_fader* pFader, float volumeBeg, float volumeEnd, ma_uint64
> lengthInFrames, ma_int64 startOffsetInFrames);
5134 | MA_API float ma_fader_get_current_volume(const ma_fader* pFader);
5135 |
5136 |
5137 |
5138 | /* Spatializer. */
5139 | typedef struct
5140 | {
5141 |     float x;
5142 |     float y;
5143 |     float z;
5144 | } ma_vec3f;
5145 |
5146 | typedef struct
5147 | {
5148 |     ma_vec3f v;
5149 |     ma_spinlock lock;
5150 | } ma_atomic_vec3f;
5151 |
5152 | typedef enum
5153 | {
5154 |     ma_attenuation_model_none, /* No distance attenuation and no spatialization. */
5155 |     ma_attenuation_model_inverse, /* Equivalent to OpenAL's AL_INVERSE_DISTANCE_CLAMPED. */
5156 |     ma_attenuation_model_linear, /* Linear attenuation. Equivalent to OpenAL's
> AL_LINEAR_DISTANCE_CLAMPED. */
5157 |     ma_attenuation_model_exponential /* Exponential attenuation. Equivalent to OpenAL's
> AL_EXPONENT_DISTANCE_CLAMPED. */
5158 | } ma_attenuation_model;
5159 |
5160 | typedef enum
5161 | {
5162 |     ma_positioning_absolute,
5163 |     ma_positioning_relative
5164 | } ma_positioning;
5165 |
5166 | typedef enum
5167 | {
5168 |     ma_handedness_right,
5169 |     ma_handedness_left
5170 | } ma_handedness;
5171 |
5172 |
5173 | typedef struct
5174 | {
5175 |     ma_uint32 channelsOut;
5176 |     ma_channel* pChannelMapOut;
5177 |     ma_handedness handedness; /* Defaults to right. Forward is -1 on the Z axis. In a left handed
> system, forward is +1 on the Z axis. */
5178 |     float coneInnerAngleInRadians;
5179 |     float coneOuterAngleInRadians;
5180 |     float coneOuterGain;
5181 |     float speedOfSound;
5182 |     ma_vec3f worldUp;
5183 | } ma_spatializer_listener_config;
5184 |
5185 | MA_API ma_spatializer_listener_config ma_spatializer_listener_config_init(ma_uint32 channelsOut);
5186 |
5187 |
5188 | typedef struct
5189 | {
5190 |     ma_spatializer_listener_config config;
5191 |     ma_atomic_vec3f position; /* The absolute position of the listener. */
5192 |     ma_atomic_vec3f direction; /* The direction the listener is facing. The world up vector is
> config.worldUp. */

```

```

5193 |     ma_atomic_vec3f velocity;
5194 |     ma_bool32 isEnabled;
5195 |
5196 |     /* Memory management. */
5197 |     ma_bool32 _ownsHeap;
5198 |     void* _pHeap;
5199 | } ma_spatializer_listener;
5200 |
5201 | MA_API ma_result ma_spatializer_listener_get_heap_size(const ma_spatializer_listener_config*
> pConfig, size_t* pHeapSizeInBytes);
5202 | MA_API ma_result ma_spatializer_listener_init_preallocated(const ma_spatializer_listener_config*
> pConfig, void* pHeap, ma_spatializer_listener* pListener);
5203 | MA_API ma_result ma_spatializer_listener_init(const ma_spatializer_listener_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_spatializer_listener* pListener);
5204 | MA_API void ma_spatializer_listener_uninit(ma_spatializer_listener* pListener, const
> ma_allocation_callbacks* pAllocationCallbacks);
5205 | MA_API ma_channel* ma_spatializer_listener_get_channel_map(ma_spatializer_listener* pListener);
5206 | MA_API void ma_spatializer_listener_set_cone(ma_spatializer_listener* pListener, float
> innerAngleInRadians, float outerAngleInRadians, float outerGain);
5207 | MA_API void ma_spatializer_listener_get_cone(const ma_spatializer_listener* pListener, float*
> pInnerAngleInRadians, float* pOuterAngleInRadians, float* pOuterGain);
5208 | MA_API void ma_spatializer_listener_set_position(ma_spatializer_listener* pListener, float x, float
> y, float z);
5209 | MA_API ma_vec3f ma_spatializer_listener_get_position(const ma_spatializer_listener* pListener);
5210 | MA_API void ma_spatializer_listener_set_direction(ma_spatializer_listener* pListener, float x, float
> y, float z);
5211 | MA_API ma_vec3f ma_spatializer_listener_get_direction(const ma_spatializer_listener* pListener);
5212 | MA_API void ma_spatializer_listener_set_velocity(ma_spatializer_listener* pListener, float x, float
> y, float z);
5213 | MA_API ma_vec3f ma_spatializer_listener_get_velocity(const ma_spatializer_listener* pListener);
5214 | MA_API void ma_spatializer_listener_set_speed_of_sound(ma_spatializer_listener* pListener, float
> speedOfSound);
5215 | MA_API float ma_spatializer_listener_get_speed_of_sound(const ma_spatializer_listener* pListener);
5216 | MA_API void ma_spatializer_listener_set_world_up(ma_spatializer_listener* pListener, float x, float
> y, float z);
5217 | MA_API ma_vec3f ma_spatializer_listener_get_world_up(const ma_spatializer_listener* pListener);
5218 | MA_API void ma_spatializer_listener_set_enabled(ma_spatializer_listener* pListener, ma_bool32
> isEnabled);
5219 | MA_API ma_bool32 ma_spatializer_listener_is_enabled(const ma_spatializer_listener* pListener);
5220 |
5221 |
5222 | typedef struct
5223 | {
5224 |     ma_uint32 channelsIn;
5225 |     ma_uint32 channelsOut;
5226 |     ma_channel* pChannelMapIn;
5227 |     ma_attenuation_model attenuationModel;
5228 |     ma_positioning positioning;
5229 |     ma_handedness handedness; /* Defaults to right. Forward is -1 on the Z axis. In a left
> handed system, forward is +1 on the Z axis. */
5230 |     float minGain;
5231 |     float maxGain;
5232 |     float minDistance;
5233 |     float maxDistance;
5234 |     float rolloff;
5235 |     float coneInnerAngleInRadians;
5236 |     float coneOuterAngleInRadians;
5237 |     float coneOuterGain;
5238 |     float dopplerFactor; /* Set to 0 to disable doppler effect. */
5239 |     float directionalAttenuationFactor; /* Set to 0 to disable directional attenuation. */
5240 |     float minSpatializationChannelGain; /* The minimal scaling factor to apply to channel gains when
> accounting for the direction of the sound relative to the listener. Must be in the range of 0..1.
> Smaller values means more aggressive directional panning, larger values means more subtle
> directional panning. */
5241 |     ma_uint32 gainSmoothTimeInFrames; /* When the gain of a channel changes during spatialization,
> the transition will be linearly interpolated over this number of frames. */
5242 | } ma_spatializer_config;
5243 |
5244 | MA_API ma_spatializer_config ma_spatializer_config_init(ma_uint32 channelsIn, ma_uint32
> channelsOut);
5245 |
5246 |
5247 | typedef struct
5248 | {
5249 |     ma_uint32 channelsIn;
5250 |     ma_uint32 channelsOut;
5251 |     ma_channel* pChannelMapIn;
5252 |     ma_attenuation_model attenuationModel;
5253 |     ma_positioning positioning;

```

```

5254 |     ma_handedness handedness;          /* Defaults to right. Forward is -1 on the Z axis. In a left
> | handed system, forward is +1 on the Z axis. */
5255 |     float minGain;
5256 |     float maxGain;
5257 |     float minDistance;
5258 |     float maxDistance;
5259 |     float rolloff;
5260 |     float coneInnerAngleInRadians;
5261 |     float coneOuterAngleInRadians;
5262 |     float coneOuterGain;
5263 |     float dopplerFactor;                /* Set to 0 to disable doppler effect. */
5264 |     float directionalAttenuationFactor; /* Set to 0 to disable directional attenuation. */
5265 |     ma_uint32 gainSmoothTimeInFrames; /* When the gain of a channel changes during spatialization,
> | the transition will be linearly interpolated over this number of frames. */
5266 |     ma_atomic_vec3f position;
5267 |     ma_atomic_vec3f direction;
5268 |     ma_atomic_vec3f velocity; /* For doppler effect. */
5269 |     float dopplerPitch; /* Will be updated by ma_spatializer_process_pcm_frames() and can be used by
> | higher level functions to apply a pitch shift for doppler effect. */
5270 |     float minSpatializationChannelGain;
5271 |     ma_gainer gainer; /* For smooth gain transitions. */
5272 |     float* pNewChannelGainsOut; /* An offset of _pHeap. Used by ma_spatializer_process_pcm_frames()
> | to store new channel gains. The number of elements in this array is equal to config.channelsOut. */
5273 |
5274 |     /* Memory management. */
5275 |     void* _pHeap;
5276 |     ma_bool32 _ownsHeap;
5277 | } ma_spatializer;
5278 |
5279 | MA_API ma_result ma_spatializer_get_heap_size(const ma_spatializer_config* pConfig, size_t*
> | pHeapSizeInBytes);
5280 | MA_API ma_result ma_spatializer_init_preallocated(const ma_spatializer_config* pConfig, void* pHeap,
> | ma_spatializer* pSpatializer);
5281 | MA_API ma_result ma_spatializer_init(const ma_spatializer_config* pConfig, const
> | ma_allocation_callbacks* pAllocationCallbacks, ma_spatializer* pSpatializer);
5282 | MA_API void ma_spatializer_uninit(ma_spatializer* pSpatializer, const ma_allocation_callbacks*
> | pAllocationCallbacks);
5283 | MA_API ma_result ma_spatializer_process_pcm_frames(ma_spatializer* pSpatializer,
> | ma_spatializer_listener* pListener, void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount);
5284 | MA_API ma_result ma_spatializer_set_master_volume(ma_spatializer* pSpatializer, float volume);
5285 | MA_API ma_result ma_spatializer_get_master_volume(const ma_spatializer* pSpatializer, float*
> | pVolume);
5286 | MA_API ma_uint32 ma_spatializer_get_input_channels(const ma_spatializer* pSpatializer);
5287 | MA_API ma_uint32 ma_spatializer_get_output_channels(const ma_spatializer* pSpatializer);
5288 | MA_API void ma_spatializer_set_attenuation_model(ma_spatializer* pSpatializer, ma_attenuation_model
> | attenuationModel);
5289 | MA_API ma_attenuation_model ma_spatializer_get_attenuation_model(const ma_spatializer*
> | pSpatializer);
5290 | MA_API void ma_spatializer_set_positioning(ma_spatializer* pSpatializer, ma_positioning
> | positioning);
5291 | MA_API ma_positioning ma_spatializer_get_positioning(const ma_spatializer* pSpatializer);
5292 | MA_API void ma_spatializer_set_rolloff(ma_spatializer* pSpatializer, float rolloff);
5293 | MA_API float ma_spatializer_get_rolloff(const ma_spatializer* pSpatializer);
5294 | MA_API void ma_spatializer_set_min_gain(ma_spatializer* pSpatializer, float minGain);
5295 | MA_API float ma_spatializer_get_min_gain(const ma_spatializer* pSpatializer);
5296 | MA_API void ma_spatializer_set_max_gain(ma_spatializer* pSpatializer, float maxGain);
5297 | MA_API float ma_spatializer_get_max_gain(const ma_spatializer* pSpatializer);
5298 | MA_API void ma_spatializer_set_min_distance(ma_spatializer* pSpatializer, float minDistance);
5299 | MA_API float ma_spatializer_get_min_distance(const ma_spatializer* pSpatializer);
5300 | MA_API void ma_spatializer_set_max_distance(ma_spatializer* pSpatializer, float maxDistance);
5301 | MA_API float ma_spatializer_get_max_distance(const ma_spatializer* pSpatializer);
5302 | MA_API void ma_spatializer_set_cone(ma_spatializer* pSpatializer, float innerAngleInRadians, float
> | outerAngleInRadians, float outerGain);
5303 | MA_API void ma_spatializer_get_cone(const ma_spatializer* pSpatializer, float* pInnerAngleInRadians,
> | float* pOuterAngleInRadians, float* pOuterGain);
5304 | MA_API void ma_spatializer_set_doppler_factor(ma_spatializer* pSpatializer, float dopplerFactor);
5305 | MA_API float ma_spatializer_get_doppler_factor(const ma_spatializer* pSpatializer);
5306 | MA_API void ma_spatializer_set_directional_attenuation_factor(ma_spatializer* pSpatializer, float
> | directionalAttenuationFactor);
5307 | MA_API float ma_spatializer_get_directional_attenuation_factor(const ma_spatializer* pSpatializer);
5308 | MA_API void ma_spatializer_set_position(ma_spatializer* pSpatializer, float x, float y, float z);
5309 | MA_API ma_vec3f ma_spatializer_get_position(const ma_spatializer* pSpatializer);
5310 | MA_API void ma_spatializer_set_direction(ma_spatializer* pSpatializer, float x, float y, float z);
5311 | MA_API ma_vec3f ma_spatializer_get_direction(const ma_spatializer* pSpatializer);
5312 | MA_API void ma_spatializer_set_velocity(ma_spatializer* pSpatializer, float x, float y, float z);
5313 | MA_API ma_vec3f ma_spatializer_get_velocity(const ma_spatializer* pSpatializer);
5314 | MA_API void ma_spatializer_get_relative_position_and_direction(const ma_spatializer* pSpatializer,
> | const ma_spatializer_listener* pListener, ma_vec3f* pRelativePos, ma_vec3f* pRelativeDir);
5315 |

```



```

5316 |
5317 |
5318 | /*****
> *****/
5319 | *****/
> *****/
5320 |
5321 | DATA CONVERSION
5322 | =====
5323 |
5324 | This section contains the APIs for data conversion. You will find everything here for channel
> mapping, sample format conversion, resampling, etc.
5325 |
5326 | *****/
> *****/
5327 | *****/
> *****/
5328 |
5329 | /*****
> *****/
5330 |
5331 | Resampling
5332 |
5333 | *****/
> *****/
5334 | typedef struct
5335 | {
5336 |     ma_format format;
5337 |     ma_uint32 channels;
5338 |     ma_uint32 sampleRateIn;
5339 |     ma_uint32 sampleRateOut;
5340 |     ma_uint32 lpfOrder;          /* The low-pass filter order. Setting this to 0 will disable
> low-pass filtering. */
5341 |     double lpfNyquistFactor; /* 0..1. Defaults to 1. 1 = Half the sampling frequency (Nyquist
> Frequency), 0.5 = Quarter the sampling frequency (half Nyquist Frequency), etc. */
5342 | } ma_linear_resampler_config;
5343 |
5344 | MA_API ma_result ma_linear_resampler_config_init(ma_format format, ma_uint32
> channels, ma_uint32 sampleRateIn, ma_uint32 sampleRateOut);
5345 |
5346 | typedef struct
5347 | {
5348 |     ma_linear_resampler_config config;
5349 |     ma_uint32 inAdvanceInt;
5350 |     ma_uint32 inAdvanceFrac;
5351 |     ma_uint32 inTimeInt;
5352 |     ma_uint32 inTimeFrac;
5353 |     union
5354 |     {
5355 |         float* f32;
5356 |         ma_int16* s16;
5357 |     } x0; /* The previous input frame. */
5358 |     union
5359 |     {
5360 |         float* f32;
5361 |         ma_int16* s16;
5362 |     } x1; /* The next input frame. */
5363 |     ma_lpf lpf;
5364 |
5365 |     /* Memory management. */
5366 |     void* _pHeap;
5367 |     ma_bool32 _ownsHeap;
5368 | } ma_linear_resampler;
5369 |
5370 | MA_API ma_result ma_linear_resampler_get_heap_size(const ma_linear_resampler_config* pConfig,
> size_t* pHeapSizeInBytes);
5371 | MA_API ma_result ma_linear_resampler_init_preallocated(const ma_linear_resampler_config* pConfig,
> void* pHeap, ma_linear_resampler* pResampler);
5372 | MA_API ma_result ma_linear_resampler_init(const ma_linear_resampler_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_linear_resampler* pResampler);
5373 | MA_API void ma_linear_resampler_uninit(ma_linear_resampler* pResampler, const
> ma_allocation_callbacks* pAllocationCallbacks);
5374 | MA_API ma_result ma_linear_resampler_process_pcm_frames(ma_linear_resampler* pResampler, const void*
> pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut);
5375 | MA_API ma_result ma_linear_resampler_set_rate(ma_linear_resampler* pResampler, ma_uint32
> sampleRateIn, ma_uint32 sampleRateOut);
5376 | MA_API ma_result ma_linear_resampler_set_rate_ratio(ma_linear_resampler* pResampler, float
> ratioInOut);
5377 | MA_API ma_uint64 ma_linear_resampler_get_input_latency(const ma_linear_resampler* pResampler);

```

```

5378 | MA_API ma_uint64 ma_linear_resampler_get_output_latency(const ma_linear_resampler* pResampler);
5379 | MA_API ma_result ma_linear_resampler_get_required_input_frame_count(const ma_linear_resampler*
> pResampler, ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount);
5380 | MA_API ma_result ma_linear_resampler_get_expected_output_frame_count(const ma_linear_resampler*
> pResampler, ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount);
5381 | MA_API ma_result ma_linear_resampler_reset(ma_linear_resampler* pResampler);
5382 |
5383 |
5384 | typedef struct ma_resampler_config ma_resampler_config;
5385 |
5386 | typedef void ma_resampling_backend;
5387 | typedef struct
5388 | {
5389 |     ma_result (* onGetHeapSize                )(void* pUserData, const ma_resampler_config* pConfig,
> size_t* pHeapSizeInBytes);
5390 |     ma_result (* onInit                      )(void* pUserData, const ma_resampler_config* pConfig,
> void* pHeap, ma_resampling_backend** ppBackend);
5391 |     void      (* onUninit                    )(void* pUserData, ma_resampling_backend* pBackend,
> const ma_allocation_callbacks* pAllocationCallbacks);
5392 |     ma_result (* onProcess                   )(void* pUserData, ma_resampling_backend* pBackend,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut);
5393 |     ma_result (* onSetRate                   )(void* pUserData, ma_resampling_backend* pBackend,
> ma_uint32 sampleRateIn, ma_uint32 sampleRateOut); /* Optional. Rate changes will be
> disabled. */
5394 |     ma_uint64 (* onGetInputLatency           )(void* pUserData, const ma_resampling_backend*
> pBackend); /* Optional. Latency will be
> reported as 0. */
5395 |     ma_uint64 (* onGetOutputLatency          )(void* pUserData, const ma_resampling_backend*
> pBackend); /* Optional. Latency will be
> reported as 0. */
5396 |     ma_result (* onGetRequiredInputFrameCount )(void* pUserData, const ma_resampling_backend*
> pBackend, ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount); /* Optional. Latency
> mitigation will be disabled. */
5397 |     ma_result (* onGetExpectedOutputFrameCount )(void* pUserData, const ma_resampling_backend*
> pBackend, ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount); /* Optional. Latency
> mitigation will be disabled. */
5398 |     ma_result (* onReset                     )(void* pUserData, ma_resampling_backend* pBackend);
5399 | } ma_resampling_backend_vtable;
5400 |
5401 | typedef enum
5402 | {
5403 |     ma_resample_algorithm_linear = 0, /* Fastest, lowest quality. Optional low-pass filtering.
> Default. */
5404 |     ma_resample_algorithm_custom,
5405 | } ma_resample_algorithm;
5406 |
5407 | struct ma_resampler_config
5408 | {
5409 |     ma_format format; /* Must be either ma_format_f32 or ma_format_s16. */
5410 |     ma_uint32 channels;
5411 |     ma_uint32 sampleRateIn;
5412 |     ma_uint32 sampleRateOut;
5413 |     ma_resample_algorithm algorithm; /* When set to ma_resample_algorithm_custom, pBackendVTable
> will be used. */
5414 |     ma_resampling_backend_vtable* pBackendVTable;
5415 |     void* pBackendUserData;
5416 |     struct
5417 |     {
5418 |         ma_uint32 lpfOrder;
5419 |     } linear;
5420 | };
5421 |
5422 | MA_API ma_resampler_config ma_resampler_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRateIn, ma_uint32 sampleRateOut, ma_resample_algorithm algorithm);
5423 |
5424 | typedef struct
5425 | {
5426 |     ma_resampling_backend* pBackend;
5427 |     ma_resampling_backend_vtable* pBackendVTable;
5428 |     void* pBackendUserData;
5429 |     ma_format format;
5430 |     ma_uint32 channels;
5431 |     ma_uint32 sampleRateIn;
5432 |     ma_uint32 sampleRateOut;
5433 |     union
5434 |     {
5435 |         ma_linear_resampler linear;
5436 |     } state; /* State for stock resamplers so we can avoid a malloc. For stock resamplers,
> pBackend will point here. */

```

```

5437 |
5438 |     /* Memory management. */
5439 |     void* _pHeap;
5440 |     ma_bool32 _ownsHeap;
5441 | } ma_resampler;
5442 |
5443 | MA_API ma_result ma_resampler_get_heap_size(const ma_resampler_config* pConfig, size_t*
> pHeapSizeInBytes);
5444 | MA_API ma_result ma_resampler_init_preallocated(const ma_resampler_config* pConfig, void* pHeap,
> ma_resampler* pResampler);
5445 |
5446 | /*
5447 | Initializes a new resampler object from a config.
5448 | */
5449 | MA_API ma_result ma_resampler_init(const ma_resampler_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_resampler* pResampler);
5450 |
5451 | /*
5452 | Uninitializes a resampler.
5453 | */
5454 | MA_API void ma_resampler_uninit(ma_resampler* pResampler, const ma_allocation_callbacks*
> pAllocationCallbacks);
5455 |
5456 | /*
5457 | Converts the given input data.
5458 |
5459 | Both the input and output frames must be in the format specified in the config when the resampler
> was initialized.
5460 |
5461 | On input, [pFrameCountOut] contains the number of output frames to process. On output it contains
> the number of output frames that
5462 | were actually processed, which may be less than the requested amount which will happen if there's
> not enough input data. You can use
5463 | ma_resampler_get_expected_output_frame_count() to know how many output frames will be processed for
> a given number of input frames.
5464 |
5465 | On input, [pFrameCountIn] contains the number of input frames contained in [pFramesIn]. On output it
> contains the number of whole
5466 | input frames that were actually processed. You can use ma_resampler_get_required_input_frame_count()
> to know how many input frames
5467 | you should provide for a given number of output frames. [pFramesIn] can be NULL, in which case
> zeroes will be used instead.
5468 |
5469 | If [pFramesOut] is NULL, a seek is performed. In this case, if [pFrameCountOut] is not NULL it will
> seek by the specified number of
5470 | output frames. Otherwise, if [pFramesCountOut] is NULL and [pFrameCountIn] is not NULL, it will seek
> by the specified number of input
5471 | frames. When seeking, [pFramesIn] is allowed to NULL, in which case the internal timing state will
> be updated, but no input will be
5472 | processed. In this case, any internal filter state will be updated as if zeroes were passed in.
5473 |
5474 | It is an error for [pFramesOut] to be non-NULL and [pFrameCountOut] to be NULL.
5475 |
5476 | It is an error for both [pFrameCountOut] and [pFrameCountIn] to be NULL.
5477 | */
5478 | MA_API ma_result ma_resampler_process_pcm_frames(ma_resampler* pResampler, const void* pFramesIn,
> ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut);
5479 |
5480 |
5481 | /*
5482 | Sets the input and output sample rate.
5483 | */
5484 | MA_API ma_result ma_resampler_set_rate(ma_resampler* pResampler, ma_uint32 sampleRateIn, ma_uint32
> sampleRateOut);
5485 |
5486 | /*
5487 | Sets the input and output sample rate as a ratio.
5488 |
5489 | The ration is in/out.
5490 | */
5491 | MA_API ma_result ma_resampler_set_rate_ratio(ma_resampler* pResampler, float ratio);
5492 |
5493 | /*
5494 | Retrieves the latency introduced by the resampler in input frames.
5495 | */
5496 | MA_API ma_uint64 ma_resampler_get_input_latency(const ma_resampler* pResampler);
5497 |
5498 | /*
5499 | Retrieves the latency introduced by the resampler in output frames.

```

```

5500 | */
5501 | MA_API ma_uint64 ma_resampler_get_output_latency(const ma_resampler* pResampler);
5502 |
5503 | /*
5504 | Calculates the number of whole input frames that would need to be read from the client in order to
> output the specified
5505 | number of output frames.
5506 |
5507 | The returned value does not include cached input frames. It only returns the number of extra frames
> that would need to be
5508 | read from the input buffer in order to output the specified number of output frames.
5509 | */
5510 | MA_API ma_result ma_resampler_get_required_input_frame_count(const ma_resampler* pResampler,
> ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount);
5511 |
5512 | /*
5513 | Calculates the number of whole output frames that would be output after fully reading and consuming
> the specified number of
5514 | input frames.
5515 | */
5516 | MA_API ma_result ma_resampler_get_expected_output_frame_count(const ma_resampler* pResampler,
> ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount);
5517 |
5518 | /*
5519 | Resets the resampler's timer and clears its internal cache.
5520 | */
5521 | MA_API ma_result ma_resampler_reset(ma_resampler* pResampler);
5522 |
5523 |
5524 | /*****
> *****/
5525 |
5526 | Channel Conversion
5527 |
5528 | *****/
> *****/
5529 | typedef enum
5530 | {
5531 |     ma_channel_conversion_path_unknown,
5532 |     ma_channel_conversion_path_passthrough,
5533 |     ma_channel_conversion_path_mono_out,    /* Converting to mono. */
5534 |     ma_channel_conversion_path_mono_in,    /* Converting from mono. */
5535 |     ma_channel_conversion_path_shuffle,    /* Simple shuffle. Will use this when all channels are
> present in both input and output channel maps, but just in a different order. */
5536 |     ma_channel_conversion_path_weights    /* Blended based on weights. */
5537 | } ma_channel_conversion_path;
5538 |
5539 | typedef enum
5540 | {
5541 |     ma_mono_expansion_mode_duplicate = 0,    /* The default. */
5542 |     ma_mono_expansion_mode_average,        /* Average the mono channel across all channels. */
5543 |     ma_mono_expansion_mode_stereo_only,    /* Duplicate to the left and right channels only and
> ignore the others. */
5544 |     ma_mono_expansion_mode_default = ma_mono_expansion_mode_duplicate
5545 | } ma_mono_expansion_mode;
5546 |
5547 | typedef struct
5548 | {
5549 |     ma_format format;
5550 |     ma_uint32 channelsIn;
5551 |     ma_uint32 channelsOut;
5552 |     const ma_channel* pChannelMapIn;
5553 |     const ma_channel* pChannelMapOut;
5554 |     ma_channel_mix_mode mixingMode;
5555 |     ma_bool32 calculateLFEFromSpatialChannels; /* When an output LFE channel is present, but no
> input LFE, set to true to set the output LFE to the average of all spatial channels (LR, FR, etc.).
> Ignored when an input LFE is present. */
5556 |     float** ppWeights; /* [in][out]. Only used when mixingMode is set to
> ma_channel_mix_mode_custom_weights. */
5557 | } ma_channel_converter_config;
5558 |
5559 | MA_API ma_channel_converter_config ma_channel_converter_config_init(ma_format format, ma_uint32
> channelsIn, const ma_channel* pChannelMapIn, ma_uint32 channelsOut, const ma_channel*
> pChannelMapOut, ma_channel_mix_mode mixingMode);
5560 |
5561 | typedef struct
5562 | {
5563 |     ma_format format;
5564 |     ma_uint32 channelsIn;

```

```

5565 |     ma_uint32 channelsOut;
5566 |     ma_channel_mix_mode mixingMode;
5567 |     ma_channel_conversion_path conversionPath;
5568 |     ma_channel* pChannelMapIn;
5569 |     ma_channel* pChannelMapOut;
5570 |     ma_uint8* pShuffleTable; /* Indexed by output channel index. */
5571 |     union
5572 |     {
5573 |         float** f32;
5574 |         ma_int32** s16;
5575 |     } weights; /* [in][out] */
5576 |
5577 |     /* Memory management. */
5578 |     void* pHeap;
5579 |     ma_bool32 _ownsHeap;
5580 | } ma_channel_converter;
5581 |
5582 | MA_API ma_result ma_channel_converter_get_heap_size(const ma_channel_converter_config* pConfig,
5583 | > size_t* pHeapSizeInBytes);
5584 | MA_API ma_result ma_channel_converter_init_preallocated(const ma_channel_converter_config* pConfig,
5585 | > void* pHeap, ma_channel_converter* pConverter);
5586 | MA_API ma_result ma_channel_converter_init(const ma_channel_converter_config* pConfig, const
5587 | > ma_allocation_callbacks* pAllocationCallbacks, ma_channel_converter* pConverter);
5588 | MA_API void ma_channel_converter_uninit(ma_channel_converter* pConverter, const
5589 | > ma_allocation_callbacks* pAllocationCallbacks);
5589 | MA_API ma_result ma_channel_converter_process_pcm_frames(ma_channel_converter* pConverter, void*
5590 | > pFramesOut, const void* pFramesIn, ma_uint64 frameCount);
5591 | MA_API ma_result ma_channel_converter_get_input_channel_map(const ma_channel_converter* pConverter,
5592 | > ma_channel* pChannelMap, size_t channelMapCap);
5593 | MA_API ma_result ma_channel_converter_get_output_channel_map(const ma_channel_converter* pConverter,
5594 | > ma_channel* pChannelMap, size_t channelMapCap);
5595 |
5596 | /*****
5597 | > *****/
5598 |
5599 | Data Conversion
5600 |
5601 | /*****
5602 | > *****/
5603 |
5604 | typedef struct
5605 | {
5606 |     ma_format formatIn;
5607 |     ma_format formatOut;
5608 |     ma_uint32 channelsIn;
5609 |     ma_uint32 channelsOut;
5610 |     ma_uint32 sampleRateIn;
5611 |     ma_uint32 sampleRateOut;
5612 |     ma_channel* pChannelMapIn;
5613 |     ma_channel* pChannelMapOut;
5614 |     ma_dither_mode ditherMode;
5615 |     ma_channel_mix_mode channelMixMode;
5616 |     ma_bool32 calculateLFEFromSpatialChannels; /* When an output LFE channel is present, but no
5617 | > input LFE, set to true to set the output LFE to the average of all spatial channels (LR, FR, etc.).
5618 | > Ignored when an input LFE is present. */
5619 |     float** ppChannelWeights; /* [in][out]. Only used when mixingMode is set to
5620 | > ma_channel_mix_mode_custom_weights. */
5621 |     ma_bool32 allowDynamicSampleRate;
5622 |     ma_resampler_config resampling;
5623 | } ma_data_converter_config;
5624 |
5625 | MA_API ma_data_converter_config ma_data_converter_config_init_default(void);
5626 | MA_API ma_data_converter_config ma_data_converter_config_init(ma_format formatIn, ma_format
5627 | > formatOut, ma_uint32 channelsIn, ma_uint32 channelsOut, ma_uint32 sampleRateIn, ma_uint32
5628 | > sampleRateOut);
5629 |
5630 | typedef enum
5631 | {
5632 |     ma_data_converter_execution_path_passthrough, /* No conversion. */
5633 |     ma_data_converter_execution_path_format_only, /* Only format conversion. */
5634 |     ma_data_converter_execution_path_channels_only, /* Only channel conversion. */
5635 |     ma_data_converter_execution_path_resample_only, /* Only resampling. */
5636 |     ma_data_converter_execution_path_resample_first, /* All conversions, but resample as the
5637 | > first step. */
5638 |     ma_data_converter_execution_path_channels_first /* All conversions, but channels as the
5639 | > first step. */
5640 | } ma_data_converter_execution_path;
5641 |

```

```

5628 | typedef struct
5629 | {
5630 |     ma_format formatIn;
5631 |     ma_format formatOut;
5632 |     ma_uint32 channelsIn;
5633 |     ma_uint32 channelsOut;
5634 |     ma_uint32 sampleRateIn;
5635 |     ma_uint32 sampleRateOut;
5636 |     ma_dither_mode ditherMode;
5637 |     ma_data_converter_execution_path executionPath; /* The execution path the data converter will
> follow when processing. */
5638 |     ma_channel_converter channelConverter;
5639 |     ma_resampler resampler;
5640 |     ma_bool8 hasPreFormatConversion;
5641 |     ma_bool8 hasPostFormatConversion;
5642 |     ma_bool8 hasChannelConverter;
5643 |     ma_bool8 hasResampler;
5644 |     ma_bool8 isPassthrough;
5645 |
5646 |     /* Memory management. */
5647 |     ma_bool8 _ownsHeap;
5648 |     void* _pHeap;
5649 | } ma_data_converter;
5650 |
5651 | MA_API ma_result ma_data_converter_get_heap_size(const ma_data_converter_config* pConfig, size_t*
> pHeapSizeInBytes);
5652 | MA_API ma_result ma_data_converter_init_preallocated(const ma_data_converter_config* pConfig, void*
> pHeap, ma_data_converter* pConverter);
5653 | MA_API ma_result ma_data_converter_init(const ma_data_converter_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_data_converter* pConverter);
5654 | MA_API void ma_data_converter_uninit(ma_data_converter* pConverter, const ma_allocation_callbacks*
> pAllocationCallbacks);
5655 | MA_API ma_result ma_data_converter_process_pcm_frames(ma_data_converter* pConverter, const void*
> pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut);
5656 | MA_API ma_result ma_data_converter_set_rate(ma_data_converter* pConverter, ma_uint32 sampleRateIn,
> ma_uint32 sampleRateOut);
5657 | MA_API ma_result ma_data_converter_set_rate_ratio(ma_data_converter* pConverter, float ratioInOut);
5658 | MA_API ma_uint64 ma_data_converter_get_input_latency(const ma_data_converter* pConverter);
5659 | MA_API ma_uint64 ma_data_converter_get_output_latency(const ma_data_converter* pConverter);
5660 | MA_API ma_result ma_data_converter_get_required_input_frame_count(const ma_data_converter*
> pConverter, ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount);
5661 | MA_API ma_result ma_data_converter_get_expected_output_frame_count(const ma_data_converter*
> pConverter, ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount);
5662 | MA_API ma_result ma_data_converter_get_input_channel_map(const ma_data_converter* pConverter,
> ma_channel* pChannelMap, size_t channelMapCap);
5663 | MA_API ma_result ma_data_converter_get_output_channel_map(const ma_data_converter* pConverter,
> ma_channel* pChannelMap, size_t channelMapCap);
5664 | MA_API ma_result ma_data_converter_reset(ma_data_converter* pConverter);
5665 |
5666 |
5667 | /*****
> *****/
5668 |
5669 | Format Conversion
5670 |
5671 | *****/
> *****/
5672 | MA_API void ma_pcm_u8_to_s16(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5673 | MA_API void ma_pcm_u8_to_s24(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5674 | MA_API void ma_pcm_u8_to_s32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5675 | MA_API void ma_pcm_u8_to_f32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5676 | MA_API void ma_pcm_s16_to_u8(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5677 | MA_API void ma_pcm_s16_to_s24(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5678 | MA_API void ma_pcm_s16_to_s32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5679 | MA_API void ma_pcm_s16_to_f32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5680 | MA_API void ma_pcm_s24_to_u8(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5681 | MA_API void ma_pcm_s24_to_s16(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5682 | MA_API void ma_pcm_s24_to_s32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);

```

```

5683 | MA_API void ma_pcm_s24_to_f32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5684 | MA_API void ma_pcm_s32_to_u8(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5685 | MA_API void ma_pcm_s32_to_s16(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5686 | MA_API void ma_pcm_s32_to_s24(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5687 | MA_API void ma_pcm_s32_to_f32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5688 | MA_API void ma_pcm_f32_to_u8(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5689 | MA_API void ma_pcm_f32_to_s16(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5690 | MA_API void ma_pcm_f32_to_s24(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5691 | MA_API void ma_pcm_f32_to_s32(void* pOut, const void* pIn, ma_uint64 count, ma_dither_mode
> ditherMode);
5692 | MA_API void ma_pcm_convert(void* pOut, ma_format formatOut, const void* pIn, ma_format formatIn,
> ma_uint64 sampleCount, ma_dither_mode ditherMode);
5693 | MA_API void ma_convert_pcm_frames_format(void* pOut, ma_format formatOut, const void* pIn, ma_format
> formatIn, ma_uint64 frameCount, ma_uint32 channels, ma_dither_mode ditherMode);
5694 |
5695 | /*
5696 | Deinterleaves an interleaved buffer.
5697 | */
5698 | MA_API void ma_deinterleave_pcm_frames(ma_format format, ma_uint32 channels, ma_uint64 frameCount,
> const void* pInterleavedPCMFrames, void** ppDeinterleavedPCMFrames);
5699 |
5700 | /*
5701 | Interleaves a group of deinterleaved buffers.
5702 | */
5703 | MA_API void ma_interleave_pcm_frames(ma_format format, ma_uint32 channels, ma_uint64 frameCount,
> const void** ppDeinterleavedPCMFrames, void* pInterleavedPCMFrames);
5704 |
5705 |
5706 | /*****
> *****/
5707 |
5708 | Channel Maps
5709 |
5710 | *****/
> *****/
5711 | /*
5712 | This is used in the shuffle table to indicate that the channel index is undefined and should be
> ignored.
5713 | */
5714 | #define MA_CHANNEL_INDEX_NULL 255
5715 |
5716 | /*
5717 | Retrieves the channel position of the specified channel in the given channel map.
5718 |
5719 | The pChannelMap parameter can be null, in which case miniaudio's default channel map will be
> assumed.
5720 | */
5721 | MA_API ma_channel ma_channel_map_get_channel(const ma_channel* pChannelMap, ma_uint32 channelCount,
> ma_uint32 channelIndex);
5722 |
5723 | /*
5724 | Initializes a blank channel map.
5725 |
5726 | When a blank channel map is specified anywhere it indicates that the native channel map should be
> used.
5727 | */
5728 | MA_API void ma_channel_map_init_blank(ma_channel* pChannelMap, ma_uint32 channels);
5729 |
5730 | /*
5731 | Helper for retrieving a standard channel map.
5732 |
5733 | The output channel map buffer must have a capacity of at least `channelMapCap`.
5734 | */
5735 | MA_API void ma_channel_map_init_standard(ma_standard_channel_map standardChannelMap, ma_channel*
> pChannelMap, size_t channelMapCap, ma_uint32 channels);
5736 |
5737 | /*
5738 | Copies a channel map.
5739 |
5740 | Both input and output channel map buffers must have a capacity of at least `channels`.
5741 | */

```



```

5742 | MA_API void ma_channel_map_copy(ma_channel* pOut, const ma_channel* pIn, ma_uint32 channels);
5743 |
5744 | /*
5745 | Copies a channel map if one is specified, otherwise copies the default channel map.
5746 |
5747 | The output buffer must have a capacity of at least `channels`. If not NULL, the input channel map
> must also have a capacity of at least `channels`.
5748 | */
5749 | MA_API void ma_channel_map_copy_or_default(ma_channel* pOut, size_t channelMapCapOut, const
> ma_channel* pIn, ma_uint32 channels);
5750 |
5751 |
5752 | /*
5753 | Determines whether or not a channel map is valid.
5754 |
5755 | A blank channel map is valid (all channels set to MA_CHANNEL_NONE). The way a blank channel map is
> handled is context specific, but
5756 | is usually treated as a passthrough.
5757 |
5758 | Invalid channel maps:
5759 | - A channel map with no channels
5760 | - A channel map with more than one channel and a mono channel
5761 |
5762 | The channel map buffer must have a capacity of at least `channels`.
5763 | */
5764 | MA_API ma_bool32 ma_channel_map_is_valid(const ma_channel* pChannelMap, ma_uint32 channels);
5765 |
5766 | /*
5767 | Helper for comparing two channel maps for equality.
5768 |
5769 | This assumes the channel count is the same between the two.
5770 |
5771 | Both channels map buffers must have a capacity of at least `channels`.
5772 | */
5773 | MA_API ma_bool32 ma_channel_map_is_equal(const ma_channel* pChannelMapA, const ma_channel*
> pChannelMapB, ma_uint32 channels);
5774 |
5775 | /*
5776 | Helper for determining if a channel map is blank (all channels set to MA_CHANNEL_NONE).
5777 |
5778 | The channel map buffer must have a capacity of at least `channels`.
5779 | */
5780 | MA_API ma_bool32 ma_channel_map_is_blank(const ma_channel* pChannelMap, ma_uint32 channels);
5781 |
5782 | /*
5783 | Helper for determining whether or not a channel is present in the given channel map.
5784 |
5785 | The channel map buffer must have a capacity of at least `channels`.
5786 | */
5787 | MA_API ma_bool32 ma_channel_map_contains_channel_position(ma_uint32 channels, const ma_channel*
> pChannelMap, ma_channel channelPosition);
5788 |
5789 | /*
5790 | Find a channel position in the given channel map. Returns MA_TRUE if the channel is found; MA_FALSE
> otherwise. The
5791 | index of the channel is output to `pChannelIndex`.
5792 |
5793 | The channel map buffer must have a capacity of at least `channels`.
5794 | */
5795 | MA_API ma_bool32 ma_channel_map_find_channel_position(ma_uint32 channels, const ma_channel*
> pChannelMap, ma_channel channelPosition, ma_uint32* pChannelIndex);
5796 |
5797 | /*
5798 | Generates a string representing the given channel map.
5799 |
5800 | This is for printing and debugging purposes, not serialization/deserialization.
5801 |
5802 | Returns the length of the string, not including the null terminator.
5803 | */
5804 | MA_API size_t ma_channel_map_to_string(const ma_channel* pChannelMap, ma_uint32 channels, char*
> pBufferOut, size_t bufferCap);
5805 |
5806 | /*
5807 | Retrieves a human readable version of a channel position.
5808 | */
5809 | MA_API const char* ma_channel_position_to_string(ma_channel channel);
5810 |
5811 |
5812 | /*****

```

```

> *****
5813 |
5814 | Conversion Helpers
5815 |
5816 | *****
> *****/
5817 |
5818 | /*
5819 | High-level helper for doing a full format conversion in one go. Returns the number of output frames.
> Call this with pOut set to NULL to
5820 | determine the required size of the output buffer. frameCountOut should be set to the capacity of
> pOut. If pOut is NULL, frameCountOut is
5821 | ignored.
5822 |
5823 | A return value of 0 indicates an error.
5824 |
5825 | This function is useful for one-off bulk conversions, but if you're streaming data you should use
> the ma_data_converter APIs instead.
5826 | */
5827 | MA_API ma_uint64 ma_convert_frames(void* pOut, ma_uint64 frameCountOut, ma_format formatOut,
> ma_uint32 channelsOut, ma_uint32 sampleRateOut, const void* pIn, ma_uint64 frameCountIn, ma_format
> formatIn, ma_uint32 channelsIn, ma_uint32 sampleRateIn);
5828 | MA_API ma_uint64 ma_convert_frames_ex(void* pOut, ma_uint64 frameCountOut, const void* pIn,
> ma_uint64 frameCountIn, const ma_data_converter_config* pConfig);
5829 |
5830 |
5831 | /*****
> *****/
5832 |
5833 | Data Source
5834 |
5835 | *****/
> *****/
5836 | typedef void ma_data_source;
5837 |
5838 | #define MA_DATA_SOURCE_SELF_MANAGED_RANGE_AND_LOOP_POINT 0x00000001
5839 |
5840 | typedef struct
5841 | {
5842 |     ma_result (* onRead)(ma_data_source* pDataSource, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead);
5843 |     ma_result (* onSeek)(ma_data_source* pDataSource, ma_uint64 frameIndex);
5844 |     ma_result (* onGetDataFormat)(ma_data_source* pDataSource, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
5845 |     ma_result (* onGetCursor)(ma_data_source* pDataSource, ma_uint64* pCursor);
5846 |     ma_result (* onGetLength)(ma_data_source* pDataSource, ma_uint64* pLength);
5847 |     ma_result (* onSetLooping)(ma_data_source* pDataSource, ma_bool32 isLooping);
5848 |     ma_uint32 flags;
5849 | } ma_data_source_vtable;
5850 |
5851 | typedef ma_data_source* (* ma_data_source_get_next_proc)(ma_data_source* pDataSource);
5852 |
5853 | typedef struct
5854 | {
5855 |     const ma_data_source_vtable* vtable;
5856 | } ma_data_source_config;
5857 |
5858 | MA_API ma_data_source_config ma_data_source_config_init(void);
5859 |
5860 |
5861 | typedef struct
5862 | {
5863 |     const ma_data_source_vtable* vtable;
5864 |     ma_uint64 rangeBegInFrames;
5865 |     ma_uint64 rangeEndInFrames; /* Set to -1 for unranged (default). */
5866 |     ma_uint64 loopBegInFrames; /* Relative to rangeBegInFrames. */
5867 |     ma_uint64 loopEndInFrames; /* Relative to rangeBegInFrames. Set to -1 for the end
> of the range. */
5868 |     ma_data_source* pCurrent; /* When non-NULL, the data source being initialized will
> act as a proxy and will route all operations to pCurrent. Used in conjunction with pNext/onGetNext
> for seamless chaining. */
5869 |     ma_data_source* pNext; /* When set to NULL, onGetNext will be used. */
5870 |     ma_data_source_get_next_proc onGetNext; /* Will be used when pNext is NULL. If both are NULL, no
> next will be used. */
5871 |     MA_ATOMIC(4, ma_bool32) isLooping;
5872 | } ma_data_source_base;
5873 |
5874 | MA_API ma_result ma_data_source_init(const ma_data_source_config* pConfig, ma_data_source*
> pDataSource);

```

```

5875 | MA_API void ma_data_source_uninit(ma_data_source* pDataSource);
5876 | MA_API ma_result ma_data_source_read_pcm_frames(ma_data_source* pDataSource, void* pFramesOut,
> ma_uint64 frameCount, ma_uint64* pFramesRead); /* Must support pFramesOut = NULL in which case a
> forward seek should be performed. */
5877 | MA_API ma_result ma_data_source_seek_pcm_frames(ma_data_source* pDataSource, ma_uint64 frameCount,
> ma_uint64* pFramesSearched); /* Can only seek forward. Equivalent to
> ma_data_source_read_pcm_frames(pDataSource, NULL, frameCount, &framesRead); */
5878 | MA_API ma_result ma_data_source_seek_to_pcm_frame(ma_data_source* pDataSource, ma_uint64
> frameIndex);
5879 | MA_API ma_result ma_data_source_seek_seconds(ma_data_source* pDataSource, float secondCount, float*
> pSecondsSearched); /* Can only seek forward. Abstraction to ma_data_source_seek_pcm_frames() */
5880 | MA_API ma_result ma_data_source_seek_to_second(ma_data_source* pDataSource, float
> seekPointInSeconds); /* Abstraction to ma_data_source_seek_to_pcm_frame() */
5881 | MA_API ma_result ma_data_source_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
5882 | MA_API ma_result ma_data_source_get_cursor_in_pcm_frames(ma_data_source* pDataSource, ma_uint64*
> pCursor);
5883 | MA_API ma_result ma_data_source_get_length_in_pcm_frames(ma_data_source* pDataSource, ma_uint64*
> pLength); /* Returns MA_NOT_IMPLEMENTED if the length is unknown or cannot be determined.
> Decoders can return this. */
5884 | MA_API ma_result ma_data_source_get_cursor_in_seconds(ma_data_source* pDataSource, float* pCursor);
5885 | MA_API ma_result ma_data_source_get_length_in_seconds(ma_data_source* pDataSource, float* pLength);
5886 | MA_API ma_result ma_data_source_set_looping(ma_data_source* pDataSource, ma_bool32 isLooping);
5887 | MA_API ma_bool32 ma_data_source_is_looping(const ma_data_source* pDataSource);
5888 | MA_API ma_result ma_data_source_set_range_in_pcm_frames(ma_data_source* pDataSource, ma_uint64
> rangeBegInFrames, ma_uint64 rangeEndInFrames);
5889 | MA_API void ma_data_source_get_range_in_pcm_frames(const ma_data_source* pDataSource, ma_uint64*
> pRangeBegInFrames, ma_uint64* pRangeEndInFrames);
5890 | MA_API ma_result ma_data_source_set_loop_point_in_pcm_frames(ma_data_source* pDataSource, ma_uint64
> loopBegInFrames, ma_uint64 loopEndInFrames);
5891 | MA_API void ma_data_source_get_loop_point_in_pcm_frames(const ma_data_source* pDataSource,
> ma_uint64* pLoopBegInFrames, ma_uint64* pLoopEndInFrames);
5892 | MA_API ma_result ma_data_source_set_current(ma_data_source* pDataSource, ma_data_source*
> pCurrentDataSource);
5893 | MA_API ma_data_source* ma_data_source_get_current(const ma_data_source* pDataSource);
5894 | MA_API ma_result ma_data_source_set_next(ma_data_source* pDataSource, ma_data_source*
> pNextDataSource);
5895 | MA_API ma_data_source* ma_data_source_get_next(const ma_data_source* pDataSource);
5896 | MA_API ma_result ma_data_source_set_next_callback(ma_data_source* pDataSource,
> ma_data_source_get_next_proc onGetNext);
5897 | MA_API ma_data_source_get_next_proc ma_data_source_get_next_callback(const ma_data_source*
> pDataSource);
5898 |
5899 |
5900 | typedef struct
5901 | {
5902 |     ma_data_source_base ds;
5903 |     ma_format format;
5904 |     ma_uint32 channels;
5905 |     ma_uint32 sampleRate;
5906 |     ma_uint64 cursor;
5907 |     ma_uint64 sizeInFrames;
5908 |     const void* pData;
5909 | } ma_audio_buffer_ref;
5910 |
5911 | MA_API ma_result ma_audio_buffer_ref_init(ma_format format, ma_uint32 channels, const void* pData,
> ma_uint64 sizeInFrames, ma_audio_buffer_ref* pAudioBufferRef);
5912 | MA_API void ma_audio_buffer_ref_uninit(ma_audio_buffer_ref* pAudioBufferRef);
5913 | MA_API ma_result ma_audio_buffer_ref_set_data(ma_audio_buffer_ref* pAudioBufferRef, const void*
> pData, ma_uint64 sizeInFrames);
5914 | MA_API ma_uint64 ma_audio_buffer_ref_read_pcm_frames(ma_audio_buffer_ref* pAudioBufferRef, void*
> pFramesOut, ma_uint64 frameCount, ma_bool32 loop);
5915 | MA_API ma_result ma_audio_buffer_ref_seek_to_pcm_frame(ma_audio_buffer_ref* pAudioBufferRef,
> ma_uint64 frameIndex);
5916 | MA_API ma_result ma_audio_buffer_ref_map(ma_audio_buffer_ref* pAudioBufferRef, void** ppFramesOut,
> ma_uint64* pFrameCount);
5917 | MA_API ma_result ma_audio_buffer_ref_unmap(ma_audio_buffer_ref* pAudioBufferRef, ma_uint64
> frameCount); /* Returns MA_AT_END if the end has been reached. This should be considered
> successful. */
5918 | MA_API ma_bool32 ma_audio_buffer_ref_at_end(const ma_audio_buffer_ref* pAudioBufferRef);
5919 | MA_API ma_result ma_audio_buffer_ref_get_cursor_in_pcm_frames(const ma_audio_buffer_ref*
> pAudioBufferRef, ma_uint64* pCursor);
5920 | MA_API ma_result ma_audio_buffer_ref_get_length_in_pcm_frames(const ma_audio_buffer_ref*
> pAudioBufferRef, ma_uint64* pLength);
5921 | MA_API ma_result ma_audio_buffer_ref_get_available_frames(const ma_audio_buffer_ref*
> pAudioBufferRef, ma_uint64* pAvailableFrames);
5922 |
5923 |
5924 |

```

```

5925 | typedef struct
5926 | {
5927 |     ma_format format;
5928 |     ma_uint32 channels;
5929 |     ma_uint32 sampleRate;
5930 |     ma_uint64 sizeInFrames;
5931 |     const void* pData; /* If set to NULL, will allocate a block of memory for you. */
5932 |     ma_allocation_callbacks allocationCallbacks;
5933 | } ma_audio_buffer_config;
5934 |
5935 | MA_API ma_result ma_audio_buffer_config_init(ma_format format, ma_uint32 channels,
> ma_uint64 sizeInFrames, const void* pData, const ma_allocation_callbacks* pAllocationCallbacks);
5936 |
5937 | typedef struct
5938 | {
5939 |     ma_audio_buffer_ref ref;
5940 |     ma_allocation_callbacks allocationCallbacks;
5941 |     ma_bool32 ownsData; /* Used to control whether or not miniaudio owns the data
> buffer. If set to true, pData will be freed in ma_audio_buffer_uninit(). */
5942 |     ma_uint8 pExtraData[1]; /* For allocating a buffer with the memory located directly
> after the other memory of the structure. */
5943 | } ma_audio_buffer;
5944 |
5945 | MA_API ma_result ma_audio_buffer_init(const ma_audio_buffer_config* pConfig, ma_audio_buffer*
> pAudioBuffer);
5946 | MA_API ma_result ma_audio_buffer_init_copy(const ma_audio_buffer_config* pConfig, ma_audio_buffer*
> pAudioBuffer);
5947 | MA_API ma_result ma_audio_buffer_alloc_and_init(const ma_audio_buffer_config* pConfig,
> ma_audio_buffer** ppAudioBuffer); /* Always copies the data. Doesn't make sense to use this
> otherwise. Use ma_audio_buffer_uninit_and_free() to uninit. */
5948 | MA_API void ma_audio_buffer_uninit(ma_audio_buffer* pAudioBuffer);
5949 | MA_API void ma_audio_buffer_uninit_and_free(ma_audio_buffer* pAudioBuffer);
5950 | MA_API ma_uint64 ma_audio_buffer_read_pcm_frames(ma_audio_buffer* pAudioBuffer, void* pFramesOut,
> ma_uint64 frameCount, ma_bool32 loop);
5951 | MA_API ma_result ma_audio_buffer_seek_to_pcm_frame(ma_audio_buffer* pAudioBuffer, ma_uint64
> frameIndex);
5952 | MA_API ma_result ma_audio_buffer_map(ma_audio_buffer* pAudioBuffer, void** ppFramesOut, ma_uint64*
> pFrameCount);
5953 | MA_API ma_result ma_audio_buffer_unmap(ma_audio_buffer* pAudioBuffer, ma_uint64 frameCount); /*
> Returns MA_AT_END if the end has been reached. This should be considered successful. */
5954 | MA_API ma_bool32 ma_audio_buffer_at_end(const ma_audio_buffer* pAudioBuffer);
5955 | MA_API ma_result ma_audio_buffer_get_cursor_in_pcm_frames(const ma_audio_buffer* pAudioBuffer,
> ma_uint64* pCursor);
5956 | MA_API ma_result ma_audio_buffer_get_length_in_pcm_frames(const ma_audio_buffer* pAudioBuffer,
> ma_uint64* pLength);
5957 | MA_API ma_result ma_audio_buffer_get_available_frames(const ma_audio_buffer* pAudioBuffer,
> ma_uint64* pAvailableFrames);
5958 |
5959 |
5960 | /*
5961 | Paged Audio Buffer
5962 | =====
5963 | A paged audio buffer is made up of a linked list of pages. It's expandable, but not shrinkable. It
5964 | can be used for cases where audio data is streamed in asynchronously while allowing data to be read
5965 | at the same time.
5966 |
5967 | This is lock-free, but not 100% thread safe. You can append a page and read from the buffer across
5968 | simultaneously across different threads, however only one thread at a time can append, and only one
5969 | thread at a time can read and seek.
5970 | */
5971 | typedef struct ma_paged_audio_buffer_page ma_paged_audio_buffer_page;
5972 | struct ma_paged_audio_buffer_page
5973 | {
5974 |     MA_ATOMIC(MA_SIZEOF_PTR, ma_paged_audio_buffer_page*) pNext;
5975 |     ma_uint64 sizeInFrames;
5976 |     ma_uint8 pAudioData[1];
5977 | };
5978 |
5979 | typedef struct
5980 | {
5981 |     ma_format format;
5982 |     ma_uint32 channels;
5983 |     ma_paged_audio_buffer_page head; /* Dummy head for the lock-free
> algorithm. Always has a size of 0. */
5984 |     MA_ATOMIC(MA_SIZEOF_PTR, ma_paged_audio_buffer_page*) pTail; /* Never null. Initially set to
> &head. */
5985 | } ma_paged_audio_buffer_data;
5986 |
5987 | MA_API ma_result ma_paged_audio_buffer_data_init(ma_format format, ma_uint32 channels,

```

```

> ma_paged_audio_buffer_data* pData);
5988 | MA_API void ma_paged_audio_buffer_data_uninit(ma_paged_audio_buffer_data* pData, const
> ma_allocation_callbacks* pAllocationCallbacks);
5989 | MA_API ma_paged_audio_buffer_page* ma_paged_audio_buffer_data_get_head(ma_paged_audio_buffer_data*
> pData);
5990 | MA_API ma_paged_audio_buffer_page* ma_paged_audio_buffer_data_get_tail(ma_paged_audio_buffer_data*
> pData);
5991 | MA_API ma_result ma_paged_audio_buffer_data_get_length_in_pcm_frames(ma_paged_audio_buffer_data*
> pData, ma_uint64* pLength);
5992 | MA_API ma_result ma_paged_audio_buffer_data_allocate_page(ma_paged_audio_buffer_data* pData,
> ma_uint64 pageSizeInFrames, const void* pInitialData, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_paged_audio_buffer_page** ppPage);
5993 | MA_API ma_result ma_paged_audio_buffer_data_free_page(ma_paged_audio_buffer_data* pData,
> ma_paged_audio_buffer_page* pPage, const ma_allocation_callbacks* pAllocationCallbacks);
5994 | MA_API ma_result ma_paged_audio_buffer_data_append_page(ma_paged_audio_buffer_data* pData,
> ma_paged_audio_buffer_page* pPage);
5995 | MA_API ma_result ma_paged_audio_buffer_data_allocate_and_append_page(ma_paged_audio_buffer_data*
> pData, ma_uint32 pageSizeInFrames, const void* pInitialData, const ma_allocation_callbacks*
> pAllocationCallbacks);
5996 |
5997 |
5998 | typedef struct
5999 | {
6000 |     ma_paged_audio_buffer_data* pData; /* Must not be null. */
6001 | } ma_paged_audio_buffer_config;
6002 |
6003 | MA_API ma_paged_audio_buffer_config ma_paged_audio_buffer_config_init(ma_paged_audio_buffer_data*
> pData);
6004 |
6005 |
6006 | typedef struct
6007 | {
6008 |     ma_data_source_base ds;
6009 |     ma_paged_audio_buffer_data* pData; /* Audio data is read from here. Cannot be null.
> */
6010 |     ma_paged_audio_buffer_page* pCurrent;
6011 |     ma_uint64 relativeCursor; /* Relative to the current page. */
6012 |     ma_uint64 absoluteCursor;
6013 | } ma_paged_audio_buffer;
6014 |
6015 | MA_API ma_result ma_paged_audio_buffer_init(const ma_paged_audio_buffer_config* pConfig,
> ma_paged_audio_buffer* pPagedAudioBuffer);
6016 | MA_API void ma_paged_audio_buffer_uninit(ma_paged_audio_buffer* pPagedAudioBuffer);
6017 | MA_API ma_result ma_paged_audio_buffer_read_pcm_frames(ma_paged_audio_buffer* pPagedAudioBuffer,
> void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead); /* Returns MA_AT_END if no more
> pages available. */
6018 | MA_API ma_result ma_paged_audio_buffer_seek_to_pcm_frame(ma_paged_audio_buffer* pPagedAudioBuffer,
> ma_uint64 frameIndex);
6019 | MA_API ma_result ma_paged_audio_buffer_get_cursor_in_pcm_frames(ma_paged_audio_buffer*
> pPagedAudioBuffer, ma_uint64* pCursor);
6020 | MA_API ma_result ma_paged_audio_buffer_get_length_in_pcm_frames(ma_paged_audio_buffer*
> pPagedAudioBuffer, ma_uint64* pLength);
6021 |
6022 |
6023 |
6024 | /*****
> *****/
6025 |
6026 | Ring Buffer
6027 |
6028 | *****/
> *****/
6029 | typedef struct
6030 | {
6031 |     void* pBuffer;
6032 |     ma_uint32 subbufferSizeInBytes;
6033 |     ma_uint32 subbufferCount;
6034 |     ma_uint32 subbufferStrideInBytes;
6035 |     MA_ATOMIC(4, ma_uint32) encodedReadOffset; /* Most significant bit is the loop flag. Lower 31
> bits contains the actual offset in bytes. Must be used atomically. */
6036 |     MA_ATOMIC(4, ma_uint32) encodedWriteOffset; /* Most significant bit is the loop flag. Lower 31
> bits contains the actual offset in bytes. Must be used atomically. */
6037 |     ma_bool8 ownsBuffer; /* Used to know whether or not miniaudio is
> responsible for free()-ing the buffer. */
6038 |     ma_bool8 clearOnWriteAcquire; /* When set, clears the acquired write buffer before
> returning from ma_rb_acquire_write(). */
6039 |     ma_allocation_callbacks allocationCallbacks;
6040 | } ma_rb;
6041 |

```

```

6042 | MA_API ma_result ma_rb_init_ex(size_t subbufferSizeInBytes, size_t subbufferCount, size_t
> subbufferStrideInBytes, void* pOptionalPreallocatedBuffer, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_rb* pRB);
6043 | MA_API ma_result ma_rb_init(size_t bufferSizeInBytes, void* pOptionalPreallocatedBuffer, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_rb* pRB);
6044 | MA_API void ma_rb_uninit(ma_rb* pRB);
6045 | MA_API void ma_rb_reset(ma_rb* pRB);
6046 | MA_API ma_result ma_rb_acquire_read(ma_rb* pRB, size_t* pSizeInBytes, void** ppBufferOut);
6047 | MA_API ma_result ma_rb_commit_read(ma_rb* pRB, size_t sizeInBytes);
6048 | MA_API ma_result ma_rb_acquire_write(ma_rb* pRB, size_t* pSizeInBytes, void** ppBufferOut);
6049 | MA_API ma_result ma_rb_commit_write(ma_rb* pRB, size_t sizeInBytes);
6050 | MA_API ma_result ma_rb_seek_read(ma_rb* pRB, size_t offsetInBytes);
6051 | MA_API ma_result ma_rb_seek_write(ma_rb* pRB, size_t offsetInBytes);
6052 | MA_API ma_int32 ma_rb_pointer_distance(ma_rb* pRB); /* Returns the distance between the write
> pointer and the read pointer. Should never be negative for a correct program. Will return the number
> of bytes that can be read before the read pointer hits the write pointer. */
6053 | MA_API ma_uint32 ma_rb_available_read(ma_rb* pRB);
6054 | MA_API ma_uint32 ma_rb_available_write(ma_rb* pRB);
6055 | MA_API size_t ma_rb_get_subbuffer_size(ma_rb* pRB);
6056 | MA_API size_t ma_rb_get_subbuffer_stride(ma_rb* pRB);
6057 | MA_API size_t ma_rb_get_subbuffer_offset(ma_rb* pRB, size_t subbufferIndex);
6058 | MA_API void* ma_rb_get_subbuffer_ptr(ma_rb* pRB, size_t subbufferIndex, void* pBuffer);
6059 |
6060 |
6061 | typedef struct
6062 | {
6063 |     ma_data_source_base ds;
6064 |     ma_rb rb;
6065 |     ma_format format;
6066 |     ma_uint32 channels;
6067 |     ma_uint32 sampleRate; /* Not required for the ring buffer itself, but useful for associating the
> data with some sample rate, particularly for data sources. */
6068 | } ma_pcm_rb;
6069 |
6070 | MA_API ma_result ma_pcm_rb_init_ex(ma_format format, ma_uint32 channels, ma_uint32
> subbufferSizeInFrames, ma_uint32 subbufferCount, ma_uint32 subbufferStrideInFrames, void*
> pOptionalPreallocatedBuffer, const ma_allocation_callbacks* pAllocationCallbacks, ma_pcm_rb* pRB);
6071 | MA_API ma_result ma_pcm_rb_init(ma_format format, ma_uint32 channels, ma_uint32 bufferSizeInFrames,
> void* pOptionalPreallocatedBuffer, const ma_allocation_callbacks* pAllocationCallbacks, ma_pcm_rb*
> pRB);
6072 | MA_API void ma_pcm_rb_uninit(ma_pcm_rb* pRB);
6073 | MA_API void ma_pcm_rb_reset(ma_pcm_rb* pRB);
6074 | MA_API ma_result ma_pcm_rb_acquire_read(ma_pcm_rb* pRB, ma_uint32* pSizeInFrames, void**
> ppBufferOut);
6075 | MA_API ma_result ma_pcm_rb_commit_read(ma_pcm_rb* pRB, ma_uint32 sizeInFrames);
6076 | MA_API ma_result ma_pcm_rb_acquire_write(ma_pcm_rb* pRB, ma_uint32* pSizeInFrames, void**
> ppBufferOut);
6077 | MA_API ma_result ma_pcm_rb_commit_write(ma_pcm_rb* pRB, ma_uint32 sizeInFrames);
6078 | MA_API ma_result ma_pcm_rb_seek_read(ma_pcm_rb* pRB, ma_uint32 offsetInFrames);
6079 | MA_API ma_result ma_pcm_rb_seek_write(ma_pcm_rb* pRB, ma_uint32 offsetInFrames);
6080 | MA_API ma_int32 ma_pcm_rb_pointer_distance(ma_pcm_rb* pRB); /* Return value is in frames. */
6081 | MA_API ma_uint32 ma_pcm_rb_available_read(ma_pcm_rb* pRB);
6082 | MA_API ma_uint32 ma_pcm_rb_available_write(ma_pcm_rb* pRB);
6083 | MA_API ma_uint32 ma_pcm_rb_get_subbuffer_size(ma_pcm_rb* pRB);
6084 | MA_API ma_uint32 ma_pcm_rb_get_subbuffer_stride(ma_pcm_rb* pRB);
6085 | MA_API ma_uint32 ma_pcm_rb_get_subbuffer_offset(ma_pcm_rb* pRB, ma_uint32 subbufferIndex);
6086 | MA_API void* ma_pcm_rb_get_subbuffer_ptr(ma_pcm_rb* pRB, ma_uint32 subbufferIndex, void* pBuffer);
6087 | MA_API ma_format ma_pcm_rb_get_format(const ma_pcm_rb* pRB);
6088 | MA_API ma_uint32 ma_pcm_rb_get_channels(const ma_pcm_rb* pRB);
6089 | MA_API ma_uint32 ma_pcm_rb_get_sample_rate(const ma_pcm_rb* pRB);
6090 | MA_API void ma_pcm_rb_set_sample_rate(ma_pcm_rb* pRB, ma_uint32 sampleRate);
6091 |
6092 |
6093 | /*
6094 | The idea of the duplex ring buffer is to act as the intermediary buffer when running two
> asynchronous devices in a duplex set up. The
6095 | capture device writes to it, and then a playback device reads from it.
6096 |
6097 | At the moment this is just a simple naive implementation, but in the future I want to implement some
> dynamic resampling to seamlessly
6098 | handle desyncs. Note that the API is work in progress and may change at any time in any version.
6099 |
6100 | The size of the buffer is based on the capture side since that's what'll be written to the buffer.
> It is based on the capture period size
6101 | in frames. The internal sample rate of the capture device is also needed in order to calculate the
> size.
6102 | */
6103 | typedef struct
6104 | {

```

```

6105 |     ma_pcm_rb rb;
6106 | } ma_duplex_rb;
6107 |
6108 | MA_API ma_result ma_duplex_rb_init(ma_format captureFormat, ma_uint32 captureChannels, ma_uint32
> sampleRate, ma_uint32 captureInternalSampleRate, ma_uint32 captureInternalPeriodSizeInFrames, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_duplex_rb* pRB);
6109 | MA_API ma_result ma_duplex_rb_uninit(ma_duplex_rb* pRB);
6110 |
6111 |
6112 | /*****
> *****/
6113 |
6114 | Miscellaneous Helpers
6115 |
6116 | *****/
> *****/
6117 | /*
6118 | Retrieves a human readable description of the given result code.
6119 | */
6120 | MA_API const char* ma_result_description(ma_result result);
6121 |
6122 | /*
6123 | malloc()
6124 | */
6125 | MA_API void* ma_malloc(size_t sz, const ma_allocation_callbacks* pAllocationCallbacks);
6126 |
6127 | /*
6128 | calloc()
6129 | */
6130 | MA_API void* ma_calloc(size_t sz, const ma_allocation_callbacks* pAllocationCallbacks);
6131 |
6132 | /*
6133 | realloc()
6134 | */
6135 | MA_API void* ma_realloc(void* p, size_t sz, const ma_allocation_callbacks* pAllocationCallbacks);
6136 |
6137 | /*
6138 | free()
6139 | */
6140 | MA_API void ma_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks);
6141 |
6142 | /*
6143 | Performs an aligned malloc, with the assumption that the alignment is a power of 2.
6144 | */
6145 | MA_API void* ma_aligned_malloc(size_t sz, size_t alignment, const ma_allocation_callbacks*
> pAllocationCallbacks);
6146 |
6147 | /*
6148 | Free's an aligned malloc'd buffer.
6149 | */
6150 | MA_API void ma_aligned_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks);
6151 |
6152 | /*
6153 | Retrieves a friendly name for a format.
6154 | */
6155 | MA_API const char* ma_get_format_name(ma_format format);
6156 |
6157 | /*
6158 | Blends two frames in floating point format.
6159 | */
6160 | MA_API void ma_blend_f32(float* pOut, float* pInA, float* pInB, float factor, ma_uint32 channels);
6161 |
6162 | /*
6163 | Retrieves the size of a sample in bytes for the given format.
6164 |
6165 | This API is efficient and is implemented using a lookup table.
6166 |
6167 | Thread Safety: SAFE
6168 | This API is pure.
6169 | */
6170 | MA_API ma_uint32 ma_get_bytes_per_sample(ma_format format);
6171 | static MA_INLINE ma_uint32 ma_get_bytes_per_frame(ma_format format, ma_uint32 channels) { return
> ma_get_bytes_per_sample(format) * channels; }
6172 |
6173 | /*
6174 | Converts a log level to a string.
6175 | */
6176 | MA_API const char* ma_log_level_to_string(ma_uint32 logLevel);
6177 |

```



```

6178 |
6179 |
6180 |
6181 | /*****
> *****/
6182 |
6183 | Synchronization
6184 |
6185 | *****/
> *****/
6186 | /*
6187 | Locks a spinlock.
6188 | */
6189 | MA_API ma_result ma_spinlock_lock(volatile ma_spinlock* pSpinlock);
6190 |
6191 | /*
6192 | Locks a spinlock, but does not yield() when looping.
6193 | */
6194 | MA_API ma_result ma_spinlock_lock_noyield(volatile ma_spinlock* pSpinlock);
6195 |
6196 | /*
6197 | Unlocks a spinlock.
6198 | */
6199 | MA_API ma_result ma_spinlock_unlock(volatile ma_spinlock* pSpinlock);
6200 |
6201 |
6202 | #ifndef MA_NO_THREADING
6203 |
6204 | /*
6205 | Creates a mutex.
6206 |
6207 | A mutex must be created from a valid context. A mutex is initially unlocked.
6208 | */
6209 | MA_API ma_result ma_mutex_init(ma_mutex* pMutex);
6210 |
6211 | /*
6212 | Deletes a mutex.
6213 | */
6214 | MA_API void ma_mutex_uninit(ma_mutex* pMutex);
6215 |
6216 | /*
6217 | Locks a mutex with an infinite timeout.
6218 | */
6219 | MA_API void ma_mutex_lock(ma_mutex* pMutex);
6220 |
6221 | /*
6222 | Unlocks a mutex.
6223 | */
6224 | MA_API void ma_mutex_unlock(ma_mutex* pMutex);
6225 |
6226 |
6227 | /*
6228 | Initializes an auto-reset event.
6229 | */
6230 | MA_API ma_result ma_event_init(ma_event* pEvent);
6231 |
6232 | /*
6233 | Uninitializes an auto-reset event.
6234 | */
6235 | MA_API void ma_event_uninit(ma_event* pEvent);
6236 |
6237 | /*
6238 | Waits for the specified auto-reset event to become signalled.
6239 | */
6240 | MA_API ma_result ma_event_wait(ma_event* pEvent);
6241 |
6242 | /*
6243 | Signals the specified auto-reset event.
6244 | */
6245 | MA_API ma_result ma_event_signal(ma_event* pEvent);
6246 |
6247 |
6248 | MA_API ma_result ma_semaphore_init(int initialValue, ma_semaphore* pSemaphore);
6249 | MA_API void ma_semaphore_uninit(ma_semaphore* pSemaphore);
6250 | MA_API ma_result ma_semaphore_wait(ma_semaphore* pSemaphore);
6251 | MA_API ma_result ma_semaphore_release(ma_semaphore* pSemaphore);
6252 | #endif /* MA_NO_THREADING */
6253 |
6254 |

```

```

6255 | /*
6256 | Fence
6257 | =====
6258 | This locks while the counter is larger than 0. Counter can be incremented and decremented by any
6259 | thread, but care needs to be taken when waiting. It is possible for one thread to acquire the
6260 | fence just as another thread returns from ma_fence_wait().
6261 |
6262 | The idea behind a fence is to allow you to wait for a group of operations to complete. When an
6263 | operation starts, the counter is incremented which locks the fence. When the operation completes,
6264 | the fence will be released which decrements the counter. ma_fence_wait() will block until the
6265 | counter hits zero.
6266 |
6267 | If threading is disabled, ma_fence_wait() will spin on the counter.
6268 | */
6269 | typedef struct
6270 | {
6271 | #ifndef MA_NO_THREADING
6272 |     ma_event e;
6273 | #endif
6274 |     ma_uint32 counter;
6275 | } ma_fence;
6276 |
6277 | MA_API ma_result ma_fence_init(ma_fence* pFence);
6278 | MA_API void ma_fence_uninit(ma_fence* pFence);
6279 | MA_API ma_result ma_fence_acquire(ma_fence* pFence);    /* Increment counter. */
6280 | MA_API ma_result ma_fence_release(ma_fence* pFence);    /* Decrement counter. */
6281 | MA_API ma_result ma_fence_wait(ma_fence* pFence);        /* Wait for counter to reach 0. */
6282 |
6283 |
6284 |
6285 | /*
6286 | Notification callback for asynchronous operations.
6287 | */
6288 | typedef void ma_async_notification;
6289 |
6290 | typedef struct
6291 | {
6292 |     void (* onSignal)(ma_async_notification* pNotification);
6293 | } ma_async_notification_callbacks;
6294 |
6295 | MA_API ma_result ma_async_notification_signal(ma_async_notification* pNotification);
6296 |
6297 |
6298 | /*
6299 | Simple polling notification.
6300 |
6301 | This just sets a variable when the notification has been signalled which is then polled with
6302 | > ma_async_notification_poll_is_signalled()
6303 | */
6304 | typedef struct
6305 | {
6306 |     ma_async_notification_callbacks cb;
6307 |     ma_bool32 signalled;
6308 | } ma_async_notification_poll;
6309 |
6310 | MA_API ma_result ma_async_notification_poll_init(ma_async_notification_poll* pNotificationPoll);
6311 | MA_API ma_bool32 ma_async_notification_poll_is_signalled(const ma_async_notification_poll*
6312 | > pNotificationPoll);
6313 |
6314 | /*
6315 | Event Notification
6316 |
6317 | This uses an ma_event. If threading is disabled (MA_NO_THREADING), initialization will fail.
6318 | */
6319 | typedef struct
6320 | {
6321 |     ma_async_notification_callbacks cb;
6322 | #ifndef MA_NO_THREADING
6323 |     ma_event e;
6324 | #endif
6325 | } ma_async_notification_event;
6326 |
6327 | MA_API ma_result ma_async_notification_event_init(ma_async_notification_event* pNotificationEvent);
6328 | MA_API ma_result ma_async_notification_event_uninit(ma_async_notification_event*
6329 | > pNotificationEvent);
6330 |
6331 | MA_API ma_result ma_async_notification_event_wait(ma_async_notification_event* pNotificationEvent);
6332 | MA_API ma_result ma_async_notification_event_signal(ma_async_notification_event*
6333 | > pNotificationEvent);

```

```

6330 |
6331 |
6332 |
6333 |
6334 | /*****
> *****/
6335 |
6336 | Job Queue
6337 |
6338 | *****/
> *****/
6339 |
6340 | /*
6341 | Slot Allocator
6342 | -----
6343 | The idea of the slot allocator is for it to be used in conjunction with a fixed sized buffer. You
> use the slot allocator to allocate an index that can be used
6344 | as the insertion point for an object.
6345 |
6346 | Slots are reference counted to help mitigate the ABA problem in the lock-free queue we use for
> tracking jobs.
6347 |
6348 | The slot index is stored in the low 32 bits. The reference counter is stored in the high 32 bits:
6349 |
6350 | +-----+-----+
6351 | | 32 Bits      | 32 Bits      |
6352 | +-----+-----+
6353 | | Reference Count | Slot Index |
6354 | +-----+-----+
6355 | */
6356 | typedef struct
6357 | {
6358 |     ma_uint32 capacity; /* The number of slots to make available. */
6359 | } ma_slot_allocator_config;
6360 |
6361 | MA_API ma_slot_allocator_config ma_slot_allocator_config_init(ma_uint32 capacity);
6362 |
6363 |
6364 | typedef struct
6365 | {
6366 |     MA_ATOMIC(4, ma_uint32) bitfield; /* Must be used atomically because the allocation and
> freeing routines need to make copies of this which must never be optimized away by the compiler. */
6367 | } ma_slot_allocator_group;
6368 |
6369 | typedef struct
6370 | {
6371 |     ma_slot_allocator_group* pGroups; /* Slots are grouped in chunks of 32. */
6372 |     ma_uint32* pSlots; /* 32 bits for reference counting for ABA mitigation. */
6373 |     ma_uint32 count; /* Allocation count. */
6374 |     ma_uint32 capacity;
6375 |
6376 |     /* Memory management. */
6377 |     ma_bool32 _ownsHeap;
6378 |     void* _pHeap;
6379 | } ma_slot_allocator;
6380 |
6381 | MA_API ma_result ma_slot_allocator_get_heap_size(const ma_slot_allocator_config* pConfig, size_t*
> pHeapSizeInBytes);
6382 | MA_API ma_result ma_slot_allocator_init_preallocated(const ma_slot_allocator_config* pConfig, void*
> pHeap, ma_slot_allocator* pAllocator);
6383 | MA_API ma_result ma_slot_allocator_init(const ma_slot_allocator_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_slot_allocator* pAllocator);
6384 | MA_API void ma_slot_allocator_uninit(ma_slot_allocator* pAllocator, const ma_allocation_callbacks*
> pAllocationCallbacks);
6385 | MA_API ma_result ma_slot_allocator_alloc(ma_slot_allocator* pAllocator, ma_uint64* pSlot);
6386 | MA_API ma_result ma_slot_allocator_free(ma_slot_allocator* pAllocator, ma_uint64 slot);
6387 |
6388 |
6389 | typedef struct ma_job ma_job;
6390 |
6391 | /*
6392 | Callback for processing a job. Each job type will have their own processing callback which will be
6393 | called by ma_job_process().
6394 | */
6395 | typedef ma_result (* ma_job_proc)(ma_job* pJob);
6396 |
6397 | /* When a job type is added here an callback needs to be added go "g_jobVTable" in the
> implementation section. */
6398 | typedef enum

```

```

6399 | {
6400 |     /* Miscellaneous. */
6401 |     MA_JOB_TYPE_QUIT = 0,
6402 |     MA_JOB_TYPE_CUSTOM,
6403 |
6404 |     /* Resource Manager. */
6405 |     MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER_NODE,
6406 |     MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER_NODE,
6407 |     MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE,
6408 |     MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER,
6409 |     MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER,
6410 |     MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_STREAM,
6411 |     MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_STREAM,
6412 |     MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_STREAM,
6413 |     MA_JOB_TYPE_RESOURCE_MANAGER_SEEK_DATA_STREAM,
6414 |
6415 |     /* Device. */
6416 |     MA_JOB_TYPE_DEVICE_AAUDIO_REROUTE,
6417 |
6418 |     /* Count. Must always be last. */
6419 |     MA_JOB_TYPE_COUNT
6420 | } ma_job_type;
6421 |
6422 | struct ma_job
6423 | {
6424 |     union
6425 |     {
6426 |         struct
6427 |         {
6428 |             ma_uint16 code;          /* Job type. */
6429 |             ma_uint16 slot;          /* Index into a ma_slot_allocator. */
6430 |             ma_uint32 refcount;
6431 |         } breakup;
6432 |         ma_uint64 allocation;
6433 |     } toc; /* 8 bytes. We encode the job code into the slot allocation data to save space. */
6434 |     MA_ATOMIC(8, ma_uint64) next; /* refcount + slot for the next item. Does not include the job
> code. */
6435 |     ma_uint32 order; /* Execution order. Used to create a data dependency and ensure a job is
> executed in order. Usage is contextual depending on the job type. */
6436 |
6437 |     union
6438 |     {
6439 |         /* Miscellaneous. */
6440 |         struct
6441 |         {
6442 |             ma_job_proc proc;
6443 |             ma_uintptr data0;
6444 |             ma_uintptr data1;
6445 |         } custom;
6446 |
6447 |         /* Resource Manager */
6448 |         union
6449 |         {
6450 |             struct
6451 |             {
6452 |                 /*ma_resource_manager**/ void* pResourceManager;
6453 |                 /*ma_resource_manager_data_buffer_node**/ void* pDataBufferNode;
6454 |                 char* pFilePath;
6455 |                 wchar_t* pFilePathW;
6456 |                 ma_uint32 flags; /* Resource manager data source
> flags that were used when initializing the data buffer. */
6457 |                 ma_async_notification* pInitNotification; /* Signalled when the data buffer
> has been initialized and the format/channels/rate can be retrieved. */
6458 |                 ma_async_notification* pDoneNotification; /* Signalled when the data buffer
> has been fully decoded. Will be passed through to MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE
> when decoding. */
6459 |                 ma_fence* pInitFence; /* Released when initialization of
> the decoder is complete. */
6460 |                 ma_fence* pDoneFence; /* Released if initialization of the
> decoder fails. Passed through to PAGE_DATA_BUFFER_NODE untouched if init is successful. */
6461 |             } loadDataBufferNode;
6462 |             struct
6463 |             {
6464 |                 /*ma_resource_manager**/ void* pResourceManager;
6465 |                 /*ma_resource_manager_data_buffer_node**/ void* pDataBufferNode;
6466 |                 ma_async_notification* pDoneNotification;
6467 |                 ma_fence* pDoneFence;
6468 |             } freeDataBufferNode;
6469 |         } struct

```

```

6470 |         {
6471 |             /*ma_resource_manager**/ void* pResourceManager;
6472 |             /*ma_resource_manager_data_buffer_node**/ void* pDataBufferNode;
6473 |             /*ma_decoder**/ void* pDecoder;
6474 |             ma_async_notification* pDoneNotification;          /* Signalled when the data buffer
> has been fully decoded. */
6475 |             ma_fence* pDoneFence;                              /* Passed through from
> LOAD_DATA_BUFFER_NODE and released when the data buffer completes decoding or an error occurs. */
6476 |         } pageDataBufferNode;
6477 |
6478 |         struct
6479 |         {
6480 |             /*ma_resource_manager_data_buffer**/ void* pDataBuffer;
6481 |             ma_async_notification* pInitNotification;          /* Signalled when the data buffer
> has been initialized and the format/channels/rate can be retrieved. */
6482 |             ma_async_notification* pDoneNotification;          /* Signalled when the data buffer
> has been fully decoded. */
6483 |             ma_fence* pInitFence;                              /* Released when the data buffer has
> been initialized and the format/channels/rate can be retrieved. */
6484 |             ma_fence* pDoneFence;                              /* Released when the data buffer has
> been fully decoded. */
6485 |             ma_uint64 rangeBegInPCMFrames;
6486 |             ma_uint64 rangeEndInPCMFrames;
6487 |             ma_uint64 loopPointBegInPCMFrames;
6488 |             ma_uint64 loopPointEndInPCMFrames;
6489 |             ma_uint32 isLooping;
6490 |         } loadDataBuffer;
6491 |         struct
6492 |         {
6493 |             /*ma_resource_manager_data_buffer**/ void* pDataBuffer;
6494 |             ma_async_notification* pDoneNotification;
6495 |             ma_fence* pDoneFence;
6496 |         } freeDataBuffer;
6497 |
6498 |         struct
6499 |         {
6500 |             /*ma_resource_manager_data_stream**/ void* pDataStream;
6501 |             char* pFilePath;                                    /* Allocated when the job is posted,
> freed by the job thread after loading. */
6502 |             wchar_t* pFilePathW;                               /* ^ As above ^. Only used if pFilePath
> is NULL. */
6503 |             ma_uint64 initialSeekPoint;
6504 |             ma_async_notification* pInitNotification;          /* Signalled after the first two pages
> have been decoded and frames can be read from the stream. */
6505 |             ma_fence* pInitFence;
6506 |         } loadDataStream;
6507 |         struct
6508 |         {
6509 |             /*ma_resource_manager_data_stream**/ void* pDataStream;
6510 |             ma_async_notification* pDoneNotification;
6511 |             ma_fence* pDoneFence;
6512 |         } freeDataStream;
6513 |         struct
6514 |         {
6515 |             /*ma_resource_manager_data_stream**/ void* pDataStream;
6516 |             ma_uint32 pageIndex;                               /* The index of the page to decode into. */
6517 |         } pageDataStream;
6518 |         struct
6519 |         {
6520 |             /*ma_resource_manager_data_stream**/ void* pDataStream;
6521 |             ma_uint64 frameIndex;
6522 |         } seekDataStream;
6523 |     } resourceManager;
6524 |
6525 |     /* Device. */
6526 |     union
6527 |     {
6528 |         union
6529 |         {
6530 |             struct
6531 |             {
6532 |                 /*ma_device**/ void* pDevice;
6533 |                 /*ma_device_type*/ ma_uint32 deviceType;
6534 |             } reroute;
6535 |             } aaudio;
6536 |         } device;
6537 |     } data;
6538 | };
6539 |

```

```

6540 | MA_API ma_job ma_job_init(ma_uint16 code);
6541 | MA_API ma_result ma_job_process(ma_job* pJob);
6542 |
6543 |
6544 | /*
6545 | When set, ma_job_queue_next() will not wait and no semaphore will be signaled in
6546 | ma_job_queue_post(). ma_job_queue_next() will return MA_NO_DATA_AVAILABLE if nothing is available.
6547 |
6548 | This flag should always be used for platforms that do not support multithreading.
6549 | */
6550 | typedef enum
6551 | {
6552 |     MA_JOB_QUEUE_FLAG_NON_BLOCKING = 0x00000001
6553 | } ma_job_queue_flags;
6554 |
6555 | typedef struct
6556 | {
6557 |     ma_uint32 flags;
6558 |     ma_uint32 capacity; /* The maximum number of jobs that can fit in the queue at a time. */
6559 | } ma_job_queue_config;
6560 |
6561 | MA_API ma_job_queue_config ma_job_queue_config_init(ma_uint32 flags, ma_uint32 capacity);
6562 |
6563 |
6564 | typedef struct
6565 | {
6566 |     ma_uint32 flags; /* Flags passed in at initialization time. */
6567 |     ma_uint32 capacity; /* The maximum number of jobs that can fit in the queue at a
> time. Set by the config. */
6568 |     MA_ATOMIC(8, ma_uint64) head; /* The first item in the list. Required for removing from the
> top of the list. */
6569 |     MA_ATOMIC(8, ma_uint64) tail; /* The last item in the list. Required for appending to the end
> of the list. */
6570 | #ifndef MA_NO_THREADING
6571 |     ma_semaphore sem; /* Only used when MA_JOB_QUEUE_FLAG_NON_BLOCKING is unset. */
6572 | #endif
6573 |     ma_slot_allocator allocator;
6574 |     ma_job* pJobs;
6575 | #ifndef MA_USE_EXPERIMENTAL_LOCK_FREE_JOB_QUEUE
6576 |     ma_spinlock lock;
6577 | #endif
6578 |
6579 |     /* Memory management. */
6580 |     void* _pHeap;
6581 |     ma_bool32 _ownsHeap;
6582 | } ma_job_queue;
6583 |
6584 | MA_API ma_result ma_job_queue_get_heap_size(const ma_job_queue_config* pConfig, size_t*
> pHeapSizeInBytes);
6585 | MA_API ma_result ma_job_queue_init_preallocated(const ma_job_queue_config* pConfig, void* pHeap,
> ma_job_queue* pQueue);
6586 | MA_API ma_result ma_job_queue_init(const ma_job_queue_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_job_queue* pQueue);
6587 | MA_API void ma_job_queue_uninit(ma_job_queue* pQueue, const ma_allocation_callbacks*
> pAllocationCallbacks);
6588 | MA_API ma_result ma_job_queue_post(ma_job_queue* pQueue, const ma_job* pJob);
6589 | MA_API ma_result ma_job_queue_next(ma_job_queue* pQueue, ma_job* pJob); /* Returns MA_CANCELLED if
> the next job is a quit job. */
6590 |
6591 |
6592 |
6593 | /*****
> *****/
6594 | /*****
> *****/
6595 |
6596 | DEVICE I/O
6597 | =====
6598 |
6599 | This section contains the APIs for device playback and capture. Here is where you'll find
> ma_device_init(), etc.
6600 |
6601 | *****/
> *****/
6602 | *****/
> *****/
6603 | #ifndef MA_NO_DEVICE_IO
6604 | /* Some backends are only supported on certain platforms. */
6605 | #if defined(MA_WIN32) && !defined(MA_XBOX)

```

```

6606 |     #define MA_SUPPORT_WASAPI
6607 |
6608 |     #if defined(MA_WIN32_DESKTOP)    /* DirectSound and WinMM backends are only supported on
> desktops. */
6609 |         #define MA_SUPPORT_DSOUND
6610 |         #define MA_SUPPORT_WINMM
6611 |         #define MA_SUPPORT_JACK    /* JACK is technically supported on Windows, but I don't know
> how many people use it in practice... */
6612 |     #endif
6613 | #endif
6614 | #if defined(MA_UNIX) && !defined(MA_ORBIS) && !defined(MA_PROSPERO)
6615 |     #if defined(MA_LINUX)
6616 |         #if !defined(MA_ANDROID) && !defined(MA_EMSCRIPTEN)    /* ALSA is not supported on Android.
> */
6617 |             #define MA_SUPPORT_ALSA
6618 |         #endif
6619 |     #endif
6620 |     #if !defined(MA_BSD) && !defined(MA_ANDROID) && !defined(MA_EMSCRIPTEN)
6621 |         #define MA_SUPPORT_PULSEAUDIO
6622 |         #define MA_SUPPORT_JACK
6623 |     #endif
6624 |     #if defined(__OpenBSD__)    /* <-- Change this to "#if defined(MA_BSD)" to enable sndio on
> all BSD flavors. */
6625 |         #define MA_SUPPORT_SNDDIO    /* sndio is only supported on OpenBSD for now. May be expanded
> later if there's demand. */
6626 |     #endif
6627 |     #if defined(__NetBSD__) || defined(__OpenBSD__)
6628 |         #define MA_SUPPORT_AUDIOL4    /* Only support audio(4) on platforms with known support. */
6629 |     #endif
6630 |     #if defined(__FreeBSD__) || defined(__DragonFly__)
6631 |         #define MA_SUPPORT_OSS    /* Only support OSS on specific platforms with known support. */
6632 |     #endif
6633 | #endif
6634 | #if defined(MA_ANDROID)
6635 |     #define MA_SUPPORT_AAUDIO
6636 |     #define MA_SUPPORT_OPENSLES
6637 | #endif
6638 | #if defined(MA_APPLE)
6639 |     #define MA_SUPPORT_COREAUDIO
6640 | #endif
6641 | #if defined(MA_EMSCRIPTEN)
6642 |     #define MA_SUPPORT_WEBAUDIO
6643 | #endif
6644 |
6645 | /* All platforms should support custom backends. */
6646 | #define MA_SUPPORT_CUSTOM
6647 |
6648 | /* Explicitly disable the Null backend for Emscripten because it uses a background thread which is
> not properly supported right now. */
6649 | #if !defined(MA_EMSCRIPTEN)
6650 | #define MA_SUPPORT_NULL
6651 | #endif
6652 |
6653 |
6654 | #if defined(MA_SUPPORT_WASAPI) && !defined(MA_NO_WASAPI) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_WASAPI))
6655 |     #define MA_HAS_WASAPI
6656 | #endif
6657 | #if defined(MA_SUPPORT_DSOUND) && !defined(MA_NO_DSOUND) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_DSOUND))
6658 |     #define MA_HAS_DSOUND
6659 | #endif
6660 | #if defined(MA_SUPPORT_WINMM) && !defined(MA_NO_WINMM) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_WINMM))
6661 |     #define MA_HAS_WINMM
6662 | #endif
6663 | #if defined(MA_SUPPORT_ALSA) && !defined(MA_NO_ALSA) && (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS)
> || defined(MA_ENABLE_ALSA))
6664 |     #define MA_HAS_ALSA
6665 | #endif
6666 | #if defined(MA_SUPPORT_PULSEAUDIO) && !defined(MA_NO_PULSEAUDIO) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_PULSEAUDIO))
6667 |     #define MA_HAS_PULSEAUDIO
6668 | #endif
6669 | #if defined(MA_SUPPORT_JACK) && !defined(MA_NO_JACK) && (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS)
> || defined(MA_ENABLE_JACK))
6670 |     #define MA_HAS_JACK
6671 | #endif
6672 | #if defined(MA_SUPPORT_COREAUDIO) && !defined(MA_NO_COREAUDIO) &&

```



```

> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_COREAUDIO))
6673 |     #define MA_HAS_COREAUDIO
6674 | #endif
6675 | #if defined(MA_SUPPORT_SNDIO) && !defined(MA_NO_SNDIO) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_SNDIO))
6676 |     #define MA_HAS_SNDIO
6677 | #endif
6678 | #if defined(MA_SUPPORT_AUDIO4) && !defined(MA_NO_AUDIO4) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_AUDIO4))
6679 |     #define MA_HAS_AUDIO4
6680 | #endif
6681 | #if defined(MA_SUPPORT_OSS) && !defined(MA_NO_OSS) && (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) ||
> defined(MA_ENABLE_OSS))
6682 |     #define MA_HAS_OSS
6683 | #endif
6684 | #if defined(MA_SUPPORT_AAUDIO) && !defined(MA_NO_AAUDIO) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_AAUDIO))
6685 |     #define MA_HAS_AAUDIO
6686 | #endif
6687 | #if defined(MA_SUPPORT_OPENS�) && !defined(MA_NO_OPENS�) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_OPENS�))
6688 |     #define MA_HAS_OPENS�
6689 | #endif
6690 | #if defined(MA_SUPPORT_WEBAUDIO) && !defined(MA_NO_WEBAUDIO) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_WEBAUDIO))
6691 |     #define MA_HAS_WEBAUDIO
6692 | #endif
6693 | #if defined(MA_SUPPORT_CUSTOM) && !defined(MA_NO_CUSTOM) &&
> (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS) || defined(MA_ENABLE_CUSTOM))
6694 |     #define MA_HAS_CUSTOM
6695 | #endif
6696 | #if defined(MA_SUPPORT_NULL) && !defined(MA_NO_NULL) && (!defined(MA_ENABLE_ONLY_SPECIFIC_BACKENDS)
> || defined(MA_ENABLE_NULL))
6697 |     #define MA_HAS_NULL
6698 | #endif
6699 |
6700 | typedef enum
6701 | {
6702 |     ma_device_state_uninitialized = 0,
6703 |     ma_device_state_stopped      = 1, /* The device's default state after initialization. */
6704 |     ma_device_state_started      = 2, /* The device is started and is requesting and/or delivering
> audio data. */
6705 |     ma_device_state_starting     = 3, /* Transitioning from a stopped state to started. */
6706 |     ma_device_state_stopping     = 4  /* Transitioning from a started state to stopped. */
6707 | } ma_device_state;
6708 |
6709 | MA_ATOMIC_SAFE_TYPE_DECL(i32, 4, device_state)
6710 |
6711 |
6712 | #ifdef MA_SUPPORT_WASAPI
6713 | /* We need a IMMNotificationClient object for WASAPI. */
6714 | typedef struct
6715 | {
6716 |     void* lpVtbl;
6717 |     ma_uint32 counter;
6718 |     ma_device* pDevice;
6719 | } ma_IMMNotificationClient;
6720 | #endif
6721 |
6722 | /* Backend enums must be in priority order. */
6723 | typedef enum
6724 | {
6725 |     ma_backend_wasapi,
6726 |     ma_backend_dsound,
6727 |     ma_backend_winmm,
6728 |     ma_backend_coreaudio,
6729 |     ma_backend_sndio,
6730 |     ma_backend_audio4,
6731 |     ma_backend_oss,
6732 |     ma_backend_pulseaudio,
6733 |     ma_backend_alas,
6734 |     ma_backend_jack,
6735 |     ma_backend_aaudio,
6736 |     ma_backend_opensl,
6737 |     ma_backend_webaudio,
6738 |     ma_backend_custom, /* <-- Custom backend, with callbacks defined by the context config. */
6739 |     ma_backend_null    /* <-- Must always be the last item. Lowest priority, and used as the
> terminator for backend enumeration. */
6740 | } ma_backend;

```

```

6741 |
6742 | #define MA_BACKEND_COUNT (ma_backend_null+1)
6743 |
6744 |
6745 | /*
6746 | Device job thread. This is used by backends that require asynchronous processing of certain
6747 | operations. It is not used by all backends.
6748 |
6749 | The device job thread is made up of a thread and a job queue. You can post a job to the thread with
6750 | ma_device_job_thread_post(). The thread will do the processing of the job.
6751 | */
6752 | typedef struct
6753 | {
6754 |     ma_bool32 noThread; /* Set this to true if you want to process jobs yourself. */
6755 |     ma_uint32 jobQueueCapacity;
6756 |     ma_uint32 jobQueueFlags;
6757 | } ma_device_job_thread_config;
6758 |
6759 | MA_API ma_device_job_thread_config ma_device_job_thread_config_init(void);
6760 |
6761 | typedef struct
6762 | {
6763 |     ma_thread thread;
6764 |     ma_job_queue jobQueue;
6765 |     ma_bool32 _hasThread;
6766 | } ma_device_job_thread;
6767 |
6768 | MA_API ma_result ma_device_job_thread_init(const ma_device_job_thread_config* pConfig, const
6769 | > ma_allocation_callbacks* pAllocationCallbacks, ma_device_job_thread* pJobThread);
6770 | MA_API void ma_device_job_thread_uninit(ma_device_job_thread* pJobThread, const
6771 | > ma_allocation_callbacks* pAllocationCallbacks);
6772 | MA_API ma_result ma_device_job_thread_post(ma_device_job_thread* pJobThread, const ma_job* pJob);
6773 | MA_API ma_result ma_device_job_thread_next(ma_device_job_thread* pJobThread, ma_job* pJob);
6774 |
6775 | /* Device notification types. */
6776 | typedef enum
6777 | {
6778 |     ma_device_notification_type_started,
6779 |     ma_device_notification_type_stopped,
6780 |     ma_device_notification_type_rerouted,
6781 |     ma_device_notification_type_interruption_began,
6782 |     ma_device_notification_type_interruption_ended,
6783 |     ma_device_notification_type_unlocked
6784 | } ma_device_notification_type;
6785 |
6786 | typedef struct
6787 | {
6788 |     ma_device* pDevice;
6789 |     ma_device_notification_type type;
6790 |     union
6791 |     {
6792 |         struct
6793 |         {
6794 |             int _unused;
6795 |         } started;
6796 |         struct
6797 |         {
6798 |             int _unused;
6799 |         } stopped;
6800 |         struct
6801 |         {
6802 |             int _unused;
6803 |         } rerouted;
6804 |         struct
6805 |         {
6806 |             int _unused;
6807 |         } interruption;
6808 |     } data;
6809 | } ma_device_notification;
6810 |
6811 | /*
6812 | The notification callback for when the application should be notified of a change to the device.
6813 |
6814 | This callback is used for notifying the application of changes such as when the device has started,
6815 | stopped, rerouted or an interruption has occurred. Note that not all backends will post all
6816 | notification types. For example, some backends will perform automatic stream routing without any
6817 | kind of notification to the host program which means miniaudio will never know about it and will

```

```

6818 | never be able to fire the rerouted notification. You should keep this in mind when designing your
6819 | program.
6820 |
6821 | The stopped notification will *not* get fired when a device is rerouted.
6822 |
6823 |
6824 | Parameters
6825 | -----
6826 | pNotification (in)
6827 |     A pointer to a structure containing information about the event. Use the `pDevice` member of
6828 |     this object to retrieve the relevant device. The `type` member can be used to discriminate
6829 |     against each of the notification types.
6830 |
6831 |
6832 | Remarks
6833 | -----
6834 | Do not restart or uninitialized the device from the callback.
6835 |
6836 | Not all notifications will be triggered by all backends, however the started and stopped events
6837 | should be reliable for all backends. Some backends do not have a good way to detect device
6838 | stoppages due to unplugging the device which may result in the stopped callback not getting
6839 | fired. This has been observed with at least one BSD variant.
6840 |
6841 | The rerouted notification is fired *after* the reroute has occurred. The stopped notification will
6842 | *not* get fired when a device is rerouted. The following backends are known to do automatic stream
6843 | rerouting, but do not have a way to be notified of the change:
6844 |
6845 |     * DirectSound
6846 |
6847 | The interruption notifications are used on mobile platforms for detecting when audio is interrupted
6848 | due to things like an incoming phone call. Currently this is only implemented on iOS. None of the
6849 | Android backends will report this notification.
6850 | */
6851 | typedef void (* ma_device_notification_proc)(const ma_device_notification* pNotification);
6852 |
6853 |
6854 | /*
6855 | The callback for processing audio data from the device.
6856 |
6857 | The data callback is fired by miniaudio whenever the device needs to have more data delivered to a
6858 | > playback device, or when a capture device has some data
6859 | > available. This is called as soon as the backend asks for more data which means it may be called
6860 | > with inconsistent frame counts. You cannot assume the
6861 | > callback will be fired with a consistent frame count.
6862 |
6863 | Parameters
6864 | -----
6865 | pDevice (in)
6866 |     A pointer to the relevant device.
6867 |
6868 | pOutput (out)
6869 |     A pointer to the output buffer that will receive audio data that will later be played back
6870 | > through the speakers. This will be non-null for a playback or
6871 | > full-duplex device and null for a capture and loopback device.
6872 |
6873 | pInput (in)
6874 |     A pointer to the buffer containing input data from a recording device. This will be non-null for
6875 | > a capture, full-duplex or loopback device and null for a
6876 | > playback device.
6877 |
6878 | frameCount (in)
6879 |     The number of PCM frames to process. Note that this will not necessarily be equal to what you
6880 | > requested when you initialized the device. The
6881 | > `periodSizeInFrames` and `periodSizeInMilliseconds` members of the device config are just hints,
6882 | > and are not necessarily exactly what you'll get. You must
6883 | > not assume this will always be the same value each time the callback is fired.
6884 |
6885 | Remarks
6886 | -----
6887 | You cannot stop and start the device from inside the callback or else you'll get a deadlock. You
6888 | > must also not uninitialized the device from inside the
6889 | > callback. The following APIs cannot be called from inside the callback:
6890 |
6891 |     ma_device_init()
6892 |     ma_device_init_ex()
6893 |     ma_device_uninit()
6894 |     ma_device_start()

```

```

6890 |     ma_device_stop()
6891 |
6892 | The proper way to stop the device is to call `ma_device_stop()` from a different thread, normally
> the main application thread.
6893 | */
6894 | typedef void (* ma_device_data_proc)(ma_device* pDevice, void* pOutput, const void* pInput,
> ma_uint32 frameCount);
6895 |
6896 |
6897 |
6898 |
6899 | /*
6900 | DEPRECATED. Use ma_device_notification_proc instead.
6901 |
6902 | The callback for when the device has been stopped.
6903 |
6904 | This will be called when the device is stopped explicitly with `ma_device_stop()` and also called
> implicitly when the device is stopped through external forces
6905 | such as being unplugged or an internal error occurring.
6906 |
6907 |
6908 | Parameters
6909 | -----
6910 | pDevice (in)
6911 |     A pointer to the device that has just stopped.
6912 |
6913 |
6914 | Remarks
6915 | -----
6916 | Do not restart or uninitialized the device from the callback.
6917 | */
6918 | typedef void (* ma_stop_proc)(ma_device* pDevice); /* DEPRECATED. Use ma_device_notification_proc
> instead. */
6919 |
6920 | typedef enum
6921 | {
6922 |     ma_device_type_playback = 1,
6923 |     ma_device_type_capture = 2,
6924 |     ma_device_type_duplex = ma_device_type_playback | ma_device_type_capture, /* 3 */
6925 |     ma_device_type_loopback = 4
6926 | } ma_device_type;
6927 |
6928 | typedef enum
6929 | {
6930 |     ma_share_mode_shared = 0,
6931 |     ma_share_mode_exclusive
6932 | } ma_share_mode;
6933 |
6934 | /* iOS/tvOS/watchOS session categories. */
6935 | typedef enum
6936 | {
6937 |     ma_ios_session_category_default = 0, /* AVAudioSessionCategoryPlayAndRecord. */
6938 |     ma_ios_session_category_none, /* Leave the session category unchanged. */
6939 |     ma_ios_session_category_ambient, /* AVAudioSessionCategoryAmbient */
6940 |     ma_ios_session_category_solo_ambient, /* AVAudioSessionCategorySoloAmbient */
6941 |     ma_ios_session_category_playback, /* AVAudioSessionCategoryPlayback */
6942 |     ma_ios_session_category_record, /* AVAudioSessionCategoryRecord */
6943 |     ma_ios_session_category_play_and_record, /* AVAudioSessionCategoryPlayAndRecord */
6944 |     ma_ios_session_category_multi_route /* AVAudioSessionCategoryMultiRoute */
6945 | } ma_ios_session_category;
6946 |
6947 | /* iOS/tvOS/watchOS session category options */
6948 | typedef enum
6949 | {
6950 |     ma_ios_session_category_option_mix_with_others = 0x01, /*
> AVAudioSessionCategoryOptionMixWithOthers */
6951 |     ma_ios_session_category_option_duck_others = 0x02, /*
> AVAudioSessionCategoryOptionDuckOthers */
6952 |     ma_ios_session_category_option_allow_bluetooth = 0x04, /*
> AVAudioSessionCategoryOptionAllowBluetooth */
6953 |     ma_ios_session_category_option_default_to_speaker = 0x08, /*
> AVAudioSessionCategoryOptionDefaultToSpeaker */
6954 |     ma_ios_session_category_option_interrupt_spoken_audio_and_mix_with_others = 0x11, /*
> AVAudioSessionCategoryOptionInterruptSpokenAudioAndMixWithOthers */
6955 |     ma_ios_session_category_option_allow_bluetooth_a2dp = 0x20, /*
> AVAudioSessionCategoryOptionAllowBluetoothA2DP */
6956 |     ma_ios_session_category_option_allow_air_play = 0x40, /*
> AVAudioSessionCategoryOptionAllowAirPlay */
6957 | } ma_ios_session_category_option;

```

```

6958 |
6959 | /* OpenSL stream types. */
6960 | typedef enum
6961 | {
6962 |     ma_opensl_stream_type_default = 0,          /* Leaves the stream type unset. */
6963 |     ma_opensl_stream_type_voice,                /* SL_ANDROID_STREAM_VOICE */
6964 |     ma_opensl_stream_type_system,               /* SL_ANDROID_STREAM_SYSTEM */
6965 |     ma_opensl_stream_type_ring,                 /* SL_ANDROID_STREAM_RING */
6966 |     ma_opensl_stream_type_media,                /* SL_ANDROID_STREAM_MEDIA */
6967 |     ma_opensl_stream_type_alarm,                /* SL_ANDROID_STREAM_ALARM */
6968 |     ma_opensl_stream_type_notification          /* SL_ANDROID_STREAM_NOTIFICATION */
6969 | } ma_opensl_stream_type;
6970 |
6971 | /* OpenSL recording presets. */
6972 | typedef enum
6973 | {
6974 |     ma_opensl_recording_preset_default = 0,      /* Leaves the input preset unset. */
6975 |     ma_opensl_recording_preset_generic,          /* SL_ANDROID_RECORDING_PRESET_GENERIC */
6976 |     ma_opensl_recording_preset_camcorder,        /* SL_ANDROID_RECORDING_PRESET_CAMCORDER */
6977 |     ma_opensl_recording_preset_voice_recognition, /* SL_ANDROID_RECORDING_PRESET_VOICE_RECOGNITION */
6978 |     ma_opensl_recording_preset_voice_communication, /*
> SL_ANDROID_RECORDING_PRESET_VOICE_COMMUNICATION */
6979 |     ma_opensl_recording_preset_voice_unprocessed /* SL_ANDROID_RECORDING_PRESET_UNPROCESSED */
6980 | } ma_opensl_recording_preset;
6981 |
6982 | /* WASAPI audio thread priority characteristics. */
6983 | typedef enum
6984 | {
6985 |     ma_wasapi_usage_default = 0,
6986 |     ma_wasapi_usage_games,
6987 |     ma_wasapi_usage_pro_audio,
6988 | } ma_wasapi_usage;
6989 |
6990 | /* AAudio usage types. */
6991 | typedef enum
6992 | {
6993 |     ma_aaudio_usage_default = 0,                /* Leaves the usage type unset. */
6994 |     ma_aaudio_usage_media,                      /* AUDIO_USAGE_MEDIA */
6995 |     ma_aaudio_usage_voice_communication,         /* AUDIO_USAGE_VOICE_COMMUNICATION */
6996 |     ma_aaudio_usage_voice_communication_signalling, /* AUDIO_USAGE_VOICE_COMMUNICATION_SIGNALLING */
6997 |     ma_aaudio_usage_alarm,                      /* AUDIO_USAGE_ALARM */
6998 |     ma_aaudio_usage_notification,               /* AUDIO_USAGE_NOTIFICATION */
6999 |     ma_aaudio_usage_notification_ringtone,      /* AUDIO_USAGE_NOTIFICATION_RINGTONE */
7000 |     ma_aaudio_usage_notification_event,         /* AUDIO_USAGE_NOTIFICATION_EVENT */
7001 |     ma_aaudio_usage_assistance_accessibility,    /* AUDIO_USAGE_ASSISTANCE_ACCESSIBILITY */
7002 |     ma_aaudio_usage_assistance_navigation_guidance, /* AUDIO_USAGE_ASSISTANCE_NAVIGATION_GUIDANCE */
7003 |     ma_aaudio_usage_assistance_sonification,    /* AUDIO_USAGE_ASSISTANCE_SONIFICATION */
7004 |     ma_aaudio_usage_game,                      /* AUDIO_USAGE_GAME */
7005 |     ma_aaudio_usage_assitant,                   /* AUDIO_USAGE_ASSISTANT */
7006 |     ma_aaudio_usage_emergency,                  /* AUDIO_SYSTEM_USAGE_EMERGENCY */
7007 |     ma_aaudio_usage_safety,                     /* AUDIO_SYSTEM_USAGE_SAFETY */
7008 |     ma_aaudio_usage_vehicle_status,             /* AUDIO_SYSTEM_USAGE_VEHICLE_STATUS */
7009 |     ma_aaudio_usage_announcement                /* AUDIO_SYSTEM_USAGE_ANNOUNCEMENT */
7010 | } ma_aaudio_usage;
7011 |
7012 | /* AAudio content types. */
7013 | typedef enum
7014 | {
7015 |     ma_aaudio_content_type_default = 0,          /* Leaves the content type unset. */
7016 |     ma_aaudio_content_type_speech,              /* AUDIO_CONTENT_TYPE_SPEECH */
7017 |     ma_aaudio_content_type_music,               /* AUDIO_CONTENT_TYPE_MUSIC */
7018 |     ma_aaudio_content_type_movie,               /* AUDIO_CONTENT_TYPE_MOVIE */
7019 |     ma_aaudio_content_type_sonification          /* AUDIO_CONTENT_TYPE_SONIFICATION */
7020 | } ma_aaudio_content_type;
7021 |
7022 | /* AAudio input presets. */
7023 | typedef enum
7024 | {
7025 |     ma_aaudio_input_preset_default = 0,          /* Leaves the input preset unset. */
7026 |     ma_aaudio_input_preset_generic,              /* AUDIO_INPUT_PRESET_GENERIC */
7027 |     ma_aaudio_input_preset_camcorder,            /* AUDIO_INPUT_PRESET_CAMCORDER */
7028 |     ma_aaudio_input_preset_voice_recognition,    /* AUDIO_INPUT_PRESET_VOICE_RECOGNITION */
7029 |     ma_aaudio_input_preset_voice_communication,  /* AUDIO_INPUT_PRESET_VOICE_COMMUNICATION */
7030 |     ma_aaudio_input_preset_unprocessed,          /* AUDIO_INPUT_PRESET_UNPROCESSED */
7031 |     ma_aaudio_input_preset_voice_performance     /* AUDIO_INPUT_PRESET_VOICE_PERFORMANCE */
7032 | } ma_aaudio_input_preset;

```

```

7033 |
7034 | typedef enum
7035 | {
7036 |     ma_audio_allow_capture_default = 0,          /* Leaves the allowed capture policy unset. */
7037 |     ma_audio_allow_capture_by_all,              /* AAUDIO_ALLOW_CAPTURE_BY_ALL */
7038 |     ma_audio_allow_capture_by_system,           /* AAUDIO_ALLOW_CAPTURE_BY_SYSTEM */
7039 |     ma_audio_allow_capture_by_none              /* AAUDIO_ALLOW_CAPTURE_BY_NONE */
7040 | } ma_audio_allowed_capture_policy;
7041 |
7042 | typedef union
7043 | {
7044 |     ma_int64 counter;
7045 |     double counterD;
7046 | } ma_timer;
7047 |
7048 | typedef union
7049 | {
7050 |     ma_wchar_win32 wasapi[64];                  /* WASAPI uses a wchar_t string for identification. */
7051 |     ma_uint8 dsound[16];                        /* DirectSound uses a GUID for identification. */
7052 |     /*UINT_PTR*/ ma_uint32 winmm;               /* When creating a device, WinMM expects a Win32 UINT_PTR for
> device identification. In practice it's actually just a UINT. */
7053 |     char alsa[256];                             /* ALSA uses a name string for identification. */
7054 |     char pulse[256];                             /* PulseAudio uses a name string for identification. */
7055 |     int jack;                                    /* JACK always uses default devices. */
7056 |     char coreaudio[256];                         /* Core Audio uses a string for identification. */
7057 |     char sndio[256];                             /* "snd/0", etc. */
7058 |     char audio4[256];                           /* "/dev/audio", etc. */
7059 |     char oss[64];                               /* "dev/dsp0", etc. "dev/dsp" for the default device. */
7060 |     ma_int32 aaudio;                             /* AAudio uses a 32-bit integer for identification. */
7061 |     ma_uint32 opensl;                            /* OpenSL|ES uses a 32-bit unsigned integer for identification.
> */
7062 |     char webaudio[32];                          /* Web Audio always uses default devices for now, but if this
> changes it'll be a GUID. */
7063 |     union
7064 |     {
7065 |         int i;
7066 |         char s[256];
7067 |         void* p;
7068 |     } custom;                                   /* The custom backend could be anything. Give them a few
> options. */
7069 |     int nullbackend;                            /* The null backend uses an integer for device IDs. */
7070 | } ma_device_id;
7071 |
7072 | MA_API ma_bool32 ma_device_id_equal(const ma_device_id* pA, const ma_device_id* pB);
7073 |
7074 |
7075 | typedef struct ma_context_config ma_context_config;
7076 | typedef struct ma_device_config ma_device_config;
7077 | typedef struct ma_backend_callbacks ma_backend_callbacks;
7078 |
7079 | #define MA_DATA_FORMAT_FLAG_EXCLUSIVE_MODE (1U << 1) /* If set, this is supported in exclusive
> mode. Otherwise not natively supported by exclusive mode. */
7080 |
7081 | #ifndef MA_MAX_DEVICE_NAME_LENGTH
7082 | #define MA_MAX_DEVICE_NAME_LENGTH 255
7083 | #endif
7084 |
7085 | typedef struct
7086 | {
7087 |     /* Basic info. This is the only information guaranteed to be filled in during device
> enumeration. */
7088 |     ma_device_id id;
7089 |     char name[MA_MAX_DEVICE_NAME_LENGTH + 1]; /* +1 for null terminator. */
7090 |     ma_bool32 isDefault;
7091 |
7092 |     ma_uint32 nativeDataFormatCount;
7093 |     struct
7094 |     {
7095 |         ma_format format; /* Sample format. If set to ma_format_unknown, all sample formats
> are supported. */
7096 |         ma_uint32 channels; /* If set to 0, all channels are supported. */
7097 |         ma_uint32 sampleRate; /* If set to 0, all sample rates are supported. */
7098 |         ma_uint32 flags; /* A combination of MA_DATA_FORMAT_FLAG_* flags. */
7099 |     } nativeDataFormats[/*ma_format_count * ma_standard_sample_rate_count * MA_MAX_CHANNELS*/ 64];
> /* Not sure how big to make this. There can be *many* permutations for virtual devices which can
> support anything. */
7100 | } ma_device_info;
7101 |
7102 | struct ma_device_config

```

```

7103 | {
7104 |     ma_device_type deviceType;
7105 |     ma_uint32 sampleRate;
7106 |     ma_uint32 periodSizeInFrames;
7107 |     ma_uint32 periodSizeInMilliseconds;
7108 |     ma_uint32 periods;
7109 |     ma_performance_profile performanceProfile;
7110 |     ma_bool8 noPreSilencedOutputBuffer; /* When set to true, the contents of the output buffer
> passed into the data callback will be left undefined rather than initialized to silence. */
7111 |     ma_bool8 noClip; /* When set to true, the contents of the output buffer
> passed into the data callback will not be clipped after returning. Only applies when the playback
> sample format is f32. */
7112 |     ma_bool8 noDisableDenormals; /* Do not disable denormals when firing the data callback.
> */
7113 |     ma_bool8 noFixedSizedCallback; /* Disables strict fixed-sized data callbacks. Setting this
> to true will result in the period size being treated only as a hint to the backend. This is an
> optimization for those who don't need fixed sized callbacks. */
7114 |     ma_device_data_proc dataCallback;
7115 |     ma_device_notification_proc notificationCallback;
7116 |     ma_stop_proc stopCallback;
7117 |     void* pUserData;
7118 |     ma_resampler_config resampling;
7119 |     struct
7120 |     {
7121 |         const ma_device_id* pDeviceID;
7122 |         ma_format format;
7123 |         ma_uint32 channels;
7124 |         ma_channel* pChannelMap;
7125 |         ma_channel_mix_mode channelMixMode;
7126 |         ma_bool32 calculateLFEFromSpatialChannels; /* When an output LFE channel is present, but no
> input LFE, set to true to set the output LFE to the average of all spatial channels (LR, FR, etc.).
> Ignored when an input LFE is present. */
7127 |         ma_share_mode shareMode;
7128 |     } playback;
7129 |     struct
7130 |     {
7131 |         const ma_device_id* pDeviceID;
7132 |         ma_format format;
7133 |         ma_uint32 channels;
7134 |         ma_channel* pChannelMap;
7135 |         ma_channel_mix_mode channelMixMode;
7136 |         ma_bool32 calculateLFEFromSpatialChannels; /* When an output LFE channel is present, but no
> input LFE, set to true to set the output LFE to the average of all spatial channels (LR, FR, etc.).
> Ignored when an input LFE is present. */
7137 |         ma_share_mode shareMode;
7138 |     } capture;
7139 |     struct
7140 |     {
7141 |         ma_wasapi_usage usage; /* When configured, uses Avrt APIs to set the thread
> characteristics. */
7142 |         ma_bool8 noAutoConvertSRC; /* When set to true, disables the use of
> AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM. */
7143 |         ma_bool8 noDefaultQualitySRC; /* When set to true, disables the use of
> AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY. */
7144 |         ma_bool8 noAutoStreamRouting; /* Disables automatic stream routing. */
7145 |         ma_bool8 noHardwareOffloading; /* Disables WASAPI's hardware offloading feature. */
7146 |         ma_uint32 loopbackProcessID; /* The process ID to include or exclude for loopback
> mode. Set to 0 to capture audio from all processes. Ignored when an explicit device ID is specified.
> */
7147 |         ma_bool8 loopbackProcessExclude; /* When set to true, excludes the process specified by
> loopbackProcessID. By default, the process will be included. */
7148 |     } wasapi;
7149 |     struct
7150 |     {
7151 |         ma_bool32 noMMap; /* Disables MMap mode. */
7152 |         ma_bool32 noAutoFormat; /* Opens the ALSA device with SND_PCM_NO_AUTO_FORMAT. */
7153 |         ma_bool32 noAutoChannels; /* Opens the ALSA device with SND_PCM_NO_AUTO_CHANNELS. */
7154 |         ma_bool32 noAutoResample; /* Opens the ALSA device with SND_PCM_NO_AUTO_RESAMPLE. */
7155 |     } alsa;
7156 |     struct
7157 |     {
7158 |         const char* pStreamNamePlayback;
7159 |         const char* pStreamNameCapture;
7160 |         int channelMap;
7161 |     } pulse;
7162 |     struct
7163 |     {
7164 |         ma_bool32 allowNominalSampleRateChange; /* Desktop only. When enabled, allows changing of

```



```

> the sample rate at the operating system level. */
7166 |     } coreaudio;
7167 |     struct
7168 |     {
7169 |         ma_opensl_stream_type streamType;
7170 |         ma_opensl_recording_preset recordingPreset;
7171 |         ma_bool32 enableCompatibilityWorkarounds;
7172 |     } openssl;
7173 |     struct
7174 |     {
7175 |         ma_aaudio_usage usage;
7176 |         ma_aaudio_content_type contentType;
7177 |         ma_aaudio_input_preset inputPreset;
7178 |         ma_aaudio_allowed_capture_policy allowedCapturePolicy;
7179 |         ma_bool32 noAutoStartAfterReroute;
7180 |         ma_bool32 enableCompatibilityWorkarounds;
7181 |         ma_bool32 allowSetBufferCapacity;
7182 |     } aaudio;
7183 | };
7184 |
7185 |
7186 | /*
7187 | The callback for handling device enumeration. This is fired from `ma_context_enumerate_devices()`.
7188 |
7189 | Parameters
7190 | -----
7191 | pContext (in)
7192 |     A pointer to the context performing the enumeration.
7193 |
7194 | deviceType (in)
7195 |     The type of the device being enumerated. This will always be either `ma_device_type_playback` or
7196 | > `ma_device_type_capture`.
7197 |
7198 | pInfo (in)
7199 |     A pointer to a `ma_device_info` containing the ID and name of the enumerated device. Note that
7200 | > this will not include detailed information about the device,
7201 | > only basic information (ID and name). The reason for this is that it would otherwise require
7202 | > opening the backend device to probe for the information which
7203 | > is too inefficient.
7204 |
7205 | pUserData (in)
7206 |     The user data pointer passed into `ma_context_enumerate_devices()`.
7207 | */
7208 |
7209 | typedef ma_bool32 (* ma_enum_devices_callback_proc)(ma_context* pContext, ma_device_type deviceType,
7210 | > const ma_device_info* pInfo, void* pUserData);
7211 |
7212 | /*
7213 | Describes some basic details about a playback or capture device.
7214 | */
7215 | typedef struct
7216 | {
7217 |     const ma_device_id* pDeviceID;
7218 |     ma_share_mode shareMode;
7219 |     ma_format format;
7220 |     ma_uint32 channels;
7221 |     ma_uint32 sampleRate;
7222 |     ma_channel channelMap[MA_MAX_CHANNELS];
7223 |     ma_uint32 periodSizeInFrames;
7224 |     ma_uint32 periodSizeInMilliseconds;
7225 |     ma_uint32 periodCount;
7226 | } ma_device_descriptor;
7227 |
7228 | /*
7229 | These are the callbacks required to be implemented for a backend. These callbacks are grouped into
7230 | > two parts: context and device. There is one context
7231 | > to many devices. A device is created from a context.
7232 |
7233 | The general flow goes like this:
7234 |
7235 | 1) A context is created with `onContextInit()`
7236 | 1a) Available devices can be enumerated with `onContextEnumerateDevices()` if required.
7237 | 1b) Detailed information about a device can be queried with `onContextGetDeviceInfo()` if
7238 | > required.
7239 | 2) A device is created from the context that was created in the first step using `onDeviceInit()`,
7240 | > and optionally a device ID that was
7241 |     selected from device enumeration via `onContextEnumerateDevices()`.
7242 | 3) A device is started or stopped with `onDeviceStart()` / `onDeviceStop()`

```

```

7237 | 4) Data is delivered to and from the device by the backend. This is always done based on the
7238 | > native format returned by the prior call
7239 | to `onDeviceInit()`. Conversion between the device's native format and the format requested by
7240 | > the application will be handled by
7241 | miniaudio internally.
7242 | Initialization of the context is quite simple. You need to do any necessary initialization of
7243 | > internal objects and then output the
7244 | callbacks defined in this structure.
7245 | Once the context has been initialized you can initialize a device. Before doing so, however, the
7246 | > application may want to know which
7247 | physical devices are available. This is where `onContextEnumerateDevices()` comes in. This is fairly
7248 | > simple. For each device, fire the
7249 | given callback with, at a minimum, the basic information filled out in `ma_device_info`. When the
7250 | > callback returns `MA_FALSE`, enumeration
7251 | needs to stop and the `onContextEnumerateDevices()` function returns with a success code.
7252 | Detailed device information can be retrieved from a device ID using `onContextGetDeviceInfo()`. This
7253 | > takes as input the device type and ID,
7254 | and on output returns detailed information about the device in `ma_device_info`. The
7255 | > `onContextGetDeviceInfo()` callback must handle the
7256 | case when the device ID is NULL, in which case information about the default device needs to be
7257 | > retrieved.
7258 | Once the context has been created and the device ID retrieved (if using anything other than the
7259 | > default device), the device can be created.
7260 | This is a little bit more complicated than initialization of the context due to its more complicated
7261 | > configuration. When initializing a
7262 | device, a duplex device may be requested. This means a separate data format needs to be specified
7263 | > for both playback and capture. On input,
7264 | the data format is set to what the application wants. On output it's set to the native format which
7265 | > should match as closely as possible to
7266 | the requested format. The conversion between the format requested by the application and the
7267 | > device's native format will be handled
7268 | internally by miniaudio.
7269 | On input, if the sample format is set to `ma_format_unknown`, the backend is free to use whatever
7270 | > sample format it desires, so long as it's
7271 | supported by miniaudio. When the channel count is set to 0, the backend should use the device's
7272 | > native channel count. The same applies for
7273 | sample rate. For the channel map, the default should be used when `ma_channel_map_is_blank()`
7274 | > returns true (all channels set to
7275 | `MA_CHANNEL_NONE`). On input, the `periodSizeInFrames` or `periodSizeInMilliseconds` option should
7276 | > always be set. The backend should
7277 | inspect both of these variables. If `periodSizeInFrames` is set, it should take priority, otherwise
7278 | > it needs to be derived from the period
7279 | size in milliseconds (`periodSizeInMilliseconds`) and the sample rate, keeping in mind that the
7280 | > sample rate may be 0, in which case the
7281 | sample rate will need to be determined before calculating the period size in frames. On output, all
7282 | > members of the `ma_device_descriptor`
7283 | object should be set to a valid value, except for `periodSizeInMilliseconds` which is optional
7284 | > (`periodSizeInFrames` *must* be set).
7285 | Starting and stopping of the device is done with `onDeviceStart()` and `onDeviceStop()` and should
7286 | > be self-explanatory. If the backend uses
7287 | asynchronous reading and writing, `onDeviceStart()` and `onDeviceStop()` should always be
7288 | > implemented.
7289 | The handling of data delivery between the application and the device is the most complicated part of
7290 | > the process. To make this a bit
7291 | easier, some helper callbacks are available. If the backend uses a blocking read/write style of API,
7292 | > the `onDeviceRead()` and
7293 | `onDeviceWrite()` callbacks can optionally be implemented. These are blocking and work just like
7294 | > reading and writing from a file. If the
7295 | backend uses a callback for data delivery, that callback must call
7296 | > `ma_device_handle_backend_data_callback()` from within its callback.
7297 | This allows miniaudio to then process any necessary data conversion and then pass it to the
7298 | > miniaudio data callback.
7299 | If the backend requires absolute flexibility with its data delivery, it can optionally implement the
7300 | > `onDeviceDataLoop()` callback
7301 | which will allow it to implement the logic that will run on the audio thread. This is much more
7302 | > advanced and is completely optional.
7303 | The audio thread should run data delivery logic in a loop while `ma_device_get_state() ==
7304 | > ma_device_state_started` and no errors have been
7305 | encountered. Do not start or stop the device here. That will be handled from outside the
7306 | > `onDeviceDataLoop()` callback.

```

```

7283 |
7284 | The invocation of the `onDeviceDataLoop()` callback will be handled by miniaudio. When you start the
> device, miniaudio will fire this
7285 | callback. When the device is stopped, the `ma_device_get_state() == ma_device_state_started`
> condition will fail and the loop will be terminated
7286 | which will then fall through to the part that stops the device. For an example on how to implement
> the `onDeviceDataLoop()` callback,
7287 | look at `ma_device_audio_thread_default_read_write()`. Implement the `onDeviceDataLoopWakeup()`
> callback if you need a mechanism to
7288 | wake up the audio thread.
7289 |
7290 | If the backend supports an optimized retrieval of device information from an initialized `ma_device`
> object, it should implement the
7291 | `onDeviceGetInfo()` callback. This is optional, in which case it will fall back to
> `onContextGetDeviceInfo()` which is less efficient.
7292 | */
7293 | struct ma_backend_callbacks
7294 | {
7295 |     ma_result (* onContextInit)(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks);
7296 |     ma_result (* onContextUninit)(ma_context* pContext);
7297 |     ma_result (* onContextEnumerateDevices)(ma_context* pContext, ma_enum_devices_callback_proc
> callback, void* pUserData);
7298 |     ma_result (* onContextGetDeviceInfo)(ma_context* pContext, ma_device_type deviceType, const
> ma_device_id* pDeviceID, ma_device_info* pDeviceInfo);
7299 |     ma_result (* onDeviceInit)(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture);
7300 |     ma_result (* onDeviceUninit)(ma_device* pDevice);
7301 |     ma_result (* onDeviceStart)(ma_device* pDevice);
7302 |     ma_result (* onDeviceStop)(ma_device* pDevice);
7303 |     ma_result (* onDeviceRead)(ma_device* pDevice, void* pFrames, ma_uint32 frameCount, ma_uint32*
> pFramesRead);
7304 |     ma_result (* onDeviceWrite)(ma_device* pDevice, const void* pFrames, ma_uint32 frameCount,
> ma_uint32* pFramesWritten);
7305 |     ma_result (* onDeviceDataLoop)(ma_device* pDevice);
7306 |     ma_result (* onDeviceDataLoopWakeup)(ma_device* pDevice);
7307 |     ma_result (* onDeviceGetInfo)(ma_device* pDevice, ma_device_type type, ma_device_info*
> pDeviceInfo);
7308 | };
7309 |
7310 | struct ma_context_config
7311 | {
7312 |     ma_log* pLog;
7313 |     ma_thread_priority threadPriority;
7314 |     size_t threadStackSize;
7315 |     void* pUserData;
7316 |     ma_allocation_callbacks allocationCallbacks;
7317 |     struct
7318 |     {
7319 |         ma_handle hWnd; /* HWND. Optional window handle to pass into SetCooperativeLevel(). Will
> default to the foreground window, and if that fails, the desktop window. */
7320 |     } dsound;
7321 |     struct
7322 |     {
7323 |         ma_bool32 useVerboseDeviceEnumeration;
7324 |     } alsa;
7325 |     struct
7326 |     {
7327 |         const char* pApplicationName;
7328 |         const char* pServerName;
7329 |         ma_bool32 tryAutoSpawn; /* Enables autospawning of the PulseAudio daemon if necessary. */
7330 |     } pulse;
7331 |     struct
7332 |     {
7333 |         ma_ios_session_category sessionCategory;
7334 |         ma_uint32 sessionCategoryOptions;
7335 |         ma_bool32 noAudioSessionActivate; /* iOS only. When set to true, does not perform an
> explicit [[AVAudioSession sharedInstance] setActive:true] on initialization. */
7336 |         ma_bool32 noAudioSessionDeactivate; /* iOS only. When set to true, does not perform an
> explicit [[AVAudioSession sharedInstance] setActive:false] on uninitialization. */
7337 |     } coreaudio;
7338 |     struct
7339 |     {
7340 |         const char* pClientName;
7341 |         ma_bool32 tryStartServer;
7342 |     } jack;
7343 |     ma_backend_callbacks custom;
7344 | };
7345 |

```

```

7346 | /* WASAPI specific structure for some commands which must run on a common thread due to bugs in
> WASAPI. */
7347 | typedef struct
7348 | {
7349 |     int code;
7350 |     ma_event* pEvent; /* This will be signalled when the event is complete. */
7351 |     union
7352 |     {
7353 |         struct
7354 |         {
7355 |             int _unused;
7356 |         } quit;
7357 |         struct
7358 |         {
7359 |             ma_device_type deviceType;
7360 |             void* pAudioClient;
7361 |             void** ppAudioClientService;
7362 |             ma_result* pResult; /* The result from creating the audio client service. */
7363 |         } createAudioClient;
7364 |         struct
7365 |         {
7366 |             ma_device* pDevice;
7367 |             ma_device_type deviceType;
7368 |         } releaseAudioClient;
7369 |     } data;
7370 | } ma_context_command_wasapi;
7371 |
7372 | struct ma_context
7373 | {
7374 |     ma_backend_callbacks callbacks;
7375 |     ma_backend backend; /* DirectSound, ALSA, etc. */
7376 |     ma_log* pLog;
7377 |     ma_log log; /* Only used if the log is owned by the context. The pLog member will be set to &log
> in this case. */
7378 |     ma_thread_priority threadPriority;
7379 |     size_t threadStackSize;
7380 |     void* pUserData;
7381 |     ma_allocation_callbacks allocationCallbacks;
7382 |     ma_mutex deviceEnumLock; /* Used to make ma_context_get_devices() thread safe. */
7383 |     ma_mutex deviceInfoLock; /* Used to make ma_context_get_device_info() thread safe. */
7384 |     ma_uint32 deviceInfoCapacity; /* Total capacity of pDeviceInfos. */
7385 |     ma_uint32 playbackDeviceInfoCount;
7386 |     ma_uint32 captureDeviceInfoCount;
7387 |     ma_device_info* pDeviceInfos; /* Playback devices first, then capture. */
7388 |
7389 |     union
7390 |     {
7391 | #ifdef MA_SUPPORT_WASAPI
7392 |         struct
7393 |         {
7394 |             ma_thread commandThread;
7395 |             ma_mutex commandLock;
7396 |             ma_semaphore commandSem;
7397 |             ma_uint32 commandIndex;
7398 |             ma_uint32 commandCount;
7399 |             ma_context_command_wasapi commands[4];
7400 |             ma_handle hAvrt;
7401 |             ma_proc AvSetMmThreadCharacteristicsA;
7402 |             ma_proc AvRevertMmThreadcharacteristics;
7403 |             ma_handle hMMDevapi;
7404 |             ma_proc ActivateAudioInterfaceAsync;
7405 |         } wasapi;
7406 | #endif
7407 | #ifdef MA_SUPPORT_DSOUND
7408 |         struct
7409 |         {
7410 |             ma_handle hWnd; /* Can be null. */
7411 |             ma_handle hDSoundDLL;
7412 |             ma_proc DirectSoundCreate;
7413 |             ma_proc DirectSoundEnumerateA;
7414 |             ma_proc DirectSoundCaptureCreate;
7415 |             ma_proc DirectSoundCaptureEnumerateA;
7416 |         } dsound;
7417 | #endif
7418 | #ifdef MA_SUPPORT_WINMM
7419 |         struct
7420 |         {
7421 |             ma_handle hWinMM;
7422 |             ma_proc waveOutGetNumDevs;

```

```
7423 |         ma_proc waveOutGetDevCapsA;
7424 |         ma_proc waveOutOpen;
7425 |         ma_proc waveOutClose;
7426 |         ma_proc waveOutPrepareHeader;
7427 |         ma_proc waveOutUnprepareHeader;
7428 |         ma_proc waveOutWrite;
7429 |         ma_proc waveOutReset;
7430 |         ma_proc waveInGetNumDevs;
7431 |         ma_proc waveInGetDevCapsA;
7432 |         ma_proc waveInOpen;
7433 |         ma_proc waveInClose;
7434 |         ma_proc waveInPrepareHeader;
7435 |         ma_proc waveInUnprepareHeader;
7436 |         ma_proc waveInAddBuffer;
7437 |         ma_proc waveInStart;
7438 |         ma_proc waveInReset;
7439 |     } winmm;
7440 | #endif
7441 | #ifndef MA_SUPPORT_ALSA
7442 |     struct
7443 |     {
7444 |         ma_handle asoundSO;
7445 |         ma_proc snd_pcm_open;
7446 |         ma_proc snd_pcm_close;
7447 |         ma_proc snd_pcm_hw_params_sizeof;
7448 |         ma_proc snd_pcm_hw_params_any;
7449 |         ma_proc snd_pcm_hw_params_set_format;
7450 |         ma_proc snd_pcm_hw_params_set_format_first;
7451 |         ma_proc snd_pcm_hw_params_get_format_mask;
7452 |         ma_proc snd_pcm_hw_params_set_channels;
7453 |         ma_proc snd_pcm_hw_params_set_channels_near;
7454 |         ma_proc snd_pcm_hw_params_set_channels_minmax;
7455 |         ma_proc snd_pcm_hw_params_set_rate_resample;
7456 |         ma_proc snd_pcm_hw_params_set_rate;
7457 |         ma_proc snd_pcm_hw_params_set_rate_near;
7458 |         ma_proc snd_pcm_hw_params_set_rate_minmax;
7459 |         ma_proc snd_pcm_hw_params_set_buffer_size_near;
7460 |         ma_proc snd_pcm_hw_params_set_periods_near;
7461 |         ma_proc snd_pcm_hw_params_set_access;
7462 |         ma_proc snd_pcm_hw_params_get_format;
7463 |         ma_proc snd_pcm_hw_params_get_channels;
7464 |         ma_proc snd_pcm_hw_params_get_channels_min;
7465 |         ma_proc snd_pcm_hw_params_get_channels_max;
7466 |         ma_proc snd_pcm_hw_params_get_rate;
7467 |         ma_proc snd_pcm_hw_params_get_rate_min;
7468 |         ma_proc snd_pcm_hw_params_get_rate_max;
7469 |         ma_proc snd_pcm_hw_params_get_buffer_size;
7470 |         ma_proc snd_pcm_hw_params_get_periods;
7471 |         ma_proc snd_pcm_hw_params_get_access;
7472 |         ma_proc snd_pcm_hw_params_test_format;
7473 |         ma_proc snd_pcm_hw_params_test_channels;
7474 |         ma_proc snd_pcm_hw_params_test_rate;
7475 |         ma_proc snd_pcm_hw_params;
7476 |         ma_proc snd_pcm_sw_params_sizeof;
7477 |         ma_proc snd_pcm_sw_params_current;
7478 |         ma_proc snd_pcm_sw_params_get_boundary;
7479 |         ma_proc snd_pcm_sw_params_set_avail_min;
7480 |         ma_proc snd_pcm_sw_params_set_start_threshold;
7481 |         ma_proc snd_pcm_sw_params_set_stop_threshold;
7482 |         ma_proc snd_pcm_sw_params;
7483 |         ma_proc snd_pcm_format_mask_sizeof;
7484 |         ma_proc snd_pcm_format_mask_test;
7485 |         ma_proc snd_pcm_get_chmap;
7486 |         ma_proc snd_pcm_state;
7487 |         ma_proc snd_pcm_prepare;
7488 |         ma_proc snd_pcm_start;
7489 |         ma_proc snd_pcm_drop;
7490 |         ma_proc snd_pcm_drain;
7491 |         ma_proc snd_pcm_reset;
7492 |         ma_proc snd_device_name_hint;
7493 |         ma_proc snd_device_name_get_hint;
7494 |         ma_proc snd_card_get_index;
7495 |         ma_proc snd_device_name_free_hint;
7496 |         ma_proc snd_pcm_mmap_begin;
7497 |         ma_proc snd_pcm_mmap_commit;
7498 |         ma_proc snd_pcm_recover;
7499 |         ma_proc snd_pcm_readi;
7500 |         ma_proc snd_pcm_writeti;
7501 |         ma_proc snd_pcm_avail;
```

```
7502 |         ma_proc snd_pcm_avail_update;
7503 |         ma_proc snd_pcm_wait;
7504 |         ma_proc snd_pcm_nonblock;
7505 |         ma_proc snd_pcm_info;
7506 |         ma_proc snd_pcm_info_sizeof;
7507 |         ma_proc snd_pcm_info_get_name;
7508 |         ma_proc snd_pcm_poll_descriptors;
7509 |         ma_proc snd_pcm_poll_descriptors_count;
7510 |         ma_proc snd_pcm_poll_descriptors_revents;
7511 |         ma_proc snd_config_update_free_global;
7512 |
7513 |         ma_mutex internalDeviceEnumLock;
7514 |         ma_bool32 useVerboseDeviceEnumeration;
7515 |     } alsa;
7516 | #endif
7517 | #ifdef MA_SUPPORT_PULSEAUDIO
7518 |     struct
7519 |     {
7520 |         ma_handle pulseS0;
7521 |         ma_proc pa_mainloop_new;
7522 |         ma_proc pa_mainloop_free;
7523 |         ma_proc pa_mainloop_quit;
7524 |         ma_proc pa_mainloop_get_api;
7525 |         ma_proc pa_mainloop_iterate;
7526 |         ma_proc pa_mainloop_wakeup;
7527 |         ma_proc pa_threaded_mainloop_new;
7528 |         ma_proc pa_threaded_mainloop_free;
7529 |         ma_proc pa_threaded_mainloop_start;
7530 |         ma_proc pa_threaded_mainloop_stop;
7531 |         ma_proc pa_threaded_mainloop_lock;
7532 |         ma_proc pa_threaded_mainloop_unlock;
7533 |         ma_proc pa_threaded_mainloop_wait;
7534 |         ma_proc pa_threaded_mainloop_signal;
7535 |         ma_proc pa_threaded_mainloop_accept;
7536 |         ma_proc pa_threaded_mainloop_get_retval;
7537 |         ma_proc pa_threaded_mainloop_get_api;
7538 |         ma_proc pa_threaded_mainloop_in_thread;
7539 |         ma_proc pa_threaded_mainloop_set_name;
7540 |         ma_proc pa_context_new;
7541 |         ma_proc pa_context_unref;
7542 |         ma_proc pa_context_connect;
7543 |         ma_proc pa_context_disconnect;
7544 |         ma_proc pa_context_set_state_callback;
7545 |         ma_proc pa_context_get_state;
7546 |         ma_proc pa_context_get_sink_info_list;
7547 |         ma_proc pa_context_get_source_info_list;
7548 |         ma_proc pa_context_get_sink_info_by_name;
7549 |         ma_proc pa_context_get_source_info_by_name;
7550 |         ma_proc pa_operation_unref;
7551 |         ma_proc pa_operation_get_state;
7552 |         ma_proc pa_channel_map_init_extend;
7553 |         ma_proc pa_channel_map_valid;
7554 |         ma_proc pa_channel_map_compatible;
7555 |         ma_proc pa_stream_new;
7556 |         ma_proc pa_stream_unref;
7557 |         ma_proc pa_stream_connect_playback;
7558 |         ma_proc pa_stream_connect_record;
7559 |         ma_proc pa_stream_disconnect;
7560 |         ma_proc pa_stream_get_state;
7561 |         ma_proc pa_stream_get_sample_spec;
7562 |         ma_proc pa_stream_get_channel_map;
7563 |         ma_proc pa_stream_get_buffer_attr;
7564 |         ma_proc pa_stream_set_buffer_attr;
7565 |         ma_proc pa_stream_get_device_name;
7566 |         ma_proc pa_stream_set_write_callback;
7567 |         ma_proc pa_stream_set_read_callback;
7568 |         ma_proc pa_stream_set_suspended_callback;
7569 |         ma_proc pa_stream_set_moved_callback;
7570 |         ma_proc pa_stream_is_suspended;
7571 |         ma_proc pa_stream_flush;
7572 |         ma_proc pa_stream_drain;
7573 |         ma_proc pa_stream_is_corked;
7574 |         ma_proc pa_stream_cork;
7575 |         ma_proc pa_stream_trigger;
7576 |         ma_proc pa_stream_begin_write;
7577 |         ma_proc pa_stream_write;
7578 |         ma_proc pa_stream_peek;
7579 |         ma_proc pa_stream_drop;
7580 |         ma_proc pa_stream_writable_size;
```

```

7581 |         ma_proc pa_stream_readable_size;
7582 |
7583 |         /*pa_mainloop*/ ma_ptr pMainLoop;
7584 |         /*pa_context*/ ma_ptr pPulseContext;
7585 |         char* pApplicationName; /* Set when the context is initialized. Used by devices for
> their local pa_context objects. */
7586 |         char* pServerName;      /* Set when the context is initialized. Used by devices for
> their local pa_context objects. */
7587 |     } pulse;
7588 | #endif
7589 | #ifdef MA_SUPPORT_JACK
7590 |     struct
7591 |     {
7592 |         ma_handle jackS0;
7593 |         ma_proc jack_client_open;
7594 |         ma_proc jack_client_close;
7595 |         ma_proc jack_client_name_size;
7596 |         ma_proc jack_set_process_callback;
7597 |         ma_proc jack_set_buffer_size_callback;
7598 |         ma_proc jack_on_shutdown;
7599 |         ma_proc jack_get_sample_rate;
7600 |         ma_proc jack_get_buffer_size;
7601 |         ma_proc jack_get_ports;
7602 |         ma_proc jack_activate;
7603 |         ma_proc jack_deactivate;
7604 |         ma_proc jack_connect;
7605 |         ma_proc jack_port_register;
7606 |         ma_proc jack_port_name;
7607 |         ma_proc jack_port_get_buffer;
7608 |         ma_proc jack_free;
7609 |
7610 |         char* pClientName;
7611 |         ma_bool32 tryStartServer;
7612 |     } jack;
7613 | #endif
7614 | #ifdef MA_SUPPORT_COREAUDIO
7615 |     struct
7616 |     {
7617 |         ma_handle hCoreFoundation;
7618 |         ma_proc CFStringGetCString;
7619 |         ma_proc CFRelease;
7620 |
7621 |         ma_handle hCoreAudio;
7622 |         ma_proc AudioObjectGetPropertyData;
7623 |         ma_proc AudioObjectGetPropertyDataSize;
7624 |         ma_proc AudioObjectSetPropertyData;
7625 |         ma_proc AudioObjectAddPropertyListener;
7626 |         ma_proc AudioObjectRemovePropertyListener;
7627 |
7628 |         ma_handle hAudioUnit; /* Could possibly be set to AudioToolbox on later versions of
> macOS. */
7629 |         ma_proc AudioComponentFindNext;
7630 |         ma_proc AudioComponentInstanceDispose;
7631 |         ma_proc AudioComponentInstanceNew;
7632 |         ma_proc AudioOutputUnitStart;
7633 |         ma_proc AudioOutputUnitStop;
7634 |         ma_proc AudioUnitAddPropertyListener;
7635 |         ma_proc AudioUnitGetPropertyInfo;
7636 |         ma_proc AudioUnitGetProperty;
7637 |         ma_proc AudioUnitSetProperty;
7638 |         ma_proc AudioUnitInitialize;
7639 |         ma_proc AudioUnitRender;
7640 |
7641 |         /*AudioComponent*/ ma_ptr component;
7642 |         ma_bool32 noAudioSessionDeactivate; /* For tracking whether or not the iOS audio session
> should be explicitly deactivated. Set from the config in ma_context_init__coreaudio(). */
7643 |     } coreaudio;
7644 | #endif
7645 | #ifdef MA_SUPPORT_SNDIO
7646 |     struct
7647 |     {
7648 |         ma_handle sndioS0;
7649 |         ma_proc sio_open;
7650 |         ma_proc sio_close;
7651 |         ma_proc sio_setpar;
7652 |         ma_proc sio_getpar;
7653 |         ma_proc sio_getcap;
7654 |         ma_proc sio_start;
7655 |         ma_proc sio_stop;

```



```

7656 |         ma_proc sio_read;
7657 |         ma_proc sio_write;
7658 |         ma_proc sio_onmove;
7659 |         ma_proc sio_nfds;
7660 |         ma_proc sio_pollfd;
7661 |         ma_proc sio_revents;
7662 |         ma_proc sio_eof;
7663 |         ma_proc sio_setvol;
7664 |         ma_proc sio_onvol;
7665 |         ma_proc sio_initpar;
7666 |     } sndio;
7667 | #endif
7668 | #ifdef MA_SUPPORT_AUDIO4
7669 |     struct
7670 |     {
7671 |         int _unused;
7672 |     } audio4;
7673 | #endif
7674 | #ifdef MA_SUPPORT_OSS
7675 |     struct
7676 |     {
7677 |         int versionMajor;
7678 |         int versionMinor;
7679 |     } oss;
7680 | #endif
7681 | #ifdef MA_SUPPORT_AAUDIO
7682 |     struct
7683 |     {
7684 |         ma_handle hAAudio; /* libaaudio.so */
7685 |         ma_proc AAudio_createStreamBuilder;
7686 |         ma_proc AAudioStreamBuilder_delete;
7687 |         ma_proc AAudioStreamBuilder_setDeviceId;
7688 |         ma_proc AAudioStreamBuilder_setDirection;
7689 |         ma_proc AAudioStreamBuilder_setSharingMode;
7690 |         ma_proc AAudioStreamBuilder_setFormat;
7691 |         ma_proc AAudioStreamBuilder_setChannelCount;
7692 |         ma_proc AAudioStreamBuilder_setSampleRate;
7693 |         ma_proc AAudioStreamBuilder_setBufferCapacityInFrames;
7694 |         ma_proc AAudioStreamBuilder_setFramesPerDataCallback;
7695 |         ma_proc AAudioStreamBuilder_setDataCallback;
7696 |         ma_proc AAudioStreamBuilder_setErrorCallback;
7697 |         ma_proc AAudioStreamBuilder_setPerformanceMode;
7698 |         ma_proc AAudioStreamBuilder_setUsage;
7699 |         ma_proc AAudioStreamBuilder_setContentType;
7700 |         ma_proc AAudioStreamBuilder_setInputPreset;
7701 |         ma_proc AAudioStreamBuilder_setAllowedCapturePolicy;
7702 |         ma_proc AAudioStreamBuilder_openStream;
7703 |         ma_proc AAudioStream_close;
7704 |         ma_proc AAudioStream_getState;
7705 |         ma_proc AAudioStream_waitForStateChange;
7706 |         ma_proc AAudioStream_getFormat;
7707 |         ma_proc AAudioStream_getChannelCount;
7708 |         ma_proc AAudioStream_getSampleRate;
7709 |         ma_proc AAudioStream_getBufferCapacityInFrames;
7710 |         ma_proc AAudioStream_getFramesPerDataCallback;
7711 |         ma_proc AAudioStream_getFramesPerBurst;
7712 |         ma_proc AAudioStream_requestStart;
7713 |         ma_proc AAudioStream_requestStop;
7714 |         ma_device_job_thread jobThread; /* For processing operations outside of the error
> callback, specifically device disconnections and rerouting. */
7715 |     } aaudio;
7716 | #endif
7717 | #ifdef MA_SUPPORT_OPENSLES
7718 |     struct
7719 |     {
7720 |         ma_handle libOpenSLES;
7721 |         ma_handle SL_IID_ENGINE;
7722 |         ma_handle SL_IID_AUDIOIODEVICECAPABILITIES;
7723 |         ma_handle SL_IID_ANDROIDSIMPLEBUFFERQUEUE;
7724 |         ma_handle SL_IID_RECORD;
7725 |         ma_handle SL_IID_PLAY;
7726 |         ma_handle SL_IID_OUTPUTMIX;
7727 |         ma_handle SL_IID_ANDROIDCONFIGURATION;
7728 |         ma_proc slCreateEngine;
7729 |     } opensl;
7730 | #endif
7731 | #ifdef MA_SUPPORT_WEBAUDIO
7732 |     struct
7733 |     {

```

```

7734 |         int _unused;
7735 |     } webaudio;
7736 | #endif
7737 | #ifdef MA_SUPPORT_NULL
7738 |     struct
7739 |     {
7740 |         int _unused;
7741 |     } null_backend;
7742 | #endif
7743 | };
7744 |
7745 |     union
7746 |     {
7747 | #if defined(MA_WIN32)
7748 |         struct
7749 |         {
7750 |             /*HMODULE*/ ma_handle hOle32DLL;
7751 |             ma_proc CoInitialize;
7752 |             ma_proc CoInitializeEx;
7753 |             ma_proc CoUninitialize;
7754 |             ma_proc CoCreateInstance;
7755 |             ma_proc CoTaskMemFree;
7756 |             ma_proc PropVariantClear;
7757 |             ma_proc StringFromGUID2;
7758 |
7759 |             /*HMODULE*/ ma_handle hUser32DLL;
7760 |             ma_proc GetForegroundWindow;
7761 |             ma_proc GetDesktopWindow;
7762 |
7763 |             /*HMODULE*/ ma_handle hAdvapi32DLL;
7764 |             ma_proc RegOpenKeyExA;
7765 |             ma_proc RegCloseKey;
7766 |             ma_proc RegQueryValueExA;
7767 |
7768 |             /*HRESULT*/ long CoInitializeResult;
7769 |         } win32;
7770 | #endif
7771 | #ifdef MA_POSIX
7772 |         struct
7773 |         {
7774 |             int _unused;
7775 |         } posix;
7776 | #endif
7777 |         int _unused;
7778 |     };
7779 | };
7780 |
7781 | struct ma_device
7782 | {
7783 |     ma_context* pContext;
7784 |     ma_device_type type;
7785 |     ma_uint32 sampleRate;
7786 |     ma_atomic_device_state state; /* The state of the device is variable and can
> change at any time on any thread. Must be used atomically. */
7787 |     ma_device_data_proc onData; /* Set once at initialization time and should not be
> changed after. */
7788 |     ma_device_notification_proc onNotification; /* Set once at initialization time and should not be
> changed after. */
7789 |     ma_stop_proc onStop; /* DEPRECATED. Use the notification callback
> instead. Set once at initialization time and should not be changed after. */
7790 |     void* pUserData; /* Application defined data. */
7791 |     ma_mutex startStopLock;
7792 |     ma_event wakeupEvent;
7793 |     ma_event startEvent;
7794 |     ma_event stopEvent;
7795 |     ma_thread thread;
7796 |     ma_result workResult; /* This is set by the worker thread after it's
> finished doing a job. */
7797 |     ma_bool8 isOwnerOfContext; /* When set to true, uninitializing the device will
> also uninitialize the context. Set to true when NULL is passed into ma_device_init(). */
7798 |     ma_bool8 noPreSilencedOutputBuffer;
7799 |     ma_bool8 noClip;
7800 |     ma_bool8 noDisableDenormals;
7801 |     ma_bool8 noFixedSizedCallback;
7802 |     ma_atomic_float masterVolumeFactor; /* Linear 0..1. Can be read and written
> simultaneously by different threads. Must be used atomically. */
7803 |     ma_duplex_rb duplexRB; /* Intermediary buffer for duplex device on
> asynchronous backends. */
7804 |     struct

```

```

7805 |     {
7806 |         ma_resample_algorithm algorithm;
7807 |         ma_resampling_backend_vtable* pBackendVTable;
7808 |         void* pBackendUserData;
7809 |         struct
7810 |         {
7811 |             ma_uint32 lpfOrder;
7812 |         } linear;
7813 |     } resampling;
7814 |     struct
7815 |     {
7816 |         ma_device_id* pID;                /* Set to NULL if using default ID, otherwise set to the
> address of "id". */
7817 |         ma_device_id id;                  /* If using an explicit device, will be set to a copy of
> the ID used for initialization. Otherwise cleared to 0. */
7818 |         char name[MA_MAX_DEVICE_NAME_LENGTH + 1]; /* Maybe temporary. Likely to
> be replaced with a query API. */
7819 |         ma_share_mode shareMode;          /* Set to whatever was passed in when the device was
> initialized. */
7820 |         ma_format format;
7821 |         ma_uint32 channels;
7822 |         ma_channel channelMap[MA_MAX_CHANNELS];
7823 |         ma_format internalFormat;
7824 |         ma_uint32 internalChannels;
7825 |         ma_uint32 internalSampleRate;
7826 |         ma_channel internalChannelMap[MA_MAX_CHANNELS];
7827 |         ma_uint32 internalPeriodSizeInFrames;
7828 |         ma_uint32 internalPeriods;
7829 |         ma_channel_mix_mode channelMixMode;
7830 |         ma_bool32 calculateLFEFromSpatialChannels;
7831 |         ma_data_converter converter;
7832 |         void* pIntermediaryBuffer;        /* For implementing fixed sized buffer callbacks. Will
> be null if using variable sized callbacks. */
7833 |         ma_uint32 intermediaryBufferCap;
7834 |         ma_uint32 intermediaryBufferLen;  /* How many valid frames are sitting in the intermediary
> buffer. */
7835 |         void* pInputCache;                /* In external format. Can be null. */
7836 |         ma_uint64 inputCacheCap;
7837 |         ma_uint64 inputCacheConsumed;
7838 |         ma_uint64 inputCacheRemaining;
7839 |     } playback;
7840 |     struct
7841 |     {
7842 |         ma_device_id* pID;                /* Set to NULL if using default ID, otherwise set to the
> address of "id". */
7843 |         ma_device_id id;                  /* If using an explicit device, will be set to a copy of
> the ID used for initialization. Otherwise cleared to 0. */
7844 |         char name[MA_MAX_DEVICE_NAME_LENGTH + 1]; /* Maybe temporary. Likely to
> be replaced with a query API. */
7845 |         ma_share_mode shareMode;          /* Set to whatever was passed in when the device was
> initialized. */
7846 |         ma_format format;
7847 |         ma_uint32 channels;
7848 |         ma_channel channelMap[MA_MAX_CHANNELS];
7849 |         ma_format internalFormat;
7850 |         ma_uint32 internalChannels;
7851 |         ma_uint32 internalSampleRate;
7852 |         ma_channel internalChannelMap[MA_MAX_CHANNELS];
7853 |         ma_uint32 internalPeriodSizeInFrames;
7854 |         ma_uint32 internalPeriods;
7855 |         ma_channel_mix_mode channelMixMode;
7856 |         ma_bool32 calculateLFEFromSpatialChannels;
7857 |         ma_data_converter converter;
7858 |         void* pIntermediaryBuffer;        /* For implementing fixed sized buffer callbacks. Will
> be null if using variable sized callbacks. */
7859 |         ma_uint32 intermediaryBufferCap;
7860 |         ma_uint32 intermediaryBufferLen;  /* How many valid frames are sitting in the intermediary
> buffer. */
7861 |     } capture;
7862 |
7863 |     union
7864 |     {
7865 | #ifdef MA_SUPPORT_WASAPI
7866 |         struct
7867 |         {
7868 |             /*IAudioClient**/ ma_ptr pAudioClientPlayback;
7869 |             /*IAudioClient**/ ma_ptr pAudioClientCapture;
7870 |             /*IAudioRenderClient**/ ma_ptr pRenderClient;
7871 |             /*IAudioCaptureClient**/ ma_ptr pCaptureClient;

```

```

7872 |         /*IMMDeviceEnumerator*/ ma_ptr pDeviceEnumerator;        /* Used for
> IMMNotificationClient notifications. Required for detecting default device changes. */
7873 |         ma_IMMNotificationClient notificationClient;
7874 |         /*HANDLE*/ ma_handle hEventPlayback;                      /* Auto reset. Initialized to
> signaled. */
7875 |         /*HANDLE*/ ma_handle hEventCapture;                      /* Auto reset. Initialized to
> unsignaled. */
7876 |         ma_uint32 actualBufferSizeInFramesPlayback;              /* Value from GetBufferSize().
> internalPeriodSizeInFrames is not set to the _actual_ buffer size when low-latency shared mode is
> being used due to the way the IAudioClient3 API works. */
7877 |         ma_uint32 actualBufferSizeInFramesCapture;
7878 |         ma_uint32 originalPeriodSizeInFrames;
7879 |         ma_uint32 originalPeriodSizeInMilliseconds;
7880 |         ma_uint32 originalPeriods;
7881 |         ma_performance_profile originalPerformanceProfile;
7882 |         ma_uint32 periodSizeInFramesPlayback;
7883 |         ma_uint32 periodSizeInFramesCapture;
7884 |         void* pMappedBufferCapture;
7885 |         ma_uint32 mappedBufferCaptureCap;
7886 |         ma_uint32 mappedBufferCaptureLen;
7887 |         void* pMappedBufferPlayback;
7888 |         ma_uint32 mappedBufferPlaybackCap;
7889 |         ma_uint32 mappedBufferPlaybackLen;
7890 |         ma_atomic_bool32 isStartedCapture;                      /* Can be read and written
> simultaneously across different threads. Must be used atomically, and must be 32-bit. */
7891 |         ma_atomic_bool32 isStartedPlayback;                      /* Can be read and written
> simultaneously across different threads. Must be used atomically, and must be 32-bit. */
7892 |         ma_uint32 loopbackProcessID;
7893 |         ma_bool8 loopbackProcessExclude;
7894 |         ma_bool8 noAutoConvertSRC;                               /* When set to true, disables
> the use of AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM. */
7895 |         ma_bool8 noDefaultQualitySRC;                           /* When set to true, disables
> the use of AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY. */
7896 |         ma_bool8 noHardwareOffloading;
7897 |         ma_bool8 allowCaptureAutoStreamRouting;
7898 |         ma_bool8 allowPlaybackAutoStreamRouting;
7899 |         ma_bool8 isDetachedPlayback;
7900 |         ma_bool8 isDetachedCapture;
7901 |         ma_wasapi_usage;
7902 |         void* hAvrtHandle;
7903 |         ma_mutex rerouteLock;
7904 |     } wasapi;
7905 | #endif
7906 | #ifdef MA_SUPPORT_DSOUND
7907 |     struct
7908 |     {
7909 |         /*LPDIRECTSOUND*/ ma_ptr pPlayback;
7910 |         /*LPDIRECTSOUNDBUFFER*/ ma_ptr pPlaybackPrimaryBuffer;
7911 |         /*LPDIRECTSOUNDBUFFER*/ ma_ptr pPlaybackBuffer;
7912 |         /*LPDIRECTSOUNDCAPTURE*/ ma_ptr pCapture;
7913 |         /*LPDIRECTSOUNDCAPTUREBUFFER*/ ma_ptr pCaptureBuffer;
7914 |     } dsound;
7915 | #endif
7916 | #ifdef MA_SUPPORT_WINMM
7917 |     struct
7918 |     {
7919 |         /*HWAVEOUT*/ ma_handle hDevicePlayback;
7920 |         /*HWAVEIN*/ ma_handle hDeviceCapture;
7921 |         /*HANDLE*/ ma_handle hEventPlayback;
7922 |         /*HANDLE*/ ma_handle hEventCapture;
7923 |         ma_uint32 fragmentSizeInFrames;
7924 |         ma_uint32 iNextHeaderPlayback;                          /* [0,periods). Used as an index into
> pWAVEHDRPlayback. */
7925 |         ma_uint32 iNextHeaderCapture;                            /* [0,periods). Used as an index into
> pWAVEHRCapture. */
7926 |         ma_uint32 headerFramesConsumedPlayback;                 /* The number of PCM frames consumed in the
> buffer in pWAVEHEADER[iNextHeader]. */
7927 |         ma_uint32 headerFramesConsumedCapture;                 /* ^^^ */
7928 |         /*WAVEHDR*/ ma_uint8* pWAVEHDRPlayback;               /* One instantiation for each period. */
7929 |         /*WAVEHDR*/ ma_uint8* pWAVEHRCapture;                 /* One instantiation for each period. */
7930 |         ma_uint8* pIntermediaryBufferPlayback;
7931 |         ma_uint8* pIntermediaryBufferCapture;
7932 |         ma_uint8* _pHeapData;                                   /* Used internally and is used for the heap
> allocated data for the intermediary buffer and the WAVEHDR structures. */
7933 |     } winmm;
7934 | #endif
7935 | #ifdef MA_SUPPORT_ALSA
7936 |     struct
7937 |     {

```

```

7938 |         /*snd_pcm_t**/ ma_ptr pPCMPlayback;
7939 |         /*snd_pcm_t**/ ma_ptr pPCMCapture;
7940 |         /*struct pollfd**/ void* pPollDescriptorsPlayback;
7941 |         /*struct pollfd**/ void* pPollDescriptorsCapture;
7942 |         int pollDescriptorCountPlayback;
7943 |         int pollDescriptorCountCapture;
7944 |         int wakeupfdPlayback; /* eventfd for waking up from poll() when the playback device is
> stopped. */
7945 |         int wakeupfdCapture; /* eventfd for waking up from poll() when the capture device is
> stopped. */
7946 |         ma_bool8 isUsingMMapPlayback;
7947 |         ma_bool8 isUsingMMapCapture;
7948 |     } alsa;
7949 | #endif
7950 | #ifdef MA_SUPPORT_PULSEAUDIO
7951 |     struct
7952 |     {
7953 |         /*pa_mainloop**/ ma_ptr pMainLoop;
7954 |         /*pa_context**/ ma_ptr pPulseContext;
7955 |         /*pa_stream**/ ma_ptr pStreamPlayback;
7956 |         /*pa_stream**/ ma_ptr pStreamCapture;
7957 |     } pulse;
7958 | #endif
7959 | #ifdef MA_SUPPORT_JACK
7960 |     struct
7961 |     {
7962 |         /*jack_client_t**/ ma_ptr pClient;
7963 |         /*jack_port_t**/ ma_ptr* ppPortsPlayback;
7964 |         /*jack_port_t**/ ma_ptr* ppPortsCapture;
7965 |         float* pIntermediaryBufferPlayback; /* Typed as a float because JACK is always floating
> point. */
7966 |         float* pIntermediaryBufferCapture;
7967 |     } jack;
7968 | #endif
7969 | #ifdef MA_SUPPORT_COREAUDIO
7970 |     struct
7971 |     {
7972 |         ma_uint32 deviceObjectIDPlayback;
7973 |         ma_uint32 deviceObjectIDCapture;
7974 |         /*AudioUnit*/ ma_ptr audioUnitPlayback;
7975 |         /*AudioUnit*/ ma_ptr audioUnitCapture;
7976 |         /*AudioBufferList*/ ma_ptr pAudioBufferList; /* Only used for input devices. */
7977 |         ma_uint32 audioBufferCapInFrames; /* Only used for input devices. The
> capacity in frames of each buffer in pAudioBufferList. */
7978 |         ma_event stopEvent;
7979 |         ma_uint32 originalPeriodSizeInFrames;
7980 |         ma_uint32 originalPeriodSizeInMilliseconds;
7981 |         ma_uint32 originalPeriods;
7982 |         ma_performance_profile originalPerformanceProfile;
7983 |         ma_bool32 isDefaultPlaybackDevice;
7984 |         ma_bool32 isDefaultCaptureDevice;
7985 |         ma_bool32 isSwitchingPlaybackDevice; /* <-- Set to true when the default device has
> changed and miniaudio is in the process of switching. */
7986 |         ma_bool32 isSwitchingCaptureDevice; /* <-- Set to true when the default device has
> changed and miniaudio is in the process of switching. */
7987 |         void* pNotificationHandler; /* Only used on mobile platforms. Obj-C object
> for handling route changes. */
7988 |     } coreaudio;
7989 | #endif
7990 | #ifdef MA_SUPPORT_SNDIO
7991 |     struct
7992 |     {
7993 |         ma_ptr handlePlayback;
7994 |         ma_ptr handleCapture;
7995 |         ma_bool32 isStartedPlayback;
7996 |         ma_bool32 isStartedCapture;
7997 |     } sndio;
7998 | #endif
7999 | #ifdef MA_SUPPORT_AUDIIO4
8000 |     struct
8001 |     {
8002 |         int fdPlayback;
8003 |         int fdCapture;
8004 |     } audio4;
8005 | #endif
8006 | #ifdef MA_SUPPORT_OSS
8007 |     struct
8008 |     {
8009 |         int fdPlayback;

```

```

8010 |         int fdCapture;
8011 |     } oss;
8012 | #endif
8013 | #ifdef MA_SUPPORT_AAUDIO
8014 |     struct
8015 |     {
8016 |         /*AAudioStream**/ ma_ptr pStreamPlayback;
8017 |         /*AAudioStream**/ ma_ptr pStreamCapture;
8018 |         ma_mutex rerouteLock;
8019 |         ma_atomic_bool32 isTearingDown;
8020 |         ma_aaudio_usage usage;
8021 |         ma_aaudio_content_type contentType;
8022 |         ma_aaudio_input_preset inputPreset;
8023 |         ma_aaudio_allowed_capture_policy allowedCapturePolicy;
8024 |         ma_bool32 noAutoStartAfterReroute;
8025 |     } aaudio;
8026 | #endif
8027 | #ifdef MA_SUPPORT_OPENS�
8028 |     struct
8029 |     {
8030 |         /*SLObjectItf*/ ma_ptr pOutputMixObj;
8031 |         /*SLOutputMixItf*/ ma_ptr pOutputMix;
8032 |         /*SLObjectItf*/ ma_ptr pAudioPlayerObj;
8033 |         /*SLPlayItf*/ ma_ptr pAudioPlayer;
8034 |         /*SLObjectItf*/ ma_ptr pAudioRecorderObj;
8035 |         /*SLRecordItf*/ ma_ptr pAudioRecorder;
8036 |         /*SLAndroidSimpleBufferQueueItf*/ ma_ptr pBufferQueuePlayback;
8037 |         /*SLAndroidSimpleBufferQueueItf*/ ma_ptr pBufferQueueCapture;
8038 |         ma_bool32 isDrainingCapture;
8039 |         ma_bool32 isDrainingPlayback;
8040 |         ma_uint32 currentBufferIndexPlayback;
8041 |         ma_uint32 currentBufferIndexCapture;
8042 |         ma_uint8* pBufferPlayback; /* This is malloc()'d and is used for storing audio
> data. Typed as ma_uint8 for easy offsetting. */
8043 |         ma_uint8* pBufferCapture;
8044 |     } openssl;
8045 | #endif
8046 | #ifdef MA_SUPPORT_WEBAUDIO
8047 |     struct
8048 |     {
8049 |         /* AudioWorklets path. */
8050 |         /* EMSCRIPTEN_WEBAUDIO_T */ int audioContext;
8051 |         /* EMSCRIPTEN_WEBAUDIO_T */ int audioWorklet;
8052 |         float* pIntermediaryBuffer;
8053 |         void* pStackBuffer;
8054 |         ma_result initResult; /* Set to MA_BUSY while initialization is in progress. */
8055 |         int deviceIndex; /* We store the device in a list on the JavaScript side. This is
> used to map our C object to the JS object. */
8056 |     } webaudio;
8057 | #endif
8058 | #ifdef MA_SUPPORT_NULL
8059 |     struct
8060 |     {
8061 |         ma_thread deviceThread;
8062 |         ma_event operationEvent;
8063 |         ma_event operationCompletionEvent;
8064 |         ma_semaphore operationSemaphore;
8065 |         ma_uint32 operation;
8066 |         ma_result operationResult;
8067 |         ma_timer timer;
8068 |         double priorRunTime;
8069 |         ma_uint32 currentPeriodFramesRemainingPlayback;
8070 |         ma_uint32 currentPeriodFramesRemainingCapture;
8071 |         ma_uint64 lastProcessedFramePlayback;
8072 |         ma_uint64 lastProcessedFrameCapture;
8073 |         ma_atomic_bool32 isStarted; /* Read and written by multiple threads. Must be used
> atomically, and must be 32-bit for compiler compatibility. */
8074 |     } null_device;
8075 | #endif
8076 | };
8077 | };
8078 | #if defined(_MSC_VER) && !defined(__clang__)
8079 |     #pragma warning(pop)
8080 | #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__
> >= 8)))
8081 |     #pragma GCC diagnostic pop /* For ISO C99 doesn't support unnamed structs/unions [-Wpedantic]
> */
8082 | #endif
8083 |

```

```

8084 | /*
8085 | Initializes a `ma_context_config` object.
8086 |
8087 |
8088 | Return Value
8089 | -----
8090 | A `ma_context_config` initialized to defaults.
8091 |
8092 |
8093 | Remarks
8094 | -----
8095 | You must always use this to initialize the default state of the `ma_context_config` object. Not
> using this will result in your program breaking when miniaudio
8096 | is updated and new members are added to `ma_context_config`. It also sets logical defaults.
8097 |
8098 | You can override members of the returned object by changing it's members directly.
8099 |
8100 |
8101 | See Also
8102 | -----
8103 | ma_context_init()
8104 | */
8105 | MA_API ma_context_config ma_context_config_init(void);
8106 |
8107 | /*
8108 | Initializes a context.
8109 |
8110 | The context is used for selecting and initializing an appropriate backend and to represent the
> backend at a more global level than that of an individual
8111 | device. There is one context to many devices, and a device is created from a context. A context is
> required to enumerate devices.
8112 |
8113 |
8114 | Parameters
8115 | -----
8116 | backends (in, optional)
8117 |     A list of backends to try initializing, in priority order. Can be NULL, in which case it uses
> default priority order.
8118 |
8119 | backendCount (in, optional)
8120 |     The number of items in `backend`. Ignored if `backend` is NULL.
8121 |
8122 | pConfig (in, optional)
8123 |     The context configuration.
8124 |
8125 | pContext (in)
8126 |     A pointer to the context object being initialized.
8127 |
8128 |
8129 | Return Value
8130 | -----
8131 | MA_SUCCESS if successful; any other error code otherwise.
8132 |
8133 |
8134 | Thread Safety
8135 | -----
8136 | Unsafe. Do not call this function across multiple threads as some backends read and write to global
> state.
8137 |
8138 |
8139 | Remarks
8140 | -----
8141 | When `backends` is NULL, the default priority order will be used. Below is a list of backends in
> priority order:
8142 |
8143 | |-----|-----|-----|
8144 | | Name | Enum Name | Supported Operating Systems |
8145 | |-----|-----|-----|
8146 | | WASAPI | ma_backend_wasapi | Windows Vista+ |
8147 | | DirectSound | ma_backend_dsound | Windows XP+ |
8148 | | WinMM | ma_backend_winmm | Windows XP+ (may work on older versions, but untested) |
8149 | | Core Audio | ma_backend_coreaudio | macOS, iOS |
8150 | | ALSA | ma_backend_alsa | Linux |
8151 | | PulseAudio | ma_backend_pulseaudio | Cross Platform (disabled on Windows, BSD and Android) |
8152 | | JACK | ma_backend_jack | Cross Platform (disabled on BSD and Android) |
8153 | | sndio | ma_backend_sndio | OpenBSD |
8154 | | audio(4) | ma_backend_audio4 | NetBSD, OpenBSD |
8155 | | OSS | ma_backend_oss | FreeBSD |
8156 | | AAudio | ma_backend_aaudio | Android 8+ |

```



```

8157 | | OpenSLES | ma_backend_opensl | Android (API level 16+) |
8158 | | Web Audio | ma_backend_webaudio | Web (via Emscripten) |
8159 | | Null | ma_backend_null | Cross Platform (not used on Web) |
8160 | |-----|-----|-----|
8161 |
8162 | The context can be configured via the `pConfig` argument. The config object is initialized with
> `ma_context_config_init()`. Individual configuration settings
8163 | can then be set directly on the structure. Below are the members of the `ma_context_config` object.
8164 |
8165 | pLog
8166 |     A pointer to the `ma_log` to post log messages to. Can be NULL if the application does not
8167 |     require logging. See the `ma_log` API for details on how to use the logging system.
8168 |
8169 | threadPriority
8170 |     The desired priority to use for the audio thread. Allowable values include the following:
8171 |
8172 |     |-----|
8173 |     | Thread Priority |
8174 |     |-----|
8175 |     | ma_thread_priority_idle |
8176 |     | ma_thread_priority_lowest |
8177 |     | ma_thread_priority_low |
8178 |     | ma_thread_priority_normal |
8179 |     | ma_thread_priority_high |
8180 |     | ma_thread_priority_highest (default) |
8181 |     | ma_thread_priority_realtime |
8182 |     | ma_thread_priority_default |
8183 |     |-----|
8184 |
8185 | threadStackSize
8186 |     The desired size of the stack for the audio thread. Defaults to the operating system's
> default.
8187 |
8188 | pUserData
8189 |     A pointer to application-defined data. This can be accessed from the context object directly
> such as `context.pUserData`.
8190 |
8191 | allocationCallbacks
8192 |     Structure containing custom allocation callbacks. Leaving this at defaults will cause it to
> use MA_MALLOC, MA_REALLOC and MA_FREE. These allocation
8193 | callbacks will be used for anything tied to the context, including devices.
8194 |
8195 | alsa.useVerboseDeviceEnumeration
8196 |     ALSA will typically enumerate many different devices which can be intrusive and not
> user-friendly. To combat this, miniaudio will enumerate only unique
8197 | card/device pairs by default. The problem with this is that you lose a bit of flexibility
> and control. Setting alsa.useVerboseDeviceEnumeration makes
8198 | it so the ALSA backend includes all devices. Defaults to false.
8199 |
8200 | pulse.pApplicationName
8201 |     PulseAudio only. The application name to use when initializing the PulseAudio context with
> `pa_context_new()`.
8202 |
8203 | pulse.pServerName
8204 |     PulseAudio only. The name of the server to connect to with `pa_context_connect()`.
8205 |
8206 | pulse.tryAutoSpawn
8207 |     PulseAudio only. Whether or not to try automatically starting the PulseAudio daemon.
> Defaults to false. If you set this to true, keep in mind that
8208 | miniaudio uses a trial and error method to find the most appropriate backend, and this will
> result in the PulseAudio daemon starting which may be
8209 | intrusive for the end user.
8210 |
8211 | coreaudio.sessionCategory
8212 |     iOS only. The session category to use for the shared AudioSession instance. Below is a list
> of allowable values and their Core Audio equivalents.
8213 |
8214 |     |-----|-----|
8215 |     | miniaudio Token | Core Audio Token |
8216 |     |-----|-----|
8217 |     | ma_ios_session_category_ambient | AVAudioSessionCategoryAmbient |
8218 |     | ma_ios_session_category_solo_ambient | AVAudioSessionCategorySoloAmbient |
8219 |     | ma_ios_session_category_playback | AVAudioSessionCategoryPlayback |
8220 |     | ma_ios_session_category_record | AVAudioSessionCategoryRecord |
8221 |     | ma_ios_session_category_play_and_record | AVAudioSessionCategoryPlayAndRecord |
8222 |     | ma_ios_session_category_multi_route | AVAudioSessionCategoryMultiRoute |
8223 |     | ma_ios_session_category_none | AVAudioSessionCategoryAmbient |
8224 |     | ma_ios_session_category_default | AVAudioSessionCategoryAmbient |
8225 |     |-----|-----|

```

```

8226 |
8227 |     coreaudio.sessionCategoryOptions
8228 |         iOS only. Session category options to use with the shared AudioSession instance. Below is a
> list of allowable values and their Core Audio equivalents.
8229 |
8230 | |-----|-----|-----|
8231 | | miniaudio Token | | Core Audio
> Token |
8232 | |-----|-----|-----|
8233 | | ma_ios_session_category_option_mix_with_others | |
> AVAudioSessionCategoryOptionMixWithOthers |
8234 | | ma_ios_session_category_option_duck_others | |
> AVAudioSessionCategoryOptionDuckOthers |
8235 | | ma_ios_session_category_option_allow_bluetooth | |
> AVAudioSessionCategoryOptionAllowBluetooth |
8236 | | ma_ios_session_category_option_default_to_speaker | |
> AVAudioSessionCategoryOptionDefaultToSpeaker |
8237 | | ma_ios_session_category_option_interrupt_spoken_audio_and_mix_with_others | |
> AVAudioSessionCategoryOptionInterruptSpokenAudioAndMixWithOthers |
8238 | | ma_ios_session_category_option_allow_bluetooth_a2dp | |
> AVAudioSessionCategoryOptionAllowBluetoothA2DP |
8239 | | ma_ios_session_category_option_allow_air_play | |
> AVAudioSessionCategoryOptionAllowAirPlay |
8240 | |-----|-----|-----|
8241 |
8242 |     coreaudio.noAudioSessionActivate
8243 |         iOS only. When set to true, does not perform an explicit [[AVAudioSession sharedInstance]
> setActive:true] on initialization.
8244 |
8245 |     coreaudio.noAudioSessionDeactivate
8246 |         iOS only. When set to true, does not perform an explicit [[AVAudioSession sharedInstance]
> setActive:false] on uninitialization.
8247 |
8248 |     jack.pClientName
8249 |         The name of the client to pass to `jack_client_open()`.
8250 |
8251 |     jack.tryStartServer
8252 |         Whether or not to try auto-starting the JACK server. Defaults to false.
8253 |
8254 |
8255 | It is recommended that only a single context is active at any given time because it's a bulky data
> structure which performs run-time linking for the
8256 | relevant backends every time it's initialized.
8257 |
8258 | The location of the context cannot change throughout it's lifetime. Consider allocating the
> `ma_context` object with `malloc()` if this is an issue. The
8259 | reason for this is that a pointer to the context is stored in the `ma_device` structure.
8260 |
8261 |
8262 | Example 1 - Default Initialization
8263 | -----
8264 | The example below shows how to initialize the context using the default configuration.
8265 |
8266 | ```c
8267 | ma_context context;
8268 | ma_result result = ma_context_init(NULL, 0, NULL, &context);
8269 | if (result != MA_SUCCESS) {
8270 |     // Error.
8271 | }
8272 | ```
8273 |
8274 |
8275 | Example 2 - Custom Configuration
8276 | -----
8277 | The example below shows how to initialize the context using custom backend priorities and a custom
> configuration. In this hypothetical example, the program
8278 | wants to prioritize ALSA over PulseAudio on Linux. They also want to avoid using the WinMM backend
> on Windows because it's latency is too high. They also
8279 | want an error to be returned if no valid backend is available which they achieve by excluding the
> Null backend.
8280 |
8281 | For the configuration, the program wants to capture any log messages so they can, for example, route
> it to a log file and user interface.
8282 |
8283 | ```c
8284 | ma_backend backends[] = {

```

```

8285 |     ma_backend_alsa,
8286 |     ma_backend_pulseaudio,
8287 |     ma_backend_wasapi,
8288 |     ma_backend_dsound
8289 | };
8290 |
8291 | ma_log log;
8292 | ma_log_init(&log);
8293 | ma_log_register_callback(&log, ma_log_callback_init(my_log_callback, pMyLogUserData));
8294 |
8295 | ma_context_config config = ma_context_config_init();
8296 | config.pLog = &log; // Specify a custom log object in the config so any logs that are posted from
> ma_context_init() are captured.
8297 |
8298 | ma_context context;
8299 | ma_result result = ma_context_init(backends, sizeof(backends)/sizeof(backends[0]), &config,
> &context);
8300 | if (result != MA_SUCCESS) {
8301 |     // Error.
8302 |     if (result == MA_NO_BACKEND) {
8303 |         // Couldn't find an appropriate backend.
8304 |     }
8305 | }
8306 |
8307 | // You could also attach a log callback post-initialization:
8308 | ma_log_register_callback(ma_context_get_log(&context), ma_log_callback_init(my_log_callback,
> pMyLogUserData));
8309 | ```
8310 |
8311 |
8312 | See Also
8313 | -----
8314 | ma_context_config_init()
8315 | ma_context_uninit()
8316 | */
8317 | MA_API ma_result ma_context_init(const ma_backend backends[], ma_uint32 backendCount, const
> ma_context_config* pConfig, ma_context* pContext);
8318 |
8319 | /*
8320 | Uninitializes a context.
8321 |
8322 |
8323 | Return Value
8324 | -----
8325 | MA_SUCCESS if successful; any other error code otherwise.
8326 |
8327 |
8328 | Thread Safety
8329 | -----
8330 | Unsafe. Do not call this function across multiple threads as some backends read and write to global
> state.
8331 |
8332 |
8333 | Remarks
8334 | -----
8335 | Results are undefined if you call this while any device created by this context is still active.
8336 |
8337 |
8338 | See Also
8339 | -----
8340 | ma_context_init()
8341 | */
8342 | MA_API ma_result ma_context_uninit(ma_context* pContext);
8343 |
8344 | /*
8345 | Retrieves the size of the ma_context object.
8346 |
8347 | This is mainly for the purpose of bindings to know how much memory to allocate.
8348 | */
8349 | MA_API size_t ma_context_sizeof(void);
8350 |
8351 | /*
8352 | Retrieves a pointer to the log object associated with this context.
8353 |
8354 |
8355 | Remarks
8356 | -----
8357 | Pass the returned pointer to `ma_log_post()`, `ma_log_postv()` or `ma_log_postf()` to post a log
8358 | message.

```

```

8359 |
8360 | You can attach your own logging callback to the log with `ma_log_register_callback()`
8361 |
8362 |
8363 | Return Value
8364 | -----
8365 | A pointer to the `ma_log` object that the context uses to post log messages. If some error occurs,
8366 | NULL will be returned.
8367 | */
8368 | MA_API ma_log* ma_context_get_log(ma_context* pContext);
8369 |
8370 | /*
8371 | Enumerates over every device (both playback and capture).
8372 |
8373 | This is a lower-level enumeration function to the easier to use `ma_context_get_devices()`. Use
8374 | > `ma_context_enumerate_devices()` if you would rather not incur
8375 | an internal heap allocation, or it simply suits your code better.
8376 | Note that this only retrieves the ID and name/description of the device. The reason for only
8377 | > retrieving basic information is that it would otherwise require
8378 | opening the backend device in order to probe it for more detailed information which can be
8379 | > inefficient. Consider using `ma_context_get_device_info()` for this,
8380 | but don't call it from within the enumeration callback.
8381 |
8382 | Returning false from the callback will stop enumeration. Returning true will continue enumeration.
8383 |
8384 | Parameters
8385 | -----
8386 | pContext (in)
8387 |     A pointer to the context performing the enumeration.
8388 | callback (in)
8389 |     The callback to fire for each enumerated device.
8390 |
8391 | pUserData (in)
8392 |     A pointer to application-defined data passed to the callback.
8393 |
8394 | Return Value
8395 | -----
8396 | MA_SUCCESS if successful; any other error code otherwise.
8397 |
8398 | Thread Safety
8399 | -----
8400 | Safe. This is guarded using a simple mutex lock.
8401 |
8402 | Remarks
8403 | -----
8404 | Do _not_ assume the first enumerated device of a given type is the default device.
8405 | Some backends and platforms may only support default playback and capture devices.
8406 |
8407 | In general, you should not do anything complicated from within the callback. In particular, do not
8408 | > try initializing a device from within the callback. Also,
8409 | do not try to call `ma_context_get_device_info()` from within the callback.
8410 | Consider using `ma_context_get_devices()` for a simpler and safer API, albeit at the expense of an
8411 | > internal heap allocation.
8412 |
8413 | Example 1 - Simple Enumeration
8414 | -----
8415 | ma_bool32 ma_device_enum_callback(ma_context* pContext, ma_device_type deviceType, const
8416 | > ma_device_info* pInfo, void* pUserData)
8417 | {
8418 |     printf("Device Name: %s\n", pInfo->name);
8419 |     return MA_TRUE;
8420 | }
8421 |
8422 | ma_result result = ma_context_enumerate_devices(&context, my_device_enum_callback, pMyUserData);
8423 | if (result != MA_SUCCESS) {
8424 |     // Error.
8425 | }
8426 |
8427 | See Also

```

```

8432 | -----
8433 | ma_context_get_devices()
8434 | */
8435 | MA_API ma_result ma_context_enumerate_devices(ma_context* pContext, ma_enum_devices_callback_proc
> callback, void* pUserData);
8436 |
8437 | /*
8438 | Retrieves basic information about every active playback and/or capture device.
8439 |
8440 | This function will allocate memory internally for the device lists and return a pointer to them
> through the `ppPlaybackDeviceInfos` and `ppCaptureDeviceInfos`
8441 | parameters. If you do not want to incur the overhead of these allocations consider using
> `ma_context_enumerate_devices()` which will instead use a callback.
8442 |
8443 | Note that this only retrieves the ID and name/description of the device. The reason for only
> retrieving basic information is that it would otherwise require
8444 | opening the backend device in order to probe it for more detailed information which can be
> inefficient. Consider using `ma_context_get_device_info()` for this,
8445 | but don't call it from within the enumeration callback.
8446 |
8447 |
8448 | Parameters
8449 | -----
8450 | pContext (in)
8451 |     A pointer to the context performing the enumeration.
8452 |
8453 | ppPlaybackDeviceInfos (out)
8454 |     A pointer to a pointer that will receive the address of a buffer containing the list of
> `ma_device_info` structures for playback devices.
8455 |
8456 | pPlaybackDeviceCount (out)
8457 |     A pointer to an unsigned integer that will receive the number of playback devices.
8458 |
8459 | ppCaptureDeviceInfos (out)
8460 |     A pointer to a pointer that will receive the address of a buffer containing the list of
> `ma_device_info` structures for capture devices.
8461 |
8462 | pCaptureDeviceCount (out)
8463 |     A pointer to an unsigned integer that will receive the number of capture devices.
8464 |
8465 |
8466 | Return Value
8467 | -----
8468 | MA_SUCCESS if successful; any other error code otherwise.
8469 |
8470 |
8471 | Thread Safety
8472 | -----
8473 | Unsafe. Since each call to this function invalidates the pointers from the previous call, you should
> not be calling this simultaneously across multiple
8474 | threads. Instead, you need to make a copy of the returned data with your own higher level
> synchronization.
8475 |
8476 |
8477 | Remarks
8478 | -----
8479 | It is not safe to assume the first device in the list is the default device.
8480 |
8481 | You can pass in NULL for the playback or capture lists in which case they'll be ignored.
8482 |
8483 | The returned pointers will become invalid upon the next call this this function, or when the context
> is uninitialized. Do not free the returned pointers.
8484 |
8485 |
8486 | See Also
8487 | -----
8488 | ma_context_enumerate_devices()
8489 | */
8490 | MA_API ma_result ma_context_get_devices(ma_context* pContext, ma_device_info**
> ppPlaybackDeviceInfos, ma_uint32* pPlaybackDeviceCount, ma_device_info** ppCaptureDeviceInfos,
> ma_uint32* pCaptureDeviceCount);
8491 |
8492 | /*
8493 | Retrieves information about a device of the given type, with the specified ID and share mode.
8494 |
8495 |
8496 | Parameters
8497 | -----
8498 | pContext (in)

```

```

8499 |     A pointer to the context performing the query.
8500 |
8501 | deviceType (in)
8502 |     The type of the device being queried. Must be either `ma_device_type_playback` or
> `ma_device_type_capture`.
8503 |
8504 | pDeviceID (in)
8505 |     The ID of the device being queried.
8506 |
8507 | pDeviceInfo (out)
8508 |     A pointer to the `ma_device_info` structure that will receive the device information.
8509 |
8510 |
8511 | Return Value
8512 | -----
8513 | MA_SUCCESS if successful; any other error code otherwise.
8514 |
8515 |
8516 | Thread Safety
8517 | -----
8518 | Safe. This is guarded using a simple mutex lock.
8519 |
8520 |
8521 | Remarks
8522 | -----
8523 | Do not call this from within the `ma_context_enumerate_devices()` callback.
8524 |
8525 | It's possible for a device to have different information and capabilities depending on whether or
> not it's opened in shared or exclusive mode. For example, in
8526 | shared mode, WASAPI always uses floating point samples for mixing, but in exclusive mode it can be
> anything. Therefore, this function allows you to specify
8527 | which share mode you want information for. Note that not all backends and devices support shared or
> exclusive mode, in which case this function will fail if
8528 | the requested share mode is unsupported.
8529 |
8530 | This leaves pDeviceInfo unmodified in the result of an error.
8531 | */
8532 | MA_API ma_result ma_context_get_device_info(ma_context* pContext, ma_device_type deviceType, const
> ma_device_id* pDeviceID, ma_device_info* pDeviceInfo);
8533 |
8534 | /*
8535 | Determines if the given context supports loopback mode.
8536 |
8537 |
8538 | Parameters
8539 | -----
8540 | pContext (in)
8541 |     A pointer to the context getting queried.
8542 |
8543 |
8544 | Return Value
8545 | -----
8546 | MA_TRUE if the context supports loopback mode; MA_FALSE otherwise.
8547 | */
8548 | MA_API ma_bool32 ma_context_is_loopback_supported(ma_context* pContext);
8549 |
8550 |
8551 |
8552 | /*
8553 | Initializes a device config with default settings.
8554 |
8555 |
8556 | Parameters
8557 | -----
8558 | deviceType (in)
8559 |     The type of the device this config is being initialized for. This must set to one of the
> following:
8560 |
8561 | |-----|
8562 | | Device Type |
8563 | |-----|
8564 | | ma_device_type_playback |
8565 | | ma_device_type_capture |
8566 | | ma_device_type_duplex |
8567 | | ma_device_type_loopback |
8568 | |-----|
8569 |
8570 |
8571 | Return Value

```

```

8572 | -----
8573 | A new device config object with default settings. You will typically want to adjust the config after
> this function returns. See remarks.
8574 |
8575 |
8576 | Thread Safety
8577 | -----
8578 | Safe.
8579 |
8580 |
8581 | Callback Safety
8582 | -----
8583 | Safe, but don't try initializing a device in a callback.
8584 |
8585 |
8586 | Remarks
8587 | -----
8588 | The returned config will be initialized to defaults. You will normally want to customize a few
> variables before initializing the device. See Example 1 for a
8589 | typical configuration which sets the sample format, channel count, sample rate, data callback and
> user data. These are usually things you will want to change
8590 | before initializing the device.
8591 |
8592 | See `ma_device_init()` for details on specific configuration options.
8593 |
8594 |
8595 | Example 1 - Simple Configuration
8596 | -----
8597 | The example below is what a program will typically want to configure for each device at a minimum.
> Notice how `ma_device_config_init()` is called first, and
8598 | then the returned object is modified directly. This is important because it ensures that your
> program continues to work as new configuration options are added
8599 | to the `ma_device_config` structure.
8600 |
8601 | ```c
8602 | ma_device_config config = ma_device_config_init(ma_device_type_playback);
8603 | config.playback.format = ma_format_f32;
8604 | config.playback.channels = 2;
8605 | config.sampleRate = 48000;
8606 | config.dataCallback = ma_data_callback;
8607 | config.pUserData = pMyUserData;
8608 | ```
8609 |
8610 |
8611 | See Also
8612 | -----
8613 | ma_device_init()
8614 | ma_device_init_ex()
8615 | */
8616 | MA_API ma_device_config ma_device_config_init(ma_device_type deviceType);
8617 |
8618 |
8619 | /*
8620 | Initializes a device.
8621 |
8622 | A device represents a physical audio device. The idea is you send or receive audio data from the
> device to either play it back through a speaker, or capture it
8623 | from a microphone. Whether or not you should send or receive data from the device (or both) depends
> on the type of device you are initializing which can be
8624 | playback, capture, full-duplex or loopback. (Note that loopback mode is only supported on select
> backends.) Sending and receiving audio data to and from the
8625 | device is done via a callback which is fired by miniaudio at periodic time intervals.
8626 |
8627 | The frequency at which data is delivered to and from a device depends on the size of its period. The
> size of the period can be defined in terms of PCM frames
8628 | or milliseconds, whichever is more convenient. Generally speaking, the smaller the period, the lower
> the latency at the expense of higher CPU usage and
8629 | increased risk of glitching due to the more frequent and granular data deliver intervals. The size
> of a period will depend on your requirements, but
8630 | miniaudio's defaults should work fine for most scenarios. If you're building a game you should leave
> this fairly small, whereas if you're building a simple
8631 | media player you can make it larger. Note that the period size you request is actually just a hint -
> miniaudio will tell the backend what you want, but the
8632 | backend is ultimately responsible for what it gives you. You cannot assume you will get exactly what
> you ask for.
8633 |
8634 | When delivering data to and from a device you need to make sure it's in the correct format which you
> can set through the device configuration. You just set the
8635 | format that you want to use and miniaudio will perform all of the necessary conversion for you

```



```

> internally. When delivering data to and from the callback you
8636 | can assume the format is the same as what you requested when you initialized the device. See Remarks
> | for more details on miniaudio's data conversion pipeline.
8637 |
8638 |
8639 | Parameters
8640 | -----
8641 | pContext (in, optional)
8642 |     A pointer to the context that owns the device. This can be null, in which case it creates a
> | default context internally.
8643 |
8644 | pConfig (in)
8645 |     A pointer to the device configuration. Cannot be null. See remarks for details.
8646 |
8647 | pDevice (out)
8648 |     A pointer to the device object being initialized.
8649 |
8650 |
8651 | Return Value
8652 | -----
8653 | MA_SUCCESS if successful; any other error code otherwise.
8654 |
8655 |
8656 | Thread Safety
8657 | -----
8658 | Unsafe. It is not safe to call this function simultaneously for different devices because some
> | backends depend on and mutate global state. The same applies to
8659 | calling this at the same time as `ma_device_uninit()`.
8660 |
8661 |
8662 | Callback Safety
8663 | -----
8664 | Unsafe. It is not safe to call this inside any callback.
8665 |
8666 |
8667 | Remarks
8668 | -----
8669 | Setting `pContext` to NULL will result in miniaudio creating a default context internally and is
> | equivalent to passing in a context initialized like so:
8670 |
8671 |     ```c
8672 |     ma_context_init(NULL, 0, NULL, &context);
8673 |     ```
8674 |
8675 | Do not set `pContext` to NULL if you are needing to open multiple devices. You can, however, use
> | NULL when initializing the first device, and then use
8676 | device.pContext for the initialization of other devices.
8677 |
8678 | The device can be configured via the `pConfig` argument. The config object is initialized with
> | `ma_device_config_init()`. Individual configuration settings can
8679 | then be set directly on the structure. Below are the members of the `ma_device_config` object.
8680 |
8681 |     deviceType
8682 |         Must be `ma_device_type_playback`, `ma_device_type_capture`, `ma_device_type_duplex` of
> | `ma_device_type_loopback`.
8683 |
8684 |     sampleRate
8685 |         The sample rate, in hertz. The most common sample rates are 48000 and 44100. Setting this to
> | 0 will use the device's native sample rate.
8686 |
8687 |     periodSizeInFrames
8688 |         The desired size of a period in PCM frames. If this is 0, `periodSizeInMilliseconds` will be
> | used instead. If both are 0 the default buffer size will
8689 | be used depending on the selected performance profile. This value affects latency. See below
> | for details.
8690 |
8691 |     periodSizeInMilliseconds
8692 |         The desired size of a period in milliseconds. If this is 0, `periodSizeInFrames` will be
> | used instead. If both are 0 the default buffer size will be
8693 | used depending on the selected performance profile. The value affects latency. See below for
> | details.
8694 |
8695 |     periods
8696 |         The number of periods making up the device's entire buffer. The total buffer size is
> | `periodSizeInFrames` or `periodSizeInMilliseconds` multiplied by
8697 | this value. This is just a hint as backends will be the ones who ultimately decide how your
> | periods will be configured.
8698 |
8699 |     performanceProfile

```

```

8700 |         A hint to miniaudio as to the performance requirements of your program. Can be either
8701 | > `ma_performance_profile_low_latency` (default) or
8702 | > `ma_performance_profile_conservative`. This mainly affects the size of default buffers and
8703 | > can usually be left at its default value.
8704 |
8705 |     noPreSilencedOutputBuffer
8706 |         When set to true, the contents of the output buffer passed into the data callback will be
8707 | > left undefined. When set to false (default), the contents of
8708 | > the output buffer will be cleared the zero. You can use this to avoid the overhead of
8709 | > zeroing out the buffer if you can guarantee that your data
8710 | > callback will write to every sample in the output buffer, or if you are doing your own
8711 | > clearing.
8712 |
8713 |     noClip
8714 |         When set to true, the contents of the output buffer are left alone after returning and it
8715 | > will be left up to the backend itself to decide whether or
8716 | > not to clip. When set to false (default), the contents of the output buffer passed into the
8717 | > data callback will be clipped after returning. This only
8718 | > applies when the playback sample format is f32.
8719 |
8720 |     noDisableDenormals
8721 |         By default, miniaudio will disable denormals when the data callback is called. Setting this
8722 | > to true will prevent the disabling of denormals.
8723 |
8724 |     noFixedSizedCallback
8725 |         Allows miniaudio to fire the data callback with any frame count. When this is set to false
8726 | > (the default), the data callback will be fired with a
8727 | > consistent frame count as specified by `periodSizeInFrames` or `periodSizeInMilliseconds`.
8728 | > When set to true, miniaudio will fire the callback with
8729 | > whatever the backend requests, which could be anything.
8730 |
8731 |     dataCallback
8732 |         The callback to fire whenever data is ready to be delivered to or from the device.
8733 |
8734 |     notificationCallback
8735 |         The callback to fire when something has changed with the device, such as whether or not it
8736 | > has been started or stopped.
8737 |
8738 |     pUserData
8739 |         The user data pointer to use with the device. You can access this directly from the device
8740 | > object like `device.pUserData`.
8741 |
8742 |     resampling.algorithm
8743 |         The resampling algorithm to use when miniaudio needs to perform resampling between the rate
8744 | > specified by `sampleRate` and the device's native rate. The
8745 | > default value is `ma_resample_algorithm_linear`, and the quality can be configured with
8746 | > `resampling.linear.lpfOrder`.
8747 |
8748 |     resampling.pBackendVTable
8749 |         A pointer to an optional vtable that can be used for plugging in a custom resampler.
8750 |
8751 |     resampling.pBackendUserData
8752 |         A pointer that will passed to callbacks in pBackendVTable.
8753 |
8754 |     resampling.linear.lpfOrder
8755 |         The linear resampler applies a low-pass filter as part of its processing for anti-aliasing.
8756 | > This setting controls the order of the filter. The higher
8757 | > the value, the better the quality, in general. Setting this to 0 will disable low-pass
8758 | > filtering altogether. The maximum value is
8759 | > `MA_MAX_FILTER_ORDER`. The default value is `min(4, MA_MAX_FILTER_ORDER)`.
8760 |
8761 |     playback.pDeviceID
8762 |         A pointer to a `ma_device_id` structure containing the ID of the playback device to
8763 | > initialize. Setting this NULL (default) will use the system's
8764 | > default playback device. Retrieve the device ID from the `ma_device_info` structure, which
8765 | > can be retrieved using device enumeration.
8766 |
8767 |     playback.format
8768 |         The sample format to use for playback. When set to `ma_format_unknown` the device's native
8769 | > format will be used. This can be retrieved after
8770 | > initialization from the device object directly with `device.playback.format`.
8771 |
8772 |     playback.channels
8773 |         The number of channels to use for playback. When set to 0 the device's native channel count
8774 | > will be used. This can be retrieved after initialization
8775 | > from the device object directly with `device.playback.channels`.
8776 |
8777 |     playback.pChannelMap
8778 |         The channel map to use for playback. When left empty, the device's native channel map will

```

```

> be used. This can be retrieved after initialization from the
8759 |     device object direct with `device.playback.pChannelMap`. When set, the buffer should contain
> `channels` items.
8760 |
8761 |     playback.shareMode
8762 |     The preferred share mode to use for playback. Can be either `ma_share_mode_shared` (default)
> or `ma_share_mode_exclusive`. Note that if you specify
8763 |     exclusive mode, but it's not supported by the backend, initialization will fail. You can
> then fall back to shared mode if desired by changing this to
8764 |     ma_share_mode_shared and reinitializing.
8765 |
8766 |     capture.pDeviceID
8767 |     A pointer to a `ma_device_id` structure containing the ID of the capture device to
> initialize. Setting this NULL (default) will use the system's
8768 |     default capture device. Retrieve the device ID from the `ma_device_info` structure, which
> can be retrieved using device enumeration.
8769 |
8770 |     capture.format
8771 |     The sample format to use for capture. When set to `ma_format_unknown` the device's native
> format will be used. This can be retrieved after
8772 |     initialization from the device object directly with `device.capture.format`.
8773 |
8774 |     capture.channels
8775 |     The number of channels to use for capture. When set to 0 the device's native channel count
> will be used. This can be retrieved after initialization
8776 |     from the device object directly with `device.capture.channels`.
8777 |
8778 |     capture.pChannelMap
8779 |     The channel map to use for capture. When left empty, the device's native channel map will be
> used. This can be retrieved after initialization from the
8780 |     device object direct with `device.capture.pChannelMap`. When set, the buffer should contain
> `channels` items.
8781 |
8782 |     capture.shareMode
8783 |     The preferred share mode to use for capture. Can be either `ma_share_mode_shared` (default)
> or `ma_share_mode_exclusive`. Note that if you specify
8784 |     exclusive mode, but it's not supported by the backend, initialization will fail. You can
> then fall back to shared mode if desired by changing this to
8785 |     ma_share_mode_shared and reinitializing.
8786 |
8787 |     wasapi.noAutoConvertSRC
8788 |     WASAPI only. When set to true, disables WASAPI's automatic resampling and forces the use of
> miniaudio's resampler. Defaults to false.
8789 |
8790 |     wasapi.noDefaultQualitySRC
8791 |     WASAPI only. Only used when `wasapi.noAutoConvertSRC` is set to false. When set to true,
> disables the use of `AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY`.
8792 |     You should usually leave this set to false, which is the default.
8793 |
8794 |     wasapi.noAutoStreamRouting
8795 |     WASAPI only. When set to true, disables automatic stream routing on the WASAPI backend.
> Defaults to false.
8796 |
8797 |     wasapi.noHardwareOffloading
8798 |     WASAPI only. When set to true, disables the use of WASAPI's hardware offloading feature.
> Defaults to false.
8799 |
8800 |     alsa.noMMap
8801 |     ALSA only. When set to true, disables MMap mode. Defaults to false.
8802 |
8803 |     alsa.noAutoFormat
8804 |     ALSA only. When set to true, disables ALSA's automatic format conversion by including the
> SND_PCM_NO_AUTO_FORMAT flag. Defaults to false.
8805 |
8806 |     alsa.noAutoChannels
8807 |     ALSA only. When set to true, disables ALSA's automatic channel conversion by including the
> SND_PCM_NO_AUTO_CHANNELS flag. Defaults to false.
8808 |
8809 |     alsa.noAutoResample
8810 |     ALSA only. When set to true, disables ALSA's automatic resampling by including the
> SND_PCM_NO_AUTO_RESAMPLE flag. Defaults to false.
8811 |
8812 |     pulse.pStreamNamePlayback
8813 |     PulseAudio only. Sets the stream name for playback.
8814 |
8815 |     pulse.pStreamNameCapture
8816 |     PulseAudio only. Sets the stream name for capture.
8817 |
8818 |     pulse.channelMap

```

```

8819 |         PulseAudio only. Sets the channel map that is requested from PulseAudio. See
      > MA_PA_CHANNEL_MAP_* constants. Defaults to MA_PA_CHANNEL_MAP_AIFF.
8820 |
8821 |         coreaudio.allowNominalSampleRateChange
8822 |         Core Audio only. Desktop only. When enabled, allows the sample rate of the device to be
      > changed at the operating system level. This
8823 |         is disabled by default in order to prevent intrusive changes to the user's system. This is
      > useful if you want to use a sample rate
8824 |         that is known to be natively supported by the hardware thereby avoiding the cost of
      > resampling. When set to true, miniaudio will
8825 |         find the closest match between the sample rate requested in the device config and the sample
      > rates natively supported by the
8826 |         hardware. When set to false, the sample rate currently set by the operating system will
      > always be used.
8827 |
8828 |         openssl.streamType
8829 |         OpenSSL only. Explicitly sets the stream type. If left unset
      > ('ma_openssl_stream_type_default'), the
8830 |         stream type will be left unset. Think of this as the type of audio you're playing.
8831 |
8832 |         openssl.recordingPreset
8833 |         OpenSSL only. Explicitly sets the type of recording your program will be doing. When left
8834 |         unset, the recording preset will be left unchanged.
8835 |
8836 |         aaudio.usage
8837 |         AAudio only. Explicitly sets the nature of the audio the program will be consuming. When
8838 |         left unset, the usage will be left unchanged.
8839 |
8840 |         aaudio.contentType
8841 |         AAudio only. Sets the content type. When left unset, the content type will be left
      > unchanged.
8842 |
8843 |         aaudio.inputPreset
8844 |         AAudio only. Explicitly sets the type of recording your program will be doing. When left
8845 |         unset, the input preset will be left unchanged.
8846 |
8847 |         aaudio.noAutoStartAfterReroute
8848 |         AAudio only. Controls whether or not the device should be automatically restarted after a
8849 |         stream reroute. When set to false (default) the device will be restarted automatically;
8850 |         otherwise the device will be stopped.
8851 |
8852 |
8853 | Once initialized, the device's config is immutable. If you need to change the config you will need
      > to initialize a new device.
8854 |
8855 | After initializing the device it will be in a stopped state. To start it, use `ma_device_start()`.
8856 |
8857 | If both `periodSizeInFrames` and `periodSizeInMilliseconds` are set to zero, it will default to
      > `MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_LOW_LATENCY` or
8858 | `MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_CONSERVATIVE`, depending on whether or not
      > `performanceProfile` is set to `ma_performance_profile_low_latency` or
8859 | `ma_performance_profile_conservative`.
8860 |
8861 | If you request exclusive mode and the backend does not support it an error will be returned. For
      > robustness, you may want to first try initializing the device
8862 | in exclusive mode, and then fall back to shared mode if required. Alternatively you can just request
      > shared mode (the default if you leave it unset in the
8863 | config) which is the most reliable option. Some backends do not have a practical way of choosing
      > whether or not the device should be exclusive or not (ALSA,
8864 | for example) in which case it just acts as a hint. Unless you have special requirements you should
      > try avoiding exclusive mode as it's intrusive to the user.
8865 | Starting with Windows 10, miniaudio will use low-latency shared mode where possible which may make
      > exclusive mode unnecessary.
8866 |
8867 | When sending or receiving data to/from a device, miniaudio will internally perform a format
      > conversion to convert between the format specified by the config
8868 | and the format used internally by the backend. If you pass in 0 for the sample format, channel
      > count, sample rate and channel map, data transmission will run
8869 | on an optimized pass-through fast path. You can retrieve the format, channel count and sample rate
      > by inspecting the `playback/capture.format`,
8870 | `playback/capture.channels` and `sampleRate` members of the device object.
8871 |
8872 | When compiling for UWP you must ensure you call this function on the main UI thread because the
      > operating system may need to present the user with a message
8873 | asking for permissions. Please refer to the official documentation for ActivateAudioInterfaceAsync()
      > for more information.
8874 |
8875 | ALSA Specific: When initializing the default device, requesting shared mode will try using the
      > "dmix" device for playback and the "dsnoop" device for capture.

```

```

8876 | If these fail it will try falling back to the "hw" device.
8877 |
8878 |
8879 | Example 1 - Simple Initialization
8880 | -----
8881 | This example shows how to initialize a simple playback device using a standard configuration. If you
> are just needing to do simple playback from the default
8882 | playback device this is usually all you need.
8883 |
8884 | ```c
8885 | ma_device_config config = ma_device_config_init(ma_device_type_playback);
8886 | config.playback.format = ma_format_f32;
8887 | config.playback.channels = 2;
8888 | config.sampleRate = 48000;
8889 | config.dataCallback = ma_data_callback;
8890 | config.pMyUserData = pMyUserData;
8891 |
8892 | ma_device device;
8893 | ma_result result = ma_device_init(NULL, &config, &device);
8894 | if (result != MA_SUCCESS) {
8895 |     // Error
8896 | }
8897 | ```
8898 |
8899 |
8900 | Example 2 - Advanced Initialization
8901 | -----
8902 | This example shows how you might do some more advanced initialization. In this hypothetical example
> we want to control the latency by setting the buffer size
8903 | and period count. We also want to allow the user to be able to choose which device to output from
> which means we need a context so we can perform device
8904 | enumeration.
8905 |
8906 | ```c
8907 | ma_context context;
8908 | ma_result result = ma_context_init(NULL, 0, NULL, &context);
8909 | if (result != MA_SUCCESS) {
8910 |     // Error
8911 | }
8912 |
8913 | ma_device_info* pPlaybackDeviceInfos;
8914 | ma_uint32 playbackDeviceCount;
8915 | result = ma_context_get_devices(&context, &pPlaybackDeviceInfos, &playbackDeviceCount, NULL, NULL);
8916 | if (result != MA_SUCCESS) {
8917 |     // Error
8918 | }
8919 |
8920 | // ... choose a device from pPlaybackDeviceInfos ...
8921 |
8922 | ma_device_config config = ma_device_config_init(ma_device_type_playback);
8923 | config.playback.pDeviceID = pMyChosenDeviceID; // <-- Get this from the `id` member of one
> of the `ma_device_info` objects returned by ma_context_get_devices().
8924 | config.playback.format = ma_format_f32;
8925 | config.playback.channels = 2;
8926 | config.sampleRate = 48000;
8927 | config.dataCallback = ma_data_callback;
8928 | config.pUserData = pMyUserData;
8929 | config.periodSizeInMilliseconds = 10;
8930 | config.periods = 3;
8931 |
8932 | ma_device device;
8933 | result = ma_device_init(&context, &config, &device);
8934 | if (result != MA_SUCCESS) {
8935 |     // Error
8936 | }
8937 | ```
8938 |
8939 |
8940 | See Also
8941 | -----
8942 | ma_device_config_init()
8943 | ma_device_uninit()
8944 | ma_device_start()
8945 | ma_context_init()
8946 | ma_context_get_devices()
8947 | ma_context_enumerate_devices()
8948 | */
8949 | MA_API ma_result ma_device_init(ma_context* pContext, const ma_device_config* pConfig, ma_device*
> pDevice);

```

```

8950 |
8951 | /*
8952 | Initializes a device without a context, with extra parameters for controlling the configuration of
> the internal self-managed context.
8953 |
8954 | This is the same as `ma_device_init()`, only instead of a context being passed in, the parameters
> from `ma_context_init()` are passed in instead. This function
8955 | allows you to configure the internally created context.
8956 |
8957 |
8958 | Parameters
8959 | -----
8960 | backends (in, optional)
8961 |     A list of backends to try initializing, in priority order. Can be NULL, in which case it uses
> default priority order.
8962 |
8963 | backendCount (in, optional)
8964 |     The number of items in `backend`. Ignored if `backend` is NULL.
8965 |
8966 | pContextConfig (in, optional)
8967 |     The context configuration.
8968 |
8969 | pConfig (in)
8970 |     A pointer to the device configuration. Cannot be null. See remarks for details.
8971 |
8972 | pDevice (out)
8973 |     A pointer to the device object being initialized.
8974 |
8975 |
8976 | Return Value
8977 | -----
8978 | MA_SUCCESS if successful; any other error code otherwise.
8979 |
8980 |
8981 | Thread Safety
8982 | -----
8983 | Unsafe. It is not safe to call this function simultaneously for different devices because some
> backends depend on and mutate global state. The same applies to
8984 | calling this at the same time as `ma_device_uninit()`.
8985 |
8986 |
8987 | Callback Safety
8988 | -----
8989 | Unsafe. It is not safe to call this inside any callback.
8990 |
8991 |
8992 | Remarks
8993 | -----
8994 | You only need to use this function if you want to configure the context differently to its defaults.
> You should never use this function if you want to manage
8995 | your own context.
8996 |
8997 | See the documentation for `ma_context_init()` for information on the different context configuration
> options.
8998 |
8999 |
9000 | See Also
9001 | -----
9002 | ma_device_init()
9003 | ma_device_uninit()
9004 | ma_device_config_init()
9005 | ma_context_init()
9006 | */
9007 | MA_API ma_result ma_device_init_ex(const ma_backend backends[], ma_uint32 backendCount, const
> ma_context_config* pContextConfig, const ma_device_config* pConfig, ma_device* pDevice);
9008 |
9009 | /*
9010 | Uninitializes a device.
9011 |
9012 | This will explicitly stop the device. You do not need to call `ma_device_stop()` beforehand, but
> it's harmless if you do.
9013 |
9014 |
9015 | Parameters
9016 | -----
9017 | pDevice (in)
9018 |     A pointer to the device to stop.
9019 |
9020 |

```

```

9021 | Return Value
9022 | -----
9023 | Nothing
9024 |
9025 |
9026 | Thread Safety
9027 | -----
9028 | Unsafe. As soon as this API is called the device should be considered undefined.
9029 |
9030 |
9031 | Callback Safety
9032 | -----
9033 | Unsafe. It is not safe to call this inside any callback. Doing this will result in a deadlock.
9034 |
9035 |
9036 | See Also
9037 | -----
9038 | ma_device_init()
9039 | ma_device_stop()
9040 | */
9041 | MA_API void ma_device_uninit(ma_device* pDevice);
9042 |
9043 |
9044 | /*
9045 | Retrieves a pointer to the context that owns the given device.
9046 | */
9047 | MA_API ma_context* ma_device_get_context(ma_device* pDevice);
9048 |
9049 | /*
9050 | Helper function for retrieving the log object associated with the context that owns this device.
9051 | */
9052 | MA_API ma_log* ma_device_get_log(ma_device* pDevice);
9053 |
9054 |
9055 | /*
9056 | Retrieves information about the device.
9057 |
9058 |
9059 | Parameters
9060 | -----
9061 | pDevice (in)
9062 |     A pointer to the device whose information is being retrieved.
9063 |
9064 | type (in)
9065 |     The device type. This parameter is required for duplex devices. When retrieving device
9066 |     information, you are doing so for an individual playback or capture device.
9067 |
9068 | pDeviceInfo (out)
9069 |     A pointer to the `ma_device_info` that will receive the device information.
9070 |
9071 |
9072 | Return Value
9073 | -----
9074 | MA_SUCCESS if successful; any other error code otherwise.
9075 |
9076 |
9077 | Thread Safety
9078 | -----
9079 | Unsafe. This should be considered unsafe because it may be calling into the backend which may or
9080 | may not be safe.
9081 |
9082 |
9083 | Callback Safety
9084 | -----
9085 | Unsafe. You should avoid calling this in the data callback because it may call into the backend
9086 | which may or may not be safe.
9087 | */
9088 | MA_API ma_result ma_device_get_info(ma_device* pDevice, ma_device_type type, ma_device_info*
> pDeviceInfo);
9089 |
9090 |
9091 | /*
9092 | Retrieves the name of the device.
9093 |
9094 |
9095 | Parameters
9096 | -----
9097 | pDevice (in)
9098 |     A pointer to the device whose information is being retrieved.

```



```

9099 |
9100 | type (in)
9101 |     The device type. This parameter is required for duplex devices. When retrieving device
9102 |     information, you are doing so for an individual playback or capture device.
9103 |
9104 | pName (out)
9105 |     A pointer to the buffer that will receive the name.
9106 |
9107 | nameCap (in)
9108 |     The capacity of the output buffer, including space for the null terminator.
9109 |
9110 | pLengthNotIncludingNullTerminator (out, optional)
9111 |     A pointer to the variable that will receive the length of the name, not including the null
9112 |     terminator.
9113 |
9114 |
9115 | Return Value
9116 | -----
9117 | MA_SUCCESS if successful; any other error code otherwise.
9118 |
9119 |
9120 | Thread Safety
9121 | -----
9122 | Unsafe. This should be considered unsafe because it may be calling into the backend which may or
9123 | may not be safe.
9124 |
9125 |
9126 | Callback Safety
9127 | -----
9128 | Unsafe. You should avoid calling this in the data callback because it may call into the backend
9129 | which may or may not be safe.
9130 |
9131 |
9132 | Remarks
9133 | -----
9134 | If the name does not fully fit into the output buffer, it'll be truncated. You can pass in NULL to
9135 | `pName` if you want to first get the length of the name for the purpose of memory allocation of the
9136 | output buffer. Allocating a buffer of size `MA_MAX_DEVICE_NAME_LENGTH + 1` should be enough for
9137 | most cases and will avoid the need for the inefficiency of calling this function twice.
9138 |
9139 | This is implemented in terms of `ma_device_get_info()`.
9140 | */
9141 | MA_API ma_result ma_device_get_name(ma_device* pDevice, ma_device_type type, char* pName, size_t
> nameCap, size_t* pLengthNotIncludingNullTerminator);
9142 |
9143 |
9144 | /*
9145 | Starts the device. For playback devices this begins playback. For capture devices it begins
> recording.
9146 |
9147 | Use `ma_device_stop()` to stop the device.
9148 |
9149 |
9150 | Parameters
9151 | -----
9152 | pDevice (in)
9153 |     A pointer to the device to start.
9154 |
9155 |
9156 | Return Value
9157 | -----
9158 | MA_SUCCESS if successful; any other error code otherwise.
9159 |
9160 |
9161 | Thread Safety
9162 | -----
9163 | Safe. It's safe to call this from any thread with the exception of the callback thread.
9164 |
9165 |
9166 | Callback Safety
9167 | -----
9168 | Unsafe. It is not safe to call this inside any callback.
9169 |
9170 |
9171 | Remarks
9172 | -----
9173 | For a playback device, this will retrieve an initial chunk of audio data from the client before
> returning. The reason for this is to ensure there is valid
9174 | audio data in the buffer, which needs to be done before the device begins playback.

```

```
9175 |
9176 | This API waits until the backend device has been started for real by the worker thread. It also
9177 | > waits on a mutex for thread-safety.
9178 |
9179 | Do not call this in any callback.
9180 |
9181 | See Also
9182 | -----
9183 | ma_device_stop()
9184 | */
9185 | MA_API ma_result ma_device_start(ma_device* pDevice);
9186 |
9187 | /*
9188 | Stops the device. For playback devices this stops playback. For capture devices it stops recording.
9189 |
9190 | Use `ma_device_start()` to start the device again.
9191 |
9192 |
9193 | Parameters
9194 | -----
9195 | pDevice (in)
9196 |     A pointer to the device to stop.
9197 |
9198 |
9199 | Return Value
9200 | -----
9201 | MA_SUCCESS if successful; any other error code otherwise.
9202 |
9203 |
9204 | Thread Safety
9205 | -----
9206 | Safe. It's safe to call this from any thread with the exception of the callback thread.
9207 |
9208 |
9209 | Callback Safety
9210 | -----
9211 | Unsafe. It is not safe to call this inside any callback. Doing this will result in a deadlock.
9212 |
9213 |
9214 | Remarks
9215 | -----
9216 | This API needs to wait on the worker thread to stop the backend device properly before returning. It
9217 | > also waits on a mutex for thread-safety. In addition, some
9218 | backends need to wait for the device to finish playback/recording of the current fragment which can
9219 | > take some time (usually proportionate to the buffer size
9220 | that was specified at initialization time).
9221 |
9222 | Backends are required to either pause the stream in-place or drain the buffer if pausing is not
9223 | > possible. The reason for this is that stopping the device and
9224 | the resuming it with ma_device_start() (which you might do when your program loses focus) may result
9225 | > in a situation where those samples are never output to the
9226 | speakers or received from the microphone which can in turn result in de-syncs.
9227 |
9228 | Do not call this in any callback.
9229 |
9230 | See Also
9231 | -----
9232 | ma_device_start()
9233 | */
9234 | MA_API ma_result ma_device_stop(ma_device* pDevice);
9235 |
9236 | /*
9237 | Determines whether or not the device is started.
9238 |
9239 |
9240 | Parameters
9241 | -----
9242 | pDevice (in)
9243 |     A pointer to the device whose start state is being retrieved.
9244 |
9245 |
9246 | Return Value
9247 | -----
9248 | True if the device is started, false otherwise.
9249 |
9250 |
9251 | Thread Safety
```

```

9249 | -----
9250 | Safe. If another thread calls `ma_device_start()` or `ma_device_stop()` at this same time as this
> function is called, there's a very small chance the return
9251 | value will be out of sync.
9252 |
9253 |
9254 | Callback Safety
9255 | -----
9256 | Safe. This is implemented as a simple accessor.
9257 |
9258 |
9259 | See Also
9260 | -----
9261 | ma_device_start()
9262 | ma_device_stop()
9263 | */
9264 | MA_API ma_bool32 ma_device_is_started(const ma_device* pDevice);
9265 |
9266 |
9267 | /*
9268 | Retrieves the state of the device.
9269 |
9270 |
9271 | Parameters
9272 | -----
9273 | pDevice (in)
9274 |     A pointer to the device whose state is being retrieved.
9275 |
9276 |
9277 | Return Value
9278 | -----
9279 | The current state of the device. The return value will be one of the following:
9280 |
9281 | +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
> +-----+
9282 | | ma_device_state_uninitialized | Will only be returned if the device is in the middle of
> initialization. |
9283 | +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
> +-----+
9284 | | ma_device_state_stopped      | The device is stopped. The initial state of the device after
> initialization. |
9285 | +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
> +-----+
9286 | | ma_device_state_started      | The device started and requesting and/or delivering audio
> data. |
9287 | +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
> +-----+
9288 | | ma_device_state_starting     | The device is in the process of starting.
> |
9289 | +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
> +-----+
9290 | | ma_device_state_stopping     | The device is in the process of stopping.
> |
9291 | +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
> +-----+
9292 |
9293 |
9294 | Thread Safety
9295 | -----
9296 | Safe. This is implemented as a simple accessor. Note that if the device is started or stopped at the
> same time as this function is called,
9297 | there's a possibility the return value could be out of sync. See remarks.
9298 |
9299 |
9300 | Callback Safety
9301 | -----
9302 | Safe. This is implemented as a simple accessor.
9303 |
9304 |
9305 | Remarks
9306 | -----
9307 | The general flow of a devices state goes like this:
9308 |
9309 | ```
9310 | ma_device_init() -> ma_device_state_uninitialized -> ma_device_state_stopped
9311 | ma_device_start() -> ma_device_state_starting      -> ma_device_state_started
9312 | ma_device_stop() -> ma_device_state_stopping      -> ma_device_state_stopped
9313 | ```
9314 |

```

```

9315 | When the state of the device is changed with `ma_device_start()` or `ma_device_stop()` at this same
> time as this function is called, the
9316 | value returned by this function could potentially be out of sync. If this is significant to your
> program you need to implement your own
9317 | synchronization.
9318 | */
9319 | MA_API ma_device_state ma_device_get_state(const ma_device* pDevice);
9320 |
9321 |
9322 | /*
9323 | Performs post backend initialization routines for setting up internal data conversion.
9324 |
9325 | This should be called whenever the backend is initialized. The only time this should be called from
9326 | outside of miniaudio is if you're implementing a custom backend, and you would only do it if you
9327 | are reinitializing the backend due to rerouting or reinitializing for some reason.
9328 |
9329 | Parameters
9330 | -----
9331 | pDevice [in]
9332 |     A pointer to the device.
9333 |
9334 | deviceType [in]
9335 |     The type of the device that was just reinitialized.
9336 |
9337 | pPlaybackDescriptor [in]
9338 |     The descriptor of the playback device containing the internal data format and buffer sizes.
9339 |
9340 | pCaptureDescriptor [in]
9341 |     The descriptor of the capture device containing the internal data format and buffer sizes.
9342 |
9343 | Return Value
9344 | -----
9345 | MA_SUCCESS if successful; any other error otherwise.
9346 |
9347 | Thread Safety
9348 | -----
9349 | Unsafe. This will be reinitializing internal data converters which may be in use by another thread.
9350 |
9351 | Callback Safety
9352 | -----
9353 | Unsafe. This will be reinitializing internal data converters which may be in use by the callback.
9354 |
9355 | Remarks
9356 | -----
9357 | For a duplex device, you can call this for only one side of the system. This is why the deviceType
9358 | is specified as a parameter rather than deriving it from the device.
9359 |
9360 | You do not need to call this manually unless you are doing a custom backend, in which case you need
9361 | only do it if you're manually performing rerouting or reinitialization.
9362 | */
9363 | MA_API ma_result ma_device_post_init(ma_device* pDevice, ma_device_type deviceType, const
> ma_device_descriptor* pPlaybackDescriptor, const ma_device_descriptor* pCaptureDescriptor);
9364 |
9365 |
9366 | /*
9367 | Sets the master volume factor for the device.
9368 |
9369 | The volume factor must be between 0 (silence) and 1 (full volume). Use
> `ma_device_set_master_volume_db()` to use decibel notation, where 0 is full volume and
9370 | values less than 0 decreases the volume.
9371 |
9372 | Parameters
9373 | -----
9374 | pDevice (in)
9375 |     A pointer to the device whose volume is being set.
9376 |
9377 | volume (in)
9378 |     The new volume factor. Must be >= 0.
9379 |
9380 | Return Value
9381 | -----
9382 | MA_SUCCESS if the volume was set successfully.

```

```

9390 | MA_INVALID_ARGS if pDevice is NULL.
9391 | MA_INVALID_ARGS if volume is negative.
9392 |
9393 |
9394 | Thread Safety
9395 | -----
9396 | Safe. This just sets a local member of the device object.
9397 |
9398 |
9399 | Callback Safety
9400 | -----
9401 | Safe. If you set the volume in the data callback, that data written to the output buffer will have
> the new volume applied.
9402 |
9403 |
9404 | Remarks
9405 | -----
9406 | This applies the volume factor across all channels.
9407 |
9408 | This does not change the operating system's volume. It only affects the volume for the given
> `ma_device` object's audio stream.
9409 |
9410 |
9411 | See Also
9412 | -----
9413 | ma_device_get_master_volume()
9414 | ma_device_set_master_volume_db()
9415 | ma_device_get_master_volume_db()
9416 | */
9417 | MA_API ma_result ma_device_set_master_volume(ma_device* pDevice, float volume);
9418 |
9419 | /*
9420 | Retrieves the master volume factor for the device.
9421 |
9422 |
9423 | Parameters
9424 | -----
9425 | pDevice (in)
9426 |     A pointer to the device whose volume factor is being retrieved.
9427 |
9428 | pVolume (in)
9429 |     A pointer to the variable that will receive the volume factor. The returned value will be in the
> range of [0, 1].
9430 |
9431 |
9432 | Return Value
9433 | -----
9434 | MA_SUCCESS if successful.
9435 | MA_INVALID_ARGS if pDevice is NULL.
9436 | MA_INVALID_ARGS if pVolume is NULL.
9437 |
9438 |
9439 | Thread Safety
9440 | -----
9441 | Safe. This just a simple member retrieval.
9442 |
9443 |
9444 | Callback Safety
9445 | -----
9446 | Safe.
9447 |
9448 |
9449 | Remarks
9450 | -----
9451 | If an error occurs, `*pVolume` will be set to 0.
9452 |
9453 |
9454 | See Also
9455 | -----
9456 | ma_device_set_master_volume()
9457 | ma_device_set_master_volume_gain_db()
9458 | ma_device_get_master_volume_gain_db()
9459 | */
9460 | MA_API ma_result ma_device_get_master_volume(ma_device* pDevice, float* pVolume);
9461 |
9462 | /*
9463 | Sets the master volume for the device as gain in decibels.
9464 |
9465 | A gain of 0 is full volume, whereas a gain of < 0 will decrease the volume.

```

```
9466 |
9467 |
9468 | Parameters
9469 | -----
9470 | pDevice (in)
9471 |     A pointer to the device whose gain is being set.
9472 |
9473 | gainDB (in)
9474 |     The new volume as gain in decibels. Must be less than or equal to 0, where 0 is full volume and
> anything less than 0 decreases the volume.
9475 |
9476 |
9477 | Return Value
9478 | -----
9479 | MA_SUCCESS if the volume was set successfully.
9480 | MA_INVALID_ARGS if pDevice is NULL.
9481 | MA_INVALID_ARGS if the gain is > 0.
9482 |
9483 |
9484 | Thread Safety
9485 | -----
9486 | Safe. This just sets a local member of the device object.
9487 |
9488 |
9489 | Callback Safety
9490 | -----
9491 | Safe. If you set the volume in the data callback, that data written to the output buffer will have
> the new volume applied.
9492 |
9493 |
9494 | Remarks
9495 | -----
9496 | This applies the gain across all channels.
9497 |
9498 | This does not change the operating system's volume. It only affects the volume for the given
> `ma_device` object's audio stream.
9499 |
9500 |
9501 | See Also
9502 | -----
9503 | ma_device_get_master_volume_gain_db()
9504 | ma_device_set_master_volume()
9505 | ma_device_get_master_volume()
9506 | */
9507 | MA_API ma_result ma_device_set_master_volume_db(ma_device* pDevice, float gainDB);
9508 |
9509 | /*
9510 | Retrieves the master gain in decibels.
9511 |
9512 |
9513 | Parameters
9514 | -----
9515 | pDevice (in)
9516 |     A pointer to the device whose gain is being retrieved.
9517 |
9518 | pGainDB (in)
9519 |     A pointer to the variable that will receive the gain in decibels. The returned value will be <=
> 0.
9520 |
9521 |
9522 | Return Value
9523 | -----
9524 | MA_SUCCESS if successful.
9525 | MA_INVALID_ARGS if pDevice is NULL.
9526 | MA_INVALID_ARGS if pGainDB is NULL.
9527 |
9528 |
9529 | Thread Safety
9530 | -----
9531 | Safe. This just a simple member retrieval.
9532 |
9533 |
9534 | Callback Safety
9535 | -----
9536 | Safe.
9537 |
9538 |
9539 | Remarks
9540 | -----
```

```

9541 | If an error occurs, `*pGainDB` will be set to 0.
9542 |
9543 |
9544 | See Also
9545 | -----
9546 | ma_device_set_master_volume_db()
9547 | ma_device_set_master_volume()
9548 | ma_device_get_master_volume()
9549 | */
9550 | MA_API ma_result ma_device_get_master_volume_db(ma_device* pDevice, float* pGainDB);
9551 |
9552 |
9553 | /*
9554 | Called from the data callback of asynchronous backends to allow miniaudio to process the data and
> fire the miniaudio data callback.
9555 |
9556 |
9557 | Parameters
9558 | -----
9559 | pDevice (in)
9560 |     A pointer to device whose processing the data callback.
9561 |
9562 | pOutput (out)
9563 |     A pointer to the buffer that will receive the output PCM frame data. On a playback device this
> must not be NULL. On a duplex device
9564 |     this can be NULL, in which case pInput must not be NULL.
9565 |
9566 | pInput (in)
9567 |     A pointer to the buffer containing input PCM frame data. On a capture device this must not be
> NULL. On a duplex device this can be
9568 |     NULL, in which case `pOutput` must not be NULL.
9569 |
9570 | frameCount (in)
9571 |     The number of frames being processed.
9572 |
9573 |
9574 | Return Value
9575 | -----
9576 | MA_SUCCESS if successful; any other result code otherwise.
9577 |
9578 |
9579 | Thread Safety
9580 | -----
9581 | This function should only ever be called from the internal data callback of the backend. It is safe
> to call this simultaneously between a
9582 | playback and capture device in duplex setups.
9583 |
9584 |
9585 | Callback Safety
9586 | -----
9587 | Do not call this from the miniaudio data callback. It should only ever be called from the internal
> data callback of the backend.
9588 |
9589 |
9590 | Remarks
9591 | -----
9592 | If both `pOutput` and `pInput` are NULL, and error will be returned. In duplex scenarios, both
> `pOutput` and `pInput` can be non-NULL, in
9593 | which case `pInput` will be processed first, followed by `pOutput`.
9594 |
9595 | If you are implementing a custom backend, and that backend uses a callback for data delivery, you'll
> need to call this from inside that
9596 | callback.
9597 | */
9598 | MA_API ma_result ma_device_handle_backend_data_callback(ma_device* pDevice, void* pOutput, const
> void* pInput, ma_uint32 frameCount);
9599 |
9600 |
9601 | /*
9602 | Calculates an appropriate buffer size from a descriptor, native sample rate and performance profile.
9603 |
9604 | This function is used by backends for helping determine an appropriately sized buffer to use with
9605 | the device depending on the values of `periodSizeInFrames` and `periodSizeInMilliseconds` in the
9606 | `pDescriptor` object. Since buffer size calculations based on time depends on the sample rate, a
9607 | best guess at the device's native sample rate is also required which is where `nativeSampleRate`
9608 | comes in. In addition, the performance profile is also needed for cases where both the period size
9609 | in frames and milliseconds are both zero.
9610 |
9611 |

```



```

9612 | Parameters
9613 | -----
9614 | pDescriptor (in)
9615 |     A pointer to device descriptor whose `periodSizeInFrames` and `periodSizeInMilliseconds` members
9616 |     will be used for the calculation of the buffer size.
9617 |
9618 | nativeSampleRate (in)
9619 |     The device's native sample rate. This is only ever used when the `periodSizeInFrames` member of
9620 |     `pDescriptor` is zero. In this case, `periodSizeInMilliseconds` will be used instead, in which
9621 |     case a sample rate is required to convert to a size in frames.
9622 |
9623 | performanceProfile (in)
9624 |     When both the `periodSizeInFrames` and `periodSizeInMilliseconds` members of `pDescriptor` are
9625 |     zero, miniaudio will fall back to a buffer size based on the performance profile. The profile
9626 |     to use for this calculation is determine by this parameter.
9627 |
9628 |
9629 | Return Value
9630 | -----
9631 | The calculated buffer size in frames.
9632 |
9633 |
9634 | Thread Safety
9635 | -----
9636 | This is safe so long as nothing modifies `pDescriptor` at the same time. However, this function
9637 | should only ever be called from within the backend's device initialization routine and therefore
9638 | shouldn't have any multithreading concerns.
9639 |
9640 |
9641 | Callback Safety
9642 | -----
9643 | This is safe to call within the data callback, but there is no reason to ever do this.
9644 |
9645 |
9646 | Remarks
9647 | -----
9648 | If `nativeSampleRate` is zero, this function will fall back to `pDescriptor->sampleRate`. If that
9649 | is also zero, `MA_DEFAULT_SAMPLE_RATE` will be used instead.
9650 | */
9651 | MA_API ma_uint32 ma_calculate_buffer_size_in_frames_from_descriptor(const ma_device_descriptor*
> pDescriptor, ma_uint32 nativeSampleRate, ma_performance_profile performanceProfile);
9652 |
9653 |
9654 |
9655 | /*
9656 | Retrieves a friendly name for a backend.
9657 | */
9658 | MA_API const char* ma_get_backend_name(ma_backend backend);
9659 |
9660 | /*
9661 | Retrieves the backend enum from the given name.
9662 | */
9663 | MA_API ma_result ma_get_backend_from_name(const char* pBackendName, ma_backend* pBackend);
9664 |
9665 | /*
9666 | Determines whether or not the given backend is available by the compilation environment.
9667 | */
9668 | MA_API ma_bool32 ma_is_backend_enabled(ma_backend backend);
9669 |
9670 | /*
9671 | Retrieves compile-time enabled backends.
9672 |
9673 |
9674 | Parameters
9675 | -----
9676 | pBackends (out, optional)
9677 |     A pointer to the buffer that will receive the enabled backends. Set to NULL to retrieve the
> backend count. Setting
9678 |     the capacity of the buffer to `MA_BACKEND_COUNT` will guarantee it's large enough for all
> backends.
9679 |
9680 | backendCap (in)
9681 |     The capacity of the `pBackends` buffer.
9682 |
9683 | pBackendCount (out)
9684 |     A pointer to the variable that will receive the enabled backend count.
9685 |
9686 |
9687 | Return Value

```

```

9688 | -----
9689 | MA_SUCCESS if successful.
9690 | MA_INVALID_ARGS if `pBackendCount` is NULL.
9691 | MA_NO_SPACE if the capacity of `pBackends` is not large enough.
9692 |
9693 | If `MA_NO_SPACE` is returned, the `pBackends` buffer will be filled with `*pBackendCount` values.
9694 |
9695 |
9696 | Thread Safety
9697 | -----
9698 | Safe.
9699 |
9700 |
9701 | Callback Safety
9702 | -----
9703 | Safe.
9704 |
9705 |
9706 | Remarks
9707 | -----
9708 | If you want to retrieve the number of backends so you can determine the capacity of `pBackends`
9709 | > buffer, you can call
9710 | this function with `pBackends` set to NULL.
9711 | This will also enumerate the null backend. If you don't want to include this you need to check for
9712 | > `ma_backend_null`
9713 | when you enumerate over the returned backends and handle it appropriately. Alternatively, you can
9714 | > disable it at
9715 | compile time with `MA_NO_NULL`.
9716 |
9717 | The returned backends are determined based on compile time settings, not the platform it's currently
9718 | > running on. For
9719 | example, PulseAudio will be returned if it was enabled at compile time, even when the user doesn't
9720 | > actually have
9721 | PulseAudio installed.
9722 |
9723 | Example 1
9724 | -----
9725 | The example below retrieves the enabled backend count using a fixed sized buffer allocated on the
9726 | > stack. The buffer is
9727 | given a capacity of `MA_BACKEND_COUNT` which will guarantee it'll be large enough to store all
9728 | > available backends.
9729 | Since `MA_BACKEND_COUNT` is always a relatively small value, this should be suitable for most
9730 | > scenarios.
9731 |
9732 | ```
9733 | ma_backend enabledBackends[MA_BACKEND_COUNT];
9734 | size_t enabledBackendCount;
9735 |
9736 | result = ma_get_enabled_backends(enabledBackends, MA_BACKEND_COUNT, &enabledBackendCount);
9737 | if (result != MA_SUCCESS) {
9738 |     // Failed to retrieve enabled backends. Should never happen in this example since all inputs are
9739 |     > valid.
9740 | }
9741 | ```
9742 |
9743 | See Also
9744 | -----
9745 | ma_is_backend_enabled()
9746 | */
9747 | MA_API ma_result ma_get_enabled_backends(ma_backend* pBackends, size_t backendCap, size_t*
9748 | > pBackendCount);
9749 |
9750 | /*
9751 | Determines whether or not loopback mode is support by a backend.
9752 | */
9753 | MA_API ma_bool32 ma_is_loopback_supported(ma_backend backend);
9754 |
9755 | #endif /* MA_NO_DEVICE_IO */
9756 |
9757 |
9758 | /*****
9759 | > *****/
9760 |
9761 | Utilities
9762 |

```

```

9756 | *****
> *****/
9757 |
9758 | /*
9759 | Calculates a buffer size in milliseconds (rounded up) from the specified number of frames and sample
> rate.
9760 | */
9761 | MA_API ma_uint32 ma_calculate_buffer_size_in_milliseconds_from_frames(ma_uint32 bufferSizeInFrames,
> ma_uint32 sampleRate);
9762 |
9763 | /*
9764 | Calculates a buffer size in frames from the specified number of milliseconds and sample rate.
9765 | */
9766 | MA_API ma_uint32 ma_calculate_buffer_size_in_frames_from_milliseconds(ma_uint32
> bufferSizeInMilliseconds, ma_uint32 sampleRate);
9767 |
9768 | /*
9769 | Copies PCM frames from one buffer to another.
9770 | */
9771 | MA_API void ma_copy_pcm_frames(void* dst, const void* src, ma_uint64 frameCount, ma_format format,
> ma_uint32 channels);
9772 |
9773 | /*
9774 | Copies silent frames into the given buffer.
9775 |
9776 | Remarks
9777 | -----
9778 | For all formats except `ma_format_u8`, the output buffer will be filled with 0. For `ma_format_u8`
> it will be filled with 128. The reason for this is that it
9779 | makes more sense for the purpose of mixing to initialize it to the center point.
9780 | */
9781 | MA_API void ma_silence_pcm_frames(void* p, ma_uint64 frameCount, ma_format format, ma_uint32
> channels);
9782 |
9783 |
9784 | /*
9785 | Offsets a pointer by the specified number of PCM frames.
9786 | */
9787 | MA_API void* ma_offset_pcm_frames_ptr(void* p, ma_uint64 offsetInFrames, ma_format format, ma_uint32
> channels);
9788 | MA_API const void* ma_offset_pcm_frames_const_ptr(const void* p, ma_uint64 offsetInFrames, ma_format
> format, ma_uint32 channels);
9789 | static MA_INLINE float* ma_offset_pcm_frames_ptr_f32(float* p, ma_uint64 offsetInFrames, ma_uint32
> channels) { return (float*)ma_offset_pcm_frames_ptr((void*)p, offsetInFrames, ma_format_f32,
> channels); }
9790 | static MA_INLINE const float* ma_offset_pcm_frames_const_ptr_f32(const float* p, ma_uint64
> offsetInFrames, ma_uint32 channels) { return (const float*)ma_offset_pcm_frames_const_ptr((const
> void*)p, offsetInFrames, ma_format_f32, channels); }
9791 |
9792 |
9793 | /*
9794 | Clips samples.
9795 | */
9796 | MA_API void ma_clip_samples_u8(ma_uint8* pDst, const ma_int16* pSrc, ma_uint64 count);
9797 | MA_API void ma_clip_samples_s16(ma_int16* pDst, const ma_int32* pSrc, ma_uint64 count);
9798 | MA_API void ma_clip_samples_s24(ma_uint8* pDst, const ma_int64* pSrc, ma_uint64 count);
9799 | MA_API void ma_clip_samples_s32(ma_int32* pDst, const ma_int64* pSrc, ma_uint64 count);
9800 | MA_API void ma_clip_samples_f32(float* pDst, const float* pSrc, ma_uint64 count);
9801 | MA_API void ma_clip_pcm_frames(void* pDst, const void* pSrc, ma_uint64 frameCount, ma_format format,
> ma_uint32 channels);
9802 |
9803 | /*
9804 | Helper for applying a volume factor to samples.
9805 |
9806 | Note that the source and destination buffers can be the same, in which case it'll perform the
> operation in-place.
9807 | */
9808 | MA_API void ma_copy_and_apply_volume_factor_u8(ma_uint8* pSamplesOut, const ma_uint8* pSamplesIn,
> ma_uint64 sampleCount, float factor);
9809 | MA_API void ma_copy_and_apply_volume_factor_s16(ma_int16* pSamplesOut, const ma_int16* pSamplesIn,
> ma_uint64 sampleCount, float factor);
9810 | MA_API void ma_copy_and_apply_volume_factor_s24(void* pSamplesOut, const void* pSamplesIn, ma_uint64
> sampleCount, float factor);
9811 | MA_API void ma_copy_and_apply_volume_factor_s32(ma_int32* pSamplesOut, const ma_int32* pSamplesIn,
> ma_uint64 sampleCount, float factor);
9812 | MA_API void ma_copy_and_apply_volume_factor_f32(float* pSamplesOut, const float* pSamplesIn,
> ma_uint64 sampleCount, float factor);
9813 |
9814 | MA_API void ma_apply_volume_factor_u8(ma_uint8* pSamples, ma_uint64 sampleCount, float factor);

```

```

9815 | MA_API void ma_apply_volume_factor_s16(ma_int16* pSamples, ma_uint64 sampleCount, float factor);
9816 | MA_API void ma_apply_volume_factor_s24(void* pSamples, ma_uint64 sampleCount, float factor);
9817 | MA_API void ma_apply_volume_factor_s32(ma_int32* pSamples, ma_uint64 sampleCount, float factor);
9818 | MA_API void ma_apply_volume_factor_f32(float* pSamples, ma_uint64 sampleCount, float factor);
9819 |
9820 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_u8(ma_uint8* pFramesOut, const ma_uint8*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor);
9821 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_s16(ma_int16* pFramesOut, const ma_int16*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor);
9822 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_s24(void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount, ma_uint32 channels, float factor);
9823 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_s32(ma_int32* pFramesOut, const ma_int32*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor);
9824 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_f32(float* pFramesOut, const float*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor);
9825 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames(void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount, ma_format format, ma_uint32 channels, float factor);
9826 |
9827 | MA_API void ma_apply_volume_factor_pcm_frames_u8(ma_uint8* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor);
9828 | MA_API void ma_apply_volume_factor_pcm_frames_s16(ma_int16* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor);
9829 | MA_API void ma_apply_volume_factor_pcm_frames_s24(void* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor);
9830 | MA_API void ma_apply_volume_factor_pcm_frames_s32(ma_int32* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor);
9831 | MA_API void ma_apply_volume_factor_pcm_frames_f32(float* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor);
9832 | MA_API void ma_apply_volume_factor_pcm_frames(void* pFrames, ma_uint64 frameCount, ma_format format,
> ma_uint32 channels, float factor);
9833 |
9834 | MA_API void ma_copy_and_apply_volume_factor_per_channel_f32(float* pFramesOut, const float*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float* pChannelGains);
9835 |
9836 |
9837 | MA_API void ma_copy_and_apply_volume_and_clip_samples_u8(ma_uint8* pDst, const ma_int16* pSrc,
> ma_uint64 count, float volume);
9838 | MA_API void ma_copy_and_apply_volume_and_clip_samples_s16(ma_int16* pDst, const ma_int32* pSrc,
> ma_uint64 count, float volume);
9839 | MA_API void ma_copy_and_apply_volume_and_clip_samples_s24(ma_uint8* pDst, const ma_int64* pSrc,
> ma_uint64 count, float volume);
9840 | MA_API void ma_copy_and_apply_volume_and_clip_samples_s32(ma_int32* pDst, const ma_int64* pSrc,
> ma_uint64 count, float volume);
9841 | MA_API void ma_copy_and_apply_volume_and_clip_samples_f32(float* pDst, const float* pSrc, ma_uint64
> count, float volume);
9842 | MA_API void ma_copy_and_apply_volume_and_clip_pcm_frames(void* pDst, const void* pSrc, ma_uint64
> frameCount, ma_format format, ma_uint32 channels, float volume);
9843 |
9844 |
9845 | /*
9846 | Helper for converting a linear factor to gain in decibels.
9847 | */
9848 | MA_API float ma_volume_linear_to_db(float factor);
9849 |
9850 | /*
9851 | Helper for converting gain in decibels to a linear factor.
9852 | */
9853 | MA_API float ma_volume_db_to_linear(float gain);
9854 |
9855 |
9856 | /*
9857 | Mixes the specified number of frames in floating point format with a volume factor.
9858 |
9859 | This will run on an optimized path when the volume is equal to 1.
9860 | */
9861 | MA_API ma_result ma_mix_pcm_frames_f32(float* pDst, const float* pSrc, ma_uint64 frameCount,
> ma_uint32 channels, float volume);
9862 |
9863 |
9864 |
9865 |
9866 | /*****
> *****/
9867 |
9868 | VFS
9869 | ===
9870 |
9871 | The VFS object (virtual file system) is what's used to customize file access. This is useful in
> cases where stdio FILE* based APIs may not be entirely

```

```

9872 | appropriate for a given situation.
9873 |
9874 | *****
> *****/
9875 | typedef void      ma_vfs;
9876 | typedef ma_handle ma_vfs_file;
9877 |
9878 | typedef enum
9879 | {
9880 |     MA_OPEN_MODE_READ  = 0x00000001,
9881 |     MA_OPEN_MODE_WRITE = 0x00000002
9882 | } ma_open_mode_flags;
9883 |
9884 | typedef enum
9885 | {
9886 |     ma_seek_origin_start,
9887 |     ma_seek_origin_current,
9888 |     ma_seek_origin_end /* Not used by decoders. */
9889 | } ma_seek_origin;
9890 |
9891 | typedef struct
9892 | {
9893 |     ma_uint64 sizeInBytes;
9894 | } ma_file_info;
9895 |
9896 | typedef struct
9897 | {
9898 |     ma_result (* onOpen) (ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode, ma_vfs_file*
> pFile);
9899 |     ma_result (* onOpenW)(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32 openMode, ma_vfs_file*
> pFile);
9900 |     ma_result (* onClose)(ma_vfs* pVFS, ma_vfs_file file);
9901 |     ma_result (* onRead) (ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t sizeInBytes, size_t*
> pBytesRead);
9902 |     ma_result (* onWrite)(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc, size_t sizeInBytes,
> size_t* pBytesWritten);
9903 |     ma_result (* onSeek) (ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset, ma_seek_origin origin);
9904 |     ma_result (* onTell) (ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor);
9905 |     ma_result (* onInfo) (ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo);
9906 | } ma_vfs_callbacks;
9907 |
9908 | MA_API ma_result ma_vfs_open(ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode, ma_vfs_file*
> pFile);
9909 | MA_API ma_result ma_vfs_open_w(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile);
9910 | MA_API ma_result ma_vfs_close(ma_vfs* pVFS, ma_vfs_file file);
9911 | MA_API ma_result ma_vfs_read(ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t sizeInBytes, size_t*
> pBytesRead);
9912 | MA_API ma_result ma_vfs_write(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc, size_t sizeInBytes,
> size_t* pBytesWritten);
9913 | MA_API ma_result ma_vfs_seek(ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset, ma_seek_origin
> origin);
9914 | MA_API ma_result ma_vfs_tell(ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor);
9915 | MA_API ma_result ma_vfs_info(ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo);
9916 | MA_API ma_result ma_vfs_open_and_read_file(ma_vfs* pVFS, const char* pFilePath, void** ppData,
> size_t* pSize, const ma_allocation_callbacks* pAllocationCallbacks);
9917 |
9918 | typedef struct
9919 | {
9920 |     ma_vfs_callbacks cb;
9921 |     ma_allocation_callbacks allocationCallbacks; /* Only used for the wchar_t version of open()
> on non-Windows platforms. */
9922 | } ma_default_vfs;
9923 |
9924 | MA_API ma_result ma_default_vfs_init(ma_default_vfs* pVFS, const ma_allocation_callbacks*
> pAllocationCallbacks);
9925 |
9926 |
9927 |
9928 | typedef ma_result (* ma_read_proc)(void* pUserData, void* pBufferOut, size_t bytesToRead, size_t*
> pBytesRead);
9929 | typedef ma_result (* ma_seek_proc)(void* pUserData, ma_int64 offset, ma_seek_origin origin);
9930 | typedef ma_result (* ma_tell_proc)(void* pUserData, ma_int64* pCursor);
9931 |
9932 |
9933 |
9934 | #if !defined(MA_NO_DECODING) || !defined(MA_NO_ENCODING)
9935 | typedef enum
9936 | {

```

```

9937 |     ma_encoding_format_unknown = 0,
9938 |     ma_encoding_format_wav,
9939 |     ma_encoding_format_flac,
9940 |     ma_encoding_format_mp3,
9941 |     ma_encoding_format_vorbis
9942 | } ma_encoding_format;
9943 | #endif
9944 |
9945 | /*****
> *****/
9946 |
9947 | Decoding
9948 | =====
9949 |
9950 | Decoders are independent of the main device API. Decoding APIs can be called freely inside the
> device's data callback, but they are not thread safe unless
9951 | you do your own synchronization.
9952 |
9953 | *****/
> *****/
9954 | #ifndef MA_NO_DECODING
9955 | typedef struct ma_decoder ma_decoder;
9956 |
9957 |
9958 | typedef struct
9959 | {
9960 |     ma_format preferredFormat;
9961 |     ma_uint32 seekPointCount; /* Set to > 0 to generate a seektable if the decoding backend
> supports it. */
9962 | } ma_decoding_backend_config;
9963 |
9964 | MA_API ma_decoding_backend_config ma_decoding_backend_config_init(ma_format preferredFormat,
> ma_uint32 seekPointCount);
9965 |
9966 |
9967 | typedef struct
9968 | {
9969 |     ma_result (* onInit      )(void* pUserData, ma_read_proc onRead, ma_seek_proc onSeek,
> ma_tell_proc onTell, void* pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_data_source** ppBackend);
9970 |     ma_result (* onInitFile  )(void* pUserData, const char* pFilePath, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend); /* Optional. */
9971 |     ma_result (* onInitFileW )(void* pUserData, const wchar_t* pFilePath, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend); /* Optional. */
9972 |     ma_result (* onInitMemory)(void* pUserData, const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend); /* Optional. */
9973 |     void (* onUninit      )(void* pUserData, ma_data_source* pBackend, const
> ma_allocation_callbacks* pAllocationCallbacks);
9974 | } ma_decoding_backend_vtable;
9975 |
9976 |
9977 | typedef ma_result (* ma_decoder_read_proc)(ma_decoder* pDecoder, void* pBufferOut, size_t
> bytesToRead, size_t* pBytesRead); /* Returns the number of bytes read. */
9978 | typedef ma_result (* ma_decoder_seek_proc)(ma_decoder* pDecoder, ma_int64 byteOffset, ma_seek_origin
> origin);
9979 | typedef ma_result (* ma_decoder_tell_proc)(ma_decoder* pDecoder, ma_int64* pCursor);
9980 |
9981 | typedef struct
9982 | {
9983 |     ma_format format; /* Set to 0 or ma_format_unknown to use the stream's internal format. */
9984 |     ma_uint32 channels; /* Set to 0 to use the stream's internal channels. */
9985 |     ma_uint32 sampleRate; /* Set to 0 to use the stream's internal sample rate. */
9986 |     ma_channel* pChannelMap;
9987 |     ma_channel_mix_mode channelMixMode;
9988 |     ma_dither_mode ditherMode;
9989 |     ma_resampler_config resampling;
9990 |     ma_allocation_callbacks allocationCallbacks;
9991 |     ma_encoding_format encodingFormat;
9992 |     ma_uint32 seekPointCount; /* When set to > 0, specifies the number of seek points to use for
> the generation of a seek table. Not all decoding backends support this. */
9993 |     ma_decoding_backend_vtable** ppCustomBackendVTables;
9994 |     ma_uint32 customBackendCount;
9995 |     void* pCustomBackendUserData;
9996 | } ma_decoder_config;
9997 |
9998 | struct ma_decoder

```

```

9999 | {
10000 |     ma_data_source_base ds;
10001 |     ma_data_source* pBackend; /* The decoding backend we'll be pulling data from.
> */
10002 |     const ma_decoding_backend_vtable* pBackendVTable; /* The vtable for the decoding backend. This
> needs to be stored so we can access the onUninit() callback. */
10003 |     void* pBackendUserData;
10004 |     ma_decoder_read_proc onRead;
10005 |     ma_decoder_seek_proc onSeek;
10006 |     ma_decoder_tell_proc onTell;
10007 |     void* pUserData;
10008 |     ma_uint64 readPointerInPCMFrames; /* In output sample rate. Used for keeping track of how
> many frames are available for decoding. */
10009 |     ma_format outputFormat;
10010 |     ma_uint32 outputChannels;
10011 |     ma_uint32 outputSampleRate;
10012 |     ma_data_converter converter; /* Data conversion is achieved by running frames through this.
> */
10013 |     void* pInputCache; /* In input format. Can be null if it's not needed. */
10014 |     ma_uint64 inputCacheCap; /* The capacity of the input cache. */
10015 |     ma_uint64 inputCacheConsumed; /* The number of frames that have been consumed in the cache.
> Used for determining the next valid frame. */
10016 |     ma_uint64 inputCacheRemaining; /* The number of valid frames remaining in the cache. */
10017 |     ma_allocation_callbacks allocationCallbacks;
10018 |     union
10019 |     {
10020 |         struct
10021 |         {
10022 |             ma_vfs* pVFS;
10023 |             ma_vfs_file file;
10024 |         } vfs;
10025 |         struct
10026 |         {
10027 |             const ma_uint8* pData;
10028 |             size_t dataSize;
10029 |             size_t currentReadPos;
10030 |         } memory; /* Only used for decoders that were opened against a block of
> memory. */
10031 |     } data;
10032 | };
10033 |
10034 | MA_API ma_decoder_config ma_decoder_config_init(ma_format outputFormat, ma_uint32 outputChannels,
> ma_uint32 outputSampleRate);
10035 | MA_API ma_decoder_config ma_decoder_config_init_default(void);
10036 |
10037 | MA_API ma_result ma_decoder_init(ma_decoder_read_proc onRead, ma_decoder_seek_proc onSeek, void*
> pUserData, const ma_decoder_config* pConfig, ma_decoder* pDecoder);
10038 | MA_API ma_result ma_decoder_init_memory(const void* pData, size_t dataSize, const ma_decoder_config*
> pConfig, ma_decoder* pDecoder);
10039 | MA_API ma_result ma_decoder_init_vfs(ma_vfs* pVFS, const char* pFilePath, const ma_decoder_config*
> pConfig, ma_decoder* pDecoder);
10040 | MA_API ma_result ma_decoder_init_vfs_w(ma_vfs* pVFS, const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder);
10041 | MA_API ma_result ma_decoder_init_file(const char* pFilePath, const ma_decoder_config* pConfig,
> ma_decoder* pDecoder);
10042 | MA_API ma_result ma_decoder_init_file_w(const wchar_t* pFilePath, const ma_decoder_config* pConfig,
> ma_decoder* pDecoder);
10043 |
10044 | /*
10045 | Uninitializes a decoder.
10046 | */
10047 | MA_API ma_result ma_decoder_uninit(ma_decoder* pDecoder);
10048 |
10049 | /*
10050 | Reads PCM frames from the given decoder.
10051 |
10052 | This is not thread safe without your own synchronization.
10053 | */
10054 | MA_API ma_result ma_decoder_read_pcm_frames(ma_decoder* pDecoder, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead);
10055 |
10056 | /*
10057 | Seeks to a PCM frame based on its absolute index.
10058 |
10059 | This is not thread safe without your own synchronization.
10060 | */
10061 | MA_API ma_result ma_decoder_seek_to_pcm_frame(ma_decoder* pDecoder, ma_uint64 frameIndex);
10062 |
10063 | /*

```



```

10064 | Retrieves the decoder's output data format.
10065 | */
10066 | MA_API ma_result ma_decoder_get_data_format(ma_decoder* pDecoder, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
10067 |
10068 | /*
10069 | Retrieves the current position of the read cursor in PCM frames.
10070 | */
10071 | MA_API ma_result ma_decoder_get_cursor_in_pcm_frames(ma_decoder* pDecoder, ma_uint64* pCursor);
10072 |
10073 | /*
10074 | Retrieves the length of the decoder in PCM frames.
10075 |
10076 | Do not call this on streams of an undefined length, such as internet radio.
10077 |
10078 | If the length is unknown or an error occurs, 0 will be returned.
10079 |
10080 | This will always return 0 for Vorbis decoders. This is due to a limitation with stb_vorbis in push
> mode which is what miniaudio
10081 | uses internally.
10082 |
10083 | For MP3's, this will decode the entire file. Do not call this in time critical scenarios.
10084 |
10085 | This function is not thread safe without your own synchronization.
10086 | */
10087 | MA_API ma_result ma_decoder_get_length_in_pcm_frames(ma_decoder* pDecoder, ma_uint64* pLength);
10088 |
10089 | /*
10090 | Retrieves the number of frames that can be read before reaching the end.
10091 |
10092 | This calls `ma_decoder_get_length_in_pcm_frames()` so you need to be aware of the rules for that
> function, in
10093 | particular ensuring you do not call it on streams of an undefined length, such as internet radio.
10094 |
10095 | If the total length of the decoder cannot be retrieved, such as with Vorbis decoders,
> `MA_NOT_IMPLEMENTED` will be
10096 | returned.
10097 | */
10098 | MA_API ma_result ma_decoder_get_available_frames(ma_decoder* pDecoder, ma_uint64* pAvailableFrames);
10099 |
10100 | /*
10101 | Helper for opening and decoding a file into a heap allocated block of memory. Free the returned
> pointer with ma_free(). On input,
10102 | pConfig should be set to what you want. On output it will be set to what you got.
10103 | */
10104 | MA_API ma_result ma_decode_from_vfs(ma_vfs* pVFS, const char* pFilePath, ma_decoder_config* pConfig,
> ma_uint64* pFrameCountOut, void** ppPCMFramesOut);
10105 | MA_API ma_result ma_decode_file(const char* pFilePath, ma_decoder_config* pConfig, ma_uint64*
> pFrameCountOut, void** ppPCMFramesOut);
10106 | MA_API ma_result ma_decode_memory(const void* pData, size_t dataSize, ma_decoder_config* pConfig,
> ma_uint64* pFrameCountOut, void** ppPCMFramesOut);
10107 |
10108 | #endif /* MA_NO_DECODING */
10109 |
10110 |
10111 | /*****
> *****/
10112 |
10113 | Encoding
10114 | =====
10115 |
10116 | Encoders do not perform any format conversion for you. If your target format does not support the
> format, and error will be returned.
10117 |
10118 | *****/
> *****/
10119 | #ifndef MA_NO_ENCODING
10120 | typedef struct ma_encoder ma_encoder;
10121 |
10122 | typedef ma_result (* ma_encoder_write_proc) (ma_encoder* pEncoder, const void* pBufferIn,
> size_t bytesToWrite, size_t* pBytesWritten);
10123 | typedef ma_result (* ma_encoder_seek_proc) (ma_encoder* pEncoder, ma_int64 offset,
> ma_seek_origin origin);
10124 | typedef ma_result (* ma_encoder_init_proc) (ma_encoder* pEncoder);
10125 | typedef void (* ma_encoder_uninit_proc) (ma_encoder* pEncoder);
10126 | typedef ma_result (* ma_encoder_write_pcm_frames_proc)(ma_encoder* pEncoder, const void* pFramesIn,
> ma_uint64 frameCount, ma_uint64* pFramesWritten);
10127 |
10128 | typedef struct

```

```

10129 | {
10130 |     ma_encoding_format encodingFormat;
10131 |     ma_format format;
10132 |     ma_uint32 channels;
10133 |     ma_uint32 sampleRate;
10134 |     ma_allocation_callbacks allocationCallbacks;
10135 | } ma_encoder_config;
10136 |
10137 | MA_API ma_encoder_config ma_encoder_config_init(ma_encoding_format encodingFormat, ma_format format,
> ma_uint32 channels, ma_uint32 sampleRate);
10138 |
10139 | struct ma_encoder
10140 | {
10141 |     ma_encoder_config config;
10142 |     ma_encoder_write_proc onWrite;
10143 |     ma_encoder_seek_proc onSeek;
10144 |     ma_encoder_init_proc onInit;
10145 |     ma_encoder_uninit_proc onUninit;
10146 |     ma_encoder_write_pcm_frames_proc onWritePCMFrames;
10147 |     void* pUserData;
10148 |     void* pInternalEncoder;
10149 |     union
10150 |     {
10151 |         struct
10152 |         {
10153 |             ma_vfs* pVFS;
10154 |             ma_vfs_file file;
10155 |         } vfs;
10156 |     } data;
10157 | };
10158 |
10159 | MA_API ma_result ma_encoder_init(ma_encoder_write_proc onWrite, ma_encoder_seek_proc onSeek, void*
> pUserData, const ma_encoder_config* pConfig, ma_encoder* pEncoder);
10160 | MA_API ma_result ma_encoder_init_vfs(ma_vfs* pVFS, const char* pFilePath, const ma_encoder_config*
> pConfig, ma_encoder* pEncoder);
10161 | MA_API ma_result ma_encoder_init_vfs_w(ma_vfs* pVFS, const wchar_t* pFilePath, const
> ma_encoder_config* pConfig, ma_encoder* pEncoder);
10162 | MA_API ma_result ma_encoder_init_file(const char* pFilePath, const ma_encoder_config* pConfig,
> ma_encoder* pEncoder);
10163 | MA_API ma_result ma_encoder_init_file_w(const wchar_t* pFilePath, const ma_encoder_config* pConfig,
> ma_encoder* pEncoder);
10164 | MA_API void ma_encoder_uninit(ma_encoder* pEncoder);
10165 | MA_API ma_result ma_encoder_write_pcm_frames(ma_encoder* pEncoder, const void* pFramesIn, ma_uint64
> frameCount, ma_uint64* pFramesWritten);
10166 |
10167 | #endif /* MA_NO_ENCODING */
10168 |
10169 |
10170 | /*****
> *****/
10171 |
10172 | Generation
10173 |
10174 | *****/
> *****/
10175 | #ifndef MA_NO_GENERATION
10176 | typedef enum
10177 | {
10178 |     ma_waveform_type_sine,
10179 |     ma_waveform_type_square,
10180 |     ma_waveform_type_triangle,
10181 |     ma_waveform_type_sawtooth
10182 | } ma_waveform_type;
10183 |
10184 | typedef struct
10185 | {
10186 |     ma_format format;
10187 |     ma_uint32 channels;
10188 |     ma_uint32 sampleRate;
10189 |     ma_waveform_type type;
10190 |     double amplitude;
10191 |     double frequency;
10192 | } ma_waveform_config;
10193 |
10194 | MA_API ma_waveform_config ma_waveform_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, ma_waveform_type type, double amplitude, double frequency);
10195 |
10196 | typedef struct
10197 | {

```

```

10198 |     ma_data_source_base ds;
10199 |     ma_waveform_config config;
10200 |     double advance;
10201 |     double time;
10202 | } ma_waveform;
10203 |
10204 | MA_API ma_result ma_waveform_init(const ma_waveform_config* pConfig, ma_waveform* pWaveform);
10205 | MA_API void ma_waveform_uninit(ma_waveform* pWaveform);
10206 | MA_API ma_result ma_waveform_read_pcm_frames(ma_waveform* pWaveform, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead);
10207 | MA_API ma_result ma_waveform_seek_to_pcm_frame(ma_waveform* pWaveform, ma_uint64 frameIndex);
10208 | MA_API ma_result ma_waveform_set_amplitude(ma_waveform* pWaveform, double amplitude);
10209 | MA_API ma_result ma_waveform_set_frequency(ma_waveform* pWaveform, double frequency);
10210 | MA_API ma_result ma_waveform_set_type(ma_waveform* pWaveform, ma_waveform_type type);
10211 | MA_API ma_result ma_waveform_set_sample_rate(ma_waveform* pWaveform, ma_uint32 sampleRate);
10212 |
10213 | typedef struct
10214 | {
10215 |     ma_format format;
10216 |     ma_uint32 channels;
10217 |     ma_uint32 sampleRate;
10218 |     double dutyCycle;
10219 |     double amplitude;
10220 |     double frequency;
10221 | } ma_pulsewave_config;
10222 |
10223 | MA_API ma_pulsewave_config ma_pulsewave_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double dutyCycle, double amplitude, double frequency);
10224 |
10225 | typedef struct
10226 | {
10227 |     ma_waveform waveform;
10228 |     ma_pulsewave_config config;
10229 | } ma_pulsewave;
10230 |
10231 | MA_API ma_result ma_pulsewave_init(const ma_pulsewave_config* pConfig, ma_pulsewave* pWaveform);
10232 | MA_API void ma_pulsewave_uninit(ma_pulsewave* pWaveform);
10233 | MA_API ma_result ma_pulsewave_read_pcm_frames(ma_pulsewave* pWaveform, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead);
10234 | MA_API ma_result ma_pulsewave_seek_to_pcm_frame(ma_pulsewave* pWaveform, ma_uint64 frameIndex);
10235 | MA_API ma_result ma_pulsewave_set_amplitude(ma_pulsewave* pWaveform, double amplitude);
10236 | MA_API ma_result ma_pulsewave_set_frequency(ma_pulsewave* pWaveform, double frequency);
10237 | MA_API ma_result ma_pulsewave_set_sample_rate(ma_pulsewave* pWaveform, ma_uint32 sampleRate);
10238 | MA_API ma_result ma_pulsewave_set_duty_cycle(ma_pulsewave* pWaveform, double dutyCycle);
10239 |
10240 | typedef enum
10241 | {
10242 |     ma_noise_type_white,
10243 |     ma_noise_type_pink,
10244 |     ma_noise_type_brownian
10245 | } ma_noise_type;
10246 |
10247 |
10248 | typedef struct
10249 | {
10250 |     ma_format format;
10251 |     ma_uint32 channels;
10252 |     ma_noise_type type;
10253 |     ma_int32 seed;
10254 |     double amplitude;
10255 |     ma_bool32 duplicateChannels;
10256 | } ma_noise_config;
10257 |
10258 | MA_API ma_noise_config ma_noise_config_init(ma_format format, ma_uint32 channels, ma_noise_type
> type, ma_int32 seed, double amplitude);
10259 |
10260 | typedef struct
10261 | {
10262 |     ma_data_source_base ds;
10263 |     ma_noise_config config;
10264 |     ma_lcg lcg;
10265 |     union
10266 |     {
10267 |         struct
10268 |         {
10269 |             double** bin;
10270 |             double* accumulation;
10271 |             ma_uint32* counter;
10272 |         } pink;

```

```

10273 |         struct
10274 |         {
10275 |             double* accumulation;
10276 |         } brownian;
10277 |     } state;
10278 |
10279 |     /* Memory management. */
10280 |     void* _pHeap;
10281 |     ma_bool32 _ownsHeap;
10282 | } ma_noise;
10283 |
10284 | MA_API ma_result ma_noise_get_heap_size(const ma_noise_config* pConfig, size_t* pHeapSizeInBytes);
10285 | MA_API ma_result ma_noise_init_preallocated(const ma_noise_config* pConfig, void* pHeap, ma_noise*
> pNoise);
10286 | MA_API ma_result ma_noise_init(const ma_noise_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_noise* pNoise);
10287 | MA_API void ma_noise_uninit(ma_noise* pNoise, const ma_allocation_callbacks* pAllocationCallbacks);
10288 | MA_API ma_result ma_noise_read_pcm_frames(ma_noise* pNoise, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead);
10289 | MA_API ma_result ma_noise_set_amplitude(ma_noise* pNoise, double amplitude);
10290 | MA_API ma_result ma_noise_set_seed(ma_noise* pNoise, ma_int32 seed);
10291 | MA_API ma_result ma_noise_set_type(ma_noise* pNoise, ma_noise_type type);
10292 |
10293 | #endif /* MA_NO_GENERATION */
10294 |
10295 |
10296 |
10297 | /*****
> *****/
10298 |
10299 | Resource Manager
10300 |
10301 | *****/
> *****/
10302 | /* The resource manager cannot be enabled if there is no decoder. */
10303 | #if !defined(MA_NO_RESOURCE_MANAGER) && defined(MA_NO_DECODING)
10304 | #define MA_NO_RESOURCE_MANAGER
10305 | #endif
10306 |
10307 | #ifndef MA_NO_RESOURCE_MANAGER
10308 | typedef struct ma_resource_manager ma_resource_manager;
10309 | typedef struct ma_resource_manager_data_buffer_node ma_resource_manager_data_buffer_node;
10310 | typedef struct ma_resource_manager_data_buffer ma_resource_manager_data_buffer;
10311 | typedef struct ma_resource_manager_data_stream ma_resource_manager_data_stream;
10312 | typedef struct ma_resource_manager_data_source ma_resource_manager_data_source;
10313 |
10314 | typedef enum
10315 | {
10316 |     MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM = 0x00000001, /* When set, does not load
> the entire data source in memory. Disk I/O will happen on job threads. */
10317 |     MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE = 0x00000002, /* Decode data before
> storing in memory. When set, decoding is done at the resource manager level rather than the mixing
> thread. Results in faster mixing, but higher memory usage. */
10318 |     MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC = 0x00000004, /* When set, the resource
> manager will load the data source asynchronously. */
10319 |     MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT = 0x00000008, /* When set, waits for
> initialization of the underlying data source before returning from
> ma_resource_manager_data_source_init(). */
10320 |     MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_UNKNOWN_LENGTH = 0x00000010, /* Gives the resource
> manager a hint that the length of the data source is unknown and calling
> `ma_data_source_get_length_in_pcm_frames()` should be avoided. */
10321 |     MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING = 0x00000020 /* When set, configures the
> data source to loop by default. */
10322 | } ma_resource_manager_data_source_flags;
10323 |
10324 |
10325 | /*
10326 | Pipeline notifications used by the resource manager. Made up of both an async notification and a
> fence, both of which are optional.
10327 | */
10328 | typedef struct
10329 | {
10330 |     ma_async_notification* pNotification;
10331 |     ma_fence* pFence;
10332 | } ma_resource_manager_pipeline_stage_notification;
10333 |
10334 | typedef struct
10335 | {
10336 |     ma_resource_manager_pipeline_stage_notification init; /* Initialization of the decoder. */

```

```

10337 |     ma_resource_manager_pipeline_stage_notification done;    /* Decoding fully completed. */
10338 | } ma_resource_manager_pipeline_notifications;
10339 |
10340 | MA_API ma_resource_manager_pipeline_notifications
> ma_resource_manager_pipeline_notifications_init(void);
10341 |
10342 |
10343 |
10344 | /* BEGIN BACKWARDS COMPATIBILITY */
10345 | /* TODO: Remove this block in version 0.12. */
10346 | #if 1
10347 | #define ma_resource_manager_job                ma_job
10348 | #define ma_resource_manager_job_init            ma_job_init
10349 | #define MA_JOB_TYPE_RESOURCE_MANAGER_QUEUE_FLAG_NON_BLOCKING MA_JOB_QUEUE_FLAG_NON_BLOCKING
10350 | #define ma_resource_manager_job_queue_config    ma_job_queue_config
10351 | #define ma_resource_manager_job_queue_config_init ma_job_queue_config_init
10352 | #define ma_resource_manager_job_queue           ma_job_queue
10353 | #define ma_resource_manager_job_queue_get_heap_size ma_job_queue_get_heap_size
10354 | #define ma_resource_manager_job_queue_init_preallocated ma_job_queue_init_preallocated
10355 | #define ma_resource_manager_job_queue_init      ma_job_queue_init
10356 | #define ma_resource_manager_job_queue_uninit    ma_job_queue_uninit
10357 | #define ma_resource_manager_job_queue_post     ma_job_queue_post
10358 | #define ma_resource_manager_job_queue_next     ma_job_queue_next
10359 | #endif
10360 | /* END BACKWARDS COMPATIBILITY */
10361 |
10362 |
10363 |
10364 |
10365 | /* Maximum job thread count will be restricted to this, but this may be removed later and replaced
> with a heap allocation thereby removing any limitation. */
10366 | #ifndef MA_RESOURCE_MANAGER_MAX_JOB_THREAD_COUNT
10367 | #define MA_RESOURCE_MANAGER_MAX_JOB_THREAD_COUNT    64
10368 | #endif
10369 |
10370 | typedef enum
10371 | {
10372 |     /* Indicates ma_resource_manager_next_job() should not block. Only valid when the job thread
> count is 0. */
10373 |     MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING = 0x00000001,
10374 |
10375 |     /* Disables any kind of multithreading. Implicitly enables
> MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING. */
10376 |     MA_RESOURCE_MANAGER_FLAG_NO_THREADING = 0x00000002
10377 | } ma_resource_manager_flags;
10378 |
10379 | typedef struct
10380 | {
10381 |     const char* pFilePath;
10382 |     const wchar_t* pFilePathW;
10383 |     const ma_resource_manager_pipeline_notifications* pNotifications;
10384 |     ma_uint64 initialSeekPointInPCMFrames;
10385 |     ma_uint64 rangeBegInPCMFrames;
10386 |     ma_uint64 rangeEndInPCMFrames;
10387 |     ma_uint64 loopPointBegInPCMFrames;
10388 |     ma_uint64 loopPointEndInPCMFrames;
10389 |     ma_uint32 flags;
10390 |     ma_bool32 isLooping;    /* Deprecated. Use the MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING flag
> in `flags` instead. */
10391 | } ma_resource_manager_data_source_config;
10392 |
10393 | MA_API ma_resource_manager_data_source_config ma_resource_manager_data_source_config_init(void);
10394 |
10395 |
10396 | typedef enum
10397 | {
10398 |     ma_resource_manager_data_supply_type_unknown = 0,    /* Used for determining whether or the data
> supply has been initialized. */
10399 |     ma_resource_manager_data_supply_type_encoded,        /* Data supply is an encoded buffer.
> Connector is ma_decoder. */
10400 |     ma_resource_manager_data_supply_type_decoded,        /* Data supply is a decoded buffer.
> Connector is ma_audio_buffer. */
10401 |     ma_resource_manager_data_supply_type_decoded_paged  /* Data supply is a linked list of decoded
> buffers. Connector is ma_paged_audio_buffer. */
10402 | } ma_resource_manager_data_supply_type;
10403 |
10404 | typedef struct
10405 | {
10406 |     MA_ATOMIC(4, ma_resource_manager_data_supply_type) type;    /* Read and written from different

```

```

> threads so needs to be accessed atomically. */
10407 |     union
10408 |     {
10409 |         struct
10410 |         {
10411 |             const void* pData;
10412 |             size_t sizeInBytes;
10413 |         } encoded;
10414 |         struct
10415 |         {
10416 |             const void* pData;
10417 |             ma_uint64 totalFrameCount;
10418 |             ma_uint64 decodedFrameCount;
10419 |             ma_format format;
10420 |             ma_uint32 channels;
10421 |             ma_uint32 sampleRate;
10422 |         } decoded;
10423 |         struct
10424 |         {
10425 |             ma_paged_audio_buffer_data data;
10426 |             ma_uint64 decodedFrameCount;
10427 |             ma_uint32 sampleRate;
10428 |         } decodedPaged;
10429 |     } backend;
10430 | } ma_resource_manager_data_supply;
10431 |
10432 | struct ma_resource_manager_data_buffer_node
10433 | {
10434 |     ma_uint32 hashedName32; /* The hashed name. This is the key. */
10435 |     ma_uint32 refCount;
10436 |     MA_ATOMIC(4, ma_result) result; /* Result from asynchronous loading. When
> loading set to MA_BUSY. When fully loaded set to MA_SUCCESS. When deleting set to MA_UNAVAILABLE. */
10437 |     MA_ATOMIC(4, ma_uint32) executionCounter; /* For allocating execution orders for jobs. */
10438 |     MA_ATOMIC(4, ma_uint32) executionPointer; /* For managing the order of execution for
> asynchronous jobs relating to this object. Incremented as jobs complete processing. */
10439 |     ma_bool32 isDataOwnedByResourceManager; /* Set to true when the underlying data buffer
> was allocated the resource manager. Set to false if it is owned by the application (via
> ma_resource_manager_register_*()). */
10440 |     ma_resource_manager_data_supply data;
10441 |     ma_resource_manager_data_buffer_node* pParent;
10442 |     ma_resource_manager_data_buffer_node* pChildLo;
10443 |     ma_resource_manager_data_buffer_node* pChildHi;
10444 | };
10445 |
10446 | struct ma_resource_manager_data_buffer
10447 | {
10448 |     ma_data_source_base ds; /* Base data source. A data buffer is a data
> source. */
10449 |     ma_resource_manager* pResourceManager; /* A pointer to the resource manager that owns
> this buffer. */
10450 |     ma_resource_manager_data_buffer_node* pNode; /* The data node. This is reference counted and
> is what supplies the data. */
10451 |     ma_uint32 flags; /* The flags that were passed used to initialize
> the buffer. */
10452 |     MA_ATOMIC(4, ma_uint32) executionCounter; /* For allocating execution orders for jobs. */
10453 |     MA_ATOMIC(4, ma_uint32) executionPointer; /* For managing the order of execution for
> asynchronous jobs relating to this object. Incremented as jobs complete processing. */
10454 |     ma_uint64 seekTargetInPCMFrames; /* Only updated by the public API. Never written
> nor read from the job thread. */
10455 |     ma_bool32 seekToCursorOnNextRead; /* On the next read we need to seek to the frame
> cursor. */
10456 |     MA_ATOMIC(4, ma_result) result; /* Keeps track of a result of decoding. Set to
> MA_BUSY while the buffer is still loading. Set to MA_SUCCESS when loading is finished successfully.
> Otherwise set to some other code. */
10457 |     MA_ATOMIC(4, ma_bool32) isLooping; /* Can be read and written by different threads
> at the same time. Must be used atomically. */
10458 |     ma_atomic_bool32 isConnectorInitialized; /* Used for asynchronous loading to ensure we
> don't try to initialize the connector multiple times while waiting for the node to fully load. */
10459 |     union
10460 |     {
10461 |         ma_decoder decoder; /* Supply type is
> ma_resource_manager_data_supply_type_encoded */
10462 |         ma_audio_buffer buffer; /* Supply type is
> ma_resource_manager_data_supply_type_decoded */
10463 |         ma_paged_audio_buffer pagedBuffer; /* Supply type is
> ma_resource_manager_data_supply_type_decoded_paged */
10464 |     } connector; /* Connects this object to the node's data supply. */
10465 | };
10466 |

```

```

10467 | struct ma_resource_manager_data_stream
10468 | {
10469 |     ma_data_source_base ds;                /* Base data source. A data stream is a data source.
> */
10470 |     ma_resource_manager* pResourceManager; /* A pointer to the resource manager that owns this
> data stream. */
10471 |     ma_uint32 flags;                       /* The flags that were passed used to initialize the
> stream. */
10472 |     ma_decoder decoder;                    /* Used for filling pages with data. This is only
> ever accessed by the job thread. The public API should never touch this. */
10473 |     ma_bool32 isDecoderInitialized;        /* Required for determining whether or not the
> decoder should be uninitialized in MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_STREAM. */
10474 |     ma_uint64 totalLengthInPCMFrames;      /* This is calculated when first loaded by the
> MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_STREAM. */
10475 |     ma_uint32 relativeCursor;              /* The playback cursor, relative to the current
> page. Only ever accessed by the public API. Never accessed by the job thread. */
10476 |     MA_ATOMIC(8, ma_uint64) absoluteCursor; /* The playback cursor, in absolute position
> starting from the start of the file. */
10477 |     ma_uint32 currentPageIndex;            /* Toggles between 0 and 1. Index 0 is the first
> half of pPageData. Index 1 is the second half. Only ever accessed by the public API. Never accessed
> by the job thread. */
10478 |     MA_ATOMIC(4, ma_uint32) executionCounter; /* For allocating execution orders for jobs. */
10479 |     MA_ATOMIC(4, ma_uint32) executionPointer; /* For managing the order of execution for
> asynchronous jobs relating to this object. Incremented as jobs complete processing. */
10480 |
10481 |     /* Written by the public API, read by the job thread. */
10482 |     MA_ATOMIC(4, ma_bool32) isLooping;      /* Whether or not the stream is looping. It's
> important to set the looping flag at the data stream level for smooth loop transitions. */
10483 |
10484 |     /* Written by the job thread, read by the public API. */
10485 |     void* pPageData;                       /* Buffer containing the decoded data of each page.
> Allocated once at initialization time. */
10486 |     MA_ATOMIC(4, ma_uint32) pageFrameCount[2]; /* The number of valid PCM frames in each page. Used
> to determine the last valid frame. */
10487 |
10488 |     /* Written and read by both the public API and the job thread. These must be atomic. */
10489 |     MA_ATOMIC(4, ma_result) result;         /* Result from asynchronous loading. When loading
> set to MA_BUSY. When initialized set to MA_SUCCESS. When deleting set to MA_UNAVAILABLE. If an error
> occurs when loading, set to an error code. */
10490 |     MA_ATOMIC(4, ma_bool32) isDecoderAtEnd; /* Whether or not the decoder has reached the end.
> */
10491 |     MA_ATOMIC(4, ma_bool32) isPageValid[2]; /* Booleans to indicate whether or not a page is
> valid. Set to false by the public API, set to true by the job thread. Set to false as the pages are
> consumed, true when they are filled. */
10492 |     MA_ATOMIC(4, ma_bool32) seekCounter;    /* When 0, no seeking is being performed. When > 0,
> a seek is being performed and reading should be delayed with MA_BUSY. */
10493 | };
10494 |
10495 | struct ma_resource_manager_data_source
10496 | {
10497 |     union
10498 |     {
10499 |         ma_resource_manager_data_buffer buffer;
10500 |         ma_resource_manager_data_stream stream;
10501 |     } backend; /* Must be the first item because we need the first item to be the data source
> callbacks for the buffer or stream. */
10502 |
10503 |     ma_uint32 flags;                       /* The flags that were passed in to
> ma_resource_manager_data_source_init(). */
10504 |     MA_ATOMIC(4, ma_uint32) executionCounter; /* For allocating execution orders for jobs. */
10505 |     MA_ATOMIC(4, ma_uint32) executionPointer; /* For managing the order of execution for
> asynchronous jobs relating to this object. Incremented as jobs complete processing. */
10506 | };
10507 |
10508 | typedef struct
10509 | {
10510 |     ma_allocation_callbacks allocationCallbacks;
10511 |     ma_log* pLog;
10512 |     ma_format decodedFormat;               /* The decoded format to use. Set to ma_format_unknown (default)
> to use the file's native format. */
10513 |     ma_uint32 decodedChannels;              /* The decoded channel count to use. Set to 0 (default) to use
> the file's native channel count. */
10514 |     ma_uint32 decodedSampleRate;           /* the decoded sample rate to use. Set to 0 (default) to use the
> file's native sample rate. */
10515 |     ma_uint32 jobThreadCount;              /* Set to 0 if you want to self-manage your job threads.
> Defaults to 1. */
10516 |     size_t jobThreadStackSize;
10517 |     ma_uint32 jobQueueCapacity;             /* The maximum number of jobs that can fit in the queue at a
> time. Defaults to MA_JOB_TYPE_RESOURCE_MANAGER_QUEUE_CAPACITY. Cannot be zero. */

```



```

10518 |     ma_uint32 flags;
10519 |     ma_vfs* pVFS; /* Can be NULL in which case defaults will be used. */
10520 |     ma_decoding_backend_vtable** ppCustomDecodingBackendVTables;
10521 |     ma_uint32 customDecodingBackendCount;
10522 |     void* pCustomDecodingBackendUserData;
10523 |     ma_resampler_config resampling;
10524 | } ma_resource_manager_config;
10525 |
10526 | MA_API ma_resource_manager_config ma_resource_manager_config_init(void);
10527 |
10528 | struct ma_resource_manager
10529 | {
10530 |     ma_resource_manager_config config;
10531 |     ma_resource_manager_data_buffer_node* pRootDataBufferNode; /* The root buffer in the binary
> tree. */
10532 | #ifndef MA_NO_THREADING
10533 |     ma_mutex dataBufferBSTLock; /* For synchronizing access to
> the data buffer binary tree. */
10534 |     ma_thread jobThreads[MA_RESOURCE_MANAGER_MAX_JOB_THREAD_COUNT]; /* The threads for executing
> jobs. */
10535 | #endif
10536 |     ma_job_queue jobQueue; /* Multi-consumer,
> multi-producer job queue for managing jobs for asynchronous decoding and streaming. */
10537 |     ma_default_vfs defaultVFS; /* Only used if a custom VFS is
> not specified. */
10538 |     ma_log log; /* Only used if no log was
> specified in the config. */
10539 | };
10540 |
10541 | /* Init. */
10542 | MA_API ma_result ma_resource_manager_init(const ma_resource_manager_config* pConfig,
> ma_resource_manager* pResourceManager);
10543 | MA_API void ma_resource_manager_uninit(ma_resource_manager* pResourceManager);
10544 | MA_API ma_log* ma_resource_manager_get_log(ma_resource_manager* pResourceManager);
10545 |
10546 | /* Registration. */
10547 | MA_API ma_result ma_resource_manager_register_file(ma_resource_manager* pResourceManager, const
> char* pFilePath, ma_uint32 flags);
10548 | MA_API ma_result ma_resource_manager_register_file_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath, ma_uint32 flags);
10549 | MA_API ma_result ma_resource_manager_register_decoded_data(ma_resource_manager* pResourceManager,
> const char* pName, const void* pData, ma_uint64 frameCount, ma_format format, ma_uint32 channels,
> ma_uint32 sampleRate); /* Does not copy. Increments the reference count if already exists and
> returns MA_SUCCESS. */
10550 | MA_API ma_result ma_resource_manager_register_decoded_data_w(ma_resource_manager* pResourceManager,
> const wchar_t* pName, const void* pData, ma_uint64 frameCount, ma_format format, ma_uint32 channels,
> ma_uint32 sampleRate);
10551 | MA_API ma_result ma_resource_manager_register_encoded_data(ma_resource_manager* pResourceManager,
> const char* pName, const void* pData, size_t sizeInBytes); /* Does not copy. Increments the
> reference count if already exists and returns MA_SUCCESS. */
10552 | MA_API ma_result ma_resource_manager_register_encoded_data_w(ma_resource_manager* pResourceManager,
> const wchar_t* pName, const void* pData, size_t sizeInBytes);
10553 | MA_API ma_result ma_resource_manager_unregister_file(ma_resource_manager* pResourceManager, const
> char* pFilePath);
10554 | MA_API ma_result ma_resource_manager_unregister_file_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath);
10555 | MA_API ma_result ma_resource_manager_unregister_data(ma_resource_manager* pResourceManager, const
> char* pName);
10556 | MA_API ma_result ma_resource_manager_unregister_data_w(ma_resource_manager* pResourceManager, const
> wchar_t* pName);
10557 |
10558 | /* Data Buffers. */
10559 | MA_API ma_result ma_resource_manager_data_buffer_init_ex(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_buffer*
> pDataBuffer);
10560 | MA_API ma_result ma_resource_manager_data_buffer_init(ma_resource_manager* pResourceManager, const
> char* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
> ma_resource_manager_data_buffer* pDataBuffer);
10561 | MA_API ma_result ma_resource_manager_data_buffer_init_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications*
> pNotifications, ma_resource_manager_data_buffer* pDataBuffer);
10562 | MA_API ma_result ma_resource_manager_data_buffer_init_copy(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_buffer* pExistingDataBuffer, ma_resource_manager_data_buffer*
> pDataBuffer);
10563 | MA_API ma_result ma_resource_manager_data_buffer_uninit(ma_resource_manager_data_buffer*
> pDataBuffer);
10564 | MA_API ma_result ma_resource_manager_data_buffer_read_pcm_frames(ma_resource_manager_data_buffer*
> pDataBuffer, void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead);
10565 | MA_API ma_result ma_resource_manager_data_buffer_seek_to_pcm_frame(ma_resource_manager_data_buffer*

```

```

    > pBuffer, ma_uint64 frameIndex);
10566 | MA_API ma_result ma_resource_manager_data_buffer_get_data_format(ma_resource_manager_data_buffer*
    > pBuffer, ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel*
    > pChannelMap, size_t channelMapCap);
10567 | MA_API ma_result
    > ma_resource_manager_data_buffer_get_cursor_in_pcm_frames(ma_resource_manager_data_buffer*
    > pBuffer, ma_uint64* pCursor);
10568 | MA_API ma_result
    > ma_resource_manager_data_buffer_get_length_in_pcm_frames(ma_resource_manager_data_buffer*
    > pBuffer, ma_uint64* pLength);
10569 | MA_API ma_result ma_resource_manager_data_buffer_result(const ma_resource_manager_data_buffer*
    > pBuffer);
10570 | MA_API ma_result ma_resource_manager_data_buffer_set_looping(ma_resource_manager_data_buffer*
    > pBuffer, ma_bool32 isLooping);
10571 | MA_API ma_bool32 ma_resource_manager_data_buffer_is_looping(const ma_resource_manager_data_buffer*
    > pBuffer);
10572 | MA_API ma_result
    > ma_resource_manager_data_buffer_get_available_frames(ma_resource_manager_data_buffer* pBuffer,
    > ma_uint64* pAvailableFrames);
10573 |
10574 | /* Data Streams. */
10575 | MA_API ma_result ma_resource_manager_data_stream_init_ex(ma_resource_manager* pResourceManager,
    > const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_stream*
    > pDataStream);
10576 | MA_API ma_result ma_resource_manager_data_stream_init(ma_resource_manager* pResourceManager, const
    > char* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
    > ma_resource_manager_data_stream* pDataStream);
10577 | MA_API ma_result ma_resource_manager_data_stream_init_w(ma_resource_manager* pResourceManager, const
    > wchar_t* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications*
    > pNotifications, ma_resource_manager_data_stream* pDataStream);
10578 | MA_API ma_result ma_resource_manager_data_stream_uninit(ma_resource_manager_data_stream*
    > pDataStream);
10579 | MA_API ma_result ma_resource_manager_data_stream_read_pcm_frames(ma_resource_manager_data_stream*
    > pDataStream, void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead);
10580 | MA_API ma_result ma_resource_manager_data_stream_seek_to_pcm_frame(ma_resource_manager_data_stream*
    > pDataStream, ma_uint64 frameIndex);
10581 | MA_API ma_result ma_resource_manager_data_stream_get_data_format(ma_resource_manager_data_stream*
    > pDataStream, ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel*
    > pChannelMap, size_t channelMapCap);
10582 | MA_API ma_result
    > ma_resource_manager_data_stream_get_cursor_in_pcm_frames(ma_resource_manager_data_stream*
    > pDataStream, ma_uint64* pCursor);
10583 | MA_API ma_result
    > ma_resource_manager_data_stream_get_length_in_pcm_frames(ma_resource_manager_data_stream*
    > pDataStream, ma_uint64* pLength);
10584 | MA_API ma_result ma_resource_manager_data_stream_result(const ma_resource_manager_data_stream*
    > pDataStream);
10585 | MA_API ma_result ma_resource_manager_data_stream_set_looping(ma_resource_manager_data_stream*
    > pDataStream, ma_bool32 isLooping);
10586 | MA_API ma_bool32 ma_resource_manager_data_stream_is_looping(const ma_resource_manager_data_stream*
    > pDataStream);
10587 | MA_API ma_result
    > ma_resource_manager_data_stream_get_available_frames(ma_resource_manager_data_stream* pDataStream,
    > ma_uint64* pAvailableFrames);
10588 |
10589 | /* Data Sources. */
10590 | MA_API ma_result ma_resource_manager_data_source_init_ex(ma_resource_manager* pResourceManager,
    > const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_source*
    > pDataSource);
10591 | MA_API ma_result ma_resource_manager_data_source_init(ma_resource_manager* pResourceManager, const
    > char* pName, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
    > ma_resource_manager_data_source* pDataSource);
10592 | MA_API ma_result ma_resource_manager_data_source_init_w(ma_resource_manager* pResourceManager, const
    > wchar_t* pName, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
    > ma_resource_manager_data_source* pDataSource);
10593 | MA_API ma_result ma_resource_manager_data_source_init_copy(ma_resource_manager* pResourceManager,
    > const ma_resource_manager_data_source* pExistingDataSource, ma_resource_manager_data_source*
    > pDataSource);
10594 | MA_API ma_result ma_resource_manager_data_source_uninit(ma_resource_manager_data_source*
    > pDataSource);
10595 | MA_API ma_result ma_resource_manager_data_source_read_pcm_frames(ma_resource_manager_data_source*
    > pDataSource, void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead);
10596 | MA_API ma_result ma_resource_manager_data_source_seek_to_pcm_frame(ma_resource_manager_data_source*
    > pDataSource, ma_uint64 frameIndex);
10597 | MA_API ma_result ma_resource_manager_data_source_get_data_format(ma_resource_manager_data_source*
    > pDataSource, ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel*
    > pChannelMap, size_t channelMapCap);
10598 | MA_API ma_result
    > ma_resource_manager_data_source_get_cursor_in_pcm_frames(ma_resource_manager_data_source*

```

```

> pDataSource, ma_uint64* pCursor);
10599 | MA_API ma_result
> ma_resource_manager_data_source_get_length_in_pcm_frames(ma_resource_manager_data_source*
> pDataSource, ma_uint64* pLength);
10600 | MA_API ma_result ma_resource_manager_data_source_result(const ma_resource_manager_data_source*
> pDataSource);
10601 | MA_API ma_result ma_resource_manager_data_source_set_looping(ma_resource_manager_data_source*
> pDataSource, ma_bool32 isLooping);
10602 | MA_API ma_bool32 ma_resource_manager_data_source_is_looping(const ma_resource_manager_data_source*
> pDataSource);
10603 | MA_API ma_result
> ma_resource_manager_data_source_get_available_frames(ma_resource_manager_data_source* pDataSource,
> ma_uint64* pAvailableFrames);
10604 |
10605 | /* Job management. */
10606 | MA_API ma_result ma_resource_manager_post_job(ma_resource_manager* pResourceManager, const ma_job*
> pJob);
10607 | MA_API ma_result ma_resource_manager_post_job_quit(ma_resource_manager* pResourceManager); /*
> Helper for posting a quit job. */
10608 | MA_API ma_result ma_resource_manager_next_job(ma_resource_manager* pResourceManager, ma_job* pJob);
10609 | MA_API ma_result ma_resource_manager_process_job(ma_resource_manager* pResourceManager, ma_job*
> pJob); /* DEPRECATED. Use ma_job_process(). Will be removed in version 0.12. */
10610 | MA_API ma_result ma_resource_manager_process_next_job(ma_resource_manager* pResourceManager); /*
> Returns MA_CANCELLED if a MA_JOB_TYPE_QUIT job is found. In non-blocking mode, returns
> MA_NO_DATA_AVAILABLE if no jobs are available. */
10611 | #endif /* MA_NO_RESOURCE_MANAGER */
10612 |
10613 |
10614 |
10615 | /*****
> *****/
10616 |
10617 | Node Graph
10618 |
10619 | *****/
> *****/
10620 | #ifndef MA_NO_NODE_GRAPH
10621 | /* Must never exceed 254. */
10622 | #ifndef MA_MAX_NODE_BUS_COUNT
10623 | #define MA_MAX_NODE_BUS_COUNT 254
10624 | #endif
10625 |
10626 | /* Used internally by miniaudio for memory management. Must never exceed MA_MAX_NODE_BUS_COUNT. */
10627 | #ifndef MA_MAX_NODE_LOCAL_BUS_COUNT
10628 | #define MA_MAX_NODE_LOCAL_BUS_COUNT 2
10629 | #endif
10630 |
10631 | /* Use this when the bus count is determined by the node instance rather than the vtable. */
10632 | #define MA_NODE_BUS_COUNT_UNKNOWN 255
10633 |
10634 |
10635 | /* For some internal memory management of ma_node_graph. */
10636 | typedef struct
10637 | {
10638 |     size_t offset;
10639 |     size_t sizeInBytes;
10640 |     unsigned char _data[1];
10641 | } ma_stack;
10642 |
10643 |
10644 | typedef struct ma_node_graph ma_node_graph;
10645 | typedef void ma_node;
10646 |
10647 |
10648 | /* Node flags. */
10649 | typedef enum
10650 | {
10651 |     MA_NODE_FLAG_PASSTHROUGH = 0x00000001,
10652 |     MA_NODE_FLAG_CONTINUOUS_PROCESSING = 0x00000002,
10653 |     MA_NODE_FLAG_ALLOW_NULL_INPUT = 0x00000004,
10654 |     MA_NODE_FLAG_DIFFERENT_PROCESSING_RATES = 0x00000008,
10655 |     MA_NODE_FLAG_SILENT_OUTPUT = 0x00000010
10656 | } ma_node_flags;
10657 |
10658 |
10659 | /* The playback state of a node. Either started or stopped. */
10660 | typedef enum
10661 | {
10662 |     ma_node_state_started = 0,

```

```

10663 |     ma_node_state_stopped = 1
10664 | } ma_node_state;
10665 |
10666 |
10667 | typedef struct
10668 | {
10669 |     /*
10670 |     Extended processing callback. This callback is used for effects that process input and output
10671 |     at different rates (i.e. they perform resampling). This is similar to the simple version, only
10672 |     they take two separate frame counts: one for input, and one for output.
10673 |
10674 |     On input, `pFrameCountOut` is equal to the capacity of the output buffer for each bus, whereas
10675 |     `pFrameCountIn` will be equal to the number of PCM frames in each of the buffers in
10676 |     > `ppFramesIn`.
10677 |
10678 |     On output, set `pFrameCountOut` to the number of PCM frames that were actually output and set
10679 |     `pFrameCountIn` to the number of input frames that were consumed.
10680 |     */
10681 |     void (* onProcess)(ma_node* pNode, const float** ppFramesIn, ma_uint32* pFrameCountIn, float**
10682 |     > ppFramesOut, ma_uint32* pFrameCountOut);
10683 |
10684 |     /*
10685 |     A callback for retrieving the number of input frames that are required to output the
10686 |     specified number of output frames. You would only want to implement this when the node performs
10687 |     resampling. This is optional, even for nodes that perform resampling, but it does offer a
10688 |     small reduction in latency as it allows miniaudio to calculate the exact number of input frames
10689 |     to read at a time instead of having to estimate.
10690 |     */
10691 |     ma_result (* onGetRequiredInputFrameCount)(ma_node* pNode, ma_uint32 outputFrameCount,
10692 |     > ma_uint32* pInputFrameCount);
10693 |
10694 |     /*
10695 |     The number of input buses. This is how many sub-buffers will be contained in the `ppFramesIn`
10696 |     parameters of the callbacks above.
10697 |     */
10698 |     ma_uint8 inputBusCount;
10699 |
10700 |     /*
10701 |     The number of output buses. This is how many sub-buffers will be contained in the `ppFramesOut`
10702 |     parameters of the callbacks above.
10703 |     */
10704 |     ma_uint8 outputBusCount;
10705 |
10706 |     /*
10707 |     Flags describing characteristics of the node. This is currently just a placeholder for some
10708 |     ideas for later on.
10709 |     */
10710 |     ma_uint32 flags;
10711 | } ma_node_vtable;
10712 |
10713 | typedef struct
10714 | {
10715 |     const ma_node_vtable* vtable;          /* Should never be null. Initialization of the node will
10716 |     > fail if so. */
10717 |     ma_node_state initialState;            /* Defaults to ma_node_state_started. */
10718 |     ma_uint32 inputBusCount;                /* Only used if the vtable specifies an input bus count of
10719 |     > `MA_NODE_BUS_COUNT_UNKNOWN`, otherwise must be set to `MA_NODE_BUS_COUNT_UNKNOWN` (default). */
10720 |     ma_uint32 outputBusCount;              /* Only used if the vtable specifies an output bus count of
10721 |     > `MA_NODE_BUS_COUNT_UNKNOWN`, otherwise be set to `MA_NODE_BUS_COUNT_UNKNOWN` (default). */
10722 |     const ma_uint32* pInputChannels;        /* The number of elements are determined by the input bus
10723 |     > count as determined by the vtable, or `inputBusCount` if the vtable specifies
10724 |     > `MA_NODE_BUS_COUNT_UNKNOWN`. */
10725 |     const ma_uint32* pOutputChannels;       /* The number of elements are determined by the output bus
10726 |     > count as determined by the vtable, or `outputBusCount` if the vtable specifies
10727 |     > `MA_NODE_BUS_COUNT_UNKNOWN`. */
10728 | } ma_node_config;
10729 |
10730 | MA_API ma_node_config ma_node_config_init(void);
10731 |
10732 | /*
10733 | A node has multiple output buses. An output bus is attached to an input bus as an item in a linked
10734 | list. Think of the input bus as a linked list, with the output bus being an item in that list.
10735 | */
10736 | typedef struct ma_node_output_bus ma_node_output_bus;
10737 | struct ma_node_output_bus
10738 | {
10739 |     /* Immutable. */
10740 |     ma_node* pNode;
10741 |     /* The node that owns this output bus.

```

```

> The input node. Will be null for dummy head and tail nodes. */
10732 |     ma_uint8 outputBusIndex;                                /* The index of the output bus on pNode
> that this output bus represents. */
10733 |     ma_uint8 channels;                                       /* The number of channels in the audio
> stream for this bus. */
10734 |
10735 |     /* Mutable via multiple threads. Must be used atomically. The weird ordering here is for packing
> reasons. */
10736 |     ma_uint8 inputNodeInputBusIndex;                        /* The index of the input bus on the
> input. Required for detaching. Will only be used within the spinlock so does not need to be atomic.
> */
10737 |     MA_ATOMIC(4, ma_uint32) flags;                          /* Some state flags for tracking the
> read state of the output buffer. A combination of MA_NODE_OUTPUT_BUS_FLAG_*. */
10738 |     MA_ATOMIC(4, ma_uint32) refCount;                       /* Reference count for some
> thread-safety when detaching. */
10739 |     MA_ATOMIC(4, ma_bool32) isAttached;                     /* This is used to prevent iteration of
> nodes that are in the middle of being detached. Used for thread safety. */
10740 |     MA_ATOMIC(4, ma_spinlock) lock;                         /* Unfortunate lock, but significantly
> simplifies the implementation. Required for thread-safe attaching and detaching. */
10741 |     MA_ATOMIC(4, float) volume;                             /* Linear. */
10742 |     MA_ATOMIC(MA_SIZEOF_PTR, ma_node_output_bus*) pNext;   /* If null, it's the tail node or
> detached. */
10743 |     MA_ATOMIC(MA_SIZEOF_PTR, ma_node_output_bus*) pPrev;   /* If null, it's the head node or
> detached. */
10744 |     MA_ATOMIC(MA_SIZEOF_PTR, ma_node*) pInputNode;         /* The node that this output bus is
> attached to. Required for detaching. */
10745 | };
10746 |
10747 | /*
10748 | A node has multiple input buses. The output buses of a node are connecting to the input busses of
10749 | another. An input bus is essentially just a linked list of output buses.
10750 | */
10751 | typedef struct ma_node_input_bus ma_node_input_bus;
10752 | struct ma_node_input_bus
10753 | {
10754 |     /* Mutable via multiple threads. */
10755 |     ma_node_output_bus head;                                /* Dummy head node for simplifying some lock-free
> thread-safety stuff. */
10756 |     MA_ATOMIC(4, ma_uint32) nextCounter;                    /* This is used to determine whether or not the input
> bus is finding the next node in the list. Used for thread safety when detaching output buses. */
10757 |     MA_ATOMIC(4, ma_spinlock) lock;                        /* Unfortunate lock, but significantly simplifies the
> implementation. Required for thread-safe attaching and detaching. */
10758 |
10759 |     /* Set once at startup. */
10760 |     ma_uint8 channels;                                       /* The number of channels in the audio stream for this
> bus. */
10761 | };
10762 |
10763 |
10764 | typedef struct ma_node_base ma_node_base;
10765 | struct ma_node_base
10766 | {
10767 |     /* These variables are set once at startup. */
10768 |     ma_node_graph* pNodeGraph;                             /* The graph this node belongs to. */
10769 |     const ma_node_vtable* vtable;
10770 |     ma_uint32 inputBusCount;
10771 |     ma_uint32 outputBusCount;
10772 |     ma_node_input_bus* pInputBuses;
10773 |     ma_node_output_bus* pOutputBuses;
10774 |     float* pCachedData;                                     /* Allocated on the heap. Fixed size. Needs to be
> stored on the heap because reading from output buses is done in separate function calls. */
10775 |     ma_uint16 cachedDataCapInFramesPerBus;                 /* The capacity of the input data cache in frames,
> per bus. */
10776 |
10777 |     /* These variables are read and written only from the audio thread. */
10778 |     ma_uint16 cachedFrameCountOut;
10779 |     ma_uint16 cachedFrameCountIn;
10780 |     ma_uint16 consumedFrameCountIn;
10781 |
10782 |     /* These variables are read and written between different threads. */
10783 |     MA_ATOMIC(4, ma_node_state) state;                     /* When set to stopped, nothing will be read,
> regardless of the times in stateTimes. */
10784 |     MA_ATOMIC(8, ma_uint64) stateTimes[2];                 /* Indexed by ma_node_state. Specifies the time
> based on the global clock that a node should be considered to be in the relevant state. */
10785 |     MA_ATOMIC(8, ma_uint64) localTime;                     /* The node's local clock. This is just a running
> sum of the number of output frames that have been processed. Can be modified by any thread with
> `ma_node_set_time()`. */
10786 |
10787 |     /* Memory management. */

```

```

10788 |     ma_node_input_bus _inputBuses[MA_MAX_NODE_LOCAL_BUS_COUNT];
10789 |     ma_node_output_bus _outputBuses[MA_MAX_NODE_LOCAL_BUS_COUNT];
10790 |     void* _pHeap;    /* A heap allocation for internal use only. pInputBuses and/or pOutputBuses will
> point to this if the bus count exceeds MA_MAX_NODE_LOCAL_BUS_COUNT. */
10791 |     ma_bool32 _ownsHeap; /* If set to true, the node owns the heap allocation and _pHeap will be
> freed in ma_node_uninit(). */
10792 | };
10793 |
10794 | MA_API ma_result ma_node_get_heap_size(ma_node_graph* pNodeGraph, const ma_node_config* pConfig,
> size_t* pHeapSizeInBytes);
10795 | MA_API ma_result ma_node_init_preallocated(ma_node_graph* pNodeGraph, const ma_node_config* pConfig,
> void* pHeap, ma_node* pNode);
10796 | MA_API ma_result ma_node_init(ma_node_graph* pNodeGraph, const ma_node_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_node* pNode);
10797 | MA_API void ma_node_uninit(ma_node* pNode, const ma_allocation_callbacks* pAllocationCallbacks);
10798 | MA_API ma_node_graph* ma_node_get_node_graph(const ma_node* pNode);
10799 | MA_API ma_uint32 ma_node_get_input_bus_count(const ma_node* pNode);
10800 | MA_API ma_uint32 ma_node_get_output_bus_count(const ma_node* pNode);
10801 | MA_API ma_uint32 ma_node_get_input_channels(const ma_node* pNode, ma_uint32 inputBusIndex);
10802 | MA_API ma_result ma_node_detach_all_output_buses(const ma_node* pNode, ma_uint32 outputBusIndex);
10803 | MA_API ma_result ma_node_attach_output_bus(ma_node* pNode, ma_uint32 outputBusIndex, ma_node*
> pOtherNode, ma_uint32 otherNodeInputBusIndex);
10804 | MA_API ma_result ma_node_detach_output_bus(ma_node* pNode, ma_uint32 outputBusIndex);
10805 | MA_API ma_result ma_node_detach_all_output_buses(ma_node* pNode);
10806 | MA_API ma_result ma_node_set_output_bus_volume(ma_node* pNode, ma_uint32 outputBusIndex, float
> volume);
10807 | MA_API float ma_node_get_output_bus_volume(const ma_node* pNode, ma_uint32 outputBusIndex);
10808 | MA_API ma_result ma_node_set_state(ma_node* pNode, ma_node_state state);
10809 | MA_API ma_node_state ma_node_get_state(const ma_node* pNode);
10810 | MA_API ma_result ma_node_set_state_time(ma_node* pNode, ma_node_state state, ma_uint64 globalTime);
10811 | MA_API ma_uint64 ma_node_get_state_time(const ma_node* pNode, ma_node_state state);
10812 | MA_API ma_node_state ma_node_get_state_by_time(const ma_node* pNode, ma_uint64 globalTime);
10813 | MA_API ma_node_state ma_node_get_state_by_time_range(const ma_node* pNode, ma_uint64 globalTimeBeg,
> ma_uint64 globalTimeEnd);
10814 | MA_API ma_uint64 ma_node_get_time(const ma_node* pNode);
10815 | MA_API ma_result ma_node_set_time(ma_node* pNode, ma_uint64 localTime);
10816 |
10817 |
10818 | typedef struct
10819 | {
10820 |     ma_uint32 channels;
10821 |     ma_uint32 processingSizeInFrames; /* This is the preferred processing size for node processing
> callbacks unless overridden by a node itself. Can be 0 in which case it will be based on the frame
> count passed into ma_node_graph_read_pcm_frames(), but will not be well defined. */
10822 |     size_t preMixStackSizeInBytes; /* Defaults to 512KB per channel. Reducing this will save
> memory, but the depth of your node graph will be more restricted. */
10823 | } ma_node_graph_config;
10824 |
10825 | MA_API ma_node_graph_config ma_node_graph_config_init(ma_uint32 channels);
10826 |
10827 |
10828 | struct ma_node_graph
10829 | {
10830 |     /* Immutable. */
10831 |     ma_node_base base; /* The node graph itself is a node so it can be connected as
> an input to different node graph. This has zero inputs and calls ma_node_graph_read_pcm_frames() to
> generate it's output. */
10832 |     ma_node_base endpoint; /* Special node that all nodes eventually connect to. Data
> is read from this node in ma_node_graph_read_pcm_frames(). */
10833 |     float* pProcessingCache; /* This will be allocated when processingSizeInFrames is
> non-zero. This is needed because ma_node_graph_read_pcm_frames() can be called with a variable
> number of frames, and we may need to do some buffering in situations where the caller requests a
> frame count that's not a multiple of processingSizeInFrames. */
10834 |     ma_uint32 processingCacheFramesRemaining;
10835 |     ma_uint32 processingSizeInFrames;
10836 |
10837 |     /* Read and written by multiple threads. */
10838 |     MA_ATOMIC(4, ma_bool32) isReading;
10839 |
10840 |     /* Modified only by the audio thread. */
10841 |     ma_stack* pPreMixStack;
10842 | };
10843 |
10844 | MA_API ma_result ma_node_graph_init(const ma_node_graph_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_node_graph* pNodeGraph);
10845 | MA_API void ma_node_graph_uninit(ma_node_graph* pNodeGraph, const ma_allocation_callbacks*
> pAllocationCallbacks);
10846 | MA_API ma_node* ma_node_graph_get_endpoint(ma_node_graph* pNodeGraph);
10847 | MA_API ma_result ma_node_graph_read_pcm_frames(ma_node_graph* pNodeGraph, void* pFramesOut,

```



```

> ma_uint64 frameCount, ma_uint64* pFramesRead);
10848 | MA_API ma_uint32 ma_node_graph_get_channels(const ma_node_graph* pNodeGraph);
10849 | MA_API ma_uint64 ma_node_graph_get_time(const ma_node_graph* pNodeGraph);
10850 | MA_API ma_result ma_node_graph_set_time(ma_node_graph* pNodeGraph, ma_uint64 globalTime);
10851 | MA_API ma_uint32 ma_node_graph_get_processing_size_in_frames(const ma_node_graph* pNodeGraph);
10852 |
10853 |
10854 |
10855 | /* Data source node. 0 input buses, 1 output bus. Used for reading from a data source. */
10856 | typedef struct
10857 | {
10858 |     ma_node_config nodeConfig;
10859 |     ma_data_source* pDataSource;
10860 | } ma_data_source_node_config;
10861 |
10862 | MA_API ma_data_source_node_config ma_data_source_node_config_init(ma_data_source* pDataSource);
10863 |
10864 |
10865 | typedef struct
10866 | {
10867 |     ma_node_base base;
10868 |     ma_data_source* pDataSource;
10869 | } ma_data_source_node;
10870 |
10871 | MA_API ma_result ma_data_source_node_init(ma_node_graph* pNodeGraph, const
> ma_data_source_node_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source_node* pDataSourceNode);
10872 | MA_API void ma_data_source_node_uninit(ma_data_source_node* pDataSourceNode, const
> ma_allocation_callbacks* pAllocationCallbacks);
10873 | MA_API ma_result ma_data_source_node_set_looping(ma_data_source_node* pDataSourceNode, ma_bool32
> isLooping);
10874 | MA_API ma_bool32 ma_data_source_node_is_looping(ma_data_source_node* pDataSourceNode);
10875 |
10876 |
10877 | /* Splitter Node. 1 input, many outputs. Used for splitting/copying a stream so it can be as input
> into two separate output nodes. */
10878 | typedef struct
10879 | {
10880 |     ma_node_config nodeConfig;
10881 |     ma_uint32 channels;
10882 |     ma_uint32 outputBusCount;
10883 | } ma_splitter_node_config;
10884 |
10885 | MA_API ma_splitter_node_config ma_splitter_node_config_init(ma_uint32 channels);
10886 |
10887 |
10888 | typedef struct
10889 | {
10890 |     ma_node_base base;
10891 | } ma_splitter_node;
10892 |
10893 | MA_API ma_result ma_splitter_node_init(ma_node_graph* pNodeGraph, const ma_splitter_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_splitter_node* pSplitterNode);
10894 | MA_API void ma_splitter_node_uninit(ma_splitter_node* pSplitterNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
10895 |
10896 |
10897 | /*
10898 | Biquad Node
10899 | */
10900 | typedef struct
10901 | {
10902 |     ma_node_config nodeConfig;
10903 |     ma_biquad_config biquad;
10904 | } ma_biquad_node_config;
10905 |
10906 | MA_API ma_biquad_node_config ma_biquad_node_config_init(ma_uint32 channels, float b0, float b1,
> float b2, float a0, float a1, float a2);
10907 |
10908 |
10909 | typedef struct
10910 | {
10911 |     ma_node_base baseNode;
10912 |     ma_biquad biquad;
10913 | } ma_biquad_node;
10914 |
10915 | MA_API ma_result ma_biquad_node_init(ma_node_graph* pNodeGraph, const ma_biquad_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_biquad_node* pNode);
10916 | MA_API ma_result ma_biquad_node_reinit(const ma_biquad_config* pConfig, ma_biquad_node* pNode);

```



```

10917 | MA_API void ma_biquad_node_uninit(ma_biquad_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
10918 |
10919 |
10920 | /*
10921 | Low Pass Filter Node
10922 | */
10923 | typedef struct
10924 | {
10925 |     ma_node_config nodeConfig;
10926 |     ma_lpf_config lpf;
10927 | } ma_lpf_node_config;
10928 |
10929 | MA_API ma_lpf_node_config ma_lpf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> cutoffFrequency, ma_uint32 order);
10930 |
10931 |
10932 | typedef struct
10933 | {
10934 |     ma_node_base baseNode;
10935 |     ma_lpf lpf;
10936 | } ma_lpf_node;
10937 |
10938 | MA_API ma_result ma_lpf_node_init(ma_node_graph* pNodeGraph, const ma_lpf_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_lpf_node* pNode);
10939 | MA_API ma_result ma_lpf_node_reinit(const ma_lpf_config* pConfig, ma_lpf_node* pNode);
10940 | MA_API void ma_lpf_node_uninit(ma_lpf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
10941 |
10942 |
10943 | /*
10944 | High Pass Filter Node
10945 | */
10946 | typedef struct
10947 | {
10948 |     ma_node_config nodeConfig;
10949 |     ma_hpf_config hpf;
10950 | } ma_hpf_node_config;
10951 |
10952 | MA_API ma_hpf_node_config ma_hpf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> cutoffFrequency, ma_uint32 order);
10953 |
10954 |
10955 | typedef struct
10956 | {
10957 |     ma_node_base baseNode;
10958 |     ma_hpf hpf;
10959 | } ma_hpf_node;
10960 |
10961 | MA_API ma_result ma_hpf_node_init(ma_node_graph* pNodeGraph, const ma_hpf_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_hpf_node* pNode);
10962 | MA_API ma_result ma_hpf_node_reinit(const ma_hpf_config* pConfig, ma_hpf_node* pNode);
10963 | MA_API void ma_hpf_node_uninit(ma_hpf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
10964 |
10965 |
10966 | /*
10967 | Band Pass Filter Node
10968 | */
10969 | typedef struct
10970 | {
10971 |     ma_node_config nodeConfig;
10972 |     ma_bpf_config bpf;
10973 | } ma_bpf_node_config;
10974 |
10975 | MA_API ma_bpf_node_config ma_bpf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> cutoffFrequency, ma_uint32 order);
10976 |
10977 |
10978 | typedef struct
10979 | {
10980 |     ma_node_base baseNode;
10981 |     ma_bpf bpf;
10982 | } ma_bpf_node;
10983 |
10984 | MA_API ma_result ma_bpf_node_init(ma_node_graph* pNodeGraph, const ma_bpf_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_bpf_node* pNode);
10985 | MA_API ma_result ma_bpf_node_reinit(const ma_bpf_config* pConfig, ma_bpf_node* pNode);
10986 | MA_API void ma_bpf_node_uninit(ma_bpf_node* pNode, const ma_allocation_callbacks*

```

```

    > pAllocationCallbacks);
10987 |
10988 |
10989 | /*
10990 | Notching Filter Node
10991 | */
10992 | typedef struct
10993 | {
10994 |     ma_node_config nodeConfig;
10995 |     ma_notch_config notch;
10996 | } ma_notch_node_config;
10997 |
10998 | MA_API ma_notch_node_config ma_notch_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
    > double q, double frequency);
10999 |
11000 |
11001 | typedef struct
11002 | {
11003 |     ma_node_base baseNode;
11004 |     ma_notch2 notch;
11005 | } ma_notch_node;
11006 |
11007 | MA_API ma_result ma_notch_node_init(ma_node_graph* pNodeGraph, const ma_notch_node_config* pConfig,
    > const ma_allocation_callbacks* pAllocationCallbacks, ma_notch_node* pNode);
11008 | MA_API ma_result ma_notch_node_reinit(const ma_notch_config* pConfig, ma_notch_node* pNode);
11009 | MA_API void ma_notch_node_uninit(ma_notch_node* pNode, const ma_allocation_callbacks*
    > pAllocationCallbacks);
11010 |
11011 |
11012 | /*
11013 | Peaking Filter Node
11014 | */
11015 | typedef struct
11016 | {
11017 |     ma_node_config nodeConfig;
11018 |     ma_peak_config peak;
11019 | } ma_peak_node_config;
11020 |
11021 | MA_API ma_peak_node_config ma_peak_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
    > gainDB, double q, double frequency);
11022 |
11023 |
11024 | typedef struct
11025 | {
11026 |     ma_node_base baseNode;
11027 |     ma_peak2 peak;
11028 | } ma_peak_node;
11029 |
11030 | MA_API ma_result ma_peak_node_init(ma_node_graph* pNodeGraph, const ma_peak_node_config* pConfig,
    > const ma_allocation_callbacks* pAllocationCallbacks, ma_peak_node* pNode);
11031 | MA_API ma_result ma_peak_node_reinit(const ma_peak_config* pConfig, ma_peak_node* pNode);
11032 | MA_API void ma_peak_node_uninit(ma_peak_node* pNode, const ma_allocation_callbacks*
    > pAllocationCallbacks);
11033 |
11034 |
11035 | /*
11036 | Low Shelf Filter Node
11037 | */
11038 | typedef struct
11039 | {
11040 |     ma_node_config nodeConfig;
11041 |     ma_loshelf_config loshelf;
11042 | } ma_loshelf_node_config;
11043 |
11044 | MA_API ma_loshelf_node_config ma_loshelf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
    > double gainDB, double q, double frequency);
11045 |
11046 |
11047 | typedef struct
11048 | {
11049 |     ma_node_base baseNode;
11050 |     ma_loshelf2 loshelf;
11051 | } ma_loshelf_node;
11052 |
11053 | MA_API ma_result ma_loshelf_node_init(ma_node_graph* pNodeGraph, const ma_loshelf_node_config*
    > pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_loshelf_node* pNode);
11054 | MA_API ma_result ma_loshelf_node_reinit(const ma_loshelf_config* pConfig, ma_loshelf_node* pNode);
11055 | MA_API void ma_loshelf_node_uninit(ma_loshelf_node* pNode, const ma_allocation_callbacks*
    > pAllocationCallbacks);

```

```

11056 |
11057 |
11058 | /*
11059 | High Shelf Filter Node
11060 | */
11061 | typedef struct
11062 | {
11063 |     ma_node_config nodeConfig;
11064 |     ma_hishelf_config hishelf;
11065 | } ma_hishelf_node_config;
11066 |
11067 | MA_API ma_result ma_hishelf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
> double gainDB, double q, double frequency);
11068 |
11069 |
11070 | typedef struct
11071 | {
11072 |     ma_node_base baseNode;
11073 |     ma_hishelf2 hishelf;
11074 | } ma_hishelf_node;
11075 |
11076 | MA_API ma_result ma_hishelf_node_init(ma_node_graph* pNodeGraph, const ma_hishelf_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_hishelf_node* pNode);
11077 | MA_API ma_result ma_hishelf_node_reinit(const ma_hishelf_config* pConfig, ma_hishelf_node* pNode);
11078 | MA_API void ma_hishelf_node_uninit(ma_hishelf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
11079 |
11080 |
11081 | typedef struct
11082 | {
11083 |     ma_node_config nodeConfig;
11084 |     ma_delay_config delay;
11085 | } ma_delay_node_config;
11086 |
11087 | MA_API ma_delay_node_config ma_delay_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
> ma_uint32 delayInFrames, float decay);
11088 |
11089 |
11090 | typedef struct
11091 | {
11092 |     ma_node_base baseNode;
11093 |     ma_delay delay;
11094 | } ma_delay_node;
11095 |
11096 | MA_API ma_result ma_delay_node_init(ma_node_graph* pNodeGraph, const ma_delay_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_delay_node* pDelayNode);
11097 | MA_API void ma_delay_node_uninit(ma_delay_node* pDelayNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
11098 | MA_API void ma_delay_node_set_wet(ma_delay_node* pDelayNode, float value);
11099 | MA_API float ma_delay_node_get_wet(const ma_delay_node* pDelayNode);
11100 | MA_API void ma_delay_node_set_dry(ma_delay_node* pDelayNode, float value);
11101 | MA_API float ma_delay_node_get_dry(const ma_delay_node* pDelayNode);
11102 | MA_API void ma_delay_node_set_decay(ma_delay_node* pDelayNode, float value);
11103 | MA_API float ma_delay_node_get_decay(const ma_delay_node* pDelayNode);
11104 | #endif /* MA_NO_NODE_GRAPH */
11105 |
11106 |
11107 | /* SECTION: miniaudio_engine.h */
11108 | /*****
> *****/
11109 |
11110 | Engine
11111 |
11112 | /*****
> *****/
11113 | #if !defined(MA_NO_ENGINE) && !defined(MA_NO_NODE_GRAPH)
11114 | typedef struct ma_engine ma_engine;
11115 | typedef struct ma_sound ma_sound;
11116 |
11117 |
11118 | /* Sound flags. */
11119 | typedef enum
11120 | {
11121 |     /* Resource manager flags. */
11122 |     MA_SOUND_FLAG_STREAM = 0x00000001, /*
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM */
11123 |     MA_SOUND_FLAG_DECODE = 0x00000002, /*
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE */
11124 |     MA_SOUND_FLAG_ASYNC = 0x00000004, /*

```

```

> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC */
11125 |     MA_SOUND_FLAG_WAIT_INIT           = 0x00000008,    /*
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT */
11126 |     MA_SOUND_FLAG_UNKNOWN_LENGTH      = 0x00000010,    /*
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_UNKNOWN_LENGTH */
11127 |     MA_SOUND_FLAG_LOOPING             = 0x00000020,    /*
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING */
11128 |
11129 |     /* ma_sound specific flags. */
11130 |     MA_SOUND_FLAG_NO_DEFAULT_ATTACHMENT = 0x00001000,    /* Do not attach to the endpoint by default.
> Useful for when setting up nodes in a complex graph system. */
11131 |     MA_SOUND_FLAG_NO_PITCH            = 0x00002000,    /* Disable pitch shifting with
> ma_sound_set_pitch() and ma_sound_group_set_pitch(). This is an optimization. */
11132 |     MA_SOUND_FLAG_NO_SPATIALIZATION    = 0x00004000    /* Disable spatialization. */
11133 | } ma_sound_flags;
11134 |
11135 | #ifndef MA_ENGINE_MAX_LISTENERS
11136 | #define MA_ENGINE_MAX_LISTENERS        4
11137 | #endif
11138 |
11139 | #define MA_LISTENER_INDEX_CLOSEST      ((ma_uint8)-1)
11140 |
11141 | typedef enum
11142 | {
11143 |     ma_engine_node_type_sound,
11144 |     ma_engine_node_type_group
11145 | } ma_engine_node_type;
11146 |
11147 | typedef struct
11148 | {
11149 |     ma_engine* pEngine;
11150 |     ma_engine_node_type type;
11151 |     ma_uint32 channelsIn;
11152 |     ma_uint32 channelsOut;
11153 |     ma_uint32 sampleRate;                /* Only used when the type is set to
> ma_engine_node_type_sound. */
11154 |     ma_uint32 volumeSmoothTimeInPCMFrames; /* The number of frames to smooth over volume changes.
> Defaults to 0 in which case no smoothing is used. */
11155 |     ma_mono_expansion_mode monoExpansionMode;
11156 |     ma_bool8 isPitchDisabled;            /* Pitching can be explicitly disabled with
> MA_SOUND_FLAG_NO_PITCH to optimize processing. */
11157 |     ma_bool8 isSpatializationDisabled;    /* Spatialization can be explicitly disabled with
> MA_SOUND_FLAG_NO_SPATIALIZATION. */
11158 |     ma_uint8 pinnedListenerIndex;        /* The index of the listener this node should always use for
> spatialization. If set to MA_LISTENER_INDEX_CLOSEST the engine will use the closest listener. */
11159 |     ma_resampler_config resampling;
11160 | } ma_engine_node_config;
11161 |
11162 | MA_API ma_engine_node_config ma_engine_node_config_init(ma_engine* pEngine, ma_engine_node_type
> type, ma_uint32 flags);
11163 |
11164 |
11165 | /* Base node object for both ma_sound and ma_sound_group. */
11166 | typedef struct
11167 | {
11168 |     ma_node_base baseNode;                /* Must be the first member for
> compatibility with the ma_node API. */
11169 |     ma_engine* pEngine;                    /* A pointer to the engine. Set based on the
> value from the config. */
11170 |     ma_uint32 sampleRate;                  /* The sample rate of the input data. For
> sounds backed by a data source, this will be the data source's sample rate. Otherwise it'll be the
> engine's sample rate. */
11171 |     ma_uint32 volumeSmoothTimeInPCMFrames;
11172 |     ma_mono_expansion_mode monoExpansionMode;
11173 |     ma_fader fader;
11174 |     ma_resampler resampler;                /* For pitch shift. */
11175 |     ma_spatializer spatializer;
11176 |     ma_panner panner;
11177 |     ma_gainer volumeGainer;                /* This will only be used if
> volumeSmoothTimeInPCMFrames is > 0. */
11178 |     ma_atomic_float volume;                /* Defaults to 1. */
11179 |     MA_ATOMIC(4, float) pitch;
11180 |     float oldPitch;                        /* For determining whether or not the
> resampler needs to be updated to reflect the new pitch. The resampler will be updated on the mixing
> thread. */
11181 |     float oldDopplerPitch;                /* For determining whether or not the
> resampler needs to be updated to take a new doppler pitch into account. */
11182 |     MA_ATOMIC(4, ma_bool32) isPitchDisabled; /* When set to true, pitching will be
> disabled which will allow the resampler to be bypassed to save some computation. */

```

```

11183 |     MA_ATOMIC(4, ma_bool32) isSpatializationDisabled;    /* Set to false by default. When set to
> false, will not have spatialisation applied. */
11184 |     MA_ATOMIC(4, ma_uint32) pinnedListenerIndex;          /* The index of the listener this node
> should always use for spatialization. If set to MA_LISTENER_INDEX_CLOSEST the engine will use the
> closest listener. */
11185 |
11186 |     /* When setting a fade, it's not done immediately in ma_sound_set_fade(). It's deferred to the
> audio thread which means we need to store the settings here. */
11187 |     struct
11188 |     {
11189 |         ma_atomic_float volumeBeg;
11190 |         ma_atomic_float volumeEnd;
11191 |         ma_atomic_uint64 fadeLengthInFrames;               /* <-- Defaults to ~(ma_uint64)0 which is
> used to indicate that no fade should be applied. */
11192 |         ma_atomic_uint64 absoluteGlobalTimeInFrames;      /* <-- The time to start the fade. */
11193 |     } fadeSettings;
11194 |
11195 |     /* Memory management. */
11196 |     ma_bool8 _ownsHeap;
11197 |     void* _pHeap;
11198 | } ma_engine_node;
11199 |
11200 | MA_API ma_result ma_engine_node_get_heap_size(const ma_engine_node_config* pConfig, size_t*
> pHeapSizeInBytes);
11201 | MA_API ma_result ma_engine_node_init_preallocated(const ma_engine_node_config* pConfig, void* pHeap,
> ma_engine_node* pEngineNode);
11202 | MA_API ma_result ma_engine_node_init(const ma_engine_node_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_engine_node* pEngineNode);
11203 | MA_API void ma_engine_node_uninit(ma_engine_node* pEngineNode, const ma_allocation_callbacks*
> pAllocationCallbacks);
11204 |
11205 |
11206 | #define MA_SOUND_SOURCE_CHANNEL_COUNT    0xFFFFFFFF
11207 |
11208 | /* Callback for when a sound reaches the end. */
11209 | typedef void (* ma_sound_end_proc)(void* pUserData, ma_sound* pSound);
11210 |
11211 | typedef struct
11212 | {
11213 |     const char* filePath;                                /* Set this to load from the resource manager. */
11214 |     const wchar_t* filePathW;                            /* Set this to load from the resource manager. */
11215 |     ma_data_source* pDataSource;                         /* Set this to load from an existing data source. */
11216 |     ma_node* pInitialAttachment;                         /* If set, the sound will be attached to an input of
> this node. This can be set to a ma_sound. If set to NULL, the sound will be attached directly to
> the endpoint unless MA_SOUND_FLAG_NO_DEFAULT_ATTACHMENT is set in `flags`. */
11217 |     ma_uint32 initialAttachmentInputBusIndex;           /* The index of the input bus of pInitialAttachment
> to attach the sound to. */
11218 |     ma_uint32 channelsIn;                                /* Ignored if using a data source as input (the data
> source's channel count will be used always). Otherwise, setting to 0 will cause the engine's
> channel count to be used. */
11219 |     ma_uint32 channelsOut;                                /* Set this to 0 (default) to use the engine's
> channel count. Set to MA_SOUND_SOURCE_CHANNEL_COUNT to use the data source's channel count (only
> used if using a data source as input). */
11220 |     ma_mono_expansion_mode monoExpansionMode;           /* Controls how the mono channel should be expanded
> to other channels when spatialization is disabled on a sound. */
11221 |     ma_uint32 flags;                                     /* A combination of MA_SOUND_FLAG_* flags. */
11222 |     ma_uint32 volumeSmoothTimeInPCMframes;              /* The number of frames to smooth over volume
> changes. Defaults to 0 in which case no smoothing is used. */
11223 |     ma_uint64 initialSeekPointInPCMframes;              /* Initializes the sound such that it's seeked to
> this location by default. */
11224 |     ma_uint64 rangeBegInPCMframes;
11225 |     ma_uint64 rangeEndInPCMframes;
11226 |     ma_uint64 loopPointBegInPCMframes;
11227 |     ma_uint64 loopPointEndInPCMframes;
11228 |     ma_sound_end_proc endCallback;                      /* Fired when the sound reaches the end. Will be
> fired from the audio thread. Do not restart, uninitialized or otherwise change the state of the sound
> from here. Instead fire an event or set a variable to indicate to a different thread to change the
> start of the sound. Will not be fired in response to a scheduled stop with
> ma_sound_set_stop_time*(). */
11229 |     void* pEndCallbackUserData;
11230 |     ma_resampler_config pitchResampling;
11231 | #ifndef MA_NO_RESOURCE_MANAGER
11232 |     ma_resource_manager_pipeline_notifications initNotifications;
11233 | #endif
11234 |     ma_fence* pDoneFence;                                /* Deprecated. Use initNotifications instead.
> Released when the resource manager has finished decoding the entire sound. Not used with streams. */
11235 |     ma_bool32 isLooping;                                /* Deprecated. Use the MA_SOUND_FLAG_LOOPING flag in
> `flags` instead. */
11236 | } ma_sound_config;

```

```

11237 |
11238 | MA_API ma_sound_config ma_sound_config_init(void); /* Deprecated. Will be removed
> in version 0.12. Use ma_sound_config_2() instead. */
11239 | MA_API ma_sound_config ma_sound_config_init_2(ma_engine* pEngine); /* Will be renamed to
> ma_sound_config_init() in version 0.12. */
11240 |
11241 | struct ma_sound
11242 | {
11243 |     ma_engine_node engineNode; /* Must be the first member for compatibility with the
> ma_node API. */
11244 |     ma_data_source* pDataSource;
11245 |     MA_ATOMIC(8, ma_uint64) seekTarget; /* The PCM frame index to seek to in the mixing thread. Set
> to ~(ma_uint64)0 to not perform any seeking. */
11246 |     MA_ATOMIC(4, ma_bool32) atEnd;
11247 |     ma_sound_end_proc endCallback;
11248 |     void* pEndCallbackUserData;
11249 |     float* pProcessingCache; /* Will be null if pDataSource is null. */
11250 |     ma_uint32 processingCacheFramesRemaining;
11251 |     ma_uint32 processingCacheCap;
11252 |     ma_bool8 ownsDataSource;
11253 |
11254 |     /*
11255 |     We're declaring a resource manager data source object here to save us a malloc when loading a
11256 |     sound via the resource manager, which I *think* will be the most common scenario.
11257 |     */
11258 | #ifndef MA_NO_RESOURCE_MANAGER
11259 |     ma_resource_manager_data_source* pResourceManagerDataSource;
11260 | #endif
11261 | };
11262 |
11263 | /* Structure specifically for sounds played with ma_engine_play_sound(). Making this a separate
> structure to reduce overhead. */
11264 | typedef struct ma_sound_inlined ma_sound_inlined;
11265 | struct ma_sound_inlined
11266 | {
11267 |     ma_sound sound;
11268 |     ma_sound_inlined* pNext;
11269 |     ma_sound_inlined* pPrev;
11270 | };
11271 |
11272 | /* A sound group is just a sound. */
11273 | typedef ma_sound_config ma_sound_group_config;
11274 | typedef ma_sound ma_sound_group;
11275 |
11276 | MA_API ma_sound_group_config ma_sound_group_config_init(void); /* Deprecated. Will
> be removed in version 0.12. Use ma_sound_config_2() instead. */
11277 | MA_API ma_sound_group_config ma_sound_group_config_init_2(ma_engine* pEngine); /* Will be renamed
> to ma_sound_config_init() in version 0.12. */
11278 |
11279 | typedef void (* ma_engine_process_proc)(void* pUserData, float* pFramesOut, ma_uint64 frameCount);
11280 |
11281 | typedef struct
11282 | {
11283 |     #if !defined(MA_NO_RESOURCE_MANAGER)
11284 |     ma_resource_manager* pResourceManager; /* Can be null in which case a resource manager
> will be created for you. */
11285 |     #endif
11286 |     #if !defined(MA_NO_DEVICE_IO)
11287 |     ma_context* pContext;
11288 |     ma_device* pDevice; /* If set, the caller is responsible for calling
> ma_engine_data_callback() in the device's data callback. */
11289 |     ma_device_id* pPlaybackDeviceID; /* The ID of the playback device to use with the
> default listener. */
11290 |     ma_device_data_proc dataCallback; /* Can be null. Can be used to provide a custom
> device data callback. */
11291 |     ma_device_notification_proc notificationCallback;
11292 |     #endif
11293 |     ma_log* pLog; /* When set to NULL, will use the context's log.
> */
11294 |     ma_uint32 listenerCount; /* Must be between 1 and
> MA_ENGINE_MAX_LISTENERS. */
11295 |     ma_uint32 channels; /* The number of channels to use when mixing and
> spatializing. When set to 0, will use the native channel count of the device. */
11296 |     ma_uint32 sampleRate; /* The sample rate. When set to 0 will use the
> native sample rate of the device. */
11297 |     ma_uint32 periodSizeInFrames; /* If set to something other than 0, updates
> will always be exactly this size. The underlying device may be a different size, but from the
> perspective of the mixer that won't matter. */
11298 |     ma_uint32 periodSizeInMilliseconds; /* Used if periodSizeInFrames is unset. */

```

```

11299 |     ma_uint32 gainSmoothTimeInFrames;                /* The number of frames to interpolate the gain
> of spatialized sounds across. If set to 0, will use gainSmoothTimeInMilliseconds. */
11300 |     ma_uint32 gainSmoothTimeInMilliseconds;          /* When set to 0, gainSmoothTimeInFrames will be
> used. If both are set to 0, a default value will be used. */
11301 |     ma_uint32 defaultVolumeSmoothTimeInPCMframes;    /* Defaults to 0. Controls the default amount of
> smoothing to apply to volume changes to sounds. High values means more smoothing at the expense of
> high latency (will take longer to reach the new volume). */
11302 |     ma_uint32 preMixStackSizeInBytes;                /* A stack is used for internal processing in
> the node graph. This allows you to configure the size of this stack. Smaller values will reduce the
> maximum depth of your node graph. You should rarely need to modify this. */
11303 |     ma_allocation_callbacks allocationCallbacks;
11304 |     ma_bool32 noAutoStart;                            /* When set to true, requires an explicit call
> to ma_engine_start(). This is false by default, meaning the engine will be started automatically in
> ma_engine_init(). */
11305 |     ma_bool32 noDevice;                                /* When set to true, don't create a default
> device. ma_engine_read_pcm_frames() can be called manually to read data. */
11306 |     ma_mono_expansion_mode monoExpansionMode;        /* Controls how the mono channel should be
> expanded to other channels when spatialization is disabled on a sound. */
11307 |     ma_vfs* pResourceManagerVFS;                    /* A pointer to a pre-allocated VFS object to
> use with the resource manager. This is ignored if pResourceManager is not NULL. */
11308 |     ma_engine_process_proc onProcess;                /* Fired at the end of each call to
> ma_engine_read_pcm_frames(). For engine's that manage their own internal device (the default
> configuration), this will be fired from the audio thread, and you do not need to call
> ma_engine_read_pcm_frames() manually in order to trigger this. */
11309 |     void* pProcessUserData;                          /* User data that's passed into onProcess. */
11310 |     ma_resampler_config resourceManagerResampling;    /* The resampling config to use with the
> resource manager. */
11311 |     ma_resampler_config pitchResampling;              /* The resampling config for the pitch and
> Doppler effects. You will typically want this to be a fast resampler. For high quality stuff, it's
> recommended that you pre-resample. */
11312 | } ma_engine_config;
11313 |
11314 | MA_API ma_engine_config ma_engine_config_init(void);
11315 |
11316 |
11317 | struct ma_engine
11318 | {
11319 |     ma_node_graph nodeGraph;                        /* An engine is a node graph. It should be able
> to be plugged into any ma_node_graph API (with a cast) which means this must be the first member of
> this struct. */
11320 | #if !defined(MA_NO_RESOURCE_MANAGER)
11321 |     ma_resource_manager* pResourceManager;
11322 | #endif
11323 | #if !defined(MA_NO_DEVICE_IO)
11324 |     ma_device* pDevice;                            /* Optionally set via the config, otherwise
> allocated by the engine in ma_engine_init(). */
11325 | #endif
11326 |     ma_log* pLog;
11327 |     ma_uint32 sampleRate;
11328 |     ma_uint32 listenerCount;
11329 |     ma_spatializer_listener listeners[MA_ENGINE_MAX_LISTENERS];
11330 |     ma_allocation_callbacks allocationCallbacks;
11331 |     ma_bool8 ownsResourceManager;
11332 |     ma_bool8 ownsDevice;
11333 |     ma_spinlock inlinedSoundLock;                  /* For synchronizing access to the inlined sound
> list. */
11334 |     ma_sound_inlined* pInlinedSoundHead;            /* The first inlined sound. Inlined sounds are
> tracked in a linked list. */
11335 |     MA_ATOMIC(4, ma_uint32) inlinedSoundCount;      /* The total number of allocated inlined sound
> objects. Used for debugging. */
11336 |     ma_uint32 gainSmoothTimeInFrames;                /* The number of frames to interpolate the gain
> of spatialized sounds across. */
11337 |     ma_uint32 defaultVolumeSmoothTimeInPCMframes;
11338 |     ma_mono_expansion_mode monoExpansionMode;
11339 |     ma_engine_process_proc onProcess;
11340 |     void* pProcessUserData;
11341 |     ma_resampler_config pitchResamplingConfig;
11342 | };
11343 |
11344 | MA_API ma_result ma_engine_init(const ma_engine_config* pConfig, ma_engine* pEngine);
11345 | MA_API void ma_engine_uninit(ma_engine* pEngine);
11346 | MA_API ma_result ma_engine_read_pcm_frames(ma_engine* pEngine, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead);
11347 | MA_API ma_node_graph* ma_engine_get_node_graph(ma_engine* pEngine);
11348 | #if !defined(MA_NO_RESOURCE_MANAGER)
11349 | MA_API ma_resource_manager* ma_engine_get_resource_manager(ma_engine* pEngine);
11350 | #endif
11351 | MA_API ma_device* ma_engine_get_device(ma_engine* pEngine);
11352 | MA_API ma_log* ma_engine_get_log(ma_engine* pEngine);

```



```

11353 | MA_API ma_node* ma_engine_get_endpoint(ma_engine* pEngine);
11354 | MA_API ma_uint64 ma_engine_get_time_in_pcm_frames(const ma_engine* pEngine);
11355 | MA_API ma_uint64 ma_engine_get_time_in_milliseconds(const ma_engine* pEngine);
11356 | MA_API ma_result ma_engine_set_time_in_pcm_frames(ma_engine* pEngine, ma_uint64 globalTime);
11357 | MA_API ma_result ma_engine_set_time_in_milliseconds(ma_engine* pEngine, ma_uint64 globalTime);
11358 | MA_API ma_uint64 ma_engine_get_time(const ma_engine* pEngine); /* Deprecated. Use
> ma_engine_get_time_in_pcm_frames(). Will be removed in version 0.12. */
11359 | MA_API ma_result ma_engine_set_time(ma_engine* pEngine, ma_uint64 globalTime); /* Deprecated. Use
> ma_engine_set_time_in_pcm_frames(). Will be removed in version 0.12. */
11360 | MA_API ma_uint32 ma_engine_get_channels(const ma_engine* pEngine);
11361 | MA_API ma_uint32 ma_engine_get_sample_rate(const ma_engine* pEngine);
11362 |
11363 | MA_API ma_result ma_engine_start(ma_engine* pEngine);
11364 | MA_API ma_result ma_engine_stop(ma_engine* pEngine);
11365 | MA_API ma_result ma_engine_set_volume(ma_engine* pEngine, float volume);
11366 | MA_API float ma_engine_get_volume(ma_engine* pEngine);
11367 | MA_API ma_result ma_engine_set_gain_db(ma_engine* pEngine, float gainDB);
11368 | MA_API float ma_engine_get_gain_db(ma_engine* pEngine);
11369 |
11370 | MA_API ma_uint32 ma_engine_get_listener_count(const ma_engine* pEngine);
11371 | MA_API ma_uint32 ma_engine_find_closest_listener(const ma_engine* pEngine, float absolutePosX, float
> absolutePosY, float absolutePosZ);
11372 | MA_API void ma_engine_listener_set_position(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z);
11373 | MA_API ma_vec3f ma_engine_listener_get_position(const ma_engine* pEngine, ma_uint32 listenerIndex);
11374 | MA_API void ma_engine_listener_set_direction(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z);
11375 | MA_API ma_vec3f ma_engine_listener_get_direction(const ma_engine* pEngine, ma_uint32 listenerIndex);
11376 | MA_API void ma_engine_listener_set_velocity(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z);
11377 | MA_API ma_vec3f ma_engine_listener_get_velocity(const ma_engine* pEngine, ma_uint32 listenerIndex);
11378 | MA_API void ma_engine_listener_set_cone(ma_engine* pEngine, ma_uint32 listenerIndex, float
> innerAngleInRadians, float outerAngleInRadians, float outerGain);
11379 | MA_API void ma_engine_listener_get_cone(const ma_engine* pEngine, ma_uint32 listenerIndex, float*
> pInnerAngleInRadians, float* pOuterAngleInRadians, float* pOuterGain);
11380 | MA_API void ma_engine_listener_set_world_up(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z);
11381 | MA_API ma_vec3f ma_engine_listener_get_world_up(const ma_engine* pEngine, ma_uint32 listenerIndex);
11382 | MA_API void ma_engine_listener_set_enabled(ma_engine* pEngine, ma_uint32 listenerIndex, ma_bool32
> isEnabled);
11383 | MA_API ma_bool32 ma_engine_listener_is_enabled(const ma_engine* pEngine, ma_uint32 listenerIndex);
11384 |
11385 | #ifndef MA_NO_RESOURCE_MANAGER
11386 | MA_API ma_result ma_engine_play_sound_ex(ma_engine* pEngine, const char* pFilePath, ma_node* pNode,
> ma_uint32 nodeInputBusIndex);
11387 | MA_API ma_result ma_engine_play_sound(ma_engine* pEngine, const char* pFilePath, ma_sound_group*
> pGroup); /* Fire and forget. */
11388 | #endif
11389 |
11390 | #ifndef MA_NO_RESOURCE_MANAGER
11391 | MA_API ma_result ma_sound_init_from_file(ma_engine* pEngine, const char* pFilePath, ma_uint32 flags,
> ma_sound_group* pGroup, ma_fence* pDoneFence, ma_sound* pSound);
11392 | MA_API ma_result ma_sound_init_from_file_w(ma_engine* pEngine, const wchar_t* pFilePath, ma_uint32
> flags, ma_sound_group* pGroup, ma_fence* pDoneFence, ma_sound* pSound);
11393 | MA_API ma_result ma_sound_init_copy(ma_engine* pEngine, const ma_sound* pExistingSound, ma_uint32
> flags, ma_sound_group* pGroup, ma_sound* pSound);
11394 | #endif
11395 | MA_API ma_result ma_sound_init_from_data_source(ma_engine* pEngine, ma_data_source* pDataSource,
> ma_uint32 flags, ma_sound_group* pGroup, ma_sound* pSound);
11396 | MA_API ma_result ma_sound_init_ex(ma_engine* pEngine, const ma_sound_config* pConfig, ma_sound*
> pSound);
11397 | MA_API void ma_sound_uninit(ma_sound* pSound);
11398 | MA_API ma_engine* ma_sound_get_engine(const ma_sound* pSound);
11399 | MA_API ma_data_source* ma_sound_get_data_source(const ma_sound* pSound);
11400 | MA_API ma_result ma_sound_start(ma_sound* pSound);
11401 | MA_API ma_result ma_sound_stop(ma_sound* pSound);
11402 | MA_API ma_result ma_sound_stop_with_fade_in_pcm_frames(ma_sound* pSound, ma_uint64
> fadeLengthInFrames); /* Will overwrite any scheduled stop and fade. If you want to restart the
> sound, first reset it with `ma_sound_reset_stop_time_and_fade()`. There are plans to make this less
> awkward in the future. */
11403 | MA_API ma_result ma_sound_stop_with_fade_in_milliseconds(ma_sound* pSound, ma_uint64
> fadeLengthInFrames); /* Will overwrite any scheduled stop and fade. If you want to restart the
> sound, first reset it with `ma_sound_reset_stop_time_and_fade()`. There are plans to make this less
> awkward in the future. */
11404 | MA_API void ma_sound_reset_start_time(ma_sound* pSound);
11405 | MA_API void ma_sound_reset_stop_time(ma_sound* pSound);
11406 | MA_API void ma_sound_reset_fade(ma_sound* pSound);
11407 | MA_API void ma_sound_reset_stop_time_and_fade(ma_sound* pSound); /* Resets fades and scheduled stop
> time. Does not seek back to the start. */

```

```

11408 | MA_API void ma_sound_set_volume(ma_sound* pSound, float volume);
11409 | MA_API float ma_sound_get_volume(const ma_sound* pSound);
11410 | MA_API void ma_sound_set_pan(ma_sound* pSound, float pan);
11411 | MA_API float ma_sound_get_pan(const ma_sound* pSound);
11412 | MA_API void ma_sound_set_pan_mode(ma_sound* pSound, ma_pan_mode panMode);
11413 | MA_API ma_pan_mode ma_sound_get_pan_mode(const ma_sound* pSound);
11414 | MA_API void ma_sound_set_pitch(ma_sound* pSound, float pitch);
11415 | MA_API float ma_sound_get_pitch(const ma_sound* pSound);
11416 | MA_API void ma_sound_set_spatialization_enabled(ma_sound* pSound, ma_bool32 enabled);
11417 | MA_API ma_bool32 ma_sound_is_spatialization_enabled(const ma_sound* pSound);
11418 | MA_API void ma_sound_set_pinned_listener_index(ma_sound* pSound, ma_uint32 listenerIndex);
11419 | MA_API ma_uint32 ma_sound_get_pinned_listener_index(const ma_sound* pSound);
11420 | MA_API ma_uint32 ma_sound_get_listener_index(const ma_sound* pSound);
11421 | MA_API ma_vec3f ma_sound_get_direction_to_listener(const ma_sound* pSound);
11422 | MA_API void ma_sound_set_position(ma_sound* pSound, float x, float y, float z);
11423 | MA_API ma_vec3f ma_sound_get_position(const ma_sound* pSound);
11424 | MA_API void ma_sound_set_direction(ma_sound* pSound, float x, float y, float z);
11425 | MA_API ma_vec3f ma_sound_get_direction(const ma_sound* pSound);
11426 | MA_API void ma_sound_set_velocity(ma_sound* pSound, float x, float y, float z);
11427 | MA_API ma_vec3f ma_sound_get_velocity(const ma_sound* pSound);
11428 | MA_API void ma_sound_set_attenuation_model(ma_sound* pSound, ma_attenuation_model attenuationModel);
11429 | MA_API ma_attenuation_model ma_sound_get_attenuation_model(const ma_sound* pSound);
11430 | MA_API void ma_sound_set_positioning(ma_sound* pSound, ma_positioning positioning);
11431 | MA_API ma_positioning ma_sound_get_positioning(const ma_sound* pSound);
11432 | MA_API void ma_sound_set_rolloff(ma_sound* pSound, float rolloff);
11433 | MA_API float ma_sound_get_rolloff(const ma_sound* pSound);
11434 | MA_API void ma_sound_set_min_gain(ma_sound* pSound, float minGain);
11435 | MA_API float ma_sound_get_min_gain(const ma_sound* pSound);
11436 | MA_API void ma_sound_set_max_gain(ma_sound* pSound, float maxGain);
11437 | MA_API float ma_sound_get_max_gain(const ma_sound* pSound);
11438 | MA_API void ma_sound_set_min_distance(ma_sound* pSound, float minDistance);
11439 | MA_API float ma_sound_get_min_distance(const ma_sound* pSound);
11440 | MA_API void ma_sound_set_max_distance(ma_sound* pSound, float maxDistance);
11441 | MA_API float ma_sound_get_max_distance(const ma_sound* pSound);
11442 | MA_API void ma_sound_set_cone(ma_sound* pSound, float innerAngleInRadians, float
> outerAngleInRadians, float outerGain);
11443 | MA_API void ma_sound_get_cone(const ma_sound* pSound, float* pInnerAngleInRadians, float*
> pOuterAngleInRadians, float* pOuterGain);
11444 | MA_API void ma_sound_set_doppler_factor(ma_sound* pSound, float dopplerFactor);
11445 | MA_API float ma_sound_get_doppler_factor(const ma_sound* pSound);
11446 | MA_API void ma_sound_set_directional_attenuation_factor(ma_sound* pSound, float
> directionalAttenuationFactor);
11447 | MA_API float ma_sound_get_directional_attenuation_factor(const ma_sound* pSound);
11448 | MA_API void ma_sound_set_fade_in_pcm_frames(ma_sound* pSound, float volumeBeg, float volumeEnd,
> ma_uint64 fadeLengthInFrames);
11449 | MA_API void ma_sound_set_fade_in_milliseconds(ma_sound* pSound, float volumeBeg, float volumeEnd,
> ma_uint64 fadeLengthInMilliseconds);
11450 | MA_API void ma_sound_set_fade_start_in_pcm_frames(ma_sound* pSound, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInFrames, ma_uint64 absoluteGlobalTimeInFrames);
11451 | MA_API void ma_sound_set_fade_start_in_milliseconds(ma_sound* pSound, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInMilliseconds, ma_uint64 absoluteGlobalTimeInMilliseconds);
11452 | MA_API float ma_sound_get_current_fade_volume(const ma_sound* pSound);
11453 | MA_API void ma_sound_set_start_time_in_pcm_frames(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInFrames);
11454 | MA_API void ma_sound_set_start_time_in_milliseconds(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInMilliseconds);
11455 | MA_API void ma_sound_set_stop_time_in_pcm_frames(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInFrames);
11456 | MA_API void ma_sound_set_stop_time_in_milliseconds(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInMilliseconds);
11457 | MA_API void ma_sound_set_stop_time_with_fade_in_pcm_frames(ma_sound* pSound, ma_uint64
> stopAbsoluteGlobalTimeInFrames, ma_uint64 fadeLengthInFrames);
11458 | MA_API void ma_sound_set_stop_time_with_fade_in_milliseconds(ma_sound* pSound, ma_uint64
> stopAbsoluteGlobalTimeInMilliseconds, ma_uint64 fadeLengthInMilliseconds);
11459 | MA_API ma_bool32 ma_sound_is_playing(const ma_sound* pSound);
11460 | MA_API ma_uint64 ma_sound_get_time_in_pcm_frames(const ma_sound* pSound);
11461 | MA_API ma_uint64 ma_sound_get_time_in_milliseconds(const ma_sound* pSound);
11462 | MA_API void ma_sound_set_looping(ma_sound* pSound, ma_bool32 isLooping);
11463 | MA_API ma_bool32 ma_sound_is_looping(const ma_sound* pSound);
11464 | MA_API ma_bool32 ma_sound_at_end(const ma_sound* pSound);
11465 | MA_API ma_result ma_sound_seek_to_pcm_frame(ma_sound* pSound, ma_uint64 frameIndex); /* Just a
> wrapper around ma_data_source_seek_to_pcm_frame(). */
11466 | MA_API ma_result ma_sound_seek_to_second(ma_sound* pSound, float seekPointInSeconds); /* Abstraction
> to ma_sound_seek_to_pcm_frame() */
11467 | MA_API ma_result ma_sound_get_data_format(const ma_sound* pSound, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
11468 | MA_API ma_result ma_sound_get_cursor_in_pcm_frames(const ma_sound* pSound, ma_uint64* pCursor);
11469 | MA_API ma_result ma_sound_get_length_in_pcm_frames(const ma_sound* pSound, ma_uint64* pLength);
11470 | MA_API ma_result ma_sound_get_cursor_in_seconds(const ma_sound* pSound, float* pCursor);

```

```

11471 | MA_API ma_result ma_sound_get_length_in_seconds(const ma_sound* pSound, float* pLength);
11472 | MA_API ma_result ma_sound_set_end_callback(ma_sound* pSound, ma_sound_end_proc callback, void*
> pUserData);
11473 |
11474 | MA_API ma_result ma_sound_group_init(ma_engine* pEngine, ma_uint32 flags, ma_sound_group*
> pParentGroup, ma_sound_group* pGroup);
11475 | MA_API ma_result ma_sound_group_init_ex(ma_engine* pEngine, const ma_sound_group_config* pConfig,
> ma_sound_group* pGroup);
11476 | MA_API void ma_sound_group_uninit(ma_sound_group* pGroup);
11477 | MA_API ma_engine* ma_sound_group_get_engine(const ma_sound_group* pGroup);
11478 | MA_API ma_result ma_sound_group_start(ma_sound_group* pGroup);
11479 | MA_API void ma_sound_group_stop(ma_sound_group* pGroup);
11480 | MA_API void ma_sound_group_set_volume(ma_sound_group* pGroup, float volume);
11481 | MA_API float ma_sound_group_get_volume(const ma_sound_group* pGroup);
11482 | MA_API void ma_sound_group_set_pan(ma_sound_group* pGroup, float pan);
11483 | MA_API float ma_sound_group_get_pan(const ma_sound_group* pGroup);
11484 | MA_API void ma_sound_group_set_pan_mode(ma_sound_group* pGroup, ma_pan_mode panMode);
11485 | MA_API ma_pan_mode ma_sound_group_get_pan_mode(const ma_sound_group* pGroup);
11486 | MA_API void ma_sound_group_set_pitch(ma_sound_group* pGroup, float pitch);
11487 | MA_API float ma_sound_group_get_pitch(const ma_sound_group* pGroup);
11488 | MA_API void ma_sound_group_set_spatialization_enabled(ma_sound_group* pGroup, ma_bool32 enabled);
11489 | MA_API ma_bool32 ma_sound_group_is_spatialization_enabled(const ma_sound_group* pGroup);
11490 | MA_API void ma_sound_group_set_pinned_listener_index(ma_sound_group* pGroup, ma_uint32
> listenerIndex);
11491 | MA_API ma_uint32 ma_sound_group_get_pinned_listener_index(const ma_sound_group* pGroup);
11492 | MA_API ma_uint32 ma_sound_group_get_listener_index(const ma_sound_group* pGroup);
11493 | MA_API ma_vec3f ma_sound_group_get_direction_to_listener(const ma_sound_group* pGroup);
11494 | MA_API void ma_sound_group_set_position(ma_sound_group* pGroup, float x, float y, float z);
11495 | MA_API ma_vec3f ma_sound_group_get_position(const ma_sound_group* pGroup);
11496 | MA_API void ma_sound_group_set_direction(ma_sound_group* pGroup, float x, float y, float z);
11497 | MA_API ma_vec3f ma_sound_group_get_direction(const ma_sound_group* pGroup);
11498 | MA_API void ma_sound_group_set_velocity(ma_sound_group* pGroup, float x, float y, float z);
11499 | MA_API ma_vec3f ma_sound_group_get_velocity(const ma_sound_group* pGroup);
11500 | MA_API void ma_sound_group_set_attenuation_model(ma_sound_group* pGroup, ma_attenuation_model
> attenuationModel);
11501 | MA_API ma_attenuation_model ma_sound_group_get_attenuation_model(const ma_sound_group* pGroup);
11502 | MA_API void ma_sound_group_set_positioning(ma_sound_group* pGroup, ma_positioning positioning);
11503 | MA_API ma_positioning ma_sound_group_get_positioning(const ma_sound_group* pGroup);
11504 | MA_API void ma_sound_group_set_rolloff(ma_sound_group* pGroup, float rolloff);
11505 | MA_API float ma_sound_group_get_rolloff(const ma_sound_group* pGroup);
11506 | MA_API void ma_sound_group_set_min_gain(ma_sound_group* pGroup, float minGain);
11507 | MA_API float ma_sound_group_get_min_gain(const ma_sound_group* pGroup);
11508 | MA_API void ma_sound_group_set_max_gain(ma_sound_group* pGroup, float maxGain);
11509 | MA_API float ma_sound_group_get_max_gain(const ma_sound_group* pGroup);
11510 | MA_API void ma_sound_group_set_min_distance(ma_sound_group* pGroup, float minDistance);
11511 | MA_API float ma_sound_group_get_min_distance(const ma_sound_group* pGroup);
11512 | MA_API void ma_sound_group_set_max_distance(ma_sound_group* pGroup, float maxDistance);
11513 | MA_API float ma_sound_group_get_max_distance(const ma_sound_group* pGroup);
11514 | MA_API void ma_sound_group_set_cone(ma_sound_group* pGroup, float innerAngleInRadians, float
> outerAngleInRadians, float outerGain);
11515 | MA_API void ma_sound_group_get_cone(const ma_sound_group* pGroup, float* pInnerAngleInRadians,
> float* pOuterAngleInRadians, float* pOuterGain);
11516 | MA_API void ma_sound_group_set_doppler_factor(ma_sound_group* pGroup, float dopplerFactor);
11517 | MA_API float ma_sound_group_get_doppler_factor(const ma_sound_group* pGroup);
11518 | MA_API void ma_sound_group_set_directional_attenuation_factor(ma_sound_group* pGroup, float
> directionalAttenuationFactor);
11519 | MA_API float ma_sound_group_get_directional_attenuation_factor(const ma_sound_group* pGroup);
11520 | MA_API void ma_sound_group_set_fade_in_pcm_frames(ma_sound_group* pGroup, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInFrames);
11521 | MA_API void ma_sound_group_set_fade_in_milliseconds(ma_sound_group* pGroup, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInMilliseconds);
11522 | MA_API float ma_sound_group_get_current_fade_volume(ma_sound_group* pGroup);
11523 | MA_API void ma_sound_group_set_start_time_in_pcm_frames(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInFrames);
11524 | MA_API void ma_sound_group_set_start_time_in_milliseconds(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInMilliseconds);
11525 | MA_API void ma_sound_group_set_stop_time_in_pcm_frames(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInFrames);
11526 | MA_API void ma_sound_group_set_stop_time_in_milliseconds(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInMilliseconds);
11527 | MA_API ma_bool32 ma_sound_group_is_playing(const ma_sound_group* pGroup);
11528 | MA_API ma_uint64 ma_sound_group_get_time_in_pcm_frames(const ma_sound_group* pGroup);
11529 | #endif /* MA_NO_ENGINE */
11530 | /* END SECTION: miniaudio_engine.h */
11531 |
11532 | #ifdef __cplusplus
11533 | }
11534 | #endif
11535 | #endif /* miniaudio_h */

```

```

11536 |
11537 |
11538 | /*
11539 | This is for preventing greying out of the implementation section.
11540 | */
11541 | #if defined(Q_CREATOR_RUN) || defined(__INTELLISENSE__) || defined(__CDT_PARSER__)
11542 | #define MINIAUDIO_IMPLEMENTATION
11543 | #endif
11544 |
11545 | /*****
> *****/
11546 | *****/
> *****/
11547 |
11548 | IMPLEMENTATION
11549 |
11550 | *****/
> *****/
11551 | *****/
> *****/
11552 | #if defined(MINIAUDIO_IMPLEMENTATION) || defined(MA_IMPLEMENTATION)
11553 | #ifndef miniaudio_c
11554 | #define miniaudio_c
11555 |
11556 | #include <assert.h>
11557 | #include <limits.h>          /* For INT_MAX */
11558 | #include <math.h>            /* sin(), etc. */
11559 | #include <stdlib.h>          /* For malloc(), free(), wcstombs(). */
11560 | #include <string.h>          /* For memset() */
11561 |
11562 | #include <stdarg.h>
11563 | #include <stdio.h>
11564 | #if !defined(_MSC_VER) && !defined(__DMC__)
11565 |     #include <strings.h>     /* For strcasecmp(). */
11566 |     #include <wchar.h>       /* For wcslen(), wcsrtombs() */
11567 | #endif
11568 | #ifdef _MSC_VER
11569 |     #include <float.h>       /* For _controlfp_s constants */
11570 | #endif
11571 |
11572 | #if defined(MA_WIN32)
11573 |     #include <windows.h>
11574 |
11575 |     /*
11576 |     There's a possibility that WIN32_LEAN_AND_MEAN has been defined which will exclude some symbols
11577 |     such as STGM_READ and CLSCTX_ALL. We need to check these and define them ourselves if they're
11578 |     unavailable.
11579 |     */
11580 |     #ifndef STGM_READ
11581 |     #define STGM_READ    0x00000000L
11582 |     #endif
11583 |     #ifndef CLSCTX_ALL
11584 |     #define CLSCTX_ALL    23
11585 |     #endif
11586 |
11587 |     /* IUnknown is used by both the WASAPI and DirectSound backends. It easier to just declare our
> version here. */
11588 |     typedef struct ma_IUnknown ma_IUnknown;
11589 | #endif
11590 |
11591 | #if !defined(MA_WIN32)
11592 |     #if !defined(MA_NO_THREADING)
11593 |         #include <sched.h>
11594 |         #include <pthread.h>    /* For pthreads. */
11595 |     #endif
11596 |
11597 |     #include <sys/time.h>       /* select() (used for ma_sleep()). */
11598 |     #include <time.h>          /* For nanosleep() */
11599 |     #include <unistd.h>
11600 | #endif
11601 |
11602 | /* For fstat(), etc. */
11603 | #if defined(MA_XBOX_NXDK)
11604 |     #include <stat.h>          /* Suggestion for NXDK: Add a sys/stat.h wrapper for compatibility. */
11605 | #else
11606 |     #include <sys/stat.h>
11607 | #endif
11608 |
11609 | #ifdef MA_EMScripten

```

```

11610 | #include <emscripten/emscripten.h>
11611 | #endif
11612 |
11613 |
11614 | /* Architecture Detection */
11615 | #if !defined(MA_64BIT) && !defined(MA_32BIT)
11616 | #ifdef _WIN32
11617 | #ifdef _WIN64
11618 | #define MA_64BIT
11619 | #else
11620 | #define MA_32BIT
11621 | #endif
11622 | #endif
11623 | #endif
11624 |
11625 | #if !defined(MA_64BIT) && !defined(MA_32BIT)
11626 | #ifdef __GNUC__
11627 | #ifdef __LP64__
11628 | #define MA_64BIT
11629 | #else
11630 | #define MA_32BIT
11631 | #endif
11632 | #endif
11633 | #endif
11634 |
11635 | #if !defined(MA_64BIT) && !defined(MA_32BIT)
11636 | #include <stdint.h>
11637 | #if INTPTR_MAX == INT64_MAX
11638 | #define MA_64BIT
11639 | #else
11640 | #define MA_32BIT
11641 | #endif
11642 | #endif
11643 |
11644 | #if defined(__arm__) || defined(_M_ARM)
11645 | #define MA_ARM32
11646 | #endif
11647 | #if defined(__arm64__) || defined(_arm64__) || defined(__aarch64__) || defined(_M_ARM64)
11648 | #define MA_ARM64
11649 | #endif
11650 |
11651 | #if defined(__x86_64__) || defined(_M_X64)
11652 | #define MA_X64
11653 | #elif defined(__i386__) || defined(_M_IX86)
11654 | #define MA_X86
11655 | #elif defined(MA_ARM32) || defined(MA_ARM64)
11656 | #define MA_ARM
11657 | #endif
11658 |
11659 | /* Intrinsics Support */
11660 | #if defined(MA_X64) || defined(MA_X86)
11661 |     #if defined(_MSC_VER) && !defined(__clang__)
11662 |         /* MSVC. */
11663 |         #if _MSC_VER >= 1400 && !defined(MA_NO_SSE2) /* 2005 */
11664 |             #define MA_SUPPORT_SSE2
11665 |         #endif
11666 |         /*#if _MSC_VER >= 1600 && !defined(MA_NO_AVX)*/ /* 2010 */
11667 |         /* #define MA_SUPPORT_AVX*/
11668 |         /*#endif*/
11669 |         #if _MSC_VER >= 1700 && !defined(MA_NO_AVX2) /* 2012 */
11670 |             #define MA_SUPPORT_AVX2
11671 |         #endif
11672 |     #else
11673 |         /* Assume GNUC-style. */
11674 |         #if defined(__SSE2__) && !defined(MA_NO_SSE2)
11675 |             #define MA_SUPPORT_SSE2
11676 |         #endif
11677 |         /*#if defined(__AVX__) && !defined(MA_NO_AVX)*/
11678 |         /* #define MA_SUPPORT_AVX*/
11679 |         /*#endif*/
11680 |         #if defined(__AVX2__) && !defined(MA_NO_AVX2)
11681 |             #define MA_SUPPORT_AVX2
11682 |         #endif
11683 |     #endif
11684 |
11685 |     /* If at this point we still haven't determined compiler support for the intrinsics just fall
> back to __has_include. */
11686 |     #if !defined(__GNUC__) && !defined(__clang__) && defined(__has_include)
11687 |         #if !defined(MA_SUPPORT_SSE2) && !defined(MA_NO_SSE2) && __has_include(<emmintrin.h>)

```

```

11688 |         #define MA_SUPPORT_SSE2
11689 |     #endif
11690 |     /*#if !defined(MA_SUPPORT_AVX)    && !defined(MA_NO_AVX)    &&
> __has_include(<immintrin.h>)*//
11691 |     /* #define MA_SUPPORT_AVX*/
11692 |     /*#endif*/
11693 |     #if !defined(MA_SUPPORT_AVX2)    && !defined(MA_NO_AVX2)    && __has_include(<immintrin.h>)
11694 |         #define MA_SUPPORT_AVX2
11695 |     #endif
11696 | #endif
11697 |
11698 |     #if defined(MA_SUPPORT_AVX2) || defined(MA_SUPPORT_AVX)
11699 |         #include <immintrin.h>
11700 |     #elif defined(MA_SUPPORT_SSE2)
11701 |         #include <emmintrin.h>
11702 |     #endif
11703 | #endif
11704 |
11705 | #if defined(MA_ARM)
11706 |     #if !defined(MA_NO_NEON) && (defined(__ARM_NEON) || defined(__aarch64__) || defined(_M_ARM64))
11707 |         #define MA_SUPPORT_NEON
11708 |         #include <arm_neon.h>
11709 |     #endif
11710 | #endif
11711 |
11712 | /* Begin globally disabled warnings. */
11713 | #if defined(_MSC_VER)
11714 |     #pragma warning(push)
11715 |     #pragma warning(disable:4752) /* found Intel(R) Advanced Vector Extensions; consider using
> /arch:AVX */
11716 |     #pragma warning(disable:4049) /* compiler limit : terminating line number emission */
11717 | #endif
11718 |
11719 | #if defined(MA_X64) || defined(MA_X86)
11720 |     #if defined(_MSC_VER) && !defined(__clang__)
11721 |         #if _MSC_VER >= 1400
11722 |             #include <intrin.h>
11723 |             static MA_INLINE void ma_cpuid(int info[4], int fid)
11724 |             {
11725 |                 __cpuid(info, fid);
11726 |             }
11727 |         #else
11728 |             #define MA_NO_CPUID
11729 |         #endif
11730 |
11731 |         #if _MSC_VER >= 1600 && (defined(_MSC_FULL_VER) && _MSC_FULL_VER >= 160040219)
11732 |             static MA_INLINE unsigned __int64 ma_xgetbv(int reg)
11733 |             {
11734 |                 return _xgetbv(reg);
11735 |             }
11736 |         #else
11737 |             #define MA_NO_XGETBV
11738 |         #endif
11739 |     #elif (defined(__GNUC__) || defined(__clang__)) && !defined(MA_ANDROID)
11740 |         static MA_INLINE void ma_cpuid(int info[4], int fid)
11741 |         {
11742 |             /*
11743 |             > work around this by just using a different register, the
11744 |             > specific register of which I'm letting the compiler decide on. The "k" prefix is used to
11745 |             > specify a 32-bit register. The {...} syntax is for
11746 |             > supporting different assembly dialects.
11747 |             >
11748 |             > What's basically happening is that we're saving and restoring the ebx register manually.
11749 |             > */
11750 |             #if defined(MA_X86) && defined(__PIC__)
11751 |                 __asm__ __volatile__ (
11752 |                     "xchg{l} {%%}ebx, %k1;"
11753 |                     "cpuid;"
11754 |                     "xchg{l} {%%}ebx, %k1;"
11755 |                     : "=a"(info[0]), "=r"(info[1]), "=c"(info[2]), "=d"(info[3]) : "a"(fid), "c"(0)
11756 |                     );
11757 |             #else
11758 |                 __asm__ __volatile__ (
11759 |                     "cpuid" : "=a"(info[0]), "=b"(info[1]), "=c"(info[2]), "=d"(info[3]) : "a"(fid),
11760 |                     > "c"(0)
11761 |                     );
11762 |             #endif
11763 |         }

```



```

11762 |
11763 |     static MA_INLINE ma_uint64 ma_xgetbv(int reg)
11764 |     {
11765 |         unsigned int hi;
11766 |         unsigned int lo;
11767 |
11768 |         __asm__ __volatile__ (
11769 |             "xgetbv" : "=a"(lo), "=d"(hi) : "c"(reg)
11770 |         );
11771 |
11772 |         return ((ma_uint64)hi << 32) | (ma_uint64)lo;
11773 |     }
11774 | #else
11775 |     #define MA_NO_CPUID
11776 |     #define MA_NO_XGETBV
11777 | #endif
11778 | #else
11779 |     #define MA_NO_CPUID
11780 |     #define MA_NO_XGETBV
11781 | #endif
11782 |
11783 | static MA_INLINE ma_bool32 ma_has_sse2(void)
11784 | {
11785 |     #if defined(MA_SUPPORT_SSE2)
11786 |         #if (defined(MA_X64) || defined(MA_X86)) && !defined(MA_NO_SSE2)
11787 |             #if defined(MA_X64)
11788 |                 return MA_TRUE; /* 64-bit targets always support SSE2. */
11789 |             #elif (defined(_M_IX86_FP) && _M_IX86_FP == 2) || defined(__SSE2__)
11790 |                 return MA_TRUE; /* If the compiler is allowed to freely generate SSE2 code we can
> assume support. */
11791 |             #else
11792 |                 #if defined(MA_NO_CPUID)
11793 |                     return MA_FALSE;
11794 |                 #else
11795 |                     int info[4];
11796 |                     ma_cpuid(info, 1);
11797 |                     return (info[3] & (1 << 26)) != 0;
11798 |                 #endif
11799 |             #endif
11800 |         #else
11801 |             return MA_FALSE; /* SSE2 is only supported on x86 and x64 architectures. */
11802 |         #endif
11803 |     #else
11804 |         return MA_FALSE; /* No compiler support. */
11805 |     #endif
11806 | }
11807 |
11808 | #if 0
11809 | static MA_INLINE ma_bool32 ma_has_avx()
11810 | {
11811 |     #if defined(MA_SUPPORT_AVX)
11812 |         #if (defined(MA_X64) || defined(MA_X86)) && !defined(MA_NO_AVX)
11813 |             #if defined(__AVX__) || defined(_AVX_)
11814 |                 return MA_TRUE; /* If the compiler is allowed to freely generate AVX code we can
> assume support. */
11815 |             #else
11816 |                 /* AVX requires both CPU and OS support. */
11817 |                 #if defined(MA_NO_CPUID) || defined(MA_NO_XGETBV)
11818 |                     return MA_FALSE;
11819 |                 #else
11820 |                     int info[4];
11821 |                     ma_cpuid(info, 1);
11822 |                     if (((info[2] & (1 << 27)) != 0) && ((info[2] & (1 << 28)) != 0)) {
11823 |                         ma_uint64 xrc = ma_xgetbv(0);
11824 |                         if ((xrc & 0x06) == 0x06) {
11825 |                             return MA_TRUE;
11826 |                         } else {
11827 |                             return MA_FALSE;
11828 |                         }
11829 |                     } else {
11830 |                         return MA_FALSE;
11831 |                     }
11832 |                 #endif
11833 |             #endif
11834 |         #else
11835 |             return MA_FALSE; /* AVX is only supported on x86 and x64 architectures. */
11836 |         #endif
11837 |     #else
11838 |         return MA_FALSE; /* No compiler support. */

```



```

11839 | #endif
11840 | }
11841 | #endif
11842 |
11843 | static MA_INLINE ma_bool32 ma_has_avx2(void)
11844 | {
11845 |     #if defined(MA_SUPPORT_AVX2)
11846 |         #if (defined(MA_X64) || defined(MA_X86)) && !defined(MA_NO_AVX2)
11847 |             #if defined(_AVX2_) || defined(__AVX2__)
11848 |                 return MA_TRUE; /* If the compiler is allowed to freely generate AVX2 code we can
> assume support. */
11849 |             #else
11850 |                 /* AVX2 requires both CPU and OS support. */
11851 |                 #if defined(MA_NO_CPUID) || defined(MA_NO_XGETBV)
11852 |                     return MA_FALSE;
11853 |                 #else
11854 |                     int info1[4];
11855 |                     int info7[4];
11856 |                     ma_cpuid(info1, 1);
11857 |                     ma_cpuid(info7, 7);
11858 |                     if (((info1[2] & (1 << 27)) != 0) && ((info7[1] & (1 << 5)) != 0)) {
11859 |                         ma_uint64 xrc = ma_xgetbv(0);
11860 |                         if ((xrc & 0x06) == 0x06) {
11861 |                             return MA_TRUE;
11862 |                         } else {
11863 |                             return MA_FALSE;
11864 |                         }
11865 |                     } else {
11866 |                         return MA_FALSE;
11867 |                     }
11868 |                 #endif
11869 |             #endif
11870 |         #else
11871 |             return MA_FALSE; /* AVX2 is only supported on x86 and x64 architectures. */
11872 |         #endif
11873 |     #else
11874 |         return MA_FALSE; /* No compiler support. */
11875 |     #endif
11876 | }
11877 |
11878 | static MA_INLINE ma_bool32 ma_has_neon(void)
11879 | {
11880 |     #if defined(MA_SUPPORT_NEON)
11881 |         #if defined(MA_ARM) && !defined(MA_NO_NEON)
11882 |             #if (defined(__ARM_NEON) || defined(__aarch64__) || defined(_M_ARM64))
11883 |                 return MA_TRUE; /* If the compiler is allowed to freely generate NEON code we can
> assume support. */
11884 |             #else
11885 |                 /* TODO: Runtime check. */
11886 |                 return MA_FALSE;
11887 |             #endif
11888 |         #else
11889 |             return MA_FALSE; /* NEON is only supported on ARM architectures. */
11890 |         #endif
11891 |     #else
11892 |         return MA_FALSE; /* No compiler support. */
11893 |     #endif
11894 | }
11895 |
11896 | #if defined(__has_builtin)
11897 |     #define MA_COMPILER_HAS_BUILTIN(x) __has_builtin(x)
11898 | #else
11899 |     #define MA_COMPILER_HAS_BUILTIN(x) 0
11900 | #endif
11901 |
11902 | #ifndef MA_ASSUME
11903 |     #if MA_COMPILER_HAS_BUILTIN(__builtin_assume)
11904 |         #define MA_ASSUME(x) __builtin_assume(x)
11905 |     #elif MA_COMPILER_HAS_BUILTIN(__builtin_unreachable)
11906 |         #define MA_ASSUME(x) do { if (!x) __builtin_unreachable(); } while (0)
11907 |     #elif defined(_MSC_VER)
11908 |         #define MA_ASSUME(x) __assume(x)
11909 |     #else
11910 |         #define MA_ASSUME(x) (void)(x)
11911 |     #endif
11912 | #endif
11913 |
11914 | #ifndef MA_RESTRICT
11915 |     #if defined(__clang__) || defined(_MSC_VER) || (defined(__GNUC__) && (__GNUC__ > 2 || (__GNUC__

```

```

> == 2 && __GNUC_MINOR__ >= 95)))
11916 |     #define MA_RESTRICT __restrict
11917 |     #else
11918 |     #define MA_RESTRICT
11919 |     #endif
11920 | #endif
11921 |
11922 | #if defined(_MSC_VER) && _MSC_VER >= 1400
11923 |     #define MA_HAS_BYTESWAP16_INTRINSIC
11924 |     #define MA_HAS_BYTESWAP32_INTRINSIC
11925 |     #define MA_HAS_BYTESWAP64_INTRINSIC
11926 | #elif defined(__clang__)
11927 |     #if MA_COMPILER_HAS_BUILTIN(__builtin_bswap16)
11928 |     #define MA_HAS_BYTESWAP16_INTRINSIC
11929 |     #endif
11930 |     #if MA_COMPILER_HAS_BUILTIN(__builtin_bswap32)
11931 |     #define MA_HAS_BYTESWAP32_INTRINSIC
11932 |     #endif
11933 |     #if MA_COMPILER_HAS_BUILTIN(__builtin_bswap64)
11934 |     #define MA_HAS_BYTESWAP64_INTRINSIC
11935 |     #endif
11936 | #elif defined(__GNUC__)
11937 |     #if ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 3))
11938 |     #define MA_HAS_BYTESWAP32_INTRINSIC
11939 |     #define MA_HAS_BYTESWAP64_INTRINSIC
11940 |     #endif
11941 |     #if ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 8))
11942 |     #define MA_HAS_BYTESWAP16_INTRINSIC
11943 |     #endif
11944 | #endif
11945 |
11946 |
11947 | static MA_INLINE ma_bool32 ma_is_little_endian(void)
11948 | {
11949 |     #if defined(MA_X86) || defined(MA_X64)
11950 |     return MA_TRUE;
11951 |     #else
11952 |     int n = 1;
11953 |     return (*(char*)&n) == 1;
11954 |     #endif
11955 | }
11956 |
11957 | static MA_INLINE ma_bool32 ma_is_big_endian(void)
11958 | {
11959 |     return !ma_is_little_endian();
11960 | }
11961 |
11962 |
11963 | static MA_INLINE ma_uint32 ma_swap_endian_uint32(ma_uint32 n)
11964 | {
11965 |     #ifndef MA_HAS_BYTESWAP32_INTRINSIC
11966 |     #if defined(_MSC_VER)
11967 |     return _byteswap_ulong(n);
11968 |     #elif defined(__GNUC__) || defined(__clang__)
11969 |     #if defined(MA_ARM) && (defined(__ARM_ARCH) && __ARM_ARCH >= 6) && !defined(MA_64BIT) /*
> <-- 64-bit inline assembly has not been tested, so disabling for now. */
11970 |     /* Inline assembly optimized implementation for ARM. In my testing, GCC does not
> generate optimized code with __builtin_bswap32(). */
11971 |     ma_uint32 r;
11972 |     __asm__ __volatile__ (
11973 |         #if defined(MA_64BIT)
11974 |         "rev %w[out], %w[in]" : [out]"=r"(r) : [in]"r"(n) /* <-- This is untested. If
> someone in the community could test this, that would be appreciated! */
11975 |         #else
11976 |         "rev %w[out], %[in]" : [out]"=r"(r) : [in]"r"(n)
11977 |         #endif
11978 |         );
11979 |     return r;
11980 |     #else
11981 |     return __builtin_bswap32(n);
11982 |     #endif
11983 |     #else
11984 |     #error "This compiler does not support the byte swap intrinsic."
11985 |     #endif
11986 |     #else
11987 |     return ((n & 0xFF000000) >> 24) |
11988 |         ((n & 0x00FF0000) >> 8) |
11989 |         ((n & 0x0000FF00) << 8) |
11990 |         ((n & 0x000000FF) << 24);

```

```

11991 | #endif
11992 | }
11993 |
11994 |
11995 | #if !defined(MA_EMSCRIPTEN)
11996 | #ifdef MA_WIN32
11997 | static void ma_sleep__win32(ma_uint32 milliseconds)
11998 | {
11999 |     Sleep((DWORD)milliseconds);
12000 | }
12001 | #endif
12002 | #ifdef MA_POSIX
12003 | static void ma_sleep__posix(ma_uint32 milliseconds)
12004 | {
12005 |     #ifdef MA_EMSCRIPTEN
12006 |         (void)milliseconds;
12007 |         MA_ASSERT(MA_FALSE); /* The Emscripten build should never sleep. */
12008 |     #else
12009 |         #if (defined(_POSIX_C_SOURCE) && _POSIX_C_SOURCE >= 199309L) || defined(MA_SWITCH)
12010 |             struct timespec ts;
12011 |             ts.tv_sec = milliseconds / 1000;
12012 |             ts.tv_nsec = milliseconds % 1000 * 1000000;
12013 |             nanosleep(&ts, NULL);
12014 |         #else
12015 |             struct timeval tv;
12016 |             tv.tv_sec = milliseconds / 1000;
12017 |             tv.tv_usec = milliseconds % 1000 * 1000;
12018 |             select(0, NULL, NULL, NULL, &tv);
12019 |         #endif
12020 |     #endif
12021 | }
12022 | #endif
12023 |
12024 | static MA_INLINE void ma_sleep(ma_uint32 milliseconds)
12025 | {
12026 |     #ifdef MA_WIN32
12027 |         ma_sleep__win32(milliseconds);
12028 |     #endif
12029 |     #ifdef MA_POSIX
12030 |         ma_sleep__posix(milliseconds);
12031 |     #endif
12032 | }
12033 | #endif
12034 |
12035 | static MA_INLINE void ma_yield(void)
12036 | {
12037 |     #if defined(__i386) || defined(_M_IX86) || defined(__x86_64__) || defined(_M_X64)
12038 |         /* x86/x64 */
12039 |         #if (defined(_MSC_VER) || defined(__WATCOMC__) || defined(__DMC__)) && !defined(__clang__)
12040 |             #if _MSC_VER >= 1400
12041 |                 __mm_pause();
12042 |             #else
12043 |                 #if defined(__DMC__)
12044 |                     /* Digital Mars does not recognize the PAUSE opcode. Fall back to NOP. */
12045 |                     __asm nop;
12046 |                 #else
12047 |                     __asm pause;
12048 |                 #endif
12049 |             #endif
12050 |         #else
12051 |             __asm __volatile__ ("rep; nop");
12052 |         #endif
12053 |     #elif (defined(__arm__) && defined(__ARM_ARCH) && __ARM_ARCH >= 7) || defined(_M_ARM64) ||
12054 | > (defined(_M_ARM) && _M_ARM >= 7) || defined(__ARM_ARCH_6K__) || defined(__ARM_ARCH_6T2__)
12055 |         /* ARM */
12056 |         #if defined(_MSC_VER)
12057 |             /* Apparently there is a __yield() intrinsic that's compatible with ARM, but I cannot find
12058 | > documentation for it nor can I find where it's declared. */
12059 |             __yield();
12060 |         #else
12061 |             __asm __volatile__ ("yield"); /* ARMv6K/ARMv6T2 and above. */
12062 |         #endif
12063 |     #else
12064 |         /* Unknown or unsupported architecture. No-op. */
12065 |     #endif
12066 | }
12067 | #define MA_MM_DENORMALS_ZERO_MASK    0x0040

```

```

12068 | #define MA_MM_FLUSH_ZERO_MASK      0x8000
12069 |
12070 | static MA_INLINE unsigned int ma_disable_denormals(void)
12071 | {
12072 |     unsigned int prevState;
12073 |
12074 |     #if defined(_MSC_VER) && !defined(MA_XBOX_NXDK)
12075 |     {
12076 |         /*
12077 |         Older versions of Visual Studio don't support the "safe" versions of _controlfp_s(). I don't
12078 |         know which version of Visual Studio first added support for _controlfp_s(), but I do know
12079 |         that VC6 lacks support. _MSC_VER = 1200 is VC6, but if you get compilation errors on older
12080 |         versions of Visual Studio, let me know and I'll make the necessary adjustment.
12081 |         */
12082 |         #if _MSC_VER <= 1200
12083 |         {
12084 |             prevState = _statusfp();
12085 |             _controlfp(prevState | _DN_FLUSH, _MCW_DN);
12086 |         }
12087 |         #else
12088 |         {
12089 |             unsigned int unused;
12090 |             _controlfp_s(&prevState, 0, 0);
12091 |             _controlfp_s(&unused, prevState | _DN_FLUSH, _MCW_DN);
12092 |         }
12093 |         #endif
12094 |     }
12095 |     #elif defined(MA_X86) || defined(MA_X64)
12096 |     {
12097 |         #if defined(MA_SUPPORT_SSE2) && defined(__SSE2__) && !(defined(__TINYC__) ||
> defined(__WATCOMC__)) /* <-- Add compilers that lack support for _mm_getcsr() and _mm_setcsr() to
> this list. */
12098 |         {
12099 |             prevState = _mm_getcsr();
12100 |             _mm_setcsr(prevState | MA_MM_DENORMALS_ZERO_MASK | MA_MM_FLUSH_ZERO_MASK);
12101 |         }
12102 |         #else
12103 |         {
12104 |             /* x86/64, but no support for _mm_getcsr()/_mm_setcsr(). May need to fall back to
> inlined assembly here. */
12105 |             prevState = 0;
12106 |         }
12107 |         #endif
12108 |     }
12109 |     #else
12110 |     {
12111 |         /* Unknown or unsupported architecture. No-op. */
12112 |         prevState = 0;
12113 |     }
12114 |     #endif
12115 |
12116 |     return prevState;
12117 | }
12118 |
12119 | static MA_INLINE void ma_restore_denormals(unsigned int prevState)
12120 | {
12121 |     #if defined(_MSC_VER) && !defined(MA_XBOX_NXDK)
12122 |     {
12123 |         /* Older versions of Visual Studio do not support _controlfp_s(). See
> ma_disable_denormals(). */
12124 |         #if _MSC_VER <= 1200
12125 |         {
12126 |             _controlfp(prevState, _MCW_DN);
12127 |         }
12128 |         #else
12129 |         {
12130 |             unsigned int unused;
12131 |             _controlfp_s(&unused, prevState, _MCW_DN);
12132 |         }
12133 |         #endif
12134 |     }
12135 |     #elif defined(MA_X86) || defined(MA_X64)
12136 |     {
12137 |         #if defined(MA_SUPPORT_SSE2) && defined(__SSE2__) && !(defined(__TINYC__) ||
> defined(__WATCOMC__)) /* <-- Add compilers that lack support for _mm_getcsr() and _mm_setcsr() to
> this list. */
12138 |         {
12139 |             _mm_setcsr(prevState);
12140 |         }

```

```

12141 |         #else
12142 |         {
12143 |             /* x88/64, but no support for _mm_getcsr()/_mm_setcsr(). May need to fall back to
> inlined assembly here. */
12144 |             (void)prevState;
12145 |         }
12146 |     #endif
12147 | }
12148 | #else
12149 | {
12150 |     /* Unknown or unsupported architecture. No-op. */
12151 |     (void)prevState;
12152 | }
12153 | #endif
12154 | }
12155 |
12156 |
12157 | #ifndef MA_ANDROID
12158 | #include <sys/system_properties.h>
12159 |
12160 | int ma_android_sdk_version()
12161 | {
12162 |     char sdkVersion[PROP_VALUE_MAX + 1] = {0, };
12163 |     if (__system_property_get("ro.build.version.sdk", sdkVersion)) {
12164 |         return atoi(sdkVersion);
12165 |     }
12166 |
12167 |     return 0;
12168 | }
12169 | #endif
12170 |
12171 |
12172 | #ifndef MA_COINIT_VALUE
12173 | #define MA_COINIT_VALUE    0    /* 0 = COINIT_MULTITHREADED */
12174 | #endif
12175 |
12176 |
12177 | #ifndef MA_FLT_MAX
12178 |     #ifndef FLT_MAX
12179 |         #define MA_FLT_MAX FLT_MAX
12180 |     #else
12181 |         #define MA_FLT_MAX 3.402823466e+38F
12182 |     #endif
12183 | #endif
12184 |
12185 |
12186 | #ifndef MA_PI
12187 | #define MA_PI    3.14159265358979323846264f
12188 | #endif
12189 | #ifndef MA_PI_D
12190 | #define MA_PI_D    3.14159265358979323846264
12191 | #endif
12192 | #ifndef MA_TAU
12193 | #define MA_TAU    6.28318530717958647693f
12194 | #endif
12195 | #ifndef MA_TAU_D
12196 | #define MA_TAU_D    6.28318530717958647693
12197 | #endif
12198 |
12199 |
12200 | /* The default format when ma_format_unknown (0) is requested when initializing a device. */
12201 | #ifndef MA_DEFAULT_FORMAT
12202 | #define MA_DEFAULT_FORMAT    ma_format_f32
12203 | #endif
12204 |
12205 | /* The default channel count to use when 0 is used when initializing a device. */
12206 | #ifndef MA_DEFAULT_CHANNELS
12207 | #define MA_DEFAULT_CHANNELS    2
12208 | #endif
12209 |
12210 | /* The default sample rate to use when 0 is used when initializing a device. */
12211 | #ifndef MA_DEFAULT_SAMPLE_RATE
12212 | #define MA_DEFAULT_SAMPLE_RATE    48000
12213 | #endif
12214 |
12215 | /* Default periods when none is specified in ma_device_init(). More periods means more work on the
> CPU. */
12216 | #ifndef MA_DEFAULT_PERIODS
12217 | #define MA_DEFAULT_PERIODS    3

```

```

12218 | #endif
12219 |
12220 | /* The default period size in milliseconds for low latency mode. */
12221 | #ifndef MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_LOW_LATENCY
12222 | #define MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_LOW_LATENCY 10
12223 | #endif
12224 |
12225 | /* The default buffer size in milliseconds for conservative mode. */
12226 | #ifndef MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_CONSERVATIVE
12227 | #define MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_CONSERVATIVE 100
12228 | #endif
12229 |
12230 | /* The default LPF filter order for linear resampling. Note that this is clamped to
> MA_MAX_FILTER_ORDER. */
12231 | #ifndef MA_DEFAULT_RESAMPLER_LPF_ORDER
12232 |     #if MA_MAX_FILTER_ORDER >= 4
12233 |         #define MA_DEFAULT_RESAMPLER_LPF_ORDER 4
12234 |     #else
12235 |         #define MA_DEFAULT_RESAMPLER_LPF_ORDER MA_MAX_FILTER_ORDER
12236 |     #endif
12237 | #endif
12238 |
12239 |
12240 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
> 6)))
12241 |     #pragma GCC diagnostic push
12242 |     #pragma GCC diagnostic ignored "-Wunused-variable"
12243 | #endif
12244 |
12245 | /* Standard sample rates, in order of priority. */
12246 | static ma_uint32 g_maStandardSampleRatePriorities[] = {
12247 |     (ma_uint32)ma_standard_sample_rate_48000,
12248 |     (ma_uint32)ma_standard_sample_rate_44100,
12249 |
12250 |     (ma_uint32)ma_standard_sample_rate_32000,
12251 |     (ma_uint32)ma_standard_sample_rate_24000,
12252 |     (ma_uint32)ma_standard_sample_rate_22050,
12253 |
12254 |     (ma_uint32)ma_standard_sample_rate_88200,
12255 |     (ma_uint32)ma_standard_sample_rate_96000,
12256 |     (ma_uint32)ma_standard_sample_rate_176400,
12257 |     (ma_uint32)ma_standard_sample_rate_192000,
12258 |
12259 |     (ma_uint32)ma_standard_sample_rate_16000,
12260 |     (ma_uint32)ma_standard_sample_rate_11025,
12261 |     (ma_uint32)ma_standard_sample_rate_8000,
12262 |
12263 |     (ma_uint32)ma_standard_sample_rate_352800,
12264 |     (ma_uint32)ma_standard_sample_rate_384000
12265 | };
12266 |
12267 | static MA_INLINE ma_bool32 ma_is_standard_sample_rate(ma_uint32 sampleRate)
12268 | {
12269 |     ma_uint32 iSampleRate;
12270 |
12271 |     for (iSampleRate = 0; iSampleRate < sizeof(g_maStandardSampleRatePriorities) /
> sizeof(g_maStandardSampleRatePriorities[0]); iSampleRate += 1) {
12272 |         if (g_maStandardSampleRatePriorities[iSampleRate] == sampleRate) {
12273 |             return MA_TRUE;
12274 |         }
12275 |     }
12276 |
12277 |     /* Getting here means the sample rate is not supported. */
12278 |     return MA_FALSE;
12279 | }
12280 |
12281 |
12282 | static ma_format g_maFormatPriorities[] = {
12283 |     ma_format_s16,          /* Most common */
12284 |     ma_format_f32,
12285 |
12286 |     /*ma_format_s24_32,*/    /* Clean alignment */
12287 |     ma_format_s32,
12288 |
12289 |     ma_format_s24,          /* Unclean alignment */
12290 |
12291 |     ma_format_u8            /* Low quality */
12292 | };
12293 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=

```

```

> 6)))
12294 | #pragma GCC diagnostic pop
12295 | #endif
12296 |
12297 |
12298 | MA_API void ma_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision)
12299 | {
12300 |     if (pMajor) {
12301 |         *pMajor = MA_VERSION_MAJOR;
12302 |     }
12303 |
12304 |     if (pMinor) {
12305 |         *pMinor = MA_VERSION_MINOR;
12306 |     }
12307 |
12308 |     if (pRevision) {
12309 |         *pRevision = MA_VERSION_REVISION;
12310 |     }
12311 | }
12312 |
12313 | MA_API const char* ma_version_string(void)
12314 | {
12315 |     return MA_VERSION_STRING;
12316 | }
12317 |
12318 |
12319 | /*****
12320 |
12321 | Standard Library Stuff
12322 |
12323 | *****/
12324 | #ifndef MA_ASSERT
12325 | #define MA_ASSERT(condition)          assert(condition)
12326 | #endif
12327 |
12328 | #ifndef MA_MALLOC
12329 | #define MA_MALLOC(sz)                 malloc((sz))
12330 | #endif
12331 | #ifndef MA_REALLOC
12332 | #define MA_REALLOC(p, sz)             realloc((p), (sz))
12333 | #endif
12334 | #ifndef MA_FREE
12335 | #define MA_FREE(p)                   free((p))
12336 | #endif
12337 |
12338 | static MA_INLINE void ma_zero_memory_default(void* p, size_t sz)
12339 | {
12340 |     if (p == NULL) {
12341 |         MA_ASSERT(sz == 0); /* If this is triggered there's an error with the calling code. */
12342 |         return;
12343 |     }
12344 |
12345 |     if (sz > 0) {
12346 |         memset(p, 0, sz);
12347 |     }
12348 | }
12349 |
12350 |
12351 | #ifndef MA_ZERO_MEMORY
12352 | #define MA_ZERO_MEMORY(p, sz)         ma_zero_memory_default((p), (sz))
12353 | #endif
12354 | #ifndef MA_COPY_MEMORY
12355 | #define MA_COPY_MEMORY(dst, src, sz)  memcpy((dst), (src), (sz))
12356 | #endif
12357 | #ifndef MA_MOVE_MEMORY
12358 | #define MA_MOVE_MEMORY(dst, src, sz)  memmove((dst), (src), (sz))
12359 | #endif
12360 |
12361 | #define MA_ZERO_OBJECT(p)             MA_ZERO_MEMORY((p), sizeof(*(p)))
12362 |
12363 | #define ma_countof(x)                  (sizeof(x) / sizeof(x[0]))
12364 | #define ma_max(x, y)                   (((x) > (y)) ? (x) : (y))
12365 | #define ma_min(x, y)                   (((x) < (y)) ? (x) : (y))
12366 | #define ma_abs(x)                      (((x) > 0) ? (x) : -(x))
12367 | #define ma_clamp(x, lo, hi)            (ma_max(lo, ma_min(x, hi)))
12368 | #define ma_offset_ptr(p, offset)      (((ma_uint8*)(p)) + (offset))
12369 | #define ma_align(x, a)                (((x) + ((a)-1)) & ~((a)-1))
12370 | #define ma_align_64(x)                ma_align(x, 8)
12371 |

```



```

12372 | #define ma_buffer_frame_capacity(buffer, channels, format) (sizeof(buffer) /
> ma_get_bytes_per_sample(format) / (channels))
12373 |
12374 | static MA_INLINE double ma_sind(double x)
12375 | {
12376 |     /* TODO: Implement custom sin(x). */
12377 |     return sin(x);
12378 | }
12379 |
12380 | static MA_INLINE double ma_expd(double x)
12381 | {
12382 |     /* TODO: Implement custom exp(x). */
12383 |     return exp(x);
12384 | }
12385 |
12386 | static MA_INLINE double ma_logd(double x)
12387 | {
12388 |     /* TODO: Implement custom log(x). */
12389 |     return log(x);
12390 | }
12391 |
12392 | static MA_INLINE double ma_powd(double x, double y)
12393 | {
12394 |     /* TODO: Implement custom pow(x, y). */
12395 |     return pow(x, y);
12396 | }
12397 |
12398 | static MA_INLINE double ma_sqrtd(double x)
12399 | {
12400 |     /* TODO: Implement custom sqrt(x). */
12401 |     return sqrt(x);
12402 | }
12403 |
12404 |
12405 | static MA_INLINE float ma_rsqrtf(float x)
12406 | {
12407 |     #if defined(MA_SUPPORT_SSE2) && !defined(MA_NO_SSE2) && (defined(MA_X64) || (defined(_M_IX86_FP)
> && _M_IX86_FP == 2) || defined(__SSE2__))
12408 |     {
12409 |         /*
12410 |         For SSE we can use RSQRTSS.
12411 |
12412 |         This Stack Overflow post suggests that compilers don't necessarily generate optimal code
12413 |         when using intrinsics:
12414 |
12415 |         https://web.archive.org/web/20221211012522/https://stackoverflow.com/questions/32687079/
> getting-fewest-instructions-for-rsqrtss-wrapper
12416 |
12417 |         I'm going to do something similar here, but a bit simpler.
12418 |         */
12419 |         #if defined(__GNUC__) || defined(__clang__)
12420 |         {
12421 |             float result;
12422 |             __asm__ __volatile__ ("rsqrtss %1, %0" : "=x"(result) : "x"(x));
12423 |             return result;
12424 |         }
12425 |         #else
12426 |         {
12427 |             return _mm_cvtss_f32(_mm_rsqrt_ss(_mm_set_ps1(x)));
12428 |         }
12429 |         #endif
12430 |     }
12431 |     #else
12432 |     {
12433 |         return 1 / (float)ma_sqrtd(x);
12434 |     }
12435 |     #endif
12436 | }
12437 |
12438 |
12439 | static MA_INLINE float ma_sinf(float x)
12440 | {
12441 |     return (float)ma_sind((float)x);
12442 | }
12443 |
12444 | static MA_INLINE double ma_cosd(double x)
12445 | {
12446 |     return ma_sind((MA_PI_D*0.5) - x);
12447 | }

```

```

12448 |
12449 | static MA_INLINE float ma_cosf(float x)
12450 | {
12451 |     return (float)ma_cosd((float)x);
12452 | }
12453 |
12454 | static MA_INLINE double ma_log10d(double x)
12455 | {
12456 |     return ma_logd(x) * 0.43429448190325182765;
12457 | }
12458 |
12459 | static MA_INLINE float ma_powf(float x, float y)
12460 | {
12461 |     return (float)ma_powd((double)x, (double)y);
12462 | }
12463 |
12464 | static MA_INLINE float ma_log10f(float x)
12465 | {
12466 |     return (float)ma_log10d((double)x);
12467 | }
12468 |
12469 |
12470 | static MA_INLINE double ma_degrees_to_radians(double degrees)
12471 | {
12472 |     return degrees * 0.01745329252;
12473 | }
12474 |
12475 | static MA_INLINE double ma_radians_to_degrees(double radians)
12476 | {
12477 |     return radians * 57.295779512896;
12478 | }
12479 |
12480 | static MA_INLINE float ma_degrees_to_radians_f(float degrees)
12481 | {
12482 |     return degrees * 0.01745329252f;
12483 | }
12484 |
12485 | static MA_INLINE float ma_radians_to_degrees_f(float radians)
12486 | {
12487 |     return radians * 57.295779512896f;
12488 | }
12489 |
12490 |
12491 | /*
12492 | Return Values:
12493 | 0: Success
12494 | 22: EINVAL
12495 | 34: ERANGE
12496 |
12497 | Not using symbolic constants for errors because I want to avoid #including errno.h
12498 |
12499 | These are marked as no-inline because of some bad code generation by Clang. None of these functions
12500 | are used in any performance-critical code within miniaudio.
12501 | */
12502 | MA_API MA_NO_INLINE int ma_strcpy_s(char* dst, size_t dstSizeInBytes, const char* src)
12503 | {
12504 |     size_t i;
12505 |
12506 |     if (dst == 0) {
12507 |         return 22;
12508 |     }
12509 |     if (dstSizeInBytes == 0) {
12510 |         return 34;
12511 |     }
12512 |     if (src == 0) {
12513 |         dst[0] = '\0';
12514 |         return 22;
12515 |     }
12516 |
12517 |     for (i = 0; i < dstSizeInBytes && src[i] != '\0'; ++i) {
12518 |         dst[i] = src[i];
12519 |     }
12520 |
12521 |     if (i < dstSizeInBytes) {
12522 |         dst[i] = '\0';
12523 |         return 0;
12524 |     }
12525 |
12526 |     dst[0] = '\0';

```

```
12527 |     return 34;
12528 | }
12529 |
12530 | MA_API MA_NO_INLINE int ma_wcscpy_s(wchar_t* dst, size_t dstCap, const wchar_t* src)
12531 | {
12532 |     size_t i;
12533 |
12534 |     if (dst == 0) {
12535 |         return 22;
12536 |     }
12537 |     if (dstCap == 0) {
12538 |         return 34;
12539 |     }
12540 |     if (src == 0) {
12541 |         dst[0] = '\\0';
12542 |         return 22;
12543 |     }
12544 |
12545 |     for (i = 0; i < dstCap && src[i] != '\\0'; ++i) {
12546 |         dst[i] = src[i];
12547 |     }
12548 |
12549 |     if (i < dstCap) {
12550 |         dst[i] = '\\0';
12551 |         return 0;
12552 |     }
12553 |
12554 |     dst[0] = '\\0';
12555 |     return 34;
12556 | }
12557 |
12558 |
12559 | MA_API MA_NO_INLINE int ma_strncpy_s(char* dst, size_t dstSizeInBytes, const char* src, size_t
> count)
12560 | {
12561 |     size_t maxcount;
12562 |     size_t i;
12563 |
12564 |     if (dst == 0) {
12565 |         return 22;
12566 |     }
12567 |     if (dstSizeInBytes == 0) {
12568 |         return 34;
12569 |     }
12570 |     if (src == 0) {
12571 |         dst[0] = '\\0';
12572 |         return 22;
12573 |     }
12574 |
12575 |     maxcount = count;
12576 |     if (count == ((size_t)-1) || count >= dstSizeInBytes) {          /* -1 = _TRUNCATE */
12577 |         maxcount = dstSizeInBytes - 1;
12578 |     }
12579 |
12580 |     for (i = 0; i < maxcount && src[i] != '\\0'; ++i) {
12581 |         dst[i] = src[i];
12582 |     }
12583 |
12584 |     if (src[i] == '\\0' || i == count || count == ((size_t)-1)) {
12585 |         dst[i] = '\\0';
12586 |         return 0;
12587 |     }
12588 |
12589 |     dst[0] = '\\0';
12590 |     return 34;
12591 | }
12592 |
12593 | MA_API MA_NO_INLINE int ma_strcat_s(char* dst, size_t dstSizeInBytes, const char* src)
12594 | {
12595 |     char* dstorig;
12596 |
12597 |     if (dst == 0) {
12598 |         return 22;
12599 |     }
12600 |     if (dstSizeInBytes == 0) {
12601 |         return 34;
12602 |     }
12603 |     if (src == 0) {
12604 |         dst[0] = '\\0';
```

```
12605 |         return 22;
12606 |     }
12607 |
12608 |     dstorig = dst;
12609 |
12610 |     while (dstSizeInBytes > 0 && dst[0] != '\0') {
12611 |         dst += 1;
12612 |         dstSizeInBytes -= 1;
12613 |     }
12614 |
12615 |     if (dstSizeInBytes == 0) {
12616 |         return 22; /* Unterminated. */
12617 |     }
12618 |
12619 |
12620 |     while (dstSizeInBytes > 0 && src[0] != '\0') {
12621 |         *dst++ = *src++;
12622 |         dstSizeInBytes -= 1;
12623 |     }
12624 |
12625 |     if (dstSizeInBytes > 0) {
12626 |         dst[0] = '\0';
12627 |     } else {
12628 |         dstorig[0] = '\0';
12629 |         return 34;
12630 |     }
12631 |
12632 |     return 0;
12633 | }
12634 |
12635 | MA_API MA_NO_INLINE int ma_strncat_s(char* dst, size_t dstSizeInBytes, const char* src, size_t
> count)
12636 | {
12637 |     char* dstorig;
12638 |
12639 |     if (dst == 0) {
12640 |         return 22;
12641 |     }
12642 |     if (dstSizeInBytes == 0) {
12643 |         return 34;
12644 |     }
12645 |     if (src == 0) {
12646 |         return 22;
12647 |     }
12648 |
12649 |     dstorig = dst;
12650 |
12651 |     while (dstSizeInBytes > 0 && dst[0] != '\0') {
12652 |         dst += 1;
12653 |         dstSizeInBytes -= 1;
12654 |     }
12655 |
12656 |     if (dstSizeInBytes == 0) {
12657 |         return 22; /* Unterminated. */
12658 |     }
12659 |
12660 |
12661 |     if (count == ((size_t)-1)) { /* _TRUNCATE */
12662 |         count = dstSizeInBytes - 1;
12663 |     }
12664 |
12665 |     while (dstSizeInBytes > 0 && src[0] != '\0' && count > 0) {
12666 |         *dst++ = *src++;
12667 |         dstSizeInBytes -= 1;
12668 |         count -= 1;
12669 |     }
12670 |
12671 |     if (dstSizeInBytes > 0) {
12672 |         dst[0] = '\0';
12673 |     } else {
12674 |         dstorig[0] = '\0';
12675 |         return 34;
12676 |     }
12677 |
12678 |     return 0;
12679 | }
12680 |
12681 | MA_API MA_NO_INLINE int ma_itoa_s(int value, char* dst, size_t dstSizeInBytes, int radix)
12682 | {
```

```

12683 | int sign;
12684 | unsigned int valueU;
12685 | char* dstEnd;
12686 |
12687 | if (dst == NULL || dstSizeInBytes == 0) {
12688 |     return 22;
12689 | }
12690 | if (radix < 2 || radix > 36) {
12691 |     dst[0] = '\0';
12692 |     return 22;
12693 | }
12694 |
12695 | sign = (value < 0 && radix == 10) ? -1 : 1;    /* The negative sign is only used when the base
> is 10. */
12696 |
12697 | if (value < 0) {
12698 |     valueU = -value;
12699 | } else {
12700 |     valueU = value;
12701 | }
12702 |
12703 | dstEnd = dst;
12704 | do
12705 | {
12706 |     int remainder = valueU % radix;
12707 |     if (remainder > 9) {
12708 |         *dstEnd = (char)((remainder - 10) + 'a');
12709 |     } else {
12710 |         *dstEnd = (char)(remainder + '0');
12711 |     }
12712 |
12713 |     dstEnd += 1;
12714 |     dstSizeInBytes -= 1;
12715 |     valueU /= radix;
12716 | } while (dstSizeInBytes > 0 && valueU > 0);
12717 |
12718 | if (dstSizeInBytes == 0) {
12719 |     dst[0] = '\0';
12720 |     return 22; /* Ran out of room in the output buffer. */
12721 | }
12722 |
12723 | if (sign < 0) {
12724 |     *dstEnd++ = '-';
12725 |     dstSizeInBytes -= 1;
12726 | }
12727 |
12728 | if (dstSizeInBytes == 0) {
12729 |     dst[0] = '\0';
12730 |     return 22; /* Ran out of room in the output buffer. */
12731 | }
12732 |
12733 | *dstEnd = '\0';
12734 |
12735 |
12736 | /* At this point the string will be reversed. */
12737 | dstEnd -= 1;
12738 | while (dst < dstEnd) {
12739 |     char temp = *dst;
12740 |     *dst = *dstEnd;
12741 |     *dstEnd = temp;
12742 |
12743 |     dst += 1;
12744 |     dstEnd -= 1;
12745 | }
12746 |
12747 | return 0;
12748 | }
12749 |
12750 | MA_API MA_NO_INLINE int ma_strncmp(const char* str1, const char* str2)
12751 | {
12752 |     if (str1 == str2) return 0;
12753 |
12754 |     /* These checks differ from the standard implementation. It's not important, but I prefer it
> just for sanity. */
12755 |     if (str1 == NULL) return -1;
12756 |     if (str2 == NULL) return 1;
12757 |
12758 |     for (;;) {
12759 |         if (str1[0] == '\0') {

```

```
12760 |         break;
12761 |     }
12762 |     if (str1[0] != str2[0]) {
12763 |         break;
12764 |     }
12765 |
12766 |     str1 += 1;
12767 |     str2 += 1;
12768 | }
12769 |
12770 | return ((unsigned char*)str1)[0] - ((unsigned char*)str2)[0];
12771 | }
12772 |
12773 | MA_API MA_NO_INLINE int ma_wcscmp(const wchar_t* str1, const wchar_t* str2)
12774 | {
12775 |     if (str1 == str2) return 0;
12776 |
12777 |     /* These checks differ from the standard implementation. It's not important, but I prefer it
> just for sanity. */
12778 |     if (str1 == NULL) return -1;
12779 |     if (str2 == NULL) return 1;
12780 |
12781 |     for (;;) {
12782 |         if (str1[0] == L'\0') {
12783 |             break;
12784 |         }
12785 |         if (str1[0] != str2[0]) {
12786 |             break;
12787 |         }
12788 |
12789 |         str1 += 1;
12790 |         str2 += 1;
12791 |     }
12792 |
12793 |     return ((unsigned short*)str1)[0] - ((unsigned short*)str2)[0];
12794 | }
12795 |
12796 | MA_API MA_NO_INLINE int ma_strappend(char* dst, size_t dstSize, const char* srcA, const char* srcB)
12797 | {
12798 |     int result;
12799 |
12800 |     result = ma_strncpy_s(dst, dstSize, srcA, (size_t)-1);
12801 |     if (result != 0) {
12802 |         return result;
12803 |     }
12804 |
12805 |     result = ma_strncat_s(dst, dstSize, srcB, (size_t)-1);
12806 |     if (result != 0) {
12807 |         return result;
12808 |     }
12809 |
12810 |     return result;
12811 | }
12812 |
12813 | MA_API MA_NO_INLINE size_t ma_wcslen(const wchar_t* str)
12814 | {
12815 |     const wchar_t* end;
12816 |
12817 |     if (str == NULL) {
12818 |         return 0;
12819 |     }
12820 |
12821 |     end = str;
12822 |     while (end[0] != L'\0') {
12823 |         end += 1;
12824 |     }
12825 |
12826 |     return end - str;
12827 | }
12828 |
12829 | MA_API MA_NO_INLINE char* ma_copy_string(const char* src, const ma_allocation_callbacks*
> pAllocationCallbacks)
12830 | {
12831 |     size_t sz;
12832 |     char* dst;
12833 |
12834 |     if (src == NULL) {
12835 |         return NULL;
12836 |     }
```

```

12837 |
12838 |     sz = strlen(src)+1;
12839 |     dst = (char*)ma_malloc(sz, pAllocationCallbacks);
12840 |     if (dst == NULL) {
12841 |         return NULL;
12842 |     }
12843 |
12844 |     ma_strcpy_s(dst, sz, src);
12845 |
12846 |     return dst;
12847 | }
12848 |
12849 | MA_API MA_NO_INLINE wchar_t* ma_copy_string_w(const wchar_t* src, const ma_allocation_callbacks*
> pAllocationCallbacks)
12850 | {
12851 |     size_t sz = ma_wcslen(src)+1;
12852 |     wchar_t* dst = (wchar_t*)ma_malloc(sz * sizeof(*dst), pAllocationCallbacks);
12853 |     if (dst == NULL) {
12854 |         return NULL;
12855 |     }
12856 |
12857 |     ma_wcscpy_s(dst, sz, src);
12858 |
12859 |     return dst;
12860 | }
12861 |
12862 |
12863 |
12864 | #include <errno.h>
12865 | static ma_result ma_result_from_errno(int e)
12866 | {
12867 |     if (e == 0) {
12868 |         return MA_SUCCESS;
12869 |     }
12870 | #ifdef EPERM
12871 |     else if (e == EPERM) { return MA_INVALID_OPERATION; }
12872 | #endif
12873 | #ifdef ENOENT
12874 |     else if (e == ENOENT) { return MA_DOES_NOT_EXIST; }
12875 | #endif
12876 | #ifdef ESRCH
12877 |     else if (e == ESRCH) { return MA_DOES_NOT_EXIST; }
12878 | #endif
12879 | #ifdef EINTR
12880 |     else if (e == EINTR) { return MA_INTERRUPT; }
12881 | #endif
12882 | #ifdef EIO
12883 |     else if (e == EIO) { return MA_IO_ERROR; }
12884 | #endif
12885 | #ifdef ENXIO
12886 |     else if (e == ENXIO) { return MA_DOES_NOT_EXIST; }
12887 | #endif
12888 | #ifdef E2BIG
12889 |     else if (e == E2BIG) { return MA_INVALID_ARGS; }
12890 | #endif
12891 | #ifdef ENOEXEC
12892 |     else if (e == ENOEXEC) { return MA_INVALID_FILE; }
12893 | #endif
12894 | #ifdef EBADF
12895 |     else if (e == EBADF) { return MA_INVALID_FILE; }
12896 | #endif
12897 | #ifdef ECHILD
12898 |     else if (e == ECHILD) { return MA_ERROR; }
12899 | #endif
12900 | #ifdef EAGAIN
12901 |     else if (e == EAGAIN) { return MA_UNAVAILABLE; }
12902 | #endif
12903 | #ifdef ENOMEM
12904 |     else if (e == ENOMEM) { return MA_OUT_OF_MEMORY; }
12905 | #endif
12906 | #ifdef EACCES
12907 |     else if (e == EACCES) { return MA_ACCESS_DENIED; }
12908 | #endif
12909 | #ifdef EFAULT
12910 |     else if (e == EFAULT) { return MA_BAD_ADDRESS; }
12911 | #endif
12912 | #ifdef ENOTBLK
12913 |     else if (e == ENOTBLK) { return MA_ERROR; }
12914 | #endif

```



```
12915 | #ifdef EBUSY
12916 |     else if (e == EBUSY) { return MA_BUSY; }
12917 | #endif
12918 | #ifdef EEXIST
12919 |     else if (e == EEXIST) { return MA_ALREADY_EXISTS; }
12920 | #endif
12921 | #ifdef EXDEV
12922 |     else if (e == EXDEV) { return MA_ERROR; }
12923 | #endif
12924 | #ifdef ENODEV
12925 |     else if (e == ENODEV) { return MA_DOES_NOT_EXIST; }
12926 | #endif
12927 | #ifdef ENOTDIR
12928 |     else if (e == ENOTDIR) { return MA_NOT_DIRECTORY; }
12929 | #endif
12930 | #ifdef EISDIR
12931 |     else if (e == EISDIR) { return MA_IS_DIRECTORY; }
12932 | #endif
12933 | #ifdef EINVAL
12934 |     else if (e == EINVAL) { return MA_INVALID_ARGS; }
12935 | #endif
12936 | #ifdef ENFILE
12937 |     else if (e == ENFILE) { return MA_TOO_MANY_OPEN_FILES; }
12938 | #endif
12939 | #ifdef EMFILE
12940 |     else if (e == EMFILE) { return MA_TOO_MANY_OPEN_FILES; }
12941 | #endif
12942 | #ifdef ENOTTY
12943 |     else if (e == ENOTTY) { return MA_INVALID_OPERATION; }
12944 | #endif
12945 | #ifdef ETXTBSY
12946 |     else if (e == ETXTBSY) { return MA_BUSY; }
12947 | #endif
12948 | #ifdef EFBIG
12949 |     else if (e == EFBIG) { return MA_TOO_BIG; }
12950 | #endif
12951 | #ifdef ENOSPC
12952 |     else if (e == ENOSPC) { return MA_NO_SPACE; }
12953 | #endif
12954 | #ifdef ESPIPE
12955 |     else if (e == ESPIPE) { return MA_BAD_SEEK; }
12956 | #endif
12957 | #ifdef EROFS
12958 |     else if (e == EROFS) { return MA_ACCESS_DENIED; }
12959 | #endif
12960 | #ifdef EMLINK
12961 |     else if (e == EMLINK) { return MA_TOO_MANY_LINKS; }
12962 | #endif
12963 | #ifdef EPIPE
12964 |     else if (e == EPIPE) { return MA_BAD_PIPE; }
12965 | #endif
12966 | #ifdef EDOM
12967 |     else if (e == EDOM) { return MA_OUT_OF_RANGE; }
12968 | #endif
12969 | #ifdef ERANGE
12970 |     else if (e == ERANGE) { return MA_OUT_OF_RANGE; }
12971 | #endif
12972 | #ifdef EDEADLK
12973 |     else if (e == EDEADLK) { return MA_DEADLOCK; }
12974 | #endif
12975 | #ifdef ENAMETOOLONG
12976 |     else if (e == ENAMETOOLONG) { return MA_PATH_TOO_LONG; }
12977 | #endif
12978 | #ifdef ENOLCK
12979 |     else if (e == ENOLCK) { return MA_ERROR; }
12980 | #endif
12981 | #ifdef ENOSYS
12982 |     else if (e == ENOSYS) { return MA_NOT_IMPLEMENTED; }
12983 | #endif
12984 | #ifdef ENOTEMPTY
12985 |     else if (e == ENOTEMPTY) { return MA_DIRECTORY_NOT_EMPTY; }
12986 | #endif
12987 | #ifdef ELOOP
12988 |     else if (e == ELOOP) { return MA_TOO_MANY_LINKS; }
12989 | #endif
12990 | #ifdef ENOMSG
12991 |     else if (e == ENOMSG) { return MA_NO_MESSAGE; }
12992 | #endif
12993 | #ifdef EIDRM
```

```
12994 |     else if (e == EIDRM) { return MA_ERROR; }
12995 | #endif
12996 | #ifdef ECHRNA
12997 |     else if (e == ECHRNA) { return MA_ERROR; }
12998 | #endif
12999 | #ifdef EL2NSYNC
13000 |     else if (e == EL2NSYNC) { return MA_ERROR; }
13001 | #endif
13002 | #ifdef EL3HLT
13003 |     else if (e == EL3HLT) { return MA_ERROR; }
13004 | #endif
13005 | #ifdef EL3RST
13006 |     else if (e == EL3RST) { return MA_ERROR; }
13007 | #endif
13008 | #ifdef ELNRNG
13009 |     else if (e == ELNRNG) { return MA_OUT_OF_RANGE; }
13010 | #endif
13011 | #ifdef EUNATCH
13012 |     else if (e == EUNATCH) { return MA_ERROR; }
13013 | #endif
13014 | #ifdef ENOCSI
13015 |     else if (e == ENOCSI) { return MA_ERROR; }
13016 | #endif
13017 | #ifdef EL2HLT
13018 |     else if (e == EL2HLT) { return MA_ERROR; }
13019 | #endif
13020 | #ifdef EBADE
13021 |     else if (e == EBADE) { return MA_ERROR; }
13022 | #endif
13023 | #ifdef EBADR
13024 |     else if (e == EBADR) { return MA_ERROR; }
13025 | #endif
13026 | #ifdef EXFULL
13027 |     else if (e == EXFULL) { return MA_ERROR; }
13028 | #endif
13029 | #ifdef ENOANO
13030 |     else if (e == ENOANO) { return MA_ERROR; }
13031 | #endif
13032 | #ifdef EBADRQC
13033 |     else if (e == EBADRQC) { return MA_ERROR; }
13034 | #endif
13035 | #ifdef EBADSLT
13036 |     else if (e == EBADSLT) { return MA_ERROR; }
13037 | #endif
13038 | #ifdef EBFONT
13039 |     else if (e == EBFONT) { return MA_INVALID_FILE; }
13040 | #endif
13041 | #ifdef ENOSTR
13042 |     else if (e == ENOSTR) { return MA_ERROR; }
13043 | #endif
13044 | #ifdef ENODATA
13045 |     else if (e == ENODATA) { return MA_NO_DATA_AVAILABLE; }
13046 | #endif
13047 | #ifdef ETIME
13048 |     else if (e == ETIME) { return MA_TIMEOUT; }
13049 | #endif
13050 | #ifdef ENOSR
13051 |     else if (e == ENOSR) { return MA_NO_DATA_AVAILABLE; }
13052 | #endif
13053 | #ifdef ENONET
13054 |     else if (e == ENONET) { return MA_NO_NETWORK; }
13055 | #endif
13056 | #ifdef ENOPKG
13057 |     else if (e == ENOPKG) { return MA_ERROR; }
13058 | #endif
13059 | #ifdef EREMOTE
13060 |     else if (e == EREMOTE) { return MA_ERROR; }
13061 | #endif
13062 | #ifdef ENOLINK
13063 |     else if (e == ENOLINK) { return MA_ERROR; }
13064 | #endif
13065 | #ifdef EADV
13066 |     else if (e == EADV) { return MA_ERROR; }
13067 | #endif
13068 | #ifdef ESRMNT
13069 |     else if (e == ESRMNT) { return MA_ERROR; }
13070 | #endif
13071 | #ifdef ECOMM
13072 |     else if (e == ECOMM) { return MA_ERROR; }
```

```
13073 | #endif
13074 | #ifdef EPROTO
13075 |     else if (e == EPROTO) { return MA_ERROR; }
13076 | #endif
13077 | #ifdef EMULTIHOP
13078 |     else if (e == EMULTIHOP) { return MA_ERROR; }
13079 | #endif
13080 | #ifdef EDOTDOT
13081 |     else if (e == EDOTDOT) { return MA_ERROR; }
13082 | #endif
13083 | #ifdef EBADMSG
13084 |     else if (e == EBADMSG) { return MA_BAD_MESSAGE; }
13085 | #endif
13086 | #ifdef EOVERFLOW
13087 |     else if (e == EOVERFLOW) { return MA_TOO_BIG; }
13088 | #endif
13089 | #ifdef ENOTUNIQ
13090 |     else if (e == ENOTUNIQ) { return MA_NOT_UNIQUE; }
13091 | #endif
13092 | #ifdef EBADFD
13093 |     else if (e == EBADFD) { return MA_ERROR; }
13094 | #endif
13095 | #ifdef EREMCHG
13096 |     else if (e == EREMCHG) { return MA_ERROR; }
13097 | #endif
13098 | #ifdef ELIBACC
13099 |     else if (e == ELIBACC) { return MA_ACCESS_DENIED; }
13100 | #endif
13101 | #ifdef ELIBBAD
13102 |     else if (e == ELIBBAD) { return MA_INVALID_FILE; }
13103 | #endif
13104 | #ifdef ELIBSCN
13105 |     else if (e == ELIBSCN) { return MA_INVALID_FILE; }
13106 | #endif
13107 | #ifdef ELIBMAX
13108 |     else if (e == ELIBMAX) { return MA_ERROR; }
13109 | #endif
13110 | #ifdef ELIBEXEC
13111 |     else if (e == ELIBEXEC) { return MA_ERROR; }
13112 | #endif
13113 | #ifdef EILSEQ
13114 |     else if (e == EILSEQ) { return MA_INVALID_DATA; }
13115 | #endif
13116 | #ifdef ERESTART
13117 |     else if (e == ERESTART) { return MA_ERROR; }
13118 | #endif
13119 | #ifdef ESTRPIPE
13120 |     else if (e == ESTRPIPE) { return MA_ERROR; }
13121 | #endif
13122 | #ifdef EUSERS
13123 |     else if (e == EUSERS) { return MA_ERROR; }
13124 | #endif
13125 | #ifdef ENOTSOCK
13126 |     else if (e == ENOTSOCK) { return MA_NOT_SOCKET; }
13127 | #endif
13128 | #ifdef EDESTADDRREQ
13129 |     else if (e == EDESTADDRREQ) { return MA_NO_ADDRESS; }
13130 | #endif
13131 | #ifdef EMSGSIZE
13132 |     else if (e == EMSGSIZE) { return MA_TOO_BIG; }
13133 | #endif
13134 | #ifdef EPROTOTYPE
13135 |     else if (e == EPROTOTYPE) { return MA_BAD_PROTOCOL; }
13136 | #endif
13137 | #ifdef ENOPROTOOPT
13138 |     else if (e == ENOPROTOOPT) { return MA_PROTOCOL_UNAVAILABLE; }
13139 | #endif
13140 | #ifdef EPROTONOSUPPORT
13141 |     else if (e == EPROTONOSUPPORT) { return MA_PROTOCOL_NOT_SUPPORTED; }
13142 | #endif
13143 | #ifdef ESOCKTNOSUPPORT
13144 |     else if (e == ESOCKTNOSUPPORT) { return MA_SOCKET_NOT_SUPPORTED; }
13145 | #endif
13146 | #ifdef EOPNOTSUPP
13147 |     else if (e == EOPNOTSUPP) { return MA_INVALID_OPERATION; }
13148 | #endif
13149 | #ifdef EPFNOSUPPORT
13150 |     else if (e == EPFNOSUPPORT) { return MA_PROTOCOL_FAMILY_NOT_SUPPORTED; }
13151 | #endif
```

```
13152 | #ifdef EAFNOSUPPORT
13153 |     else if (e == EAFNOSUPPORT) { return MA_ADDRESS_FAMILY_NOT_SUPPORTED; }
13154 | #endif
13155 | #ifdef EADDRINUSE
13156 |     else if (e == EADDRINUSE) { return MA_ALREADY_IN_USE; }
13157 | #endif
13158 | #ifdef EADDRNOTAVAIL
13159 |     else if (e == EADDRNOTAVAIL) { return MA_ERROR; }
13160 | #endif
13161 | #ifdef ENETDOWN
13162 |     else if (e == ENETDOWN) { return MA_NO_NETWORK; }
13163 | #endif
13164 | #ifdef ENETUNREACH
13165 |     else if (e == ENETUNREACH) { return MA_NO_NETWORK; }
13166 | #endif
13167 | #ifdef ENETRESET
13168 |     else if (e == ENETRESET) { return MA_NO_NETWORK; }
13169 | #endif
13170 | #ifdef ECONNABORTED
13171 |     else if (e == ECONNABORTED) { return MA_NO_NETWORK; }
13172 | #endif
13173 | #ifdef ECONNRESET
13174 |     else if (e == ECONNRESET) { return MA_CONNECTION_RESET; }
13175 | #endif
13176 | #ifdef ENOBUFS
13177 |     else if (e == ENOBUFS) { return MA_NO_SPACE; }
13178 | #endif
13179 | #ifdef EISCONN
13180 |     else if (e == EISCONN) { return MA_ALREADY_CONNECTED; }
13181 | #endif
13182 | #ifdef ENOTCONN
13183 |     else if (e == ENOTCONN) { return MA_NOT_CONNECTED; }
13184 | #endif
13185 | #ifdef ESHUTDOWN
13186 |     else if (e == ESHUTDOWN) { return MA_ERROR; }
13187 | #endif
13188 | #ifdef ETOOMANYREFS
13189 |     else if (e == ETOOMANYREFS) { return MA_ERROR; }
13190 | #endif
13191 | #ifdef ETIMEDOUT
13192 |     else if (e == ETIMEDOUT) { return MA_TIMEOUT; }
13193 | #endif
13194 | #ifdef ECONNREFUSED
13195 |     else if (e == ECONNREFUSED) { return MA_CONNECTION_REFUSED; }
13196 | #endif
13197 | #ifdef EHOSTDOWN
13198 |     else if (e == EHOSTDOWN) { return MA_NO_HOST; }
13199 | #endif
13200 | #ifdef EHOSTUNREACH
13201 |     else if (e == EHOSTUNREACH) { return MA_NO_HOST; }
13202 | #endif
13203 | #ifdef EALREADY
13204 |     else if (e == EALREADY) { return MA_IN_PROGRESS; }
13205 | #endif
13206 | #ifdef EINPROGRESS
13207 |     else if (e == EINPROGRESS) { return MA_IN_PROGRESS; }
13208 | #endif
13209 | #ifdef ESTALE
13210 |     else if (e == ESTALE) { return MA_INVALID_FILE; }
13211 | #endif
13212 | #ifdef EUCLEAN
13213 |     else if (e == EUCLEAN) { return MA_ERROR; }
13214 | #endif
13215 | #ifdef ENOTNAM
13216 |     else if (e == ENOTNAM) { return MA_ERROR; }
13217 | #endif
13218 | #ifdef ENAVAIL
13219 |     else if (e == ENAVAIL) { return MA_ERROR; }
13220 | #endif
13221 | #ifdef EISNAM
13222 |     else if (e == EISNAM) { return MA_ERROR; }
13223 | #endif
13224 | #ifdef EREMOTEIO
13225 |     else if (e == EREMOTEIO) { return MA_IO_ERROR; }
13226 | #endif
13227 | #ifdef EDQUOT
13228 |     else if (e == EDQUOT) { return MA_NO_SPACE; }
13229 | #endif
13230 | #ifdef ENOMEDIUM
```

```

13231 |     else if (e == ENOMEDIUM) { return MA_DOES_NOT_EXIST; }
13232 | #endif
13233 | #ifdef EMEDIUMTYPE
13234 |     else if (e == EMEDIUMTYPE) { return MA_ERROR; }
13235 | #endif
13236 | #ifdef ECANCELED
13237 |     else if (e == ECANCELED) { return MA_CANCELLED; }
13238 | #endif
13239 | #ifdef ENOKEY
13240 |     else if (e == ENOKEY) { return MA_ERROR; }
13241 | #endif
13242 | #ifdef EKEYEXPIRED
13243 |     else if (e == EKEYEXPIRED) { return MA_ERROR; }
13244 | #endif
13245 | #ifdef EKEYREVOKED
13246 |     else if (e == EKEYREVOKED) { return MA_ERROR; }
13247 | #endif
13248 | #ifdef EKEYREJECTED
13249 |     else if (e == EKEYREJECTED) { return MA_ERROR; }
13250 | #endif
13251 | #ifdef EOWNERDEAD
13252 |     else if (e == EOWNERDEAD) { return MA_ERROR; }
13253 | #endif
13254 | #ifdef ENOTRECOVERABLE
13255 |     else if (e == ENOTRECOVERABLE) { return MA_ERROR; }
13256 | #endif
13257 | #ifdef ERFKILL
13258 |     else if (e == ERFKILL) { return MA_ERROR; }
13259 | #endif
13260 | #ifdef EHWPOISON
13261 |     else if (e == EHWPOISON) { return MA_ERROR; }
13262 | #endif
13263 |     else {
13264 |         return MA_ERROR;
13265 |     }
13266 | }
13267 |
13268 | MA_API ma_result ma_fopen(FILE** ppFile, const char* pFilePath, const char* pOpenMode)
13269 | {
13270 |     #if defined(_MSC_VER) && _MSC_VER >= 1400
13271 |         errno_t err;
13272 |     #endif
13273 |
13274 |     if (ppFile != NULL) {
13275 |         *ppFile = NULL; /* Safety. */
13276 |     }
13277 |
13278 |     if (pFilePath == NULL || pOpenMode == NULL || ppFile == NULL) {
13279 |         return MA_INVALID_ARGS;
13280 |     }
13281 |
13282 |     #if (defined(_MSC_VER) && _MSC_VER >= 1400) && !defined(MA_XBOX_NXDK)
13283 |         err = fopen_s(ppFile, pFilePath, pOpenMode);
13284 |         if (err != 0) {
13285 |             return ma_result_from_errno(err);
13286 |         }
13287 |     #else
13288 |         #if defined(_WIN32) || defined(__APPLE__)
13289 |             *ppFile = fopen(pFilePath, pOpenMode);
13290 |         #else
13291 |             #if defined(_FILE_OFFSET_BITS) && _FILE_OFFSET_BITS == 64 && defined(_LARGEFILE64_SOURCE)
13292 |                 *ppFile = fopen64(pFilePath, pOpenMode);
13293 |             #else
13294 |                 *ppFile = fopen(pFilePath, pOpenMode);
13295 |             #endif
13296 |         #endif
13297 |         if (*ppFile == NULL) {
13298 |             ma_result result = ma_result_from_errno(errno);
13299 |             if (result == MA_SUCCESS) {
13300 |                 result = MA_ERROR; /* Just a safety check to make sure we never ever return success
> when pFile == NULL. */
13301 |             }
13302 |
13303 |             return result;
13304 |         }
13305 |     #endif
13306 |
13307 |     return MA_SUCCESS;
13308 | }

```

```

13309 |
13310 |
13311 |
13312 | /*
13313 | _w fopen() isn't always available in all compilation environments.
13314 |
13315 |     * Windows only.
13316 |     * MSVC seems to support it universally as far back as VC6 from what I can tell (haven't checked
> further back).
13317 |     * MinGW-64 (both 32- and 64-bit) seems to support it.
13318 |     * MinGW wraps it in !defined(__STRICT_ANSI__).
13319 |     * OpenWatcom wraps it in !defined(_NO_EXT_KEYS).
13320 |
13321 | This can be reviewed as compatibility issues arise. The preference is to use _w fopen_s() and
> _w fopen() as opposed to the wcsrtombs()
13322 | fallback, so if you notice your compiler not detecting this properly I'm happy to look at adding
> support.
13323 | */
13324 | #if defined(_WIN32) && !defined(MA_XBOX_NXDK)
13325 |     #if defined(_MSC_VER) || defined(__MINGW64__) || (!defined(__STRICT_ANSI__) &&
> !defined(_NO_EXT_KEYS))
13326 |         #define MA_HAS_WFOPEN
13327 |     #endif
13328 | #endif
13329 |
13330 | MA_API ma_result ma_w fopen(FILE** ppFile, const wchar_t* pFilePath, const wchar_t* pOpenMode, const
> ma_allocation_callbacks* pAllocationCallbacks)
13331 | {
13332 |     if (ppFile != NULL) {
13333 |         *ppFile = NULL; /* Safety. */
13334 |     }
13335 |
13336 |     if (pFilePath == NULL || pOpenMode == NULL || ppFile == NULL) {
13337 |         return MA_INVALID_ARGS;
13338 |     }
13339 |
13340 |     #if defined(MA_HAS_WFOPEN)
13341 |     {
13342 |         /* Use _w fopen() on Windows. */
13343 |         #if defined(_MSC_VER) && _MSC_VER >= 1400
13344 |         {
13345 |             errno_t err = _w fopen_s(ppFile, pFilePath, pOpenMode);
13346 |             if (err != 0) {
13347 |                 return ma_result_from_errno(err);
13348 |             }
13349 |         }
13350 |         #else
13351 |         {
13352 |             *ppFile = _w fopen(pFilePath, pOpenMode);
13353 |             if (*ppFile == NULL) {
13354 |                 return ma_result_from_errno(errno);
13355 |             }
13356 |         }
13357 |         #endif
13358 |
13359 |         (void)pAllocationCallbacks;
13360 |     }
13361 |     #elif !defined(MA_XBOX_NXDK) && !defined(MA_DOS) /* If your compiler does not support
> wcsrtombs(), add it here. */
13362 |     {
13363 |         /*
13364 |         Use fopen() on anything other than Windows. Requires a conversion. This is annoying because
> fopen() is locale specific. The only real way I can
13365 |         think of to do this is with wcsrtombs(). Note that wcsrtombs() is apparently not thread-safe
> because it uses a static global mbstate_t object for
13366 |         maintaining state. I've checked this with -std=c89 and it works, but if somebody get's a
> compiler error I'll look into improving compatibility.
13367 |         */
13368 |         mbstate_t mbs;
13369 |         size_t lenMB;
13370 |         const wchar_t* pFilePathTemp = pFilePath;
13371 |         char* pFilePathMB = NULL;
13372 |         char pOpenModeMB[32] = {0};
13373 |
13374 |         /* Get the length first. */
13375 |         MA_ZERO_OBJECT(&mbs);
13376 |         lenMB = wcsrtombs(NULL, &pFilePathTemp, 0, &mbs);
13377 |         if (lenMB == (size_t)-1) {
13378 |             return ma_result_from_errno(errno);

```

```

13379 |     }
13380 |
13381 |     pFilePathMB = (char*)ma_malloc(lenMB + 1, pAllocationCallbacks);
13382 |     if (pFilePathMB == NULL) {
13383 |         return MA_OUT_OF_MEMORY;
13384 |     }
13385 |
13386 |     pFilePathTemp = pFilePath;
13387 |     MA_ZERO_OBJECT(&mbs);
13388 |     wcsrtombs(pFilePathMB, &pFilePathTemp, lenMB + 1, &mbs);
13389 |
13390 |     /* The open mode should always consist of ASCII characters so we should be able to do a
> trivial conversion. */
13391 |     {
13392 |         size_t i = 0;
13393 |         for (;;) {
13394 |             if (pOpenMode[i] == 0) {
13395 |                 pOpenModeMB[i] = '\\0';
13396 |                 break;
13397 |             }
13398 |
13399 |             pOpenModeMB[i] = (char)pOpenMode[i];
13400 |             i += 1;
13401 |         }
13402 |     }
13403 |
13404 |     *ppFile = fopen(pFilePathMB, pOpenModeMB);
13405 |
13406 |     ma_free(pFilePathMB, pAllocationCallbacks);
13407 | }
13408 | #else
13409 | {
13410 |     /* Getting here means there is no way to open the file with a wide character string. */
13411 |     *ppFile = NULL;
13412 | }
13413 | #endif
13414 |
13415 | if (*ppFile == NULL) {
13416 |     return MA_ERROR;
13417 | }
13418 |
13419 | return MA_SUCCESS;
13420 | }
13421 |
13422 |
13423 |
13424 | static MA_INLINE void ma_copy_memory_64(void* dst, const void* src, ma_uint64 sizeInBytes)
13425 | {
13426 |     #if MA_SIZE_MAX > 0xFFFFFFFF
13427 |         MA_COPY_MEMORY(dst, src, (size_t)sizeInBytes);
13428 |     #else
13429 |         while (sizeInBytes > 0) {
13430 |             ma_uint64 bytesToCopyNow = sizeInBytes;
13431 |             if (bytesToCopyNow > MA_SIZE_MAX) {
13432 |                 bytesToCopyNow = MA_SIZE_MAX;
13433 |             }
13434 |
13435 |             MA_COPY_MEMORY(dst, src, (size_t)bytesToCopyNow); /* Safe cast to size_t. */
13436 |
13437 |             sizeInBytes -= bytesToCopyNow;
13438 |             dst = (void*)((ma_uint8*)dst + bytesToCopyNow);
13439 |             src = (const void*)((const ma_uint8*)src + bytesToCopyNow);
13440 |         }
13441 |     #endif
13442 | }
13443 |
13444 | static MA_INLINE void ma_zero_memory_64(void* dst, ma_uint64 sizeInBytes)
13445 | {
13446 |     #if MA_SIZE_MAX > 0xFFFFFFFF
13447 |         MA_ZERO_MEMORY(dst, (size_t)sizeInBytes);
13448 |     #else
13449 |         while (sizeInBytes > 0) {
13450 |             ma_uint64 bytesToZeroNow = sizeInBytes;
13451 |             if (bytesToZeroNow > MA_SIZE_MAX) {
13452 |                 bytesToZeroNow = MA_SIZE_MAX;
13453 |             }
13454 |
13455 |             MA_ZERO_MEMORY(dst, (size_t)bytesToZeroNow); /* Safe cast to size_t. */
13456 |         }

```



```

13457 |         sizeInBytes -= bytesToZeroNow;
13458 |         dst = (void*)((ma_uint8*)dst + bytesToZeroNow);
13459 |     }
13460 | #endif
13461 | }
13462 |
13463 |
13464 | /* Thanks to good old Bit Twiddling Hacks for this one:
> http://graphics.stanford.edu/~seander/bithacks.html#RoundUpPowerOf2 */
13465 | static MA_INLINE unsigned int ma_next_power_of_2(unsigned int x)
13466 | {
13467 |     x--;
13468 |     x |= x >> 1;
13469 |     x |= x >> 2;
13470 |     x |= x >> 4;
13471 |     x |= x >> 8;
13472 |     x |= x >> 16;
13473 |     x++;
13474 |
13475 |     return x;
13476 | }
13477 |
13478 | static MA_INLINE unsigned int ma_prev_power_of_2(unsigned int x)
13479 | {
13480 |     return ma_next_power_of_2(x) >> 1;
13481 | }
13482 |
13483 | static MA_INLINE unsigned int ma_round_to_power_of_2(unsigned int x)
13484 | {
13485 |     unsigned int prev = ma_prev_power_of_2(x);
13486 |     unsigned int next = ma_next_power_of_2(x);
13487 |     if ((next - x) > (x - prev)) {
13488 |         return prev;
13489 |     } else {
13490 |         return next;
13491 |     }
13492 | }
13493 |
13494 | static MA_INLINE unsigned int ma_count_set_bits(unsigned int x)
13495 | {
13496 |     unsigned int count = 0;
13497 |     while (x != 0) {
13498 |         if (x & 1) {
13499 |             count += 1;
13500 |         }
13501 |
13502 |         x = x >> 1;
13503 |     }
13504 |
13505 |     return count;
13506 | }
13507 |
13508 |
13509 |
13510 | /*****
> *****/
13511 |
13512 | Allocation Callbacks
13513 |
13514 | *****/
> *****/
13515 | static void* ma__malloc_default(size_t sz, void* pUserData)
13516 | {
13517 |     (void)pUserData;
13518 |     return MA_MALLOC(sz);
13519 | }
13520 |
13521 | static void* ma__realloc_default(void* p, size_t sz, void* pUserData)
13522 | {
13523 |     (void)pUserData;
13524 |     return MA_REALLOC(p, sz);
13525 | }
13526 |
13527 | static void ma__free_default(void* p, void* pUserData)
13528 | {
13529 |     (void)pUserData;
13530 |     MA_FREE(p);
13531 | }
13532 |

```

```

13533 | static ma_allocation_callbacks ma_allocation_callbacks_init_default(void)
13534 | {
13535 |     ma_allocation_callbacks callbacks;
13536 |     callbacks.pUserData = NULL;
13537 |     callbacks.onMalloc = ma__malloc_default;
13538 |     callbacks.onRealloc = ma__realloc_default;
13539 |     callbacks.onFree = ma__free_default;
13540 |
13541 |     return callbacks;
13542 | }
13543 |
13544 | static ma_result ma_allocation_callbacks_init_copy(ma_allocation_callbacks* pDst, const
> ma_allocation_callbacks* pSrc)
13545 | {
13546 |     if (pDst == NULL) {
13547 |         return MA_INVALID_ARGS;
13548 |     }
13549 |
13550 |     if (pSrc == NULL) {
13551 |         *pDst = ma_allocation_callbacks_init_default();
13552 |     } else {
13553 |         if (pSrc->pUserData == NULL && pSrc->onFree == NULL && pSrc->onMalloc == NULL &&
> pSrc->onRealloc == NULL) {
13554 |             *pDst = ma_allocation_callbacks_init_default();
13555 |         } else {
13556 |             if (pSrc->onFree == NULL || (pSrc->onMalloc == NULL && pSrc->onRealloc == NULL)) {
13557 |                 return MA_INVALID_ARGS; /* Invalid allocation callbacks. */
13558 |             } else {
13559 |                 *pDst = *pSrc;
13560 |             }
13561 |         }
13562 |     }
13563 |
13564 |     return MA_SUCCESS;
13565 | }
13566 |
13567 |
13568 |
13569 |
13570 | /*****
> *****/
13571 |
13572 | Logging
13573 |
13574 | *****/
> *****/
13575 | #ifndef ma_va_copy
13576 |     #if !defined(_MSC_VER) || _MSC_VER >= 1800
13577 |         #if (defined(__GNUC__) && __GNUC__ < 3)
13578 |             #define ma_va_copy(dst, src) ((dst) = (src)) /* This is untested. Not sure if this is
> correct for old GCC. */
13579 |         #else
13580 |             #define ma_va_copy(dst, src) va_copy((dst), (src))
13581 |         #endif
13582 |     #else
13583 |         #define ma_va_copy(dst, src) ((dst) = (src))
13584 |     #endif
13585 | #endif
13586 |
13587 | MA_API const char* ma_log_level_to_string(ma_uint32 logLevel)
13588 | {
13589 |     switch (logLevel)
13590 |     {
13591 |         case MA_LOG_LEVEL_DEBUG: return "DEBUG";
13592 |         case MA_LOG_LEVEL_INFO: return "INFO";
13593 |         case MA_LOG_LEVEL_WARNING: return "WARNING";
13594 |         case MA_LOG_LEVEL_ERROR: return "ERROR";
13595 |         default: return "ERROR";
13596 |     }
13597 | }
13598 |
13599 | #if defined(MA_DEBUG_OUTPUT)
13600 | #if defined(MA_ANDROID)
13601 |     #include <android/log.h>
13602 | #endif
13603 |
13604 | /* Customize this to use a specific tag in __android_log_print() for debug output messages. */
13605 | #ifndef MA_ANDROID_LOG_TAG
13606 | #define MA_ANDROID_LOG_TAG "miniaudio"

```

```

13607 | #endif
13608 |
13609 | void ma_log_callback_debug(void* pUserData, ma_uint32 level, const char* pMessage)
13610 | {
13611 |     (void)pUserData;
13612 |
13613 |     /* Special handling for some platforms. */
13614 |     #if defined(MA_ANDROID)
13615 |     {
13616 |         /* Android. */
13617 |         __android_log_print(ANDROID_LOG_DEBUG, MA_ANDROID_LOG_TAG, "%s: %s",
> ma_log_level_to_string(level), pMessage);
13618 |     }
13619 |     #else
13620 |     {
13621 |         /* Everything else. */
13622 |         printf("%s: %s", ma_log_level_to_string(level), pMessage);
13623 |     }
13624 |     #endif
13625 | }
13626 | #endif
13627 |
13628 | MA_API ma_log_callback ma_log_callback_init(ma_log_callback_proc onLog, void* pUserData)
13629 | {
13630 |     ma_log_callback callback;
13631 |
13632 |     MA_ZERO_OBJECT(&callback);
13633 |     callback.onLog = onLog;
13634 |     callback.pUserData = pUserData;
13635 |
13636 |     return callback;
13637 | }
13638 |
13639 |
13640 | MA_API ma_result ma_log_init(const ma_allocation_callbacks* pAllocationCallbacks, ma_log* pLog)
13641 | {
13642 |     if (pLog == NULL) {
13643 |         return MA_INVALID_ARGS;
13644 |     }
13645 |
13646 |     MA_ZERO_OBJECT(pLog);
13647 |     ma_allocation_callbacks_init_copy(&pLog->allocationCallbacks, pAllocationCallbacks);
13648 |
13649 |     /* We need a mutex for thread safety. */
13650 |     #ifndef MA_NO_THREADING
13651 |     {
13652 |         ma_result result = ma_mutex_init(&pLog->lock);
13653 |         if (result != MA_SUCCESS) {
13654 |             return result;
13655 |         }
13656 |     }
13657 |     #endif
13658 |
13659 |     /* If we're using debug output, enable it. */
13660 |     #if defined(MA_DEBUG_OUTPUT)
13661 |     {
13662 |         ma_log_register_callback(pLog, ma_log_callback_init(ma_log_callback_debug, NULL)); /*
> Doesn't really matter if this fails. */
13663 |     }
13664 |     #endif
13665 |
13666 |     return MA_SUCCESS;
13667 | }
13668 |
13669 | MA_API void ma_log_uninit(ma_log* pLog)
13670 | {
13671 |     if (pLog == NULL) {
13672 |         return;
13673 |     }
13674 |
13675 |     #ifndef MA_NO_THREADING
13676 |     ma_mutex_uninit(&pLog->lock);
13677 |     #endif
13678 | }
13679 |
13680 | static void ma_log_lock(ma_log* pLog)
13681 | {
13682 |     #ifndef MA_NO_THREADING
13683 |     ma_mutex_lock(&pLog->lock);

```

```

13684 | #else
13685 |     (void)pLog;
13686 | #endif
13687 | }
13688 |
13689 | static void ma_log_unlock(ma_log* pLog)
13690 | {
13691 |     #ifndef MA_NO_THREADING
13692 |         ma_mutex_unlock(&pLog->lock);
13693 |     #else
13694 |         (void)pLog;
13695 |     #endif
13696 | }
13697 |
13698 | MA_API ma_result ma_log_register_callback(ma_log* pLog, ma_log_callback callback)
13699 | {
13700 |     ma_result result = MA_SUCCESS;
13701 |
13702 |     if (pLog == NULL || callback.onLog == NULL) {
13703 |         return MA_INVALID_ARGS;
13704 |     }
13705 |
13706 |     ma_log_lock(pLog);
13707 |     {
13708 |         if (pLog->callbackCount == ma_countof(pLog->callbacks)) {
13709 |             result = MA_OUT_OF_MEMORY; /* Reached the maximum allowed log callbacks. */
13710 |         } else {
13711 |             pLog->callbacks[pLog->callbackCount] = callback;
13712 |             pLog->callbackCount += 1;
13713 |         }
13714 |     }
13715 |     ma_log_unlock(pLog);
13716 |
13717 |     return result;
13718 | }
13719 |
13720 | MA_API ma_result ma_log_unregister_callback(ma_log* pLog, ma_log_callback callback)
13721 | {
13722 |     if (pLog == NULL) {
13723 |         return MA_INVALID_ARGS;
13724 |     }
13725 |
13726 |     ma_log_lock(pLog);
13727 |     {
13728 |         ma_uint32 iLog;
13729 |         for (iLog = 0; iLog < pLog->callbackCount; ) {
13730 |             if (pLog->callbacks[iLog].onLog == callback.onLog) {
13731 |                 /* Found. Move everything down a slot. */
13732 |                 ma_uint32 jLog;
13733 |                 for (jLog = iLog; jLog < pLog->callbackCount-1; jLog += 1) {
13734 |                     pLog->callbacks[jLog] = pLog->callbacks[jLog + 1];
13735 |                 }
13736 |
13737 |                 pLog->callbackCount -= 1;
13738 |             } else {
13739 |                 /* Not found. */
13740 |                 iLog += 1;
13741 |             }
13742 |         }
13743 |     }
13744 |     ma_log_unlock(pLog);
13745 |
13746 |     return MA_SUCCESS;
13747 | }
13748 |
13749 | MA_API ma_result ma_log_post(ma_log* pLog, ma_uint32 level, const char* pMessage)
13750 | {
13751 |     if (pLog == NULL || pMessage == NULL) {
13752 |         return MA_INVALID_ARGS;
13753 |     }
13754 |
13755 |     ma_log_lock(pLog);
13756 |     {
13757 |         ma_uint32 iLog;
13758 |         for (iLog = 0; iLog < pLog->callbackCount; iLog += 1) {
13759 |             if (pLog->callbacks[iLog].onLog) {
13760 |                 pLog->callbacks[iLog].onLog(pLog->callbacks[iLog].pUserData, level, pMessage);
13761 |             }
13762 |         }

```

```

13763 |     }
13764 |     ma_log_unlock(pLog);
13765 |
13766 |     return MA_SUCCESS;
13767 | }
13768 |
13769 |
13770 | /*
13771 | We need to emulate _vscprintf() for the VC6 build. This can be more efficient, but since it's only
> VC6, and it's just a
13772 | logging function, I'm happy to keep this simple. In the VC6 build we can implement this in terms of
> _vsnprintf().
13773 | */
13774 | #if defined(_MSC_VER) && _MSC_VER < 1900
13775 | static int ma_vscprintf(const ma_allocation_callbacks* pAllocationCallbacks, const char* format,
> va_list args)
13776 | {
13777 |     #if _MSC_VER > 1200
13778 |         return _vscprintf(format, args);
13779 |     #else
13780 |         int result;
13781 |         char* pTempBuffer = NULL;
13782 |         size_t tempBufferCap = 1024;
13783 |
13784 |         if (format == NULL) {
13785 |             errno = EINVAL;
13786 |             return -1;
13787 |         }
13788 |
13789 |         for (;;) {
13790 |             char* pNewTempBuffer = (char*)ma_realloc(pTempBuffer, tempBufferCap, pAllocationCallbacks);
13791 |             if (pNewTempBuffer == NULL) {
13792 |                 ma_free(pTempBuffer, pAllocationCallbacks);
13793 |                 errno = ENOMEM;
13794 |                 return -1; /* Out of memory. */
13795 |             }
13796 |
13797 |             pTempBuffer = pNewTempBuffer;
13798 |
13799 |             result = _vsnprintf(pTempBuffer, tempBufferCap, format, args);
13800 |             ma_free(pTempBuffer, NULL);
13801 |
13802 |             if (result != -1) {
13803 |                 break; /* Got it. */
13804 |             }
13805 |
13806 |             /* Buffer wasn't big enough. Ideally it'd be nice to use an error code to know the reason
> for sure, but this is reliable enough. */
13807 |             tempBufferCap *= 2;
13808 |         }
13809 |
13810 |         return result;
13811 |     #endif
13812 | }
13813 | #endif
13814 |
13815 | MA_API ma_result ma_log_postv(ma_log* pLog, ma_uint32 level, const char* pFormat, va_list args)
13816 | {
13817 |     if (pLog == NULL || pFormat == NULL) {
13818 |         return MA_INVALID_ARGS;
13819 |     }
13820 |
13821 |     #if (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L) || (!defined(_MSC_VER) ||
> _MSC_VER >= 1900) && !defined(__STRICT_ANSI__) && !defined(_NO_EXT_KEYS) || (defined(__cplusplus)
> && __cplusplus >= 201103L)
13822 |     {
13823 |         ma_result result;
13824 |         int length;
13825 |         char pFormattedMessageStack[1024];
13826 |         char* pFormattedMessageHeap = NULL;
13827 |         va_list args2;
13828 |
13829 |         /* First try formatting into our fixed sized stack allocated buffer. If this is too small
> we'll fallback to a heap allocation. */
13830 |         ma_va_copy(args2, args);
13831 |         {
13832 |             length = vsnprintf(pFormattedMessageStack, sizeof(pFormattedMessageStack), pFormat,
> args2);
13833 |         }

```

```

13834 |         va_end(args2);
13835 |
13836 |         if (length < 0) {
13837 |             return MA_INVALID_OPERATION;    /* An error occurred when trying to convert the buffer.
> */
13838 |         }
13839 |
13840 |         if ((size_t)length < sizeof(pFormattedMessageStack)) {
13841 |             /* The string was written to the stack. */
13842 |             result = ma_log_post(pLog, level, pFormattedMessageStack);
13843 |         } else {
13844 |             /* The stack buffer was too small, try the heap. */
13845 |             pFormattedMessageHeap = (char*)ma_malloc(length + 1, &pLog->allocationCallbacks);
13846 |             if (pFormattedMessageHeap == NULL) {
13847 |                 return MA_OUT_OF_MEMORY;
13848 |             }
13849 |
13850 |             length = vsnprintf(pFormattedMessageHeap, length + 1, pFormat, args);
13851 |             if (length < 0) {
13852 |                 ma_free(pFormattedMessageHeap, &pLog->allocationCallbacks);
13853 |                 return MA_INVALID_OPERATION;
13854 |             }
13855 |
13856 |             result = ma_log_post(pLog, level, pFormattedMessageHeap);
13857 |             ma_free(pFormattedMessageHeap, &pLog->allocationCallbacks);
13858 |         }
13859 |
13860 |         return result;
13861 |     }
13862 |     #else
13863 |     {
13864 |         /*
13865 |         Without snprintf() we need to first measure the string and then heap allocate it. I'm only
> aware of Visual Studio having support for this without snprintf(), so we'll
13866 |         need to restrict this branch to Visual Studio. For other compilers we need to just not
> support formatted logging because I don't want the security risk of overflowing
13867 |         a fixed sized stack allocated buffer.
13868 |         */
13869 |         #if defined(_MSC_VER) && _MSC_VER >= 1200    /* 1200 = VC6 */
13870 |         {
13871 |             ma_result result;
13872 |             int formattedLen;
13873 |             char* pFormattedMessage = NULL;
13874 |             va_list args2;
13875 |
13876 |             ma_va_copy(args2, args);
13877 |             {
13878 |                 formattedLen = ma_vscprintf(&pLog->allocationCallbacks, pFormat, args2);
13879 |             }
13880 |             va_end(args2);
13881 |
13882 |             if (formattedLen <= 0) {
13883 |                 return MA_INVALID_OPERATION;
13884 |             }
13885 |
13886 |             pFormattedMessage = (char*)ma_malloc(formattedLen + 1, &pLog->allocationCallbacks);
13887 |             if (pFormattedMessage == NULL) {
13888 |                 return MA_OUT_OF_MEMORY;
13889 |             }
13890 |
13891 |             /* We'll get errors on newer versions of Visual Studio if we try to use vsprintf(). */
13892 |             #if _MSC_VER >= 1400    /* 1400 = Visual Studio 2005 */
13893 |             {
13894 |                 vsprintf_s(pFormattedMessage, formattedLen + 1, pFormat, args);
13895 |             }
13896 |             #else
13897 |             {
13898 |                 vsprintf(pFormattedMessage, pFormat, args);
13899 |             }
13900 |             #endif
13901 |
13902 |             result = ma_log_post(pLog, level, pFormattedMessage);
13903 |             ma_free(pFormattedMessage, &pLog->allocationCallbacks);
13904 |
13905 |             return result;
13906 |         }
13907 |     #else
13908 |     {
13909 |         /* Can't do anything because we don't have a safe way of to emulate vsnprintf() without

```

```

> a manual solution. */
13910 |         (void)level;
13911 |         (void)args;
13912 |
13913 |         return MA_INVALID_OPERATION;
13914 |     }
13915 | #endif
13916 | }
13917 | #endif
13918 | }
13919 |
13920 | MA_API ma_result ma_log_postf(ma_log* pLog, ma_uint32 level, const char* pFormat, ...)
13921 | {
13922 |     ma_result result;
13923 |     va_list args;
13924 |
13925 |     if (pLog == NULL || pFormat == NULL) {
13926 |         return MA_INVALID_ARGS;
13927 |     }
13928 |
13929 |     va_start(args, pFormat);
13930 |     {
13931 |         result = ma_log_postv(pLog, level, pFormat, args);
13932 |     }
13933 |     va_end(args);
13934 |
13935 |     return result;
13936 | }
13937 |
13938 |
13939 |
13940 | static MA_INLINE ma_uint8 ma_clip_u8(ma_int32 x)
13941 | {
13942 |     return (ma_uint8)(ma_clamp(x, -128, 127) + 128);
13943 | }
13944 |
13945 | static MA_INLINE ma_int16 ma_clip_s16(ma_int32 x)
13946 | {
13947 |     return (ma_int16)ma_clamp(x, -32768, 32767);
13948 | }
13949 |
13950 | static MA_INLINE ma_int64 ma_clip_s24(ma_int64 x)
13951 | {
13952 |     return (ma_int64)ma_clamp(x, -8388608, 8388607);
13953 | }
13954 |
13955 | static MA_INLINE ma_int32 ma_clip_s32(ma_int64 x)
13956 | {
13957 |     /* This dance is to silence warnings with -std=c89. A good compiler should be able to optimize
> this away. */
13958 |     ma_int64 clipMin;
13959 |     ma_int64 clipMax;
13960 |     clipMin = -((ma_int64)2147483647 + 1);
13961 |     clipMax = (ma_int64)2147483647;
13962 |
13963 |     return (ma_int32)ma_clamp(x, clipMin, clipMax);
13964 | }
13965 |
13966 | static MA_INLINE float ma_clip_f32(float x)
13967 | {
13968 |     if (x < -1) return -1;
13969 |     if (x > +1) return +1;
13970 |     return x;
13971 | }
13972 |
13973 |
13974 | static MA_INLINE float ma_mix_f32(float x, float y, float a)
13975 | {
13976 |     return x*(1-a) + y*a;
13977 | }
13978 | static MA_INLINE float ma_mix_f32_fast(float x, float y, float a)
13979 | {
13980 |     float r0 = (y - x);
13981 |     float r1 = r0*a;
13982 |     return x + r1;
13983 |     /*return x + (y - x)*a;*/
13984 | }
13985 |
13986 | #if defined(MA_SUPPORT_SSE2)

```



```

13987 | static MA_INLINE __m128 ma_mix_f32_fast__sse2(__m128 x, __m128 y, __m128 a)
13988 | {
13989 |     return _mm_add_ps(x, _mm_mul_ps(_mm_sub_ps(y, x), a));
13990 | }
13991 | #endif
13992 | #if defined(MA_SUPPORT_AVX2)
13993 | static MA_INLINE __m256 ma_mix_f32_fast__avx2(__m256 x, __m256 y, __m256 a)
13994 | {
13995 |     return _mm256_add_ps(x, _mm256_mul_ps(_mm256_sub_ps(y, x), a));
13996 | }
13997 | #endif
13998 | #if defined(MA_SUPPORT_NEON)
13999 | static MA_INLINE float32x4_t ma_mix_f32_fast__neon(float32x4_t x, float32x4_t y, float32x4_t a)
14000 | {
14001 |     return vaddq_f32(x, vmulq_f32(vsubq_f32(y, x), a));
14002 | }
14003 | #endif
14004 |
14005 |
14006 | static MA_INLINE double ma_mix_f64(double x, double y, double a)
14007 | {
14008 |     return x*(1-a) + y*a;
14009 | }
14010 | static MA_INLINE double ma_mix_f64_fast(double x, double y, double a)
14011 | {
14012 |     return x + (y - x)*a;
14013 | }
14014 |
14015 | static MA_INLINE float ma_scale_to_range_f32(float x, float lo, float hi)
14016 | {
14017 |     return lo + x*(hi-lo);
14018 | }
14019 |
14020 |
14021 | /*
14022 | Greatest common factor using Euclid's algorithm iteratively.
14023 | */
14024 | static MA_INLINE ma_uint32 ma_gcf_u32(ma_uint32 a, ma_uint32 b)
14025 | {
14026 |     for (;;) {
14027 |         if (b == 0) {
14028 |             break;
14029 |         } else {
14030 |             ma_uint32 t = a;
14031 |             a = b;
14032 |             b = t % a;
14033 |         }
14034 |     }
14035 |
14036 |     return a;
14037 | }
14038 |
14039 |
14040 | static ma_uint32 ma_ffs_32(ma_uint32 x)
14041 | {
14042 |     ma_uint32 i;
14043 |
14044 |     /* Just a naive implementation just to get things working for now. Will optimize this later. */
14045 |     for (i = 0; i < 32; i += 1) {
14046 |         if ((x & (1U << i)) != 0) {
14047 |             return i;
14048 |         }
14049 |     }
14050 |
14051 |     return i;
14052 | }
14053 |
14054 | static MA_INLINE ma_int16 ma_float_to_fixed_16(float x)
14055 | {
14056 |     return (ma_int16)(x * (1 << 8));
14057 | }
14058 |
14059 |
14060 | /*
14061 | Random Number Generation
14062 |
14063 | miniaudio uses the LCG random number generation algorithm. This is good enough for audio.
14064 |
14065 |

```

```

14066 | Note that miniaudio's global LCG implementation uses global state which is _not_ thread-local. When
> this is called across
14067 | multiple threads, results will be unpredictable. However, it won't crash and results will still be
> random enough for
14068 | miniaudio's purposes.
14069 | */
14070 | #ifndef MA_DEFAULT_LCG_SEED
14071 | #define MA_DEFAULT_LCG_SEED 4321
14072 | #endif
14073 |
14074 | #define MA_LCG_M    2147483647
14075 | #define MA_LCG_A    48271
14076 | #define MA_LCG_C    0
14077 |
14078 | static ma_lcg g_maLCG = {MA_DEFAULT_LCG_SEED}; /* Non-zero initial seed. Use ma_lcg_seed() to use an
> explicit seed. */
14079 |
14080 | static MA_INLINE void ma_lcg_seed(ma_lcg* pLCG, ma_int32 seed)
14081 | {
14082 |     MA_ASSERT(pLCG != NULL);
14083 |     pLCG->state = seed;
14084 | }
14085 |
14086 | static MA_INLINE ma_int32 ma_lcg_rand_s32(ma_lcg* pLCG)
14087 | {
14088 |     pLCG->state = (MA_LCG_A * pLCG->state + MA_LCG_C) % MA_LCG_M;
14089 |     return pLCG->state;
14090 | }
14091 |
14092 | static MA_INLINE ma_uint32 ma_lcg_rand_u32(ma_lcg* pLCG)
14093 | {
14094 |     return (ma_uint32)ma_lcg_rand_s32(pLCG);
14095 | }
14096 |
14097 | static MA_INLINE ma_int16 ma_lcg_rand_s16(ma_lcg* pLCG)
14098 | {
14099 |     return (ma_int16)(ma_lcg_rand_s32(pLCG) & 0xFFFF);
14100 | }
14101 |
14102 | static MA_INLINE double ma_lcg_rand_f64(ma_lcg* pLCG)
14103 | {
14104 |     return ma_lcg_rand_s32(pLCG) / (double)0x7FFFFFFF;
14105 | }
14106 |
14107 | static MA_INLINE float ma_lcg_rand_f32(ma_lcg* pLCG)
14108 | {
14109 |     return (float)ma_lcg_rand_f64(pLCG);
14110 | }
14111 |
14112 | static MA_INLINE float ma_lcg_rand_range_f32(ma_lcg* pLCG, float lo, float hi)
14113 | {
14114 |     return ma_scale_to_range_f32(ma_lcg_rand_f32(pLCG), lo, hi);
14115 | }
14116 |
14117 | static MA_INLINE ma_int32 ma_lcg_rand_range_s32(ma_lcg* pLCG, ma_int32 lo, ma_int32 hi)
14118 | {
14119 |     if (lo == hi) {
14120 |         return lo;
14121 |     }
14122 |
14123 |     return lo + ma_lcg_rand_u32(pLCG) / (0xFFFFFFFF / (hi - lo + 1) + 1);
14124 | }
14125 |
14126 |
14127 | #if 0 /* Currently unused. */
14128 | static MA_INLINE void ma_seed(ma_int32 seed)
14129 | {
14130 |     ma_lcg_seed(&g_maLCG, seed);
14131 | }
14132 |
14133 | static MA_INLINE ma_int32 ma_rand_s32(void)
14134 | {
14135 |     return ma_lcg_rand_s32(&g_maLCG);
14136 | }
14137 |
14138 | static MA_INLINE ma_uint32 ma_rand_u32(void)
14139 | {
14140 |     return ma_lcg_rand_u32(&g_maLCG);
14141 | }

```

```

14142 |
14143 | static MA_INLINE double ma_rand_f64(void)
14144 | {
14145 |     return ma_lcg_rand_f64(&g_maLCG);
14146 | }
14147 |
14148 | static MA_INLINE float ma_rand_f32(void)
14149 | {
14150 |     return ma_lcg_rand_f32(&g_maLCG);
14151 | }
14152 | #endif
14153 |
14154 | static MA_INLINE float ma_rand_range_f32(float lo, float hi)
14155 | {
14156 |     return ma_lcg_rand_range_f32(&g_maLCG, lo, hi);
14157 | }
14158 |
14159 | static MA_INLINE ma_int32 ma_rand_range_s32(ma_int32 lo, ma_int32 hi)
14160 | {
14161 |     return ma_lcg_rand_range_s32(&g_maLCG, lo, hi);
14162 | }
14163 |
14164 |
14165 | static MA_INLINE float ma_dither_f32_rectangle(float ditherMin, float ditherMax)
14166 | {
14167 |     return ma_rand_range_f32(ditherMin, ditherMax);
14168 | }
14169 |
14170 | static MA_INLINE float ma_dither_f32_triangle(float ditherMin, float ditherMax)
14171 | {
14172 |     float a = ma_rand_range_f32(ditherMin, 0);
14173 |     float b = ma_rand_range_f32(0, ditherMax);
14174 |     return a + b;
14175 | }
14176 |
14177 | static MA_INLINE float ma_dither_f32(ma_dither_mode ditherMode, float ditherMin, float ditherMax)
14178 | {
14179 |     if (ditherMode == ma_dither_mode_rectangle) {
14180 |         return ma_dither_f32_rectangle(ditherMin, ditherMax);
14181 |     }
14182 |     if (ditherMode == ma_dither_mode_triangle) {
14183 |         return ma_dither_f32_triangle(ditherMin, ditherMax);
14184 |     }
14185 |
14186 |     return 0;
14187 | }
14188 |
14189 | static MA_INLINE ma_int32 ma_dither_s32(ma_dither_mode ditherMode, ma_int32 ditherMin, ma_int32
> ditherMax)
14190 | {
14191 |     if (ditherMode == ma_dither_mode_rectangle) {
14192 |         ma_int32 a = ma_rand_range_s32(ditherMin, ditherMax);
14193 |         return a;
14194 |     }
14195 |     if (ditherMode == ma_dither_mode_triangle) {
14196 |         ma_int32 a = ma_rand_range_s32(ditherMin, 0);
14197 |         ma_int32 b = ma_rand_range_s32(0, ditherMax);
14198 |         return a + b;
14199 |     }
14200 |
14201 |     return 0;
14202 | }
14203 |
14204 |
14205 | /*****
> *****/
14206 |
14207 | Atomics
14208 |
14209 | *****/
> *****/
14210 | /* c89atomic.h begin */
14211 | #ifndef ma_atomic_h
14212 | #define ma_atomic_h
14213 | #if defined(__cplusplus)
14214 | extern "C" {
14215 | #endif
14216 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
> 6)))

```

```

14217 | #pragma GCC diagnostic push
14218 | #pragma GCC diagnostic ignored "-Wlong-long"
14219 | #if defined(__clang__)
14220 |     #pragma GCC diagnostic ignored "-Wc++11-long-long"
14221 | #endif
14222 | #endif
14223 | typedef int ma_atomic_memory_order;
14224 | #if !defined(MA_ATOMIC_MODERN_MSVC) && \
14225 |     !defined(MA_ATOMIC_LEGACY_MSVC) && \
14226 |     !defined(MA_ATOMIC_LEGACY_MSVC_ASM) && \
14227 |     !defined(MA_ATOMIC_MODERN_GCC) && \
14228 |     !defined(MA_ATOMIC_LEGACY_GCC) && \
14229 |     !defined(MA_ATOMIC_LEGACY_GCC_ASM)
14230 | #if defined(_MSC_VER) || defined(_WATCOMC_) || defined(__DMC__) || defined(__BORLANDC__)
14231 |     #if (defined(_MSC_VER) && _MSC_VER > 1600)
14232 |         #define MA_ATOMIC_MODERN_MSVC
14233 |     #else
14234 |         #if defined(MA_X64)
14235 |             #define MA_ATOMIC_LEGACY_MSVC
14236 |         #else
14237 |             #define MA_ATOMIC_LEGACY_MSVC_ASM
14238 |         #endif
14239 |     #endif
14240 | #elif (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >= 7))) ||
> defined(__clang__)
14241 |     #define MA_ATOMIC_MODERN_GCC
14242 | #else
14243 |     #if defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >= 1))
14244 |         #define MA_ATOMIC_LEGACY_GCC
14245 |     #else
14246 |         #define MA_ATOMIC_LEGACY_GCC_ASM
14247 |     #endif
14248 | #endif
14249 | #endif
14250 | #if defined(MA_ATOMIC_MODERN_MSVC) || defined(MA_ATOMIC_LEGACY_MSVC)
14251 |     #include <intrin.h>
14252 |     #define ma_atomic_memory_order_relaxed 1
14253 |     #define ma_atomic_memory_order_consume 2
14254 |     #define ma_atomic_memory_order_acquire 3
14255 |     #define ma_atomic_memory_order_release 4
14256 |     #define ma_atomic_memory_order_acq_rel 5
14257 |     #define ma_atomic_memory_order_seq_cst 6
14258 |     #define MA_ATOMIC_MSVC_ARM_INTRINSIC_NORETURN(dst, src, order, intrin, ma_atomicType, msvcType)
> \
14259 |         switch (order) \
14260 |         { \
14261 |             case ma_atomic_memory_order_relaxed: \
14262 |                 { \
14263 |                     intrin##_nf((volatile msvcType*)dst, (msvcType)src); \
14264 |                 } break; \
14265 |             case ma_atomic_memory_order_consume: \
14266 |             case ma_atomic_memory_order_acquire: \
14267 |                 { \
14268 |                     intrin##_acq((volatile msvcType*)dst, (msvcType)src); \
14269 |                 } break; \
14270 |             case ma_atomic_memory_order_release: \
14271 |                 { \
14272 |                     intrin##_rel((volatile msvcType*)dst, (msvcType)src); \
14273 |                 } break; \
14274 |             case ma_atomic_memory_order_acq_rel: \
14275 |             case ma_atomic_memory_order_seq_cst: \
14276 |             default: \
14277 |                 { \
14278 |                     intrin((volatile msvcType*)dst, (msvcType)src); \
14279 |                 } break; \
14280 |         } \
14281 |     #define MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, intrin, ma_atomicType, msvcType) \
14282 |         ma_atomicType result; \
14283 |         switch (order) \
14284 |         { \
14285 |             case ma_atomic_memory_order_relaxed: \
14286 |                 { \
14287 |                     result = (ma_atomicType)intrin##_nf((volatile msvcType*)dst, (msvcType)src); \
14288 |                 } break; \
14289 |             case ma_atomic_memory_order_consume: \
14290 |             case ma_atomic_memory_order_acquire: \
14291 |                 { \
14292 |                     result = (ma_atomicType)intrin##_acq((volatile msvcType*)dst, (msvcType)src); \
14293 |                 } break; \

```

```

14294 |         case ma_atomic_memory_order_release: \
14295 |         { \
14296 |             result = (ma_atomicType)intrin##_rel((volatile msvcType*)dst, (msvcType)src); \
14297 |         } break; \
14298 |         case ma_atomic_memory_order_acq_rel: \
14299 |         case ma_atomic_memory_order_seq_cst: \
14300 |         default: \
14301 |         { \
14302 |             result = (ma_atomicType)intrin((volatile msvcType*)dst, (msvcType)src); \
14303 |         } break; \
14304 |     } \
14305 |     return result;
14306 |     typedef ma_uint32 ma_atomic_flag;
14307 |     static MA_INLINE ma_atomic_flag ma_atomic_flag_test_and_set_explicit(volatile ma_atomic_flag*
> dst, ma_atomic_memory_order order)
14308 |     {
14309 |         #if defined(MA_ARM)
14310 |         {
14311 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, 1, order, _InterlockedExchange, ma_atomic_flag, long);
14312 |         }
14313 |         #else
14314 |         {
14315 |             (void)order;
14316 |             return (ma_atomic_flag)_InterlockedExchange((volatile long*)dst, (long)1);
14317 |         }
14318 |         #endif
14319 |     }
14320 |     static MA_INLINE void ma_atomic_flag_clear_explicit(volatile ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14321 |     {
14322 |         #if defined(MA_ARM)
14323 |         {
14324 |             MA_ATOMIC_MSVC_ARM_INTRINSIC_NORETURN(dst, 0, order, _InterlockedExchange,
> ma_atomic_flag, long);
14325 |         }
14326 |         #else
14327 |         {
14328 |             (void)order;
14329 |             _InterlockedExchange((volatile long*)dst, (long)0);
14330 |         }
14331 |         #endif
14332 |     }
14333 |     static MA_INLINE ma_atomic_flag ma_atomic_flag_load_explicit(volatile const ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14334 |     {
14335 |         (void)order;
14336 |         return (ma_uint32)_InterlockedCompareExchange((volatile long*)dst, 0, 0);
14337 |     }
14338 | #endif
14339 | #if defined(MA_ATOMIC_LEGACY_MSVC_ASM)
14340 | #define ma_atomic_memory_order_relaxed 1
14341 | #define ma_atomic_memory_order_consume 2
14342 | #define ma_atomic_memory_order_acquire 3
14343 | #define ma_atomic_memory_order_release 4
14344 | #define ma_atomic_memory_order_acq_rel 5
14345 | #define ma_atomic_memory_order_seq_cst 6
14346 | typedef ma_uint32 ma_atomic_flag;
14347 | static MA_INLINE ma_atomic_flag ma_atomic_flag_test_and_set_explicit(volatile ma_atomic_flag*
> dst, ma_atomic_memory_order order)
14348 | {
14349 |     ma_atomic_flag result = 0;
14350 |     (void)order;
14351 |     __asm {
14352 |         mov ecx, dst
14353 |         mov eax, 1
14354 |         xchg [ecx], eax
14355 |         mov result, eax
14356 |     }
14357 |     return result;
14358 | }
14359 | static MA_INLINE void ma_atomic_flag_clear_explicit(volatile ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14360 | {
14361 |     if (order == ma_atomic_memory_order_relaxed) {
14362 |         __asm {
14363 |             mov esi, dst
14364 |             mov dword ptr [esi], 0
14365 |         }
14366 |     } else {

```

```

14367 |         __asm {
14368 |             mov esi, dst
14369 |             mov eax, 0
14370 |             xchg [esi], eax
14371 |         }
14372 |     }
14373 | }
14374 | static MA_INLINE ma_atomic_flag ma_atomic_flag_load_explicit(volatile const ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14375 | {
14376 |     ma_atomic_flag result = 0;
14377 |     if (order == ma_atomic_memory_order_relaxed) {
14378 |         __asm {
14379 |             mov esi, dst
14380 |             mov eax, [esi]
14381 |             mov result, eax
14382 |         }
14383 |     } else if (order <= ma_atomic_memory_order_release) {
14384 |         __asm {
14385 |             mov esi, dst
14386 |             mov eax, [esi]
14387 |             lock add dword ptr [esp], 0
14388 |             mov result, eax
14389 |         }
14390 |     } else {
14391 |         __asm {
14392 |             lock add dword ptr [esp], 0
14393 |             mov esi, dst
14394 |             mov eax, [esi]
14395 |             mov result, eax
14396 |             lock add dword ptr [esp], 0
14397 |         }
14398 |     }
14399 |     return result;
14400 | }
14401 | #endif
14402 | #if defined(MA_ATOMIC_MODERN_GCC)
14403 | #define ma_atomic_memory_order_relaxed    __ATOMIC_RELAXED
14404 | #define ma_atomic_memory_order_consume    __ATOMIC_CONSUME
14405 | #define ma_atomic_memory_order_acquire    __ATOMIC_ACQUIRE
14406 | #define ma_atomic_memory_order_release    __ATOMIC_RELEASE
14407 | #define ma_atomic_memory_order_acq_rel    __ATOMIC_ACQ_REL
14408 | #define ma_atomic_memory_order_seq_cst    __ATOMIC_SEQ_CST
14409 | typedef ma_uint32 ma_atomic_flag;
14410 | #define ma_atomic_flag_test_and_set_explicit(dst, order) __atomic_exchange_n(dst, 1, order)
14411 | #define ma_atomic_flag_clear_explicit(dst, order)      __atomic_store_n(dst, 0, order)
14412 | #define ma_atomic_flag_load_explicit(dst, order)        __atomic_load_n(dst, order)
14413 | #endif
14414 | #if defined(MA_ATOMIC_LEGACY_GCC)
14415 | #define ma_atomic_memory_order_relaxed 1
14416 | #define ma_atomic_memory_order_consume 2
14417 | #define ma_atomic_memory_order_acquire 3
14418 | #define ma_atomic_memory_order_release 4
14419 | #define ma_atomic_memory_order_acq_rel 5
14420 | #define ma_atomic_memory_order_seq_cst 6
14421 | typedef ma_uint32 ma_atomic_flag;
14422 | static MA_INLINE ma_atomic_flag ma_atomic_flag_test_and_set_explicit(volatile ma_atomic_flag*
> dst, ma_atomic_memory_order order)
14423 | {
14424 |     if (order > ma_atomic_memory_order_acquire) {
14425 |         __sync_synchronize();
14426 |     }
14427 |     return __sync_lock_test_and_set(dst, 1);
14428 | }
14429 | static MA_INLINE void ma_atomic_flag_clear_explicit(volatile ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14430 | {
14431 |     if (order > ma_atomic_memory_order_release) {
14432 |         __sync_synchronize();
14433 |     }
14434 |     __sync_lock_release(dst);
14435 | }
14436 | static MA_INLINE ma_atomic_flag ma_atomic_flag_load_explicit(volatile const ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14437 | {
14438 |     (void)order;
14439 |     return __sync_val_compare_and_swap((ma_atomic_flag*)dst, 0, 0);
14440 | }
14441 | #endif

```

```

14442 | #if defined(MA_ATOMIC_LEGACY_GCC_ASM)
14443 |     #define ma_atomic_memory_order_relaxed 1
14444 |     #define ma_atomic_memory_order_consume 2
14445 |     #define ma_atomic_memory_order_acquire 3
14446 |     #define ma_atomic_memory_order_release 4
14447 |     #define ma_atomic_memory_order_acq_rel 5
14448 |     #define ma_atomic_memory_order_seq_cst 6
14449 |     #if defined(MA_X86)
14450 |         #define ma_atomic_thread_fence(order) __asm__ __volatile__("lock; addl $0, (%esp)" :::
> "memory")
14451 |     #elif defined(MA_X64)
14452 |         #define ma_atomic_thread_fence(order) __asm__ __volatile__("lock; addq $0, (%rsp)" :::
> "memory")
14453 |     #else
14454 |         #error Unsupported architecture.
14455 |     #endif
14456 |     #define MA_ATOMIC_XCHG_GCC_X86(instructionSizeSuffix, result, dst, src) \
14457 |         __asm__ __volatile__(\
14458 |             "xchg" instructionSizeSuffix " %0, %1" \
14459 |             : "=r"(result), \
14460 |             "=m"(*dst) \
14461 |             : "0"(src), \
14462 |             "m"(*dst) \
14463 |             : "memory" \
14464 |         )
14465 |     #define MA_ATOMIC_LOAD_RELAXED_GCC_X86(instructionSizeSuffix, result, dst) \
14466 |         __asm__ __volatile__(\
14467 |             "mov" instructionSizeSuffix " %1, %0" \
14468 |             : "=r"(result) \
14469 |             : "m"(*dst) \
14470 |         )
14471 |     #define MA_ATOMIC_LOAD_RELEASE_GCC_X86(instructionSizeSuffix, result, dst) \
14472 |         ma_atomic_thread_fence(ma_atomic_memory_order_release); \
14473 |         __asm__ __volatile__(\
14474 |             "mov" instructionSizeSuffix " %1, %0" \
14475 |             : "=r"(result) \
14476 |             : "m"(*dst) \
14477 |             : "memory" \
14478 |         )
14479 |     #define MA_ATOMIC_LOAD_SEQ_CST_GCC_X86(instructionSizeSuffix, result, dst) \
14480 |         ma_atomic_thread_fence(ma_atomic_memory_order_seq_cst); \
14481 |         __asm__ __volatile__(\
14482 |             "mov" instructionSizeSuffix " %1, %0" \
14483 |             : "=r"(result) \
14484 |             : "m"(*dst) \
14485 |             : "memory" \
14486 |         );
14487 |         ma_atomic_thread_fence(ma_atomic_memory_order_seq_cst)
14488 |         typedef ma_uint32 ma_atomic_flag;
14489 |         static MA_INLINE ma_atomic_flag ma_atomic_flag_test_and_set_explicit(volatile ma_atomic_flag*
> dst, ma_atomic_memory_order order)
14490 |         {
14491 |             ma_atomic_flag result;
14492 |             #if defined(MA_X86) || defined(MA_X64)
14493 |             {
14494 |                 (void)order;
14495 |                 MA_ATOMIC_XCHG_GCC_X86("l", result, dst, 1);
14496 |             }
14497 |             #else
14498 |             {
14499 |                 #error Unsupported architecture.
14500 |             }
14501 |             #endif
14502 |             return result;
14503 |         }
14504 |         static MA_INLINE void ma_atomic_flag_clear_explicit(volatile ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14505 |         {
14506 |             #if defined(MA_X86) || defined(MA_X64)
14507 |             {
14508 |                 if (order == ma_atomic_memory_order_relaxed) {
14509 |                     __asm__ __volatile__(\
14510 |                         "movl $0, %0" \
14511 |                         : "=m"(*dst) \
14512 |                     );
14513 |                 } else if (order == ma_atomic_memory_order_release) {
14514 |                     __asm__ __volatile__(\
14515 |                         "movl $0, %0" \
14516 |                         : "=m"(*dst)

```



```

14517 |         :
14518 |         : "memory"
14519 |     );
14520 |     } else {
14521 |         ma_atomic_flag tmp = 0;
14522 |         __asm__ __volatile__(
14523 |             "xchgl %0, %1"
14524 |             : "=r"(tmp),
14525 |             "=m"(*dst)
14526 |             : "0"(tmp),
14527 |             "m"(*dst)
14528 |             : "memory"
14529 |         );
14530 |     }
14531 | }
14532 | #else
14533 | {
14534 |     #error Unsupported architecture.
14535 | }
14536 | #endif
14537 | }
14538 | static MA_INLINE ma_atomic_flag ma_atomic_flag_load_explicit(volatile const ma_atomic_flag* dst,
> ma_atomic_memory_order order)
14539 | {
14540 |     #if defined(MA_X86) || defined(MA_X64)
14541 |     {
14542 |         ma_atomic_flag result;
14543 |         if (order == ma_atomic_memory_order_relaxed) {
14544 |             MA_ATOMIC_LOAD_RELAXED_GCC_X86("l", result, dst);
14545 |         } else if (order <= ma_atomic_memory_order_release) {
14546 |             MA_ATOMIC_LOAD_RELEASE_GCC_X86("l", result, dst);
14547 |         } else {
14548 |             MA_ATOMIC_LOAD_SEQ_CST_GCC_X86("l", result, dst);
14549 |         }
14550 |         return result;
14551 |     }
14552 |     #else
14553 |     {
14554 |         #error Unsupported architecture.
14555 |     }
14556 |     #endif
14557 | }
14558 | #endif
14559 | #define ma_atomic_flag_test_and_set(dst) ma_atomic_flag_test_and_set_explicit(dst,
> ma_atomic_memory_order_acquire)
14560 | #define ma_atomic_flag_clear(dst) ma_atomic_flag_clear_explicit(dst,
> ma_atomic_memory_order_release)
14561 | typedef ma_atomic_flag ma_atomic_spinlock;
14562 | static MA_INLINE void ma_atomic_spinlock_lock(volatile ma_atomic_spinlock* pSpinlock)
14563 | {
14564 |     for (;;) {
14565 |         if (ma_atomic_flag_test_and_set_explicit(pSpinlock, ma_atomic_memory_order_acquire) == 0) {
14566 |             break;
14567 |         }
14568 |         while (ma_atomic_flag_load_explicit(pSpinlock, ma_atomic_memory_order_relaxed) == 1) {
14569 |         }
14570 |     }
14571 | }
14572 | static MA_INLINE void ma_atomic_spinlock_unlock(volatile ma_atomic_spinlock* pSpinlock)
14573 | {
14574 |     ma_atomic_flag_clear_explicit(pSpinlock, ma_atomic_memory_order_release);
14575 | }
14576 | ma_atomic_spinlock ma_atomic_global_lock;
14577 | #if defined(MA_ATOMIC_MODERN_MSVC) || defined(MA_ATOMIC_LEGACY_MSVC) ||
> defined(MA_ATOMIC_LEGACY_MSVC_ASM) || defined(MA_ATOMIC_LEGACY_GCC) ||
> defined(MA_ATOMIC_LEGACY_GCC_ASM)
14578 |     #if defined(MA_X64) || (defined(MA_X86) && ((defined(__GNUC__) && defined(__i486__)) ||
> (defined(_M_IX86) && _M_IX86 >= 400)))
14579 |         #if defined(MA_ATOMIC_LEGACY_MSVC) && defined(MA_X64)
14580 |         #else
14581 |             #define MA_ATOMIC_IS_LOCK_FREE_8 1
14582 |             #define MA_ATOMIC_IS_LOCK_FREE_16 1
14583 |         #endif
14584 |         #define MA_ATOMIC_IS_LOCK_FREE_32 1
14585 |         #if defined(MA_X64) || (defined(MA_X86) && ((defined(__GNUC__) && defined(__i586__)) ||
> (defined(_M_IX86) && _M_IX86 >= 500)))
14586 |             #define MA_ATOMIC_IS_LOCK_FREE_64 1
14587 |         #else
14588 |         #endif

```

```

14589 | #else
14590 | #endif
14591 | #if defined(MA_ARM32) || defined(MA_ARM64)
14592 |     #define MA_ATOMIC_IS_LOCK_FREE_8 1
14593 |     #define MA_ATOMIC_IS_LOCK_FREE_16 1
14594 |     #define MA_ATOMIC_IS_LOCK_FREE_32 1
14595 |     #if defined(MA_ARM64) || defined(__ARM_ARCH_7A__) || defined(__ARM_ARCH_7R__) ||
> defined(__ARM_ARCH_6K__) || defined(__ARM_ARCH_6Z__) || defined(__ARM_ARCH_6ZK__)
14596 |         #define MA_ATOMIC_IS_LOCK_FREE_64 1
14597 |     #endif
14598 | #endif
14599 | #if defined(MA_ATOMIC_PPC32) || defined(MA_ATOMIC_PPC64)
14600 |     #if (defined(__GNUC__) && (__GNUC__ < 4 || (__GNUC__ == 4 && __GNUC_MINOR__ < 7))) &&
> !defined(__clang__)
14601 |     #else
14602 |         #define MA_ATOMIC_IS_LOCK_FREE_8 1
14603 |         #define MA_ATOMIC_IS_LOCK_FREE_16 1
14604 |     #endif
14605 |     #define MA_ATOMIC_IS_LOCK_FREE_32 1
14606 |     #if defined(MA_ATOMIC_PPC64)
14607 |         #define MA_ATOMIC_IS_LOCK_FREE_64 1
14608 |     #endif
14609 | #endif
14610 | static MA_INLINE ma_bool32 ma_atomic_is_lock_free_8(volatile void* ptr)
14611 | {
14612 |     (void)ptr;
14613 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
14614 |         return 1;
14615 |     #else
14616 |         return 0;
14617 |     #endif
14618 | }
14619 | static MA_INLINE ma_bool32 ma_atomic_is_lock_free_16(volatile void* ptr)
14620 | {
14621 |     (void)ptr;
14622 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
14623 |         return 1;
14624 |     #else
14625 |         return 0;
14626 |     #endif
14627 | }
14628 | static MA_INLINE ma_bool32 ma_atomic_is_lock_free_32(volatile void* ptr)
14629 | {
14630 |     (void)ptr;
14631 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
14632 |         return 1;
14633 |     #else
14634 |         return 0;
14635 |     #endif
14636 | }
14637 | static MA_INLINE ma_bool32 ma_atomic_is_lock_free_64(volatile void* ptr)
14638 | {
14639 |     (void)ptr;
14640 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
14641 |         return 1;
14642 |     #else
14643 |         return 0;
14644 |     #endif
14645 | }
14646 | #endif
14647 | #define MA_ATOMIC_COMPARE_AND_SWAP_LOCK(sizeInBits, dst, expected, replacement) \
14648 |     ma_uint##sizeInBits result; \
14649 |     ma_atomic_spinlock_lock(&ma_atomic_global_lock); \
14650 |     { \
14651 |         result = *dst; \
14652 |         if (result == expected) { \
14653 |             *dst = replacement; \
14654 |         } \
14655 |     } \
14656 |     ma_atomic_spinlock_unlock(&ma_atomic_global_lock); \
14657 |     return result
14658 | #define MA_ATOMIC_LOAD_EXPLICIT_LOCK(sizeInBits, ptr, order) \
14659 |     ma_uint##sizeInBits result; \
14660 |     ma_atomic_spinlock_lock(&ma_atomic_global_lock); \
14661 |     { \
14662 |         result = *ptr; \
14663 |         (void)order; \
14664 |     } \
14665 |     ma_atomic_spinlock_unlock(&ma_atomic_global_lock); \

```

```

14666 |     return result
14667 | #define MA_ATOMIC_STORE_EXPLICIT_LOCK(sizeInBits, dst, src, order) \
14668 |     ma_atomic_spinlock_lock(&ma_atomic_global_lock); \
14669 |     { \
14670 |         *dst = src; \
14671 |         (void)order; \
14672 |     } \
14673 |     ma_atomic_spinlock_unlock(&ma_atomic_global_lock)
14674 | #define MA_ATOMIC_STORE_EXPLICIT_CAS(sizeInBits, dst, src, order) \
14675 |     ma_uint##sizeInBits oldValue; \
14676 |     do { \
14677 |         oldValue = ma_atomic_load_explicit_##sizeInBits(dst, ma_atomic_memory_order_relaxed); \
14678 |     } while (ma_atomic_compare_and_swap_##sizeInBits(dst, oldValue, src) != oldValue); \
14679 |     (void)order
14680 | #define MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(sizeInBits, dst, src, order) \
14681 |     ma_uint##sizeInBits result; \
14682 |     ma_atomic_spinlock_lock(&ma_atomic_global_lock); \
14683 |     { \
14684 |         result = *dst; \
14685 |         *dst = src; \
14686 |         (void)order; \
14687 |     } \
14688 |     ma_atomic_spinlock_unlock(&ma_atomic_global_lock); \
14689 |     return result
14690 | #define MA_ATOMIC_EXCHANGE_EXPLICIT_CAS(sizeInBits, dst, src, order) \
14691 |     ma_uint##sizeInBits oldValue; \
14692 |     do { \
14693 |         oldValue = ma_atomic_load_explicit_##sizeInBits(dst, ma_atomic_memory_order_relaxed); \
14694 |     } while (ma_atomic_compare_and_swap_##sizeInBits(dst, oldValue, src) != oldValue); \
14695 |     (void)order; \
14696 |     return oldValue
14697 | #define MA_ATOMIC_FETCH_ADD_LOCK(sizeInBits, dst, src, order) \
14698 |     ma_uint##sizeInBits result; \
14699 |     ma_atomic_spinlock_lock(&ma_atomic_global_lock); \
14700 |     { \
14701 |         result = *dst; \
14702 |         *dst += src; \
14703 |         (void)order; \
14704 |     } \
14705 |     ma_atomic_spinlock_unlock(&ma_atomic_global_lock); \
14706 |     return result
14707 | #define MA_ATOMIC_FETCH_ADD_CAS(sizeInBits, dst, src, order) \
14708 |     ma_uint##sizeInBits oldValue; \
14709 |     ma_uint##sizeInBits newValue; \
14710 |     do { \
14711 |         oldValue = ma_atomic_load_explicit_##sizeInBits(dst, ma_atomic_memory_order_relaxed); \
14712 |         newValue = oldValue + src; \
14713 |     } while (ma_atomic_compare_and_swap_##sizeInBits(dst, oldValue, newValue) != oldValue); \
14714 |     (void)order; \
14715 |     return oldValue
14716 | #define MA_ATOMIC_FETCH_AND_CAS(sizeInBits, dst, src, order) \
14717 |     ma_uint##sizeInBits oldValue; \
14718 |     ma_uint##sizeInBits newValue; \
14719 |     do { \
14720 |         oldValue = ma_atomic_load_explicit_##sizeInBits(dst, ma_atomic_memory_order_relaxed); \
14721 |         newValue = (ma_uint##sizeInBits)(oldValue & src); \
14722 |     } while (ma_atomic_compare_and_swap_##sizeInBits(dst, oldValue, newValue) != oldValue); \
14723 |     (void)order; \
14724 |     return oldValue
14725 | #define MA_ATOMIC_FETCH_OR_CAS(sizeInBits, dst, src, order) \
14726 |     ma_uint##sizeInBits oldValue; \
14727 |     ma_uint##sizeInBits newValue; \
14728 |     do { \
14729 |         oldValue = ma_atomic_load_explicit_##sizeInBits(dst, ma_atomic_memory_order_relaxed); \
14730 |         newValue = (ma_uint##sizeInBits)(oldValue | src); \
14731 |     } while (ma_atomic_compare_and_swap_##sizeInBits(dst, oldValue, newValue) != oldValue); \
14732 |     (void)order; \
14733 |     return oldValue
14734 | #define MA_ATOMIC_FETCH_XOR_CAS(sizeInBits, dst, src, order) \
14735 |     ma_uint##sizeInBits oldValue; \
14736 |     ma_uint##sizeInBits newValue; \
14737 |     do { \
14738 |         oldValue = ma_atomic_load_explicit_##sizeInBits(dst, ma_atomic_memory_order_relaxed); \
14739 |         newValue = (ma_uint##sizeInBits)(oldValue ^ src); \
14740 |     } while (ma_atomic_compare_and_swap_##sizeInBits(dst, oldValue, newValue) != oldValue); \
14741 |     (void)order; \
14742 |     return oldValue
14743 | #if defined(MA_ATOMIC_MODERN_MSVC) || defined(MA_ATOMIC_LEGACY_MSVC)
14744 |     #define MA_ATOMIC_MSVC_ARM_INTRINSIC_COMPARE_EXCHANGE(ptr, expected, replacement, order, intrin,

```

```

> ma_atomicType, msvcType) \
14745 |     ma_atomicType result; \
14746 |     switch (order) \
14747 |     { \
14748 |         case ma_atomic_memory_order_relaxed: \
14749 |         { \
14750 |             result = (ma_atomicType)intrin##_nf((volatile msvcType*)ptr, (msvcType)expected,
> (msvcType)replacement); \
14751 |         } break; \
14752 |         case ma_atomic_memory_order_consume: \
14753 |         case ma_atomic_memory_order_acquire: \
14754 |         { \
14755 |             result = (ma_atomicType)intrin##_acq((volatile msvcType*)ptr, (msvcType)expected,
> (msvcType)replacement); \
14756 |         } break; \
14757 |         case ma_atomic_memory_order_release: \
14758 |         { \
14759 |             result = (ma_atomicType)intrin##_rel((volatile msvcType*)ptr, (msvcType)expected,
> (msvcType)replacement); \
14760 |         } break; \
14761 |         case ma_atomic_memory_order_acq_rel: \
14762 |         case ma_atomic_memory_order_seq_cst: \
14763 |         default: \
14764 |         { \
14765 |             result = (ma_atomicType)intrin((volatile msvcType*)ptr, (msvcType)expected,
> (msvcType)replacement); \
14766 |         } break; \
14767 |     } \
14768 |     return result;
14769 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
14770 |     #define ma_atomic_compare_and_swap_8(dst, expected, replacement) (ma_uint8
> )_InterlockedCompareExchange8((volatile char*)dst, (char)replacement, (char)expected)
14771 |     #else
14772 |     static MA_INLINE ma_uint8 __stdcall ma_atomic_compare_and_swap_8(volatile ma_uint8* dst,
> ma_uint8 expected, ma_uint8 replacement)
14773 |     {
14774 |         MA_ATOMIC_COMPARE_AND_SWAP_LOCK(8, dst, expected, replacement);
14775 |     }
14776 |     #endif
14777 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
14778 |     #define ma_atomic_compare_and_swap_16(dst, expected, replacement)
> (ma_uint16)_InterlockedCompareExchange16((volatile short*)dst, (short)replacement, (short)expected)
14779 |     #else
14780 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_compare_and_swap_16(volatile ma_uint16* dst,
> ma_uint16 expected, ma_uint16 replacement)
14781 |     {
14782 |         MA_ATOMIC_COMPARE_AND_SWAP_LOCK(16, dst, expected, replacement);
14783 |     }
14784 |     #endif
14785 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
14786 |     #define ma_atomic_compare_and_swap_32(dst, expected, replacement)
> (ma_uint32)_InterlockedCompareExchange((volatile long*)dst, (long)replacement, (long)expected)
14787 |     #else
14788 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_compare_and_swap_32(volatile ma_uint32* dst,
> ma_uint32 expected, ma_uint32 replacement)
14789 |     {
14790 |         MA_ATOMIC_COMPARE_AND_SWAP_LOCK(32, dst, expected, replacement);
14791 |     }
14792 |     #endif
14793 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
14794 |     #define ma_atomic_compare_and_swap_64(dst, expected, replacement)
> (ma_uint64)_InterlockedCompareExchange64((volatile ma_int64*)dst, (ma_int64)replacement,
> (ma_int64)expected)
14795 |     #else
14796 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_compare_and_swap_64(volatile ma_uint64* dst,
> ma_uint64 expected, ma_uint64 replacement)
14797 |     {
14798 |         MA_ATOMIC_COMPARE_AND_SWAP_LOCK(64, dst, expected, replacement);
14799 |     }
14800 |     #endif
14801 |     static MA_INLINE ma_uint8 ma_atomic_load_explicit_8(volatile const ma_uint8* ptr,
> ma_atomic_memory_order order)
14802 |     {
14803 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
14804 |         {
14805 |             #if defined(MA_ARM)
14806 |             {
14807 |                 MA_ATOMIC_MSVC_ARM_INTRINSIC_COMPARE_EXCHANGE(ptr, 0, 0, order,
> _InterlockedCompareExchange8, ma_uint8, char);

```

```

14808 |     }
14809 |     #else
14810 |     {
14811 |         (void)order;
14812 |         return ma_atomic_compare_and_swap_8((volatile ma_uint8*)ptr, 0, 0);
14813 |     }
14814 |     #endif
14815 | }
14816 | #else
14817 | {
14818 |     MA_ATOMIC_LOAD_EXPLICIT_LOCK(8, ptr, order);
14819 | }
14820 | #endif
14821 | }
14822 | static MA_INLINE ma_uint16 ma_atomic_load_explicit_16(volatile const ma_uint16* ptr,
> ma_atomic_memory_order order)
14823 | {
14824 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
14825 |     {
14826 |         #if defined(MA_ARM)
14827 |         {
14828 |             MA_ATOMIC_MSVC_ARM_INTRINSIC_COMPARE_EXCHANGE(ptr, 0, 0, order,
> _InterlockedCompareExchange16, ma_uint16, short);
14829 |         }
14830 |         #else
14831 |         {
14832 |             (void)order;
14833 |             return ma_atomic_compare_and_swap_16((volatile ma_uint16*)ptr, 0, 0);
14834 |         }
14835 |         #endif
14836 |     }
14837 |     #else
14838 |     {
14839 |         MA_ATOMIC_LOAD_EXPLICIT_LOCK(16, ptr, order);
14840 |     }
14841 |     #endif
14842 | }
14843 | static MA_INLINE ma_uint32 ma_atomic_load_explicit_32(volatile const ma_uint32* ptr,
> ma_atomic_memory_order order)
14844 | {
14845 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
14846 |     {
14847 |         #if defined(MA_ARM)
14848 |         {
14849 |             MA_ATOMIC_MSVC_ARM_INTRINSIC_COMPARE_EXCHANGE(ptr, 0, 0, order,
> _InterlockedCompareExchange, ma_uint32, long);
14850 |         }
14851 |         #else
14852 |         {
14853 |             (void)order;
14854 |             return ma_atomic_compare_and_swap_32((volatile ma_uint32*)ptr, 0, 0);
14855 |         }
14856 |         #endif
14857 |     }
14858 |     #else
14859 |     {
14860 |         MA_ATOMIC_LOAD_EXPLICIT_LOCK(32, ptr, order);
14861 |     }
14862 |     #endif
14863 | }
14864 | static MA_INLINE ma_uint64 ma_atomic_load_explicit_64(volatile const ma_uint64* ptr,
> ma_atomic_memory_order order)
14865 | {
14866 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
14867 |     {
14868 |         #if defined(MA_ARM)
14869 |         {
14870 |             MA_ATOMIC_MSVC_ARM_INTRINSIC_COMPARE_EXCHANGE(ptr, 0, 0, order,
> _InterlockedCompareExchange64, ma_uint64, long long);
14871 |         }
14872 |         #else
14873 |         {
14874 |             (void)order;
14875 |             return ma_atomic_compare_and_swap_64((volatile ma_uint64*)ptr, 0, 0);
14876 |         }
14877 |         #endif
14878 |     }
14879 |     #else
14880 |     {

```

```

14881 |         MA_ATOMIC_LOAD_EXPLICIT_LOCK(64, ptr, order);
14882 |     }
14883 |     #endif
14884 | }
14885 | static MA_INLINE ma_uint8 __stdcall ma_atomic_exchange_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
14886 | {
14887 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
14888 |     {
14889 |         #if defined(MA_ARM)
14890 |         {
14891 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchange8, ma_uint8,
> char);
14892 |         }
14893 |         #else
14894 |         {
14895 |             (void)order;
14896 |             return (ma_uint8)_InterlockedExchange8((volatile char*)dst, (char)src);
14897 |         }
14898 |         #endif
14899 |     }
14900 |     #else
14901 |     {
14902 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(8, dst, src, order);
14903 |     }
14904 |     #endif
14905 | }
14906 | static MA_INLINE ma_uint16 __stdcall ma_atomic_exchange_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
14907 | {
14908 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
14909 |     {
14910 |         #if defined(MA_ARM)
14911 |         {
14912 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchange16, ma_uint16,
> short);
14913 |         }
14914 |         #else
14915 |         {
14916 |             (void)order;
14917 |             return (ma_uint16)_InterlockedExchange16((volatile short*)dst, (short)src);
14918 |         }
14919 |         #endif
14920 |     }
14921 |     #else
14922 |     {
14923 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(16, dst, src, order);
14924 |     }
14925 |     #endif
14926 | }
14927 | static MA_INLINE ma_uint32 __stdcall ma_atomic_exchange_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
14928 | {
14929 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
14930 |     {
14931 |         #if defined(MA_ARM)
14932 |         {
14933 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchange, ma_uint32,
> long);
14934 |         }
14935 |         #else
14936 |         {
14937 |             (void)order;
14938 |             return (ma_uint32)_InterlockedExchange((volatile long*)dst, (long)src);
14939 |         }
14940 |         #endif
14941 |     }
14942 |     #else
14943 |     {
14944 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(32, dst, src, order);
14945 |     }
14946 |     #endif
14947 | }
14948 | static MA_INLINE ma_uint64 __stdcall ma_atomic_exchange_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
14949 | {
14950 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
14951 |     {
14952 |         #if defined(MA_32BIT)

```

```

14953 |         {
14954 |             MA_ATOMIC_EXCHANGE_EXPLICIT_CAS(64, dst, src, order);
14955 |         }
14956 |     #else
14957 |     {
14958 |         #if defined(MA_ARM)
14959 |         {
14960 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchange64, ma_uint64,
> long long);
14961 |         }
14962 |     #else
14963 |     {
14964 |         (void)order;
14965 |         return (ma_uint64)_InterlockedExchange64((volatile long long*)dst, (long
> long)src);
14966 |     }
14967 |     #endif
14968 | }
14969 | #endif
14970 | }
14971 | #else
14972 | {
14973 |     MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(64, dst, src, order);
14974 | }
14975 | #endif
14976 | }
14977 | static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_add_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
14978 | {
14979 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
14980 |     {
14981 |         #if defined(MA_ARM)
14982 |         {
14983 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchangeAdd8, ma_uint8,
> char);
14984 |         }
14985 |     #else
14986 |     {
14987 |         (void)order;
14988 |         return (ma_uint8)_InterlockedExchangeAdd8((volatile char*)dst, (char)src);
14989 |     }
14990 |     #endif
14991 | }
14992 | #else
14993 | {
14994 |     MA_ATOMIC_FETCH_ADD_LOCK(8, dst, src, order);
14995 | }
14996 | #endif
14997 | }
14998 | static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_add_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
14999 | {
15000 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15001 |     {
15002 |         #if defined(MA_ARM)
15003 |         {
15004 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchangeAdd16, ma_uint16,
> short);
15005 |         }
15006 |     #else
15007 |     {
15008 |         (void)order;
15009 |         return (ma_uint16)_InterlockedExchangeAdd16((volatile short*)dst, (short)src);
15010 |     }
15011 |     #endif
15012 | }
15013 | #else
15014 | {
15015 |     MA_ATOMIC_FETCH_ADD_LOCK(16, dst, src, order);
15016 | }
15017 | #endif
15018 | }
15019 | static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_add_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15020 | {
15021 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15022 |     {
15023 |         #if defined(MA_ARM)
15024 |         {

```



```

15025 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchangeAdd, ma_uint32,
> long);
15026 |         }
15027 |     #else
15028 |     {
15029 |         (void)order;
15030 |         return (ma_uint32)_InterlockedExchangeAdd((volatile long*)dst, (long)src);
15031 |     }
15032 |     #endif
15033 | }
15034 | #else
15035 | {
15036 |     MA_ATOMIC_FETCH_ADD_LOCK(32, dst, src, order);
15037 | }
15038 | #endif
15039 | }
15040 | static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_add_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15041 | {
15042 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15043 |     {
15044 |         #if defined(MA_32BIT)
15045 |         {
15046 |             MA_ATOMIC_FETCH_ADD_CAS(64, dst, src, order);
15047 |         }
15048 |         #else
15049 |         {
15050 |             #if defined(MA_ARM)
15051 |             {
15052 |                 MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedExchangeAdd64,
> ma_uint64, long long);
15053 |             }
15054 |             #else
15055 |             {
15056 |                 (void)order;
15057 |                 return (ma_uint64)_InterlockedExchangeAdd64((volatile long long*)dst, (long
> long)src);
15058 |             }
15059 |             #endif
15060 |         }
15061 |         #endif
15062 |     }
15063 |     #else
15064 |     {
15065 |         MA_ATOMIC_FETCH_ADD_LOCK(64, dst, src, order);
15066 |     }
15067 |     #endif
15068 | }
15069 | static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_sub_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15070 | {
15071 |     return ma_atomic_fetch_add_explicit_8(dst, (ma_uint8)(-(ma_int8)src), order);
15072 | }
15073 | static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_sub_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15074 | {
15075 |     return ma_atomic_fetch_add_explicit_16(dst, (ma_uint16)(-(ma_int16)src), order);
15076 | }
15077 | static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_sub_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15078 | {
15079 |     return ma_atomic_fetch_add_explicit_32(dst, (ma_uint32)(-(ma_int32)src), order);
15080 | }
15081 | static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_sub_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15082 | {
15083 |     return ma_atomic_fetch_add_explicit_64(dst, (ma_uint64)(-(ma_int64)src), order);
15084 | }
15085 | static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_and_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15086 | {
15087 |     #if defined(MA_ARM)
15088 |     {
15089 |         MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedAnd8, ma_uint8, char);
15090 |     }
15091 |     #else
15092 |     {
15093 |         MA_ATOMIC_FETCH_AND_CAS(8, dst, src, order);
15094 |     }

```

```
15095 |         #endif
15096 |     }
15097 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_and_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15098 |     {
15099 |         #if defined(MA_ARM)
15100 |         {
15101 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedAnd16, ma_uint16, short);
15102 |         }
15103 |         #else
15104 |         {
15105 |             MA_ATOMIC_FETCH_AND_CAS(16, dst, src, order);
15106 |         }
15107 |         #endif
15108 |     }
15109 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_and_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15110 |     {
15111 |         #if defined(MA_ARM)
15112 |         {
15113 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedAnd, ma_uint32, long);
15114 |         }
15115 |         #else
15116 |         {
15117 |             MA_ATOMIC_FETCH_AND_CAS(32, dst, src, order);
15118 |         }
15119 |         #endif
15120 |     }
15121 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_and_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15122 |     {
15123 |         #if defined(MA_ARM)
15124 |         {
15125 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedAnd64, ma_uint64, long long);
15126 |         }
15127 |         #else
15128 |         {
15129 |             MA_ATOMIC_FETCH_AND_CAS(64, dst, src, order);
15130 |         }
15131 |         #endif
15132 |     }
15133 |     static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_or_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15134 |     {
15135 |         #if defined(MA_ARM)
15136 |         {
15137 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedOr8, ma_uint8, char);
15138 |         }
15139 |         #else
15140 |         {
15141 |             MA_ATOMIC_FETCH_OR_CAS(8, dst, src, order);
15142 |         }
15143 |         #endif
15144 |     }
15145 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_or_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15146 |     {
15147 |         #if defined(MA_ARM)
15148 |         {
15149 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedOr16, ma_uint16, short);
15150 |         }
15151 |         #else
15152 |         {
15153 |             MA_ATOMIC_FETCH_OR_CAS(16, dst, src, order);
15154 |         }
15155 |         #endif
15156 |     }
15157 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_or_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15158 |     {
15159 |         #if defined(MA_ARM)
15160 |         {
15161 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedOr, ma_uint32, long);
15162 |         }
15163 |         #else
15164 |         {
15165 |             MA_ATOMIC_FETCH_OR_CAS(32, dst, src, order);
15166 |         }
15167 |         #endif
```

```

15168 |     }
15169 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_or_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15170 |     {
15171 |         #if defined(MA_ARM)
15172 |         {
15173 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedOr64, ma_uint64, long long);
15174 |         }
15175 |         #else
15176 |         {
15177 |             MA_ATOMIC_FETCH_OR_CAS(64, dst, src, order);
15178 |         }
15179 |         #endif
15180 |     }
15181 |     static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_xor_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15182 |     {
15183 |         #if defined(MA_ARM)
15184 |         {
15185 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedXor8, ma_uint8, char);
15186 |         }
15187 |         #else
15188 |         {
15189 |             MA_ATOMIC_FETCH_XOR_CAS(8, dst, src, order);
15190 |         }
15191 |         #endif
15192 |     }
15193 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_xor_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15194 |     {
15195 |         #if defined(MA_ARM)
15196 |         {
15197 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedXor16, ma_uint16, short);
15198 |         }
15199 |         #else
15200 |         {
15201 |             MA_ATOMIC_FETCH_XOR_CAS(16, dst, src, order);
15202 |         }
15203 |         #endif
15204 |     }
15205 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_xor_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15206 |     {
15207 |         #if defined(MA_ARM)
15208 |         {
15209 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedXor, ma_uint32, long);
15210 |         }
15211 |         #else
15212 |         {
15213 |             MA_ATOMIC_FETCH_XOR_CAS(32, dst, src, order);
15214 |         }
15215 |         #endif
15216 |     }
15217 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_xor_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15218 |     {
15219 |         #if defined(MA_ARM)
15220 |         {
15221 |             MA_ATOMIC_MSVC_ARM_INTRINSIC(dst, src, order, _InterlockedXor64, ma_uint64, long long);
15222 |         }
15223 |         #else
15224 |         {
15225 |             MA_ATOMIC_FETCH_XOR_CAS(64, dst, src, order);
15226 |         }
15227 |         #endif
15228 |     }
15229 |     #define ma_atomic_store_explicit_8( dst, src, order) (void)ma_atomic_exchange_explicit_8( dst,
> src, order)
15230 |     #define ma_atomic_store_explicit_16(dst, src, order) (void)ma_atomic_exchange_explicit_16(dst,
> src, order)
15231 |     #define ma_atomic_store_explicit_32(dst, src, order) (void)ma_atomic_exchange_explicit_32(dst,
> src, order)
15232 |     #define ma_atomic_store_explicit_64(dst, src, order) (void)ma_atomic_exchange_explicit_64(dst,
> src, order)
15233 |     #if defined(MA_X64)
15234 |     #define ma_atomic_thread_fence(order)    __faststorefence(), (void)order
15235 |     #elif defined(MA_ARM64)
15236 |     #define ma_atomic_thread_fence(order)    __dmb(_ARM64_BARRIER_ISH), (void)order
15237 |     #else

```

```

15238 |         static MA_INLINE void ma_atomic_thread_fence(ma_atomic_memory_order order)
15239 |         {
15240 |             volatile ma_uint32 barrier = 0;
15241 |             ma_atomic_fetch_add_explicit_32(&barrier, 0, order);
15242 |         }
15243 |     #endif
15244 |     #define ma_atomic_signal_fence(order) _ReadWriteBarrier(), (void)order
15245 | #endif
15246 | #if defined(MA_ATOMIC_LEGACY_MSVC_ASM)
15247 |     static MA_INLINE ma_uint8 __stdcall ma_atomic_compare_and_swap_8(volatile ma_uint8* dst,
> ma_uint8 expected, ma_uint8 replacement)
15248 |     {
15249 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15250 |         {
15251 |             ma_uint8 result = 0;
15252 |             __asm {
15253 |                 mov ecx, dst
15254 |                 mov al, expected
15255 |                 mov dl, replacement
15256 |                 lock cmpxchg [ecx], dl
15257 |                 mov result, al
15258 |             }
15259 |             return result;
15260 |         }
15261 |         #else
15262 |         {
15263 |             MA_ATOMIC_COMPARE_AND_SWAP_LOCK(8, dst, expected, replacement);
15264 |         }
15265 |         #endif
15266 |     }
15267 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_compare_and_swap_16(volatile ma_uint16* dst,
> ma_uint16 expected, ma_uint16 replacement)
15268 |     {
15269 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15270 |         {
15271 |             ma_uint16 result = 0;
15272 |             __asm {
15273 |                 mov ecx, dst
15274 |                 mov ax, expected
15275 |                 mov dx, replacement
15276 |                 lock cmpxchg [ecx], dx
15277 |                 mov result, ax
15278 |             }
15279 |             return result;
15280 |         }
15281 |         #else
15282 |         {
15283 |             MA_ATOMIC_COMPARE_AND_SWAP_LOCK(16, dst, expected, replacement);
15284 |         }
15285 |         #endif
15286 |     }
15287 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_compare_and_swap_32(volatile ma_uint32* dst,
> ma_uint32 expected, ma_uint32 replacement)
15288 |     {
15289 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15290 |         {
15291 |             ma_uint32 result = 0;
15292 |             __asm {
15293 |                 mov ecx, dst
15294 |                 mov eax, expected
15295 |                 mov edx, replacement
15296 |                 lock cmpxchg [ecx], edx
15297 |                 mov result, eax
15298 |             }
15299 |             return result;
15300 |         }
15301 |         #else
15302 |         {
15303 |             MA_ATOMIC_COMPARE_AND_SWAP_LOCK(32, dst, expected, replacement);
15304 |         }
15305 |         #endif
15306 |     }
15307 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_compare_and_swap_64(volatile ma_uint64* dst,
> ma_uint64 expected, ma_uint64 replacement)
15308 |     {
15309 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15310 |         {
15311 |             ma_uint32 resultEAX = 0;
15312 |             ma_uint32 resultEDX = 0;

```

```

15313 |         __asm {
15314 |             mov esi, dst
15315 |             mov eax, dword ptr expected
15316 |             mov edx, dword ptr expected + 4
15317 |             mov ebx, dword ptr replacement
15318 |             mov ecx, dword ptr replacement + 4
15319 |             lock cmpxchg8b qword ptr [esi]
15320 |             mov resultEAX, eax
15321 |             mov resultEDX, edx
15322 |         }
15323 |         return ((ma_uint64)resultEDX << 32) | resultEAX;
15324 |     }
15325 | #else
15326 | {
15327 |     MA_ATOMIC_COMPARE_AND_SWAP_LOCK(64, dst, expected, replacement);
15328 | }
15329 | #endif
15330 | }
15331 | static MA_INLINE ma_uint8 ma_atomic_load_explicit_8(volatile const ma_uint8* dst,
> ma_atomic_memory_order order)
15332 | {
15333 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15334 |     {
15335 |         ma_uint8 result = 0;
15336 |         if (order == ma_atomic_memory_order_relaxed) {
15337 |             __asm {
15338 |                 mov esi, dst
15339 |                 mov al, [esi]
15340 |                 mov result, al
15341 |             }
15342 |         } else if (order <= ma_atomic_memory_order_release) {
15343 |             __asm {
15344 |                 mov esi, dst
15345 |                 mov al, [esi]
15346 |                 lock add dword ptr [esp], 0
15347 |                 mov result, al
15348 |             }
15349 |         } else {
15350 |             __asm {
15351 |                 lock add dword ptr [esp], 0
15352 |                 mov esi, dst
15353 |                 mov al, [esi]
15354 |                 mov result, al
15355 |                 lock add dword ptr [esp], 0
15356 |             }
15357 |         }
15358 |         return result;
15359 |     }
15360 | #else
15361 | {
15362 |     MA_ATOMIC_LOAD_EXPLICIT_LOCK(8, dst, order);
15363 | }
15364 | #endif
15365 | }
15366 | static MA_INLINE ma_uint16 ma_atomic_load_explicit_16(volatile const ma_uint16* dst,
> ma_atomic_memory_order order)
15367 | {
15368 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15369 |     {
15370 |         ma_uint16 result = 0;
15371 |         if (order == ma_atomic_memory_order_relaxed) {
15372 |             __asm {
15373 |                 mov esi, dst
15374 |                 mov ax, [esi]
15375 |                 mov result, ax
15376 |             }
15377 |         } else if (order <= ma_atomic_memory_order_release) {
15378 |             __asm {
15379 |                 mov esi, dst
15380 |                 mov ax, [esi]
15381 |                 lock add dword ptr [esp], 0
15382 |                 mov result, ax
15383 |             }
15384 |         } else {
15385 |             __asm {
15386 |                 lock add dword ptr [esp], 0
15387 |                 mov esi, dst
15388 |                 mov ax, [esi]
15389 |                 mov result, ax

```

```

15390 |         lock add dword ptr [esp], 0
15391 |     }
15392 | }
15393 |     return result;
15394 | }
15395 | #else
15396 | {
15397 |     MA_ATOMIC_LOAD_EXPLICIT_LOCK(16, dst, order);
15398 | }
15399 | #endif
15400 | }
15401 | static MA_INLINE ma_uint32 ma_atomic_load_explicit_32(volatile const ma_uint32* dst,
> ma_atomic_memory_order order)
15402 | {
15403 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15404 |     {
15405 |         ma_uint32 result = 0;
15406 |         if (order == ma_atomic_memory_order_relaxed) {
15407 |             __asm {
15408 |                 mov esi, dst
15409 |                 mov eax, [esi]
15410 |                 mov result, eax
15411 |             }
15412 |         } else if (order <= ma_atomic_memory_order_release) {
15413 |             __asm {
15414 |                 mov esi, dst
15415 |                 mov eax, [esi]
15416 |                 lock add dword ptr [esp], 0
15417 |                 mov result, eax
15418 |             }
15419 |         } else {
15420 |             __asm {
15421 |                 lock add dword ptr [esp], 0
15422 |                 mov esi, dst
15423 |                 mov eax, [esi]
15424 |                 mov result, eax
15425 |                 lock add dword ptr [esp], 0
15426 |             }
15427 |         }
15428 |         return result;
15429 |     }
15430 | #else
15431 | {
15432 |     MA_ATOMIC_LOAD_EXPLICIT_LOCK(32, dst, order);
15433 | }
15434 | #endif
15435 | }
15436 | static MA_INLINE ma_uint64 ma_atomic_load_explicit_64(volatile const ma_uint64* dst,
> ma_atomic_memory_order order)
15437 | {
15438 |     (void)order;
15439 |     return ma_atomic_compare_and_swap_64((volatile ma_uint64*)dst, 0, 0);
15440 | }
15441 | static MA_INLINE void __stdcall ma_atomic_store_explicit_8(volatile ma_uint8* dst, ma_uint8 src,
> ma_atomic_memory_order order)
15442 | {
15443 |     if (order == ma_atomic_memory_order_relaxed) {
15444 |         __asm {
15445 |             mov esi, dst
15446 |             mov al, src
15447 |             mov [esi], al
15448 |         }
15449 |     } else {
15450 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15451 |         {
15452 |             __asm {
15453 |                 mov esi, dst
15454 |                 mov al, src
15455 |                 xchg [esi], al
15456 |             }
15457 |         }
15458 |         #else
15459 |         {
15460 |             MA_ATOMIC_STORE_EXPLICIT_LOCK(8, dst, src, order);
15461 |         }
15462 |         #endif
15463 |     }
15464 | }
15465 | static MA_INLINE void __stdcall ma_atomic_store_explicit_16(volatile ma_uint16* dst, ma_uint16

```

```

> src, ma_atomic_memory_order order)
15466 | {
15467 |     if (order == ma_atomic_memory_order_relaxed) {
15468 |         __asm {
15469 |             mov esi, dst
15470 |             mov ax, src
15471 |             mov [esi], ax
15472 |         }
15473 |     } else {
15474 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15475 |         {
15476 |             __asm {
15477 |                 mov esi, dst
15478 |                 mov ax, src
15479 |                 xchg [esi], ax
15480 |             }
15481 |         }
15482 |         #else
15483 |         {
15484 |             MA_ATOMIC_STORE_EXPLICIT_LOCK(16, dst, src, order);
15485 |         }
15486 |         #endif
15487 |     }
15488 | }
15489 | static MA_INLINE void __stdcall ma_atomic_store_explicit_32(volatile ma_uint32* dst, ma_uint32
> src, ma_atomic_memory_order order)
15490 | {
15491 |     if (order == ma_atomic_memory_order_relaxed) {
15492 |         __asm {
15493 |             mov esi, dst
15494 |             mov eax, src
15495 |             mov [esi], eax
15496 |         }
15497 |     } else {
15498 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15499 |         {
15500 |             __asm {
15501 |                 mov esi, dst
15502 |                 mov eax, src
15503 |                 xchg [esi], eax
15504 |             }
15505 |         }
15506 |         #else
15507 |         {
15508 |             MA_ATOMIC_STORE_EXPLICIT_LOCK(32, dst, src, order);
15509 |         }
15510 |         #endif
15511 |     }
15512 | }
15513 | static MA_INLINE void __stdcall ma_atomic_store_explicit_64(volatile ma_uint64* dst, ma_uint64
> src, ma_atomic_memory_order order)
15514 | {
15515 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15516 |     {
15517 |         MA_ATOMIC_STORE_EXPLICIT_CAS(64, dst, src, order);
15518 |     }
15519 |     #else
15520 |     {
15521 |         MA_ATOMIC_STORE_EXPLICIT_LOCK(64, dst, src, order);
15522 |     }
15523 |     #endif
15524 | }
15525 | static MA_INLINE ma_uint8 __stdcall ma_atomic_exchange_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15526 | {
15527 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15528 |     {
15529 |         ma_uint8 result = 0;
15530 |         (void)order;
15531 |         __asm {
15532 |             mov ecx, dst
15533 |             mov al, src
15534 |             lock xchg [ecx], al
15535 |             mov result, al
15536 |         }
15537 |         return result;
15538 |     }
15539 |     #else
15540 |     {

```



```

15541 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(8, dst, src, order);
15542 |     }
15543 |     #endif
15544 | }
15545 | static MA_INLINE ma_uint16 __stdcall ma_atomic_exchange_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15546 | {
15547 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15548 |     {
15549 |         ma_uint16 result = 0;
15550 |         (void)order;
15551 |         __asm {
15552 |             mov ecx, dst
15553 |             mov ax, src
15554 |             lock xchg [ecx], ax
15555 |             mov result, ax
15556 |         }
15557 |         return result;
15558 |     }
15559 |     #else
15560 |     {
15561 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(16, dst, src, order);
15562 |     }
15563 |     #endif
15564 | }
15565 | static MA_INLINE ma_uint32 __stdcall ma_atomic_exchange_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15566 | {
15567 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15568 |     {
15569 |         ma_uint32 result = 0;
15570 |         (void)order;
15571 |         __asm {
15572 |             mov ecx, dst
15573 |             mov eax, src
15574 |             xchg [ecx], eax
15575 |             mov result, eax
15576 |         }
15577 |         return result;
15578 |     }
15579 |     #else
15580 |     {
15581 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(32, dst, src, order);
15582 |     }
15583 |     #endif
15584 | }
15585 | static MA_INLINE ma_uint64 __stdcall ma_atomic_exchange_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15586 | {
15587 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15588 |     {
15589 |         MA_ATOMIC_EXCHANGE_EXPLICIT_CAS(64, dst, src, order);
15590 |     }
15591 |     #else
15592 |     {
15593 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(64, dst, src, order);
15594 |     }
15595 |     #endif
15596 | }
15597 | static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_add_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15598 | {
15599 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15600 |     {
15601 |         ma_uint8 result = 0;
15602 |         (void)order;
15603 |         __asm {
15604 |             mov ecx, dst
15605 |             mov al, src
15606 |             lock xadd [ecx], al
15607 |             mov result, al
15608 |         }
15609 |         return result;
15610 |     }
15611 |     #else
15612 |     {
15613 |         MA_ATOMIC_FETCH_ADD_LOCK(8, dst, src, order);
15614 |     }
15615 |     #endif

```

```
15616 |     }
15617 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_add_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15618 |     {
15619 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15620 |         {
15621 |             ma_uint16 result = 0;
15622 |             (void)order;
15623 |             __asm {
15624 |                 mov ecx, dst
15625 |                 mov ax, src
15626 |                 lock xadd [ecx], ax
15627 |                 mov result, ax
15628 |             }
15629 |             return result;
15630 |         }
15631 |         #else
15632 |         {
15633 |             MA_ATOMIC_FETCH_ADD_LOCK(16, dst, src, order);
15634 |         }
15635 |         #endif
15636 |     }
15637 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_add_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15638 |     {
15639 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15640 |         {
15641 |             ma_uint32 result = 0;
15642 |             (void)order;
15643 |             __asm {
15644 |                 mov ecx, dst
15645 |                 mov eax, src
15646 |                 lock xadd [ecx], eax
15647 |                 mov result, eax
15648 |             }
15649 |             return result;
15650 |         }
15651 |         #else
15652 |         {
15653 |             MA_ATOMIC_FETCH_ADD_LOCK(32, dst, src, order);
15654 |         }
15655 |         #endif
15656 |     }
15657 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_add_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15658 |     {
15659 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15660 |         {
15661 |             MA_ATOMIC_FETCH_ADD_CAS(64, dst, src, order);
15662 |         }
15663 |         #else
15664 |         {
15665 |             MA_ATOMIC_FETCH_ADD_LOCK(64, dst, src, order);
15666 |         }
15667 |         #endif
15668 |     }
15669 |     static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_sub_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15670 |     {
15671 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15672 |         {
15673 |             ma_uint8 result = 0;
15674 |             (void)order;
15675 |             __asm {
15676 |                 mov ecx, dst
15677 |                 mov al, src
15678 |                 neg al
15679 |                 lock xadd [ecx], al
15680 |                 mov result, al
15681 |             }
15682 |             return result;
15683 |         }
15684 |         #else
15685 |         {
15686 |             MA_ATOMIC_FETCH_ADD_LOCK(8, dst, (ma_uint8)(-(ma_int8)src), order);
15687 |         }
15688 |         #endif
15689 |     }
15690 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_sub_explicit_16(volatile ma_uint16* dst,
```

```

> ma_uint16 src, ma_atomic_memory_order order)
15691 | {
15692 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15693 |     {
15694 |         ma_uint16 result = 0;
15695 |         (void)order;
15696 |         __asm {
15697 |             mov ecx, dst
15698 |             mov ax, src
15699 |             neg ax
15700 |             lock xadd [ecx], ax
15701 |             mov result, ax
15702 |         }
15703 |         return result;
15704 |     }
15705 |     #else
15706 |     {
15707 |         MA_ATOMIC_FETCH_ADD_LOCK(16, dst, (ma_uint16)(-(ma_int16)src), order);
15708 |     }
15709 |     #endif
15710 | }
15711 | static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_sub_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15712 | {
15713 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15714 |     {
15715 |         ma_uint32 result = 0;
15716 |         (void)order;
15717 |         __asm {
15718 |             mov ecx, dst
15719 |             mov eax, src
15720 |             neg eax
15721 |             lock xadd [ecx], eax
15722 |             mov result, eax
15723 |         }
15724 |         return result;
15725 |     }
15726 |     #else
15727 |     {
15728 |         MA_ATOMIC_FETCH_ADD_LOCK(32, dst, (ma_uint32)(-(ma_int32)src), order);
15729 |     }
15730 |     #endif
15731 | }
15732 | static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_sub_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15733 | {
15734 |     MA_ATOMIC_FETCH_ADD_CAS(64, dst, (ma_uint64)(-(ma_int64)src), order);
15735 | }
15736 | static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_and_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15737 | {
15738 |     MA_ATOMIC_FETCH_AND_CAS(8, dst, src, order);
15739 | }
15740 | static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_and_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15741 | {
15742 |     MA_ATOMIC_FETCH_AND_CAS(16, dst, src, order);
15743 | }
15744 | static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_and_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15745 | {
15746 |     MA_ATOMIC_FETCH_AND_CAS(32, dst, src, order);
15747 | }
15748 | static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_and_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15749 | {
15750 |     MA_ATOMIC_FETCH_AND_CAS(64, dst, src, order);
15751 | }
15752 | static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_or_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15753 | {
15754 |     MA_ATOMIC_FETCH_OR_CAS(8, dst, src, order);
15755 | }
15756 | static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_or_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15757 | {
15758 |     MA_ATOMIC_FETCH_OR_CAS(16, dst, src, order);
15759 | }
15760 | static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_or_explicit_32(volatile ma_uint32* dst,

```

```

> ma_uint32 src, ma_atomic_memory_order order)
15761 |     {
15762 |         MA_ATOMIC_FETCH_OR_CAS(32, dst, src, order);
15763 |     }
15764 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_or_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15765 |     {
15766 |         MA_ATOMIC_FETCH_OR_CAS(64, dst, src, order);
15767 |     }
15768 |     static MA_INLINE ma_uint8 __stdcall ma_atomic_fetch_xor_explicit_8(volatile ma_uint8* dst,
> ma_uint8 src, ma_atomic_memory_order order)
15769 |     {
15770 |         MA_ATOMIC_FETCH_XOR_CAS(8, dst, src, order);
15771 |     }
15772 |     static MA_INLINE ma_uint16 __stdcall ma_atomic_fetch_xor_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
15773 |     {
15774 |         MA_ATOMIC_FETCH_XOR_CAS(16, dst, src, order);
15775 |     }
15776 |     static MA_INLINE ma_uint32 __stdcall ma_atomic_fetch_xor_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
15777 |     {
15778 |         MA_ATOMIC_FETCH_XOR_CAS(32, dst, src, order);
15779 |     }
15780 |     static MA_INLINE ma_uint64 __stdcall ma_atomic_fetch_xor_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
15781 |     {
15782 |         MA_ATOMIC_FETCH_XOR_CAS(64, dst, src, order);
15783 |     }
15784 |     static MA_INLINE void __stdcall ma_atomic_thread_fence(ma_atomic_memory_order order)
15785 |     {
15786 |         (void)order;
15787 |         __asm {
15788 |             lock add dword ptr [esp], 0
15789 |         }
15790 |     }
15791 |     #define ma_atomic_signal_fence(order) __asm {}; (void)order
15792 | #endif
15793 | #if defined(MA_ATOMIC_MODERN_GCC)
15794 |     #define MA_ATOMIC_HAS_NATIVE_COMPARE_EXCHANGE
15795 |     #define ma_atomic_thread_fence(order)                __atomic_thread_fence(order)
15796 |     #define ma_atomic_signal_fence(order)                __atomic_signal_fence(order)
15797 |     #define ma_atomic_is_lock_free_8(ptr)                __atomic_is_lock_free(1, ptr)
15798 |     #define ma_atomic_is_lock_free_16(ptr)               __atomic_is_lock_free(2, ptr)
15799 |     #define ma_atomic_is_lock_free_32(ptr)               __atomic_is_lock_free(4, ptr)
15800 |     #define ma_atomic_is_lock_free_64(ptr)               __atomic_is_lock_free(8, ptr)
15801 |     #define ma_atomic_store_explicit_8( dst, src, order) __atomic_store_n(dst, src,
> order)
15802 |     #define ma_atomic_store_explicit_16(dst, src, order) __atomic_store_n(dst, src,
> order)
15803 |     #define ma_atomic_store_explicit_32(dst, src, order) __atomic_store_n(dst, src,
> order)
15804 |     #define ma_atomic_store_explicit_64(dst, src, order) __atomic_store_n(dst, src,
> order)
15805 |     #define ma_atomic_load_explicit_8( dst, order)       __atomic_load_n(dst, order)
15806 |     #define ma_atomic_load_explicit_16(dst, order)       __atomic_load_n(dst, order)
15807 |     #define ma_atomic_load_explicit_32(dst, order)       __atomic_load_n(dst, order)
15808 |     #define ma_atomic_load_explicit_64(dst, order)       __atomic_load_n(dst, order)
15809 |     #define ma_atomic_exchange_explicit_8( dst, src, order) __atomic_exchange_n(dst, src,
> order)
15810 |     #define ma_atomic_exchange_explicit_16(dst, src, order) __atomic_exchange_n(dst, src,
> order)
15811 |     #define ma_atomic_exchange_explicit_32(dst, src, order) __atomic_exchange_n(dst, src,
> order)
15812 |     #define ma_atomic_exchange_explicit_64(dst, src, order) __atomic_exchange_n(dst, src,
> order)
15813 |     #define ma_atomic_compare_exchange_strong_explicit_8( dst, expected, replacement, successOrder,
> failureOrder) __atomic_compare_exchange_n(dst, expected, replacement, 0, successOrder,
> failureOrder)
15814 |     #define ma_atomic_compare_exchange_strong_explicit_16(dst, expected, replacement, successOrder,
> failureOrder) __atomic_compare_exchange_n(dst, expected, replacement, 0, successOrder,
> failureOrder)
15815 |     #define ma_atomic_compare_exchange_strong_explicit_32(dst, expected, replacement, successOrder,
> failureOrder) __atomic_compare_exchange_n(dst, expected, replacement, 0, successOrder,
> failureOrder)
15816 |     #define ma_atomic_compare_exchange_strong_explicit_64(dst, expected, replacement, successOrder,
> failureOrder) __atomic_compare_exchange_n(dst, expected, replacement, 0, successOrder,
> failureOrder)
15817 |     #define ma_atomic_compare_exchange_weak_explicit_8( dst, expected, replacement, successOrder,

```

```

> failureOrder)    __atomic_compare_exchange_n(dst, expected, replacement, 1, successOrder,
> failureOrder)
15818 | #define ma_atomic_compare_exchange_weak_explicit_16(dst, expected, replacement, successOrder,
> failureOrder)    __atomic_compare_exchange_n(dst, expected, replacement, 1, successOrder,
> failureOrder)
15819 | #define ma_atomic_compare_exchange_weak_explicit_32(dst, expected, replacement, successOrder,
> failureOrder)    __atomic_compare_exchange_n(dst, expected, replacement, 1, successOrder,
> failureOrder)
15820 | #define ma_atomic_compare_exchange_weak_explicit_64(dst, expected, replacement, successOrder,
> failureOrder)    __atomic_compare_exchange_n(dst, expected, replacement, 1, successOrder,
> failureOrder)
15821 | #define ma_atomic_fetch_add_explicit_8( dst, src, order)    __atomic_fetch_add(dst, src,
> order)
15822 | #define ma_atomic_fetch_add_explicit_16(dst, src, order)    __atomic_fetch_add(dst, src,
> order)
15823 | #define ma_atomic_fetch_add_explicit_32(dst, src, order)    __atomic_fetch_add(dst, src,
> order)
15824 | #define ma_atomic_fetch_add_explicit_64(dst, src, order)    __atomic_fetch_add(dst, src,
> order)
15825 | #define ma_atomic_fetch_sub_explicit_8( dst, src, order)    __atomic_fetch_sub(dst, src,
> order)
15826 | #define ma_atomic_fetch_sub_explicit_16(dst, src, order)    __atomic_fetch_sub(dst, src,
> order)
15827 | #define ma_atomic_fetch_sub_explicit_32(dst, src, order)    __atomic_fetch_sub(dst, src,
> order)
15828 | #define ma_atomic_fetch_sub_explicit_64(dst, src, order)    __atomic_fetch_sub(dst, src,
> order)
15829 | #define ma_atomic_fetch_or_explicit_8( dst, src, order)    __atomic_fetch_or(dst, src,
> order)
15830 | #define ma_atomic_fetch_or_explicit_16(dst, src, order)    __atomic_fetch_or(dst, src,
> order)
15831 | #define ma_atomic_fetch_or_explicit_32(dst, src, order)    __atomic_fetch_or(dst, src,
> order)
15832 | #define ma_atomic_fetch_or_explicit_64(dst, src, order)    __atomic_fetch_or(dst, src,
> order)
15833 | #define ma_atomic_fetch_xor_explicit_8( dst, src, order)    __atomic_fetch_xor(dst, src,
> order)
15834 | #define ma_atomic_fetch_xor_explicit_16(dst, src, order)    __atomic_fetch_xor(dst, src,
> order)
15835 | #define ma_atomic_fetch_xor_explicit_32(dst, src, order)    __atomic_fetch_xor(dst, src,
> order)
15836 | #define ma_atomic_fetch_xor_explicit_64(dst, src, order)    __atomic_fetch_xor(dst, src,
> order)
15837 | #define ma_atomic_fetch_and_explicit_8( dst, src, order)    __atomic_fetch_and(dst, src,
> order)
15838 | #define ma_atomic_fetch_and_explicit_16(dst, src, order)    __atomic_fetch_and(dst, src,
> order)
15839 | #define ma_atomic_fetch_and_explicit_32(dst, src, order)    __atomic_fetch_and(dst, src,
> order)
15840 | #define ma_atomic_fetch_and_explicit_64(dst, src, order)    __atomic_fetch_and(dst, src,
> order)
15841 | static MA_INLINE ma_uint8 ma_atomic_compare_and_swap_8(volatile ma_uint8* dst, ma_uint8
> expected, ma_uint8 replacement)
15842 | {
15843 |     __atomic_compare_exchange_n(dst, &expected, replacement, 0, __ATOMIC_SEQ_CST,
> __ATOMIC_SEQ_CST);
15844 |     return expected;
15845 | }
15846 | static MA_INLINE ma_uint16 ma_atomic_compare_and_swap_16(volatile ma_uint16* dst, ma_uint16
> expected, ma_uint16 replacement)
15847 | {
15848 |     __atomic_compare_exchange_n(dst, &expected, replacement, 0, __ATOMIC_SEQ_CST,
> __ATOMIC_SEQ_CST);
15849 |     return expected;
15850 | }
15851 | static MA_INLINE ma_uint32 ma_atomic_compare_and_swap_32(volatile ma_uint32* dst, ma_uint32
> expected, ma_uint32 replacement)
15852 | {
15853 |     __atomic_compare_exchange_n(dst, &expected, replacement, 0, __ATOMIC_SEQ_CST,
> __ATOMIC_SEQ_CST);
15854 |     return expected;
15855 | }
15856 | #if defined(__clang__)
15857 |     #pragma clang diagnostic push
15858 |     #if __clang_major__ >= 8
15859 |         #pragma clang diagnostic ignored "-Watomic-alignment"
15860 |     #endif
15861 | #endif
15862 | static MA_INLINE ma_uint64 ma_atomic_compare_and_swap_64(volatile ma_uint64* dst, ma_uint64

```

```

> expected, ma_uint64 replacement)
15863 |     {
15864 |         __atomic_compare_exchange_n(dst, &expected, replacement, 0, __ATOMIC_SEQ_CST,
> __ATOMIC_SEQ_CST);
15865 |         return expected;
15866 |     }
15867 |     #if defined(__clang__)
15868 |         #pragma clang diagnostic pop
15869 |     #endif
15870 | #endif
15871 | #if defined(MA_ATOMIC_LEGACY_GCC) || defined(MA_ATOMIC_LEGACY_GCC_ASM)
15872 |     #define ma_atomic_signal_fence(order)    __asm__ __volatile__(":::"memory")
15873 |     #if defined(MA_ATOMIC_LEGACY_GCC)
15874 |         #define ma_atomic_thread_fence(order) __sync_synchronize(), (void)order
15875 |         static MA_INLINE ma_uint8 ma_atomic_compare_and_swap_8(volatile ma_uint8* dst, ma_uint8
> expected, ma_uint8 replacement)
15876 |         {
15877 |             #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15878 |             {
15879 |                 return __sync_val_compare_and_swap(dst, expected, replacement);
15880 |             }
15881 |             #else
15882 |             {
15883 |                 MA_ATOMIC_COMPARE_AND_SWAP_LOCK(8, dst, expected, replacement);
15884 |             }
15885 |             #endif
15886 |         }
15887 |         static MA_INLINE ma_uint16 ma_atomic_compare_and_swap_16(volatile ma_uint16* dst, ma_uint16
> expected, ma_uint16 replacement)
15888 |         {
15889 |             #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15890 |             {
15891 |                 return __sync_val_compare_and_swap(dst, expected, replacement);
15892 |             }
15893 |             #else
15894 |             {
15895 |                 MA_ATOMIC_COMPARE_AND_SWAP_LOCK(16, dst, expected, replacement);
15896 |             }
15897 |             #endif
15898 |         }
15899 |         static MA_INLINE ma_uint32 ma_atomic_compare_and_swap_32(volatile ma_uint32* dst, ma_uint32
> expected, ma_uint32 replacement)
15900 |         {
15901 |             #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15902 |             {
15903 |                 return __sync_val_compare_and_swap(dst, expected, replacement);
15904 |             }
15905 |             #else
15906 |             {
15907 |                 MA_ATOMIC_COMPARE_AND_SWAP_LOCK(32, dst, expected, replacement);
15908 |             }
15909 |             #endif
15910 |         }
15911 |         static MA_INLINE ma_uint64 ma_atomic_compare_and_swap_64(volatile ma_uint64* dst, ma_uint64
> expected, ma_uint64 replacement)
15912 |         {
15913 |             #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15914 |             {
15915 |                 return __sync_val_compare_and_swap(dst, expected, replacement);
15916 |             }
15917 |             #else
15918 |             {
15919 |                 MA_ATOMIC_COMPARE_AND_SWAP_LOCK(64, dst, expected, replacement);
15920 |             }
15921 |             #endif
15922 |         }
15923 |         static MA_INLINE ma_uint8 ma_atomic_load_explicit_8(volatile const ma_uint8* ptr,
> ma_atomic_memory_order order)
15924 |         {
15925 |             #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15926 |             {
15927 |                 (void)order;
15928 |                 return ma_atomic_compare_and_swap_8((ma_uint8*)ptr, 0, 0);
15929 |             }
15930 |             #else
15931 |             {
15932 |                 MA_ATOMIC_LOAD_EXPLICIT_LOCK(8, ptr, order);
15933 |             }
15934 |             #endif

```

```

15935 |     }
15936 |     static MA_INLINE ma_uint16 ma_atomic_load_explicit_16(volatile const ma_uint16* ptr,
> ma_atomic_memory_order order)
15937 |     {
15938 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15939 |         {
15940 |             (void)order;
15941 |             return ma_atomic_compare_and_swap_16((ma_uint16*)ptr, 0, 0);
15942 |         }
15943 |         #else
15944 |         {
15945 |             MA_ATOMIC_LOAD_EXPLICIT_LOCK(16, ptr, order);
15946 |         }
15947 |         #endif
15948 |     }
15949 |     static MA_INLINE ma_uint32 ma_atomic_load_explicit_32(volatile const ma_uint32* ptr,
> ma_atomic_memory_order order)
15950 |     {
15951 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
15952 |         {
15953 |             (void)order;
15954 |             return ma_atomic_compare_and_swap_32((ma_uint32*)ptr, 0, 0);
15955 |         }
15956 |         #else
15957 |         {
15958 |             MA_ATOMIC_LOAD_EXPLICIT_LOCK(32, ptr, order);
15959 |         }
15960 |         #endif
15961 |     }
15962 |     static MA_INLINE ma_uint64 ma_atomic_load_explicit_64(volatile const ma_uint64* ptr,
> ma_atomic_memory_order order)
15963 |     {
15964 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
15965 |         {
15966 |             (void)order;
15967 |             return ma_atomic_compare_and_swap_64((ma_uint64*)ptr, 0, 0);
15968 |         }
15969 |         #else
15970 |         {
15971 |             MA_ATOMIC_LOAD_EXPLICIT_LOCK(64, ptr, order);
15972 |         }
15973 |         #endif
15974 |     }
15975 |     static MA_INLINE ma_uint8 ma_atomic_exchange_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
15976 |     {
15977 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
15978 |         {
15979 |             if (order > ma_atomic_memory_order_acquire) {
15980 |                 __sync_synchronize();
15981 |             }
15982 |             return __sync_lock_test_and_set(dst, src);
15983 |         }
15984 |         #else
15985 |         {
15986 |             MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(8, dst, src, order);
15987 |         }
15988 |         #endif
15989 |     }
15990 |     static MA_INLINE ma_uint16 ma_atomic_exchange_explicit_16(volatile ma_uint16* dst, ma_uint16
> src, ma_atomic_memory_order order)
15991 |     {
15992 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
15993 |         {
15994 |             if (order > ma_atomic_memory_order_acquire) {
15995 |                 __sync_synchronize();
15996 |             }
15997 |             return __sync_lock_test_and_set(dst, src);
15998 |         }
15999 |         #else
16000 |         {
16001 |             MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(16, dst, src, order);
16002 |         }
16003 |         #endif
16004 |     }
16005 |     static MA_INLINE ma_uint32 ma_atomic_exchange_explicit_32(volatile ma_uint32* dst, ma_uint32
> src, ma_atomic_memory_order order)
16006 |     {
16007 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)

```



```

16008 |         {
16009 |             if (order > ma_atomic_memory_order_acquire) {
16010 |                 __sync_synchronize();
16011 |             }
16012 |             return __sync_lock_test_and_set(dst, src);
16013 |         }
16014 |         #else
16015 |         {
16016 |             MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(32, dst, src, order);
16017 |         }
16018 |         #endif
16019 |     }
16020 |     static MA_INLINE ma_uint64 ma_atomic_exchange_explicit_64(volatile ma_uint64* dst, ma_uint64
> src, ma_atomic_memory_order order)
16021 |     {
16022 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
16023 |         {
16024 |             if (order > ma_atomic_memory_order_acquire) {
16025 |                 __sync_synchronize();
16026 |             }
16027 |             return __sync_lock_test_and_set(dst, src);
16028 |         }
16029 |         #else
16030 |         {
16031 |             MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(64, dst, src, order);
16032 |         }
16033 |         #endif
16034 |     }
16035 |     #define ma_atomic_store_explicit_8( dst, src, order) (void)ma_atomic_exchange_explicit_8
> (dst, src, order)
16036 |     #define ma_atomic_store_explicit_16(dst, src, order)
> (void)ma_atomic_exchange_explicit_16(dst, src, order)
16037 |     #define ma_atomic_store_explicit_32(dst, src, order)
> (void)ma_atomic_exchange_explicit_32(dst, src, order)
16038 |     #define ma_atomic_store_explicit_64(dst, src, order)
> (void)ma_atomic_exchange_explicit_64(dst, src, order)
16039 |     static MA_INLINE ma_uint8 ma_atomic_fetch_add_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16040 |     {
16041 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
16042 |         {
16043 |             (void)order;
16044 |             return __sync_fetch_and_add(dst, src);
16045 |         }
16046 |         #else
16047 |         {
16048 |             MA_ATOMIC_FETCH_ADD_LOCK(8, dst, src, order);
16049 |         }
16050 |         #endif
16051 |     }
16052 |     static MA_INLINE ma_uint16 ma_atomic_fetch_add_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16053 |     {
16054 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
16055 |         {
16056 |             (void)order;
16057 |             return __sync_fetch_and_add(dst, src);
16058 |         }
16059 |         #else
16060 |         {
16061 |             MA_ATOMIC_FETCH_ADD_LOCK(16, dst, src, order);
16062 |         }
16063 |         #endif
16064 |     }
16065 |     static MA_INLINE ma_uint32 ma_atomic_fetch_add_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16066 |     {
16067 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
16068 |         {
16069 |             (void)order;
16070 |             return __sync_fetch_and_add(dst, src);
16071 |         }
16072 |         #else
16073 |         {
16074 |             MA_ATOMIC_FETCH_ADD_LOCK(32, dst, src, order);
16075 |         }
16076 |         #endif
16077 |     }
16078 |     static MA_INLINE ma_uint64 ma_atomic_fetch_add_explicit_64(volatile ma_uint64* dst,

```

```

> ma_uint64 src, ma_atomic_memory_order order)
16079 |     {
16080 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
16081 |         {
16082 |             (void)order;
16083 |             return __sync_fetch_and_add(dst, src);
16084 |         }
16085 |         #else
16086 |         {
16087 |             MA_ATOMIC_FETCH_ADD_LOCK(64, dst, src, order);
16088 |         }
16089 |         #endif
16090 |     }
16091 |     static MA_INLINE ma_uint8 ma_atomic_fetch_sub_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16092 |     {
16093 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
16094 |         {
16095 |             (void)order;
16096 |             return __sync_fetch_and_sub(dst, src);
16097 |         }
16098 |         #else
16099 |         {
16100 |             MA_ATOMIC_FETCH_ADD_LOCK(8, dst, (ma_uint8)(-(ma_int8)src), order);
16101 |         }
16102 |         #endif
16103 |     }
16104 |     static MA_INLINE ma_uint16 ma_atomic_fetch_sub_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16105 |     {
16106 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
16107 |         {
16108 |             (void)order;
16109 |             return __sync_fetch_and_sub(dst, src);
16110 |         }
16111 |         #else
16112 |         {
16113 |             MA_ATOMIC_FETCH_ADD_LOCK(16, dst, (ma_uint16)(-(ma_int16)src), order);
16114 |         }
16115 |         #endif
16116 |     }
16117 |     static MA_INLINE ma_uint32 ma_atomic_fetch_sub_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16118 |     {
16119 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
16120 |         {
16121 |             (void)order;
16122 |             return __sync_fetch_and_sub(dst, src);
16123 |         }
16124 |         #else
16125 |         {
16126 |             MA_ATOMIC_FETCH_ADD_LOCK(32, dst, (ma_uint32)(-(ma_int32)src), order);
16127 |         }
16128 |         #endif
16129 |     }
16130 |     static MA_INLINE ma_uint64 ma_atomic_fetch_sub_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16131 |     {
16132 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
16133 |         {
16134 |             (void)order;
16135 |             return __sync_fetch_and_sub(dst, src);
16136 |         }
16137 |         #else
16138 |         {
16139 |             MA_ATOMIC_FETCH_ADD_LOCK(64, dst, (ma_uint64)(-(ma_int64)src), order);
16140 |         }
16141 |         #endif
16142 |     }
16143 |     static MA_INLINE ma_uint8 ma_atomic_fetch_and_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16144 |     {
16145 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
16146 |         {
16147 |             (void)order;
16148 |             return __sync_fetch_and_and(dst, src);
16149 |         }
16150 |         #else
16151 |         {

```

```

16152 |         MA_ATOMIC_FETCH_AND_CAS(8, dst, src, order);
16153 |     }
16154 |     #endif
16155 | }
16156 | static MA_INLINE ma_uint16 ma_atomic_fetch_and_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16157 | {
16158 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
16159 |     {
16160 |         (void)order;
16161 |         return __sync_fetch_and_and(dst, src);
16162 |     }
16163 |     #else
16164 |     {
16165 |         MA_ATOMIC_FETCH_AND_CAS(16, dst, src, order);
16166 |     }
16167 |     #endif
16168 | }
16169 | static MA_INLINE ma_uint32 ma_atomic_fetch_and_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16170 | {
16171 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
16172 |     {
16173 |         (void)order;
16174 |         return __sync_fetch_and_and(dst, src);
16175 |     }
16176 |     #else
16177 |     {
16178 |         MA_ATOMIC_FETCH_AND_CAS(32, dst, src, order);
16179 |     }
16180 |     #endif
16181 | }
16182 | static MA_INLINE ma_uint64 ma_atomic_fetch_and_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16183 | {
16184 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
16185 |     {
16186 |         (void)order;
16187 |         return __sync_fetch_and_and(dst, src);
16188 |     }
16189 |     #else
16190 |     {
16191 |         MA_ATOMIC_FETCH_AND_CAS(64, dst, src, order);
16192 |     }
16193 |     #endif
16194 | }
16195 | static MA_INLINE ma_uint8 ma_atomic_fetch_or_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16196 | {
16197 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
16198 |     {
16199 |         (void)order;
16200 |         return __sync_fetch_and_or(dst, src);
16201 |     }
16202 |     #else
16203 |     {
16204 |         MA_ATOMIC_FETCH_OR_CAS(8, dst, src, order);
16205 |     }
16206 |     #endif
16207 | }
16208 | static MA_INLINE ma_uint16 ma_atomic_fetch_or_explicit_16(volatile ma_uint16* dst, ma_uint16
> src, ma_atomic_memory_order order)
16209 | {
16210 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
16211 |     {
16212 |         (void)order;
16213 |         return __sync_fetch_and_or(dst, src);
16214 |     }
16215 |     #else
16216 |     {
16217 |         MA_ATOMIC_FETCH_OR_CAS(16, dst, src, order);
16218 |     }
16219 |     #endif
16220 | }
16221 | static MA_INLINE ma_uint32 ma_atomic_fetch_or_explicit_32(volatile ma_uint32* dst, ma_uint32
> src, ma_atomic_memory_order order)
16222 | {
16223 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
16224 |     {

```

```
16225 |         (void)order;
16226 |         return __sync_fetch_and_or(dst, src);
16227 |     }
16228 |     #else
16229 |     {
16230 |         MA_ATOMIC_FETCH_OR_CAS(32, dst, src, order);
16231 |     }
16232 |     #endif
16233 | }
16234 | static MA_INLINE ma_uint64 ma_atomic_fetch_or_explicit_64(volatile ma_uint64* dst, ma_uint64
> src, ma_atomic_memory_order order)
16235 | {
16236 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
16237 |     {
16238 |         (void)order;
16239 |         return __sync_fetch_and_or(dst, src);
16240 |     }
16241 |     #else
16242 |     {
16243 |         MA_ATOMIC_FETCH_OR_CAS(64, dst, src, order);
16244 |     }
16245 |     #endif
16246 | }
16247 | static MA_INLINE ma_uint8 ma_atomic_fetch_xor_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16248 | {
16249 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8)
16250 |     {
16251 |         (void)order;
16252 |         return __sync_fetch_and_xor(dst, src);
16253 |     }
16254 |     #else
16255 |     {
16256 |         MA_ATOMIC_FETCH_XOR_CAS(8, dst, src, order);
16257 |     }
16258 |     #endif
16259 | }
16260 | static MA_INLINE ma_uint16 ma_atomic_fetch_xor_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16261 | {
16262 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16)
16263 |     {
16264 |         (void)order;
16265 |         return __sync_fetch_and_xor(dst, src);
16266 |     }
16267 |     #else
16268 |     {
16269 |         MA_ATOMIC_FETCH_XOR_CAS(16, dst, src, order);
16270 |     }
16271 |     #endif
16272 | }
16273 | static MA_INLINE ma_uint32 ma_atomic_fetch_xor_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16274 | {
16275 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32)
16276 |     {
16277 |         (void)order;
16278 |         return __sync_fetch_and_xor(dst, src);
16279 |     }
16280 |     #else
16281 |     {
16282 |         MA_ATOMIC_FETCH_XOR_CAS(32, dst, src, order);
16283 |     }
16284 |     #endif
16285 | }
16286 | static MA_INLINE ma_uint64 ma_atomic_fetch_xor_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16287 | {
16288 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64)
16289 |     {
16290 |         (void)order;
16291 |         return __sync_fetch_and_xor(dst, src);
16292 |     }
16293 |     #else
16294 |     {
16295 |         MA_ATOMIC_FETCH_XOR_CAS(64, dst, src, order);
16296 |     }
16297 |     #endif
16298 | }
```

```

16299 |     #elif defined(MA_ATOMIC_LEGACY_GCC_ASM)
16300 |     #define MA_ATOMIC_CMPXCHG_GCC_X86(instructionSizeSuffix, result, dst, expected, replacement)
16301 |     \
16302 |     __asm__ __volatile__(
16303 |         "lock; cmpxchg" instructionSizeSuffix " %2, %1" \
16304 |         : "=a"(result), \
16305 |         : "r"(replacement), \
16306 |         : "0"(expected), \
16307 |         : "m"(*dst) \
16308 |         : "cc", "memory")
16309 |     #define MA_ATOMIC_XADD_GCC_X86(instructionSizeSuffix, result, dst, src) \
16310 |     __asm__ __volatile__(
16311 |         "lock; xadd" instructionSizeSuffix " %0, %1" \
16312 |         : "=a"(result), \
16313 |         : "m"(*dst) \
16314 |         : "0"(src), \
16315 |         : "m"(*dst) \
16316 |         : "cc", "memory")
16317 |     static MA_INLINE ma_uint8 ma_atomic_compare_and_swap_8(volatile ma_uint8* dst, ma_uint8
> expected, ma_uint8 replacement)
16318 |     {
16319 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_8) && (defined(MA_X86) || defined(MA_X64))
16320 |         {
16321 |             ma_uint8 result;
16322 |             #if defined(MA_X86) || defined(MA_X64)
16323 |             {
16324 |                 MA_ATOMIC_CMPXCHG_GCC_X86("b", result, dst, expected, replacement);
16325 |             }
16326 |             #else
16327 |             {
16328 |                 #error Unsupported architecture.
16329 |             }
16330 |             #endif
16331 |             return result;
16332 |         }
16333 |         #else
16334 |         {
16335 |             MA_ATOMIC_COMPARE_AND_SWAP_LOCK(8, dst, expected, replacement);
16336 |         }
16337 |         #endif
16338 |     }
16339 |     static MA_INLINE ma_uint16 ma_atomic_compare_and_swap_16(volatile ma_uint16* dst, ma_uint16
> expected, ma_uint16 replacement)
16340 |     {
16341 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16) && (defined(MA_X86) || defined(MA_X64))
16342 |         {
16343 |             ma_uint16 result;
16344 |             #if defined(MA_X86) || defined(MA_X64)
16345 |             {
16346 |                 MA_ATOMIC_CMPXCHG_GCC_X86("w", result, dst, expected, replacement);
16347 |             }
16348 |             #else
16349 |             {
16350 |                 #error Unsupported architecture.
16351 |             }
16352 |             #endif
16353 |             return result;
16354 |         }
16355 |         #else
16356 |         {
16357 |             MA_ATOMIC_COMPARE_AND_SWAP_LOCK(16, dst, expected, replacement);
16358 |         }
16359 |         #endif
16360 |     }
16361 |     static MA_INLINE ma_uint32 ma_atomic_compare_and_swap_32(volatile ma_uint32* dst, ma_uint32
> expected, ma_uint32 replacement)
16362 |     {
16363 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32) && (defined(MA_X86) || defined(MA_X64))
16364 |         {
16365 |             ma_uint32 result;
16366 |             #if defined(MA_X86) || defined(MA_X64)
16367 |             {
16368 |                 MA_ATOMIC_CMPXCHG_GCC_X86("l", result, dst, expected, replacement);
16369 |             }
16370 |             #else
16371 |             {
16372 |                 #error Unsupported architecture.
16373 |             }

```

```

16374 |         #endif
16375 |         return result;
16376 |     }
16377 |     #else
16378 |     {
16379 |         MA_ATOMIC_COMPARE_AND_SWAP_LOCK(32, dst, expected, replacement);
16380 |     }
16381 |     #endif
16382 | }
16383 | static MA_INLINE ma_uint64 ma_atomic_compare_and_swap_64(volatile ma_uint64* dst, ma_uint64
> expected, ma_uint64 replacement)
16384 | {
16385 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64) && (defined(MA_X86) || defined(MA_X64))
16386 |     {
16387 |         ma_uint64 result;
16388 |         #if defined(MA_X86)
16389 |         {
16390 |             ma_uint32 resultEAX;
16391 |             ma_uint32 resultEDX;
16392 |             __asm__ __volatile__(
16393 |                 "pushl %%ebx\n"
16394 |                 "movl  %4, %%ebx\n"
16395 |                 "lock cmpxchg8b (%edi)\n"
16396 |                 "popl  %%ebx\n"
16397 |                 : "=a"(resultEAX),
16398 |                   "=d"(resultEDX)
16399 |                 : "a"((ma_uint32)(expected & 0xFFFFFFFF)),
16400 |                   "d"((ma_uint32)(expected >> 32)),
16401 |                   "r"((ma_uint32)(replacement & 0xFFFFFFFF)),
16402 |                   "c"((ma_uint32)(replacement >> 32)),
16403 |                   "D"(dst)
16404 |                 : "memory", "cc");
16405 |             result = ((ma_uint64)resultEDX << 32) | resultEAX;
16406 |         }
16407 |         #elif defined(MA_X64)
16408 |         {
16409 |             MA_ATOMIC_CMPXCHG_GCC_X86("q", result, dst, expected, replacement);
16410 |         }
16411 |         #else
16412 |         {
16413 |             #error Unsupported architecture.
16414 |         }
16415 |         #endif
16416 |         return result;
16417 |     }
16418 |     #else
16419 |     {
16420 |         MA_ATOMIC_COMPARE_AND_SWAP_LOCK(64, dst, expected, replacement);
16421 |     }
16422 |     #endif
16423 | }
16424 | static MA_INLINE ma_uint8 ma_atomic_load_explicit_8(volatile const ma_uint8* dst,
> ma_atomic_memory_order order)
16425 | {
16426 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8) && (defined(MA_X86) || defined(MA_X64))
16427 |     {
16428 |         ma_uint8 result;
16429 |         #if defined(MA_X86) || defined(MA_X64)
16430 |         {
16431 |             if (order == ma_atomic_memory_order_relaxed) {
16432 |                 MA_ATOMIC_LOAD_RELAXED_GCC_X86("b", result, dst);
16433 |             } else if (order <= ma_atomic_memory_order_release) {
16434 |                 MA_ATOMIC_LOAD_RELEASE_GCC_X86("b", result, dst);
16435 |             } else {
16436 |                 MA_ATOMIC_LOAD_SEQ_CST_GCC_X86("b", result, dst);
16437 |             }
16438 |         }
16439 |         #else
16440 |         {
16441 |             #error Unsupported architecture.
16442 |         }
16443 |         #endif
16444 |         return result;
16445 |     }
16446 |     #else
16447 |     {
16448 |         MA_ATOMIC_LOAD_EXPLICIT_LOCK(8, dst, order);
16449 |     }
16450 |     #endif

```

```

16451 |     }
16452 |     static MA_INLINE ma_uint16 ma_atomic_load_explicit_16(volatile const ma_uint16* dst,
> ma_atomic_memory_order order)
16453 |     {
16454 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_16) && (defined(MA_X86) || defined(MA_X64))
16455 |         {
16456 |             ma_uint16 result;
16457 |             #if defined(MA_X86) || defined(MA_X64)
16458 |             {
16459 |                 if (order == ma_atomic_memory_order_relaxed) {
16460 |                     MA_ATOMIC_LOAD_RELAXED_GCC_X86("w", result, dst);
16461 |                 } else if (order <= ma_atomic_memory_order_release) {
16462 |                     MA_ATOMIC_LOAD_RELEASE_GCC_X86("w", result, dst);
16463 |                 } else {
16464 |                     MA_ATOMIC_LOAD_SEQ_CST_GCC_X86("w", result, dst);
16465 |                 }
16466 |             }
16467 |             #else
16468 |             {
16469 |                 #error Unsupported architecture.
16470 |             }
16471 |             #endif
16472 |             return result;
16473 |         }
16474 |         #else
16475 |         {
16476 |             MA_ATOMIC_LOAD_EXPLICIT_LOCK(16, dst, order);
16477 |         }
16478 |         #endif
16479 |     }
16480 |     static MA_INLINE ma_uint32 ma_atomic_load_explicit_32(volatile const ma_uint32* dst,
> ma_atomic_memory_order order)
16481 |     {
16482 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_32) && (defined(MA_X86) || defined(MA_X64))
16483 |         {
16484 |             ma_uint32 result;
16485 |             #if defined(MA_X86) || defined(MA_X64)
16486 |             {
16487 |                 if (order == ma_atomic_memory_order_relaxed) {
16488 |                     MA_ATOMIC_LOAD_RELAXED_GCC_X86("l", result, dst);
16489 |                 } else if (order <= ma_atomic_memory_order_release) {
16490 |                     MA_ATOMIC_LOAD_RELEASE_GCC_X86("l", result, dst);
16491 |                 } else {
16492 |                     MA_ATOMIC_LOAD_SEQ_CST_GCC_X86("l", result, dst);
16493 |                 }
16494 |             }
16495 |             #else
16496 |             {
16497 |                 #error Unsupported architecture.
16498 |             }
16499 |             #endif
16500 |             return result;
16501 |         }
16502 |         #else
16503 |         {
16504 |             MA_ATOMIC_LOAD_EXPLICIT_LOCK(32, dst, order);
16505 |         }
16506 |         #endif
16507 |     }
16508 |     static MA_INLINE ma_uint64 ma_atomic_load_explicit_64(volatile const ma_uint64* dst,
> ma_atomic_memory_order order)
16509 |     {
16510 |         #if defined(MA_ATOMIC_IS_LOCK_FREE_64) && (defined(MA_X86) || defined(MA_X64))
16511 |         {
16512 |             ma_uint64 result;
16513 |             #if defined(MA_X64)
16514 |             {
16515 |                 if (order == ma_atomic_memory_order_relaxed) {
16516 |                     MA_ATOMIC_LOAD_RELAXED_GCC_X86("q", result, dst);
16517 |                 } else if (order <= ma_atomic_memory_order_release) {
16518 |                     MA_ATOMIC_LOAD_RELEASE_GCC_X86("q", result, dst);
16519 |                 } else {
16520 |                     MA_ATOMIC_LOAD_SEQ_CST_GCC_X86("q", result, dst);
16521 |                 }
16522 |             }
16523 |             #elif defined(MA_X86)
16524 |             {
16525 |                 (void)order;
16526 |                 return ma_atomic_compare_and_swap_64((volatile ma_uint64*)dst, 0, 0);

```



```

16527 |         }
16528 |         #else
16529 |         {
16530 |             #error Unsupported architecture.
16531 |         }
16532 |         #endif
16533 |         return result;
16534 |     }
16535 |     #else
16536 |     {
16537 |         MA_ATOMIC_LOAD_EXPLICIT_LOCK(64, dst, order);
16538 |     }
16539 |     #endif
16540 | }
16541 | static MA_INLINE ma_uint8 ma_atomic_exchange_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16542 | {
16543 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8) && (defined(MA_X86) || defined(MA_X64))
16544 |     {
16545 |         ma_uint8 result;
16546 |         (void)order;
16547 |         #if defined(MA_X86) || defined(MA_X64)
16548 |         {
16549 |             MA_ATOMIC_XCHG_GCC_X86("b", result, dst, src);
16550 |         }
16551 |         #else
16552 |         {
16553 |             #error Unsupported architecture.
16554 |         }
16555 |         #endif
16556 |         return result;
16557 |     }
16558 |     #else
16559 |     {
16560 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(8, dst, src, order);
16561 |     }
16562 |     #endif
16563 | }
16564 | static MA_INLINE ma_uint16 ma_atomic_exchange_explicit_16(volatile ma_uint16* dst, ma_uint16
> src, ma_atomic_memory_order order)
16565 | {
16566 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16) && (defined(MA_X86) || defined(MA_X64))
16567 |     {
16568 |         ma_uint16 result;
16569 |         (void)order;
16570 |         #if defined(MA_X86) || defined(MA_X64)
16571 |         {
16572 |             MA_ATOMIC_XCHG_GCC_X86("w", result, dst, src);
16573 |         }
16574 |         #else
16575 |         {
16576 |             #error Unsupported architecture.
16577 |         }
16578 |         #endif
16579 |         return result;
16580 |     }
16581 |     #else
16582 |     {
16583 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(16, dst, src, order);
16584 |     }
16585 |     #endif
16586 | }
16587 | static MA_INLINE ma_uint32 ma_atomic_exchange_explicit_32(volatile ma_uint32* dst, ma_uint32
> src, ma_atomic_memory_order order)
16588 | {
16589 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32) && (defined(MA_X86) || defined(MA_X64))
16590 |     {
16591 |         ma_uint32 result;
16592 |         (void)order;
16593 |         #if defined(MA_X86) || defined(MA_X64)
16594 |         {
16595 |             MA_ATOMIC_XCHG_GCC_X86("l", result, dst, src);
16596 |         }
16597 |         #else
16598 |         {
16599 |             #error Unsupported architecture.
16600 |         }
16601 |         #endif
16602 |         return result;

```

```

16603 |     }
16604 |     #else
16605 |     {
16606 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(32, dst, src, order);
16607 |     }
16608 |     #endif
16609 | }
16610 | static MA_INLINE ma_uint64 ma_atomic_exchange_explicit_64(volatile ma_uint64* dst, ma_uint64
> src, ma_atomic_memory_order order)
16611 | {
16612 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64) && (defined(MA_X86) || defined(MA_X64))
16613 |     {
16614 |         ma_uint64 result;
16615 |         (void)order;
16616 |         #if defined(MA_X86)
16617 |         {
16618 |             MA_ATOMIC_EXCHANGE_EXPLICIT_CAS(64, dst, src, order);
16619 |         }
16620 |         #elif defined(MA_X64)
16621 |         {
16622 |             MA_ATOMIC_XCHG_GCC_X86("q", result, dst, src);
16623 |         }
16624 |         #else
16625 |         {
16626 |             #error Unsupported architecture.
16627 |         }
16628 |         #endif
16629 |         return result;
16630 |     }
16631 |     #else
16632 |     {
16633 |         MA_ATOMIC_EXCHANGE_EXPLICIT_LOCK(64, dst, src, order);
16634 |     }
16635 |     #endif
16636 | }
16637 | static MA_INLINE void ma_atomic_store_explicit_8(volatile ma_uint8* dst, ma_uint8 src,
> ma_atomic_memory_order order)
16638 | {
16639 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8) && (defined(MA_X86) || defined(MA_X64))
16640 |     {
16641 |         #if defined(MA_X86) || defined(MA_X64)
16642 |         {
16643 |             if (order == ma_atomic_memory_order_relaxed) {
16644 |                 __asm__ __volatile__ (
16645 |                     "movb %1, %0"
16646 |                     : "=m"(*dst)
16647 |                     : "r"(src)
16648 |                     );
16649 |             } else {
16650 |                 __asm__ __volatile__ (
16651 |                     "xchgb %1, %0"
16652 |                     : "=m"(*dst)
16653 |                     : "r"(src)
16654 |                     : "memory"
16655 |                     );
16656 |             }
16657 |         }
16658 |         #else
16659 |         {
16660 |             #error Unsupported architecture.
16661 |         }
16662 |         #endif
16663 |     }
16664 |     #else
16665 |     {
16666 |         MA_ATOMIC_STORE_EXPLICIT_LOCK(8, dst, src, order);
16667 |     }
16668 |     #endif
16669 | }
16670 | static MA_INLINE void ma_atomic_store_explicit_16(volatile ma_uint16* dst, ma_uint16 src,
> ma_atomic_memory_order order)
16671 | {
16672 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16) && (defined(MA_X86) || defined(MA_X64))
16673 |     {
16674 |         #if defined(MA_X86) || defined(MA_X64)
16675 |         {
16676 |             if (order == ma_atomic_memory_order_relaxed) {
16677 |                 __asm__ __volatile__ (
16678 |                     "movw %1, %0"

```

```

16679 |             : "=m"(*dst)
16680 |             : "r"(src)
16681 |         );
16682 |     } else {
16683 |         __asm__ __volatile__ (
16684 |             "xchgw %1, %0"
16685 |             : "=m"(*dst)
16686 |             : "r"(src)
16687 |             : "memory"
16688 |         );
16689 |     }
16690 | }
16691 | #else
16692 | {
16693 |     #error Unsupported architecture.
16694 | }
16695 | #endif
16696 | }
16697 | #else
16698 | {
16699 |     MA_ATOMIC_STORE_EXPLICIT_LOCK(16, dst, src, order);
16700 | }
16701 | #endif
16702 | }
16703 | static MA_INLINE void ma_atomic_store_explicit_32(volatile ma_uint32* dst, ma_uint32 src,
> ma_atomic_memory_order order)
16704 | {
16705 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32) && (defined(MA_X86) || defined(MA_X64))
16706 |     {
16707 |         #if defined(MA_X86) || defined(MA_X64)
16708 |         {
16709 |             if (order == ma_atomic_memory_order_relaxed) {
16710 |                 __asm__ __volatile__ (
16711 |                     "movl %1, %0"
16712 |                     : "=m"(*dst)
16713 |                     : "r"(src)
16714 |                 );
16715 |             } else {
16716 |                 __asm__ __volatile__ (
16717 |                     "xchgl %1, %0"
16718 |                     : "=m"(*dst)
16719 |                     : "r"(src)
16720 |                     : "memory"
16721 |                 );
16722 |             }
16723 |         }
16724 |     }
16725 |     #else
16726 |     {
16727 |         #error Unsupported architecture.
16728 |     }
16729 |     #endif
16730 | }
16731 | #else
16732 | {
16733 |     MA_ATOMIC_STORE_EXPLICIT_LOCK(32, dst, src, order);
16734 | }
16735 | #endif
16736 | }
16737 | static MA_INLINE void ma_atomic_store_explicit_64(volatile ma_uint64* dst, ma_uint64 src,
> ma_atomic_memory_order order)
16738 | {
16739 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64) && (defined(MA_X86) || defined(MA_X64))
16740 |     {
16741 |         #if defined(MA_X64)
16742 |         {
16743 |             if (order == ma_atomic_memory_order_relaxed) {
16744 |                 __asm__ __volatile__ (
16745 |                     "movq %1, %0"
16746 |                     : "=m"(*dst)
16747 |                     : "r"(src)
16748 |                 );
16749 |             } else {
16750 |                 __asm__ __volatile__ (
16751 |                     "xchq %1, %0"
16752 |                     : "=m"(*dst)
16753 |                     : "r"(src)
16754 |                     : "memory"
16755 |                 );
16756 |             }
16757 |         }
16758 |     }
16759 |     #else
16760 |     {
16761 |         #error Unsupported architecture.
16762 |     }
16763 |     #endif
16764 | }
16765 | #else
16766 | {
16767 |     MA_ATOMIC_STORE_EXPLICIT_LOCK(64, dst, src, order);
16768 | }
16769 | #endif
16770 | }

```

```
16756 |         }
16757 |         #else
16758 |         {
16759 |             MA_ATOMIC_STORE_EXPLICIT_CAS(64, dst, src, order);
16760 |         }
16761 |         #endif
16762 |     }
16763 |     #else
16764 |     {
16765 |         MA_ATOMIC_STORE_EXPLICIT_LOCK(64, dst, src, order);
16766 |     }
16767 |     #endif
16768 | }
16769 | static MA_INLINE ma_uint8 ma_atomic_fetch_add_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16770 | {
16771 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_8) && (defined(MA_X86) || defined(MA_X64))
16772 |     {
16773 |         #if defined(MA_X86) || defined(MA_X64)
16774 |         {
16775 |             ma_uint8 result;
16776 |             (void)order;
16777 |             MA_ATOMIC_XADD_GCC_X86("b", result, dst, src);
16778 |             return result;
16779 |         }
16780 |         #else
16781 |         {
16782 |             #error Unsupported architecture.
16783 |         }
16784 |         #endif
16785 |     }
16786 |     #else
16787 |     {
16788 |         MA_ATOMIC_FETCH_ADD_LOCK(8, dst, src, order);
16789 |     }
16790 |     #endif
16791 | }
16792 | static MA_INLINE ma_uint16 ma_atomic_fetch_add_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16793 | {
16794 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_16) && (defined(MA_X86) || defined(MA_X64))
16795 |     {
16796 |         #if defined(MA_X86) || defined(MA_X64)
16797 |         {
16798 |             ma_uint16 result;
16799 |             (void)order;
16800 |             MA_ATOMIC_XADD_GCC_X86("w", result, dst, src);
16801 |             return result;
16802 |         }
16803 |         #else
16804 |         {
16805 |             #error Unsupported architecture.
16806 |         }
16807 |         #endif
16808 |     }
16809 |     #else
16810 |     {
16811 |         MA_ATOMIC_FETCH_ADD_LOCK(16, dst, src, order);
16812 |     }
16813 |     #endif
16814 | }
16815 | static MA_INLINE ma_uint32 ma_atomic_fetch_add_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16816 | {
16817 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_32) && (defined(MA_X86) || defined(MA_X64))
16818 |     {
16819 |         #if defined(MA_X86) || defined(MA_X64)
16820 |         {
16821 |             ma_uint32 result;
16822 |             (void)order;
16823 |             MA_ATOMIC_XADD_GCC_X86("l", result, dst, src);
16824 |             return result;
16825 |         }
16826 |         #else
16827 |         {
16828 |             #error Unsupported architecture.
16829 |         }
16830 |         #endif
16831 |     }
```

```

16832 |         #else
16833 |         {
16834 |             MA_ATOMIC_FETCH_ADD_LOCK(32, dst, src, order);
16835 |         }
16836 |     #endif
16837 | }
16838 | static MA_INLINE ma_uint64 ma_atomic_fetch_add_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16839 | {
16840 |     #if defined(MA_ATOMIC_IS_LOCK_FREE_64) && (defined(MA_X86) || defined(MA_X64))
16841 |     {
16842 |         #if defined(MA_X86)
16843 |         {
16844 |             MA_ATOMIC_FETCH_ADD_CAS(64, dst, src, order);
16845 |         }
16846 |         #elif defined(MA_X64)
16847 |         {
16848 |             ma_uint64 result;
16849 |             MA_ATOMIC_XADD_GCC_X86("q", result, dst, src);
16850 |             (void)order;
16851 |             return result;
16852 |         }
16853 |         #else
16854 |         {
16855 |             #error Unsupported architecture.
16856 |         }
16857 |     #endif
16858 |     }
16859 |     #else
16860 |     {
16861 |         MA_ATOMIC_FETCH_ADD_LOCK(64, dst, src, order);
16862 |     }
16863 |     #endif
16864 | }
16865 | static MA_INLINE ma_uint8 ma_atomic_fetch_sub_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16866 | {
16867 |     return ma_atomic_fetch_add_explicit_8(dst, (ma_uint8)(-(ma_int8)src), order);
16868 | }
16869 | static MA_INLINE ma_uint16 ma_atomic_fetch_sub_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16870 | {
16871 |     return ma_atomic_fetch_add_explicit_16(dst, (ma_uint16)(-(ma_int16)src), order);
16872 | }
16873 | static MA_INLINE ma_uint32 ma_atomic_fetch_sub_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16874 | {
16875 |     return ma_atomic_fetch_add_explicit_32(dst, (ma_uint32)(-(ma_int32)src), order);
16876 | }
16877 | static MA_INLINE ma_uint64 ma_atomic_fetch_sub_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16878 | {
16879 |     return ma_atomic_fetch_add_explicit_64(dst, (ma_uint64)(-(ma_int64)src), order);
16880 | }
16881 | static MA_INLINE ma_uint8 ma_atomic_fetch_and_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16882 | {
16883 |     MA_ATOMIC_FETCH_AND_CAS(8, dst, src, order);
16884 | }
16885 | static MA_INLINE ma_uint16 ma_atomic_fetch_and_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16886 | {
16887 |     MA_ATOMIC_FETCH_AND_CAS(16, dst, src, order);
16888 | }
16889 | static MA_INLINE ma_uint32 ma_atomic_fetch_and_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16890 | {
16891 |     MA_ATOMIC_FETCH_AND_CAS(32, dst, src, order);
16892 | }
16893 | static MA_INLINE ma_uint64 ma_atomic_fetch_and_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16894 | {
16895 |     MA_ATOMIC_FETCH_AND_CAS(64, dst, src, order);
16896 | }
16897 | static MA_INLINE ma_uint8 ma_atomic_fetch_or_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16898 | {
16899 |     MA_ATOMIC_FETCH_OR_CAS(8, dst, src, order);
16900 | }

```

```

16901 |         static MA_INLINE ma_uint16 ma_atomic_fetch_or_explicit_16(volatile ma_uint16* dst, ma_uint16
> src, ma_atomic_memory_order order)
16902 |         {
16903 |             MA_ATOMIC_FETCH_OR_CAS(16, dst, src, order);
16904 |         }
16905 |         static MA_INLINE ma_uint32 ma_atomic_fetch_or_explicit_32(volatile ma_uint32* dst, ma_uint32
> src, ma_atomic_memory_order order)
16906 |         {
16907 |             MA_ATOMIC_FETCH_OR_CAS(32, dst, src, order);
16908 |         }
16909 |         static MA_INLINE ma_uint64 ma_atomic_fetch_or_explicit_64(volatile ma_uint64* dst, ma_uint64
> src, ma_atomic_memory_order order)
16910 |         {
16911 |             MA_ATOMIC_FETCH_OR_CAS(64, dst, src, order);
16912 |         }
16913 |         static MA_INLINE ma_uint8 ma_atomic_fetch_xor_explicit_8(volatile ma_uint8* dst, ma_uint8
> src, ma_atomic_memory_order order)
16914 |         {
16915 |             MA_ATOMIC_FETCH_XOR_CAS(8, dst, src, order);
16916 |         }
16917 |         static MA_INLINE ma_uint16 ma_atomic_fetch_xor_explicit_16(volatile ma_uint16* dst,
> ma_uint16 src, ma_atomic_memory_order order)
16918 |         {
16919 |             MA_ATOMIC_FETCH_XOR_CAS(16, dst, src, order);
16920 |         }
16921 |         static MA_INLINE ma_uint32 ma_atomic_fetch_xor_explicit_32(volatile ma_uint32* dst,
> ma_uint32 src, ma_atomic_memory_order order)
16922 |         {
16923 |             MA_ATOMIC_FETCH_XOR_CAS(32, dst, src, order);
16924 |         }
16925 |         static MA_INLINE ma_uint64 ma_atomic_fetch_xor_explicit_64(volatile ma_uint64* dst,
> ma_uint64 src, ma_atomic_memory_order order)
16926 |         {
16927 |             MA_ATOMIC_FETCH_XOR_CAS(64, dst, src, order);
16928 |         }
16929 |     #else
16930 |     #error Unsupported compiler.
16931 | #endif
16932 | #endif
16933 | #if !defined(MA_ATOMIC_HAS_NATIVE_COMPARE_EXCHANGE)
16934 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_8(volatile ma_uint8* dst,
> ma_uint8* expected, ma_uint8 replacement, ma_atomic_memory_order successOrder,
> ma_atomic_memory_order failureOrder)
16935 |     {
16936 |         ma_uint8 result;
16937 |         (void)successOrder;
16938 |         (void)failureOrder;
16939 |         result = ma_atomic_compare_and_swap_8(dst, *expected, replacement);
16940 |         if (result == *expected) {
16941 |             return 1;
16942 |         } else {
16943 |             *expected = result;
16944 |             return 0;
16945 |         }
16946 |     }
16947 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_16(volatile ma_uint16*
> dst, ma_uint16* expected, ma_uint16 replacement, ma_atomic_memory_order successOrder,
> ma_atomic_memory_order failureOrder)
16948 |     {
16949 |         ma_uint16 result;
16950 |         (void)successOrder;
16951 |         (void)failureOrder;
16952 |         result = ma_atomic_compare_and_swap_16(dst, *expected, replacement);
16953 |         if (result == *expected) {
16954 |             return 1;
16955 |         } else {
16956 |             *expected = result;
16957 |             return 0;
16958 |         }
16959 |     }
16960 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_32(volatile ma_uint32*
> dst, ma_uint32* expected, ma_uint32 replacement, ma_atomic_memory_order successOrder,
> ma_atomic_memory_order failureOrder)
16961 |     {
16962 |         ma_uint32 result;
16963 |         (void)successOrder;
16964 |         (void)failureOrder;
16965 |         result = ma_atomic_compare_and_swap_32(dst, *expected, replacement);
16966 |         if (result == *expected) {

```

```

16967 |         return 1;
16968 |     } else {
16969 |         *expected = result;
16970 |         return 0;
16971 |     }
16972 | }
16973 | static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_64(volatile ma_uint64*
> dst, volatile ma_uint64* expected, ma_uint64 replacement, ma_atomic_memory_order successOrder,
> ma_atomic_memory_order failureOrder)
16974 | {
16975 |     ma_uint64 result;
16976 |     (void)successOrder;
16977 |     (void)failureOrder;
16978 |     result = ma_atomic_compare_and_swap_64(dst, *expected, replacement);
16979 |     if (result == *expected) {
16980 |         return 1;
16981 |     } else {
16982 |         *expected = result;
16983 |         return 0;
16984 |     }
16985 | }
16986 | #define ma_atomic_compare_exchange_weak_explicit_8( dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_8( dst, expected, replacement,
> successOrder, failureOrder)
16987 | #define ma_atomic_compare_exchange_weak_explicit_16(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_16(dst, expected, replacement,
> successOrder, failureOrder)
16988 | #define ma_atomic_compare_exchange_weak_explicit_32(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_32(dst, expected, replacement,
> successOrder, failureOrder)
16989 | #define ma_atomic_compare_exchange_weak_explicit_64(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_64(dst, expected, replacement,
> successOrder, failureOrder)
16990 | #endif
16991 | #if defined(MA_64BIT)
16992 |     static MA_INLINE ma_bool32 ma_atomic_is_lock_free_ptr(volatile void** ptr)
16993 |     {
16994 |         return ma_atomic_is_lock_free_64((volatile ma_uint64*)ptr);
16995 |     }
16996 |     static MA_INLINE void* ma_atomic_load_explicit_ptr(volatile void** ptr, ma_atomic_memory_order
> order)
16997 |     {
16998 |         return (void*)ma_atomic_load_explicit_64((volatile ma_uint64*)ptr, order);
16999 |     }
17000 |     static MA_INLINE void ma_atomic_store_explicit_ptr(volatile void** dst, void* src,
> ma_atomic_memory_order order)
17001 |     {
17002 |         ma_atomic_store_explicit_64((volatile ma_uint64*)dst, (ma_uint64)src, order);
17003 |     }
17004 |     static MA_INLINE void* ma_atomic_exchange_explicit_ptr(volatile void** dst, void* src,
> ma_atomic_memory_order order)
17005 |     {
17006 |         return (void*)ma_atomic_exchange_explicit_64((volatile ma_uint64*)dst, (ma_uint64)src,
> order);
17007 |     }
17008 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_ptr(volatile void** dst,
> void** expected, void* replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
> failureOrder)
17009 |     {
17010 |         return ma_atomic_compare_exchange_strong_explicit_64((volatile ma_uint64*)dst,
> (ma_uint64*)expected, (ma_uint64)replacement, successOrder, failureOrder);
17011 |     }
17012 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_weak_explicit_ptr(volatile void** dst,
> void** expected, void* replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
> failureOrder)
17013 |     {
17014 |         return ma_atomic_compare_exchange_weak_explicit_64((volatile ma_uint64*)dst,
> (ma_uint64*)expected, (ma_uint64)replacement, successOrder, failureOrder);
17015 |     }
17016 |     static MA_INLINE void* ma_atomic_compare_and_swap_ptr(volatile void** dst, void* expected, void*
> replacement)
17017 |     {
17018 |         return (void*)ma_atomic_compare_and_swap_64((volatile ma_uint64*)dst, (ma_uint64)expected,
> (ma_uint64)replacement);
17019 |     }
17020 | #elif defined(MA_32BIT)
17021 |     static MA_INLINE ma_bool32 ma_atomic_is_lock_free_ptr(volatile void** ptr)
17022 |     {
17023 |         return ma_atomic_is_lock_free_32((volatile ma_uint32*)ptr);

```



```

17024 |     }
17025 |     static MA_INLINE void* ma_atomic_load_explicit_ptr(volatile void** ptr, ma_atomic_memory_order
> order)
17026 |     {
17027 |         return (void*)ma_atomic_load_explicit_32((volatile ma_uint32*)ptr, order);
17028 |     }
17029 |     static MA_INLINE void ma_atomic_store_explicit_ptr(volatile void** dst, void* src,
> ma_atomic_memory_order order)
17030 |     {
17031 |         ma_atomic_store_explicit_32((volatile ma_uint32*)dst, (ma_uint32)src, order);
17032 |     }
17033 |     static MA_INLINE void* ma_atomic_exchange_explicit_ptr(volatile void** dst, void* src,
> ma_atomic_memory_order order)
17034 |     {
17035 |         return (void*)ma_atomic_exchange_explicit_32((volatile ma_uint32*)dst, (ma_uint32)src,
> order);
17036 |     }
17037 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_ptr(volatile void** dst,
> void** expected, void* replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
> failureOrder)
17038 |     {
17039 |         return ma_atomic_compare_exchange_strong_explicit_32((volatile ma_uint32*)dst,
> (ma_uint32*)expected, (ma_uint32)replacement, successOrder, failureOrder);
17040 |     }
17041 |     static MA_INLINE ma_bool32 ma_atomic_compare_exchange_weak_explicit_ptr(volatile void** dst,
> void** expected, void* replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
> failureOrder)
17042 |     {
17043 |         return ma_atomic_compare_exchange_weak_explicit_32((volatile ma_uint32*)dst,
> (ma_uint32*)expected, (ma_uint32)replacement, successOrder, failureOrder);
17044 |     }
17045 |     static MA_INLINE void* ma_atomic_compare_and_swap_ptr(volatile void** dst, void* expected, void*
> replacement)
17046 |     {
17047 |         return (void*)ma_atomic_compare_and_swap_32((volatile ma_uint32*)dst, (ma_uint32)expected,
> (ma_uint32)replacement);
17048 |     }
17049 | #else
17050 |     #error Unsupported architecture.
17051 | #endif
17052 | #define ma_atomic_store_ptr(dst, src)
> ma_atomic_store_explicit_ptr((volatile void**)dst, (void*)src, ma_atomic_memory_order_seq_cst)
17053 | #define ma_atomic_load_ptr(ptr)
> ma_atomic_load_explicit_ptr((volatile void**)ptr, ma_atomic_memory_order_seq_cst)
17054 | #define ma_atomic_exchange_ptr(dst, src)
> ma_atomic_exchange_explicit_ptr((volatile void**)dst, (void*)src, ma_atomic_memory_order_seq_cst)
17055 | #define ma_atomic_compare_exchange_strong_ptr(dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_ptr((volatile void**)dst, (void**)expected,
> (void*)replacement, ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17056 | #define ma_atomic_compare_exchange_weak_ptr(dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_ptr((volatile void**)dst, (void**)expected,
> (void*)replacement, ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17057 | #define ma_atomic_store_8( dst, src)
> dst, src, ma_atomic_memory_order_seq_cst) ma_atomic_store_explicit_8(
17058 | #define ma_atomic_store_16(dst, src)
> ma_atomic_store_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17059 | #define ma_atomic_store_32(dst, src)
> ma_atomic_store_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17060 | #define ma_atomic_store_64(dst, src)
> ma_atomic_store_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17061 | #define ma_atomic_load_8( ptr)
> ptr, ma_atomic_memory_order_seq_cst) ma_atomic_load_explicit_8(
17062 | #define ma_atomic_load_16(ptr)
> ma_atomic_load_explicit_16(ptr, ma_atomic_memory_order_seq_cst)
17063 | #define ma_atomic_load_32(ptr)
> ma_atomic_load_explicit_32(ptr, ma_atomic_memory_order_seq_cst)
17064 | #define ma_atomic_load_64(ptr)
> ma_atomic_load_explicit_64(ptr, ma_atomic_memory_order_seq_cst)
17065 | #define ma_atomic_exchange_8( dst, src)
> ma_atomic_exchange_explicit_8( dst, src, ma_atomic_memory_order_seq_cst)
17066 | #define ma_atomic_exchange_16(dst, src)
> ma_atomic_exchange_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17067 | #define ma_atomic_exchange_32(dst, src)
> ma_atomic_exchange_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17068 | #define ma_atomic_exchange_64(dst, src)
> ma_atomic_exchange_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17069 | #define ma_atomic_compare_exchange_strong_8( dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_8( dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)

```

```

17070 | #define ma_atomic_compare_exchange_strong_16(dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_16(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17071 | #define ma_atomic_compare_exchange_strong_32(dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_32(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17072 | #define ma_atomic_compare_exchange_strong_64(dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_64(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17073 | #define ma_atomic_compare_exchange_weak_8( dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_8( dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17074 | #define ma_atomic_compare_exchange_weak_16( dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_16(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17075 | #define ma_atomic_compare_exchange_weak_32( dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_32(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17076 | #define ma_atomic_compare_exchange_weak_64( dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_64(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17077 | #define ma_atomic_fetch_add_8( dst, src)
> ma_atomic_fetch_add_explicit_8( dst, src, ma_atomic_memory_order_seq_cst)
17078 | #define ma_atomic_fetch_add_16(dst, src)
> ma_atomic_fetch_add_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17079 | #define ma_atomic_fetch_add_32(dst, src)
> ma_atomic_fetch_add_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17080 | #define ma_atomic_fetch_add_64(dst, src)
> ma_atomic_fetch_add_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17081 | #define ma_atomic_fetch_sub_8( dst, src)
> ma_atomic_fetch_sub_explicit_8( dst, src, ma_atomic_memory_order_seq_cst)
17082 | #define ma_atomic_fetch_sub_16(dst, src)
> ma_atomic_fetch_sub_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17083 | #define ma_atomic_fetch_sub_32(dst, src)
> ma_atomic_fetch_sub_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17084 | #define ma_atomic_fetch_sub_64(dst, src)
> ma_atomic_fetch_sub_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17085 | #define ma_atomic_fetch_or_8( dst, src)
> ma_atomic_fetch_or_explicit_8( dst, src, ma_atomic_memory_order_seq_cst)
17086 | #define ma_atomic_fetch_or_16(dst, src)
> ma_atomic_fetch_or_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17087 | #define ma_atomic_fetch_or_32(dst, src)
> ma_atomic_fetch_or_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17088 | #define ma_atomic_fetch_or_64(dst, src)
> ma_atomic_fetch_or_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17089 | #define ma_atomic_fetch_xor_8( dst, src)
> ma_atomic_fetch_xor_explicit_8( dst, src, ma_atomic_memory_order_seq_cst)
17090 | #define ma_atomic_fetch_xor_16(dst, src)
> ma_atomic_fetch_xor_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17091 | #define ma_atomic_fetch_xor_32(dst, src)
> ma_atomic_fetch_xor_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17092 | #define ma_atomic_fetch_xor_64(dst, src)
> ma_atomic_fetch_xor_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17093 | #define ma_atomic_fetch_and_8( dst, src)
> ma_atomic_fetch_and_explicit_8( dst, src, ma_atomic_memory_order_seq_cst)
17094 | #define ma_atomic_fetch_and_16(dst, src)
> ma_atomic_fetch_and_explicit_16(dst, src, ma_atomic_memory_order_seq_cst)
17095 | #define ma_atomic_fetch_and_32(dst, src)
> ma_atomic_fetch_and_explicit_32(dst, src, ma_atomic_memory_order_seq_cst)
17096 | #define ma_atomic_fetch_and_64(dst, src)
> ma_atomic_fetch_and_explicit_64(dst, src, ma_atomic_memory_order_seq_cst)
17097 | #define ma_atomic_store_explicit_i8( dst, src, order) ma_atomic_store_explicit_8(
> (ma_uint8*)dst, (ma_uint8*)src, order)
17098 | #define ma_atomic_store_explicit_i16(dst, src, order)
> ma_atomic_store_explicit_16((ma_uint16*)dst, (ma_uint16*)src, order)
17099 | #define ma_atomic_store_explicit_i32(dst, src, order)
> ma_atomic_store_explicit_32((ma_uint32*)dst, (ma_uint32*)src, order)
17100 | #define ma_atomic_store_explicit_i64(dst, src, order)
> ma_atomic_store_explicit_64((ma_uint64*)dst, (ma_uint64*)src, order)
17101 | #define ma_atomic_load_explicit_i8( ptr, order) (ma_int8
> )ma_atomic_load_explicit_8( (ma_uint8*)ptr, order)
17102 | #define ma_atomic_load_explicit_i16(ptr, order)
> (ma_int16)ma_atomic_load_explicit_16((ma_uint16*)ptr, order)
17103 | #define ma_atomic_load_explicit_i32(ptr, order)
> (ma_int32)ma_atomic_load_explicit_32((ma_uint32*)ptr, order)
17104 | #define ma_atomic_load_explicit_i64(ptr, order)
> (ma_int64)ma_atomic_load_explicit_64((ma_uint64*)ptr, order)
17105 | #define ma_atomic_exchange_explicit_i8( dst, src, order) (ma_int8
> )ma_atomic_exchange_explicit_8 ((ma_uint8*)dst, (ma_uint8*)src, order)

```

```

17106 | #define ma_atomic_exchange_explicit_i16(dst, src, order)
> (ma_int16)ma_atomic_exchange_explicit_16((ma_uint16*)dst, (ma_uint16)src, order)
17107 | #define ma_atomic_exchange_explicit_i32(dst, src, order)
> (ma_int32)ma_atomic_exchange_explicit_32((ma_uint32*)dst, (ma_uint32)src, order)
17108 | #define ma_atomic_exchange_explicit_i64(dst, src, order)
> (ma_int64)ma_atomic_exchange_explicit_64((ma_uint64*)dst, (ma_uint64)src, order)
17109 | #define ma_atomic_compare_exchange_strong_explicit_i8( dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_8( (ma_uint8* )dst, (ma_uint8* )expected,
> (ma_uint8 )replacement, successOrder, failureOrder)
17110 | #define ma_atomic_compare_exchange_strong_explicit_i16(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_16((ma_uint16*)dst, (ma_uint16*)expected,
> (ma_uint16)replacement, successOrder, failureOrder)
17111 | #define ma_atomic_compare_exchange_strong_explicit_i32(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_32((ma_uint32*)dst, (ma_uint32*)expected,
> (ma_uint32)replacement, successOrder, failureOrder)
17112 | #define ma_atomic_compare_exchange_strong_explicit_i64(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_strong_explicit_64((ma_uint64*)dst, (ma_uint64*)expected,
> (ma_uint64)replacement, successOrder, failureOrder)
17113 | #define ma_atomic_compare_exchange_weak_explicit_i8( dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_weak_explicit_8( (ma_uint8* )dst, (ma_uint8* )expected,
> (ma_uint8 )replacement, successOrder, failureOrder)
17114 | #define ma_atomic_compare_exchange_weak_explicit_i16(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_weak_explicit_16((ma_uint16*)dst, (ma_uint16*)expected,
> (ma_uint16)replacement, successOrder, failureOrder)
17115 | #define ma_atomic_compare_exchange_weak_explicit_i32(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_weak_explicit_32((ma_uint32*)dst, (ma_uint32*)expected,
> (ma_uint32)replacement, successOrder, failureOrder)
17116 | #define ma_atomic_compare_exchange_weak_explicit_i64(dst, expected, replacement, successOrder,
> failureOrder) ma_atomic_compare_exchange_weak_explicit_64((ma_uint64*)dst, (ma_uint64*)expected,
> (ma_uint64)replacement, successOrder, failureOrder)
17117 | #define ma_atomic_fetch_add_explicit_i8( dst, src, order) (ma_int8
> )ma_atomic_fetch_add_explicit_8( (ma_uint8* )dst, (ma_uint8 )src, order)
17118 | #define ma_atomic_fetch_add_explicit_i16(dst, src, order)
> (ma_int16)ma_atomic_fetch_add_explicit_16((ma_uint16*)dst, (ma_uint16)src, order)
17119 | #define ma_atomic_fetch_add_explicit_i32(dst, src, order)
> (ma_int32)ma_atomic_fetch_add_explicit_32((ma_uint32*)dst, (ma_uint32)src, order)
17120 | #define ma_atomic_fetch_add_explicit_i64(dst, src, order)
> (ma_int64)ma_atomic_fetch_add_explicit_64((ma_uint64*)dst, (ma_uint64)src, order)
17121 | #define ma_atomic_fetch_sub_explicit_i8( dst, src, order) (ma_int8
> )ma_atomic_fetch_sub_explicit_8( (ma_uint8* )dst, (ma_uint8 )src, order)
17122 | #define ma_atomic_fetch_sub_explicit_i16(dst, src, order)
> (ma_int16)ma_atomic_fetch_sub_explicit_16((ma_uint16*)dst, (ma_uint16)src, order)
17123 | #define ma_atomic_fetch_sub_explicit_i32(dst, src, order)
> (ma_int32)ma_atomic_fetch_sub_explicit_32((ma_uint32*)dst, (ma_uint32)src, order)
17124 | #define ma_atomic_fetch_sub_explicit_i64(dst, src, order)
> (ma_int64)ma_atomic_fetch_sub_explicit_64((ma_uint64*)dst, (ma_uint64)src, order)
17125 | #define ma_atomic_fetch_or_explicit_i8( dst, src, order) (ma_int8
> )ma_atomic_fetch_or_explicit_8( (ma_uint8* )dst, (ma_uint8 )src, order)
17126 | #define ma_atomic_fetch_or_explicit_i16(dst, src, order)
> (ma_int16)ma_atomic_fetch_or_explicit_16((ma_uint16*)dst, (ma_uint16)src, order)
17127 | #define ma_atomic_fetch_or_explicit_i32(dst, src, order)
> (ma_int32)ma_atomic_fetch_or_explicit_32((ma_uint32*)dst, (ma_uint32)src, order)
17128 | #define ma_atomic_fetch_or_explicit_i64(dst, src, order)
> (ma_int64)ma_atomic_fetch_or_explicit_64((ma_uint64*)dst, (ma_uint64)src, order)
17129 | #define ma_atomic_fetch_xor_explicit_i8( dst, src, order) (ma_int8
> )ma_atomic_fetch_xor_explicit_8( (ma_uint8* )dst, (ma_uint8 )src, order)
17130 | #define ma_atomic_fetch_xor_explicit_i16(dst, src, order)
> (ma_int16)ma_atomic_fetch_xor_explicit_16((ma_uint16*)dst, (ma_uint16)src, order)
17131 | #define ma_atomic_fetch_xor_explicit_i32(dst, src, order)
> (ma_int32)ma_atomic_fetch_xor_explicit_32((ma_uint32*)dst, (ma_uint32)src, order)
17132 | #define ma_atomic_fetch_xor_explicit_i64(dst, src, order)
> (ma_int64)ma_atomic_fetch_xor_explicit_64((ma_uint64*)dst, (ma_uint64)src, order)
17133 | #define ma_atomic_fetch_and_explicit_i8( dst, src, order) (ma_int8
> )ma_atomic_fetch_and_explicit_8( (ma_uint8* )dst, (ma_uint8 )src, order)
17134 | #define ma_atomic_fetch_and_explicit_i16(dst, src, order)
> (ma_int16)ma_atomic_fetch_and_explicit_16((ma_uint16*)dst, (ma_uint16)src, order)
17135 | #define ma_atomic_fetch_and_explicit_i32(dst, src, order)
> (ma_int32)ma_atomic_fetch_and_explicit_32((ma_uint32*)dst, (ma_uint32)src, order)
17136 | #define ma_atomic_fetch_and_explicit_i64(dst, src, order)
> (ma_int64)ma_atomic_fetch_and_explicit_64((ma_uint64*)dst, (ma_uint64)src, order)
17137 | #define ma_atomic_store_i8( dst, src) ma_atomic_store_explicit_i8(
> dst, src, ma_atomic_memory_order_seq_cst)
17138 | #define ma_atomic_store_i16(dst, src)
> ma_atomic_store_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17139 | #define ma_atomic_store_i32(dst, src)
> ma_atomic_store_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17140 | #define ma_atomic_store_i64(dst, src)
> ma_atomic_store_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)
17141 | #define ma_atomic_load_i8( ptr) ma_atomic_load_explicit_i8(

```

```

    > ptr, ma_atomic_memory_order_seq_cst)
17142 | #define ma_atomic_load_i16(ptr)
    > ma_atomic_load_explicit_i16(ptr, ma_atomic_memory_order_seq_cst)
17143 | #define ma_atomic_load_i32(ptr)
    > ma_atomic_load_explicit_i32(ptr, ma_atomic_memory_order_seq_cst)
17144 | #define ma_atomic_load_i64(ptr)
    > ma_atomic_load_explicit_i64(ptr, ma_atomic_memory_order_seq_cst)
17145 | #define ma_atomic_exchange_i8( dst, src)
    > ma_atomic_exchange_explicit_i8( dst, src, ma_atomic_memory_order_seq_cst)
17146 | #define ma_atomic_exchange_i16(dst, src)
    > ma_atomic_exchange_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17147 | #define ma_atomic_exchange_i32(dst, src)
    > ma_atomic_exchange_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17148 | #define ma_atomic_exchange_i64(dst, src)
    > ma_atomic_exchange_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)
17149 | #define ma_atomic_compare_exchange_strong_i8( dst, expected, replacement)
    > ma_atomic_compare_exchange_strong_explicit_i8( dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17150 | #define ma_atomic_compare_exchange_strong_i16(dst, expected, replacement)
    > ma_atomic_compare_exchange_strong_explicit_i16(dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17151 | #define ma_atomic_compare_exchange_strong_i32(dst, expected, replacement)
    > ma_atomic_compare_exchange_strong_explicit_i32(dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17152 | #define ma_atomic_compare_exchange_strong_i64(dst, expected, replacement)
    > ma_atomic_compare_exchange_strong_explicit_i64(dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17153 | #define ma_atomic_compare_exchange_weak_i8( dst, expected, replacement)
    > ma_atomic_compare_exchange_weak_explicit_i8( dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17154 | #define ma_atomic_compare_exchange_weak_i16(dst, expected, replacement)
    > ma_atomic_compare_exchange_weak_explicit_i16(dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17155 | #define ma_atomic_compare_exchange_weak_i32(dst, expected, replacement)
    > ma_atomic_compare_exchange_weak_explicit_i32(dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17156 | #define ma_atomic_compare_exchange_weak_i64(dst, expected, replacement)
    > ma_atomic_compare_exchange_weak_explicit_i64(dst, expected, replacement,
    > ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17157 | #define ma_atomic_fetch_add_i8( dst, src)
    > ma_atomic_fetch_add_explicit_i8( dst, src, ma_atomic_memory_order_seq_cst)
17158 | #define ma_atomic_fetch_add_i16(dst, src)
    > ma_atomic_fetch_add_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17159 | #define ma_atomic_fetch_add_i32(dst, src)
    > ma_atomic_fetch_add_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17160 | #define ma_atomic_fetch_add_i64(dst, src)
    > ma_atomic_fetch_add_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)
17161 | #define ma_atomic_fetch_sub_i8( dst, src)
    > ma_atomic_fetch_sub_explicit_i8( dst, src, ma_atomic_memory_order_seq_cst)
17162 | #define ma_atomic_fetch_sub_i16(dst, src)
    > ma_atomic_fetch_sub_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17163 | #define ma_atomic_fetch_sub_i32(dst, src)
    > ma_atomic_fetch_sub_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17164 | #define ma_atomic_fetch_sub_i64(dst, src)
    > ma_atomic_fetch_sub_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)
17165 | #define ma_atomic_fetch_or_i8( dst, src)
    > ma_atomic_fetch_or_explicit_i8( dst, src, ma_atomic_memory_order_seq_cst)
17166 | #define ma_atomic_fetch_or_i16(dst, src)
    > ma_atomic_fetch_or_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17167 | #define ma_atomic_fetch_or_i32(dst, src)
    > ma_atomic_fetch_or_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17168 | #define ma_atomic_fetch_or_i64(dst, src)
    > ma_atomic_fetch_or_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)
17169 | #define ma_atomic_fetch_xor_i8( dst, src)
    > ma_atomic_fetch_xor_explicit_i8( dst, src, ma_atomic_memory_order_seq_cst)
17170 | #define ma_atomic_fetch_xor_i16(dst, src)
    > ma_atomic_fetch_xor_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17171 | #define ma_atomic_fetch_xor_i32(dst, src)
    > ma_atomic_fetch_xor_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17172 | #define ma_atomic_fetch_xor_i64(dst, src)
    > ma_atomic_fetch_xor_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)
17173 | #define ma_atomic_fetch_and_i8( dst, src)
    > ma_atomic_fetch_and_explicit_i8( dst, src, ma_atomic_memory_order_seq_cst)
17174 | #define ma_atomic_fetch_and_i16(dst, src)
    > ma_atomic_fetch_and_explicit_i16(dst, src, ma_atomic_memory_order_seq_cst)
17175 | #define ma_atomic_fetch_and_i32(dst, src)
    > ma_atomic_fetch_and_explicit_i32(dst, src, ma_atomic_memory_order_seq_cst)
17176 | #define ma_atomic_fetch_and_i64(dst, src)
    > ma_atomic_fetch_and_explicit_i64(dst, src, ma_atomic_memory_order_seq_cst)

```

```

17177 | #define ma_atomic_compare_and_swap_i8( dst, expected, dedsiored)          (ma_int8
> )ma_atomic_compare_and_swap_8( (ma_uint8*)dst, (ma_uint8)expected, (ma_uint8)dedsiored)
17178 | #define ma_atomic_compare_and_swap_i16(dst, expected, dedsiored)
> (ma_int16)ma_atomic_compare_and_swap_16((ma_uint16*)dst, (ma_uint16)expected, (ma_uint16)dedsiored)
17179 | #define ma_atomic_compare_and_swap_i32(dst, expected, dedsiored)
> (ma_int32)ma_atomic_compare_and_swap_32((ma_uint32*)dst, (ma_uint32)expected, (ma_uint32)dedsiored)
17180 | #define ma_atomic_compare_and_swap_i64(dst, expected, dedsiored)
> (ma_int64)ma_atomic_compare_and_swap_64((ma_uint64*)dst, (ma_uint64)expected, (ma_uint64)dedsiored)
17181 | typedef union
17182 | {
17183 |     ma_uint32 i;
17184 |     float f;
17185 | } ma_atomic_if32;
17186 | typedef union
17187 | {
17188 |     ma_uint64 i;
17189 |     double f;
17190 | } ma_atomic_if64;
17191 | #define ma_atomic_clear_explicit_f32(ptr, order)
> ma_atomic_clear_explicit_32((ma_uint32*)ptr, order)
17192 | #define ma_atomic_clear_explicit_f64(ptr, order)
> ma_atomic_clear_explicit_64((ma_uint64*)ptr, order)
17193 | static MA_INLINE void ma_atomic_store_explicit_f32(volatile float* dst, float src,
> ma_atomic_memory_order order)
17194 | {
17195 |     ma_atomic_if32 x;
17196 |     x.f = src;
17197 |     ma_atomic_store_explicit_32((volatile ma_uint32*)dst, x.i, order);
17198 | }
17199 | static MA_INLINE void ma_atomic_store_explicit_f64(volatile double* dst, double src,
> ma_atomic_memory_order order)
17200 | {
17201 |     ma_atomic_if64 x;
17202 |     x.f = src;
17203 |     ma_atomic_store_explicit_64((volatile ma_uint64*)dst, x.i, order);
17204 | }
17205 | static MA_INLINE float ma_atomic_load_explicit_f32(volatile const float* ptr, ma_atomic_memory_order
> order)
17206 | {
17207 |     ma_atomic_if32 r;
17208 |     r.i = ma_atomic_load_explicit_32((volatile const ma_uint32*)ptr, order);
17209 |     return r.f;
17210 | }
17211 | static MA_INLINE double ma_atomic_load_explicit_f64(volatile const double* ptr,
> ma_atomic_memory_order order)
17212 | {
17213 |     ma_atomic_if64 r;
17214 |     r.i = ma_atomic_load_explicit_64((volatile const ma_uint64*)ptr, order);
17215 |     return r.f;
17216 | }
17217 | static MA_INLINE float ma_atomic_exchange_explicit_f32(volatile float* dst, float src,
> ma_atomic_memory_order order)
17218 | {
17219 |     ma_atomic_if32 r;
17220 |     ma_atomic_if32 x;
17221 |     x.f = src;
17222 |     r.i = ma_atomic_exchange_explicit_32((volatile ma_uint32*)dst, x.i, order);
17223 |     return r.f;
17224 | }
17225 | static MA_INLINE double ma_atomic_exchange_explicit_f64(volatile double* dst, double src,
> ma_atomic_memory_order order)
17226 | {
17227 |     ma_atomic_if64 r;
17228 |     ma_atomic_if64 x;
17229 |     x.f = src;
17230 |     r.i = ma_atomic_exchange_explicit_64((volatile ma_uint64*)dst, x.i, order);
17231 |     return r.f;
17232 | }
17233 | static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_f32(volatile float* dst,
> float* expected, float replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
> failureOrder)
17234 | {
17235 |     ma_atomic_if32 d;
17236 |     d.f = replacement;
17237 |     return ma_atomic_compare_exchange_strong_explicit_32((volatile ma_uint32*)dst,
> (ma_uint32*)expected, d.i, successOrder, failureOrder);
17238 | }
17239 | static MA_INLINE ma_bool32 ma_atomic_compare_exchange_strong_explicit_f64(volatile double* dst,
> double* expected, double replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order

```

```

    > failureOrder)
17240 | {
17241 |     ma_atomic_if64 d;
17242 |     d.f = replacement;
17243 |     return ma_atomic_compare_exchange_strong_explicit_64((volatile ma_uint64*)dst,
    > (ma_uint64*)expected, d.i, successOrder, failureOrder);
17244 | }
17245 | static MA_INLINE ma_bool32 ma_atomic_compare_exchange_weak_explicit_f32(volatile float* dst, float*
    > expected, float replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
    > failureOrder)
17246 | {
17247 |     ma_atomic_if32 d;
17248 |     d.f = replacement;
17249 |     return ma_atomic_compare_exchange_weak_explicit_32((volatile ma_uint32*)dst,
    > (ma_uint32*)expected, d.i, successOrder, failureOrder);
17250 | }
17251 | static MA_INLINE ma_bool32 ma_atomic_compare_exchange_weak_explicit_f64(volatile double* dst,
    > double* expected, double replacement, ma_atomic_memory_order successOrder, ma_atomic_memory_order
    > failureOrder)
17252 | {
17253 |     ma_atomic_if64 d;
17254 |     d.f = replacement;
17255 |     return ma_atomic_compare_exchange_weak_explicit_64((volatile ma_uint64*)dst,
    > (ma_uint64*)expected, d.i, successOrder, failureOrder);
17256 | }
17257 | static MA_INLINE float ma_atomic_fetch_add_explicit_f32(volatile float* dst, float src,
    > ma_atomic_memory_order order)
17258 | {
17259 |     ma_atomic_if32 r;
17260 |     ma_atomic_if32 x;
17261 |     x.f = src;
17262 |     r.i = ma_atomic_fetch_add_explicit_32((volatile ma_uint32*)dst, x.i, order);
17263 |     return r.f;
17264 | }
17265 | static MA_INLINE double ma_atomic_fetch_add_explicit_f64(volatile double* dst, double src,
    > ma_atomic_memory_order order)
17266 | {
17267 |     ma_atomic_if64 r;
17268 |     ma_atomic_if64 x;
17269 |     x.f = src;
17270 |     r.i = ma_atomic_fetch_add_explicit_64((volatile ma_uint64*)dst, x.i, order);
17271 |     return r.f;
17272 | }
17273 | static MA_INLINE float ma_atomic_fetch_sub_explicit_f32(volatile float* dst, float src,
    > ma_atomic_memory_order order)
17274 | {
17275 |     ma_atomic_if32 r;
17276 |     ma_atomic_if32 x;
17277 |     x.f = src;
17278 |     r.i = ma_atomic_fetch_sub_explicit_32((volatile ma_uint32*)dst, x.i, order);
17279 |     return r.f;
17280 | }
17281 | static MA_INLINE double ma_atomic_fetch_sub_explicit_f64(volatile double* dst, double src,
    > ma_atomic_memory_order order)
17282 | {
17283 |     ma_atomic_if64 r;
17284 |     ma_atomic_if64 x;
17285 |     x.f = src;
17286 |     r.i = ma_atomic_fetch_sub_explicit_64((volatile ma_uint64*)dst, x.i, order);
17287 |     return r.f;
17288 | }
17289 | static MA_INLINE float ma_atomic_fetch_or_explicit_f32(volatile float* dst, float src,
    > ma_atomic_memory_order order)
17290 | {
17291 |     ma_atomic_if32 r;
17292 |     ma_atomic_if32 x;
17293 |     x.f = src;
17294 |     r.i = ma_atomic_fetch_or_explicit_32((volatile ma_uint32*)dst, x.i, order);
17295 |     return r.f;
17296 | }
17297 | static MA_INLINE double ma_atomic_fetch_or_explicit_f64(volatile double* dst, double src,
    > ma_atomic_memory_order order)
17298 | {
17299 |     ma_atomic_if64 r;
17300 |     ma_atomic_if64 x;
17301 |     x.f = src;
17302 |     r.i = ma_atomic_fetch_or_explicit_64((volatile ma_uint64*)dst, x.i, order);
17303 |     return r.f;
17304 | }

```



```

17305 | static MA_INLINE float ma_atomic_fetch_xor_explicit_f32(volatile float* dst, float src,
> ma_atomic_memory_order order)
17306 | {
17307 |     ma_atomic_if32 r;
17308 |     ma_atomic_if32 x;
17309 |     x.f = src;
17310 |     r.i = ma_atomic_fetch_xor_explicit_32((volatile ma_uint32*)dst, x.i, order);
17311 |     return r.f;
17312 | }
17313 | static MA_INLINE double ma_atomic_fetch_xor_explicit_f64(volatile double* dst, double src,
> ma_atomic_memory_order order)
17314 | {
17315 |     ma_atomic_if64 r;
17316 |     ma_atomic_if64 x;
17317 |     x.f = src;
17318 |     r.i = ma_atomic_fetch_xor_explicit_64((volatile ma_uint64*)dst, x.i, order);
17319 |     return r.f;
17320 | }
17321 | static MA_INLINE float ma_atomic_fetch_and_explicit_f32(volatile float* dst, float src,
> ma_atomic_memory_order order)
17322 | {
17323 |     ma_atomic_if32 r;
17324 |     ma_atomic_if32 x;
17325 |     x.f = src;
17326 |     r.i = ma_atomic_fetch_and_explicit_32((volatile ma_uint32*)dst, x.i, order);
17327 |     return r.f;
17328 | }
17329 | static MA_INLINE double ma_atomic_fetch_and_explicit_f64(volatile double* dst, double src,
> ma_atomic_memory_order order)
17330 | {
17331 |     ma_atomic_if64 r;
17332 |     ma_atomic_if64 x;
17333 |     x.f = src;
17334 |     r.i = ma_atomic_fetch_and_explicit_64((volatile ma_uint64*)dst, x.i, order);
17335 |     return r.f;
17336 | }
17337 | #define ma_atomic_clear_f32(ptr)                                (float
> )ma_atomic_clear_explicit_f32(ptr, ma_atomic_memory_order_seq_cst)
17338 | #define ma_atomic_clear_f64(ptr)                                (double)ma_atomic_clear_explicit_f64(ptr, ma_atomic_memory_order_seq_cst)
17339 | #define ma_atomic_store_f32(dst, src)                          ma_atomic_store_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17340 | #define ma_atomic_store_f64(dst, src)                          ma_atomic_store_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17341 | #define ma_atomic_load_f32(ptr)                                (float
> )ma_atomic_load_explicit_f32(ptr, ma_atomic_memory_order_seq_cst)
17342 | #define ma_atomic_load_f64(ptr)                                (double)ma_atomic_load_explicit_f64(ptr, ma_atomic_memory_order_seq_cst)
17343 | #define ma_atomic_exchange_f32(dst, src)                      (float
> )ma_atomic_exchange_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17344 | #define ma_atomic_exchange_f64(dst, src)                      (double)ma_atomic_exchange_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17345 | #define ma_atomic_compare_exchange_strong_f32(dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_f32(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17346 | #define ma_atomic_compare_exchange_strong_f64(dst, expected, replacement)
> ma_atomic_compare_exchange_strong_explicit_f64(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17347 | #define ma_atomic_compare_exchange_weak_f32(dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_f32(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17348 | #define ma_atomic_compare_exchange_weak_f64(dst, expected, replacement)
> ma_atomic_compare_exchange_weak_explicit_f64(dst, expected, replacement,
> ma_atomic_memory_order_seq_cst, ma_atomic_memory_order_seq_cst)
17349 | #define ma_atomic_fetch_add_f32(dst, src)                    ma_atomic_fetch_add_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17350 | #define ma_atomic_fetch_add_f64(dst, src)                    ma_atomic_fetch_add_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17351 | #define ma_atomic_fetch_sub_f32(dst, src)                    ma_atomic_fetch_sub_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17352 | #define ma_atomic_fetch_sub_f64(dst, src)                    ma_atomic_fetch_sub_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17353 | #define ma_atomic_fetch_or_f32(dst, src)                     ma_atomic_fetch_or_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17354 | #define ma_atomic_fetch_or_f64(dst, src)                     ma_atomic_fetch_or_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17355 | #define ma_atomic_fetch_xor_f32(dst, src)                    ma_atomic_fetch_xor_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17356 | #define ma_atomic_fetch_xor_f64(dst, src)                    ma_atomic_fetch_xor_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)

```



```

> ma_atomic_fetch_xor_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17357 | #define ma_atomic_fetch_and_f32(dst, src)
> ma_atomic_fetch_and_explicit_f32(dst, src, ma_atomic_memory_order_seq_cst)
17358 | #define ma_atomic_fetch_and_f64(dst, src)
> ma_atomic_fetch_and_explicit_f64(dst, src, ma_atomic_memory_order_seq_cst)
17359 | static MA_INLINE float ma_atomic_compare_and_swap_f32(volatile float* dst, float expected, float
> replacement)
17360 | {
17361 |     ma_atomic_if32 r;
17362 |     ma_atomic_if32 e, d;
17363 |     e.f = expected;
17364 |     d.f = replacement;
17365 |     r.i = ma_atomic_compare_and_swap_32((volatile ma_uint32*)dst, e.i, d.i);
17366 |     return r.f;
17367 | }
17368 | static MA_INLINE double ma_atomic_compare_and_swap_f64(volatile double* dst, double expected, double
> replacement)
17369 | {
17370 |     ma_atomic_if64 r;
17371 |     ma_atomic_if64 e, d;
17372 |     e.f = expected;
17373 |     d.f = replacement;
17374 |     r.i = ma_atomic_compare_and_swap_64((volatile ma_uint64*)dst, e.i, d.i);
17375 |     return r.f;
17376 | }
17377 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
> 6)))
17378 |     #pragma GCC diagnostic pop
17379 | #endif
17380 | #if defined(__cplusplus)
17381 | }
17382 | #endif
17383 | #endif
17384 | /* c89atomic.h end */
17385 |
17386 | #define MA_ATOMIC_SAFE_TYPE_IMPL(c89TypeExtension, type) \
17387 |     static MA_INLINE ma_##type ma_atomic_##type##_get(ma_atomic_##type* x) \
17388 |     { \
17389 |         return (ma_##type)ma_atomic_load_##c89TypeExtension(&x->value); \
17390 |     } \
17391 |     static MA_INLINE void ma_atomic_##type##_set(ma_atomic_##type* x, ma_##type value) \
17392 |     { \
17393 |         ma_atomic_store_##c89TypeExtension(&x->value, value); \
17394 |     } \
17395 |     static MA_INLINE ma_##type ma_atomic_##type##_exchange(ma_atomic_##type* x, ma_##type value) \
17396 |     { \
17397 |         return (ma_##type)ma_atomic_exchange_##c89TypeExtension(&x->value, value); \
17398 |     } \
17399 |     static MA_INLINE ma_bool32 ma_atomic_##type##_compare_exchange(ma_atomic_##type* x, ma_##type*
> expected, ma_##type desired) \
17400 |     { \
17401 |         return ma_atomic_compare_exchange_weak_##c89TypeExtension(&x->value, expected, desired); \
17402 |     } \
17403 |     static MA_INLINE ma_##type ma_atomic_##type##_fetch_add(ma_atomic_##type* x, ma_##type y) \
17404 |     { \
17405 |         return (ma_##type)ma_atomic_fetch_add_##c89TypeExtension(&x->value, y); \
17406 |     } \
17407 |     static MA_INLINE ma_##type ma_atomic_##type##_fetch_sub(ma_atomic_##type* x, ma_##type y) \
17408 |     { \
17409 |         return (ma_##type)ma_atomic_fetch_sub_##c89TypeExtension(&x->value, y); \
17410 |     } \
17411 |     static MA_INLINE ma_##type ma_atomic_##type##_fetch_or(ma_atomic_##type* x, ma_##type y) \
17412 |     { \
17413 |         return (ma_##type)ma_atomic_fetch_or_##c89TypeExtension(&x->value, y); \
17414 |     } \
17415 |     static MA_INLINE ma_##type ma_atomic_##type##_fetch_xor(ma_atomic_##type* x, ma_##type y) \
17416 |     { \
17417 |         return (ma_##type)ma_atomic_fetch_xor_##c89TypeExtension(&x->value, y); \
17418 |     } \
17419 |     static MA_INLINE ma_##type ma_atomic_##type##_fetch_and(ma_atomic_##type* x, ma_##type y) \
17420 |     { \
17421 |         return (ma_##type)ma_atomic_fetch_and_##c89TypeExtension(&x->value, y); \
17422 |     } \
17423 |     static MA_INLINE ma_##type ma_atomic_##type##_compare_and_swap(ma_atomic_##type* x, ma_##type
> expected, ma_##type desired) \
17424 |     { \
17425 |         return (ma_##type)ma_atomic_compare_and_swap_##c89TypeExtension(&x->value, expected,
> desired); \
17426 |     } \

```

```

17427 |
17428 | #define MA_ATOMIC_SAFE_TYPE_IMPL_PTR(type) \
17429 |     static MA_INLINE ma_##type* ma_atomic_ptr_##type##_get(ma_atomic_ptr_##type* x) \
17430 |     { \
17431 |         return ma_atomic_load_ptr((void**)&x->value); \
17432 |     } \
17433 |     static MA_INLINE void ma_atomic_ptr_##type##_set(ma_atomic_ptr_##type* x, ma_##type* value) \
17434 |     { \
17435 |         ma_atomic_store_ptr((void**)&x->value, (void*)value); \
17436 |     } \
17437 |     static MA_INLINE ma_##type* ma_atomic_ptr_##type##_exchange(ma_atomic_ptr_##type* x, ma_##type*
> value) \
17438 |     { \
17439 |         return ma_atomic_exchange_ptr((void**)&x->value, (void*)value); \
17440 |     } \
17441 |     static MA_INLINE ma_bool32 ma_atomic_ptr_##type##_compare_exchange(ma_atomic_ptr_##type* x,
> ma_##type** expected, ma_##type* desired) \
17442 |     { \
17443 |         return ma_atomic_compare_exchange_weak_ptr((void**)&x->value, (void*)expected,
> (void*)desired); \
17444 |     } \
17445 |     static MA_INLINE ma_##type* ma_atomic_ptr_##type##_compare_and_swap(ma_atomic_ptr_##type* x,
> ma_##type* expected, ma_##type* desired) \
17446 |     { \
17447 |         return (ma_##type*)ma_atomic_compare_and_swap_ptr((void**)&x->value, (void*)expected,
> (void*)desired); \
17448 |     } \
17449 |
17450 | MA_ATOMIC_SAFE_TYPE_IMPL(32, uint32)
17451 | MA_ATOMIC_SAFE_TYPE_IMPL(i32, int32)
17452 | MA_ATOMIC_SAFE_TYPE_IMPL(64, uint64)
17453 | MA_ATOMIC_SAFE_TYPE_IMPL(f32, float)
17454 | MA_ATOMIC_SAFE_TYPE_IMPL(32, bool32)
17455 |
17456 | #if !defined(MA_NO_DEVICE_IO)
17457 | MA_ATOMIC_SAFE_TYPE_IMPL(i32, device_state)
17458 | #endif
17459 |
17460 |
17461 | MA_API ma_uint64 ma_calculate_frame_count_after_resampling(ma_uint32 sampleRateOut, ma_uint32
> sampleRateIn, ma_uint64 frameCountIn)
17462 | {
17463 |     /* This is based on the calculation in ma_linear_resampler_get_expected_output_frame_count(). */
17464 |     ma_uint64 outputFrameCount;
17465 |     ma_uint64 preliminaryInputFrameCountFromFrac;
17466 |     ma_uint64 preliminaryInputFrameCount;
17467 |
17468 |     if (sampleRateIn == 0 || sampleRateOut == 0 || frameCountIn == 0) {
17469 |         return 0;
17470 |     }
17471 |
17472 |     if (sampleRateOut == sampleRateIn) {
17473 |         return frameCountIn;
17474 |     }
17475 |
17476 |     outputFrameCount = (frameCountIn * sampleRateOut) / sampleRateIn;
17477 |
17478 |     preliminaryInputFrameCountFromFrac = (outputFrameCount * (sampleRateIn / sampleRateOut)) /
> sampleRateOut;
17479 |     preliminaryInputFrameCount = (outputFrameCount * (sampleRateIn % sampleRateOut)) +
> preliminaryInputFrameCountFromFrac;
17480 |
17481 |     if (preliminaryInputFrameCount <= frameCountIn) {
17482 |         outputFrameCount += 1;
17483 |     }
17484 |
17485 |     return outputFrameCount;
17486 | }
17487 |
17488 | #ifndef MA_DATA_CONVERTER_STACK_BUFFER_SIZE
17489 | #define MA_DATA_CONVERTER_STACK_BUFFER_SIZE    4096
17490 | #endif
17491 |
17492 |
17493 |
17494 | #if defined(MA_WIN32)
17495 | static ma_result ma_result_from_GetLastError(DWORD error)
17496 | {
17497 |     switch (error)

```

```

17498 | {
17499 |     case ERROR_SUCCESS:          return MA_SUCCESS;
17500 |     case ERROR_PATH_NOT_FOUND:   return MA_DOES_NOT_EXIST;
17501 |     case ERROR_TOO_MANY_OPEN_FILES: return MA_TOO_MANY_OPEN_FILES;
17502 |     case ERROR_NOT_ENOUGH_MEMORY: return MA_OUT_OF_MEMORY;
17503 |     case ERROR_DISK_FULL:        return MA_NO_SPACE;
17504 |     case ERROR_HANDLE_EOF:        return MA_AT_END;
17505 |     case ERROR_NEGATIVE_SEEK:     return MA_BAD_SEEK;
17506 |     case ERROR_INVALID_PARAMETER: return MA_INVALID_ARGS;
17507 |     case ERROR_ACCESS_DENIED:     return MA_ACCESS_DENIED;
17508 |     case ERROR_SEM_TIMEOUT:       return MA_TIMEOUT;
17509 |     case ERROR_FILE_NOT_FOUND:    return MA_DOES_NOT_EXIST;
17510 |     default: break;
17511 | }
17512 |
17513 |     return MA_ERROR;
17514 | }
17515 | #endif /* MA_WIN32 */
17516 |
17517 |
17518 | /*****
17519 |
17520 | Threading
17521 |
17522 | *****/
17523 | static MA_INLINE ma_result ma_spinlock_lock_ex(volatile ma_spinlock* pSpinlock, ma_bool32 yield)
17524 | {
17525 |     if (pSpinlock == NULL) {
17526 |         return MA_INVALID_ARGS;
17527 |     }
17528 |
17529 |     for (;;) {
17530 |         if (ma_atomic_exchange_explicit_32(pSpinlock, 1, ma_atomic_memory_order_acquire) == 0) {
17531 |             break;
17532 |         }
17533 |
17534 |         while (ma_atomic_load_explicit_32(pSpinlock, ma_atomic_memory_order_relaxed) == 1) {
17535 |             if (yield) {
17536 |                 ma_yield();
17537 |             }
17538 |         }
17539 |     }
17540 |
17541 |     return MA_SUCCESS;
17542 | }
17543 |
17544 | MA_API ma_result ma_spinlock_lock(volatile ma_spinlock* pSpinlock)
17545 | {
17546 |     return ma_spinlock_lock_ex(pSpinlock, MA_TRUE);
17547 | }
17548 |
17549 | MA_API ma_result ma_spinlock_lock_noyield(volatile ma_spinlock* pSpinlock)
17550 | {
17551 |     return ma_spinlock_lock_ex(pSpinlock, MA_FALSE);
17552 | }
17553 |
17554 | MA_API ma_result ma_spinlock_unlock(volatile ma_spinlock* pSpinlock)
17555 | {
17556 |     if (pSpinlock == NULL) {
17557 |         return MA_INVALID_ARGS;
17558 |     }
17559 |
17560 |     ma_atomic_store_explicit_32(pSpinlock, 0, ma_atomic_memory_order_release);
17561 |     return MA_SUCCESS;
17562 | }
17563 |
17564 |
17565 | #ifndef MA_NO_THREADING
17566 | #if defined(MA_POSIX)
17567 |     #define MA_THREADCALL
17568 |     typedef void* ma_thread_result;
17569 | #elif defined(MA_WIN32)
17570 |     #define MA_THREADCALL WINAPI
17571 |     typedef unsigned long ma_thread_result;
17572 | #endif
17573 |
17574 | typedef ma_thread_result (MA_THREADCALL * ma_thread_entry_proc)(void* pData);
17575 |
17576 | #ifdef MA_POSIX

```

```

17577 | static ma_result ma_thread_create__posix(ma_thread* pThread, ma_thread_priority priority, size_t
> stackSize, ma_thread_entry_proc entryProc, void* pData)
17578 | {
17579 |     int result;
17580 |     pthread_attr_t* pAttr = NULL;
17581 |
17582 |     #if !defined(MA_EMSCRIPTEEN) && !defined(MA_3DS) && !defined(MA_SWITCH)
17583 |     /* Try setting the thread priority. It's not critical if anything fails here. */
17584 |     pthread_attr_t attr;
17585 |     if (pthread_attr_init(&attr) == 0) {
17586 |         int scheduler = -1;
17587 |
17588 |         /* We successfully initialized our attributes object so we can assign the pointer so it's
> passed into pthread_create(). */
17589 |         pAttr = &attr;
17590 |
17591 |         /* We need to set the scheduler policy. Only do this if the OS supports
> pthread_attr_setschedpolicy() */
17592 |         #if !defined(MA_BEOS)
17593 |         {
17594 |             if (priority == ma_thread_priority_idle) {
17595 |                 #ifdef SCHED_IDLE
17596 |                 if (pthread_attr_setschedpolicy(&attr, SCHED_IDLE) == 0) {
17597 |                     scheduler = SCHED_IDLE;
17598 |                 }
17599 |                 #endif
17600 |             } else if (priority == ma_thread_priority_realtime) {
17601 |                 #ifdef SCHED_FIFO
17602 |                 if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) == 0) {
17603 |                     scheduler = SCHED_FIFO;
17604 |                 }
17605 |                 #endif
17606 |                 #ifdef MA_LINUX
17607 |                 } else {
17608 |                     scheduler = sched_getscheduler(0);
17609 |                 }
17610 |             }
17611 |         }
17612 |         #endif
17613 |
17614 |         #if defined(_POSIX_THREAD_ATTR_STACKSIZE) && _POSIX_THREAD_ATTR_STACKSIZE >= 0
17615 |         {
17616 |             if (stackSize > 0) {
17617 |                 pthread_attr_setstacksize(&attr, stackSize);
17618 |             }
17619 |         }
17620 |         #else
17621 |         {
17622 |             (void)stackSize; /* Suppress unused parameter warning. */
17623 |         }
17624 |         #endif
17625 |
17626 |
17627 |         if (scheduler != -1) {
17628 |             int priorityMin = sched_get_priority_min(scheduler);
17629 |             int priorityMax = sched_get_priority_max(scheduler);
17630 |             int priorityStep = (priorityMax - priorityMin) / 7; /* 7 = number of priorities
> supported by miniaudio. */
17631 |
17632 |             struct sched_param sched;
17633 |             if (priorityMin != -1 && priorityMax != -1 && pthread_attr_getschedparam(&attr, &sched)
> == 0) {
17634 |                 if (priority == ma_thread_priority_idle) {
17635 |                     sched.sched_priority = priorityMin;
17636 |                 } else if (priority == ma_thread_priority_realtime) {
17637 |                     #if defined(MA_PTHREAD_REALTIME_THREAD_PRIORITY)
17638 |                     {
17639 |                         sched.sched_priority = MA_PTHREAD_REALTIME_THREAD_PRIORITY;
17640 |                     }
17641 |                     #else
17642 |                     {
17643 |                         sched.sched_priority = priorityMax;
17644 |                     }
17645 |                     #endif
17646 |                 } else {
17647 |                     sched.sched_priority += ((int)priority + 5) * priorityStep; /* +5 because the
> lowest priority is -5. */
17648 |                 }
17649 |             }

```

```

17650 |         if (sched.sched_priority < priorityMin) {
17651 |             sched.sched_priority = priorityMin;
17652 |         }
17653 |         if (sched.sched_priority > priorityMax) {
17654 |             sched.sched_priority = priorityMax;
17655 |         }
17656 |
17657 |         /* I'm not treating a failure of setting the priority as a critical error so not
> aborting on failure here. */
17658 |         if (pthread_attr_setschedparam(&attr, &sched) == 0) {
17659 |             #if !defined(MA_ANDROID) || (defined(__ANDROID_API__) && __ANDROID_API__ >= 28)
17660 |             {
17661 |                 pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
17662 |             }
17663 |             #endif
17664 |         }
17665 |     }
17666 | }
17667 | }
17668 | #else
17669 | /* It's the emscripten build. We'll have a few unused parameters. */
17670 | (void)priority;
17671 | (void)stackSize;
17672 | #endif
17673 |
17674 | result = pthread_create((pthread_t*)pThread, pAttr, entryProc, pData);
17675 |
17676 | /* The thread attributes object is no longer required. */
17677 | if (pAttr != NULL) {
17678 |     pthread_attr_destroy(pAttr);
17679 | }
17680 |
17681 | if (result != 0) {
17682 |     /*
17683 |     > This case,
17684 |     fall back to a normal priority thread.
17685 |
17686 |     I'm including a compile-time option here to disable this functionality for those who have a
> hard
17687 |     requirement on realtime threads and would rather an explicit failure.
17688 |     */
17689 |     #ifndef MA_NO_PTHREAD_REALTIME_PRIORITY_FALLBACK
17690 |     {
17691 |         if (result == EPERM && priority == ma_thread_priority_realtime) {
17692 |             > return ma_thread_create__posix(pThread, ma_thread_priority_normal, stackSize,
entryProc, pData);
17693 |         }
17694 |     }
17695 |     #endif
17696 |
17697 |     return ma_result_from_errno(result);
17698 | }
17699 |
17700 | return MA_SUCCESS;
17701 | }
17702 |
17703 | static void ma_thread_wait__posix(ma_thread* pThread)
17704 | {
17705 |     pthread_join((pthread_t*)pThread, NULL);
17706 | }
17707 |
17708 |
17709 | static ma_result ma_mutex_init__posix(ma_mutex* pMutex)
17710 | {
17711 |     int result;
17712 |
17713 |     if (pMutex == NULL) {
17714 |         return MA_INVALID_ARGS;
17715 |     }
17716 |
17717 |     MA_ZERO_OBJECT(pMutex);
17718 |
17719 |     result = pthread_mutex_init((pthread_mutex_t*)pMutex, NULL);
17720 |     if (result != 0) {
17721 |         return ma_result_from_errno(result);
17722 |     }
17723 | }
17724 |
17724 | return MA_SUCCESS;

```

```

17725 | }
17726 |
17727 | static void ma_mutex_uninit__posix(ma_mutex* pMutex)
17728 | {
17729 |     pthread_mutex_destroy((pthread_mutex_t*)pMutex);
17730 | }
17731 |
17732 | static void ma_mutex_lock__posix(ma_mutex* pMutex)
17733 | {
17734 |     pthread_mutex_lock((pthread_mutex_t*)pMutex);
17735 | }
17736 |
17737 | static void ma_mutex_unlock__posix(ma_mutex* pMutex)
17738 | {
17739 |     pthread_mutex_unlock((pthread_mutex_t*)pMutex);
17740 | }
17741 |
17742 |
17743 | static ma_result ma_event_init__posix(ma_event* pEvent)
17744 | {
17745 |     int result;
17746 |
17747 |     result = pthread_mutex_init((pthread_mutex_t*)&pEvent->lock, NULL);
17748 |     if (result != 0) {
17749 |         return ma_result_from_errno(result);
17750 |     }
17751 |
17752 |     result = pthread_cond_init((pthread_cond_t*)&pEvent->cond, NULL);
17753 |     if (result != 0) {
17754 |         pthread_mutex_destroy((pthread_mutex_t*)&pEvent->lock);
17755 |         return ma_result_from_errno(result);
17756 |     }
17757 |
17758 |     pEvent->value = 0;
17759 |     return MA_SUCCESS;
17760 | }
17761 |
17762 | static void ma_event_uninit__posix(ma_event* pEvent)
17763 | {
17764 |     pthread_cond_destroy((pthread_cond_t*)&pEvent->cond);
17765 |     pthread_mutex_destroy((pthread_mutex_t*)&pEvent->lock);
17766 | }
17767 |
17768 | static ma_result ma_event_wait__posix(ma_event* pEvent)
17769 | {
17770 |     pthread_mutex_lock((pthread_mutex_t*)&pEvent->lock);
17771 |     {
17772 |         while (pEvent->value == 0) {
17773 |             pthread_cond_wait((pthread_cond_t*)&pEvent->cond, (pthread_mutex_t*)&pEvent->lock);
17774 |         }
17775 |         pEvent->value = 0; /* Auto-reset. */
17776 |     }
17777 |     pthread_mutex_unlock((pthread_mutex_t*)&pEvent->lock);
17778 |
17779 |     return MA_SUCCESS;
17780 | }
17781 |
17782 | static ma_result ma_event_signal__posix(ma_event* pEvent)
17783 | {
17784 |     pthread_mutex_lock((pthread_mutex_t*)&pEvent->lock);
17785 |     {
17786 |         pEvent->value = 1;
17787 |         pthread_cond_signal((pthread_cond_t*)&pEvent->cond);
17788 |     }
17789 |     pthread_mutex_unlock((pthread_mutex_t*)&pEvent->lock);
17790 |
17791 |     return MA_SUCCESS;
17792 | }
17793 |
17794 |
17795 | static ma_result ma_semaphore_init__posix(int initialValue, ma_semaphore* pSemaphore)
17796 | {
17797 |     int result;
17798 |
17799 |     if (pSemaphore == NULL) {
17800 |         return MA_INVALID_ARGS;
17801 |     }
17802 |
17803 |     pSemaphore->value = initialValue;

```

```

17804 |
17805 |     result = pthread_mutex_init((pthread_mutex_t*)&pSemaphore->lock, NULL);
17806 |     if (result != 0) {
17807 |         return ma_result_from_errno(result); /* Failed to create mutex. */
17808 |     }
17809 |
17810 |     result = pthread_cond_init((pthread_cond_t*)&pSemaphore->cond, NULL);
17811 |     if (result != 0) {
17812 |         pthread_mutex_destroy((pthread_mutex_t*)&pSemaphore->lock);
17813 |         return ma_result_from_errno(result); /* Failed to create condition variable. */
17814 |     }
17815 |
17816 |     return MA_SUCCESS;
17817 | }
17818 |
17819 | static void ma_semaphore_uninit__posix(ma_semaphore* pSemaphore)
17820 | {
17821 |     if (pSemaphore == NULL) {
17822 |         return;
17823 |     }
17824 |
17825 |     pthread_cond_destroy((pthread_cond_t*)&pSemaphore->cond);
17826 |     pthread_mutex_destroy((pthread_mutex_t*)&pSemaphore->lock);
17827 | }
17828 |
17829 | static ma_result ma_semaphore_wait__posix(ma_semaphore* pSemaphore)
17830 | {
17831 |     if (pSemaphore == NULL) {
17832 |         return MA_INVALID_ARGS;
17833 |     }
17834 |
17835 |     pthread_mutex_lock((pthread_mutex_t*)&pSemaphore->lock);
17836 |     {
17837 |         /* We need to wait on a condition variable before escaping. We can't return from this
17838 | > function until the semaphore has been signaled. */
17839 |         while (pSemaphore->value == 0) {
17840 |             pthread_cond_wait((pthread_cond_t*)&pSemaphore->cond,
17841 | > (pthread_mutex_t*)&pSemaphore->lock);
17842 |         }
17843 |
17844 |         pSemaphore->value -= 1;
17845 |     }
17846 |     pthread_mutex_unlock((pthread_mutex_t*)&pSemaphore->lock);
17847 |
17848 |     return MA_SUCCESS;
17849 | }
17850 |
17851 | static ma_result ma_semaphore_release__posix(ma_semaphore* pSemaphore)
17852 | {
17853 |     if (pSemaphore == NULL) {
17854 |         return MA_INVALID_ARGS;
17855 |     }
17856 |
17857 |     pthread_mutex_lock((pthread_mutex_t*)&pSemaphore->lock);
17858 |     {
17859 |         pSemaphore->value += 1;
17860 |         pthread_cond_signal((pthread_cond_t*)&pSemaphore->cond);
17861 |     }
17862 |     pthread_mutex_unlock((pthread_mutex_t*)&pSemaphore->lock);
17863 |
17864 |     return MA_SUCCESS;
17865 | }
17866 | #elif defined(MA_WIN32)
17867 | static int ma_thread_priority_to_win32(ma_thread_priority priority)
17868 | {
17869 |     switch (priority) {
17870 |         case ma_thread_priority_idle:      return THREAD_PRIORITY_IDLE;
17871 |         case ma_thread_priority_lowest:    return THREAD_PRIORITY_LOWEST;
17872 |         case ma_thread_priority_low:       return THREAD_PRIORITY_BELOW_NORMAL;
17873 |         case ma_thread_priority_normal:    return THREAD_PRIORITY_NORMAL;
17874 |         case ma_thread_priority_high:      return THREAD_PRIORITY_ABOVE_NORMAL;
17875 |         case ma_thread_priority_highest:   return THREAD_PRIORITY_HIGHEST;
17876 |         case ma_thread_priority_realtime:  return THREAD_PRIORITY_TIME_CRITICAL;
17877 |         default:                           return THREAD_PRIORITY_NORMAL;
17878 |     }
17879 | }
17880 |
17881 | static ma_result ma_thread_create__win32(ma_thread* pThread, ma_thread_priority priority, size_t
17882 | > stackSize, ma_thread_entry_proc entryProc, void* pData)

```



```

17880 | {
17881 |     DWORD threadID; /* Not used. Only used for passing into CreateThread() so it doesn't fail on
> Windows 98. */
17882 |
17883 |     *pThread = CreateThread(NULL, stackSize, entryProc, pData, 0, &threadID);
17884 |     if (*pThread == NULL) {
17885 |         return ma_result_from_GetLastError(GetLastError());
17886 |     }
17887 |
17888 |     SetThreadPriority((HANDLE)*pThread, ma_thread_priority_to_win32(priority));
17889 |
17890 |     return MA_SUCCESS;
17891 | }
17892 |
17893 | static void ma_thread_wait__win32(ma_thread* pThread)
17894 | {
17895 |     WaitForSingleObject((HANDLE)*pThread, INFINITE);
17896 |     CloseHandle((HANDLE)*pThread);
17897 | }
17898 |
17899 |
17900 | static ma_result ma_mutex_init__win32(ma_mutex* pMutex)
17901 | {
17902 |     *pMutex = CreateEventA(NULL, FALSE, TRUE, NULL);
17903 |     if (*pMutex == NULL) {
17904 |         return ma_result_from_GetLastError(GetLastError());
17905 |     }
17906 |
17907 |     return MA_SUCCESS;
17908 | }
17909 |
17910 | static void ma_mutex_uninit__win32(ma_mutex* pMutex)
17911 | {
17912 |     CloseHandle((HANDLE)*pMutex);
17913 | }
17914 |
17915 | static void ma_mutex_lock__win32(ma_mutex* pMutex)
17916 | {
17917 |     WaitForSingleObject((HANDLE)*pMutex, INFINITE);
17918 | }
17919 |
17920 | static void ma_mutex_unlock__win32(ma_mutex* pMutex)
17921 | {
17922 |     SetEvent((HANDLE)*pMutex);
17923 | }
17924 |
17925 |
17926 | static ma_result ma_event_init__win32(ma_event* pEvent)
17927 | {
17928 |     *pEvent = CreateEventA(NULL, FALSE, FALSE, NULL);
17929 |     if (*pEvent == NULL) {
17930 |         return ma_result_from_GetLastError(GetLastError());
17931 |     }
17932 |
17933 |     return MA_SUCCESS;
17934 | }
17935 |
17936 | static void ma_event_uninit__win32(ma_event* pEvent)
17937 | {
17938 |     CloseHandle((HANDLE)*pEvent);
17939 | }
17940 |
17941 | static ma_result ma_event_wait__win32(ma_event* pEvent)
17942 | {
17943 |     DWORD result = WaitForSingleObject((HANDLE)*pEvent, INFINITE);
17944 |     if (result == WAIT_OBJECT_0) {
17945 |         return MA_SUCCESS;
17946 |     }
17947 |
17948 |     if (result == WAIT_TIMEOUT) {
17949 |         return MA_TIMEOUT;
17950 |     }
17951 |
17952 |     return ma_result_from_GetLastError(GetLastError());
17953 | }
17954 |
17955 | static ma_result ma_event_signal__win32(ma_event* pEvent)
17956 | {
17957 |     BOOL result = SetEvent((HANDLE)*pEvent);

```

```

17958 |     if (result == 0) {
17959 |         return ma_result_from_GetLastError(GetLastError());
17960 |     }
17961 |
17962 |     return MA_SUCCESS;
17963 | }
17964 |
17965 |
17966 | static ma_result ma_semaphore_init_win32(int initialValue, ma_semaphore* pSemaphore)
17967 | {
17968 |     *pSemaphore = CreateSemaphore(NULL, (LONG)initialValue, LONG_MAX, NULL);
17969 |     if (*pSemaphore == NULL) {
17970 |         return ma_result_from_GetLastError(GetLastError());
17971 |     }
17972 |
17973 |     return MA_SUCCESS;
17974 | }
17975 |
17976 | static void ma_semaphore_uninit_win32(ma_semaphore* pSemaphore)
17977 | {
17978 |     CloseHandle((HANDLE)*pSemaphore);
17979 | }
17980 |
17981 | static ma_result ma_semaphore_wait_win32(ma_semaphore* pSemaphore)
17982 | {
17983 |     DWORD result = WaitForSingleObject((HANDLE)*pSemaphore, INFINITE);
17984 |     if (result == WAIT_OBJECT_0) {
17985 |         return MA_SUCCESS;
17986 |     }
17987 |
17988 |     if (result == WAIT_TIMEOUT) {
17989 |         return MA_TIMEOUT;
17990 |     }
17991 |
17992 |     return ma_result_from_GetLastError(GetLastError());
17993 | }
17994 |
17995 | static ma_result ma_semaphore_release_win32(ma_semaphore* pSemaphore)
17996 | {
17997 |     BOOL result = ReleaseSemaphore((HANDLE)*pSemaphore, 1, NULL);
17998 |     if (result == 0) {
17999 |         return ma_result_from_GetLastError(GetLastError());
18000 |     }
18001 |
18002 |     return MA_SUCCESS;
18003 | }
18004 | #endif
18005 |
18006 | typedef struct
18007 | {
18008 |     ma_thread_entry_proc entryProc;
18009 |     void* pData;
18010 |     ma_allocation_callbacks allocationCallbacks;
18011 | } ma_thread_proxy_data;
18012 |
18013 | static ma_thread_result MA_THREADCALL ma_thread_entry_proxy(void* pData)
18014 | {
18015 |     ma_thread_proxy_data* pProxyData = (ma_thread_proxy_data*)pData;
18016 |     ma_thread_entry_proc entryProc;
18017 |     void* pEntryProcData;
18018 |     ma_thread_result result;
18019 |
18020 |     #if defined(MA_ON_THREAD_ENTRY)
18021 |         MA_ON_THREAD_ENTRY
18022 |     #endif
18023 |
18024 |     entryProc = pProxyData->entryProc;
18025 |     pEntryProcData = pProxyData->pData;
18026 |
18027 |     /* Free the proxy data before getting into the real thread entry proc. */
18028 |     ma_free(pProxyData, &pProxyData->allocationCallbacks);
18029 |
18030 |     result = entryProc(pEntryProcData);
18031 |
18032 |     #if defined(MA_ON_THREAD_EXIT)
18033 |         MA_ON_THREAD_EXIT
18034 |     #endif
18035 |
18036 |     return result;

```

```

18037 | }
18038 |
18039 | static ma_result ma_thread_create(ma_thread* pThread, ma_thread_priority priority, size_t stackSize,
> ma_thread_entry_proc entryProc, void* pData, const ma_allocation_callbacks* pAllocationCallbacks)
18040 | {
18041 |     ma_result result;
18042 |     ma_thread_proxy_data* pProxyData;
18043 |
18044 |     if (pThread == NULL || entryProc == NULL) {
18045 |         return MA_INVALID_ARGS;
18046 |     }
18047 |
18048 |     pProxyData = (ma_thread_proxy_data*)ma_malloc(sizeof(*pProxyData), pAllocationCallbacks); /*
> Will be freed by the proxy entry proc. */
18049 |     if (pProxyData == NULL) {
18050 |         return MA_OUT_OF_MEMORY;
18051 |     }
18052 |
18053 |     #if defined(MA_THREAD_DEFAULT_STACK_SIZE)
18054 |         if (stackSize == 0) {
18055 |             stackSize = MA_THREAD_DEFAULT_STACK_SIZE;
18056 |         }
18057 |     #endif
18058 |
18059 |     pProxyData->entryProc = entryProc;
18060 |     pProxyData->pData      = pData;
18061 |     ma_allocation_callbacks_init_copy(&pProxyData->allocationCallbacks, pAllocationCallbacks);
18062 |
18063 |     #if defined(MA_POSIX)
18064 |         result = ma_thread_create__posix(pThread, priority, stackSize, ma_thread_entry_proxy,
> pProxyData);
18065 |     #elif defined(MA_WIN32)
18066 |         result = ma_thread_create__win32(pThread, priority, stackSize, ma_thread_entry_proxy,
> pProxyData);
18067 |     #endif
18068 |
18069 |     if (result != MA_SUCCESS) {
18070 |         ma_free(pProxyData, pAllocationCallbacks);
18071 |         return result;
18072 |     }
18073 |
18074 |     return MA_SUCCESS;
18075 | }
18076 |
18077 | static void ma_thread_wait(ma_thread* pThread)
18078 | {
18079 |     if (pThread == NULL) {
18080 |         return;
18081 |     }
18082 |
18083 |     #if defined(MA_POSIX)
18084 |         ma_thread_wait__posix(pThread);
18085 |     #elif defined(MA_WIN32)
18086 |         ma_thread_wait__win32(pThread);
18087 |     #endif
18088 | }
18089 |
18090 |
18091 | MA_API ma_result ma_mutex_init(ma_mutex* pMutex)
18092 | {
18093 |     if (pMutex == NULL) {
18094 |         MA_ASSERT(MA_FALSE); /* Fire an assert so the caller is aware of this bug. */
18095 |         return MA_INVALID_ARGS;
18096 |     }
18097 |
18098 |     #if defined(MA_POSIX)
18099 |         return ma_mutex_init__posix(pMutex);
18100 |     #elif defined(MA_WIN32)
18101 |         return ma_mutex_init__win32(pMutex);
18102 |     #endif
18103 | }
18104 |
18105 | MA_API void ma_mutex_uninit(ma_mutex* pMutex)
18106 | {
18107 |     if (pMutex == NULL) {
18108 |         return;
18109 |     }
18110 |
18111 |     #if defined(MA_POSIX)

```

```

18112 |     ma_mutex_uninit__posix(pMutex);
18113 | #elif defined(MA_WIN32)
18114 |     ma_mutex_uninit__win32(pMutex);
18115 | #endif
18116 | }
18117 |
18118 | MA_API void ma_mutex_lock(ma_mutex* pMutex)
18119 | {
18120 |     if (pMutex == NULL) {
18121 |         MA_ASSERT(MA_FALSE);    /* Fire an assert so the caller is aware of this bug. */
18122 |         return;
18123 |     }
18124 |
18125 | #if defined(MA_POSIX)
18126 |     ma_mutex_lock__posix(pMutex);
18127 | #elif defined(MA_WIN32)
18128 |     ma_mutex_lock__win32(pMutex);
18129 | #endif
18130 | }
18131 |
18132 | MA_API void ma_mutex_unlock(ma_mutex* pMutex)
18133 | {
18134 |     if (pMutex == NULL) {
18135 |         MA_ASSERT(MA_FALSE);    /* Fire an assert so the caller is aware of this bug. */
18136 |         return;
18137 |     }
18138 |
18139 | #if defined(MA_POSIX)
18140 |     ma_mutex_unlock__posix(pMutex);
18141 | #elif defined(MA_WIN32)
18142 |     ma_mutex_unlock__win32(pMutex);
18143 | #endif
18144 | }
18145 |
18146 |
18147 | MA_API ma_result ma_event_init(ma_event* pEvent)
18148 | {
18149 |     if (pEvent == NULL) {
18150 |         MA_ASSERT(MA_FALSE);    /* Fire an assert so the caller is aware of this bug. */
18151 |         return MA_INVALID_ARGS;
18152 |     }
18153 |
18154 | #if defined(MA_POSIX)
18155 |     return ma_event_init__posix(pEvent);
18156 | #elif defined(MA_WIN32)
18157 |     return ma_event_init__win32(pEvent);
18158 | #endif
18159 | }
18160 |
18161 | #if 0
18162 | static ma_result ma_event_alloc_and_init(ma_event** ppEvent, ma_allocation_callbacks*
> pAllocationCallbacks)
18163 | {
18164 |     ma_result result;
18165 |     ma_event* pEvent;
18166 |
18167 |     if (ppEvent == NULL) {
18168 |         return MA_INVALID_ARGS;
18169 |     }
18170 |
18171 |     *ppEvent = NULL;
18172 |
18173 |     pEvent = ma_malloc(sizeof(*pEvent), pAllocationCallbacks);
18174 |     if (pEvent == NULL) {
18175 |         return MA_OUT_OF_MEMORY;
18176 |     }
18177 |
18178 |     result = ma_event_init(pEvent);
18179 |     if (result != MA_SUCCESS) {
18180 |         ma_free(pEvent, pAllocationCallbacks);
18181 |         return result;
18182 |     }
18183 |
18184 |     *ppEvent = pEvent;
18185 |     return result;
18186 | }
18187 | #endif
18188 |
18189 | MA_API void ma_event_uninit(ma_event* pEvent)

```

```

18190 | {
18191 |     if (pEvent == NULL) {
18192 |         return;
18193 |     }
18194 |
18195 |     #if defined(MA_POSIX)
18196 |         ma_event_uninit__posix(pEvent);
18197 |     #elif defined(MA_WIN32)
18198 |         ma_event_uninit__win32(pEvent);
18199 |     #endif
18200 | }
18201 |
18202 | #if 0
18203 | static void ma_event_uninit_and_free(ma_event* pEvent, ma_allocation_callbacks*
> pAllocationCallbacks)
18204 | {
18205 |     if (pEvent == NULL) {
18206 |         return;
18207 |     }
18208 |
18209 |     ma_event_uninit(pEvent);
18210 |     ma_free(pEvent, pAllocationCallbacks);
18211 | }
18212 | #endif
18213 |
18214 | MA_API ma_result ma_event_wait(ma_event* pEvent)
18215 | {
18216 |     if (pEvent == NULL) {
18217 |         MA_ASSERT(MA_FALSE); /* Fire an assert to the caller is aware of this bug. */
18218 |         return MA_INVALID_ARGS;
18219 |     }
18220 |
18221 |     #if defined(MA_POSIX)
18222 |         return ma_event_wait__posix(pEvent);
18223 |     #elif defined(MA_WIN32)
18224 |         return ma_event_wait__win32(pEvent);
18225 |     #endif
18226 | }
18227 |
18228 | MA_API ma_result ma_event_signal(ma_event* pEvent)
18229 | {
18230 |     if (pEvent == NULL) {
18231 |         MA_ASSERT(MA_FALSE); /* Fire an assert to the caller is aware of this bug. */
18232 |         return MA_INVALID_ARGS;
18233 |     }
18234 |
18235 |     #if defined(MA_POSIX)
18236 |         return ma_event_signal__posix(pEvent);
18237 |     #elif defined(MA_WIN32)
18238 |         return ma_event_signal__win32(pEvent);
18239 |     #endif
18240 | }
18241 |
18242 |
18243 | MA_API ma_result ma_semaphore_init(int initialValue, ma_semaphore* pSemaphore)
18244 | {
18245 |     if (pSemaphore == NULL) {
18246 |         MA_ASSERT(MA_FALSE); /* Fire an assert so the caller is aware of this bug. */
18247 |         return MA_INVALID_ARGS;
18248 |     }
18249 |
18250 |     #if defined(MA_POSIX)
18251 |         return ma_semaphore_init__posix(initialValue, pSemaphore);
18252 |     #elif defined(MA_WIN32)
18253 |         return ma_semaphore_init__win32(initialValue, pSemaphore);
18254 |     #endif
18255 | }
18256 |
18257 | MA_API void ma_semaphore_uninit(ma_semaphore* pSemaphore)
18258 | {
18259 |     if (pSemaphore == NULL) {
18260 |         MA_ASSERT(MA_FALSE); /* Fire an assert so the caller is aware of this bug. */
18261 |         return;
18262 |     }
18263 |
18264 |     #if defined(MA_POSIX)
18265 |         ma_semaphore_uninit__posix(pSemaphore);
18266 |     #elif defined(MA_WIN32)
18267 |         ma_semaphore_uninit__win32(pSemaphore);

```

```

18268 | #endif
18269 | }
18270 |
18271 | MA_API ma_result ma_semaphore_wait(ma_semaphore* pSemaphore)
18272 | {
18273 |     if (pSemaphore == NULL) {
18274 |         MA_ASSERT(MA_FALSE); /* Fire an assert so the caller is aware of this bug. */
18275 |         return MA_INVALID_ARGS;
18276 |     }
18277 |
18278 | #if defined(MA_POSIX)
18279 |     return ma_semaphore_wait__posix(pSemaphore);
18280 | #elif defined(MA_WIN32)
18281 |     return ma_semaphore_wait__win32(pSemaphore);
18282 | #endif
18283 | }
18284 |
18285 | MA_API ma_result ma_semaphore_release(ma_semaphore* pSemaphore)
18286 | {
18287 |     if (pSemaphore == NULL) {
18288 |         MA_ASSERT(MA_FALSE); /* Fire an assert so the caller is aware of this bug. */
18289 |         return MA_INVALID_ARGS;
18290 |     }
18291 |
18292 | #if defined(MA_POSIX)
18293 |     return ma_semaphore_release__posix(pSemaphore);
18294 | #elif defined(MA_WIN32)
18295 |     return ma_semaphore_release__win32(pSemaphore);
18296 | #endif
18297 | }
18298 | #else
18299 | /* MA_NO_THREADING is set which means threading is disabled. Threading is required by some API
> families. If any of these are enabled we need to throw an error. */
18300 | #ifndef MA_NO_DEVICE_IO
18301 | #error "MA_NO_THREADING cannot be used without MA_NO_DEVICE_IO";
18302 | #endif
18303 | #endif /* MA_NO_THREADING */
18304 |
18305 |
18306 |
18307 | #define MA_FENCE_COUNTER_MAX    0x7FFFFFFF
18308 |
18309 | MA_API ma_result ma_fence_init(ma_fence* pFence)
18310 | {
18311 |     if (pFence == NULL) {
18312 |         return MA_INVALID_ARGS;
18313 |     }
18314 |
18315 |     MA_ZERO_OBJECT(pFence);
18316 |     pFence->counter = 0;
18317 |
18318 | #ifndef MA_NO_THREADING
18319 |     {
18320 |         ma_result result;
18321 |
18322 |         result = ma_event_init(&pFence->e);
18323 |         if (result != MA_SUCCESS) {
18324 |             return result;
18325 |         }
18326 |     }
18327 | #endif
18328 |
18329 |     return MA_SUCCESS;
18330 | }
18331 |
18332 | MA_API void ma_fence_uninit(ma_fence* pFence)
18333 | {
18334 |     if (pFence == NULL) {
18335 |         return;
18336 |     }
18337 |
18338 | #ifndef MA_NO_THREADING
18339 |     {
18340 |         ma_event_uninit(&pFence->e);
18341 |     }
18342 | #endif
18343 |
18344 |     MA_ZERO_OBJECT(pFence);
18345 | }

```

```

18346 |
18347 | MA_API ma_result ma_fence_acquire(ma_fence* pFence)
18348 | {
18349 |     if (pFence == NULL) {
18350 |         return MA_INVALID_ARGS;
18351 |     }
18352 |
18353 |     for (;;) {
18354 |         ma_uint32 oldCounter = ma_atomic_load_32(&pFence->counter);
18355 |         ma_uint32 newCounter = oldCounter + 1;
18356 |
18357 |         /* Make sure we're not about to exceed our maximum value. */
18358 |         if (newCounter > MA_FENCE_COUNTER_MAX) {
18359 |             MA_ASSERT(MA_FALSE);
18360 |             return MA_OUT_OF_RANGE;
18361 |         }
18362 |
18363 |         if (ma_atomic_compare_exchange_weak_32(&pFence->counter, &oldCounter, newCounter)) {
18364 |             return MA_SUCCESS;
18365 |         } else {
18366 |             if (oldCounter == MA_FENCE_COUNTER_MAX) {
18367 |                 MA_ASSERT(MA_FALSE);
18368 |                 return MA_OUT_OF_RANGE; /* The other thread took the last available slot. Abort. */
18369 |             }
18370 |         }
18371 |     }
18372 |
18373 |     /* Should never get here. */
18374 |     /*return MA_SUCCESS;*/
18375 | }
18376 |
18377 | MA_API ma_result ma_fence_release(ma_fence* pFence)
18378 | {
18379 |     if (pFence == NULL) {
18380 |         return MA_INVALID_ARGS;
18381 |     }
18382 |
18383 |     for (;;) {
18384 |         ma_uint32 oldCounter = ma_atomic_load_32(&pFence->counter);
18385 |         ma_uint32 newCounter = oldCounter - 1;
18386 |
18387 |         if (oldCounter == 0) {
18388 |             MA_ASSERT(MA_FALSE);
18389 |             return MA_INVALID_OPERATION; /* Acquire/release mismatch. */
18390 |         }
18391 |
18392 |         if (ma_atomic_compare_exchange_weak_32(&pFence->counter, &oldCounter, newCounter)) {
18393 |             #ifndef MA_NO_THREADING
18394 |             {
18395 |                 if (newCounter == 0) {
18396 |                     ma_event_signal(&pFence->e); /* <-- ma_fence_wait() will be waiting on this.
> */
18397 |                 }
18398 |             }
18399 |             #endif
18400 |
18401 |             return MA_SUCCESS;
18402 |         } else {
18403 |             if (oldCounter == 0) {
18404 |                 MA_ASSERT(MA_FALSE);
18405 |                 return MA_INVALID_OPERATION; /* Another thread has taken the 0 slot.
> Acquire/release mismatch. */
18406 |             }
18407 |         }
18408 |     }
18409 |
18410 |     /* Should never get here. */
18411 |     /*return MA_SUCCESS;*/
18412 | }
18413 |
18414 | MA_API ma_result ma_fence_wait(ma_fence* pFence)
18415 | {
18416 |     if (pFence == NULL) {
18417 |         return MA_INVALID_ARGS;
18418 |     }
18419 |
18420 |     for (;;) {
18421 |         ma_uint32 counter;
18422 |

```



```

18423 |         counter = ma_atomic_load_32(&pFence->counter);
18424 |         if (counter == 0) {
18425 |             /*
18426 |              * Counter has hit zero. By the time we get here some other thread may have acquired the
18427 |              * fence again, but that is where the caller needs to take care with how they see the fence.
18428 |              */
18429 |             return MA_SUCCESS;
18430 |         }
18431 |
18432 |         /* Getting here means the counter is > 0. We'll need to wait for something to happen. */
18433 |         #ifndef MA_NO_THREADING
18434 |         {
18435 |             ma_result result;
18436 |
18437 |             result = ma_event_wait(&pFence->e);
18438 |             if (result != MA_SUCCESS) {
18439 |                 return result;
18440 |             }
18441 |         }
18442 |         #endif
18443 |     }
18444 |
18445 |     /* Should never get here. */
18446 |     /*return MA_INVALID_OPERATION;*/
18447 | }
18448 |
18449 |
18450 | MA_API ma_result ma_async_notification_signal(ma_async_notification* pNotification)
18451 | {
18452 |     ma_async_notification_callbacks* pNotificationCallbacks =
18453 | > (ma_async_notification_callbacks*)pNotification;
18454 |
18455 |     if (pNotification == NULL) {
18456 |         return MA_INVALID_ARGS;
18457 |     }
18458 |
18459 |     if (pNotificationCallbacks->onSignal == NULL) {
18460 |         return MA_NOT_IMPLEMENTED;
18461 |     }
18462 |
18463 |     pNotificationCallbacks->onSignal(pNotification);
18464 |     return MA_INVALID_ARGS;
18465 | }
18466 |
18467 | static void ma_async_notification_poll__on_signal(ma_async_notification* pNotification)
18468 | {
18469 |     ((ma_async_notification_poll*)pNotification)->signalled = MA_TRUE;
18470 | }
18471 |
18472 | MA_API ma_result ma_async_notification_poll_init(ma_async_notification_poll* pNotificationPoll)
18473 | {
18474 |     if (pNotificationPoll == NULL) {
18475 |         return MA_INVALID_ARGS;
18476 |     }
18477 |
18478 |     pNotificationPoll->cb.onSignal = ma_async_notification_poll__on_signal;
18479 |     pNotificationPoll->signalled = MA_FALSE;
18480 |
18481 |     return MA_SUCCESS;
18482 | }
18483 |
18484 | MA_API ma_bool32 ma_async_notification_poll_is_signalled(const ma_async_notification_poll*
18485 | > pNotificationPoll)
18486 | {
18487 |     if (pNotificationPoll == NULL) {
18488 |         return MA_FALSE;
18489 |     }
18490 |
18491 |     return pNotificationPoll->signalled;
18492 | }
18493 |
18494 | static void ma_async_notification_event__on_signal(ma_async_notification* pNotification)
18495 | {
18496 |     ma_async_notification_event_signal((ma_async_notification_event*)pNotification);
18497 | }
18498 |
18499 | MA_API ma_result ma_async_notification_event_init(ma_async_notification_event* pNotificationEvent)

```

```
18500 | {
18501 |     if (pNotificationEvent == NULL) {
18502 |         return MA_INVALID_ARGS;
18503 |     }
18504 |
18505 |     pNotificationEvent->cb.onSignal = ma_async_notification_event__on_signal;
18506 |
18507 |     #ifndef MA_NO_THREADING
18508 |     {
18509 |         ma_result result;
18510 |
18511 |         result = ma_event_init(&pNotificationEvent->e);
18512 |         if (result != MA_SUCCESS) {
18513 |             return result;
18514 |         }
18515 |
18516 |         return MA_SUCCESS;
18517 |     }
18518 |     #else
18519 |     {
18520 |         return MA_NOT_IMPLEMENTED; /* Threading is disabled. */
18521 |     }
18522 |     #endif
18523 | }
18524 |
18525 | MA_API ma_result ma_async_notification_event_uninit(ma_async_notification_event* pNotificationEvent)
18526 | {
18527 |     if (pNotificationEvent == NULL) {
18528 |         return MA_INVALID_ARGS;
18529 |     }
18530 |
18531 |     #ifndef MA_NO_THREADING
18532 |     {
18533 |         ma_event_uninit(&pNotificationEvent->e);
18534 |         return MA_SUCCESS;
18535 |     }
18536 |     #else
18537 |     {
18538 |         return MA_NOT_IMPLEMENTED; /* Threading is disabled. */
18539 |     }
18540 |     #endif
18541 | }
18542 |
18543 | MA_API ma_result ma_async_notification_event_wait(ma_async_notification_event* pNotificationEvent)
18544 | {
18545 |     if (pNotificationEvent == NULL) {
18546 |         return MA_INVALID_ARGS;
18547 |     }
18548 |
18549 |     #ifndef MA_NO_THREADING
18550 |     {
18551 |         return ma_event_wait(&pNotificationEvent->e);
18552 |     }
18553 |     #else
18554 |     {
18555 |         return MA_NOT_IMPLEMENTED; /* Threading is disabled. */
18556 |     }
18557 |     #endif
18558 | }
18559 |
18560 | MA_API ma_result ma_async_notification_event_signal(ma_async_notification_event* pNotificationEvent)
18561 | {
18562 |     if (pNotificationEvent == NULL) {
18563 |         return MA_INVALID_ARGS;
18564 |     }
18565 |
18566 |     #ifndef MA_NO_THREADING
18567 |     {
18568 |         return ma_event_signal(&pNotificationEvent->e);
18569 |     }
18570 |     #else
18571 |     {
18572 |         return MA_NOT_IMPLEMENTED; /* Threading is disabled. */
18573 |     }
18574 |     #endif
18575 | }
18576 |
18577 |
18578 |
```

```

18579 | /*****
> *****/
18580 |
18581 | Job Queue
18582 |
18583 | *****/
> *****/
18584 | MA_API ma_slot_allocator_config ma_slot_allocator_config_init(ma_uint32 capacity)
18585 | {
18586 |     ma_slot_allocator_config config;
18587 |
18588 |     MA_ZERO_OBJECT(&config);
18589 |     config.capacity = capacity;
18590 |
18591 |     return config;
18592 | }
18593 |
18594 |
18595 | static MA_INLINE ma_uint32 ma_slot_allocator_calculate_group_capacity(ma_uint32 slotCapacity)
18596 | {
18597 |     ma_uint32 cap = slotCapacity / 32;
18598 |     if ((slotCapacity % 32) != 0) {
18599 |         cap += 1;
18600 |     }
18601 |
18602 |     return cap;
18603 | }
18604 |
18605 | static MA_INLINE ma_uint32 ma_slot_allocator_group_capacity(const ma_slot_allocator* pAllocator)
18606 | {
18607 |     return ma_slot_allocator_calculate_group_capacity(pAllocator->capacity);
18608 | }
18609 |
18610 |
18611 | typedef struct
18612 | {
18613 |     size_t sizeInBytes;
18614 |     size_t groupsOffset;
18615 |     size_t slotsOffset;
18616 | } ma_slot_allocator_heap_layout;
18617 |
18618 | static ma_result ma_slot_allocator_get_heap_layout(const ma_slot_allocator_config* pConfig,
> ma_slot_allocator_heap_layout* pHeapLayout)
18619 | {
18620 |     MA_ASSERT(pHeapLayout != NULL);
18621 |
18622 |     MA_ZERO_OBJECT(pHeapLayout);
18623 |
18624 |     if (pConfig == NULL) {
18625 |         return MA_INVALID_ARGS;
18626 |     }
18627 |
18628 |     if (pConfig->capacity == 0) {
18629 |         return MA_INVALID_ARGS;
18630 |     }
18631 |
18632 |     pHeapLayout->sizeInBytes = 0;
18633 |
18634 |     /* Groups. */
18635 |     pHeapLayout->groupsOffset = pHeapLayout->sizeInBytes;
18636 |     pHeapLayout->sizeInBytes +=
> ma_align_64(ma_slot_allocator_calculate_group_capacity(pConfig->capacity) *
> sizeof(ma_slot_allocator_group));
18637 |
18638 |     /* Slots. */
18639 |     pHeapLayout->slotsOffset = pHeapLayout->sizeInBytes;
18640 |     pHeapLayout->sizeInBytes += ma_align_64(pConfig->capacity * sizeof(ma_uint32));
18641 |
18642 |     return MA_SUCCESS;
18643 | }
18644 |
18645 | MA_API ma_result ma_slot_allocator_get_heap_size(const ma_slot_allocator_config* pConfig, size_t*
> pHeapSizeInBytes)
18646 | {
18647 |     ma_result result;
18648 |     ma_slot_allocator_heap_layout layout;
18649 |
18650 |     if (pHeapSizeInBytes == NULL) {
18651 |         return MA_INVALID_ARGS;

```

```

18652 |     }
18653 |
18654 |     *pHeapSizeInBytes = 0;
18655 |
18656 |     result = ma_slot_allocator_get_heap_layout(pConfig, &layout);
18657 |     if (result != MA_SUCCESS) {
18658 |         return result;
18659 |     }
18660 |
18661 |     *pHeapSizeInBytes = layout.sizeInBytes;
18662 |
18663 |     return result;
18664 | }
18665 |
18666 | MA_API ma_result ma_slot_allocator_init_preallocated(const ma_slot_allocator_config* pConfig, void*
> pHeap, ma_slot_allocator* pAllocator)
18667 | {
18668 |     ma_result result;
18669 |     ma_slot_allocator_heap_layout heapLayout;
18670 |
18671 |     if (pAllocator == NULL) {
18672 |         return MA_INVALID_ARGS;
18673 |     }
18674 |
18675 |     MA_ZERO_OBJECT(pAllocator);
18676 |
18677 |     if (pHeap == NULL) {
18678 |         return MA_INVALID_ARGS;
18679 |     }
18680 |
18681 |     result = ma_slot_allocator_get_heap_layout(pConfig, &heapLayout);
18682 |     if (result != MA_SUCCESS) {
18683 |         return result;
18684 |     }
18685 |
18686 |     pAllocator->pHeap = pHeap;
18687 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
18688 |
18689 |     pAllocator->pGroups = (ma_slot_allocator_group*)ma_offset_ptr(pHeap, heapLayout.groupsOffset);
18690 |     pAllocator->pSlots = (ma_uint32*)ma_offset_ptr(pHeap, heapLayout.slotsOffset);
18691 |     pAllocator->capacity = pConfig->capacity;
18692 |
18693 |     return MA_SUCCESS;
18694 | }
18695 |
18696 | MA_API ma_result ma_slot_allocator_init(const ma_slot_allocator_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_slot_allocator* pAllocator)
18697 | {
18698 |     ma_result result;
18699 |     size_t heapSizeInBytes;
18700 |     void* pHeap;
18701 |
18702 |     result = ma_slot_allocator_get_heap_size(pConfig, &heapSizeInBytes);
18703 |     if (result != MA_SUCCESS) {
18704 |         return result; /* Failed to retrieve the size of the heap allocation. */
18705 |     }
18706 |
18707 |     if (heapSizeInBytes > 0) {
18708 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
18709 |         if (pHeap == NULL) {
18710 |             return MA_OUT_OF_MEMORY;
18711 |         }
18712 |     } else {
18713 |         pHeap = NULL;
18714 |     }
18715 |
18716 |     result = ma_slot_allocator_init_preallocated(pConfig, pHeap, pAllocator);
18717 |     if (result != MA_SUCCESS) {
18718 |         ma_free(pHeap, pAllocationCallbacks);
18719 |         return result;
18720 |     }
18721 |
18722 |     pAllocator->ownsHeap = MA_TRUE;
18723 |     return MA_SUCCESS;
18724 | }
18725 |
18726 | MA_API void ma_slot_allocator_uninit(ma_slot_allocator* pAllocator, const ma_allocation_callbacks*
> pAllocationCallbacks)
18727 | {

```

```

18728 |     if (pAllocator == NULL) {
18729 |         return;
18730 |     }
18731 |
18732 |     if (pAllocator->ownsHeap) {
18733 |         ma_free(pAllocator->pHeap, pAllocationCallbacks);
18734 |     }
18735 | }
18736 |
18737 | MA_API ma_result ma_slot_allocator_alloc(ma_slot_allocator* pAllocator, ma_uint64* pSlot)
18738 | {
18739 |     ma_uint32 iAttempt;
18740 |     const ma_uint32 maxAttempts = 2;    /* The number of iterations to perform until returning
> MA_OUT_OF_MEMORY if no slots can be found. */
18741 |
18742 |     if (pAllocator == NULL || pSlot == NULL) {
18743 |         return MA_INVALID_ARGS;
18744 |     }
18745 |
18746 |     for (iAttempt = 0; iAttempt < maxAttempts; iAttempt += 1) {
18747 |         /* We need to acquire a suitable bitfield first. This is a bitfield that's got an available
> slot within it. */
18748 |         ma_uint32 iGroup;
18749 |         for (iGroup = 0; iGroup < ma_slot_allocator_group_capacity(pAllocator); iGroup += 1) {
18750 |             /* CAS */
18751 |             for (;;) {
18752 |                 ma_uint32 oldBitfield;
18753 |                 ma_uint32 newBitfield;
18754 |                 ma_uint32 bitOffset;
18755 |
18756 |                 oldBitfield = ma_atomic_load_32(&pAllocator->pGroups[iGroup].bitfield); /* <-- This
> copy must happen. The compiler must not optimize this away. */
18757 |
18758 |                 /* Fast check to see if anything is available. */
18759 |                 if (oldBitfield == 0xFFFFFFFF) {
18760 |                     break; /* No available bits in this bitfield. */
18761 |                 }
18762 |
18763 |                 bitOffset = ma_ffs_32(~oldBitfield);
18764 |                 MA_ASSERT(bitOffset < 32);
18765 |
18766 |                 newBitfield = oldBitfield | (1 << bitOffset);
18767 |
18768 |                 if (ma_atomic_compare_and_swap_32(&pAllocator->pGroups[iGroup].bitfield,
> oldBitfield, newBitfield) == oldBitfield) {
18769 |                     ma_uint32 slotIndex;
18770 |
18771 |                     /* Increment the counter as soon as possible to have other threads report
> out-of-memory sooner than later. */
18772 |                     ma_atomic_fetch_add_32(&pAllocator->count, 1);
18773 |
18774 |                     /* The slot index is required for constructing the output value. */
18775 |                     slotIndex = (iGroup << 5) + bitOffset; /* iGroup << 5 = iGroup * 32 */
18776 |                     if (slotIndex >= pAllocator->capacity) {
18777 |                         return MA_OUT_OF_MEMORY;
18778 |                     }
18779 |
18780 |                     /* Increment the reference count before constructing the output value. */
18781 |                     pAllocator->pSlots[slotIndex] += 1;
18782 |
18783 |                     /* Construct the output value. */
18784 |                     *pSlot = (((ma_uint64)pAllocator->pSlots[slotIndex] << 32) | slotIndex);
18785 |
18786 |                     return MA_SUCCESS;
18787 |                 }
18788 |             }
18789 |         }
18790 |
18791 |         /* We weren't able to find a slot. If it's because we've reached our capacity we need to
> return MA_OUT_OF_MEMORY. Otherwise we need to do another iteration and try again. */
18792 |         if (pAllocator->count < pAllocator->capacity) {
18793 |             ma_yield();
18794 |         } else {
18795 |             return MA_OUT_OF_MEMORY;
18796 |         }
18797 |     }
18798 |
18799 |     /* We couldn't find a slot within the maximum number of attempts. */
18800 |     return MA_OUT_OF_MEMORY;

```

```

18801 | }
18802 |
18803 | MA_API ma_result ma_slot_allocator_free(ma_slot_allocator* pAllocator, ma_uint64 slot)
18804 | {
18805 |     ma_uint32 iGroup;
18806 |     ma_uint32 iBit;
18807 |
18808 |     if (pAllocator == NULL) {
18809 |         return MA_INVALID_ARGS;
18810 |     }
18811 |
18812 |     iGroup = (ma_uint32)((slot & 0xFFFFFFFF) >> 5); /* slot / 32 */
18813 |     iBit   = (ma_uint32)((slot & 0xFFFFFFFF) & 31); /* slot % 32 */
18814 |
18815 |     if (iGroup >= ma_slot_allocator_group_capacity(pAllocator)) {
18816 |         return MA_INVALID_ARGS;
18817 |     }
18818 |
18819 |     MA_ASSERT(iBit < 32); /* This must be true due to the logic we used to actually calculate it.
> */
18820 |
18821 |     while (ma_atomic_load_32(&pAllocator->count) > 0) {
18822 |         /* CAS */
18823 |         ma_uint32 oldBitfield;
18824 |         ma_uint32 newBitfield;
18825 |
18826 |         oldBitfield = ma_atomic_load_32(&pAllocator->pGroups[iGroup].bitfield); /* <-- This copy
> must happen. The compiler must not optimize this away. */
18827 |         newBitfield = oldBitfield & ~(1 << iBit);
18828 |
18829 |         /* Debugging for checking for double-frees. */
18830 |         #if defined(MA_DEBUG_OUTPUT)
18831 |         {
18832 |             if ((oldBitfield & (1 << iBit)) == 0) {
18833 |                 MA_ASSERT(MA_FALSE); /* Double free detected.*/
18834 |             }
18835 |         }
18836 |         #endif
18837 |
18838 |         if (ma_atomic_compare_and_swap_32(&pAllocator->pGroups[iGroup].bitfield, oldBitfield,
> newBitfield) == oldBitfield) {
18839 |             ma_atomic_fetch_sub_32(&pAllocator->count, 1);
18840 |             return MA_SUCCESS;
18841 |         }
18842 |     }
18843 |
18844 |     /* Getting here means there are no allocations available for freeing. */
18845 |     return MA_INVALID_OPERATION;
18846 | }
18847 |
18848 |
18849 | #define MA_JOB_ID_NONE      ~((ma_uint64)0)
18850 | #define MA_JOB_SLOT_NONE   (ma_uint16)(~0)
18851 |
18852 | static MA_INLINE ma_uint32 ma_job_extract_refcount(ma_uint64 toc)
18853 | {
18854 |     return (ma_uint32)(toc >> 32);
18855 | }
18856 |
18857 | static MA_INLINE ma_uint16 ma_job_extract_slot(ma_uint64 toc)
18858 | {
18859 |     return (ma_uint16)(toc & 0x0000FFFF);
18860 | }
18861 |
18862 | #if 0 /* Currently unused, but might make use of this later. */
18863 | static MA_INLINE ma_uint16 ma_job_extract_code(ma_uint64 toc)
18864 | {
18865 |     return (ma_uint16)((toc & 0xFFFF0000) >> 16);
18866 | }
18867 | #endif
18868 |
18869 | static MA_INLINE ma_uint64 ma_job_toc_to_allocation(ma_uint64 toc)
18870 | {
18871 |     return ((ma_uint64)ma_job_extract_refcount(toc) << 32) | (ma_uint64)ma_job_extract_slot(toc);
18872 | }
18873 |
18874 | static MA_INLINE ma_uint64 ma_job_set_refcount(ma_uint64 toc, ma_uint32 refcount)
18875 | {
18876 |     /* Clear the reference count first. */

```

```

18877 |     toc = toc & ~(ma_uint64)0xFFFFFFFF << 32);
18878 |     toc = toc | ((ma_uint64)refcount << 32);
18879 |
18880 |     return toc;
18881 | }
18882 |
18883 |
18884 | MA_API ma_job ma_job_init(ma_uint16 code)
18885 | {
18886 |     ma_job job;
18887 |
18888 |     MA_ZERO_OBJECT(&job);
18889 |     job.toc.breakup.code = code;
18890 |     job.toc.breakup.slot = MA_JOB_SLOT_NONE;    /* Temp value. Will be allocated when posted to a
> queue. */
18891 |     job.next = MA_JOB_ID_NONE;
18892 |
18893 |     return job;
18894 | }
18895 |
18896 |
18897 | static ma_result ma_job_process_noop(ma_job* pJob);
18898 | static ma_result ma_job_process_quit(ma_job* pJob);
18899 | static ma_result ma_job_process_custom(ma_job* pJob);
18900 | static ma_result ma_job_process_resource_manager_load_data_buffer_node(ma_job* pJob);
18901 | static ma_result ma_job_process_resource_manager_free_data_buffer_node(ma_job* pJob);
18902 | static ma_result ma_job_process_resource_manager_page_data_buffer_node(ma_job* pJob);
18903 | static ma_result ma_job_process_resource_manager_load_data_buffer(ma_job* pJob);
18904 | static ma_result ma_job_process_resource_manager_free_data_buffer(ma_job* pJob);
18905 | static ma_result ma_job_process_resource_manager_load_data_stream(ma_job* pJob);
18906 | static ma_result ma_job_process_resource_manager_free_data_stream(ma_job* pJob);
18907 | static ma_result ma_job_process_resource_manager_page_data_stream(ma_job* pJob);
18908 | static ma_result ma_job_process_resource_manager_seek_data_stream(ma_job* pJob);
18909 |
18910 | #if !defined(MA_NO_DEVICE_IO)
18911 | static ma_result ma_job_process_device_aaudio_reroute(ma_job* pJob);
18912 | #endif
18913 |
18914 | static ma_job_proc g_jobVTable[MA_JOB_TYPE_COUNT] =
18915 | {
18916 |     /* Miscellaneous. */
18917 |     ma_job_process_quit,                /* MA_JOB_TYPE_QUIT */
18918 |     ma_job_process_custom,              /* MA_JOB_TYPE_CUSTOM */
18919 |
18920 |     /* Resource Manager. */
18921 |     ma_job_process_resource_manager_load_data_buffer_node, /*
> MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER_NODE */
18922 |     ma_job_process_resource_manager_free_data_buffer_node, /*
> MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER_NODE */
18923 |     ma_job_process_resource_manager_page_data_buffer_node, /*
> MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE */
18924 |     ma_job_process_resource_manager_load_data_buffer,      /*
> MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER */
18925 |     ma_job_process_resource_manager_free_data_buffer,      /*
> MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER */
18926 |     ma_job_process_resource_manager_load_data_stream,      /*
> MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_STREAM */
18927 |     ma_job_process_resource_manager_free_data_stream,      /*
> MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_STREAM */
18928 |     ma_job_process_resource_manager_page_data_stream,      /*
> MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_STREAM */
18929 |     ma_job_process_resource_manager_seek_data_stream,      /*
> MA_JOB_TYPE_RESOURCE_MANAGER_SEEK_DATA_STREAM */
18930 |
18931 |     /* Device. */
18932 |     #if !defined(MA_NO_DEVICE_IO)
18933 |     ma_job_process_device_aaudio_reroute    /* MA_JOB_TYPE_DEVICE_AAUDIO_REROUTE
> */
18934 |     #endif
18935 | };
18936 |
18937 | MA_API ma_result ma_job_process(ma_job* pJob)
18938 | {
18939 |     if (pJob == NULL) {
18940 |         return MA_INVALID_ARGS;
18941 |     }
18942 |
18943 |     if (pJob->toc.breakup.code >= MA_JOB_TYPE_COUNT) {
18944 |         return MA_INVALID_OPERATION;

```



```

18945 |     }
18946 |
18947 |     return g_jobVTable[pJob->toc.breakup.code](pJob);
18948 | }
18949 |
18950 | static ma_result ma_job_process__noop(ma_job* pJob)
18951 | {
18952 |     MA_ASSERT(pJob != NULL);
18953 |
18954 |     /* No-op. */
18955 |     (void)pJob;
18956 |
18957 |     return MA_SUCCESS;
18958 | }
18959 |
18960 | static ma_result ma_job_process__quit(ma_job* pJob)
18961 | {
18962 |     return ma_job_process__noop(pJob);
18963 | }
18964 |
18965 | static ma_result ma_job_process__custom(ma_job* pJob)
18966 | {
18967 |     MA_ASSERT(pJob != NULL);
18968 |
18969 |     /* No-op if there's no callback. */
18970 |     if (pJob->data.custom.proc == NULL) {
18971 |         return MA_SUCCESS;
18972 |     }
18973 |
18974 |     return pJob->data.custom.proc(pJob);
18975 | }
18976 |
18977 |
18978 |
18979 | MA_API ma_job_queue_config ma_job_queue_config_init(ma_uint32 flags, ma_uint32 capacity)
18980 | {
18981 |     ma_job_queue_config config;
18982 |
18983 |     config.flags    = flags;
18984 |     config.capacity = capacity;
18985 |
18986 |     return config;
18987 | }
18988 |
18989 |
18990 | typedef struct
18991 | {
18992 |     size_t sizeInBytes;
18993 |     size_t allocatorOffset;
18994 |     size_t jobsOffset;
18995 | } ma_job_queue_heap_layout;
18996 |
18997 | static ma_result ma_job_queue_get_heap_layout(const ma_job_queue_config* pConfig,
18998 | > ma_job_queue_heap_layout* pHeapLayout)
18999 | {
19000 |     ma_result result;
19001 |
19002 |     MA_ASSERT(pHeapLayout != NULL);
19003 |
19004 |     MA_ZERO_OBJECT(pHeapLayout);
19005 |
19006 |     if (pConfig == NULL) {
19007 |         return MA_INVALID_ARGS;
19008 |     }
19009 |
19010 |     if (pConfig->capacity == 0) {
19011 |         return MA_INVALID_ARGS;
19012 |     }
19013 |
19014 |     pHeapLayout->sizeInBytes = 0;
19015 |
19016 |     /* Allocator. */
19017 |     {
19018 |         ma_slot_allocator_config allocatorConfig;
19019 |         size_t allocatorHeapSizeInBytes;
19020 |
19021 |         allocatorConfig = ma_slot_allocator_config_init(pConfig->capacity);
19022 |         result = ma_slot_allocator_get_heap_size(&allocatorConfig, &allocatorHeapSizeInBytes);
19023 |         if (result != MA_SUCCESS) {

```

```

19023 |         return result;
19024 |     }
19025 |
19026 |     pHeapLayout->allocatorOffset = pHeapLayout->sizeInBytes;
19027 |     pHeapLayout->sizeInBytes    += allocatorHeapSizeInBytes;
19028 | }
19029 |
19030 | /* Jobs. */
19031 | pHeapLayout->jobsOffset    = pHeapLayout->sizeInBytes;
19032 | pHeapLayout->sizeInBytes += ma_align_64(pConfig->capacity * sizeof(ma_job));
19033 |
19034 | return MA_SUCCESS;
19035 | }
19036 |
19037 | MA_API ma_result ma_job_queue_get_heap_size(const ma_job_queue_config* pConfig, size_t*
> pHeapSizeInBytes)
19038 | {
19039 |     ma_result result;
19040 |     ma_job_queue_heap_layout layout;
19041 |
19042 |     if (pHeapSizeInBytes == NULL) {
19043 |         return MA_INVALID_ARGS;
19044 |     }
19045 |
19046 |     *pHeapSizeInBytes = 0;
19047 |
19048 |     result = ma_job_queue_get_heap_layout(pConfig, &layout);
19049 |     if (result != MA_SUCCESS) {
19050 |         return result;
19051 |     }
19052 |
19053 |     *pHeapSizeInBytes = layout.sizeInBytes;
19054 |
19055 |     return MA_SUCCESS;
19056 | }
19057 |
19058 | MA_API ma_result ma_job_queue_init_preallocated(const ma_job_queue_config* pConfig, void* pHeap,
> ma_job_queue* pQueue)
19059 | {
19060 |     ma_result result;
19061 |     ma_job_queue_heap_layout heapLayout;
19062 |     ma_slot_allocator_config allocatorConfig;
19063 |
19064 |     if (pQueue == NULL) {
19065 |         return MA_INVALID_ARGS;
19066 |     }
19067 |
19068 |     MA_ZERO_OBJECT(pQueue);
19069 |
19070 |     result = ma_job_queue_get_heap_layout(pConfig, &heapLayout);
19071 |     if (result != MA_SUCCESS) {
19072 |         return result;
19073 |     }
19074 |
19075 |     pQueue->pHeap = pHeap;
19076 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
19077 |
19078 |     pQueue->flags    = pConfig->flags;
19079 |     pQueue->capacity = pConfig->capacity;
19080 |     pQueue->pJobs    = (ma_job*)ma_offset_ptr(pHeap, heapLayout.jobsOffset);
19081 |
19082 |     allocatorConfig = ma_slot_allocator_config_init(pConfig->capacity);
19083 |     result = ma_slot_allocator_init_preallocated(&allocatorConfig, ma_offset_ptr(pHeap,
> heapLayout.allocatorOffset), &pQueue->allocator);
19084 |     if (result != MA_SUCCESS) {
19085 |         return result;
19086 |     }
19087 |
19088 |     /* We need a semaphore if we're running in non-blocking mode. If threading is disabled we need
> to return an error. */
19089 |     if ((pQueue->flags & MA_JOB_QUEUE_FLAG_NON_BLOCKING) == 0) {
19090 |         #ifndef MA_NO_THREADING
19091 |         {
19092 |             ma_semaphore_init(0, &pQueue->sem);
19093 |         }
19094 |         #else
19095 |         {
19096 |             /* Threading is disabled and we've requested non-blocking mode. */
19097 |             return MA_INVALID_OPERATION;

```

```

19098 |     }
19099 |     #endif
19100 | }
19101 |
19102 | /*
19103 | Our queue needs to be initialized with a free standing node. This should always be slot 0.
> Required for the lock free algorithm. The first job in the queue is
19104 | just a dummy item for giving us the first item in the list which is stored in the "next" member.
19105 | */
19106 | ma_slot_allocator_alloc(&pQueue->allocator, &pQueue->head); /* Will never fail. */
19107 | pQueue->pJobs[ma_job_extract_slot(pQueue->head)].next = MA_JOB_ID_NONE;
19108 | pQueue->tail = pQueue->head;
19109 |
19110 | return MA_SUCCESS;
19111 | }
19112 |
19113 | MA_API ma_result ma_job_queue_init(const ma_job_queue_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_job_queue* pQueue)
19114 | {
19115 |     ma_result result;
19116 |     size_t heapSizeInBytes;
19117 |     void* pHeap;
19118 |
19119 |     result = ma_job_queue_get_heap_size(pConfig, &heapSizeInBytes);
19120 |     if (result != MA_SUCCESS) {
19121 |         return result;
19122 |     }
19123 |
19124 |     if (heapSizeInBytes > 0) {
19125 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
19126 |         if (pHeap == NULL) {
19127 |             return MA_OUT_OF_MEMORY;
19128 |         }
19129 |     } else {
19130 |         pHeap = NULL;
19131 |     }
19132 |
19133 |     result = ma_job_queue_init_preallocated(pConfig, pHeap, pQueue);
19134 |     if (result != MA_SUCCESS) {
19135 |         ma_free(pHeap, pAllocationCallbacks);
19136 |         return result;
19137 |     }
19138 |
19139 |     pQueue->_ownsHeap = MA_TRUE;
19140 |     return MA_SUCCESS;
19141 | }
19142 |
19143 | MA_API void ma_job_queue_uninit(ma_job_queue* pQueue, const ma_allocation_callbacks*
> pAllocationCallbacks)
19144 | {
19145 |     if (pQueue == NULL) {
19146 |         return;
19147 |     }
19148 |
19149 |     /* All we need to do is uninitialized the semaphore. */
19150 |     if ((pQueue->flags & MA_JOB_QUEUE_FLAG_NON_BLOCKING) == 0) {
19151 |         #ifndef MA_NO_THREADING
19152 |         {
19153 |             ma_semaphore_uninit(&pQueue->sem);
19154 |         }
19155 |         #else
19156 |         {
19157 |             MA_ASSERT(MA_FALSE); /* Should never get here. Should have been checked at
> initialization time. */
19158 |         }
19159 |         #endif
19160 |     }
19161 |
19162 |     ma_slot_allocator_uninit(&pQueue->allocator, pAllocationCallbacks);
19163 |
19164 |     if (pQueue->_ownsHeap) {
19165 |         ma_free(pQueue->pHeap, pAllocationCallbacks);
19166 |     }
19167 | }
19168 |
19169 | static ma_bool32 ma_job_queue_cas(volatile ma_uint64* dst, ma_uint64 expected, ma_uint64 desired)
19170 | {
19171 |     /* The new counter is taken from the expected value. */
19172 |     return ma_atomic_compare_and_swap_64(dst, expected, ma_job_set_refcount(desired,

```

```

    > ma_job_extract_refcount(expected) + 1)) == expected;
19173 | }
19174 |
19175 | MA_API ma_result ma_job_queue_post(ma_job_queue* pQueue, const ma_job* pJob)
19176 | {
19177 |     /*
19178 |     Lock free queue implementation based on the paper by Michael and Scott: Nonblocking Algorithms
    > and Preemption-Safe Locking on Multiprogrammed Shared Memory Multiprocessors
19179 |     */
19180 |     ma_result result;
19181 |     ma_uint64 slot;
19182 |     ma_uint64 tail;
19183 |     ma_uint64 next;
19184 |
19185 |     if (pQueue == NULL || pJob == NULL) {
19186 |         return MA_INVALID_ARGS;
19187 |     }
19188 |
19189 |     /* We need a new slot. */
19190 |     result = ma_slot_allocator_alloc(&pQueue->allocator, &slot);
19191 |     if (result != MA_SUCCESS) {
19192 |         return result; /* Probably ran out of slots. If so, MA_OUT_OF_MEMORY will be returned. */
19193 |     }
19194 |
19195 |     /* At this point we should have a slot to place the job. */
19196 |     MA_ASSERT(ma_job_extract_slot(slot) < pQueue->capacity);
19197 |
19198 |     /* We need to put the job into memory before we do anything. */
19199 |     pQueue->pJobs[ma_job_extract_slot(slot)] = *pJob;
19200 |     pQueue->pJobs[ma_job_extract_slot(slot)].toc.allocation = slot; /* This
    > will overwrite the job code. */
19201 |     pQueue->pJobs[ma_job_extract_slot(slot)].toc.breakup.code = pJob->toc.breakup.code; /* The job
    > code needs to be applied again because the line above overwrote it. */
19202 |     pQueue->pJobs[ma_job_extract_slot(slot)].next = MA_JOB_ID_NONE; /* Reset
    > for safety. */
19203 |
19204 |     #ifndef MA_USE_EXPERIMENTAL_LOCK_FREE_JOB_QUEUE
19205 |     ma_spinlock_lock(&pQueue->lock);
19206 |     #endif
19207 |     {
19208 |         /* The job is stored in memory so now we need to add it to our linked list. We only ever add
    > items to the end of the list. */
19209 |         for (;;) {
19210 |             tail = ma_atomic_load_64(&pQueue->tail);
19211 |             next = ma_atomic_load_64(&pQueue->pJobs[ma_job_extract_slot(tail)].next);
19212 |
19213 |             if (ma_job_toc_to_allocation(tail) ==
    > ma_job_toc_to_allocation(ma_atomic_load_64(&pQueue->tail))) {
19214 |                 if (ma_job_extract_slot(next) == 0xFFFF) {
19215 |                     if (ma_job_queue_cas(&pQueue->pJobs[ma_job_extract_slot(tail)].next, next,
    > slot)) {
19216 |                         break;
19217 |                     }
19218 |                 } else {
19219 |                     ma_job_queue_cas(&pQueue->tail, tail, ma_job_extract_slot(next));
19220 |                 }
19221 |             }
19222 |         }
19223 |         ma_job_queue_cas(&pQueue->tail, tail, slot);
19224 |     }
19225 |     #ifndef MA_USE_EXPERIMENTAL_LOCK_FREE_JOB_QUEUE
19226 |     ma_spinlock_unlock(&pQueue->lock);
19227 |     #endif
19228 |
19229 |
19230 |     /* Signal the semaphore as the last step if we're using synchronous mode. */
19231 |     if ((pQueue->flags & MA_JOB_QUEUE_FLAG_NON_BLOCKING) == 0) {
19232 |         #ifndef MA_NO_THREADING
19233 |         {
19234 |             ma_semaphore_release(&pQueue->sem);
19235 |         }
19236 |         #else
19237 |         {
19238 |             MA_ASSERT(MA_FALSE); /* Should never get here. Should have been checked at
    > initialization time. */
19239 |         }
19240 |         #endif
19241 |     }
19242 | }

```

```

19243 |     return MA_SUCCESS;
19244 | }
19245 |
19246 | MA_API ma_result ma_job_queue_next(ma_job_queue* pQueue, ma_job* pJob)
19247 | {
19248 |     ma_uint64 head;
19249 |     ma_uint64 tail;
19250 |     ma_uint64 next;
19251 |
19252 |     if (pQueue == NULL || pJob == NULL) {
19253 |         return MA_INVALID_ARGS;
19254 |     }
19255 |
19256 |     /* If we're running in synchronous mode we'll need to wait on a semaphore. */
19257 |     if ((pQueue->flags & MA_JOB_QUEUE_FLAG_NON_BLOCKING) == 0) {
19258 |         #ifndef MA_NO_THREADING
19259 |         {
19260 |             ma_semaphore_wait(&pQueue->sem);
19261 |         }
19262 |         #else
19263 |         {
19264 |             MA_ASSERT(MA_FALSE);    /* Should never get here. Should have been checked at
> initialization time. */
19265 |         }
19266 |         #endif
19267 |     }
19268 |
19269 |     #ifndef MA_USE_EXPERIMENTAL_LOCK_FREE_JOB_QUEUE
19270 |     ma_spinlock_lock(&pQueue->lock);
19271 |     #endif
19272 |     {
19273 |         /*
19274 |         BUG: In lock-free mode, multiple threads can be in this section of code. The "head" variable
> in the loop below
19275 |         is stored. One thread can fall through to the freeing of this item while another is still
> using "head" for the
19276 |         retrieval of the "next" variable.
19277 |
19278 |         The slot allocator might need to make use of some reference counting to ensure it's only
> truly freed when
19279 |         there are no more references to the item. This must be fixed before removing these locks.
19280 |         */
19281 |
19282 |         /* Now we need to remove the root item from the list. */
19283 |         for (;;) {
19284 |             head = ma_atomic_load_64(&pQueue->head);
19285 |             tail = ma_atomic_load_64(&pQueue->tail);
19286 |             next = ma_atomic_load_64(&pQueue->pJobs[ma_job_extract_slot(head)].next);
19287 |
19288 |             if (ma_job_toc_to_allocation(head) ==
> ma_job_toc_to_allocation(ma_atomic_load_64(&pQueue->head))) {
19289 |                 if (ma_job_extract_slot(head) == ma_job_extract_slot(tail)) {
19290 |                     if (ma_job_extract_slot(next) == 0xFFFF) {
19291 |                         #ifndef MA_USE_EXPERIMENTAL_LOCK_FREE_JOB_QUEUE
19292 |                         ma_spinlock_unlock(&pQueue->lock);
19293 |                         #endif
19294 |                         return MA_NO_DATA_AVAILABLE;
19295 |                     }
19296 |                     ma_job_queue_cas(&pQueue->tail, tail, ma_job_extract_slot(next));
19297 |                 } else {
19298 |                     *pJob = pQueue->pJobs[ma_job_extract_slot(next)];
19299 |                     if (ma_job_queue_cas(&pQueue->head, head, ma_job_extract_slot(next))) {
19300 |                         break;
19301 |                     }
19302 |                 }
19303 |             }
19304 |         }
19305 |     }
19306 |     #ifndef MA_USE_EXPERIMENTAL_LOCK_FREE_JOB_QUEUE
19307 |     ma_spinlock_unlock(&pQueue->lock);
19308 |     #endif
19309 |
19310 |     ma_slot_allocator_free(&pQueue->allocator, head);
19311 |
19312 |     /*
19313 |     If it's a quit job make sure it's put back on the queue to ensure other threads have an
> opportunity to detect it and terminate naturally. We
19314 |     could instead just leave it on the queue, but that would involve fiddling with the lock-free
> code above and I want to keep that as simple as

```

```

19315 |     possible.
19316 |     */
19317 |     if (pJob->toc.breakup.code == MA_JOB_TYPE_QUIT) {
19318 |         ma_job_queue_post(pQueue, pJob);
19319 |         return MA_CANCELLED; /* Return a cancelled status just in case the thread is checking
> return codes and not properly checking for a quit job. */
19320 |     }
19321 |
19322 |     return MA_SUCCESS;
19323 | }
19324 |
19325 |
19326 |
19327 | /*****
19328 |
19329 | Dynamic Linking
19330 |
19331 | *****/
19332 | /* Disable run-time linking on certain backends and platforms. */
19333 | #ifndef MA_NO_RUNTIME_LINKING
19334 |     #if defined(MA_EMSCRIPTEN) || defined(MA_ORBIS) || defined(MA_PROSPERO) || defined(MA_SWITCH) ||
> defined(MA_DOS)
19335 |         #define MA_NO_RUNTIME_LINKING
19336 |     #endif
19337 | #endif
19338 |
19339 | #ifdef MA_POSIX
19340 |     /* No need for dlfcn.h if we're not using runtime linking. */
19341 |     #ifndef MA_NO_RUNTIME_LINKING
19342 |         #include <dlfcn.h>
19343 |     #endif
19344 | #endif
19345 |
19346 | MA_API ma_handle ma_dlopen(ma_log* pLog, const char* filename)
19347 | {
19348 |     #ifndef MA_NO_RUNTIME_LINKING
19349 |     {
19350 |         ma_handle handle;
19351 |
19352 |         ma_log_postf(pLog, MA_LOG_LEVEL_DEBUG, "Loading library: %s\n", filename);
19353 |
19354 |         #ifdef MA_WIN32
19355 |             /* From MSDN: Desktop applications cannot use LoadPackagedLibrary; if a desktop
> application calls this function it fails with APPMODEL_ERROR_NO_PACKAGE.*/
19356 |             #if !defined(MA_WIN32_UWP) || !(defined(WINAPI_FAMILY) &&
> ((defined(WINAPI_FAMILY_PHONE_APP) && WINAPI_FAMILY == WINAPI_FAMILY_PHONE_APP)))
19357 |                 handle = (ma_handle)LoadLibraryA(filename);
19358 |             #else
19359 |                 /* *sigh* It appears there is no ANSI version of LoadPackagedLibrary()... */
19360 |                 WCHAR filenameW[4096];
19361 |                 if (MultiByteToWideChar(CP_UTF8, 0, filename, -1, filenameW, ma_countof(filenameW))
> == 0) {
19362 |                     handle = NULL;
19363 |                 } else {
19364 |                     handle = (ma_handle)LoadPackagedLibrary(filenameW, 0);
19365 |                 }
19366 |             #endif
19367 |         #else
19368 |             handle = (ma_handle)dlopen(filename, RTLD_NOW);
19369 |         #endif
19370 |
19371 |         /*
19372 |         I'm not considering failure to load a library an error nor a warning because seamlessly
> falling
19373 |         through to a lower-priority
19374 |         backend is a deliberate design choice. Instead I'm logging it as an informational message.
19375 |         */
19376 |         if (handle == NULL) {
19377 |             ma_log_postf(pLog, MA_LOG_LEVEL_INFO, "Failed to load library: %s\n", filename);
19378 |         }
19379 |         return handle;
19380 |     }
19381 |     #else
19382 |     {
19383 |         /* Runtime linking is disabled. */
19384 |         (void)pLog;
19385 |         (void)filename;
19386 |         return NULL;
19387 |     }

```

```

19388 |     #endif
19389 | }
19390 |
19391 | MA_API void ma_dlclose(ma_log* pLog, ma_handle handle)
19392 | {
19393 |     #ifndef MA_NO_RUNTIME_LINKING
19394 |     {
19395 |         #ifdef MA_WIN32
19396 |         {
19397 |             FreeLibrary((HMODULE)handle);
19398 |         }
19399 |         #else
19400 |         {
19401 |             /* Hack for Android bug (see https://github.com/android/ndk/issues/360). Calling
> dlclose() pre-API 28 may segfault. */
19402 |             #if !defined(MA_ANDROID) || (defined(__ANDROID_API__) && __ANDROID_API__ >= 28)
19403 |             {
19404 |                 dlclose((void*)handle);
19405 |             }
19406 |             #else
19407 |             {
19408 |                 (void)handle;
19409 |             }
19410 |             #endif
19411 |         }
19412 |         #endif
19413 |
19414 |         (void)pLog;
19415 |     }
19416 |     #else
19417 |     {
19418 |         /* Runtime linking is disabled. */
19419 |         (void)pLog;
19420 |         (void)handle;
19421 |     }
19422 |     #endif
19423 | }
19424 |
19425 | MA_API ma_proc ma_dlsym(ma_log* pLog, ma_handle handle, const char* symbol)
19426 | {
19427 |     #ifndef MA_NO_RUNTIME_LINKING
19428 |     {
19429 |         ma_proc proc;
19430 |
19431 |         ma_log_postf(pLog, MA_LOG_LEVEL_DEBUG, "Loading symbol: %s\n", symbol);
19432 |
19433 |         #ifdef _WIN32
19434 |         {
19435 |             proc = (ma_proc)GetProcAddress((HMODULE)handle, symbol);
19436 |         }
19437 |         #else
19438 |         {
19439 |             #if (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >= 8))) ||
> defined(__clang__)
19440 |                 #pragma GCC diagnostic push
19441 |                 #pragma GCC diagnostic ignored "-Wpedantic"
19442 |                 #endif
19443 |                 proc = (ma_proc)dlsym((void*)handle, symbol);
19444 |                 #if (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >= 8))) ||
> defined(__clang__)
19445 |                     #pragma GCC diagnostic pop
19446 |                     #endif
19447 |             }
19448 |             #endif
19449 |
19450 |             if (proc == NULL) {
19451 |                 ma_log_postf(pLog, MA_LOG_LEVEL_WARNING, "Failed to load symbol: %s\n", symbol);
19452 |             }
19453 |
19454 |             (void)pLog; /* It's possible for pContext to be unused. */
19455 |             return proc;
19456 |         }
19457 |         #else
19458 |         {
19459 |             /* Runtime linking is disabled. */
19460 |             (void)pLog;
19461 |             (void)handle;
19462 |             (void)symbol;
19463 |             return NULL;

```



```

19464 |     }
19465 |     #endif
19466 | }
19467 |
19468 |
19469 |
19470 | /*****
> *****/
19471 | *****/
> *****/
19472 |
19473 | DEVICE I/O
19474 | =====
19475 |
19476 | *****/
> *****/
19477 | *****/
> *****/
19478 |
19479 | #ifdef MA_APPLE
19480 |     #include <AvailabilityMacros.h>
19481 | #endif
19482 |
19483 | #ifndef MA_NO_DEVICE_IO
19484 |
19485 | #if defined(MA_APPLE) && (MAC_OS_X_VERSION_MIN_REQUIRED < 101200)
19486 |     #include <mach/mach_time.h> /* For mach_absolute_time() */
19487 | #endif
19488 |
19489 | #ifdef MA_POSIX
19490 |     #include <sys/types.h>
19491 | #endif
19492 |
19493 | /* This must be set to at least 26. */
19494 | #ifndef MA_AAUDIO_MIN_ANDROID_SDK_VERSION
19495 | #define MA_AAUDIO_MIN_ANDROID_SDK_VERSION 27
19496 | #endif
19497 |
19498 |
19499 | MA_API void ma_device_info_add_native_data_format(ma_device_info* pDeviceInfo, ma_format format,
> ma_uint32 channels, ma_uint32 sampleRate, ma_uint32 flags)
19500 | {
19501 |     if (pDeviceInfo == NULL) {
19502 |         return;
19503 |     }
19504 |
19505 |     if (pDeviceInfo->nativeDataFormatCount < ma_countof(pDeviceInfo->nativeDataFormats)) {
19506 |         pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format = format;
19507 |         pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels = channels;
19508 |         pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate = sampleRate;
19509 |         pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags = flags;
19510 |         pDeviceInfo->nativeDataFormatCount += 1;
19511 |     }
19512 | }
19513 |
19514 |
19515 | typedef struct
19516 | {
19517 |     ma_backend backend;
19518 |     const char* pName;
19519 | } ma_backend_info;
19520 |
19521 | static ma_backend_info gBackendInfo[] = /* Indexed by the backend enum. Must be in the order
> backends are declared in the ma_backend enum. */
19522 | {
19523 |     {ma_backend_wasapi, "WASAPI"},
19524 |     {ma_backend_dsound, "DirectSound"},
19525 |     {ma_backend_winmm, "WinMM"},
19526 |     {ma_backend_coreaudio, "Core Audio"},
19527 |     {ma_backend_sndio, "sndio"},
19528 |     {ma_backend_audio4, "audio(4)"},
19529 |     {ma_backend_oss, "OSS"},
19530 |     {ma_backend_pulseaudio, "PulseAudio"},
19531 |     {ma_backend_alsa, "ALSA"},
19532 |     {ma_backend_jack, "JACK"},
19533 |     {ma_backend_aaudio, "AAudio"},
19534 |     {ma_backend_opensl, "OpenSL|ES"},
19535 |     {ma_backend_webaudio, "Web Audio"},
19536 |     {ma_backend_custom, "Custom"},

```

```
19537 |     {ma_backend_null,      "Null"}
19538 | };
19539 |
19540 | MA_API const char* ma_get_backend_name(ma_backend backend)
19541 | {
19542 |     if (backend < 0 || backend >= (int)ma_countof(gBackendInfo)) {
19543 |         return "Unknown";
19544 |     }
19545 |
19546 |     return gBackendInfo[backend].pName;
19547 | }
19548 |
19549 | MA_API ma_result ma_get_backend_from_name(const char* pBackendName, ma_backend* pBackend)
19550 | {
19551 |     size_t iBackend;
19552 |
19553 |     if (pBackendName == NULL) {
19554 |         return MA_INVALID_ARGS;
19555 |     }
19556 |
19557 |     for (iBackend = 0; iBackend < ma_countof(gBackendInfo); iBackend += 1) {
19558 |         if (ma_strcmp(pBackendName, gBackendInfo[iBackend].pName) == 0) {
19559 |             if (pBackend != NULL) {
19560 |                 *pBackend = gBackendInfo[iBackend].backend;
19561 |             }
19562 |
19563 |             return MA_SUCCESS;
19564 |         }
19565 |     }
19566 |
19567 |     /* Getting here means the backend name is unknown. */
19568 |     return MA_INVALID_ARGS;
19569 | }
19570 |
19571 | MA_API ma_bool32 ma_is_backend_enabled(ma_backend backend)
19572 | {
19573 |     /*
19574 |     > This looks a little bit gross, but we want all backends to be included in the switch to avoid
19575 |     > warnings on some compilers
19576 |     > about some enums not being handled by the switch statement.
19577 |     */
19578 |     switch (backend)
19579 |     {
19580 |         case ma_backend_wasapi:
19581 |             #if defined(MA_HAS_WASAPI)
19582 |                 return MA_TRUE;
19583 |             #else
19584 |                 return MA_FALSE;
19585 |             #endif
19586 |         case ma_backend_dsound:
19587 |             #if defined(MA_HAS_DSOUND)
19588 |                 return MA_TRUE;
19589 |             #else
19590 |                 return MA_FALSE;
19591 |             #endif
19592 |         case ma_backend_winmm:
19593 |             #if defined(MA_HAS_WINMM)
19594 |                 return MA_TRUE;
19595 |             #else
19596 |                 return MA_FALSE;
19597 |             #endif
19598 |         case ma_backend_coreaudio:
19599 |             #if defined(MA_HAS_COREAUDIO)
19600 |                 return MA_TRUE;
19601 |             #else
19602 |                 return MA_FALSE;
19603 |             #endif
19604 |         case ma_backend_sndio:
19605 |             #if defined(MA_HAS_SNDIO)
19606 |                 return MA_TRUE;
19607 |             #else
19608 |                 return MA_FALSE;
19609 |             #endif
19610 |         case ma_backend_audio4:
19611 |             #if defined(MA_HAS_AUDIO4)
19612 |                 return MA_TRUE;
19613 |             #else
19614 |                 return MA_FALSE;
19615 |             #endif
19616 |     }
```

```

19615 |     case ma_backend_oss:
19616 |     #if defined(MA_HAS_OSS)
19617 |         return MA_TRUE;
19618 |     #else
19619 |         return MA_FALSE;
19620 |     #endif
19621 |     case ma_backend_pulseaudio:
19622 |     #if defined(MA_HAS_PULSEAUDIO)
19623 |         return MA_TRUE;
19624 |     #else
19625 |         return MA_FALSE;
19626 |     #endif
19627 |     case ma_backend_alsa:
19628 |     #if defined(MA_HAS_ALSA)
19629 |         return MA_TRUE;
19630 |     #else
19631 |         return MA_FALSE;
19632 |     #endif
19633 |     case ma_backend_jack:
19634 |     #if defined(MA_HAS_JACK)
19635 |         return MA_TRUE;
19636 |     #else
19637 |         return MA_FALSE;
19638 |     #endif
19639 |     case ma_backend_aaudio:
19640 |     #if defined(MA_HAS_AAUDIO)
19641 |         #if defined(MA_ANDROID)
19642 |         {
19643 |             return ma_android_sdk_version() >= MA_AAUDIO_MIN_ANDROID_SDK_VERSION;
19644 |         }
19645 |         #else
19646 |             return MA_FALSE;
19647 |         #endif
19648 |     #else
19649 |         return MA_FALSE;
19650 |     #endif
19651 |     case ma_backend_opensl:
19652 |     #if defined(MA_HAS_OPENSLE)
19653 |         #if defined(MA_ANDROID)
19654 |         {
19655 |             return ma_android_sdk_version() >= 9;
19656 |         }
19657 |         #else
19658 |             return MA_TRUE;
19659 |         #endif
19660 |     #else
19661 |         return MA_FALSE;
19662 |     #endif
19663 |     case ma_backend_webaudio:
19664 |     #if defined(MA_HAS_WEBAUDIO)
19665 |         return MA_TRUE;
19666 |     #else
19667 |         return MA_FALSE;
19668 |     #endif
19669 |     case ma_backend_custom:
19670 |     #if defined(MA_HAS_CUSTOM)
19671 |         return MA_TRUE;
19672 |     #else
19673 |         return MA_FALSE;
19674 |     #endif
19675 |     case ma_backend_null:
19676 |     #if defined(MA_HAS_NULL)
19677 |         return MA_TRUE;
19678 |     #else
19679 |         return MA_FALSE;
19680 |     #endif
19681 |     default: return MA_FALSE;
19682 | }
19683 | }
19684 | }
19685 |
19686 | MA_API ma_result ma_get_enabled_backends(ma_backend* pBackends, size_t backendCap, size_t*
> pBackendCount)
19687 | {
19688 |     size_t backendCount;
19689 |     size_t iBackend;
19690 |     ma_result result = MA_SUCCESS;
19691 |
19692 |     if (pBackendCount == NULL) {

```

```

19693 |         return MA_INVALID_ARGS;
19694 |     }
19695 |
19696 |     backendCount = 0;
19697 |
19698 |     for (iBackend = 0; iBackend <= ma_backend_null; iBackend += 1) {
19699 |         ma_backend backend = (ma_backend)iBackend;
19700 |
19701 |         if (ma_is_backend_enabled(backend)) {
19702 |             /* The backend is enabled. Try adding it to the list. If there's no room, MA_NO_SPACE
> needs to be returned. */
19703 |             if (backendCount == backendCap) {
19704 |                 result = MA_NO_SPACE;
19705 |                 break;
19706 |             } else {
19707 |                 pBackends[backendCount] = backend;
19708 |                 backendCount += 1;
19709 |             }
19710 |         }
19711 |     }
19712 |
19713 |     if (pBackendCount != NULL) {
19714 |         *pBackendCount = backendCount;
19715 |     }
19716 |
19717 |     return result;
19718 | }
19719 |
19720 | MA_API ma_bool32 ma_is_loopback_supported(ma_backend backend)
19721 | {
19722 |     switch (backend)
19723 |     {
19724 |         case ma_backend_wasapi:      return MA_TRUE;
19725 |         case ma_backend_dsound:      return MA_FALSE;
19726 |         case ma_backend_winmm:       return MA_FALSE;
19727 |         case ma_backend_coreaudio:   return MA_FALSE;
19728 |         case ma_backend_sndio:       return MA_FALSE;
19729 |         case ma_backend_audio4:      return MA_FALSE;
19730 |         case ma_backend_oss:         return MA_FALSE;
19731 |         case ma_backend_pulseaudio:  return MA_FALSE;
19732 |         case ma_backend_alsa:        return MA_FALSE;
19733 |         case ma_backend_jack:        return MA_FALSE;
19734 |         case ma_backend_aaudio:      return MA_FALSE;
19735 |         case ma_backend_opensl:      return MA_FALSE;
19736 |         case ma_backend_webaudio:    return MA_FALSE;
19737 |         case ma_backend_custom:      return MA_FALSE;    /* <-- Will depend on the implementation of
> the backend. */
19738 |         case ma_backend_null:        return MA_FALSE;
19739 |         default:                     return MA_FALSE;
19740 |     }
19741 | }
19742 |
19743 |
19744 |
19745 | #if defined(MA_WIN32) && !defined(MA_XBOX)
19746 | /* WASAPI error codes. */
19747 | #define MA_AUDCLNT_E_NOT_INITIALIZED ((HRESULT)0x88890001)
19748 | #define MA_AUDCLNT_E_ALREADY_INITIALIZED ((HRESULT)0x88890002)
19749 | #define MA_AUDCLNT_E_WRONG_ENDPOINT_TYPE ((HRESULT)0x88890003)
19750 | #define MA_AUDCLNT_E_DEVICE_INVALIDATED ((HRESULT)0x88890004)
19751 | #define MA_AUDCLNT_E_NOT_STOPPED ((HRESULT)0x88890005)
19752 | #define MA_AUDCLNT_E_BUFFER_TOO_LARGE ((HRESULT)0x88890006)
19753 | #define MA_AUDCLNT_E_OUT_OF_ORDER ((HRESULT)0x88890007)
19754 | #define MA_AUDCLNT_E_UNSUPPORTED_FORMAT ((HRESULT)0x88890008)
19755 | #define MA_AUDCLNT_E_INVALID_SIZE ((HRESULT)0x88890009)
19756 | #define MA_AUDCLNT_E_DEVICE_IN_USE ((HRESULT)0x8889000A)
19757 | #define MA_AUDCLNT_E_BUFFER_OPERATION_PENDING ((HRESULT)0x8889000B)
19758 | #define MA_AUDCLNT_E_THREAD_NOT_REGISTERED ((HRESULT)0x8889000C)
19759 | #define MA_AUDCLNT_E_NO_SINGLE_PROCESS ((HRESULT)0x8889000D)
19760 | #define MA_AUDCLNT_E_EXCLUSIVE_MODE_NOT_ALLOWED ((HRESULT)0x8889000E)
19761 | #define MA_AUDCLNT_E_ENDPOINT_CREATE_FAILED ((HRESULT)0x8889000F)
19762 | #define MA_AUDCLNT_E_SERVICE_NOT_RUNNING ((HRESULT)0x88890010)
19763 | #define MA_AUDCLNT_E_EVENTHANDLE_NOT_EXPECTED ((HRESULT)0x88890011)
19764 | #define MA_AUDCLNT_E_EXCLUSIVE_MODE_ONLY ((HRESULT)0x88890012)
19765 | #define MA_AUDCLNT_E_BUFDURATION_PERIOD_NOT_EQUAL ((HRESULT)0x88890013)
19766 | #define MA_AUDCLNT_E_EVENTHANDLE_NOT_SET ((HRESULT)0x88890014)
19767 | #define MA_AUDCLNT_E_INCORRECT_BUFFER_SIZE ((HRESULT)0x88890015)
19768 | #define MA_AUDCLNT_E_BUFFER_SIZE_ERROR ((HRESULT)0x88890016)
19769 | #define MA_AUDCLNT_E_CPUUSAGE_EXCEEDED ((HRESULT)0x88890017)

```

```

19770 | #define MA_AUDCLNT_E_BUFFER_ERROR ((HRESULT)0x88890018)
19771 | #define MA_AUDCLNT_E_BUFFER_SIZE_NOT_ALIGNED ((HRESULT)0x88890019)
19772 | #define MA_AUDCLNT_E_INVALID_DEVICE_PERIOD ((HRESULT)0x88890020)
19773 | #define MA_AUDCLNT_E_INVALID_STREAM_FLAG ((HRESULT)0x88890021)
19774 | #define MA_AUDCLNT_E_ENDPOINT_OFFLOAD_NOT_CAPABLE ((HRESULT)0x88890022)
19775 | #define MA_AUDCLNT_E_OUT_OF_OFFLOAD_RESOURCES ((HRESULT)0x88890023)
19776 | #define MA_AUDCLNT_E_OFFLOAD_MODE_ONLY ((HRESULT)0x88890024)
19777 | #define MA_AUDCLNT_E_NONOFFLOAD_MODE_ONLY ((HRESULT)0x88890025)
19778 | #define MA_AUDCLNT_E_RESOURCES_INVALIDATED ((HRESULT)0x88890026)
19779 | #define MA_AUDCLNT_E_RAW_MODE_UNSUPPORTED ((HRESULT)0x88890027)
19780 | #define MA_AUDCLNT_E_ENGINE_PERIODICITY_LOCKED ((HRESULT)0x88890028)
19781 | #define MA_AUDCLNT_E_ENGINE_FORMAT_LOCKED ((HRESULT)0x88890029)
19782 | #define MA_AUDCLNT_E_HEADTRACKING_ENABLED ((HRESULT)0x88890030)
19783 | #define MA_AUDCLNT_E_HEADTRACKING_UNSUPPORTED ((HRESULT)0x88890040)
19784 | #define MA_AUDCLNT_S_BUFFER_EMPTY ((HRESULT)0x08890001)
19785 | #define MA_AUDCLNT_S_THREAD_ALREADY_REGISTERED ((HRESULT)0x08890002)
19786 | #define MA_AUDCLNT_S_POSITION_STALLED ((HRESULT)0x08890003)
19787 |
19788 | #define MA_DS_OK ((HRESULT)0)
19789 | #define MA_DS_NO_VIRTUALIZATION ((HRESULT)0x0878000A)
19790 | #define MA_DSERR_ALLOCATED ((HRESULT)0x8878000A)
19791 | #define MA_DSERR_CONTROLUNAVAIL ((HRESULT)0x8878001E)
19792 | #define MA_DSERR_INVALIDPARAM ((HRESULT)0x80070057) /*E_INVALIDARG*/
19793 | #define MA_DSERR_INVALIDCALL ((HRESULT)0x88780032)
19794 | #define MA_DSERR_GENERIC ((HRESULT)0x80004005) /*E_FAIL*/
19795 | #define MA_DSERR_PRIORLEVELNEEDED ((HRESULT)0x88780046)
19796 | #define MA_DSERR_OUTOFMEMORY ((HRESULT)0x8007000E) /*E_OUTOFMEMORY*/
19797 | #define MA_DSERR_BADFORMAT ((HRESULT)0x88780064)
19798 | #define MA_DSERR_UNSUPPORTED ((HRESULT)0x80004001) /*E_NOTIMPL*/
19799 | #define MA_DSERR_NODRIVER ((HRESULT)0x88780078)
19800 | #define MA_DSERR_ALREADYINITIALIZED ((HRESULT)0x88780082)
19801 | #define MA_DSERR_NOAGGREGATION ((HRESULT)0x80040110) /*CLASS_E_NOAGGREGATION*/
19802 | #define MA_DSERR_BUFFERLOST ((HRESULT)0x88780096)
19803 | #define MA_DSERR_OTHERAPPHASPRIO ((HRESULT)0x887800A0)
19804 | #define MA_DSERR_UNINITIALIZED ((HRESULT)0x887800AA)
19805 | #define MA_DSERR_NOINTERFACE ((HRESULT)0x80004002) /*E_NOINTERFACE*/
19806 | #define MA_DSERR_ACCESSDENIED ((HRESULT)0x80070005) /*E_ACCESSDENIED*/
19807 | #define MA_DSERR_BUFFERTOOSMALL ((HRESULT)0x887800B4)
19808 | #define MA_DSERR_DS8_REQUIRED ((HRESULT)0x887800BE)
19809 | #define MA_DSERR_SENDBUFFERLOOP ((HRESULT)0x887800C8)
19810 | #define MA_DSERR_BADSENDBUFFERGUID ((HRESULT)0x887800D2)
19811 | #define MA_DSERR_OBJECTNOTFOUND ((HRESULT)0x88781161)
19812 | #define MA_DSERR_FXUNAVAILABLE ((HRESULT)0x887800DC)
19813 |
19814 | static ma_result ma_result_from_HRESULT(HRESULT hr)
19815 | {
19816 |     switch (hr)
19817 |     {
19818 |         case NOERROR: return MA_SUCCESS;
19819 |         /*case S_OK: return MA_SUCCESS;*/
19820 |
19821 |         case E_POINTER: return MA_INVALID_ARGS;
19822 |         case E_UNEXPECTED: return MA_ERROR;
19823 |         case E_NOTIMPL: return MA_NOT_IMPLEMENTED;
19824 |         case E_OUTOFMEMORY: return MA_OUT_OF_MEMORY;
19825 |         case E_INVALIDARG: return MA_INVALID_ARGS;
19826 |         case E_NOINTERFACE: return MA_API_NOT_FOUND;
19827 |         case E_HANDLE: return MA_INVALID_ARGS;
19828 |         case E_ABORT: return MA_ERROR;
19829 |         case E_FAIL: return MA_ERROR;
19830 |         case E_ACCESSDENIED: return MA_ACCESS_DENIED;
19831 |
19832 |         /* WASAPI */
19833 |         case MA_AUDCLNT_E_NOT_INITIALIZED: return MA_DEVICE_NOT_INITIALIZED;
19834 |         case MA_AUDCLNT_E_ALREADY_INITIALIZED: return MA_DEVICE_ALREADY_INITIALIZED;
19835 |         case MA_AUDCLNT_E_WRONG_ENDPOINT_TYPE: return MA_INVALID_ARGS;
19836 |         case MA_AUDCLNT_E_DEVICE_INVALIDATED: return MA_UNAVAILABLE;
19837 |         case MA_AUDCLNT_E_NOT_STOPPED: return MA_DEVICE_NOT_STOPPED;
19838 |         case MA_AUDCLNT_E_BUFFER_TOO_LARGE: return MA_TOO_BIG;
19839 |         case MA_AUDCLNT_E_OUT_OF_ORDER: return MA_INVALID_OPERATION;
19840 |         case MA_AUDCLNT_E_UNSUPPORTED_FORMAT: return MA_FORMAT_NOT_SUPPORTED;
19841 |         case MA_AUDCLNT_E_INVALID_SIZE: return MA_INVALID_ARGS;
19842 |         case MA_AUDCLNT_E_DEVICE_IN_USE: return MA_BUSY;
19843 |         case MA_AUDCLNT_E_BUFFER_OPERATION_PENDING: return MA_INVALID_OPERATION;
19844 |         case MA_AUDCLNT_E_THREAD_NOT_REGISTERED: return MA_DOES_NOT_EXIST;
19845 |         case MA_AUDCLNT_E_NO_SINGLE_PROCESS: return MA_INVALID_OPERATION;
19846 |         case MA_AUDCLNT_E_EXCLUSIVE_MODE_NOT_ALLOWED: return MA_SHARE_MODE_NOT_SUPPORTED;
19847 |         case MA_AUDCLNT_E_ENDPOINT_CREATE_FAILED: return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
19848 |         case MA_AUDCLNT_E_SERVICE_NOT_RUNNING: return MA_NOT_CONNECTED;

```

```

19849 | case MA_AUDCLNT_E_EVENTHANDLE_NOT_EXPECTED: return MA_INVALID_ARGS;
19850 | case MA_AUDCLNT_E_EXCLUSIVE_MODE_ONLY: return MA_SHARE_MODE_NOT_SUPPORTED;
19851 | case MA_AUDCLNT_E_BUFURATION_PERIOD_NOT_EQUAL: return MA_INVALID_ARGS;
19852 | case MA_AUDCLNT_E_EVENTHANDLE_NOT_SET: return MA_INVALID_ARGS;
19853 | case MA_AUDCLNT_E_INCORRECT_BUFFER_SIZE: return MA_INVALID_ARGS;
19854 | case MA_AUDCLNT_E_BUFFER_SIZE_ERROR: return MA_INVALID_ARGS;
19855 | case MA_AUDCLNT_E_CPUUSAGE_EXCEEDED: return MA_ERROR;
19856 | case MA_AUDCLNT_E_BUFFER_ERROR: return MA_ERROR;
19857 | case MA_AUDCLNT_E_BUFFER_SIZE_NOT_ALIGNED: return MA_INVALID_ARGS;
19858 | case MA_AUDCLNT_E_INVALID_DEVICE_PERIOD: return MA_INVALID_ARGS;
19859 | case MA_AUDCLNT_E_INVALID_STREAM_FLAG: return MA_INVALID_ARGS;
19860 | case MA_AUDCLNT_E_ENDPOINT_OFFLOAD_NOT_CAPABLE: return MA_INVALID_OPERATION;
19861 | case MA_AUDCLNT_E_OUT_OF_OFFLOAD_RESOURCES: return MA_OUT_OF_MEMORY;
19862 | case MA_AUDCLNT_E_OFFLOAD_MODE_ONLY: return MA_INVALID_OPERATION;
19863 | case MA_AUDCLNT_E_NONOFFLOAD_MODE_ONLY: return MA_INVALID_OPERATION;
19864 | case MA_AUDCLNT_E_RESOURCES_INVALIDATED: return MA_INVALID_DATA;
19865 | case MA_AUDCLNT_E_RAW_MODE_UNSUPPORTED: return MA_INVALID_OPERATION;
19866 | case MA_AUDCLNT_E_ENGINE_PERIODICITY_LOCKED: return MA_INVALID_OPERATION;
19867 | case MA_AUDCLNT_E_ENGINE_FORMAT_LOCKED: return MA_INVALID_OPERATION;
19868 | case MA_AUDCLNT_E_HEADTRACKING_ENABLED: return MA_INVALID_OPERATION;
19869 | case MA_AUDCLNT_E_HEADTRACKING_UNSUPPORTED: return MA_INVALID_OPERATION;
19870 | case MA_AUDCLNT_S_BUFFER_EMPTY: return MA_NO_SPACE;
19871 | case MA_AUDCLNT_S_THREAD_ALREADY_REGISTERED: return MA_ALREADY_EXISTS;
19872 | case MA_AUDCLNT_S_POSITION_STALLED: return MA_ERROR;
19873 |
19874 | /* DirectSound */
19875 | /*case MA_DS_OK: return MA_SUCCESS;*/ /* S_OK */
19876 | case MA_DS_NO_VIRTUALIZATION: return MA_SUCCESS;
19877 | case MA_DSERR_ALLOCATED: return MA_ALREADY_IN_USE;
19878 | case MA_DSERR_CONTROLUNAVAIL: return MA_INVALID_OPERATION;
19879 | /*case MA_DSERR_INVALIDPARAM: return MA_INVALID_ARGS;*/ /*
> E_INVALIDARG */
19880 | case MA_DSERR_INVALIDCALL: return MA_INVALID_OPERATION;
19881 | /*case MA_DSERR_GENERIC: return MA_ERROR;*/ /* E_FAIL */
19882 | case MA_DSERR_PRIORLEVELNEEDED: return MA_INVALID_OPERATION;
19883 | /*case MA_DSERR_OUTOFMEMORY: return MA_OUT_OF_MEMORY;*/ /*
> E_OUTOFMEMORY */
19884 | case MA_DSERR_BADFORMAT: return MA_FORMAT_NOT_SUPPORTED;
19885 | /*case MA_DSERR_UNSUPPORTED: return MA_NOT_IMPLEMENTED;*/ /* E_NOTIMPL
> */
19886 | case MA_DSERR_NODRIVER: return MA_FAILED_TO_INIT_BACKEND;
19887 | case MA_DSERR_ALREADYINITIALIZED: return MA_DEVICE_ALREADY_INITIALIZED;
19888 | case MA_DSERR_NOAGGREGATION: return MA_ERROR;
19889 | case MA_DSERR_BUFFERLOST: return MA_UNAVAILABLE;
19890 | case MA_DSERR_OTHERAPPHASPRIO: return MA_ACCESS_DENIED;
19891 | case MA_DSERR_UNINITIALIZED: return MA_DEVICE_NOT_INITIALIZED;
19892 | /*case MA_DSERR_NOINTERFACE: return MA_API_NOT_FOUND;*/ /*
> E_NOINTERFACE */
19893 | /*case MA_DSERR_ACCESSDENIED: return MA_ACCESS_DENIED;*/ /*
> E_ACCESSDENIED */
19894 | case MA_DSERR_BUFFERTOOSMALL: return MA_NO_SPACE;
19895 | case MA_DSERR_DS8_REQUIRED: return MA_INVALID_OPERATION;
19896 | case MA_DSERR_SENDBUFFERGUID: return MA_DEADLOCK;
19897 | case MA_DSERR_BADSENDBUFFERGUID: return MA_INVALID_ARGS;
19898 | case MA_DSERR_OBJECTNOTFOUND: return MA_NO_DEVICE;
19899 | case MA_DSERR_FXUNAVAILABLE: return MA_UNAVAILABLE;
19900 |
19901 | default: return MA_ERROR;
19902 | }
19903 | }
19904 |
19905 | /* PROPVARIANT */
19906 | #define MA_VT_LPWSTR 31
19907 | #define MA_VT_BLOB 65
19908 |
19909 | #if defined(_MSC_VER) && !defined(__clang__)
19910 | #pragma warning(push)
19911 | #pragma warning(disable:4201) /* nonstandard extension used: nameless struct/union */
19912 | #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__
> = 8)))
19913 | #pragma GCC diagnostic push
19914 | #pragma GCC diagnostic ignored "-Wpedantic" /* For ISO C99 doesn't support unnamed
> structs/unions [-Wpedantic] */
19915 | #if defined(__clang__)
19916 | #pragma GCC diagnostic ignored "-Wc11-extensions" /* anonymous unions are a C11 extension
> */
19917 | #endif
19918 | #endif
19919 | typedef struct

```

```

19920 | {
19921 |     WORD vt;
19922 |     WORD wReserved1;
19923 |     WORD wReserved2;
19924 |     WORD wReserved3;
19925 |     union
19926 |     {
19927 |         struct
19928 |         {
19929 |             ULONG cbSize;
19930 |             BYTE* pBlobData;
19931 |         } blob;
19932 |         WCHAR* pwszVal;
19933 |         char pad[16]; /* Just to ensure the size of the struct matches the official version. */
19934 |     };
19935 | } MA_PROPVARIANT;
19936 | #if defined(_MSC_VER) && !defined(__clang__)
19937 |     #pragma warning(pop)
19938 | #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__
> >= 8)))
19939 |     #pragma GCC diagnostic pop
19940 | #endif
19941 |
19942 | typedef HRESULT (WINAPI * MA_PFN_CoInitialize)(void* pvReserved);
19943 | typedef HRESULT (WINAPI * MA_PFN_CoInitializeEx)(void* pvReserved, DWORD dwCoInit);
19944 | typedef void (WINAPI * MA_PFN_CoUninitialize)(void);
19945 | typedef HRESULT (WINAPI * MA_PFN_CoCreateInstance)(const IID* rclsid, void* pUnkOuter, DWORD
> dwClsContext, const IID* riid, void* ppv);
19946 | typedef void (WINAPI * MA_PFN_CoTaskMemFree)(void* pv);
19947 | typedef HRESULT (WINAPI * MA_PFN_PropVariantClear)(MA_PROPVARIANT *pvar);
19948 | typedef int (WINAPI * MA_PFN_StringFromGUID2)(const GUID* const rguid, WCHAR* lpsz, int cchMax);
19949 |
19950 | typedef HWND (WINAPI * MA_PFN_GetForegroundWindow)(void);
19951 | typedef HWND (WINAPI * MA_PFN_GetDesktopWindow)(void);
19952 |
19953 | #if defined(MA_WIN32_DESKTOP)
19954 | /* Microsoft documents these APIs as returning LSTATUS, but the Win32 API shipping with some
> compilers do not define it. It's just a LONG. */
19955 | typedef LONG (WINAPI * MA_PFN_RegOpenKeyExA)(HKEY hKey, const char* lpSubKey, DWORD ulOptions,
> DWORD samDesired, HKEY* phkResult);
19956 | typedef LONG (WINAPI * MA_PFN_RegCloseKey)(HKEY hKey);
19957 | typedef LONG (WINAPI * MA_PFN_RegQueryValueExA)(HKEY hKey, const char* lpValueName, DWORD*
> lpReserved, DWORD* lpType, BYTE* lpData, DWORD* lpcbData);
19958 | #endif /* MA_WIN32_DESKTOP */
19959 |
19960 | static GUID MA_GUID_KSDATAFORMAT_SUBTYPE_PCM = {0x00000001, 0x0000, 0x0010, {0x80, 0x00,
> 0x00, 0xaa, 0x00, 0x38, 0x9b, 0x71}};
19961 | static GUID MA_GUID_KSDATAFORMAT_SUBTYPE_IEEE_FLOAT = {0x00000003, 0x0000, 0x0010, {0x80, 0x00,
> 0x00, 0xaa, 0x00, 0x38, 0x9b, 0x71}};
19962 | /*static GUID MA_GUID_KSDATAFORMAT_SUBTYPE_ALAW = {0x00000006, 0x0000, 0x0010, {0x80, 0x00,
> 0x00, 0xaa, 0x00, 0x38, 0x9b, 0x71}};*/
19963 | /*static GUID MA_GUID_KSDATAFORMAT_SUBTYPE_MULAW = {0x00000007, 0x0000, 0x0010, {0x80, 0x00,
> 0x00, 0xaa, 0x00, 0x38, 0x9b, 0x71}};*/
19964 |
19965 | MA_API size_t ma_strlen_WCHAR(const WCHAR* str)
19966 | {
19967 |     size_t len = 0;
19968 |     while (str[len] != '\0') {
19969 |         len += 1;
19970 |     }
19971 |
19972 |     return len;
19973 | }
19974 |
19975 | MA_API int ma_strcmp_WCHAR(const WCHAR *s1, const WCHAR *s2)
19976 | {
19977 |     while (*s1 != '\0' && *s1 == *s2) {
19978 |         s1 += 1;
19979 |         s2 += 1;
19980 |     }
19981 |
19982 |     return *s1 - *s2;
19983 | }
19984 |
19985 | MA_API int ma_strcpy_s_WCHAR(WCHAR* dst, size_t dstCap, const WCHAR* src)
19986 | {
19987 |     size_t i;
19988 |
19989 |     if (dst == 0) {

```



```

19990 |         return 22;
19991 |     }
19992 |     if (dstCap == 0) {
19993 |         return 34;
19994 |     }
19995 |     if (src == 0) {
19996 |         dst[0] = '\0';
19997 |         return 22;
19998 |     }
19999 |
20000 |     for (i = 0; i < dstCap && src[i] != '\0'; ++i) {
20001 |         dst[i] = src[i];
20002 |     }
20003 |
20004 |     if (i < dstCap) {
20005 |         dst[i] = '\0';
20006 |         return 0;
20007 |     }
20008 |
20009 |     dst[0] = '\0';
20010 |     return 34;
20011 | }
20012 | #endif /* MA_WIN32 */
20013 |
20014 |
20015 | #define MA_DEFAULT_PLAYBACK_DEVICE_NAME    "Default Playback Device"
20016 | #define MA_DEFAULT_CAPTURE_DEVICE_NAME    "Default Capture Device"
20017 |
20018 |
20019 |
20020 |
20021 | /*****
20022 |
20023 | Timing
20024 |
20025 | *****/
20026 | #if defined(MA_WIN32) && !defined(MA_POSIX)
20027 |     static LARGE_INTEGER g_ma_TimerFrequency; /* <-- Initialized to zero since it's static. */
20028 |     static MA_INLINE void ma_timer_init(ma_timer* pTimer)
20029 |     {
20030 |         LARGE_INTEGER counter;
20031 |
20032 |         if (g_ma_TimerFrequency.QuadPart == 0) {
20033 |             QueryPerformanceFrequency(&g_ma_TimerFrequency);
20034 |         }
20035 |
20036 |         QueryPerformanceCounter(&counter);
20037 |         pTimer->counter = counter.QuadPart;
20038 |     }
20039 |
20040 |     static MA_INLINE double ma_timer_get_time_in_seconds(ma_timer* pTimer)
20041 |     {
20042 |         LARGE_INTEGER counter;
20043 |         if (!QueryPerformanceCounter(&counter)) {
20044 |             return 0;
20045 |         }
20046 |
20047 |         return (double)(counter.QuadPart - pTimer->counter) / g_ma_TimerFrequency.QuadPart;
20048 |     }
20049 | #elif defined(MA_APPLE) && (MAC_OS_X_VERSION_MIN_REQUIRED < 101200)
20050 |     static ma_uint64 g_ma_TimerFrequency = 0;
20051 |     static MA_INLINE void ma_timer_init(ma_timer* pTimer)
20052 |     {
20053 |         mach_timebase_info_data_t baseTime;
20054 |         mach_timebase_info(&baseTime);
20055 |         g_ma_TimerFrequency = (baseTime.denom * 1e9) / baseTime.numer;
20056 |
20057 |         pTimer->counter = mach_absolute_time();
20058 |     }
20059 |
20060 |     static MA_INLINE double ma_timer_get_time_in_seconds(ma_timer* pTimer)
20061 |     {
20062 |         ma_uint64 newTimeCounter = mach_absolute_time();
20063 |         ma_uint64 oldTimeCounter = pTimer->counter;
20064 |
20065 |         return (newTimeCounter - oldTimeCounter) / g_ma_TimerFrequency;
20066 |     }
20067 | #elif defined(MA_EMSCRIPTEEN)
20068 |     static MA_INLINE void ma_timer_init(ma_timer* pTimer)

```

```

20069 |     {
20070 |         pTimer->counterD = emscripten_get_now();
20071 |     }
20072 |
20073 |     static MA_INLINE double ma_timer_get_time_in_seconds(ma_timer* pTimer)
20074 |     {
20075 |         return (emscripten_get_now() - pTimer->counterD) / 1000;    /* Emscripten is in
> milliseconds. */
20076 |     }
20077 | #else
20078 |     #if defined(_POSIX_C_SOURCE) && _POSIX_C_SOURCE >= 199309L
20079 |         #if defined(CLOCK_MONOTONIC)
20080 |             #define MA_CLOCK_ID CLOCK_MONOTONIC
20081 |         #else
20082 |             #define MA_CLOCK_ID CLOCK_REALTIME
20083 |         #endif
20084 |
20085 |         static MA_INLINE void ma_timer_init(ma_timer* pTimer)
20086 |         {
20087 |             struct timespec newTime;
20088 |             clock_gettime(MA_CLOCK_ID, &newTime);
20089 |
20090 |             pTimer->counter = ((ma_int64)newTime.tv_sec * 1000000000) + newTime.tv_nsec;
20091 |         }
20092 |
20093 |         static MA_INLINE double ma_timer_get_time_in_seconds(ma_timer* pTimer)
20094 |         {
20095 |             ma_uint64 newTimeCounter;
20096 |             ma_uint64 oldTimeCounter;
20097 |
20098 |             struct timespec newTime;
20099 |             clock_gettime(MA_CLOCK_ID, &newTime);
20100 |
20101 |             newTimeCounter = ((ma_uint64)newTime.tv_sec * 1000000000) + newTime.tv_nsec;
20102 |             oldTimeCounter = pTimer->counter;
20103 |
20104 |             return (newTimeCounter - oldTimeCounter) / 1000000000.0;
20105 |         }
20106 |     #else
20107 |         static MA_INLINE void ma_timer_init(ma_timer* pTimer)
20108 |         {
20109 |             struct timeval newTime;
20110 |             gettimeofday(&newTime, NULL);
20111 |
20112 |             pTimer->counter = ((ma_int64)newTime.tv_sec * 1000000) + newTime.tv_usec;
20113 |         }
20114 |
20115 |         static MA_INLINE double ma_timer_get_time_in_seconds(ma_timer* pTimer)
20116 |         {
20117 |             ma_uint64 newTimeCounter;
20118 |             ma_uint64 oldTimeCounter;
20119 |
20120 |             struct timeval newTime;
20121 |             gettimeofday(&newTime, NULL);
20122 |
20123 |             newTimeCounter = ((ma_uint64)newTime.tv_sec * 1000000) + newTime.tv_usec;
20124 |             oldTimeCounter = pTimer->counter;
20125 |
20126 |             return (newTimeCounter - oldTimeCounter) / 1000000.0;
20127 |         }
20128 |     #endif
20129 | #endif
20130 |
20131 |
20132 |
20133 | #if 0
20134 | static ma_uint32 ma_get_closest_standard_sample_rate(ma_uint32 sampleRateIn)
20135 | {
20136 |     ma_uint32 closestRate = 0;
20137 |     ma_uint32 closestDiff = 0xFFFFFFFF;
20138 |     size_t iStandardRate;
20139 |
20140 |     for (iStandardRate = 0; iStandardRate < ma_countof(g_maStandardSampleRatePriorities);
> ++iStandardRate) {
20141 |         ma_uint32 standardRate = g_maStandardSampleRatePriorities[iStandardRate];
20142 |         ma_uint32 diff;
20143 |
20144 |         if (sampleRateIn > standardRate) {
20145 |             diff = sampleRateIn - standardRate;

```

```

20146 |         } else {
20147 |             diff = standardRate - sampleRateIn;
20148 |         }
20149 |
20150 |         if (diff == 0) {
20151 |             return standardRate;    /* The input sample rate is a standard rate. */
20152 |         }
20153 |
20154 |         if (closestDiff > diff) {
20155 |             closestDiff = diff;
20156 |             closestRate = standardRate;
20157 |         }
20158 |     }
20159 |
20160 |     return closestRate;
20161 | }
20162 | #endif
20163 |
20164 |
20165 | static MA_INLINE unsigned int ma_device_disable_denormals(ma_device* pDevice)
20166 | {
20167 |     MA_ASSERT(pDevice != NULL);
20168 |
20169 |     if (!pDevice->noDisableDenormals) {
20170 |         return ma_disable_denormals();
20171 |     } else {
20172 |         return 0;
20173 |     }
20174 | }
20175 |
20176 | static MA_INLINE void ma_device_restore_denormals(ma_device* pDevice, unsigned int prevState)
20177 | {
20178 |     MA_ASSERT(pDevice != NULL);
20179 |
20180 |     if (!pDevice->noDisableDenormals) {
20181 |         ma_restore_denormals(prevState);
20182 |     } else {
20183 |         /* Do nothing. */
20184 |         (void)prevState;
20185 |     }
20186 | }
20187 |
20188 | static ma_device_notification ma_device_notification_init(ma_device* pDevice,
> ma_device_notification_type type)
20189 | {
20190 |     ma_device_notification notification;
20191 |
20192 |     MA_ZERO_OBJECT(&notification);
20193 |     notification.pDevice = pDevice;
20194 |     notification.type     = type;
20195 |
20196 |     return notification;
20197 | }
20198 |
20199 | static void ma_device__on_notification(ma_device_notification notification)
20200 | {
20201 |     MA_ASSERT(notification.pDevice != NULL);
20202 |
20203 |     if (notification.pDevice->onNotification != NULL) {
20204 |         notification.pDevice->onNotification(&notification);
20205 |     }
20206 |
20207 |     /* TEMP FOR COMPATIBILITY: If it's a stopped notification, fire the onStop callback as well.
> This is only for backwards compatibility and will be removed. */
20208 |     if (notification.pDevice->onStop != NULL && notification.type ==
> ma_device_notification_type_stopped) {
20209 |         notification.pDevice->onStop(notification.pDevice);
20210 |     }
20211 | }
20212 |
20213 | static void ma_device__on_notification_started(ma_device* pDevice)
20214 | {
20215 |     ma_device__on_notification(ma_device_notification_init(pDevice,
> ma_device_notification_type_started));
20216 | }
20217 |
20218 | static void ma_device__on_notification_stopped(ma_device* pDevice)
20219 | {
20220 |     ma_device__on_notification(ma_device_notification_init(pDevice,

```

```

> ma_device_notification_type_stopped));
20221 | }
20222 |
20223 | /* Not all platforms support reroute notifications. */
20224 | #if !defined(MA_EMSCRIPTEN)
20225 | static void ma_device__on_notification_rerouted(ma_device* pDevice)
20226 | {
20227 |     ma_device__on_notification(ma_device_notification_init(pDevice,
> ma_device_notification_type_rerouted));
20228 | }
20229 | #endif
20230 |
20231 | #if defined(MA_EMSCRIPTEN)
20232 | #ifdef __cplusplus
20233 | extern "C" {
20234 | #endif
20235 | void EMSCRIPTEN_KEEPALIVE ma_device__on_notification_unlocked(ma_device* pDevice)
20236 | {
20237 |     ma_device__on_notification(ma_device_notification_init(pDevice,
> ma_device_notification_type_unlocked));
20238 | }
20239 | #ifdef __cplusplus
20240 | }
20241 | #endif
20242 | #endif
20243 |
20244 |
20245 | static void ma_device__on_data_inner(ma_device* pDevice, void* pFramesOut, const void* pFramesIn,
> ma_uint32 frameCount)
20246 | {
20247 |     MA_ASSERT(pDevice != NULL);
20248 |     MA_ASSERT(pDevice->onData != NULL);
20249 |
20250 |     if (!pDevice->noPreSilencedOutputBuffer && pFramesOut != NULL) {
20251 |         ma_silence_pcm_frames(pFramesOut, frameCount, pDevice->playback.format,
> pDevice->playback.channels);
20252 |     }
20253 |
20254 |     pDevice->onData(pDevice, pFramesOut, pFramesIn, frameCount);
20255 | }
20256 |
20257 | static void ma_device__on_data(ma_device* pDevice, void* pFramesOut, const void* pFramesIn,
> ma_uint32 frameCount)
20258 | {
20259 |     MA_ASSERT(pDevice != NULL);
20260 |
20261 |     /* Don't read more data from the client if we're in the process of stopping. */
20262 |     if (ma_device_get_state(pDevice) == ma_device_state_stopping) {
20263 |         return;
20264 |     }
20265 |
20266 |     if (pDevice->noFixedSizedCallback) {
20267 |         /* Fast path. Not using a fixed sized callback. Process directly from the specified buffers.
> */
20268 |         ma_device__on_data_inner(pDevice, pFramesOut, pFramesIn, frameCount);
20269 |     } else {
20270 |         /* Slow path. Using a fixed sized callback. Need to use the intermediary buffer. */
20271 |         ma_uint32 totalFramesProcessed = 0;
20272 |
20273 |         while (totalFramesProcessed < frameCount) {
20274 |             ma_uint32 totalFramesRemaining = frameCount - totalFramesProcessed;
20275 |             ma_uint32 framesToProcessThisIteration = 0;
20276 |
20277 |             if (pFramesIn != NULL) {
20278 |                 /* Capturing. Write to the intermediary buffer. If there's no room, fire the
> callback to empty it. */
20279 |                 if (pDevice->capture.intermediaryBufferLen < pDevice->capture.intermediaryBufferCap)
> {
20280 |                     /* There's some room left in the intermediary buffer. Write to it without firing
> the callback. */
20281 |                     framesToProcessThisIteration = totalFramesRemaining;
20282 |                     if (framesToProcessThisIteration > pDevice->capture.intermediaryBufferCap -
> pDevice->capture.intermediaryBufferLen) {
20283 |                         framesToProcessThisIteration = pDevice->capture.intermediaryBufferCap -
> pDevice->capture.intermediaryBufferLen;
20284 |                     }
20285 |
20286 |                     ma_copy_pcm_frames(
20287 |                         ma_offset_pcm_frames_ptr(pDevice->capture.pIntermediaryBuffer,

```

```

20288 |         pDevice->capture.intermediaryBufferLen, pDevice->capture.format, pDevice->capture.channels),
20289 |         ma_offset_pcm_frames_const_ptr(pFramesIn, totalFramesProcessed,
20290 |         pDevice->capture.format, pDevice->capture.channels),
20291 |         framesToProcessThisIteration,
20292 |         pDevice->capture.format, pDevice->capture.channels);
20293 |     }
20294 |
20295 |     if (pDevice->capture.intermediaryBufferLen ==
20296 |         pDevice->capture.intermediaryBufferCap) {
20297 |         /* No room left in the intermediary buffer. Fire the data callback. */
20298 |         if (pDevice->type == ma_device_type_duplex) {
20299 |             /* We'll do the duplex data callback later after we've processed the
20300 |             playback data. */
20301 |             } else {
20302 |                 ma_device__on_data_inner(pDevice, NULL,
20303 |                 pDevice->capture.pIntermediaryBuffer, pDevice->capture.intermediaryBufferCap);
20304 |             }
20305 |         }
20306 |     }
20307 |
20308 |     if (pFramesOut != NULL) {
20309 |         /* Playing back. Read from the intermediary buffer. If there's nothing in it, fire
20310 |         the callback to fill it. */
20311 |         if (pDevice->playback.intermediaryBufferLen > 0) {
20312 |             /* There's some content in the intermediary buffer. Read from that without
20313 |             firing the callback. */
20314 |             if (pDevice->type == ma_device_type_duplex) {
20315 |                 /* The frames processed this iteration for a duplex device will always be
20316 |                 based on the capture side. Leave it unmodified. */
20317 |             } else {
20318 |                 framesToProcessThisIteration = totalFramesRemaining;
20319 |                 if (framesToProcessThisIteration > pDevice->playback.intermediaryBufferLen)
20320 |                 {
20321 |                     framesToProcessThisIteration = pDevice->playback.intermediaryBufferLen;
20322 |                 }
20323 |             }
20324 |             ma_copy_pcm_frames(
20325 |                 ma_offset_pcm_frames_ptr(pFramesOut, totalFramesProcessed,
20326 |                 pDevice->playback.format, pDevice->playback.channels),
20327 |                 ma_offset_pcm_frames_ptr(pDevice->playback.pIntermediaryBuffer,
20328 |                 pDevice->playback.intermediaryBufferCap - pDevice->playback.intermediaryBufferLen,
20329 |                 pDevice->playback.format, pDevice->playback.channels),
20330 |                 framesToProcessThisIteration,
20331 |                 pDevice->playback.format, pDevice->playback.channels);
20332 |             pDevice->playback.intermediaryBufferLen -= framesToProcessThisIteration;
20333 |         }
20334 |         if (pDevice->playback.intermediaryBufferLen == 0) {
20335 |             /* There's nothing in the intermediary buffer. Fire the data callback to fill
20336 |             it. */
20337 |             if (pDevice->type == ma_device_type_duplex) {
20338 |                 /* In duplex mode, the data callback will be fired later. Nothing to do
20339 |                 here. */
20340 |             } else {
20341 |                 ma_device__on_data_inner(pDevice, pDevice->playback.pIntermediaryBuffer,
20342 |                 NULL, pDevice->playback.intermediaryBufferCap);
20343 |                 /* The intermediary buffer has just been filled. */
20344 |                 pDevice->playback.intermediaryBufferLen =
20345 |                 pDevice->playback.intermediaryBufferCap;
20346 |             }
20347 |         }
20348 |     }
20349 |
20350 |     /* If we're in duplex mode we might need to do a refill of the data. */
20351 |     if (pDevice->type == ma_device_type_duplex) {
20352 |         if (pDevice->capture.intermediaryBufferLen ==
20353 |             pDevice->capture.intermediaryBufferCap) {
20354 |             ma_device__on_data_inner(pDevice, pDevice->playback.pIntermediaryBuffer,
20355 |             pDevice->capture.pIntermediaryBuffer, pDevice->capture.intermediaryBufferCap);
20356 |             pDevice->playback.intermediaryBufferLen =

```

```

> pDevice->playback.intermediaryBufferCap; /* The playback buffer will have just been filled. */
20349 |         pDevice->capture.intermediaryBufferLen = 0;
>         /* The intermediary buffer has just been drained. */
20350 |     }
20351 | }
20352 |
20353 |     /* Make sure this is only incremented once in the duplex case. */
20354 |     totalFramesProcessed += framesToProcessThisIteration;
20355 | }
20356 | }
20357 | }
20358 |
20359 | static void ma_device__handle_data_callback(ma_device* pDevice, void* pFramesOut, const void*
> pFramesIn, ma_uint32 frameCount)
20360 | {
20361 |     float masterVolumeFactor;
20362 |
20363 |     ma_device_get_master_volume(pDevice, &masterVolumeFactor); /* Use ma_device_get_master_volume()
> to ensure the volume is loaded atomically. */
20364 |
20365 |     if (pDevice->onData) {
20366 |         unsigned int prevDenormalState = ma_device_disable_denormals(pDevice);
20367 |         {
20368 |             /* Volume control of input makes things a bit awkward because the input buffer is
> read-only. We'll need to use a temp buffer and loop in this case. */
20369 |             if (pFramesIn != NULL && masterVolumeFactor != 1) {
20370 |                 ma_uint8 tempFramesIn[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20371 |                 ma_uint32 bpfCapture = ma_get_bytes_per_frame(pDevice->capture.format,
> pDevice->capture.channels);
20372 |                 ma_uint32 bpfPlayback = ma_get_bytes_per_frame(pDevice->playback.format,
> pDevice->playback.channels);
20373 |                 ma_uint32 totalFramesProcessed = 0;
20374 |                 while (totalFramesProcessed < frameCount) {
20375 |                     ma_uint32 framesToProcessThisIteration = frameCount - totalFramesProcessed;
20376 |                     if (framesToProcessThisIteration > sizeof(tempFramesIn)/bpfCapture) {
20377 |                         framesToProcessThisIteration = sizeof(tempFramesIn)/bpfCapture;
20378 |                     }
20379 |
20380 |                     ma_copy_and_apply_volume_factor_pcm_frames(tempFramesIn,
> ma_offset_ptr(pFramesIn, totalFramesProcessed*bpfCapture), framesToProcessThisIteration,
> pDevice->capture.format, pDevice->capture.channels, masterVolumeFactor);
20381 |
20382 |                     ma_device__on_data(pDevice, ma_offset_ptr(pFramesOut,
> totalFramesProcessed*bpfPlayback), tempFramesIn, framesToProcessThisIteration);
20383 |
20384 |                     totalFramesProcessed += framesToProcessThisIteration;
20385 |                 }
20386 |             } else {
20387 |                 ma_device__on_data(pDevice, pFramesOut, pFramesIn, frameCount);
20388 |             }
20389 |
20390 |             /* Volume control and clipping for playback devices. */
20391 |             if (pFramesOut != NULL) {
20392 |                 if (masterVolumeFactor != 1) {
20393 |                     if (pFramesIn == NULL) { /* <-- In full-duplex situations, the volume will
> have been applied to the input samples before the data callback. Applying it again post-callback
> will incorrectly compound it. */
20394 |                         ma_apply_volume_factor_pcm_frames(pFramesOut, frameCount,
> pDevice->playback.format, pDevice->playback.channels, masterVolumeFactor);
20395 |                     }
20396 |                 }
20397 |
20398 |                 if (!pDevice->noClip && pDevice->playback.format == ma_format_f32) {
20399 |                     ma_clip_samples_f32((float*)pFramesOut, (const float*)pFramesOut, frameCount *
> pDevice->playback.channels); /* Intentionally specifying the same pointer for both input and
> output for in-place processing. */
20400 |                 }
20401 |             }
20402 |         }
20403 |         ma_device_restore_denormals(pDevice, prevDenormalState);
20404 |     } else {
20405 |         /* No data callback. Just silence the output. */
20406 |         if (pFramesOut != NULL) {
20407 |             ma_silence_pcm_frames(pFramesOut, frameCount, pDevice->playback.format,
> pDevice->playback.channels);
20408 |         }
20409 |     }
20410 | }
20411 |

```

```

20412 |
20413 |
20414 | /* A helper function for reading sample data from the client. */
20415 | static void ma_device__read_frames_from_client(ma_device* pDevice, ma_uint32 frameCount, void*
> pFramesOut)
20416 | {
20417 |     MA_ASSERT(pDevice != NULL);
20418 |     MA_ASSERT(frameCount > 0);
20419 |     MA_ASSERT(pFramesOut != NULL);
20420 |
20421 |     if (pDevice->playback.converter.isPassthrough) {
20422 |         ma_device__handle_data_callback(pDevice, pFramesOut, NULL, frameCount);
20423 |     } else {
20424 |         ma_result result;
20425 |         ma_uint64 totalFramesReadOut;
20426 |         void* pRunningFramesOut;
20427 |
20428 |         totalFramesReadOut = 0;
20429 |         pRunningFramesOut = pFramesOut;
20430 |
20431 |         /*
20432 |         We run slightly different logic depending on whether or not we're using a heap-allocated
20433 |         buffer for caching input data. This will be the case if the data converter does not have
20434 |         the ability to retrieve the required input frame count for a given output frame count.
20435 |         */
20436 |         if (pDevice->playback.pInputCache != NULL) {
20437 |             while (totalFramesReadOut < frameCount) {
20438 |                 ma_uint64 framesToReadThisIterationIn;
20439 |                 ma_uint64 framesToReadThisIterationOut;
20440 |
20441 |                 /* If there's any data available in the cache, that needs to get processed first. */
20442 |                 if (pDevice->playback.inputCacheRemaining > 0) {
20443 |                     framesToReadThisIterationOut = (frameCount - totalFramesReadOut);
20444 |                     framesToReadThisIterationIn = framesToReadThisIterationOut;
20445 |                     if (framesToReadThisIterationIn > pDevice->playback.inputCacheRemaining) {
20446 |                         framesToReadThisIterationIn = pDevice->playback.inputCacheRemaining;
20447 |                     }
20448 |
20449 |                     result = ma_data_converter_process_pcm_frames(&pDevice->playback.converter,
> ma_offset_pcm_frames_ptr(pDevice->playback.pInputCache, pDevice->playback.inputCacheConsumed,
> pDevice->playback.format, pDevice->playback.channels), &framesToReadThisIterationIn,
> pRunningFramesOut, &framesToReadThisIterationOut);
20450 |                     if (result != MA_SUCCESS) {
20451 |                         break;
20452 |                     }
20453 |
20454 |                     pDevice->playback.inputCacheConsumed += framesToReadThisIterationIn;
20455 |                     pDevice->playback.inputCacheRemaining -= framesToReadThisIterationIn;
20456 |
20457 |                     totalFramesReadOut += framesToReadThisIterationOut;
20458 |                     pRunningFramesOut = ma_offset_ptr(pRunningFramesOut,
> framesToReadThisIterationOut * ma_get_bytes_per_frame(pDevice->playback.internalFormat,
> pDevice->playback.internalChannels));
20459 |
20460 |                     if (framesToReadThisIterationIn == 0 && framesToReadThisIterationOut == 0) {
20461 |                         break; /* We're done. */
20462 |                     }
20463 |                 }
20464 |             }
20465 |             /* Getting here means there's no data in the cache and we need to fill it up with
> data from the client. */
20466 |             if (pDevice->playback.inputCacheRemaining == 0) {
20467 |                 ma_device__handle_data_callback(pDevice, pDevice->playback.pInputCache, NULL,
> (ma_uint32)pDevice->playback.inputCacheCap);
20468 |
20469 |                 pDevice->playback.inputCacheConsumed = 0;
20470 |                 pDevice->playback.inputCacheRemaining = pDevice->playback.inputCacheCap;
20471 |             }
20472 |         }
20473 |     } else {
20474 |         while (totalFramesReadOut < frameCount) {
20475 |             ma_uint8 pIntermediaryBuffer[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In client
> format. */
20476 |             ma_uint64 intermediaryBufferCap = sizeof(pIntermediaryBuffer) /
> ma_get_bytes_per_frame(pDevice->playback.format, pDevice->playback.channels);
20477 |             ma_uint64 framesToReadThisIterationIn;
20478 |             ma_uint64 framesReadThisIterationIn;
20479 |             ma_uint64 framesToReadThisIterationOut;
20480 |             ma_uint64 framesReadThisIterationOut;

```



```

20481 |         ma_uint64 requiredInputFrameCount;
20482 |
20483 |         framesToReadThisIterationOut = (frameCount - totalFramesReadOut);
20484 |         framesToReadThisIterationIn = framesToReadThisIterationOut;
20485 |         if (framesToReadThisIterationIn > intermediaryBufferCap) {
20486 |             framesToReadThisIterationIn = intermediaryBufferCap;
20487 |         }
20488 |
20489 |         ma_data_converter_get_required_input_frame_count(&pDevice->playback.converter,
> framesToReadThisIterationOut, &requiredInputFrameCount);
20490 |         if (framesToReadThisIterationIn > requiredInputFrameCount) {
20491 |             framesToReadThisIterationIn = requiredInputFrameCount;
20492 |         }
20493 |
20494 |         ma_device__handle_data_callback(pDevice, pIntermediaryBuffer, NULL,
> (ma_uint32)framesToReadThisIterationIn);
20495 |
20496 |         /*
20497 |         At this point we have our decoded data in input format and now we need to convert to
> output format. Note that even if we didn't read any
20498 |         input frames, we still want to try processing frames because there may some output
> frames generated from cached input data.
20499 |         */
20500 |         framesReadThisIterationIn = framesToReadThisIterationIn;
20501 |         framesReadThisIterationOut = framesToReadThisIterationOut;
20502 |         result = ma_data_converter_process_pcm_frames(&pDevice->playback.converter,
> pIntermediaryBuffer, &framesReadThisIterationIn, pRunningFramesOut, &framesReadThisIterationOut);
20503 |         if (result != MA_SUCCESS) {
20504 |             break;
20505 |         }
20506 |
20507 |         totalFramesReadOut += framesReadThisIterationOut;
20508 |         pRunningFramesOut = ma_offset_ptr(pRunningFramesOut, framesReadThisIterationOut *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels));
20509 |
20510 |         if (framesReadThisIterationIn == 0 && framesReadThisIterationOut == 0) {
20511 |             break; /* We're done. */
20512 |         }
20513 |     }
20514 | }
20515 | }
20516 | }
20517 |
20518 | /* A helper for sending sample data to the client. */
20519 | static void ma_device__send_frames_to_client(ma_device* pDevice, ma_uint32 frameCountInDeviceFormat,
> const void* pFramesInDeviceFormat)
20520 | {
20521 |     MA_ASSERT(pDevice != NULL);
20522 |     MA_ASSERT(frameCountInDeviceFormat > 0);
20523 |     MA_ASSERT(pFramesInDeviceFormat != NULL);
20524 |
20525 |     if (pDevice->capture.converter.isPassthrough) {
20526 |         ma_device__handle_data_callback(pDevice, NULL, pFramesInDeviceFormat,
> frameCountInDeviceFormat);
20527 |     } else {
20528 |         ma_result result;
20529 |         ma_uint8 pFramesInClientFormat[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20530 |         ma_uint64 framesInClientFormatCap = sizeof(pFramesInClientFormat) /
> ma_get_bytes_per_frame(pDevice->capture.format, pDevice->capture.channels);
20531 |         ma_uint64 totalDeviceFramesProcessed = 0;
20532 |         ma_uint64 totalClientFramesProcessed = 0;
20533 |         const void* pRunningFramesInDeviceFormat = pFramesInDeviceFormat;
20534 |
20535 |         /* We just keep going until we've exhausted all of our input frames and cannot generate any
> more output frames. */
20536 |         for (;;) {
20537 |             ma_uint64 deviceFramesProcessedThisIteration;
20538 |             ma_uint64 clientFramesProcessedThisIteration;
20539 |
20540 |             deviceFramesProcessedThisIteration = (frameCountInDeviceFormat -
> totalDeviceFramesProcessed);
20541 |             clientFramesProcessedThisIteration = framesInClientFormatCap;
20542 |
20543 |             result = ma_data_converter_process_pcm_frames(&pDevice->capture.converter,
> pRunningFramesInDeviceFormat, &deviceFramesProcessedThisIteration, pFramesInClientFormat,
> &clientFramesProcessedThisIteration);
20544 |             if (result != MA_SUCCESS) {
20545 |                 break;
20546 |             }

```

```

20547 |
20548 |         if (clientFramesProcessedThisIteration > 0) {
20549 |             ma_device__handle_data_callback(pDevice, NULL, pFramesInClientFormat,
> (ma_uint32)clientFramesProcessedThisIteration); /* Safe cast. */
20550 |         }
20551 |
20552 |         pRunningFramesInDeviceFormat = ma_offset_ptr(pRunningFramesInDeviceFormat,
> deviceFramesProcessedThisIteration * ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels));
20553 |         totalDeviceFramesProcessed += deviceFramesProcessedThisIteration;
20554 |         totalClientFramesProcessed += clientFramesProcessedThisIteration;
20555 |
20556 |         /* This is just to silence a warning. I might want to use this variable later so leaving
> in place for now. */
20557 |         (void)totalClientFramesProcessed;
20558 |
20559 |         if (deviceFramesProcessedThisIteration == 0 && clientFramesProcessedThisIteration == 0)
> {
20560 |             break; /* We're done. */
20561 |         }
20562 |     }
20563 | }
20564 | }
20565 |
20566 | static ma_result ma_device__handle_duplex_callback_capture(ma_device* pDevice, ma_uint32
> frameCountInDeviceFormat, const void* pFramesInDeviceFormat, ma_pcm_rb* pRB)
20567 | {
20568 |     ma_result result;
20569 |     ma_uint32 totalDeviceFramesProcessed = 0;
20570 |     const void* pRunningFramesInDeviceFormat = pFramesInDeviceFormat;
20571 |
20572 |     MA_ASSERT(pDevice != NULL);
20573 |     MA_ASSERT(frameCountInDeviceFormat > 0);
20574 |     MA_ASSERT(pFramesInDeviceFormat != NULL);
20575 |     MA_ASSERT(pRB != NULL);
20576 |
20577 |     /* Write to the ring buffer. The ring buffer is in the client format which means we need to
> convert. */
20578 |     for (;;) {
20579 |         ma_uint32 framesToProcessInDeviceFormat = (frameCountInDeviceFormat -
> totalDeviceFramesProcessed);
20580 |         ma_uint32 framesToProcessInClientFormat = MA_DATA_CONVERTER_STACK_BUFFER_SIZE /
> ma_get_bytes_per_frame(pDevice->capture.format, pDevice->capture.channels);
20581 |         ma_uint64 framesProcessedInDeviceFormat;
20582 |         ma_uint64 framesProcessedInClientFormat;
20583 |         void* pFramesInClientFormat;
20584 |
20585 |         result = ma_pcm_rb_acquire_write(pRB, &framesToProcessInClientFormat,
> &pFramesInClientFormat);
20586 |         if (result != MA_SUCCESS) {
20587 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "Failed to acquire capture
> PCM frames from ring buffer.");
20588 |             break;
20589 |         }
20590 |
20591 |         if (framesToProcessInClientFormat == 0) {
20592 |             if (ma_pcm_rb_pointer_distance(pRB) == (ma_int32)ma_pcm_rb_get_subbuffer_size(pRB)) {
20593 |                 break; /* Overrun. Not enough room in the ring buffer for input frame. Excess
> frames are dropped. */
20594 |             }
20595 |         }
20596 |
20597 |         /* Convert. */
20598 |         framesProcessedInDeviceFormat = framesToProcessInDeviceFormat;
20599 |         framesProcessedInClientFormat = framesToProcessInClientFormat;
20600 |         result = ma_data_converter_process_pcm_frames(&pDevice->capture.converter,
> pRunningFramesInDeviceFormat, &framesProcessedInDeviceFormat, pFramesInClientFormat,
> &framesProcessedInClientFormat);
20601 |         if (result != MA_SUCCESS) {
20602 |             break;
20603 |         }
20604 |
20605 |         result = ma_pcm_rb_commit_write(pRB, (ma_uint32)framesProcessedInClientFormat); /* Safe
> cast. */
20606 |         if (result != MA_SUCCESS) {
20607 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "Failed to commit capture
> PCM frames to ring buffer.");
20608 |             break;
20609 |         }

```

```

20610 |
20611 |         pRunningFramesInDeviceFormat = ma_offset_ptr(pRunningFramesInDeviceFormat,
> framesProcessedInDeviceFormat * ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels));
20612 |         totalDeviceFramesProcessed += (ma_uint32)framesProcessedInDeviceFormat; /* Safe cast. */
20613 |
20614 |         /* We're done when we're unable to process any client nor device frames. */
20615 |         if (framesProcessedInClientFormat == 0 && framesProcessedInDeviceFormat == 0) {
20616 |             break; /* Done. */
20617 |         }
20618 |     }
20619 |
20620 |     return MA_SUCCESS;
20621 | }
20622 |
20623 | static ma_result ma_device__handle_duplex_callback_playback(ma_device* pDevice, ma_uint32
> frameCount, void* pFramesInInternalFormat, ma_pcm_rb* pRB)
20624 | {
20625 |     ma_result result;
20626 |     ma_uint8 silentInputFrames[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20627 |     ma_uint32 totalFramesReadOut = 0;
20628 |
20629 |     MA_ASSERT(pDevice != NULL);
20630 |     MA_ASSERT(frameCount > 0);
20631 |     MA_ASSERT(pFramesInInternalFormat != NULL);
20632 |     MA_ASSERT(pRB != NULL);
20633 |     MA_ASSERT(pDevice->playback.pInputCache != NULL);
20634 |
20635 |     /*
20636 |     Sitting in the ring buffer should be captured data from the capture callback in external format.
> If there's not enough data in there for
20637 |     the whole frameCount frames we just use silence instead for the input data.
20638 |     */
20639 |     MA_ZERO_MEMORY(silentInputFrames, sizeof(silentInputFrames));
20640 |
20641 |     while (totalFramesReadOut < frameCount && ma_device_is_started(pDevice)) {
20642 |         /*
20643 |         We should have a buffer allocated on the heap. Any playback frames still sitting in there
20644 |         need to be sent to the internal device before we process any more data from the client.
20645 |         */
20646 |         if (pDevice->playback.inputCacheRemaining > 0) {
20647 |             ma_uint64 framesConvertedIn = pDevice->playback.inputCacheRemaining;
20648 |             ma_uint64 framesConvertedOut = (frameCount - totalFramesReadOut);
20649 |             ma_data_converter_process_pcm_frames(&pDevice->playback.converter,
> ma_offset_pcm_frames_ptr(pDevice->playback.pInputCache, pDevice->playback.inputCacheConsumed,
> pDevice->playback.format, pDevice->playback.channels), &framesConvertedIn, pFramesInInternalFormat,
> &framesConvertedOut);
20650 |
20651 |             pDevice->playback.inputCacheConsumed += framesConvertedIn;
20652 |             pDevice->playback.inputCacheRemaining -= framesConvertedIn;
20653 |
20654 |             totalFramesReadOut += (ma_uint32)framesConvertedOut; /* Safe cast. */
20655 |             pFramesInInternalFormat = ma_offset_ptr(pFramesInInternalFormat, framesConvertedOut *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels));
20656 |         }
20657 |
20658 |         /* If there's no more data in the cache we'll need to fill it with some. */
20659 |         if (totalFramesReadOut < frameCount && pDevice->playback.inputCacheRemaining == 0) {
20660 |             ma_uint32 inputFrameCount;
20661 |             void* pInputFrames;
20662 |
20663 |             inputFrameCount = (ma_uint32)pDevice->playback.inputCacheCap;
20664 |             result = ma_pcm_rb_acquire_read(pRB, &inputFrameCount, &pInputFrames);
20665 |             if (result == MA_SUCCESS) {
20666 |                 if (inputFrameCount > 0) {
20667 |                     ma_device__handle_data_callback(pDevice, pDevice->playback.pInputCache,
> pInputFrames, inputFrameCount);
20668 |                 } else {
20669 |                     if (ma_pcm_rb_pointer_distance(pRB) == 0) {
20670 |                         break; /* Underrun. */
20671 |                     }
20672 |                 }
20673 |             } else {
20674 |                 /* No capture data available. Feed in silence. */
20675 |                 inputFrameCount = (ma_uint32)ma_min(pDevice->playback.inputCacheCap,
> sizeof(silentInputFrames) / ma_get_bytes_per_frame(pDevice->capture.format,
> pDevice->capture.channels));
20676 |                 ma_device__handle_data_callback(pDevice, pDevice->playback.pInputCache,
> silentInputFrames, inputFrameCount);

```

```

20677 |         }
20678 |
20679 |         pDevice->playback.inputCacheConsumed = 0;
20680 |         pDevice->playback.inputCacheRemaining = inputFrameCount;
20681 |
20682 |         result = ma_pcm_rb_commit_read(pRB, inputFrameCount);
20683 |         if (result != MA_SUCCESS) {
20684 |             return result; /* Should never happen. */
20685 |         }
20686 |     }
20687 | }
20688 |
20689 | return MA_SUCCESS;
20690 | }
20691 |
20692 | /* A helper for changing the state of the device. */
20693 | static MA_INLINE void ma_device__set_state(ma_device* pDevice, ma_device_state newState)
20694 | {
20695 |     ma_atomic_device_state_set(&pDevice->state, newState);
20696 | }
20697 |
20698 |
20699 |
20700 | MA_API ma_uint32 ma_get_format_priority_index(ma_format format) /* Lower = better. */
20701 | {
20702 |     ma_uint32 i;
20703 |     for (i = 0; i < ma_countof(g_maFormatPriorities); ++i) {
20704 |         if (g_maFormatPriorities[i] == format) {
20705 |             return i;
20706 |         }
20707 |     }
20708 |
20709 |     /* Getting here means the format could not be found or is equal to ma_format_unknown. */
20710 |     return (ma_uint32)-1;
20711 | }
20712 |
20713 | static ma_result ma_device__post_init_setup(ma_device* pDevice, ma_device_type deviceType);
20714 |
20715 | static ma_bool32 ma_device_descriptor_is_valid(const ma_device_descriptor* pDeviceDescriptor)
20716 | {
20717 |     if (pDeviceDescriptor == NULL) {
20718 |         return MA_FALSE;
20719 |     }
20720 |
20721 |     if (pDeviceDescriptor->format == ma_format_unknown) {
20722 |         return MA_FALSE;
20723 |     }
20724 |
20725 |     if (pDeviceDescriptor->channels == 0 || pDeviceDescriptor->channels > MA_MAX_CHANNELS) {
20726 |         return MA_FALSE;
20727 |     }
20728 |
20729 |     if (pDeviceDescriptor->sampleRate == 0) {
20730 |         return MA_FALSE;
20731 |     }
20732 |
20733 |     return MA_TRUE;
20734 | }
20735 |
20736 |
20737 | static ma_result ma_device_audio_thread__default_read_write(ma_device* pDevice)
20738 | {
20739 |     ma_result result = MA_SUCCESS;
20740 |     ma_bool32 exitLoop = MA_FALSE;
20741 |     ma_uint8 capturedDeviceData[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20742 |     ma_uint8 playbackDeviceData[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20743 |     ma_uint32 capturedDeviceDataCapInFrames = 0;
20744 |     ma_uint32 playbackDeviceDataCapInFrames = 0;
20745 |
20746 |     MA_ASSERT(pDevice != NULL);
20747 |
20748 |     /* Just some quick validation on the device type and the available callbacks. */
20749 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex ||
> pDevice->type == ma_device_type_loopback) {
20750 |         if (pDevice->pContext->callbacks.onDeviceRead == NULL) {
20751 |             return MA_NOT_IMPLEMENTED;
20752 |         }
20753 |
20754 |         capturedDeviceDataCapInFrames = sizeof(capturedDeviceData) /

```

```

> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels);
20755 |     }
20756 |
20757 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
20758 |         if (pDevice->pContext->callbacks.onDeviceWrite == NULL) {
20759 |             return MA_NOT_IMPLEMENTED;
20760 |         }
20761 |
20762 |         playbackDeviceDataCapInFrames = sizeof(playbackDeviceData) /
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
20763 |     }
20764 |
20765 |     /* NOTE: The device was started outside of this function, in the worker thread. */
20766 |
20767 |     while (ma_device_get_state(pDevice) == ma_device_state_started && !exitLoop) {
20768 |         switch (pDevice->type) {
20769 |             case ma_device_type_duplex:
20770 |             {
20771 |                 /* The process is: onDeviceRead() -> convert -> callback -> convert ->
> onDeviceWrite() */
20772 |                 ma_uint32 totalCapturedDeviceFramesProcessed = 0;
20773 |                 ma_uint32 capturedDevicePeriodSizeInFrames =
> ma_min(pDevice->capture.internalPeriodSizeInFrames, pDevice->playback.internalPeriodSizeInFrames);
20774 |
20775 |                 while (totalCapturedDeviceFramesProcessed < capturedDevicePeriodSizeInFrames) {
20776 |                     ma_uint32 capturedDeviceFramesRemaining;
20777 |                     ma_uint32 capturedDeviceFramesProcessed;
20778 |                     ma_uint32 capturedDeviceFramesToProcess;
20779 |                     ma_uint32 capturedDeviceFramesToTryProcessing = capturedDevicePeriodSizeInFrames
> - totalCapturedDeviceFramesProcessed;
20780 |                     if (capturedDeviceFramesToTryProcessing > capturedDeviceDataCapInFrames) {
20781 |                         capturedDeviceFramesToTryProcessing = capturedDeviceDataCapInFrames;
20782 |                     }
20783 |
20784 |                     result = pDevice->pContext->callbacks.onDeviceRead(pDevice, capturedDeviceData,
> capturedDeviceFramesToTryProcessing, &capturedDeviceFramesToProcess);
20785 |                     if (result != MA_SUCCESS) {
20786 |                         exitLoop = MA_TRUE;
20787 |                         break;
20788 |                     }
20789 |
20790 |                     capturedDeviceFramesRemaining = capturedDeviceFramesToProcess;
20791 |                     capturedDeviceFramesProcessed = 0;
20792 |
20793 |                     /* At this point we have our captured data in device format and we now need to
> convert it to client format. */
20794 |                     for (;;) {
20795 |                         ma_uint8 capturedClientData[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20796 |                         ma_uint8 playbackClientData[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
20797 |                         ma_uint32 capturedClientDataCapInFrames = sizeof(capturedClientData) /
> ma_get_bytes_per_frame(pDevice->capture.format, pDevice->capture.channels);
20798 |                         ma_uint32 playbackClientDataCapInFrames = sizeof(playbackClientData) /
> ma_get_bytes_per_frame(pDevice->playback.format, pDevice->playback.channels);
20799 |                         ma_uint64 capturedClientFramesToProcessThisIteration =
> ma_min(capturedClientDataCapInFrames, playbackClientDataCapInFrames);
20800 |                         ma_uint64 capturedDeviceFramesToProcessThisIteration =
> capturedDeviceFramesRemaining;
20801 |                         ma_uint8* pRunningCapturedDeviceFrames = ma_offset_ptr(capturedDeviceData,
> capturedDeviceFramesProcessed * ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels));
20802 |
20803 |                         /* Convert capture data from device format to client format. */
20804 |                         result = ma_data_converter_process_pcm_frames(&pDevice->capture.converter,
> pRunningCapturedDeviceFrames, &capturedDeviceFramesToProcessThisIteration, capturedClientData,
> &capturedClientFramesToProcessThisIteration);
20805 |                         if (result != MA_SUCCESS) {
20806 |                             break;
20807 |                         }
20808 |
20809 |                         /*
20810 |                         If we weren't able to generate any output frames it must mean we've
> exhausted all of our input. The only time this would not be the case is if capturedClientData was
> too small
20811 |                         which should never be the case when it's of the size
> MA_DATA_CONVERTER_STACK_BUFFER_SIZE.
20812 |                         */
20813 |                         if (capturedClientFramesToProcessThisIteration == 0) {
20814 |                             break;
20815 |                         }

```

```

20816 |
20817 |             ma_device__handle_data_callback(pDevice, playbackClientData,
> capturedClientData, (ma_uint32)capturedClientFramesToProcessThisIteration); /* Safe cast .*/
20818 |
20819 |             capturedDeviceFramesProcessed +=
> (ma_uint32)capturedDeviceFramesToProcessThisIteration; /* Safe cast. */
20820 |             capturedDeviceFramesRemaining -=
> (ma_uint32)capturedDeviceFramesToProcessThisIteration; /* Safe cast. */
20821 |
20822 |             /* At this point the playbackClientData buffer should be holding data that
> needs to be written to the device. */
20823 |             for (;;) {
20824 |                 ma_uint64 convertedClientFrameCount =
> capturedClientFramesToProcessThisIteration;
20825 |                 ma_uint64 convertedDeviceFrameCount = playbackDeviceDataCapInFrames;
20826 |                 result =
> ma_data_converter_process_pcm_frames(&pDevice->playback.converter, playbackClientData,
> &convertedClientFrameCount, playbackDeviceData, &convertedDeviceFrameCount);
20827 |                 if (result != MA_SUCCESS) {
20828 |                     break;
20829 |                 }
20830 |
20831 |                 result = pDevice->pContext->callbacks.onDeviceWrite(pDevice,
> playbackDeviceData, (ma_uint32)convertedDeviceFrameCount, NULL); /* Safe cast. */
20832 |                 if (result != MA_SUCCESS) {
20833 |                     exitLoop = MA_TRUE;
20834 |                     break;
20835 |                 }
20836 |
20837 |                 capturedClientFramesToProcessThisIteration -=
> (ma_uint32)convertedClientFrameCount; /* Safe cast. */
20838 |                 if (capturedClientFramesToProcessThisIteration == 0) {
20839 |                     break;
20840 |                 }
20841 |             }
20842 |
20843 |             /* In case an error happened from ma_device_write__null()... */
20844 |             if (result != MA_SUCCESS) {
20845 |                 exitLoop = MA_TRUE;
20846 |                 break;
20847 |             }
20848 |         }
20849 |
20850 |         /* Make sure we don't get stuck in the inner loop. */
20851 |         if (capturedDeviceFramesProcessed == 0) {
20852 |             break;
20853 |         }
20854 |
20855 |         totalCapturedDeviceFramesProcessed += capturedDeviceFramesProcessed;
20856 |     }
20857 | } break;
20858 |
20859 | case ma_device_type_capture:
20860 | case ma_device_type_loopback:
20861 | {
20862 |     ma_uint32 periodSizeInFrames = pDevice->capture.internalPeriodSizeInFrames;
20863 |     ma_uint32 framesReadThisPeriod = 0;
20864 |     while (framesReadThisPeriod < periodSizeInFrames) {
20865 |         ma_uint32 framesRemainingInPeriod = periodSizeInFrames - framesReadThisPeriod;
20866 |         ma_uint32 framesProcessed;
20867 |         ma_uint32 framesToReadThisIteration = framesRemainingInPeriod;
20868 |         if (framesToReadThisIteration > capturedDeviceDataCapInFrames) {
20869 |             framesToReadThisIteration = capturedDeviceDataCapInFrames;
20870 |         }
20871 |
20872 |         result = pDevice->pContext->callbacks.onDeviceRead(pDevice, capturedDeviceData,
> framesToReadThisIteration, &framesProcessed);
20873 |         if (result != MA_SUCCESS) {
20874 |             exitLoop = MA_TRUE;
20875 |             break;
20876 |         }
20877 |
20878 |         /* Make sure we don't get stuck in the inner loop. */
20879 |         if (framesProcessed == 0) {
20880 |             break;
20881 |         }
20882 |
20883 |         ma_device__send_frames_to_client(pDevice, framesProcessed, capturedDeviceData);
20884 |     }

```

```

20885 |             framesReadThisPeriod += framesProcessed;
20886 |         }
20887 |     } break;
20888 |
20889 |     case ma_device_type_playback:
20890 |     {
20891 |         /* We write in chunks of the period size, but use a stack allocated buffer for the
> intermediary. */
20892 |         ma_uint32 periodSizeInFrames = pDevice->playback.internalPeriodSizeInFrames;
20893 |         ma_uint32 framesWrittenThisPeriod = 0;
20894 |         while (framesWrittenThisPeriod < periodSizeInFrames) {
20895 |             ma_uint32 framesRemainingInPeriod = periodSizeInFrames -
> framesWrittenThisPeriod;
20896 |             ma_uint32 framesProcessed;
20897 |             ma_uint32 framesToWriteThisIteration = framesRemainingInPeriod;
20898 |             if (framesToWriteThisIteration > playbackDeviceDataCapInFrames) {
20899 |                 framesToWriteThisIteration = playbackDeviceDataCapInFrames;
20900 |             }
20901 |
20902 |             ma_device__read_frames_from_client(pDevice, framesToWriteThisIteration,
> playbackDeviceData);
20903 |
20904 |             result = pDevice->pContext->callbacks.onDeviceWrite(pDevice, playbackDeviceData,
> framesToWriteThisIteration, &framesProcessed);
20905 |             if (result != MA_SUCCESS) {
20906 |                 exitLoop = MA_TRUE;
20907 |                 break;
20908 |             }
20909 |
20910 |             /* Make sure we don't get stuck in the inner loop. */
20911 |             if (framesProcessed == 0) {
20912 |                 break;
20913 |             }
20914 |
20915 |             framesWrittenThisPeriod += framesProcessed;
20916 |         }
20917 |     } break;
20918 |
20919 |     /* Should never get here. */
20920 |     default: break;
20921 | }
20922 | }
20923 |
20924 | return result;
20925 | }
20926 |
20927 |
20928 |
20929 | /*****
20930 |
20931 | Null Backend
20932 |
20933 | *****/
20934 | #ifndef MA_HAS_NULL
20935 |
20936 | #define MA_DEVICE_OP_NONE__NULL    0
20937 | #define MA_DEVICE_OP_START__NULL   1
20938 | #define MA_DEVICE_OP_SUSPEND__NULL 2
20939 | #define MA_DEVICE_OP_KILL__NULL    3
20940 |
20941 | static ma_thread_result MA_THREADCALL ma_device_thread__null(void* pData)
20942 | {
20943 |     ma_device* pDevice = (ma_device*)pData;
20944 |     MA_ASSERT(pDevice != NULL);
20945 |
20946 |     for (;;) { /* Keep the thread alive until the device is uninitialized. */
20947 |         ma_uint32 operation;
20948 |
20949 |         /* Wait for an operation to be requested. */
20950 |         ma_event_wait(&pDevice->null_device.operationEvent);
20951 |
20952 |         /* At this point an event should have been triggered. */
20953 |         operation = pDevice->null_device.operation;
20954 |
20955 |         /* Starting the device needs to put the thread into a loop. */
20956 |         if (operation == MA_DEVICE_OP_START__NULL) {
20957 |             /* Reset the timer just in case. */
20958 |             ma_timer_init(&pDevice->null_device.timer);
20959 |

```



```

20960 |         /* Getting here means a suspend or kill operation has been requested. */
20961 |         pDevice->null_device.operationResult = MA_SUCCESS;
20962 |         ma_event_signal(&pDevice->null_device.operationCompletionEvent);
20963 |         ma_semaphore_release(&pDevice->null_device.operationSemaphore);
20964 |         continue;
20965 |     }
20966 |
20967 |     /* Suspending the device means we need to stop the timer and just continue the loop. */
20968 |     if (operation == MA_DEVICE_OP_SUSPEND_NULL) {
20969 |         /* We need to add the current run time to the prior run time, then reset the timer. */
20970 |         pDevice->null_device.priorRunTime +=
> ma_timer_get_time_in_seconds(&pDevice->null_device.timer);
20971 |         ma_timer_init(&pDevice->null_device.timer);
20972 |
20973 |         /* We're done. */
20974 |         pDevice->null_device.operationResult = MA_SUCCESS;
20975 |         ma_event_signal(&pDevice->null_device.operationCompletionEvent);
20976 |         ma_semaphore_release(&pDevice->null_device.operationSemaphore);
20977 |         continue;
20978 |     }
20979 |
20980 |     /* Killing the device means we need to get out of this loop so that this thread can
> terminate. */
20981 |     if (operation == MA_DEVICE_OP_KILL_NULL) {
20982 |         pDevice->null_device.operationResult = MA_SUCCESS;
20983 |         ma_event_signal(&pDevice->null_device.operationCompletionEvent);
20984 |         ma_semaphore_release(&pDevice->null_device.operationSemaphore);
20985 |         break;
20986 |     }
20987 |
20988 |     /* Getting a signal on a "none" operation probably means an error. Return invalid operation.
> */
20989 |     if (operation == MA_DEVICE_OP_NONE_NULL) {
20990 |         MA_ASSERT(MA_FALSE); /* <-- Trigger this in debug mode to ensure developers are aware
> they're doing something wrong (or there's a bug in a miniaudio). */
20991 |         pDevice->null_device.operationResult = MA_INVALID_OPERATION;
20992 |         ma_event_signal(&pDevice->null_device.operationCompletionEvent);
20993 |         ma_semaphore_release(&pDevice->null_device.operationSemaphore);
20994 |         continue; /* Continue the loop. Don't terminate. */
20995 |     }
20996 | }
20997 |
20998 |     return (ma_thread_result)0;
20999 | }
21000 |
21001 | static ma_result ma_device_do_operation__null(ma_device* pDevice, ma_uint32 operation)
21002 | {
21003 |     ma_result result;
21004 |
21005 |     /*
21006 |     > TODO: Need to review this and consider just using mutual exclusion. I think the original
motivation
21007 |     for this was to just post the event to a queue and return immediately, but that has since
> changed
21008 |     and now this function is synchronous. I think this can be simplified to just use a mutex.
21009 |     */
21010 |
21011 |     /*
21012 |     > The first thing to do is wait for an operation slot to become available. We only have a single
slot for this, but we could extend this later
21013 |     to support queuing of operations.
21014 |     */
21015 |     result = ma_semaphore_wait(&pDevice->null_device.operationSemaphore);
21016 |     if (result != MA_SUCCESS) {
21017 |         return result; /* Failed to wait for the event. */
21018 |     }
21019 |
21020 |     /*
21021 |     > When we get here it means the background thread is not referencing the operation code and it can
be changed. After changing this we need to
21022 |     signal an event to the worker thread to let it know that it can start work.
21023 |     */
21024 |     pDevice->null_device.operation = operation;
21025 |
21026 |     /* Once the operation code has been set, the worker thread can start work. */
21027 |     if (ma_event_signal(&pDevice->null_device.operationEvent) != MA_SUCCESS) {
21028 |         return MA_ERROR;
21029 |     }
21030 | }

```

```

21031 |     /* We want everything to be synchronous so we're going to wait for the worker thread to complete
> it's operation. */
21032 |     if (ma_event_wait(&pDevice->null_device.operationCompletionEvent) != MA_SUCCESS) {
21033 |         return MA_ERROR;
21034 |     }
21035 |
21036 |     return pDevice->null_device.operationResult;
21037 | }
21038 |
21039 | static ma_uint64 ma_device_get_total_run_time_in_frames__null(ma_device* pDevice)
21040 | {
21041 |     ma_uint32 internalSampleRate;
21042 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
21043 |         internalSampleRate = pDevice->capture.internalSampleRate;
21044 |     } else {
21045 |         internalSampleRate = pDevice->playback.internalSampleRate;
21046 |     }
21047 |
21048 |     return (ma_uint64)((pDevice->null_device.priorRunTime +
> ma_timer_get_time_in_seconds(&pDevice->null_device.timer)) * internalSampleRate);
21049 | }
21050 |
21051 | static ma_result ma_context_enumerate_devices__null(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
21052 | {
21053 |     ma_bool32 cbResult = MA_TRUE;
21054 |
21055 |     MA_ASSERT(pContext != NULL);
21056 |     MA_ASSERT(callback != NULL);
21057 |
21058 |     /* Playback. */
21059 |     if (cbResult) {
21060 |         ma_device_info deviceInfo;
21061 |         MA_ZERO_OBJECT(&deviceInfo);
21062 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), "NULL Playback Device", (size_t)-1);
21063 |         deviceInfo.isDefault = MA_TRUE; /* Only one playback and capture device for the null
> backend, so might as well mark as default. */
21064 |         cbResult = callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
21065 |     }
21066 |
21067 |     /* Capture. */
21068 |     if (cbResult) {
21069 |         ma_device_info deviceInfo;
21070 |         MA_ZERO_OBJECT(&deviceInfo);
21071 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), "NULL Capture Device", (size_t)-1);
21072 |         deviceInfo.isDefault = MA_TRUE; /* Only one playback and capture device for the null
> backend, so might as well mark as default. */
21073 |         cbResult = callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
21074 |     }
21075 |
21076 |     (void)cbResult; /* Silence a static analysis warning. */
21077 |
21078 |     return MA_SUCCESS;
21079 | }
21080 |
21081 | static ma_result ma_context_get_device_info__null(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
21082 | {
21083 |     MA_ASSERT(pContext != NULL);
21084 |
21085 |     if (pDeviceID != NULL && pDeviceID->nullbackend != 0) {
21086 |         return MA_NO_DEVICE; /* Don't know the device. */
21087 |     }
21088 |
21089 |     /* Name / Description */
21090 |     if (deviceType == ma_device_type_playback) {
21091 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), "NULL Playback Device",
> (size_t)-1);
21092 |     } else {
21093 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), "NULL Capture Device",
> (size_t)-1);
21094 |     }
21095 |
21096 |     pDeviceInfo->isDefault = MA_TRUE; /* Only one playback and capture device for the null
> backend, so might as well mark as default. */
21097 |
21098 |     /* Support everything on the null backend. */
21099 |     pDeviceInfo->nativeDataFormats[0].format = ma_format_unknown;
21100 |     pDeviceInfo->nativeDataFormats[0].channels = 0;

```

```

21101 |     pDeviceInfo->nativeDataFormats[0].sampleRate = 0;
21102 |     pDeviceInfo->nativeDataFormats[0].flags      = 0;
21103 |     pDeviceInfo->nativeDataFormatCount = 1;
21104 |
21105 |     (void)pContext;
21106 |     return MA_SUCCESS;
21107 | }
21108 |
21109 |
21110 | static ma_result ma_device_uninit__null(ma_device* pDevice)
21111 | {
21112 |     MA_ASSERT(pDevice != NULL);
21113 |
21114 |     /* Keep it clean and wait for the device thread to finish before returning. */
21115 |     ma_device_do_operation__null(pDevice, MA_DEVICE_OP_KILL__NULL);
21116 |
21117 |     /* Wait for the thread to finish before continuing. */
21118 |     ma_thread_wait(&pDevice->null_device.deviceThread);
21119 |
21120 |     /* At this point the loop in the device thread is as good as terminated so we can uninitialized
> our events. */
21121 |     ma_semaphore_uninit(&pDevice->null_device.operationSemaphore);
21122 |     ma_event_uninit(&pDevice->null_device.operationCompletionEvent);
21123 |     ma_event_uninit(&pDevice->null_device.operationEvent);
21124 |
21125 |     return MA_SUCCESS;
21126 | }
21127 |
21128 | static ma_result ma_device_init__null(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
21129 | {
21130 |     ma_result result;
21131 |
21132 |     MA_ASSERT(pDevice != NULL);
21133 |
21134 |     MA_ZERO_OBJECT(&pDevice->null_device);
21135 |
21136 |     if (pConfig->deviceType == ma_device_type_loopback) {
21137 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
21138 |     }
21139 |
21140 |     /* The null backend supports everything exactly as we specify it. */
21141 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
21142 |         pDescriptorCapture->format      = (pDescriptorCapture->format      != ma_format_unknown) ?
> pDescriptorCapture->format      : MA_DEFAULT_FORMAT;
21143 |         pDescriptorCapture->channels    = (pDescriptorCapture->channels    != 0) ?
> pDescriptorCapture->channels    : MA_DEFAULT_CHANNELS;
21144 |         pDescriptorCapture->sampleRate = (pDescriptorCapture->sampleRate != 0) ?
> pDescriptorCapture->sampleRate : MA_DEFAULT_SAMPLE_RATE;
21145 |
21146 |         if (pDescriptorCapture->channelMap[0] == MA_CHANNEL_NONE) {
21147 |             ma_channel_map_init_standard(ma_standard_channel_map_default,
> pDescriptorCapture->channelMap, ma_countof(pDescriptorCapture->channelMap),
> pDescriptorCapture->channels);
21148 |         }
21149 |
21150 |         pDescriptorCapture->periodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptorCapture,
> pDescriptorCapture->sampleRate, pConfig->performanceProfile);
21151 |     }
21152 |
21153 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
21154 |         pDescriptorPlayback->format      = (pDescriptorPlayback->format      != ma_format_unknown) ?
> pDescriptorPlayback->format      : MA_DEFAULT_FORMAT;
21155 |         pDescriptorPlayback->channels    = (pDescriptorPlayback->channels    != 0) ?
> pDescriptorPlayback->channels    : MA_DEFAULT_CHANNELS;
21156 |         pDescriptorPlayback->sampleRate = (pDescriptorPlayback->sampleRate != 0) ?
> pDescriptorPlayback->sampleRate : MA_DEFAULT_SAMPLE_RATE;
21157 |
21158 |         if (pDescriptorPlayback->channelMap[0] == MA_CHANNEL_NONE) {
21159 |             ma_channel_map_init_standard(ma_standard_channel_map_default,
> pDescriptorPlayback->channelMap, ma_countof(pDescriptorCapture->channelMap),
> pDescriptorPlayback->channels);
21160 |         }
21161 |
21162 |         pDescriptorPlayback->periodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptorPlayback,

```

```

> pDescriptorPlayback->sampleRate, pConfig->performanceProfile);
21163 |     }
21164 |
21165 |     /*
21166 |     In order to get timing right, we need to create a thread that does nothing but keeps track of
> the timer. This timer is started when the
21167 |     first period is "written" to it, and then stopped in ma_device_stop__null().
21168 |     */
21169 |     result = ma_event_init(&pDevice->null_device.operationEvent);
21170 |     if (result != MA_SUCCESS) {
21171 |         return result;
21172 |     }
21173 |
21174 |     result = ma_event_init(&pDevice->null_device.operationCompletionEvent);
21175 |     if (result != MA_SUCCESS) {
21176 |         return result;
21177 |     }
21178 |
21179 |     result = ma_semaphore_init(1, &pDevice->null_device.operationSemaphore);    /* <-- It's
> important that the initial value is set to 1. */
21180 |     if (result != MA_SUCCESS) {
21181 |         return result;
21182 |     }
21183 |
21184 |     result = ma_thread_create(&pDevice->null_device.deviceThread, pDevice->pContext->threadPriority,
> 0, ma_device_thread__null, pDevice, &pDevice->pContext->allocationCallbacks);
21185 |     if (result != MA_SUCCESS) {
21186 |         return result;
21187 |     }
21188 |
21189 |     return MA_SUCCESS;
21190 | }
21191 |
21192 | static ma_result ma_device_start__null(ma_device* pDevice)
21193 | {
21194 |     MA_ASSERT(pDevice != NULL);
21195 |
21196 |     ma_device_do_operation__null(pDevice, MA_DEVICE_OP_START__NULL);
21197 |
21198 |     ma_atomic_bool32_set(&pDevice->null_device.isStarted, MA_TRUE);
21199 |     return MA_SUCCESS;
21200 | }
21201 |
21202 | static ma_result ma_device_stop__null(ma_device* pDevice)
21203 | {
21204 |     MA_ASSERT(pDevice != NULL);
21205 |
21206 |     ma_device_do_operation__null(pDevice, MA_DEVICE_OP_SUSPEND__NULL);
21207 |
21208 |     ma_atomic_bool32_set(&pDevice->null_device.isStarted, MA_FALSE);
21209 |     return MA_SUCCESS;
21210 | }
21211 |
21212 | static ma_bool32 ma_device_is_started__null(ma_device* pDevice)
21213 | {
21214 |     MA_ASSERT(pDevice != NULL);
21215 |
21216 |     return ma_atomic_bool32_get(&pDevice->null_device.isStarted);
21217 | }
21218 |
21219 | static ma_result ma_device_write__null(ma_device* pDevice, const void* pPCMFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
21220 | {
21221 |     ma_result result = MA_SUCCESS;
21222 |     ma_uint32 totalPCMFramesProcessed;
21223 |     ma_bool32 wasStartedOnEntry;
21224 |
21225 |     if (pFramesWritten != NULL) {
21226 |         *pFramesWritten = 0;
21227 |     }
21228 |
21229 |     wasStartedOnEntry = ma_device_is_started__null(pDevice);
21230 |
21231 |     /* Keep going until everything has been read. */
21232 |     totalPCMFramesProcessed = 0;
21233 |     while (totalPCMFramesProcessed < frameCount) {
21234 |         ma_uint64 targetFrame;
21235 |
21236 |         /* If there are any frames remaining in the current period, consume those first. */

```

```

21237 |         if (pDevice->null_device.currentPeriodFramesRemainingPlayback > 0) {
21238 |             ma_uint32 framesRemaining = (frameCount - totalPCMFramesProcessed);
21239 |             ma_uint32 framesToProcess = pDevice->null_device.currentPeriodFramesRemainingPlayback;
21240 |             if (framesToProcess > framesRemaining) {
21241 |                 framesToProcess = framesRemaining;
21242 |             }
21243 |
21244 |             /* We don't actually do anything with pPCMFrames, so just mark it as unused to prevent a
> warning. */
21245 |             (void)pPCMFrames;
21246 |
21247 |             pDevice->null_device.currentPeriodFramesRemainingPlayback -= framesToProcess;
21248 |             totalPCMFramesProcessed += framesToProcess;
21249 |         }
21250 |
21251 |         /* If we've consumed the current period we'll need to mark it as such an ensure the device
> is started if it's not already. */
21252 |         if (pDevice->null_device.currentPeriodFramesRemainingPlayback == 0) {
21253 |             pDevice->null_device.currentPeriodFramesRemainingPlayback = 0;
21254 |
21255 |             if (!ma_device_is_started__null(pDevice) && !wasStartedOnEntry) {
21256 |                 result = ma_device_start__null(pDevice);
21257 |                 if (result != MA_SUCCESS) {
21258 |                     break;
21259 |                 }
21260 |             }
21261 |         }
21262 |
21263 |         /* If we've consumed the whole buffer we can return now. */
21264 |         MA_ASSERT(totalPCMFramesProcessed <= frameCount);
21265 |         if (totalPCMFramesProcessed == frameCount) {
21266 |             break;
21267 |         }
21268 |
21269 |         /* Getting here means we've still got more frames to consume, we but need to wait for it to
> become available. */
21270 |         targetFrame = pDevice->null_device.lastProcessedFramePlayback;
21271 |         for (;;) {
21272 |             ma_uint64 currentFrame;
21273 |
21274 |             /* Stop waiting if the device has been stopped. */
21275 |             if (!ma_device_is_started__null(pDevice)) {
21276 |                 break;
21277 |             }
21278 |
21279 |             currentFrame = ma_device_get_total_run_time_in_frames__null(pDevice);
21280 |             if (currentFrame >= targetFrame) {
21281 |                 break;
21282 |             }
21283 |
21284 |             /* Getting here means we haven't yet reached the target sample, so continue waiting. */
21285 |             ma_sleep(10);
21286 |         }
21287 |
21288 |         pDevice->null_device.lastProcessedFramePlayback +=
> pDevice->playback.internalPeriodSizeInFrames;
21289 |         pDevice->null_device.currentPeriodFramesRemainingPlayback =
> pDevice->playback.internalPeriodSizeInFrames;
21290 |     }
21291 |
21292 |     if (pFramesWritten != NULL) {
21293 |         *pFramesWritten = totalPCMFramesProcessed;
21294 |     }
21295 |
21296 |     return result;
21297 | }
21298 |
21299 | static ma_result ma_device_read__null(ma_device* pDevice, void* pPCMFrames, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
21300 | {
21301 |     ma_result result = MA_SUCCESS;
21302 |     ma_uint32 totalPCMFramesProcessed;
21303 |
21304 |     if (pFramesRead != NULL) {
21305 |         *pFramesRead = 0;
21306 |     }
21307 |
21308 |     /* Keep going until everything has been read. */
21309 |     totalPCMFramesProcessed = 0;

```

```

21310 |     while (totalPCMFramesProcessed < frameCount) {
21311 |         ma_uint64 targetFrame;
21312 |
21313 |         /* If there are any frames remaining in the current period, consume those first. */
21314 |         if (pDevice->null_device.currentPeriodFramesRemainingCapture > 0) {
21315 |             ma_uint32 bpf = ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels);
21316 |             ma_uint32 framesRemaining = (frameCount - totalPCMFramesProcessed);
21317 |             ma_uint32 framesToProcess = pDevice->null_device.currentPeriodFramesRemainingCapture;
21318 |             if (framesToProcess > framesRemaining) {
21319 |                 framesToProcess = framesRemaining;
21320 |             }
21321 |
21322 |             /* We need to ensure the output buffer is zeroed. */
21323 |             MA_ZERO_MEMORY(ma_offset_ptr(pPCMFrames, totalPCMFramesProcessed*bpf),
> framesToProcess*bpf);
21324 |
21325 |             pDevice->null_device.currentPeriodFramesRemainingCapture -= framesToProcess;
21326 |             totalPCMFramesProcessed += framesToProcess;
21327 |         }
21328 |
21329 |         /* If we've consumed the current period we'll need to mark it as such an ensure the device
> is started if it's not already. */
21330 |         if (pDevice->null_device.currentPeriodFramesRemainingCapture == 0) {
21331 |             pDevice->null_device.currentPeriodFramesRemainingCapture = 0;
21332 |         }
21333 |
21334 |         /* If we've consumed the whole buffer we can return now. */
21335 |         MA_ASSERT(totalPCMFramesProcessed <= frameCount);
21336 |         if (totalPCMFramesProcessed == frameCount) {
21337 |             break;
21338 |         }
21339 |
21340 |         /* Getting here means we've still got more frames to consume, we but need to wait for it to
> become available. */
21341 |         targetFrame = pDevice->null_device.lastProcessedFrameCapture +
> pDevice->capture.internalPeriodSizeInFrames;
21342 |         for (;;) {
21343 |             ma_uint64 currentFrame;
21344 |
21345 |             /* Stop waiting if the device has been stopped. */
21346 |             if (!ma_device_is_started__null(pDevice)) {
21347 |                 break;
21348 |             }
21349 |
21350 |             currentFrame = ma_device_get_total_run_time_in_frames__null(pDevice);
21351 |             if (currentFrame >= targetFrame) {
21352 |                 break;
21353 |             }
21354 |
21355 |             /* Getting here means we haven't yet reached the target sample, so continue waiting. */
21356 |             ma_sleep(10);
21357 |         }
21358 |
21359 |         pDevice->null_device.lastProcessedFrameCapture +=
> pDevice->capture.internalPeriodSizeInFrames;
21360 |         pDevice->null_device.currentPeriodFramesRemainingCapture =
> pDevice->capture.internalPeriodSizeInFrames;
21361 |     }
21362 |
21363 |     if (pFramesRead != NULL) {
21364 |         *pFramesRead = totalPCMFramesProcessed;
21365 |     }
21366 |
21367 |     return result;
21368 | }
21369 |
21370 | static ma_result ma_context_uninit__null(ma_context* pContext)
21371 | {
21372 |     MA_ASSERT(pContext != NULL);
21373 |     MA_ASSERT(pContext->backend == ma_backend_null);
21374 |
21375 |     (void)pContext;
21376 |     return MA_SUCCESS;
21377 | }
21378 |
21379 | static ma_result ma_context_init__null(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
21380 | {

```

```

21381 |     MA_ASSERT(pContext != NULL);
21382 |
21383 |     (void)pConfig;
21384 |     (void)pContext;
21385 |
21386 |     pCallbacks->onContextInit          = ma_context_init__null;
21387 |     pCallbacks->onContextUninit        = ma_context_uninit__null;
21388 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__null;
21389 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__null;
21390 |     pCallbacks->onDeviceInit          = ma_device_init__null;
21391 |     pCallbacks->onDeviceUninit        = ma_device_uninit__null;
21392 |     pCallbacks->onDeviceStart         = ma_device_start__null;
21393 |     pCallbacks->onDeviceStop          = ma_device_stop__null;
21394 |     pCallbacks->onDeviceRead          = ma_device_read__null;
21395 |     pCallbacks->onDeviceWrite         = ma_device_write__null;
21396 |     pCallbacks->onDeviceDataLoop      = NULL; /* Our backend is asynchronous with a blocking
> read-write API which means we can get miniaudio to deal with the audio thread. */
21397 |
21398 |     /* The null backend always works. */
21399 |     return MA_SUCCESS;
21400 | }
21401 | #endif
21402 |
21403 |
21404 |
21405 | /*****
21406 |
21407 | WIN32 COMMON
21408 |
21409 | *****/
21410 | #if defined(MA_WIN32) && !defined(MA_XBOX)
21411 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
21412 |     #define ma_CoInitializeEx(pContext, pvReserved, dwCoInit)
> ((pContext->win32.CoInitializeEx) ?
> ((MA_PFN_CoInitializeEx)pContext->win32.CoInitializeEx)(pvReserved, dwCoInit) :
> ((MA_PFN_CoInitialize)pContext->win32.CoInitialize)(pvReserved))
21413 |     #define ma_CoUninitialize(pContext)
> ((MA_PFN_CoUninitialize)pContext->win32.CoUninitialize)()
21414 |     #define ma_CoCreateInstance(pContext, rclsid, pUnkOuter, dwClsContext, riid, ppv)
> ((MA_PFN_CoCreateInstance)pContext->win32.CoCreateInstance)(rclsid, pUnkOuter, dwClsContext, riid,
> ppv)
21415 |     #define ma_CoTaskMemFree(pContext, pv)
> ((MA_PFN_CoTaskMemFree)pContext->win32.CoTaskMemFree)(pv)
21416 |     #define ma_PropVariantClear(pContext, pvar)
> ((MA_PFN_PropVariantClear)pContext->win32.PropVariantClear)(pvar)
21417 | #else
21418 |     #define ma_CoInitializeEx(pContext, pvReserved, dwCoInit)
> CoInitializeEx(pvReserved, dwCoInit)
21419 |     #define ma_CoUninitialize(pContext)
> CoUninitialize()
21420 |     #define ma_CoCreateInstance(pContext, rclsid, pUnkOuter, dwClsContext, riid, ppv)
> CoCreateInstance(rclsid, pUnkOuter, dwClsContext, riid, ppv)
21421 |     #define ma_CoTaskMemFree(pContext, pv)
> CoTaskMemFree(pv)
21422 |     #define ma_PropVariantClear(pContext, pvar)
> PropVariantClear(pvar)
21423 | #endif
21424 |
21425 | #if !defined(MAXULONG_PTR) && !defined(__WATCOMC__) && !defined(MA_XBOX_NXDK)
21426 | typedef size_t DWORD_PTR;
21427 | #endif
21428 |
21429 | #if !defined(WAVE_FORMAT_1M08)
21430 | #define WAVE_FORMAT_1M08      0x00000001
21431 | #define WAVE_FORMAT_1S08      0x00000002
21432 | #define WAVE_FORMAT_1M16      0x00000004
21433 | #define WAVE_FORMAT_1S16      0x00000008
21434 | #define WAVE_FORMAT_2M08      0x00000010
21435 | #define WAVE_FORMAT_2S08      0x00000020
21436 | #define WAVE_FORMAT_2M16      0x00000040
21437 | #define WAVE_FORMAT_2S16      0x00000080
21438 | #define WAVE_FORMAT_4M08      0x00000100
21439 | #define WAVE_FORMAT_4S08      0x00000200
21440 | #define WAVE_FORMAT_4M16      0x00000400
21441 | #define WAVE_FORMAT_4S16      0x00000800
21442 | #endif
21443 |
21444 | #if !defined(WAVE_FORMAT_44M08)
21445 | #define WAVE_FORMAT_44M08     0x00000100

```



```

21446 | #define WAVE_FORMAT_44S08    0x00000200
21447 | #define WAVE_FORMAT_44M16    0x00000400
21448 | #define WAVE_FORMAT_44S16    0x00000800
21449 | #define WAVE_FORMAT_48M08    0x00001000
21450 | #define WAVE_FORMAT_48S08    0x00002000
21451 | #define WAVE_FORMAT_48M16    0x00004000
21452 | #define WAVE_FORMAT_48S16    0x00008000
21453 | #define WAVE_FORMAT_96M08    0x00010000
21454 | #define WAVE_FORMAT_96S08    0x00020000
21455 | #define WAVE_FORMAT_96M16    0x00040000
21456 | #define WAVE_FORMAT_96S16    0x00080000
21457 | #endif
21458 |
21459 | #ifndef SPEAKER_FRONT_LEFT
21460 | #define SPEAKER_FRONT_LEFT    0x1
21461 | #define SPEAKER_FRONT_RIGHT   0x2
21462 | #define SPEAKER_FRONT_CENTER  0x4
21463 | #define SPEAKER_LOW_FREQUENCY 0x8
21464 | #define SPEAKER_BACK_LEFT     0x10
21465 | #define SPEAKER_BACK_RIGHT    0x20
21466 | #define SPEAKER_FRONT_LEFT_OF_CENTER 0x40
21467 | #define SPEAKER_FRONT_RIGHT_OF_CENTER 0x80
21468 | #define SPEAKER_BACK_CENTER    0x100
21469 | #define SPEAKER_SIDE_LEFT      0x200
21470 | #define SPEAKER_SIDE_RIGHT     0x400
21471 | #define SPEAKER_TOP_CENTER     0x800
21472 | #define SPEAKER_TOP_FRONT_LEFT 0x1000
21473 | #define SPEAKER_TOP_FRONT_CENTER 0x2000
21474 | #define SPEAKER_TOP_FRONT_RIGHT 0x4000
21475 | #define SPEAKER_TOP_BACK_LEFT  0x8000
21476 | #define SPEAKER_TOP_BACK_CENTER 0x10000
21477 | #define SPEAKER_TOP_BACK_RIGHT 0x20000
21478 | #endif
21479 |
21480 | /*
21481 | Implement our own version of MA_WAVEFORMATEXTENSIBLE so we can avoid a header. Be careful with this
21482 | because MA_WAVEFORMATEX has an extra two bytes over standard WAVEFORMATEX due to padding. The
21483 | standard version uses tight packing, but for compiler compatibility we're not doing that with ours.
21484 | */
21485 | typedef struct
21486 | {
21487 |     WORD wFormatTag;
21488 |     WORD nChannels;
21489 |     DWORD nSamplesPerSec;
21490 |     DWORD nAvgBytesPerSec;
21491 |     WORD nBlockAlign;
21492 |     WORD wBitsPerSample;
21493 |     WORD cbSize;
21494 | } MA_WAVEFORMATEX;
21495 |
21496 | typedef struct
21497 | {
21498 |     WORD wFormatTag;
21499 |     WORD nChannels;
21500 |     DWORD nSamplesPerSec;
21501 |     DWORD nAvgBytesPerSec;
21502 |     WORD nBlockAlign;
21503 |     WORD wBitsPerSample;
21504 |     WORD cbSize;
21505 |     union
21506 |     {
21507 |         WORD wValidBitsPerSample;
21508 |         WORD wSamplesPerBlock;
21509 |         WORD wReserved;
21510 |     } Samples;
21511 |     DWORD dwChannelMask;
21512 |     GUID SubFormat;
21513 | } MA_WAVEFORMATEXTENSIBLE;
21514 |
21515 |
21516 |
21517 | #ifndef WAVE_FORMAT_EXTENSIBLE
21518 | #define WAVE_FORMAT_EXTENSIBLE 0xFFFFE
21519 | #endif
21520 |
21521 | #ifndef WAVE_FORMAT_PCM
21522 | #define WAVE_FORMAT_PCM        1
21523 | #endif
21524 |

```

```

21525 | #ifndef WAVE_FORMAT_IEEE_FLOAT
21526 | #define WAVE_FORMAT_IEEE_FLOAT 0x0003
21527 | #endif
21528 |
21529 | /* Converts an individual Win32-style channel identifier (SPEAKER_FRONT_LEFT, etc.) to miniaudio. */
21530 | static ma_uint8 ma_channel_id_to_ma_win32(DWORD id)
21531 | {
21532 |     switch (id)
21533 |     {
21534 |         case SPEAKER_FRONT_LEFT:      return MA_CHANNEL_FRONT_LEFT;
21535 |         case SPEAKER_FRONT_RIGHT:     return MA_CHANNEL_FRONT_RIGHT;
21536 |         case SPEAKER_FRONT_CENTER:    return MA_CHANNEL_FRONT_CENTER;
21537 |         case SPEAKER_LOW_FREQUENCY:   return MA_CHANNEL_LFE;
21538 |         case SPEAKER_BACK_LEFT:       return MA_CHANNEL_BACK_LEFT;
21539 |         case SPEAKER_BACK_RIGHT:      return MA_CHANNEL_BACK_RIGHT;
21540 |         case SPEAKER_FRONT_LEFT_OF_CENTER: return MA_CHANNEL_FRONT_LEFT_CENTER;
21541 |         case SPEAKER_FRONT_RIGHT_OF_CENTER: return MA_CHANNEL_FRONT_RIGHT_CENTER;
21542 |         case SPEAKER_BACK_CENTER:     return MA_CHANNEL_BACK_CENTER;
21543 |         case SPEAKER_SIDE_LEFT:       return MA_CHANNEL_SIDE_LEFT;
21544 |         case SPEAKER_SIDE_RIGHT:      return MA_CHANNEL_SIDE_RIGHT;
21545 |         case SPEAKER_TOP_CENTER:      return MA_CHANNEL_TOP_CENTER;
21546 |         case SPEAKER_TOP_FRONT_LEFT:  return MA_CHANNEL_TOP_FRONT_LEFT;
21547 |         case SPEAKER_TOP_FRONT_CENTER: return MA_CHANNEL_TOP_FRONT_CENTER;
21548 |         case SPEAKER_TOP_FRONT_RIGHT: return MA_CHANNEL_TOP_FRONT_RIGHT;
21549 |         case SPEAKER_TOP_BACK_LEFT:   return MA_CHANNEL_TOP_BACK_LEFT;
21550 |         case SPEAKER_TOP_BACK_CENTER: return MA_CHANNEL_TOP_BACK_CENTER;
21551 |         case SPEAKER_TOP_BACK_RIGHT:  return MA_CHANNEL_TOP_BACK_RIGHT;
21552 |         default: return 0;
21553 |     }
21554 | }
21555 |
21556 | /* Converts an individual miniaudio channel identifier (MA_CHANNEL_FRONT_LEFT, etc.) to Win32-style.
> */
21557 | static DWORD ma_channel_id_to_win32(DWORD id)
21558 | {
21559 |     switch (id)
21560 |     {
21561 |         case MA_CHANNEL_MONO:      return SPEAKER_FRONT_CENTER;
21562 |         case MA_CHANNEL_FRONT_LEFT: return SPEAKER_FRONT_LEFT;
21563 |         case MA_CHANNEL_FRONT_RIGHT: return SPEAKER_FRONT_RIGHT;
21564 |         case MA_CHANNEL_FRONT_CENTER: return SPEAKER_FRONT_CENTER;
21565 |         case MA_CHANNEL_LFE:        return SPEAKER_LOW_FREQUENCY;
21566 |         case MA_CHANNEL_BACK_LEFT:   return SPEAKER_BACK_LEFT;
21567 |         case MA_CHANNEL_BACK_RIGHT:  return SPEAKER_BACK_RIGHT;
21568 |         case MA_CHANNEL_FRONT_LEFT_CENTER: return SPEAKER_FRONT_LEFT_OF_CENTER;
21569 |         case MA_CHANNEL_FRONT_RIGHT_CENTER: return SPEAKER_FRONT_RIGHT_OF_CENTER;
21570 |         case MA_CHANNEL_BACK_CENTER:   return SPEAKER_BACK_CENTER;
21571 |         case MA_CHANNEL_SIDE_LEFT:     return SPEAKER_SIDE_LEFT;
21572 |         case MA_CHANNEL_SIDE_RIGHT:    return SPEAKER_SIDE_RIGHT;
21573 |         case MA_CHANNEL_TOP_CENTER:    return SPEAKER_TOP_CENTER;
21574 |         case MA_CHANNEL_TOP_FRONT_LEFT: return SPEAKER_TOP_FRONT_LEFT;
21575 |         case MA_CHANNEL_TOP_FRONT_CENTER: return SPEAKER_TOP_FRONT_CENTER;
21576 |         case MA_CHANNEL_TOP_FRONT_RIGHT: return SPEAKER_TOP_FRONT_RIGHT;
21577 |         case MA_CHANNEL_TOP_BACK_LEFT:  return SPEAKER_TOP_BACK_LEFT;
21578 |         case MA_CHANNEL_TOP_BACK_CENTER: return SPEAKER_TOP_BACK_CENTER;
21579 |         case MA_CHANNEL_TOP_BACK_RIGHT: return SPEAKER_TOP_BACK_RIGHT;
21580 |         default: return 0;
21581 |     }
21582 | }
21583 |
21584 | /* Converts a channel mapping to a Win32-style channel mask. */
21585 | static DWORD ma_channel_map_to_channel_mask_win32(const ma_channel* pChannelMap, ma_uint32
> channels)
21586 | {
21587 |     DWORD dwChannelMask = 0;
21588 |     ma_uint32 iChannel;
21589 |
21590 |     for (iChannel = 0; iChannel < channels; ++iChannel) {
21591 |         dwChannelMask |= ma_channel_id_to_win32(pChannelMap[iChannel]);
21592 |     }
21593 |
21594 |     return dwChannelMask;
21595 | }
21596 |
21597 | /* Converts a Win32-style channel mask to a miniaudio channel map. */
21598 | static void ma_channel_mask_to_channel_map_win32(DWORD dwChannelMask, ma_uint32 channels,
> ma_channel* pChannelMap)
21599 | {
21600 |     /* If the channel mask is set to 0, just assume a default Win32 channel map. */

```

```

21601 |     if (dwChannelMask == 0) {
21602 |         ma_channel_map_init_standard(ma_standard_channel_map_microsoft, pChannelMap, channels,
> channels);
21603 |     } else {
21604 |         if (channels == 1 && (dwChannelMask & SPEAKER_FRONT_CENTER) != 0) {
21605 |             pChannelMap[0] = MA_CHANNEL_MONO;
21606 |         } else {
21607 |             /* Just iterate over each bit. */
21608 |             ma_uint32 iChannel = 0;
21609 |             ma_uint32 iBit;
21610 |
21611 |             for (iBit = 0; iBit < 32 && iChannel < channels; ++iBit) {
21612 |                 DWORD bitValue = (dwChannelMask & (1UL << iBit));
21613 |                 if (bitValue != 0) {
21614 |                     /* The bit is set. */
21615 |                     pChannelMap[iChannel] = ma_channel_id_to_ma_win32(bitValue);
21616 |                     iChannel += 1;
21617 |                 }
21618 |             }
21619 |         }
21620 |     }
21621 | }
21622 |
21623 | #ifdef __cplusplus
21624 | static ma_bool32 ma_is_guid_equal(const void* a, const void* b)
21625 | {
21626 |     return IsEqualGUID(*(const GUID*)a, *(const GUID*)b);
21627 | }
21628 | #else
21629 | #define ma_is_guid_equal(a, b) IsEqualGUID((const GUID*)a, (const GUID*)b)
21630 | #endif
21631 |
21632 | static MA_INLINE ma_bool32 ma_is_guid_null(const void* guid)
21633 | {
21634 |     static GUID nullguid = {0x00000000, 0x0000, 0x0000, {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
> 0x00}};
21635 |     return ma_is_guid_equal(guid, &nullguid);
21636 | }
21637 |
21638 | static ma_format ma_format_from_WAVEFORMATEX(const MA_WAVEFORMATEX* pWF)
21639 | {
21640 |     MA_ASSERT(pWF != NULL);
21641 |
21642 |     if (pWF->wFormatTag == WAVE_FORMAT_EXTENSIBLE) {
21643 |         const MA_WAVEFORMATEXTENSIBLE* pWFEX = (const MA_WAVEFORMATEXTENSIBLE*)pWF;
21644 |         if (ma_is_guid_equal(&pWFEX->SubFormat, &MA_GUID_KSDATAFORMAT_SUBTYPE_PCM)) {
21645 |             if (pWFEX->Samples.wValidBitsPerSample == 32) {
21646 |                 return ma_format_s32;
21647 |             }
21648 |             if (pWFEX->Samples.wValidBitsPerSample == 24) {
21649 |                 if (pWFEX->wBitsPerSample == 32) {
21650 |                     return ma_format_s32;
21651 |                 }
21652 |                 if (pWFEX->wBitsPerSample == 24) {
21653 |                     return ma_format_s24;
21654 |                 }
21655 |             }
21656 |             if (pWFEX->Samples.wValidBitsPerSample == 16) {
21657 |                 return ma_format_s16;
21658 |             }
21659 |             if (pWFEX->Samples.wValidBitsPerSample == 8) {
21660 |                 return ma_format_u8;
21661 |             }
21662 |         }
21663 |         if (ma_is_guid_equal(&pWFEX->SubFormat, &MA_GUID_KSDATAFORMAT_SUBTYPE_IEEE_FLOAT)) {
21664 |             if (pWFEX->Samples.wValidBitsPerSample == 32) {
21665 |                 return ma_format_f32;
21666 |             }
21667 |             /*
21668 |             if (pWFEX->Samples.wValidBitsPerSample == 64) {
21669 |                 return ma_format_f64;
21670 |             }
21671 |             */
21672 |         }
21673 |     } else {
21674 |         if (pWF->wFormatTag == WAVE_FORMAT_PCM) {
21675 |             if (pWF->wBitsPerSample == 32) {
21676 |                 return ma_format_s32;
21677 |             }

```

```

21678 |         if (pWF->wBitsPerSample == 24) {
21679 |             return ma_format_s24;
21680 |         }
21681 |         if (pWF->wBitsPerSample == 16) {
21682 |             return ma_format_s16;
21683 |         }
21684 |         if (pWF->wBitsPerSample == 8) {
21685 |             return ma_format_u8;
21686 |         }
21687 |     }
21688 |     if (pWF->wFormatTag == WAVE_FORMAT_IEEE_FLOAT) {
21689 |         if (pWF->wBitsPerSample == 32) {
21690 |             return ma_format_f32;
21691 |         }
21692 |         if (pWF->wBitsPerSample == 64) {
21693 |             /*return ma_format_f64;*/
21694 |         }
21695 |     }
21696 | }
21697 |
21698 |     return ma_format_unknown;
21699 | }
21700 | #endif
21701 |
21702 |
21703 | /*****
21704 |
21705 | WASAPI Backend
21706 |
21707 | *****/
21708 | #ifdef MA_HAS_WASAPI
21709 | #if 0
21710 | #if defined(_MSC_VER)
21711 |     #pragma warning(push)
21712 |     #pragma warning(disable:4091) /* 'typedef ': ignored on left of '' when no variable is
> declared */
21713 | #endif
21714 | #include <audioclient.h>
21715 | #include <mmdeviceapi.h>
21716 | #if defined(_MSC_VER)
21717 |     #pragma warning(pop)
21718 | #endif
21719 | #endif /* 0 */
21720 |
21721 | static ma_result ma_device_reroute__wasapi(ma_device* pDevice, ma_device_type deviceType);
21722 |
21723 | /* Some compilers don't define VerifyVersionInfoW. Need to write this ourselves. */
21724 | #define MA_WIN32_WINNT_VISTA    0x0600
21725 | #define MA_VER_MINORVERSION     0x01
21726 | #define MA_VER_MAJORVERSION     0x02
21727 | #define MA_VER_SERVICEPACKMAJOR 0x20
21728 | #define MA_VER_GREATER_EQUAL    0x03
21729 |
21730 | typedef struct {
21731 |     DWORD dwOSVersionInfoSize;
21732 |     DWORD dwMajorVersion;
21733 |     DWORD dwMinorVersion;
21734 |     DWORD dwBuildNumber;
21735 |     DWORD dwPlatformId;
21736 |     WCHAR szCSDVersion[128];
21737 |     WORD wServicePackMajor;
21738 |     WORD wServicePackMinor;
21739 |     WORD wSuiteMask;
21740 |     BYTE wProductType;
21741 |     BYTE wReserved;
21742 | } ma_OSVERSIONINFOEXW;
21743 |
21744 | typedef BOOL (WINAPI * ma_PFNVerifyVersionInfoW)(ma_OSVERSIONINFOEXW* lpVersionInfo, DWORD
> dwTypeMask, DWORDLONG dwlConditionMask);
21745 | typedef ULONGLONG (WINAPI * ma_PFNVerSetConditionMask)(ULONGLONG dwlConditionMask, DWORD
> dwTypeBitMask, BYTE dwConditionMask);
21746 |
21747 |
21748 | #ifndef PROPERTYKEY_DEFINED
21749 | #define PROPERTYKEY_DEFINED
21750 | #ifndef __WATCOMC__
21751 | typedef struct
21752 | {
21753 |     GUID fmtid;

```

```

21754 |     DWORD pid;
21755 | } PROPERTYKEY;
21756 | #endif
21757 | #endif
21758 |
21759 | /* Some compilers don't define PropVariantInit(). We just do this ourselves since it's just a
> memset(). */
21760 | static MA_INLINE void ma_PropVariantInit(MA_PROPVARIANT* pProp)
21761 | {
21762 |     MA_ZERO_OBJECT(pProp);
21763 | }
21764 |
21765 |
21766 | static const PROPERTYKEY MA_PKEY_Device_FriendlyName = {{0xA45C254E, 0xDF1C, 0x4EFD,
> {0x80, 0x20, 0x67, 0xD1, 0x46, 0xA8, 0x50, 0xE0}}, 14};
21767 | static const PROPERTYKEY MA_PKEY_AudioEngine_DeviceFormat = {{0xF19F064D, 0x82C, 0x4E27,
> {0xBC, 0x73, 0x68, 0x82, 0xA1, 0xBB, 0x8E, 0x4C}}, 0};
21768 |
21769 | static const IID MA_IID_IUnknown = {0x00000000, 0x0000, 0x0000,
> {0xC0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x46}}; /* 00000000-0000-0000-C000-000000000046 */
21770 | #if !defined(MA_WIN32_DESKTOP) && !defined(MA_WIN32_GDK)
21771 | static const IID MA_IID_IAgileObject = {0x94EA2B94, 0xE9CC, 0x49E0,
> {0xC0, 0xFF, 0xEE, 0x64, 0xCA, 0x8F, 0x5B, 0x90}}; /* 94EA2B94-E9CC-49E0-C0FF-EE64CA8F5B90 */
21772 | #endif
21773 |
21774 | static const IID MA_IID_IAudioClient = {0x1CB9AD4C, 0xDBFA, 0x4C32,
> {0xB1, 0x78, 0xC2, 0xF5, 0x68, 0xA7, 0x03, 0xB2}}; /* 1CB9AD4C-DBFA-4C32-B178-C2F568A703B2 =
> __uuidof(IAudioClient) */
21775 | static const IID MA_IID_IAudioClient2 = {0x726778CD, 0xF60A, 0x4EDA,
> {0x82, 0xDE, 0xE4, 0x76, 0x10, 0xCD, 0x78, 0xAA}}; /* 726778CD-F60A-4EDA-82DE-E47610CD78AA =
> __uuidof(IAudioClient2) */
21776 | static const IID MA_IID_IAudioClient3 = {0x7ED4EE07, 0x8E67, 0x4CD4,
> {0x8C, 0x1A, 0x2B, 0x7A, 0x59, 0x87, 0xAD, 0x42}}; /* 7ED4EE07-8E67-4CD4-8C1A-2B7A5987AD42 =
> __uuidof(IAudioClient3) */
21777 | static const IID MA_IID_IAudioRenderClient = {0xF294ACFC, 0x3146, 0x4483,
> {0xA7, 0xBF, 0xAD, 0xDC, 0xA7, 0xC2, 0x60, 0xE2}}; /* F294ACFC-3146-4483-A7BF-ADDCA7C260E2 =
> __uuidof(IAudioRenderClient) */
21778 | static const IID MA_IID_IAudioCaptureClient = {0xC8ADB64, 0xE71E, 0x48A0,
> {0xA4, 0xDE, 0x18, 0x5C, 0x39, 0x5C, 0xD3, 0x17}}; /* C8ADB64-E71E-48A0-A4DE-185C395CD317 =
> __uuidof(IAudioCaptureClient) */
21779 | static const IID MA_IID_IMMNotificationClient = {0x7991EEC9, 0x7E89, 0x4D85,
> {0x83, 0x90, 0x6C, 0x70, 0x3C, 0xEC, 0x60, 0xC0}}; /* 7991EEC9-7E89-4D85-8390-6C703CEC60C0 =
> __uuidof(IMMNotificationClient) */
21780 | #if !defined(MA_WIN32_DESKTOP) && !defined(MA_WIN32_GDK)
21781 | static const IID MA_IID_DEVINTERFACE_AUDIO_RENDER = {0xE6327CAD, 0xDCEC, 0x4949,
> {0xAE, 0x8A, 0x99, 0x1E, 0x97, 0x6A, 0x79, 0xD2}}; /* E6327CAD-DCEC-4949-AE8A-991E976A79D2 */
21782 | static const IID MA_IID_DEVINTERFACE_AUDIO_CAPTURE = {0x2EEF81BE, 0x33FA, 0x4800,
> {0x96, 0x70, 0x1C, 0xD4, 0x74, 0x97, 0x2C, 0x3F}}; /* 2EEF81BE-33FA-4800-9670-1CD474972C3F */
21783 | static const IID MA_IID_IActivateAudioInterfaceCompletionHandler = {0x41D949AB, 0x9862, 0x444A,
> {0x80, 0xF6, 0xC2, 0x61, 0x33, 0x4D, 0xA5, 0xEB}}; /* 41D949AB-9862-444A-80F6-C261334DA5EB */
21784 | #endif
21785 |
21786 | static const IID MA_CLSID_MMDeviceEnumerator = {0xBCDE0395, 0xE52F, 0x467C,
> {0x8E, 0x3D, 0xC4, 0x57, 0x92, 0x91, 0x69, 0x2E}}; /* BCDE0395-E52F-467C-8E3D-C4579291692E =
> __uuidof(MMDeviceEnumerator) */
21787 | static const IID MA_IID_IMMDeviceEnumerator = {0xA95664D2, 0x9614, 0x4F35,
> {0xA7, 0x46, 0xDE, 0x8D, 0xB6, 0x36, 0x17, 0xE6}}; /* A95664D2-9614-4F35-A746-DE8DB63617E6 =
> __uuidof(IMMDeviceEnumerator) */
21788 |
21789 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
21790 | #define MA_MM_DEVICE_STATE_ACTIVE 1
21791 | #define MA_MM_DEVICE_STATE_DISABLED 2
21792 | #define MA_MM_DEVICE_STATE_NOTPRESENT 4
21793 | #define MA_MM_DEVICE_STATE_UNPLUGGED 8
21794 |
21795 | typedef struct ma_IMMDeviceEnumerator ma_IMMDeviceEnumerator;
21796 | typedef struct ma_IMMDeviceCollection ma_IMMDeviceCollection;
21797 | typedef struct ma_IMMDevice ma_IMMDevice;
21798 | #else
21799 | typedef struct ma_IActivateAudioInterfaceCompletionHandler
> ma_IActivateAudioInterfaceCompletionHandler;
21800 | typedef struct ma_IActivateAudioInterfaceAsyncOperation ma_IActivateAudioInterfaceAsyncOperation;
21801 | #endif
21802 | typedef struct ma_IPropertyStore ma_IPropertyStore;
21803 | typedef struct ma_IAudioClient ma_IAudioClient;
21804 | typedef struct ma_IAudioClient2 ma_IAudioClient2;
21805 | typedef struct ma_IAudioClient3 ma_IAudioClient3;
21806 | typedef struct ma_IAudioRenderClient ma_IAudioRenderClient;
21807 | typedef struct ma_IAudioCaptureClient ma_IAudioCaptureClient;

```

```

21808 |
21809 | typedef ma_int64                                MA_REFERENCE_TIME;
21810 |
21811 | #define MA_AUDCLNT_STREAMFLAGS_CROSSPROCESS      0x00010000
21812 | #define MA_AUDCLNT_STREAMFLAGS_LOOPBACK          0x00020000
21813 | #define MA_AUDCLNT_STREAMFLAGS_EVENTCALLBACK     0x00040000
21814 | #define MA_AUDCLNT_STREAMFLAGS_NOPERSIST         0x00080000
21815 | #define MA_AUDCLNT_STREAMFLAGS_RATEADJUST        0x00100000
21816 | #define MA_AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY 0x08000000
21817 | #define MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM     0x80000000
21818 | #define MA_AUDCLNT_SESSIONFLAGS_EXPIREWHENUNOWNED 0x10000000
21819 | #define MA_AUDCLNT_SESSIONFLAGS_DISPLAY_HIDE      0x20000000
21820 | #define MA_AUDCLNT_SESSIONFLAGS_DISPLAY_HIDEWHENEXPIRED 0x40000000
21821 |
21822 | /* Buffer flags. */
21823 | #define MA_AUDCLNT_BUFFERFLAGS_DATA_DISCONTINUITY 1
21824 | #define MA_AUDCLNT_BUFFERFLAGS_SILENT              2
21825 | #define MA_AUDCLNT_BUFFERFLAGS_TIMESTAMP_ERROR     4
21826 |
21827 | typedef enum
21828 | {
21829 |     ma_eRender = 0,
21830 |     ma_eCapture = 1,
21831 |     ma_eAll = 2
21832 | } ma_EDataFlow;
21833 |
21834 | typedef enum
21835 | {
21836 |     ma_eConsole = 0,
21837 |     ma_eMultimedia = 1,
21838 |     ma_eCommunications = 2
21839 | } ma_ERole;
21840 |
21841 | typedef enum
21842 | {
21843 |     MA_AUDCLNT_SHAREMODE_SHARED,
21844 |     MA_AUDCLNT_SHAREMODE_EXCLUSIVE
21845 | } MA_AUDCLNT_SHAREMODE;
21846 |
21847 | typedef enum
21848 | {
21849 |     MA_AudioCategory_Other = 0 /* <-- miniaudio is only caring about Other. */
21850 | } MA_AUDIO_STREAM_CATEGORY;
21851 |
21852 | typedef enum
21853 | {
21854 |     MA_AUDCLNT_STREAMOPTIONS_NONE,
21855 |     MA_AUDCLNT_STREAMOPTIONS_RAW,
21856 |     MA_AUDCLNT_STREAMOPTIONS_MATCH_FORMAT,
21857 |     MA_AUDCLNT_STREAMOPTIONS_AMBISONICS,
21858 |     MA_AUDCLNT_STREAMOPTIONS_POST_VOLUME_LOOPBACK
21859 | } MA_AUDCLNT_STREAMOPTIONS;
21860 |
21861 | typedef struct
21862 | {
21863 |     ma_uint32 cbSize;
21864 |     BOOL bIsOffload;
21865 |     MA_AUDIO_STREAM_CATEGORY eCategory;
21866 |     MA_AUDCLNT_STREAMOPTIONS Options;
21867 | } ma_AudioClientProperties;
21868 |
21869 | /* IUnknown */
21870 | typedef struct
21871 | {
21872 |     /* IUnknown */
21873 |     HRESULT ( STDMETHODCALLTYPE * QueryInterface)(ma_IUnknown* pThis, const IID* const riid, void**
> ppObject);
21874 |     ULONG ( STDMETHODCALLTYPE * AddRef) (ma_IUnknown* pThis);
21875 |     ULONG ( STDMETHODCALLTYPE * Release) (ma_IUnknown* pThis);
21876 | } ma_IUnknownVtbl;
21877 | struct ma_IUnknown
21878 | {
21879 |     ma_IUnknownVtbl* lpVtbl;
21880 | };
21881 | static MA_INLINE HRESULT ma_IUnknown_QueryInterface(ma_IUnknown* pThis, const IID* const riid,
> void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
21882 | static MA_INLINE ULONG ma_IUnknown_AddRef(ma_IUnknown* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
21883 | static MA_INLINE ULONG ma_IUnknown_Release(ma_IUnknown* pThis)

```

```

>                                     { return pThis->lpVtbl->Release(pThis); }
21884 |
21885 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
21886 |     /* IMMNotificationClient */
21887 |     typedef struct
21888 |     {
21889 |         /* IUnknown */
21890 |         HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IMMNotificationClient* pThis, const IID*
> const riid, void** ppObject);
21891 |         ULONG (STDMETHODCALLTYPE * AddRef) (ma_IMMNotificationClient* pThis);
21892 |         ULONG (STDMETHODCALLTYPE * Release) (ma_IMMNotificationClient* pThis);
21893 |
21894 |         /* IMMNotificationClient */
21895 |         HRESULT (STDMETHODCALLTYPE * OnDeviceStateChanged) (ma_IMMNotificationClient* pThis, const
> WCHAR* pDeviceID, DWORD dwNewState);
21896 |         HRESULT (STDMETHODCALLTYPE * OnDeviceAdded) (ma_IMMNotificationClient* pThis, const
> WCHAR* pDeviceID);
21897 |         HRESULT (STDMETHODCALLTYPE * OnDeviceRemoved) (ma_IMMNotificationClient* pThis, const
> WCHAR* pDeviceID);
21898 |         HRESULT (STDMETHODCALLTYPE * OnDefaultDeviceChanged)(ma_IMMNotificationClient* pThis,
> ma_EDataFlow dataFlow, ma_ERole role, const WCHAR* pDefaultDeviceID);
21899 |         HRESULT (STDMETHODCALLTYPE * OnPropertyValueChanged)(ma_IMMNotificationClient* pThis, const
> WCHAR* pDeviceID, const PROPERTYKEY key);
21900 |     } ma_IMMNotificationClientVtbl;
21901 |
21902 |     /* IMMDeviceEnumerator */
21903 |     typedef struct
21904 |     {
21905 |         /* IUnknown */
21906 |         HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IMMDeviceEnumerator* pThis, const IID* const
> riid, void** ppObject);
21907 |         ULONG (STDMETHODCALLTYPE * AddRef) (ma_IMMDeviceEnumerator* pThis);
21908 |         ULONG (STDMETHODCALLTYPE * Release) (ma_IMMDeviceEnumerator* pThis);
21909 |
21910 |         /* IMMDeviceEnumerator */
21911 |         HRESULT (STDMETHODCALLTYPE * EnumAudioEndpoints) (ma_IMMDeviceEnumerator*
> pThis, ma_EDataFlow dataFlow, DWORD dwStateMask, ma_IMMDeviceCollection** ppDevices);
21912 |         HRESULT (STDMETHODCALLTYPE * GetDefaultAudioEndpoint) (ma_IMMDeviceEnumerator*
> pThis, ma_EDataFlow dataFlow, ma_ERole role, ma_IMMDevice** ppEndpoint);
21913 |         HRESULT (STDMETHODCALLTYPE * GetDevice) (ma_IMMDeviceEnumerator*
> pThis, const WCHAR* pID, ma_IMMDevice** ppDevice);
21914 |         HRESULT (STDMETHODCALLTYPE * RegisterEndpointNotificationCallback) (ma_IMMDeviceEnumerator*
> pThis, ma_IMMNotificationClient* pClient);
21915 |         HRESULT (STDMETHODCALLTYPE * UnregisterEndpointNotificationCallback)(ma_IMMDeviceEnumerator*
> pThis, ma_IMMNotificationClient* pClient);
21916 |     } ma_IMMDeviceEnumeratorVtbl;
21917 |     struct ma_IMMDeviceEnumerator
21918 |     {
21919 |         ma_IMMDeviceEnumeratorVtbl* lpVtbl;
21920 |     };
21921 |     static MA_INLINE HRESULT ma_IMMDeviceEnumerator_QueryInterface(ma_IMMDeviceEnumerator* pThis,
> const IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid,
> ppObject); }
21922 |     static MA_INLINE ULONG ma_IMMDeviceEnumerator_AddRef(ma_IMMDeviceEnumerator* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
21923 |     static MA_INLINE ULONG ma_IMMDeviceEnumerator_Release(ma_IMMDeviceEnumerator* pThis)
> { return pThis->lpVtbl->Release(pThis); }
21924 |     static MA_INLINE HRESULT ma_IMMDeviceEnumerator_EnumAudioEndpoints(ma_IMMDeviceEnumerator*
> pThis, ma_EDataFlow dataFlow, DWORD dwStateMask, ma_IMMDeviceCollection** ppDevices) { return
> pThis->lpVtbl->EnumAudioEndpoints(pThis, dataFlow, dwStateMask, ppDevices); }
21925 |     static MA_INLINE HRESULT ma_IMMDeviceEnumerator_GetDefaultAudioEndpoint(ma_IMMDeviceEnumerator*
> pThis, ma_EDataFlow dataFlow, ma_ERole role, ma_IMMDevice** ppEndpoint) { return
> pThis->lpVtbl->GetDefaultAudioEndpoint(pThis, dataFlow, role, ppEndpoint); }
21926 |     static MA_INLINE HRESULT ma_IMMDeviceEnumerator_GetDevice(ma_IMMDeviceEnumerator* pThis, const
> WCHAR* pID, ma_IMMDevice** ppDevice) { return pThis->lpVtbl->GetDevice(pThis, pID, ppDevice); }
21927 |     static MA_INLINE HRESULT
> ma_IMMDeviceEnumerator_RegisterEndpointNotificationCallback(ma_IMMDeviceEnumerator* pThis,
> ma_IMMNotificationClient* pClient) { return
> pThis->lpVtbl->RegisterEndpointNotificationCallback(pThis, pClient); }
21928 |     static MA_INLINE HRESULT
> ma_IMMDeviceEnumerator_UnregisterEndpointNotificationCallback(ma_IMMDeviceEnumerator* pThis,
> ma_IMMNotificationClient* pClient) { return
> pThis->lpVtbl->UnregisterEndpointNotificationCallback(pThis, pClient); }
21929 |
21930 |
21931 |     /* IMMDeviceCollection */
21932 |     typedef struct
21933 |     {
21934 |         /* IUnknown */

```



```

21935 |         HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IMMDeviceCollection* pThis, const IID* const
> riid, void** ppObject);
21936 |         ULONG (STDMETHODCALLTYPE * AddRef) (ma_IMMDeviceCollection* pThis);
21937 |         ULONG (STDMETHODCALLTYPE * Release) (ma_IMMDeviceCollection* pThis);
21938 |
21939 |         /* IMMDeviceCollection */
21940 |         HRESULT (STDMETHODCALLTYPE * GetCount)(ma_IMMDeviceCollection* pThis, UINT* pDevices);
21941 |         HRESULT (STDMETHODCALLTYPE * Item) (ma_IMMDeviceCollection* pThis, UINT nDevice,
> ma_IMMDevice** ppDevice);
21942 |     } ma_IMMDeviceCollectionVtbl;
21943 |     struct ma_IMMDeviceCollection
21944 |     {
21945 |         ma_IMMDeviceCollectionVtbl* lpVtbl;
21946 |     };
21947 |     static MA_INLINE HRESULT ma_IMMDeviceCollection_QueryInterface(ma_IMMDeviceCollection* pThis,
> const IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid,
> ppObject); }
21948 |     static MA_INLINE ULONG ma_IMMDeviceCollection_AddRef(ma_IMMDeviceCollection* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
21949 |     static MA_INLINE ULONG ma_IMMDeviceCollection_Release(ma_IMMDeviceCollection* pThis)
> { return pThis->lpVtbl->Release(pThis); }
21950 |     static MA_INLINE HRESULT ma_IMMDeviceCollection_GetCount(ma_IMMDeviceCollection* pThis, UINT*
> pDevices) { return pThis->lpVtbl->GetCount(pThis, pDevices); }
21951 |     static MA_INLINE HRESULT ma_IMMDeviceCollection_Item(ma_IMMDeviceCollection* pThis, UINT
> nDevice, ma_IMMDevice** ppDevice) { return pThis->lpVtbl->Item(pThis, nDevice, ppDevice);
> }
21952 |
21953 |
21954 |     /* IMMDevice */
21955 |     typedef struct
21956 |     {
21957 |         /* IUnknown */
21958 |         HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IMMDevice* pThis, const IID* const riid,
> void** ppObject);
21959 |         ULONG (STDMETHODCALLTYPE * AddRef) (ma_IMMDevice* pThis);
21960 |         ULONG (STDMETHODCALLTYPE * Release) (ma_IMMDevice* pThis);
21961 |
21962 |         /* IMMDevice */
21963 |         HRESULT (STDMETHODCALLTYPE * Activate) (ma_IMMDevice* pThis, const IID* const iid,
> DWORD dwClsCtx, MA_PROPVARIANT* pActivationParams, void** ppInterface);
21964 |         HRESULT (STDMETHODCALLTYPE * OpenPropertyStore)(ma_IMMDevice* pThis, DWORD stgmAccess,
> ma_IPropertyStore** ppProperties);
21965 |         HRESULT (STDMETHODCALLTYPE * GetId) (ma_IMMDevice* pThis, WCHAR** pID);
21966 |         HRESULT (STDMETHODCALLTYPE * GetState) (ma_IMMDevice* pThis, DWORD *pState);
21967 |     } ma_IMMDeviceVtbl;
21968 |     struct ma_IMMDevice
21969 |     {
21970 |         ma_IMMDeviceVtbl* lpVtbl;
21971 |     };
21972 |     static MA_INLINE HRESULT ma_IMMDevice_QueryInterface(ma_IMMDevice* pThis, const IID* const riid,
> void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
21973 |     static MA_INLINE ULONG ma_IMMDevice_AddRef(ma_IMMDevice* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
21974 |     static MA_INLINE ULONG ma_IMMDevice_Release(ma_IMMDevice* pThis)
> { return pThis->lpVtbl->Release(pThis); }
21975 |     static MA_INLINE HRESULT ma_IMMDevice_Activate(ma_IMMDevice* pThis, const IID* const iid, DWORD
> dwClsCtx, MA_PROPVARIANT* pActivationParams, void** ppInterface) { return
> pThis->lpVtbl->Activate(pThis, iid, dwClsCtx, pActivationParams, ppInterface); }
21976 |     static MA_INLINE HRESULT ma_IMMDevice_OpenPropertyStore(ma_IMMDevice* pThis, DWORD stgmAccess,
> ma_IPropertyStore** ppProperties) { return pThis->lpVtbl->OpenPropertyStore(pThis, stgmAccess,
> ppProperties); }
21977 |     static MA_INLINE HRESULT ma_IMMDevice_GetId(ma_IMMDevice* pThis, WCHAR** pID)
> { return pThis->lpVtbl->GetId(pThis, pID); }
21978 |     static MA_INLINE HRESULT ma_IMMDevice_GetState(ma_IMMDevice* pThis, DWORD *pState)
> { return pThis->lpVtbl->GetState(pThis, pState); }
21979 | #else
21980 |     /* IActivateAudioInterfaceAsyncOperation */
21981 |     typedef struct
21982 |     {
21983 |         /* IUnknown */
21984 |         HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IActivateAudioInterfaceAsyncOperation*
> pThis, const IID* const riid, void** ppObject);
21985 |         ULONG (STDMETHODCALLTYPE * AddRef) (ma_IActivateAudioInterfaceAsyncOperation*
> pThis);
21986 |         ULONG (STDMETHODCALLTYPE * Release) (ma_IActivateAudioInterfaceAsyncOperation*
> pThis);
21987 |
21988 |         /* IActivateAudioInterfaceAsyncOperation */
21989 |         HRESULT (STDMETHODCALLTYPE * GetActivateResult)(ma_IActivateAudioInterfaceAsyncOperation*

```

```

    > pThis, HRESULT *pActivateResult, ma_IUnknown** ppActivatedInterface);
21990 |     } ma_IActivateAudioInterfaceAsyncOperationVtbl;
21991 |     struct ma_IActivateAudioInterfaceAsyncOperation
21992 |     {
21993 |         ma_IActivateAudioInterfaceAsyncOperationVtbl* lpVtbl;
21994 |     };
21995 |     static MA_INLINE HRESULT
    > ma_IActivateAudioInterfaceAsyncOperation_QueryInterface(ma_IActivateAudioInterfaceAsyncOperation*
    > pThis, const IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid,
    > ppObject); }
21996 |     static MA_INLINE ULONG
    > ma_IActivateAudioInterfaceAsyncOperation_AddRef(ma_IActivateAudioInterfaceAsyncOperation* pThis)
    > { return pThis->lpVtbl->AddRef(pThis); }
21997 |     static MA_INLINE ULONG
    > ma_IActivateAudioInterfaceAsyncOperation_Release(ma_IActivateAudioInterfaceAsyncOperation* pThis)
    > { return pThis->lpVtbl->Release(pThis); }
21998 |     static MA_INLINE HRESULT
    > ma_IActivateAudioInterfaceAsyncOperation_GetActivateResult(ma_IActivateAudioInterfaceAsyncOperation*
    > pThis, HRESULT *pActivateResult, ma_IUnknown** ppActivatedInterface) { return
    > pThis->lpVtbl->GetActivateResult(pThis, pActivateResult, ppActivatedInterface); }
21999 | #endif
22000 |
22001 | /* IPropertyStore */
22002 | typedef struct
22003 | {
22004 |     /* IUnknown */
22005 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IPropertyStore* pThis, const IID* const riid,
    > void** ppObject);
22006 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IPropertyStore* pThis);
22007 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IPropertyStore* pThis);
22008 |
22009 |     /* IPropertyStore */
22010 |     HRESULT (STDMETHODCALLTYPE * GetCount)(ma_IPropertyStore* pThis, DWORD* pPropCount);
22011 |     HRESULT (STDMETHODCALLTYPE * GetAt) (ma_IPropertyStore* pThis, DWORD propIndex, PROPERTYKEY*
    > pPropKey);
22012 |     HRESULT (STDMETHODCALLTYPE * GetValue)(ma_IPropertyStore* pThis, const PROPERTYKEY* const pKey,
    > MA_PROPVARIANT* pPropVar);
22013 |     HRESULT (STDMETHODCALLTYPE * SetValue)(ma_IPropertyStore* pThis, const PROPERTYKEY* const pKey,
    > const MA_PROPVARIANT* const pPropVar);
22014 |     HRESULT (STDMETHODCALLTYPE * Commit) (ma_IPropertyStore* pThis);
22015 | } ma_IPropertyStoreVtbl;
22016 | struct ma_IPropertyStore
22017 | {
22018 |     ma_IPropertyStoreVtbl* lpVtbl;
22019 | };
22020 | static MA_INLINE HRESULT ma_IPropertyStore_QueryInterface(ma_IPropertyStore* pThis, const IID* const
    > riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
22021 | static MA_INLINE ULONG ma_IPropertyStore_AddRef(ma_IPropertyStore* pThis)
    > { return pThis->lpVtbl->AddRef(pThis); }
22022 | static MA_INLINE ULONG ma_IPropertyStore_Release(ma_IPropertyStore* pThis)
    > { return pThis->lpVtbl->Release(pThis); }
22023 | static MA_INLINE HRESULT ma_IPropertyStore_GetCount(ma_IPropertyStore* pThis, DWORD* pPropCount)
    > { return pThis->lpVtbl->GetCount(pThis, pPropCount); }
22024 | static MA_INLINE HRESULT ma_IPropertyStore_GetAt(ma_IPropertyStore* pThis, DWORD propIndex,
    > PROPERTYKEY* pPropKey) { return pThis->lpVtbl->GetAt(pThis, propIndex, pPropKey); }
22025 | static MA_INLINE HRESULT ma_IPropertyStore_GetValue(ma_IPropertyStore* pThis, const PROPERTYKEY*
    > const pKey, MA_PROPVARIANT* pPropVar) { return pThis->lpVtbl->GetValue(pThis, pKey, pPropVar); }
22026 | static MA_INLINE HRESULT ma_IPropertyStore_SetValue(ma_IPropertyStore* pThis, const PROPERTYKEY*
    > const pKey, const MA_PROPVARIANT* const pPropVar) { return pThis->lpVtbl->SetValue(pThis, pKey,
    > pPropVar); }
22027 | static MA_INLINE HRESULT ma_IPropertyStore_Commit(ma_IPropertyStore* pThis)
    > { return pThis->lpVtbl->Commit(pThis); }
22028 |
22029 |
22030 | /* IAudioClient */
22031 | typedef struct
22032 | {
22033 |     /* IUnknown */
22034 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IAudioClient* pThis, const IID* const riid,
    > void** ppObject);
22035 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IAudioClient* pThis);
22036 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IAudioClient* pThis);
22037 |
22038 |     /* IAudioClient */
22039 |     HRESULT (STDMETHODCALLTYPE * Initialize) (ma_IAudioClient* pThis, MA_AUDCLNT_SHAREMODE
    > shareMode, DWORD streamFlags, MA_REFERENCE_TIME bufferDuration, MA_REFERENCE_TIME periodicity, const
    > MA_WAVEFORMATEX* pFormat, const GUID* pAudioSessionGuid);
22040 |     HRESULT (STDMETHODCALLTYPE * GetBufferSize) (ma_IAudioClient* pThis, ma_uint32*
    > pNumBufferFrames);

```

```

22041 |     HRESULT (STDMETHODCALLTYPE * GetStreamLatency) (ma_IAudioClient* pThis, MA_REFERENCE_TIME*
> pLatency);
22042 |     HRESULT (STDMETHODCALLTYPE * GetCurrentPadding)(ma_IAudioClient* pThis, ma_uint32*
> pNumPaddingFrames);
22043 |     HRESULT (STDMETHODCALLTYPE * IsFormatSupported)(ma_IAudioClient* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, const MA_WAVEFORMATEX* pFormat, MA_WAVEFORMATEX** ppClosestMatch);
22044 |     HRESULT (STDMETHODCALLTYPE * GetMixFormat) (ma_IAudioClient* pThis, MA_WAVEFORMATEX**
> ppDeviceFormat);
22045 |     HRESULT (STDMETHODCALLTYPE * GetDevicePeriod) (ma_IAudioClient* pThis, MA_REFERENCE_TIME*
> pDefaultDevicePeriod, MA_REFERENCE_TIME* pMinimumDevicePeriod);
22046 |     HRESULT (STDMETHODCALLTYPE * Start) (ma_IAudioClient* pThis);
22047 |     HRESULT (STDMETHODCALLTYPE * Stop) (ma_IAudioClient* pThis);
22048 |     HRESULT (STDMETHODCALLTYPE * Reset) (ma_IAudioClient* pThis);
22049 |     HRESULT (STDMETHODCALLTYPE * SetEventHandle) (ma_IAudioClient* pThis, HANDLE eventHandle);
22050 |     HRESULT (STDMETHODCALLTYPE * GetService) (ma_IAudioClient* pThis, const IID* const riid,
> void** pp);
22051 | } ma_IAudioClientVtbl;
22052 | struct ma_IAudioClient
22053 | {
22054 |     ma_IAudioClientVtbl* lpVtbl;
22055 | };
22056 | static MA_INLINE HRESULT ma_IAudioClient_QueryInterface(ma_IAudioClient* pThis, const IID* const
> riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
22057 | static MA_INLINE ULONG ma_IAudioClient_AddRef(ma_IAudioClient* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
22058 | static MA_INLINE ULONG ma_IAudioClient_Release(ma_IAudioClient* pThis)
> { return pThis->lpVtbl->Release(pThis); }
22059 | static MA_INLINE HRESULT ma_IAudioClient_Initialize(ma_IAudioClient* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, DWORD streamFlags, MA_REFERENCE_TIME bufferDuration, MA_REFERENCE_TIME periodicity, const
> MA_WAVEFORMATEX* pFormat, const GUID* pAudioSessionGuid) { return pThis->lpVtbl->Initialize(pThis,
> shareMode, streamFlags, bufferDuration, periodicity, pFormat, pAudioSessionGuid); }
22060 | static MA_INLINE HRESULT ma_IAudioClient_GetBufferSize(ma_IAudioClient* pThis, ma_uint32*
> pNumBufferFrames) { return pThis->lpVtbl->GetBufferSize(pThis, pNumBufferFrames); }
22061 | static MA_INLINE HRESULT ma_IAudioClient_GetStreamLatency(ma_IAudioClient* pThis, MA_REFERENCE_TIME*
> pLatency) { return pThis->lpVtbl->GetStreamLatency(pThis, pLatency); }
22062 | static MA_INLINE HRESULT ma_IAudioClient_GetCurrentPadding(ma_IAudioClient* pThis, ma_uint32*
> pNumPaddingFrames) { return pThis->lpVtbl->GetCurrentPadding(pThis, pNumPaddingFrames); }
22063 | static MA_INLINE HRESULT ma_IAudioClient_IsFormatSupported(ma_IAudioClient* pThis,
> MA_AUDCLNT_SHAREMODE shareMode, const MA_WAVEFORMATEX* pFormat, MA_WAVEFORMATEX** ppClosestMatch) {
> return pThis->lpVtbl->IsFormatSupported(pThis, shareMode, pFormat, ppClosestMatch); }
22064 | static MA_INLINE HRESULT ma_IAudioClient_GetMixFormat(ma_IAudioClient* pThis, MA_WAVEFORMATEX**
> ppDeviceFormat) { return pThis->lpVtbl->GetMixFormat(pThis, ppDeviceFormat); }
22065 | static MA_INLINE HRESULT ma_IAudioClient_GetDevicePeriod(ma_IAudioClient* pThis, MA_REFERENCE_TIME*
> pDefaultDevicePeriod, MA_REFERENCE_TIME* pMinimumDevicePeriod) { return
> pThis->lpVtbl->GetDevicePeriod(pThis, pDefaultDevicePeriod, pMinimumDevicePeriod); }
22066 | static MA_INLINE HRESULT ma_IAudioClient_Start(ma_IAudioClient* pThis)
> { return pThis->lpVtbl->Start(pThis); }
22067 | static MA_INLINE HRESULT ma_IAudioClient_Stop(ma_IAudioClient* pThis)
> { return pThis->lpVtbl->Stop(pThis); }
22068 | static MA_INLINE HRESULT ma_IAudioClient_Reset(ma_IAudioClient* pThis)
> { return pThis->lpVtbl->Reset(pThis); }
22069 | static MA_INLINE HRESULT ma_IAudioClient_SetEventHandle(ma_IAudioClient* pThis, HANDLE eventHandle)
> { return pThis->lpVtbl->SetEventHandle(pThis, eventHandle); }
22070 | static MA_INLINE HRESULT ma_IAudioClient_GetService(ma_IAudioClient* pThis, const IID* const riid,
> void** pp) { return pThis->lpVtbl->GetService(pThis, riid, pp); }
22071 |
22072 | /* IAudioClient2 */
22073 | typedef struct
22074 | {
22075 |     /* IUnknown */
22076 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IAudioClient2* pThis, const IID* const riid,
> void** ppObject);
22077 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IAudioClient2* pThis);
22078 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IAudioClient2* pThis);
22079 |
22080 |     /* IAudioClient */
22081 |     HRESULT (STDMETHODCALLTYPE * Initialize) (ma_IAudioClient2* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, DWORD streamFlags, MA_REFERENCE_TIME bufferDuration, MA_REFERENCE_TIME periodicity, const
> MA_WAVEFORMATEX* pFormat, const GUID* pAudioSessionGuid);
22082 |     HRESULT (STDMETHODCALLTYPE * GetBufferSize) (ma_IAudioClient2* pThis, ma_uint32*
> pNumBufferFrames);
22083 |     HRESULT (STDMETHODCALLTYPE * GetStreamLatency) (ma_IAudioClient2* pThis, MA_REFERENCE_TIME*
> pLatency);
22084 |     HRESULT (STDMETHODCALLTYPE * GetCurrentPadding)(ma_IAudioClient2* pThis, ma_uint32*
> pNumPaddingFrames);
22085 |     HRESULT (STDMETHODCALLTYPE * IsFormatSupported)(ma_IAudioClient2* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, const MA_WAVEFORMATEX* pFormat, MA_WAVEFORMATEX** ppClosestMatch);
22086 |     HRESULT (STDMETHODCALLTYPE * GetMixFormat) (ma_IAudioClient2* pThis, MA_WAVEFORMATEX**
> ppDeviceFormat);

```

```

22087 |     HRESULT (STDMETHODCALLTYPE * GetDevicePeriod) (ma_IAudioClient2* pThis, MA_REFERENCE_TIME*
> pDefaultDevicePeriod, MA_REFERENCE_TIME* pMinimumDevicePeriod);
22088 |     HRESULT (STDMETHODCALLTYPE * Start) (ma_IAudioClient2* pThis);
22089 |     HRESULT (STDMETHODCALLTYPE * Stop) (ma_IAudioClient2* pThis);
22090 |     HRESULT (STDMETHODCALLTYPE * Reset) (ma_IAudioClient2* pThis);
22091 |     HRESULT (STDMETHODCALLTYPE * SetEventHandle) (ma_IAudioClient2* pThis, HANDLE eventHandle);
22092 |     HRESULT (STDMETHODCALLTYPE * GetService) (ma_IAudioClient2* pThis, const IID* const riid,
> void** pp);
22093 |
22094 |     /* IAudioClient2 */
22095 |     HRESULT (STDMETHODCALLTYPE * IsOffloadCapable) (ma_IAudioClient2* pThis,
> MA_AUDIO_STREAM_CATEGORY category, BOOL* pOffloadCapable);
22096 |     HRESULT (STDMETHODCALLTYPE * SetClientProperties)(ma_IAudioClient2* pThis, const
> ma_AudioClientProperties* pProperties);
22097 |     HRESULT (STDMETHODCALLTYPE * GetBufferSizeLimits)(ma_IAudioClient2* pThis, const
> MA_WAVEFORMATEX* pFormat, BOOL eventDriven, MA_REFERENCE_TIME* pMinBufferDuration,
> MA_REFERENCE_TIME* pMaxBufferDuration);
22098 | } ma_IAudioClient2Vtbl;
22099 | struct ma_IAudioClient2
22100 | {
22101 |     ma_IAudioClient2Vtbl* lpVtbl;
22102 | };
22103 | static MA_INLINE HRESULT ma_IAudioClient2_QueryInterface(ma_IAudioClient2* pThis, const IID* const
> riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
22104 | static MA_INLINE ULONG ma_IAudioClient2_AddRef(ma_IAudioClient2* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
22105 | static MA_INLINE ULONG ma_IAudioClient2_Release(ma_IAudioClient2* pThis)
> { return pThis->lpVtbl->Release(pThis); }
22106 | static MA_INLINE HRESULT ma_IAudioClient2_Initialize(ma_IAudioClient2* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, DWORD streamFlags, MA_REFERENCE_TIME bufferDuration, MA_REFERENCE_TIME periodicity, const
> MA_WAVEFORMATEX* pFormat, const GUID* pAudioSessionGuid) { return pThis->lpVtbl->Initialize(pThis,
> shareMode, streamFlags, bufferDuration, periodicity, pFormat, pAudioSessionGuid); }
22107 | static MA_INLINE HRESULT ma_IAudioClient2_GetBufferSize(ma_IAudioClient2* pThis, ma_uint32*
> pNumBufferFrames) { return pThis->lpVtbl->GetBufferSize(pThis, pNumBufferFrames); }
22108 | static MA_INLINE HRESULT ma_IAudioClient2_GetStreamLatency(ma_IAudioClient2* pThis,
> MA_REFERENCE_TIME* pLatency) { return pThis->lpVtbl->GetStreamLatency(pThis, pLatency); }
> }
22109 | static MA_INLINE HRESULT ma_IAudioClient2_GetCurrentPadding(ma_IAudioClient2* pThis, ma_uint32*
> pNumPaddingFrames) { return pThis->lpVtbl->GetCurrentPadding(pThis, pNumPaddingFrames); }
22110 | static MA_INLINE HRESULT ma_IAudioClient2_IsFormatSupported(ma_IAudioClient2* pThis,
> MA_AUDCLNT_SHAREMODE shareMode, const MA_WAVEFORMATEX* pFormat, MA_WAVEFORMATEX** ppClosestMatch) {
> return pThis->lpVtbl->IsFormatSupported(pThis, shareMode, pFormat, ppClosestMatch); }
22111 | static MA_INLINE HRESULT ma_IAudioClient2_GetMixFormat(ma_IAudioClient2* pThis, MA_WAVEFORMATEX**
> ppDeviceFormat) { return pThis->lpVtbl->GetMixFormat(pThis, ppDeviceFormat); }
22112 | static MA_INLINE HRESULT ma_IAudioClient2_GetDevicePeriod(ma_IAudioClient2* pThis,
> MA_REFERENCE_TIME* pDefaultDevicePeriod, MA_REFERENCE_TIME* pMinimumDevicePeriod) { return
> pThis->lpVtbl->GetDevicePeriod(pThis, pDefaultDevicePeriod, pMinimumDevicePeriod); }
22113 | static MA_INLINE HRESULT ma_IAudioClient2_Start(ma_IAudioClient2* pThis)
> { return pThis->lpVtbl->Start(pThis); }
22114 | static MA_INLINE HRESULT ma_IAudioClient2_Stop(ma_IAudioClient2* pThis)
> { return pThis->lpVtbl->Stop(pThis); }
22115 | static MA_INLINE HRESULT ma_IAudioClient2_Reset(ma_IAudioClient2* pThis)
> { return pThis->lpVtbl->Reset(pThis); }
22116 | static MA_INLINE HRESULT ma_IAudioClient2_SetEventHandle(ma_IAudioClient2* pThis, HANDLE
> eventHandle) { return pThis->lpVtbl->SetEventHandle(pThis, eventHandle); }
22117 | static MA_INLINE HRESULT ma_IAudioClient2_GetService(ma_IAudioClient2* pThis, const IID* const riid,
> void** pp) { return pThis->lpVtbl->GetService(pThis, riid, pp); }
22118 | static MA_INLINE HRESULT ma_IAudioClient2_IsOffloadCapable(ma_IAudioClient2* pThis,
> MA_AUDIO_STREAM_CATEGORY category, BOOL* pOffloadCapable) { return
> pThis->lpVtbl->IsOffloadCapable(pThis, category, pOffloadCapable); }
22119 | static MA_INLINE HRESULT ma_IAudioClient2_SetClientProperties(ma_IAudioClient2* pThis, const
> ma_AudioClientProperties* pProperties) { return pThis->lpVtbl->SetClientProperties(pThis,
> pProperties); }
22120 | static MA_INLINE HRESULT ma_IAudioClient2_GetBufferSizeLimits(ma_IAudioClient2* pThis, const
> MA_WAVEFORMATEX* pFormat, BOOL eventDriven, MA_REFERENCE_TIME* pMinBufferDuration,
> MA_REFERENCE_TIME* pMaxBufferDuration) { return pThis->lpVtbl->GetBufferSizeLimits(pThis, pFormat,
> eventDriven, pMinBufferDuration, pMaxBufferDuration); }
22121 |
22122 |
22123 | /* IAudioClient3 */
22124 | typedef struct
22125 | {
22126 |     /* IUnknown */
22127 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IAudioClient3* pThis, const IID* const riid,
> void** ppObject);
22128 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IAudioClient3* pThis);
22129 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IAudioClient3* pThis);
22130 |
22131 |     /* IAudioClient */

```

```

22132 |     HRESULT (STDMETHODCALLTYPE * Initialize)          (ma_IAudioClient3* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, DWORD streamFlags, MA_REFERENCE_TIME bufferDuration, MA_REFERENCE_TIME periodicity, const
> MA_WAVEFORMATEX* pFormat, const GUID* pAudioSessionGuid);
22133 |     HRESULT (STDMETHODCALLTYPE * GetBufferSize)      (ma_IAudioClient3* pThis, ma_uint32*
> pNumBufferFrames);
22134 |     HRESULT (STDMETHODCALLTYPE * GetStreamLatency)   (ma_IAudioClient3* pThis, MA_REFERENCE_TIME*
> pLatency);
22135 |     HRESULT (STDMETHODCALLTYPE * GetCurrentPadding)(ma_IAudioClient3* pThis, ma_uint32*
> pNumPaddingFrames);
22136 |     HRESULT (STDMETHODCALLTYPE * IsFormatSupported)(ma_IAudioClient3* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, const MA_WAVEFORMATEX* pFormat, MA_WAVEFORMATEX** ppClosestMatch);
22137 |     HRESULT (STDMETHODCALLTYPE * GetMixFormat)       (ma_IAudioClient3* pThis, MA_WAVEFORMATEX**
> ppDeviceFormat);
22138 |     HRESULT (STDMETHODCALLTYPE * GetDevicePeriod)    (ma_IAudioClient3* pThis, MA_REFERENCE_TIME*
> pDefaultDevicePeriod, MA_REFERENCE_TIME* pMinimumDevicePeriod);
22139 |     HRESULT (STDMETHODCALLTYPE * Start)              (ma_IAudioClient3* pThis);
22140 |     HRESULT (STDMETHODCALLTYPE * Stop)              (ma_IAudioClient3* pThis);
22141 |     HRESULT (STDMETHODCALLTYPE * Reset)             (ma_IAudioClient3* pThis);
22142 |     HRESULT (STDMETHODCALLTYPE * SetEventHandle)     (ma_IAudioClient3* pThis, HANDLE eventHandle);
22143 |     HRESULT (STDMETHODCALLTYPE * GetService)        (ma_IAudioClient3* pThis, const IID* const riid,
> void** pp);
22144 |
22145 |     /* IAudioClient2 */
22146 |     HRESULT (STDMETHODCALLTYPE * IsOffloadCapable)   (ma_IAudioClient3* pThis,
> MA_AUDIO_STREAM_CATEGORY category, BOOL* pOffloadCapable);
22147 |     HRESULT (STDMETHODCALLTYPE * SetClientProperties)(ma_IAudioClient3* pThis, const
> ma_AudioClientProperties* pProperties);
22148 |     HRESULT (STDMETHODCALLTYPE * GetBufferSizeLimits)(ma_IAudioClient3* pThis, const
> MA_WAVEFORMATEX* pFormat, BOOL eventDriven, MA_REFERENCE_TIME* pMinBufferDuration,
> MA_REFERENCE_TIME* pMaxBufferDuration);
22149 |
22150 |     /* IAudioClient3 */
22151 |     HRESULT (STDMETHODCALLTYPE * GetSharedModeEnginePeriod) (ma_IAudioClient3* pThis, const
> MA_WAVEFORMATEX* pFormat, ma_uint32* pDefaultPeriodInFrames, ma_uint32* pFundamentalPeriodInFrames,
> ma_uint32* pMinPeriodInFrames, ma_uint32* pMaxPeriodInFrames);
22152 |     HRESULT (STDMETHODCALLTYPE * GetCurrentSharedModeEnginePeriod)(ma_IAudioClient3* pThis,
> MA_WAVEFORMATEX** ppFormat, ma_uint32* pCurrentPeriodInFrames);
22153 |     HRESULT (STDMETHODCALLTYPE * InitializeSharedAudioStream) (ma_IAudioClient3* pThis, DWORD
> streamFlags, ma_uint32 periodInFrames, const MA_WAVEFORMATEX* pFormat, const GUID*
> pAudioSessionGuid);
22154 | } ma_IAudioClient3Vtbl;
22155 | struct ma_IAudioClient3
22156 | {
22157 |     ma_IAudioClient3Vtbl* lpVtbl;
22158 | };
22159 | static MA_INLINE HRESULT ma_IAudioClient3_QueryInterface(ma_IAudioClient3* pThis, const IID* const
> riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
22160 | static MA_INLINE ULONG ma_IAudioClient3_AddRef(ma_IAudioClient3* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
22161 | static MA_INLINE ULONG ma_IAudioClient3_Release(ma_IAudioClient3* pThis)
> { return pThis->lpVtbl->Release(pThis); }
22162 | static MA_INLINE HRESULT ma_IAudioClient3_Initialize(ma_IAudioClient3* pThis, MA_AUDCLNT_SHAREMODE
> shareMode, DWORD streamFlags, MA_REFERENCE_TIME bufferDuration, MA_REFERENCE_TIME periodicity, const
> MA_WAVEFORMATEX* pFormat, const GUID* pAudioSessionGuid) { return pThis->lpVtbl->Initialize(pThis,
> shareMode, streamFlags, bufferDuration, periodicity, pFormat, pAudioSessionGuid); }
22163 | static MA_INLINE HRESULT ma_IAudioClient3_GetBufferSize(ma_IAudioClient3* pThis, ma_uint32*
> pNumBufferFrames) { return pThis->lpVtbl->GetBufferSize(pThis, pNumBufferFrames); }
22164 | static MA_INLINE HRESULT ma_IAudioClient3_GetStreamLatency(ma_IAudioClient3* pThis,
> MA_REFERENCE_TIME* pLatency) { return pThis->lpVtbl->GetStreamLatency(pThis, pLatency); }
> }
22165 | static MA_INLINE HRESULT ma_IAudioClient3_GetCurrentPadding(ma_IAudioClient3* pThis, ma_uint32*
> pNumPaddingFrames) { return pThis->lpVtbl->GetCurrentPadding(pThis, pNumPaddingFrames); }
22166 | static MA_INLINE HRESULT ma_IAudioClient3_IsFormatSupported(ma_IAudioClient3* pThis,
> MA_AUDCLNT_SHAREMODE shareMode, const MA_WAVEFORMATEX* pFormat, MA_WAVEFORMATEX** ppClosestMatch) {
> return pThis->lpVtbl->IsFormatSupported(pThis, shareMode, pFormat, ppClosestMatch); }
22167 | static MA_INLINE HRESULT ma_IAudioClient3_GetMixFormat(ma_IAudioClient3* pThis, MA_WAVEFORMATEX**
> ppDeviceFormat) { return pThis->lpVtbl->GetMixFormat(pThis, ppDeviceFormat); }
22168 | static MA_INLINE HRESULT ma_IAudioClient3_GetDevicePeriod(ma_IAudioClient3* pThis,
> MA_REFERENCE_TIME* pDefaultDevicePeriod, MA_REFERENCE_TIME* pMinimumDevicePeriod) { return
> pThis->lpVtbl->GetDevicePeriod(pThis, pDefaultDevicePeriod, pMinimumDevicePeriod); }
22169 | static MA_INLINE HRESULT ma_IAudioClient3_Start(ma_IAudioClient3* pThis)
> { return pThis->lpVtbl->Start(pThis); }
22170 | static MA_INLINE HRESULT ma_IAudioClient3_Stop(ma_IAudioClient3* pThis)
> { return pThis->lpVtbl->Stop(pThis); }
22171 | static MA_INLINE HRESULT ma_IAudioClient3_Reset(ma_IAudioClient3* pThis)
> { return pThis->lpVtbl->Reset(pThis); }
22172 | static MA_INLINE HRESULT ma_IAudioClient3_SetEventHandle(ma_IAudioClient3* pThis, HANDLE
> eventHandle) { return pThis->lpVtbl->SetEventHandle(pThis, eventHandle); }
22173 | static MA_INLINE HRESULT ma_IAudioClient3_GetService(ma_IAudioClient3* pThis, const IID* const riid,

```



```

> void** pp) { return pThis->lpVtbl->GetService(pThis, riid, pp); }
22174 | static MA_INLINE HRESULT ma_IAudioClient3_IsOffloadCapable(ma_IAudioClient3* pThis,
> MA_AUDIO_STREAM_CATEGORY category, BOOL* pOffloadCapable) { return
> pThis->lpVtbl->IsOffloadCapable(pThis, category, pOffloadCapable); }
22175 | static MA_INLINE HRESULT ma_IAudioClient3_SetClientProperties(ma_IAudioClient3* pThis, const
> ma_AudioClientProperties* pProperties) { return pThis->lpVtbl->SetClientProperties(pThis,
> pProperties); }
22176 | static MA_INLINE HRESULT ma_IAudioClient3_GetBufferSizeLimits(ma_IAudioClient3* pThis, const
> MA_WAVEFORMATEX* pFormat, BOOL eventDriven, MA_REFERENCE_TIME* pMinBufferDuration,
> MA_REFERENCE_TIME* pMaxBufferDuration) { return pThis->lpVtbl->GetBufferSizeLimits(pThis, pFormat,
> eventDriven, pMinBufferDuration, pMaxBufferDuration); }
22177 | static MA_INLINE HRESULT ma_IAudioClient3_GetSharedModeEnginePeriod(ma_IAudioClient3* pThis, const
> MA_WAVEFORMATEX* pFormat, ma_uint32* pDefaultPeriodInFrames, ma_uint32* pFundamentalPeriodInFrames,
> ma_uint32* pMinPeriodInFrames, ma_uint32* pMaxPeriodInFrames) { return
> pThis->lpVtbl->GetSharedModeEnginePeriod(pThis, pFormat, pDefaultPeriodInFrames,
> pFundamentalPeriodInFrames, pMinPeriodInFrames, pMaxPeriodInFrames); }
22178 | static MA_INLINE HRESULT ma_IAudioClient3_GetCurrentSharedModeEnginePeriod(ma_IAudioClient3* pThis,
> MA_WAVEFORMATEX** ppFormat, ma_uint32* pCurrentPeriodInFrames) { return
> pThis->lpVtbl->GetCurrentSharedModeEnginePeriod(pThis, ppFormat, pCurrentPeriodInFrames); }
22179 | static MA_INLINE HRESULT ma_IAudioClient3_InitializeSharedAudioStream(ma_IAudioClient3* pThis, DWORD
> streamFlags, ma_uint32 periodInFrames, const MA_WAVEFORMATEX* pFormat, const GUID*
> pAudioSessionGUID) { return pThis->lpVtbl->InitializeSharedAudioStream(pThis, streamFlags,
> periodInFrames, pFormat, pAudioSessionGUID); }
22180 |
22181 |
22182 | /* IAudioRenderClient */
22183 | typedef struct
22184 | {
22185 |     /* IUnknown */
22186 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IAudioRenderClient* pThis, const IID* const
> riid, void** ppObject);
22187 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IAudioRenderClient* pThis);
22188 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IAudioRenderClient* pThis);
22189 |
22190 |     /* IAudioRenderClient */
22191 |     HRESULT (STDMETHODCALLTYPE * GetBuffer) (ma_IAudioRenderClient* pThis, ma_uint32
> numFramesRequested, BYTE** ppData);
22192 |     HRESULT (STDMETHODCALLTYPE * ReleaseBuffer)(ma_IAudioRenderClient* pThis, ma_uint32
> numFramesWritten, DWORD dwFlags);
22193 | } ma_IAudioRenderClientVtbl;
22194 | struct ma_IAudioRenderClient
22195 | {
22196 |     ma_IAudioRenderClientVtbl* lpVtbl;
22197 | };
22198 | static MA_INLINE HRESULT ma_IAudioRenderClient_QueryInterface(ma_IAudioRenderClient* pThis, const
> IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
22199 | static MA_INLINE ULONG ma_IAudioRenderClient_AddRef(ma_IAudioRenderClient* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
22200 | static MA_INLINE ULONG ma_IAudioRenderClient_Release(ma_IAudioRenderClient* pThis)
> { return pThis->lpVtbl->Release(pThis); }
22201 | static MA_INLINE HRESULT ma_IAudioRenderClient_GetBuffer(ma_IAudioRenderClient* pThis, ma_uint32
> numFramesRequested, BYTE** ppData) { return pThis->lpVtbl->GetBuffer(pThis, numFramesRequested,
> ppData); }
22202 | static MA_INLINE HRESULT ma_IAudioRenderClient_ReleaseBuffer(ma_IAudioRenderClient* pThis, ma_uint32
> numFramesWritten, DWORD dwFlags) { return pThis->lpVtbl->ReleaseBuffer(pThis, numFramesWritten,
> dwFlags); }
22203 |
22204 |
22205 | /* IAudioCaptureClient */
22206 | typedef struct
22207 | {
22208 |     /* IUnknown */
22209 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IAudioCaptureClient* pThis, const IID* const
> riid, void** ppObject);
22210 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IAudioCaptureClient* pThis);
22211 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IAudioCaptureClient* pThis);
22212 |
22213 |     /* IAudioRenderClient */
22214 |     HRESULT (STDMETHODCALLTYPE * GetBuffer) (ma_IAudioCaptureClient* pThis, BYTE** ppData,
> ma_uint32* pNumFramesToRead, DWORD* pFlags, ma_uint64* pDevicePosition, ma_uint64* pQPCPosition);
22215 |     HRESULT (STDMETHODCALLTYPE * ReleaseBuffer) (ma_IAudioCaptureClient* pThis, ma_uint32
> numFramesRead);
22216 |     HRESULT (STDMETHODCALLTYPE * GetNextPacketSize)(ma_IAudioCaptureClient* pThis, ma_uint32*
> pNumFramesInNextPacket);
22217 | } ma_IAudioCaptureClientVtbl;
22218 | struct ma_IAudioCaptureClient
22219 | {
22220 |     ma_IAudioCaptureClientVtbl* lpVtbl;
22221 | };

```

```

22222 | static MA_INLINE HRESULT ma_IAudioCaptureClient_QueryInterface(ma_IAudioCaptureClient* pThis, const
> IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
22223 | static MA_INLINE ULONG ma_IAudioCaptureClient_AddRef(ma_IAudioCaptureClient* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
22224 | static MA_INLINE ULONG ma_IAudioCaptureClient_Release(ma_IAudioCaptureClient* pThis)
> { return pThis->lpVtbl->Release(pThis); }
22225 | static MA_INLINE HRESULT ma_IAudioCaptureClient_GetBuffer(ma_IAudioCaptureClient* pThis, BYTE**
> ppData, ma_uint32* pNumFramesToRead, DWORD* pFlags, ma_uint64* pDevicePosition, ma_uint64*
> pQPCPosition) { return pThis->lpVtbl->GetBuffer(pThis, ppData, pNumFramesToRead, pFlags,
> pDevicePosition, pQPCPosition); }
22226 | static MA_INLINE HRESULT ma_IAudioCaptureClient_ReleaseBuffer(ma_IAudioCaptureClient* pThis,
> ma_uint32 numFramesRead) { return pThis->lpVtbl->ReleaseBuffer(pThis,
> numFramesRead); }
22227 | static MA_INLINE HRESULT ma_IAudioCaptureClient_GetNextPacketSize(ma_IAudioCaptureClient* pThis,
> ma_uint32* pNumFramesInNextPacket) { return pThis->lpVtbl->GetNextPacketSize(pThis,
> pNumFramesInNextPacket); }
22228 |
22229 | #if defined(MA_WIN32_UWP)
22230 | /* mmdevapi Functions */
22231 | typedef HRESULT (WINAPI * MA_PFN_ActivateAudioInterfaceAsync)(const wchar_t* deviceInterfacePath,
> const IID* riid, MA_PROPVARIANT* activationParams, ma_IActivateAudioInterfaceCompletionHandler*
> completionHandler, ma_IActivateAudioInterfaceAsyncOperation** activationOperation);
22232 | #endif
22233 |
22234 | /* Avrt Functions */
22235 | typedef HANDLE (WINAPI * MA_PFN_AvSetMmThreadCharacteristicsA)(const char* TaskName, DWORD*
> TaskIndex);
22236 | typedef BOOL (WINAPI * MA_PFN_AvRevertMmThreadCharacteristics)(HANDLE AvrtHandle);
22237 |
22238 | #if !defined(MA_WIN32_DESKTOP) && !defined(MA_WIN32_GDK)
22239 | typedef struct ma_completion_handler_uwp ma_completion_handler_uwp;
22240 |
22241 | typedef struct
22242 | {
22243 |     /* IUnknown */
22244 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_completion_handler_uwp* pThis, const IID* const
> riid, void** ppObject);
22245 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_completion_handler_uwp* pThis);
22246 |     ULONG (STDMETHODCALLTYPE * Release) (ma_completion_handler_uwp* pThis);
22247 |
22248 |     /* IActivateAudioInterfaceCompletionHandler */
22249 |     HRESULT (STDMETHODCALLTYPE * ActivateCompleted)(ma_completion_handler_uwp* pThis,
> ma_IActivateAudioInterfaceAsyncOperation* pActivateOperation);
22250 | } ma_completion_handler_uwp_vtbl;
22251 | struct ma_completion_handler_uwp
22252 | {
22253 |     ma_completion_handler_uwp_vtbl* lpVtbl;
22254 |     MA_ATOMIC(4, ma_uint32) counter;
22255 |     HANDLE hEvent;
22256 | };
22257 |
22258 | static HRESULT STDMETHODCALLTYPE ma_completion_handler_uwp_QueryInterface(ma_completion_handler_uwp*
> pThis, const IID* const riid, void** ppObject)
22259 | {
22260 |     /*
22261 |     We need to "implement" IAgileObject which is just an indicator that's used internally by WASAPI
> for some multithreading management. To
22262 |     "implement" this, we just make sure we return pThis when the IAgileObject is requested.
22263 |     */
22264 |     if (!ma_is_guid_equal(riid, &MA_IID_IUnknown) && !ma_is_guid_equal(riid,
> &MA_IID_IActivateAudioInterfaceCompletionHandler) && !ma_is_guid_equal(riid, &MA_IID_IAgileObject))
> {
22265 |         *ppObject = NULL;
22266 |         return E_NOINTERFACE;
22267 |     }
22268 |
22269 |     /* Getting here means the IID is IUnknown or IMMNotificationClient. */
22270 |     *ppObject = (void*)pThis;
22271 |     ((ma_completion_handler_uwp_vtbl*)pThis->lpVtbl->AddRef(pThis);
22272 |     return S_OK;
22273 | }
22274 |
22275 | static ULONG STDMETHODCALLTYPE ma_completion_handler_uwp_AddRef(ma_completion_handler_uwp* pThis)
22276 | {
22277 |     return (ULONG)ma_atomic_fetch_add_32(&pThis->counter, 1) + 1;
22278 | }
22279 |
22280 | static ULONG STDMETHODCALLTYPE ma_completion_handler_uwp_Release(ma_completion_handler_uwp* pThis)
22281 | {

```



```

22282 |     ma_uint32 newRefCount = ma_atomic_fetch_sub_32(&pThis->counter, 1) - 1;
22283 |     if (newRefCount == 0) {
22284 |         return 0; /* We don't free anything here because we never allocate the object on the heap.
> */
22285 |     }
22286 |
22287 |     return (ULONG)newRefCount;
22288 | }
22289 |
22290 | static HRESULT STDMETHODCALLTYPE
> ma_completion_handler_uwp_ActivateCompleted(ma_completion_handler_uwp* pThis,
> ma_IActivateAudioInterfaceAsyncOperation* pActivateOperation)
22291 | {
22292 |     (void)pActivateOperation;
22293 |     SetEvent(pThis->hEvent);
22294 |     return S_OK;
22295 | }
22296 |
22297 |
22298 | static ma_completion_handler_uwp_vtbl g_maCompletionHandlerVtblInstance = {
22299 |     ma_completion_handler_uwp_QueryInterface,
22300 |     ma_completion_handler_uwp_AddRef,
22301 |     ma_completion_handler_uwp_Release,
22302 |     ma_completion_handler_uwp_ActivateCompleted
22303 | };
22304 |
22305 | static ma_result ma_completion_handler_uwp_init(ma_completion_handler_uwp* pHandler)
22306 | {
22307 |     MA_ASSERT(pHandler != NULL);
22308 |     MA_ZERO_OBJECT(pHandler);
22309 |
22310 |     pHandler->lpVtbl = &g_maCompletionHandlerVtblInstance;
22311 |     pHandler->counter = 1;
22312 |     pHandler->hEvent = CreateEventA(NULL, FALSE, FALSE, NULL);
22313 |     if (pHandler->hEvent == NULL) {
22314 |         return ma_result_from_GetLastError(GetLastError());
22315 |     }
22316 |
22317 |     return MA_SUCCESS;
22318 | }
22319 |
22320 | static void ma_completion_handler_uwp_uninit(ma_completion_handler_uwp* pHandler)
22321 | {
22322 |     if (pHandler->hEvent != NULL) {
22323 |         CloseHandle(pHandler->hEvent);
22324 |     }
22325 | }
22326 |
22327 | static void ma_completion_handler_uwp_wait(ma_completion_handler_uwp* pHandler)
22328 | {
22329 |     WaitForSingleObject((HANDLE)pHandler->hEvent, INFINITE);
22330 | }
22331 | #endif /* !MA_WIN32_DESKTOP */
22332 |
22333 | /* We need a virtual table for our notification client object that's used for detecting changes to
> the default device. */
22334 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
22335 | static HRESULT STDMETHODCALLTYPE ma_IMMNotificationClient_QueryInterface(ma_IMMNotificationClient*
> pThis, const IID* const riid, void** ppObject)
22336 | {
22337 |     /*
22338 |     We care about two interfaces - IUnknown and IMMNotificationClient. If the requested IID is
> something else
22339 |     we just return E_NOINTERFACE. Otherwise we need to increment the reference counter and return
> S_OK.
22340 |     */
22341 |     if (!ma_is_guid_equal(riid, &MA_IID_IUnknown) && !ma_is_guid_equal(riid,
> &MA_IID_IMMNotificationClient)) {
22342 |         *ppObject = NULL;
22343 |         return E_NOINTERFACE;
22344 |     }
22345 |
22346 |     /* Getting here means the IID is IUnknown or IMMNotificationClient. */
22347 |     *ppObject = (void*)pThis;
22348 |     ((ma_IMMNotificationClientVtbl*)pThis->lpVtbl)->AddRef(pThis);
22349 |     return S_OK;
22350 | }
22351 |
22352 | static ULONG STDMETHODCALLTYPE ma_IMMNotificationClient_AddRef(ma_IMMNotificationClient* pThis)

```

```

22353 | {
22354 |     return (ULONG)ma_atomic_fetch_add_32(&pThis->counter, 1) + 1;
22355 | }
22356 |
22357 | static ULONG STDMETHODCALLTYPE ma_IMMNotificationClient_Release(ma_IMMNotificationClient* pThis)
22358 | {
22359 |     ma_uint32 newRefCount = ma_atomic_fetch_sub_32(&pThis->counter, 1) - 1;
22360 |     if (newRefCount == 0) {
22361 |         return 0; /* We don't free anything here because we never allocate the object on the heap.
> */
22362 |     }
22363 |
22364 |     return (ULONG)newRefCount;
22365 | }
22366 |
22367 | static HRESULT STDMETHODCALLTYPE
> ma_IMMNotificationClient_OnDeviceStateChanged(ma_IMMNotificationClient* pThis, const WCHAR*
> pDeviceID, DWORD dwNewState)
22368 | {
22369 |     ma_bool32 isThisDevice = MA_FALSE;
22370 |     ma_bool32 isCapture     = MA_FALSE;
22371 |     ma_bool32 isPlayback    = MA_FALSE;
22372 |
22373 | #ifdef MA_DEBUG_OUTPUT
22374 |     /*ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG,
> "IMMNotificationClient_OnDeviceStateChanged(pDeviceID=%S, dwNewState=%u)\n", (pDeviceID != NULL) ?
> pDeviceID : L"(NULL)", (unsigned int)dwNewState);*/
22375 | #endif
22376 |
22377 |     /*
22378 |     There have been reports of a hang when a playback device is disconnected. The idea with this
> code is to explicitly stop the device if we detect
22379 |     that the device is disabled or has been unplugged.
22380 |     */
22381 |     if (pThis->pDevice->wasapi.allowCaptureAutoStreamRouting && (pThis->pDevice->type ==
> ma_device_type_capture || pThis->pDevice->type == ma_device_type_duplex || pThis->pDevice->type ==
> ma_device_type_loopback)) {
22382 |         isCapture = MA_TRUE;
22383 |         if (ma_strcmp_WCHAR(pThis->pDevice->capture.id.wasapi, pDeviceID) == 0) {
22384 |             isThisDevice = MA_TRUE;
22385 |         }
22386 |     }
22387 |
22388 |     if (pThis->pDevice->wasapi.allowPlaybackAutoStreamRouting && (pThis->pDevice->type ==
> ma_device_type_playback || pThis->pDevice->type == ma_device_type_duplex)) {
22389 |         isPlayback = MA_TRUE;
22390 |         if (ma_strcmp_WCHAR(pThis->pDevice->playback.id.wasapi, pDeviceID) == 0) {
22391 |             isThisDevice = MA_TRUE;
22392 |         }
22393 |     }
22394 |
22395 |
22396 |     /*
22397 |     If the device ID matches our device we need to mark our device as detached and stop it. When a
22398 |     device is added in OnDeviceAdded(), we'll restart it. We only mark it as detached if the device
22399 |     was started at the time of being removed.
22400 |     */
22401 |     if (isThisDevice) {
22402 |         if ((dwNewState & MA_MM_DEVICE_STATE_ACTIVE) == 0) {
22403 |             /*
22404 |             Unplugged or otherwise unavailable. Mark as detached if we were in a playing state.
> We'll
22405 |             use this to determine whether or not we need to automatically start the device when it's
22406 |             plugged back in again.
22407 |             */
22408 |             if (ma_device_get_state(pThis->pDevice) == ma_device_state_started) {
22409 |                 if (isPlayback) {
22410 |                     pThis->pDevice->wasapi.isDetachedPlayback = MA_TRUE;
22411 |                 }
22412 |                 if (isCapture) {
22413 |                     pThis->pDevice->wasapi.isDetachedCapture = MA_TRUE;
22414 |                 }
22415 |
22416 |                 ma_device_stop(pThis->pDevice);
22417 |             }
22418 |         }
22419 |
22420 |         if ((dwNewState & MA_MM_DEVICE_STATE_ACTIVE) != 0) {
22421 |             /* The device was activated. If we were detached, we need to start it again. */

```

```

22422 |         ma_bool8 tryRestartingDevice = MA_FALSE;
22423 |
22424 |         if (isPlayback) {
22425 |             if (pThis->pDevice->wasapi.isDetachedPlayback) {
22426 |                 pThis->pDevice->wasapi.isDetachedPlayback = MA_FALSE;
22427 |                 ma_device_reroute_wasapi(pThis->pDevice, ma_device_type_playback);
22428 |                 tryRestartingDevice = MA_TRUE;
22429 |             }
22430 |         }
22431 |
22432 |         if (isCapture) {
22433 |             if (pThis->pDevice->wasapi.isDetachedCapture) {
22434 |                 pThis->pDevice->wasapi.isDetachedCapture = MA_FALSE;
22435 |                 ma_device_reroute_wasapi(pThis->pDevice, (pThis->pDevice->type ==
> ma_device_type_loopback) ? ma_device_type_loopback : ma_device_type_capture);
22436 |                 tryRestartingDevice = MA_TRUE;
22437 |             }
22438 |         }
22439 |
22440 |         if (tryRestartingDevice) {
22441 |             if (pThis->pDevice->wasapi.isDetachedPlayback == MA_FALSE &&
> pThis->pDevice->wasapi.isDetachedCapture == MA_FALSE) {
22442 |                 ma_device_start(pThis->pDevice);
22443 |             }
22444 |         }
22445 |     }
22446 | }
22447 |
22448 |     return S_OK;
22449 | }
22450 |
22451 | static HRESULT STDMETHODCALLTYPE ma_IMMNotificationClient_OnDeviceAdded(ma_IMMNotificationClient*
> pThis, const WCHAR* pDeviceID)
22452 | {
22453 |     #ifdef MA_DEBUG_OUTPUT
22454 |         /*ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG,
> "IMMNotificationClient_OnDeviceAdded(pDeviceID=%S)\n", (pDeviceID != NULL) ? pDeviceID :
> L"(NULL)");*/
22455 |     #endif
22456 |
22457 |     /* We don't need to worry about this event for our purposes. */
22458 |     (void)pThis;
22459 |     (void)pDeviceID;
22460 |     return S_OK;
22461 | }
22462 |
22463 | static HRESULT STDMETHODCALLTYPE ma_IMMNotificationClient_OnDeviceRemoved(ma_IMMNotificationClient*
> pThis, const WCHAR* pDeviceID)
22464 | {
22465 |     #ifdef MA_DEBUG_OUTPUT
22466 |         /*ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG,
> "IMMNotificationClient_OnDeviceRemoved(pDeviceID=%S)\n", (pDeviceID != NULL) ? pDeviceID :
> L"(NULL)");*/
22467 |     #endif
22468 |
22469 |     /* We don't need to worry about this event for our purposes. */
22470 |     (void)pThis;
22471 |     (void)pDeviceID;
22472 |     return S_OK;
22473 | }
22474 |
22475 | static HRESULT STDMETHODCALLTYPE
> ma_IMMNotificationClient_OnDefaultDeviceChanged(ma_IMMNotificationClient* pThis, ma_EDataFlow
> dataFlow, ma_ERole role, const WCHAR* pDefaultDeviceID)
22476 | {
22477 |     #ifdef MA_DEBUG_OUTPUT
22478 |         /*ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG,
> "IMMNotificationClient_OnDefaultDeviceChanged(dataFlow=%d, role=%d, pDefaultDeviceID=%S)\n",
> dataFlow, role, (pDefaultDeviceID != NULL) ? pDefaultDeviceID : L"(NULL)");*/
22479 |     #endif
22480 |
22481 |     (void)role;
22482 |
22483 |     /* We only care about devices with the same data flow as the current device. */
22484 |     if ((pThis->pDevice->type == ma_device_type_playback && dataFlow != ma_eRender) ||
22485 |         (pThis->pDevice->type == ma_device_type_capture && dataFlow != ma_eCapture) ||
22486 |         (pThis->pDevice->type == ma_device_type_loopback && dataFlow != ma_eRender)) {
22487 |         ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG, "[WASAPI] Stream
> rerouting abandoned because dataFlow does match device type.\n");

```

```

22488 |         return S_OK;
22489 |     }
22490 |
22491 |     /* We need to consider dataFlow as ma_eCapture if device is ma_device_type_loopback */
22492 |     if (pThis->pDevice->type == ma_device_type_loopback) {
22493 |         dataFlow = ma_eCapture;
22494 |     }
22495 |
22496 |     /* Don't do automatic stream routing if we're not allowed. */
22497 |     if ((dataFlow == ma_eRender && pThis->pDevice->wasapi.allowPlaybackAutoStreamRouting ==
> MA_FALSE) ||
22498 |         (dataFlow == ma_eCapture && pThis->pDevice->wasapi.allowCaptureAutoStreamRouting ==
> MA_FALSE)) {
22499 |         ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG, "[WASAPI] Stream
> rerouting abandoned because automatic stream routing has been disabled by the device config.\n");
22500 |         return S_OK;
22501 |     }
22502 |
22503 |     /*
22504 |     Not currently supporting automatic stream routing in exclusive mode. This is not working
> correctly on my machine due to
22505 |     AUDCLNT_E_DEVICE_IN_USE errors when reinitializing the device. If this is a bug in miniaudio, we
> can try re-enabling this once
22506 |     it's fixed.
22507 |     */
22508 |     if ((dataFlow == ma_eRender && pThis->pDevice->playback.shareMode == ma_share_mode_exclusive)
> ||
22509 |         (dataFlow == ma_eCapture && pThis->pDevice->capture.shareMode == ma_share_mode_exclusive))
> {
22510 |         ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG, "[WASAPI] Stream
> rerouting abandoned because the device shared mode is exclusive.\n");
22511 |         return S_OK;
22512 |     }
22513 |
22514 |
22515 |
22516 |     /*
22517 |     Second attempt at device rerouting. We're going to retrieve the device's state at the time of
22518 |     the route change. We're then going to stop the device, reinitialize the device, and then start
22519 |     it again if the state before stopping was ma_device_state_started.
22520 |     */
22521 |     {
22522 |         ma_uint32 previousState = ma_device_get_state(pThis->pDevice);
22523 |         ma_bool8 restartDevice = MA_FALSE;
22524 |
22525 |         if (previousState == ma_device_state_uninitialized || previousState ==
> ma_device_state_starting) {
22526 |             ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG, "[WASAPI] Stream
> rerouting abandoned because the device is in the process of starting.\n");
22527 |             return S_OK;
22528 |         }
22529 |
22530 |         if (previousState == ma_device_state_started) {
22531 |             ma_device_stop(pThis->pDevice);
22532 |             restartDevice = MA_TRUE;
22533 |         }
22534 |
22535 |         if (pDefaultDeviceID != NULL) { /* <-- The input device ID will be null if there's no other
> device available. */
22536 |             ma_mutex_lock(&pThis->pDevice->wasapi.rerouteLock);
22537 |             {
22538 |                 if (dataFlow == ma_eRender) {
22539 |                     ma_device_reroute__wasapi(pThis->pDevice, ma_device_type_playback);
22540 |
22541 |                     if (pThis->pDevice->wasapi.isDetachedPlayback) {
22542 |                         pThis->pDevice->wasapi.isDetachedPlayback = MA_FALSE;
22543 |
22544 |                         if (pThis->pDevice->type == ma_device_type_duplex &&
> pThis->pDevice->wasapi.isDetachedCapture) {
22545 |                             restartDevice = MA_FALSE; /* It's a duplex device and the capture side
> is detached. We cannot be restarting the device just yet. */
22546 |                         }
22547 |                         else {
22548 |                             restartDevice = MA_TRUE; /* It's not a duplex device, or the capture
> side is also attached so we can go ahead and restart the device. */
22549 |                         }
22550 |                     }
22551 |                 }
22552 |             }

```

```

22553 |         ma_device_reroute__wasapi(pThis->pDevice, (pThis->pDevice->type ==
> ma_device_type_loopback) ? ma_device_type_loopback : ma_device_type_capture);
22554 |
22555 |         if (pThis->pDevice->wasapi.isDetachedCapture) {
22556 |             pThis->pDevice->wasapi.isDetachedCapture = MA_FALSE;
22557 |
22558 |             if (pThis->pDevice->type == ma_device_type_duplex &&
> pThis->pDevice->wasapi.isDetachedPlayback) {
22559 |                 restartDevice = MA_FALSE; /* It's a duplex device and the playback
> side is detached. We cannot be restarting the device just yet. */
22560 |             }
22561 |             else {
22562 |                 restartDevice = MA_TRUE; /* It's not a duplex device, or the playback
> side is also attached so we can go ahead and restart the device. */
22563 |             }
22564 |         }
22565 |     }
22566 | }
22567 | ma_mutex_unlock(&pThis->pDevice->wasapi.rerouteLock);
22568 |
22569 | if (restartDevice) {
22570 |     ma_device_start(pThis->pDevice);
22571 | }
22572 | }
22573 | }
22574 |
22575 | return S_OK;
22576 | }
22577 |
22578 | static HRESULT STDMETHODCALLTYPE
> ma_IMMNotificationClient_OnPropertyValueChanged(ma_IMMNotificationClient* pThis, const WCHAR*
> pDeviceID, const PROPERTYKEY key)
22579 | {
22580 | #ifdef MA_DEBUG_OUTPUT
22581 |     /*ma_log_postf(ma_device_get_log(pThis->pDevice), MA_LOG_LEVEL_DEBUG,
> "IMMNotificationClient_OnPropertyValueChanged(pDeviceID=%S)\n", (pDeviceID != NULL) ? pDeviceID :
> L"(NULL)");*/
22582 | #endif
22583 |
22584 |     (void)pThis;
22585 |     (void)pDeviceID;
22586 |     (void)key;
22587 |     return S_OK;
22588 | }
22589 |
22590 | static ma_IMMNotificationClientVtbl g_maNotificationClientVtbl = {
22591 |     ma_IMMNotificationClient_QueryInterface,
22592 |     ma_IMMNotificationClient_AddRef,
22593 |     ma_IMMNotificationClient_Release,
22594 |     ma_IMMNotificationClient_OnDeviceStateChanged,
22595 |     ma_IMMNotificationClient_OnDeviceAdded,
22596 |     ma_IMMNotificationClient_OnDeviceRemoved,
22597 |     ma_IMMNotificationClient_OnDefaultDeviceChanged,
22598 |     ma_IMMNotificationClient_OnPropertyValueChanged
22599 | };
22600 | #endif /* MA_WIN32_DESKTOP */
22601 |
22602 | static const char* ma_to_usage_string__wasapi(ma_wasapi_usage usage)
22603 | {
22604 |     switch (usage)
22605 |     {
22606 |         case ma_wasapi_usage_default: return NULL;
22607 |         case ma_wasapi_usage_games: return "Games";
22608 |         case ma_wasapi_usage_pro_audio: return "Pro Audio";
22609 |         default: break;
22610 |     }
22611 |
22612 |     return NULL;
22613 | }
22614 |
22615 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
22616 | typedef ma_IMMDevice ma_WASAPIDeviceInterface;
22617 | #else
22618 | typedef ma_IUnknown ma_WASAPIDeviceInterface;
22619 | #endif
22620 |
22621 |
22622 | #define MA_CONTEXT_COMMAND_QUIT__WASAPI 1
22623 | #define MA_CONTEXT_COMMAND_CREATE_IAUDIOCLIENT__WASAPI 2

```

```

22624 | #define MA_CONTEXT_COMMAND_RELEASE_IAUDIOCLIENT__WASAPI 3
22625 |
22626 | static ma_context_command_wasapi ma_context_init_command_wasapi(int code)
22627 | {
22628 |     ma_context_command_wasapi cmd;
22629 |
22630 |     MA_ZERO_OBJECT(&cmd);
22631 |     cmd.code = code;
22632 |
22633 |     return cmd;
22634 | }
22635 |
22636 | static ma_result ma_context_post_command_wasapi(ma_context* pContext, const
> ma_context_command_wasapi* pCmd)
22637 | {
22638 |     /* For now we are doing everything synchronously, but I might relax this later if the need
> arises. */
22639 |     ma_result result;
22640 |     ma_bool32 isUsingLocalEvent = MA_FALSE;
22641 |     ma_event localEvent;
22642 |
22643 |     MA_ASSERT(pContext != NULL);
22644 |     MA_ASSERT(pCmd != NULL);
22645 |
22646 |     if (pCmd->pEvent == NULL) {
22647 |         isUsingLocalEvent = MA_TRUE;
22648 |
22649 |         result = ma_event_init(&localEvent);
22650 |         if (result != MA_SUCCESS) {
22651 |             return result; /* Failed to create the event for this command. */
22652 |         }
22653 |     }
22654 |
22655 |     /* Here is where we add the command to the list. If there's not enough room we'll spin until
> there is. */
22656 |     ma_mutex_lock(&pContext->wasapi.commandLock);
22657 |     {
22658 |         ma_uint32 index;
22659 |
22660 |         /* Spin until we've got some space available. */
22661 |         while (pContext->wasapi.commandCount == ma_countof(pContext->wasapi.commands)) {
22662 |             ma_yield();
22663 |         }
22664 |
22665 |         /* Space is now available. Can safely add to the list. */
22666 |         index = (pContext->wasapi.commandIndex + pContext->wasapi.commandCount) %
> ma_countof(pContext->wasapi.commands);
22667 |         pContext->wasapi.commands[index] = *pCmd;
22668 |         pContext->wasapi.commands[index].pEvent = &localEvent;
22669 |         pContext->wasapi.commandCount += 1;
22670 |
22671 |         /* Now that the command has been added, release the semaphore so
> ma_context_next_command_wasapi() can return. */
22672 |         ma_semaphore_release(&pContext->wasapi.commandSem);
22673 |     }
22674 |     ma_mutex_unlock(&pContext->wasapi.commandLock);
22675 |
22676 |     if (isUsingLocalEvent) {
22677 |         ma_event_wait(&localEvent);
22678 |         ma_event_uninit(&localEvent);
22679 |     }
22680 |
22681 |     return MA_SUCCESS;
22682 | }
22683 |
22684 | static ma_result ma_context_next_command_wasapi(ma_context* pContext, ma_context_command_wasapi*
> pCmd)
22685 | {
22686 |     ma_result result = MA_SUCCESS;
22687 |
22688 |     MA_ASSERT(pContext != NULL);
22689 |     MA_ASSERT(pCmd != NULL);
22690 |
22691 |     result = ma_semaphore_wait(&pContext->wasapi.commandSem);
22692 |     if (result == MA_SUCCESS) {
22693 |         ma_mutex_lock(&pContext->wasapi.commandLock);
22694 |         {
22695 |             *pCmd = pContext->wasapi.commands[pContext->wasapi.commandIndex];
22696 |             pContext->wasapi.commandIndex = (pContext->wasapi.commandIndex + 1) %

```

```

> ma_countof(pContext->wasapi.commands);
22697 |         pContext->wasapi.commandCount -= 1;
22698 |     }
22699 |     ma_mutex_unlock(&pContext->wasapi.commandLock);
22700 | }
22701 |
22702 |     return result;
22703 | }
22704 |
22705 | static ma_thread_result MA_THREADCALL ma_context_command_thread_wasapi(void* pUserData)
22706 | {
22707 |     ma_result result;
22708 |     ma_context* pContext = (ma_context*)pUserData;
22709 |     MA_ASSERT(pContext != NULL);
22710 |
22711 |     for (;;) {
22712 |         ma_context_command_wasapi cmd;
22713 |         result = ma_context_next_command_wasapi(pContext, &cmd);
22714 |         if (result != MA_SUCCESS) {
22715 |             break;
22716 |         }
22717 |
22718 |         switch (cmd.code)
22719 |         {
22720 |             case MA_CONTEXT_COMMAND_QUIT__WASAPI:
22721 |             {
22722 |                 /* Do nothing. Handled after the switch. */
22723 |             } break;
22724 |
22725 |             case MA_CONTEXT_COMMAND_CREATE_IAUDIOCLIENT__WASAPI:
22726 |             {
22727 |                 if (cmd.data.createAudioClient.deviceType == ma_device_type_playback) {
22728 |                     *cmd.data.createAudioClient.pResult = ma_result_from_HRESULT(ma_IAudioClient_Get
> Service((ma_IAudioClient*)cmd.data.createAudioClient.pAudioClient, &MA_IID_IAudioRenderClient,
> cmd.data.createAudioClient.ppAudioClientService));
22729 |                 } else {
22730 |                     *cmd.data.createAudioClient.pResult = ma_result_from_HRESULT(ma_IAudioClient_Get
> Service((ma_IAudioClient*)cmd.data.createAudioClient.pAudioClient, &MA_IID_IAudioCaptureClient,
> cmd.data.createAudioClient.ppAudioClientService));
22731 |                 }
22732 |             } break;
22733 |
22734 |             case MA_CONTEXT_COMMAND_RELEASE_IAUDIOCLIENT__WASAPI:
22735 |             {
22736 |                 if (cmd.data.releaseAudioClient.deviceType == ma_device_type_playback) {
22737 |                     if (cmd.data.releaseAudioClient.pDevice->wasapi.pAudioClientPlayback != NULL) {
22738 |                         ma_IAudioClient_Release((ma_IAudioClient*)cmd.data.releaseAudioClient.pDevic
> e->wasapi.pAudioClientPlayback);
22739 |                         cmd.data.releaseAudioClient.pDevice->wasapi.pAudioClientPlayback = NULL;
22740 |                     }
22741 |                 }
22742 |
22743 |                 if (cmd.data.releaseAudioClient.deviceType == ma_device_type_capture) {
22744 |                     if (cmd.data.releaseAudioClient.pDevice->wasapi.pAudioClientCapture != NULL) {
22745 |                         ma_IAudioClient_Release((ma_IAudioClient*)cmd.data.releaseAudioClient.pDevic
> e->wasapi.pAudioClientCapture);
22746 |                         cmd.data.releaseAudioClient.pDevice->wasapi.pAudioClientCapture = NULL;
22747 |                     }
22748 |                 }
22749 |             } break;
22750 |
22751 |             default:
22752 |             {
22753 |                 /* Unknown command. Ignore it, but trigger an assert in debug mode so we're aware of
> it. */
22754 |                 MA_ASSERT(MA_FALSE);
22755 |             } break;
22756 |         }
22757 |
22758 |         if (cmd.pEvent != NULL) {
22759 |             ma_event_signal(cmd.pEvent);
22760 |         }
22761 |
22762 |         if (cmd.code == MA_CONTEXT_COMMAND_QUIT__WASAPI) {
22763 |             break; /* Received a quit message. Get out of here. */
22764 |         }
22765 |     }
22766 |
22767 |     return (ma_thread_result)0;

```



```

22768 | }
22769 |
22770 | static ma_result ma_device_create_IAudioClient_service_wasapi(ma_context* pContext, ma_device_type
> deviceType, ma_IAudioClient* pAudioClient, void** ppAudioClientService)
22771 | {
22772 |     ma_result result;
22773 |     ma_result cmdResult;
22774 |     ma_context_command_wasapi cmd =
> ma_context_init_command_wasapi(MA_CONTEXT_COMMAND_CREATE_IAUDIOCLIENT__WASAPI);
22775 |     cmd.data.createAudioClient.deviceType      = deviceType;
22776 |     cmd.data.createAudioClient.pAudioClient    = (void*)pAudioClient;
22777 |     cmd.data.createAudioClient.ppAudioClientService = ppAudioClientService;
22778 |     cmd.data.createAudioClient.pResult        = &cmdResult; /* Declared locally, but won't
> be dereferenced after this function returns since execution of the command will wait here. */
22779 |
22780 |     result = ma_context_post_command_wasapi(pContext, &cmd); /* This will not return until the
> command has actually been run. */
22781 |     if (result != MA_SUCCESS) {
22782 |         return result;
22783 |     }
22784 |
22785 |     return *cmd.data.createAudioClient.pResult;
22786 | }
22787 |
22788 | #if 0 /* Not used at the moment, but leaving here for future use. */
22789 | static ma_result ma_device_release_IAudioClient_service_wasapi(ma_device* pDevice, ma_device_type
> deviceType)
22790 | {
22791 |     ma_result result;
22792 |     ma_context_command_wasapi cmd =
> ma_context_init_command_wasapi(MA_CONTEXT_COMMAND_RELEASE_IAUDIOCLIENT__WASAPI);
22793 |     cmd.data.releaseAudioClient.pDevice      = pDevice;
22794 |     cmd.data.releaseAudioClient.deviceType   = deviceType;
22795 |
22796 |     result = ma_context_post_command_wasapi(pDevice->pContext, &cmd); /* This will not return
> until the command has actually been run. */
22797 |     if (result != MA_SUCCESS) {
22798 |         return result;
22799 |     }
22800 |
22801 |     return MA_SUCCESS;
22802 | }
22803 | #endif
22804 |
22805 |
22806 | static void ma_add_native_data_format_to_device_info_from_WAVEFORMATEX(const MA_WAVEFORMATEX* pWF,
> ma_share_mode shareMode, ma_device_info* pInfo)
22807 | {
22808 |     MA_ASSERT(pWF != NULL);
22809 |     MA_ASSERT(pInfo != NULL);
22810 |
22811 |     if (pInfo->nativeDataFormatCount >= ma_countof(pInfo->nativeDataFormats)) {
22812 |         return; /* Too many data formats. Need to ignore this one. Don't think this should ever
> happen with WASAPI. */
22813 |     }
22814 |
22815 |     pInfo->nativeDataFormats[pInfo->nativeDataFormatCount].format      =
> ma_format_from_WAVEFORMATEX(pWF);
22816 |     pInfo->nativeDataFormats[pInfo->nativeDataFormatCount].channels    = pWF->nChannels;
22817 |     pInfo->nativeDataFormats[pInfo->nativeDataFormatCount].sampleRate = pWF->nSamplesPerSec;
22818 |     pInfo->nativeDataFormats[pInfo->nativeDataFormatCount].flags      = (shareMode ==
> ma_share_mode_exclusive) ? MA_DATA_FORMAT_FLAG_EXCLUSIVE_MODE : 0;
22819 |     pInfo->nativeDataFormatCount += 1;
22820 | }
22821 |
22822 | static ma_result ma_context_get_device_info_from_IAudioClient__wasapi(ma_context* pContext,
> /*ma_IMMDevice**/void* pMMDDevice, ma_IAudioClient* pAudioClient, ma_device_info* pInfo)
22823 | {
22824 |     HRESULT hr;
22825 |     MA_WAVEFORMATEX* pWF = NULL;
22826 |
22827 |     MA_ASSERT(pAudioClient != NULL);
22828 |     MA_ASSERT(pInfo != NULL);
22829 |
22830 |     /* Shared Mode. We use GetMixFormat() here. */
22831 |     hr = ma_IAudioClient_GetMixFormat((ma_IAudioClient*)pAudioClient, (MA_WAVEFORMATEX*)&pWF);
22832 |     if (SUCCEEDED(hr)) {
22833 |         ma_add_native_data_format_to_device_info_from_WAVEFORMATEX(pWF, ma_share_mode_shared,
> pInfo);

```

```

22834 |     } else {
22835 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to retrieve
> mix format for device info retrieval.");
22836 |         return ma_result_from_HRESULT(hr);
22837 |     }
22838 |
22839 |     /*
22840 |     Exclusive Mode. We repeatedly call IsFormatSupported() here. This is not currently supported on
22841 |     UWP. Failure to retrieve the exclusive mode format is not considered an error, so from here on
22842 |     out, MA_SUCCESS is guaranteed to be returned.
22843 |     */
22844 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
22845 |     {
22846 |         ma_IPropertyStore *pProperties;
22847 |
22848 |         /*
22849 |         The first thing to do is get the format from PKEY_AudioEngine_DeviceFormat. This should give
> us a channel count we assume is
22850 |         correct which will simplify our searching.
22851 |         */
22852 |         hr = ma_IMMDevice_OpenPropertyStore((ma_IMMDevice*)pMMDevice, STGM_READ, &pProperties);
22853 |         if (SUCCEEDED(hr)) {
22854 |             MA_PROPVARIANT var;
22855 |             ma_PropVariantInit(&var);
22856 |
22857 |             hr = ma_IPropertyStore_GetValue(pProperties, &MA_PKEY_AudioEngine_DeviceFormat, &var);
22858 |             if (SUCCEEDED(hr)) {
22859 |                 pWF = (MA_WAVEFORMATEX*)var.blob.pBlobData;
22860 |
22861 |                 /*
22862 |                 In my testing, the format returned by PKEY_AudioEngine_DeviceFormat is suitable for
> exclusive mode so we check this format
22863 |                 first. If this fails, fall back to a search.
22864 |                 */
22865 |                 hr = ma_IAudioClient_IsFormatSupported((ma_IAudioClient*)pAudioClient,
> MA_AUDCLNT_SHAREMODE_EXCLUSIVE, pWF, NULL);
22866 |                 if (SUCCEEDED(hr)) {
22867 |                     /* The format returned by PKEY_AudioEngine_DeviceFormat is supported. */
22868 |                     ma_add_native_data_format_to_device_info_from_WAVEFORMATEX(pWF,
> ma_share_mode_exclusive, pInfo);
22869 |                 } else {
22870 |                     /*
22871 |                     The format returned by PKEY_AudioEngine_DeviceFormat is not supported, so fall
> back to a search. We assume the channel
22872 |                     count returned by MA_PKEY_AudioEngine_DeviceFormat is valid and correct. For
> simplicity we're only returning one format.
22873 |                     */
22874 |                     ma_uint32 channels = pWF->nChannels;
22875 |                     ma_channel defaultChannelMap[MA_MAX_CHANNELS];
22876 |                     MA_WAVEFORMATEXTENSIBLE wf;
22877 |                     ma_bool32 found;
22878 |                     ma_uint32 iFormat;
22879 |
22880 |                     /* Make sure we don't overflow the channel map. */
22881 |                     if (channels > MA_MAX_CHANNELS) {
22882 |                         channels = MA_MAX_CHANNELS;
22883 |                     }
22884 |
22885 |                     ma_channel_map_init_standard(ma_standard_channel_map_microsoft,
> defaultChannelMap, ma_countof(defaultChannelMap), channels);
22886 |
22887 |                     MA_ZERO_OBJECT(&wf);
22888 |                     wf.cbSize = sizeof(wf);
22889 |                     wf.wFormatTag = WAVE_FORMAT_EXTENSIBLE;
22890 |                     wf.nChannels = (WORD)channels;
22891 |                     wf.dwChannelMask = ma_channel_map_to_channel_mask_win32(defaultChannelMap,
> channels);
22892 |
22893 |                     found = MA_FALSE;
22894 |                     for (iFormat = 0; iFormat < ma_countof(g_maFormatPriorities); ++iFormat) {
22895 |                         ma_format format = g_maFormatPriorities[iFormat];
22896 |                         ma_uint32 iSampleRate;
22897 |
22898 |                         wf.wBitsPerSample = (WORD)(ma_get_bytes_per_sample(format)*8);
22899 |                         wf.nBlockAlign = (WORD)(wf.nChannels * wf.wBitsPerSample / 8);
22900 |                         wf.nAvgBytesPerSec = wf.nBlockAlign * wf.nSamplesPerSec;
22901 |                         wf.Samples.wValidBitsPerSample = /*(format == ma_format_s24_32) ? 24 :*/
> wf.wBitsPerSample;
22902 |                         if (format == ma_format_f32) {

```

```

22903 |             wf.SubFormat = MA_GUID_KSDATAFORMAT_SUBTYPE_IEEE_FLOAT;
22904 |         } else {
22905 |             wf.SubFormat = MA_GUID_KSDATAFORMAT_SUBTYPE_PCM;
22906 |         }
22907 |
22908 |         for (iSampleRate = 0; iSampleRate <
> ma_countof(g_maStandardSampleRatePriorities); ++iSampleRate) {
22909 |             wf.nSamplesPerSec = g_maStandardSampleRatePriorities[iSampleRate];
22910 |
22911 |             hr = ma_IAudioClient_IsFormatSupported((ma_IAudioClient*)pAudioClient,
> MA_AUDCLNT_SHAREMODE_EXCLUSIVE, (MA_WAVEFORMATEX*)&wf, NULL);
22912 |             if (SUCCEEDED(hr)) {
22913 |
> ma_add_native_data_format_to_device_info_from_WAVEFORMATEX((MA_WAVEFORMATEX*)&wf,
> ma_share_mode_exclusive, pInfo);
22914 |                 found = MA_TRUE;
22915 |                 break;
22916 |             }
22917 |         }
22918 |
22919 |         if (found) {
22920 |             break;
22921 |         }
22922 |     }
22923 |
22924 |     ma_PropVariantClear(pContext, &var);
22925 |
22926 |     if (!found) {
22927 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "[WASAPI]
> Failed to find suitable device format for device info retrieval.");
22928 |     }
22929 | }
22930 | } else {
22931 |     ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "[WASAPI] Failed to
> retrieve device format for device info retrieval.");
22932 | }
22933 |
22934 |     ma_IPropertyStore_Release(pProperties);
22935 | } else {
22936 |     ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "[WASAPI] Failed to
> open property store for device info retrieval.");
22937 | }
22938 | }
22939 | #else
22940 | {
22941 |     (void)pMMDDevice; /* Unused. */
22942 | }
22943 | #endif
22944 |
22945 |     return MA_SUCCESS;
22946 | }
22947 |
22948 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
22949 | static ma_EDataFlow ma_device_type_to_EDataFlow(ma_device_type deviceType)
22950 | {
22951 |     if (deviceType == ma_device_type_playback) {
22952 |         return ma_eRender;
22953 |     } else if (deviceType == ma_device_type_capture) {
22954 |         return ma_eCapture;
22955 |     } else {
22956 |         MA_ASSERT(MA_FALSE);
22957 |         return ma_eRender; /* Should never hit this. */
22958 |     }
22959 | }
22960 |
22961 | static ma_result ma_context_create_IMMDeviceEnumerator_wasapi(ma_context* pContext,
> ma_IMMDeviceEnumerator** ppDeviceEnumerator)
22962 | {
22963 |     HRESULT hr;
22964 |     ma_IMMDeviceEnumerator* pDeviceEnumerator;
22965 |
22966 |     MA_ASSERT(pContext != NULL);
22967 |     MA_ASSERT(ppDeviceEnumerator != NULL);
22968 |
22969 |     *ppDeviceEnumerator = NULL; /* Safety. */
22970 |
22971 |     hr = ma_CoCreateInstance(pContext, &MA_CLSID_MMDeviceEnumerator, NULL, CLSCTX_ALL,
> &MA_IID_IMMDeviceEnumerator, (void*)&pDeviceEnumerator);
22972 |     if (FAILED(hr)) {

```

```

22973 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> device enumerator.");
22974 |         return ma_result_from_HRESULT(hr);
22975 |     }
22976 |
22977 |     *ppDeviceEnumerator = pDeviceEnumerator;
22978 |
22979 |     return MA_SUCCESS;
22980 | }
22981 |
22982 | static WCHAR* ma_context_get_default_device_id_from_IMMDeviceEnumerator__wasapi(ma_context*
> pContext, ma_IMMDeviceEnumerator* pDeviceEnumerator, ma_device_type deviceType)
22983 | {
22984 |     HRESULT hr;
22985 |     ma_IMMDevice* pMMDefaultDevice = NULL;
22986 |     WCHAR* pDefaultDeviceID = NULL;
22987 |     ma_EDataFlow dataFlow;
22988 |     ma_ERole role;
22989 |
22990 |     MA_ASSERT(pContext != NULL);
22991 |     MA_ASSERT(pDeviceEnumerator != NULL);
22992 |
22993 |     (void)pContext;
22994 |
22995 |     /* Grab the EDataFlow type from the device type. */
22996 |     dataFlow = ma_device_type_to_EDataFlow(deviceType);
22997 |
22998 |     /* The role is always eConsole, but we may make this configurable later. */
22999 |     role = ma_eConsole;
23000 |
23001 |     hr = ma_IMMDeviceEnumerator_GetDefaultAudioEndpoint(pDeviceEnumerator, dataFlow, role,
> &pMMDefaultDevice);
23002 |     if (FAILED(hr)) {
23003 |         return NULL;
23004 |     }
23005 |
23006 |     hr = ma_IMMDevice_GetId(pMMDefaultDevice, &pDefaultDeviceID);
23007 |
23008 |     ma_IMMDevice_Release(pMMDefaultDevice);
23009 |     pMMDefaultDevice = NULL;
23010 |
23011 |     if (FAILED(hr)) {
23012 |         return NULL;
23013 |     }
23014 |
23015 |     return pDefaultDeviceID;
23016 | }
23017 |
23018 | static WCHAR* ma_context_get_default_device_id_wasapi(ma_context* pContext, ma_device_type
> deviceType) /* Free the returned pointer with ma_CoTaskMemFree() */
23019 | {
23020 |     ma_result result;
23021 |     ma_IMMDeviceEnumerator* pDeviceEnumerator;
23022 |     WCHAR* pDefaultDeviceID = NULL;
23023 |
23024 |     MA_ASSERT(pContext != NULL);
23025 |
23026 |     result = ma_context_create_IMMDeviceEnumerator__wasapi(pContext, &pDeviceEnumerator);
23027 |     if (result != MA_SUCCESS) {
23028 |         return NULL;
23029 |     }
23030 |
23031 |     pDefaultDeviceID = ma_context_get_default_device_id_from_IMMDeviceEnumerator__wasapi(pContext,
> pDeviceEnumerator, deviceType);
23032 |
23033 |     ma_IMMDeviceEnumerator_Release(pDeviceEnumerator);
23034 |     return pDefaultDeviceID;
23035 | }
23036 |
23037 | static ma_result ma_context_get_MMDevice__wasapi(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_IMMDevice** ppMMDevice)
23038 | {
23039 |     ma_IMMDeviceEnumerator* pDeviceEnumerator;
23040 |     HRESULT hr;
23041 |     HRESULT CoInitializeResult;
23042 |
23043 |     MA_ASSERT(pContext != NULL);
23044 |     MA_ASSERT(ppMMDevice != NULL);
23045 |

```

```

23046 |     /*
23047 |     This weird COM init/uninit here is a hack to work around a crash when changing devices. What is
> happening is
23048 |     WASAPI fires a callback from another thread when the device is changed. It's from that thread
> where this
23049 |     function is getting called. What I'm suspecting is that the other thread is not initializing COM
> which in turn
23050 |     results in CoCreateInstance() failing.
23051 |
23052 |     The community has reported that this seems to fix the crash. There are future plans to move all
> WASAPI operation
23053 |     over to a single thread to make everything safer, but in the meantime while we wait for that to
> come online I'm
23054 |     happy enough to use this hack instead.
23055 |
23056 |     CoUninitialize should only be called if we successfully initialized. S_OK and S_FALSE both mean
> that we need to
23057 |     call CoUninitialize since the internal ref count was increased. RPC_E_CHANGED_MODE means that
> CoInitializeEx was
23058 |     called with a different COINIT value, and we don't call CoUninitialize in that case. Other
> errors are possible,
23059 |     so we check for S_OK and S_FALSE specifically.
23060 |     */
23061 |     CoInitializeResult = ma_CoInitializeEx(pContext, NULL, MA_COINIT_VALUE);
23062 |     {
23063 |         hr = ma_CoCreateInstance(pContext, &MA_CLSID_MMDeviceEnumerator, NULL, CLSCTX_ALL,
> &MA_IID_IMMDeviceEnumerator, (void*)&pDeviceEnumerator);
23064 |     }
23065 |     if (CoInitializeResult == S_OK || CoInitializeResult == S_FALSE) { ma_CoUninitialize(pContext);
> }
23066 |
23067 |     if (FAILED(hr)) { /* <-- This is checking the call above to ma_CoCreateInstance(). */
23068 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> IMMDeviceEnumerator.\n");
23069 |         return ma_result_from_HRESULT(hr);
23070 |     }
23071 |
23072 |     if (pDeviceID == NULL) {
23073 |         hr = ma_IMMDeviceEnumerator_GetDefaultAudioEndpoint(pDeviceEnumerator, (deviceType ==
> ma_device_type_capture) ? ma_eCapture : ma_eRender, ma_eConsole, ppMMDDevice);
23074 |     } else {
23075 |         hr = ma_IMMDeviceEnumerator_GetDevice(pDeviceEnumerator, pDeviceID->wasapi, ppMMDDevice);
23076 |     }
23077 |
23078 |     ma_IMMDeviceEnumerator_Release(pDeviceEnumerator);
23079 |     if (FAILED(hr)) {
23080 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to retrieve
> IMMDevice.\n");
23081 |         return ma_result_from_HRESULT(hr);
23082 |     }
23083 |
23084 |     return MA_SUCCESS;
23085 | }
23086 |
23087 | static ma_result ma_context_get_device_id_from_MMDevice__wasapi(ma_context* pContext, ma_IMMDevice*
> pMMDDevice, ma_device_id* pDeviceID)
23088 | {
23089 |     WCHAR* pDeviceIDString;
23090 |     HRESULT hr;
23091 |
23092 |     MA_ASSERT(pDeviceID != NULL);
23093 |
23094 |     hr = ma_IMMDevice_GetId(pMMDDevice, &pDeviceIDString);
23095 |     if (SUCCEEDED(hr)) {
23096 |         size_t idlen = ma_strlen_WCHAR(pDeviceIDString);
23097 |         if (idlen+1 > ma_countof(pDeviceID->wasapi)) {
23098 |             ma_CoTaskMemFree(pContext, pDeviceIDString);
23099 |             MA_ASSERT(MA_FALSE); /* NOTE: If this is triggered, please report it. It means the
> format of the ID must have changed and is too long to fit in our fixed sized buffer. */
23100 |             return MA_ERROR;
23101 |         }
23102 |
23103 |         MA_COPY_MEMORY(pDeviceID->wasapi, pDeviceIDString, idlen * sizeof(wchar_t));
23104 |         pDeviceID->wasapi[idlen] = '\0';
23105 |
23106 |         ma_CoTaskMemFree(pContext, pDeviceIDString);
23107 |
23108 |         return MA_SUCCESS;
23109 |     }

```

```

23110 |
23111 |     return MA_ERROR;
23112 | }
23113 |
23114 | static ma_result ma_context_get_device_info_from_MMDevice_wasapi(ma_context* pContext,
> ma_IMMDevice* pMMDevice, WCHAR* pDefaultDeviceID, ma_bool32 onlySimpleInfo, ma_device_info* pInfo)
23115 | {
23116 |     ma_result result;
23117 |     HRESULT hr;
23118 |
23119 |     MA_ASSERT(pContext != NULL);
23120 |     MA_ASSERT(pMMDevice != NULL);
23121 |     MA_ASSERT(pInfo != NULL);
23122 |
23123 |     /* ID. */
23124 |     result = ma_context_get_device_id_from_MMDevice_wasapi(pContext, pMMDevice, &pInfo->id);
23125 |     if (result == MA_SUCCESS) {
23126 |         if (pDefaultDeviceID != NULL) {
23127 |             if (ma_strcmp_WCHAR(pInfo->id.wasapi, pDefaultDeviceID) == 0) {
23128 |                 pInfo->isDefault = MA_TRUE;
23129 |             }
23130 |         }
23131 |     }
23132 |
23133 |     /* Description / Friendly Name */
23134 |     {
23135 |         ma_IPropertyStore *pProperties;
23136 |         hr = ma_IMMDevice_OpenPropertyStore(pMMDevice, STGM_READ, &pProperties);
23137 |         if (SUCCEEDED(hr)) {
23138 |             MA_PROPVARIANT var;
23139 |
23140 |             ma_PropVariantInit(&var);
23141 |             hr = ma_IPropertyStore_GetValue(pProperties, &MA_PKEY_Device_FriendlyName, &var);
23142 |             if (SUCCEEDED(hr)) {
23143 |                 WideCharToMultiByte(CP_UTF8, 0, var.pwszVal, -1, pInfo->name, sizeof(pInfo->name),
> 0, FALSE);
23144 |                 ma_PropVariantClear(pContext, &var);
23145 |             }
23146 |
23147 |             ma_IPropertyStore_Release(pProperties);
23148 |         }
23149 |     }
23150 |
23151 |     /* Format */
23152 |     if (!onlySimpleInfo) {
23153 |         ma_IAudioClient* pAudioClient;
23154 |         hr = ma_IMMDevice_Activate(pMMDevice, &MA_IID_IAudioClient, CLSCTX_ALL, NULL,
> (void*)&pAudioClient);
23155 |         if (SUCCEEDED(hr)) {
23156 |             result = ma_context_get_device_info_from_IAudioClient_wasapi(pContext, pMMDevice,
> pAudioClient, pInfo);
23157 |
23158 |             ma_IAudioClient_Release(pAudioClient);
23159 |             return result;
23160 |         } else {
23161 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to
> activate audio client for device info retrieval.");
23162 |             return ma_result_from_HRESULT(hr);
23163 |         }
23164 |     }
23165 |
23166 |     return MA_SUCCESS;
23167 | }
23168 |
23169 | static ma_result ma_context_enumerate_devices_by_type_wasapi(ma_context* pContext,
> ma_IMMDeviceEnumerator* pDeviceEnumerator, ma_device_type deviceType, ma_enum_devices_callback_proc
> callback, void* pUserData)
23170 | {
23171 |     ma_result result = MA_SUCCESS;
23172 |     UINT deviceCount;
23173 |     HRESULT hr;
23174 |     ma_uint32 iDevice;
23175 |     WCHAR* pDefaultDeviceID = NULL;
23176 |     ma_IMMDeviceCollection* pDeviceCollection = NULL;
23177 |
23178 |     MA_ASSERT(pContext != NULL);
23179 |     MA_ASSERT(callback != NULL);
23180 |
23181 |     /* Grab the default device. We use this to know whether or not flag the returned device info as

```

```

> being the default. */
23182 |     pDefaultDeviceID = ma_context_get_default_device_id_from_IMMDeviceEnumerator__wasapi(pContext,
> pDeviceEnumerator, deviceType);
23183 |
23184 |     /* We need to enumerate the devices which returns a device collection. */
23185 |     hr = ma_IMMDeviceEnumerator_EnumAudioEndpoints(pDeviceEnumerator,
> ma_device_type_to_EDataFlow(deviceType), MA_MM_DEVICE_STATE_ACTIVE, &pDeviceCollection);
23186 |     if (SUCCEEDED(hr)) {
23187 |         hr = ma_IMMDeviceCollection_GetCount(pDeviceCollection, &deviceCount);
23188 |         if (FAILED(hr)) {
23189 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to get
> device count.\n");
23190 |             result = ma_result_from_HRESULT(hr);
23191 |             goto done;
23192 |         }
23193 |
23194 |         for (iDevice = 0; iDevice < deviceCount; ++iDevice) {
23195 |             ma_device_info deviceInfo;
23196 |             ma_IMMDevice* pMMDDevice;
23197 |
23198 |             MA_ZERO_OBJECT(&deviceInfo);
23199 |
23200 |             hr = ma_IMMDeviceCollection_Item(pDeviceCollection, iDevice, &pMMDDevice);
23201 |             if (SUCCEEDED(hr)) {
23202 |                 result = ma_context_get_device_info_from_MMDevice__wasapi(pContext, pMMDDevice,
> pDefaultDeviceID, MA_TRUE, &deviceInfo); /* MA_TRUE = onlySimpleInfo. */
23203 |
23204 |                 ma_IMMDevice_Release(pMMDDevice);
23205 |                 if (result == MA_SUCCESS) {
23206 |                     ma_bool32 cbResult = callback(pContext, deviceType, &deviceInfo, pUserData);
23207 |                     if (cbResult == MA_FALSE) {
23208 |                         break;
23209 |                     }
23210 |                 }
23211 |             }
23212 |         }
23213 |     }
23214 |
23215 | done:
23216 |     if (pDefaultDeviceID != NULL) {
23217 |         ma_CoTaskMemFree(pContext, pDefaultDeviceID);
23218 |         pDefaultDeviceID = NULL;
23219 |     }
23220 |
23221 |     if (pDeviceCollection != NULL) {
23222 |         ma_IMMDeviceCollection_Release(pDeviceCollection);
23223 |         pDeviceCollection = NULL;
23224 |     }
23225 |
23226 |     return result;
23227 | }
23228 |
23229 | static ma_result ma_context_get_IAudioClient_Desktop__wasapi(ma_context* pContext, ma_device_type
> deviceType, const ma_device_id* pDeviceID, MA_PROPVARIANT* pActivationParams, ma_IAudioClient**
> ppAudioClient, ma_IMMDevice** ppMMDDevice)
23230 | {
23231 |     ma_result result;
23232 |     HRESULT hr;
23233 |
23234 |     MA_ASSERT(pContext != NULL);
23235 |     MA_ASSERT(ppAudioClient != NULL);
23236 |     MA_ASSERT(ppMMDDevice != NULL);
23237 |
23238 |     result = ma_context_get_MMDevice__wasapi(pContext, deviceType, pDeviceID, ppMMDDevice);
23239 |     if (result != MA_SUCCESS) {
23240 |         return result;
23241 |     }
23242 |
23243 |     hr = ma_IMMDevice_Activate(*ppMMDDevice, &MA_IID_IAudioClient, CLSCTX_ALL, pActivationParams,
> (void**)ppAudioClient);
23244 |     if (FAILED(hr)) {
23245 |         return ma_result_from_HRESULT(hr);
23246 |     }
23247 |
23248 |     return MA_SUCCESS;
23249 | }
23250 | #else
23251 | static ma_result ma_context_get_IAudioClient_UWP__wasapi(ma_context* pContext, ma_device_type
> deviceType, const ma_device_id* pDeviceID, MA_PROPVARIANT* pActivationParams, ma_IAudioClient**

```



```

> ppAudioClient, ma_IUnknown** ppActivatedInterface)
23252 | {
23253 |     ma_IActivateAudioInterfaceAsyncOperation *pAsyncOp = NULL;
23254 |     ma_completion_handler_uwp completionHandler;
23255 |     IID iid;
23256 |     WCHAR* iidStr;
23257 |     HRESULT hr;
23258 |     ma_result result;
23259 |     HRESULT activateResult;
23260 |     ma_IUnknown* pActivatedInterface;
23261 |
23262 |     MA_ASSERT(pContext != NULL);
23263 |     MA_ASSERT(ppAudioClient != NULL);
23264 |
23265 |     if (pDeviceID != NULL) {
23266 |         iidStr = (WCHAR*)pDeviceID->wasapi;
23267 |     } else {
23268 |         if (deviceType == ma_device_type_capture) {
23269 |             iid = MA_IID_DEVINTERFACE_AUDIO_CAPTURE;
23270 |         } else {
23271 |             iid = MA_IID_DEVINTERFACE_AUDIO_RENDER;
23272 |         }
23273 |
23274 |         #if defined(__cplusplus)
23275 |             hr = StringFromIID(iid, &iidStr);
23276 |         #else
23277 |             hr = StringFromIID(&iid, &iidStr);
23278 |         #endif
23279 |         if (FAILED(hr)) {
23280 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to
> convert device IID to string for ActivateAudioInterfaceAsync(). Out of memory.\n");
23281 |             return ma_result_from_HRESULT(hr);
23282 |         }
23283 |     }
23284 |
23285 |     result = ma_completion_handler_uwp_init(&completionHandler);
23286 |     if (result != MA_SUCCESS) {
23287 |         ma_CoTaskMemFree(pContext, iidStr);
23288 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> event for waiting for ActivateAudioInterfaceAsync().\n");
23289 |         return result;
23290 |     }
23291 |
23292 |     hr = ((MA_PFN_ActivateAudioInterfaceAsync)pContext->wasapi.ActivateAudioInterfaceAsync)(iidStr,
> &MA_IID_IAudioClient, pActivationParams,
> (ma_IActivateAudioInterfaceCompletionHandler*)&completionHandler,
> (ma_IActivateAudioInterfaceAsyncOperation*)&pAsyncOp);
23293 |     if (FAILED(hr)) {
23294 |         ma_completion_handler_uwp_uninit(&completionHandler);
23295 |         ma_CoTaskMemFree(pContext, iidStr);
23296 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI]
> ActivateAudioInterfaceAsync() failed.\n");
23297 |         return ma_result_from_HRESULT(hr);
23298 |     }
23299 |
23300 |     if (pDeviceID == NULL) {
23301 |         ma_CoTaskMemFree(pContext, iidStr);
23302 |     }
23303 |
23304 |     /* Wait for the async operation for finish. */
23305 |     ma_completion_handler_uwp_wait(&completionHandler);
23306 |     ma_completion_handler_uwp_uninit(&completionHandler);
23307 |
23308 |     hr = ma_IActivateAudioInterfaceAsyncOperation_GetActivateResult(pAsyncOp, &activateResult,
> &pActivatedInterface);
23309 |     ma_IActivateAudioInterfaceAsyncOperation_Release(pAsyncOp);
23310 |
23311 |     if (FAILED(hr) || FAILED(activateResult)) {
23312 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to activate
> device.\n");
23313 |         return FAILED(hr) ? ma_result_from_HRESULT(hr) : ma_result_from_HRESULT(activateResult);
23314 |     }
23315 |
23316 |     /* Here is where we grab the IAudioClient interface. */
23317 |     hr = ma_IUnknown_QueryInterface(pActivatedInterface, &MA_IID_IAudioClient,
> (void**)ppAudioClient);
23318 |     if (FAILED(hr)) {
23319 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to query
> IAudioClient interface.\n");

```

```

23320 |         return ma_result_from_HRESULT(hr);
23321 |     }
23322 |
23323 |     if (ppActivatedInterface) {
23324 |         *ppActivatedInterface = pActivatedInterface;
23325 |     } else {
23326 |         ma_IUnknown_Release(pActivatedInterface);
23327 |     }
23328 |
23329 |     return MA_SUCCESS;
23330 | }
23331 | #endif
23332 |
23333 |
23334 | /* https://docs.microsoft.com/en-us/windows/win32/api/audioclientactivationparams/ne-audioclientacti
> vationparams-audioclient_activation_type */
23335 | typedef enum
23336 | {
23337 |     MA_AUDIOCLIENT_ACTIVATION_TYPE_DEFAULT,
23338 |     MA_AUDIOCLIENT_ACTIVATION_TYPE_PROCESS_LOOPBACK
23339 | } MA_AUDIOCLIENT_ACTIVATION_TYPE;
23340 |
23341 | /* https://docs.microsoft.com/en-us/windows/win32/api/audioclientactivationparams/ne-audioclientacti
> vationparams-process_loopback_mode */
23342 | typedef enum
23343 | {
23344 |     MA_PROCESS_LOOPBACK_MODE_INCLUDE_TARGET_PROCESS_TREE,
23345 |     MA_PROCESS_LOOPBACK_MODE_EXCLUDE_TARGET_PROCESS_TREE
23346 | } MA_PROCESS_LOOPBACK_MODE;
23347 |
23348 | /* https://docs.microsoft.com/en-us/windows/win32/api/audioclientactivationparams/ns-audioclientacti
> vationparams-audioclient_process_loopback_params */
23349 | typedef struct
23350 | {
23351 |     DWORD TargetProcessId;
23352 |     MA_PROCESS_LOOPBACK_MODE ProcessLoopbackMode;
23353 | } MA_AUDIOCLIENT_PROCESS_LOOPBACK_PARAMS;
23354 |
23355 | #if defined(_MSC_VER) && !defined(__clang__)
23356 |     #pragma warning(push)
23357 |     #pragma warning(disable:4201) /* nonstandard extension used: nameless struct/union */
23358 | #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__
> >= 8)))
23359 |     #pragma GCC diagnostic push
23360 |     #pragma GCC diagnostic ignored "-Wpedantic" /* For ISO C99 doesn't support unnamed
> structs/unions [-Wpedantic] */
23361 |     #if defined(__clang__)
23362 |         #pragma GCC diagnostic ignored "-Wc11-extensions" /* anonymous unions are a C11 extension
> */
23363 |     #endif
23364 | #endif
23365 | /* https://docs.microsoft.com/en-us/windows/win32/api/audioclientactivationparams/ns-audioclientacti
> vationparams-audioclient_activation_params */
23366 | typedef struct
23367 | {
23368 |     MA_AUDIOCLIENT_ACTIVATION_TYPE ActivationType;
23369 |     union
23370 |     {
23371 |         MA_AUDIOCLIENT_PROCESS_LOOPBACK_PARAMS ProcessLoopbackParams;
23372 |     };
23373 | } MA_AUDIOCLIENT_ACTIVATION_PARAMS;
23374 | #if defined(_MSC_VER) && !defined(__clang__)
23375 |     #pragma warning(pop)
23376 | #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__
> >= 8)))
23377 |     #pragma GCC diagnostic pop
23378 | #endif
23379 |
23380 | #define MA_VIRTUAL_AUDIO_DEVICE_PROCESS_LOOPBACK L"VAD\\Process_Loopback"
23381 |
23382 | static ma_result ma_context_get_IAudioClient__wasapi(ma_context* pContext, ma_device_type
> deviceType, const ma_device_id* pDeviceID, ma_uint32 loopbackProcessID, ma_bool32
> loopbackProcessExclude, ma_IAudioClient** ppAudioClient, ma_WASAPIDeviceInterface**
> ppDeviceInterface)
23383 | {
23384 |     ma_result result;
23385 |     ma_bool32 usingProcessLoopback = MA_FALSE;
23386 |     MA_AUDIOCLIENT_ACTIVATION_PARAMS audioclientActivationParams;
23387 |     MA_PROPVARIANT activationParams;

```

```

23388 |     MA_PROPVARIANT* pActivationParams = NULL;
23389 |     ma_device_id virtualDeviceID;
23390 |
23391 |     /* Activation parameters specific to loopback mode. Note that process-specific loopback will
> only work when a default device ID is specified. */
23392 |     if (deviceType == ma_device_type_loopback && loopbackProcessID != 0 && pDeviceID == NULL) {
23393 |         usingProcessLoopback = MA_TRUE;
23394 |     }
23395 |
23396 |     if (usingProcessLoopback) {
23397 |         MA_ZERO_OBJECT(&audioclientActivationParams);
23398 |         audioclientActivationParams.ActivationType =
> MA_AUDIOCLIENT_ACTIVATION_TYPE_PROCESS_LOOPBACK;
23399 |         audioclientActivationParams.ProcessLoopbackParams.ProcessLoopbackMode =
> (loopbackProcessExclude) ? MA_PROCESS_LOOPBACK_MODE_EXCLUDE_TARGET_PROCESS_TREE :
> MA_PROCESS_LOOPBACK_MODE_INCLUDE_TARGET_PROCESS_TREE;
23400 |         audioclientActivationParams.ProcessLoopbackParams.TargetProcessId =
> (DWORD)loopbackProcessID;
23401 |
23402 |         ma_PropVariantInit(&activationParams);
23403 |         activationParams.vt = MA_VT_BLOB;
23404 |         activationParams.blob.cbSize = sizeof(audioclientActivationParams);
23405 |         activationParams.blob.pBlobData = (BYTE*)&audioclientActivationParams;
23406 |         pActivationParams = &activationParams;
23407 |
23408 |         /* When requesting a specific device ID we need to use a special device ID. */
23409 |         MA_COPY_MEMORY(virtualDeviceID.wasapi, MA_VIRTUAL_AUDIO_DEVICE_PROCESS_LOOPBACK,
> (ma_wcslen(MA_VIRTUAL_AUDIO_DEVICE_PROCESS_LOOPBACK) + 1) * sizeof(wchar_t)); /* +1 for the null
> terminator. */
23410 |         pDeviceID = &virtualDeviceID;
23411 |     } else {
23412 |         pActivationParams = NULL; /* No activation parameters required. */
23413 |     }
23414 |
23415 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
23416 |     result = ma_context_get_IAudioClient_Desktop__wasapi(pContext, deviceType, pDeviceID,
> pActivationParams, ppAudioClient, ppDeviceInterface);
23417 | #else
23418 |     result = ma_context_get_IAudioClient_UWP__wasapi(pContext, deviceType, pDeviceID,
> pActivationParams, ppAudioClient, ppDeviceInterface);
23419 | #endif
23420 |
23421 |     /*
23422 |     If loopback mode was requested with a process ID and initialization failed, it could be because
> it's
23423 |     trying to run on an older version of Windows where it's not supported. We need to let the caller
23424 |     know about this with a log message.
23425 |     */
23426 |     if (result != MA_SUCCESS) {
23427 |         if (usingProcessLoopback) {
23428 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Loopback mode
> requested to %s process ID %u, but initialization failed. Support for this feature begins with
> Windows 10 Build 20348. Confirm your version of Windows or consider not using process-specific
> loopback.\n", (loopbackProcessExclude) ? "exclude" : "include", loopbackProcessID);
23429 |         }
23430 |     }
23431 |
23432 |     return result;
23433 | }
23434 |
23435 |
23436 | static ma_result ma_context_enumerate_devices__wasapi(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
23437 | {
23438 |     /* Different enumeration for desktop and UWP. */
23439 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
23440 |         /* Desktop */
23441 |         HRESULT hr;
23442 |         ma_IMMDeviceEnumerator* pDeviceEnumerator;
23443 |
23444 |         hr = ma_CoCreateInstance(pContext, &MA_CLSID_MMDeviceEnumerator, NULL, CLSCTX_ALL,
> &MA_IID_IMMDeviceEnumerator, (void*)&pDeviceEnumerator);
23445 |         if (FAILED(hr)) {
23446 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> device enumerator.");
23447 |             return ma_result_from_HRESULT(hr);
23448 |         }
23449 |
23450 |         ma_context_enumerate_devices_by_type__wasapi(pContext, pDeviceEnumerator,

```

```

> ma_device_type_playback, callback, pUserData);
23451 |     ma_context_enumerate_devices_by_type_wasapi(pContext, pDeviceEnumerator,
> ma_device_type_capture, callback, pUserData);
23452 |
23453 |     ma_IMMDeviceEnumerator_Release(pDeviceEnumerator);
23454 | #else
23455 |     /*
23456 |     UWP
23457 |
23458 |     The MMDevice API is only supported on desktop applications. For now, while I'm still figuring
> out how to properly enumerate
23459 |     over devices without using MMDevice, I'm restricting devices to defaults.
23460 |
23461 |     Hint: DeviceInformation::FindAllAsync() with DeviceClass.AudioCapture/AudioRender. https://blogs
> .windows.com/buildingapps/2014/05/15/real-time-audio-in-windows-store-and-windows-phone-apps/
23462 |     */
23463 |     if (callback) {
23464 |         ma_bool32 cbResult = MA_TRUE;
23465 |
23466 |         /* Playback. */
23467 |         if (cbResult) {
23468 |             ma_device_info deviceInfo;
23469 |             MA_ZERO_OBJECT(&deviceInfo);
23470 |             ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
23471 |             deviceInfo.isDefault = MA_TRUE;
23472 |             cbResult = callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
23473 |         }
23474 |
23475 |         /* Capture. */
23476 |         if (cbResult) {
23477 |             ma_device_info deviceInfo;
23478 |             MA_ZERO_OBJECT(&deviceInfo);
23479 |             ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
23480 |             deviceInfo.isDefault = MA_TRUE;
23481 |             cbResult = callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
23482 |         }
23483 |     }
23484 | #endif
23485 |     return MA_SUCCESS;
23486 | }
23487 | }
23488 |
23489 | static ma_result ma_context_get_device_info_wasapi(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
23490 | {
23491 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
23492 |         ma_result result;
23493 |         ma_IMMDevice* pMMDevice = NULL;
23494 |         WCHAR* pDefaultDeviceID = NULL;
23495 |
23496 |         result = ma_context_get_MMDevice_wasapi(pContext, deviceType, pDeviceID, &pMMDevice);
23497 |         if (result != MA_SUCCESS) {
23498 |             return result;
23499 |         }
23500 |
23501 |         /* We need the default device ID so we can set the isDefault flag in the device info. */
23502 |         pDefaultDeviceID = ma_context_get_default_device_id_wasapi(pContext, deviceType);
23503 |
23504 |         result = ma_context_get_device_info_from_MMDevice_wasapi(pContext, pMMDevice, pDefaultDeviceID,
> MA_FALSE, pDeviceInfo); /* MA_FALSE = !onlySimpleInfo. */
23505 |
23506 |         if (pDefaultDeviceID != NULL) {
23507 |             ma_CoTaskMemFree(pContext, pDefaultDeviceID);
23508 |             pDefaultDeviceID = NULL;
23509 |         }
23510 |
23511 |         ma_IMMDevice_Release(pMMDevice);
23512 |
23513 |         return result;
23514 |     #else
23515 |         ma_IAudioClient* pAudioClient;
23516 |         ma_result result;
23517 |
23518 |         /* UWP currently only uses default devices. */
23519 |         if (deviceType == ma_device_type_playback) {
23520 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);

```

```

23521 |     } else {
23522 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
23523 |     }
23524 |
23525 |     result = ma_context_get_IAudioClient_UWP__wasapi(pContext, deviceType, pDeviceID, NULL,
> &pAudioClient, NULL);
23526 |     if (result != MA_SUCCESS) {
23527 |         return result;
23528 |     }
23529 |
23530 |     result = ma_context_get_device_info_from_IAudioClient__wasapi(pContext, NULL, pAudioClient,
> pDeviceInfo);
23531 |
23532 |     pDeviceInfo->isDefault = MA_TRUE; /* UWP only supports default devices. */
23533 |
23534 |     ma_IAudioClient_Release(pAudioClient);
23535 |     return result;
23536 | #endif
23537 | }
23538 |
23539 | static ma_result ma_device_uninit__wasapi(ma_device* pDevice)
23540 | {
23541 |     MA_ASSERT(pDevice != NULL);
23542 |
23543 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
23544 |     {
23545 |         if (pDevice->wasapi.pDeviceEnumerator) {
23546 |             ((ma_IMMDeviceEnumerator*)pDevice->wasapi.pDeviceEnumerator)->lpVtbl->UnregisterEndpoint
> NotificationCallback((ma_IMMDeviceEnumerator*)pDevice->wasapi.pDeviceEnumerator,
> &pDevice->wasapi.notificationClient);
23547 |
> ma_IMMDeviceEnumerator_Release((ma_IMMDeviceEnumerator*)pDevice->wasapi.pDeviceEnumerator);
23548 |         }
23549 |
23550 |         ma_mutex_uninit(&pDevice->wasapi.rerouteLock);
23551 |     }
23552 |     #endif
23553 |
23554 |     if (pDevice->wasapi.pRenderClient) {
23555 |         if (pDevice->wasapi.pMappedBufferPlayback != NULL) {
23556 |
> ma_IAudioRenderClient_ReleaseBuffer((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient,
> pDevice->wasapi.mappedBufferPlaybackCap, 0);
23557 |             pDevice->wasapi.pMappedBufferPlayback = NULL;
23558 |             pDevice->wasapi.mappedBufferPlaybackCap = 0;
23559 |             pDevice->wasapi.mappedBufferPlaybackLen = 0;
23560 |         }
23561 |
23562 |         ma_IAudioRenderClient_Release((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient);
23563 |     }
23564 |     if (pDevice->wasapi.pCaptureClient) {
23565 |         if (pDevice->wasapi.pMappedBufferCapture != NULL) {
23566 |
> ma_IAudioCaptureClient_ReleaseBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> pDevice->wasapi.mappedBufferCaptureCap);
23567 |             pDevice->wasapi.pMappedBufferCapture = NULL;
23568 |             pDevice->wasapi.mappedBufferCaptureCap = 0;
23569 |             pDevice->wasapi.mappedBufferCaptureLen = 0;
23570 |         }
23571 |
23572 |         ma_IAudioCaptureClient_Release((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient);
23573 |     }
23574 |
23575 |     if (pDevice->wasapi.pAudioClientPlayback) {
23576 |         ma_IAudioClient_Release((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback);
23577 |     }
23578 |     if (pDevice->wasapi.pAudioClientCapture) {
23579 |         ma_IAudioClient_Release((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
23580 |     }
23581 |
23582 |     if (pDevice->wasapi.hEventPlayback) {
23583 |         CloseHandle((HANDLE)pDevice->wasapi.hEventPlayback);
23584 |     }
23585 |     if (pDevice->wasapi.hEventCapture) {
23586 |         CloseHandle((HANDLE)pDevice->wasapi.hEventCapture);
23587 |     }
23588 |
23589 |     return MA_SUCCESS;

```

```

23590 | }
23591 |
23592 |
23593 | typedef struct
23594 | {
23595 |     /* Input. */
23596 |     ma_format formatIn;
23597 |     ma_uint32 channelsIn;
23598 |     ma_uint32 sampleRateIn;
23599 |     ma_channel channelMapIn[MA_MAX_CHANNELS];
23600 |     ma_uint32 periodSizeInFramesIn;
23601 |     ma_uint32 periodSizeInMillisecondsIn;
23602 |     ma_uint32 periodsIn;
23603 |     ma_share_mode shareMode;
23604 |     ma_performance_profile performanceProfile;
23605 |     ma_bool32 noAutoConvertSRC;
23606 |     ma_bool32 noDefaultQualitySRC;
23607 |     ma_bool32 noHardwareOffloading;
23608 |     ma_uint32 loopbackProcessID;
23609 |     ma_bool32 loopbackProcessExclude;
23610 |
23611 |     /* Output. */
23612 |     ma_IAudioClient* pAudioClient;
23613 |     ma_IAudioRenderClient* pRenderClient;
23614 |     ma_IAudioCaptureClient* pCaptureClient;
23615 |     ma_format formatOut;
23616 |     ma_uint32 channelsOut;
23617 |     ma_uint32 sampleRateOut;
23618 |     ma_channel channelMapOut[MA_MAX_CHANNELS];
23619 |     ma_uint32 periodSizeInFramesOut;
23620 |     ma_uint32 periodsOut;
23621 |     ma_bool32 usingAudioClient3;
23622 |     char deviceName[256];
23623 |     ma_device_id id;
23624 | } ma_device_init_internal_data_wasapi;
23625 |
23626 | static ma_result ma_device_init_internal_wasapi(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_init_internal_data_wasapi* pData)
23627 | {
23628 |     HRESULT hr;
23629 |     ma_result result = MA_SUCCESS;
23630 |     const char* errorMsg = "";
23631 |     MA_AUDCLNT_SHAREMODE shareMode = MA_AUDCLNT_SHAREMODE_SHARED;
23632 |     DWORD streamFlags = 0;
23633 |     MA_REFERENCE_TIME periodDurationInMicroseconds;
23634 |     ma_bool32 wasInitializedUsingIAudioClient3 = MA_FALSE;
23635 |     MA_WAVEFORMATEXTENSIBLE wf;
23636 |     ma_WASAPIDeviceInterface* pDeviceInterface = NULL;
23637 |     ma_IAudioClient2* pAudioClient2;
23638 |     ma_uint32 nativeSampleRate;
23639 |     ma_bool32 usingProcessLoopback = MA_FALSE;
23640 |
23641 |     MA_ASSERT(pContext != NULL);
23642 |     MA_ASSERT(pData != NULL);
23643 |
23644 |     /* This function is only used to initialize one device type: either playback, capture or
> loopback. Never full-duplex. */
23645 |     if (deviceType == ma_device_type_duplex) {
23646 |         return MA_INVALID_ARGS;
23647 |     }
23648 |
23649 |     usingProcessLoopback = deviceType == ma_device_type_loopback && pData->loopbackProcessID != 0 &&
> pDeviceID == NULL;
23650 |
23651 |     pData->pAudioClient = NULL;
23652 |     pData->pRenderClient = NULL;
23653 |     pData->pCaptureClient = NULL;
23654 |
23655 |     streamFlags = MA_AUDCLNT_STREAMFLAGS_EVENTCALLBACK;
23656 |     if (!pData->noAutoConvertSRC && pData->sampleRateIn != 0 && pData->shareMode !=
> ma_share_mode_exclusive) { /* <-- Exclusive streams must use the native sample rate. */
23657 |         streamFlags |= MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM;
23658 |     }
23659 |     if (!pData->noDefaultQualitySRC && pData->sampleRateIn != 0 && (streamFlags &
> MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM) != 0) {
23660 |         streamFlags |= MA_AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY;
23661 |     }
23662 |     if (deviceType == ma_device_type_loopback) {
23663 |         streamFlags |= MA_AUDCLNT_STREAMFLAGS_LOOPBACK;

```

```

23664 |     }
23665 |
23666 |     result = ma_context_get_IAudioClient__wasapi(pContext, deviceType, pDeviceID,
> pData->loopbackProcessID, pData->loopbackProcessExclude, &pData->pAudioClient, &pDeviceInterface);
23667 |     if (result != MA_SUCCESS) {
23668 |         goto done;
23669 |     }
23670 |
23671 |     MA_ZERO_OBJECT(&wf);
23672 |
23673 |     /* Try enabling hardware offloading. */
23674 |     if (!pData->noHardwareOffloading) {
23675 |         hr = ma_IAudioClient_QueryInterface(pData->pAudioClient, &MA_IID_IAudioClient2,
> (void**)&pAudioClient2);
23676 |         if (SUCCEEDED(hr)) {
23677 |             BOOL isHardwareOffloadingSupported = 0;
23678 |             hr = ma_IAudioClient2_IsOffloadCapable(pAudioClient2, MA_AudioCategory_Other,
> &isHardwareOffloadingSupported);
23679 |             if (SUCCEEDED(hr) && isHardwareOffloadingSupported) {
23680 |                 ma_AudioClientProperties clientProperties;
23681 |                 MA_ZERO_OBJECT(&clientProperties);
23682 |                 clientProperties.cbSize = sizeof(clientProperties);
23683 |                 clientProperties.bIsOffload = 1;
23684 |                 clientProperties.eCategory = MA_AudioCategory_Other;
23685 |                 ma_IAudioClient2_SetClientProperties(pAudioClient2, &clientProperties);
23686 |             }
23687 |
23688 |             pAudioClient2->lpVtbl->Release(pAudioClient2);
23689 |         }
23690 |     }
23691 |
23692 |     /* Here is where we try to determine the best format to use with the device. If the client if
> wanting exclusive mode, first try finding the best format for that. If this fails, fall back to
> shared mode. */
23693 |     result = MA_FORMAT_NOT_SUPPORTED;
23694 |     if (pData->shareMode == ma_share_mode_exclusive) {
23695 |         #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
23696 |             /* In exclusive mode on desktop we always use the backend's native format. */
23697 |             ma_IPropertyStore* pStore = NULL;
23698 |             hr = ma_IMMDevice_OpenPropertyStore(pDeviceInterface, STGM_READ, &pStore);
23699 |             if (SUCCEEDED(hr)) {
23700 |                 MA_PROPVARIANT prop;
23701 |                 ma_PropVariantInit(&prop);
23702 |                 hr = ma_IPropertyStore_GetValue(pStore, &MA_PKEY_AudioEngine_DeviceFormat, &prop);
23703 |                 if (SUCCEEDED(hr)) {
23704 |                     MA_WAVEFORMATEX* pActualFormat = (MA_WAVEFORMATEX*)prop.blob.pBlobData;
23705 |                     hr = ma_IAudioClient_IsFormatSupported((ma_IAudioClient*)pData->pAudioClient,
> MA_AUDCLNT_SHAREMODE_EXCLUSIVE, pActualFormat, NULL);
23706 |                     if (SUCCEEDED(hr)) {
23707 |                         MA_COPY_MEMORY(&wf, pActualFormat, sizeof(MA_WAVEFORMATEXTENSIBLE));
23708 |                     }
23709 |
23710 |                     ma_PropVariantClear(pContext, &prop);
23711 |                 }
23712 |
23713 |                 ma_IPropertyStore_Release(pStore);
23714 |             }
23715 |         #else
23716 |             /*
23717 |             I do not know how to query the device's native format on UWP so for now I'm just disabling
> support for
23718 |             exclusive mode. The alternative is to enumerate over different formats and check
> IsFormatSupported()
23719 |             until you find one that works.
23720 |
23721 |             TODO: Add support for exclusive mode to UWP.
23722 |             */
23723 |             hr = S_FALSE;
23724 |         #endif
23725 |
23726 |         if (hr == S_OK) {
23727 |             shareMode = MA_AUDCLNT_SHAREMODE_EXCLUSIVE;
23728 |             result = MA_SUCCESS;
23729 |         } else {
23730 |             result = MA_SHARE_MODE_NOT_SUPPORTED;
23731 |         }
23732 |     } else {
23733 |         /* In shared mode we are always using the format reported by the operating system. */
23734 |         MA_WAVEFORMATEXTENSIBLE* pNativeFormat = NULL;

```



```

23735 |         hr = ma_IAudioClient_GetMixFormat((ma_IAudioClient*)pData->pAudioClient,
> (MA_WAVEFORMATEX*)&pNativeFormat);
23736 |         if (hr != S_OK) {
23737 |             /* When using process-specific loopback, GetMixFormat() seems to always fail. */
23738 |             if (usingProcessLoopback) {
23739 |                 wf.wFormatTag = WAVE_FORMAT_IEEE_FLOAT;
23740 |                 wf.nChannels = 2;
23741 |                 wf.nSamplesPerSec = 44100;
23742 |                 wf.wBitsPerSample = 32;
23743 |                 wf.nBlockAlign = wf.nChannels * wf.wBitsPerSample / 8;
23744 |                 wf.nAvgBytesPerSec = wf.nSamplesPerSec * wf.nBlockAlign;
23745 |                 wf.cbSize = sizeof(MA_WAVEFORMATEX);
23746 |
23747 |                 result = MA_SUCCESS;
23748 |             } else {
23749 |                 result = MA_FORMAT_NOT_SUPPORTED;
23750 |             }
23751 |         } else {
23752 |             /*
23753 |             I've seen cases where cbSize will be set to sizeof(WAVEFORMATEX) even though the
> structure itself
23754 |             is given the format tag of WAVE_FORMAT_EXTENSIBLE. If the format tag is
> WAVE_FORMAT_EXTENSIBLE
23755 |             want to make sure we copy the whole WAVEFORMATEX structure. Otherwise we'll have
> to be
23756 |             safe and only copy the WAVEFORMATEX part.
23757 |             */
23758 |             if (pNativeFormat->wFormatTag == WAVE_FORMAT_EXTENSIBLE) {
23759 |                 MA_COPY_MEMORY(&wf, pNativeFormat, sizeof(MA_WAVEFORMATEX));
23760 |             } else {
23761 |                 /* I've seen a case where cbSize was set to 0. Assume sizeof(WAVEFORMATEX) in this
> case. */
23762 |                 size_t cbSize = pNativeFormat->cbSize;
23763 |                 if (cbSize == 0) {
23764 |                     cbSize = sizeof(MA_WAVEFORMATEX);
23765 |                 }
23766 |
23767 |                 /* Make sure we don't copy more than the capacity of `wf`. */
23768 |                 if (cbSize > sizeof(wf)) {
23769 |                     cbSize = sizeof(wf);
23770 |                 }
23771 |
23772 |                 MA_COPY_MEMORY(&wf, pNativeFormat, cbSize);
23773 |             }
23774 |
23775 |             result = MA_SUCCESS;
23776 |         }
23777 |
23778 |         ma_CoTaskMemFree(pContext, pNativeFormat);
23779 |
23780 |         shareMode = MA_AUDCLNT_SHAREMODE_SHARED;
23781 |     }
23782 |
23783 |     /* Return an error if we still haven't found a format. */
23784 |     if (result != MA_SUCCESS) {
23785 |         errorMsg = "[WASAPI] Failed to find best device mix format.";
23786 |         goto done;
23787 |     }
23788 |
23789 |     /*
23790 |     Override the native sample rate with the one requested by the caller, but only if we're not
> using the default sample rate. We'll use
23791 |     WASAPI to perform the sample rate conversion.
23792 |     */
23793 |     nativeSampleRate = wf.nSamplesPerSec;
23794 |     if (streamFlags & MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM) {
23795 |         wf.nSamplesPerSec = (pData->sampleRateIn != 0) ? pData->sampleRateIn :
> MA_DEFAULT_SAMPLE_RATE;
23796 |         wf.nAvgBytesPerSec = wf.nSamplesPerSec * wf.nBlockAlign;
23797 |     }
23798 |
23799 |     pData->formatOut = ma_format_from_WAVEFORMATEX((MA_WAVEFORMATEX*)&wf);
23800 |     if (pData->formatOut == ma_format_unknown) {
23801 |         /*
23802 |         The format isn't supported. This is almost certainly because the exclusive mode format isn't
> supported by miniaudio. We need to return MA_SHARE_MODE_NOT_SUPPORTED
23803 |         in this case so that the caller can detect it and fall back to shared mode if desired. We
> should never get here if shared mode was requested, but just for
23804 |         completeness we'll check for it and return MA_FORMAT_NOT_SUPPORTED.

```

```

23805 |         */
23806 |         if (shareMode == MA_AUDCLNT_SHAREMODE_EXCLUSIVE) {
23807 |             result = MA_SHARE_MODE_NOT_SUPPORTED;
23808 |         } else {
23809 |             result = MA_FORMAT_NOT_SUPPORTED;
23810 |         }
23811 |
23812 |         errorMsg = "[WASAPI] Native format not supported.";
23813 |         goto done;
23814 |     }
23815 |
23816 |     pData->channelsOut = wf.nChannels;
23817 |     pData->sampleRateOut = wf.nSamplesPerSec;
23818 |
23819 |     /*
23820 |     Get the internal channel map based on the channel mask. There is a possibility that
> GetMixFormat() returns
23821 |     a WAVEFORMATEX instead of a WAVEFORMATEXTENSIBLE, in which case the channel mask will be
> undefined. In this
23822 |     case we'll just use the default channel map.
23823 |     */
23824 |     if (wf.wFormatTag == WAVE_FORMAT_EXTENSIBLE || wf.cbSize >= sizeof(MA_WAVEFORMATEXTENSIBLE)) {
23825 |         ma_channel_mask_to_channel_map_win32(wf.dwChannelMask, pData->channelsOut,
> pData->channelMapOut);
23826 |     } else {
23827 |         ma_channel_map_init_standard(ma_standard_channel_map_microsoft, pData->channelMapOut,
> ma_countof(pData->channelMapOut), pData->channelsOut);
23828 |     }
23829 |
23830 |     /* Period size. */
23831 |     pData->periodsOut = (pData->periodsIn != 0) ? pData->periodsIn : MA_DEFAULT_PERIODS;
23832 |     pData->periodSizeInFramesOut = pData->periodSizeInFramesIn;
23833 |     if (pData->periodSizeInFramesOut == 0) {
23834 |         if (pData->periodSizeInMillisecondsIn == 0) {
23835 |             if (pData->performanceProfile == ma_performance_profile_low_latency) {
23836 |                 pData->periodSizeInFramesOut = ma_calculate_buffer_size_in_frames_from_milliseconds(
> MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_LOW_LATENCY, wf.nSamplesPerSec);
23837 |             } else {
23838 |                 pData->periodSizeInFramesOut = ma_calculate_buffer_size_in_frames_from_milliseconds(
> MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_CONSERVATIVE, wf.nSamplesPerSec);
23839 |             }
23840 |         } else {
23841 |             pData->periodSizeInFramesOut =
> ma_calculate_buffer_size_in_frames_from_milliseconds(pData->periodSizeInMillisecondsIn,
> wf.nSamplesPerSec);
23842 |         }
23843 |     }
23844 |
23845 |     periodDurationInMicroseconds = ((ma_uint64)pData->periodSizeInFramesOut * 1000 * 1000) /
> wf.nSamplesPerSec;
23846 |
23847 |
23848 |     /* Slightly different initialization for shared and exclusive modes. We try exclusive mode
> first, and if it fails, fall back to shared mode. */
23849 |     if (shareMode == MA_AUDCLNT_SHAREMODE_EXCLUSIVE) {
23850 |         MA_REFERENCE_TIME bufferDuration = periodDurationInMicroseconds * pData->periodsOut * 10;
23851 |
23852 |         /*
23853 |         If the periodicity is too small, Initialize() will fail with
> AUDCLNT_E_INVALID_DEVICE_PERIOD. In this case we should just keep increasing
23854 |         it and trying it again.
23855 |         */
23856 |         hr = E_FAIL;
23857 |         for (;;) {
23858 |             hr = ma_IAudioClient_Initialize((ma_IAudioClient*)pData->pAudioClient, shareMode,
> streamFlags, bufferDuration, bufferDuration, (MA_WAVEFORMATEX*)&wf, NULL);
23859 |             if (hr == MA_AUDCLNT_E_INVALID_DEVICE_PERIOD) {
23860 |                 if (bufferDuration > 500*10000) {
23861 |                     break;
23862 |                 } else {
23863 |                     if (bufferDuration == 0) { /* <-- Just a sanity check to prevent an infinite
> loop. Should never happen, but it makes me feel better. */
23864 |                         break;
23865 |                     }
23866 |
23867 |                     bufferDuration = bufferDuration * 2;
23868 |                     continue;
23869 |                 }
23870 |             } else {

```

```

23871 |         break;
23872 |     }
23873 | }
23874 |
23875 |     if (hr == MA_AUDCLNT_E_BUFFER_SIZE_NOT_ALIGNED) {
23876 |         ma_uint32 bufferSizeInFrames;
23877 |         hr = ma_IAudioClient_GetBufferSize((ma_IAudioClient*)pData->pAudioClient,
> &bufferSizeInFrames);
23878 |         if (SUCCEEDED(hr)) {
23879 |             bufferDuration = (MA_REFERENCE_TIME)((10000.0 * 1000 / wf.nSamplesPerSec *
> bufferSizeInFrames) + 0.5);
23880 |
23881 |             /* Unfortunately we need to release and re-acquire the audio client according to
> MSDN. Seems silly - why not just call IAudioClient_Initialize() again?! */
23882 |             ma_IAudioClient_Release((ma_IAudioClient*)pData->pAudioClient);
23883 |
23884 |             #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
23885 |             hr = ma_IMMDevice_Activate(pDeviceInterface, &MA_IID_IAudioClient, CLSCTX_ALL, NULL,
> (void**)&pData->pAudioClient);
23886 |             #else
23887 |             hr = ma_IUnknown_QueryInterface(pDeviceInterface, &MA_IID_IAudioClient,
> (void**)&pData->pAudioClient);
23888 |             #endif
23889 |
23890 |             if (SUCCEEDED(hr)) {
23891 |                 hr = ma_IAudioClient_Initialize((ma_IAudioClient*)pData->pAudioClient,
> shareMode, streamFlags, bufferDuration, bufferDuration, (MA_WAVEFORMATEX*)&wf, NULL);
23892 |             }
23893 |         }
23894 |     }
23895 |
23896 |     if (FAILED(hr)) {
23897 |         /* Failed to initialize in exclusive mode. Don't fall back to shared mode - instead tell
> the client about it. They can reinitialize in shared mode if they want. */
23898 |         if (hr == E_ACCESSDENIED) {
23899 |             errorMsg = "[WASAPI] Failed to initialize device in exclusive mode. Access denied.",
> result = MA_ACCESS_DENIED;
23900 |         } else if (hr == MA_AUDCLNT_E_DEVICE_IN_USE) {
23901 |             errorMsg = "[WASAPI] Failed to initialize device in exclusive mode. Device in use.",
> result = MA_BUSY;
23902 |         } else {
23903 |             errorMsg = "[WASAPI] Failed to initialize device in exclusive mode."; result =
> ma_result_from_HRESULT(hr);
23904 |         }
23905 |         goto done;
23906 |     }
23907 | }
23908 |
23909 |     if (shareMode == MA_AUDCLNT_SHAREMODE_SHARED) {
23910 |         /*
23911 |         Low latency shared mode via IAudioClient3.
23912 |
23913 |         NOTE
23914 |         ====
23915 |         Contrary to the documentation on MSDN (https://docs.microsoft.com/en-us/windows/win32/api/audioclient/nf-audioclient-iaudioclient3-initializesharedaudiostream), the
> use of AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM and AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY with
> IAudioClient3_InitializeSharedAudioStream() absolutely does not work. Using
23917 |         any of these flags will result in HRESULT code 0x88890021. The other problem is that calling
> IAudioClient3_GetSharedModeEnginePeriod() with a sample rate different to
23918 |         that returned by IAudioClient_GetMixFormat() also results in an error. I'm therefore
> disabling low-latency shared mode with AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM.
23919 |         */
23920 |         #ifndef MA_WASAPI_NO_LOW_LATENCY_SHARED_MODE
23921 |         {
23922 |             if ((streamFlags & MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM) == 0 || nativeSampleRate ==
> wf.nSamplesPerSec) {
23923 |                 ma_IAudioClient3* pAudioClient3 = NULL;
23924 |                 hr = ma_IAudioClient_QueryInterface(pData->pAudioClient, &MA_IID_IAudioClient3,
> (void**)&pAudioClient3);
23925 |                 if (SUCCEEDED(hr)) {
23926 |                     ma_uint32 defaultPeriodInFrames;
23927 |                     ma_uint32 fundamentalPeriodInFrames;
23928 |                     ma_uint32 minPeriodInFrames;
23929 |                     ma_uint32 maxPeriodInFrames;
23930 |                     hr = ma_IAudioClient3_GetSharedModeEnginePeriod(pAudioClient3,
> (MA_WAVEFORMATEX*)&wf, &defaultPeriodInFrames, &fundamentalPeriodInFrames, &minPeriodInFrames,
> &maxPeriodInFrames);
23931 |                     if (SUCCEEDED(hr)) {

```

```

23932 |                 ma_uint32 desiredPeriodInFrames = pData->periodSizeInFramesOut;
23933 |                 ma_uint32 actualPeriodInFrames = desiredPeriodInFrames;
23934 |
23935 |                 /* Make sure the period size is a multiple of fundamentalPeriodInFrames. */
23936 |                 actualPeriodInFrames = actualPeriodInFrames / fundamentalPeriodInFrames;
23937 |                 actualPeriodInFrames = actualPeriodInFrames * fundamentalPeriodInFrames;
23938 |
23939 |                 /* The period needs to be clamped between minPeriodInFrames and
> maxPeriodInFrames. */
23940 |                 actualPeriodInFrames = ma_clamp(actualPeriodInFrames, minPeriodInFrames,
> maxPeriodInFrames);
23941 |
23942 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "[WASAPI]
> Trying IAudioClient3_InitializeSharedAudioStream(actualPeriodInFrames=%d)\n", actualPeriodInFrames);
23943 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "
> defaultPeriodInFrames=%d\n", defaultPeriodInFrames);
23944 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "
> fundamentalPeriodInFrames=%d\n", fundamentalPeriodInFrames);
23945 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "
> minPeriodInFrames=%d\n", minPeriodInFrames);
23946 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "
> maxPeriodInFrames=%d\n", maxPeriodInFrames);
23947 |
23948 |                 /* If the client requested a largish buffer than we don't actually want to
> use low latency shared mode because it forces small buffers. */
23949 |                 if (actualPeriodInFrames >= desiredPeriodInFrames) {
23950 |                     /*
23951 |                     MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM |
> MA_AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY must not be in the stream flags. If either of these are
> specified,
23952 |                     IAudioClient3_InitializeSharedAudioStream() will fail.
23953 |                     */
23954 |                     hr = ma_IAudioClient3_InitializeSharedAudioStream(pAudioClient3,
> streamFlags & ~(MA_AUDCLNT_STREAMFLAGS_AUTOCONVERTPCM | MA_AUDCLNT_STREAMFLAGS_SRC_DEFAULT_QUALITY),
> actualPeriodInFrames, (MA_WAVEFORMATEX*)&wf, NULL);
23955 |                     if (SUCCEEDED(hr)) {
23956 |                         wasInitializedUsingIAudioClient3 = MA_TRUE;
23957 |                         pData->periodSizeInFramesOut = actualPeriodInFrames;
23958 |
23959 |                         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG,
> "[WASAPI] Using IAudioClient3\n");
23960 |                         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "
> periodSizeInFramesOut=%d\n", pData->periodSizeInFramesOut);
23961 |                     } else {
23962 |                         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG,
> "[WASAPI] IAudioClient3_InitializeSharedAudioStream failed. Falling back to IAudioClient.\n");
23963 |                     }
23964 |                 } else {
23965 |                     ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "[WASAPI]
> Not using IAudioClient3 because the desired period size is larger than the maximum supported by
> IAudioClient3.\n");
23966 |                 }
23967 |                 } else {
23968 |                     ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "[WASAPI]
> IAudioClient3_GetSharedModeEnginePeriod failed. Falling back to IAudioClient.\n");
23969 |                 }
23970 |
23971 |                 ma_IAudioClient3_Release(pAudioClient3);
23972 |                 pAudioClient3 = NULL;
23973 |             }
23974 |         }
23975 |     }
23976 |     #else
23977 |     {
23978 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "[WASAPI] Not using
> IAudioClient3 because MA_WASAPI_NO_LOW_LATENCY_SHARED_MODE is enabled.\n");
23979 |     }
23980 |     #endif
23981 |
23982 |     /* If we don't have an IAudioClient3 then we need to use the normal initialization routine.
> */
23983 |     if (!wasInitializedUsingIAudioClient3) {
23984 |         MA_REFERENCE_TIME bufferDuration = periodDurationInMicroseconds * pData->periodsOut *
> 10; /* <-- Multiply by 10 for microseconds to 100-nanoseconds. */
23985 |         hr = ma_IAudioClient_Initialize((ma_IAudioClient*)&pAudioClient, shareMode,
> streamFlags, bufferDuration, 0, (const MA_WAVEFORMATEX*)&wf, NULL);
23986 |         if (FAILED(hr)) {
23987 |             if (hr == E_ACCESSDENIED) {
23988 |                 errorMsg = "[WASAPI] Failed to initialize device. Access denied.", result =

```

```

> MA_ACCESS_DENIED;
23989 |         } else if (hr == MA_AUDCLNT_E_DEVICE_IN_USE) {
23990 |             errorMsg = "[WASAPI] Failed to initialize device. Device in use.", result =
> MA_BUSY;
23991 |         } else {
23992 |             errorMsg = "[WASAPI] Failed to initialize device.", result =
> ma_result_from_HRESULT(hr);
23993 |         }
23994 |
23995 |         goto done;
23996 |     }
23997 | }
23998 | }
23999 |
24000 |     if (!wasInitializedUsingIAudioClient3) {
24001 |         ma_uint32 bufferSizeInFrames = 0;
24002 |         hr = ma_IAudioClient_GetBufferSize((ma_IAudioClient*)pData->pAudioClient,
> &bufferSizeInFrames);
24003 |         if (FAILED(hr)) {
24004 |             errorMsg = "[WASAPI] Failed to get audio client's actual buffer size.", result =
> ma_result_from_HRESULT(hr);
24005 |             goto done;
24006 |         }
24007 |
24008 |         /*
24009 |         When using process loopback mode, retrieval of the buffer size seems to result in totally
24010 |         incorrect values. In this case we'll just assume it's the same size as what we requested
24011 |         when we initialized the client.
24012 |         */
24013 |         if (usingProcessLoopback) {
24014 |             bufferSizeInFrames = (ma_uint32)((periodDurationInMicroseconds * pData->periodsOut) *
> pData->sampleRateOut / 1000000);
24015 |         }
24016 |
24017 |         pData->periodSizeInFramesOut = bufferSizeInFrames / pData->periodsOut;
24018 |     }
24019 |
24020 |     pData->usingAudioClient3 = wasInitializedUsingIAudioClient3;
24021 |
24022 |
24023 |     if (deviceType == ma_device_type_playback) {
24024 |         result = ma_device_create_IAudioClient_service_wasapi(pContext, deviceType,
> (ma_IAudioClient*)pData->pAudioClient, (void**)&pData->pRenderClient);
24025 |     } else {
24026 |         result = ma_device_create_IAudioClient_service_wasapi(pContext, deviceType,
> (ma_IAudioClient*)pData->pAudioClient, (void**)&pData->pCaptureClient);
24027 |     }
24028 |
24029 |     /*if (FAILED(hr)) {*/
24030 |     if (result != MA_SUCCESS) {
24031 |         errorMsg = "[WASAPI] Failed to get audio client service.";
24032 |         goto done;
24033 |     }
24034 |
24035 |
24036 |     /* Grab the name of the device. */
24037 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
24038 |     {
24039 |         ma_IPropertyStore *pProperties;
24040 |         hr = ma_IMMDevice_OpenPropertyStore(pDeviceInterface, STGM_READ, &pProperties);
24041 |         if (SUCCEEDED(hr)) {
24042 |             MA_PROPVARIANT varName;
24043 |             ma_PropVariantInit(&varName);
24044 |             hr = ma_IPropertyStore_GetValue(pProperties, &MA_PKEY_Device_FriendlyName, &varName);
24045 |             if (SUCCEEDED(hr)) {
24046 |                 WideCharToMultiByte(CP_UTF8, 0, varName.pwszVal, -1, pData->deviceName,
> sizeof(pData->deviceName), 0, FALSE);
24047 |                 ma_PropVariantClear(pContext, &varName);
24048 |             }
24049 |
24050 |             ma_IPropertyStore_Release(pProperties);
24051 |         }
24052 |     }
24053 |     #endif
24054 |
24055 |     /*
24056 |     For the WASAPI backend we need to know the actual IDs of the device in order to do automatic
24057 |     stream routing so that IDs can be compared and we can determine which device has been detached
24058 |     and whether or not it matches with our ma_device.

```

```

24059 | */
24060 | #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
24061 | {
24062 |     /* Desktop */
24063 |     ma_context_get_device_id_from_MMDevice_wasapi(pContext, pDeviceInterface, &pData->id);
24064 | }
24065 | #else
24066 | {
24067 |     /* UWP */
24068 |     /* TODO: Implement me. Need to figure out how to get the ID of the default device. */
24069 | }
24070 | #endif
24071 |
24072 | done:
24073 |     /* Clean up. */
24074 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
24075 |     if (pDeviceInterface != NULL) {
24076 |         ma_IMMDevice_Release(pDeviceInterface);
24077 |     }
24078 |     #else
24079 |     if (pDeviceInterface != NULL) {
24080 |         ma_IUnknown_Release(pDeviceInterface);
24081 |     }
24082 |     #endif
24083 |
24084 |     if (result != MA_SUCCESS) {
24085 |         if (pData->pRenderClient) {
24086 |             ma_IAudioRenderClient_Release((ma_IAudioRenderClient*)pData->pRenderClient);
24087 |             pData->pRenderClient = NULL;
24088 |         }
24089 |         if (pData->pCaptureClient) {
24090 |             ma_IAudioCaptureClient_Release((ma_IAudioCaptureClient*)pData->pCaptureClient);
24091 |             pData->pCaptureClient = NULL;
24092 |         }
24093 |         if (pData->pAudioClient) {
24094 |             ma_IAudioClient_Release((ma_IAudioClient*)pData->pAudioClient);
24095 |             pData->pAudioClient = NULL;
24096 |         }
24097 |
24098 |         if (errorMsg != NULL && errorMsg[0] != '\0') {
24099 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "%s\n", errorMsg);
24100 |         }
24101 |
24102 |         return result;
24103 |     } else {
24104 |         return MA_SUCCESS;
24105 |     }
24106 | }
24107 |
24108 | static ma_result ma_device_reinit_wasapi(ma_device* pDevice, ma_device_type deviceType)
24109 | {
24110 |     ma_device_init_internal_data_wasapi data;
24111 |     ma_result result;
24112 |
24113 |     MA_ASSERT(pDevice != NULL);
24114 |
24115 |     /* We only re-initialize the playback or capture device. Never a full-duplex device. */
24116 |     if (deviceType == ma_device_type_duplex) {
24117 |         return MA_INVALID_ARGS;
24118 |     }
24119 |
24120 |     /*
24121 |     Before reinitializing the device we need to free the previous audio clients.
24122 |
24123 |     There's a known memory leak here. We will be calling this from the routing change callback that
24124 |     is fired by WASAPI. If we attempt to release the IAudioClient we will deadlock. In my opinion
24125 |     this is a bug. I'm not sure what I need to do to handle this cleanly, but I think we'll probably
24126 |     need some system where we post an event, but delay the execution of it until the callback has
24127 |     returned. I'm not sure how to do this reliably, however. I have set up some infrastructure for
24128 |     a command thread which might be useful for this.
24129 |     */
24130 |
24131 |     if (deviceType == ma_device_type_capture || deviceType == ma_device_type_loopback) {
24132 |         if (pDevice->wasapi.pCaptureClient) {
24133 |             ma_IAudioCaptureClient_Release((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient);
24134 |             pDevice->wasapi.pCaptureClient = NULL;
24135 |         }
24136 |
24137 |         if (pDevice->wasapi.pAudioClientCapture) {

```

```

24138 |         /*ma_device_release_IAudioClient_service__wasapi(pDevice, ma_device_type_capture);*/
24139 |         pDevice->wasapi.pAudioClientCapture = NULL;
24140 |     }
24141 | }
24142 |
24143 | if (deviceType == ma_device_type_playback) {
24144 |     if (pDevice->wasapi.pRenderClient) {
24145 |         ma_IAudioRenderClient_Release((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient);
24146 |         pDevice->wasapi.pRenderClient = NULL;
24147 |     }
24148 |
24149 |     if (pDevice->wasapi.pAudioClientPlayback) {
24150 |         /*ma_device_release_IAudioClient_service__wasapi(pDevice, ma_device_type_playback);*/
24151 |         pDevice->wasapi.pAudioClientPlayback = NULL;
24152 |     }
24153 | }
24154 |
24155 |
24156 | if (deviceType == ma_device_type_playback) {
24157 |     data.formatIn = pDevice->playback.format;
24158 |     data.channelsIn = pDevice->playback.channels;
24159 |     MA_COPY_MEMORY(data.channelMapIn, pDevice->playback.channelMap,
> sizeof(pDevice->playback.channelMap));
24160 |     data.shareMode = pDevice->playback.shareMode;
24161 | } else {
24162 |     data.formatIn = pDevice->capture.format;
24163 |     data.channelsIn = pDevice->capture.channels;
24164 |     MA_COPY_MEMORY(data.channelMapIn, pDevice->capture.channelMap,
> sizeof(pDevice->capture.channelMap));
24165 |     data.shareMode = pDevice->capture.shareMode;
24166 | }
24167 |
24168 | data.sampleRateIn = pDevice->sampleRate;
24169 | data.periodSizeInFramesIn = pDevice->wasapi.originalPeriodSizeInFrames;
24170 | data.periodSizeInMillisecondsIn = pDevice->wasapi.originalPeriodSizeInMilliseconds;
24171 | data.periodsIn = pDevice->wasapi.originalPeriods;
24172 | data.performanceProfile = pDevice->wasapi.originalPerformanceProfile;
24173 | data.noAutoConvertSRC = pDevice->wasapi.noAutoConvertSRC;
24174 | data.noDefaultQualitySRC = pDevice->wasapi.noDefaultQualitySRC;
24175 | data.noHardwareOffloading = pDevice->wasapi.noHardwareOffloading;
24176 | data.loopbackProcessID = pDevice->wasapi.loopbackProcessID;
24177 | data.loopbackProcessExclude = pDevice->wasapi.loopbackProcessExclude;
24178 | result = ma_device_init_internal__wasapi(pDevice->pContext, deviceType, NULL, &data);
24179 | if (result != MA_SUCCESS) {
24180 |     return result;
24181 | }
24182 |
24183 | /* At this point we have some new objects ready to go. We need to uninitialized the previous ones
> and then set the new ones. */
24184 | if (deviceType == ma_device_type_capture || deviceType == ma_device_type_loopback) {
24185 |     pDevice->wasapi.pAudioClientCapture = data.pAudioClient;
24186 |     pDevice->wasapi.pCaptureClient = data.pCaptureClient;
24187 |
24188 |     pDevice->capture.internalFormat = data.formatOut;
24189 |     pDevice->capture.internalChannels = data.channelsOut;
24190 |     pDevice->capture.internalSampleRate = data.sampleRateOut;
24191 |     MA_COPY_MEMORY(pDevice->capture.internalChannelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
24192 |     pDevice->capture.internalPeriodSizeInFrames = data.periodSizeInFramesOut;
24193 |     pDevice->capture.internalPeriods = data.periodsOut;
24194 |     ma_strcpy_s(pDevice->capture.name, sizeof(pDevice->capture.name), data.deviceName);
24195 |
24196 |     ma_IAudioClient_SetEventHandle((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture,
> (HANDLE)pDevice->wasapi.hEventCapture);
24197 |
24198 |     pDevice->wasapi.periodSizeInFramesCapture = data.periodSizeInFramesOut;
24199 |     ma_IAudioClient_GetBufferSize((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture,
> &pDevice->wasapi.actualBufferSizeInFramesCapture);
24200 |
24201 |     /* We must always have a valid ID. */
24202 |     ma_strcpy_s_WCHAR(pDevice->capture.id.wasapi, sizeof(pDevice->capture.id.wasapi),
> data.id.wasapi);
24203 | }
24204 |
24205 | if (deviceType == ma_device_type_playback) {
24206 |     pDevice->wasapi.pAudioClientPlayback = data.pAudioClient;
24207 |     pDevice->wasapi.pRenderClient = data.pRenderClient;
24208 |
24209 |     pDevice->playback.internalFormat = data.formatOut;

```



```

24210 |         pDevice->playback.internalChannels          = data.channelsOut;
24211 |         pDevice->playback.internalSampleRate        = data.sampleRateOut;
24212 |         MA_COPY_MEMORY(pDevice->playback.internalChannelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
24213 |         pDevice->playback.internalPeriodSizeInFrames = data.periodSizeInFramesOut;
24214 |         pDevice->playback.internalPeriods           = data.periodsOut;
24215 |         ma_strcpy_s(pDevice->playback.name, sizeof(pDevice->playback.name), data.deviceName);
24216 |
24217 |         ma_IAudioClient_SetEventHandle((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback,
> (HANDLE)pDevice->wasapi.hEventPlayback);
24218 |
24219 |         pDevice->wasapi.periodSizeInFramesPlayback = data.periodSizeInFramesOut;
24220 |         ma_IAudioClient_GetBufferSize((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback,
> &pDevice->wasapi.actualBufferSizeInFramesPlayback);
24221 |
24222 |         /* We must always have a valid ID because rerouting will look at it. */
24223 |         ma_strcpy_s_WCHAR(pDevice->playback.id.wasapi, sizeof(pDevice->playback.id.wasapi),
> data.id.wasapi);
24224 |     }
24225 |
24226 |     return MA_SUCCESS;
24227 | }
24228 |
24229 | static ma_result ma_device_init_wasapi(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
24230 | {
24231 |     ma_result result = MA_SUCCESS;
24232 |
24233 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
24234 |     HRESULT hr;
24235 |     ma_IMMDeviceEnumerator* pDeviceEnumerator;
24236 |     #endif
24237 |
24238 |     MA_ASSERT(pDevice != NULL);
24239 |
24240 |     MA_ZERO_OBJECT(&pDevice->wasapi);
24241 |     pDevice->wasapi.usage          = pConfig->wasapi.usage;
24242 |     pDevice->wasapi.noAutoConvertSRC = pConfig->wasapi.noAutoConvertSRC;
24243 |     pDevice->wasapi.noDefaultQualitySRC = pConfig->wasapi.noDefaultQualitySRC;
24244 |     pDevice->wasapi.noHardwareOffloading = pConfig->wasapi.noHardwareOffloading;
24245 |     pDevice->wasapi.loopbackProcessID = pConfig->wasapi.loopbackProcessID;
24246 |     pDevice->wasapi.loopbackProcessExclude = pConfig->wasapi.loopbackProcessExclude;
24247 |
24248 |     /* Exclusive mode is not allowed with loopback. */
24249 |     if (pConfig->deviceType == ma_device_type_loopback && pConfig->playback.shareMode ==
> ma_share_mode_exclusive) {
24250 |         return MA_INVALID_DEVICE_CONFIG;
24251 |     }
24252 |
24253 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex || pConfig->deviceType == ma_device_type_loopback) {
24254 |         ma_device_init_internal_data_wasapi data;
24255 |         data.formatIn          = pDescriptorCapture->format;
24256 |         data.channelsIn        = pDescriptorCapture->channels;
24257 |         data.sampleRateIn      = pDescriptorCapture->sampleRate;
24258 |         MA_COPY_MEMORY(data.channelMapIn, pDescriptorCapture->channelMap,
> sizeof(pDescriptorCapture->channelMap));
24259 |         data.periodSizeInFramesIn = pDescriptorCapture->periodSizeInFrames;
24260 |         data.periodSizeInMillisecondsIn = pDescriptorCapture->periodSizeInMilliseconds;
24261 |         data.periodsIn          = pDescriptorCapture->periodCount;
24262 |         data.shareMode          = pDescriptorCapture->shareMode;
24263 |         data.performanceProfile = pConfig->performanceProfile;
24264 |         data.noAutoConvertSRC   = pConfig->wasapi.noAutoConvertSRC;
24265 |         data.noDefaultQualitySRC = pConfig->wasapi.noDefaultQualitySRC;
24266 |         data.noHardwareOffloading = pConfig->wasapi.noHardwareOffloading;
24267 |         data.loopbackProcessID   = pConfig->wasapi.loopbackProcessID;
24268 |         data.loopbackProcessExclude = pConfig->wasapi.loopbackProcessExclude;
24269 |
24270 |         result = ma_device_init_internal_wasapi(pDevice->pContext, (pConfig->deviceType ==
> ma_device_type_loopback) ? ma_device_type_loopback : ma_device_type_capture,
> pDescriptorCapture->pDeviceID, &data);
24271 |         if (result != MA_SUCCESS) {
24272 |             return result;
24273 |         }
24274 |
24275 |         pDevice->wasapi.pAudioClientCapture          = data.pAudioClient;
24276 |         pDevice->wasapi.pCaptureClient               = data.pCaptureClient;
24277 |         pDevice->wasapi.originalPeriodSizeInMilliseconds =
> pDescriptorCapture->periodSizeInMilliseconds;

```

```

24278 |         pDevice->wasapi.originalPeriodSizeInFrames      = pDescriptorCapture->periodSizeInFrames;
24279 |         pDevice->wasapi.originalPeriods                  = pDescriptorCapture->periodCount;
24280 |         pDevice->wasapi.originalPerformanceProfile       = pConfig->performanceProfile;
24281 |
24282 |         /*
24283 |         The event for capture needs to be manual reset for the same reason as playback. We keep the
> initial state set to unsignaled,
24284 |         however, because we want to block until we actually have something for the first call to
> ma_device_read().
24285 |         */
24286 |         pDevice->wasapi.hEventCapture = (ma_handle)CreateEventA(NULL, FALSE, FALSE, NULL); /* Auto
> reset, unsignaled by default. */
24287 |         if (pDevice->wasapi.hEventCapture == NULL) {
24288 |             result = ma_result_from_GetLastError(GetLastError());
24289 |
24290 |             if (pDevice->wasapi.pCaptureClient != NULL) {
24291 |
> ma_IAudioCaptureClient_Release((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient);
24292 |             pDevice->wasapi.pCaptureClient = NULL;
24293 |         }
24294 |         if (pDevice->wasapi.pAudioClientCapture != NULL) {
24295 |             ma_IAudioClient_Release((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
24296 |             pDevice->wasapi.pAudioClientCapture = NULL;
24297 |         }
24298 |
24299 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> event for capture.");
24300 |         return result;
24301 |     }
24302 |     ma_IAudioClient_SetEventHandle((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture,
> (HANDLE)pDevice->wasapi.hEventCapture);
24303 |
24304 |     pDevice->wasapi.periodSizeInFramesCapture = data.periodSizeInFramesOut;
24305 |     ma_IAudioClient_GetBufferSize((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture,
> &pDevice->wasapi.actualBufferSizeInFramesCapture);
24306 |
24307 |     /* We must always have a valid ID. */
24308 |     ma_strcpy_s_WCHAR(pDevice->capture.id.wasapi, sizeof(pDevice->capture.id.wasapi),
> data.id.wasapi);
24309 |
24310 |     /* The descriptor needs to be updated with actual values. */
24311 |     pDescriptorCapture->format      = data.formatOut;
24312 |     pDescriptorCapture->channels     = data.channelsOut;
24313 |     pDescriptorCapture->sampleRate  = data.sampleRateOut;
24314 |     MA_COPY_MEMORY(pDescriptorCapture->channelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
24315 |     pDescriptorCapture->periodSizeInFrames = data.periodSizeInFramesOut;
24316 |     pDescriptorCapture->periodCount      = data.periodsOut;
24317 | }
24318 |
24319 | if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
24320 |     ma_device_init_internal_data_wasapi data;
24321 |     data.formatIn      = pDescriptorPlayback->format;
24322 |     data.channelsIn    = pDescriptorPlayback->channels;
24323 |     data.sampleRateIn  = pDescriptorPlayback->sampleRate;
24324 |     MA_COPY_MEMORY(data.channelMapIn, pDescriptorPlayback->channelMap,
> sizeof(pDescriptorPlayback->channelMap));
24325 |     data.periodSizeInFramesIn = pDescriptorPlayback->periodSizeInFrames;
24326 |     data.periodSizeInMillisecondsIn = pDescriptorPlayback->periodSizeInMilliseconds;
24327 |     data.periodsIn      = pDescriptorPlayback->periodCount;
24328 |     data.shareMode      = pDescriptorPlayback->shareMode;
24329 |     data.performanceProfile = pConfig->performanceProfile;
24330 |     data.noAutoConvertSRC = pConfig->wasapi.noAutoConvertSRC;
24331 |     data.noDefaultQualitySRC = pConfig->wasapi.noDefaultQualitySRC;
24332 |     data.noHardwareOffloading = pConfig->wasapi.noHardwareOffloading;
24333 |     data.loopbackProcessID = pConfig->wasapi.loopbackProcessID;
24334 |     data.loopbackProcessExclude = pConfig->wasapi.loopbackProcessExclude;
24335 |
24336 |     result = ma_device_init_internal__wasapi(pDevice->pContext, ma_device_type_playback,
> pDescriptorPlayback->pDeviceID, &data);
24337 |     if (result != MA_SUCCESS) {
24338 |         if (pConfig->deviceType == ma_device_type_duplex) {
24339 |             if (pDevice->wasapi.pCaptureClient != NULL) {
24340 |
> ma_IAudioCaptureClient_Release((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient);
24341 |             pDevice->wasapi.pCaptureClient = NULL;
24342 |         }
24343 |         if (pDevice->wasapi.pAudioClientCapture != NULL) {

```

```

24344 |         ma_IAudioClient_Release((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
24345 |         pDevice->wasapi.pAudioClientCapture = NULL;
24346 |     }
24347 |
24348 |     CloseHandle((HANDLE)pDevice->wasapi.hEventCapture);
24349 |     pDevice->wasapi.hEventCapture = NULL;
24350 | }
24351 |     return result;
24352 | }
24353 |
24354 |     pDevice->wasapi.pAudioClientPlayback = data.pAudioClient;
24355 |     pDevice->wasapi.pRenderClient = data.pRenderClient;
24356 |     pDevice->wasapi.originalPeriodSizeInMilliseconds =
> pDescriptorPlayback->periodSizeInMilliseconds;
24357 |     pDevice->wasapi.originalPeriodSizeInFrames = pDescriptorPlayback->periodSizeInFrames;
24358 |     pDevice->wasapi.originalPeriods = pDescriptorPlayback->periodCount;
24359 |     pDevice->wasapi.originalPerformanceProfile = pConfig->performanceProfile;
24360 |
24361 |     /*
24362 |     The event for playback is needs to be manual reset because we want to explicitly control the
> fact that it becomes signalled
24363 |     only after the whole available space has been filled, never before.
24364 |
24365 |     The playback event also needs to be initially set to a signaled state so that the first call
> to ma_device_write() is able
24366 |     to get passed WaitForMultipleObjects().
24367 |     */
24368 |     pDevice->wasapi.hEventPlayback = (ma_handle)CreateEventA(NULL, FALSE, TRUE, NULL); /* Auto
> reset, signaled by default. */
24369 |     if (pDevice->wasapi.hEventPlayback == NULL) {
24370 |         result = ma_result_from_GetLastError(GetLastError());
24371 |
24372 |         if (pConfig->deviceType == ma_device_type_duplex) {
24373 |             if (pDevice->wasapi.pCaptureClient != NULL) {
24374 |
> ma_IAudioCaptureClient_Release((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient);
24375 |                 pDevice->wasapi.pCaptureClient = NULL;
24376 |             }
24377 |             if (pDevice->wasapi.pAudioClientCapture != NULL) {
24378 |                 ma_IAudioClient_Release((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
24379 |                 pDevice->wasapi.pAudioClientCapture = NULL;
24380 |             }
24381 |
24382 |             CloseHandle((HANDLE)pDevice->wasapi.hEventCapture);
24383 |             pDevice->wasapi.hEventCapture = NULL;
24384 |         }
24385 |
24386 |         if (pDevice->wasapi.pRenderClient != NULL) {
24387 |
> ma_IAudioRenderClient_Release((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient);
24388 |             pDevice->wasapi.pRenderClient = NULL;
24389 |         }
24390 |         if (pDevice->wasapi.pAudioClientPlayback != NULL) {
24391 |             ma_IAudioClient_Release((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback);
24392 |             pDevice->wasapi.pAudioClientPlayback = NULL;
24393 |         }
24394 |
24395 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> event for playback.");
24396 |         return result;
24397 |     }
24398 |     ma_IAudioClient_SetEventHandle((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback,
> (HANDLE)pDevice->wasapi.hEventPlayback);
24399 |
24400 |     pDevice->wasapi.periodSizeInFramesPlayback = data.periodSizeInFramesOut;
24401 |     ma_IAudioClient_GetBufferSize((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback,
> &pDevice->wasapi.actualBufferSizeInFramesPlayback);
24402 |
24403 |     /* We must always have a valid ID because rerouting will look at it. */
24404 |     ma_strcpy_s_WCHAR(pDevice->playback.id.wasapi, sizeof(pDevice->playback.id.wasapi),
> data.id.wasapi);
24405 |
24406 |     /* The descriptor needs to be updated with actual values. */
24407 |     pDescriptorPlayback->format = data.formatOut;
24408 |     pDescriptorPlayback->channels = data.channelsOut;
24409 |     pDescriptorPlayback->sampleRate = data.sampleRateOut;
24410 |     MA_COPY_MEMORY(pDescriptorPlayback->channelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
24411 |     pDescriptorPlayback->periodSizeInFrames = data.periodSizeInFramesOut;

```

```

24412 |         pDescriptorPlayback->periodCount      = data.periodsOut;
24413 |     }
24414 |
24415 |     /*
24416 |     We need to register a notification client to detect when the device has been disabled, unplugged
> or re-routed (when the default device changes). When
24417 |     we are connecting to the default device we want to do automatic stream routing when the device
> is disabled or unplugged. Otherwise we want to just
24418 |     stop the device outright and let the application handle it.
24419 |     */
24420 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
24421 |         if (pConfig->wasapi.noAutoStreamRouting == MA_FALSE) {
24422 |             if ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex || pConfig->deviceType == ma_device_type_loopback) &&
> pConfig->capture.pDeviceID == NULL) {
24423 |                 pDevice->wasapi.allowCaptureAutoStreamRouting = MA_TRUE;
24424 |             }
24425 |             if ((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pConfig->playback.pDeviceID == NULL) {
24426 |                 pDevice->wasapi.allowPlaybackAutoStreamRouting = MA_TRUE;
24427 |             }
24428 |         }
24429 |
24430 |         ma_mutex_init(&pDevice->wasapi.rerouteLock);
24431 |
24432 |         hr = ma_CoCreateInstance(pDevice->pContext, &MA_CLSID_MMDeviceEnumerator, NULL, CLSCTX_ALL,
> &MA_IID_IMMDeviceEnumerator, (void*)&pDeviceEnumerator);
24433 |         if (FAILED(hr)) {
24434 |             ma_device_uninit_wasapi(pDevice);
24435 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to create
> device enumerator.");
24436 |             return ma_result_from_HRESULT(hr);
24437 |         }
24438 |
24439 |         pDevice->wasapi.notificationClient.lpVtbl = (void*)&g_maNotificationClientVtbl;
24440 |         pDevice->wasapi.notificationClient.counter = 1;
24441 |         pDevice->wasapi.notificationClient.pDevice = pDevice;
24442 |
24443 |         hr = pDeviceEnumerator->lpVtbl->RegisterEndpointNotificationCallback(pDeviceEnumerator,
> &pDevice->wasapi.notificationClient);
24444 |         if (SUCCEEDED(hr)) {
24445 |             pDevice->wasapi.pDeviceEnumerator = (ma_ptr)pDeviceEnumerator;
24446 |         } else {
24447 |             /* Not the end of the world if we fail to register the notification callback. We just won't
> support automatic stream routing. */
24448 |             ma_IMMDeviceEnumerator_Release(pDeviceEnumerator);
24449 |         }
24450 |     #endif
24451 |
24452 |     ma_atomic_bool32_set(&pDevice->wasapi.isStartedCapture, MA_FALSE);
24453 |     ma_atomic_bool32_set(&pDevice->wasapi.isStartedPlayback, MA_FALSE);
24454 |
24455 |     return MA_SUCCESS;
24456 | }
24457 |
24458 | static ma_result ma_device_get_available_frames_wasapi(ma_device* pDevice, ma_IAudioClient*
> pAudioClient, ma_uint32* pFrameCount)
24459 | {
24460 |     ma_uint32 paddingFramesCount;
24461 |     HRESULT hr;
24462 |     ma_share_mode shareMode;
24463 |
24464 |     MA_ASSERT(pDevice != NULL);
24465 |     MA_ASSERT(pFrameCount != NULL);
24466 |
24467 |     *pFrameCount = 0;
24468 |
24469 |     if ((ma_ptr)pAudioClient != pDevice->wasapi.pAudioClientPlayback && (ma_ptr)pAudioClient !=
> pDevice->wasapi.pAudioClientCapture) {
24470 |         return MA_INVALID_OPERATION;
24471 |     }
24472 |
24473 |     /*
24474 |     I've had a report that GetCurrentPadding() is returning a frame count of 0 which is preventing
24475 |     higher level function calls from doing anything because it thinks nothing is available. I have
24476 |     taken a look at the documentation and it looks like this is unnecessary in exclusive mode.
24477 |
24478 |     From Microsoft's documentation:
24479 |

```

```

24480 |         For an exclusive-mode rendering or capture stream that was initialized with the
24481 |         AUDCLNT_STREAMFLAGS_EVENTCALLBACK flag, the client typically has no use for the padding
24482 |         value reported by GetCurrentPadding. Instead, the client accesses an entire buffer during
24483 |         each processing pass.
24484 |
24485 |         Considering this, I'm going to skip GetCurrentPadding() for exclusive mode and just report the
24486 |         entire buffer. This depends on the caller making sure they wait on the event handler.
24487 |         */
24488 |         shareMode = ((ma_ptr)pAudioClient == pDevice->wasapi.pAudioClientPlayback) ?
> pDevice->playback.shareMode : pDevice->capture.shareMode;
24489 |         if (shareMode == ma_share_mode_shared) {
24490 |             /* Shared mode. */
24491 |             hr = ma_IAudioClient_GetCurrentPadding(pAudioClient, &paddingFramesCount);
24492 |             if (FAILED(hr)) {
24493 |                 return ma_result_from_HRESULT(hr);
24494 |             }
24495 |
24496 |             if ((ma_ptr)pAudioClient == pDevice->wasapi.pAudioClientPlayback) {
24497 |                 *pFrameCount = pDevice->wasapi.actualBufferSizeInFramesPlayback - paddingFramesCount;
24498 |             } else {
24499 |                 *pFrameCount = paddingFramesCount;
24500 |             }
24501 |         } else {
24502 |             /* Exclusive mode. */
24503 |             if ((ma_ptr)pAudioClient == pDevice->wasapi.pAudioClientPlayback) {
24504 |                 *pFrameCount = pDevice->wasapi.actualBufferSizeInFramesPlayback;
24505 |             } else {
24506 |                 *pFrameCount = pDevice->wasapi.actualBufferSizeInFramesCapture;
24507 |             }
24508 |         }
24509 |
24510 |         return MA_SUCCESS;
24511 |     }
24512 |
24513 |
24514 | static ma_result ma_device_reroute__wasapi(ma_device* pDevice, ma_device_type deviceType)
24515 | {
24516 |     ma_result result;
24517 |
24518 |     if (deviceType == ma_device_type_duplex) {
24519 |         return MA_INVALID_ARGS;
24520 |     }
24521 |
24522 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "=== CHANGING DEVICE ===\n");
24523 |
24524 |     result = ma_device_reinit__wasapi(pDevice, deviceType);
24525 |     if (result != MA_SUCCESS) {
24526 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[WASAPI] Reinitializing
> device after route change failed.\n");
24527 |         return result;
24528 |     }
24529 |
24530 |     ma_device__post_init_setup(pDevice, deviceType);
24531 |     ma_device__on_notification_rerouted(pDevice);
24532 |
24533 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "=== DEVICE CHANGED ===\n");
24534 |
24535 |     return MA_SUCCESS;
24536 | }
24537 |
24538 | static ma_result ma_device_start__wasapi_nolock(ma_device* pDevice)
24539 | {
24540 |     HRESULT hr;
24541 |
24542 |     if (pDevice->pContext->wasapi.hAvrt) {
24543 |         const char* pTaskName = ma_to_usage_string__wasapi(pDevice->wasapi.usage);
24544 |         if (pTaskName) {
24545 |             DWORD idx = 0;
24546 |             pDevice->wasapi.hAvrtHandle = (ma_handle)((MA_PFN_AvSetMmThreadCharacteristicsA)pDevice-
> pContext->wasapi.AvSetMmThreadCharacteristicsA)(pTaskName, &idx);
24547 |         }
24548 |     }
24549 |
24550 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex ||
> pDevice->type == ma_device_type_loopback) {
24551 |         hr = ma_IAudioClient_Start((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
24552 |         if (FAILED(hr)) {
24553 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to start
> internal capture device. HRESULT = %d.", (int)hr);

```

```

24554 |         return ma_result_from_HRESULT(hr);
24555 |     }
24556 |
24557 |     ma_atomic_bool32_set(&pDevice->wasapi.isStartedCapture, MA_TRUE);
24558 | }
24559 |
24560 | if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
24561 |     hr = ma_IAudioClient_Start((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback);
24562 |     if (FAILED(hr)) {
24563 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to start
> internal playback device. HRESULT = %d.", (int)hr);
24564 |         return ma_result_from_HRESULT(hr);
24565 |     }
24566 |
24567 |     ma_atomic_bool32_set(&pDevice->wasapi.isStartedPlayback, MA_TRUE);
24568 | }
24569 |
24570 | return MA_SUCCESS;
24571 | }
24572 |
24573 | static ma_result ma_device_start_wasapi(ma_device* pDevice)
24574 | {
24575 |     ma_result result;
24576 |
24577 |     MA_ASSERT(pDevice != NULL);
24578 |
24579 |     /* Wait for any rerouting to finish before attempting to start the device. */
24580 |     ma_mutex_lock(&pDevice->wasapi.rerouteLock);
24581 |     {
24582 |         result = ma_device_start_wasapi_nolock(pDevice);
24583 |     }
24584 |     ma_mutex_unlock(&pDevice->wasapi.rerouteLock);
24585 |
24586 |     return result;
24587 | }
24588 |
24589 | static ma_result ma_device_stop_wasapi_nolock(ma_device* pDevice)
24590 | {
24591 |     ma_result result;
24592 |     HRESULT hr;
24593 |
24594 |     MA_ASSERT(pDevice != NULL);
24595 |
24596 |     if (pDevice->wasapi.hAvrtHandle) {
24597 |         ((MA_PFN_AvRevertMmThreadCharacteristics)pDevice->pContext->wasapi.AvRevertMmThreadcharacter
> istics)((HANDLE)pDevice->wasapi.hAvrtHandle);
24598 |         pDevice->wasapi.hAvrtHandle = NULL;
24599 |     }
24600 |
24601 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex ||
> pDevice->type == ma_device_type_loopback) {
24602 |         /* If we have a mapped buffer we need to release it. */
24603 |         if (pDevice->wasapi.pMappedBufferCapture != NULL) {
24604 |             ma_IAudioCaptureClient_ReleaseBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> pDevice->wasapi.mappedBufferCaptureCap);
24605 |             pDevice->wasapi.pMappedBufferCapture = NULL;
24606 |             pDevice->wasapi.mappedBufferCaptureCap = 0;
24607 |             pDevice->wasapi.mappedBufferCaptureLen = 0;
24608 |         }
24609 |
24610 |         hr = ma_IAudioClient_Stop((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
24611 |         if (FAILED(hr)) {
24612 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to stop
> internal capture device.");
24613 |             return ma_result_from_HRESULT(hr);
24614 |         }
24615 |
24616 |         /* The audio client needs to be reset otherwise restarting will fail. */
24617 |         hr = ma_IAudioClient_Reset((ma_IAudioClient*)pDevice->wasapi.pAudioClientCapture);
24618 |         if (FAILED(hr)) {
24619 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to reset
> internal capture device.");
24620 |             return ma_result_from_HRESULT(hr);
24621 |         }
24622 |
24623 |         ma_atomic_bool32_set(&pDevice->wasapi.isStartedCapture, MA_FALSE);
24624 |     }
24625 | }

```



```

24626 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
24627 |         if (pDevice->wasapi.pMappedBufferPlayback != NULL) {
24628 |             ma_silence_pcm_frames(
24629 |                 ma_offset_pcm_frames_ptr(pDevice->wasapi.pMappedBufferPlayback,
> pDevice->wasapi.mappedBufferPlaybackLen, pDevice->playback.internalFormat,
> pDevice->playback.internalChannels),
24630 |                 pDevice->wasapi.mappedBufferPlaybackCap - pDevice->wasapi.mappedBufferPlaybackLen,
24631 |                 pDevice->playback.internalFormat, pDevice->playback.internalChannels
24632 |             );
24633 |
> ma_IAudioRenderClient_ReleaseBuffer((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient,
> pDevice->wasapi.mappedBufferPlaybackCap, 0);
24634 |         pDevice->wasapi.pMappedBufferPlayback = NULL;
24635 |         pDevice->wasapi.mappedBufferPlaybackCap = 0;
24636 |         pDevice->wasapi.mappedBufferPlaybackLen = 0;
24637 |     }
24638 |
24639 |     /*
24640 |     The buffer needs to be drained before stopping the device. Not doing this will result in the
> last few frames not getting output to
24641 |     the speakers. This is a problem for very short sounds because it'll result in a significant
> portion of it not getting played.
24642 |     */
24643 |     if (ma_atomic_bool32_get(&pDevice->wasapi.isStartedPlayback)) {
24644 |         /* We need to make sure we put a timeout here or else we'll risk getting stuck in a
> deadlock in some cases. */
24645 |         DWORD waitTime = (pDevice->wasapi.actualBufferSizeInFramesPlayback * 1000) /
> pDevice->playback.internalSampleRate;
24646 |
24647 |         if (pDevice->playback.shareMode == ma_share_mode_exclusive) {
24648 |             WaitForSingleObject((HANDLE)pDevice->wasapi.hEventPlayback, waitTime);
24649 |         } else {
24650 |             ma_uint32 prevFramesAvailablePlayback = (ma_uint32)-1;
24651 |             ma_uint32 framesAvailablePlayback;
24652 |             for (;;) {
24653 |                 result = ma_device__get_available_frames_wasapi(pDevice,
> (ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback, &framesAvailablePlayback);
24654 |                 if (result != MA_SUCCESS) {
24655 |                     break;
24656 |                 }
24657 |
24658 |                 if (framesAvailablePlayback >= pDevice->wasapi.actualBufferSizeInFramesPlayback)
> {
24659 |                     break;
24660 |                 }
24661 |
24662 |                 /*
24663 |                 Just a safety check to avoid an infinite loop. If this iteration results in a
> situation where the number of available frames
24664 |                 has not changed, get out of the loop. I don't think this should ever happen, but
> I think it's nice to have just in case.
24665 |                 */
24666 |                 if (framesAvailablePlayback == prevFramesAvailablePlayback) {
24667 |                     break;
24668 |                 }
24669 |                 prevFramesAvailablePlayback = framesAvailablePlayback;
24670 |
24671 |                 ResetEvent((HANDLE)pDevice->wasapi.hEventPlayback); /* Manual reset. */
24672 |                 WaitForSingleObject((HANDLE)pDevice->wasapi.hEventPlayback, waitTime);
24673 |             }
24674 |         }
24675 |     }
24676 |
24677 |     hr = ma_IAudioClient_Stop((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback);
24678 |     if (FAILED(hr)) {
24679 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to stop
> internal playback device.");
24680 |         return ma_result_from_HRESULT(hr);
24681 |     }
24682 |
24683 |     /* The audio client needs to be reset otherwise restarting will fail. */
24684 |     {
24685 |         ma_int32 retries = 5;
24686 |
24687 |         while ((hr =
> ma_IAudioClient_Reset((ma_IAudioClient*)pDevice->wasapi.pAudioClientPlayback)) ==
> MA_AUDCLNT_E_BUFFER_OPERATION_PENDING && retries > 0) {
24688 |             ma_sleep(10);
24689 |             retries -= 1;

```



```

24690 |     }
24691 | }
24692 |
24693 |     if (FAILED(hr)) {
24694 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to reset
> internal playback device.");
24695 |         return ma_result_from_HRESULT(hr);
24696 |     }
24697 |
24698 |     ma_atomic_bool32_set(&pDevice->wasapi.isStartedPlayback, MA_FALSE);
24699 | }
24700 |
24701 |     return MA_SUCCESS;
24702 | }
24703 |
24704 | static ma_result ma_device_stop_wasapi(ma_device* pDevice)
24705 | {
24706 |     ma_result result;
24707 |
24708 |     MA_ASSERT(pDevice != NULL);
24709 |
24710 |     /* Wait for any rerouting to finish before attempting to stop the device. */
24711 |     ma_mutex_lock(&pDevice->wasapi.rerouteLock);
24712 |     {
24713 |         result = ma_device_stop_wasapi_nolock(pDevice);
24714 |     }
24715 |     ma_mutex_unlock(&pDevice->wasapi.rerouteLock);
24716 |
24717 |     return result;
24718 | }
24719 |
24720 |
24721 | #ifndef MA_WASAPI_WAIT_TIMEOUT_MILLISECONDS
24722 | #define MA_WASAPI_WAIT_TIMEOUT_MILLISECONDS 5000
24723 | #endif
24724 |
24725 | static ma_result ma_device_read_wasapi(ma_device* pDevice, void* pFrames, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
24726 | {
24727 |     ma_result result = MA_SUCCESS;
24728 |     ma_uint32 totalFramesProcessed = 0;
24729 |
24730 |     /*
24731 |     When reading, we need to get a buffer and process all of it before releasing it. Because the
24732 |     frame count (frameCount) can be different to the size of the buffer, we'll need to cache the
24733 |     pointer to the buffer.
24734 |     */
24735 |
24736 |     /* Keep running until we've processed the requested number of frames. */
24737 |     while (ma_device_get_state(pDevice) == ma_device_state_started && totalFramesProcessed <
> frameCount) {
24738 |         ma_uint32 framesRemaining = frameCount - totalFramesProcessed;
24739 |
24740 |         /* If we have a mapped data buffer, consume that first. */
24741 |         if (pDevice->wasapi.pMappedBufferCapture != NULL) {
24742 |             /* We have a cached data pointer so consume that before grabbing another one from
> WASAPI. */
24743 |             ma_uint32 framesToProcessNow = framesRemaining;
24744 |             if (framesToProcessNow > pDevice->wasapi.mappedBufferCaptureLen) {
24745 |                 framesToProcessNow = pDevice->wasapi.mappedBufferCaptureLen;
24746 |             }
24747 |
24748 |             /* Now just copy the data over to the output buffer. */
24749 |             ma_copy_pcm_frames(
24750 |                 ma_offset_pcm_frames_ptr(pFrames, totalFramesProcessed,
> pDevice->capture.internalFormat, pDevice->capture.internalChannels),
24751 |                 ma_offset_pcm_frames_const_ptr(pDevice->wasapi.pMappedBufferCapture,
> pDevice->wasapi.mappedBufferCaptureCap - pDevice->wasapi.mappedBufferCaptureLen,
> pDevice->capture.internalFormat, pDevice->capture.internalChannels),
24752 |                 framesToProcessNow,
24753 |                 pDevice->capture.internalFormat, pDevice->capture.internalChannels
24754 |             );
24755 |
24756 |             totalFramesProcessed += framesToProcessNow;
24757 |             pDevice->wasapi.mappedBufferCaptureLen -= framesToProcessNow;
24758 |
24759 |             /* If the data buffer has been fully consumed we need to release it. */
24760 |             if (pDevice->wasapi.mappedBufferCaptureLen == 0) {
24761 |

```

```

> ma_IAudioCaptureClient_ReleaseBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> pDevice->wasapi.mappedBufferCaptureCap);
24762 |         pDevice->wasapi.pMappedBufferCapture = NULL;
24763 |         pDevice->wasapi.mappedBufferCaptureCap = 0;
24764 |     }
24765 | } else {
24766 |     /* We don't have any cached data pointer, so grab another one. */
24767 |     HRESULT hr;
24768 |     DWORD flags = 0;
24769 |
24770 |     /* First just ask WASAPI for a data buffer. If it's not available, we'll wait for more.
> */
24771 |     hr =
> ma_IAudioCaptureClient_GetBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> (BYTE**)&pDevice->wasapi.pMappedBufferCapture, &pDevice->wasapi.mappedBufferCaptureCap, &flags,
> NULL, NULL);
24772 |     if (hr == S_OK) {
24773 |         /* We got a data buffer. Continue to the next loop iteration which will then read
> from the mapped pointer. */
24774 |         pDevice->wasapi.mappedBufferCaptureLen = pDevice->wasapi.mappedBufferCaptureCap;
24775 |
24776 |         /*
24777 |         There have been reports that indicate that at times the
> AUDCLNT_BUFFERFLAGS_DATA_DISCONTINUITY is reported for every
24778 |         call to IAudioCaptureClient_GetBuffer() above which results in spamming of the debug
> messages below. To partially
24779 |         work around this, I'm only outputting these messages when MA_DEBUG_OUTPUT is
> explicitly defined. The better solution
24780 |         would be to figure out why the flag is always getting reported.
24781 |         */
24782 |         #if defined(MA_DEBUG_OUTPUT)
24783 |         {
24784 |             if (flags != 0) {
24785 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[WASAPI]
> Capture Flags: %ld\n", flags);
24786 |
24787 |                 if ((flags & MA_AUDCLNT_BUFFERFLAGS_DATA_DISCONTINUITY) != 0) {
24788 |                     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[WASAPI]
> Data discontinuity (possible overrun). Attempting recovery. mappedBufferCaptureCap=%d\n",
> pDevice->wasapi.mappedBufferCaptureCap);
24789 |                 }
24790 |             }
24791 |         }
24792 |     #endif
24793 |
24794 |     /* Overrun detection. */
24795 |     if ((flags & MA_AUDCLNT_BUFFERFLAGS_DATA_DISCONTINUITY) != 0) {
24796 |         /* Glitched. Probably due to an overrun. */
24797 |
24798 |         /*
24799 |         If we got an overrun it probably means we're straddling the end of the buffer.
> In normal capture
24800 |         mode this is the fault of the client application because they're responsible for
> ensuring data is
24801 |         processed fast enough. In duplex mode, however, the processing of audio is tied
> to the playback
24802 |         device, so this can possibly be the result of a timing de-sync.
24803 |
24804 |         In capture mode we're not going to do any kind of recovery because the real fix
> is for the client
24805 |         application to process faster. In duplex mode, we'll treat this as a desync and
> reset the buffers
24806 |         to prevent a never-ending sequence of glitches due to straddling the end of the
> buffer.
24807 |         */
24808 |         if (pDevice->type == ma_device_type_duplex) {
24809 |             /*
24810 |             Experiment:
24811 |
24812 |             If we empty out the *entire* buffer we may end up putting ourselves into an
> underrun position
24813 |             which isn't really any better than the overrun we're probably in right now.
> Instead we'll just
24814 |             empty out about half.
24815 |             */
24816 |             ma_uint32 i;
24817 |             ma_uint32 periodCount = (pDevice->wasapi.actualBufferSizeInFramesCapture /
> pDevice->wasapi.periodSizeInFramesCapture);
24818 |             ma_uint32 iterationCount = periodCount / 2;

```

```

24819 |             if ((periodCount % 2) > 0) {
24820 |                 iterationCount += 1;
24821 |             }
24822 |
24823 |             for (i = 0; i < iterationCount; i += 1) {
24824 |                 hr =
> ma_IAudioCaptureClient_ReleaseBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> pDevice->wasapi.mappedBufferCaptureCap);
24825 |                 if (FAILED(hr)) {
24826 |                     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG,
> "[WASAPI] Data discontinuity recovery: IAudioCaptureClient_ReleaseBuffer() failed with %ld.\n", hr);
24827 |                     break;
24828 |                 }
24829 |
24830 |                 flags = 0;
24831 |                 hr =
> ma_IAudioCaptureClient_GetBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> (BYTE**)&pDevice->wasapi.pMappedBufferCapture, &pDevice->wasapi.mappedBufferCaptureCap, &flags,
> NULL, NULL);
24832 |                 if (hr == MA_AUDCLNT_S_BUFFER_EMPTY || FAILED(hr)) {
24833 |                     /*
24834 |                     The buffer has been completely emptied or an error occurred. In this
> case we'll need
24835 |                     to reset the state of the mapped buffer which will trigger the next
> iteration to get
24836 |                     a fresh buffer from WASAPI.
24837 |                     */
24838 |                     pDevice->wasapi.pMappedBufferCapture = NULL;
24839 |                     pDevice->wasapi.mappedBufferCaptureCap = 0;
24840 |                     pDevice->wasapi.mappedBufferCaptureLen = 0;
24841 |
24842 |                     if (hr == MA_AUDCLNT_S_BUFFER_EMPTY) {
24843 |                         if ((flags & MA_AUDCLNT_BUFFERFLAGS_DATA_DISCONTINUITY) != 0) {
24844 |                             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG,
> "[WASAPI] Data discontinuity recovery: Buffer emptied, and data discontinuity still reported.\n");
24845 |                         } else {
24846 |                             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG,
> "[WASAPI] Data discontinuity recovery: Buffer emptied.\n");
24847 |                         }
24848 |                     }
24849 |
24850 |                     if (FAILED(hr)) {
24851 |                         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG,
> "[WASAPI] Data discontinuity recovery: IAudioCaptureClient_GetBuffer() failed with %ld.\n", hr);
24852 |                     }
24853 |
24854 |                     break;
24855 |                 }
24856 |             }
24857 |
24858 |             /* If at this point we have a valid buffer mapped, make sure the buffer
> length is set appropriately. */
24859 |             if (pDevice->wasapi.pMappedBufferCapture != NULL) {
24860 |                 pDevice->wasapi.mappedBufferCaptureLen =
> pDevice->wasapi.mappedBufferCaptureCap;
24861 |             }
24862 |         }
24863 |     }
24864 |
24865 |     continue;
24866 | } else {
24867 |     if (hr == MA_AUDCLNT_S_BUFFER_EMPTY || hr == MA_AUDCLNT_E_BUFFER_ERROR) {
24868 |         /*
24869 |         No data is available. We need to wait for more. There's two situations to
> consider
24870 |         here. The first is normal capture mode. If this times out it probably means the
24871 |         microphone isn't delivering data for whatever reason. In this case we'll just
24872 |         abort the read and return whatever we were able to get. The other situations is
24873 |         loopback mode, in which case a timeout probably just means the nothing is
> playing
24874 |         through the speakers.
24875 |         */
24876 |
24877 |         /* Experiment: Use a shorter timeout for loopback mode. */
24878 |         DWORD timeoutInMilliseconds = MA_WASAPI_WAIT_TIMEOUT_MILLISECONDS;
24879 |         if (pDevice->type == ma_device_type_loopback) {
24880 |             timeoutInMilliseconds = 10;
24881 |         }
24882 |     }

```

```

24883 |         if (WaitForSingleObject((HANDLE)pDevice->wasapi.hEventCapture,
> timeoutInMilliseconds) != WAIT_OBJECT_0) {
24884 |             if (pDevice->type == ma_device_type_loopback) {
24885 |                 continue; /* Keep waiting in loopback mode. */
24886 |             } else {
24887 |                 result = MA_ERROR;
24888 |                 break; /* Wait failed. */
24889 |             }
24890 |         }
24891 |
24892 |         /* At this point we should be able to loop back to the start of the loop and try
> retrieving a data buffer again. */
24893 |     } else {
24894 |         /* An error occurred and we need to abort. */
24895 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to
> retrieve internal buffer from capture device in preparation for reading from the device. HRESULT =
> %d. Stopping device.\n", (int)hr);
24896 |         result = ma_result_from_HRESULT(hr);
24897 |         break;
24898 |     }
24899 | }
24900 | }
24901 | }
24902 |
24903 | /*
24904 | If we were unable to process the entire requested frame count, but we still have a mapped
> buffer,
24905 | there's a good chance either an error occurred or the device was stopped mid-read. In this case
24906 | we'll need to make sure the buffer is released.
24907 | */
24908 | if (totalFramesProcessed < frameCount && pDevice->wasapi.pMappedBufferCapture != NULL) {
24909 |
> ma_IAudioCaptureClient_ReleaseBuffer((ma_IAudioCaptureClient*)pDevice->wasapi.pCaptureClient,
> pDevice->wasapi.mappedBufferCaptureCap);
24910 |     pDevice->wasapi.pMappedBufferCapture = NULL;
24911 |     pDevice->wasapi.mappedBufferCaptureCap = 0;
24912 |     pDevice->wasapi.mappedBufferCaptureLen = 0;
24913 | }
24914 |
24915 | if (pFramesRead != NULL) {
24916 |     *pFramesRead = totalFramesProcessed;
24917 | }
24918 |
24919 | return result;
24920 | }
24921 |
24922 | static ma_result ma_device_write_wasapi(ma_device* pDevice, const void* pFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
24923 | {
24924 |     ma_result result = MA_SUCCESS;
24925 |     ma_uint32 totalFramesProcessed = 0;
24926 |
24927 |     /* Keep writing to the device until it's stopped or we've consumed all of our input. */
24928 |     while (ma_device_get_state(pDevice) == ma_device_state_started && totalFramesProcessed <
> frameCount) {
24929 |         ma_uint32 framesRemaining = frameCount - totalFramesProcessed;
24930 |
24931 |         /*
24932 |         We're going to do this in a similar way to capture. We'll first check if the cached data
> pointer
24933 |         is valid, and if so, read from that. Otherwise We will call IAudioRenderClient_GetBuffer()
> with
24934 |         a requested buffer size equal to our actual period size. If it returns
> AUDCLNT_E_BUFFER_TOO_LARGE
24935 |         it means we need to wait for some data to become available.
24936 |         */
24937 |         if (pDevice->wasapi.pMappedBufferPlayback != NULL) {
24938 |             /* We still have some space available in the mapped data buffer. Write to it. */
24939 |             ma_uint32 framesToProcessNow = framesRemaining;
24940 |             if (framesToProcessNow > (pDevice->wasapi.mappedBufferPlaybackCap -
> pDevice->wasapi.mappedBufferPlaybackLen)) {
24941 |                 framesToProcessNow = (pDevice->wasapi.mappedBufferPlaybackCap -
> pDevice->wasapi.mappedBufferPlaybackLen);
24942 |             }
24943 |
24944 |             /* Now just copy the data over to the output buffer. */
24945 |             ma_copy_pcm_frames(
24946 |                 ma_offset_pcm_frames_ptr(pDevice->wasapi.pMappedBufferPlayback,
> pDevice->wasapi.mappedBufferPlaybackLen, pDevice->playback.internalFormat,

```

```

> pDevice->playback.internalChannels),
24947 |         ma_offset_pcm_frames_const_ptr(pFrames, totalFramesProcessed,
> pDevice->playback.internalFormat, pDevice->playback.internalChannels),
24948 |         framesToProcessNow,
24949 |         pDevice->playback.internalFormat, pDevice->playback.internalChannels
24950 |     );
24951 |
24952 |     totalFramesProcessed += framesToProcessNow;
24953 |     pDevice->wasapi.mappedBufferPlaybackLen += framesToProcessNow;
24954 |
24955 |     /* If the data buffer has been fully consumed we need to release it. */
24956 |     if (pDevice->wasapi.mappedBufferPlaybackLen == pDevice->wasapi.mappedBufferPlaybackCap)
> {
24957 |
> ma_IAudioRenderClient_ReleaseBuffer((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient,
> pDevice->wasapi.mappedBufferPlaybackCap, 0);
24958 |     pDevice->wasapi.pMappedBufferPlayback = NULL;
24959 |     pDevice->wasapi.mappedBufferPlaybackCap = 0;
24960 |     pDevice->wasapi.mappedBufferPlaybackLen = 0;
24961 |
24962 |     /*
24963 |     In exclusive mode we need to wait here. Exclusive mode is weird because GetBuffer()
> never
24964 |     seems to return AUDCLNT_E_BUFFER_TOO_LARGE, which is what we normally use to
> determine
24965 |     whether or not we need to wait for more data.
24966 |     */
24967 |     if (pDevice->playback.shareMode == ma_share_mode_exclusive) {
24968 |         if (WaitForSingleObject((HANDLE)pDevice->wasapi.hEventPlayback,
> MA_WASAPI_WAIT_TIMEOUT_MILLISECONDS) != WAIT_OBJECT_0) {
24969 |             result = MA_ERROR;
24970 |             break; /* Wait failed. Probably timed out. */
24971 |         }
24972 |     }
24973 | }
24974 | } else {
24975 |     /* We don't have a mapped data buffer so we'll need to get one. */
24976 |     HRESULT hr;
24977 |     ma_uint32 bufferSizeInFrames;
24978 |
24979 |     /* Special rules for exclusive mode. */
24980 |     if (pDevice->playback.shareMode == ma_share_mode_exclusive) {
24981 |         bufferSizeInFrames = pDevice->wasapi.actualBufferSizeInFramesPlayback;
24982 |     } else {
24983 |         bufferSizeInFrames = pDevice->wasapi.periodSizeInFramesPlayback;
24984 |     }
24985 |
24986 |     hr =
> ma_IAudioRenderClient_GetBuffer((ma_IAudioRenderClient*)pDevice->wasapi.pRenderClient,
> bufferSizeInFrames, (BYTE**)&pDevice->wasapi.pMappedBufferPlayback);
24987 |     if (hr == S_OK) {
24988 |         /* We have data available. */
24989 |         pDevice->wasapi.mappedBufferPlaybackCap = bufferSizeInFrames;
24990 |         pDevice->wasapi.mappedBufferPlaybackLen = 0;
24991 |     } else {
24992 |         if (hr == MA_AUDCLNT_E_BUFFER_TOO_LARGE || hr == MA_AUDCLNT_E_BUFFER_ERROR) {
24993 |             /* Not enough data available. We need to wait for more. */
24994 |             if (WaitForSingleObject((HANDLE)pDevice->wasapi.hEventPlayback,
> MA_WASAPI_WAIT_TIMEOUT_MILLISECONDS) != WAIT_OBJECT_0) {
24995 |                 result = MA_ERROR;
24996 |                 break; /* Wait failed. Probably timed out. */
24997 |             }
24998 |         } else {
24999 |             /* Some error occurred. We'll need to abort. */
25000 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WASAPI] Failed to
> retrieve internal buffer from playback device in preparation for writing to the device. HRESULT =
> %d. Stopping device.\n", (int)hr);
25001 |             result = ma_result_from_HRESULT(hr);
25002 |             break;
25003 |         }
25004 |     }
25005 | }
25006 | }
25007 |
25008 | if (pFramesWritten != NULL) {
25009 |     *pFramesWritten = totalFramesProcessed;
25010 | }
25011 |
25012 | return result;

```

```

25013 | }
25014 |
25015 | static ma_result ma_device_data_loop_wakeup_wasapi(ma_device* pDevice)
25016 | {
25017 |     MA_ASSERT(pDevice != NULL);
25018 |
25019 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex ||
> pDevice->type == ma_device_type_loopback) {
25020 |         SetEvent((HANDLE)pDevice->wasapi.hEventCapture);
25021 |     }
25022 |
25023 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
25024 |         SetEvent((HANDLE)pDevice->wasapi.hEventPlayback);
25025 |     }
25026 |
25027 |     return MA_SUCCESS;
25028 | }
25029 |
25030 |
25031 | static ma_result ma_context_uninit_wasapi(ma_context* pContext)
25032 | {
25033 |     ma_context_command_wasapi cmd =
> ma_context_init_command_wasapi(MA_CONTEXT_COMMAND_QUIT__WASAPI);
25034 |
25035 |     MA_ASSERT(pContext != NULL);
25036 |     MA_ASSERT(pContext->backend == ma_backend_wasapi);
25037 |
25038 |     ma_context_post_command_wasapi(pContext, &cmd);
25039 |     ma_thread_wait(&pContext->wasapi.commandThread);
25040 |
25041 |     if (pContext->wasapi.hAvrt) {
25042 |         ma_dlclose(ma_context_get_log(pContext), pContext->wasapi.hAvrt);
25043 |         pContext->wasapi.hAvrt = NULL;
25044 |     }
25045 |
25046 |     #if defined(MA_WIN32_UWP)
25047 |     {
25048 |         if (pContext->wasapi.hMMDevapi) {
25049 |             ma_dlclose(ma_context_get_log(pContext), pContext->wasapi.hMMDevapi);
25050 |             pContext->wasapi.hMMDevapi = NULL;
25051 |         }
25052 |     }
25053 |     #endif
25054 |
25055 |     /* Only after the thread has been terminated can we uninitialize the sync objects for the
> command thread. */
25056 |     ma_semaphore_uninit(&pContext->wasapi.commandSem);
25057 |     ma_mutex_uninit(&pContext->wasapi.commandLock);
25058 |
25059 |     return MA_SUCCESS;
25060 | }
25061 |
25062 | static ma_result ma_context_init_wasapi(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
25063 | {
25064 |     ma_result result = MA_SUCCESS;
25065 |
25066 |     MA_ASSERT(pContext != NULL);
25067 |
25068 |     (void)pConfig;
25069 |
25070 |     #ifdef MA_WIN32_DESKTOP
25071 |     /*
25072 |     WASAPI is only supported in Vista SP1 and newer. The reason for SP1 and not the base version of
> Vista is that event-driven
25073 |     exclusive mode does not work until SP1.
25074 |
25075 |     Unfortunately older compilers don't define these functions so we need to dynamically load them
> in order to avoid a link error.
25076 |     */
25077 |     {
25078 |         ma_OSVERSIONINFOEXW osv;
25079 |         ma_handle kernel32DLL;
25080 |         ma_PFNVerifyVersionInfoW _VerifyVersionInfoW;
25081 |         ma_PFNVerSetConditionMask _VerSetConditionMask;
25082 |
25083 |         kernel32DLL = ma_dlopen(ma_context_get_log(pContext), "kernel32.dll");
25084 |         if (kernel32DLL == NULL) {
25085 |             return MA_NO_BACKEND;

```

```

25086 |     }
25087 |
25088 |     _VerifyVersionInfoW = (ma_PFNVerifyVersionInfoW )ma_dlsym(ma_context_get_log(pContext),
> kernel32DLL, "VerifyVersionInfoW");
25089 |     _VerSetConditionMask = (ma_PFNVerSetConditionMask)ma_dlsym(ma_context_get_log(pContext),
> kernel32DLL, "VerSetConditionMask");
25090 |     if (_VerifyVersionInfoW == NULL || _VerSetConditionMask == NULL) {
25091 |         ma_dlclose(ma_context_get_log(pContext), kernel32DLL);
25092 |         return MA_NO_BACKEND;
25093 |     }
25094 |
25095 |     MA_ZERO_OBJECT(&osvi);
25096 |     osvi.dwOSVersionInfoSize = sizeof(osvi);
25097 |     osvi.dwMajorVersion = ((MA_WIN32_WINNT_VISTA >> 8) & 0xFF);
25098 |     osvi.dwMinorVersion = ((MA_WIN32_WINNT_VISTA >> 0) & 0xFF);
25099 |     osvi.wServicePackMajor = 1;
25100 |     if (_VerifyVersionInfoW(&osvi, MA_VER_MAJORVERSION | MA_VER_MINORVERSION |
> MA_VER_SERVICEPACKMAJOR, _VerSetConditionMask(_VerSetConditionMask(_VerSetConditionMask(0,
> MA_VER_MAJORVERSION, MA_VER_GREATER_EQUAL), MA_VER_MINORVERSION, MA_VER_GREATER_EQUAL),
> MA_VER_SERVICEPACKMAJOR, MA_VER_GREATER_EQUAL))) {
25101 |         result = MA_SUCCESS;
25102 |     } else {
25103 |         result = MA_NO_BACKEND;
25104 |     }
25105 |
25106 |     ma_dlclose(ma_context_get_log(pContext), kernel32DLL);
25107 | }
25108 | #endif
25109 |
25110 | if (result != MA_SUCCESS) {
25111 |     return result;
25112 | }
25113 |
25114 | MA_ZERO_OBJECT(&pContext->wasapi);
25115 |
25116 |
25117 | #if defined(MA_WIN32_UWP)
25118 | {
25119 |     /* Link to mmdevapi so we can get access to ActivateAudioInterfaceAsync(). */
25120 |     pContext->wasapi.hMMDevapi = ma_dlopen(ma_context_get_log(pContext), "mmdevapi.dll");
25121 |     if (pContext->wasapi.hMMDevapi) {
25122 |         pContext->wasapi.ActivateAudioInterfaceAsync = ma_dlsym(ma_context_get_log(pContext),
> pContext->wasapi.hMMDevapi, "ActivateAudioInterfaceAsync");
25123 |         if (pContext->wasapi.ActivateAudioInterfaceAsync == NULL) {
25124 |             ma_dlclose(ma_context_get_log(pContext), pContext->wasapi.hMMDevapi);
25125 |             return MA_NO_BACKEND; /* ActivateAudioInterfaceAsync() could not be loaded. */
25126 |         }
25127 |     } else {
25128 |         return MA_NO_BACKEND; /* Failed to load mmdevapi.dll which is required for
> ActivateAudioInterfaceAsync() */
25129 |     }
25130 | }
25131 | #endif
25132 |
25133 | /* Optionally use the Avrt API to specify the audio thread's latency sensitivity requirements */
25134 | pContext->wasapi.hAvrt = ma_dlopen(ma_context_get_log(pContext), "avrt.dll");
25135 | if (pContext->wasapi.hAvrt) {
25136 |     pContext->wasapi.AvSetMmThreadCharacteristicsA = ma_dlsym(ma_context_get_log(pContext),
> pContext->wasapi.hAvrt, "AvSetMmThreadCharacteristicsA");
25137 |     pContext->wasapi.AvRevertMmThreadCharacteristics = ma_dlsym(ma_context_get_log(pContext),
> pContext->wasapi.hAvrt, "AvRevertMmThreadCharacteristics");
25138 |
25139 |     /* If either function could not be found, disable use of avrt entirely. */
25140 |     if (!pContext->wasapi.AvSetMmThreadCharacteristicsA ||
> !pContext->wasapi.AvRevertMmThreadCharacteristics) {
25141 |         pContext->wasapi.AvSetMmThreadCharacteristicsA = NULL;
25142 |         pContext->wasapi.AvRevertMmThreadCharacteristics = NULL;
25143 |         ma_dlclose(ma_context_get_log(pContext), pContext->wasapi.hAvrt);
25144 |         pContext->wasapi.hAvrt = NULL;
25145 |     }
25146 | }
25147 |
25148 |
25149 | /*
25150 | Annoyingly, WASAPI does not allow you to release an IAudioClient object from a different thread
25151 | than the one that retrieved it with GetService(). This can result in a deadlock in two
25152 | situations:
25153 |
25154 | 1) When calling ma_device_uninit() from a different thread to ma_device_init(); and

```



```

25155 |         2) When uninitializing and reinitializing the internal IAudioClient object in response to
25156 |             automatic stream routing.
25157 |
25158 |         We could define ma_device_uninit() such that it must be called on the same thread as
25159 |         ma_device_init(). We could also just not release the IAudioClient when performing automatic
25160 |         stream routing to avoid the deadlock. Neither of these are acceptable solutions in my view so
25161 |         we're going to have to work around this with a worker thread. This is not ideal, but I can't
25162 |         think of a better way to do this.
25163 |
25164 |         More information about this can be found here:
25165 |
25166 |         > https://docs.microsoft.com/en-us/windows/win32/api/audioclient/nn-audioclient-iaudiorenderclient
25167 |
25168 |         Note this section:
25169 |
25170 |         When releasing an IAudioRenderClient interface instance, the client must call the
25171 |         > interface's
25172 |         Release method from the same thread as the call to IAudioClient::GetService that created the
25173 |         object.
25174 |         */
25175 |         {
25176 |             result = ma_mutex_init(&pContext->wasapi.commandLock);
25177 |             if (result != MA_SUCCESS) {
25178 |                 return result;
25179 |             }
25180 |
25181 |             result = ma_semaphore_init(0, &pContext->wasapi.commandSem);
25182 |             if (result != MA_SUCCESS) {
25183 |                 ma_mutex_uninit(&pContext->wasapi.commandLock);
25184 |                 return result;
25185 |             }
25186 |
25187 |             result = ma_thread_create(&pContext->wasapi.commandThread, ma_thread_priority_normal, 0,
25188 |             > ma_context_command_thread__wasapi, pContext, &pContext->allocationCallbacks);
25189 |             if (result != MA_SUCCESS) {
25190 |                 ma_semaphore_uninit(&pContext->wasapi.commandSem);
25191 |                 ma_mutex_uninit(&pContext->wasapi.commandLock);
25192 |                 return result;
25193 |             }
25194 |         }
25195 |
25196 |         pCallbacks->onContextInit = ma_context_init__wasapi;
25197 |         pCallbacks->onContextUninit = ma_context_uninit__wasapi;
25198 |         pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__wasapi;
25199 |         pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__wasapi;
25200 |         pCallbacks->onDeviceInit = ma_device_init__wasapi;
25201 |         pCallbacks->onDeviceUninit = ma_device_uninit__wasapi;
25202 |         pCallbacks->onDeviceStart = ma_device_start__wasapi;
25203 |         pCallbacks->onDeviceStop = ma_device_stop__wasapi;
25204 |         pCallbacks->onDeviceRead = ma_device_read__wasapi;
25205 |         pCallbacks->onDeviceWrite = ma_device_write__wasapi;
25206 |         pCallbacks->onDeviceDataLoop = NULL;
25207 |         pCallbacks->onDeviceDataLoopWakeup = ma_device_data_loop_wakeup__wasapi;
25208 |
25209 |         return MA_SUCCESS;
25210 |     }
25211 | #endif
25212 | /*****
25213 |
25214 | DirectSound Backend
25215 |
25216 | *****/
25217 | #ifdef MA_HAS_DSOUND
25218 | /*#include <dsound.h>*/
25219 |
25220 | /*static const GUID MA_GUID_IID_DirectSoundNotify = {0xb0210783, 0x89cd, 0x11d0, {0xaf, 0x08, 0x00,
25221 | > 0xa0, 0xc9, 0x25, 0xcd, 0x16}};*/
25222 |
25223 | /* miniaudio only uses priority or exclusive modes. */
25224 | #define MA_DSSCL_NORMAL 1
25225 | #define MA_DSSCL_PRIORITY 2
25226 | #define MA_DSSCL_EXCLUSIVE 3
25227 | #define MA_DSSCL_WRITEPRIMARY 4
25228 | #define MA_DSCAPS_PRIMARYMONO 0x00000001
25229 | #define MA_DSCAPS_PRIMARYSTEREO 0x00000002

```

```

25230 | #define MA_DSCAPS_PRIMARY8BIT          0x00000004
25231 | #define MA_DSCAPS_PRIMARY16BIT         0x00000008
25232 | #define MA_DSCAPS_CONTINUOUSRATE       0x00000010
25233 | #define MA_DSCAPS_EMULDRIVER           0x00000020
25234 | #define MA_DSCAPS_CERTIFIED            0x00000040
25235 | #define MA_DSCAPS_SECONDARYMONO        0x00000100
25236 | #define MA_DSCAPS_SECONDARYSTEREO      0x00000200
25237 | #define MA_DSCAPS_SECONDARY8BIT        0x00000400
25238 | #define MA_DSCAPS_SECONDARY16BIT       0x00000800
25239 |
25240 | #define MA_DSBCAPS_PRIMARYBUFFER       0x00000001
25241 | #define MA_DSBCAPS_STATIC               0x00000002
25242 | #define MA_DSBCAPS_LOCHARDWARE          0x00000004
25243 | #define MA_DSBCAPS_LOCSOFTWARE          0x00000008
25244 | #define MA_DSBCAPS_CTRL3D               0x00000010
25245 | #define MA_DSBCAPS_CTRLFREQUENCY        0x00000020
25246 | #define MA_DSBCAPS_CTRLPAN              0x00000040
25247 | #define MA_DSBCAPS_CTRLVOLUME           0x00000080
25248 | #define MA_DSBCAPS_CTRLPOSITIONNOTIFY   0x00000100
25249 | #define MA_DSBCAPS_CTRLFX              0x00000200
25250 | #define MA_DSBCAPS_STICKYFOCUS           0x00004000
25251 | #define MA_DSBCAPS_GLOBALFOCUS          0x00008000
25252 | #define MA_DSBCAPS_GETCURRENTPOSITION2  0x00010000
25253 | #define MA_DSBCAPS_MUTE3DATMAXDISTANCE  0x00020000
25254 | #define MA_DSBCAPS_LOCDEFER             0x00040000
25255 | #define MA_DSBCAPS_TRUEPLAYPOSITION     0x00080000
25256 |
25257 | #define MA_DSBPLAY_LOOPING              0x00000001
25258 | #define MA_DSBPLAY_LOCHARDWARE          0x00000002
25259 | #define MA_DSBPLAY_LOCSOFTWARE          0x00000004
25260 | #define MA_DSBPLAY_TERMINATEBY_TIME     0x00000008
25261 | #define MA_DSBPLAY_TERMINATEBY_DISTANCE 0x00000010
25262 | #define MA_DSBPLAY_TERMINATEBY_PRIORITY 0x00000020
25263 |
25264 | #define MA_DSBSTATUS_PLAYING            0x00000001
25265 | #define MA_DSBSTATUS_BUFFERLOST         0x00000002
25266 | #define MA_DSBSTATUS_LOOPING            0x00000004
25267 | #define MA_DSBSTATUS_LOCHARDWARE        0x00000008
25268 | #define MA_DSBSTATUS_LOCSOFTWARE        0x00000010
25269 | #define MA_DSBSTATUS_TERMINATED         0x00000020
25270 |
25271 | #define MA_DSCBSTART_LOOPING            0x00000001
25272 |
25273 | typedef struct
25274 | {
25275 |     DWORD dwSize;
25276 |     DWORD dwFlags;
25277 |     DWORD dwBufferBytes;
25278 |     DWORD dwReserved;
25279 |     MA_WAVEFORMATEX* lpwfxFormat;
25280 |     GUID guid3DAlgorithm;
25281 | } MA_DSBBUFFERDESC;
25282 |
25283 | typedef struct
25284 | {
25285 |     DWORD dwSize;
25286 |     DWORD dwFlags;
25287 |     DWORD dwBufferBytes;
25288 |     DWORD dwReserved;
25289 |     MA_WAVEFORMATEX* lpwfxFormat;
25290 |     DWORD dwFXCount;
25291 |     void* lpDSCFXDesc; /* <-- miniaudio doesn't use this, so set to void*. */
25292 | } MA_DSCBUFFERDESC;
25293 |
25294 | typedef struct
25295 | {
25296 |     DWORD dwSize;
25297 |     DWORD dwFlags;
25298 |     DWORD dwMinSecondarySampleRate;
25299 |     DWORD dwMaxSecondarySampleRate;
25300 |     DWORD dwPrimaryBuffers;
25301 |     DWORD dwMaxHwMixingAllBuffers;
25302 |     DWORD dwMaxHwMixingStaticBuffers;
25303 |     DWORD dwMaxHwMixingStreamingBuffers;
25304 |     DWORD dwFreeHwMixingAllBuffers;
25305 |     DWORD dwFreeHwMixingStaticBuffers;
25306 |     DWORD dwFreeHwMixingStreamingBuffers;
25307 |     DWORD dwMaxHw3DAllBuffers;
25308 |     DWORD dwMaxHw3DStaticBuffers;

```

```

25309 |     DWORD dwMaxHw3DStreamingBuffers;
25310 |     DWORD dwFreeHw3DAllBuffers;
25311 |     DWORD dwFreeHw3DStaticBuffers;
25312 |     DWORD dwFreeHw3DStreamingBuffers;
25313 |     DWORD dwTotalHwMemBytes;
25314 |     DWORD dwFreeHwMemBytes;
25315 |     DWORD dwMaxContigFreeHwMemBytes;
25316 |     DWORD dwUnlockTransferRateHwBuffers;
25317 |     DWORD dwPlayCpuOverheadSwBuffers;
25318 |     DWORD dwReserved1;
25319 |     DWORD dwReserved2;
25320 | } MA_DSCAPS;
25321 |
25322 | typedef struct
25323 | {
25324 |     DWORD dwSize;
25325 |     DWORD dwFlags;
25326 |     DWORD dwBufferBytes;
25327 |     DWORD dwUnlockTransferRate;
25328 |     DWORD dwPlayCpuOverhead;
25329 | } MA_DSBCAPS;
25330 |
25331 | typedef struct
25332 | {
25333 |     DWORD dwSize;
25334 |     DWORD dwFlags;
25335 |     DWORD dwFormats;
25336 |     DWORD dwChannels;
25337 | } MA_DSCCAPS;
25338 |
25339 | typedef struct
25340 | {
25341 |     DWORD dwSize;
25342 |     DWORD dwFlags;
25343 |     DWORD dwBufferBytes;
25344 |     DWORD dwReserved;
25345 | } MA_DSCBCAPS;
25346 |
25347 | typedef struct
25348 | {
25349 |     DWORD dwOffset;
25350 |     HANDLE hEventNotify;
25351 | } MA_DSBPOSITIONNOTIFY;
25352 |
25353 | typedef struct ma_IDirectSound          ma_IDirectSound;
25354 | typedef struct ma_IDirectSoundBuffer    ma_IDirectSoundBuffer;
25355 | typedef struct ma_IDirectSoundCapture    ma_IDirectSoundCapture;
25356 | typedef struct ma_IDirectSoundCaptureBuffer ma_IDirectSoundCaptureBuffer;
25357 | typedef struct ma_IDirectSoundNotify     ma_IDirectSoundNotify;
25358 |
25359 |
25360 | /*
25361 | COM objects. The way these work is that you have a vtable (a list of function pointers, kind of
25362 | like how C++ works internally), and then you have a structure with a single member, which is a
25363 | pointer to the vtable. The vtable is where the methods of the object are defined. Methods need
25364 | to be in a specific order, and parent classes need to have their methods declared first.
25365 | */
25366 |
25367 | /* IDirectSound */
25368 | typedef struct
25369 | {
25370 |     /* IUnknown */
25371 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IDirectSound* pThis, const IID* const riid,
> void** ppObject);
25372 |     ULONG (STDMETHODCALLTYPE * AddRef)          (ma_IDirectSound* pThis);
25373 |     ULONG (STDMETHODCALLTYPE * Release)         (ma_IDirectSound* pThis);
25374 |
25375 |     /* IDirectSound */
25376 |     HRESULT (STDMETHODCALLTYPE * CreateSoundBuffer) (ma_IDirectSound* pThis, const
> MA_DSBUFFERDESC* pDSBufferDesc, ma_IDirectSoundBuffer** ppDSBuffer, void* pUnkOuter);
25377 |     HRESULT (STDMETHODCALLTYPE * GetCaps)          (ma_IDirectSound* pThis, MA_DSCAPS* pDSCaps);
25378 |     HRESULT (STDMETHODCALLTYPE * DuplicateSoundBuffer)(ma_IDirectSound* pThis,
> ma_IDirectSoundBuffer* pDSBufferOriginal, ma_IDirectSoundBuffer** ppDSBufferDuplicate);
25379 |     HRESULT (STDMETHODCALLTYPE * SetCooperativeLevel) (ma_IDirectSound* pThis, HWND hwnd, DWORD
> dwLevel);
25380 |     HRESULT (STDMETHODCALLTYPE * Compact)          (ma_IDirectSound* pThis);
25381 |     HRESULT (STDMETHODCALLTYPE * GetSpeakerConfig) (ma_IDirectSound* pThis, DWORD*
> pSpeakerConfig);
25382 |     HRESULT (STDMETHODCALLTYPE * SetSpeakerConfig) (ma_IDirectSound* pThis, DWORD

```

```

> dwSpeakerConfig);
25383 | HRESULT (STDMETHODCALLTYPE * Initialize)          (ma_IDirectSound* pThis, const GUID*
> pGuidDevice);
25384 | } ma_IDirectSoundVtbl;
25385 | struct ma_IDirectSound
25386 | {
25387 |     ma_IDirectSoundVtbl* lpVtbl;
25388 | };
25389 | static MA_INLINE HRESULT ma_IDirectSound_QueryInterface(ma_IDirectSound* pThis, const IID* const
> riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
25390 | static MA_INLINE ULONG ma_IDirectSound_AddRef(ma_IDirectSound* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
25391 | static MA_INLINE ULONG ma_IDirectSound_Release(ma_IDirectSound* pThis)
> { return pThis->lpVtbl->Release(pThis); }
25392 | static MA_INLINE HRESULT ma_IDirectSound_CreateSoundBuffer(ma_IDirectSound* pThis, const
> MA_DSBUFFERDESC* pDSBufferDesc, ma_IDirectSoundBuffer** ppDSBuffer, void* pUnkOuter) { return
> pThis->lpVtbl->CreateSoundBuffer(pThis, pDSBufferDesc, ppDSBuffer, pUnkOuter); }
25393 | static MA_INLINE HRESULT ma_IDirectSound_GetCaps(ma_IDirectSound* pThis, MA_DSCAPS* pDSCaps)
> { return pThis->lpVtbl->GetCaps(pThis, pDSCaps); }
25394 | static MA_INLINE HRESULT ma_IDirectSound_DuplicateSoundBuffer(ma_IDirectSound* pThis,
> ma_IDirectSoundBuffer* pDSBufferOriginal, ma_IDirectSoundBuffer** ppDSBufferDuplicate) { return
> pThis->lpVtbl->DuplicateSoundBuffer(pThis, pDSBufferOriginal, ppDSBufferDuplicate); }
25395 | static MA_INLINE HRESULT ma_IDirectSound_SetCooperativeLevel(ma_IDirectSound* pThis, HWND hwnd,
> DWORD dwLevel) { return pThis->lpVtbl->SetCooperativeLevel(pThis, hwnd, dwLevel); }
25396 | static MA_INLINE HRESULT ma_IDirectSound_Compact(ma_IDirectSound* pThis)
> { return pThis->lpVtbl->Compact(pThis); }
25397 | static MA_INLINE HRESULT ma_IDirectSound_GetSpeakerConfig(ma_IDirectSound* pThis, DWORD*
> pSpeakerConfig) { return pThis->lpVtbl->GetSpeakerConfig(pThis, pSpeakerConfig); }
25398 | static MA_INLINE HRESULT ma_IDirectSound_SetSpeakerConfig(ma_IDirectSound* pThis, DWORD
> dwSpeakerConfig) { return pThis->lpVtbl->SetSpeakerConfig(pThis, dwSpeakerConfig); }
25399 | static MA_INLINE HRESULT ma_IDirectSound_Initialize(ma_IDirectSound* pThis, const GUID* pGuidDevice)
> { return pThis->lpVtbl->Initialize(pThis, pGuidDevice); }
25400 |
25401 |
25402 | /* IDirectSoundBuffer */
25403 | typedef struct
25404 | {
25405 |     /* IUnknown */
25406 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IDirectSoundBuffer* pThis, const IID* const
> riid, void** ppObject);
25407 |     ULONG (STDMETHODCALLTYPE * AddRef)          (ma_IDirectSoundBuffer* pThis);
25408 |     ULONG (STDMETHODCALLTYPE * Release)         (ma_IDirectSoundBuffer* pThis);
25409 |
25410 |     /* IDirectSoundBuffer */
25411 |     HRESULT (STDMETHODCALLTYPE * GetCaps)        (ma_IDirectSoundBuffer* pThis, MA_DSBCAPS*
> pDSBufferCaps);
25412 |     HRESULT (STDMETHODCALLTYPE * GetCurrentPosition)(ma_IDirectSoundBuffer* pThis, DWORD*
> pCurrentPlayCursor, DWORD* pCurrentWriteCursor);
25413 |     HRESULT (STDMETHODCALLTYPE * GetFormat)       (ma_IDirectSoundBuffer* pThis, MA_WAVEFORMATEX*
> pFormat, DWORD dwSizeAllocated, DWORD* pSizeWritten);
25414 |     HRESULT (STDMETHODCALLTYPE * GetVolume)       (ma_IDirectSoundBuffer* pThis, LONG* pVolume);
25415 |     HRESULT (STDMETHODCALLTYPE * GetPan)          (ma_IDirectSoundBuffer* pThis, LONG* pPan);
25416 |     HRESULT (STDMETHODCALLTYPE * GetFrequency)    (ma_IDirectSoundBuffer* pThis, DWORD*
> pFrequency);
25417 |     HRESULT (STDMETHODCALLTYPE * GetStatus)       (ma_IDirectSoundBuffer* pThis, DWORD* pStatus);
25418 |     HRESULT (STDMETHODCALLTYPE * Initialize)      (ma_IDirectSoundBuffer* pThis, ma_IDirectSound*
> pDirectSound, const MA_DSBUFFERDESC* pDSBufferDesc);
25419 |     HRESULT (STDMETHODCALLTYPE * Lock)           (ma_IDirectSoundBuffer* pThis, DWORD dwOffset,
> DWORD dwBytes, void** ppAudioPtr1, DWORD* pAudioBytes1, void** ppAudioPtr2, DWORD* pAudioBytes2,
> DWORD dwFlags);
25420 |     HRESULT (STDMETHODCALLTYPE * Play)           (ma_IDirectSoundBuffer* pThis, DWORD
> dwReserved1, DWORD dwPriority, DWORD dwFlags);
25421 |     HRESULT (STDMETHODCALLTYPE * SetCurrentPosition)(ma_IDirectSoundBuffer* pThis, DWORD
> dwNewPosition);
25422 |     HRESULT (STDMETHODCALLTYPE * SetFormat)       (ma_IDirectSoundBuffer* pThis, const
> MA_WAVEFORMATEX* pFormat);
25423 |     HRESULT (STDMETHODCALLTYPE * SetVolume)       (ma_IDirectSoundBuffer* pThis, LONG volume);
25424 |     HRESULT (STDMETHODCALLTYPE * SetPan)          (ma_IDirectSoundBuffer* pThis, LONG pan);
25425 |     HRESULT (STDMETHODCALLTYPE * SetFrequency)    (ma_IDirectSoundBuffer* pThis, DWORD
> dwFrequency);
25426 |     HRESULT (STDMETHODCALLTYPE * Stop)           (ma_IDirectSoundBuffer* pThis);
25427 |     HRESULT (STDMETHODCALLTYPE * Unlock)         (ma_IDirectSoundBuffer* pThis, void* pAudioPtr1,
> DWORD dwAudioBytes1, void* pAudioPtr2, DWORD dwAudioBytes2);
25428 |     HRESULT (STDMETHODCALLTYPE * Restore)        (ma_IDirectSoundBuffer* pThis);
25429 | } ma_IDirectSoundBufferVtbl;
25430 | struct ma_IDirectSoundBuffer
25431 | {
25432 |     ma_IDirectSoundBufferVtbl* lpVtbl;
25433 | };

```

```

25434 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_QueryInterface(ma_IDirectSoundBuffer* pThis, const
> IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
25435 | static MA_INLINE ULONG ma_IDirectSoundBuffer_AddRef(ma_IDirectSoundBuffer* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
25436 | static MA_INLINE ULONG ma_IDirectSoundBuffer_Release(ma_IDirectSoundBuffer* pThis)
> { return pThis->lpVtbl->Release(pThis); }
25437 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetCaps(ma_IDirectSoundBuffer* pThis, MA_DSBCAPS*
> pDSBufferCaps) { return pThis->lpVtbl->GetCaps(pThis, pDSBufferCaps); }
25438 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetCurrentPosition(ma_IDirectSoundBuffer* pThis,
> DWORD* pCurrentPlayCursor, DWORD* pCurrentWriteCursor) { return
> pThis->lpVtbl->GetCurrentPosition(pThis, pCurrentPlayCursor, pCurrentWriteCursor); }
25439 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetFormat(ma_IDirectSoundBuffer* pThis,
> MA_WAVEFORMATEX* pFormat, DWORD dwSizeAllocated, DWORD* pSizeWritten) { return
> pThis->lpVtbl->GetFormat(pThis, pFormat, dwSizeAllocated, pSizeWritten); }
25440 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetVolume(ma_IDirectSoundBuffer* pThis, LONG*
> pVolume) { return pThis->lpVtbl->GetVolume(pThis, pVolume); }
25441 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetPan(ma_IDirectSoundBuffer* pThis, LONG* pPan)
> { return pThis->lpVtbl->GetPan(pThis, pPan); }
25442 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetFrequency(ma_IDirectSoundBuffer* pThis, DWORD*
> pFrequency) { return pThis->lpVtbl->GetFrequency(pThis, pFrequency); }
25443 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_GetStatus(ma_IDirectSoundBuffer* pThis, DWORD*
> pStatus) { return pThis->lpVtbl->GetStatus(pThis, pStatus); }
25444 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_Initialize(ma_IDirectSoundBuffer* pThis,
> ma_IDirectSound* pDirectSound, const MA_DSBUFFERDESC* pDSBufferDesc) { return
> pThis->lpVtbl->Initialize(pThis, pDirectSound, pDSBufferDesc); }
25445 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_Lock(ma_IDirectSoundBuffer* pThis, DWORD dwOffset,
> DWORD dwBytes, void** ppAudioPtr1, DWORD* pAudioBytes1, void** ppAudioPtr2, DWORD* pAudioBytes2,
> DWORD dwFlags) { return pThis->lpVtbl->Lock(pThis, dwOffset, dwBytes, ppAudioPtr1, pAudioBytes1,
> ppAudioPtr2, pAudioBytes2, dwFlags); }
25446 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_Play(ma_IDirectSoundBuffer* pThis, DWORD dwReserved1,
> DWORD dwPriority, DWORD dwFlags) { return pThis->lpVtbl->Play(pThis, dwReserved1, dwPriority,
> dwFlags); }
25447 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_SetCurrentPosition(ma_IDirectSoundBuffer* pThis,
> DWORD dwNewPosition) { return pThis->lpVtbl->SetCurrentPosition(pThis,
> dwNewPosition); }
25448 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_SetFormat(ma_IDirectSoundBuffer* pThis, const
> MA_WAVEFORMATEX* pFormat) { return pThis->lpVtbl->SetFormat(pThis, pFormat); }
25449 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_SetVolume(ma_IDirectSoundBuffer* pThis, LONG volume)
> { return pThis->lpVtbl->SetVolume(pThis, volume); }
25450 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_SetPan(ma_IDirectSoundBuffer* pThis, LONG pan)
> { return pThis->lpVtbl->SetPan(pThis, pan); }
25451 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_SetFrequency(ma_IDirectSoundBuffer* pThis, DWORD
> dwFrequency) { return pThis->lpVtbl->SetFrequency(pThis, dwFrequency); }
25452 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_Stop(ma_IDirectSoundBuffer* pThis)
> { return pThis->lpVtbl->Stop(pThis); }
25453 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_Unlock(ma_IDirectSoundBuffer* pThis, void*
> pAudioPtr1, DWORD dwAudioBytes1, void* pAudioPtr2, DWORD dwAudioBytes2) { return
> pThis->lpVtbl->Unlock(pThis, pAudioPtr1, dwAudioBytes1, pAudioPtr2, dwAudioBytes2); }
25454 | static MA_INLINE HRESULT ma_IDirectSoundBuffer_Restore(ma_IDirectSoundBuffer* pThis)
> { return pThis->lpVtbl->Restore(pThis); }
25455 |
25456 |
25457 | /* IDirectSoundCapture */
25458 | typedef struct
25459 | {
25460 |     /* IUnknown */
25461 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IDirectSoundCapture* pThis, const IID* const
> riid, void** ppObject);
25462 |     ULONG (STDMETHODCALLTYPE * AddRef) (ma_IDirectSoundCapture* pThis);
25463 |     ULONG (STDMETHODCALLTYPE * Release) (ma_IDirectSoundCapture* pThis);
25464 |
25465 |     /* IDirectSoundCapture */
25466 |     HRESULT (STDMETHODCALLTYPE * CreateCaptureBuffer)(ma_IDirectSoundCapture* pThis, const
> MA_DSBUFFERDESC* pDSBufferDesc, ma_IDirectSoundCaptureBuffer** ppDSBuffer, void* pUnkOuter);
25467 |     HRESULT (STDMETHODCALLTYPE * GetCaps) (ma_IDirectSoundCapture* pThis, MA_DSBCAPS*
> pDSBCaps);
25468 |     HRESULT (STDMETHODCALLTYPE * Initialize) (ma_IDirectSoundCapture* pThis, const GUID*
> pGuidDevice);
25469 | } ma_IDirectSoundCaptureVtbl;
25470 | struct ma_IDirectSoundCapture
25471 | {
25472 |     ma_IDirectSoundCaptureVtbl* lpVtbl;
25473 | };
25474 | static MA_INLINE HRESULT ma_IDirectSoundCapture_QueryInterface (ma_IDirectSoundCapture* pThis,
> const IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid,
> ppObject); }
25475 | static MA_INLINE ULONG ma_IDirectSoundCapture_AddRef (ma_IDirectSoundCapture* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
25476 | static MA_INLINE ULONG ma_IDirectSoundCapture_Release (ma_IDirectSoundCapture* pThis)

```



```

>                                     { return pThis->lpVtbl->Release(pThis); }
25477 | static MA_INLINE HRESULT ma_IDirectSoundCapture_CreateCaptureBuffer(ma_IDirectSoundCapture* pThis,
> const MA_DSCBUFFERDESC* pDSCBufferDesc, ma_IDirectSoundCaptureBuffer** ppDSCBuffer, void* pUnkOuter)
> { return pThis->lpVtbl->CreateCaptureBuffer(pThis, pDSCBufferDesc, ppDSCBuffer, pUnkOuter); }
25478 | static MA_INLINE HRESULT ma_IDirectSoundCapture_GetCaps(ma_IDirectSoundCapture* pThis,
> MA_DSCBCAPS* pDSCBCaps) { return pThis->lpVtbl->GetCaps(pThis, pDSCBCaps); }
25479 | static MA_INLINE HRESULT ma_IDirectSoundCapture_Initialize(ma_IDirectSoundCapture* pThis,
> const GUID* pGuidDevice) { return pThis->lpVtbl->Initialize(pThis, pGuidDevice); }
25480 |
25481 |
25482 | /* IDirectSoundCaptureBuffer */
25483 | typedef struct
25484 | {
25485 |     /* IUnknown */
25486 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IDirectSoundCaptureBuffer* pThis, const IID*
> const riid, void** ppObject);
25487 |     ULONG (STDMETHODCALLTYPE * AddRef)(ma_IDirectSoundCaptureBuffer* pThis);
25488 |     ULONG (STDMETHODCALLTYPE * Release)(ma_IDirectSoundCaptureBuffer* pThis);
25489 |
25490 |     /* IDirectSoundCaptureBuffer */
25491 |     HRESULT (STDMETHODCALLTYPE * GetCaps)(ma_IDirectSoundCaptureBuffer* pThis,
> MA_DSCBCAPS* pDSCBCaps);
25492 |     HRESULT (STDMETHODCALLTYPE * GetCurrentPosition)(ma_IDirectSoundCaptureBuffer* pThis, DWORD*
> pCapturePosition, DWORD* pReadPosition);
25493 |     HRESULT (STDMETHODCALLTYPE * GetFormat)(ma_IDirectSoundCaptureBuffer* pThis,
> MA_WAVEFORMATEX* pFormat, DWORD dwSizeAllocated, DWORD* pSizeWritten);
25494 |     HRESULT (STDMETHODCALLTYPE * GetStatus)(ma_IDirectSoundCaptureBuffer* pThis, DWORD*
> pStatus);
25495 |     HRESULT (STDMETHODCALLTYPE * Initialize)(ma_IDirectSoundCaptureBuffer* pThis,
> ma_IDirectSoundCapture* pDirectSoundCapture, const MA_DSCBUFFERDESC* pDSCBufferDesc);
25496 |     HRESULT (STDMETHODCALLTYPE * Lock)(ma_IDirectSoundCaptureBuffer* pThis, DWORD
> dwOffset, DWORD dwBytes, void** ppAudioPtr1, DWORD* pAudioBytes1, void** ppAudioPtr2, DWORD*
> pAudioBytes2, DWORD dwFlags);
25497 |     HRESULT (STDMETHODCALLTYPE * Start)(ma_IDirectSoundCaptureBuffer* pThis, DWORD
> dwFlags);
25498 |     HRESULT (STDMETHODCALLTYPE * Stop)(ma_IDirectSoundCaptureBuffer* pThis);
25499 |     HRESULT (STDMETHODCALLTYPE * Unlock)(ma_IDirectSoundCaptureBuffer* pThis, void*
> pAudioPtr1, DWORD dwAudioBytes1, void* pAudioPtr2, DWORD dwAudioBytes2);
25500 | } ma_IDirectSoundCaptureBufferVtbl;
25501 | struct ma_IDirectSoundCaptureBuffer
25502 | {
25503 |     ma_IDirectSoundCaptureBufferVtbl* lpVtbl;
25504 | };
25505 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_QueryInterface(ma_IDirectSoundCaptureBuffer*
> pThis, const IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid,
> ppObject); }
25506 | static MA_INLINE ULONG ma_IDirectSoundCaptureBuffer_AddRef(ma_IDirectSoundCaptureBuffer* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
25507 | static MA_INLINE ULONG ma_IDirectSoundCaptureBuffer_Release(ma_IDirectSoundCaptureBuffer* pThis)
> { return pThis->lpVtbl->Release(pThis); }
25508 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_GetCaps(ma_IDirectSoundCaptureBuffer* pThis,
> MA_DSCBCAPS* pDSCBCaps) { return pThis->lpVtbl->GetCaps(pThis, pDSCBCaps); }
25509 | static MA_INLINE HRESULT
> ma_IDirectSoundCaptureBuffer_GetCurrentPosition(ma_IDirectSoundCaptureBuffer* pThis, DWORD*
> pCapturePosition, DWORD* pReadPosition) { return pThis->lpVtbl->GetCurrentPosition(pThis,
> pCapturePosition, pReadPosition); }
25510 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_GetFormat(ma_IDirectSoundCaptureBuffer* pThis,
> MA_WAVEFORMATEX* pFormat, DWORD dwSizeAllocated, DWORD* pSizeWritten) { return
> pThis->lpVtbl->GetFormat(pThis, pFormat, dwSizeAllocated, pSizeWritten); }
25511 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_GetStatus(ma_IDirectSoundCaptureBuffer* pThis,
> DWORD* pStatus) { return pThis->lpVtbl->GetStatus(pThis, pStatus); }
25512 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_Initialize(ma_IDirectSoundCaptureBuffer*
> pThis, ma_IDirectSoundCapture* pDirectSoundCapture, const MA_DSCBUFFERDESC* pDSCBufferDesc) { return
> pThis->lpVtbl->Initialize(pThis, pDirectSoundCapture, pDSCBufferDesc); }
25513 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_Lock(ma_IDirectSoundCaptureBuffer* pThis,
> DWORD dwOffset, DWORD dwBytes, void** ppAudioPtr1, DWORD* pAudioBytes1, void** ppAudioPtr2, DWORD*
> pAudioBytes2, DWORD dwFlags) { return pThis->lpVtbl->Lock(pThis, dwOffset, dwBytes, ppAudioPtr1,
> pAudioBytes1, ppAudioPtr2, pAudioBytes2, dwFlags); }
25514 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_Start(ma_IDirectSoundCaptureBuffer* pThis,
> DWORD dwFlags) { return pThis->lpVtbl->Start(pThis, dwFlags); }
25515 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_Stop(ma_IDirectSoundCaptureBuffer* pThis)
> { return pThis->lpVtbl->Stop(pThis); }
25516 | static MA_INLINE HRESULT ma_IDirectSoundCaptureBuffer_Unlock(ma_IDirectSoundCaptureBuffer* pThis,
> void* pAudioPtr1, DWORD dwAudioBytes1, void* pAudioPtr2, DWORD dwAudioBytes2) { return
> pThis->lpVtbl->Unlock(pThis, pAudioPtr1, dwAudioBytes1, pAudioPtr2, dwAudioBytes2); }
25517 |
25518 |
25519 | /* IDirectSoundNotify */
25520 | typedef struct

```

```

25521 | {
25522 |     /* IUnknown */
25523 |     HRESULT (STDMETHODCALLTYPE * QueryInterface)(ma_IDirectSoundNotify* pThis, const IID* const
> riid, void** ppObject);
25524 |     ULONG (STDMETHODCALLTYPE * AddRef)          (ma_IDirectSoundNotify* pThis);
25525 |     ULONG (STDMETHODCALLTYPE * Release)         (ma_IDirectSoundNotify* pThis);
25526 |
25527 |     /* IDirectSoundNotify */
25528 |     HRESULT (STDMETHODCALLTYPE * SetNotificationPositions)(ma_IDirectSoundNotify* pThis, DWORD
> dwPositionNotifies, const MA_DSBPOSITIONNOTIFY* pPositionNotifies);
25529 | } ma_IDirectSoundNotifyVtbl;
25530 | struct ma_IDirectSoundNotify
25531 | {
25532 |     ma_IDirectSoundNotifyVtbl* lpVtbl;
25533 | };
25534 | static MA_INLINE HRESULT ma_IDirectSoundNotify_QueryInterface(ma_IDirectSoundNotify* pThis, const
> IID* const riid, void** ppObject) { return pThis->lpVtbl->QueryInterface(pThis, riid, ppObject); }
25535 | static MA_INLINE ULONG ma_IDirectSoundNotify_AddRef(ma_IDirectSoundNotify* pThis)
> { return pThis->lpVtbl->AddRef(pThis); }
25536 | static MA_INLINE ULONG ma_IDirectSoundNotify_Release(ma_IDirectSoundNotify* pThis)
> { return pThis->lpVtbl->Release(pThis); }
25537 | static MA_INLINE HRESULT ma_IDirectSoundNotify_SetNotificationPositions(ma_IDirectSoundNotify*
> pThis, DWORD dwPositionNotifies, const MA_DSBPOSITIONNOTIFY* pPositionNotifies) { return
> pThis->lpVtbl->SetNotificationPositions(pThis, dwPositionNotifies, pPositionNotifies); }
25538 |
25539 |
25540 | typedef BOOL (CALLBACK * ma_DSEnumCallbackAProc) (GUID* pDeviceGUID, const char*
> pDeviceDescription, const char* pModule, void* pContext);
25541 | typedef HRESULT (WINAPI * ma_DirectSoundCreateProc) (const GUID* pcGuidDevice,
> ma_IDirectSound** ppDS8, ma_IUnknown* pUnkOuter);
25542 | typedef HRESULT (WINAPI * ma_DirectSoundEnumerateAProc) (ma_DSEnumCallbackAProc
> pDSEnumCallback, void* pContext);
25543 | typedef HRESULT (WINAPI * ma_DirectSoundCaptureCreateProc) (const GUID* pcGuidDevice,
> ma_IDirectSoundCapture** ppDSC8, ma_IUnknown* pUnkOuter);
25544 | typedef HRESULT (WINAPI * ma_DirectSoundCaptureEnumerateAProc)(ma_DSEnumCallbackAProc
> pDSEnumCallback, void* pContext);
25545 |
25546 | static ma_uint32 ma_get_best_sample_rate_within_range(ma_uint32 sampleRateMin, ma_uint32
> sampleRateMax)
25547 | {
25548 |     /* Normalize the range in case we were given something stupid. */
25549 |     if (sampleRateMin < (ma_uint32)ma_standard_sample_rate_min) {
25550 |         sampleRateMin = (ma_uint32)ma_standard_sample_rate_min;
25551 |     }
25552 |     if (sampleRateMax > (ma_uint32)ma_standard_sample_rate_max) {
25553 |         sampleRateMax = (ma_uint32)ma_standard_sample_rate_max;
25554 |     }
25555 |     if (sampleRateMin > sampleRateMax) {
25556 |         sampleRateMin = sampleRateMax;
25557 |     }
25558 |
25559 |     if (sampleRateMin == sampleRateMax) {
25560 |         return sampleRateMax;
25561 |     } else {
25562 |         size_t iStandardRate;
25563 |         for (iStandardRate = 0; iStandardRate < ma_countof(g_maStandardSampleRatePriorities);
> ++iStandardRate) {
25564 |             ma_uint32 standardRate = g_maStandardSampleRatePriorities[iStandardRate];
25565 |             if (standardRate >= sampleRateMin && standardRate <= sampleRateMax) {
25566 |                 return standardRate;
25567 |             }
25568 |         }
25569 |     }
25570 |
25571 |     /* Should never get here. */
25572 |     MA_ASSERT(MA_FALSE);
25573 |     return 0;
25574 | }
25575 |
25576 | /*
25577 | > Retrieves the channel count and channel map for the given speaker configuration. If the speaker
> configuration is unknown,
25578 | the channel count and channel map will be left unmodified.
25579 | */
25580 | static void ma_get_channels_from_speaker_config_dsound(DWORD speakerConfig, WORD* pChannelsOut,
> DWORD* pChannelMapOut)
25581 | {
25582 |     WORD channels;
25583 |     DWORD channelMap;

```



```

25584 |
25585 |     channels = 0;
25586 |     if (pChannelsOut != NULL) {
25587 |         channels = *pChannelsOut;
25588 |     }
25589 |
25590 |     channelMap = 0;
25591 |     if (pChannelMapOut != NULL) {
25592 |         channelMap = *pChannelMapOut;
25593 |     }
25594 |
25595 |     /*
25596 |     The speaker configuration is a combination of speaker config and speaker geometry. The lower 8
> bits is what we care about. The upper
25597 |     16 bits is for the geometry.
25598 |     */
25599 |     switch ((BYTE)(speakerConfig)) {
25600 |         case 1 /*DSSPEAKER_HEADPHONE*/: channels = 2; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT; break;
25601 |         case 2 /*DSSPEAKER_MONO*/: channels = 1; channelMap =
> SPEAKER_FRONT_CENTER; break;
25602 |         case 3 /*DSSPEAKER_QUAD*/: channels = 4; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT | SPEAKER_BACK_LEFT | SPEAKER_BACK_RIGHT; break;
25603 |         case 4 /*DSSPEAKER_STEREO*/: channels = 2; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT; break;
25604 |         case 5 /*DSSPEAKER_SURROUND*/: channels = 4; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT | SPEAKER_FRONT_CENTER | SPEAKER_BACK_CENTER; break;
25605 |         case 6 /*DSSPEAKER_5POINT1_BACK*/ /*DSSPEAKER_5POINT1*/: channels = 6; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT | SPEAKER_FRONT_CENTER | SPEAKER_LOW_FREQUENCY |
> SPEAKER_BACK_LEFT | SPEAKER_BACK_RIGHT; break;
25606 |         case 7 /*DSSPEAKER_7POINT1_WIDE*/ /*DSSPEAKER_7POINT1*/: channels = 8; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT | SPEAKER_FRONT_CENTER | SPEAKER_LOW_FREQUENCY |
> SPEAKER_BACK_LEFT | SPEAKER_BACK_RIGHT | SPEAKER_FRONT_LEFT_OF_CENTER |
> SPEAKER_FRONT_RIGHT_OF_CENTER; break;
25607 |         case 8 /*DSSPEAKER_7POINT1_SURROUND*/: channels = 8; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT | SPEAKER_FRONT_CENTER | SPEAKER_LOW_FREQUENCY |
> SPEAKER_BACK_LEFT | SPEAKER_BACK_RIGHT | SPEAKER_SIDE_LEFT | SPEAKER_SIDE_RIGHT; break;
25608 |         case 9 /*DSSPEAKER_5POINT1_SURROUND*/: channels = 6; channelMap =
> SPEAKER_FRONT_LEFT | SPEAKER_FRONT_RIGHT | SPEAKER_FRONT_CENTER | SPEAKER_LOW_FREQUENCY |
> SPEAKER_SIDE_LEFT | SPEAKER_SIDE_RIGHT; break;
25609 |     default: break;
25610 |     }
25611 |
25612 |     if (pChannelsOut != NULL) {
25613 |         *pChannelsOut = channels;
25614 |     }
25615 |
25616 |     if (pChannelMapOut != NULL) {
25617 |         *pChannelMapOut = channelMap;
25618 |     }
25619 | }
25620 |
25621 |
25622 | static ma_result ma_context_create_IDirectSound_dsound(ma_context* pContext, ma_share_mode
> shareMode, const ma_device_id* pDeviceID, ma_IDirectSound** ppDirectSound)
25623 | {
25624 |     ma_IDirectSound* pDirectSound;
25625 |     HWND hWnd;
25626 |     HRESULT hr;
25627 |
25628 |     MA_ASSERT(pContext != NULL);
25629 |     MA_ASSERT(ppDirectSound != NULL);
25630 |
25631 |     *ppDirectSound = NULL;
25632 |     pDirectSound = NULL;
25633 |
25634 |     if (FAILED(((ma_DirectSoundCreateProc)pContext->dsound.DirectSoundCreate)((pDeviceID == NULL) ?
> NULL : (const GUID*)pDeviceID->dsound, &pDirectSound, NULL))) {
25635 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[DirectSound]
> DirectSoundCreate() failed for playback device.");
25636 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
25637 |     }
25638 |
25639 |     /* The cooperative level must be set before doing anything else. */
25640 |     hWnd = (HWND)pContext->dsound.hWnd;
25641 |     if (hWnd == 0) {
25642 |         hWnd = ((MA_PFN_GetForegroundWindow)pContext->win32.GetForegroundWindow)();
25643 |         if (hWnd == 0) {
25644 |             hWnd = ((MA_PFN_GetDesktopWindow)pContext->win32.GetDesktopWindow)();

```

```

25645 |     }
25646 | }
25647 |
25648 |     hr = ma_IDirectSound_SetCooperativeLevel(pDirectSound, hWnd, (shareMode ==
> ma_share_mode_exclusive) ? MA_DSSCL_EXCLUSIVE : MA_DSSCL_PRIORITY);
25649 |     if (FAILED(hr)) {
25650 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSound_SetCooperativeLevel() failed for playback device.");
25651 |         return ma_result_from_HRESULT(hr);
25652 |     }
25653 |
25654 |     *ppDirectSound = pDirectSound;
25655 |     return MA_SUCCESS;
25656 | }
25657 |
25658 | static ma_result ma_context_create_IDirectSoundCapture_dsound(ma_context* pContext, ma_share_mode
> shareMode, const ma_device_id* pDeviceID, ma_IDirectSoundCapture** ppDirectSoundCapture)
25659 | {
25660 |     ma_IDirectSoundCapture* pDirectSoundCapture;
25661 |     HRESULT hr;
25662 |
25663 |     MA_ASSERT(pContext != NULL);
25664 |     MA_ASSERT(ppDirectSoundCapture != NULL);
25665 |
25666 |     /* DirectSound does not support exclusive mode for capture. */
25667 |     if (shareMode == ma_share_mode_exclusive) {
25668 |         return MA_SHARE_MODE_NOT_SUPPORTED;
25669 |     }
25670 |
25671 |     *ppDirectSoundCapture = NULL;
25672 |     pDirectSoundCapture = NULL;
25673 |
25674 |     hr = ((ma_DirectSoundCaptureCreateProc)pContext->dsound.DirectSoundCaptureCreate)((pDeviceID ==
> NULL) ? NULL : (const GUID*)pDeviceID->dsound, &pDirectSoundCapture, NULL);
25675 |     if (FAILED(hr)) {
25676 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[DirectSound]
> DirectSoundCaptureCreate() failed for capture device.");
25677 |         return ma_result_from_HRESULT(hr);
25678 |     }
25679 |
25680 |     *ppDirectSoundCapture = pDirectSoundCapture;
25681 |     return MA_SUCCESS;
25682 | }
25683 |
25684 | static ma_result ma_context_get_format_info_for_IDirectSoundCapture_dsound(ma_context* pContext,
> ma_IDirectSoundCapture* pDirectSoundCapture, WORD* pChannels, WORD* pBitsPerSample, DWORD*
> pSampleRate)
25685 | {
25686 |     HRESULT hr;
25687 |     MA_DSCCAPS caps;
25688 |     WORD bitsPerSample;
25689 |     DWORD sampleRate;
25690 |
25691 |     MA_ASSERT(pContext != NULL);
25692 |     MA_ASSERT(pDirectSoundCapture != NULL);
25693 |
25694 |     if (pChannels) {
25695 |         *pChannels = 0;
25696 |     }
25697 |     if (pBitsPerSample) {
25698 |         *pBitsPerSample = 0;
25699 |     }
25700 |     if (pSampleRate) {
25701 |         *pSampleRate = 0;
25702 |     }
25703 |
25704 |     MA_ZERO_OBJECT(&caps);
25705 |     caps.dwSize = sizeof(caps);
25706 |     hr = ma_IDirectSoundCapture_GetCaps(pDirectSoundCapture, &caps);
25707 |     if (FAILED(hr)) {
25708 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundCapture_GetCaps() failed for capture device.");
25709 |         return ma_result_from_HRESULT(hr);
25710 |     }
25711 |
25712 |     if (pChannels) {
25713 |         *pChannels = (WORD)caps.dwChannels;
25714 |     }
25715 |

```

```

25716 | /* The device can support multiple formats. We just go through the different formats in order of
> | priority and pick the first one. This the same type of system as the WinMM backend. */
25717 |     bitsPerSample = 16;
25718 |     sampleRate = 48000;
25719 |
25720 |     if (caps.dwChannels == 1) {
25721 |         if ((caps.dwFormats & WAVE_FORMAT_48M16) != 0) {
25722 |             sampleRate = 48000;
25723 |         } else if ((caps.dwFormats & WAVE_FORMAT_44M16) != 0) {
25724 |             sampleRate = 44100;
25725 |         } else if ((caps.dwFormats & WAVE_FORMAT_2M16) != 0) {
25726 |             sampleRate = 22050;
25727 |         } else if ((caps.dwFormats & WAVE_FORMAT_1M16) != 0) {
25728 |             sampleRate = 11025;
25729 |         } else if ((caps.dwFormats & WAVE_FORMAT_96M16) != 0) {
25730 |             sampleRate = 96000;
25731 |         } else {
25732 |             bitsPerSample = 8;
25733 |             if ((caps.dwFormats & WAVE_FORMAT_48M08) != 0) {
25734 |                 sampleRate = 48000;
25735 |             } else if ((caps.dwFormats & WAVE_FORMAT_44M08) != 0) {
25736 |                 sampleRate = 44100;
25737 |             } else if ((caps.dwFormats & WAVE_FORMAT_2M08) != 0) {
25738 |                 sampleRate = 22050;
25739 |             } else if ((caps.dwFormats & WAVE_FORMAT_1M08) != 0) {
25740 |                 sampleRate = 11025;
25741 |             } else if ((caps.dwFormats & WAVE_FORMAT_96M08) != 0) {
25742 |                 sampleRate = 96000;
25743 |             } else {
25744 |                 bitsPerSample = 16; /* Didn't find it. Just fall back to 16-bit. */
25745 |             }
25746 |         }
25747 |     } else if (caps.dwChannels == 2) {
25748 |         if ((caps.dwFormats & WAVE_FORMAT_48S16) != 0) {
25749 |             sampleRate = 48000;
25750 |         } else if ((caps.dwFormats & WAVE_FORMAT_44S16) != 0) {
25751 |             sampleRate = 44100;
25752 |         } else if ((caps.dwFormats & WAVE_FORMAT_2S16) != 0) {
25753 |             sampleRate = 22050;
25754 |         } else if ((caps.dwFormats & WAVE_FORMAT_1S16) != 0) {
25755 |             sampleRate = 11025;
25756 |         } else if ((caps.dwFormats & WAVE_FORMAT_96S16) != 0) {
25757 |             sampleRate = 96000;
25758 |         } else {
25759 |             bitsPerSample = 8;
25760 |             if ((caps.dwFormats & WAVE_FORMAT_48S08) != 0) {
25761 |                 sampleRate = 48000;
25762 |             } else if ((caps.dwFormats & WAVE_FORMAT_44S08) != 0) {
25763 |                 sampleRate = 44100;
25764 |             } else if ((caps.dwFormats & WAVE_FORMAT_2S08) != 0) {
25765 |                 sampleRate = 22050;
25766 |             } else if ((caps.dwFormats & WAVE_FORMAT_1S08) != 0) {
25767 |                 sampleRate = 11025;
25768 |             } else if ((caps.dwFormats & WAVE_FORMAT_96S08) != 0) {
25769 |                 sampleRate = 96000;
25770 |             } else {
25771 |                 bitsPerSample = 16; /* Didn't find it. Just fall back to 16-bit. */
25772 |             }
25773 |         }
25774 |     }
25775 |
25776 |     if (pBitsPerSample) {
25777 |         *pBitsPerSample = bitsPerSample;
25778 |     }
25779 |     if (pSampleRate) {
25780 |         *pSampleRate = sampleRate;
25781 |     }
25782 |
25783 |     return MA_SUCCESS;
25784 | }
25785 |
25786 |
25787 | typedef struct
25788 | {
25789 |     ma_context* pContext;
25790 |     ma_device_type deviceType;
25791 |     ma_enum_devices_callback_proc callback;
25792 |     void* pUserData;
25793 |     ma_bool32 terminated;

```

```

25794 | } ma_context_enumerate_devices_callback_data__dsound;
25795 |
25796 | static BOOL CALLBACK ma_context_enumerate_devices_callback__dsound(GUID* lpGuid, const char*
> lpcstrDescription, const char* lpcstrModule, void* lpContext)
25797 | {
25798 |     ma_context_enumerate_devices_callback_data__dsound* pData =
> (ma_context_enumerate_devices_callback_data__dsound*)lpContext;
25799 |     ma_device_info deviceInfo;
25800 |
25801 |     (void)lpcstrModule;
25802 |
25803 |     MA_ZERO_OBJECT(&deviceInfo);
25804 |
25805 |     /* ID. */
25806 |     if (lpGuid != NULL) {
25807 |         MA_COPY_MEMORY(deviceInfo.id.dsound, lpGuid, 16);
25808 |     } else {
25809 |         MA_ZERO_MEMORY(deviceInfo.id.dsound, 16);
25810 |         deviceInfo.isDefault = MA_TRUE;
25811 |     }
25812 |
25813 |     /* Name / Description */
25814 |     ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), lpcstrDescription, (size_t)-1);
25815 |
25816 |
25817 |     /* Call the callback function, but make sure we stop enumerating if the callee requested so. */
25818 |     MA_ASSERT(pData != NULL);
25819 |     pData->terminated = (pData->callback(pData->pContext, pData->deviceType, &deviceInfo,
> pData->pUserData) == MA_FALSE);
25820 |     if (pData->terminated) {
25821 |         return FALSE; /* Stop enumeration. */
25822 |     } else {
25823 |         return TRUE; /* Continue enumeration. */
25824 |     }
25825 | }
25826 |
25827 | static ma_result ma_context_enumerate_devices__dsound(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
25828 | {
25829 |     ma_context_enumerate_devices_callback_data__dsound data;
25830 |
25831 |     MA_ASSERT(pContext != NULL);
25832 |     MA_ASSERT(callback != NULL);
25833 |
25834 |     data.pContext = pContext;
25835 |     data.callback = callback;
25836 |     data.pUserData = pUserData;
25837 |     data.terminated = MA_FALSE;
25838 |
25839 |     /* Playback. */
25840 |     if (!data.terminated) {
25841 |         data.deviceType = ma_device_type_playback;
25842 |         ((ma_DirectSoundEnumerateAProc)pContext->dsound.DirectSoundEnumerateA)(ma_context_enumerate_
> devices_callback__dsound, &data);
25843 |     }
25844 |
25845 |     /* Capture. */
25846 |     if (!data.terminated) {
25847 |         data.deviceType = ma_device_type_capture;
25848 |         ((ma_DirectSoundCaptureEnumerateAProc)pContext->dsound.DirectSoundCaptureEnumerateA)(ma_cont
> ext_enumerate_devices_callback__dsound, &data);
25849 |     }
25850 |
25851 |     return MA_SUCCESS;
25852 | }
25853 |
25854 |
25855 | typedef struct
25856 | {
25857 |     const ma_device_id* pDeviceID;
25858 |     ma_device_info* pDeviceInfo;
25859 |     ma_bool32 found;
25860 | } ma_context_get_device_info_callback_data__dsound;
25861 |
25862 | static BOOL CALLBACK ma_context_get_device_info_callback__dsound(GUID* lpGuid, const char*
> lpcstrDescription, const char* lpcstrModule, void* lpContext)
25863 | {
25864 |     ma_context_get_device_info_callback_data__dsound* pData =
> (ma_context_get_device_info_callback_data__dsound*)lpContext;

```

```

25865 |     MA_ASSERT(pData != NULL);
25866 |
25867 |     if ((pData->pDeviceID == NULL || ma_is_guid_null(pData->pDeviceID->dsound)) && (lpGuid == NULL
> || ma_is_guid_null(lpGuid))) {
25868 |         /* Default device. */
25869 |         ma_strncpy_s(pData->pDeviceInfo->name, sizeof(pData->pDeviceInfo->name), lpCTSTRDescription,
> (size_t)-1);
25870 |         pData->pDeviceInfo->isDefault = MA_TRUE;
25871 |         pData->found = MA_TRUE;
25872 |         return FALSE; /* Stop enumeration. */
25873 |     } else {
25874 |         /* Not the default device. */
25875 |         if (lpGuid != NULL && pData->pDeviceID != NULL) {
25876 |             if (memcmp(pData->pDeviceID->dsound, lpGuid, sizeof(pData->pDeviceID->dsound)) == 0) {
25877 |                 ma_strncpy_s(pData->pDeviceInfo->name, sizeof(pData->pDeviceInfo->name),
> lpCTSTRDescription, (size_t)-1);
25878 |                 pData->found = MA_TRUE;
25879 |                 return FALSE; /* Stop enumeration. */
25880 |             }
25881 |         }
25882 |     }
25883 |
25884 |     (void)lpCTSTRModule;
25885 |     return TRUE;
25886 | }
25887 |
25888 | static ma_result ma_context_get_device_info_dsound(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
25889 | {
25890 |     ma_result result;
25891 |     HRESULT hr;
25892 |
25893 |     if (pDeviceID != NULL) {
25894 |         ma_context_get_device_info_callback_data_dsound data;
25895 |
25896 |         /* ID. */
25897 |         MA_COPY_MEMORY(pDeviceInfo->id.dsound, pDeviceID->dsound, 16);
25898 |
25899 |         /* Name / Description. This is retrieved by enumerating over each device until we find that
> one that matches the input ID. */
25900 |         data.pDeviceID = pDeviceID;
25901 |         data.pDeviceInfo = pDeviceInfo;
25902 |         data.found = MA_FALSE;
25903 |         if (deviceType == ma_device_type_playback) {
25904 |             ((ma_DirectSoundEnumerateAProc)pContext->dsound.DirectSoundEnumerateA)(ma_context_get_de
> vice_info_callback_dsound, &data);
25905 |         } else {
25906 |             ((ma_DirectSoundCaptureEnumerateAProc)pContext->dsound.DirectSoundCaptureEnumerateA)(ma_
> context_get_device_info_callback_dsound, &data);
25907 |         }
25908 |
25909 |         if (!data.found) {
25910 |             return MA_NO_DEVICE;
25911 |         }
25912 |     } else {
25913 |         /* I don't think there's a way to get the name of the default device with DirectSound. In
> this case we just need to use defaults. */
25914 |
25915 |         /* ID */
25916 |         MA_ZERO_MEMORY(pDeviceInfo->id.dsound, 16);
25917 |
25918 |         /* Name / Description */
25919 |         if (deviceType == ma_device_type_playback) {
25920 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name),
> MA_DEFAULT_PLAYBACK_DEVICE_NAME, (size_t)-1);
25921 |         } else {
25922 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name),
> MA_DEFAULT_CAPTURE_DEVICE_NAME, (size_t)-1);
25923 |         }
25924 |
25925 |         pDeviceInfo->isDefault = MA_TRUE;
25926 |     }
25927 |
25928 |     /* Retrieving detailed information is slightly different depending on the device type. */
25929 |     if (deviceType == ma_device_type_playback) {
25930 |         /* Playback. */
25931 |         ma_IDirectSound* pDirectSound;
25932 |         MA_DSCAPS caps;
25933 |         WORD channels;

```

```

25934 |
25935 |         result = ma_context_create_IDirectSound__dsound(pContext, ma_share_mode_shared, pDeviceID,
> &pDirectSound);
25936 |         if (result != MA_SUCCESS) {
25937 |             return result;
25938 |         }
25939 |
25940 |         MA_ZERO_OBJECT(&caps);
25941 |         caps.dwSize = sizeof(caps);
25942 |         hr = ma_IDirectSound_GetCaps(pDirectSound, &caps);
25943 |         if (FAILED(hr)) {
25944 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSound_GetCaps() failed for playback device.");
25945 |             return ma_result_from_HRESULT(hr);
25946 |         }
25947 |
25948 |
25949 |         /* Channels. Only a single channel count is reported for DirectSound. */
25950 |         if ((caps.dwFlags & MA_DSCAPS_PRIMARYSTEREO) != 0) {
25951 |             /* It supports at least stereo, but could support more. */
25952 |             DWORD speakerConfig;
25953 |
25954 |             channels = 2;
25955 |
25956 |             /* Look at the speaker configuration to get a better idea on the channel count. */
25957 |             hr = ma_IDirectSound_GetSpeakerConfig(pDirectSound, &speakerConfig);
25958 |             if (SUCCEEDED(hr)) {
25959 |                 ma_get_channels_from_speaker_config__dsound(speakerConfig, &channels, NULL);
25960 |             }
25961 |         } else {
25962 |             /* It does not support stereo, which means we are stuck with mono. */
25963 |             channels = 1;
25964 |         }
25965 |
25966 |
25967 |         /*
25968 |         In DirectSound, our native formats are centered around sample rates. All formats are
> supported, and we're only reporting a single channel
25969 |         count. However, DirectSound can report a range of supported sample rates. We're only going
> to include standard rates known by miniaudio
25970 |         in order to keep the size of this within reason.
25971 |         */
25972 |         if ((caps.dwFlags & MA_DSCAPS_CONTINUOUSRATE) != 0) {
25973 |             /* Multiple sample rates are supported. We'll report in order of our preferred sample
> rates. */
25974 |             size_t iStandardSampleRate;
25975 |             for (iStandardSampleRate = 0; iStandardSampleRate <
> ma_countof(g_maStandardSampleRatePriorities); iStandardSampleRate += 1) {
25976 |                 ma_uint32 sampleRate = g_maStandardSampleRatePriorities[iStandardSampleRate];
25977 |                 if (sampleRate >= caps.dwMinSecondarySampleRate && sampleRate <=
> caps.dwMaxSecondarySampleRate) {
25978 |                     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format      =
> ma_format_unknown;
25979 |                     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels    =
> channels;
25980 |                     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate =
> sampleRate;
25981 |                     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags      =
> 0;
25982 |                     pDeviceInfo->nativeDataFormatCount += 1;
25983 |                 }
25984 |             }
25985 |         } else {
25986 |             /* Only a single sample rate is supported. */
25987 |             pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format      =
> ma_format_unknown;
25988 |             pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels    =
> channels;
25989 |             pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate =
> caps.dwMaxSecondarySampleRate;
25990 |             pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags      = 0;
25991 |             pDeviceInfo->nativeDataFormatCount += 1;
25992 |         }
25993 |
25994 |         ma_IDirectSound_Release(pDirectSound);
25995 |     } else {
25996 |         /*
25997 |         Capture. This is a little different to playback due to the say the supported formats are
> reported. Technically capture

```

```

25998 |         devices can support a number of different formats, but for simplicity and consistency with
> ma_device_init() I'm just
25999 |         reporting the best format.
26000 |         */
26001 |         ma_IDirectSoundCapture* pDirectSoundCapture;
26002 |         WORD channels;
26003 |         WORD bitsPerSample;
26004 |         DWORD sampleRate;
26005 |
26006 |         result = ma_context_create_IDirectSoundCapture_dsound(pContext, ma_share_mode_shared,
> pDeviceID, &pDirectSoundCapture);
26007 |         if (result != MA_SUCCESS) {
26008 |             return result;
26009 |         }
26010 |
26011 |         result = ma_context_get_format_info_for_IDirectSoundCapture_dsound(pContext,
> pDirectSoundCapture, &channels, &bitsPerSample, &sampleRate);
26012 |         if (result != MA_SUCCESS) {
26013 |             ma_IDirectSoundCapture_Release(pDirectSoundCapture);
26014 |             return result;
26015 |         }
26016 |
26017 |         ma_IDirectSoundCapture_Release(pDirectSoundCapture);
26018 |
26019 |         /* The format is always an integer format and is based on the bits per sample. */
26020 |         if (bitsPerSample == 8) {
26021 |             pDeviceInfo->nativeDataFormats[0].format = ma_format_u8;
26022 |         } else if (bitsPerSample == 16) {
26023 |             pDeviceInfo->nativeDataFormats[0].format = ma_format_s16;
26024 |         } else if (bitsPerSample == 24) {
26025 |             pDeviceInfo->nativeDataFormats[0].format = ma_format_s24;
26026 |         } else if (bitsPerSample == 32) {
26027 |             pDeviceInfo->nativeDataFormats[0].format = ma_format_s32;
26028 |         } else {
26029 |             return MA_FORMAT_NOT_SUPPORTED;
26030 |         }
26031 |
26032 |         pDeviceInfo->nativeDataFormats[0].channels = channels;
26033 |         pDeviceInfo->nativeDataFormats[0].sampleRate = sampleRate;
26034 |         pDeviceInfo->nativeDataFormats[0].flags = 0;
26035 |         pDeviceInfo->nativeDataFormatCount = 1;
26036 |     }
26037 |
26038 |     return MA_SUCCESS;
26039 | }
26040 |
26041 |
26042 |
26043 | static ma_result ma_device_uninit_dsound(ma_device* pDevice)
26044 | {
26045 |     MA_ASSERT(pDevice != NULL);
26046 |
26047 |     if (pDevice->dsound.pCaptureBuffer != NULL) {
26048 |
> ma_IDirectSoundCaptureBuffer_Release((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer);
26049 |     }
26050 |     if (pDevice->dsound.pCapture != NULL) {
26051 |         ma_IDirectSoundCapture_Release((ma_IDirectSoundCapture*)pDevice->dsound.pCapture);
26052 |     }
26053 |
26054 |     if (pDevice->dsound.pPlaybackBuffer != NULL) {
26055 |         ma_IDirectSoundBuffer_Release((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer);
26056 |     }
26057 |     if (pDevice->dsound.pPlaybackPrimaryBuffer != NULL) {
26058 |
> ma_IDirectSoundBuffer_Release((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackPrimaryBuffer);
26059 |     }
26060 |     if (pDevice->dsound.pPlayback != NULL) {
26061 |         ma_IDirectSound_Release((ma_IDirectSound*)pDevice->dsound.pPlayback);
26062 |     }
26063 |
26064 |     return MA_SUCCESS;
26065 | }
26066 |
26067 | static ma_result ma_config_to_WAVEFORMATEXTENSIBLE(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, const ma_channel* pChannelMap, MA_WAVEFORMATEXTENSIBLE* pWF)
26068 | {
26069 |     GUID subformat;
26070 |

```



```

26071 |     if (format == ma_format_unknown) {
26072 |         format = MA_DEFAULT_FORMAT;
26073 |     }
26074 |
26075 |     if (channels == 0) {
26076 |         channels = MA_DEFAULT_CHANNELS;
26077 |     }
26078 |
26079 |     if (sampleRate == 0) {
26080 |         sampleRate = MA_DEFAULT_SAMPLE_RATE;
26081 |     }
26082 |
26083 |     switch (format)
26084 |     {
26085 |         case ma_format_u8:
26086 |         case ma_format_s16:
26087 |         case ma_format_s24:
26088 |         /*case ma_format_s24_32:*/
26089 |         case ma_format_s32:
26090 |         {
26091 |             subformat = MA_GUID_KSDATAFORMAT_SUBTYPE_PCM;
26092 |         } break;
26093 |
26094 |         case ma_format_f32:
26095 |         {
26096 |             subformat = MA_GUID_KSDATAFORMAT_SUBTYPE_IEEE_FLOAT;
26097 |         } break;
26098 |
26099 |         default:
26100 |             return MA_FORMAT_NOT_SUPPORTED;
26101 |     }
26102 |
26103 |     MA_ZERO_OBJECT(pWF);
26104 |     pWF->cbSize = sizeof(*pWF);
26105 |     pWF->wFormatTag = WAVE_FORMAT_EXTENSIBLE;
26106 |     pWF->nChannels = (WORD)channels;
26107 |     pWF->nSamplesPerSec = (DWORD)sampleRate;
26108 |     pWF->wBitsPerSample = (WORD)(ma_get_bytes_per_sample(format)*8);
26109 |     pWF->nBlockAlign = (WORD)(pWF->nChannels * pWF->wBitsPerSample / 8);
26110 |     pWF->nAvgBytesPerSec = pWF->nBlockAlign * pWF->nSamplesPerSec;
26111 |     pWF->Samples.wValidBitsPerSample = pWF->wBitsPerSample;
26112 |     pWF->dwChannelMask = ma_channel_map_to_channel_mask_win32(pChannelMap, channels);
26113 |     pWF->SubFormat = subformat;
26114 |
26115 |     return MA_SUCCESS;
26116 | }
26117 |
26118 | static ma_uint32 ma_calculate_period_size_in_frames_from_descriptor_dsound(const
> ma_device_descriptor* pDescriptor, ma_uint32 nativeSampleRate, ma_performance_profile
> performanceProfile)
26119 | {
26120 |     /*
26121 |     DirectSound has a minimum period size of 20ms. In practice, this doesn't seem to be enough for
26122 |     reliable glitch-free processing so going to use 30ms instead.
26123 |     */
26124 |     ma_uint32 minPeriodSizeInFrames = ma_calculate_buffer_size_in_frames_from_milliseconds(30,
> nativeSampleRate);
26125 |     ma_uint32 periodSizeInFrames;
26126 |
26127 |     periodSizeInFrames = ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor,
> nativeSampleRate, performanceProfile);
26128 |     if (periodSizeInFrames < minPeriodSizeInFrames) {
26129 |         periodSizeInFrames = minPeriodSizeInFrames;
26130 |     }
26131 |
26132 |     return periodSizeInFrames;
26133 | }
26134 |
26135 | static ma_result ma_device_init_dsound(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
26136 | {
26137 |     ma_result result;
26138 |     HRESULT hr;
26139 |
26140 |     MA_ASSERT(pDevice != NULL);
26141 |
26142 |     MA_ZERO_OBJECT(&pDevice->dsound);
26143 |
26144 |     if (pConfig->deviceType == ma_device_type_loopback) {

```

```

26145 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
26146 |     }
26147 |
26148 |     /*
26149 |     Unfortunately DirectSound uses different APIs and data structures for playback and capture
> devices. We need to initialize
26150 |     the capture device first because we'll want to match its buffer size and period count on the
> playback side if we're using
26151 |     full-duplex mode.
26152 |     */
26153 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
26154 |         MA_WAVEFORMATEXTENSIBLE wf;
26155 |         MA_DSCBUFFERDESC descDS;
26156 |         ma_uint32 periodSizeInFrames;
26157 |         ma_uint32 periodCount;
26158 |         char rawdata[1024]; /* <-- Ugly hack to avoid a malloc() due to a crappy DirectSound API. */
26159 |         MA_WAVEFORMATEXTENSIBLE* pActualFormat;
26160 |
26161 |         result = ma_config_to_WAVEFORMATEXTENSIBLE(pDescriptorCapture->format,
> pDescriptorCapture->channels, pDescriptorCapture->sampleRate, pDescriptorCapture->channelMap, &wf);
26162 |         if (result != MA_SUCCESS) {
26163 |             return result;
26164 |         }
26165 |
26166 |         result = ma_context_create_IDirectSoundCapture__dsound(pDevice->pContext,
> pDescriptorCapture->shareMode, pDescriptorCapture->pDeviceID,
> (ma_IDirectSoundCapture**) &pDevice->dsound.pCapture);
26167 |         if (result != MA_SUCCESS) {
26168 |             ma_device_uninit__dsound(pDevice);
26169 |             return result;
26170 |         }
26171 |
26172 |         result = ma_context_get_format_info_for_IDirectSoundCapture__dsound(pDevice->pContext,
> (ma_IDirectSoundCapture*)pDevice->dsound.pCapture, &wf.nChannels, &wf.wBitsPerSample,
> &wf.nSamplesPerSec);
26173 |         if (result != MA_SUCCESS) {
26174 |             ma_device_uninit__dsound(pDevice);
26175 |             return result;
26176 |         }
26177 |
26178 |         wf.nBlockAlign = (WORD)(wf.nChannels * wf.wBitsPerSample / 8);
26179 |         wf.nAvgBytesPerSec = wf.nBlockAlign * wf.nSamplesPerSec;
26180 |         wf.Samples.wValidBitsPerSample = wf.wBitsPerSample;
26181 |         wf.SubFormat = MA_GUID_KSDATAFORMAT_SUBTYPE_PCM;
26182 |
26183 |         /* The size of the buffer must be a clean multiple of the period count. */
26184 |         periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__dsound(pDescriptorCapture, wf.nSamplesPerSec,
> pConfig->performanceProfile);
26185 |         periodCount = (pDescriptorCapture->periodCount > 0) ? pDescriptorCapture->periodCount :
> MA_DEFAULT_PERIODS;
26186 |
26187 |         MA_ZERO_OBJECT(&descDS);
26188 |         descDS.dwSize = sizeof(descDS);
26189 |         descDS.dwFlags = 0;
26190 |         descDS.dwBufferBytes = periodSizeInFrames * periodCount * wf.nBlockAlign;
26191 |         descDS.lpwfxFormat = (MA_WAVEFORMATEX*)&wf;
26192 |         hr =
> ma_IDirectSoundCapture_CreateCaptureBuffer((ma_IDirectSoundCapture*)pDevice->dsound.pCapture,
> &descDS, (ma_IDirectSoundCaptureBuffer**) &pDevice->dsound.pCaptureBuffer, NULL);
26193 |         if (FAILED(hr)) {
26194 |             ma_device_uninit__dsound(pDevice);
26195 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundCapture_CreateCaptureBuffer() failed for capture device.");
26196 |             return ma_result_from_HRESULT(hr);
26197 |         }
26198 |
26199 |         /* Get the _actual_ properties of the buffer. */
26200 |         pActualFormat = (MA_WAVEFORMATEXTENSIBLE*)rawdata;
26201 |         hr = ma_IDirectSoundCaptureBuffer_GetFormat((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.p
> CaptureBuffer, (MA_WAVEFORMATEX*)pActualFormat, sizeof(rawdata), NULL);
26202 |         if (FAILED(hr)) {
26203 |             ma_device_uninit__dsound(pDevice);
26204 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound] Failed to
> retrieve the actual format of the capture device's buffer.");
26205 |             return ma_result_from_HRESULT(hr);
26206 |         }
26207 |

```

```

26208 |         /* We can now start setting the output data formats. */
26209 |         pDescriptorCapture->format =
> ma_format_from_WAVEFORMATEX((MA_WAVEFORMATEX*)pActualFormat);
26210 |         pDescriptorCapture->channels = pActualFormat->nChannels;
26211 |         pDescriptorCapture->sampleRate = pActualFormat->nSamplesPerSec;
26212 |
26213 |         /* Get the native channel map based on the channel mask. */
26214 |         if (pActualFormat->wFormatTag == WAVE_FORMAT_EXTENSIBLE) {
26215 |             ma_channel_mask_to_channel_map_win32(pActualFormat->dwChannelMask,
> pDescriptorCapture->channels, pDescriptorCapture->channelMap);
26216 |         } else {
26217 |             ma_channel_mask_to_channel_map_win32(wf.dwChannelMask, pDescriptorCapture->channels,
> pDescriptorCapture->channelMap);
26218 |         }
26219 |
26220 |         /*
26221 |         After getting the actual format the size of the buffer in frames may have actually changed.
> However, we want this to be as close to what the
26222 |         user has asked for as possible, so let's go ahead and release the old capture buffer and
> create a new one in this case.
26223 |         */
26224 |         if (periodSizeInFrames != (descDS.dwBufferBytes /
> ma_get_bytes_per_frame(pDescriptorCapture->format, pDescriptorCapture->channels) / periodCount)) {
26225 |             descDS.dwBufferBytes = periodSizeInFrames *
> ma_get_bytes_per_frame(pDescriptorCapture->format, pDescriptorCapture->channels) * periodCount;
26226 |
> ma_IDirectSoundCaptureBuffer_Release((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer);
26227 |
26228 |             hr =
> ma_IDirectSoundCapture_CreateCaptureBuffer((ma_IDirectSoundCapture*)pDevice->dsound.pCapture,
> &descDS, (ma_IDirectSoundCaptureBuffer*)&pDevice->dsound.pCaptureBuffer, NULL);
26229 |             if (FAILED(hr)) {
26230 |                 ma_device_uninit_dsound(pDevice);
26231 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound] Second
> attempt at IDirectSoundCapture_CreateCaptureBuffer() failed for capture device.");
26232 |                 return ma_result_from_HRESULT(hr);
26233 |             }
26234 |         }
26235 |
26236 |         /* DirectSound should give us a buffer exactly the size we asked for. */
26237 |         pDescriptorCapture->periodSizeInFrames = periodSizeInFrames;
26238 |         pDescriptorCapture->periodCount = periodCount;
26239 |     }
26240 |
26241 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
26242 |         MA_WAVEFORMATEXTENSIBLE wf;
26243 |         MA_DSBUFFERDESC descDSPPrimary;
26244 |         MA_DSCAPS caps;
26245 |         char rawdata[1024]; /* <-- Ugly hack to avoid a malloc() due to a crappy DirectSound API. */
26246 |         MA_WAVEFORMATEXTENSIBLE* pActualFormat;
26247 |         ma_uint32 periodSizeInFrames;
26248 |         ma_uint32 periodCount;
26249 |         MA_DSBUFFERDESC descDS;
26250 |         WORD nativeChannelCount;
26251 |         DWORD nativeChannelMask = 0;
26252 |
26253 |         result = ma_config_to_WAVEFORMATEXTENSIBLE(pDescriptorPlayback->format,
> pDescriptorPlayback->channels, pDescriptorPlayback->sampleRate, pDescriptorPlayback->channelMap,
> &wf);
26254 |         if (result != MA_SUCCESS) {
26255 |             return result;
26256 |         }
26257 |
26258 |         result = ma_context_create_IDirectSound_dsound(pDevice->pContext,
> pDescriptorPlayback->shareMode, pDescriptorPlayback->pDeviceID,
> (ma_IDirectSound*)&pDevice->dsound.pPlayback);
26259 |         if (result != MA_SUCCESS) {
26260 |             ma_device_uninit_dsound(pDevice);
26261 |             return result;
26262 |         }
26263 |
26264 |         MA_ZERO_OBJECT(&descDSPPrimary);
26265 |         descDSPPrimary.dwSize = sizeof(MA_DSBUFFERDESC);
26266 |         descDSPPrimary.dwFlags = MA_DSBCAPS_PRIMARYBUFFER | MA_DSBCAPS_CTRLVOLUME;
26267 |         hr = ma_IDirectSound_CreateSoundBuffer((ma_IDirectSound*)pDevice->dsound.pPlayback,
> &descDSPPrimary, (ma_IDirectSoundBuffer*)&pDevice->dsound.pPlaybackPrimaryBuffer, NULL);
26268 |         if (FAILED(hr)) {
26269 |             ma_device_uninit_dsound(pDevice);

```

```

26270 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSound_CreateSoundBuffer() failed for playback device's primary buffer.");
26271 |         return ma_result_from_HRESULT(hr);
26272 |     }
26273 |
26274 |
26275 |     /* We may want to make some adjustments to the format if we are using defaults. */
26276 |     MA_ZERO_OBJECT(&caps);
26277 |     caps.dwSize = sizeof(caps);
26278 |     hr = ma_IDirectSound_GetCaps((ma_IDirectSound*)pDevice->dsound.pPlayback, &caps);
26279 |     if (FAILED(hr)) {
26280 |         ma_device_uninit_dsound(pDevice);
26281 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSound_GetCaps() failed for playback device.");
26282 |         return ma_result_from_HRESULT(hr);
26283 |     }
26284 |
26285 |     if ((caps.dwFlags & MA_DSCAPS_PRIMARYSTEREO) != 0) {
26286 |         DWORD speakerConfig;
26287 |
26288 |         /* It supports at least stereo, but could support more. */
26289 |         nativeChannelCount = 2;
26290 |
26291 |         /* Look at the speaker configuration to get a better idea on the channel count. */
26292 |         if
> (SUCCEEDED(ma_IDirectSound_GetSpeakerConfig((ma_IDirectSound*)pDevice->dsound.pPlayback,
> &speakerConfig))) {
26293 |             ma_get_channels_from_speaker_config_dsound(speakerConfig, &nativeChannelCount,
> &nativeChannelMask);
26294 |         }
26295 |     } else {
26296 |         /* It does not support stereo, which means we are stuck with mono. */
26297 |         nativeChannelCount = 1;
26298 |         nativeChannelMask = 0x00000001;
26299 |     }
26300 |
26301 |     if (pDescriptorPlayback->channels == 0) {
26302 |         wf.nChannels = nativeChannelCount;
26303 |         wf.dwChannelMask = nativeChannelMask;
26304 |     }
26305 |
26306 |     if (pDescriptorPlayback->sampleRate == 0) {
26307 |         /* We base the sample rate on the values returned by GetCaps(). */
26308 |         if ((caps.dwFlags & MA_DSCAPS_CONTINUOUSRATE) != 0) {
26309 |             wf.nSamplesPerSec =
> ma_get_best_sample_rate_within_range(caps.dwMinSecondarySampleRate, caps.dwMaxSecondarySampleRate);
26310 |         } else {
26311 |             wf.nSamplesPerSec = caps.dwMaxSecondarySampleRate;
26312 |         }
26313 |     }
26314 |
26315 |     wf.nBlockAlign = (WORD)(wf.nChannels * wf.wBitsPerSample / 8);
26316 |     wf.nAvgBytesPerSec = wf.nBlockAlign * wf.nSamplesPerSec;
26317 |
26318 |     /*
26319 |     From MSDN:
26320 |
26321 |     The method succeeds even if the hardware does not support the requested format; DirectSound
> sets the buffer to the closest
26322 |     supported format. To determine whether this has happened, an application can call the
> GetFormat method for the primary buffer
26323 |     and compare the result with the format that was requested with the SetFormat method.
26324 |     */
26325 |     hr =
> ma_IDirectSoundBuffer_SetFormat((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackPrimaryBuffer,
> (MA_WAVEFORMATEX*)&wf);
26326 |     if (FAILED(hr)) {
26327 |         /*
26328 |         If setting of the format failed we'll try again with some fallback settings. On Windows
> 98 I have
26329 |         observed that IEEE_FLOAT does not work. We'll therefore enforce PCM. I also had issues
> where a
26330 |         sample rate of 48000 did not work correctly. Not sure if it was a driver issue or not,
> but will
26331 |         use 44100 for the sample rate.
26332 |         */
26333 |         wf.cbSize = 18; /* NOTE: Don't use sizeof(MA_WAVEFORMATEX) here because it's
> got an extra 2 bytes due to padding. */
26334 |         wf.wFormatTag = WAVE_FORMAT_PCM;

```

```

26335 |         wf.wBitsPerSample = 16;
26336 |         wf.nChannels       = nativeChannelCount;
26337 |         wf.nSamplesPerSec  = 44100;
26338 |         wf.nBlockAlign     = wf.nChannels * (wf.wBitsPerSample / 8);
26339 |         wf.nAvgBytesPerSec = wf.nSamplesPerSec * wf.nBlockAlign;
26340 |
26341 |         hr =
> ma_IDirectSoundBuffer_SetFormat((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackPrimaryBuffer,
> (MA_WAVEFORMATEX*)&wf);
26342 |         if (FAILED(hr)) {
26343 |             ma_device_uninit_dsound(pDevice);
26344 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound] Failed to
> set format of playback device's primary buffer.");
26345 |             return ma_result_from_HRESULT(hr);
26346 |         }
26347 |     }
26348 |
26349 |     /* Get the _actual_ properties of the buffer. */
26350 |     pActualFormat = (MA_WAVEFORMATEXTENSIBLE*)rawdata;
26351 |     hr =
> ma_IDirectSoundBuffer_GetFormat((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackPrimaryBuffer,
> (MA_WAVEFORMATEX*)pActualFormat, sizeof(rawdata), NULL);
26352 |     if (FAILED(hr)) {
26353 |         ma_device_uninit_dsound(pDevice);
26354 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound] Failed to
> retrieve the actual format of the playback device's primary buffer.");
26355 |         return ma_result_from_HRESULT(hr);
26356 |     }
26357 |
26358 |     /* We now have enough information to start setting some output properties. */
26359 |     pDescriptorPlayback->format =
> ma_format_from_WAVEFORMATEX((MA_WAVEFORMATEX*)pActualFormat);
26360 |     pDescriptorPlayback->channels = pActualFormat->nChannels;
26361 |     pDescriptorPlayback->sampleRate = pActualFormat->nSamplesPerSec;
26362 |
26363 |     /* Get the internal channel map based on the channel mask. */
26364 |     if (pActualFormat->wFormatTag == WAVE_FORMAT_EXTENSIBLE) {
26365 |         ma_channel_mask_to_channel_map_win32(pActualFormat->dwChannelMask,
> pDescriptorPlayback->channels, pDescriptorPlayback->channelMap);
26366 |     } else {
26367 |         ma_channel_mask_to_channel_map_win32(wf.dwChannelMask, pDescriptorPlayback->channels,
> pDescriptorPlayback->channelMap);
26368 |     }
26369 |
26370 |     /* The size of the buffer must be a clean multiple of the period count. */
26371 |     periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor_dsound(pDescriptorPlayback,
> pDescriptorPlayback->sampleRate, pConfig->performanceProfile);
26372 |     periodCount = (pDescriptorPlayback->periodCount > 0) ? pDescriptorPlayback->periodCount :
> MA_DEFAULT_PERIODS;
26373 |
26374 |     /*
26375 |     Meaning of dwFlags (from MSDN):
26376 |
26377 |     DSBCAPS_CTRLPOSITIONNOTIFY
26378 |         The buffer has position notification capability.
26379 |
26380 |     DSBCAPS_GLOBALFOCUS
26381 |         With this flag set, an application using DirectSound can continue to play its buffers if
> the user switches focus to
26382 |         another application, even if the new application uses DirectSound.
26383 |
26384 |     DSBCAPS_GETCURRENTPOSITION2
26385 |         In the first version of DirectSound, the play cursor was significantly ahead of the actual
> playing sound on emulated
26386 |         sound cards; it was directly behind the write cursor. Now, if the
> DSBCAPS_GETCURRENTPOSITION2 flag is specified, the
26387 |         application can get a more accurate play cursor.
26388 |     */
26389 |     MA_ZERO_OBJECT(&descDS);
26390 |     descDS.dwSize = sizeof(descDS);
26391 |     descDS.dwFlags = MA_DSBCAPS_CTRLPOSITIONNOTIFY | MA_DSBCAPS_GLOBALFOCUS |
> MA_DSBCAPS_GETCURRENTPOSITION2;
26392 |     descDS.dwBufferBytes = periodSizeInFrames * periodCount *
> ma_get_bytes_per_frame(pDescriptorPlayback->format, pDescriptorPlayback->channels);
26393 |     descDS.lpwfxFormat = (MA_WAVEFORMATEX*)pActualFormat;
26394 |     hr = ma_IDirectSound_CreateSoundBuffer((ma_IDirectSound*)pDevice->dsound.pPlayback, &descDS,
> (ma_IDirectSoundBuffer**) &pDevice->dsound.pPlaybackBuffer, NULL);
26395 |     if (FAILED(hr)) {

```

```

26396 |         ma_device_uninit_dsound(pDevice);
26397 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSound_CreateSoundBuffer() failed for playback device's secondary buffer.");
26398 |         return ma_result_from_HRESULT(hr);
26399 |     }
26400 |
26401 |     /* DirectSound should give us a buffer exactly the size we asked for. */
26402 |     pDescriptorPlayback->periodSizeInFrames = periodSizeInFrames;
26403 |     pDescriptorPlayback->periodCount        = periodCount;
26404 | }
26405 |
26406 |     return MA_SUCCESS;
26407 | }
26408 |
26409 |
26410 | static ma_result ma_device_data_loop_dsound(ma_device* pDevice)
26411 | {
26412 |     ma_result result = MA_SUCCESS;
26413 |     ma_uint32 bpfDeviceCapture = ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels);
26414 |     ma_uint32 bpfDevicePlayback = ma_get_bytes_per_frame(pDevice->playback.internalFormat,
> pDevice->playback.internalChannels);
26415 |     HRESULT hr;
26416 |     DWORD lockOffsetInBytesCapture;
26417 |     DWORD lockSizeInBytesCapture;
26418 |     DWORD mappedSizeInBytesCapture;
26419 |     DWORD mappedDeviceFramesProcessedCapture;
26420 |     void* pMappedDeviceBufferCapture;
26421 |     DWORD lockOffsetInBytesPlayback;
26422 |     DWORD lockSizeInBytesPlayback;
26423 |     DWORD mappedSizeInBytesPlayback;
26424 |     void* pMappedDeviceBufferPlayback;
26425 |     DWORD prevReadCursorInBytesCapture = 0;
26426 |     DWORD prevPlayCursorInBytesPlayback = 0;
26427 |     ma_bool32 physicalPlayCursorLoopFlagPlayback = 0;
26428 |     DWORD virtualWriteCursorInBytesPlayback = 0;
26429 |     ma_bool32 virtualWriteCursorLoopFlagPlayback = 0;
26430 |     ma_bool32 isPlaybackDeviceStarted = MA_FALSE;
26431 |     ma_uint32 framesWrittenToPlaybackDevice = 0; /* For knowing whether or not the playback device
> needs to be started. */
26432 |     ma_uint32 waitTimeInMilliseconds = 1;
26433 |     DWORD playbackBufferStatus = 0;
26434 |
26435 |     MA_ASSERT(pDevice != NULL);
26436 |
26437 |     /* The first thing to do is start the capture device. The playback device is only started after
> the first period is written. */
26438 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
26439 |         hr =
> ma_IDirectSoundCaptureBuffer_Start((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer,
> MA_DSCBSTART_LOOPING);
26440 |         if (FAILED(hr)) {
26441 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundCaptureBuffer_Start() failed.");
26442 |             return ma_result_from_HRESULT(hr);
26443 |         }
26444 |     }
26445 |
26446 |     while (ma_device_get_state(pDevice) == ma_device_state_started) {
26447 |         switch (pDevice->type)
26448 |         {
26449 |             case ma_device_type_duplex:
26450 |             {
26451 |                 DWORD physicalCaptureCursorInBytes;
26452 |                 DWORD physicalReadCursorInBytes;
26453 |                 hr = ma_IDirectSoundCaptureBuffer_GetCurrentPosition((ma_IDirectSoundCaptureBuffer*)
> pDevice->dsound.pCaptureBuffer, &physicalCaptureCursorInBytes, &physicalReadCursorInBytes);
26454 |                 if (FAILED(hr)) {
26455 |                     return ma_result_from_HRESULT(hr);
26456 |                 }
26457 |
26458 |                 /* If nothing is available we just sleep for a bit and return from this iteration.
> */
26459 |                 if (physicalReadCursorInBytes == prevReadCursorInBytesCapture) {
26460 |                     ma_sleep(waitTimeInMilliseconds);
26461 |                     continue; /* Nothing is available in the capture buffer. */
26462 |                 }
26463 |
26464 |                 /*

```



```

26465 |         The current position has moved. We need to map all of the captured samples and write
> | them to the playback device, making sure
26466 |         we don't return until every frame has been copied over.
26467 |         */
26468 |         if (prevReadCursorInBytesCapture < physicalReadCursorInBytes) {
26469 |             /* The capture position has not looped. This is the simple case. */
26470 |             lockOffsetInBytesCapture = prevReadCursorInBytesCapture;
26471 |             lockSizeInBytesCapture = (physicalReadCursorInBytes -
> | prevReadCursorInBytesCapture);
26472 |         } else {
26473 |             /*
26474 |             The capture position has looped. This is the more complex case. Map to the end
> | of the buffer. If this does not return anything,
26475 |             do it again from the start.
26476 |             */
26477 |             if (prevReadCursorInBytesCapture <
> | pDevice->capture.internalPeriodSizeInFrames*pDevice->capture.internalPeriods*bpfDeviceCapture) {
26478 |                 /* Lock up to the end of the buffer. */
26479 |                 lockOffsetInBytesCapture = prevReadCursorInBytesCapture;
26480 |                 lockSizeInBytesCapture =
> | (pDevice->capture.internalPeriodSizeInFrames*pDevice->capture.internalPeriods*bpfDeviceCapture) -
> | prevReadCursorInBytesCapture;
26481 |             } else {
26482 |                 /* Lock starting from the start of the buffer. */
26483 |                 lockOffsetInBytesCapture = 0;
26484 |                 lockSizeInBytesCapture = physicalReadCursorInBytes;
26485 |             }
26486 |         }
26487 |
26488 |         if (lockSizeInBytesCapture == 0) {
26489 |             ma_sleep(waitTimeInMilliseconds);
26490 |             continue; /* Nothing is available in the capture buffer. */
26491 |         }
26492 |
26493 |         hr =
> | ma_IDirectSoundCaptureBuffer_Lock((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer,
> | lockOffsetInBytesCapture, lockSizeInBytesCapture, &pMappedDeviceBufferCapture,
> | &mappedSizeInBytesCapture, NULL, NULL, 0);
26494 |         if (FAILED(hr)) {
26495 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> | Failed to map buffer from capture device in preparation for writing to the device.");
26496 |             return ma_result_from_HRESULT(hr);
26497 |         }
26498 |
26499 |
26500 |         /* At this point we have some input data that we need to output. We do not return
> | until every mapped frame of the input data is written to the playback device. */
26501 |         mappedDeviceFramesProcessedCapture = 0;
26502 |
26503 |         for (;;) { /* Keep writing to the playback device. */
26504 |             ma_uint8 inputFramesInClientFormat[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
26505 |             ma_uint32 inputFramesInClientFormatCap = sizeof(inputFramesInClientFormat) /
> | ma_get_bytes_per_frame(pDevice->capture.format, pDevice->capture.channels);
26506 |             ma_uint8 outputFramesInClientFormat[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
26507 |             ma_uint32 outputFramesInClientFormatCap = sizeof(outputFramesInClientFormat) /
> | ma_get_bytes_per_frame(pDevice->playback.format, pDevice->playback.channels);
26508 |             ma_uint32 outputFramesInClientFormatCount;
26509 |             ma_uint32 outputFramesInClientFormatConsumed = 0;
26510 |             ma_uint64 clientCapturedFramesToProcess = ma_min(inputFramesInClientFormatCap,
> | outputFramesInClientFormatCap);
26511 |             ma_uint64 deviceCapturedFramesToProcess = (mappedSizeInBytesCapture /
> | bpfDeviceCapture) - mappedDeviceFramesProcessedCapture;
26512 |             void* pRunningMappedDeviceBufferCapture =
> | ma_offset_ptr(pMappedDeviceBufferCapture, mappedDeviceFramesProcessedCapture * bpfDeviceCapture);
26513 |
26514 |             result = ma_data_converter_process_pcm_frames(&pDevice->capture.converter,
> | pRunningMappedDeviceBufferCapture, &deviceCapturedFramesToProcess, inputFramesInClientFormat,
> | &clientCapturedFramesToProcess);
26515 |             if (result != MA_SUCCESS) {
26516 |                 break;
26517 |             }
26518 |
26519 |             outputFramesInClientFormatCount = (ma_uint32)clientCapturedFramesToProcess;
26520 |             mappedDeviceFramesProcessedCapture += (ma_uint32)deviceCapturedFramesToProcess;
26521 |
26522 |             ma_device__handle_data_callback(pDevice, outputFramesInClientFormat,
> | inputFramesInClientFormat, (ma_uint32)clientCapturedFramesToProcess);
26523 |
26524 |             /* At this point we have input and output data in client format. All we need to

```



```

> do now is convert it to the output device format. This may take a few passes. */
26525 |         for (;;) {
26526 |             ma_uint32 framesWrittenThisIteration;
26527 |             DWORD physicalPlayCursorInBytes;
26528 |             DWORD physicalWriteCursorInBytes;
26529 |             DWORD availableBytesPlayback;
26530 |             DWORD silentPaddingInBytes = 0; /* <-- Must be initialized to 0. */
26531 |
26532 |             /* We need the physical play and write cursors. */
26533 |             if (FAILED(ma_IDirectSoundBuffer_GetCurrentPosition((ma_IDirectSoundBuffer*)
> pDevice->dsound.pPlaybackBuffer, &physicalPlayCursorInBytes, &physicalWriteCursorInBytes))) {
26534 |                 break;
26535 |             }
26536 |
26537 |             if (physicalPlayCursorInBytes < prevPlayCursorInBytesPlayback) {
26538 |                 physicalPlayCursorLoopFlagPlayback =
> !physicalPlayCursorLoopFlagPlayback;
26539 |             }
26540 |             prevPlayCursorInBytesPlayback = physicalPlayCursorInBytes;
26541 |
26542 |             /* If there's any bytes available for writing we can do that now. The space
> between the virtual cursor position and play cursor. */
26543 |             if (physicalPlayCursorLoopFlagPlayback ==
> virtualWriteCursorLoopFlagPlayback) {
26544 |                 /* Same loop iteration. The available bytes wraps all the way around
> from the virtual write cursor to the physical play cursor. */
26545 |                 if (physicalPlayCursorInBytes <= virtualWriteCursorInBytesPlayback) {
26546 |                     availableBytesPlayback =
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) -
> virtualWriteCursorInBytesPlayback;
26547 |                     availableBytesPlayback += physicalPlayCursorInBytes; /* Wrap
> around. */
26548 |                 } else {
26549 |                     /* This is an error. */
26550 |                     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING,
> "[DirectSound] (Duplex/Playback): Play cursor has moved in front of the write cursor (same loop
> iteration). physicalPlayCursorInBytes=%ld, virtualWriteCursorInBytes=%ld.\n",
> physicalPlayCursorInBytes, virtualWriteCursorInBytesPlayback);
26551 |                     availableBytesPlayback = 0;
26552 |                 }
26553 |             } else {
26554 |                 /* Different loop iterations. The available bytes only goes from the
> virtual write cursor to the physical play cursor. */
26555 |                 if (physicalPlayCursorInBytes >= virtualWriteCursorInBytesPlayback) {
26556 |                     availableBytesPlayback = physicalPlayCursorInBytes -
> virtualWriteCursorInBytesPlayback;
26557 |                 } else {
26558 |                     /* This is an error. */
26559 |                     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING,
> "[DirectSound] (Duplex/Playback): Write cursor has moved behind the play cursor (different loop
> iterations). physicalPlayCursorInBytes=%ld, virtualWriteCursorInBytes=%ld.\n",
> physicalPlayCursorInBytes, virtualWriteCursorInBytesPlayback);
26560 |                     availableBytesPlayback = 0;
26561 |                 }
26562 |             }
26563 |
26564 |             /* If there's no room available for writing we need to wait for more. */
26565 |             if (availableBytesPlayback == 0) {
26566 |                 /* If we haven't started the device yet, this will never get beyond 0.
> In this case we need to get the device started. */
26567 |                 if (!isPlaybackDeviceStarted) {
26568 |                     hr =
> ma_IDirectSoundBuffer_Play((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer, 0, 0,
> MA_DSPLAY_LOOPING);
26569 |                     if (FAILED(hr)) {
26570 |
> ma_IDirectSoundCaptureBuffer_Stop((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer);
26571 |                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR,
> "[DirectSound] IDirectSoundBuffer_Play() failed.");
26572 |                     return ma_result_from_HRESULT(hr);
26573 |                 }
26574 |                 isPlaybackDeviceStarted = MA_TRUE;
26575 |             } else {
26576 |                 ma_sleep(waitTimeInMilliseconds);
26577 |                 continue;
26578 |             }
26579 |         }
26580 |     }
26581 | }

```

```

26582 |                                     /* Getting here means there room available somewhere. We limit this to
> either the end of the buffer or the physical play cursor, whichever is closest. */
26583 |                                     lockOffsetInBytesPlayback = virtualWriteCursorInBytesPlayback;
26584 |                                     if (physicalPlayCursorLoopFlagPlayback ==
> virtualWriteCursorLoopFlagPlayback) {
26585 |                                     /* Same loop iteration. Go up to the end of the buffer. */
26586 |                                     lockSizeInBytesPlayback =
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) -
> virtualWriteCursorInBytesPlayback;
26587 |                                     } else {
26588 |                                     /* Different loop iterations. Go up to the physical play cursor. */
26589 |                                     lockSizeInBytesPlayback = physicalPlayCursorInBytes -
> virtualWriteCursorInBytesPlayback;
26590 |                                     }
26591 |
26592 |                                     hr =
> ma_IDirectSoundBuffer_Lock((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> lockOffsetInBytesPlayback, lockSizeInBytesPlayback, &pMappedDeviceBufferPlayback,
> &mappedSizeInBytesPlayback, NULL, NULL, 0);
26593 |                                     if (FAILED(hr)) {
26594 |                                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR,
> "[DirectSound] Failed to map buffer from playback device in preparation for writing to the
> device.");
26595 |                                     result = ma_result_from_HRESULT(hr);
26596 |                                     break;
26597 |                                     }
26598 |
26599 |                                     /*
26600 |                                     Experiment: If the playback buffer is being starved, pad it with some
> silence to get it back in sync. This will cause a glitch, but it may prevent
26601 |                                     endless glitching due to it constantly running out of data.
26602 |                                     */
26603 |                                     if (isPlaybackDeviceStarted) {
26604 |                                     DWORD bytesQueuedForPlayback =
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) -
> availableBytesPlayback;
26605 |                                     if (bytesQueuedForPlayback <
> (pDevice->playback.internalPeriodSizeInFrames*bpfDevicePlayback)) {
26606 |                                     silentPaddingInBytes =
> (pDevice->playback.internalPeriodSizeInFrames*2*bpfDevicePlayback) - bytesQueuedForPlayback;
26607 |                                     if (silentPaddingInBytes > lockSizeInBytesPlayback) {
26608 |                                     silentPaddingInBytes = lockSizeInBytesPlayback;
26609 |                                     }
26610 |
26611 |                                     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING,
> "[DirectSound] (Duplex/Playback) Playback buffer starved. availableBytesPlayback=%ld,
> silentPaddingInBytes=%ld\n", availableBytesPlayback, silentPaddingInBytes);
26612 |                                     }
26613 |                                     }
26614 |
26615 |                                     /* At this point we have a buffer for output. */
26616 |                                     if (silentPaddingInBytes > 0) {
26617 |                                     MA_ZERO_MEMORY(pMappedDeviceBufferPlayback, silentPaddingInBytes);
26618 |                                     framesWrittenThisIteration = silentPaddingInBytes/bpfDevicePlayback;
26619 |                                     } else {
26620 |                                     ma_uint64 convertedFrameCountIn = (outputFramesInClientFormatCount -
> outputFramesInClientFormatConsumed);
26621 |                                     ma_uint64 convertedFrameCountOut =
> mappedSizeInBytesPlayback/bpfDevicePlayback;
26622 |                                     void* pConvertedFramesIn = ma_offset_ptr(outputFramesInClientFormat,
> outputFramesInClientFormatConsumed * bpfDevicePlayback);
26623 |                                     void* pConvertedFramesOut = pMappedDeviceBufferPlayback;
26624 |
26625 |                                     result =
> ma_data_converter_process_pcm_frames(&pDevice->playback.converter, pConvertedFramesIn,
> &convertedFrameCountIn, pConvertedFramesOut, &convertedFrameCountOut);
26626 |                                     if (result != MA_SUCCESS) {
26627 |                                     break;
26628 |                                     }
26629 |
26630 |                                     outputFramesInClientFormatConsumed += (ma_uint32)convertedFrameCountOut;
26631 |                                     framesWrittenThisIteration = (ma_uint32)convertedFrameCountOut;
26632 |                                     }
26633 |
26634 |
26635 |                                     hr =
> ma_IDirectSoundBuffer_Unlock((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> pMappedDeviceBufferPlayback, framesWrittenThisIteration*bpfDevicePlayback, NULL, 0);
26636 |                                     if (FAILED(hr)) {

```

```

26637 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR,
> "[DirectSound] Failed to unlock internal buffer from playback device after writing to the device.");
26638 |             result = ma_result_from_HRESULT(hr);
26639 |             break;
26640 |         }
26641 |
26642 |             virtualWriteCursorInBytesPlayback +=
> framesWrittenThisIteration*bpfDevicePlayback;
26643 |             if ((virtualWriteCursorInBytesPlayback/bpfDevicePlayback) ==
> pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods) {
26644 |                 virtualWriteCursorInBytesPlayback = 0;
26645 |                 virtualWriteCursorLoopFlagPlayback =
> !virtualWriteCursorLoopFlagPlayback;
26646 |             }
26647 |
26648 |             /*
26649 |             We may need to start the device. We want two full periods to be written
> before starting the playback device. Having an extra period adds
26650 |             a bit of a buffer to prevent the playback buffer from getting starved.
26651 |             */
26652 |             framesWrittenToPlaybackDevice += framesWrittenThisIteration;
26653 |             if (!isPlaybackDeviceStarted && framesWrittenToPlaybackDevice >=
> (pDevice->playback.internalPeriodSizeInFrames*2)) {
26654 |                 hr =
> ma_IDirectSoundBuffer_Play((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer, 0, 0,
> MA_DSBPLAY_LOOPING);
26655 |                 if (FAILED(hr)) {
26656 |
> ma_IDirectSoundCaptureBuffer_Stop((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer);
26657 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR,
> "[DirectSound] IDirectSoundBuffer_Play() failed.");
26658 |                 return ma_result_from_HRESULT(hr);
26659 |             }
26660 |             isPlaybackDeviceStarted = MA_TRUE;
26661 |         }
26662 |
26663 |             if (framesWrittenThisIteration <
> mappedSizeInBytesPlayback/bpfDevicePlayback) {
26664 |                 break; /* We're finished with the output data.*/
26665 |             }
26666 |         }
26667 |
26668 |         if (clientCapturedFramesToProcess == 0) {
26669 |             break; /* We just consumed every input sample. */
26670 |         }
26671 |     }
26672 |
26673 |
26674 |     /* At this point we're done with the mapped portion of the capture buffer. */
26675 |     hr =
> ma_IDirectSoundCaptureBuffer_Unlock((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer,
> pMappedDeviceBufferCapture, mappedSizeInBytesCapture, NULL, 0);
26676 |     if (FAILED(hr)) {
26677 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> Failed to unlock internal buffer from capture device after reading from the device.");
26678 |         return ma_result_from_HRESULT(hr);
26679 |     }
26680 |     prevReadCursorInBytesCapture = (lockOffsetInBytesCapture +
> mappedSizeInBytesCapture);
26681 |     } break;
26682 |
26683 |
26684 |
26685 |     case ma_device_type_capture:
26686 |     {
26687 |         DWORD physicalCaptureCursorInBytes;
26688 |         DWORD physicalReadCursorInBytes;
26689 |         hr = ma_IDirectSoundCaptureBuffer_GetCurrentPosition((ma_IDirectSoundCaptureBuffer*)
> pDevice->dsound.pCaptureBuffer, &physicalCaptureCursorInBytes, &physicalReadCursorInBytes);
26690 |         if (FAILED(hr)) {
26691 |             return MA_ERROR;
26692 |         }
26693 |
26694 |         /* If the previous capture position is the same as the current position we need to
> wait a bit longer. */
26695 |         if (prevReadCursorInBytesCapture == physicalReadCursorInBytes) {
26696 |             ma_sleep(waitTimeInMilliseconds);
26697 |             continue;
26698 |         }

```

```

26699 |
26700 |         /* Getting here means we have capture data available. */
26701 |         if (prevReadCursorInBytesCapture < physicalReadCursorInBytes) {
26702 |             /* The capture position has not looped. This is the simple case. */
26703 |             lockOffsetInBytesCapture = prevReadCursorInBytesCapture;
26704 |             lockSizeInBytesCapture = (physicalReadCursorInBytes -
> prevReadCursorInBytesCapture);
26705 |         } else {
26706 |             /*
26707 |             The capture position has looped. This is the more complex case. Map to the end
> of the buffer. If this does not return anything,
26708 |             do it again from the start.
26709 |             */
26710 |             if (prevReadCursorInBytesCapture <
> pDevice->capture.internalPeriodSizeInFrames*pDevice->capture.internalPeriods*bpfDeviceCapture) {
26711 |                 /* Lock up to the end of the buffer. */
26712 |                 lockOffsetInBytesCapture = prevReadCursorInBytesCapture;
26713 |                 lockSizeInBytesCapture =
> (pDevice->capture.internalPeriodSizeInFrames*pDevice->capture.internalPeriods*bpfDeviceCapture) -
> prevReadCursorInBytesCapture;
26714 |             } else {
26715 |                 /* Lock starting from the start of the buffer. */
26716 |                 lockOffsetInBytesCapture = 0;
26717 |                 lockSizeInBytesCapture = physicalReadCursorInBytes;
26718 |             }
26719 |         }
26720 |
26721 |         if (lockSizeInBytesCapture < pDevice->capture.internalPeriodSizeInFrames) {
26722 |             ma_sleep(waitTimeInMilliseconds);
26723 |             continue; /* Nothing is available in the capture buffer. */
26724 |         }
26725 |
26726 |         hr =
> ma_IDirectSoundCaptureBuffer_Lock((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer,
> lockOffsetInBytesCapture, lockSizeInBytesCapture, &pMappedDeviceBufferCapture,
> &mappedSizeInBytesCapture, NULL, 0);
26727 |         if (FAILED(hr)) {
26728 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> Failed to map buffer from capture device in preparation for writing to the device.");
26729 |             result = ma_result_from_HRESULT(hr);
26730 |         }
26731 |
26732 |         if (lockSizeInBytesCapture != mappedSizeInBytesCapture) {
26733 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[DirectSound]
> (Capture) lockSizeInBytesCapture=%ld != mappedSizeInBytesCapture=%ld\n", lockSizeInBytesCapture,
> mappedSizeInBytesCapture);
26734 |         }
26735 |
26736 |         ma_device__send_frames_to_client(pDevice, mappedSizeInBytesCapture/bpfDeviceCapture,
> pMappedDeviceBufferCapture);
26737 |
26738 |         hr =
> ma_IDirectSoundCaptureBuffer_Unlock((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer,
> pMappedDeviceBufferCapture, mappedSizeInBytesCapture, NULL, 0);
26739 |         if (FAILED(hr)) {
26740 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> Failed to unlock internal buffer from capture device after reading from the device.");
26741 |             return ma_result_from_HRESULT(hr);
26742 |         }
26743 |         prevReadCursorInBytesCapture = lockOffsetInBytesCapture + mappedSizeInBytesCapture;
26744 |
26745 |         if (prevReadCursorInBytesCapture ==
> (pDevice->capture.internalPeriodSizeInFrames*pDevice->capture.internalPeriods*bpfDeviceCapture)) {
26746 |             prevReadCursorInBytesCapture = 0;
26747 |         }
26748 |     } break;
26749 |
26750 |
26751 |
26752 |     case ma_device_type_playback:
26753 |     {
26754 |         DWORD availableBytesPlayback;
26755 |         DWORD physicalPlayCursorInBytes;
26756 |         DWORD physicalWriteCursorInBytes;
26757 |         hr =
> ma_IDirectSoundBuffer_GetCurrentPosition((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> &physicalPlayCursorInBytes, &physicalWriteCursorInBytes);
26758 |         if (FAILED(hr)) {
26759 |             break;

```

```

26760 |         }
26761 |
26762 |         hr =
> ma_IDirectSoundBuffer_GetStatus((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> &playbackBufferStatus);
26763 |         if (SUCCEEDED(hr) && (playbackBufferStatus & MA_DSBSTATUS_PLAYING) == 0 &&
> isPlaybackDeviceStarted) {
26764 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[DirectSound]
> Attempting to resume audio due to state: %d.", (int)playbackBufferStatus);
26765 |             hr =
> ma_IDirectSoundBuffer_Play((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer, 0, 0,
> MA_DSBPLAY_LOOPING);
26766 |             if (FAILED(hr)) {
26767 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundBuffer_Play() failed after attempting to resume from state %d.",
> (int)playbackBufferStatus);
26768 |                 return ma_result_from_HRESULT(hr);
26769 |             }
26770 |
26771 |             isPlaybackDeviceStarted = MA_TRUE;
26772 |             ma_sleep(waitTimeInMilliseconds);
26773 |             continue;
26774 |         }
26775 |
26776 |         if (physicalPlayCursorInBytes < prevPlayCursorInBytesPlayback) {
26777 |             physicalPlayCursorLoopFlagPlayback = !physicalPlayCursorLoopFlagPlayback;
26778 |         }
26779 |         prevPlayCursorInBytesPlayback = physicalPlayCursorInBytes;
26780 |
26781 |         /* If there's any bytes available for writing we can do that now. The space between
> the virtual cursor position and play cursor. */
26782 |         if (physicalPlayCursorLoopFlagPlayback == virtualWriteCursorLoopFlagPlayback) {
26783 |             /* Same loop iteration. The available bytes wraps all the way around from the
> virtual write cursor to the physical play cursor. */
26784 |             if (physicalPlayCursorInBytes <= virtualWriteCursorInBytesPlayback) {
26785 |                 availableBytesPlayback =
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) -
> virtualWriteCursorInBytesPlayback;
26786 |                 availableBytesPlayback += physicalPlayCursorInBytes; /* Wrap around. */
26787 |             } else {
26788 |                 /* This is an error. */
26789 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING,
> "[DirectSound] (Playback): Play cursor has moved in front of the write cursor (same loop
> iterations). physicalPlayCursorInBytes=%ld, virtualWriteCursorInBytes=%ld.\n",
> physicalPlayCursorInBytes, virtualWriteCursorInBytesPlayback);
26790 |                 availableBytesPlayback = 0;
26791 |             }
26792 |         } else {
26793 |             /* Different loop iterations. The available bytes only goes from the virtual
> write cursor to the physical play cursor. */
26794 |             if (physicalPlayCursorInBytes >= virtualWriteCursorInBytesPlayback) {
26795 |                 availableBytesPlayback = physicalPlayCursorInBytes -
> virtualWriteCursorInBytesPlayback;
26796 |             } else {
26797 |                 /* This is an error. */
26798 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING,
> "[DirectSound] (Playback): Write cursor has moved behind the play cursor (different loop
> iterations). physicalPlayCursorInBytes=%ld, virtualWriteCursorInBytes=%ld.\n",
> physicalPlayCursorInBytes, virtualWriteCursorInBytesPlayback);
26799 |                 availableBytesPlayback = 0;
26800 |             }
26801 |         }
26802 |
26803 |         /* If there's no room available for writing we need to wait for more. */
26804 |         if (availableBytesPlayback < pDevice->playback.internalPeriodSizeInFrames) {
26805 |             /* If we haven't started the device yet, this will never get beyond 0. In this
> case we need to get the device started. */
26806 |             if (availableBytesPlayback == 0 && !isPlaybackDeviceStarted) {
26807 |                 hr =
> ma_IDirectSoundBuffer_Play((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer, 0, 0,
> MA_DSBPLAY_LOOPING);
26808 |                 if (FAILED(hr)) {
26809 |                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR,
> "[DirectSound] IDirectSoundBuffer_Play() failed.");
26810 |                     return ma_result_from_HRESULT(hr);
26811 |                 }
26812 |                 isPlaybackDeviceStarted = MA_TRUE;
26813 |             } else {
26814 |                 ma_sleep(waitTimeInMilliseconds);

```

```

26815 |             continue;
26816 |         }
26817 |     }
26818 |
26819 |     /* Getting here means there room available somewhere. We limit this to either the
> end of the buffer or the physical play cursor, whichever is closest. */
26820 |     lockOffsetInBytesPlayback = virtualWriteCursorInBytesPlayback;
26821 |     if (physicalPlayCursorLoopFlagPlayback == virtualWriteCursorLoopFlagPlayback) {
26822 |         /* Same loop iteration. Go up to the end of the buffer. */
26823 |         lockSizeInBytesPlayback =
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) -
> virtualWriteCursorInBytesPlayback;
26824 |     } else {
26825 |         /* Different loop iterations. Go up to the physical play cursor. */
26826 |         lockSizeInBytesPlayback = physicalPlayCursorInBytes -
> virtualWriteCursorInBytesPlayback;
26827 |     }
26828 |
26829 |     hr =
> ma_IDirectSoundBuffer_Lock((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> lockOffsetInBytesPlayback, lockSizeInBytesPlayback, &pMappedDeviceBufferPlayback,
> &mappedSizeInBytesPlayback, NULL, NULL, 0);
26830 |     if (FAILED(hr)) {
26831 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> Failed to map buffer from playback device in preparation for writing to the device.");
26832 |         result = ma_result_from_HRESULT(hr);
26833 |         break;
26834 |     }
26835 |
26836 |     /* At this point we have a buffer for output. */
26837 |     ma_device__read_frames_from_client(pDevice,
> (mappedSizeInBytesPlayback/bpfDevicePlayback), pMappedDeviceBufferPlayback);
26838 |
26839 |     hr =
> ma_IDirectSoundBuffer_Unlock((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> pMappedDeviceBufferPlayback, mappedSizeInBytesPlayback, NULL, 0);
26840 |     if (FAILED(hr)) {
26841 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> Failed to unlock internal buffer from playback device after writing to the device.");
26842 |         result = ma_result_from_HRESULT(hr);
26843 |         break;
26844 |     }
26845 |
26846 |     virtualWriteCursorInBytesPlayback += mappedSizeInBytesPlayback;
26847 |     if (virtualWriteCursorInBytesPlayback ==
> pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) {
26848 |         virtualWriteCursorInBytesPlayback = 0;
26849 |         virtualWriteCursorLoopFlagPlayback = !virtualWriteCursorLoopFlagPlayback;
26850 |     }
26851 |
26852 |     /*
26853 |     We may need to start the device. We want two full periods to be written before
> starting the playback device. Having an extra period adds
26854 |     a bit of a buffer to prevent the playback buffer from getting starved.
26855 |     */
26856 |     framesWrittenToPlaybackDevice += mappedSizeInBytesPlayback/bpfDevicePlayback;
26857 |     if (!isPlaybackDeviceStarted && framesWrittenToPlaybackDevice >=
> pDevice->playback.internalPeriodSizeInFrames) {
26858 |         hr =
> ma_IDirectSoundBuffer_Play((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer, 0, 0,
> MA_DSBPLAY_LOOPING);
26859 |         if (FAILED(hr)) {
26860 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundBuffer_Play() failed.");
26861 |             return ma_result_from_HRESULT(hr);
26862 |         }
26863 |         isPlaybackDeviceStarted = MA_TRUE;
26864 |     }
26865 |     } break;
26866 |
26867 |
26868 |     default: return MA_INVALID_ARGS; /* Invalid device type. */
26869 | }
26870 |
26871 | if (result != MA_SUCCESS) {
26872 |     return result;
26873 | }
26874 | }
26875 |

```



```

26876 |     /* Getting here means the device is being stopped. */
26877 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
26878 |         hr =
> ma_IDirectSoundCaptureBuffer_Stop((ma_IDirectSoundCaptureBuffer*)pDevice->dsound.pCaptureBuffer);
26879 |         if (FAILED(hr)) {
26880 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundCaptureBuffer_Stop() failed.");
26881 |             return ma_result_from_HRESULT(hr);
26882 |         }
26883 |     }
26884 |
26885 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
26886 |         /* The playback device should be drained before stopping. All we do is wait until the
> available bytes is equal to the size of the buffer. */
26887 |         if (isPlaybackDeviceStarted) {
26888 |             for (;;) {
26889 |                 DWORD availableBytesPlayback = 0;
26890 |                 DWORD physicalPlayCursorInBytes;
26891 |                 DWORD physicalWriteCursorInBytes;
26892 |                 hr =
> ma_IDirectSoundBuffer_GetCurrentPosition((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> &physicalPlayCursorInBytes, &physicalWriteCursorInBytes);
26893 |                 if (FAILED(hr)) {
26894 |                     break;
26895 |                 }
26896 |
26897 |                 if (physicalPlayCursorInBytes < prevPlayCursorInBytesPlayback) {
26898 |                     physicalPlayCursorLoopFlagPlayback = !physicalPlayCursorLoopFlagPlayback;
26899 |                 }
26900 |                 prevPlayCursorInBytesPlayback = physicalPlayCursorInBytes;
26901 |
26902 |                 if (physicalPlayCursorLoopFlagPlayback == virtualWriteCursorLoopFlagPlayback) {
26903 |                     /* Same loop iteration. The available bytes wraps all the way around from the
> virtual write cursor to the physical play cursor. */
26904 |                     if (physicalPlayCursorInBytes <= virtualWriteCursorInBytesPlayback) {
26905 |                         availableBytesPlayback =
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback) -
> virtualWriteCursorInBytesPlayback;
26906 |                         availableBytesPlayback += physicalPlayCursorInBytes;    /* Wrap around. */
26907 |                     } else {
26908 |                         break;
26909 |                     }
26910 |                 } else {
26911 |                     /* Different loop iterations. The available bytes only goes from the virtual
> write cursor to the physical play cursor. */
26912 |                     if (physicalPlayCursorInBytes >= virtualWriteCursorInBytesPlayback) {
26913 |                         availableBytesPlayback = physicalPlayCursorInBytes -
> virtualWriteCursorInBytesPlayback;
26914 |                     } else {
26915 |                         break;
26916 |                     }
26917 |                 }
26918 |
26919 |                 if (availableBytesPlayback >=
> (pDevice->playback.internalPeriodSizeInFrames*pDevice->playback.internalPeriods*bpfDevicePlayback))
> {
26920 |                     break;
26921 |                 }
26922 |
26923 |                 ma_sleep(waitTimeInMilliseconds);
26924 |             }
26925 |         }
26926 |
26927 |         hr = ma_IDirectSoundBuffer_Stop((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer);
26928 |         if (FAILED(hr)) {
26929 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[DirectSound]
> IDirectSoundBuffer_Stop() failed.");
26930 |             return ma_result_from_HRESULT(hr);
26931 |         }
26932 |
26933 |
> ma_IDirectSoundBuffer_SetCurrentPosition((ma_IDirectSoundBuffer*)pDevice->dsound.pPlaybackBuffer,
> 0);
26934 |     }
26935 |
26936 |     return MA_SUCCESS;
26937 | }
26938 |
26939 | static ma_result ma_context_uninit_dsound(ma_context* pContext)

```



```

26940 | {
26941 |     MA_ASSERT(pContext != NULL);
26942 |     MA_ASSERT(pContext->backend == ma_backend_dsound);
26943 |
26944 |     ma_dlclose(ma_context_get_log(pContext), pContext->dsound.hDSoundDLL);
26945 |
26946 |     return MA_SUCCESS;
26947 | }
26948 |
26949 | static ma_result ma_context_init_dsound(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
26950 | {
26951 |     MA_ASSERT(pContext != NULL);
26952 |
26953 |     (void)pConfig;
26954 |
26955 |     pContext->dsound.hDSoundDLL = ma_dlopen(ma_context_get_log(pContext), "dsound.dll");
26956 |     if (pContext->dsound.hDSoundDLL == NULL) {
26957 |         return MA_API_NOT_FOUND;
26958 |     }
26959 |
26960 |     pContext->dsound.DirectSoundCreate = ma_dlsym(ma_context_get_log(pContext),
> pContext->dsound.hDSoundDLL, "DirectSoundCreate");
26961 |     pContext->dsound.DirectSoundEnumerateA = ma_dlsym(ma_context_get_log(pContext),
> pContext->dsound.hDSoundDLL, "DirectSoundEnumerateA");
26962 |     pContext->dsound.DirectSoundCaptureCreate = ma_dlsym(ma_context_get_log(pContext),
> pContext->dsound.hDSoundDLL, "DirectSoundCaptureCreate");
26963 |     pContext->dsound.DirectSoundCaptureEnumerateA = ma_dlsym(ma_context_get_log(pContext),
> pContext->dsound.hDSoundDLL, "DirectSoundCaptureEnumerateA");
26964 |
26965 |     /*
26966 |     We need to support all functions or nothing. DirectSound with Windows 95 seems to not work too
26967 |     well in my testing. For example, it's missing DirectSoundCaptureEnumerateA(). This is a
> convenient
26968 |     place to just disable the DirectSound backend for Windows 95.
26969 |     */
26970 |     if (pContext->dsound.DirectSoundCreate == NULL ||
26971 |         pContext->dsound.DirectSoundEnumerateA == NULL ||
26972 |         pContext->dsound.DirectSoundCaptureCreate == NULL ||
26973 |         pContext->dsound.DirectSoundCaptureEnumerateA == NULL) {
26974 |         return MA_API_NOT_FOUND;
26975 |     }
26976 |
26977 |     pContext->dsound.hWnd = pConfig->dsound.hWnd;
26978 |
26979 |     pCallbacks->onContextInit = ma_context_init_dsound;
26980 |     pCallbacks->onContextUninit = ma_context_uninit_dsound;
26981 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices_dsound;
26982 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info_dsound;
26983 |     pCallbacks->onDeviceInit = ma_device_init_dsound;
26984 |     pCallbacks->onDeviceUninit = ma_device_uninit_dsound;
26985 |     pCallbacks->onDeviceStart = NULL; /* Not used. Started in onDeviceDataLoop. */
26986 |     pCallbacks->onDeviceStop = NULL; /* Not used. Stopped in onDeviceDataLoop. */
26987 |     pCallbacks->onDeviceRead = NULL; /* Not used. Data is read directly in
> onDeviceDataLoop. */
26988 |     pCallbacks->onDeviceWrite = NULL; /* Not used. Data is written directly in
> onDeviceDataLoop. */
26989 |     pCallbacks->onDeviceDataLoop = ma_device_data_loop_dsound;
26990 |
26991 |     return MA_SUCCESS;
26992 | }
26993 | #endif
26994 |
26995 |
26996 |
26997 | /*****
26998 |
26999 | WinMM Backend
27000 |
27001 | *****/
27002 | #ifdef MA_HAS_WINMM
27003 |
27004 | /*
27005 | Some build configurations will exclude the WinMM API. An example is when WIN32_LEAN_AND_MEAN
27006 | is defined. We need to define the types and functions we need manually.
27007 | */
27008 | #define MA_MMSYSERR_NOERROR 0
27009 | #define MA_MMSYSERR_ERROR 1
27010 | #define MA_MMSYSERR_BADDEVICEID 2

```

```
27011 | #define MA_MMSYSERR_INVALIDHANDLE 5
27012 | #define MA_MMSYSERR_NOMEM 7
27013 | #define MA_MMSYSERR_INVALIDFLAG 10
27014 | #define MA_MMSYSERR_INVALIDPARAM 11
27015 | #define MA_MMSYSERR_HANDLEBUSY 12
27016 |
27017 | #define MA_CALLBACK_EVENT 0x00050000
27018 | #define MA_WAVE_ALLOWSYNC 0x0002
27019 |
27020 | #define MA_WHDR_DONE 0x00000001
27021 | #define MA_WHDR_PREPARED 0x00000002
27022 | #define MA_WHDR_BEGINLOOP 0x00000004
27023 | #define MA_WHDR_ENDLOOP 0x00000008
27024 | #define MA_WHDR_INQUEUE 0x00000010
27025 |
27026 | #define MA_MAXPNAMELEN 32
27027 |
27028 | typedef void* MA_HWAVEIN;
27029 | typedef void* MA_HWAVEOUT;
27030 | typedef UINT MA_MMRESULT;
27031 | typedef UINT MA_MMVERSION;
27032 |
27033 | typedef struct
27034 | {
27035 |     WORD wMid;
27036 |     WORD wPid;
27037 |     MA_MMVERSION vDriverVersion;
27038 |     CHAR szPname[MA_MAXPNAMELEN];
27039 |     DWORD dwFormats;
27040 |     WORD wChannels;
27041 |     WORD wReserved1;
27042 | } MA_WAVEINCAPSA;
27043 |
27044 | typedef struct
27045 | {
27046 |     WORD wMid;
27047 |     WORD wPid;
27048 |     MA_MMVERSION vDriverVersion;
27049 |     CHAR szPname[MA_MAXPNAMELEN];
27050 |     DWORD dwFormats;
27051 |     WORD wChannels;
27052 |     WORD wReserved1;
27053 |     DWORD dwSupport;
27054 | } MA_WAVEOUTCAPSA;
27055 |
27056 | typedef struct tagWAVEHDR
27057 | {
27058 |     char* lpData;
27059 |     DWORD dwBufferLength;
27060 |     DWORD dwBytesRecorded;
27061 |     DWORD_PTR dwUser;
27062 |     DWORD dwFlags;
27063 |     DWORD dwLoops;
27064 |     struct tagWAVEHDR* lpNext;
27065 |     DWORD_PTR reserved;
27066 | } MA_WAVEHDR;
27067 |
27068 | typedef struct
27069 | {
27070 |     WORD wMid;
27071 |     WORD wPid;
27072 |     MA_MMVERSION vDriverVersion;
27073 |     CHAR szPname[MA_MAXPNAMELEN];
27074 |     DWORD dwFormats;
27075 |     WORD wChannels;
27076 |     WORD wReserved1;
27077 |     DWORD dwSupport;
27078 |     GUID ManufacturerGuid;
27079 |     GUID ProductGuid;
27080 |     GUID NameGuid;
27081 | } MA_WAVEOUTCAPS2A;
27082 |
27083 | typedef struct
27084 | {
27085 |     WORD wMid;
27086 |     WORD wPid;
27087 |     MA_MMVERSION vDriverVersion;
27088 |     CHAR szPname[MA_MAXPNAMELEN];
27089 |     DWORD dwFormats;
```

```

27090 |     WORD wChannels;
27091 |     WORD wReserved1;
27092 |     GUID ManufacturerGuid;
27093 |     GUID ProductGuid;
27094 |     GUID NameGuid;
27095 | } MA_WAVEINCAPS2A;
27096 |
27097 | typedef UINT (WINAPI * MA_PFN_waveOutGetNumDevs)(void);
27098 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutGetDevCapsA)(ma_uintptr uDeviceID, MA_WAVEOUTCAPSA*
> pwoc, UINT cbwoc);
27099 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutOpen)(MA_HWAVEOUT* phwo, UINT uDeviceID, const
> MA_WAVEFORMATEX* pwf, DWORD_PTR dwCallback, DWORD_PTR dwInstance, DWORD fdwOpen);
27100 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutClose)(MA_HWAVEOUT hwo);
27101 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutPrepareHeader)(MA_HWAVEOUT hwo, MA_WAVEHDR* pwh, UINT
> cbwh);
27102 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutUnprepareHeader)(MA_HWAVEOUT hwo, MA_WAVEHDR* pwh, UINT
> cbwh);
27103 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutWrite)(MA_HWAVEOUT hwo, MA_WAVEHDR* pwh, UINT cbwh);
27104 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveOutReset)(MA_HWAVEOUT hwo);
27105 | typedef UINT (WINAPI * MA_PFN_waveInGetNumDevs)(void);
27106 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInGetDevCapsA)(ma_uintptr uDeviceID, MA_WAVEINCAPSA* pwic,
> UINT cbwic);
27107 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInOpen)(MA_HWAVEIN* phwi, UINT uDeviceID, const
> MA_WAVEFORMATEX* pwf, DWORD_PTR dwCallback, DWORD_PTR dwInstance, DWORD fdwOpen);
27108 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInClose)(MA_HWAVEIN hwi);
27109 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInPrepareHeader)(MA_HWAVEIN hwi, MA_WAVEHDR* pwh, UINT
> cbwh);
27110 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInUnprepareHeader)(MA_HWAVEIN hwi, MA_WAVEHDR* pwh, UINT
> cbwh);
27111 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInAddBuffer)(MA_HWAVEIN hwi, MA_WAVEHDR* pwh, UINT cbwh);
27112 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInStart)(MA_HWAVEIN hwi);
27113 | typedef MA_MMRESULT (WINAPI * MA_PFN_waveInReset)(MA_HWAVEIN hwi);
27114 |
27115 | static ma_result ma_result_from_MMRESULT(MA_MMRESULT resultMM)
27116 | {
27117 |     switch (resultMM)
27118 |     {
27119 |         case MA_MMSYSERR_NOERROR: return MA_SUCCESS;
27120 |         case MA_MMSYSERR_BADDEVICEID: return MA_INVALID_ARGS;
27121 |         case MA_MMSYSERR_INVALIDHANDLE: return MA_INVALID_ARGS;
27122 |         case MA_MMSYSERR_NOMEM: return MA_OUT_OF_MEMORY;
27123 |         case MA_MMSYSERR_INVALIDFLAG: return MA_INVALID_ARGS;
27124 |         case MA_MMSYSERR_INVALIDPARAM: return MA_INVALID_ARGS;
27125 |         case MA_MMSYSERR_HANDLEBUSY: return MA_BUSY;
27126 |         case MA_MMSYSERR_ERROR: return MA_ERROR;
27127 |         default: return MA_ERROR;
27128 |     }
27129 | }
27130 |
27131 | static char* ma_find_last_character(char* str, char ch)
27132 | {
27133 |     char* last;
27134 |
27135 |     if (str == NULL) {
27136 |         return NULL;
27137 |     }
27138 |
27139 |     last = NULL;
27140 |     while (*str != '\0') {
27141 |         if (*str == ch) {
27142 |             last = str;
27143 |         }
27144 |
27145 |         str += 1;
27146 |     }
27147 |
27148 |     return last;
27149 | }
27150 |
27151 | static ma_uint32 ma_get_period_size_in_bytes(ma_uint32 periodSizeInFrames, ma_format format,
> ma_uint32 channels)
27152 | {
27153 |     return periodSizeInFrames * ma_get_bytes_per_frame(format, channels);
27154 | }
27155 |
27156 |
27157 | /*
27158 | Our own "WAVECAPS" structure that contains generic information shared between WAVEOUTCAPS2 and
> WAVEINCAPS2 so

```

```

27159 | we can do things generically and typesafely. Names are being kept the same for consistency.
27160 | */
27161 | typedef struct
27162 | {
27163 |     CHAR szPname[MA_MAXPNAMELEN];
27164 |     DWORD dwFormats;
27165 |     WORD wChannels;
27166 |     GUID NameGuid;
27167 | } MA_WAVECAPSA;
27168 |
27169 | static ma_result ma_get_best_info_from_formats_flags_winmm(DWORD dwFormats, WORD channels, WORD*
> pBitsPerSample, DWORD* pSampleRate)
27170 | {
27171 |     WORD bitsPerSample = 0;
27172 |     DWORD sampleRate = 0;
27173 |
27174 |     if (pBitsPerSample) {
27175 |         *pBitsPerSample = 0;
27176 |     }
27177 |     if (pSampleRate) {
27178 |         *pSampleRate = 0;
27179 |     }
27180 |
27181 |     if (channels == 1) {
27182 |         bitsPerSample = 16;
27183 |         if ((dwFormats & WAVE_FORMAT_48M16) != 0) {
27184 |             sampleRate = 48000;
27185 |         } else if ((dwFormats & WAVE_FORMAT_44M16) != 0) {
27186 |             sampleRate = 44100;
27187 |         } else if ((dwFormats & WAVE_FORMAT_2M16) != 0) {
27188 |             sampleRate = 22050;
27189 |         } else if ((dwFormats & WAVE_FORMAT_1M16) != 0) {
27190 |             sampleRate = 11025;
27191 |         } else if ((dwFormats & WAVE_FORMAT_96M16) != 0) {
27192 |             sampleRate = 96000;
27193 |         } else {
27194 |             bitsPerSample = 8;
27195 |             if ((dwFormats & WAVE_FORMAT_48M08) != 0) {
27196 |                 sampleRate = 48000;
27197 |             } else if ((dwFormats & WAVE_FORMAT_44M08) != 0) {
27198 |                 sampleRate = 44100;
27199 |             } else if ((dwFormats & WAVE_FORMAT_2M08) != 0) {
27200 |                 sampleRate = 22050;
27201 |             } else if ((dwFormats & WAVE_FORMAT_1M08) != 0) {
27202 |                 sampleRate = 11025;
27203 |             } else if ((dwFormats & WAVE_FORMAT_96M08) != 0) {
27204 |                 sampleRate = 96000;
27205 |             } else {
27206 |                 return MA_FORMAT_NOT_SUPPORTED;
27207 |             }
27208 |         }
27209 |     } else {
27210 |         bitsPerSample = 16;
27211 |         if ((dwFormats & WAVE_FORMAT_48S16) != 0) {
27212 |             sampleRate = 48000;
27213 |         } else if ((dwFormats & WAVE_FORMAT_44S16) != 0) {
27214 |             sampleRate = 44100;
27215 |         } else if ((dwFormats & WAVE_FORMAT_2S16) != 0) {
27216 |             sampleRate = 22050;
27217 |         } else if ((dwFormats & WAVE_FORMAT_1S16) != 0) {
27218 |             sampleRate = 11025;
27219 |         } else if ((dwFormats & WAVE_FORMAT_96S16) != 0) {
27220 |             sampleRate = 96000;
27221 |         } else {
27222 |             bitsPerSample = 8;
27223 |             if ((dwFormats & WAVE_FORMAT_48S08) != 0) {
27224 |                 sampleRate = 48000;
27225 |             } else if ((dwFormats & WAVE_FORMAT_44S08) != 0) {
27226 |                 sampleRate = 44100;
27227 |             } else if ((dwFormats & WAVE_FORMAT_2S08) != 0) {
27228 |                 sampleRate = 22050;
27229 |             } else if ((dwFormats & WAVE_FORMAT_1S08) != 0) {
27230 |                 sampleRate = 11025;
27231 |             } else if ((dwFormats & WAVE_FORMAT_96S08) != 0) {
27232 |                 sampleRate = 96000;
27233 |             } else {
27234 |                 return MA_FORMAT_NOT_SUPPORTED;
27235 |             }
27236 |         }
27237 |     }

```

```

27237 |     }
27238 |
27239 |     if (pBitsPerSample) {
27240 |         *pBitsPerSample = bitsPerSample;
27241 |     }
27242 |     if (pSampleRate) {
27243 |         *pSampleRate = sampleRate;
27244 |     }
27245 |
27246 |     return MA_SUCCESS;
27247 | }
27248 |
27249 | static ma_result ma_formats_flags_to_WAVEFORMATEX__winmm(DWORD dwFormats, WORD channels,
> MA_WAVEFORMATEX* pWF)
27250 | {
27251 |     ma_result result;
27252 |
27253 |     MA_ASSERT(pWF != NULL);
27254 |
27255 |     MA_ZERO_OBJECT(pWF);
27256 |     pWF->cbSize = sizeof(*pWF);
27257 |     pWF->wFormatTag = WAVE_FORMAT_PCM;
27258 |     pWF->nChannels = (WORD)channels;
27259 |     if (pWF->nChannels > 2) {
27260 |         pWF->nChannels = 2;
27261 |     }
27262 |
27263 |     result = ma_get_best_info_from_formats_flags__winmm(dwFormats, channels, &pWF->wBitsPerSample,
> &pWF->nSamplesPerSec);
27264 |     if (result != MA_SUCCESS) {
27265 |         return result;
27266 |     }
27267 |
27268 |     pWF->nBlockAlign = (WORD)(pWF->nChannels * pWF->wBitsPerSample / 8);
27269 |     pWF->nAvgBytesPerSec = pWF->nBlockAlign * pWF->nSamplesPerSec;
27270 |
27271 |     return MA_SUCCESS;
27272 | }
27273 |
27274 | static ma_result ma_context_get_device_info_from_WAVECAPS(ma_context* pContext, MA_WAVECAPSA* pCaps,
> ma_device_info* pDeviceInfo)
27275 | {
27276 |     WORD bitsPerSample;
27277 |     DWORD sampleRate;
27278 |     ma_result result;
27279 |
27280 |     MA_ASSERT(pContext != NULL);
27281 |     MA_ASSERT(pCaps != NULL);
27282 |     MA_ASSERT(pDeviceInfo != NULL);
27283 |
27284 |     /*
27285 |     Name / Description
27286 |
27287 |     Unfortunately the name specified in WAVE(OUT/IN)CAPS2 is limited to 31 characters. This results
> in an unprofessional looking
27288 |     situation where the names of the devices are truncated. To help work around this, we need to
> look at the name GUID and try
27289 |     looking in the registry for the full name. If we can't find it there, we need to just fall back
> to the default name.
27290 |     */
27291 |
27292 |     /* Set the default to begin with. */
27293 |     ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), pCaps->szPname, (size_t)-1);
27294 |
27295 |     /*
27296 |     Now try the registry. There's a few things to consider here:
27297 |     - The name GUID can be null, in which case we just need to stick to the original 31
> characters.
27298 |     - If the name GUID is not present in the registry we'll also need to stick to the original 31
> characters.
27299 |     - I like consistency, so I want the returned device names to be consistent with those returned
> by WASAPI and DirectSound. The
27300 |     problem, however is that WASAPI and DirectSound use "<component> (<name>)" format (such as
> "Speakers (High Definition Audio)"),
27301 |     but WinMM does not specify the component name. From my admittedly limited testing, I've notice
> the component name seems to
27302 |     usually fit within the 31 characters of the fixed sized buffer, so what I'm going to do is
> parse that string for the component
27303 |     name, and then concatenate the name from the registry.

```

```

27304 |     */
27305 |     if (!ma_is_guid_null(&pCaps->NameGuid)) {
27306 |         WCHAR guidStrW[256];
27307 |         if (((MA_PFN_StringFromGUID2)pContext->win32.StringFromGUID2)(&pCaps->NameGuid, guidStrW,
> ma_countof(guidStrW)) > 0) {
27308 |             char guidStr[256];
27309 |             char keyStr[1024];
27310 |             HKEY hKey;
27311 |
27312 |             WideCharToMultiByte(CP_UTF8, 0, guidStrW, -1, guidStr, sizeof(guidStr), 0, FALSE);
27313 |
27314 |             ma_strcpy_s(keyStr, sizeof(keyStr),
> "SYSTEM\\CurrentControlSet\\Control\\MediaCategories\\");
27315 |             ma_strcat_s(keyStr, sizeof(keyStr), guidStr);
27316 |
27317 |             if (((MA_PFN_RegOpenKeyExA)pContext->win32.RegOpenKeyExA)(HKEY_LOCAL_MACHINE, keyStr, 0,
> KEY_READ, &hKey) == ERROR_SUCCESS) {
27318 |                 BYTE nameFromReg[512];
27319 |                 DWORD nameFromRegSize = sizeof(nameFromReg);
27320 |                 LONG resultWin32 = ((MA_PFN_RegQueryValueExA)pContext->win32.RegQueryValueExA)(hKey,
> "Name", 0, NULL, (BYTE*)nameFromReg, (DWORD*)&nameFromRegSize);
27321 |                 ((MA_PFN_RegCloseKey)pContext->win32.RegCloseKey)(hKey);
27322 |
27323 |                 if (resultWin32 == ERROR_SUCCESS) {
27324 |                     /* We have the value from the registry, so now we need to construct the name
> string. */
27325 |                     char name[1024];
27326 |                     if (ma_strcpy_s(name, sizeof(name), pDeviceInfo->name) == 0) {
27327 |                         char* nameBeg = ma_find_last_character(name, '(');
27328 |                         if (nameBeg != NULL) {
27329 |                             size_t leadingLen = (nameBeg - name);
27330 |                             ma_strncpy_s(nameBeg + 1, sizeof(name) - leadingLen, (const
> char*)nameFromReg, (size_t)-1);
27331 |
27332 |                             /* The closing ")", if it can fit. */
27333 |                             if (leadingLen + nameFromRegSize < sizeof(name)-1) {
27334 |                                 ma_strcat_s(name, sizeof(name), ")");
27335 |                             }
27336 |
27337 |                             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), name,
> (size_t)-1);
27338 |                         }
27339 |                     }
27340 |                 }
27341 |             }
27342 |         }
27343 |     }
27344 |
27345 |     result = ma_get_best_info_from_formats_flags_winmm(pCaps->dwFormats, pCaps->wChannels,
> &bitsPerSample, &sampleRate);
27347 |     if (result != MA_SUCCESS) {
27348 |         return result;
27349 |     }
27350 |
27351 |     if (bitsPerSample == 8) {
27352 |         pDeviceInfo->nativeDataFormats[0].format = ma_format_u8;
27353 |     } else if (bitsPerSample == 16) {
27354 |         pDeviceInfo->nativeDataFormats[0].format = ma_format_s16;
27355 |     } else if (bitsPerSample == 24) {
27356 |         pDeviceInfo->nativeDataFormats[0].format = ma_format_s24;
27357 |     } else if (bitsPerSample == 32) {
27358 |         pDeviceInfo->nativeDataFormats[0].format = ma_format_s32;
27359 |     } else {
27360 |         return MA_FORMAT_NOT_SUPPORTED;
27361 |     }
27362 |     pDeviceInfo->nativeDataFormats[0].channels = pCaps->wChannels;
27363 |     pDeviceInfo->nativeDataFormats[0].sampleRate = sampleRate;
27364 |     pDeviceInfo->nativeDataFormats[0].flags = 0;
27365 |     pDeviceInfo->nativeDataFormatCount = 1;
27366 |
27367 |     return MA_SUCCESS;
27368 | }
27369 |
27370 | static ma_result ma_context_get_device_info_from_WAVEOUTCAPS2(ma_context* pContext,
> MA_WAVEOUTCAPS2A* pCaps, ma_device_info* pDeviceInfo)
27371 | {
27372 |     MA_WAVECAPSA caps;
27373 |

```

```

27374 |     MA_ASSERT(pContext != NULL);
27375 |     MA_ASSERT(pCaps != NULL);
27376 |     MA_ASSERT(pDeviceInfo != NULL);
27377 |
27378 |     MA_COPY_MEMORY(caps.szPname, pCaps->szPname, sizeof(caps.szPname));
27379 |     caps.dwFormats = pCaps->dwFormats;
27380 |     caps.wChannels = pCaps->wChannels;
27381 |     caps.NameGuid = pCaps->NameGuid;
27382 |     return ma_context_get_device_info_from_WAVECAPS(pContext, &caps, pDeviceInfo);
27383 | }
27384 |
27385 | static ma_result ma_context_get_device_info_from_WAVEINCAPS2(ma_context* pContext, MA_WAVEINCAPS2*
> pCaps, ma_device_info* pDeviceInfo)
27386 | {
27387 |     MA_WAVECAPSA caps;
27388 |
27389 |     MA_ASSERT(pContext != NULL);
27390 |     MA_ASSERT(pCaps != NULL);
27391 |     MA_ASSERT(pDeviceInfo != NULL);
27392 |
27393 |     MA_COPY_MEMORY(caps.szPname, pCaps->szPname, sizeof(caps.szPname));
27394 |     caps.dwFormats = pCaps->dwFormats;
27395 |     caps.wChannels = pCaps->wChannels;
27396 |     caps.NameGuid = pCaps->NameGuid;
27397 |     return ma_context_get_device_info_from_WAVECAPS(pContext, &caps, pDeviceInfo);
27398 | }
27399 |
27400 |
27401 | static ma_result ma_context_enumerate_devices__winmm(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
27402 | {
27403 |     UINT playbackDeviceCount;
27404 |     UINT captureDeviceCount;
27405 |     UINT iPlaybackDevice;
27406 |     UINT iCaptureDevice;
27407 |
27408 |     MA_ASSERT(pContext != NULL);
27409 |     MA_ASSERT(callback != NULL);
27410 |
27411 |     /* Playback. */
27412 |     playbackDeviceCount = ((MA_PFN_waveOutGetNumDevs)pContext->winmm.waveOutGetNumDevs)();
27413 |     for (iPlaybackDevice = 0; iPlaybackDevice < playbackDeviceCount; ++iPlaybackDevice) {
27414 |         MA_MMRESULT result;
27415 |         MA_WAVEOUTCAPS2A caps;
27416 |
27417 |         MA_ZERO_OBJECT(&caps);
27418 |
27419 |         result = ((MA_PFN_waveOutGetDevCapsA)pContext->winmm.waveOutGetDevCapsA)(iPlaybackDevice,
> (MA_WAVEOUTCAPSA*)&caps, sizeof(caps));
27420 |         if (result == MA_MMSYSERR_NOERROR) {
27421 |             ma_device_info deviceInfo;
27422 |
27423 |             MA_ZERO_OBJECT(&deviceInfo);
27424 |             deviceInfo.id.winmm = iPlaybackDevice;
27425 |
27426 |             /* The first enumerated device is the default device. */
27427 |             if (iPlaybackDevice == 0) {
27428 |                 deviceInfo.isDefault = MA_TRUE;
27429 |             }
27430 |
27431 |             if (ma_context_get_device_info_from_WAVEOUTCAPS2(pContext, &caps, &deviceInfo) ==
> MA_SUCCESS) {
27432 |                 ma_bool32 cbResult = callback(pContext, ma_device_type_playback, &deviceInfo,
> pUserData);
27433 |                 if (cbResult == MA_FALSE) {
27434 |                     return MA_SUCCESS; /* Enumeration was stopped. */
27435 |                 }
27436 |             }
27437 |         }
27438 |     }
27439 |
27440 |     /* Capture. */
27441 |     captureDeviceCount = ((MA_PFN_waveInGetNumDevs)pContext->winmm.waveInGetNumDevs)();
27442 |     for (iCaptureDevice = 0; iCaptureDevice < captureDeviceCount; ++iCaptureDevice) {
27443 |         MA_MMRESULT result;
27444 |         MA_WAVEINCAPS2A caps;
27445 |
27446 |         MA_ZERO_OBJECT(&caps);
27447 |

```



```

27448 |         result = ((MA_PFN_waveInGetDevCapsA)pContext->winmm.waveInGetDevCapsA)(iCaptureDevice,
> (MA_WAVEINCAPSA*)&caps, sizeof(caps));
27449 |         if (result == MA_MMSYSERR_NOERROR) {
27450 |             ma_device_info deviceInfo;
27451 |
27452 |             MA_ZERO_OBJECT(&deviceInfo);
27453 |             deviceInfo.id.winmm = iCaptureDevice;
27454 |
27455 |             /* The first enumerated device is the default device. */
27456 |             if (iCaptureDevice == 0) {
27457 |                 deviceInfo.isDefault = MA_TRUE;
27458 |             }
27459 |
27460 |             if (ma_context_get_device_info_from_WAVEINCAPS2(pContext, &caps, &deviceInfo) ==
> MA_SUCCESS) {
27461 |                 ma_bool32 cbResult = callback(pContext, ma_device_type_capture, &deviceInfo,
> pUserData);
27462 |                 if (cbResult == MA_FALSE) {
27463 |                     return MA_SUCCESS; /* Enumeration was stopped. */
27464 |                 }
27465 |             }
27466 |         }
27467 |     }
27468 |
27469 |     return MA_SUCCESS;
27470 | }
27471 |
27472 | static ma_result ma_context_get_device_info_winmm(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
27473 | {
27474 |     UINT winMMDeviceID;
27475 |
27476 |     MA_ASSERT(pContext != NULL);
27477 |
27478 |     winMMDeviceID = 0;
27479 |     if (pDeviceID != NULL) {
27480 |         winMMDeviceID = (UINT)pDeviceID->winmm;
27481 |     }
27482 |
27483 |     pDeviceInfo->id.winmm = winMMDeviceID;
27484 |
27485 |     /* The first ID is the default device. */
27486 |     if (winMMDeviceID == 0) {
27487 |         pDeviceInfo->isDefault = MA_TRUE;
27488 |     }
27489 |
27490 |     if (deviceType == ma_device_type_playback) {
27491 |         MA_MMRESULT result;
27492 |         MA_WAVEOUTCAPS2A caps;
27493 |
27494 |         MA_ZERO_OBJECT(&caps);
27495 |
27496 |         result = ((MA_PFN_waveOutGetDevCapsA)pContext->winmm.waveOutGetDevCapsA)(winMMDeviceID,
> (MA_WAVEOUTCAPSA*)&caps, sizeof(caps));
27497 |         if (result == MA_MMSYSERR_NOERROR) {
27498 |             return ma_context_get_device_info_from_WAVEOUTCAPS2(pContext, &caps, pDeviceInfo);
27499 |         }
27500 |     } else {
27501 |         MA_MMRESULT result;
27502 |         MA_WAVEINCAPS2A caps;
27503 |
27504 |         MA_ZERO_OBJECT(&caps);
27505 |
27506 |         result = ((MA_PFN_waveInGetDevCapsA)pContext->winmm.waveInGetDevCapsA)(winMMDeviceID,
> (MA_WAVEINCAPSA*)&caps, sizeof(caps));
27507 |         if (result == MA_MMSYSERR_NOERROR) {
27508 |             return ma_context_get_device_info_from_WAVEINCAPS2(pContext, &caps, pDeviceInfo);
27509 |         }
27510 |     }
27511 |
27512 |     return MA_NO_DEVICE;
27513 | }
27514 |
27515 |
27516 | static ma_result ma_device_uninit__winmm(ma_device* pDevice)
27517 | {
27518 |     MA_ASSERT(pDevice != NULL);
27519 |
27520 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {

```

```

27521 |         ((MA_PFN_waveInClose)pDevice->pContext->winmm.waveInClose)((MA_HWAVEIN)pDevice->winmm.hDevic
> eCapture);
27522 |         CloseHandle((HANDLE)pDevice->winmm.hEventCapture);
27523 |     }
27524 |
27525 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
27526 |         ((MA_PFN_waveOutReset)pDevice->pContext->winmm.waveOutReset)((MA_HWAVEOUT)pDevice->winmm.hDe
> vicePlayback);
27527 |         ((MA_PFN_waveOutClose)pDevice->pContext->winmm.waveOutClose)((MA_HWAVEOUT)pDevice->winmm.hDe
> vicePlayback);
27528 |         CloseHandle((HANDLE)pDevice->winmm.hEventPlayback);
27529 |     }
27530 |
27531 |     ma_free(pDevice->winmm._pHeapData, &pDevice->pContext->allocationCallbacks);
27532 |
27533 |     MA_ZERO_OBJECT(&pDevice->winmm);    /* Safety. */
27534 |
27535 |     return MA_SUCCESS;
27536 | }
27537 |
27538 | static ma_uint32 ma_calculate_period_size_in_frames_from_descriptor__winmm(const
> ma_device_descriptor* pDescriptor, ma_uint32 nativeSampleRate, ma_performance_profile
> performanceProfile)
27539 | {
27540 |     /* WinMM has a minimum period size of 40ms. */
27541 |     ma_uint32 minPeriodSizeInFrames = ma_calculate_buffer_size_in_frames_from_milliseconds(40,
> nativeSampleRate);
27542 |     ma_uint32 periodSizeInFrames;
27543 |
27544 |     periodSizeInFrames = ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor,
> nativeSampleRate, performanceProfile);
27545 |     if (periodSizeInFrames < minPeriodSizeInFrames) {
27546 |         periodSizeInFrames = minPeriodSizeInFrames;
27547 |     }
27548 |
27549 |     return periodSizeInFrames;
27550 | }
27551 |
27552 | static ma_result ma_device_init__winmm(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
27553 | {
27554 |     const char* errorMsg = "";
27555 |     ma_result errorCode = MA_ERROR;
27556 |     ma_result result = MA_SUCCESS;
27557 |     ma_uint32 heapSize;
27558 |     UINT winMMDeviceIDPlayback = 0;
27559 |     UINT winMMDeviceIDCapture = 0;
27560 |
27561 |     MA_ASSERT(pDevice != NULL);
27562 |
27563 |     MA_ZERO_OBJECT(&pDevice->winmm);
27564 |
27565 |     if (pConfig->deviceType == ma_device_type_loopback) {
27566 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
27567 |     }
27568 |
27569 |     /* No exclusive mode with WinMM. */
27570 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->shareMode == ma_share_mode_exclusive) ||
27571 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->shareMode == ma_share_mode_exclusive)) {
27572 |         return MA_SHARE_MODE_NOT_SUPPORTED;
27573 |     }
27574 |
27575 |     if (pDescriptorPlayback->pDeviceID != NULL) {
27576 |         winMMDeviceIDPlayback = (UINT)pDescriptorPlayback->pDeviceID->winmm;
27577 |     }
27578 |     if (pDescriptorCapture->pDeviceID != NULL) {
27579 |         winMMDeviceIDCapture = (UINT)pDescriptorCapture->pDeviceID->winmm;
27580 |     }
27581 |
27582 |     /* The capture device needs to be initialized first. */
27583 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
27584 |         MA_WAVEINCAPSA caps;
27585 |         MA_WAVEFORMATEX wf;
27586 |         MA_MMRESULT resultMM;
27587 |
27588 |         /* We use an event to know when a new fragment needs to be enqueued. */

```

```

27589 |         pDevice->winmm.hEventCapture = (ma_handle)CreateEventA(NULL, TRUE, TRUE, NULL);
27590 |         if (pDevice->winmm.hEventCapture == NULL) {
27591 |             errorMsg = "[WinMM] Failed to create event for fragment enqueueing for the capture
> device.", errorCode = ma_result_from_GetLastError(GetLastError());
27592 |             goto on_error;
27593 |         }
27594 |
27595 |         /* The format should be based on the device's actual format. */
27596 |         if
> (((MA_PFN_waveInGetDevCapsA)pDevice->pContext->winmm.waveInGetDevCapsA)(winMMDeviceIDCapture, &caps,
> sizeof(caps)) != MA_MMSYSERR_NOERROR) {
27597 |             errorMsg = "[WinMM] Failed to retrieve internal device caps.", errorCode =
> MA_FORMAT_NOT_SUPPORTED;
27598 |             goto on_error;
27599 |         }
27600 |
27601 |         result = ma_formats_flags_to_WAVEFORMATEX__winmm(caps.dwFormats, caps.wChannels, &wf);
27602 |         if (result != MA_SUCCESS) {
27603 |             errorMsg = "[WinMM] Could not find appropriate format for internal device.", errorCode =
> result;
27604 |             goto on_error;
27605 |         }
27606 |
27607 |         resultMM = ((MA_PFN_waveInOpen)pDevice->pContext->winmm.waveInOpen)((MA_HWAVEIN*)&pDevice->w
> inmm.hDeviceCapture, winMMDeviceIDCapture, &wf, (DWORD_PTR)pDevice->winmm.hEventCapture,
> (DWORD_PTR)pDevice, MA_CALLBACK_EVENT | MA_WAVE_ALLOWSYNC);
27608 |         if (resultMM != MA_MMSYSERR_NOERROR) {
27609 |             errorMsg = "[WinMM] Failed to open capture device.", errorCode =
> MA_FAILED_TO_OPEN_BACKEND_DEVICE;
27610 |             goto on_error;
27611 |         }
27612 |
27613 |         pDescriptorCapture->format = ma_format_from_WAVEFORMATEX(&wf);
27614 |         pDescriptorCapture->channels = wf.nChannels;
27615 |         pDescriptorCapture->sampleRate = wf.nSamplesPerSec;
27616 |         ma_channel_map_init_standard(ma_standard_channel_map_microsoft,
> pDescriptorCapture->channelMap, ma_countof(pDescriptorCapture->channelMap),
> pDescriptorCapture->channels);
27617 |         pDescriptorCapture->periodCount = pDescriptorCapture->periodCount;
27618 |         pDescriptorCapture->periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__winmm(pDescriptorCapture,
> pDescriptorCapture->sampleRate, pConfig->performanceProfile);
27619 |     }
27620 |
27621 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
27622 |         MA_WAVEOUTCAPSA caps;
27623 |         MA_WAVEFORMATEX wf;
27624 |         MA_MMRESULT resultMM;
27625 |
27626 |         /* We use an event to know when a new fragment needs to be enqueued. */
27627 |         pDevice->winmm.hEventPlayback = (ma_handle)CreateEventA(NULL, TRUE, TRUE, NULL);
27628 |         if (pDevice->winmm.hEventPlayback == NULL) {
27629 |             errorMsg = "[WinMM] Failed to create event for fragment enqueueing for the playback
> device.", errorCode = ma_result_from_GetLastError(GetLastError());
27630 |             goto on_error;
27631 |         }
27632 |
27633 |         /* The format should be based on the device's actual format. */
27634 |         if
> (((MA_PFN_waveOutGetDevCapsA)pDevice->pContext->winmm.waveOutGetDevCapsA)(winMMDeviceIDPlayback,
> &caps, sizeof(caps)) != MA_MMSYSERR_NOERROR) {
27635 |             errorMsg = "[WinMM] Failed to retrieve internal device caps.", errorCode =
> MA_FORMAT_NOT_SUPPORTED;
27636 |             goto on_error;
27637 |         }
27638 |
27639 |         result = ma_formats_flags_to_WAVEFORMATEX__winmm(caps.dwFormats, caps.wChannels, &wf);
27640 |         if (result != MA_SUCCESS) {
27641 |             errorMsg = "[WinMM] Could not find appropriate format for internal device.", errorCode =
> result;
27642 |             goto on_error;
27643 |         }
27644 |
27645 |         resultMM = ((MA_PFN_waveOutOpen)pDevice->pContext->winmm.waveOutOpen)((MA_HWAVEOUT*)&pDevice
> ->winmm.hDevicePlayback, winMMDeviceIDPlayback, &wf, (DWORD_PTR)pDevice->winmm.hEventPlayback,
> (DWORD_PTR)pDevice, MA_CALLBACK_EVENT | MA_WAVE_ALLOWSYNC);
27646 |         if (resultMM != MA_MMSYSERR_NOERROR) {
27647 |             errorMsg = "[WinMM] Failed to open playback device.", errorCode =

```

```

> MA_FAILED_TO_OPEN_BACKEND_DEVICE;
27648 |         goto on_error;
27649 |     }
27650 |
27651 |     pDescriptorPlayback->format          = ma_format_from_WAVEFORMATEX(&wf);
27652 |     pDescriptorPlayback->channels        = wf.nChannels;
27653 |     pDescriptorPlayback->sampleRate      = wf.nSamplesPerSec;
27654 |     ma_channel_map_init_standard(ma_standard_channel_map_microsoft,
> pDescriptorPlayback->channelMap, ma_countof(pDescriptorPlayback->channelMap),
> pDescriptorPlayback->channels);
27655 |     pDescriptorPlayback->periodCount     = pDescriptorPlayback->periodCount;
27656 |     pDescriptorPlayback->periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__winmm(pDescriptorPlayback,
> pDescriptorPlayback->sampleRate, pConfig->performanceProfile);
27657 |     }
27658 |
27659 |     /*
27660 |     The heap allocated data is allocated like so:
27661 |
27662 |     [Capture WAVEHDRs][Playback WAVEHDRs][Capture Intermediary Buffer][Playback Intermediary Buffer]
27663 |     */
27664 |     heapSize = 0;
27665 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
27666 |         heapSize += sizeof(MA_WAVEHDR)*pDescriptorCapture->periodCount +
> (pDescriptorCapture->periodSizeInFrames * pDescriptorCapture->periodCount *
> ma_get_bytes_per_frame(pDescriptorCapture->format, pDescriptorCapture->channels));
27667 |     }
27668 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
27669 |         heapSize += sizeof(MA_WAVEHDR)*pDescriptorPlayback->periodCount +
> (pDescriptorPlayback->periodSizeInFrames * pDescriptorPlayback->periodCount *
> ma_get_bytes_per_frame(pDescriptorPlayback->format, pDescriptorPlayback->channels));
27670 |     }
27671 |
27672 |     pDevice->winmm._pHeapData = (ma_uint8*)ma_calloc(heapSize,
> &pDevice->pContext->allocationCallbacks);
27673 |     if (pDevice->winmm._pHeapData == NULL) {
27674 |         errorMsg = "[WinMM] Failed to allocate memory for the intermediary buffer.", errorCode =
> MA_OUT_OF_MEMORY;
27675 |         goto on_error;
27676 |     }
27677 |
27678 |     MA_ZERO_MEMORY(pDevice->winmm._pHeapData, heapSize);
27679 |
27680 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
27681 |         ma_uint32 iPeriod;
27682 |
27683 |         if (pConfig->deviceType == ma_device_type_capture) {
27684 |             pDevice->winmm.pWAVEHDRCapture = pDevice->winmm._pHeapData;
27685 |             pDevice->winmm.pIntermediaryBufferCapture = pDevice->winmm._pHeapData +
> (sizeof(MA_WAVEHDR)*(pDescriptorCapture->periodCount));
27686 |         } else {
27687 |             pDevice->winmm.pWAVEHDRCapture = pDevice->winmm._pHeapData;
27688 |             pDevice->winmm.pIntermediaryBufferCapture = pDevice->winmm._pHeapData +
> (sizeof(MA_WAVEHDR)*(pDescriptorCapture->periodCount + pDescriptorPlayback->periodCount));
27689 |         }
27690 |
27691 |         /* Prepare headers. */
27692 |         for (iPeriod = 0; iPeriod < pDescriptorCapture->periodCount; ++iPeriod) {
27693 |             ma_uint32 periodSizeInBytes =
> ma_get_period_size_in_bytes(pDescriptorCapture->periodSizeInFrames, pDescriptorCapture->format,
> pDescriptorCapture->channels);
27694 |
27695 |             ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod].lpData =
> (char*)(pDevice->winmm.pIntermediaryBufferCapture + (periodSizeInBytes*iPeriod));
27696 |             ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod].dwBufferLength =
> periodSizeInBytes;
27697 |             ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod].dwFlags = 0L;
27698 |             ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod].dwLoops = 0L;
27699 |             ((MA_PFN_waveInPrepareHeader)pDevice->pContext->winmm.waveInPrepareHeader)((MA_HWAVEIN)p
> Device->winmm.hDeviceCapture, &((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod],
> sizeof(MA_WAVEHDR));
27700 |
27701 |             /*
27702 |             The user data of the MA_WAVEHDR structure is a single flag the controls whether or not
> it is ready for writing. Consider it to be named "isLocked". A value of 0 means
27703 |             it's unlocked and available for writing. A value of 1 means it's locked.

```

```

27704 |         */
27705 |         ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod].dwUser = 0;
27706 |     }
27707 | }
27708 |
27709 | if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
27710 |     ma_uint32 iPeriod;
27711 |
27712 |     if (pConfig->deviceType == ma_device_type_playback) {
27713 |         pDevice->winmm.pWAVEHDRPlayback = pDevice->winmm._pHeapData;
27714 |         pDevice->winmm.pIntermediaryBufferPlayback = pDevice->winmm._pHeapData +
> (sizeof(MA_WAVEHDR)*pDescriptorPlayback->periodCount);
27715 |     } else {
27716 |         pDevice->winmm.pWAVEHDRPlayback = pDevice->winmm._pHeapData +
> (sizeof(MA_WAVEHDR)*(pDescriptorCapture->periodCount));
27717 |         pDevice->winmm.pIntermediaryBufferPlayback = pDevice->winmm._pHeapData +
> (sizeof(MA_WAVEHDR)*(pDescriptorCapture->periodCount + pDescriptorPlayback->periodCount)) + (pDescr
> ptorCapture->periodSizeInFrames*pDescriptorCapture->periodCount*ma_get_bytes_per_frame(pDescriptorCa
> pture->format, pDescriptorCapture->channels));
27718 |     }
27719 |
27720 |     /* Prepare headers. */
27721 |     for (iPeriod = 0; iPeriod < pDescriptorPlayback->periodCount; ++iPeriod) {
27722 |         ma_uint32 periodSizeInBytes =
> ma_get_period_size_in_bytes(pDescriptorPlayback->periodSizeInFrames, pDescriptorPlayback->format,
> pDescriptorPlayback->channels);
27723 |
27724 |         ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod].lpData =
> (char*)(pDevice->winmm.pIntermediaryBufferPlayback + (periodSizeInBytes*iPeriod));
27725 |         ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod].dwBufferLength =
> periodSizeInBytes;
27726 |         ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod].dwFlags = 0L;
27727 |         ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod].dwLoops = 0L;
27728 |         ((MA_PFN_waveOutPrepareHeader)pDevice->pContext->winmm.waveOutPrepareHeader)((MA_HWAVEOU
> T)pDevice->winmm.hDevicePlayback, &((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod],
> sizeof(MA_WAVEHDR));
27729 |
27730 |         /*
27731 |         The user data of the MA_WAVEHDR structure is a single flag the controls whether or not
> it is ready for writing. Consider it to be named "isLocked". A value of 0 means
27732 |         it's unlocked and available for writing. A value of 1 means it's locked.
27733 |         */
27734 |         ((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod].dwUser = 0;
27735 |     }
27736 | }
27737 |
27738 | return MA_SUCCESS;
27739 |
27740 | on_error:
27741 | if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
27742 |     if (pDevice->winmm.pWAVEHDRCapture != NULL) {
27743 |         ma_uint32 iPeriod;
27744 |         for (iPeriod = 0; iPeriod < pDescriptorCapture->periodCount; ++iPeriod) {
27745 |             ((MA_PFN_waveInUnprepareHeader)pDevice->pContext->winmm.waveInUnprepareHeader)((MA_H
> WAVEIN)pDevice->winmm.hDeviceCapture, &((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod],
> sizeof(MA_WAVEHDR));
27746 |         }
27747 |     }
27748 |
27749 |     ((MA_PFN_waveInClose)pDevice->pContext->winmm.waveInClose)((MA_HWAVEIN)pDevice->winmm.hDevic
> eCapture);
27750 | }
27751 |
27752 | if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
27753 |     if (pDevice->winmm.pWAVEHDRCapture != NULL) {
27754 |         ma_uint32 iPeriod;
27755 |         for (iPeriod = 0; iPeriod < pDescriptorPlayback->periodCount; ++iPeriod) {
27756 |             ((MA_PFN_waveOutUnprepareHeader)pDevice->pContext->winmm.waveOutUnprepareHeader)((MA
> _HWAVEOUT)pDevice->winmm.hDevicePlayback, &((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback)[iPeriod],
> sizeof(MA_WAVEHDR));
27757 |         }
27758 |     }
27759 |
27760 |     ((MA_PFN_waveOutClose)pDevice->pContext->winmm.waveOutClose)((MA_HWAVEOUT)pDevice->winmm.hDe
> vicePlayback);
27761 | }
27762 |
27763 | ma_free(pDevice->winmm._pHeapData, &pDevice->pContext->allocationCallbacks);

```

```

27764 |
27765 |     if (errorMsg != NULL && errorMsg[0] != '\0') {
27766 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "%s", errorMsg);
27767 |     }
27768 |
27769 |     return errorCode;
27770 | }
27771 |
27772 | static ma_result ma_device_start_winmm(ma_device* pDevice)
27773 | {
27774 |     MA_ASSERT(pDevice != NULL);
27775 |
27776 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
27777 |         MA_MMRESULT resultMM;
27778 |         MA_WAVEHDR* pWAVEHDR;
27779 |         ma_uint32 iPeriod;
27780 |
27781 |         pWAVEHDR = (MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture;
27782 |
27783 |         /* Make sure the event is reset to a non-signaled state to ensure we don't prematurely
> return from WaitForSingleObject(). */
27784 |         ResetEvent((HANDLE)pDevice->winmm.hEventCapture);
27785 |
27786 |         /* To start the device we attach all of the buffers and then start it. As the buffers are
> filled with data we will get notifications. */
27787 |         for (iPeriod = 0; iPeriod < pDevice->capture.internalPeriods; ++iPeriod) {
27788 |             resultMM = ((MA_PFN_waveInAddBuffer)pDevice->pContext->winmm.waveInAddBuffer)((MA_HWAVEI
> N)pDevice->winmm.hDeviceCapture, &((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[iPeriod],
> sizeof(MA_WAVEHDR));
27789 |             if (resultMM != MA_MMSYSERR_NOERROR) {
27790 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WinMM] Failed to
> attach input buffers to capture device in preparation for capture.");
27791 |                 return ma_result_from_MMRESULT(resultMM);
27792 |             }
27793 |
27794 |             /* Make sure all of the buffers start out locked. We don't want to access them until the
> backend tells us we can. */
27795 |             pWAVEHDR[iPeriod].dwUser = 1; /* 1 = locked. */
27796 |         }
27797 |
27798 |         /* Capture devices need to be explicitly started, unlike playback devices. */
27799 |         resultMM = ((MA_PFN_waveInStart)pDevice->pContext->winmm.waveInStart)((MA_HWAVEIN)pDevice->w
> inmm.hDeviceCapture);
27800 |         if (resultMM != MA_MMSYSERR_NOERROR) {
27801 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WinMM] Failed to start
> backend device.");
27802 |             return ma_result_from_MMRESULT(resultMM);
27803 |         }
27804 |     }
27805 |
27806 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
27807 |         /* Don't need to do anything for playback. It'll be started automatically in
> ma_device_start_winmm(). */
27808 |     }
27809 |
27810 |     return MA_SUCCESS;
27811 | }
27812 |
27813 | static ma_result ma_device_stop_winmm(ma_device* pDevice)
27814 | {
27815 |     MA_MMRESULT resultMM;
27816 |
27817 |     MA_ASSERT(pDevice != NULL);
27818 |
27819 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
27820 |         if (pDevice->winmm.hDeviceCapture == NULL) {
27821 |             return MA_INVALID_ARGS;
27822 |         }
27823 |
27824 |         resultMM = ((MA_PFN_waveInReset)pDevice->pContext->winmm.waveInReset)((MA_HWAVEIN)pDevice->w
> inmm.hDeviceCapture);
27825 |         if (resultMM != MA_MMSYSERR_NOERROR) {
27826 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[WinMM] WARNING: Failed
> to reset capture device.");
27827 |         }
27828 |     }
27829 |
27830 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
27831 |         ma_uint32 iPeriod;

```



```

27832 |         MA_WAVEHDR* pWAVEHDR;
27833 |
27834 |         if (pDevice->winmm.hDevicePlayback == NULL) {
27835 |             return MA_INVALID_ARGS;
27836 |         }
27837 |
27838 |         /* We need to drain the device. To do this we just loop over each header and if it's locked
> just wait for the event. */
27839 |         pWAVEHDR = (MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback;
27840 |         for (iPeriod = 0; iPeriod < pDevice->playback.internalPeriods; iPeriod += 1) {
27841 |             if (pWAVEHDR[iPeriod].dwUser == 1) { /* 1 = locked. */
27842 |                 if (WaitForSingleObject((HANDLE)pDevice->winmm.hEventPlayback, INFINITE) !=
> WAIT_OBJECT_0) {
27843 |                     break; /* An error occurred so just abandon ship and stop the device without
> draining. */
27844 |                 }
27845 |
27846 |                 pWAVEHDR[iPeriod].dwUser = 0;
27847 |             }
27848 |         }
27849 |
27850 |         resultMM = ((MA_PFN_waveOutReset)pDevice->pContext->winmm.waveOutReset)((MA_HWAVEOUT)pDevice
> ->winmm.hDevicePlayback);
27851 |         if (resultMM != MA_MMSYSERR_NOERROR) {
27852 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[winMM] WARNING: Failed
> to reset playback device.");
27853 |         }
27854 |     }
27855 |
27856 |     return MA_SUCCESS;
27857 | }
27858 |
27859 | static ma_result ma_device_write_winmm(ma_device* pDevice, const void* pPCMFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
27860 | {
27861 |     ma_result result = MA_SUCCESS;
27862 |     MA_MMRESULT resultMM;
27863 |     ma_uint32 totalFramesWritten;
27864 |     MA_WAVEHDR* pWAVEHDR;
27865 |
27866 |     MA_ASSERT(pDevice != NULL);
27867 |     MA_ASSERT(pPCMFrames != NULL);
27868 |
27869 |     if (pFramesWritten != NULL) {
27870 |         *pFramesWritten = 0;
27871 |     }
27872 |
27873 |     pWAVEHDR = (MA_WAVEHDR*)pDevice->winmm.pWAVEHDRPlayback;
27874 |
27875 |     /* Keep processing as much data as possible. */
27876 |     totalFramesWritten = 0;
27877 |     while (totalFramesWritten < frameCount) {
27878 |         /* If the current header has some space available we need to write part of it. */
27879 |         if (pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwUser == 0) { /* 0 = unlocked. */
27880 |             /*
27881 |             This header has room in it. We copy as much of it as we can. If we end up fully
> consuming the buffer we need to
27882 |             write it out and move on to the next iteration.
27883 |             */
27884 |             ma_uint32 bpf = ma_get_bytes_per_frame(pDevice->playback.internalFormat,
> pDevice->playback.internalChannels);
27885 |             ma_uint32 framesRemainingInHeader =
> (pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwBufferLength/bpf) -
> pDevice->winmm.headerFramesConsumedPlayback;
27886 |
27887 |             ma_uint32 framesToCopy = ma_min(framesRemainingInHeader, (frameCount -
> totalFramesWritten));
27888 |             const void* pSrc = ma_offset_ptr(pPCMFrames, totalFramesWritten*bpf);
27889 |             void* pDst = ma_offset_ptr(pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].lpData,
> pDevice->winmm.headerFramesConsumedPlayback*bpf);
27890 |             MA_COPY_MEMORY(pDst, pSrc, framesToCopy*bpf);
27891 |
27892 |             pDevice->winmm.headerFramesConsumedPlayback += framesToCopy;
27893 |             totalFramesWritten += framesToCopy;
27894 |
27895 |             /* If we've consumed the buffer entirely we need to write it out to the device. */
27896 |             if (pDevice->winmm.headerFramesConsumedPlayback ==
> (pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwBufferLength/bpf)) {
27897 |                 pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwUser = 1; /* 1 = locked.

```



```

> */
27898 |         pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwFlags &= ~MA_WHDR_DONE; /* <-- Need
> to make sure the WHDR_DONE flag is unset. */
27899 |
27900 |         /* Make sure the event is reset to a non-signaled state to ensure we don't
> prematurely return from WaitForSingleObject(). */
27901 |         ResetEvent((HANDLE)pDevice->winmm.hEventPlayback);
27902 |
27903 |         /* The device will be started here. */
27904 |         resultMM = ((MA_PFN_waveOutWrite)pDevice->pContext->winmm.waveOutWrite)((MA_HWAVEOUT
> )pDevice->winmm.hDevicePlayback, &pWAVEHDR[pDevice->winmm.iNextHeaderPlayback], sizeof(MA_WAVEHDR));
27905 |         if (resultMM != MA_MMSYSERR_NOERROR) {
27906 |             result = ma_result_from_MMRESULT(resultMM);
27907 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[winMM]
> waveOutWrite() failed.");
27908 |             break;
27909 |         }
27910 |
27911 |         /* Make sure we move to the next header. */
27912 |         pDevice->winmm.iNextHeaderPlayback = (pDevice->winmm.iNextHeaderPlayback + 1) %
> pDevice->playback.internalPeriods;
27913 |         pDevice->winmm.headerFramesConsumedPlayback = 0;
27914 |     }
27915 |
27916 |     /* If at this point we have consumed the entire input buffer we can return. */
27917 |     MA_ASSERT(totalFramesWritten <= frameCount);
27918 |     if (totalFramesWritten == frameCount) {
27919 |         break;
27920 |     }
27921 |
27922 |     /* Getting here means there's more to process. */
27923 |     continue;
27924 | }
27925 |
27926 |     /* Getting here means there isn't enough room in the buffer and we need to wait for one to
> become available. */
27927 |     if (WaitForSingleObject((HANDLE)pDevice->winmm.hEventPlayback, INFINITE) != WAIT_OBJECT_0) {
27928 |         result = MA_ERROR;
27929 |         break;
27930 |     }
27931 |
27932 |     /* Something happened. If the next buffer has been marked as done we need to reset a bit of
> state. */
27933 |     if ((pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwFlags & MA_WHDR_DONE) != 0) {
27934 |         pWAVEHDR[pDevice->winmm.iNextHeaderPlayback].dwUser = 0; /* 0 = unlocked (make it
> available for writing). */
27935 |         pDevice->winmm.headerFramesConsumedPlayback = 0;
27936 |     }
27937 |
27938 |     /* If the device has been stopped we need to break. */
27939 |     if (ma_device_get_state(pDevice) != ma_device_state_started) {
27940 |         break;
27941 |     }
27942 | }
27943 |
27944 | if (pFramesWritten != NULL) {
27945 |     *pFramesWritten = totalFramesWritten;
27946 | }
27947 |
27948 | return result;
27949 | }
27950 |
27951 | static ma_result ma_device_read__winmm(ma_device* pDevice, void* pPCMFrames, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
27952 | {
27953 |     ma_result result = MA_SUCCESS;
27954 |     MA_MMRESULT resultMM;
27955 |     ma_uint32 totalFramesRead;
27956 |     MA_WAVEHDR* pWAVEHDR;
27957 |
27958 |     MA_ASSERT(pDevice != NULL);
27959 |     MA_ASSERT(pPCMFrames != NULL);
27960 |
27961 |     if (pFramesRead != NULL) {
27962 |         *pFramesRead = 0;
27963 |     }
27964 |
27965 |     pWAVEHDR = (MA_WAVEHDR*)pDevice->winmm.pWAVEHDCapture;
27966 |

```

```

27967 |     /* Keep processing as much data as possible. */
27968 |     totalFramesRead = 0;
27969 |     while (totalFramesRead < frameCount) {
27970 |         /* If the current header has some space available we need to write part of it. */
27971 |         if (pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwUser == 0) { /* 0 = unlocked. */
27972 |             /* The buffer is available for reading. If we fully consume it we need to add it back to
> the buffer. */
27973 |             ma_uint32 bpf = ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels);
27974 |             ma_uint32 framesRemainingInHeader =
> (pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwBufferLength/bpf) -
> pDevice->winmm.headerFramesConsumedCapture;
27975 |
27976 |             ma_uint32 framesToCopy = ma_min(framesRemainingInHeader, (frameCount -
> totalFramesRead));
27977 |             const void* pSrc = ma_offset_ptr(pWAVEHDR[pDevice->winmm.iNextHeaderCapture].lpData,
> pDevice->winmm.headerFramesConsumedCapture*bpf);
27978 |             void* pDst = ma_offset_ptr(pPCMFrames, totalFramesRead*bpf);
27979 |             MA_COPY_MEMORY(pDst, pSrc, framesToCopy*bpf);
27980 |
27981 |             pDevice->winmm.headerFramesConsumedCapture += framesToCopy;
27982 |             totalFramesRead += framesToCopy;
27983 |
27984 |             /* If we've consumed the buffer entirely we need to add it back to the device. */
27985 |             if (pDevice->winmm.headerFramesConsumedCapture ==
> (pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwBufferLength/bpf)) {
27986 |                 pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwUser = 1; /* 1 = locked. */
27987 |                 pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwFlags &= ~MA_WHDR_DONE; /* <-- Need to
> make sure the WHDR_DONE flag is unset. */
27988 |
27989 |                 /* Make sure the event is reset to a non-signaled state to ensure we don't
> prematurely return from WaitForSingleObject(). */
27990 |                 ResetEvent((HANDLE)pDevice->winmm.hEventCapture);
27991 |
27992 |                 /* The device will be started here. */
27993 |                 resultMM = ((MA_PFN_waveInAddBuffer)pDevice->pContext->winmm.waveInAddBuffer)((MA_HW
> AVEIN)pDevice->winmm.hDeviceCapture,
> &((MA_WAVEHDR*)pDevice->winmm.pWAVEHDRCapture)[pDevice->winmm.iNextHeaderCapture],
> sizeof(MA_WAVEHDR));
27994 |                 if (resultMM != MA_MMSYSERR_NOERROR) {
27995 |                     result = ma_result_from_MMRESULT(resultMM);
27996 |                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[WinMM]
> waveInAddBuffer() failed.");
27997 |                     break;
27998 |                 }
27999 |
28000 |                 /* Make sure we move to the next header. */
28001 |                 pDevice->winmm.iNextHeaderCapture = (pDevice->winmm.iNextHeaderCapture + 1) %
> pDevice->capture.internalPeriods;
28002 |                 pDevice->winmm.headerFramesConsumedCapture = 0;
28003 |             }
28004 |
28005 |             /* If at this point we have filled the entire input buffer we can return. */
28006 |             MA_ASSERT(totalFramesRead <= frameCount);
28007 |             if (totalFramesRead == frameCount) {
28008 |                 break;
28009 |             }
28010 |
28011 |             /* Getting here means there's more to process. */
28012 |             continue;
28013 |         }
28014 |
28015 |         /* Getting here means there isn't enough any data left to send to the client which means we
> need to wait for more. */
28016 |         if (WaitForSingleObject((HANDLE)pDevice->winmm.hEventCapture, INFINITE) != WAIT_OBJECT_0) {
28017 |             result = MA_ERROR;
28018 |             break;
28019 |         }
28020 |
28021 |         /* Something happened. If the next buffer has been marked as done we need to reset a bit of
> state. */
28022 |         if ((pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwFlags & MA_WHDR_DONE) != 0) {
28023 |             pWAVEHDR[pDevice->winmm.iNextHeaderCapture].dwUser = 0; /* 0 = unlocked (make it
> available for reading). */
28024 |             pDevice->winmm.headerFramesConsumedCapture = 0;
28025 |         }
28026 |
28027 |         /* If the device has been stopped we need to break. */
28028 |         if (ma_device_get_state(pDevice) != ma_device_state_started) {

```

```

28029 |         break;
28030 |     }
28031 | }
28032 |
28033 | if (pFramesRead != NULL) {
28034 |     *pFramesRead = totalFramesRead;
28035 | }
28036 |
28037 | return result;
28038 | }
28039 |
28040 | static ma_result ma_context_uninit__winmm(ma_context* pContext)
28041 | {
28042 |     MA_ASSERT(pContext != NULL);
28043 |     MA_ASSERT(pContext->backend == ma_backend_winmm);
28044 |
28045 |     ma_dlclose(ma_context_get_log(pContext), pContext->winmm.hWinMM);
28046 |     return MA_SUCCESS;
28047 | }
28048 |
28049 | static ma_result ma_context_init__winmm(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
28050 | {
28051 |     MA_ASSERT(pContext != NULL);
28052 |
28053 |     (void)pConfig;
28054 |
28055 |     pContext->winmm.hWinMM = ma_dlopen(ma_context_get_log(pContext), "winmm.dll");
28056 |     if (pContext->winmm.hWinMM == NULL) {
28057 |         return MA_NO_BACKEND;
28058 |     }
28059 |
28060 |     pContext->winmm.waveOutGetNumDevs = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutGetNumDevs");
28061 |     pContext->winmm.waveOutGetDevCapsA = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutGetDevCapsA");
28062 |     pContext->winmm.waveOutOpen = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutOpen");
28063 |     pContext->winmm.waveOutClose = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutClose");
28064 |     pContext->winmm.waveOutPrepareHeader = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutPrepareHeader");
28065 |     pContext->winmm.waveOutUnprepareHeader = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutUnprepareHeader");
28066 |     pContext->winmm.waveOutWrite = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutWrite");
28067 |     pContext->winmm.waveOutReset = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveOutReset");
28068 |     pContext->winmm.waveInGetNumDevs = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInGetNumDevs");
28069 |     pContext->winmm.waveInGetDevCapsA = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInGetDevCapsA");
28070 |     pContext->winmm.waveInOpen = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInOpen");
28071 |     pContext->winmm.waveInClose = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInClose");
28072 |     pContext->winmm.waveInPrepareHeader = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInPrepareHeader");
28073 |     pContext->winmm.waveInUnprepareHeader = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInUnprepareHeader");
28074 |     pContext->winmm.waveInAddBuffer = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInAddBuffer");
28075 |     pContext->winmm.waveInStart = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInStart");
28076 |     pContext->winmm.waveInReset = ma_dlsym(ma_context_get_log(pContext),
> pContext->winmm.hWinMM, "waveInReset");
28077 |
28078 |     pCallbacks->onContextInit = ma_context_init__winmm;
28079 |     pCallbacks->onContextUninit = ma_context_uninit__winmm;
28080 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__winmm;
28081 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__winmm;
28082 |     pCallbacks->onDeviceInit = ma_device_init__winmm;
28083 |     pCallbacks->onDeviceUninit = ma_device_uninit__winmm;
28084 |     pCallbacks->onDeviceStart = ma_device_start__winmm;
28085 |     pCallbacks->onDeviceStop = ma_device_stop__winmm;
28086 |     pCallbacks->onDeviceRead = ma_device_read__winmm;
28087 |     pCallbacks->onDeviceWrite = ma_device_write__winmm;
28088 |     pCallbacks->onDeviceDataLoop = NULL; /* This is a blocking read-write API, so this
> can be NULL since miniaudio will manage the audio thread for us. */

```

```

28089 |
28090 |     return MA_SUCCESS;
28091 | }
28092 | #endif
28093 |
28094 |
28095 |
28096 |
28097 | /*****
28098 |
28099 | ALSA Backend
28100 |
28101 | *****/
28102 | #ifdef MA_HAS_ALSA
28103 |
28104 | #include <poll.h>          /* poll(), struct pollfd */
28105 | #include <sys/eventfd.h>  /* eventfd() */
28106 |
28107 | #ifdef MA_NO_RUNTIME_LINKING
28108 |
28109 | /* asoundlib.h marks some functions with "inline" which isn't always supported. Need to emulate it.
> */
28110 | #if !defined(__cplusplus)
28111 |     #if defined(__STRICT_ANSI__)
28112 |         #if !defined(inline)
28113 |             #define inline __inline__ __attribute__((always_inline))
28114 |             #define MA_INLINE_DEFINED
28115 |         #endif
28116 |     #endif
28117 | #endif
28118 | #include <alsa/asoundlib.h>
28119 | #if defined(MA_INLINE_DEFINED)
28120 |     #undef inline
28121 |     #undef MA_INLINE_DEFINED
28122 | #endif
28123 |
28124 | typedef snd_pcm_uframes_t          ma_snd_pcm_uframes_t;
28125 | typedef snd_pcm_sframes_t          ma_snd_pcm_sframes_t;
28126 | typedef snd_pcm_stream_t           ma_snd_pcm_stream_t;
28127 | typedef snd_pcm_format_t           ma_snd_pcm_format_t;
28128 | typedef snd_pcm_access_t           ma_snd_pcm_access_t;
28129 | typedef snd_pcm_t                  ma_snd_pcm_t;
28130 | typedef snd_pcm_hw_params_t        ma_snd_pcm_hw_params_t;
28131 | typedef snd_pcm_sw_params_t        ma_snd_pcm_sw_params_t;
28132 | typedef snd_pcm_format_mask_t      ma_snd_pcm_format_mask_t;
28133 | typedef snd_pcm_info_t             ma_snd_pcm_info_t;
28134 | typedef snd_pcm_channel_area_t     ma_snd_pcm_channel_area_t;
28135 | typedef snd_pcm_chmap_t            ma_snd_pcm_chmap_t;
28136 | typedef snd_pcm_state_t            ma_snd_pcm_state_t;
28137 |
28138 | /* snd_pcm_state_t */
28139 | #define MA_SND_PCM_STATE_XRUN      SND_PCM_STATE_XRUN
28140 |
28141 | /* snd_pcm_stream_t */
28142 | #define MA_SND_PCM_STREAM_PLAYBACK SND_PCM_STREAM_PLAYBACK
28143 | #define MA_SND_PCM_STREAM_CAPTURE  SND_PCM_STREAM_CAPTURE
28144 |
28145 | /* snd_pcm_format_t */
28146 | #define MA_SND_PCM_FORMAT_UNKNOWN  SND_PCM_FORMAT_UNKNOWN
28147 | #define MA_SND_PCM_FORMAT_U8       SND_PCM_FORMAT_U8
28148 | #define MA_SND_PCM_FORMAT_S16_LE   SND_PCM_FORMAT_S16_LE
28149 | #define MA_SND_PCM_FORMAT_S16_BE   SND_PCM_FORMAT_S16_BE
28150 | #define MA_SND_PCM_FORMAT_S24_LE   SND_PCM_FORMAT_S24_LE
28151 | #define MA_SND_PCM_FORMAT_S24_BE   SND_PCM_FORMAT_S24_BE
28152 | #define MA_SND_PCM_FORMAT_S32_LE   SND_PCM_FORMAT_S32_LE
28153 | #define MA_SND_PCM_FORMAT_S32_BE   SND_PCM_FORMAT_S32_BE
28154 | #define MA_SND_PCM_FORMAT_FLOAT_LE  SND_PCM_FORMAT_FLOAT_LE
28155 | #define MA_SND_PCM_FORMAT_FLOAT_BE  SND_PCM_FORMAT_FLOAT_BE
28156 | #define MA_SND_PCM_FORMAT_FLOAT64_LE SND_PCM_FORMAT_FLOAT64_LE
28157 | #define MA_SND_PCM_FORMAT_FLOAT64_BE SND_PCM_FORMAT_FLOAT64_BE
28158 | #define MA_SND_PCM_FORMAT_MU_LAW    SND_PCM_FORMAT_MU_LAW
28159 | #define MA_SND_PCM_FORMAT_A_LAW     SND_PCM_FORMAT_A_LAW
28160 | #define MA_SND_PCM_FORMAT_S24_3LE   SND_PCM_FORMAT_S24_3LE
28161 | #define MA_SND_PCM_FORMAT_S24_3BE   SND_PCM_FORMAT_S24_3BE
28162 |
28163 | /* ma_snd_pcm_access_t */
28164 | #define MA_SND_PCM_ACCESS_MMAP_INTERLEAVED  SND_PCM_ACCESS_MMAP_INTERLEAVED
28165 | #define MA_SND_PCM_ACCESS_MMAP_NONINTERLEAVED SND_PCM_ACCESS_MMAP_NONINTERLEAVED
28166 | #define MA_SND_PCM_ACCESS_MMAP_COMPLEX      SND_PCM_ACCESS_MMAP_COMPLEX

```

```

28167 | #define MA_SND_PCM_ACCESS_RW_INTERLEAVED      SND_PCM_ACCESS_RW_INTERLEAVED
28168 | #define MA_SND_PCM_ACCESS_RW_NONINTERLEAVED   SND_PCM_ACCESS_RW_NONINTERLEAVED
28169 |
28170 | /* Channel positions. */
28171 | #define MA_SND_CHMAP_UNKNOWN                   SND_CHMAP_UNKNOWN
28172 | #define MA_SND_CHMAP_NA                       SND_CHMAP_NA
28173 | #define MA_SND_CHMAP_MONO                     SND_CHMAP_MONO
28174 | #define MA_SND_CHMAP_FL                       SND_CHMAP_FL
28175 | #define MA_SND_CHMAP_FR                       SND_CHMAP_FR
28176 | #define MA_SND_CHMAP_RL                       SND_CHMAP_RL
28177 | #define MA_SND_CHMAP_RR                       SND_CHMAP_RR
28178 | #define MA_SND_CHMAP_FC                       SND_CHMAP_FC
28179 | #define MA_SND_CHMAP_LFE                      SND_CHMAP_LFE
28180 | #define MA_SND_CHMAP_SL                       SND_CHMAP_SL
28181 | #define MA_SND_CHMAP_SR                       SND_CHMAP_SR
28182 | #define MA_SND_CHMAP_RC                       SND_CHMAP_RC
28183 | #define MA_SND_CHMAP_FLC                      SND_CHMAP_FLC
28184 | #define MA_SND_CHMAP_FRC                      SND_CHMAP_FRC
28185 | #define MA_SND_CHMAP_RLC                      SND_CHMAP_RLC
28186 | #define MA_SND_CHMAP_RRC                      SND_CHMAP_RRC
28187 | #define MA_SND_CHMAP_FLW                      SND_CHMAP_FLW
28188 | #define MA_SND_CHMAP_FRW                      SND_CHMAP_FRW
28189 | #define MA_SND_CHMAP_FLH                      SND_CHMAP_FLH
28190 | #define MA_SND_CHMAP_FCH                      SND_CHMAP_FCH
28191 | #define MA_SND_CHMAP_FRH                      SND_CHMAP_FRH
28192 | #define MA_SND_CHMAP_TC                       SND_CHMAP_TC
28193 | #define MA_SND_CHMAP_TFL                      SND_CHMAP_TFL
28194 | #define MA_SND_CHMAP_TFR                      SND_CHMAP_TFR
28195 | #define MA_SND_CHMAP_TFC                      SND_CHMAP_TFC
28196 | #define MA_SND_CHMAP_TRL                      SND_CHMAP_TRL
28197 | #define MA_SND_CHMAP_TRR                      SND_CHMAP_TRR
28198 | #define MA_SND_CHMAP_TRC                      SND_CHMAP_TRC
28199 | #define MA_SND_CHMAP_TFLC                     SND_CHMAP_TFLC
28200 | #define MA_SND_CHMAP_TFRC                     SND_CHMAP_TFRC
28201 | #define MA_SND_CHMAP_TSL                      SND_CHMAP_TSL
28202 | #define MA_SND_CHMAP_TSR                      SND_CHMAP_TSR
28203 | #define MA_SND_CHMAP_LLFE                     SND_CHMAP_LLFE
28204 | #define MA_SND_CHMAP_RLFE                     SND_CHMAP_RLFE
28205 | #define MA_SND_CHMAP_BC                       SND_CHMAP_BC
28206 | #define MA_SND_CHMAP_BLC                      SND_CHMAP_BLC
28207 | #define MA_SND_CHMAP_BRC                      SND_CHMAP_BRC
28208 |
28209 | /* Open mode flags. */
28210 | #define MA_SND_PCM_NO_AUTO_RESAMPLE            SND_PCM_NO_AUTO_RESAMPLE
28211 | #define MA_SND_PCM_NO_AUTO_CHANNELS            SND_PCM_NO_AUTO_CHANNELS
28212 | #define MA_SND_PCM_NO_AUTO_FORMAT              SND_PCM_NO_AUTO_FORMAT
28213 | #else
28214 | #include <errno.h> /* For EPIPE, etc. */
28215 | typedef unsigned long
28216 | typedef long
28217 | typedef int
28218 | typedef int
28219 | typedef int
28220 | typedef int
28221 | typedef struct ma_snd_pcm_t
28222 | typedef struct ma_snd_pcm_hw_params_t
28223 | typedef struct ma_snd_pcm_sw_params_t
28224 | typedef struct ma_snd_pcm_format_mask_t
28225 | typedef struct ma_snd_pcm_info_t
28226 | typedef struct
28227 | {
28228 |     void* addr;
28229 |     unsigned int first;
28230 |     unsigned int step;
28231 | } ma_snd_pcm_channel_area_t;
28232 | typedef struct
28233 | {
28234 |     unsigned int channels;
28235 |     unsigned int pos[1];
28236 | } ma_snd_pcm_chmap_t;
28237 |
28238 | /* snd_pcm_state_t */
28239 | #define MA_SND_PCM_STATE_OPEN                   0
28240 | #define MA_SND_PCM_STATE_SETUP                   1
28241 | #define MA_SND_PCM_STATE_PREPARED                 2
28242 | #define MA_SND_PCM_STATE_RUNNING                  3
28243 | #define MA_SND_PCM_STATE_XRUN                     4
28244 | #define MA_SND_PCM_STATE_DRAINING                 5
28245 | #define MA_SND_PCM_STATE_PAUSED                   6

```

```

28246 | #define MA_SND_PCM_STATE_SUSPENDED          7
28247 | #define MA_SND_PCM_STATE_DISCONNECTED      8
28248 |
28249 | /* snd_pcm_stream_t */
28250 | #define MA_SND_PCM_STREAM_PLAYBACK        0
28251 | #define MA_SND_PCM_STREAM_CAPTURE         1
28252 |
28253 | /* snd_pcm_format_t */
28254 | #define MA_SND_PCM_FORMAT_UNKNOWN         -1
28255 | #define MA_SND_PCM_FORMAT_U8              1
28256 | #define MA_SND_PCM_FORMAT_S16_LE         2
28257 | #define MA_SND_PCM_FORMAT_S16_BE         3
28258 | #define MA_SND_PCM_FORMAT_S24_LE         6
28259 | #define MA_SND_PCM_FORMAT_S24_BE         7
28260 | #define MA_SND_PCM_FORMAT_S32_LE        10
28261 | #define MA_SND_PCM_FORMAT_S32_BE        11
28262 | #define MA_SND_PCM_FORMAT_FLOAT_LE       14
28263 | #define MA_SND_PCM_FORMAT_FLOAT_BE       15
28264 | #define MA_SND_PCM_FORMAT_FLOAT64_LE     16
28265 | #define MA_SND_PCM_FORMAT_FLOAT64_BE     17
28266 | #define MA_SND_PCM_FORMAT_MU_LAW        20
28267 | #define MA_SND_PCM_FORMAT_A_LAW         21
28268 | #define MA_SND_PCM_FORMAT_S24_3LE       32
28269 | #define MA_SND_PCM_FORMAT_S24_3BE       33
28270 |
28271 | /* snd_pcm_access_t */
28272 | #define MA_SND_PCM_ACCESS_MMAP_INTERLEAVED 0
28273 | #define MA_SND_PCM_ACCESS_MMAP_NONINTERLEAVED 1
28274 | #define MA_SND_PCM_ACCESS_MMAP_COMPLEX    2
28275 | #define MA_SND_PCM_ACCESS_RW_INTERLEAVED 3
28276 | #define MA_SND_PCM_ACCESS_RW_NONINTERLEAVED 4
28277 |
28278 | /* Channel positions. */
28279 | #define MA_SND_CHMAP_UNKNOWN              0
28280 | #define MA_SND_CHMAP_NA                  1
28281 | #define MA_SND_CHMAP_MONO                2
28282 | #define MA_SND_CHMAP_FL                  3
28283 | #define MA_SND_CHMAP_FR                  4
28284 | #define MA_SND_CHMAP_RL                  5
28285 | #define MA_SND_CHMAP_RR                  6
28286 | #define MA_SND_CHMAP_FC                  7
28287 | #define MA_SND_CHMAP_LFE                 8
28288 | #define MA_SND_CHMAP_SL                  9
28289 | #define MA_SND_CHMAP_SR                 10
28290 | #define MA_SND_CHMAP_RC                 11
28291 | #define MA_SND_CHMAP_FLC                 12
28292 | #define MA_SND_CHMAP_FRC                 13
28293 | #define MA_SND_CHMAP_RLC                 14
28294 | #define MA_SND_CHMAP_RRC                 15
28295 | #define MA_SND_CHMAP_FLW                 16
28296 | #define MA_SND_CHMAP_FRW                 17
28297 | #define MA_SND_CHMAP_FLH                 18
28298 | #define MA_SND_CHMAP_FCH                 19
28299 | #define MA_SND_CHMAP_FRH                 20
28300 | #define MA_SND_CHMAP_TC                  21
28301 | #define MA_SND_CHMAP_TFL                 22
28302 | #define MA_SND_CHMAP_TFR                 23
28303 | #define MA_SND_CHMAP_TFC                 24
28304 | #define MA_SND_CHMAP_TRL                 25
28305 | #define MA_SND_CHMAP_TRR                 26
28306 | #define MA_SND_CHMAP_TRC                 27
28307 | #define MA_SND_CHMAP_TFLC                 28
28308 | #define MA_SND_CHMAP_TFRC                 29
28309 | #define MA_SND_CHMAP_TSL                 30
28310 | #define MA_SND_CHMAP_TSR                 31
28311 | #define MA_SND_CHMAP_LLFE                 32
28312 | #define MA_SND_CHMAP_RLFE                 33
28313 | #define MA_SND_CHMAP_BC                  34
28314 | #define MA_SND_CHMAP_BLC                 35
28315 | #define MA_SND_CHMAP_BRC                 36
28316 |
28317 | /* Open mode flags. */
28318 | #define MA_SND_PCM_NO_AUTO_RESAMPLE       0x00010000
28319 | #define MA_SND_PCM_NO_AUTO_CHANNELS       0x00020000
28320 | #define MA_SND_PCM_NO_AUTO_FORMAT        0x00040000
28321 | #endif
28322 |
28323 | typedef int (* ma_snd_pcm_open_proc) (ma_snd_pcm_t **pcm,
> const char *name, ma_snd_pcm_stream_t stream, int mode);

```



```

28324 | typedef int          (* ma_snd_pcm_close_proc)          (ma_snd_pcm_t *pcm);
28325 | typedef size_t       (* ma_snd_pcm_hw_params_sizeof_proc) (void);
28326 | typedef int          (* ma_snd_pcm_hw_params_any_proc)   (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params);
28327 | typedef int          (* ma_snd_pcm_hw_params_set_format_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_format_t val);
28328 | typedef int          (* ma_snd_pcm_hw_params_set_format_first_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_format_t *format);
28329 | typedef void         (* ma_snd_pcm_hw_params_get_format_mask_proc)
> (ma_snd_pcm_hw_params_t *params, ma_snd_pcm_format_mask_t *mask);
28330 | typedef int          (* ma_snd_pcm_hw_params_set_channels_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int val);
28331 | typedef int          (* ma_snd_pcm_hw_params_set_channels_near_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int *val);
28332 | typedef int          (* ma_snd_pcm_hw_params_set_channels_minmax_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int *minimum, unsigned int *maximum);
28333 | typedef int          (* ma_snd_pcm_hw_params_set_rate_resample_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int val);
28334 | typedef int          (* ma_snd_pcm_hw_params_set_rate_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int val, int *dir);
28335 | typedef int          (* ma_snd_pcm_hw_params_set_rate_near_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int *val, int *dir);
28336 | typedef int          (* ma_snd_pcm_hw_params_set_rate_minmax_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int *min, int *mindir, unsigned int *max, int *maxdir);
28337 | typedef int          (* ma_snd_pcm_hw_params_set_buffer_size_near_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_uframes_t *val);
28338 | typedef int          (* ma_snd_pcm_hw_params_set_periods_near_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int *val, int *dir);
28339 | typedef int          (* ma_snd_pcm_hw_params_set_access_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_access_t *access);
28340 | typedef int          (* ma_snd_pcm_hw_params_get_format_proc) (const
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_format_t *format);
28341 | typedef int          (* ma_snd_pcm_hw_params_get_channels_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *val);
28342 | typedef int          (* ma_snd_pcm_hw_params_get_channels_min_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *val);
28343 | typedef int          (* ma_snd_pcm_hw_params_get_channels_max_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *val);
28344 | typedef int          (* ma_snd_pcm_hw_params_get_rate_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *rate, int *dir);
28345 | typedef int          (* ma_snd_pcm_hw_params_get_rate_min_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *rate, int *dir);
28346 | typedef int          (* ma_snd_pcm_hw_params_get_rate_max_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *rate, int *dir);
28347 | typedef int          (* ma_snd_pcm_hw_params_get_buffer_size_proc) (const
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_uframes_t *val);
28348 | typedef int          (* ma_snd_pcm_hw_params_get_periods_proc) (const
> ma_snd_pcm_hw_params_t *params, unsigned int *val, int *dir);
28349 | typedef int          (* ma_snd_pcm_hw_params_get_access_proc) (const
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_access_t *access);
28350 | typedef int          (* ma_snd_pcm_hw_params_test_format_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, ma_snd_pcm_format_t val);
28351 | typedef int          (* ma_snd_pcm_hw_params_test_channels_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int val);
28352 | typedef int          (* ma_snd_pcm_hw_params_test_rate_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params, unsigned int val, int *dir);
28353 | typedef int          (* ma_snd_pcm_hw_params_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_hw_params_t *params);
28354 | typedef size_t       (* ma_snd_pcm_sw_params_sizeof_proc) (void);
28355 | typedef int          (* ma_snd_pcm_sw_params_current_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_sw_params_t *params);
28356 | typedef int          (* ma_snd_pcm_sw_params_get_boundary_proc) (const
> ma_snd_pcm_sw_params_t *params, ma_snd_pcm_uframes_t *val);
28357 | typedef int          (* ma_snd_pcm_sw_params_set_avail_min_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_sw_params_t *params, ma_snd_pcm_uframes_t val);
28358 | typedef int          (* ma_snd_pcm_sw_params_set_start_threshold_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_sw_params_t *params, ma_snd_pcm_uframes_t val);
28359 | typedef int          (* ma_snd_pcm_sw_params_set_stop_threshold_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_sw_params_t *params, ma_snd_pcm_uframes_t val);
28360 | typedef int          (* ma_snd_pcm_sw_params_proc) (ma_snd_pcm_t *pcm,
> ma_snd_pcm_sw_params_t *params);
28361 | typedef size_t       (* ma_snd_pcm_format_mask_sizeof_proc) (void);
28362 | typedef int          (* ma_snd_pcm_format_mask_test_proc) (const
> ma_snd_pcm_format_mask_t *mask, ma_snd_pcm_format_t val);
28363 | typedef ma_snd_pcm_chmap_t * (* ma_snd_pcm_get_chmap_proc) (ma_snd_pcm_t *pcm);
28364 | typedef ma_snd_pcm_state_t (* ma_snd_pcm_state_proc) (ma_snd_pcm_t *pcm);
28365 | typedef int          (* ma_snd_pcm_prepare_proc) (ma_snd_pcm_t *pcm);
28366 | typedef int          (* ma_snd_pcm_start_proc) (ma_snd_pcm_t *pcm);
28367 | typedef int          (* ma_snd_pcm_drop_proc) (ma_snd_pcm_t *pcm);

```



```

28368 | typedef int                (* ma_snd_pcm_drain_proc)                (ma_snd_pcm_t *pcm);
28369 | typedef int                (* ma_snd_pcm_reset_proc)               (ma_snd_pcm_t *pcm);
28370 | typedef int                (* ma_snd_device_name_hint_proc)        (int card, const char
> *iface, void ***hints);
28371 | typedef char *             (* ma_snd_device_name_get_hint_proc)    (const void *hint,
> const char *id);
28372 | typedef int                (* ma_snd_card_get_index_proc)          (const char *name);
28373 | typedef int                (* ma_snd_device_name_free_hint_proc)   (void **hints);
28374 | typedef int                (* ma_snd_pcm_mmap_begin_proc)          (ma_snd_pcm_t *pcm,
> const ma_snd_pcm_channel_area_t **areas, ma_snd_pcm_uframes_t *offset, ma_snd_pcm_uframes_t
> *frames);
28375 | typedef ma_snd_pcm_sframes_t (* ma_snd_pcm_mmap_commit_proc)      (ma_snd_pcm_t *pcm,
> ma_snd_pcm_uframes_t offset, ma_snd_pcm_uframes_t frames);
28376 | typedef int                (* ma_snd_pcm_recover_proc)             (ma_snd_pcm_t *pcm,
> int err, int silent);
28377 | typedef ma_snd_pcm_sframes_t (* ma_snd_pcm_readi_proc)            (ma_snd_pcm_t *pcm,
> void *buffer, ma_snd_pcm_uframes_t size);
28378 | typedef ma_snd_pcm_sframes_t (* ma_snd_pcm_writei_proc)           (ma_snd_pcm_t *pcm,
> const void *buffer, ma_snd_pcm_uframes_t size);
28379 | typedef ma_snd_pcm_sframes_t (* ma_snd_pcm_avail_proc)             (ma_snd_pcm_t *pcm);
28380 | typedef ma_snd_pcm_sframes_t (* ma_snd_pcm_avail_update_proc)      (ma_snd_pcm_t *pcm);
28381 | typedef int                (* ma_snd_pcm_wait_proc)               (ma_snd_pcm_t *pcm,
> int timeout);
28382 | typedef int                (* ma_snd_pcm_nonblock_proc)            (ma_snd_pcm_t *pcm,
> int nonblock);
28383 | typedef int                (* ma_snd_pcm_info_proc)                (ma_snd_pcm_t *pcm,
> ma_snd_pcm_info_t* info);
28384 | typedef size_t             (* ma_snd_pcm_info_sizeof_proc)         (void);
28385 | typedef const char*        (* ma_snd_pcm_info_get_name_proc)       (const
> ma_snd_pcm_info_t* info);
28386 | typedef int                (* ma_snd_pcm_poll_descriptors_proc)    (ma_snd_pcm_t *pcm,
> struct pollfd *pfd, unsigned int space);
28387 | typedef int                (* ma_snd_pcm_poll_descriptors_count_proc) (ma_snd_pcm_t *pcm);
28388 | typedef int                (* ma_snd_pcm_poll_descriptors_revents_proc) (ma_snd_pcm_t *pcm,
> struct pollfd *pfd, unsigned int nfds, unsigned short *revents);
28389 | typedef int                (* ma_snd_config_update_free_global_proc) (void);
28390 |
28391 | /* This array specifies each of the common devices that can be used for both playback and capture.
> */
28392 | static const char* g_maCommonDeviceNamesALSA[] = {
28393 |     "default",
28394 |     "null",
28395 |     "pulse",
28396 |     "jack"
28397 | };
28398 |
28399 | /* This array allows us to blacklist specific playback devices. */
28400 | static const char* g_maBlacklistedPlaybackDeviceNamesALSA[] = {
28401 |     ""
28402 | };
28403 |
28404 | /* This array allows us to blacklist specific capture devices. */
28405 | static const char* g_maBlacklistedCaptureDeviceNamesALSA[] = {
28406 |     ""
28407 | };
28408 |
28409 |
28410 | static ma_snd_pcm_format_t ma_convert_ma_format_to_alsa_format(ma_format format)
28411 | {
28412 |     ma_snd_pcm_format_t ALSAFormats[] = {
28413 |         MA_SND_PCM_FORMAT_UNKNOWN, /* ma_format_unknown */
28414 |         MA_SND_PCM_FORMAT_U8,      /* ma_format_u8 */
28415 |         MA_SND_PCM_FORMAT_S16_LE,  /* ma_format_s16 */
28416 |         MA_SND_PCM_FORMAT_S24_3LE, /* ma_format_s24 */
28417 |         MA_SND_PCM_FORMAT_S32_LE,  /* ma_format_s32 */
28418 |         MA_SND_PCM_FORMAT_FLOAT_LE /* ma_format_f32 */
28419 |     };
28420 |
28421 |     if (ma_is_big_endian()) {
28422 |         ALSAFormats[0] = MA_SND_PCM_FORMAT_UNKNOWN;
28423 |         ALSAFormats[1] = MA_SND_PCM_FORMAT_U8;
28424 |         ALSAFormats[2] = MA_SND_PCM_FORMAT_S16_BE;
28425 |         ALSAFormats[3] = MA_SND_PCM_FORMAT_S24_3BE;
28426 |         ALSAFormats[4] = MA_SND_PCM_FORMAT_S32_BE;
28427 |         ALSAFormats[5] = MA_SND_PCM_FORMAT_FLOAT_BE;
28428 |     }
28429 |
28430 |     return ALSAFormats[format];
28431 | }

```

```

28432 |
28433 | static ma_format ma_format_from_alsa(ma_snd_pcm_format_t formatALSA)
28434 | {
28435 |     if (ma_is_little_endian()) {
28436 |         switch (formatALSA) {
28437 |             case MA_SND_PCM_FORMAT_S16_LE: return ma_format_s16;
28438 |             case MA_SND_PCM_FORMAT_S24_3LE: return ma_format_s24;
28439 |             case MA_SND_PCM_FORMAT_S32_LE: return ma_format_s32;
28440 |             case MA_SND_PCM_FORMAT_FLOAT_LE: return ma_format_f32;
28441 |             default: break;
28442 |         }
28443 |     } else {
28444 |         switch (formatALSA) {
28445 |             case MA_SND_PCM_FORMAT_S16_BE: return ma_format_s16;
28446 |             case MA_SND_PCM_FORMAT_S24_3BE: return ma_format_s24;
28447 |             case MA_SND_PCM_FORMAT_S32_BE: return ma_format_s32;
28448 |             case MA_SND_PCM_FORMAT_FLOAT_BE: return ma_format_f32;
28449 |             default: break;
28450 |         }
28451 |     }
28452 |
28453 |     /* Endian agnostic. */
28454 |     switch (formatALSA) {
28455 |         case MA_SND_PCM_FORMAT_U8: return ma_format_u8;
28456 |         default: return ma_format_unknown;
28457 |     }
28458 | }
28459 |
28460 | static ma_channel ma_convert_alsa_channel_position_to_ma_channel(unsigned int alsaChannelPos)
28461 | {
28462 |     switch (alsaChannelPos)
28463 |     {
28464 |         case MA_SND_CHMAP_MONO: return MA_CHANNEL_MONO;
28465 |         case MA_SND_CHMAP_FL: return MA_CHANNEL_FRONT_LEFT;
28466 |         case MA_SND_CHMAP_FR: return MA_CHANNEL_FRONT_RIGHT;
28467 |         case MA_SND_CHMAP_RL: return MA_CHANNEL_BACK_LEFT;
28468 |         case MA_SND_CHMAP_RR: return MA_CHANNEL_BACK_RIGHT;
28469 |         case MA_SND_CHMAP_FC: return MA_CHANNEL_FRONT_CENTER;
28470 |         case MA_SND_CHMAP_LFE: return MA_CHANNEL_LFE;
28471 |         case MA_SND_CHMAP_SL: return MA_CHANNEL_SIDE_LEFT;
28472 |         case MA_SND_CHMAP_SR: return MA_CHANNEL_SIDE_RIGHT;
28473 |         case MA_SND_CHMAP_RC: return MA_CHANNEL_BACK_CENTER;
28474 |         case MA_SND_CHMAP_FLC: return MA_CHANNEL_FRONT_LEFT_CENTER;
28475 |         case MA_SND_CHMAP_FRC: return MA_CHANNEL_FRONT_RIGHT_CENTER;
28476 |         case MA_SND_CHMAP_RLC: return 0;
28477 |         case MA_SND_CHMAP_RRC: return 0;
28478 |         case MA_SND_CHMAP_FLW: return 0;
28479 |         case MA_SND_CHMAP_FRW: return 0;
28480 |         case MA_SND_CHMAP_FLH: return 0;
28481 |         case MA_SND_CHMAP_FCH: return 0;
28482 |         case MA_SND_CHMAP_FRH: return 0;
28483 |         case MA_SND_CHMAP_TC: return MA_CHANNEL_TOP_CENTER;
28484 |         case MA_SND_CHMAP_TFL: return MA_CHANNEL_TOP_FRONT_LEFT;
28485 |         case MA_SND_CHMAP_TFR: return MA_CHANNEL_TOP_FRONT_RIGHT;
28486 |         case MA_SND_CHMAP_TFC: return MA_CHANNEL_TOP_FRONT_CENTER;
28487 |         case MA_SND_CHMAP_TRL: return MA_CHANNEL_TOP_BACK_LEFT;
28488 |         case MA_SND_CHMAP_TRR: return MA_CHANNEL_TOP_BACK_RIGHT;
28489 |         case MA_SND_CHMAP_TRC: return MA_CHANNEL_TOP_BACK_CENTER;
28490 |         default: break;
28491 |     }
28492 |
28493 |     return 0;
28494 | }
28495 |
28496 | static ma_bool32 ma_is_common_device_name__alsa(const char* name)
28497 | {
28498 |     size_t iName;
28499 |     for (iName = 0; iName < ma_countof(g_maCommonDeviceNamesALSA); ++iName) {
28500 |         if (ma_strcmp(name, g_maCommonDeviceNamesALSA[iName]) == 0) {
28501 |             return MA_TRUE;
28502 |         }
28503 |     }
28504 |
28505 |     return MA_FALSE;
28506 | }
28507 |
28508 |
28509 | static ma_bool32 ma_is_playback_device_blacklisted__alsa(const char* name)
28510 | {

```

```

28511 |     size_t iName;
28512 |     for (iName = 0; iName < ma_countof(g_maBlacklistedPlaybackDeviceNamesALSA); ++iName) {
28513 |         if (ma_strcmp(name, g_maBlacklistedPlaybackDeviceNamesALSA[iName]) == 0) {
28514 |             return MA_TRUE;
28515 |         }
28516 |     }
28517 |
28518 |     return MA_FALSE;
28519 | }
28520 |
28521 | static ma_bool32 ma_is_capture_device_blacklisted__alsa(const char* name)
28522 | {
28523 |     size_t iName;
28524 |     for (iName = 0; iName < ma_countof(g_maBlacklistedCaptureDeviceNamesALSA); ++iName) {
28525 |         if (ma_strcmp(name, g_maBlacklistedCaptureDeviceNamesALSA[iName]) == 0) {
28526 |             return MA_TRUE;
28527 |         }
28528 |     }
28529 |
28530 |     return MA_FALSE;
28531 | }
28532 |
28533 | static ma_bool32 ma_is_device_blacklisted__alsa(ma_device_type deviceType, const char* name)
28534 | {
28535 |     if (deviceType == ma_device_type_playback) {
28536 |         return ma_is_playback_device_blacklisted__alsa(name);
28537 |     } else {
28538 |         return ma_is_capture_device_blacklisted__alsa(name);
28539 |     }
28540 | }
28541 |
28542 |
28543 | static const char* ma_find_char(const char* str, char c, int* index)
28544 | {
28545 |     int i = 0;
28546 |     for (;;) {
28547 |         if (str[i] == '\0') {
28548 |             if (index) *index = -1;
28549 |             return NULL;
28550 |         }
28551 |
28552 |         if (str[i] == c) {
28553 |             if (index) *index = i;
28554 |             return str + i;
28555 |         }
28556 |
28557 |         i += 1;
28558 |     }
28559 |
28560 |     /* Should never get here, but treat it as though the character was not found to make me feel
> better inside. */
28561 |     if (index) *index = -1;
28562 |     return NULL;
28563 | }
28564 |
28565 | static ma_bool32 ma_is_device_name_in_hw_format__alsa(const char* hwid)
28566 | {
28567 |     /* This function is just checking whether or not hwid is in "hw:%d,%d" format. */
28568 |
28569 |     int commaPos;
28570 |     const char* dev;
28571 |     int i;
28572 |
28573 |     if (hwid == NULL) {
28574 |         return MA_FALSE;
28575 |     }
28576 |
28577 |     if (hwid[0] != 'h' || hwid[1] != 'w' || hwid[2] != ':') {
28578 |         return MA_FALSE;
28579 |     }
28580 |
28581 |     hwid += 3;
28582 |
28583 |     dev = ma_find_char(hwid, ',', &commaPos);
28584 |     if (dev == NULL) {
28585 |         return MA_FALSE;
28586 |     } else {
28587 |         dev += 1; /* Skip past the ",". */
28588 |     }

```

```

28589 |
28590 |     /* Check if the part between the ":" and the "," contains only numbers. If not, return false. */
28591 |     for (i = 0; i < commaPos; ++i) {
28592 |         if (hwid[i] < '0' || hwid[i] > '9') {
28593 |             return MA_FALSE;
28594 |         }
28595 |     }
28596 |
28597 |     /* Check if everything after the "," is numeric. If not, return false. */
28598 |     i = 0;
28599 |     while (dev[i] != '\0') {
28600 |         if (dev[i] < '0' || dev[i] > '9') {
28601 |             return MA_FALSE;
28602 |         }
28603 |         i += 1;
28604 |     }
28605 |
28606 |     return MA_TRUE;
28607 | }
28608 |
28609 | static int ma_convert_device_name_to_hw_format__alsa(ma_context* pContext, char* dst, size_t
> dstSize, const char* src) /* Returns 0 on success, non-0 on error. */
28610 | {
28611 |     /* src should look something like this: "hw:CARD=I82801AAICH,DEV=0" */
28612 |
28613 |     int colonPos;
28614 |     int commaPos;
28615 |     char card[256];
28616 |     const char* dev;
28617 |     int cardIndex;
28618 |
28619 |     if (dst == NULL) {
28620 |         return -1;
28621 |     }
28622 |     if (dstSize < 7) {
28623 |         return -1; /* Absolute minimum size of the output buffer is 7 bytes. */
28624 |     }
28625 |
28626 |     *dst = '\0'; /* Safety. */
28627 |     if (src == NULL) {
28628 |         return -1;
28629 |     }
28630 |
28631 |     /* If the input name is already in "hw:%d,%d" format, just return that verbatim. */
28632 |     if (ma_is_device_name_in_hw_format__alsa(src)) {
28633 |         return ma_strcpy_s(dst, dstSize, src);
28634 |     }
28635 |
28636 |     src = ma_find_char(src, ':', &colonPos);
28637 |     if (src == NULL) {
28638 |         return -1; /* Couldn't find a colon */
28639 |     }
28640 |
28641 |     dev = ma_find_char(src, ',', &commaPos);
28642 |     if (dev == NULL) {
28643 |         dev = "0";
28644 |         ma_strncpy_s(card, sizeof(card), src+6, (size_t)-1); /* +6 = ":CARD=" */
28645 |     } else {
28646 |         dev = dev + 5; /* +5 = ",DEV=" */
28647 |         ma_strncpy_s(card, sizeof(card), src+6, commaPos-6); /* +6 = ":CARD=" */
28648 |     }
28649 |
28650 |     cardIndex = ((ma_snd_card_get_index_proc)pContext->alsa.snd_card_get_index)(card);
28651 |     if (cardIndex < 0) {
28652 |         return -2; /* Failed to retrieve the card index. */
28653 |     }
28654 |
28655 |
28656 |     /* Construction. */
28657 |     dst[0] = 'h'; dst[1] = 'w'; dst[2] = ':';
28658 |     if (ma_itoa_s(cardIndex, dst+3, dstSize-3, 10) != 0) {
28659 |         return -3;
28660 |     }
28661 |     if (ma_strcat_s(dst, dstSize, ",") != 0) {
28662 |         return -3;
28663 |     }
28664 |     if (ma_strcat_s(dst, dstSize, dev) != 0) {
28665 |         return -3;
28666 |     }

```

```

28667 |
28668 |     return 0;
28669 | }
28670 |
28671 | static ma_bool32 ma_does_id_exist_in_list__alsa(ma_device_id* pUniqueIDs, ma_uint32 count, const
> char* pHWID)
28672 | {
28673 |     ma_uint32 i;
28674 |
28675 |     MA_ASSERT(pHWID != NULL);
28676 |
28677 |     for (i = 0; i < count; ++i) {
28678 |         if (ma_strcmp(pUniqueIDs[i].alsa, pHWID) == 0) {
28679 |             return MA_TRUE;
28680 |         }
28681 |     }
28682 |
28683 |     return MA_FALSE;
28684 | }
28685 |
28686 |
28687 | static ma_result ma_context_open_pcm__alsa(ma_context* pContext, ma_share_mode shareMode,
> ma_device_type deviceType, const ma_device_id* pDeviceID, int openMode, ma_snd_pcm_t** ppPCM)
28688 | {
28689 |     ma_snd_pcm_t* pPCM;
28690 |     ma_snd_pcm_stream_t stream;
28691 |
28692 |     MA_ASSERT(pContext != NULL);
28693 |     MA_ASSERT(ppPCM != NULL);
28694 |
28695 |     *ppPCM = NULL;
28696 |     pPCM = NULL;
28697 |
28698 |     stream = (deviceType == ma_device_type_playback) ? MA_SND_PCM_STREAM_PLAYBACK :
> MA_SND_PCM_STREAM_CAPTURE;
28699 |
28700 |     if (pDeviceID == NULL) {
28701 |         ma_bool32 isDeviceOpen;
28702 |         size_t i;
28703 |
28704 |         /*
28705 |         We're opening the default device. I don't know if trying anything other than "default" is
> necessary, but it makes
28706 |         me feel better to try as hard as we can get to get _something_ working.
28707 |         */
28708 |         const char* defaultDeviceNames[] = {
28709 |             "default",
28710 |             NULL,
28711 |             NULL,
28712 |             NULL,
28713 |             NULL,
28714 |             NULL,
28715 |             NULL
28716 |         };
28717 |
28718 |         if (shareMode == ma_share_mode_exclusive) {
28719 |             defaultDeviceNames[1] = "hw";
28720 |             defaultDeviceNames[2] = "hw:0";
28721 |             defaultDeviceNames[3] = "hw:0,0";
28722 |         } else {
28723 |             if (deviceType == ma_device_type_playback) {
28724 |                 defaultDeviceNames[1] = "dmix";
28725 |                 defaultDeviceNames[2] = "dmix:0";
28726 |                 defaultDeviceNames[3] = "dmix:0,0";
28727 |             } else {
28728 |                 defaultDeviceNames[1] = "dsnoop";
28729 |                 defaultDeviceNames[2] = "dsnoop:0";
28730 |                 defaultDeviceNames[3] = "dsnoop:0,0";
28731 |             }
28732 |             defaultDeviceNames[4] = "hw";
28733 |             defaultDeviceNames[5] = "hw:0";
28734 |             defaultDeviceNames[6] = "hw:0,0";
28735 |         }
28736 |
28737 |         isDeviceOpen = MA_FALSE;
28738 |         for (i = 0; i < ma_countof(defaultDeviceNames); ++i) {
28739 |             if (defaultDeviceNames[i] != NULL && defaultDeviceNames[i][0] != '\0') {
28740 |                 if (((ma_snd_pcm_open_proc)pContext->alsa.snd_pcm_open)(&pPCM,
> defaultDeviceNames[i], stream, openMode) == 0) {

```

```

28741 |             isDeviceOpen = MA_TRUE;
28742 |             break;
28743 |         }
28744 |     }
28745 | }
28746 |
28747 |     if (!isDeviceOpen) {
28748 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[ALSA] snd_pcm_open()
> failed when trying to open an appropriate default device.");
28749 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
28750 |     }
28751 | } else {
28752 |     /*
28753 |     We're trying to open a specific device. There's a few things to consider here:
28754 |
28755 |     miniaudio recognizes a special format of device id that excludes the "hw", "dmix", etc.
> prefix. It looks like this: ":0,0", ":0,1", etc. When
28756 |     an ID of this format is specified, it indicates to miniaudio that it can try different
> combinations of plugins ("hw", "dmix", etc.) until it
28757 |     finds an appropriate one that works. This comes in very handy when trying to open a device
> in shared mode ("dmix"), vs exclusive mode ("hw").
28758 |     */
28759 |
28760 |     /* May end up needing to make small adjustments to the ID, so make a copy. */
28761 |     ma_device_id deviceID = *pDeviceID;
28762 |     int resultALSA = -ENODEV;
28763 |
28764 |     if (deviceID.alsa[0] != ':') {
28765 |         /* The ID is not in ":0,0" format. Use the ID exactly as-is. */
28766 |         resultALSA = ((ma_snd_pcm_open_proc)pContext->alsa.snd_pcm_open)(&pPCM, deviceID.alsa,
> stream, openMode);
28767 |     } else {
28768 |         char hwid[256];
28769 |
28770 |         /* The ID is in ":0,0" format. Try different plugins depending on the shared mode. */
28771 |         if (deviceID.alsa[1] == '\0') {
28772 |             deviceID.alsa[0] = '\0'; /* An ID of ":" should be converted to "". */
28773 |         }
28774 |
28775 |         if (shareMode == ma_share_mode_shared) {
28776 |             if (deviceType == ma_device_type_playback) {
28777 |                 ma_strcpy_s(hwid, sizeof(hwid), "dmix");
28778 |             } else {
28779 |                 ma_strcpy_s(hwid, sizeof(hwid), "dsnoop");
28780 |             }
28781 |
28782 |             if (ma_strcat_s(hwid, sizeof(hwid), deviceID.alsa) == 0) {
28783 |                 resultALSA = ((ma_snd_pcm_open_proc)pContext->alsa.snd_pcm_open)(&pPCM, hwid,
> stream, openMode);
28784 |             }
28785 |         }
28786 |
28787 |         /* If at this point we still don't have an open device it means we're either
> preferencing exclusive mode or opening with "dmix"/"dsnoop" failed. */
28788 |         if (resultALSA != 0) {
28789 |             ma_strcpy_s(hwid, sizeof(hwid), "hw");
28790 |             if (ma_strcat_s(hwid, sizeof(hwid), deviceID.alsa) == 0) {
28791 |                 resultALSA = ((ma_snd_pcm_open_proc)pContext->alsa.snd_pcm_open)(&pPCM, hwid,
> stream, openMode);
28792 |             }
28793 |         }
28794 |     }
28795 |
28796 |     if (resultALSA < 0) {
28797 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[ALSA] snd_pcm_open()
> failed.");
28798 |         return ma_result_from_errno(-resultALSA);
28799 |     }
28800 | }
28801 |
28802 | *ppPCM = pPCM;
28803 | return MA_SUCCESS;
28804 | }
28805 |
28806 |
28807 | static ma_result ma_context_enumerate_devices_alsa(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
28808 | {
28809 |     int resultALSA;

```

```

28810 |     ma_bool32 cbResult = MA_TRUE;
28811 |     char** ppDeviceHints;
28812 |     ma_device_id* pUniqueIDs = NULL;
28813 |     ma_uint32 uniqueIDCount = 0;
28814 |     char** ppNextDeviceHint;
28815 |
28816 |     MA_ASSERT(pContext != NULL);
28817 |     MA_ASSERT(callback != NULL);
28818 |
28819 |     ma_mutex_lock(&pContext->alsa.internalDeviceEnumLock);
28820 |
28821 |     resultALSA = ((ma_snd_device_name_hint_proc)pContext->alsa.snd_device_name_hint)(-1, "pcm",
> (void**)&ppDeviceHints);
28822 |     if (resultALSA < 0) {
28823 |         ma_mutex_unlock(&pContext->alsa.internalDeviceEnumLock);
28824 |         return ma_result_from_errno(-resultALSA);
28825 |     }
28826 |
28827 |     ppNextDeviceHint = ppDeviceHints;
28828 |     while (*ppNextDeviceHint != NULL) {
28829 |         char* NAME =
> ((ma_snd_device_name_get_hint_proc)pContext->alsa.snd_device_name_get_hint)(*ppNextDeviceHint,
> "NAME");
28830 |         char* DESC =
> ((ma_snd_device_name_get_hint_proc)pContext->alsa.snd_device_name_get_hint)(*ppNextDeviceHint,
> "DESC");
28831 |         char* IOID =
> ((ma_snd_device_name_get_hint_proc)pContext->alsa.snd_device_name_get_hint)(*ppNextDeviceHint,
> "IOID");
28832 |         ma_device_type deviceType = ma_device_type_playback;
28833 |         ma_bool32 stopEnumeration = MA_FALSE;
28834 |         char hwid[sizeof(pUniqueIDs->alsa)];
28835 |         ma_device_info deviceInfo;
28836 |
28837 |         if ((IOID == NULL || ma_strcmp(IOID, "Output") == 0)) {
28838 |             deviceType = ma_device_type_playback;
28839 |         }
28840 |         if ((IOID != NULL && ma_strcmp(IOID, "Input" ) == 0)) {
28841 |             deviceType = ma_device_type_capture;
28842 |         }
28843 |
28844 |         if (NAME != NULL) {
28845 |             if (pContext->alsa.useVerboseDeviceEnumeration) {
28846 |                 /* Verbose mode. Use the name exactly as-is. */
28847 |                 ma_strncpy_s(hwid, sizeof(hwid), NAME, (size_t)-1);
28848 |             } else {
28849 |                 /* Simplified mode. Use ":%d,%d" format. */
28850 |                 if (ma_convert_device_name_to_hw_format__alsa(pContext, hwid, sizeof(hwid), NAME) ==
> 0) {
28851 |                     /*
28852 |                     > At this point, hwid looks like "hw:0,0". In simplified enumeration mode, we
28853 |                     > actually want to strip off the
28854 |                     > plugin name so it looks like ":%d,%d". The reason for this is that this special
28855 |                     > format is detected at device
28856 |                     > initialization time and is used as an indicator to try to use the most
28857 |                     > appropriate plugin depending on the
28858 |                     > device type and sharing mode.
28859 |                     */
28860 |                     char* dst = hwid;
28861 |                     char* src = hwid+2;
28862 |                     while ((*dst++ = *src++));
28863 |                 } else {
28864 |                     /* Conversion to "hw:%d,%d" failed. Just use the name as-is. */
28865 |                     ma_strncpy_s(hwid, sizeof(hwid), NAME, (size_t)-1);
28866 |                 }
28867 |
28868 |                 if (ma_does_id_exist_in_list__alsa(pUniqueIDs, uniqueIDCount, hwid)) {
28869 |                     goto next_device; /* The device has already been enumerated. Move on to the
> next one. */
28870 |                 } else {
28871 |                     /* The device has not yet been enumerated. Make sure it's added to our list so
> that it's not enumerated again. */
28872 |                     size_t newCapacity = sizeof(*pUniqueIDs) * (uniqueIDCount + 1);
28873 |                     ma_device_id* pNewUniqueIDs = (ma_device_id*)ma_realloc(pUniqueIDs, newCapacity,
> &pContext->allocationCallbacks);
28874 |                     if (pNewUniqueIDs == NULL) {
28875 |                         goto next_device; /* Failed to allocate memory. */
28876 |                     }

```



```

28875 |         pUniqueIDs = pNewUniqueIDs;
28876 |         MA_COPY_MEMORY(pUniqueIDs[uniqueIDCount].alsa, hwid, sizeof(hwid));
28877 |         uniqueIDCount += 1;
28878 |     }
28879 | }
28880 | } else {
28881 |     MA_ZERO_MEMORY(hwid, sizeof(hwid));
28882 | }
28883 |
28884 | MA_ZERO_OBJECT(&deviceInfo);
28885 | ma_strncpy_s(deviceInfo.id.alsa, sizeof(deviceInfo.id.alsa), hwid, (size_t)-1);
28886 |
28887 | /*
28888 | There's no good way to determine whether or not a device is the default on Linux. We're just
> going to do something simple and
28889 | just use the name of "default" as the indicator.
28890 | */
28891 | if (ma_strcmp(deviceInfo.id.alsa, "default") == 0) {
28892 |     deviceInfo.isDefault = MA_TRUE;
28893 | }
28894 |
28895 |
28896 | /*
28897 | DESC is the friendly name. We treat this slightly differently depending on whether or not we
> are using verbose
28898 | device enumeration. In verbose mode we want to take the entire description so that the
> end-user can distinguish
28899 | between the subdevices of each card/dev pair. In simplified mode, however, we only want the
> first part of the
28900 | description.
28901 |
28902 | The value in DESC seems to be split into two lines, with the first line being the name of
> the device and the
28903 | second line being a description of the device. I don't like having the description be across
> two lines because
28904 | it makes formatting ugly and annoying. I'm therefore deciding to put it all on a single line
> with the second line
28905 | being put into parentheses. In simplified mode I'm just stripping the second line entirely.
28906 | */
28907 | if (DESC != NULL) {
28908 |     int lfPos;
28909 |     const char* line2 = ma_find_char(DESC, '\n', &lfPos);
28910 |     if (line2 != NULL) {
28911 |         line2 += 1; /* Skip past the new-line character. */
28912 |
28913 |         if (pContext->alsa.useVerboseDeviceEnumeration) {
28914 |             /* Verbose mode. Put the second line in brackets. */
28915 |             ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), DESC, lfPos);
28916 |             ma_strcat_s (deviceInfo.name, sizeof(deviceInfo.name), " (");
28917 |             ma_strcat_s (deviceInfo.name, sizeof(deviceInfo.name), line2);
28918 |             ma_strcat_s (deviceInfo.name, sizeof(deviceInfo.name), ")");
28919 |         } else {
28920 |             /* Simplified mode. Strip the second line entirely. */
28921 |             ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), DESC, lfPos);
28922 |         }
28923 |     } else {
28924 |         /* There's no second line. Just copy the whole description. */
28925 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), DESC, (size_t)-1);
28926 |     }
28927 | }
28928 |
28929 | if (!ma_is_device_blacklisted__alsa(deviceType, NAME)) {
28930 |     cbResult = callback(pContext, deviceType, &deviceInfo, pUserData);
28931 | }
28932 |
28933 | /*
28934 | Some devices are both playback and capture, but they are only enumerated by ALSA once. We
> need to fire the callback
28935 | again for the other device type in this case. We do this for known devices and where the
> IOID hint is NULL, which
28936 | means both Input and Output.
28937 | */
28938 | if (cbResult) {
28939 |     if (ma_is_common_device_name__alsa(NAME) || IOID == NULL) {
28940 |         if (deviceType == ma_device_type_playback) {
28941 |             if (!ma_is_capture_device_blacklisted__alsa(NAME)) {
28942 |                 cbResult = callback(pContext, ma_device_type_capture, &deviceInfo,
> pUserData);
28943 |             }

```

```

28944 |         } else {
28945 |             if (!ma_is_playback_device_blacklisted__alsa(NAME)) {
28946 |                 cbResult = callback(pContext, ma_device_type_playback, &deviceInfo,
> pUserData);
28947 |             }
28948 |         }
28949 |     }
28950 | }
28951 |
28952 |     if (cbResult == MA_FALSE) {
28953 |         stopEnumeration = MA_TRUE;
28954 |     }
28955 |
28956 |     next_device:
28957 |         free(NAME);
28958 |         free(DESC);
28959 |         free(IOID);
28960 |         ppNextDeviceHint += 1;
28961 |
28962 |         /* We need to stop enumeration if the callback returned false. */
28963 |         if (stopEnumeration) {
28964 |             break;
28965 |         }
28966 |     }
28967 |
28968 |     ma_free(pUniqueIDs, &pContext->allocationCallbacks);
28969 |     ((ma_snd_device_name_free_hint_proc)pContext->alsa.snd_device_name_free_hint)((void**)ppDeviceHi
> nts);
28970 |
28971 |     ma_mutex_unlock(&pContext->alsa.internalDeviceEnumLock);
28972 |
28973 |     return MA_SUCCESS;
28974 | }
28975 |
28976 |
28977 | typedef struct
28978 | {
28979 |     ma_device_type deviceType;
28980 |     const ma_device_id* pDeviceID;
28981 |     ma_share_mode shareMode;
28982 |     ma_device_info* pDeviceInfo;
28983 |     ma_bool32 foundDevice;
28984 | } ma_context_get_device_info_enum_callback_data__alsa;
28985 |
28986 | static ma_bool32 ma_context_get_device_info_enum_callback__alsa(ma_context* pContext, ma_device_type
> deviceType, const ma_device_info* pDeviceInfo, void* pUserData)
28987 | {
28988 |     ma_context_get_device_info_enum_callback_data__alsa* pData =
> (ma_context_get_device_info_enum_callback_data__alsa*)pUserData;
28989 |     MA_ASSERT(pData != NULL);
28990 |
28991 |     (void)pContext;
28992 |
28993 |     if (pData->pDeviceID == NULL && ma_strcmp(pDeviceInfo->id.alsa, "default") == 0) {
28994 |         ma_strncpy_s(pData->pDeviceInfo->name, sizeof(pData->pDeviceInfo->name), pDeviceInfo->name,
> (size_t)-1);
28995 |         pData->foundDevice = MA_TRUE;
28996 |     } else {
28997 |         if (pData->deviceType == deviceType && (pData->pDeviceID != NULL &&
> ma_strcmp(pData->pDeviceID->alsa, pDeviceInfo->id.alsa) == 0)) {
28998 |             ma_strncpy_s(pData->pDeviceInfo->name, sizeof(pData->pDeviceInfo->name),
> pDeviceInfo->name, (size_t)-1);
28999 |             pData->foundDevice = MA_TRUE;
29000 |         }
29001 |     }
29002 |
29003 |     /* Keep enumerating until we have found the device. */
29004 |     return !pData->foundDevice;
29005 | }
29006 |
29007 | static void ma_context_test_rate_and_add_native_data_format__alsa(ma_context* pContext,
> ma_snd_pcm_t* pPCM, ma_snd_pcm_hw_params_t* pHWParams, ma_format format, ma_uint32 channels,
> ma_uint32 sampleRate, ma_uint32 flags, ma_device_info* pDeviceInfo)
29008 | {
29009 |     MA_ASSERT(pPCM != NULL);
29010 |     MA_ASSERT(pHWParams != NULL);
29011 |     MA_ASSERT(pDeviceInfo != NULL);
29012 |
29013 |     if (pDeviceInfo->nativeDataFormatCount < ma_countof(pDeviceInfo->nativeDataFormats) &&

```

```

> ((ma_snd_pcm_hw_params_test_rate_proc)pContext->alsa.snd_pcm_hw_params_test_rate)(pPCM, pHWParams,
> sampleRate, 0) == 0) {
29014 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format    = format;
29015 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels  = channels;
29016 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate = sampleRate;
29017 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags     = flags;
29018 |     pDeviceInfo->nativeDataFormatCount += 1;
29019 | }
29020 | }
29021 |
29022 | static void ma_context_iterate_rates_and_add_native_data_format__alsa(ma_context* pContext,
> ma_snd_pcm_t* pPCM, ma_snd_pcm_hw_params_t* pHWParams, ma_format format, ma_uint32 channels,
> ma_uint32 flags, ma_device_info* pDeviceInfo)
29023 | {
29024 |     ma_uint32 iSampleRate;
29025 |     unsigned int minSampleRate;
29026 |     unsigned int maxSampleRate;
29027 |     int sampleRateDir; /* Not used. Just passed into snd_pcm_hw_params_get_rate_min/max(). */
29028 |
29029 |     /* There could be a range. */
29030 |
> ((ma_snd_pcm_hw_params_get_rate_min_proc)pContext->alsa.snd_pcm_hw_params_get_rate_min)(pHWParams,
> &minSampleRate, &sampleRateDir);
29031 |
> ((ma_snd_pcm_hw_params_get_rate_max_proc)pContext->alsa.snd_pcm_hw_params_get_rate_max)(pHWParams,
> &maxSampleRate, &sampleRateDir);
29032 |
29033 |     /* Make sure our sample rates are clamped to sane values. Stupid devices like "pulse" will
> reports rates like "1" which is ridiculous. */
29034 |     minSampleRate = ma_clamp(minSampleRate, (unsigned int)ma_standard_sample_rate_min, (unsigned
> int)ma_standard_sample_rate_max);
29035 |     maxSampleRate = ma_clamp(maxSampleRate, (unsigned int)ma_standard_sample_rate_min, (unsigned
> int)ma_standard_sample_rate_max);
29036 |
29037 |     for (iSampleRate = 0; iSampleRate < ma_countof(g_maStandardSampleRatePriorities); iSampleRate +=
> 1) {
29038 |         ma_uint32 standardSampleRate = g_maStandardSampleRatePriorities[iSampleRate];
29039 |
29040 |         if (standardSampleRate >= minSampleRate && standardSampleRate <= maxSampleRate) {
29041 |             ma_context_test_rate_and_add_native_data_format__alsa(pContext, pPCM, pHWParams, format,
> channels, standardSampleRate, flags, pDeviceInfo);
29042 |         }
29043 |     }
29044 |
29045 |     /* Now make sure our min and max rates are included just in case they aren't in the range of our
> standard rates. */
29046 |     if (!ma_is_standard_sample_rate(minSampleRate)) {
29047 |         ma_context_test_rate_and_add_native_data_format__alsa(pContext, pPCM, pHWParams, format,
> channels, minSampleRate, flags, pDeviceInfo);
29048 |     }
29049 |
29050 |     if (!ma_is_standard_sample_rate(maxSampleRate) && maxSampleRate != minSampleRate) {
29051 |         ma_context_test_rate_and_add_native_data_format__alsa(pContext, pPCM, pHWParams, format,
> channels, maxSampleRate, flags, pDeviceInfo);
29052 |     }
29053 | }
29054 |
29055 | static ma_result ma_context_get_device_info__alsa(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
29056 | {
29057 |     ma_context_get_device_info_enum_callback_data__alsa data;
29058 |     ma_result result;
29059 |     int resultALSA;
29060 |     ma_snd_pcm_t* pPCM;
29061 |     ma_snd_pcm_hw_params_t* pHWParams;
29062 |     ma_uint32 iFormat;
29063 |     ma_uint32 iChannel;
29064 |
29065 |     MA_ASSERT(pContext != NULL);
29066 |
29067 |     /* We just enumerate to find basic information about the device. */
29068 |     data.deviceType = deviceType;
29069 |     data.pDeviceID = pDeviceID;
29070 |     data.pDeviceInfo = pDeviceInfo;
29071 |     data.foundDevice = MA_FALSE;
29072 |     result = ma_context_enumerate_devices__alsa(pContext,
> ma_context_get_device_info_enum_callback__alsa, &data);
29073 |     if (result != MA_SUCCESS) {
29074 |         return result;

```

```

29075 |     }
29076 |
29077 |     if (!data.foundDevice) {
29078 |         return MA_NO_DEVICE;
29079 |     }
29080 |
29081 |     if (ma_strcmp(pDeviceInfo->id.alsa, "default") == 0) {
29082 |         pDeviceInfo->isDefault = MA_TRUE;
29083 |     }
29084 |
29085 |     /* For detailed info we need to open the device. */
29086 |     result = ma_context_open_pcm_alsa(pContext, ma_share_mode_shared, deviceType, pDeviceID, 0,
> &pPCM);
29087 |     if (result != MA_SUCCESS) {
29088 |         return result;
29089 |     }
29090 |
29091 |     /* We need to initialize a HW parameters object in order to know what formats are supported. */
29092 |     pHWParams = (ma_snd_pcm_hw_params_t*)ma_malloc((ma_snd_pcm_hw_params_sizeof_proc)pContext->alsa
> .snd_pcm_hw_params_sizeof(), &pContext->allocationCallbacks);
29093 |     if (pHWParams == NULL) {
29094 |         ((ma_snd_pcm_close_proc)pContext->alsa.snd_pcm_close)(pPCM);
29095 |         return MA_OUT_OF_MEMORY;
29096 |     }
29097 |
29098 |     resultALSA = ((ma_snd_pcm_hw_params_any_proc)pContext->alsa.snd_pcm_hw_params_any)(pPCM,
> pHWParams);
29099 |     if (resultALSA < 0) {
29100 |         ma_free(pHWParams, &pContext->allocationCallbacks);
29101 |         ((ma_snd_pcm_close_proc)pContext->alsa.snd_pcm_close)(pPCM);
29102 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to initialize
> hardware parameters. snd_pcm_hw_params_any() failed.");
29103 |         return ma_result_from_errno(-resultALSA);
29104 |     }
29105 |
29106 |     /*
29107 |     Some ALSA devices can support many permutations of formats, channels and rates. We only support
29108 |     a fixed number of permutations which means we need to employ some strategies to ensure the best
29109 |     combinations are returned. An example is the "pulse" device which can do its own data conversion
29110 |     in software and as a result can support any combination of format, channels and rate.
29111 |
29112 |     We want to ensure that the first data formats are the best. We have a list of favored sample
29113 |     formats and sample rates, so these will be the basis of our iteration.
29114 |     */
29115 |
29116 |     /* Formats. We just iterate over our standard formats and test them, making sure we reset the
> configuration space each iteration. */
29117 |     for (iFormat = 0; iFormat < ma_countof(g_maFormatPriorities); iFormat += 1) {
29118 |         ma_format format = g_maFormatPriorities[iFormat];
29119 |
29120 |         /*
29121 |         For each format we need to make sure we reset the configuration space so we don't return
29122 |         channel counts and rates that aren't compatible with a format.
29123 |         */
29124 |         ((ma_snd_pcm_hw_params_any_proc)pContext->alsa.snd_pcm_hw_params_any)(pPCM, pHWParams);
29125 |
29126 |         /* Test the format first. If this fails it means the format is not supported and we can skip
> it. */
29127 |         if
> (((ma_snd_pcm_hw_params_test_format_proc)pContext->alsa.snd_pcm_hw_params_test_format)(pPCM,
> pHWParams, ma_convert_ma_format_to_alsa_format(format)) == 0) {
29128 |             /* The format is supported. */
29129 |             unsigned int minChannels;
29130 |             unsigned int maxChannels;
29131 |
29132 |             /*
29133 |             The configuration space needs to be restricted to this format so we can get an accurate
29134 |             picture of which sample rates and channel counts are support with this format.
29135 |             */
29136 |
> ((ma_snd_pcm_hw_params_set_format_proc)pContext->alsa.snd_pcm_hw_params_set_format)(pPCM, pHWParams,
> ma_convert_ma_format_to_alsa_format(format));
29137 |
29138 |             /* Now we need to check for supported channels. */
29139 |             ((ma_snd_pcm_hw_params_get_channels_min_proc)pContext->alsa.snd_pcm_hw_params_get_channe
> ls_min)(pHWParams, &minChannels);
29140 |             ((ma_snd_pcm_hw_params_get_channels_max_proc)pContext->alsa.snd_pcm_hw_params_get_channe
> ls_max)(pHWParams, &maxChannels);
29141 |

```

```

29142 |         if (minChannels > MA_MAX_CHANNELS) {
29143 |             continue; /* Too many channels. */
29144 |         }
29145 |         if (maxChannels < MA_MIN_CHANNELS) {
29146 |             continue; /* Not enough channels. */
29147 |         }
29148 |
29149 |         /*
29150 |         Make sure the channel count is clamped. This is mainly intended for the max channels
29151 |         because some devices can report an unbound maximum.
29152 |         */
29153 |         minChannels = ma_clamp(minChannels, MA_MIN_CHANNELS, MA_MAX_CHANNELS);
29154 |         maxChannels = ma_clamp(maxChannels, MA_MIN_CHANNELS, MA_MAX_CHANNELS);
29155 |
29156 |         if (minChannels == MA_MIN_CHANNELS && maxChannels == MA_MAX_CHANNELS) {
29157 |             /* The device supports all channels. Don't iterate over every single one. Instead
> just set the channels to 0 which means all channels are supported. */
29158 |             ma_context_iterate_rates_and_add_native_data_format_alsa(pContext, pPCM, pHWParams,
> format, 0, 0, pDeviceInfo); /* Intentionally setting the channel count to 0 as that means all
> channels are supported. */
29159 |         } else {
29160 |             /* The device only supports a specific set of channels. We need to iterate over all
> of them. */
29161 |             for (iChannel = minChannels; iChannel <= maxChannels; iChannel += 1) {
29162 |                 /* Test the channel before applying it to the configuration space. */
29163 |                 unsigned int channels = iChannel;
29164 |
29165 |                 /* Make sure our channel range is reset before testing again or else we'll
> always fail the test. */
29166 |                 ((ma_snd_pcm_hw_params_any_proc)pContext->alsa.snd_pcm_hw_params_any)(pPCM,
> pHWParams);
29167 |                 ((ma_snd_pcm_hw_params_set_format_proc)pContext->alsa.snd_pcm_hw_params_set_format)(pPCM, pHWParams,
> ma_convert_ma_format_to_alsa_format(format));
29168 |
29169 |                 if
> (((ma_snd_pcm_hw_params_test_channels_proc)pContext->alsa.snd_pcm_hw_params_test_channels)(pPCM,
> pHWParams, channels) == 0) {
29170 |                     /* The channel count is supported. */
29171 |
29172 |                     /* The configuration space now needs to be restricted to the channel count
> before extracting the sample rate. */
29173 |                     ((ma_snd_pcm_hw_params_set_channels_proc)pContext->alsa.snd_pcm_hw_params_set_channels)(pPCM,
> pHWParams, channels);
29174 |
29175 |                     /* Only after the configuration space has been restricted to the specific
> channel count should we iterate over our sample rates. */
29176 |                     ma_context_iterate_rates_and_add_native_data_format_alsa(pContext, pPCM,
> pHWParams, format, channels, 0, pDeviceInfo);
29177 |                 } else {
29178 |                     /* The channel count is not supported. Skip. */
29179 |                 }
29180 |             }
29181 |         }
29182 |     } else {
29183 |         /* The format is not supported. Skip. */
29184 |     }
29185 | }
29186 |
29187 | ma_free(pHWParams, &pContext->allocationCallbacks);
29188 |
29189 | ((ma_snd_pcm_close_proc)pContext->alsa.snd_pcm_close)(pPCM);
29190 | return MA_SUCCESS;
29191 | }
29192 |
29193 | static ma_result ma_device_uninit__alsa(ma_device* pDevice)
29194 | {
29195 |     MA_ASSERT(pDevice != NULL);
29196 |
29197 |     if ((ma_snd_pcm_t*)pDevice->alsa.pPCMCapture) {
29198 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)((ma_snd_pcm_t*)pDevice->alsa.
> pPCMCapture);
29199 |         close(pDevice->alsa.wakeupfdCapture);
29200 |         ma_free(pDevice->alsa.pPollDescriptorsCapture, &pDevice->pContext->allocationCallbacks);
29201 |     }
29202 |
29203 |     if ((ma_snd_pcm_t*)pDevice->alsa.pPCMPlayback) {
29204 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)((ma_snd_pcm_t*)pDevice->alsa.

```

```

    > pPCMPPlayback);
29205 |     close(pDevice->alsa.wakeupfdPlayback);
29206 |     ma_free(pDevice->alsa.pPollDescriptorsPlayback, &pDevice->pContext->allocationCallbacks);
29207 | }
29208 |
29209 |     return MA_SUCCESS;
29210 | }
29211 |
29212 | static ma_result ma_device_init_by_type_alsa(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptor, ma_device_type deviceType)
29213 | {
29214 |     ma_result result;
29215 |     int resultALSA;
29216 |     ma_snd_pcm_t* pPCM;
29217 |     ma_bool32 isUsingMMap;
29218 |     ma_snd_pcm_format_t formatALSA;
29219 |     ma_format internalFormat;
29220 |     ma_uint32 internalChannels;
29221 |     ma_uint32 internalSampleRate;
29222 |     ma_channel internalChannelMap[MA_MAX_CHANNELS];
29223 |     ma_uint32 internalPeriodSizeInFrames;
29224 |     ma_uint32 internalPeriods;
29225 |     int openMode;
29226 |     ma_snd_pcm_hw_params_t* pHWParams;
29227 |     ma_snd_pcm_sw_params_t* pSWParams;
29228 |     ma_snd_pcm_uframes_t bufferBoundary;
29229 |     int pollDescriptorCount;
29230 |     struct pollfd* pPollDescriptors;
29231 |     int wakeupfd;
29232 |
29233 |     MA_ASSERT(pConfig != NULL);
29234 |     MA_ASSERT(deviceType != ma_device_type_duplex); /* This function should only be called for
> playback_or_capture, never duplex. */
29235 |     MA_ASSERT(pDevice != NULL);
29236 |
29237 |     formatALSA = ma_convert_ma_format_to_alsa_format(pDescriptor->format);
29238 |
29239 |     openMode = 0;
29240 |     if (pConfig->alsa.noAutoResample) {
29241 |         openMode |= MA_SND_PCM_NO_AUTO_RESAMPLE;
29242 |     }
29243 |     if (pConfig->alsa.noAutoChannels) {
29244 |         openMode |= MA_SND_PCM_NO_AUTO_CHANNELS;
29245 |     }
29246 |     if (pConfig->alsa.noAutoFormat) {
29247 |         openMode |= MA_SND_PCM_NO_AUTO_FORMAT;
29248 |     }
29249 |
29250 |     result = ma_context_open_pcm_alsa(pDevice->pContext, pDescriptor->shareMode, deviceType,
> pDescriptor->pDeviceID, openMode, &pPCM);
29251 |     if (result != MA_SUCCESS) {
29252 |         return result;
29253 |     }
29254 |
29255 |
29256 |     /* Hardware parameters. */
29257 |     pHWParams = (ma_snd_pcm_hw_params_t*)ma_malloc(((ma_snd_pcm_hw_params_sizeof_proc)pDevice->pCont
> ext->alsa.snd_pcm_hw_params_sizeof()), &pDevice->pContext->allocationCallbacks);
29258 |     if (pHWParams == NULL) {
29259 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29260 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to allocate
> memory for hardware parameters.");
29261 |         return MA_OUT_OF_MEMORY;
29262 |     }
29263 |
29264 |     resultALSA =
> ((ma_snd_pcm_hw_params_any_proc)pDevice->pContext->alsa.snd_pcm_hw_params_any)(pPCM, pHWParams);
29265 |     if (resultALSA < 0) {
29266 |         ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29267 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29268 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to initialize
> hardware parameters. snd_pcm_hw_params_any() failed.");
29269 |         return ma_result_from_errno(-resultALSA);
29270 |     }
29271 |
29272 |     /* MMAP Mode. Try using interleaved MMAP access. If this fails, fall back to standard
> readi/writei. */
29273 |     isUsingMMap = MA_FALSE;
29274 |     #if 0 /* NOTE: MMAP mode temporarily disabled. */

```

```

29275 |     if (deviceType != ma_device_type_capture) { /* <-- Disabling MMAP mode for capture devices
> because I apparently do not have a device that supports it which means I can't test it...
> Contributions welcome. */
29276 |         if (!pConfig->alsa.noMMap) {
29277 |             if
> (((ma_snd_pcm_hw_params_set_access_proc)pDevice->pContext->alsa.snd_pcm_hw_params_set_access)(pPCM,
> pHWParams, MA_SND_PCM_ACCESS_MMAP_INTERLEAVED) == 0) {
29278 |                 pDevice->alsa.isUsingMMap = MA_TRUE;
29279 |             }
29280 |         }
29281 |     }
29282 | #endif
29283 |
29284 |     if (!isUsingMMap) {
29285 |         resultALSA =
> ((ma_snd_pcm_hw_params_set_access_proc)pDevice->pContext->alsa.snd_pcm_hw_params_set_access)(pPCM,
> pHWParams, MA_SND_PCM_ACCESS_RW_INTERLEAVED);
29286 |         if (resultALSA < 0) {
29287 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29288 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29289 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set access
> mode to neither SND_PCM_ACCESS_MMAP_INTERLEAVED nor SND_PCM_ACCESS_RW_INTERLEAVED.
> snd_pcm_hw_params_set_access() failed.");
29290 |             return ma_result_from_errno(-resultALSA);
29291 |         }
29292 |     }
29293 |
29294 |     /*
29295 |     Most important properties first. The documentation for OSS (yes, I know this is ALSA!)
> recommends format, channels, then sample rate. I can't
29296 |     find any documentation for ALSA specifically, so I'm going to copy the recommendation for OSS.
29297 |     */
29298 |
29299 |     /* Format. */
29300 |     {
29301 |         /*
29302 |         At this point we should have a list of supported formats, so now we need to find the best
> one. We first check if the requested format is
29303 |         supported, and if so, use that one. If it's not supported, we just run through a list of
> formats and try to find the best one.
29304 |         */
29305 |         if (formatALSA == MA_SND_PCM_FORMAT_UNKNOWN ||
> ((ma_snd_pcm_hw_params_test_format_proc)pDevice->pContext->alsa.snd_pcm_hw_params_test_format)(pPCM,
> pHWParams, formatALSA) != 0) {
29306 |             /* We're either requesting the native format or the specified format is not supported.
> */
29307 |             size_t iFormat;
29308 |
29309 |             formatALSA = MA_SND_PCM_FORMAT_UNKNOWN;
29310 |             for (iFormat = 0; iFormat < ma_countof(g_maFormatPriorities); ++iFormat) {
29311 |                 if (((ma_snd_pcm_hw_params_test_format_proc)pDevice->pContext->alsa.snd_pcm_hw_param
> s_test_format)(pPCM, pHWParams, ma_convert_ma_format_to_alsa_format(g_maFormatPriorities[iFormat]))
> == 0) {
29312 |                     formatALSA = ma_convert_ma_format_to_alsa_format(g_maFormatPriorities[iFormat]);
29313 |                     break;
29314 |                 }
29315 |             }
29316 |
29317 |             if (formatALSA == MA_SND_PCM_FORMAT_UNKNOWN) {
29318 |                 ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29319 |                 ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29320 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Format not
> supported. The device does not support any miniaudio formats.");
29321 |                 return MA_FORMAT_NOT_SUPPORTED;
29322 |             }
29323 |         }
29324 |
29325 |         resultALSA =
> ((ma_snd_pcm_hw_params_set_format_proc)pDevice->pContext->alsa.snd_pcm_hw_params_set_format)(pPCM,
> pHWParams, formatALSA);
29326 |         if (resultALSA < 0) {
29327 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29328 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29329 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Format not
> supported. snd_pcm_hw_params_set_format() failed.");
29330 |             return ma_result_from_errno(-resultALSA);
29331 |         }
29332 |
29333 |         internalFormat = ma_format_from_alsa(formatALSA);

```



```

29334 |         if (internalFormat == ma_format_unknown) {
29335 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29336 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29337 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] The chosen format is
> not supported by miniaudio.");
29338 |             return MA_FORMAT_NOT_SUPPORTED;
29339 |         }
29340 |     }
29341 |
29342 |     /* Channels. */
29343 |     {
29344 |         unsigned int channels = pDescriptor->channels;
29345 |         if (channels == 0) {
29346 |             channels = MA_DEFAULT_CHANNELS;
29347 |         }
29348 |
29349 |         resultALSA = ((ma_snd_pcm_hw_params_set_channels_near_proc)pDevice->pContext->alsa.snd_pcm_h
> w_params_set_channels_near)(pPCM, pHWParams, &channels);
29350 |         if (resultALSA < 0) {
29351 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29352 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29353 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set
> channel count. snd_pcm_hw_params_set_channels_near() failed.");
29354 |             return ma_result_from_errno(-resultALSA);
29355 |         }
29356 |
29357 |         internalChannels = (ma_uint32)channels;
29358 |     }
29359 |
29360 |     /* Sample Rate */
29361 |     {
29362 |         unsigned int sampleRate;
29363 |
29364 |         /*
29365 |         > It appears there's either a bug in ALSA, a bug in some drivers, or I'm doing something
> silly; but having resampling enabled causes
29366 |         > problems with some device configurations when used in conjunction with MMAP access mode. To
> fix this problem we need to disable
29367 |         > resampling.
29368 |
29369 |         > To reproduce this problem, open the "plug:dmix" device, and set the sample rate to 44100.
> Internally, it looks like dmix uses a
29370 |         > sample rate of 48000. The hardware parameters will get set correctly with no errors, but it
> looks like the 44100 -> 48000 resampling
29371 |         > doesn't work properly - but only with MMAP access mode. You will notice skipping/crackling
> in the audio, and it'll run at a slightly
29372 |         > faster rate.
29373 |
29374 |         > miniaudio has built-in support for sample rate conversion (albeit low quality at the
> moment), so disabling resampling should be fine
29375 |         > for us. The only problem is that it won't be taking advantage of any kind of
> hardware-accelerated resampling and it won't be very
29376 |         > good quality until I get a chance to improve the quality of miniaudio's software sample rate
> conversion.
29377 |
29378 |         > I don't currently know if the dmix plugin is the only one with this error. Indeed, this is
> the only one I've been able to reproduce
29379 |         > this error with. In the future, we may want to restrict the disabling of resampling to only
> known bad plugins.
29380 |         */
29381 |         ((ma_snd_pcm_hw_params_set_rate_resample_proc)pDevice->pContext->alsa.snd_pcm_hw_params_set_
> rate_resample)(pPCM, pHWParams, 0);
29382 |
29383 |         sampleRate = pDescriptor->sampleRate;
29384 |         if (sampleRate == 0) {
29385 |             sampleRate = MA_DEFAULT_SAMPLE_RATE;
29386 |         }
29387 |
29388 |         resultALSA = ((ma_snd_pcm_hw_params_set_rate_near_proc)pDevice->pContext->alsa.snd_pcm_hw_pa
> rams_set_rate_near)(pPCM, pHWParams, &sampleRate, 0);
29389 |         if (resultALSA < 0) {
29390 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29391 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29392 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Sample rate not
> supported. snd_pcm_hw_params_set_rate_near() failed.");
29393 |             return ma_result_from_errno(-resultALSA);
29394 |         }
29395 |
29396 |         internalSampleRate = (ma_uint32)sampleRate;

```

```

29397 |     }
29398 |
29399 |     /* Periods. */
29400 |     {
29401 |         ma_uint32 periods = pDescriptor->periodCount;
29402 |
29403 |         resultALSA = ((ma_snd_pcm_hw_params_set_periods_near_proc)pDevice->pContext->alsa.snd_pcm_hw
> _params_set_periods_near)(pPCM, pHWParams, &periods, NULL);
29404 |         if (resultALSA < 0) {
29405 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29406 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29407 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set period
> count. snd_pcm_hw_params_set_periods_near() failed.");
29408 |             return ma_result_from_errno(-resultALSA);
29409 |         }
29410 |
29411 |         internalPeriods = periods;
29412 |     }
29413 |
29414 |     /* Buffer Size */
29415 |     {
29416 |         ma_snd_pcm_uframes_t actualBufferSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor, internalSampleRate,
> pConfig->performanceProfile) * internalPeriods;
29417 |
29418 |         resultALSA = ((ma_snd_pcm_hw_params_set_buffer_size_near_proc)pDevice->pContext->alsa.snd_pc
> m_hw_params_set_buffer_size_near)(pPCM, pHWParams, &actualBufferSizeInFrames);
29419 |         if (resultALSA < 0) {
29420 |             ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29421 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29422 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set buffer
> size for device. snd_pcm_hw_params_set_buffer_size() failed.");
29423 |             return ma_result_from_errno(-resultALSA);
29424 |         }
29425 |
29426 |         internalPeriodSizeInFrames = actualBufferSizeInFrames / internalPeriods;
29427 |     }
29428 |
29429 |     /* Apply hardware parameters. */
29430 |     resultALSA = ((ma_snd_pcm_hw_params_proc)pDevice->pContext->alsa.snd_pcm_hw_params)(pPCM,
> pHWParams);
29431 |     if (resultALSA < 0) {
29432 |         ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29433 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29434 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set hardware
> parameters. snd_pcm_hw_params() failed.");
29435 |         return ma_result_from_errno(-resultALSA);
29436 |     }
29437 |
29438 |     ma_free(pHWParams, &pDevice->pContext->allocationCallbacks);
29439 |     pHWParams = NULL;
29440 |
29441 |
29442 |     /* Software parameters. */
29443 |     pSWParams = (ma_snd_pcm_sw_params_t*)ma_malloc(((ma_snd_pcm_sw_params_sizeof_proc)pDevice->pCont
> ext->alsa.snd_pcm_sw_params_sizeof)(), &pDevice->pContext->allocationCallbacks);
29444 |     if (pSWParams == NULL) {
29445 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29446 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to allocate
> memory for software parameters.");
29447 |         return MA_OUT_OF_MEMORY;
29448 |     }
29449 |
29450 |     resultALSA =
> ((ma_snd_pcm_sw_params_current_proc)pDevice->pContext->alsa.snd_pcm_sw_params_current)(pPCM,
> pSWParams);
29451 |     if (resultALSA < 0) {
29452 |         ma_free(pSWParams, &pDevice->pContext->allocationCallbacks);
29453 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29454 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to initialize
> software parameters. snd_pcm_sw_params_current() failed.");
29455 |         return ma_result_from_errno(-resultALSA);
29456 |     }
29457 |
29458 |     resultALSA = ((ma_snd_pcm_sw_params_set_avail_min_proc)pDevice->pContext->alsa.snd_pcm_sw_params
> _set_avail_min)(pPCM, pSWParams, ma_prev_power_of_2(internalPeriodSizeInFrames));
29459 |     if (resultALSA < 0) {
29460 |         ma_free(pSWParams, &pDevice->pContext->allocationCallbacks);
29461 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);

```

```

29462 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA]
> snd_pcm_sw_params_set_avail_min() failed.");
29463 |         return ma_result_from_errno(-resultALSA);
29464 |     }
29465 |
29466 |     resultALSA = ((ma_snd_pcm_sw_params_get_boundary_proc)pDevice->pContext->alsa.snd_pcm_sw_params_
> get_boundary)(pSWParams, &bufferBoundary);
29467 |     if (resultALSA < 0) {
29468 |         bufferBoundary = internalPeriodSizeInFrames * internalPeriods;
29469 |     }
29470 |
29471 |     if (deviceType == ma_device_type_playback && !isUsingMMap) { /* Only playback devices in
> write/read mode need a start threshold. */
29472 |         /*
29473 |         Subtle detail here with the start threshold. When in playback-only mode (no full-duplex) we
> can set the start threshold to
29474 |         the size of a period. But for full-duplex we need to set it such that it is at least two
> periods.
29475 |         */
29476 |         resultALSA = ((ma_snd_pcm_sw_params_set_start_threshold_proc)pDevice->pContext->alsa.snd_pcm
> _sw_params_set_start_threshold)(pPCM, pSWParams, internalPeriodSizeInFrames*2);
29477 |         if (resultALSA < 0) {
29478 |             ma_free(pSWParams, &pDevice->pContext->allocationCallbacks);
29479 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29480 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set start
> threshold for playback device. snd_pcm_sw_params_set_start_threshold() failed.");
29481 |             return ma_result_from_errno(-resultALSA);
29482 |         }
29483 |
29484 |         resultALSA = ((ma_snd_pcm_sw_params_set_stop_threshold_proc)pDevice->pContext->alsa.snd_pcm_
> sw_params_set_stop_threshold)(pPCM, pSWParams, bufferBoundary);
29485 |         if (resultALSA < 0) { /* Set to boundary to loop instead of stop in the event of an xrun. */
29486 |             ma_free(pSWParams, &pDevice->pContext->allocationCallbacks);
29487 |             ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29488 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set stop
> threshold for playback device. snd_pcm_sw_params_set_stop_threshold() failed.");
29489 |             return ma_result_from_errno(-resultALSA);
29490 |         }
29491 |     }
29492 |
29493 |     resultALSA = ((ma_snd_pcm_sw_params_proc)pDevice->pContext->alsa.snd_pcm_sw_params)(pPCM,
> pSWParams);
29494 |     if (resultALSA < 0) {
29495 |         ma_free(pSWParams, &pDevice->pContext->allocationCallbacks);
29496 |         ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29497 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to set software
> parameters. snd_pcm_sw_params() failed.");
29498 |         return ma_result_from_errno(-resultALSA);
29499 |     }
29500 |
29501 |     ma_free(pSWParams, &pDevice->pContext->allocationCallbacks);
29502 |     pSWParams = NULL;
29503 |
29504 |
29505 |     /* Grab the internal channel map. For now we're not going to bother trying to change the channel
> map and instead just do it ourselves. */
29506 |     {
29507 |         ma_snd_pcm_chmap_t* pChmap = NULL;
29508 |         if (pDevice->pContext->alsa.snd_pcm_get_chmap != NULL) {
29509 |             pChmap = ((ma_snd_pcm_get_chmap_proc)pDevice->pContext->alsa.snd_pcm_get_chmap)(pPCM);
29510 |         }
29511 |
29512 |         if (pChmap != NULL) {
29513 |             ma_uint32 iChannel;
29514 |
29515 |             /* There are cases where the returned channel map can have a different channel count
> than was returned by snd_pcm_hw_params_set_channels_near(). */
29516 |             if (pChmap->channels >= internalChannels) {
29517 |                 /* Drop excess channels. */
29518 |                 for (iChannel = 0; iChannel < internalChannels; ++iChannel) {
29519 |                     internalChannelMap[iChannel] =
> ma_convert_alsa_channel_position_to_ma_channel(pChmap->pos[iChannel]);
29520 |                 }
29521 |             } else {
29522 |                 ma_uint32 i;
29523 |
29524 |                 /*
29525 |                 Excess channels use defaults. Do an initial fill with defaults, overwrite the first
> pChmap->channels, validate to ensure there are no duplicate

```

```

29526 |         channels. If validation fails, fall back to defaults.
29527 |         */
29528 |         ma_bool32 isValid = MA_TRUE;
29529 |
29530 |         /* Fill with defaults. */
29531 |         ma_channel_map_init_standard(ma_standard_channel_map_alsa, internalChannelMap,
> ma_countof(internalChannelMap), internalChannels);
29532 |
29533 |         /* Overwrite first pChmap->channels channels. */
29534 |         for (iChannel = 0; iChannel < pChmap->channels; ++iChannel) {
29535 |             internalChannelMap[iChannel] =
> ma_convert_alsa_channel_position_to_ma_channel(pChmap->pos[iChannel]);
29536 |         }
29537 |
29538 |         /* Validate. */
29539 |         for (i = 0; i < internalChannels && isValid; ++i) {
29540 |             ma_uint32 j;
29541 |             for (j = i+1; j < internalChannels; ++j) {
29542 |                 if (internalChannelMap[i] == internalChannelMap[j]) {
29543 |                     isValid = MA_FALSE;
29544 |                     break;
29545 |                 }
29546 |             }
29547 |         }
29548 |
29549 |         /* If our channel map is invalid, fall back to defaults. */
29550 |         if (!isValid) {
29551 |             ma_channel_map_init_standard(ma_standard_channel_map_alsa, internalChannelMap,
> ma_countof(internalChannelMap), internalChannels);
29552 |         }
29553 |     }
29554 |
29555 |     free(pChmap);
29556 |     pChmap = NULL;
29557 | } else {
29558 |     /* Could not retrieve the channel map. Fall back to a hard-coded assumption. */
29559 |     ma_channel_map_init_standard(ma_standard_channel_map_alsa, internalChannelMap,
> ma_countof(internalChannelMap), internalChannels);
29560 | }
29561 | }
29562 |
29563 |
29564 | /*
29565 | We need to retrieve the poll descriptors so we can use poll() to wait for data to become
29566 | available for reading or writing. There's no well defined maximum for this so we're just going
29567 | to allocate this on the heap.
29568 | */
29569 | pollDescriptorCount = ((ma_snd_pcm_poll_descriptors_count_proc)pDevice->pContext->alsa.snd_pcm_p
> oll_descriptors_count)(pPCM);
29570 | if (pollDescriptorCount <= 0) {
29571 |     ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29572 |     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to retrieve poll
> descriptors count.");
29573 |     return MA_ERROR;
29574 | }
29575 |
29576 | pPollDescriptors = (struct pollfd*)ma_malloc(sizeof(*pPollDescriptors) * (pollDescriptorCount +
> 1), &pDevice->pContext->allocationCallbacks); /* +1 because we want room for the wakeup
> descriptor. */
29577 | if (pPollDescriptors == NULL) {
29578 |     ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29579 |     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to allocate
> memory for poll descriptors.");
29580 |     return MA_OUT_OF_MEMORY;
29581 | }
29582 |
29583 | /*
29584 | We need an eventfd to wakeup from poll() and avoid a deadlock in situations where the driver
29585 | never returns from writei() and readi(). This has been observed with the "pulse" device.
29586 | */
29587 | wakeupfd = eventfd(0, 0);
29588 | if (wakeupfd < 0) {
29589 |     ma_free(pPollDescriptors, &pDevice->pContext->allocationCallbacks);
29590 |     ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29591 |     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to create eventfd
> for poll wakeup.");
29592 |     return ma_result_from_errno(errno);
29593 | }
29594 |

```

```

29595 | /* We'll place the wakeup fd at the start of the buffer. */
29596 | pPollDescriptors[0].fd = wakeupfd;
29597 | pPollDescriptors[0].events = POLLIN; /* We only care about waiting to read from the wakeup
> file descriptor. */
29598 | pPollDescriptors[0].revents = 0;
29599 |
29600 | /* We can now extract the PCM poll descriptors which we place after the wakeup descriptor. */
29601 | pollDescriptorCount =
> ((ma_snd_pcm_poll_descriptors_proc)pDevice->pContext->alsa.snd_pcm_poll_descriptors)(pPCM,
> pPollDescriptors + 1, pollDescriptorCount); /* +1 because we want to place these descriptors
> after the wakeup descriptor. */
29602 | if (pollDescriptorCount <= 0) {
29603 |     close(wakeupfd);
29604 |     ma_free(pPollDescriptors, &pDevice->pContext->allocationCallbacks);
29605 |     ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29606 |     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to retrieve poll
> descriptors.");
29607 |     return MA_ERROR;
29608 | }
29609 |
29610 | if (deviceType == ma_device_type_capture) {
29611 |     pDevice->alsa.pollDescriptorCountCapture = pollDescriptorCount;
29612 |     pDevice->alsa.pPollDescriptorsCapture = pPollDescriptors;
29613 |     pDevice->alsa.wakeupfdCapture = wakeupfd;
29614 | } else {
29615 |     pDevice->alsa.pollDescriptorCountPlayback = pollDescriptorCount;
29616 |     pDevice->alsa.pPollDescriptorsPlayback = pPollDescriptors;
29617 |     pDevice->alsa.wakeupfdPlayback = wakeupfd;
29618 | }
29619 |
29620 |
29621 | /* We're done. Prepare the device. */
29622 | resultALSA = ((ma_snd_pcm_prepare_proc)pDevice->pContext->alsa.snd_pcm_prepare)(pPCM);
29623 | if (resultALSA < 0) {
29624 |     close(wakeupfd);
29625 |     ma_free(pPollDescriptors, &pDevice->pContext->allocationCallbacks);
29626 |     ((ma_snd_pcm_close_proc)pDevice->pContext->alsa.snd_pcm_close)(pPCM);
29627 |     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to prepare
> device.");
29628 |     return ma_result_from_errno(-resultALSA);
29629 | }
29630 |
29631 |
29632 | if (deviceType == ma_device_type_capture) {
29633 |     pDevice->alsa.pPCM Capture = (ma_ptr)pPCM;
29634 |     pDevice->alsa.isUsingMMapCapture = isUsingMMap;
29635 | } else {
29636 |     pDevice->alsa.pPCMPlayback = (ma_ptr)pPCM;
29637 |     pDevice->alsa.isUsingMMapPlayback = isUsingMMap;
29638 | }
29639 |
29640 | pDescriptor->format = internalFormat;
29641 | pDescriptor->channels = internalChannels;
29642 | pDescriptor->sampleRate = internalSampleRate;
29643 | ma_channel_map_copy(pDescriptor->channelMap, internalChannelMap, ma_min(internalChannels,
> MA_MAX_CHANNELS));
29644 | pDescriptor->periodSizeInFrames = internalPeriodSizeInFrames;
29645 | pDescriptor->periodCount = internalPeriods;
29646 |
29647 | return MA_SUCCESS;
29648 | }
29649 |
29650 | static ma_result ma_device_init_alsa(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
29651 | {
29652 |     MA_ASSERT(pDevice != NULL);
29653 |
29654 |     MA_ZERO_OBJECT(&pDevice->alsa);
29655 |
29656 |     if (pConfig->deviceType == ma_device_type_loopback) {
29657 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
29658 |     }
29659 |
29660 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
29661 |         ma_result result = ma_device_init_by_type_alsa(pDevice, pConfig, pDescriptorCapture,
> ma_device_type_capture);
29662 |         if (result != MA_SUCCESS) {
29663 |             return result;

```

```

29664 |     }
29665 | }
29666 |
29667 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
29668 |         ma_result result = ma_device_init_by_type__alsa(pDevice, pConfig, pDescriptorPlayback,
> ma_device_type_playback);
29669 |         if (result != MA_SUCCESS) {
29670 |             return result;
29671 |         }
29672 |     }
29673 |
29674 |     return MA_SUCCESS;
29675 | }
29676 |
29677 | static ma_result ma_device_start__alsa(ma_device* pDevice)
29678 | {
29679 |     int resultALSA;
29680 |
29681 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
29682 |         resultALSA = ((ma_snd_pcm_start_proc)pDevice->pContext->alsa.snd_pcm_start)((ma_snd_pcm_t*)p
> Device->alsa.pPCMCapture);
29683 |         if (resultALSA < 0) {
29684 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to start
> capture device.");
29685 |             return ma_result_from_errno(-resultALSA);
29686 |         }
29687 |     }
29688 |
29689 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
29690 |         /*
29691 |         When data is written to the device we wait for the device to get ready to receive data with
> poll(). In my testing
29692 |         I have observed that poll() can sometimes block forever unless the device is started
> explicitly with snd_pcm_start()
29693 |         or some data is written with snd_pcm_wrotei().
29694 |
29695 |         To resolve this I've decided to do an explicit start with snd_pcm_start(). The problem with
> this is that the device
29696 |         is started without any data in the internal buffer which will result in an immediate
> underrun. If instead we were
29697 |         to call into snd_pcm_wrotei() in an attempt to prevent the underrun, we would run the risk
> of a weird deadlock
29698 |         issue as documented inside ma_device_write__alsa().
29699 |         */
29700 |         resultALSA = ((ma_snd_pcm_start_proc)pDevice->pContext->alsa.snd_pcm_start)((ma_snd_pcm_t*)p
> Device->alsa.pPCMPlayback);
29701 |         if (resultALSA < 0) {
29702 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to start
> playback device.");
29703 |             return ma_result_from_errno(-resultALSA);
29704 |         }
29705 |     }
29706 |
29707 |     return MA_SUCCESS;
29708 | }
29709 |
29710 | static ma_result ma_device_stop__alsa(ma_device* pDevice)
29711 | {
29712 |     /*
29713 |     The stop callback will get called on the worker thread after read/write__alsa() has returned. At
> this point there is
29714 |     a small chance that our wakeupfd has not been cleared. We'll clear that out now if applicable.
29715 |     */
29716 |     int resultPoll;
29717 |     int resultRead;
29718 |
29719 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
29720 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Dropping capture
> device...\n");
29721 |         ((ma_snd_pcm_drop_proc)pDevice->pContext->alsa.snd_pcm_drop)((ma_snd_pcm_t*)pDevice->alsa.pP
> CMCapture);
29722 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Dropping capture device
> successful.\n");
29723 |
29724 |         /* We need to prepare the device again, otherwise we won't be able to restart the device. */
29725 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Preparing capture
> device...\n");
29726 |         if ((ma_snd_pcm_prepare_proc)pDevice->pContext->alsa.snd_pcm_prepare)((ma_snd_pcm_t*)pDevic

```



```

> e->alsa.pCMCapture) < 0) {
29727 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Preparing capture
> device failed.\n");
29728 |     } else {
29729 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Preparing capture
> device successful.\n");
29730 |     }
29731 |
29732 |     /* Clear the wakeupfd. */
29733 |     resultPoll = poll((struct pollfd*)pDevice->alsa.pPollDescriptorsCapture, 1, 0);
29734 |     if (resultPoll > 0) {
29735 |         ma_uint64 t;
29736 |         resultRead = read(((struct pollfd*)pDevice->alsa.pPollDescriptorsCapture)[0].fd, &t,
> sizeof(t));
29737 |         if (resultRead != sizeof(t)) {
29738 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Failed to read
> from capture wakeupfd. read() = %d\n", resultRead);
29739 |         }
29740 |     }
29741 | }
29742 |
29743 | if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
29744 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Dropping playback
> device...\n");
29745 |     ((ma_snd_pcm_drop_proc)pDevice->pContext->alsa.snd_pcm_drop)((ma_snd_pcm_t*)pDevice->alsa.pP
> CMPlayback);
29746 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Dropping playback
> device successful.\n");
29747 |
29748 |     /* We need to prepare the device again, otherwise we won't be able to restart the device. */
29749 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Preparing playback
> device...\n");
29750 |     if (((ma_snd_pcm_prepare_proc)pDevice->pContext->alsa.snd_pcm_prepare)((ma_snd_pcm_t*)pDevic
> e->alsa.pCMPlayback) < 0) {
29751 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Preparing playback
> device failed.\n");
29752 |     } else {
29753 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Preparing playback
> device successful.\n");
29754 |     }
29755 |
29756 |     /* Clear the wakeupfd. */
29757 |     resultPoll = poll((struct pollfd*)pDevice->alsa.pPollDescriptorsPlayback, 1, 0);
29758 |     if (resultPoll > 0) {
29759 |         ma_uint64 t;
29760 |         resultRead = read(((struct pollfd*)pDevice->alsa.pPollDescriptorsPlayback)[0].fd, &t,
> sizeof(t));
29761 |         if (resultRead != sizeof(t)) {
29762 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Failed to read
> from playback wakeupfd. read() = %d\n", resultRead);
29763 |         }
29764 |     }
29765 | }
29766 |
29767 |     return MA_SUCCESS;
29768 | }
29769 |
29770 | static ma_result ma_device_wait__alsa(ma_device* pDevice, ma_snd_pcm_t* pPCM, struct pollfd*
> pPollDescriptors, int pollDescriptorCount, short requiredEvent)
29771 | {
29772 |     for (;;) {
29773 |         unsigned short revents;
29774 |         int resultALSA;
29775 |         int resultPoll = poll(pPollDescriptors, pollDescriptorCount, -1);
29776 |         if (resultPoll < 0) {
29777 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[ALSA] poll()
> failed.\n");
29778 |
29779 |             /*
29780 |              There have been reports that poll() is returning an error randomly and that instead of
29781 |              returning an error, simply trying again will work. I'm experimenting with adopting this
29782 |              advice.
29783 |              */
29784 |             continue;
29785 |             /*return ma_result_from_errno(errno);*/
29786 |         }
29787 |
29788 |         /*
29789 |         Before checking the ALSA poll descriptor flag we need to check if the wakeup descriptor

```



```

29790 |         has had it's POLLIN flag set. If so, we need to actually read the data and then exit the
29791 |         function. The wakeup descriptor will be the first item in the descriptors buffer.
29792 |         */
29793 |         if ((pPollDescriptors[0].revents & POLLIN) != 0) {
29794 |             ma_uint64 t;
29795 |             int resultRead = read(pPollDescriptors[0].fd, &t, sizeof(t));    /* <-- Important that
> we read here so that the next write() does not block. */
29796 |             if (resultRead < 0) {
29797 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] read()
> failed.\n");
29798 |                 return ma_result_from_errno(errno);
29799 |             }
29800 |
29801 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] POLLIN set for
> wakeupfd\n");
29802 |             return MA_DEVICE_NOT_STARTED;
29803 |         }
29804 |
29805 |         /*
29806 |         Getting here means that some data should be able to be read. We need to use ALSA to
29807 |         translate the revents flags for us.
29808 |         */
29809 |         resultALSA = ((ma_snd_pcm_poll_descriptors_revents_proc)pDevice->pContext->alsa.snd_pcm_poll
> _descriptors_revents)(pPCM, pPollDescriptors + 1, pollDescriptorCount - 1, &revents);    /* +1, -1 to
> ignore the wakeup descriptor. */
29810 |         if (resultALSA < 0) {
29811 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA]
> snd_pcm_poll_descriptors_revents() failed.\n");
29812 |             return ma_result_from_errno(-resultALSA);
29813 |         }
29814 |
29815 |         if ((revents & POLLERR) != 0) {
29816 |             ma_snd_pcm_state_t state =
> ((ma_snd_pcm_state_proc)pDevice->pContext->alsa.snd_pcm_state)(pPCM);
29817 |             if (state == MA_SND_PCM_STATE_XRUN) {
29818 |                 /* The PCM is in a xrun state. This will be recovered from at a higher level. We can
> disregard this. */
29819 |             } else {
29820 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[ALSA] POLLERR
> detected. status = %d\n", ((ma_snd_pcm_state_proc)pDevice->pContext->alsa.snd_pcm_state)(pPCM));
29821 |             }
29822 |         }
29823 |
29824 |         if ((revents & requiredEvent) == requiredEvent) {
29825 |             break;    /* We're done. Data available for reading or writing. */
29826 |         }
29827 |     }
29828 |
29829 |     return MA_SUCCESS;
29830 | }
29831 |
29832 | static ma_result ma_device_wait_read__alsa(ma_device* pDevice)
29833 | {
29834 |     return ma_device_wait__alsa(pDevice, (ma_snd_pcm_t*)pDevice->alsa.pPCMCapture, (struct
> pollfd*)pDevice->alsa.pPollDescriptorsCapture, pDevice->alsa.pollDescriptorCountCapture + 1,
> POLLIN);    /* +1 to account for the wakeup descriptor. */
29835 | }
29836 |
29837 | static ma_result ma_device_wait_write__alsa(ma_device* pDevice)
29838 | {
29839 |     return ma_device_wait__alsa(pDevice, (ma_snd_pcm_t*)pDevice->alsa.pPCMPlayback, (struct
> pollfd*)pDevice->alsa.pPollDescriptorsPlayback, pDevice->alsa.pollDescriptorCountPlayback + 1,
> POLLOUT);    /* +1 to account for the wakeup descriptor. */
29840 | }
29841 |
29842 | static ma_result ma_device_read__alsa(ma_device* pDevice, void* pFramesOut, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
29843 | {
29844 |     ma_snd_pcm_sframes_t resultALSA = 0;
29845 |
29846 |     MA_ASSERT(pDevice != NULL);
29847 |     MA_ASSERT(pFramesOut != NULL);
29848 |
29849 |     if (pFramesRead != NULL) {
29850 |         *pFramesRead = 0;
29851 |     }
29852 |
29853 |     while (ma_device_get_state(pDevice) == ma_device_state_started) {
29854 |         ma_result result;

```

```

29855 |
29856 |         /* The first thing to do is wait for data to become available for reading. This will return
> an error code if the device has been stopped. */
29857 |         result = ma_device_wait_read_alsa(pDevice);
29858 |         if (result != MA_SUCCESS) {
29859 |             return result;
29860 |         }
29861 |
29862 |         /* Getting here means we should have data available. */
29863 |         resultALSA = ((ma_snd_pcm_readi_proc)pDevice->pContext->alsa.snd_pcm_readi)((ma_snd_pcm_t*)p
> Device->alsa.pPCMCapture, pFramesOut, frameCount);
29864 |         if (resultALSA >= 0) {
29865 |             break; /* Success. */
29866 |         } else {
29867 |             if (resultALSA == -EAGAIN) {
29868 |                 /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "EGAIN (read)\n");*/
29869 |                 continue; /* Try again. */
29870 |             } else if (resultALSA == -EPIPE) {
29871 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "EPIPE (read)\n");
29872 |
29873 |                 /* Overrun. Recover and try again. If this fails we need to return an error. */
29874 |                 resultALSA = ((ma_snd_pcm_recover_proc)pDevice->pContext->alsa.snd_pcm_recover)((ma_
> snd_pcm_t*)pDevice->alsa.pPCMCapture, resultALSA, MA_TRUE);
29875 |                 if (resultALSA < 0) {
29876 |                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to
> recover device after overrun.");
29877 |                     return ma_result_from_errno((int)-resultALSA);
29878 |                 }
29879 |
29880 |                 resultALSA = ((ma_snd_pcm_start_proc)pDevice->pContext->alsa.snd_pcm_start)((ma_snd_
> pcm_t*)pDevice->alsa.pPCMCapture);
29881 |                 if (resultALSA < 0) {
29882 |                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to
> start device after underrun.");
29883 |                     return ma_result_from_errno((int)-resultALSA);
29884 |                 }
29885 |
29886 |                 continue; /* Try reading again. */
29887 |             }
29888 |         }
29889 |     }
29890 |
29891 |     if (pFramesRead != NULL) {
29892 |         *pFramesRead = resultALSA;
29893 |     }
29894 |
29895 |     return MA_SUCCESS;
29896 | }
29897 |
29898 | static ma_result ma_device_write_alsa(ma_device* pDevice, const void* pFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
29899 | {
29900 |     ma_snd_pcm_sframes_t resultALSA = 0;
29901 |
29902 |     MA_ASSERT(pDevice != NULL);
29903 |     MA_ASSERT(pFrames != NULL);
29904 |
29905 |     if (pFramesWritten != NULL) {
29906 |         *pFramesWritten = 0;
29907 |     }
29908 |
29909 |     while (ma_device_get_state(pDevice) == ma_device_state_started) {
29910 |         ma_result result;
29911 |
29912 |         /* The first thing to do is wait for space to become available for writing. This will return
> an error code if the device has been stopped. */
29913 |         result = ma_device_wait_write_alsa(pDevice);
29914 |         if (result != MA_SUCCESS) {
29915 |             return result;
29916 |         }
29917 |
29918 |         resultALSA = ((ma_snd_pcm_writei_proc)pDevice->pContext->alsa.snd_pcm_writei)((ma_snd_pcm_t*
> )pDevice->alsa.pPCMPlayback, pFrames, frameCount);
29919 |         if (resultALSA >= 0) {
29920 |             break; /* Success. */
29921 |         } else {
29922 |             if (resultALSA == -EAGAIN) {
29923 |                 /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "EGAIN (write)\n");*/
29924 |                 continue; /* Try again. */

```

```

29925 |         } else if (resultALSA == -EPIPE) {
29926 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "EPIPE (write)\n");
29927 |
29928 |             /* Underrun. Recover and try again. If this fails we need to return an error. */
29929 |             resultALSA = ((ma_snd_pcm_recover_proc)pDevice->pContext->alsa.snd_pcm_recover)((ma_
> snd_pcm_t*)pDevice->alsa.pCMPLPlayback, resultALSA, MA_TRUE); /* MA_TRUE=silent (don't print
> anything on error). */
29930 |             if (resultALSA < 0) {
29931 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to
> recover device after underrun.");
29932 |                 return ma_result_from_errno((int)-resultALSA);
29933 |             }
29934 |
29935 |             /*
29936 |             In my testing I have had a situation where writei() does not automatically restart
> the device even though I've set it
29937 |             up as such in the software parameters. What will happen is writei() will block
> indefinitely even though the number of
29938 |             frames is well beyond the auto-start threshold. To work around this I've needed to
> add an explicit start here. Not sure
29939 |             if this is me just being stupid and not recovering the device properly, but this
> definitely feels like something isn't
29940 |             quite right here.
29941 |             */
29942 |             resultALSA = ((ma_snd_pcm_start_proc)pDevice->pContext->alsa.snd_pcm_start)((ma_snd_
> pcm_t*)pDevice->alsa.pCMPLPlayback);
29943 |             if (resultALSA < 0) {
29944 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] Failed to
> start device after underrun.");
29945 |                 return ma_result_from_errno((int)-resultALSA);
29946 |             }
29947 |
29948 |             continue; /* Try writing again. */
29949 |         }
29950 |     }
29951 | }
29952 |
29953 | if (pFramesWritten != NULL) {
29954 |     *pFramesWritten = resultALSA;
29955 | }
29956 |
29957 | return MA_SUCCESS;
29958 | }
29959 |
29960 | static ma_result ma_device_data_loop_wakeup__alsa(ma_device* pDevice)
29961 | {
29962 |     ma_uint64 t = 1;
29963 |     int resultWrite = 0;
29964 |
29965 |     MA_ASSERT(pDevice != NULL);
29966 |
29967 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Waking up...\n");
29968 |
29969 |     /* Write to an eventfd to trigger a wakeup from poll() and abort any reading or writing. */
29970 |     if (pDevice->alsa.pPollDescriptorsCapture != NULL) {
29971 |         resultWrite = write(pDevice->alsa.wakeupfdCapture, &t, sizeof(t));
29972 |     }
29973 |     if (pDevice->alsa.pPollDescriptorsPlayback != NULL) {
29974 |         resultWrite = write(pDevice->alsa.wakeupfdPlayback, &t, sizeof(t));
29975 |     }
29976 |
29977 |     if (resultWrite < 0) {
29978 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[ALSA] write() failed.\n");
29979 |         return ma_result_from_errno(errno);
29980 |     }
29981 |
29982 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[ALSA] Waking up completed
> successfully.\n");
29983 |
29984 |     return MA_SUCCESS;
29985 | }
29986 |
29987 | static ma_result ma_context_uninit__alsa(ma_context* pContext)
29988 | {
29989 |     MA_ASSERT(pContext != NULL);
29990 |     MA_ASSERT(pContext->backend == ma_backend_alsa);
29991 |
29992 |     /* Clean up memory for memory leak checkers. */
29993 |     ((ma_snd_config_update_free_global_proc)pContext->alsa.snd_config_update_free_global)();

```

```

29994 |
29995 | #ifndef MA_NO_RUNTIME_LINKING
29996 |     ma_dlclose(ma_context_get_log(pContext), pContext->alsa.asoundSO);
29997 | #endif
29998 |
29999 |     ma_mutex_uninit(&pContext->alsa.internalDeviceEnumLock);
30000 |
30001 |     return MA_SUCCESS;
30002 | }
30003 |
30004 | static ma_result ma_context_init_alsa(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
30005 | {
30006 |     ma_result result;
30007 | #ifndef MA_NO_RUNTIME_LINKING
30008 |     const char* libasoundNames[] = {
30009 |         "libasound.so.2",
30010 |         "libasound.so"
30011 |     };
30012 |     size_t i;
30013 |
30014 |     for (i = 0; i < ma_countof(libasoundNames); ++i) {
30015 |         pContext->alsa.asoundSO = ma_dlopen(ma_context_get_log(pContext), libasoundNames[i]);
30016 |         if (pContext->alsa.asoundSO != NULL) {
30017 |             break;
30018 |         }
30019 |     }
30020 |
30021 |     if (pContext->alsa.asoundSO == NULL) {
30022 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "[ALSA] Failed to open shared
> object.\n");
30023 |         return MA_NO_BACKEND;
30024 |     }
30025 |
30026 |     pContext->alsa.snd_pcm_open =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_open");
30027 |     pContext->alsa.snd_pcm_close =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_close");
30028 |     pContext->alsa.snd_pcm_hw_params_sizeof =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_sizeof");
30029 |     pContext->alsa.snd_pcm_hw_params_any =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_hw_params_any");
30030 |     pContext->alsa.snd_pcm_hw_params_set_format =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_format");
30031 |     pContext->alsa.snd_pcm_hw_params_set_format_first =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_format_first");
30032 |     pContext->alsa.snd_pcm_hw_params_get_format_mask =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_get_format_mask");
30033 |     pContext->alsa.snd_pcm_hw_params_set_channels =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_channels");
30034 |     pContext->alsa.snd_pcm_hw_params_set_channels_near =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_channels_near");
30035 |     pContext->alsa.snd_pcm_hw_params_set_channels_minmax =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_channels_minmax");
30036 |     pContext->alsa.snd_pcm_hw_params_set_rate_resample =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_rate_resample");
30037 |     pContext->alsa.snd_pcm_hw_params_set_rate =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_rate");
30038 |     pContext->alsa.snd_pcm_hw_params_set_rate_near =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_rate_near");
30039 |     pContext->alsa.snd_pcm_hw_params_set_buffer_size_near =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_buffer_size_near");
30040 |     pContext->alsa.snd_pcm_hw_params_set_periods_near =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_periods_near");
30041 |     pContext->alsa.snd_pcm_hw_params_set_access =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_hw_params_set_access");

```

```

30042 |     pContext->alsa.snd_pcm_hw_params_get_format           =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_format");
30043 |     pContext->alsa.snd_pcm_hw_params_get_channels         =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_channels");
30044 |     pContext->alsa.snd_pcm_hw_params_get_channels_min     =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_channels_min");
30045 |     pContext->alsa.snd_pcm_hw_params_get_channels_max     =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_channels_max");
30046 |     pContext->alsa.snd_pcm_hw_params_get_rate             =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_rate");
30047 |     pContext->alsa.snd_pcm_hw_params_get_rate_min        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_rate_min");
30048 |     pContext->alsa.snd_pcm_hw_params_get_rate_max        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_rate_max");
30049 |     pContext->alsa.snd_pcm_hw_params_get_buffer_size      =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_buffer_size");
30050 |     pContext->alsa.snd_pcm_hw_params_get_periods         =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_periods");
30051 |     pContext->alsa.snd_pcm_hw_params_get_access          =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_get_access");
30052 |     pContext->alsa.snd_pcm_hw_params_test_format         =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_test_format");
30053 |     pContext->alsa.snd_pcm_hw_params_test_channels       =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_test_channels");
30054 |     pContext->alsa.snd_pcm_hw_params_test_rate           =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_hw_params_test_rate");
30055 |     pContext->alsa.snd_pcm_hw_params                     =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_hw_params");
30056 |     pContext->alsa.snd_pcm_sw_params_sizeof              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_sw_params_sizeof");
30057 |     pContext->alsa.snd_pcm_sw_params_current             =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_sw_params_current");
30058 |     pContext->alsa.snd_pcm_sw_params_get_boundary        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_sw_params_get_boundary");
30059 |     pContext->alsa.snd_pcm_sw_params_set_avail_min       =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_sw_params_set_avail_min");
30060 |     pContext->alsa.snd_pcm_sw_params_set_start_threshold =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_sw_params_set_start_threshold");
30061 |     pContext->alsa.snd_pcm_sw_params_set_stop_threshold  =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_sw_params_set_stop_threshold");
30062 |     pContext->alsa.snd_pcm_sw_params                     =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_sw_params");
30063 |     pContext->alsa.snd_pcm_format_mask_sizeof            =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_format_mask_sizeof");
30064 |     pContext->alsa.snd_pcm_format_mask_test             =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0,
> "snd_pcm_format_mask_test");
30065 |     pContext->alsa.snd_pcm_get_chmap                     =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_get_chmap");
30066 |     pContext->alsa.snd_pcm_state                         =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_state");
30067 |     pContext->alsa.snd_pcm_prepare                      =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_prepare");
30068 |     pContext->alsa.snd_pcm_start                        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_start");
30069 |     pContext->alsa.snd_pcm_drop                         =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_drop");
30070 |     pContext->alsa.snd_pcm_drain                        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundS0, "snd_pcm_drain");

```

```

30071 |     pContext->alsa.snd_pcm_reset                                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_reset");
30072 |     pContext->alsa.snd_device_name_hint                        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_device_name_hint");
30073 |     pContext->alsa.snd_device_name_get_hint                    =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_device_name_get_hint");
30074 |     pContext->alsa.snd_card_get_index                          =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_card_get_index");
30075 |     pContext->alsa.snd_device_name_free_hint                    =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_device_name_free_hint");
30076 |     pContext->alsa.snd_pcm_mmap_begin                          =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_mmap_begin");
30077 |     pContext->alsa.snd_pcm_mmap_commit                          =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_mmap_commit");
30078 |     pContext->alsa.snd_pcm_recover                              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_recover");
30079 |     pContext->alsa.snd_pcm_readi                                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_readi");
30080 |     pContext->alsa.snd_pcm_writei                              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_writei");
30081 |     pContext->alsa.snd_pcm_avail                                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_avail");
30082 |     pContext->alsa.snd_pcm_avail_update                        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_avail_update");
30083 |     pContext->alsa.snd_pcm_wait                                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_wait");
30084 |     pContext->alsa.snd_pcm_nonblock                            =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_nonblock");
30085 |     pContext->alsa.snd_pcm_info                                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_info");
30086 |     pContext->alsa.snd_pcm_info_sizeof                        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_info_sizeof");
30087 |     pContext->alsa.snd_pcm_info_get_name                        =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO, "snd_pcm_info_get_name");
30088 |     pContext->alsa.snd_pcm_poll_descriptors                    =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_poll_descriptors");
30089 |     pContext->alsa.snd_pcm_poll_descriptors_count              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_poll_descriptors_count");
30090 |     pContext->alsa.snd_pcm_poll_descriptors_revents            =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_pcm_poll_descriptors_revents");
30091 |     pContext->alsa.snd_config_update_free_global                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->alsa.asoundSO,
> "snd_config_update_free_global");
30092 | #else
30093 |     /* The system below is just for type safety. */
30094 |     ma_snd_pcm_open_proc                                       _snd_pcm_open                        =
> snd_pcm_open;
30095 |     ma_snd_pcm_close_proc                                       _snd_pcm_close                      =
> snd_pcm_close;
30096 |     ma_snd_pcm_hw_params_sizeof_proc                           _snd_pcm_hw_params_sizeof          =
> snd_pcm_hw_params_sizeof;
30097 |     ma_snd_pcm_hw_params_any_proc                              _snd_pcm_hw_params_any              =
> snd_pcm_hw_params_any;
30098 |     ma_snd_pcm_hw_params_set_format_proc                       _snd_pcm_hw_params_set_format      =
> snd_pcm_hw_params_set_format;
30099 |     ma_snd_pcm_hw_params_set_format_first_proc                _snd_pcm_hw_params_set_format_first =
> snd_pcm_hw_params_set_format_first;
30100 |     ma_snd_pcm_hw_params_get_format_mask_proc                  _snd_pcm_hw_params_get_format_mask  =
> snd_pcm_hw_params_get_format_mask;
30101 |     ma_snd_pcm_hw_params_set_channels_proc                      _snd_pcm_hw_params_set_channels     =
> snd_pcm_hw_params_set_channels;
30102 |     ma_snd_pcm_hw_params_set_channels_near_proc                _snd_pcm_hw_params_set_channels_near =
> snd_pcm_hw_params_set_channels_near;
30103 |     ma_snd_pcm_hw_params_set_channels_minmax_proc              _snd_pcm_hw_params_set_channels_minmax =
> snd_pcm_hw_params_set_channels_minmax;
30104 |     ma_snd_pcm_hw_params_set_rate_resample_proc                _snd_pcm_hw_params_set_rate_resample =
> snd_pcm_hw_params_set_rate_resample;
30105 |     ma_snd_pcm_hw_params_set_rate_proc                         _snd_pcm_hw_params_set_rate         =
> snd_pcm_hw_params_set_rate;
30106 |     ma_snd_pcm_hw_params_set_rate_near_proc                    _snd_pcm_hw_params_set_rate_near    =
> snd_pcm_hw_params_set_rate_near;
30107 |     ma_snd_pcm_hw_params_set_rate_minmax_proc                  _snd_pcm_hw_params_set_rate_minmax  =
> snd_pcm_hw_params_set_rate_minmax;
30108 |     ma_snd_pcm_hw_params_set_buffer_size_near_proc             _snd_pcm_hw_params_set_buffer_size_near =

```


	> snd_pcm_hw_params_set_buffer_size_near;		
30109	ma_snd_pcm_hw_params_set_periods_near_proc	_snd_pcm_hw_params_set_periods_near	=
	> snd_pcm_hw_params_set_periods_near;		
30110	ma_snd_pcm_hw_params_set_access_proc	_snd_pcm_hw_params_set_access	=
	> snd_pcm_hw_params_set_access;		
30111	ma_snd_pcm_hw_params_get_format_proc	_snd_pcm_hw_params_get_format	=
	> snd_pcm_hw_params_get_format;		
30112	ma_snd_pcm_hw_params_get_channels_proc	_snd_pcm_hw_params_get_channels	=
	> snd_pcm_hw_params_get_channels;		
30113	ma_snd_pcm_hw_params_get_channels_min_proc	_snd_pcm_hw_params_get_channels_min	=
	> snd_pcm_hw_params_get_channels_min;		
30114	ma_snd_pcm_hw_params_get_channels_max_proc	_snd_pcm_hw_params_get_channels_max	=
	> snd_pcm_hw_params_get_channels_max;		
30115	ma_snd_pcm_hw_params_get_rate_proc	_snd_pcm_hw_params_get_rate	=
	> snd_pcm_hw_params_get_rate;		
30116	ma_snd_pcm_hw_params_get_rate_min_proc	_snd_pcm_hw_params_get_rate_min	=
	> snd_pcm_hw_params_get_rate_min;		
30117	ma_snd_pcm_hw_params_get_rate_max_proc	_snd_pcm_hw_params_get_rate_max	=
	> snd_pcm_hw_params_get_rate_max;		
30118	ma_snd_pcm_hw_params_get_buffer_size_proc	_snd_pcm_hw_params_get_buffer_size	=
	> snd_pcm_hw_params_get_buffer_size;		
30119	ma_snd_pcm_hw_params_get_periods_proc	_snd_pcm_hw_params_get_periods	=
	> snd_pcm_hw_params_get_periods;		
30120	ma_snd_pcm_hw_params_get_access_proc	_snd_pcm_hw_params_get_access	=
	> snd_pcm_hw_params_get_access;		
30121	ma_snd_pcm_hw_params_test_format_proc	_snd_pcm_hw_params_test_format	=
	> snd_pcm_hw_params_test_format;		
30122	ma_snd_pcm_hw_params_test_channels_proc	_snd_pcm_hw_params_test_channels	=
	> snd_pcm_hw_params_test_channels;		
30123	ma_snd_pcm_hw_params_test_rate_proc	_snd_pcm_hw_params_test_rate	=
	> snd_pcm_hw_params_test_rate;		
30124	ma_snd_pcm_hw_params_proc	_snd_pcm_hw_params	=
	> snd_pcm_hw_params;		
30125	ma_snd_pcm_sw_params_sizeof_proc	_snd_pcm_sw_params_sizeof	=
	> snd_pcm_sw_params_sizeof;		
30126	ma_snd_pcm_sw_params_current_proc	_snd_pcm_sw_params_current	=
	> snd_pcm_sw_params_current;		
30127	ma_snd_pcm_sw_params_get_boundary_proc	_snd_pcm_sw_params_get_boundary	=
	> snd_pcm_sw_params_get_boundary;		
30128	ma_snd_pcm_sw_params_set_avail_min_proc	_snd_pcm_sw_params_set_avail_min	=
	> snd_pcm_sw_params_set_avail_min;		
30129	ma_snd_pcm_sw_params_set_start_threshold_proc	_snd_pcm_sw_params_set_start_threshold	=
	> snd_pcm_sw_params_set_start_threshold;		
30130	ma_snd_pcm_sw_params_set_stop_threshold_proc	_snd_pcm_sw_params_set_stop_threshold	=
	> snd_pcm_sw_params_set_stop_threshold;		
30131	ma_snd_pcm_sw_params_proc	_snd_pcm_sw_params	=
	> snd_pcm_sw_params;		
30132	ma_snd_pcm_format_mask_sizeof_proc	_snd_pcm_format_mask_sizeof	=
	> snd_pcm_format_mask_sizeof;		
30133	ma_snd_pcm_format_mask_test_proc	_snd_pcm_format_mask_test	=
	> snd_pcm_format_mask_test;		
30134	ma_snd_pcm_get_chmap_proc	_snd_pcm_get_chmap	=
	> snd_pcm_get_chmap;		
30135	ma_snd_pcm_state_proc	_snd_pcm_state	=
	> snd_pcm_state;		
30136	ma_snd_pcm_prepare_proc	_snd_pcm_prepare	=
	> snd_pcm_prepare;		
30137	ma_snd_pcm_start_proc	_snd_pcm_start	=
	> snd_pcm_start;		
30138	ma_snd_pcm_drop_proc	_snd_pcm_drop	=
	> snd_pcm_drop;		
30139	ma_snd_pcm_drain_proc	_snd_pcm_drain	=
	> snd_pcm_drain;		
30140	ma_snd_pcm_reset_proc	_snd_pcm_reset	=
	> snd_pcm_reset;		
30141	ma_snd_device_name_hint_proc	_snd_device_name_hint	=
	> snd_device_name_hint;		
30142	ma_snd_device_name_get_hint_proc	_snd_device_name_get_hint	=
	> snd_device_name_get_hint;		
30143	ma_snd_card_get_index_proc	_snd_card_get_index	=
	> snd_card_get_index;		
30144	ma_snd_device_name_free_hint_proc	_snd_device_name_free_hint	=
	> snd_device_name_free_hint;		
30145	ma_snd_pcm_mmap_begin_proc	_snd_pcm_mmap_begin	=
	> snd_pcm_mmap_begin;		
30146	ma_snd_pcm_mmap_commit_proc	_snd_pcm_mmap_commit	=
	> snd_pcm_mmap_commit;		
30147	ma_snd_pcm_recover_proc	_snd_pcm_recover	=
	> snd_pcm_recover;		


```

30148 |     ma_snd_pcm_readi_proc                _snd_pcm_readi                =
> snd_pcm_readi;
30149 |     ma_snd_pcm_writei_proc               _snd_pcm_writei               =
> snd_pcm_writei;
30150 |     ma_snd_pcm_avail_proc                _snd_pcm_avail                =
> snd_pcm_avail;
30151 |     ma_snd_pcm_avail_update_proc         _snd_pcm_avail_update         =
> snd_pcm_avail_update;
30152 |     ma_snd_pcm_wait_proc                 _snd_pcm_wait                  =
> snd_pcm_wait;
30153 |     ma_snd_pcm_nonblock_proc              _snd_pcm_nonblock              =
> snd_pcm_nonblock;
30154 |     ma_snd_pcm_info_proc                  _snd_pcm_info                  =
> snd_pcm_info;
30155 |     ma_snd_pcm_info_sizeof_proc           _snd_pcm_info_sizeof           =
> snd_pcm_info_sizeof;
30156 |     ma_snd_pcm_info_get_name_proc         _snd_pcm_info_get_name        =
> snd_pcm_info_get_name;
30157 |     ma_snd_pcm_poll_descriptors_proc      _snd_pcm_poll_descriptors     =
> snd_pcm_poll_descriptors;
30158 |     ma_snd_pcm_poll_descriptors_count_proc _snd_pcm_poll_descriptors_count =
> snd_pcm_poll_descriptors_count;
30159 |     ma_snd_pcm_poll_descriptors_revents_proc _snd_pcm_poll_descriptors_revents =
> snd_pcm_poll_descriptors_revents;
30160 |     ma_snd_config_update_free_global_proc _snd_config_update_free_global =
> snd_config_update_free_global;
30161 |
30162 |     pContext->alsa.snd_pcm_open           = (ma_proc)_snd_pcm_open;
30163 |     pContext->alsa.snd_pcm_close          = (ma_proc)_snd_pcm_close;
30164 |     pContext->alsa.snd_pcm_hw_params_sizeof = (ma_proc)_snd_pcm_hw_params_sizeof;
30165 |     pContext->alsa.snd_pcm_hw_params_any   = (ma_proc)_snd_pcm_hw_params_any;
30166 |     pContext->alsa.snd_pcm_hw_params_set_format = (ma_proc)_snd_pcm_hw_params_set_format;
30167 |     pContext->alsa.snd_pcm_hw_params_set_format_first =
> (ma_proc)_snd_pcm_hw_params_set_format_first;
30168 |     pContext->alsa.snd_pcm_hw_params_get_format_mask =
> (ma_proc)_snd_pcm_hw_params_get_format_mask;
30169 |     pContext->alsa.snd_pcm_hw_params_set_channels =
> (ma_proc)_snd_pcm_hw_params_set_channels;
30170 |     pContext->alsa.snd_pcm_hw_params_set_channels_near =
> (ma_proc)_snd_pcm_hw_params_set_channels_near;
30171 |     pContext->alsa.snd_pcm_hw_params_set_channels_minmax =
> (ma_proc)_snd_pcm_hw_params_set_channels_minmax;
30172 |     pContext->alsa.snd_pcm_hw_params_set_rate_resample =
> (ma_proc)_snd_pcm_hw_params_set_rate_resample;
30173 |     pContext->alsa.snd_pcm_hw_params_set_rate = (ma_proc)_snd_pcm_hw_params_set_rate;
30174 |     pContext->alsa.snd_pcm_hw_params_set_rate_near =
> (ma_proc)_snd_pcm_hw_params_set_rate_near;
30175 |     pContext->alsa.snd_pcm_hw_params_set_rate_minmax =
> (ma_proc)_snd_pcm_hw_params_set_rate_minmax;
30176 |     pContext->alsa.snd_pcm_hw_params_set_buffer_size_near =
> (ma_proc)_snd_pcm_hw_params_set_buffer_size_near;
30177 |     pContext->alsa.snd_pcm_hw_params_set_periods_near =
> (ma_proc)_snd_pcm_hw_params_set_periods_near;
30178 |     pContext->alsa.snd_pcm_hw_params_set_access = (ma_proc)_snd_pcm_hw_params_set_access;
30179 |     pContext->alsa.snd_pcm_hw_params_get_format = (ma_proc)_snd_pcm_hw_params_get_format;
30180 |     pContext->alsa.snd_pcm_hw_params_get_channels =
> (ma_proc)_snd_pcm_hw_params_get_channels;
30181 |     pContext->alsa.snd_pcm_hw_params_get_channels_min =
> (ma_proc)_snd_pcm_hw_params_get_channels_min;
30182 |     pContext->alsa.snd_pcm_hw_params_get_channels_max =
> (ma_proc)_snd_pcm_hw_params_get_channels_max;
30183 |     pContext->alsa.snd_pcm_hw_params_get_rate = (ma_proc)_snd_pcm_hw_params_get_rate;
30184 |     pContext->alsa.snd_pcm_hw_params_get_rate_min =
> (ma_proc)_snd_pcm_hw_params_get_rate_min;
30185 |     pContext->alsa.snd_pcm_hw_params_get_rate_max =
> (ma_proc)_snd_pcm_hw_params_get_rate_max;
30186 |     pContext->alsa.snd_pcm_hw_params_get_buffer_size =
> (ma_proc)_snd_pcm_hw_params_get_buffer_size;
30187 |     pContext->alsa.snd_pcm_hw_params_get_periods = (ma_proc)_snd_pcm_hw_params_get_periods;
30188 |     pContext->alsa.snd_pcm_hw_params_get_access = (ma_proc)_snd_pcm_hw_params_get_access;
30189 |     pContext->alsa.snd_pcm_hw_params_test_format = (ma_proc)_snd_pcm_hw_params_test_format;
30190 |     pContext->alsa.snd_pcm_hw_params_test_channels =
> (ma_proc)_snd_pcm_hw_params_test_channels;
30191 |     pContext->alsa.snd_pcm_hw_params_test_rate = (ma_proc)_snd_pcm_hw_params_test_rate;
30192 |     pContext->alsa.snd_pcm_hw_params       = (ma_proc)_snd_pcm_hw_params;
30193 |     pContext->alsa.snd_pcm_sw_params_sizeof = (ma_proc)_snd_pcm_sw_params_sizeof;
30194 |     pContext->alsa.snd_pcm_sw_params_current = (ma_proc)_snd_pcm_sw_params_current;
30195 |     pContext->alsa.snd_pcm_sw_params_get_boundary =
> (ma_proc)_snd_pcm_sw_params_get_boundary;

```

```

30196 |     pContext->alsa.snd_pcm_sw_params_set_avail_min      =
> (ma_proc)_snd_pcm_sw_params_set_avail_min;
30197 |     pContext->alsa.snd_pcm_sw_params_set_start_threshold =
> (ma_proc)_snd_pcm_sw_params_set_start_threshold;
30198 |     pContext->alsa.snd_pcm_sw_params_set_stop_threshold =
> (ma_proc)_snd_pcm_sw_params_set_stop_threshold;
30199 |     pContext->alsa.snd_pcm_sw_params                   = (ma_proc)_snd_pcm_sw_params;
30200 |     pContext->alsa.snd_pcm_format_mask_sizeof           = (ma_proc)_snd_pcm_format_mask_sizeof;
30201 |     pContext->alsa.snd_pcm_format_mask_test             = (ma_proc)_snd_pcm_format_mask_test;
30202 |     pContext->alsa.snd_pcm_get_chmap                    = (ma_proc)_snd_pcm_get_chmap;
30203 |     pContext->alsa.snd_pcm_state                        = (ma_proc)_snd_pcm_state;
30204 |     pContext->alsa.snd_pcm_prepare                     = (ma_proc)_snd_pcm_prepare;
30205 |     pContext->alsa.snd_pcm_start                       = (ma_proc)_snd_pcm_start;
30206 |     pContext->alsa.snd_pcm_drop                        = (ma_proc)_snd_pcm_drop;
30207 |     pContext->alsa.snd_pcm_drain                      = (ma_proc)_snd_pcm_drain;
30208 |     pContext->alsa.snd_pcm_reset                      = (ma_proc)_snd_pcm_reset;
30209 |     pContext->alsa.snd_device_name_hint                 = (ma_proc)_snd_device_name_hint;
30210 |     pContext->alsa.snd_device_name_get_hint            = (ma_proc)_snd_device_name_get_hint;
30211 |     pContext->alsa.snd_card_get_index                  = (ma_proc)_snd_card_get_index;
30212 |     pContext->alsa.snd_device_name_free_hint           = (ma_proc)_snd_device_name_free_hint;
30213 |     pContext->alsa.snd_pcm_mmap_begin                  = (ma_proc)_snd_pcm_mmap_begin;
30214 |     pContext->alsa.snd_pcm_mmap_commit                 = (ma_proc)_snd_pcm_mmap_commit;
30215 |     pContext->alsa.snd_pcm_recover                    = (ma_proc)_snd_pcm_recover;
30216 |     pContext->alsa.snd_pcm_readi                      = (ma_proc)_snd_pcm_readi;
30217 |     pContext->alsa.snd_pcm_writei                     = (ma_proc)_snd_pcm_writei;
30218 |     pContext->alsa.snd_pcm_avail                      = (ma_proc)_snd_pcm_avail;
30219 |     pContext->alsa.snd_pcm_avail_update               = (ma_proc)_snd_pcm_avail_update;
30220 |     pContext->alsa.snd_pcm_wait                      = (ma_proc)_snd_pcm_wait;
30221 |     pContext->alsa.snd_pcm_nonblock                   = (ma_proc)_snd_pcm_nonblock;
30222 |     pContext->alsa.snd_pcm_info                      = (ma_proc)_snd_pcm_info;
30223 |     pContext->alsa.snd_pcm_info_sizeof                = (ma_proc)_snd_pcm_info_sizeof;
30224 |     pContext->alsa.snd_pcm_info_get_name              = (ma_proc)_snd_pcm_info_get_name;
30225 |     pContext->alsa.snd_pcm_poll_descriptors           = (ma_proc)_snd_pcm_poll_descriptors;
30226 |     pContext->alsa.snd_pcm_poll_descriptors_count      =
> (ma_proc)_snd_pcm_poll_descriptors_count;
30227 |     pContext->alsa.snd_pcm_poll_descriptors_revents    =
> (ma_proc)_snd_pcm_poll_descriptors_revents;
30228 |     pContext->alsa.snd_config_update_free_global       = (ma_proc)_snd_config_update_free_global;
30229 | #endif
30230 |
30231 |     pContext->alsa.useVerboseDeviceEnumeration = pConfig->alsa.useVerboseDeviceEnumeration;
30232 |
30233 |     result = ma_mutex_init(&pContext->alsa.internalDeviceEnumLock);
30234 |     if (result != MA_SUCCESS) {
30235 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[ALSA] WARNING: Failed to
> initialize mutex for internal device enumeration.");
30236 |         return result;
30237 |     }
30238 |
30239 |     pCallbacks->onContextInit      = ma_context_init__alsa;
30240 |     pCallbacks->onContextUninit    = ma_context_uninit__alsa;
30241 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__alsa;
30242 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__alsa;
30243 |     pCallbacks->onDeviceInit      = ma_device_init__alsa;
30244 |     pCallbacks->onDeviceUninit    = ma_device_uninit__alsa;
30245 |     pCallbacks->onDeviceStart     = ma_device_start__alsa;
30246 |     pCallbacks->onDeviceStop      = ma_device_stop__alsa;
30247 |     pCallbacks->onDeviceRead      = ma_device_read__alsa;
30248 |     pCallbacks->onDeviceWrite     = ma_device_write__alsa;
30249 |     pCallbacks->onDeviceDataLoop  = NULL;
30250 |     pCallbacks->onDeviceDataLoopWakeup = ma_device_data_loop_wakeup__alsa;
30251 |
30252 |     return MA_SUCCESS;
30253 | }
30254 | #endif /* MA_HAS_ALSA */
30255 |
30256 |
30257 |
30258 | /*****
30259 |
30260 | PulseAudio Backend
30261 |
30262 | *****/
30263 | #ifdef MA_HAS_PULSEAUDIO
30264 | /*
30265 | The PulseAudio API, along with Apple's Core Audio, is the worst of the mainstream audio APIs. This
> is a brief description of what's going on
30266 | in the PulseAudio backend. I apologize if this gets a bit ranty for your liking - you might want to
> skip this discussion.

```

```

30267 |
30268 | PulseAudio has something they call the "Simple API", which unfortunately isn't suitable for
> miniaudio. I've not seen anywhere where it
30269 | allows you to enumerate over devices, nor does it seem to support the ability to stop and start
> streams. Looking at the documentation, it
30270 | appears as though the stream is constantly running and you prevent sound from being emitted or
> captured by simply not calling the read or
30271 | write functions. This is not a professional solution as it would be much better to *actually* stop
> the underlying stream. Perhaps the
30272 | simple API has some smarts to do this automatically, but I'm not sure. Another limitation with the
> simple API is that it seems inefficient
30273 | when you want to have multiple streams to a single context. For these reasons, miniaudio is not
> using the simple API.
30274 |
30275 | Since we're not using the simple API, we're left with the asynchronous API as our only other option.
> And boy, is this where it starts to
30276 | get fun, and I don't mean that in a good way...
30277 |
30278 | The problems start with the very name of the API - "asynchronous". Yes, this is an asynchronous
> oriented API which means your commands
30279 | don't immediately take effect. You instead need to issue your commands, and then wait for them to
> complete. The waiting mechanism is
30280 | enabled through the use of a "main loop". In the asynchronous API you cannot get away from the main
> loop, and the main loop is where almost
30281 | all of PulseAudio's problems stem from.
30282 |
30283 | When you first initialize PulseAudio you need an object referred to as "main loop". You can
> implement this yourself by defining your own
30284 | vtable, but it's much easier to just use one of the built-in main loop implementations. There's two
> generic implementations called
30285 | pa_mainloop and pa_threaded_mainloop, and another implementation specific to GLib called
> pa_glib_mainloop. We're using pa_threaded_mainloop
30286 | because it simplifies management of the worker thread. The idea of the main loop object is pretty
> self explanatory - you're supposed to use
30287 | it to implement a worker thread which runs in a loop. The main loop is where operations are actually
> executed.
30288 |
30289 | To initialize the main loop, you just use `pa_threaded_mainloop_new()`. This is the first function
> you'll call. You can then get a pointer
30290 | to the vtable with `pa_threaded_mainloop_get_api()` (the main loop vtable is called
> `pa_mainloop_api`). Again, you can bypass the threaded
30291 | main loop object entirely and just implement `pa_mainloop_api` directly, but there's no need for it
> unless you're doing something extremely
30292 | specialized such as if you want to integrate it into your application's existing main loop
> infrastructure.
30293 |
30294 | (EDIT 2021-01-26: miniaudio is no longer using `pa_threaded_mainloop` due to this issue:
> https://github.com/mackron/miniaudio/issues/262.
30295 | It is now using `pa_mainloop` which turns out to be a simpler solution anyway. The rest of this rant
> still applies, however.)
30296 |
30297 | Once you have your main loop vtable (the `pa_mainloop_api` object) you can create the PulseAudio
> context. This is very similar to
30298 | miniaudio's context and they map to each other quite well. You have one context to many streams,
> which is basically the same as miniaudio's
30299 | one `ma_context` to many `ma_device`s. Here's where it starts to get annoying, however. When you
> first create the PulseAudio context, which
30300 | is done with `pa_context_new()`, it's not actually connected to anything. When you connect, you call
> `pa_context_connect()`. However, if
30301 | you remember, PulseAudio is an asynchronous API. That means you cannot just assume the context is
> connected after `pa_context_connect()`
30302 | has returned. You instead need to wait for it to connect. To do this, you need to either wait for a
> callback to get fired, which you can
30303 | set with `pa_context_set_state_callback()`, or you can continuously poll the context's state. Either
> way, you need to run this in a loop.
30304 | All objects from here out are created from the context, and, I believe, you can't be creating these
> objects until the context is connected.
30305 | This waiting loop is therefore unavoidable. In order for the waiting to ever complete, however, the
> main loop needs to be running. Before
30306 | attempting to connect the context, the main loop needs to be started with
> `pa_threaded_mainloop_start()`.
30307 |
30308 | The reason for this asynchronous design is to support cases where you're connecting to a remote
> server, say through a local network or an
30309 | internet connection. However, the *VAST* majority of cases don't involve this at all - they just
> connect to a local "server" running on the
30310 | host machine. The fact that this would be the default rather than making `pa_context_connect()`
> synchronous tends to boggle the mind.
30311 |

```

```

30312 | Once the context has been created and connected you can start creating a stream. A PulseAudio stream
> | is analogous to miniaudio's device.
30313 | The initialization of a stream is fairly standard - you configure some attributes (analogous to
> | miniaudio's device config) and then call
30314 | `pa_stream_new()` to actually create it. Here is where we start to get into "operations". When
> | configuring the stream, you can get
30315 | information about the source (such as sample format, sample rate, etc.), however it's not
> | synchronous. Instead, a `pa_operation` object
30316 | is returned from `pa_context_get_source_info_by_name()` (capture) or
> | `pa_context_get_sink_info_by_name()` (playback). Then, you need to
30317 | run a loop (again!) to wait for the operation to complete which you can determine via a callback or
> | polling, just like we did with the
30318 | context. Then, as an added bonus, you need to decrement the reference counter of the `pa_operation`
> | object to ensure memory is cleaned up.
30319 | All of that just to retrieve basic information about a device!
30320 |
30321 | Once the basic information about the device has been retrieved, miniaudio can now create the stream
> | with `ma_stream_new()`. Like the
30322 | context, this needs to be connected. But we need to be careful here, because we're now about to
> | introduce one of the most horrific design
30323 | choices in PulseAudio.
30324 |
30325 | PulseAudio allows you to specify a callback that is fired when data can be written to or read from a
> | stream. The language is important here
30326 | because PulseAudio takes it literally, specifically the "can be". You would think these callbacks
> | would be appropriate as the place for
30327 | writing and reading data to and from the stream, and that would be right, except when it's not. When
> | you initialize the stream, you can
30328 | set a flag that tells PulseAudio to not start the stream automatically. This is required because
> | miniaudio does not auto-start devices
30329 | straight after initialization - you need to call `ma_device_start()` manually. The problem is that
> | even when this flag is specified,
30330 | PulseAudio will immediately fire its write or read callback. This is *technically* correct (based on
> | the wording in the documentation)
30331 | because indeed, data *can* be written at this point. The problem is that it's not *practical*. It
> | makes sense that the write/read callback
30332 | would be where a program will want to write or read data to or from the stream, but when it's called
> | before the application has even
30333 | requested that the stream be started, it's just not practical because the program probably isn't
> | ready for any kind of data delivery at
30334 | that point (it may still need to load files or whatnot). Instead, this callback should only be fired
> | when the application requests the
30335 | stream be started which is how it works with literally *every* other callback-based audio API. Since
> | miniaudio forbids firing of the data
30336 | callback until the device has been started (as it should be with *all* callback based APIs), logic
> | needs to be added to ensure miniaudio
30337 | doesn't just blindly fire the application-defined data callback from within the PulseAudio callback
> | before the stream has actually been
30338 | started. The device state is used for this - if the state is anything other than
> | `ma_device_state_starting` or `ma_device_state_started`, the main data
30339 | callback is not fired.
30340 |
30341 | This, unfortunately, is not the end of the problems with the PulseAudio write callback. Any normal
> | callback based audio API will
30342 | continuously fire the callback at regular intervals based on the size of the internal buffer. This
> | will only ever be fired when the device
30343 | is running, and will be fired regardless of whether or not the user actually wrote anything to the
> | device/stream. This not the case in
30344 | PulseAudio. In PulseAudio, the data callback will *only* be called if you wrote something to it
> | previously. That means, if you don't call
30345 | `pa_stream_write()`, the callback will not get fired. On the surface you wouldn't think this would
> | matter because you should be always
30346 | writing data, and if you don't have anything to write, just write silence. That's fine until you
> | want to drain the stream. You see, if
30347 | you're continuously writing data to the stream, the stream will never get drained! That means in
> | order to drain the stream, you need to
30348 | *not* write data to it! But remember, when you don't write data to the stream, the callback won't
> | get fired again! Why is draining
30349 | important? Because that's how we've defined stopping to work in miniaudio. In miniaudio, stopping
> | the device requires it to be drained
30350 | before returning from ma_device_stop(). So we've stopped the device, which requires us to drain, but
> | draining requires us to *not* write
30351 | data to the stream (or else it won't ever complete draining), but not writing to the stream means
> | the callback won't get fired again!
30352 |
30353 | This becomes a problem when stopping and then restarting the device. When the device is stopped,
> | it's drained, which requires us to *not*
30354 | write anything to the stream. But then, since we didn't write anything to it, the write callback
> | will *never* get called again if we just

```

```

30355 | resume the stream naively. This means that starting the stream requires us to write data to the
> stream from outside the callback. This
30356 | disconnect is something PulseAudio has got seriously wrong - there should only ever be a single
> source of data delivery, that being the
30357 | callback. (I have tried using `pa_stream_flush()` to trigger the write callback to fire, but this
> just doesn't work for some reason.)
30358 |
30359 | Once you've created the stream, you need to connect it which involves the whole waiting procedure.
> This is the same process as the context,
30360 | only this time you'll poll for the state with `pa_stream_get_status()`. The starting and stopping of
> a streaming is referred to as
30361 | "corking" in PulseAudio. The analogy is corking a barrel. To start the stream, you uncork it, to
> stop it you cork it. Personally I think
30362 | it's silly - why would you not just call it "starting" and "stopping" like any other normal audio
> API? Anyway, the act of corking is, you
30363 | guessed it, asynchronous. This means you'll need our waiting loop as usual. Again, why this
> asynchronous design is the default is
30364 | absolutely beyond me. Would it really be that hard to just make it run synchronously?
30365 |
30366 | Teardown is pretty simple (what?!). It's just a matter of calling the relevant `_unref()` function
> on each object in reverse order that
30367 | they were initialized in.
30368 |
30369 | That's about it from the PulseAudio side. A bit ranty, I know, but they really need to fix that main
> loop and callback system. They're
30370 | embarrassingly unpractical. The main loop thing is an easy fix - have synchronous versions of all
> APIs. If an application wants these to
30371 | run asynchronously, they can execute them in a separate thread themselves. The desire to run these
> asynchronously is such a niche
30372 | requirement - it makes no sense to make it the default. The stream write callback needs to be
> change, or an alternative provided, that is
30373 | constantly fired, regardless of whether or not `pa_stream_write()` has been called, and it needs to
> take a pointer to a buffer as a
30374 | parameter which the program just writes to directly rather than having to call
> `pa_stream_writable_size()` and `pa_stream_write()`. These
30375 | changes alone will change PulseAudio from one of the worst audio APIs to one of the best.
30376 | */
30377 |
30378 |
30379 | /*
30380 | It is assumed pulseaudio.h is available when linking at compile time. When linking at compile time,
> we use the declarations in the header
30381 | to check for type safety. We cannot do this when linking at run time because the header might not be
> available.
30382 | */
30383 | #ifdef MA_NO_RUNTIME_LINKING
30384 |
30385 | /* pulseaudio.h marks some functions with "inline" which isn't always supported. Need to emulate it.
> */
30386 | #if !defined(__cplusplus)
30387 |     #if defined(__STRICT_ANSI__)
30388 |         #if !defined(inline)
30389 |             #define inline __inline__ __attribute__((always_inline))
30390 |             #define MA_INLINE_DEFINED
30391 |         #endif
30392 |     #endif
30393 | #endif
30394 | #include <pulse/pulseaudio.h>
30395 | #if defined(MA_INLINE_DEFINED)
30396 |     #undef inline
30397 |     #undef MA_INLINE_DEFINED
30398 | #endif
30399 |
30400 | #define MA_PA_OK PA_OK
30401 | #define MA_PA_ERR_ACCESS PA_ERR_ACCESS
30402 | #define MA_PA_ERR_INVALID PA_ERR_INVALID
30403 | #define MA_PA_ERR_NOENTITY PA_ERR_NOENTITY
30404 | #define MA_PA_ERR_NOTSUPPORTED PA_ERR_NOTSUPPORTED
30405 |
30406 | #define MA_PA_CHANNELS_MAX PA_CHANNELS_MAX
30407 | #define MA_PA_RATE_MAX PA_RATE_MAX
30408 |
30409 | typedef pa_context_flags_t ma_pa_context_flags_t;
30410 | #define MA_PA_CONTEXT_NOFLAGS PA_CONTEXT_NOFLAGS
30411 | #define MA_PA_CONTEXT_NOAUTOSPAWN PA_CONTEXT_NOAUTOSPAWN
30412 | #define MA_PA_CONTEXT_NOFAIL PA_CONTEXT_NOFAIL
30413 |
30414 | typedef pa_stream_flags_t ma_pa_stream_flags_t;
30415 | #define MA_PA_STREAM_NOFLAGS PA_STREAM_NOFLAGS

```

30416	#define MA_PA_STREAM_START_CORKED	PA_STREAM_START_CORKED
30417	#define MA_PA_STREAM_INTERPOLATE_TIMING	PA_STREAM_INTERPOLATE_TIMING
30418	#define MA_PA_STREAM_NOT_MONOTONIC	PA_STREAM_NOT_MONOTONIC
30419	#define MA_PA_STREAM_AUTO_TIMING_UPDATE	PA_STREAM_AUTO_TIMING_UPDATE
30420	#define MA_PA_STREAM_NO_REMAP_CHANNELS	PA_STREAM_NO_REMAP_CHANNELS
30421	#define MA_PA_STREAM_NO_REMIX_CHANNELS	PA_STREAM_NO_REMIX_CHANNELS
30422	#define MA_PA_STREAM_FIX_FORMAT	PA_STREAM_FIX_FORMAT
30423	#define MA_PA_STREAM_FIX_RATE	PA_STREAM_FIX_RATE
30424	#define MA_PA_STREAM_FIX_CHANNELS	PA_STREAM_FIX_CHANNELS
30425	#define MA_PA_STREAM_DONT_MOVE	PA_STREAM_DONT_MOVE
30426	#define MA_PA_STREAM_VARIABLE_RATE	PA_STREAM_VARIABLE_RATE
30427	#define MA_PA_STREAM_PEAK_DETECT	PA_STREAM_PEAK_DETECT
30428	#define MA_PA_STREAM_START_MUTED	PA_STREAM_START_MUTED
30429	#define MA_PA_STREAM_ADJUST_LATENCY	PA_STREAM_ADJUST_LATENCY
30430	#define MA_PA_STREAM_EARLY_REQUESTS	PA_STREAM_EARLY_REQUESTS
30431	#define MA_PA_STREAM_DONT_INHIBIT_AUTO_SUSPEND	PA_STREAM_DONT_INHIBIT_AUTO_SUSPEND
30432	#define MA_PA_STREAM_START_UNMUTED	PA_STREAM_START_UNMUTED
30433	#define MA_PA_STREAM_FAIL_ON_SUSPEND	PA_STREAM_FAIL_ON_SUSPEND
30434	#define MA_PA_STREAM_RELATIVE_VOLUME	PA_STREAM_RELATIVE_VOLUME
30435	#define MA_PA_STREAM_PASSTHROUGH	PA_STREAM_PASSTHROUGH
30436		
30437	typedef pa_sink_flags_t ma_pa_sink_flags_t;	
30438	#define MA_PA_SINK_NOFLAGS	PA_SINK_NOFLAGS
30439	#define MA_PA_SINK_HW_VOLUME_CTRL	PA_SINK_HW_VOLUME_CTRL
30440	#define MA_PA_SINK_LATENCY	PA_SINK_LATENCY
30441	#define MA_PA_SINK_HARDWARE	PA_SINK_HARDWARE
30442	#define MA_PA_SINK_NETWORK	PA_SINK_NETWORK
30443	#define MA_PA_SINK_HW_MUTE_CTRL	PA_SINK_HW_MUTE_CTRL
30444	#define MA_PA_SINK_DECIBEL_VOLUME	PA_SINK_DECIBEL_VOLUME
30445	#define MA_PA_SINK_FLAT_VOLUME	PA_SINK_FLAT_VOLUME
30446	#define MA_PA_SINK_DYNAMIC_LATENCY	PA_SINK_DYNAMIC_LATENCY
30447	#define MA_PA_SINK_SET_FORMATS	PA_SINK_SET_FORMATS
30448		
30449	typedef pa_source_flags_t ma_pa_source_flags_t;	
30450	#define MA_PA_SOURCE_NOFLAGS	PA_SOURCE_NOFLAGS
30451	#define MA_PA_SOURCE_HW_VOLUME_CTRL	PA_SOURCE_HW_VOLUME_CTRL
30452	#define MA_PA_SOURCE_LATENCY	PA_SOURCE_LATENCY
30453	#define MA_PA_SOURCE_HARDWARE	PA_SOURCE_HARDWARE
30454	#define MA_PA_SOURCE_NETWORK	PA_SOURCE_NETWORK
30455	#define MA_PA_SOURCE_HW_MUTE_CTRL	PA_SOURCE_HW_MUTE_CTRL
30456	#define MA_PA_SOURCE_DECIBEL_VOLUME	PA_SOURCE_DECIBEL_VOLUME
30457	#define MA_PA_SOURCE_DYNAMIC_LATENCY	PA_SOURCE_DYNAMIC_LATENCY
30458	#define MA_PA_SOURCE_FLAT_VOLUME	PA_SOURCE_FLAT_VOLUME
30459		
30460	typedef pa_context_state_t ma_pa_context_state_t;	
30461	#define MA_PA_CONTEXT_UNCONNECTED	PA_CONTEXT_UNCONNECTED
30462	#define MA_PA_CONTEXT_CONNECTING	PA_CONTEXT_CONNECTING
30463	#define MA_PA_CONTEXT_AUTHORIZING	PA_CONTEXT_AUTHORIZING
30464	#define MA_PA_CONTEXT_SETTING_NAME	PA_CONTEXT_SETTING_NAME
30465	#define MA_PA_CONTEXT_READY	PA_CONTEXT_READY
30466	#define MA_PA_CONTEXT_FAILED	PA_CONTEXT_FAILED
30467	#define MA_PA_CONTEXT_TERMINATED	PA_CONTEXT_TERMINATED
30468		
30469	typedef pa_stream_state_t ma_pa_stream_state_t;	
30470	#define MA_PA_STREAM_UNCONNECTED	PA_STREAM_UNCONNECTED
30471	#define MA_PA_STREAM_CREATING	PA_STREAM_CREATING
30472	#define MA_PA_STREAM_READY	PA_STREAM_READY
30473	#define MA_PA_STREAM_FAILED	PA_STREAM_FAILED
30474	#define MA_PA_STREAM_TERMINATED	PA_STREAM_TERMINATED
30475		
30476	typedef pa_operation_state_t ma_pa_operation_state_t;	
30477	#define MA_PA_OPERATION_RUNNING	PA_OPERATION_RUNNING
30478	#define MA_PA_OPERATION_DONE	PA_OPERATION_DONE
30479	#define MA_PA_OPERATION_CANCELLED	PA_OPERATION_CANCELLED
30480		
30481	typedef pa_sink_state_t ma_pa_sink_state_t;	
30482	#define MA_PA_SINK_INVALID_STATE	PA_SINK_INVALID_STATE
30483	#define MA_PA_SINK_RUNNING	PA_SINK_RUNNING
30484	#define MA_PA_SINK_IDLE	PA_SINK_IDLE
30485	#define MA_PA_SINK_SUSPENDED	PA_SINK_SUSPENDED
30486		
30487	typedef pa_source_state_t ma_pa_source_state_t;	
30488	#define MA_PA_SOURCE_INVALID_STATE	PA_SOURCE_INVALID_STATE
30489	#define MA_PA_SOURCE_RUNNING	PA_SOURCE_RUNNING
30490	#define MA_PA_SOURCE_IDLE	PA_SOURCE_IDLE
30491	#define MA_PA_SOURCE_SUSPENDED	PA_SOURCE_SUSPENDED
30492		
30493	typedef pa_seek_mode_t ma_pa_seek_mode_t;	
30494	#define MA_PA_SEEK_RELATIVE	PA_SEEK_RELATIVE


```

30495 | #define MA_PA_SEEK_ABSOLUTE          PA_SEEK_ABSOLUTE
30496 | #define MA_PA_SEEK_RELATIVE_ON_READ  PA_SEEK_RELATIVE_ON_READ
30497 | #define MA_PA_SEEK_RELATIVE_END      PA_SEEK_RELATIVE_END
30498 |
30499 | typedef pa_channel_position_t ma_pa_channel_position_t;
30500 | #define MA_PA_CHANNEL_POSITION_INVALID PA_CHANNEL_POSITION_INVALID
30501 | #define MA_PA_CHANNEL_POSITION_MONO   PA_CHANNEL_POSITION_MONO
30502 | #define MA_PA_CHANNEL_POSITION_FRONT_LEFT PA_CHANNEL_POSITION_FRONT_LEFT
30503 | #define MA_PA_CHANNEL_POSITION_FRONT_RIGHT PA_CHANNEL_POSITION_FRONT_RIGHT
30504 | #define MA_PA_CHANNEL_POSITION_FRONT_CENTER PA_CHANNEL_POSITION_FRONT_CENTER
30505 | #define MA_PA_CHANNEL_POSITION_REAR_CENTER PA_CHANNEL_POSITION_REAR_CENTER
30506 | #define MA_PA_CHANNEL_POSITION_REAR_LEFT PA_CHANNEL_POSITION_REAR_LEFT
30507 | #define MA_PA_CHANNEL_POSITION_REAR_RIGHT PA_CHANNEL_POSITION_REAR_RIGHT
30508 | #define MA_PA_CHANNEL_POSITION_LFE     PA_CHANNEL_POSITION_LFE
30509 | #define MA_PA_CHANNEL_POSITION_FRONT_LEFT_OF_CENTER PA_CHANNEL_POSITION_FRONT_LEFT_OF_CENTER
30510 | #define MA_PA_CHANNEL_POSITION_FRONT_RIGHT_OF_CENTER PA_CHANNEL_POSITION_FRONT_RIGHT_OF_CENTER
30511 | #define MA_PA_CHANNEL_POSITION_SIDE_LEFT PA_CHANNEL_POSITION_SIDE_LEFT
30512 | #define MA_PA_CHANNEL_POSITION_SIDE_RIGHT PA_CHANNEL_POSITION_SIDE_RIGHT
30513 | #define MA_PA_CHANNEL_POSITION_AUX0     PA_CHANNEL_POSITION_AUX0
30514 | #define MA_PA_CHANNEL_POSITION_AUX1     PA_CHANNEL_POSITION_AUX1
30515 | #define MA_PA_CHANNEL_POSITION_AUX2     PA_CHANNEL_POSITION_AUX2
30516 | #define MA_PA_CHANNEL_POSITION_AUX3     PA_CHANNEL_POSITION_AUX3
30517 | #define MA_PA_CHANNEL_POSITION_AUX4     PA_CHANNEL_POSITION_AUX4
30518 | #define MA_PA_CHANNEL_POSITION_AUX5     PA_CHANNEL_POSITION_AUX5
30519 | #define MA_PA_CHANNEL_POSITION_AUX6     PA_CHANNEL_POSITION_AUX6
30520 | #define MA_PA_CHANNEL_POSITION_AUX7     PA_CHANNEL_POSITION_AUX7
30521 | #define MA_PA_CHANNEL_POSITION_AUX8     PA_CHANNEL_POSITION_AUX8
30522 | #define MA_PA_CHANNEL_POSITION_AUX9     PA_CHANNEL_POSITION_AUX9
30523 | #define MA_PA_CHANNEL_POSITION_AUX10    PA_CHANNEL_POSITION_AUX10
30524 | #define MA_PA_CHANNEL_POSITION_AUX11    PA_CHANNEL_POSITION_AUX11
30525 | #define MA_PA_CHANNEL_POSITION_AUX12    PA_CHANNEL_POSITION_AUX12
30526 | #define MA_PA_CHANNEL_POSITION_AUX13    PA_CHANNEL_POSITION_AUX13
30527 | #define MA_PA_CHANNEL_POSITION_AUX14    PA_CHANNEL_POSITION_AUX14
30528 | #define MA_PA_CHANNEL_POSITION_AUX15    PA_CHANNEL_POSITION_AUX15
30529 | #define MA_PA_CHANNEL_POSITION_AUX16    PA_CHANNEL_POSITION_AUX16
30530 | #define MA_PA_CHANNEL_POSITION_AUX17    PA_CHANNEL_POSITION_AUX17
30531 | #define MA_PA_CHANNEL_POSITION_AUX18    PA_CHANNEL_POSITION_AUX18
30532 | #define MA_PA_CHANNEL_POSITION_AUX19    PA_CHANNEL_POSITION_AUX19
30533 | #define MA_PA_CHANNEL_POSITION_AUX20    PA_CHANNEL_POSITION_AUX20
30534 | #define MA_PA_CHANNEL_POSITION_AUX21    PA_CHANNEL_POSITION_AUX21
30535 | #define MA_PA_CHANNEL_POSITION_AUX22    PA_CHANNEL_POSITION_AUX22
30536 | #define MA_PA_CHANNEL_POSITION_AUX23    PA_CHANNEL_POSITION_AUX23
30537 | #define MA_PA_CHANNEL_POSITION_AUX24    PA_CHANNEL_POSITION_AUX24
30538 | #define MA_PA_CHANNEL_POSITION_AUX25    PA_CHANNEL_POSITION_AUX25
30539 | #define MA_PA_CHANNEL_POSITION_AUX26    PA_CHANNEL_POSITION_AUX26
30540 | #define MA_PA_CHANNEL_POSITION_AUX27    PA_CHANNEL_POSITION_AUX27
30541 | #define MA_PA_CHANNEL_POSITION_AUX28    PA_CHANNEL_POSITION_AUX28
30542 | #define MA_PA_CHANNEL_POSITION_AUX29    PA_CHANNEL_POSITION_AUX29
30543 | #define MA_PA_CHANNEL_POSITION_AUX30    PA_CHANNEL_POSITION_AUX30
30544 | #define MA_PA_CHANNEL_POSITION_AUX31    PA_CHANNEL_POSITION_AUX31
30545 | #define MA_PA_CHANNEL_POSITION_TOP_CENTER PA_CHANNEL_POSITION_TOP_CENTER
30546 | #define MA_PA_CHANNEL_POSITION_TOP_FRONT_LEFT PA_CHANNEL_POSITION_TOP_FRONT_LEFT
30547 | #define MA_PA_CHANNEL_POSITION_TOP_FRONT_RIGHT PA_CHANNEL_POSITION_TOP_FRONT_RIGHT
30548 | #define MA_PA_CHANNEL_POSITION_TOP_FRONT_CENTER PA_CHANNEL_POSITION_TOP_FRONT_CENTER
30549 | #define MA_PA_CHANNEL_POSITION_TOP_REAR_LEFT PA_CHANNEL_POSITION_TOP_REAR_LEFT
30550 | #define MA_PA_CHANNEL_POSITION_TOP_REAR_RIGHT PA_CHANNEL_POSITION_TOP_REAR_RIGHT
30551 | #define MA_PA_CHANNEL_POSITION_TOP_REAR_CENTER PA_CHANNEL_POSITION_TOP_REAR_CENTER
30552 | #define MA_PA_CHANNEL_POSITION_LEFT     PA_CHANNEL_POSITION_LEFT
30553 | #define MA_PA_CHANNEL_POSITION_RIGHT    PA_CHANNEL_POSITION_RIGHT
30554 | #define MA_PA_CHANNEL_POSITION_CENTER   PA_CHANNEL_POSITION_CENTER
30555 | #define MA_PA_CHANNEL_POSITION_SUBWOOFER PA_CHANNEL_POSITION_SUBWOOFER
30556 |
30557 | typedef pa_channel_map_def_t ma_pa_channel_map_def_t;
30558 | #define MA_PA_CHANNEL_MAP_AIFF          PA_CHANNEL_MAP_AIFF
30559 | #define MA_PA_CHANNEL_MAP_ALSA          PA_CHANNEL_MAP_ALSA
30560 | #define MA_PA_CHANNEL_MAP_AUX           PA_CHANNEL_MAP_AUX
30561 | #define MA_PA_CHANNEL_MAP_WAVEEX        PA_CHANNEL_MAP_WAVEEX
30562 | #define MA_PA_CHANNEL_MAP_OSS           PA_CHANNEL_MAP_OSS
30563 | #define MA_PA_CHANNEL_MAP_DEFAULT       PA_CHANNEL_MAP_DEFAULT
30564 |
30565 | typedef pa_sample_format_t ma_pa_sample_format_t;
30566 | #define MA_PA_SAMPLE_INVALID            PA_SAMPLE_INVALID
30567 | #define MA_PA_SAMPLE_U8                 PA_SAMPLE_U8
30568 | #define MA_PA_SAMPLE_ALAW               PA_SAMPLE_ALAW
30569 | #define MA_PA_SAMPLE_ULAW               PA_SAMPLE_ULAW
30570 | #define MA_PA_SAMPLE_S16LE              PA_SAMPLE_S16LE
30571 | #define MA_PA_SAMPLE_S16BE              PA_SAMPLE_S16BE
30572 | #define MA_PA_SAMPLE_FLOAT32LE          PA_SAMPLE_FLOAT32LE
30573 | #define MA_PA_SAMPLE_FLOAT32BE          PA_SAMPLE_FLOAT32BE

```



```

30574 | #define MA_PA_SAMPLE_S32LE          PA_SAMPLE_S32LE
30575 | #define MA_PA_SAMPLE_S32BE          PA_SAMPLE_S32BE
30576 | #define MA_PA_SAMPLE_S24LE          PA_SAMPLE_S24LE
30577 | #define MA_PA_SAMPLE_S24BE          PA_SAMPLE_S24BE
30578 | #define MA_PA_SAMPLE_S24_32LE        PA_SAMPLE_S24_32LE
30579 | #define MA_PA_SAMPLE_S24_32BE        PA_SAMPLE_S24_32BE
30580 |
30581 | typedef pa_mainloop                ma_pa_mainloop;
30582 | typedef pa_threaded_mainloop        ma_pa_threaded_mainloop;
30583 | typedef pa_mainloop_api             ma_pa_mainloop_api;
30584 | typedef pa_context                  ma_pa_context;
30585 | typedef pa_operation                 ma_pa_operation;
30586 | typedef pa_stream                   ma_pa_stream;
30587 | typedef pa_spawn_api                ma_pa_spawn_api;
30588 | typedef pa_buffer_attr              ma_pa_buffer_attr;
30589 | typedef pa_channel_map              ma_pa_channel_map;
30590 | typedef pa_cvolume                  ma_pa_cvolume;
30591 | typedef pa_sample_spec              ma_pa_sample_spec;
30592 | typedef pa_sink_info                ma_pa_sink_info;
30593 | typedef pa_source_info              ma_pa_source_info;
30594 |
30595 | typedef pa_context_notify_cb_t      ma_pa_context_notify_cb_t;
30596 | typedef pa_sink_info_cb_t           ma_pa_sink_info_cb_t;
30597 | typedef pa_source_info_cb_t         ma_pa_source_info_cb_t;
30598 | typedef pa_stream_success_cb_t      ma_pa_stream_success_cb_t;
30599 | typedef pa_stream_request_cb_t      ma_pa_stream_request_cb_t;
30600 | typedef pa_stream_notify_cb_t       ma_pa_stream_notify_cb_t;
30601 | typedef pa_free_cb_t               ma_pa_free_cb_t;
30602 | #else
30603 | #define MA_PA_OK                     0
30604 | #define MA_PA_ERR_ACCESS              1
30605 | #define MA_PA_ERR_INVALID            2
30606 | #define MA_PA_ERR_NOENTITY           5
30607 | #define MA_PA_ERR_NOTSUPPORTED       19
30608 |
30609 | #define MA_PA_CHANNELS_MAX           32
30610 | #define MA_PA_RATE_MAX               384000
30611 |
30612 | typedef int ma_pa_context_flags_t;
30613 | #define MA_PA_CONTEXT_NOFLAGS        0x00000000
30614 | #define MA_PA_CONTEXT_NOAUTOSPAWN    0x00000001
30615 | #define MA_PA_CONTEXT_NOFAIL        0x00000002
30616 |
30617 | typedef int ma_pa_stream_flags_t;
30618 | #define MA_PA_STREAM_NOFLAGS         0x00000000
30619 | #define MA_PA_STREAM_START_CORKED    0x00000001
30620 | #define MA_PA_STREAM_INTERPOLATE_TIMING 0x00000002
30621 | #define MA_PA_STREAM_NOT_MONOTONIC  0x00000004
30622 | #define MA_PA_STREAM_AUTO_TIMING_UPDATE 0x00000008
30623 | #define MA_PA_STREAM_NO_REMAP_CHANNELS 0x00000010
30624 | #define MA_PA_STREAM_NO_REMIX_CHANNELS 0x00000020
30625 | #define MA_PA_STREAM_FIX_FORMAT      0x00000040
30626 | #define MA_PA_STREAM_FIX_RATE        0x00000080
30627 | #define MA_PA_STREAM_FIX_CHANNELS    0x00000100
30628 | #define MA_PA_STREAM_DONT_MOVE        0x00000200
30629 | #define MA_PA_STREAM_VARIABLE_RATE    0x00000400
30630 | #define MA_PA_STREAM_PEAK_DETECT      0x00000800
30631 | #define MA_PA_STREAM_START_MUTED      0x00001000
30632 | #define MA_PA_STREAM_ADJUST_LATENCY   0x00002000
30633 | #define MA_PA_STREAM_EARLY_REQUESTS   0x00004000
30634 | #define MA_PA_STREAM_DONT_INHIBIT_AUTO_SUSPEND 0x00008000
30635 | #define MA_PA_STREAM_START_UNMUTED    0x00010000
30636 | #define MA_PA_STREAM_FAIL_ON_SUSPEND  0x00020000
30637 | #define MA_PA_STREAM_RELATIVE_VOLUME  0x00040000
30638 | #define MA_PA_STREAM_PASSTHROUGH      0x00080000
30639 |
30640 | typedef int ma_pa_sink_flags_t;
30641 | #define MA_PA_SINK_NOFLAGS            0x00000000
30642 | #define MA_PA_SINK_HW_VOLUME_CTRL     0x00000001
30643 | #define MA_PA_SINK_LATENCY            0x00000002
30644 | #define MA_PA_SINK_HARDWARE           0x00000004
30645 | #define MA_PA_SINK_NETWORK            0x00000008
30646 | #define MA_PA_SINK_HW_MUTE_CTRL       0x00000010
30647 | #define MA_PA_SINK_DECIBEL_VOLUME     0x00000020
30648 | #define MA_PA_SINK_FLAT_VOLUME        0x00000040
30649 | #define MA_PA_SINK_DYNAMIC_LATENCY    0x00000080
30650 | #define MA_PA_SINK_SET_FORMATS        0x00000100
30651 |
30652 | typedef int ma_pa_source_flags_t;

```

```

30653 | #define MA_PA_SOURCE_NOFLAGS                0x00000000
30654 | #define MA_PA_SOURCE_HW_VOLUME_CTRL        0x00000001
30655 | #define MA_PA_SOURCE_LATENCY                0x00000002
30656 | #define MA_PA_SOURCE_HARDWARE               0x00000004
30657 | #define MA_PA_SOURCE_NETWORK                0x00000008
30658 | #define MA_PA_SOURCE_HW_MUTE_CTRL           0x00000010
30659 | #define MA_PA_SOURCE_DECIBEL_VOLUME         0x00000020
30660 | #define MA_PA_SOURCE_DYNAMIC_LATENCY        0x00000040
30661 | #define MA_PA_SOURCE_FLAT_VOLUME            0x00000080
30662 |
30663 | typedef int ma_pa_context_state_t;
30664 | #define MA_PA_CONTEXT_UNCONNECTED           0
30665 | #define MA_PA_CONTEXT_CONNECTING            1
30666 | #define MA_PA_CONTEXT_AUTHORIZING           2
30667 | #define MA_PA_CONTEXT_SETTING_NAME          3
30668 | #define MA_PA_CONTEXT_READY                 4
30669 | #define MA_PA_CONTEXT_FAILED                5
30670 | #define MA_PA_CONTEXT_TERMINATED            6
30671 |
30672 | typedef int ma_pa_stream_state_t;
30673 | #define MA_PA_STREAM_UNCONNECTED            0
30674 | #define MA_PA_STREAM_CREATING               1
30675 | #define MA_PA_STREAM_READY                  2
30676 | #define MA_PA_STREAM_FAILED                 3
30677 | #define MA_PA_STREAM_TERMINATED             4
30678 |
30679 | typedef int ma_pa_operation_state_t;
30680 | #define MA_PA_OPERATION_RUNNING             0
30681 | #define MA_PA_OPERATION_DONE                 1
30682 | #define MA_PA_OPERATION_CANCELLED           2
30683 |
30684 | typedef int ma_pa_sink_state_t;
30685 | #define MA_PA_SINK_INVALID_STATE            -1
30686 | #define MA_PA_SINK_RUNNING                  0
30687 | #define MA_PA_SINK_IDLE                     1
30688 | #define MA_PA_SINK_SUSPENDED                2
30689 |
30690 | typedef int ma_pa_source_state_t;
30691 | #define MA_PA_SOURCE_INVALID_STATE           -1
30692 | #define MA_PA_SOURCE_RUNNING                0
30693 | #define MA_PA_SOURCE_IDLE                   1
30694 | #define MA_PA_SOURCE_SUSPENDED              2
30695 |
30696 | typedef int ma_pa_seek_mode_t;
30697 | #define MA_PA_SEEK_RELATIVE                 0
30698 | #define MA_PA_SEEK_ABSOLUTE                 1
30699 | #define MA_PA_SEEK_RELATIVE_ON_READ         2
30700 | #define MA_PA_SEEK_RELATIVE_END             3
30701 |
30702 | typedef int ma_pa_channel_position_t;
30703 | #define MA_PA_CHANNEL_POSITION_INVALID       -1
30704 | #define MA_PA_CHANNEL_POSITION_MONO         0
30705 | #define MA_PA_CHANNEL_POSITION_FRONT_LEFT   1
30706 | #define MA_PA_CHANNEL_POSITION_FRONT_RIGHT  2
30707 | #define MA_PA_CHANNEL_POSITION_FRONT_CENTER 3
30708 | #define MA_PA_CHANNEL_POSITION_REAR_CENTER  4
30709 | #define MA_PA_CHANNEL_POSITION_REAR_LEFT    5
30710 | #define MA_PA_CHANNEL_POSITION_REAR_RIGHT   6
30711 | #define MA_PA_CHANNEL_POSITION_LFE          7
30712 | #define MA_PA_CHANNEL_POSITION_FRONT_LEFT_OF_CENTER 8
30713 | #define MA_PA_CHANNEL_POSITION_FRONT_RIGHT_OF_CENTER 9
30714 | #define MA_PA_CHANNEL_POSITION_SIDE_LEFT    10
30715 | #define MA_PA_CHANNEL_POSITION_SIDE_RIGHT   11
30716 | #define MA_PA_CHANNEL_POSITION_AUX0         12
30717 | #define MA_PA_CHANNEL_POSITION_AUX1         13
30718 | #define MA_PA_CHANNEL_POSITION_AUX2         14
30719 | #define MA_PA_CHANNEL_POSITION_AUX3         15
30720 | #define MA_PA_CHANNEL_POSITION_AUX4         16
30721 | #define MA_PA_CHANNEL_POSITION_AUX5         17
30722 | #define MA_PA_CHANNEL_POSITION_AUX6         18
30723 | #define MA_PA_CHANNEL_POSITION_AUX7         19
30724 | #define MA_PA_CHANNEL_POSITION_AUX8         20
30725 | #define MA_PA_CHANNEL_POSITION_AUX9         21
30726 | #define MA_PA_CHANNEL_POSITION_AUX10        22
30727 | #define MA_PA_CHANNEL_POSITION_AUX11        23
30728 | #define MA_PA_CHANNEL_POSITION_AUX12        24
30729 | #define MA_PA_CHANNEL_POSITION_AUX13        25
30730 | #define MA_PA_CHANNEL_POSITION_AUX14        26
30731 | #define MA_PA_CHANNEL_POSITION_AUX15        27

```

```

30732 | #define MA_PA_CHANNEL_POSITION_AUX16      28
30733 | #define MA_PA_CHANNEL_POSITION_AUX17      29
30734 | #define MA_PA_CHANNEL_POSITION_AUX18      30
30735 | #define MA_PA_CHANNEL_POSITION_AUX19      31
30736 | #define MA_PA_CHANNEL_POSITION_AUX20      32
30737 | #define MA_PA_CHANNEL_POSITION_AUX21      33
30738 | #define MA_PA_CHANNEL_POSITION_AUX22      34
30739 | #define MA_PA_CHANNEL_POSITION_AUX23      35
30740 | #define MA_PA_CHANNEL_POSITION_AUX24      36
30741 | #define MA_PA_CHANNEL_POSITION_AUX25      37
30742 | #define MA_PA_CHANNEL_POSITION_AUX26      38
30743 | #define MA_PA_CHANNEL_POSITION_AUX27      39
30744 | #define MA_PA_CHANNEL_POSITION_AUX28      40
30745 | #define MA_PA_CHANNEL_POSITION_AUX29      41
30746 | #define MA_PA_CHANNEL_POSITION_AUX30      42
30747 | #define MA_PA_CHANNEL_POSITION_AUX31      43
30748 | #define MA_PA_CHANNEL_POSITION_TOP_CENTER  44
30749 | #define MA_PA_CHANNEL_POSITION_TOP_FRONT_LEFT  45
30750 | #define MA_PA_CHANNEL_POSITION_TOP_FRONT_RIGHT  46
30751 | #define MA_PA_CHANNEL_POSITION_TOP_FRONT_CENTER  47
30752 | #define MA_PA_CHANNEL_POSITION_TOP_REAR_LEFT  48
30753 | #define MA_PA_CHANNEL_POSITION_TOP_REAR_RIGHT  49
30754 | #define MA_PA_CHANNEL_POSITION_TOP_REAR_CENTER  50
30755 | #define MA_PA_CHANNEL_POSITION_LEFT        MA_PA_CHANNEL_POSITION_FRONT_LEFT
30756 | #define MA_PA_CHANNEL_POSITION_RIGHT       MA_PA_CHANNEL_POSITION_FRONT_RIGHT
30757 | #define MA_PA_CHANNEL_POSITION_CENTER      MA_PA_CHANNEL_POSITION_FRONT_CENTER
30758 | #define MA_PA_CHANNEL_POSITION_SUBWOOFER    MA_PA_CHANNEL_POSITION_LFE
30759 |
30760 | typedef int ma_pa_channel_map_def_t;
30761 | #define MA_PA_CHANNEL_MAP_AIFF            0
30762 | #define MA_PA_CHANNEL_MAP_ALSA            1
30763 | #define MA_PA_CHANNEL_MAP_AUX             2
30764 | #define MA_PA_CHANNEL_MAP_WAVEEX         3
30765 | #define MA_PA_CHANNEL_MAP_OSS             4
30766 | #define MA_PA_CHANNEL_MAP_DEFAULT        MA_PA_CHANNEL_MAP_AIFF
30767 |
30768 | typedef int ma_pa_sample_format_t;
30769 | #define MA_PA_SAMPLE_INVALID             -1
30770 | #define MA_PA_SAMPLE_U8                  0
30771 | #define MA_PA_SAMPLE_ALAW                1
30772 | #define MA_PA_SAMPLE_ULAW                2
30773 | #define MA_PA_SAMPLE_S16LE               3
30774 | #define MA_PA_SAMPLE_S16BE               4
30775 | #define MA_PA_SAMPLE_FLOAT32LE           5
30776 | #define MA_PA_SAMPLE_FLOAT32BE           6
30777 | #define MA_PA_SAMPLE_S32LE               7
30778 | #define MA_PA_SAMPLE_S32BE               8
30779 | #define MA_PA_SAMPLE_S24LE               9
30780 | #define MA_PA_SAMPLE_S24BE              10
30781 | #define MA_PA_SAMPLE_S24_32LE            11
30782 | #define MA_PA_SAMPLE_S24_32BE            12
30783 |
30784 | typedef struct ma_pa_mainloop             ma_pa_mainloop;
30785 | typedef struct ma_pa_threaded_mainloop    ma_pa_threaded_mainloop;
30786 | typedef struct ma_pa_mainloop_api         ma_pa_mainloop_api;
30787 | typedef struct ma_pa_context              ma_pa_context;
30788 | typedef struct ma_pa_operation            ma_pa_operation;
30789 | typedef struct ma_pa_stream               ma_pa_stream;
30790 | typedef struct ma_pa_spawn_api            ma_pa_spawn_api;
30791 |
30792 | typedef struct
30793 | {
30794 |     ma_uint32 maxlength;
30795 |     ma_uint32 tlength;
30796 |     ma_uint32 prebuf;
30797 |     ma_uint32 minreq;
30798 |     ma_uint32 fragsize;
30799 | } ma_pa_buffer_attr;
30800 |
30801 | typedef struct
30802 | {
30803 |     ma_uint8 channels;
30804 |     ma_pa_channel_position_t map[MA_PA_CHANNELS_MAX];
30805 | } ma_pa_channel_map;
30806 |
30807 | typedef struct
30808 | {
30809 |     ma_uint8 channels;
30810 |     ma_uint32 values[MA_PA_CHANNELS_MAX];

```

```

30811 | } ma_pa_cvolume;
30812 |
30813 | typedef struct
30814 | {
30815 |     ma_pa_sample_format_t format;
30816 |     ma_uint32 rate;
30817 |     ma_uint8 channels;
30818 | } ma_pa_sample_spec;
30819 |
30820 | typedef struct
30821 | {
30822 |     const char* name;
30823 |     ma_uint32 index;
30824 |     const char* description;
30825 |     ma_pa_sample_spec sample_spec;
30826 |     ma_pa_channel_map channel_map;
30827 |     ma_uint32 owner_module;
30828 |     ma_pa_cvolume volume;
30829 |     int mute;
30830 |     ma_uint32 monitor_source;
30831 |     const char* monitor_source_name;
30832 |     ma_uint64 latency;
30833 |     const char* driver;
30834 |     ma_pa_sink_flags_t flags;
30835 |     void* proplist;
30836 |     ma_uint64 configured_latency;
30837 |     ma_uint32 base_volume;
30838 |     ma_pa_sink_state_t state;
30839 |     ma_uint32 n_volume_steps;
30840 |     ma_uint32 card;
30841 |     ma_uint32 n_ports;
30842 |     void** ports;
30843 |     void* active_port;
30844 |     ma_uint8 n_formats;
30845 |     void** formats;
30846 | } ma_pa_sink_info;
30847 |
30848 | typedef struct
30849 | {
30850 |     const char *name;
30851 |     ma_uint32 index;
30852 |     const char *description;
30853 |     ma_pa_sample_spec sample_spec;
30854 |     ma_pa_channel_map channel_map;
30855 |     ma_uint32 owner_module;
30856 |     ma_pa_cvolume volume;
30857 |     int mute;
30858 |     ma_uint32 monitor_of_sink;
30859 |     const char *monitor_of_sink_name;
30860 |     ma_uint64 latency;
30861 |     const char *driver;
30862 |     ma_pa_source_flags_t flags;
30863 |     void* proplist;
30864 |     ma_uint64 configured_latency;
30865 |     ma_uint32 base_volume;
30866 |     ma_pa_source_state_t state;
30867 |     ma_uint32 n_volume_steps;
30868 |     ma_uint32 card;
30869 |     ma_uint32 n_ports;
30870 |     void** ports;
30871 |     void* active_port;
30872 |     ma_uint8 n_formats;
30873 |     void** formats;
30874 | } ma_pa_source_info;
30875 |
30876 | typedef void (* ma_pa_context_notify_cb_t)(ma_pa_context* c, void* userdata);
30877 | typedef void (* ma_pa_sink_info_cb_t)      (ma_pa_context* c, const ma_pa_sink_info* i, int eol,
> void* userdata);
30878 | typedef void (* ma_pa_source_info_cb_t)    (ma_pa_context* c, const ma_pa_source_info* i, int eol,
> void* userdata);
30879 | typedef void (* ma_pa_stream_success_cb_t)(ma_pa_stream* s, int success, void* userdata);
30880 | typedef void (* ma_pa_stream_request_cb_t)(ma_pa_stream* s, size_t nbytes, void* userdata);
30881 | typedef void (* ma_pa_stream_notify_cb_t) (ma_pa_stream* s, void* userdata);
30882 | typedef void (* ma_pa_free_cb_t)          (void* p);
30883 | #endif
30884 |
30885 |
30886 | typedef ma_pa_mainloop*      (* ma_pa_mainloop_new_proc)      (void);
30887 | typedef void                 (* ma_pa_mainloop_free_proc)     (ma_pa_mainloop* m);

```

```

30888 | typedef void                (* ma_pa_mainloop_quit_proc)                (ma_pa_mainloop* m,
> int retval);
30889 | typedef ma_pa_mainloop_api* (* ma_pa_mainloop_get_api_proc)            (ma_pa_mainloop* m);
30890 | typedef int                 (* ma_pa_mainloop_iterate_proc)            (ma_pa_mainloop* m,
> int block, int* retval);
30891 | typedef void                (* ma_pa_mainloop_wakeup_proc)              (ma_pa_mainloop* m);
30892 | typedef ma_pa_threaded_mainloop* (* ma_pa_threaded_mainloop_new_proc)  (void);
30893 | typedef void                (* ma_pa_threaded_mainloop_free_proc)
> (ma_pa_threaded_mainloop* m);
30894 | typedef int                 (* ma_pa_threaded_mainloop_start_proc)
> (ma_pa_threaded_mainloop* m);
30895 | typedef void                (* ma_pa_threaded_mainloop_stop_proc)
> (ma_pa_threaded_mainloop* m);
30896 | typedef void                (* ma_pa_threaded_mainloop_lock_proc)
> (ma_pa_threaded_mainloop* m);
30897 | typedef void                (* ma_pa_threaded_mainloop_unlock_proc)
> (ma_pa_threaded_mainloop* m);
30898 | typedef void                (* ma_pa_threaded_mainloop_wait_proc)
> (ma_pa_threaded_mainloop* m);
30899 | typedef void                (* ma_pa_threaded_mainloop_signal_proc)
> (ma_pa_threaded_mainloop* m, int wait_for_accept);
30900 | typedef void                (* ma_pa_threaded_mainloop_accept_proc)
> (ma_pa_threaded_mainloop* m);
30901 | typedef int                 (* ma_pa_threaded_mainloop_get_retval_proc) (const
> ma_pa_threaded_mainloop* m);
30902 | typedef ma_pa_mainloop_api* (* ma_pa_threaded_mainloop_get_api_proc)
> (ma_pa_threaded_mainloop* m);
30903 | typedef int                 (* ma_pa_threaded_mainloop_in_thread_proc)
> (ma_pa_threaded_mainloop* m);
30904 | typedef void                (* ma_pa_threaded_mainloop_set_name_proc)
> (ma_pa_threaded_mainloop* m, const char* name);
30905 | typedef ma_pa_context*      (* ma_pa_context_new_proc)                  (ma_pa_mainloop_api*
> mainloop, const char* name);
30906 | typedef void                (* ma_pa_context_unref_proc)                (ma_pa_context* c);
30907 | typedef int                 (* ma_pa_context_connect_proc)              (ma_pa_context* c,
> const char* server, ma_pa_context_flags_t flags, const ma_pa_spawn_api* api);
30908 | typedef void                (* ma_pa_context_disconnect_proc)            (ma_pa_context* c);
30909 | typedef void                (* ma_pa_context_set_state_callback_proc)    (ma_pa_context* c,
> ma_pa_context_notify_cb_t cb, void* userdata);
30910 | typedef ma_pa_context_state_t (* ma_pa_context_get_state_proc)           (const ma_pa_context*
> c);
30911 | typedef ma_pa_operation*     (* ma_pa_context_get_sink_info_list_proc)    (ma_pa_context* c,
> ma_pa_sink_info_cb_t cb, void* userdata);
30912 | typedef ma_pa_operation*     (* ma_pa_context_get_source_info_list_proc) (ma_pa_context* c,
> ma_pa_source_info_cb_t cb, void* userdata);
30913 | typedef ma_pa_operation*     (* ma_pa_context_get_sink_info_by_name_proc) (ma_pa_context* c,
> const char* name, ma_pa_sink_info_cb_t cb, void* userdata);
30914 | typedef ma_pa_operation*     (* ma_pa_context_get_source_info_by_name_proc)(ma_pa_context* c,
> const char* name, ma_pa_source_info_cb_t cb, void* userdata);
30915 | typedef void                (* ma_pa_operation_unref_proc)              (ma_pa_operation* o);
30916 | typedef ma_pa_operation_state_t (* ma_pa_operation_get_state_proc)       (const
> ma_pa_operation* o);
30917 | typedef ma_pa_channel_map*    (* ma_pa_channel_map_init_extend_proc)      (ma_pa_channel_map*
> m, unsigned channels, ma_pa_channel_map_def_t def);
30918 | typedef int                 (* ma_pa_channel_map_valid_proc)             (const
> ma_pa_channel_map* m);
30919 | typedef int                 (* ma_pa_channel_map_compatible_proc)         (const
> ma_pa_channel_map* m, const ma_pa_sample_spec* ss);
30920 | typedef ma_pa_stream*        (* ma_pa_stream_new_proc)                   (ma_pa_context* c,
> const char* name, const ma_pa_sample_spec* ss, const ma_pa_channel_map* map);
30921 | typedef void                (* ma_pa_stream_unref_proc)                  (ma_pa_stream* s);
30922 | typedef int                 (* ma_pa_stream_connect_playback_proc)        (ma_pa_stream* s,
> const char* dev, const ma_pa_buffer_attr* attr, ma_pa_stream_flags_t flags, const ma_pa_cvolume*
> volume, ma_pa_stream* sync_stream);
30923 | typedef int                 (* ma_pa_stream_connect_record_proc)          (ma_pa_stream* s,
> const char* dev, const ma_pa_buffer_attr* attr, ma_pa_stream_flags_t flags);
30924 | typedef int                 (* ma_pa_stream_disconnect_proc)              (ma_pa_stream* s);
30925 | typedef ma_pa_stream_state_t (* ma_pa_stream_get_state_proc)              (const ma_pa_stream*
> s);
30926 | typedef const ma_pa_sample_spec* (* ma_pa_stream_get_sample_spec_proc)    (ma_pa_stream* s);
30927 | typedef const ma_pa_channel_map* (* ma_pa_stream_get_channel_map_proc)    (ma_pa_stream* s);
30928 | typedef const ma_pa_buffer_attr* (* ma_pa_stream_get_buffer_attr_proc)    (ma_pa_stream* s);
30929 | typedef ma_pa_operation*     (* ma_pa_stream_set_buffer_attr_proc)        (ma_pa_stream* s,
> const ma_pa_buffer_attr* attr, ma_pa_stream_success_cb_t cb, void* userdata);
30930 | typedef const char*          (* ma_pa_stream_get_device_name_proc)        (const ma_pa_stream*
> s);
30931 | typedef void                (* ma_pa_stream_set_write_callback_proc)      (ma_pa_stream* s,
> ma_pa_stream_request_cb_t cb, void* userdata);
30932 | typedef void                (* ma_pa_stream_set_read_callback_proc)       (ma_pa_stream* s,

```

```

> ma_pa_stream_request_cb_t cb, void* userdata);
30933 | typedef void (* ma_pa_stream_set_suspended_callback_proc) (ma_pa_stream* s,
> ma_pa_stream_notify_cb_t cb, void* userdata);
30934 | typedef void (* ma_pa_stream_set_moved_callback_proc) (ma_pa_stream* s,
> ma_pa_stream_notify_cb_t cb, void* userdata);
30935 | typedef int (* ma_pa_stream_is_suspended_proc) (const ma_pa_stream*
> s);
30936 | typedef ma_pa_operation* (* ma_pa_stream_flush_proc) (ma_pa_stream* s,
> ma_pa_stream_success_cb_t cb, void* userdata);
30937 | typedef ma_pa_operation* (* ma_pa_stream_drain_proc) (ma_pa_stream* s,
> ma_pa_stream_success_cb_t cb, void* userdata);
30938 | typedef int (* ma_pa_stream_is_corked_proc) (const ma_pa_stream*
> s);
30939 | typedef ma_pa_operation* (* ma_pa_stream_cork_proc) (ma_pa_stream* s, int
> b, ma_pa_stream_success_cb_t cb, void* userdata);
30940 | typedef ma_pa_operation* (* ma_pa_stream_trigger_proc) (ma_pa_stream* s,
> ma_pa_stream_success_cb_t cb, void* userdata);
30941 | typedef int (* ma_pa_stream_begin_write_proc) (ma_pa_stream* s,
> void** data, size_t* nbytes);
30942 | typedef int (* ma_pa_stream_write_proc) (ma_pa_stream* s,
> const void* data, size_t nbytes, ma_pa_free_cb_t free_cb, int64_t offset, ma_pa_seek_mode_t seek);
30943 | typedef int (* ma_pa_stream_peek_proc) (ma_pa_stream* s,
> const void** data, size_t* nbytes);
30944 | typedef int (* ma_pa_stream_drop_proc) (ma_pa_stream* s);
30945 | typedef size_t (* ma_pa_stream_writable_size_proc) (const ma_pa_stream*
> s);
30946 | typedef size_t (* ma_pa_stream_readable_size_proc) (const ma_pa_stream*
> s);
30947 |
30948 | typedef struct
30949 | {
30950 |     ma_uint32 count;
30951 |     ma_uint32 capacity;
30952 |     ma_device_info* pInfo;
30953 | } ma_pulse_device_enum_data;
30954 |
30955 | static ma_result ma_result_from_pulse(int result)
30956 | {
30957 |     if (result < 0) {
30958 |         return MA_ERROR;
30959 |     }
30960 |
30961 |     switch (result) {
30962 |         case MA_PA_OK: return MA_SUCCESS;
30963 |         case MA_PA_ERR_ACCESS: return MA_ACCESS_DENIED;
30964 |         case MA_PA_ERR_INVALID: return MA_INVALID_ARGS;
30965 |         case MA_PA_ERR_NOENTITY: return MA_NO_DEVICE;
30966 |         default: return MA_ERROR;
30967 |     }
30968 | }
30969 |
30970 | #if 0
30971 | static ma_pa_sample_format_t ma_format_to_pulse(ma_format format)
30972 | {
30973 |     if (ma_is_little_endian()) {
30974 |         switch (format) {
30975 |             case ma_format_s16: return MA_PA_SAMPLE_S16LE;
30976 |             case ma_format_s24: return MA_PA_SAMPLE_S24LE;
30977 |             case ma_format_s32: return MA_PA_SAMPLE_S32LE;
30978 |             case ma_format_f32: return MA_PA_SAMPLE_FLOAT32LE;
30979 |             default: break;
30980 |         }
30981 |     } else {
30982 |         switch (format) {
30983 |             case ma_format_s16: return MA_PA_SAMPLE_S16BE;
30984 |             case ma_format_s24: return MA_PA_SAMPLE_S24BE;
30985 |             case ma_format_s32: return MA_PA_SAMPLE_S32BE;
30986 |             case ma_format_f32: return MA_PA_SAMPLE_FLOAT32BE;
30987 |             default: break;
30988 |         }
30989 |     }
30990 |
30991 |     /* Endian agnostic. */
30992 |     switch (format) {
30993 |         case ma_format_u8: return MA_PA_SAMPLE_U8;
30994 |         default: return MA_PA_SAMPLE_INVALID;
30995 |     }
30996 | }
30997 | #endif

```



```

30998 |
30999 | static ma_format ma_format_from_pulse(ma_pa_sample_format_t format)
31000 | {
31001 |     if (ma_is_little_endian()) {
31002 |         switch (format) {
31003 |             case MA_PA_SAMPLE_S16LE:    return ma_format_s16;
31004 |             case MA_PA_SAMPLE_S24LE:    return ma_format_s24;
31005 |             case MA_PA_SAMPLE_S32LE:    return ma_format_s32;
31006 |             case MA_PA_SAMPLE_FLOAT32LE: return ma_format_f32;
31007 |             default: break;
31008 |         }
31009 |     } else {
31010 |         switch (format) {
31011 |             case MA_PA_SAMPLE_S16BE:    return ma_format_s16;
31012 |             case MA_PA_SAMPLE_S24BE:    return ma_format_s24;
31013 |             case MA_PA_SAMPLE_S32BE:    return ma_format_s32;
31014 |             case MA_PA_SAMPLE_FLOAT32BE: return ma_format_f32;
31015 |             default: break;
31016 |         }
31017 |     }
31018 |
31019 |     /* Endian agnostic. */
31020 |     switch (format) {
31021 |         case MA_PA_SAMPLE_U8: return ma_format_u8;
31022 |         default: return ma_format_unknown;
31023 |     }
31024 | }
31025 |
31026 | static ma_channel ma_channel_position_from_pulse(ma_pa_channel_position_t position)
31027 | {
31028 |     switch (position)
31029 |     {
31030 |         case MA_PA_CHANNEL_POSITION_INVALID:    return MA_CHANNEL_NONE;
31031 |         case MA_PA_CHANNEL_POSITION_MONO:      return MA_CHANNEL_MONO;
31032 |         case MA_PA_CHANNEL_POSITION_FRONT_LEFT: return MA_CHANNEL_FRONT_LEFT;
31033 |         case MA_PA_CHANNEL_POSITION_FRONT_RIGHT: return MA_CHANNEL_FRONT_RIGHT;
31034 |         case MA_PA_CHANNEL_POSITION_FRONT_CENTER: return MA_CHANNEL_FRONT_CENTER;
31035 |         case MA_PA_CHANNEL_POSITION_REAR_CENTER: return MA_CHANNEL_BACK_CENTER;
31036 |         case MA_PA_CHANNEL_POSITION_REAR_LEFT:  return MA_CHANNEL_BACK_LEFT;
31037 |         case MA_PA_CHANNEL_POSITION_REAR_RIGHT: return MA_CHANNEL_BACK_RIGHT;
31038 |         case MA_PA_CHANNEL_POSITION_LFE:        return MA_CHANNEL_LFE;
31039 |         case MA_PA_CHANNEL_POSITION_FRONT_LEFT_OF_CENTER: return MA_CHANNEL_FRONT_LEFT_CENTER;
31040 |         case MA_PA_CHANNEL_POSITION_FRONT_RIGHT_OF_CENTER: return MA_CHANNEL_FRONT_RIGHT_CENTER;
31041 |         case MA_PA_CHANNEL_POSITION_SIDE_LEFT:  return MA_CHANNEL_SIDE_LEFT;
31042 |         case MA_PA_CHANNEL_POSITION_SIDE_RIGHT: return MA_CHANNEL_SIDE_RIGHT;
31043 |         case MA_PA_CHANNEL_POSITION_AUX0:       return MA_CHANNEL_AUX_0;
31044 |         case MA_PA_CHANNEL_POSITION_AUX1:       return MA_CHANNEL_AUX_1;
31045 |         case MA_PA_CHANNEL_POSITION_AUX2:       return MA_CHANNEL_AUX_2;
31046 |         case MA_PA_CHANNEL_POSITION_AUX3:       return MA_CHANNEL_AUX_3;
31047 |         case MA_PA_CHANNEL_POSITION_AUX4:       return MA_CHANNEL_AUX_4;
31048 |         case MA_PA_CHANNEL_POSITION_AUX5:       return MA_CHANNEL_AUX_5;
31049 |         case MA_PA_CHANNEL_POSITION_AUX6:       return MA_CHANNEL_AUX_6;
31050 |         case MA_PA_CHANNEL_POSITION_AUX7:       return MA_CHANNEL_AUX_7;
31051 |         case MA_PA_CHANNEL_POSITION_AUX8:       return MA_CHANNEL_AUX_8;
31052 |         case MA_PA_CHANNEL_POSITION_AUX9:       return MA_CHANNEL_AUX_9;
31053 |         case MA_PA_CHANNEL_POSITION_AUX10:      return MA_CHANNEL_AUX_10;
31054 |         case MA_PA_CHANNEL_POSITION_AUX11:      return MA_CHANNEL_AUX_11;
31055 |         case MA_PA_CHANNEL_POSITION_AUX12:      return MA_CHANNEL_AUX_12;
31056 |         case MA_PA_CHANNEL_POSITION_AUX13:      return MA_CHANNEL_AUX_13;
31057 |         case MA_PA_CHANNEL_POSITION_AUX14:      return MA_CHANNEL_AUX_14;
31058 |         case MA_PA_CHANNEL_POSITION_AUX15:      return MA_CHANNEL_AUX_15;
31059 |         case MA_PA_CHANNEL_POSITION_AUX16:      return MA_CHANNEL_AUX_16;
31060 |         case MA_PA_CHANNEL_POSITION_AUX17:      return MA_CHANNEL_AUX_17;
31061 |         case MA_PA_CHANNEL_POSITION_AUX18:      return MA_CHANNEL_AUX_18;
31062 |         case MA_PA_CHANNEL_POSITION_AUX19:      return MA_CHANNEL_AUX_19;
31063 |         case MA_PA_CHANNEL_POSITION_AUX20:      return MA_CHANNEL_AUX_20;
31064 |         case MA_PA_CHANNEL_POSITION_AUX21:      return MA_CHANNEL_AUX_21;
31065 |         case MA_PA_CHANNEL_POSITION_AUX22:      return MA_CHANNEL_AUX_22;
31066 |         case MA_PA_CHANNEL_POSITION_AUX23:      return MA_CHANNEL_AUX_23;
31067 |         case MA_PA_CHANNEL_POSITION_AUX24:      return MA_CHANNEL_AUX_24;
31068 |         case MA_PA_CHANNEL_POSITION_AUX25:      return MA_CHANNEL_AUX_25;
31069 |         case MA_PA_CHANNEL_POSITION_AUX26:      return MA_CHANNEL_AUX_26;
31070 |         case MA_PA_CHANNEL_POSITION_AUX27:      return MA_CHANNEL_AUX_27;
31071 |         case MA_PA_CHANNEL_POSITION_AUX28:      return MA_CHANNEL_AUX_28;
31072 |         case MA_PA_CHANNEL_POSITION_AUX29:      return MA_CHANNEL_AUX_29;
31073 |         case MA_PA_CHANNEL_POSITION_AUX30:      return MA_CHANNEL_AUX_30;
31074 |         case MA_PA_CHANNEL_POSITION_AUX31:      return MA_CHANNEL_AUX_31;
31075 |         case MA_PA_CHANNEL_POSITION_TOP_CENTER: return MA_CHANNEL_TOP_CENTER;
31076 |         case MA_PA_CHANNEL_POSITION_TOP_FRONT_LEFT: return MA_CHANNEL_TOP_FRONT_LEFT;

```



```

31077 |         case MA_PA_CHANNEL_POSITION_TOP_FRONT_RIGHT:      return MA_CHANNEL_TOP_FRONT_RIGHT;
31078 |         case MA_PA_CHANNEL_POSITION_TOP_FRONT_CENTER:    return MA_CHANNEL_TOP_FRONT_CENTER;
31079 |         case MA_PA_CHANNEL_POSITION_TOP_REAR_LEFT:       return MA_CHANNEL_TOP_BACK_LEFT;
31080 |         case MA_PA_CHANNEL_POSITION_TOP_REAR_RIGHT:      return MA_CHANNEL_TOP_BACK_RIGHT;
31081 |         case MA_PA_CHANNEL_POSITION_TOP_REAR_CENTER:     return MA_CHANNEL_TOP_BACK_CENTER;
31082 |         default: return MA_CHANNEL_NONE;
31083 |     }
31084 | }
31085 |
31086 | #if 0
31087 | static ma_pa_channel_position_t ma_channel_position_to_pulse(ma_channel position)
31088 | {
31089 |     switch (position)
31090 |     {
31091 |         case MA_CHANNEL_NONE:                return MA_PA_CHANNEL_POSITION_INVALID;
31092 |         case MA_CHANNEL_FRONT_LEFT:           return MA_PA_CHANNEL_POSITION_FRONT_LEFT;
31093 |         case MA_CHANNEL_FRONT_RIGHT:          return MA_PA_CHANNEL_POSITION_FRONT_RIGHT;
31094 |         case MA_CHANNEL_FRONT_CENTER:         return MA_PA_CHANNEL_POSITION_FRONT_CENTER;
31095 |         case MA_CHANNEL_LFE:                 return MA_PA_CHANNEL_POSITION_LFE;
31096 |         case MA_CHANNEL_BACK_LEFT:            return MA_PA_CHANNEL_POSITION_REAR_LEFT;
31097 |         case MA_CHANNEL_BACK_RIGHT:           return MA_PA_CHANNEL_POSITION_REAR_RIGHT;
31098 |         case MA_CHANNEL_FRONT_LEFT_CENTER:    return MA_PA_CHANNEL_POSITION_FRONT_LEFT_OF_CENTER;
31099 |         case MA_CHANNEL_FRONT_RIGHT_CENTER:   return MA_PA_CHANNEL_POSITION_FRONT_RIGHT_OF_CENTER;
31100 |         case MA_CHANNEL_BACK_CENTER:          return MA_PA_CHANNEL_POSITION_REAR_CENTER;
31101 |         case MA_CHANNEL_SIDE_LEFT:            return MA_PA_CHANNEL_POSITION_SIDE_LEFT;
31102 |         case MA_CHANNEL_SIDE_RIGHT:           return MA_PA_CHANNEL_POSITION_SIDE_RIGHT;
31103 |         case MA_CHANNEL_TOP_CENTER:           return MA_PA_CHANNEL_POSITION_TOP_CENTER;
31104 |         case MA_CHANNEL_TOP_FRONT_LEFT:       return MA_PA_CHANNEL_POSITION_TOP_FRONT_LEFT;
31105 |         case MA_CHANNEL_TOP_FRONT_CENTER:     return MA_PA_CHANNEL_POSITION_TOP_FRONT_CENTER;
31106 |         case MA_CHANNEL_TOP_FRONT_RIGHT:      return MA_PA_CHANNEL_POSITION_TOP_FRONT_RIGHT;
31107 |         case MA_CHANNEL_TOP_BACK_LEFT:        return MA_PA_CHANNEL_POSITION_TOP_REAR_LEFT;
31108 |         case MA_CHANNEL_TOP_BACK_CENTER:      return MA_PA_CHANNEL_POSITION_TOP_REAR_CENTER;
31109 |         case MA_CHANNEL_TOP_BACK_RIGHT:       return MA_PA_CHANNEL_POSITION_TOP_REAR_RIGHT;
31110 |         case MA_CHANNEL_19:                   return MA_PA_CHANNEL_POSITION_AUX18;
31111 |         case MA_CHANNEL_20:                   return MA_PA_CHANNEL_POSITION_AUX19;
31112 |         case MA_CHANNEL_21:                   return MA_PA_CHANNEL_POSITION_AUX20;
31113 |         case MA_CHANNEL_22:                   return MA_PA_CHANNEL_POSITION_AUX21;
31114 |         case MA_CHANNEL_23:                   return MA_PA_CHANNEL_POSITION_AUX22;
31115 |         case MA_CHANNEL_24:                   return MA_PA_CHANNEL_POSITION_AUX23;
31116 |         case MA_CHANNEL_25:                   return MA_PA_CHANNEL_POSITION_AUX24;
31117 |         case MA_CHANNEL_26:                   return MA_PA_CHANNEL_POSITION_AUX25;
31118 |         case MA_CHANNEL_27:                   return MA_PA_CHANNEL_POSITION_AUX26;
31119 |         case MA_CHANNEL_28:                   return MA_PA_CHANNEL_POSITION_AUX27;
31120 |         case MA_CHANNEL_29:                   return MA_PA_CHANNEL_POSITION_AUX28;
31121 |         case MA_CHANNEL_30:                   return MA_PA_CHANNEL_POSITION_AUX29;
31122 |         case MA_CHANNEL_31:                   return MA_PA_CHANNEL_POSITION_AUX30;
31123 |         case MA_CHANNEL_32:                   return MA_PA_CHANNEL_POSITION_AUX31;
31124 |         default: return (ma_pa_channel_position_t)position;
31125 |     }
31126 | }
31127 | #endif
31128 |
31129 | static ma_result ma_wait_for_operation__pulse(ma_context* pContext, ma_ptr pMainLoop,
> ma_pa_operation* pOP)
31130 | {
31131 |     int resultPA;
31132 |     ma_pa_operation_state_t state;
31133 |
31134 |     MA_ASSERT(pContext != NULL);
31135 |     MA_ASSERT(pOP != NULL);
31136 |
31137 |     for (;;) {
31138 |         state = ((ma_pa_operation_get_state_proc)pContext->pulse.pa_operation_get_state)(pOP);
31139 |         if (state != MA_PA_OPERATION_RUNNING) {
31140 |             break; /* Done. */
31141 |         }
31142 |
31143 |         resultPA =
> ((ma_pa_mainloop_iterate_proc)pContext->pulse.pa_mainloop_iterate)((ma_pa_mainloop*)pMainLoop, 1,
> NULL);
31144 |         if (resultPA < 0) {
31145 |             return ma_result_from_pulse(resultPA);
31146 |         }
31147 |     }
31148 |
31149 |     return MA_SUCCESS;
31150 | }
31151 |
31152 | static ma_result ma_wait_for_operation_and_unref__pulse(ma_context* pContext, ma_ptr pMainLoop,

```

```

> ma_pa_operation* pOP)
31153 | {
31154 |     ma_result result;
31155 |
31156 |     if (pOP == NULL) {
31157 |         return MA_INVALID_ARGS;
31158 |     }
31159 |
31160 |     result = ma_wait_for_operation__pulse(pContext, pMainLoop, pOP);
31161 |     ((ma_pa_operation_unref_proc)pContext->pulse.pa_operation_unref)(pOP);
31162 |
31163 |     return result;
31164 | }
31165 |
31166 | static ma_result ma_wait_for_pa_context_to_connect__pulse(ma_context* pContext, ma_ptr pMainLoop,
> ma_ptr pPulseContext)
31167 | {
31168 |     int resultPA;
31169 |     ma_pa_context_state_t state;
31170 |
31171 |     for (;;) {
31172 |         state =
> ((ma_pa_context_get_state_proc)pContext->pulse.pa_context_get_state)((ma_pa_context*)pPulseContext);
31173 |         if (state == MA_PA_CONTEXT_READY) {
31174 |             break; /* Done. */
31175 |         }
31176 |
31177 |         if (state == MA_PA_CONTEXT_FAILED || state == MA_PA_CONTEXT_TERMINATED) {
31178 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[PulseAudio] An error
> occurred while connecting the PulseAudio context.");
31179 |             return MA_ERROR;
31180 |         }
31181 |
31182 |         resultPA =
> ((ma_pa_mainloop_iterate_proc)pContext->pulse.pa_mainloop_iterate)((ma_pa_mainloop*)pMainLoop, 1,
> NULL);
31183 |         if (resultPA < 0) {
31184 |             return ma_result_from_pulse(resultPA);
31185 |         }
31186 |     }
31187 |
31188 |     /* Should never get here. */
31189 |     return MA_SUCCESS;
31190 | }
31191 |
31192 | static ma_result ma_wait_for_pa_stream_to_connect__pulse(ma_context* pContext, ma_ptr pMainLoop,
> ma_ptr pStream)
31193 | {
31194 |     int resultPA;
31195 |     ma_pa_stream_state_t state;
31196 |
31197 |     for (;;) {
31198 |         state =
> ((ma_pa_stream_get_state_proc)pContext->pulse.pa_stream_get_state)((ma_pa_stream*)pStream);
31199 |         if (state == MA_PA_STREAM_READY) {
31200 |             break; /* Done. */
31201 |         }
31202 |
31203 |         if (state == MA_PA_STREAM_FAILED || state == MA_PA_STREAM_TERMINATED) {
31204 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[PulseAudio] An error
> occurred while connecting the PulseAudio stream.");
31205 |             return MA_ERROR;
31206 |         }
31207 |
31208 |         resultPA =
> ((ma_pa_mainloop_iterate_proc)pContext->pulse.pa_mainloop_iterate)((ma_pa_mainloop*)pMainLoop, 1,
> NULL);
31209 |         if (resultPA < 0) {
31210 |             return ma_result_from_pulse(resultPA);
31211 |         }
31212 |     }
31213 |
31214 |     return MA_SUCCESS;
31215 | }
31216 |
31217 |
31218 | static ma_result ma_init_pa_mainloop_and_pa_context__pulse(ma_context* pContext, const char*
> pApplicationName, const char* pServerName, ma_bool32 tryAutoSpawn, ma_ptr* ppMainLoop, ma_ptr*
> ppPulseContext)

```

```

31219 | {
31220 |     ma_result result;
31221 |     ma_ptr pMainLoop;
31222 |     ma_ptr pPulseContext;
31223 |
31224 |     MA_ASSERT(ppMainLoop != NULL);
31225 |     MA_ASSERT(ppPulseContext != NULL);
31226 |
31227 |     /* The PulseAudio context maps well to miniaudio's notion of a context. The pa_context object
> will be initialized as part of the ma_context. */
31228 |     pMainLoop = ((ma_pa_mainloop_new_proc)pContext->pulse.pa_mainloop_new)();
31229 |     if (pMainLoop == NULL) {
31230 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> create mainloop.");
31231 |         return MA_FAILED_TO_INIT_BACKEND;
31232 |     }
31233 |
31234 |     pPulseContext = ((ma_pa_context_new_proc)pContext->pulse.pa_context_new)((ma_pa_mainloop_get_ap
> i_proc)pContext->pulse.pa_mainloop_get_api)((ma_pa_mainloop*)pMainLoop), pApplicationName);
31235 |     if (pPulseContext == NULL) {
31236 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> create PulseAudio context.");
31237 |         ((ma_pa_mainloop_free_proc)pContext->pulse.pa_mainloop_free)((ma_pa_mainloop*)(pMainLoop));
31238 |         return MA_FAILED_TO_INIT_BACKEND;
31239 |     }
31240 |
31241 |     /* Now we need to connect to the context. Everything is asynchronous so we need to wait for it
> to connect before returning. */
31242 |     result = ma_result_from_pulse(((ma_pa_context_connect_proc)pContext->pulse.pa_context_connect)((
> ma_pa_context*)pPulseContext, pServerName, (tryAutoSpawn) ? MA_PA_CONTEXT_NOFLAGS :
> MA_PA_CONTEXT_NOAUTOSPAWN, NULL));
31243 |     if (result != MA_SUCCESS) {
31244 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> connect PulseAudio context.");
31245 |
> ((ma_pa_context_unref_proc)pContext->pulse.pa_context_unref)((ma_pa_context*)(pPulseContext));
31246 |         ((ma_pa_mainloop_free_proc)pContext->pulse.pa_mainloop_free)((ma_pa_mainloop*)(pMainLoop));
31247 |         return result;
31248 |     }
31249 |
31250 |     /* Since ma_context_init() runs synchronously we need to wait for the PulseAudio context to
> connect before we return. */
31251 |     result = ma_wait_for_pa_context_to_connect__pulse(pContext, pMainLoop, pPulseContext);
31252 |     if (result != MA_SUCCESS) {
31253 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[PulseAudio] Waiting for
> connection failed.");
31254 |
> ((ma_pa_context_unref_proc)pContext->pulse.pa_context_unref)((ma_pa_context*)(pPulseContext));
31255 |         ((ma_pa_mainloop_free_proc)pContext->pulse.pa_mainloop_free)((ma_pa_mainloop*)(pMainLoop));
31256 |         return result;
31257 |     }
31258 |
31259 |     *ppMainLoop = pMainLoop;
31260 |     *ppPulseContext = pPulseContext;
31261 |
31262 |     return MA_SUCCESS;
31263 | }
31264 |
31265 |
31266 | static void ma_device_sink_info_callback(ma_pa_context* pPulseContext, const ma_pa_sink_info* pInfo,
> int endOfList, void* pUserData)
31267 | {
31268 |     ma_pa_sink_info* pInfoOut;
31269 |
31270 |     if (endOfList > 0) {
31271 |         return;
31272 |     }
31273 |
31274 |     /*
31275 |     There has been a report that indicates that pInfo can be null which results
31276 |     in a null pointer dereference below. We'll check for this for safety.
31277 |     */
31278 |     if (pInfo == NULL) {
31279 |         return;
31280 |     }
31281 |
31282 |     pInfoOut = (ma_pa_sink_info*)pUserData;
31283 |     MA_ASSERT(pInfoOut != NULL);
31284 |

```

```

31285 |     *pInfoOut = *pInfo;
31286 |
31287 |     (void)pPulseContext; /* Unused. */
31288 | }
31289 |
31290 | static void ma_device_source_info_callback(ma_pa_context* pPulseContext, const ma_pa_source_info*
> pInfo, int endOfList, void* pUserData)
31291 | {
31292 |     ma_pa_source_info* pInfoOut;
31293 |
31294 |     if (endOfList > 0) {
31295 |         return;
31296 |     }
31297 |
31298 |     /*
31299 |     There has been a report that indicates that pInfo can be null which results
31300 |     in a null pointer dereference below. We'll check for this for safety.
31301 |     */
31302 |     if (pInfo == NULL) {
31303 |         return;
31304 |     }
31305 |
31306 |     pInfoOut = (ma_pa_source_info*)pUserData;
31307 |     MA_ASSERT(pInfoOut != NULL);
31308 |
31309 |     *pInfoOut = *pInfo;
31310 |
31311 |     (void)pPulseContext; /* Unused. */
31312 | }
31313 |
31314 | #if 0
31315 | static void ma_device_sink_name_callback(ma_pa_context* pPulseContext, const ma_pa_sink_info* pInfo,
> int endOfList, void* pUserData)
31316 | {
31317 |     ma_device* pDevice;
31318 |
31319 |     if (endOfList > 0) {
31320 |         return;
31321 |     }
31322 |
31323 |     pDevice = (ma_device*)pUserData;
31324 |     MA_ASSERT(pDevice != NULL);
31325 |
31326 |     ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name), pInfo->description,
> (size_t)-1);
31327 |
31328 |     (void)pPulseContext; /* Unused. */
31329 | }
31330 |
31331 | static void ma_device_source_name_callback(ma_pa_context* pPulseContext, const ma_pa_source_info*
> pInfo, int endOfList, void* pUserData)
31332 | {
31333 |     ma_device* pDevice;
31334 |
31335 |     if (endOfList > 0) {
31336 |         return;
31337 |     }
31338 |
31339 |     pDevice = (ma_device*)pUserData;
31340 |     MA_ASSERT(pDevice != NULL);
31341 |
31342 |     ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name), pInfo->description,
> (size_t)-1);
31343 |
31344 |     (void)pPulseContext; /* Unused. */
31345 | }
31346 | #endif
31347 |
31348 | static ma_result ma_context_get_sink_info_pulse(ma_context* pContext, const char* pDeviceName,
> ma_pa_sink_info* pSinkInfo)
31349 | {
31350 |     ma_pa_operation* pOP;
31351 |
31352 |     pOP = ((ma_pa_context_get_sink_info_by_name_proc)pContext->pulse.pa_context_get_sink_info_by_nam
> e)((ma_pa_context*)pContext->pulse.pPulseContext, pDeviceName, ma_device_sink_info_callback,
> pSinkInfo);
31353 |     if (pOP == NULL) {
31354 |         return MA_ERROR;
31355 |     }

```

```

31356 |
31357 |     return ma_wait_for_operation_and_unref__pulse(pContext, pContext->pulse.pMainLoop, pOP);
31358 | }
31359 |
31360 | static ma_result ma_context_get_source_info__pulse(ma_context* pContext, const char* pDeviceName,
> ma_pa_source_info* pSourceInfo)
31361 | {
31362 |     ma_pa_operation* pOP;
31363 |
31364 |     pOP = ((ma_pa_context_get_source_info_by_name_proc)pContext->pulse.pa_context_get_source_info_by
> _name)((ma_pa_context*)pContext->pulse.pPulseContext, pDeviceName, ma_device_source_info_callback,
> pSourceInfo);
31365 |     if (pOP == NULL) {
31366 |         return MA_ERROR;
31367 |     }
31368 |
31369 |     return ma_wait_for_operation_and_unref__pulse(pContext, pContext->pulse.pMainLoop, pOP);
31370 | }
31371 |
31372 | static ma_result ma_context_get_default_device_index__pulse(ma_context* pContext, ma_device_type
> deviceType, ma_uint32* pIndex)
31373 | {
31374 |     ma_result result;
31375 |
31376 |     MA_ASSERT(pContext != NULL);
31377 |     MA_ASSERT(pIndex != NULL);
31378 |
31379 |     if (pIndex != NULL) {
31380 |         *pIndex = (ma_uint32)-1;
31381 |     }
31382 |
31383 |     if (deviceType == ma_device_type_playback) {
31384 |         ma_pa_sink_info sinkInfo;
31385 |         result = ma_context_get_sink_info__pulse(pContext, NULL, &sinkInfo);
31386 |         if (result != MA_SUCCESS) {
31387 |             return result;
31388 |         }
31389 |
31390 |         if (pIndex != NULL) {
31391 |             *pIndex = sinkInfo.index;
31392 |         }
31393 |     }
31394 |
31395 |     if (deviceType == ma_device_type_capture) {
31396 |         ma_pa_source_info sourceInfo;
31397 |         result = ma_context_get_source_info__pulse(pContext, NULL, &sourceInfo);
31398 |         if (result != MA_SUCCESS) {
31399 |             return result;
31400 |         }
31401 |
31402 |         if (pIndex != NULL) {
31403 |             *pIndex = sourceInfo.index;
31404 |         }
31405 |     }
31406 |
31407 |     return MA_SUCCESS;
31408 | }
31409 |
31410 |
31411 | typedef struct
31412 | {
31413 |     ma_context* pContext;
31414 |     ma_enum_devices_callback_proc callback;
31415 |     void* pUserData;
31416 |     ma_bool32 isTerminated;
31417 |     ma_uint32 defaultDeviceIndexPlayback;
31418 |     ma_uint32 defaultDeviceIndexCapture;
31419 | } ma_context_enumerate_devices_callback_data__pulse;
31420 |
31421 | static void ma_context_enumerate_devices_sink_callback__pulse(ma_pa_context* pPulseContext, const
> ma_pa_sink_info* pSinkInfo, int endOfList, void* pUserData)
31422 | {
31423 |     ma_context_enumerate_devices_callback_data__pulse* pData =
> (ma_context_enumerate_devices_callback_data__pulse*)pUserData;
31424 |     ma_device_info deviceInfo;
31425 |
31426 |     MA_ASSERT(pData != NULL);
31427 |
31428 |     if (endOfList || pData->isTerminated) {

```

```

31429 |         return;
31430 |     }
31431 |
31432 |     MA_ZERO_OBJECT(&deviceInfo);
31433 |
31434 |     /* The name from PulseAudio is the ID for miniaudio. */
31435 |     if (pSinkInfo->name != NULL) {
31436 |         ma_strncpy_s(deviceInfo.id.pulse, sizeof(deviceInfo.id.pulse), pSinkInfo->name, (size_t)-1);
31437 |     }
31438 |
31439 |     /* The description from PulseAudio is the name for miniaudio. */
31440 |     if (pSinkInfo->description != NULL) {
31441 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), pSinkInfo->description, (size_t)-1);
31442 |     }
31443 |
31444 |     if (pSinkInfo->index == pData->defaultDeviceIndexPlayback) {
31445 |         deviceInfo.isDefault = MA_TRUE;
31446 |     }
31447 |
31448 |     pData->isTerminated = !pData->callback(pData->pContext, ma_device_type_playback, &deviceInfo,
> pData->pUserData);
31449 |
31450 |     (void)pPulseContext; /* Unused. */
31451 | }
31452 |
31453 | static void ma_context_enumerate_devices_source_callback_pulse(ma_pa_context* pPulseContext, const
> ma_pa_source_info* pSourceInfo, int endOfList, void* pUserData)
31454 | {
31455 |     ma_context_enumerate_devices_callback_data_pulse* pData =
> (ma_context_enumerate_devices_callback_data_pulse*)pUserData;
31456 |     ma_device_info deviceInfo;
31457 |
31458 |     MA_ASSERT(pData != NULL);
31459 |
31460 |     if (endOfList || pData->isTerminated) {
31461 |         return;
31462 |     }
31463 |
31464 |     MA_ZERO_OBJECT(&deviceInfo);
31465 |
31466 |     /* The name from PulseAudio is the ID for miniaudio. */
31467 |     if (pSourceInfo->name != NULL) {
31468 |         ma_strncpy_s(deviceInfo.id.pulse, sizeof(deviceInfo.id.pulse), pSourceInfo->name,
> (size_t)-1);
31469 |     }
31470 |
31471 |     /* The description from PulseAudio is the name for miniaudio. */
31472 |     if (pSourceInfo->description != NULL) {
31473 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), pSourceInfo->description,
> (size_t)-1);
31474 |     }
31475 |
31476 |     if (pSourceInfo->index == pData->defaultDeviceIndexCapture) {
31477 |         deviceInfo.isDefault = MA_TRUE;
31478 |     }
31479 |
31480 |     pData->isTerminated = !pData->callback(pData->pContext, ma_device_type_capture, &deviceInfo,
> pData->pUserData);
31481 |
31482 |     (void)pPulseContext; /* Unused. */
31483 | }
31484 |
31485 | static ma_result ma_context_enumerate_devices_pulse(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
31486 | {
31487 |     ma_result result = MA_SUCCESS;
31488 |     ma_context_enumerate_devices_callback_data_pulse callbackData;
31489 |     ma_pa_operation* pOP = NULL;
31490 |
31491 |     MA_ASSERT(pContext != NULL);
31492 |     MA_ASSERT(callback != NULL);
31493 |
31494 |     callbackData.pContext = pContext;
31495 |     callbackData.callback = callback;
31496 |     callbackData.pUserData = pUserData;
31497 |     callbackData.isTerminated = MA_FALSE;
31498 |     callbackData.defaultDeviceIndexPlayback = (ma_uint32)-1;
31499 |     callbackData.defaultDeviceIndexCapture = (ma_uint32)-1;
31500 |

```

```

31501 |     /* We need to get the index of the default devices. */
31502 |     ma_context_get_default_device_index__pulse(pContext, ma_device_type_playback,
> &callbackData.defaultDeviceIndexPlayback);
31503 |     ma_context_get_default_device_index__pulse(pContext, ma_device_type_capture,
> &callbackData.defaultDeviceIndexCapture);
31504 |
31505 |     /* Playback. */
31506 |     if (!callbackData.isTerminated) {
31507 |         pOP = ((ma_pa_context_get_sink_info_list_proc)pContext->pulse.pa_context_get_sink_info_list)
> ((ma_pa_context*)(pContext->pulse.pPulseContext), ma_context_enumerate_devices_sink_callback__pulse,
> &callbackData);
31508 |         if (pOP == NULL) {
31509 |             result = MA_ERROR;
31510 |             goto done;
31511 |         }
31512 |
31513 |         result = ma_wait_for_operation__pulse(pContext, pContext->pulse.pMainLoop, pOP);
31514 |         ((ma_pa_operation_unref_proc)pContext->pulse.pa_operation_unref)(pOP);
31515 |
31516 |         if (result != MA_SUCCESS) {
31517 |             goto done;
31518 |         }
31519 |     }
31520 |
31521 |
31522 |     /* Capture. */
31523 |     if (!callbackData.isTerminated) {
31524 |         pOP = ((ma_pa_context_get_source_info_list_proc)pContext->pulse.pa_context_get_source_info_l
> ist)((ma_pa_context*)(pContext->pulse.pPulseContext),
> ma_context_enumerate_devices_source_callback__pulse, &callbackData);
31525 |         if (pOP == NULL) {
31526 |             result = MA_ERROR;
31527 |             goto done;
31528 |         }
31529 |
31530 |         result = ma_wait_for_operation__pulse(pContext, pContext->pulse.pMainLoop, pOP);
31531 |         ((ma_pa_operation_unref_proc)pContext->pulse.pa_operation_unref)(pOP);
31532 |
31533 |         if (result != MA_SUCCESS) {
31534 |             goto done;
31535 |         }
31536 |     }
31537 |
31538 | done:
31539 |     return result;
31540 | }
31541 |
31542 |
31543 | typedef struct
31544 | {
31545 |     ma_device_info* pDeviceInfo;
31546 |     ma_uint32 defaultDeviceIndex;
31547 |     ma_bool32 foundDevice;
31548 | } ma_context_get_device_info_callback_data__pulse;
31549 |
31550 | static void ma_context_get_device_info_sink_callback__pulse(ma_pa_context* pPulseContext, const
> ma_pa_sink_info* pInfo, int endOfList, void* pUserData)
31551 | {
31552 |     ma_context_get_device_info_callback_data__pulse* pData =
> (ma_context_get_device_info_callback_data__pulse*)pUserData;
31553 |
31554 |     if (endOfList > 0) {
31555 |         return;
31556 |     }
31557 |
31558 |     MA_ASSERT(pData != NULL);
31559 |     pData->foundDevice = MA_TRUE;
31560 |
31561 |     if (pInfo->name != NULL) {
31562 |         ma_strncpy_s(pData->pDeviceInfo->id.pulse, sizeof(pData->pDeviceInfo->id.pulse),
> pInfo->name, (size_t)-1);
31563 |     }
31564 |
31565 |     if (pInfo->description != NULL) {
31566 |         ma_strncpy_s(pData->pDeviceInfo->name, sizeof(pData->pDeviceInfo->name), pInfo->description,
> (size_t)-1);
31567 |     }
31568 |
31569 |     /*

```



```

31570 |     We're just reporting a single data format here. I think technically PulseAudio might support
31571 |     all formats, but I don't trust that PulseAudio will do *anything* right, so I'm just going to
31572 |     report the "native" device format.
31573 |     */
31574 |     pData->pDeviceInfo->nativeDataFormats[0].format      =
> ma_format_from_pulse(pInfo->sample_spec.format);
31575 |     pData->pDeviceInfo->nativeDataFormats[0].channels    = pInfo->sample_spec.channels;
31576 |     pData->pDeviceInfo->nativeDataFormats[0].sampleRate = pInfo->sample_spec.rate;
31577 |     pData->pDeviceInfo->nativeDataFormats[0].flags      = 0;
31578 |     pData->pDeviceInfo->nativeDataFormatCount = 1;
31579 |
31580 |     if (pData->defaultDeviceIndex == pInfo->index) {
31581 |         pData->pDeviceInfo->isDefault = MA_TRUE;
31582 |     }
31583 |
31584 |     (void)pPulseContext; /* Unused. */
31585 | }
31586 |
31587 | static void ma_context_get_device_info_source_callback__pulse(ma_pa_context* pPulseContext, const
> ma_pa_source_info* pInfo, int endOfList, void* pUserData)
31588 | {
31589 |     ma_context_get_device_info_callback_data__pulse* pData =
> (ma_context_get_device_info_callback_data__pulse*)pUserData;
31590 |
31591 |     if (endOfList > 0) {
31592 |         return;
31593 |     }
31594 |
31595 |     MA_ASSERT(pData != NULL);
31596 |     pData->foundDevice = MA_TRUE;
31597 |
31598 |     if (pInfo->name != NULL) {
31599 |         ma_strncpy_s(pData->pDeviceInfo->id.pulse, sizeof(pData->pDeviceInfo->id.pulse),
> pInfo->name, (size_t)-1);
31600 |     }
31601 |
31602 |     if (pInfo->description != NULL) {
31603 |         ma_strncpy_s(pData->pDeviceInfo->name, sizeof(pData->pDeviceInfo->name), pInfo->description,
> (size_t)-1);
31604 |     }
31605 |
31606 |     /*
31607 |     We're just reporting a single data format here. I think technically PulseAudio might support
31608 |     all formats, but I don't trust that PulseAudio will do *anything* right, so I'm just going to
31609 |     report the "native" device format.
31610 |     */
31611 |     pData->pDeviceInfo->nativeDataFormats[0].format      =
> ma_format_from_pulse(pInfo->sample_spec.format);
31612 |     pData->pDeviceInfo->nativeDataFormats[0].channels    = pInfo->sample_spec.channels;
31613 |     pData->pDeviceInfo->nativeDataFormats[0].sampleRate = pInfo->sample_spec.rate;
31614 |     pData->pDeviceInfo->nativeDataFormats[0].flags      = 0;
31615 |     pData->pDeviceInfo->nativeDataFormatCount = 1;
31616 |
31617 |     if (pData->defaultDeviceIndex == pInfo->index) {
31618 |         pData->pDeviceInfo->isDefault = MA_TRUE;
31619 |     }
31620 |
31621 |     (void)pPulseContext; /* Unused. */
31622 | }
31623 |
31624 | static ma_result ma_context_get_device_info__pulse(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
31625 | {
31626 |     ma_result result = MA_SUCCESS;
31627 |     ma_context_get_device_info_callback_data__pulse callbackData;
31628 |     ma_pa_operation* pOP = NULL;
31629 |     const char* pDeviceName = NULL;
31630 |
31631 |     MA_ASSERT(pContext != NULL);
31632 |
31633 |     callbackData.pDeviceInfo = pDeviceInfo;
31634 |     callbackData.foundDevice = MA_FALSE;
31635 |
31636 |     if (pDeviceID != NULL) {
31637 |         pDeviceName = pDeviceID->pulse;
31638 |     } else {
31639 |         pDeviceName = NULL;
31640 |     }
31641 |

```

```

31642 |     result = ma_context_get_default_device_index__pulse(pContext, deviceType,
> &callbackData.defaultDeviceIndex);
31643 |
31644 |     if (deviceType == ma_device_type_playback) {
31645 |         pOP = ((ma_pa_context_get_sink_info_by_name_proc)pContext->pulse.pa_context_get_sink_info_by
> _name)((ma_pa_context*)(pContext->pulse.pPulseContext), pDeviceName,
> ma_context_get_device_info_sink_callback__pulse, &callbackData);
31646 |     } else {
31647 |         pOP = ((ma_pa_context_get_source_info_by_name_proc)pContext->pulse.pa_context_get_source_inf
> o_by_name)((ma_pa_context*)(pContext->pulse.pPulseContext), pDeviceName,
> ma_context_get_device_info_source_callback__pulse, &callbackData);
31648 |     }
31649 |
31650 |     if (pOP != NULL) {
31651 |         ma_wait_for_operation_and_unref__pulse(pContext, pContext->pulse.pMainLoop, pOP);
31652 |     } else {
31653 |         result = MA_ERROR;
31654 |         goto done;
31655 |     }
31656 |
31657 |     if (!callbackData.foundDevice) {
31658 |         result = MA_NO_DEVICE;
31659 |         goto done;
31660 |     }
31661 |
31662 | done:
31663 |     return result;
31664 | }
31665 |
31666 | static ma_result ma_device_uninit__pulse(ma_device* pDevice)
31667 | {
31668 |     ma_context* pContext;
31669 |
31670 |     MA_ASSERT(pDevice != NULL);
31671 |
31672 |     pContext = pDevice->pContext;
31673 |     MA_ASSERT(pContext != NULL);
31674 |
31675 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
31676 |         ((ma_pa_stream_disconnect_proc)pContext->pulse.pa_stream_disconnect)((ma_pa_stream*)pDevice->
> pulse.pStreamCapture);
31677 |         ((ma_pa_stream_unref_proc)pContext->pulse.pa_stream_unref)((ma_pa_stream*)pDevice->pulse.pSt
> reamCapture);
31678 |     }
31679 |
31680 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
31681 |         ((ma_pa_stream_disconnect_proc)pContext->pulse.pa_stream_disconnect)((ma_pa_stream*)pDevice->
> pulse.pStreamPlayback);
31682 |         ((ma_pa_stream_unref_proc)pContext->pulse.pa_stream_unref)((ma_pa_stream*)pDevice->pulse.pSt
> reamPlayback);
31683 |     }
31684 |
31685 |     if (pDevice->type == ma_device_type_duplex) {
31686 |         ma_duplex_rb_uninit(&pDevice->duplexRB);
31687 |     }
31688 |
31689 |     ((ma_pa_context_disconnect_proc)pContext->pulse.pa_context_disconnect)((ma_pa_context*)pDevice->
> pulse.pPulseContext);
31690 |     ((ma_pa_context_unref_proc)pContext->pulse.pa_context_unref)((ma_pa_context*)pDevice->pulse.pPul
> seContext);
31691 |     ((ma_pa_mainloop_free_proc)pContext->pulse.pa_mainloop_free)((ma_pa_mainloop*)pDevice->pulse.pMa
> inLoop);
31692 |
31693 |     return MA_SUCCESS;
31694 | }
31695 |
31696 | static ma_pa_buffer_attr ma_device__pa_buffer_attr_new(ma_uint32 periodSizeInFrames, ma_uint32
> periods, const ma_pa_sample_spec* ss)
31697 | {
31698 |     ma_pa_buffer_attr attr;
31699 |     attr.maxlength = periodSizeInFrames * periods *
> ma_get_bytes_per_frame(ma_format_from_pulse(ss->format), ss->channels);
31700 |     attr.tlength = attr.maxlength / periods;
31701 |     attr.prebuf = (ma_uint32)-1;
31702 |     attr.minreq = (ma_uint32)-1;
31703 |     attr.fragsize = attr.maxlength / periods;
31704 |
31705 |     return attr;
31706 | }

```

```

31707 |
31708 | static ma_pa_stream* ma_device__pa_stream_new_pulse(ma_device* pDevice, const char* pStreamName,
> const ma_pa_sample_spec* ss, const ma_pa_channel_map* cmap)
31709 | {
31710 |     static ma_atomic_uint32 g_StreamCounter = { 0 };
31711 |     char actualStreamName[256];
31712 |
31713 |     if (pStreamName != NULL) {
31714 |         ma_strncpy_s(actualStreamName, sizeof(actualStreamName), pStreamName, (size_t)-1);
31715 |     } else {
31716 |         const char* pBaseName = "miniaudio:";
31717 |         size_t baseNameLen = strlen(pBaseName);
31718 |         ma_strcpy_s(actualStreamName, sizeof(actualStreamName), pBaseName);
31719 |         ma_itoa_s((int)ma_atomic_uint32_get(&g_StreamCounter), actualStreamName + baseNameLen,
> sizeof(actualStreamName)-baseNameLen, 10);
31720 |     }
31721 |     ma_atomic_uint32_fetch_add(&g_StreamCounter, 1);
31722 |
31723 |     return ((ma_pa_stream_new_proc)pDevice->pContext->pulse.pa_stream_new)((ma_pa_context*)pDevice->
> pulse.pPulseContext, actualStreamName, ss, cmap);
31724 | }
31725 |
31726 |
31727 | static void ma_device_on_read_pulse(ma_pa_stream* pStream, size_t byteCount, void* pUserData)
31728 | {
31729 |     ma_device* pDevice = (ma_device*)pUserData;
31730 |     ma_uint32 bpf;
31731 |     ma_uint32 deviceState;
31732 |     ma_uint64 frameCount;
31733 |     ma_uint64 framesProcessed;
31734 |
31735 |     MA_ASSERT(pDevice != NULL);
31736 |
31737 |     /*
31738 |     Don't do anything if the device isn't initialized yet. Yes, this can happen because PulseAudio
31739 |     can fire this callback before the stream has even started. Ridiculous.
31740 |     */
31741 |     deviceState = ma_device_get_state(pDevice);
31742 |     if (deviceState != ma_device_state_starting && deviceState != ma_device_state_started) {
31743 |         return;
31744 |     }
31745 |
31746 |     bpf = ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels);
31747 |     MA_ASSERT(bpf > 0);
31748 |
31749 |     frameCount = byteCount / bpf;
31750 |     framesProcessed = 0;
31751 |
31752 |     while (ma_device_get_state(pDevice) == ma_device_state_started && framesProcessed < frameCount)
> {
31753 |         const void* pMappedPCMFrames;
31754 |         size_t bytesMapped;
31755 |         ma_uint64 framesMapped;
31756 |
31757 |         int pulseResult = ((ma_pa_stream_peek_proc)pDevice->pContext->pulse.pa_stream_peek)(pStream,
> &pMappedPCMFrames, &bytesMapped);
31758 |         if (pulseResult < 0) {
31759 |             break; /* Failed to map. Abort. */
31760 |         }
31761 |
31762 |         framesMapped = bytesMapped / bpf;
31763 |         if (framesMapped > 0) {
31764 |             if (pMappedPCMFrames != NULL) {
31765 |                 ma_device_handle_backend_data_callback(pDevice, NULL, pMappedPCMFrames,
> framesMapped);
31766 |             } else {
31767 |                 /* It's a hole. */
31768 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[PulseAudio]
> ma_device_on_read_pulse: Hole.\n");
31769 |             }
31770 |
31771 |             pulseResult =
> ((ma_pa_stream_drop_proc)pDevice->pContext->pulse.pa_stream_drop)(pStream);
31772 |             if (pulseResult < 0) {
31773 |                 break; /* Failed to drop the buffer. */
31774 |             }
31775 |
31776 |             framesProcessed += framesMapped;

```

```

31777 |
31778 |         } else {
31779 |             /* Nothing was mapped. Just abort. */
31780 |             break;
31781 |         }
31782 |     }
31783 | }
31784 |
31785 | static ma_result ma_device_write_to_stream__pulse(ma_device* pDevice, ma_pa_stream* pStream,
> ma_uint64* pFramesProcessed)
31786 | {
31787 |     ma_result result = MA_SUCCESS;
31788 |     ma_uint64 framesProcessed = 0;
31789 |     size_t bytesMapped;
31790 |     ma_uint32 bpf;
31791 |     ma_uint32 deviceState;
31792 |
31793 |     MA_ASSERT(pDevice != NULL);
31794 |     MA_ASSERT(pStream != NULL);
31795 |
31796 |     bpf = ma_get_bytes_per_frame(pDevice->playback.internalFormat,
> pDevice->playback.internalChannels);
31797 |     MA_ASSERT(bpf > 0);
31798 |
31799 |     deviceState = ma_device_get_state(pDevice);
31800 |
31801 |     bytesMapped =
> ((ma_pa_stream_writable_size_proc)pDevice->pContext->pulse.pa_stream_writable_size)(pStream);
31802 |     if (bytesMapped != (size_t)-1) {
31803 |         if (bytesMapped > 0) {
31804 |             ma_uint64 framesMapped;
31805 |             void* pMappedPCMFrames;
31806 |             int pulseResult =
> ((ma_pa_stream_begin_write_proc)pDevice->pContext->pulse.pa_stream_begin_write)(pStream,
> &pMappedPCMFrames, &bytesMapped);
31807 |             if (pulseResult < 0) {
31808 |                 result = ma_result_from_pulse(pulseResult);
31809 |                 goto done;
31810 |             }
31811 |
31812 |             framesMapped = bytesMapped / bpf;
31813 |
31814 |             if (deviceState == ma_device_state_started || deviceState == ma_device_state_starting) {
> /* Check for starting state just in case this is being used to do the initial fill. */
31815 |                 ma_device_handle_backend_data_callback(pDevice, pMappedPCMFrames, NULL,
> framesMapped);
31816 |             } else {
31817 |                 /* Device is not started. Write silence. */
31818 |                 ma_silence_pcm_frames(pMappedPCMFrames, framesMapped, pDevice->playback.format,
> pDevice->playback.channels);
31819 |             }
31820 |
31821 |             pulseResult =
> ((ma_pa_stream_write_proc)pDevice->pContext->pulse.pa_stream_write)(pStream, pMappedPCMFrames,
> bytesMapped, NULL, 0, MA_PA_SEEK_RELATIVE);
31822 |             if (pulseResult < 0) {
31823 |                 result = ma_result_from_pulse(pulseResult);
31824 |                 goto done; /* Failed to write data to stream. */
31825 |             }
31826 |
31827 |             framesProcessed += framesMapped;
31828 |         } else {
31829 |             result = MA_SUCCESS; /* No data available for writing. */
31830 |             goto done;
31831 |         }
31832 |     } else {
31833 |         result = MA_ERROR; /* Failed to retrieve the writable size. Abort. */
31834 |         goto done;
31835 |     }
31836 |
31837 | done:
31838 |     if (pFramesProcessed != NULL) {
31839 |         *pFramesProcessed = framesProcessed;
31840 |     }
31841 |
31842 |     return result;
31843 | }
31844 |
31845 | static void ma_device_on_write__pulse(ma_pa_stream* pStream, size_t byteCount, void* pUserData)

```

```

31846 | {
31847 |     ma_device* pDevice = (ma_device*)pUserData;
31848 |     ma_uint32 bpf;
31849 |     ma_uint64 frameCount;
31850 |     ma_uint64 framesProcessed;
31851 |     ma_uint32 deviceState;
31852 |     ma_result result;
31853 |
31854 |     MA_ASSERT(pDevice != NULL);
31855 |
31856 |     /*
31857 |     Don't do anything if the device isn't initialized yet. Yes, this can happen because PulseAudio
31858 |     can fire this callback before the stream has even started. Ridiculous.
31859 |     */
31860 |     deviceState = ma_device_get_state(pDevice);
31861 |     if (deviceState != ma_device_state_starting && deviceState != ma_device_state_started) {
31862 |         return;
31863 |     }
31864 |
31865 |     bpf = ma_get_bytes_per_frame(pDevice->playback.internalFormat,
> pDevice->playback.internalChannels);
31866 |     MA_ASSERT(bpf > 0);
31867 |
31868 |     frameCount = byteCount / bpf;
31869 |     framesProcessed = 0;
31870 |
31871 |     while (framesProcessed < frameCount) {
31872 |         ma_uint64 framesProcessedThisIteration;
31873 |
31874 |         /* Don't keep trying to process frames if the device isn't started. */
31875 |         deviceState = ma_device_get_state(pDevice);
31876 |         if (deviceState != ma_device_state_starting && deviceState != ma_device_state_started) {
31877 |             break;
31878 |         }
31879 |
31880 |         result = ma_device_write_to_stream__pulse(pDevice, pStream, &framesProcessedThisIteration);
31881 |         if (result != MA_SUCCESS) {
31882 |             break;
31883 |         }
31884 |
31885 |         framesProcessed += framesProcessedThisIteration;
31886 |     }
31887 | }
31888 |
31889 | static void ma_device_on_suspended__pulse(ma_pa_stream* pStream, void* pUserData)
31890 | {
31891 |     ma_device* pDevice = (ma_device*)pUserData;
31892 |     int suspended;
31893 |
31894 |     (void)pStream;
31895 |
31896 |     suspended =
> ((ma_pa_stream_is_suspended_proc)pDevice->pContext->pulse.pa_stream_is_suspended)(pStream);
31897 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[Pulse] Device suspended state
> changed. pa_stream_is_suspended() returned %d.\n", suspended);
31898 |
31899 |     if (suspended < 0) {
31900 |         return;
31901 |     }
31902 |
31903 |     if (suspended == 1) {
31904 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[Pulse] Device suspended state
> changed. Suspended.\n");
31905 |         ma_device__on_notification_stopped(pDevice);
31906 |     } else {
31907 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "[Pulse] Device suspended state
> changed. Resumed.\n");
31908 |         ma_device__on_notification_started(pDevice);
31909 |     }
31910 | }
31911 |
31912 | static void ma_device_on_rerouted__pulse(ma_pa_stream* pStream, void* pUserData)
31913 | {
31914 |     ma_device* pDevice = (ma_device*)pUserData;
31915 |
31916 |     (void)pStream;
31917 |     (void)pUserData;
31918 |
31919 |     ma_device__on_notification_rerouted(pDevice);

```

```

31920 | }
31921 |
31922 | static ma_uint32 ma_calculate_period_size_in_frames_from_descriptor__pulse(const
> ma_device_descriptor* pDescriptor, ma_uint32 nativeSampleRate, ma_performance_profile
> performanceProfile)
31923 | {
31924 |     /*
31925 |      * There have been reports from users where buffers of < ~20ms result glitches when running through
31926 |      * PipeWire. To work around this we're going to have to use a different default buffer size.
31927 |      */
31928 |     const ma_uint32 defaultPeriodSizeInMilliseconds_LowLatency = 25;
31929 |     const ma_uint32 defaultPeriodSizeInMilliseconds_Conservative =
> MA_DEFAULT_PERIOD_SIZE_IN_MILLISECONDS_CONSERVATIVE;
31930 |
31931 |     MA_ASSERT(nativeSampleRate != 0);
31932 |
31933 |     if (pDescriptor->periodSizeInFrames == 0) {
31934 |         if (pDescriptor->periodSizeInMilliseconds == 0) {
31935 |             if (performanceProfile == ma_performance_profile_low_latency) {
31936 |                 return
> ma_calculate_buffer_size_in_frames_from_milliseconds(defaultPeriodSizeInMilliseconds_LowLatency,
> nativeSampleRate);
31937 |             } else {
31938 |                 return
> ma_calculate_buffer_size_in_frames_from_milliseconds(defaultPeriodSizeInMilliseconds_Conservative,
> nativeSampleRate);
31939 |             }
31940 |         } else {
31941 |             return
> ma_calculate_buffer_size_in_frames_from_milliseconds(pDescriptor->periodSizeInMilliseconds,
> nativeSampleRate);
31942 |         }
31943 |     } else {
31944 |         return pDescriptor->periodSizeInFrames;
31945 |     }
31946 | }
31947 |
31948 | static ma_result ma_device_init__pulse(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
31949 | {
31950 |     /*
31951 |      * Notes for PulseAudio:
31952 |
31953 |      * - When both the period size in frames and milliseconds are 0, we default to miniaudio's
31954 |      *   default buffer sizes rather than leaving it up to PulseAudio because I don't trust
31955 |      *   PulseAudio to give us any kind of reasonable latency by default.
31956 |
31957 |      * - Do not ever, *ever* forget to use MA_PA_STREAM_ADJUST_LATENCY. If you don't specify this
31958 |      *   flag, capture mode will just not work properly until you open another PulseAudio app.
31959 |      */
31960 |
31961 |     ma_result result = MA_SUCCESS;
31962 |     int error = 0;
31963 |     const char* devPlayback = NULL;
31964 |     const char* devCapture = NULL;
31965 |     ma_format format = ma_format_unknown;
31966 |     ma_uint32 channels = 0;
31967 |     ma_uint32 sampleRate = 0;
31968 |     ma_pa_sink_info sinkInfo;
31969 |     ma_pa_source_info sourceInfo;
31970 |     ma_pa_sample_spec ss;
31971 |     ma_pa_channel_map cmap;
31972 |     ma_pa_buffer_attr attr;
31973 |     const ma_pa_sample_spec* pActualSS = NULL;
31974 |     const ma_pa_buffer_attr* pActualAttr = NULL;
31975 |     const ma_pa_channel_map* pActualChannelMap = NULL;
31976 |     ma_uint32 iChannel;
31977 |     int streamFlags;
31978 |
31979 |     MA_ASSERT(pDevice != NULL);
31980 |     MA_ZERO_OBJECT(&pDevice->pulse);
31981 |
31982 |     if (pConfig->deviceType == ma_device_type_loopback) {
31983 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
31984 |     }
31985 |
31986 |     /* No exclusive mode with the PulseAudio backend. */
31987 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pConfig->playback.shareMode == ma_share_mode_exclusive) ||

```

```

31988 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pConfig->capture.shareMode == ma_share_mode_exclusive)) {
31989 |             return MA_SHARE_MODE_NOT_SUPPORTED;
31990 |         }
31991 |
31992 |         if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
31993 |             if (pDescriptorPlayback->pDeviceID != NULL) {
31994 |                 devPlayback = pDescriptorPlayback->pDeviceID->pulse;
31995 |             }
31996 |
31997 |             format      = pDescriptorPlayback->format;
31998 |             channels     = pDescriptorPlayback->channels;
31999 |             sampleRate   = pDescriptorPlayback->sampleRate;
32000 |         }
32001 |
32002 |         if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
32003 |             if (pDescriptorCapture->pDeviceID != NULL) {
32004 |                 devCapture = pDescriptorCapture->pDeviceID->pulse;
32005 |             }
32006 |
32007 |             format      = pDescriptorCapture->format;
32008 |             channels     = pDescriptorCapture->channels;
32009 |             sampleRate   = pDescriptorCapture->sampleRate;
32010 |         }
32011 |
32012 |
32013 |
32014 |         result = ma_init_pa_mainloop_and_pa_context__pulse(pDevice->pContext,
> pDevice->pContext->pulse.pApplicationName, pDevice->pContext->pulse.pServerName, MA_FALSE,
> &pDevice->pulse.pMainLoop, &pDevice->pulse.pPulseContext);
32015 |         if (result != MA_SUCCESS) {
32016 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> initialize PA mainloop and context for device.\n");
32017 |             return result;
32018 |         }
32019 |
32020 |         if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
32021 |             result = ma_context_get_source_info__pulse(pDevice->pContext, devCapture, &sourceInfo);
32022 |             if (result != MA_SUCCESS) {
32023 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> retrieve source info for capture device.");
32024 |                 goto on_error0;
32025 |             }
32026 |
32027 |             ss = sourceInfo.sample_spec;
32028 |             cmap = sourceInfo.channel_map;
32029 |
32030 |             /* Use the requested channel count if we have one. */
32031 |             if (pDescriptorCapture->channels != 0) {
32032 |                 ss.channels = pDescriptorCapture->channels;
32033 |             }
32034 |
32035 |             /* PulseAudio has a maximum channel count of 32. We'll get a crash if this is exceeded. */
32036 |             if (ss.channels > 32) {
32037 |                 ss.channels = 32;
32038 |             }
32039 |
32040 |             /* Use a default channel map. */
32041 |
> ((ma_pa_channel_map_init_extend_proc)pDevice->pContext->pulse.pa_channel_map_init_extend>(&cmap,
> ss.channels, (ma_pa_channel_map_def_t)pConfig->pulse.channelMap);
32042 |
32043 |             /* Use the requested sample rate if one was specified. */
32044 |             if (pDescriptorCapture->sampleRate != 0) {
32045 |                 ss.rate = pDescriptorCapture->sampleRate;
32046 |             }
32047 |             streamFlags = MA_PA_STREAM_START_CORKED | MA_PA_STREAM_ADJUST_LATENCY;
32048 |
32049 |             if (ma_format_from_pulse(ss.format) == ma_format_unknown) {
32050 |                 if (ma_is_little_endian()) {
32051 |                     ss.format = MA_PA_SAMPLE_FLOAT32LE;
32052 |                 } else {
32053 |                     ss.format = MA_PA_SAMPLE_FLOAT32BE;
32054 |                 }
32055 |                 streamFlags |= MA_PA_STREAM_FIX_FORMAT;
32056 |                 ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio]

```



```

> sample_spec.format not supported by miniaudio. Defaulting to PA_SAMPLE_FLOAT32.\n");
32057 |     }
32058 |     if (ss.rate == 0) {
32059 |         ss.rate = MA_DEFAULT_SAMPLE_RATE;
32060 |         streamFlags |= MA_PA_STREAM_FIX_RATE;
32061 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio]
> sample_spec.rate = 0. Defaulting to %d.\n", ss.rate);
32062 |     }
32063 |     if (ss.channels == 0) {
32064 |         ss.channels = MA_DEFAULT_CHANNELS;
32065 |         streamFlags |= MA_PA_STREAM_FIX_CHANNELS;
32066 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio]
> sample_spec.channels = 0. Defaulting to %d.\n", ss.channels);
32067 |     }
32068 |
32069 |     /* We now have enough information to calculate our actual period size in frames. */
32070 |     pDescriptorCapture->periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__pulse(pDescriptorCapture, ss.rate,
> pConfig->performanceProfile);
32071 |
32072 |     attr = ma_device__pa_buffer_attr_new(pDescriptorCapture->periodSizeInFrames,
> pDescriptorCapture->periodCount, &ss);
32073 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Capture attr:
> maxlength=%d, tlength=%d, prebuf=%d, minreq=%d, fragsize=%d; periodSizeInFrames=%d\n",
> attr.maxlength, attr.tlength, attr.prebuf, attr.minreq, attr.fragsize,
> pDescriptorCapture->periodSizeInFrames);
32074 |
32075 |     pDevice->pulse.pStreamCapture = ma_device__pa_stream_new__pulse(pDevice,
> pConfig->pulse.pStreamNameCapture, &ss, &cmap);
32076 |     if (pDevice->pulse.pStreamCapture == NULL) {
32077 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> create PulseAudio capture stream.\n");
32078 |         result = MA_ERROR;
32079 |         goto on_error0;
32080 |     }
32081 |
32082 |
32083 |     /* The callback needs to be set before connecting the stream. */
32084 |     ((ma_pa_stream_set_read_callback_proc)pDevice->pContext->pulse.pa_stream_set_read_callback)(
> (ma_pa_stream*)pDevice->pulse.pStreamCapture, ma_device_on_read__pulse, pDevice);
32085 |
32086 |     /* State callback for checking when the device has been corked. */
32087 |     ((ma_pa_stream_set_suspended_callback_proc)pDevice->pContext->pulse.pa_stream_set_suspended_
> callback)((ma_pa_stream*)pDevice->pulse.pStreamCapture, ma_device_on_suspended__pulse, pDevice);
32088 |
32089 |     /* Rerouting notification. */
32090 |     ((ma_pa_stream_set_moved_callback_proc)pDevice->pContext->pulse.pa_stream_set_moved_callback
> )((ma_pa_stream*)pDevice->pulse.pStreamCapture, ma_device_on_rerouted__pulse, pDevice);
32091 |
32092 |
32093 |     /* Connect after we've got all of our internal state set up. */
32094 |     if (devCapture != NULL) {
32095 |         streamFlags |= MA_PA_STREAM_DONT_MOVE;
32096 |     }
32097 |
32098 |     error = ((ma_pa_stream_connect_record_proc)pDevice->pContext->pulse.pa_stream_connect_record
> )((ma_pa_stream*)pDevice->pulse.pStreamCapture, devCapture, &attr,
> (ma_pa_stream_flags_t)streamFlags);
32099 |     if (error != MA_PA_OK) {
32100 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> connect PulseAudio capture stream.");
32101 |         result = ma_result_from_pulse(error);
32102 |         goto on_error1;
32103 |     }
32104 |
32105 |     result = ma_wait_for_pa_stream_to_connect__pulse(pDevice->pContext,
> pDevice->pulse.pMainLoop, (ma_pa_stream*)pDevice->pulse.pStreamCapture);
32106 |     if (result != MA_SUCCESS) {
32107 |         goto on_error2;
32108 |     }
32109 |
32110 |
32111 |     /* Internal format. */
32112 |     pActualSS = ((ma_pa_stream_get_sample_spec_proc)pDevice->pContext->pulse.pa_stream_get_sampl
> e_spec)((ma_pa_stream*)pDevice->pulse.pStreamCapture);
32113 |     if (pActualSS != NULL) {
32114 |         ss = *pActualSS;
32115 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Capture sample
> spec: format=%s, channels=%d, rate=%d\n", ma_get_format_name(ma_format_from_pulse(ss.format)),

```

```

> ss.channels, ss.rate);
32116 |     } else {
32117 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Failed to
> retrieve capture sample spec.\n");
32118 |     }
32119 |
32120 |     pDescriptorCapture->format      = ma_format_from_pulse(ss.format);
32121 |     pDescriptorCapture->channels    = ss.channels;
32122 |     pDescriptorCapture->sampleRate = ss.rate;
32123 |
32124 |     if (pDescriptorCapture->format == ma_format_unknown || pDescriptorCapture->channels == 0 ||
> pDescriptorCapture->sampleRate == 0) {
32125 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Capture
> sample spec is invalid. Device unusable by miniaudio. format=%s, channels=%d, sampleRate=%d.\n",
> ma_get_format_name(pDescriptorCapture->format), pDescriptorCapture->channels,
> pDescriptorCapture->sampleRate);
32126 |         result = MA_ERROR;
32127 |         goto on_error4;
32128 |     }
32129 |
32130 |
32131 |     /* Internal channel map. */
32132 |     pActualChannelMap = ((ma_pa_stream_get_channel_map_proc)pDevice->pContext->pulse.pa_stream_g
> et_channel_map)((ma_pa_stream*)pDevice->pulse.pStreamCapture);
32133 |     if (pActualChannelMap == NULL) {
32134 |         pActualChannelMap = &cmap; /* Fallback just in case. */
32135 |     }
32136 |
32137 |     /*
32138 |     Bug in PipeWire. There have been reports that PipeWire is returning AUX channels when
> reporting
32139 |     the channel map. To somewhat workaround this, I'm hacking in a hard coded channel map for
> mono
32140 |     and stereo. In this case it should be safe to assume mono = MONO and stereo = LEFT/RIGHT.
> For
32141 |     all other channel counts we need to just put up with whatever PipeWire reports and hope it
> gets
32142 |     fixed sooner than later. I might remove this hack later.
32143 |     */
32144 |     if (pDescriptorCapture->channels > 2) {
32145 |         for (iChannel = 0; iChannel < pDescriptorCapture->channels; iChannel += 1) {
32146 |             pDescriptorCapture->channelMap[iChannel] =
> ma_channel_position_from_pulse(pActualChannelMap->map[iChannel]);
32147 |         }
32148 |     } else {
32149 |         /* Hack for mono and stereo. */
32150 |         if (pDescriptorCapture->channels == 1) {
32151 |             pDescriptorCapture->channelMap[0] = MA_CHANNEL_MONO;
32152 |         } else if (pDescriptorCapture->channels == 2) {
32153 |             pDescriptorCapture->channelMap[0] = MA_CHANNEL_FRONT_LEFT;
32154 |             pDescriptorCapture->channelMap[1] = MA_CHANNEL_FRONT_RIGHT;
32155 |         } else {
32156 |             MA_ASSERT(MA_FALSE); /* Should never hit this. */
32157 |         }
32158 |     }
32159 |
32160 |
32161 |     /* Buffer. */
32162 |     pActualAttr = ((ma_pa_stream_get_buffer_attr_proc)pDevice->pContext->pulse.pa_stream_get_buf
> fer_attr)((ma_pa_stream*)pDevice->pulse.pStreamCapture);
32163 |     if (pActualAttr != NULL) {
32164 |         attr = *pActualAttr;
32165 |     }
32166 |
32167 |     if (attr.fragsize > 0) {
32168 |         pDescriptorCapture->periodCount = ma_max(attr.maxlength / attr.fragsize, 1);
32169 |     } else {
32170 |         pDescriptorCapture->periodCount = 1;
32171 |     }
32172 |
32173 |     pDescriptorCapture->periodSizeInFrames = attr.maxlength /
> ma_get_bytes_per_frame(pDescriptorCapture->format, pDescriptorCapture->channels) /
> pDescriptorCapture->periodCount;
32174 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Capture actual
> attr: maxlength=%d, tlength=%d, prebuf=%d, minreq=%d, fragsize=%d; periodSizeInFrames=%d\n",
> attr.maxlength, attr.tlength, attr.prebuf, attr.minreq, attr.fragsize,
> pDescriptorCapture->periodSizeInFrames);
32175 | }
32176 |

```

```

32177 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
32178 |         result = ma_context_get_sink_info__pulse(pDevice->pContext, devPlayback, &sinkInfo);
32179 |         if (result != MA_SUCCESS) {
32180 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> retrieve sink info for playback device.\n");
32181 |             goto on_error2;
32182 |         }
32183 |
32184 |         ss = sinkInfo.sample_spec;
32185 |         cmap = sinkInfo.channel_map;
32186 |
32187 |         /* Use the requested channel count if we have one. */
32188 |         if (pDescriptorPlayback->channels != 0) {
32189 |             ss.channels = pDescriptorPlayback->channels;
32190 |         }
32191 |
32192 |         /* PulseAudio has a maximum channel count of 32. We'll get a crash if this is exceeded. */
32193 |         if (ss.channels > 32) {
32194 |             ss.channels = 32;
32195 |         }
32196 |
32197 |         /* Use a default channel map. */
32198 |
> ((ma_pa_channel_map_init_extend_proc)pDevice->pContext->pulse.pa_channel_map_init_extend>(&cmap,
> ss.channels, (ma_pa_channel_map_def_t)pConfig->pulse.channelMap);
32199 |
32200 |
32201 |         /* Use the requested sample rate if one was specified. */
32202 |         if (pDescriptorPlayback->sampleRate != 0) {
32203 |             ss.rate = pDescriptorPlayback->sampleRate;
32204 |         }
32205 |
32206 |         streamFlags = MA_PA_STREAM_START_CORKED | MA_PA_STREAM_ADJUST_LATENCY;
32207 |         if (ma_format_from_pulse(ss.format) == ma_format_unknown) {
32208 |             if (ma_is_little_endian()) {
32209 |                 ss.format = MA_PA_SAMPLE_FLOAT32LE;
32210 |             } else {
32211 |                 ss.format = MA_PA_SAMPLE_FLOAT32BE;
32212 |             }
32213 |             streamFlags |= MA_PA_STREAM_FIX_FORMAT;
32214 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio]
> sample_spec.format not supported by miniaudio. Defaulting to PA_SAMPLE_FLOAT32.\n");
32215 |         }
32216 |         if (ss.rate == 0) {
32217 |             ss.rate = MA_DEFAULT_SAMPLE_RATE;
32218 |             streamFlags |= MA_PA_STREAM_FIX_RATE;
32219 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio]
> sample_spec.rate = 0. Defaulting to %d.\n", ss.rate);
32220 |         }
32221 |         if (ss.channels == 0) {
32222 |             ss.channels = MA_DEFAULT_CHANNELS;
32223 |             streamFlags |= MA_PA_STREAM_FIX_CHANNELS;
32224 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio]
> sample_spec.channels = 0. Defaulting to %d.\n", ss.channels);
32225 |         }
32226 |
32227 |         /* We now have enough information to calculate the actual buffer size in frames. */
32228 |         pDescriptorPlayback->periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__pulse(pDescriptorPlayback, ss.rate,
> pConfig->performanceProfile);
32229 |
32230 |         attr = ma_device__pa_buffer_attr_new(pDescriptorPlayback->periodSizeInFrames,
> pDescriptorPlayback->periodCount, &ss);
32231 |
32232 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Playback attr:
> maxlen=%d, tlength=%d, prebuf=%d, minreq=%d, fragsize=%d; periodSizeInFrames=%d\n",
> attr.maxlength, attr.tlength, attr.prebuf, attr.minreq, attr.fragsize,
> pDescriptorPlayback->periodSizeInFrames);
32233 |
32234 |         pDevice->pulse.pStreamPlayback = ma_device__pa_stream_new__pulse(pDevice,
> pConfig->pulse.pStreamNamePlayback, &ss, &cmap);
32235 |         if (pDevice->pulse.pStreamPlayback == NULL) {
32236 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> create PulseAudio playback stream.\n");
32237 |             result = MA_ERROR;
32238 |             goto on_error2;
32239 |         }
32240 |

```

```

32241 |
32242 |     /*
32243 |     Note that this callback will be fired as soon as the stream is connected, even though it's
> started as corked. The callback needs to handle a
32244 |     device state of ma_device_state_uninitialized.
32245 |     */
32246 |     ((ma_pa_stream_set_write_callback_proc)pDevice->pContext->pulse.pa_stream_set_write_callback
> )((ma_pa_stream*)pDevice->pulse.pStreamPlayback, ma_device_on_write__pulse, pDevice);
32247 |
32248 |     /* State callback for checking when the device has been corked. */
32249 |     ((ma_pa_stream_set_suspended_callback_proc)pDevice->pContext->pulse.pa_stream_set_suspended_
> callback)((ma_pa_stream*)pDevice->pulse.pStreamPlayback, ma_device_on_suspended__pulse, pDevice);
32250 |
32251 |     /* Rerouting notification. */
32252 |     ((ma_pa_stream_set_moved_callback_proc)pDevice->pContext->pulse.pa_stream_set_moved_callback
> )((ma_pa_stream*)pDevice->pulse.pStreamPlayback, ma_device_on_rerouted__pulse, pDevice);
32253 |
32254 |
32255 |     /* Connect after we've got all of our internal state set up. */
32256 |     if (devPlayback != NULL) {
32257 |         streamFlags |= MA_PA_STREAM_DONT_MOVE;
32258 |     }
32259 |
32260 |     error = ((ma_pa_stream_connect_playback_proc)pDevice->pContext->pulse.pa_stream_connect_play
> back)((ma_pa_stream*)pDevice->pulse.pStreamPlayback, devPlayback, &attr,
> (ma_pa_stream_flags_t)streamFlags, NULL, NULL);
32261 |     if (error != MA_PA_OK) {
32262 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> connect PulseAudio playback stream.");
32263 |         result = ma_result_from_pulse(error);
32264 |         goto on_error3;
32265 |     }
32266 |
32267 |     result = ma_wait_for_pa_stream_to_connect__pulse(pDevice->pContext,
> pDevice->pulse.pMainLoop, (ma_pa_stream*)pDevice->pulse.pStreamPlayback);
32268 |     if (result != MA_SUCCESS) {
32269 |         goto on_error3;
32270 |     }
32271 |
32272 |
32273 |     /* Internal format. */
32274 |     pActualSS = ((ma_pa_stream_get_sample_spec_proc)pDevice->pContext->pulse.pa_stream_get_sampl
> e_spec)((ma_pa_stream*)pDevice->pulse.pStreamPlayback);
32275 |     if (pActualSS != NULL) {
32276 |         ss = *pActualSS;
32277 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Playback
> sample spec: format=%s, channels=%d, rate=%d\n",
> ma_get_format_name(ma_format_from_pulse(ss.format)), ss.channels, ss.rate);
32278 |     } else {
32279 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Failed to
> retrieve playback sample spec.\n");
32280 |     }
32281 |
32282 |     pDescriptorPlayback->format = ma_format_from_pulse(ss.format);
32283 |     pDescriptorPlayback->channels = ss.channels;
32284 |     pDescriptorPlayback->sampleRate = ss.rate;
32285 |
32286 |     if (pDescriptorPlayback->format == ma_format_unknown || pDescriptorPlayback->channels == 0
> || pDescriptorPlayback->sampleRate == 0) {
32287 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Playback
> sample spec is invalid. Device unusable by miniaudio. format=%s, channels=%d, sampleRate=%d.\n",
> ma_get_format_name(pDescriptorPlayback->format), pDescriptorPlayback->channels,
> pDescriptorPlayback->sampleRate);
32288 |         result = MA_ERROR;
32289 |         goto on_error4;
32290 |     }
32291 |
32292 |
32293 |     /* Internal channel map. */
32294 |     pActualChannelMap = ((ma_pa_stream_get_channel_map_proc)pDevice->pContext->pulse.pa_stream_g
> et_channel_map)((ma_pa_stream*)pDevice->pulse.pStreamPlayback);
32295 |     if (pActualChannelMap == NULL) {
32296 |         pActualChannelMap = &cmap; /* Fallback just in case. */
32297 |     }
32298 |
32299 |     /*
32300 |     Bug in PipeWire. There have been reports that PipeWire is returning AUX channels when
> reporting
32301 |     the channel map. To somewhat workaround this, I'm hacking in a hard coded channel map for

```

```

> mono
32302 |         and stereo. In this case it should be safe to assume mono = MONO and stereo = LEFT/RIGHT.
> For
32303 |         all other channel counts we need to just put up with whatever PipeWire reports and hope it
> gets
32304 |         fixed sooner than later. I might remove this hack later.
32305 |         */
32306 |         if (pDescriptorPlayback->channels > 2) {
32307 |             for (iChannel = 0; iChannel < pDescriptorPlayback->channels; iChannel += 1) {
32308 |                 pDescriptorPlayback->channelMap[iChannel] =
> ma_channel_position_from_pulse(pActualChannelMap->map[iChannel]);
32309 |             }
32310 |         } else {
32311 |             /* Hack for mono and stereo. */
32312 |             if (pDescriptorPlayback->channels == 1) {
32313 |                 pDescriptorPlayback->channelMap[0] = MA_CHANNEL_MONO;
32314 |             } else if (pDescriptorPlayback->channels == 2) {
32315 |                 pDescriptorPlayback->channelMap[0] = MA_CHANNEL_FRONT_LEFT;
32316 |                 pDescriptorPlayback->channelMap[1] = MA_CHANNEL_FRONT_RIGHT;
32317 |             } else {
32318 |                 MA_ASSERT(MA_FALSE); /* Should never hit this. */
32319 |             }
32320 |         }
32321 |
32322 |
32323 |         /* Buffer. */
32324 |         pActualAttr = ((ma_pa_stream_get_buffer_attr_proc)pDevice->pContext->pulse.pa_stream_get_buf
> fer_attr)((ma_pa_stream*)pDevice->pulse.pStreamPlayback);
32325 |         if (pActualAttr != NULL) {
32326 |             attr = *pActualAttr;
32327 |         }
32328 |
32329 |         if (attr.tlength > 0) {
32330 |             pDescriptorPlayback->periodCount = ma_max(attr.maxlength / attr.tlength, 1);
32331 |         } else {
32332 |             pDescriptorPlayback->periodCount = 1;
32333 |         }
32334 |
32335 |         pDescriptorPlayback->periodSizeInFrames = attr.maxlength /
> ma_get_bytes_per_frame(pDescriptorPlayback->format, pDescriptorPlayback->channels) /
> pDescriptorPlayback->periodCount;
32336 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[PulseAudio] Playback actual
> attr: maxlength=%d, tlength=%d, prebuf=%d, minreq=%d, fragsize=%d; internalPeriodSizeInFrames=%d\n",
> attr.maxlength, attr.tlength, attr.prebuf, attr.minreq, attr.fragsize,
> pDescriptorPlayback->periodSizeInFrames);
32337 |     }
32338 |
32339 |
32340 |     /*
32341 |     We need a ring buffer for handling duplex mode. We can use the main duplex ring buffer in the
> main
32342 |     part of the ma_device struct. We cannot, however, depend on ma_device_init() initializing this
> for
32343 |     us later on because that will only do it if it's a fully asynchronous backend - i.e. the
32344 |     onDeviceDataLoop callback is NULL, which is not the case for PulseAudio.
32345 |     */
32346 |     if (pConfig->deviceType == ma_device_type_duplex) {
32347 |         ma_format rbFormat = (format != ma_format_unknown) ? format :
> pDescriptorCapture->format;
32348 |         ma_uint32 rbChannels = (channels > 0) ? channels :
> pDescriptorCapture->channels;
32349 |         ma_uint32 rbSampleRate = (sampleRate > 0) ? sampleRate :
> pDescriptorCapture->sampleRate;
32350 |
32351 |         result = ma_duplex_rb_init(rbFormat, rbChannels, rbSampleRate,
> pDescriptorCapture->sampleRate, pDescriptorCapture->periodSizeInFrames,
> &pDevice->pContext->allocationCallbacks, &pDevice->duplexRB);
32352 |         if (result != MA_SUCCESS) {
32353 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to
> initialize ring buffer. %s.\n", ma_result_description(result));
32354 |             goto on_error4;
32355 |         }
32356 |     }
32357 |
32358 |     return MA_SUCCESS;
32359 |
32360 |
32361 | on_error4:
32362 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==

```

```

> ma_device_type_duplex) {
32363 |     ((ma_pa_stream_disconnect_proc)pDevice->pContext->pulse.pa_stream_disconnect)((ma_pa_stream*
> )pDevice->pulse.pStreamPlayback);
32364 | }
32365 | on_error3:
32366 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
32367 |         ((ma_pa_stream_unref_proc)pDevice->pContext->pulse.pa_stream_unref)((ma_pa_stream*)pDevice->
> pulse.pStreamPlayback);
32368 |     }
32369 | on_error2:
32370 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
32371 |         ((ma_pa_stream_disconnect_proc)pDevice->pContext->pulse.pa_stream_disconnect)((ma_pa_stream*
> )pDevice->pulse.pStreamCapture);
32372 |     }
32373 | on_error1:
32374 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
32375 |         ((ma_pa_stream_unref_proc)pDevice->pContext->pulse.pa_stream_unref)((ma_pa_stream*)pDevice->
> pulse.pStreamCapture);
32376 |     }
32377 | on_error0:
32378 |     return result;
32379 | }
32380 |
32381 |
32382 | static void ma_pulse_operation_complete_callback(ma_pa_stream* pStream, int success, void*
> pUserData)
32383 | {
32384 |     ma_bool32* pIsSuccessful = (ma_bool32*)pUserData;
32385 |     MA_ASSERT(pIsSuccessful != NULL);
32386 |
32387 |     *pIsSuccessful = (ma_bool32)success;
32388 |
32389 |     (void)pStream; /* Unused. */
32390 | }
32391 |
32392 | static ma_result ma_device_cork_stream_pulse(ma_device* pDevice, ma_device_type deviceType, int
> cork)
32393 | {
32394 |     ma_context* pContext = pDevice->pContext;
32395 |     ma_bool32 wasSuccessful;
32396 |     ma_pa_stream* pStream;
32397 |     ma_pa_operation* pOP;
32398 |     ma_result result;
32399 |
32400 |     /* This should not be called with a duplex device type. */
32401 |     if (deviceType == ma_device_type_duplex) {
32402 |         return MA_INVALID_ARGS;
32403 |     }
32404 |
32405 |     wasSuccessful = MA_FALSE;
32406 |
32407 |     pStream = (ma_pa_stream*)((deviceType == ma_device_type_capture) ? pDevice->pulse.pStreamCapture
> : pDevice->pulse.pStreamPlayback);
32408 |     MA_ASSERT(pStream != NULL);
32409 |
32410 |     pOP = ((ma_pa_stream_cork_proc)pContext->pulse.pa_stream_cork)(pStream, cork,
> ma_pulse_operation_complete_callback, &wasSuccessful);
32411 |     if (pOP == NULL) {
32412 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to cork
> PulseAudio stream.");
32413 |         return MA_ERROR;
32414 |     }
32415 |
32416 |     result = ma_wait_for_operation_and_unref__pulse(pDevice->pContext, pDevice->pulse.pMainLoop,
> pOP);
32417 |     if (result != MA_SUCCESS) {
32418 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] An error occurred
> while waiting for the PulseAudio stream to cork.");
32419 |         return result;
32420 |     }
32421 |
32422 |     if (!wasSuccessful) {
32423 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[PulseAudio] Failed to %s
> PulseAudio stream.", (cork) ? "stop" : "start");
32424 |         return MA_ERROR;
32425 |     }

```

```

32426 |
32427 |     return MA_SUCCESS;
32428 | }
32429 |
32430 | static ma_result ma_device_start_pulse(ma_device* pDevice)
32431 | {
32432 |     ma_result result;
32433 |
32434 |     MA_ASSERT(pDevice != NULL);
32435 |
32436 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
32437 |         result = ma_device_cork_stream_pulse(pDevice, ma_device_type_capture, 0);
32438 |         if (result != MA_SUCCESS) {
32439 |             return result;
32440 |         }
32441 |     }
32442 |
32443 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
32444 |         /*
32445 |         We need to fill some data before uncorking. Not doing this will result in the write callback
32446 |         never getting fired. We're not going to abort if writing fails because I still want the
> device
32447 |         to get uncorked.
32448 |         */
32449 |         ma_device_write_to_stream_pulse(pDevice, (ma_pa_stream*)(pDevice->pulse.pStreamPlayback),
> NULL); /* No need to check the result here. Always want to fall through an uncork.*/
32450 |
32451 |         result = ma_device_cork_stream_pulse(pDevice, ma_device_type_playback, 0);
32452 |         if (result != MA_SUCCESS) {
32453 |             return result;
32454 |         }
32455 |     }
32456 |
32457 |     return MA_SUCCESS;
32458 | }
32459 |
32460 | static ma_result ma_device_stop_pulse(ma_device* pDevice)
32461 | {
32462 |     ma_result result;
32463 |
32464 |     MA_ASSERT(pDevice != NULL);
32465 |
32466 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
32467 |         result = ma_device_cork_stream_pulse(pDevice, ma_device_type_capture, 1);
32468 |         if (result != MA_SUCCESS) {
32469 |             return result;
32470 |         }
32471 |     }
32472 |
32473 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
32474 |         /*
32475 |         Ideally we would drain the device here, but there's been cases where PulseAudio seems to be
32476 |         broken on some systems to the point where no audio processing seems to happen. When this
32477 |         happens, draining never completes and we get stuck here. For now I'm disabling draining of
32478 |         the device so we don't just freeze the application.
32479 |         */
32480 |         #if 0
32481 |         ma_pa_operation* pOP = ((ma_pa_stream_drain_proc)pDevice->pContext->pulse.pa_stream_drain)((
> ma_pa_stream*)pDevice->pulse.pStreamPlayback, ma_pulse_operation_complete_callback, &wasSuccessful);
32482 |         ma_wait_for_operation_and_unref_pulse(pDevice->pContext, pDevice->pulse.pMainLoop, pOP);
32483 |         #endif
32484 |
32485 |         result = ma_device_cork_stream_pulse(pDevice, ma_device_type_playback, 1);
32486 |         if (result != MA_SUCCESS) {
32487 |             return result;
32488 |         }
32489 |     }
32490 |
32491 |     return MA_SUCCESS;
32492 | }
32493 |
32494 | static ma_result ma_device_data_loop_pulse(ma_device* pDevice)
32495 | {
32496 |     int resultPA;
32497 |
32498 |     MA_ASSERT(pDevice != NULL);
32499 |
32500 |     /* NOTE: Don't start the device here. It'll be done at a higher level. */
32501 |

```



```

32502 |     /*
32503 |     All data is handled through callbacks. All we need to do is iterate over the main loop and let
32504 |     the callbacks deal with it.
32505 |     */
32506 |     while (ma_device_get_state(pDevice) == ma_device_state_started) {
32507 |         resultPA = ((ma_pa_mainloop_iterate_proc)pDevice->pContext->pulse.pa_mainloop_iterate)((ma_p
> a_mainloop*)pDevice->pulse.pMainLoop, 1, NULL);
32508 |         if (resultPA < 0) {
32509 |             break;
32510 |         }
32511 |     }
32512 |
32513 |     /* NOTE: Don't stop the device here. It'll be done at a higher level. */
32514 |     return MA_SUCCESS;
32515 | }
32516 |
32517 | static ma_result ma_device_data_loop_wakeup__pulse(ma_device* pDevice)
32518 | {
32519 |     MA_ASSERT(pDevice != NULL);
32520 |
32521 |     ((ma_pa_mainloop_wakeup_proc)pDevice->pContext->pulse.pa_mainloop_wakeup)((ma_pa_mainloop*)pDevi
> ce->pulse.pMainLoop);
32522 |
32523 |     return MA_SUCCESS;
32524 | }
32525 |
32526 | static ma_result ma_context_uninit__pulse(ma_context* pContext)
32527 | {
32528 |     MA_ASSERT(pContext != NULL);
32529 |     MA_ASSERT(pContext->backend == ma_backend_pulseaudio);
32530 |
32531 |     ((ma_pa_context_disconnect_proc)pContext->pulse.pa_context_disconnect)((ma_pa_context*)pContext->
> pulse.pPulseContext);
32532 |     ((ma_pa_context_unref_proc)pContext->pulse.pa_context_unref)((ma_pa_context*)pContext->pulse.pPu
> lseContext);
32533 |     ((ma_pa_mainloop_free_proc)pContext->pulse.pa_mainloop_free)((ma_pa_mainloop*)pContext->pulse.pM
> ainLoop);
32534 |
32535 |     ma_free(pContext->pulse.pServerName, &pContext->allocationCallbacks);
32536 |     ma_free(pContext->pulse.pApplicationName, &pContext->allocationCallbacks);
32537 |
32538 | #ifndef MA_NO_RUNTIME_LINKING
32539 |     ma_dlclose(ma_context_get_log(pContext), pContext->pulse.pulseSO);
32540 | #endif
32541 |
32542 |     return MA_SUCCESS;
32543 | }
32544 |
32545 | static ma_result ma_context_init__pulse(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
32546 | {
32547 |     ma_result result;
32548 | #ifndef MA_NO_RUNTIME_LINKING
32549 |     const char* libpulseNames[] = {
32550 |         "libpulse.so",
32551 |         "libpulse.so.0"
32552 |     };
32553 |     size_t i;
32554 |
32555 |     for (i = 0; i < ma_countof(libpulseNames); ++i) {
32556 |         pContext->pulse.pulseSO = ma_dlopen(ma_context_get_log(pContext), libpulseNames[i]);
32557 |         if (pContext->pulse.pulseSO != NULL) {
32558 |             break;
32559 |         }
32560 |     }
32561 |
32562 |     if (pContext->pulse.pulseSO == NULL) {
32563 |         return MA_NO_BACKEND;
32564 |     }
32565 |
32566 |     pContext->pulse.pa_mainloop_new =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseSO, "pa_mainloop_new");
32567 |     pContext->pulse.pa_mainloop_free =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseSO, "pa_mainloop_free");
32568 |     pContext->pulse.pa_mainloop_quit =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseSO, "pa_mainloop_quit");
32569 |     pContext->pulse.pa_mainloop_get_api =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseSO, "pa_mainloop_get_api");
32570 |     pContext->pulse.pa_mainloop_iterate =

```

```

> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_mainloop_iterate");
32571 |     pContext->pulse.pa_mainloop_wakeup =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_mainloop_wakeup");
32572 |     pContext->pulse.pa_threaded_mainloop_new =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_new");
32573 |     pContext->pulse.pa_threaded_mainloop_free =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_free");
32574 |     pContext->pulse.pa_threaded_mainloop_start =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_start");
32575 |     pContext->pulse.pa_threaded_mainloop_stop =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_stop");
32576 |     pContext->pulse.pa_threaded_mainloop_lock =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_lock");
32577 |     pContext->pulse.pa_threaded_mainloop_unlock =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_unlock");
32578 |     pContext->pulse.pa_threaded_mainloop_wait =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_wait");
32579 |     pContext->pulse.pa_threaded_mainloop_signal =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_signal");
32580 |     pContext->pulse.pa_threaded_mainloop_accept =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_accept");
32581 |     pContext->pulse.pa_threaded_mainloop_get_retval =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_get_retval");
32582 |     pContext->pulse.pa_threaded_mainloop_get_api =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_get_api");
32583 |     pContext->pulse.pa_threaded_mainloop_in_thread =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_in_thread");
32584 |     pContext->pulse.pa_threaded_mainloop_set_name =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_threaded_mainloop_set_name");
32585 |     pContext->pulse.pa_context_new =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_context_new");
32586 |     pContext->pulse.pa_context_unref =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_context_unref");
32587 |     pContext->pulse.pa_context_connect =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_context_connect");
32588 |     pContext->pulse.pa_context_disconnect =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_context_disconnect");
32589 |     pContext->pulse.pa_context_set_state_callback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_context_set_state_callback");
32590 |     pContext->pulse.pa_context_get_state =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_context_get_state");
32591 |     pContext->pulse.pa_context_get_sink_info_list =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_context_get_sink_info_list");
32592 |     pContext->pulse.pa_context_get_source_info_list =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_context_get_source_info_list");
32593 |     pContext->pulse.pa_context_get_sink_info_by_name =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_context_get_sink_info_by_name");
32594 |     pContext->pulse.pa_context_get_source_info_by_name =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_context_get_source_info_by_name");
32595 |     pContext->pulse.pa_operation_unref =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_operation_unref");
32596 |     pContext->pulse.pa_operation_get_state =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_operation_get_state");
32597 |     pContext->pulse.pa_channel_map_init_extend =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_channel_map_init_extend");
32598 |     pContext->pulse.pa_channel_map_valid =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_channel_map_valid");
32599 |     pContext->pulse.pa_channel_map_compatible =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_channel_map_compatible");

```

```

32600 |     pContext->pulse.pa_stream_new =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_new");
32601 |     pContext->pulse.pa_stream_unref =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_unref");
32602 |     pContext->pulse.pa_stream_connect_playback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_connect_playback");
32603 |     pContext->pulse.pa_stream_connect_record =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_connect_record");
32604 |     pContext->pulse.pa_stream_disconnect =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_disconnect");
32605 |     pContext->pulse.pa_stream_get_state =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_get_state");
32606 |     pContext->pulse.pa_stream_get_sample_spec =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_get_sample_spec");
32607 |     pContext->pulse.pa_stream_get_channel_map =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_get_channel_map");
32608 |     pContext->pulse.pa_stream_get_buffer_attr =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_get_buffer_attr");
32609 |     pContext->pulse.pa_stream_set_buffer_attr =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_set_buffer_attr");
32610 |     pContext->pulse.pa_stream_get_device_name =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_get_device_name");
32611 |     pContext->pulse.pa_stream_set_write_callback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_set_write_callback");
32612 |     pContext->pulse.pa_stream_set_read_callback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_set_read_callback");
32613 |     pContext->pulse.pa_stream_set_suspended_callback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_set_suspended_callback");
32614 |     pContext->pulse.pa_stream_set_moved_callback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0,
> "pa_stream_set_moved_callback");
32615 |     pContext->pulse.pa_stream_is_suspended =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_is_suspended");
32616 |     pContext->pulse.pa_stream_flush =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_flush");
32617 |     pContext->pulse.pa_stream_drain =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_drain");
32618 |     pContext->pulse.pa_stream_is_corked =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_is_corked");
32619 |     pContext->pulse.pa_stream_cork =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_cork");
32620 |     pContext->pulse.pa_stream_trigger =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_trigger");
32621 |     pContext->pulse.pa_stream_begin_write =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_begin_write");
32622 |     pContext->pulse.pa_stream_write =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_write");
32623 |     pContext->pulse.pa_stream_peek =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_peek");
32624 |     pContext->pulse.pa_stream_drop =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_drop");
32625 |     pContext->pulse.pa_stream_writable_size =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_writable_size");
32626 |     pContext->pulse.pa_stream_readable_size =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->pulse.pulseS0, "pa_stream_readable_size");
32627 | #else
32628 |     /* This strange assignment system is just for type safety. */
32629 |     ma_pa_mainloop_new_proc _pa_mainloop_new = pa_mainloop_new;
32630 |     ma_pa_mainloop_free_proc _pa_mainloop_free =
> pa_mainloop_free;
32631 |     ma_pa_mainloop_quit_proc _pa_mainloop_quit =
> pa_mainloop_quit;
32632 |     ma_pa_mainloop_get_api_proc _pa_mainloop_get_api =
> pa_mainloop_get_api;
32633 |     ma_pa_mainloop_iterate_proc _pa_mainloop_iterate =
> pa_mainloop_iterate;
32634 |     ma_pa_mainloop_wakeup_proc _pa_mainloop_wakeup =
> pa_mainloop_wakeup;
32635 |     ma_pa_threaded_mainloop_new_proc _pa_threaded_mainloop_new =

```

```

> pa_threaded_mainloop_new;
32636 |   ma_pa_threaded_mainloop_free_proc   _pa_threaded_mainloop_free   =
> pa_threaded_mainloop_free;
32637 |   ma_pa_threaded_mainloop_start_proc  _pa_threaded_mainloop_start  =
> pa_threaded_mainloop_start;
32638 |   ma_pa_threaded_mainloop_stop_proc   _pa_threaded_mainloop_stop   =
> pa_threaded_mainloop_stop;
32639 |   ma_pa_threaded_mainloop_lock_proc   _pa_threaded_mainloop_lock   =
> pa_threaded_mainloop_lock;
32640 |   ma_pa_threaded_mainloop_unlock_proc _pa_threaded_mainloop_unlock =
> pa_threaded_mainloop_unlock;
32641 |   ma_pa_threaded_mainloop_wait_proc   _pa_threaded_mainloop_wait   =
> pa_threaded_mainloop_wait;
32642 |   ma_pa_threaded_mainloop_signal_proc  _pa_threaded_mainloop_signal =
> pa_threaded_mainloop_signal;
32643 |   ma_pa_threaded_mainloop_accept_proc  _pa_threaded_mainloop_accept =
> pa_threaded_mainloop_accept;
32644 |   ma_pa_threaded_mainloop_get_retval_proc _pa_threaded_mainloop_get_retval =
> pa_threaded_mainloop_get_retval;
32645 |   ma_pa_threaded_mainloop_get_api_proc  _pa_threaded_mainloop_get_api =
> pa_threaded_mainloop_get_api;
32646 |   ma_pa_threaded_mainloop_in_thread_proc _pa_threaded_mainloop_in_thread =
> pa_threaded_mainloop_in_thread;
32647 |   ma_pa_threaded_mainloop_set_name_proc _pa_threaded_mainloop_set_name =
> pa_threaded_mainloop_set_name;
32648 |   ma_pa_context_new_proc               _pa_context_new               = pa_context_new;
32649 |   ma_pa_context_unref_proc             _pa_context_unref            =
> pa_context_unref;
32650 |   ma_pa_context_connect_proc           _pa_context_connect           =
> pa_context_connect;
32651 |   ma_pa_context_disconnect_proc         _pa_context_disconnect        =
> pa_context_disconnect;
32652 |   ma_pa_context_set_state_callback_proc _pa_context_set_state_callback =
> pa_context_set_state_callback;
32653 |   ma_pa_context_get_state_proc          _pa_context_get_state         =
> pa_context_get_state;
32654 |   ma_pa_context_get_sink_info_list_proc _pa_context_get_sink_info_list =
> pa_context_get_sink_info_list;
32655 |   ma_pa_context_get_source_info_list_proc _pa_context_get_source_info_list =
> pa_context_get_source_info_list;
32656 |   ma_pa_context_get_sink_info_by_name_proc _pa_context_get_sink_info_by_name =
> pa_context_get_sink_info_by_name;
32657 |   ma_pa_context_get_source_info_by_name_proc _pa_context_get_source_info_by_name =
> pa_context_get_source_info_by_name;
32658 |   ma_pa_operation_unref_proc           _pa_operation_unref           =
> pa_operation_unref;
32659 |   ma_pa_operation_get_state_proc        _pa_operation_get_state       =
> pa_operation_get_state;
32660 |   ma_pa_channel_map_init_extend_proc    _pa_channel_map_init_extend    =
> pa_channel_map_init_extend;
32661 |   ma_pa_channel_map_valid_proc          _pa_channel_map_valid         =
> pa_channel_map_valid;
32662 |   ma_pa_channel_map_compatible_proc     _pa_channel_map_compatible     =
> pa_channel_map_compatible;
32663 |   ma_pa_stream_new_proc                 _pa_stream_new                 = pa_stream_new;
32664 |   ma_pa_stream_unref_proc               _pa_stream_unref               = pa_stream_unref;
32665 |   ma_pa_stream_connect_playback_proc    _pa_stream_connect_playback    =
> pa_stream_connect_playback;
32666 |   ma_pa_stream_connect_record_proc      _pa_stream_connect_record     =
> pa_stream_connect_record;
32667 |   ma_pa_stream_disconnect_proc          _pa_stream_disconnect         =
> pa_stream_disconnect;
32668 |   ma_pa_stream_get_state_proc           _pa_stream_get_state          =
> pa_stream_get_state;
32669 |   ma_pa_stream_get_sample_spec_proc     _pa_stream_get_sample_spec     =
> pa_stream_get_sample_spec;
32670 |   ma_pa_stream_get_channel_map_proc     _pa_stream_get_channel_map     =
> pa_stream_get_channel_map;
32671 |   ma_pa_stream_get_buffer_attr_proc     _pa_stream_get_buffer_attr     =
> pa_stream_get_buffer_attr;
32672 |   ma_pa_stream_set_buffer_attr_proc     _pa_stream_set_buffer_attr     =
> pa_stream_set_buffer_attr;
32673 |   ma_pa_stream_get_device_name_proc     _pa_stream_get_device_name     =
> pa_stream_get_device_name;
32674 |   ma_pa_stream_set_write_callback_proc   _pa_stream_set_write_callback  =
> pa_stream_set_write_callback;
32675 |   ma_pa_stream_set_read_callback_proc    _pa_stream_set_read_callback   =
> pa_stream_set_read_callback;
32676 |   ma_pa_stream_set_suspended_callback_proc _pa_stream_set_suspended_callback =

```

```

> pa_stream_set_suspended_callback;
32677 |   ma_pa_stream_set_moved_callback_proc   _pa_stream_set_moved_callback   =
> pa_stream_set_moved_callback;
32678 |   ma_pa_stream_is_suspended_proc         _pa_stream_is_suspended         =
> pa_stream_is_suspended;
32679 |   ma_pa_stream_flush_proc                _pa_stream_flush                = pa_stream_flush;
32680 |   ma_pa_stream_drain_proc                _pa_stream_drain                = pa_stream_drain;
32681 |   ma_pa_stream_is_corked_proc            _pa_stream_is_corked            =
> pa_stream_is_corked;
32682 |   ma_pa_stream_cork_proc                 _pa_stream_cork                 = pa_stream_cork;
32683 |   ma_pa_stream_trigger_proc              _pa_stream_trigger              =
> pa_stream_trigger;
32684 |   ma_pa_stream_begin_write_proc          _pa_stream_begin_write          =
> pa_stream_begin_write;
32685 |   ma_pa_stream_write_proc                _pa_stream_write                = pa_stream_write;
32686 |   ma_pa_stream_peek_proc                 _pa_stream_peek                 = pa_stream_peek;
32687 |   ma_pa_stream_drop_proc                 _pa_stream_drop                 = pa_stream_drop;
32688 |   ma_pa_stream_writable_size_proc         _pa_stream_writable_size        =
> pa_stream_writable_size;
32689 |   ma_pa_stream_readable_size_proc        _pa_stream_readable_size        =
> pa_stream_readable_size;
32690 |
32691 |   pContext->pulse.pa_mainloop_new         = (ma_proc)_pa_mainloop_new;
32692 |   pContext->pulse.pa_mainloop_free        = (ma_proc)_pa_mainloop_free;
32693 |   pContext->pulse.pa_mainloop_quit        = (ma_proc)_pa_mainloop_quit;
32694 |   pContext->pulse.pa_mainloop_get_api     = (ma_proc)_pa_mainloop_get_api;
32695 |   pContext->pulse.pa_mainloop_iterate     = (ma_proc)_pa_mainloop_iterate;
32696 |   pContext->pulse.pa_mainloop_wakeup      = (ma_proc)_pa_mainloop_wakeup;
32697 |   pContext->pulse.pa_threaded_mainloop_new = (ma_proc)_pa_threaded_mainloop_new;
32698 |   pContext->pulse.pa_threaded_mainloop_free = (ma_proc)_pa_threaded_mainloop_free;
32699 |   pContext->pulse.pa_threaded_mainloop_start = (ma_proc)_pa_threaded_mainloop_start;
32700 |   pContext->pulse.pa_threaded_mainloop_stop = (ma_proc)_pa_threaded_mainloop_stop;
32701 |   pContext->pulse.pa_threaded_mainloop_lock = (ma_proc)_pa_threaded_mainloop_lock;
32702 |   pContext->pulse.pa_threaded_mainloop_unlock = (ma_proc)_pa_threaded_mainloop_unlock;
32703 |   pContext->pulse.pa_threaded_mainloop_wait = (ma_proc)_pa_threaded_mainloop_wait;
32704 |   pContext->pulse.pa_threaded_mainloop_signal = (ma_proc)_pa_threaded_mainloop_signal;
32705 |   pContext->pulse.pa_threaded_mainloop_accept = (ma_proc)_pa_threaded_mainloop_accept;
32706 |   pContext->pulse.pa_threaded_mainloop_get_retval = (ma_proc)_pa_threaded_mainloop_get_retval;
32707 |   pContext->pulse.pa_threaded_mainloop_get_api = (ma_proc)_pa_threaded_mainloop_get_api;
32708 |   pContext->pulse.pa_threaded_mainloop_in_thread = (ma_proc)_pa_threaded_mainloop_in_thread;
32709 |   pContext->pulse.pa_threaded_mainloop_set_name = (ma_proc)_pa_threaded_mainloop_set_name;
32710 |   pContext->pulse.pa_context_new          = (ma_proc)_pa_context_new;
32711 |   pContext->pulse.pa_context_unref        = (ma_proc)_pa_context_unref;
32712 |   pContext->pulse.pa_context_connect      = (ma_proc)_pa_context_connect;
32713 |   pContext->pulse.pa_context_disconnect   = (ma_proc)_pa_context_disconnect;
32714 |   pContext->pulse.pa_context_set_state_callback = (ma_proc)_pa_context_set_state_callback;
32715 |   pContext->pulse.pa_context_get_state    = (ma_proc)_pa_context_get_state;
32716 |   pContext->pulse.pa_context_get_sink_info_list = (ma_proc)_pa_context_get_sink_info_list;
32717 |   pContext->pulse.pa_context_get_source_info_list = (ma_proc)_pa_context_get_source_info_list;
32718 |   pContext->pulse.pa_context_get_sink_info_by_name = (ma_proc)_pa_context_get_sink_info_by_name;
32719 |   pContext->pulse.pa_context_get_source_info_by_name =
> (ma_proc)_pa_context_get_source_info_by_name;
32720 |   pContext->pulse.pa_operation_unref      = (ma_proc)_pa_operation_unref;
32721 |   pContext->pulse.pa_operation_get_state  = (ma_proc)_pa_operation_get_state;
32722 |   pContext->pulse.pa_channel_map_init_extend = (ma_proc)_pa_channel_map_init_extend;
32723 |   pContext->pulse.pa_channel_map_valid    = (ma_proc)_pa_channel_map_valid;
32724 |   pContext->pulse.pa_channel_map_compatible = (ma_proc)_pa_channel_map_compatible;
32725 |   pContext->pulse.pa_stream_new           = (ma_proc)_pa_stream_new;
32726 |   pContext->pulse.pa_stream_unref         = (ma_proc)_pa_stream_unref;
32727 |   pContext->pulse.pa_stream_connect_playback = (ma_proc)_pa_stream_connect_playback;
32728 |   pContext->pulse.pa_stream_connect_record = (ma_proc)_pa_stream_connect_record;
32729 |   pContext->pulse.pa_stream_disconnect    = (ma_proc)_pa_stream_disconnect;
32730 |   pContext->pulse.pa_stream_get_state     = (ma_proc)_pa_stream_get_state;
32731 |   pContext->pulse.pa_stream_get_sample_spec = (ma_proc)_pa_stream_get_sample_spec;
32732 |   pContext->pulse.pa_stream_get_channel_map = (ma_proc)_pa_stream_get_channel_map;
32733 |   pContext->pulse.pa_stream_get_buffer_attr = (ma_proc)_pa_stream_get_buffer_attr;
32734 |   pContext->pulse.pa_stream_set_buffer_attr = (ma_proc)_pa_stream_set_buffer_attr;
32735 |   pContext->pulse.pa_stream_get_device_name = (ma_proc)_pa_stream_get_device_name;
32736 |   pContext->pulse.pa_stream_set_write_callback = (ma_proc)_pa_stream_set_write_callback;
32737 |   pContext->pulse.pa_stream_set_read_callback = (ma_proc)_pa_stream_set_read_callback;
32738 |   pContext->pulse.pa_stream_set_suspended_callback = (ma_proc)_pa_stream_set_suspended_callback;
32739 |   pContext->pulse.pa_stream_set_moved_callback = (ma_proc)_pa_stream_set_moved_callback;
32740 |   pContext->pulse.pa_stream_is_suspended = (ma_proc)_pa_stream_is_suspended;
32741 |   pContext->pulse.pa_stream_flush         = (ma_proc)_pa_stream_flush;
32742 |   pContext->pulse.pa_stream_drain         = (ma_proc)_pa_stream_drain;
32743 |   pContext->pulse.pa_stream_is_corked     = (ma_proc)_pa_stream_is_corked;
32744 |   pContext->pulse.pa_stream_cork          = (ma_proc)_pa_stream_cork;
32745 |   pContext->pulse.pa_stream_trigger       = (ma_proc)_pa_stream_trigger;
32746 |   pContext->pulse.pa_stream_begin_write   = (ma_proc)_pa_stream_begin_write;

```



```

32747 |     pContext->pulse.pa_stream_write           = (ma_proc)_pa_stream_write;
32748 |     pContext->pulse.pa_stream_peek           = (ma_proc)_pa_stream_peek;
32749 |     pContext->pulse.pa_stream_drop           = (ma_proc)_pa_stream_drop;
32750 |     pContext->pulse.pa_stream_writable_size   = (ma_proc)_pa_stream_writable_size;
32751 |     pContext->pulse.pa_stream_readable_size   = (ma_proc)_pa_stream_readable_size;
32752 | #endif
32753 |
32754 |     /* We need to make a copy of the application and server names so we can pass them to the
> pa_context of each device. */
32755 |     pContext->pulse.pApplicationName = ma_copy_string(pConfig->pulse.pApplicationName,
> &pContext->allocationCallbacks);
32756 |     if (pContext->pulse.pApplicationName == NULL && pConfig->pulse.pApplicationName != NULL) {
32757 |         return MA_OUT_OF_MEMORY;
32758 |     }
32759 |
32760 |     pContext->pulse.pServerName = ma_copy_string(pConfig->pulse.pServerName,
> &pContext->allocationCallbacks);
32761 |     if (pContext->pulse.pServerName == NULL && pConfig->pulse.pServerName != NULL) {
32762 |         ma_free(pContext->pulse.pApplicationName, &pContext->allocationCallbacks);
32763 |         return MA_OUT_OF_MEMORY;
32764 |     }
32765 |
32766 |     result = ma_init_pa_mainloop_and_pa_context__pulse(pContext, pConfig->pulse.pApplicationName,
> pConfig->pulse.pServerName, pConfig->pulse.tryAutoSpawn, &pContext->pulse.pMainLoop,
> &pContext->pulse.pPulseContext);
32767 |     if (result != MA_SUCCESS) {
32768 |         ma_free(pContext->pulse.pServerName, &pContext->allocationCallbacks);
32769 |         ma_free(pContext->pulse.pApplicationName, &pContext->allocationCallbacks);
32770 |         #ifndef MA_NO_RUNTIME_LINKING
32771 |             ma_dlclose(ma_context_get_log(pContext), pContext->pulse.pulseSO);
32772 |         #endif
32773 |         return result;
32774 |     }
32775 |
32776 |     /* With pa_mainloop we run a synchronous backend, but we implement our own main loop. */
32777 |     pCallbacks->onContextInit                 = ma_context_init__pulse;
32778 |     pCallbacks->onContextUninit                = ma_context_uninit__pulse;
32779 |     pCallbacks->onContextEnumerateDevices      = ma_context_enumerate_devices__pulse;
32780 |     pCallbacks->onContextGetDeviceInfo         = ma_context_get_device_info__pulse;
32781 |     pCallbacks->onDeviceInit                  = ma_device_init__pulse;
32782 |     pCallbacks->onDeviceUninit                = ma_device_uninit__pulse;
32783 |     pCallbacks->onDeviceStart                 = ma_device_start__pulse;
32784 |     pCallbacks->onDeviceStop                  = ma_device_stop__pulse;
32785 |     pCallbacks->onDeviceRead                  = NULL; /* Not used because we're implementing
> onDeviceDataLoop. */
32786 |     pCallbacks->onDeviceWrite                 = NULL; /* Not used because we're implementing
> onDeviceDataLoop. */
32787 |     pCallbacks->onDeviceDataLoop              = ma_device_data_loop__pulse;
32788 |     pCallbacks->onDeviceDataLoopWakeup        = ma_device_data_loop_wakeup__pulse;
32789 |
32790 |     return MA_SUCCESS;
32791 | }
32792 | #endif
32793 |
32794 |
32795 | /*****
32796 |
32797 | JACK Backend
32798 |
32799 | *****/
32800 | #ifdef MA_HAS_JACK
32801 |
32802 | /* It is assumed jack.h is available when compile-time linking is being used. */
32803 | #ifdef MA_NO_RUNTIME_LINKING
32804 | #include <jack/jack.h>
32805 |
32806 | typedef jack_nframes_t      ma_jack_nframes_t;
32807 | typedef jack_options_t      ma_jack_options_t;
32808 | typedef jack_status_t       ma_jack_status_t;
32809 | typedef jack_client_t       ma_jack_client_t;
32810 | typedef jack_port_t         ma_jack_port_t;
32811 | typedef JackProcessCallback ma_JackProcessCallback;
32812 | typedef JackBufferSizeCallback ma_JackBufferSizeCallback;
32813 | typedef JackShutdownCallback ma_JackShutdownCallback;
32814 | #define MA_JACK_DEFAULT_AUDIO_TYPE JACK_DEFAULT_AUDIO_TYPE
32815 | #define ma_JackNullOption JackNullOption
32816 | #define ma_JackNoStartServer JackNoStartServer
32817 | #define ma_JackPortIsInput JackPortIsInput
32818 | #define ma_JackPortIsOutput JackPortIsOutput

```

```

32819 | #define ma_JackPortIsPhysical      JackPortIsPhysical
32820 | #else
32821 | typedef ma_uint32                  ma_jack_nframes_t;
32822 | typedef int                        ma_jack_options_t;
32823 | typedef int                        ma_jack_status_t;
32824 | typedef struct ma_jack_client_t ma_jack_client_t;
32825 | typedef struct ma_jack_port_t  ma_jack_port_t;
32826 | typedef int (* ma_JackProcessCallback) (ma_jack_nframes_t nframes, void* arg);
32827 | typedef int (* ma_JackBufferSizeCallback)(ma_jack_nframes_t nframes, void* arg);
32828 | typedef void (* ma_JackShutdownCallback) (void* arg);
32829 | #define MA_JACK_DEFAULT_AUDIO_TYPE "32 bit float mono audio"
32830 | #define ma_JackNullOption          0
32831 | #define ma_JackNoStartServer       1
32832 | #define ma_JackPortIsInput         1
32833 | #define ma_JackPortIsOutput        2
32834 | #define ma_JackPortIsPhysical      4
32835 | #endif
32836 |
32837 | typedef ma_jack_client_t* (* ma_jack_client_open_proc)      (const char* client_name,
> ma_jack_options_t options, ma_jack_status_t* status, ...);
32838 | typedef int (* ma_jack_client_close_proc) (ma_jack_client_t* client);
32839 | typedef int (* ma_jack_client_name_size_proc) (void);
32840 | typedef int (* ma_jack_set_process_callback_proc) (ma_jack_client_t* client,
> ma_JackProcessCallback process_callback, void* arg);
32841 | typedef int (* ma_jack_set_buffer_size_callback_proc)(ma_jack_client_t* client,
> ma_JackBufferSizeCallback bufsize_callback, void* arg);
32842 | typedef void (* ma_jack_on_shutdown_proc) (ma_jack_client_t* client,
> ma_JackShutdownCallback function, void* arg);
32843 | typedef ma_jack_nframes_t (* ma_jack_get_sample_rate_proc) (ma_jack_client_t* client);
32844 | typedef ma_jack_nframes_t (* ma_jack_get_buffer_size_proc) (ma_jack_client_t* client);
32845 | typedef const char** (* ma_jack_get_ports_proc) (ma_jack_client_t* client, const
> char* port_name_pattern, const char* type_name_pattern, unsigned long flags);
32846 | typedef int (* ma_jack_activate_proc) (ma_jack_client_t* client);
32847 | typedef int (* ma_jack_deactivate_proc) (ma_jack_client_t* client);
32848 | typedef int (* ma_jack_connect_proc) (ma_jack_client_t* client, const
> char* source_port, const char* destination_port);
32849 | typedef ma_jack_port_t* (* ma_jack_port_register_proc) (ma_jack_client_t* client, const
> char* port_name, const char* port_type, unsigned long flags, unsigned long buffer_size);
32850 | typedef const char* (* ma_jack_port_name_proc) (const ma_jack_port_t* port);
32851 | typedef void* (* ma_jack_port_get_buffer_proc) (ma_jack_port_t* port,
> ma_jack_nframes_t nframes);
32852 | typedef void (* ma_jack_free_proc) (void* ptr);
32853 |
32854 | static ma_result ma_context_open_client__jack(ma_context* pContext, ma_jack_client_t** ppClient)
32855 | {
32856 |     size_t maxClientNameSize;
32857 |     char clientName[256];
32858 |     ma_jack_status_t status;
32859 |     ma_jack_client_t* pClient;
32860 |
32861 |     MA_ASSERT(pContext != NULL);
32862 |     MA_ASSERT(ppClient != NULL);
32863 |
32864 |     if (ppClient) {
32865 |         *ppClient = NULL;
32866 |     }
32867 |
32868 |     maxClientNameSize = ((ma_jack_client_name_size_proc)pContext->jack.jack_client_name_size)(); /*
> Includes null terminator. */
32869 |     ma_strncpy_s(clientName, ma_min(sizeof(clientName), maxClientNameSize),
> (pContext->jack.pClientName != NULL) ? pContext->jack.pClientName : "miniaudio", (size_t)-1);
32870 |
32871 |     pClient = ((ma_jack_client_open_proc)pContext->jack.jack_client_open)(clientName,
> (pContext->jack.tryStartServer) ? ma_JackNullOption : ma_JackNoStartServer, &status, NULL);
32872 |     if (pClient == NULL) {
32873 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
32874 |     }
32875 |
32876 |     if (ppClient) {
32877 |         *ppClient = pClient;
32878 |     }
32879 |
32880 |     return MA_SUCCESS;
32881 | }
32882 |
32883 |
32884 | static ma_result ma_context_enumerate_devices__jack(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
32885 | {

```



```

32886 |     ma_bool32 cbResult = MA_TRUE;
32887 |
32888 |     MA_ASSERT(pContext != NULL);
32889 |     MA_ASSERT(callback != NULL);
32890 |
32891 |     /* Playback. */
32892 |     if (cbResult) {
32893 |         ma_device_info deviceInfo;
32894 |         MA_ZERO_OBJECT(&deviceInfo);
32895 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
32896 |         deviceInfo.isDefault = MA_TRUE; /* JACK only uses default devices. */
32897 |         cbResult = callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
32898 |     }
32899 |
32900 |     /* Capture. */
32901 |     if (cbResult) {
32902 |         ma_device_info deviceInfo;
32903 |         MA_ZERO_OBJECT(&deviceInfo);
32904 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
32905 |         deviceInfo.isDefault = MA_TRUE; /* JACK only uses default devices. */
32906 |         cbResult = callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
32907 |     }
32908 |
32909 |     (void)cbResult; /* For silencing a static analysis warning. */
32910 |
32911 |     return MA_SUCCESS;
32912 | }
32913 |
32914 | static ma_result ma_context_get_device_info__jack(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
32915 | {
32916 |     ma_jack_client_t* pClient;
32917 |     ma_result result;
32918 |     const char** ppPorts;
32919 |
32920 |     MA_ASSERT(pContext != NULL);
32921 |
32922 |     if (pDeviceID != NULL && pDeviceID->jack != 0) {
32923 |         return MA_NO_DEVICE; /* Don't know the device. */
32924 |     }
32925 |
32926 |     /* Name / Description */
32927 |     if (deviceType == ma_device_type_playback) {
32928 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
32929 |     } else {
32930 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
32931 |     }
32932 |
32933 |     /* Jack only uses default devices. */
32934 |     pDeviceInfo->isDefault = MA_TRUE;
32935 |
32936 |     /* Jack only supports f32 and has a specific channel count and sample rate. */
32937 |     pDeviceInfo->nativeDataFormats[0].format = ma_format_f32;
32938 |
32939 |     /* The channel count and sample rate can only be determined by opening the device. */
32940 |     result = ma_context_open_client__jack(pContext, &pClient);
32941 |     if (result != MA_SUCCESS) {
32942 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[JACK] Failed to open
> client.");
32943 |         return result;
32944 |     }
32945 |
32946 |     pDeviceInfo->nativeDataFormats[0].sampleRate =
> ((ma_jack_get_sample_rate_proc)pContext->jack.jack_get_sample_rate)((ma_jack_client_t*)pClient);
32947 |     pDeviceInfo->nativeDataFormats[0].channels = 0;
32948 |
32949 |     ppPorts = ((ma_jack_get_ports_proc)pContext->jack.jack_get_ports)((ma_jack_client_t*)pClient,
> NULL, MA_JACK_DEFAULT_AUDIO_TYPE, ma_JackPortIsPhysical | ((deviceType == ma_device_type_playback) ?
> ma_JackPortIsInput : ma_JackPortIsOutput));
32950 |     if (ppPorts == NULL) {
32951 |         ((ma_jack_client_close_proc)pContext->jack.jack_client_close)((ma_jack_client_t*)pClient);
32952 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[JACK] Failed to query
> physical ports.");
32953 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
32954 |     }

```

```

32955 |
32956 |     while (ppPorts[pDeviceInfo->nativeDataFormats[0].channels] != NULL) {
32957 |         pDeviceInfo->nativeDataFormats[0].channels += 1;
32958 |     }
32959 |
32960 |     pDeviceInfo->nativeDataFormats[0].flags = 0;
32961 |     pDeviceInfo->nativeDataFormatCount = 1;
32962 |
32963 |     ((ma_jack_free_proc)pContext->jack.jack_free)((void*)ppPorts);
32964 |     ((ma_jack_client_close_proc)pContext->jack.jack_client_close)((ma_jack_client_t*)pClient);
32965 |
32966 |     (void)pContext;
32967 |     return MA_SUCCESS;
32968 | }
32969 |
32970 |
32971 | static ma_result ma_device_uninit__jack(ma_device* pDevice)
32972 | {
32973 |     ma_context* pContext;
32974 |
32975 |     MA_ASSERT(pDevice != NULL);
32976 |
32977 |     pContext = pDevice->pContext;
32978 |     MA_ASSERT(pContext != NULL);
32979 |
32980 |     if (pDevice->jack.pClient != NULL) {
32981 |         ((ma_jack_client_close_proc)pContext->jack.jack_client_close)((ma_jack_client_t*)pDevice->ja
> ck.pClient);
32982 |     }
32983 |
32984 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
32985 |         ma_free(pDevice->jack.pIntermediaryBufferCapture, &pDevice->pContext->allocationCallbacks);
32986 |         ma_free(pDevice->jack.ppPortsCapture, &pDevice->pContext->allocationCallbacks);
32987 |     }
32988 |
32989 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
32990 |         ma_free(pDevice->jack.pIntermediaryBufferPlayback, &pDevice->pContext->allocationCallbacks);
32991 |         ma_free(pDevice->jack.ppPortsPlayback, &pDevice->pContext->allocationCallbacks);
32992 |     }
32993 |
32994 |     return MA_SUCCESS;
32995 | }
32996 |
32997 | static void ma_device__jack_shutdown_callback(void* pUserData)
32998 | {
32999 |     /* JACK died. Stop the device. */
33000 |     ma_device* pDevice = (ma_device*)pUserData;
33001 |     MA_ASSERT(pDevice != NULL);
33002 |
33003 |     ma_device_stop(pDevice);
33004 | }
33005 |
33006 | static int ma_device__jack_buffer_size_callback(ma_jack_nframes_t frameCount, void* pUserData)
33007 | {
33008 |     ma_device* pDevice = (ma_device*)pUserData;
33009 |     MA_ASSERT(pDevice != NULL);
33010 |
33011 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
33012 |         size_t newBufferSize = frameCount * (pDevice->capture.internalChannels *
> ma_get_bytes_per_sample(pDevice->capture.internalFormat));
33013 |         float* pNewBuffer = (float*)ma_calloc(newBufferSize,
> &pDevice->pContext->allocationCallbacks);
33014 |         if (pNewBuffer == NULL) {
33015 |             return MA_OUT_OF_MEMORY;
33016 |         }
33017 |
33018 |         ma_free(pDevice->jack.pIntermediaryBufferCapture, &pDevice->pContext->allocationCallbacks);
33019 |
33020 |         pDevice->jack.pIntermediaryBufferCapture = pNewBuffer;
33021 |         pDevice->playback.internalPeriodSizeInFrames = frameCount;
33022 |     }
33023 |
33024 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
33025 |         size_t newBufferSize = frameCount * (pDevice->playback.internalChannels *
> ma_get_bytes_per_sample(pDevice->playback.internalFormat));
33026 |         float* pNewBuffer = (float*)ma_calloc(newBufferSize,
> &pDevice->pContext->allocationCallbacks);
33027 |         if (pNewBuffer == NULL) {
33028 |             return MA_OUT_OF_MEMORY;

```

```

33029 |     }
33030 |
33031 |     ma_free(pDevice->jack.pIntermediaryBufferPlayback, &pDevice->pContext->allocationCallbacks);
33032 |
33033 |     pDevice->jack.pIntermediaryBufferPlayback = pNewBuffer;
33034 |     pDevice->playback.internalPeriodSizeInFrames = frameCount;
33035 | }
33036 |
33037 | return 0;
33038 | }
33039 |
33040 | static int ma_device__jack_process_callback(ma_jack_nframes_t frameCount, void* pUserData)
33041 | {
33042 |     ma_device* pDevice;
33043 |     ma_context* pContext;
33044 |     ma_uint32 iChannel;
33045 |
33046 |     pDevice = (ma_device*)pUserData;
33047 |     MA_ASSERT(pDevice != NULL);
33048 |
33049 |     pContext = pDevice->pContext;
33050 |     MA_ASSERT(pContext != NULL);
33051 |
33052 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
33053 |         /* Channels need to be interleaved. */
33054 |         for (iChannel = 0; iChannel < pDevice->capture.internalChannels; ++iChannel) {
33055 |             const float* pSrc = (const float*)((ma_jack_port_get_buffer_proc)pContext->jack.jack_por
> t_get_buffer)((ma_jack_port_t*)pDevice->jack.ppPortsCapture[iChannel], frameCount);
33056 |             if (pSrc != NULL) {
33057 |                 float* pDst = pDevice->jack.pIntermediaryBufferCapture + iChannel;
33058 |                 ma_jack_nframes_t iFrame;
33059 |                 for (iFrame = 0; iFrame < frameCount; ++iFrame) {
33060 |                     *pDst = *pSrc;
33061 |
33062 |                     pDst += pDevice->capture.internalChannels;
33063 |                     pSrc += 1;
33064 |                 }
33065 |             }
33066 |         }
33067 |
33068 |         ma_device_handle_backend_data_callback(pDevice, NULL,
> pDevice->jack.pIntermediaryBufferCapture, frameCount);
33069 |     }
33070 |
33071 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
33072 |         ma_device_handle_backend_data_callback(pDevice, pDevice->jack.pIntermediaryBufferPlayback,
> NULL, frameCount);
33073 |
33074 |         /* Channels need to be deinterleaved. */
33075 |         for (iChannel = 0; iChannel < pDevice->playback.internalChannels; ++iChannel) {
33076 |             float* pDst = (float*)((ma_jack_port_get_buffer_proc)pContext->jack.jack_port_get_buffer
> )((ma_jack_port_t*)pDevice->jack.ppPortsPlayback[iChannel], frameCount);
33077 |             if (pDst != NULL) {
33078 |                 const float* pSrc = pDevice->jack.pIntermediaryBufferPlayback + iChannel;
33079 |                 ma_jack_nframes_t iFrame;
33080 |                 for (iFrame = 0; iFrame < frameCount; ++iFrame) {
33081 |                     *pDst = *pSrc;
33082 |
33083 |                     pDst += 1;
33084 |                     pSrc += pDevice->playback.internalChannels;
33085 |                 }
33086 |             }
33087 |         }
33088 |     }
33089 |
33090 |     return 0;
33091 | }
33092 |
33093 | static ma_result ma_device_init__jack(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
33094 | {
33095 |     ma_result result;
33096 |     ma_uint32 periodSizeInFrames;
33097 |
33098 |     MA_ASSERT(pConfig != NULL);
33099 |     MA_ASSERT(pDevice != NULL);
33100 |
33101 |     if (pConfig->deviceType == ma_device_type_loopback) {
33102 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Loopback mode not

```

```

> supported.");
33103 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
33104 |     }
33105 |
33106 |     /* Only supporting default devices with JACK. */
33107 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->pDeviceID != NULL &&
> pDescriptorPlayback->pDeviceID->jack != 0) ||
33108 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->pDeviceID != NULL &&
> pDescriptorCapture->pDeviceID->jack != 0)) {
33109 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Only default devices are
> supported.");
33110 |         return MA_NO_DEVICE;
33111 |     }
33112 |
33113 |     /* No exclusive mode with the JACK backend. */
33114 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->shareMode == ma_share_mode_exclusive) ||
33115 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->shareMode == ma_share_mode_exclusive)) {
33116 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Exclusive mode not
> supported.");
33117 |         return MA_SHARE_MODE_NOT_SUPPORTED;
33118 |     }
33119 |
33120 |     /* Open the client. */
33121 |     result = ma_context_open_client__jack(pDevice->pContext,
> (ma_jack_client_t*)&pDevice->jack.pClient);
33122 |     if (result != MA_SUCCESS) {
33123 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to open
> client.");
33124 |         return result;
33125 |     }
33126 |
33127 |     /* Callbacks. */
33128 |     if (((ma_jack_set_process_callback_proc)pDevice->pContext->jack.jack_set_process_callback)((ma_j
> ack_client_t*)pDevice->jack.pClient, ma_device__jack_process_callback, pDevice) != 0) {
33129 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to set process
> callback.");
33130 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
33131 |     }
33132 |     if (((ma_jack_set_buffer_size_callback_proc)pDevice->pContext->jack.jack_set_buffer_size_callback
> k)((ma_jack_client_t*)pDevice->jack.pClient, ma_device__jack_buffer_size_callback, pDevice) != 0) {
33133 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to set buffer
> size callback.");
33134 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
33135 |     }
33136 |
33137 |     ((ma_jack_on_shutdown_proc)pDevice->pContext->jack.jack_on_shutdown)((ma_jack_client_t*)pDevice-
> >jack.pClient, ma_device__jack_shutdown_callback, pDevice);
33138 |
33139 |
33140 |     /* The buffer size in frames can change. */
33141 |     periodSizeInFrames = ((ma_jack_get_buffer_size_proc)pDevice->pContext->jack.jack_get_buffer_size
> >)((ma_jack_client_t*)pDevice->jack.pClient);
33142 |
33143 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
33144 |         ma_uint32 iPort;
33145 |         const char** ppPorts;
33146 |
33147 |         pDescriptorCapture->format = ma_format_f32;
33148 |         pDescriptorCapture->channels = 0;
33149 |         pDescriptorCapture->sampleRate = ((ma_jack_get_sample_rate_proc)pDevice->pContext->jack.jack
> _get_sample_rate)((ma_jack_client_t*)pDevice->jack.pClient);
33150 |         ma_channel_map_init_standard(ma_standard_channel_map_alsa, pDescriptorCapture->channelMap,
> ma_countof(pDescriptorCapture->channelMap), pDescriptorCapture->channels);
33151 |
33152 |         ppPorts = ((ma_jack_get_ports_proc)pDevice->pContext->jack.jack_get_ports)((ma_jack_client_t
> *)pDevice->jack.pClient, NULL, MA_JACK_DEFAULT_AUDIO_TYPE, ma_JackPortIsPhysical |
> ma_JackPortIsOutput);
33153 |         if (ppPorts == NULL) {
33154 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to query
> physical ports.");
33155 |             return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
33156 |         }
33157 |
33158 |         /* Need to count the number of ports first so we can allocate some memory. */

```

```

33159 |         while (ppPorts[pDescriptorCapture->channels] != NULL) {
33160 |             pDescriptorCapture->channels += 1;
33161 |         }
33162 |
33163 |         pDevice->jack.ppPortsCapture = (ma_ptr*)ma_malloc(sizeof(*pDevice->jack.ppPortsCapture) *
> pDescriptorCapture->channels, &pDevice->pContext->allocationCallbacks);
33164 |         if (pDevice->jack.ppPortsCapture == NULL) {
33165 |             return MA_OUT_OF_MEMORY;
33166 |         }
33167 |
33168 |         for (iPort = 0; iPort < pDescriptorCapture->channels; iPort += 1) {
33169 |             char name[64];
33170 |             ma_strcpy_s(name, sizeof(name), "capture");
33171 |             ma_itoa_s((int)iPort, name+7, sizeof(name)-7, 10); /* 7 = length of "capture" */
33172 |
33173 |             pDevice->jack.ppPortsCapture[iPort] = ((ma_jack_port_register_proc)pDevice->pContext->ja
> ck.jack_port_register)((ma_jack_client_t*)pDevice->jack.pClient, name, MA_JACK_DEFAULT_AUDIO_TYPE,
> ma_JackPortIsInput, 0);
33174 |             if (pDevice->jack.ppPortsCapture[iPort] == NULL) {
33175 |                 ((ma_jack_free_proc)pDevice->pContext->jack.jack_free)((void*)ppPorts);
33176 |                 ma_device_uninit__jack(pDevice);
33177 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to
> register ports.");
33178 |                 return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
33179 |             }
33180 |         }
33181 |
33182 |         ((ma_jack_free_proc)pDevice->pContext->jack.jack_free)((void*)ppPorts);
33183 |
33184 |         pDescriptorCapture->periodSizeInFrames = periodSizeInFrames;
33185 |         pDescriptorCapture->periodCount = 1; /* There's no notion of a period in JACK. Just
> set to 1. */
33186 |
33187 |         pDevice->jack.pIntermediaryBufferCapture =
> (float*)ma_calloc(pDescriptorCapture->periodSizeInFrames *
> ma_get_bytes_per_frame(pDescriptorCapture->format, pDescriptorCapture->channels),
> &pDevice->pContext->allocationCallbacks);
33188 |         if (pDevice->jack.pIntermediaryBufferCapture == NULL) {
33189 |             ma_device_uninit__jack(pDevice);
33190 |             return MA_OUT_OF_MEMORY;
33191 |         }
33192 |     }
33193 |
33194 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
33195 |         ma_uint32 iPort;
33196 |         const char** ppPorts;
33197 |
33198 |         pDescriptorPlayback->format = ma_format_f32;
33199 |         pDescriptorPlayback->channels = 0;
33200 |         pDescriptorPlayback->sampleRate = ((ma_jack_get_sample_rate_proc)pDevice->pContext->jack.jac
> k_get_sample_rate)((ma_jack_client_t*)pDevice->jack.pClient);
33201 |         ma_channel_map_init_standard(ma_standard_channel_map_alsa, pDescriptorPlayback->channelMap,
> ma_countof(pDescriptorPlayback->channelMap), pDescriptorPlayback->channels);
33202 |
33203 |         ppPorts = ((ma_jack_get_ports_proc)pDevice->pContext->jack.jack_get_ports)((ma_jack_client_t
> *)pDevice->jack.pClient, NULL, MA_JACK_DEFAULT_AUDIO_TYPE, ma_JackPortIsPhysical |
> ma_JackPortIsInput);
33204 |         if (ppPorts == NULL) {
33205 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to query
> physical ports.");
33206 |             return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
33207 |         }
33208 |
33209 |         /* Need to count the number of ports first so we can allocate some memory. */
33210 |         while (ppPorts[pDescriptorPlayback->channels] != NULL) {
33211 |             pDescriptorPlayback->channels += 1;
33212 |         }
33213 |
33214 |         pDevice->jack.ppPortsPlayback = (ma_ptr*)ma_malloc(sizeof(*pDevice->jack.ppPortsPlayback) *
> pDescriptorPlayback->channels, &pDevice->pContext->allocationCallbacks);
33215 |         if (pDevice->jack.ppPortsPlayback == NULL) {
33216 |             ma_free(pDevice->jack.ppPortsCapture, &pDevice->pContext->allocationCallbacks);
33217 |             return MA_OUT_OF_MEMORY;
33218 |         }
33219 |
33220 |         for (iPort = 0; iPort < pDescriptorPlayback->channels; iPort += 1) {
33221 |             char name[64];
33222 |             ma_strcpy_s(name, sizeof(name), "playback");

```

```

33223 |         ma_itoa_s((int)iPort, name+8, sizeof(name)-8, 10); /* 8 = length of "playback" */
33224 |
33225 |         pDevice->jack.ppPortsPlayback[iPort] = ((ma_jack_port_register_proc)pDevice->pContext->j
> ack.jack_port_register)((ma_jack_client_t*)pDevice->jack.pClient, name, MA_JACK_DEFAULT_AUDIO_TYPE,
> ma_JackPortIsOutput, 0);
33226 |         if (pDevice->jack.ppPortsPlayback[iPort] == NULL) {
33227 |             ((ma_jack_free_proc)pDevice->pContext->jack.jack_free)((void*)ppPorts);
33228 |             ma_device_uninit__jack(pDevice);
33229 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to
> register ports.");
33230 |             return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
33231 |         }
33232 |     }
33233 |
33234 |     ((ma_jack_free_proc)pDevice->pContext->jack.jack_free)((void*)ppPorts);
33235 |
33236 |     pDescriptorPlayback->periodSizeInFrames = periodSizeInFrames;
33237 |     pDescriptorPlayback->periodCount = 1; /* There's no notion of a period in JACK.
> Just set to 1. */
33238 |
33239 |     pDevice->jack.pIntermediaryBufferPlayback =
> (float*)ma_malloc(pDescriptorPlayback->periodSizeInFrames *
> ma_get_bytes_per_frame(pDescriptorPlayback->format, pDescriptorPlayback->channels),
> &pDevice->pContext->allocationCallbacks);
33240 |     if (pDevice->jack.pIntermediaryBufferPlayback == NULL) {
33241 |         ma_device_uninit__jack(pDevice);
33242 |         return MA_OUT_OF_MEMORY;
33243 |     }
33244 | }
33245 |
33246 | return MA_SUCCESS;
33247 | }
33248 |
33249 |
33250 | static ma_result ma_device_start__jack(ma_device* pDevice)
33251 | {
33252 |     ma_context* pContext = pDevice->pContext;
33253 |     int resultJACK;
33254 |     size_t i;
33255 |
33256 |     resultJACK =
> ((ma_jack_activate_proc)pContext->jack.jack_activate)((ma_jack_client_t*)pDevice->jack.pClient);
33257 |     if (resultJACK != 0) {
33258 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to activate the
> JACK client.");
33259 |         return MA_FAILED_TO_START_BACKEND_DEVICE;
33260 |     }
33261 |
33262 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
33263 |         const char** ppServerPorts =
> ((ma_jack_get_ports_proc)pContext->jack.jack_get_ports)((ma_jack_client_t*)pDevice->jack.pClient,
> NULL, MA_JACK_DEFAULT_AUDIO_TYPE, ma_JackPortIsPhysical | ma_JackPortIsOutput);
33264 |         if (ppServerPorts == NULL) {
33265 |             > ((ma_jack_deactivate_proc)pContext->jack.jack_deactivate)((ma_jack_client_t*)pDevice->jack.pClient);
33266 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to retrieve
> physical ports.");
33267 |             return MA_ERROR;
33268 |         }
33269 |
33270 |         for (i = 0; ppServerPorts[i] != NULL; ++i) {
33271 |             const char* pServerPort = ppServerPorts[i];
33272 |             const char* pClientPort = ((ma_jack_port_name_proc)pContext->jack.jack_port_name)((ma_ja
> ck_port_t*)pDevice->jack.ppPortsCapture[i]);
33273 |
33274 |             resultJACK =
> ((ma_jack_connect_proc)pContext->jack.jack_connect)((ma_jack_client_t*)pDevice->jack.pClient,
> pServerPort, pClientPort);
33275 |             if (resultJACK != 0) {
33276 |                 ((ma_jack_free_proc)pContext->jack.jack_free)((void*)ppServerPorts);
33277 |
> ((ma_jack_deactivate_proc)pContext->jack.jack_deactivate)((ma_jack_client_t*)pDevice->jack.pClient);
33278 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to
> connect ports.");
33279 |                 return MA_ERROR;
33280 |             }
33281 |         }
33282 |
33283 |         ((ma_jack_free_proc)pContext->jack.jack_free)((void*)ppServerPorts);

```

```

33284 |     }
33285 |
33286 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
33287 |         const char** ppServerPorts =
> ((ma_jack_get_ports_proc)pContext->jack.jack_get_ports)((ma_jack_client_t*)pDevice->jack.pClient,
> NULL, MA_JACK_DEFAULT_AUDIO_TYPE, ma_JackPortIsPhysical | ma_JackPortIsInput);
33288 |         if (ppServerPorts == NULL) {
33289 |
> ((ma_jack_deactivate_proc)pContext->jack.jack_deactivate)((ma_jack_client_t*)pDevice->jack.pClient);
33290 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to retrieve
> physical ports.");
33291 |             return MA_ERROR;
33292 |         }
33293 |
33294 |         for (i = 0; ppServerPorts[i] != NULL; ++i) {
33295 |             const char* pServerPort = ppServerPorts[i];
33296 |             const char* pClientPort = ((ma_jack_port_name_proc)pContext->jack.jack_port_name)((ma_ja
> ck_port_t*)pDevice->jack.ppPortsPlayback[i]);
33297 |
33298 |             resultJACK =
> ((ma_jack_connect_proc)pContext->jack.jack_connect)((ma_jack_client_t*)pDevice->jack.pClient,
> pClientPort, pServerPort);
33299 |             if (resultJACK != 0) {
33300 |                 ((ma_jack_free_proc)pContext->jack.jack_free)((void*)ppServerPorts);
33301 |
> ((ma_jack_deactivate_proc)pContext->jack.jack_deactivate)((ma_jack_client_t*)pDevice->jack.pClient);
33302 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] Failed to
> connect ports.");
33303 |                 return MA_ERROR;
33304 |             }
33305 |         }
33306 |
33307 |         ((ma_jack_free_proc)pContext->jack.jack_free)((void*)ppServerPorts);
33308 |     }
33309 |
33310 |     return MA_SUCCESS;
33311 | }
33312 |
33313 | static ma_result ma_device_stop__jack(ma_device* pDevice)
33314 | {
33315 |     ma_context* pContext = pDevice->pContext;
33316 |
33317 |     if
> ((ma_jack_deactivate_proc)pContext->jack.jack_deactivate)((ma_jack_client_t*)pDevice->jack.pClient)
> != 0) {
33318 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[JACK] An error occurred when
> deactivating the JACK client.");
33319 |         return MA_ERROR;
33320 |     }
33321 |
33322 |     ma_device__on_notification_stopped(pDevice);
33323 |
33324 |     return MA_SUCCESS;
33325 | }
33326 |
33327 |
33328 | static ma_result ma_context_uninit__jack(ma_context* pContext)
33329 | {
33330 |     MA_ASSERT(pContext != NULL);
33331 |     MA_ASSERT(pContext->backend == ma_backend_jack);
33332 |
33333 |     ma_free(pContext->jack.pClientName, &pContext->allocationCallbacks);
33334 |     pContext->jack.pClientName = NULL;
33335 |
33336 | #ifndef MA_NO_RUNTIME_LINKING
33337 |     ma_dlclose(ma_context_get_log(pContext), pContext->jack.jackSO);
33338 | #endif
33339 |
33340 |     return MA_SUCCESS;
33341 | }
33342 |
33343 | static ma_result ma_context_init__jack(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
33344 | {
33345 | #ifndef MA_NO_RUNTIME_LINKING
33346 |     const char* libjackNames[] = {
33347 | #if defined(MA_WIN32)
33348 |         "libjack.dll",
33349 |         "libjack64.dll"

```



```

33350 | #endif
33351 | #if defined(MA_UNIX)
33352 |     "libjack.so",
33353 |     "libjack.so.0"
33354 | #endif
33355 | };
33356 | size_t i;
33357 |
33358 | for (i = 0; i < ma_countof(libjackNames); ++i) {
33359 |     pContext->jack.jackSO = ma_dlopen(ma_context_get_log(pContext), libjackNames[i]);
33360 |     if (pContext->jack.jackSO != NULL) {
33361 |         break;
33362 |     }
33363 | }
33364 |
33365 | if (pContext->jack.jackSO == NULL) {
33366 |     return MA_NO_BACKEND;
33367 | }
33368 |
33369 | pContext->jack.jack_client_open          = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_client_open");
33370 | pContext->jack.jack_client_close         = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_client_close");
33371 | pContext->jack.jack_client_name_size     = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_client_name_size");
33372 | pContext->jack.jack_set_process_callback = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_set_process_callback");
33373 | pContext->jack.jack_set_buffer_size_callback = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_set_buffer_size_callback");
33374 | pContext->jack.jack_on_shutdown          = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_on_shutdown");
33375 | pContext->jack.jack_get_sample_rate      = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_get_sample_rate");
33376 | pContext->jack.jack_get_buffer_size      = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_get_buffer_size");
33377 | pContext->jack.jack_get_ports            = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_get_ports");
33378 | pContext->jack.jack_activate             = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_activate");
33379 | pContext->jack.jack_deactivate           = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_deactivate");
33380 | pContext->jack.jack_connect              = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_connect");
33381 | pContext->jack.jack_port_register        = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_port_register");
33382 | pContext->jack.jack_port_name            = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_port_name");
33383 | pContext->jack.jack_port_get_buffer      = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_port_get_buffer");
33384 | pContext->jack.jack_free                 = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->jack.jackSO, "jack_free");
33385 | #else
33386 | /*
33387 | This strange assignment system is here just to ensure type safety of miniaudio's function
> pointer
33388 | types. If anything differs slightly the compiler should throw a warning.
33389 | */
33390 | ma_jack_client_open_proc      _jack_client_open          = jack_client_open;
33391 | ma_jack_client_close_proc     _jack_client_close         = jack_client_close;
33392 | ma_jack_client_name_size_proc _jack_client_name_size     = jack_client_name_size;
33393 | ma_jack_set_process_callback_proc _jack_set_process_callback =
> jack_set_process_callback;
33394 | ma_jack_set_buffer_size_callback_proc _jack_set_buffer_size_callback =
> jack_set_buffer_size_callback;
33395 | ma_jack_on_shutdown_proc      _jack_on_shutdown          = jack_on_shutdown;
33396 | ma_jack_get_sample_rate_proc   _jack_get_sample_rate      = jack_get_sample_rate;
33397 | ma_jack_get_buffer_size_proc   _jack_get_buffer_size      = jack_get_buffer_size;
33398 | ma_jack_get_ports_proc        _jack_get_ports            = jack_get_ports;
33399 | ma_jack_activate_proc         _jack_activate             = jack_activate;
3400 | ma_jack_deactivate_proc       _jack_deactivate           = jack_deactivate;
3401 | ma_jack_connect_proc          _jack_connect              = jack_connect;
3402 | ma_jack_port_register_proc     _jack_port_register        = jack_port_register;
3403 | ma_jack_port_name_proc        _jack_port_name            = jack_port_name;
3404 | ma_jack_port_get_buffer_proc   _jack_port_get_buffer      = jack_port_get_buffer;
3405 | ma_jack_free_proc             _jack_free                 = jack_free;
3406 |
3407 | pContext->jack.jack_client_open          = (ma_proc)_jack_client_open;
3408 | pContext->jack.jack_client_close         = (ma_proc)_jack_client_close;
3409 | pContext->jack.jack_client_name_size     = (ma_proc)_jack_client_name_size;

```

```

33410 | pContext->jack.jack_set_process_callback = (ma_proc)_jack_set_process_callback;
33411 | pContext->jack.jack_set_buffer_size_callback = (ma_proc)_jack_set_buffer_size_callback;
33412 | pContext->jack.jack_on_shutdown = (ma_proc)_jack_on_shutdown;
33413 | pContext->jack.jack_get_sample_rate = (ma_proc)_jack_get_sample_rate;
33414 | pContext->jack.jack_get_buffer_size = (ma_proc)_jack_get_buffer_size;
33415 | pContext->jack.jack_get_ports = (ma_proc)_jack_get_ports;
33416 | pContext->jack.jack_activate = (ma_proc)_jack_activate;
33417 | pContext->jack.jack_deactivate = (ma_proc)_jack_deactivate;
33418 | pContext->jack.jack_connect = (ma_proc)_jack_connect;
33419 | pContext->jack.jack_port_register = (ma_proc)_jack_port_register;
33420 | pContext->jack.jack_port_name = (ma_proc)_jack_port_name;
33421 | pContext->jack.jack_port_get_buffer = (ma_proc)_jack_port_get_buffer;
33422 | pContext->jack.jack_free = (ma_proc)_jack_free;
33423 | #endif
33424 |
33425 |     if (pConfig->jack.pClientName != NULL) {
33426 |         pContext->jack.pClientName = ma_copy_string(pConfig->jack.pClientName,
> &pContext->allocationCallbacks);
33427 |     }
33428 |     pContext->jack.tryStartServer = pConfig->jack.tryStartServer;
33429 |
33430 |     /*
33431 |     Getting here means the JACK library is installed, but it doesn't necessarily mean it's usable.
> We need to quickly test this by connecting
33432 |     a temporary client.
33433 |     */
33434 |     {
33435 |         ma_jack_client_t* pDummyClient;
33436 |         ma_result result = ma_context_open_client__jack(pContext, &pDummyClient);
33437 |         if (result != MA_SUCCESS) {
33438 |             ma_free(pContext->jack.pClientName, &pContext->allocationCallbacks);
33439 |             #ifndef MA_NO_RUNTIME_LINKING
33440 |                 ma_dlcclose(ma_context_get_log(pContext), pContext->jack.jackSO);
33441 |             #endif
33442 |             return MA_NO_BACKEND;
33443 |         }
33444 |     }
33445 |
> ((ma_jack_client_close_proc)pContext->jack.jack_client_close)((ma_jack_client_t*)pDummyClient);
33446 |     }
33447 |
33448 |
33449 |     pCallbacks->onContextInit = ma_context_init__jack;
33450 |     pCallbacks->onContextUninit = ma_context_uninit__jack;
33451 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__jack;
33452 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__jack;
33453 |     pCallbacks->onDeviceInit = ma_device_init__jack;
33454 |     pCallbacks->onDeviceUninit = ma_device_uninit__jack;
33455 |     pCallbacks->onDeviceStart = ma_device_start__jack;
33456 |     pCallbacks->onDeviceStop = ma_device_stop__jack;
33457 |     pCallbacks->onDeviceRead = NULL; /* Not used because JACK is asynchronous. */
33458 |     pCallbacks->onDeviceWrite = NULL; /* Not used because JACK is asynchronous. */
33459 |     pCallbacks->onDeviceDataLoop = NULL; /* Not used because JACK is asynchronous. */
33460 |
33461 |     return MA_SUCCESS;
33462 | }
33463 | #endif /* MA_HAS_JACK */
33464 |
33465 |
33466 |
33467 | /*****
33468 |
33469 | Core Audio Backend
33470 |
33471 | References
33472 | =====
33473 | - Technical Note TN2091: Device input using the HAL Output Audio Unit
33474 |   https://developer.apple.com/library/archive/technotes/tn2091/_index.html
33475 |
33476 | *****/
33477 | #ifdef MA_HAS_COREAUDIO
33478 | #include <TargetConditionals.h>
33479 |
33480 | #if defined(TARGET_OS_IPHONE) && TARGET_OS_IPHONE == 1
33481 |     #define MA_APPLE_MOBILE
33482 | #if defined(TARGET_OS_TV) && TARGET_OS_TV == 1
33483 |     #define MA_APPLE_TV
33484 | #endif
33485 | #if defined(TARGET_OS_WATCH) && TARGET_OS_WATCH == 1

```

```

33486 |     #define MA_APPLE_WATCH
33487 | #endif
33488 | #if __has_feature(objc_arc)
33489 |     #define MA_BRIDGE_TRANSFER    __bridge_transfer
33490 |     #define MA_BRIDGE_RETAINED    __bridge_retained
33491 | #else
33492 |     #define MA_BRIDGE_TRANSFER
33493 |     #define MA_BRIDGE_RETAINED
33494 | #endif
33495 | #else
33496 |     #define MA_APPLE_DESKTOP
33497 | #endif
33498 |
33499 | #if defined(MA_APPLE_DESKTOP)
33500 | #include <CoreAudio/CoreAudio.h>
33501 | #else
33502 | #include <AVFoundation/AVFoundation.h>
33503 | #endif
33504 |
33505 | #include <AudioToolbox/AudioToolbox.h>
33506 |
33507 | /* CoreFoundation */
33508 | typedef Boolean (* ma_CFStringGetCString_proc)(CFStringRef theString, char* buffer, CFIndex
> bufferSize, CFStringEncoding encoding);
33509 | typedef void (* ma_CFRelease_proc)(CTypeRef cf);
33510 |
33511 | /* CoreAudio */
33512 | #if defined(MA_APPLE_DESKTOP)
33513 | typedef OSStatus (* ma_AudioObjectGetPropertyData_proc)(AudioObjectID inObjectID, const
> AudioObjectPropertyAddress* inAddress, UInt32 inQualifierDataSize, const void* inQualifierData,
> UInt32* ioDataSize, void* outData);
33514 | typedef OSStatus (* ma_AudioObjectGetPropertyDataSize_proc)(AudioObjectID inObjectID, const
> AudioObjectPropertyAddress* inAddress, UInt32 inQualifierDataSize, const void* inQualifierData,
> UInt32* outDataSize);
33515 | typedef OSStatus (* ma_AudioObjectSetPropertyData_proc)(AudioObjectID inObjectID, const
> AudioObjectPropertyAddress* inAddress, UInt32 inQualifierDataSize, const void* inQualifierData,
> UInt32 inDataSize, const void* inData);
33516 | typedef OSStatus (* ma_AudioObjectAddPropertyListener_proc)(AudioObjectID inObjectID, const
> AudioObjectPropertyAddress* inAddress, AudioObjectPropertyListenerProc inListener, void*
> inClientData);
33517 | typedef OSStatus (* ma_AudioObjectRemovePropertyListener_proc)(AudioObjectID inObjectID, const
> AudioObjectPropertyAddress* inAddress, AudioObjectPropertyListenerProc inListener, void*
> inClientData);
33518 | #endif
33519 |
33520 | /* AudioToolbox */
33521 | typedef AudioComponent (* ma_AudioComponentFindNext_proc)(AudioComponent inComponent, const
> AudioComponentDescription* inDesc);
33522 | typedef OSStatus (* ma_AudioComponentInstanceDispose_proc)(AudioComponentInstance inInstance);
33523 | typedef OSStatus (* ma_AudioComponentInstanceNew_proc)(AudioComponent inComponent,
> AudioComponentInstance* outInstance);
33524 | typedef OSStatus (* ma_AudioOutputUnitStart_proc)(AudioUnit inUnit);
33525 | typedef OSStatus (* ma_AudioOutputUnitStop_proc)(AudioUnit inUnit);
33526 | typedef OSStatus (* ma_AudioUnitAddPropertyListener_proc)(AudioUnit inUnit, AudioUnitPropertyID
> inID, AudioUnitPropertyListenerProc inProc, void* inProcUserData);
33527 | typedef OSStatus (* ma_AudioUnitGetPropertyInfo_proc)(AudioUnit inUnit, AudioUnitPropertyID inID,
> AudioUnitScope inScope, AudioUnitElement inElement, UInt32* outDataSize, Boolean* outWriteable);
33528 | typedef OSStatus (* ma_AudioUnitGetProperty_proc)(AudioUnit inUnit, AudioUnitPropertyID inID,
> AudioUnitScope inScope, AudioUnitElement inElement, void* outData, UInt32* ioDataSize);
33529 | typedef OSStatus (* ma_AudioUnitSetProperty_proc)(AudioUnit inUnit, AudioUnitPropertyID inID,
> AudioUnitScope inScope, AudioUnitElement inElement, const void* inData, UInt32 inDataSize);
33530 | typedef OSStatus (* ma_AudioUnitInitialize_proc)(AudioUnit inUnit);
33531 | typedef OSStatus (* ma_AudioUnitRender_proc)(AudioUnit inUnit, AudioUnitRenderActionFlags*
> ioActionFlags, const AudioTimeStamp* inTimeStamp, UInt32 inOutputBusNumber, UInt32 inNumberFrames,
> AudioBufferList* ioData);
33532 |
33533 |
33534 | #define MA_COREAUDIO_OUTPUT_BUS    0
33535 | #define MA_COREAUDIO_INPUT_BUS     1
33536 |
33537 | #if defined(MA_APPLE_DESKTOP)
33538 | static ma_result ma_device_reinit_internal__coreaudio(ma_device* pDevice, ma_device_type deviceType,
> ma_bool32 disposePreviousAudioUnit);
33539 | #endif
33540 |
33541 | /*
33542 | Core Audio
33543 |
33544 | So far, Core Audio has been the worst backend to work with due to being both unintuitive and having

```

```

> almost no documentation
33545 | apart from comments in the headers (which admittedly are quite good). For my own purposes, and for
> anybody out there whose
33546 | needing to figure out how this darn thing works, I'm going to outline a few things here.
33547 |
33548 | Since miniaudio is a fairly low-level API, one of the things it needs is control over specific
> devices, and it needs to be
33549 | able to identify whether or not it can be used as playback and/or capture. The AudioObject API is
> the only one I've seen
33550 | that supports this level of detail. There was some public domain sample code I stumbled across that
> used the AudioComponent
33551 | and AudioUnit APIs, but I couldn't see anything that gave low-level control over device selection
> and capabilities (the
33552 | distinction between playback and capture in particular). Therefore, miniaudio is using the
> AudioObject API.
33553 |
33554 | Most (all?) functions in the AudioObject API take a AudioObjectID as its input. This is the device
> identifier. When
33555 | retrieving global information, such as the device list, you use kAudioObjectSystemObject. When
> retrieving device-specific
33556 | data, you pass in the ID for that device. In order to retrieve device-specific IDs you need to
> enumerate over each of the
33557 | devices. This is done using the AudioObjectGetPropertyDataSize() and AudioObjectGetPropertyData()
> APIs which seem to be
33558 | the central APIs for retrieving information about the system and specific devices.
33559 |
33560 | To use the AudioObjectGetPropertyData() API you need to use the notion of a property address. A
> property address is a
33561 | structure with three variables and is used to identify which property you are getting or setting.
> The first is the "selector"
33562 | which is basically the specific property that you're wanting to retrieve or set. The second is the
> "scope", which is
33563 | typically set to kAudioObjectPropertyScopeGlobal, kAudioObjectPropertyScopeInput for input-specific
> properties and
33564 | kAudioObjectPropertyScopeOutput for output-specific properties. The last is the "element" which is
> always set to
33565 | kAudioObjectPropertyElementMain in miniaudio's case. I don't know of any cases where this would be
> set to anything different.
33566 |
33567 | Back to the earlier issue of device retrieval, you first use the AudioObjectGetPropertyDataSize()
> API to retrieve the size
33568 | of the raw data which is just a list of AudioDeviceID's. You use the kAudioObjectSystemObject
> AudioObjectID, and a property
33569 | address with the kAudioHardwarePropertyDevices selector and the kAudioObjectPropertyScopeGlobal
> scope. Once you have the
33570 | size, allocate a block of memory of that size and then call AudioObjectGetPropertyData(). The data
> is just a list of
33571 | AudioDeviceID's so just do "dataSize/sizeof(AudioDeviceID)" to know the device count.
33572 | */
33573 |
33574 | #if defined(MA_APPLE_MOBILE)
33575 | static void ma_device__on_notification_interruption_began(ma_device* pDevice)
33576 | {
33577 |     ma_device__on_notification(ma_device_notification_init(pDevice,
> ma_device_notification_type_interruption_began));
33578 | }
33579 |
33580 | static void ma_device__on_notification_interruption_ended(ma_device* pDevice)
33581 | {
33582 |     ma_device__on_notification(ma_device_notification_init(pDevice,
> ma_device_notification_type_interruption_ended));
33583 | }
33584 | #endif
33585 |
33586 | static ma_result ma_result_from_OSStatus(OSStatus status)
33587 | {
33588 |     switch (status)
33589 |     {
33590 |         case noErr: return MA_SUCCESS;
33591 |         #if defined(MA_APPLE_DESKTOP)
33592 |         case kAudioHardwareNotRunningError: return MA_DEVICE_NOT_STARTED;
33593 |         case kAudioHardwareUnspecifiedError: return MA_ERROR;
33594 |         case kAudioHardwareUnknownPropertyError: return MA_INVALID_ARGS;
33595 |         case kAudioHardwareBadPropertySizeError: return MA_INVALID_OPERATION;
33596 |         case kAudioHardwareIllegalOperationError: return MA_INVALID_OPERATION;
33597 |         case kAudioHardwareBadObjectError: return MA_INVALID_ARGS;
33598 |         case kAudioHardwareBadDeviceError: return MA_INVALID_ARGS;
33599 |         case kAudioHardwareBadStreamError: return MA_INVALID_ARGS;
33600 |         case kAudioHardwareUnsupportedOperationError: return MA_INVALID_OPERATION;

```

```

33601 |         case kAudioDeviceUnsupportedFormatError:    return MA_FORMAT_NOT_SUPPORTED;
33602 |         case kAudioDevicePermissionsError:         return MA_ACCESS_DENIED;
33603 |     #endif
33604 |     default:                                         return MA_ERROR;
33605 | }
33606 | }
33607 |
33608 | #if 0
33609 | static ma_channel ma_channel_from_AudioChannelBitmap(AudioChannelBitmap bit)
33610 | {
33611 |     switch (bit)
33612 |     {
33613 |         case kAudioChannelBit_Left:                 return MA_CHANNEL_LEFT;
33614 |         case kAudioChannelBit_Right:                return MA_CHANNEL_RIGHT;
33615 |         case kAudioChannelBit_Center:               return MA_CHANNEL_FRONT_CENTER;
33616 |         case kAudioChannelBit_LFEScreen:            return MA_CHANNEL_LFE;
33617 |         case kAudioChannelBit_LeftSurround:         return MA_CHANNEL_BACK_LEFT;
33618 |         case kAudioChannelBit_RightSurround:        return MA_CHANNEL_BACK_RIGHT;
33619 |         case kAudioChannelBit_LeftCenter:           return MA_CHANNEL_FRONT_LEFT_CENTER;
33620 |         case kAudioChannelBit_RightCenter:          return MA_CHANNEL_FRONT_RIGHT_CENTER;
33621 |         case kAudioChannelBit_CenterSurround:       return MA_CHANNEL_BACK_CENTER;
33622 |         case kAudioChannelBit_LeftSurroundDirect:   return MA_CHANNEL_SIDE_LEFT;
33623 |         case kAudioChannelBit_RightSurroundDirect:  return MA_CHANNEL_SIDE_RIGHT;
33624 |         case kAudioChannelBit_TopCenterSurround:    return MA_CHANNEL_TOP_CENTER;
33625 |         case kAudioChannelBit_VerticalHeightLeft:   return MA_CHANNEL_TOP_FRONT_LEFT;
33626 |         case kAudioChannelBit_VerticalHeightCenter: return MA_CHANNEL_TOP_FRONT_CENTER;
33627 |         case kAudioChannelBit_VerticalHeightRight:  return MA_CHANNEL_TOP_FRONT_RIGHT;
33628 |         case kAudioChannelBit_TopBackLeft:          return MA_CHANNEL_TOP_BACK_LEFT;
33629 |         case kAudioChannelBit_TopBackCenter:        return MA_CHANNEL_TOP_BACK_CENTER;
33630 |         case kAudioChannelBit_TopBackRight:         return MA_CHANNEL_TOP_BACK_RIGHT;
33631 |         default:                                     return MA_CHANNEL_NONE;
33632 |     }
33633 | }
33634 | #endif
33635 |
33636 | static ma_result ma_format_from_AudioStreamBasicDescription(const AudioStreamBasicDescription*
> pDescription, ma_format* pFormatOut)
33637 | {
33638 |     MA_ASSERT(pDescription != NULL);
33639 |     MA_ASSERT(pFormatOut != NULL);
33640 |
33641 |     *pFormatOut = ma_format_unknown;    /* Safety. */
33642 |
33643 |     /* There's a few things miniaudio doesn't support. */
33644 |     if (pDescription->mFormatID != kAudioFormatLinearPCM) {
33645 |         return MA_FORMAT_NOT_SUPPORTED;
33646 |     }
33647 |
33648 |     /* We don't support any non-packed formats that are aligned high. */
33649 |     if ((pDescription->mFormatFlags & kLinearPCMFormatFlagIsAlignedHigh) != 0) {
33650 |         return MA_FORMAT_NOT_SUPPORTED;
33651 |     }
33652 |
33653 |     /* Only supporting native-endian. */
33654 |     if ((ma_is_little_endian() && (pDescription->mFormatFlags & kAudioFormatFlagIsBigEndian) != 0)
> || (ma_is_big_endian() && (pDescription->mFormatFlags & kAudioFormatFlagIsBigEndian) == 0)) {
33655 |         return MA_FORMAT_NOT_SUPPORTED;
33656 |     }
33657 |
33658 |     /* We are not currently supporting non-interleaved formats (this will be added in a future
> version of miniaudio). */
33659 |     /*if ((pDescription->mFormatFlags & kAudioFormatFlagIsNonInterleaved) != 0) {
33660 |         return MA_FORMAT_NOT_SUPPORTED;
33661 |     }*/
33662 |
33663 |     if ((pDescription->mFormatFlags & kLinearPCMFormatFlagIsFloat) != 0) {
33664 |         if (pDescription->mBitsPerChannel == 32) {
33665 |             *pFormatOut = ma_format_f32;
33666 |             return MA_SUCCESS;
33667 |         }
33668 |     } else {
33669 |         if ((pDescription->mFormatFlags & kLinearPCMFormatFlagIsSignedInteger) != 0) {
33670 |             if (pDescription->mBitsPerChannel == 16) {
33671 |                 *pFormatOut = ma_format_s16;
33672 |                 return MA_SUCCESS;
33673 |             } else if (pDescription->mBitsPerChannel == 24) {
33674 |                 if (pDescription->mBytesPerFrame == (pDescription->mBitsPerChannel/8 *
> pDescription->mChannelsPerFrame)) {
33675 |                     *pFormatOut = ma_format_s24;

```

```

33676 |         return MA_SUCCESS;
33677 |     } else {
33678 |         if (pDescription->mBytesPerFrame/pDescription->mChannelsPerFrame ==
> sizeof(ma_int32)) {
33679 |             /* TODO: Implement ma_format_s24_32. */
33680 |             /**pFormatOut = ma_format_s24_32;*/
33681 |             /*return MA_SUCCESS;*/
33682 |             return MA_FORMAT_NOT_SUPPORTED;
33683 |         }
33684 |     }
33685 |     } else if (pDescription->mBitsPerChannel == 32) {
33686 |         *pFormatOut = ma_format_s32;
33687 |         return MA_SUCCESS;
33688 |     }
33689 |     } else {
33690 |         if (pDescription->mBitsPerChannel == 8) {
33691 |             *pFormatOut = ma_format_u8;
33692 |             return MA_SUCCESS;
33693 |         }
33694 |     }
33695 | }
33696 |
33697 | /* Getting here means the format is not supported. */
33698 | return MA_FORMAT_NOT_SUPPORTED;
33699 | }
33700 |
33701 | #if defined(MA_APPLE_DESKTOP)
33702 | static ma_channel ma_channel_from_AudioChannelLabel(AudioChannelLabel label)
33703 | {
33704 |     switch (label)
33705 |     {
33706 |         case kAudioChannelLabel_Unknown:           return MA_CHANNEL_NONE;
33707 |         case kAudioChannelLabel_Unused:            return MA_CHANNEL_NONE;
33708 |         case kAudioChannelLabel_UseCoordinates:    return MA_CHANNEL_NONE;
33709 |         case kAudioChannelLabel_Left:              return MA_CHANNEL_LEFT;
33710 |         case kAudioChannelLabel_Right:             return MA_CHANNEL_RIGHT;
33711 |         case kAudioChannelLabel_Center:            return MA_CHANNEL_FRONT_CENTER;
33712 |         case kAudioChannelLabel_LFEScreen:         return MA_CHANNEL_LFE;
33713 |         case kAudioChannelLabel_LeftSurround:      return MA_CHANNEL_BACK_LEFT;
33714 |         case kAudioChannelLabel_RightSurround:     return MA_CHANNEL_BACK_RIGHT;
33715 |         case kAudioChannelLabel_LeftCenter:        return MA_CHANNEL_FRONT_LEFT_CENTER;
33716 |         case kAudioChannelLabel_RightCenter:       return MA_CHANNEL_FRONT_RIGHT_CENTER;
33717 |         case kAudioChannelLabel_CenterSurround:    return MA_CHANNEL_BACK_CENTER;
33718 |         case kAudioChannelLabel_LeftSurroundDirect: return MA_CHANNEL_SIDE_LEFT;
33719 |         case kAudioChannelLabel_RightSurroundDirect: return MA_CHANNEL_SIDE_RIGHT;
33720 |         case kAudioChannelLabel_TopCenterSurround: return MA_CHANNEL_TOP_CENTER;
33721 |         case kAudioChannelLabel_VerticalHeightLeft: return MA_CHANNEL_TOP_FRONT_LEFT;
33722 |         case kAudioChannelLabel_VerticalHeightCenter: return MA_CHANNEL_TOP_FRONT_CENTER;
33723 |         case kAudioChannelLabel_VerticalHeightRight: return MA_CHANNEL_TOP_FRONT_RIGHT;
33724 |         case kAudioChannelLabel_TopBackLeft:      return MA_CHANNEL_TOP_BACK_LEFT;
33725 |         case kAudioChannelLabel_TopBackCenter:    return MA_CHANNEL_TOP_BACK_CENTER;
33726 |         case kAudioChannelLabel_TopBackRight:     return MA_CHANNEL_TOP_BACK_RIGHT;
33727 |         case kAudioChannelLabel_RearSurroundLeft:  return MA_CHANNEL_BACK_LEFT;
33728 |         case kAudioChannelLabel_RearSurroundRight: return MA_CHANNEL_BACK_RIGHT;
33729 |         case kAudioChannelLabel_LeftWide:         return MA_CHANNEL_SIDE_LEFT;
33730 |         case kAudioChannelLabel_RightWide:        return MA_CHANNEL_SIDE_RIGHT;
33731 |         case kAudioChannelLabel_LFE2:             return MA_CHANNEL_LFE;
33732 |         case kAudioChannelLabel_LeftTotal:        return MA_CHANNEL_LEFT;
33733 |         case kAudioChannelLabel_RightTotal:       return MA_CHANNEL_RIGHT;
33734 |         case kAudioChannelLabel_HearingImpaired:  return MA_CHANNEL_NONE;
33735 |         case kAudioChannelLabel_Narration:        return MA_CHANNEL_MONO;
33736 |         case kAudioChannelLabel_Mono:             return MA_CHANNEL_MONO;
33737 |         case kAudioChannelLabel_DialogCentricMix: return MA_CHANNEL_MONO;
33738 |         case kAudioChannelLabel_CenterSurroundDirect: return MA_CHANNEL_BACK_CENTER;
33739 |         case kAudioChannelLabel_Haptic:           return MA_CHANNEL_NONE;
33740 |         case kAudioChannelLabel_Ambisonic_W:      return MA_CHANNEL_NONE;
33741 |         case kAudioChannelLabel_Ambisonic_X:      return MA_CHANNEL_NONE;
33742 |         case kAudioChannelLabel_Ambisonic_Y:      return MA_CHANNEL_NONE;
33743 |         case kAudioChannelLabel_Ambisonic_Z:      return MA_CHANNEL_NONE;
33744 |         case kAudioChannelLabel_MS_Mid:           return MA_CHANNEL_LEFT;
33745 |         case kAudioChannelLabel_MS_Side:          return MA_CHANNEL_RIGHT;
33746 |         case kAudioChannelLabel_XY_X:            return MA_CHANNEL_LEFT;
33747 |         case kAudioChannelLabel_XY_Y:            return MA_CHANNEL_RIGHT;
33748 |         case kAudioChannelLabel_HeadphonesLeft:   return MA_CHANNEL_LEFT;
33749 |         case kAudioChannelLabel_HeadphonesRight:  return MA_CHANNEL_RIGHT;
33750 |         case kAudioChannelLabel_ClickTrack:       return MA_CHANNEL_NONE;
33751 |         case kAudioChannelLabel_ForeignLanguage:  return MA_CHANNEL_NONE;
33752 |         case kAudioChannelLabel_Discrete:         return MA_CHANNEL_NONE;
33753 |         case kAudioChannelLabel_Discrete_0:       return MA_CHANNEL_AUX_0;

```



```

33754 |         case kAudioChannelLabel_Discrete_1:             return MA_CHANNEL_AUX_1;
33755 |         case kAudioChannelLabel_Discrete_2:             return MA_CHANNEL_AUX_2;
33756 |         case kAudioChannelLabel_Discrete_3:             return MA_CHANNEL_AUX_3;
33757 |         case kAudioChannelLabel_Discrete_4:             return MA_CHANNEL_AUX_4;
33758 |         case kAudioChannelLabel_Discrete_5:             return MA_CHANNEL_AUX_5;
33759 |         case kAudioChannelLabel_Discrete_6:             return MA_CHANNEL_AUX_6;
33760 |         case kAudioChannelLabel_Discrete_7:             return MA_CHANNEL_AUX_7;
33761 |         case kAudioChannelLabel_Discrete_8:             return MA_CHANNEL_AUX_8;
33762 |         case kAudioChannelLabel_Discrete_9:             return MA_CHANNEL_AUX_9;
33763 |         case kAudioChannelLabel_Discrete_10:            return MA_CHANNEL_AUX_10;
33764 |         case kAudioChannelLabel_Discrete_11:            return MA_CHANNEL_AUX_11;
33765 |         case kAudioChannelLabel_Discrete_12:            return MA_CHANNEL_AUX_12;
33766 |         case kAudioChannelLabel_Discrete_13:            return MA_CHANNEL_AUX_13;
33767 |         case kAudioChannelLabel_Discrete_14:            return MA_CHANNEL_AUX_14;
33768 |         case kAudioChannelLabel_Discrete_15:            return MA_CHANNEL_AUX_15;
33769 |         case kAudioChannelLabel_Discrete_65535:         return MA_CHANNEL_NONE;
33770 |
33771 |     #if 0 /* Introduced in a later version of macOS. */
33772 |         case kAudioChannelLabel_HOA_ACN:                return MA_CHANNEL_NONE;
33773 |         case kAudioChannelLabel_HOA_ACN_0:              return MA_CHANNEL_AUX_0;
33774 |         case kAudioChannelLabel_HOA_ACN_1:              return MA_CHANNEL_AUX_1;
33775 |         case kAudioChannelLabel_HOA_ACN_2:              return MA_CHANNEL_AUX_2;
33776 |         case kAudioChannelLabel_HOA_ACN_3:              return MA_CHANNEL_AUX_3;
33777 |         case kAudioChannelLabel_HOA_ACN_4:              return MA_CHANNEL_AUX_4;
33778 |         case kAudioChannelLabel_HOA_ACN_5:              return MA_CHANNEL_AUX_5;
33779 |         case kAudioChannelLabel_HOA_ACN_6:              return MA_CHANNEL_AUX_6;
33780 |         case kAudioChannelLabel_HOA_ACN_7:              return MA_CHANNEL_AUX_7;
33781 |         case kAudioChannelLabel_HOA_ACN_8:              return MA_CHANNEL_AUX_8;
33782 |         case kAudioChannelLabel_HOA_ACN_9:              return MA_CHANNEL_AUX_9;
33783 |         case kAudioChannelLabel_HOA_ACN_10:             return MA_CHANNEL_AUX_10;
33784 |         case kAudioChannelLabel_HOA_ACN_11:             return MA_CHANNEL_AUX_11;
33785 |         case kAudioChannelLabel_HOA_ACN_12:             return MA_CHANNEL_AUX_12;
33786 |         case kAudioChannelLabel_HOA_ACN_13:             return MA_CHANNEL_AUX_13;
33787 |         case kAudioChannelLabel_HOA_ACN_14:             return MA_CHANNEL_AUX_14;
33788 |         case kAudioChannelLabel_HOA_ACN_15:             return MA_CHANNEL_AUX_15;
33789 |         case kAudioChannelLabel_HOA_ACN_65024:          return MA_CHANNEL_NONE;
33790 |     #endif
33791 |
33792 |     default:                                             return MA_CHANNEL_NONE;
33793 | }
33794 | }
33795 |
33796 | static ma_result ma_get_channel_map_from_AudioChannelLayout(AudioChannelLayout* pChannelLayout,
> ma_channel* pChannelMap, size_t channelMapCap)
33797 | {
33798 |     MA_ASSERT(pChannelLayout != NULL);
33799 |
33800 |     if (pChannelLayout->mChannelLayoutTag == kAudioChannelLayoutTag_UseChannelDescriptions) {
33801 |         UInt32 iChannel;
33802 |         for (iChannel = 0; iChannel < pChannelLayout->mNumberChannelDescriptions && iChannel <
> channelMapCap; ++iChannel) {
33803 |             pChannelMap[iChannel] =
> ma_channel_from_AudioChannelLabel(pChannelLayout->mChannelDescriptions[iChannel].mChannelLabel);
33804 |         }
33805 |     } else
33806 |     #if 0
33807 |         if (pChannelLayout->mChannelLayoutTag == kAudioChannelLayoutTag_UseChannelBitmap) {
33808 |             /* This is the same kind of system that's used by Windows audio APIs. */
33809 |             UInt32 iChannel = 0;
33810 |             UInt32 iBit;
33811 |             AudioChannelBitmap bitmap = pChannelLayout->mChannelBitmap;
33812 |             for (iBit = 0; iBit < 32 && iChannel < channelMapCap; ++iBit) {
33813 |                 AudioChannelBitmap bit = bitmap & (1 << iBit);
33814 |                 if (bit != 0) {
33815 |                     pChannelMap[iChannel++] = ma_channel_from_AudioChannelBit(bit);
33816 |                 }
33817 |             }
33818 |         } else
33819 |     #endif
33820 |     {
33821 |         /*
33822 |         Need to use the tag to determine the channel map. For now I'm just assuming a default
> channel map, but later on this should
33823 |         be updated to determine the mapping based on the tag.
33824 |         */
33825 |         UInt32 channelCount;
33826 |
33827 |         /* Our channel map retrieval APIs below take 32-bit integers, so we'll want to clamp the
> channel map capacity. */

```



```

33828 |         if (channelMapCap > 0xFFFFFFFF) {
33829 |             channelMapCap = 0xFFFFFFFF;
33830 |         }
33831 |
33832 |         channelCount =
> ma_min(AudioChannelLayoutTag_GetNumberOfChannels(pChannelLayout->mChannelLayoutTag),
> (UInt32)channelMapCap);
33833 |
33834 |         switch (pChannelLayout->mChannelLayoutTag)
33835 |         {
33836 |             case kAudioChannelLayoutTag_Mono:
33837 |             case kAudioChannelLayoutTag_Stereo:
33838 |             case kAudioChannelLayoutTag_StereoHeadphones:
33839 |             case kAudioChannelLayoutTag_MatrixStereo:
33840 |             case kAudioChannelLayoutTag_MidSide:
33841 |             case kAudioChannelLayoutTag_XY:
33842 |             case kAudioChannelLayoutTag_Binaural:
33843 |             case kAudioChannelLayoutTag_Ambisonic_B_Format:
33844 |             {
33845 |                 ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap,
> channelMapCap, channelCount);
33846 |             } break;
33847 |
33848 |             case kAudioChannelLayoutTag_Octagonal:
33849 |             {
33850 |                 pChannelMap[7] = MA_CHANNEL_SIDE_RIGHT;
33851 |                 pChannelMap[6] = MA_CHANNEL_SIDE_LEFT;
33852 |             } MA_FALLTHROUGH; /* Intentional fallthrough. */
33853 |             case kAudioChannelLayoutTag_Hexagonal:
33854 |             {
33855 |                 pChannelMap[5] = MA_CHANNEL_BACK_CENTER;
33856 |             } MA_FALLTHROUGH; /* Intentional fallthrough. */
33857 |             case kAudioChannelLayoutTag_Pentagonal:
33858 |             {
33859 |                 pChannelMap[4] = MA_CHANNEL_FRONT_CENTER;
33860 |             } MA_FALLTHROUGH; /* Intentional fallthrough. */
33861 |             case kAudioChannelLayoutTag_Quadraphonic:
33862 |             {
33863 |                 pChannelMap[3] = MA_CHANNEL_BACK_RIGHT;
33864 |                 pChannelMap[2] = MA_CHANNEL_BACK_LEFT;
33865 |                 pChannelMap[1] = MA_CHANNEL_RIGHT;
33866 |                 pChannelMap[0] = MA_CHANNEL_LEFT;
33867 |             } break;
33868 |
33869 |             /* TODO: Add support for more tags here. */
33870 |
33871 |             default:
33872 |             {
33873 |                 ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap,
> channelMapCap, channelCount);
33874 |             } break;
33875 |         }
33876 |     }
33877 |
33878 |     return MA_SUCCESS;
33879 | }
33880 |
33881 | #if (defined(MAC_OS_VERSION_12_0) && MAC_OS_X_VERSION_MAX_ALLOWED >= MAC_OS_VERSION_12_0) || \
33882 |     (defined(__IPHONE_15_0) && __IPHONE_OS_VERSION_MAX_ALLOWED >= __IPHONE_15_0)
33883 | #define AUDIO_OBJECT_PROPERTY_ELEMENT kAudioObjectPropertyElementMain
33884 | #else
33885 | /* kAudioObjectPropertyElementMaster is deprecated. */
33886 | #define AUDIO_OBJECT_PROPERTY_ELEMENT kAudioObjectPropertyElementMaster
33887 | #endif
33888 |
33889 | /* kAudioDevicePropertyScope* were renamed to kAudioObjectPropertyScope* in 10.8. */
33890 | #if !defined(MAC_OS_X_VERSION_10_8) || (MAC_OS_X_VERSION_MIN_REQUIRED < MAC_OS_X_VERSION_10_8)
33891 | #define kAudioObjectPropertyScopeInput kAudioDevicePropertyScopeInput
33892 | #define kAudioObjectPropertyScopeOutput kAudioDevicePropertyScopeOutput
33893 | #endif
33894 |
33895 | static ma_result ma_get_device_object_ids_coreaudio(ma_context* pContext, UInt32* pDeviceCount,
> AudioObjectID** ppDeviceObjectIDs) /* NOTE: Free the returned buffer with ma_free(). */
33896 | {
33897 |     AudioObjectPropertyAddress propAddressDevices;
33898 |     UInt32 deviceObjectsDataSize;
33899 |     OSStatus status;
33900 |     AudioObjectID* pDeviceObjectIDs;
33901 |

```

```

33902 |     MA_ASSERT(pContext != NULL);
33903 |     MA_ASSERT(pDeviceCount != NULL);
33904 |     MA_ASSERT(ppDeviceObjectIDs != NULL);
33905 |
33906 |     /* Safety. */
33907 |     *pDeviceCount = 0;
33908 |     *ppDeviceObjectIDs = NULL;
33909 |
33910 |     propAddressDevices.mSelector = kAudioHardwarePropertyDevices;
33911 |     propAddressDevices.mScope    = kAudioObjectPropertyScopeGlobal;
33912 |     propAddressDevices.mElement  = AUDIO_OBJECT_PROPERTY_ELEMENT;
33913 |
33914 |     status = ((ma_AudioObjectGetPropertyDataSize_proc)pContext->coreaudio.AudioObjectGetPropertyData
> Size)(kAudioObjectSystemObject, &propAddressDevices, 0, NULL, &deviceObjectsDataSize);
33915 |     if (status != noErr) {
33916 |         return ma_result_from_OSStatus(status);
33917 |     }
33918 |
33919 |     pDeviceObjectIDs = (AudioObjectID*)ma_malloc(deviceObjectsDataSize,
> &pContext->allocationCallbacks);
33920 |     if (pDeviceObjectIDs == NULL) {
33921 |         return MA_OUT_OF_MEMORY;
33922 |     }
33923 |
33924 |     status = ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(kA
> udioObjectSystemObject, &propAddressDevices, 0, NULL, &deviceObjectsDataSize, pDeviceObjectIDs);
33925 |     if (status != noErr) {
33926 |         ma_free(pDeviceObjectIDs, &pContext->allocationCallbacks);
33927 |         return ma_result_from_OSStatus(status);
33928 |     }
33929 |
33930 |     *pDeviceCount = deviceObjectsDataSize / sizeof(AudioObjectID);
33931 |     *ppDeviceObjectIDs = pDeviceObjectIDs;
33932 |
33933 |     return MA_SUCCESS;
33934 | }
33935 |
33936 | static ma_result ma_get_AudioObject_uid_as_CFStringRef(ma_context* pContext, AudioObjectID objectID,
> CFStringRef* pUID)
33937 | {
33938 |     AudioObjectPropertyAddress propAddress;
33939 |     UInt32 dataSize;
33940 |     OSStatus status;
33941 |
33942 |     MA_ASSERT(pContext != NULL);
33943 |
33944 |     propAddress.mSelector = kAudioDevicePropertyDeviceUID;
33945 |     propAddress.mScope    = kAudioObjectPropertyScopeGlobal;
33946 |     propAddress.mElement  = AUDIO_OBJECT_PROPERTY_ELEMENT;
33947 |
33948 |     dataSize = sizeof(*pUID);
33949 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(objectID,
> &propAddress, 0, NULL, &dataSize, pUID);
33950 |     if (status != noErr) {
33951 |         return ma_result_from_OSStatus(status);
33952 |     }
33953 |
33954 |     return MA_SUCCESS;
33955 | }
33956 |
33957 | static ma_result ma_get_AudioObject_uid(ma_context* pContext, AudioObjectID objectID, size_t
> bufferSize, char* bufferOut)
33958 | {
33959 |     CFStringRef uid;
33960 |     ma_result result;
33961 |
33962 |     MA_ASSERT(pContext != NULL);
33963 |
33964 |     result = ma_get_AudioObject_uid_as_CFStringRef(pContext, objectID, &uid);
33965 |     if (result != MA_SUCCESS) {
33966 |         return result;
33967 |     }
33968 |
33969 |     if (!((ma_CFStringGetCString_proc)pContext->coreaudio.CFStringGetCString)(uid, bufferOut,
> bufferSize, kCFStringEncodingUTF8)) {
33970 |         return MA_ERROR;
33971 |     }
33972 | }

```

```

33973 | ((ma_CFRelease_proc)pContext->coreaudio.CFRelease)(uid);
33974 | return MA_SUCCESS;
33975 | }
33976 |
33977 | static ma_result ma_get_AudioObject_name(ma_context* pContext, AudioObjectID objectID, size_t
> bufferSize, char* bufferOut)
33978 | {
33979 |     AudioObjectPropertyAddress propAddress;
33980 |     CFStringRef deviceName = NULL;
33981 |     UInt32 dataSize;
33982 |     OSStatus status;
33983 |
33984 |     MA_ASSERT(pContext != NULL);
33985 |
33986 |     propAddress.mSelector = kAudioDevicePropertyDeviceNameCFString;
33987 |     propAddress.mScope    = kAudioObjectPropertyScopeGlobal;
33988 |     propAddress.mElement  = AUDIO_OBJECT_PROPERTY_ELEMENT;
33989 |
33990 |     dataSize = sizeof(deviceName);
33991 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(objectID,
> &propAddress, 0, NULL, &dataSize, &deviceName);
33992 |     if (status != noErr) {
33993 |         return ma_result_from_OSStatus(status);
33994 |     }
33995 |
33996 |     if (!((ma_CFStringGetCString_proc)pContext->coreaudio.CFStringGetCString)(deviceName, bufferOut,
> bufferSize, kCFStringEncodingUTF8)) {
33997 |         return MA_ERROR;
33998 |     }
33999 |
34000 |     ((ma_CFRelease_proc)pContext->coreaudio.CFRelease)(deviceName);
34001 |     return MA_SUCCESS;
34002 | }
34003 |
34004 | static ma_bool32 ma_does_AudioObject_support_scope(ma_context* pContext, AudioObjectID
> deviceObjectID, AudioObjectPropertyScope scope)
34005 | {
34006 |     AudioObjectPropertyAddress propAddress;
34007 |     UInt32 dataSize;
34008 |     OSStatus status;
34009 |     AudioBufferList* pBufferList;
34010 |     ma_bool32 isSupported;
34011 |
34012 |     MA_ASSERT(pContext != NULL);
34013 |
34014 |     /* To know whether or not a device is an input device we need ot look at the stream
> configuration. If it has an output channel it's a playback device. */
34015 |     propAddress.mSelector = kAudioDevicePropertyStreamConfiguration;
34016 |     propAddress.mScope    = scope;
34017 |     propAddress.mElement  = AUDIO_OBJECT_PROPERTY_ELEMENT;
34018 |
34019 |     status = ((ma_AudioObjectGetPropertyDataSize_proc)pContext->coreaudio.AudioObjectGetPropertyData
> Size)(deviceObjectID, &propAddress, 0, NULL, &dataSize);
34020 |     if (status != noErr) {
34021 |         return MA_FALSE;
34022 |     }
34023 |
34024 |     pBufferList = (AudioBufferList*)ma_malloc(dataSize, &pContext->allocationCallbacks);
34025 |     if (pBufferList == NULL) {
34026 |         return MA_FALSE; /* Out of memory. */
34027 |     }
34028 |
34029 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, &dataSize, pBufferList);
34030 |     if (status != noErr) {
34031 |         ma_free(pBufferList, &pContext->allocationCallbacks);
34032 |         return MA_FALSE;
34033 |     }
34034 |
34035 |     isSupported = MA_FALSE;
34036 |     if (pBufferList->mNumberBuffers > 0) {
34037 |         isSupported = MA_TRUE;
34038 |     }
34039 |
34040 |     ma_free(pBufferList, &pContext->allocationCallbacks);
34041 |     return isSupported;
34042 | }

```

```

34043 |
34044 | static ma_bool32 ma_does_AudioObject_support_playback(ma_context* pContext, AudioObjectID
> deviceObjectID)
34045 | {
34046 |     return ma_does_AudioObject_support_scope(pContext, deviceObjectID,
> kAudioObjectPropertyScopeOutput);
34047 | }
34048 |
34049 | static ma_bool32 ma_does_AudioObject_support_capture(ma_context* pContext, AudioObjectID
> deviceObjectID)
34050 | {
34051 |     return ma_does_AudioObject_support_scope(pContext, deviceObjectID,
> kAudioObjectPropertyScopeInput);
34052 | }
34053 |
34054 |
34055 | static ma_result ma_get_AudioObject_stream_descriptions(ma_context* pContext, AudioObjectID
> deviceObjectID, ma_device_type deviceType, UInt32* pDescriptionCount, AudioStreamRangedDescription**
> ppDescriptions) /* NOTE: Free the returned pointer with ma_free(). */
34056 | {
34057 |     AudioObjectPropertyAddress propAddress;
34058 |     UInt32 dataSize;
34059 |     OSStatus status;
34060 |     AudioStreamRangedDescription* pDescriptions;
34061 |
34062 |     MA_ASSERT(pContext != NULL);
34063 |     MA_ASSERT(pDescriptionCount != NULL);
34064 |     MA_ASSERT(ppDescriptions != NULL);
34065 |
34066 |     /*
34067 |     > TODO: Experiment with kAudioStreamPropertyAvailablePhysicalFormats instead of (or in addition
to) kAudioStreamPropertyAvailableVirtualFormats. My
34068 |     > MacBook Pro uses s24/32 format, however, which miniaudio does not currently support.
34069 |     */
34070 |     propAddress.mSelector = kAudioStreamPropertyAvailableVirtualFormats;
34071 |     > /*kAudioStreamPropertyAvailablePhysicalFormats;*/
34072 |     propAddress.mScope = (deviceType == ma_device_type_playback) ?
> kAudioObjectPropertyScopeOutput : kAudioObjectPropertyScopeInput;
34073 |     propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
34074 |
34075 |     status = ((ma_AudioObjectGetPropertyDataSize_proc)pContext->coreaudio.AudioObjectGetPropertyData
> Size)(deviceObjectID, &propAddress, 0, NULL, &dataSize);
34076 |     if (status != noErr) {
34077 |         return ma_result_from_OSStatus(status);
34078 |     }
34079 |
34080 |     pDescriptions = (AudioStreamRangedDescription*)ma_malloc(dataSize,
> &pContext->allocationCallbacks);
34081 |     if (pDescriptions == NULL) {
34082 |         return MA_OUT_OF_MEMORY;
34083 |     }
34084 |
34085 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, &dataSize, pDescriptions);
34086 |     if (status != noErr) {
34087 |         ma_free(pDescriptions, &pContext->allocationCallbacks);
34088 |         return ma_result_from_OSStatus(status);
34089 |     }
34090 |
34091 |     *pDescriptionCount = dataSize / sizeof(*pDescriptions);
34092 |     *ppDescriptions = pDescriptions;
34093 |     return MA_SUCCESS;
34094 | }
34095 |
34096 | static ma_result ma_get_AudioObject_channel_layout(ma_context* pContext, AudioObjectID
> deviceObjectID, ma_device_type deviceType, AudioChannelLayout** ppChannelLayout) /* NOTE: Free the
> returned pointer with ma_free(). */
34097 | {
34098 |     AudioObjectPropertyAddress propAddress;
34099 |     UInt32 dataSize;
34100 |     OSStatus status;
34101 |     AudioChannelLayout* pChannelLayout;
34102 |
34103 |     MA_ASSERT(pContext != NULL);
34104 |     MA_ASSERT(ppChannelLayout != NULL);
34105 |
34106 |     *ppChannelLayout = NULL; /* Safety. */

```

```

34107 |
34108 |     propAddress.mSelector = kAudioDevicePropertyPreferredChannelLayout;
34109 |     propAddress.mScope   = (deviceType == ma_device_type_playback) ?
> kAudioObjectPropertyScopeOutput : kAudioObjectPropertyScopeInput;
34110 |     propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
34111 |
34112 |     status = ((ma_AudioObjectGetPropertyDataSize_proc)pContext->coreaudio.AudioObjectGetPropertyData
> Size)(deviceObjectID, &propAddress, 0, NULL, &dataSize);
34113 |     if (status != noErr) {
34114 |         return ma_result_from_OSStatus(status);
34115 |     }
34116 |
34117 |     pChannelLayout = (AudioChannelLayout*)ma_malloc(dataSize, &pContext->allocationCallbacks);
34118 |     if (pChannelLayout == NULL) {
34119 |         return MA_OUT_OF_MEMORY;
34120 |     }
34121 |
34122 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, &dataSize, pChannelLayout);
34123 |     if (status != noErr) {
34124 |         ma_free(pChannelLayout, &pContext->allocationCallbacks);
34125 |         return ma_result_from_OSStatus(status);
34126 |     }
34127 |
34128 |     *ppChannelLayout = pChannelLayout;
34129 |     return MA_SUCCESS;
34130 | }
34131 |
34132 | static ma_result ma_get_AudioObject_channel_count(ma_context* pContext, AudioObjectID
> deviceObjectID, ma_device_type deviceType, ma_uint32* pChannelCount)
34133 | {
34134 |     AudioChannelLayout* pChannelLayout;
34135 |     ma_result result;
34136 |
34137 |     MA_ASSERT(pContext != NULL);
34138 |     MA_ASSERT(pChannelCount != NULL);
34139 |
34140 |     *pChannelCount = 0; /* Safety. */
34141 |
34142 |     result = ma_get_AudioObject_channel_layout(pContext, deviceObjectID, deviceType,
> &pChannelLayout);
34143 |     if (result != MA_SUCCESS) {
34144 |         return result;
34145 |     }
34146 |
34147 |     if (pChannelLayout->mChannelLayoutTag == kAudioChannelLayoutTag_UseChannelDescriptions) {
34148 |         *pChannelCount = pChannelLayout->mNumberChannelDescriptions;
34149 |     } else if (pChannelLayout->mChannelLayoutTag == kAudioChannelLayoutTag_UseChannelBitmap) {
34150 |         *pChannelCount = ma_count_set_bits(pChannelLayout->mChannelBitmap);
34151 |     } else {
34152 |         *pChannelCount =
> AudioChannelLayoutTag_GetNumberOfChannels(pChannelLayout->mChannelLayoutTag);
34153 |     }
34154 |
34155 |     ma_free(pChannelLayout, &pContext->allocationCallbacks);
34156 |     return MA_SUCCESS;
34157 | }
34158 |
34159 | #if 0
34160 | static ma_result ma_get_AudioObject_channel_map(ma_context* pContext, AudioObjectID deviceObjectID,
> ma_device_type deviceType, ma_channel* pChannelMap, size_t channelMapCap)
34161 | {
34162 |     AudioChannelLayout* pChannelLayout;
34163 |     ma_result result;
34164 |
34165 |     MA_ASSERT(pContext != NULL);
34166 |
34167 |     result = ma_get_AudioObject_channel_layout(pContext, deviceObjectID, deviceType,
> &pChannelLayout);
34168 |     if (result != MA_SUCCESS) {
34169 |         return result; /* Rather than always failing here, would it be more robust to simply assume
> a default? */
34170 |     }
34171 |
34172 |     result = ma_get_channel_map_from_AudioChannelLayout(pChannelLayout, pChannelMap, channelMapCap);
34173 |     if (result != MA_SUCCESS) {
34174 |         ma_free(pChannelLayout, &pContext->allocationCallbacks);
34175 |         return result;

```

```

34176 |     }
34177 |
34178 |     ma_free(pChannelLayout, &pContext->allocationCallbacks);
34179 |     return result;
34180 | }
34181 | #endif
34182 |
34183 | static ma_result ma_get_AudioObject_sample_rates(ma_context* pContext, AudioObjectID deviceObjectID,
> ma_device_type deviceType, UInt32* pSampleRateRangesCount, AudioValueRange** ppSampleRateRanges)
> /* NOTE: Free the returned pointer with ma_free(). */
34184 | {
34185 |     AudioObjectPropertyAddress propAddress;
34186 |     UInt32 dataSize;
34187 |     OSStatus status;
34188 |     AudioValueRange* pSampleRateRanges;
34189 |
34190 |     MA_ASSERT(pContext != NULL);
34191 |     MA_ASSERT(pSampleRateRangesCount != NULL);
34192 |     MA_ASSERT(ppSampleRateRanges != NULL);
34193 |
34194 |     /* Safety. */
34195 |     *pSampleRateRangesCount = 0;
34196 |     *ppSampleRateRanges = NULL;
34197 |
34198 |     propAddress.mSelector = kAudioDevicePropertyAvailableNominalSampleRates;
34199 |     propAddress.mScope = (deviceType == ma_device_type_playback) ?
> kAudioObjectPropertyScopeOutput : kAudioObjectPropertyScopeInput;
34200 |     propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
34201 |
34202 |     status = ((ma_AudioObjectGetPropertyDataSize_proc)pContext->coreaudio.AudioObjectGetPropertyData
> Size)(deviceObjectID, &propAddress, 0, NULL, &dataSize);
34203 |     if (status != noErr) {
34204 |         return ma_result_from_OSStatus(status);
34205 |     }
34206 |
34207 |     pSampleRateRanges = (AudioValueRange*)ma_malloc(dataSize, &pContext->allocationCallbacks);
34208 |     if (pSampleRateRanges == NULL) {
34209 |         return MA_OUT_OF_MEMORY;
34210 |     }
34211 |
34212 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, &dataSize, pSampleRateRanges);
34213 |     if (status != noErr) {
34214 |         ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34215 |         return ma_result_from_OSStatus(status);
34216 |     }
34217 |
34218 |     *pSampleRateRangesCount = dataSize / sizeof(*pSampleRateRanges);
34219 |     *ppSampleRateRanges = pSampleRateRanges;
34220 |     return MA_SUCCESS;
34221 | }
34222 |
34223 | #if 0
34224 | static ma_result ma_get_AudioObject_get_closest_sample_rate(ma_context* pContext, AudioObjectID
> deviceObjectID, ma_device_type deviceType, ma_uint32 sampleRateIn, ma_uint32* pSampleRateOut)
34225 | {
34226 |     UInt32 sampleRateRangeCount;
34227 |     AudioValueRange* pSampleRateRanges;
34228 |     ma_result result;
34229 |
34230 |     MA_ASSERT(pContext != NULL);
34231 |     MA_ASSERT(pSampleRateOut != NULL);
34232 |
34233 |     *pSampleRateOut = 0;    /* Safety. */
34234 |
34235 |     result = ma_get_AudioObject_sample_rates(pContext, deviceObjectID, deviceType,
> &sampleRateRangeCount, &pSampleRateRanges);
34236 |     if (result != MA_SUCCESS) {
34237 |         return result;
34238 |     }
34239 |
34240 |     if (sampleRateRangeCount == 0) {
34241 |         ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34242 |         return MA_ERROR;    /* Should never hit this case should we? */
34243 |     }
34244 |
34245 |     if (sampleRateIn == 0) {
34246 |         /* Search in order of miniaudio's preferred priority. */

```

```

34247 |         UInt32 iMALSampleRate;
34248 |         for (iMALSampleRate = 0; iMALSampleRate < ma_countof(g_maStandardSampleRatePriorities);
> ++iMALSampleRate) {
34249 |             ma_uint32 malSampleRate = g_maStandardSampleRatePriorities[iMALSampleRate];
34250 |             UInt32 iCASampleRate;
34251 |             for (iCASampleRate = 0; iCASampleRate < sampleRateRangeCount; ++iCASampleRate) {
34252 |                 AudioValueRange caSampleRate = pSampleRateRanges[iCASampleRate];
34253 |                 if (caSampleRate.mMinimum <= malSampleRate && caSampleRate.mMaximum >=
> malSampleRate) {
34254 |                     *pSampleRateOut = malSampleRate;
34255 |                     ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34256 |                     return MA_SUCCESS;
34257 |                 }
34258 |             }
34259 |         }
34260 |
34261 |         /*
34262 |         If we get here it means none of miniaudio's standard sample rates matched any of the
> supported sample rates from the device. In this
34263 |         case we just fall back to the first one reported by Core Audio.
34264 |         */
34265 |         MA_ASSERT(sampleRateRangeCount > 0);
34266 |
34267 |         *pSampleRateOut = pSampleRateRanges[0].mMinimum;
34268 |         ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34269 |         return MA_SUCCESS;
34270 |     } else {
34271 |         /* Find the closest match to this sample rate. */
34272 |         UInt32 currentAbsoluteDifference = INT32_MAX;
34273 |         UInt32 iCurrentClosestRange = (UInt32)-1;
34274 |         UInt32 iRange;
34275 |         for (iRange = 0; iRange < sampleRateRangeCount; ++iRange) {
34276 |             if (pSampleRateRanges[iRange].mMinimum <= sampleRateIn &&
> pSampleRateRanges[iRange].mMaximum >= sampleRateIn) {
34277 |                 *pSampleRateOut = sampleRateIn;
34278 |                 ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34279 |                 return MA_SUCCESS;
34280 |             } else {
34281 |                 UInt32 absoluteDifference;
34282 |                 if (pSampleRateRanges[iRange].mMinimum > sampleRateIn) {
34283 |                     absoluteDifference = pSampleRateRanges[iRange].mMinimum - sampleRateIn;
34284 |                 } else {
34285 |                     absoluteDifference = sampleRateIn - pSampleRateRanges[iRange].mMaximum;
34286 |                 }
34287 |
34288 |                 if (currentAbsoluteDifference > absoluteDifference) {
34289 |                     currentAbsoluteDifference = absoluteDifference;
34290 |                     iCurrentClosestRange = iRange;
34291 |                 }
34292 |             }
34293 |         }
34294 |
34295 |         MA_ASSERT(iCurrentClosestRange != (UInt32)-1);
34296 |
34297 |         *pSampleRateOut = pSampleRateRanges[iCurrentClosestRange].mMinimum;
34298 |         ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34299 |         return MA_SUCCESS;
34300 |     }
34301 |
34302 |     /* Should never get here, but it would mean we weren't able to find any suitable sample rates.
> */
34303 |     /*ma_free(pSampleRateRanges, &pContext->allocationCallbacks);*/
34304 |     /*return MA_ERROR;*/
34305 | }
34306 | #endif
34307 |
34308 | static ma_result ma_get_AudioObject_closest_buffer_size_in_frames(ma_context* pContext,
> AudioObjectID deviceObjectID, ma_device_type deviceType, ma_uint32 bufferSizeInFramesIn, ma_uint32*
> pBufferSizeInFramesOut)
34309 | {
34310 |     AudioObjectPropertyAddress propAddress;
34311 |     AudioValueRange bufferSizeRange;
34312 |     UInt32 dataSize;
34313 |     OSStatus status;
34314 |
34315 |     MA_ASSERT(pContext != NULL);
34316 |     MA_ASSERT(pBufferSizeInFramesOut != NULL);
34317 |
34318 |     *pBufferSizeInFramesOut = 0;    /* Safety. */

```



```

34319 |
34320 |     propAddress.mSelector = kAudioDevicePropertyBufferFrameSizeRange;
34321 |     propAddress.mScope   = (deviceType == ma_device_type_playback) ?
> kAudioObjectPropertyScopeOutput : kAudioObjectPropertyScopeInput;
34322 |     propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
34323 |
34324 |     dataSize = sizeof(bufferSizeRange);
34325 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, &dataSize, &bufferSizeRange);
34326 |     if (status != noErr) {
34327 |         return ma_result_from_OSStatus(status);
34328 |     }
34329 |
34330 |     /* This is just a clamp. */
34331 |     if (bufferSizeInFramesIn < bufferSizeRange.mMinimum) {
34332 |         *pBufferSizeInFramesOut = (ma_uint32)bufferSizeRange.mMinimum;
34333 |     } else if (bufferSizeInFramesIn > bufferSizeRange.mMaximum) {
34334 |         *pBufferSizeInFramesOut = (ma_uint32)bufferSizeRange.mMaximum;
34335 |     } else {
34336 |         *pBufferSizeInFramesOut = bufferSizeInFramesIn;
34337 |     }
34338 |
34339 |     return MA_SUCCESS;
34340 | }
34341 |
34342 | static ma_result ma_set_AudioObject_buffer_size_in_frames(ma_context* pContext, AudioObjectID
> deviceObjectID, ma_device_type deviceType, ma_uint32* pPeriodSizeInOut)
34343 | {
34344 |     ma_result result;
34345 |     ma_uint32 chosenBufferSizeInFrames;
34346 |     AudioObjectPropertyAddress propAddress;
34347 |     UInt32 dataSize;
34348 |     OSStatus status;
34349 |
34350 |     MA_ASSERT(pContext != NULL);
34351 |
34352 |     result = ma_get_AudioObject_closest_buffer_size_in_frames(pContext, deviceObjectID, deviceType,
> *pPeriodSizeInOut, &chosenBufferSizeInFrames);
34353 |     if (result != MA_SUCCESS) {
34354 |         return result;
34355 |     }
34356 |
34357 |     /* Try setting the size of the buffer... If this fails we just use whatever is currently set. */
34358 |     propAddress.mSelector = kAudioDevicePropertyBufferFrameSize;
34359 |     propAddress.mScope   = (deviceType == ma_device_type_playback) ?
> kAudioObjectPropertyScopeOutput : kAudioObjectPropertyScopeInput;
34360 |     propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
34361 |
34362 |
> ((ma_AudioObjectSetPropertyData_proc)pContext->coreaudio.AudioObjectSetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, sizeof(chosenBufferSizeInFrames), &chosenBufferSizeInFrames);
34363 |
34364 |     /* Get the actual size of the buffer. */
34365 |     dataSize = sizeof(*pPeriodSizeInOut);
34366 |     status =
> ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, &dataSize, &chosenBufferSizeInFrames);
34367 |     if (status != noErr) {
34368 |         return ma_result_from_OSStatus(status);
34369 |     }
34370 |
34371 |     *pPeriodSizeInOut = chosenBufferSizeInFrames;
34372 |     return MA_SUCCESS;
34373 | }
34374 |
34375 | static ma_result ma_find_default_AudioObjectID(ma_context* pContext, ma_device_type deviceType,
> AudioObjectID* pDeviceObjectID)
34376 | {
34377 |     AudioObjectPropertyAddress propAddressDefaultDevice;
34378 |     UInt32 defaultDeviceObjectIDSize = sizeof(AudioObjectID);
34379 |     AudioObjectID defaultDeviceObjectID;
34380 |     OSStatus status;
34381 |
34382 |     MA_ASSERT(pContext != NULL);
34383 |     MA_ASSERT(pDeviceObjectID != NULL);
34384 |
34385 |     /* Safety. */
34386 |     *pDeviceObjectID = 0;

```

```

34387 |
34388 |     propAddressDefaultDevice.mScope = kAudioObjectPropertyScopeGlobal;
34389 |     propAddressDefaultDevice.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
34390 |     if (deviceType == ma_device_type_playback) {
34391 |         propAddressDefaultDevice.mSelector = kAudioHardwarePropertyDefaultOutputDevice;
34392 |     } else {
34393 |         propAddressDefaultDevice.mSelector = kAudioHardwarePropertyDefaultInputDevice;
34394 |     }
34395 |
34396 |     defaultDeviceObjectIDSize = sizeof(AudioObjectID);
34397 |     status = ((ma_AudioObjectGetPropertyData_proc)pContext->coreaudio.AudioObjectGetPropertyData)(kA
> udioObjectSystemObject, &propAddressDefaultDevice, 0, NULL, &defaultDeviceObjectIDSize,
> &defaultDeviceObjectID);
34398 |     if (status == noErr) {
34399 |         *pDeviceObjectID = defaultDeviceObjectID;
34400 |         return MA_SUCCESS;
34401 |     }
34402 |
34403 |     /* If we get here it means we couldn't find the device. */
34404 |     return MA_NO_DEVICE;
34405 | }
34406 |
34407 | static ma_result ma_find_AudioObjectID(ma_context* pContext, ma_device_type deviceType, const
> ma_device_id* pDeviceID, AudioObjectID* pDeviceObjectID)
34408 | {
34409 |     MA_ASSERT(pContext != NULL);
34410 |     MA_ASSERT(pDeviceObjectID != NULL);
34411 |
34412 |     /* Safety. */
34413 |     *pDeviceObjectID = 0;
34414 |
34415 |     if (pDeviceID == NULL) {
34416 |         /* Default device. */
34417 |         return ma_find_default_AudioObjectID(pContext, deviceType, pDeviceObjectID);
34418 |     } else {
34419 |         /* Explicit device. */
34420 |         UInt32 deviceCount;
34421 |         AudioObjectID* pDeviceObjectIDs;
34422 |         ma_result result;
34423 |         UInt32 iDevice;
34424 |
34425 |         result = ma_get_device_object_ids__coreaudio(pContext, &deviceCount, &pDeviceObjectIDs);
34426 |         if (result != MA_SUCCESS) {
34427 |             return result;
34428 |         }
34429 |
34430 |         for (iDevice = 0; iDevice < deviceCount; ++iDevice) {
34431 |             AudioObjectID deviceObjectID = pDeviceObjectIDs[iDevice];
34432 |
34433 |             char uid[256];
34434 |             if (ma_get_AudioObject_uid(pContext, deviceObjectID, sizeof(uid), uid) != MA_SUCCESS) {
34435 |                 continue;
34436 |             }
34437 |
34438 |             if (deviceType == ma_device_type_playback) {
34439 |                 if (ma_does_AudioObject_support_playback(pContext, deviceObjectID)) {
34440 |                     if (strcmp(uid, pDeviceID->coreaudio) == 0) {
34441 |                         *pDeviceObjectID = deviceObjectID;
34442 |                         ma_free(pDeviceObjectIDs, &pContext->allocationCallbacks);
34443 |                         return MA_SUCCESS;
34444 |                     }
34445 |                 }
34446 |             } else {
34447 |                 if (ma_does_AudioObject_support_capture(pContext, deviceObjectID)) {
34448 |                     if (strcmp(uid, pDeviceID->coreaudio) == 0) {
34449 |                         *pDeviceObjectID = deviceObjectID;
34450 |                         ma_free(pDeviceObjectIDs, &pContext->allocationCallbacks);
34451 |                         return MA_SUCCESS;
34452 |                     }
34453 |                 }
34454 |             }
34455 |         }
34456 |
34457 |         ma_free(pDeviceObjectIDs, &pContext->allocationCallbacks);
34458 |     }
34459 |
34460 |     /* If we get here it means we couldn't find the device. */
34461 |     return MA_NO_DEVICE;
34462 | }

```

```

34463 |
34464 |
34465 | static ma_result ma_find_best_format_coreaudio(ma_context* pContext, AudioObjectID deviceObjectID,
> ma_device_type deviceType, ma_format format, ma_uint32 channels, ma_uint32 sampleRate, const
> AudioStreamBasicDescription* pOrigFormat, AudioStreamBasicDescription* pFormat)
34466 | {
34467 |     UInt32 deviceFormatDescriptionCount;
34468 |     AudioStreamRangedDescription* pDeviceFormatDescriptions;
34469 |     ma_result result;
34470 |     ma_uint32 desiredSampleRate;
34471 |     ma_uint32 desiredChannelCount;
34472 |     ma_format desiredFormat;
34473 |     AudioStreamBasicDescription bestDeviceFormatSoFar;
34474 |     ma_bool32 hasSupportedFormat;
34475 |     UInt32 iFormat;
34476 |
34477 |     result = ma_get_AudioObject_stream_descriptions(pContext, deviceObjectID, deviceType,
> &deviceFormatDescriptionCount, &pDeviceFormatDescriptions);
34478 |     if (result != MA_SUCCESS) {
34479 |         return result;
34480 |     }
34481 |
34482 |     desiredSampleRate = sampleRate;
34483 |     if (desiredSampleRate == 0) {
34484 |         desiredSampleRate = (ma_uint32)pOrigFormat->mSampleRate;
34485 |     }
34486 |
34487 |     desiredChannelCount = channels;
34488 |     if (desiredChannelCount == 0) {
34489 |         desiredChannelCount = pOrigFormat->mChannelsPerFrame;
34490 |     }
34491 |
34492 |     desiredFormat = format;
34493 |     if (desiredFormat == ma_format_unknown) {
34494 |         result = ma_format_from_AudioStreamBasicDescription(pOrigFormat, &desiredFormat);
34495 |         if (result != MA_SUCCESS || desiredFormat == ma_format_unknown) {
34496 |             desiredFormat = g_maFormatPriorities[0];
34497 |         }
34498 |     }
34499 |
34500 |     /*
34501 |     If we get here it means we don't have an exact match to what the client is asking for. We'll
> need to find the closest one. The next
34502 |     loop will check for formats that have the same sample rate to what we're asking for. If there
> is, we prefer that one in all cases.
34503 |     */
34504 |     MA_ZERO_OBJECT(&bestDeviceFormatSoFar);
34505 |
34506 |     hasSupportedFormat = MA_FALSE;
34507 |     for (iFormat = 0; iFormat < deviceFormatDescriptionCount; ++iFormat) {
34508 |         ma_format formatFromDescription;
34509 |         ma_result formatResult =
> ma_format_from_AudioStreamBasicDescription(&pDeviceFormatDescriptions[iFormat].mFormat,
> &formatFromDescription);
34510 |         if (formatResult == MA_SUCCESS && formatFromDescription != ma_format_unknown) {
34511 |             hasSupportedFormat = MA_TRUE;
34512 |             bestDeviceFormatSoFar = pDeviceFormatDescriptions[iFormat].mFormat;
34513 |             break;
34514 |         }
34515 |     }
34516 |
34517 |     if (!hasSupportedFormat) {
34518 |         ma_free(pDeviceFormatDescriptions, &pContext->allocationCallbacks);
34519 |         return MA_FORMAT_NOT_SUPPORTED;
34520 |     }
34521 |
34522 |
34523 |     for (iFormat = 0; iFormat < deviceFormatDescriptionCount; ++iFormat) {
34524 |         AudioStreamBasicDescription thisDeviceFormat = pDeviceFormatDescriptions[iFormat].mFormat;
34525 |         ma_format thisSampleFormat;
34526 |         ma_result formatResult;
34527 |         ma_format bestSampleFormatSoFar;
34528 |
34529 |         /* If the format is not supported by miniaudio we need to skip this one entirely. */
34530 |         formatResult =
> ma_format_from_AudioStreamBasicDescription(&pDeviceFormatDescriptions[iFormat].mFormat,
> &thisSampleFormat);
34531 |         if (formatResult != MA_SUCCESS || thisSampleFormat == ma_format_unknown) {
34532 |             continue; /* The format is not supported by miniaudio. Skip. */

```

```

34533 |     }
34534 |
34535 |     ma_format_from_AudioStreamBasicDescription(&bestDeviceFormatSoFar, &bestSampleFormatSoFar);
34536 |
34537 |     /* Getting here means the format is supported by miniaudio which makes this format a
> candidate. */
34538 |     if (thisDeviceFormat.mSampleRate != desiredSampleRate) {
34539 |         /*
34540 |         The sample rate does not match, but this format could still be usable, although it's a
> very low priority. If the best format
34541 |         so far has an equal sample rate we can just ignore this one.
34542 |         */
34543 |         if (bestDeviceFormatSoFar.mSampleRate == desiredSampleRate) {
34544 |             continue; /* The best sample rate so far has the same sample rate as what we
> requested which means it's still the best so far. Skip this format. */
34545 |         } else {
34546 |             /* In this case, neither the best format so far nor this one have the same sample
> rate. Check the channel count next. */
34547 |             if (thisDeviceFormat.mChannelsPerFrame != desiredChannelCount) {
34548 |                 /* This format has a different sample rate _and_ a different channel count. */
34549 |                 if (bestDeviceFormatSoFar.mChannelsPerFrame == desiredChannelCount) {
34550 |                     continue; /* No change to the best format. */
34551 |                 } else {
34552 |                     /*
34553 |                     Both this format and the best so far have different sample rates and
> different channel counts. Whichever has the
34554 |                     best format is the new best.
34555 |                     */
34556 |                     if (ma_get_format_priority_index(thisSampleFormat) <
> ma_get_format_priority_index(bestSampleFormatSoFar)) {
34557 |                         bestDeviceFormatSoFar = thisDeviceFormat;
34558 |                         continue;
34559 |                     } else {
34560 |                         continue; /* No change to the best format. */
34561 |                     }
34562 |                 }
34563 |             } else {
34564 |                 /* This format has a different sample rate but the desired channel count. */
34565 |                 if (bestDeviceFormatSoFar.mChannelsPerFrame == desiredChannelCount) {
34566 |                     /* Both this format and the best so far have the desired channel count.
> Whichever has the best format is the new best. */
34567 |                     if (ma_get_format_priority_index(thisSampleFormat) <
> ma_get_format_priority_index(bestSampleFormatSoFar)) {
34568 |                         bestDeviceFormatSoFar = thisDeviceFormat;
34569 |                         continue;
34570 |                     } else {
34571 |                         continue; /* No change to the best format for now. */
34572 |                     }
34573 |                 } else {
34574 |                     /* This format has the desired channel count, but the best so far does not.
> We have a new best. */
34575 |                     bestDeviceFormatSoFar = thisDeviceFormat;
34576 |                     continue;
34577 |                 }
34578 |             }
34579 |         }
34580 |     } else {
34581 |         /*
34582 |         The sample rates match which makes this format a very high priority contender. If the
> best format so far has a different
34583 |         sample rate it needs to be replaced with this one.
34584 |         */
34585 |         if (bestDeviceFormatSoFar.mSampleRate != desiredSampleRate) {
34586 |             bestDeviceFormatSoFar = thisDeviceFormat;
34587 |             continue;
34588 |         } else {
34589 |             /* In this case both this format and the best format so far have the same sample
> rate. Check the channel count next. */
34590 |             if (thisDeviceFormat.mChannelsPerFrame == desiredChannelCount) {
34591 |                 /*
34592 |                 In this case this format has the same channel count as what the client is
> requesting. If the best format so far has
34593 |                 a different count, this one becomes the new best.
34594 |                 */
34595 |                 if (bestDeviceFormatSoFar.mChannelsPerFrame != desiredChannelCount) {
34596 |                     bestDeviceFormatSoFar = thisDeviceFormat;
34597 |                     continue;
34598 |                 } else {
34599 |                     /* In this case both this format and the best so far have the ideal sample

```

```

> rate and channel count. Check the format. */
34600 |         if (thisSampleFormat == desiredFormat) {
34601 |             bestDeviceFormatSoFar = thisDeviceFormat;
34602 |             break; /* Found the exact match. */
34603 |         } else {
34604 |             /* The formats are different. The new best format is the one with the
> highest priority format according to miniaudio. */
34605 |             if (ma_get_format_priority_index(thisSampleFormat) <
> ma_get_format_priority_index(bestSampleFormatSoFar)) {
34606 |                 bestDeviceFormatSoFar = thisDeviceFormat;
34607 |                 continue;
34608 |             } else {
34609 |                 continue; /* No change to the best format for now. */
34610 |             }
34611 |         }
34612 |     }
34613 | } else {
34614 |     /*
34615 |     In this case the channel count is different to what the client has requested. If
> the best so far has the same channel
34616 |     count as the requested count then it remains the best.
34617 |     */
34618 |     if (bestDeviceFormatSoFar.mChannelsPerFrame == desiredChannelCount) {
34619 |         continue;
34620 |     } else {
34621 |         /*
34622 |         This is the case where both have the same sample rate (good) but different
> channel counts. Right now both have about
34623 |         the same priority, but we need to compare the format now.
34624 |         */
34625 |         if (thisSampleFormat == bestSampleFormatSoFar) {
34626 |             if (ma_get_format_priority_index(thisSampleFormat) <
> ma_get_format_priority_index(bestSampleFormatSoFar)) {
34627 |                 bestDeviceFormatSoFar = thisDeviceFormat;
34628 |                 continue;
34629 |             } else {
34630 |                 continue; /* No change to the best format for now. */
34631 |             }
34632 |         }
34633 |     }
34634 | }
34635 | }
34636 | }
34637 | }
34638 |
34639 | *pFormat = bestDeviceFormatSoFar;
34640 |
34641 | ma_free(pDeviceFormatDescriptions, &pContext->allocationCallbacks);
34642 | return MA_SUCCESS;
34643 | }
34644 |
34645 | static ma_result ma_get_AudioUnit_channel_map(ma_context* pContext, AudioUnit audioUnit,
> ma_device_type deviceType, ma_channel* pChannelMap, size_t channelMapCap)
34646 | {
34647 |     AudioUnitScope deviceScope;
34648 |     AudioUnitElement deviceBus;
34649 |     UInt32 channelLayoutSize;
34650 |     OSStatus status;
34651 |     AudioChannelLayout* pChannelLayout;
34652 |     ma_result result;
34653 |
34654 |     MA_ASSERT(pContext != NULL);
34655 |
34656 |     if (deviceType == ma_device_type_playback) {
34657 |         deviceScope = kAudioUnitScope_Input;
34658 |         deviceBus = MA_COREAUDIO_OUTPUT_BUS;
34659 |     } else {
34660 |         deviceScope = kAudioUnitScope_Output;
34661 |         deviceBus = MA_COREAUDIO_INPUT_BUS;
34662 |     }
34663 |
34664 |     status =
> ((ma_AudioUnitGetPropertyInfo_proc)pContext->coreaudio.AudioUnitGetPropertyInfo)(audioUnit,
> kAudioUnitProperty_AudioChannelLayout, deviceScope, deviceBus, &channelLayoutSize, NULL);
34665 |     if (status != noErr) {
34666 |         return ma_result_from_OSStatus(status);
34667 |     }
34668 |
34669 |     pChannelLayout = (AudioChannelLayout*)ma_malloc(channelLayoutSize,

```

```

> &pContext->allocationCallbacks);
34670 |     if (pChannelLayout == NULL) {
34671 |         return MA_OUT_OF_MEMORY;
34672 |     }
34673 |
34674 |     status = ((ma_AudioUnitGetProperty_proc)pContext->coreaudio.AudioUnitGetProperty)(audioUnit,
> kAudioUnitProperty_AudioChannelLayout, deviceScope, deviceBus, pChannelLayout, &channelLayoutSize);
34675 |     if (status != noErr) {
34676 |         ma_free(pChannelLayout, &pContext->allocationCallbacks);
34677 |         return ma_result_from_OSStatus(status);
34678 |     }
34679 |
34680 |     result = ma_get_channel_map_from_AudioChannelLayout(pChannelLayout, pChannelMap, channelMapCap);
34681 |     if (result != MA_SUCCESS) {
34682 |         ma_free(pChannelLayout, &pContext->allocationCallbacks);
34683 |         return result;
34684 |     }
34685 |
34686 |     ma_free(pChannelLayout, &pContext->allocationCallbacks);
34687 |     return MA_SUCCESS;
34688 | }
34689 | #endif /* MA_APPLE_DESKTOP */
34690 |
34691 |
34692 | #if !defined(MA_APPLE_DESKTOP)
34693 | static void ma_AVAudioSessionPortDescription_to_device_info(AVAudioSessionPortDescription*
> pPortDesc, ma_device_info* pInfo)
34694 | {
34695 |     MA_ZERO_OBJECT(pInfo);
34696 |     ma_strncpy_s(pInfo->name, sizeof(pInfo->name), [pPortDesc.portName UTF8String],
> (size_t)-1);
34697 |     ma_strncpy_s(pInfo->id.coreaudio, sizeof(pInfo->id.coreaudio), [pPortDesc.UID UTF8String],
> (size_t)-1);
34698 | }
34699 | #endif
34700 |
34701 | static ma_result ma_context_enumerate_devices__coreaudio(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
34702 | {
34703 |     #if defined(MA_APPLE_DESKTOP)
34704 |         UInt32 deviceCount;
34705 |         AudioObjectID* pDeviceObjectIDs;
34706 |         AudioObjectID defaultDeviceObjectIDPlayback;
34707 |         AudioObjectID defaultDeviceObjectIDCapture;
34708 |         ma_result result;
34709 |         UInt32 iDevice;
34710 |
34711 |         ma_find_default_AudioObjectID(pContext, ma_device_type_playback,
> &defaultDeviceObjectIDPlayback); /* OK if this fails. */
34712 |         ma_find_default_AudioObjectID(pContext, ma_device_type_capture, &defaultDeviceObjectIDCapture);
> /* OK if this fails. */
34713 |
34714 |         result = ma_get_device_object_ids__coreaudio(pContext, &deviceCount, &pDeviceObjectIDs);
34715 |         if (result != MA_SUCCESS) {
34716 |             return result;
34717 |         }
34718 |
34719 |         for (iDevice = 0; iDevice < deviceCount; ++iDevice) {
34720 |             AudioObjectID deviceObjectID = pDeviceObjectIDs[iDevice];
34721 |             ma_device_info info;
34722 |
34723 |             MA_ZERO_OBJECT(&info);
34724 |             if (ma_get_AudioObject_uid(pContext, deviceObjectID, sizeof(info.id.coreaudio),
> info.id.coreaudio) != MA_SUCCESS) {
34725 |                 continue;
34726 |             }
34727 |             if (ma_get_AudioObject_name(pContext, deviceObjectID, sizeof(info.name), info.name) !=
> MA_SUCCESS) {
34728 |                 continue;
34729 |             }
34730 |
34731 |             if (ma_does_AudioObject_support_playback(pContext, deviceObjectID)) {
34732 |                 if (deviceObjectID == defaultDeviceObjectIDPlayback) {
34733 |                     info.isDefault = MA_TRUE;
34734 |                 }
34735 |
34736 |                 if (!callback(pContext, ma_device_type_playback, &info, pUserData)) {
34737 |                     break;
34738 |                 }

```

```

34739 |     }
34740 |     if (ma_does_AudioObject_support_capture(pContext, deviceObjectID)) {
34741 |         if (deviceObjectID == defaultDeviceObjectIDCapture) {
34742 |             info.isDefault = MA_TRUE;
34743 |         }
34744 |
34745 |         if (!callback(pContext, ma_device_type_capture, &info, pUserData)) {
34746 |             break;
34747 |         }
34748 |     }
34749 | }
34750 |
34751 | ma_free(pDeviceObjectIDs, &pContext->allocationCallbacks);
34752 | #else
34753 |     ma_device_info info;
34754 |     NSArray *pInputs = [[[AVAAudioSession sharedInstance] currentRoute] inputs];
34755 |     NSArray *pOutputs = [[[AVAAudioSession sharedInstance] currentRoute] outputs];
34756 |
34757 |     for (AVAAudioSessionPortDescription* pPortDesc in pOutputs) {
34758 |         ma_AVAudioSessionPortDescription_to_device_info(pPortDesc, &info);
34759 |         if (!callback(pContext, ma_device_type_playback, &info, pUserData)) {
34760 |             return MA_SUCCESS;
34761 |         }
34762 |     }
34763 |
34764 |     for (AVAAudioSessionPortDescription* pPortDesc in pInputs) {
34765 |         ma_AVAudioSessionPortDescription_to_device_info(pPortDesc, &info);
34766 |         if (!callback(pContext, ma_device_type_capture, &info, pUserData)) {
34767 |             return MA_SUCCESS;
34768 |         }
34769 |     }
34770 | #endif
34771 |
34772 |     return MA_SUCCESS;
34773 | }
34774 |
34775 | static ma_result ma_context_get_device_info_coreaudio(ma_context* pContext, ma_device_type
> deviceType, const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
34776 | {
34777 |     ma_result result;
34778 |
34779 |     MA_ASSERT(pContext != NULL);
34780 |
34781 |     #if defined(MA_APPLE_DESKTOP)
34782 |         /* Desktop */
34783 |         {
34784 |             AudioObjectID deviceObjectID;
34785 |             AudioObjectID defaultDeviceObjectID;
34786 |             UInt32 streamDescriptionCount;
34787 |             AudioStreamRangedDescription* pStreamDescriptions;
34788 |             UInt32 iStreamDescription;
34789 |             UInt32 sampleRateRangeCount;
34790 |             AudioValueRange* pSampleRateRanges;
34791 |
34792 |             ma_find_default_AudioObjectID(pContext, deviceType, &defaultDeviceObjectID); /* OK if
> this fails. */
34793 |
34794 |             result = ma_find_AudioObjectID(pContext, deviceType, pDeviceID, &deviceObjectID);
34795 |             if (result != MA_SUCCESS) {
34796 |                 return result;
34797 |             }
34798 |
34799 |             result = ma_get_AudioObject_uid(pContext, deviceObjectID, sizeof(pDeviceInfo->id.coreaudio),
> pDeviceInfo->id.coreaudio);
34800 |             if (result != MA_SUCCESS) {
34801 |                 return result;
34802 |             }
34803 |
34804 |             result = ma_get_AudioObject_name(pContext, deviceObjectID, sizeof(pDeviceInfo->name),
> pDeviceInfo->name);
34805 |             if (result != MA_SUCCESS) {
34806 |                 return result;
34807 |             }
34808 |
34809 |             if (deviceObjectID == defaultDeviceObjectID) {
34810 |                 pDeviceInfo->isDefault = MA_TRUE;
34811 |             }
34812 |
34813 |             /*

```



```

34814 |         There could be a large number of permutations here. Fortunately there is only a single
> channel count
34815 |         being reported which reduces this quite a bit. For sample rates we're only reporting those
> that are
34816 |         one of miniaudio's recognized "standard" rates. If there are still more formats than can fit
> into
34817 |         our fixed sized array we'll just need to truncate them. This is unlikely and will probably
> only happen
34818 |         if some driver performs software data conversion and therefore reports every possible format
> and
34819 |         sample rate.
34820 |         */
34821 |         pDeviceInfo->nativeDataFormatCount = 0;
34822 |
34823 |         /* Formats. */
34824 |         {
34825 |             ma_format uniqueFormats[ma_format_count];
34826 |             ma_uint32 uniqueFormatCount = 0;
34827 |             ma_uint32 channels;
34828 |
34829 |             /* Channels. */
34830 |             result = ma_get_AudioObject_channel_count(pContext, deviceObjectID, deviceType,
> &channels);
34831 |             if (result != MA_SUCCESS) {
34832 |                 return result;
34833 |             }
34834 |
34835 |             /* Formats. */
34836 |             result = ma_get_AudioObject_stream_descriptions(pContext, deviceObjectID, deviceType,
> &streamDescriptionCount, &pStreamDescriptions);
34837 |             if (result != MA_SUCCESS) {
34838 |                 return result;
34839 |             }
34840 |
34841 |             for (iStreamDescription = 0; iStreamDescription < streamDescriptionCount;
> ++iStreamDescription) {
34842 |                 ma_format format;
34843 |                 ma_bool32 hasFormatBeenHandled = MA_FALSE;
34844 |                 ma_uint32 iOutputFormat;
34845 |                 ma_uint32 iSampleRate;
34846 |
34847 |                 result =
> ma_format_from_AudioStreamBasicDescription(&pStreamDescriptions[iStreamDescription].mFormat,
> &format);
34848 |                 if (result != MA_SUCCESS) {
34849 |                     continue;
34850 |                 }
34851 |
34852 |                 MA_ASSERT(format != ma_format_unknown);
34853 |
34854 |                 /* Make sure the format isn't already in the output list. */
34855 |                 for (iOutputFormat = 0; iOutputFormat < uniqueFormatCount; ++iOutputFormat) {
34856 |                     if (uniqueFormats[iOutputFormat] == format) {
34857 |                         hasFormatBeenHandled = MA_TRUE;
34858 |                         break;
34859 |                     }
34860 |                 }
34861 |
34862 |                 /* If we've already handled this format just skip it. */
34863 |                 if (hasFormatBeenHandled) {
34864 |                     continue;
34865 |                 }
34866 |
34867 |                 uniqueFormats[uniqueFormatCount] = format;
34868 |                 uniqueFormatCount += 1;
34869 |
34870 |                 /* Sample Rates */
34871 |                 result = ma_get_AudioObject_sample_rates(pContext, deviceObjectID, deviceType,
> &sampleRateRangeCount, &pSampleRateRanges);
34872 |                 if (result != MA_SUCCESS) {
34873 |                     return result;
34874 |                 }
34875 |
34876 |                 /*
34877 |                 Annoyingly Core Audio reports a sample rate range. We just get all the standard
> rates that are
34878 |                 between this range.
34879 |                 */
34880 |                 for (iSampleRate = 0; iSampleRate < sampleRateRangeCount; ++iSampleRate) {

```

```

34881 |             ma_uint32 iStandardSampleRate;
34882 |             for (iStandardSampleRate = 0; iStandardSampleRate <
> ma_countof(g_maStandardSampleRatePriorities); iStandardSampleRate += 1) {
34883 |                 ma_uint32 standardSampleRate =
> g_maStandardSampleRatePriorities[iStandardSampleRate];
34884 |                 if (standardSampleRate >= pSampleRateRanges[iSampleRate].mMinimum &&
> standardSampleRate <= pSampleRateRanges[iSampleRate].mMaximum) {
34885 |                     /* We have a new data format. Add it to the list. */
34886 |
> pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format    = format;
34887 |
> pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels    = channels;
34888 |
> pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate = standardSampleRate;
34889 |                     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags
>
= 0;
34890 |                     pDeviceInfo->nativeDataFormatCount += 1;
34891 |
34892 |                     if (pDeviceInfo->nativeDataFormatCount >=
> ma_countof(pDeviceInfo->nativeDataFormats)) {
34893 |                         break; /* No more room for any more formats. */
34894 |                     }
34895 |                 }
34896 |             }
34897 |         }
34898 |
34899 |         ma_free(pSampleRateRanges, &pContext->allocationCallbacks);
34900 |
34901 |         if (pDeviceInfo->nativeDataFormatCount >=
> ma_countof(pDeviceInfo->nativeDataFormats)) {
34902 |             break; /* No more room for any more formats. */
34903 |         }
34904 |     }
34905 |
34906 |     ma_free(pStreamDescriptions, &pContext->allocationCallbacks);
34907 | }
34908 | }
34909 | #else
34910 | /* Mobile */
34911 | {
34912 |     AudioComponentDescription desc;
34913 |     AudioComponent component;
34914 |     AudioUnit audioUnit;
34915 |     OSStatus status;
34916 |     AudioUnitScope formatScope;
34917 |     AudioUnitElement formatElement;
34918 |     AudioStreamBasicDescription bestFormat;
34919 |     UInt32 propSize;
34920 |
34921 |     /* We want to ensure we use a consistent device name to device enumeration. */
34922 |     if (pDeviceID != NULL && pDeviceID->coreaudio[0] != '\0') {
34923 |         ma_bool32 found = MA_FALSE;
34924 |         if (deviceType == ma_device_type_playback) {
34925 |             NSArray *pOutputs = [[[AVAudioSession sharedInstance] currentRoute] outputs];
34926 |             for (AVAudioSessionPortDescription* pPortDesc in pOutputs) {
34927 |                 if (strcmp(pDeviceID->coreaudio, [pPortDesc.UUID UTF8String]) == 0) {
34928 |                     ma_AVAudioSessionPortDescription_to_device_info(pPortDesc, pDeviceInfo);
34929 |                     found = MA_TRUE;
34930 |                     break;
34931 |                 }
34932 |             }
34933 |         } else {
34934 |             NSArray *pInputs = [[[AVAudioSession sharedInstance] currentRoute] inputs];
34935 |             for (AVAudioSessionPortDescription* pPortDesc in pInputs) {
34936 |                 if (strcmp(pDeviceID->coreaudio, [pPortDesc.UUID UTF8String]) == 0) {
34937 |                     ma_AVAudioSessionPortDescription_to_device_info(pPortDesc, pDeviceInfo);
34938 |                     found = MA_TRUE;
34939 |                     break;
34940 |                 }
34941 |             }
34942 |         }
34943 |
34944 |         if (!found) {
34945 |             return MA_DOES_NOT_EXIST;
34946 |         }
34947 |     } else {
34948 |         if (deviceType == ma_device_type_playback) {
34949 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name),
> MA_DEFAULT_PLAYBACK_DEVICE_NAME, (size_t)-1);

```

```

34950 |         } else {
34951 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name),
> MA_DEFAULT_CAPTURE_DEVICE_NAME, (size_t)-1);
34952 |         }
34953 |     }
34954 |
34955 |
34956 |     /*
34957 |     Retrieving device information is more annoying on mobile than desktop. For simplicity I'm
> locking this down to whatever format is
34958 |     reported on a temporary I/O unit. The problem, however, is that this doesn't return a value
> for the sample rate which we need to
34959 |     retrieve from the AVAudioSession shared instance.
34960 |     */
34961 |     desc.componentType = kAudioUnitType_Output;
34962 |     desc.componentSubType = kAudioUnitSubType_RemoteIO;
34963 |     desc.componentManufacturer = kAudioUnitManufacturer_Apple;
34964 |     desc.componentFlags = 0;
34965 |     desc.componentFlagsMask = 0;
34966 |
34967 |     component =
> ((ma_AudioComponentFindNext_proc)pContext->coreaudio.AudioComponentFindNext)(NULL, &desc);
34968 |     if (component == NULL) {
34969 |         return MA_FAILED_TO_INIT_BACKEND;
34970 |     }
34971 |
34972 |     status =
> ((ma_AudioComponentInstanceNew_proc)pContext->coreaudio.AudioComponentInstanceNew)(component,
> &audioUnit);
34973 |     if (status != noErr) {
34974 |         return ma_result_from_OSStatus(status);
34975 |     }
34976 |
34977 |     formatScope = (deviceType == ma_device_type_playback) ? kAudioUnitScope_Input :
> kAudioUnitScope_Output;
34978 |     formatElement = (deviceType == ma_device_type_playback) ? MA_COREAUDIO_OUTPUT_BUS :
> MA_COREAUDIO_INPUT_BUS;
34979 |
34980 |     propSize = sizeof(bestFormat);
34981 |     status = ((ma_AudioUnitGetProperty_proc)pContext->coreaudio.AudioUnitGetProperty)(audioUnit,
> kAudioUnitProperty_StreamFormat, formatScope, formatElement, &bestFormat, &propSize);
34982 |     if (status != noErr) {
34983 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(audioUnit);
34984 |         return ma_result_from_OSStatus(status);
34985 |     }
34986 |
34987 |     ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispose)(a
> udioUnit);
34988 |     audioUnit = NULL;
34989 |
34990 |     /* Only a single format is being reported for iOS. */
34991 |     pDeviceInfo->nativeDataFormatCount = 1;
34992 |
34993 |     result = ma_format_from_AudioStreamBasicDescription(&bestFormat,
> &pDeviceInfo->nativeDataFormats[0].format);
34994 |     if (result != MA_SUCCESS) {
34995 |         return result;
34996 |     }
34997 |
34998 |     pDeviceInfo->nativeDataFormats[0].channels = bestFormat.mChannelsPerFrame;
34999 |
35000 |     /*
35001 |     It looks like Apple are wanting to push the whole AVAudioSession thing. Thus, we need to use
> that to determine device settings. To do
35002 |     this we just get the shared instance and inspect.
35003 |     */
35004 |     @autoreleasepool {
35005 |         AVAudioSession* pAudioSession = [AVAudioSession sharedInstance];
35006 |         MA_ASSERT(pAudioSession != NULL);
35007 |
35008 |         pDeviceInfo->nativeDataFormats[0].sampleRate = (ma_uint32)pAudioSession.sampleRate;
35009 |     }
35010 | }
35011 | #endif
35012 |
35013 |     (void)pDeviceInfo; /* Unused. */
35014 |     return MA_SUCCESS;
35015 | }

```

```

35016 |
35017 | static AudioBufferList* ma_allocate_AudioBufferList__coreaudio(ma_uint32 sizeInFrames, ma_format
> format, ma_uint32 channels, ma_stream_layout layout, const ma_allocation_callbacks*
> pAllocationCallbacks)
35018 | {
35019 |     AudioBufferList* pBufferList;
35020 |     UInt32 audioBufferSizeInBytes;
35021 |     size_t allocationSize;
35022 |
35023 |     MA_ASSERT(sizeInFrames > 0);
35024 |     MA_ASSERT(format != ma_format_unknown);
35025 |     MA_ASSERT(channels > 0);
35026 |
35027 |     allocationSize = sizeof(AudioBufferList) - sizeof(AudioBuffer); /* Subtract sizeof(AudioBuffer)
> because that part is dynamically sized. */
35028 |     if (layout == ma_stream_layout_interleaved) {
35029 |         /* Interleaved case. This is the simple case because we just have one buffer. */
35030 |         allocationSize += sizeof(AudioBuffer) * 1;
35031 |     } else {
35032 |         /* Non-interleaved case. This is the more complex case because there's more than one buffer.
> */
35033 |         allocationSize += sizeof(AudioBuffer) * channels;
35034 |     }
35035 |
35036 |     allocationSize += sizeInFrames * ma_get_bytes_per_frame(format, channels);
35037 |
35038 |     pBufferList = (AudioBufferList*)ma_malloc(allocationSize, pAllocationCallbacks);
35039 |     if (pBufferList == NULL) {
35040 |         return NULL;
35041 |     }
35042 |
35043 |     audioBufferSizeInBytes = (UInt32)(sizeInFrames * ma_get_bytes_per_sample(format));
35044 |
35045 |     if (layout == ma_stream_layout_interleaved) {
35046 |         pBufferList->mNumberBuffers = 1;
35047 |         pBufferList->mBuffers[0].mNumberChannels = channels;
35048 |         pBufferList->mBuffers[0].mDataByteSize = audioBufferSizeInBytes * channels;
35049 |         pBufferList->mBuffers[0].mData = (ma_uint8*)pBufferList + sizeof(AudioBufferList);
35050 |     } else {
35051 |         ma_uint32 iBuffer;
35052 |         pBufferList->mNumberBuffers = channels;
35053 |         for (iBuffer = 0; iBuffer < pBufferList->mNumberBuffers; ++iBuffer) {
35054 |             pBufferList->mBuffers[iBuffer].mNumberChannels = 1;
35055 |             pBufferList->mBuffers[iBuffer].mDataByteSize = audioBufferSizeInBytes;
35056 |             pBufferList->mBuffers[iBuffer].mData = (ma_uint8*)pBufferList +
> ((sizeof(AudioBufferList) - sizeof(AudioBuffer)) + (sizeof(AudioBuffer) * channels)) +
> (audioBufferSizeInBytes * iBuffer);
35057 |         }
35058 |     }
35059 |
35060 |     return pBufferList;
35061 | }
35062 |
35063 | static ma_result ma_device_realloc_AudioBufferList__coreaudio(ma_device* pDevice, ma_uint32
> sizeInFrames, ma_format format, ma_uint32 channels, ma_stream_layout layout)
35064 | {
35065 |     MA_ASSERT(pDevice != NULL);
35066 |     MA_ASSERT(format != ma_format_unknown);
35067 |     MA_ASSERT(channels > 0);
35068 |
35069 |     /* Only resize the buffer if necessary. */
35070 |     if (pDevice->coreaudio.audioBufferCapInFrames < sizeInFrames) {
35071 |         AudioBufferList* pNewAudioBufferList;
35072 |
35073 |         pNewAudioBufferList = ma_allocate_AudioBufferList__coreaudio(sizeInFrames, format, channels,
> layout, &pDevice->pContext->allocationCallbacks);
35074 |         if (pNewAudioBufferList == NULL) {
35075 |             return MA_OUT_OF_MEMORY;
35076 |         }
35077 |
35078 |         /* At this point we'll have a new AudioBufferList and we can free the old one. */
35079 |         ma_free(pDevice->coreaudio.pAudioBufferList, &pDevice->pContext->allocationCallbacks);
35080 |         pDevice->coreaudio.pAudioBufferList = pNewAudioBufferList;
35081 |         pDevice->coreaudio.audioBufferCapInFrames = sizeInFrames;
35082 |     }
35083 |
35084 |     /* Getting here means the capacity of the audio is fine. */
35085 |     return MA_SUCCESS;
35086 | }

```

```

35087 |
35088 |
35089 | static OSStatus ma_on_output__coreaudio(void* pUserData, AudioUnitRenderActionFlags* pActionFlags,
> const AudioTimeStamp* pTimeStamp, UInt32 busNumber, UInt32 frameCount, AudioBufferList* pBufferList)
35090 | {
35091 |     ma_device* pDevice = (ma_device*)pUserData;
35092 |     ma_stream_layout layout;
35093 |
35094 |     MA_ASSERT(pDevice != NULL);
35095 |
35096 |     /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "INFO: Output Callback:
> busNumber=%d, frameCount=%d, mNumberBuffers=%d\n", (int)busNumber, (int)frameCount,
> (int)pBufferList->mNumberBuffers);*/
35097 |
35098 |     /* We need to check whether or not we are outputting interleaved or non-interleaved samples. The
> way we do this is slightly different for each type. */
35099 |     layout = ma_stream_layout_interleaved;
35100 |     if (pBufferList->mBuffers[0].mNumberChannels != pDevice->playback.internalChannels) {
35101 |         layout = ma_stream_layout_deinterleaved;
35102 |     }
35103 |
35104 |     if (layout == ma_stream_layout_interleaved) {
35105 |         /* For now we can assume everything is interleaved. */
35106 |         UInt32 iBuffer;
35107 |         for (iBuffer = 0; iBuffer < pBufferList->mNumberBuffers; ++iBuffer) {
35108 |             if (pBufferList->mBuffers[iBuffer].mNumberChannels ==
> pDevice->playback.internalChannels) {
35109 |                 ma_uint32 frameCountForThisBuffer = pBufferList->mBuffers[iBuffer].mDataByteSize /
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
35110 |                 if (frameCountForThisBuffer > 0) {
35111 |                     ma_device_handle_backend_data_callback(pDevice,
> pBufferList->mBuffers[iBuffer].mData, NULL, frameCountForThisBuffer);
35112 |                 }
35113 |
35114 |                 /*a_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, " frameCount=%d,
> mNumberChannels=%d, mDataByteSize=%d\n", (int)frameCount,
> (int)pBufferList->mBuffers[iBuffer].mNumberChannels,
> (int)pBufferList->mBuffers[iBuffer].mDataByteSize);*/
35115 |             } else {
35116 |                 /*
35117 |                 This case is where the number of channels in the output buffer do not match our
> internal channels. It could mean that it's
35118 |                 not interleaved, in which case we can't handle right now since miniaudio does not
> yet support non-interleaved streams. We just
35119 |                 output silence here.
35120 |                 */
35121 |                 MA_ZERO_MEMORY(pBufferList->mBuffers[iBuffer].mData,
> pBufferList->mBuffers[iBuffer].mDataByteSize);
35122 |                 /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, " WARNING:
> Outputting silence. frameCount=%d, mNumberChannels=%d, mDataByteSize=%d\n", (int)frameCount,
> (int)pBufferList->mBuffers[iBuffer].mNumberChannels,
> (int)pBufferList->mBuffers[iBuffer].mDataByteSize);*/
35123 |             }
35124 |         }
35125 |     } else {
35126 |         /* This is the deinterleaved case. We need to update each buffer in groups of
> internalChannels. This assumes each buffer is the same size. */
35127 |         MA_ASSERT(pDevice->playback.internalChannels <= MA_MAX_CHANNELS); /* This should have been
> validated at initialization time. */
35128 |
35129 |         /*
35130 |         For safety we'll check that the internal channels is a multiple of the buffer count. If it's
> not it means something
35131 |         very strange has happened and we're not going to support it.
35132 |         */
35133 |         if ((pBufferList->mNumberBuffers % pDevice->playback.internalChannels) == 0) {
35134 |             ma_uint8 tempBuffer[4096];
35135 |             UInt32 iBuffer;
35136 |
35137 |             for (iBuffer = 0; iBuffer < pBufferList->mNumberBuffers; iBuffer +=
> pDevice->playback.internalChannels) {
35138 |                 ma_uint32 frameCountPerBuffer = pBufferList->mBuffers[iBuffer].mDataByteSize /
> ma_get_bytes_per_sample(pDevice->playback.internalFormat);
35139 |                 ma_uint32 framesRemaining = frameCountPerBuffer;
35140 |
35141 |                 while (framesRemaining > 0) {
35142 |                     void* ppDeinterleavedBuffers[MA_MAX_CHANNELS];
35143 |                     ma_uint32 iChannel;
35144 |                     ma_uint32 framesToRead = sizeof(tempBuffer) /

```

```

> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
35145 |         if (framesToRead > framesRemaining) {
35146 |             framesToRead = framesRemaining;
35147 |         }
35148 |
35149 |         ma_device_handle_backend_data_callback(pDevice, tempBuffer, NULL, framesToRead);
35150 |
35151 |         for (iChannel = 0; iChannel < pDevice->playback.internalChannels; ++iChannel) {
35152 |             ppDeinterleavedBuffers[iChannel] =
> (void*)ma_offset_ptr(pBufferList->mBuffers[iBuffer+iChannel].mData, (frameCountPerBuffer -
> framesRemaining) * ma_get_bytes_per_sample(pDevice->playback.internalFormat));
35153 |         }
35154 |
35155 |         ma_deinterleave_pcm_frames(pDevice->playback.internalFormat,
> pDevice->playback.internalChannels, framesToRead, tempBuffer, ppDeinterleavedBuffers);
35156 |
35157 |         framesRemaining -= framesToRead;
35158 |     }
35159 | }
35160 | }
35161 | }
35162 |
35163 | (void)pActionFlags;
35164 | (void)pTimeStamp;
35165 | (void)busNumber;
35166 | (void)frameCount;
35167 |
35168 | return noErr;
35169 | }
35170 |
35171 | static OSStatus ma_on_input__coreaudio(void* pUserData, AudioUnitRenderActionFlags* pActionFlags,
> const AudioTimeStamp* pTimeStamp, UInt32 busNumber, UInt32 frameCount, AudioBufferList*
> pUnusedBufferList)
35172 | {
35173 |     ma_device* pDevice = (ma_device*)pUserData;
35174 |     AudioBufferList* pRenderedBufferList;
35175 |     ma_result result;
35176 |     ma_stream_layout layout;
35177 |     ma_uint32 iBuffer;
35178 |     OSStatus status;
35179 |
35180 |     MA_ASSERT(pDevice != NULL);
35181 |
35182 |     pRenderedBufferList = (AudioBufferList*)pDevice->coreaudio.pAudioBufferList;
35183 |     MA_ASSERT(pRenderedBufferList);
35184 |
35185 |     /* We need to check whether or not we are outputting interleaved or non-interleaved samples. The
> way we do this is slightly different for each type. */
35186 |     layout = ma_stream_layout_interleaved;
35187 |     if (pRenderedBufferList->mBuffers[0].mNumberChannels != pDevice->capture.internalChannels) {
35188 |         layout = ma_stream_layout_deinterleaved;
35189 |     }
35190 |
35191 |     /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "INFO: Input Callback:
> busNumber=%d, frameCount=%d, mNumberBuffers=%d\n", (int)busNumber, (int)frameCount,
> (int)pRenderedBufferList->mNumberBuffers);*/
35192 |
35193 |     /*
35194 |     > There has been a situation reported where frame count passed into this function is greater than
> the capacity of
35195 |     our capture buffer. There doesn't seem to be a reliable way to determine what the maximum frame
> count will be,
35196 |     so we need to instead resort to dynamically reallocating our buffer to ensure it's large enough
> to capture the
35197 |     number of frames requested by this callback.
35198 |     */
35199 |     result = ma_device_realloc_AudioBufferList__coreaudio(pDevice, frameCount,
> pDevice->capture.internalFormat, pDevice->capture.internalChannels, layout);
35200 |     if (result != MA_SUCCESS) {
35201 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "Failed to allocate
> AudioBufferList for capture.\n");
35202 |         return noErr;
35203 |     }
35204 |
35205 |     pRenderedBufferList = (AudioBufferList*)pDevice->coreaudio.pAudioBufferList;
35206 |     MA_ASSERT(pRenderedBufferList);
35207 |
35208 |     /*
35209 |     When you call AudioUnitRender(), Core Audio tries to be helpful by setting the mDataByteSize to

```

```

> the number of bytes
35210 |     that were actually rendered. The problem with this is that the next call can fail with -50 due
> to the size no longer
35211 |     being set to the capacity of the buffer, but instead the size in bytes of the previous render.
> This will cause a
35212 |     problem when a future call to this callback specifies a larger number of frames.
35213 |
35214 |     To work around this we need to explicitly set the size of each buffer to their respective size
> in bytes.
35215 |     */
35216 |     for (iBuffer = 0; iBuffer < pRenderedBufferList->mNumberBuffers; ++iBuffer) {
35217 |         pRenderedBufferList->mBuffers[iBuffer].mDataByteSize =
> pDevice->coreaudio.audioBufferCapInFrames * ma_get_bytes_per_sample(pDevice->capture.internalFormat)
> * pRenderedBufferList->mBuffers[iBuffer].mNumberChannels;
35218 |         /*printf("DEBUG: mDataByteSize = %d\n",
> (int)pRenderedBufferList->mBuffers[iBuffer].mDataByteSize);*/
35219 |     }
35220 |
35221 |     status = ((ma_AudioUnitRender_proc)pDevice->pContext->coreaudio.AudioUnitRender)((AudioUnit)pDev
> ice->coreaudio.audioUnitCapture, pActionFlags, pTimeStamp, busNumber, frameCount,
> pRenderedBufferList);
35222 |     if (status != noErr) {
35223 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "ERROR: AudioUnitRender()
> failed with %d.\n", (int)status);
35224 |         return status;
35225 |     }
35226 |
35227 |     if (layout == ma_stream_layout_interleaved) {
35228 |         for (iBuffer = 0; iBuffer < pRenderedBufferList->mNumberBuffers; ++iBuffer) {
35229 |             if (pRenderedBufferList->mBuffers[iBuffer].mNumberChannels ==
> pDevice->capture.internalChannels) {
35230 |                 ma_device_handle_backend_data_callback(pDevice, NULL,
> pRenderedBufferList->mBuffers[iBuffer].mData, frameCount);
35231 |                 /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, "
> mDataByteSize=%d.\n", (int)pRenderedBufferList->mBuffers[iBuffer].mDataByteSize);*/
35232 |             } else {
35233 |                 /*
35234 |                 This case is where the number of channels in the output buffer do not match our
> internal channels. It could mean that it's
35235 |                 not interleaved, in which case we can't handle right now since miniaudio does not
> yet support non-interleaved streams.
35236 |                 */
35237 |                 ma_uint8 silentBuffer[4096];
35238 |                 ma_uint32 framesRemaining;
35239 |
35240 |                 MA_ZERO_MEMORY(silentBuffer, sizeof(silentBuffer));
35241 |
35242 |                 framesRemaining = frameCount;
35243 |                 while (framesRemaining > 0) {
35244 |                     ma_uint32 framesToSend = sizeof(silentBuffer) /
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels);
35245 |                     if (framesToSend > framesRemaining) {
35246 |                         framesToSend = framesRemaining;
35247 |                     }
35248 |
35249 |                     ma_device_handle_backend_data_callback(pDevice, NULL, silentBuffer,
> framesToSend);
35250 |
35251 |                     framesRemaining -= framesToSend;
35252 |                 }
35253 |
35254 |                 /*ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_DEBUG, " WARNING:
> Outputting silence. frameCount=%d, mNumberChannels=%d, mDataByteSize=%d\n", (int)frameCount,
> (int)pRenderedBufferList->mBuffers[iBuffer].mNumberChannels,
> (int)pRenderedBufferList->mBuffers[iBuffer].mDataByteSize);*/
35255 |             }
35256 |         }
35257 |     } else {
35258 |         /* This is the deinterleaved case. We need to interleave the audio data before sending it to
> the client. This assumes each buffer is the same size. */
35259 |         MA_ASSERT(pDevice->capture.internalChannels <= MA_MAX_CHANNELS); /* This should have been
> validated at initialization time. */
35260 |
35261 |         /*
35262 |         For safety we'll check that the internal channels is a multiple of the buffer count. If it's
> not it means something
35263 |         very strange has happened and we're not going to support it.
35264 |         */
35265 |         if ((pRenderedBufferList->mNumberBuffers % pDevice->capture.internalChannels) == 0) {

```



```

35266 |         ma_uint8 tempBuffer[4096];
35267 |         for (iBuffer = 0; iBuffer < pRenderedBufferList->mNumberBuffers; iBuffer +=
> pDevice->capture.internalChannels) {
35268 |             ma_uint32 framesRemaining = frameCount;
35269 |             while (framesRemaining > 0) {
35270 |                 void* ppDeinterleavedBuffers[MA_MAX_CHANNELS];
35271 |                 ma_uint32 iChannel;
35272 |                 ma_uint32 framesToSend = sizeof(tempBuffer) /
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels);
35273 |                 if (framesToSend > framesRemaining) {
35274 |                     framesToSend = framesRemaining;
35275 |                 }
35276 |
35277 |                 for (iChannel = 0; iChannel < pDevice->capture.internalChannels; ++iChannel) {
35278 |                     ppDeinterleavedBuffers[iChannel] =
> (void*)ma_offset_ptr(pRenderedBufferList->mBuffers[iBuffer+iChannel].mData, (frameCount -
> framesRemaining) * ma_get_bytes_per_sample(pDevice->capture.internalFormat));
35279 |                 }
35280 |
35281 |                 ma_interleave_pcm_frames(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels, framesToSend, (const void**)ppDeinterleavedBuffers, tempBuffer);
35282 |                 ma_device_handle_backend_data_callback(pDevice, NULL, tempBuffer, framesToSend);
35283 |
35284 |                 framesRemaining -= framesToSend;
35285 |             }
35286 |         }
35287 |     }
35288 | }
35289 |
35290 | (void)pActionFlags;
35291 | (void)pTimeStamp;
35292 | (void)busNumber;
35293 | (void)frameCount;
35294 | (void)pUnusedBufferList;
35295 |
35296 | return noErr;
35297 | }
35298 |
35299 | static void on_start_stop_coreaudio(void* pUserData, AudioUnit audioUnit, AudioUnitPropertyID
> propertyID, AudioUnitScope scope, AudioUnitElement element)
35300 | {
35301 |     ma_device* pDevice = (ma_device*)pUserData;
35302 |     MA_ASSERT(pDevice != NULL);
35303 |
35304 |     /* Don't do anything if it looks like we're just reinitializing due to a device switch. */
35305 |     if (((audioUnit == pDevice->coreaudio.audioUnitPlayback) &&
> pDevice->coreaudio.isSwitchingPlaybackDevice) ||
35306 |         ((audioUnit == pDevice->coreaudio.audioUnitCapture) &&
> pDevice->coreaudio.isSwitchingCaptureDevice)) {
35307 |         return;
35308 |     }
35309 |
35310 |     /*
35311 |     There's been a report of a deadlock here when triggered by ma_device_uninit(). It looks like
35312 |     AudioUnitGetProperty (called below) and AudioComponentInstanceDispose (called in
> ma_device_uninit)
35313 |     can try waiting on the same lock. I'm going to try working around this by not calling any Core
35314 |     Audio APIs in the callback when the device has been stopped or uninitialized.
35315 |     */
35316 |     if (ma_device_get_state(pDevice) == ma_device_state_uninitialized ||
> ma_device_get_state(pDevice) == ma_device_state_stopping || ma_device_get_state(pDevice) ==
> ma_device_state_stopped) {
35317 |         ma_device__on_notification_stopped(pDevice);
35318 |     } else {
35319 |         UInt32 isRunning;
35320 |         UInt32 isRunningSize = sizeof(isRunning);
35321 |         OSSStatus status =
> ((ma_AudioUnitGetProperty_proc)pDevice->pContext->coreaudio.AudioUnitGetProperty)(audioUnit,
> kAudioOutputUnitProperty_IsRunning, scope, element, &isRunning, &isRunningSize);
35322 |         if (status != noErr) {
35323 |             goto done; /* Don't really know what to do in this case... just ignore it, I suppose...
> */
35324 |         }
35325 |
35326 |         if (!isRunning) {
35327 |             /*
35328 |             The stop event is a bit annoying in Core Audio because it will be called when we
> automatically switch the default device. Some scenarios to consider:
35329 |

```

```

35330 |         1) When the device is unplugged, this will be called _before_ the default device change
> notification.
35331 |         2) When the device is changed via the default device change notification, this will be
> called _after_ the switch.
35332 |
35333 |         For case #1, we just check if there's a new default device available. If so, we just
> ignore the stop event. For case #2 we check a flag.
35334 |         */
35335 |         if (((audioUnit == pDevice->coreaudio.audioUnitPlayback) &&
> pDevice->coreaudio.isDefaultPlaybackDevice) ||
35336 |             ((audioUnit == pDevice->coreaudio.audioUnitCapture) &&
> pDevice->coreaudio.isDefaultCaptureDevice)) {
35337 |             /*
35338 |             It looks like the device is switching through an external event, such as the user
> unplugging the device or changing the default device
35339 |             via the operating system's sound settings. If we're re-initializing the device, we
> just terminate because we want the stopping of the
35340 |             device to be seamless to the client (we don't want them receiving the stopped event
> and thinking that the device has stopped when it
35341 |             hasn't!).
35342 |             */
35343 |             if (((audioUnit == pDevice->coreaudio.audioUnitPlayback) &&
> pDevice->coreaudio.isSwitchingPlaybackDevice) ||
35344 |                 ((audioUnit == pDevice->coreaudio.audioUnitCapture) &&
> pDevice->coreaudio.isSwitchingCaptureDevice)) {
35345 |                 goto done;
35346 |             }
35347 |
35348 |             /*
35349 |             Getting here means the device is not reinitializing which means it may have been
> unplugged. From what I can see, it looks like Core Audio
35350 |             will try switching to the new default device seamlessly. We need to somehow find a
> way to determine whether or not Core Audio will most
35351 |             likely be successful in switching to the new device.
35352 |
35353 |             TODO: Try to predict if Core Audio will switch devices. If not, the stopped callback
> needs to be posted.
35354 |             */
35355 |             goto done;
35356 |         }
35357 |
35358 |         /* Getting here means we need to stop the device. */
35359 |         ma_device__on_notification_stopped(pDevice);
35360 |     }
35361 | }
35362 |
35363 | (void)propertyID; /* Unused. */
35364 |
35365 | done:
35366 |     /* Always signal the stop event. It's possible for the "else" case to get hit which can happen
> during an interruption. */
35367 |     ma_event_signal(&pDevice->coreaudio.stopEvent);
35368 | }
35369 |
35370 | #if defined(MA_APPLE_DESKTOP)
35371 | static ma_spinlock g_DeviceTrackingInitLock_CoreAudio = 0; /* A spinlock for mutal exclusion of the
> init/uninit of the global tracking data. Initialization to 0 is what we need. */
35372 | static ma_uint32 g_DeviceTrackingInitCounter_CoreAudio = 0;
35373 | static ma_mutex g_DeviceTrackingMutex_CoreAudio;
35374 | static ma_device** g_ppTrackedDevices_CoreAudio = NULL;
35375 | static ma_uint32 g_TrackedDeviceCap_CoreAudio = 0;
35376 | static ma_uint32 g_TrackedDeviceCount_CoreAudio = 0;
35377 |
35378 | static OSStatus ma_default_device_changed__coreaudio(AudioObjectID objectID, UInt32 addressCount,
> const AudioObjectPropertyAddress* pAddresses, void* pUserData)
35379 | {
35380 |     ma_device_type deviceType;
35381 |
35382 |     /* Not sure if I really need to check this, but it makes me feel better. */
35383 |     if (addressCount == 0) {
35384 |         return noErr;
35385 |     }
35386 |
35387 |     if (pAddresses[0].mSelector == kAudioHardwarePropertyDefaultOutputDevice) {
35388 |         deviceType = ma_device_type_playback;
35389 |     } else if (pAddresses[0].mSelector == kAudioHardwarePropertyDefaultInputDevice) {
35390 |         deviceType = ma_device_type_capture;
35391 |     } else {
35392 |         return noErr; /* Should never hit this. */

```

```

35393 |     }
35394 |
35395 |     ma_mutex_lock(&g_DeviceTrackingMutex_CoreAudio);
35396 |     {
35397 |         ma_uint32 iDevice;
35398 |         for (iDevice = 0; iDevice < g_TrackedDeviceCount_CoreAudio; iDevice += 1) {
35399 |             ma_result reinitResult;
35400 |             ma_device* pDevice;
35401 |
35402 |             pDevice = g_ppTrackedDevices_CoreAudio[iDevice];
35403 |             if (pDevice->type == deviceType || pDevice->type == ma_device_type_duplex) {
35404 |                 if (deviceType == ma_device_type_playback) {
35405 |                     pDevice->coreaudio.isSwitchingPlaybackDevice = MA_TRUE;
35406 |                     reinitResult = ma_device_reinit_internal_coreaudio(pDevice, deviceType,
> MA_TRUE);
35407 |                     pDevice->coreaudio.isSwitchingPlaybackDevice = MA_FALSE;
35408 |                 } else {
35409 |                     pDevice->coreaudio.isSwitchingCaptureDevice = MA_TRUE;
35410 |                     reinitResult = ma_device_reinit_internal_coreaudio(pDevice, deviceType,
> MA_TRUE);
35411 |                     pDevice->coreaudio.isSwitchingCaptureDevice = MA_FALSE;
35412 |                 }
35413 |
35414 |                 if (reinitResult == MA_SUCCESS) {
35415 |                     ma_device__post_init_setup(pDevice, deviceType);
35416 |
35417 |                     /* Restart the device if required. If this fails we need to stop the device
> entirely. */
35418 |                     if (ma_device_get_state(pDevice) == ma_device_state_started) {
35419 |                         OSStatus status;
35420 |                         if (deviceType == ma_device_type_playback) {
35421 |                             status = ((ma_AudioOutputUnitStart_proc)pDevice->pContext->coreaudio.Aud
> ioOutputUnitStart)((AudioUnit)pDevice->coreaudio.audioUnitPlayback);
35422 |                             if (status != noErr) {
35423 |                                 if (pDevice->type == ma_device_type_duplex) {
35424 |                                     ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.Audio
> OutputUnitStop)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
35425 |                                 }
35426 |                                 ma_device__set_state(pDevice, ma_device_state_stopped);
35427 |                             }
35428 |                         } else if (deviceType == ma_device_type_capture) {
35429 |                             status = ((ma_AudioOutputUnitStart_proc)pDevice->pContext->coreaudio.Aud
> ioOutputUnitStart)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
35430 |                             if (status != noErr) {
35431 |                                 if (pDevice->type == ma_device_type_duplex) {
35432 |                                     ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.Audio
> OutputUnitStop)((AudioUnit)pDevice->coreaudio.audioUnitPlayback);
35433 |                                 }
35434 |                                 ma_device__set_state(pDevice, ma_device_state_stopped);
35435 |                             }
35436 |                         }
35437 |                     }
35438 |
35439 |                     ma_device__on_notification_rerouted(pDevice);
35440 |                 }
35441 |             }
35442 |         }
35443 |     }
35444 |     ma_mutex_unlock(&g_DeviceTrackingMutex_CoreAudio);
35445 |
35446 |     /* Unused parameters. */
35447 |     (void)objectID;
35448 |     (void)pUserData;
35449 |
35450 |     return noErr;
35451 | }
35452 |
35453 | static ma_result ma_context__init_device_tracking_coreaudio(ma_context* pContext)
35454 | {
35455 |     MA_ASSERT(pContext != NULL);
35456 |
35457 |     ma_spinlock_lock(&g_DeviceTrackingInitLock_CoreAudio);
35458 |     {
35459 |         /* Don't do anything if we've already initialized device tracking. */
35460 |         if (g_DeviceTrackingInitCounter_CoreAudio == 0) {
35461 |             AudioObjectPropertyAddress propAddress;
35462 |             propAddress.mScope = kAudioObjectPropertyScopeGlobal;
35463 |             propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
35464 |

```

```

35465 |         ma_mutex_init(&g_DeviceTrackingMutex_CoreAudio);
35466 |
35467 |         propAddress.mSelector = kAudioHardwarePropertyDefaultInputDevice;
35468 |         ((ma_AudioObjectAddPropertyListener_proc)pContext->coreaudio.AudioObjectAddPropertyListe
> ner)(kAudioObjectSystemObject, &propAddress, &ma_default_device_changed__coreaudio, NULL);
35469 |
35470 |         propAddress.mSelector = kAudioHardwarePropertyDefaultOutputDevice;
35471 |         ((ma_AudioObjectAddPropertyListener_proc)pContext->coreaudio.AudioObjectAddPropertyListe
> ner)(kAudioObjectSystemObject, &propAddress, &ma_default_device_changed__coreaudio, NULL);
35472 |
35473 |     }
35474 |     g_DeviceTrackingInitCounter_CoreAudio += 1;
35475 | }
35476 | ma_spinlock_unlock(&g_DeviceTrackingInitLock_CoreAudio);
35477 |
35478 | return MA_SUCCESS;
35479 | }
35480 |
35481 | static ma_result ma_context__uninit_device_tracking__coreaudio(ma_context* pContext)
35482 | {
35483 |     MA_ASSERT(pContext != NULL);
35484 |
35485 |     ma_spinlock_lock(&g_DeviceTrackingInitLock_CoreAudio);
35486 |     {
35487 |         if (g_DeviceTrackingInitCounter_CoreAudio > 0)
35488 |             g_DeviceTrackingInitCounter_CoreAudio -= 1;
35489 |
35490 |         if (g_DeviceTrackingInitCounter_CoreAudio == 0) {
35491 |             AudioObjectPropertyAddress propAddress;
35492 |             propAddress.mScope = kAudioObjectPropertyScopeGlobal;
35493 |             propAddress.mElement = AUDIO_OBJECT_PROPERTY_ELEMENT;
35494 |
35495 |             propAddress.mSelector = kAudioHardwarePropertyDefaultInputDevice;
35496 |             ((ma_AudioObjectRemovePropertyListener_proc)pContext->coreaudio.AudioObjectRemovePropert
> yListener)(kAudioObjectSystemObject, &propAddress, &ma_default_device_changed__coreaudio, NULL);
35497 |
35498 |             propAddress.mSelector = kAudioHardwarePropertyDefaultOutputDevice;
35499 |             ((ma_AudioObjectRemovePropertyListener_proc)pContext->coreaudio.AudioObjectRemovePropert
> yListener)(kAudioObjectSystemObject, &propAddress, &ma_default_device_changed__coreaudio, NULL);
35500 |
35501 |             /* At this point there should be no tracked devices. If not there's an error somewhere.
> */
35502 |             if (g_ppTrackedDevices_CoreAudio != NULL) {
35503 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "You have
> uninitialized all contexts while an associated device is still active.");
35504 |                 ma_spinlock_unlock(&g_DeviceTrackingInitLock_CoreAudio);
35505 |                 return MA_INVALID_OPERATION;
35506 |             }
35507 |
35508 |             ma_mutex_uninit(&g_DeviceTrackingMutex_CoreAudio);
35509 |         }
35510 |     }
35511 |     ma_spinlock_unlock(&g_DeviceTrackingInitLock_CoreAudio);
35512 |
35513 |     return MA_SUCCESS;
35514 | }
35515 |
35516 | static ma_result ma_device__track__coreaudio(ma_device* pDevice)
35517 | {
35518 |     MA_ASSERT(pDevice != NULL);
35519 |
35520 |     ma_mutex_lock(&g_DeviceTrackingMutex_CoreAudio);
35521 |     {
35522 |         /* Allocate memory if required. */
35523 |         if (g_TrackedDeviceCap_CoreAudio <= g_TrackedDeviceCount_CoreAudio) {
35524 |             ma_uint32 newCap;
35525 |             ma_device** ppNewDevices;
35526 |
35527 |             newCap = g_TrackedDeviceCap_CoreAudio * 2;
35528 |             if (newCap == 0) {
35529 |                 newCap = 1;
35530 |             }
35531 |
35532 |             ppNewDevices = (ma_device**)ma_realloc(g_ppTrackedDevices_CoreAudio,
> sizeof(*g_ppTrackedDevices_CoreAudio)*newCap, &pDevice->pContext->allocationCallbacks);
35533 |             if (ppNewDevices == NULL) {
35534 |                 ma_mutex_unlock(&g_DeviceTrackingMutex_CoreAudio);
35535 |                 return MA_OUT_OF_MEMORY;
35536 |             }

```

```

35537 |
35538 |         g_ppTrackedDevices_CoreAudio = ppNewDevices;
35539 |         g_TrackedDeviceCap_CoreAudio = newCap;
35540 |     }
35541 |
35542 |     g_ppTrackedDevices_CoreAudio[g_TrackedDeviceCount_CoreAudio] = pDevice;
35543 |     g_TrackedDeviceCount_CoreAudio += 1;
35544 | }
35545 | ma_mutex_unlock(&g_DeviceTrackingMutex_CoreAudio);
35546 |
35547 | return MA_SUCCESS;
35548 | }
35549 |
35550 | static ma_result ma_device__untrack__coreaudio(ma_device* pDevice)
35551 | {
35552 |     MA_ASSERT(pDevice != NULL);
35553 |
35554 |     ma_mutex_lock(&g_DeviceTrackingMutex_CoreAudio);
35555 |     {
35556 |         ma_uint32 iDevice;
35557 |         for (iDevice = 0; iDevice < g_TrackedDeviceCount_CoreAudio; iDevice += 1) {
35558 |             if (g_ppTrackedDevices_CoreAudio[iDevice] == pDevice) {
35559 |                 /* We've found the device. We now need to remove it from the list. */
35560 |                 ma_uint32 jDevice;
35561 |                 for (jDevice = iDevice; jDevice < g_TrackedDeviceCount_CoreAudio-1; jDevice += 1) {
35562 |                     g_ppTrackedDevices_CoreAudio[jDevice] = g_ppTrackedDevices_CoreAudio[jDevice+1];
35563 |                 }
35564 |
35565 |                 g_TrackedDeviceCount_CoreAudio -= 1;
35566 |
35567 |                 /* If there's nothing else in the list we need to free memory. */
35568 |                 if (g_TrackedDeviceCount_CoreAudio == 0) {
35569 |                     ma_free(g_ppTrackedDevices_CoreAudio, &pDevice->pContext->allocationCallbacks);
35570 |                     g_ppTrackedDevices_CoreAudio = NULL;
35571 |                     g_TrackedDeviceCap_CoreAudio = 0;
35572 |                 }
35573 |
35574 |                 break;
35575 |             }
35576 |         }
35577 |     }
35578 |     ma_mutex_unlock(&g_DeviceTrackingMutex_CoreAudio);
35579 |
35580 |     return MA_SUCCESS;
35581 | }
35582 | #endif
35583 |
35584 | #if defined(MA_APPLE_MOBILE)
35585 | @interface ma_ios_notification_handler:NSObject {
35586 |     ma_device* m_pDevice;
35587 | }
35588 | @end
35589 |
35590 | @implementation ma_ios_notification_handler
35591 | -(id)init:(ma_device*)pDevice
35592 | {
35593 |     self = [super init];
35594 |     m_pDevice = pDevice;
35595 |
35596 |     /* For route changes. */
35597 |     [[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(handle_route_change:)
35598 | > name:AVAAudioSessionRouteChangeNotification object:[AVAAudioSession sharedInstance]];
35599 |
35600 |     /* For interruptions. */
35601 |     [[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(handle_interruption:)
35602 | > name:AVAAudioSessionInterruptionNotification object:[AVAAudioSession sharedInstance]];
35603 |
35604 |     return self;
35605 | }
35606 |
35607 | -(void)dealloc
35608 | {
35609 |     [self remove_handler];
35610 |
35611 |     #if defined(__has_feature)
35612 |     #if !__has_feature(objc_arc)
35613 |         [super dealloc];
35614 |     #endif
35615 | #endif

```

```

35614 | }
35615 |
35616 | -(void)remove_handler
35617 | {
35618 |     [[NSNotificationCenter defaultCenter] removeObserver:self
> name:AVAudioSessionRouteChangeNotification object:nil];
35619 |     [[NSNotificationCenter defaultCenter] removeObserver:self
> name:AVAudioSessionInterruptionNotification object:nil];
35620 | }
35621 |
35622 | -(void)handle_interruption:(NSNotification*)pNotification
35623 | {
35624 |     NSInteger type = [[[pNotification userInfo] objectForKey:AVAudioSessionInterruptionTypeKey]
> integerValue];
35625 |     switch (type)
35626 |     {
35627 |         case AVAudioSessionInterruptionTypeBegan:
35628 |         {
35629 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio]
> Interruption: AVAudioSessionInterruptionTypeBegan\n");
35630 |
35631 |             /*
35632 |              Core Audio will have stopped the internal device automatically, but we need explicitly
35633 |              stop it at a higher level to ensure miniaudio-specific state is updated for consistency.
35634 |              */
35635 |             ma_device_stop(m_pDevice);
35636 |
35637 |             /*
35638 |              Fire the notification after the device has been stopped to ensure it's in the correct
35639 |              state when the notification handler is invoked.
35640 |              */
35641 |             ma_device__on_notification_interruption_began(m_pDevice);
35642 |         } break;
35643 |
35644 |         case AVAudioSessionInterruptionTypeEnded:
35645 |         {
35646 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio]
> Interruption: AVAudioSessionInterruptionTypeEnded\n");
35647 |             ma_device__on_notification_interruption_ended(m_pDevice);
35648 |         } break;
35649 |     }
35650 | }
35651 |
35652 | -(void)handle_route_change:(NSNotification*)pNotification
35653 | {
35654 |     AVAudioSession* pSession = [AVAudioSession sharedInstance];
35655 |
35656 |     NSInteger reason = [[[pNotification userInfo] objectForKey:AVAudioSessionRouteChangeReasonKey]
> integerValue];
35657 |     switch (reason)
35658 |     {
35659 |         case AVAudioSessionRouteChangeReasonOldDeviceUnavailable:
35660 |         {
35661 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonOldDeviceUnavailable\n");
35662 |         } break;
35663 |
35664 |         case AVAudioSessionRouteChangeReasonNewDeviceAvailable:
35665 |         {
35666 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonNewDeviceAvailable\n");
35667 |         } break;
35668 |
35669 |         case AVAudioSessionRouteChangeReasonNoSuitableRouteForCategory:
35670 |         {
35671 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonNoSuitableRouteForCategory\n");
35672 |         } break;
35673 |
35674 |         case AVAudioSessionRouteChangeReasonWakeFromSleep:
35675 |         {
35676 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonWakeFromSleep\n");
35677 |         } break;
35678 |
35679 |         case AVAudioSessionRouteChangeReasonOverride:
35680 |         {
35681 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonOverride\n");

```

```

35682 |         } break;
35683 |
35684 |         case AVAudioSessionRouteChangeReasonCategoryChange:
35685 |         {
35686 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonCategoryChange\n");
35687 |         } break;
35688 |
35689 |         case AVAudioSessionRouteChangeReasonUnknown:
35690 |         default:
35691 |         {
35692 |             ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_INFO, "[Core Audio] Route
> Changed: AVAudioSessionRouteChangeReasonUnknown\n");
35693 |         } break;
35694 |     }
35695 |
35696 |     ma_log_postf(ma_device_get_log(m_pDevice), MA_LOG_LEVEL_DEBUG, "[Core Audio] Changing Route.
> inputNumberChannels=%d; outputNumberOfChannels=%d\n", (int)pSession.inputNumberOfChannels,
> (int)pSession.outputNumberOfChannels);
35697 |
35698 |     /* Let the application know about the route change. */
35699 |     ma_device__on_notification_rerouted(m_pDevice);
35700 | }
35701 | @end
35702 | #endif
35703 |
35704 | static ma_result ma_device_uninit__coreaudio(ma_device* pDevice)
35705 | {
35706 |     MA_ASSERT(pDevice != NULL);
35707 |     MA_ASSERT(ma_device_get_state(pDevice) == ma_device_state_uninitialized);
35708 |
35709 |     #if defined(MA_APPLE_DESKTOP)
35710 |     /*
35711 |     > Make sure we're no longer tracking the device. It doesn't matter if we call this for a
> non-default device because it'll
35712 |     just gracefully ignore it.
35713 |     */
35714 |     ma_device__untrack__coreaudio(pDevice);
35715 |     #endif
35716 |     #if defined(MA_APPLE_MOBILE)
35717 |     if (pDevice->coreaudio.pNotificationHandler != NULL) {
35718 |         ma_ios_notification_handler* pNotificationHandler = (MA_BRIDGE_TRANSFER
> ma_ios_notification_handler*)pDevice->coreaudio.pNotificationHandler;
35719 |         [pNotificationHandler remove_handler];
35720 |     }
35721 |     #endif
35722 |
35723 |     if (pDevice->coreaudio.audioUnitCapture != NULL) {
35724 |         ((ma_AudioComponentInstanceDispose_proc)pDevice->pContext->coreaudio.AudioComponentInstanceD
> ispose)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
35725 |     }
35726 |     if (pDevice->coreaudio.audioUnitPlayback != NULL) {
35727 |         ((ma_AudioComponentInstanceDispose_proc)pDevice->pContext->coreaudio.AudioComponentInstanceD
> ispose)((AudioUnit)pDevice->coreaudio.audioUnitPlayback);
35728 |     }
35729 |
35730 |     if (pDevice->coreaudio.pAudioBufferList) {
35731 |         ma_free(pDevice->coreaudio.pAudioBufferList, &pDevice->pContext->allocationCallbacks);
35732 |     }
35733 |
35734 |     return MA_SUCCESS;
35735 | }
35736 |
35737 | typedef struct
35738 | {
35739 |     ma_bool32 allowNominalSampleRateChange;
35740 |
35741 |     /* Input. */
35742 |     ma_format formatIn;
35743 |     ma_uint32 channelsIn;
35744 |     ma_uint32 sampleRateIn;
35745 |     ma_channel channelMapIn[MA_MAX_CHANNELS];
35746 |     ma_uint32 periodSizeInFramesIn;
35747 |     ma_uint32 periodSizeInMillisecondsIn;
35748 |     ma_uint32 periodsIn;
35749 |     ma_share_mode shareMode;
35750 |     ma_performance_profile performanceProfile;
35751 |     ma_bool32 registerStopEvent;
35752 | }

```



```

35753 | /* Output. */
35754 | #if defined(MA_APPLE_DESKTOP)
35755 |     AudioObjectID deviceObjectID;
35756 | #endif
35757 |     AudioComponent component;
35758 |     AudioUnit audioUnit;
35759 |     AudioBufferList* pAudioBufferList; /* Only used for input devices. */
35760 |     ma_format formatOut;
35761 |     ma_uint32 channelsOut;
35762 |     ma_uint32 sampleRateOut;
35763 |     ma_channel channelMapOut[MA_MAX_CHANNELS];
35764 |     ma_uint32 periodSizeInFramesOut;
35765 |     ma_uint32 periodsOut;
35766 |     char deviceName[256];
35767 | } ma_device_init_internal_data__coreaudio;
35768 |
35769 | static ma_result ma_device_init_internal__coreaudio(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_init_internal_data__coreaudio* pData, void*
> pDevice_DoNotReference) /* <-- pDevice is typed as void* intentionally so as to avoid accidentally
> referencing it. */
35770 | {
35771 |     ma_result result = MA_SUCCESS;
35772 |     OSStatus status;
35773 |     UInt32 enableIOFlag;
35774 |     AudioStreamBasicDescription bestFormat;
35775 |     ma_uint32 actualPeriodSizeInFrames;
35776 |     AURenderCallbackStruct callbackInfo;
35777 | #if defined(MA_APPLE_DESKTOP)
35778 |     AudioObjectID deviceObjectID;
35779 | #endif
35780 |
35781 |     /* This API should only be used for a single device type: playback or capture. No full-duplex
> mode. */
35782 |     if (deviceType == ma_device_type_duplex) {
35783 |         return MA_INVALID_ARGS;
35784 |     }
35785 |
35786 |     MA_ASSERT(pContext != NULL);
35787 |     MA_ASSERT(deviceType == ma_device_type_playback || deviceType == ma_device_type_capture);
35788 |
35789 | #if defined(MA_APPLE_DESKTOP)
35790 |     pData->deviceObjectID = 0;
35791 | #endif
35792 |     pData->component = NULL;
35793 |     pData->audioUnit = NULL;
35794 |     pData->pAudioBufferList = NULL;
35795 |
35796 | #if defined(MA_APPLE_DESKTOP)
35797 |     result = ma_find_AudioObjectID(pContext, deviceType, pDeviceID, &deviceObjectID);
35798 |     if (result != MA_SUCCESS) {
35799 |         return result;
35800 |     }
35801 |
35802 |     pData->deviceObjectID = deviceObjectID;
35803 | #endif
35804 |
35805 |     /* Core audio doesn't really use the notion of a period so we can leave this unmodified, but not
> too over the top. */
35806 |     pData->periodsOut = pData->periodsIn;
35807 |     if (pData->periodsOut == 0) {
35808 |         pData->periodsOut = MA_DEFAULT_PERIODS;
35809 |     }
35810 |     if (pData->periodsOut > 16) {
35811 |         pData->periodsOut = 16;
35812 |     }
35813 |
35814 |
35815 |     /* Audio unit. */
35816 |     status = ((ma_AudioComponentInstanceNew_proc)pContext->coreaudio.AudioComponentInstanceNew)((Aud
> ioComponent)pContext->coreaudio.component, (AudioUnit*)&pData->audioUnit);
35817 |     if (status != noErr) {
35818 |         return ma_result_from_OSStatus(status);
35819 |     }
35820 |
35821 |
35822 |     /* The input/output buses need to be explicitly enabled and disabled. We set the flag based on
> the output unit first, then we just swap it for input. */
35823 |     enableIOFlag = 1;
35824 |     if (deviceType == ma_device_type_capture) {

```

```

35825 |         enableIOFlag = 0;
35826 |     }
35827 |
35828 |     status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioOutputUnitProperty_EnableIO, kAudioUnitScope_Output, MA_COREAUDIO_OUTPUT_BUS, &enableIOFlag,
> sizeof(enableIOFlag));
35829 |     if (status != noErr) {
35830 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispose)(p
> Data->audioUnit);
35831 |         return ma_result_from_OSStatus(status);
35832 |     }
35833 |
35834 |     enableIOFlag = (enableIOFlag == 0) ? 1 : 0;
35835 |     status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioOutputUnitProperty_EnableIO, kAudioUnitScope_Input, MA_COREAUDIO_INPUT_BUS, &enableIOFlag,
> sizeof(enableIOFlag));
35836 |     if (status != noErr) {
35837 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispose)(p
> Data->audioUnit);
35838 |         return ma_result_from_OSStatus(status);
35839 |     }
35840 |
35841 |
35842 |     /* Set the device to use with this audio unit. This is only used on desktop since we are using
> defaults on mobile. */
35843 |     #if defined(MA_APPLE_DESKTOP)
35844 |     status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioOutputUnitProperty_CurrentDevice, kAudioUnitScope_Global, 0, &deviceObjectID,
> sizeof(deviceObjectID));
35845 |     if (status != noErr) {
35846 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispose)(p
> Data->audioUnit);
35847 |         return ma_result_from_OSStatus(result);
35848 |     }
35849 |     #else
35850 |     /*
35851 |     For some reason it looks like Apple is only allowing selection of the input device. There does
> not appear to be any way to change
35852 |     the default output route. I have no idea why this is like this, but for now we'll only be able
> to configure capture devices.
35853 |     */
35854 |     if (pDeviceID != NULL) {
35855 |         if (deviceType == ma_device_type_capture) {
35856 |             ma_bool32 found = MA_FALSE;
35857 |             NSArray *pInputs = [[[AVAAudioSession sharedInstance] currentRoute] inputs];
35858 |             for (AVAAudioSessionPortDescription* pPortDesc in pInputs) {
35859 |                 if (strcmp(pDeviceID->coreaudio, [pPortDesc.UUID UTF8String]) == 0) {
35860 |                     [[AVAAudioSession sharedInstance] setPreferredInput:pPortDesc error:nil];
35861 |                     found = MA_TRUE;
35862 |                     break;
35863 |                 }
35864 |             }
35865 |
35866 |             if (found == MA_FALSE) {
35867 |                 return MA_DOES_NOT_EXIST;
35868 |             }
35869 |         }
35870 |     }
35871 |     #endif
35872 |
35873 |     /*
35874 |     Format. This is the hardest part of initialization because there's a few variables to take into
> account.
35875 |     1) The format must be supported by the device.
35876 |     2) The format must be supported miniaudio.
35877 |     3) There's a priority that miniaudio prefers.
35878 |
35879 |     Ideally we would like to use a format that's as close to the hardware as possible so we can get
> as close to a passthrough as possible. The
35880 |     most important property is the sample rate. miniaudio can do format conversion for any sample
> rate and channel count, but cannot do the same
35881 |     for the sample data format. If the sample data format is not supported by miniaudio it must be
> ignored completely.
35882 |
35883 |     On mobile platforms this is a bit different. We just force the use of whatever the audio unit's
> current format is set to.

```

```

35884 |     */
35885 |     {
35886 |         AudioStreamBasicDescription origFormat;
35887 |         UInt32 origFormatSize = sizeof(origFormat);
35888 |         AudioUnitScope formatScope = (deviceType == ma_device_type_playback) ?
> kAudioUnitScope_Input : kAudioUnitScope_Output;
35889 |         AudioUnitElement formatElement = (deviceType == ma_device_type_playback) ?
> MA_COREAUDIO_OUTPUT_BUS : MA_COREAUDIO_INPUT_BUS;
35890 |
35891 |         if (deviceType == ma_device_type_playback) {
35892 |             status =
> ((ma_AudioUnitGetProperty_proc)pContext->coreaudio.AudioUnitGetProperty)(pData->audioUnit,
> kAudioUnitProperty_StreamFormat, kAudioUnitScope_Output, MA_COREAUDIO_OUTPUT_BUS, &origFormat,
> &origFormatSize);
35893 |         } else {
35894 |             status =
> ((ma_AudioUnitGetProperty_proc)pContext->coreaudio.AudioUnitGetProperty)(pData->audioUnit,
> kAudioUnitProperty_StreamFormat, kAudioUnitScope_Input, MA_COREAUDIO_INPUT_BUS, &origFormat,
> &origFormatSize);
35895 |         }
35896 |         if (status != noErr) {
35897 |             ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
35898 |             return ma_result_from_OSStatus(status);
35899 |         }
35900 |
35901 |         #if defined(MA_APPLE_DESKTOP)
35902 |             result = ma_find_best_format_coreaudio(pContext, deviceObjectID, deviceType,
> pData->formatIn, pData->channelsIn, pData->sampleRateIn, &origFormat, &bestFormat);
35903 |             if (result != MA_SUCCESS) {
35904 |                 ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
35905 |                 return result;
35906 |             }
35907 |
35908 |             /*
35909 |             Technical Note TN2091: Device input using the HAL Output Audio Unit
35910 |             https://developer.apple.com/library/archive/technotes/tn2091/\_index.html
35911 |
35912 |             This documentation says the following:
35913 |
35914 |             The internal AudioConverter can handle any *simple* conversion. Typically, this means
> that a client can specify ANY
35915 |             variant of the PCM formats. Consequently, the device's sample rate should match the
> desired sample rate. If sample rate
35916 |             conversion is needed, it can be accomplished by buffering the input and converting the
> data on a separate thread with
35917 |             another AudioConverter.
35918 |
35919 |             The important part here is the mention that it can handle *simple* conversions, which does
> *not* include sample rate. We
35920 |             therefore want to ensure the sample rate stays consistent. This document is specifically for
> input, but I'm going to play it
35921 |             safe and apply the same rule to output as well.
35922 |
35923 |             I have tried going against the documentation by setting the sample rate anyway, but this
> just results in AudioUnitRender()
35924 |             returning a result code of -10863. I have also tried changing the format directly on the
> input scope on the input bus, but
35925 |             this just results in `ca_require: IsStreamFormatWritable(inScope, inElement) NotWritable`
> when trying to set the format.
35926 |
35927 |             Something that does seem to work, however, has been setting the nominal sample rate on the
> device object. The problem with
35928 |             this, however, is that it actually changes the sample rate at the operating system level and
> not just the application. This
35929 |             could be intrusive to the user, however, so I don't think it's wise to make this the
> default. Instead I'm making this a
35930 |             configuration option. When the `coreaudio.allowNominalSampleRateChange` config option is set
> to true, changing the sample
35931 |             rate will be allowed. Otherwise it'll be fixed to the current sample rate. To check the
> system-defined sample rate, run
35932 |             the Audio MIDI Setup program that comes installed on macOS and observe how the sample rate
> changes as the sample rate is
35933 |             changed by miniaudio.
35934 |             */
35935 |             if (pData->allowNominalSampleRateChange) {
35936 |                 AudioValueRange sampleRateRange;
35937 |                 AudioObjectPropertyAddress propAddress;

```

```

35938 |
35939 |         sampleRateRange.mMinimum = bestFormat.mSampleRate;
35940 |         sampleRateRange.mMaximum = bestFormat.mSampleRate;
35941 |
35942 |         propAddress.mSelector = kAudioDevicePropertyNominalSampleRate;
35943 |         propAddress.mScope    = (deviceType == ma_device_type_playback) ?
> kAudioObjectPropertyScopeOutput : kAudioObjectPropertyScopeInput;
35944 |         propAddress.mElement  = AUDIO_OBJECT_PROPERTY_ELEMENT;
35945 |
35946 |         status =
> ((ma_AudioObjectSetPropertyData_proc)pContext->coreaudio.AudioObjectSetPropertyData)(deviceObjectID,
> &propAddress, 0, NULL, sizeof(sampleRateRange), &sampleRateRange);
35947 |         if (status != noErr) {
35948 |             bestFormat.mSampleRate = origFormat.mSampleRate;
35949 |         }
35950 |     } else {
35951 |         bestFormat.mSampleRate = origFormat.mSampleRate;
35952 |     }
35953 |
35954 |     status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioUnitProperty_StreamFormat, formatScope, formatElement, &bestFormat, sizeof(bestFormat));
35955 |     if (status != noErr) {
35956 |         /* We failed to set the format, so fall back to the current format of the audio unit. */
35957 |         bestFormat = origFormat;
35958 |     }
35959 |     #else
35960 |         bestFormat = origFormat;
35961 |
35962 |         /*
35963 |         Sample rate is a little different here because for some reason
> kAudioUnitProperty_StreamFormat returns 0... Oh well. We need to instead try
35964 |         setting the sample rate to what the user has requested and then just see the results of it.
> Need to use some Objective-C here for this since
35965 |         it depends on Apple's AVAudioSession API. To do this we just get the shared AVAudioSession
> instance and then set it. Note that from what I
35966 |         can tell, it looks like the sample rate is shared between playback and capture for
> everything.
35967 |         */
35968 |         @autoreleasepool {
35969 |             AVAudioSession* pAudioSession = [AVAudioSession sharedInstance];
35970 |             MA_ASSERT(pAudioSession != NULL);
35971 |
35972 |             [pAudioSession setPreferredSampleRate:(double)pData->sampleRateIn error:nil];
35973 |             bestFormat.mSampleRate = pAudioSession.sampleRate;
35974 |
35975 |             /*
35976 |             I've had a report that the channel count returned by AudioUnitGetProperty above is
> inconsistent with
35977 |             AVAudioSession outputNumberOfChannels. I'm going to try using the AVAudioSession values
> instead.
35978 |
35979 |             UPDATE 20/02/2025:
35980 |             When testing on the simulator with an iPhone 15 and iOS 17 I get an error when
> initializing the audio
35981 |             unit if set the input channels to pAudioSession.inputNumberOfChannels. What is happening
> is the channel
35982 |             count returned from AudioUnitGetProperty() above is set to 2, but pAudioSession is
> reporting a channel
35983 |             count of 1. When this happens, the call to AudioUnitSetProprty() below just down below
> will succeed, but
35984 |             AudioUnitInitialize() further down will fail. The only solution I have come up with is
> to not set the
35985 |             channel count to pAudioSession.inputNumberOfChannels.
35986 |             */
35987 |             if (deviceType == ma_device_type_playback) {
35988 |                 bestFormat.mChannelsPerFrame = (UInt32)pAudioSession.outputNumberOfChannels;
35989 |             }
35990 |
35991 |             #if 0
35992 |             if (deviceType == ma_device_type_capture) {
35993 |                 /*printf("DEBUG: bestFormat.mChannelsPerFrame = %d;
> pAudioSession.inputNumberOfChannels = %d\n", (int)bestFormat.mChannelsPerFrame,
> (int)pAudioSession.inputNumberOfChannels);*/
35994 |                 bestFormat.mChannelsPerFrame = (UInt32)pAudioSession.inputNumberOfChannels;
35995 |             }
35996 |             #endif
35997 |         }
35998 |

```

```

35999 |
36000 |         status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioUnitProperty_StreamFormat, formatScope, formatElement, &bestFormat, sizeof(bestFormat));
36001 |         if (status != noErr) {
36002 |             ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
36003 |             return ma_result_from_OSStatus(status);
36004 |         }
36005 |     #endif
36006 |
36007 |     result = ma_format_from_AudioStreamBasicDescription(&bestFormat, &pData->formatOut);
36008 |     if (result != MA_SUCCESS) {
36009 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
36010 |         return result;
36011 |     }
36012 |
36013 |     if (pData->formatOut == ma_format_unknown) {
36014 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
36015 |         return MA_FORMAT_NOT_SUPPORTED;
36016 |     }
36017 |
36018 |     pData->channelsOut = bestFormat.mChannelsPerFrame;
36019 |     pData->sampleRateOut = (ma_uint32)bestFormat.mSampleRate;
36020 | }
36021 |
36022 | /* Clamp the channel count for safety. */
36023 | if (pData->channelsOut > MA_MAX_CHANNELS) {
36024 |     pData->channelsOut = MA_MAX_CHANNELS;
36025 | }
36026 |
36027 | /*
36028 | Internal channel map. This is weird in my testing. If I use the AudioObject to get the
36029 | channel map, the channel descriptions are set to "Unknown" for some reason. To work around
36030 | this it looks like retrieving it from the AudioUnit will work. However, and this is where
36031 | it gets weird, it doesn't seem to work with capture devices, nor at all on iOS... Therefore
36032 | I'm going to fall back to a default assumption in these cases.
36033 | */
36034 | #if defined(MA_APPLE_DESKTOP)
36035 |     result = ma_get_AudioUnit_channel_map(pContext, pData->audioUnit, deviceType,
> pData->channelMapOut, pData->channelsOut);
36036 |     if (result != MA_SUCCESS) {
36037 |         #if 0
36038 |             /* Try falling back to the channel map from the AudioObject. */
36039 |             result = ma_get_AudioObject_channel_map(pContext, deviceObjectID, deviceType,
> pData->channelMapOut, pData->channelsOut);
36040 |             if (result != MA_SUCCESS) {
36041 |                 return result;
36042 |             }
36043 |         #else
36044 |             /* Fall back to default assumptions. */
36045 |             ma_channel_map_init_standard(ma_standard_channel_map_default, pData->channelMapOut,
> ma_countof(pData->channelMapOut), pData->channelsOut);
36046 |         #endif
36047 |     }
36048 | #else
36049 |     /* TODO: Figure out how to get the channel map using AVAudioSession. */
36050 |     ma_channel_map_init_standard(ma_standard_channel_map_default, pData->channelMapOut,
> ma_countof(pData->channelMapOut), pData->channelsOut);
36051 | #endif
36052 |
36053 |
36054 | /* Buffer size. Not allowing this to be configurable on iOS. */
36055 | if (pData->periodSizeInFramesIn == 0) {
36056 |     if (pData->periodSizeInMillisecondsIn == 0) {
36057 |         if (pData->performanceProfile == ma_performance_profile_low_latency) {
36058 |             actualPeriodSizeInFrames = ma_calculate_buffer_size_in_frames_from_milliseconds(MA_D
> EFAULT_PERIOD_SIZE_IN_MILLISECONDS_LOW_LATENCY, pData->sampleRateOut);
36059 |         } else {
36060 |             actualPeriodSizeInFrames = ma_calculate_buffer_size_in_frames_from_milliseconds(MA_D
> EFAULT_PERIOD_SIZE_IN_MILLISECONDS_CONSERVATIVE, pData->sampleRateOut);
36061 |         }
36062 |     } else {
36063 |         actualPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_milliseconds(pData->periodSizeInMillisecondsIn,
> pData->sampleRateOut);
36064 |     }

```

```

36065 |     } else {
36066 |         actualPeriodSizeInFrames = pData->periodSizeInFramesIn;
36067 |     }
36068 |
36069 | #if defined(MA_APPLE_DESKTOP)
36070 |     result = ma_set_AudioObject_buffer_size_in_frames(pContext, deviceObjectID, deviceType,
> &actualPeriodSizeInFrames);
36071 |     if (result != MA_SUCCESS) {
36072 |         return result;
36073 |     }
36074 | #else
36075 |     /*
36076 |     On iOS, the size of the IO buffer needs to be specified in seconds and is a floating point
36077 |     number. I don't trust any potential truncation errors due to converting from float to integer
36078 |     so I'm going to explicitly set the actual period size to the next power of 2.
36079 |     */
36080 |     @autoreleasepool {
36081 |         AVAudioSession* pAudioSession = [AVAudioSession sharedInstance];
36082 |         MA_ASSERT(pAudioSession != NULL);
36083 |
36084 |         [pAudioSession setPreferredIOBufferDuration:((float)actualPeriodSizeInFrames /
> pAudioSession.sampleRate) error:nil];
36085 |         actualPeriodSizeInFrames = ma_next_power_of_2((ma_uint32)(pAudioSession.IOBufferDuration *
> pAudioSession.sampleRate));
36086 |     }
36087 | #endif
36088 |
36089 |
36090 |     /*
36091 |     During testing I discovered that the buffer size can be too big. You'll get an error like this:
36092 |
36093 |         kAudioUnitErr_TooManyFramesToProcess : inFramesToProcess=4096, mMaxFramesPerSlice=512
36094 |
36095 |     Note how inFramesToProcess is smaller than mMaxFramesPerSlice. To fix, we need to set
> kAudioUnitProperty_MaximumFramesPerSlice to that
36096 |     of the size of our buffer, or do it the other way around and set our buffer size to the
> kAudioUnitProperty_MaximumFramesPerSlice.
36097 |     */
36098 |     status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioUnitProperty_MaximumFramesPerSlice, kAudioUnitScope_Global, 0, &actualPeriodSizeInFrames,
> sizeof(actualPeriodSizeInFrames));
36099 |     if (status != noErr) {
36100 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispose)(p
> Data->audioUnit);
36101 |         return ma_result_from_OSStatus(status);
36102 |     }
36103 |
36104 |     pData->periodSizeInFramesOut = (ma_uint32)actualPeriodSizeInFrames;
36105 |
36106 |     /* We need a buffer list if this is an input device. We render into this in the input callback.
> */
36107 |     if (deviceType == ma_device_type_capture) {
36108 |         ma_bool32 isInterleaved = (bestFormat.mFormatFlags & kAudioFormatFlagIsNonInterleaved) == 0;
36109 |         AudioBufferList* pBufferList;
36110 |
36111 |         pBufferList = ma_allocate_AudioBufferList__coreaudio(pData->periodSizeInFramesOut,
> pData->formatOut, pData->channelsOut, (isInterleaved) ? ma_stream_layout_interleaved :
> ma_stream_layout_deinterleaved, &pContext->allocationCallbacks);
36112 |         if (pBufferList == NULL) {
36113 |             ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
36114 |             return MA_OUT_OF_MEMORY;
36115 |         }
36116 |
36117 |         pData->pAudioBufferList = pBufferList;
36118 |     }
36119 |
36120 |     /* Callbacks. */
36121 |     callbackInfo.inputProcRefCon = pDevice_DoNotReference;
36122 |     if (deviceType == ma_device_type_playback) {
36123 |         callbackInfo.inputProc = ma_on_output__coreaudio;
36124 |         status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioUnitProperty_SetRenderCallback, kAudioUnitScope_Global, 0, &callbackInfo,
> sizeof(callbackInfo));
36125 |         if (status != noErr) {
36126 |             ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);

```

```

36127 |         return ma_result_from_OSStatus(status);
36128 |     }
36129 | } else {
36130 |     callbackInfo.inputProc = ma_on_input_coreaudio;
36131 |     status =
> ((ma_AudioUnitSetProperty_proc)pContext->coreaudio.AudioUnitSetProperty)(pData->audioUnit,
> kAudioOutputUnitProperty_SetInputCallback, kAudioUnitScope_Global, 0, &callbackInfo,
> sizeof(callbackInfo));
36132 |     if (status != noErr) {
36133 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
36134 |         return ma_result_from_OSStatus(status);
36135 |     }
36136 | }
36137 |
36138 | /* We need to listen for stop events. */
36139 | if (pData->registerStopEvent) {
36140 |     status = ((ma_AudioUnitAddPropertyListener_proc)pContext->coreaudio.AudioUnitAddPropertyList
> ener)(pData->audioUnit, kAudioOutputUnitProperty_IsRunning, on_start_stop_coreaudio,
> pDevice_DoNotReference);
36141 |     if (status != noErr) {
36142 |         ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispos
> e)(pData->audioUnit);
36143 |         return ma_result_from_OSStatus(status);
36144 |     }
36145 | }
36146 |
36147 | /* Initialize the audio unit. */
36148 | status =
> ((ma_AudioUnitInitialize_proc)pContext->coreaudio.AudioUnitInitialize)(pData->audioUnit);
36149 | if (status != noErr) {
36150 |     ma_free(pData->pAudioBufferList, &pContext->allocationCallbacks);
36151 |     pData->pAudioBufferList = NULL;
36152 |     ((ma_AudioComponentInstanceDispose_proc)pContext->coreaudio.AudioComponentInstanceDispose)(p
> Data->audioUnit);
36153 |     return ma_result_from_OSStatus(status);
36154 | }
36155 |
36156 | /* Grab the name. */
36157 | #if defined(MA_APPLE_DESKTOP)
36158 |     ma_get_AudioObject_name(pContext, deviceObjectID, sizeof(pData->deviceName), pData->deviceName);
36159 | #else
36160 |     if (deviceType == ma_device_type_playback) {
36161 |         ma_strcpy_s(pData->deviceName, sizeof(pData->deviceName), MA_DEFAULT_PLAYBACK_DEVICE_NAME);
36162 |     } else {
36163 |         ma_strcpy_s(pData->deviceName, sizeof(pData->deviceName), MA_DEFAULT_CAPTURE_DEVICE_NAME);
36164 |     }
36165 | #endif
36166 |
36167 |     return result;
36168 | }
36169 |
36170 | #if defined(MA_APPLE_DESKTOP)
36171 | static ma_result ma_device_reinit_internal_coreaudio(ma_device* pDevice, ma_device_type deviceType,
> ma_bool32 disposePreviousAudioUnit)
36172 | {
36173 |     ma_device_init_internal_data_coreaudio data;
36174 |     ma_result result;
36175 |
36176 |     /* This should only be called for playback or capture, not duplex. */
36177 |     if (deviceType == ma_device_type_duplex) {
36178 |         return MA_INVALID_ARGS;
36179 |     }
36180 |
36181 |     data.allowNominalSampleRateChange = MA_FALSE; /* Don't change the nominal sample rate when
> switching devices. */
36182 |
36183 |     if (deviceType == ma_device_type_capture) {
36184 |         data.formatIn = pDevice->capture.format;
36185 |         data.channelsIn = pDevice->capture.channels;
36186 |         data.sampleRateIn = pDevice->sampleRate;
36187 |         MA_COPY_MEMORY(data.channelMapIn, pDevice->capture.channelMap,
> sizeof(pDevice->capture.channelMap));
36188 |         data.shareMode = pDevice->capture.shareMode;
36189 |         data.performanceProfile = pDevice->coreaudio.originalPerformanceProfile;
36190 |         data.registerStopEvent = MA_TRUE;
36191 |
36192 |         if (disposePreviousAudioUnit) {
36193 |             ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.AudioOutputUnitStop)((AudioUn

```



```

> it)pDevice->coreaudio.audioUnitCapture);
36194 |         ((ma_AudioComponentInstanceDispose_proc)pDevice->pContext->coreaudio.AudioComponentInsta
> nceDispose)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
36195 |     }
36196 |     if (pDevice->coreaudio.pAudioBufferList) {
36197 |         ma_free(pDevice->coreaudio.pAudioBufferList, &pDevice->pContext->allocationCallbacks);
36198 |     }
36199 | } else if (deviceType == ma_device_type_playback) {
36200 |     data.formatIn = pDevice->playback.format;
36201 |     data.channelsIn = pDevice->playback.channels;
36202 |     data.sampleRateIn = pDevice->sampleRate;
36203 |     MA_COPY_MEMORY(data.channelMapIn, pDevice->playback.channelMap,
> sizeof(pDevice->playback.channelMap));
36204 |     data.shareMode = pDevice->playback.shareMode;
36205 |     data.performanceProfile = pDevice->coreaudio.originalPerformanceProfile;
36206 |     data.registerStopEvent = (pDevice->type != ma_device_type_duplex);
36207 |
36208 |     if (disposePreviousAudioUnit) {
36209 |         ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.AudioOutputUnitStop)((AudioUn
> it)pDevice->coreaudio.audioUnitPlayback);
36210 |         ((ma_AudioComponentInstanceDispose_proc)pDevice->pContext->coreaudio.AudioComponentInsta
> nceDispose)((AudioUnit)pDevice->coreaudio.audioUnitPlayback);
36211 |     }
36212 | }
36213 | data.periodSizeInFramesIn = pDevice->coreaudio.originalPeriodSizeInFrames;
36214 | data.periodSizeInMillisecondsIn = pDevice->coreaudio.originalPeriodSizeInMilliseconds;
36215 | data.periodsIn = pDevice->coreaudio.originalPeriods;
36216 |
36217 | /* Need at least 3 periods for duplex. */
36218 | if (data.periodsIn < 3 && pDevice->type == ma_device_type_duplex) {
36219 |     data.periodsIn = 3;
36220 | }
36221 |
36222 | result = ma_device_init_internal__coreaudio(pDevice->pContext, deviceType, NULL, &data,
> (void*)pDevice);
36223 | if (result != MA_SUCCESS) {
36224 |     return result;
36225 | }
36226 |
36227 | if (deviceType == ma_device_type_capture) {
36228 |     #if defined(MA_APPLE_DESKTOP)
36229 |         pDevice->coreaudio.deviceObjectIDCapture = (ma_uint32)data.deviceObjectID;
36230 |         ma_get_AudioObject_uid(pDevice->pContext, pDevice->coreaudio.deviceObjectIDCapture,
> sizeof(pDevice->capture.id.coreaudio), pDevice->capture.id.coreaudio);
36231 |     #endif
36232 |     pDevice->coreaudio.audioUnitCapture = (ma_ptr)data.audioUnit;
36233 |     pDevice->coreaudio.pAudioBufferList = (ma_ptr)data.pAudioBufferList;
36234 |     pDevice->coreaudio.audioBufferCapInFrames = data.periodSizeInFramesOut;
36235 |
36236 |     pDevice->capture.internalFormat = data.formatOut;
36237 |     pDevice->capture.internalChannels = data.channelsOut;
36238 |     pDevice->capture.internalSampleRate = data.sampleRateOut;
36239 |     MA_COPY_MEMORY(pDevice->capture.internalChannelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
36240 |     pDevice->capture.internalPeriodSizeInFrames = data.periodSizeInFramesOut;
36241 |     pDevice->capture.internalPeriods = data.periodsOut;
36242 | } else if (deviceType == ma_device_type_playback) {
36243 |     #if defined(MA_APPLE_DESKTOP)
36244 |         pDevice->coreaudio.deviceObjectIDPlayback = (ma_uint32)data.deviceObjectID;
36245 |         ma_get_AudioObject_uid(pDevice->pContext, pDevice->coreaudio.deviceObjectIDPlayback,
> sizeof(pDevice->playback.id.coreaudio), pDevice->playback.id.coreaudio);
36246 |     #endif
36247 |     pDevice->coreaudio.audioUnitPlayback = (ma_ptr)data.audioUnit;
36248 |
36249 |     pDevice->playback.internalFormat = data.formatOut;
36250 |     pDevice->playback.internalChannels = data.channelsOut;
36251 |     pDevice->playback.internalSampleRate = data.sampleRateOut;
36252 |     MA_COPY_MEMORY(pDevice->playback.internalChannelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
36253 |     pDevice->playback.internalPeriodSizeInFrames = data.periodSizeInFramesOut;
36254 |     pDevice->playback.internalPeriods = data.periodsOut;
36255 | }
36256 |
36257 | return MA_SUCCESS;
36258 | }
36259 | #endif /* MA_APPLE_DESKTOP */
36260 |
36261 | static ma_result ma_device_init__coreaudio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)

```

```

36262 | {
36263 |     ma_result result;
36264 |
36265 |     MA_ASSERT(pDevice != NULL);
36266 |     MA_ASSERT(pConfig != NULL);
36267 |
36268 |     if (pConfig->deviceType == ma_device_type_loopback) {
36269 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
36270 |     }
36271 |
36272 |     /* No exclusive mode with the Core Audio backend for now. */
36273 |     if (((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->shareMode == ma_share_mode_exclusive) ||
36274 |         ((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->shareMode == ma_share_mode_exclusive)) {
36275 |         return MA_SHARE_MODE_NOT_SUPPORTED;
36276 |     }
36277 |
36278 |     /* Capture needs to be initialized first. */
36279 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
36280 |         ma_device_init_internal_data_coreaudio data;
36281 |         data.allowNominalSampleRateChange = pConfig->coreaudio.allowNominalSampleRateChange;
36282 |         data.formatIn = pDescriptorCapture->format;
36283 |         data.channelsIn = pDescriptorCapture->channels;
36284 |         data.sampleRateIn = pDescriptorCapture->sampleRate;
36285 |         MA_COPY_MEMORY(data.channelMapIn, pDescriptorCapture->channelMap,
> sizeof(pDescriptorCapture->channelMap));
36286 |         data.periodSizeInFramesIn = pDescriptorCapture->periodSizeInFrames;
36287 |         data.periodSizeInMillisecondsIn = pDescriptorCapture->periodSizeInMilliseconds;
36288 |         data.periodsIn = pDescriptorCapture->periodCount;
36289 |         data.shareMode = pDescriptorCapture->shareMode;
36290 |         data.performanceProfile = pConfig->performanceProfile;
36291 |         data.registerStopEvent = MA_TRUE;
36292 |
36293 |         /* Need at least 3 periods for duplex. */
36294 |         if (data.periodsIn < 3 && pConfig->deviceType == ma_device_type_duplex) {
36295 |             data.periodsIn = 3;
36296 |         }
36297 |
36298 |         result = ma_device_init_internal_coreaudio(pDevice->pContext, ma_device_type_capture,
> pDescriptorCapture->pDeviceID, &data, (void*)pDevice);
36299 |         if (result != MA_SUCCESS) {
36300 |             return result;
36301 |         }
36302 |
36303 |         pDevice->coreaudio.isDefaultCaptureDevice = (pConfig->capture.pDeviceID == NULL);
36304 |         #if defined(MA_APPLE_DESKTOP)
36305 |         pDevice->coreaudio.deviceObjectIDCapture = (ma_uint32)data.deviceObjectID;
36306 |         #endif
36307 |         pDevice->coreaudio.audioUnitCapture = (ma_ptr)data.audioUnit;
36308 |         pDevice->coreaudio.pAudioBufferList = (ma_ptr)data.pAudioBufferList;
36309 |         pDevice->coreaudio.audioBufferCapInFrames = data.periodSizeInFramesOut;
36310 |         pDevice->coreaudio.originalPeriodSizeInFrames =
> pDescriptorCapture->periodSizeInFrames;
36311 |         pDevice->coreaudio.originalPeriodSizeInMilliseconds =
> pDescriptorCapture->periodSizeInMilliseconds;
36312 |         pDevice->coreaudio.originalPeriods = pDescriptorCapture->periodCount;
36313 |         pDevice->coreaudio.originalPerformanceProfile = pConfig->performanceProfile;
36314 |
36315 |         pDescriptorCapture->format = data.formatOut;
36316 |         pDescriptorCapture->channels = data.channelsOut;
36317 |         pDescriptorCapture->sampleRate = data.sampleRateOut;
36318 |         MA_COPY_MEMORY(pDescriptorCapture->channelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
36319 |         pDescriptorCapture->periodSizeInFrames = data.periodSizeInFramesOut;
36320 |         pDescriptorCapture->periodCount = data.periodsOut;
36321 |
36322 |         #if defined(MA_APPLE_DESKTOP)
36323 |         ma_get_AudioObject_uid(pDevice->pContext, pDevice->coreaudio.deviceObjectIDCapture,
> sizeof(pDevice->capture.id.coreaudio), pDevice->capture.id.coreaudio);
36324 |
36325 |         /*
36326 |         If we are using the default device we'll need to listen for changes to the system's default
> device so we can seamlessly
36327 |         switch the device in the background.
36328 |         */
36329 |         if (pConfig->capture.pDeviceID == NULL) {
36330 |             ma_device__track__coreaudio(pDevice);

```

```

36331 |     }
36332 |     #endif
36333 | }
36334 |
36335 |     /* Playback. */
36336 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
36337 |         ma_device_init_internal_data__coreaudio data;
36338 |         data.allowNominalSampleRateChange = pConfig->coreaudio.allowNominalSampleRateChange;
36339 |         data.formatIn = pDescriptorPlayback->format;
36340 |         data.channelsIn = pDescriptorPlayback->channels;
36341 |         data.sampleRateIn = pDescriptorPlayback->sampleRate;
36342 |         MA_COPY_MEMORY(data.channelMapIn, pDescriptorPlayback->channelMap,
> sizeof(pDescriptorPlayback->channelMap));
36343 |         data.shareMode = pDescriptorPlayback->shareMode;
36344 |         data.performanceProfile = pConfig->performanceProfile;
36345 |
36346 |         /* In full-duplex mode we want the playback buffer to be the same size as the capture
> buffer. */
36347 |         if (pConfig->deviceType == ma_device_type_duplex) {
36348 |             data.periodSizeInFramesIn = pDescriptorCapture->periodSizeInFrames;
36349 |             data.periodsIn = pDescriptorCapture->periodCount;
36350 |             data.registerStopEvent = MA_FALSE;
36351 |         } else {
36352 |             data.periodSizeInFramesIn = pDescriptorPlayback->periodSizeInFrames;
36353 |             data.periodSizeInMillisecondsIn = pDescriptorPlayback->periodSizeInMilliseconds;
36354 |             data.periodsIn = pDescriptorPlayback->periodCount;
36355 |             data.registerStopEvent = MA_TRUE;
36356 |         }
36357 |
36358 |         result = ma_device_init_internal__coreaudio(pDevice->pContext, ma_device_type_playback,
> pDescriptorPlayback->pDeviceID, &data, (void*)pDevice);
36359 |         if (result != MA_SUCCESS) {
36360 |             if (pConfig->deviceType == ma_device_type_duplex) {
36361 |                 ((ma_AudioComponentInstanceDispose_proc)pDevice->pContext->coreaudio.AudioComponentI
> nstanceDispose)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
36362 |                 if (pDevice->coreaudio.pAudioBufferList) {
36363 |                     ma_free(pDevice->coreaudio.pAudioBufferList,
> &pDevice->pContext->allocationCallbacks);
36364 |                 }
36365 |             }
36366 |             return result;
36367 |         }
36368 |
36369 |         pDevice->coreaudio.isDefaultPlaybackDevice = (pConfig->playback.pDeviceID == NULL);
36370 |         #if defined(MA_APPLE_DESKTOP)
36371 |         pDevice->coreaudio.deviceObjectIDPlayback = (ma_uint32)data.deviceObjectID;
36372 |         #endif
36373 |         pDevice->coreaudio.audioUnitPlayback = (ma_ptr)data.audioUnit;
36374 |         pDevice->coreaudio.originalPeriodSizeInFrames =
> pDescriptorPlayback->periodSizeInFrames;
36375 |         pDevice->coreaudio.originalPeriodSizeInMilliseconds =
> pDescriptorPlayback->periodSizeInMilliseconds;
36376 |         pDevice->coreaudio.originalPeriods = pDescriptorPlayback->periodCount;
36377 |         pDevice->coreaudio.originalPerformanceProfile = pConfig->performanceProfile;
36378 |
36379 |         pDescriptorPlayback->format = data.formatOut;
36380 |         pDescriptorPlayback->channels = data.channelsOut;
36381 |         pDescriptorPlayback->sampleRate = data.sampleRateOut;
36382 |         MA_COPY_MEMORY(pDescriptorPlayback->channelMap, data.channelMapOut,
> sizeof(data.channelMapOut));
36383 |         pDescriptorPlayback->periodSizeInFrames = data.periodSizeInFramesOut;
36384 |         pDescriptorPlayback->periodCount = data.periodsOut;
36385 |
36386 |         #if defined(MA_APPLE_DESKTOP)
36387 |         ma_get_AudioObject_uid(pDevice->pContext, pDevice->coreaudio.deviceObjectIDPlayback,
> sizeof(pDevice->playback.id.coreaudio), pDevice->playback.id.coreaudio);
36388 |
36389 |         /*
36390 |         If we are using the default device we'll need to listen for changes to the system's default
> device so we can seamlessly
36391 |         switch the device in the background.
36392 |         */
36393 |         if (pDescriptorPlayback->pDeviceID == NULL && (pConfig->deviceType != ma_device_type_duplex
> || pDescriptorCapture->pDeviceID != NULL)) {
36394 |             ma_device__track__coreaudio(pDevice);
36395 |         }
36396 |     #endif
36397 | }

```

```

36398 |
36399 |
36400 |
36401 |     /*
36402 |     When stopping the device, a callback is called on another thread. We need to wait for this
> callback
36403 |     before returning from ma_device_stop(). This event is used for this.
36404 |     */
36405 |     ma_event_init(&pDevice->coreaudio.stopEvent);
36406 |
36407 |     /*
36408 |     We need to detect when a route has changed so we can update the data conversion pipeline
> accordingly. This is done
36409 |     differently on non-Desktop Apple platforms.
36410 |     */
36411 |     #if defined(MA_APPLE_MOBILE)
36412 |     pDevice->coreaudio.pNotificationHandler = (MA_BRIDGE_RETAINED
> void*)[[ma_ios_notification_handler alloc] init:pDevice];
36413 |     #endif
36414 |
36415 |     return MA_SUCCESS;
36416 | }
36417 |
36418 |
36419 | static ma_result ma_device_start_coreaudio(ma_device* pDevice)
36420 | {
36421 |     MA_ASSERT(pDevice != NULL);
36422 |
36423 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
36424 |         OSStatus status = ((ma_AudioOutputUnitStart_proc)pDevice->pContext->coreaudio.AudioOutputUnit
> tStart)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
36425 |         if (status != noErr) {
36426 |             return ma_result_from_OSStatus(status);
36427 |         }
36428 |     }
36429 |
36430 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
36431 |         OSStatus status = ((ma_AudioOutputUnitStart_proc)pDevice->pContext->coreaudio.AudioOutputUnit
> tStart)((AudioUnit)pDevice->coreaudio.audioUnitPlayback);
36432 |         if (status != noErr) {
36433 |             if (pDevice->type == ma_device_type_duplex) {
36434 |                 ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.AudioOutputUnitStop)((Aud
> ioUnit)pDevice->coreaudio.audioUnitCapture);
36435 |             }
36436 |             return ma_result_from_OSStatus(status);
36437 |         }
36438 |     }
36439 |
36440 |     return MA_SUCCESS;
36441 | }
36442 |
36443 | static ma_result ma_device_stop_coreaudio(ma_device* pDevice)
36444 | {
36445 |     MA_ASSERT(pDevice != NULL);
36446 |
36447 |     /* It's not clear from the documentation whether or not AudioOutputUnitStop() actually drains
> the device or not. */
36448 |
36449 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
36450 |         OSStatus status = ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.AudioOutputUnit
> Stop)((AudioUnit)pDevice->coreaudio.audioUnitCapture);
36451 |         if (status != noErr) {
36452 |             return ma_result_from_OSStatus(status);
36453 |         }
36454 |     }
36455 |
36456 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
36457 |         OSStatus status = ((ma_AudioOutputUnitStop_proc)pDevice->pContext->coreaudio.AudioOutputUnit
> Stop)((AudioUnit)pDevice->coreaudio.audioUnitPlayback);
36458 |         if (status != noErr) {
36459 |             return ma_result_from_OSStatus(status);
36460 |         }
36461 |     }
36462 |
36463 |     /* We need to wait for the callback to finish before returning. */
36464 |     ma_event_wait(&pDevice->coreaudio.stopEvent);
36465 |     return MA_SUCCESS;
36466 | }
36467 |

```

```

36468 |
36469 | static ma_result ma_context_uninit__coreaudio(ma_context* pContext)
36470 | {
36471 |     MA_ASSERT(pContext != NULL);
36472 |     MA_ASSERT(pContext->backend == ma_backend_coreaudio);
36473 |
36474 | #if defined(MA_APPLE_MOBILE)
36475 |     if (!pContext->coreaudio.noAudioSessionDeactivate) {
36476 |         if (![AVAudioSession sharedInstance] setActive:false error:nil) {
36477 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "Failed to deactivate
> audio session.");
36478 |             return MA_FAILED_TO_INIT_BACKEND;
36479 |         }
36480 |     }
36481 | #endif
36482 |
36483 | #if !defined(MA_NO_RUNTIME_LINKING) && !defined(MA_APPLE_MOBILE)
36484 |     ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hAudioUnit);
36485 |     ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreAudio);
36486 |     ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreFoundation);
36487 | #endif
36488 |
36489 | #if !defined(MA_APPLE_MOBILE)
36490 |     ma_context__uninit_device_tracking__coreaudio(pContext);
36491 | #endif
36492 |
36493 |     (void)pContext;
36494 |     return MA_SUCCESS;
36495 | }
36496 |
36497 | #if defined(MA_APPLE_MOBILE) && defined(__IPHONE_12_0)
36498 | static AVAudioSessionCategory ma_to_AVAudioSessionCategory(ma_ios_session_category category)
36499 | {
36500 |     /* The "default" and "none" categories are treated different and should not be used as an input
> into this function. */
36501 |     MA_ASSERT(category != ma_ios_session_category_default);
36502 |     MA_ASSERT(category != ma_ios_session_category_none);
36503 |
36504 |     switch (category) {
36505 |         case ma_ios_session_category_ambient:           return AVAudioSessionCategoryAmbient;
36506 |         case ma_ios_session_category_solo_ambient:      return AVAudioSessionCategorySoloAmbient;
36507 |         case ma_ios_session_category_playback:          return AVAudioSessionCategoryPlayback;
36508 |         case ma_ios_session_category_record:            return AVAudioSessionCategoryRecord;
36509 |         case ma_ios_session_category_play_and_record:   return AVAudioSessionCategoryPlayAndRecord;
36510 |         case ma_ios_session_category_multi_route:       return AVAudioSessionCategoryMultiRoute;
36511 |         case ma_ios_session_category_none:              return AVAudioSessionCategoryAmbient;
36512 |         case ma_ios_session_category_default:           return AVAudioSessionCategoryAmbient;
36513 |         default:                                         return AVAudioSessionCategoryAmbient;
36514 |     }
36515 | }
36516 | #endif
36517 |
36518 | static ma_result ma_context_init__coreaudio(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
36519 | {
36520 | #if !defined(MA_APPLE_MOBILE)
36521 |     ma_result result;
36522 | #endif
36523 |
36524 |     MA_ASSERT(pConfig != NULL);
36525 |     MA_ASSERT(pContext != NULL);
36526 |
36527 | #if defined(MA_APPLE_MOBILE)
36528 |     @autoreleasepool {
36529 |         AVAudioSession* pAudioSession = [AVAudioSession sharedInstance];
36530 |         AVAudioSessionCategoryOptions options = pConfig->coreaudio.sessionCategoryOptions;
36531 |
36532 |         MA_ASSERT(pAudioSession != NULL);
36533 |
36534 |         if (pConfig->coreaudio.sessionCategory == ma_ios_session_category_default) {
36535 |             /*
36536 |             I'm going to use trial and error to determine our default session category. First we'll
> try PlayAndRecord. If that fails
36537 |             we'll try Playback and if that fails we'll try record. If all of these fail we'll just
> not set the category.
36538 |             */
36539 |             #if !defined(MA_APPLE_TV) && !defined(MA_APPLE_WATCH)
36540 |                 options |= AVAudioSessionCategoryOptionDefaultToSpeaker;
36541 |             #endif

```

```

36542 |
36543 |         if ([pAudioSession setCategory: AVAudioSessionCategoryPlayAndRecord withOptions:options
> error:nil]) {
36544 |             /* Using PlayAndRecord */
36545 |         } else if ([pAudioSession setCategory: AVAudioSessionCategoryPlayback
> withOptions:options error:nil]) {
36546 |             /* Using Playback */
36547 |         } else if ([pAudioSession setCategory: AVAudioSessionCategoryRecord withOptions:options
> error:nil]) {
36548 |             /* Using Record */
36549 |         } else {
36550 |             /* Leave as default? */
36551 |         }
36552 |     } else {
36553 |         if (pConfig->coreaudio.sessionCategory != ma_ios_session_category_none) {
36554 |             #if defined(__IPHONE_12_0)
36555 |                 if (![pAudioSession setCategory:
> ma_to_AVAudioSessionCategory(pConfig->coreaudio.sessionCategory) withOptions:options error:nil]) {
36556 |                     return MA_INVALID_OPERATION; /* Failed to set session category. */
36557 |                 }
36558 |             #else
36559 |                 /* Ignore the session category on version 11 and older, but post a warning. */
36560 |                 ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "Session category
> only supported in iOS 12 and newer.");
36561 |             #endif
36562 |         }
36563 |     }
36564 |
36565 |     if (!pConfig->coreaudio.noAudioSessionActivate) {
36566 |         if (![pAudioSession setActive:true error:nil]) {
36567 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "Failed to activate
> audio session.");
36568 |             return MA_FAILED_TO_INIT_BACKEND;
36569 |         }
36570 |     }
36571 | }
36572 | #endif
36573 |
36574 | #if !defined(MA_NO_RUNTIME_LINKING) && !defined(MA_APPLE_MOBILE)
36575 |     pContext->coreaudio.hCoreFoundation = ma_dlopen(ma_context_get_log(pContext),
> "/System/Library/Frameworks/CoreFoundation.framework/CoreFoundation");
36576 |     if (pContext->coreaudio.hCoreFoundation == NULL) {
36577 |         return MA_API_NOT_FOUND;
36578 |     }
36579 |
36580 |     pContext->coreaudio.CFStringGetCString = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreFoundation, "CFStringGetCString");
36581 |     pContext->coreaudio.CFRelease = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreFoundation, "CFRelease");
36582 |
36583 |
36584 |     pContext->coreaudio.hCoreAudio = ma_dlopen(ma_context_get_log(pContext),
> "/System/Library/Frameworks/CoreAudio.framework/CoreAudio");
36585 |     if (pContext->coreaudio.hCoreAudio == NULL) {
36586 |         ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreFoundation);
36587 |         return MA_API_NOT_FOUND;
36588 |     }
36589 |
36590 |     pContext->coreaudio.AudioObjectGetPropertyData = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreAudio, "AudioObjectGetPropertyData");
36591 |     pContext->coreaudio.AudioObjectGetPropertyDataSize = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreAudio, "AudioObjectGetPropertyDataSize");
36592 |     pContext->coreaudio.AudioObjectSetPropertyData = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreAudio, "AudioObjectSetPropertyData");
36593 |     pContext->coreaudio.AudioObjectAddPropertyListener = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreAudio, "AudioObjectAddPropertyListener");
36594 |     pContext->coreaudio.AudioObjectRemovePropertyListener = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hCoreAudio, "AudioObjectRemovePropertyListener");
36595 |
36596 |     /*
36597 |     > It looks like Apple has moved some APIs from AudioUnit into AudioToolbox on more recent versions
> of macOS. They are still
36598 |     defined in AudioUnit, but just in case they decide to remove them from there entirely I'm going
> to implement a fallback.
36599 |     The way it'll work is that it'll first try AudioUnit, and if the required symbols are not
> present there we'll fall back to
36600 |     AudioToolbox.
36601 |     */
36602 |     pContext->coreaudio.hAudioUnit = ma_dlopen(ma_context_get_log(pContext),

```



```

> "/System/Library/Frameworks/AudioUnit.framework/AudioUnit");
36603 |     if (pContext->coreaudio.hAudioUnit == NULL) {
36604 |         ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreAudio);
36605 |         ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreFoundation);
36606 |         return MA_API_NOT_FOUND;
36607 |     }
36608 |
36609 |     if (ma_dlsym(ma_context_get_log(pContext), pContext->coreaudio.hAudioUnit,
> "AudioComponentFindNext") == NULL) {
36610 |         /* Couldn't find the required symbols in AudioUnit, so fall back to AudioToolbox. */
36611 |         ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hAudioUnit);
36612 |         pContext->coreaudio.hAudioUnit = ma_dlopen(ma_context_get_log(pContext),
> "/System/Library/Frameworks/AudioToolbox.framework/AudioToolbox");
36613 |         if (pContext->coreaudio.hAudioUnit == NULL) {
36614 |             ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreAudio);
36615 |             ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreFoundation);
36616 |             return MA_API_NOT_FOUND;
36617 |         }
36618 |     }
36619 |
36620 |     pContext->coreaudio.AudioComponentFindNext = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioComponentFindNext");
36621 |     pContext->coreaudio.AudioComponentInstanceDispose = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioComponentInstanceDispose");
36622 |     pContext->coreaudio.AudioComponentInstanceNew = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioComponentInstanceNew");
36623 |     pContext->coreaudio.AudioOutputUnitStart = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioOutputUnitStart");
36624 |     pContext->coreaudio.AudioOutputUnitStop = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioOutputUnitStop");
36625 |     pContext->coreaudio.AudioUnitAddPropertyListener = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioUnitAddPropertyListener");
36626 |     pContext->coreaudio.AudioUnitGetPropertyInfo = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioUnitGetPropertyInfo");
36627 |     pContext->coreaudio.AudioUnitGetProperty = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioUnitGetProperty");
36628 |     pContext->coreaudio.AudioUnitSetProperty = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioUnitSetProperty");
36629 |     pContext->coreaudio.AudioUnitInitialize = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioUnitInitialize");
36630 |     pContext->coreaudio.AudioUnitRender = ma_dlsym(ma_context_get_log(pContext),
> pContext->coreaudio.hAudioUnit, "AudioUnitRender");
36631 | #else
36632 |     pContext->coreaudio.CFStringGetCString = (ma_proc)CFStringGetCString;
36633 |     pContext->coreaudio.CFRelease = (ma_proc)CFRelease;
36634 |
36635 |     #if defined(MA_APPLE_DESKTOP)
36636 |     pContext->coreaudio.AudioObjectGetPropertyData = (ma_proc)AudioObjectGetPropertyData;
36637 |     pContext->coreaudio.AudioObjectGetPropertyDataSize = (ma_proc)AudioObjectGetPropertyDataSize;
36638 |     pContext->coreaudio.AudioObjectSetPropertyData = (ma_proc)AudioObjectSetPropertyData;
36639 |     pContext->coreaudio.AudioObjectAddPropertyListener = (ma_proc)AudioObjectAddPropertyListener;
36640 |     pContext->coreaudio.AudioObjectRemovePropertyListener = (ma_proc)AudioObjectRemovePropertyListener;
> (ma_proc)AudioObjectRemovePropertyListener;
36641 | #endif
36642 |
36643 |     pContext->coreaudio.AudioComponentFindNext = (ma_proc)AudioComponentFindNext;
36644 |     pContext->coreaudio.AudioComponentInstanceDispose = (ma_proc)AudioComponentInstanceDispose;
36645 |     pContext->coreaudio.AudioComponentInstanceNew = (ma_proc)AudioComponentInstanceNew;
36646 |     pContext->coreaudio.AudioOutputUnitStart = (ma_proc)AudioOutputUnitStart;
36647 |     pContext->coreaudio.AudioOutputUnitStop = (ma_proc)AudioOutputUnitStop;
36648 |     pContext->coreaudio.AudioUnitAddPropertyListener = (ma_proc)AudioUnitAddPropertyListener;
36649 |     pContext->coreaudio.AudioUnitGetPropertyInfo = (ma_proc)AudioUnitGetPropertyInfo;
36650 |     pContext->coreaudio.AudioUnitGetProperty = (ma_proc)AudioUnitGetProperty;
36651 |     pContext->coreaudio.AudioUnitSetProperty = (ma_proc)AudioUnitSetProperty;
36652 |     pContext->coreaudio.AudioUnitInitialize = (ma_proc)AudioUnitInitialize;
36653 |     pContext->coreaudio.AudioUnitRender = (ma_proc)AudioUnitRender;
36654 | #endif
36655 |
36656 |     /* Audio component. */
36657 |     {
36658 |         AudioComponentDescription desc;
36659 |         desc.componentType = kAudioUnitType_Output;
36660 |         #if defined(MA_APPLE_DESKTOP)
36661 |         desc.componentSubType = kAudioUnitSubType_HALOutput;
36662 |         #else
36663 |         desc.componentSubType = kAudioUnitSubType_RemoteIO;
36664 |         #endif
36665 |         desc.componentManufacturer = kAudioUnitManufacturer_Apple;
36666 |         desc.componentFlags = 0;

```



```

36667 |         desc.componentFlagsMask    = 0;
36668 |
36669 |         pContext->coreaudio.component =
36670 | > ((ma_AudioComponentFindNext_proc)pContext->coreaudio.AudioComponentFindNext)(NULL, &desc);
36671 |         if (pContext->coreaudio.component == NULL) {
36672 |             #if !defined(MA_NO_RUNTIME_LINKING) && !defined(MA_APPLE_MOBILE)
36673 |                 ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hAudioUnit);
36674 |                 ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreAudio);
36675 |                 ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreFoundation);
36676 |             #endif
36677 |             return MA_FAILED_TO_INIT_BACKEND;
36678 |         }
36679 |
36680 |         #if !defined(MA_APPLE_MOBILE)
36681 |             result = ma_context__init_device_tracking__coreaudio(pContext);
36682 |             if (result != MA_SUCCESS) {
36683 |                 #if !defined(MA_NO_RUNTIME_LINKING) && !defined(MA_APPLE_MOBILE)
36684 |                     ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hAudioUnit);
36685 |                     ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreAudio);
36686 |                     ma_dlclose(ma_context_get_log(pContext), pContext->coreaudio.hCoreFoundation);
36687 |                 #endif
36688 |                 return result;
36689 |             }
36690 |         #endif
36691 |
36692 |         pContext->coreaudio.noAudioSessionDeactivate = pConfig->coreaudio.noAudioSessionDeactivate;
36693 |
36694 |         pCallbacks->onContextInit          = ma_context_init__coreaudio;
36695 |         pCallbacks->onContextUninit        = ma_context_uninit__coreaudio;
36696 |         pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__coreaudio;
36697 |         pCallbacks->onContextGetDeviceInfo  = ma_context_get_device_info__coreaudio;
36698 |         pCallbacks->onDeviceInit           = ma_device_init__coreaudio;
36699 |         pCallbacks->onDeviceUninit         = ma_device_uninit__coreaudio;
36700 |         pCallbacks->onDeviceStart          = ma_device_start__coreaudio;
36701 |         pCallbacks->onDeviceStop           = ma_device_stop__coreaudio;
36702 |         pCallbacks->onDeviceRead           = NULL;
36703 |         pCallbacks->onDeviceWrite          = NULL;
36704 |         pCallbacks->onDeviceDataLoop       = NULL;
36705 |
36706 |         return MA_SUCCESS;
36707 |     }
36708 | #endif /* MA_HAS_COREAUDIO */
36709 |
36710 |
36711 |
36712 | /*****
36713 |
36714 | sndio Backend
36715 |
36716 | *****/
36717 | #ifdef MA_HAS_SNDIO
36718 | #include <fcntl.h>
36719 |
36720 | /*
36721 | > Only supporting OpenBSD. This did not work very well at all on FreeBSD when I tried it. Not sure if
36722 | > this is due
36723 | > to miniaudio's implementation or if it's some kind of system configuration issue, but basically the
36724 | > default device
36725 | > just doesn't emit any sound, or at times you'll hear tiny pieces. I will consider enabling this when
36726 | > there's
36727 | > demand for it or if I can get it tested and debugged more thoroughly.
36728 | */
36729 | #if 0
36730 | #if defined(__NetBSD__) || defined(__OpenBSD__)
36731 | #include <sys/audioio.h>
36732 | #endif
36733 | #if defined(__FreeBSD__) || defined(__DragonFly__)
36734 | #include <sys/soundcard.h>
36735 | #endif
36736 | #endif
36737 |
36738 | #define MA_SIO_DEVANY    "default"
36739 | #define MA_SIO_PLAY      1
36740 | #define MA_SIO_REC        2
36741 | #define MA_SIO_NENC       8
36742 | #define MA_SIO_NCHAN      8
36743 | #define MA_SIO_NRATE     16
36744 | #define MA_SIO_NCONF      4

```

```

36742 |
36743 | struct ma_sio_hdl; /* <-- Opaque */
36744 |
36745 | struct ma_sio_par
36746 | {
36747 |     unsigned int bits;
36748 |     unsigned int bps;
36749 |     unsigned int sig;
36750 |     unsigned int le;
36751 |     unsigned int msb;
36752 |     unsigned int rchan;
36753 |     unsigned int pchan;
36754 |     unsigned int rate;
36755 |     unsigned int bufsz;
36756 |     unsigned int xrun;
36757 |     unsigned int round;
36758 |     unsigned int appbufsz;
36759 |     int __pad[3];
36760 |     unsigned int __magic;
36761 | };
36762 |
36763 | struct ma_sio_enc
36764 | {
36765 |     unsigned int bits;
36766 |     unsigned int bps;
36767 |     unsigned int sig;
36768 |     unsigned int le;
36769 |     unsigned int msb;
36770 | };
36771 |
36772 | struct ma_sio_conf
36773 | {
36774 |     unsigned int enc;
36775 |     unsigned int rchan;
36776 |     unsigned int pchan;
36777 |     unsigned int rate;
36778 | };
36779 |
36780 | struct ma_sio_cap
36781 | {
36782 |     struct ma_sio_enc enc[MA_SIO_NENC];
36783 |     unsigned int rchan[MA_SIO_NCHAN];
36784 |     unsigned int pchan[MA_SIO_NCHAN];
36785 |     unsigned int rate[MA_SIO_NRATE];
36786 |     int __pad[7];
36787 |     unsigned int nconf;
36788 |     struct ma_sio_conf confs[MA_SIO_NCONF];
36789 | };
36790 |
36791 | typedef struct ma_sio_hdl* (* ma_sio_open_proc) (const char*, unsigned int, int);
36792 | typedef void (* ma_sio_close_proc) (struct ma_sio_hdl*);
36793 | typedef int (* ma_sio_setpar_proc) (struct ma_sio_hdl*, struct ma_sio_par*);
36794 | typedef int (* ma_sio_getpar_proc) (struct ma_sio_hdl*, struct ma_sio_par*);
36795 | typedef int (* ma_sio_getcap_proc) (struct ma_sio_hdl*, struct ma_sio_cap*);
36796 | typedef size_t (* ma_sio_write_proc) (struct ma_sio_hdl*, const void*, size_t);
36797 | typedef size_t (* ma_sio_read_proc) (struct ma_sio_hdl*, void*, size_t);
36798 | typedef int (* ma_sio_start_proc) (struct ma_sio_hdl*);
36799 | typedef int (* ma_sio_stop_proc) (struct ma_sio_hdl*);
36800 | typedef int (* ma_sio_initpar_proc)(struct ma_sio_par*);
36801 |
36802 | static ma_uint32 ma_get_standard_sample_rate_priority_index__sndio(ma_uint32 sampleRate) /* Lower
> = higher priority */
36803 | {
36804 |     ma_uint32 i;
36805 |     for (i = 0; i < ma_countof(g_maStandardSampleRatePriorities); ++i) {
36806 |         if (g_maStandardSampleRatePriorities[i] == sampleRate) {
36807 |             return i;
36808 |         }
36809 |     }
36810 |
36811 |     return (ma_uint32)-1;
36812 | }
36813 |
36814 | static ma_format ma_format_from_sio_enc__sndio(unsigned int bits, unsigned int bps, unsigned int
> sig, unsigned int le, unsigned int msb)
36815 | {
36816 |     /* We only support native-endian right now. */
36817 |     if ((ma_is_little_endian() && le == 0) || (ma_is_big_endian() && le == 1)) {
36818 |         return ma_format_unknown;

```

```

36819 |     }
36820 |
36821 |     if (bits == 8 && bps == 1 && sig == 0) {
36822 |         return ma_format_u8;
36823 |     }
36824 |     if (bits == 16 && bps == 2 && sig == 1) {
36825 |         return ma_format_s16;
36826 |     }
36827 |     if (bits == 24 && bps == 3 && sig == 1) {
36828 |         return ma_format_s24;
36829 |     }
36830 |     if (bits == 24 && bps == 4 && sig == 1 && msb == 0) {
36831 |         /*return ma_format_s24_s32;*/
36832 |     }
36833 |     if (bits == 32 && bps == 4 && sig == 1) {
36834 |         return ma_format_s32;
36835 |     }
36836 |
36837 |     return ma_format_unknown;
36838 | }
36839 |
36840 | static ma_format ma_find_best_format_from_sio_cap_sndio(struct ma_sio_cap* caps)
36841 | {
36842 |     ma_format bestFormat;
36843 |     unsigned int iConfig;
36844 |
36845 |     MA_ASSERT(caps != NULL);
36846 |
36847 |     bestFormat = ma_format_unknown;
36848 |     for (iConfig = 0; iConfig < caps->nconf; iConfig += 1) {
36849 |         unsigned int iEncoding;
36850 |         for (iEncoding = 0; iEncoding < MA_SIO_NENC; iEncoding += 1) {
36851 |             unsigned int bits;
36852 |             unsigned int bps;
36853 |             unsigned int sig;
36854 |             unsigned int le;
36855 |             unsigned int msb;
36856 |             ma_format format;
36857 |
36858 |             if ((caps->confs[iConfig].enc & (1UL << iEncoding)) == 0) {
36859 |                 continue;
36860 |             }
36861 |
36862 |             bits = caps->enc[iEncoding].bits;
36863 |             bps = caps->enc[iEncoding].bps;
36864 |             sig = caps->enc[iEncoding].sig;
36865 |             le = caps->enc[iEncoding].le;
36866 |             msb = caps->enc[iEncoding].msb;
36867 |             format = ma_format_from_sio_enc_sndio(bits, bps, sig, le, msb);
36868 |             if (format == ma_format_unknown) {
36869 |                 continue; /* Format not supported. */
36870 |             }
36871 |
36872 |             if (bestFormat == ma_format_unknown) {
36873 |                 bestFormat = format;
36874 |             } else {
36875 |                 if (ma_get_format_priority_index(bestFormat) > ma_get_format_priority_index(format))
> { /* <-- Lower = better. */
36876 |                     bestFormat = format;
36877 |                 }
36878 |             }
36879 |         }
36880 |     }
36881 |
36882 |     return bestFormat;
36883 | }
36884 |
36885 | static ma_uint32 ma_find_best_channels_from_sio_cap_sndio(struct ma_sio_cap* caps, ma_device_type
> deviceType, ma_format requiredFormat)
36886 | {
36887 |     ma_uint32 maxChannels;
36888 |     unsigned int iConfig;
36889 |
36890 |     MA_ASSERT(caps != NULL);
36891 |     MA_ASSERT(requiredFormat != ma_format_unknown);
36892 |
36893 |     /* Just pick whatever configuration has the most channels. */
36894 |     maxChannels = 0;
36895 |     for (iConfig = 0; iConfig < caps->nconf; iConfig += 1) {

```

```

36896 |         /* The encoding should be of requiredFormat. */
36897 |         unsigned int iEncoding;
36898 |         for (iEncoding = 0; iEncoding < MA_SIO_NENC; iEncoding += 1) {
36899 |             unsigned int iChannel;
36900 |             unsigned int bits;
36901 |             unsigned int bps;
36902 |             unsigned int sig;
36903 |             unsigned int le;
36904 |             unsigned int msb;
36905 |             ma_format format;
36906 |
36907 |             if ((caps->confs[iConfig].enc & (1UL << iEncoding)) == 0) {
36908 |                 continue;
36909 |             }
36910 |
36911 |             bits = caps->enc[iEncoding].bits;
36912 |             bps = caps->enc[iEncoding].bps;
36913 |             sig = caps->enc[iEncoding].sig;
36914 |             le = caps->enc[iEncoding].le;
36915 |             msb = caps->enc[iEncoding].msb;
36916 |             format = ma_format_from_sio_enc_sndio(bits, bps, sig, le, msb);
36917 |             if (format != requiredFormat) {
36918 |                 continue;
36919 |             }
36920 |
36921 |             /* Getting here means the format is supported. Iterate over each channel count and grab
> the biggest one. */
36922 |             for (iChannel = 0; iChannel < MA_SIO_NCHAN; iChannel += 1) {
36923 |                 unsigned int chan = 0;
36924 |                 unsigned int channels;
36925 |
36926 |                 if (deviceType == ma_device_type_playback) {
36927 |                     chan = caps->confs[iConfig].pchan;
36928 |                 } else {
36929 |                     chan = caps->confs[iConfig].rchan;
36930 |                 }
36931 |
36932 |                 if ((chan & (1UL << iChannel)) == 0) {
36933 |                     continue;
36934 |                 }
36935 |
36936 |                 if (deviceType == ma_device_type_playback) {
36937 |                     channels = caps->pchan[iChannel];
36938 |                 } else {
36939 |                     channels = caps->rchan[iChannel];
36940 |                 }
36941 |
36942 |                 if (maxChannels < channels) {
36943 |                     maxChannels = channels;
36944 |                 }
36945 |             }
36946 |         }
36947 |     }
36948 |
36949 |     return maxChannels;
36950 | }
36951 |
36952 | static ma_uint32 ma_find_best_sample_rate_from_sio_cap_sndio(struct ma_sio_cap* caps,
> ma_device_type deviceType, ma_format requiredFormat, ma_uint32 requiredChannels)
36953 | {
36954 |     ma_uint32 firstSampleRate;
36955 |     ma_uint32 bestSampleRate;
36956 |     unsigned int iConfig;
36957 |
36958 |     MA_ASSERT(caps != NULL);
36959 |     MA_ASSERT(requiredFormat != ma_format_unknown);
36960 |     MA_ASSERT(requiredChannels > 0);
36961 |     MA_ASSERT(requiredChannels <= MA_MAX_CHANNELS);
36962 |
36963 |     firstSampleRate = 0; /* <-- If the device does not support a standard rate we'll fall back to
> the first one that's found. */
36964 |     bestSampleRate = 0;
36965 |
36966 |     for (iConfig = 0; iConfig < caps->nconf; iConfig += 1) {
36967 |         /* The encoding should be of requiredFormat. */
36968 |         unsigned int iEncoding;
36969 |         for (iEncoding = 0; iEncoding < MA_SIO_NENC; iEncoding += 1) {
36970 |             unsigned int iChannel;
36971 |             unsigned int bits;

```

```

36972 |         unsigned int bps;
36973 |         unsigned int sig;
36974 |         unsigned int le;
36975 |         unsigned int msb;
36976 |         ma_format format;
36977 |
36978 |         if ((caps->confs[iConfig].enc & (1UL << iEncoding)) == 0) {
36979 |             continue;
36980 |         }
36981 |
36982 |         bits = caps->enc[iEncoding].bits;
36983 |         bps = caps->enc[iEncoding].bps;
36984 |         sig = caps->enc[iEncoding].sig;
36985 |         le = caps->enc[iEncoding].le;
36986 |         msb = caps->enc[iEncoding].msb;
36987 |         format = ma_format_from_sio_enc_sndio(bits, bps, sig, le, msb);
36988 |         if (format != requiredFormat) {
36989 |             continue;
36990 |         }
36991 |
36992 |         /* Getting here means the format is supported. Iterate over each channel count and grab
> the biggest one. */
36993 |         for (iChannel = 0; iChannel < MA_SIO_NCHAN; iChannel += 1) {
36994 |             unsigned int chan = 0;
36995 |             unsigned int channels;
36996 |             unsigned int iRate;
36997 |
36998 |             if (deviceType == ma_device_type_playback) {
36999 |                 chan = caps->confs[iConfig].pchan;
37000 |             } else {
37001 |                 chan = caps->confs[iConfig].rchan;
37002 |             }
37003 |
37004 |             if ((chan & (1UL << iChannel)) == 0) {
37005 |                 continue;
37006 |             }
37007 |
37008 |             if (deviceType == ma_device_type_playback) {
37009 |                 channels = caps->pchan[iChannel];
37010 |             } else {
37011 |                 channels = caps->rchan[iChannel];
37012 |             }
37013 |
37014 |             if (channels != requiredChannels) {
37015 |                 continue;
37016 |             }
37017 |
37018 |             /* Getting here means we have found a compatible encoding/channel pair. */
37019 |             for (iRate = 0; iRate < MA_SIO_NRATE; iRate += 1) {
37020 |                 ma_uint32 rate = (ma_uint32)caps->rate[iRate];
37021 |                 ma_uint32 ratePriority;
37022 |
37023 |                 if (firstSampleRate == 0) {
37024 |                     firstSampleRate = rate;
37025 |                 }
37026 |
37027 |                 /* Disregard this rate if it's not a standard one. */
37028 |                 ratePriority = ma_get_standard_sample_rate_priority_index_sndio(rate);
37029 |                 if (ratePriority == (ma_uint32)-1) {
37030 |                     continue;
37031 |                 }
37032 |
37033 |                 if (ma_get_standard_sample_rate_priority_index_sndio(bestSampleRate) >
> ratePriority) { /* Lower = better. */
37034 |                     bestSampleRate = rate;
37035 |                 }
37036 |             }
37037 |         }
37038 |     }
37039 | }
37040 |
37041 | /* If a standard sample rate was not found just fall back to the first one that was iterated. */
37042 | if (bestSampleRate == 0) {
37043 |     bestSampleRate = firstSampleRate;
37044 | }
37045 |
37046 | return bestSampleRate;
37047 | }
37048 |

```

```

37049 |
37050 | static ma_result ma_context_enumerate_devices_sndio(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
37051 | {
37052 |     ma_bool32 isTerminating = MA_FALSE;
37053 |     struct ma_sio_hdl* handle;
37054 |
37055 |     MA_ASSERT(pContext != NULL);
37056 |     MA_ASSERT(callback != NULL);
37057 |
37058 |     /* sndio doesn't seem to have a good device enumeration API, so I'm therefore only enumerating
> over default devices for now. */
37059 |
37060 |     /* Playback. */
37061 |     if (!isTerminating) {
37062 |         handle = ((ma_sio_open_proc)pContext->sndio.sio_open)(MA_SIO_DEVANY, MA_SIO_PLAY, 0);
37063 |         if (handle != NULL) {
37064 |             /* Supports playback. */
37065 |             ma_device_info deviceInfo;
37066 |             MA_ZERO_OBJECT(&deviceInfo);
37067 |             ma_strcpy_s(deviceInfo.id.sndio, sizeof(deviceInfo.id.sndio), MA_SIO_DEVANY);
37068 |             ma_strcpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_PLAYBACK_DEVICE_NAME);
37069 |
37070 |             isTerminating = !callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
37071 |
37072 |             ((ma_sio_close_proc)pContext->sndio.sio_close)(handle);
37073 |         }
37074 |     }
37075 |
37076 |     /* Capture. */
37077 |     if (!isTerminating) {
37078 |         handle = ((ma_sio_open_proc)pContext->sndio.sio_open)(MA_SIO_DEVANY, MA_SIO_REC, 0);
37079 |         if (handle != NULL) {
37080 |             /* Supports capture. */
37081 |             ma_device_info deviceInfo;
37082 |             MA_ZERO_OBJECT(&deviceInfo);
37083 |             ma_strcpy_s(deviceInfo.id.sndio, sizeof(deviceInfo.id.sndio), "default");
37084 |             ma_strcpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_CAPTURE_DEVICE_NAME);
37085 |
37086 |             isTerminating = !callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
37087 |
37088 |             ((ma_sio_close_proc)pContext->sndio.sio_close)(handle);
37089 |         }
37090 |     }
37091 |
37092 |     return MA_SUCCESS;
37093 | }
37094 |
37095 | static ma_result ma_context_get_device_info_sndio(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
37096 | {
37097 |     char devid[256];
37098 |     struct ma_sio_hdl* handle;
37099 |     struct ma_sio_cap caps;
37100 |     unsigned int iConfig;
37101 |
37102 |     MA_ASSERT(pContext != NULL);
37103 |
37104 |     /* We need to open the device before we can get information about it. */
37105 |     if (pDeviceID == NULL) {
37106 |         ma_strcpy_s(devid, sizeof(devid), MA_SIO_DEVANY);
37107 |         ma_strcpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), (deviceType ==
> ma_device_type_playback) ? MA_DEFAULT_PLAYBACK_DEVICE_NAME : MA_DEFAULT_CAPTURE_DEVICE_NAME);
37108 |     } else {
37109 |         ma_strcpy_s(devid, sizeof(devid), pDeviceID->sndio);
37110 |         ma_strcpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), devid);
37111 |     }
37112 |
37113 |     handle = ((ma_sio_open_proc)pContext->sndio.sio_open)(devid, (deviceType ==
> ma_device_type_playback) ? MA_SIO_PLAY : MA_SIO_REC, 0);
37114 |     if (handle == NULL) {
37115 |         return MA_NO_DEVICE;
37116 |     }
37117 |
37118 |     if (((ma_sio_getcap_proc)pContext->sndio.sio_getcap)(handle, &caps) == 0) {
37119 |         return MA_ERROR;
37120 |     }
37121 |
37122 |     pDeviceInfo->nativeDataFormatCount = 0;

```

```

37123 |
37124 |     for (iConfig = 0; iConfig < caps.nconf; iConfig += 1) {
37125 |         /*
37126 |             The main thing we care about is that the encoding is supported by miniaudio. If it is, we
> want to give
37127 |             preference to some formats over others.
37128 |         */
37129 |         unsigned int iEncoding;
37130 |         unsigned int iChannel;
37131 |         unsigned int iRate;
37132 |
37133 |         for (iEncoding = 0; iEncoding < MA_SIO_NENC; iEncoding += 1) {
37134 |             unsigned int bits;
37135 |             unsigned int bps;
37136 |             unsigned int sig;
37137 |             unsigned int le;
37138 |             unsigned int msb;
37139 |             ma_format format;
37140 |
37141 |             if ((caps.confs[iConfig].enc & (1UL << iEncoding)) == 0) {
37142 |                 continue;
37143 |             }
37144 |
37145 |             bits = caps.enc[iEncoding].bits;
37146 |             bps = caps.enc[iEncoding].bps;
37147 |             sig = caps.enc[iEncoding].sig;
37148 |             le = caps.enc[iEncoding].le;
37149 |             msb = caps.enc[iEncoding].msb;
37150 |             format = ma_format_from_sio_enc_sndio(bits, bps, sig, le, msb);
37151 |             if (format == ma_format_unknown) {
37152 |                 continue; /* Format not supported. */
37153 |             }
37154 |
37155 |             /* Channels. */
37156 |             for (iChannel = 0; iChannel < MA_SIO_NCHAN; iChannel += 1) {
37157 |                 unsigned int chan = 0;
37158 |                 unsigned int channels;
37159 |
37160 |                 if (deviceType == ma_device_type_playback) {
37161 |                     chan = caps.confs[iConfig].pchan;
37162 |                 } else {
37163 |                     chan = caps.confs[iConfig].rchan;
37164 |                 }
37165 |
37166 |                 if ((chan & (1UL << iChannel)) == 0) {
37167 |                     continue;
37168 |                 }
37169 |
37170 |                 if (deviceType == ma_device_type_playback) {
37171 |                     channels = caps.pchan[iChannel];
37172 |                 } else {
37173 |                     channels = caps.rchan[iChannel];
37174 |                 }
37175 |
37176 |                 /* Sample Rates. */
37177 |                 for (iRate = 0; iRate < MA_SIO_NRATE; iRate += 1) {
37178 |                     if ((caps.confs[iConfig].rate & (1UL << iRate)) != 0) {
37179 |                         ma_device_info_add_native_data_format(pDeviceInfo, format, channels,
37180 | > caps.rate[iRate], 0);
37181 |                     }
37182 |                 }
37183 |             }
37184 |         }
37185 |     }
37186 | }
37187 |
37188 | ((ma_sio_close_proc)pContext->sndio.sio_close)(handle);
37189 | return MA_SUCCESS;
37190 | }
37191 |
37192 | static ma_result ma_device_uninit__sndio(ma_device* pDevice)
37193 | {
37194 |     MA_ASSERT(pDevice != NULL);
37195 |
37196 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
37197 |         ((ma_sio_close_proc)pDevice->pContext->sndio.sio_close)((struct
> ma_sio_hdl*)pDevice->sndio.handleCapture);
37198 |     }

```



```

37199 |
37200 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
37201 |         ((ma_sio_close_proc)pDevice->pContext->sndio.sio_close)((struct
> ma_sio_hdl*)pDevice->sndio.handlePlayback);
37202 |     }
37203 |
37204 |     return MA_SUCCESS;
37205 | }
37206 |
37207 | static ma_result ma_device_init_handle_sndio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptor, ma_device_type deviceType)
37208 | {
37209 |     const char* pDeviceName;
37210 |     ma_ptr handle;
37211 |     int openFlags = 0;
37212 |     struct ma_sio_cap caps;
37213 |     struct ma_sio_par par;
37214 |     const ma_device_id* pDeviceID;
37215 |     ma_format format;
37216 |     ma_uint32 channels;
37217 |     ma_uint32 sampleRate;
37218 |     ma_format internalFormat;
37219 |     ma_uint32 internalChannels;
37220 |     ma_uint32 internalSampleRate;
37221 |     ma_uint32 internalPeriodSizeInFrames;
37222 |     ma_uint32 internalPeriods;
37223 |
37224 |     MA_ASSERT(pConfig != NULL);
37225 |     MA_ASSERT(deviceType != ma_device_type_duplex);
37226 |     MA_ASSERT(pDevice != NULL);
37227 |
37228 |     if (deviceType == ma_device_type_capture) {
37229 |         openFlags = MA_SIO_REC;
37230 |     } else {
37231 |         openFlags = MA_SIO_PLAY;
37232 |     }
37233 |
37234 |     pDeviceID = pDescriptor->pDeviceID;
37235 |     format = pDescriptor->format;
37236 |     channels = pDescriptor->channels;
37237 |     sampleRate = pDescriptor->sampleRate;
37238 |
37239 |     pDeviceName = MA_SIO_DEVANY;
37240 |     if (pDeviceID != NULL) {
37241 |         pDeviceName = pDeviceID->sndio;
37242 |     }
37243 |
37244 |     handle = (ma_ptr)((ma_sio_open_proc)pDevice->pContext->sndio.sio_open)(pDeviceName, openFlags,
> 0);
37245 |     if (handle == NULL) {
37246 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[sndio] Failed to open
> device.");
37247 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
37248 |     }
37249 |
37250 |     /* We need to retrieve the device caps to determine the most appropriate format to use. */
37251 |     if (((ma_sio_getcap_proc)pDevice->pContext->sndio.sio_getcap)((struct ma_sio_hdl*)handle, &caps)
> == 0) {
37252 |         ((ma_sio_close_proc)pDevice->pContext->sndio.sio_close)((struct ma_sio_hdl*)handle);
37253 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[sndio] Failed to retrieve
> device caps.");
37254 |         return MA_ERROR;
37255 |     }
37256 |
37257 |     /*
37258 |     Note: sndio reports a huge range of available channels. This is inconvenient for us because
> there's no real
37259 |     way, as far as I can tell, to get the _actual_ channel count of the device. I'm therefore
> restricting this
37260 |     to the requested channels, regardless of whether or not the default channel count is requested.
37261 |
37262 |     For hardware devices, I'm suspecting only a single channel count will be reported and we can
> safely use the
37263 |     value returned by ma_find_best_channels_from_sio_cap_sndio().
37264 |     */
37265 |     if (deviceType == ma_device_type_capture) {
37266 |         if (format == ma_format_unknown) {
37267 |             format = ma_find_best_format_from_sio_cap_sndio(&caps);
37268 |         }

```

```

37269 |
37270 |         if (channels == 0) {
37271 |             if (strlen(pDeviceName) > strlen("rsnd/") && strncmp(pDeviceName, "rsnd/",
> strlen("rsnd/")) == 0) {
37272 |                 channels = ma_find_best_channels_from_sio_cap_sndio(&caps, deviceType, format);
37273 |             } else {
37274 |                 channels = MA_DEFAULT_CHANNELS;
37275 |             }
37276 |         }
37277 |     } else {
37278 |         if (format == ma_format_unknown) {
37279 |             format = ma_find_best_format_from_sio_cap_sndio(&caps);
37280 |         }
37281 |
37282 |         if (channels == 0) {
37283 |             if (strlen(pDeviceName) > strlen("rsnd/") && strncmp(pDeviceName, "rsnd/",
> strlen("rsnd/")) == 0) {
37284 |                 channels = ma_find_best_channels_from_sio_cap_sndio(&caps, deviceType, format);
37285 |             } else {
37286 |                 channels = MA_DEFAULT_CHANNELS;
37287 |             }
37288 |         }
37289 |     }
37290 |
37291 |     if (sampleRate == 0) {
37292 |         sampleRate = ma_find_best_sample_rate_from_sio_cap_sndio(&caps, pConfig->deviceType,
> format, channels);
37293 |     }
37294 |
37295 |
37296 |     ((ma_sio_initpar_proc)pDevice->pContext->sndio.sio_initpar)(&par);
37297 |     par.msb = 0;
37298 |     par.le = ma_is_little_endian();
37299 |
37300 |     switch (format) {
37301 |         case ma_format_u8:
37302 |         {
37303 |             par.bits = 8;
37304 |             par.bps = 1;
37305 |             par.sig = 0;
37306 |         } break;
37307 |
37308 |         case ma_format_s24:
37309 |         {
37310 |             par.bits = 24;
37311 |             par.bps = 3;
37312 |             par.sig = 1;
37313 |         } break;
37314 |
37315 |         case ma_format_s32:
37316 |         {
37317 |             par.bits = 32;
37318 |             par.bps = 4;
37319 |             par.sig = 1;
37320 |         } break;
37321 |
37322 |         case ma_format_s16:
37323 |         case ma_format_f32:
37324 |         case ma_format_unknown:
37325 |         default:
37326 |         {
37327 |             par.bits = 16;
37328 |             par.bps = 2;
37329 |             par.sig = 1;
37330 |         } break;
37331 |     }
37332 |
37333 |     if (deviceType == ma_device_type_capture) {
37334 |         par.rchan = channels;
37335 |     } else {
37336 |         par.pchan = channels;
37337 |     }
37338 |
37339 |     par.rate = sampleRate;
37340 |
37341 |     internalPeriodSizeInFrames = ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor,
> par.rate, pConfig->performanceProfile);
37342 |
37343 |     par.round = internalPeriodSizeInFrames;

```

```

37344 |     par.appbufsz = par.round * pDescriptor->periodCount;
37345 |
37346 |     if (((ma_sio_setpar_proc)pDevice->pContext->sndio.sio_setpar)((struct ma_sio_hdl*)handle, &par)
> == 0) {
37347 |         ((ma_sio_close_proc)pDevice->pContext->sndio.sio_close)((struct ma_sio_hdl*)handle);
37348 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[sndio] Failed to set buffer
> size.");
37349 |         return MA_ERROR;
37350 |     }
37351 |
37352 |     if (((ma_sio_getpar_proc)pDevice->pContext->sndio.sio_getpar)((struct ma_sio_hdl*)handle, &par)
> == 0) {
37353 |         ((ma_sio_close_proc)pDevice->pContext->sndio.sio_close)((struct ma_sio_hdl*)handle);
37354 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[sndio] Failed to retrieve
> buffer size.");
37355 |         return MA_ERROR;
37356 |     }
37357 |
37358 |     internalFormat          = ma_format_from_sio_enc__sndio(par.bits, par.bps, par.sig, par.le,
> par.msb);
37359 |     internalChannels        = (deviceType == ma_device_type_capture) ? par.rchan : par.pchan;
37360 |     internalSampleRate      = par.rate;
37361 |     internalPeriods         = par.appbufsz / par.round;
37362 |     internalPeriodSizeInFrames = par.round;
37363 |
37364 |     if (deviceType == ma_device_type_capture) {
37365 |         pDevice->sndio.handleCapture = handle;
37366 |     } else {
37367 |         pDevice->sndio.handlePlayback = handle;
37368 |     }
37369 |
37370 |     pDescriptor->format          = internalFormat;
37371 |     pDescriptor->channels        = internalChannels;
37372 |     pDescriptor->sampleRate      = internalSampleRate;
37373 |     ma_channel_map_init_standard(ma_standard_channel_map_sndio, pDescriptor->channelMap,
> ma_countof(pDescriptor->channelMap), internalChannels);
37374 |     pDescriptor->periodSizeInFrames = internalPeriodSizeInFrames;
37375 |     pDescriptor->periodCount      = internalPeriods;
37376 |
37377 |     return MA_SUCCESS;
37378 | }
37379 |
37380 | static ma_result ma_device_init__sndio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
37381 | {
37382 |     MA_ASSERT(pDevice != NULL);
37383 |
37384 |     MA_ZERO_OBJECT(&pDevice->sndio);
37385 |
37386 |     if (pConfig->deviceType == ma_device_type_loopback) {
37387 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
37388 |     }
37389 |
37390 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
37391 |         ma_result result = ma_device_init_handle__sndio(pDevice, pConfig, pDescriptorCapture,
> ma_device_type_capture);
37392 |         if (result != MA_SUCCESS) {
37393 |             return result;
37394 |         }
37395 |     }
37396 |
37397 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
37398 |         ma_result result = ma_device_init_handle__sndio(pDevice, pConfig, pDescriptorPlayback,
> ma_device_type_playback);
37399 |         if (result != MA_SUCCESS) {
37400 |             return result;
37401 |         }
37402 |     }
37403 |
37404 |     return MA_SUCCESS;
37405 | }
37406 |
37407 | static ma_result ma_device_start__sndio(ma_device* pDevice)
37408 | {
37409 |     MA_ASSERT(pDevice != NULL);
37410 |
37411 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {

```

```

37412 |         ((ma_sio_start_proc)pDevice->pContext->sndio.sio_start)((struct
> ma_sio_hdl*)pDevice->sndio.handleCapture);
37413 |     }
37414 |
37415 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
37416 |         ((ma_sio_start_proc)pDevice->pContext->sndio.sio_start)((struct
> ma_sio_hdl*)pDevice->sndio.handlePlayback); /* <-- Doesn't actually playback until data is
> written. */
37417 |     }
37418 |
37419 |     return MA_SUCCESS;
37420 | }
37421 |
37422 | static ma_result ma_device_stop_sndio(ma_device* pDevice)
37423 | {
37424 |     MA_ASSERT(pDevice != NULL);
37425 |
37426 |     /*
37427 |     From the documentation:
37428 |
37429 |     The sio_stop() function puts the audio subsystem in the same state as before sio_start() is
> called. It stops recording, drains the play buffer and then
37430 |     stops playback. If samples to play are queued but playback hasn't started yet then playback
> is forced immediately; playback will actually stop once the
37431 |     buffer is drained. In no case are samples in the play buffer discarded.
37432 |
37433 |     Therefore, sio_stop() performs all of the necessary draining for us.
37434 |     */
37435 |
37436 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
37437 |         ((ma_sio_stop_proc)pDevice->pContext->sndio.sio_stop)((struct
> ma_sio_hdl*)pDevice->sndio.handleCapture);
37438 |     }
37439 |
37440 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
37441 |         ((ma_sio_stop_proc)pDevice->pContext->sndio.sio_stop)((struct
> ma_sio_hdl*)pDevice->sndio.handlePlayback);
37442 |     }
37443 |
37444 |     return MA_SUCCESS;
37445 | }
37446 |
37447 | static ma_result ma_device_write_sndio(ma_device* pDevice, const void* pPCMFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
37448 | {
37449 |     int result;
37450 |
37451 |     if (pFramesWritten != NULL) {
37452 |         *pFramesWritten = 0;
37453 |     }
37454 |
37455 |     result = ((ma_sio_write_proc)pDevice->pContext->sndio.sio_write)((struct
> ma_sio_hdl*)pDevice->sndio.handlePlayback, pPCMFrames, frameCount *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels));
37456 |     if (result == 0) {
37457 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[sndio] Failed to send data
> from the client to the device.");
37458 |         return MA_IO_ERROR;
37459 |     }
37460 |
37461 |     if (pFramesWritten != NULL) {
37462 |         *pFramesWritten = frameCount;
37463 |     }
37464 |
37465 |     return MA_SUCCESS;
37466 | }
37467 |
37468 | static ma_result ma_device_read_sndio(ma_device* pDevice, void* pPCMFrames, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
37469 | {
37470 |     int result;
37471 |
37472 |     if (pFramesRead != NULL) {
37473 |         *pFramesRead = 0;
37474 |     }
37475 |
37476 |     result = ((ma_sio_read_proc)pDevice->pContext->sndio.sio_read)((struct
> ma_sio_hdl*)pDevice->sndio.handleCapture, pPCMFrames, frameCount *
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels));

```

```

37477 |     if (result == 0) {
37478 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[sndio] Failed to read data
> from the device to be sent to the device.");
37479 |         return MA_IO_ERROR;
37480 |     }
37481 |
37482 |     if (pFramesRead != NULL) {
37483 |         *pFramesRead = frameCount;
37484 |     }
37485 |
37486 |     return MA_SUCCESS;
37487 | }
37488 |
37489 | static ma_result ma_context_uninit__sndio(ma_context* pContext)
37490 | {
37491 |     MA_ASSERT(pContext != NULL);
37492 |     MA_ASSERT(pContext->backend == ma_backend_sndio);
37493 |
37494 |     (void)pContext;
37495 |     return MA_SUCCESS;
37496 | }
37497 |
37498 | static ma_result ma_context_init__sndio(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
37499 | {
37500 | #ifndef MA_NO_RUNTIME_LINKING
37501 |     const char* libsndioNames[] = {
37502 |         "libsndio.so"
37503 |     };
37504 |     size_t i;
37505 |
37506 |     for (i = 0; i < ma_countof(libsndioNames); ++i) {
37507 |         pContext->sndio.sndioSO = ma_dlopen(ma_context_get_log(pContext), libsndioNames[i]);
37508 |         if (pContext->sndio.sndioSO != NULL) {
37509 |             break;
37510 |         }
37511 |     }
37512 |
37513 |     if (pContext->sndio.sndioSO == NULL) {
37514 |         return MA_NO_BACKEND;
37515 |     }
37516 |
37517 |     pContext->sndio.sio_open = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_open");
37518 |     pContext->sndio.sio_close = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_close");
37519 |     pContext->sndio.sio_setpar = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_setpar");
37520 |     pContext->sndio.sio_getpar = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_getpar");
37521 |     pContext->sndio.sio_getcap = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_getcap");
37522 |     pContext->sndio.sio_write = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_write");
37523 |     pContext->sndio.sio_read = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_read");
37524 |     pContext->sndio.sio_start = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_start");
37525 |     pContext->sndio.sio_stop = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_stop");
37526 |     pContext->sndio.sio_initpar = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->sndio.sndioSO, "sio_initpar");
37527 | #else
37528 |     pContext->sndio.sio_open = sio_open;
37529 |     pContext->sndio.sio_close = sio_close;
37530 |     pContext->sndio.sio_setpar = sio_setpar;
37531 |     pContext->sndio.sio_getpar = sio_getpar;
37532 |     pContext->sndio.sio_getcap = sio_getcap;
37533 |     pContext->sndio.sio_write = sio_write;
37534 |     pContext->sndio.sio_read = sio_read;
37535 |     pContext->sndio.sio_start = sio_start;
37536 |     pContext->sndio.sio_stop = sio_stop;
37537 |     pContext->sndio.sio_initpar = sio_initpar;
37538 | #endif
37539 |
37540 |     pCallbacks->onContextInit = ma_context_init__sndio;
37541 |     pCallbacks->onContextUninit = ma_context_uninit__sndio;
37542 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__sndio;
37543 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__sndio;

```

```

37544 |     pCallbacks->onDeviceInit           = ma_device_init__sndio;
37545 |     pCallbacks->onDeviceUninit         = ma_device_uninit__sndio;
37546 |     pCallbacks->onDeviceStart          = ma_device_start__sndio;
37547 |     pCallbacks->onDeviceStop           = ma_device_stop__sndio;
37548 |     pCallbacks->onDeviceRead           = ma_device_read__sndio;
37549 |     pCallbacks->onDeviceWrite          = ma_device_write__sndio;
37550 |     pCallbacks->onDeviceDataLoop       = NULL;
37551 |
37552 |     (void)pConfig;
37553 |     return MA_SUCCESS;
37554 | }
37555 | #endif /* MA_HAS_SNDIO */
37556 |
37557 |
37558 |
37559 | /*****
37560 |
37561 | audio(4) Backend
37562 |
37563 | *****/
37564 | #ifdef MA_HAS_AUDIO4
37565 | #include <fcntl.h>
37566 | #include <poll.h>
37567 | #include <errno.h>
37568 | #include <sys/stat.h>
37569 | #include <sys/types.h>
37570 | #include <sys/ioctl.h>
37571 | #include <sys/audioio.h>
37572 |
37573 | #ifdef __NetBSD__
37574 | #include <sys/param.h>
37575 | #endif
37576 |
37577 | #if defined(__OpenBSD__)
37578 | #include <sys/param.h>
37579 | #if defined(OpenBSD) && OpenBSD >= 201709
37580 | #define MA_AUDIO4_USE_NEW_API
37581 | #endif
37582 | #endif
37583 |
37584 | static void ma_construct_device_id__audio4(char* id, size_t idSize, const char* base, int
> deviceIndex)
37585 | {
37586 |     size_t baseLen;
37587 |
37588 |     MA_ASSERT(id != NULL);
37589 |     MA_ASSERT(idSize > 0);
37590 |     MA_ASSERT(deviceIndex >= 0);
37591 |
37592 |     baseLen = strlen(base);
37593 |     MA_ASSERT(idSize > baseLen);
37594 |
37595 |     ma_strcpy_s(id, idSize, base);
37596 |     ma_itoa_s(deviceIndex, id+baseLen, idSize-baseLen, 10);
37597 | }
37598 |
37599 | static ma_result ma_extract_device_index_from_id__audio4(const char* id, const char* base, int*
> pIndexOut)
37600 | {
37601 |     size_t idLen;
37602 |     size_t baseLen;
37603 |     const char* deviceIndexStr;
37604 |
37605 |     MA_ASSERT(id != NULL);
37606 |     MA_ASSERT(base != NULL);
37607 |     MA_ASSERT(pIndexOut != NULL);
37608 |
37609 |     idLen = strlen(id);
37610 |     baseLen = strlen(base);
37611 |     if (idLen <= baseLen) {
37612 |         return MA_ERROR; /* Doesn't look like the id starts with the base. */
37613 |     }
37614 |
37615 |     if (strncmp(id, base, baseLen) != 0) {
37616 |         return MA_ERROR; /* ID does not begin with base. */
37617 |     }
37618 |
37619 |     deviceIndexStr = id + baseLen;
37620 |     if (deviceIndexStr[0] == '\\0') {

```

```

37621 |         return MA_ERROR;  /* No index specified in the ID. */
37622 |     }
37623 |
37624 |     if (pIndexOut) {
37625 |         *pIndexOut = atoi(deviceIndexStr);
37626 |     }
37627 |
37628 |     return MA_SUCCESS;
37629 | }
37630 |
37631 |
37632 | #if !defined(MA_AUDIO4_USE_NEW_API)  /* Old API */
37633 | static ma_format ma_format_from_encoding__audio4(unsigned int encoding, unsigned int precision)
37634 | {
37635 |     if (precision == 8 && (encoding == AUDIO_ENCODING_ULINEAR || encoding == AUDIO_ENCODING_ULINEAR
> || encoding == AUDIO_ENCODING_ULINEAR_LE || encoding == AUDIO_ENCODING_ULINEAR_BE)) {
37636 |         return ma_format_u8;
37637 |     } else {
37638 |         if (ma_is_little_endian() && encoding == AUDIO_ENCODING_SLINEAR_LE) {
37639 |             if (precision == 16) {
37640 |                 return ma_format_s16;
37641 |             } else if (precision == 24) {
37642 |                 return ma_format_s24;
37643 |             } else if (precision == 32) {
37644 |                 return ma_format_s32;
37645 |             }
37646 |         } else if (ma_is_big_endian() && encoding == AUDIO_ENCODING_SLINEAR_BE) {
37647 |             if (precision == 16) {
37648 |                 return ma_format_s16;
37649 |             } else if (precision == 24) {
37650 |                 return ma_format_s24;
37651 |             } else if (precision == 32) {
37652 |                 return ma_format_s32;
37653 |             }
37654 |         }
37655 |     }
37656 |
37657 |     return ma_format_unknown; /* Encoding not supported. */
37658 | }
37659 |
37660 | static void ma_encoding_from_format__audio4(ma_format format, unsigned int* pEncoding, unsigned int*
> pPrecision)
37661 | {
37662 |     MA_ASSERT(pEncoding != NULL);
37663 |     MA_ASSERT(pPrecision != NULL);
37664 |
37665 |     switch (format)
37666 |     {
37667 |         case ma_format_u8:
37668 |         {
37669 |             *pEncoding = AUDIO_ENCODING_ULINEAR;
37670 |             *pPrecision = 8;
37671 |         } break;
37672 |
37673 |         case ma_format_s24:
37674 |         {
37675 |             *pEncoding = (ma_is_little_endian()) ? AUDIO_ENCODING_SLINEAR_LE :
> AUDIO_ENCODING_SLINEAR_BE;
37676 |             *pPrecision = 24;
37677 |         } break;
37678 |
37679 |         case ma_format_s32:
37680 |         {
37681 |             *pEncoding = (ma_is_little_endian()) ? AUDIO_ENCODING_SLINEAR_LE :
> AUDIO_ENCODING_SLINEAR_BE;
37682 |             *pPrecision = 32;
37683 |         } break;
37684 |
37685 |         case ma_format_s16:
37686 |         case ma_format_f32:
37687 |         case ma_format_unknown:
37688 |         default:
37689 |         {
37690 |             *pEncoding = (ma_is_little_endian()) ? AUDIO_ENCODING_SLINEAR_LE :
> AUDIO_ENCODING_SLINEAR_BE;
37691 |             *pPrecision = 16;
37692 |         } break;
37693 |     }
37694 | }

```



```

37695 |
37696 | static ma_format ma_format_from_prinfo__audio4(struct audio_prinfo* prinfo)
37697 | {
37698 |     return ma_format_from_encoding__audio4(prinfo->encoding, prinfo->precision);
37699 | }
37700 |
37701 | static ma_format ma_best_format_from_fd__audio4(int fd, ma_format preferredFormat)
37702 | {
37703 |     audio_encoding_t encoding;
37704 |     ma_uint32 iFormat;
37705 |     int counter = 0;
37706 |
37707 |     /* First check to see if the preferred format is supported. */
37708 |     if (preferredFormat != ma_format_unknown) {
37709 |         counter = 0;
37710 |         for (;;) {
37711 |             MA_ZERO_OBJECT(&encoding);
37712 |             encoding.index = counter;
37713 |             if (ioctl(fd, AUDIO_GETENC, &encoding) < 0) {
37714 |                 break;
37715 |             }
37716 |
37717 |             if (preferredFormat == ma_format_from_encoding__audio4(encoding.encoding,
> encoding.precison)) {
37718 |                 return preferredFormat; /* Found the preferred format. */
37719 |             }
37720 |
37721 |             /* Getting here means this encoding does not match our preferred format so we need to
> more on to the next encoding. */
37722 |             counter += 1;
37723 |         }
37724 |     }
37725 |
37726 |     /* Getting here means our preferred format is not supported, so fall back to our standard
> priorities. */
37727 |     for (iFormat = 0; iFormat < ma_countof(g_maFormatPriorities); iFormat += 1) {
37728 |         ma_format format = g_maFormatPriorities[iFormat];
37729 |
37730 |         counter = 0;
37731 |         for (;;) {
37732 |             MA_ZERO_OBJECT(&encoding);
37733 |             encoding.index = counter;
37734 |             if (ioctl(fd, AUDIO_GETENC, &encoding) < 0) {
37735 |                 break;
37736 |             }
37737 |
37738 |             if (format == ma_format_from_encoding__audio4(encoding.encoding, encoding.precison)) {
37739 |                 return format; /* Found a workable format. */
37740 |             }
37741 |
37742 |             /* Getting here means this encoding does not match our preferred format so we need to
> more on to the next encoding. */
37743 |             counter += 1;
37744 |         }
37745 |     }
37746 |
37747 |     /* Getting here means not appropriate format was found. */
37748 |     return ma_format_unknown;
37749 | }
37750 | #else
37751 | static ma_format ma_format_from_swpar__audio4(struct audio_swpar* par)
37752 | {
37753 |     if (par->bits == 8 && par->bps == 1 && par->sig == 0) {
37754 |         return ma_format_u8;
37755 |     }
37756 |     if (par->bits == 16 && par->bps == 2 && par->sig == 1 && par->le == ma_is_little_endian()) {
37757 |         return ma_format_s16;
37758 |     }
37759 |     if (par->bits == 24 && par->bps == 3 && par->sig == 1 && par->le == ma_is_little_endian()) {
37760 |         return ma_format_s24;
37761 |     }
37762 |     if (par->bits == 32 && par->bps == 4 && par->sig == 1 && par->le == ma_is_little_endian()) {
37763 |         return ma_format_f32;
37764 |     }
37765 |
37766 |     /* Format not supported. */
37767 |     return ma_format_unknown;
37768 | }
37769 | #endif

```

```

37770 |
37771 | static ma_result ma_context_get_device_info_from_fd__audio4(ma_context* pContext, ma_device_type
> deviceType, int fd, ma_device_info* pDeviceInfo)
37772 | {
37773 |     audio_device_t fdDevice;
37774 |
37775 |     MA_ASSERT(pContext != NULL);
37776 |     MA_ASSERT(fd >= 0);
37777 |     MA_ASSERT(pDeviceInfo != NULL);
37778 |
37779 |     (void)pContext;
37780 |     (void)deviceType;
37781 |
37782 |     if (ioctl(fd, AUDIO_GETDEV, &fdDevice) < 0) {
37783 |         return MA_ERROR; /* Failed to retrieve device info. */
37784 |     }
37785 |
37786 |     /* Name. */
37787 |     ma_strcpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), fdDevice.name);
37788 |
37789 |     #if !defined(MA_AUDIO4_USE_NEW_API)
37790 |     {
37791 |         audio_info_t fdInfo;
37792 |         int counter = 0;
37793 |         ma_uint32 channels;
37794 |         ma_uint32 sampleRate;
37795 |
37796 |         #if defined(__NetBSD__) && (__NetBSD_Version__ >= 900000000)
37797 |             if (ioctl(fd, AUDIO_GETFORMAT, &fdInfo) < 0) {
37798 |                 return MA_ERROR;
37799 |             }
37800 |         #else
37801 |             if (ioctl(fd, AUDIO_GETINFO, &fdInfo) < 0) {
37802 |                 return MA_ERROR;
37803 |             }
37804 |         #endif
37805 |
37806 |         if (deviceType == ma_device_type_playback) {
37807 |             channels = fdInfo.play.channels;
37808 |             sampleRate = fdInfo.play.sample_rate;
37809 |         } else {
37810 |             channels = fdInfo.record.channels;
37811 |             sampleRate = fdInfo.record.sample_rate;
37812 |         }
37813 |
37814 |         /* Supported formats. We get this by looking at the encodings. */
37815 |         pDeviceInfo->nativeDataFormatCount = 0;
37816 |         for (;;) {
37817 |             audio_encoding_t encoding;
37818 |             ma_format format;
37819 |
37820 |             MA_ZERO_OBJECT(&encoding);
37821 |             encoding.index = counter;
37822 |             if (ioctl(fd, AUDIO_GETENC, &encoding) < 0) {
37823 |                 break;
37824 |             }
37825 |
37826 |             format = ma_format_from_encoding__audio4(encoding.encoding, encoding.precision);
37827 |             if (format != ma_format_unknown) {
37828 |                 ma_device_info_add_native_data_format(pDeviceInfo, format, channels, sampleRate, 0);
37829 |             }
37830 |
37831 |             counter += 1;
37832 |         }
37833 |     }
37834 |     #else
37835 |     {
37836 |         struct audio_swpar fdPar;
37837 |         ma_format format;
37838 |         ma_uint32 channels;
37839 |         ma_uint32 sampleRate;
37840 |
37841 |         if (ioctl(fd, AUDIO_GETPAR, &fdPar) < 0) {
37842 |             return MA_ERROR;
37843 |         }
37844 |
37845 |         format = ma_format_from_swpar__audio4(&fdPar);
37846 |         if (format == ma_format_unknown) {
37847 |             return MA_FORMAT_NOT_SUPPORTED;

```

```

37848 |     }
37849 |
37850 |     if (deviceType == ma_device_type_playback) {
37851 |         channels = fdPar.pchan;
37852 |     } else {
37853 |         channels = fdPar.rchan;
37854 |     }
37855 |
37856 |     sampleRate = fdPar.rate;
37857 |
37858 |     pDeviceInfo->nativeDataFormatCount = 0;
37859 |     ma_device_info_add_native_data_format(pDeviceInfo, format, channels, sampleRate, 0);
37860 | }
37861 | #endif
37862 |
37863 |     return MA_SUCCESS;
37864 | }
37865 |
37866 | static ma_result ma_context_enumerate_devices__audio4(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
37867 | {
37868 |     const int maxDevices = 64;
37869 |     char devpath[256];
37870 |     int iDevice;
37871 |
37872 |     MA_ASSERT(pContext != NULL);
37873 |     MA_ASSERT(callback != NULL);
37874 |
37875 |     /*
37876 |     Every device will be named "/dev/audioN", with a "/dev/audiocctlN" equivalent. We use the
> "/dev/audiocctlN"
37877 |     version here since we can open it even when another process has control of the "/dev/audioN"
> device.
37878 |     */
37879 |     for (iDevice = 0; iDevice < maxDevices; ++iDevice) {
37880 |         struct stat st;
37881 |         int fd;
37882 |         ma_bool32 isTerminating = MA_FALSE;
37883 |
37884 |         ma_strcpy_s(devpath, sizeof(devpath), "/dev/audiocctl");
37885 |         ma_itoa_s(iDevice, devpath+strlen(devpath), sizeof(devpath)-strlen(devpath), 10);
37886 |
37887 |         if (stat(devpath, &st) < 0) {
37888 |             break;
37889 |         }
37890 |
37891 |         /* The device exists, but we need to check if it's usable as playback and/or capture. */
37892 |
37893 |         /* Playback. */
37894 |         if (!isTerminating) {
37895 |             fd = open(devpath, O_RDONLY, 0);
37896 |             if (fd >= 0) {
37897 |                 /* Supports playback. */
37898 |                 ma_device_info deviceInfo;
37899 |                 MA_ZERO_OBJECT(&deviceInfo);
37900 |                 ma_construct_device_id__audio4(deviceInfo.id.audio4, sizeof(deviceInfo.id.audio4),
> "/dev/audio", iDevice);
37901 |                 if (ma_context_get_device_info_from_fd__audio4(pContext, ma_device_type_playback,
> fd, &deviceInfo) == MA_SUCCESS) {
37902 |                     isTerminating = !callback(pContext, ma_device_type_playback, &deviceInfo,
> pUserData);
37903 |                 }
37904 |
37905 |                 close(fd);
37906 |             }
37907 |         }
37908 |
37909 |         /* Capture. */
37910 |         if (!isTerminating) {
37911 |             fd = open(devpath, O_WRONLY, 0);
37912 |             if (fd >= 0) {
37913 |                 /* Supports capture. */
37914 |                 ma_device_info deviceInfo;
37915 |                 MA_ZERO_OBJECT(&deviceInfo);
37916 |                 ma_construct_device_id__audio4(deviceInfo.id.audio4, sizeof(deviceInfo.id.audio4),
> "/dev/audio", iDevice);
37917 |                 if (ma_context_get_device_info_from_fd__audio4(pContext, ma_device_type_capture, fd,
> &deviceInfo) == MA_SUCCESS) {
37918 |                     isTerminating = !callback(pContext, ma_device_type_capture, &deviceInfo,

```

```

    > pUserData);
37919 |         }
37920 |
37921 |         close(fd);
37922 |     }
37923 | }
37924 |
37925 |     if (isTerminating) {
37926 |         break;
37927 |     }
37928 | }
37929 |
37930 |     return MA_SUCCESS;
37931 | }
37932 |
37933 | static ma_result ma_context_get_device_info__audio4(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
37934 | {
37935 |     int fd = -1;
37936 |     int deviceIndex = -1;
37937 |     char ctlid[256];
37938 |     ma_result result;
37939 |
37940 |     MA_ASSERT(pContext != NULL);
37941 |
37942 |     /*
37943 |     We need to open the "/dev/audiocctlN" device to get the info. To do this we need to extract the
> number
37944 |     from the device ID which will be in "/dev/audioN" format.
37945 |     */
37946 |     if (pDeviceID == NULL) {
37947 |         /* Default device. */
37948 |         ma_strcpy_s(ctlid, sizeof(ctlid), "/dev/audiocctl");
37949 |     } else {
37950 |         /* Specific device. We need to convert from "/dev/audioN" to "/dev/audiocctlN". */
37951 |         result = ma_extract_device_index_from_id__audio4(pDeviceID->audio4, "/dev/audio",
> &deviceIndex);
37952 |         if (result != MA_SUCCESS) {
37953 |             return result;
37954 |         }
37955 |
37956 |         ma_construct_device_id__audio4(ctlid, sizeof(ctlid), "/dev/audiocctl", deviceIndex);
37957 |     }
37958 |
37959 |     fd = open(ctlid, (deviceType == ma_device_type_playback) ? O_WRONLY : O_RDONLY, 0);
37960 |     if (fd == -1) {
37961 |         return MA_NO_DEVICE;
37962 |     }
37963 |
37964 |     if (deviceIndex == -1) {
37965 |         ma_strcpy_s(pDeviceInfo->id.audio4, sizeof(pDeviceInfo->id.audio4), "/dev/audio");
37966 |     } else {
37967 |         ma_construct_device_id__audio4(pDeviceInfo->id.audio4, sizeof(pDeviceInfo->id.audio4),
> "/dev/audio", deviceIndex);
37968 |     }
37969 |
37970 |     result = ma_context_get_device_info_from_fd__audio4(pContext, deviceType, fd, pDeviceInfo);
37971 |
37972 |     close(fd);
37973 |     return result;
37974 | }
37975 |
37976 | static ma_result ma_device_uninit__audio4(ma_device* pDevice)
37977 | {
37978 |     MA_ASSERT(pDevice != NULL);
37979 |
37980 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
37981 |         close(pDevice->audio4.fdCapture);
37982 |     }
37983 |
37984 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
37985 |         close(pDevice->audio4.fdPlayback);
37986 |     }
37987 |
37988 |     return MA_SUCCESS;
37989 | }
37990 |
37991 | static ma_result ma_device_init_fd__audio4(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptor, ma_device_type deviceType)

```

```

37992 | {
37993 |     const char* pDefaultDeviceNames[] = {
37994 |         "/dev/audio",
37995 |         "/dev/audio0"
37996 |     };
37997 |     const char* pDefaultDeviceCtlNames[] = {
37998 |         "/dev/audiocctl",
37999 |         "/dev/audiocctl0"
38000 |     };
38001 |     int fd;
38002 |     int fdFlags = 0;
38003 |     size_t iDefaultDevice = (size_t)-1;
38004 |     ma_format internalFormat;
38005 |     ma_uint32 internalChannels;
38006 |     ma_uint32 internalSampleRate;
38007 |     ma_uint32 internalPeriodSizeInFrames;
38008 |     ma_uint32 internalPeriods;
38009 |
38010 |     MA_ASSERT(pConfig != NULL);
38011 |     MA_ASSERT(deviceType != ma_device_type_duplex);
38012 |     MA_ASSERT(pDevice != NULL);
38013 |
38014 |     /* The first thing to do is open the file. */
38015 |     if (deviceType == ma_device_type_capture) {
38016 |         fdFlags = O_RDONLY;
38017 |     } else {
38018 |         fdFlags = O_WRONLY;
38019 |     }
38020 |     /*fdFlags |= O_NONBLOCK;*/
38021 |
38022 |     /* Find the index of the default device as a start. We'll use this index later. Set it to
> (size_t)-1 otherwise. */
38023 |     if (pDescriptor->pDeviceID == NULL) {
38024 |         /* Default device. */
38025 |         for (iDefaultDevice = 0; iDefaultDevice < ma_countof(pDefaultDeviceNames); ++iDefaultDevice)
> {
38026 |             fd = open(pDefaultDeviceNames[iDefaultDevice], fdFlags, 0);
38027 |             if (fd != -1) {
38028 |                 break;
38029 |             }
38030 |         }
38031 |     } else {
38032 |         /* Specific device. */
38033 |         fd = open(pDescriptor->pDeviceID->audio4, fdFlags, 0);
38034 |
38035 |         for (iDefaultDevice = 0; iDefaultDevice < ma_countof(pDefaultDeviceNames); iDefaultDevice +=
> 1) {
38036 |             if (ma_strcmp(pDefaultDeviceNames[iDefaultDevice], pDescriptor->pDeviceID->audio4) == 0)
> {
38037 |                 break;
38038 |             }
38039 |         }
38040 |
38041 |         if (iDefaultDevice == ma_countof(pDefaultDeviceNames)) {
38042 |             iDefaultDevice = (size_t)-1;
38043 |         }
38044 |     }
38045 |
38046 |     if (fd == -1) {
38047 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to open
> device.");
38048 |         return ma_result_from_errno(errno);
38049 |     }
38050 |
38051 |     #if !defined(MA_AUDIOI04_USE_NEW_API) /* Old API */
38052 |     {
38053 |         audio_info_t fdInfo;
38054 |         int fdInfoResult = -1;
38055 |
38056 |         /*
38057 |         The documentation is a little bit unclear to me as to how it handles formats. It says the
38058 |         following:
38059 |
38060 |         Regardless of formats supported by underlying driver, the audio driver accepts the
38061 |         following formats.
38062 |
38063 |         By then the next sentence says this:
38064 |
38065 |         `encoding` and `precision` are one of the values obtained by AUDIO_GETENC.

```

```

38066 |
38067 |         It sounds like a direct contradiction to me. I'm going to play this safe any only use the
38068 |         best sample format returned by AUDIO_GETENC. If the requested format is supported we'll
38069 |         use that, but otherwise we'll just use our standard format priorities to pick an
38070 |         appropriate one.
38071 |         */
38072 |         AUDIO_INITINFO(&fdInfo);
38073 |
38074 |         /*
38075 |         Get the default format from the audioctl file if we're asking for a default device. If we
38076 |         retrieve it from /dev/audio it'll default to mono 8000Hz.
38077 |         */
38078 |         if (iDefaultDevice != (size_t)-1) {
38079 |             /* We're using a default device. Get the info from the /dev/audioctl file instead of
> /dev/audio. */
38080 |             int fdctl = open(pDefaultDeviceCtlNames[iDefaultDevice], fdFlags, 0);
38081 |             if (fdctl != -1) {
38082 |                 #if defined(__NetBSD__) && (__NetBSD_Version__ >= 900000000)
38083 |                     fdInfoResult = ioctl(fdctl, AUDIO_GETFORMAT, &fdInfo);
38084 |                 #else
38085 |                     fdInfoResult = ioctl(fdctl, AUDIO_GETINFO, &fdInfo);
38086 |                 #endif
38087 |                 close(fdctl);
38088 |             }
38089 |         }
38090 |
38091 |         if (fdInfoResult == -1) {
38092 |             /* We still don't have the default device info so just retrieve it from the main audio
> device. */
38093 |             if (ioctl(fd, AUDIO_GETINFO, &fdInfo) < 0) {
38094 |                 close(fd);
38095 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] AUDIO_GETINFO
> failed.");
38096 |                 return ma_result_from_errno(errno);
38097 |             }
38098 |         }
38099 |
38100 |         /* We get the driver to do as much of the data conversion as possible. */
38101 |         if (deviceType == ma_device_type_capture) {
38102 |             fdInfo.mode = AUMODE_RECORD;
38103 |             ma_encoding_from_format__audio4(ma_best_format_from_fd__audio4(fd, pDescriptor->format),
> &fdInfo.record.encoding, &fdInfo.record.precision);
38104 |
38105 |             if (pDescriptor->channels != 0) {
38106 |                 fdInfo.record.channels = ma_clamp(pDescriptor->channels, 1, 12); /* From the
> documentation: `channels` ranges from 1 to 12. */
38107 |             }
38108 |
38109 |             if (pDescriptor->sampleRate != 0) {
38110 |                 fdInfo.record.sample_rate = ma_clamp(pDescriptor->sampleRate, 1000, 192000); /*
> From the documentation: `frequency` ranges from 1000Hz to 192000Hz. (They mean `sample_rate` instead
> of `frequency`.) */
38111 |             }
38112 |             } else {
38113 |                 fdInfo.mode = AUMODE_PLAY;
38114 |                 ma_encoding_from_format__audio4(ma_best_format_from_fd__audio4(fd, pDescriptor->format),
> &fdInfo.play.encoding, &fdInfo.play.precision);
38115 |
38116 |                 if (pDescriptor->channels != 0) {
38117 |                     fdInfo.play.channels = ma_clamp(pDescriptor->channels, 1, 12); /* From the
> documentation: `channels` ranges from 1 to 12. */
38118 |                 }
38119 |
38120 |                 if (pDescriptor->sampleRate != 0) {
38121 |                     fdInfo.play.sample_rate = ma_clamp(pDescriptor->sampleRate, 1000, 192000); /*
> From the documentation: `frequency` ranges from 1000Hz to 192000Hz. (They mean `sample_rate` instead
> of `frequency`.) */
38122 |                 }
38123 |             }
38124 |
38125 |             if (ioctl(fd, AUDIO_SETINFO, &fdInfo) < 0) {
38126 |                 close(fd);
38127 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to set
> device format. AUDIO_SETINFO failed.");
38128 |                 return ma_result_from_errno(errno);
38129 |             }
38130 |
38131 |             if (ioctl(fd, AUDIO_GETINFO, &fdInfo) < 0) {
38132 |                 close(fd);

```

```

38133 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] AUDIO_GETINFO
> failed.");
38134 |         return ma_result_from_errno(errno);
38135 |     }
38136 |
38137 |     if (deviceType == ma_device_type_capture) {
38138 |         internalFormat = ma_format_from_prinfo_audio4(&fdInfo.record);
38139 |         internalChannels = fdInfo.record.channels;
38140 |         internalSampleRate = fdInfo.record.sample_rate;
38141 |     } else {
38142 |         internalFormat = ma_format_from_prinfo_audio4(&fdInfo.play);
38143 |         internalChannels = fdInfo.play.channels;
38144 |         internalSampleRate = fdInfo.play.sample_rate;
38145 |     }
38146 |
38147 |     if (internalFormat == ma_format_unknown) {
38148 |         close(fd);
38149 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] The device's
> internal device format is not supported by miniaudio. The device is unusable.");
38150 |         return MA_FORMAT_NOT_SUPPORTED;
38151 |     }
38152 |
38153 |     /* Buffer. */
38154 |     {
38155 |         ma_uint32 internalPeriodSizeInBytes;
38156 |
38157 |         internalPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor, internalSampleRate,
> pConfig->performanceProfile);
38158 |
38159 |         internalPeriodSizeInBytes = internalPeriodSizeInFrames *
> ma_get_bytes_per_frame(internalFormat, internalChannels);
38160 |         if (internalPeriodSizeInBytes < 16) {
38161 |             internalPeriodSizeInBytes = 16;
38162 |         }
38163 |
38164 |         internalPeriods = pDescriptor->periodCount;
38165 |         if (internalPeriods < 2) {
38166 |             internalPeriods = 2;
38167 |         }
38168 |
38169 |         /* What miniaudio calls a period, audio4 calls a block. */
38170 |         AUDIO_INITINFO(&fdInfo);
38171 |         fdInfo.hiwat = internalPeriods;
38172 |         fdInfo.lowat = internalPeriods-1;
38173 |         fdInfo.blocksize = internalPeriodSizeInBytes;
38174 |         if (ioctl(fd, AUDIO_SETINFO, &fdInfo) < 0) {
38175 |             close(fd);
38176 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to set
> internal buffer size. AUDIO_SETINFO failed.");
38177 |             return ma_result_from_errno(errno);
38178 |         }
38179 |
38180 |         internalPeriods = fdInfo.hiwat;
38181 |         internalPeriodSizeInFrames = fdInfo.blocksize / ma_get_bytes_per_frame(internalFormat,
> internalChannels);
38182 |     }
38183 | }
38184 | #else
38185 | {
38186 |     struct audio_swpar fdPar;
38187 |
38188 |     /* We need to retrieve the format of the device so we can know the channel count and sample
> rate. Then we can calculate the buffer size. */
38189 |     if (ioctl(fd, AUDIO_GETPAR, &fdPar) < 0) {
38190 |         close(fd);
38191 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to retrieve
> initial device parameters.");
38192 |         return ma_result_from_errno(errno);
38193 |     }
38194 |
38195 |     internalFormat = ma_format_from_swpar_audio4(&fdPar);
38196 |     internalChannels = (deviceType == ma_device_type_capture) ? fdPar.rchan : fdPar.pchan;
38197 |     internalSampleRate = fdPar.rate;
38198 |
38199 |     if (internalFormat == ma_format_unknown) {
38200 |         close(fd);
38201 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] The device's
> internal device format is not supported by miniaudio. The device is unusable.");

```



```

38202 |         return MA_FORMAT_NOT_SUPPORTED;
38203 |     }
38204 |
38205 |     /* Buffer. */
38206 |     {
38207 |         ma_uint32 internalPeriodSizeInBytes;
38208 |
38209 |         internalPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor, internalSampleRate,
> pConfig->performanceProfile);
38210 |
38211 |         /* What miniaudio calls a period, audio4 calls a block. */
38212 |         internalPeriodSizeInBytes = internalPeriodSizeInFrames *
> ma_get_bytes_per_frame(internalFormat, internalChannels);
38213 |         if (internalPeriodSizeInBytes < 16) {
38214 |             internalPeriodSizeInBytes = 16;
38215 |         }
38216 |
38217 |         fdPar.nblks = pDescriptor->periodCount;
38218 |         fdPar.round = internalPeriodSizeInBytes;
38219 |
38220 |         if (ioctl(fd, AUDIO_SETPAR, &fdPar) < 0) {
38221 |             close(fd);
38222 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to set
> device parameters.");
38223 |             return ma_result_from_errno(errno);
38224 |         }
38225 |
38226 |         if (ioctl(fd, AUDIO_GETPAR, &fdPar) < 0) {
38227 |             close(fd);
38228 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to
> retrieve actual device parameters.");
38229 |             return ma_result_from_errno(errno);
38230 |         }
38231 |     }
38232 |
38233 |     internalFormat          = ma_format_from_swpar__audio4(&fdPar);
38234 |     internalChannels        = (deviceType == ma_device_type_capture) ? fdPar.rchan :
> fdPar.pchan;
38235 |     internalSampleRate      = fdPar.rate;
38236 |     internalPeriods        = fdPar.nblks;
38237 |     internalPeriodSizeInFrames = fdPar.round / ma_get_bytes_per_frame(internalFormat,
> internalChannels);
38238 |     }
38239 |     #endif
38240 |
38241 |     if (internalFormat == ma_format_unknown) {
38242 |         close(fd);
38243 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] The device's internal
> device format is not supported by miniaudio. The device is unusable.");
38244 |         return MA_FORMAT_NOT_SUPPORTED;
38245 |     }
38246 |
38247 |     if (deviceType == ma_device_type_capture) {
38248 |         pDevice->audio4.fdCapture = fd;
38249 |     } else {
38250 |         pDevice->audio4.fdPlayback = fd;
38251 |     }
38252 |
38253 |     pDescriptor->format          = internalFormat;
38254 |     pDescriptor->channels        = internalChannels;
38255 |     pDescriptor->sampleRate      = internalSampleRate;
38256 |     ma_channel_map_init_standard(ma_standard_channel_map_sound4, pDescriptor->channelMap,
> ma_countof(pDescriptor->channelMap), internalChannels);
38257 |     pDescriptor->periodSizeInFrames = internalPeriodSizeInFrames;
38258 |     pDescriptor->periodCount      = internalPeriods;
38259 |
38260 |     return MA_SUCCESS;
38261 | }
38262 |
38263 | static ma_result ma_device_init__audio4(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
38264 | {
38265 |     MA_ASSERT(pDevice != NULL);
38266 |
38267 |     MA_ZERO_OBJECT(&pDevice->audio4);
38268 |
38269 |     if (pConfig->deviceType == ma_device_type_loopback) {
38270 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;

```

```

38271 |     }
38272 |
38273 |     pDevice->audio4.fdCapture = -1;
38274 |     pDevice->audio4.fdPlayback = -1;
38275 |
38276 |     /*
38277 |     The version of the operating system dictates whether or not the device is exclusive or shared.
> NetBSD
38278 |     introduced in-kernel mixing which means it's shared. All other BSD flavours are exclusive as far
> as
38279 |     I'm aware.
38280 |     */
38281 |     #if defined(__NetBSD_Version__) && __NetBSD_Version__ >= 800000000
38282 |     /* NetBSD 8.0+ */
38283 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->shareMode == ma_share_mode_exclusive) ||
38284 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->shareMode == ma_share_mode_exclusive)) {
38285 |         return MA_SHARE_MODE_NOT_SUPPORTED;
38286 |     }
38287 |     #else
38288 |     /* All other flavors. */
38289 |     #endif
38290 |
38291 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
38292 |         ma_result result = ma_device_init_fd__audio4(pDevice, pConfig, pDescriptorCapture,
> ma_device_type_capture);
38293 |         if (result != MA_SUCCESS) {
38294 |             return result;
38295 |         }
38296 |     }
38297 |
38298 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
38299 |         ma_result result = ma_device_init_fd__audio4(pDevice, pConfig, pDescriptorPlayback,
> ma_device_type_playback);
38300 |         if (result != MA_SUCCESS) {
38301 |             if (pConfig->deviceType == ma_device_type_duplex) {
38302 |                 close(pDevice->audio4.fdCapture);
38303 |             }
38304 |             return result;
38305 |         }
38306 |     }
38307 |
38308 |     return MA_SUCCESS;
38309 | }
38310 |
38311 | static ma_result ma_device_start__audio4(ma_device* pDevice)
38312 | {
38313 |     MA_ASSERT(pDevice != NULL);
38314 |
38315 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
38316 |         if (pDevice->audio4.fdCapture == -1) {
38317 |             return MA_INVALID_ARGS;
38318 |         }
38319 |     }
38320 |
38321 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
38322 |         if (pDevice->audio4.fdPlayback == -1) {
38323 |             return MA_INVALID_ARGS;
38324 |         }
38325 |     }
38326 |
38327 |     return MA_SUCCESS;
38328 | }
38329 |
38330 | static ma_result ma_device_stop_fd__audio4(ma_device* pDevice, int fd)
38331 | {
38332 |     if (fd == -1) {
38333 |         return MA_INVALID_ARGS;
38334 |     }
38335 |
38336 |     #if !defined(MA_AUDIO4_USE_NEW_API)
38337 |     if (ioctl(fd, AUDIO_FLUSH, 0) < 0) {
38338 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to stop device.
> AUDIO_FLUSH failed.");
38339 |         return ma_result_from_errno(errno);
38340 |     }

```

```

38341 | #else
38342 |     if (ioctl(fd, AUDIO_STOP, 0) < 0) {
38343 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to stop device.
> AUDIO_STOP failed.");
38344 |         return ma_result_from_errno(errno);
38345 |     }
38346 | #endif
38347 |
38348 |     return MA_SUCCESS;
38349 | }
38350 |
38351 | static ma_result ma_device_stop__audio4(ma_device* pDevice)
38352 | {
38353 |     MA_ASSERT(pDevice != NULL);
38354 |
38355 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
38356 |         ma_result result;
38357 |
38358 |         result = ma_device_stop_fd__audio4(pDevice, pDevice->audio4.fdCapture);
38359 |         if (result != MA_SUCCESS) {
38360 |             return result;
38361 |         }
38362 |     }
38363 |
38364 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
38365 |         ma_result result;
38366 |
38367 |         /* Drain the device first. If this fails we'll just need to flush without draining.
> Unfortunately draining isn't available on newer version of OpenBSD. */
38368 |         #if !defined(MA_AUDIO4_USE_NEW_API)
38369 |             ioctl(pDevice->audio4.fdPlayback, AUDIO_DRAIN, 0);
38370 |         #endif
38371 |
38372 |         /* Here is where the device is stopped immediately. */
38373 |         result = ma_device_stop_fd__audio4(pDevice, pDevice->audio4.fdPlayback);
38374 |         if (result != MA_SUCCESS) {
38375 |             return result;
38376 |         }
38377 |     }
38378 |
38379 |     return MA_SUCCESS;
38380 | }
38381 |
38382 | static ma_result ma_device_write__audio4(ma_device* pDevice, const void* pPCMFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
38383 | {
38384 |     int result;
38385 |
38386 |     if (pFramesWritten != NULL) {
38387 |         *pFramesWritten = 0;
38388 |     }
38389 |
38390 |     result = write(pDevice->audio4.fdPlayback, pPCMFrames, frameCount *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels));
38391 |     if (result < 0) {
38392 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to write data
> to the device.");
38393 |         return ma_result_from_errno(errno);
38394 |     }
38395 |
38396 |     if (pFramesWritten != NULL) {
38397 |         *pFramesWritten = (ma_uint32)result /
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
38398 |     }
38399 |
38400 |     return MA_SUCCESS;
38401 | }
38402 |
38403 | static ma_result ma_device_read__audio4(ma_device* pDevice, void* pPCMFrames, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
38404 | {
38405 |     int result;
38406 |
38407 |     if (pFramesRead != NULL) {
38408 |         *pFramesRead = 0;
38409 |     }
38410 |
38411 |     result = read(pDevice->audio4.fdCapture, pPCMFrames, frameCount *
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels));

```

```

38412 |     if (result < 0) {
38413 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[audio4] Failed to read data
> from the device.");
38414 |         return ma_result_from_errno(errno);
38415 |     }
38416 |
38417 |     if (pFramesRead != NULL) {
38418 |         *pFramesRead = (ma_uint32)result / ma_get_bytes_per_frame(pDevice->capture.internalFormat,
> pDevice->capture.internalChannels);
38419 |     }
38420 |
38421 |     return MA_SUCCESS;
38422 | }
38423 |
38424 | static ma_result ma_context_uninit__audio4(ma_context* pContext)
38425 | {
38426 |     MA_ASSERT(pContext != NULL);
38427 |     MA_ASSERT(pContext->backend == ma_backend_audio4);
38428 |
38429 |     (void)pContext;
38430 |     return MA_SUCCESS;
38431 | }
38432 |
38433 | static ma_result ma_context_init__audio4(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
38434 | {
38435 |     MA_ASSERT(pContext != NULL);
38436 |
38437 |     (void)pConfig;
38438 |
38439 |     pCallbacks->onContextInit          = ma_context_init__audio4;
38440 |     pCallbacks->onContextUninit        = ma_context_uninit__audio4;
38441 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__audio4;
38442 |     pCallbacks->onContextGetDeviceInfo  = ma_context_get_device_info__audio4;
38443 |     pCallbacks->onDeviceInit           = ma_device_init__audio4;
38444 |     pCallbacks->onDeviceUninit         = ma_device_uninit__audio4;
38445 |     pCallbacks->onDeviceStart          = ma_device_start__audio4;
38446 |     pCallbacks->onDeviceStop           = ma_device_stop__audio4;
38447 |     pCallbacks->onDeviceRead           = ma_device_read__audio4;
38448 |     pCallbacks->onDeviceWrite          = ma_device_write__audio4;
38449 |     pCallbacks->onDeviceDataLoop       = NULL;
38450 |
38451 |     return MA_SUCCESS;
38452 | }
38453 | #endif /* MA_HAS_AUDIO4 */
38454 |
38455 |
38456 | /*****
38457 |
38458 | OSS Backend
38459 |
38460 | *****/
38461 | #ifdef MA_HAS_OSS
38462 | #include <sys/ioctl.h>
38463 | #include <unistd.h>
38464 | #include <fcntl.h>
38465 | #include <sys/soundcard.h>
38466 |
38467 | #ifndef SNDCTL_DSP_HALT
38468 | #define SNDCTL_DSP_HALT SNDCTL_DSP_RESET
38469 | #endif
38470 |
38471 | #define MA_OSS_DEFAULT_DEVICE_NAME  "/dev/dsp"
38472 |
38473 | static int ma_open_temp_device__oss(void)
38474 | {
38475 |     /* The OSS sample code uses "/dev/mixer" as the device for getting system properties so I'm
> going to do the same. */
38476 |     int fd = open("/dev/mixer", O_RDONLY, 0);
38477 |     if (fd >= 0) {
38478 |         return fd;
38479 |     }
38480 |
38481 |     return -1;
38482 | }
38483 |
38484 | static ma_result ma_context_open_device__oss(ma_context* pContext, ma_device_type deviceType, const
> ma_device_id* pDeviceID, ma_share_mode shareMode, int* pfd)
38485 | {

```

```

38486 |     const char* deviceName;
38487 |     int flags;
38488 |
38489 |     MA_ASSERT(pContext != NULL);
38490 |     MA_ASSERT(pfd != NULL);
38491 |     (void)pContext;
38492 |
38493 |     *pfd = -1;
38494 |
38495 |     /* This function should only be called for playback or capture, not duplex. */
38496 |     if (deviceType == ma_device_type_duplex) {
38497 |         return MA_INVALID_ARGS;
38498 |     }
38499 |
38500 |     deviceName = MA_OSS_DEFAULT_DEVICE_NAME;
38501 |     if (pDeviceID != NULL) {
38502 |         deviceName = pDeviceID->oss;
38503 |     }
38504 |
38505 |     flags = (deviceType == ma_device_type_playback) ? O_WRONLY : O_RDONLY;
38506 |     if (shareMode == ma_share_mode_exclusive) {
38507 |         flags |= O_EXCL;
38508 |     }
38509 |
38510 |     *pfd = open(deviceName, flags, 0);
38511 |     if (*pfd == -1) {
38512 |         return ma_result_from_errno(errno);
38513 |     }
38514 |
38515 |     return MA_SUCCESS;
38516 | }
38517 |
38518 | static ma_result ma_context_enumerate_devices__oss(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
38519 | {
38520 |     int fd;
38521 |     oss_sysinfo si;
38522 |     int result;
38523 |
38524 |     MA_ASSERT(pContext != NULL);
38525 |     MA_ASSERT(callback != NULL);
38526 |
38527 |     fd = ma_open_temp_device__oss();
38528 |     if (fd == -1) {
38529 |         ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[OSS] Failed to open a
> temporary device for retrieving system information used for device enumeration.");
38530 |         return MA_NO_BACKEND;
38531 |     }
38532 |
38533 |     result = ioctl(fd, SNDCTL_SYSINFO, &si);
38534 |     if (result != -1) {
38535 |         int iAudioDevice;
38536 |         for (iAudioDevice = 0; iAudioDevice < si.numaudios; ++iAudioDevice) {
38537 |             oss_audioinfo ai;
38538 |             ai.dev = iAudioDevice;
38539 |             result = ioctl(fd, SNDCTL_AUDIOINFO, &ai);
38540 |             if (result != -1) {
38541 |                 if (ai.devnode[0] != '\0') { /* <-- Can be blank, according to documentation. */
38542 |                     ma_device_info deviceInfo;
38543 |                     ma_bool32 isTerminating = MA_FALSE;
38544 |
38545 |                     MA_ZERO_OBJECT(&deviceInfo);
38546 |
38547 |                     /* ID */
38548 |                     ma_strncpy_s(deviceInfo.id.oss, sizeof(deviceInfo.id.oss), ai.devnode,
> (size_t)-1);
38549 |
38550 |                     /*
38551 |                     The human readable device name should be in the "ai.handle" variable, but it can
38552 |                     sometimes be empty in which case we just fall back to "ai.name" which is less
> user
38553 |                     friendly, but usually has a value.
38554 |                     */
38555 |                     if (ai.handle[0] != '\0') {
38556 |                         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), ai.handle,
> (size_t)-1);
38557 |                     } else {
38558 |                         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), ai.name, (size_t)-1);
38559 |                     }

```

```

38560 |
38561 |         /* The device can be both playback and capture. */
38562 |         if (!isTerminating && (ai.caps & PCM_CAP_OUTPUT) != 0) {
38563 |             isTerminating = !callback(pContext, ma_device_type_playback, &deviceInfo,
> pUserData);
38564 |         }
38565 |         if (!isTerminating && (ai.caps & PCM_CAP_INPUT) != 0) {
38566 |             isTerminating = !callback(pContext, ma_device_type_capture, &deviceInfo,
> pUserData);
38567 |         }
38568 |
38569 |         if (isTerminating) {
38570 |             break;
38571 |         }
38572 |     }
38573 | }
38574 | }
38575 | } else {
38576 |     close(fd);
38577 |     ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[OSS] Failed to retrieve
> system information for device enumeration.");
38578 |     return MA_NO_BACKEND;
38579 | }
38580 |
38581 | close(fd);
38582 | return MA_SUCCESS;
38583 | }
38584 |
38585 | static void ma_context_add_native_data_format_oss(ma_context* pContext, oss_audioinfo* pAudioInfo,
> ma_format format, ma_device_info* pDeviceInfo)
38586 | {
38587 |     unsigned int minChannels;
38588 |     unsigned int maxChannels;
38589 |     unsigned int iRate;
38590 |
38591 |     MA_ASSERT(pContext != NULL);
38592 |     MA_ASSERT(pAudioInfo != NULL);
38593 |     MA_ASSERT(pDeviceInfo != NULL);
38594 |
38595 |     /* If we support all channels we just report 0. */
38596 |     minChannels = ma_clamp(pAudioInfo->min_channels, MA_MIN_CHANNELS, MA_MAX_CHANNELS);
38597 |     maxChannels = ma_clamp(pAudioInfo->max_channels, MA_MIN_CHANNELS, MA_MAX_CHANNELS);
38598 |
38599 |     /*
38600 |     OSS has this annoying thing where sample rates can be reported in two ways. We prefer
> explicitness,
38601 |     which OSS has in the form of nrates/rates, however there are times where nrates can be 0, in
> which
38602 |     case we'll need to use min_rate and max_rate and report only standard rates.
38603 |     */
38604 |     if (pAudioInfo->nrates > 0) {
38605 |         for (iRate = 0; iRate < pAudioInfo->nrates; iRate += 1) {
38606 |             unsigned int rate = pAudioInfo->rates[iRate];
38607 |
38608 |             if (minChannels == MA_MIN_CHANNELS && maxChannels == MA_MAX_CHANNELS) {
38609 |                 ma_device_info_add_native_data_format(pDeviceInfo, format, 0, rate, 0); /* Set the
> channel count to 0 to indicate that all channel counts are supported. */
38610 |             } else {
38611 |                 unsigned int iChannel;
38612 |                 for (iChannel = minChannels; iChannel <= maxChannels; iChannel += 1) {
38613 |                     ma_device_info_add_native_data_format(pDeviceInfo, format, iChannel, rate, 0);
38614 |                 }
38615 |             }
38616 |         }
38617 |     } else {
38618 |         for (iRate = 0; iRate < ma_countof(g_maStandardSampleRatePriorities); iRate += 1) {
38619 |             ma_uint32 standardRate = g_maStandardSampleRatePriorities[iRate];
38620 |
38621 |             if (standardRate >= (ma_uint32)pAudioInfo->min_rate && standardRate <=
> (ma_uint32)pAudioInfo->max_rate) {
38622 |                 if (minChannels == MA_MIN_CHANNELS && maxChannels == MA_MAX_CHANNELS) {
38623 |                     ma_device_info_add_native_data_format(pDeviceInfo, format, 0, standardRate, 0);
> /* Set the channel count to 0 to indicate that all channel counts are supported. */
38624 |                 } else {
38625 |                     unsigned int iChannel;
38626 |                     for (iChannel = minChannels; iChannel <= maxChannels; iChannel += 1) {
38627 |                         ma_device_info_add_native_data_format(pDeviceInfo, format, iChannel,
> standardRate, 0);
38628 |                     }

```

```

38629 |         }
38630 |     }
38631 | }
38632 | }
38633 | }
38634 |
38635 | static ma_result ma_context_get_device_info__oss(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
38636 | {
38637 |     ma_bool32 foundDevice;
38638 |     int fdTemp;
38639 |     oss_sysinfo si;
38640 |     int result;
38641 |
38642 |     MA_ASSERT(pContext != NULL);
38643 |
38644 |     /* Handle the default device a little differently. */
38645 |     if (pDeviceID == NULL) {
38646 |         if (deviceType == ma_device_type_playback) {
38647 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name),
> MA_DEFAULT_PLAYBACK_DEVICE_NAME, (size_t)-1);
38648 |         } else {
38649 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name),
> MA_DEFAULT_CAPTURE_DEVICE_NAME, (size_t)-1);
38650 |         }
38651 |
38652 |         return MA_SUCCESS;
38653 |     }
38654 |
38655 |
38656 |     /* If we get here it means we are _not_ using the default device. */
38657 |     foundDevice = MA_FALSE;
38658 |
38659 |     fdTemp = ma_open_temp_device__oss();
38660 |     if (fdTemp == -1) {
38661 |         ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[OSS] Failed to open a
> temporary device for retrieving system information used for device enumeration.");
38662 |         return MA_NO_BACKEND;
38663 |     }
38664 |
38665 |     result = ioctl(fdTemp, SNDCTL_SYSINFO, &si);
38666 |     if (result != -1) {
38667 |         int iAudioDevice;
38668 |         for (iAudioDevice = 0; iAudioDevice < si.numaudios; ++iAudioDevice) {
38669 |             oss_audioinfo ai;
38670 |             ai.dev = iAudioDevice;
38671 |             result = ioctl(fdTemp, SNDCTL_AUDIOINFO, &ai);
38672 |             if (result != -1) {
38673 |                 if (ma_strcmp(ai.devnode, pDeviceID->oss) == 0) {
38674 |                     /* It has the same name, so now just confirm the type. */
38675 |                     if ((deviceType == ma_device_type_playback && ((ai.caps & PCM_CAP_OUTPUT) != 0))
> ||
38676 |                         (deviceType == ma_device_type_capture && ((ai.caps & PCM_CAP_INPUT) !=
> 0))) {
38677 |                         unsigned int formatMask;
38678 |
38679 |                         /* ID */
38680 |                         ma_strncpy_s(pDeviceInfo->id.oss, sizeof(pDeviceInfo->id.oss), ai.devnode,
> (size_t)-1);
38681 |
38682 |                         /*
38683 |                         The human readable device name should be in the "ai.handle" variable, but it
> can
38684 |                         sometimes be empty in which case we just fall back to "ai.name" which is
> less user
38685 |                         friendly, but usually has a value.
38686 |                         */
38687 |                         if (ai.handle[0] != '\0') {
38688 |                             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), ai.handle,
> (size_t)-1);
38689 |                         } else {
38690 |                             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), ai.name,
> (size_t)-1);
38691 |                         }
38692 |
38693 |
38694 |                         pDeviceInfo->nativeDataFormatCount = 0;
38695 |
38696 |                         if (deviceType == ma_device_type_playback) {

```



```

38697 |             formatMask = ai.oformats;
38698 |         } else {
38699 |             formatMask = ai.iformats;
38700 |         }
38701 |
38702 |         if (((formatMask & AFMT_S16_LE) != 0 && ma_is_little_endian()) ||
> (AFMT_S16_BE && ma_is_big_endian())) {
38703 |             ma_context_add_native_data_format__oss(pContext, &ai, ma_format_s16,
> pDeviceInfo);
38704 |         }
38705 |         if (((formatMask & AFMT_S32_LE) != 0 && ma_is_little_endian()) ||
> (AFMT_S32_BE && ma_is_big_endian())) {
38706 |             ma_context_add_native_data_format__oss(pContext, &ai, ma_format_s32,
> pDeviceInfo);
38707 |         }
38708 |         if ((formatMask & AFMT_U8) != 0) {
38709 |             ma_context_add_native_data_format__oss(pContext, &ai, ma_format_u8,
> pDeviceInfo);
38710 |         }
38711 |
38712 |         foundDevice = MA_TRUE;
38713 |         break;
38714 |     }
38715 | }
38716 | }
38717 | }
38718 | } else {
38719 |     close(fdTemp);
38720 |     ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[OSS] Failed to retrieve
> system information for device enumeration.");
38721 |     return MA_NO_BACKEND;
38722 | }
38723 |
38724 |
38725 |     close(fdTemp);
38726 |
38727 |     if (!foundDevice) {
38728 |         return MA_NO_DEVICE;
38729 |     }
38730 |
38731 |     return MA_SUCCESS;
38732 | }
38733 |
38734 | static ma_result ma_device_uninit__oss(ma_device* pDevice)
38735 | {
38736 |     MA_ASSERT(pDevice != NULL);
38737 |
38738 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
38739 |         close(pDevice->oss.fdCapture);
38740 |     }
38741 |
38742 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
38743 |         close(pDevice->oss.fdPlayback);
38744 |     }
38745 |
38746 |     return MA_SUCCESS;
38747 | }
38748 |
38749 | static int ma_format_to_oss(ma_format format)
38750 | {
38751 |     int ossFormat = AFMT_U8;
38752 |     switch (format) {
38753 |         case ma_format_s16: ossFormat = (ma_is_little_endian()) ? AFMT_S16_LE : AFMT_S16_BE; break;
38754 |         case ma_format_s24: ossFormat = (ma_is_little_endian()) ? AFMT_S32_LE : AFMT_S32_BE; break;
38755 |         case ma_format_s32: ossFormat = (ma_is_little_endian()) ? AFMT_S32_LE : AFMT_S32_BE; break;
38756 |         case ma_format_f32: ossFormat = (ma_is_little_endian()) ? AFMT_S16_LE : AFMT_S16_BE; break;
38757 |         case ma_format_u8:
38758 |             default: ossFormat = AFMT_U8; break;
38759 |     }
38760 |
38761 |     return ossFormat;
38762 | }
38763 |
38764 | static ma_format ma_format_from_oss(int ossFormat)
38765 | {
38766 |     if (ossFormat == AFMT_U8) {
38767 |         return ma_format_u8;
38768 |     } else {
38769 |         if (ma_is_little_endian()) {

```

```

38770 |         switch (ossFormat) {
38771 |             case AFMT_S16_LE: return ma_format_s16;
38772 |             case AFMT_S32_LE: return ma_format_s32;
38773 |             default: return ma_format_unknown;
38774 |         }
38775 |     } else {
38776 |         switch (ossFormat) {
38777 |             case AFMT_S16_BE: return ma_format_s16;
38778 |             case AFMT_S32_BE: return ma_format_s32;
38779 |             default: return ma_format_unknown;
38780 |         }
38781 |     }
38782 | }
38783 |
38784 | return ma_format_unknown;
38785 | }
38786 |
38787 | static ma_result ma_device_init_fd_oss(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptor, ma_device_type deviceType)
38788 | {
38789 |     ma_result result;
38790 |     int ossResult;
38791 |     int fd;
38792 |     const ma_device_id* pDeviceID = NULL;
38793 |     ma_share_mode shareMode;
38794 |     int ossFormat;
38795 |     int ossChannels;
38796 |     int ossSampleRate;
38797 |     int ossFragment;
38798 |
38799 |     MA_ASSERT(pDevice != NULL);
38800 |     MA_ASSERT(pConfig != NULL);
38801 |     MA_ASSERT(deviceType != ma_device_type_duplex);
38802 |
38803 |     pDeviceID      = pDescriptor->pDeviceID;
38804 |     shareMode      = pDescriptor->shareMode;
38805 |     ossFormat      = ma_format_to_oss((pDescriptor->format != ma_format_unknown) ?
> pDescriptor->format : ma_format_s16); /* Use s16 by default because OSS doesn't like floating point.
> */
38806 |     ossChannels    = (int)(pDescriptor->channels > 0) ? pDescriptor->channels :
> MA_DEFAULT_CHANNELS;
38807 |     ossSampleRate  = (int)(pDescriptor->sampleRate > 0) ? pDescriptor->sampleRate :
> MA_DEFAULT_SAMPLE_RATE;
38808 |
38809 |     result = ma_context_open_device__oss(pDevice->pContext, deviceType, pDeviceID, shareMode, &fd);
38810 |     if (result != MA_SUCCESS) {
38811 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to open device.");
38812 |         return result;
38813 |     }
38814 |
38815 |     /*
38816 |     The OSS documentation is very clear about the order we should be initializing the device's
> properties:
38817 |     1) Format
38818 |     2) Channels
38819 |     3) Sample rate.
38820 |     */
38821 |
38822 |     /* Format. */
38823 |     ossResult = ioctl(fd, SNDCTL_DSP_SETFMT, &ossFormat);
38824 |     if (ossResult == -1) {
38825 |         close(fd);
38826 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to set format.");
38827 |         return ma_result_from_errno(errno);
38828 |     }
38829 |
38830 |     /* Channels. */
38831 |     ossResult = ioctl(fd, SNDCTL_DSP_CHANNELS, &ossChannels);
38832 |     if (ossResult == -1) {
38833 |         close(fd);
38834 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to set channel
> count.");
38835 |         return ma_result_from_errno(errno);
38836 |     }
38837 |
38838 |     /* Sample Rate. */
38839 |     ossResult = ioctl(fd, SNDCTL_DSP_SPEED, &ossSampleRate);
38840 |     if (ossResult == -1) {
38841 |         close(fd);

```

```

38842 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to set sample
> rate.");
38843 |         return ma_result_from_errno(errno);
38844 |     }
38845 |
38846 |     /*
38847 |     Buffer.
38848 |
38849 |     The documentation says that the fragment settings should be set as soon as possible, but I'm not
> sure if
38850 |     it should be done before or after format/channels/rate.
38851 |
38852 |     OSS wants the fragment size in bytes and a power of 2. When setting, we specify the power, not
> the actual
38853 |     value.
38854 |     */
38855 |     {
38856 |         ma_uint32 periodSizeInFrames;
38857 |         ma_uint32 periodSizeInBytes;
38858 |         ma_uint32 ossFragmentSizePower;
38859 |
38860 |         periodSizeInFrames = ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor,
> (ma_uint32)ossSampleRate, pConfig->performanceProfile);
38861 |
38862 |         periodSizeInBytes = ma_round_to_power_of_2(periodSizeInFrames *
> ma_get_bytes_per_frame(ma_format_from_oss(ossFormat), ossChannels));
38863 |         if (periodSizeInBytes < 16) {
38864 |             periodSizeInBytes = 16;
38865 |         }
38866 |
38867 |         ossFragmentSizePower = 4;
38868 |         periodSizeInBytes >= 4;
38869 |         while (periodSizeInBytes >= 1) {
38870 |             ossFragmentSizePower += 1;
38871 |         }
38872 |
38873 |         ossFragment = (int)((pConfig->periods < 16) | ossFragmentSizePower);
38874 |         ossResult = ioctl(fd, SNDCTL_DSP_SETFRAGMENT, &ossFragment);
38875 |         if (ossResult == -1) {
38876 |             close(fd);
38877 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to set
> fragment size and period count.");
38878 |             return ma_result_from_errno(errno);
38879 |         }
38880 |     }
38881 |
38882 |     /* Internal settings. */
38883 |     if (deviceType == ma_device_type_capture) {
38884 |         pDevice->oss.fdCapture = fd;
38885 |     } else {
38886 |         pDevice->oss.fdPlayback = fd;
38887 |     }
38888 |
38889 |     pDescriptor->format = ma_format_from_oss(ossFormat);
38890 |     pDescriptor->channels = ossChannels;
38891 |     pDescriptor->sampleRate = ossSampleRate;
38892 |     ma_channel_map_init_standard(ma_standard_channel_map_sound4, pDescriptor->channelMap,
> ma_countof(pDescriptor->channelMap), pDescriptor->channels);
38893 |     pDescriptor->periodCount = (ma_uint32)(ossFragment > 16);
38894 |     pDescriptor->periodSizeInFrames = (ma_uint32)(1 < (ossFragment & 0xFFFF)) /
> ma_get_bytes_per_frame(pDescriptor->format, pDescriptor->channels);
38895 |
38896 |     if (pDescriptor->format == ma_format_unknown) {
38897 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] The device's internal
> format is not supported by miniaudio.");
38898 |         return MA_FORMAT_NOT_SUPPORTED;
38899 |     }
38900 |
38901 |     return MA_SUCCESS;
38902 | }
38903 |
38904 | static ma_result ma_device_init_oss(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
38905 | {
38906 |     MA_ASSERT(pDevice != NULL);
38907 |     MA_ASSERT(pConfig != NULL);
38908 |
38909 |     MA_ZERO_OBJECT(&pDevice->oss);
38910 | }

```

```

38911 |     if (pConfig->deviceType == ma_device_type_loopback) {
38912 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
38913 |     }
38914 |
38915 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
38916 |         ma_result result = ma_device_init_fd_oss(pDevice, pConfig, pDescriptorCapture,
> ma_device_type_capture);
38917 |         if (result != MA_SUCCESS) {
38918 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to open
> device.");
38919 |             return result;
38920 |         }
38921 |     }
38922 |
38923 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
38924 |         ma_result result = ma_device_init_fd_oss(pDevice, pConfig, pDescriptorPlayback,
> ma_device_type_playback);
38925 |         if (result != MA_SUCCESS) {
38926 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to open
> device.");
38927 |             return result;
38928 |         }
38929 |     }
38930 |
38931 |     return MA_SUCCESS;
38932 | }
38933 |
38934 | /*
38935 | Note on Starting and Stopping
38936 | =====
38937 | In the past I was using SNDCTL_DSP_HALT to stop the device, however this results in issues when
38938 | trying to resume the device again. If we use SNDCTL_DSP_HALT, the next write() or read() will
38939 | fail. Instead what we need to do is just not write or read to and from the device when the
38940 | device is not running.
38941 |
38942 | As a result, both the start and stop functions for OSS are just empty stubs. The starting and
38943 | stopping logic is handled by ma_device_write_oss() and ma_device_read_oss(). These will check
38944 | the device state, and if the device is stopped they will simply not do any kind of processing.
38945 |
38946 | The downside to this technique is that I've noticed a fairly lengthy delay in stopping the
38947 | device, up to a second. This is on a virtual machine, and as such might just be due to the
38948 | virtual drivers, but I'm not fully sure. I am not sure how to work around this problem so for
38949 | the moment that's just how it's going to have to be.
38950 |
38951 | When starting the device, OSS will automatically start it when write() or read() is called.
38952 | */
38953 | static ma_result ma_device_start_oss(ma_device* pDevice)
38954 | {
38955 |     MA_ASSERT(pDevice != NULL);
38956 |
38957 |     /* The device is automatically started with reading and writing. */
38958 |     (void)pDevice;
38959 |
38960 |     return MA_SUCCESS;
38961 | }
38962 |
38963 | static ma_result ma_device_stop_oss(ma_device* pDevice)
38964 | {
38965 |     MA_ASSERT(pDevice != NULL);
38966 |
38967 |     /* See note above on why this is empty. */
38968 |     (void)pDevice;
38969 |
38970 |     return MA_SUCCESS;
38971 | }
38972 |
38973 | static ma_result ma_device_write_oss(ma_device* pDevice, const void* pPCMFrames, ma_uint32
> frameCount, ma_uint32* pFramesWritten)
38974 | {
38975 |     int resultOSS;
38976 |     ma_uint32 deviceState;
38977 |
38978 |     if (pFramesWritten != NULL) {
38979 |         *pFramesWritten = 0;
38980 |     }
38981 |
38982 |     /* Don't do any processing if the device is stopped. */

```

```

38983 |     deviceState = ma_device_get_state(pDevice);
38984 |     if (deviceState != ma_device_state_started && deviceState != ma_device_state_starting) {
38985 |         return MA_SUCCESS;
38986 |     }
38987 |
38988 |     resultOSS = write(pDevice->oss.fdPlayback, pPCMFrames, frameCount *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels));
38989 |     if (resultOSS < 0) {
38990 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to send data from
> the client to the device.");
38991 |         return ma_result_from_errno(errno);
38992 |     }
38993 |
38994 |     if (pFramesWritten != NULL) {
38995 |         *pFramesWritten = (ma_uint32)resultOSS /
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
38996 |     }
38997 |
38998 |     return MA_SUCCESS;
38999 | }
39000 |
39001 | static ma_result ma_device_read__oss(ma_device* pDevice, void* pPCMFrames, ma_uint32 frameCount,
> ma_uint32* pFramesRead)
39002 | {
39003 |     int resultOSS;
39004 |     ma_uint32 deviceState;
39005 |
39006 |     if (pFramesRead != NULL) {
39007 |         *pFramesRead = 0;
39008 |     }
39009 |
39010 |     /* Don't do any processing if the device is stopped. */
39011 |     deviceState = ma_device_get_state(pDevice);
39012 |     if (deviceState != ma_device_state_started && deviceState != ma_device_state_starting) {
39013 |         return MA_SUCCESS;
39014 |     }
39015 |
39016 |     resultOSS = read(pDevice->oss.fdCapture, pPCMFrames, frameCount *
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels));
39017 |     if (resultOSS < 0) {
39018 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OSS] Failed to read data from
> the device to be sent to the client.");
39019 |         return ma_result_from_errno(errno);
39020 |     }
39021 |
39022 |     if (pFramesRead != NULL) {
39023 |         *pFramesRead = (ma_uint32)resultOSS /
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels);
39024 |     }
39025 |
39026 |     return MA_SUCCESS;
39027 | }
39028 |
39029 | static ma_result ma_context_uninit__oss(ma_context* pContext)
39030 | {
39031 |     MA_ASSERT(pContext != NULL);
39032 |     MA_ASSERT(pContext->backend == ma_backend_oss);
39033 |
39034 |     (void)pContext;
39035 |     return MA_SUCCESS;
39036 | }
39037 |
39038 | static ma_result ma_context_init__oss(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
39039 | {
39040 |     int fd;
39041 |     int ossVersion;
39042 |     int result;
39043 |
39044 |     MA_ASSERT(pContext != NULL);
39045 |
39046 |     (void)pConfig;
39047 |
39048 |     /* Try opening a temporary device first so we can get version information. This is closed at the
> end. */
39049 |     fd = ma_open_temp_device__oss();
39050 |     if (fd == -1) {
39051 |         ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[OSS] Failed to open
> temporary device for retrieving system properties."); /* Looks like OSS isn't installed, or there

```

```

> are no available devices. */
39052 |         return MA_NO_BACKEND;
39053 |     }
39054 |
39055 |     /* Grab the OSS version. */
39056 |     ossVersion = 0;
39057 |     result = ioctl(fd, OSS_GETVERSION, &ossVersion);
39058 |     if (result == -1) {
39059 |         close(fd);
39060 |         ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_ERROR, "[OSS] Failed to retrieve OSS
> version.");
39061 |         return MA_NO_BACKEND;
39062 |     }
39063 |
39064 |     /* The file handle to temp device is no longer needed. Close ASAP. */
39065 |     close(fd);
39066 |
39067 |     pContext->oss.versionMajor = ((ossVersion & 0xFF0000) >> 16);
39068 |     pContext->oss.versionMinor = ((ossVersion & 0x00FF00) >> 8);
39069 |
39070 |     pCallbacks->onContextInit          = ma_context_init__oss;
39071 |     pCallbacks->onContextUninit        = ma_context_uninit__oss;
39072 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__oss;
39073 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__oss;
39074 |     pCallbacks->onDeviceInit           = ma_device_init__oss;
39075 |     pCallbacks->onDeviceUninit         = ma_device_uninit__oss;
39076 |     pCallbacks->onDeviceStart          = ma_device_start__oss;
39077 |     pCallbacks->onDeviceStop           = ma_device_stop__oss;
39078 |     pCallbacks->onDeviceRead           = ma_device_read__oss;
39079 |     pCallbacks->onDeviceWrite          = ma_device_write__oss;
39080 |     pCallbacks->onDeviceDataLoop       = NULL;
39081 |
39082 |     return MA_SUCCESS;
39083 | }
39084 | #endif /* MA_HAS_OSS */
39085 |
39086 |
39087 |
39088 |
39089 |
39090 | /*****
39091 |
39092 | AAudio Backend
39093 |
39094 | *****/
39095 | #ifndef MA_HAS_AAUDIO
39096 |
39097 | #ifndef MA_NO_RUNTIME_LINKING
39098 |     #include <AAudio/AAudio.h>
39099 | #endif
39100 |
39101 | typedef int32_t          ma_aaudio_result_t;
39102 | typedef int32_t          ma_aaudio_direction_t;
39103 | typedef int32_t          ma_aaudio_sharing_mode_t;
39104 | typedef int32_t          ma_aaudio_format_t;
39105 | typedef int32_t          ma_aaudio_stream_state_t;
39106 | typedef int32_t          ma_aaudio_performance_mode_t;
39107 | typedef int32_t          ma_aaudio_usage_t;
39108 | typedef int32_t          ma_aaudio_content_type_t;
39109 | typedef int32_t          ma_aaudio_input_preset_t;
39110 | typedef int32_t          ma_aaudio_allowed_capture_policy_t;
39111 | typedef int32_t          ma_aaudio_data_callback_result_t;
39112 | typedef struct ma_AAudioStreamBuilder_t* ma_AAudioStreamBuilder;
39113 | typedef struct ma_AAudioStream_t* ma_AAudioStream;
39114 |
39115 | #define MA_AAUDIO_UNSPECIFIED          0
39116 |
39117 | /* Result codes. miniaudio only cares about the success code. */
39118 | #define MA_AAUDIO_OK                  0
39119 |
39120 | /* Directions. */
39121 | #define MA_AAUDIO_DIRECTION_OUTPUT    0
39122 | #define MA_AAUDIO_DIRECTION_INPUT    1
39123 |
39124 | /* Sharing modes. */
39125 | #define MA_AAUDIO_SHARING_MODE_EXCLUSIVE 0
39126 | #define MA_AAUDIO_SHARING_MODE_SHARED   1
39127 |
39128 | /* Formats. */

```

```

39129 | #define MA_AUDIO_FORMAT_PCM_I16 1
39130 | #define MA_AUDIO_FORMAT_PCM_FLOAT 2
39131 |
39132 | /* Stream states. */
39133 | #define MA_AUDIO_STREAM_STATE_UNINITIALIZED 0
39134 | #define MA_AUDIO_STREAM_STATE_UNKNOWN 1
39135 | #define MA_AUDIO_STREAM_STATE_OPEN 2
39136 | #define MA_AUDIO_STREAM_STATE_STARTING 3
39137 | #define MA_AUDIO_STREAM_STATE_STARTED 4
39138 | #define MA_AUDIO_STREAM_STATE_PAUSING 5
39139 | #define MA_AUDIO_STREAM_STATE_PAUSED 6
39140 | #define MA_AUDIO_STREAM_STATE_FLUSHING 7
39141 | #define MA_AUDIO_STREAM_STATE_FLUSHED 8
39142 | #define MA_AUDIO_STREAM_STATE_STOPPING 9
39143 | #define MA_AUDIO_STREAM_STATE_STOPPED 10
39144 | #define MA_AUDIO_STREAM_STATE_CLOSING 11
39145 | #define MA_AUDIO_STREAM_STATE_CLOSED 12
39146 | #define MA_AUDIO_STREAM_STATE_DISCONNECTED 13
39147 |
39148 | /* Performance modes. */
39149 | #define MA_AUDIO_PERFORMANCE_MODE_NONE 10
39150 | #define MA_AUDIO_PERFORMANCE_MODE_POWER_SAVING 11
39151 | #define MA_AUDIO_PERFORMANCE_MODE_LOW_LATENCY 12
39152 |
39153 | /* Usage types. */
39154 | #define MA_AUDIO_USAGE_MEDIA 1
39155 | #define MA_AUDIO_USAGE_VOICE_COMMUNICATION 2
39156 | #define MA_AUDIO_USAGE_VOICE_COMMUNICATION_SIGNALLING 3
39157 | #define MA_AUDIO_USAGE_ALARM 4
39158 | #define MA_AUDIO_USAGE_NOTIFICATION 5
39159 | #define MA_AUDIO_USAGE_NOTIFICATION_RINGTONE 6
39160 | #define MA_AUDIO_USAGE_NOTIFICATION_EVENT 10
39161 | #define MA_AUDIO_USAGE_ASSISTANCE_ACCESSIBILITY 11
39162 | #define MA_AUDIO_USAGE_ASSISTANCE_NAVIGATION_GUIDANCE 12
39163 | #define MA_AUDIO_USAGE_ASSISTANCE_SONIFICATION 13
39164 | #define MA_AUDIO_USAGE_GAME 14
39165 | #define MA_AUDIO_USAGE_ASSISTANT 16
39166 | #define MA_AUDIO_SYSTEM_USAGE_EMERGENCY 1000
39167 | #define MA_AUDIO_SYSTEM_USAGE_SAFETY 1001
39168 | #define MA_AUDIO_SYSTEM_USAGE_VEHICLE_STATUS 1002
39169 | #define MA_AUDIO_SYSTEM_USAGE_ANNOUNCEMENT 1003
39170 |
39171 | /* Content types. */
39172 | #define MA_AUDIO_CONTENT_TYPE_SPEECH 1
39173 | #define MA_AUDIO_CONTENT_TYPE_MUSIC 2
39174 | #define MA_AUDIO_CONTENT_TYPE_MOVIE 3
39175 | #define MA_AUDIO_CONTENT_TYPE_SONIFICATION 4
39176 |
39177 | /* Input presets. */
39178 | #define MA_AUDIO_INPUT_PRESET_GENERIC 1
39179 | #define MA_AUDIO_INPUT_PRESET_CAMCORDER 5
39180 | #define MA_AUDIO_INPUT_PRESET_VOICE_RECOGNITION 6
39181 | #define MA_AUDIO_INPUT_PRESET_VOICE_COMMUNICATION 7
39182 | #define MA_AUDIO_INPUT_PRESET_UNPROCESSED 9
39183 | #define MA_AUDIO_INPUT_PRESET_VOICE_PERFORMANCE 10
39184 |
39185 | /* Allowed Capture Policies */
39186 | #define MA_AUDIO_ALLOW_CAPTURE_BY_ALL 1
39187 | #define MA_AUDIO_ALLOW_CAPTURE_BY_SYSTEM 2
39188 | #define MA_AUDIO_ALLOW_CAPTURE_BY_NONE 3
39189 |
39190 | /* Callback results. */
39191 | #define MA_AUDIO_CALLBACK_RESULT_CONTINUE 0
39192 | #define MA_AUDIO_CALLBACK_RESULT_STOP 1
39193 |
39194 |
39195 | typedef ma_audio_data_callback_result_t (* ma_AudioStream_dataCallback) (ma_AudioStream* pStream,
> void* pUserData, void* pAudioData, int32_t numFrames);
39196 | typedef void (* ma_AudioStream_errorCallback)(ma_AudioStream *pStream,
> void *pUserData, ma_audio_result_t error);
39197 |
39198 | typedef ma_audio_result_t (* MA_PFN_Audio_createStreamBuilder)
> (ma_AudioStreamBuilder** ppBuilder);
39199 | typedef ma_audio_result_t (* MA_PFN_AudioStreamBuilder_delete)
> (ma_AudioStreamBuilder* pBuilder);
39200 | typedef void (* MA_PFN_AudioStreamBuilder_setDeviceId)
> (ma_AudioStreamBuilder* pBuilder, int32_t deviceId);
39201 | typedef void (* MA_PFN_AudioStreamBuilder_setDirection)
> (ma_AudioStreamBuilder* pBuilder, ma_audio_direction_t direction);

```



```

39202 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setSharingMode)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_sharing_mode_t sharingMode);
39203 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setFormat)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_format_t format);
39204 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setChannelCount)
> (ma_AAUDIOStreamBuilder* pBuilder, int32_t channelCount);
39205 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setSampleRate)
> (ma_AAUDIOStreamBuilder* pBuilder, int32_t sampleRate);
39206 | typedef void                                (*
> MA_PFN_AAUDIOStreamBuilder_setBufferCapacityInFrames)(ma_AAUDIOStreamBuilder* pBuilder, int32_t
> numFrames);
39207 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setFramesPerDataCallback)
> (ma_AAUDIOStreamBuilder* pBuilder, int32_t numFrames);
39208 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setDataCallback)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_AAUDIOStream_dataCallback callback, void* pUserData);
39209 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setErrorCallback)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_AAUDIOStream_errorCallback callback, void* pUserData);
39210 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setPerformanceMode)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_performance_mode_t mode);
39211 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setUsage)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_usage_t contentType);
39212 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setContentType)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_content_type_t contentType);
39213 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setInputPreset)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_input_preset_t inputPreset);
39214 | typedef void                                (* MA_PFN_AAUDIOStreamBuilder_setAllowedCapturePolicy)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_aaudio_allowed_capture_policy_t policy);
39215 | typedef ma_aaudio_result_t                  (* MA_PFN_AAUDIOStreamBuilder_openStream)
> (ma_AAUDIOStreamBuilder* pBuilder, ma_AAUDIOStream** ppStream);
39216 | typedef ma_aaudio_result_t                  (* MA_PFN_AAUDIOStream_close)
> (ma_AAUDIOStream* pStream);
39217 | typedef ma_aaudio_stream_state_t            (* MA_PFN_AAUDIOStream_getState)
> (ma_AAUDIOStream* pStream);
39218 | typedef ma_aaudio_result_t                  (* MA_PFN_AAUDIOStream_waitForStateChange)
> (ma_AAUDIOStream* pStream, ma_aaudio_stream_state_t inputState, ma_aaudio_stream_state_t*
> pNextState, int64_t timeoutInNanoseconds);
39219 | typedef ma_aaudio_format_t                  (* MA_PFN_AAUDIOStream_getFormat)
> (ma_AAUDIOStream* pStream);
39220 | typedef int32_t                             (* MA_PFN_AAUDIOStream_getChannelCount)
> (ma_AAUDIOStream* pStream);
39221 | typedef int32_t                             (* MA_PFN_AAUDIOStream_getSampleRate)
> (ma_AAUDIOStream* pStream);
39222 | typedef int32_t                             (* MA_PFN_AAUDIOStream_getBufferCapacityInFrames)
> (ma_AAUDIOStream* pStream);
39223 | typedef int32_t                             (* MA_PFN_AAUDIOStream_getFramesPerDataCallback)
> (ma_AAUDIOStream* pStream);
39224 | typedef int32_t                             (* MA_PFN_AAUDIOStream_getFramesPerBurst)
> (ma_AAUDIOStream* pStream);
39225 | typedef ma_aaudio_result_t                  (* MA_PFN_AAUDIOStream_requestStart)
> (ma_AAUDIOStream* pStream);
39226 | typedef ma_aaudio_result_t                  (* MA_PFN_AAUDIOStream_requestStop)
> (ma_AAUDIOStream* pStream);
39227 |
39228 | static ma_result ma_result_from_aaudio(ma_aaudio_result_t resultAA)
39229 | {
39230 |     switch (resultAA)
39231 |     {
39232 |         case MA_AAUDIO_OK: return MA_SUCCESS;
39233 |         default: break;
39234 |     }
39235 |
39236 |     return MA_ERROR;
39237 | }
39238 |
39239 | static ma_aaudio_usage_t ma_to_usage__aaudio(ma_aaudio_usage usage)
39240 | {
39241 |     switch (usage) {
39242 |         case ma_aaudio_usage_media:                return MA_AAUDIO_USAGE_MEDIA;
39243 |         case ma_aaudio_usage_voice_communication:  return
> MA_AAUDIO_USAGE_VOICE_COMMUNICATION;
39244 |         case ma_aaudio_usage_voice_communication_signalling: return
> MA_AAUDIO_USAGE_VOICE_COMMUNICATION_SIGNALLING;
39245 |         case ma_aaudio_usage_alarm:                return MA_AAUDIO_USAGE_ALARM;
39246 |         case ma_aaudio_usage_notification:          return MA_AAUDIO_USAGE_NOTIFICATION;
39247 |         case ma_aaudio_usage_notification_ringtone: return
> MA_AAUDIO_USAGE_NOTIFICATION_RINGTONE;
39248 |         case ma_aaudio_usage_notification_event:    return
> MA_AAUDIO_USAGE_NOTIFICATION_EVENT;
39249 |         case ma_aaudio_usage_assistance_accessibility: return

```

```

> MA_AUDIO_USAGE_ASSISTANCE_ACCESSIBILITY;
39250 |     case ma_audio_usage_assistance_navigation_guidance: return
> MA_AUDIO_USAGE_ASSISTANCE_NAVIGATION_GUIDANCE;
39251 |     case ma_audio_usage_assistance_sonification:         return
> MA_AUDIO_USAGE_ASSISTANCE_SONIFICATION;
39252 |     case ma_audio_usage_game:                             return MA_AUDIO_USAGE_GAME;
39253 |     case ma_audio_usage_assistant:                       return MA_AUDIO_USAGE_ASSISTANT;
39254 |     case ma_audio_usage_emergency:                       return
> MA_AUDIO_SYSTEM_USAGE_EMERGENCY;
39255 |     case ma_audio_usage_safety:                           return MA_AUDIO_SYSTEM_USAGE_SAFETY;
39256 |     case ma_audio_usage_vehicle_status:                   return
> MA_AUDIO_SYSTEM_USAGE_VEHICLE_STATUS;
39257 |     case ma_audio_usage_announcement:                     return
> MA_AUDIO_SYSTEM_USAGE_ANNOUNCEMENT;
39258 |     default: break;
39259 | }
39260 |
39261 | return MA_AUDIO_USAGE_MEDIA;
39262 | }
39263 |
39264 | static ma_audio_content_type_t ma_to_content_type__audio(ma_audio_content_type contentType)
39265 | {
39266 |     switch (contentType) {
39267 |         case ma_audio_content_type_speech:               return MA_AUDIO_CONTENT_TYPE_SPEECH;
39268 |         case ma_audio_content_type_music:                return MA_AUDIO_CONTENT_TYPE_MUSIC;
39269 |         case ma_audio_content_type_movie:                return MA_AUDIO_CONTENT_TYPE_MOVIE;
39270 |         case ma_audio_content_type_sonification:         return MA_AUDIO_CONTENT_TYPE_SONIFICATION;
39271 |         default: break;
39272 |     }
39273 |
39274 |     return MA_AUDIO_CONTENT_TYPE_SPEECH;
39275 | }
39276 |
39277 | static ma_audio_input_preset_t ma_to_input_preset__audio(ma_audio_input_preset inputPreset)
39278 | {
39279 |     switch (inputPreset) {
39280 |         case ma_audio_input_preset_generic:              return MA_AUDIO_INPUT_PRESET_GENERIC;
39281 |         case ma_audio_input_preset_camcorder:           return MA_AUDIO_INPUT_PRESET_CAMCORDER;
39282 |         case ma_audio_input_preset_voice_recognition:    return
> MA_AUDIO_INPUT_PRESET_VOICE_RECOGNITION;
39283 |         case ma_audio_input_preset_voice_communication: return
> MA_AUDIO_INPUT_PRESET_VOICE_COMMUNICATION;
39284 |         case ma_audio_input_preset_unprocessed:         return MA_AUDIO_INPUT_PRESET_UNPROCESSED;
39285 |         case ma_audio_input_preset_voice_performance:   return
> MA_AUDIO_INPUT_PRESET_VOICE_PERFORMANCE;
39286 |         default: break;
39287 |     }
39288 |
39289 |     return MA_AUDIO_INPUT_PRESET_GENERIC;
39290 | }
39291 |
39292 | static ma_audio_allowed_capture_policy_t
> ma_to_allowed_capture_policy__audio(ma_audio_allowed_capture_policy allowedCapturePolicy)
39293 | {
39294 |     switch (allowedCapturePolicy) {
39295 |         case ma_audio_allow_capture_by_all:              return MA_AUDIO_ALLOW_CAPTURE_BY_ALL;
39296 |         case ma_audio_allow_capture_by_system:           return MA_AUDIO_ALLOW_CAPTURE_BY_SYSTEM;
39297 |         case ma_audio_allow_capture_by_none:             return MA_AUDIO_ALLOW_CAPTURE_BY_NONE;
39298 |         default: break;
39299 |     }
39300 |
39301 |     return MA_AUDIO_ALLOW_CAPTURE_BY_ALL;
39302 | }
39303 |
39304 | static void ma_stream_error_callback__audio(ma_AudioStream* pStream, void* pUserData,
> ma_audio_result_t error)
39305 | {
39306 |     ma_result result;
39307 |     ma_job job;
39308 |     ma_device* pDevice = (ma_device*)pUserData;
39309 |     MA_ASSERT(pDevice != NULL);
39310 |
39311 |     (void)error;
39312 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[AAudio] ERROR CALLBACK: error=%d",
> AAStream_getState()=%d\n", error,
> ((MA_PFN_AAStream_getState)pDevice->pContext->audio.AAStream_getState)(pStream));
39313 |
39314 |     /*
39315 |     When we get an error, we'll assume that the stream is in an erroneous state and needs to be

```

```

> restarted. From the documentation,
39316 |     we cannot do this from the error callback. Therefore we are going to use an event thread for the
> AAudio backend to do this
39317 |     cleanly and safely.
39318 |     */
39319 |     if (ma_atomic_bool32_get(&pDevice->aaudio.isTearingDown)) {
39320 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[AAudio] Device Disconnected.
> Tearing down device.\n");
39321 |     }
39322 |     else {
39323 |         job = ma_job_init(MA_JOB_TYPE_DEVICE_AAUDIO_REROUTE);
39324 |         job.data.device.aaudio.reroute.pDevice = pDevice;
39325 |
39326 |         if (pStream == pDevice->aaudio.pStreamCapture) {
39327 |             job.data.device.aaudio.reroute.deviceType = ma_device_type_capture;
39328 |         } else {
39329 |             job.data.device.aaudio.reroute.deviceType = ma_device_type_playback;
39330 |         }
39331 |
39332 |         result = ma_device_job_thread_post(&pDevice->pContext->aaudio.jobThread, &job);
39333 |         if (result != MA_SUCCESS) {
39334 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[AAudio] Device
> Disconnected. Failed to post job for rerouting.\n");
39335 |             return;
39336 |         }
39337 |     }
39338 | }
39339 |
39340 | static ma_aaudio_data_callback_result_t ma_stream_data_callback_capture__audio(ma_AAudioStream*
> pStream, void* pUserData, void* pAudioData, int32_t frameCount)
39341 | {
39342 |     ma_device* pDevice = (ma_device*)pUserData;
39343 |     MA_ASSERT(pDevice != NULL);
39344 |
39345 |     if (frameCount > 0) {
39346 |         ma_device_handle_backend_data_callback(pDevice, NULL, pAudioData, (ma_uint32)frameCount);
39347 |     }
39348 |
39349 |     (void)pStream;
39350 |     return MA_AAUDIO_CALLBACK_RESULT_CONTINUE;
39351 | }
39352 |
39353 | static ma_aaudio_data_callback_result_t ma_stream_data_callback_playback__audio(ma_AAudioStream*
> pStream, void* pUserData, void* pAudioData, int32_t frameCount)
39354 | {
39355 |     ma_device* pDevice = (ma_device*)pUserData;
39356 |     MA_ASSERT(pDevice != NULL);
39357 |
39358 |     /*
39359 |     I've had a report that AAudio can sometimes post a frame count of 0. We need to check for that
> here
39360 |     so we don't get any errors at a deeper level. I'm doing the same with the capture side for
> safety,
39361 |     though I've not yet had any reports about that one.
39362 |     */
39363 |     if (frameCount > 0) {
39364 |         ma_device_handle_backend_data_callback(pDevice, pAudioData, NULL, (ma_uint32)frameCount);
39365 |     }
39366 |
39367 |     (void)pStream;
39368 |     return MA_AAUDIO_CALLBACK_RESULT_CONTINUE;
39369 | }
39370 |
39371 | static ma_result ma_create_and_configure_AAudioStreamBuilder__audio(ma_context* pContext, const
> ma_device_id* pDeviceID, ma_device_type deviceType, ma_share_mode shareMode, const
> ma_device_descriptor* pDescriptor, const ma_device_config* pConfig, ma_device* pDevice,
> ma_AAudioStreamBuilder** ppBuilder)
39372 | {
39373 |     ma_AAudioStreamBuilder* pBuilder;
39374 |     ma_aaudio_result_t resultAA;
39375 |
39376 |     /* Safety. */
39377 |     *ppBuilder = NULL;
39378 |
39379 |     resultAA =
> ((MA_PFN_AAudio_createStreamBuilder)pContext->aaudio.AAudio_createStreamBuilder>(&pBuilder);
39380 |     if (resultAA != MA_AAUDIO_OK) {
39381 |         return ma_result_from_aaudio(resultAA);
39382 |     }

```

```

39383 |
39384 |     if (pDeviceID != NULL) {
39385 |
> ((MA_PFN_AAUDIOStreamBuilder_setDeviceId)pContext->aaudio.AAudioStreamBuilder_setDeviceId)(pBuilder,
> pDeviceID->aaudio);
39386 |     }
39387 |
39388 |     ((MA_PFN_AAUDIOStreamBuilder_setDirection)pContext->aaudio.AAudioStreamBuilder_setDirection)(pBu
> ilder, (deviceType == ma_device_type_playback) ? MA_AAUDIO_DIRECTION_OUTPUT :
> MA_AAUDIO_DIRECTION_INPUT);
39389 |     ((MA_PFN_AAUDIOStreamBuilder_setSharingMode)pContext->aaudio.AAudioStreamBuilder_setSharingMode)
> (pBuilder, (shareMode == ma_share_mode_shared) ? MA_AAUDIO_SHARING_MODE_SHARED :
> MA_AAUDIO_SHARING_MODE_EXCLUSIVE);
39390 |
39391 |
39392 |     /* If we have a device descriptor make sure we configure the stream builder to take our
> requested parameters. */
39393 |     if (pDescriptor != NULL) {
39394 |         MA_ASSERT(pConfig != NULL); /* We must have a device config if we also have a descriptor.
> The config is required for AAudio specific configuration options. */
39395 |
39396 |         if (pDescriptor->sampleRate != 0) {
39397 |             ((MA_PFN_AAUDIOStreamBuilder_setSampleRate)pContext->aaudio.AAudioStreamBuilder_setSampl
> eRate)(pBuilder, pDescriptor->sampleRate);
39398 |         }
39399 |
39400 |         if (pDescriptor->channels != 0) {
39401 |             ((MA_PFN_AAUDIOStreamBuilder_setChannelCount)pContext->aaudio.AAudioStreamBuilder_setCha
> nnelCount)(pBuilder, pDescriptor->channels);
39402 |         }
39403 |
39404 |         if (pDescriptor->format != ma_format_unknown) {
39405 |
> ((MA_PFN_AAUDIOStreamBuilder_setFormat)pContext->aaudio.AAudioStreamBuilder_setFormat)(pBuilder,
> (pDescriptor->format == ma_format_s16) ? MA_AAUDIO_FORMAT_PCM_I16 : MA_AAUDIO_FORMAT_PCM_FLOAT);
39406 |         }
39407 |
39408 |
39409 |         /*
39410 |         There have been reports where setting the frames per data callback results in an error.
39411 |         In particular, re-routing may inadvertently switch from low-latency mode, resulting in a
> less stable
39412 |         stream from the legacy path (AudioStreamLegacy). To address this, we simply don't set the
> value. It
39413 |         can still be set if it's explicitly requested via the aaudio.allowSetBufferCapacity variable
> in the
39414 |         device config.
39415 |         */
39416 |         if ((!pConfig->aaudio.enableCompatibilityWorkarounds || ma_android_sdk_version() > 30) &&
> pConfig->aaudio.allowSetBufferCapacity) {
39417 |             /*
39418 |             AAudio is annoying when it comes to its buffer calculation stuff because it doesn't let
> you
39419 |             retrieve the actual sample rate until after you've opened the stream. But you need to
> configure
39420 |             the buffer capacity before you open the stream... :/
39421 |
39422 |             To solve, we're just going to assume MA_DEFAULT_SAMPLE_RATE (48000) and move on.
39423 |             */
39424 |             ma_uint32 bufferCapacityInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptor, pDescriptor->sampleRate,
> pConfig->performanceProfile) * pDescriptor->periodCount;
39425 |
39426 |             ((MA_PFN_AAUDIOStreamBuilder_setBufferCapacityInFrames)pContext->aaudio.AAudioStreamBuil
> der_setBufferCapacityInFrames)(pBuilder, bufferCapacityInFrames);
39427 |             ((MA_PFN_AAUDIOStreamBuilder_setFramesPerDataCallback)pContext->aaudio.AAudioStreamBuil
> der_setFramesPerDataCallback)(pBuilder, bufferCapacityInFrames / pDescriptor->periodCount);
39428 |         }
39429 |
39430 |         if (deviceType == ma_device_type_capture) {
39431 |             if (pConfig->aaudio.inputPreset != ma_aaudio_input_preset_default &&
> pContext->aaudio.AAudioStreamBuilder_setInputPreset != NULL) {
39432 |                 ((MA_PFN_AAUDIOStreamBuilder_setInputPreset)pContext->aaudio.AAudioStreamBuilder_set
> InputPreset)(pBuilder, ma_to_input_preset__aaudio(pConfig->aaudio.inputPreset));
39433 |             }
39434 |
39435 |             ((MA_PFN_AAUDIOStreamBuilder_setDataCallback)pContext->aaudio.AAudioStreamBuilder_setDat
> aCallback)(pBuilder, ma_stream_data_callback_capture__aaudio, (void*)pDevice);
39436 |         } else {

```

```

39437 |         if (pConfig->aaudio.usage != ma_aaudio_usage_default &&
> pContext->aaudio.AAudioStreamBuilder_setUsage != NULL) {
39438 |
39439 |         }
39440 |
39441 |         if (pConfig->aaudio.contentType != ma_aaudio_content_type_default &&
> pContext->aaudio.AAudioStreamBuilder_setContentType != NULL) {
39442 |             ((MA_PFN_AAudioStreamBuilder_setContentType)pContext->aaudio.AAudioStreamBuilder_set
> ContentType)(pBuilder, ma_to_content_type__audio(pConfig->aaudio.contentType));
39443 |         }
39444 |
39445 |         if (pConfig->aaudio.allowedCapturePolicy != ma_aaudio_allow_capture_default &&
> pContext->aaudio.AAudioStreamBuilder_setAllowedCapturePolicy != NULL) {
39446 |             ((MA_PFN_AAudioStreamBuilder_setAllowedCapturePolicy)pContext->aaudio.AAudioStreamBu
> ilder_setAllowedCapturePolicy)(pBuilder,
> ma_to_allowed_capture_policy__audio(pConfig->aaudio.allowedCapturePolicy));
39447 |         }
39448 |
39449 |         ((MA_PFN_AAudioStreamBuilder_setDataCallback)pContext->aaudio.AAudioStreamBuilder_setDat
> aCallback)(pBuilder, ma_stream_data_callback_playback__audio, (void*)pDevice);
39450 |     }
39451 |
39452 |     /*
39453 |     If we set AAUDIO_PERFORMANCE_MODE_LOW_LATENCY, we allow for MMAP (non-legacy path).
39454 |     Since there's a mapping between miniaudio's performance profiles and AAudio's performance
> modes, let's use it.
39455 |     Beware though, with a conservative performance profile, AAudio will indeed take the legacy
> path.
39456 |     */
39457 |     ((MA_PFN_AAudioStreamBuilder_setPerformanceMode)pContext->aaudio.AAudioStreamBuilder_setPerf
> ormanceMode)(pBuilder, (pConfig->performanceProfile == ma_performance_profile_low_latency) ?
> MA_AAUDIO_PERFORMANCE_MODE_LOW_LATENCY : MA_AAUDIO_PERFORMANCE_MODE_NONE);
39458 |
39459 |     /* We need to set an error callback to detect device changes. */
39460 |     if (pDevice != NULL) { /* <- pDevice should never be null if pDescriptor is not null,
> which is always the case if we hit this branch. Check anyway for safety. */
39461 |         ((MA_PFN_AAudioStreamBuilder_setErrorCallback)pContext->aaudio.AAudioStreamBuilder_setEr
> rorCallback)(pBuilder, ma_stream_error_callback__audio, (void*)pDevice);
39462 |     }
39463 | }
39464 |
39465 | *ppBuilder = pBuilder;
39466 |
39467 | return MA_SUCCESS;
39468 | }
39469 |
39470 | static ma_result ma_open_stream_and_close_builder__audio(ma_context* pContext,
> ma_AAudioStreamBuilder* pBuilder, ma_AAudioStream** ppStream)
39471 | {
39472 |     ma_result result;
39473 |
39474 |     result = ma_result_from_aaudio(((MA_PFN_AAudioStreamBuilder_openStream)pContext->aaudio.AAudioSt
> reamBuilder_openStream)(pBuilder, ppStream));
39475 |     ((MA_PFN_AAudioStreamBuilder_delete)pContext->aaudio.AAudioStreamBuilder_delete)(pBuilder);
39476 |
39477 |     return result;
39478 | }
39479 |
39480 | static ma_result ma_open_stream_basic__audio(ma_context* pContext, const ma_device_id* pDeviceID,
> ma_device_type deviceType, ma_share_mode shareMode, ma_AAudioStream** ppStream)
39481 | {
39482 |     ma_result result;
39483 |     ma_AAudioStreamBuilder* pBuilder;
39484 |
39485 |     *ppStream = NULL;
39486 |
39487 |     result = ma_create_and_configure_AAudioStreamBuilder__audio(pContext, pDeviceID, deviceType,
> shareMode, NULL, NULL, NULL, &pBuilder);
39488 |     if (result != MA_SUCCESS) {
39489 |         return result;
39490 |     }
39491 |
39492 |     /* Let's give AAudio a hint to avoid the legacy path (AudioStreamLegacy). */
39493 |     ((MA_PFN_AAudioStreamBuilder_setPerformanceMode)pContext->aaudio.AAudioStreamBuilder_setPerforma
> nceMode)(pBuilder, MA_AAUDIO_PERFORMANCE_MODE_LOW_LATENCY);
39494 |
39495 |     return ma_open_stream_and_close_builder__audio(pContext, pBuilder, ppStream);

```

```

39496 | }
39497 |
39498 | static ma_result ma_open_stream__audio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_type deviceType, const ma_device_descriptor* pDescriptor, ma_AudioStream** ppStream)
39499 | {
39500 |     ma_result result;
39501 |     ma_AudioStreamBuilder* pBuilder;
39502 |
39503 |     MA_ASSERT(pDevice != NULL);
39504 |     MA_ASSERT(pDescriptor != NULL);
39505 |     MA_ASSERT(deviceType != ma_device_type_duplex); /* This function should not be called for a
> full-duplex device type. */
39506 |
39507 |     *ppStream = NULL;
39508 |
39509 |     result = ma_create_and_configure_AudioStreamBuilder__audio(pDevice->pContext,
> pDescriptor->pDeviceID, deviceType, pDescriptor->shareMode, pDescriptor, pConfig, pDevice,
> &pBuilder);
39510 |     if (result != MA_SUCCESS) {
39511 |         return result;
39512 |     }
39513 |
39514 |     return ma_open_stream_and_close_builder__audio(pDevice->pContext, pBuilder, ppStream);
39515 | }
39516 |
39517 | static ma_result ma_close_stream__audio(ma_context* pContext, ma_AudioStream* pStream)
39518 | {
39519 |     if (pStream == NULL) {
39520 |         return MA_INVALID_ARGS;
39521 |     }
39522 |
39523 |     return
> ma_result_from_audio(((MA_PFN_AudioStream_close)pContext->audio.AudioStream_close)(pStream));
39524 | }
39525 |
39526 | static ma_bool32 ma_has_default_device__audio(ma_context* pContext, ma_device_type deviceType)
39527 | {
39528 |     /* The only way to know this is to try creating a stream. */
39529 |     ma_AudioStream* pStream;
39530 |     ma_result result = ma_open_stream_basic__audio(pContext, NULL, deviceType,
> ma_share_mode_shared, &pStream);
39531 |     if (result != MA_SUCCESS) {
39532 |         return MA_FALSE;
39533 |     }
39534 |
39535 |     ma_close_stream__audio(pContext, pStream);
39536 |     return MA_TRUE;
39537 | }
39538 |
39539 | static ma_result ma_wait_for_simple_state_transition__audio(ma_context* pContext, ma_AudioStream*
> pStream, ma_audio_stream_state_t oldState, ma_audio_stream_state_t newState)
39540 | {
39541 |     ma_audio_stream_state_t actualNewState;
39542 |     ma_audio_result_t resultAA =
> ((MA_PFN_AudioStream_waitForStateChange)pContext->audio.AudioStream_waitForStateChange)(pStream,
> oldState, &actualNewState, 5000000000); /* 5 second timeout. */
39543 |     if (resultAA != MA_AUDIO_OK) {
39544 |         return ma_result_from_audio(resultAA);
39545 |     }
39546 |
39547 |     if (newState != actualNewState) {
39548 |         return MA_ERROR; /* Failed to transition into the expected state. */
39549 |     }
39550 |
39551 |     return MA_SUCCESS;
39552 | }
39553 |
39554 |
39555 | static ma_result ma_context_enumerate_devices__audio(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
39556 | {
39557 |     ma_bool32 cbResult = MA_TRUE;
39558 |
39559 |     MA_ASSERT(pContext != NULL);
39560 |     MA_ASSERT(callback != NULL);
39561 |
39562 |     /* Unfortunately Audio does not have an enumeration API. Therefore I'm only going to report
> default devices, but only if it can instantiate a stream. */
39563 |

```



```

39564 | /* Playback. */
39565 | if (cbResult) {
39566 |     ma_device_info deviceInfo;
39567 |     MA_ZERO_OBJECT(&deviceInfo);
39568 |     deviceInfo.id.audio = MA_AUDIO_UNSPECIFIED;
39569 |     ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
39570 |
39571 |     if (ma_has_default_device__audio(pContext, ma_device_type_playback)) {
39572 |         cbResult = callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
39573 |     }
39574 | }
39575 |
39576 | /* Capture. */
39577 | if (cbResult) {
39578 |     ma_device_info deviceInfo;
39579 |     MA_ZERO_OBJECT(&deviceInfo);
39580 |     deviceInfo.id.audio = MA_AUDIO_UNSPECIFIED;
39581 |     ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
39582 |
39583 |     if (ma_has_default_device__audio(pContext, ma_device_type_capture)) {
39584 |         cbResult = callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
39585 |     }
39586 | }
39587 |
39588 | return MA_SUCCESS;
39589 | }
39590 |
39591 | static void ma_context_add_native_data_format_from_AAUDIOStream_ex__audio(ma_context* pContext,
> ma_AAUDIOStream* pStream, ma_format format, ma_uint32 flags, ma_device_info* pDeviceInfo)
39592 | {
39593 |     MA_ASSERT(pContext != NULL);
39594 |     MA_ASSERT(pStream != NULL);
39595 |     MA_ASSERT(pDeviceInfo != NULL);
39596 |
39597 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format = format;
39598 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels =
> ((MA_PFN_AAUDIOStream_getChannelCount)pContext->audio.AAUDIOStream_getChannelCount)(pStream);
39599 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate =
> ((MA_PFN_AAUDIOStream_getSampleRate)pContext->audio.AAUDIOStream_getSampleRate)(pStream);
39600 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags = flags;
39601 |     pDeviceInfo->nativeDataFormatCount += 1;
39602 | }
39603 |
39604 | static void ma_context_add_native_data_format_from_AAUDIOStream__audio(ma_context* pContext,
> ma_AAUDIOStream* pStream, ma_uint32 flags, ma_device_info* pDeviceInfo)
39605 | {
39606 |     /* AAUDIO supports s16 and f32. */
39607 |     ma_context_add_native_data_format_from_AAUDIOStream_ex__audio(pContext, pStream, ma_format_f32,
> flags, pDeviceInfo);
39608 |     ma_context_add_native_data_format_from_AAUDIOStream_ex__audio(pContext, pStream, ma_format_s16,
> flags, pDeviceInfo);
39609 | }
39610 |
39611 | static ma_result ma_context_get_device_info__audio(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
39612 | {
39613 |     ma_AAUDIOStream* pStream;
39614 |     ma_result result;
39615 |
39616 |     MA_ASSERT(pContext != NULL);
39617 |
39618 |     /* ID */
39619 |     if (pDeviceID != NULL) {
39620 |         pDeviceInfo->id.audio = pDeviceID->audio;
39621 |     } else {
39622 |         pDeviceInfo->id.audio = MA_AUDIO_UNSPECIFIED;
39623 |     }
39624 |
39625 |     /* Name */
39626 |     if (deviceType == ma_device_type_playback) {
39627 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
39628 |     } else {
39629 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
39630 |     }
39631 | }

```



```

39632 |
39633 |     pDeviceInfo->nativeDataFormatCount = 0;
39634 |
39635 |     /* We'll need to open the device to get accurate sample rate and channel count information. */
39636 |     result = ma_open_stream_basic__audio(pContext, pDeviceID, deviceType, ma_share_mode_shared,
> &pStream);
39637 |     if (result != MA_SUCCESS) {
39638 |         return result;
39639 |     }
39640 |
39641 |     ma_context_add_native_data_format_from_AAUDIOStream__audio(pContext, pStream, 0, pDeviceInfo);
39642 |
39643 |     ma_close_stream__audio(pContext, pStream);
39644 |     pStream = NULL;
39645 |
39646 |     return MA_SUCCESS;
39647 | }
39648 |
39649 | static ma_result ma_close_streams__audio(ma_device* pDevice)
39650 | {
39651 |     MA_ASSERT(pDevice != NULL);
39652 |
39653 |     /* When rerouting, streams may have been closed and never re-opened. Hence the extra checks
> below. */
39654 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
39655 |         ma_close_stream__audio(pDevice->pContext,
> (ma_AAUDIOStream*)pDevice->audio.pStreamCapture);
39656 |         pDevice->audio.pStreamCapture = NULL;
39657 |     }
39658 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
39659 |         ma_close_stream__audio(pDevice->pContext,
> (ma_AAUDIOStream*)pDevice->audio.pStreamPlayback);
39660 |         pDevice->audio.pStreamPlayback = NULL;
39661 |     }
39662 |
39663 |     return MA_SUCCESS;
39664 | }
39665 |
39666 | static ma_result ma_device_uninit__audio(ma_device* pDevice)
39667 | {
39668 |     MA_ASSERT(pDevice != NULL);
39669 |
39670 |     /*
39671 |     Note: Closing the streams may cause a timeout error, which would then trigger rerouting in our
> error callback.
39672 |     We must not schedule a reroute when device is getting destroyed.
39673 |     */
39674 |     ma_atomic_bool32_set(&pDevice->audio.isTearingDown, MA_TRUE);
39675 |
39676 |     /* Wait for any rerouting to finish before attempting to close the streams. */
39677 |     ma_mutex_lock(&pDevice->audio.rerouteLock);
39678 |     {
39679 |         ma_close_streams__audio(pDevice);
39680 |     }
39681 |     ma_mutex_unlock(&pDevice->audio.rerouteLock);
39682 |
39683 |     /* Destroy rerouting lock. */
39684 |     ma_mutex_uninit(&pDevice->audio.rerouteLock);
39685 |
39686 |     return MA_SUCCESS;
39687 | }
39688 |
39689 | static ma_result ma_device_init_by_type__audio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_type deviceType, ma_device_descriptor* pDescriptor, ma_AAUDIOStream** ppStream)
39690 | {
39691 |     ma_result result;
39692 |     int32_t bufferCapacityInFrames;
39693 |     int32_t framesPerDataCallback;
39694 |     ma_AAUDIOStream* pStream;
39695 |
39696 |     MA_ASSERT(pDevice != NULL);
39697 |     MA_ASSERT(pConfig != NULL);
39698 |     MA_ASSERT(pDescriptor != NULL);
39699 |
39700 |     *ppStream = NULL; /* Safety. */
39701 |
39702 |     /* First step is to open the stream. From there we'll be able to extract the internal
> configuration. */
39703 |     result = ma_open_stream__audio(pDevice, pConfig, deviceType, pDescriptor, &pStream);

```

```

39704 |     if (result != MA_SUCCESS) {
39705 |         return result; /* Failed to open the AAudio stream. */
39706 |     }
39707 |
39708 |     /* Now extract the internal configuration. */
39709 |     pDescriptor->format =
> ((MA_PFN_AAudioStream_getFormat)pDevice->pContext->aaudio.AAudioStream_getFormat)(pStream) ==
> MA_AUDIO_FORMAT_PCM_I16) ? ma_format_s16 : ma_format_f32;
39710 |     pDescriptor->channels = ((MA_PFN_AAudioStream_getChannelCount)pDevice->pContext->aaudio.AAudio
> Stream_getChannelCount)(pStream);
39711 |     pDescriptor->sampleRate =
> ((MA_PFN_AAudioStream_getSampleRate)pDevice->pContext->aaudio.AAudioStream_getSampleRate)(pStream);
39712 |
39713 |     /* For the channel map we need to be sure we don't overflow any buffers. */
39714 |     if (pDescriptor->channels <= MA_MAX_CHANNELS) {
39715 |         ma_channel_map_init_standard(ma_standard_channel_map_default, pDescriptor->channelMap,
> ma_countof(pDescriptor->channelMap), pDescriptor->channels); /* <-- Cannot find info on channel
> order, so assuming a default. */
39716 |     } else {
39717 |         ma_channel_map_init_blank(pDescriptor->channelMap, MA_MAX_CHANNELS); /* Too many channels.
> Use a blank channel map. */
39718 |     }
39719 |
39720 |     bufferCapacityInFrames = ((MA_PFN_AAudioStream_getBufferCapacityInFrames)pDevice->pContext->aud
> io.AAudioStream_getBufferCapacityInFrames)(pStream);
39721 |     framesPerDataCallback = ((MA_PFN_AAudioStream_getFramesPerDataCallback)pDevice->pContext->aaudio
> .AAudioStream_getFramesPerDataCallback)(pStream);
39722 |
39723 |     if (framesPerDataCallback > 0) {
39724 |         pDescriptor->periodSizeInFrames = framesPerDataCallback;
39725 |         pDescriptor->periodCount = bufferCapacityInFrames / framesPerDataCallback;
39726 |     } else {
39727 |         pDescriptor->periodSizeInFrames = bufferCapacityInFrames;
39728 |         pDescriptor->periodCount = 1;
39729 |     }
39730 |
39731 |     *ppStream = pStream;
39732 |
39733 |     return MA_SUCCESS;
39734 | }
39735 |
39736 | static ma_result ma_device_init_streams__aaudio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
39737 | {
39738 |     ma_result result;
39739 |
39740 |     MA_ASSERT(pDevice != NULL);
39741 |
39742 |     if (pConfig->deviceType == ma_device_type_loopback) {
39743 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
39744 |     }
39745 |
39746 |     pDevice->aaudio.usage = pConfig->aaudio.usage;
39747 |     pDevice->aaudio.contentType = pConfig->aaudio.contentType;
39748 |     pDevice->aaudio.inputPreset = pConfig->aaudio.inputPreset;
39749 |     pDevice->aaudio.allowedCapturePolicy = pConfig->aaudio.allowedCapturePolicy;
39750 |     pDevice->aaudio.noAutoStartAfterReroute = pConfig->aaudio.noAutoStartAfterReroute;
39751 |
39752 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
39753 |         result = ma_device_init_by_type__aaudio(pDevice, pConfig, ma_device_type_capture,
> pDescriptorCapture, (ma_AAudioStream**)&pDevice->aaudio.pStreamCapture);
39754 |         if (result != MA_SUCCESS) {
39755 |             return result;
39756 |         }
39757 |     }
39758 |
39759 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
39760 |         result = ma_device_init_by_type__aaudio(pDevice, pConfig, ma_device_type_playback,
> pDescriptorPlayback, (ma_AAudioStream**)&pDevice->aaudio.pStreamPlayback);
39761 |         if (result != MA_SUCCESS) {
39762 |             return result;
39763 |         }
39764 |     }
39765 |
39766 |     return MA_SUCCESS;
39767 | }
39768 |

```

```

39769 | static ma_result ma_device_init__audio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
39770 | {
39771 |     ma_result result;
39772 |
39773 |     MA_ASSERT(pDevice != NULL);
39774 |
39775 |     result = ma_device_init_streams__audio(pDevice, pConfig, pDescriptorPlayback,
> pDescriptorCapture);
39776 |     if (result != MA_SUCCESS) {
39777 |         return result;
39778 |     }
39779 |
39780 |     result = ma_mutex_init(&pDevice->audio.rerouteLock);
39781 |     if (result != MA_SUCCESS) {
39782 |         return result;
39783 |     }
39784 |
39785 |     return MA_SUCCESS;
39786 | }
39787 |
39788 | static ma_result ma_device_start_stream__audio(ma_device* pDevice, ma_AudioStream* pStream)
39789 | {
39790 |     ma_audio_result_t resultAA;
39791 |     ma_audio_stream_state_t currentState;
39792 |
39793 |     MA_ASSERT(pDevice != NULL);
39794 |
39795 |     if (pStream == NULL) {
39796 |         return MA_INVALID_ARGS;
39797 |     }
39798 |
39799 |     resultAA =
> ((MA_PFN_AudioStream_requestStart)pDevice->pContext->audio.AAudioStream_requestStart)(pStream);
39800 |     if (resultAA != MA_AUDIO_OK) {
39801 |         return ma_result_from_audio(resultAA);
39802 |     }
39803 |
39804 |     /* Do we actually need to wait for the device to transition into its started state? */
39805 |
39806 |     /* The device should be in either a starting or started state. If it's not set to started we
> need to wait for it to transition. It should go from starting to started. */
39807 |     currentState =
> ((MA_PFN_AudioStream_getState)pDevice->pContext->audio.AAudioStream_getState)(pStream);
39808 |     if (currentState != MA_AUDIO_STREAM_STATE_STARTED) {
39809 |         ma_result result;
39810 |
39811 |         if (currentState != MA_AUDIO_STREAM_STATE_STARTING) {
39812 |             return MA_ERROR; /* Expecting the stream to be a starting or started state. */
39813 |         }
39814 |
39815 |         result = ma_wait_for_simple_state_transition__audio(pDevice->pContext, pStream,
> currentState, MA_AUDIO_STREAM_STATE_STARTED);
39816 |         if (result != MA_SUCCESS) {
39817 |             return result;
39818 |         }
39819 |     }
39820 |
39821 |     return MA_SUCCESS;
39822 | }
39823 |
39824 | static ma_result ma_device_stop_stream__audio(ma_device* pDevice, ma_AudioStream* pStream)
39825 | {
39826 |     ma_audio_result_t resultAA;
39827 |     ma_audio_stream_state_t currentState;
39828 |
39829 |     MA_ASSERT(pDevice != NULL);
39830 |
39831 |     if (pStream == NULL) {
39832 |         return MA_INVALID_ARGS;
39833 |     }
39834 |
39835 |     /*
39836 |     From the AAudio documentation:
39837 |
39838 |     The stream will stop after all of the data currently buffered has been played.
39839 |
39840 |     This maps with miniaudio's requirement that device's be drained which means we don't need to
> implement any draining logic.

```

```

39841 |     */
39842 |     currentState =
> ((MA_PFN_AAudioStream_getState)pDevice->pContext->aaudio.AAudioStream_getState)(pStream);
39843 |     if (currentState == MA_AAUDIO_STREAM_STATE_DISCONNECTED) {
39844 |         return MA_SUCCESS; /* The device is disconnected. Don't try stopping it. */
39845 |     }
39846 |
39847 |     resultAA =
> ((MA_PFN_AAudioStream_requestStop)pDevice->pContext->aaudio.AAudioStream_requestStop)(pStream);
39848 |     if (resultAA != MA_AAUDIO_OK) {
39849 |         return ma_result_from_aaudio(resultAA);
39850 |     }
39851 |
39852 |     /* The device should be in either a stopping or stopped state. If it's not set to started we
> need to wait for it to transition. It should go from stopping to stopped. */
39853 |     currentState =
> ((MA_PFN_AAudioStream_getState)pDevice->pContext->aaudio.AAudioStream_getState)(pStream);
39854 |     if (currentState != MA_AAUDIO_STREAM_STATE_STOPPED) {
39855 |         ma_result result;
39856 |
39857 |         if (currentState != MA_AAUDIO_STREAM_STATE_STOPPING) {
39858 |             return MA_ERROR; /* Expecting the stream to be a stopping or stopped state. */
39859 |         }
39860 |
39861 |         result = ma_wait_for_simple_state_transition__aaudio(pDevice->pContext, pStream,
> currentState, MA_AAUDIO_STREAM_STATE_STOPPED);
39862 |         if (result != MA_SUCCESS) {
39863 |             return result;
39864 |         }
39865 |     }
39866 |
39867 |     return MA_SUCCESS;
39868 | }
39869 |
39870 | static ma_result ma_device_start__aaudio(ma_device* pDevice)
39871 | {
39872 |     MA_ASSERT(pDevice != NULL);
39873 |
39874 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
39875 |         ma_result result = ma_device_start_stream__aaudio(pDevice,
> (ma_AAudioStream*)pDevice->aaudio.pStreamCapture);
39876 |         if (result != MA_SUCCESS) {
39877 |             return result;
39878 |         }
39879 |     }
39880 |
39881 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
39882 |         ma_result result = ma_device_start_stream__aaudio(pDevice,
> (ma_AAudioStream*)pDevice->aaudio.pStreamPlayback);
39883 |         if (result != MA_SUCCESS) {
39884 |             if (pDevice->type == ma_device_type_duplex) {
39885 |                 ma_device_stop_stream__aaudio(pDevice,
> (ma_AAudioStream*)pDevice->aaudio.pStreamCapture);
39886 |             }
39887 |             return result;
39888 |         }
39889 |     }
39890 |
39891 |     return MA_SUCCESS;
39892 | }
39893 |
39894 | static ma_result ma_device_stop__aaudio(ma_device* pDevice)
39895 | {
39896 |     MA_ASSERT(pDevice != NULL);
39897 |
39898 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
39899 |         ma_result result = ma_device_stop_stream__aaudio(pDevice,
> (ma_AAudioStream*)pDevice->aaudio.pStreamCapture);
39900 |         if (result != MA_SUCCESS) {
39901 |             return result;
39902 |         }
39903 |     }
39904 |
39905 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
39906 |         ma_result result = ma_device_stop_stream__aaudio(pDevice,
> (ma_AAudioStream*)pDevice->aaudio.pStreamPlayback);
39907 |         if (result != MA_SUCCESS) {
39908 |             return result;
39909 |         }

```

```

39910 |     }
39911 |
39912 |     ma_device__on_notification_stopped(pDevice);
39913 |
39914 |     return MA_SUCCESS;
39915 | }
39916 |
39917 | static ma_result ma_device_reinit__aaudio(ma_device* pDevice, ma_device_type deviceType)
39918 | {
39919 |     const ma_int32 maxAttempts = 4; /* Reasonable retry limit. */
39920 |
39921 |     ma_result result;
39922 |     ma_int32 iAttempt;
39923 |
39924 |     MA_ASSERT(pDevice != NULL);
39925 |
39926 |     /* We got disconnected! Retry a few times, until we find a connected device! */
39927 |     iAttempt = 0;
39928 |     while (iAttempt++ < maxAttempts) {
39929 |         /* Device tearing down? No need to reroute! */
39930 |         if (ma_atomic_bool32_get(&pDevice->aaudio.isTearingDown)) {
39931 |             result = MA_SUCCESS; /* Caller should continue as normal. */
39932 |             break;
39933 |         }
39934 |
39935 |         /* The first thing to do is close the streams. */
39936 |         ma_close_streams__aaudio(pDevice);
39937 |
39938 |         /* Now we need to reinitialize each streams. The hardest part with this is just filling
> output the config and descriptors. */
39939 |         ma_device_config deviceConfig;
39940 |         ma_device_descriptor descriptorPlayback;
39941 |         ma_device_descriptor descriptorCapture;
39942 |
39943 |         deviceConfig = ma_device_config_init(deviceType);
39944 |         deviceConfig.playback.pDeviceID = NULL; /* Only doing rerouting with default
> devices. */
39945 |         deviceConfig.playback.shareMode = pDevice->playback.shareMode;
39946 |         deviceConfig.playback.format = pDevice->playback.format;
39947 |         deviceConfig.playback.channels = pDevice->playback.channels;
39948 |         deviceConfig.capture.pDeviceID = NULL; /* Only doing rerouting with default
> devices. */
39949 |         deviceConfig.capture.shareMode = pDevice->capture.shareMode;
39950 |         deviceConfig.capture.format = pDevice->capture.format;
39951 |         deviceConfig.capture.channels = pDevice->capture.channels;
39952 |         deviceConfig.sampleRate = pDevice->sampleRate;
39953 |         deviceConfig.aaudio.usage = pDevice->aaudio.usage;
39954 |         deviceConfig.aaudio.contentType = pDevice->aaudio.contentType;
39955 |         deviceConfig.aaudio.inputPreset = pDevice->aaudio.inputPreset;
39956 |         deviceConfig.aaudio.allowedCapturePolicy = pDevice->aaudio.allowedCapturePolicy;
39957 |         deviceConfig.aaudio.noAutoStartAfterReroute = pDevice->aaudio.noAutoStartAfterReroute;
39958 |         deviceConfig.periods = 1;
39959 |
39960 |         /* Try to get an accurate period size. */
39961 |         if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
39962 |             deviceConfig.periodSizeInFrames = pDevice->playback.internalPeriodSizeInFrames;
39963 |         } else {
39964 |             deviceConfig.periodSizeInFrames = pDevice->capture.internalPeriodSizeInFrames;
39965 |         }
39966 |
39967 |         if (deviceType == ma_device_type_capture || deviceType == ma_device_type_duplex ||
> deviceType == ma_device_type_loopback) {
39968 |             descriptorCapture.pDeviceID = deviceConfig.capture.pDeviceID;
39969 |             descriptorCapture.shareMode = deviceConfig.capture.shareMode;
39970 |             descriptorCapture.format = deviceConfig.capture.format;
39971 |             descriptorCapture.channels = deviceConfig.capture.channels;
39972 |             descriptorCapture.sampleRate = deviceConfig.sampleRate;
39973 |             descriptorCapture.periodSizeInFrames = deviceConfig.periodSizeInFrames;
39974 |             descriptorCapture.periodCount = deviceConfig.periods;
39975 |         }
39976 |
39977 |         if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
39978 |             descriptorPlayback.pDeviceID = deviceConfig.playback.pDeviceID;
39979 |             descriptorPlayback.shareMode = deviceConfig.playback.shareMode;
39980 |             descriptorPlayback.format = deviceConfig.playback.format;
39981 |             descriptorPlayback.channels = deviceConfig.playback.channels;
39982 |             descriptorPlayback.sampleRate = deviceConfig.sampleRate;
39983 |             descriptorPlayback.periodSizeInFrames = deviceConfig.periodSizeInFrames;
39984 |             descriptorPlayback.periodCount = deviceConfig.periods;

```

```

39985 |     }
39986 |
39987 |     result = ma_device_init_streams__audio(pDevice, &deviceConfig, &descriptorPlayback,
> &descriptorCapture);
39988 |     if (result != MA_SUCCESS) {
39989 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[AAudio] Failed to create
> stream after route change.");
39990 |         /* Reroute failed! */
39991 |         break;
39992 |     }
39993 |
39994 |     result = ma_device_post_init(pDevice, deviceType, &descriptorPlayback, &descriptorCapture);
39995 |     if (result != MA_SUCCESS) {
39996 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_WARNING, "[AAudio] Failed to
> initialize device after route change.");
39997 |         ma_close_streams__audio(pDevice);
39998 |         /* Reroute failed! */
39999 |         break;
40000 |     }
40001 |
40002 |     /* We'll only ever do this in response to a reroute. */
40003 |     ma_device__on_notification_rerouted(pDevice);
40004 |
40005 |     /* If the device is started, start the streams. Maybe make this configurable? */
40006 |     if (ma_device_get_state(pDevice) == ma_device_state_started) {
40007 |         if (pDevice->audio.noAutoStartAfterReroute == MA_FALSE) {
40008 |             result = ma_device_start__audio(pDevice);
40009 |             if (result != MA_SUCCESS) {
40010 |                 if (iAttempt < maxAttempts) {
40011 |                     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[AAudio] Failed
> to start stream after route change, retrying(%d)", iAttempt);
40012 |                 } else {
40013 |                     ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[AAudio] Failed
> to start stream after route change, giving up.");
40014 |                 }
40015 |             }
40016 |         } else {
40017 |             ma_device_stop(pDevice); /* Do a full device stop so we set internal state
> correctly. */
40018 |         }
40019 |     }
40020 |
40021 |     if (result == MA_SUCCESS) {
40022 |         /* Reroute successful! */
40023 |         break;
40024 |     }
40025 | }
40026 |
40027 | return result;
40028 | }
40029 |
40030 | static ma_result ma_device_get_info__audio(ma_device* pDevice, ma_device_type type, ma_device_info*
> pDeviceInfo)
40031 | {
40032 |     ma_AAUDIOStream* pStream = NULL;
40033 |
40034 |     MA_ASSERT(pDevice != NULL);
40035 |     MA_ASSERT(type != ma_device_type_duplex);
40036 |     MA_ASSERT(pDeviceInfo != NULL);
40037 |
40038 |     if (type == ma_device_type_capture) {
40039 |         pStream = (ma_AAUDIOStream*)pDevice->audio.pStreamCapture;
40040 |         pDeviceInfo->id.audio = pDevice->capture.id.audio;
40041 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1); /* Only supporting default devices. */
40042 |     }
40043 |     if (type == ma_device_type_playback) {
40044 |         pStream = (ma_AAUDIOStream*)pDevice->audio.pStreamPlayback;
40045 |         pDeviceInfo->id.audio = pDevice->playback.id.audio;
40046 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1); /* Only supporting default devices. */
40047 |     }
40048 |
40049 |     /* Safety. Should never happen. */
40050 |     if (pStream == NULL) {
40051 |         return MA_INVALID_OPERATION;
40052 |     }
40053 |
40054 |     pDeviceInfo->nativeDataFormatCount = 0;

```

```

40055 |     ma_context_add_native_data_format_from_AAUDIOStream__aaudio(pDevice->pContext, pStream, 0,
> pDeviceInfo);
40056 |
40057 |     return MA_SUCCESS;
40058 | }
40059 |
40060 |
40061 | static ma_result ma_context_uninit__aaudio(ma_context* pContext)
40062 | {
40063 |     MA_ASSERT(pContext != NULL);
40064 |     MA_ASSERT(pContext->backend == ma_backend_aaudio);
40065 |
40066 |     ma_device_job_thread_uninit(&pContext->audio.jobThread, &pContext->allocationCallbacks);
40067 |
40068 |     ma_dlclose(ma_context_get_log(pContext), pContext->aaudio.hAAudio);
40069 |     pContext->aaudio.hAAudio = NULL;
40070 |
40071 |     return MA_SUCCESS;
40072 | }
40073 |
40074 | static ma_result ma_context_init__aaudio(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
40075 | {
40076 |     #if !defined(MA_NO_RUNTIME_LINKING)
40077 |         size_t i;
40078 |         const char* libNames[] = {
40079 |             "libaaudio.so"
40080 |         };
40081 |
40082 |         for (i = 0; i < ma_countof(libNames); ++i) {
40083 |             pContext->aaudio.hAAudio = ma_dlopen(ma_context_get_log(pContext), libNames[i]);
40084 |             if (pContext->aaudio.hAAudio != NULL) {
40085 |                 break;
40086 |             }
40087 |         }
40088 |
40089 |         if (pContext->aaudio.hAAudio == NULL) {
40090 |             return MA_FAILED_TO_INIT_BACKEND;
40091 |         }
40092 |
40093 |         pContext->aaudio.AAudio_createStreamBuilder =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudio_createStreamBuilder");
40094 |         pContext->aaudio.AAudioStreamBuilder_delete =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_delete");
40095 |         pContext->aaudio.AAudioStreamBuilder_setDeviceId =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setDeviceId");
40096 |         pContext->aaudio.AAudioStreamBuilder_setDirection =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setDirection");
40097 |         pContext->aaudio.AAudioStreamBuilder_setSharingMode =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setSharingMode");
40098 |         pContext->aaudio.AAudioStreamBuilder_setFormat =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setFormat");
40099 |         pContext->aaudio.AAudioStreamBuilder_setChannelCount =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setChannelCount");
40100 |         pContext->aaudio.AAudioStreamBuilder_setSampleRate =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setSampleRate");
40101 |         pContext->aaudio.AAudioStreamBuilder_setBufferCapacityInFrames =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setBufferCapacityInFrames");
40102 |         pContext->aaudio.AAudioStreamBuilder_setFramesPerDataCallback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setFramesPerDataCallback");
40103 |         pContext->aaudio.AAudioStreamBuilder_setDataCallback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setDataCallback");
40104 |         pContext->aaudio.AAudioStreamBuilder_setErrorCallback =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setErrorCallback");
40105 |         pContext->aaudio.AAudioStreamBuilder_setPerformanceMode =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setPerformanceMode");

```



```

40106 |     pContext->aaudio.AAudioStreamBuilder_setUsage                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setUsage");
40107 |     pContext->aaudio.AAudioStreamBuilder_setContentType          =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setContentType");
40108 |     pContext->aaudio.AAudioStreamBuilder_setInputPreset          =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setInputPreset");
40109 |     pContext->aaudio.AAudioStreamBuilder_setAllowedCapturePolicy =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_setAllowedCapturePolicy");
40110 |     pContext->aaudio.AAudioStreamBuilder_openStream              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStreamBuilder_openStream");
40111 |     pContext->aaudio.AAudioStream_close                           =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio, "AAudioStream_close");
40112 |     pContext->aaudio.AAudioStream_getState                       =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio, "AAudioStream_getState");
40113 |     pContext->aaudio.AAudioStream_waitForStateChange              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_waitForStateChange");
40114 |     pContext->aaudio.AAudioStream_getFormat                      =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio, "AAudioStream_getFormat");
40115 |     pContext->aaudio.AAudioStream_getChannelCount                =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_getChannelCount");
40116 |     pContext->aaudio.AAudioStream_getSampleRate                  =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_getSampleRate");
40117 |     pContext->aaudio.AAudioStream_getBufferCapacityInFrames      =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_getBufferCapacityInFrames");
40118 |     pContext->aaudio.AAudioStream_getFramesPerDataCallback       =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_getFramesPerDataCallback");
40119 |     pContext->aaudio.AAudioStream_getFramesPerBurst              =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_getFramesPerBurst");
40120 |     pContext->aaudio.AAudioStream_requestStart                   =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_requestStart");
40121 |     pContext->aaudio.AAudioStream_requestStop                    =
> (ma_proc)ma_dlsym(ma_context_get_log(pContext), pContext->aaudio.hAAudio,
> "AAudioStream_requestStop");
40122 | #else
40123 |     pContext->aaudio.AAudio_createStreamBuilder                  =
> (ma_proc)AAudio_createStreamBuilder;
40124 |     pContext->aaudio.AAudioStreamBuilder_delete                  =
> (ma_proc)AAudioStreamBuilder_delete;
40125 |     pContext->aaudio.AAudioStreamBuilder_setDeviceId             =
> (ma_proc)AAudioStreamBuilder_setDeviceId;
40126 |     pContext->aaudio.AAudioStreamBuilder_setDirection           =
> (ma_proc)AAudioStreamBuilder_setDirection;
40127 |     pContext->aaudio.AAudioStreamBuilder_setSharingMode          =
> (ma_proc)AAudioStreamBuilder_setSharingMode;
40128 |     pContext->aaudio.AAudioStreamBuilder_setFormat               =
> (ma_proc)AAudioStreamBuilder_setFormat;
40129 |     pContext->aaudio.AAudioStreamBuilder_setChannelCount         =
> (ma_proc)AAudioStreamBuilder_setChannelCount;
40130 |     pContext->aaudio.AAudioStreamBuilder_setSampleRate           =
> (ma_proc)AAudioStreamBuilder_setSampleRate;
40131 |     pContext->aaudio.AAudioStreamBuilder_setBufferCapacityInFrames =
> (ma_proc)AAudioStreamBuilder_setBufferCapacityInFrames;
40132 |     pContext->aaudio.AAudioStreamBuilder_setFramesPerDataCallback =
> (ma_proc)AAudioStreamBuilder_setFramesPerDataCallback;
40133 |     pContext->aaudio.AAudioStreamBuilder_setDataCallback         =
> (ma_proc)AAudioStreamBuilder_setDataCallback;
40134 |     pContext->aaudio.AAudioStreamBuilder_setErrorCallback        =
> (ma_proc)AAudioStreamBuilder_setErrorCallback;
40135 |     pContext->aaudio.AAudioStreamBuilder_setPerformanceMode      =
> (ma_proc)AAudioStreamBuilder_setPerformanceMode;
40136 |     pContext->aaudio.AAudioStreamBuilder_setUsage                =
> (ma_proc)AAudioStreamBuilder_setUsage;
40137 |     pContext->aaudio.AAudioStreamBuilder_setContentType          =
> (ma_proc)AAudioStreamBuilder_setContentType;
40138 |     pContext->aaudio.AAudioStreamBuilder_setInputPreset          =
> (ma_proc)AAudioStreamBuilder_setInputPreset;
40139 |     #if defined(__ANDROID_API__) && __ANDROID_API__ >= 29

```

```

40140 |     pContext->aaudio.AAudioStreamBuilder_setAllowedCapturePolicy    =
> (ma_proc)AAudioStreamBuilder_setAllowedCapturePolicy;
40141 |     #endif
40142 |     pContext->aaudio.AAudioStreamBuilder_openStream                =
> (ma_proc)AAudioStreamBuilder_openStream;
40143 |     pContext->aaudio.AAudioStream_close                            = (ma_proc)AAudioStream_close;
40144 |     pContext->aaudio.AAudioStream_getState                        = (ma_proc)AAudioStream_getState;
40145 |     pContext->aaudio.AAudioStream_waitForStateChange              =
> (ma_proc)AAudioStream_waitForStateChange;
40146 |     pContext->aaudio.AAudioStream_getFormat                      =
> (ma_proc)AAudioStream_getFormat;
40147 |     pContext->aaudio.AAudioStream_getChannelCount                =
> (ma_proc)AAudioStream_getChannelCount;
40148 |     pContext->aaudio.AAudioStream_getSampleRate                  =
> (ma_proc)AAudioStream_getSampleRate;
40149 |     pContext->aaudio.AAudioStream_getBufferCapacityInFrames      =
> (ma_proc)AAudioStream_getBufferCapacityInFrames;
40150 |     pContext->aaudio.AAudioStream_getFramesPerDataCallback       =
> (ma_proc)AAudioStream_getFramesPerDataCallback;
40151 |     pContext->aaudio.AAudioStream_getFramesPerBurst              =
> (ma_proc)AAudioStream_getFramesPerBurst;
40152 |     pContext->aaudio.AAudioStream_requestStart                    =
> (ma_proc)AAudioStream_requestStart;
40153 |     pContext->aaudio.AAudioStream_requestStop                    =
> (ma_proc)AAudioStream_requestStop;
40154 | #endif
40155 |
40156 |     pCallbacks->onContextInit                                     = ma_context_init__aaudio;
40157 |     pCallbacks->onContextUninit                                   = ma_context_uninit__aaudio;
40158 |     pCallbacks->onContextEnumerateDevices                       = ma_context_enumerate_devices__aaudio;
40159 |     pCallbacks->onContextGetDeviceInfo                         = ma_context_get_device_info__aaudio;
40160 |     pCallbacks->onDeviceInit                                     = ma_device_init__aaudio;
40161 |     pCallbacks->onDeviceUninit                                   = ma_device_uninit__aaudio;
40162 |     pCallbacks->onDeviceStart                                    = ma_device_start__aaudio;
40163 |     pCallbacks->onDeviceStop                                     = ma_device_stop__aaudio;
40164 |     pCallbacks->onDeviceRead                                    = NULL; /* Not used because AAudio is asynchronous. */
40165 |     pCallbacks->onDeviceWrite                                    = NULL; /* Not used because AAudio is asynchronous. */
40166 |     pCallbacks->onDeviceDataLoop                                = NULL; /* Not used because AAudio is asynchronous. */
40167 |     pCallbacks->onDeviceGetInfo                                  = ma_device_get_info__aaudio;
40168 |
40169 |
40170 |     /* We need a job thread so we can deal with rerouting. */
40171 |     {
40172 |         ma_result result;
40173 |         ma_device_job_thread_config jobThreadConfig;
40174 |
40175 |         jobThreadConfig = ma_device_job_thread_config_init();
40176 |
40177 |         result = ma_device_job_thread_init(&jobThreadConfig, &pContext->allocationCallbacks,
> &pContext->aaudio.jobThread);
40178 |         if (result != MA_SUCCESS) {
40179 |             ma_dlclose(ma_context_get_log(pContext), pContext->aaudio.hAAudio);
40180 |             pContext->aaudio.hAAudio = NULL;
40181 |             return result;
40182 |         }
40183 |     }
40184 |
40185 |
40186 |     (void)pConfig;
40187 |     return MA_SUCCESS;
40188 | }
40189 |
40190 | static ma_result ma_job_process__device__aaudio_reroute(ma_job* pJob)
40191 | {
40192 |     ma_result result = MA_SUCCESS;
40193 |     ma_device* pDevice;
40194 |
40195 |     MA_ASSERT(pJob != NULL);
40196 |
40197 |     pDevice = (ma_device*)pJob->data.device.aaudio.reroute.pDevice;
40198 |     MA_ASSERT(pDevice != NULL);
40199 |
40200 |     ma_mutex_lock(&pDevice->aaudio.rerouteLock);
40201 |     {
40202 |         /* Here is where we need to reroute the device. To do this we need to uninitialized the
> stream and reinitialize it. */
40203 |         result = ma_device_reinit__aaudio(pDevice,
> (ma_device_type)pJob->data.device.aaudio.reroute.deviceType);
40204 |         if (result != MA_SUCCESS) {

```

```

40205 |             /*
40206 |             Getting here means we failed to reroute the device. The best thing I can think of here
> is to
40207 |             just stop the device.
40208 |             */
40209 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[AAudio] Stopping device
> due to reroute failure.");
40210 |             ma_device_stop(pDevice);
40211 |         }
40212 |     }
40213 |     ma_mutex_unlock(&pDevice->aaudio.rerouteLock);
40214 |
40215 |     return result;
40216 | }
40217 | #else
40218 | /* Getting here means there is no AAudio backend so we need a no-op job implementation. */
40219 | static ma_result ma_job_process_device_aaudio_reroute(ma_job* pJob)
40220 | {
40221 |     return ma_job_process_noop(pJob);
40222 | }
40223 | #endif /* AAudio */
40224 |
40225 |
40226 | /*****
40227 |
40228 | OpenSL|ES Backend
40229 |
40230 | *****/
40231 | #ifdef MA_HAS_OPENS
40232 | #include <SLES/OpenSLES.h>
40233 | #ifdef MA_ANDROID
40234 | #include <SLES/OpenSLES_Android.h>
40235 | #endif
40236 |
40237 | typedef SLresult (SLAPIENTRY * ma_slCreateEngine_proc)(SLObjectItf* pEngine, SLuint32 numOptions,
> SLEngineOption* pEngineOptions, SLuint32 numInterfaces, SLInterfaceID* pInterfaceIds, SLboolean*
> pInterfaceRequired);
40238 |
40239 | /* OpenSL|ES has one-per-application objects :( */
40240 | static SLObjectItf g_maEngineObjectSL = NULL;
40241 | static SLEngineItf g_maEngineSL = NULL;
40242 | static ma_uint32 g_maOpenSLInitCounter = 0;
40243 | static ma_spinlock g_maOpenSLSpinlock = 0; /* For init/uninit. */
40244 |
40245 | #define MA_OPENS_OBJ(p)      (*((SLObjectItf)(p)))
40246 | #define MA_OPENS_OUTPUTMIX(p)  (*((SLOutputMixItf)(p)))
40247 | #define MA_OPENS_PLAY(p)      (*((SLPlayItf)(p)))
40248 | #define MA_OPENS_RECORD(p)    (*((SLRecordItf)(p)))
40249 |
40250 | #ifdef MA_ANDROID
40251 | #define MA_OPENS_BUFFERQUEUE(p) (*((SLAndroidSimpleBufferQueueItf)(p)))
40252 | #else
40253 | #define MA_OPENS_BUFFERQUEUE(p) (*((SLBufferQueueItf)(p)))
40254 | #endif
40255 |
40256 | static ma_result ma_result_from_OpenSL(SLuint32 result)
40257 | {
40258 |     switch (result)
40259 |     {
40260 |         case SL_RESULT_SUCCESS:                return MA_SUCCESS;
40261 |         case SL_RESULT_PRECONDITIONS_VIOLATED: return MA_ERROR;
40262 |         case SL_RESULT_PARAMETER_INVALID:       return MA_INVALID_ARGS;
40263 |         case SL_RESULT_MEMORY_FAILURE:          return MA_OUT_OF_MEMORY;
40264 |         case SL_RESULT_RESOURCE_ERROR:          return MA_INVALID_DATA;
40265 |         case SL_RESULT_RESOURCE_LOST:           return MA_ERROR;
40266 |         case SL_RESULT_IO_ERROR:                return MA_IO_ERROR;
40267 |         case SL_RESULT_BUFFER_INSUFFICIENT:     return MA_NO_SPACE;
40268 |         case SL_RESULT_CONTENT_CORRUPTED:       return MA_INVALID_DATA;
40269 |         case SL_RESULT_CONTENT_UNSUPPORTED:     return MA_FORMAT_NOT_SUPPORTED;
40270 |         case SL_RESULT_CONTENT_NOT_FOUND:       return MA_ERROR;
40271 |         case SL_RESULT_PERMISSION_DENIED:       return MA_ACCESS_DENIED;
40272 |         case SL_RESULT_FEATURE_UNSUPPORTED:      return MA_NOT_IMPLEMENTED;
40273 |         case SL_RESULT_INTERNAL_ERROR:          return MA_ERROR;
40274 |         case SL_RESULT_UNKNOWN_ERROR:           return MA_ERROR;
40275 |         case SL_RESULT_OPERATION_ABORTED:       return MA_ERROR;
40276 |         case SL_RESULT_CONTROL_LOST:            return MA_ERROR;
40277 |         default:                                return MA_ERROR;
40278 |     }
40279 | }

```

```

40280 |
40281 | /* Converts an individual OpenSL-style channel identifier (SL_SPEAKER_FRONT_LEFT, etc.) to
> miniaudio. */
40282 | static ma_uint8 ma_channel_id_to_ma_opensl(SLuint32 id)
40283 | {
40284 |     switch (id)
40285 |     {
40286 |         case SL_SPEAKER_FRONT_LEFT:         return MA_CHANNEL_FRONT_LEFT;
40287 |         case SL_SPEAKER_FRONT_RIGHT:        return MA_CHANNEL_FRONT_RIGHT;
40288 |         case SL_SPEAKER_FRONT_CENTER:        return MA_CHANNEL_FRONT_CENTER;
40289 |         case SL_SPEAKER_LOW_FREQUENCY:       return MA_CHANNEL_LFE;
40290 |         case SL_SPEAKER_BACK_LEFT:           return MA_CHANNEL_BACK_LEFT;
40291 |         case SL_SPEAKER_BACK_RIGHT:          return MA_CHANNEL_BACK_RIGHT;
40292 |         case SL_SPEAKER_FRONT_LEFT_OF_CENTER: return MA_CHANNEL_FRONT_LEFT_CENTER;
40293 |         case SL_SPEAKER_FRONT_RIGHT_OF_CENTER: return MA_CHANNEL_FRONT_RIGHT_CENTER;
40294 |         case SL_SPEAKER_BACK_CENTER:         return MA_CHANNEL_BACK_CENTER;
40295 |         case SL_SPEAKER_SIDE_LEFT:           return MA_CHANNEL_SIDE_LEFT;
40296 |         case SL_SPEAKER_SIDE_RIGHT:          return MA_CHANNEL_SIDE_RIGHT;
40297 |         case SL_SPEAKER_TOP_CENTER:          return MA_CHANNEL_TOP_CENTER;
40298 |         case SL_SPEAKER_TOP_FRONT_LEFT:      return MA_CHANNEL_TOP_FRONT_LEFT;
40299 |         case SL_SPEAKER_TOP_FRONT_CENTER:    return MA_CHANNEL_TOP_FRONT_CENTER;
40300 |         case SL_SPEAKER_TOP_FRONT_RIGHT:     return MA_CHANNEL_TOP_FRONT_RIGHT;
40301 |         case SL_SPEAKER_TOP_BACK_LEFT:       return MA_CHANNEL_TOP_BACK_LEFT;
40302 |         case SL_SPEAKER_TOP_BACK_CENTER:     return MA_CHANNEL_TOP_BACK_CENTER;
40303 |         case SL_SPEAKER_TOP_BACK_RIGHT:      return MA_CHANNEL_TOP_BACK_RIGHT;
40304 |         default: return 0;
40305 |     }
40306 | }
40307 |
40308 | /* Converts an individual miniaudio channel identifier (MA_CHANNEL_FRONT_LEFT, etc.) to
> OpenSL-style. */
40309 | static SLuint32 ma_channel_id_to_opensl(ma_uint8 id)
40310 | {
40311 |     switch (id)
40312 |     {
40313 |         case MA_CHANNEL_MONO:                return SL_SPEAKER_FRONT_CENTER;
40314 |         case MA_CHANNEL_FRONT_LEFT:          return SL_SPEAKER_FRONT_LEFT;
40315 |         case MA_CHANNEL_FRONT_RIGHT:         return SL_SPEAKER_FRONT_RIGHT;
40316 |         case MA_CHANNEL_FRONT_CENTER:        return SL_SPEAKER_FRONT_CENTER;
40317 |         case MA_CHANNEL_LFE:                 return SL_SPEAKER_LOW_FREQUENCY;
40318 |         case MA_CHANNEL_BACK_LEFT:           return SL_SPEAKER_BACK_LEFT;
40319 |         case MA_CHANNEL_BACK_RIGHT:          return SL_SPEAKER_BACK_RIGHT;
40320 |         case MA_CHANNEL_FRONT_LEFT_CENTER:    return SL_SPEAKER_FRONT_LEFT_OF_CENTER;
40321 |         case MA_CHANNEL_FRONT_RIGHT_CENTER:   return SL_SPEAKER_FRONT_RIGHT_OF_CENTER;
40322 |         case MA_CHANNEL_BACK_CENTER:         return SL_SPEAKER_BACK_CENTER;
40323 |         case MA_CHANNEL_SIDE_LEFT:           return SL_SPEAKER_SIDE_LEFT;
40324 |         case MA_CHANNEL_SIDE_RIGHT:          return SL_SPEAKER_SIDE_RIGHT;
40325 |         case MA_CHANNEL_TOP_CENTER:          return SL_SPEAKER_TOP_CENTER;
40326 |         case MA_CHANNEL_TOP_FRONT_LEFT:      return SL_SPEAKER_TOP_FRONT_LEFT;
40327 |         case MA_CHANNEL_TOP_FRONT_CENTER:    return SL_SPEAKER_TOP_FRONT_CENTER;
40328 |         case MA_CHANNEL_TOP_FRONT_RIGHT:     return SL_SPEAKER_TOP_FRONT_RIGHT;
40329 |         case MA_CHANNEL_TOP_BACK_LEFT:       return SL_SPEAKER_TOP_BACK_LEFT;
40330 |         case MA_CHANNEL_TOP_BACK_CENTER:     return SL_SPEAKER_TOP_BACK_CENTER;
40331 |         case MA_CHANNEL_TOP_BACK_RIGHT:      return SL_SPEAKER_TOP_BACK_RIGHT;
40332 |         default: return 0;
40333 |     }
40334 | }
40335 |
40336 | /* Converts a channel mapping to an OpenSL-style channel mask. */
40337 | static SLuint32 ma_channel_map_to_channel_mask_opensl(const ma_channel* pChannelMap, ma_uint32
> channels)
40338 | {
40339 |     SLuint32 channelMask = 0;
40340 |     ma_uint32 iChannel;
40341 |     for (iChannel = 0; iChannel < channels; ++iChannel) {
40342 |         channelMask |= ma_channel_id_to_opensl(pChannelMap[iChannel]);
40343 |     }
40344 |
40345 |     return channelMask;
40346 | }
40347 |
40348 | /* Converts an OpenSL-style channel mask to a miniaudio channel map. */
40349 | static void ma_channel_mask_to_channel_map_opensl(SLuint32 channelMask, ma_uint32 channels,
> ma_channel* pChannelMap)
40350 | {
40351 |     if (channels == 1 && channelMask == 0) {
40352 |         pChannelMap[0] = MA_CHANNEL_MONO;
40353 |     } else if (channels == 2 && channelMask == 0) {
40354 |         pChannelMap[0] = MA_CHANNEL_FRONT_LEFT;

```

```

40355 |     pChannelMap[1] = MA_CHANNEL_FRONT_RIGHT;
40356 | } else {
40357 |     if (channels == 1 && (channelMask & SL_SPEAKER_FRONT_CENTER) != 0) {
40358 |         pChannelMap[0] = MA_CHANNEL_MONO;
40359 |     } else {
40360 |         /* Just iterate over each bit. */
40361 |         ma_uint32 iChannel = 0;
40362 |         ma_uint32 iBit;
40363 |         for (iBit = 0; iBit < 32 && iChannel < channels; ++iBit) {
40364 |             SLuint32 bitValue = (channelMask & (1UL << iBit));
40365 |             if (bitValue != 0) {
40366 |                 /* The bit is set. */
40367 |                 pChannelMap[iChannel] = ma_channel_id_to_ma_opensl(bitValue);
40368 |                 iChannel += 1;
40369 |             }
40370 |         }
40371 |     }
40372 | }
40373 | }
40374 |
40375 | static SLuint32 ma_round_to_standard_sample_rate__opensl(SLuint32 samplesPerSec)
40376 | {
40377 |     if (samplesPerSec <= SL_SAMPLINGRATE_8) {
40378 |         return SL_SAMPLINGRATE_8;
40379 |     }
40380 |     if (samplesPerSec <= SL_SAMPLINGRATE_11_025) {
40381 |         return SL_SAMPLINGRATE_11_025;
40382 |     }
40383 |     if (samplesPerSec <= SL_SAMPLINGRATE_12) {
40384 |         return SL_SAMPLINGRATE_12;
40385 |     }
40386 |     if (samplesPerSec <= SL_SAMPLINGRATE_16) {
40387 |         return SL_SAMPLINGRATE_16;
40388 |     }
40389 |     if (samplesPerSec <= SL_SAMPLINGRATE_22_05) {
40390 |         return SL_SAMPLINGRATE_22_05;
40391 |     }
40392 |     if (samplesPerSec <= SL_SAMPLINGRATE_24) {
40393 |         return SL_SAMPLINGRATE_24;
40394 |     }
40395 |     if (samplesPerSec <= SL_SAMPLINGRATE_32) {
40396 |         return SL_SAMPLINGRATE_32;
40397 |     }
40398 |     if (samplesPerSec <= SL_SAMPLINGRATE_44_1) {
40399 |         return SL_SAMPLINGRATE_44_1;
40400 |     }
40401 |     if (samplesPerSec <= SL_SAMPLINGRATE_48) {
40402 |         return SL_SAMPLINGRATE_48;
40403 |     }
40404 |
40405 |     /* Android doesn't support more than 48000. */
40406 | #ifndef MA_ANDROID
40407 |     if (samplesPerSec <= SL_SAMPLINGRATE_64) {
40408 |         return SL_SAMPLINGRATE_64;
40409 |     }
40410 |     if (samplesPerSec <= SL_SAMPLINGRATE_88_2) {
40411 |         return SL_SAMPLINGRATE_88_2;
40412 |     }
40413 |     if (samplesPerSec <= SL_SAMPLINGRATE_96) {
40414 |         return SL_SAMPLINGRATE_96;
40415 |     }
40416 |     if (samplesPerSec <= SL_SAMPLINGRATE_192) {
40417 |         return SL_SAMPLINGRATE_192;
40418 |     }
40419 | #endif
40420 |
40421 |     return SL_SAMPLINGRATE_16;
40422 | }
40423 |
40424 |
40425 | static SLint32 ma_to_stream_type__opensl(ma_opensl_stream_type streamType)
40426 | {
40427 |     switch (streamType) {
40428 |         case ma_opensl_stream_type_voice:         return SL_ANDROID_STREAM_VOICE;
40429 |         case ma_opensl_stream_type_system:        return SL_ANDROID_STREAM_SYSTEM;
40430 |         case ma_opensl_stream_type_ring:          return SL_ANDROID_STREAM_RING;
40431 |         case ma_opensl_stream_type_media:         return SL_ANDROID_STREAM_MEDIA;
40432 |         case ma_opensl_stream_type_alarm:         return SL_ANDROID_STREAM_ALARM;
40433 |         case ma_opensl_stream_type_notification:  return SL_ANDROID_STREAM_NOTIFICATION;

```

```

40434 |         default: break;
40435 |     }
40436 |
40437 |     return SL_ANDROID_STREAM_VOICE;
40438 | }
40439 |
40440 | static SLint32 ma_to_recording_preset__openssl(ma_opensl_recording_preset recordingPreset)
40441 | {
40442 |     switch (recordingPreset) {
40443 |         case ma_opensl_recording_preset_generic:           return
> SL_ANDROID_RECORDING_PRESET_GENERIC;
40444 |         case ma_opensl_recording_preset_camcorder:         return
> SL_ANDROID_RECORDING_PRESET_CAMCORDER;
40445 |         case ma_opensl_recording_preset_voice_recognition: return
> SL_ANDROID_RECORDING_PRESET_VOICE_RECOGNITION;
40446 |         case ma_opensl_recording_preset_voice_communication: return
> SL_ANDROID_RECORDING_PRESET_VOICE_COMMUNICATION;
40447 |         case ma_opensl_recording_preset_voice_unprocessed: return
> SL_ANDROID_RECORDING_PRESET_UNPROCESSED;
40448 |         default: break;
40449 |     }
40450 |
40451 |     return SL_ANDROID_RECORDING_PRESET_NONE;
40452 | }
40453 |
40454 |
40455 | static ma_result ma_context_enumerate_devices__openssl(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
40456 | {
40457 |     ma_bool32 cbResult;
40458 |
40459 |     MA_ASSERT(pContext != NULL);
40460 |     MA_ASSERT(callback != NULL);
40461 |
40462 |     MA_ASSERT(g_maOpenSLInitCounter > 0); /* <-- If you trigger this it means you've either not
> initialized the context, or you've uninitialized it and then attempted to enumerate devices. */
40463 |     if (g_maOpenSLInitCounter == 0) {
40464 |         return MA_INVALID_OPERATION;
40465 |     }
40466 |
40467 |     /*
40468 |     TODO: Test Me.
40469 |
40470 |     This is currently untested, so for now we are just returning default devices.
40471 |     */
40472 |     #if 0 && !defined(MA_ANDROID)
40473 |         ma_bool32 isTerminated = MA_FALSE;
40474 |
40475 |         SLuint32 pDeviceIDs[128];
40476 |         SLint32 deviceCount = sizeof(pDeviceIDs) / sizeof(pDeviceIDs[0]);
40477 |
40478 |         SLAudioIODeviceCapabilitiesItf deviceCaps;
40479 |         SLresult resultSL = (*g_maEngineObjectSL)->GetInterface(g_maEngineObjectSL,
> (SLInterfaceID)pContext->openssl.SL_IID_AUDIOIODEVICECAPABILITIES, &deviceCaps);
40480 |         if (resultSL != SL_RESULT_SUCCESS) {
40481 |             /* The interface may not be supported so just report a default device. */
40482 |             goto return_default_device;
40483 |         }
40484 |
40485 |         /* Playback */
40486 |         if (!isTerminated) {
40487 |             resultSL = (*deviceCaps)->GetAvailableAudioOutputs(deviceCaps, &deviceCount, pDeviceIDs);
40488 |             if (resultSL != SL_RESULT_SUCCESS) {
40489 |                 return ma_result_from_OpenSL(resultSL);
40490 |             }
40491 |
40492 |             for (SLint32 iDevice = 0; iDevice < deviceCount; ++iDevice) {
40493 |                 ma_device_info deviceInfo;
40494 |                 MA_ZERO_OBJECT(&deviceInfo);
40495 |                 deviceInfo.id.opensl = pDeviceIDs[iDevice];
40496 |
40497 |                 SLAudioOutputDescriptor desc;
40498 |                 resultSL = (*deviceCaps)->QueryAudioOutputCapabilities(deviceCaps, deviceInfo.id.opensl,
> &desc);
40499 |                 if (resultSL == SL_RESULT_SUCCESS) {
40500 |                     ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), (const
> char*)desc.pDeviceName, (size_t)-1);
40501 |
40502 |                     ma_bool32 cbResult = callback(pContext, ma_device_type_playback, &deviceInfo,

```



```

> pUserData);
40503 |         if (cbResult == MA_FALSE) {
40504 |             isTerminated = MA_TRUE;
40505 |             break;
40506 |         }
40507 |     }
40508 | }
40509 | }
40510 |
40511 | /* Capture */
40512 | if (!isTerminated) {
40513 |     resultSL = (*deviceCaps)->GetAvailableAudioInputs(deviceCaps, &deviceCount, pDeviceIDs);
40514 |     if (resultSL != SL_RESULT_SUCCESS) {
40515 |         return ma_result_from_OpenSL(resultSL);
40516 |     }
40517 |
40518 |     for (SLint32 iDevice = 0; iDevice < deviceCount; ++iDevice) {
40519 |         ma_device_info deviceInfo;
40520 |         MA_ZERO_OBJECT(&deviceInfo);
40521 |         deviceInfo.id.opensl = pDeviceIDs[iDevice];
40522 |
40523 |         SLAudioInputDescriptor desc;
40524 |         resultSL = (*deviceCaps)->QueryAudioInputCapabilities(deviceCaps, deviceInfo.id.opensl,
> &desc);
40525 |         if (resultSL == SL_RESULT_SUCCESS) {
40526 |             ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), (const char*)desc.deviceName,
> (size_t)-1);
40527 |
40528 |             ma_bool32 cbResult = callback(pContext, ma_device_type_capture, &deviceInfo,
> pUserData);
40529 |             if (cbResult == MA_FALSE) {
40530 |                 isTerminated = MA_TRUE;
40531 |                 break;
40532 |             }
40533 |         }
40534 |     }
40535 | }
40536 |
40537 | return MA_SUCCESS;
40538 | #else
40539 | goto return_default_device;
40540 | #endif
40541 |
40542 | return_default_device::
40543 |     cbResult = MA_TRUE;
40544 |
40545 |     /* Playback. */
40546 |     if (cbResult) {
40547 |         ma_device_info deviceInfo;
40548 |         MA_ZERO_OBJECT(&deviceInfo);
40549 |         deviceInfo.id.opensl = SL_DEFAULTDEVICEID_AUDIOOUTPUT;
40550 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
40551 |         cbResult = callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
40552 |     }
40553 |
40554 |     /* Capture. */
40555 |     if (cbResult) {
40556 |         ma_device_info deviceInfo;
40557 |         MA_ZERO_OBJECT(&deviceInfo);
40558 |         deviceInfo.id.opensl = SL_DEFAULTDEVICEID_AUDIOINPUT;
40559 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
40560 |         cbResult = callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
40561 |     }
40562 |
40563 |     return MA_SUCCESS;
40564 | }
40565 |
40566 | static void ma_context_add_data_format_ex_opensl(ma_context* pContext, ma_format format, ma_uint32
> channels, ma_uint32 sampleRate, ma_device_info* pDeviceInfo)
40567 | {
40568 |     MA_ASSERT(pContext != NULL);
40569 |     MA_ASSERT(pDeviceInfo != NULL);
40570 |
40571 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].format = format;
40572 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].channels = channels;
40573 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].sampleRate = sampleRate;
40574 |     pDeviceInfo->nativeDataFormats[pDeviceInfo->nativeDataFormatCount].flags = 0;

```



```

40575 |     pDeviceInfo->nativeDataFormatCount += 1;
40576 | }
40577 |
40578 | static void ma_context_add_data_format__openssl(ma_context* pContext, ma_format format,
> ma_device_info* pDeviceInfo)
40579 | {
40580 |     ma_uint32 minChannels    = 1;
40581 |     ma_uint32 maxChannels    = 2;
40582 |     ma_uint32 minSampleRate = (ma_uint32)ma_standard_sample_rate_8000;
40583 |     ma_uint32 maxSampleRate = (ma_uint32)ma_standard_sample_rate_48000;
40584 |     ma_uint32 iChannel;
40585 |     ma_uint32 iSampleRate;
40586 |
40587 |     MA_ASSERT(pContext    != NULL);
40588 |     MA_ASSERT(pDeviceInfo != NULL);
40589 |
40590 |     /*
40591 |     Each sample format can support mono and stereo, and we'll support a small subset of standard
40592 |     rates (up to 48000). A better solution would be to somehow find a native sample rate.
40593 |     */
40594 |     for (iChannel = minChannels; iChannel < maxChannels; iChannel += 1) {
40595 |         for (iSampleRate = 0; iSampleRate < ma_countof(g_maStandardSampleRatePriorities);
> iSampleRate += 1) {
40596 |             ma_uint32 standardSampleRate = g_maStandardSampleRatePriorities[iSampleRate];
40597 |             if (standardSampleRate >= minSampleRate && standardSampleRate <= maxSampleRate) {
40598 |                 ma_context_add_data_format_ex__openssl(pContext, format, iChannel,
> standardSampleRate, pDeviceInfo);
40599 |             }
40600 |         }
40601 |     }
40602 | }
40603 |
40604 | static ma_result ma_context_get_device_info__openssl(ma_context* pContext, ma_device_type deviceType,
> const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
40605 | {
40606 |     MA_ASSERT(pContext != NULL);
40607 |
40608 |     MA_ASSERT(g_maOpenSLInitCounter > 0); /* <-- If you trigger this it means you've either not
> initialized the context, or you've uninitialized it and then attempted to get device info. */
40609 |     if (g_maOpenSLInitCounter == 0) {
40610 |         return MA_INVALID_OPERATION;
40611 |     }
40612 |
40613 |     /*
40614 |     TODO: Test Me.
40615 |
40616 |     This is currently untested, so for now we are just returning default devices.
40617 |     */
40618 |     #if 0 && !defined(MA_ANDROID)
40619 |         SLAudioIODeviceCapabilitiesItf deviceCaps;
40620 |         SLresult resultSL = (*g_maEngineObjectSL)->GetInterface(g_maEngineObjectSL,
> (SLInterfaceID)pContext->openssl.SL_IID_AUDIOIODEVICECAPABILITIES, &deviceCaps);
40621 |         if (resultSL != SL_RESULT_SUCCESS) {
40622 |             /* The interface may not be supported so just report a default device. */
40623 |             goto return_default_device;
40624 |         }
40625 |
40626 |         if (deviceType == ma_device_type_playback) {
40627 |             SLAudioOutputDescriptor desc;
40628 |             resultSL = (*deviceCaps)->QueryAudioOutputCapabilities(deviceCaps, pDeviceID->openssl,
> &desc);
40629 |             if (resultSL != SL_RESULT_SUCCESS) {
40630 |                 return ma_result_from_OpenSL(resultSL);
40631 |             }
40632 |
40633 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), (const char*)desc.pDeviceName,
> (size_t)-1);
40634 |         } else {
40635 |             SLAudioInputDescriptor desc;
40636 |             resultSL = (*deviceCaps)->QueryAudioInputCapabilities(deviceCaps, pDeviceID->openssl, &desc);
40637 |             if (resultSL != SL_RESULT_SUCCESS) {
40638 |                 return ma_result_from_OpenSL(resultSL);
40639 |             }
40640 |
40641 |             ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), (const char*)desc.deviceName,
> (size_t)-1);
40642 |         }
40643 |
40644 |         goto return_detailed_info;

```

```

40645 | #else
40646 |     goto return_default_device;
40647 | #endif
40648 |
40649 | return_default_device:
40650 |     if (pDeviceID != NULL) {
40651 |         if ((deviceType == ma_device_type_playback && pDeviceID->opensl !=
> SL_DEFAULTDEVICEID_AUDIOOUTPUT) ||
40652 |             (deviceType == ma_device_type_capture && pDeviceID->opensl !=
> SL_DEFAULTDEVICEID_AUDIOINPUT)) {
40653 |             return MA_NO_DEVICE; /* Don't know the device. */
40654 |         }
40655 |     }
40656 |
40657 |     /* ID and Name / Description */
40658 |     if (deviceType == ma_device_type_playback) {
40659 |         pDeviceInfo->id.opensl = SL_DEFAULTDEVICEID_AUDIOOUTPUT;
40660 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
40661 |     } else {
40662 |         pDeviceInfo->id.opensl = SL_DEFAULTDEVICEID_AUDIOINPUT;
40663 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
40664 |     }
40665 |
40666 |     pDeviceInfo->isDefault = MA_TRUE;
40667 |
40668 |     goto return_detailed_info;
40669 |
40670 |
40671 | return_detailed_info:
40672 |
40673 |     /*
40674 |     For now we're just outputting a set of values that are supported by the API but not necessarily
> supported
40675 |     by the device natively. Later on we should work on this so that it more closely reflects the
> device's
40676 |     actual native format.
40677 |     */
40678 |     pDeviceInfo->nativeDataFormatCount = 0;
40679 |     #if defined(MA_ANDROID) && __ANDROID_API__ >= 21
40680 |     ma_context_add_data_format__opensl(pContext, ma_format_f32, pDeviceInfo);
40681 |     #endif
40682 |     ma_context_add_data_format__opensl(pContext, ma_format_s16, pDeviceInfo);
40683 |     ma_context_add_data_format__opensl(pContext, ma_format_u8, pDeviceInfo);
40684 |
40685 |     return MA_SUCCESS;
40686 | }
40687 |
40688 |
40689 | #ifdef MA_ANDROID
40690 | /*void ma_buffer_queue_callback_capture__opensl_android(SLAndroidSimpleBufferQueueItf pBufferQueue,
> SLuint32 eventFlags, const void* pBuffer, SLuint32 bufferSize, SLuint32 dataUsed, void* pContext)*/
40691 | static void ma_buffer_queue_callback_capture__opensl_android(SLAndroidSimpleBufferQueueItf
> pBufferQueue, void* pUserData)
40692 | {
40693 |     ma_device* pDevice = (ma_device*)pUserData;
40694 |     size_t periodSizeInBytes;
40695 |     ma_uint8* pBuffer;
40696 |     SLresult resultSL;
40697 |
40698 |     MA_ASSERT(pDevice != NULL);
40699 |
40700 |     (void)pBufferQueue;
40701 |
40702 |     /*
40703 |     For now, don't do anything unless the buffer was fully processed. From what I can tell, it looks
> like
40704 |     OpenSL|ES 1.1 improves on buffer queues to the point that we could much more intelligently
> handle this,
40705 |     but unfortunately it looks like Android is only supporting OpenSL|ES 1.0.1 for now :(
40706 |     */
40707 |
40708 |     /* Don't do anything if the device is not started. */
40709 |     if (ma_device_get_state(pDevice) != ma_device_state_started) {
40710 |         return;
40711 |     }
40712 |
40713 |     /* Don't do anything if the device is being drained. */

```

```

40714 |     if (pDevice->opensl.isDrainingCapture) {
40715 |         return;
40716 |     }
40717 |
40718 |     periodSizeInBytes = pDevice->capture.internalPeriodSizeInFrames *
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels);
40719 |     pBuffer = pDevice->opensl.pBufferCapture + (pDevice->opensl.currentBufferIndexCapture *
> periodSizeInBytes);
40720 |
40721 |     ma_device_handle_backend_data_callback(pDevice, NULL, pBuffer,
> pDevice->capture.internalPeriodSizeInFrames);
40722 |
40723 |     resultSL = MA_OPENS�_BUFFERQUEUE(pDevice->opensl.pBufferQueueCapture)->Enqueue((SLAndroidSimpleB
> ufferQueueItf)pDevice->opensl.pBufferQueueCapture, pBuffer, periodSizeInBytes);
40724 |     if (resultSL != SL_RESULT_SUCCESS) {
40725 |         return;
40726 |     }
40727 |
40728 |     pDevice->opensl.currentBufferIndexCapture = (pDevice->opensl.currentBufferIndexCapture + 1) %
> pDevice->capture.internalPeriods;
40729 | }
40730 |
40731 | static void ma_buffer_queue_callback_playback__opensl_android(SLAndroidSimpleBufferQueueItf
> pBufferQueue, void* pUserData)
40732 | {
40733 |     ma_device* pDevice = (ma_device*)pUserData;
40734 |     size_t periodSizeInBytes;
40735 |     ma_uint8* pBuffer;
40736 |     SLresult resultSL;
40737 |
40738 |     MA_ASSERT(pDevice != NULL);
40739 |
40740 |     (void)pBufferQueue;
40741 |
40742 |     /* Don't do anything if the device is not started. */
40743 |     if (ma_device_get_state(pDevice) != ma_device_state_started) {
40744 |         return;
40745 |     }
40746 |
40747 |     /* Don't do anything if the device is being drained. */
40748 |     if (pDevice->opensl.isDrainingPlayback) {
40749 |         return;
40750 |     }
40751 |
40752 |     periodSizeInBytes = pDevice->playback.internalPeriodSizeInFrames *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
40753 |     pBuffer = pDevice->opensl.pBufferPlayback + (pDevice->opensl.currentBufferIndexPlayback *
> periodSizeInBytes);
40754 |
40755 |     ma_device_handle_backend_data_callback(pDevice, pBuffer, NULL,
> pDevice->playback.internalPeriodSizeInFrames);
40756 |
40757 |     resultSL = MA_OPENS�_BUFFERQUEUE(pDevice->opensl.pBufferQueuePlayback)->Enqueue((SLAndroidSimple
> BufferQueueItf)pDevice->opensl.pBufferQueuePlayback, pBuffer, periodSizeInBytes);
40758 |     if (resultSL != SL_RESULT_SUCCESS) {
40759 |         return;
40760 |     }
40761 |
40762 |     pDevice->opensl.currentBufferIndexPlayback = (pDevice->opensl.currentBufferIndexPlayback + 1) %
> pDevice->playback.internalPeriods;
40763 | }
40764 | #endif
40765 |
40766 | static ma_result ma_device_uninit__opensl(ma_device* pDevice)
40767 | {
40768 |     MA_ASSERT(pDevice != NULL);
40769 |
40770 |     MA_ASSERT(g_maOpenSLInitCounter > 0); /* <-- If you trigger this it means you've either not
> initialized the context, or you've uninitialized it before uninitialized the device. */
40771 |     if (g_maOpenSLInitCounter == 0) {
40772 |         return MA_INVALID_OPERATION;
40773 |     }
40774 |
40775 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
40776 |         if (pDevice->opensl.pAudioRecorderObj) {
40777 |             MA_OPENS�_OBJ(pDevice->opensl.pAudioRecorderObj)->Destroy((SLObjectItf)pDevice->opensl.p
> AudioRecorderObj);
40778 |         }
40779 |     }

```

```

40780 |         ma_free(pDevice->opensl.pBufferCapture, &pDevice->pContext->allocationCallbacks);
40781 |     }
40782 |
40783 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
40784 |         if (pDevice->opensl.pAudioPlayerObj) {
40785 |             MA_OPENSL_OBJ(pDevice->opensl.pAudioPlayerObj)->Destroy((SLObjectItf)pDevice->opensl.pAu
> dioPlayerObj);
40786 |         }
40787 |         if (pDevice->opensl.pOutputMixObj) {
40788 |             MA_OPENSL_OBJ(pDevice->opensl.pOutputMixObj)->Destroy((SLObjectItf)pDevice->opensl.pOutputMixObj);
40789 |         }
40790 |
40791 |         ma_free(pDevice->opensl.pBufferPlayback, &pDevice->pContext->allocationCallbacks);
40792 |     }
40793 |
40794 |     return MA_SUCCESS;
40795 | }
40796 |
40797 | #if defined(MA_ANDROID) && __ANDROID_API__ >= 21
40798 | typedef SLAndroidDataFormat_PCM_EX ma_SLDataFormat_PCM;
40799 | #else
40800 | typedef SLDataFormat_PCM          ma_SLDataFormat_PCM;
40801 | #endif
40802 |
40803 | static ma_result ma_SLDataFormat_PCM_init__opensl(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, const ma_channel* channelMap, ma_SLDataFormat_PCM* pDataFormat)
40804 | {
40805 |     /* We need to convert our format/channels/rate so that they aren't set to default. */
40806 |     if (format == ma_format_unknown) {
40807 |         format = MA_DEFAULT_FORMAT;
40808 |     }
40809 |     if (channels == 0) {
40810 |         channels = MA_DEFAULT_CHANNELS;
40811 |     }
40812 |     if (sampleRate == 0) {
40813 |         sampleRate = MA_DEFAULT_SAMPLE_RATE;
40814 |     }
40815 |
40816 |     #if defined(MA_ANDROID) && __ANDROID_API__ >= 21
40817 |         if (format == ma_format_f32) {
40818 |             pDataFormat->formatType = SL_ANDROID_DATAFORMAT_PCM_EX;
40819 |             pDataFormat->representation = SL_ANDROID_PCM_REPRESENTATION_FLOAT;
40820 |         } else {
40821 |             pDataFormat->formatType = SL_DATAFORMAT_PCM;
40822 |         }
40823 |     #else
40824 |         pDataFormat->formatType = SL_DATAFORMAT_PCM;
40825 |     #endif
40826 |
40827 |     pDataFormat->numChannels = channels;
40828 |     ((SLDataFormat_PCM*)pDataFormat)->samplesPerSec =
> ma_round_to_standard_sample_rate__opensl(sampleRate * 1000); /* In millihertz. Annoyingly, the
> sample rate variable is named differently between SLAndroidDataFormat_PCM_EX and SLDataFormat_PCM */
40829 |     pDataFormat->bitsPerSample = ma_get_bytes_per_sample(format) * 8;
40830 |     pDataFormat->channelMask = ma_channel_map_to_channel_mask__opensl(channelMap, channels);
40831 |     pDataFormat->endianness = (ma_is_little_endian()) ? SL_BYTEORDER_LITTLEENDIAN :
> SL_BYTEORDER_BIGENDIAN;
40832 |
40833 |     /*
40834 |     Android has a few restrictions on the format as documented here:
> https://developer.android.com/ndk/guides/audio/opensl-for-android.html
40835 |     - Only mono and stereo is supported.
40836 |     - Only u8 and s16 formats are supported.
40837 |     - Maximum sample rate of 48000.
40838 |     */
40839 |     #ifdef MA_ANDROID
40840 |         if (pDataFormat->numChannels > 2) {
40841 |             pDataFormat->numChannels = 2;
40842 |         }
40843 |     #if __ANDROID_API__ >= 21
40844 |         if (pDataFormat->formatType == SL_ANDROID_DATAFORMAT_PCM_EX) {
40845 |             /* It's floating point. */
40846 |             MA_ASSERT(pDataFormat->representation == SL_ANDROID_PCM_REPRESENTATION_FLOAT);
40847 |             if (pDataFormat->bitsPerSample > 32) {
40848 |                 pDataFormat->bitsPerSample = 32;
40849 |             }
40850 |         } else {
40851 |             if (pDataFormat->bitsPerSample > 16) {

```

```

40852 |         pDataFormat->bitsPerSample = 16;
40853 |     }
40854 | }
40855 | #else
40856 |     if (pDataFormat->bitsPerSample > 16) {
40857 |         pDataFormat->bitsPerSample = 16;
40858 |     }
40859 | #endif
40860 |     if (((SLDataFormat_PCM*)pDataFormat)->samplesPerSec > SL_SAMPLINGRATE_48) {
40861 |         ((SLDataFormat_PCM*)pDataFormat)->samplesPerSec = SL_SAMPLINGRATE_48;
40862 |     }
40863 | #endif
40864 |
40865 |     pDataFormat->containerSize = pDataFormat->bitsPerSample; /* Always tightly packed for now. */
40866 |
40867 |     return MA_SUCCESS;
40868 | }
40869 |
40870 | static ma_result ma_deconstruct_SLDataFormat_PCM__opensl(ma_SLDataFormat_PCM* pDataFormat,
> ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
40871 | {
40872 |     ma_bool32 isFloatingPoint = MA_FALSE;
40873 |     #if defined(MA_ANDROID) && __ANDROID_API__ >= 21
40874 |         if (pDataFormat->formatType == SL_ANDROID_DATAFORMAT_PCM_EX) {
40875 |             MA_ASSERT(pDataFormat->representation == SL_ANDROID_PCM_REPRESENTATION_FLOAT);
40876 |             isFloatingPoint = MA_TRUE;
40877 |         }
40878 |     #endif
40879 |     if (isFloatingPoint) {
40880 |         if (pDataFormat->bitsPerSample == 32) {
40881 |             *pFormat = ma_format_f32;
40882 |         }
40883 |     } else {
40884 |         if (pDataFormat->bitsPerSample == 8) {
40885 |             *pFormat = ma_format_u8;
40886 |         } else if (pDataFormat->bitsPerSample == 16) {
40887 |             *pFormat = ma_format_s16;
40888 |         } else if (pDataFormat->bitsPerSample == 24) {
40889 |             *pFormat = ma_format_s24;
40890 |         } else if (pDataFormat->bitsPerSample == 32) {
40891 |             *pFormat = ma_format_s32;
40892 |         }
40893 |     }
40894 |
40895 |     *pChannels = pDataFormat->numChannels;
40896 |     *pSampleRate = ((SLDataFormat_PCM*)pDataFormat)->samplesPerSec / 1000;
40897 |     ma_channel_mask_to_channel_map__opensl(pDataFormat->channelMask,
> ma_min(pDataFormat->numChannels, channelMapCap), pChannelMap);
40898 |
40899 |     return MA_SUCCESS;
40900 | }
40901 |
40902 | static ma_result ma_device_init__opensl(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
40903 | {
40904 |     #ifdef MA_ANDROID
40905 |         SLDataLocator_AndroidSimpleBufferQueue queue;
40906 |         SLresult resultSL;
40907 |         size_t bufferSizeInBytes;
40908 |         SLInterfaceID itfIDs[2];
40909 |         const SLboolean itfIDsRequired[] = {
40910 |             SL_BOOLEAN_TRUE, /* SL_IID_ANDROIDSIMPLEBUFFERQUEUE */
40911 |             SL_BOOLEAN_FALSE /* SL_IID_ANDROIDCONFIGURATION */
40912 |         };
40913 |     #endif
40914 |
40915 |     MA_ASSERT(g_maOpenSLInitCounter > 0); /* <-- If you trigger this it means you've either not
> initialized the context, or you've uninitialized it and then attempted to initialize a new device.
> */
40916 |     if (g_maOpenSLInitCounter == 0) {
40917 |         return MA_INVALID_OPERATION;
40918 |     }
40919 |
40920 |     if (pConfig->deviceType == ma_device_type_loopback) {
40921 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
40922 |     }
40923 |
40924 |     /*

```

```

40925 |     For now, only supporting Android implementations of OpenSL|ES since that's the only one I've
40926 |     been able to test with and I currently depend on Android-specific extensions (simple buffer
40927 |     queues).
40928 |     */
40929 | #ifdef MA_ANDROID
40930 |     itfIDs[0] = (SLInterfaceID)pDevice->pContext->opensl.SL_IID_ANDROIDSIMPLEBUFFERQUEUE;
40931 |     itfIDs[1] = (SLInterfaceID)pDevice->pContext->opensl.SL_IID_ANDROIDCONFIGURATION;
40932 |
40933 |     /* No exclusive mode with OpenSL|ES. */
40934 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->shareMode == ma_share_mode_exclusive) ||
40935 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->shareMode == ma_share_mode_exclusive)) {
40936 |         return MA_SHARE_MODE_NOT_SUPPORTED;
40937 |     }
40938 |
40939 |     /* Now we can start initializing the device properly. */
40940 |     MA_ASSERT(pDevice != NULL);
40941 |     MA_ZERO_OBJECT(&pDevice->opensl);
40942 |
40943 |     queue.locatorType = SL_DATALOCATOR_ANDROIDSIMPLEBUFFERQUEUE;
40944 |
40945 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
40946 |         ma_SLDataFormat_PCM pcm;
40947 |         SLDataLocator_IODEvice locatorDevice;
40948 |         SLDataSource source;
40949 |         SLDataSink sink;
40950 |         SLAndroidConfigurationItf pRecorderConfig;
40951 |
40952 |         ma_SLDataFormat_PCM_init__opensl(pDescriptorCapture->format, pDescriptorCapture->channels,
> pDescriptorCapture->sampleRate, pDescriptorCapture->channelMap, &pcm);
40953 |
40954 |         locatorDevice.locatorType = SL_DATALOCATOR_IODEVICE;
40955 |         locatorDevice.deviceType = SL_IODEVICE_AUDIOINPUT;
40956 |         locatorDevice.deviceID = SL_DEFAULTDEVICEID_AUDIOINPUT; /* Must always use the default
> device with Android. */
40957 |         locatorDevice.device = NULL;
40958 |
40959 |         source.pLocator = &locatorDevice;
40960 |         source.pFormat = NULL;
40961 |
40962 |         queue.numBuffers = pDescriptorCapture->periodCount;
40963 |
40964 |         sink.pLocator = &queue;
40965 |         sink.pFormat = (SLDataFormat_PCM*)&pcm;
40966 |
40967 |         resultSL = (*g_maEngineSL)->CreateAudioRecorder(g_maEngineSL,
> (SLObjectItf*)&pDevice->opensl.pAudioRecorderObj, &source, &sink, ma_countof(itfIDs), itfIDs,
> itfIDsRequired);
40968 |         if (resultSL == SL_RESULT_CONTENT_UNSUPPORTED || resultSL == SL_RESULT_PARAMETER_INVALID) {
40969 |             /* Unsupported format. Fall back to something safer and try again. If this fails, just
> abort. */
40970 |             pcm.formatType = SL_DATAFORMAT_PCM;
40971 |             pcm.numChannels = 1;
40972 |             ((SLDataFormat_PCM*)&pcm)->samplesPerSec = SL_SAMPLINGRATE_16; /* The name of the
> sample rate variable is different between SLAndroidDataFormat_PCM_EX and SLDataFormat_PCM. */
40973 |             pcm.bitsPerSample = 16;
40974 |             pcm.containerSize = pcm.bitsPerSample; /* Always tightly packed for now. */
40975 |             pcm.channelMask = 0;
40976 |             resultSL = (*g_maEngineSL)->CreateAudioRecorder(g_maEngineSL,
> (SLObjectItf*)&pDevice->opensl.pAudioRecorderObj, &source, &sink, ma_countof(itfIDs), itfIDs,
> itfIDsRequired);
40977 |         }
40978 |
40979 |         if (resultSL != SL_RESULT_SUCCESS) {
40980 |             ma_device_uninit__opensl(pDevice);
40981 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to create
> audio recorder.");
40982 |             return ma_result_from_OpenSL(resultSL);
40983 |         }
40984 |
40985 |
40986 |         /* Set the recording preset before realizing the player. */
40987 |         if (pConfig->opensl.recordingPreset != ma_opensl_recording_preset_default) {
40988 |             resultSL = MA_OPENS�_OBJ(pDevice->opensl.pAudioRecorderObj)->GetInterface((SLObjectItf)p
> Device->opensl.pAudioRecorderObj,
> (SLInterfaceID)pDevice->pContext->opensl.SL_IID_ANDROIDCONFIGURATION, &pRecorderConfig);
40989 |             if (resultSL == SL_RESULT_SUCCESS) {

```

```

40990 |         SLint32 recordingPreset =
> ma_to_recording_preset__openssl(pConfig->openssl.recordingPreset);
40991 |         resultSL = (*pRecorderConfig)->SetConfiguration(pRecorderConfig,
> SL_ANDROID_KEY_RECORDING_PRESET, &recordingPreset, sizeof(SLint32));
40992 |         if (resultSL != SL_RESULT_SUCCESS) {
40993 |             /* Failed to set the configuration. Just keep going. */
40994 |         }
40995 |     }
40996 | }
40997 |
40998 |     resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioRecorderObj)->Realize((SLObjectItf)pDevice->o
> pensl.pAudioRecorderObj, SL_BOOLEAN_FALSE);
40999 |     if (resultSL != SL_RESULT_SUCCESS) {
41000 |         ma_device_uninit__openssl(pDevice);
41001 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to realize
> audio recorder.");
41002 |         return ma_result_from_OpenSL(resultSL);
41003 |     }
41004 |
41005 |     resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioRecorderObj)->GetInterface((SLObjectItf)pDevi
> ce->openssl.pAudioRecorderObj, (SLInterfaceID)pDevice->pContext->openssl.SL_IID_RECORD,
> &pDevice->openssl.pAudioRecorder);
41006 |     if (resultSL != SL_RESULT_SUCCESS) {
41007 |         ma_device_uninit__openssl(pDevice);
41008 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to retrieve
> SL_IID_RECORD interface.");
41009 |         return ma_result_from_OpenSL(resultSL);
41010 |     }
41011 |
41012 |     resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioRecorderObj)->GetInterface((SLObjectItf)pDevi
> ce->openssl.pAudioRecorderObj,
> (SLInterfaceID)pDevice->pContext->openssl.SL_IID_ANDROIDSIMPLEBUFFERQUEUE,
> &pDevice->openssl.pBufferQueueCapture);
41013 |     if (resultSL != SL_RESULT_SUCCESS) {
41014 |         ma_device_uninit__openssl(pDevice);
41015 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to retrieve
> SL_IID_ANDROIDSIMPLEBUFFERQUEUE interface.");
41016 |         return ma_result_from_OpenSL(resultSL);
41017 |     }
41018 |
41019 |     resultSL = MA_OPENSSL_BUFFERQUEUE(pDevice->openssl.pBufferQueueCapture)->RegisterCallback((SLA
> ndroidSimpleBufferQueueItf)pDevice->openssl.pBufferQueueCapture,
> ma_buffer_queue_callback_capture__openssl_android, pDevice);
41020 |     if (resultSL != SL_RESULT_SUCCESS) {
41021 |         ma_device_uninit__openssl(pDevice);
41022 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to register
> buffer queue callback.");
41023 |         return ma_result_from_OpenSL(resultSL);
41024 |     }
41025 |
41026 |     /* The internal format is determined by the "pcm" object. */
41027 |     ma_deconstruct_SLDataFormat_PCM__openssl(&pcm, &pDescriptorCapture->format,
> &pDescriptorCapture->channels, &pDescriptorCapture->sampleRate, pDescriptorCapture->channelMap,
> ma_countof(pDescriptorCapture->channelMap));
41028 |
41029 |     /* Buffer. */
41030 |     pDescriptorCapture->periodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptorCapture,
> pDescriptorCapture->sampleRate, pConfig->performanceProfile);
41031 |     pDevice->openssl.currentBufferIndexCapture = 0;
41032 |
41033 |     bufferSizeInBytes = pDescriptorCapture->periodSizeInFrames *
> ma_get_bytes_per_frame(pDescriptorCapture->format, pDescriptorCapture->channels) *
> pDescriptorCapture->periodCount;
41034 |     pDevice->openssl.pBufferCapture = (ma_uint8*)ma_calloc(bufferSizeInBytes,
> &pDevice->pContext->allocationCallbacks);
41035 |     if (pDevice->openssl.pBufferCapture == NULL) {
41036 |         ma_device_uninit__openssl(pDevice);
41037 |         ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to allocate
> memory for data buffer.");
41038 |         return MA_OUT_OF_MEMORY;
41039 |     }
41040 |     MA_ZERO_MEMORY(pDevice->openssl.pBufferCapture, bufferSizeInBytes);
41041 | }
41042 |
41043 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
41044 |         ma_SLDataFormat_PCM pcm;
41045 |         SLDataSource source;

```



```

41046 |         SLDataLocator_OutputMix outmixLocator;
41047 |         SLDataSink sink;
41048 |         SLAndroidConfigurationItf pPlayerConfig;
41049 |
41050 |         ma_SLDataFormat_PCM_init__opensl(pDescriptorPlayback->format, pDescriptorPlayback->channels,
> pDescriptorPlayback->sampleRate, pDescriptorPlayback->channelMap, &pcm);
41051 |
41052 |         resultSL = (*g_maEngineSL)->CreateOutputMix(g_maEngineSL,
> (SLObjectItf*)&pDevice->opensl.pOutputMixObj, 0, NULL, NULL);
41053 |         if (resultSL != SL_RESULT_SUCCESS) {
41054 |             ma_device_uninit__opensl(pDevice);
41055 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to create
> output mix.");
41056 |             return ma_result_from_OpenSL(resultSL);
41057 |         }
41058 |
41059 |         resultSL =
> MA_OPENSL_OBJ(pDevice->opensl.pOutputMixObj)->Realize((SLObjectItf)pDevice->opensl.pOutputMixObj,
> SL_BOOLEAN_FALSE);
41060 |         if (resultSL != SL_RESULT_SUCCESS) {
41061 |             ma_device_uninit__opensl(pDevice);
41062 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to realize
> output mix object.");
41063 |             return ma_result_from_OpenSL(resultSL);
41064 |         }
41065 |
41066 |         resultSL = MA_OPENSL_OBJ(pDevice->opensl.pOutputMixObj)->GetInterface((SLObjectItf)pDevice->
> opensl.pOutputMixObj, (SLInterfaceID)pDevice->pContext->opensl.SL_IID_OUTPUTMIX,
> &pDevice->opensl.pOutputMix);
41067 |         if (resultSL != SL_RESULT_SUCCESS) {
41068 |             ma_device_uninit__opensl(pDevice);
41069 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to retrieve
> SL_IID_OUTPUTMIX interface.");
41070 |             return ma_result_from_OpenSL(resultSL);
41071 |         }
41072 |
41073 |         /* Set the output device. */
41074 |         if (pDescriptorPlayback->pDeviceID != NULL) {
41075 |             SLuint32 deviceID_OpenSL = pDescriptorPlayback->pDeviceID->opensl;
41076 |
> MA_OPENSL_OUTPUTMIX(pDevice->opensl.pOutputMix)->ReRoute((SLOutputMixItf)pDevice->opensl.pOutputMix,
> 1, &deviceID_OpenSL);
41077 |         }
41078 |
41079 |         queue.numBuffers = pDescriptorPlayback->periodCount;
41080 |
41081 |         source.pLocator = &queue;
41082 |         source.pFormat = (SLDataFormat_PCM*)&pcm;
41083 |
41084 |         outmixLocator.locatorType = SL_DATALOCATOR_OUTPUTMIX;
41085 |         outmixLocator.outputMix = (SLObjectItf)pDevice->opensl.pOutputMixObj;
41086 |
41087 |         sink.pLocator = &outmixLocator;
41088 |         sink.pFormat = NULL;
41089 |
41090 |         resultSL = (*g_maEngineSL)->CreateAudioPlayer(g_maEngineSL,
> (SLObjectItf*)&pDevice->opensl.pAudioPlayerObj, &source, &sink, ma_countof(itfIDs), itfIDs,
> itfIDsRequired);
41091 |         if (resultSL == SL_RESULT_CONTENT_UNSUPPORTED || resultSL == SL_RESULT_PARAMETER_INVALID) {
41092 |             /* Unsupported format. Fall back to something safer and try again. If this fails, just
> abort. */
41093 |             pcm.formatType = SL_DATAFORMAT_PCM;
41094 |             pcm.numChannels = 2;
41095 |             ((SLDataFormat_PCM*)&pcm)->samplesPerSec = SL_SAMPLINGRATE_16;
41096 |             pcm.bitsPerSample = 16;
41097 |             pcm.containerSize = pcm.bitsPerSample; /* Always tightly packed for now. */
41098 |             pcm.channelMask = SL_SPEAKER_FRONT_LEFT | SL_SPEAKER_FRONT_RIGHT;
41099 |             resultSL = (*g_maEngineSL)->CreateAudioPlayer(g_maEngineSL,
> (SLObjectItf*)&pDevice->opensl.pAudioPlayerObj, &source, &sink, ma_countof(itfIDs), itfIDs,
> itfIDsRequired);
41100 |         }
41101 |
41102 |         if (resultSL != SL_RESULT_SUCCESS) {
41103 |             ma_device_uninit__opensl(pDevice);
41104 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to create
> audio player.");
41105 |             return ma_result_from_OpenSL(resultSL);
41106 |         }
41107 |

```

```

41108 |
41109 |         /* Set the stream type before realizing the player. */
41110 |         if (pConfig->openssl.streamType != ma_opensl_stream_type_default) {
41111 |             resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioPlayerObj)->GetInterface((SLObjectItf)pDe
> vice->openssl.pAudioPlayerObj, (SLInterfaceID)pDevice->pContext->openssl.SL_IID_ANDROIDCONFIGURATION,
> &pPlayerConfig);
41112 |             if (resultSL == SL_RESULT_SUCCESS) {
41113 |                 SLint32 streamType = ma_to_stream_type__openssl(pConfig->openssl.streamType);
41114 |                 resultSL = (*pPlayerConfig)->SetConfiguration(pPlayerConfig,
> SL_ANDROID_KEY_STREAM_TYPE, &streamType, sizeof(SLint32));
41115 |                 if (resultSL != SL_RESULT_SUCCESS) {
41116 |                     /* Failed to set the configuration. Just keep going. */
41117 |                 }
41118 |             }
41119 |         }
41120 |
41121 |         resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioPlayerObj)->Realize((SLObjectItf)pDevice->ope
> nsl.pAudioPlayerObj, SL_BOOLEAN_FALSE);
41122 |         if (resultSL != SL_RESULT_SUCCESS) {
41123 |             ma_device_uninit__openssl(pDevice);
41124 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to realize
> audio player.");
41125 |             return ma_result_from_OpenSL(resultSL);
41126 |         }
41127 |
41128 |         resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioPlayerObj)->GetInterface((SLObjectItf)pDevice
> ->openssl.pAudioPlayerObj, (SLInterfaceID)pDevice->pContext->openssl.SL_IID_PLAY,
> &pDevice->openssl.pAudioPlayer);
41129 |         if (resultSL != SL_RESULT_SUCCESS) {
41130 |             ma_device_uninit__openssl(pDevice);
41131 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to retrieve
> SL_IID_PLAY interface.");
41132 |             return ma_result_from_OpenSL(resultSL);
41133 |         }
41134 |
41135 |         resultSL = MA_OPENSSL_OBJ(pDevice->openssl.pAudioPlayerObj)->GetInterface((SLObjectItf)pDevice
> ->openssl.pAudioPlayerObj, (SLInterfaceID)pDevice->pContext->openssl.SL_IID_ANDROIDSIMPLEBUFFERQUEUE,
> &pDevice->openssl.pBufferQueuePlayback);
41136 |         if (resultSL != SL_RESULT_SUCCESS) {
41137 |             ma_device_uninit__openssl(pDevice);
41138 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to retrieve
> SL_IID_ANDROIDSIMPLEBUFFERQUEUE interface.");
41139 |             return ma_result_from_OpenSL(resultSL);
41140 |         }
41141 |
41142 |         resultSL = MA_OPENSSL_BUFFERQUEUE(pDevice->openssl.pBufferQueuePlayback)->RegisterCallback((SL
> AndroidSimpleBufferQueueItf)pDevice->openssl.pBufferQueuePlayback,
> ma_buffer_queue_callback_playback__openssl_android, pDevice);
41143 |         if (resultSL != SL_RESULT_SUCCESS) {
41144 |             ma_device_uninit__openssl(pDevice);
41145 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to register
> buffer queue callback.");
41146 |             return ma_result_from_OpenSL(resultSL);
41147 |         }
41148 |
41149 |         /* The internal format is determined by the "pcm" object. */
41150 |         ma_deconstruct_SLDataFormat_PCM__openssl(&pcm, &pDescriptorPlayback->format,
> &pDescriptorPlayback->channels, &pDescriptorPlayback->sampleRate, pDescriptorPlayback->channelMap,
> ma_countof(pDescriptorPlayback->channelMap));
41151 |
41152 |         /* Buffer. */
41153 |         pDescriptorPlayback->periodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_descriptor(pDescriptorPlayback,
> pDescriptorPlayback->sampleRate, pConfig->performanceProfile);
41154 |         pDevice->openssl.currentBufferIndexPlayback = 0;
41155 |
41156 |         bufferSizeInBytes = pDescriptorPlayback->periodSizeInFrames *
> ma_get_bytes_per_frame(pDescriptorPlayback->format, pDescriptorPlayback->channels) *
> pDescriptorPlayback->periodCount;
41157 |         pDevice->openssl.pBufferPlayback = (ma_uint8*)ma_malloc(bufferSizeInBytes,
> &pDevice->pContext->allocationCallbacks);
41158 |         if (pDevice->openssl.pBufferPlayback == NULL) {
41159 |             ma_device_uninit__openssl(pDevice);
41160 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to allocate
> memory for data buffer.");
41161 |             return MA_OUT_OF_MEMORY;
41162 |         }
41163 |         MA_ZERO_MEMORY(pDevice->openssl.pBufferPlayback, bufferSizeInBytes);
41164 |     }

```

```

41165 |
41166 |     return MA_SUCCESS;
41167 | #else
41168 |     return MA_NO_BACKEND; /* Non-Android implementations are not supported. */
41169 | #endif
41170 | }
41171 |
41172 | static ma_result ma_device_start_opensl(ma_device* pDevice)
41173 | {
41174 |     SLresult resultSL;
41175 |     size_t periodSizeInBytes;
41176 |     ma_uint32 iPeriod;
41177 |
41178 |     MA_ASSERT(pDevice != NULL);
41179 |
41180 |     MA_ASSERT(g_maOpenSLInitCounter > 0); /* <-- If you trigger this it means you've either not
> initialized the context, or you've uninitialized it and then attempted to start the device. */
41181 |     if (g_maOpenSLInitCounter == 0) {
41182 |         return MA_INVALID_OPERATION;
41183 |     }
41184 |
41185 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
41186 |         resultSL = MA_OPENS�_RECORD(pDevice->opensl.pAudioRecorder)->SetRecordState((SLRecordItf)pDe
> vice->opensl.pAudioRecorder, SL_RECORDSTATE_RECORDING);
41187 |         if (resultSL != SL_RESULT_SUCCESS) {
41188 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to start
> internal capture device.");
41189 |             return ma_result_from_OpenSL(resultSL);
41190 |         }
41191 |
41192 |         periodSizeInBytes = pDevice->capture.internalPeriodSizeInFrames *
> ma_get_bytes_per_frame(pDevice->capture.internalFormat, pDevice->capture.internalChannels);
41193 |         for (iPeriod = 0; iPeriod < pDevice->capture.internalPeriods; ++iPeriod) {
41194 |             resultSL = MA_OPENS�_BUFFERQUEUE(pDevice->opensl.pBufferQueueCapture)->Enqueue((SLAndroi
> dSimpleBufferQueueItf)pDevice->opensl.pBufferQueueCapture, pDevice->opensl.pBufferCapture +
> (periodSizeInBytes * iPeriod), periodSizeInBytes);
41195 |             if (resultSL != SL_RESULT_SUCCESS) {
41196 |                 MA_OPENS�_RECORD(pDevice->opensl.pAudioRecorder)->SetRecordState((SLRecordItf)pDevic
> e->opensl.pAudioRecorder, SL_RECORDSTATE_STOPPED);
41197 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to
> enqueue buffer for capture device.");
41198 |                 return ma_result_from_OpenSL(resultSL);
41199 |             }
41200 |         }
41201 |     }
41202 |
41203 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
41204 |         resultSL =
> MA_OPENS�_PLAY(pDevice->opensl.pAudioPlayer)->SetPlayState((SLPlayItf)pDevice->opensl.pAudioPlayer,
> SL_PLAYSTATE_PLAYING);
41205 |         if (resultSL != SL_RESULT_SUCCESS) {
41206 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to start
> internal playback device.");
41207 |             return ma_result_from_OpenSL(resultSL);
41208 |         }
41209 |
41210 |         /* In playback mode (no duplex) we need to load some initial buffers. In duplex mode we need
> to enqueue silent buffers. */
41211 |         if (pDevice->type == ma_device_type_duplex) {
41212 |             MA_ZERO_MEMORY(pDevice->opensl.pBufferPlayback,
> pDevice->playback.internalPeriodSizeInFrames * pDevice->playback.internalPeriods *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels));
41213 |         } else {
41214 |             ma_device__read_frames_from_client(pDevice, pDevice->playback.internalPeriodSizeInFrames
> * pDevice->playback.internalPeriods, pDevice->opensl.pBufferPlayback);
41215 |         }
41216 |
41217 |         periodSizeInBytes = pDevice->playback.internalPeriodSizeInFrames *
> ma_get_bytes_per_frame(pDevice->playback.internalFormat, pDevice->playback.internalChannels);
41218 |         for (iPeriod = 0; iPeriod < pDevice->playback.internalPeriods; ++iPeriod) {
41219 |             resultSL = MA_OPENS�_BUFFERQUEUE(pDevice->opensl.pBufferQueuePlayback)->Enqueue((SLAndroi
> dSimpleBufferQueueItf)pDevice->opensl.pBufferQueuePlayback, pDevice->opensl.pBufferPlayback +
> (periodSizeInBytes * iPeriod), periodSizeInBytes);
41220 |             if (resultSL != SL_RESULT_SUCCESS) {
41221 |
> MA_OPENS�_PLAY(pDevice->opensl.pAudioPlayer)->SetPlayState((SLPlayItf)pDevice->opensl.pAudioPlayer,
> SL_PLAYSTATE_STOPPED);
41222 |                 ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to
> enqueue buffer for playback device.");

```

```

41223 |         return ma_result_from_OpenSL(resultSL);
41224 |     }
41225 | }
41226 | }
41227 |
41228 |     return MA_SUCCESS;
41229 | }
41230 |
41231 | static ma_result ma_device_drain_opensl(ma_device* pDevice, ma_device_type deviceType)
41232 | {
41233 |     SLAndroidSimpleBufferQueueItf pBufferQueue;
41234 |
41235 |     MA_ASSERT(deviceType == ma_device_type_capture || deviceType == ma_device_type_playback);
41236 |
41237 |     if (pDevice->type == ma_device_type_capture) {
41238 |         pBufferQueue = (SLAndroidSimpleBufferQueueItf)pDevice->opensl.pBufferQueueCapture;
41239 |         pDevice->opensl.isDrainingCapture = MA_TRUE;
41240 |     } else {
41241 |         pBufferQueue = (SLAndroidSimpleBufferQueueItf)pDevice->opensl.pBufferQueuePlayback;
41242 |         pDevice->opensl.isDrainingPlayback = MA_TRUE;
41243 |     }
41244 |
41245 |     for (;;) {
41246 |         SLAndroidSimpleBufferQueueState state;
41247 |
41248 |         MA_OPENSL_BUFFERQUEUE(pBufferQueue)->GetState(pBufferQueue, &state);
41249 |         if (state.count == 0) {
41250 |             break;
41251 |         }
41252 |
41253 |         ma_sleep(10);
41254 |     }
41255 |
41256 |     if (pDevice->type == ma_device_type_capture) {
41257 |         pDevice->opensl.isDrainingCapture = MA_FALSE;
41258 |     } else {
41259 |         pDevice->opensl.isDrainingPlayback = MA_FALSE;
41260 |     }
41261 |
41262 |     return MA_SUCCESS;
41263 | }
41264 |
41265 | static ma_result ma_device_stop_opensl(ma_device* pDevice)
41266 | {
41267 |     SLresult resultSL;
41268 |
41269 |     MA_ASSERT(pDevice != NULL);
41270 |
41271 |     MA_ASSERT(g_maOpenSLInitCounter > 0); /* <-- If you trigger this it means you've either not
> initialized the context, or you've uninitialized it before stopping/uninitializing the device. */
41272 |     if (g_maOpenSLInitCounter == 0) {
41273 |         return MA_INVALID_OPERATION;
41274 |     }
41275 |
41276 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex) {
41277 |         ma_device_drain_opensl(pDevice, ma_device_type_capture);
41278 |
41279 |         resultSL = MA_OPENSL_RECORD(pDevice->opensl.pAudioRecorder)->SetRecordState((SLRecordItf)pDe
> vice->opensl.pAudioRecorder, SL_RECORDSTATE_STOPPED);
41280 |         if (resultSL != SL_RESULT_SUCCESS) {
41281 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to stop
> internal capture device.");
41282 |             return ma_result_from_OpenSL(resultSL);
41283 |         }
41284 |
41285 |         MA_OPENSL_BUFFERQUEUE(pDevice->opensl.pBufferQueueCapture)->Clear((SLAndroidSimpleBufferQueu
> eItf)pDevice->opensl.pBufferQueueCapture);
41286 |     }
41287 |
41288 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
41289 |         ma_device_drain_opensl(pDevice, ma_device_type_playback);
41290 |
41291 |         resultSL =
> MA_OPENSL_PLAY(pDevice->opensl.pAudioPlayer)->SetPlayState((SLPlayItf)pDevice->opensl.pAudioPlayer,
> SL_PLAYSTATE_STOPPED);
41292 |         if (resultSL != SL_RESULT_SUCCESS) {
41293 |             ma_log_post(ma_device_get_log(pDevice), MA_LOG_LEVEL_ERROR, "[OpenSL] Failed to stop
> internal playback device.");
41294 |             return ma_result_from_OpenSL(resultSL);

```

```

41295 |     }
41296 |
41297 |     MA_OPENS�_BUFFERQUEUE(pDevice->opensl.pBufferQueuePlayback)->Clear((SLAndroidSimpleBufferQue
> ueIf)pDevice->opensl.pBufferQueuePlayback);
41298 |     }
41299 |
41300 |     /* Make sure the client is aware that the device has stopped. There may be an OpenSL|ES callback
> for this, but I haven't found it. */
41301 |     ma_device_on_notification_stopped(pDevice);
41302 |
41303 |     return MA_SUCCESS;
41304 | }
41305 |
41306 |
41307 | static ma_result ma_context_uninit__opensl(ma_context* pContext)
41308 | {
41309 |     MA_ASSERT(pContext != NULL);
41310 |     MA_ASSERT(pContext->backend == ma_backend_opensl);
41311 |     (void)pContext;
41312 |
41313 |     /* Uninit global data. */
41314 |     ma_spinlock_lock(&g_maOpenSLSpinlock);
41315 |     {
41316 |         MA_ASSERT(g_maOpenSLInitCounter > 0); /* If you've triggered this, it means you have
> ma_context_init/uninit mismatch. Each successful call to ma_context_init() must be matched up with a
> call to ma_context_uninit(). */
41317 |
41318 |         g_maOpenSLInitCounter -= 1;
41319 |         if (g_maOpenSLInitCounter == 0) {
41320 |             (*g_maEngineObjectSL)->Destroy(g_maEngineObjectSL);
41321 |         }
41322 |     }
41323 |     ma_spinlock_unlock(&g_maOpenSLSpinlock);
41324 |
41325 |     return MA_SUCCESS;
41326 | }
41327 |
41328 | static ma_result ma_dlsym_SLInterfaceID__opensl(ma_context* pContext, const char* pName, ma_handle*
> pHandle)
41329 | {
41330 |     /* We need to return an error if the symbol cannot be found. This is important because there
> have been reports that some symbols do not exist. */
41331 |     ma_handle* p = (ma_handle*)ma_dlsym(ma_context_get_log(pContext), pContext->opensl.libOpenSLES,
> pName);
41332 |     if (p == NULL) {
41333 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_INFO, "[OpenSL] Cannot find symbol
> %s", pName);
41334 |         return MA_NO_BACKEND;
41335 |     }
41336 |
41337 |     *pHandle = *p;
41338 |     return MA_SUCCESS;
41339 | }
41340 |
41341 | static ma_result ma_context_init_engine_nolock__opensl(ma_context* pContext)
41342 | {
41343 |     g_maOpenSLInitCounter += 1;
41344 |     if (g_maOpenSLInitCounter == 1) {
41345 |         SLresult resultSL;
41346 |
41347 |         resultSL = ((ma_slCreateEngine_proc)pContext->opensl.slCreateEngine)(&g_maEngineObjectSL, 0,
> NULL, 0, NULL, NULL);
41348 |         if (resultSL != SL_RESULT_SUCCESS) {
41349 |             g_maOpenSLInitCounter -= 1;
41350 |             return ma_result_from_OpenSL(resultSL);
41351 |         }
41352 |
41353 |         (*g_maEngineObjectSL)->Realize(g_maEngineObjectSL, SL_BOOLEAN_FALSE);
41354 |
41355 |         resultSL = (*g_maEngineObjectSL)->GetInterface(g_maEngineObjectSL,
> (SLInterfaceID)pContext->opensl.SL_IID_ENGINE, &g_maEngineSL);
41356 |         if (resultSL != SL_RESULT_SUCCESS) {
41357 |             (*g_maEngineObjectSL)->Destroy(g_maEngineObjectSL);
41358 |             g_maOpenSLInitCounter -= 1;
41359 |             return ma_result_from_OpenSL(resultSL);
41360 |         }
41361 |     }
41362 |
41363 |     return MA_SUCCESS;

```

```

41364 | }
41365 |
41366 | static ma_result ma_context_init_opensl(ma_context* pContext, const ma_context_config* pConfig,
> ma_backend_callbacks* pCallbacks)
41367 | {
41368 |     ma_result result;
41369 |
41370 |     #if !defined(MA_NO_RUNTIME_LINKING)
41371 |         size_t i;
41372 |         const char* libOpenSLESNames[] = {
41373 |             "libOpenSLES.so"
41374 |         };
41375 |     #endif
41376 |
41377 |     MA_ASSERT(pContext != NULL);
41378 |
41379 |     (void)pConfig;
41380 |
41381 |     #if !defined(MA_NO_RUNTIME_LINKING)
41382 |         /*
41383 |         Dynamically link against libOpenSLES.so. I have now had multiple reports that
> SL_IID_ANDROIDSIMPLEBUFFERQUEUE cannot be found. One
41384 |         report was happening at compile time and another at runtime. To try working around this, I'm
> going to link to libOpenSLES at runtime
41385 |         and extract the symbols rather than reference them directly. This should, hopefully, fix these
> issues as the compiler won't see any
41386 |         references to the symbols and will hopefully skip the checks.
41387 |         */
41388 |         for (i = 0; i < ma_countof(libOpenSLESNames); i += 1) {
41389 |             pContext->opensl.libOpenSLES = ma_dlopen(ma_context_get_log(pContext), libOpenSLESNames[i]);
41390 |             if (pContext->opensl.libOpenSLES != NULL) {
41391 |                 break;
41392 |             }
41393 |         }
41394 |
41395 |         if (pContext->opensl.libOpenSLES == NULL) {
41396 |             ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_INFO, "[OpenSL] Could not find
> libOpenSLES.so");
41397 |             return MA_NO_BACKEND;
41398 |         }
41399 |
41400 |         result = ma_dlsym_SLInterfaceID__opensl(pContext, "SL_IID_ENGINE",
> &pContext->opensl.SL_IID_ENGINE);
41401 |         if (result != MA_SUCCESS) {
41402 |             ma_dlclosure(ma_context_get_log(pContext), pContext->opensl.libOpenSLES);
41403 |             return result;
41404 |         }
41405 |
41406 |         result = ma_dlsym_SLInterfaceID__opensl(pContext, "SL_IID_AUDIOIODEVICECAPABILITIES",
> &pContext->opensl.SL_IID_AUDIOIODEVICECAPABILITIES);
41407 |         if (result != MA_SUCCESS) {
41408 |             ma_dlclosure(ma_context_get_log(pContext), pContext->opensl.libOpenSLES);
41409 |             return result;
41410 |         }
41411 |
41412 |         result = ma_dlsym_SLInterfaceID__opensl(pContext, "SL_IID_ANDROIDSIMPLEBUFFERQUEUE",
> &pContext->opensl.SL_IID_ANDROIDSIMPLEBUFFERQUEUE);
41413 |         if (result != MA_SUCCESS) {
41414 |             ma_dlclosure(ma_context_get_log(pContext), pContext->opensl.libOpenSLES);
41415 |             return result;
41416 |         }
41417 |
41418 |         result = ma_dlsym_SLInterfaceID__opensl(pContext, "SL_IID_RECORD",
> &pContext->opensl.SL_IID_RECORD);
41419 |         if (result != MA_SUCCESS) {
41420 |             ma_dlclosure(ma_context_get_log(pContext), pContext->opensl.libOpenSLES);
41421 |             return result;
41422 |         }
41423 |
41424 |         result = ma_dlsym_SLInterfaceID__opensl(pContext, "SL_IID_PLAY", &pContext->opensl.SL_IID_PLAY);
41425 |         if (result != MA_SUCCESS) {
41426 |             ma_dlclosure(ma_context_get_log(pContext), pContext->opensl.libOpenSLES);
41427 |             return result;
41428 |         }
41429 |
41430 |         result = ma_dlsym_SLInterfaceID__opensl(pContext, "SL_IID_OUTPUTMIX",
> &pContext->opensl.SL_IID_OUTPUTMIX);
41431 |         if (result != MA_SUCCESS) {
41432 |             ma_dlclosure(ma_context_get_log(pContext), pContext->opensl.libOpenSLES);

```

```

41433 |         return result;
41434 |     }
41435 |
41436 |     result = ma_dlsym_SLInterfaceID__opengl(pContext, "SL_IID_ANDROIDCONFIGURATION",
> &pContext->opengl.SL_IID_ANDROIDCONFIGURATION);
41437 |     if (result != MA_SUCCESS) {
41438 |         ma_dlclose(ma_context_get_log(pContext), pContext->opengl.libOpenSLES);
41439 |         return result;
41440 |     }
41441 |
41442 |     pContext->opengl.slCreateEngine = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
> pContext->opengl.libOpenSLES, "slCreateEngine");
41443 |     if (pContext->opengl.slCreateEngine == NULL) {
41444 |         ma_dlclose(ma_context_get_log(pContext), pContext->opengl.libOpenSLES);
41445 |         ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_INFO, "[OpenGL] Cannot find symbol
> slCreateEngine.");
41446 |         return MA_NO_BACKEND;
41447 |     }
41448 | #else
41449 |     pContext->opengl.SL_IID_ENGINE = (ma_handle)SL_IID_ENGINE;
41450 |     pContext->opengl.SL_IID_AUDIOIODEVICECAPABILITIES = (ma_handle)SL_IID_AUDIOIODEVICECAPABILITIES;
41451 |     pContext->opengl.SL_IID_ANDROIDSIMPLEBUFFERQUEUE = (ma_handle)SL_IID_ANDROIDSIMPLEBUFFERQUEUE;
41452 |     pContext->opengl.SL_IID_RECORD = (ma_handle)SL_IID_RECORD;
41453 |     pContext->opengl.SL_IID_PLAY = (ma_handle)SL_IID_PLAY;
41454 |     pContext->opengl.SL_IID_OUTPUTMIX = (ma_handle)SL_IID_OUTPUTMIX;
41455 |     pContext->opengl.SL_IID_ANDROIDCONFIGURATION = (ma_handle)SL_IID_ANDROIDCONFIGURATION;
41456 |     pContext->opengl.slCreateEngine = (ma_proc)slCreateEngine;
41457 | #endif
41458 |
41459 |
41460 |     /* Initialize global data first if applicable. */
41461 |     ma_spinlock_lock(&g_maOpenSLSpinlock);
41462 |     {
41463 |         result = ma_context_init_engine_nolock__opengl(pContext);
41464 |     }
41465 |     ma_spinlock_unlock(&g_maOpenSLSpinlock);
41466 |
41467 |     if (result != MA_SUCCESS) {
41468 |         ma_dlclose(ma_context_get_log(pContext), pContext->opengl.libOpenSLES);
41469 |         ma_log_post(ma_context_get_log(pContext), MA_LOG_LEVEL_INFO, "[OpenGL] Failed to initialize
> OpenGL engine.");
41470 |         return result;
41471 |     }
41472 |
41473 |     pCallbacks->onContextInit = ma_context_init__opengl;
41474 |     pCallbacks->onContextUninit = ma_context_uninit__opengl;
41475 |     pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__opengl;
41476 |     pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__opengl;
41477 |     pCallbacks->onDeviceInit = ma_device_init__opengl;
41478 |     pCallbacks->onDeviceUninit = ma_device_uninit__opengl;
41479 |     pCallbacks->onDeviceStart = ma_device_start__opengl;
41480 |     pCallbacks->onDeviceStop = ma_device_stop__opengl;
41481 |     pCallbacks->onDeviceRead = NULL; /* Not needed because OpenGL|ES is asynchronous.
> */
41482 |     pCallbacks->onDeviceWrite = NULL; /* Not needed because OpenGL|ES is asynchronous.
> */
41483 |     pCallbacks->onDeviceDataLoop = NULL; /* Not needed because OpenGL|ES is asynchronous.
> */
41484 |
41485 |     return MA_SUCCESS;
41486 | }
41487 | #endif /* OpenGL|ES */
41488 |
41489 |
41490 | /*****
41491 |
41492 | Web Audio Backend
41493 |
41494 | *****/
41495 | #ifdef MA_HAS_WEBAUDIO
41496 | #include <emscripten/emscripten.h>
41497 |
41498 | #ifndef MA_EMSCRIPTEN_MAJOR
41499 |     #if defined(__EMSCRIPTEN_MAJOR__)
41500 |         #define MA_EMSCRIPTEN_MAJOR __EMSCRIPTEN_MAJOR__
41501 |     #else
41502 |         #define MA_EMSCRIPTEN_MAJOR __EMSCRIPTEN_major__
41503 |     #endif
41504 | #endif

```



```

41505 | #ifndef MA_EMSCRIPTEN_MINOR
41506 |     #if defined(__EMSCRIPTEN_MINOR__)
41507 |         #define MA_EMSCRIPTEN_MINOR __EMSCRIPTEN_MINOR__
41508 |     #else
41509 |         #define MA_EMSCRIPTEN_MINOR __EMSCRIPTEN_minor__
41510 |     #endif
41511 | #endif
41512 | #ifndef MA_EMSCRIPTEN_TINY
41513 |     #if defined(__EMSCRIPTEN_TINY__)
41514 |         #define MA_EMSCRIPTEN_TINY __EMSCRIPTEN_TINY__
41515 |     #else
41516 |         #define MA_EMSCRIPTEN_TINY __EMSCRIPTEN_tiny__
41517 |     #endif
41518 | #endif
41519 |
41520 | #if (MA_EMSCRIPTEN_MAJOR > 3) || (MA_EMSCRIPTEN_MAJOR == 3 && (MA_EMSCRIPTEN_MINOR > 1 ||
> (MA_EMSCRIPTEN_MINOR == 1 && MA_EMSCRIPTEN_TINY >= 32)))
41521 |     #include <emscripten/webaudio.h>
41522 |     #define MA_SUPPORT_AUDIO_WORKLETS
41523 |
41524 |     #if (MA_EMSCRIPTEN_MAJOR > 3) || (MA_EMSCRIPTEN_MAJOR == 3 && (MA_EMSCRIPTEN_MINOR > 1 ||
> (MA_EMSCRIPTEN_MINOR == 1 && MA_EMSCRIPTEN_TINY >= 70)))
41525 |         #define MA_SUPPORT_AUDIO_WORKLETS_VARIABLE_BUFFER_SIZE
41526 |     #endif
41527 | #endif
41528 |
41529 | #if defined(MA_ENABLE_AUDIO_WORKLETS) && defined(MA_SUPPORT_AUDIO_WORKLETS)
41530 |     #define MA_USE_AUDIO_WORKLETS
41531 | #endif
41532 |
41533 | /* The thread stack size must be a multiple of 16. */
41534 | #ifndef MA_AUDIO_WORKLETS_THREAD_STACK_SIZE
41535 | #define MA_AUDIO_WORKLETS_THREAD_STACK_SIZE    131072
41536 | #endif
41537 |
41538 | #if defined(MA_USE_AUDIO_WORKLETS)
41539 | #define MA_WEBAUDIO_LATENCY_HINT_BALANCED      "balanced"
41540 | #define MA_WEBAUDIO_LATENCY_HINT_INTERACTIVE  "interactive"
41541 | #define MA_WEBAUDIO_LATENCY_HINT_PLAYBACK     "playback"
41542 | #endif
41543 |
41544 | static ma_bool32 ma_is_capture_supported__webaudio()
41545 | {
41546 |     return EM_ASM_INT({
41547 |         return (navigator.mediaDevices !== undefined && navigator.mediaDevices.getUserMedia !==
> undefined);
41548 |     }, 0) != 0; /* Must pass in a dummy argument for C99 compatibility. */
41549 | }
41550 |
41551 | #ifdef __cplusplus
41552 | extern "C" {
41553 | #endif
41554 | void* EMSCRIPTEN_KEEPALIVE ma_malloc_emscripten(size_t sz, const ma_allocation_callbacks*
> pAllocationCallbacks)
41555 | {
41556 |     return ma_malloc(sz, pAllocationCallbacks);
41557 | }
41558 |
41559 | void EMSCRIPTEN_KEEPALIVE ma_free_emscripten(void* p, const ma_allocation_callbacks*
> pAllocationCallbacks)
41560 | {
41561 |     ma_free(p, pAllocationCallbacks);
41562 | }
41563 |
41564 | void EMSCRIPTEN_KEEPALIVE ma_device_process_pcm_frames_capture__webaudio(ma_device* pDevice, int
> frameCount, float* pFrames)
41565 | {
41566 |     ma_device_handle_backend_data_callback(pDevice, NULL, pFrames, (ma_uint32)frameCount);
41567 | }
41568 |
41569 | void EMSCRIPTEN_KEEPALIVE ma_device_process_pcm_frames_playback__webaudio(ma_device* pDevice, int
> frameCount, float* pFrames)
41570 | {
41571 |     ma_device_handle_backend_data_callback(pDevice, pFrames, NULL, (ma_uint32)frameCount);
41572 | }
41573 | #ifdef __cplusplus
41574 | }
41575 | #endif
41576 |

```

```

41577 | static ma_result ma_context_enumerate_devices__webaudio(ma_context* pContext,
> ma_enum_devices_callback_proc callback, void* pUserData)
41578 | {
41579 |     ma_bool32 cbResult = MA_TRUE;
41580 |
41581 |     MA_ASSERT(pContext != NULL);
41582 |     MA_ASSERT(callback != NULL);
41583 |
41584 |     /* Only supporting default devices for now. */
41585 |
41586 |     /* Playback. */
41587 |     if (cbResult) {
41588 |         ma_device_info deviceInfo;
41589 |         MA_ZERO_OBJECT(&deviceInfo);
41590 |         ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
41591 |         deviceInfo.isDefault = MA_TRUE; /* Only supporting default devices. */
41592 |         cbResult = callback(pContext, ma_device_type_playback, &deviceInfo, pUserData);
41593 |     }
41594 |
41595 |     /* Capture. */
41596 |     if (cbResult) {
41597 |         if (ma_is_capture_supported__webaudio()) {
41598 |             ma_device_info deviceInfo;
41599 |             MA_ZERO_OBJECT(&deviceInfo);
41600 |             ma_strncpy_s(deviceInfo.name, sizeof(deviceInfo.name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
41601 |             deviceInfo.isDefault = MA_TRUE; /* Only supporting default devices. */
41602 |             cbResult = callback(pContext, ma_device_type_capture, &deviceInfo, pUserData);
41603 |         }
41604 |     }
41605 |
41606 |     return MA_SUCCESS;
41607 | }
41608 |
41609 | static ma_result ma_context_get_device_info__webaudio(ma_context* pContext, ma_device_type
> deviceType, const ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
41610 | {
41611 |     MA_ASSERT(pContext != NULL);
41612 |
41613 |     if (deviceType == ma_device_type_capture && !ma_is_capture_supported__webaudio()) {
41614 |         return MA_NO_DEVICE;
41615 |     }
41616 |
41617 |     MA_ZERO_MEMORY(pDeviceInfo->id.webaudio, sizeof(pDeviceInfo->id.webaudio));
41618 |
41619 |     /* Only supporting default devices for now. */
41620 |     (void)pDeviceID;
41621 |     if (deviceType == ma_device_type_playback) {
41622 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_PLAYBACK_DEVICE_NAME,
> (size_t)-1);
41623 |     } else {
41624 |         ma_strncpy_s(pDeviceInfo->name, sizeof(pDeviceInfo->name), MA_DEFAULT_CAPTURE_DEVICE_NAME,
> (size_t)-1);
41625 |     }
41626 |
41627 |     /* Only supporting default devices. */
41628 |     pDeviceInfo->isDefault = MA_TRUE;
41629 |
41630 |     /* Web Audio can support any number of channels and sample rates. It only supports f32 formats,
> however. */
41631 |     pDeviceInfo->nativeDataFormats[0].flags = 0;
41632 |     pDeviceInfo->nativeDataFormats[0].format = ma_format_unknown;
41633 |     pDeviceInfo->nativeDataFormats[0].channels = 0; /* All channels are supported. */
41634 |     pDeviceInfo->nativeDataFormats[0].sampleRate = EM_ASM_INT({
41635 |         try {
41636 |             var temp = new (window.AudioContext || window.webkitAudioContext)();
41637 |             var sampleRate = temp.sampleRate;
41638 |             temp.close();
41639 |             return sampleRate;
41640 |         } catch(e) {
41641 |             return 0;
41642 |         }
41643 |     }, 0); /* Must pass in a dummy argument for C99 compatibility. */
41644 |
41645 |     if (pDeviceInfo->nativeDataFormats[0].sampleRate == 0) {
41646 |         return MA_NO_DEVICE;
41647 |     }
41648 | }

```

```

41649 |     pDeviceInfo->nativeDataFormatCount = 1;
41650 |
41651 |     return MA_SUCCESS;
41652 | }
41653 |
41654 | static ma_result ma_device_uninit__webaudio(ma_device* pDevice)
41655 | {
41656 |     MA_ASSERT(pDevice != NULL);
41657 |
41658 |     #if defined(MA_USE_AUDIO_WORKLETS)
41659 |     {
41660 |         EM_ASM({
41661 |             var device = window.miniaudio.get_device_by_index($0);
41662 |
41663 |             if (device.streamNode !== undefined) {
41664 |                 device.streamNode.disconnect();
41665 |                 device.streamNode = undefined;
41666 |             }
41667 |
41668 |             device.pDevice = undefined;
41669 |         }, pDevice->webaudio.deviceIndex);
41670 |
41671 |         emscripten_destroy_web_audio_node(pDevice->webaudio.audioWorklet);
41672 |         emscripten_destroy_audio_context(pDevice->webaudio.audioContext);
41673 |         ma_free(pDevice->webaudio.pStackBuffer, &pDevice->pContext->allocationCallbacks);
41674 |     }
41675 |     #else
41676 |     {
41677 |         EM_ASM({
41678 |             var device = window.miniaudio.get_device_by_index($0);
41679 |
41680 |             /* Make sure all nodes are disconnected and marked for collection. */
41681 |             if (device.scriptNode !== undefined) {
41682 |                 device.scriptNode.onaudioprocess = function(e) {}; /* We want to reset the callback
> to ensure it doesn't get called after AudioContext.close() has returned. Shouldn't happen since
> we're disconnecting, but just to be safe... */
41683 |                 device.scriptNode.disconnect();
41684 |                 device.scriptNode = undefined;
41685 |             }
41686 |
41687 |             if (device.streamNode !== undefined) {
41688 |                 device.streamNode.disconnect();
41689 |                 device.streamNode = undefined;
41690 |             }
41691 |
41692 |             /*
41693 |             > this, hence why we want
41694 |             to clear the callback before closing.
41695 |             */
41696 |             device.webaudio.close();
41697 |             device.webaudio = undefined;
41698 |             device.pDevice = undefined;
41699 |         }, pDevice->webaudio.deviceIndex);
41700 |     }
41701 |     #endif
41702 |
41703 |     /* Clean up the device on the JS side. */
41704 |     EM_ASM({
41705 |         window.miniaudio.untrack_device_by_index($0);
41706 |     }, pDevice->webaudio.deviceIndex);
41707 |
41708 |     ma_free(pDevice->webaudio.pIntermediaryBuffer, &pDevice->pContext->allocationCallbacks);
41709 |
41710 |     return MA_SUCCESS;
41711 | }
41712 |
41713 | #if !defined(MA_USE_AUDIO_WORKLETS)
41714 | static ma_uint32 ma_calculate_period_size_in_frames_from_descriptor__webaudio(const
> ma_device_descriptor* pDescriptor, ma_uint32 nativeSampleRate, ma_performance_profile
> performanceProfile)
41715 | {
41716 |     /*
41717 |     > There have been reports of the default buffer size being too small on some browsers. If we're
> using
41718 |     the default buffer size, we'll make sure the period size is bigger than our standard defaults.
41719 |     */
41720 |     ma_uint32 periodSizeInFrames;
41721 |

```

```

41722 |     if (nativeSampleRate == 0) {
41723 |         nativeSampleRate = MA_DEFAULT_SAMPLE_RATE;
41724 |     }
41725 |
41726 |     if (pDescriptor->periodSizeInFrames == 0) {
41727 |         if (pDescriptor->periodSizeInMilliseconds == 0) {
41728 |             if (performanceProfile == ma_performance_profile_low_latency) {
41729 |                 periodSizeInFrames = ma_calculate_buffer_size_in_frames_from_milliseconds(33,
> nativeSampleRate); /* 1 frame @ 30 FPS */
41730 |             } else {
41731 |                 periodSizeInFrames = ma_calculate_buffer_size_in_frames_from_milliseconds(333,
> nativeSampleRate);
41732 |             }
41733 |         } else {
41734 |             periodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_milliseconds(pDescriptor->periodSizeInMilliseconds,
> nativeSampleRate);
41735 |         }
41736 |     } else {
41737 |         periodSizeInFrames = pDescriptor->periodSizeInFrames;
41738 |     }
41739 |
41740 |     /* The size of the buffer must be a power of 2 and between 256 and 16384. */
41741 |     if (periodSizeInFrames < 256) {
41742 |         periodSizeInFrames = 256;
41743 |     } else if (periodSizeInFrames > 16384) {
41744 |         periodSizeInFrames = 16384;
41745 |     } else {
41746 |         periodSizeInFrames = ma_next_power_of_2(periodSizeInFrames);
41747 |     }
41748 |
41749 |     return periodSizeInFrames;
41750 | }
41751 | #endif
41752 |
41753 |
41754 | #if defined(MA_USE_AUDIO_WORKLETS)
41755 | typedef struct
41756 | {
41757 |     ma_device* pDevice;
41758 |     const ma_device_config* pConfig;
41759 |     ma_device_descriptor* pDescriptorPlayback;
41760 |     ma_device_descriptor* pDescriptorCapture;
41761 | } ma_audio_worklet_thread_initialized_data;
41762 |
41763 | static EM_BOOL ma_audio_worklet_process_callback_webaudio(int inputCount, const AudioSampleFrame*
> pInputs, int outputCount, AudioSampleFrame* pOutputs, int paramCount, const AudioParamFrame*
> pParams, void* pUserData)
41764 | {
41765 |     ma_device* pDevice = (ma_device*)pUserData;
41766 |     ma_uint32 frameCount;
41767 |
41768 |     (void)paramCount;
41769 |     (void)pParams;
41770 |
41771 |     /*
41772 |     The Emscripten documentation says that it'll always be 128 frames being passed in. Hard coding
> it like that feels
41773 |     like a very bad idea to me. Even if it's hard coded in the backend, the API and documentation
> should always refer
41774 |     to variables instead of a hard coded number. In any case, will follow along for the time being.
41775 |
41776 |     Unfortunately the audio data is not interleaved so we'll need to convert it before we give the
> data to miniaudio
41777 |     for further processing.
41778 |     */
41779 |     if (pDevice->type == ma_device_type_playback) {
41780 |         frameCount = pDevice->playback.internalPeriodSizeInFrames;
41781 |     } else {
41782 |         frameCount = pDevice->capture.internalPeriodSizeInFrames;
41783 |     }
41784 |
41785 |     /*
41786 |     If this is called by the device has not yet been started we need to return early, making sure we
> output silence to
41787 |     the output buffer.
41788 |     */
41789 |     if (ma_device_get_state(pDevice) != ma_device_state_started) {
41790 |         for (int i = 0; i < outputCount; i += 1) {

```

```

41791 |         MA_ZERO_MEMORY(pOutputs[i].data, pOutputs[i].numberOfChannels * frameCount *
> sizeof(float));
41792 |     }
41793 |
41794 |     return EM_TRUE;
41795 | }
41796 |
41797 | if (inputCount > 0) {
41798 |     /* Input data needs to be interleaved before we hand it to the client. */
41799 |     for (ma_uint32 iChannel = 0; iChannel < pDevice->capture.internalChannels; iChannel += 1) {
41800 |         for (ma_uint32 iFrame = 0; iFrame < frameCount; iFrame += 1) {
41801 |             pDevice->webaudio.pIntermediaryBuffer[iFrame*pDevice->capture.internalChannels +
> iChannel] = pInputs[0].data[frameCount*iChannel + iFrame];
41802 |         }
41803 |     }
41804 |
41805 |     ma_device_process_pcm_frames_capture__webaudio(pDevice, frameCount,
> pDevice->webaudio.pIntermediaryBuffer);
41806 | }
41807 |
41808 | if (outputCount > 0) {
41809 |     /* If it's a capture-only device, we'll need to output silence. */
41810 |     if (pDevice->type == ma_device_type_capture) {
41811 |         for (int i = 0; i < outputCount; i += 1) {
41812 |             MA_ZERO_MEMORY(pOutputs[i].data, pOutputs[i].numberOfChannels * frameCount *
> sizeof(float));
41813 |         }
41814 |     } else {
41815 |         ma_device_process_pcm_frames_playback__webaudio(pDevice, frameCount,
> pDevice->webaudio.pIntermediaryBuffer);
41816 |     }
41817 |     /* We've read the data from the client. Now we need to deinterleave the buffer and
> output to the output buffer. */
41818 |     for (ma_uint32 iChannel = 0; iChannel < pDevice->playback.internalChannels; iChannel +=
> 1) {
41819 |         for (ma_uint32 iFrame = 0; iFrame < frameCount; iFrame += 1) {
41820 |             pOutputs[0].data[frameCount*iChannel + iFrame] =
> pDevice->webaudio.pIntermediaryBuffer[iFrame*pDevice->playback.internalChannels + iChannel];
41821 |         }
41822 |     }
41823 |     /*
41824 |     Just above we output data to the first output buffer. Here we just make sure we're
> putting silence into any
41826 |     remaining output buffers.
41827 |     */
41828 |     for (int i = 1; i < outputCount; i += 1) { /* <-- Note that the counter starts at 1
> instead of 0. */
41829 |         MA_ZERO_MEMORY(pOutputs[i].data, pOutputs[i].numberOfChannels * frameCount *
> sizeof(float));
41830 |     }
41831 | }
41832 | }
41833 |
41834 | return EM_TRUE;
41835 | }
41836 |
41837 |
41838 | static void ma_audio_worklet_processor_created__webaudio(EMSCRIPTEN_WEBAUDIO_T audioContext, EM_BOOL
> success, void* pUserData)
41839 | {
41840 |     ma_audio_worklet_thread_initialized_data* pParameters =
> (ma_audio_worklet_thread_initialized_data*)pUserData;
41841 |     EmscriptenAudioWorkletNodeCreateOptions audioWorkletOptions;
41842 |     int channels = 0;
41843 |     size_t intermediaryBufferSizeInFrames;
41844 |     int sampleRate;
41845 |
41846 |     if (success == EM_FALSE) {
41847 |         pParameters->pDevice->webaudio.initResult = MA_ERROR;
41848 |         ma_free(pParameters, &pParameters->pDevice->pContext->allocationCallbacks);
41849 |         return;
41850 |     }
41851 |
41852 |     /* The next step is to initialize the audio worklet node. */
41853 |     MA_ZERO_OBJECT(&audioWorkletOptions);
41854 |
41855 |     /*
41856 |     The way channel counts work with Web Audio is confusing. As far as I can tell, there's no way to

```

```

> know the channel
41857 |     count from MediaStreamAudioSourceNode (what we use for capture)? The only way to have control is
> to configure an
41858 |     output channel count on the capture side. This is slightly confusing for capture mode because
> intuitively you
41859 |     wouldn't actually connect an output to an input-only node, but this is what we'll have to do in
> order to have
41860 |     proper control over the channel count. In the capture case, we'll have to output silence to its
> output node.
41861 |     */
41862 |     if (pParameters->pConfig->deviceType == ma_device_type_capture) {
41863 |         channels = (int)((pParameters->pDescriptorCapture->channels > 0) ?
> pParameters->pDescriptorCapture->channels : MA_DEFAULT_CHANNELS);
41864 |         audioWorkletOptions.numberOfInputs = 1;
41865 |     } else {
41866 |         channels = (int)((pParameters->pDescriptorPlayback->channels > 0) ?
> pParameters->pDescriptorPlayback->channels : MA_DEFAULT_CHANNELS);
41867 |
41868 |         if (pParameters->pConfig->deviceType == ma_device_type_duplex) {
41869 |             audioWorkletOptions.numberOfInputs = 1;
41870 |         } else {
41871 |             audioWorkletOptions.numberOfInputs = 0;
41872 |         }
41873 |     }
41874 |
41875 |     audioWorkletOptions.numberOfOutputs = 1;
41876 |     audioWorkletOptions.outputChannelCounts = &channels;
41877 |
41878 |
41879 |     /*
41880 |     Now that we know the channel count to use we can allocate the intermediary buffer. The
41881 |     intermediary buffer is used for interleaving and deinterleaving.
41882 |     */
41883 |     #if defined(MA_SUPPORT_AUDIO_WORKLETS_VARIABLE_BUFFER_SIZE)
41884 |     {
41885 |         intermediaryBufferSizeInFrames =
> (size_t)emscripten_audio_context_quantum_size(audioContext);
41886 |     }
41887 |     #else
41888 |     {
41889 |         intermediaryBufferSizeInFrames = 128;
41890 |     }
41891 |     #endif
41892 |
41893 |     pParameters->pDevice->webaudio.pIntermediaryBuffer =
> (float*)ma_malloc(intermediaryBufferSizeInFrames * (ma_uint32)channels * sizeof(float),
> &pParameters->pDevice->pContext->allocationCallbacks);
41894 |     if (pParameters->pDevice->webaudio.pIntermediaryBuffer == NULL) {
41895 |         pParameters->pDevice->webaudio.initResult = MA_OUT_OF_MEMORY;
41896 |         ma_free(pParameters, &pParameters->pDevice->pContext->allocationCallbacks);
41897 |         return;
41898 |     }
41899 |
41900 |     pParameters->pDevice->webaudio.audioWorklet =
> emscripten_create_wasm_audio_worklet_node(audioContext, "miniaudio", &audioWorkletOptions,
> &ma_audio_worklet_process_callback_webaudio, pParameters->pDevice);
41901 |
41902 |     /* With the audio worklet initialized we can now attach it to the graph. */
41903 |     if (pParameters->pConfig->deviceType == ma_device_type_capture ||
> pParameters->pConfig->deviceType == ma_device_type_duplex) {
41904 |         ma_result attachmentResult = (ma_result)EM_ASM_INT({
41905 |             var getUserMediaResult = 0;
41906 |             var audioWorklet = emscriptenGetAudioObject($0);
41907 |             var audioContext = emscriptenGetAudioObject($1);
41908 |
41909 |             navigator.mediaDevices.getUserMedia({audio:true, video:false})
41910 |                 .then(function(stream) {
41911 |                     audioContext.streamNode = audioContext.createMediaStreamSource(stream);
41912 |                     audioContext.streamNode.connect(audioWorklet);
41913 |                     audioWorklet.connect(audioContext.destination);
41914 |                     getUserMediaResult = 0; /* 0 = MA_SUCCESS */
41915 |                 })
41916 |                 .catch(function(error) {
41917 |                     console.log("navigator.mediaDevices.getUserMedia Failed: " + error);
41918 |                     getUserMediaResult = -1; /* -1 = MA_ERROR */
41919 |                 });
41920 |
41921 |             return getUserMediaResult;
41922 |         }, pParameters->pDevice->webaudio.audioWorklet, audioContext);

```

```

41923 |
41924 |         if (attachmentResult != MA_SUCCESS) {
41925 |             ma_log_postf(ma_device_get_log(pParameters->pDevice), MA_LOG_LEVEL_ERROR, "Web Audio:
> Failed to connect capture node.");
41926 |             emscripten_destroy_web_audio_node(pParameters->pDevice->webaudio.audioWorklet);
41927 |             pParameters->pDevice->webaudio.initResult = attachmentResult;
41928 |             ma_free(pParameters, &pParameters->pDevice->pContext->allocationCallbacks);
41929 |             return;
41930 |         }
41931 |     }
41932 |
41933 |     /* If it's playback only we can now attach the worklet node to the graph. This has already been
> done for the duplex case. */
41934 |     if (pParameters->pConfig->deviceType == ma_device_type_playback) {
41935 |         ma_result attachmentResult = (ma_result)EM_ASM_INT({
41936 |             var audioWorklet = emscriptenGetAudioObject($0);
41937 |             var audioContext = emscriptenGetAudioObject($1);
41938 |             audioWorklet.connect(audioContext.destination);
41939 |             return 0; /* 0 = MA_SUCCESS */
41940 |         }, pParameters->pDevice->webaudio.audioWorklet, audioContext);
41941 |
41942 |         if (attachmentResult != MA_SUCCESS) {
41943 |             ma_log_postf(ma_device_get_log(pParameters->pDevice), MA_LOG_LEVEL_ERROR, "Web Audio:
> Failed to connect playback node.");
41944 |             pParameters->pDevice->webaudio.initResult = attachmentResult;
41945 |             ma_free(pParameters, &pParameters->pDevice->pContext->allocationCallbacks);
41946 |             return;
41947 |         }
41948 |     }
41949 |
41950 |     /* We need to update the descriptors so that they reflect the internal data format. Both capture
> and playback should be the same. */
41951 |     sampleRate = EM_ASM_INT({ return emscriptenGetAudioObject($0).sampleRate; }, audioContext);
41952 |
41953 |     if (pParameters->pDescriptorCapture != NULL) {
41954 |         pParameters->pDescriptorCapture->format = ma_format_f32;
41955 |         pParameters->pDescriptorCapture->channels = (ma_uint32)channels;
41956 |         pParameters->pDescriptorCapture->sampleRate = (ma_uint32)sampleRate;
41957 |         ma_channel_map_init_standard(ma_standard_channel_map_webaudio,
> pParameters->pDescriptorCapture->channelMap,
> ma_countof(pParameters->pDescriptorCapture->channelMap), pParameters->pDescriptorCapture->channels);
41958 |         pParameters->pDescriptorCapture->periodSizeInFrames = intermediaryBufferSizeInFrames;
41959 |         pParameters->pDescriptorCapture->periodCount = 1;
41960 |     }
41961 |
41962 |     if (pParameters->pDescriptorPlayback != NULL) {
41963 |         pParameters->pDescriptorPlayback->format = ma_format_f32;
41964 |         pParameters->pDescriptorPlayback->channels = (ma_uint32)channels;
41965 |         pParameters->pDescriptorPlayback->sampleRate = (ma_uint32)sampleRate;
41966 |         ma_channel_map_init_standard(ma_standard_channel_map_webaudio,
> pParameters->pDescriptorPlayback->channelMap,
> ma_countof(pParameters->pDescriptorPlayback->channelMap),
> pParameters->pDescriptorPlayback->channels);
41967 |         pParameters->pDescriptorPlayback->periodSizeInFrames = intermediaryBufferSizeInFrames;
41968 |         pParameters->pDescriptorPlayback->periodCount = 1;
41969 |     }
41970 |
41971 |     /* At this point we're done and we can return. */
41972 |     ma_log_postf(ma_device_get_log(pParameters->pDevice), MA_LOG_LEVEL_DEBUG, "AudioWorklets:
> Created worklet node: %d\n", pParameters->pDevice->webaudio.audioWorklet);
41973 |     pParameters->pDevice->webaudio.initResult = MA_SUCCESS;
41974 |     ma_free(pParameters, &pParameters->pDevice->pContext->allocationCallbacks);
41975 | }
41976 |
41977 | static void ma_audio_worklet_thread_initialized__webaudio(EMSCRIPTEN_WEBAUDIO_T audioContext,
> EM_BOOL success, void* pUserData)
41978 | {
41979 |     ma_audio_worklet_thread_initialized_data* pParameters =
> (ma_audio_worklet_thread_initialized_data*)pUserData;
41980 |     WebAudioWorkletProcessorCreateOptions workletProcessorOptions;
41981 |
41982 |     MA_ASSERT(pParameters != NULL);
41983 |
41984 |     if (success == EM_FALSE) {
41985 |         pParameters->pDevice->webaudio.initResult = MA_ERROR;
41986 |         return;
41987 |     }
41988 |
41989 |     MA_ZERO_OBJECT(&workletProcessorOptions);

```



```

41990 |     workletProcessorOptions.name = "miniaudio"; /* I'm not entirely sure what to call this. Does
> this need to be globally unique, or does it need only be unique for a given AudioContext? */
41991 |
41992 |     emscripten_create_wasm_audio_worklet_processor_async(audioContext, &workletProcessorOptions,
> ma_audio_worklet_processor_created__webaudio, pParameters);
41993 | }
41994 | #endif
41995 |
41996 | static ma_result ma_device_init__webaudio(ma_device* pDevice, const ma_device_config* pConfig,
> ma_device_descriptor* pDescriptorPlayback, ma_device_descriptor* pDescriptorCapture)
41997 | {
41998 |     if (pConfig->deviceType == ma_device_type_loopback) {
41999 |         return MA_DEVICE_TYPE_NOT_SUPPORTED;
42000 |     }
42001 |
42002 |     /* No exclusive mode with Web Audio. */
42003 |     if (((pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorPlayback->shareMode == ma_share_mode_exclusive) ||
42004 |         ((pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) && pDescriptorCapture->shareMode == ma_share_mode_exclusive)) {
42005 |         return MA_SHARE_MODE_NOT_SUPPORTED;
42006 |     }
42007 |
42008 |     /*
42009 |     With AudioWorklets we'll have just a single AudioContext. I'm not sure why I'm not doing this
> for ScriptProcessorNode so
42010 |     it might be worthwhile to look into that as well.
42011 |     */
42012 |     #if defined(MA_USE_AUDIO_WORKLETS)
42013 |     {
42014 |         EmscriptenWebAudioCreateAttributes audioContextAttributes;
42015 |         ma_audio_worklet_thread_initialized_data* pInitParameters;
42016 |         void* pStackBuffer;
42017 |
42018 |         if (pConfig->performanceProfile == ma_performance_profile_conservative) {
42019 |             audioContextAttributes.latencyHint = MA_WEBAUDIO_LATENCY_HINT_PLAYBACK;
42020 |         } else {
42021 |             audioContextAttributes.latencyHint = MA_WEBAUDIO_LATENCY_HINT_INTERACTIVE;
42022 |         }
42023 |
42024 |         /*
42025 |         In my testing, Firefox does not seem to capture audio data properly if the sample rate is
> set
42026 |         to anything other than 48K. This does not seem to be the case for other browsers. For this
> reason,
42027 |         if the device type is anything other than playback, we'll leave the sample rate as-is and
> let the
42028 |         browser pick the appropriate rate for us.
42029 |         */
42030 |         if (pConfig->deviceType == ma_device_type_playback) {
42031 |             audioContextAttributes.sampleRate = pDescriptorPlayback->sampleRate;
42032 |         } else {
42033 |             audioContextAttributes.sampleRate = 0;
42034 |         }
42035 |
42036 |         /* It's not clear if this can return an error. None of the tests in the Emscripten
> repository check for this, so neither am I for now. */
42037 |         pDevice->webaudio.audioContext = emscripten_create_audio_context(&audioContextAttributes);
42038 |
42039 |         /*
42040 |         With the context created we can now create the worklet. We can only have a single worklet
> per audio
42041 |         context which means we'll need to craft this appropriately to handle duplex devices
> correctly.
42042 |         */
42043 |
42044 |         /*
42045 |         We now need to create a worker thread. This is a bit weird because we need to allocate our
42046 |         own buffer for the thread's stack. The stack needs to be aligned to 16 bytes. I'm going to
42047 |         allocate this on the heap to keep it simple.
42048 |         */
42049 |         pStackBuffer = ma_aligned_malloc(MA_AUDIO_WORKLETS_THREAD_STACK_SIZE, 16,
> &pDevice->pContext->allocationCallbacks);
42050 |         if (pStackBuffer == NULL) {
42051 |             emscripten_destroy_audio_context(pDevice->webaudio.audioContext);
42052 |             return MA_OUT_OF_MEMORY;
42053 |         }
42054 |
42055 |         /* Our thread initialization parameters need to be allocated on the heap so they don't go

```

```

> out of scope. */
42056 |         pInitParameters =
> (ma_audio_worklet_thread_initialized_data*)ma_malloc(sizeof(*pInitParameters),
> &pDevice->pContext->allocationCallbacks);
42057 |         if (pInitParameters == NULL) {
42058 |             ma_free(pStackBuffer, &pDevice->pContext->allocationCallbacks);
42059 |             emscripten_destroy_audio_context(pDevice->webaudio.audioContext);
42060 |             return MA_OUT_OF_MEMORY;
42061 |         }
42062 |
42063 |         pInitParameters->pDevice = pDevice;
42064 |         pInitParameters->pConfig = pConfig;
42065 |         pInitParameters->pDescriptorPlayback = pDescriptorPlayback;
42066 |         pInitParameters->pDescriptorCapture = pDescriptorCapture;
42067 |
42068 |         /*
42069 |         We need to flag the device as not yet initialized so we can wait on it later. Unfortunately
> all of
42070 |         the Emscripten WebAudio stuff is asynchronous.
42071 |         */
42072 |         pDevice->webaudio.initResult = MA_BUSY;
42073 |         {
42074 |             emscripten_start_wasm_audio_worklet_thread_async(pDevice->webaudio.audioContext,
> pStackBuffer, MA_AUDIO_WORKLETS_THREAD_STACK_SIZE, ma_audio_worklet_thread_initialized__webaudio,
> pInitParameters);
42075 |         }
42076 |         while (pDevice->webaudio.initResult == MA_BUSY) { emscripten_sleep(1); } /* We must wait
> for initialization to complete. We're just spinning here. The emscripten_sleep() call is why we need
> to build with '-sASYNCIFY'. */
42077 |
42078 |         /* Initialization is now complete. Descriptors were updated when the worklet was
> initialized. */
42079 |         if (pDevice->webaudio.initResult != MA_SUCCESS) {
42080 |             ma_free(pStackBuffer, &pDevice->pContext->allocationCallbacks);
42081 |             emscripten_destroy_audio_context(pDevice->webaudio.audioContext);
42082 |             return pDevice->webaudio.initResult;
42083 |         }
42084 |
42085 |         /* We need to add an entry to the miniaudio.devices list on the JS side so we can do some
> JS/C interop. */
42086 |         pDevice->webaudio.deviceIndex = EM_ASM_INT({
42087 |             return window.miniaudio.track_device({
42088 |                 webaudio: emscriptenGetAudioObject($0),
42089 |                 state: 1, /* 1 = ma_device_state_stopped */
42090 |                 pDevice: $1
42091 |             });
42092 |         }, pDevice->webaudio.audioContext, pDevice);
42093 |
42094 |         return MA_SUCCESS;
42095 |     }
42096 |     #else
42097 |     {
42098 |         /* ScriptProcessorNode. This path requires us to do almost everything in JS, but we'll do as
> much as we can in C. */
42099 |         ma_uint32 deviceIndex;
42100 |         ma_uint32 channels;
42101 |         ma_uint32 sampleRate;
42102 |         ma_uint32 periodSizeInFrames;
42103 |
42104 |         /* The channel count will depend on the device type. If it's a capture, use its, otherwise
> use the playback side. */
42105 |         if (pConfig->deviceType == ma_device_type_capture) {
42106 |             channels = (pDescriptorCapture->channels > 0) ? pDescriptorCapture->channels :
> MA_DEFAULT_CHANNELS;
42107 |         } else {
42108 |             channels = (pDescriptorPlayback->channels > 0) ? pDescriptorPlayback->channels :
> MA_DEFAULT_CHANNELS;
42109 |         }
42110 |
42111 |         /*
42112 |         When testing in Firefox, I've seen it where capture mode fails if the sample rate is changed
> to anything other than it's
42113 |         native rate. For this reason we're leaving the sample rate untouched for capture devices.
42114 |         */
42115 |         if (pConfig->deviceType == ma_device_type_playback) {
42116 |             sampleRate = pDescriptorPlayback->sampleRate;
42117 |         } else {
42118 |             sampleRate = 0; /* Let the browser decide when capturing. */
42119 |         }

```

```

42120 |
42121 |         /* The period size needs to be a power of 2. */
42122 |         if (pConfig->deviceType == ma_device_type_capture) {
42123 |             periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__webaudio(pDescriptorCapture, sampleRate,
> pConfig->performanceProfile);
42124 |         } else {
42125 |             periodSizeInFrames =
> ma_calculate_period_size_in_frames_from_descriptor__webaudio(pDescriptorPlayback, sampleRate,
> pConfig->performanceProfile);
42126 |         }
42127 |
42128 |         /* We need an intermediary buffer for doing interleaving and deinterleaving. */
42129 |         pDevice->webaudio.pIntermediaryBuffer = (float*)ma_malloc(periodSizeInFrames * channels *
> sizeof(float), &pDevice->pContext->allocationCallbacks);
42130 |         if (pDevice->webaudio.pIntermediaryBuffer == NULL) {
42131 |             return MA_OUT_OF_MEMORY;
42132 |         }
42133 |
42134 |         deviceIndex = EM_ASM_INT({
42135 |             var deviceType = $0;
42136 |             var channels = $1;
42137 |             var sampleRate = $2;
42138 |             var bufferSize = $3;
42139 |             var pIntermediaryBuffer = $4;
42140 |             var pDevice = $5;
42141 |
42142 |             if (typeof(window.miniaudio) === 'undefined') {
42143 |                 return -1; /* Context not initialized. */
42144 |             }
42145 |
42146 |             var device = {};
42147 |
42148 |             /* First thing we need is an AudioContext. */
42149 |             var audioContextOptions = {};
42150 |             if (deviceType == window.miniaudio.device_type.playback && sampleRate != 0) {
42151 |                 audioContextOptions.sampleRate = sampleRate;
42152 |             }
42153 |
42154 |             device.webaudio = new (window.AudioContext ||
> window.webkitAudioContext)(audioContextOptions);
42155 |             device.webaudio.suspend(); /* The AudioContext must be created in a suspended state. */
42156 |             device.state = window.miniaudio.device_state.stopped;
42157 |
42158 |             /*
42159 |             We need to create a ScriptProcessorNode. The channel situation is the same as the
> AudioWorklet path in that we
42160 |             need to specify an output and configure the channel count there.
42161 |             */
42162 |             var channelCountIn = 0;
42163 |             var channelCountOut = channels;
42164 |             if (deviceType != window.miniaudio.device_type.playback) {
42165 |                 channelCountIn = channels;
42166 |             }
42167 |
42168 |             device.scriptNode = device.webaudio.createScriptProcessor(bufferSize, channelCountIn,
> channelCountOut);
42169 |
42170 |             /* The node processing callback. */
42171 |             device.scriptNode.onaudioprocess = function(e) {
42172 |                 if (device.intermediaryBufferView == null || device.intermediaryBufferView.length ==
> 0) {
42173 |                     device.intermediaryBufferView = new Float32Array(HEAPF32.buffer,
> pIntermediaryBuffer, bufferSize * channels);
42174 |                 }
42175 |
42176 |                 /* Do the capture side first. */
42177 |                 if (deviceType == window.miniaudio.device_type.capture || deviceType ==
> window.miniaudio.device_type.duplex) {
42178 |                     /* The data must be interleaved before being processed miniaudio. */
42179 |                     for (var iChannel = 0; iChannel < channels; iChannel += 1) {
42180 |                         var inputBuffer = e.inputBuffer.getChannelData(iChannel);
42181 |                         var intermediaryBuffer = device.intermediaryBufferView;
42182 |
42183 |                         for (var iFrame = 0; iFrame < bufferSize; iFrame += 1) {
42184 |                             intermediaryBuffer[iFrame*channels + iChannel] = inputBuffer[iFrame];
42185 |                         }
42186 |                     }
42187 |                 }

```

```

42188 |         _ma_device_process_pcm_frames_capture__webaudio(pDevice, bufferSize,
> pIntermediaryBuffer);
42189 |     }
42190 |
42191 |     if (deviceType == window.miniaudio.device_type.playback || deviceType ==
> window.miniaudio.device_type.duplex) {
42192 |         _ma_device_process_pcm_frames_playback__webaudio(pDevice, bufferSize,
> pIntermediaryBuffer);
42193 |
42194 |         for (var iChannel = 0; iChannel < e.outputBuffer.numberOfChannels; ++iChannel) {
42195 |             var outputBuffer = e.outputBuffer.getChannelData(iChannel);
42196 |             var intermediaryBuffer = device.intermediaryBufferView;
42197 |
42198 |             for (var iFrame = 0; iFrame < bufferSize; iFrame += 1) {
42199 |                 outputBuffer[iFrame] = intermediaryBuffer[iFrame*channels + iChannel];
42200 |             }
42201 |         }
42202 |     } else {
42203 |         /* It's a capture-only device. Make sure the output is silenced. */
42204 |         for (var iChannel = 0; iChannel < e.outputBuffer.numberOfChannels; ++iChannel) {
42205 |             e.outputBuffer.getChannelData(iChannel).fill(0.0);
42206 |         }
42207 |     }
42208 | };
42209 |
42210 |     /* Now we need to connect our node to the graph. */
42211 |     if (deviceType == window.miniaudio.device_type.capture || deviceType ==
> window.miniaudio.device_type.duplex) {
42212 |         navigator.mediaDevices.getUserMedia({audio:true, video:false})
42213 |         .then(function(stream) {
42214 |             device.streamNode = device.webaudio.createMediaStreamSource(stream);
42215 |             device.streamNode.connect(device.scriptNode);
42216 |             device.scriptNode.connect(device.webaudio.destination);
42217 |         })
42218 |         .catch(function(error) {
42219 |             console.log("Failed to get user media: " + error);
42220 |         });
42221 |     }
42222 |
42223 |     if (deviceType == window.miniaudio.device_type.playback) {
42224 |         device.scriptNode.connect(device.webaudio.destination);
42225 |     }
42226 |
42227 |     device.pDevice = pDevice;
42228 |
42229 |     return window.miniaudio.track_device(device);
42230 | }, pConfig->deviceType, channels, sampleRate, periodSizeInFrames,
> pDevice->webaudio.pIntermediaryBuffer, pDevice);
42231 |
42232 |     if (deviceIndex < 0) {
42233 |         return MA_FAILED_TO_OPEN_BACKEND_DEVICE;
42234 |     }
42235 |
42236 |     pDevice->webaudio.deviceIndex = deviceIndex;
42237 |
42238 |     /* Grab the sample rate from the audio context directly. */
42239 |     sampleRate = (ma_uint32)EM_ASM_INT({ return
> window.miniaudio.get_device_by_index($0).webaudio.sampleRate; }, deviceIndex);
42240 |
42241 |     if (pDescriptorCapture != NULL) {
42242 |         pDescriptorCapture->format = ma_format_f32;
42243 |         pDescriptorCapture->channels = channels;
42244 |         pDescriptorCapture->sampleRate = sampleRate;
42245 |         ma_channel_map_init_standard(ma_standard_channel_map_webaudio,
> pDescriptorCapture->channelMap, ma_countof(pDescriptorCapture->channelMap),
> pDescriptorCapture->channels);
42246 |         pDescriptorCapture->periodSizeInFrames = periodSizeInFrames;
42247 |         pDescriptorCapture->periodCount = 1;
42248 |     }
42249 |
42250 |     if (pDescriptorPlayback != NULL) {
42251 |         pDescriptorPlayback->format = ma_format_f32;
42252 |         pDescriptorPlayback->channels = channels;
42253 |         pDescriptorPlayback->sampleRate = sampleRate;
42254 |         ma_channel_map_init_standard(ma_standard_channel_map_webaudio,
> pDescriptorPlayback->channelMap, ma_countof(pDescriptorPlayback->channelMap),
> pDescriptorPlayback->channels);
42255 |         pDescriptorPlayback->periodSizeInFrames = periodSizeInFrames;
42256 |         pDescriptorPlayback->periodCount = 1;

```

```

42257 |     }
42258 |
42259 |     return MA_SUCCESS;
42260 | }
42261 | #endif
42262 | }
42263 |
42264 | static ma_result ma_device_start__webaudio(ma_device* pDevice)
42265 | {
42266 |     MA_ASSERT(pDevice != NULL);
42267 |
42268 |     EM_ASM({
42269 |         var device = window.miniaudio.get_device_by_index($0);
42270 |         device.webaudio.resume();
42271 |         device.state = window.miniaudio.device_state.started;
42272 |     }, pDevice->webaudio.deviceIndex);
42273 |
42274 |     return MA_SUCCESS;
42275 | }
42276 |
42277 | static ma_result ma_device_stop__webaudio(ma_device* pDevice)
42278 | {
42279 |     MA_ASSERT(pDevice != NULL);
42280 |
42281 |     /*
42282 |     From the WebAudio API documentation for AudioContext.suspend():
42283 |
42284 |     Suspends the progression of AudioContext's currentTime, allows any current context
42285 | > processing blocks that are already processed to be played to the
42286 |     destination, and then allows the system to release its claim on audio hardware.
42287 |
42288 |     I read this to mean that "any current context processing blocks" are processed by suspend() -
42289 | > i.e. They they are drained. We therefore shouldn't need to
42290 |     do any kind of explicit draining.
42291 |     */
42292 |     EM_ASM({
42293 |         var device = window.miniaudio.get_device_by_index($0);
42294 |         device.webaudio.suspend();
42295 |         device.state = window.miniaudio.device_state.stopped;
42296 |     }, pDevice->webaudio.deviceIndex);
42297 |
42298 |     ma_device__on_notification_stopped(pDevice);
42299 |
42300 |     return MA_SUCCESS;
42301 | }
42302 |
42303 | static ma_result ma_context_uninit__webaudio(ma_context* pContext)
42304 | {
42305 |     MA_ASSERT(pContext != NULL);
42306 |     MA_ASSERT(pContext->backend == ma_backend_webaudio);
42307 |
42308 |     (void)pContext; /* Unused. */
42309 |
42310 |     /* Remove the global miniaudio object from window if there are no more references to it. */
42311 |     EM_ASM({
42312 |         if (typeof(window.miniaudio) !== 'undefined') {
42313 |             window.miniaudio.unlock_event_types.map(function(event_type) {
42314 |                 document.removeEventListener(event_type, window.miniaudio.unlock, true);
42315 |             });
42316 |
42317 |             window.miniaudio.referenceCount -= 1;
42318 |             if (window.miniaudio.referenceCount === 0) {
42319 |                 delete window.miniaudio;
42320 |             }
42321 |         }
42322 |     });
42323 |
42324 |     return MA_SUCCESS;
42325 | }
42326 |
42327 | static ma_result ma_context_init__webaudio(ma_context* pContext, const ma_context_config* pConfig,
42328 | > ma_backend_callbacks* pCallbacks)
42329 | {
42330 |     int resultFromJS;
42331 |
42332 |     MA_ASSERT(pContext != NULL);
42333 |
42334 |     (void)pConfig; /* Unused. */
42335 |

```

```

42333 | /* Here is where our global JavaScript object is initialized. */
42334 | resultFromJS = EM_ASM_INT({
42335 |     if (typeof window === 'undefined' || (window.AudioContext || window.webkitAudioContext) ===
> undefined) {
42336 |         return 0; /* Web Audio not supported. */
42337 |     }
42338 |
42339 |     if (typeof(window.miniaudio) === 'undefined') {
42340 |         window.miniaudio = {
42341 |             referenceCount: 0
42342 |         };
42343 |
42344 |         /* Device types. */
42345 |         window.miniaudio.device_type = {};
42346 |         window.miniaudio.device_type.playback = $0;
42347 |         window.miniaudio.device_type.capture = $1;
42348 |         window.miniaudio.device_type.duplex = $2;
42349 |
42350 |         /* Device states. */
42351 |         window.miniaudio.device_state = {};
42352 |         window.miniaudio.device_state.stopped = $3;
42353 |         window.miniaudio.device_state.started = $4;
42354 |
42355 |         /* Device cache for mapping devices to indexes for JavaScript/C interop. */
42356 |         let miniaudio = window.miniaudio;
42357 |         miniaudio.devices = [];
42358 |
42359 |         miniaudio.track_device = function(device) {
42360 |             /* Try inserting into a free slot first. */
42361 |             for (var iDevice = 0; iDevice < miniaudio.devices.length; ++iDevice) {
42362 |                 if (miniaudio.devices[iDevice] == null) {
42363 |                     miniaudio.devices[iDevice] = device;
42364 |                     return iDevice;
42365 |                 }
42366 |             }
42367 |
42368 |             /* Getting here means there is no empty slots in the array so we just push to the
> end. */
42369 |             miniaudio.devices.push(device);
42370 |             return miniaudio.devices.length - 1;
42371 |         };
42372 |
42373 |         miniaudio.untrack_device_by_index = function(deviceIndex) {
42374 |             /* We just set the device's slot to null. The slot will get reused in the next call
> to ma_track_device. */
42375 |             miniaudio.devices[deviceIndex] = null;
42376 |
42377 |             /* Trim the array if possible. */
42378 |             while (miniaudio.devices.length > 0) {
42379 |                 if (miniaudio.devices[miniaudio.devices.length-1] == null) {
42380 |                     miniaudio.devices.pop();
42381 |                 } else {
42382 |                     break;
42383 |                 }
42384 |             }
42385 |         };
42386 |
42387 |         miniaudio.untrack_device = function(device) {
42388 |             for (var iDevice = 0; iDevice < miniaudio.devices.length; ++iDevice) {
42389 |                 if (miniaudio.devices[iDevice] == device) {
42390 |                     return miniaudio.untrack_device_by_index(iDevice);
42391 |                 }
42392 |             }
42393 |         };
42394 |
42395 |         miniaudio.get_device_by_index = function(deviceIndex) {
42396 |             return miniaudio.devices[deviceIndex];
42397 |         };
42398 |
42399 |         miniaudio.unlock_event_types = (function(){
42400 |             return ['touchend', 'click'];
42401 |         })();
42402 |
42403 |         miniaudio.unlock = function() {
42404 |             for(var i = 0; i < miniaudio.devices.length; ++i) {
42405 |                 var device = miniaudio.devices[i];
42406 |                 if (device != null &&
42407 |                     device.webaudio != null &&
42408 |                     device.state === miniaudio.device_state.started) {

```

```

42409 |
42410 |         device.webaudio.resume().then(() => {
42411 |             _ma_device__on_notification_unlocked(device.pDevice);
42412 |         },
42413 |         (error) => {console.error("Failed to resume audiocontext", error);
42414 |         });
42415 |     }
42416 | }
42417 | miniaudio.unlock_event_types.map(function(event_type) {
42418 |     document.removeEventListener(event_type, miniaudio.unlock, true);
42419 | });
42420 | };
42421 |
42422 | miniaudio.unlock_event_types.map(function(event_type) {
42423 |     document.addEventListener(event_type, miniaudio.unlock, true);
42424 | });
42425 | }
42426 |
42427 | window.miniaudio.referenceCount += 1;
42428 |
42429 | return 1;
42430 | }, ma_device_type_playback, ma_device_type_capture, ma_device_type_duplex,
> ma_device_state_stopped, ma_device_state_started);
42431 |
42432 | if (resultFromJS != 1) {
42433 |     return MA_FAILED_TO_INIT_BACKEND;
42434 | }
42435 |
42436 | pCallbacks->onContextInit          = ma_context_init__webaudio;
42437 | pCallbacks->onContextUninit        = ma_context_uninit__webaudio;
42438 | pCallbacks->onContextEnumerateDevices = ma_context_enumerate_devices__webaudio;
42439 | pCallbacks->onContextGetDeviceInfo = ma_context_get_device_info__webaudio;
42440 | pCallbacks->onDeviceInit           = ma_device_init__webaudio;
42441 | pCallbacks->onDeviceUninit         = ma_device_uninit__webaudio;
42442 | pCallbacks->onDeviceStart          = ma_device_start__webaudio;
42443 | pCallbacks->onDeviceStop           = ma_device_stop__webaudio;
42444 | pCallbacks->onDeviceRead            = NULL; /* Not needed because WebAudio is asynchronous.
> */
42445 | pCallbacks->onDeviceWrite            = NULL; /* Not needed because WebAudio is asynchronous.
> */
42446 | pCallbacks->onDeviceDataLoop         = NULL; /* Not needed because WebAudio is asynchronous.
> */
42447 |
42448 | return MA_SUCCESS;
42449 | }
42450 | #endif /* MA_HAS_WEBAUDIO */
42451 |
42452 |
42453 |
42454 | static ma_bool32 ma__is_channel_map_valid(const ma_channel* pChannelMap, ma_uint32 channels)
42455 | {
42456 |     /* A blank channel map should be allowed, in which case it should use an appropriate default
> which will depend on context. */
42457 |     if (pChannelMap != NULL && pChannelMap[0] != MA_CHANNEL_NONE) {
42458 |         ma_uint32 iChannel;
42459 |
42460 |         if (channels == 0 || channels > MA_MAX_CHANNELS) {
42461 |             return MA_FALSE; /* Channel count out of range. */
42462 |         }
42463 |
42464 |         /* A channel cannot be present in the channel map more than once. */
42465 |         for (iChannel = 0; iChannel < channels; ++iChannel) {
42466 |             ma_uint32 jChannel;
42467 |             for (jChannel = iChannel + 1; jChannel < channels; ++jChannel) {
42468 |                 if (pChannelMap[iChannel] == pChannelMap[jChannel]) {
42469 |                     return MA_FALSE;
42470 |                 }
42471 |             }
42472 |         }
42473 |     }
42474 |
42475 |     return MA_TRUE;
42476 | }
42477 |
42478 |
42479 | static ma_bool32 ma_context_is_backend_asynchronous(ma_context* pContext)
42480 | {
42481 |     MA_ASSERT(pContext != NULL);
42482 |

```



```

42483 |     if (pContext->callbacks.onDeviceRead == NULL && pContext->callbacks.onDeviceWrite == NULL) {
42484 |         if (pContext->callbacks.onDeviceDataLoop == NULL) {
42485 |             return MA_TRUE;
42486 |         } else {
42487 |             return MA_FALSE;
42488 |         }
42489 |     } else {
42490 |         return MA_FALSE;
42491 |     }
42492 | }
42493 |
42494 |
42495 | static ma_result ma_device__post_init_setup(ma_device* pDevice, ma_device_type deviceType)
42496 | {
42497 |     ma_result result;
42498 |
42499 |     MA_ASSERT(pDevice != NULL);
42500 |
42501 |     if (deviceType == ma_device_type_capture || deviceType == ma_device_type_duplex || deviceType ==
> ma_device_type_loopback) {
42502 |         if (pDevice->capture.format == ma_format_unknown) {
42503 |             pDevice->capture.format = pDevice->capture.internalFormat;
42504 |         }
42505 |         if (pDevice->capture.channels == 0) {
42506 |             pDevice->capture.channels = pDevice->capture.internalChannels;
42507 |         }
42508 |         if (pDevice->capture.channelMap[0] == MA_CHANNEL_NONE) {
42509 |             MA_ASSERT(pDevice->capture.channels <= MA_MAX_CHANNELS);
42510 |             if (pDevice->capture.internalChannels == pDevice->capture.channels) {
42511 |                 ma_channel_map_copy(pDevice->capture.channelMap,
> pDevice->capture.internalChannelMap, pDevice->capture.channels);
42512 |             } else {
42513 |                 if (pDevice->capture.channelMixMode == ma_channel_mix_mode_simple) {
42514 |                     ma_channel_map_init_blank(pDevice->capture.channelMap,
> pDevice->capture.channels);
42515 |                 } else {
42516 |                     ma_channel_map_init_standard(ma_standard_channel_map_default,
> pDevice->capture.channelMap, ma_countof(pDevice->capture.channelMap), pDevice->capture.channels);
42517 |                 }
42518 |             }
42519 |         }
42520 |     }
42521 |
42522 |     if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
42523 |         if (pDevice->playback.format == ma_format_unknown) {
42524 |             pDevice->playback.format = pDevice->playback.internalFormat;
42525 |         }
42526 |         if (pDevice->playback.channels == 0) {
42527 |             pDevice->playback.channels = pDevice->playback.internalChannels;
42528 |         }
42529 |         if (pDevice->playback.channelMap[0] == MA_CHANNEL_NONE) {
42530 |             MA_ASSERT(pDevice->playback.channels <= MA_MAX_CHANNELS);
42531 |             if (pDevice->playback.internalChannels == pDevice->playback.channels) {
42532 |                 ma_channel_map_copy(pDevice->playback.channelMap,
> pDevice->playback.internalChannelMap, pDevice->playback.channels);
42533 |             } else {
42534 |                 if (pDevice->playback.channelMixMode == ma_channel_mix_mode_simple) {
42535 |                     ma_channel_map_init_blank(pDevice->playback.channelMap,
> pDevice->playback.channels);
42536 |                 } else {
42537 |                     ma_channel_map_init_standard(ma_standard_channel_map_default,
> pDevice->playback.channelMap, ma_countof(pDevice->playback.channelMap), pDevice->playback.channels);
42538 |                 }
42539 |             }
42540 |         }
42541 |     }
42542 |
42543 |     if (pDevice->sampleRate == 0) {
42544 |         if (deviceType == ma_device_type_capture || deviceType == ma_device_type_duplex ||
> deviceType == ma_device_type_loopback) {
42545 |             pDevice->sampleRate = pDevice->capture.internalSampleRate;
42546 |         } else {
42547 |             pDevice->sampleRate = pDevice->playback.internalSampleRate;
42548 |         }
42549 |     }
42550 |
42551 |     /* Data converters. */
42552 |     if (deviceType == ma_device_type_capture || deviceType == ma_device_type_duplex || deviceType ==
> ma_device_type_loopback) {

```

```

42553 |         /* Converting from internal device format to client format. */
42554 |         ma_data_converter_config converterConfig = ma_data_converter_config_init_default();
42555 |         converterConfig.formatIn                = pDevice->capture.internalFormat;
42556 |         converterConfig.channelsIn               = pDevice->capture.internalChannels;
42557 |         converterConfig.sampleRateIn            = pDevice->capture.internalSampleRate;
42558 |         converterConfig.pChannelMapIn           = pDevice->capture.internalChannelMap;
42559 |         converterConfig.formatOut               = pDevice->capture.format;
42560 |         converterConfig.channelsOut             = pDevice->capture.channels;
42561 |         converterConfig.sampleRateOut          = pDevice->sampleRate;
42562 |         converterConfig.pChannelMapOut          = pDevice->capture.channelMap;
42563 |         converterConfig.channelMixMode          = pDevice->capture.channelMixMode;
42564 |         converterConfig.calculateLFEFromSpatialChannels =
> pDevice->capture.calculateLFEFromSpatialChannels;
42565 |         converterConfig.allowDynamicSampleRate = MA_FALSE;
42566 |         converterConfig.resampling.algorithm    = pDevice->resampling.algorithm;
42567 |         converterConfig.resampling.linear.lpfOrder = pDevice->resampling.linear.lpfOrder;
42568 |         converterConfig.resampling.pBackendVTable = pDevice->resampling.pBackendVTable;
42569 |         converterConfig.resampling.pBackendUserData = pDevice->resampling.pBackendUserData;
42570 |
42571 |         /* Make sure the old converter is uninitialized first. */
42572 |         if (ma_device_get_state(pDevice) != ma_device_state_uninitialized) {
42573 |             ma_data_converter_uninit(&pDevice->capture.converter,
> &pDevice->pContext->allocationCallbacks);
42574 |         }
42575 |
42576 |         result = ma_data_converter_init(&converterConfig, &pDevice->pContext->allocationCallbacks,
> &pDevice->capture.converter);
42577 |         if (result != MA_SUCCESS) {
42578 |             return result;
42579 |         }
42580 |     }
42581 |
42582 |     if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
42583 |         /* Converting from client format to device format. */
42584 |         ma_data_converter_config converterConfig = ma_data_converter_config_init_default();
42585 |         converterConfig.formatIn                = pDevice->playback.format;
42586 |         converterConfig.channelsIn              = pDevice->playback.channels;
42587 |         converterConfig.sampleRateIn           = pDevice->sampleRate;
42588 |         converterConfig.pChannelMapIn          = pDevice->playback.channelMap;
42589 |         converterConfig.formatOut              = pDevice->playback.internalFormat;
42590 |         converterConfig.channelsOut            = pDevice->playback.internalChannels;
42591 |         converterConfig.sampleRateOut          = pDevice->playback.internalSampleRate;
42592 |         converterConfig.pChannelMapOut         = pDevice->playback.internalChannelMap;
42593 |         converterConfig.channelMixMode         = pDevice->playback.channelMixMode;
42594 |         converterConfig.calculateLFEFromSpatialChannels =
> pDevice->playback.calculateLFEFromSpatialChannels;
42595 |         converterConfig.allowDynamicSampleRate = MA_FALSE;
42596 |         converterConfig.resampling.algorithm    = pDevice->resampling.algorithm;
42597 |         converterConfig.resampling.linear.lpfOrder = pDevice->resampling.linear.lpfOrder;
42598 |         converterConfig.resampling.pBackendVTable = pDevice->resampling.pBackendVTable;
42599 |         converterConfig.resampling.pBackendUserData = pDevice->resampling.pBackendUserData;
42600 |
42601 |         /* Make sure the old converter is uninitialized first. */
42602 |         if (ma_device_get_state(pDevice) != ma_device_state_uninitialized) {
42603 |             ma_data_converter_uninit(&pDevice->playback.converter,
> &pDevice->pContext->allocationCallbacks);
42604 |         }
42605 |
42606 |         result = ma_data_converter_init(&converterConfig, &pDevice->pContext->allocationCallbacks,
> &pDevice->playback.converter);
42607 |         if (result != MA_SUCCESS) {
42608 |             return result;
42609 |         }
42610 |     }
42611 |
42612 |     /*
42613 |
42614 |     If the device is doing playback (ma_device_type_playback or ma_device_type_duplex), there's
42615 |     a couple of situations where we'll need a heap allocated cache.
42616 |
42617 |     The first is a duplex device for backends that use a callback for data delivery. The reason
42618 |     this is needed is that the input stage needs to have a buffer to place the input data while it
42619 |     waits for the playback stage, after which the miniaudio data callback will get fired. This is
42620 |     not needed for backends that use a blocking API because miniaudio manages temporary buffers on
42621 |     the stack to achieve this.
42622 |
42623 |     The other situation is when the data converter does not have the ability to query the number
42624 |     of input frames that are required in order to process a given number of output frames. When
42625 |     performing data conversion, it's useful if miniaudio know exactly how many frames it needs

```

```

42626 | from the client in order to generate a given number of output frames. This way, only exactly
42627 | the number of frames are needed to be read from the client which means no cache is necessary.
42628 | On the other hand, if miniaudio doesn't know how many frames to read, it is forced to read
42629 | in fixed sized chunks and then cache any residual unused input frames, those of which will be
42630 | processed at a later stage.
42631 | */
42632 | if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
42633 |     ma_uint64 unused;
42634 |
42635 |     pDevice->playback.inputCacheConsumed = 0;
42636 |     pDevice->playback.inputCacheRemaining = 0;
42637 |
42638 |     if (pDevice->type == ma_device_type_duplex ||
>                                     /* Duplex. backend may decide
> to use ma_device_handle_backend_data_callback() which will require this cache. */
42639 |         ma_data_converter_get_required_input_frame_count(&pDevice->playback.converter, 1,
> &unused) != MA_SUCCESS) /* Data conversion required input frame calculation not supported. */
42640 |     {
42641 |         /* We need a heap allocated cache. We want to size this based on the period size. */
42642 |         void* pNewInputCache;
42643 |         ma_uint64 newInputCacheCap;
42644 |         ma_uint64 newInputCacheSizeInBytes;
42645 |
42646 |         newInputCacheCap =
> ma_calculate_frame_count_after_resampling(pDevice->playback.internalSampleRate, pDevice->sampleRate,
> pDevice->playback.internalPeriodSizeInFrames);
42647 |
42648 |         newInputCacheSizeInBytes = newInputCacheCap *
> ma_get_bytes_per_frame(pDevice->playback.format, pDevice->playback.channels);
42649 |         if (newInputCacheSizeInBytes > MA_SIZE_MAX) {
42650 |             ma_free(pDevice->playback.pInputCache, &pDevice->pContext->allocationCallbacks);
42651 |             pDevice->playback.pInputCache = NULL;
42652 |             pDevice->playback.inputCacheCap = 0;
42653 |             return MA_OUT_OF_MEMORY; /* Allocation too big. Should never hit this, but makes
> the cast below safer for 32-bit builds. */
42654 |         }
42655 |
42656 |         pNewInputCache = ma_realloc(pDevice->playback.pInputCache,
> (size_t)newInputCacheSizeInBytes, &pDevice->pContext->allocationCallbacks);
42657 |         if (pNewInputCache == NULL) {
42658 |             ma_free(pDevice->playback.pInputCache, &pDevice->pContext->allocationCallbacks);
42659 |             pDevice->playback.pInputCache = NULL;
42660 |             pDevice->playback.inputCacheCap = 0;
42661 |             return MA_OUT_OF_MEMORY;
42662 |         }
42663 |
42664 |         pDevice->playback.pInputCache = pNewInputCache;
42665 |         pDevice->playback.inputCacheCap = newInputCacheCap;
42666 |     } else {
42667 |         /* Heap allocation not required. Make sure we clear out the old cache just in case this
> function was called in response to a route change. */
42668 |         ma_free(pDevice->playback.pInputCache, &pDevice->pContext->allocationCallbacks);
42669 |         pDevice->playback.pInputCache = NULL;
42670 |         pDevice->playback.inputCacheCap = 0;
42671 |     }
42672 | }
42673 |
42674 | return MA_SUCCESS;
42675 | }
42676 |
42677 | MA_API ma_result ma_device_post_init(ma_device* pDevice, ma_device_type deviceType, const
> ma_device_descriptor* pDescriptorPlayback, const ma_device_descriptor* pDescriptorCapture)
42678 | {
42679 |     ma_result result;
42680 |
42681 |     if (pDevice == NULL) {
42682 |         return MA_INVALID_ARGS;
42683 |     }
42684 |
42685 |     /* Capture. */
42686 |     if (deviceType == ma_device_type_capture || deviceType == ma_device_type_duplex || deviceType ==
> ma_device_type_loopback) {
42687 |         if (ma_device_descriptor_is_valid(pDescriptorCapture) == MA_FALSE) {
42688 |             return MA_INVALID_ARGS;
42689 |         }
42690 |
42691 |         pDevice->capture.internalFormat = pDescriptorCapture->format;
42692 |         pDevice->capture.internalChannels = pDescriptorCapture->channels;
42693 |         pDevice->capture.internalSampleRate = pDescriptorCapture->sampleRate;

```

```

42694 |         MA_COPY_MEMORY(pDevice->capture.internalChannelMap, pDescriptorCapture->channelMap,
> sizeof(pDescriptorCapture->channelMap));
42695 |         pDevice->capture.internalPeriodSizeInFrames = pDescriptorCapture->periodSizeInFrames;
42696 |         pDevice->capture.internalPeriods = pDescriptorCapture->periodCount;
42697 |
42698 |         if (pDevice->capture.internalPeriodSizeInFrames == 0) {
42699 |             pDevice->capture.internalPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_milliseconds(pDescriptorCapture->periodSizeInMilliseconds,
> pDescriptorCapture->sampleRate);
42700 |         }
42701 |     }
42702 |
42703 |     /* Playback. */
42704 |     if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
42705 |         if (ma_device_descriptor_is_valid(pDescriptorPlayback) == MA_FALSE) {
42706 |             return MA_INVALID_ARGS;
42707 |         }
42708 |
42709 |         pDevice->playback.internalFormat = pDescriptorPlayback->format;
42710 |         pDevice->playback.internalChannels = pDescriptorPlayback->channels;
42711 |         pDevice->playback.internalSampleRate = pDescriptorPlayback->sampleRate;
42712 |         MA_COPY_MEMORY(pDevice->playback.internalChannelMap, pDescriptorPlayback->channelMap,
> sizeof(pDescriptorPlayback->channelMap));
42713 |         pDevice->playback.internalPeriodSizeInFrames = pDescriptorPlayback->periodSizeInFrames;
42714 |         pDevice->playback.internalPeriods = pDescriptorPlayback->periodCount;
42715 |
42716 |         if (pDevice->playback.internalPeriodSizeInFrames == 0) {
42717 |             pDevice->playback.internalPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_milliseconds(pDescriptorPlayback->periodSizeInMilliseconds,
> pDescriptorPlayback->sampleRate);
42718 |         }
42719 |     }
42720 |
42721 |     /*
42722 |     The name of the device can be retrieved from device info. This may be temporary and replaced
> with a `ma_device_get_info(pDevice, deviceType)` instead.
42723 |     For loopback devices, we need to retrieve the name of the playback device.
42724 |     */
42725 |     {
42726 |         ma_device_info deviceInfo;
42727 |
42728 |         if (deviceType == ma_device_type_capture || deviceType == ma_device_type_duplex ||
> deviceType == ma_device_type_loopback) {
42729 |             result = ma_device_get_info(pDevice, ma_device_type_capture, &deviceInfo);
42730 |             if (result == MA_SUCCESS) {
42731 |                 ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name), deviceInfo.name,
> (size_t)-1);
42732 |             } else {
42733 |                 /* We failed to retrieve the device info. Fall back to a default name. */
42734 |                 if (pDescriptorCapture->pDeviceID == NULL) {
42735 |                     ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name),
> MA_DEFAULT_CAPTURE_DEVICE_NAME, (size_t)-1);
42736 |                 } else {
42737 |                     ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name), "Capture
> Device", (size_t)-1);
42738 |                 }
42739 |             }
42740 |         }
42741 |
42742 |         if (deviceType == ma_device_type_playback || deviceType == ma_device_type_duplex) {
42743 |             result = ma_device_get_info(pDevice, ma_device_type_playback, &deviceInfo);
42744 |             if (result == MA_SUCCESS) {
42745 |                 ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name),
> deviceInfo.name, (size_t)-1);
42746 |             } else {
42747 |                 /* We failed to retrieve the device info. Fall back to a default name. */
42748 |                 if (pDescriptorPlayback->pDeviceID == NULL) {
42749 |                     ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name),
> MA_DEFAULT_PLAYBACK_DEVICE_NAME, (size_t)-1);
42750 |                 } else {
42751 |                     ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name), "Playback
> Device", (size_t)-1);
42752 |                 }
42753 |             }
42754 |         }
42755 |     }
42756 |
42757 |     /* Update data conversion. */
42758 |     return ma_device__post_init_setup(pDevice, deviceType); /* TODO: Should probably rename

```

```

> ma_device__post_init_setup() to something better. */
42759 | }
42760 |
42761 |
42762 | static ma_thread_result MA_THREADCALL ma_worker_thread(void* pData)
42763 | {
42764 |     ma_device* pDevice = (ma_device*)pData;
42765 |     #if defined(MA_WIN32) && !defined(MA_XBOX)
42766 |         HRESULT CoInitializeResult;
42767 |     #endif
42768 |
42769 |     MA_ASSERT(pDevice != NULL);
42770 |
42771 |     #if defined(MA_WIN32) && !defined(MA_XBOX)
42772 |         CoInitializeResult = ma_CoInitializeEx(pDevice->pContext, NULL, MA_COINIT_VALUE);
42773 |     #endif
42774 |
42775 |     /*
42776 |      * When the device is being initialized its initial state is set to ma_device_state_uninitialized.
42777 |      * Before returning from
42778 |      * ma_device_init(), the state needs to be set to something valid. In miniaudio the device's
42779 |      * default state immediately
42780 |      * after initialization is stopped, so therefore we need to mark the device as such. miniaudio will
42781 |      * wait on the worker
42782 |      * thread to signal an event to know when the worker thread is ready for action.
42783 |      */
42784 |     ma_device__set_state(pDevice, ma_device_state_stopped);
42785 |     ma_event_signal(&pDevice->stopEvent);
42786 |
42787 |     for (;;) { /* <-- This loop just keeps the thread alive. The main audio loop is inside. */
42788 |         ma_result startResult;
42789 |         ma_result stopResult; /* <-- This will store the result from onDeviceStop(). If it returns
42790 |         > an error, we don't fire the stopped notification callback. */
42791 |
42792 |         /* We wait on an event to know when something has requested that the device be started and
42793 |         > the main loop entered. */
42794 |         ma_event_wait(&pDevice->wakeupEvent);
42795 |
42796 |         /* Default result code. */
42797 |         pDevice->workResult = MA_SUCCESS;
42798 |
42799 |         /* If the reason for the wake up is that we are terminating, just break from the loop. */
42800 |         if (ma_device_get_state(pDevice) == ma_device_state_uninitialized) {
42801 |             break;
42802 |         }
42803 |
42804 |         /*
42805 |          * Getting to this point means the device is wanting to get started. The function that has
42806 |          > requested that the device
42807 |          > be started will be waiting on an event (pDevice->startEvent) which means we need to make
42808 |          > sure we signal the event
42809 |          > in both the success and error case. It's important that the state of the device is set
42810 |          > _before_ signaling the event.
42811 |          */
42812 |         MA_ASSERT(ma_device_get_state(pDevice) == ma_device_state_starting);
42813 |
42814 |         /* If the device has a start callback, start it now. */
42815 |         if (pDevice->pContext->callbacks.onDeviceStart != NULL) {
42816 |             startResult = pDevice->pContext->callbacks.onDeviceStart(pDevice);
42817 |         } else {
42818 |             startResult = MA_SUCCESS;
42819 |         }
42820 |
42821 |         /*
42822 |          * If starting was not successful we'll need to loop back to the start and wait for something
42823 |          * to happen (pDevice->wakeupEvent).
42824 |          */
42825 |         if (startResult != MA_SUCCESS) {
42826 |             pDevice->workResult = startResult;
42827 |             ma_event_signal(&pDevice->startEvent); /* <-- Always signal the start event so
42828 |             > ma_device_start() can return as it'll be waiting on it. */
42829 |             continue;
42830 |         }
42831 |
42832 |         /* Make sure the state is set appropriately. */
42833 |         ma_device__set_state(pDevice, ma_device_state_started); /* <-- Set this before signaling the
42834 |         > event so that the state is always guaranteed to be good after ma_device_start() has returned. */
42835 |         ma_event_signal(&pDevice->startEvent);
42836 |     }

```

```

42827 |         ma_device__on_notification_started(pDevice);
42828 |
42829 |         if (pDevice->pContext->callbacks.onDeviceDataLoop != NULL) {
42830 |             pDevice->pContext->callbacks.onDeviceDataLoop(pDevice);
42831 |         } else {
42832 |             /* The backend is not using a custom main loop implementation, so now fall back to the
> blocking read-write implementation. */
42833 |             ma_device_audio_thread__default_read_write(pDevice);
42834 |         }
42835 |
42836 |         /* Getting here means we have broken from the main loop which happens the application has
> requested that device be stopped. */
42837 |         if (pDevice->pContext->callbacks.onDeviceStop != NULL) {
42838 |             stopResult = pDevice->pContext->callbacks.onDeviceStop(pDevice);
42839 |         } else {
42840 |             stopResult = MA_SUCCESS;    /* No stop callback with the backend. Just assume
> successful. */
42841 |         }
42842 |
42843 |         /*
42844 |         After the device has stopped, make sure an event is posted. Don't post a stopped event if
42845 |         stopping failed. This can happen on some backends when the underlying stream has been
42846 |         stopped due to the device being physically unplugged or disabled via an OS setting.
42847 |         */
42848 |         if (stopResult == MA_SUCCESS) {
42849 |             ma_device__on_notification_stopped(pDevice);
42850 |         }
42851 |
42852 |         /* If we stopped because the device has been uninitialized, abort now. */
42853 |         if (ma_device_get_state(pDevice) == ma_device_state_uninitialized) {
42854 |             break;
42855 |         }
42856 |
42857 |         /* A function somewhere is waiting for the device to have stopped for real so we need to
> signal an event to allow it to continue. */
42858 |         ma_device__set_state(pDevice, ma_device_state_stopped);
42859 |         ma_event_signal(&pDevice->stopEvent);
42860 |     }
42861 |
42862 |     #if defined(MA_WIN32) && !defined(MA_XBOX)
42863 |         if (CoInitializeResult == S_OK || CoInitializeResult == S_FALSE) {
42864 |             ma_CoUninitialize(pDevice->pContext);
42865 |         }
42866 |     #endif
42867 |
42868 |     return (ma_thread_result)0;
42869 | }
42870 |
42871 |
42872 | /* Helper for determining whether or not the given device is initialized. */
42873 | static ma_bool32 ma_device__is_initialized(ma_device* pDevice)
42874 | {
42875 |     if (pDevice == NULL) {
42876 |         return MA_FALSE;
42877 |     }
42878 |
42879 |     return ma_device_get_state(pDevice) != ma_device_state_uninitialized;
42880 | }
42881 |
42882 |
42883 | #ifdef MA_WIN32
42884 | static ma_result ma_context_uninit_backend_apis__win32(ma_context* pContext)
42885 | {
42886 |     /* For some reason UWP complains when CoUninitialize() is called. I'm just not going to call it
> on UWP. */
42887 |     #if defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)
42888 |     {
42889 |         /* TODO: Remove this once the new single threaded backend system is in place in 0.12. */
42890 |         #if !defined(MA_XBOX)
42891 |         {
42892 |             if (pContext->win32.CoInitializeResult == S_OK || pContext->win32.CoInitializeResult ==
> S_FALSE) {
42893 |                 ma_CoUninitialize(pContext);    /* TODO: Remove this once the new single threaded
> backend system is in place in 0.12. */
42894 |             }
42895 |         }
42896 |     }
42897 |     #endif
42898 |
42899 |     #if defined(MA_WIN32_DESKTOP)

```



```

42899 |         ma_dlclose(ma_context_get_log(pContext), pContext->win32.hUser32DLL);
42900 |         ma_dlclose(ma_context_get_log(pContext), pContext->win32.hAdvapi32DLL);
42901 |     #endif
42902 |
42903 |         ma_dlclose(ma_context_get_log(pContext), pContext->win32.hOle32DLL);
42904 |     }
42905 |     #else
42906 |     {
42907 |         (void)pContext;
42908 |     }
42909 |     #endif
42910 |
42911 |     return MA_SUCCESS;
42912 | }
42913 |
42914 | static ma_result ma_context_init_backend_apis_win32(ma_context* pContext)
42915 | {
42916 |     /*
42917 |     > TODO: Reassess all of this stuff and move everything to the relevant backends. For example, I
42918 |     > think
42919 |     > GetForegroundWindow() and GetDesktopWindow() are only used by the DirectSound backend.
42920 |     > */
42921 |     #if (defined(MA_WIN32_DESKTOP) || defined(MA_WIN32_GDK)) && !defined(MA_XBOX)
42922 |     {
42923 |         #if defined(MA_WIN32_DESKTOP)
42924 |         {
42925 |             /* User32.dll */
42926 |             pContext->win32.hUser32DLL = ma_dlopen(ma_context_get_log(pContext), "user32.dll");
42927 |             if (pContext->win32.hUser32DLL == NULL) {
42928 |                 return MA_FAILED_TO_INIT_BACKEND;
42929 |             }
42930 |             pContext->win32.GetForegroundWindow = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42931 | > pContext->win32.hUser32DLL, "GetForegroundWindow");
42932 |             pContext->win32.GetDesktopWindow = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42933 | > pContext->win32.hUser32DLL, "GetDesktopWindow");
42934 |
42935 |             /* Advapi32.dll */
42936 |             pContext->win32.hAdvapi32DLL = ma_dlopen(ma_context_get_log(pContext), "advapi32.dll");
42937 |             if (pContext->win32.hAdvapi32DLL == NULL) {
42938 |                 return MA_FAILED_TO_INIT_BACKEND;
42939 |             }
42940 |             pContext->win32.RegOpenKeyExA = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42941 | > pContext->win32.hAdvapi32DLL, "RegOpenKeyExA");
42942 |             pContext->win32.RegCloseKey = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42943 | > pContext->win32.hAdvapi32DLL, "RegCloseKey");
42944 |             pContext->win32.RegQueryValueExA = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42945 | > pContext->win32.hAdvapi32DLL, "RegQueryValueExA");
42946 |             #endif
42947 |
42948 |             /* Ole32.dll */
42949 |             pContext->win32.hOle32DLL = ma_dlopen(ma_context_get_log(pContext), "ole32.dll");
42950 |             if (pContext->win32.hOle32DLL == NULL) {
42951 |                 return MA_FAILED_TO_INIT_BACKEND;
42952 |             }
42953 |             pContext->win32.CoInitialize = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42954 | > pContext->win32.hOle32DLL, "CoInitialize");
42955 |             pContext->win32.CoInitializeEx = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42956 | > pContext->win32.hOle32DLL, "CoInitializeEx");
42957 |             pContext->win32.CoUninitialize = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42958 | > pContext->win32.hOle32DLL, "CoUninitialize");
42959 |             pContext->win32.CoCreateInstance = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42960 | > pContext->win32.hOle32DLL, "CoCreateInstance");
42961 |             pContext->win32.CoTaskMemFree = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42962 | > pContext->win32.hOle32DLL, "CoTaskMemFree");
42963 |             pContext->win32.PropVariantClear = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42964 | > pContext->win32.hOle32DLL, "PropVariantClear");
42965 |             pContext->win32.StringFromGUID2 = (ma_proc)ma_dlsym(ma_context_get_log(pContext),
42966 | > pContext->win32.hOle32DLL, "StringFromGUID2");
42967 |             #endif
42968 |             #else
42969 |             {
42970 |                 (void)pContext; /* Unused. */
42971 |             }
42972 |             #endif
42973 |         }
42974 |     }
42975 |     #endif

```



```

42965 |
42966 |     /* TODO: Remove this once the new single threaded backend system is in place in 0.12. */
42967 |     #if !defined(MA_XBOX)
42968 |     {
42969 |         pContext->win32.CoInitializeResult = ma_CoInitializeEx(pContext, NULL, MA_COINIT_VALUE);
42970 |     }
42971 |     #endif
42972 |
42973 |     return MA_SUCCESS;
42974 | }
42975 | #else
42976 | static ma_result ma_context_uninit_backend_apis__nix(ma_context* pContext)
42977 | {
42978 |     (void)pContext;
42979 |
42980 |     return MA_SUCCESS;
42981 | }
42982 |
42983 | static ma_result ma_context_init_backend_apis__nix(ma_context* pContext)
42984 | {
42985 |     (void)pContext;
42986 |
42987 |     return MA_SUCCESS;
42988 | }
42989 | #endif
42990 |
42991 | static ma_result ma_context_init_backend_apis(ma_context* pContext)
42992 | {
42993 |     ma_result result;
42994 |     #ifdef MA_WIN32
42995 |         result = ma_context_init_backend_apis__win32(pContext);
42996 |     #else
42997 |         result = ma_context_init_backend_apis__nix(pContext);
42998 |     #endif
42999 |
43000 |     return result;
43001 | }
43002 |
43003 | static ma_result ma_context_uninit_backend_apis(ma_context* pContext)
43004 | {
43005 |     ma_result result;
43006 |     #ifdef MA_WIN32
43007 |         result = ma_context_uninit_backend_apis__win32(pContext);
43008 |     #else
43009 |         result = ma_context_uninit_backend_apis__nix(pContext);
43010 |     #endif
43011 |
43012 |     return result;
43013 | }
43014 |
43015 |
43016 | /* The default capacity doesn't need to be too big. */
43017 | #ifndef MA_DEFAULT_DEVICE_JOB_QUEUE_CAPACITY
43018 | #define MA_DEFAULT_DEVICE_JOB_QUEUE_CAPACITY    32
43019 | #endif
43020 |
43021 | MA_API ma_device_job_thread_config ma_device_job_thread_config_init(void)
43022 | {
43023 |     ma_device_job_thread_config config;
43024 |
43025 |     MA_ZERO_OBJECT(&config);
43026 |     config.noThread          = MA_FALSE;
43027 |     config.jobQueueCapacity = MA_DEFAULT_DEVICE_JOB_QUEUE_CAPACITY;
43028 |     config.jobQueueFlags    = 0;
43029 |
43030 |     return config;
43031 | }
43032 |
43033 |
43034 | static ma_thread_result MA_THREADCALL ma_device_job_thread_entry(void* pUserData)
43035 | {
43036 |     ma_device_job_thread* pJobThread = (ma_device_job_thread*)pUserData;
43037 |     MA_ASSERT(pJobThread != NULL);
43038 |
43039 |     for (;;) {
43040 |         ma_result result;
43041 |         ma_job job;
43042 |
43043 |         result = ma_device_job_thread_next(pJobThread, &job);

```

```

43044 |         if (result != MA_SUCCESS) {
43045 |             break;
43046 |         }
43047 |
43048 |         if (job.toc.breakup.code == MA_JOB_TYPE_QUIT) {
43049 |             break;
43050 |         }
43051 |
43052 |         ma_job_process(&job);
43053 |     }
43054 |
43055 |     return (ma_thread_result)0;
43056 | }
43057 |
43058 | MA_API ma_result ma_device_job_thread_init(const ma_device_job_thread_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_device_job_thread* pJobThread)
43059 | {
43060 |     ma_result result;
43061 |     ma_job_queue_config jobQueueConfig;
43062 |
43063 |     if (pJobThread == NULL) {
43064 |         return MA_INVALID_ARGS;
43065 |     }
43066 |
43067 |     MA_ZERO_OBJECT(pJobThread);
43068 |
43069 |     if (pConfig == NULL) {
43070 |         return MA_INVALID_ARGS;
43071 |     }
43072 |
43073 |
43074 |     /* Initialize the job queue before the thread to ensure it's in a valid state. */
43075 |     jobQueueConfig = ma_job_queue_config_init(pConfig->jobQueueFlags, pConfig->jobQueueCapacity);
43076 |
43077 |     result = ma_job_queue_init(&jobQueueConfig, pAllocationCallbacks, &pJobThread->jobQueue);
43078 |     if (result != MA_SUCCESS) {
43079 |         return result; /* Failed to initialize job queue. */
43080 |     }
43081 |
43082 |
43083 |     /* The thread needs to be initialized after the job queue to ensure the thread doesn't try to
> access it prematurely. */
43084 |     if (pConfig->noThread == MA_FALSE) {
43085 |         result = ma_thread_create(&pJobThread->thread, ma_thread_priority_normal, 0,
> ma_device_job_thread_entry, pJobThread, pAllocationCallbacks);
43086 |         if (result != MA_SUCCESS) {
43087 |             ma_job_queue_uninit(&pJobThread->jobQueue, pAllocationCallbacks);
43088 |             return result; /* Failed to create the job thread. */
43089 |         }
43090 |
43091 |         pJobThread->_hasThread = MA_TRUE;
43092 |     } else {
43093 |         pJobThread->_hasThread = MA_FALSE;
43094 |     }
43095 |
43096 |
43097 |     return MA_SUCCESS;
43098 | }
43099 |
43100 | MA_API void ma_device_job_thread_uninit(ma_device_job_thread* pJobThread, const
> ma_allocation_callbacks* pAllocationCallbacks)
43101 | {
43102 |     if (pJobThread == NULL) {
43103 |         return;
43104 |     }
43105 |
43106 |     /* The first thing to do is post a quit message to the job queue. If we're using a thread we'll
> need to wait for it. */
43107 |     {
43108 |         ma_job job = ma_job_init(MA_JOB_TYPE_QUIT);
43109 |         ma_device_job_thread_post(pJobThread, &job);
43110 |     }
43111 |
43112 |     /* Wait for the thread to terminate naturally. */
43113 |     if (pJobThread->_hasThread) {
43114 |         ma_thread_wait(&pJobThread->thread);
43115 |     }
43116 |
43117 |     /* At this point the thread should be terminated so we can safely uninitialized the job queue. */

```

```

43118 |     ma_job_queue_uninit(&pJobThread->jobQueue, pAllocationCallbacks);
43119 | }
43120 |
43121 | MA_API ma_result ma_device_job_thread_post(ma_device_job_thread* pJobThread, const ma_job* pJob)
43122 | {
43123 |     if (pJobThread == NULL || pJob == NULL) {
43124 |         return MA_INVALID_ARGS;
43125 |     }
43126 |
43127 |     return ma_job_queue_post(&pJobThread->jobQueue, pJob);
43128 | }
43129 |
43130 | MA_API ma_result ma_device_job_thread_next(ma_device_job_thread* pJobThread, ma_job* pJob)
43131 | {
43132 |     if (pJob == NULL) {
43133 |         return MA_INVALID_ARGS;
43134 |     }
43135 |
43136 |     MA_ZERO_OBJECT(pJob);
43137 |
43138 |     if (pJobThread == NULL) {
43139 |         return MA_INVALID_ARGS;
43140 |     }
43141 |
43142 |     return ma_job_queue_next(&pJobThread->jobQueue, pJob);
43143 | }
43144 |
43145 |
43146 | MA_API ma_bool32 ma_device_id_equal(const ma_device_id* pA, const ma_device_id* pB)
43147 | {
43148 |     size_t i;
43149 |
43150 |     if (pA == NULL || pB == NULL) {
43151 |         return MA_FALSE;
43152 |     }
43153 |
43154 |     for (i = 0; i < sizeof(ma_device_id); i += 1) {
43155 |         if (((const char*)pA)[i] != ((const char*)pB)[i]) {
43156 |             return MA_FALSE;
43157 |         }
43158 |     }
43159 |
43160 |     return MA_TRUE;
43161 | }
43162 |
43163 |
43164 |
43165 | MA_API ma_context_config ma_context_config_init(void)
43166 | {
43167 |     ma_context_config config;
43168 |     MA_ZERO_OBJECT(&config);
43169 |
43170 |     return config;
43171 | }
43172 |
43173 | MA_API ma_result ma_context_init(const ma_backend backends[], ma_uint32 backendCount, const
> ma_context_config* pConfig, ma_context* pContext)
43174 | {
43175 |     ma_result result;
43176 |     ma_context_config defaultConfig;
43177 |     ma_backend defaultBackends[ma_backend_null+1];
43178 |     ma_uint32 iBackend;
43179 |     ma_backend* pBackendsToIterate;
43180 |     ma_uint32 backendsToIterateCount;
43181 |
43182 |     if (pContext == NULL) {
43183 |         return MA_INVALID_ARGS;
43184 |     }
43185 |
43186 |     MA_ZERO_OBJECT(pContext);
43187 |
43188 |     /* Always make sure the config is set first to ensure properties are available as soon as
> possible. */
43189 |     if (pConfig == NULL) {
43190 |         defaultConfig = ma_context_config_init();
43191 |         pConfig = &defaultConfig;
43192 |     }
43193 |
43194 |     /* Allocation callbacks need to come first because they'll be passed around to other areas. */

```

```

43195 |     result = ma_allocation_callbacks_init_copy(&pContext->allocationCallbacks,
> &pConfig->allocationCallbacks);
43196 |     if (result != MA_SUCCESS) {
43197 |         return result;
43198 |     }
43199 |
43200 |     /* Get a lot set up first so we can start logging ASAP. */
43201 |     if (pConfig->pLog != NULL) {
43202 |         pContext->pLog = pConfig->pLog;
43203 |     } else {
43204 |         result = ma_log_init(&pContext->allocationCallbacks, &pContext->log);
43205 |         if (result == MA_SUCCESS) {
43206 |             pContext->pLog = &pContext->log;
43207 |         } else {
43208 |             pContext->pLog = NULL; /* Logging is not available. */
43209 |         }
43210 |     }
43211 |
43212 |     pContext->threadPriority = pConfig->threadPriority;
43213 |     pContext->threadStackSize = pConfig->threadStackSize;
43214 |     pContext->pUserData = pConfig->pUserData;
43215 |
43216 |     /* Backend APIs need to be initialized first. This is where external libraries will be loaded
> and linked. */
43217 |     result = ma_context_init_backend_apis(pContext);
43218 |     if (result != MA_SUCCESS) {
43219 |         return result;
43220 |     }
43221 |
43222 |     for (iBackend = 0; iBackend <= ma_backend_null; ++iBackend) {
43223 |         defaultBackends[iBackend] = (ma_backend)iBackend;
43224 |     }
43225 |
43226 |     pBackendsToIterate = (ma_backend*)backends;
43227 |     backendsToIterateCount = backendCount;
43228 |     if (pBackendsToIterate == NULL) {
43229 |         pBackendsToIterate = (ma_backend*)defaultBackends;
43230 |         backendsToIterateCount = ma_countof(defaultBackends);
43231 |     }
43232 |
43233 |     MA_ASSERT(pBackendsToIterate != NULL);
43234 |
43235 |     for (iBackend = 0; iBackend < backendsToIterateCount; iBackend += 1) {
43236 |         ma_backend backend = pBackendsToIterate[iBackend];
43237 |
43238 |         /* Make sure all callbacks are reset so we don't accidentally drag in any from previously
> failed initialization attempts. */
43239 |         MA_ZERO_OBJECT(&pContext->callbacks);
43240 |
43241 |         /* These backends are using the new callback system. */
43242 |         switch (backend) {
43243 |             #ifdef MA_HAS_WASAPI
43244 |             case ma_backend_wasapi:
43245 |             {
43246 |                 pContext->callbacks.onContextInit = ma_context_init__wasapi;
43247 |             } break;
43248 |             #endif
43249 |             #ifdef MA_HAS_DSOUND
43250 |             case ma_backend_dsound:
43251 |             {
43252 |                 pContext->callbacks.onContextInit = ma_context_init__dsound;
43253 |             } break;
43254 |             #endif
43255 |             #ifdef MA_HAS_WINMM
43256 |             case ma_backend_winmm:
43257 |             {
43258 |                 pContext->callbacks.onContextInit = ma_context_init__winmm;
43259 |             } break;
43260 |             #endif
43261 |             #ifdef MA_HAS_COREAUDIO
43262 |             case ma_backend_coreaudio:
43263 |             {
43264 |                 pContext->callbacks.onContextInit = ma_context_init__coreaudio;
43265 |             } break;
43266 |             #endif
43267 |             #ifdef MA_HAS_SNDIO
43268 |             case ma_backend_sndio:
43269 |             {
43270 |                 pContext->callbacks.onContextInit = ma_context_init__sndio;

```

```

43271 |         } break;
43272 |     #endif
43273 |     #ifdef MA_HAS_AUDIO4
43274 |         case ma_backend_audio4:
43275 |         {
43276 |             pContext->callbacks.onContextInit = ma_context_init__audio4;
43277 |         } break;
43278 |     #endif
43279 |     #ifdef MA_HAS_OSS
43280 |         case ma_backend_oss:
43281 |         {
43282 |             pContext->callbacks.onContextInit = ma_context_init__oss;
43283 |         } break;
43284 |     #endif
43285 |     #ifdef MA_HAS_PULSEAUDIO
43286 |         case ma_backend_pulseaudio:
43287 |         {
43288 |             pContext->callbacks.onContextInit = ma_context_init__pulse;
43289 |         } break;
43290 |     #endif
43291 |     #ifdef MA_HAS_ALSA
43292 |         case ma_backend_alsa:
43293 |         {
43294 |             pContext->callbacks.onContextInit = ma_context_init__alsa;
43295 |         } break;
43296 |     #endif
43297 |     #ifdef MA_HAS_JACK
43298 |         case ma_backend_jack:
43299 |         {
43300 |             pContext->callbacks.onContextInit = ma_context_init__jack;
43301 |         } break;
43302 |     #endif
43303 |     #ifdef MA_HAS_AAUDIO
43304 |         case ma_backend_aaudio:
43305 |         {
43306 |             if (ma_is_backend_enabled(backend)) {
43307 |                 pContext->callbacks.onContextInit = ma_context_init__aaudio;
43308 |             }
43309 |         } break;
43310 |     #endif
43311 |     #ifdef MA_HAS_OPENSF
43312 |         case ma_backend_opensf:
43313 |         {
43314 |             if (ma_is_backend_enabled(backend)) {
43315 |                 pContext->callbacks.onContextInit = ma_context_init__opensf;
43316 |             }
43317 |         } break;
43318 |     #endif
43319 |     #ifdef MA_HAS_WEB_AUDIO
43320 |         case ma_backend_webaudio:
43321 |         {
43322 |             pContext->callbacks.onContextInit = ma_context_init__webaudio;
43323 |         } break;
43324 |     #endif
43325 |     #ifdef MA_HAS_CUSTOM
43326 |         case ma_backend_custom:
43327 |         {
43328 |             /* Slightly different logic for custom backends. Custom backends can optionally set
43329 | > all of their callbacks in the config. */
43330 |             pContext->callbacks = pConfig->custom;
43331 |         } break;
43332 |     #endif
43333 |     #ifdef MA_HAS_NULL
43334 |         case ma_backend_null:
43335 |         {
43336 |             pContext->callbacks.onContextInit = ma_context_init__null;
43337 |         } break;
43338 |     #endif
43339 |         default: break;
43340 |     }
43341 |
43342 |     if (pContext->callbacks.onContextInit != NULL) {
43343 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "Attempting to initialize
43344 | > %s backend...\n", ma_get_backend_name(backend));
43345 |         result = pContext->callbacks.onContextInit(pContext, pConfig, &pContext->callbacks);
43346 |     } else {
43347 |         /* Getting here means the onContextInit callback is not set which means the backend is
43348 | > not enabled. Special case for the custom backend. */

```

```

43347 |         if (backend != ma_backend_custom) {
43348 |             result = MA_BACKEND_NOT_ENABLED;
43349 |         } else {
43350 |             #if !defined(MA_HAS_CUSTOM)
43351 |                 result = MA_BACKEND_NOT_ENABLED;
43352 |             #else
43353 |                 result = MA_NO_BACKEND;
43354 |             #endif
43355 |         }
43356 |     }
43357 |
43358 |     /* If this iteration was successful, return. */
43359 |     if (result == MA_SUCCESS) {
43360 |         result = ma_mutex_init(&pContext->deviceEnumLock);
43361 |         if (result != MA_SUCCESS) {
43362 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "Failed to
> initialize mutex for device enumeration. ma_context_get_devices() is not thread safe.\n");
43363 |         }
43364 |
43365 |         result = ma_mutex_init(&pContext->deviceInfoLock);
43366 |         if (result != MA_SUCCESS) {
43367 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_WARNING, "Failed to
> initialize mutex for device info retrieval. ma_context_get_device_info() is not thread safe.\n");
43368 |         }
43369 |
43370 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "System
> Architecture:\n");
43371 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "  Endian: %s\n",
> ma_is_little_endian() ? "LE" : "BE");
43372 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "  SSE2:  %s\n",
> ma_has_sse2()      ? "YES" : "NO");
43373 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "  AVX2:  %s\n",
> ma_has_avx2()      ? "YES" : "NO");
43374 |         ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "  NEON:  %s\n",
> ma_has_neon()      ? "YES" : "NO");
43375 |
43376 |         pContext->backend = backend;
43377 |         return result;
43378 |     } else {
43379 |         if (result == MA_BACKEND_NOT_ENABLED) {
43380 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "%s backend is
> disabled.\n", ma_get_backend_name(backend));
43381 |         } else {
43382 |             ma_log_postf(ma_context_get_log(pContext), MA_LOG_LEVEL_DEBUG, "Failed to initialize
> %s backend.\n", ma_get_backend_name(backend));
43383 |         }
43384 |     }
43385 | }
43386 |
43387 | /* If we get here it means an error occurred. */
43388 | MA_ZERO_OBJECT(pContext); /* Safety. */
43389 | return MA_NO_BACKEND;
43390 | }
43391 |
43392 | MA_API ma_result ma_context_uninit(ma_context* pContext)
43393 | {
43394 |     if (pContext == NULL) {
43395 |         return MA_INVALID_ARGS;
43396 |     }
43397 |
43398 |     if (pContext->callbacks.onContextUninit != NULL) {
43399 |         pContext->callbacks.onContextUninit(pContext);
43400 |     }
43401 |
43402 |     ma_mutex_uninit(&pContext->deviceEnumLock);
43403 |     ma_mutex_uninit(&pContext->deviceInfoLock);
43404 |     ma_free(pContext->pDeviceInfos, &pContext->allocationCallbacks);
43405 |     ma_context_uninit_backend_apis(pContext);
43406 |
43407 |     if (pContext->pLog == &pContext->log) {
43408 |         ma_log_uninit(&pContext->log);
43409 |     }
43410 |
43411 |     return MA_SUCCESS;
43412 | }
43413 |
43414 | MA_API size_t ma_context_sizeof(void)
43415 | {
43416 |     return sizeof(ma_context);

```

```

43417 | }
43418 |
43419 |
43420 | MA_API ma_log* ma_context_get_log(ma_context* pContext)
43421 | {
43422 |     if (pContext == NULL) {
43423 |         return NULL;
43424 |     }
43425 |
43426 |     return pContext->pLog;
43427 | }
43428 |
43429 |
43430 | MA_API ma_result ma_context_enumerate_devices(ma_context* pContext, ma_enum_devices_callback_proc
> callback, void* pUserData)
43431 | {
43432 |     ma_result result;
43433 |
43434 |     if (pContext == NULL || callback == NULL) {
43435 |         return MA_INVALID_ARGS;
43436 |     }
43437 |
43438 |     if (pContext->callbacks.onContextEnumerateDevices == NULL) {
43439 |         return MA_INVALID_OPERATION;
43440 |     }
43441 |
43442 |     ma_mutex_lock(&pContext->deviceEnumLock);
43443 |     {
43444 |         result = pContext->callbacks.onContextEnumerateDevices(pContext, callback, pUserData);
43445 |     }
43446 |     ma_mutex_unlock(&pContext->deviceEnumLock);
43447 |
43448 |     return result;
43449 | }
43450 |
43451 |
43452 | static ma_bool32 ma_context_get_devices__enum_callback(ma_context* pContext, ma_device_type
> deviceType, const ma_device_info* pInfo, void* pUserData)
43453 | {
43454 |     /*
43455 |     We need to insert the device info into our main internal buffer. Where it goes depends on the
> device type. If it's a capture device
43456 |     it's just appended to the end. If it's a playback device it's inserted just before the first
> capture device.
43457 |     */
43458 |
43459 |     /*
43460 |     First make sure we have room. Since the number of devices we add to the list is usually
> relatively small I've decided to use a
43461 |     simple fixed size increment for buffer expansion.
43462 |     */
43463 |     const ma_uint32 bufferExpansionCount = 2;
43464 |     const ma_uint32 totalDeviceInfoCount = pContext->playbackDeviceInfoCount +
> pContext->captureDeviceInfoCount;
43465 |
43466 |     if (totalDeviceInfoCount >= pContext->deviceInfoCapacity) {
43467 |         ma_uint32 newCapacity = pContext->deviceInfoCapacity + bufferExpansionCount;
43468 |         ma_device_info* pNewInfos = (ma_device_info*)ma_realloc(pContext->pDeviceInfos,
> sizeof(*pContext->pDeviceInfos)*newCapacity, &pContext->allocationCallbacks);
43469 |         if (pNewInfos == NULL) {
43470 |             return MA_FALSE; /* Out of memory. */
43471 |         }
43472 |
43473 |         pContext->pDeviceInfos = pNewInfos;
43474 |         pContext->deviceInfoCapacity = newCapacity;
43475 |     }
43476 |
43477 |     if (deviceType == ma_device_type_playback) {
43478 |         /* Playback. Insert just before the first capture device. */
43479 |
43480 |         /* The first thing to do is move all of the capture devices down a slot. */
43481 |         ma_uint32 iFirstCaptureDevice = pContext->playbackDeviceInfoCount;
43482 |         size_t iCaptureDevice;
43483 |         for (iCaptureDevice = totalDeviceInfoCount; iCaptureDevice > iFirstCaptureDevice;
> --iCaptureDevice) {
43484 |             pContext->pDeviceInfos[iCaptureDevice] = pContext->pDeviceInfos[iCaptureDevice-1];
43485 |         }
43486 |
43487 |         /* Now just insert where the first capture device was before moving it down a slot. */

```



```

43488 |         pContext->pDeviceInfos[iFirstCaptureDevice] = *pInfo;
43489 |         pContext->playbackDeviceInfoCount += 1;
43490 |     } else {
43491 |         /* Capture. Insert at the end. */
43492 |         pContext->pDeviceInfos[totalDeviceInfoCount] = *pInfo;
43493 |         pContext->captureDeviceInfoCount += 1;
43494 |     }
43495 |
43496 |     (void)pUserData;
43497 |     return MA_TRUE;
43498 | }
43499 |
43500 | MA_API ma_result ma_context_get_devices(ma_context* pContext, ma_device_info**
> ppPlaybackDeviceInfos, ma_uint32* pPlaybackDeviceCount, ma_device_info** ppCaptureDeviceInfos,
> ma_uint32* pCaptureDeviceCount)
43501 | {
43502 |     ma_result result;
43503 |
43504 |     /* Safety. */
43505 |     if (ppPlaybackDeviceInfos != NULL) *ppPlaybackDeviceInfos = NULL;
43506 |     if (pPlaybackDeviceCount != NULL) *pPlaybackDeviceCount = 0;
43507 |     if (ppCaptureDeviceInfos != NULL) *ppCaptureDeviceInfos = NULL;
43508 |     if (pCaptureDeviceCount != NULL) *pCaptureDeviceCount = 0;
43509 |
43510 |     if (pContext == NULL) {
43511 |         return MA_INVALID_ARGS;
43512 |     }
43513 |
43514 |     if (pContext->callbacks.onContextEnumerateDevices == NULL) {
43515 |         return MA_INVALID_OPERATION;
43516 |     }
43517 |
43518 |     /* Note that we don't use ma_context_enumerate_devices() here because we want to do locking at a
> higher level. */
43519 |     ma_mutex_lock(&pContext->deviceEnumLock);
43520 |     {
43521 |         /* Reset everything first. */
43522 |         pContext->playbackDeviceInfoCount = 0;
43523 |         pContext->captureDeviceInfoCount = 0;
43524 |
43525 |         /* Now enumerate over available devices. */
43526 |         result = pContext->callbacks.onContextEnumerateDevices(pContext,
> ma_context_get_devices__enum_callback, NULL);
43527 |         if (result == MA_SUCCESS) {
43528 |             /* Playback devices. */
43529 |             if (ppPlaybackDeviceInfos != NULL) {
43530 |                 *ppPlaybackDeviceInfos = pContext->pDeviceInfos;
43531 |             }
43532 |             if (pPlaybackDeviceCount != NULL) {
43533 |                 *pPlaybackDeviceCount = pContext->playbackDeviceInfoCount;
43534 |             }
43535 |
43536 |             /* Capture devices. */
43537 |             if (ppCaptureDeviceInfos != NULL) {
43538 |                 *ppCaptureDeviceInfos = pContext->pDeviceInfos;
43539 |                 /* Capture devices come after playback devices. */
43540 |                 if (pContext->playbackDeviceInfoCount > 0) {
43541 |                     /* Conditional, because NULL+0 is undefined behavior. */
43542 |                     *ppCaptureDeviceInfos += pContext->playbackDeviceInfoCount;
43543 |                 }
43544 |             }
43545 |             if (pCaptureDeviceCount != NULL) {
43546 |                 *pCaptureDeviceCount = pContext->captureDeviceInfoCount;
43547 |             }
43548 |         }
43549 |     }
43550 |     ma_mutex_unlock(&pContext->deviceEnumLock);
43551 |
43552 |     return result;
43553 | }
43554 |
43555 | MA_API ma_result ma_context_get_device_info(ma_context* pContext, ma_device_type deviceType, const
> ma_device_id* pDeviceID, ma_device_info* pDeviceInfo)
43556 | {
43557 |     ma_result result;
43558 |     ma_device_info deviceInfo;
43559 |
43560 |     /* NOTE: Do not clear pDeviceInfo on entry. The reason is the pDeviceID may actually point to
> pDeviceInfo->id which will break things. */

```

```

43561 |     if (pContext == NULL || pDeviceInfo == NULL) {
43562 |         return MA_INVALID_ARGS;
43563 |     }
43564 |
43565 |     MA_ZERO_OBJECT(&deviceInfo);
43566 |
43567 |     /* Help the backend out by copying over the device ID if we have one. */
43568 |     if (pDeviceID != NULL) {
43569 |         MA_COPY_MEMORY(&deviceInfo.id, pDeviceID, sizeof(*pDeviceID));
43570 |     }
43571 |
43572 |     if (pContext->callbacks.onContextGetDeviceInfo == NULL) {
43573 |         return MA_INVALID_OPERATION;
43574 |     }
43575 |
43576 |     ma_mutex_lock(&pContext->deviceInfoLock);
43577 |     {
43578 |         result = pContext->callbacks.onContextGetDeviceInfo(pContext, deviceType, pDeviceID,
> &deviceInfo);
43579 |     }
43580 |     ma_mutex_unlock(&pContext->deviceInfoLock);
43581 |
43582 |     *pDeviceInfo = deviceInfo;
43583 |     return result;
43584 | }
43585 |
43586 | MA_API ma_bool32 ma_context_is_loopback_supported(ma_context* pContext)
43587 | {
43588 |     if (pContext == NULL) {
43589 |         return MA_FALSE;
43590 |     }
43591 |
43592 |     return ma_is_loopback_supported(pContext->backend);
43593 | }
43594 |
43595 |
43596 | MA_API ma_device_config ma_device_config_init(ma_device_type deviceType)
43597 | {
43598 |     ma_device_config config;
43599 |     MA_ZERO_OBJECT(&config);
43600 |     config.deviceType = deviceType;
43601 |     config.resampling = ma_resampler_config_init(ma_format_unknown, 0, 0, 0,
> ma_resample_algorithm_linear); /* Format/channels/rate don't matter here. */
43602 |
43603 |     return config;
43604 | }
43605 |
43606 | MA_API ma_result ma_device_init(ma_context* pContext, const ma_device_config* pConfig, ma_device*
> pDevice)
43607 | {
43608 |     ma_result result;
43609 |     ma_device_descriptor descriptorPlayback;
43610 |     ma_device_descriptor descriptorCapture;
43611 |
43612 |     /* The context can be null, in which case we self-manage it. */
43613 |     if (pContext == NULL) {
43614 |         return ma_device_init_ex(NULL, 0, NULL, pConfig, pDevice);
43615 |     }
43616 |
43617 |     if (pDevice == NULL) {
43618 |         return MA_INVALID_ARGS;
43619 |     }
43620 |
43621 |     MA_ZERO_OBJECT(pDevice);
43622 |
43623 |     if (pConfig == NULL) {
43624 |         return MA_INVALID_ARGS;
43625 |     }
43626 |
43627 |     /* Check that we have our callbacks defined. */
43628 |     if (pContext->callbacks.onDeviceInit == NULL) {
43629 |         return MA_INVALID_OPERATION;
43630 |     }
43631 |
43632 |     /* Basic config validation. */
43633 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex) {
43634 |         if (pConfig->capture.channels > MA_MAX_CHANNELS) {
43635 |             return MA_INVALID_ARGS;

```

```

43636 |     }
43637 |
43638 |     if (!ma_is_channel_map_valid(pConfig->capture.pChannelMap, pConfig->capture.channels)) {
43639 |         return MA_INVALID_ARGS;
43640 |     }
43641 | }
43642 |
43643 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex || pConfig->deviceType == ma_device_type_loopback) {
43644 |         if (pConfig->playback.channels > MA_MAX_CHANNELS) {
43645 |             return MA_INVALID_ARGS;
43646 |         }
43647 |
43648 |         if (!ma_is_channel_map_valid(pConfig->playback.pChannelMap, pConfig->playback.channels)) {
43649 |             return MA_INVALID_ARGS;
43650 |         }
43651 |     }
43652 |
43653 |     pDevice->pContext = pContext;
43654 |
43655 |     /* Set the user data and log callback ASAP to ensure it is available for the entire
> initialization process. */
43656 |     pDevice->pUserData      = pConfig->pUserData;
43657 |     pDevice->onData         = pConfig->dataCallback;
43658 |     pDevice->onNotification = pConfig->notificationCallback;
43659 |     pDevice->onStop         = pConfig->stopCallback;
43660 |
43661 |     if (pConfig->playback.pDeviceID != NULL) {
43662 |         MA_COPY_MEMORY(&pDevice->playback.id, pConfig->playback.pDeviceID,
> sizeof(pDevice->playback.id));
43663 |         pDevice->playback.pID = &pDevice->playback.id;
43664 |     } else {
43665 |         pDevice->playback.pID = NULL;
43666 |     }
43667 |
43668 |     if (pConfig->capture.pDeviceID != NULL) {
43669 |         MA_COPY_MEMORY(&pDevice->capture.id, pConfig->capture.pDeviceID,
> sizeof(pDevice->capture.id));
43670 |         pDevice->capture.pID = &pDevice->capture.id;
43671 |     } else {
43672 |         pDevice->capture.pID = NULL;
43673 |     }
43674 |
43675 |     pDevice->noPreSilencedOutputBuffer = pConfig->noPreSilencedOutputBuffer;
43676 |     pDevice->noClip                     = pConfig->noClip;
43677 |     pDevice->noDisableDenormals         = pConfig->noDisableDenormals;
43678 |     pDevice->noFixedSizedCallback       = pConfig->noFixedSizedCallback;
43679 |     ma_atomic_float_set(&pDevice->masterVolumeFactor, 1);
43680 |
43681 |     pDevice->type                     = pConfig->deviceType;
43682 |     pDevice->sampleRate               = pConfig->sampleRate;
43683 |     pDevice->resampling.algorithm      = pConfig->resampling.algorithm;
43684 |     pDevice->resampling.linear.lpfOrder = pConfig->resampling.linear.lpfOrder;
43685 |     pDevice->resampling.pBackendVTable = pConfig->resampling.pBackendVTable;
43686 |     pDevice->resampling.pBackendUserData = pConfig->resampling.pBackendUserData;
43687 |
43688 |     pDevice->capture.shareMode         = pConfig->capture.shareMode;
43689 |     pDevice->capture.format             = pConfig->capture.format;
43690 |     pDevice->capture.channels           = pConfig->capture.channels;
43691 |     ma_channel_map_copy_or_default(pDevice->capture.channelMap,
> ma_countof(pDevice->capture.channelMap), pConfig->capture.pChannelMap, pConfig->capture.channels);
43692 |     pDevice->capture.channelMixMode     = pConfig->capture.channelMixMode;
43693 |     pDevice->capture.calculateLFEFromSpatialChannels =
> pConfig->capture.calculateLFEFromSpatialChannels;
43694 |
43695 |     pDevice->playback.shareMode         = pConfig->playback.shareMode;
43696 |     pDevice->playback.format             = pConfig->playback.format;
43697 |     pDevice->playback.channels           = pConfig->playback.channels;
43698 |     ma_channel_map_copy_or_default(pDevice->playback.channelMap,
> ma_countof(pDevice->playback.channelMap), pConfig->playback.pChannelMap,
> pConfig->playback.channels);
43699 |     pDevice->playback.channelMixMode     = pConfig->playback.channelMixMode;
43700 |     pDevice->playback.calculateLFEFromSpatialChannels =
> pConfig->playback.calculateLFEFromSpatialChannels;
43701 |
43702 |     result = ma_mutex_init(&pDevice->startStopLock);
43703 |     if (result != MA_SUCCESS) {
43704 |         return result;
43705 |     }

```

```

43706 |
43707 |     /*
43708 |     > When the device is started, the worker thread is the one that does the actual startup of the
43709 |     > backend device. We
43710 |     > use a semaphore to wait for the background thread to finish the work. The same applies for
43711 |     > stopping the device.
43712 |     Each of these semaphores is released internally by the worker thread when the work is completed.
43713 |     > The start
43714 |     semaphore is also used to wake up the worker thread.
43715 |     */
43716 |     result = ma_event_init(&pDevice->wakeupEvent);
43717 |     if (result != MA_SUCCESS) {
43718 |         ma_mutex_uninit(&pDevice->startStopLock);
43719 |         return result;
43720 |     }
43721 |     result = ma_event_init(&pDevice->startEvent);
43722 |     if (result != MA_SUCCESS) {
43723 |         ma_event_uninit(&pDevice->wakeupEvent);
43724 |         ma_mutex_uninit(&pDevice->startStopLock);
43725 |         return result;
43726 |     }
43727 |     result = ma_event_init(&pDevice->stopEvent);
43728 |     if (result != MA_SUCCESS) {
43729 |         ma_event_uninit(&pDevice->startEvent);
43730 |         ma_event_uninit(&pDevice->wakeupEvent);
43731 |         ma_mutex_uninit(&pDevice->startStopLock);
43732 |         return result;
43733 |     }
43734 |
43735 |
43736 |     MA_ZERO_OBJECT(&descriptorPlayback);
43737 |     descriptorPlayback.pDeviceID          = pConfig->playback.pDeviceID;
43738 |     descriptorPlayback.shareMode          = pConfig->playback.shareMode;
43739 |     descriptorPlayback.format              = pConfig->playback.format;
43740 |     descriptorPlayback.channels            = pConfig->playback.channels;
43741 |     descriptorPlayback.sampleRate          = pConfig->sampleRate;
43742 |     ma_channel_map_copy_or_default(descriptorPlayback.channelMap,
43743 | > ma_countof(descriptorPlayback.channelMap), pConfig->playback.pChannelMap,
43744 | > pConfig->playback.channels);
43745 |     descriptorPlayback.periodSizeInFrames = pConfig->periodSizeInFrames;
43746 |     descriptorPlayback.periodSizeInMilliseconds = pConfig->periodSizeInMilliseconds;
43747 |     descriptorPlayback.periodCount         = pConfig->periods;
43748 |
43749 |     if (descriptorPlayback.periodCount == 0) {
43750 |         descriptorPlayback.periodCount = MA_DEFAULT_PERIODS;
43751 |     }
43752 |
43753 |     MA_ZERO_OBJECT(&descriptorCapture);
43754 |     descriptorCapture.pDeviceID          = pConfig->capture.pDeviceID;
43755 |     descriptorCapture.shareMode          = pConfig->capture.shareMode;
43756 |     descriptorCapture.format              = pConfig->capture.format;
43757 |     descriptorCapture.channels            = pConfig->capture.channels;
43758 |     descriptorCapture.sampleRate          = pConfig->sampleRate;
43759 |     ma_channel_map_copy_or_default(descriptorCapture.channelMap,
43760 | > ma_countof(descriptorCapture.channelMap), pConfig->capture.pChannelMap, pConfig->capture.channels);
43761 |     descriptorCapture.periodSizeInFrames = pConfig->periodSizeInFrames;
43762 |     descriptorCapture.periodSizeInMilliseconds = pConfig->periodSizeInMilliseconds;
43763 |     descriptorCapture.periodCount         = pConfig->periods;
43764 |
43765 |     if (descriptorCapture.periodCount == 0) {
43766 |         descriptorCapture.periodCount = MA_DEFAULT_PERIODS;
43767 |     }
43768 |
43769 |     result = pContext->callbacks.onDeviceInit(pDevice, pConfig, &descriptorPlayback,
43770 | > &descriptorCapture);
43771 |     if (result != MA_SUCCESS) {
43772 |         ma_event_uninit(&pDevice->startEvent);
43773 |         ma_event_uninit(&pDevice->wakeupEvent);
43774 |         ma_mutex_uninit(&pDevice->startStopLock);
43775 |         return result;
43776 |     }
43777 |     #if 0
43778 |     /*

```

```

43778 |     On output the descriptors will contain the *actual* data format of the device. We need this to
> know how to convert the data between
43779 |     the requested format and the internal format.
43780 |     */
43781 |     if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex || pConfig->deviceType == ma_device_type_loopback) {
43782 |         if (!ma_device_descriptor_is_valid(&descriptorCapture)) {
43783 |             ma_device_uninit(pDevice);
43784 |             return MA_INVALID_ARGS;
43785 |         }
43786 |
43787 |         pDevice->capture.internalFormat          = descriptorCapture.format;
43788 |         pDevice->capture.internalChannels         = descriptorCapture.channels;
43789 |         pDevice->capture.internalSampleRate       = descriptorCapture.sampleRate;
43790 |         ma_channel_map_copy(pDevice->capture.internalChannelMap, descriptorCapture.channelMap,
> descriptorCapture.channels);
43791 |         pDevice->capture.internalPeriodSizeInFrames = descriptorCapture.periodSizeInFrames;
43792 |         pDevice->capture.internalPeriods           = descriptorCapture.periodCount;
43793 |
43794 |         if (pDevice->capture.internalPeriodSizeInFrames == 0) {
43795 |             pDevice->capture.internalPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_milliseconds(descriptorCapture.periodSizeInMilliseconds,
> descriptorCapture.sampleRate);
43796 |         }
43797 |     }
43798 |
43799 |     if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
43800 |         if (!ma_device_descriptor_is_valid(&descriptorPlayback)) {
43801 |             ma_device_uninit(pDevice);
43802 |             return MA_INVALID_ARGS;
43803 |         }
43804 |
43805 |         pDevice->playback.internalFormat          = descriptorPlayback.format;
43806 |         pDevice->playback.internalChannels         = descriptorPlayback.channels;
43807 |         pDevice->playback.internalSampleRate       = descriptorPlayback.sampleRate;
43808 |         ma_channel_map_copy(pDevice->playback.internalChannelMap, descriptorPlayback.channelMap,
> descriptorPlayback.channels);
43809 |         pDevice->playback.internalPeriodSizeInFrames = descriptorPlayback.periodSizeInFrames;
43810 |         pDevice->playback.internalPeriods           = descriptorPlayback.periodCount;
43811 |
43812 |         if (pDevice->playback.internalPeriodSizeInFrames == 0) {
43813 |             pDevice->playback.internalPeriodSizeInFrames =
> ma_calculate_buffer_size_in_frames_from_milliseconds(descriptorPlayback.periodSizeInMilliseconds,
> descriptorPlayback.sampleRate);
43814 |         }
43815 |     }
43816 |
43817 |     /*
43818 |     The name of the device can be retrieved from device info. This may be temporary and replaced
> with a `ma_device_get_info(pDevice, deviceType)` instead.
43820 |     For loopback devices, we need to retrieve the name of the playback device.
43821 |     */
43822 |     {
43823 |         ma_device_info deviceInfo;
43824 |
43825 |         if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex || pConfig->deviceType == ma_device_type_loopback) {
43826 |             result = ma_device_get_info(pDevice, (pConfig->deviceType == ma_device_type_loopback) ?
> ma_device_type_playback : ma_device_type_capture, &deviceInfo);
43827 |             if (result == MA_SUCCESS) {
43828 |                 ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name), deviceInfo.name,
> (size_t)-1);
43829 |             } else {
43830 |                 /* We failed to retrieve the device info. Fall back to a default name. */
43831 |                 if (descriptorCapture.pDeviceID == NULL) {
43832 |                     ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name),
> MA_DEFAULT_CAPTURE_DEVICE_NAME, (size_t)-1);
43833 |                 } else {
43834 |                     ma_strncpy_s(pDevice->capture.name, sizeof(pDevice->capture.name), "Capture
> Device", (size_t)-1);
43835 |                 }
43836 |             }
43837 |         }
43838 |
43839 |         if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
43840 |             result = ma_device_get_info(pDevice, ma_device_type_playback, &deviceInfo);

```

```

43841 |         if (result == MA_SUCCESS) {
43842 |             ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name),
> deviceInfo.name, (size_t)-1);
43843 |         } else {
43844 |             /* We failed to retrieve the device info. Fall back to a default name. */
43845 |             if (descriptorPlayback.pDeviceID == NULL) {
43846 |                 ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name),
> MA_DEFAULT_PLAYBACK_DEVICE_NAME, (size_t)-1);
43847 |             } else {
43848 |                 ma_strncpy_s(pDevice->playback.name, sizeof(pDevice->playback.name), "Playback
> Device", (size_t)-1);
43849 |             }
43850 |         }
43851 |     }
43852 | }
43853 |
43854 |
43855 |     ma_device_post_init_setup(pDevice, pConfig->deviceType);
43856 | #endif
43857 |
43858 |     result = ma_device_post_init(pDevice, pConfig->deviceType, &descriptorPlayback,
> &descriptorCapture);
43859 |     if (result != MA_SUCCESS) {
43860 |         ma_device_uninit(pDevice);
43861 |         return result;
43862 |     }
43863 |
43864 |
43865 |     /*
43866 |     If we're using fixed sized callbacks we'll need to make use of an intermediary buffer. Needs to
43867 |     be done after post_init_setup() because we'll need access to the sample rate.
43868 |     */
43869 |     if (pConfig->noFixedSizedCallback == MA_FALSE) {
43870 |         /* We're using a fixed sized data callback so we'll need an intermediary buffer. */
43871 |         ma_uint32 intermediaryBufferCap = pConfig->periodSizeInFrames;
43872 |         if (intermediaryBufferCap == 0) {
43873 |             intermediaryBufferCap =
> ma_calculate_buffer_size_in_frames_from_milliseconds(pConfig->periodSizeInMilliseconds,
> pDevice->sampleRate);
43874 |         }
43875 |
43876 |         if (pConfig->deviceType == ma_device_type_capture || pConfig->deviceType ==
> ma_device_type_duplex || pConfig->deviceType == ma_device_type_loopback) {
43877 |             ma_uint32 intermediaryBufferSizeInBytes;
43878 |
43879 |             pDevice->capture.intermediaryBufferLen = 0;
43880 |             pDevice->capture.intermediaryBufferCap = intermediaryBufferCap;
43881 |             if (pDevice->capture.intermediaryBufferCap == 0) {
43882 |                 pDevice->capture.intermediaryBufferCap =
> pDevice->capture.internalPeriodSizeInFrames;
43883 |             }
43884 |
43885 |             intermediaryBufferSizeInBytes = pDevice->capture.intermediaryBufferCap *
> ma_get_bytes_per_frame(pDevice->capture.format, pDevice->capture.channels);
43886 |
43887 |             pDevice->capture.pIntermediaryBuffer = ma_malloc((size_t)intermediaryBufferSizeInBytes,
> &pContext->allocationCallbacks);
43888 |             if (pDevice->capture.pIntermediaryBuffer == NULL) {
43889 |                 ma_device_uninit(pDevice);
43890 |                 return MA_OUT_OF_MEMORY;
43891 |             }
43892 |
43893 |             /* Silence the buffer for safety. */
43894 |             ma_silence_pcm_frames(pDevice->capture.pIntermediaryBuffer,
> pDevice->capture.intermediaryBufferCap, pDevice->capture.format, pDevice->capture.channels);
43895 |             pDevice->capture.intermediaryBufferLen = pDevice->capture.intermediaryBufferCap;
43896 |         }
43897 |
43898 |         if (pConfig->deviceType == ma_device_type_playback || pConfig->deviceType ==
> ma_device_type_duplex) {
43899 |             ma_uint64 intermediaryBufferSizeInBytes;
43900 |
43901 |             pDevice->playback.intermediaryBufferLen = 0;
43902 |             if (pConfig->deviceType == ma_device_type_duplex) {
43903 |                 pDevice->playback.intermediaryBufferCap = pDevice->capture.intermediaryBufferCap;
> /* In duplex mode, make sure the intermediary buffer is always the same size as the capture side. */
43904 |             } else {
43905 |                 pDevice->playback.intermediaryBufferCap = intermediaryBufferCap;
43906 |                 if (pDevice->playback.intermediaryBufferCap == 0) {

```

```

43907 |         pDevice->playback.intermediaryBufferCap =
> pDevice->playback.internalPeriodSizeInFrames;
43908 |     }
43909 | }
43910 |
43911 |     intermediaryBufferSizeInBytes = pDevice->playback.intermediaryBufferCap *
> ma_get_bytes_per_frame(pDevice->playback.format, pDevice->playback.channels);
43912 |
43913 |     pDevice->playback.pIntermediaryBuffer = ma_malloc((size_t)intermediaryBufferSizeInBytes,
> &pContext->allocationCallbacks);
43914 |     if (pDevice->playback.pIntermediaryBuffer == NULL) {
43915 |         ma_device_uninit(pDevice);
43916 |         return MA_OUT_OF_MEMORY;
43917 |     }
43918 |
43919 |     /* Silence the buffer for safety. */
43920 |     ma_silence_pcm_frames(pDevice->playback.pIntermediaryBuffer,
> pDevice->playback.intermediaryBufferCap, pDevice->playback.format, pDevice->playback.channels);
43921 |     pDevice->playback.intermediaryBufferLen = 0;
43922 | }
43923 | } else {
43924 |     /* Not using a fixed sized data callback so no need for an intermediary buffer. */
43925 | }
43926 |
43927 |
43928 |     /* Some backends don't require the worker thread. */
43929 |     if (!ma_context_is_backend_asynchronous(pContext)) {
43930 |         /* The worker thread. */
43931 |         result = ma_thread_create(&pDevice->thread, pContext->threadPriority,
> pContext->threadStackSize, ma_worker_thread, pDevice, &pContext->allocationCallbacks);
43932 |         if (result != MA_SUCCESS) {
43933 |             ma_device_uninit(pDevice);
43934 |             return result;
43935 |         }
43936 |
43937 |         /* Wait for the worker thread to put the device into its stopped state for real. */
43938 |         ma_event_wait(&pDevice->stopEvent);
43939 |         MA_ASSERT(ma_device_get_state(pDevice) == ma_device_state_stopped);
43940 |     } else {
43941 |         /*
43942 |         If the backend is asynchronous and the device is duplex, we'll need an intermediary ring
> buffer. Note that this needs to be done
43943 |         after ma_device__post_init_setup().
43944 |         */
43945 |         if (ma_context_is_backend_asynchronous(pContext)) {
43946 |             if (pConfig->deviceType == ma_device_type_duplex) {
43947 |                 result = ma_duplex_rb_init(pDevice->capture.format, pDevice->capture.channels,
> pDevice->sampleRate, pDevice->capture.internalSampleRate,
> pDevice->capture.internalPeriodSizeInFrames, &pDevice->pContext->allocationCallbacks,
> &pDevice->duplexRB);
43948 |                 if (result != MA_SUCCESS) {
43949 |                     ma_device_uninit(pDevice);
43950 |                     return result;
43951 |                 }
43952 |             }
43953 |         }
43954 |
43955 |         ma_device__set_state(pDevice, ma_device_state_stopped);
43956 |     }
43957 |
43958 |     /* Log device information. */
43959 |     {
43960 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "[%s]\n",
> ma_get_backend_name(pDevice->pContext->backend));
43961 |         if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex ||
> pDevice->type == ma_device_type_loopback) {
43962 |             char name[MA_MAX_DEVICE_NAME_LENGTH + 1];
43963 |             ma_device_get_name(pDevice, ma_device_type_capture, name, sizeof(name), NULL);
43964 |
43965 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "  %s (%s)\n", name,
> "Capture");
43966 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Format:      %s ->
> %s\n", ma_get_format_name(pDevice->capture.internalFormat),
> ma_get_format_name(pDevice->capture.format));
43967 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channels:    %d ->
> %d\n", pDevice->capture.internalChannels, pDevice->capture.channels);
43968 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Sample Rate: %d ->
> %d\n", pDevice->capture.internalSampleRate, pDevice->sampleRate);
43969 |             ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Buffer Size: %d*%d

```



```

    > (%d)\n", pDevice->capture.internalPeriodSizeInFrames, pDevice->capture.internalPeriods,
    > (pDevice->capture.internalPeriodSizeInFrames * pDevice->capture.internalPeriods));
43970 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Conversion:\n");
43971 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Pre Format
    > Conversion: %s\n", pDevice->capture.converter.hasPreFormatConversion ? "YES" : "NO");
43972 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Post Format
    > Conversion: %s\n", pDevice->capture.converter.hasPostFormatConversion ? "YES" : "NO");
43973 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channel Routing:
    > %s\n", pDevice->capture.converter.hasChannelConverter ? "YES" : "NO");
43974 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Resampling:
    > %s\n", pDevice->capture.converter.hasResampler ? "YES" : "NO");
43975 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Passthrough:
    > %s\n", pDevice->capture.converter.isPassthrough ? "YES" : "NO");
43976 |     {
43977 |         char channelMapStr[1024];
43978 |         ma_channel_map_to_string(pDevice->capture.internalChannelMap,
    > pDevice->capture.internalChannels, channelMapStr, sizeof(channelMapStr));
43979 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channel Map In:
    > {%s}\n", channelMapStr);
43980 |
43981 |         ma_channel_map_to_string(pDevice->capture.channelMap, pDevice->capture.channels,
    > channelMapStr, sizeof(channelMapStr));
43982 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channel Map Out:
    > {%s}\n", channelMapStr);
43983 |     }
43984 | }
43985 | if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
43986 |     char name[MA_MAX_DEVICE_NAME_LENGTH + 1];
43987 |     ma_device_get_name(pDevice, ma_device_type_playback, name, sizeof(name), NULL);
43988 |
43989 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    %s (%s)\n", name,
    > "Playback");
43990 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Format:      %s ->
    > %s\n", ma_get_format_name(pDevice->playback.format),
    > ma_get_format_name(pDevice->playback.internalFormat));
43991 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channels:    %d ->
    > %d\n", pDevice->playback.channels, pDevice->playback.internalChannels);
43992 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Sample Rate: %d ->
    > %d\n", pDevice->sampleRate, pDevice->playback.internalSampleRate);
43993 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Buffer Size: %d*%d
    > (%d)\n", pDevice->playback.internalPeriodSizeInFrames, pDevice->playback.internalPeriods,
    > (pDevice->playback.internalPeriodSizeInFrames * pDevice->playback.internalPeriods));
43994 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Conversion:\n");
43995 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Pre Format
    > Conversion: %s\n", pDevice->playback.converter.hasPreFormatConversion ? "YES" : "NO");
43996 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Post Format
    > Conversion: %s\n", pDevice->playback.converter.hasPostFormatConversion ? "YES" : "NO");
43997 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channel Routing:
    > %s\n", pDevice->playback.converter.hasChannelConverter ? "YES" : "NO");
43998 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Resampling:
    > %s\n", pDevice->playback.converter.hasResampler ? "YES" : "NO");
43999 |     ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Passthrough:
    > %s\n", pDevice->playback.converter.isPassthrough ? "YES" : "NO");
44000 |     {
44001 |         char channelMapStr[1024];
44002 |         ma_channel_map_to_string(pDevice->playback.channelMap, pDevice->playback.channels,
    > channelMapStr, sizeof(channelMapStr));
44003 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channel Map In:
    > {%s}\n", channelMapStr);
44004 |
44005 |         ma_channel_map_to_string(pDevice->playback.internalChannelMap,
    > pDevice->playback.internalChannels, channelMapStr, sizeof(channelMapStr));
44006 |         ma_log_postf(ma_device_get_log(pDevice), MA_LOG_LEVEL_INFO, "    Channel Map Out:
    > {%s}\n", channelMapStr);
44007 |     }
44008 | }
44009 | }
44010 |
44011 | MA_ASSERT(ma_device_get_state(pDevice) == ma_device_state_stopped);
44012 | return MA_SUCCESS;
44013 | }
44014 |
44015 | MA_API ma_result ma_device_init_ex(const ma_backend backends[], ma_uint32 backendCount, const
    > ma_context_config* pContextConfig, const ma_device_config* pConfig, ma_device* pDevice)
44016 | {
44017 |     ma_result result;
44018 |     ma_context* pContext;
44019 |     ma_backend defaultBackends[ma_backend_null+1];
44020 |     ma_uint32 iBackend;

```

```

44021 |     ma_backend* pBackendsToIterate;
44022 |     ma_uint32 backendsToIterateCount;
44023 |     ma_allocation_callbacks allocationCallbacks;
44024 |
44025 |     if (pConfig == NULL) {
44026 |         return MA_INVALID_ARGS;
44027 |     }
44028 |
44029 |     if (pContextConfig != NULL) {
44030 |         result = ma_allocation_callbacks_init_copy(&allocationCallbacks,
> &pContextConfig->allocationCallbacks);
44031 |         if (result != MA_SUCCESS) {
44032 |             return result;
44033 |         }
44034 |     } else {
44035 |         allocationCallbacks = ma_allocation_callbacks_init_default();
44036 |     }
44037 |
44038 |     pContext = (ma_context*)ma_malloc(sizeof(*pContext), &allocationCallbacks);
44039 |     if (pContext == NULL) {
44040 |         return MA_OUT_OF_MEMORY;
44041 |     }
44042 |
44043 |     for (iBackend = 0; iBackend <= ma_backend_null; ++iBackend) {
44044 |         defaultBackends[iBackend] = (ma_backend)iBackend;
44045 |     }
44046 |
44047 |     pBackendsToIterate = (ma_backend*)backends;
44048 |     backendsToIterateCount = backendCount;
44049 |     if (pBackendsToIterate == NULL) {
44050 |         pBackendsToIterate = (ma_backend*)defaultBackends;
44051 |         backendsToIterateCount = ma_countof(defaultBackends);
44052 |     }
44053 |
44054 |     result = MA_NO_BACKEND;
44055 |
44056 |     for (iBackend = 0; iBackend < backendsToIterateCount; ++iBackend) {
44057 |         /*
44058 |         This is a hack for iOS. If the context config is null, there's a good chance the
44059 |         `ma_device_init(NULL, &deviceConfig, pDevice);` pattern is being used. In this
44060 |         case, set the session category based on the device type.
44061 |         */
44062 |         #if defined(MA_APPLE_MOBILE)
44063 |             ma_context_config contextConfig;
44064 |
44065 |             if (pContextConfig == NULL) {
44066 |                 contextConfig = ma_context_config_init();
44067 |                 switch (pConfig->deviceType) {
44068 |                     case ma_device_type_duplex: {
44069 |                         contextConfig.coreaudio.sessionCategory =
> ma_ios_session_category_play_and_record;
44070 |                     } break;
44071 |                     case ma_device_type_capture: {
44072 |                         contextConfig.coreaudio.sessionCategory = ma_ios_session_category_record;
44073 |                     } break;
44074 |                     case ma_device_type_playback:
44075 |                     default: {
44076 |                         contextConfig.coreaudio.sessionCategory = ma_ios_session_category_playback;
44077 |                     } break;
44078 |                 }
44079 |
44080 |                 pContextConfig = &contextConfig;
44081 |             }
44082 |         #endif
44083 |
44084 |         result = ma_context_init(&pBackendsToIterate[iBackend], 1, pContextConfig, pContext);
44085 |         if (result == MA_SUCCESS) {
44086 |             result = ma_device_init(pContext, pConfig, pDevice);
44087 |             if (result == MA_SUCCESS) {
44088 |                 break; /* Success. */
44089 |             } else {
44090 |                 ma_context_uninit(pContext); /* Failure. */
44091 |             }
44092 |         }
44093 |     }
44094 |
44095 |     if (result != MA_SUCCESS) {
44096 |         ma_free(pContext, &allocationCallbacks);
44097 |         return result;

```

```

44098 |     }
44099 |
44100 |     pDevice->isOwnerOfContext = MA_TRUE;
44101 |     return result;
44102 | }
44103 |
44104 | MA_API void ma_device_uninit(ma_device* pDevice)
44105 | {
44106 |     if (!ma_device__is_initialized(pDevice)) {
44107 |         return;
44108 |     }
44109 |
44110 |     /*
44111 |     It's possible for the miniaudio side of the device and the backend to not be in sync due to
44112 |     system-level situations such as the computer being put into sleep mode and the backend not
44113 |     notifying miniaudio of the fact the device has stopped. It's possible for this to result in a
44114 |     deadlock due to miniaudio thinking the device is in a running state, when in fact it's not
44115 |     running at all. For this reason I am no longer explicitly stopping the device. I don't think
44116 |     this should affect anyone in practice since uninitializing the backend will naturally stop the
44117 |     device anyway.
44118 |     */
44119 |     #if 0
44120 |     {
44121 |         /* Make sure the device is stopped first. The backends will probably handle this naturally,
> but I like to do it explicitly for my own sanity. */
44122 |         if (ma_device_is_started(pDevice)) {
44123 |             ma_device_stop(pDevice);
44124 |         }
44125 |     }
44126 |     #endif
44127 |
44128 |     /* Putting the device into an uninitialized state will make the worker thread return. */
44129 |     ma_device__set_state(pDevice, ma_device_state_uninitialized);
44130 |
44131 |     /* Wake up the worker thread and wait for it to properly terminate. */
44132 |     if (!ma_context_is_backend_asynchronous(pDevice->pContext)) {
44133 |         ma_event_signal(&pDevice->wakeupEvent);
44134 |         ma_thread_wait(&pDevice->thread);
44135 |     }
44136 |
44137 |     if (pDevice->pContext->callbacks.onDeviceUninit != NULL) {
44138 |         pDevice->pContext->callbacks.onDeviceUninit(pDevice);
44139 |     }
44140 |
44141 |
44142 |     ma_event_uninit(&pDevice->stopEvent);
44143 |     ma_event_uninit(&pDevice->startEvent);
44144 |     ma_event_uninit(&pDevice->wakeupEvent);
44145 |     ma_mutex_uninit(&pDevice->startStopLock);
44146 |
44147 |     if (ma_context_is_backend_asynchronous(pDevice->pContext)) {
44148 |         if (pDevice->type == ma_device_type_duplex) {
44149 |             ma_duplex_rb_uninit(&pDevice->duplexRB);
44150 |         }
44151 |     }
44152 |
44153 |     if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_duplex ||
> pDevice->type == ma_device_type_loopback) {
44154 |         ma_data_converter_uninit(&pDevice->capture.converter,
> &pDevice->pContext->allocationCallbacks);
44155 |     }
44156 |     if (pDevice->type == ma_device_type_playback || pDevice->type == ma_device_type_duplex) {
44157 |         ma_data_converter_uninit(&pDevice->playback.converter,
> &pDevice->pContext->allocationCallbacks);
44158 |     }
44159 |
44160 |     if (pDevice->playback.pInputCache != NULL) {
44161 |         ma_free(pDevice->playback.pInputCache, &pDevice->pContext->allocationCallbacks);
44162 |     }
44163 |
44164 |     if (pDevice->capture.pIntermediaryBuffer != NULL) {
44165 |         ma_free(pDevice->capture.pIntermediaryBuffer, &pDevice->pContext->allocationCallbacks);
44166 |     }
44167 |     if (pDevice->playback.pIntermediaryBuffer != NULL) {
44168 |         ma_free(pDevice->playback.pIntermediaryBuffer, &pDevice->pContext->allocationCallbacks);
44169 |     }
44170 |
44171 |     if (pDevice->isOwnerOfContext) {
44172 |         ma_allocation_callbacks allocationCallbacks = pDevice->pContext->allocationCallbacks;

```

```

44173 |
44174 |         ma_context_uninit(pDevice->pContext);
44175 |         ma_free(pDevice->pContext, &allocationCallbacks);
44176 |     }
44177 |
44178 |     MA_ZERO_OBJECT(pDevice);
44179 | }
44180 |
44181 | MA_API ma_context* ma_device_get_context(ma_device* pDevice)
44182 | {
44183 |     if (pDevice == NULL) {
44184 |         return NULL;
44185 |     }
44186 |
44187 |     return pDevice->pContext;
44188 | }
44189 |
44190 | MA_API ma_log* ma_device_get_log(ma_device* pDevice)
44191 | {
44192 |     return ma_context_get_log(ma_device_get_context(pDevice));
44193 | }
44194 |
44195 | MA_API ma_result ma_device_get_info(ma_device* pDevice, ma_device_type type, ma_device_info*
> pDeviceInfo)
44196 | {
44197 |     if (pDeviceInfo == NULL) {
44198 |         return MA_INVALID_ARGS;
44199 |     }
44200 |
44201 |     MA_ZERO_OBJECT(pDeviceInfo);
44202 |
44203 |     if (pDevice == NULL) {
44204 |         return MA_INVALID_ARGS;
44205 |     }
44206 |
44207 |     /* If the onDeviceGetInfo() callback is set, use that. Otherwise we'll fall back to
> ma_context_get_device_info(). */
44208 |     if (pDevice->pContext->callbacks.onDeviceGetInfo != NULL) {
44209 |         return pDevice->pContext->callbacks.onDeviceGetInfo(pDevice, type, pDeviceInfo);
44210 |     }
44211 |
44212 |     /* Getting here means onDeviceGetInfo is not implemented so we need to fall back to an
> alternative. */
44213 |     if (type == ma_device_type_playback) {
44214 |         return ma_context_get_device_info(pDevice->pContext, type, pDevice->playback.pID,
> pDeviceInfo);
44215 |     } else {
44216 |         /*
44217 |         Here we're getting the capture side, which is the branch we'll be entering for a loopback
44218 |         device, since loopback is capturing. However, if the device is using the default device ID,
44219 |         it won't get the correct information because it'll think we're asking for the default
44220 |         capture device, where in fact for loopback we want the default *playback* device. We'll do
44221 |         a bit of a hack here to make sure we get the correct info.
44222 |         */
44223 |         if (pDevice->type == ma_device_type_loopback && pDevice->capture.pID == NULL) {
44224 |             type = ma_device_type_playback;
44225 |         }
44226 |
44227 |         return ma_context_get_device_info(pDevice->pContext, type, pDevice->capture.pID,
> pDeviceInfo);
44228 |     }
44229 | }
44230 |
44231 | MA_API ma_result ma_device_get_name(ma_device* pDevice, ma_device_type type, char* pName, size_t
> nameCap, size_t* pLengthNotIncludingNullTerminator)
44232 | {
44233 |     ma_result result;
44234 |     ma_device_info deviceInfo;
44235 |
44236 |     if (pLengthNotIncludingNullTerminator != NULL) {
44237 |         *pLengthNotIncludingNullTerminator = 0;
44238 |     }
44239 |
44240 |     if (pName != NULL && nameCap > 0) {
44241 |         pName[0] = '\0';
44242 |     }
44243 |
44244 |     result = ma_device_get_info(pDevice, type, &deviceInfo);
44245 |     if (result != MA_SUCCESS) {

```

```

44246 |     return result;
44247 | }
44248 |
44249 | if (pName != NULL) {
44250 |     ma_strncpy_s(pName, nameCap, deviceInfo.name, (size_t)-1);
44251 |
44252 |     /*
44253 |     For safety, make sure the length is based on the truncated output string rather than the
44254 |     source. Otherwise the caller might assume the output buffer contains more content than it
44255 |     actually does.
44256 |     */
44257 |     if (pLengthNotIncludingNullTerminator != NULL) {
44258 |         *pLengthNotIncludingNullTerminator = strlen(pName);
44259 |     }
44260 | } else {
44261 |     /* Name not specified. Just report the length of the source string. */
44262 |     if (pLengthNotIncludingNullTerminator != NULL) {
44263 |         *pLengthNotIncludingNullTerminator = strlen(deviceInfo.name);
44264 |     }
44265 | }
44266 |
44267 | return MA_SUCCESS;
44268 | }
44269 |
44270 | MA_API ma_result ma_device_start(ma_device* pDevice)
44271 | {
44272 |     ma_result result;
44273 |
44274 |     if (pDevice == NULL) {
44275 |         return MA_INVALID_ARGS;
44276 |     }
44277 |
44278 |     if (ma_device_get_state(pDevice) == ma_device_state_uninitialized) {
44279 |         return MA_INVALID_OPERATION; /* Not initialized. */
44280 |     }
44281 |
44282 |     if (ma_device_get_state(pDevice) == ma_device_state_started) {
44283 |         return MA_SUCCESS; /* Already started. */
44284 |     }
44285 |
44286 |     ma_mutex_lock(&pDevice->startStopLock);
44287 |     {
44288 |         /*
44289 |         We need to check again if the device is in a started state because it's possible for one
44290 |         > thread to have started the device
44291 |         while another was waiting on the mutex.
44292 |         */
44293 |         if (ma_device_get_state(pDevice) == ma_device_state_started) {
44294 |             ma_mutex_unlock(&pDevice->startStopLock);
44295 |             return MA_SUCCESS; /* Already started. */
44296 |         }
44297 |
44298 |         /* Starting and stopping are wrapped in a mutex which means we can assert that the device is
44299 |         > in a stopped or paused state. */
44300 |         MA_ASSERT(ma_device_get_state(pDevice) == ma_device_state_stopped);
44301 |
44302 |         ma_device__set_state(pDevice, ma_device_state_starting);
44303 |
44304 |         /* Asynchronous backends need to be handled differently. */
44305 |         if (ma_context_is_backend_asynchronous(pDevice->pContext)) {
44306 |             if (pDevice->pContext->callbacks.onDeviceStart != NULL) {
44307 |                 result = pDevice->pContext->callbacks.onDeviceStart(pDevice);
44308 |             } else {
44309 |                 result = MA_INVALID_OPERATION;
44310 |             }
44311 |
44312 |             if (result == MA_SUCCESS) {
44313 |                 ma_device__set_state(pDevice, ma_device_state_started);
44314 |                 ma_device__on_notification_started(pDevice);
44315 |             }
44316 |         } else {
44317 |             /*
44318 |             Synchronous backends are started by signaling an event that's being waited on in the
44319 |             > worker thread. We first wake up the
44320 |             thread and then wait for the start event.
44321 |             */
44322 |             ma_event_signal(&pDevice->wakeupEvent);
44323 |
44324 |             /*

```

```

44322 |         Wait for the worker thread to finish starting the device. Note that the worker thread
> will be the one who puts the device
44323 |         into the started state. Don't call ma_device__set_state() here.
44324 |         */
44325 |         ma_event_wait(&pDevice->startEvent);
44326 |         result = pDevice->workResult;
44327 |     }
44328 |
44329 |     /* We changed the state from stopped to started, so if we failed, make sure we put the state
> back to stopped. */
44330 |     if (result != MA_SUCCESS) {
44331 |         ma_device__set_state(pDevice, ma_device_state_stopped);
44332 |     }
44333 | }
44334 |     ma_mutex_unlock(&pDevice->startStopLock);
44335 |
44336 |     return result;
44337 | }
44338 |
44339 | MA_API ma_result ma_device_stop(ma_device* pDevice)
44340 | {
44341 |     ma_result result;
44342 |
44343 |     if (pDevice == NULL) {
44344 |         return MA_INVALID_ARGS;
44345 |     }
44346 |
44347 |     if (ma_device_get_state(pDevice) == ma_device_state_uninitialized) {
44348 |         return MA_INVALID_OPERATION;    /* Not initialized. */
44349 |     }
44350 |
44351 |     if (ma_device_get_state(pDevice) == ma_device_state_stopped) {
44352 |         return MA_SUCCESS; /* Already stopped. */
44353 |     }
44354 |
44355 |     ma_mutex_lock(&pDevice->startStopLock);
44356 |     {
44357 |         /*
> thread to have stopped the device
44359 |         while another was waiting on the mutex.
44360 |         */
44361 |         if (ma_device_get_state(pDevice) == ma_device_state_stopped) {
44362 |             ma_mutex_unlock(&pDevice->startStopLock);
44363 |             return MA_SUCCESS; /* Already stopped. */
44364 |         }
44365 |
44366 |         /* Starting and stopping are wrapped in a mutex which means we can assert that the device is
> in a started or paused state. */
44367 |         MA_ASSERT(ma_device_get_state(pDevice) == ma_device_state_started);
44368 |
44369 |         ma_device__set_state(pDevice, ma_device_state_stopping);
44370 |
44371 |         /* Asynchronous backends need to be handled differently. */
44372 |         if (ma_context_is_backend_asynchronous(pDevice->pContext)) {
44373 |             /* Asynchronous backends must have a stop operation. */
44374 |             if (pDevice->pContext->callbacks.onDeviceStop != NULL) {
44375 |                 result = pDevice->pContext->callbacks.onDeviceStop(pDevice);
44376 |             } else {
44377 |                 result = MA_INVALID_OPERATION;
44378 |             }
44379 |
44380 |             ma_device__set_state(pDevice, ma_device_state_stopped);
44381 |         } else {
44382 |             /*
> call the stop callback here. If
44384 |             the backend is implementing its own audio thread loop we'll need to wake it up if
> required. Note that we need to make
44385 |             sure the state of the device is *not* playing right now, which it shouldn't be since we
> set it above. This is super
44386 |             important though, so I'm asserting it here as well for extra safety in case we
> accidentally change something later.
44387 |             */
44388 |             MA_ASSERT(ma_device_get_state(pDevice) != ma_device_state_started);
44389 |
44390 |             if (pDevice->pContext->callbacks.onDeviceDataLoopWakeup != NULL) {
44391 |                 pDevice->pContext->callbacks.onDeviceDataLoopWakeup(pDevice);
44392 |             }

```

```

44393 |
44394 |         /*
44395 |         We need to wait for the worker thread to become available for work before returning.
> Note that the worker thread will be
44396 |         the one who puts the device into the stopped state. Don't call ma_device__set_state()
> here.
44397 |         */
44398 |         ma_event_wait(&pDevice->stopEvent);
44399 |         result = MA_SUCCESS;
44400 |     }
44401 |
44402 |     /*
44403 |     This is a safety measure to ensure the internal buffer has been cleared so any leftover
44404 |     does not get played the next time the device starts. Ideally this should be drained by
44405 |     the backend first.
44406 |     */
44407 |     pDevice->playback.intermediaryBufferLen = 0;
44408 |     pDevice->playback.inputCacheConsumed = 0;
44409 |     pDevice->playback.inputCacheRemaining = 0;
44410 | }
44411 | ma_mutex_unlock(&pDevice->startStopLock);
44412 |
44413 | return result;
44414 | }
44415 |
44416 | MA_API ma_bool32 ma_device_is_started(const ma_device* pDevice)
44417 | {
44418 |     return ma_device_get_state(pDevice) == ma_device_state_started;
44419 | }
44420 |
44421 | MA_API ma_device_state ma_device_get_state(const ma_device* pDevice)
44422 | {
44423 |     if (pDevice == NULL) {
44424 |         return ma_device_state_uninitialized;
44425 |     }
44426 |
44427 |     return ma_atomic_device_state_get((ma_atomic_device_state*)&pDevice->state); /* Naughty cast
> to get rid of a const warning. */
44428 | }
44429 |
44430 | MA_API ma_result ma_device_set_master_volume(ma_device* pDevice, float volume)
44431 | {
44432 |     if (pDevice == NULL) {
44433 |         return MA_INVALID_ARGS;
44434 |     }
44435 |
44436 |     if (volume < 0.0f) {
44437 |         return MA_INVALID_ARGS;
44438 |     }
44439 |
44440 |     ma_atomic_float_set(&pDevice->masterVolumeFactor, volume);
44441 |
44442 |     return MA_SUCCESS;
44443 | }
44444 |
44445 | MA_API ma_result ma_device_get_master_volume(ma_device* pDevice, float* pVolume)
44446 | {
44447 |     if (pVolume == NULL) {
44448 |         return MA_INVALID_ARGS;
44449 |     }
44450 |
44451 |     if (pDevice == NULL) {
44452 |         *pVolume = 0;
44453 |         return MA_INVALID_ARGS;
44454 |     }
44455 |
44456 |     *pVolume = ma_atomic_float_get(&pDevice->masterVolumeFactor);
44457 |
44458 |     return MA_SUCCESS;
44459 | }
44460 |
44461 | MA_API ma_result ma_device_set_master_volume_db(ma_device* pDevice, float gainDB)
44462 | {
44463 |     if (gainDB > 0) {
44464 |         return MA_INVALID_ARGS;
44465 |     }
44466 |
44467 |     return ma_device_set_master_volume(pDevice, ma_volume_db_to_linear(gainDB));
44468 | }

```



```

44469 |
44470 | MA_API ma_result ma_device_get_master_volume_db(ma_device* pDevice, float* pGainDB)
44471 | {
44472 |     float factor;
44473 |     ma_result result;
44474 |
44475 |     if (pGainDB == NULL) {
44476 |         return MA_INVALID_ARGS;
44477 |     }
44478 |
44479 |     result = ma_device_get_master_volume(pDevice, &factor);
44480 |     if (result != MA_SUCCESS) {
44481 |         *pGainDB = 0;
44482 |         return result;
44483 |     }
44484 |
44485 |     *pGainDB = ma_volume_linear_to_db(factor);
44486 |
44487 |     return MA_SUCCESS;
44488 | }
44489 |
44490 |
44491 | MA_API ma_result ma_device_handle_backend_data_callback(ma_device* pDevice, void* pOutput, const
> void* pInput, ma_uint32 frameCount)
44492 | {
44493 |     if (pDevice == NULL) {
44494 |         return MA_INVALID_ARGS;
44495 |     }
44496 |
44497 |     if (pOutput == NULL && pInput == NULL) {
44498 |         return MA_INVALID_ARGS;
44499 |     }
44500 |
44501 |     /*
44502 |     > There is an assert deeper in the code that checks that frameCount > 0. Since this is a public
> facing
44503 |     API we'll need to check for that here. I've had reports that AAudio can sometimes post a frame
> count
44504 |     of 0.
44505 |     */
44506 |     if (frameCount == 0) {
44507 |         return MA_INVALID_ARGS;
44508 |     }
44509 |
44510 |     if (pDevice->type == ma_device_type_duplex) {
44511 |         if (pInput != NULL) {
44512 |             ma_device_handle_duplex_callback_capture(pDevice, frameCount, pInput,
> &pDevice->duplexRB.rb);
44513 |         }
44514 |
44515 |         if (pOutput != NULL) {
44516 |             ma_device_handle_duplex_callback_playback(pDevice, frameCount, pOutput,
> &pDevice->duplexRB.rb);
44517 |         }
44518 |     } else {
44519 |         if (pDevice->type == ma_device_type_capture || pDevice->type == ma_device_type_loopback) {
44520 |             if (pInput == NULL) {
44521 |                 return MA_INVALID_ARGS;
44522 |             }
44523 |
44524 |             ma_device__send_frames_to_client(pDevice, frameCount, pInput);
44525 |         }
44526 |
44527 |         if (pDevice->type == ma_device_type_playback) {
44528 |             if (pOutput == NULL) {
44529 |                 return MA_INVALID_ARGS;
44530 |             }
44531 |
44532 |             ma_device__read_frames_from_client(pDevice, frameCount, pOutput);
44533 |         }
44534 |     }
44535 |
44536 |     return MA_SUCCESS;
44537 | }
44538 |
44539 | MA_API ma_uint32 ma_calculate_buffer_size_in_frames_from_descriptor(const ma_device_descriptor*
> pDescriptor, ma_uint32 nativeSampleRate, ma_performance_profile performanceProfile)
44540 | {
44541 |     if (pDescriptor == NULL) {

```

```

44542 |         return 0;
44543 |     }
44544 |
44545 |     /*
44546 |     We must have a non-0 native sample rate, but some backends don't allow retrieval of this at the
44547 |     time when the size of the buffer needs to be determined. In this case we need to just take a
> best
44548 |     guess and move on. We'll try using the sample rate in pDescriptor first. If that's not set we'll
44549 |     just fall back to MA_DEFAULT_SAMPLE_RATE.
44550 |     */
44551 |     if (nativeSampleRate == 0) {
44552 |         nativeSampleRate = pDescriptor->sampleRate;
44553 |     }
44554 |     if (nativeSampleRate == 0) {
44555 |         nativeSampleRate = MA_DEFAULT_SAMPLE_RATE;
44556 |     }
44557 |
44558 |     MA_ASSERT(nativeSampleRate != 0);
44559 |
44560 |     if (pDescriptor->periodSizeInFrames == 0) {
44561 |         if (pDescriptor->periodSizeInMilliseconds == 0) {
44562 |             if (performanceProfile == ma_performance_profile_low_latency) {
44563 |                 return ma_calculate_buffer_size_in_frames_from_milliseconds(MA_DEFAULT_PERIOD_SIZE_I
> N_MILLISECONDS_LOW_LATENCY, nativeSampleRate);
44564 |             } else {
44565 |                 return ma_calculate_buffer_size_in_frames_from_milliseconds(MA_DEFAULT_PERIOD_SIZE_I
> N_MILLISECONDS_CONSERVATIVE, nativeSampleRate);
44566 |             }
44567 |         } else {
44568 |             return
> ma_calculate_buffer_size_in_frames_from_milliseconds(pDescriptor->periodSizeInMilliseconds,
> nativeSampleRate);
44569 |         }
44570 |     } else {
44571 |         return pDescriptor->periodSizeInFrames;
44572 |     }
44573 | }
44574 | #endif /* MA_NO_DEVICE_IO */
44575 |
44576 |
44577 | MA_API ma_uint32 ma_calculate_buffer_size_in_milliseconds_from_frames(ma_uint32 bufferSizeInFrames,
> ma_uint32 sampleRate)
44578 | {
44579 |     /* Prevent a division by zero. */
44580 |     if (sampleRate == 0) {
44581 |         return 0;
44582 |     }
44583 |
44584 |     return (bufferSizeInFrames*1000 + (sampleRate - 1)) / sampleRate;
44585 | }
44586 |
44587 | MA_API ma_uint32 ma_calculate_buffer_size_in_frames_from_milliseconds(ma_uint32
> bufferSizeInMilliseconds, ma_uint32 sampleRate)
44588 | {
44589 |     /* Prevent a division by zero. */
44590 |     if (sampleRate == 0) {
44591 |         return 0;
44592 |     }
44593 |
44594 |     return bufferSizeInMilliseconds*sampleRate / 1000;
44595 | }
44596 |
44597 | MA_API void ma_copy_pcm_frames(void* dst, const void* src, ma_uint64 frameCount, ma_format format,
> ma_uint32 channels)
44598 | {
44599 |     if (dst == src) {
44600 |         return; /* No-op. */
44601 |     }
44602 |
44603 |     ma_copy_memory_64(dst, src, frameCount * ma_get_bytes_per_frame(format, channels));
44604 | }
44605 |
44606 | MA_API void ma_silence_pcm_frames(void* p, ma_uint64 frameCount, ma_format format, ma_uint32
> channels)
44607 | {
44608 |     if (format == ma_format_u8) {
44609 |         ma_uint64 sampleCount = frameCount * channels;
44610 |         ma_uint64 iSample;
44611 |         for (iSample = 0; iSample < sampleCount; iSample += 1) {

```

```

44612 |         ((ma_uint8*)p)[iSample] = 128;
44613 |     }
44614 | } else {
44615 |     ma_zero_memory_64(p, frameCount * ma_get_bytes_per_frame(format, channels));
44616 | }
44617 | }
44618 |
44619 | MA_API void* ma_offset_pcm_frames_ptr(void* p, ma_uint64 offsetInFrames, ma_format format, ma_uint32
> channels)
44620 | {
44621 |     return ma_offset_ptr(p, offsetInFrames * ma_get_bytes_per_frame(format, channels));
44622 | }
44623 |
44624 | MA_API const void* ma_offset_pcm_frames_const_ptr(const void* p, ma_uint64 offsetInFrames, ma_format
> format, ma_uint32 channels)
44625 | {
44626 |     return ma_offset_ptr(p, offsetInFrames * ma_get_bytes_per_frame(format, channels));
44627 | }
44628 |
44629 |
44630 | MA_API void ma_clip_samples_u8(ma_uint8* pDst, const ma_int16* pSrc, ma_uint64 count)
44631 | {
44632 |     ma_uint64 iSample;
44633 |
44634 |     MA_ASSERT(pDst != NULL);
44635 |     MA_ASSERT(pSrc != NULL);
44636 |
44637 |     for (iSample = 0; iSample < count; iSample += 1) {
44638 |         pDst[iSample] = ma_clip_u8(pSrc[iSample]);
44639 |     }
44640 | }
44641 |
44642 | MA_API void ma_clip_samples_s16(ma_int16* pDst, const ma_int32* pSrc, ma_uint64 count)
44643 | {
44644 |     ma_uint64 iSample;
44645 |
44646 |     MA_ASSERT(pDst != NULL);
44647 |     MA_ASSERT(pSrc != NULL);
44648 |
44649 |     for (iSample = 0; iSample < count; iSample += 1) {
44650 |         pDst[iSample] = ma_clip_s16(pSrc[iSample]);
44651 |     }
44652 | }
44653 |
44654 | MA_API void ma_clip_samples_s24(ma_uint8* pDst, const ma_int64* pSrc, ma_uint64 count)
44655 | {
44656 |     ma_uint64 iSample;
44657 |
44658 |     MA_ASSERT(pDst != NULL);
44659 |     MA_ASSERT(pSrc != NULL);
44660 |
44661 |     for (iSample = 0; iSample < count; iSample += 1) {
44662 |         ma_int64 s = ma_clip_s24(pSrc[iSample]);
44663 |         pDst[iSample*3 + 0] = (ma_uint8)((s & 0x000000FF) >> 0);
44664 |         pDst[iSample*3 + 1] = (ma_uint8)((s & 0x0000FF00) >> 8);
44665 |         pDst[iSample*3 + 2] = (ma_uint8)((s & 0x00FF0000) >> 16);
44666 |     }
44667 | }
44668 |
44669 | MA_API void ma_clip_samples_s32(ma_int32* pDst, const ma_int64* pSrc, ma_uint64 count)
44670 | {
44671 |     ma_uint64 iSample;
44672 |
44673 |     MA_ASSERT(pDst != NULL);
44674 |     MA_ASSERT(pSrc != NULL);
44675 |
44676 |     for (iSample = 0; iSample < count; iSample += 1) {
44677 |         pDst[iSample] = ma_clip_s32(pSrc[iSample]);
44678 |     }
44679 | }
44680 |
44681 | MA_API void ma_clip_samples_f32(float* pDst, const float* pSrc, ma_uint64 count)
44682 | {
44683 |     ma_uint64 iSample;
44684 |
44685 |     MA_ASSERT(pDst != NULL);
44686 |     MA_ASSERT(pSrc != NULL);
44687 |
44688 |     for (iSample = 0; iSample < count; iSample += 1) {

```

```

44689 |         pDst[iSample] = ma_clip_f32(pSrc[iSample]);
44690 |     }
44691 | }
44692 |
44693 | MA_API void ma_clip_pcm_frames(void* pDst, const void* pSrc, ma_uint64 frameCount, ma_format format,
> ma_uint32 channels)
44694 | {
44695 |     ma_uint64 sampleCount;
44696 |
44697 |     MA_ASSERT(pDst != NULL);
44698 |     MA_ASSERT(pSrc != NULL);
44699 |
44700 |     sampleCount = frameCount * channels;
44701 |
44702 |     switch (format) {
44703 |         case ma_format_u8: ma_clip_samples_u8((ma_uint8*)pDst, (const ma_int16*)pSrc,
> sampleCount); break;
44704 |         case ma_format_s16: ma_clip_samples_s16((ma_int16*)pDst, (const ma_int32*)pSrc,
> sampleCount); break;
44705 |         case ma_format_s24: ma_clip_samples_s24((ma_uint8*)pDst, (const ma_int64*)pSrc,
> sampleCount); break;
44706 |         case ma_format_s32: ma_clip_samples_s32((ma_int32*)pDst, (const ma_int64*)pSrc,
> sampleCount); break;
44707 |         case ma_format_f32: ma_clip_samples_f32((float*)pDst, (const float*)pSrc,
> sampleCount); break;
44708 |
44709 |         /* Do nothing if we don't know the format. We're including these here to silence a compiler
> warning about enums not being handled by the switch. */
44710 |         case ma_format_unknown:
44711 |         case ma_format_count:
44712 |             break;
44713 |     }
44714 | }
44715 |
44716 |
44717 | MA_API void ma_copy_and_apply_volume_factor_u8(ma_uint8* pSamplesOut, const ma_uint8* pSamplesIn,
> ma_uint64 sampleCount, float factor)
44718 | {
44719 |     ma_uint64 iSample;
44720 |
44721 |     if (pSamplesOut == NULL || pSamplesIn == NULL) {
44722 |         return;
44723 |     }
44724 |
44725 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
44726 |         pSamplesOut[iSample] = (ma_uint8)(pSamplesIn[iSample] * factor);
44727 |     }
44728 | }
44729 |
44730 | MA_API void ma_copy_and_apply_volume_factor_s16(ma_int16* pSamplesOut, const ma_int16* pSamplesIn,
> ma_uint64 sampleCount, float factor)
44731 | {
44732 |     ma_uint64 iSample;
44733 |
44734 |     if (pSamplesOut == NULL || pSamplesIn == NULL) {
44735 |         return;
44736 |     }
44737 |
44738 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
44739 |         pSamplesOut[iSample] = (ma_int16)(pSamplesIn[iSample] * factor);
44740 |     }
44741 | }
44742 |
44743 | MA_API void ma_copy_and_apply_volume_factor_s24(void* pSamplesOut, const void* pSamplesIn, ma_uint64
> sampleCount, float factor)
44744 | {
44745 |     ma_uint64 iSample;
44746 |     ma_uint8* pSamplesOut8;
44747 |     ma_uint8* pSamplesIn8;
44748 |
44749 |     if (pSamplesOut == NULL || pSamplesIn == NULL) {
44750 |         return;
44751 |     }
44752 |
44753 |     pSamplesOut8 = (ma_uint8*)pSamplesOut;
44754 |     pSamplesIn8 = (ma_uint8*)pSamplesIn;
44755 |
44756 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
44757 |         ma_int32 sampleS32;

```

```

44758 |
44759 |         sampleS32 = (ma_int32)(((ma_uint32)(pSamplesIn[iSample*3+0]) << 8) |
> ((ma_uint32)(pSamplesIn[iSample*3+1]) << 16) | ((ma_uint32)(pSamplesIn[iSample*3+2])) << 24);
44760 |         sampleS32 = (ma_int32)(sampleS32 * factor);
44761 |
44762 |         pSamplesOut8[iSample*3+0] = (ma_uint8)(((ma_uint32)sampleS32 & 0x0000FF00) >> 8);
44763 |         pSamplesOut8[iSample*3+1] = (ma_uint8)(((ma_uint32)sampleS32 & 0x00FF0000) >> 16);
44764 |         pSamplesOut8[iSample*3+2] = (ma_uint8)(((ma_uint32)sampleS32 & 0xFF000000) >> 24);
44765 |     }
44766 | }
44767 |
44768 | MA_API void ma_copy_and_apply_volume_factor_s32(ma_int32* pSamplesOut, const ma_int32* pSamplesIn,
> ma_uint64 sampleCount, float factor)
44769 | {
44770 |     ma_uint64 iSample;
44771 |
44772 |     if (pSamplesOut == NULL || pSamplesIn == NULL) {
44773 |         return;
44774 |     }
44775 |
44776 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
44777 |         pSamplesOut[iSample] = (ma_int32)(pSamplesIn[iSample] * factor);
44778 |     }
44779 | }
44780 |
44781 | MA_API void ma_copy_and_apply_volume_factor_f32(float* pSamplesOut, const float* pSamplesIn,
> ma_uint64 sampleCount, float factor)
44782 | {
44783 |     ma_uint64 iSample;
44784 |
44785 |     if (pSamplesOut == NULL || pSamplesIn == NULL) {
44786 |         return;
44787 |     }
44788 |
44789 |     if (factor == 1) {
44790 |         if (pSamplesOut == pSamplesIn) {
44791 |             /* In place. No-op. */
44792 |         } else {
44793 |             /* Just a copy. */
44794 |             for (iSample = 0; iSample < sampleCount; iSample += 1) {
44795 |                 pSamplesOut[iSample] = pSamplesIn[iSample];
44796 |             }
44797 |         }
44798 |     } else {
44799 |         for (iSample = 0; iSample < sampleCount; iSample += 1) {
44800 |             pSamplesOut[iSample] = pSamplesIn[iSample] * factor;
44801 |         }
44802 |     }
44803 | }
44804 |
44805 | MA_API void ma_apply_volume_factor_u8(ma_uint8* pSamples, ma_uint64 sampleCount, float factor)
44806 | {
44807 |     ma_copy_and_apply_volume_factor_u8(pSamples, pSamples, sampleCount, factor);
44808 | }
44809 |
44810 | MA_API void ma_apply_volume_factor_s16(ma_int16* pSamples, ma_uint64 sampleCount, float factor)
44811 | {
44812 |     ma_copy_and_apply_volume_factor_s16(pSamples, pSamples, sampleCount, factor);
44813 | }
44814 |
44815 | MA_API void ma_apply_volume_factor_s24(void* pSamples, ma_uint64 sampleCount, float factor)
44816 | {
44817 |     ma_copy_and_apply_volume_factor_s24(pSamples, pSamples, sampleCount, factor);
44818 | }
44819 |
44820 | MA_API void ma_apply_volume_factor_s32(ma_int32* pSamples, ma_uint64 sampleCount, float factor)
44821 | {
44822 |     ma_copy_and_apply_volume_factor_s32(pSamples, pSamples, sampleCount, factor);
44823 | }
44824 |
44825 | MA_API void ma_apply_volume_factor_f32(float* pSamples, ma_uint64 sampleCount, float factor)
44826 | {
44827 |     ma_copy_and_apply_volume_factor_f32(pSamples, pSamples, sampleCount, factor);
44828 | }
44829 |
44830 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_u8(ma_uint8* pFramesOut, const ma_uint8*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor)
44831 | {
44832 |     ma_copy_and_apply_volume_factor_u8(pFramesOut, pFramesIn, frameCount*channels, factor);

```

```

44833 | }
44834 |
44835 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_s16(ma_int16* pFramesOut, const ma_int16*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor)
44836 | {
44837 |     ma_copy_and_apply_volume_factor_s16(pFramesOut, pFramesIn, frameCount*channels, factor);
44838 | }
44839 |
44840 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_s24(void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount, ma_uint32 channels, float factor)
44841 | {
44842 |     ma_copy_and_apply_volume_factor_s24(pFramesOut, pFramesIn, frameCount*channels, factor);
44843 | }
44844 |
44845 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_s32(ma_int32* pFramesOut, const ma_int32*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor)
44846 | {
44847 |     ma_copy_and_apply_volume_factor_s32(pFramesOut, pFramesIn, frameCount*channels, factor);
44848 | }
44849 |
44850 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames_f32(float* pFramesOut, const float*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float factor)
44851 | {
44852 |     ma_copy_and_apply_volume_factor_f32(pFramesOut, pFramesIn, frameCount*channels, factor);
44853 | }
44854 |
44855 | MA_API void ma_copy_and_apply_volume_factor_pcm_frames(void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount, ma_format format, ma_uint32 channels, float factor)
44856 | {
44857 |     switch (format)
44858 |     {
44859 |         case ma_format_u8: ma_copy_and_apply_volume_factor_pcm_frames_u8 ((ma_uint8*)pFramesOut, (const
> ma_uint8*)pFramesIn, frameCount, channels, factor); return;
44860 |         case ma_format_s16: ma_copy_and_apply_volume_factor_pcm_frames_s16((ma_int16*)pFramesOut, (const
> ma_int16*)pFramesIn, frameCount, channels, factor); return;
44861 |         case ma_format_s24: ma_copy_and_apply_volume_factor_pcm_frames_s24(
> pFramesOut,
> pFramesIn, frameCount, channels, factor); return;
44862 |         case ma_format_s32: ma_copy_and_apply_volume_factor_pcm_frames_s32((ma_int32*)pFramesOut, (const
> ma_int32*)pFramesIn, frameCount, channels, factor); return;
44863 |         case ma_format_f32: ma_copy_and_apply_volume_factor_pcm_frames_f32(
> (float*)pFramesOut,
> (const float*)pFramesIn, frameCount, channels, factor); return;
44864 |         default: return; /* Do nothing. */
44865 |     }
44866 | }
44867 |
44868 | MA_API void ma_apply_volume_factor_pcm_frames_u8(ma_uint8* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor)
44869 | {
44870 |     ma_copy_and_apply_volume_factor_pcm_frames_u8(pFrames, pFrames, frameCount, channels, factor);
44871 | }
44872 |
44873 | MA_API void ma_apply_volume_factor_pcm_frames_s16(ma_int16* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor)
44874 | {
44875 |     ma_copy_and_apply_volume_factor_pcm_frames_s16(pFrames, pFrames, frameCount, channels, factor);
44876 | }
44877 |
44878 | MA_API void ma_apply_volume_factor_pcm_frames_s24(void* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor)
44879 | {
44880 |     ma_copy_and_apply_volume_factor_pcm_frames_s24(pFrames, pFrames, frameCount, channels, factor);
44881 | }
44882 |
44883 | MA_API void ma_apply_volume_factor_pcm_frames_s32(ma_int32* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor)
44884 | {
44885 |     ma_copy_and_apply_volume_factor_pcm_frames_s32(pFrames, pFrames, frameCount, channels, factor);
44886 | }
44887 |
44888 | MA_API void ma_apply_volume_factor_pcm_frames_f32(float* pFrames, ma_uint64 frameCount, ma_uint32
> channels, float factor)
44889 | {
44890 |     ma_copy_and_apply_volume_factor_pcm_frames_f32(pFrames, pFrames, frameCount, channels, factor);
44891 | }
44892 |
44893 | MA_API void ma_apply_volume_factor_pcm_frames(void* pFramesOut, ma_uint64 frameCount, ma_format
> format, ma_uint32 channels, float factor)
44894 | {
44895 |     ma_copy_and_apply_volume_factor_pcm_frames(pFramesOut, pFramesOut, frameCount, format, channels,

```

```

> factor);
44896 | }
44897 |
44898 |
44899 | MA_API void ma_copy_and_apply_volume_factor_per_channel_f32(float* pFramesOut, const float*
> pFramesIn, ma_uint64 frameCount, ma_uint32 channels, float* pChannelGains)
44900 | {
44901 |     ma_uint64 iFrame;
44902 |
44903 |     if (channels == 2) {
44904 |         /* TODO: Do an optimized implementation for stereo and mono. Can do a SIMD optimized
> implementation as well. */
44905 |     }
44906 |
44907 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
44908 |         ma_uint32 iChannel;
44909 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
44910 |             pFramesOut[iFrame * channels + iChannel] = pFramesIn[iFrame * channels + iChannel] *
> pChannelGains[iChannel];
44911 |         }
44912 |     }
44913 | }
44914 |
44915 |
44916 |
44917 | static MA_INLINE ma_int16 ma_apply_volume_unclipped_u8(ma_int16 x, ma_int16 volume)
44918 | {
44919 |     return (ma_int16)(((ma_int32)x * (ma_int32)volume) >> 8);
44920 | }
44921 |
44922 | static MA_INLINE ma_int32 ma_apply_volume_unclipped_s16(ma_int32 x, ma_int16 volume)
44923 | {
44924 |     return (ma_int32)((x * volume) >> 8);
44925 | }
44926 |
44927 | static MA_INLINE ma_int64 ma_apply_volume_unclipped_s24(ma_int64 x, ma_int16 volume)
44928 | {
44929 |     return (ma_int64)((x * volume) >> 8);
44930 | }
44931 |
44932 | static MA_INLINE ma_int64 ma_apply_volume_unclipped_s32(ma_int64 x, ma_int16 volume)
44933 | {
44934 |     return (ma_int64)((x * volume) >> 8);
44935 | }
44936 |
44937 | static MA_INLINE float ma_apply_volume_unclipped_f32(float x, float volume)
44938 | {
44939 |     return x * volume;
44940 | }
44941 |
44942 |
44943 | MA_API void ma_copy_and_apply_volume_and_clip_samples_u8(ma_uint8* pDst, const ma_int16* pSrc,
> ma_uint64 count, float volume)
44944 | {
44945 |     ma_uint64 iSample;
44946 |     ma_int16 volumeFixed;
44947 |
44948 |     MA_ASSERT(pDst != NULL);
44949 |     MA_ASSERT(pSrc != NULL);
44950 |
44951 |     volumeFixed = ma_float_to_fixed_16(volume);
44952 |
44953 |     for (iSample = 0; iSample < count; iSample += 1) {
44954 |         pDst[iSample] = ma_clip_u8(ma_apply_volume_unclipped_u8(pSrc[iSample], volumeFixed));
44955 |     }
44956 | }
44957 |
44958 | MA_API void ma_copy_and_apply_volume_and_clip_samples_s16(ma_int16* pDst, const ma_int32* pSrc,
> ma_uint64 count, float volume)
44959 | {
44960 |     ma_uint64 iSample;
44961 |     ma_int16 volumeFixed;
44962 |
44963 |     MA_ASSERT(pDst != NULL);
44964 |     MA_ASSERT(pSrc != NULL);
44965 |
44966 |     volumeFixed = ma_float_to_fixed_16(volume);
44967 |
44968 |     for (iSample = 0; iSample < count; iSample += 1) {

```



```

44969 |         pDst[iSample] = ma_clip_s16(ma_apply_volume_unclipped_s16(pSrc[iSample], volumeFixed));
44970 |     }
44971 | }
44972 |
44973 | MA_API void ma_copy_and_apply_volume_and_clip_samples_s24(ma_uint8* pDst, const ma_int64* pSrc,
> ma_uint64 count, float volume)
44974 | {
44975 |     ma_uint64 iSample;
44976 |     ma_int16 volumeFixed;
44977 |
44978 |     MA_ASSERT(pDst != NULL);
44979 |     MA_ASSERT(pSrc != NULL);
44980 |
44981 |     volumeFixed = ma_float_to_fixed_16(volume);
44982 |
44983 |     for (iSample = 0; iSample < count; iSample += 1) {
44984 |         ma_int64 s = ma_clip_s24(ma_apply_volume_unclipped_s24(pSrc[iSample], volumeFixed));
44985 |         pDst[iSample*3 + 0] = (ma_uint8)((s & 0x000000FF) >> 0);
44986 |         pDst[iSample*3 + 1] = (ma_uint8)((s & 0x0000FF00) >> 8);
44987 |         pDst[iSample*3 + 2] = (ma_uint8)((s & 0x00FF0000) >> 16);
44988 |     }
44989 | }
44990 |
44991 | MA_API void ma_copy_and_apply_volume_and_clip_samples_s32(ma_int32* pDst, const ma_int64* pSrc,
> ma_uint64 count, float volume)
44992 | {
44993 |     ma_uint64 iSample;
44994 |     ma_int16 volumeFixed;
44995 |
44996 |     MA_ASSERT(pDst != NULL);
44997 |     MA_ASSERT(pSrc != NULL);
44998 |
44999 |     volumeFixed = ma_float_to_fixed_16(volume);
45000 |
45001 |     for (iSample = 0; iSample < count; iSample += 1) {
45002 |         pDst[iSample] = ma_clip_s32(ma_apply_volume_unclipped_s32(pSrc[iSample], volumeFixed));
45003 |     }
45004 | }
45005 |
45006 | MA_API void ma_copy_and_apply_volume_and_clip_samples_f32(float* pDst, const float* pSrc, ma_uint64
> count, float volume)
45007 | {
45008 |     ma_uint64 iSample;
45009 |
45010 |     MA_ASSERT(pDst != NULL);
45011 |     MA_ASSERT(pSrc != NULL);
45012 |
45013 |     /* For the f32 case we need to make sure this supports in-place processing where the input and
> output buffers are the same. */
45014 |
45015 |     for (iSample = 0; iSample < count; iSample += 1) {
45016 |         pDst[iSample] = ma_clip_f32(ma_apply_volume_unclipped_f32(pSrc[iSample], volume));
45017 |     }
45018 | }
45019 |
45020 | MA_API void ma_copy_and_apply_volume_and_clip_pcm_frames(void* pDst, const void* pSrc, ma_uint64
> frameCount, ma_format format, ma_uint32 channels, float volume)
45021 | {
45022 |     MA_ASSERT(pDst != NULL);
45023 |     MA_ASSERT(pSrc != NULL);
45024 |
45025 |     if (volume == 1) {
45026 |         ma_clip_pcm_frames(pDst, pSrc, frameCount, format, channels); /* Optimized case for volume
> = 1. */
45027 |     } else if (volume == 0) {
45028 |         ma_silence_pcm_frames(pDst, frameCount, format, channels); /* Optimized case for volume
> = 0. */
45029 |     } else {
45030 |         ma_uint64 sampleCount = frameCount * channels;
45031 |
45032 |         switch (format) {
45033 |             case ma_format_u8: ma_copy_and_apply_volume_and_clip_samples_u8((ma_uint8*)pDst,
> (const ma_int16*)pSrc, sampleCount, volume); break;
45034 |             case ma_format_s16: ma_copy_and_apply_volume_and_clip_samples_s16((ma_int16*)pDst,
> (const ma_int32*)pSrc, sampleCount, volume); break;
45035 |             case ma_format_s24: ma_copy_and_apply_volume_and_clip_samples_s24((ma_uint8*)pDst,
> (const ma_int64*)pSrc, sampleCount, volume); break;
45036 |             case ma_format_s32: ma_copy_and_apply_volume_and_clip_samples_s32((ma_int32*)pDst,
> (const ma_int64*)pSrc, sampleCount, volume); break;

```

```

45037 |         case ma_format_f32: ma_copy_and_apply_volume_and_clip_samples_f32((float*)pDst,
> (const float*)pSrc, sampleCount, volume); break;
45038 |
45039 |         /* Do nothing if we don't know the format. We're including these here to silence a
> compiler warning about enums not being handled by the switch. */
45040 |         case ma_format_unknown:
45041 |         case ma_format_count:
45042 |             break;
45043 |     }
45044 | }
45045 | }
45046 |
45047 |
45048 |
45049 | MA_API float ma_volume_linear_to_db(float factor)
45050 | {
45051 |     return 20*ma_log10f(factor);
45052 | }
45053 |
45054 | MA_API float ma_volume_db_to_linear(float gain)
45055 | {
45056 |     return ma_powf(10, gain/20.0f);
45057 | }
45058 |
45059 |
45060 | MA_API ma_result ma_mix_pcm_frames_f32(float* pDst, const float* pSrc, ma_uint64 frameCount,
> ma_uint32 channels, float volume)
45061 | {
45062 |     ma_uint64 iSample;
45063 |     ma_uint64 sampleCount;
45064 |
45065 |     if (pDst == NULL || pSrc == NULL || channels == 0) {
45066 |         return MA_INVALID_ARGS;
45067 |     }
45068 |
45069 |     if (volume == 0) {
45070 |         return MA_SUCCESS; /* No changes if the volume is 0. */
45071 |     }
45072 |
45073 |     sampleCount = frameCount * channels;
45074 |
45075 |     if (volume == 1) {
45076 |         for (iSample = 0; iSample < sampleCount; iSample += 1) {
45077 |             pDst[iSample] += pSrc[iSample];
45078 |         }
45079 |     } else {
45080 |         for (iSample = 0; iSample < sampleCount; iSample += 1) {
45081 |             pDst[iSample] += ma_apply_volume_unclipped_f32(pSrc[iSample], volume);
45082 |         }
45083 |     }
45084 |
45085 |     return MA_SUCCESS;
45086 | }
45087 |
45088 |
45089 |
45090 | /*****
> *****/
45091 |
45092 | Format Conversion
45093 |
45094 | *****/
> *****/
45095 |
45096 | static MA_INLINE ma_int16 ma_pcm_sample_f32_to_s16(float x)
45097 | {
45098 |     return (ma_int16)(x * 32767.0f);
45099 | }
45100 |
45101 | static MA_INLINE ma_int16 ma_pcm_sample_u8_to_s16_no_scale(ma_uint8 x)
45102 | {
45103 |     return (ma_int16)((ma_int16)x - 128);
45104 | }
45105 |
45106 | static MA_INLINE ma_int64 ma_pcm_sample_s24_to_s32_no_scale(const ma_uint8* x)
45107 | {
45108 |     return (ma_int64)((((ma_uint64)x[0] << 40) | ((ma_uint64)x[1] << 48) | ((ma_uint64)x[2] << 56))
> >> 40; /* Make sure the sign bits are maintained. */
45109 | }

```

```

45110 |
45111 | static MA_INLINE void ma_pcm_sample_s32_to_s24_no_scale(ma_int64 x, ma_uint8* s24)
45112 | {
45113 |     s24[0] = (ma_uint8)((x & 0x000000FF) >> 0);
45114 |     s24[1] = (ma_uint8)((x & 0x0000FF00) >> 8);
45115 |     s24[2] = (ma_uint8)((x & 0x00FF0000) >> 16);
45116 | }
45117 |
45118 |
45119 | /* u8 */
45120 | MA_API void ma_pcm_u8_to_u8(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45121 | {
45122 |     (void)ditherMode;
45123 |     ma_copy_memory_64(dst, src, count * sizeof(ma_uint8));
45124 | }
45125 |
45126 |
45127 | static MA_INLINE void ma_pcm_u8_to_s16__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45128 | {
45129 |     ma_int16* dst_s16 = (ma_int16*)dst;
45130 |     const ma_uint8* src_u8 = (const ma_uint8*)src;
45131 |
45132 |     ma_uint64 i;
45133 |     for (i = 0; i < count; i += 1) {
45134 |         ma_int16 x = src_u8[i];
45135 |         x = (ma_int16)(x - 128);
45136 |         x = (ma_int16)(x << 8);
45137 |         dst_s16[i] = x;
45138 |     }
45139 |
45140 |     (void)ditherMode;
45141 | }
45142 |
45143 | static MA_INLINE void ma_pcm_u8_to_s16__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45144 | {
45145 |     ma_pcm_u8_to_s16__reference(dst, src, count, ditherMode);
45146 | }
45147 |
45148 | #if defined(MA_SUPPORT_SSE2)
45149 | static MA_INLINE void ma_pcm_u8_to_s16__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45150 | {
45151 |     ma_pcm_u8_to_s16__optimized(dst, src, count, ditherMode);
45152 | }
45153 | #endif
45154 | #if defined(MA_SUPPORT_NEON)
45155 | static MA_INLINE void ma_pcm_u8_to_s16__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45156 | {
45157 |     ma_pcm_u8_to_s16__optimized(dst, src, count, ditherMode);
45158 | }
45159 | #endif
45160 |
45161 | MA_API void ma_pcm_u8_to_s16(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45162 | {
45163 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45164 |         ma_pcm_u8_to_s16__reference(dst, src, count, ditherMode);
45165 |     #else
45166 |         # if defined(MA_SUPPORT_SSE2)
45167 |             if (ma_has_sse2()) {
45168 |                 ma_pcm_u8_to_s16__sse2(dst, src, count, ditherMode);
45169 |             } else
45170 |             #elif defined(MA_SUPPORT_NEON)
45171 |                 if (ma_has_neon()) {
45172 |                     ma_pcm_u8_to_s16__neon(dst, src, count, ditherMode);
45173 |                 } else
45174 |                 #endif
45175 |                 {
45176 |                     ma_pcm_u8_to_s16__optimized(dst, src, count, ditherMode);
45177 |                 }
45178 |         #endif
45179 |     }
45180 |
45181 |
45182 | static MA_INLINE void ma_pcm_u8_to_s24__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45183 | {

```

```

45184 |     ma_uint8* dst_s24 = (ma_uint8*)dst;
45185 |     const ma_uint8* src_u8 = (const ma_uint8*)src;
45186 |
45187 |     ma_uint64 i;
45188 |     for (i = 0; i < count; i += 1) {
45189 |         ma_int16 x = src_u8[i];
45190 |         x = (ma_int16)(x - 128);
45191 |
45192 |         dst_s24[i*3+0] = 0;
45193 |         dst_s24[i*3+1] = 0;
45194 |         dst_s24[i*3+2] = (ma_uint8)((ma_int8)x);
45195 |     }
45196 |
45197 |     (void)ditherMode;
45198 | }
45199 |
45200 | static MA_INLINE void ma_pcm_u8_to_s24__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45201 | {
45202 |     ma_pcm_u8_to_s24__reference(dst, src, count, ditherMode);
45203 | }
45204 |
45205 | #if defined(MA_SUPPORT_SSE2)
45206 | static MA_INLINE void ma_pcm_u8_to_s24__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45207 | {
45208 |     ma_pcm_u8_to_s24__optimized(dst, src, count, ditherMode);
45209 | }
45210 | #endif
45211 | #if defined(MA_SUPPORT_NEON)
45212 | static MA_INLINE void ma_pcm_u8_to_s24__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45213 | {
45214 |     ma_pcm_u8_to_s24__optimized(dst, src, count, ditherMode);
45215 | }
45216 | #endif
45217 |
45218 | MA_API void ma_pcm_u8_to_s24(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45219 | {
45220 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45221 |         ma_pcm_u8_to_s24__reference(dst, src, count, ditherMode);
45222 |     #else
45223 |         # if defined(MA_SUPPORT_SSE2)
45224 |             if (ma_has_sse2()) {
45225 |                 ma_pcm_u8_to_s24__sse2(dst, src, count, ditherMode);
45226 |             } else
45227 |             #elif defined(MA_SUPPORT_NEON)
45228 |                 if (ma_has_neon()) {
45229 |                     ma_pcm_u8_to_s24__neon(dst, src, count, ditherMode);
45230 |                 } else
45231 |                 #endif
45232 |                 {
45233 |                     ma_pcm_u8_to_s24__optimized(dst, src, count, ditherMode);
45234 |                 }
45235 |     #endif
45236 | }
45237 |
45238 |
45239 | static MA_INLINE void ma_pcm_u8_to_s32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45240 | {
45241 |     ma_int32* dst_s32 = (ma_int32*)dst;
45242 |     const ma_uint8* src_u8 = (const ma_uint8*)src;
45243 |
45244 |     ma_uint64 i;
45245 |     for (i = 0; i < count; i += 1) {
45246 |         ma_int32 x = src_u8[i];
45247 |         x = x - 128;
45248 |         x = x << 24;
45249 |         dst_s32[i] = x;
45250 |     }
45251 |
45252 |     (void)ditherMode;
45253 | }
45254 |
45255 | static MA_INLINE void ma_pcm_u8_to_s32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45256 | {
45257 |     ma_pcm_u8_to_s32__reference(dst, src, count, ditherMode);

```

```

45258 | }
45259 |
45260 | #if defined(MA_SUPPORT_SSE2)
45261 | static MA_INLINE void ma_pcm_u8_to_s32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45262 | {
45263 |     ma_pcm_u8_to_s32__optimized(dst, src, count, ditherMode);
45264 | }
45265 | #endif
45266 | #if defined(MA_SUPPORT_NEON)
45267 | static MA_INLINE void ma_pcm_u8_to_s32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45268 | {
45269 |     ma_pcm_u8_to_s32__optimized(dst, src, count, ditherMode);
45270 | }
45271 | #endif
45272 |
45273 | MA_API void ma_pcm_u8_to_s32(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45274 | {
45275 |     #ifndef MA_USE_REFERENCE_CONVERSION_APIS
45276 |         ma_pcm_u8_to_s32__reference(dst, src, count, ditherMode);
45277 |     #else
45278 |         # if defined(MA_SUPPORT_SSE2)
45279 |             if (ma_has_sse2()) {
45280 |                 ma_pcm_u8_to_s32__sse2(dst, src, count, ditherMode);
45281 |             } else
45282 |             #elif defined(MA_SUPPORT_NEON)
45283 |                 if (ma_has_neon()) {
45284 |                     ma_pcm_u8_to_s32__neon(dst, src, count, ditherMode);
45285 |                 } else
45286 |                 #endif
45287 |                 {
45288 |                     ma_pcm_u8_to_s32__optimized(dst, src, count, ditherMode);
45289 |                 }
45290 |             #endif
45291 |         }
45292 |     }
45293 |
45294 | static MA_INLINE void ma_pcm_u8_to_f32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45295 | {
45296 |     float* dst_f32 = (float*)dst;
45297 |     const ma_uint8* src_u8 = (const ma_uint8*)src;
45298 |
45299 |     ma_uint64 i;
45300 |     for (i = 0; i < count; i += 1) {
45301 |         float x = (float)src_u8[i];
45302 |         x = x * 0.00784313725490196078f;    /* 0..255 to 0..2 */
45303 |         x = x - 1;                        /* 0..2 to -1..1 */
45304 |
45305 |         dst_f32[i] = x;
45306 |     }
45307 |
45308 |     (void)ditherMode;
45309 | }
45310 |
45311 | static MA_INLINE void ma_pcm_u8_to_f32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45312 | {
45313 |     ma_pcm_u8_to_f32__reference(dst, src, count, ditherMode);
45314 | }
45315 |
45316 | #if defined(MA_SUPPORT_SSE2)
45317 | static MA_INLINE void ma_pcm_u8_to_f32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45318 | {
45319 |     ma_pcm_u8_to_f32__optimized(dst, src, count, ditherMode);
45320 | }
45321 | #endif
45322 | #if defined(MA_SUPPORT_NEON)
45323 | static MA_INLINE void ma_pcm_u8_to_f32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45324 | {
45325 |     ma_pcm_u8_to_f32__optimized(dst, src, count, ditherMode);
45326 | }
45327 | #endif
45328 |
45329 | MA_API void ma_pcm_u8_to_f32(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45330 | {

```

```

45331 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45332 |     ma_pcm_u8_to_f32__reference(dst, src, count, ditherMode);
45333 | #else
45334 |     # if defined(MA_SUPPORT_SSE2)
45335 |         if (ma_has_sse2()) {
45336 |             ma_pcm_u8_to_f32__sse2(dst, src, count, ditherMode);
45337 |         } else
45338 |         #elif defined(MA_SUPPORT_NEON)
45339 |             if (ma_has_neon()) {
45340 |                 ma_pcm_u8_to_f32__neon(dst, src, count, ditherMode);
45341 |             } else
45342 |             #endif
45343 |             {
45344 |                 ma_pcm_u8_to_f32__optimized(dst, src, count, ditherMode);
45345 |             }
45346 | #endif
45347 | }
45348 |
45349 |
45350 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45351 | static MA_INLINE void ma_pcm_interleave_u8__reference(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
45352 | {
45353 |     ma_uint8* dst_u8 = (ma_uint8*)dst;
45354 |     const ma_uint8** src_u8 = (const ma_uint8**)src;
45355 |
45356 |     ma_uint64 iFrame;
45357 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
45358 |         ma_uint32 iChannel;
45359 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
45360 |             dst_u8[iFrame*channels + iChannel] = src_u8[iChannel][iFrame];
45361 |         }
45362 |     }
45363 | }
45364 | #else
45365 | static MA_INLINE void ma_pcm_interleave_u8__optimized(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
45366 | {
45367 |     ma_uint8* dst_u8 = (ma_uint8*)dst;
45368 |     const ma_uint8** src_u8 = (const ma_uint8**)src;
45369 |
45370 |     if (channels == 1) {
45371 |         ma_copy_memory_64(dst, src[0], frameCount * sizeof(ma_uint8));
45372 |     } else if (channels == 2) {
45373 |         ma_uint64 iFrame;
45374 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
45375 |             dst_u8[iFrame*2 + 0] = src_u8[0][iFrame];
45376 |             dst_u8[iFrame*2 + 1] = src_u8[1][iFrame];
45377 |         }
45378 |     } else {
45379 |         ma_uint64 iFrame;
45380 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
45381 |             ma_uint32 iChannel;
45382 |             for (iChannel = 0; iChannel < channels; iChannel += 1) {
45383 |                 dst_u8[iFrame*channels + iChannel] = src_u8[iChannel][iFrame];
45384 |             }
45385 |         }
45386 |     }
45387 | }
45388 | #endif
45389 |
45390 | MA_API void ma_pcm_interleave_u8(void* dst, const void** src, ma_uint64 frameCount, ma_uint32
> channels)
45391 | {
45392 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45393 |     ma_pcm_interleave_u8__reference(dst, src, frameCount, channels);
45394 | #else
45395 |     ma_pcm_interleave_u8__optimized(dst, src, frameCount, channels);
45396 | #endif
45397 | }
45398 |
45399 |
45400 | static MA_INLINE void ma_pcm_deinterleave_u8__reference(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
45401 | {
45402 |     ma_uint8** dst_u8 = (ma_uint8**)dst;
45403 |     const ma_uint8* src_u8 = (const ma_uint8*)src;
45404 |
45405 |     ma_uint64 iFrame;

```

```

45406 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
45407 |         ma_uint32 iChannel;
45408 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
45409 |             dst_u8[iChannel][iFrame] = src_u8[iFrame*channels + iChannel];
45410 |         }
45411 |     }
45412 | }
45413 |
45414 | static MA_INLINE void ma_pcm_deinterleave_u8__optimized(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
45415 | {
45416 |     ma_pcm_deinterleave_u8__reference(dst, src, frameCount, channels);
45417 | }
45418 |
45419 | MA_API void ma_pcm_deinterleave_u8(void** dst, const void* src, ma_uint64 frameCount, ma_uint32
> channels)
45420 | {
45421 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45422 |     ma_pcm_deinterleave_u8__reference(dst, src, frameCount, channels);
45423 | #else
45424 |     ma_pcm_deinterleave_u8__optimized(dst, src, frameCount, channels);
45425 | #endif
45426 | }
45427 |
45428 |
45429 | /* s16 */
45430 | static MA_INLINE void ma_pcm_s16_to_u8__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45431 | {
45432 |     ma_uint8* dst_u8 = (ma_uint8*)dst;
45433 |     const ma_int16* src_s16 = (const ma_int16*)src;
45434 |
45435 |     if (ditherMode == ma_dither_mode_none) {
45436 |         ma_uint64 i;
45437 |         for (i = 0; i < count; i += 1) {
45438 |             ma_int16 x = src_s16[i];
45439 |             x = (ma_int16)(x >> 8);
45440 |             x = (ma_int16)(x + 128);
45441 |             dst_u8[i] = (ma_uint8)x;
45442 |         }
45443 |     } else {
45444 |         ma_uint64 i;
45445 |         for (i = 0; i < count; i += 1) {
45446 |             ma_int16 x = src_s16[i];
45447 |
45448 |             /* Dither. Don't overflow. */
45449 |             ma_int32 dither = ma_dither_s32(ditherMode, -0x80, 0x7F);
45450 |             if ((x + dither) <= 0x7FFF) {
45451 |                 x = (ma_int16)(x + dither);
45452 |             } else {
45453 |                 x = 0x7FFF;
45454 |             }
45455 |
45456 |             x = (ma_int16)(x >> 8);
45457 |             x = (ma_int16)(x + 128);
45458 |             dst_u8[i] = (ma_uint8)x;
45459 |         }
45460 |     }
45461 | }
45462 |
45463 | static MA_INLINE void ma_pcm_s16_to_u8__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45464 | {
45465 |     ma_pcm_s16_to_u8__reference(dst, src, count, ditherMode);
45466 | }
45467 |
45468 | #if defined(MA_SUPPORT_SSE2)
45469 | static MA_INLINE void ma_pcm_s16_to_u8__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45470 | {
45471 |     ma_pcm_s16_to_u8__optimized(dst, src, count, ditherMode);
45472 | }
45473 | #endif
45474 | #if defined(MA_SUPPORT_NEON)
45475 | static MA_INLINE void ma_pcm_s16_to_u8__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45476 | {
45477 |     ma_pcm_s16_to_u8__optimized(dst, src, count, ditherMode);
45478 | }

```



```

45479 | #endif
45480 |
45481 | MA_API void ma_pcm_s16_to_u8(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45482 | {
45483 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45484 |     ma_pcm_s16_to_u8__reference(dst, src, count, ditherMode);
45485 | #else
45486 |     # if defined(MA_SUPPORT_SSE2)
45487 |         if (ma_has_sse2()) {
45488 |             ma_pcm_s16_to_u8__sse2(dst, src, count, ditherMode);
45489 |         } else
45490 |         #elif defined(MA_SUPPORT_NEON)
45491 |             if (ma_has_neon()) {
45492 |                 ma_pcm_s16_to_u8__neon(dst, src, count, ditherMode);
45493 |             } else
45494 |             #endif
45495 |             {
45496 |                 ma_pcm_s16_to_u8__optimized(dst, src, count, ditherMode);
45497 |             }
45498 | #endif
45499 | }
45500 |
45501 |
45502 | MA_API void ma_pcm_s16_to_s16(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45503 | {
45504 |     (void)ditherMode;
45505 |     ma_copy_memory_64(dst, src, count * sizeof(ma_int16));
45506 | }
45507 |
45508 |
45509 | static MA_INLINE void ma_pcm_s16_to_s24__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45510 | {
45511 |     ma_uint8* dst_s24 = (ma_uint8*)dst;
45512 |     const ma_int16* src_s16 = (const ma_int16*)src;
45513 |
45514 |     ma_uint64 i;
45515 |     for (i = 0; i < count; i += 1) {
45516 |         dst_s24[i*3+0] = 0;
45517 |         dst_s24[i*3+1] = (ma_uint8)(src_s16[i] & 0xFF);
45518 |         dst_s24[i*3+2] = (ma_uint8)(src_s16[i] >> 8);
45519 |     }
45520 |
45521 |     (void)ditherMode;
45522 | }
45523 |
45524 | static MA_INLINE void ma_pcm_s16_to_s24__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45525 | {
45526 |     ma_pcm_s16_to_s24__reference(dst, src, count, ditherMode);
45527 | }
45528 |
45529 | #if defined(MA_SUPPORT_SSE2)
45530 | static MA_INLINE void ma_pcm_s16_to_s24__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45531 | {
45532 |     ma_pcm_s16_to_s24__optimized(dst, src, count, ditherMode);
45533 | }
45534 | #endif
45535 | #if defined(MA_SUPPORT_NEON)
45536 | static MA_INLINE void ma_pcm_s16_to_s24__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45537 | {
45538 |     ma_pcm_s16_to_s24__optimized(dst, src, count, ditherMode);
45539 | }
45540 | #endif
45541 |
45542 | MA_API void ma_pcm_s16_to_s24(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45543 | {
45544 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45545 |     ma_pcm_s16_to_s24__reference(dst, src, count, ditherMode);
45546 | #else
45547 |     # if defined(MA_SUPPORT_SSE2)
45548 |         if (ma_has_sse2()) {
45549 |             ma_pcm_s16_to_s24__sse2(dst, src, count, ditherMode);
45550 |         } else
45551 |         #elif defined(MA_SUPPORT_NEON)

```

```

45552 |         if (ma_has_neon()) {
45553 |             ma_pcm_s16_to_s24__neon(dst, src, count, ditherMode);
45554 |         } else
45555 |         #endif
45556 |         {
45557 |             ma_pcm_s16_to_s24__optimized(dst, src, count, ditherMode);
45558 |         }
45559 |     #endif
45560 | }
45561 |
45562 |
45563 | static MA_INLINE void ma_pcm_s16_to_s32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45564 | {
45565 |     ma_int32* dst_s32 = (ma_int32*)dst;
45566 |     const ma_int16* src_s16 = (const ma_int16*)src;
45567 |
45568 |     ma_uint64 i;
45569 |     for (i = 0; i < count; i += 1) {
45570 |         dst_s32[i] = (ma_int32)src_s16[i] << 16;
45571 |     }
45572 |
45573 |     (void)ditherMode;
45574 | }
45575 |
45576 | static MA_INLINE void ma_pcm_s16_to_s32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45577 | {
45578 |     ma_pcm_s16_to_s32__reference(dst, src, count, ditherMode);
45579 | }
45580 |
45581 | #if defined(MA_SUPPORT_SSE2)
45582 | static MA_INLINE void ma_pcm_s16_to_s32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45583 | {
45584 |     ma_pcm_s16_to_s32__optimized(dst, src, count, ditherMode);
45585 | }
45586 | #endif
45587 | #if defined(MA_SUPPORT_NEON)
45588 | static MA_INLINE void ma_pcm_s16_to_s32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45589 | {
45590 |     ma_pcm_s16_to_s32__optimized(dst, src, count, ditherMode);
45591 | }
45592 | #endif
45593 |
45594 | MA_API void ma_pcm_s16_to_s32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45595 | {
45596 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45597 |         ma_pcm_s16_to_s32__reference(dst, src, count, ditherMode);
45598 |     #else
45599 |         # if defined(MA_SUPPORT_SSE2)
45600 |             if (ma_has_sse2()) {
45601 |                 ma_pcm_s16_to_s32__sse2(dst, src, count, ditherMode);
45602 |             } else
45603 |             #elif defined(MA_SUPPORT_NEON)
45604 |                 if (ma_has_neon()) {
45605 |                     ma_pcm_s16_to_s32__neon(dst, src, count, ditherMode);
45606 |                 } else
45607 |                 #endif
45608 |                 {
45609 |                     ma_pcm_s16_to_s32__optimized(dst, src, count, ditherMode);
45610 |                 }
45611 |             #endif
45612 |         }
45613 |
45614 |
45615 | static MA_INLINE void ma_pcm_s16_to_f32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45616 | {
45617 |     float* dst_f32 = (float*)dst;
45618 |     const ma_int16* src_s16 = (const ma_int16*)src;
45619 |
45620 |     ma_uint64 i;
45621 |     for (i = 0; i < count; i += 1) {
45622 |         float x = (float)src_s16[i];
45623 |
45624 |     #if 0

```

```

45625 |         /* The accurate way. */
45626 |         x = x + 32768.0f;                /* -32768..32767 to 0..65535 */
45627 |         x = x * 0.00003051804379339284f; /* 0..65535 to 0..2 */
45628 |         x = x - 1;                       /* 0..2 to -1..1 */
45629 |     #else
45630 |         /* The fast way. */
45631 |         x = x * 0.000030517578125f;      /* -32768..32767 to -1..0.999969482421875 */
45632 |     #endif
45633 |
45634 |         dst_f32[i] = x;
45635 |     }
45636 |
45637 |     (void)ditherMode;
45638 | }
45639 |
45640 | static MA_INLINE void ma_pcm_s16_to_f32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45641 | {
45642 |     ma_pcm_s16_to_f32__reference(dst, src, count, ditherMode);
45643 | }
45644 |
45645 | #if defined(MA_SUPPORT_SSE2)
45646 | static MA_INLINE void ma_pcm_s16_to_f32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45647 | {
45648 |     ma_pcm_s16_to_f32__optimized(dst, src, count, ditherMode);
45649 | }
45650 | #endif
45651 | #if defined(MA_SUPPORT_NEON)
45652 | static MA_INLINE void ma_pcm_s16_to_f32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45653 | {
45654 |     ma_pcm_s16_to_f32__optimized(dst, src, count, ditherMode);
45655 | }
45656 | #endif
45657 |
45658 | MA_API void ma_pcm_s16_to_f32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45659 | {
45660 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45661 |         ma_pcm_s16_to_f32__reference(dst, src, count, ditherMode);
45662 |     #else
45663 |         # if defined(MA_SUPPORT_SSE2)
45664 |             if (ma_has_sse2()) {
45665 |                 ma_pcm_s16_to_f32__sse2(dst, src, count, ditherMode);
45666 |             } else
45667 |             #elif defined(MA_SUPPORT_NEON)
45668 |                 if (ma_has_neon()) {
45669 |                     ma_pcm_s16_to_f32__neon(dst, src, count, ditherMode);
45670 |                 } else
45671 |                 #endif
45672 |                 {
45673 |                     ma_pcm_s16_to_f32__optimized(dst, src, count, ditherMode);
45674 |                 }
45675 |         #endif
45676 |     }
45677 |
45678 |
45679 | static MA_INLINE void ma_pcm_interleave_s16__reference(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
45680 | {
45681 |     ma_int16* dst_s16 = (ma_int16*)dst;
45682 |     const ma_int16** src_s16 = (const ma_int16**)src;
45683 |
45684 |     ma_uint64 iFrame;
45685 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
45686 |         ma_uint32 iChannel;
45687 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
45688 |             dst_s16[iFrame*channels + iChannel] = src_s16[iChannel][iFrame];
45689 |         }
45690 |     }
45691 | }
45692 |
45693 | static MA_INLINE void ma_pcm_interleave_s16__optimized(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
45694 | {
45695 |     ma_pcm_interleave_s16__reference(dst, src, frameCount, channels);
45696 | }
45697 |

```

```

45698 | MA_API void ma_pcm_interleave_s16(void* dst, const void** src, ma_uint64 frameCount, ma_uint32
> channels)
45699 | {
45700 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45701 |     ma_pcm_interleave_s16__reference(dst, src, frameCount, channels);
45702 | #else
45703 |     ma_pcm_interleave_s16__optimized(dst, src, frameCount, channels);
45704 | #endif
45705 | }
45706 |
45707 |
45708 | static MA_INLINE void ma_pcm_deinterleave_s16__reference(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
45709 | {
45710 |     ma_int16** dst_s16 = (ma_int16**)dst;
45711 |     const ma_int16* src_s16 = (const ma_int16*)src;
45712 |
45713 |     ma_uint64 iFrame;
45714 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
45715 |         ma_uint32 iChannel;
45716 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
45717 |             dst_s16[iChannel][iFrame] = src_s16[iFrame*channels + iChannel];
45718 |         }
45719 |     }
45720 | }
45721 |
45722 | static MA_INLINE void ma_pcm_deinterleave_s16__optimized(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
45723 | {
45724 |     ma_pcm_deinterleave_s16__reference(dst, src, frameCount, channels);
45725 | }
45726 |
45727 | MA_API void ma_pcm_deinterleave_s16(void** dst, const void* src, ma_uint64 frameCount, ma_uint32
> channels)
45728 | {
45729 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45730 |     ma_pcm_deinterleave_s16__reference(dst, src, frameCount, channels);
45731 | #else
45732 |     ma_pcm_deinterleave_s16__optimized(dst, src, frameCount, channels);
45733 | #endif
45734 | }
45735 |
45736 |
45737 | /* s24 */
45738 | static MA_INLINE void ma_pcm_s24_to_u8__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45739 | {
45740 |     ma_uint8* dst_u8 = (ma_uint8*)dst;
45741 |     const ma_uint8* src_s24 = (const ma_uint8*)src;
45742 |
45743 |     if (ditherMode == ma_dither_mode_none) {
45744 |         ma_uint64 i;
45745 |         for (i = 0; i < count; i += 1) {
45746 |             dst_u8[i] = (ma_uint8)((ma_int8)src_s24[i*3 + 2] + 128);
45747 |         }
45748 |     } else {
45749 |         ma_uint64 i;
45750 |         for (i = 0; i < count; i += 1) {
45751 |             ma_int32 x = (ma_int32)((((ma_uint32)(src_s24[i*3+0]) << 8) |
> ((ma_uint32)(src_s24[i*3+1]) << 16) | ((ma_uint32)(src_s24[i*3+2]) << 24));
45752 |
45753 |             /* Dither. Don't overflow. */
45754 |             ma_int32 dither = ma_dither_s32(ditherMode, -0x800000, 0x7FFFFFFF);
45755 |             if ((ma_int64)x + dither <= 0x7FFFFFFF) {
45756 |                 x = x + dither;
45757 |             } else {
45758 |                 x = 0x7FFFFFFF;
45759 |             }
45760 |
45761 |             x = x >> 24;
45762 |             x = x + 128;
45763 |             dst_u8[i] = (ma_uint8)x;
45764 |         }
45765 |     }
45766 | }
45767 |
45768 | static MA_INLINE void ma_pcm_s24_to_u8__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45769 | {

```

```

45770 |     ma_pcm_s24_to_u8__reference(dst, src, count, ditherMode);
45771 | }
45772 |
45773 | #if defined(MA_SUPPORT_SSE2)
45774 | static MA_INLINE void ma_pcm_s24_to_u8__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45775 | {
45776 |     ma_pcm_s24_to_u8__optimized(dst, src, count, ditherMode);
45777 | }
45778 | #endif
45779 | #if defined(MA_SUPPORT_NEON)
45780 | static MA_INLINE void ma_pcm_s24_to_u8__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45781 | {
45782 |     ma_pcm_s24_to_u8__optimized(dst, src, count, ditherMode);
45783 | }
45784 | #endif
45785 |
45786 | MA_API void ma_pcm_s24_to_u8(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
45787 | {
45788 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45789 |     ma_pcm_s24_to_u8__reference(dst, src, count, ditherMode);
45790 | #else
45791 |     # if defined(MA_SUPPORT_SSE2)
45792 |         if (ma_has_sse2()) {
45793 |             ma_pcm_s24_to_u8__sse2(dst, src, count, ditherMode);
45794 |         } else
45795 |         #elif defined(MA_SUPPORT_NEON)
45796 |             if (ma_has_neon()) {
45797 |                 ma_pcm_s24_to_u8__neon(dst, src, count, ditherMode);
45798 |             } else
45799 |             #endif
45800 |             {
45801 |                 ma_pcm_s24_to_u8__optimized(dst, src, count, ditherMode);
45802 |             }
45803 | #endif
45804 | }
45805 |
45806 |
45807 | static MA_INLINE void ma_pcm_s24_to_s16__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45808 | {
45809 |     ma_int16* dst_s16 = (ma_int16*)dst;
45810 |     const ma_uint8* src_s24 = (const ma_uint8*)src;
45811 |
45812 |     if (ditherMode == ma_dither_mode_none) {
45813 |         ma_uint64 i;
45814 |         for (i = 0; i < count; i += 1) {
45815 |             ma_uint16 dst_lo = ((ma_uint16)src_s24[i*3 + 1]);
45816 |             ma_uint16 dst_hi = (ma_uint16)((ma_uint16)src_s24[i*3 + 2] << 8);
45817 |             dst_s16[i] = (ma_int16)(dst_lo | dst_hi);
45818 |         }
45819 |     } else {
45820 |         ma_uint64 i;
45821 |         for (i = 0; i < count; i += 1) {
45822 |             ma_int32 x = (ma_int32)((ma_uint32)(src_s24[i*3+0]) << 8) |
> ((ma_uint32)(src_s24[i*3+1]) << 16) | ((ma_uint32)(src_s24[i*3+2])) << 24);
45823 |
45824 |             /* Dither. Don't overflow. */
45825 |             ma_int32 dither = ma_dither_s32(ditherMode, -0x8000, 0x7FFF);
45826 |             if ((ma_int64)x + dither <= 0x7FFFFFFF) {
45827 |                 x = x + dither;
45828 |             } else {
45829 |                 x = 0x7FFFFFFF;
45830 |             }
45831 |
45832 |             x = x >> 16;
45833 |             dst_s16[i] = (ma_int16)x;
45834 |         }
45835 |     }
45836 | }
45837 |
45838 | static MA_INLINE void ma_pcm_s24_to_s16__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45839 | {
45840 |     ma_pcm_s24_to_s16__reference(dst, src, count, ditherMode);
45841 | }
45842 |
45843 | #if defined(MA_SUPPORT_SSE2)

```

```

45844 | static MA_INLINE void ma_pcm_s24_to_s16__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45845 | {
45846 |     ma_pcm_s24_to_s16__optimized(dst, src, count, ditherMode);
45847 | }
45848 | #endif
45849 | #if defined(MA_SUPPORT_NEON)
45850 | static MA_INLINE void ma_pcm_s24_to_s16__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45851 | {
45852 |     ma_pcm_s24_to_s16__optimized(dst, src, count, ditherMode);
45853 | }
45854 | #endif
45855 |
45856 | MA_API void ma_pcm_s24_to_s16(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45857 | {
45858 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45859 |     ma_pcm_s24_to_s16__reference(dst, src, count, ditherMode);
45860 | #else
45861 |     # if defined(MA_SUPPORT_SSE2)
45862 |         if (ma_has_sse2()) {
45863 |             ma_pcm_s24_to_s16__sse2(dst, src, count, ditherMode);
45864 |         } else
45865 |         #elif defined(MA_SUPPORT_NEON)
45866 |             if (ma_has_neon()) {
45867 |                 ma_pcm_s24_to_s16__neon(dst, src, count, ditherMode);
45868 |             } else
45869 |             #endif
45870 |             {
45871 |                 ma_pcm_s24_to_s16__optimized(dst, src, count, ditherMode);
45872 |             }
45873 | #endif
45874 | }
45875 |
45876 |
45877 | MA_API void ma_pcm_s24_to_s24(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45878 | {
45879 |     (void)ditherMode;
45880 |
45881 |     ma_copy_memory_64(dst, src, count * 3);
45882 | }
45883 |
45884 |
45885 | static MA_INLINE void ma_pcm_s24_to_s32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45886 | {
45887 |     ma_int32* dst_s32 = (ma_int32*)dst;
45888 |     const ma_uint8* src_s24 = (const ma_uint8*)src;
45889 |
45890 |     ma_uint64 i;
45891 |     for (i = 0; i < count; i += 1) {
45892 |         dst_s32[i] = (ma_int32)((((ma_uint32)(src_s24[i*3+0]) << 8) | ((ma_uint32)(src_s24[i*3+1]) <<
> 16) | ((ma_uint32)(src_s24[i*3+2])) << 24);
45893 |     }
45894 |
45895 |     (void)ditherMode;
45896 | }
45897 |
45898 | static MA_INLINE void ma_pcm_s24_to_s32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45899 | {
45900 |     ma_pcm_s24_to_s32__reference(dst, src, count, ditherMode);
45901 | }
45902 |
45903 | #if defined(MA_SUPPORT_SSE2)
45904 | static MA_INLINE void ma_pcm_s24_to_s32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45905 | {
45906 |     ma_pcm_s24_to_s32__optimized(dst, src, count, ditherMode);
45907 | }
45908 | #endif
45909 | #if defined(MA_SUPPORT_NEON)
45910 | static MA_INLINE void ma_pcm_s24_to_s32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45911 | {
45912 |     ma_pcm_s24_to_s32__optimized(dst, src, count, ditherMode);
45913 | }

```

```

45914 | #endif
45915 |
45916 | MA_API void ma_pcm_s24_to_s32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45917 | {
45918 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45919 |     ma_pcm_s24_to_s32__reference(dst, src, count, ditherMode);
45920 | #else
45921 |     # if defined(MA_SUPPORT_SSE2)
45922 |         if (ma_has_sse2()) {
45923 |             ma_pcm_s24_to_s32__sse2(dst, src, count, ditherMode);
45924 |         } else
45925 |         #elif defined(MA_SUPPORT_NEON)
45926 |             if (ma_has_neon()) {
45927 |                 ma_pcm_s24_to_s32__neon(dst, src, count, ditherMode);
45928 |             } else
45929 |             #endif
45930 |             {
45931 |                 ma_pcm_s24_to_s32__optimized(dst, src, count, ditherMode);
45932 |             }
45933 | #endif
45934 | }
45935 |
45936 |
45937 | static MA_INLINE void ma_pcm_s24_to_f32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45938 | {
45939 |     float* dst_f32 = (float*)dst;
45940 |     const ma_uint8* src_s24 = (const ma_uint8*)src;
45941 |
45942 |     ma_uint64 i;
45943 |     for (i = 0; i < count; i += 1) {
45944 |         float x = (float)(((ma_int32)((ma_uint32)(src_s24[i*3+0]) << 8) |
> ((ma_uint32)(src_s24[i*3+1]) << 16) | ((ma_uint32)(src_s24[i*3+2]) << 24)) >> 8);
45945 |
45946 |         #if 0
45947 |             /* The accurate way. */
45948 |             x = x + 8388608.0f; /* -8388608..8388607 to 0..16777215 */
45949 |             x = x * 0.00000011920929665621f; /* 0..16777215 to 0..2 */
45950 |             x = x - 1; /* 0..2 to -1..1 */
45951 |         #else
45952 |             /* The fast way. */
45953 |             x = x * 0.00000011920928955078125f; /* -8388608..8388607 to -1..0.999969482421875 */
45954 |         #endif
45955 |
45956 |         dst_f32[i] = x;
45957 |     }
45958 |
45959 |     (void)ditherMode;
45960 | }
45961 |
45962 | static MA_INLINE void ma_pcm_s24_to_f32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45963 | {
45964 |     ma_pcm_s24_to_f32__reference(dst, src, count, ditherMode);
45965 | }
45966 |
45967 | #if defined(MA_SUPPORT_SSE2)
45968 | static MA_INLINE void ma_pcm_s24_to_f32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45969 | {
45970 |     ma_pcm_s24_to_f32__optimized(dst, src, count, ditherMode);
45971 | }
45972 | #endif
45973 | #if defined(MA_SUPPORT_NEON)
45974 | static MA_INLINE void ma_pcm_s24_to_f32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
45975 | {
45976 |     ma_pcm_s24_to_f32__optimized(dst, src, count, ditherMode);
45977 | }
45978 | #endif
45979 |
45980 | MA_API void ma_pcm_s24_to_f32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
45981 | {
45982 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
45983 |     ma_pcm_s24_to_f32__reference(dst, src, count, ditherMode);
45984 | #else
45985 |     # if defined(MA_SUPPORT_SSE2)

```



```

45986 |         if (ma_has_sse2()) {
45987 |             ma_pcm_s24_to_f32__sse2(dst, src, count, ditherMode);
45988 |         } else
45989 |         #elif defined(MA_SUPPORT_NEON)
45990 |             if (ma_has_neon()) {
45991 |                 ma_pcm_s24_to_f32__neon(dst, src, count, ditherMode);
45992 |             } else
45993 |             #endif
45994 |             {
45995 |                 ma_pcm_s24_to_f32__optimized(dst, src, count, ditherMode);
45996 |             }
45997 |         #endif
45998 |     }
45999 | }
46000 |
46001 | static MA_INLINE void ma_pcm_interleave_s24__reference(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
46002 | {
46003 |     ma_uint8* dst8 = (ma_uint8*)dst;
46004 |     const ma_uint8** src8 = (const ma_uint8**)src;
46005 |
46006 |     ma_uint64 iFrame;
46007 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
46008 |         ma_uint32 iChannel;
46009 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
46010 |             dst8[iFrame*3*channels + iChannel*3 + 0] = src8[iChannel][iFrame*3 + 0];
46011 |             dst8[iFrame*3*channels + iChannel*3 + 1] = src8[iChannel][iFrame*3 + 1];
46012 |             dst8[iFrame*3*channels + iChannel*3 + 2] = src8[iChannel][iFrame*3 + 2];
46013 |         }
46014 |     }
46015 | }
46016 |
46017 | static MA_INLINE void ma_pcm_interleave_s24__optimized(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
46018 | {
46019 |     ma_pcm_interleave_s24__reference(dst, src, frameCount, channels);
46020 | }
46021 |
46022 | MA_API void ma_pcm_interleave_s24(void* dst, const void** src, ma_uint64 frameCount, ma_uint32
> channels)
46023 | {
46024 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46025 |         ma_pcm_interleave_s24__reference(dst, src, frameCount, channels);
46026 |     #else
46027 |         ma_pcm_interleave_s24__optimized(dst, src, frameCount, channels);
46028 |     #endif
46029 | }
46030 |
46031 |
46032 | static MA_INLINE void ma_pcm_deinterleave_s24__reference(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
46033 | {
46034 |     ma_uint8** dst8 = (ma_uint8**)dst;
46035 |     const ma_uint8* src8 = (const ma_uint8*)src;
46036 |
46037 |     ma_uint32 iFrame;
46038 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
46039 |         ma_uint32 iChannel;
46040 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
46041 |             dst8[iChannel][iFrame*3 + 0] = src8[iFrame*3*channels + iChannel*3 + 0];
46042 |             dst8[iChannel][iFrame*3 + 1] = src8[iFrame*3*channels + iChannel*3 + 1];
46043 |             dst8[iChannel][iFrame*3 + 2] = src8[iFrame*3*channels + iChannel*3 + 2];
46044 |         }
46045 |     }
46046 | }
46047 |
46048 | static MA_INLINE void ma_pcm_deinterleave_s24__optimized(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
46049 | {
46050 |     ma_pcm_deinterleave_s24__reference(dst, src, frameCount, channels);
46051 | }
46052 |
46053 | MA_API void ma_pcm_deinterleave_s24(void** dst, const void* src, ma_uint64 frameCount, ma_uint32
> channels)
46054 | {
46055 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46056 |         ma_pcm_deinterleave_s24__reference(dst, src, frameCount, channels);
46057 |     #else
46058 |         ma_pcm_deinterleave_s24__optimized(dst, src, frameCount, channels);

```

```

46059 | #endif
46060 | }
46061 |
46062 |
46063 |
46064 | /* s32 */
46065 | static MA_INLINE void ma_pcm_s32_to_u8__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46066 | {
46067 |     ma_uint8* dst_u8 = (ma_uint8*)dst;
46068 |     const ma_int32* src_s32 = (const ma_int32*)src;
46069 |
46070 |     if (ditherMode == ma_dither_mode_none) {
46071 |         ma_uint64 i;
46072 |         for (i = 0; i < count; i += 1) {
46073 |             ma_int32 x = src_s32[i];
46074 |             x = x >> 24;
46075 |             x = x + 128;
46076 |             dst_u8[i] = (ma_uint8)x;
46077 |         }
46078 |     } else {
46079 |         ma_uint64 i;
46080 |         for (i = 0; i < count; i += 1) {
46081 |             ma_int32 x = src_s32[i];
46082 |
46083 |             /* Dither. Don't overflow. */
46084 |             ma_int32 dither = ma_dither_s32(ditherMode, -0x800000, 0x7FFFFFFF);
46085 |             if ((ma_int64)x + dither <= 0x7FFFFFFF) {
46086 |                 x = x + dither;
46087 |             } else {
46088 |                 x = 0x7FFFFFFF;
46089 |             }
46090 |
46091 |             x = x >> 24;
46092 |             x = x + 128;
46093 |             dst_u8[i] = (ma_uint8)x;
46094 |         }
46095 |     }
46096 | }
46097 |
46098 | static MA_INLINE void ma_pcm_s32_to_u8__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46099 | {
46100 |     ma_pcm_s32_to_u8__reference(dst, src, count, ditherMode);
46101 | }
46102 |
46103 | #if defined(MA_SUPPORT_SSE2)
46104 | static MA_INLINE void ma_pcm_s32_to_u8__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46105 | {
46106 |     ma_pcm_s32_to_u8__optimized(dst, src, count, ditherMode);
46107 | }
46108 | #endif
46109 | #if defined(MA_SUPPORT_NEON)
46110 | static MA_INLINE void ma_pcm_s32_to_u8__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46111 | {
46112 |     ma_pcm_s32_to_u8__optimized(dst, src, count, ditherMode);
46113 | }
46114 | #endif
46115 |
46116 | MA_API void ma_pcm_s32_to_u8(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
46117 | {
46118 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46119 |         ma_pcm_s32_to_u8__reference(dst, src, count, ditherMode);
46120 |     #else
46121 |         # if defined(MA_SUPPORT_SSE2)
46122 |             if (ma_has_sse2()) {
46123 |                 ma_pcm_s32_to_u8__sse2(dst, src, count, ditherMode);
46124 |             } else
46125 |             #elif defined(MA_SUPPORT_NEON)
46126 |                 if (ma_has_neon()) {
46127 |                     ma_pcm_s32_to_u8__neon(dst, src, count, ditherMode);
46128 |                 } else
46129 |                 #endif
46130 |                 {
46131 |                     ma_pcm_s32_to_u8__optimized(dst, src, count, ditherMode);
46132 |                 }
46133 |     #endif

```

```

46134 | }
46135 |
46136 |
46137 | static MA_INLINE void ma_pcm_s32_to_s16__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46138 | {
46139 |     ma_int16* dst_s16 = (ma_int16*)dst;
46140 |     const ma_int32* src_s32 = (const ma_int32*)src;
46141 |
46142 |     if (ditherMode == ma_dither_mode_none) {
46143 |         ma_uint64 i;
46144 |         for (i = 0; i < count; i += 1) {
46145 |             ma_int32 x = src_s32[i];
46146 |             x = x >> 16;
46147 |             dst_s16[i] = (ma_int16)x;
46148 |         }
46149 |     } else {
46150 |         ma_uint64 i;
46151 |         for (i = 0; i < count; i += 1) {
46152 |             ma_int32 x = src_s32[i];
46153 |
46154 |             /* Dither. Don't overflow. */
46155 |             ma_int32 dither = ma_dither_s32(ditherMode, -0x8000, 0x7FFF);
46156 |             if ((ma_int64)x + dither <= 0x7FFFFFFF) {
46157 |                 x = x + dither;
46158 |             } else {
46159 |                 x = 0x7FFFFFFF;
46160 |             }
46161 |
46162 |             x = x >> 16;
46163 |             dst_s16[i] = (ma_int16)x;
46164 |         }
46165 |     }
46166 | }
46167 |
46168 | static MA_INLINE void ma_pcm_s32_to_s16__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46169 | {
46170 |     ma_pcm_s32_to_s16__reference(dst, src, count, ditherMode);
46171 | }
46172 |
46173 | #if defined(MA_SUPPORT_SSE2)
46174 | static MA_INLINE void ma_pcm_s32_to_s16__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46175 | {
46176 |     ma_pcm_s32_to_s16__optimized(dst, src, count, ditherMode);
46177 | }
46178 | #endif
46179 | #if defined(MA_SUPPORT_NEON)
46180 | static MA_INLINE void ma_pcm_s32_to_s16__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46181 | {
46182 |     ma_pcm_s32_to_s16__optimized(dst, src, count, ditherMode);
46183 | }
46184 | #endif
46185 |
46186 | MA_API void ma_pcm_s32_to_s16(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46187 | {
46188 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46189 |         ma_pcm_s32_to_s16__reference(dst, src, count, ditherMode);
46190 |     #else
46191 |         # if defined(MA_SUPPORT_SSE2)
46192 |             if (ma_has_sse2()) {
46193 |                 ma_pcm_s32_to_s16__sse2(dst, src, count, ditherMode);
46194 |             } else
46195 |             #elif defined(MA_SUPPORT_NEON)
46196 |                 if (ma_has_neon()) {
46197 |                     ma_pcm_s32_to_s16__neon(dst, src, count, ditherMode);
46198 |                 } else
46199 |                 #endif
46200 |                 {
46201 |                     ma_pcm_s32_to_s16__optimized(dst, src, count, ditherMode);
46202 |                 }
46203 |             #endif
46204 |         }
46205 |
46206 |
46207 | static MA_INLINE void ma_pcm_s32_to_s24__reference(void* dst, const void* src, ma_uint64 count,

```

```

> ma_dither_mode ditherMode)
46208 | {
46209 |     ma_uint8* dst_s24 = (ma_uint8*)dst;
46210 |     const ma_int32* src_s32 = (const ma_int32*)src;
46211 |
46212 |     ma_uint64 i;
46213 |     for (i = 0; i < count; i += 1) {
46214 |         ma_uint32 x = (ma_uint32)src_s32[i];
46215 |         dst_s24[i*3+0] = (ma_uint8)((x & 0x0000FF00) >> 8);
46216 |         dst_s24[i*3+1] = (ma_uint8)((x & 0x00FF0000) >> 16);
46217 |         dst_s24[i*3+2] = (ma_uint8)((x & 0xFF000000) >> 24);
46218 |     }
46219 |
46220 |     (void)ditherMode; /* No dithering for s32 -> s24. */
46221 | }
46222 |
46223 | static MA_INLINE void ma_pcm_s32_to_s24_optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46224 | {
46225 |     ma_pcm_s32_to_s24_reference(dst, src, count, ditherMode);
46226 | }
46227 |
46228 | #if defined(MA_SUPPORT_SSE2)
46229 | static MA_INLINE void ma_pcm_s32_to_s24_sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46230 | {
46231 |     ma_pcm_s32_to_s24_optimized(dst, src, count, ditherMode);
46232 | }
46233 | #endif
46234 | #if defined(MA_SUPPORT_NEON)
46235 | static MA_INLINE void ma_pcm_s32_to_s24_neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46236 | {
46237 |     ma_pcm_s32_to_s24_optimized(dst, src, count, ditherMode);
46238 | }
46239 | #endif
46240 |
46241 | MA_API void ma_pcm_s32_to_s24(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46242 | {
46243 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46244 |     ma_pcm_s32_to_s24_reference(dst, src, count, ditherMode);
46245 | #else
46246 |     # if defined(MA_SUPPORT_SSE2)
46247 |         if (ma_has_sse2()) {
46248 |             ma_pcm_s32_to_s24_sse2(dst, src, count, ditherMode);
46249 |         } else
46250 |         #elif defined(MA_SUPPORT_NEON)
46251 |             if (ma_has_neon()) {
46252 |                 ma_pcm_s32_to_s24_neon(dst, src, count, ditherMode);
46253 |             } else
46254 |             #endif
46255 |             {
46256 |                 ma_pcm_s32_to_s24_optimized(dst, src, count, ditherMode);
46257 |             }
46258 |     #endif
46259 | }
46260 |
46261 |
46262 | MA_API void ma_pcm_s32_to_s32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46263 | {
46264 |     (void)ditherMode;
46265 |
46266 |     ma_copy_memory_64(dst, src, count * sizeof(ma_int32));
46267 | }
46268 |
46269 |
46270 | static MA_INLINE void ma_pcm_s32_to_f32_reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46271 | {
46272 |     float* dst_f32 = (float*)dst;
46273 |     const ma_int32* src_s32 = (const ma_int32*)src;
46274 |
46275 |     ma_uint64 i;
46276 |     for (i = 0; i < count; i += 1) {
46277 |         double x = src_s32[i];
46278 |
46279 |     #if 0

```

```

46280 |         x = x + 2147483648.0;
46281 |         x = x * 0.0000000004656612873077392578125;
46282 |         x = x - 1;
46283 |     #else
46284 |         x = x / 2147483648.0;
46285 |     #endif
46286 |
46287 |         dst_f32[i] = (float)x;
46288 |     }
46289 |
46290 |     (void)ditherMode;    /* No dithering for s32 -> f32. */
46291 | }
46292 |
46293 | static MA_INLINE void ma_pcm_s32_to_f32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46294 | {
46295 |     ma_pcm_s32_to_f32__reference(dst, src, count, ditherMode);
46296 | }
46297 |
46298 | #if defined(MA_SUPPORT_SSE2)
46299 | static MA_INLINE void ma_pcm_s32_to_f32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46300 | {
46301 |     ma_pcm_s32_to_f32__optimized(dst, src, count, ditherMode);
46302 | }
46303 | #endif
46304 | #if defined(MA_SUPPORT_NEON)
46305 | static MA_INLINE void ma_pcm_s32_to_f32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46306 | {
46307 |     ma_pcm_s32_to_f32__optimized(dst, src, count, ditherMode);
46308 | }
46309 | #endif
46310 |
46311 | MA_API void ma_pcm_s32_to_f32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46312 | {
46313 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46314 |         ma_pcm_s32_to_f32__reference(dst, src, count, ditherMode);
46315 |     #else
46316 |         # if defined(MA_SUPPORT_SSE2)
46317 |             if (ma_has_sse2()) {
46318 |                 ma_pcm_s32_to_f32__sse2(dst, src, count, ditherMode);
46319 |             } else
46320 |             #elif defined(MA_SUPPORT_NEON)
46321 |                 if (ma_has_neon()) {
46322 |                     ma_pcm_s32_to_f32__neon(dst, src, count, ditherMode);
46323 |                 } else
46324 |                 #endif
46325 |                 {
46326 |                     ma_pcm_s32_to_f32__optimized(dst, src, count, ditherMode);
46327 |                 }
46328 |     #endif
46329 | }
46330 |
46331 |
46332 | static MA_INLINE void ma_pcm_interleave_s32__reference(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
46333 | {
46334 |     ma_int32* dst_s32 = (ma_int32*)dst;
46335 |     const ma_int32** src_s32 = (const ma_int32**)src;
46336 |
46337 |     ma_uint64 iFrame;
46338 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
46339 |         ma_uint32 iChannel;
46340 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
46341 |             dst_s32[iFrame*channels + iChannel] = src_s32[iChannel][iFrame];
46342 |         }
46343 |     }
46344 | }
46345 |
46346 | static MA_INLINE void ma_pcm_interleave_s32__optimized(void* dst, const void** src, ma_uint64
> frameCount, ma_uint32 channels)
46347 | {
46348 |     ma_pcm_interleave_s32__reference(dst, src, frameCount, channels);
46349 | }
46350 |
46351 | MA_API void ma_pcm_interleave_s32(void* dst, const void** src, ma_uint64 frameCount, ma_uint32
> channels)

```

```

46352 | {
46353 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46354 |     ma_pcm_interleave_s32__reference(dst, src, frameCount, channels);
46355 | #else
46356 |     ma_pcm_interleave_s32__optimized(dst, src, frameCount, channels);
46357 | #endif
46358 | }
46359 |
46360 |
46361 | static MA_INLINE void ma_pcm_deinterleave_s32__reference(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
46362 | {
46363 |     ma_int32** dst_s32 = (ma_int32**)dst;
46364 |     const ma_int32* src_s32 = (const ma_int32*)src;
46365 |
46366 |     ma_uint64 iFrame;
46367 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
46368 |         ma_uint32 iChannel;
46369 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
46370 |             dst_s32[iChannel][iFrame] = src_s32[iFrame*channels + iChannel];
46371 |         }
46372 |     }
46373 | }
46374 |
46375 | static MA_INLINE void ma_pcm_deinterleave_s32__optimized(void** dst, const void* src, ma_uint64
> frameCount, ma_uint32 channels)
46376 | {
46377 |     ma_pcm_deinterleave_s32__reference(dst, src, frameCount, channels);
46378 | }
46379 |
46380 | MA_API void ma_pcm_deinterleave_s32(void** dst, const void* src, ma_uint64 frameCount, ma_uint32
> channels)
46381 | {
46382 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46383 |     ma_pcm_deinterleave_s32__reference(dst, src, frameCount, channels);
46384 | #else
46385 |     ma_pcm_deinterleave_s32__optimized(dst, src, frameCount, channels);
46386 | #endif
46387 | }
46388 |
46389 |
46390 | /* f32 */
46391 | static MA_INLINE void ma_pcm_f32_to_u8__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46392 | {
46393 |     ma_uint64 i;
46394 |
46395 |     ma_uint8* dst_u8 = (ma_uint8*)dst;
46396 |     const float* src_f32 = (const float*)src;
46397 |
46398 |     float ditherMin = 0;
46399 |     float ditherMax = 0;
46400 |     if (ditherMode != ma_dither_mode_none) {
46401 |         ditherMin = 1.0f / -128;
46402 |         ditherMax = 1.0f / 127;
46403 |     }
46404 |
46405 |     for (i = 0; i < count; i += 1) {
46406 |         float x = src_f32[i];
46407 |         x = x + ma_dither_f32(ditherMode, ditherMin, ditherMax);
46408 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x)); /* clip */
46409 |         x = x + 1; /* -1..1 to 0..2 */
46410 |         x = x * 127.5f; /* 0..2 to 0..255 */
46411 |
46412 |         dst_u8[i] = (ma_uint8)x;
46413 |     }
46414 | }
46415 |
46416 | static MA_INLINE void ma_pcm_f32_to_u8__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46417 | {
46418 |     ma_pcm_f32_to_u8__reference(dst, src, count, ditherMode);
46419 | }
46420 |
46421 | #if defined(MA_SUPPORT_SSE2)
46422 | static MA_INLINE void ma_pcm_f32_to_u8__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46423 | {
46424 |     ma_pcm_f32_to_u8__optimized(dst, src, count, ditherMode);

```

```

46425 | }
46426 | #endif
46427 | #if defined(MA_SUPPORT_NEON)
46428 | static MA_INLINE void ma_pcm_f32_to_u8__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46429 | {
46430 |     ma_pcm_f32_to_u8__optimized(dst, src, count, ditherMode);
46431 | }
46432 | #endif
46433 |
46434 | MA_API void ma_pcm_f32_to_u8(void* dst, const void* src, ma_uint64 count, ma_dither_mode ditherMode)
46435 | {
46436 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46437 |     ma_pcm_f32_to_u8__reference(dst, src, count, ditherMode);
46438 | #else
46439 |     # if defined(MA_SUPPORT_SSE2)
46440 |         if (ma_has_sse2()) {
46441 |             ma_pcm_f32_to_u8__sse2(dst, src, count, ditherMode);
46442 |         } else
46443 |         #elif defined(MA_SUPPORT_NEON)
46444 |             if (ma_has_neon()) {
46445 |                 ma_pcm_f32_to_u8__neon(dst, src, count, ditherMode);
46446 |             } else
46447 |             #endif
46448 |             {
46449 |                 ma_pcm_f32_to_u8__optimized(dst, src, count, ditherMode);
46450 |             }
46451 | #endif
46452 | }
46453 |
46454 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46455 | static MA_INLINE void ma_pcm_f32_to_s16__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46456 | {
46457 |     ma_uint64 i;
46458 |
46459 |     ma_int16* dst_s16 = (ma_int16*)dst;
46460 |     const float* src_f32 = (const float*)src;
46461 |
46462 |     float ditherMin = 0;
46463 |     float ditherMax = 0;
46464 |     if (ditherMode != ma_dither_mode_none) {
46465 |         ditherMin = 1.0f / -32768;
46466 |         ditherMax = 1.0f / 32767;
46467 |     }
46468 |
46469 |     for (i = 0; i < count; i += 1) {
46470 |         float x = src_f32[i];
46471 |         x = x + ma_dither_f32(ditherMode, ditherMin, ditherMax);
46472 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x)); /* clip */
46473 |
46474 | #if 0
46475 |         /* The accurate way. */
46476 |         x = x + 1; /* -1..1 to 0..2 */
46477 |         x = x * 32767.5f; /* 0..2 to 0..65535 */
46478 |         x = x - 32768.0f; /* 0...65535 to -32768..32767 */
46479 | #else
46480 |         /* The fast way. */
46481 |         x = x * 32767.0f; /* -1..1 to -32767..32767 */
46482 | #endif
46483 |
46484 |         dst_s16[i] = (ma_int16)x;
46485 |     }
46486 | }
46487 | #else
46488 | static MA_INLINE void ma_pcm_f32_to_s16__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46489 | {
46490 |     ma_uint64 i;
46491 |     ma_uint64 i4;
46492 |     ma_uint64 count4;
46493 |
46494 |     ma_int16* dst_s16 = (ma_int16*)dst;
46495 |     const float* src_f32 = (const float*)src;
46496 |
46497 |     float ditherMin = 0;
46498 |     float ditherMax = 0;
46499 |     if (ditherMode != ma_dither_mode_none) {
46500 |         ditherMin = 1.0f / -32768;

```



```

46501 |         ditherMax = 1.0f / 32767;
46502 |     }
46503 |
46504 |     /* Unrolled. */
46505 |     i = 0;
46506 |     count4 = count >> 2;
46507 |     for (i4 = 0; i4 < count4; i4 += 1) {
46508 |         float d0 = ma_dither_f32(ditherMode, ditherMin, ditherMax);
46509 |         float d1 = ma_dither_f32(ditherMode, ditherMin, ditherMax);
46510 |         float d2 = ma_dither_f32(ditherMode, ditherMin, ditherMax);
46511 |         float d3 = ma_dither_f32(ditherMode, ditherMin, ditherMax);
46512 |
46513 |         float x0 = src_f32[i+0];
46514 |         float x1 = src_f32[i+1];
46515 |         float x2 = src_f32[i+2];
46516 |         float x3 = src_f32[i+3];
46517 |
46518 |         x0 = x0 + d0;
46519 |         x1 = x1 + d1;
46520 |         x2 = x2 + d2;
46521 |         x3 = x3 + d3;
46522 |
46523 |         x0 = ((x0 < -1) ? -1 : ((x0 > 1) ? 1 : x0));
46524 |         x1 = ((x1 < -1) ? -1 : ((x1 > 1) ? 1 : x1));
46525 |         x2 = ((x2 < -1) ? -1 : ((x2 > 1) ? 1 : x2));
46526 |         x3 = ((x3 < -1) ? -1 : ((x3 > 1) ? 1 : x3));
46527 |
46528 |         x0 = x0 * 32767.0f;
46529 |         x1 = x1 * 32767.0f;
46530 |         x2 = x2 * 32767.0f;
46531 |         x3 = x3 * 32767.0f;
46532 |
46533 |         dst_s16[i+0] = (ma_int16)x0;
46534 |         dst_s16[i+1] = (ma_int16)x1;
46535 |         dst_s16[i+2] = (ma_int16)x2;
46536 |         dst_s16[i+3] = (ma_int16)x3;
46537 |
46538 |         i += 4;
46539 |     }
46540 |
46541 |     /* Leftover. */
46542 |     for (; i < count; i += 1) {
46543 |         float x = src_f32[i];
46544 |         x = x + ma_dither_f32(ditherMode, ditherMin, ditherMax);
46545 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x)); /* clip */
46546 |         x = x * 32767.0f; /* -1..1 to -32767..32767 */
46547 |
46548 |         dst_s16[i] = (ma_int16)x;
46549 |     }
46550 | }
46551 |
46552 | #if defined(MA_SUPPORT_SSE2)
46553 | static MA_INLINE void ma_pcm_f32_to_s16__sse2(void* dst, const void* src, ma_uint64 count,
46554 | > ma_dither_mode ditherMode)
46555 | {
46556 |     ma_uint64 i;
46557 |     ma_uint64 i8;
46558 |     ma_uint64 count8;
46559 |     ma_int16* dst_s16;
46560 |     const float* src_f32;
46561 |     float ditherMin;
46562 |     float ditherMax;
46563 |
46564 |     /* Both the input and output buffers need to be aligned to 16 bytes. */
46565 |     if (((ma_uintptr)dst & 15) != 0 || ((ma_uintptr)src & 15) != 0) {
46566 |         ma_pcm_f32_to_s16__optimized(dst, src, count, ditherMode);
46567 |         return;
46568 |     }
46569 |
46570 |     dst_s16 = (ma_int16*)dst;
46571 |     src_f32 = (const float*)src;
46572 |
46573 |     ditherMin = 0;
46574 |     ditherMax = 0;
46575 |     if (ditherMode != ma_dither_mode_none) {
46576 |         ditherMin = 1.0f / -32768;
46577 |         ditherMax = 1.0f / 32767;
46578 |     }

```

```

46579 |     i = 0;
46580 |
46581 |     /* SSE2. SSE allows us to output 8 s16's at a time which means our loop is unrolled 8 times. */
46582 |     count8 = count >> 3;
46583 |     for (i8 = 0; i8 < count8; i8 += 1) {
46584 |         __m128 d0;
46585 |         __m128 d1;
46586 |         __m128 x0;
46587 |         __m128 x1;
46588 |
46589 |         if (ditherMode == ma_dither_mode_none) {
46590 |             d0 = _mm_set1_ps(0);
46591 |             d1 = _mm_set1_ps(0);
46592 |         } else if (ditherMode == ma_dither_mode_rectangle) {
46593 |             d0 = _mm_set_ps(
46594 |                 ma_dither_f32_rectangle(ditherMin, ditherMax),
46595 |                 ma_dither_f32_rectangle(ditherMin, ditherMax),
46596 |                 ma_dither_f32_rectangle(ditherMin, ditherMax),
46597 |                 ma_dither_f32_rectangle(ditherMin, ditherMax)
46598 |             );
46599 |             d1 = _mm_set_ps(
46600 |                 ma_dither_f32_rectangle(ditherMin, ditherMax),
46601 |                 ma_dither_f32_rectangle(ditherMin, ditherMax),
46602 |                 ma_dither_f32_rectangle(ditherMin, ditherMax),
46603 |                 ma_dither_f32_rectangle(ditherMin, ditherMax)
46604 |             );
46605 |         } else {
46606 |             d0 = _mm_set_ps(
46607 |                 ma_dither_f32_triangle(ditherMin, ditherMax),
46608 |                 ma_dither_f32_triangle(ditherMin, ditherMax),
46609 |                 ma_dither_f32_triangle(ditherMin, ditherMax),
46610 |                 ma_dither_f32_triangle(ditherMin, ditherMax)
46611 |             );
46612 |             d1 = _mm_set_ps(
46613 |                 ma_dither_f32_triangle(ditherMin, ditherMax),
46614 |                 ma_dither_f32_triangle(ditherMin, ditherMax),
46615 |                 ma_dither_f32_triangle(ditherMin, ditherMax),
46616 |                 ma_dither_f32_triangle(ditherMin, ditherMax)
46617 |             );
46618 |         }
46619 |
46620 |         x0 = *((__m128*)(src_f32 + i) + 0);
46621 |         x1 = *((__m128*)(src_f32 + i) + 1);
46622 |
46623 |         x0 = _mm_add_ps(x0, d0);
46624 |         x1 = _mm_add_ps(x1, d1);
46625 |
46626 |         x0 = _mm_mul_ps(x0, _mm_set1_ps(32767.0f));
46627 |         x1 = _mm_mul_ps(x1, _mm_set1_ps(32767.0f));
46628 |
46629 |         _mm_stream_si128((__m128i*)(dst_s16 + i)), _mm_packs_epi32(_mm_cvttss_epi32(x0),
46630 | > _mm_cvttss_epi32(x1));
46631 |
46632 |         i += 8;
46633 |     }
46634 |
46635 |     /* Leftover. */
46636 |     for (; i < count; i += 1) {
46637 |         float x = src_f32[i];
46638 |         x = x + ma_dither_f32(ditherMode, ditherMin, ditherMax);
46639 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x)); /* clip */
46640 |         x = x * 32767.0f; /* -1..1 to -32767..32767 */
46641 |
46642 |         dst_s16[i] = (ma_int16)x;
46643 |     }
46644 | }
46645 | #endif /* SSE2 */
46646 |
46647 | #if defined(MA_SUPPORT_NEON)
46648 | static MA_INLINE void ma_pcm_f32_to_s16_neon(void* dst, const void* src, ma_uint64 count,
46649 | > ma_dither_mode ditherMode)
46650 | {
46651 |     ma_uint64 i;
46652 |     ma_uint64 i8;
46653 |     ma_uint64 count8;
46654 |     ma_int16* dst_s16;
46655 |     const float* src_f32;
46656 |     float ditherMin;

```

```

46656 | float ditherMax;
46657 |
46658 | if (!ma_has_neon()) {
46659 |     ma_pcm_f32_to_s16_optimized(dst, src, count, ditherMode);
46660 |     return;
46661 | }
46662 |
46663 | /* Both the input and output buffers need to be aligned to 16 bytes. */
46664 | if (((ma_uintptr_t)dst & 15) != 0) || (((ma_uintptr_t)src & 15) != 0) {
46665 |     ma_pcm_f32_to_s16_optimized(dst, src, count, ditherMode);
46666 |     return;
46667 | }
46668 |
46669 | dst_s16 = (ma_int16_t*)dst;
46670 | src_f32 = (const float*)src;
46671 |
46672 | ditherMin = 0;
46673 | ditherMax = 0;
46674 | if (ditherMode != ma_dither_mode_none) {
46675 |     ditherMin = 1.0f / -32768;
46676 |     ditherMax = 1.0f / 32767;
46677 | }
46678 |
46679 | i = 0;
46680 |
46681 | /* NEON. NEON allows us to output 8 s16's at a time which means our loop is unrolled 8 times. */
46682 | count8 = count >> 3;
46683 | for (i8 = 0; i8 < count8; i8 += 1) {
46684 |     float32x4_t d0;
46685 |     float32x4_t d1;
46686 |     float32x4_t x0;
46687 |     float32x4_t x1;
46688 |     int32x4_t i0;
46689 |     int32x4_t i1;
46690 |
46691 |     if (ditherMode == ma_dither_mode_none) {
46692 |         d0 = vmovq_n_f32(0);
46693 |         d1 = vmovq_n_f32(0);
46694 |     } else if (ditherMode == ma_dither_mode_rectangle) {
46695 |         float d0v[4];
46696 |         float d1v[4];
46697 |
46698 |         d0v[0] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46699 |         d0v[1] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46700 |         d0v[2] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46701 |         d0v[3] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46702 |         d0 = vld1q_f32(d0v);
46703 |
46704 |         d1v[0] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46705 |         d1v[1] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46706 |         d1v[2] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46707 |         d1v[3] = ma_dither_f32_rectangle(ditherMin, ditherMax);
46708 |         d1 = vld1q_f32(d1v);
46709 |     } else {
46710 |         float d0v[4];
46711 |         float d1v[4];
46712 |
46713 |         d0v[0] = ma_dither_f32_triangle(ditherMin, ditherMax);
46714 |         d0v[1] = ma_dither_f32_triangle(ditherMin, ditherMax);
46715 |         d0v[2] = ma_dither_f32_triangle(ditherMin, ditherMax);
46716 |         d0v[3] = ma_dither_f32_triangle(ditherMin, ditherMax);
46717 |         d0 = vld1q_f32(d0v);
46718 |
46719 |         d1v[0] = ma_dither_f32_triangle(ditherMin, ditherMax);
46720 |         d1v[1] = ma_dither_f32_triangle(ditherMin, ditherMax);
46721 |         d1v[2] = ma_dither_f32_triangle(ditherMin, ditherMax);
46722 |         d1v[3] = ma_dither_f32_triangle(ditherMin, ditherMax);
46723 |         d1 = vld1q_f32(d1v);
46724 |     }
46725 |
46726 |     x0 = ((float32x4_t*)(src_f32 + i) + 0);
46727 |     x1 = ((float32x4_t*)(src_f32 + i) + 1);
46728 |
46729 |     x0 = vaddq_f32(x0, d0);
46730 |     x1 = vaddq_f32(x1, d1);
46731 |
46732 |     x0 = vmulq_n_f32(x0, 32767.0f);
46733 |     x1 = vmulq_n_f32(x1, 32767.0f);
46734 |

```

```

46735 |         i0 = vcvtq_s32_f32(x0);
46736 |         i1 = vcvtq_s32_f32(x1);
46737 |         *((int16x8_t*)(dst_s16 + i)) = vcombine_s16(vqmovn_s32(i0), vqmovn_s32(i1));
46738 |
46739 |         i += 8;
46740 |     }
46741 |
46742 |
46743 |     /* Leftover. */
46744 |     for (; i < count; i += 1) {
46745 |         float x = src_f32[i];
46746 |         x = x + ma_dither_f32(ditherMode, ditherMin, ditherMax);
46747 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x)); /* clip */
46748 |         x = x * 32767.0f; /* -1..1 to -32767..32767 */
46749 |
46750 |         dst_s16[i] = (ma_int16)x;
46751 |     }
46752 | }
46753 | #endif /* Neon */
46754 | #endif /* MA_USE_REFERENCE_CONVERSION_APIS */
46755 |
46756 | MA_API void ma_pcm_f32_to_s16(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46757 | {
46758 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46759 |     ma_pcm_f32_to_s16__reference(dst, src, count, ditherMode);
46760 | #else
46761 |     # if defined(MA_SUPPORT_SSE2)
46762 |         if (ma_has_sse2()) {
46763 |             ma_pcm_f32_to_s16__sse2(dst, src, count, ditherMode);
46764 |         } else
46765 |         #elif defined(MA_SUPPORT_NEON)
46766 |             if (ma_has_neon()) {
46767 |                 ma_pcm_f32_to_s16__neon(dst, src, count, ditherMode);
46768 |             } else
46769 |             #endif
46770 |             {
46771 |                 ma_pcm_f32_to_s16__optimized(dst, src, count, ditherMode);
46772 |             }
46773 | #endif
46774 | }
46775 |
46776 |
46777 | static MA_INLINE void ma_pcm_f32_to_s24__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46778 | {
46779 |     ma_uint8* dst_s24 = (ma_uint8*)dst;
46780 |     const float* src_f32 = (const float*)src;
46781 |
46782 |     ma_uint64 i;
46783 |     for (i = 0; i < count; i += 1) {
46784 |         ma_int32 r;
46785 |         float x = src_f32[i];
46786 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x)); /* clip */
46787 |
46788 |         #if 0
46789 |             /* The accurate way. */
46790 |             x = x + 1; /* -1..1 to 0..2 */
46791 |             x = x * 8388607.5f; /* 0..2 to 0..16777215 */
46792 |             x = x - 8388608.0f; /* 0..16777215 to -8388608..8388607 */
46793 |         #else
46794 |             /* The fast way. */
46795 |             x = x * 8388607.0f; /* -1..1 to -8388607..8388607 */
46796 |         #endif
46797 |
46798 |         r = (ma_int32)x;
46799 |         dst_s24[(i*3)+0] = (ma_uint8)((r & 0x0000FF) >> 0);
46800 |         dst_s24[(i*3)+1] = (ma_uint8)((r & 0x00FF00) >> 8);
46801 |         dst_s24[(i*3)+2] = (ma_uint8)((r & 0xFF0000) >> 16);
46802 |     }
46803 |
46804 |     (void)ditherMode; /* No dithering for f32 -> s24. */
46805 | }
46806 |
46807 | static MA_INLINE void ma_pcm_f32_to_s24__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46808 | {
46809 |     ma_pcm_f32_to_s24__reference(dst, src, count, ditherMode);
46810 | }

```

```

46811 |
46812 | #if defined(MA_SUPPORT_SSE2)
46813 | static MA_INLINE void ma_pcm_f32_to_s24__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46814 | {
46815 |     ma_pcm_f32_to_s24__optimized(dst, src, count, ditherMode);
46816 | }
46817 | #endif
46818 | #if defined(MA_SUPPORT_NEON)
46819 | static MA_INLINE void ma_pcm_f32_to_s24__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46820 | {
46821 |     ma_pcm_f32_to_s24__optimized(dst, src, count, ditherMode);
46822 | }
46823 | #endif
46824 |
46825 | MA_API void ma_pcm_f32_to_s24(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46826 | {
46827 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46828 |     ma_pcm_f32_to_s24__reference(dst, src, count, ditherMode);
46829 | #else
46830 |     # if defined(MA_SUPPORT_SSE2)
46831 |         if (ma_has_sse2()) {
46832 |             ma_pcm_f32_to_s24__sse2(dst, src, count, ditherMode);
46833 |         } else
46834 |         #elif defined(MA_SUPPORT_NEON)
46835 |             if (ma_has_neon()) {
46836 |                 ma_pcm_f32_to_s24__neon(dst, src, count, ditherMode);
46837 |             } else
46838 |             #endif
46839 |             {
46840 |                 ma_pcm_f32_to_s24__optimized(dst, src, count, ditherMode);
46841 |             }
46842 | #endif
46843 | }
46844 |
46845 |
46846 | static MA_INLINE void ma_pcm_f32_to_s32__reference(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46847 | {
46848 |     ma_int32* dst_s32 = (ma_int32*)dst;
46849 |     const float* src_f32 = (const float*)src;
46850 |
46851 |     ma_uint32 i;
46852 |     for (i = 0; i < count; i += 1) {
46853 |         double x = src_f32[i];
46854 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x));    /* clip */
46855 |
46856 |         #if 0
46857 |             /* The accurate way. */
46858 |             x = x + 1;                                /* -1..1 to 0..2 */
46859 |             x = x * 2147483647.5;                      /* 0..2 to 0..4294967295 */
46860 |             x = x - 2147483648.0;                      /* 0...4294967295 to -2147483648..2147483647 */
46861 |         #else
46862 |             /* The fast way. */
46863 |             x = x * 2147483647.0;                      /* -1..1 to -2147483647..2147483647 */
46864 |         #endif
46865 |
46866 |         dst_s32[i] = (ma_int32)x;
46867 |     }
46868 |
46869 |     (void)ditherMode;    /* No dithering for f32 -> s32. */
46870 | }
46871 |
46872 | static MA_INLINE void ma_pcm_f32_to_s32__optimized(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46873 | {
46874 |     ma_pcm_f32_to_s32__reference(dst, src, count, ditherMode);
46875 | }
46876 |
46877 | #if defined(MA_SUPPORT_SSE2)
46878 | static MA_INLINE void ma_pcm_f32_to_s32__sse2(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46879 | {
46880 |     ma_pcm_f32_to_s32__optimized(dst, src, count, ditherMode);
46881 | }
46882 | #endif
46883 | #if defined(MA_SUPPORT_NEON)

```

```

46884 | static MA_INLINE void ma_pcm_f32_to_s32__neon(void* dst, const void* src, ma_uint64 count,
> ma_dither_mode ditherMode)
46885 | {
46886 |     ma_pcm_f32_to_s32__optimized(dst, src, count, ditherMode);
46887 | }
46888 | #endif
46889 |
46890 | MA_API void ma_pcm_f32_to_s32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46891 | {
46892 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46893 |     ma_pcm_f32_to_s32__reference(dst, src, count, ditherMode);
46894 | #else
46895 |     # if defined(MA_SUPPORT_SSE2)
46896 |         if (ma_has_sse2()) {
46897 |             ma_pcm_f32_to_s32__sse2(dst, src, count, ditherMode);
46898 |         } else
46899 |         #elif defined(MA_SUPPORT_NEON)
46900 |             if (ma_has_neon()) {
46901 |                 ma_pcm_f32_to_s32__neon(dst, src, count, ditherMode);
46902 |             } else
46903 |             #endif
46904 |             {
46905 |                 ma_pcm_f32_to_s32__optimized(dst, src, count, ditherMode);
46906 |             }
46907 | #endif
46908 | }
46909 |
46910 |
46911 | MA_API void ma_pcm_f32_to_f32(void* dst, const void* src, ma_uint64 count, ma_dither_mode
> ditherMode)
46912 | {
46913 |     (void)ditherMode;
46914 |
46915 |     ma_copy_memory_64(dst, src, count * sizeof(float));
46916 | }
46917 |
46918 |
46919 | static void ma_pcm_interleave_f32__reference(void* dst, const void** src, ma_uint64 frameCount,
> ma_uint32 channels)
46920 | {
46921 |     float* dst_f32 = (float*)dst;
46922 |     const float** src_f32 = (const float**)src;
46923 |
46924 |     ma_uint64 iFrame;
46925 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
46926 |         ma_uint32 iChannel;
46927 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
46928 |             dst_f32[iFrame*channels + iChannel] = src_f32[iChannel][iFrame];
46929 |         }
46930 |     }
46931 | }
46932 |
46933 | static void ma_pcm_interleave_f32__optimized(void* dst, const void** src, ma_uint64 frameCount,
> ma_uint32 channels)
46934 | {
46935 |     ma_pcm_interleave_f32__reference(dst, src, frameCount, channels);
46936 | }
46937 |
46938 | MA_API void ma_pcm_interleave_f32(void* dst, const void** src, ma_uint64 frameCount, ma_uint32
> channels)
46939 | {
46940 | #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46941 |     ma_pcm_interleave_f32__reference(dst, src, frameCount, channels);
46942 | #else
46943 |     ma_pcm_interleave_f32__optimized(dst, src, frameCount, channels);
46944 | #endif
46945 | }
46946 |
46947 |
46948 | static void ma_pcm_deinterleave_f32__reference(void** dst, const void* src, ma_uint64 frameCount,
> ma_uint32 channels)
46949 | {
46950 |     float** dst_f32 = (float**)dst;
46951 |     const float* src_f32 = (const float*)src;
46952 |
46953 |     ma_uint64 iFrame;
46954 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
46955 |         ma_uint32 iChannel;

```

```

46956 |         for (iChannel = 0; iChannel < channels; iChannel += 1) {
46957 |             dst_f32[iChannel][iFrame] = src_f32[iFrame*channels + iChannel];
46958 |         }
46959 |     }
46960 | }
46961 |
46962 | static void ma_pcm_deinterleave_f32__optimized(void** dst, const void* src, ma_uint64 frameCount,
> ma_uint32 channels)
46963 | {
46964 |     ma_pcm_deinterleave_f32__reference(dst, src, frameCount, channels);
46965 | }
46966 |
46967 | MA_API void ma_pcm_deinterleave_f32(void** dst, const void* src, ma_uint64 frameCount, ma_uint32
> channels)
46968 | {
46969 |     #ifdef MA_USE_REFERENCE_CONVERSION_APIS
46970 |         ma_pcm_deinterleave_f32__reference(dst, src, frameCount, channels);
46971 |     #else
46972 |         ma_pcm_deinterleave_f32__optimized(dst, src, frameCount, channels);
46973 |     #endif
46974 | }
46975 |
46976 |
46977 | MA_API void ma_pcm_convert(void* pOut, ma_format formatOut, const void* pIn, ma_format formatIn,
> ma_uint64 sampleCount, ma_dither_mode ditherMode)
46978 | {
46979 |     if (formatOut == formatIn) {
46980 |         ma_copy_memory_64(pOut, pIn, sampleCount * ma_get_bytes_per_sample(formatOut));
46981 |         return;
46982 |     }
46983 |
46984 |     switch (formatIn)
46985 |     {
46986 |         case ma_format_u8:
46987 |         {
46988 |             switch (formatOut)
46989 |             {
46990 |                 case ma_format_s16: ma_pcm_u8_to_s16(pOut, pIn, sampleCount, ditherMode); return;
46991 |                 case ma_format_s24: ma_pcm_u8_to_s24(pOut, pIn, sampleCount, ditherMode); return;
46992 |                 case ma_format_s32: ma_pcm_u8_to_s32(pOut, pIn, sampleCount, ditherMode); return;
46993 |                 case ma_format_f32: ma_pcm_u8_to_f32(pOut, pIn, sampleCount, ditherMode); return;
46994 |                 default: break;
46995 |             }
46996 |         } break;
46997 |
46998 |         case ma_format_s16:
46999 |         {
47000 |             switch (formatOut)
47001 |             {
47002 |                 case ma_format_u8: ma_pcm_s16_to_u8( pOut, pIn, sampleCount, ditherMode); return;
47003 |                 case ma_format_s24: ma_pcm_s16_to_s24(pOut, pIn, sampleCount, ditherMode); return;
47004 |                 case ma_format_s32: ma_pcm_s16_to_s32(pOut, pIn, sampleCount, ditherMode); return;
47005 |                 case ma_format_f32: ma_pcm_s16_to_f32(pOut, pIn, sampleCount, ditherMode); return;
47006 |                 default: break;
47007 |             }
47008 |         } break;
47009 |
47010 |         case ma_format_s24:
47011 |         {
47012 |             switch (formatOut)
47013 |             {
47014 |                 case ma_format_u8: ma_pcm_s24_to_u8( pOut, pIn, sampleCount, ditherMode); return;
47015 |                 case ma_format_s16: ma_pcm_s24_to_s16(pOut, pIn, sampleCount, ditherMode); return;
47016 |                 case ma_format_s32: ma_pcm_s24_to_s32(pOut, pIn, sampleCount, ditherMode); return;
47017 |                 case ma_format_f32: ma_pcm_s24_to_f32(pOut, pIn, sampleCount, ditherMode); return;
47018 |                 default: break;
47019 |             }
47020 |         } break;
47021 |
47022 |         case ma_format_s32:
47023 |         {
47024 |             switch (formatOut)
47025 |             {
47026 |                 case ma_format_u8: ma_pcm_s32_to_u8( pOut, pIn, sampleCount, ditherMode); return;
47027 |                 case ma_format_s16: ma_pcm_s32_to_s16(pOut, pIn, sampleCount, ditherMode); return;
47028 |                 case ma_format_s24: ma_pcm_s32_to_s24(pOut, pIn, sampleCount, ditherMode); return;
47029 |                 case ma_format_f32: ma_pcm_s32_to_f32(pOut, pIn, sampleCount, ditherMode); return;
47030 |                 default: break;
47031 |             }

```



```

47032 |         } break;
47033 |
47034 |         case ma_format_f32:
47035 |         {
47036 |             switch (formatOut)
47037 |             {
47038 |                 case ma_format_u8: ma_pcm_f32_to_u8( pOut, pIn, sampleCount, ditherMode); return;
47039 |                 case ma_format_s16: ma_pcm_f32_to_s16(pOut, pIn, sampleCount, ditherMode); return;
47040 |                 case ma_format_s24: ma_pcm_f32_to_s24(pOut, pIn, sampleCount, ditherMode); return;
47041 |                 case ma_format_s32: ma_pcm_f32_to_s32(pOut, pIn, sampleCount, ditherMode); return;
47042 |                 default: break;
47043 |             }
47044 |         } break;
47045 |
47046 |         default: break;
47047 |     }
47048 | }
47049 |
47050 | MA_API void ma_convert_pcm_frames_format(void* pOut, ma_format formatOut, const void* pIn, ma_format
> formatIn, ma_uint64 frameCount, ma_uint32 channels, ma_dither_mode ditherMode)
47051 | {
47052 |     ma_pcm_convert(pOut, formatOut, pIn, formatIn, frameCount * channels, ditherMode);
47053 | }
47054 |
47055 | MA_API void ma_deinterleave_pcm_frames(ma_format format, ma_uint32 channels, ma_uint64 frameCount,
> const void* pInterleavedPCMFrames, void** ppDeinterleavedPCMFrames)
47056 | {
47057 |     if (pInterleavedPCMFrames == NULL || ppDeinterleavedPCMFrames == NULL) {
47058 |         return; /* Invalid args. */
47059 |     }
47060 |
47061 |     /* For efficiency we do this per format. */
47062 |     switch (format) {
47063 |         case ma_format_s16:
47064 |         {
47065 |             const ma_int16* pSrcS16 = (const ma_int16*)pInterleavedPCMFrames;
47066 |             ma_uint64 iPCMFrame;
47067 |             for (iPCMFrame = 0; iPCMFrame < frameCount; ++iPCMFrame) {
47068 |                 ma_uint32 iChannel;
47069 |                 for (iChannel = 0; iChannel < channels; ++iChannel) {
47070 |                     ma_int16* pDstS16 = (ma_int16*)ppDeinterleavedPCMFrames[iChannel];
47071 |                     pDstS16[iPCMFrame] = pSrcS16[iPCMFrame*channels+iChannel];
47072 |                 }
47073 |             }
47074 |         } break;
47075 |
47076 |         case ma_format_f32:
47077 |         {
47078 |             const float* pSrcF32 = (const float*)pInterleavedPCMFrames;
47079 |             ma_uint64 iPCMFrame;
47080 |             for (iPCMFrame = 0; iPCMFrame < frameCount; ++iPCMFrame) {
47081 |                 ma_uint32 iChannel;
47082 |                 for (iChannel = 0; iChannel < channels; ++iChannel) {
47083 |                     float* pDstF32 = (float*)ppDeinterleavedPCMFrames[iChannel];
47084 |                     pDstF32[iPCMFrame] = pSrcF32[iPCMFrame*channels+iChannel];
47085 |                 }
47086 |             }
47087 |         } break;
47088 |
47089 |         default:
47090 |         {
47091 |             ma_uint32 sampleSizeInBytes = ma_get_bytes_per_sample(format);
47092 |             ma_uint64 iPCMFrame;
47093 |             for (iPCMFrame = 0; iPCMFrame < frameCount; ++iPCMFrame) {
47094 |                 ma_uint32 iChannel;
47095 |                 for (iChannel = 0; iChannel < channels; ++iChannel) {
47096 |                     void* pDst = ma_offset_ptr(ppDeinterleavedPCMFrames[iChannel],
> iPCMFrame*sampleSizeInBytes);
47097 |                     const void* pSrc = ma_offset_ptr(pInterleavedPCMFrames,
> (iPCMFrame*channels+iChannel)*sampleSizeInBytes);
47098 |                     memcpy(pDst, pSrc, sampleSizeInBytes);
47099 |                 }
47100 |             }
47101 |         } break;
47102 |     }
47103 | }
47104 |
47105 | MA_API void ma_interleave_pcm_frames(ma_format format, ma_uint32 channels, ma_uint64 frameCount,
> const void** ppDeinterleavedPCMFrames, void* pInterleavedPCMFrames)

```

```

47106 | {
47107 |     switch (format)
47108 |     {
47109 |         case ma_format_s16:
47110 |         {
47111 |             ma_int16* pDstS16 = (ma_int16*)pInterleavedPCMFrames;
47112 |             ma_uint64 iPCMFrame;
47113 |             for (iPCMFrame = 0; iPCMFrame < frameCount; ++iPCMFrame) {
47114 |                 ma_uint32 iChannel;
47115 |                 for (iChannel = 0; iChannel < channels; ++iChannel) {
47116 |                     const ma_int16* pSrcS16 = (const ma_int16*)ppDeinterleavedPCMFrames[iChannel];
47117 |                     pDstS16[iPCMFrame*channels+iChannel] = pSrcS16[iPCMFrame];
47118 |                 }
47119 |             }
47120 |         } break;
47121 |
47122 |         case ma_format_f32:
47123 |         {
47124 |             float* pDstF32 = (float*)pInterleavedPCMFrames;
47125 |             ma_uint64 iPCMFrame;
47126 |             for (iPCMFrame = 0; iPCMFrame < frameCount; ++iPCMFrame) {
47127 |                 ma_uint32 iChannel;
47128 |                 for (iChannel = 0; iChannel < channels; ++iChannel) {
47129 |                     const float* pSrcF32 = (const float*)ppDeinterleavedPCMFrames[iChannel];
47130 |                     pDstF32[iPCMFrame*channels+iChannel] = pSrcF32[iPCMFrame];
47131 |                 }
47132 |             }
47133 |         } break;
47134 |
47135 |         default:
47136 |         {
47137 |             ma_uint32 sampleSizeInBytes = ma_get_bytes_per_sample(format);
47138 |             ma_uint64 iPCMFrame;
47139 |             for (iPCMFrame = 0; iPCMFrame < frameCount; ++iPCMFrame) {
47140 |                 ma_uint32 iChannel;
47141 |                 for (iChannel = 0; iChannel < channels; ++iChannel) {
47142 |                     void* pDst = ma_offset_ptr(pInterleavedPCMFrames,
47143 | > (iPCMFrame*channels+iChannel)*sampleSizeInBytes);
47144 |                     const void* pSrc = ma_offset_ptr(ppDeinterleavedPCMFrames[iChannel],
47145 | > iPCMFrame*sampleSizeInBytes);
47146 |                     memcpy(pDst, pSrc, sampleSizeInBytes);
47147 |                 }
47148 |             } break;
47149 |         }
47150 |     }
47151 | }
47152 | /*****
47153 | > *****/
47154 | Biquad Filter
47155 |
47156 | *****/
47157 | > *****/
47157 | #ifndef MA_BIQUAD_FIXED_POINT_SHIFT
47158 | #define MA_BIQUAD_FIXED_POINT_SHIFT 14
47159 | #endif
47160 |
47161 | static ma_int32 ma_biquad_float_to_fp(double x)
47162 | {
47163 |     return (ma_int32)(x * (1 << MA_BIQUAD_FIXED_POINT_SHIFT));
47164 | }
47165 |
47166 | MA_API ma_biquad_config ma_biquad_config_init(ma_format format, ma_uint32 channels, double b0,
47167 | > double b1, double b2, double a0, double a1, double a2)
47168 | {
47169 |     ma_biquad_config config;
47170 |
47171 |     MA_ZERO_OBJECT(&config);
47172 |     config.format = format;
47173 |     config.channels = channels;
47174 |     config.b0 = b0;
47175 |     config.b1 = b1;
47176 |     config.b2 = b2;
47177 |     config.a0 = a0;
47178 |     config.a1 = a1;
47179 |     config.a2 = a2;

```

```

47180 |     return config;
47181 | }
47182 |
47183 |
47184 | typedef struct
47185 | {
47186 |     size_t sizeInBytes;
47187 |     size_t r10ffset;
47188 |     size_t r20ffset;
47189 | } ma_biquad_heap_layout;
47190 |
47191 | static ma_result ma_biquad_get_heap_layout(const ma_biquad_config* pConfig, ma_biquad_heap_layout*
> pHeapLayout)
47192 | {
47193 |     MA_ASSERT(pHeapLayout != NULL);
47194 |
47195 |     MA_ZERO_OBJECT(pHeapLayout);
47196 |
47197 |     if (pConfig == NULL) {
47198 |         return MA_INVALID_ARGS;
47199 |     }
47200 |
47201 |     if (pConfig->channels == 0) {
47202 |         return MA_INVALID_ARGS;
47203 |     }
47204 |
47205 |     pHeapLayout->sizeInBytes = 0;
47206 |
47207 |     /* R0 */
47208 |     pHeapLayout->r10ffset = pHeapLayout->sizeInBytes;
47209 |     pHeapLayout->sizeInBytes += sizeof(ma_biquad_coefficient) * pConfig->channels;
47210 |
47211 |     /* R1 */
47212 |     pHeapLayout->r20ffset = pHeapLayout->sizeInBytes;
47213 |     pHeapLayout->sizeInBytes += sizeof(ma_biquad_coefficient) * pConfig->channels;
47214 |
47215 |     /* Make sure allocation size is aligned. */
47216 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
47217 |
47218 |     return MA_SUCCESS;
47219 | }
47220 |
47221 | MA_API ma_result ma_biquad_get_heap_size(const ma_biquad_config* pConfig, size_t* pHeapSizeInBytes)
47222 | {
47223 |     ma_result result;
47224 |     ma_biquad_heap_layout heapLayout;
47225 |
47226 |     if (pHeapSizeInBytes == NULL) {
47227 |         return MA_INVALID_ARGS;
47228 |     }
47229 |
47230 |     *pHeapSizeInBytes = 0;
47231 |
47232 |     result = ma_biquad_get_heap_layout(pConfig, &heapLayout);
47233 |     if (result != MA_SUCCESS) {
47234 |         return result;
47235 |     }
47236 |
47237 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
47238 |
47239 |     return MA_SUCCESS;
47240 | }
47241 |
47242 | MA_API ma_result ma_biquad_init_preallocated(const ma_biquad_config* pConfig, void* pHeap,
> ma_biquad* pBQ)
47243 | {
47244 |     ma_result result;
47245 |     ma_biquad_heap_layout heapLayout;
47246 |
47247 |     if (pBQ == NULL) {
47248 |         return MA_INVALID_ARGS;
47249 |     }
47250 |
47251 |     MA_ZERO_OBJECT(pBQ);
47252 |
47253 |     result = ma_biquad_get_heap_layout(pConfig, &heapLayout);
47254 |     if (result != MA_SUCCESS) {
47255 |         return result;
47256 |     }

```

```

47257 |
47258 |     pBQ->pHeap = pHeap;
47259 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
47260 |
47261 |     pBQ->pR1 = (ma_biquad_coefficient*)ma_offset_ptr(pHeap, heapLayout.r1Offset);
47262 |     pBQ->pR2 = (ma_biquad_coefficient*)ma_offset_ptr(pHeap, heapLayout.r2Offset);
47263 |
47264 |     return ma_biquad_reinit(pConfig, pBQ);
47265 | }
47266 |
47267 | MA_API ma_result ma_biquad_init(const ma_biquad_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_biquad* pBQ)
47268 | {
47269 |     ma_result result;
47270 |     size_t heapSizeInBytes;
47271 |     void* pHeap;
47272 |
47273 |     result = ma_biquad_get_heap_size(pConfig, &heapSizeInBytes);
47274 |     if (result != MA_SUCCESS) {
47275 |         return result;
47276 |     }
47277 |
47278 |     if (heapSizeInBytes > 0) {
47279 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
47280 |         if (pHeap == NULL) {
47281 |             return MA_OUT_OF_MEMORY;
47282 |         }
47283 |     } else {
47284 |         pHeap = NULL;
47285 |     }
47286 |
47287 |     result = ma_biquad_init_preallocated(pConfig, pHeap, pBQ);
47288 |     if (result != MA_SUCCESS) {
47289 |         ma_free(pHeap, pAllocationCallbacks);
47290 |         return result;
47291 |     }
47292 |
47293 |     pBQ->ownsHeap = MA_TRUE;
47294 |     return MA_SUCCESS;
47295 | }
47296 |
47297 | MA_API void ma_biquad_uninit(ma_biquad* pBQ, const ma_allocation_callbacks* pAllocationCallbacks)
47298 | {
47299 |     if (pBQ == NULL) {
47300 |         return;
47301 |     }
47302 |
47303 |     if (pBQ->ownsHeap) {
47304 |         ma_free(pBQ->pHeap, pAllocationCallbacks);
47305 |     }
47306 | }
47307 |
47308 | MA_API ma_result ma_biquad_reinit(const ma_biquad_config* pConfig, ma_biquad* pBQ)
47309 | {
47310 |     if (pBQ == NULL || pConfig == NULL) {
47311 |         return MA_INVALID_ARGS;
47312 |     }
47313 |
47314 |     if (pConfig->a0 == 0) {
47315 |         return MA_INVALID_ARGS; /* Division by zero. */
47316 |     }
47317 |
47318 |     /* Only supporting f32 and s16. */
47319 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
47320 |         return MA_INVALID_ARGS;
47321 |     }
47322 |
47323 |     /* The format cannot be changed after initialization. */
47324 |     if (pBQ->format != ma_format_unknown && pBQ->format != pConfig->format) {
47325 |         return MA_INVALID_OPERATION;
47326 |     }
47327 |
47328 |     /* The channel count cannot be changed after initialization. */
47329 |     if (pBQ->channels != 0 && pBQ->channels != pConfig->channels) {
47330 |         return MA_INVALID_OPERATION;
47331 |     }
47332 |
47333 |     pBQ->format = pConfig->format;

```

```

47335 |     pBQ->channels = pConfig->channels;
47336 |
47337 |     /* Normalize. */
47338 |     if (pConfig->format == ma_format_f32) {
47339 |         pBQ->b0.f32 = (float)(pConfig->b0 / pConfig->a0);
47340 |         pBQ->b1.f32 = (float)(pConfig->b1 / pConfig->a0);
47341 |         pBQ->b2.f32 = (float)(pConfig->b2 / pConfig->a0);
47342 |         pBQ->a1.f32 = (float)(pConfig->a1 / pConfig->a0);
47343 |         pBQ->a2.f32 = (float)(pConfig->a2 / pConfig->a0);
47344 |     } else {
47345 |         pBQ->b0.s32 = ma_biquad_float_to_fp(pConfig->b0 / pConfig->a0);
47346 |         pBQ->b1.s32 = ma_biquad_float_to_fp(pConfig->b1 / pConfig->a0);
47347 |         pBQ->b2.s32 = ma_biquad_float_to_fp(pConfig->b2 / pConfig->a0);
47348 |         pBQ->a1.s32 = ma_biquad_float_to_fp(pConfig->a1 / pConfig->a0);
47349 |         pBQ->a2.s32 = ma_biquad_float_to_fp(pConfig->a2 / pConfig->a0);
47350 |     }
47351 |
47352 |     return MA_SUCCESS;
47353 | }
47354 |
47355 | MA_API ma_result ma_biquad_clear_cache(ma_biquad* pBQ)
47356 | {
47357 |     if (pBQ == NULL) {
47358 |         return MA_INVALID_ARGS;
47359 |     }
47360 |
47361 |     if (pBQ->format == ma_format_f32) {
47362 |         pBQ->pR1->f32 = 0;
47363 |         pBQ->pR2->f32 = 0;
47364 |     } else {
47365 |         pBQ->pR1->s32 = 0;
47366 |         pBQ->pR2->s32 = 0;
47367 |     }
47368 |
47369 |     return MA_SUCCESS;
47370 | }
47371 |
47372 | static MA_INLINE void ma_biquad_process_pcm_frame_f32__direct_form_2_transposed(ma_biquad* pBQ,
> float* pY, const float* pX)
47373 | {
47374 |     ma_uint32 c;
47375 |     const ma_uint32 channels = pBQ->channels;
47376 |     const float b0 = pBQ->b0.f32;
47377 |     const float b1 = pBQ->b1.f32;
47378 |     const float b2 = pBQ->b2.f32;
47379 |     const float a1 = pBQ->a1.f32;
47380 |     const float a2 = pBQ->a2.f32;
47381 |
47382 |     MA_ASSUME(channels > 0);
47383 |     for (c = 0; c < channels; c += 1) {
47384 |         float r1 = pBQ->pR1[c].f32;
47385 |         float r2 = pBQ->pR2[c].f32;
47386 |         float x = pX[c];
47387 |         float y;
47388 |
47389 |         y = b0*x + r1;
47390 |         r1 = b1*x - a1*y + r2;
47391 |         r2 = b2*x - a2*y;
47392 |
47393 |         pY[c] = y;
47394 |         pBQ->pR1[c].f32 = r1;
47395 |         pBQ->pR2[c].f32 = r2;
47396 |     }
47397 | }
47398 |
47399 | static MA_INLINE void ma_biquad_process_pcm_frame_f32(ma_biquad* pBQ, float* pY, const float* pX)
47400 | {
47401 |     ma_biquad_process_pcm_frame_f32__direct_form_2_transposed(pBQ, pY, pX);
47402 | }
47403 |
47404 | static MA_INLINE void ma_biquad_process_pcm_frame_s16__direct_form_2_transposed(ma_biquad* pBQ,
> ma_int16* pY, const ma_int16* pX)
47405 | {
47406 |     ma_uint32 c;
47407 |     const ma_uint32 channels = pBQ->channels;
47408 |     const ma_int32 b0 = pBQ->b0.s32;
47409 |     const ma_int32 b1 = pBQ->b1.s32;
47410 |     const ma_int32 b2 = pBQ->b2.s32;
47411 |     const ma_int32 a1 = pBQ->a1.s32;

```

```

47412 |     const ma_int32 a2 = pBQ->a2.s32;
47413 |
47414 |     MA_ASSUME(channels > 0);
47415 |     for (c = 0; c < channels; c += 1) {
47416 |         ma_int32 r1 = pBQ->pR1[c].s32;
47417 |         ma_int32 r2 = pBQ->pR2[c].s32;
47418 |         ma_int32 x  = pX[c];
47419 |         ma_int32 y;
47420 |
47421 |         y = (b0*x      + r1) >> MA_BIQUAD_FIXED_POINT_SHIFT;
47422 |         r1 = (b1*x - a1*y + r2);
47423 |         r2 = (b2*x - a2*y);
47424 |
47425 |         pY[c] = (ma_int16)ma_clamp(y, -32768, 32767);
47426 |         pBQ->pR1[c].s32 = r1;
47427 |         pBQ->pR2[c].s32 = r2;
47428 |     }
47429 | }
47430 |
47431 | static MA_INLINE void ma_biquad_process_pcm_frame_s16(ma_biquad* pBQ, ma_int16* pY, const ma_int16*
> pX)
47432 | {
47433 |     ma_biquad_process_pcm_frame_s16__direct_form_2_transposed(pBQ, pY, pX);
47434 | }
47435 |
47436 | MA_API ma_result ma_biquad_process_pcm_frames(ma_biquad* pBQ, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
47437 | {
47438 |     ma_uint32 n;
47439 |
47440 |     if (pBQ == NULL || pFramesOut == NULL || pFramesIn == NULL) {
47441 |         return MA_INVALID_ARGS;
47442 |     }
47443 |
47444 |     /* Note that the logic below needs to support in-place filtering. That is, it must support the
> case where pFramesOut and pFramesIn are the same. */
47445 |
47446 |     if (pBQ->format == ma_format_f32) {
47447 |         /* */ float* pY = (float*)pFramesOut;
47448 |         const float* pX = (const float*)pFramesIn;
47449 |
47450 |         for (n = 0; n < frameCount; n += 1) {
47451 |             ma_biquad_process_pcm_frame_f32__direct_form_2_transposed(pBQ, pY, pX);
47452 |             pY += pBQ->channels;
47453 |             pX += pBQ->channels;
47454 |         }
47455 |     } else if (pBQ->format == ma_format_s16) {
47456 |         /* */ ma_int16* pY = (ma_int16*)pFramesOut;
47457 |         const ma_int16* pX = (const ma_int16*)pFramesIn;
47458 |
47459 |         for (n = 0; n < frameCount; n += 1) {
47460 |             ma_biquad_process_pcm_frame_s16__direct_form_2_transposed(pBQ, pY, pX);
47461 |             pY += pBQ->channels;
47462 |             pX += pBQ->channels;
47463 |         }
47464 |     } else {
47465 |         MA_ASSERT(MA_FALSE);
47466 |         return MA_INVALID_ARGS; /* Format not supported. Should never hit this because it's checked
> in ma_biquad_init() and ma_biquad_reinit(). */
47467 |     }
47468 |
47469 |     return MA_SUCCESS;
47470 | }
47471 |
47472 | MA_API ma_uint32 ma_biquad_get_latency(const ma_biquad* pBQ)
47473 | {
47474 |     if (pBQ == NULL) {
47475 |         return 0;
47476 |     }
47477 |
47478 |     return 2;
47479 | }
47480 |
47481 |
47482 | /*****
> *****/
47483 |
47484 | Low-Pass Filter
47485 |

```

```

47486 | *****
> *****/
47487 | MA_API ma_lpf1_config ma_lpf1_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency)
47488 | {
47489 |     ma_lpf1_config config;
47490 |
47491 |     MA_ZERO_OBJECT(&config);
47492 |     config.format = format;
47493 |     config.channels = channels;
47494 |     config.sampleRate = sampleRate;
47495 |     config.cutoffFrequency = cutoffFrequency;
47496 |     config.q = 0.5;
47497 |
47498 |     return config;
47499 | }
47500 |
47501 | MA_API ma_lpf2_config ma_lpf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency, double q)
47502 | {
47503 |     ma_lpf2_config config;
47504 |
47505 |     MA_ZERO_OBJECT(&config);
47506 |     config.format = format;
47507 |     config.channels = channels;
47508 |     config.sampleRate = sampleRate;
47509 |     config.cutoffFrequency = cutoffFrequency;
47510 |     config.q = q;
47511 |
47512 |     /* Q cannot be 0 or else it'll result in a division by 0. In this case just default to 0.707107.
> */
47513 |     if (config.q == 0) {
47514 |         config.q = 0.707107;
47515 |     }
47516 |
47517 |     return config;
47518 | }
47519 |
47520 |
47521 | typedef struct
47522 | {
47523 |     size_t sizeInBytes;
47524 |     size_t r10ffset;
47525 | } ma_lpf1_heap_layout;
47526 |
47527 | static ma_result ma_lpf1_get_heap_layout(const ma_lpf1_config* pConfig, ma_lpf1_heap_layout*
> pHeapLayout)
47528 | {
47529 |     MA_ASSERT(pHeapLayout != NULL);
47530 |
47531 |     MA_ZERO_OBJECT(pHeapLayout);
47532 |
47533 |     if (pConfig == NULL) {
47534 |         return MA_INVALID_ARGS;
47535 |     }
47536 |
47537 |     if (pConfig->channels == 0) {
47538 |         return MA_INVALID_ARGS;
47539 |     }
47540 |
47541 |     pHeapLayout->sizeInBytes = 0;
47542 |
47543 |     /* R1 */
47544 |     pHeapLayout->r10ffset = pHeapLayout->sizeInBytes;
47545 |     pHeapLayout->sizeInBytes += sizeof(ma_biquad_coefficient) * pConfig->channels;
47546 |
47547 |     /* Make sure allocation size is aligned. */
47548 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
47549 |
47550 |     return MA_SUCCESS;
47551 | }
47552 |
47553 | MA_API ma_result ma_lpf1_get_heap_size(const ma_lpf1_config* pConfig, size_t* pHeapSizeInBytes)
47554 | {
47555 |     ma_result result;
47556 |     ma_lpf1_heap_layout heapLayout;
47557 |
47558 |     if (pHeapSizeInBytes == NULL) {
47559 |         return MA_INVALID_ARGS;

```



```

47560 |     }
47561 |
47562 |     result = ma_lpf1_get_heap_layout(pConfig, &heapLayout);
47563 |     if (result != MA_SUCCESS) {
47564 |         return result;
47565 |     }
47566 |
47567 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
47568 |
47569 |     return MA_SUCCESS;
47570 | }
47571 |
47572 | MA_API ma_result ma_lpf1_init_preallocated(const ma_lpf1_config* pConfig, void* pHeap, ma_lpf1*
> pLPF)
47573 | {
47574 |     ma_result result;
47575 |     ma_lpf1_heap_layout heapLayout;
47576 |
47577 |     if (pLPF == NULL) {
47578 |         return MA_INVALID_ARGS;
47579 |     }
47580 |
47581 |     MA_ZERO_OBJECT(pLPF);
47582 |
47583 |     result = ma_lpf1_get_heap_layout(pConfig, &heapLayout);
47584 |     if (result != MA_SUCCESS) {
47585 |         return result;
47586 |     }
47587 |
47588 |     pLPF->pHeap = pHeap;
47589 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
47590 |
47591 |     pLPF->pR1 = (ma_biquad_coefficient*)ma_offset_ptr(pHeap, heapLayout.r10ffset);
47592 |
47593 |     return ma_lpf1_reinit(pConfig, pLPF);
47594 | }
47595 |
47596 | MA_API ma_result ma_lpf1_init(const ma_lpf1_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_lpf1* pLPF)
47597 | {
47598 |     ma_result result;
47599 |     size_t heapSizeInBytes;
47600 |     void* pHeap;
47601 |
47602 |     result = ma_lpf1_get_heap_size(pConfig, &heapSizeInBytes);
47603 |     if (result != MA_SUCCESS) {
47604 |         return result;
47605 |     }
47606 |
47607 |     if (heapSizeInBytes > 0) {
47608 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
47609 |         if (pHeap == NULL) {
47610 |             return MA_OUT_OF_MEMORY;
47611 |         }
47612 |     } else {
47613 |         pHeap = NULL;
47614 |     }
47615 |
47616 |     result = ma_lpf1_init_preallocated(pConfig, pHeap, pLPF);
47617 |     if (result != MA_SUCCESS) {
47618 |         ma_free(pHeap, pAllocationCallbacks);
47619 |         return result;
47620 |     }
47621 |
47622 |     pLPF->ownsHeap = MA_TRUE;
47623 |     return MA_SUCCESS;
47624 | }
47625 |
47626 | MA_API void ma_lpf1_uninit(ma_lpf1* pLPF, const ma_allocation_callbacks* pAllocationCallbacks)
47627 | {
47628 |     if (pLPF == NULL) {
47629 |         return;
47630 |     }
47631 |
47632 |     if (pLPF->ownsHeap) {
47633 |         ma_free(pLPF->pHeap, pAllocationCallbacks);
47634 |     }
47635 | }
47636 |

```

```

47637 | MA_API ma_result ma_lpf1_reinit(const ma_lpf1_config* pConfig, ma_lpf1* pLPF)
47638 | {
47639 |     double a;
47640 |
47641 |     if (pLPF == NULL || pConfig == NULL) {
47642 |         return MA_INVALID_ARGS;
47643 |     }
47644 |
47645 |     /* Only supporting f32 and s16. */
47646 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
47647 |         return MA_INVALID_ARGS;
47648 |     }
47649 |
47650 |     /* The format cannot be changed after initialization. */
47651 |     if (pLPF->format != ma_format_unknown && pLPF->format != pConfig->format) {
47652 |         return MA_INVALID_OPERATION;
47653 |     }
47654 |
47655 |     /* The channel count cannot be changed after initialization. */
47656 |     if (pLPF->channels != 0 && pLPF->channels != pConfig->channels) {
47657 |         return MA_INVALID_OPERATION;
47658 |     }
47659 |
47660 |     pLPF->format = pConfig->format;
47661 |     pLPF->channels = pConfig->channels;
47662 |
47663 |     a = ma_expd(-2 * MA_PI_D * pConfig->cutoffFrequency / pConfig->sampleRate);
47664 |     if (pConfig->format == ma_format_f32) {
47665 |         pLPF->a.f32 = (float)a;
47666 |     } else {
47667 |         pLPF->a.s32 = ma_biquad_float_to_fp(a);
47668 |     }
47669 |
47670 |     return MA_SUCCESS;
47671 | }
47672 |
47673 | MA_API ma_result ma_lpf1_clear_cache(ma_lpf1* pLPF)
47674 | {
47675 |     if (pLPF == NULL) {
47676 |         return MA_INVALID_ARGS;
47677 |     }
47678 |
47679 |     if (pLPF->format == ma_format_f32) {
47680 |         pLPF->a.f32 = 0;
47681 |     } else {
47682 |         pLPF->a.s32 = 0;
47683 |     }
47684 |
47685 |     return MA_SUCCESS;
47686 | }
47687 |
47688 | static MA_INLINE void ma_lpf1_process_pcm_frame_f32(ma_lpf1* pLPF, float* pY, const float* pX)
47689 | {
47690 |     ma_uint32 c;
47691 |     const ma_uint32 channels = pLPF->channels;
47692 |     const float a = pLPF->a.f32;
47693 |     const float b = 1 - a;
47694 |
47695 |     MA_ASSUME(channels > 0);
47696 |     for (c = 0; c < channels; c += 1) {
47697 |         float r1 = pLPF->pR1[c].f32;
47698 |         float x = pX[c];
47699 |         float y;
47700 |
47701 |         y = b*x + a*r1;
47702 |
47703 |         pY[c] = y;
47704 |         pLPF->pR1[c].f32 = y;
47705 |     }
47706 | }
47707 |
47708 | static MA_INLINE void ma_lpf1_process_pcm_frame_s16(ma_lpf1* pLPF, ma_int16* pY, const ma_int16* pX)
47709 | {
47710 |     ma_uint32 c;
47711 |     const ma_uint32 channels = pLPF->channels;
47712 |     const ma_int32 a = pLPF->a.s32;
47713 |     const ma_int32 b = ((1 << MA_BIQUAD_FIXED_POINT_SHIFT) - a);
47714 |
47715 |     MA_ASSUME(channels > 0);

```

```

47716 |     for (c = 0; c < channels; c += 1) {
47717 |         ma_int32 r1 = pLPF->pR1[c].s32;
47718 |         ma_int32 x  = pX[c];
47719 |         ma_int32 y;
47720 |
47721 |         y = (b*x + a*r1) >> MA_BIQUAD_FIXED_POINT_SHIFT;
47722 |
47723 |         pY[c]          = (ma_int16)y;
47724 |         pLPF->pR1[c].s32 = (ma_int32)y;
47725 |     }
47726 | }
47727 |
47728 | MA_API ma_result ma_lpf1_process_pcm_frames(ma_lpf1* pLPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount)
47729 | {
47730 |     ma_uint32 n;
47731 |
47732 |     if (pLPF == NULL || pFramesOut == NULL || pFramesIn == NULL) {
47733 |         return MA_INVALID_ARGS;
47734 |     }
47735 |
47736 |     /* Note that the logic below needs to support in-place filtering. That is, it must support the
> case where pFramesOut and pFramesIn are the same. */
47737 |
47738 |     if (pLPF->format == ma_format_f32) {
47739 |         /* */ float* pY = (float*)pFramesOut;
47740 |         const float* pX = (const float*)pFramesIn;
47741 |
47742 |         for (n = 0; n < frameCount; n += 1) {
47743 |             ma_lpf1_process_pcm_frame_f32(pLPF, pY, pX);
47744 |             pY += pLPF->channels;
47745 |             pX += pLPF->channels;
47746 |         }
47747 |     } else if (pLPF->format == ma_format_s16) {
47748 |         /* */ ma_int16* pY = (ma_int16*)pFramesOut;
47749 |         const ma_int16* pX = (const ma_int16*)pFramesIn;
47750 |
47751 |         for (n = 0; n < frameCount; n += 1) {
47752 |             ma_lpf1_process_pcm_frame_s16(pLPF, pY, pX);
47753 |             pY += pLPF->channels;
47754 |             pX += pLPF->channels;
47755 |         }
47756 |     } else {
47757 |         MA_ASSERT(MA_FALSE);
47758 |         return MA_INVALID_ARGS; /* Format not supported. Should never hit this because it's checked
> in ma_biquad_init() and ma_biquad_reinit(). */
47759 |     }
47760 |
47761 |     return MA_SUCCESS;
47762 | }
47763 |
47764 | MA_API ma_uint32 ma_lpf1_get_latency(const ma_lpf1* pLPF)
47765 | {
47766 |     if (pLPF == NULL) {
47767 |         return 0;
47768 |     }
47769 |
47770 |     return 1;
47771 | }
47772 |
47773 |
47774 | static MA_INLINE ma_biquad_config ma_lpf2__get_biquad_config(const ma_lpf2_config* pConfig)
47775 | {
47776 |     ma_biquad_config bqConfig;
47777 |     double q;
47778 |     double w;
47779 |     double s;
47780 |     double c;
47781 |     double a;
47782 |
47783 |     MA_ASSERT(pConfig != NULL);
47784 |
47785 |     q = pConfig->q;
47786 |     w = 2 * MA_PI_D * pConfig->cutoffFrequency / pConfig->sampleRate;
47787 |     s = ma_sind(w);
47788 |     c = ma_cosd(w);
47789 |     a = s / (2*q);
47790 |
47791 |     bqConfig.b0 = (1 - c) / 2;

```

```

47792 |     bqConfig.b1 = 1 - c;
47793 |     bqConfig.b2 = (1 - c) / 2;
47794 |     bqConfig.a0 = 1 + a;
47795 |     bqConfig.a1 = -2 * c;
47796 |     bqConfig.a2 = 1 - a;
47797 |
47798 |     bqConfig.format = pConfig->format;
47799 |     bqConfig.channels = pConfig->channels;
47800 |
47801 |     return bqConfig;
47802 | }
47803 |
47804 | MA_API ma_result ma_lpf2_get_heap_size(const ma_lpf2_config* pConfig, size_t* pHeapSizeInBytes)
47805 | {
47806 |     ma_biquad_config bqConfig;
47807 |     bqConfig = ma_lpf2__get_biquad_config(pConfig);
47808 |
47809 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
47810 | }
47811 |
47812 | MA_API ma_result ma_lpf2_init_preallocated(const ma_lpf2_config* pConfig, void* pHeap, ma_lpf2*
> pLPF)
47813 | {
47814 |     ma_result result;
47815 |     ma_biquad_config bqConfig;
47816 |
47817 |     if (pLPF == NULL) {
47818 |         return MA_INVALID_ARGS;
47819 |     }
47820 |
47821 |     MA_ZERO_OBJECT(pLPF);
47822 |
47823 |     if (pConfig == NULL) {
47824 |         return MA_INVALID_ARGS;
47825 |     }
47826 |
47827 |     bqConfig = ma_lpf2__get_biquad_config(pConfig);
47828 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pLPF->bq);
47829 |     if (result != MA_SUCCESS) {
47830 |         return result;
47831 |     }
47832 |
47833 |     return MA_SUCCESS;
47834 | }
47835 |
47836 | MA_API ma_result ma_lpf2_init(const ma_lpf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_lpf2* pLPF)
47837 | {
47838 |     ma_result result;
47839 |     size_t heapSizeInBytes;
47840 |     void* pHeap;
47841 |
47842 |     result = ma_lpf2_get_heap_size(pConfig, &heapSizeInBytes);
47843 |     if (result != MA_SUCCESS) {
47844 |         return result;
47845 |     }
47846 |
47847 |     if (heapSizeInBytes > 0) {
47848 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
47849 |         if (pHeap == NULL) {
47850 |             return MA_OUT_OF_MEMORY;
47851 |         }
47852 |     } else {
47853 |         pHeap = NULL;
47854 |     }
47855 |
47856 |     result = ma_lpf2_init_preallocated(pConfig, pHeap, pLPF);
47857 |     if (result != MA_SUCCESS) {
47858 |         ma_free(pHeap, pAllocationCallbacks);
47859 |         return result;
47860 |     }
47861 |
47862 |     pLPF->bq._ownsHeap = MA_TRUE; /* <-- This will cause the biquad to take ownership of the heap
> and free it when it's uninitialized. */
47863 |     return MA_SUCCESS;
47864 | }
47865 |
47866 | MA_API void ma_lpf2_uninit(ma_lpf2* pLPF, const ma_allocation_callbacks* pAllocationCallbacks)
47867 | {

```

```

47868 |     if (pLPF == NULL) {
47869 |         return;
47870 |     }
47871 |
47872 |     ma_biquad_uninit(&pLPF->bq, pAllocationCallbacks); /* <-- This will free the heap allocation.
> */
47873 | }
47874 |
47875 | MA_API ma_result ma_lpf2_reinit(const ma_lpf2_config* pConfig, ma_lpf2* pLPF)
47876 | {
47877 |     ma_result result;
47878 |     ma_biquad_config bqConfig;
47879 |
47880 |     if (pLPF == NULL || pConfig == NULL) {
47881 |         return MA_INVALID_ARGS;
47882 |     }
47883 |
47884 |     bqConfig = ma_lpf2__get_biquad_config(pConfig);
47885 |     result = ma_biquad_reinit(&bqConfig, &pLPF->bq);
47886 |     if (result != MA_SUCCESS) {
47887 |         return result;
47888 |     }
47889 |
47890 |     return MA_SUCCESS;
47891 | }
47892 |
47893 | MA_API ma_result ma_lpf2_clear_cache(ma_lpf2* pLPF)
47894 | {
47895 |     if (pLPF == NULL) {
47896 |         return MA_INVALID_ARGS;
47897 |     }
47898 |
47899 |     ma_biquad_clear_cache(&pLPF->bq);
47900 |
47901 |     return MA_SUCCESS;
47902 | }
47903 |
47904 | static MA_INLINE void ma_lpf2_process_pcm_frame_s16(ma_lpf2* pLPF, ma_int16* pFrameOut, const
> ma_int16* pFrameIn)
47905 | {
47906 |     ma_biquad_process_pcm_frame_s16(&pLPF->bq, pFrameOut, pFrameIn);
47907 | }
47908 |
47909 | static MA_INLINE void ma_lpf2_process_pcm_frame_f32(ma_lpf2* pLPF, float* pFrameOut, const float*
> pFrameIn)
47910 | {
47911 |     ma_biquad_process_pcm_frame_f32(&pLPF->bq, pFrameOut, pFrameIn);
47912 | }
47913 |
47914 | MA_API ma_result ma_lpf2_process_pcm_frames(ma_lpf2* pLPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount)
47915 | {
47916 |     if (pLPF == NULL) {
47917 |         return MA_INVALID_ARGS;
47918 |     }
47919 |
47920 |     return ma_biquad_process_pcm_frames(&pLPF->bq, pFramesOut, pFramesIn, frameCount);
47921 | }
47922 |
47923 | MA_API ma_uint32 ma_lpf2_get_latency(const ma_lpf2* pLPF)
47924 | {
47925 |     if (pLPF == NULL) {
47926 |         return 0;
47927 |     }
47928 |
47929 |     return ma_biquad_get_latency(&pLPF->bq);
47930 | }
47931 |
47932 |
47933 | MA_API ma_lpf_config ma_lpf_config_init(ma_format format, ma_uint32 channels, ma_uint32 sampleRate,
> double cutoffFrequency, ma_uint32 order)
47934 | {
47935 |     ma_lpf_config config;
47936 |
47937 |     MA_ZERO_OBJECT(&config);
47938 |     config.format = format;
47939 |     config.channels = channels;
47940 |     config.sampleRate = sampleRate;
47941 |     config.cutoffFrequency = cutoffFrequency;

```

```

47942 |         config.order          = ma_min(order, MA_MAX_FILTER_ORDER);
47943 |
47944 |         return config;
47945 |     }
47946 |
47947 |
47948 |     typedef struct
47949 |     {
47950 |         size_t sizeInBytes;
47951 |         size_t lpf10offset;
47952 |         size_t lpf20offset; /* Offset of the first second order filter. Subsequent filters will come
> straight after, and will each have the same heap size. */
47953 |     } ma_lpf_heap_layout;
47954 |
47955 |     static void ma_lpf_calculate_sub_lpf_counts(ma_uint32 order, ma_uint32* pLPF1Count, ma_uint32*
> pLPF2Count)
47956 |     {
47957 |         MA_ASSERT(pLPF1Count != NULL);
47958 |         MA_ASSERT(pLPF2Count != NULL);
47959 |
47960 |         *pLPF1Count = order % 2;
47961 |         *pLPF2Count = order / 2;
47962 |     }
47963 |
47964 |     static ma_result ma_lpf_get_heap_layout(const ma_lpf_config* pConfig, ma_lpf_heap_layout*
> pHeapLayout)
47965 |     {
47966 |         ma_result result;
47967 |         ma_uint32 lpf1Count;
47968 |         ma_uint32 lpf2Count;
47969 |         ma_uint32 ilpf1;
47970 |         ma_uint32 ilpf2;
47971 |
47972 |         MA_ASSERT(pHeapLayout != NULL);
47973 |
47974 |         MA_ZERO_OBJECT(pHeapLayout);
47975 |
47976 |         if (pConfig == NULL) {
47977 |             return MA_INVALID_ARGS;
47978 |         }
47979 |
47980 |         if (pConfig->channels == 0) {
47981 |             return MA_INVALID_ARGS;
47982 |         }
47983 |
47984 |         if (pConfig->order > MA_MAX_FILTER_ORDER) {
47985 |             return MA_INVALID_ARGS;
47986 |         }
47987 |
47988 |         ma_lpf_calculate_sub_lpf_counts(pConfig->order, &lpf1Count, &lpf2Count);
47989 |
47990 |         pHeapLayout->sizeInBytes = 0;
47991 |
47992 |         /* LPF 1 */
47993 |         pHeapLayout->lpf10offset = pHeapLayout->sizeInBytes;
47994 |         for (ilpf1 = 0; ilpf1 < lpf1Count; ilpf1 += 1) {
47995 |             size_t lpf1HeapSizeInBytes;
47996 |             ma_lpf1_config lpf1Config = ma_lpf1_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency);
47997 |
47998 |             result = ma_lpf1_get_heap_size(&lpf1Config, &lpf1HeapSizeInBytes);
47999 |             if (result != MA_SUCCESS) {
48000 |                 return result;
48001 |             }
48002 |
48003 |             pHeapLayout->sizeInBytes += sizeof(ma_lpf1) + lpf1HeapSizeInBytes;
48004 |         }
48005 |
48006 |         /* LPF 2 */
48007 |         pHeapLayout->lpf20offset = pHeapLayout->sizeInBytes;
48008 |         for (ilpf2 = 0; ilpf2 < lpf2Count; ilpf2 += 1) {
48009 |             size_t lpf2HeapSizeInBytes;
48010 |             ma_lpf2_config lpf2Config = ma_lpf2_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency, 0.707107); /* <-- The "q" parameter does not matter
> for the purpose of calculating the heap size. */
48011 |
48012 |             result = ma_lpf2_get_heap_size(&lpf2Config, &lpf2HeapSizeInBytes);
48013 |             if (result != MA_SUCCESS) {
48014 |                 return result;

```

```

48015 |     }
48016 |
48017 |     pHeapLayout->sizeInBytes += sizeof(ma_lpf2) + lpf2HeapSizeInBytes;
48018 | }
48019 |
48020 | /* Make sure allocation size is aligned. */
48021 | pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
48022 |
48023 | return MA_SUCCESS;
48024 | }
48025 |
48026 | static ma_result ma_lpf_reinit__internal(const ma_lpf_config* pConfig, void* pHeap, ma_lpf* pLPF,
> ma_bool32 isNew)
48027 | {
48028 |     ma_result result;
48029 |     ma_uint32 lpf1Count;
48030 |     ma_uint32 lpf2Count;
48031 |     ma_uint32 ilpf1;
48032 |     ma_uint32 ilpf2;
48033 |     ma_lpf_heap_layout heapLayout; /* Only used if isNew is true. */
48034 |
48035 |     if (pLPF == NULL || pConfig == NULL) {
48036 |         return MA_INVALID_ARGS;
48037 |     }
48038 |
48039 |     /* Only supporting f32 and s16. */
48040 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
48041 |         return MA_INVALID_ARGS;
48042 |     }
48043 |
48044 |     /* The format cannot be changed after initialization. */
48045 |     if (pLPF->format != ma_format_unknown && pLPF->format != pConfig->format) {
48046 |         return MA_INVALID_OPERATION;
48047 |     }
48048 |
48049 |     /* The channel count cannot be changed after initialization. */
48050 |     if (pLPF->channels != 0 && pLPF->channels != pConfig->channels) {
48051 |         return MA_INVALID_OPERATION;
48052 |     }
48053 |
48054 |     if (pConfig->order > MA_MAX_FILTER_ORDER) {
48055 |         return MA_INVALID_ARGS;
48056 |     }
48057 |
48058 |     ma_lpf_calculate_sub_lpf_counts(pConfig->order, &lpf1Count, &lpf2Count);
48059 |
48060 |     /* The filter order can't change between reinit. */
48061 |     if (!isNew) {
48062 |         if (pLPF->lpf1Count != lpf1Count || pLPF->lpf2Count != lpf2Count) {
48063 |             return MA_INVALID_OPERATION;
48064 |         }
48065 |     }
48066 |
48067 |     if (isNew) {
48068 |         result = ma_lpf_get_heap_layout(pConfig, &heapLayout);
48069 |         if (result != MA_SUCCESS) {
48070 |             return result;
48071 |         }
48072 |
48073 |         pLPF->pHeap = pHeap;
48074 |         MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
48075 |
48076 |         pLPF->pLPF1 = (ma_lpf1*)ma_offset_ptr(pHeap, heapLayout.lpf1Offset);
48077 |         pLPF->pLPF2 = (ma_lpf2*)ma_offset_ptr(pHeap, heapLayout.lpf2Offset);
48078 |     } else {
48079 |         MA_ZERO_OBJECT(&heapLayout); /* To silence a compiler warning. */
48080 |     }
48081 |
48082 |     for (ilpf1 = 0; ilpf1 < lpf1Count; ilpf1 += 1) {
48083 |         ma_lpf1_config lpf1Config = ma_lpf1_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency);
48084 |
48085 |         if (isNew) {
48086 |             size_t lpf1HeapSizeInBytes;
48087 |
48088 |             result = ma_lpf1_get_heap_size(&lpf1Config, &lpf1HeapSizeInBytes);
48089 |             if (result == MA_SUCCESS) {
48090 |                 result = ma_lpf1_init_preallocated(&lpf1Config, ma_offset_ptr(pHeap,
> heapLayout.lpf1Offset + (sizeof(ma_lpf1) * lpf1Count) + (ilpf1 * lpf1HeapSizeInBytes)),

```



```

> &pLPF->pLPF1[ilpf1]);
48091 |     }
48092 |     } else {
48093 |         result = ma_lpf1_reinit(&lpf1Config, &pLPF->pLPF1[ilpf1]);
48094 |     }
48095 |
48096 |     if (result != MA_SUCCESS) {
48097 |         ma_uint32 jlpf1;
48098 |
48099 |         for (jlpf1 = 0; jlpf1 < ilpf1; jlpf1 += 1) {
48100 |             ma_lpf1_uninit(&pLPF->pLPF1[jlpf1], NULL); /* No need for allocation callbacks here
> since we used a preallocated heap allocation. */
48101 |         }
48102 |
48103 |         return result;
48104 |     }
48105 | }
48106 |
48107 | for (ilpf2 = 0; ilpf2 < lpf2Count; ilpf2 += 1) {
48108 |     ma_lpf2_config lpf2Config;
48109 |     double q;
48110 |     double a;
48111 |
48112 |     /* Tempting to use 0.707107, but won't result in a Butterworth filter if the order is > 2.
> */
48113 |     if (lpf1Count == 1) {
48114 |         a = (1 + ilpf2*1) * (MA_PI_D/(pConfig->order*1)); /* Odd order. */
48115 |     } else {
48116 |         a = (1 + ilpf2*2) * (MA_PI_D/(pConfig->order*2)); /* Even order. */
48117 |     }
48118 |     q = 1 / (2*ma_cosd(a));
48119 |
48120 |     lpf2Config = ma_lpf2_config_init(pConfig->format, pConfig->channels, pConfig->sampleRate,
> pConfig->cutoffFrequency, q);
48121 |
48122 |     if (isNew) {
48123 |         size_t lpf2HeapSizeInBytes;
48124 |
48125 |         result = ma_lpf2_get_heap_size(&lpf2Config, &lpf2HeapSizeInBytes);
48126 |         if (result == MA_SUCCESS) {
48127 |             result = ma_lpf2_init_preallocated(&lpf2Config, ma_offset_ptr(pHeap,
> heapLayout.lpf2Offset + (sizeof(ma_lpf2) * lpf2Count) + (ilpf2 * lpf2HeapSizeInBytes)),
> &pLPF->pLPF2[ilpf2]);
48128 |         }
48129 |     } else {
48130 |         result = ma_lpf2_reinit(&lpf2Config, &pLPF->pLPF2[ilpf2]);
48131 |     }
48132 |
48133 |     if (result != MA_SUCCESS) {
48134 |         ma_uint32 jlpf1;
48135 |         ma_uint32 jlpf2;
48136 |
48137 |         for (jlpf1 = 0; jlpf1 < lpf1Count; jlpf1 += 1) {
48138 |             ma_lpf1_uninit(&pLPF->pLPF1[jlpf1], NULL); /* No need for allocation callbacks here
> since we used a preallocated heap allocation. */
48139 |         }
48140 |
48141 |         for (jlpf2 = 0; jlpf2 < ilpf2; jlpf2 += 1) {
48142 |             ma_lpf2_uninit(&pLPF->pLPF2[jlpf2], NULL); /* No need for allocation callbacks here
> since we used a preallocated heap allocation. */
48143 |         }
48144 |
48145 |         return result;
48146 |     }
48147 | }
48148 |
48149 | pLPF->lpf1Count = lpf1Count;
48150 | pLPF->lpf2Count = lpf2Count;
48151 | pLPF->format = pConfig->format;
48152 | pLPF->channels = pConfig->channels;
48153 | pLPF->sampleRate = pConfig->sampleRate;
48154 |
48155 | return MA_SUCCESS;
48156 | }
48157 |
48158 | MA_API ma_result ma_lpf_get_heap_size(const ma_lpf_config* pConfig, size_t* pHeapSizeInBytes)
48159 | {
48160 |     ma_result result;
48161 |     ma_lpf_heap_layout heapLayout;

```

```

48162 |
48163 |     if (pHeapSizeInBytes == NULL) {
48164 |         return MA_INVALID_ARGS;
48165 |     }
48166 |
48167 |     *pHeapSizeInBytes = 0;
48168 |
48169 |     result = ma_lpf_get_heap_layout(pConfig, &heapLayout);
48170 |     if (result != MA_SUCCESS) {
48171 |         return result;
48172 |     }
48173 |
48174 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
48175 |
48176 |     return result;
48177 | }
48178 |
48179 | MA_API ma_result ma_lpf_init_preallocated(const ma_lpf_config* pConfig, void* pHeap, ma_lpf* pLPF)
48180 | {
48181 |     if (pLPF == NULL) {
48182 |         return MA_INVALID_ARGS;
48183 |     }
48184 |
48185 |     MA_ZERO_OBJECT(pLPF);
48186 |
48187 |     return ma_lpf_reinit__internal(pConfig, pHeap, pLPF, /*isNew*/MA_TRUE);
48188 | }
48189 |
48190 | MA_API ma_result ma_lpf_init(const ma_lpf_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_lpf* pLPF)
48191 | {
48192 |     ma_result result;
48193 |     size_t heapSizeInBytes;
48194 |     void* pHeap;
48195 |
48196 |     result = ma_lpf_get_heap_size(pConfig, &heapSizeInBytes);
48197 |     if (result != MA_SUCCESS) {
48198 |         return result;
48199 |     }
48200 |
48201 |     if (heapSizeInBytes > 0) {
48202 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
48203 |         if (pHeap == NULL) {
48204 |             return MA_OUT_OF_MEMORY;
48205 |         }
48206 |     } else {
48207 |         pHeap = NULL;
48208 |     }
48209 |
48210 |     result = ma_lpf_init_preallocated(pConfig, pHeap, pLPF);
48211 |     if (result != MA_SUCCESS) {
48212 |         ma_free(pHeap, pAllocationCallbacks);
48213 |         return result;
48214 |     }
48215 |
48216 |     pLPF->ownsHeap = MA_TRUE;
48217 |     return MA_SUCCESS;
48218 | }
48219 |
48220 | MA_API void ma_lpf_uninit(ma_lpf* pLPF, const ma_allocation_callbacks* pAllocationCallbacks)
48221 | {
48222 |     ma_uint32 ilpf1;
48223 |     ma_uint32 ilpf2;
48224 |
48225 |     if (pLPF == NULL) {
48226 |         return;
48227 |     }
48228 |
48229 |     for (ilpf1 = 0; ilpf1 < pLPF->lpf1Count; ilpf1 += 1) {
48230 |         ma_lpf1_uninit(&pLPF->pLPF1[ilpf1], pAllocationCallbacks);
48231 |     }
48232 |
48233 |     for (ilpf2 = 0; ilpf2 < pLPF->lpf2Count; ilpf2 += 1) {
48234 |         ma_lpf2_uninit(&pLPF->pLPF2[ilpf2], pAllocationCallbacks);
48235 |     }
48236 |
48237 |     if (pLPF->ownsHeap) {
48238 |         ma_free(pLPF->pHeap, pAllocationCallbacks);
48239 |     }

```

```

48240 | }
48241 |
48242 | MA_API ma_result ma_lpf_reinit(const ma_lpf_config* pConfig, ma_lpf* pLPF)
48243 | {
48244 |     return ma_lpf_reinit__internal(pConfig, NULL, pLPF, /*isNew*/MA_FALSE);
48245 | }
48246 |
48247 | MA_API ma_result ma_lpf_clear_cache(ma_lpf* pLPF)
48248 | {
48249 |     ma_uint32 ilpf1;
48250 |     ma_uint32 ilpf2;
48251 |
48252 |     if (pLPF == NULL) {
48253 |         return MA_INVALID_ARGS;
48254 |     }
48255 |
48256 |     for (ilpf1 = 0; ilpf1 < pLPF->lpf1Count; ilpf1 += 1) {
48257 |         ma_lpf1_clear_cache(&pLPF->pLPF1[ilpf1]);
48258 |     }
48259 |
48260 |     for (ilpf2 = 0; ilpf2 < pLPF->lpf2Count; ilpf2 += 1) {
48261 |         ma_lpf2_clear_cache(&pLPF->pLPF2[ilpf2]);
48262 |     }
48263 |
48264 |     return MA_SUCCESS;
48265 | }
48266 |
48267 | static MA_INLINE void ma_lpf_process_pcm_frame_f32(ma_lpf* pLPF, float* pY, const void* pX)
48268 | {
48269 |     ma_uint32 ilpf1;
48270 |     ma_uint32 ilpf2;
48271 |
48272 |     MA_ASSERT(pLPF->format == ma_format_f32);
48273 |
48274 |     MA_MOVE_MEMORY(pY, pX, ma_get_bytes_per_frame(pLPF->format, pLPF->channels));
48275 |
48276 |     for (ilpf1 = 0; ilpf1 < pLPF->lpf1Count; ilpf1 += 1) {
48277 |         ma_lpf1_process_pcm_frame_f32(&pLPF->pLPF1[ilpf1], pY, pY);
48278 |     }
48279 |
48280 |     for (ilpf2 = 0; ilpf2 < pLPF->lpf2Count; ilpf2 += 1) {
48281 |         ma_lpf2_process_pcm_frame_f32(&pLPF->pLPF2[ilpf2], pY, pY);
48282 |     }
48283 | }
48284 |
48285 | static MA_INLINE void ma_lpf_process_pcm_frame_s16(ma_lpf* pLPF, ma_int16* pY, const ma_int16* pX)
48286 | {
48287 |     ma_uint32 ilpf1;
48288 |     ma_uint32 ilpf2;
48289 |
48290 |     MA_ASSERT(pLPF->format == ma_format_s16);
48291 |
48292 |     MA_MOVE_MEMORY(pY, pX, ma_get_bytes_per_frame(pLPF->format, pLPF->channels));
48293 |
48294 |     for (ilpf1 = 0; ilpf1 < pLPF->lpf1Count; ilpf1 += 1) {
48295 |         ma_lpf1_process_pcm_frame_s16(&pLPF->pLPF1[ilpf1], pY, pY);
48296 |     }
48297 |
48298 |     for (ilpf2 = 0; ilpf2 < pLPF->lpf2Count; ilpf2 += 1) {
48299 |         ma_lpf2_process_pcm_frame_s16(&pLPF->pLPF2[ilpf2], pY, pY);
48300 |     }
48301 | }
48302 |
48303 | MA_API ma_result ma_lpf_process_pcm_frames(ma_lpf* pLPF, void* pFramesOut, const void* pFramesIn,
48304 | > ma_uint64 frameCount)
48305 | {
48306 |     ma_result result;
48307 |     ma_uint32 ilpf1;
48308 |     ma_uint32 ilpf2;
48309 |
48310 |     if (pLPF == NULL) {
48311 |         return MA_INVALID_ARGS;
48312 |     }
48313 |
48314 |     /* Faster path for in-place. */
48315 |     if (pFramesOut == pFramesIn) {
48316 |         for (ilpf1 = 0; ilpf1 < pLPF->lpf1Count; ilpf1 += 1) {
48317 |             result = ma_lpf1_process_pcm_frames(&pLPF->pLPF1[ilpf1], pFramesOut, pFramesOut,
48318 | > frameCount);

```

```

48317 |         if (result != MA_SUCCESS) {
48318 |             return result;
48319 |         }
48320 |     }
48321 |
48322 |     for (ilpf2 = 0; ilpf2 < pLPF->lpf2Count; ilpf2 += 1) {
48323 |         result = ma_lpf2_process_pcm_frames(&pLPF->pLPF2[ilpf2], pFramesOut, pFramesOut,
> frameCount);
48324 |         if (result != MA_SUCCESS) {
48325 |             return result;
48326 |         }
48327 |     }
48328 | }
48329 |
48330 | /* Slightly slower path for copying. */
48331 | if (pFramesOut != pFramesIn) {
48332 |     ma_uint32 iFrame;
48333 |
48334 |     /* */ if (pLPF->format == ma_format_f32) {
48335 |         /* */ float* pFramesOutF32 = (float*)pFramesOut;
48336 |         const float* pFramesInF32 = (const float*)pFramesIn;
48337 |
48338 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
48339 |             ma_lpf_process_pcm_frame_f32(pLPF, pFramesOutF32, pFramesInF32);
48340 |             pFramesOutF32 += pLPF->channels;
48341 |             pFramesInF32 += pLPF->channels;
48342 |         }
48343 |     } else if (pLPF->format == ma_format_s16) {
48344 |         /* */ ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
48345 |         const ma_int16* pFramesInS16 = (const ma_int16*)pFramesIn;
48346 |
48347 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
48348 |             ma_lpf_process_pcm_frame_s16(pLPF, pFramesOutS16, pFramesInS16);
48349 |             pFramesOutS16 += pLPF->channels;
48350 |             pFramesInS16 += pLPF->channels;
48351 |         }
48352 |     } else {
48353 |         MA_ASSERT(MA_FALSE);
48354 |         return MA_INVALID_OPERATION; /* Should never hit this. */
48355 |     }
48356 | }
48357 |
48358 | return MA_SUCCESS;
48359 | }
48360 |
48361 | MA_API ma_uint32 ma_lpf_get_latency(const ma_lpf* pLPF)
48362 | {
48363 |     if (pLPF == NULL) {
48364 |         return 0;
48365 |     }
48366 |
48367 |     return pLPF->lpf2Count*2 + pLPF->lpf1Count;
48368 | }
48369 |
48370 |
48371 | /*****
> *****/
48372 |
48373 | High-Pass Filtering
48374 |
48375 | *****/
> *****/
48376 | MA_API ma_hpf1_config ma_hpf1_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency)
48377 | {
48378 |     ma_hpf1_config config;
48379 |
48380 |     MA_ZERO_OBJECT(&config);
48381 |     config.format = format;
48382 |     config.channels = channels;
48383 |     config.sampleRate = sampleRate;
48384 |     config.cutoffFrequency = cutoffFrequency;
48385 |
48386 |     return config;
48387 | }
48388 |
48389 | MA_API ma_hpf2_config ma_hpf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency, double q)
48390 | {

```

```

48391 |     ma_hpf2_config config;
48392 |
48393 |     MA_ZERO_OBJECT(&config);
48394 |     config.format = format;
48395 |     config.channels = channels;
48396 |     config.sampleRate = sampleRate;
48397 |     config.cutoffFrequency = cutoffFrequency;
48398 |     config.q = q;
48399 |
48400 |     /* Q cannot be 0 or else it'll result in a division by 0. In this case just default to 0.707107.
> */
48401 |     if (config.q == 0) {
48402 |         config.q = 0.707107;
48403 |     }
48404 |
48405 |     return config;
48406 | }
48407 |
48408 |
48409 | typedef struct
48410 | {
48411 |     size_t sizeInBytes;
48412 |     size_t r10ffset;
48413 | } ma_hpf1_heap_layout;
48414 |
48415 | static ma_result ma_hpf1_get_heap_layout(const ma_hpf1_config* pConfig, ma_hpf1_heap_layout*
> pHeapLayout)
48416 | {
48417 |     MA_ASSERT(pHeapLayout != NULL);
48418 |
48419 |     MA_ZERO_OBJECT(pHeapLayout);
48420 |
48421 |     if (pConfig == NULL) {
48422 |         return MA_INVALID_ARGS;
48423 |     }
48424 |
48425 |     if (pConfig->channels == 0) {
48426 |         return MA_INVALID_ARGS;
48427 |     }
48428 |
48429 |     pHeapLayout->sizeInBytes = 0;
48430 |
48431 |     /* R1 */
48432 |     pHeapLayout->r10ffset = pHeapLayout->sizeInBytes;
48433 |     pHeapLayout->sizeInBytes += sizeof(ma_biquad_coefficient) * pConfig->channels;
48434 |
48435 |     /* Make sure allocation size is aligned. */
48436 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
48437 |
48438 |     return MA_SUCCESS;
48439 | }
48440 |
48441 | MA_API ma_result ma_hpf1_get_heap_size(const ma_hpf1_config* pConfig, size_t* pHeapSizeInBytes)
48442 | {
48443 |     ma_result result;
48444 |     ma_hpf1_heap_layout heapLayout;
48445 |
48446 |     if (pHeapSizeInBytes == NULL) {
48447 |         return MA_INVALID_ARGS;
48448 |     }
48449 |
48450 |     result = ma_hpf1_get_heap_layout(pConfig, &heapLayout);
48451 |     if (result != MA_SUCCESS) {
48452 |         return result;
48453 |     }
48454 |
48455 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
48456 |
48457 |     return MA_SUCCESS;
48458 | }
48459 |
48460 | MA_API ma_result ma_hpf1_init_preallocated(const ma_hpf1_config* pConfig, void* pHeap, ma_hpf1*
> pLPF)
48461 | {
48462 |     ma_result result;
48463 |     ma_hpf1_heap_layout heapLayout;
48464 |
48465 |     if (pLPF == NULL) {
48466 |         return MA_INVALID_ARGS;

```

```

48467 |     }
48468 |
48469 |     MA_ZERO_OBJECT(pLPF);
48470 |
48471 |     result = ma_hpf1_get_heap_layout(pConfig, &heapLayout);
48472 |     if (result != MA_SUCCESS) {
48473 |         return result;
48474 |     }
48475 |
48476 |     pLPF->pHeap = pHeap;
48477 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
48478 |
48479 |     pLPF->pR1 = (ma_biquad_coefficient*)ma_offset_ptr(pHeap, heapLayout.r1offset);
48480 |
48481 |     return ma_hpf1_reinit(pConfig, pLPF);
48482 | }
48483 |
48484 | MA_API ma_result ma_hpf1_init(const ma_hpf1_config* pConfig, const ma_allocation_callbacks*
48485 | > pAllocationCallbacks, ma_hpf1* pLPF)
48486 | {
48487 |     ma_result result;
48488 |     size_t heapSizeInBytes;
48489 |     void* pHeap;
48490 |
48491 |     result = ma_hpf1_get_heap_size(pConfig, &heapSizeInBytes);
48492 |     if (result != MA_SUCCESS) {
48493 |         return result;
48494 |     }
48495 |
48496 |     if (heapSizeInBytes > 0) {
48497 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
48498 |         if (pHeap == NULL) {
48499 |             return MA_OUT_OF_MEMORY;
48500 |         }
48501 |     } else {
48502 |         pHeap = NULL;
48503 |     }
48504 |
48505 |     result = ma_hpf1_init_preallocated(pConfig, pHeap, pLPF);
48506 |     if (result != MA_SUCCESS) {
48507 |         ma_free(pHeap, pAllocationCallbacks);
48508 |         return result;
48509 |     }
48510 |
48511 |     pLPF->ownsHeap = MA_TRUE;
48512 |     return MA_SUCCESS;
48513 | }
48514 |
48515 | MA_API void ma_hpf1_uninit(ma_hpf1* pHPF, const ma_allocation_callbacks* pAllocationCallbacks)
48516 | {
48517 |     if (pHPF == NULL) {
48518 |         return;
48519 |     }
48520 |
48521 |     if (pHPF->ownsHeap) {
48522 |         ma_free(pHPF->pHeap, pAllocationCallbacks);
48523 |     }
48524 | }
48525 |
48526 | MA_API ma_result ma_hpf1_reinit(const ma_hpf1_config* pConfig, ma_hpf1* pHPF)
48527 | {
48528 |     double a;
48529 |
48530 |     if (pHPF == NULL || pConfig == NULL) {
48531 |         return MA_INVALID_ARGS;
48532 |     }
48533 |
48534 |     /* Only supporting f32 and s16. */
48535 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
48536 |         return MA_INVALID_ARGS;
48537 |     }
48538 |
48539 |     /* The format cannot be changed after initialization. */
48540 |     if (pHPF->format != ma_format_unknown && pHPF->format != pConfig->format) {
48541 |         return MA_INVALID_OPERATION;
48542 |     }
48543 |
48544 |     /* The channel count cannot be changed after initialization. */
48545 |     if (pHPF->channels != 0 && pHPF->channels != pConfig->channels) {

```

```

48545 |         return MA_INVALID_OPERATION;
48546 |     }
48547 |
48548 |     pHPF->format    = pConfig->format;
48549 |     pHPF->channels   = pConfig->channels;
48550 |
48551 |     a = ma_expd(-2 * MA_PI_D * pConfig->cutoffFrequency / pConfig->sampleRate);
48552 |     if (pConfig->format == ma_format_f32) {
48553 |         pHPF->a.f32 = (float)a;
48554 |     } else {
48555 |         pHPF->a.s32 = ma_biquad_float_to_fp(a);
48556 |     }
48557 |
48558 |     return MA_SUCCESS;
48559 | }
48560 |
48561 | static MA_INLINE void ma_hpf1_process_pcm_frame_f32(ma_hpf1* pHPF, float* pY, const float* pX)
48562 | {
48563 |     ma_uint32 c;
48564 |     const ma_uint32 channels = pHPF->channels;
48565 |     const float a = 1 - pHPF->a.f32;
48566 |     const float b = 1 - a;
48567 |
48568 |     MA_ASSUME(channels > 0);
48569 |     for (c = 0; c < channels; c += 1) {
48570 |         float r1 = pHPF->pR1[c].f32;
48571 |         float x  = pX[c];
48572 |         float y;
48573 |
48574 |         y = b*x - a*r1;
48575 |
48576 |         pY[c] = y;
48577 |         pHPF->pR1[c].f32 = y;
48578 |     }
48579 | }
48580 |
48581 | static MA_INLINE void ma_hpf1_process_pcm_frame_s16(ma_hpf1* pHPF, ma_int16* pY, const ma_int16* pX)
48582 | {
48583 |     ma_uint32 c;
48584 |     const ma_uint32 channels = pHPF->channels;
48585 |     const ma_int32 a = ((1 << MA_BIQUAD_FIXED_POINT_SHIFT) - pHPF->a.s32);
48586 |     const ma_int32 b = ((1 << MA_BIQUAD_FIXED_POINT_SHIFT) - a);
48587 |
48588 |     MA_ASSUME(channels > 0);
48589 |     for (c = 0; c < channels; c += 1) {
48590 |         ma_int32 r1 = pHPF->pR1[c].s32;
48591 |         ma_int32 x  = pX[c];
48592 |         ma_int32 y;
48593 |
48594 |         y = (b*x - a*r1) >> MA_BIQUAD_FIXED_POINT_SHIFT;
48595 |
48596 |         pY[c] = (ma_int16)y;
48597 |         pHPF->pR1[c].s32 = (ma_int32)y;
48598 |     }
48599 | }
48600 |
48601 | MA_API ma_result ma_hpf1_process_pcm_frames(ma_hpf1* pHPF, void* pFramesOut, const void* pFramesIn,
48602 | > ma_uint64 frameCount)
48603 | {
48604 |     ma_uint32 n;
48605 |
48606 |     if (pHPF == NULL || pFramesOut == NULL || pFramesIn == NULL) {
48607 |         return MA_INVALID_ARGS;
48608 |     }
48609 |
48610 |     /* Note that the logic below needs to support in-place filtering. That is, it must support the
48611 | > case where pFramesOut and pFramesIn are the same. */
48612 |
48613 |     if (pHPF->format == ma_format_f32) {
48614 |         /* */ float* pY = (float*)pFramesOut;
48615 |         const float* pX = (const float*)pFramesIn;
48616 |
48617 |         for (n = 0; n < frameCount; n += 1) {
48618 |             ma_hpf1_process_pcm_frame_f32(pHPF, pY, pX);
48619 |             pY += pHPF->channels;
48620 |             pX += pHPF->channels;
48621 |         }
48622 |     } else if (pHPF->format == ma_format_s16) {
48623 |         /* */ ma_int16* pY = (ma_int16*)pFramesOut;

```



```

48622 |         const ma_int16* pX = (const ma_int16*)pFramesIn;
48623 |
48624 |         for (n = 0; n < frameCount; n += 1) {
48625 |             ma_hpf1_process_pcm_frame_s16(pHPF, pY, pX);
48626 |             pY += pHPF->channels;
48627 |             pX += pHPF->channels;
48628 |         }
48629 |     } else {
48630 |         MA_ASSERT(MA_FALSE);
48631 |         return MA_INVALID_ARGS; /* Format not supported. Should never hit this because it's checked
> in ma_biquad_init() and ma_biquad_reinit(). */
48632 |     }
48633 |
48634 |     return MA_SUCCESS;
48635 | }
48636 |
48637 | MA_API ma_uint32 ma_hpf1_get_latency(const ma_hpf1* pHPF)
48638 | {
48639 |     if (pHPF == NULL) {
48640 |         return 0;
48641 |     }
48642 |
48643 |     return 1;
48644 | }
48645 |
48646 |
48647 | static MA_INLINE ma_biquad_config ma_hpf2__get_biquad_config(const ma_hpf2_config* pConfig)
48648 | {
48649 |     ma_biquad_config bqConfig;
48650 |     double q;
48651 |     double w;
48652 |     double s;
48653 |     double c;
48654 |     double a;
48655 |
48656 |     MA_ASSERT(pConfig != NULL);
48657 |
48658 |     q = pConfig->q;
48659 |     w = 2 * MA_PI_D * pConfig->cutoffFrequency / pConfig->sampleRate;
48660 |     s = ma_sind(w);
48661 |     c = ma_cosd(w);
48662 |     a = s / (2*q);
48663 |
48664 |     bqConfig.b0 = (1 + c) / 2;
48665 |     bqConfig.b1 = -(1 + c);
48666 |     bqConfig.b2 = (1 + c) / 2;
48667 |     bqConfig.a0 = 1 + a;
48668 |     bqConfig.a1 = -2 * c;
48669 |     bqConfig.a2 = 1 - a;
48670 |
48671 |     bqConfig.format = pConfig->format;
48672 |     bqConfig.channels = pConfig->channels;
48673 |
48674 |     return bqConfig;
48675 | }
48676 |
48677 | MA_API ma_result ma_hpf2_get_heap_size(const ma_hpf2_config* pConfig, size_t* pHeapSizeInBytes)
48678 | {
48679 |     ma_biquad_config bqConfig;
48680 |     bqConfig = ma_hpf2__get_biquad_config(pConfig);
48681 |
48682 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
48683 | }
48684 |
48685 | MA_API ma_result ma_hpf2_init_preallocated(const ma_hpf2_config* pConfig, void* pHeap, ma_hpf2*
> pHPF)
48686 | {
48687 |     ma_result result;
48688 |     ma_biquad_config bqConfig;
48689 |
48690 |     if (pHPF == NULL) {
48691 |         return MA_INVALID_ARGS;
48692 |     }
48693 |
48694 |     MA_ZERO_OBJECT(pHPF);
48695 |
48696 |     if (pConfig == NULL) {
48697 |         return MA_INVALID_ARGS;
48698 |     }

```

```

48699 |
48700 |     bqConfig = ma_hpf2__get_biquad_config(pConfig);
48701 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pHPF->bq);
48702 |     if (result != MA_SUCCESS) {
48703 |         return result;
48704 |     }
48705 |
48706 |     return MA_SUCCESS;
48707 | }
48708 |
48709 | MA_API ma_result ma_hpf2_init(const ma_hpf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hpf2* pHPF)
48710 | {
48711 |     ma_result result;
48712 |     size_t heapSizeInBytes;
48713 |     void* pHeap;
48714 |
48715 |     result = ma_hpf2_get_heap_size(pConfig, &heapSizeInBytes);
48716 |     if (result != MA_SUCCESS) {
48717 |         return result;
48718 |     }
48719 |
48720 |     if (heapSizeInBytes > 0) {
48721 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
48722 |         if (pHeap == NULL) {
48723 |             return MA_OUT_OF_MEMORY;
48724 |         }
48725 |     } else {
48726 |         pHeap = NULL;
48727 |     }
48728 |
48729 |     result = ma_hpf2_init_preallocated(pConfig, pHeap, pHPF);
48730 |     if (result != MA_SUCCESS) {
48731 |         ma_free(pHeap, pAllocationCallbacks);
48732 |         return result;
48733 |     }
48734 |
48735 |     pHPF->bq.ownsHeap = MA_TRUE; /* <-- This will cause the biquad to take ownership of the heap
> and free it when it's uninitialized. */
48736 |     return MA_SUCCESS;
48737 | }
48738 |
48739 | MA_API void ma_hpf2_uninit(ma_hpf2* pHPF, const ma_allocation_callbacks* pAllocationCallbacks)
48740 | {
48741 |     if (pHPF == NULL) {
48742 |         return;
48743 |     }
48744 |
48745 |     ma_biquad_uninit(&pHPF->bq, pAllocationCallbacks); /* <-- This will free the heap allocation.
> */
48746 | }
48747 |
48748 | MA_API ma_result ma_hpf2_reinit(const ma_hpf2_config* pConfig, ma_hpf2* pHPF)
48749 | {
48750 |     ma_result result;
48751 |     ma_biquad_config bqConfig;
48752 |
48753 |     if (pHPF == NULL || pConfig == NULL) {
48754 |         return MA_INVALID_ARGS;
48755 |     }
48756 |
48757 |     bqConfig = ma_hpf2__get_biquad_config(pConfig);
48758 |     result = ma_biquad_reinit(&bqConfig, &pHPF->bq);
48759 |     if (result != MA_SUCCESS) {
48760 |         return result;
48761 |     }
48762 |
48763 |     return MA_SUCCESS;
48764 | }
48765 |
48766 | static MA_INLINE void ma_hpf2_process_pcm_frame_s16(ma_hpf2* pHPF, ma_int16* pFrameOut, const
> ma_int16* pFrameIn)
48767 | {
48768 |     ma_biquad_process_pcm_frame_s16(&pHPF->bq, pFrameOut, pFrameIn);
48769 | }
48770 |
48771 | static MA_INLINE void ma_hpf2_process_pcm_frame_f32(ma_hpf2* pHPF, float* pFrameOut, const float*
> pFrameIn)
48772 | {

```

```

48773 |     ma_biquad_process_pcm_frame_f32(&pHPF->bq, pFrameOut, pFrameIn);
48774 | }
48775 |
48776 | MA_API ma_result ma_hpf2_process_pcm_frames(ma_hpf2* pHPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount)
48777 | {
48778 |     if (pHPF == NULL) {
48779 |         return MA_INVALID_ARGS;
48780 |     }
48781 |
48782 |     return ma_biquad_process_pcm_frames(&pHPF->bq, pFramesOut, pFramesIn, frameCount);
48783 | }
48784 |
48785 | MA_API ma_uint32 ma_hpf2_get_latency(const ma_hpf2* pHPF)
48786 | {
48787 |     if (pHPF == NULL) {
48788 |         return 0;
48789 |     }
48790 |
48791 |     return ma_biquad_get_latency(&pHPF->bq);
48792 | }
48793 |
48794 |
48795 | MA_API ma_hpf_config ma_hpf_config_init(ma_format format, ma_uint32 channels, ma_uint32 sampleRate,
> double cutoffFrequency, ma_uint32 order)
48796 | {
48797 |     ma_hpf_config config;
48798 |
48799 |     MA_ZERO_OBJECT(&config);
48800 |     config.format      = format;
48801 |     config.channels    = channels;
48802 |     config.sampleRate  = sampleRate;
48803 |     config.cutoffFrequency = cutoffFrequency;
48804 |     config.order       = ma_min(order, MA_MAX_FILTER_ORDER);
48805 |
48806 |     return config;
48807 | }
48808 |
48809 |
48810 | typedef struct
48811 | {
48812 |     size_t sizeInBytes;
48813 |     size_t hpf10ffset;
48814 |     size_t hpf20ffset; /* Offset of the first second order filter. Subsequent filters will come
> straight after, and will each have the same heap size. */
48815 | } ma_hpf_heap_layout;
48816 |
48817 | static void ma_hpf_calculate_sub_hpf_counts(ma_uint32 order, ma_uint32* pHPF1Count, ma_uint32*
> pHPF2Count)
48818 | {
48819 |     MA_ASSERT(pHPF1Count != NULL);
48820 |     MA_ASSERT(pHPF2Count != NULL);
48821 |
48822 |     *pHPF1Count = order % 2;
48823 |     *pHPF2Count = order / 2;
48824 | }
48825 |
48826 | static ma_result ma_hpf_get_heap_layout(const ma_hpf_config* pConfig, ma_hpf_heap_layout*
> pHeapLayout)
48827 | {
48828 |     ma_result result;
48829 |     ma_uint32 hpf1Count;
48830 |     ma_uint32 hpf2Count;
48831 |     ma_uint32 ihpf1;
48832 |     ma_uint32 ihpf2;
48833 |
48834 |     MA_ASSERT(pHeapLayout != NULL);
48835 |
48836 |     MA_ZERO_OBJECT(pHeapLayout);
48837 |
48838 |     if (pConfig == NULL) {
48839 |         return MA_INVALID_ARGS;
48840 |     }
48841 |
48842 |     if (pConfig->channels == 0) {
48843 |         return MA_INVALID_ARGS;
48844 |     }
48845 |
48846 |     if (pConfig->order > MA_MAX_FILTER_ORDER) {

```

```

48847 |         return MA_INVALID_ARGS;
48848 |     }
48849 |
48850 |     ma_hpf_calculate_sub_hpf_counts(pConfig->order, &hpf1Count, &hpf2Count);
48851 |
48852 |     pHeapLayout->sizeInBytes = 0;
48853 |
48854 |     /* HPF 1 */
48855 |     pHeapLayout->hpf1Offset = pHeapLayout->sizeInBytes;
48856 |     for (ihpf1 = 0; ihpf1 < hpf1Count; ihpf1 += 1) {
48857 |         size_t hpf1HeapSizeInBytes;
48858 |         ma_hpf1_config hpf1Config = ma_hpf1_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency);
48859 |
48860 |         result = ma_hpf1_get_heap_size(&hpf1Config, &hpf1HeapSizeInBytes);
48861 |         if (result != MA_SUCCESS) {
48862 |             return result;
48863 |         }
48864 |
48865 |         pHeapLayout->sizeInBytes += sizeof(ma_hpf1) + hpf1HeapSizeInBytes;
48866 |     }
48867 |
48868 |     /* HPF 2 */
48869 |     pHeapLayout->hpf2Offset = pHeapLayout->sizeInBytes;
48870 |     for (ihpf2 = 0; ihpf2 < hpf2Count; ihpf2 += 1) {
48871 |         size_t hpf2HeapSizeInBytes;
48872 |         ma_hpf2_config hpf2Config = ma_hpf2_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency, 0.707107); /* <-- The "q" parameter does not matter
> for the purpose of calculating the heap size. */
48873 |
48874 |         result = ma_hpf2_get_heap_size(&hpf2Config, &hpf2HeapSizeInBytes);
48875 |         if (result != MA_SUCCESS) {
48876 |             return result;
48877 |         }
48878 |
48879 |         pHeapLayout->sizeInBytes += sizeof(ma_hpf2) + hpf2HeapSizeInBytes;
48880 |     }
48881 |
48882 |     /* Make sure allocation size is aligned. */
48883 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
48884 |
48885 |     return MA_SUCCESS;
48886 | }
48887 |
48888 | static ma_result ma_hpf_reinit__internal(const ma_hpf_config* pConfig, void* pHeap, ma_hpf* pHPF,
> ma_bool32 isNew)
48889 | {
48890 |     ma_result result;
48891 |     ma_uint32 hpf1Count;
48892 |     ma_uint32 hpf2Count;
48893 |     ma_uint32 ihpf1;
48894 |     ma_uint32 ihpf2;
48895 |     ma_hpf_heap_layout heapLayout; /* Only used if isNew is true. */
48896 |
48897 |     if (pHPF == NULL || pConfig == NULL) {
48898 |         return MA_INVALID_ARGS;
48899 |     }
48900 |
48901 |     /* Only supporting f32 and s16. */
48902 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
48903 |         return MA_INVALID_ARGS;
48904 |     }
48905 |
48906 |     /* The format cannot be changed after initialization. */
48907 |     if (pHPF->format != ma_format_unknown && pHPF->format != pConfig->format) {
48908 |         return MA_INVALID_OPERATION;
48909 |     }
48910 |
48911 |     /* The channel count cannot be changed after initialization. */
48912 |     if (pHPF->channels != 0 && pHPF->channels != pConfig->channels) {
48913 |         return MA_INVALID_OPERATION;
48914 |     }
48915 |
48916 |     if (pConfig->order > MA_MAX_FILTER_ORDER) {
48917 |         return MA_INVALID_ARGS;
48918 |     }
48919 |
48920 |     ma_hpf_calculate_sub_hpf_counts(pConfig->order, &hpf1Count, &hpf2Count);
48921 |

```

```

48922 | /* The filter order can't change between reinit. */
48923 | if (!isNew) {
48924 |     if (pHPF->hpf1Count != hpf1Count || pHPF->hpf2Count != hpf2Count) {
48925 |         return MA_INVALID_OPERATION;
48926 |     }
48927 | }
48928 |
48929 | if (isNew) {
48930 |     result = ma_hpf_get_heap_layout(pConfig, &heapLayout);
48931 |     if (result != MA_SUCCESS) {
48932 |         return result;
48933 |     }
48934 |
48935 |     pHPF->pHeap = pHeap;
48936 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
48937 |
48938 |     pHPF->pHPF1 = (ma_hpf1*)ma_offset_ptr(pHeap, heapLayout.hpf1Offset);
48939 |     pHPF->pHPF2 = (ma_hpf2*)ma_offset_ptr(pHeap, heapLayout.hpf2Offset);
48940 | } else {
48941 |     MA_ZERO_OBJECT(&heapLayout); /* To silence a compiler warning. */
48942 | }
48943 |
48944 | for (ihpf1 = 0; ihpf1 < hpf1Count; ihpf1 += 1) {
48945 |     ma_hpf1_config hpf1Config = ma_hpf1_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency);
48946 |
48947 |     if (isNew) {
48948 |         size_t hpf1HeapSizeInBytes;
48949 |
48950 |         result = ma_hpf1_get_heap_size(&hpf1Config, &hpf1HeapSizeInBytes);
48951 |         if (result == MA_SUCCESS) {
48952 |             result = ma_hpf1_init_preallocated(&hpf1Config, ma_offset_ptr(pHeap,
> heapLayout.hpf1Offset + (sizeof(ma_hpf1) * hpf1Count) + (ihpf1 * hpf1HeapSizeInBytes)),
> &pHPF->pHPF1[ihpf1]);
48953 |         }
48954 |     } else {
48955 |         result = ma_hpf1_reinit(&hpf1Config, &pHPF->pHPF1[ihpf1]);
48956 |     }
48957 |
48958 |     if (result != MA_SUCCESS) {
48959 |         ma_uint32 jhpf1;
48960 |
48961 |         for (jhpf1 = 0; jhpf1 < ihpf1; jhpf1 += 1) {
48962 |             ma_hpf1_uninit(&pHPF->pHPF1[jhpf1], NULL); /* No need for allocation callbacks here
> since we used a preallocated heap allocation. */
48963 |         }
48964 |
48965 |         return result;
48966 |     }
48967 | }
48968 |
48969 | for (ihpf2 = 0; ihpf2 < hpf2Count; ihpf2 += 1) {
48970 |     ma_hpf2_config hpf2Config;
48971 |     double q;
48972 |     double a;
48973 |
48974 |     /* Tempting to use 0.707107, but won't result in a Butterworth filter if the order is > 2.
> */
48975 |     if (hpf1Count == 1) {
48976 |         a = (1 + ihpf2*1) * (MA_PI_D/(pConfig->order*1)); /* Odd order. */
48977 |     } else {
48978 |         a = (1 + ihpf2*2) * (MA_PI_D/(pConfig->order*2)); /* Even order. */
48979 |     }
48980 |     q = 1 / (2*ma_cosd(a));
48981 |
48982 |     hpf2Config = ma_hpf2_config_init(pConfig->format, pConfig->channels, pConfig->sampleRate,
> pConfig->cutoffFrequency, q);
48983 |
48984 |     if (isNew) {
48985 |         size_t hpf2HeapSizeInBytes;
48986 |
48987 |         result = ma_hpf2_get_heap_size(&hpf2Config, &hpf2HeapSizeInBytes);
48988 |         if (result == MA_SUCCESS) {
48989 |             result = ma_hpf2_init_preallocated(&hpf2Config, ma_offset_ptr(pHeap,
> heapLayout.hpf2Offset + (sizeof(ma_hpf2) * hpf2Count) + (ihpf2 * hpf2HeapSizeInBytes)),
> &pHPF->pHPF2[ihpf2]);
48990 |         }
48991 |     } else {
48992 |         result = ma_hpf2_reinit(&hpf2Config, &pHPF->pHPF2[ihpf2]);

```

```

48993 |     }
48994 |
48995 |     if (result != MA_SUCCESS) {
48996 |         ma_uint32 jhpf1;
48997 |         ma_uint32 jhpf2;
48998 |
48999 |         for (jhpf1 = 0; jhpf1 < hpf1Count; jhpf1 += 1) {
49000 |             ma_hpf1_uninit(&pHPF->pHPF1[jhpf1], NULL); /* No need for allocation callbacks here
> since we used a preallocated heap allocation. */
49001 |         }
49002 |
49003 |         for (jhpf2 = 0; jhpf2 < ihpf2; jhpf2 += 1) {
49004 |             ma_hpf2_uninit(&pHPF->pHPF2[jhpf2], NULL); /* No need for allocation callbacks here
> since we used a preallocated heap allocation. */
49005 |         }
49006 |
49007 |         return result;
49008 |     }
49009 | }
49010 |
49011 | pHPF->hpf1Count = hpf1Count;
49012 | pHPF->hpf2Count = hpf2Count;
49013 | pHPF->format = pConfig->format;
49014 | pHPF->channels = pConfig->channels;
49015 | pHPF->sampleRate = pConfig->sampleRate;
49016 |
49017 | return MA_SUCCESS;
49018 | }
49019 |
49020 | MA_API ma_result ma_hpf_get_heap_size(const ma_hpf_config* pConfig, size_t* pHeapSizeInBytes)
49021 | {
49022 |     ma_result result;
49023 |     ma_hpf_heap_layout heapLayout;
49024 |
49025 |     if (pHeapSizeInBytes == NULL) {
49026 |         return MA_INVALID_ARGS;
49027 |     }
49028 |
49029 |     *pHeapSizeInBytes = 0;
49030 |
49031 |     result = ma_hpf_get_heap_layout(pConfig, &heapLayout);
49032 |     if (result != MA_SUCCESS) {
49033 |         return result;
49034 |     }
49035 |
49036 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
49037 |
49038 |     return result;
49039 | }
49040 |
49041 | MA_API ma_result ma_hpf_init_preallocated(const ma_hpf_config* pConfig, void* pHeap, ma_hpf* pLPF)
49042 | {
49043 |     if (pLPF == NULL) {
49044 |         return MA_INVALID_ARGS;
49045 |     }
49046 |
49047 |     MA_ZERO_OBJECT(pLPF);
49048 |
49049 |     return ma_hpf_reinit__internal(pConfig, pHeap, pLPF, /*isNew*/MA_TRUE);
49050 | }
49051 |
49052 | MA_API ma_result ma_hpf_init(const ma_hpf_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hpf* pHPF)
49053 | {
49054 |     ma_result result;
49055 |     size_t heapSizeInBytes;
49056 |     void* pHeap;
49057 |
49058 |     result = ma_hpf_get_heap_size(pConfig, &heapSizeInBytes);
49059 |     if (result != MA_SUCCESS) {
49060 |         return result;
49061 |     }
49062 |
49063 |     if (heapSizeInBytes > 0) {
49064 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
49065 |         if (pHeap == NULL) {
49066 |             return MA_OUT_OF_MEMORY;
49067 |         }
49068 |     } else {

```

```

49069 |     pHeap = NULL;
49070 | }
49071 |
49072 |     result = ma_hpf_init_preallocated(pConfig, pHeap, pHPF);
49073 |     if (result != MA_SUCCESS) {
49074 |         ma_free(pHeap, pAllocationCallbacks);
49075 |         return result;
49076 |     }
49077 |
49078 |     pHPF->ownsHeap = MA_TRUE;
49079 |     return MA_SUCCESS;
49080 | }
49081 |
49082 | MA_API void ma_hpf_uninit(ma_hpf* pHPF, const ma_allocation_callbacks* pAllocationCallbacks)
49083 | {
49084 |     ma_uint32 ihpf1;
49085 |     ma_uint32 ihpf2;
49086 |
49087 |     if (pHPF == NULL) {
49088 |         return;
49089 |     }
49090 |
49091 |     for (ihpf1 = 0; ihpf1 < pHPF->hpf1Count; ihpf1 += 1) {
49092 |         ma_hpf1_uninit(&pHPF->pHPF1[ihpf1], pAllocationCallbacks);
49093 |     }
49094 |
49095 |     for (ihpf2 = 0; ihpf2 < pHPF->hpf2Count; ihpf2 += 1) {
49096 |         ma_hpf2_uninit(&pHPF->pHPF2[ihpf2], pAllocationCallbacks);
49097 |     }
49098 |
49099 |     if (pHPF->ownsHeap) {
49100 |         ma_free(pHPF->pHeap, pAllocationCallbacks);
49101 |     }
49102 | }
49103 |
49104 | MA_API ma_result ma_hpf_reinit(const ma_hpf_config* pConfig, ma_hpf* pHPF)
49105 | {
49106 |     return ma_hpf_reinit_internal(pConfig, NULL, pHPF, /*isNew*/MA_FALSE);
49107 | }
49108 |
49109 | MA_API ma_result ma_hpf_process_pcm_frames(ma_hpf* pHPF, void* pFramesOut, const void* pFramesIn,
49110 | > ma_uint64 frameCount)
49111 | {
49112 |     ma_result result;
49113 |     ma_uint32 ihpf1;
49114 |     ma_uint32 ihpf2;
49115 |
49116 |     if (pHPF == NULL) {
49117 |         return MA_INVALID_ARGS;
49118 |     }
49119 |
49120 |     /* Faster path for in-place. */
49121 |     if (pFramesOut == pFramesIn) {
49122 |         for (ihpf1 = 0; ihpf1 < pHPF->hpf1Count; ihpf1 += 1) {
49123 |             result = ma_hpf1_process_pcm_frames(&pHPF->pHPF1[ihpf1], pFramesOut, pFramesOut,
49124 | > frameCount);
49125 |             if (result != MA_SUCCESS) {
49126 |                 return result;
49127 |             }
49128 |
49129 |             for (ihpf2 = 0; ihpf2 < pHPF->hpf2Count; ihpf2 += 1) {
49130 |                 result = ma_hpf2_process_pcm_frames(&pHPF->pHPF2[ihpf2], pFramesOut, pFramesOut,
49131 | > frameCount);
49132 |                 if (result != MA_SUCCESS) {
49133 |                     return result;
49134 |                 }
49135 |             }
49136 |
49137 |             /* Slightly slower path for copying. */
49138 |             if (pFramesOut != pFramesIn) {
49139 |                 ma_uint32 iFrame;
49140 |
49141 |                 /* */ if (pHPF->format == ma_format_f32) {
49142 |                     /* */ float* pFramesOutF32 = (float*)pFramesOut;
49143 |                     const float* pFramesInF32 = (const float*)pFramesIn;
49144 |
49145 |                     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {

```



```

49145 |         MA_COPY_MEMORY(pFramesOutF32, pFramesInF32, ma_get_bytes_per_frame(pHPF->format,
> pHPF->channels));
49146 |
49147 |         for (ihpf1 = 0; ihpf1 < pHPF->hpf1Count; ihpf1 += 1) {
49148 |             ma_hpf1_process_pcm_frame_f32(&pHPF->pHPF1[ihpf1], pFramesOutF32,
> pFramesOutF32);
49149 |         }
49150 |
49151 |         for (ihpf2 = 0; ihpf2 < pHPF->hpf2Count; ihpf2 += 1) {
49152 |             ma_hpf2_process_pcm_frame_f32(&pHPF->pHPF2[ihpf2], pFramesOutF32,
> pFramesOutF32);
49153 |         }
49154 |
49155 |         pFramesOutF32 += pHPF->channels;
49156 |         pFramesInF32 += pHPF->channels;
49157 |     }
49158 | } else if (pHPF->format == ma_format_s16) {
49159 |     /* */ ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
49160 |     const ma_int16* pFramesInS16 = (const ma_int16*)pFramesIn;
49161 |
49162 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
49163 |         MA_COPY_MEMORY(pFramesOutS16, pFramesInS16, ma_get_bytes_per_frame(pHPF->format,
> pHPF->channels));
49164 |
49165 |         for (ihpf1 = 0; ihpf1 < pHPF->hpf1Count; ihpf1 += 1) {
49166 |             ma_hpf1_process_pcm_frame_s16(&pHPF->pHPF1[ihpf1], pFramesOutS16,
> pFramesOutS16);
49167 |         }
49168 |
49169 |         for (ihpf2 = 0; ihpf2 < pHPF->hpf2Count; ihpf2 += 1) {
49170 |             ma_hpf2_process_pcm_frame_s16(&pHPF->pHPF2[ihpf2], pFramesOutS16,
> pFramesOutS16);
49171 |         }
49172 |
49173 |         pFramesOutS16 += pHPF->channels;
49174 |         pFramesInS16 += pHPF->channels;
49175 |     }
49176 | } else {
49177 |     MA_ASSERT(MA_FALSE);
49178 |     return MA_INVALID_OPERATION;    /* Should never hit this. */
49179 | }
49180 | }
49181 |
49182 | return MA_SUCCESS;
49183 | }
49184 |
49185 | MA_API ma_uint32 ma_hpf_get_latency(const ma_hpf* pHPF)
49186 | {
49187 |     if (pHPF == NULL) {
49188 |         return 0;
49189 |     }
49190 |
49191 |     return pHPF->hpf2Count*2 + pHPF->hpf1Count;
49192 | }
49193 |
49194 |
49195 | /*****
> *****/
49196 |
49197 | Band-Pass Filtering
49198 |
49199 | *****/
> *****/
49200 | MA_API ma_bpf2_config ma_bpf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double cutoffFrequency, double q)
49201 | {
49202 |     ma_bpf2_config config;
49203 |
49204 |     MA_ZERO_OBJECT(&config);
49205 |     config.format = format;
49206 |     config.channels = channels;
49207 |     config.sampleRate = sampleRate;
49208 |     config.cutoffFrequency = cutoffFrequency;
49209 |     config.q = q;
49210 |
49211 |     /* Q cannot be 0 or else it'll result in a division by 0. In this case just default to 0.707107.
> */
49212 |     if (config.q == 0) {
49213 |         config.q = 0.707107;

```

```

49214 |     }
49215 |
49216 |     return config;
49217 | }
49218 |
49219 |
49220 | static MA_INLINE ma_biquad_config ma_bpf2__get_biquad_config(const ma_bpf2_config* pConfig)
49221 | {
49222 |     ma_biquad_config bqConfig;
49223 |     double q;
49224 |     double w;
49225 |     double s;
49226 |     double c;
49227 |     double a;
49228 |
49229 |     MA_ASSERT(pConfig != NULL);
49230 |
49231 |     q = pConfig->q;
49232 |     w = 2 * MA_PI_D * pConfig->cutoffFrequency / pConfig->sampleRate;
49233 |     s = ma_sind(w);
49234 |     c = ma_cosd(w);
49235 |     a = s / (2*q);
49236 |
49237 |     bqConfig.b0 = q * a;
49238 |     bqConfig.b1 = 0;
49239 |     bqConfig.b2 = -q * a;
49240 |     bqConfig.a0 = 1 + a;
49241 |     bqConfig.a1 = -2 * c;
49242 |     bqConfig.a2 = 1 - a;
49243 |
49244 |     bqConfig.format = pConfig->format;
49245 |     bqConfig.channels = pConfig->channels;
49246 |
49247 |     return bqConfig;
49248 | }
49249 |
49250 | MA_API ma_result ma_bpf2_get_heap_size(const ma_bpf2_config* pConfig, size_t* pHeapSizeInBytes)
49251 | {
49252 |     ma_biquad_config bqConfig;
49253 |     bqConfig = ma_bpf2__get_biquad_config(pConfig);
49254 |
49255 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
49256 | }
49257 |
49258 | MA_API ma_result ma_bpf2_init_preallocated(const ma_bpf2_config* pConfig, void* pHeap, ma_bpf2*
> pBPF)
49259 | {
49260 |     ma_result result;
49261 |     ma_biquad_config bqConfig;
49262 |
49263 |     if (pBPF == NULL) {
49264 |         return MA_INVALID_ARGS;
49265 |     }
49266 |
49267 |     MA_ZERO_OBJECT(pBPF);
49268 |
49269 |     if (pConfig == NULL) {
49270 |         return MA_INVALID_ARGS;
49271 |     }
49272 |
49273 |     bqConfig = ma_bpf2__get_biquad_config(pConfig);
49274 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pBPF->bq);
49275 |     if (result != MA_SUCCESS) {
49276 |         return result;
49277 |     }
49278 |
49279 |     return MA_SUCCESS;
49280 | }
49281 |
49282 | MA_API ma_result ma_bpf2_init(const ma_bpf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_bpf2* pBPF)
49283 | {
49284 |     ma_result result;
49285 |     size_t heapSizeInBytes;
49286 |     void* pHeap;
49287 |
49288 |     result = ma_bpf2_get_heap_size(pConfig, &heapSizeInBytes);
49289 |     if (result != MA_SUCCESS) {
49290 |         return result;

```

```

49291 |     }
49292 |
49293 |     if (heapSizeInBytes > 0) {
49294 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
49295 |         if (pHeap == NULL) {
49296 |             return MA_OUT_OF_MEMORY;
49297 |         }
49298 |     } else {
49299 |         pHeap = NULL;
49300 |     }
49301 |
49302 |     result = ma_bpf2_init_preallocated(pConfig, pHeap, pBPF);
49303 |     if (result != MA_SUCCESS) {
49304 |         ma_free(pHeap, pAllocationCallbacks);
49305 |         return result;
49306 |     }
49307 |
49308 |     pBPF->bq._ownsHeap = MA_TRUE; /* <-- This will cause the biquad to take ownership of the heap
> and free it when it's uninitialized. */
49309 |     return MA_SUCCESS;
49310 | }
49311 |
49312 | MA_API void ma_bpf2_uninit(ma_bpf2* pBPF, const ma_allocation_callbacks* pAllocationCallbacks)
49313 | {
49314 |     if (pBPF == NULL) {
49315 |         return;
49316 |     }
49317 |
49318 |     ma_biquad_uninit(&pBPF->bq, pAllocationCallbacks); /* <-- This will free the heap allocation.
> */
49319 | }
49320 |
49321 | MA_API ma_result ma_bpf2_reinit(const ma_bpf2_config* pConfig, ma_bpf2* pBPF)
49322 | {
49323 |     ma_result result;
49324 |     ma_biquad_config bqConfig;
49325 |
49326 |     if (pBPF == NULL || pConfig == NULL) {
49327 |         return MA_INVALID_ARGS;
49328 |     }
49329 |
49330 |     bqConfig = ma_bpf2__get_biquad_config(pConfig);
49331 |     result = ma_biquad_reinit(&bqConfig, &pBPF->bq);
49332 |     if (result != MA_SUCCESS) {
49333 |         return result;
49334 |     }
49335 |
49336 |     return MA_SUCCESS;
49337 | }
49338 |
49339 | static MA_INLINE void ma_bpf2_process_pcm_frame_s16(ma_bpf2* pBPF, ma_int16* pFrameOut, const
> ma_int16* pFrameIn)
49340 | {
49341 |     ma_biquad_process_pcm_frame_s16(&pBPF->bq, pFrameOut, pFrameIn);
49342 | }
49343 |
49344 | static MA_INLINE void ma_bpf2_process_pcm_frame_f32(ma_bpf2* pBPF, float* pFrameOut, const float*
> pFrameIn)
49345 | {
49346 |     ma_biquad_process_pcm_frame_f32(&pBPF->bq, pFrameOut, pFrameIn);
49347 | }
49348 |
49349 | MA_API ma_result ma_bpf2_process_pcm_frames(ma_bpf2* pBPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount)
49350 | {
49351 |     if (pBPF == NULL) {
49352 |         return MA_INVALID_ARGS;
49353 |     }
49354 |
49355 |     return ma_biquad_process_pcm_frames(&pBPF->bq, pFramesOut, pFramesIn, frameCount);
49356 | }
49357 |
49358 | MA_API ma_uint32 ma_bpf2_get_latency(const ma_bpf2* pBPF)
49359 | {
49360 |     if (pBPF == NULL) {
49361 |         return 0;
49362 |     }
49363 |
49364 |     return ma_biquad_get_latency(&pBPF->bq);

```

```

49365 | }
49366 |
49367 |
49368 | MA_API ma_bpf_config ma_bpf_config_init(ma_format format, ma_uint32 channels, ma_uint32 sampleRate,
> double cutoffFrequency, ma_uint32 order)
49369 | {
49370 |     ma_bpf_config config;
49371 |
49372 |     MA_ZERO_OBJECT(&config);
49373 |     config.format      = format;
49374 |     config.channels    = channels;
49375 |     config.sampleRate  = sampleRate;
49376 |     config.cutoffFrequency = cutoffFrequency;
49377 |     config.order       = ma_min(order, MA_MAX_FILTER_ORDER);
49378 |
49379 |     return config;
49380 | }
49381 |
49382 |
49383 | typedef struct
49384 | {
49385 |     size_t sizeInBytes;
49386 |     size_t bpf2Offset;
49387 | } ma_bpf_heap_layout;
49388 |
49389 | static ma_result ma_bpf_get_heap_layout(const ma_bpf_config* pConfig, ma_bpf_heap_layout*
> pHeapLayout)
49390 | {
49391 |     ma_result result;
49392 |     ma_uint32 bpf2Count;
49393 |     ma_uint32 ibpf2;
49394 |
49395 |     MA_ASSERT(pHeapLayout != NULL);
49396 |
49397 |     MA_ZERO_OBJECT(pHeapLayout);
49398 |
49399 |     if (pConfig == NULL) {
49400 |         return MA_INVALID_ARGS;
49401 |     }
49402 |
49403 |     if (pConfig->order > MA_MAX_FILTER_ORDER) {
49404 |         return MA_INVALID_ARGS;
49405 |     }
49406 |
49407 |     /* We must have an even number of order. */
49408 |     if ((pConfig->order & 0x1) != 0) {
49409 |         return MA_INVALID_ARGS;
49410 |     }
49411 |
49412 |     bpf2Count = pConfig->order / 2;
49413 |
49414 |     pHeapLayout->sizeInBytes = 0;
49415 |
49416 |     /* BPF 2 */
49417 |     pHeapLayout->bpf2Offset = pHeapLayout->sizeInBytes;
49418 |     for (ibpf2 = 0; ibpf2 < bpf2Count; ibpf2 += 1) {
49419 |         size_t bpf2HeapSizeInBytes;
49420 |         ma_bpf2_config bpf2Config = ma_bpf2_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRate, pConfig->cutoffFrequency, 0.707107); /* <-- The "q" parameter does not matter
> for the purpose of calculating the heap size. */
49421 |
49422 |         result = ma_bpf2_get_heap_size(&bpf2Config, &bpf2HeapSizeInBytes);
49423 |         if (result != MA_SUCCESS) {
49424 |             return result;
49425 |         }
49426 |
49427 |         pHeapLayout->sizeInBytes += sizeof(ma_bpf2) + bpf2HeapSizeInBytes;
49428 |     }
49429 |
49430 |     /* Make sure allocation size is aligned. */
49431 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
49432 |
49433 |     return MA_SUCCESS;
49434 | }
49435 |
49436 | static ma_result ma_bpf_reinit__internal(const ma_bpf_config* pConfig, void* pHeap, ma_bpf* pBPF,
> ma_bool32 isNew)
49437 | {
49438 |     ma_result result;

```

```

49439 |     ma_uint32 bpf2Count;
49440 |     ma_uint32 ibpf2;
49441 |     ma_bpf_heap_layout heapLayout; /* Only used if isNew is true. */
49442 |
49443 |     if (pBPF == NULL || pConfig == NULL) {
49444 |         return MA_INVALID_ARGS;
49445 |     }
49446 |
49447 |     /* Only supporting f32 and s16. */
49448 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
49449 |         return MA_INVALID_ARGS;
49450 |     }
49451 |
49452 |     /* The format cannot be changed after initialization. */
49453 |     if (pBPF->format != ma_format_unknown && pBPF->format != pConfig->format) {
49454 |         return MA_INVALID_OPERATION;
49455 |     }
49456 |
49457 |     /* The channel count cannot be changed after initialization. */
49458 |     if (pBPF->channels != 0 && pBPF->channels != pConfig->channels) {
49459 |         return MA_INVALID_OPERATION;
49460 |     }
49461 |
49462 |     if (pConfig->order > MA_MAX_FILTER_ORDER) {
49463 |         return MA_INVALID_ARGS;
49464 |     }
49465 |
49466 |     /* We must have an even number of order. */
49467 |     if ((pConfig->order & 0x1) != 0) {
49468 |         return MA_INVALID_ARGS;
49469 |     }
49470 |
49471 |     bpf2Count = pConfig->order / 2;
49472 |
49473 |     /* The filter order can't change between reinit. */
49474 |     if (!isNew) {
49475 |         if (pBPF->bpf2Count != bpf2Count) {
49476 |             return MA_INVALID_OPERATION;
49477 |         }
49478 |     }
49479 |
49480 |     if (isNew) {
49481 |         result = ma_bpf_get_heap_layout(pConfig, &heapLayout);
49482 |         if (result != MA_SUCCESS) {
49483 |             return result;
49484 |         }
49485 |
49486 |         pBPF->pHeap = pHeap;
49487 |         MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
49488 |
49489 |         pBPF->pBPF2 = (ma_bpf2*)ma_offset_ptr(pHeap, heapLayout.bpf2Offset);
49490 |     } else {
49491 |         MA_ZERO_OBJECT(&heapLayout);
49492 |     }
49493 |
49494 |     for (ibpf2 = 0; ibpf2 < bpf2Count; ibpf2 += 1) {
49495 |         ma_bpf2_config bpf2Config;
49496 |         double q;
49497 |
49498 |         /* TODO: Calculate Q to make this a proper Butterworth filter. */
49499 |         q = 0.707107;
49500 |
49501 |         bpf2Config = ma_bpf2_config_init(pConfig->format, pConfig->channels, pConfig->sampleRate,
> pConfig->cutoffFrequency, q);
49502 |
49503 |         if (isNew) {
49504 |             size_t bpf2HeapSizeInBytes;
49505 |
49506 |             result = ma_bpf2_get_heap_size(&bpf2Config, &bpf2HeapSizeInBytes);
49507 |             if (result == MA_SUCCESS) {
49508 |                 result = ma_bpf2_init_preallocated(&bpf2Config, ma_offset_ptr(pHeap,
> heapLayout.bpf2Offset + (sizeof(ma_bpf2) * bpf2Count) + (ibpf2 * bpf2HeapSizeInBytes)),
> &pBPF->pBPF2[ibpf2]);
49509 |             }
49510 |         } else {
49511 |             result = ma_bpf2_reinit(&bpf2Config, &pBPF->pBPF2[ibpf2]);
49512 |         }
49513 |
49514 |         if (result != MA_SUCCESS) {

```

```

49515 |         return result;
49516 |     }
49517 | }
49518 |
49519 |     pBPF->bpf2Count = bpf2Count;
49520 |     pBPF->format     = pConfig->format;
49521 |     pBPF->channels    = pConfig->channels;
49522 |
49523 |     return MA_SUCCESS;
49524 | }
49525 |
49526 |
49527 | MA_API ma_result ma_bpf_get_heap_size(const ma_bpf_config* pConfig, size_t* pHeapSizeInBytes)
49528 | {
49529 |     ma_result result;
49530 |     ma_bpf_heap_layout heapLayout;
49531 |
49532 |     if (pHeapSizeInBytes == NULL) {
49533 |         return MA_INVALID_ARGS;
49534 |     }
49535 |
49536 |     *pHeapSizeInBytes = 0;
49537 |
49538 |     result = ma_bpf_get_heap_layout(pConfig, &heapLayout);
49539 |     if (result != MA_SUCCESS) {
49540 |         return result;
49541 |     }
49542 |
49543 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
49544 |
49545 |     return MA_SUCCESS;
49546 | }
49547 |
49548 | MA_API ma_result ma_bpf_init_preallocated(const ma_bpf_config* pConfig, void* pHeap, ma_bpf* pBPF)
49549 | {
49550 |     if (pBPF == NULL) {
49551 |         return MA_INVALID_ARGS;
49552 |     }
49553 |
49554 |     MA_ZERO_OBJECT(pBPF);
49555 |
49556 |     return ma_bpf_reinit__internal(pConfig, pHeap, pBPF, /*isNew*/MA_TRUE);
49557 | }
49558 |
49559 | MA_API ma_result ma_bpf_init(const ma_bpf_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_bpf* pBPF)
49560 | {
49561 |     ma_result result;
49562 |     size_t heapSizeInBytes;
49563 |     void* pHeap;
49564 |
49565 |     result = ma_bpf_get_heap_size(pConfig, &heapSizeInBytes);
49566 |     if (result != MA_SUCCESS) {
49567 |         return result;
49568 |     }
49569 |
49570 |     if (heapSizeInBytes > 0) {
49571 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
49572 |         if (pHeap == NULL) {
49573 |             return MA_OUT_OF_MEMORY;
49574 |         }
49575 |     } else {
49576 |         pHeap = NULL;
49577 |     }
49578 |
49579 |     result = ma_bpf_init_preallocated(pConfig, pHeap, pBPF);
49580 |     if (result != MA_SUCCESS) {
49581 |         ma_free(pHeap, pAllocationCallbacks);
49582 |         return result;
49583 |     }
49584 |
49585 |     pBPF->_ownsHeap = MA_TRUE;
49586 |     return MA_SUCCESS;
49587 | }
49588 |
49589 | MA_API void ma_bpf_uninit(ma_bpf* pBPF, const ma_allocation_callbacks* pAllocationCallbacks)
49590 | {
49591 |     ma_uint32 ibpf2;
49592 |

```

```

49593 |     if (pBPF == NULL) {
49594 |         return;
49595 |     }
49596 |
49597 |     for (ibpf2 = 0; ibpf2 < pBPF->bp2Count; ibpf2 += 1) {
49598 |         ma_bp2_uninit(&pBPF->pBP2[ibpf2], pAllocationCallbacks);
49599 |     }
49600 |
49601 |     if (pBPF->ownsHeap) {
49602 |         ma_free(pBPF->pHeap, pAllocationCallbacks);
49603 |     }
49604 | }
49605 |
49606 | MA_API ma_result ma_bp2_reinit(const ma_bp2_config* pConfig, ma_bp2* pBPF)
49607 | {
49608 |     return ma_bp2_reinit__internal(pConfig, NULL, pBPF, /*isNew*/MA_FALSE);
49609 | }
49610 |
49611 | MA_API ma_result ma_bp2_process_pcm_frames(ma_bp2* pBPF, void* pFramesOut, const void* pFramesIn,
> ma_uint64 frameCount)
49612 | {
49613 |     ma_result result;
49614 |     ma_uint32 ibpf2;
49615 |
49616 |     if (pBPF == NULL) {
49617 |         return MA_INVALID_ARGS;
49618 |     }
49619 |
49620 |     /* Faster path for in-place. */
49621 |     if (pFramesOut == pFramesIn) {
49622 |         for (ibpf2 = 0; ibpf2 < pBPF->bp2Count; ibpf2 += 1) {
49623 |             result = ma_bp2_process_pcm_frames(&pBPF->pBP2[ibpf2], pFramesOut, pFramesOut,
> frameCount);
49624 |             if (result != MA_SUCCESS) {
49625 |                 return result;
49626 |             }
49627 |         }
49628 |     }
49629 |
49630 |     /* Slightly slower path for copying. */
49631 |     if (pFramesOut != pFramesIn) {
49632 |         ma_uint32 iFrame;
49633 |
49634 |         /* */ if (pBPF->format == ma_format_f32) {
49635 |             /* */ float* pFramesOutF32 = (float*)pFramesOut;
49636 |             const float* pFramesInF32 = (const float*)pFramesIn;
49637 |
49638 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
49639 |                 MA_COPY_MEMORY(pFramesOutF32, pFramesInF32, ma_get_bytes_per_frame(pBPF->format,
> pBPF->channels));
49640 |
49641 |                 for (ibpf2 = 0; ibpf2 < pBPF->bp2Count; ibpf2 += 1) {
49642 |                     ma_bp2_process_pcm_frame_f32(&pBPF->pBP2[ibpf2], pFramesOutF32,
> pFramesOutF32);
49643 |                 }
49644 |
49645 |                 pFramesOutF32 += pBPF->channels;
49646 |                 pFramesInF32 += pBPF->channels;
49647 |             }
49648 |         } else if (pBPF->format == ma_format_s16) {
49649 |             /* */ ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
49650 |             const ma_int16* pFramesInS16 = (const ma_int16*)pFramesIn;
49651 |
49652 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
49653 |                 MA_COPY_MEMORY(pFramesOutS16, pFramesInS16, ma_get_bytes_per_frame(pBPF->format,
> pBPF->channels));
49654 |
49655 |                 for (ibpf2 = 0; ibpf2 < pBPF->bp2Count; ibpf2 += 1) {
49656 |                     ma_bp2_process_pcm_frame_s16(&pBPF->pBP2[ibpf2], pFramesOutS16,
> pFramesOutS16);
49657 |                 }
49658 |
49659 |                 pFramesOutS16 += pBPF->channels;
49660 |                 pFramesInS16 += pBPF->channels;
49661 |             }
49662 |         } else {
49663 |             MA_ASSERT(MA_FALSE);
49664 |             return MA_INVALID_OPERATION; /* Should never hit this. */
49665 |         }

```



```

49666 |     }
49667 |
49668 |     return MA_SUCCESS;
49669 | }
49670 |
49671 | MA_API ma_uint32 ma_bpf_get_latency(const ma_bpf* pBPF)
49672 | {
49673 |     if (pBPF == NULL) {
49674 |         return 0;
49675 |     }
49676 |
49677 |     return pBPF->bpf2Count*2;
49678 | }
49679 |
49680 |
49681 | /*****
> *****/
49682 |
49683 | Notching Filter
49684 |
49685 | *****/
> *****/
49686 | MA_API ma_notch2_config ma_notch2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double q, double frequency)
49687 | {
49688 |     ma_notch2_config config;
49689 |
49690 |     MA_ZERO_OBJECT(&config);
49691 |     config.format      = format;
49692 |     config.channels    = channels;
49693 |     config.sampleRate  = sampleRate;
49694 |     config.q           = q;
49695 |     config.frequency   = frequency;
49696 |
49697 |     if (config.q == 0) {
49698 |         config.q = 0.707107;
49699 |     }
49700 |
49701 |     return config;
49702 | }
49703 |
49704 |
49705 | static MA_INLINE ma_biquad_config ma_notch2__get_biquad_config(const ma_notch2_config* pConfig)
49706 | {
49707 |     ma_biquad_config bqConfig;
49708 |     double q;
49709 |     double w;
49710 |     double s;
49711 |     double c;
49712 |     double a;
49713 |
49714 |     MA_ASSERT(pConfig != NULL);
49715 |
49716 |     q = pConfig->q;
49717 |     w = 2 * MA_PI_D * pConfig->frequency / pConfig->sampleRate;
49718 |     s = ma_sind(w);
49719 |     c = ma_cosd(w);
49720 |     a = s / (2*q);
49721 |
49722 |     bqConfig.b0 = 1;
49723 |     bqConfig.b1 = -2 * c;
49724 |     bqConfig.b2 = 1;
49725 |     bqConfig.a0 = 1 + a;
49726 |     bqConfig.a1 = -2 * c;
49727 |     bqConfig.a2 = 1 - a;
49728 |
49729 |     bqConfig.format = pConfig->format;
49730 |     bqConfig.channels = pConfig->channels;
49731 |
49732 |     return bqConfig;
49733 | }
49734 |
49735 | MA_API ma_result ma_notch2_get_heap_size(const ma_notch2_config* pConfig, size_t* pHeapSizeInBytes)
49736 | {
49737 |     ma_biquad_config bqConfig;
49738 |     bqConfig = ma_notch2__get_biquad_config(pConfig);
49739 |
49740 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
49741 | }

```

```

49742 |
49743 | MA_API ma_result ma_notch2_init_preallocated(const ma_notch2_config* pConfig, void* pHeap,
> ma_notch2* pFilter)
49744 | {
49745 |     ma_result result;
49746 |     ma_biquad_config bqConfig;
49747 |
49748 |     if (pFilter == NULL) {
49749 |         return MA_INVALID_ARGS;
49750 |     }
49751 |
49752 |     MA_ZERO_OBJECT(pFilter);
49753 |
49754 |     if (pConfig == NULL) {
49755 |         return MA_INVALID_ARGS;
49756 |     }
49757 |
49758 |     bqConfig = ma_notch2__get_biquad_config(pConfig);
49759 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pFilter->bq);
49760 |     if (result != MA_SUCCESS) {
49761 |         return result;
49762 |     }
49763 |
49764 |     return MA_SUCCESS;
49765 | }
49766 |
49767 | MA_API ma_result ma_notch2_init(const ma_notch2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_notch2* pFilter)
49768 | {
49769 |     ma_result result;
49770 |     size_t heapSizeInBytes;
49771 |     void* pHeap;
49772 |
49773 |     result = ma_notch2_get_heap_size(pConfig, &heapSizeInBytes);
49774 |     if (result != MA_SUCCESS) {
49775 |         return result;
49776 |     }
49777 |
49778 |     if (heapSizeInBytes > 0) {
49779 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
49780 |         if (pHeap == NULL) {
49781 |             return MA_OUT_OF_MEMORY;
49782 |         }
49783 |     } else {
49784 |         pHeap = NULL;
49785 |     }
49786 |
49787 |     result = ma_notch2_init_preallocated(pConfig, pHeap, pFilter);
49788 |     if (result != MA_SUCCESS) {
49789 |         ma_free(pHeap, pAllocationCallbacks);
49790 |         return result;
49791 |     }
49792 |
49793 |     pFilter->bq.ownsHeap = MA_TRUE;    /* <-- This will cause the biquad to take ownership of the
> heap and free it when it's uninitialized. */
49794 |     return MA_SUCCESS;
49795 | }
49796 |
49797 | MA_API void ma_notch2_uninit(ma_notch2* pFilter, const ma_allocation_callbacks*
> pAllocationCallbacks)
49798 | {
49799 |     if (pFilter == NULL) {
49800 |         return;
49801 |     }
49802 |
49803 |     ma_biquad_uninit(&pFilter->bq, pAllocationCallbacks);    /* <-- This will free the heap
> allocation. */
49804 | }
49805 |
49806 | MA_API ma_result ma_notch2_reinit(const ma_notch2_config* pConfig, ma_notch2* pFilter)
49807 | {
49808 |     ma_result result;
49809 |     ma_biquad_config bqConfig;
49810 |
49811 |     if (pFilter == NULL || pConfig == NULL) {
49812 |         return MA_INVALID_ARGS;
49813 |     }
49814 |
49815 |     bqConfig = ma_notch2__get_biquad_config(pConfig);

```

```

49816 |     result = ma_biquad_reinit(&bqConfig, &pFilter->bq);
49817 |     if (result != MA_SUCCESS) {
49818 |         return result;
49819 |     }
49820 |
49821 |     return MA_SUCCESS;
49822 | }
49823 |
49824 | static MA_INLINE void ma_notch2_process_pcm_frame_s16(ma_notch2* pFilter, ma_int16* pFrameOut, const
> ma_int16* pFrameIn)
49825 | {
49826 |     ma_biquad_process_pcm_frame_s16(&pFilter->bq, pFrameOut, pFrameIn);
49827 | }
49828 |
49829 | static MA_INLINE void ma_notch2_process_pcm_frame_f32(ma_notch2* pFilter, float* pFrameOut, const
> float* pFrameIn)
49830 | {
49831 |     ma_biquad_process_pcm_frame_f32(&pFilter->bq, pFrameOut, pFrameIn);
49832 | }
49833 |
49834 | MA_API ma_result ma_notch2_process_pcm_frames(ma_notch2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
49835 | {
49836 |     if (pFilter == NULL) {
49837 |         return MA_INVALID_ARGS;
49838 |     }
49839 |
49840 |     return ma_biquad_process_pcm_frames(&pFilter->bq, pFramesOut, pFramesIn, frameCount);
49841 | }
49842 |
49843 | MA_API ma_uint32 ma_notch2_get_latency(const ma_notch2* pFilter)
49844 | {
49845 |     if (pFilter == NULL) {
49846 |         return 0;
49847 |     }
49848 |
49849 |     return ma_biquad_get_latency(&pFilter->bq);
49850 | }
49851 |
49852 |
49853 |
49854 | /*****
> *****/
49855 |
49856 | Peaking EQ Filter
49857 |
49858 | *****/
> *****/
49859 | MA_API ma_peak2_config ma_peak2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double gainDB, double q, double frequency)
49860 | {
49861 |     ma_peak2_config config;
49862 |
49863 |     MA_ZERO_OBJECT(&config);
49864 |     config.format = format;
49865 |     config.channels = channels;
49866 |     config.sampleRate = sampleRate;
49867 |     config.gainDB = gainDB;
49868 |     config.q = q;
49869 |     config.frequency = frequency;
49870 |
49871 |     if (config.q == 0) {
49872 |         config.q = 0.707107;
49873 |     }
49874 |
49875 |     return config;
49876 | }
49877 |
49878 |
49879 | static MA_INLINE ma_biquad_config ma_peak2__get_biquad_config(const ma_peak2_config* pConfig)
49880 | {
49881 |     ma_biquad_config bqConfig;
49882 |     double q;
49883 |     double w;
49884 |     double s;
49885 |     double c;
49886 |     double a;
49887 |     double A;
49888 |

```

```

49889 |     MA_ASSERT(pConfig != NULL);
49890 |
49891 |     q = pConfig->q;
49892 |     w = 2 * MA_PI_D * pConfig->frequency / pConfig->sampleRate;
49893 |     s = ma_sind(w);
49894 |     c = ma_cosd(w);
49895 |     a = s / (2*q);
49896 |     A = ma_powd(10, (pConfig->gainDB / 40));
49897 |
49898 |     bqConfig.b0 = 1 + (a * A);
49899 |     bqConfig.b1 = -2 * c;
49900 |     bqConfig.b2 = 1 - (a * A);
49901 |     bqConfig.a0 = 1 + (a / A);
49902 |     bqConfig.a1 = -2 * c;
49903 |     bqConfig.a2 = 1 - (a / A);
49904 |
49905 |     bqConfig.format = pConfig->format;
49906 |     bqConfig.channels = pConfig->channels;
49907 |
49908 |     return bqConfig;
49909 | }
49910 |
49911 | MA_API ma_result ma_peak2_get_heap_size(const ma_peak2_config* pConfig, size_t* pHeapSizeInBytes)
49912 | {
49913 |     ma_biquad_config bqConfig;
49914 |     bqConfig = ma_peak2__get_biquad_config(pConfig);
49915 |
49916 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
49917 | }
49918 |
49919 | MA_API ma_result ma_peak2_init_preallocated(const ma_peak2_config* pConfig, void* pHeap, ma_peak2*
> pFilter)
49920 | {
49921 |     ma_result result;
49922 |     ma_biquad_config bqConfig;
49923 |
49924 |     if (pFilter == NULL) {
49925 |         return MA_INVALID_ARGS;
49926 |     }
49927 |
49928 |     MA_ZERO_OBJECT(pFilter);
49929 |
49930 |     if (pConfig == NULL) {
49931 |         return MA_INVALID_ARGS;
49932 |     }
49933 |
49934 |     bqConfig = ma_peak2__get_biquad_config(pConfig);
49935 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pFilter->bq);
49936 |     if (result != MA_SUCCESS) {
49937 |         return result;
49938 |     }
49939 |
49940 |     return MA_SUCCESS;
49941 | }
49942 |
49943 | MA_API ma_result ma_peak2_init(const ma_peak2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_peak2* pFilter)
49944 | {
49945 |     ma_result result;
49946 |     size_t heapSizeInBytes;
49947 |     void* pHeap;
49948 |
49949 |     result = ma_peak2_get_heap_size(pConfig, &heapSizeInBytes);
49950 |     if (result != MA_SUCCESS) {
49951 |         return result;
49952 |     }
49953 |
49954 |     if (heapSizeInBytes > 0) {
49955 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
49956 |         if (pHeap == NULL) {
49957 |             return MA_OUT_OF_MEMORY;
49958 |         }
49959 |     } else {
49960 |         pHeap = NULL;
49961 |     }
49962 |
49963 |     result = ma_peak2_init_preallocated(pConfig, pHeap, pFilter);
49964 |     if (result != MA_SUCCESS) {
49965 |         ma_free(pHeap, pAllocationCallbacks);

```

```

49966 |         return result;
49967 |     }
49968 |
49969 |     pFilter->bq_ownsHeap = MA_TRUE;    /* <-- This will cause the biquad to take ownership of the
> heap and free it when it's uninitialized. */
49970 |     return MA_SUCCESS;
49971 | }
49972 |
49973 | MA_API void ma_peak2_uninit(ma_peak2* pFilter, const ma_allocation_callbacks* pAllocationCallbacks)
49974 | {
49975 |     if (pFilter == NULL) {
49976 |         return;
49977 |     }
49978 |
49979 |     ma_biquad_uninit(&pFilter->bq, pAllocationCallbacks);    /* <-- This will free the heap
> allocation. */
49980 | }
49981 |
49982 | MA_API ma_result ma_peak2_reinit(const ma_peak2_config* pConfig, ma_peak2* pFilter)
49983 | {
49984 |     ma_result result;
49985 |     ma_biquad_config bqConfig;
49986 |
49987 |     if (pFilter == NULL || pConfig == NULL) {
49988 |         return MA_INVALID_ARGS;
49989 |     }
49990 |
49991 |     bqConfig = ma_peak2_get_biquad_config(pConfig);
49992 |     result = ma_biquad_reinit(&bqConfig, &pFilter->bq);
49993 |     if (result != MA_SUCCESS) {
49994 |         return result;
49995 |     }
49996 |
49997 |     return MA_SUCCESS;
49998 | }
49999 |
50000 | static MA_INLINE void ma_peak2_process_pcm_frame_s16(ma_peak2* pFilter, ma_int16* pFrameOut, const
> ma_int16* pFrameIn)
50001 | {
50002 |     ma_biquad_process_pcm_frame_s16(&pFilter->bq, pFrameOut, pFrameIn);
50003 | }
50004 |
50005 | static MA_INLINE void ma_peak2_process_pcm_frame_f32(ma_peak2* pFilter, float* pFrameOut, const
> float* pFrameIn)
50006 | {
50007 |     ma_biquad_process_pcm_frame_f32(&pFilter->bq, pFrameOut, pFrameIn);
50008 | }
50009 |
50010 | MA_API ma_result ma_peak2_process_pcm_frames(ma_peak2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
50011 | {
50012 |     if (pFilter == NULL) {
50013 |         return MA_INVALID_ARGS;
50014 |     }
50015 |
50016 |     return ma_biquad_process_pcm_frames(&pFilter->bq, pFramesOut, pFramesIn, frameCount);
50017 | }
50018 |
50019 | MA_API ma_uint32 ma_peak2_get_latency(const ma_peak2* pFilter)
50020 | {
50021 |     if (pFilter == NULL) {
50022 |         return 0;
50023 |     }
50024 |
50025 |     return ma_biquad_get_latency(&pFilter->bq);
50026 | }
50027 |
50028 |
50029 | /*****
> *****/
50030 |
50031 | Low Shelf Filter
50032 |
50033 | *****/
> *****/
50034 | MA_API ma_loshelf2_config ma_loshelf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double gainDB, double shelfSlope, double frequency)
50035 | {
50036 |     ma_loshelf2_config config;

```

```

50037 |
50038 |     MA_ZERO_OBJECT(&config);
50039 |     config.format      = format;
50040 |     config.channels    = channels;
50041 |     config.sampleRate = sampleRate;
50042 |     config.gainDB      = gainDB;
50043 |     config.shelfSlope = shelfSlope;
50044 |     config.frequency  = frequency;
50045 |
50046 |     return config;
50047 | }
50048 |
50049 |
50050 | static MA_INLINE ma_biquad_config ma_loshelf2__get_biquad_config(const ma_loshelf2_config* pConfig)
50051 | {
50052 |     ma_biquad_config bqConfig;
50053 |     double w;
50054 |     double s;
50055 |     double c;
50056 |     double A;
50057 |     double S;
50058 |     double a;
50059 |     double sqrtA;
50060 |
50061 |     MA_ASSERT(pConfig != NULL);
50062 |
50063 |     w = 2 * MA_PI_D * pConfig->frequency / pConfig->sampleRate;
50064 |     s = ma_sind(w);
50065 |     c = ma_cosd(w);
50066 |     A = ma_powd(10, (pConfig->gainDB / 40));
50067 |     S = pConfig->shelfSlope;
50068 |     a = s/2 * ma_sqrttd((A + 1/A) * (1/S - 1) + 2);
50069 |     sqrtA = 2*ma_sqrttd(A)*a;
50070 |
50071 |     bqConfig.b0 = A * ((A + 1) - (A - 1)*c + sqrtA);
50072 |     bqConfig.b1 = 2 * A * ((A - 1) - (A + 1)*c);
50073 |     bqConfig.b2 = A * ((A + 1) - (A - 1)*c - sqrtA);
50074 |     bqConfig.a0 = (A + 1) + (A - 1)*c + sqrtA;
50075 |     bqConfig.a1 = -2 * ((A - 1) + (A + 1)*c);
50076 |     bqConfig.a2 = (A + 1) + (A - 1)*c - sqrtA;
50077 |
50078 |     bqConfig.format = pConfig->format;
50079 |     bqConfig.channels = pConfig->channels;
50080 |
50081 |     return bqConfig;
50082 | }
50083 |
50084 | MA_API ma_result ma_loshelf2_get_heap_size(const ma_loshelf2_config* pConfig, size_t*
> pHeapSizeInBytes)
50085 | {
50086 |     ma_biquad_config bqConfig;
50087 |     bqConfig = ma_loshelf2__get_biquad_config(pConfig);
50088 |
50089 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
50090 | }
50091 |
50092 | MA_API ma_result ma_loshelf2_init_preallocated(const ma_loshelf2_config* pConfig, void* pHeap,
> ma_loshelf2* pFilter)
50093 | {
50094 |     ma_result result;
50095 |     ma_biquad_config bqConfig;
50096 |
50097 |     if (pFilter == NULL) {
50098 |         return MA_INVALID_ARGS;
50099 |     }
50100 |
50101 |     MA_ZERO_OBJECT(pFilter);
50102 |
50103 |     if (pConfig == NULL) {
50104 |         return MA_INVALID_ARGS;
50105 |     }
50106 |
50107 |     bqConfig = ma_loshelf2__get_biquad_config(pConfig);
50108 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pFilter->bq);
50109 |     if (result != MA_SUCCESS) {
50110 |         return result;
50111 |     }
50112 |
50113 |     return MA_SUCCESS;

```

```

50114 | }
50115 |
50116 | MA_API ma_result ma_loshelf2_init(const ma_loshelf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_loshelf2* pFilter)
50117 | {
50118 |     ma_result result;
50119 |     size_t heapSizeInBytes;
50120 |     void* pHeap;
50121 |
50122 |     result = ma_loshelf2_get_heap_size(pConfig, &heapSizeInBytes);
50123 |     if (result != MA_SUCCESS) {
50124 |         return result;
50125 |     }
50126 |
50127 |     if (heapSizeInBytes > 0) {
50128 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
50129 |         if (pHeap == NULL) {
50130 |             return MA_OUT_OF_MEMORY;
50131 |         }
50132 |     } else {
50133 |         pHeap = NULL;
50134 |     }
50135 |
50136 |     result = ma_loshelf2_init_preallocated(pConfig, pHeap, pFilter);
50137 |     if (result != MA_SUCCESS) {
50138 |         ma_free(pHeap, pAllocationCallbacks);
50139 |         return result;
50140 |     }
50141 |
50142 |     pFilter->bq.ownsHeap = MA_TRUE; /* <-- This will cause the biquad to take ownership of the
> heap and free it when it's uninitialized. */
50143 |     return MA_SUCCESS;
50144 | }
50145 |
50146 | MA_API void ma_loshelf2_uninit(ma_loshelf2* pFilter, const ma_allocation_callbacks*
> pAllocationCallbacks)
50147 | {
50148 |     if (pFilter == NULL) {
50149 |         return;
50150 |     }
50151 |
50152 |     ma_biquad_uninit(&pFilter->bq, pAllocationCallbacks); /* <-- This will free the heap
> allocation. */
50153 | }
50154 |
50155 | MA_API ma_result ma_loshelf2_reinit(const ma_loshelf2_config* pConfig, ma_loshelf2* pFilter)
50156 | {
50157 |     ma_result result;
50158 |     ma_biquad_config bqConfig;
50159 |
50160 |     if (pFilter == NULL || pConfig == NULL) {
50161 |         return MA_INVALID_ARGS;
50162 |     }
50163 |
50164 |     bqConfig = ma_loshelf2__get_biquad_config(pConfig);
50165 |     result = ma_biquad_reinit(&bqConfig, &pFilter->bq);
50166 |     if (result != MA_SUCCESS) {
50167 |         return result;
50168 |     }
50169 |
50170 |     return MA_SUCCESS;
50171 | }
50172 |
50173 | static MA_INLINE void ma_loshelf2_process_pcm_frame_s16(ma_loshelf2* pFilter, ma_int16* pFrameOut,
> const ma_int16* pFrameIn)
50174 | {
50175 |     ma_biquad_process_pcm_frame_s16(&pFilter->bq, pFrameOut, pFrameIn);
50176 | }
50177 |
50178 | static MA_INLINE void ma_loshelf2_process_pcm_frame_f32(ma_loshelf2* pFilter, float* pFrameOut,
> const float* pFrameIn)
50179 | {
50180 |     ma_biquad_process_pcm_frame_f32(&pFilter->bq, pFrameOut, pFrameIn);
50181 | }
50182 |
50183 | MA_API ma_result ma_loshelf2_process_pcm_frames(ma_loshelf2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
50184 | {
50185 |     if (pFilter == NULL) {

```



```

50186 |         return MA_INVALID_ARGS;
50187 |     }
50188 |
50189 |     return ma_biquad_process_pcm_frames(&pFilter->bq, pFramesOut, pFramesIn, frameCount);
50190 | }
50191 |
50192 | MA_API ma_uint32 ma_loshelf2_get_latency(const ma_loshelf2* pFilter)
50193 | {
50194 |     if (pFilter == NULL) {
50195 |         return 0;
50196 |     }
50197 |
50198 |     return ma_biquad_get_latency(&pFilter->bq);
50199 | }
50200 |
50201 |
50202 | /*****
> *****/
50203 |
50204 | High Shelf Filter
50205 |
50206 | *****/
> *****/
50207 | MA_API ma_hishelf2_config ma_hishelf2_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double gainDB, double shelfSlope, double frequency)
50208 | {
50209 |     ma_hishelf2_config config;
50210 |
50211 |     MA_ZERO_OBJECT(&config);
50212 |     config.format      = format;
50213 |     config.channels    = channels;
50214 |     config.sampleRate  = sampleRate;
50215 |     config.gainDB      = gainDB;
50216 |     config.shelfSlope  = shelfSlope;
50217 |     config.frequency   = frequency;
50218 |
50219 |     return config;
50220 | }
50221 |
50222 |
50223 | static MA_INLINE ma_biquad_config ma_hishelf2__get_biquad_config(const ma_hishelf2_config* pConfig)
50224 | {
50225 |     ma_biquad_config bqConfig;
50226 |     double w;
50227 |     double s;
50228 |     double c;
50229 |     double A;
50230 |     double S;
50231 |     double a;
50232 |     double sqrtA;
50233 |
50234 |     MA_ASSERT(pConfig != NULL);
50235 |
50236 |     w = 2 * MA_PI_D * pConfig->frequency / pConfig->sampleRate;
50237 |     s = ma_sind(w);
50238 |     c = ma_cosd(w);
50239 |     A = ma_powd(10, (pConfig->gainDB / 40));
50240 |     S = pConfig->shelfSlope;
50241 |     a = s/2 * ma_sqrtd((A + 1/A) * (1/S - 1) + 2);
50242 |     sqrtA = 2*ma_sqrtd(A)*a;
50243 |
50244 |     bqConfig.b0 = A * ((A + 1) + (A - 1)*c + sqrtA);
50245 |     bqConfig.b1 = -2 * A * ((A - 1) + (A + 1)*c);
50246 |     bqConfig.b2 = A * ((A + 1) + (A - 1)*c - sqrtA);
50247 |     bqConfig.a0 = (A + 1) - (A - 1)*c + sqrtA;
50248 |     bqConfig.a1 = 2 * ((A - 1) - (A + 1)*c);
50249 |     bqConfig.a2 = (A + 1) - (A - 1)*c - sqrtA;
50250 |
50251 |     bqConfig.format = pConfig->format;
50252 |     bqConfig.channels = pConfig->channels;
50253 |
50254 |     return bqConfig;
50255 | }
50256 |
50257 | MA_API ma_result ma_hishelf2_get_heap_size(const ma_hishelf2_config* pConfig, size_t*
> pHeapSizeInBytes)
50258 | {
50259 |     ma_biquad_config bqConfig;
50260 |     bqConfig = ma_hishelf2__get_biquad_config(pConfig);

```

```

50261 |
50262 |     return ma_biquad_get_heap_size(&bqConfig, pHeapSizeInBytes);
50263 | }
50264 |
50265 | MA_API ma_result ma_hishelf2_init_preallocated(const ma_hishelf2_config* pConfig, void* pHeap,
> ma_hishelf2* pFilter)
50266 | {
50267 |     ma_result result;
50268 |     ma_biquad_config bqConfig;
50269 |
50270 |     if (pFilter == NULL) {
50271 |         return MA_INVALID_ARGS;
50272 |     }
50273 |
50274 |     MA_ZERO_OBJECT(pFilter);
50275 |
50276 |     if (pConfig == NULL) {
50277 |         return MA_INVALID_ARGS;
50278 |     }
50279 |
50280 |     bqConfig = ma_hishelf2_get_biquad_config(pConfig);
50281 |     result = ma_biquad_init_preallocated(&bqConfig, pHeap, &pFilter->bq);
50282 |     if (result != MA_SUCCESS) {
50283 |         return result;
50284 |     }
50285 |
50286 |     return MA_SUCCESS;
50287 | }
50288 |
50289 | MA_API ma_result ma_hishelf2_init(const ma_hishelf2_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_hishelf2* pFilter)
50290 | {
50291 |     ma_result result;
50292 |     size_t heapSizeInBytes;
50293 |     void* pHeap;
50294 |
50295 |     result = ma_hishelf2_get_heap_size(pConfig, &heapSizeInBytes);
50296 |     if (result != MA_SUCCESS) {
50297 |         return result;
50298 |     }
50299 |
50300 |     if (heapSizeInBytes > 0) {
50301 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
50302 |         if (pHeap == NULL) {
50303 |             return MA_OUT_OF_MEMORY;
50304 |         }
50305 |     } else {
50306 |         pHeap = NULL;
50307 |     }
50308 |
50309 |     result = ma_hishelf2_init_preallocated(pConfig, pHeap, pFilter);
50310 |     if (result != MA_SUCCESS) {
50311 |         ma_free(pHeap, pAllocationCallbacks);
50312 |         return result;
50313 |     }
50314 |
50315 |     pFilter->bq.ownsHeap = MA_TRUE; /* <-- This will cause the biquad to take ownership of the
> heap and free it when it's uninitialized. */
50316 |     return MA_SUCCESS;
50317 | }
50318 |
50319 | MA_API void ma_hishelf2_uninit(ma_hishelf2* pFilter, const ma_allocation_callbacks*
> pAllocationCallbacks)
50320 | {
50321 |     if (pFilter == NULL) {
50322 |         return;
50323 |     }
50324 |
50325 |     ma_biquad_uninit(&pFilter->bq, pAllocationCallbacks); /* <-- This will free the heap
> allocation. */
50326 | }
50327 |
50328 | MA_API ma_result ma_hishelf2_reinit(const ma_hishelf2_config* pConfig, ma_hishelf2* pFilter)
50329 | {
50330 |     ma_result result;
50331 |     ma_biquad_config bqConfig;
50332 |
50333 |     if (pFilter == NULL || pConfig == NULL) {
50334 |         return MA_INVALID_ARGS;

```

```

50335 |     }
50336 |
50337 |     bqConfig = ma_hishef2_get_biquad_config(pConfig);
50338 |     result = ma_biquad_reinit(&bqConfig, &pFilter->bq);
50339 |     if (result != MA_SUCCESS) {
50340 |         return result;
50341 |     }
50342 |
50343 |     return MA_SUCCESS;
50344 | }
50345 |
50346 | static MA_INLINE void ma_hishef2_process_pcm_frame_s16(ma_hishef2* pFilter, ma_int16* pFrameOut,
> const ma_int16* pFrameIn)
50347 | {
50348 |     ma_biquad_process_pcm_frame_s16(&pFilter->bq, pFrameOut, pFrameIn);
50349 | }
50350 |
50351 | static MA_INLINE void ma_hishef2_process_pcm_frame_f32(ma_hishef2* pFilter, float* pFrameOut,
> const float* pFrameIn)
50352 | {
50353 |     ma_biquad_process_pcm_frame_f32(&pFilter->bq, pFrameOut, pFrameIn);
50354 | }
50355 |
50356 | MA_API ma_result ma_hishef2_process_pcm_frames(ma_hishef2* pFilter, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
50357 | {
50358 |     if (pFilter == NULL) {
50359 |         return MA_INVALID_ARGS;
50360 |     }
50361 |
50362 |     return ma_biquad_process_pcm_frames(&pFilter->bq, pFramesOut, pFramesIn, frameCount);
50363 | }
50364 |
50365 | MA_API ma_uint32 ma_hishef2_get_latency(const ma_hishef2* pFilter)
50366 | {
50367 |     if (pFilter == NULL) {
50368 |         return 0;
50369 |     }
50370 |
50371 |     return ma_biquad_get_latency(&pFilter->bq);
50372 | }
50373 |
50374 |
50375 |
50376 | /*
50377 | Delay
50378 | */
50379 | MA_API ma_delay_config ma_delay_config_init(ma_uint32 channels, ma_uint32 sampleRate, ma_uint32
> delayInFrames, float decay)
50380 | {
50381 |     ma_delay_config config;
50382 |
50383 |     MA_ZERO_OBJECT(&config);
50384 |     config.channels      = channels;
50385 |     config.sampleRate    = sampleRate;
50386 |     config.delayInFrames = delayInFrames;
50387 |     config.delayStart    = (decay == 0) ? MA_TRUE : MA_FALSE; /* Delay the start if it looks like
> we're not configuring an echo. */
50388 |     config.wet           = 1;
50389 |     config.dry           = 1;
50390 |     config.decay         = decay;
50391 |
50392 |     return config;
50393 | }
50394 |
50395 |
50396 | MA_API ma_result ma_delay_init(const ma_delay_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_delay* pDelay)
50397 | {
50398 |     if (pDelay == NULL) {
50399 |         return MA_INVALID_ARGS;
50400 |     }
50401 |
50402 |     MA_ZERO_OBJECT(pDelay);
50403 |
50404 |     if (pConfig == NULL) {
50405 |         return MA_INVALID_ARGS;
50406 |     }
50407 |

```

```

50408 |     if (pConfig->decay < 0 || pConfig->decay > 1) {
50409 |         return MA_INVALID_ARGS;
50410 |     }
50411 |
50412 |     pDelay->config          = *pConfig;
50413 |     pDelay->bufferSizeInFrames = pConfig->delayInFrames;
50414 |     pDelay->cursor          = 0;
50415 |
50416 |     pDelay->pBuffer = (float*)ma_malloc((size_t)(pDelay->bufferSizeInFrames *
> ma_get_bytes_per_frame(ma_format_f32, pConfig->channels)), pAllocationCallbacks);
50417 |     if (pDelay->pBuffer == NULL) {
50418 |         return MA_OUT_OF_MEMORY;
50419 |     }
50420 |
50421 |     ma_silence_pcm_frames(pDelay->pBuffer, pDelay->bufferSizeInFrames, ma_format_f32,
> pConfig->channels);
50422 |
50423 |     return MA_SUCCESS;
50424 | }
50425 |
50426 | MA_API void ma_delay_uninit(ma_delay* pDelay, const ma_allocation_callbacks* pAllocationCallbacks)
50427 | {
50428 |     if (pDelay == NULL) {
50429 |         return;
50430 |     }
50431 |
50432 |     ma_free(pDelay->pBuffer, pAllocationCallbacks);
50433 | }
50434 |
50435 | MA_API ma_result ma_delay_process_pcm_frames(ma_delay* pDelay, void* pFramesOut, const void*
> pFramesIn, ma_uint32 frameCount)
50436 | {
50437 |     ma_uint32 iFrame;
50438 |     ma_uint32 iChannel;
50439 |     float* pFramesOutF32 = (float*)pFramesOut;
50440 |     const float* pFramesInF32 = (const float*)pFramesIn;
50441 |
50442 |     if (pDelay == NULL || pFramesOut == NULL || pFramesIn == NULL) {
50443 |         return MA_INVALID_ARGS;
50444 |     }
50445 |
50446 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
50447 |         for (iChannel = 0; iChannel < pDelay->config.channels; iChannel += 1) {
50448 |             ma_uint32 iBuffer = (pDelay->cursor * pDelay->config.channels) + iChannel;
50449 |
50450 |             if (pDelay->config.delayStart) {
50451 |                 /* Delayed start. */
50452 |
50453 |                 /* Read */
50454 |                 pFramesOutF32[iChannel] = pDelay->pBuffer[iBuffer] * pDelay->config.wet;
50455 |
50456 |                 /* Feedback */
50457 |                 pDelay->pBuffer[iBuffer] = (pDelay->pBuffer[iBuffer] * pDelay->config.decay) +
> (pFramesInF32[iChannel] * pDelay->config.dry);
50458 |             } else {
50459 |                 /* Immediate start */
50460 |
50461 |                 /* Feedback */
50462 |                 pDelay->pBuffer[iBuffer] = (pDelay->pBuffer[iBuffer] * pDelay->config.decay) +
> (pFramesInF32[iChannel] * pDelay->config.dry);
50463 |             }
50464 |
50465 |             /* Read */
50466 |             pFramesOutF32[iChannel] = pDelay->pBuffer[iBuffer] * pDelay->config.wet;
50467 |         }
50468 |
50469 |         pDelay->cursor = (pDelay->cursor + 1) % pDelay->bufferSizeInFrames;
50470 |
50471 |         pFramesOutF32 += pDelay->config.channels;
50472 |         pFramesInF32  += pDelay->config.channels;
50473 |     }
50474 |
50475 |     return MA_SUCCESS;
50476 | }
50477 |
50478 | MA_API void ma_delay_set_wet(ma_delay* pDelay, float value)
50479 | {
50480 |     if (pDelay == NULL) {
50481 |         return;

```

```
50482 |     }
50483 |
50484 |     pDelay->config.wet = value;
50485 | }
50486 |
50487 | MA_API float ma_delay_get_wet(const ma_delay* pDelay)
50488 | {
50489 |     if (pDelay == NULL) {
50490 |         return 0;
50491 |     }
50492 |
50493 |     return pDelay->config.wet;
50494 | }
50495 |
50496 | MA_API void ma_delay_set_dry(ma_delay* pDelay, float value)
50497 | {
50498 |     if (pDelay == NULL) {
50499 |         return;
50500 |     }
50501 |
50502 |     pDelay->config.dry = value;
50503 | }
50504 |
50505 | MA_API float ma_delay_get_dry(const ma_delay* pDelay)
50506 | {
50507 |     if (pDelay == NULL) {
50508 |         return 0;
50509 |     }
50510 |
50511 |     return pDelay->config.dry;
50512 | }
50513 |
50514 | MA_API void ma_delay_set_decay(ma_delay* pDelay, float value)
50515 | {
50516 |     if (pDelay == NULL) {
50517 |         return;
50518 |     }
50519 |
50520 |     pDelay->config.decay = value;
50521 | }
50522 |
50523 | MA_API float ma_delay_get_decay(const ma_delay* pDelay)
50524 | {
50525 |     if (pDelay == NULL) {
50526 |         return 0;
50527 |     }
50528 |
50529 |     return pDelay->config.decay;
50530 | }
50531 |
50532 |
50533 | MA_API ma_gainer_config ma_gainer_config_init(ma_uint32 channels, ma_uint32 smoothTimeInFrames)
50534 | {
50535 |     ma_gainer_config config;
50536 |
50537 |     MA_ZERO_OBJECT(&config);
50538 |     config.channels = channels;
50539 |     config.smoothTimeInFrames = smoothTimeInFrames;
50540 |
50541 |     return config;
50542 | }
50543 |
50544 |
50545 | typedef struct
50546 | {
50547 |     size_t sizeInBytes;
50548 |     size_t oldGainsOffset;
50549 |     size_t newGainsOffset;
50550 | } ma_gainer_heap_layout;
50551 |
50552 | static ma_result ma_gainer_get_heap_layout(const ma_gainer_config* pConfig, ma_gainer_heap_layout*
> pHeapLayout)
50553 | {
50554 |     MA_ASSERT(pHeapLayout != NULL);
50555 |
50556 |     MA_ZERO_OBJECT(pHeapLayout);
50557 |
50558 |     if (pConfig == NULL) {
50559 |         return MA_INVALID_ARGS;
```

```

50560 |     }
50561 |
50562 |     if (pConfig->channels == 0) {
50563 |         return MA_INVALID_ARGS;
50564 |     }
50565 |
50566 |     pHeapLayout->sizeInBytes = 0;
50567 |
50568 |     /* Old gains. */
50569 |     pHeapLayout->oldGainsOffset = pHeapLayout->sizeInBytes;
50570 |     pHeapLayout->sizeInBytes += sizeof(float) * pConfig->channels;
50571 |
50572 |     /* New gains. */
50573 |     pHeapLayout->newGainsOffset = pHeapLayout->sizeInBytes;
50574 |     pHeapLayout->sizeInBytes += sizeof(float) * pConfig->channels;
50575 |
50576 |     /* Alignment. */
50577 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
50578 |
50579 |     return MA_SUCCESS;
50580 | }
50581 |
50582 |
50583 | MA_API ma_result ma_gainer_get_heap_size(const ma_gainer_config* pConfig, size_t* pHeapSizeInBytes)
50584 | {
50585 |     ma_result result;
50586 |     ma_gainer_heap_layout heapLayout;
50587 |
50588 |     if (pHeapSizeInBytes == NULL) {
50589 |         return MA_INVALID_ARGS;
50590 |     }
50591 |
50592 |     *pHeapSizeInBytes = 0;
50593 |
50594 |     result = ma_gainer_get_heap_layout(pConfig, &heapLayout);
50595 |     if (result != MA_SUCCESS) {
50596 |         return MA_INVALID_ARGS;
50597 |     }
50598 |
50599 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
50600 |
50601 |     return MA_SUCCESS;
50602 | }
50603 |
50604 |
50605 | MA_API ma_result ma_gainer_init_preallocated(const ma_gainer_config* pConfig, void* pHeap,
> ma_gainer* pGainer)
50606 | {
50607 |     ma_result result;
50608 |     ma_gainer_heap_layout heapLayout;
50609 |     ma_uint32 iChannel;
50610 |
50611 |     if (pGainer == NULL) {
50612 |         return MA_INVALID_ARGS;
50613 |     }
50614 |
50615 |     MA_ZERO_OBJECT(pGainer);
50616 |
50617 |     if (pConfig == NULL || pHeap == NULL) {
50618 |         return MA_INVALID_ARGS;
50619 |     }
50620 |
50621 |     result = ma_gainer_get_heap_layout(pConfig, &heapLayout);
50622 |     if (result != MA_SUCCESS) {
50623 |         return result;
50624 |     }
50625 |
50626 |     pGainer->_pHeap = pHeap;
50627 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
50628 |
50629 |     pGainer->pOldGains = (float*)ma_offset_ptr(pHeap, heapLayout.oldGainsOffset);
50630 |     pGainer->pNewGains = (float*)ma_offset_ptr(pHeap, heapLayout.newGainsOffset);
50631 |     pGainer->masterVolume = 1;
50632 |
50633 |     pGainer->config = *pConfig;
50634 |     pGainer->t       = (ma_uint32)-1; /* No interpolation by default. */
50635 |
50636 |     for (iChannel = 0; iChannel < pConfig->channels; iChannel += 1) {
50637 |         pGainer->pOldGains[iChannel] = 1;

```

```

50638 |         pGainer->pNewGains[iChannel] = 1;
50639 |     }
50640 |
50641 |     return MA_SUCCESS;
50642 | }
50643 |
50644 | MA_API ma_result ma_gainer_init(const ma_gainer_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_gainer* pGainer)
50645 | {
50646 |     ma_result result;
50647 |     size_t heapSizeInBytes;
50648 |     void* pHeap;
50649 |
50650 |     result = ma_gainer_get_heap_size(pConfig, &heapSizeInBytes);
50651 |     if (result != MA_SUCCESS) {
50652 |         return result; /* Failed to retrieve the size of the heap allocation. */
50653 |     }
50654 |
50655 |     if (heapSizeInBytes > 0) {
50656 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
50657 |         if (pHeap == NULL) {
50658 |             return MA_OUT_OF_MEMORY;
50659 |         }
50660 |     } else {
50661 |         pHeap = NULL;
50662 |     }
50663 |
50664 |     result = ma_gainer_init_preallocated(pConfig, pHeap, pGainer);
50665 |     if (result != MA_SUCCESS) {
50666 |         ma_free(pHeap, pAllocationCallbacks);
50667 |         return result;
50668 |     }
50669 |
50670 |     pGainer->_ownsHeap = MA_TRUE;
50671 |     return MA_SUCCESS;
50672 | }
50673 |
50674 | MA_API void ma_gainer_uninit(ma_gainer* pGainer, const ma_allocation_callbacks*
> pAllocationCallbacks)
50675 | {
50676 |     if (pGainer == NULL) {
50677 |         return;
50678 |     }
50679 |
50680 |     if (pGainer->_ownsHeap) {
50681 |         ma_free(pGainer->_pHeap, pAllocationCallbacks);
50682 |     }
50683 | }
50684 |
50685 | static float ma_gainer_calculate_current_gain(const ma_gainer* pGainer, ma_uint32 channel)
50686 | {
50687 |     float a = (float)pGainer->t / pGainer->config.smoothTimeInFrames;
50688 |     return ma_mix_f32_fast(pGainer->pOldGains[channel], pGainer->pNewGains[channel], a);
50689 | }
50690 |
50691 | static /*__attribute__((noinline))*/ ma_result ma_gainer_process_pcm_frames_internal(ma_gainer *
> pGainer, void* MA_RESTRICT pFramesOut, const void* MA_RESTRICT pFramesIn, ma_uint64 frameCount)
50692 | {
50693 |     ma_uint64 iFrame;
50694 |     ma_uint32 iChannel;
50695 |     ma_uint64 interpolatedFrameCount;
50696 |
50697 |     MA_ASSERT(pGainer != NULL);
50698 |
50699 |     /*
50700 |     We don't necessarily need to apply a linear interpolation for the entire frameCount frames. When
50701 |     linear interpolation is not needed we can do a simple volume adjustment which will be more
50702 |     efficient than a lerp with an alpha value of 1.
50703 |
50704 |     To do this, all we need to do is determine how many frames need to have a lerp applied. Then we
50705 |     just process that number of frames with linear interpolation. After that we run on an optimized
50706 |     path which just applies the new gains without a lerp.
50707 |     */
50708 |     if (pGainer->t >= pGainer->config.smoothTimeInFrames) {
50709 |         interpolatedFrameCount = 0;
50710 |     } else {
50711 |         interpolatedFrameCount = pGainer->t - pGainer->config.smoothTimeInFrames;
50712 |         if (interpolatedFrameCount > frameCount) {
50713 |             interpolatedFrameCount = frameCount;

```



```

50714 |     }
50715 | }
50716 |
50717 | /*
50718 | Start off with our interpolated frames. When we do this, we'll adjust frameCount and our
> pointers
50719 | so that the fast path can work naturally without consideration of the interpolated path.
50720 | */
50721 | if (interpolatedFrameCount > 0) {
50722 |     /* We can allow the input and output buffers to be null in which case we'll just update the
> internal timer. */
50723 |     if (pFramesOut != NULL && pFramesIn != NULL) {
50724 |         /*
50725 |         All we're really doing here is moving the old gains towards the new gains. We don't want
> to
50726 |         be modifying the gains inside the ma_gainer object because that will break things.
> Instead
50727 |         we can make a copy here on the stack. For extreme channel counts we can fall back to a
> slower
50728 |         implementation which just uses a standard lerp.
50729 |         */
50730 |         float* pFramesOutF32 = (float*)pFramesOut;
50731 |         const float* pFramesInF32 = (const float*)pFramesIn;
50732 |         float a = (float)pGainer->t / pGainer->config.smoothTimeInFrames;
50733 |         float d = 1.0f / pGainer->config.smoothTimeInFrames;
50734 |
50735 |         if (pGainer->config.channels <= 32) {
50736 |             float pRunningGain[32];
50737 |             float pRunningGainDelta[32]; /* Could this be heap-allocated as part of the
> ma_gainer object? */
50738 |
50739 |             /* Initialize the running gain. */
50740 |             for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
50741 |                 float t = (pGainer->pNewGains[iChannel] - pGainer->pOldGains[iChannel]) *
> pGainer->masterVolume;
50742 |                 pRunningGainDelta[iChannel] = t * d;
50743 |                 pRunningGain[iChannel] = (pGainer->pOldGains[iChannel] * pGainer->masterVolume)
> + (t * a);
50744 |             }
50745 |
50746 |             iFrame = 0;
50747 |
50748 |             /* Optimized paths for common channel counts. This is mostly just experimenting with
> some SIMD ideas. It's not necessarily final. */
50749 |             if (pGainer->config.channels == 2) {
50750 |                 #if defined(MA_SUPPORT_SSE2)
50751 |                     if (ma_has_sse2()) {
50752 |                         ma_uint64 unrolledLoopCount = interpolatedFrameCount >> 1;
50753 |
50754 |                         /* Expand some arrays so we can have a clean SIMD loop below. */
50755 |                         __m128 runningGainDelta0 = _mm_set_ps(pRunningGainDelta[1],
> pRunningGainDelta[0], pRunningGainDelta[1], pRunningGainDelta[0]);
50756 |                         __m128 runningGain0 = _mm_set_ps(pRunningGain[1] +
> pRunningGainDelta[1], pRunningGain[0] + pRunningGainDelta[0], pRunningGain[1], pRunningGain[0]);
50757 |
50758 |                         for (; iFrame < unrolledLoopCount; iFrame += 1) {
50759 |                             _mm_storeu_ps(&pFramesOutF32[iFrame*4 + 0],
> _mm_mul_ps(_mm_loadu_ps(&pFramesInF32[iFrame*4 + 0]), runningGain0));
50760 |                             runningGain0 = _mm_add_ps(runningGain0, runningGainDelta0);
50761 |                         }
50762 |
50763 |                         iFrame = unrolledLoopCount << 1;
50764 |                     } else
50765 |                 #endif
50766 |                 {
50767 |                     /*
50768 |                     Two different scalar implementations here. Clang (and I assume GCC) will
> vectorize
50769 |                     both of these, but the bottom version results in a nicer vectorization with
> less
50770 |                     instructions emitted. The problem, however, is that the bottom version runs
> slower
50771 |                     when compiled with MSVC. The top version will be partially vectorized by
> MSVC.
50772 |                     */
50773 |                     #if defined(_MSC_VER) && !defined(__clang__)
50774 |                         ma_uint64 unrolledLoopCount = interpolatedFrameCount >> 1;
50775 |
50776 |                         /* Expand some arrays so we can have a clean 4x SIMD operation in the loop.

```

```

> */
50777 |                 pRunningGainDelta[2] = pRunningGainDelta[0];
50778 |                 pRunningGainDelta[3] = pRunningGainDelta[1];
50779 |                 pRunningGain[2] = pRunningGain[0] + pRunningGainDelta[0];
50780 |                 pRunningGain[3] = pRunningGain[1] + pRunningGainDelta[1];
50781 |
50782 |                 for (; iFrame < unrolledLoopCount; iFrame += 1) {
50783 |                     pFramesOutF32[iFrame*4 + 0] = pFramesInF32[iFrame*4 + 0] *
> pRunningGain[0];
50784 |                     pFramesOutF32[iFrame*4 + 1] = pFramesInF32[iFrame*4 + 1] *
> pRunningGain[1];
50785 |                     pFramesOutF32[iFrame*4 + 2] = pFramesInF32[iFrame*4 + 2] *
> pRunningGain[2];
50786 |                     pFramesOutF32[iFrame*4 + 3] = pFramesInF32[iFrame*4 + 3] *
> pRunningGain[3];
50787 |
50788 |                     /* Move the running gain forward towards the new gain. */
50789 |                     pRunningGain[0] += pRunningGainDelta[0];
50790 |                     pRunningGain[1] += pRunningGainDelta[1];
50791 |                     pRunningGain[2] += pRunningGainDelta[2];
50792 |                     pRunningGain[3] += pRunningGainDelta[3];
50793 |                 }
50794 |
50795 |                 iFrame = unrolledLoopCount << 1;
50796 |             #else
50797 |                 for (; iFrame < interpolatedFrameCount; iFrame += 1) {
50798 |                     for (iChannel = 0; iChannel < 2; iChannel += 1) {
50799 |                         pFramesOutF32[iFrame*2 + iChannel] = pFramesInF32[iFrame*2 +
> iChannel] * pRunningGain[iChannel];
50800 |                     }
50801 |
50802 |                     for (iChannel = 0; iChannel < 2; iChannel += 1) {
50803 |                         pRunningGain[iChannel] += pRunningGainDelta[iChannel];
50804 |                     }
50805 |                 }
50806 |             #endif
50807 |         }
50808 |     } else if (pGainer->config.channels == 6) {
50809 |         #if defined(MA_SUPPORT_SSE2)
50810 |             if (ma_has_sse2()) {
50811 |                 /*
50812 |                 For 6 channels things are a bit more complicated because 6 isn't cleanly
> divisible by 4. We need to do 2 frames
50813 |                 at a time, meaning we'll be doing 12 samples in a group. Like the stereo
> case we'll need to expand some arrays
50814 |                 so we can do clean 4x SIMD operations.
50815 |                 */
50816 |                 ma_uint64 unrolledLoopCount = interpolatedFrameCount >> 1;
50817 |
50818 |                 /* Expand some arrays so we can have a clean SIMD loop below. */
50819 |                 __m128 runningGainDelta0 = _mm_set_ps(pRunningGainDelta[3],
> pRunningGainDelta[2], pRunningGainDelta[1], pRunningGainDelta[0]);
50820 |                 __m128 runningGainDelta1 = _mm_set_ps(pRunningGainDelta[1],
> pRunningGainDelta[0], pRunningGainDelta[5], pRunningGainDelta[4]);
50821 |                 __m128 runningGainDelta2 = _mm_set_ps(pRunningGainDelta[5],
> pRunningGainDelta[4], pRunningGainDelta[3], pRunningGainDelta[2]);
50822 |
50823 |                 __m128 runningGain0 = _mm_set_ps(pRunningGain[3],
> pRunningGain[2],
> pRunningGain[0]);
50824 |                 __m128 runningGain1 = _mm_set_ps(pRunningGain[1] +
> pRunningGainDelta[1], pRunningGain[0] + pRunningGainDelta[0], pRunningGain[5],
> pRunningGain[4]);
50825 |                 __m128 runningGain2 = _mm_set_ps(pRunningGain[5] +
> pRunningGainDelta[5], pRunningGain[4] + pRunningGainDelta[4], pRunningGain[3] +
> pRunningGainDelta[3], pRunningGain[2] + pRunningGainDelta[2]);
50826 |
50827 |                 for (; iFrame < unrolledLoopCount; iFrame += 1) {
50828 |                     __mm_storeu_ps(&pFramesOutF32[iFrame*12 + 0],
> __mm_mul_ps(__mm_loadu_ps(&pFramesInF32[iFrame*12 + 0]), runningGain0));
50829 |                     __mm_storeu_ps(&pFramesOutF32[iFrame*12 + 4],
> __mm_mul_ps(__mm_loadu_ps(&pFramesInF32[iFrame*12 + 4]), runningGain1));
50830 |                     __mm_storeu_ps(&pFramesOutF32[iFrame*12 + 8],
> __mm_mul_ps(__mm_loadu_ps(&pFramesInF32[iFrame*12 + 8]), runningGain2));
50831 |
50832 |                     runningGain0 = _mm_add_ps(runningGain0, runningGainDelta0);
50833 |                     runningGain1 = _mm_add_ps(runningGain1, runningGainDelta1);
50834 |                     runningGain2 = _mm_add_ps(runningGain2, runningGainDelta2);
50835 |                 }

```

```

50836 |
50837 |         iFrame = unrolledLoopCount << 1;
50838 |     } else
50839 |     #endif
50840 |     {
50841 |         for (; iFrame < interpolatedFrameCount; iFrame += 1) {
50842 |             for (iChannel = 0; iChannel < 6; iChannel += 1) {
50843 |                 pFramesOutF32[iFrame*6 + iChannel] = pFramesInF32[iFrame*6 +
> iChannel] * pRunningGain[iChannel];
50844 |             }
50845 |
50846 |             /* Move the running gain forward towards the new gain. */
50847 |             for (iChannel = 0; iChannel < 6; iChannel += 1) {
50848 |                 pRunningGain[iChannel] += pRunningGainDelta[iChannel];
50849 |             }
50850 |         }
50851 |     }
50852 | } else if (pGainer->config.channels == 8) {
50853 |     /* For 8 channels we can just go over frame by frame and do all eight channels
> as 2 separate 4x SIMD operations. */
50854 |     #if defined(MA_SUPPORT_SSE2)
50855 |         if (ma_has_sse2()) {
50856 |             __m128 runningGainDelta0 = _mm_loadu_ps(&pRunningGainDelta[0]);
50857 |             __m128 runningGainDelta1 = _mm_loadu_ps(&pRunningGainDelta[4]);
50858 |             __m128 runningGain0 = _mm_loadu_ps(&pRunningGain[0]);
50859 |             __m128 runningGain1 = _mm_loadu_ps(&pRunningGain[4]);
50860 |
50861 |             for (; iFrame < interpolatedFrameCount; iFrame += 1) {
50862 |                 _mm_storeu_ps(&pFramesOutF32[iFrame*8 + 0],
> _mm_mul_ps(_mm_loadu_ps(&pFramesInF32[iFrame*8 + 0]), runningGain0));
50863 |                 _mm_storeu_ps(&pFramesOutF32[iFrame*8 + 4],
> _mm_mul_ps(_mm_loadu_ps(&pFramesInF32[iFrame*8 + 4]), runningGain1));
50864 |
50865 |                 runningGain0 = _mm_add_ps(runningGain0, runningGainDelta0);
50866 |                 runningGain1 = _mm_add_ps(runningGain1, runningGainDelta1);
50867 |             }
50868 |         } else
50869 |         #endif
50870 |         {
50871 |             /* This is crafted so that it auto-vectorizes when compiled with Clang. */
50872 |             for (; iFrame < interpolatedFrameCount; iFrame += 1) {
50873 |                 for (iChannel = 0; iChannel < 8; iChannel += 1) {
50874 |                     pFramesOutF32[iFrame*8 + iChannel] = pFramesInF32[iFrame*8 +
> iChannel] * pRunningGain[iChannel];
50875 |                 }
50876 |
50877 |                 /* Move the running gain forward towards the new gain. */
50878 |                 for (iChannel = 0; iChannel < 8; iChannel += 1) {
50879 |                     pRunningGain[iChannel] += pRunningGainDelta[iChannel];
50880 |                 }
50881 |             }
50882 |         }
50883 |     }
50884 |
50885 |     for (; iFrame < interpolatedFrameCount; iFrame += 1) {
50886 |         for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
50887 |             pFramesOutF32[iFrame*pGainer->config.channels + iChannel] =
> pFramesInF32[iFrame*pGainer->config.channels + iChannel] * pRunningGain[iChannel];
50888 |             pRunningGain[iChannel] += pRunningGainDelta[iChannel];
50889 |         }
50890 |     }
50891 | } else {
50892 |     /* Slower path for extreme channel counts where we can't fit enough on the stack. We
> could also move this to the heap as part of the ma_gainer object which might even be better since
> it'll only be updated when the gains actually change. */
50893 |     for (iFrame = 0; iFrame < interpolatedFrameCount; iFrame += 1) {
50894 |         for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
50895 |             pFramesOutF32[iFrame*pGainer->config.channels + iChannel] =
> pFramesInF32[iFrame*pGainer->config.channels + iChannel] *
> ma_mix_f32_fast(pGainer->pOldGains[iChannel], pGainer->pNewGains[iChannel], a) *
> pGainer->masterVolume;
50896 |         }
50897 |
50898 |         a += d;
50899 |     }
50900 | }
50901 |
50902 | pFramesOut = ma_offset_ptr(pFramesOut, interpolatedFrameCount * sizeof(float));
50903 | pFramesIn = ma_offset_ptr(pFramesIn, interpolatedFrameCount * sizeof(float));

```

```

50904 |     }
50905 |
50906 |     frameCount -= interpolatedFrameCount;
50907 |
50908 |     /* Make sure the timer is updated. */
50909 |     pGainer->t = (ma_uint32)ma_min(pGainer->t + interpolatedFrameCount,
> pGainer->config.smoothTimeInFrames);
50910 | }
50911 |
50912 | /* All we need to do here is apply the new gains using an optimized path. */
50913 | if (pFramesOut != NULL && pFramesIn != NULL) {
50914 |     if (pGainer->config.channels <= 32) {
50915 |         float gains[32];
50916 |         for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
50917 |             gains[iChannel] = pGainer->pNewGains[iChannel] * pGainer->masterVolume;
50918 |         }
50919 |
50920 |         ma_copy_and_apply_volume_factor_per_channel_f32((float*)pFramesOut, (const
> float*)pFramesIn, frameCount, pGainer->config.channels, gains);
50921 |     } else {
50922 |         /* Slow path. Too many channels to fit on the stack. Need to apply a master volume as a
> separate path. */
50923 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
50924 |             for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
50925 |                 ((float*)pFramesOut)[iFrame*pGainer->config.channels + iChannel] = ((const
> float*)pFramesIn)[iFrame*pGainer->config.channels + iChannel] * pGainer->pNewGains[iChannel] *
> pGainer->masterVolume;
50926 |             }
50927 |         }
50928 |     }
50929 | }
50930 |
50931 | /* Now that some frames have been processed we need to make sure future changes to the gain are
> interpolated. */
50932 | if (pGainer->t == (ma_uint32)-1) {
50933 |     pGainer->t = (ma_uint32)ma_min(pGainer->config.smoothTimeInFrames, frameCount);
50934 | }
50935 |
50936 | #if 0
50937 |     if (pGainer->t >= pGainer->config.smoothTimeInFrames) {
50938 |         /* Fast path. No gain calculation required. */
50939 |         ma_copy_and_apply_volume_factor_per_channel_f32(pFramesOutF32, pFramesInF32, frameCount,
> pGainer->config.channels, pGainer->pNewGains);
50940 |         ma_apply_volume_factor_f32(pFramesOutF32, frameCount * pGainer->config.channels,
> pGainer->masterVolume);
50941 |     }
50942 |     /* Now that some frames have been processed we need to make sure future changes to the gain
> are interpolated. */
50943 |     if (pGainer->t == (ma_uint32)-1) {
50944 |         pGainer->t = pGainer->config.smoothTimeInFrames;
50945 |     }
50946 | } else {
50947 |     /* Slow path. Need to interpolate the gain for each channel individually. */
50948 |
50949 |     /* We can allow the input and output buffers to be null in which case we'll just update the
> internal timer. */
50950 |     if (pFramesOut != NULL && pFramesIn != NULL) {
50951 |         float a = (float)pGainer->t / pGainer->config.smoothTimeInFrames;
50952 |         float d = 1.0f / pGainer->config.smoothTimeInFrames;
50953 |         ma_uint32 channelCount = pGainer->config.channels;
50954 |
50955 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
50956 |             for (iChannel = 0; iChannel < channelCount; iChannel += 1) {
50957 |                 pFramesOutF32[iChannel] = pFramesInF32[iChannel] *
> ma_mix_f32_fast(pGainer->pOldGains[iChannel], pGainer->pNewGains[iChannel], a) *
> pGainer->masterVolume;
50958 |             }
50959 |
50960 |             pFramesOutF32 += channelCount;
50961 |             pFramesInF32 += channelCount;
50962 |
50963 |             a += d;
50964 |             if (a > 1) {
50965 |                 a = 1;
50966 |             }
50967 |         }
50968 |     }
50969 |
50970 |     pGainer->t = (ma_uint32)ma_min(pGainer->t + frameCount, pGainer->config.smoothTimeInFrames);

```

```

50971 |
50972 |     #if 0    /* Reference implementation. */
50973 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
50974 |             /* We can allow the input and output buffers to be null in which case we'll just update
> the internal timer. */
50975 |             if (pFramesOut != NULL && pFramesIn != NULL) {
50976 |                 for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
50977 |                     pFramesOutF32[iFrame * pGainer->config.channels + iChannel] =
> pFramesInF32[iFrame * pGainer->config.channels + iChannel] *
> ma_gainer_calculate_current_gain(pGainer, iChannel) * pGainer->masterVolume;
50978 |                 }
50979 |             }
50980 |
50981 |             /* Move interpolation time forward, but don't go beyond our smoothing time. */
50982 |             pGainer->t = ma_min(pGainer->t + 1, pGainer->config.smoothTimeInFrames);
50983 |         }
50984 |     #endif
50985 | }
50986 | #endif
50987 |
50988 |     return MA_SUCCESS;
50989 | }
50990 |
50991 | MA_API ma_result ma_gainer_process_pcm_frames(ma_gainer* pGainer, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
50992 | {
50993 |     if (pGainer == NULL) {
50994 |         return MA_INVALID_ARGS;
50995 |     }
50996 |
50997 |     /*
50998 |     ma_gainer_process_pcm_frames_internal() marks pFramesOut and pFramesIn with MA_RESTRICT which
50999 |     helps with auto-vectorization.
51000 |     */
51001 |     return ma_gainer_process_pcm_frames_internal(pGainer, pFramesOut, pFramesIn, frameCount);
51002 | }
51003 |
51004 | static void ma_gainer_set_gain_by_index(ma_gainer* pGainer, float newGain, ma_uint32 iChannel)
51005 | {
51006 |     pGainer->pOldGains[iChannel] = ma_gainer_calculate_current_gain(pGainer, iChannel);
51007 |     pGainer->pNewGains[iChannel] = newGain;
51008 | }
51009 |
51010 | static void ma_gainer_reset_smoothing_time(ma_gainer* pGainer)
51011 | {
51012 |     if (pGainer->t == (ma_uint32)-1) {
51013 |         pGainer->t = pGainer->config.smoothTimeInFrames;    /* No smoothing required for initial
> gains setting. */
51014 |     } else {
51015 |         pGainer->t = 0;
51016 |     }
51017 | }
51018 |
51019 | MA_API ma_result ma_gainer_set_gain(ma_gainer* pGainer, float newGain)
51020 | {
51021 |     ma_uint32 iChannel;
51022 |
51023 |     if (pGainer == NULL) {
51024 |         return MA_INVALID_ARGS;
51025 |     }
51026 |
51027 |     for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
51028 |         ma_gainer_set_gain_by_index(pGainer, newGain, iChannel);
51029 |     }
51030 |
51031 |     /* The smoothing time needs to be reset to ensure we always interpolate by the configured
> smoothing time, but only if it's not the first setting. */
51032 |     ma_gainer_reset_smoothing_time(pGainer);
51033 |
51034 |     return MA_SUCCESS;
51035 | }
51036 |
51037 | MA_API ma_result ma_gainer_set_gains(ma_gainer* pGainer, float* pNewGains)
51038 | {
51039 |     ma_uint32 iChannel;
51040 |
51041 |     if (pGainer == NULL || pNewGains == NULL) {
51042 |         return MA_INVALID_ARGS;
51043 |     }

```

```

51044 |
51045 |     for (iChannel = 0; iChannel < pGainer->config.channels; iChannel += 1) {
51046 |         ma_gainer_set_gain_by_index(pGainer, pNewGains[iChannel], iChannel);
51047 |     }
51048 |
51049 |     /* The smoothing time needs to be reset to ensure we always interpolate by the configured
> smoothing time, but only if it's not the first setting. */
51050 |     ma_gainer_reset_smoothing_time(pGainer);
51051 |
51052 |     return MA_SUCCESS;
51053 | }
51054 |
51055 | MA_API ma_result ma_gainer_set_master_volume(ma_gainer* pGainer, float volume)
51056 | {
51057 |     if (pGainer == NULL) {
51058 |         return MA_INVALID_ARGS;
51059 |     }
51060 |
51061 |     pGainer->masterVolume = volume;
51062 |
51063 |     return MA_SUCCESS;
51064 | }
51065 |
51066 | MA_API ma_result ma_gainer_get_master_volume(const ma_gainer* pGainer, float* pVolume)
51067 | {
51068 |     if (pGainer == NULL || pVolume == NULL) {
51069 |         return MA_INVALID_ARGS;
51070 |     }
51071 |
51072 |     *pVolume = pGainer->masterVolume;
51073 |
51074 |     return MA_SUCCESS;
51075 | }
51076 |
51077 |
51078 | MA_API ma_panner_config ma_panner_config_init(ma_format format, ma_uint32 channels)
51079 | {
51080 |     ma_panner_config config;
51081 |
51082 |     MA_ZERO_OBJECT(&config);
51083 |     config.format = format;
51084 |     config.channels = channels;
51085 |     config.mode = ma_pan_mode_balance; /* Set to balancing mode by default because it's
> consistent with other audio engines and most likely what the caller is expecting. */
51086 |     config.pan = 0;
51087 |
51088 |     return config;
51089 | }
51090 |
51091 |
51092 | MA_API ma_result ma_panner_init(const ma_panner_config* pConfig, ma_panner* pPanner)
51093 | {
51094 |     if (pPanner == NULL) {
51095 |         return MA_INVALID_ARGS;
51096 |     }
51097 |
51098 |     MA_ZERO_OBJECT(pPanner);
51099 |
51100 |     if (pConfig == NULL) {
51101 |         return MA_INVALID_ARGS;
51102 |     }
51103 |
51104 |     pPanner->format = pConfig->format;
51105 |     pPanner->channels = pConfig->channels;
51106 |     pPanner->mode = pConfig->mode;
51107 |     pPanner->pan = pConfig->pan;
51108 |
51109 |     return MA_SUCCESS;
51110 | }
51111 |
51112 | static void ma_stereo_balance_pcm_frames_f32(float* pFramesOut, const float* pFramesIn, ma_uint64
> frameCount, float pan)
51113 | {
51114 |     ma_uint64 iFrame;
51115 |
51116 |     if (pan > 0) {
51117 |         float factor = 1.0f - pan;
51118 |         if (pFramesOut == pFramesIn) {
51119 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {

```

```

51120 |         pFramesOut[iFrame*2 + 0] = pFramesIn[iFrame*2 + 0] * factor;
51121 |     }
51122 | } else {
51123 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
51124 |         pFramesOut[iFrame*2 + 0] = pFramesIn[iFrame*2 + 0] * factor;
51125 |         pFramesOut[iFrame*2 + 1] = pFramesIn[iFrame*2 + 1];
51126 |     }
51127 | }
51128 | } else {
51129 |     float factor = 1.0f + pan;
51130 |     if (pFramesOut == pFramesIn) {
51131 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
51132 |             pFramesOut[iFrame*2 + 1] = pFramesIn[iFrame*2 + 1] * factor;
51133 |         }
51134 |     } else {
51135 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
51136 |             pFramesOut[iFrame*2 + 0] = pFramesIn[iFrame*2 + 0];
51137 |             pFramesOut[iFrame*2 + 1] = pFramesIn[iFrame*2 + 1] * factor;
51138 |         }
51139 |     }
51140 | }
51141 | }
51142 |
51143 | static void ma_stereo_balance_pcm_frames(void* pFramesOut, const void* pFramesIn, ma_uint64
> frameCount, ma_format format, float pan)
51144 | {
51145 |     if (pan == 0) {
51146 |         /* Fast path. No panning required. */
51147 |         if (pFramesOut == pFramesIn) {
51148 |             /* No-op */
51149 |         } else {
51150 |             ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, format, 2);
51151 |         }
51152 |
51153 |         return;
51154 |     }
51155 |
51156 |     switch (format) {
51157 |         case ma_format_f32: ma_stereo_balance_pcm_frames_f32((float*)pFramesOut, (float*)pFramesIn,
> frameCount, pan); break;
51158 |
51159 |         /* Unknown format. Just copy. */
51160 |         default:
51161 |         {
51162 |             ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, format, 2);
51163 |         } break;
51164 |     }
51165 | }
51166 |
51167 |
51168 | static void ma_stereo_pan_pcm_frames_f32(float* pFramesOut, const float* pFramesIn, ma_uint64
> frameCount, float pan)
51169 | {
51170 |     ma_uint64 iFrame;
51171 |
51172 |     if (pan > 0) {
51173 |         float factorL0 = 1.0f - pan;
51174 |         float factorL1 = 0.0f + pan;
51175 |
51176 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
51177 |             float sample0 = (pFramesIn[iFrame*2 + 0] * factorL0);
51178 |             float sample1 = (pFramesIn[iFrame*2 + 0] * factorL1) + pFramesIn[iFrame*2 + 1];
51179 |
51180 |             pFramesOut[iFrame*2 + 0] = sample0;
51181 |             pFramesOut[iFrame*2 + 1] = sample1;
51182 |         }
51183 |     } else {
51184 |         float factorR0 = 0.0f - pan;
51185 |         float factorR1 = 1.0f + pan;
51186 |
51187 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
51188 |             float sample0 = pFramesIn[iFrame*2 + 0] + (pFramesIn[iFrame*2 + 1] * factorR0);
51189 |             float sample1 =
51190 |                 (pFramesIn[iFrame*2 + 1] * factorR1);
51191 |
51192 |             pFramesOut[iFrame*2 + 0] = sample0;
51193 |             pFramesOut[iFrame*2 + 1] = sample1;
51194 |         }
51195 |     }

```



```

51196 |
51197 | static void ma_stereo_pan_pcm_frames(void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount,
> ma_format format, float pan)
51198 | {
51199 |     if (pan == 0) {
51200 |         /* Fast path. No panning required. */
51201 |         if (pFramesOut == pFramesIn) {
51202 |             /* No-op */
51203 |         } else {
51204 |             ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, format, 2);
51205 |         }
51206 |
51207 |         return;
51208 |     }
51209 |
51210 |     switch (format) {
51211 |         case ma_format_f32: ma_stereo_pan_pcm_frames_f32((float*)pFramesOut, (float*)pFramesIn,
> frameCount, pan); break;
51212 |
51213 |         /* Unknown format. Just copy. */
51214 |         default:
51215 |         {
51216 |             ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, format, 2);
51217 |         } break;
51218 |     }
51219 | }
51220 |
51221 | MA_API ma_result ma_panner_process_pcm_frames(ma_panner* pPanner, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
51222 | {
51223 |     if (pPanner == NULL || pFramesOut == NULL || pFramesIn == NULL) {
51224 |         return MA_INVALID_ARGS;
51225 |     }
51226 |
51227 |     if (pPanner->channels == 2) {
51228 |         /* Stereo case. For now assume channel 0 is left and channel right is 1, but should probably
> add support for a channel map. */
51229 |         if (pPanner->mode == ma_pan_mode_balance) {
51230 |             ma_stereo_balance_pcm_frames(pFramesOut, pFramesIn, frameCount, pPanner->format,
> pPanner->pan);
51231 |         } else {
51232 |             ma_stereo_pan_pcm_frames(pFramesOut, pFramesIn, frameCount, pPanner->format,
> pPanner->pan);
51233 |         }
51234 |     } else {
51235 |         if (pPanner->channels == 1) {
51236 |             /* Panning has no effect on mono streams. */
51237 |             ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, pPanner->format,
> pPanner->channels);
51238 |         } else {
51239 |             /* For now we're not going to support non-stereo set ups. Not sure how I want to handle
> this case just yet. */
51240 |             ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, pPanner->format,
> pPanner->channels);
51241 |         }
51242 |     }
51243 |
51244 |     return MA_SUCCESS;
51245 | }
51246 |
51247 | MA_API void ma_panner_set_mode(ma_panner* pPanner, ma_pan_mode mode)
51248 | {
51249 |     if (pPanner == NULL) {
51250 |         return;
51251 |     }
51252 |
51253 |     pPanner->mode = mode;
51254 | }
51255 |
51256 | MA_API ma_pan_mode ma_panner_get_mode(const ma_panner* pPanner)
51257 | {
51258 |     if (pPanner == NULL) {
51259 |         return ma_pan_mode_balance;
51260 |     }
51261 |
51262 |     return pPanner->mode;
51263 | }
51264 |
51265 | MA_API void ma_panner_set_pan(ma_panner* pPanner, float pan)

```

```

51266 | {
51267 |     if (pPanner == NULL) {
51268 |         return;
51269 |     }
51270 |
51271 |     pPanner->pan = ma_clamp(pan, -1.0f, 1.0f);
51272 | }
51273 |
51274 | MA_API float ma_panner_get_pan(const ma_panner* pPanner)
51275 | {
51276 |     if (pPanner == NULL) {
51277 |         return 0;
51278 |     }
51279 |
51280 |     return pPanner->pan;
51281 | }
51282 |
51283 |
51284 |
51285 |
51286 | MA_API ma_fader_config ma_fader_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate)
51287 | {
51288 |     ma_fader_config config;
51289 |
51290 |     MA_ZERO_OBJECT(&config);
51291 |     config.format      = format;
51292 |     config.channels    = channels;
51293 |     config.sampleRate = sampleRate;
51294 |
51295 |     return config;
51296 | }
51297 |
51298 |
51299 | MA_API ma_result ma_fader_init(const ma_fader_config* pConfig, ma_fader* pFader)
51300 | {
51301 |     if (pFader == NULL) {
51302 |         return MA_INVALID_ARGS;
51303 |     }
51304 |
51305 |     MA_ZERO_OBJECT(pFader);
51306 |
51307 |     if (pConfig == NULL) {
51308 |         return MA_INVALID_ARGS;
51309 |     }
51310 |
51311 |     /* Only f32 is supported for now. */
51312 |     if (pConfig->format != ma_format_f32) {
51313 |         return MA_INVALID_ARGS;
51314 |     }
51315 |
51316 |     pFader->config      = *pConfig;
51317 |     pFader->volumeBeg   = 1;
51318 |     pFader->volumeEnd   = 1;
51319 |     pFader->lengthInFrames = 0;
51320 |     pFader->cursorInFrames = 0;
51321 |
51322 |     return MA_SUCCESS;
51323 | }
51324 |
51325 | MA_API ma_result ma_fader_process_pcm_frames(ma_fader* pFader, void* pFramesOut, const void*
> pFramesIn, ma_uint64 frameCount)
51326 | {
51327 |     if (pFader == NULL) {
51328 |         return MA_INVALID_ARGS;
51329 |     }
51330 |
51331 |     /* If the cursor is still negative we need to just copy the absolute number of those frames, but
> no more than frameCount. */
51332 |     if (pFader->cursorInFrames < 0) {
51333 |         ma_uint64 absCursorInFrames = (ma_uint64)0 - pFader->cursorInFrames;
51334 |         if (absCursorInFrames > frameCount) {
51335 |             absCursorInFrames = frameCount;
51336 |         }
51337 |
51338 |         ma_copy_pcm_frames(pFramesOut, pFramesIn, absCursorInFrames, pFader->config.format,
> pFader->config.channels);
51339 |
51340 |         pFader->cursorInFrames += absCursorInFrames;

```

```

51341 |         frameCount -= absCursorInFrames;
51342 |         pFramesOut = ma_offset_ptr(pFramesOut, ma_get_bytes_per_frame(pFader->config.format,
> pFader->config.channels)*absCursorInFrames);
51343 |         pFramesIn = ma_offset_ptr(pFramesIn, ma_get_bytes_per_frame(pFader->config.format,
> pFader->config.channels)*absCursorInFrames);
51344 |     }
51345 |
51346 |     if (pFader->cursorInFrames >= 0) {
51347 |         /*
51348 |         For now we need to clamp frameCount so that the cursor never overflows 32-bits. This is
> required for
51349 |         the conversion to a float which we use for the linear interpolation. This might be changed
> later.
51350 |         */
51351 |         if (frameCount + pFader->cursorInFrames > UINT_MAX) {
51352 |             frameCount = UINT_MAX - pFader->cursorInFrames;
51353 |         }
51354 |
51355 |         /* Optimized path if volumeBeg and volumeEnd are equal. */
51356 |         if (pFader->volumeBeg == pFader->volumeEnd) {
51357 |             if (pFader->volumeBeg == 1) {
51358 |                 /* Straight copy. */
51359 |                 ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, pFader->config.format,
> pFader->config.channels);
51360 |             } else {
51361 |                 /* Copy with volume. */
51362 |                 ma_copy_and_apply_volume_and_clip_pcm_frames(pFramesOut, pFramesIn, frameCount,
> pFader->config.format, pFader->config.channels, pFader->volumeBeg);
51363 |             }
51364 |         } else {
51365 |             /* Slower path. Volumes are different, so may need to do an interpolation. */
51366 |             if ((ma_uint64)pFader->cursorInFrames >= pFader->lengthInFrames) {
51367 |                 /* Fast path. We've gone past the end of the fade period so just apply the end
> volume to all samples. */
51368 |                 ma_copy_and_apply_volume_and_clip_pcm_frames(pFramesOut, pFramesIn, frameCount,
> pFader->config.format, pFader->config.channels, pFader->volumeEnd);
51369 |             } else {
51370 |                 /* Slow path. This is where we do the actual fading. */
51371 |                 ma_uint64 iFrame;
51372 |                 ma_uint32 iChannel;
51373 |
51374 |                 /* For now we only support f32. Support for other formats might be added later. */
51375 |                 if (pFader->config.format == ma_format_f32) {
51376 |                     const float* pFramesInF32 = (const float*)pFramesIn;
51377 |                     /* */ float* pFramesOutF32 = (float*)pFramesOut;
51378 |
51379 |                     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
51380 |                         float a = (ma_uint32)ma_min(pFader->cursorInFrames + iFrame,
> pFader->lengthInFrames) / (float)((ma_uint32)pFader->lengthInFrames); /* Safe cast due to the
> frameCount clamp at the top of this function. */
51381 |                         float volume = ma_mix_f32_fast(pFader->volumeBeg, pFader->volumeEnd, a);
51382 |
51383 |                         for (iChannel = 0; iChannel < pFader->config.channels; iChannel += 1) {
51384 |                             pFramesOutF32[iFrame*pFader->config.channels + iChannel] =
> pFramesInF32[iFrame*pFader->config.channels + iChannel] * volume;
51385 |                         }
51386 |                     }
51387 |                 } else {
51388 |                     return MA_NOT_IMPLEMENTED;
51389 |                 }
51390 |             }
51391 |         }
51392 |     }
51393 |
51394 |     pFader->cursorInFrames += frameCount;
51395 |
51396 |     return MA_SUCCESS;
51397 | }
51398 |
51399 | MA_API void ma_fader_get_data_format(const ma_fader* pFader, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate)
51400 | {
51401 |     if (pFader == NULL) {
51402 |         return;
51403 |     }
51404 |
51405 |     if (pFormat != NULL) {
51406 |         *pFormat = pFader->config.format;
51407 |     }

```

```

51408 |
51409 |     if (pChannels != NULL) {
51410 |         *pChannels = pFader->config.channels;
51411 |     }
51412 |
51413 |     if (pSampleRate != NULL) {
51414 |         *pSampleRate = pFader->config.sampleRate;
51415 |     }
51416 | }
51417 |
51418 | MA_API void ma_fader_set_fade(ma_fader* pFader, float volumeBeg, float volumeEnd, ma_uint64
> lengthInFrames)
51419 | {
51420 |     ma_fader_set_fade_ex(pFader, volumeBeg, volumeEnd, lengthInFrames, 0);
51421 | }
51422 |
51423 | MA_API void ma_fader_set_fade_ex(ma_fader* pFader, float volumeBeg, float volumeEnd, ma_uint64
> lengthInFrames, ma_int64 startOffsetInFrames)
51424 | {
51425 |     if (pFader == NULL) {
51426 |         return;
51427 |     }
51428 |
51429 |     /* If the volume is negative, use current volume. */
51430 |     if (volumeBeg < 0) {
51431 |         volumeBeg = ma_fader_get_current_volume(pFader);
51432 |     }
51433 |
51434 |     /*
51435 |     The length needs to be clamped to 32-bits due to how we convert it to a float for linear
51436 |     interpolation reasons. I might change this requirement later, but for now it's not important.
51437 |     */
51438 |     if (lengthInFrames > UINT_MAX) {
51439 |         lengthInFrames = UINT_MAX;
51440 |     }
51441 |
51442 |     /* The start offset needs to be clamped to ensure it doesn't overflow a signed number. */
51443 |     if (startOffsetInFrames > INT_MAX) {
51444 |         startOffsetInFrames = INT_MAX;
51445 |     }
51446 |
51447 |     pFader->volumeBeg      = volumeBeg;
51448 |     pFader->volumeEnd      = volumeEnd;
51449 |     pFader->lengthInFrames = lengthInFrames;
51450 |     pFader->cursorInFrames = -startOffsetInFrames;
51451 | }
51452 |
51453 | MA_API float ma_fader_get_current_volume(const ma_fader* pFader)
51454 | {
51455 |     if (pFader == NULL) {
51456 |         return 0.0f;
51457 |     }
51458 |
51459 |     /* Any frames prior to the start of the fade period will be at unfaded volume. */
51460 |     if (pFader->cursorInFrames < 0) {
51461 |         return 1.0f;
51462 |     }
51463 |
51464 |     /* The current volume depends on the position of the cursor. */
51465 |     if (pFader->cursorInFrames == 0) {
51466 |         return pFader->volumeBeg;
51467 |     } else if ((ma_uint64)pFader->cursorInFrames >= pFader->lengthInFrames) { /* Safe case because
> the < 0 case was checked above. */
51468 |         return pFader->volumeEnd;
51469 |     } else {
51470 |         /* The cursor is somewhere inside the fading period. We can figure this out with a simple
> linear interpolation between volumeBeg and volumeEnd based on our cursor position. */
51471 |         return ma_mix_f32_fast(pFader->volumeBeg, pFader->volumeEnd,
> (ma_uint32)pFader->cursorInFrames / (float)((ma_uint32)pFader->lengthInFrames)); /* Safe cast to
> uint32 because we clamp it in ma_fader_process_pcm_frames(). */
51472 |     }
51473 | }
51474 |
51475 |
51476 |
51477 |
51478 |
51479 | MA_API ma_vec3f ma_vec3f_init_3f(float x, float y, float z)
51480 | {

```

```
51481 |     ma_vec3f v;
51482 |
51483 |     v.x = x;
51484 |     v.y = y;
51485 |     v.z = z;
51486 |
51487 |     return v;
51488 | }
51489 |
51490 | MA_API ma_vec3f ma_vec3f_sub(ma_vec3f a, ma_vec3f b)
51491 | {
51492 |     return ma_vec3f_init_3f(
51493 |         a.x - b.x,
51494 |         a.y - b.y,
51495 |         a.z - b.z
51496 |     );
51497 | }
51498 |
51499 | MA_API ma_vec3f ma_vec3f_neg(ma_vec3f a)
51500 | {
51501 |     return ma_vec3f_init_3f(
51502 |         -a.x,
51503 |         -a.y,
51504 |         -a.z
51505 |     );
51506 | }
51507 |
51508 | MA_API float ma_vec3f_dot(ma_vec3f a, ma_vec3f b)
51509 | {
51510 |     return a.x*b.x + a.y*b.y + a.z*b.z;
51511 | }
51512 |
51513 | MA_API float ma_vec3f_len2(ma_vec3f v)
51514 | {
51515 |     return ma_vec3f_dot(v, v);
51516 | }
51517 |
51518 | MA_API float ma_vec3f_len(ma_vec3f v)
51519 | {
51520 |     return (float)ma_sqrtf(ma_vec3f_len2(v));
51521 | }
51522 |
51523 |
51524 |
51525 | MA_API float ma_vec3f_dist(ma_vec3f a, ma_vec3f b)
51526 | {
51527 |     return ma_vec3f_len(ma_vec3f_sub(a, b));
51528 | }
51529 |
51530 | MA_API ma_vec3f ma_vec3f_normalize(ma_vec3f v)
51531 | {
51532 |     float invLen;
51533 |     float len2 = ma_vec3f_len2(v);
51534 |     if (len2 == 0) {
51535 |         return ma_vec3f_init_3f(0, 0, 0);
51536 |     }
51537 |
51538 |     invLen = ma_sqrtf(len2);
51539 |     v.x *= invLen;
51540 |     v.y *= invLen;
51541 |     v.z *= invLen;
51542 |
51543 |     return v;
51544 | }
51545 |
51546 | MA_API ma_vec3f ma_vec3f_cross(ma_vec3f a, ma_vec3f b)
51547 | {
51548 |     return ma_vec3f_init_3f(
51549 |         a.y*b.z - a.z*b.y,
51550 |         a.z*b.x - a.x*b.z,
51551 |         a.x*b.y - a.y*b.x
51552 |     );
51553 | }
51554 |
51555 |
51556 | MA_API void ma_atomic_vec3f_init(ma_atomic_vec3f* v, ma_vec3f value)
51557 | {
51558 |     v->v = value;
51559 |     v->lock = 0;    /* Important this is initialized to 0. */
```

```

51560 | }
51561 |
51562 | MA_API void ma_atomic_vec3f_set(ma_atomic_vec3f* v, ma_vec3f value)
51563 | {
51564 |     ma_spinlock_lock(&v->lock);
51565 |     {
51566 |         v->v = value;
51567 |     }
51568 |     ma_spinlock_unlock(&v->lock);
51569 | }
51570 |
51571 | MA_API ma_vec3f ma_atomic_vec3f_get(ma_atomic_vec3f* v)
51572 | {
51573 |     ma_vec3f r;
51574 |
51575 |     ma_spinlock_lock(&v->lock);
51576 |     {
51577 |         r = v->v;
51578 |     }
51579 |     ma_spinlock_unlock(&v->lock);
51580 |
51581 |     return r;
51582 | }
51583 |
51584 |
51585 |
51586 | static void ma_channel_map_apply_f32(float* pFramesOut, const ma_channel* pChannelMapOut, ma_uint32
> channelsOut, const float* pFramesIn, const ma_channel* pChannelMapIn, ma_uint32 channelsIn,
> ma_uint64 frameCount, ma_channel_mix_mode mode, ma_mono_expansion_mode monoExpansionMode);
51587 | static ma_bool32 ma_is_spatial_channel_position(ma_channel channelPosition);
51588 |
51589 |
51590 | #ifndef MA_DEFAULT_SPEED_OF_SOUND
51591 | #define MA_DEFAULT_SPEED_OF_SOUND 343.3f
51592 | #endif
51593 |
51594 | /*
51595 | These vectors represent the direction that speakers are facing from the center point. They're used
51596 | for panning in the spatializer. Must be normalized.
51597 | */
51598 | static ma_vec3f g_maChannelDirections[MA_CHANNEL_POSITION_COUNT] = {
51599 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_NONE */
51600 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_MONO */
51601 |     { -0.7071f, 0.0f, -0.7071f }, /* MA_CHANNEL_FRONT_LEFT */
51602 |     { +0.7071f, 0.0f, -0.7071f }, /* MA_CHANNEL_FRONT_RIGHT */
51603 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_FRONT_CENTER */
51604 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_LFE */
51605 |     { -0.7071f, 0.0f, +0.7071f }, /* MA_CHANNEL_BACK_LEFT */
51606 |     { +0.7071f, 0.0f, +0.7071f }, /* MA_CHANNEL_BACK_RIGHT */
51607 |     { -0.3162f, 0.0f, -0.9487f }, /* MA_CHANNEL_FRONT_LEFT_CENTER */
51608 |     { +0.3162f, 0.0f, -0.9487f }, /* MA_CHANNEL_FRONT_RIGHT_CENTER */
51609 |     { 0.0f, 0.0f, +1.0f }, /* MA_CHANNEL_BACK_CENTER */
51610 |     { -1.0f, 0.0f, 0.0f }, /* MA_CHANNEL_SIDE_LEFT */
51611 |     { +1.0f, 0.0f, 0.0f }, /* MA_CHANNEL_SIDE_RIGHT */
51612 |     { 0.0f, +1.0f, 0.0f }, /* MA_CHANNEL_TOP_CENTER */
51613 |     { -0.5774f, +0.5774f, -0.5774f }, /* MA_CHANNEL_TOP_FRONT_LEFT */
51614 |     { 0.0f, +0.7071f, -0.7071f }, /* MA_CHANNEL_TOP_FRONT_CENTER */
51615 |     { +0.5774f, +0.5774f, -0.5774f }, /* MA_CHANNEL_TOP_FRONT_RIGHT */
51616 |     { -0.5774f, +0.5774f, +0.5774f }, /* MA_CHANNEL_TOP_BACK_LEFT */
51617 |     { 0.0f, +0.7071f, +0.7071f }, /* MA_CHANNEL_TOP_BACK_CENTER */
51618 |     { +0.5774f, +0.5774f, +0.5774f }, /* MA_CHANNEL_TOP_BACK_RIGHT */
51619 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_0 */
51620 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_1 */
51621 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_2 */
51622 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_3 */
51623 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_4 */
51624 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_5 */
51625 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_6 */
51626 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_7 */
51627 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_8 */
51628 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_9 */
51629 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_10 */
51630 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_11 */
51631 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_12 */
51632 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_13 */
51633 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_14 */
51634 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_15 */
51635 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_16 */
51636 |     { 0.0f, 0.0f, -1.0f }, /* MA_CHANNEL_AUX_17 */

```

```

51637 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_18 */
51638 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_19 */
51639 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_20 */
51640 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_21 */
51641 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_22 */
51642 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_23 */
51643 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_24 */
51644 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_25 */
51645 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_26 */
51646 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_27 */
51647 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_28 */
51648 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_29 */
51649 | { 0.0f,    0.0f,   -1.0f   }, /* MA_CHANNEL_AUX_30 */
51650 | { 0.0f,    0.0f,   -1.0f   } /* MA_CHANNEL_AUX_31 */
51651 | };
51652 |
51653 | static ma_vec3f ma_get_channel_direction(ma_channel channel)
51654 | {
51655 |     if (channel >= MA_CHANNEL_POSITION_COUNT) {
51656 |         return ma_vec3f_init_3f(0, 0, -1);
51657 |     } else {
51658 |         return g_maChannelDirections[channel];
51659 |     }
51660 | }
51661 |
51662 |
51663 |
51664 | static float ma_attenuation_inverse(float distance, float minDistance, float maxDistance, float
> rolloff)
51665 | {
51666 |     if (minDistance >= maxDistance) {
51667 |         return 1; /* To avoid division by zero. Do not attenuate. */
51668 |     }
51669 |
51670 |     return minDistance / (minDistance + rolloff * (ma_clamp(distance, minDistance, maxDistance) -
> minDistance));
51671 | }
51672 |
51673 | static float ma_attenuation_linear(float distance, float minDistance, float maxDistance, float
> rolloff)
51674 | {
51675 |     if (minDistance >= maxDistance) {
51676 |         return 1; /* To avoid division by zero. Do not attenuate. */
51677 |     }
51678 |
51679 |     return 1 - rolloff * (ma_clamp(distance, minDistance, maxDistance) - minDistance) / (maxDistance
> - minDistance);
51680 | }
51681 |
51682 | static float ma_attenuation_exponential(float distance, float minDistance, float maxDistance, float
> rolloff)
51683 | {
51684 |     if (minDistance >= maxDistance) {
51685 |         return 1; /* To avoid division by zero. Do not attenuate. */
51686 |     }
51687 |
51688 |     return (float)ma_powd(ma_clamp(distance, minDistance, maxDistance) / minDistance, -rolloff);
51689 | }
51690 |
51691 |
51692 | /*
51693 | Doppler Effect calculation taken from the OpenAL spec, with two main differences:
51694 |
51695 | 1) The source to listener vector will have already been calculated at an earlier step so we can
51696 | just use that directly. We need only the position of the source relative to the origin.
51697 |
51698 | 2) We don't scale by a frequency because we actually just want the ratio which we'll plug straight
51699 | into the resampler directly.
51700 | */
51701 | static float ma_doppler_pitch(ma_vec3f relativePosition, ma_vec3f sourceVelocity, ma_vec3f
> listenVelocity, float speedOfSound, float dopplerFactor)
51702 | {
51703 |     float len;
51704 |     float vls;
51705 |     float vss;
51706 |
51707 |     len = ma_vec3f_len(relativePosition);
51708 |
51709 |     /*

```



```

51710 |   There's a case where the position of the source will be right on top of the listener in which
51711 |   case the length will be 0 and we'll end up with a division by zero. We can just return a ratio
51712 |   of 1.0 in this case. This is not considered in the OpenAL spec, but is necessary.
51713 |   */
51714 |   if (len == 0) {
51715 |       return 1.0;
51716 |   }
51717 |
51718 |   vls = ma_vec3f_dot(relativePosition, listenVelocity) / len;
51719 |   vss = ma_vec3f_dot(relativePosition, sourceVelocity) / len;
51720 |
51721 |   vls = ma_min(vls, speedOfSound / dopplerFactor);
51722 |   vss = ma_min(vss, speedOfSound / dopplerFactor);
51723 |
51724 |   return (speedOfSound - dopplerFactor*vls) / (speedOfSound - dopplerFactor*vss);
51725 | }
51726 |
51727 |
51728 | static void ma_get_default_channel_map_for_spatializer(ma_channel* pChannelMap, size_t
> channelMapCap, ma_uint32 channelCount)
51729 | {
51730 |     /*
51731 |     Special case for stereo. Want to default the left and right speakers to side left and side
51732 |     right so that they're facing directly down the X axis rather than slightly forward. Not
51733 |     doing this will result in sounds being quieter when behind the listener. This might
51734 |     actually be good for some scenarios, but I don't think it's an appropriate default because
51735 |     it can be a bit unexpected.
51736 |     */
51737 |     if (channelCount == 2) {
51738 |         pChannelMap[0] = MA_CHANNEL_SIDE_LEFT;
51739 |         pChannelMap[1] = MA_CHANNEL_SIDE_RIGHT;
51740 |     } else {
51741 |         ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> channelCount);
51742 |     }
51743 | }
51744 |
51745 |
51746 | MA_API ma_spatializer_listener_config ma_spatializer_listener_config_init(ma_uint32 channelsOut)
51747 | {
51748 |     ma_spatializer_listener_config config;
51749 |
51750 |     MA_ZERO_OBJECT(&config);
51751 |     config.channelsOut = channelsOut;
51752 |     config.pChannelMapOut = NULL;
51753 |     config.handedness = ma_handedness_right;
51754 |     config.worldUp = ma_vec3f_init_3f(0, 1, 0);
51755 |     config.coneInnerAngleInRadians = 6.283185f; /* 360 degrees. */
51756 |     config.coneOuterAngleInRadians = 6.283185f; /* 360 degrees. */
51757 |     config.coneOuterGain = 0;
51758 |     config.speedOfSound = 343.3f; /* Same as OpenAL. Used for doppler effect. */
51759 |
51760 |     return config;
51761 | }
51762 |
51763 |
51764 | typedef struct
51765 | {
51766 |     size_t sizeInBytes;
51767 |     size_t channelMapOutOffset;
51768 | } ma_spatializer_listener_heap_layout;
51769 |
51770 | static ma_result ma_spatializer_listener_get_heap_layout(const ma_spatializer_listener_config*
> pConfig, ma_spatializer_listener_heap_layout* pHeapLayout)
51771 | {
51772 |     MA_ASSERT(pHeapLayout != NULL);
51773 |
51774 |     MA_ZERO_OBJECT(pHeapLayout);
51775 |
51776 |     if (pConfig == NULL) {
51777 |         return MA_INVALID_ARGS;
51778 |     }
51779 |
51780 |     if (pConfig->channelsOut == 0) {
51781 |         return MA_INVALID_ARGS;
51782 |     }
51783 |
51784 |     pHeapLayout->sizeInBytes = 0;
51785 |

```

```

51786 | /* Channel map. We always need this, even for passthroughs. */
51787 | pHeapLayout->channelMapOutOffset = pHeapLayout->sizeInBytes;
51788 | pHeapLayout->sizeInBytes += ma_align_64(sizeof(*pConfig->pChannelMapOut) *
> pConfig->channelsOut);
51789 |
51790 | return MA_SUCCESS;
51791 | }
51792 |
51793 |
51794 | MA_API ma_result ma_spatializer_listener_get_heap_size(const ma_spatializer_listener_config*
> pConfig, size_t* pHeapSizeInBytes)
51795 | {
51796 |     ma_result result;
51797 |     ma_spatializer_listener_heap_layout heapLayout;
51798 |
51799 |     if (pHeapSizeInBytes == NULL) {
51800 |         return MA_INVALID_ARGS;
51801 |     }
51802 |
51803 |     *pHeapSizeInBytes = 0;
51804 |
51805 |     result = ma_spatializer_listener_get_heap_layout(pConfig, &heapLayout);
51806 |     if (result != MA_SUCCESS) {
51807 |         return result;
51808 |     }
51809 |
51810 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
51811 |
51812 |     return MA_SUCCESS;
51813 | }
51814 |
51815 | MA_API ma_result ma_spatializer_listener_init_preallocated(const ma_spatializer_listener_config*
> pConfig, void* pHeap, ma_spatializer_listener* pListener)
51816 | {
51817 |     ma_result result;
51818 |     ma_spatializer_listener_heap_layout heapLayout;
51819 |
51820 |     if (pListener == NULL) {
51821 |         return MA_INVALID_ARGS;
51822 |     }
51823 |
51824 |     MA_ZERO_OBJECT(pListener);
51825 |
51826 |     result = ma_spatializer_listener_get_heap_layout(pConfig, &heapLayout);
51827 |     if (result != MA_SUCCESS) {
51828 |         return result;
51829 |     }
51830 |
51831 |     pListener->pHeap = pHeap;
51832 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
51833 |
51834 |     pListener->config = *pConfig;
51835 |     ma_atomic_vec3f_init(&pListener->position, ma_vec3f_init_3f(0, 0, 0));
51836 |     ma_atomic_vec3f_init(&pListener->direction, ma_vec3f_init_3f(0, 0, -1));
51837 |     ma_atomic_vec3f_init(&pListener->velocity, ma_vec3f_init_3f(0, 0, 0));
51838 |     pListener->isEnabled = MA_TRUE;
51839 |
51840 |     /* Swap the forward direction if we're left handed (it was initialized based on right handed).
> */
51841 |     if (pListener->config.handedness == ma_handedness_left) {
51842 |         ma_vec3f negDir = ma_vec3f_neg(ma_spatializer_listener_get_direction(pListener));
51843 |         ma_spatializer_listener_set_direction(pListener, negDir.x, negDir.y, negDir.z);
51844 |     }
51845 |
51846 |
51847 |     /* We must always have a valid channel map. */
51848 |     pListener->config.pChannelMapOut = (ma_channel*)ma_offset_ptr(pHeap,
> heapLayout.channelMapOutOffset);
51849 |
51850 |     /* Use a slightly different default channel map for stereo. */
51851 |     if (pConfig->pChannelMapOut == NULL) {
51852 |         ma_get_default_channel_map_for_spatializer(pListener->config.pChannelMapOut,
> pConfig->channelsOut, pConfig->channelsOut);
51853 |     } else {
51854 |         ma_channel_map_copy_or_default(pListener->config.pChannelMapOut, pConfig->channelsOut,
> pConfig->pChannelMapOut, pConfig->channelsOut);
51855 |     }
51856 |
51857 |     return MA_SUCCESS;

```

```

51858 | }
51859 |
51860 | MA_API ma_result ma_spatializer_listener_init(const ma_spatializer_listener_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_spatializer_listener* pListener)
51861 | {
51862 |     ma_result result;
51863 |     size_t heapSizeInBytes;
51864 |     void* pHeap;
51865 |
51866 |     result = ma_spatializer_listener_get_heap_size(pConfig, &heapSizeInBytes);
51867 |     if (result != MA_SUCCESS) {
51868 |         return result;
51869 |     }
51870 |
51871 |     if (heapSizeInBytes > 0) {
51872 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
51873 |         if (pHeap == NULL) {
51874 |             return MA_OUT_OF_MEMORY;
51875 |         }
51876 |     } else {
51877 |         pHeap = NULL;
51878 |     }
51879 |
51880 |     result = ma_spatializer_listener_init_preallocated(pConfig, pHeap, pListener);
51881 |     if (result != MA_SUCCESS) {
51882 |         ma_free(pHeap, pAllocationCallbacks);
51883 |         return result;
51884 |     }
51885 |
51886 |     pListener->ownsHeap = MA_TRUE;
51887 |     return MA_SUCCESS;
51888 | }
51889 |
51890 | MA_API void ma_spatializer_listener_uninit(ma_spatializer_listener* pListener, const
> ma_allocation_callbacks* pAllocationCallbacks)
51891 | {
51892 |     if (pListener == NULL) {
51893 |         return;
51894 |     }
51895 |
51896 |     if (pListener->ownsHeap) {
51897 |         ma_free(pListener->pHeap, pAllocationCallbacks);
51898 |     }
51899 | }
51900 |
51901 | MA_API ma_channel* ma_spatializer_listener_get_channel_map(ma_spatializer_listener* pListener)
51902 | {
51903 |     if (pListener == NULL) {
51904 |         return NULL;
51905 |     }
51906 |
51907 |     return pListener->config.pChannelMapOut;
51908 | }
51909 |
51910 | MA_API void ma_spatializer_listener_set_cone(ma_spatializer_listener* pListener, float
> innerAngleInRadians, float outerAngleInRadians, float outerGain)
51911 | {
51912 |     if (pListener == NULL) {
51913 |         return;
51914 |     }
51915 |
51916 |     pListener->config.coneInnerAngleInRadians = innerAngleInRadians;
51917 |     pListener->config.coneOuterAngleInRadians = outerAngleInRadians;
51918 |     pListener->config.coneOuterGain = outerGain;
51919 | }
51920 |
51921 | MA_API void ma_spatializer_listener_get_cone(const ma_spatializer_listener* pListener, float*
> pInnerAngleInRadians, float* pOuterAngleInRadians, float* pOuterGain)
51922 | {
51923 |     if (pListener == NULL) {
51924 |         return;
51925 |     }
51926 |
51927 |     if (pInnerAngleInRadians != NULL) {
51928 |         *pInnerAngleInRadians = pListener->config.coneInnerAngleInRadians;
51929 |     }
51930 |
51931 |     if (pOuterAngleInRadians != NULL) {
51932 |         *pOuterAngleInRadians = pListener->config.coneOuterAngleInRadians;

```

```

51933 |     }
51934 |
51935 |     if (pOuterGain != NULL) {
51936 |         *pOuterGain = pListener->config.coneOuterGain;
51937 |     }
51938 | }
51939 |
51940 | MA_API void ma_spatializer_listener_set_position(ma_spatializer_listener* pListener, float x, float
> y, float z)
51941 | {
51942 |     if (pListener == NULL) {
51943 |         return;
51944 |     }
51945 |
51946 |     ma_atomic_vec3f_set(&pListener->position, ma_vec3f_init_3f(x, y, z));
51947 | }
51948 |
51949 | MA_API ma_vec3f ma_spatializer_listener_get_position(const ma_spatializer_listener* pListener)
51950 | {
51951 |     if (pListener == NULL) {
51952 |         return ma_vec3f_init_3f(0, 0, 0);
51953 |     }
51954 |
51955 |     return ma_atomic_vec3f_get((ma_atomic_vec3f*)&pListener->position); /* Naughty const-cast. It's
> just for atomically loading the vec3 which should be safe. */
51956 | }
51957 |
51958 | MA_API void ma_spatializer_listener_set_direction(ma_spatializer_listener* pListener, float x, float
> y, float z)
51959 | {
51960 |     if (pListener == NULL) {
51961 |         return;
51962 |     }
51963 |
51964 |     ma_atomic_vec3f_set(&pListener->direction, ma_vec3f_init_3f(x, y, z));
51965 | }
51966 |
51967 | MA_API ma_vec3f ma_spatializer_listener_get_direction(const ma_spatializer_listener* pListener)
51968 | {
51969 |     if (pListener == NULL) {
51970 |         return ma_vec3f_init_3f(0, 0, -1);
51971 |     }
51972 |
51973 |     return ma_atomic_vec3f_get((ma_atomic_vec3f*)&pListener->direction); /* Naughty const-cast.
> It's just for atomically loading the vec3 which should be safe. */
51974 | }
51975 |
51976 | MA_API void ma_spatializer_listener_set_velocity(ma_spatializer_listener* pListener, float x, float
> y, float z)
51977 | {
51978 |     if (pListener == NULL) {
51979 |         return;
51980 |     }
51981 |
51982 |     ma_atomic_vec3f_set(&pListener->velocity, ma_vec3f_init_3f(x, y, z));
51983 | }
51984 |
51985 | MA_API ma_vec3f ma_spatializer_listener_get_velocity(const ma_spatializer_listener* pListener)
51986 | {
51987 |     if (pListener == NULL) {
51988 |         return ma_vec3f_init_3f(0, 0, 0);
51989 |     }
51990 |
51991 |     return ma_atomic_vec3f_get((ma_atomic_vec3f*)&pListener->velocity); /* Naughty const-cast. It's
> just for atomically loading the vec3 which should be safe. */
51992 | }
51993 |
51994 | MA_API void ma_spatializer_listener_set_speed_of_sound(ma_spatializer_listener* pListener, float
> speedOfSound)
51995 | {
51996 |     if (pListener == NULL) {
51997 |         return;
51998 |     }
51999 |
52000 |     pListener->config.speedOfSound = speedOfSound;
52001 | }
52002 |
52003 | MA_API float ma_spatializer_listener_get_speed_of_sound(const ma_spatializer_listener* pListener)
52004 | {

```

```

52005 |     if (pListener == NULL) {
52006 |         return 0;
52007 |     }
52008 |
52009 |     return pListener->config.speedOfSound;
52010 | }
52011 |
52012 | MA_API void ma_spatializer_listener_set_world_up(ma_spatializer_listener* pListener, float x, float
> y, float z)
52013 | {
52014 |     if (pListener == NULL) {
52015 |         return;
52016 |     }
52017 |
52018 |     pListener->config.worldUp = ma_vec3f_init_3f(x, y, z);
52019 | }
52020 |
52021 | MA_API ma_vec3f ma_spatializer_listener_get_world_up(const ma_spatializer_listener* pListener)
52022 | {
52023 |     if (pListener == NULL) {
52024 |         return ma_vec3f_init_3f(0, 1, 0);
52025 |     }
52026 |
52027 |     return pListener->config.worldUp;
52028 | }
52029 |
52030 | MA_API void ma_spatializer_listener_set_enabled(ma_spatializer_listener* pListener, ma_bool32
> isEnabled)
52031 | {
52032 |     if (pListener == NULL) {
52033 |         return;
52034 |     }
52035 |
52036 |     pListener->isEnabled = isEnabled;
52037 | }
52038 |
52039 | MA_API ma_bool32 ma_spatializer_listener_is_enabled(const ma_spatializer_listener* pListener)
52040 | {
52041 |     if (pListener == NULL) {
52042 |         return MA_FALSE;
52043 |     }
52044 |
52045 |     return pListener->isEnabled;
52046 | }
52047 |
52048 |
52049 |
52050 |
52051 | MA_API ma_spatializer_config ma_spatializer_config_init(ma_uint32 channelsIn, ma_uint32 channelsOut)
52052 | {
52053 |     ma_spatializer_config config;
52054 |
52055 |     MA_ZERO_OBJECT(&config);
52056 |     config.channelsIn           = channelsIn;
52057 |     config.channelsOut          = channelsOut;
52058 |     config.pChannelMapIn        = NULL;
52059 |     config.attenuationModel      = ma_attenuation_model_inverse;
52060 |     config.positioning           = ma_positioning_absolute;
52061 |     config.handedness           = ma_handedness_right;
52062 |     config.minGain              = 0;
52063 |     config.maxGain              = 1;
52064 |     config.minDistance          = 1;
52065 |     config.maxDistance          = MA_FLT_MAX;
52066 |     config.rolloff               = 1;
52067 |     config.coneInnerAngleInRadians = 6.283185f; /* 360 degrees. */
52068 |     config.coneOuterAngleInRadians = 6.283185f; /* 360 degrees. */
52069 |     config.coneOuterGain        = 0.0f;
52070 |     config.dopplerFactor        = 1;
52071 |     config.directionalAttenuationFactor = 1;
52072 |     config.minSpatializationChannelGain = 0.2f;
52073 |     config.gainSmoothTimeInFrames = 360;          /* 7.5ms @ 48K. */
52074 |
52075 |     return config;
52076 | }
52077 |
52078 |
52079 | static ma_gainer_config ma_spatializer_gainer_config_init(const ma_spatializer_config* pConfig)
52080 | {
52081 |     MA_ASSERT(pConfig != NULL);

```

```

52082 |     return ma_gainer_config_init(pConfig->channelsOut, pConfig->gainSmoothTimeInFrames);
52083 | }
52084 |
52085 | static ma_result ma_spatializer_validate_config(const ma_spatializer_config* pConfig)
52086 | {
52087 |     MA_ASSERT(pConfig != NULL);
52088 |
52089 |     if (pConfig->channelsIn == 0 || pConfig->channelsOut == 0) {
52090 |         return MA_INVALID_ARGS;
52091 |     }
52092 |
52093 |     return MA_SUCCESS;
52094 | }
52095 |
52096 | typedef struct
52097 | {
52098 |     size_t sizeInBytes;
52099 |     size_t channelMapInOffset;
52100 |     size_t newChannelGainsOffset;
52101 |     size_t gainerOffset;
52102 | } ma_spatializer_heap_layout;
52103 |
52104 | static ma_result ma_spatializer_get_heap_layout(const ma_spatializer_config* pConfig,
> ma_spatializer_heap_layout* pHeapLayout)
52105 | {
52106 |     ma_result result;
52107 |
52108 |     MA_ASSERT(pHeapLayout != NULL);
52109 |
52110 |     MA_ZERO_OBJECT(pHeapLayout);
52111 |
52112 |     if (pConfig == NULL) {
52113 |         return MA_INVALID_ARGS;
52114 |     }
52115 |
52116 |     result = ma_spatializer_validate_config(pConfig);
52117 |     if (result != MA_SUCCESS) {
52118 |         return result;
52119 |     }
52120 |
52121 |     pHeapLayout->sizeInBytes = 0;
52122 |
52123 |     /* Channel map. */
52124 |     pHeapLayout->channelMapInOffset = MA_SIZE_MAX; /* <-- MA_SIZE_MAX indicates no allocation
> necessary. */
52125 |     if (pConfig->pChannelMapIn != NULL) {
52126 |         pHeapLayout->channelMapInOffset = pHeapLayout->sizeInBytes;
52127 |         pHeapLayout->sizeInBytes += ma_align_64(sizeof(*pConfig->pChannelMapIn) *
> pConfig->channelsIn);
52128 |     }
52129 |
52130 |     /* New channel gains for output. */
52131 |     pHeapLayout->newChannelGainsOffset = pHeapLayout->sizeInBytes;
52132 |     pHeapLayout->sizeInBytes += ma_align_64(sizeof(float) * pConfig->channelsOut);
52133 |
52134 |     /* Gainer. */
52135 |     {
52136 |         size_t gainerHeapSizeInBytes;
52137 |         ma_gainer_config gainerConfig;
52138 |
52139 |         gainerConfig = ma_spatializer_gainer_config_init(pConfig);
52140 |
52141 |         result = ma_gainer_get_heap_size(&gainerConfig, &gainerHeapSizeInBytes);
52142 |         if (result != MA_SUCCESS) {
52143 |             return result;
52144 |         }
52145 |
52146 |         pHeapLayout->gainerOffset = pHeapLayout->sizeInBytes;
52147 |         pHeapLayout->sizeInBytes += ma_align_64(gainerHeapSizeInBytes);
52148 |     }
52149 |
52150 |     return MA_SUCCESS;
52151 | }
52152 |
52153 | MA_API ma_result ma_spatializer_get_heap_size(const ma_spatializer_config* pConfig, size_t*
> pHeapSizeInBytes)
52154 | {
52155 |     ma_result result;
52156 |     ma_spatializer_heap_layout heapLayout;

```

```

52157 |
52158 |     if (pHeapSizeInBytes == NULL) {
52159 |         return MA_INVALID_ARGS;
52160 |     }
52161 |
52162 |     *pHeapSizeInBytes = 0; /* Safety. */
52163 |
52164 |     result = ma_spatializer_get_heap_layout(pConfig, &heapLayout);
52165 |     if (result != MA_SUCCESS) {
52166 |         return result;
52167 |     }
52168 |
52169 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
52170 |
52171 |     return MA_SUCCESS;
52172 | }
52173 |
52174 |
52175 | MA_API ma_result ma_spatializer_init_preallocated(const ma_spatializer_config* pConfig, void* pHeap,
> ma_spatializer* pSpatializer)
52176 | {
52177 |     ma_result result;
52178 |     ma_spatializer_heap_layout heapLayout;
52179 |     ma_gainer_config gainerConfig;
52180 |
52181 |     if (pSpatializer == NULL) {
52182 |         return MA_INVALID_ARGS;
52183 |     }
52184 |
52185 |     MA_ZERO_OBJECT(pSpatializer);
52186 |
52187 |     if (pConfig == NULL || pHeap == NULL) {
52188 |         return MA_INVALID_ARGS;
52189 |     }
52190 |
52191 |     result = ma_spatializer_get_heap_layout(pConfig, &heapLayout);
52192 |     if (result != MA_SUCCESS) {
52193 |         return result;
52194 |     }
52195 |
52196 |     pSpatializer->pHeap = pHeap;
52197 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
52198 |
52199 |     pSpatializer->channelsIn          = pConfig->channelsIn;
52200 |     pSpatializer->channelsOut         = pConfig->channelsOut;
52201 |     pSpatializer->attenuationModel    = pConfig->attenuationModel;
52202 |     pSpatializer->positioning         = pConfig->positioning;
52203 |     pSpatializer->handedness          = pConfig->handedness;
52204 |     pSpatializer->minGain             = pConfig->minGain;
52205 |     pSpatializer->maxGain             = pConfig->maxGain;
52206 |     pSpatializer->minDistance         = pConfig->minDistance;
52207 |     pSpatializer->maxDistance         = pConfig->maxDistance;
52208 |     pSpatializer->rolloff             = pConfig->rolloff;
52209 |     pSpatializer->coneInnerAngleInRadians = pConfig->coneInnerAngleInRadians;
52210 |     pSpatializer->coneOuterAngleInRadians = pConfig->coneOuterAngleInRadians;
52211 |     pSpatializer->coneOuterGain       = pConfig->coneOuterGain;
52212 |     pSpatializer->dopplerFactor       = pConfig->dopplerFactor;
52213 |     pSpatializer->minSpatializationChannelGain = pConfig->minSpatializationChannelGain;
52214 |     pSpatializer->directionalAttenuationFactor = pConfig->directionalAttenuationFactor;
52215 |     pSpatializer->gainSmoothTimeInFrames = pConfig->gainSmoothTimeInFrames;
52216 |     ma_atomic_vec3f_init(&pSpatializer->position, ma_vec3f_init_3f(0, 0, 0));
52217 |     ma_atomic_vec3f_init(&pSpatializer->direction, ma_vec3f_init_3f(0, 0, -1));
52218 |     ma_atomic_vec3f_init(&pSpatializer->velocity, ma_vec3f_init_3f(0, 0, 0));
52219 |     pSpatializer->dopplerPitch        = 1;
52220 |
52221 |     /* Swap the forward direction if we're left handed (it was initialized based on right handed).
> */
52222 |     if (pSpatializer->handedness == ma_handedness_left) {
52223 |         ma_vec3f negDir = ma_vec3f_neg(ma_spatializer_get_direction(pSpatializer));
52224 |         ma_spatializer_set_direction(pSpatializer, negDir.x, negDir.y, negDir.z);
52225 |     }
52226 |
52227 |     /* Channel map. This will be on the heap. */
52228 |     if (pConfig->pChannelMapIn != NULL) {
52229 |         pSpatializer->pChannelMapIn = (ma_channel*)ma_offset_ptr(pHeap,
> heapLayout.channelMapInOffset);
52230 |         ma_channel_map_copy_or_default(pSpatializer->pChannelMapIn, pSpatializer->channelsIn,
> pConfig->pChannelMapIn, pSpatializer->channelsIn);
52231 |     }

```



```

52232 |
52233 |     /* New channel gains for output channels. */
52234 |     pSpatializer->NewChannelGainsOut = (float*)ma_offset_ptr(pHeap,
> heapLayout.newChannelGainsOffset);
52235 |
52236 |     /* Gainer. */
52237 |     gainerConfig = ma_spatializer_gainer_config_init(pConfig);
52238 |
52239 |     result = ma_gainer_init_preallocated(&gainerConfig, ma_offset_ptr(pHeap,
> heapLayout.gainerOffset), &pSpatializer->gainer);
52240 |     if (result != MA_SUCCESS) {
52241 |         return result; /* Failed to initialize the gainer. */
52242 |     }
52243 |
52244 |     return MA_SUCCESS;
52245 | }
52246 |
52247 | MA_API ma_result ma_spatializer_init(const ma_spatializer_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_spatializer* pSpatializer)
52248 | {
52249 |     ma_result result;
52250 |     size_t heapSizeInBytes;
52251 |     void* pHeap;
52252 |
52253 |     /* We'll need a heap allocation to retrieve the size. */
52254 |     result = ma_spatializer_get_heap_size(pConfig, &heapSizeInBytes);
52255 |     if (result != MA_SUCCESS) {
52256 |         return result;
52257 |     }
52258 |
52259 |     if (heapSizeInBytes > 0) {
52260 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
52261 |         if (pHeap == NULL) {
52262 |             return MA_OUT_OF_MEMORY;
52263 |         }
52264 |     } else {
52265 |         pHeap = NULL;
52266 |     }
52267 |
52268 |     result = ma_spatializer_init_preallocated(pConfig, pHeap, pSpatializer);
52269 |     if (result != MA_SUCCESS) {
52270 |         ma_free(pHeap, pAllocationCallbacks);
52271 |         return result;
52272 |     }
52273 |
52274 |     pSpatializer->ownsHeap = MA_TRUE;
52275 |     return MA_SUCCESS;
52276 | }
52277 |
52278 | MA_API void ma_spatializer_uninit(ma_spatializer* pSpatializer, const ma_allocation_callbacks*
> pAllocationCallbacks)
52279 | {
52280 |     if (pSpatializer == NULL) {
52281 |         return;
52282 |     }
52283 |
52284 |     ma_gainer_uninit(&pSpatializer->gainer, pAllocationCallbacks);
52285 |
52286 |     if (pSpatializer->ownsHeap) {
52287 |         ma_free(pSpatializer->pHeap, pAllocationCallbacks);
52288 |     }
52289 | }
52290 |
52291 | static float ma_calculate_angular_gain(ma_vec3f dirA, ma_vec3f dirB, float coneInnerAngleInRadians,
> float coneOuterAngleInRadians, float coneOuterGain)
52292 | {
52293 |     /*
52294 |     Angular attenuation.
52295 |
52296 |     Unlike distance gain, the math for this is not specified by the OpenAL spec so we'll just go
> ahead and figure
52297 |     this out for ourselves at the expense of possibly being inconsistent with other implementations.
52298 |
52299 |     To do cone attenuation, I'm just using the same math that we'd use to implement a basic
> spotlight in OpenGL. We
52300 |     just need to get the direction from the source to the listener and then do a dot product against
> that and the
52301 |     direction of the spotlight. Then we just compare that dot product against the cosine of the
> inner and outer

```

```

52302 |     angles. If the dot product is greater than the outer angle, we just use coneOuterGain. If it's
> less than
52303 |     the inner angle, we just use a gain of 1. Otherwise we linearly interpolate between 1 and
> coneOuterGain.
52304 |     */
52305 |     if (coneInnerAngleInRadians < 6.283185f) {
52306 |         float angularGain = 1;
52307 |         float cutoffInner = (float)ma_cosd(coneInnerAngleInRadians*0.5f);
52308 |         float cutoffOuter = (float)ma_cosd(coneOuterAngleInRadians*0.5f);
52309 |         float d;
52310 |
52311 |         d = ma_vec3f_dot(dirA, dirB);
52312 |
52313 |         if (d > cutoffInner) {
52314 |             /* It's inside the inner angle. */
52315 |             angularGain = 1;
52316 |         } else {
52317 |             /* It's outside the inner angle. */
52318 |             if (d > cutoffOuter) {
52319 |                 /* It's between the inner and outer angle. We need to linearly interpolate between 1
> and coneOuterGain. */
52320 |                 angularGain = ma_mix_f32(coneOuterGain, 1, (d - cutoffOuter) / (cutoffInner -
> cutoffOuter));
52321 |             } else {
52322 |                 /* It's outside the outer angle. */
52323 |                 angularGain = coneOuterGain;
52324 |             }
52325 |         }
52326 |
52327 |         /*printf("d = %f; cutoffInner = %f; cutoffOuter = %f; angularGain = %f\n", d, cutoffInner,
> cutoffOuter, angularGain);*/
52328 |         return angularGain;
52329 |     } else {
52330 |         /* Inner angle is 360 degrees so no need to do any attenuation. */
52331 |         return 1;
52332 |     }
52333 | }
52334 |
52335 | MA_API ma_result ma_spatializer_process_pcm_frames(ma_spatializer* pSpatializer,
> ma_spatializer_listener* pListener, void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount)
52336 | {
52337 |     ma_channel* pChannelMapIn;
52338 |     ma_channel* pChannelMapOut;
52339 |
52340 |     if (pSpatializer == NULL || pListener == NULL) {
52341 |         return MA_INVALID_ARGS;
52342 |     }
52343 |
52344 |     pChannelMapIn = pSpatializer->pChannelMapIn;
52345 |     pChannelMapOut = pListener->config.pChannelMapOut;
52346 |
52347 |     /* If we're not spatializing we need to run an optimized path. */
52348 |     if (ma_atomic_load_i32(&pSpatializer->attenuationModel) == ma_attenuation_model_none) {
52349 |         if (ma_spatializer_listener_is_enabled(pListener)) {
52350 |             /* No attenuation is required, but we'll need to do some channel conversion. */
52351 |             if (pSpatializer->channelsIn == pSpatializer->channelsOut) {
52352 |                 ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, ma_format_f32,
> pSpatializer->channelsIn);
52353 |             } else {
52354 |                 ma_channel_map_apply_f32((float*)pFramesOut, pChannelMapOut,
> pSpatializer->channelsOut, (const float*)pFramesIn, pChannelMapIn, pSpatializer->channelsIn,
> frameCount, ma_channel_mix_mode_rectangular, ma_mono_expansion_mode_default); /* Safe casts to
> float* because f32 is the only supported format. */
52355 |             }
52356 |         } else {
52357 |             /* The listener is disabled. Output silence. */
52358 |             ma_silence_pcm_frames(pFramesOut, frameCount, ma_format_f32, pSpatializer->channelsOut);
52359 |         }
52360 |
52361 |         /*
52362 |         We're not doing attenuation so don't bother with doppler for now. I'm not sure if this is
52363 |         the correct thinking so might need to review this later.
52364 |         */
52365 |         pSpatializer->dopplerPitch = 1;
52366 |     } else {
52367 |         /*
52368 |         Let's first determine which listener the sound is closest to. Need to keep in mind that we
52369 |         might not have a world or any listeners, in which case we just spatializer based on the
52370 |         listener being positioned at the origin (0, 0, 0).

```

```

52371 |      */
52372 |      ma_vec3f relativePosNormalized;
52373 |      ma_vec3f relativePos;    /* The position relative to the listener. */
52374 |      ma_vec3f relativeDir;    /* The direction of the sound, relative to the listener. */
52375 |      ma_vec3f listenerVel;    /* The velocity of the listener. For doppler pitch calculation. */
52376 |      float speedOfSound;
52377 |      float distance = 0;
52378 |      float gain = 1;
52379 |      ma_uint32 iChannel;
52380 |      const ma_uint32 channelsOut = pSpatializer->channelsOut;
52381 |      const ma_uint32 channelsIn  = pSpatializer->channelsIn;
52382 |      float minDistance = ma_spatializer_get_min_distance(pSpatializer);
52383 |      float maxDistance = ma_spatializer_get_max_distance(pSpatializer);
52384 |      float rolloff = ma_spatializer_get_rolloff(pSpatializer);
52385 |      float dopplerFactor = ma_spatializer_get_doppler_factor(pSpatializer);
52386 |
52387 |      /*
52388 |      We'll need the listener velocity for doppler pitch calculations. The speed of sound is
52389 |      defined by the listener, so we'll grab that here too.
52390 |      */
52391 |      listenerVel = ma_spatializer_listener_get_velocity(pListener);
52392 |      speedOfSound = pListener->config.speedOfSound;
52393 |
52394 |      if (ma_spatializer_get_positioning(pSpatializer) == ma_positioning_relative) {
52395 |          relativePos = ma_spatializer_get_position(pSpatializer);
52396 |          relativeDir = ma_spatializer_get_direction(pSpatializer);
52397 |      } else {
52398 |          /*
52399 |          We're using absolute positioning. We need to transform the sound's position and
52400 |          direction so that it's relative to listener. Later on we'll use this for determining
52401 |          the factors to apply to each channel to apply the panning effect.
52402 |          */
52403 |          ma_spatializer_get_relative_position_and_direction(pSpatializer, pListener,
> &relativePos, &relativeDir);
52404 |      }
52405 |
52406 |      distance = ma_vec3f_len(relativePos);
52407 |
52408 |      /* We've gathered the data, so now we can apply some spatialization. */
52409 |      switch (ma_spatializer_get_attenuation_model(pSpatializer)) {
52410 |          case ma_attenuation_model_inverse:
52411 |          {
52412 |              gain = ma_attenuation_inverse(distance, minDistance, maxDistance, rolloff);
52413 |          } break;
52414 |          case ma_attenuation_model_linear:
52415 |          {
52416 |              gain = ma_attenuation_linear(distance, minDistance, maxDistance, rolloff);
52417 |          } break;
52418 |          case ma_attenuation_model_exponential:
52419 |          {
52420 |              gain = ma_attenuation_exponential(distance, minDistance, maxDistance, rolloff);
52421 |          } break;
52422 |          case ma_attenuation_model_none:
52423 |          default:
52424 |          {
52425 |              gain = 1;
52426 |          } break;
52427 |      }
52428 |
52429 |      /* Normalize the position. */
52430 |      if (distance > 0.001f) {
52431 |          float distanceInv = 1/distance;
52432 |          relativePosNormalized = relativePos;
52433 |          relativePosNormalized.x *= distanceInv;
52434 |          relativePosNormalized.y *= distanceInv;
52435 |          relativePosNormalized.z *= distanceInv;
52436 |      } else {
52437 |          distance = 0;
52438 |          relativePosNormalized = ma_vec3f_init_3f(0, 0, 0);
52439 |      }
52440 |
52441 |      /*
52442 |      Angular attenuation.
52443 |
52444 |      Unlike distance gain, the math for this is not specified by the OpenAL spec so we'll just go
> ahead and figure
52445 |      this out for ourselves at the expense of possibly being inconsistent with other
> implementations.
52446 |

```

```

52447 |         To do cone attenuation, I'm just using the same math that we'd use to implement a basic
> spotlight in OpenGL. We
52448 |         just need to get the direction from the source to the listener and then do a dot product
> against that and the
52449 |         direction of the spotlight. Then we just compare that dot product against the cosine of the
> inner and outer
52450 |         angles. If the dot product is greater than the outer angle, we just use coneOuterGain. If
> it's less than
52451 |         the inner angle, we just use a gain of 1. Otherwise we linearly interpolate between 1 and
> coneOuterGain.
52452 |         */
52453 |         if (distance > 0) {
52454 |             /* Source angular gain. */
52455 |             float spatializerConeInnerAngle;
52456 |             float spatializerConeOuterAngle;
52457 |             float spatializerConeOuterGain;
52458 |             ma_spatializer_get_cone(pSpatializer, &spatializerConeInnerAngle,
> &spatializerConeOuterAngle, &spatializerConeOuterGain);
52459 |
52460 |             gain *= ma_calculate_angular_gain(relativeDir, ma_vec3f_neg(relativePosNormalized),
> spatializerConeInnerAngle, spatializerConeOuterAngle, spatializerConeOuterGain);
52461 |
52462 |             /*
52463 |             We're supporting angular gain on the listener as well for those who want to reduce the
> volume of sounds that
52464 |             are positioned behind the listener. On default settings, this will have no effect.
52465 |             */
52466 |             if (pListener != NULL && pListener->config.coneInnerAngleInRadians < 6.283185f) {
52467 |                 ma_vec3f listenerDirection;
52468 |                 float listenerInnerAngle;
52469 |                 float listenerOuterAngle;
52470 |                 float listenerOuterGain;
52471 |
52472 |                 if (pListener->config.handedness == ma_handedness_right) {
52473 |                     listenerDirection = ma_vec3f_init_3f(0, 0, -1);
52474 |                 } else {
52475 |                     listenerDirection = ma_vec3f_init_3f(0, 0, +1);
52476 |                 }
52477 |
52478 |                 listenerInnerAngle = pListener->config.coneInnerAngleInRadians;
52479 |                 listenerOuterAngle = pListener->config.coneOuterAngleInRadians;
52480 |                 listenerOuterGain = pListener->config.coneOuterGain;
52481 |
52482 |                 gain *= ma_calculate_angular_gain(listenerDirection, relativePosNormalized,
> listenerInnerAngle, listenerOuterAngle, listenerOuterGain);
52483 |             }
52484 |             } else {
52485 |                 /* The sound is right on top of the listener. Don't do any angular attenuation. */
52486 |             }
52487 |
52488 |             /* Clamp the gain. */
52489 |             gain = ma_clamp(gain, ma_spatializer_get_min_gain(pSpatializer),
> ma_spatializer_get_max_gain(pSpatializer));
52490 |
52491 |             /*
52492 |             The gain needs to be applied per-channel here. The spatialization code below will be
> changing the per-channel
52493 |             gains which will then eventually be passed into the gainer which will deal with smoothing
> the gain transitions
52494 |             to avoid harsh changes in gain.
52495 |             */
52496 |             for (iChannel = 0; iChannel < channelsOut; iChannel += 1) {
52497 |                 pSpatializer->pNewChannelGainsOut[iChannel] = gain;
52498 |             }
52499 |
52500 |             /*
52501 |             Convert to our output channel count. If the listener is disabled we just output silence
> here. We cannot ignore
52502 |             the whole section of code here because we need to update some internal spatialization state.
52503 |             */
52504 |             if (ma_spatializer_listener_is_enabled(pListener)) {
52505 |                 ma_channel_map_apply_f32((float*)pFramesOut, pChannelMapOut, channelsOut, (const
> float*)pFramesIn, pChannelMapIn, channelsIn, frameCount, ma_channel_mix_mode_rectangular,
> ma_mono_expansion_mode_default);
52506 |             } else {
52507 |                 ma_silence_pcm_frames(pFramesOut, frameCount, ma_format_f32, pSpatializer->channelsOut);
52508 |             }
52509 |
52510 |

```

```

52511 |
52512 |         /*
52513 |         Panning. This is where we'll apply the gain and convert to the output channel count. We have
> an optimized path for
52514 |         when we're converting to a mono stream. In that case we don't really need to do any panning
> - we just apply the
52515 |         gain to the final output.
52516 |         */
52517 |         /*printf("distance=%f; gain=%f\n", distance, gain);*/
52518 |
52519 |         /* We must have a valid channel map here to ensure we spatialize properly. */
52520 |         MA_ASSERT(pChannelMapOut != NULL);
52521 |
52522 |         /*
52523 |         We're not converting to mono so we'll want to apply some panning. This is where the feeling
> of something being
52524 |         to the left, right, infront or behind the listener is calculated. I'm just using a basic
> model here. Note that
52525 |         the code below is not based on any specific algorithm. I'm just implementing this off the
> top of my head and
52526 |         seeing how it goes. There might be better ways to do this.
52527 |
52528 |         To determine the direction of the sound relative to a speaker I'm using dot products. Each
> speaker
52529 |         is given a
52530 |         direction. For example, the left channel in a stereo system will be -1 on the X axis and the
> right channel will
52531 |         be +1 on the X axis. A dot product is performed against the direction vector of the channel
> and the normalized
52532 |         position of the sound.
52533 |         */
52534 |         /*
52535 |         Calculate our per-channel gains. We do this based on the normalized relative position of the
> sound and it's
52536 |         relation to the direction of the channel.
52537 |         */
52538 |         if (distance > 0) {
52539 |             ma_vec3f unitPos = relativePos;
52540 |             float distanceInv = 1/distance;
52541 |             unitPos.x *= distanceInv;
52542 |             unitPos.y *= distanceInv;
52543 |             unitPos.z *= distanceInv;
52544 |
52545 |             for (iChannel = 0; iChannel < channelsOut; iChannel += 1) {
52546 |                 ma_channel channelOut;
52547 |                 float d;
52548 |                 float dMin;
52549 |
52550 |                 channelOut = ma_channel_map_get_channel(pChannelMapOut, channelsOut, iChannel);
52551 |                 if (ma_is_spatial_channel_position(channelOut)) {
52552 |                     d = ma_mix_f32_fast(1, ma_vec3f_dot(unitPos,
> ma_get_channel_direction(channelOut)),
> ma_spatializer_get_directional_attenuation_factor(pSpatializer));
52553 |                 } else {
52554 |                     d = 1; /* It's not a spatial channel so there's no real notion of direction. */
52555 |                 }
52556 |
52557 |                 /*
52558 |                 In my testing, if the panning effect is too aggressive it makes spatialization feel
> uncomfortable.
52559 |                 The "dMin" variable below is used to control the aggressiveness of the panning
> effect. When set to
52560 |                 0, panning will be most extreme and any sounds that are positioned on the opposite
> side of the
52561 |                 speaker will be completely silent from that speaker. Not only does this feel
> uncomfortable, it
52562 |                 doesn't even remotely represent the real world at all because sounds that come from
> your right side
52563 |                 are still clearly audible from your left side. Setting "dMin" to 1 will result in no
> panning at
52564 |                 all, which is also not ideal. By setting it to something greater than 0, the
> spatialization effect
52565 |                 becomes much less dramatic and a lot more bearable.
52566 |
52567 |                 Summary: 0 = more extreme panning; 1 = no panning.
52568 |                 */
52569 |                 dMin = pSpatializer->minSpatializationChannelGain;
52570 |
52571 |                 /*

```

```

52572 |           At this point, "d" will be positive if the sound is on the same side as the channel
> and negative if
52573 |           it's on the opposite side. It will be in the range of -1..1. There's two ways I can
> think of to
52574 |           calculate a panning value. The first is to simply convert it to 0..1, however this
> has a problem
52575 |           which I'm not entirely happy with. Considering a stereo system, when a sound is
> positioned right
52576 |           in front of the listener it'll result in each speaker getting a gain of 0.5. I don't
> know if I like
52577 |           the idea of having a scaling factor of 0.5 being applied to a sound when it's
> sitting right in front
52578 |           of the listener. I would intuitively expect that to be played at full volume, or
> close to it.
52579 |
52580 |           The second idea I think of is to only apply a reduction in gain when the sound is on
> the opposite
52581 |           side of the speaker. That is, reduce the gain only when the dot product is negative.
> The problem
52582 |           with this is that there will not be any attenuation as the sound sweeps around the
> 180 degrees
52583 |           where the dot product is positive. The idea with this option is that you leave the
> gain at 1 when
52584 |           the sound is being played on the same side as the speaker and then you just reduce
> the volume when
52585 |           the sound is on the other side.
52586 |
52587 |           The summarize, I think the first option should give a better sense of
> spatialization, but the second
52588 |           option is better for preserving the sound's power.
52589 |
52590 |           UPDATE: In my testing, I find the first option to sound better. You can feel the
> sense of space a
52591 |           bit better, but you can also hear the reduction in volume when it's right in front.
52592 |           */
52593 |           #if 1
52594 |           {
52595 |               /*
52596 |               Scale the dot product from -1..1 to 0..1. Will result in a sound directly in
> front losing power
52597 |               by being played at 0.5 gain.
52598 |               */
52599 |               d = (d + 1) * 0.5f; /* -1..1 to 0..1 */
52600 |               d = ma_max(d, dMin);
52601 |               pSpatializer->pNewChannelGainsOut[iChannel] *= d;
52602 |           }
52603 |           #else
52604 |           {
52605 |               /*
52606 |               Only reduce the volume of the sound if it's on the opposite side. This path
> keeps the volume more
52607 |               consistent, but comes at the expense of a worse sense of space and positioning.
52608 |               */
52609 |               if (d < 0) {
52610 |                   d += 1; /* Move into the positive range. */
52611 |                   d = ma_max(d, dMin);
52612 |                   channelGainsOut[iChannel] *= d;
52613 |               }
52614 |           }
52615 |           #endif
52616 |       }
52617 |   } else {
52618 |       /* Assume the sound is right on top of us. Don't do any panning. */
52619 |   }
52620 |
52621 |       /* Now we need to apply the volume to each channel. This needs to run through the gainer to
> ensure we get a smooth volume transition. */
52622 |       ma_gainer_set_gains(&pSpatializer->gainer, pSpatializer->pNewChannelGainsOut);
52623 |       ma_gainer_process_pcm_frames(&pSpatializer->gainer, pFramesOut, pFramesOut, frameCount);
52624 |
52625 |       /*
52626 |       Before leaving we'll want to update our doppler pitch so that the caller can apply some
52627 |       pitch shifting if they desire. Note that we need to negate the relative position here
52628 |       because the doppler calculation needs to be source-to-listener, but ours is listener-to-
52629 |       source.
52630 |       */
52631 |       if (dopplerFactor > 0) {
52632 |           pSpatializer->dopplerPitch =
> ma_doppler_pitch(ma_vec3f_sub(ma_spatializer_listener_get_position(pListener),

```

```

> ma_spatializer_get_position(pSpatializer)), ma_spatializer_get_velocity(pSpatializer), listenerVel,
> speedOfSound, dopplerFactor);
52633 |     } else {
52634 |         pSpatializer->dopplerPitch = 1;
52635 |     }
52636 | }
52637 |
52638 | return MA_SUCCESS;
52639 | }
52640 |
52641 | MA_API ma_result ma_spatializer_set_master_volume(ma_spatializer* pSpatializer, float volume)
52642 | {
52643 |     if (pSpatializer == NULL) {
52644 |         return MA_INVALID_ARGS;
52645 |     }
52646 |
52647 |     return ma_gainer_set_master_volume(&pSpatializer->gainer, volume);
52648 | }
52649 |
52650 | MA_API ma_result ma_spatializer_get_master_volume(const ma_spatializer* pSpatializer, float*
> pVolume)
52651 | {
52652 |     if (pSpatializer == NULL) {
52653 |         return MA_INVALID_ARGS;
52654 |     }
52655 |
52656 |     return ma_gainer_get_master_volume(&pSpatializer->gainer, pVolume);
52657 | }
52658 |
52659 | MA_API ma_uint32 ma_spatializer_get_input_channels(const ma_spatializer* pSpatializer)
52660 | {
52661 |     if (pSpatializer == NULL) {
52662 |         return 0;
52663 |     }
52664 |
52665 |     return pSpatializer->channelsIn;
52666 | }
52667 |
52668 | MA_API ma_uint32 ma_spatializer_get_output_channels(const ma_spatializer* pSpatializer)
52669 | {
52670 |     if (pSpatializer == NULL) {
52671 |         return 0;
52672 |     }
52673 |
52674 |     return pSpatializer->channelsOut;
52675 | }
52676 |
52677 | MA_API void ma_spatializer_set_attenuation_model(ma_spatializer* pSpatializer, ma_attenuation_model
> attenuationModel)
52678 | {
52679 |     if (pSpatializer == NULL) {
52680 |         return;
52681 |     }
52682 |
52683 |     ma_atomic_exchange_i32(&pSpatializer->attenuationModel, attenuationModel);
52684 | }
52685 |
52686 | MA_API ma_attenuation_model ma_spatializer_get_attenuation_model(const ma_spatializer* pSpatializer)
52687 | {
52688 |     if (pSpatializer == NULL) {
52689 |         return ma_attenuation_model_none;
52690 |     }
52691 |
52692 |     return (ma_attenuation_model)ma_atomic_load_i32(&pSpatializer->attenuationModel);
52693 | }
52694 |
52695 | MA_API void ma_spatializer_set_positioning(ma_spatializer* pSpatializer, ma_positioning positioning)
52696 | {
52697 |     if (pSpatializer == NULL) {
52698 |         return;
52699 |     }
52700 |
52701 |     ma_atomic_exchange_i32(&pSpatializer->positioning, positioning);
52702 | }
52703 |
52704 | MA_API ma_positioning ma_spatializer_get_positioning(const ma_spatializer* pSpatializer)
52705 | {
52706 |     if (pSpatializer == NULL) {
52707 |         return ma_positioning_absolute;

```



```
52708 |     }
52709 |
52710 |     return (ma_positioning)ma_atomic_load_i32(&pSpatializer->positioning);
52711 | }
52712 |
52713 | MA_API void ma_spatializer_set_rolloff(ma_spatializer* pSpatializer, float rolloff)
52714 | {
52715 |     if (pSpatializer == NULL) {
52716 |         return;
52717 |     }
52718 |
52719 |     ma_atomic_exchange_f32(&pSpatializer->rolloff, rolloff);
52720 | }
52721 |
52722 | MA_API float ma_spatializer_get_rolloff(const ma_spatializer* pSpatializer)
52723 | {
52724 |     if (pSpatializer == NULL) {
52725 |         return 0;
52726 |     }
52727 |
52728 |     return ma_atomic_load_f32(&pSpatializer->rolloff);
52729 | }
52730 |
52731 | MA_API void ma_spatializer_set_min_gain(ma_spatializer* pSpatializer, float minGain)
52732 | {
52733 |     if (pSpatializer == NULL) {
52734 |         return;
52735 |     }
52736 |
52737 |     ma_atomic_exchange_f32(&pSpatializer->minGain, minGain);
52738 | }
52739 |
52740 | MA_API float ma_spatializer_get_min_gain(const ma_spatializer* pSpatializer)
52741 | {
52742 |     if (pSpatializer == NULL) {
52743 |         return 0;
52744 |     }
52745 |
52746 |     return ma_atomic_load_f32(&pSpatializer->minGain);
52747 | }
52748 |
52749 | MA_API void ma_spatializer_set_max_gain(ma_spatializer* pSpatializer, float maxGain)
52750 | {
52751 |     if (pSpatializer == NULL) {
52752 |         return;
52753 |     }
52754 |
52755 |     ma_atomic_exchange_f32(&pSpatializer->maxGain, maxGain);
52756 | }
52757 |
52758 | MA_API float ma_spatializer_get_max_gain(const ma_spatializer* pSpatializer)
52759 | {
52760 |     if (pSpatializer == NULL) {
52761 |         return 0;
52762 |     }
52763 |
52764 |     return ma_atomic_load_f32(&pSpatializer->maxGain);
52765 | }
52766 |
52767 | MA_API void ma_spatializer_set_min_distance(ma_spatializer* pSpatializer, float minDistance)
52768 | {
52769 |     if (pSpatializer == NULL) {
52770 |         return;
52771 |     }
52772 |
52773 |     ma_atomic_exchange_f32(&pSpatializer->minDistance, minDistance);
52774 | }
52775 |
52776 | MA_API float ma_spatializer_get_min_distance(const ma_spatializer* pSpatializer)
52777 | {
52778 |     if (pSpatializer == NULL) {
52779 |         return 0;
52780 |     }
52781 |
52782 |     return ma_atomic_load_f32(&pSpatializer->minDistance);
52783 | }
52784 |
52785 | MA_API void ma_spatializer_set_max_distance(ma_spatializer* pSpatializer, float maxDistance)
52786 | {
```

```
52787 |     if (pSpatializer == NULL) {
52788 |         return;
52789 |     }
52790 |
52791 |     ma_atomic_exchange_f32(&pSpatializer->maxDistance, maxDistance);
52792 | }
52793 |
52794 | MA_API float ma_spatializer_get_max_distance(const ma_spatializer* pSpatializer)
52795 | {
52796 |     if (pSpatializer == NULL) {
52797 |         return 0;
52798 |     }
52799 |
52800 |     return ma_atomic_load_f32(&pSpatializer->maxDistance);
52801 | }
52802 |
52803 | MA_API void ma_spatializer_set_cone(ma_spatializer* pSpatializer, float innerAngleInRadians, float
> outerAngleInRadians, float outerGain)
52804 | {
52805 |     if (pSpatializer == NULL) {
52806 |         return;
52807 |     }
52808 |
52809 |     ma_atomic_exchange_f32(&pSpatializer->coneInnerAngleInRadians, innerAngleInRadians);
52810 |     ma_atomic_exchange_f32(&pSpatializer->coneOuterAngleInRadians, outerAngleInRadians);
52811 |     ma_atomic_exchange_f32(&pSpatializer->coneOuterGain, outerGain);
52812 | }
52813 |
52814 | MA_API void ma_spatializer_get_cone(const ma_spatializer* pSpatializer, float* pInnerAngleInRadians,
> float* pOuterAngleInRadians, float* pOuterGain)
52815 | {
52816 |     if (pSpatializer == NULL) {
52817 |         return;
52818 |     }
52819 |
52820 |     if (pInnerAngleInRadians != NULL) {
52821 |         *pInnerAngleInRadians = ma_atomic_load_f32(&pSpatializer->coneInnerAngleInRadians);
52822 |     }
52823 |
52824 |     if (pOuterAngleInRadians != NULL) {
52825 |         *pOuterAngleInRadians = ma_atomic_load_f32(&pSpatializer->coneOuterAngleInRadians);
52826 |     }
52827 |
52828 |     if (pOuterGain != NULL) {
52829 |         *pOuterGain = ma_atomic_load_f32(&pSpatializer->coneOuterGain);
52830 |     }
52831 | }
52832 |
52833 | MA_API void ma_spatializer_set_doppler_factor(ma_spatializer* pSpatializer, float dopplerFactor)
52834 | {
52835 |     if (pSpatializer == NULL) {
52836 |         return;
52837 |     }
52838 |
52839 |     ma_atomic_exchange_f32(&pSpatializer->dopplerFactor, dopplerFactor);
52840 | }
52841 |
52842 | MA_API float ma_spatializer_get_doppler_factor(const ma_spatializer* pSpatializer)
52843 | {
52844 |     if (pSpatializer == NULL) {
52845 |         return 1;
52846 |     }
52847 |
52848 |     return ma_atomic_load_f32(&pSpatializer->dopplerFactor);
52849 | }
52850 |
52851 | MA_API void ma_spatializer_set_directional_attenuation_factor(ma_spatializer* pSpatializer, float
> directionalAttenuationFactor)
52852 | {
52853 |     if (pSpatializer == NULL) {
52854 |         return;
52855 |     }
52856 |
52857 |     ma_atomic_exchange_f32(&pSpatializer->directionalAttenuationFactor,
> directionalAttenuationFactor);
52858 | }
52859 |
52860 | MA_API float ma_spatializer_get_directional_attenuation_factor(const ma_spatializer* pSpatializer)
52861 | {
```

```

52862 |     if (pSpatializer == NULL) {
52863 |         return 1;
52864 |     }
52865 |
52866 |     return ma_atomic_load_f32(&pSpatializer->directionalAttenuationFactor);
52867 | }
52868 |
52869 | MA_API void ma_spatializer_set_position(ma_spatializer* pSpatializer, float x, float y, float z)
52870 | {
52871 |     if (pSpatializer == NULL) {
52872 |         return;
52873 |     }
52874 |
52875 |     ma_atomic_vec3f_set(&pSpatializer->position, ma_vec3f_init_3f(x, y, z));
52876 | }
52877 |
52878 | MA_API ma_vec3f ma_spatializer_get_position(const ma_spatializer* pSpatializer)
52879 | {
52880 |     if (pSpatializer == NULL) {
52881 |         return ma_vec3f_init_3f(0, 0, 0);
52882 |     }
52883 |
52884 |     return ma_atomic_vec3f_get((ma_atomic_vec3f*)&pSpatializer->position); /* Naughty const-cast.
> It's just for atomically loading the vec3 which should be safe. */
52885 | }
52886 |
52887 | MA_API void ma_spatializer_set_direction(ma_spatializer* pSpatializer, float x, float y, float z)
52888 | {
52889 |     if (pSpatializer == NULL) {
52890 |         return;
52891 |     }
52892 |
52893 |     ma_atomic_vec3f_set(&pSpatializer->direction, ma_vec3f_init_3f(x, y, z));
52894 | }
52895 |
52896 | MA_API ma_vec3f ma_spatializer_get_direction(const ma_spatializer* pSpatializer)
52897 | {
52898 |     if (pSpatializer == NULL) {
52899 |         return ma_vec3f_init_3f(0, 0, -1);
52900 |     }
52901 |
52902 |     return ma_atomic_vec3f_get((ma_atomic_vec3f*)&pSpatializer->direction); /* Naughty const-cast.
> It's just for atomically loading the vec3 which should be safe. */
52903 | }
52904 |
52905 | MA_API void ma_spatializer_set_velocity(ma_spatializer* pSpatializer, float x, float y, float z)
52906 | {
52907 |     if (pSpatializer == NULL) {
52908 |         return;
52909 |     }
52910 |
52911 |     ma_atomic_vec3f_set(&pSpatializer->velocity, ma_vec3f_init_3f(x, y, z));
52912 | }
52913 |
52914 | MA_API ma_vec3f ma_spatializer_get_velocity(const ma_spatializer* pSpatializer)
52915 | {
52916 |     if (pSpatializer == NULL) {
52917 |         return ma_vec3f_init_3f(0, 0, 0);
52918 |     }
52919 |
52920 |     return ma_atomic_vec3f_get((ma_atomic_vec3f*)&pSpatializer->velocity); /* Naughty const-cast.
> It's just for atomically loading the vec3 which should be safe. */
52921 | }
52922 |
52923 | MA_API void ma_spatializer_get_relative_position_and_direction(const ma_spatializer* pSpatializer,
> const ma_spatializer_listener* pListener, ma_vec3f* pRelativePos, ma_vec3f* pRelativeDir)
52924 | {
52925 |     if (pRelativePos != NULL) {
52926 |         pRelativePos->x = 0;
52927 |         pRelativePos->y = 0;
52928 |         pRelativePos->z = 0;
52929 |     }
52930 |
52931 |     if (pRelativeDir != NULL) {
52932 |         pRelativeDir->x = 0;
52933 |         pRelativeDir->y = 0;
52934 |         pRelativeDir->z = -1;
52935 |     }
52936 | }

```

```

52937 |     if (pSpatializer == NULL) {
52938 |         return;
52939 |     }
52940 |
52941 |     if (pListener == NULL || ma_spatializer_get_positioning(pSpatializer) ==
> ma_positioning_relative) {
52942 |         /* There's no listener or we're using relative positioning. */
52943 |         if (pRelativePos != NULL) {
52944 |             *pRelativePos = ma_spatializer_get_position(pSpatializer);
52945 |         }
52946 |         if (pRelativeDir != NULL) {
52947 |             *pRelativeDir = ma_spatializer_get_direction(pSpatializer);
52948 |         }
52949 |     } else {
52950 |         ma_vec3f spatializerPosition;
52951 |         ma_vec3f spatializerDirection;
52952 |         ma_vec3f listenerPosition;
52953 |         ma_vec3f listenerDirection;
52954 |         ma_vec3f v;
52955 |         ma_vec3f axisX;
52956 |         ma_vec3f axisY;
52957 |         ma_vec3f axisZ;
52958 |         float m[4][4];
52959 |
52960 |         spatializerPosition = ma_spatializer_get_position(pSpatializer);
52961 |         spatializerDirection = ma_spatializer_get_direction(pSpatializer);
52962 |         listenerPosition = ma_spatializer_listener_get_position(pListener);
52963 |         listenerDirection = ma_spatializer_listener_get_direction(pListener);
52964 |
52965 |         /*
52966 |         We need to calculate the right vector from our forward and up vectors. This is done with
52967 |         a cross product.
52968 |         */
52969 |         axisZ = ma_vec3f_normalize(listenerDirection); /*
> Normalization required here because we can't trust the caller. */
52970 |         axisX = ma_vec3f_normalize(ma_vec3f_cross(axisZ, pListener->config.worldUp)); /*
> Normalization required here because the world up vector may not be perpendicular with the forward
> vector. */
52971 |
52972 |         /*
52973 |         The calculation of axisX above can result in a zero-length vector if the listener is
52974 |         looking straight up on the Y axis. We'll need to fall back to a +X in this case so that
52975 |         the calculations below don't fall apart. This is where a quaternion based listener and
52976 |         sound orientation would come in handy.
52977 |         */
52978 |         if (ma_vec3f_len2(axisX) == 0) {
52979 |             axisX = ma_vec3f_init_3f(1, 0, 0);
52980 |         }
52981 |
52982 |         axisY = ma_vec3f_cross(axisX, axisZ); /* No
> normalization is required here because axisX and axisZ are unit length and perpendicular. */
52983 |
52984 |         /*
52985 |         We need to swap the X axis if we're left handed because otherwise the cross product above
52986 |         will have resulted in it pointing in the wrong direction (right handed was assumed in the
52987 |         cross products above).
52988 |         */
52989 |         if (pListener->config.handedness == ma_handedness_left) {
52990 |             axisX = ma_vec3f_neg(axisX);
52991 |         }
52992 |
52993 |         /* Lookat. */
52994 |         m[0][0] = axisX.x; m[1][0] = axisX.y; m[2][0] = axisX.z; m[3][0] = -ma_vec3f_dot(axisX,
> listenerPosition);
52995 |         m[0][1] = axisY.x; m[1][1] = axisY.y; m[2][1] = axisY.z; m[3][1] = -ma_vec3f_dot(axisY,
> listenerPosition);
52996 |         m[0][2] = -axisZ.x; m[1][2] = -axisZ.y; m[2][2] = -axisZ.z; m[3][2] =
> -ma_vec3f_dot(ma_vec3f_neg(axisZ), listenerPosition);
52997 |         m[0][3] = 0; m[1][3] = 0; m[2][3] = 0; m[3][3] = 1;
52998 |
52999 |         /*
53000 |         Multiply the lookat matrix by the spatializer position to transform it to listener
53001 |         space. This allows calculations to work based on the sound being relative to the
53002 |         origin which makes things simpler.
53003 |         */
53004 |         if (pRelativePos != NULL) {
53005 |             v = spatializerPosition;
53006 |             pRelativePos->x = m[0][0] * v.x + m[1][0] * v.y + m[2][0] * v.z + m[3][0] * 1;
53007 |             pRelativePos->y = m[0][1] * v.x + m[1][1] * v.y + m[2][1] * v.z + m[3][1] * 1;

```

```

53008 |         pRelativePos->z = m[0][2] * v.x + m[1][2] * v.y + m[2][2] * v.z + m[3][2] * 1;
53009 |     }
53010 |
53011 |     /*
53012 |     The direction of the sound needs to also be transformed so that it's relative to the
53013 |     rotation of the listener.
53014 |     */
53015 |     if (pRelativeDir != NULL) {
53016 |         v = spatializerDirection;
53017 |         pRelativeDir->x = m[0][0] * v.x + m[1][0] * v.y + m[2][0] * v.z;
53018 |         pRelativeDir->y = m[0][1] * v.x + m[1][1] * v.y + m[2][1] * v.z;
53019 |         pRelativeDir->z = m[0][2] * v.x + m[1][2] * v.y + m[2][2] * v.z;
53020 |     }
53021 | }
53022 | }
53023 |
53024 |
53025 |
53026 |
53027 | /*****
> *****/
53028 |
53029 | Resampling
53030 |
53031 | *****/
> *****/
53032 | MA_API ma_linear_resampler_config ma_linear_resampler_config_init(ma_format format, ma_uint32
> channels, ma_uint32 sampleRateIn, ma_uint32 sampleRateOut)
53033 | {
53034 |     ma_linear_resampler_config config;
53035 |     MA_ZERO_OBJECT(&config);
53036 |     config.format = format;
53037 |     config.channels = channels;
53038 |     config.sampleRateIn = sampleRateIn;
53039 |     config.sampleRateOut = sampleRateOut;
53040 |     config.lpfOrder = ma_min(MA_DEFAULT_RESAMPLER_LPF_ORDER, MA_MAX_FILTER_ORDER);
53041 |     config.lpfNyquistFactor = 1;
53042 |
53043 |     return config;
53044 | }
53045 |
53046 |
53047 | typedef struct
53048 | {
53049 |     size_t sizeInBytes;
53050 |     size_t x0Offset;
53051 |     size_t x1Offset;
53052 |     size_t lpfOffset;
53053 | } ma_linear_resampler_heap_layout;
53054 |
53055 |
53056 | static void ma_linear_resampler_adjust_timer_for_new_rate(ma_linear_resampler* pResampler, ma_uint32
> oldSampleRateOut, ma_uint32 newSampleRateOut)
53057 | {
53058 |     /*
53059 |     So what's happening here? Basically we need to adjust the fractional component of the time
> advance based on the new rate. The old time advance will
53060 |     be based on the old sample rate, but we are needing to adjust it to that it's based on the new
> sample rate.
53061 |     */
53062 |     ma_uint32 oldRateTimeWhole = pResampler->inTimeFrac / oldSampleRateOut; /* <-- This should
> almost never be anything other than 0, but leaving it here to make this more general and robust just
> in case. */
53063 |     ma_uint32 oldRateTimeFrac = pResampler->inTimeFrac % oldSampleRateOut;
53064 |
53065 |     pResampler->inTimeFrac =
53066 |         (oldRateTimeWhole * newSampleRateOut) +
53067 |         ((oldRateTimeFrac * newSampleRateOut) / oldSampleRateOut);
53068 |
53069 |     /* Make sure the fractional part is less than the output sample rate. */
53070 |     pResampler->inTimeInt += pResampler->inTimeFrac / pResampler->config.sampleRateOut;
53071 |     pResampler->inTimeFrac = pResampler->inTimeFrac % pResampler->config.sampleRateOut;
53072 | }
53073 |
53074 | static ma_result ma_linear_resampler_set_rate_internal(ma_linear_resampler* pResampler, void* pHeap,
> ma_linear_resampler_heap_layout* pHeapLayout, ma_uint32 sampleRateIn, ma_uint32 sampleRateOut,
> ma_bool32 isResamplerAlreadyInitialized)
53075 | {
53076 |     ma_result result;

```

```

53077 |     ma_uint32 gcf;
53078 |     ma_uint32 lpfSampleRate;
53079 |     double lpfCutoffFrequency;
53080 |     ma_lpf_config lpfConfig;
53081 |     ma_uint32 oldSampleRateOut; /* Required for adjusting time advance down the bottom. */
53082 |
53083 |     if (pResampler == NULL) {
53084 |         return MA_INVALID_ARGS;
53085 |     }
53086 |
53087 |     if (sampleRateIn == 0 || sampleRateOut == 0) {
53088 |         return MA_INVALID_ARGS;
53089 |     }
53090 |
53091 |     oldSampleRateOut = pResampler->config.sampleRateOut;
53092 |
53093 |     pResampler->config.sampleRateIn = sampleRateIn;
53094 |     pResampler->config.sampleRateOut = sampleRateOut;
53095 |
53096 |     /* Simplify the sample rate. */
53097 |     gcf = ma_gcf_u32(pResampler->config.sampleRateIn, pResampler->config.sampleRateOut);
53098 |     pResampler->config.sampleRateIn /= gcf;
53099 |     pResampler->config.sampleRateOut /= gcf;
53100 |
53101 |     /* Always initialize the low-pass filter, even when the order is 0. */
53102 |     if (pResampler->config.lpfOrder > MA_MAX_FILTER_ORDER) {
53103 |         return MA_INVALID_ARGS;
53104 |     }
53105 |
53106 |     lpfSampleRate = (ma_uint32)(ma_max(pResampler->config.sampleRateIn,
> pResampler->config.sampleRateOut));
53107 |     lpfCutoffFrequency = (double)(ma_min(pResampler->config.sampleRateIn,
> pResampler->config.sampleRateOut) * 0.5 * pResampler->config.lpfNyquistFactor);
53108 |
53109 |     lpfConfig = ma_lpf_config_init(pResampler->config.format, pResampler->config.channels,
> lpfSampleRate, lpfCutoffFrequency, pResampler->config.lpfOrder);
53110 |
53111 |     /*
53112 |     > If the resampler is already initialized we don't want to do a fresh initialization of the
> low-pass filter because it will result in the cached frames
53113 |     > getting cleared. Instead we re-initialize the filter which will maintain any cached frames.
53114 |     */
53115 |     if (isResamplerAlreadyInitialized) {
53116 |         result = ma_lpf_reinit(&lpfConfig, &pResampler->lpf);
53117 |     } else {
53118 |         result = ma_lpf_init_preallocated(&lpfConfig, ma_offset_ptr(pHeap, pHeapLayout->lpfOffset),
> &pResampler->lpf);
53119 |     }
53120 |
53121 |     if (result != MA_SUCCESS) {
53122 |         return result;
53123 |     }
53124 |
53125 |
53126 |     pResampler->inAdvanceInt = pResampler->config.sampleRateIn / pResampler->config.sampleRateOut;
53127 |     pResampler->inAdvanceFrac = pResampler->config.sampleRateIn % pResampler->config.sampleRateOut;
53128 |
53129 |     /* Our timer was based on the old rate. We need to adjust it so that it's based on the new rate.
> */
53130 |     ma_linear_resampler_adjust_timer_for_new_rate(pResampler, oldSampleRateOut,
> pResampler->config.sampleRateOut);
53131 |
53132 |     return MA_SUCCESS;
53133 | }
53134 |
53135 | static ma_result ma_linear_resampler_get_heap_layout(const ma_linear_resampler_config* pConfig,
> ma_linear_resampler_heap_layout* pHeapLayout)
53136 | {
53137 |     MA_ASSERT(pHeapLayout != NULL);
53138 |
53139 |     MA_ZERO_OBJECT(pHeapLayout);
53140 |
53141 |     if (pConfig == NULL) {
53142 |         return MA_INVALID_ARGS;
53143 |     }
53144 |
53145 |     if (pConfig->format != ma_format_f32 && pConfig->format != ma_format_s16) {
53146 |         return MA_INVALID_ARGS;
53147 |     }

```

```

53148 |
53149 |     if (pConfig->channels == 0) {
53150 |         return MA_INVALID_ARGS;
53151 |     }
53152 |
53153 |     pHeapLayout->sizeInBytes = 0;
53154 |
53155 |     /* x0 */
53156 |     pHeapLayout->x0Offset = pHeapLayout->sizeInBytes;
53157 |     if (pConfig->format == ma_format_f32) {
53158 |         pHeapLayout->sizeInBytes += sizeof(float) * pConfig->channels;
53159 |     } else {
53160 |         pHeapLayout->sizeInBytes += sizeof(ma_int16) * pConfig->channels;
53161 |     }
53162 |
53163 |     /* x1 */
53164 |     pHeapLayout->x1Offset = pHeapLayout->sizeInBytes;
53165 |     if (pConfig->format == ma_format_f32) {
53166 |         pHeapLayout->sizeInBytes += sizeof(float) * pConfig->channels;
53167 |     } else {
53168 |         pHeapLayout->sizeInBytes += sizeof(ma_int16) * pConfig->channels;
53169 |     }
53170 |
53171 |     /* LPF */
53172 |     pHeapLayout->lpfOffset = ma_align_64(pHeapLayout->sizeInBytes);
53173 |     {
53174 |         ma_result result;
53175 |         size_t lpfHeapSizeInBytes;
53176 |         ma_lpf_config lpfConfig = ma_lpf_config_init(pConfig->format, pConfig->channels, 1, 1,
> pConfig->lpfOrder); /* Sample rate and cutoff frequency do not matter. */
53177 |
53178 |         result = ma_lpf_get_heap_size(&lpfConfig, &lpfHeapSizeInBytes);
53179 |         if (result != MA_SUCCESS) {
53180 |             return result;
53181 |         }
53182 |
53183 |         pHeapLayout->sizeInBytes += lpfHeapSizeInBytes;
53184 |     }
53185 |
53186 |     /* Make sure allocation size is aligned. */
53187 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
53188 |
53189 |     return MA_SUCCESS;
53190 | }
53191 |
53192 | MA_API ma_result ma_linear_resampler_get_heap_size(const ma_linear_resampler_config* pConfig,
> size_t* pHeapSizeInBytes)
53193 | {
53194 |     ma_result result;
53195 |     ma_linear_resampler_heap_layout heapLayout;
53196 |
53197 |     if (pHeapSizeInBytes == NULL) {
53198 |         return MA_INVALID_ARGS;
53199 |     }
53200 |
53201 |     *pHeapSizeInBytes = 0;
53202 |
53203 |     result = ma_linear_resampler_get_heap_layout(pConfig, &heapLayout);
53204 |     if (result != MA_SUCCESS) {
53205 |         return result;
53206 |     }
53207 |
53208 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
53209 |
53210 |     return MA_SUCCESS;
53211 | }
53212 |
53213 | MA_API ma_result ma_linear_resampler_init_preallocated(const ma_linear_resampler_config* pConfig,
> void* pHeap, ma_linear_resampler* pResampler)
53214 | {
53215 |     ma_result result;
53216 |     ma_linear_resampler_heap_layout heapLayout;
53217 |
53218 |     if (pResampler == NULL) {
53219 |         return MA_INVALID_ARGS;
53220 |     }
53221 |
53222 |     MA_ZERO_OBJECT(pResampler);
53223 |

```



```

53224 |     result = ma_linear_resampler_get_heap_layout(pConfig, &heapLayout);
53225 |     if (result != MA_SUCCESS) {
53226 |         return result;
53227 |     }
53228 |
53229 |     pResampler->config = *pConfig;
53230 |
53231 |     pResampler->pHeap = pHeap;
53232 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
53233 |
53234 |     if (pConfig->format == ma_format_f32) {
53235 |         pResampler->x0.f32 = (float*)ma_offset_ptr(pHeap, heapLayout.x0offset);
53236 |         pResampler->x1.f32 = (float*)ma_offset_ptr(pHeap, heapLayout.x1offset);
53237 |     } else {
53238 |         pResampler->x0.s16 = (ma_int16*)ma_offset_ptr(pHeap, heapLayout.x0offset);
53239 |         pResampler->x1.s16 = (ma_int16*)ma_offset_ptr(pHeap, heapLayout.x1offset);
53240 |     }
53241 |
53242 |     /* Setting the rate will set up the filter and time advances for us. */
53243 |     result = ma_linear_resampler_set_rate_internal(pResampler, pHeap, &heapLayout,
> pConfig->sampleRateIn, pConfig->sampleRateOut, /* isResamplerAlreadyInitialized = */ MA_FALSE);
53244 |     if (result != MA_SUCCESS) {
53245 |         return result;
53246 |     }
53247 |
53248 |     pResampler->inTimeInt = 1; /* Set this to one to force an input sample to always be loaded for
> the first output frame. */
53249 |     pResampler->inTimeFrac = 0;
53250 |
53251 |     return MA_SUCCESS;
53252 | }
53253 |
53254 | MA_API ma_result ma_linear_resampler_init(const ma_linear_resampler_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_linear_resampler* pResampler)
53255 | {
53256 |     ma_result result;
53257 |     size_t heapSizeInBytes;
53258 |     void* pHeap;
53259 |
53260 |     result = ma_linear_resampler_get_heap_size(pConfig, &heapSizeInBytes);
53261 |     if (result != MA_SUCCESS) {
53262 |         return result;
53263 |     }
53264 |
53265 |     if (heapSizeInBytes > 0) {
53266 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
53267 |         if (pHeap == NULL) {
53268 |             return MA_OUT_OF_MEMORY;
53269 |         }
53270 |     } else {
53271 |         pHeap = NULL;
53272 |     }
53273 |
53274 |     result = ma_linear_resampler_init_preallocated(pConfig, pHeap, pResampler);
53275 |     if (result != MA_SUCCESS) {
53276 |         ma_free(pHeap, pAllocationCallbacks);
53277 |         return result;
53278 |     }
53279 |
53280 |     pResampler->ownsHeap = MA_TRUE;
53281 |     return MA_SUCCESS;
53282 | }
53283 |
53284 | MA_API void ma_linear_resampler_uninit(ma_linear_resampler* pResampler, const
> ma_allocation_callbacks* pAllocationCallbacks)
53285 | {
53286 |     if (pResampler == NULL) {
53287 |         return;
53288 |     }
53289 |
53290 |     ma_lpf_uninit(&pResampler->lpf, pAllocationCallbacks);
53291 |
53292 |     if (pResampler->ownsHeap) {
53293 |         ma_free(pResampler->pHeap, pAllocationCallbacks);
53294 |     }
53295 | }
53296 |
53297 | static MA_INLINE ma_int16 ma_linear_resampler_mix_s16(ma_int16 x, ma_int16 y, ma_int32 a, const
> ma_int32 shift)

```

```

53298 | {
53299 |     ma_int32 b;
53300 |     ma_int32 c;
53301 |     ma_int32 r;
53302 |
53303 |     MA_ASSERT(a <= (1<<shift));
53304 |
53305 |     b = x * ((1<<shift) - a);
53306 |     c = y * a;
53307 |     r = b + c;
53308 |
53309 |     return (ma_int16)(r >> shift);
53310 | }
53311 |
53312 | static void ma_linear_resampler_interpolate_frame_s16(ma_linear_resampler* pResampler, ma_int16*
> MA_RESTRICT pFrameOut)
53313 | {
53314 |     ma_uint32 c;
53315 |     ma_uint32 a;
53316 |     const ma_uint32 channels = pResampler->config.channels;
53317 |     const ma_uint32 shift = 12;
53318 |
53319 |     MA_ASSERT(pResampler != NULL);
53320 |     MA_ASSERT(pFrameOut != NULL);
53321 |
53322 |     a = (pResampler->inTimeFrac << shift) / pResampler->config.sampleRateOut;
53323 |
53324 |     MA_ASSUME(channels > 0);
53325 |     for (c = 0; c < channels; c += 1) {
53326 |         ma_int16 s = ma_linear_resampler_mix_s16(pResampler->x0.s16[c], pResampler->x1.s16[c], a,
> shift);
53327 |         pFrameOut[c] = s;
53328 |     }
53329 | }
53330 |
53331 |
53332 | static void ma_linear_resampler_interpolate_frame_f32(ma_linear_resampler* pResampler, float*
> MA_RESTRICT pFrameOut)
53333 | {
53334 |     ma_uint32 c;
53335 |     float a;
53336 |     const ma_uint32 channels = pResampler->config.channels;
53337 |
53338 |     MA_ASSERT(pResampler != NULL);
53339 |     MA_ASSERT(pFrameOut != NULL);
53340 |
53341 |     a = (float)pResampler->inTimeFrac / pResampler->config.sampleRateOut;
53342 |
53343 |     MA_ASSUME(channels > 0);
53344 |     for (c = 0; c < channels; c += 1) {
53345 |         float s = ma_mix_f32_fast(pResampler->x0.f32[c], pResampler->x1.f32[c], a);
53346 |         pFrameOut[c] = s;
53347 |     }
53348 | }
53349 |
53350 | static ma_result ma_linear_resampler_process_pcm_frames_s16_downsample(ma_linear_resampler*
> pResampler, const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64*
> pFrameCountOut)
53351 | {
53352 |     const ma_int16* pFramesInS16;
53353 |     /* */ ma_int16* pFramesOutS16;
53354 |     ma_uint64 frameCountIn;
53355 |     ma_uint64 frameCountOut;
53356 |     ma_uint64 framesProcessedIn;
53357 |     ma_uint64 framesProcessedOut;
53358 |
53359 |     MA_ASSERT(pResampler != NULL);
53360 |     MA_ASSERT(pFrameCountIn != NULL);
53361 |     MA_ASSERT(pFrameCountOut != NULL);
53362 |
53363 |     pFramesInS16 = (const ma_int16*)pFramesIn;
53364 |     pFramesOutS16 = (ma_int16*)pFramesOut;
53365 |     frameCountIn = *pFrameCountIn;
53366 |     frameCountOut = *pFrameCountOut;
53367 |     framesProcessedIn = 0;
53368 |     framesProcessedOut = 0;
53369 |
53370 |     while (framesProcessedOut < frameCountOut) {
53371 |         /* Before interpolating we need to load the buffers. When doing this we need to ensure we

```

```

> run every input sample through the filter. */
53372 | while (pResampler->inTimeInt > 0 && frameCountIn > framesProcessedIn) {
53373 |     ma_uint32 iChannel;
53374 |
53375 |     if (pFramesInS16 != NULL) {
53376 |         for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53377 |             pResampler->x0.s16[iChannel] = pResampler->x1.s16[iChannel];
53378 |             pResampler->x1.s16[iChannel] = pFramesInS16[iChannel];
53379 |         }
53380 |         pFramesInS16 += pResampler->config.channels;
53381 |     } else {
53382 |         for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53383 |             pResampler->x0.s16[iChannel] = pResampler->x1.s16[iChannel];
53384 |             pResampler->x1.s16[iChannel] = 0;
53385 |         }
53386 |     }
53387 |
53388 |     /* Filter. Do not apply filtering if sample rates are the same or else you'll get
> dangerous glitching. */
53389 |     if (pResampler->config.sampleRateIn != pResampler->config.sampleRateOut) {
53390 |         ma_lpf_process_pcm_frame_s16(&pResampler->lpf, pResampler->x1.s16,
> pResampler->x1.s16);
53391 |     }
53392 |
53393 |     framesProcessedIn += 1;
53394 |     pResampler->inTimeInt -= 1;
53395 | }
53396 |
53397 | if (pResampler->inTimeInt > 0) {
53398 |     break; /* Ran out of input data. */
53399 | }
53400 |
53401 | /* Getting here means the frames have been loaded and filtered and we can generate the next
> output frame. */
53402 | if (pFramesOutS16 != NULL) {
53403 |     MA_ASSERT(pResampler->inTimeInt == 0);
53404 |     ma_linear_resampler_interpolate_frame_s16(pResampler, pFramesOutS16);
53405 |
53406 |     pFramesOutS16 += pResampler->config.channels;
53407 | }
53408 |
53409 | framesProcessedOut += 1;
53410 |
53411 | /* Advance time forward. */
53412 | pResampler->inTimeInt += pResampler->inAdvanceInt;
53413 | pResampler->inTimeFrac += pResampler->inAdvanceFrac;
53414 | if (pResampler->inTimeFrac >= pResampler->config.sampleRateOut) {
53415 |     pResampler->inTimeFrac -= pResampler->config.sampleRateOut;
53416 |     pResampler->inTimeInt += 1;
53417 | }
53418 | }
53419 |
53420 | *pFrameCountIn = framesProcessedIn;
53421 | *pFrameCountOut = framesProcessedOut;
53422 |
53423 | return MA_SUCCESS;
53424 | }
53425 |
53426 | static ma_result ma_linear_resampler_process_pcm_frames_s16_upsample(ma_linear_resampler*
> pResampler, const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64*
> pFrameCountOut)
53427 | {
53428 |     const ma_int16* pFramesInS16;
53429 |     /* */ ma_int16* pFramesOutS16;
53430 |     ma_uint64 frameCountIn;
53431 |     ma_uint64 frameCountOut;
53432 |     ma_uint64 framesProcessedIn;
53433 |     ma_uint64 framesProcessedOut;
53434 |
53435 |     MA_ASSERT(pResampler != NULL);
53436 |     MA_ASSERT(pFrameCountIn != NULL);
53437 |     MA_ASSERT(pFrameCountOut != NULL);
53438 |
53439 |     pFramesInS16 = (const ma_int16*)pFramesIn;
53440 |     pFramesOutS16 = (ma_int16*)pFramesOut;
53441 |     frameCountIn = *pFrameCountIn;
53442 |     frameCountOut = *pFrameCountOut;
53443 |     framesProcessedIn = 0;
53444 |     framesProcessedOut = 0;

```

```

53445 |
53446 |     while (framesProcessedOut < frameCountOut) {
53447 |         /* Before interpolating we need to load the buffers. */
53448 |         while (pResampler->inTimeInt > 0 && frameCountIn > framesProcessedIn) {
53449 |             ma_uint32 iChannel;
53450 |
53451 |             if (pFramesInS16 != NULL) {
53452 |                 for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53453 |                     pResampler->x0.s16[iChannel] = pResampler->x1.s16[iChannel];
53454 |                     pResampler->x1.s16[iChannel] = pFramesInS16[iChannel];
53455 |                 }
53456 |                 pFramesInS16 += pResampler->config.channels;
53457 |             } else {
53458 |                 for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53459 |                     pResampler->x0.s16[iChannel] = pResampler->x1.s16[iChannel];
53460 |                     pResampler->x1.s16[iChannel] = 0;
53461 |                 }
53462 |             }
53463 |
53464 |             framesProcessedIn += 1;
53465 |             pResampler->inTimeInt -= 1;
53466 |         }
53467 |
53468 |         if (pResampler->inTimeInt > 0) {
53469 |             break; /* Ran out of input data. */
53470 |         }
53471 |
53472 |         /* Getting here means the frames have been loaded and we can generate the next output frame.
> */
53473 |         if (pFramesOutS16 != NULL) {
53474 |             MA_ASSERT(pResampler->inTimeInt == 0);
53475 |             ma_linear_resampler_interpolate_frame_s16(pResampler, pFramesOutS16);
53476 |
53477 |             /* Filter. Do not apply filtering if sample rates are the same or else you'll get
> dangerous glitching. */
53478 |             if (pResampler->config.sampleRateIn != pResampler->config.sampleRateOut) {
53479 |                 ma_lpf_process_pcm_frame_s16(&pResampler->lpf, pFramesOutS16, pFramesOutS16);
53480 |             }
53481 |
53482 |             pFramesOutS16 += pResampler->config.channels;
53483 |         }
53484 |
53485 |         framesProcessedOut += 1;
53486 |
53487 |         /* Advance time forward. */
53488 |         pResampler->inTimeInt += pResampler->inAdvanceInt;
53489 |         pResampler->inTimeFrac += pResampler->inAdvanceFrac;
53490 |         if (pResampler->inTimeFrac >= pResampler->config.sampleRateOut) {
53491 |             pResampler->inTimeFrac -= pResampler->config.sampleRateOut;
53492 |             pResampler->inTimeInt += 1;
53493 |         }
53494 |     }
53495 |
53496 |     *pFrameCountIn = framesProcessedIn;
53497 |     *pFrameCountOut = framesProcessedOut;
53498 |
53499 |     return MA_SUCCESS;
53500 | }
53501 |
53502 | static ma_result ma_linear_resampler_process_pcm_frames_s16(ma_linear_resampler* pResampler, const
> void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
53503 | {
53504 |     MA_ASSERT(pResampler != NULL);
53505 |
53506 |     if (pResampler->config.sampleRateIn > pResampler->config.sampleRateOut) {
53507 |         return ma_linear_resampler_process_pcm_frames_s16_downsample(pResampler, pFramesIn,
> pFrameCountIn, pFramesOut, pFrameCountOut);
53508 |     } else {
53509 |         return ma_linear_resampler_process_pcm_frames_s16_upsample(pResampler, pFramesIn,
> pFrameCountIn, pFramesOut, pFrameCountOut);
53510 |     }
53511 | }
53512 |
53513 |
53514 | static ma_result ma_linear_resampler_process_pcm_frames_f32_downsample(ma_linear_resampler*
> pResampler, const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64*
> pFrameCountOut)
53515 | {
53516 |     const float* pFramesInF32;

```

```

53517 | /* */ float* pFramesOutF32;
53518 | ma_uint64 frameCountIn;
53519 | ma_uint64 frameCountOut;
53520 | ma_uint64 framesProcessedIn;
53521 | ma_uint64 framesProcessedOut;
53522 |
53523 | MA_ASSERT(pResampler != NULL);
53524 | MA_ASSERT(pFrameCountIn != NULL);
53525 | MA_ASSERT(pFrameCountOut != NULL);
53526 |
53527 | pFramesInF32 = (const float*)pFramesIn;
53528 | pFramesOutF32 = (float*)pFramesOut;
53529 | frameCountIn = *pFrameCountIn;
53530 | frameCountOut = *pFrameCountOut;
53531 | framesProcessedIn = 0;
53532 | framesProcessedOut = 0;
53533 |
53534 | while (framesProcessedOut < frameCountOut) {
53535 |     /* Before interpolating we need to load the buffers. When doing this we need to ensure we
> run every input sample through the filter. */
53536 |     while (pResampler->inTimeInt > 0 && frameCountIn > framesProcessedIn) {
53537 |         ma_uint32 iChannel;
53538 |
53539 |         if (pFramesInF32 != NULL) {
53540 |             for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53541 |                 pResampler->x0.f32[iChannel] = pResampler->x1.f32[iChannel];
53542 |                 pResampler->x1.f32[iChannel] = pFramesInF32[iChannel];
53543 |             }
53544 |             pFramesInF32 += pResampler->config.channels;
53545 |         } else {
53546 |             for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53547 |                 pResampler->x0.f32[iChannel] = pResampler->x1.f32[iChannel];
53548 |                 pResampler->x1.f32[iChannel] = 0;
53549 |             }
53550 |         }
53551 |
53552 |         /* Filter. Do not apply filtering if sample rates are the same or else you'll get
> dangerous glitching. */
53553 |         if (pResampler->config.sampleRateIn != pResampler->config.sampleRateOut) {
53554 |             ma_lpf_process_pcm_frame_f32(&pResampler->lpf, pResampler->x1.f32,
> pResampler->x1.f32);
53555 |         }
53556 |
53557 |         framesProcessedIn += 1;
53558 |         pResampler->inTimeInt -= 1;
53559 |     }
53560 |
53561 |     if (pResampler->inTimeInt > 0) {
53562 |         break; /* Ran out of input data. */
53563 |     }
53564 |
53565 |     /* Getting here means the frames have been loaded and filtered and we can generate the next
> output frame. */
53566 |     if (pFramesOutF32 != NULL) {
53567 |         MA_ASSERT(pResampler->inTimeInt == 0);
53568 |         ma_linear_resampler_interpolate_frame_f32(pResampler, pFramesOutF32);
53569 |
53570 |         pFramesOutF32 += pResampler->config.channels;
53571 |     }
53572 |
53573 |     framesProcessedOut += 1;
53574 |
53575 |     /* Advance time forward. */
53576 |     pResampler->inTimeInt += pResampler->inAdvanceInt;
53577 |     pResampler->inTimeFrac += pResampler->inAdvanceFrac;
53578 |     if (pResampler->inTimeFrac >= pResampler->config.sampleRateOut) {
53579 |         pResampler->inTimeFrac -= pResampler->config.sampleRateOut;
53580 |         pResampler->inTimeInt += 1;
53581 |     }
53582 | }
53583 |
53584 | *pFrameCountIn = framesProcessedIn;
53585 | *pFrameCountOut = framesProcessedOut;
53586 |
53587 | return MA_SUCCESS;
53588 | }
53589 |
53590 | static ma_result ma_linear_resampler_process_pcm_frames_f32_upsample(ma_linear_resampler*
> pResampler, const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64*

```

```

> pFrameCountOut)
53591 | {
53592 |     const float* pFramesInF32;
53593 |     /* */ float* pFramesOutF32;
53594 |     ma_uint64 frameCountIn;
53595 |     ma_uint64 frameCountOut;
53596 |     ma_uint64 framesProcessedIn;
53597 |     ma_uint64 framesProcessedOut;
53598 |
53599 |     MA_ASSERT(pResampler != NULL);
53600 |     MA_ASSERT(pFrameCountIn != NULL);
53601 |     MA_ASSERT(pFrameCountOut != NULL);
53602 |
53603 |     pFramesInF32 = (const float*)pFramesIn;
53604 |     pFramesOutF32 = (float*)pFramesOut;
53605 |     frameCountIn = *pFrameCountIn;
53606 |     frameCountOut = *pFrameCountOut;
53607 |     framesProcessedIn = 0;
53608 |     framesProcessedOut = 0;
53609 |
53610 |     while (framesProcessedOut < frameCountOut) {
53611 |         /* Before interpolating we need to load the buffers. */
53612 |         while (pResampler->inTimeInt > 0 && frameCountIn > framesProcessedIn) {
53613 |             ma_uint32 iChannel;
53614 |
53615 |             if (pFramesInF32 != NULL) {
53616 |                 for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53617 |                     pResampler->x0.f32[iChannel] = pResampler->x1.f32[iChannel];
53618 |                     pResampler->x1.f32[iChannel] = pFramesInF32[iChannel];
53619 |                 }
53620 |                 pFramesInF32 += pResampler->config.channels;
53621 |             } else {
53622 |                 for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53623 |                     pResampler->x0.f32[iChannel] = pResampler->x1.f32[iChannel];
53624 |                     pResampler->x1.f32[iChannel] = 0;
53625 |                 }
53626 |             }
53627 |
53628 |             framesProcessedIn += 1;
53629 |             pResampler->inTimeInt -= 1;
53630 |         }
53631 |
53632 |         if (pResampler->inTimeInt > 0) {
53633 |             break; /* Ran out of input data. */
53634 |         }
53635 |
53636 |         /* Getting here means the frames have been loaded and we can generate the next output frame.
> */
53637 |         if (pFramesOutF32 != NULL) {
53638 |             MA_ASSERT(pResampler->inTimeInt == 0);
53639 |             ma_linear_resampler_interpolate_frame_f32(pResampler, pFramesOutF32);
53640 |
53641 |             /* Filter. Do not apply filtering if sample rates are the same or else you'll get
> dangerous glitching. */
53642 |             if (pResampler->config.sampleRateIn != pResampler->config.sampleRateOut) {
53643 |                 ma_lpf_process_pcm_frame_f32(&pResampler->lpf, pFramesOutF32, pFramesOutF32);
53644 |             }
53645 |
53646 |             pFramesOutF32 += pResampler->config.channels;
53647 |         }
53648 |
53649 |         framesProcessedOut += 1;
53650 |
53651 |         /* Advance time forward. */
53652 |         pResampler->inTimeInt += pResampler->inAdvanceInt;
53653 |         pResampler->inTimeFrac += pResampler->inAdvanceFrac;
53654 |         if (pResampler->inTimeFrac >= pResampler->config.sampleRateOut) {
53655 |             pResampler->inTimeFrac -= pResampler->config.sampleRateOut;
53656 |             pResampler->inTimeInt += 1;
53657 |         }
53658 |     }
53659 |
53660 |     *pFrameCountIn = framesProcessedIn;
53661 |     *pFrameCountOut = framesProcessedOut;
53662 |
53663 |     return MA_SUCCESS;
53664 | }
53665 |
53666 | static ma_result ma_linear_resampler_process_pcm_frames_f32(ma_linear_resampler* pResampler, const

```

```

> void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
53667 | {
53668 |     MA_ASSERT(pResampler != NULL);
53669 |
53670 |     if (pResampler->config.sampleRateIn > pResampler->config.sampleRateOut) {
53671 |         return ma_linear_resampler_process_pcm_frames_f32_downsample(pResampler, pFramesIn,
> pFrameCountIn, pFramesOut, pFrameCountOut);
53672 |     } else {
53673 |         return ma_linear_resampler_process_pcm_frames_f32_upsample(pResampler, pFramesIn,
> pFrameCountIn, pFramesOut, pFrameCountOut);
53674 |     }
53675 | }
53676 |
53677 |
53678 | MA_API ma_result ma_linear_resampler_process_pcm_frames(ma_linear_resampler* pResampler, const void*
> pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
53679 | {
53680 |     if (pResampler == NULL) {
53681 |         return MA_INVALID_ARGS;
53682 |     }
53683 |
53684 |     /* */ if (pResampler->config.format == ma_format_s16) {
53685 |         return ma_linear_resampler_process_pcm_frames_s16(pResampler, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
53686 |     } else if (pResampler->config.format == ma_format_f32) {
53687 |         return ma_linear_resampler_process_pcm_frames_f32(pResampler, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
53688 |     } else {
53689 |         /* Should never get here. Getting here means the format is not supported and you didn't
> check the return value of ma_linear_resampler_init(). */
53690 |         MA_ASSERT(MA_FALSE);
53691 |         return MA_INVALID_ARGS;
53692 |     }
53693 | }
53694 |
53695 |
53696 | MA_API ma_result ma_linear_resampler_set_rate(ma_linear_resampler* pResampler, ma_uint32
> sampleRateIn, ma_uint32 sampleRateOut)
53697 | {
53698 |     return ma_linear_resampler_set_rate_internal(pResampler, NULL, NULL, sampleRateIn,
> sampleRateOut, /* isResamplerAlreadyInitialized = */ MA_TRUE);
53699 | }
53700 |
53701 | MA_API ma_result ma_linear_resampler_set_rate_ratio(ma_linear_resampler* pResampler, float
> ratioInOut)
53702 | {
53703 |     ma_uint32 n;
53704 |     ma_uint32 d;
53705 |
53706 |     if (pResampler == NULL) {
53707 |         return MA_INVALID_ARGS;
53708 |     }
53709 |
53710 |     if (ratioInOut <= 0) {
53711 |         return MA_INVALID_ARGS;
53712 |     }
53713 |
53714 |     d = 1000000;
53715 |     n = (ma_uint32)(ratioInOut * d);
53716 |
53717 |     if (n == 0) {
53718 |         return MA_INVALID_ARGS; /* Ratio too small. */
53719 |     }
53720 |
53721 |     MA_ASSERT(n != 0);
53722 |
53723 |     return ma_linear_resampler_set_rate(pResampler, n, d);
53724 | }
53725 |
53726 | MA_API ma_uint64 ma_linear_resampler_get_input_latency(const ma_linear_resampler* pResampler)
53727 | {
53728 |     if (pResampler == NULL) {
53729 |         return 0;
53730 |     }
53731 |
53732 |     return 1 + ma_lpf_get_latency(&pResampler->lpf);
53733 | }
53734 |
53735 | MA_API ma_uint64 ma_linear_resampler_get_output_latency(const ma_linear_resampler* pResampler)

```



```

53736 | {
53737 |     if (pResampler == NULL) {
53738 |         return 0;
53739 |     }
53740 |
53741 |     return ma_linear_resampler_get_input_latency(pResampler) * pResampler->config.sampleRateOut /
> pResampler->config.sampleRateIn;
53742 | }
53743 |
53744 | MA_API ma_result ma_linear_resampler_get_required_input_frame_count(const ma_linear_resampler*
> pResampler, ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount)
53745 | {
53746 |     ma_uint64 inputFrameCount;
53747 |
53748 |     if (pInputFrameCount == NULL) {
53749 |         return MA_INVALID_ARGS;
53750 |     }
53751 |
53752 |     *pInputFrameCount = 0;
53753 |
53754 |     if (pResampler == NULL) {
53755 |         return MA_INVALID_ARGS;
53756 |     }
53757 |
53758 |     if (outputFrameCount == 0) {
53759 |         return MA_SUCCESS;
53760 |     }
53761 |
53762 |     /* Any whole input frames are consumed before the first output frame is generated. */
53763 |     inputFrameCount = pResampler->inTimeInt;
53764 |     outputFrameCount -= 1;
53765 |
53766 |     /* The rest of the output frames can be calculated in constant time. */
53767 |     inputFrameCount += outputFrameCount * pResampler->inAdvanceInt;
53768 |     inputFrameCount += (pResampler->inTimeFrac + (outputFrameCount * pResampler->inAdvanceFrac)) /
> pResampler->config.sampleRateOut;
53769 |
53770 |     *pInputFrameCount = inputFrameCount;
53771 |
53772 |     return MA_SUCCESS;
53773 | }
53774 |
53775 | MA_API ma_result ma_linear_resampler_get_expected_output_frame_count(const ma_linear_resampler*
> pResampler, ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount)
53776 | {
53777 |     ma_uint64 outputFrameCount;
53778 |     ma_uint64 preliminaryInputFrameCountFromFrac;
53779 |     ma_uint64 preliminaryInputFrameCount;
53780 |
53781 |     if (pOutputFrameCount == NULL) {
53782 |         return MA_INVALID_ARGS;
53783 |     }
53784 |
53785 |     *pOutputFrameCount = 0;
53786 |
53787 |     if (pResampler == NULL) {
53788 |         return MA_INVALID_ARGS;
53789 |     }
53790 |
53791 |     /*
53792 |     The first step is to get a preliminary output frame count. This will either be exactly equal to
> what we need, or less by 1. We need to
53793 |     determine how many input frames will be consumed by this value. If it's greater than our
> original input frame count it means we won't
53794 |     be able to generate an extra frame because we will have run out of input data. Otherwise we will
> have enough input for the generation
53795 |     of an extra output frame. This add-by-one logic is necessary due to how the data loading logic
> works when processing frames.
53796 |     */
53797 |     outputFrameCount = (inputFrameCount * pResampler->config.sampleRateOut) /
> pResampler->config.sampleRateIn;
53798 |
53799 |     /*
53800 |     We need to determine how many *whole* input frames will have been processed to generate our
> preliminary output frame count. This is
53801 |     used in the logic below to determine whether or not we need to add an extra output frame.
53802 |     */
53803 |     preliminaryInputFrameCountFromFrac = (pResampler->inTimeFrac +
> outputFrameCount*pResampler->inAdvanceFrac) / pResampler->config.sampleRateOut;

```

```

53804 |     preliminaryInputFrameCount      = (pResampler->inTimeInt  +
> outputFrameCount*pResampler->inAdvanceInt ) + preliminaryInputFrameCountFromFrac;
53805 |
53806 |     /*
53807 |     If the total number of *whole* input frames that would be required to generate our preliminary
> output frame count is greater than
53808 |     the amount of whole input frames we have available as input we need to *not* add an extra output
> frame as there won't be enough data
53809 |     to actually process. Otherwise we need to add the extra output frame.
53810 |     */
53811 |     if (preliminaryInputFrameCount <= inputFrameCount) {
53812 |         outputFrameCount += 1;
53813 |     }
53814 |
53815 |     *pOutputFrameCount = outputFrameCount;
53816 |
53817 |     return MA_SUCCESS;
53818 | }
53819 |
53820 | MA_API ma_result ma_linear_resampler_reset(ma_linear_resampler* pResampler)
53821 | {
53822 |     ma_uint32 iChannel;
53823 |
53824 |     if (pResampler == NULL) {
53825 |         return MA_INVALID_ARGS;
53826 |     }
53827 |
53828 |     /* Timers need to be cleared back to zero. */
53829 |     pResampler->inTimeInt = 1; /* Set this to one to force an input sample to always be loaded for
> the first output frame. */
53830 |     pResampler->inTimeFrac = 0;
53831 |
53832 |     /* Cached samples need to be cleared. */
53833 |     if (pResampler->config.format == ma_format_f32) {
53834 |         for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53835 |             pResampler->x0.f32[iChannel] = 0;
53836 |             pResampler->x1.f32[iChannel] = 0;
53837 |         }
53838 |     } else {
53839 |         for (iChannel = 0; iChannel < pResampler->config.channels; iChannel += 1) {
53840 |             pResampler->x0.s16[iChannel] = 0;
53841 |             pResampler->x1.s16[iChannel] = 0;
53842 |         }
53843 |     }
53844 |
53845 |     /* The low pass filter needs to have its cache reset. */
53846 |     ma_lpf_clear_cache(&pResampler->lpf);
53847 |
53848 |     return MA_SUCCESS;
53849 | }
53850 |
53851 |
53852 |
53853 | /* Linear resampler backend vtable. */
53854 | static ma_linear_resampler_config ma_resampling_backend_get_config__linear(const
> ma_resampler_config* pConfig)
53855 | {
53856 |     ma_linear_resampler_config linearConfig;
53857 |
53858 |     linearConfig = ma_linear_resampler_config_init(pConfig->format, pConfig->channels,
> pConfig->sampleRateIn, pConfig->sampleRateOut);
53859 |     linearConfig.lpfOrder = pConfig->linear.lpfOrder;
53860 |
53861 |     return linearConfig;
53862 | }
53863 |
53864 | static ma_result ma_resampling_backend_get_heap_size__linear(void* pUserData, const
> ma_resampler_config* pConfig, size_t* pHeapSizeInBytes)
53865 | {
53866 |     ma_linear_resampler_config linearConfig;
53867 |
53868 |     (void)pUserData;
53869 |
53870 |     linearConfig = ma_resampling_backend_get_config__linear(pConfig);
53871 |
53872 |     return ma_linear_resampler_get_heap_size(&linearConfig, pHeapSizeInBytes);
53873 | }
53874 |
53875 | static ma_result ma_resampling_backend_init__linear(void* pUserData, const ma_resampler_config*

```

```

> pConfig, void* pHeap, ma_resampling_backend** ppBackend)
53876 | {
53877 |     ma_resampler* pResampler = (ma_resampler*)pUserData;
53878 |     ma_result result;
53879 |     ma_linear_resampler_config linearConfig;
53880 |
53881 |     (void)pUserData;
53882 |
53883 |     linearConfig = ma_resampling_backend_get_config__linear(pConfig);
53884 |
53885 |     result = ma_linear_resampler_init_preallocated(&linearConfig, pHeap, &pResampler->state.linear);
53886 |     if (result != MA_SUCCESS) {
53887 |         return result;
53888 |     }
53889 |
53890 |     *ppBackend = &pResampler->state.linear;
53891 |
53892 |     return MA_SUCCESS;
53893 | }
53894 |
53895 | static void ma_resampling_backend_uninit__linear(void* pUserData, ma_resampling_backend* pBackend,
> const ma_allocation_callbacks* pAllocationCallbacks)
53896 | {
53897 |     (void)pUserData;
53898 |
53899 |     ma_linear_resampler_uninit((ma_linear_resampler*)pBackend, pAllocationCallbacks);
53900 | }
53901 |
53902 | static ma_result ma_resampling_backend_process__linear(void* pUserData, ma_resampling_backend*
> pBackend, const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64*
> pFrameCountOut)
53903 | {
53904 |     (void)pUserData;
53905 |
53906 |     return ma_linear_resampler_process_pcm_frames((ma_linear_resampler*)pBackend, pFramesIn,
> pFrameCountIn, pFramesOut, pFrameCountOut);
53907 | }
53908 |
53909 | static ma_result ma_resampling_backend_set_rate__linear(void* pUserData, ma_resampling_backend*
> pBackend, ma_uint32 sampleRateIn, ma_uint32 sampleRateOut)
53910 | {
53911 |     (void)pUserData;
53912 |
53913 |     return ma_linear_resampler_set_rate((ma_linear_resampler*)pBackend, sampleRateIn,
> sampleRateOut);
53914 | }
53915 |
53916 | static ma_uint64 ma_resampling_backend_get_input_latency__linear(void* pUserData, const
> ma_resampling_backend* pBackend)
53917 | {
53918 |     (void)pUserData;
53919 |
53920 |     return ma_linear_resampler_get_input_latency((const ma_linear_resampler*)pBackend);
53921 | }
53922 |
53923 | static ma_uint64 ma_resampling_backend_get_output_latency__linear(void* pUserData, const
> ma_resampling_backend* pBackend)
53924 | {
53925 |     (void)pUserData;
53926 |
53927 |     return ma_linear_resampler_get_output_latency((const ma_linear_resampler*)pBackend);
53928 | }
53929 |
53930 | static ma_result ma_resampling_backend_get_required_input_frame_count__linear(void* pUserData, const
> ma_resampling_backend* pBackend, ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount)
53931 | {
53932 |     (void)pUserData;
53933 |
53934 |     return ma_linear_resampler_get_required_input_frame_count((const ma_linear_resampler*)pBackend,
> outputFrameCount, pInputFrameCount);
53935 | }
53936 |
53937 | static ma_result ma_resampling_backend_get_expected_output_frame_count__linear(void* pUserData,
> const ma_resampling_backend* pBackend, ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount)
53938 | {
53939 |     (void)pUserData;
53940 |
53941 |     return ma_linear_resampler_get_expected_output_frame_count((const ma_linear_resampler*)pBackend,
> inputFrameCount, pOutputFrameCount);

```

```

53942 | }
53943 |
53944 | static ma_result ma_resampling_backend_reset__linear(void* pUserData, ma_resampling_backend*
> pBackend)
53945 | {
53946 |     (void)pUserData;
53947 |
53948 |     return ma_linear_resampler_reset((ma_linear_resampler*)pBackend);
53949 | }
53950 |
53951 | static ma_resampling_backend_vtable g_ma_linear_resampler_vtable =
53952 | {
53953 |     ma_resampling_backend_get_heap_size__linear,
53954 |     ma_resampling_backend_init__linear,
53955 |     ma_resampling_backend_uninit__linear,
53956 |     ma_resampling_backend_process__linear,
53957 |     ma_resampling_backend_set_rate__linear,
53958 |     ma_resampling_backend_get_input_latency__linear,
53959 |     ma_resampling_backend_get_output_latency__linear,
53960 |     ma_resampling_backend_get_required_input_frame_count__linear,
53961 |     ma_resampling_backend_get_expected_output_frame_count__linear,
53962 |     ma_resampling_backend_reset__linear
53963 | };
53964 |
53965 |
53966 |
53967 | MA_API ma_resampler_config ma_resampler_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRateIn, ma_uint32 sampleRateOut, ma_resample_algorithm algorithm)
53968 | {
53969 |     ma_resampler_config config;
53970 |
53971 |     MA_ZERO_OBJECT(&config);
53972 |     config.format = format;
53973 |     config.channels = channels;
53974 |     config.sampleRateIn = sampleRateIn;
53975 |     config.sampleRateOut = sampleRateOut;
53976 |     config.algorithm = algorithm;
53977 |
53978 |     /* Linear. */
53979 |     config.linear.lpfOrder = ma_min(MA_DEFAULT_RESAMPLER_LPF_ORDER, MA_MAX_FILTER_ORDER);
53980 |
53981 |     return config;
53982 | }
53983 |
53984 | static ma_result ma_resampler_get_vtable(const ma_resampler_config* pConfig, ma_resampler*
> pResampler, ma_resampling_backend_vtable** ppVTable, void** ppUserData)
53985 | {
53986 |     MA_ASSERT(pConfig != NULL);
53987 |     MA_ASSERT(ppVTable != NULL);
53988 |     MA_ASSERT(ppUserData != NULL);
53989 |
53990 |     /* Safety. */
53991 |     *ppVTable = NULL;
53992 |     *ppUserData = NULL;
53993 |
53994 |     switch (pConfig->algorithm)
53995 |     {
53996 |         case ma_resample_algorithm_linear:
53997 |         {
53998 |             *ppVTable = &g_ma_linear_resampler_vtable;
53999 |             *ppUserData = pResampler;
54000 |         } break;
54001 |
54002 |         case ma_resample_algorithm_custom:
54003 |         {
54004 |             *ppVTable = pConfig->pBackendVTable;
54005 |             *ppUserData = pConfig->pBackendUserData;
54006 |         } break;
54007 |
54008 |         default: return MA_INVALID_ARGS;
54009 |     }
54010 |
54011 |     return MA_SUCCESS;
54012 | }
54013 |
54014 | MA_API ma_result ma_resampler_get_heap_size(const ma_resampler_config* pConfig, size_t*
> pHeapSizeInBytes)
54015 | {
54016 |     ma_result result;

```

```

54017 |     ma_resampling_backend_vtable* pVTable;
54018 |     void* pVTableUserData;
54019 |
54020 |     if (pHeapSizeInBytes == NULL) {
54021 |         return MA_INVALID_ARGS;
54022 |     }
54023 |
54024 |     *pHeapSizeInBytes = 0;
54025 |
54026 |     if (pConfig == NULL) {
54027 |         return MA_INVALID_ARGS;
54028 |     }
54029 |
54030 |     result = ma_resampler_get_vtable(pConfig, NULL, &pVTable, &pVTableUserData);
54031 |     if (result != MA_SUCCESS) {
54032 |         return result;
54033 |     }
54034 |
54035 |     if (pVTable == NULL || pVTable->onGetHeapSize == NULL) {
54036 |         return MA_NOT_IMPLEMENTED;
54037 |     }
54038 |
54039 |     result = pVTable->onGetHeapSize(pVTableUserData, pConfig, pHeapSizeInBytes);
54040 |     if (result != MA_SUCCESS) {
54041 |         return result;
54042 |     }
54043 |
54044 |     return MA_SUCCESS;
54045 | }
54046 |
54047 | MA_API ma_result ma_resampler_init_preallocated(const ma_resampler_config* pConfig, void* pHeap,
> ma_resampler* pResampler)
54048 | {
54049 |     ma_result result;
54050 |
54051 |     if (pResampler == NULL) {
54052 |         return MA_INVALID_ARGS;
54053 |     }
54054 |
54055 |     MA_ZERO_OBJECT(pResampler);
54056 |
54057 |     if (pConfig == NULL) {
54058 |         return MA_INVALID_ARGS;
54059 |     }
54060 |
54061 |     pResampler->pHeap = pHeap;
54062 |     pResampler->format = pConfig->format;
54063 |     pResampler->channels = pConfig->channels;
54064 |     pResampler->sampleRateIn = pConfig->sampleRateIn;
54065 |     pResampler->sampleRateOut = pConfig->sampleRateOut;
54066 |
54067 |     result = ma_resampler_get_vtable(pConfig, pResampler, &pResampler->pBackendVTable,
> &pResampler->pBackendUserData);
54068 |     if (result != MA_SUCCESS) {
54069 |         return result;
54070 |     }
54071 |
54072 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onInit == NULL) {
54073 |         return MA_NOT_IMPLEMENTED; /* onInit not implemented. */
54074 |     }
54075 |
54076 |     result = pResampler->pBackendVTable->onInit(pResampler->pBackendUserData, pConfig, pHeap,
> &pResampler->pBackend);
54077 |     if (result != MA_SUCCESS) {
54078 |         return result;
54079 |     }
54080 |
54081 |     return MA_SUCCESS;
54082 | }
54083 |
54084 | MA_API ma_result ma_resampler_init(const ma_resampler_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_resampler* pResampler)
54085 | {
54086 |     ma_result result;
54087 |     size_t heapSizeInBytes;
54088 |     void* pHeap;
54089 |
54090 |     result = ma_resampler_get_heap_size(pConfig, &heapSizeInBytes);
54091 |     if (result != MA_SUCCESS) {

```

```

54092 |         return result;
54093 |     }
54094 |
54095 |     if (heapSizeInBytes > 0) {
54096 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
54097 |         if (pHeap == NULL) {
54098 |             return MA_OUT_OF_MEMORY;
54099 |         }
54100 |     } else {
54101 |         pHeap = NULL;
54102 |     }
54103 |
54104 |     result = ma_resampler_init_preallocated(pConfig, pHeap, pResampler);
54105 |     if (result != MA_SUCCESS) {
54106 |         ma_free(pHeap, pAllocationCallbacks);
54107 |         return result;
54108 |     }
54109 |
54110 |     pResampler->ownsHeap = MA_TRUE;
54111 |     return MA_SUCCESS;
54112 | }
54113 |
54114 | MA_API void ma_resampler_uninit(ma_resampler* pResampler, const ma_allocation_callbacks*
> pAllocationCallbacks)
54115 | {
54116 |     if (pResampler == NULL) {
54117 |         return;
54118 |     }
54119 |
54120 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onUninit == NULL) {
54121 |         return;
54122 |     }
54123 |
54124 |     pResampler->pBackendVTable->onUninit(pResampler->pBackendUserData, pResampler->pBackend,
> pAllocationCallbacks);
54125 |
54126 |     if (pResampler->ownsHeap) {
54127 |         ma_free(pResampler->pHeap, pAllocationCallbacks);
54128 |     }
54129 | }
54130 |
54131 | MA_API ma_result ma_resampler_process_pcm_frames(ma_resampler* pResampler, const void* pFramesIn,
> ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
54132 | {
54133 |     if (pResampler == NULL) {
54134 |         return MA_INVALID_ARGS;
54135 |     }
54136 |
54137 |     if (pFrameCountOut == NULL && pFrameCountIn == NULL) {
54138 |         return MA_INVALID_ARGS;
54139 |     }
54140 |
54141 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onProcess == NULL) {
54142 |         return MA_NOT_IMPLEMENTED;
54143 |     }
54144 |
54145 |     return pResampler->pBackendVTable->onProcess(pResampler->pBackendUserData, pResampler->pBackend,
> pFramesIn, pFrameCountIn, pFramesOut, pFrameCountOut);
54146 | }
54147 |
54148 | MA_API ma_result ma_resampler_set_rate(ma_resampler* pResampler, ma_uint32 sampleRateIn, ma_uint32
> sampleRateOut)
54149 | {
54150 |     ma_result result;
54151 |
54152 |     if (pResampler == NULL) {
54153 |         return MA_INVALID_ARGS;
54154 |     }
54155 |
54156 |     if (sampleRateIn == 0 || sampleRateOut == 0) {
54157 |         return MA_INVALID_ARGS;
54158 |     }
54159 |
54160 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onSetRate == NULL) {
54161 |         return MA_NOT_IMPLEMENTED;
54162 |     }
54163 |
54164 |     result = pResampler->pBackendVTable->onSetRate(pResampler->pBackendUserData,
> pResampler->pBackend, sampleRateIn, sampleRateOut);

```

```

54165 |     if (result != MA_SUCCESS) {
54166 |         return result;
54167 |     }
54168 |
54169 |     pResampler->sampleRateIn = sampleRateIn;
54170 |     pResampler->sampleRateOut = sampleRateOut;
54171 |
54172 |     return MA_SUCCESS;
54173 | }
54174 |
54175 | MA_API ma_result ma_resampler_set_rate_ratio(ma_resampler* pResampler, float ratio)
54176 | {
54177 |     ma_uint32 n;
54178 |     ma_uint32 d;
54179 |
54180 |     if (pResampler == NULL) {
54181 |         return MA_INVALID_ARGS;
54182 |     }
54183 |
54184 |     if (ratio <= 0) {
54185 |         return MA_INVALID_ARGS;
54186 |     }
54187 |
54188 |     d = 1000;
54189 |     n = (ma_uint32)(ratio * d);
54190 |
54191 |     if (n == 0) {
54192 |         return MA_INVALID_ARGS; /* Ratio too small. */
54193 |     }
54194 |
54195 |     MA_ASSERT(n != 0);
54196 |
54197 |     return ma_resampler_set_rate(pResampler, n, d);
54198 | }
54199 |
54200 | MA_API ma_uint64 ma_resampler_get_input_latency(const ma_resampler* pResampler)
54201 | {
54202 |     if (pResampler == NULL) {
54203 |         return 0;
54204 |     }
54205 |
54206 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onGetInputLatency == NULL)
> {
54207 |         return 0;
54208 |     }
54209 |
54210 |     return pResampler->pBackendVTable->onGetInputLatency(pResampler->pBackendUserData,
> pResampler->pBackend);
54211 | }
54212 |
54213 | MA_API ma_uint64 ma_resampler_get_output_latency(const ma_resampler* pResampler)
54214 | {
54215 |     if (pResampler == NULL) {
54216 |         return 0;
54217 |     }
54218 |
54219 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onGetOutputLatency ==
> NULL) {
54220 |         return 0;
54221 |     }
54222 |
54223 |     return pResampler->pBackendVTable->onGetOutputLatency(pResampler->pBackendUserData,
> pResampler->pBackend);
54224 | }
54225 |
54226 | MA_API ma_result ma_resampler_get_required_input_frame_count(const ma_resampler* pResampler,
> ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount)
54227 | {
54228 |     if (pInputFrameCount == NULL) {
54229 |         return MA_INVALID_ARGS;
54230 |     }
54231 |
54232 |     *pInputFrameCount = 0;
54233 |
54234 |     if (pResampler == NULL) {
54235 |         return MA_INVALID_ARGS;
54236 |     }
54237 |
54238 |     if (pResampler->pBackendVTable == NULL ||

```



```

> pResampler->pBackendVTable->onGetRequiredInputFrameCount == NULL) {
54239 |     return MA_NOT_IMPLEMENTED;
54240 | }
54241 |
54242 |     return pResampler->pBackendVTable->onGetRequiredInputFrameCount(pResampler->pBackendUserData,
> pResampler->pBackend, outputFrameCount, pInputFrameCount);
54243 | }
54244 |
54245 | MA_API ma_result ma_resampler_get_expected_output_frame_count(const ma_resampler* pResampler,
> ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount)
54246 | {
54247 |     if (pOutputFrameCount == NULL) {
54248 |         return MA_INVALID_ARGS;
54249 |     }
54250 |
54251 |     *pOutputFrameCount = 0;
54252 |
54253 |     if (pResampler == NULL) {
54254 |         return MA_INVALID_ARGS;
54255 |     }
54256 |
54257 |     if (pResampler->pBackendVTable == NULL ||
> pResampler->pBackendVTable->onGetExpectedOutputFrameCount == NULL) {
54258 |         return MA_NOT_IMPLEMENTED;
54259 |     }
54260 |
54261 |     return pResampler->pBackendVTable->onGetExpectedOutputFrameCount(pResampler->pBackendUserData,
> pResampler->pBackend, inputFrameCount, pOutputFrameCount);
54262 | }
54263 |
54264 | MA_API ma_result ma_resampler_reset(ma_resampler* pResampler)
54265 | {
54266 |     if (pResampler == NULL) {
54267 |         return MA_INVALID_ARGS;
54268 |     }
54269 |
54270 |     if (pResampler->pBackendVTable == NULL || pResampler->pBackendVTable->onReset == NULL) {
54271 |         return MA_NOT_IMPLEMENTED;
54272 |     }
54273 |
54274 |     return pResampler->pBackendVTable->onReset(pResampler->pBackendUserData, pResampler->pBackend);
54275 | }
54276 |
54277 | /*****
> *****/
54278 |
54279 | Channel Conversion
54280 |
54281 | *****/
> *****/
54282 | #ifndef MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT
54283 | #define MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT 12
54284 | #endif
54285 |
54286 | #define MA_PLANE_LEFT 0
54287 | #define MA_PLANE_RIGHT 1
54288 | #define MA_PLANE_FRONT 2
54289 | #define MA_PLANE_BACK 3
54290 | #define MA_PLANE_BOTTOM 4
54291 | #define MA_PLANE_TOP 5
54292 |
54293 | static float g_maChannelPlaneRatios[MA_CHANNEL_POSITION_COUNT][6] = {
54294 |     { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_NONE */
54295 |     { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_MONO */
54296 |     { 0.5f, 0.0f, 0.5f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_FRONT_LEFT */
54297 |     { 0.0f, 0.5f, 0.5f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_FRONT_RIGHT */
54298 |     { 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_FRONT_CENTER */
54299 |     { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_LFE */
54300 |     { 0.5f, 0.0f, 0.0f, 0.5f, 0.0f, 0.0f}, /* MA_CHANNEL_BACK_LEFT */
54301 |     { 0.0f, 0.5f, 0.0f, 0.5f, 0.0f, 0.0f}, /* MA_CHANNEL_BACK_RIGHT */
54302 |     { 0.25f, 0.0f, 0.75f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_FRONT_LEFT_CENTER */
54303 |     { 0.0f, 0.25f, 0.75f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_FRONT_RIGHT_CENTER */
54304 |     { 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f}, /* MA_CHANNEL_BACK_CENTER */
54305 |     { 1.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_SIDE_LEFT */
54306 |     { 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_SIDE_RIGHT */
54307 |     { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f}, /* MA_CHANNEL_TOP_CENTER */
54308 |     { 0.33f, 0.0f, 0.33f, 0.0f, 0.0f, 0.34f}, /* MA_CHANNEL_TOP_FRONT_LEFT */
54309 |     { 0.0f, 0.0f, 0.5f, 0.0f, 0.0f, 0.5f}, /* MA_CHANNEL_TOP_FRONT_CENTER */
54310 |     { 0.0f, 0.33f, 0.33f, 0.0f, 0.0f, 0.34f}, /* MA_CHANNEL_TOP_FRONT_RIGHT */

```

```

54311 | { 0.33f, 0.0f, 0.0f, 0.33f, 0.0f, 0.34f}, /* MA_CHANNEL_TOP_BACK_LEFT */
54312 | { 0.0f, 0.0f, 0.0f, 0.5f, 0.0f, 0.5f}, /* MA_CHANNEL_TOP_BACK_CENTER */
54313 | { 0.0f, 0.33f, 0.0f, 0.33f, 0.0f, 0.34f}, /* MA_CHANNEL_TOP_BACK_RIGHT */
54314 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_0 */
54315 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_1 */
54316 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_2 */
54317 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_3 */
54318 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_4 */
54319 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_5 */
54320 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_6 */
54321 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_7 */
54322 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_8 */
54323 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_9 */
54324 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_10 */
54325 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_11 */
54326 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_12 */
54327 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_13 */
54328 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_14 */
54329 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_15 */
54330 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_16 */
54331 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_17 */
54332 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_18 */
54333 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_19 */
54334 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_20 */
54335 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_21 */
54336 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_22 */
54337 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_23 */
54338 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_24 */
54339 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_25 */
54340 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_26 */
54341 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_27 */
54342 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_28 */
54343 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_29 */
54344 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_30 */
54345 | { 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f}, /* MA_CHANNEL_AUX_31 */
54346 | };
54347 |
54348 | static float ma_calculate_channel_position_rectangular_weight(ma_channel channelPositionA,
> ma_channel channelPositionB)
54349 | {
54350 |     /*
54351 |     Imagine the following simplified example: You have a single input speaker which is the
> front/left speaker which you want to convert to
54352 |     the following output configuration:
54353 |
54354 |     - front/left
54355 |     - side/left
54356 |     - back/left
54357 |
54358 |     The front/left output is easy - it the same speaker position so it receives the full
> contribution of the front/left input. The amount
54359 |     of contribution to apply to the side/left and back/left speakers, however, is a bit more
> complicated.
54360 |
54361 |     Imagine the front/left speaker as emitting audio from two planes - the front plane and the left
> plane. You can think of the front/left
54362 |     speaker emitting half of its total volume from the front, and the other half from the left.
> Since part of its volume is being emitted
54363 |     from the left side, and the side/left and back/left channels also emit audio from the left
> plane, one would expect that they would
54364 |     receive some amount of contribution from front/left speaker. The amount of contribution depends
> on how many planes are shared between
54365 |     the two speakers. Note that in the examples below I've added a top/front/left speaker as an
> example just to show how the math works
54366 |     across 3 spatial dimensions.
54367 |
54368 |     The first thing to do is figure out how each speaker's volume is spread over each of plane:
54369 |     - front/left:    2 planes (front and left)    = 1/2 = half its total volume on each plane
54370 |     - side/left:     1 plane (left only)          = 1/1 = entire volume from left plane
54371 |     - back/left:     2 planes (back and left)      = 1/2 = half its total volume on each plane
54372 |     - top/front/left: 3 planes (top, front and left) = 1/3 = one third its total volume on each
> plane
54373 |
54374 |     The amount of volume each channel contributes to each of its planes is what controls how much it
> is willing to given and take to other
54375 |     channels on the same plane. The volume that is willing to the given by one channel is multiplied
> by the volume that is willing to be
54376 |     taken by the other to produce the final contribution.
54377 |     */

```

```

54378 |
54379 |     /* Contribution = Sum(Volume to Give * Volume to Take) */
54380 |     float contribution =
54381 |         g_maChannelPlaneRatios[channelPositionA][0] * g_maChannelPlaneRatios[channelPositionB][0] +
54382 |         g_maChannelPlaneRatios[channelPositionA][1] * g_maChannelPlaneRatios[channelPositionB][1] +
54383 |         g_maChannelPlaneRatios[channelPositionA][2] * g_maChannelPlaneRatios[channelPositionB][2] +
54384 |         g_maChannelPlaneRatios[channelPositionA][3] * g_maChannelPlaneRatios[channelPositionB][3] +
54385 |         g_maChannelPlaneRatios[channelPositionA][4] * g_maChannelPlaneRatios[channelPositionB][4] +
54386 |         g_maChannelPlaneRatios[channelPositionA][5] * g_maChannelPlaneRatios[channelPositionB][5];
54387 |
54388 |     return contribution;
54389 | }
54390 |
54391 | MA_API ma_channel_converter_config ma_channel_converter_config_init(ma_format format, ma_uint32
> channelsIn, const ma_channel* pChannelMapIn, ma_uint32 channelsOut, const ma_channel*
> pChannelMapOut, ma_channel_mix_mode mixingMode)
54392 | {
54393 |     ma_channel_converter_config config;
54394 |
54395 |     MA_ZERO_OBJECT(&config);
54396 |     config.format = format;
54397 |     config.channelsIn = channelsIn;
54398 |     config.channelsOut = channelsOut;
54399 |     config.pChannelMapIn = pChannelMapIn;
54400 |     config.pChannelMapOut = pChannelMapOut;
54401 |     config.mixingMode = mixingMode;
54402 |
54403 |     return config;
54404 | }
54405 |
54406 | static ma_int32 ma_channel_converter_float_to_fixed(float x)
54407 | {
54408 |     return (ma_int32)(x * (1<<MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT));
54409 | }
54410 |
54411 | static ma_uint32 ma_channel_map_get_spatial_channel_count(const ma_channel* pChannelMap, ma_uint32
> channels)
54412 | {
54413 |     ma_uint32 spatialChannelCount = 0;
54414 |     ma_uint32 iChannel;
54415 |
54416 |     MA_ASSERT(pChannelMap != NULL);
54417 |     MA_ASSERT(channels > 0);
54418 |
54419 |     for (iChannel = 0; iChannel < channels; ++iChannel) {
54420 |         if (ma_is_spatial_channel_position(ma_channel_map_get_channel(pChannelMap, channels,
> iChannel))) {
54421 |             spatialChannelCount++;
54422 |         }
54423 |     }
54424 |
54425 |     return spatialChannelCount;
54426 | }
54427 |
54428 | static ma_bool32 ma_is_spatial_channel_position(ma_channel channelPosition)
54429 | {
54430 |     int i;
54431 |
54432 |     if (channelPosition == MA_CHANNEL_NONE || channelPosition == MA_CHANNEL_MONO || channelPosition
> == MA_CHANNEL_LFE) {
54433 |         return MA_FALSE;
54434 |     }
54435 |
54436 |     if (channelPosition >= MA_CHANNEL_AUX_0) {
54437 |         return MA_FALSE;
54438 |     }
54439 |
54440 |     for (i = 0; i < 6; ++i) { /* Each side of a cube. */
54441 |         if (g_maChannelPlaneRatios[channelPosition][i] != 0) {
54442 |             return MA_TRUE;
54443 |         }
54444 |     }
54445 |
54446 |     return MA_FALSE;
54447 | }
54448 |
54449 |
54450 | static ma_bool32 ma_channel_map_is_passthrough(const ma_channel* pChannelMapIn, ma_uint32
> channelsIn, const ma_channel* pChannelMapOut, ma_uint32 channelsOut)

```

```

54451 | {
54452 |     if (channelsOut == channelsIn) {
54453 |         return ma_channel_map_is_equal(pChannelMapOut, pChannelMapIn, channelsOut);
54454 |     } else {
54455 |         return MA_FALSE;    /* Channel counts differ, so cannot be a passthrough. */
54456 |     }
54457 | }
54458 |
54459 | static ma_channel_conversion_path ma_channel_map_get_conversion_path(const ma_channel*
> pChannelMapIn, ma_uint32 channelsIn, const ma_channel* pChannelMapOut, ma_uint32 channelsOut,
> ma_channel_mix_mode mode)
54460 | {
54461 |     if (ma_channel_map_is_passthrough(pChannelMapIn, channelsIn, pChannelMapOut, channelsOut)) {
54462 |         return ma_channel_conversion_path_passthrough;
54463 |     }
54464 |
54465 |     if (channelsOut == 1 && (pChannelMapOut == NULL || pChannelMapOut[0] == MA_CHANNEL_MONO)) {
54466 |         return ma_channel_conversion_path_mono_out;
54467 |     }
54468 |
54469 |     if (channelsIn == 1 && (pChannelMapIn == NULL || pChannelMapIn[0] == MA_CHANNEL_MONO)) {
54470 |         return ma_channel_conversion_path_mono_in;
54471 |     }
54472 |
54473 |     if (mode == ma_channel_mix_mode_custom_weights) {
54474 |         return ma_channel_conversion_path_weights;
54475 |     }
54476 |
54477 |     /*
54478 |     We can use a simple shuffle if both channel maps have the same channel count and all channel
54479 |     positions are present in both.
54480 |     */
54481 |     if (channelsIn == channelsOut) {
54482 |         ma_uint32 iChannelIn;
54483 |         ma_bool32 areAllChannelPositionsPresent = MA_TRUE;
54484 |         for (iChannelIn = 0; iChannelIn < channelsIn; ++iChannelIn) {
54485 |             ma_bool32 isInputChannelPositionInOutput =
> ma_channel_map_contains_channel_position(channelsOut, pChannelMapOut,
> ma_channel_map_get_channel(pChannelMapIn, channelsIn, iChannelIn));
54486 |             if (!isInputChannelPositionInOutput) {
54487 |                 areAllChannelPositionsPresent = MA_FALSE;
54488 |                 break;
54489 |             }
54490 |         }
54491 |
54492 |         if (areAllChannelPositionsPresent) {
54493 |             return ma_channel_conversion_path_shuffle;
54494 |         }
54495 |     }
54496 |
54497 |     /* Getting here means we'll need to use weights. */
54498 |     return ma_channel_conversion_path_weights;
54499 | }
54500 |
54501 |
54502 | static ma_result ma_channel_map_build_shuffle_table(const ma_channel* pChannelMapIn, ma_uint32
> channelCountIn, const ma_channel* pChannelMapOut, ma_uint32 channelCountOut, ma_uint8*
> pShuffleTable)
54503 | {
54504 |     ma_uint32 iChannelIn;
54505 |     ma_uint32 iChannelOut;
54506 |
54507 |     if (pShuffleTable == NULL || channelCountIn == 0 || channelCountOut == 0) {
54508 |         return MA_INVALID_ARGS;
54509 |     }
54510 |
54511 |     /*
54512 |     When building the shuffle table we just do a 1:1 mapping based on the first occurrence of a
> channel. If the
54513 |     input channel has more than one occurrence of a channel position, the second one will be
> ignored.
54514 |     */
54515 |     for (iChannelOut = 0; iChannelOut < channelCountOut; iChannelOut += 1) {
54516 |         ma_channel channelOut;
54517 |
54518 |         /* Default to MA_CHANNEL_INDEX_NULL so that if a mapping is not found it'll be set
> appropriately. */
54519 |         pShuffleTable[iChannelOut] = MA_CHANNEL_INDEX_NULL;
54520 |     }

```

```

54521 |     channelOut = ma_channel_map_get_channel(pChannelMapOut, channelCountOut, iChannelOut);
54522 |     for (iChannelIn = 0; iChannelIn < channelCountIn; iChannelIn += 1) {
54523 |         ma_channel channelIn;
54524 |
54525 |         channelIn = ma_channel_map_get_channel(pChannelMapIn, channelCountIn, iChannelIn);
54526 |         if (channelOut == channelIn) {
54527 |             pShuffleTable[iChannelOut] = (ma_uint8)iChannelIn;
54528 |             break;
54529 |         }
54530 |
54531 |         /*
54532 |         Getting here means the channels don't exactly match, but we are going to support some
54533 |         relaxed matching for practicality. If, for example, there are two stereo channel maps,
54534 |         but one uses front left/right and the other uses side left/right, it makes logical
54535 |         sense to just map these. The way we'll do it is we'll check if there is a logical
54536 |         corresponding mapping, and if so, apply it, but we will *not* break from the loop,
54537 |         thereby giving the loop a chance to find an exact match later which will take priority.
54538 |         */
54539 |         switch (channelOut)
54540 |         {
54541 |             /* Left channels. */
54542 |             case MA_CHANNEL_FRONT_LEFT:
54543 |             case MA_CHANNEL_SIDE_LEFT:
54544 |             {
54545 |                 switch (channelIn) {
54546 |                     case MA_CHANNEL_FRONT_LEFT:
54547 |                     case MA_CHANNEL_SIDE_LEFT:
54548 |                     {
54549 |                         pShuffleTable[iChannelOut] = (ma_uint8)iChannelIn;
54550 |                         } break;
54551 |                     }
54552 |                 } break;
54553 |
54554 |             /* Right channels. */
54555 |             case MA_CHANNEL_FRONT_RIGHT:
54556 |             case MA_CHANNEL_SIDE_RIGHT:
54557 |             {
54558 |                 switch (channelIn) {
54559 |                     case MA_CHANNEL_FRONT_RIGHT:
54560 |                     case MA_CHANNEL_SIDE_RIGHT:
54561 |                     {
54562 |                         pShuffleTable[iChannelOut] = (ma_uint8)iChannelIn;
54563 |                         } break;
54564 |                     }
54565 |                 } break;
54566 |
54567 |             default: break;
54568 |         }
54569 |     }
54570 | }
54571 |
54572 | return MA_SUCCESS;
54573 | }
54574 |
54575 |
54576 | static void ma_channel_map_apply_shuffle_table_u8(ma_uint8* pFramesOut, ma_uint32 channelsOut, const
> ma_uint8* pFramesIn, ma_uint32 channelsIn, ma_uint64 frameCount, const ma_uint8* pShuffleTable)
54577 | {
54578 |     ma_uint64 iFrame;
54579 |     ma_uint32 iChannelOut;
54580 |
54581 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54582 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54583 |             ma_uint8 iChannelIn = pShuffleTable[iChannelOut];
54584 |             if (iChannelIn < channelsIn) { /* For safety, and to deal with MA_CHANNEL_INDEX_NULL.
> */
54585 |                 pFramesOut[iChannelOut] = pFramesIn[iChannelIn];
54586 |             } else {
54587 |                 pFramesOut[iChannelOut] = 0;
54588 |             }
54589 |         }
54590 |
54591 |         pFramesOut += channelsOut;
54592 |         pFramesIn += channelsIn;
54593 |     }
54594 | }
54595 |
54596 | static void ma_channel_map_apply_shuffle_table_s16(ma_int16* pFramesOut, ma_uint32 channelsOut,
> const ma_int16* pFramesIn, ma_uint32 channelsIn, ma_uint64 frameCount, const ma_uint8*

```

```

> pShuffleTable)
54597 | {
54598 |     ma_uint64 iFrame;
54599 |     ma_uint32 iChannelOut;
54600 |
54601 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54602 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54603 |             ma_uint8 iChannelIn = pShuffleTable[iChannelOut];
54604 |             if (iChannelIn < channelsIn) { /* For safety, and to deal with MA_CHANNEL_INDEX_NULL.
> */
54605 |                 pFramesOut[iChannelOut] = pFramesIn[iChannelIn];
54606 |             } else {
54607 |                 pFramesOut[iChannelOut] = 0;
54608 |             }
54609 |         }
54610 |
54611 |         pFramesOut += channelsOut;
54612 |         pFramesIn += channelsIn;
54613 |     }
54614 | }
54615 |
54616 | static void ma_channel_map_apply_shuffle_table_s24(ma_uint8* pFramesOut, ma_uint32 channelsOut,
> const ma_uint8* pFramesIn, ma_uint32 channelsIn, ma_uint64 frameCount, const ma_uint8*
> pShuffleTable)
54617 | {
54618 |     ma_uint64 iFrame;
54619 |     ma_uint32 iChannelOut;
54620 |
54621 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54622 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54623 |             ma_uint8 iChannelIn = pShuffleTable[iChannelOut];
54624 |             if (iChannelIn < channelsIn) { /* For safety, and to deal with MA_CHANNEL_INDEX_NULL.
> */
54625 |                 pFramesOut[iChannelOut*3 + 0] = pFramesIn[iChannelIn*3 + 0];
54626 |                 pFramesOut[iChannelOut*3 + 1] = pFramesIn[iChannelIn*3 + 1];
54627 |                 pFramesOut[iChannelOut*3 + 2] = pFramesIn[iChannelIn*3 + 2];
54628 |             } else {
54629 |                 pFramesOut[iChannelOut*3 + 0] = 0;
54630 |             } pFramesOut[iChannelOut*3 + 1] = 0;
54631 |             pFramesOut[iChannelOut*3 + 2] = 0;
54632 |
54633 |             pFramesOut += channelsOut*3;
54634 |             pFramesIn += channelsIn*3;
54635 |         }
54636 |     }
54637 | }
54638 | static void ma_channel_map_apply_shuffle_table_s32(ma_int32* pFramesOut, ma_uint32 channelsOut,
> const ma_int32* pFramesIn, ma_uint32 channelsIn, ma_uint64 frameCount, const ma_uint8*
> pShuffleTable)
54639 | {
54640 |     ma_uint64 iFrame;
54641 |     ma_uint32 iChannelOut;
54642 |
54643 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54644 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54645 |             ma_uint8 iChannelIn = pShuffleTable[iChannelOut];
54646 |             if (iChannelIn < channelsIn) { /* For safety, and to deal with MA_CHANNEL_INDEX_NULL.
> */
54647 |                 pFramesOut[iChannelOut] = pFramesIn[iChannelIn];
54648 |             } else {
54649 |                 pFramesOut[iChannelOut] = 0;
54650 |             }
54651 |         }
54652 |
54653 |         pFramesOut += channelsOut;
54654 |         pFramesIn += channelsIn;
54655 |     }
54656 | }
54657 |
54658 | static void ma_channel_map_apply_shuffle_table_f32(float* pFramesOut, ma_uint32 channelsOut, const
> float* pFramesIn, ma_uint32 channelsIn, ma_uint64 frameCount, const ma_uint8* pShuffleTable)
54659 | {
54660 |     ma_uint64 iFrame;
54661 |     ma_uint32 iChannelOut;
54662 |
54663 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54664 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54665 |             ma_uint8 iChannelIn = pShuffleTable[iChannelOut];
54666 |             if (iChannelIn < channelsIn) { /* For safety, and to deal with MA_CHANNEL_INDEX_NULL.

```

```

> */
54667 |         pFramesOut[iChannelOut] = pFramesIn[iChannelIn];
54668 |     } else {
54669 |         pFramesOut[iChannelOut] = 0;
54670 |     }
54671 | }
54672 |
54673 |     pFramesOut += channelsOut;
54674 |     pFramesIn += channelsIn;
54675 | }
54676 | }
54677 |
54678 | static ma_result ma_channel_map_apply_shuffle_table(void* pFramesOut, ma_uint32 channelsOut, const
> void* pFramesIn, ma_uint32 channelsIn, ma_uint64 frameCount, const ma_uint8* pShuffleTable,
> ma_format format)
54679 | {
54680 |     if (pFramesOut == NULL || pFramesIn == NULL || channelsOut == 0 || pShuffleTable == NULL) {
54681 |         return MA_INVALID_ARGS;
54682 |     }
54683 |
54684 |     switch (format)
54685 |     {
54686 |         case ma_format_u8:
54687 |         {
54688 |             ma_channel_map_apply_shuffle_table_u8((ma_uint8*)pFramesOut, channelsOut, (const
> ma_uint8*)pFramesIn, channelsIn, frameCount, pShuffleTable);
54689 |         } break;
54690 |
54691 |         case ma_format_s16:
54692 |         {
54693 |             ma_channel_map_apply_shuffle_table_s16((ma_int16*)pFramesOut, channelsOut, (const
> ma_int16*)pFramesIn, channelsIn, frameCount, pShuffleTable);
54694 |         } break;
54695 |
54696 |         case ma_format_s24:
54697 |         {
54698 |             ma_channel_map_apply_shuffle_table_s24((ma_uint8*)pFramesOut, channelsOut, (const
> ma_uint8*)pFramesIn, channelsIn, frameCount, pShuffleTable);
54699 |         } break;
54700 |
54701 |         case ma_format_s32:
54702 |         {
54703 |             ma_channel_map_apply_shuffle_table_s32((ma_int32*)pFramesOut, channelsOut, (const
> ma_int32*)pFramesIn, channelsIn, frameCount, pShuffleTable);
54704 |         } break;
54705 |
54706 |         case ma_format_f32:
54707 |         {
54708 |             ma_channel_map_apply_shuffle_table_f32((float*)pFramesOut, channelsOut, (const
> float*)pFramesIn, channelsIn, frameCount, pShuffleTable);
54709 |         } break;
54710 |
54711 |         default: return MA_INVALID_ARGS;    /* Unknown format. */
54712 |     }
54713 |
54714 |     return MA_SUCCESS;
54715 | }
54716 |
54717 | static ma_result ma_channel_map_apply_mono_out_f32(float* pFramesOut, const float* pFramesIn, const
> ma_channel* pChannelMapIn, ma_uint32 channelsIn, ma_uint64 frameCount)
54718 | {
54719 |     ma_uint64 iFrame;
54720 |     ma_uint32 iChannelIn;
54721 |     ma_uint32 accumulationCount;
54722 |
54723 |     if (pFramesOut == NULL || pFramesIn == NULL || channelsIn == 0) {
54724 |         return MA_INVALID_ARGS;
54725 |     }
54726 |
54727 |     /* In this case the output stream needs to be the average of all channels, ignoring NONE. */
54728 |
54729 |     /* A quick pre-processing step to get the accumulation counter since we're ignoring NONE
> channels. */
54730 |     accumulationCount = 0;
54731 |     for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
54732 |         if (ma_channel_map_get_channel(pChannelMapIn, channelsIn, iChannelIn) != MA_CHANNEL_NONE) {
54733 |             accumulationCount += 1;
54734 |         }
54735 |     }

```



```

54736 |
54737 |     if (accumulationCount > 0) { /* <-- Prevent a division by zero. */
54738 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54739 |             float accumulation = 0;
54740 |
54741 |             for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
54742 |                 ma_channel channelIn = ma_channel_map_get_channel(pChannelMapIn, channelsIn,
> iChannelIn);
54743 |                 if (channelIn != MA_CHANNEL_NONE) {
54744 |                     accumulation += pFramesIn[iChannelIn];
54745 |                 }
54746 |             }
54747 |
54748 |             pFramesOut[0] = accumulation / accumulationCount;
54749 |             pFramesOut += 1;
54750 |             pFramesIn += channelsIn;
54751 |         }
54752 |     } else {
54753 |         ma_silence_pcm_frames(pFramesOut, frameCount, ma_format_f32, 1);
54754 |     }
54755 |
54756 |     return MA_SUCCESS;
54757 | }
54758 |
54759 | static ma_result ma_channel_map_apply_mono_in_f32(float* MA_RESTRICT pFramesOut, const ma_channel*
> pChannelMapOut, ma_uint32 channelsOut, const float* MA_RESTRICT pFramesIn, ma_uint64 frameCount,
> ma_mono_expansion_mode monoExpansionMode)
54760 | {
54761 |     ma_uint64 iFrame;
54762 |     ma_uint32 iChannelOut;
54763 |
54764 |     if (pFramesOut == NULL || channelsOut == 0 || pFramesIn == NULL) {
54765 |         return MA_INVALID_ARGS;
54766 |     }
54767 |
54768 |     /* Note that the MA_CHANNEL_NONE channel must be ignored in all cases. */
54769 |     switch (monoExpansionMode)
54770 |     {
54771 |         case ma_mono_expansion_mode_average:
54772 |         {
54773 |             float weight;
54774 |             ma_uint32 validChannelCount = 0;
54775 |
54776 |             for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54777 |                 ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut, channelsOut,
> iChannelOut);
54778 |                 if (channelOut != MA_CHANNEL_NONE) {
54779 |                     validChannelCount += 1;
54780 |                 }
54781 |             }
54782 |
54783 |             weight = 1.0f / validChannelCount;
54784 |
54785 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54786 |                 for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54787 |                     ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut, channelsOut,
> iChannelOut);
54788 |                     if (channelOut != MA_CHANNEL_NONE) {
54789 |                         pFramesOut[iChannelOut] = pFramesIn[0] * weight;
54790 |                     }
54791 |                 }
54792 |
54793 |                 pFramesOut += channelsOut;
54794 |                 pFramesIn += 1;
54795 |             }
54796 |             } break;
54797 |
54798 |         case ma_mono_expansion_mode_stereo_only:
54799 |         {
54800 |             if (channelsOut >= 2) {
54801 |                 ma_uint32 iChannelLeft = (ma_uint32)-1;
54802 |                 ma_uint32 iChannelRight = (ma_uint32)-1;
54803 |
54804 |                 /*
54805 |                 We first need to find our stereo channels. We prefer front-left and front-right, but
54806 |                 if they're not available, we'll also try side-left and side-right. If neither are
54807 |                 available we'll fall through to the default case below.
54808 |                 */
54809 |                 for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {

```

```

54810 |         ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut, channelsOut,
> iChannelOut);
54811 |         if (channelOut == MA_CHANNEL_SIDE_LEFT) {
54812 |             iChannelLeft = iChannelOut;
54813 |         }
54814 |         if (channelOut == MA_CHANNEL_SIDE_RIGHT) {
54815 |             iChannelRight = iChannelOut;
54816 |         }
54817 |     }
54818 |
54819 |     for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54820 |         ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut, channelsOut,
> iChannelOut);
54821 |         if (channelOut == MA_CHANNEL_FRONT_LEFT) {
54822 |             iChannelLeft = iChannelOut;
54823 |         }
54824 |         if (channelOut == MA_CHANNEL_FRONT_RIGHT) {
54825 |             iChannelRight = iChannelOut;
54826 |         }
54827 |     }
54828 |
54829 |
54830 |     if (iChannelLeft != (ma_uint32)-1 && iChannelRight != (ma_uint32)-1) {
54831 |         /* We found our stereo channels so we can duplicate the signal across those
> channels. */
54832 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54833 |             for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54834 |                 ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut,
> channelsOut, iChannelOut);
54835 |                 if (channelOut != MA_CHANNEL_NONE) {
54836 |                     if (iChannelOut == iChannelLeft || iChannelOut == iChannelRight) {
54837 |                         pFramesOut[iChannelOut] = pFramesIn[0];
54838 |                     } else {
54839 |                         pFramesOut[iChannelOut] = 0.0f;
54840 |                     }
54841 |                 }
54842 |             }
54843 |
54844 |             pFramesOut += channelsOut;
54845 |             pFramesIn += 1;
54846 |         }
54847 |
54848 |         break; /* Get out of the switch. */
54849 |     } else {
54850 |         /* Fallthrough. Does not have left and right channels. */
54851 |         goto default_handler;
54852 |     }
54853 | } else {
54854 |     /* Fallthrough. Does not have stereo channels. */
54855 |     goto default_handler;
54856 | }
54857 | }; /* Fallthrough. See comments above. */
54858 |
54859 | case ma_mono_expansion_mode_duplicate:
54860 | default:
54861 | {
54862 |     default_handler:
54863 |     {
54864 |         if (channelsOut <= MA_MAX_CHANNELS) {
54865 |             ma_bool32 hasEmptyChannel = MA_FALSE;
54866 |             ma_channel channelPositions[MA_MAX_CHANNELS];
54867 |             for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54868 |                 channelPositions[iChannelOut] = ma_channel_map_get_channel(pChannelMapOut,
> channelsOut, iChannelOut);
54869 |                 if (channelPositions[iChannelOut] == MA_CHANNEL_NONE) {
54870 |                     hasEmptyChannel = MA_TRUE;
54871 |                 }
54872 |             }
54873 |
54874 |             if (hasEmptyChannel == MA_FALSE) {
54875 |                 /*
54876 |                 Faster path when there's no MA_CHANNEL_NONE channel positions. This should
> hopefully
54877 |                 help the compiler with auto-vectorization.
54878 |                 */
54879 |                 if (channelsOut == 2) {
54880 |                     #if defined(MA_SUPPORT_SSE2)
54881 |                         if (ma_has_sse2()) {
54882 |                             /* We want to do two frames in each iteration. */

```

```

54883 |         ma_uint64 unrolledFrameCount = frameCount >> 1;
54884 |
54885 |         for (iFrame = 0; iFrame < unrolledFrameCount; iFrame += 1) {
54886 |             __m128 in0 = __mm_set1_ps(pFramesIn[iFrame*2 + 0]);
54887 |             __m128 in1 = __mm_set1_ps(pFramesIn[iFrame*2 + 1]);
54888 |             __mm_storeu_ps(&pFramesOut[iFrame*4 + 0], __mm_shuffle_ps(in0,
> in1, _MM_SHUFFLE(0, 0, 0, 0)));
54889 |         }
54890 |
54891 |         /* Tail. */
54892 |         iFrame = unrolledFrameCount << 1;
54893 |         goto generic_on_fastpath;
54894 |     } else
54895 | #endif
54896 |     {
54897 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54898 |             for (iChannelOut = 0; iChannelOut < 2; iChannelOut += 1) {
54899 |                 pFramesOut[iFrame*2 + iChannelOut] = pFramesIn[iFrame];
54900 |             }
54901 |         }
54902 |     }
54903 | } else if (channelsOut == 6) {
54904 | #if defined(MA_SUPPORT_SSE2)
54905 |     if (ma_has_sse2()) {
54906 |         /* We want to do two frames in each iteration so we can have a
> multiple of 4 samples. */
54907 |         ma_uint64 unrolledFrameCount = frameCount >> 1;
54908 |
54909 |         for (iFrame = 0; iFrame < unrolledFrameCount; iFrame += 1) {
54910 |             __m128 in0 = __mm_set1_ps(pFramesIn[iFrame*2 + 0]);
54911 |             __m128 in1 = __mm_set1_ps(pFramesIn[iFrame*2 + 1]);
54912 |
54913 |             __mm_storeu_ps(&pFramesOut[iFrame*12 + 0], in0);
54914 |             __mm_storeu_ps(&pFramesOut[iFrame*12 + 4], __mm_shuffle_ps(in0,
> in1, _MM_SHUFFLE(0, 0, 0, 0)));
54915 |             __mm_storeu_ps(&pFramesOut[iFrame*12 + 8], in1);
54916 |         }
54917 |
54918 |         /* Tail. */
54919 |         iFrame = unrolledFrameCount << 1;
54920 |         goto generic_on_fastpath;
54921 |     } else
54922 | #endif
54923 |     {
54924 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54925 |             for (iChannelOut = 0; iChannelOut < 6; iChannelOut += 1) {
54926 |                 pFramesOut[iFrame*6 + iChannelOut] = pFramesIn[iFrame];
54927 |             }
54928 |         }
54929 |     }
54930 | } else if (channelsOut == 8) {
54931 | #if defined(MA_SUPPORT_SSE2)
54932 |     if (ma_has_sse2()) {
54933 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54934 |             __m128 in = __mm_set1_ps(pFramesIn[iFrame]);
54935 |             __mm_storeu_ps(&pFramesOut[iFrame*8 + 0], in);
54936 |             __mm_storeu_ps(&pFramesOut[iFrame*8 + 4], in);
54937 |         }
54938 |     } else
54939 | #endif
54940 |     {
54941 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54942 |             for (iChannelOut = 0; iChannelOut < 8; iChannelOut += 1) {
54943 |                 pFramesOut[iFrame*8 + iChannelOut] = pFramesIn[iFrame];
54944 |             }
54945 |         }
54946 |     }
54947 | } else {
54948 |     iFrame = 0;
54949 |
54950 |     #if defined(MA_SUPPORT_SSE2) /* For silencing a warning with non-x86
> builds. */
54951 |     generic_on_fastpath:
54952 |     #endif
54953 |     {
54954 |         for (; iFrame < frameCount; iFrame += 1) {
54955 |             for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut +=
> 1) {
54956 |                 pFramesOut[iFrame*channelsOut + iChannelOut] =

```

```

> pFramesIn[iFrame];
54957 |         }
54958 |     }
54959 | }
54960 | }
54961 | } else {
54962 |     /* Slow path. Need to handle MA_CHANNEL_NONE. */
54963 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54964 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54965 |             if (channelPositions[iChannelOut] != MA_CHANNEL_NONE) {
54966 |                 pFramesOut[iFrame*channelsOut + iChannelOut] =
> pFramesIn[iFrame];
54967 |             }
54968 |         }
54969 |     }
54970 | }
54971 | } else {
54972 |     /* Slow path. Too many channels to store on the stack. */
54973 |     for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
54974 |         for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
54975 |             ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut,
> channelsOut, iChannelOut);
54976 |             if (channelOut != MA_CHANNEL_NONE) {
54977 |                 pFramesOut[iFrame*channelsOut + iChannelOut] = pFramesIn[iFrame];
54978 |             }
54979 |         }
54980 |     }
54981 | }
54982 | } break;
54983 | }
54984 | }
54985 |
54986 | return MA_SUCCESS;
54987 | }
54988 |
54989 | static void ma_channel_map_apply_f32(float* pFramesOut, const ma_channel* pChannelMapOut, ma_uint32
> channelsOut, const float* pFramesIn, const ma_channel* pChannelMapIn, ma_uint32 channelsIn,
> ma_uint64 frameCount, ma_channel_mix_mode mode, ma_mono_expansion_mode monoExpansionMode)
54990 | {
54991 |     ma_channel_conversion_path conversionPath = ma_channel_map_get_conversion_path(pChannelMapIn,
> channelsIn, pChannelMapOut, channelsOut, mode);
54992 |
54993 |     /* Optimized Path: Passthrough */
54994 |     if (conversionPath == ma_channel_conversion_path_passthrough) {
54995 |         ma_copy_pcm_frames(pFramesOut, pFramesIn, frameCount, ma_format_f32, channelsOut);
54996 |         return;
54997 |     }
54998 |
54999 |     /* Special Path: Mono Output. */
55000 |     if (conversionPath == ma_channel_conversion_path_mono_out) {
55001 |         ma_channel_map_apply_mono_out_f32(pFramesOut, pFramesIn, pChannelMapIn, channelsIn,
> frameCount);
55002 |         return;
55003 |     }
55004 |
55005 |     /* Special Path: Mono Input. */
55006 |     if (conversionPath == ma_channel_conversion_path_mono_in) {
55007 |         ma_channel_map_apply_mono_in_f32(pFramesOut, pChannelMapOut, channelsOut, pFramesIn,
> frameCount, monoExpansionMode);
55008 |         return;
55009 |     }
55010 |
55011 |     /* Getting here means we aren't running on an optimized conversion path. */
55012 |     if (channelsOut <= MA_MAX_CHANNELS) {
55013 |         ma_result result;
55014 |
55015 |         if (mode == ma_channel_mix_mode_simple) {
55016 |             ma_channel shuffleTable[MA_MAX_CHANNELS];
55017 |
55018 |             result = ma_channel_map_build_shuffle_table(pChannelMapIn, channelsIn, pChannelMapOut,
> channelsOut, shuffleTable);
55019 |             if (result != MA_SUCCESS) {
55020 |                 return;
55021 |             }
55022 |
55023 |             result = ma_channel_map_apply_shuffle_table(pFramesOut, channelsOut, pFramesIn,
> channelsIn, frameCount, shuffleTable, ma_format_f32);
55024 |             if (result != MA_SUCCESS) {
55025 |                 return;

```

```

55026 |     }
55027 |     } else {
55028 |         ma_uint32 iFrame;
55029 |         ma_uint32 iChannelOut;
55030 |         ma_uint32 iChannelIn;
55031 |         float weights[32][32]; /* Do not use MA_MAX_CHANNELS here! */
55032 |
55033 |         /*
55034 |         If we have a small enough number of channels, pre-compute the weights. Otherwise we'll
> just need to
55035 |         fall back to a slower path because otherwise we'll run out of stack space.
55036 |         */
55037 |         if (channelsIn <= ma_countof(weights) && channelsOut <= ma_countof(weights)) {
55038 |             /* Pre-compute weights. */
55039 |             for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
55040 |                 ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut, channelsOut,
> iChannelOut);
55041 |                 for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
55042 |                     ma_channel channelIn = ma_channel_map_get_channel(pChannelMapIn, channelsIn,
> iChannelIn);
55043 |                     weights[iChannelOut][iChannelIn] =
> ma_calculate_channel_position_rectangular_weight(channelOut, channelIn);
55044 |                 }
55045 |             }
55046 |
55047 |             iFrame = 0;
55048 |
55049 |             /* Experiment: Try an optimized unroll for some specific cases to see how it
> improves performance. RESULT: Good gains. */
55050 |             if (channelsOut == 8) {
55051 |                 /* Experiment 2: Expand the inner loop to see what kind of different it makes.
> RESULT: Small, but worthwhile gain. */
55052 |                 if (channelsIn == 2) {
55053 |                     for (; iFrame < frameCount; iFrame += 1) {
55054 |                         float accumulation[8] = { 0, 0, 0, 0, 0, 0, 0, 0 };
55055 |
55056 |                         accumulation[0] += pFramesIn[iFrame*2 + 0] * weights[0][0];
55057 |                         accumulation[1] += pFramesIn[iFrame*2 + 0] * weights[1][0];
55058 |                         accumulation[2] += pFramesIn[iFrame*2 + 0] * weights[2][0];
55059 |                         accumulation[3] += pFramesIn[iFrame*2 + 0] * weights[3][0];
55060 |                         accumulation[4] += pFramesIn[iFrame*2 + 0] * weights[4][0];
55061 |                         accumulation[5] += pFramesIn[iFrame*2 + 0] * weights[5][0];
55062 |                         accumulation[6] += pFramesIn[iFrame*2 + 0] * weights[6][0];
55063 |                         accumulation[7] += pFramesIn[iFrame*2 + 0] * weights[7][0];
55064 |
55065 |                         accumulation[0] += pFramesIn[iFrame*2 + 1] * weights[0][1];
55066 |                         accumulation[1] += pFramesIn[iFrame*2 + 1] * weights[1][1];
55067 |                         accumulation[2] += pFramesIn[iFrame*2 + 1] * weights[2][1];
55068 |                         accumulation[3] += pFramesIn[iFrame*2 + 1] * weights[3][1];
55069 |                         accumulation[4] += pFramesIn[iFrame*2 + 1] * weights[4][1];
55070 |                         accumulation[5] += pFramesIn[iFrame*2 + 1] * weights[5][1];
55071 |                         accumulation[6] += pFramesIn[iFrame*2 + 1] * weights[6][1];
55072 |                         accumulation[7] += pFramesIn[iFrame*2 + 1] * weights[7][1];
55073 |
55074 |                         pFramesOut[iFrame*8 + 0] = accumulation[0];
55075 |                         pFramesOut[iFrame*8 + 1] = accumulation[1];
55076 |                         pFramesOut[iFrame*8 + 2] = accumulation[2];
55077 |                         pFramesOut[iFrame*8 + 3] = accumulation[3];
55078 |                         pFramesOut[iFrame*8 + 4] = accumulation[4];
55079 |                         pFramesOut[iFrame*8 + 5] = accumulation[5];
55080 |                         pFramesOut[iFrame*8 + 6] = accumulation[6];
55081 |                         pFramesOut[iFrame*8 + 7] = accumulation[7];
55082 |                     }
55083 |                 } else {
55084 |                     /* When outputting to 8 channels, we can do everything in groups of two 4x
> SIMD operations. */
55085 |                     for (; iFrame < frameCount; iFrame += 1) {
55086 |                         float accumulation[8] = { 0, 0, 0, 0, 0, 0, 0, 0 };
55087 |
55088 |                         for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
55089 |                             accumulation[0] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[0][iChannelIn];
55090 |                             accumulation[1] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[1][iChannelIn];
55091 |                             accumulation[2] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[2][iChannelIn];
55092 |                             accumulation[3] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[3][iChannelIn];
55093 |                             accumulation[4] += pFramesIn[iFrame*channelsIn + iChannelIn] *

```

```

> weights[4][iChannelIn];
55094 |                                     accumulation[5] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[5][iChannelIn];
55095 |                                     accumulation[6] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[6][iChannelIn];
55096 |                                     accumulation[7] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[7][iChannelIn];
55097 |                                     }
55098 |
55099 |                                     pFramesOut[iFrame*8 + 0] = accumulation[0];
55100 |                                     pFramesOut[iFrame*8 + 1] = accumulation[1];
55101 |                                     pFramesOut[iFrame*8 + 2] = accumulation[2];
55102 |                                     pFramesOut[iFrame*8 + 3] = accumulation[3];
55103 |                                     pFramesOut[iFrame*8 + 4] = accumulation[4];
55104 |                                     pFramesOut[iFrame*8 + 5] = accumulation[5];
55105 |                                     pFramesOut[iFrame*8 + 6] = accumulation[6];
55106 |                                     pFramesOut[iFrame*8 + 7] = accumulation[7];
55107 |                                     }
55108 |                                     }
55109 |                             } else if (channelsOut == 6) {
55110 |                                 /*
55111 |                                 When outputting to 6 channels we unfortunately don't have a nice multiple of 4
> to do 4x SIMD operations. Instead we'll
55112 |                                 expand our weights and do two frames at a time.
55113 |                                 */
55114 |                                 for (; iFrame < frameCount; iFrame += 1) {
55115 |                                     float accumulation[12] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
55116 |
55117 |                                     for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
55118 |                                         accumulation[0] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[0][iChannelIn];
55119 |                                         accumulation[1] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[1][iChannelIn];
55120 |                                         accumulation[2] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[2][iChannelIn];
55121 |                                         accumulation[3] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[3][iChannelIn];
55122 |                                         accumulation[4] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[4][iChannelIn];
55123 |                                         accumulation[5] += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[5][iChannelIn];
55124 |                                     }
55125 |
55126 |                                     pFramesOut[iFrame*6 + 0] = accumulation[0];
55127 |                                     pFramesOut[iFrame*6 + 1] = accumulation[1];
55128 |                                     pFramesOut[iFrame*6 + 2] = accumulation[2];
55129 |                                     pFramesOut[iFrame*6 + 3] = accumulation[3];
55130 |                                     pFramesOut[iFrame*6 + 4] = accumulation[4];
55131 |                                     pFramesOut[iFrame*6 + 5] = accumulation[5];
55132 |                                     }
55133 |                                 }
55134 |
55135 |                                 /* Leftover frames. */
55136 |                                 for (; iFrame < frameCount; iFrame += 1) {
55137 |                                     for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
55138 |                                         float accumulation = 0;
55139 |
55140 |                                         for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
55141 |                                             accumulation += pFramesIn[iFrame*channelsIn + iChannelIn] *
> weights[iChannelOut][iChannelIn];
55142 |                                         }
55143 |
55144 |                                         pFramesOut[iFrame*channelsOut + iChannelOut] = accumulation;
55145 |                                     }
55146 |                                 }
55147 |                             } else {
55148 |                                 /* Cannot pre-compute weights because not enough room in stack-allocated buffer. */
55149 |                                 for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
55150 |                                     for (iChannelOut = 0; iChannelOut < channelsOut; iChannelOut += 1) {
55151 |                                         float accumulation = 0;
55152 |                                         ma_channel channelOut = ma_channel_map_get_channel(pChannelMapOut,
> channelsOut, iChannelOut);
55153 |
55154 |                                         for (iChannelIn = 0; iChannelIn < channelsIn; iChannelIn += 1) {
55155 |                                             ma_channel channelIn = ma_channel_map_get_channel(pChannelMapIn,
> channelsIn, iChannelIn);
55156 |                                             accumulation += pFramesIn[iFrame*channelsIn + iChannelIn] *
> ma_calculate_channel_position_rectangular_weight(channelOut, channelIn);
55157 |                                         }

```

```

55158 |
55159 |         pFramesOut[iFrame*channelsOut + iChannelOut] = accumulation;
55160 |     }
55161 | }
55162 |
55163 | }
55164 | } else {
55165 |     /* Fall back to silence. If you hit this, what are you doing with so many channels?! */
55166 |     ma_silence_pcm_frames(pFramesOut, frameCount, ma_format_f32, channelsOut);
55167 | }
55168 | }
55169 |
55170 |
55171 | typedef struct
55172 | {
55173 |     size_t sizeInBytes;
55174 |     size_t channelMapInOffset;
55175 |     size_t channelMapOutOffset;
55176 |     size_t shuffleTableOffset;
55177 |     size_t weightsOffset;
55178 | } ma_channel_converter_heap_layout;
55179 |
55180 | static ma_channel_conversion_path ma_channel_converter_config_get_conversion_path(const
> ma_channel_converter_config* pConfig)
55181 | {
55182 |     return ma_channel_map_get_conversion_path(pConfig->pChannelMapIn, pConfig->channelsIn,
> pConfig->pChannelMapOut, pConfig->channelsOut, pConfig->mixingMode);
55183 | }
55184 |
55185 | static ma_result ma_channel_converter_get_heap_layout(const ma_channel_converter_config* pConfig,
> ma_channel_converter_heap_layout* pHeapLayout)
55186 | {
55187 |     ma_channel_conversion_path conversionPath;
55188 |
55189 |     MA_ASSERT(pHeapLayout != NULL);
55190 |
55191 |     if (pConfig == NULL) {
55192 |         return MA_INVALID_ARGS;
55193 |     }
55194 |
55195 |     if (pConfig->channelsIn == 0 || pConfig->channelsOut == 0) {
55196 |         return MA_INVALID_ARGS;
55197 |     }
55198 |
55199 |     if (!ma_channel_map_is_valid(pConfig->pChannelMapIn, pConfig->channelsIn)) {
55200 |         return MA_INVALID_ARGS;
55201 |     }
55202 |
55203 |     if (!ma_channel_map_is_valid(pConfig->pChannelMapOut, pConfig->channelsOut)) {
55204 |         return MA_INVALID_ARGS;
55205 |     }
55206 |
55207 |     pHeapLayout->sizeInBytes = 0;
55208 |
55209 |     /* Input channel map. Only need to allocate this if we have an input channel map (otherwise
> default channel map is assumed). */
55210 |     pHeapLayout->channelMapInOffset = pHeapLayout->sizeInBytes;
55211 |     if (pConfig->pChannelMapIn != NULL) {
55212 |         pHeapLayout->sizeInBytes += sizeof(ma_channel) * pConfig->channelsIn;
55213 |     }
55214 |
55215 |     /* Output channel map. Only need to allocate this if we have an output channel map (otherwise
> default channel map is assumed). */
55216 |     pHeapLayout->channelMapOutOffset = pHeapLayout->sizeInBytes;
55217 |     if (pConfig->pChannelMapOut != NULL) {
55218 |         pHeapLayout->sizeInBytes += sizeof(ma_channel) * pConfig->channelsOut;
55219 |     }
55220 |
55221 |     /* Alignment for the next section. */
55222 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
55223 |
55224 |     /* Whether or not we use weights of a shuffle table depends on the channel map themselves and
> the algorithm we've chosen. */
55225 |     conversionPath = ma_channel_converter_config_get_conversion_path(pConfig);
55226 |
55227 |     /* Shuffle table */
55228 |     pHeapLayout->shuffleTableOffset = pHeapLayout->sizeInBytes;
55229 |     if (conversionPath == ma_channel_conversion_path_shuffle) {
55230 |         pHeapLayout->sizeInBytes += sizeof(ma_uint8) * pConfig->channelsOut;

```



```

55231 |     }
55232 |
55233 |     /* Weights */
55234 |     pHeapLayout->weightsOffset = pHeapLayout->sizeInBytes;
55235 |     if (conversionPath == ma_channel_conversion_path_weights) {
55236 |         pHeapLayout->sizeInBytes += sizeof(float*) * pConfig->channelsIn;
55237 |         pHeapLayout->sizeInBytes += sizeof(float ) * pConfig->channelsIn * pConfig->channelsOut;
55238 |     }
55239 |
55240 |     /* Make sure allocation size is aligned. */
55241 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
55242 |
55243 |     return MA_SUCCESS;
55244 | }
55245 |
55246 | MA_API ma_result ma_channel_converter_get_heap_size(const ma_channel_converter_config* pConfig,
> size_t* pHeapSizeInBytes)
55247 | {
55248 |     ma_result result;
55249 |     ma_channel_converter_heap_layout heapLayout;
55250 |
55251 |     if (pHeapSizeInBytes == NULL) {
55252 |         return MA_INVALID_ARGS;
55253 |     }
55254 |
55255 |     *pHeapSizeInBytes = 0;
55256 |
55257 |     result = ma_channel_converter_get_heap_layout(pConfig, &heapLayout);
55258 |     if (result != MA_SUCCESS) {
55259 |         return result;
55260 |     }
55261 |
55262 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
55263 |
55264 |     return MA_SUCCESS;
55265 | }
55266 |
55267 | MA_API ma_result ma_channel_converter_init_preallocated(const ma_channel_converter_config* pConfig,
> void* pHeap, ma_channel_converter* pConverter)
55268 | {
55269 |     ma_result result;
55270 |     ma_channel_converter_heap_layout heapLayout;
55271 |
55272 |     if (pConverter == NULL) {
55273 |         return MA_INVALID_ARGS;
55274 |     }
55275 |
55276 |     MA_ZERO_OBJECT(pConverter);
55277 |
55278 |     result = ma_channel_converter_get_heap_layout(pConfig, &heapLayout);
55279 |     if (result != MA_SUCCESS) {
55280 |         return result;
55281 |     }
55282 |
55283 |     pConverter->pHeap = pHeap;
55284 |     MA_ZERO_MEMORY(pConverter->pHeap, heapLayout.sizeInBytes);
55285 |
55286 |     pConverter->format      = pConfig->format;
55287 |     pConverter->channelsIn  = pConfig->channelsIn;
55288 |     pConverter->channelsOut = pConfig->channelsOut;
55289 |     pConverter->mixingMode  = pConfig->mixingMode;
55290 |
55291 |     if (pConfig->pChannelMapIn != NULL) {
55292 |         pConverter->pChannelMapIn = (ma_channel*)ma_offset_ptr(pHeap,
> heapLayout.channelMapInOffset);
55293 |         ma_channel_map_copy_or_default(pConverter->pChannelMapIn, pConfig->channelsIn,
> pConfig->pChannelMapIn, pConfig->channelsIn);
55294 |     } else {
55295 |         pConverter->pChannelMapIn = NULL; /* Use default channel map. */
55296 |     }
55297 |
55298 |     if (pConfig->pChannelMapOut != NULL) {
55299 |         pConverter->pChannelMapOut = (ma_channel*)ma_offset_ptr(pHeap,
> heapLayout.channelMapOutOffset);
55300 |         ma_channel_map_copy_or_default(pConverter->pChannelMapOut, pConfig->channelsOut,
> pConfig->pChannelMapOut, pConfig->channelsOut);
55301 |     } else {
55302 |         pConverter->pChannelMapOut = NULL; /* Use default channel map. */
55303 |     }

```

```

55304 |
55305 |     pConverter->conversionPath = ma_channel_converter_config_get_conversion_path(pConfig);
55306 |
55307 |     if (pConverter->conversionPath == ma_channel_conversion_path_shuffle) {
55308 |         pConverter->pShuffleTable = (ma_uint8*)ma_offset_ptr(pHeap, heapLayout.shuffleTableOffset);
55309 |         ma_channel_map_build_shuffle_table(pConverter->pChannelMapIn, pConverter->channelsIn,
> pConverter->pChannelMapOut, pConverter->channelsOut, pConverter->pShuffleTable);
55310 |     }
55311 |
55312 |     if (pConverter->conversionPath == ma_channel_conversion_path_weights) {
55313 |         ma_uint32 iChannelIn;
55314 |         ma_uint32 iChannelOut;
55315 |
55316 |         if (pConverter->format == ma_format_f32) {
55317 |             pConverter->weights.f32 = (float**)ma_offset_ptr(pHeap, heapLayout.weightsOffset);
55318 |             for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; iChannelIn += 1) {
55319 |                 pConverter->weights.f32[iChannelIn] = (float*)ma_offset_ptr(pHeap,
> heapLayout.weightsOffset + ((sizeof(float) * pConverter->channelsIn) + (sizeof(float) *
> pConverter->channelsOut * iChannelIn)));
55320 |             }
55321 |         } else {
55322 |             pConverter->weights.s16 = (ma_int32**)ma_offset_ptr(pHeap, heapLayout.weightsOffset);
55323 |             for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; iChannelIn += 1) {
55324 |                 pConverter->weights.s16[iChannelIn] = (ma_int32*)ma_offset_ptr(pHeap,
> heapLayout.weightsOffset + ((sizeof(ma_int32) * pConverter->channelsIn) + (sizeof(ma_int32) *
> pConverter->channelsOut * iChannelIn)));
55325 |             }
55326 |         }
55327 |
55328 |         /* Silence our weights by default. */
55329 |         for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; iChannelIn += 1) {
55330 |             for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; iChannelOut += 1) {
55331 |                 if (pConverter->format == ma_format_f32) {
55332 |                     pConverter->weights.f32[iChannelIn][iChannelOut] = 0.0f;
55333 |                 } else {
55334 |                     pConverter->weights.s16[iChannelIn][iChannelOut] = 0;
55335 |                 }
55336 |             }
55337 |         }
55338 |
55339 |         /*
55340 |         We now need to fill out our weights table. This is determined by the mixing mode.
55341 |         */
55342 |
55343 |         /* In all cases we need to make sure all channels that are present in both channel maps have
> a 1:1 mapping. */
55344 |         for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55345 |             ma_channel channelPosIn = ma_channel_map_get_channel(pConverter->pChannelMapIn,
> pConverter->channelsIn, iChannelIn);
55346 |
55347 |             for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55348 |                 ma_channel channelPosOut = ma_channel_map_get_channel(pConverter->pChannelMapOut,
> pConverter->channelsOut, iChannelOut);
55349 |
55350 |                 if (channelPosIn == channelPosOut) {
55351 |                     float weight = 1;
55352 |
55353 |                     if (pConverter->format == ma_format_f32) {
55354 |                         pConverter->weights.f32[iChannelIn][iChannelOut] = weight;
55355 |                     } else {
55356 |                         pConverter->weights.s16[iChannelIn][iChannelOut] =
> ma_channel_converter_float_to_fixed(weight);
55357 |                     }
55358 |                 }
55359 |             }
55360 |         }
55361 |
55362 |         switch (pConverter->mixingMode)
55363 |         {
55364 |             case ma_channel_mix_mode_custom_weights:
55365 |             {
55366 |                 if (pConfig->ppWeights == NULL) {
55367 |                     return MA_INVALID_ARGS; /* Config specified a custom weights mixing mode, but no
> custom weights have been specified. */
55368 |                 }
55369 |
55370 |                 for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; iChannelIn += 1) {
55371 |                     for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; iChannelOut += 1) {
55372 |                         float weight = pConfig->ppWeights[iChannelIn][iChannelOut];

```

```

55373 |
55374 |             if (pConverter->format == ma_format_f32) {
55375 |                 pConverter->weights.f32[iChannelIn][iChannelOut] = weight;
55376 |             } else {
55377 |                 pConverter->weights.s16[iChannelIn][iChannelOut] =
> ma_channel_converter_float_to_fixed(weight);
55378 |             }
55379 |         }
55380 |     }
55381 | } break;
55382 |
55383 |     case ma_channel_mix_mode_simple:
55384 |     {
55385 |         /*
55386 |         In simple mode, only set weights for channels that have exactly matching types,
> leave the rest at
55387 |         zero. The 1:1 mappings have already been covered before this switch statement.
55388 |         */
55389 |     } break;
55390 |
55391 |     case ma_channel_mix_mode_rectangular:
55392 |     default:
55393 |     {
55394 |         /* Unmapped input channels. */
55395 |         for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55396 |             ma_channel channelPosIn = ma_channel_map_get_channel(pConverter->pChannelMapIn,
> pConverter->channelsIn, iChannelIn);
55397 |
55398 |             if (ma_is_spatial_channel_position(channelPosIn)) {
55399 |                 if (!ma_channel_map_contains_channel_position(pConverter->channelsOut,
> pConverter->pChannelMapOut, channelPosIn)) {
55400 |                     for (iChannelOut = 0; iChannelOut < pConverter->channelsOut;
> ++iChannelOut) {
55401 |                         ma_channel channelPosOut =
> ma_channel_map_get_channel(pConverter->pChannelMapOut, pConverter->channelsOut, iChannelOut);
55402 |
55403 |                         if (ma_is_spatial_channel_position(channelPosOut)) {
55404 |                             float weight = 0;
55405 |                             if (pConverter->mixingMode == ma_channel_mix_mode_rectangular) {
55406 |                                 weight =
> ma_calculate_channel_position_rectangular_weight(channelPosIn, channelPosOut);
55407 |                             }
55408 |
55409 |                             /* Only apply the weight if we haven't already got some
> contribution from the respective channels. */
55410 |                             if (pConverter->format == ma_format_f32) {
55411 |                                 if (pConverter->weights.f32[iChannelIn][iChannelOut] == 0) {
55412 |                                     pConverter->weights.f32[iChannelIn][iChannelOut] =
> weight;
55413 |                                 }
55414 |                                 } else {
55415 |                                     if (pConverter->weights.s16[iChannelIn][iChannelOut] == 0) {
55416 |                                         pConverter->weights.s16[iChannelIn][iChannelOut] =
> ma_channel_converter_float_to_fixed(weight);
55417 |                                     }
55418 |                                 }
55419 |                             }
55420 |                         }
55421 |                     }
55422 |                 }
55423 |             }
55424 |
55425 |             /* Unmapped output channels. */
55426 |             for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55427 |                 ma_channel channelPosOut =
> ma_channel_map_get_channel(pConverter->pChannelMapOut, pConverter->channelsOut, iChannelOut);
55428 |
55429 |                 if (ma_is_spatial_channel_position(channelPosOut)) {
55430 |                     if (!ma_channel_map_contains_channel_position(pConverter->channelsIn,
> pConverter->pChannelMapIn, channelPosOut)) {
55431 |                         for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn)
> {
55432 |                             ma_channel channelPosIn =
> ma_channel_map_get_channel(pConverter->pChannelMapIn, pConverter->channelsIn, iChannelIn);
55433 |
55434 |                             if (ma_is_spatial_channel_position(channelPosIn)) {
55435 |                                 float weight = 0;
55436 |                                 if (pConverter->mixingMode == ma_channel_mix_mode_rectangular) {
55437 |                                     weight =

```

CSE 4102 | Application, tests, configuration, and library source

```

55501 |         if (pHeap == NULL) {
55502 |             return MA_OUT_OF_MEMORY;
55503 |         }
55504 |     } else {
55505 |         pHeap = NULL;
55506 |     }
55507 |
55508 |     result = ma_channel_converter_init_preallocated(pConfig, pHeap, pConverter);
55509 |     if (result != MA_SUCCESS) {
55510 |         ma_free(pHeap, pAllocationCallbacks);
55511 |         return result;
55512 |     }
55513 |
55514 |     pConverter->_ownsHeap = MA_TRUE;
55515 |     return MA_SUCCESS;
55516 | }
55517 |
55518 | MA_API void ma_channel_converter_uninit(ma_channel_converter* pConverter, const
> ma_allocation_callbacks* pAllocationCallbacks)
55519 | {
55520 |     if (pConverter == NULL) {
55521 |         return;
55522 |     }
55523 |
55524 |     if (pConverter->_ownsHeap) {
55525 |         ma_free(pConverter->_pHeap, pAllocationCallbacks);
55526 |     }
55527 | }
55528 |
55529 | static ma_result ma_channel_converter_process_pcm_frames_passthrough(ma_channel_converter*
> pConverter, void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount)
55530 | {
55531 |     MA_ASSERT(pConverter != NULL);
55532 |     MA_ASSERT(pFramesOut != NULL);
55533 |     MA_ASSERT(pFramesIn != NULL);
55534 |
55535 |     ma_copy_memory_64(pFramesOut, pFramesIn, frameCount * ma_get_bytes_per_frame(pConverter->format,
> pConverter->channelsOut));
55536 |     return MA_SUCCESS;
55537 | }
55538 |
55539 | static ma_result ma_channel_converter_process_pcm_frames_shuffle(ma_channel_converter* pConverter,
> void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount)
55540 | {
55541 |     MA_ASSERT(pConverter != NULL);
55542 |     MA_ASSERT(pFramesOut != NULL);
55543 |     MA_ASSERT(pFramesIn != NULL);
55544 |     MA_ASSERT(pConverter->channelsIn == pConverter->channelsOut);
55545 |
55546 |     return ma_channel_map_apply_shuffle_table(pFramesOut, pConverter->channelsOut, pFramesIn,
> pConverter->channelsIn, frameCount, pConverter->pShuffleTable, pConverter->format);
55547 | }
55548 |
55549 | static ma_result ma_channel_converter_process_pcm_frames_mono_in(ma_channel_converter* pConverter,
> void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount)
55550 | {
55551 |     ma_uint64 iFrame;
55552 |
55553 |     MA_ASSERT(pConverter != NULL);
55554 |     MA_ASSERT(pFramesOut != NULL);
55555 |     MA_ASSERT(pFramesIn != NULL);
55556 |     MA_ASSERT(pConverter->channelsIn == 1);
55557 |
55558 |     switch (pConverter->format)
55559 |     {
55560 |         case ma_format_u8:
55561 |         {
55562 |             /* */ ma_uint8* pFramesOutU8 = (ma_uint8*)pFramesOut;
55563 |             const ma_uint8* pFramesInU8 = (const ma_uint8*)pFramesIn;
55564 |
55565 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55566 |                 ma_uint32 iChannel;
55567 |                 for (iChannel = 0; iChannel < pConverter->channelsOut; iChannel += 1) {
55568 |                     pFramesOutU8[iFrame*pConverter->channelsOut + iChannel] = pFramesInU8[iFrame];
55569 |                 }
55570 |             }
55571 |             } break;
55572 |
55573 |         case ma_format_s16:

```

```

55574 |     {
55575 |         /* */ ma_int16* pFramesOutS16 = (    ma_int16*)pFramesOut;
55576 |         const ma_int16* pFramesInS16  = (const ma_int16*)pFramesIn;
55577 |
55578 |         if (pConverter->channelsOut == 2) {
55579 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55580 |                 pFramesOutS16[iFrame*2 + 0] = pFramesInS16[iFrame];
55581 |                 pFramesOutS16[iFrame*2 + 1] = pFramesInS16[iFrame];
55582 |             }
55583 |         } else {
55584 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55585 |                 ma_uint32 iChannel;
55586 |                 for (iChannel = 0; iChannel < pConverter->channelsOut; iChannel += 1) {
55587 |                     pFramesOutS16[iFrame*pConverter->channelsOut + iChannel] =
> pFramesInS16[iFrame];
55588 |                 }
55589 |             }
55590 |         }
55591 |     } break;
55592 |
55593 |     case ma_format_s24:
55594 |     {
55595 |         /* */ ma_uint8* pFramesOutS24 = (    ma_uint8*)pFramesOut;
55596 |         const ma_uint8* pFramesInS24  = (const ma_uint8*)pFramesIn;
55597 |
55598 |         for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55599 |             ma_uint32 iChannel;
55600 |             for (iChannel = 0; iChannel < pConverter->channelsOut; iChannel += 1) {
55601 |                 ma_uint64 iSampleOut = iFrame*pConverter->channelsOut + iChannel;
55602 |                 ma_uint64 iSampleIn  = iFrame;
55603 |                 pFramesOutS24[iSampleOut*3 + 0] = pFramesInS24[iSampleIn*3 + 0];
55604 |                 pFramesOutS24[iSampleOut*3 + 1] = pFramesInS24[iSampleIn*3 + 1];
55605 |                 pFramesOutS24[iSampleOut*3 + 2] = pFramesInS24[iSampleIn*3 + 2];
55606 |             }
55607 |         }
55608 |     } break;
55609 |
55610 |     case ma_format_s32:
55611 |     {
55612 |         /* */ ma_int32* pFramesOutS32 = (    ma_int32*)pFramesOut;
55613 |         const ma_int32* pFramesInS32  = (const ma_int32*)pFramesIn;
55614 |
55615 |         for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55616 |             ma_uint32 iChannel;
55617 |             for (iChannel = 0; iChannel < pConverter->channelsOut; iChannel += 1) {
55618 |                 pFramesOutS32[iFrame*pConverter->channelsOut + iChannel] = pFramesInS32[iFrame];
55619 |             }
55620 |         }
55621 |     } break;
55622 |
55623 |     case ma_format_f32:
55624 |     {
55625 |         /* */ float* pFramesOutF32 = (    float*)pFramesOut;
55626 |         const float* pFramesInF32  = (const float*)pFramesIn;
55627 |
55628 |         if (pConverter->channelsOut == 2) {
55629 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55630 |                 pFramesOutF32[iFrame*2 + 0] = pFramesInF32[iFrame];
55631 |                 pFramesOutF32[iFrame*2 + 1] = pFramesInF32[iFrame];
55632 |             }
55633 |         } else {
55634 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55635 |                 ma_uint32 iChannel;
55636 |                 for (iChannel = 0; iChannel < pConverter->channelsOut; iChannel += 1) {
55637 |                     pFramesOutF32[iFrame*pConverter->channelsOut + iChannel] =
> pFramesInF32[iFrame];
55638 |                 }
55639 |             }
55640 |         }
55641 |     } break;
55642 |
55643 |     default: return MA_INVALID_OPERATION;    /* Unknown format. */
55644 | }
55645 |
55646 |     return MA_SUCCESS;
55647 | }
55648 |
55649 | static ma_result ma_channel_converter_process_pcm_frames__mono_out(ma_channel_converter* pConverter,
> void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount)

```

```

55650 | {
55651 |     ma_uint64 iFrame;
55652 |     ma_uint32 iChannel;
55653 |
55654 |     MA_ASSERT(pConverter != NULL);
55655 |     MA_ASSERT(pFramesOut != NULL);
55656 |     MA_ASSERT(pFramesIn != NULL);
55657 |     MA_ASSERT(pConverter->channelsOut == 1);
55658 |
55659 |     switch (pConverter->format)
55660 |     {
55661 |         case ma_format_u8:
55662 |         {
55663 |             /* */ ma_uint8* pFramesOutU8 = (ma_uint8*)pFramesOut;
55664 |             const ma_uint8* pFramesInU8 = (const ma_uint8*)pFramesIn;
55665 |
55666 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55667 |                 ma_int32 t = 0;
55668 |                 for (iChannel = 0; iChannel < pConverter->channelsIn; iChannel += 1) {
55669 |                     t += ma_pcm_sample_u8_to_s16_no_scale(pFramesInU8[iFrame*pConverter->channelsIn
> + iChannel]);
55670 |                 }
55671 |
55672 |                 pFramesOutU8[iFrame] = ma_clip_u8(t / pConverter->channelsOut);
55673 |             }
55674 |             break;
55675 |
55676 |         case ma_format_s16:
55677 |         {
55678 |             /* */ ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
55679 |             const ma_int16* pFramesInS16 = (const ma_int16*)pFramesIn;
55680 |
55681 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55682 |                 ma_int32 t = 0;
55683 |                 for (iChannel = 0; iChannel < pConverter->channelsIn; iChannel += 1) {
55684 |                     t += pFramesInS16[iFrame*pConverter->channelsIn + iChannel];
55685 |                 }
55686 |
55687 |                 pFramesOutS16[iFrame] = (ma_int16)(t / pConverter->channelsIn);
55688 |             }
55689 |             break;
55690 |
55691 |         case ma_format_s24:
55692 |         {
55693 |             /* */ ma_uint8* pFramesOutS24 = (ma_uint8*)pFramesOut;
55694 |             const ma_uint8* pFramesInS24 = (const ma_uint8*)pFramesIn;
55695 |
55696 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55697 |                 ma_int64 t = 0;
55698 |                 for (iChannel = 0; iChannel < pConverter->channelsIn; iChannel += 1) {
55699 |                     t +=
> ma_pcm_sample_s24_to_s32_no_scale(&pFramesInS24[(iFrame*pConverter->channelsIn + iChannel)*3]);
55700 |                 }
55701 |
55702 |                 ma_pcm_sample_s32_to_s24_no_scale(t / pConverter->channelsIn,
> &pFramesOutS24[iFrame*3]);
55703 |             }
55704 |             break;
55705 |
55706 |         case ma_format_s32:
55707 |         {
55708 |             /* */ ma_int32* pFramesOutS32 = (ma_int32*)pFramesOut;
55709 |             const ma_int32* pFramesInS32 = (const ma_int32*)pFramesIn;
55710 |
55711 |             for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55712 |                 ma_int64 t = 0;
55713 |                 for (iChannel = 0; iChannel < pConverter->channelsIn; iChannel += 1) {
55714 |                     t += pFramesInS32[iFrame*pConverter->channelsIn + iChannel];
55715 |                 }
55716 |
55717 |                 pFramesOutS32[iFrame] = (ma_int32)(t / pConverter->channelsIn);
55718 |             }
55719 |             break;
55720 |
55721 |         case ma_format_f32:
55722 |         {
55723 |             /* */ float* pFramesOutF32 = (float*)pFramesOut;
55724 |             const float* pFramesInF32 = (const float*)pFramesIn;
55725 |

```



```

55726 |         for (iFrame = 0; iFrame < frameCount; ++iFrame) {
55727 |             float t = 0;
55728 |             for (iChannel = 0; iChannel < pConverter->channelsIn; iChannel += 1) {
55729 |                 t += pFramesInF32[iFrame*pConverter->channelsIn + iChannel];
55730 |             }
55731 |
55732 |             pFramesOutF32[iFrame] = t / pConverter->channelsIn;
55733 |         }
55734 |     } break;
55735 |
55736 |     default: return MA_INVALID_OPERATION;    /* Unknown format. */
55737 | }
55738 |
55739 | return MA_SUCCESS;
55740 | }
55741 |
55742 | static ma_result ma_channel_converter_process_pcm_frames__weights(ma_channel_converter* pConverter,
> void* pFramesOut, const void* pFramesIn, ma_uint64 frameCount)
55743 | {
55744 |     ma_uint32 iFrame;
55745 |     ma_uint32 iChannelIn;
55746 |     ma_uint32 iChannelOut;
55747 |
55748 |     MA_ASSERT(pConverter != NULL);
55749 |     MA_ASSERT(pFramesOut != NULL);
55750 |     MA_ASSERT(pFramesIn != NULL);
55751 |
55752 |     /* This is the more complicated case. Each of the output channels is accumulated with 0 or more
> input channels. */
55753 |
55754 |     /* Clear. */
55755 |     ma_zero_memory_64(pFramesOut, frameCount * ma_get_bytes_per_frame(pConverter->format,
> pConverter->channelsOut));
55756 |
55757 |     /* Accumulate. */
55758 |     switch (pConverter->format)
55759 |     {
55760 |     case ma_format_u8:
55761 |     {
55762 |         /* */ ma_uint8* pFramesOutU8 = (ma_uint8*)pFramesOut;
55763 |         const ma_uint8* pFramesInU8 = (const ma_uint8*)pFramesIn;
55764 |
55765 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
55766 |             for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55767 |                 for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55768 |                     ma_int16 u8_0 =
> ma_pcm_sample_u8_to_s16_no_scale(pFramesOutU8[iFrame*pConverter->channelsOut + iChannelOut]);
55769 |                     ma_int16 u8_I = ma_pcm_sample_u8_to_s16_no_scale(pFramesInU8
> [iFrame*pConverter->channelsIn + iChannelIn]);
55770 |                     ma_int32 s = (ma_int32)ma_clamp(u8_0 + ((u8_I *
> pConverter->weights.s16[iChannelIn][iChannelOut]) >> MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT), -128,
> 127);
55771 |                     pFramesOutU8[iFrame*pConverter->channelsOut + iChannelOut] =
> ma_clip_u8((ma_int16)s);
55772 |                 }
55773 |             }
55774 |         } break;
55775 |
55776 |     case ma_format_s16:
55777 |     {
55778 |         /* */ ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
55779 |         const ma_int16* pFramesInS16 = (const ma_int16*)pFramesIn;
55780 |
55781 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
55782 |             for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55783 |                 for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55784 |                     ma_int32 s = pFramesOutS16[iFrame*pConverter->channelsOut + iChannelOut];
55785 |                     s += (pFramesInS16[iFrame*pConverter->channelsIn + iChannelIn] *
> pConverter->weights.s16[iChannelIn][iChannelOut]) >> MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT;
55787 |                     pFramesOutS16[iFrame*pConverter->channelsOut + iChannelOut] =
> (ma_int16)ma_clamp(s, -32768, 32767);
55788 |                 }
55789 |             }
55790 |         } break;
55791 |
55792 |     } break;
55793 |
55794 |     case ma_format_s24:

```

```

55795 |         {
55796 |             /* */ ma_uint8* pFramesOutS24 = (    ma_uint8*)pFramesOut;
55797 |             const ma_uint8* pFramesInS24  = (const ma_uint8*)pFramesIn;
55798 |
55799 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
55800 |                 for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55801 |                     for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55802 |                         ma_int64 s24_0 =
> ma_pcm_sample_s24_to_s32_no_scale(&pFramesOutS24[(iFrame*pConverter->channelsOut + iChannelOut)*3]);
55803 |                         ma_int64 s24_I = ma_pcm_sample_s24_to_s32_no_scale(&pFramesInS24
> [(iFrame*pConverter->channelsIn + iChannelIn)*3]);
55804 |                         ma_int64 s24  = (ma_int32)ma_clamp(s24_0 + ((s24_I *
> pConverter->weights.s16[iChannelIn][iChannelOut]) >> MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT),
> -8388608, 8388607);
55805 |                         ma_pcm_sample_s32_to_s24_no_scale(s24,
> &pFramesOutS24[(iFrame*pConverter->channelsOut + iChannelOut)*3]);
55806 |                     }
55807 |                 }
55808 |             }
55809 |         } break;
55810 |
55811 |         case ma_format_s32:
55812 |         {
55813 |             /* */ ma_int32* pFramesOutS32 = (    ma_int32*)pFramesOut;
55814 |             const ma_int32* pFramesInS32  = (const ma_int32*)pFramesIn;
55815 |
55816 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
55817 |                 for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55818 |                     for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55819 |                         ma_int64 s = pFramesOutS32[iFrame*pConverter->channelsOut + iChannelOut];
55820 |                         s += ((ma_int64)pFramesInS32[iFrame*pConverter->channelsIn + iChannelIn] *
> pConverter->weights.s16[iChannelIn][iChannelOut]) >> MA_CHANNEL_CONVERTER_FIXED_POINT_SHIFT;
55821 |
55822 |                         pFramesOutS32[iFrame*pConverter->channelsOut + iChannelOut] =
> ma_clip_s32(s);
55823 |                     }
55824 |                 }
55825 |             }
55826 |         } break;
55827 |
55828 |         case ma_format_f32:
55829 |         {
55830 |             /* */ float* pFramesOutF32 = (    float*)pFramesOut;
55831 |             const float* pFramesInF32  = (const float*)pFramesIn;
55832 |
55833 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
55834 |                 for (iChannelIn = 0; iChannelIn < pConverter->channelsIn; ++iChannelIn) {
55835 |                     for (iChannelOut = 0; iChannelOut < pConverter->channelsOut; ++iChannelOut) {
55836 |                         pFramesOutF32[iFrame*pConverter->channelsOut + iChannelOut] +=
> pFramesInF32[iFrame*pConverter->channelsIn + iChannelIn] *
> pConverter->weights.f32[iChannelIn][iChannelOut];
55837 |                     }
55838 |                 }
55839 |             }
55840 |         } break;
55841 |
55842 |         default: return MA_INVALID_OPERATION;    /* Unknown format. */
55843 |     }
55844 |
55845 |     return MA_SUCCESS;
55846 | }
55847 |
55848 | MA_API ma_result ma_channel_converter_process_pcm_frames(ma_channel_converter* pConverter, void*
> pFramesOut, const void* pFramesIn, ma_uint64 frameCount)
55849 | {
55850 |     if (pConverter == NULL) {
55851 |         return MA_INVALID_ARGS;
55852 |     }
55853 |
55854 |     if (pFramesOut == NULL) {
55855 |         return MA_INVALID_ARGS;
55856 |     }
55857 |
55858 |     if (pFramesIn == NULL) {
55859 |         ma_zero_memory_64(pFramesOut, frameCount * ma_get_bytes_per_frame(pConverter->format,
> pConverter->channelsOut));
55860 |         return MA_SUCCESS;
55861 |     }
55862 | }

```

```

55863 |     switch (pConverter->conversionPath)
55864 |     {
55865 |         case ma_channel_conversion_path_passthrough: return
> ma_channel_converter_process_pcm_frames_passthrough(pConverter, pFramesOut, pFramesIn, frameCount);
55866 |         case ma_channel_conversion_path_mono_out: return
> ma_channel_converter_process_pcm_frames_mono_out(pConverter, pFramesOut, pFramesIn, frameCount);
55867 |         case ma_channel_conversion_path_mono_in: return
> ma_channel_converter_process_pcm_frames_mono_in(pConverter, pFramesOut, pFramesIn, frameCount);
55868 |         case ma_channel_conversion_path_shuffle: return
> ma_channel_converter_process_pcm_frames_shuffle(pConverter, pFramesOut, pFramesIn, frameCount);
55869 |         case ma_channel_conversion_path_weights:
55870 |             default:
55871 |             {
55872 |                 return ma_channel_converter_process_pcm_frames_weights(pConverter, pFramesOut,
> pFramesIn, frameCount);
55873 |             }
55874 |     }
55875 | }
55876 |
55877 | MA_API ma_result ma_channel_converter_get_input_channel_map(const ma_channel_converter* pConverter,
> ma_channel* pChannelMap, size_t channelMapCap)
55878 | {
55879 |     if (pConverter == NULL || pChannelMap == NULL) {
55880 |         return MA_INVALID_ARGS;
55881 |     }
55882 |
55883 |     ma_channel_map_copy_or_default(pChannelMap, channelMapCap, pConverter->pChannelMapIn,
> pConverter->channelsIn);
55884 |
55885 |     return MA_SUCCESS;
55886 | }
55887 |
55888 | MA_API ma_result ma_channel_converter_get_output_channel_map(const ma_channel_converter* pConverter,
> ma_channel* pChannelMap, size_t channelMapCap)
55889 | {
55890 |     if (pConverter == NULL || pChannelMap == NULL) {
55891 |         return MA_INVALID_ARGS;
55892 |     }
55893 |
55894 |     ma_channel_map_copy_or_default(pChannelMap, channelMapCap, pConverter->pChannelMapOut,
> pConverter->channelsOut);
55895 |
55896 |     return MA_SUCCESS;
55897 | }
55898 |
55899 |
55900 | /*****
> *****/
55901 |
55902 | Data Conversion
55903 |
55904 | *****/
> *****/
55905 | MA_API ma_data_converter_config ma_data_converter_config_init_default(void)
55906 | {
55907 |     ma_data_converter_config config;
55908 |     MA_ZERO_OBJECT(&config);
55909 |
55910 |     config.ditherMode = ma_dither_mode_none;
55911 |     config.resampling.algorithm = ma_resample_algorithm_linear;
55912 |     config.allowDynamicSampleRate = MA_FALSE; /* Disable dynamic sample rates by default because
> dynamic rate adjustments should be quite rare and it allows an optimization for cases when the in
> and out sample rates are the same. */
55913 |
55914 |     /* Linear resampling defaults. */
55915 |     config.resampling.linear.lpfOrder = 1;
55916 |
55917 |     return config;
55918 | }
55919 |
55920 | MA_API ma_data_converter_config ma_data_converter_config_init(ma_format formatIn, ma_format
> formatOut, ma_uint32 channelsIn, ma_uint32 channelsOut, ma_uint32 sampleRateIn, ma_uint32
> sampleRateOut)
55921 | {
55922 |     ma_data_converter_config config = ma_data_converter_config_init_default();
55923 |     config.formatIn = formatIn;
55924 |     config.formatOut = formatOut;
55925 |     config.channelsIn = channelsIn;
55926 |     config.channelsOut = channelsOut;

```

```

55927 |     config.sampleRateIn = sampleRateIn;
55928 |     config.sampleRateOut = sampleRateOut;
55929 |
55930 |     return config;
55931 | }
55932 |
55933 |
55934 | typedef struct
55935 | {
55936 |     size_t sizeInBytes;
55937 |     size_t channelConverterOffset;
55938 |     size_t resamplerOffset;
55939 | } ma_data_converter_heap_layout;
55940 |
55941 | static ma_bool32 ma_data_converter_config_is_resampler_required(const ma_data_converter_config*
> pConfig)
55942 | {
55943 |     MA_ASSERT(pConfig != NULL);
55944 |
55945 |     return pConfig->allowDynamicSampleRate || pConfig->sampleRateIn != pConfig->sampleRateOut;
55946 | }
55947 |
55948 | static ma_format ma_data_converter_config_get_mid_format(const ma_data_converter_config* pConfig)
55949 | {
55950 |     MA_ASSERT(pConfig != NULL);
55951 |
55952 |     /*
55953 |     We want to avoid as much data conversion as possible. The channel converter and linear
55954 |     resampler both support s16 and f32 natively. We need to decide on the format to use for this
55955 |     stage. We call this the mid format because it's used in the middle stage of the conversion
55956 |     pipeline. If the output format is either s16 or f32 we use that one. If that is not the case it
55957 |     will do the same thing for the input format. If it's neither we just use f32. If we are using a
55958 |     custom resampling backend, we can only guarantee that f32 will be supported so we'll be forced
55959 |     to use that if resampling is required.
55960 |     */
55961 |     if (ma_data_converter_config_is_resampler_required(pConfig) && pConfig->resampling.algorithm !=
> ma_resample_algorithm_linear) {
55962 |         return ma_format_f32; /* <-- Force f32 since that is the only one we can guarantee will be
> supported by the resampler. */
55963 |     } else {
55964 |         /* */ if (pConfig->formatOut == ma_format_s16 || pConfig->formatOut == ma_format_f32) {
55965 |             return pConfig->formatOut;
55966 |         } else if (pConfig->formatIn == ma_format_s16 || pConfig->formatIn == ma_format_f32) {
55967 |             return pConfig->formatIn;
55968 |         } else {
55969 |             return ma_format_f32;
55970 |         }
55971 |     }
55972 | }
55973 |
55974 | static ma_channel_converter_config ma_channel_converter_config_init_from_data_converter_config(const
> ma_data_converter_config* pConfig)
55975 | {
55976 |     ma_channel_converter_config channelConverterConfig;
55977 |
55978 |     MA_ASSERT(pConfig != NULL);
55979 |
55980 |     channelConverterConfig =
> ma_channel_converter_config_init(ma_data_converter_config_get_mid_format(pConfig),
> pConfig->channelsIn, pConfig->pChannelMapIn, pConfig->channelsOut, pConfig->pChannelMapOut,
> pConfig->channelMixMode);
55981 |     channelConverterConfig.ppWeights = pConfig->ppChannelWeights;
55982 |     channelConverterConfig.calculateLFEFromSpatialChannels =
> pConfig->calculateLFEFromSpatialChannels;
55983 |
55984 |     return channelConverterConfig;
55985 | }
55986 |
55987 | static ma_resampler_config ma_resampler_config_init_from_data_converter_config(const
> ma_data_converter_config* pConfig)
55988 | {
55989 |     ma_resampler_config resamplerConfig;
55990 |     ma_uint32 resamplerChannels;
55991 |
55992 |     MA_ASSERT(pConfig != NULL);
55993 |
55994 |     /* The resampler is the most expensive part of the conversion process, so we need to do it at
> the stage where the channel count is at it's lowest. */
55995 |     if (pConfig->channelsIn < pConfig->channelsOut) {

```

```

55996 |         resamplerChannels = pConfig->channelsIn;
55997 |     } else {
55998 |         resamplerChannels = pConfig->channelsOut;
55999 |     }
56000 |
56001 |     resamplerConfig = ma_resampler_config_init(ma_data_converter_config_get_mid_format(pConfig),
> resamplerChannels, pConfig->sampleRateIn, pConfig->sampleRateOut, pConfig->resampling.algorithm);
56002 |     resamplerConfig.linear = pConfig->resampling.linear;
56003 |     resamplerConfig.pBackendVTable = pConfig->resampling.pBackendVTable;
56004 |     resamplerConfig.pBackendUserData = pConfig->resampling.pBackendUserData;
56005 |
56006 |     return resamplerConfig;
56007 | }
56008 |
56009 | static ma_result ma_data_converter_get_heap_layout(const ma_data_converter_config* pConfig,
> ma_data_converter_heap_layout* pHeapLayout)
56010 | {
56011 |     ma_result result;
56012 |
56013 |     MA_ASSERT(pHeapLayout != NULL);
56014 |
56015 |     MA_ZERO_OBJECT(pHeapLayout);
56016 |
56017 |     if (pConfig == NULL) {
56018 |         return MA_INVALID_ARGS;
56019 |     }
56020 |
56021 |     if (pConfig->channelsIn == 0 || pConfig->channelsOut == 0) {
56022 |         return MA_INVALID_ARGS;
56023 |     }
56024 |
56025 |     pHeapLayout->sizeInBytes = 0;
56026 |
56027 |     /* Channel converter. */
56028 |     pHeapLayout->channelConverterOffset = pHeapLayout->sizeInBytes;
56029 |     {
56030 |         size_t heapSizeInBytes;
56031 |         ma_channel_converter_config channelConverterConfig =
> ma_channel_converter_config_init_from_data_converter_config(pConfig);
56032 |
56033 |         result = ma_channel_converter_get_heap_size(&channelConverterConfig, &heapSizeInBytes);
56034 |         if (result != MA_SUCCESS) {
56035 |             return result;
56036 |         }
56037 |
56038 |         pHeapLayout->sizeInBytes += heapSizeInBytes;
56039 |     }
56040 |
56041 |     /* Resampler. */
56042 |     pHeapLayout->resamplerOffset = pHeapLayout->sizeInBytes;
56043 |     if (ma_data_converter_config_is_resampler_required(pConfig)) {
56044 |         size_t heapSizeInBytes;
56045 |         ma_resampler_config resamplerConfig =
> ma_resampler_config_init_from_data_converter_config(pConfig);
56046 |
56047 |         result = ma_resampler_get_heap_size(&resamplerConfig, &heapSizeInBytes);
56048 |         if (result != MA_SUCCESS) {
56049 |             return result;
56050 |         }
56051 |
56052 |         pHeapLayout->sizeInBytes += heapSizeInBytes;
56053 |     }
56054 |
56055 |     /* Make sure allocation size is aligned. */
56056 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
56057 |
56058 |     return MA_SUCCESS;
56059 | }
56060 |
56061 | MA_API ma_result ma_data_converter_get_heap_size(const ma_data_converter_config* pConfig, size_t*
> pHeapSizeInBytes)
56062 | {
56063 |     ma_result result;
56064 |     ma_data_converter_heap_layout heapLayout;
56065 |
56066 |     if (pHeapSizeInBytes == NULL) {
56067 |         return MA_INVALID_ARGS;
56068 |     }
56069 |

```

```

56070 |     *pHeapSizeInBytes = 0;
56071 |
56072 |     result = ma_data_converter_get_heap_layout(pConfig, &heapLayout);
56073 |     if (result != MA_SUCCESS) {
56074 |         return result;
56075 |     }
56076 |
56077 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
56078 |
56079 |     return MA_SUCCESS;
56080 | }
56081 |
56082 | MA_API ma_result ma_data_converter_init_preallocated(const ma_data_converter_config* pConfig, void*
> pHeap, ma_data_converter* pConverter)
56083 | {
56084 |     ma_result result;
56085 |     ma_data_converter_heap_layout heapLayout;
56086 |     ma_format midFormat;
56087 |     ma_bool32 isResamplingRequired;
56088 |
56089 |     if (pConverter == NULL) {
56090 |         return MA_INVALID_ARGS;
56091 |     }
56092 |
56093 |     MA_ZERO_OBJECT(pConverter);
56094 |
56095 |     result = ma_data_converter_get_heap_layout(pConfig, &heapLayout);
56096 |     if (result != MA_SUCCESS) {
56097 |         return result;
56098 |     }
56099 |
56100 |     pConverter->pHeap = pHeap;
56101 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
56102 |
56103 |     pConverter->formatIn      = pConfig->formatIn;
56104 |     pConverter->formatOut     = pConfig->formatOut;
56105 |     pConverter->channelsIn    = pConfig->channelsIn;
56106 |     pConverter->channelsOut   = pConfig->channelsOut;
56107 |     pConverter->sampleRateIn  = pConfig->sampleRateIn;
56108 |     pConverter->sampleRateOut = pConfig->sampleRateOut;
56109 |     pConverter->ditherMode    = pConfig->ditherMode;
56110 |
56111 |     /*
56112 |     Determine if resampling is required. We need to do this so we can determine an appropriate
56113 |     mid format to use. If resampling is required, the mid format must be ma_format_f32 since
56114 |     that is the only one that is guaranteed to supported by custom resampling backends.
56115 |     */
56116 |     isResamplingRequired = ma_data_converter_config_is_resampler_required(pConfig);
56117 |     midFormat = ma_data_converter_config_get_mid_format(pConfig);
56118 |
56119 |
56120 |     /* Channel converter. We always initialize this, but we check if it configures itself as a
> passthrough to determine whether or not it's needed. */
56121 |     {
56122 |         ma_channel_converter_config channelConverterConfig =
> ma_channel_converter_config_init_from_data_converter_config(pConfig);
56123 |
56124 |         result = ma_channel_converter_init_preallocated(&channelConverterConfig,
> ma_offset_ptr(pHeap, heapLayout.channelConverterOffset), &pConverter->channelConverter);
56125 |         if (result != MA_SUCCESS) {
56126 |             return result;
56127 |         }
56128 |
56129 |         /* If the channel converter is not a passthrough we need to enable it. Otherwise we can skip
> it. */
56130 |         if (pConverter->channelConverter.conversionPath != ma_channel_conversion_path_passthrough) {
56131 |             pConverter->hasChannelConverter = MA_TRUE;
56132 |         }
56133 |     }
56134 |
56135 |
56136 |     /* Resampler. */
56137 |     if (isResamplingRequired) {
56138 |         ma_resampler_config resamplerConfig =
> ma_resampler_config_init_from_data_converter_config(pConfig);
56139 |
56140 |         result = ma_resampler_init_preallocated(&resamplerConfig, ma_offset_ptr(pHeap,
> heapLayout.resamplerOffset), &pConverter->resampler);
56141 |         if (result != MA_SUCCESS) {

```

```

56142 |         return result;
56143 |     }
56144 |
56145 |     pConverter->hasResampler = MA_TRUE;
56146 | }
56147 |
56148 |
56149 |     /* We can simplify pre- and post-format conversion if we have neither channel conversion nor
> resampling. */
56150 |     if (pConverter->hasChannelConverter == MA_FALSE && pConverter->hasResampler == MA_FALSE) {
56151 |         /* We have neither channel conversion nor resampling so we'll only need one of pre- or
> post-format conversion, or none if the input and output formats are the same. */
56152 |         if (pConverter->formatIn == pConverter->formatOut) {
56153 |             /* The formats are the same so we can just pass through. */
56154 |             pConverter->hasPreFormatConversion = MA_FALSE;
56155 |             pConverter->hasPostFormatConversion = MA_FALSE;
56156 |         } else {
56157 |             /* The formats are different so we need to do either pre- or post-format conversion. It
> doesn't matter which. */
56158 |             pConverter->hasPreFormatConversion = MA_FALSE;
56159 |             pConverter->hasPostFormatConversion = MA_TRUE;
56160 |         }
56161 |     } else {
56162 |         /* We have a channel converter and/or resampler so we'll need channel conversion based on
> the mid format. */
56163 |         if (pConverter->formatIn != midFormat) {
56164 |             pConverter->hasPreFormatConversion = MA_TRUE;
56165 |         }
56166 |         if (pConverter->formatOut != midFormat) {
56167 |             pConverter->hasPostFormatConversion = MA_TRUE;
56168 |         }
56169 |     }
56170 |
56171 |     /* We can enable passthrough optimizations if applicable. Note that we'll only be able to do
> this if the sample rate is static. */
56172 |     if (pConverter->hasPreFormatConversion == MA_FALSE &&
56173 |         pConverter->hasPostFormatConversion == MA_FALSE &&
56174 |         pConverter->hasChannelConverter == MA_FALSE &&
56175 |         pConverter->hasResampler == MA_FALSE) {
56176 |         pConverter->isPassthrough = MA_TRUE;
56177 |     }
56178 |
56179 |
56180 |     /* We now need to determine our execution path. */
56181 |     if (pConverter->isPassthrough) {
56182 |         pConverter->executionPath = ma_data_converter_execution_path_passthrough;
56183 |     } else {
56184 |         if (pConverter->channelsIn < pConverter->channelsOut) {
56185 |             /* Do resampling first, if necessary. */
56186 |             MA_ASSERT(pConverter->hasChannelConverter == MA_TRUE);
56187 |
56188 |             if (pConverter->hasResampler) {
56189 |                 pConverter->executionPath = ma_data_converter_execution_path_resample_first;
56190 |             } else {
56191 |                 pConverter->executionPath = ma_data_converter_execution_path_channels_only;
56192 |             }
56193 |         } else {
56194 |             /* Do channel conversion first, if necessary. */
56195 |             if (pConverter->hasChannelConverter) {
56196 |                 if (pConverter->hasResampler) {
56197 |                     pConverter->executionPath = ma_data_converter_execution_path_channels_first;
56198 |                 } else {
56199 |                     pConverter->executionPath = ma_data_converter_execution_path_channels_only;
56200 |                 }
56201 |             } else {
56202 |                 /* Channel routing not required. */
56203 |                 if (pConverter->hasResampler) {
56204 |                     pConverter->executionPath = ma_data_converter_execution_path_resample_only;
56205 |                 } else {
56206 |                     pConverter->executionPath = ma_data_converter_execution_path_format_only;
56207 |                 }
56208 |             }
56209 |         }
56210 |     }
56211 |
56212 |     return MA_SUCCESS;
56213 | }
56214 |
56215 | MA_API ma_result ma_data_converter_init(const ma_data_converter_config* pConfig, const

```



```

> ma_allocation_callbacks* pAllocationCallbacks, ma_data_converter* pConverter)
56216 | {
56217 |     ma_result result;
56218 |     size_t heapSizeInBytes;
56219 |     void* pHeap;
56220 |
56221 |     result = ma_data_converter_get_heap_size(pConfig, &heapSizeInBytes);
56222 |     if (result != MA_SUCCESS) {
56223 |         return result;
56224 |     }
56225 |
56226 |     if (heapSizeInBytes > 0) {
56227 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
56228 |         if (pHeap == NULL) {
56229 |             return MA_OUT_OF_MEMORY;
56230 |         }
56231 |     } else {
56232 |         pHeap = NULL;
56233 |     }
56234 |
56235 |     result = ma_data_converter_init_preallocated(pConfig, pHeap, pConverter);
56236 |     if (result != MA_SUCCESS) {
56237 |         ma_free(pHeap, pAllocationCallbacks);
56238 |         return result;
56239 |     }
56240 |
56241 |     pConverter->ownsHeap = MA_TRUE;
56242 |     return MA_SUCCESS;
56243 | }
56244 |
56245 | MA_API void ma_data_converter_uninit(ma_data_converter* pConverter, const ma_allocation_callbacks*
> pAllocationCallbacks)
56246 | {
56247 |     if (pConverter == NULL) {
56248 |         return;
56249 |     }
56250 |
56251 |     if (pConverter->hasResampler) {
56252 |         ma_resampler_uninit(&pConverter->resampler, pAllocationCallbacks);
56253 |     }
56254 |
56255 |     ma_channel_converter_uninit(&pConverter->channelConverter, pAllocationCallbacks);
56256 |
56257 |     if (pConverter->ownsHeap) {
56258 |         ma_free(pConverter->pHeap, pAllocationCallbacks);
56259 |     }
56260 | }
56261 |
56262 | static ma_result ma_data_converter_process_pcm_frames_passthrough(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56263 | {
56264 |     ma_uint64 frameCountIn;
56265 |     ma_uint64 frameCountOut;
56266 |     ma_uint64 frameCount;
56267 |
56268 |     MA_ASSERT(pConverter != NULL);
56269 |
56270 |     frameCountIn = 0;
56271 |     if (pFrameCountIn != NULL) {
56272 |         frameCountIn = *pFrameCountIn;
56273 |     }
56274 |
56275 |     frameCountOut = 0;
56276 |     if (pFrameCountOut != NULL) {
56277 |         frameCountOut = *pFrameCountOut;
56278 |     }
56279 |
56280 |     frameCount = ma_min(frameCountIn, frameCountOut);
56281 |
56282 |     if (pFramesOut != NULL) {
56283 |         if (pFramesIn != NULL) {
56284 |             ma_copy_memory_64(pFramesOut, pFramesIn, frameCount *
> ma_get_bytes_per_frame(pConverter->formatOut, pConverter->channelsOut));
56285 |         } else {
56286 |             ma_zero_memory_64(pFramesOut, frameCount *
> ma_get_bytes_per_frame(pConverter->formatOut, pConverter->channelsOut));
56287 |         }
56288 |     }
56289 | }

```

```
56290 |     if (pFrameCountIn != NULL) {
56291 |         *pFrameCountIn = frameCount;
56292 |     }
56293 |     if (pFrameCountOut != NULL) {
56294 |         *pFrameCountOut = frameCount;
56295 |     }
56296 |
56297 |     return MA_SUCCESS;
56298 | }
56299 |
56300 | static ma_result ma_data_converter_process_pcm_frames__format_only(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56301 | {
56302 |     ma_uint64 frameCountIn;
56303 |     ma_uint64 frameCountOut;
56304 |     ma_uint64 frameCount;
56305 |
56306 |     MA_ASSERT(pConverter != NULL);
56307 |
56308 |     frameCountIn = 0;
56309 |     if (pFrameCountIn != NULL) {
56310 |         frameCountIn = *pFrameCountIn;
56311 |     }
56312 |
56313 |     frameCountOut = 0;
56314 |     if (pFrameCountOut != NULL) {
56315 |         frameCountOut = *pFrameCountOut;
56316 |     }
56317 |
56318 |     frameCount = ma_min(frameCountIn, frameCountOut);
56319 |
56320 |     if (pFramesOut != NULL) {
56321 |         if (pFramesIn != NULL) {
56322 |             ma_convert_pcm_frames_format(pFramesOut, pConverter->formatOut, pFramesIn,
> pConverter->formatIn, frameCount, pConverter->channelsIn, pConverter->ditherMode);
56323 |         } else {
56324 |             ma_zero_memory_64(pFramesOut, frameCount * ma_get_bytes_per_frame(pConverter->formatOut,
> pConverter->channelsOut));
56325 |         }
56326 |     }
56327 |
56328 |     if (pFrameCountIn != NULL) {
56329 |         *pFrameCountIn = frameCount;
56330 |     }
56331 |     if (pFrameCountOut != NULL) {
56332 |         *pFrameCountOut = frameCount;
56333 |     }
56334 |
56335 |     return MA_SUCCESS;
56336 | }
56337 |
56338 |
56339 | static ma_result
> ma_data_converter_process_pcm_frames__resample_with_format_conversion(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56340 | {
56341 |     ma_result result = MA_SUCCESS;
56342 |     ma_uint64 frameCountIn;
56343 |     ma_uint64 frameCountOut;
56344 |     ma_uint64 framesProcessedIn;
56345 |     ma_uint64 framesProcessedOut;
56346 |
56347 |     MA_ASSERT(pConverter != NULL);
56348 |
56349 |     frameCountIn = 0;
56350 |     if (pFrameCountIn != NULL) {
56351 |         frameCountIn = *pFrameCountIn;
56352 |     }
56353 |
56354 |     frameCountOut = 0;
56355 |     if (pFrameCountOut != NULL) {
56356 |         frameCountOut = *pFrameCountOut;
56357 |     }
56358 |
56359 |     framesProcessedIn = 0;
56360 |     framesProcessedOut = 0;
56361 |
56362 |     while (framesProcessedOut < frameCountOut) {
56363 |         ma_uint8 pTempBufferOut[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
```

```

56364 |         const ma_uint32 tempBufferOutCap = sizeof(pTempBufferOut) /
> ma_get_bytes_per_frame(pConverter->resampler.format, pConverter->resampler.channels);
56365 |         const void* pFramesInThisIteration;
56366 |         /* */ void* pFramesOutThisIteration;
56367 |         ma_uint64 frameCountInThisIteration;
56368 |         ma_uint64 frameCountOutThisIteration;
56369 |
56370 |         if (pFramesIn != NULL) {
56371 |             pFramesInThisIteration = ma_offset_ptr(pFramesIn, framesProcessedIn *
> ma_get_bytes_per_frame(pConverter->formatIn, pConverter->channelsIn));
56372 |         } else {
56373 |             pFramesInThisIteration = NULL;
56374 |         }
56375 |
56376 |         if (pFramesOut != NULL) {
56377 |             pFramesOutThisIteration = ma_offset_ptr(pFramesOut, framesProcessedOut *
> ma_get_bytes_per_frame(pConverter->formatOut, pConverter->channelsOut));
56378 |         } else {
56379 |             pFramesOutThisIteration = NULL;
56380 |         }
56381 |
56382 |         /* Do a pre format conversion if necessary. */
56383 |         if (pConverter->hasPreFormatConversion) {
56384 |             ma_uint8 pTempBufferIn[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
56385 |             const ma_uint32 tempBufferInCap = sizeof(pTempBufferIn) /
> ma_get_bytes_per_frame(pConverter->resampler.format, pConverter->resampler.channels);
56386 |
56387 |             frameCountInThisIteration = (frameCountIn - framesProcessedIn);
56388 |             if (frameCountInThisIteration > tempBufferInCap) {
56389 |                 frameCountInThisIteration = tempBufferInCap;
56390 |             }
56391 |
56392 |             if (pConverter->hasPostFormatConversion) {
56393 |                 if (frameCountInThisIteration > tempBufferOutCap) {
56394 |                     frameCountInThisIteration = tempBufferOutCap;
56395 |                 }
56396 |             }
56397 |
56398 |             if (pFramesInThisIteration != NULL) {
56399 |                 ma_convert_pcm_frames_format(pTempBufferIn, pConverter->resampler.format,
> pFramesInThisIteration, pConverter->formatIn, frameCountInThisIteration, pConverter->channelsIn,
> pConverter->ditherMode);
56400 |             } else {
56401 |                 MA_ZERO_MEMORY(pTempBufferIn, sizeof(pTempBufferIn));
56402 |             }
56403 |
56404 |             frameCountOutThisIteration = (frameCountOut - framesProcessedOut);
56405 |
56406 |             if (pConverter->hasPostFormatConversion) {
56407 |                 /* Both input and output conversion required. Output to the temp buffer. */
56408 |                 if (frameCountOutThisIteration > tempBufferOutCap) {
56409 |                     frameCountOutThisIteration = tempBufferOutCap;
56410 |                 }
56411 |
56412 |                 result = ma_resampler_process_pcm_frames(&pConverter->resampler, pTempBufferIn,
> &frameCountInThisIteration, pTempBufferOut, &frameCountOutThisIteration);
56413 |             } else {
56414 |                 /* Only pre-format required. Output straight to the output buffer. */
56415 |                 result = ma_resampler_process_pcm_frames(&pConverter->resampler, pTempBufferIn,
> &frameCountInThisIteration, pFramesOutThisIteration, &frameCountOutThisIteration);
56416 |             }
56417 |
56418 |             if (result != MA_SUCCESS) {
56419 |                 break;
56420 |             }
56421 |         } else {
56422 |             /* No pre-format required. Just read straight from the input buffer. */
56423 |             MA_ASSERT(pConverter->hasPostFormatConversion == MA_TRUE);
56424 |
56425 |             frameCountInThisIteration = (frameCountIn - framesProcessedIn);
56426 |             frameCountOutThisIteration = (frameCountOut - framesProcessedOut);
56427 |             if (frameCountOutThisIteration > tempBufferOutCap) {
56428 |                 frameCountOutThisIteration = tempBufferOutCap;
56429 |             }
56430 |
56431 |             result = ma_resampler_process_pcm_frames(&pConverter->resampler, pFramesInThisIteration,
> &frameCountInThisIteration, pTempBufferOut, &frameCountOutThisIteration);
56432 |             if (result != MA_SUCCESS) {
56433 |                 break;

```

```

56434 |     }
56435 | }
56436 |
56437 | /* If we are doing a post format conversion we need to do that now. */
56438 | if (pConverter->hasPostFormatConversion) {
56439 |     if (pFramesOutThisIteration != NULL) {
56440 |         ma_convert_pcm_frames_format(pFramesOutThisIteration, pConverter->formatOut,
> pTempBufferOut, pConverter->resampler.format, frameCountOutThisIteration,
> pConverter->resampler.channels, pConverter->ditherMode);
56441 |     }
56442 | }
56443 |
56444 | framesProcessedIn += frameCountInThisIteration;
56445 | framesProcessedOut += frameCountOutThisIteration;
56446 |
56447 | MA_ASSERT(framesProcessedIn <= frameCountIn);
56448 | MA_ASSERT(framesProcessedOut <= frameCountOut);
56449 |
56450 | if (frameCountOutThisIteration == 0) {
56451 |     break; /* Consumed all of our input data. */
56452 | }
56453 | }
56454 |
56455 | if (pFrameCountIn != NULL) {
56456 |     *pFrameCountIn = framesProcessedIn;
56457 | }
56458 | if (pFrameCountOut != NULL) {
56459 |     *pFrameCountOut = framesProcessedOut;
56460 | }
56461 |
56462 | return result;
56463 | }
56464 |
56465 | static ma_result ma_data_converter_process_pcm_frames__resample_only(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56466 | {
56467 |     MA_ASSERT(pConverter != NULL);
56468 |
56469 |     if (pConverter->hasPreFormatConversion == MA_FALSE && pConverter->hasPostFormatConversion ==
> MA_FALSE) {
56470 |         /* Neither pre- nor post-format required. This is simple case where only resampling is
> required. */
56471 |         return ma_resampler_process_pcm_frames(&pConverter->resampler, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
56472 |     } else {
56473 |         /* Format conversion required. */
56474 |         return ma_data_converter_process_pcm_frames__resample_with_format_conversion(pConverter,
> pFramesIn, pFrameCountIn, pFramesOut, pFrameCountOut);
56475 |     }
56476 | }
56477 |
56478 | static ma_result ma_data_converter_process_pcm_frames__channels_only(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56479 | {
56480 |     ma_result result;
56481 |     ma_uint64 frameCountIn;
56482 |     ma_uint64 frameCountOut;
56483 |     ma_uint64 frameCount;
56484 |
56485 |     MA_ASSERT(pConverter != NULL);
56486 |
56487 |     frameCountIn = 0;
56488 |     if (pFrameCountIn != NULL) {
56489 |         frameCountIn = *pFrameCountIn;
56490 |     }
56491 |
56492 |     frameCountOut = 0;
56493 |     if (pFrameCountOut != NULL) {
56494 |         frameCountOut = *pFrameCountOut;
56495 |     }
56496 |
56497 |     frameCount = ma_min(frameCountIn, frameCountOut);
56498 |
56499 |     if (pConverter->hasPreFormatConversion == MA_FALSE && pConverter->hasPostFormatConversion ==
> MA_FALSE) {
56500 |         /* No format conversion required. */
56501 |         result = ma_channel_converter_process_pcm_frames(&pConverter->channelConverter, pFramesOut,
> pFramesIn, frameCount);
56502 |         if (result != MA_SUCCESS) {

```

```

56503 |         return result;
56504 |     }
56505 | } else {
56506 |     /* Format conversion required. */
56507 |     ma_uint64 framesProcessed = 0;
56508 |
56509 |     while (framesProcessed < frameCount) {
56510 |         ma_uint8 pTempBufferOut[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
56511 |         const ma_uint32 tempBufferOutCap = sizeof(pTempBufferOut) /
> ma_get_bytes_per_frame(pConverter->channelConverter.format,
> pConverter->channelConverter.channelsOut);
56512 |         const void* pFramesInThisIteration;
56513 |         /* */ void* pFramesOutThisIteration;
56514 |         ma_uint64 frameCountThisIteration;
56515 |
56516 |         if (pFramesIn != NULL) {
56517 |             pFramesInThisIteration = ma_offset_ptr(pFramesIn, framesProcessed *
> ma_get_bytes_per_frame(pConverter->formatIn, pConverter->channelsIn));
56518 |         } else {
56519 |             pFramesInThisIteration = NULL;
56520 |         }
56521 |
56522 |         if (pFramesOut != NULL) {
56523 |             pFramesOutThisIteration = ma_offset_ptr(pFramesOut, framesProcessed *
> ma_get_bytes_per_frame(pConverter->formatOut, pConverter->channelsOut));
56524 |         } else {
56525 |             pFramesOutThisIteration = NULL;
56526 |         }
56527 |
56528 |         /* Do a pre format conversion if necessary. */
56529 |         if (pConverter->hasPreFormatConversion) {
56530 |             ma_uint8 pTempBufferIn[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
56531 |             const ma_uint32 tempBufferInCap = sizeof(pTempBufferIn) /
> ma_get_bytes_per_frame(pConverter->channelConverter.format,
> pConverter->channelConverter.channelsIn);
56532 |
56533 |             frameCountThisIteration = (frameCount - framesProcessed);
56534 |             if (frameCountThisIteration > tempBufferInCap) {
56535 |                 frameCountThisIteration = tempBufferInCap;
56536 |             }
56537 |
56538 |             if (pConverter->hasPostFormatConversion) {
56539 |                 if (frameCountThisIteration > tempBufferOutCap) {
56540 |                     frameCountThisIteration = tempBufferOutCap;
56541 |                 }
56542 |             }
56543 |
56544 |             if (pFramesInThisIteration != NULL) {
56545 |                 ma_convert_pcm_frames_format(pTempBufferIn, pConverter->channelConverter.format,
> pFramesInThisIteration, pConverter->formatIn, frameCountThisIteration, pConverter->channelsIn,
> pConverter->ditherMode);
56546 |             } else {
56547 |                 MA_ZERO_MEMORY(pTempBufferIn, sizeof(pTempBufferIn));
56548 |             }
56549 |
56550 |             if (pConverter->hasPostFormatConversion) {
56551 |                 /* Both input and output conversion required. Output to the temp buffer. */
56552 |                 result = ma_channel_converter_process_pcm_frames(&pConverter->channelConverter,
> pTempBufferOut, pTempBufferIn, frameCountThisIteration);
56553 |             } else {
56554 |                 /* Only pre-format required. Output straight to the output buffer. */
56555 |                 result = ma_channel_converter_process_pcm_frames(&pConverter->channelConverter,
> pFramesOutThisIteration, pTempBufferIn, frameCountThisIteration);
56556 |             }
56557 |
56558 |             if (result != MA_SUCCESS) {
56559 |                 break;
56560 |             }
56561 |         } else {
56562 |             /* No pre-format required. Just read straight from the input buffer. */
56563 |             MA_ASSERT(pConverter->hasPostFormatConversion == MA_TRUE);
56564 |
56565 |             frameCountThisIteration = (frameCount - framesProcessed);
56566 |             if (frameCountThisIteration > tempBufferOutCap) {
56567 |                 frameCountThisIteration = tempBufferOutCap;
56568 |             }
56569 |
56570 |             result = ma_channel_converter_process_pcm_frames(&pConverter->channelConverter,
> pTempBufferOut, pFramesInThisIteration, frameCountThisIteration);

```

```

56571 |             if (result != MA_SUCCESS) {
56572 |                 break;
56573 |             }
56574 |         }
56575 |
56576 |         /* If we are doing a post format conversion we need to do that now. */
56577 |         if (pConverter->hasPostFormatConversion) {
56578 |             if (pFramesOutThisIteration != NULL) {
56579 |                 ma_convert_pcm_frames_format(pFramesOutThisIteration, pConverter->formatOut,
> pTempBufferOut, pConverter->channelConverter.format, frameCountThisIteration,
> pConverter->channelConverter.channelsOut, pConverter->ditherMode);
56580 |             }
56581 |         }
56582 |
56583 |         framesProcessed += frameCountThisIteration;
56584 |     }
56585 | }
56586 |
56587 | if (pFrameCountIn != NULL) {
56588 |     *pFrameCountIn = frameCount;
56589 | }
56590 | if (pFrameCountOut != NULL) {
56591 |     *pFrameCountOut = frameCount;
56592 | }
56593 |
56594 | return MA_SUCCESS;
56595 | }
56596 |
56597 | static ma_result ma_data_converter_process_pcm_frames__resample_first(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56598 | {
56599 |     ma_result result;
56600 |     ma_uint64 frameCountIn;
56601 |     ma_uint64 frameCountOut;
56602 |     ma_uint64 framesProcessedIn;
56603 |     ma_uint64 framesProcessedOut;
56604 |     ma_uint8 pTempBufferIn[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In resampler format. */
56605 |     ma_uint64 tempBufferInCap;
56606 |     ma_uint8 pTempBufferMid[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In resampler format, channel
> converter input format. */
56607 |     ma_uint64 tempBufferMidCap;
56608 |     ma_uint8 pTempBufferOut[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In channel converter output
> format. */
56609 |     ma_uint64 tempBufferOutCap;
56610 |
56611 |     MA_ASSERT(pConverter != NULL);
56612 |     MA_ASSERT(pConverter->resampler.format == pConverter->channelConverter.format);
56613 |     MA_ASSERT(pConverter->resampler.channels == pConverter->channelConverter.channelsIn);
56614 |     MA_ASSERT(pConverter->resampler.channels < pConverter->channelConverter.channelsOut);
56615 |
56616 |     frameCountIn = 0;
56617 |     if (pFrameCountIn != NULL) {
56618 |         frameCountIn = *pFrameCountIn;
56619 |     }
56620 |
56621 |     frameCountOut = 0;
56622 |     if (pFrameCountOut != NULL) {
56623 |         frameCountOut = *pFrameCountOut;
56624 |     }
56625 |
56626 |     framesProcessedIn = 0;
56627 |     framesProcessedOut = 0;
56628 |
56629 |     tempBufferInCap = sizeof(pTempBufferIn) / ma_get_bytes_per_frame(pConverter->resampler.format,
> pConverter->resampler.channels);
56630 |     tempBufferMidCap = sizeof(pTempBufferIn) / ma_get_bytes_per_frame(pConverter->resampler.format,
> pConverter->resampler.channels);
56631 |     tempBufferOutCap = sizeof(pTempBufferOut) /
> ma_get_bytes_per_frame(pConverter->channelConverter.format,
> pConverter->channelConverter.channelsOut);
56632 |
56633 |     while (framesProcessedOut < frameCountOut) {
56634 |         ma_uint64 frameCountInThisIteration;
56635 |         ma_uint64 frameCountOutThisIteration;
56636 |         const void* pRunningFramesIn = NULL;
56637 |         void* pRunningFramesOut = NULL;
56638 |         const void* pResampleBufferIn;
56639 |         void* pChannelsBufferOut;
56640 |

```

```

56641 |         if (pFramesIn != NULL) {
56642 |             pRunningFramesIn = ma_offset_ptr(pFramesIn, framesProcessedIn *
> ma_get_bytes_per_frame(pConverter->formatIn, pConverter->channelsIn));
56643 |         }
56644 |         if (pFramesOut != NULL) {
56645 |             pRunningFramesOut = ma_offset_ptr(pFramesOut, framesProcessedOut *
> ma_get_bytes_per_frame(pConverter->formatOut, pConverter->channelsOut));
56646 |         }
56647 |
56648 |         /* Run input data through the resampler and output it to the temporary buffer. */
56649 |         frameCountInThisIteration = (frameCountIn - framesProcessedIn);
56650 |
56651 |         if (pConverter->hasPreFormatConversion) {
56652 |             if (frameCountInThisIteration > tempBufferInCap) {
56653 |                 frameCountInThisIteration = tempBufferInCap;
56654 |             }
56655 |         }
56656 |
56657 |         frameCountOutThisIteration = (frameCountOut - framesProcessedOut);
56658 |         if (frameCountOutThisIteration > tempBufferMidCap) {
56659 |             frameCountOutThisIteration = tempBufferMidCap;
56660 |         }
56661 |
56662 |         /* We can't read more frames than can fit in the output buffer. */
56663 |         if (pConverter->hasPostFormatConversion) {
56664 |             if (frameCountOutThisIteration > tempBufferOutCap) {
56665 |                 frameCountOutThisIteration = tempBufferOutCap;
56666 |             }
56667 |         }
56668 |
56669 |         /* We need to ensure we don't try to process too many input frames that we run out of room
> in the output buffer. If this happens we'll end up glitching. */
56670 |
56671 |         /*
56672 |         We need to try to predict how many input frames will be required for the resampler. If the
56673 |         resampler can tell us, we'll use that. Otherwise we'll need to make a best guess. The
> further
56674 |         off we are from this, the more wasted format conversions we'll end up doing.
56675 |         */
56676 |         #if 1
56677 |         {
56678 |             ma_uint64 requiredInputFrameCount;
56679 |
56680 |             result = ma_resampler_get_required_input_frame_count(&pConverter->resampler,
> frameCountOutThisIteration, &requiredInputFrameCount);
56681 |             if (result != MA_SUCCESS) {
56682 |                 /* Fall back to a best guess. */
56683 |                 requiredInputFrameCount = (frameCountOutThisIteration *
> pConverter->resampler.sampleRateIn) / pConverter->resampler.sampleRateOut;
56684 |             }
56685 |
56686 |             if (frameCountInThisIteration > requiredInputFrameCount) {
56687 |                 frameCountInThisIteration = requiredInputFrameCount;
56688 |             }
56689 |         }
56690 |         #endif
56691 |
56692 |         if (pConverter->hasPreFormatConversion) {
56693 |             if (pFramesIn != NULL) {
56694 |                 ma_convert_pcm_frames_format(pTempBufferIn, pConverter->resampler.format,
> pRunningFramesIn, pConverter->formatIn, frameCountInThisIteration, pConverter->channelsIn,
> pConverter->ditherMode);
56695 |                 pResampleBufferIn = pTempBufferIn;
56696 |             } else {
56697 |                 pResampleBufferIn = NULL;
56698 |             }
56699 |         } else {
56700 |             pResampleBufferIn = pRunningFramesIn;
56701 |         }
56702 |
56703 |         result = ma_resampler_process_pcm_frames(&pConverter->resampler, pResampleBufferIn,
> &frameCountInThisIteration, pTempBufferMid, &frameCountOutThisIteration);
56704 |         if (result != MA_SUCCESS) {
56705 |             return result;
56706 |         }
56707 |
56708 |
56709 |         /*
56710 |         The input data has been resampled so now we need to run it through the channel converter.

```



```

> The input data is always contained in pTempBufferMid. We only need to do
56711 |     this part if we have an output buffer.
56712 |     */
56713 |     if (pFramesOut != NULL) {
56714 |         if (pConverter->hasPostFormatConversion) {
56715 |             pChannelsBufferOut = pTempBufferOut;
56716 |         } else {
56717 |             pChannelsBufferOut = pRunningFramesOut;
56718 |         }
56719 |
56720 |         result = ma_channel_converter_process_pcm_frames(&pConverter->channelConverter,
> pChannelsBufferOut, pTempBufferMid, frameCountOutThisIteration);
56721 |         if (result != MA_SUCCESS) {
56722 |             return result;
56723 |         }
56724 |
56725 |         /* Finally we do post format conversion. */
56726 |         if (pConverter->hasPostFormatConversion) {
56727 |             ma_convert_pcm_frames_format(pRunningFramesOut, pConverter->formatOut,
> pChannelsBufferOut, pConverter->channelConverter.format, frameCountOutThisIteration,
> pConverter->channelConverter.channelsOut, pConverter->ditherMode);
56728 |         }
56729 |     }
56730 |
56731 |
56732 |     framesProcessedIn += frameCountInThisIteration;
56733 |     framesProcessedOut += frameCountOutThisIteration;
56734 |
56735 |     MA_ASSERT(framesProcessedIn <= frameCountIn);
56736 |     MA_ASSERT(framesProcessedOut <= frameCountOut);
56737 |
56738 |     if (frameCountOutThisIteration == 0) {
56739 |         break; /* Consumed all of our input data. */
56740 |     }
56741 | }
56742 |
56743 | if (pFrameCountIn != NULL) {
56744 |     *pFrameCountIn = framesProcessedIn;
56745 | }
56746 | if (pFrameCountOut != NULL) {
56747 |     *pFrameCountOut = framesProcessedOut;
56748 | }
56749 |
56750 | return MA_SUCCESS;
56751 | }
56752 |
56753 | static ma_result ma_data_converter_process_pcm_frames_channels_first(ma_data_converter* pConverter,
> const void* pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56754 | {
56755 |     ma_result result;
56756 |     ma_uint64 frameCountIn;
56757 |     ma_uint64 frameCountOut;
56758 |     ma_uint64 framesProcessedIn;
56759 |     ma_uint64 framesProcessedOut;
56760 |     ma_uint8 pTempBufferIn[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In resampler format. */
56761 |     ma_uint64 tempBufferInCap;
56762 |     ma_uint8 pTempBufferMid[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In resampler format, channel
> converter input format. */
56763 |     ma_uint64 tempBufferMidCap;
56764 |     ma_uint8 pTempBufferOut[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In channel converter output
> format. */
56765 |     ma_uint64 tempBufferOutCap;
56766 |
56767 |     MA_ASSERT(pConverter != NULL);
56768 |     MA_ASSERT(pConverter->resampler.format == pConverter->channelConverter.format);
56769 |     MA_ASSERT(pConverter->resampler.channels == pConverter->channelConverter.channelsOut);
56770 |     MA_ASSERT(pConverter->resampler.channels <= pConverter->channelConverter.channelsIn);
56771 |
56772 |     frameCountIn = 0;
56773 |     if (pFrameCountIn != NULL) {
56774 |         frameCountIn = *pFrameCountIn;
56775 |     }
56776 |
56777 |     frameCountOut = 0;
56778 |     if (pFrameCountOut != NULL) {
56779 |         frameCountOut = *pFrameCountOut;
56780 |     }
56781 |
56782 |     framesProcessedIn = 0;

```

```

56783 |     framesProcessedOut = 0;
56784 |
56785 |     tempBufferInCap = sizeof(pTempBufferIn) /
> ma_get_bytes_per_frame(pConverter->channelConverter.format,
> pConverter->channelConverter.channelsIn);
56786 |     tempBufferMidCap = sizeof(pTempBufferIn) /
> ma_get_bytes_per_frame(pConverter->channelConverter.format,
> pConverter->channelConverter.channelsOut);
56787 |     tempBufferOutCap = sizeof(pTempBufferOut) / ma_get_bytes_per_frame(pConverter->resampler.format,
> pConverter->resampler.channels);
56788 |
56789 |     while (framesProcessedOut < frameCountOut) {
56790 |         ma_uint64 frameCountInThisIteration;
56791 |         ma_uint64 frameCountOutThisIteration;
56792 |         const void* pRunningFramesIn = NULL;
56793 |         void* pRunningFramesOut = NULL;
56794 |         const void* pChannelsBufferIn;
56795 |         void* pResampleBufferOut;
56796 |
56797 |         if (pFramesIn != NULL) {
56798 |             pRunningFramesIn = ma_offset_ptr(pFramesIn, framesProcessedIn *
> ma_get_bytes_per_frame(pConverter->formatIn, pConverter->channelsIn));
56799 |         }
56800 |         if (pFramesOut != NULL) {
56801 |             pRunningFramesOut = ma_offset_ptr(pFramesOut, framesProcessedOut *
> ma_get_bytes_per_frame(pConverter->formatOut, pConverter->channelsOut));
56802 |         }
56803 |
56804 |         /*
56805 |         Before doing any processing we need to determine how many frames we should try processing
56806 |         this iteration, for both input and output. The resampler requires us to perform format and
56807 |         channel conversion before passing any data into it. If we get our input count wrong, we'll
56808 |         end up performing redundant pre-processing. This isn't the end of the world, but it does
56809 |         result in some inefficiencies proportionate to how far our estimates are off.
56810 |
56811 |         If the resampler has a means to calculate exactly how much we'll need, we'll use that.
56812 |         Otherwise we'll make a best guess. In order to do this, we'll need to calculate the output
56813 |         frame count first.
56814 |         */
56815 |         frameCountOutThisIteration = (frameCountOut - framesProcessedOut);
56816 |         if (frameCountOutThisIteration > tempBufferMidCap) {
56817 |             frameCountOutThisIteration = tempBufferMidCap;
56818 |         }
56819 |
56820 |         if (pConverter->hasPostFormatConversion) {
56821 |             if (frameCountOutThisIteration > tempBufferOutCap) {
56822 |                 frameCountOutThisIteration = tempBufferOutCap;
56823 |             }
56824 |         }
56825 |
56826 |         /* Now that we have the output frame count we can determine the input frame count. */
56827 |         frameCountInThisIteration = (frameCountIn - framesProcessedIn);
56828 |         if (pConverter->hasPreFormatConversion) {
56829 |             if (frameCountInThisIteration > tempBufferInCap) {
56830 |                 frameCountInThisIteration = tempBufferInCap;
56831 |             }
56832 |         }
56833 |
56834 |         if (frameCountInThisIteration > tempBufferMidCap) {
56835 |             frameCountInThisIteration = tempBufferMidCap;
56836 |         }
56837 |
56838 |         #if 1
56839 |         {
56840 |             ma_uint64 requiredInputFrameCount;
56841 |
56842 |             result = ma_resampler_get_required_input_frame_count(&pConverter->resampler,
> frameCountOutThisIteration, &requiredInputFrameCount);
56843 |             if (result != MA_SUCCESS) {
56844 |                 /* Fall back to a best guess. */
56845 |                 requiredInputFrameCount = (frameCountOutThisIteration *
> pConverter->resampler.sampleRateIn) / pConverter->resampler.sampleRateOut;
56846 |             }
56847 |
56848 |             if (frameCountInThisIteration > requiredInputFrameCount) {
56849 |                 frameCountInThisIteration = requiredInputFrameCount;
56850 |             }
56851 |         }
56852 |         #endif

```

```

56853 |
56854 |
56855 |     /* Pre format conversion. */
56856 |     if (pConverter->hasPreFormatConversion) {
56857 |         if (pRunningFramesIn != NULL) {
56858 |             ma_convert_pcm_frames_format(pTempBufferIn, pConverter->channelConverter.format,
> pRunningFramesIn, pConverter->formatIn, frameCountInThisIteration, pConverter->channelsIn,
> pConverter->ditherMode);
56859 |             pChannelsBufferIn = pTempBufferIn;
56860 |         } else {
56861 |             pChannelsBufferIn = NULL;
56862 |         }
56863 |     } else {
56864 |         pChannelsBufferIn = pRunningFramesIn;
56865 |     }
56866 |
56867 |
56868 |     /* Channel conversion. */
56869 |     result = ma_channel_converter_process_pcm_frames(&pConverter->channelConverter,
> pTempBufferMid, pChannelsBufferIn, frameCountInThisIteration);
56870 |     if (result != MA_SUCCESS) {
56871 |         return result;
56872 |     }
56873 |
56874 |
56875 |     /* Resampling. */
56876 |     if (pConverter->hasPostFormatConversion) {
56877 |         pResampleBufferOut = pTempBufferOut;
56878 |     } else {
56879 |         pResampleBufferOut = pRunningFramesOut;
56880 |     }
56881 |
56882 |     result = ma_resampler_process_pcm_frames(&pConverter->resampler, pTempBufferMid,
> &frameCountInThisIteration, pResampleBufferOut, &frameCountOutThisIteration);
56883 |     if (result != MA_SUCCESS) {
56884 |         return result;
56885 |     }
56886 |
56887 |
56888 |     /* Post format conversion. */
56889 |     if (pConverter->hasPostFormatConversion) {
56890 |         if (pRunningFramesOut != NULL) {
56891 |             ma_convert_pcm_frames_format(pRunningFramesOut, pConverter->formatOut,
> pResampleBufferOut, pConverter->resampler.format, frameCountOutThisIteration,
> pConverter->channelsOut, pConverter->ditherMode);
56892 |         }
56893 |     }
56894 |
56895 |
56896 |     framesProcessedIn += frameCountInThisIteration;
56897 |     framesProcessedOut += frameCountOutThisIteration;
56898 |
56899 |     MA_ASSERT(framesProcessedIn <= frameCountIn);
56900 |     MA_ASSERT(framesProcessedOut <= frameCountOut);
56901 |
56902 |     if (frameCountOutThisIteration == 0) {
56903 |         break; /* Consumed all of our input data. */
56904 |     }
56905 | }
56906 |
56907 | if (pFrameCountIn != NULL) {
56908 |     *pFrameCountIn = framesProcessedIn;
56909 | }
56910 | if (pFrameCountOut != NULL) {
56911 |     *pFrameCountOut = framesProcessedOut;
56912 | }
56913 |
56914 | return MA_SUCCESS;
56915 | }
56916 |
56917 | MA_API ma_result ma_data_converter_process_pcm_frames(ma_data_converter* pConverter, const void*
> pFramesIn, ma_uint64* pFrameCountIn, void* pFramesOut, ma_uint64* pFrameCountOut)
56918 | {
56919 |     if (pConverter == NULL) {
56920 |         return MA_INVALID_ARGS;
56921 |     }
56922 |
56923 |     switch (pConverter->executionPath)
56924 |     {

```

```

56925 |         case ma_data_converter_execution_path_passthrough:    return
> ma_data_converter_process_pcm_frames__passthrough(pConverter, pFramesIn, pFrameCountIn, pFramesOut,
> pFrameCountOut);
56926 |         case ma_data_converter_execution_path_format_only:    return
> ma_data_converter_process_pcm_frames__format_only(pConverter, pFramesIn, pFrameCountIn, pFramesOut,
> pFrameCountOut);
56927 |         case ma_data_converter_execution_path_channels_only:  return
> ma_data_converter_process_pcm_frames__channels_only(pConverter, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
56928 |         case ma_data_converter_execution_path_resample_only:  return
> ma_data_converter_process_pcm_frames__resample_only(pConverter, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
56929 |         case ma_data_converter_execution_path_resample_first: return
> ma_data_converter_process_pcm_frames__resample_first(pConverter, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
56930 |         case ma_data_converter_execution_path_channels_first: return
> ma_data_converter_process_pcm_frames__channels_first(pConverter, pFramesIn, pFrameCountIn,
> pFramesOut, pFrameCountOut);
56931 |         default: return MA_INVALID_OPERATION;    /* Should never hit this. */
56932 |     }
56933 | }
56934 |
56935 | MA_API ma_result ma_data_converter_set_rate(ma_data_converter* pConverter, ma_uint32 sampleRateIn,
> ma_uint32 sampleRateOut)
56936 | {
56937 |     if (pConverter == NULL) {
56938 |         return MA_INVALID_ARGS;
56939 |     }
56940 |
56941 |     if (pConverter->hasResampler == MA_FALSE) {
56942 |         return MA_INVALID_OPERATION;    /* Dynamic resampling not enabled. */
56943 |     }
56944 |
56945 |     return ma_resampler_set_rate(&pConverter->resampler, sampleRateIn, sampleRateOut);
56946 | }
56947 |
56948 | MA_API ma_result ma_data_converter_set_rate_ratio(ma_data_converter* pConverter, float ratioInOut)
56949 | {
56950 |     if (pConverter == NULL) {
56951 |         return MA_INVALID_ARGS;
56952 |     }
56953 |
56954 |     if (pConverter->hasResampler == MA_FALSE) {
56955 |         return MA_INVALID_OPERATION;    /* Dynamic resampling not enabled. */
56956 |     }
56957 |
56958 |     return ma_resampler_set_rate_ratio(&pConverter->resampler, ratioInOut);
56959 | }
56960 |
56961 | MA_API ma_uint64 ma_data_converter_get_input_latency(const ma_data_converter* pConverter)
56962 | {
56963 |     if (pConverter == NULL) {
56964 |         return 0;
56965 |     }
56966 |
56967 |     if (pConverter->hasResampler) {
56968 |         return ma_resampler_get_input_latency(&pConverter->resampler);
56969 |     }
56970 |
56971 |     return 0;    /* No latency without a resampler. */
56972 | }
56973 |
56974 | MA_API ma_uint64 ma_data_converter_get_output_latency(const ma_data_converter* pConverter)
56975 | {
56976 |     if (pConverter == NULL) {
56977 |         return 0;
56978 |     }
56979 |
56980 |     if (pConverter->hasResampler) {
56981 |         return ma_resampler_get_output_latency(&pConverter->resampler);
56982 |     }
56983 |
56984 |     return 0;    /* No latency without a resampler. */
56985 | }
56986 |
56987 | MA_API ma_result ma_data_converter_get_required_input_frame_count(const ma_data_converter*
> pConverter, ma_uint64 outputFrameCount, ma_uint64* pInputFrameCount)
56988 | {
56989 |     if (pInputFrameCount == NULL) {

```

```
56990 |         return MA_INVALID_ARGS;
56991 |     }
56992 |
56993 |     *pInputFrameCount = 0;
56994 |
56995 |     if (pConverter == NULL) {
56996 |         return MA_INVALID_ARGS;
56997 |     }
56998 |
56999 |     if (pConverter->hasResampler) {
57000 |         return ma_resampler_get_required_input_frame_count(&pConverter->resampler, outputFrameCount,
> pInputFrameCount);
57001 |     } else {
57002 |         *pInputFrameCount = outputFrameCount;    /* 1:1 */
57003 |         return MA_SUCCESS;
57004 |     }
57005 | }
57006 |
57007 | MA_API ma_result ma_data_converter_get_expected_output_frame_count(const ma_data_converter*
> pConverter, ma_uint64 inputFrameCount, ma_uint64* pOutputFrameCount)
57008 | {
57009 |     if (pOutputFrameCount == NULL) {
57010 |         return MA_INVALID_ARGS;
57011 |     }
57012 |
57013 |     *pOutputFrameCount = 0;
57014 |
57015 |     if (pConverter == NULL) {
57016 |         return MA_INVALID_ARGS;
57017 |     }
57018 |
57019 |     if (pConverter->hasResampler) {
57020 |         return ma_resampler_get_expected_output_frame_count(&pConverter->resampler, inputFrameCount,
> pOutputFrameCount);
57021 |     } else {
57022 |         *pOutputFrameCount = inputFrameCount;    /* 1:1 */
57023 |         return MA_SUCCESS;
57024 |     }
57025 | }
57026 |
57027 | MA_API ma_result ma_data_converter_get_input_channel_map(const ma_data_converter* pConverter,
> ma_channel* pChannelMap, size_t channelMapCap)
57028 | {
57029 |     if (pConverter == NULL || pChannelMap == NULL) {
57030 |         return MA_INVALID_ARGS;
57031 |     }
57032 |
57033 |     if (pConverter->hasChannelConverter) {
57034 |         ma_channel_converter_get_output_channel_map(&pConverter->channelConverter, pChannelMap,
> channelMapCap);
57035 |     } else {
57036 |         ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pConverter->channelsOut);
57037 |     }
57038 |
57039 |     return MA_SUCCESS;
57040 | }
57041 |
57042 | MA_API ma_result ma_data_converter_get_output_channel_map(const ma_data_converter* pConverter,
> ma_channel* pChannelMap, size_t channelMapCap)
57043 | {
57044 |     if (pConverter == NULL || pChannelMap == NULL) {
57045 |         return MA_INVALID_ARGS;
57046 |     }
57047 |
57048 |     if (pConverter->hasChannelConverter) {
57049 |         ma_channel_converter_get_input_channel_map(&pConverter->channelConverter, pChannelMap,
> channelMapCap);
57050 |     } else {
57051 |         ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pConverter->channelsIn);
57052 |     }
57053 |
57054 |     return MA_SUCCESS;
57055 | }
57056 |
57057 | MA_API ma_result ma_data_converter_reset(ma_data_converter* pConverter)
57058 | {
57059 |     if (pConverter == NULL) {
```

```

57060 |         return MA_INVALID_ARGS;
57061 |     }
57062 |
57063 |     /* There's nothing to do if we're not resampling. */
57064 |     if (pConverter->hasResampler == MA_FALSE) {
57065 |         return MA_SUCCESS;
57066 |     }
57067 |
57068 |     return ma_resampler_reset(&pConverter->resampler);
57069 | }
57070 |
57071 |
57072 |
57073 | /*****
57074 | > *****/
57075 | Channel Maps
57076 |
57077 | *****/
57078 | static ma_channel ma_channel_map_init_standard_channel(ma_standard_channel_map standardChannelMap,
57079 | > ma_uint32 channelCount, ma_uint32 channelIndex);
57080 | MA_API ma_channel ma_channel_map_get_channel(const ma_channel* pChannelMap, ma_uint32 channelCount,
57081 | > ma_uint32 channelIndex)
57082 | {
57083 |     if (pChannelMap == NULL) {
57084 |         return ma_channel_map_init_standard_channel(ma_standard_channel_map_default, channelCount,
57085 | > channelIndex);
57086 |     } else {
57087 |         if (channelIndex >= channelCount) {
57088 |             return MA_CHANNEL_NONE;
57089 |         }
57090 |         return pChannelMap[channelIndex];
57091 |     }
57092 | }
57093 | MA_API void ma_channel_map_init_blank(ma_channel* pChannelMap, ma_uint32 channels)
57094 | {
57095 |     if (pChannelMap == NULL) {
57096 |         return;
57097 |     }
57098 |
57099 |     MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channels);
57100 | }
57101 |
57102 |
57103 | static ma_channel ma_channel_map_init_standard_channel_microsoft(ma_uint32 channelCount, ma_uint32
57104 | > channelIndex)
57105 | {
57106 |     if (channelCount == 0 || channelIndex >= channelCount) {
57107 |         return MA_CHANNEL_NONE;
57108 |     }
57109 |
57110 |     /* This is the Microsoft channel map. Based off the speaker configurations mentioned here: https
57111 | > ://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/content/ksmedia/ns-ksmedia-ksaudio_channel_
57112 | > config */
57113 |     switch (channelCount)
57114 |     {
57115 |         case 0: return MA_CHANNEL_NONE;
57116 |
57117 |         case 1:
57118 |         {
57119 |             return MA_CHANNEL_MONO;
57120 |         } break;
57121 |
57122 |         case 2:
57123 |         {
57124 |             switch (channelIndex) {
57125 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57126 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57127 |             }
57128 |         } break;
57129 |
57130 |         case 3: /* No defined, but best guess. */
57131 |         {
57132 |             switch (channelIndex) {
57133 |                 case 0: return MA_CHANNEL_FRONT_LEFT;

```

```

57131 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57132 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57133 |     }
57134 | } break;
57135 |
57136 | case 4:
57137 | {
57138 |     switch (channelIndex) {
57139 |         #ifndef MA_USE_QUAD_MICROSOFT_CHANNEL_MAP
57140 |             /* Surround. Using the Surround profile has the advantage of the 3rd channel
> (MA_CHANNEL_FRONT_CENTER) mapping nicely with higher channel counts. */
57141 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57142 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57143 |             case 2: return MA_CHANNEL_FRONT_CENTER;
57144 |             case 3: return MA_CHANNEL_BACK_CENTER;
57145 |         #else
57146 |             /* Quad. */
57147 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57148 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57149 |             case 2: return MA_CHANNEL_BACK_LEFT;
57150 |             case 3: return MA_CHANNEL_BACK_RIGHT;
57151 |         #endif
57152 |     }
57153 | } break;
57154 |
57155 | case 5: /* Not defined, but best guess. */
57156 | {
57157 |     switch (channelIndex) {
57158 |         case 0: return MA_CHANNEL_FRONT_LEFT;
57159 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57160 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57161 |         case 3: return MA_CHANNEL_BACK_LEFT;
57162 |         case 4: return MA_CHANNEL_BACK_RIGHT;
57163 |     }
57164 | } break;
57165 |
57166 | case 6:
57167 | {
57168 |     switch (channelIndex) {
57169 |         case 0: return MA_CHANNEL_FRONT_LEFT;
57170 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57171 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57172 |         case 3: return MA_CHANNEL_LFE;
57173 |         case 4: return MA_CHANNEL_SIDE_LEFT;
57174 |         case 5: return MA_CHANNEL_SIDE_RIGHT;
57175 |     }
57176 | } break;
57177 |
57178 | case 7: /* Not defined, but best guess. */
57179 | {
57180 |     switch (channelIndex) {
57181 |         case 0: return MA_CHANNEL_FRONT_LEFT;
57182 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57183 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57184 |         case 3: return MA_CHANNEL_LFE;
57185 |         case 4: return MA_CHANNEL_BACK_CENTER;
57186 |         case 5: return MA_CHANNEL_SIDE_LEFT;
57187 |         case 6: return MA_CHANNEL_SIDE_RIGHT;
57188 |     }
57189 | } break;
57190 |
57191 | case 8:
57192 | default:
57193 | {
57194 |     switch (channelIndex) {
57195 |         case 0: return MA_CHANNEL_FRONT_LEFT;
57196 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57197 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57198 |         case 3: return MA_CHANNEL_LFE;
57199 |         case 4: return MA_CHANNEL_BACK_LEFT;
57200 |         case 5: return MA_CHANNEL_BACK_RIGHT;
57201 |         case 6: return MA_CHANNEL_SIDE_LEFT;
57202 |         case 7: return MA_CHANNEL_SIDE_RIGHT;
57203 |     }
57204 | } break;
57205 | }
57206 |
57207 | if (channelCount > 8) {
57208 |     if (channelIndex < 32) { /* We have 32 AUX channels. */

```



```
57209 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 8));
57210 |     }
57211 | }
57212 |
57213 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57214 |     return MA_CHANNEL_NONE;
57215 | }
57216 |
57217 | static ma_channel ma_channel_map_init_standard_channel_alsa(ma_uint32 channelCount, ma_uint32
> channelIndex)
57218 | {
57219 |     switch (channelCount)
57220 |     {
57221 |         case 0: return MA_CHANNEL_NONE;
57222 |
57223 |         case 1:
57224 |         {
57225 |             return MA_CHANNEL_MONO;
57226 |         } break;
57227 |
57228 |         case 2:
57229 |         {
57230 |             switch (channelIndex) {
57231 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57232 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57233 |             }
57234 |         } break;
57235 |
57236 |         case 3:
57237 |         {
57238 |             switch (channelIndex) {
57239 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57240 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57241 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57242 |             }
57243 |         } break;
57244 |
57245 |         case 4:
57246 |         {
57247 |             switch (channelIndex) {
57248 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57249 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57250 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57251 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57252 |             }
57253 |         } break;
57254 |
57255 |         case 5:
57256 |         {
57257 |             switch (channelIndex) {
57258 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57259 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57260 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57261 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57262 |                 case 4: return MA_CHANNEL_FRONT_CENTER;
57263 |             }
57264 |         } break;
57265 |
57266 |         case 6:
57267 |         {
57268 |             switch (channelIndex) {
57269 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57270 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57271 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57272 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57273 |                 case 4: return MA_CHANNEL_FRONT_CENTER;
57274 |                 case 5: return MA_CHANNEL_LFE;
57275 |             }
57276 |         } break;
57277 |
57278 |         case 7:
57279 |         {
57280 |             switch (channelIndex) {
57281 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57282 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57283 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57284 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57285 |                 case 4: return MA_CHANNEL_FRONT_CENTER;
```

```

57286 |         case 5: return MA_CHANNEL_LFE;
57287 |         case 6: return MA_CHANNEL_BACK_CENTER;
57288 |     }
57289 | } break;
57290 |
57291 |     case 8:
57292 |     default:
57293 |     {
57294 |         switch (channelIndex) {
57295 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57296 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57297 |             case 2: return MA_CHANNEL_BACK_LEFT;
57298 |             case 3: return MA_CHANNEL_BACK_RIGHT;
57299 |             case 4: return MA_CHANNEL_FRONT_CENTER;
57300 |             case 5: return MA_CHANNEL_LFE;
57301 |             case 6: return MA_CHANNEL_SIDE_LEFT;
57302 |             case 7: return MA_CHANNEL_SIDE_RIGHT;
57303 |         }
57304 |     } break;
57305 | }
57306 |
57307 | if (channelCount > 8) {
57308 |     if (channelIndex < 32) { /* We have 32 AUX channels. */
57309 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 8));
57310 |     }
57311 | }
57312 |
57313 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57314 |     return MA_CHANNEL_NONE;
57315 | }
57316 |
57317 | static ma_channel ma_channel_map_init_standard_channel_rfc3551(ma_uint32 channelCount, ma_uint32
> channelIndex)
57318 | {
57319 |     switch (channelCount)
57320 |     {
57321 |         case 0: return MA_CHANNEL_NONE;
57322 |
57323 |         case 1:
57324 |         {
57325 |             return MA_CHANNEL_MONO;
57326 |         } break;
57327 |
57328 |         case 2:
57329 |         {
57330 |             switch (channelIndex) {
57331 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57332 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57333 |             }
57334 |         } break;
57335 |
57336 |         case 3:
57337 |         {
57338 |             switch (channelIndex) {
57339 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57340 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57341 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57342 |             }
57343 |         } break;
57344 |
57345 |         case 4:
57346 |         {
57347 |             switch (channelIndex) {
57348 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57349 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57350 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57351 |                 case 3: return MA_CHANNEL_BACK_CENTER;
57352 |             }
57353 |         } break;
57354 |
57355 |         case 5:
57356 |         {
57357 |             switch (channelIndex) {
57358 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57359 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57360 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57361 |                 case 3: return MA_CHANNEL_BACK_LEFT;
57362 |                 case 4: return MA_CHANNEL_BACK_RIGHT;

```

```
57363 |     }
57364 |     } break;
57365 |
57366 |     case 6:
57367 |     default:
57368 |     {
57369 |         switch (channelIndex) {
57370 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57371 |             case 1: return MA_CHANNEL_SIDE_LEFT;
57372 |             case 2: return MA_CHANNEL_FRONT_CENTER;
57373 |             case 3: return MA_CHANNEL_FRONT_RIGHT;
57374 |             case 4: return MA_CHANNEL_SIDE_RIGHT;
57375 |             case 5: return MA_CHANNEL_BACK_CENTER;
57376 |         }
57377 |     } break;
57378 | }
57379 |
57380 | if (channelCount > 6) {
57381 |     if (channelIndex < 32) { /* We have 32 AUX channels. */
57382 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 6));
57383 |     }
57384 | }
57385 |
57386 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57387 | return MA_CHANNEL_NONE;
57388 | }
57389 |
57390 | static ma_channel ma_channel_map_init_standard_channel_flac(ma_uint32 channelCount, ma_uint32
> channelIndex)
57391 | {
57392 |     switch (channelCount)
57393 |     {
57394 |         case 0: return MA_CHANNEL_NONE;
57395 |
57396 |         case 1:
57397 |         {
57398 |             return MA_CHANNEL_MONO;
57399 |         } break;
57400 |
57401 |         case 2:
57402 |         {
57403 |             switch (channelIndex) {
57404 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57405 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57406 |             }
57407 |         } break;
57408 |
57409 |         case 3:
57410 |         {
57411 |             switch (channelIndex) {
57412 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57413 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57414 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57415 |             }
57416 |         } break;
57417 |
57418 |         case 4:
57419 |         {
57420 |             switch (channelIndex) {
57421 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57422 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57423 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57424 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57425 |             }
57426 |         } break;
57427 |
57428 |         case 5:
57429 |         {
57430 |             switch (channelIndex) {
57431 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57432 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57433 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57434 |                 case 3: return MA_CHANNEL_BACK_LEFT;
57435 |                 case 4: return MA_CHANNEL_BACK_RIGHT;
57436 |             }
57437 |         } break;
57438 |
57439 |         case 6:
```

```

57440 |     {
57441 |         switch (channelIndex) {
57442 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57443 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57444 |             case 2: return MA_CHANNEL_FRONT_CENTER;
57445 |             case 3: return MA_CHANNEL_LFE;
57446 |             case 4: return MA_CHANNEL_BACK_LEFT;
57447 |             case 5: return MA_CHANNEL_BACK_RIGHT;
57448 |         }
57449 |     } break;
57450 |
57451 | case 7:
57452 | {
57453 |     switch (channelIndex) {
57454 |         case 0: return MA_CHANNEL_FRONT_LEFT;
57455 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57456 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57457 |         case 3: return MA_CHANNEL_LFE;
57458 |         case 4: return MA_CHANNEL_BACK_CENTER;
57459 |         case 5: return MA_CHANNEL_SIDE_LEFT;
57460 |         case 6: return MA_CHANNEL_SIDE_RIGHT;
57461 |     }
57462 | } break;
57463 |
57464 | case 8:
57465 | default:
57466 | {
57467 |     switch (channelIndex) {
57468 |         case 0: return MA_CHANNEL_FRONT_LEFT;
57469 |         case 1: return MA_CHANNEL_FRONT_RIGHT;
57470 |         case 2: return MA_CHANNEL_FRONT_CENTER;
57471 |         case 3: return MA_CHANNEL_LFE;
57472 |         case 4: return MA_CHANNEL_BACK_LEFT;
57473 |         case 5: return MA_CHANNEL_BACK_RIGHT;
57474 |         case 6: return MA_CHANNEL_SIDE_LEFT;
57475 |         case 7: return MA_CHANNEL_SIDE_RIGHT;
57476 |     }
57477 | } break;
57478 | }
57479 |
57480 | if (channelCount > 8) {
57481 |     if (channelIndex < 32) { /* We have 32 AUX channels. */
57482 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 8));
57483 |     }
57484 | }
57485 |
57486 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57487 | return MA_CHANNEL_NONE;
57488 | }
57489 |
57490 | static ma_channel ma_channel_map_init_standard_channel_vorbis(ma_uint32 channelCount, ma_uint32
> channelIndex)
57491 | {
57492 |     switch (channelCount)
57493 |     {
57494 |         case 0: return MA_CHANNEL_NONE;
57495 |
57496 |         case 1:
57497 |         {
57498 |             return MA_CHANNEL_MONO;
57499 |         } break;
57500 |
57501 |         case 2:
57502 |         {
57503 |             switch (channelIndex) {
57504 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57505 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57506 |             }
57507 |         } break;
57508 |
57509 |         case 3:
57510 |         {
57511 |             switch (channelIndex) {
57512 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57513 |                 case 1: return MA_CHANNEL_FRONT_CENTER;
57514 |                 case 2: return MA_CHANNEL_FRONT_RIGHT;
57515 |             }
57516 |         } break;

```

```

57517 |
57518 |     case 4:
57519 |     {
57520 |         switch (channelIndex) {
57521 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57522 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57523 |             case 2: return MA_CHANNEL_BACK_LEFT;
57524 |             case 3: return MA_CHANNEL_BACK_RIGHT;
57525 |         }
57526 |     } break;
57527 |
57528 |     case 5:
57529 |     {
57530 |         switch (channelIndex) {
57531 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57532 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57533 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57534 |             case 3: return MA_CHANNEL_BACK_LEFT;
57535 |             case 4: return MA_CHANNEL_BACK_RIGHT;
57536 |         }
57537 |     } break;
57538 |
57539 |     case 6:
57540 |     {
57541 |         switch (channelIndex) {
57542 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57543 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57544 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57545 |             case 3: return MA_CHANNEL_BACK_LEFT;
57546 |             case 4: return MA_CHANNEL_BACK_RIGHT;
57547 |             case 5: return MA_CHANNEL_LFE;
57548 |         }
57549 |     } break;
57550 |
57551 |     case 7:
57552 |     {
57553 |         switch (channelIndex) {
57554 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57555 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57556 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57557 |             case 3: return MA_CHANNEL_SIDE_LEFT;
57558 |             case 4: return MA_CHANNEL_SIDE_RIGHT;
57559 |             case 5: return MA_CHANNEL_BACK_CENTER;
57560 |             case 6: return MA_CHANNEL_LFE;
57561 |         }
57562 |     } break;
57563 |
57564 |     case 8:
57565 |     default:
57566 |     {
57567 |         switch (channelIndex) {
57568 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57569 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57570 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57571 |             case 3: return MA_CHANNEL_SIDE_LEFT;
57572 |             case 4: return MA_CHANNEL_SIDE_RIGHT;
57573 |             case 5: return MA_CHANNEL_BACK_LEFT;
57574 |             case 6: return MA_CHANNEL_BACK_RIGHT;
57575 |             case 7: return MA_CHANNEL_LFE;
57576 |         }
57577 |     } break;
57578 | }
57579 |
57580 | if (channelCount > 8) {
57581 |     if (channelIndex < 32) { /* We have 32 AUX channels. */
57582 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 8));
57583 |     }
57584 | }
57585 |
57586 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57587 |     return MA_CHANNEL_NONE;
57588 | }
57589 |
57590 | static ma_channel ma_channel_map_init_standard_channel_sound4(ma_uint32 channelCount, ma_uint32
> channelIndex)
57591 | {
57592 |     switch (channelCount)
57593 |     {

```

```
57594 |         case 0: return MA_CHANNEL_NONE;
57595 |
57596 |     case 1:
57597 |     {
57598 |         return MA_CHANNEL_MONO;
57599 |     } break;
57600 |
57601 |     case 2:
57602 |     {
57603 |         switch (channelIndex) {
57604 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57605 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57606 |         }
57607 |     } break;
57608 |
57609 |     case 3:
57610 |     {
57611 |         switch (channelIndex) {
57612 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57613 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57614 |             case 2: return MA_CHANNEL_FRONT_CENTER;
57615 |         }
57616 |     } break;
57617 |
57618 |     case 4:
57619 |     {
57620 |         switch (channelIndex) {
57621 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57622 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57623 |             case 2: return MA_CHANNEL_BACK_LEFT;
57624 |             case 3: return MA_CHANNEL_BACK_RIGHT;
57625 |         }
57626 |     } break;
57627 |
57628 |     case 5:
57629 |     {
57630 |         switch (channelIndex) {
57631 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57632 |             case 1: return MA_CHANNEL_FRONT_RIGHT;
57633 |             case 2: return MA_CHANNEL_FRONT_CENTER;
57634 |             case 3: return MA_CHANNEL_BACK_LEFT;
57635 |             case 4: return MA_CHANNEL_BACK_RIGHT;
57636 |         }
57637 |     } break;
57638 |
57639 |     case 6:
57640 |     {
57641 |         switch (channelIndex) {
57642 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57643 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57644 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57645 |             case 3: return MA_CHANNEL_BACK_LEFT;
57646 |             case 4: return MA_CHANNEL_BACK_RIGHT;
57647 |             case 5: return MA_CHANNEL_LFE;
57648 |         }
57649 |     } break;
57650 |
57651 |     case 7:
57652 |     {
57653 |         switch (channelIndex) {
57654 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57655 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57656 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57657 |             case 3: return MA_CHANNEL_SIDE_LEFT;
57658 |             case 4: return MA_CHANNEL_SIDE_RIGHT;
57659 |             case 5: return MA_CHANNEL_BACK_CENTER;
57660 |             case 6: return MA_CHANNEL_LFE;
57661 |         }
57662 |     } break;
57663 |
57664 |     case 8:
57665 |     default:
57666 |     {
57667 |         switch (channelIndex) {
57668 |             case 0: return MA_CHANNEL_FRONT_LEFT;
57669 |             case 1: return MA_CHANNEL_FRONT_CENTER;
57670 |             case 2: return MA_CHANNEL_FRONT_RIGHT;
57671 |             case 3: return MA_CHANNEL_SIDE_LEFT;
57672 |             case 4: return MA_CHANNEL_SIDE_RIGHT;
```

```

57673 |             case 5: return MA_CHANNEL_BACK_LEFT;
57674 |             case 6: return MA_CHANNEL_BACK_RIGHT;
57675 |             case 7: return MA_CHANNEL_LFE;
57676 |         }
57677 |     } break;
57678 | }
57679 |
57680 | if (channelCount > 8) {
57681 |     if (channelIndex < 32) { /* We have 32 AUX channels. */
57682 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 8));
57683 |     }
57684 | }
57685 |
57686 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57687 |     return MA_CHANNEL_NONE;
57688 | }
57689 |
57690 | static ma_channel ma_channel_map_init_standard_channel_sndio(ma_uint32 channelCount, ma_uint32
> channelIndex)
57691 | {
57692 |     switch (channelCount)
57693 |     {
57694 |         case 0: return MA_CHANNEL_NONE;
57695 |
57696 |         case 1:
57697 |         {
57698 |             return MA_CHANNEL_MONO;
57699 |         } break;
57700 |
57701 |         case 2:
57702 |         {
57703 |             switch (channelIndex) {
57704 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57705 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57706 |             }
57707 |         } break;
57708 |
57709 |         case 3: /* No defined, but best guess. */
57710 |         {
57711 |             switch (channelIndex) {
57712 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57713 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57714 |                 case 2: return MA_CHANNEL_FRONT_CENTER;
57715 |             }
57716 |         } break;
57717 |
57718 |         case 4:
57719 |         {
57720 |             switch (channelIndex) {
57721 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57722 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57723 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57724 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57725 |             }
57726 |         } break;
57727 |
57728 |         case 5: /* Not defined, but best guess. */
57729 |         {
57730 |             switch (channelIndex) {
57731 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57732 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57733 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57734 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57735 |                 case 4: return MA_CHANNEL_FRONT_CENTER;
57736 |             }
57737 |         } break;
57738 |
57739 |         case 6:
57740 |         default:
57741 |         {
57742 |             switch (channelIndex) {
57743 |                 case 0: return MA_CHANNEL_FRONT_LEFT;
57744 |                 case 1: return MA_CHANNEL_FRONT_RIGHT;
57745 |                 case 2: return MA_CHANNEL_BACK_LEFT;
57746 |                 case 3: return MA_CHANNEL_BACK_RIGHT;
57747 |                 case 4: return MA_CHANNEL_FRONT_CENTER;
57748 |                 case 5: return MA_CHANNEL_LFE;
57749 |             }

```



```

57750 |     } break;
57751 | }
57752 |
57753 | if (channelCount > 6) {
57754 |     if (channelIndex < 32) { /* We have 32 AUX channels. */
57755 |         return (ma_channel)(MA_CHANNEL_AUX_0 + (channelIndex - 6));
57756 |     }
57757 | }
57758 |
57759 | /* Getting here means we don't know how to map the channel position so just return
> MA_CHANNEL_NONE. */
57760 |     return MA_CHANNEL_NONE;
57761 | }
57762 |
57763 |
57764 | static ma_channel ma_channel_map_init_standard_channel(ma_standard_channel_map standardChannelMap,
> ma_uint32 channelCount, ma_uint32 channelIndex)
57765 | {
57766 |     if (channelCount == 0 || channelIndex >= channelCount) {
57767 |         return MA_CHANNEL_NONE;
57768 |     }
57769 |
57770 |     switch (standardChannelMap)
57771 |     {
57772 |         case ma_standard_channel_map_alsa:
57773 |         {
57774 |             return ma_channel_map_init_standard_channel_alsa(channelCount, channelIndex);
57775 |         } break;
57776 |
57777 |         case ma_standard_channel_map_rfc3551:
57778 |         {
57779 |             return ma_channel_map_init_standard_channel_rfc3551(channelCount, channelIndex);
57780 |         } break;
57781 |
57782 |         case ma_standard_channel_map_flac:
57783 |         {
57784 |             return ma_channel_map_init_standard_channel_flac(channelCount, channelIndex);
57785 |         } break;
57786 |
57787 |         case ma_standard_channel_map_vorbis:
57788 |         {
57789 |             return ma_channel_map_init_standard_channel_vorbis(channelCount, channelIndex);
57790 |         } break;
57791 |
57792 |         case ma_standard_channel_map_sound4:
57793 |         {
57794 |             return ma_channel_map_init_standard_channel_sound4(channelCount, channelIndex);
57795 |         } break;
57796 |
57797 |         case ma_standard_channel_map_sndio:
57798 |         {
57799 |             return ma_channel_map_init_standard_channel_sndio(channelCount, channelIndex);
57800 |         } break;
57801 |
57802 |         case ma_standard_channel_map_microsoft: /* Also default. */
57803 |         /*case ma_standard_channel_map_default;*/
57804 |         default:
57805 |         {
57806 |             return ma_channel_map_init_standard_channel_microsoft(channelCount, channelIndex);
57807 |         } break;
57808 |     }
57809 | }
57810 |
57811 | MA_API void ma_channel_map_init_standard(ma_standard_channel_map standardChannelMap, ma_channel*
> pChannelMap, size_t channelMapCap, ma_uint32 channels)
57812 | {
57813 |     ma_uint32 iChannel;
57814 |
57815 |     if (pChannelMap == NULL || channelMapCap == 0 || channels == 0) {
57816 |         return;
57817 |     }
57818 |
57819 |     for (iChannel = 0; iChannel < channels; iChannel += 1) {
57820 |         if (channelMapCap == 0) {
57821 |             break; /* Ran out of room. */
57822 |         }
57823 |
57824 |         pChannelMap[0] = ma_channel_map_init_standard_channel(standardChannelMap, channels,
> iChannel);

```

```

57825 |         pChannelMap += 1;
57826 |         channelMapCap -= 1;
57827 |     }
57828 | }
57829 |
57830 | MA_API void ma_channel_map_copy(ma_channel* pOut, const ma_channel* pIn, ma_uint32 channels)
57831 | {
57832 |     if (pOut != NULL && pIn != NULL && channels > 0) {
57833 |         MA_COPY_MEMORY(pOut, pIn, sizeof(*pOut) * channels);
57834 |     }
57835 | }
57836 |
57837 | MA_API void ma_channel_map_copy_or_default(ma_channel* pOut, size_t channelMapCapOut, const
> ma_channel* pIn, ma_uint32 channels)
57838 | {
57839 |     if (pOut == NULL || channels == 0) {
57840 |         return;
57841 |     }
57842 |
57843 |     if (pIn != NULL) {
57844 |         ma_channel_map_copy(pOut, pIn, channels);
57845 |     } else {
57846 |         ma_channel_map_init_standard(ma_standard_channel_map_default, pOut, channelMapCapOut,
> channels);
57847 |     }
57848 | }
57849 |
57850 | MA_API ma_bool32 ma_channel_map_is_valid(const ma_channel* pChannelMap, ma_uint32 channels)
57851 | {
57852 |     /* A channel count of 0 is invalid. */
57853 |     if (channels == 0) {
57854 |         return MA_FALSE;
57855 |     }
57856 |
57857 |     /* It does not make sense to have a mono channel when there is more than 1 channel. */
57858 |     if (channels > 1) {
57859 |         ma_uint32 iChannel;
57860 |         for (iChannel = 0; iChannel < channels; ++iChannel) {
57861 |             if (ma_channel_map_get_channel(pChannelMap, channels, iChannel) == MA_CHANNEL_MONO) {
57862 |                 return MA_FALSE;
57863 |             }
57864 |         }
57865 |     }
57866 |
57867 |     return MA_TRUE;
57868 | }
57869 |
57870 | MA_API ma_bool32 ma_channel_map_is_equal(const ma_channel* pChannelMapA, const ma_channel*
> pChannelMapB, ma_uint32 channels)
57871 | {
57872 |     ma_uint32 iChannel;
57873 |
57874 |     if (pChannelMapA == pChannelMapB) {
57875 |         return MA_TRUE;
57876 |     }
57877 |
57878 |     for (iChannel = 0; iChannel < channels; ++iChannel) {
57879 |         if (ma_channel_map_get_channel(pChannelMapA, channels, iChannel) !=
> ma_channel_map_get_channel(pChannelMapB, channels, iChannel)) {
57880 |             return MA_FALSE;
57881 |         }
57882 |     }
57883 |
57884 |     return MA_TRUE;
57885 | }
57886 |
57887 | MA_API ma_bool32 ma_channel_map_is_blank(const ma_channel* pChannelMap, ma_uint32 channels)
57888 | {
57889 |     ma_uint32 iChannel;
57890 |
57891 |     /* A null channel map is equivalent to the default channel map. */
57892 |     if (pChannelMap == NULL) {
57893 |         return MA_FALSE;
57894 |     }
57895 |
57896 |     for (iChannel = 0; iChannel < channels; ++iChannel) {
57897 |         if (pChannelMap[iChannel] != MA_CHANNEL_NONE) {
57898 |             return MA_FALSE;
57899 |         }

```

```

57900 |     }
57901 |
57902 |     return MA_TRUE;
57903 | }
57904 |
57905 | MA_API ma_bool32 ma_channel_map_contains_channel_position(ma_uint32 channels, const ma_channel*
> pChannelMap, ma_channel channelPosition)
57906 | {
57907 |     return ma_channel_map_find_channel_position(channels, pChannelMap, channelPosition, NULL);
57908 | }
57909 |
57910 | MA_API ma_bool32 ma_channel_map_find_channel_position(ma_uint32 channels, const ma_channel*
> pChannelMap, ma_channel channelPosition, ma_uint32* pChannelIndex)
57911 | {
57912 |     ma_uint32 iChannel;
57913 |
57914 |     if (pChannelIndex != NULL) {
57915 |         *pChannelIndex = (ma_uint32)-1;
57916 |     }
57917 |
57918 |     for (iChannel = 0; iChannel < channels; ++iChannel) {
57919 |         if (ma_channel_map_get_channel(pChannelMap, channels, iChannel) == channelPosition) {
57920 |             if (pChannelIndex != NULL) {
57921 |                 *pChannelIndex = iChannel;
57922 |             }
57923 |
57924 |             return MA_TRUE;
57925 |         }
57926 |     }
57927 |
57928 |     /* Getting here means the channel position was not found. */
57929 |     return MA_FALSE;
57930 | }
57931 |
57932 | MA_API size_t ma_channel_map_to_string(const ma_channel* pChannelMap, ma_uint32 channels, char*
> pBufferOut, size_t bufferCap)
57933 | {
57934 |     size_t len;
57935 |     ma_uint32 iChannel;
57936 |
57937 |     len = 0;
57938 |
57939 |     for (iChannel = 0; iChannel < channels; iChannel += 1) {
57940 |         const char* pChannelStr =
> ma_channel_position_to_string(ma_channel_map_get_channel(pChannelMap, channels, iChannel));
57941 |         size_t channelStrLen = strlen(pChannelStr);
57942 |
57943 |         /* Append the string if necessary. */
57944 |         if (pBufferOut != NULL && bufferCap > len + channelStrLen) {
57945 |             MA_COPY_MEMORY(pBufferOut + len, pChannelStr, channelStrLen);
57946 |         }
57947 |         len += channelStrLen;
57948 |
57949 |         /* Append a space if it's not the last item. */
57950 |         if (iChannel+1 < channels) {
57951 |             if (pBufferOut != NULL && bufferCap > len + 1) {
57952 |                 pBufferOut[len] = ' ';
57953 |             }
57954 |             len += 1;
57955 |         }
57956 |     }
57957 |
57958 |     /* Null terminate. Don't increment the length here. */
57959 |     if (pBufferOut != NULL) {
57960 |         if (bufferCap > len) {
57961 |             pBufferOut[len] = '\0';
57962 |         } else if (bufferCap > 0) {
57963 |             pBufferOut[bufferCap - 1] = '\0';
57964 |         }
57965 |     }
57966 |
57967 |     return len;
57968 | }
57969 |
57970 | MA_API const char* ma_channel_position_to_string(ma_channel channel)
57971 | {
57972 |     switch (channel)
57973 |     {
57974 |         case MA_CHANNEL_NONE : return "CHANNEL_NONE";

```

```

57975 |         case MA_CHANNEL_MONO           : return "CHANNEL_MONO";
57976 |         case MA_CHANNEL_FRONT_LEFT     : return "CHANNEL_FRONT_LEFT";
57977 |         case MA_CHANNEL_FRONT_RIGHT    : return "CHANNEL_FRONT_RIGHT";
57978 |         case MA_CHANNEL_FRONT_CENTER   : return "CHANNEL_FRONT_CENTER";
57979 |         case MA_CHANNEL_LFE            : return "CHANNEL_LFE";
57980 |         case MA_CHANNEL_BACK_LEFT      : return "CHANNEL_BACK_LEFT";
57981 |         case MA_CHANNEL_BACK_RIGHT     : return "CHANNEL_BACK_RIGHT";
57982 |         case MA_CHANNEL_FRONT_LEFT_CENTER : return "CHANNEL_FRONT_LEFT_CENTER";
57983 |         case MA_CHANNEL_FRONT_RIGHT_CENTER : return "CHANNEL_FRONT_RIGHT_CENTER";
57984 |         case MA_CHANNEL_BACK_CENTER    : return "CHANNEL_BACK_CENTER";
57985 |         case MA_CHANNEL_SIDE_LEFT      : return "CHANNEL_SIDE_LEFT";
57986 |         case MA_CHANNEL_SIDE_RIGHT     : return "CHANNEL_SIDE_RIGHT";
57987 |         case MA_CHANNEL_TOP_CENTER      : return "CHANNEL_TOP_CENTER";
57988 |         case MA_CHANNEL_TOP_FRONT_LEFT : return "CHANNEL_TOP_FRONT_LEFT";
57989 |         case MA_CHANNEL_TOP_FRONT_CENTER : return "CHANNEL_TOP_FRONT_CENTER";
57990 |         case MA_CHANNEL_TOP_FRONT_RIGHT : return "CHANNEL_TOP_FRONT_RIGHT";
57991 |         case MA_CHANNEL_TOP_BACK_LEFT  : return "CHANNEL_TOP_BACK_LEFT";
57992 |         case MA_CHANNEL_TOP_BACK_CENTER : return "CHANNEL_TOP_BACK_CENTER";
57993 |         case MA_CHANNEL_TOP_BACK_RIGHT : return "CHANNEL_TOP_BACK_RIGHT";
57994 |         case MA_CHANNEL_AUX_0          : return "CHANNEL_AUX_0";
57995 |         case MA_CHANNEL_AUX_1          : return "CHANNEL_AUX_1";
57996 |         case MA_CHANNEL_AUX_2          : return "CHANNEL_AUX_2";
57997 |         case MA_CHANNEL_AUX_3          : return "CHANNEL_AUX_3";
57998 |         case MA_CHANNEL_AUX_4          : return "CHANNEL_AUX_4";
57999 |         case MA_CHANNEL_AUX_5          : return "CHANNEL_AUX_5";
58000 |         case MA_CHANNEL_AUX_6          : return "CHANNEL_AUX_6";
58001 |         case MA_CHANNEL_AUX_7          : return "CHANNEL_AUX_7";
58002 |         case MA_CHANNEL_AUX_8          : return "CHANNEL_AUX_8";
58003 |         case MA_CHANNEL_AUX_9          : return "CHANNEL_AUX_9";
58004 |         case MA_CHANNEL_AUX_10         : return "CHANNEL_AUX_10";
58005 |         case MA_CHANNEL_AUX_11         : return "CHANNEL_AUX_11";
58006 |         case MA_CHANNEL_AUX_12         : return "CHANNEL_AUX_12";
58007 |         case MA_CHANNEL_AUX_13         : return "CHANNEL_AUX_13";
58008 |         case MA_CHANNEL_AUX_14         : return "CHANNEL_AUX_14";
58009 |         case MA_CHANNEL_AUX_15         : return "CHANNEL_AUX_15";
58010 |         case MA_CHANNEL_AUX_16         : return "CHANNEL_AUX_16";
58011 |         case MA_CHANNEL_AUX_17         : return "CHANNEL_AUX_17";
58012 |         case MA_CHANNEL_AUX_18         : return "CHANNEL_AUX_18";
58013 |         case MA_CHANNEL_AUX_19         : return "CHANNEL_AUX_19";
58014 |         case MA_CHANNEL_AUX_20         : return "CHANNEL_AUX_20";
58015 |         case MA_CHANNEL_AUX_21         : return "CHANNEL_AUX_21";
58016 |         case MA_CHANNEL_AUX_22         : return "CHANNEL_AUX_22";
58017 |         case MA_CHANNEL_AUX_23         : return "CHANNEL_AUX_23";
58018 |         case MA_CHANNEL_AUX_24         : return "CHANNEL_AUX_24";
58019 |         case MA_CHANNEL_AUX_25         : return "CHANNEL_AUX_25";
58020 |         case MA_CHANNEL_AUX_26         : return "CHANNEL_AUX_26";
58021 |         case MA_CHANNEL_AUX_27         : return "CHANNEL_AUX_27";
58022 |         case MA_CHANNEL_AUX_28         : return "CHANNEL_AUX_28";
58023 |         case MA_CHANNEL_AUX_29         : return "CHANNEL_AUX_29";
58024 |         case MA_CHANNEL_AUX_30         : return "CHANNEL_AUX_30";
58025 |         case MA_CHANNEL_AUX_31         : return "CHANNEL_AUX_31";
58026 |         default: break;
58027 |     }
58028 |
58029 |     return "UNKNOWN";
58030 | }
58031 |
58032 |
58033 |
58034 | /*****
> *****/
58035 |
58036 | Conversion Helpers
58037 |
58038 | *****/
> *****/
58039 | MA_API ma_uint64 ma_convert_frames(void* pOut, ma_uint64 frameCountOut, ma_format formatOut,
> ma_uint32 channelsOut, ma_uint32 sampleRateOut, const void* pIn, ma_uint64 frameCountIn, ma_format
> formatIn, ma_uint32 channelsIn, ma_uint32 sampleRateIn)
58040 | {
58041 |     ma_data_converter_config config;
58042 |
58043 |     config = ma_data_converter_config_init(formatIn, formatOut, channelsIn, channelsOut,
> sampleRateIn, sampleRateOut);
58044 |     config.resampling.linear.lpfOrder = ma_min(MA_DEFAULT_RESAMPLER_LPF_ORDER, MA_MAX_FILTER_ORDER);
58045 |
58046 |     return ma_convert_frames_ex(pOut, frameCountOut, pIn, frameCountIn, &config);
58047 | }
58048 |

```

```

58049 | MA_API ma_uint64 ma_convert_frames_ex(void* pOut, ma_uint64 frameCountOut, const void* pIn,
> ma_uint64 frameCountIn, const ma_data_converter_config* pConfig)
58050 | {
58051 |     ma_result result;
58052 |     ma_data_converter converter;
58053 |
58054 |     if (frameCountIn == 0 || pConfig == NULL) {
58055 |         return 0;
58056 |     }
58057 |
58058 |     result = ma_data_converter_init(pConfig, NULL, &converter);
58059 |     if (result != MA_SUCCESS) {
58060 |         return 0; /* Failed to initialize the data converter. */
58061 |     }
58062 |
58063 |     if (pOut == NULL) {
58064 |         result = ma_data_converter_get_expected_output_frame_count(&converter, frameCountIn,
> &frameCountOut);
58065 |         if (result != MA_SUCCESS) {
58066 |             if (result == MA_NOT_IMPLEMENTED) {
58067 |                 /* No way to calculate the number of frames, so we'll need to brute force it and
> loop. */
58068 |                     frameCountOut = 0;
58069 |
58070 |                     while (frameCountIn > 0) {
58071 |                         ma_uint64 framesProcessedIn = frameCountIn;
58072 |                         ma_uint64 framesProcessedOut = 0xFFFFFFFF;
58073 |
58074 |                         result = ma_data_converter_process_pcm_frames(&converter, pIn,
> &framesProcessedIn, NULL, &framesProcessedOut);
58075 |                         if (result != MA_SUCCESS) {
58076 |                             break;
58077 |                         }
58078 |
58079 |                         frameCountIn -= framesProcessedIn;
58080 |                     }
58081 |                 }
58082 |             } else {
58083 |                 result = ma_data_converter_process_pcm_frames(&converter, pIn, &frameCountIn, pOut,
> &frameCountOut);
58084 |                 if (result != MA_SUCCESS) {
58085 |                     frameCountOut = 0;
58086 |                 }
58087 |             }
58088 |         }
58089 |
58090 |         ma_data_converter_uninit(&converter, NULL);
58091 |         return frameCountOut;
58092 |     }
58093 | }
58094 |
58095 | /*****
> *****/
58096 |
58097 | Ring Buffer
58098 |
58099 | *****/
> *****/
58100 | static MA_INLINE ma_uint32 ma_rb__extract_offset_in_bytes(ma_uint32 encodedOffset)
58101 | {
58102 |     return encodedOffset & 0x7FFFFFFF;
58103 | }
58104 |
58105 | static MA_INLINE ma_uint32 ma_rb__extract_offset_loop_flag(ma_uint32 encodedOffset)
58106 | {
58107 |     return encodedOffset & 0x80000000;
58108 | }
58109 |
58110 | static MA_INLINE void* ma_rb__get_read_ptr(ma_rb* pRB)
58111 | {
58112 |     MA_ASSERT(pRB != NULL);
58113 |     return ma_offset_ptr(pRB->pBuffer,
> ma_rb__extract_offset_in_bytes(ma_atomic_load_32(&pRB->encodedReadOffset)));
58114 | }
58115 |
58116 | static MA_INLINE void* ma_rb__get_write_ptr(ma_rb* pRB)
58117 | {
58118 |     MA_ASSERT(pRB != NULL);
58119 |     return ma_offset_ptr(pRB->pBuffer,

```

```

> ma_rb__extract_offset_in_bytes(ma_atomic_load_32(&pRB->encodedWriteOffset)));
58120 | }
58121 |
58122 | static MA_INLINE ma_uint32 ma_rb__construct_offset(ma_uint32 offsetInBytes, ma_uint32
> offsetLoopFlag)
58123 | {
58124 |     return offsetLoopFlag | offsetInBytes;
58125 | }
58126 |
58127 | static MA_INLINE void ma_rb__deconstruct_offset(ma_uint32 encodedOffset, ma_uint32* pOffsetInBytes,
> ma_uint32* pOffsetLoopFlag)
58128 | {
58129 |     MA_ASSERT(pOffsetInBytes != NULL);
58130 |     MA_ASSERT(pOffsetLoopFlag != NULL);
58131 |
58132 |     *pOffsetInBytes = ma_rb__extract_offset_in_bytes(encodedOffset);
58133 |     *pOffsetLoopFlag = ma_rb__extract_offset_loop_flag(encodedOffset);
58134 | }
58135 |
58136 |
58137 | MA_API ma_result ma_rb_init_ex(size_t subbufferSizeInBytes, size_t subbufferCount, size_t
> subbufferStrideInBytes, void* pOptionalPreallocatedBuffer, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_rb* pRB)
58138 | {
58139 |     ma_result result;
58140 |     const ma_uint32 maxSubBufferSize = 0x7FFFFFFF - (MA_SIMD_ALIGNMENT-1);
58141 |
58142 |     if (pRB == NULL) {
58143 |         return MA_INVALID_ARGS;
58144 |     }
58145 |
58146 |     if (subbufferSizeInBytes == 0 || subbufferCount == 0) {
58147 |         return MA_INVALID_ARGS;
58148 |     }
58149 |
58150 |     if (subbufferSizeInBytes > maxSubBufferSize) {
58151 |         return MA_INVALID_ARGS; /* Maximum buffer size is ~2GB. The most significant bit is a
> flag for use internally. */
58152 |     }
58153 |
58154 |
58155 |     MA_ZERO_OBJECT(pRB);
58156 |
58157 |     result = ma_allocation_callbacks_init_copy(&pRB->allocationCallbacks, pAllocationCallbacks);
58158 |     if (result != MA_SUCCESS) {
58159 |         return result;
58160 |     }
58161 |
58162 |     pRB->subbufferSizeInBytes = (ma_uint32)subbufferSizeInBytes;
58163 |     pRB->subbufferCount = (ma_uint32)subbufferCount;
58164 |
58165 |     if (pOptionalPreallocatedBuffer != NULL) {
58166 |         pRB->subbufferStrideInBytes = (ma_uint32)subbufferStrideInBytes;
58167 |         pRB->pBuffer = pOptionalPreallocatedBuffer;
58168 |     } else {
58169 |         size_t bufferSizeInBytes;
58170 |
58171 |         /*
58172 |         > Here is where we allocate our own buffer. We always want to align this to MA_SIMD_ALIGNMENT
> for future SIMD optimization opportunity. To do this
58173 |         we need to make sure the stride is a multiple of MA_SIMD_ALIGNMENT.
58174 |         */
58175 |         pRB->subbufferStrideInBytes = ma_align(pRB->subbufferSizeInBytes, MA_SIMD_ALIGNMENT);
58176 |
58177 |         bufferSizeInBytes = (size_t)pRB->subbufferCount*pRB->subbufferStrideInBytes;
58178 |         pRB->pBuffer = ma_aligned_malloc(bufferSizeInBytes, MA_SIMD_ALIGNMENT,
> &pRB->allocationCallbacks);
58179 |         if (pRB->pBuffer == NULL) {
58180 |             return MA_OUT_OF_MEMORY;
58181 |         }
58182 |
58183 |         MA_ZERO_MEMORY(pRB->pBuffer, bufferSizeInBytes);
58184 |         pRB->ownsBuffer = MA_TRUE;
58185 |     }
58186 |
58187 |     return MA_SUCCESS;
58188 | }
58189 |
58190 | MA_API ma_result ma_rb_init(size_t bufferSizeInBytes, void* pOptionalPreallocatedBuffer, const

```

```

> ma_allocation_callbacks* pAllocationCallbacks, ma_rb* pRB)
58191 | {
58192 |     return ma_rb_init_ex(bufferSizeInBytes, 1, 0, pOptionalPreallocatedBuffer, pAllocationCallbacks,
> pRB);
58193 | }
58194 |
58195 | MA_API void ma_rb_uninit(ma_rb* pRB)
58196 | {
58197 |     if (pRB == NULL) {
58198 |         return;
58199 |     }
58200 |
58201 |     if (pRB->ownsBuffer) {
58202 |         ma_aligned_free(pRB->pBuffer, &pRB->allocationCallbacks);
58203 |     }
58204 | }
58205 |
58206 | MA_API void ma_rb_reset(ma_rb* pRB)
58207 | {
58208 |     if (pRB == NULL) {
58209 |         return;
58210 |     }
58211 |
58212 |     ma_atomic_exchange_32(&pRB->encodedReadOffset, 0);
58213 |     ma_atomic_exchange_32(&pRB->encodedWriteOffset, 0);
58214 | }
58215 |
58216 | MA_API ma_result ma_rb_acquire_read(ma_rb* pRB, size_t* pSizeInBytes, void** ppBufferOut)
58217 | {
58218 |     ma_uint32 writeOffset;
58219 |     ma_uint32 writeOffsetInBytes;
58220 |     ma_uint32 writeOffsetLoopFlag;
58221 |     ma_uint32 readOffset;
58222 |     ma_uint32 readOffsetInBytes;
58223 |     ma_uint32 readOffsetLoopFlag;
58224 |     size_t bytesAvailable;
58225 |     size_t bytesRequested;
58226 |
58227 |     if (pRB == NULL || pSizeInBytes == NULL || ppBufferOut == NULL) {
58228 |         return MA_INVALID_ARGS;
58229 |     }
58230 |
58231 |     /* The returned buffer should never move ahead of the write pointer. */
58232 |     writeOffset = ma_atomic_load_32(&pRB->encodedWriteOffset);
58233 |     ma_rb_deconstruct_offset(writeOffset, &writeOffsetInBytes, &writeOffsetLoopFlag);
58234 |
58235 |     readOffset = ma_atomic_load_32(&pRB->encodedReadOffset);
58236 |     ma_rb_deconstruct_offset(readOffset, &readOffsetInBytes, &readOffsetLoopFlag);
58237 |
58238 |     /*
58239 |     The number of bytes available depends on whether or not the read and write pointers are on the
> same loop iteration. If so, we
58240 |     can only read up to the write pointer. If not, we can only read up to the end of the buffer.
58241 |     */
58242 |     if (readOffsetLoopFlag == writeOffsetLoopFlag) {
58243 |         bytesAvailable = writeOffsetInBytes - readOffsetInBytes;
58244 |     } else {
58245 |         bytesAvailable = pRB->subbufferSizeInBytes - readOffsetInBytes;
58246 |     }
58247 |
58248 |     bytesRequested = *pSizeInBytes;
58249 |     if (bytesRequested > bytesAvailable) {
58250 |         bytesRequested = bytesAvailable;
58251 |     }
58252 |
58253 |     *pSizeInBytes = bytesRequested;
58254 |     (*ppBufferOut) = ma_rb_get_read_ptr(pRB);
58255 |
58256 |     return MA_SUCCESS;
58257 | }
58258 |
58259 | MA_API ma_result ma_rb_commit_read(ma_rb* pRB, size_t sizeInBytes)
58260 | {
58261 |     ma_uint32 readOffset;
58262 |     ma_uint32 readOffsetInBytes;
58263 |     ma_uint32 readOffsetLoopFlag;
58264 |     ma_uint32 newReadOffsetInBytes;
58265 |     ma_uint32 newReadOffsetLoopFlag;
58266 |

```



```

58267 |     if (pRB == NULL) {
58268 |         return MA_INVALID_ARGS;
58269 |     }
58270 |
58271 |     readOffset = ma_atomic_load_32(&pRB->encodedReadOffset);
58272 |     ma_rb__deconstruct_offset(readOffset, &readOffsetInBytes, &readOffsetLoopFlag);
58273 |
58274 |     /* Check that sizeInBytes is correct. It should never go beyond the end of the buffer. */
58275 |     newReadOffsetInBytes = (ma_uint32)(readOffsetInBytes + sizeInBytes);
58276 |     if (newReadOffsetInBytes > pRB->subbufferSizeInBytes) {
58277 |         return MA_INVALID_ARGS; /* <-- sizeInBytes will cause the read offset to overflow. */
58278 |     }
58279 |
58280 |     /* Move the read pointer back to the start if necessary. */
58281 |     newReadOffsetLoopFlag = readOffsetLoopFlag;
58282 |     if (newReadOffsetInBytes == pRB->subbufferSizeInBytes) {
58283 |         newReadOffsetInBytes = 0;
58284 |         newReadOffsetLoopFlag ^= 0x80000000;
58285 |     }
58286 |
58287 |     ma_atomic_exchange_32(&pRB->encodedReadOffset, ma_rb__construct_offset(newReadOffsetInBytes,
> newReadOffsetLoopFlag));
58288 |
58289 |     return MA_SUCCESS;
58290 | }
58291 |
58292 | MA_API ma_result ma_rb_acquire_write(ma_rb* pRB, size_t* pSizeInBytes, void** ppBufferOut)
58293 | {
58294 |     ma_uint32 readOffset;
58295 |     ma_uint32 readOffsetInBytes;
58296 |     ma_uint32 readOffsetLoopFlag;
58297 |     ma_uint32 writeOffset;
58298 |     ma_uint32 writeOffsetInBytes;
58299 |     ma_uint32 writeOffsetLoopFlag;
58300 |     size_t bytesAvailable;
58301 |     size_t bytesRequested;
58302 |
58303 |     if (pRB == NULL || pSizeInBytes == NULL || ppBufferOut == NULL) {
58304 |         return MA_INVALID_ARGS;
58305 |     }
58306 |
58307 |     /* The returned buffer should never overtake the read buffer. */
58308 |     readOffset = ma_atomic_load_32(&pRB->encodedReadOffset);
58309 |     ma_rb__deconstruct_offset(readOffset, &readOffsetInBytes, &readOffsetLoopFlag);
58310 |
58311 |     writeOffset = ma_atomic_load_32(&pRB->encodedWriteOffset);
58312 |     ma_rb__deconstruct_offset(writeOffset, &writeOffsetInBytes, &writeOffsetLoopFlag);
58313 |
58314 |     /*
58315 |     In the case of writing, if the write pointer and the read pointer are on the same loop iteration
> we can only
58316 |     write up to the end of the buffer. Otherwise we can only write up to the read pointer. The write
> pointer should
58317 |     never overtake the read pointer.
58318 |     */
58319 |     if (writeOffsetLoopFlag == readOffsetLoopFlag) {
58320 |         bytesAvailable = pRB->subbufferSizeInBytes - writeOffsetInBytes;
58321 |     } else {
58322 |         bytesAvailable = readOffsetInBytes - writeOffsetInBytes;
58323 |     }
58324 |
58325 |     bytesRequested = *pSizeInBytes;
58326 |     if (bytesRequested > bytesAvailable) {
58327 |         bytesRequested = bytesAvailable;
58328 |     }
58329 |
58330 |     *pSizeInBytes = bytesRequested;
58331 |     *ppBufferOut = ma_rb__get_write_ptr(pRB);
58332 |
58333 |     /* Clear the buffer if desired. */
58334 |     if (pRB->clearOnWriteAcquire) {
58335 |         MA_ZERO_MEMORY(*ppBufferOut, *pSizeInBytes);
58336 |     }
58337 |
58338 |     return MA_SUCCESS;
58339 | }
58340 |
58341 | MA_API ma_result ma_rb_commit_write(ma_rb* pRB, size_t sizeInBytes)
58342 | {

```

```

58343 |     ma_uint32 writeOffset;
58344 |     ma_uint32 writeOffsetInBytes;
58345 |     ma_uint32 writeOffsetLoopFlag;
58346 |     ma_uint32 newWriteOffsetInBytes;
58347 |     ma_uint32 newWriteOffsetLoopFlag;
58348 |
58349 |     if (pRB == NULL) {
58350 |         return MA_INVALID_ARGS;
58351 |     }
58352 |
58353 |     writeOffset = ma_atomic_load_32(&pRB->encodedWriteOffset);
58354 |     ma_rb__deconstruct_offset(writeOffset, &writeOffsetInBytes, &writeOffsetLoopFlag);
58355 |
58356 |     /* Check that sizeInBytes is correct. It should never go beyond the end of the buffer. */
58357 |     newWriteOffsetInBytes = (ma_uint32)(writeOffsetInBytes + sizeInBytes);
58358 |     if (newWriteOffsetInBytes > pRB->subbufferSizeInBytes) {
58359 |         return MA_INVALID_ARGS; /* <-- sizeInBytes will cause the read offset to overflow. */
58360 |     }
58361 |
58362 |     /* Move the read pointer back to the start if necessary. */
58363 |     newWriteOffsetLoopFlag = writeOffsetLoopFlag;
58364 |     if (newWriteOffsetInBytes == pRB->subbufferSizeInBytes) {
58365 |         newWriteOffsetInBytes = 0;
58366 |         newWriteOffsetLoopFlag ^= 0x80000000;
58367 |     }
58368 |
58369 |     ma_atomic_exchange_32(&pRB->encodedWriteOffset, ma_rb__construct_offset(newWriteOffsetInBytes,
> newWriteOffsetLoopFlag));
58370 |
58371 |     return MA_SUCCESS;
58372 | }
58373 |
58374 | MA_API ma_result ma_rb_seek_read(ma_rb* pRB, size_t offsetInBytes)
58375 | {
58376 |     ma_uint32 readOffset;
58377 |     ma_uint32 readOffsetInBytes;
58378 |     ma_uint32 readOffsetLoopFlag;
58379 |     ma_uint32 writeOffset;
58380 |     ma_uint32 writeOffsetInBytes;
58381 |     ma_uint32 writeOffsetLoopFlag;
58382 |     ma_uint32 newReadOffsetInBytes;
58383 |     ma_uint32 newReadOffsetLoopFlag;
58384 |
58385 |     if (pRB == NULL || offsetInBytes > pRB->subbufferSizeInBytes) {
58386 |         return MA_INVALID_ARGS;
58387 |     }
58388 |
58389 |     readOffset = ma_atomic_load_32(&pRB->encodedReadOffset);
58390 |     ma_rb__deconstruct_offset(readOffset, &readOffsetInBytes, &readOffsetLoopFlag);
58391 |
58392 |     writeOffset = ma_atomic_load_32(&pRB->encodedWriteOffset);
58393 |     ma_rb__deconstruct_offset(writeOffset, &writeOffsetInBytes, &writeOffsetLoopFlag);
58394 |
58395 |     newReadOffsetLoopFlag = readOffsetLoopFlag;
58396 |
58397 |     /* We cannot go past the write buffer. */
58398 |     if (readOffsetLoopFlag == writeOffsetLoopFlag) {
58399 |         if ((readOffsetInBytes + offsetInBytes) > writeOffsetInBytes) {
58400 |             newReadOffsetInBytes = writeOffsetInBytes;
58401 |         } else {
58402 |             newReadOffsetInBytes = (ma_uint32)(readOffsetInBytes + offsetInBytes);
58403 |         }
58404 |     } else {
58405 |         /* May end up looping. */
58406 |         if ((readOffsetInBytes + offsetInBytes) >= pRB->subbufferSizeInBytes) {
58407 |             newReadOffsetInBytes = (ma_uint32)(readOffsetInBytes + offsetInBytes) -
> pRB->subbufferSizeInBytes;
58408 |             newReadOffsetLoopFlag ^= 0x80000000; /* <-- Looped. */
58409 |         } else {
58410 |             newReadOffsetInBytes = (ma_uint32)(readOffsetInBytes + offsetInBytes);
58411 |         }
58412 |     }
58413 |
58414 |     ma_atomic_exchange_32(&pRB->encodedReadOffset, ma_rb__construct_offset(newReadOffsetInBytes,
> newReadOffsetLoopFlag));
58415 |     return MA_SUCCESS;
58416 | }
58417 |
58418 | MA_API ma_result ma_rb_seek_write(ma_rb* pRB, size_t offsetInBytes)

```

```

58419 | {
58420 |     ma_uint32 readOffset;
58421 |     ma_uint32 readOffsetInBytes;
58422 |     ma_uint32 readOffsetLoopFlag;
58423 |     ma_uint32 writeOffset;
58424 |     ma_uint32 writeOffsetInBytes;
58425 |     ma_uint32 writeOffsetLoopFlag;
58426 |     ma_uint32 newWriteOffsetInBytes;
58427 |     ma_uint32 newWriteOffsetLoopFlag;
58428 |
58429 |     if (pRB == NULL) {
58430 |         return MA_INVALID_ARGS;
58431 |     }
58432 |
58433 |     readOffset = ma_atomic_load_32(&pRB->encodedReadOffset);
58434 |     ma_rb__deconstruct_offset(readOffset, &readOffsetInBytes, &readOffsetLoopFlag);
58435 |
58436 |     writeOffset = ma_atomic_load_32(&pRB->encodedWriteOffset);
58437 |     ma_rb__deconstruct_offset(writeOffset, &writeOffsetInBytes, &writeOffsetLoopFlag);
58438 |
58439 |     newWriteOffsetLoopFlag = writeOffsetLoopFlag;
58440 |
58441 |     /* We cannot go past the write buffer. */
58442 |     if (readOffsetLoopFlag == writeOffsetLoopFlag) {
58443 |         /* May end up looping. */
58444 |         if ((writeOffsetInBytes + offsetInBytes) >= pRB->subbufferSizeInBytes) {
58445 |             newWriteOffsetInBytes = (ma_uint32)(writeOffsetInBytes + offsetInBytes) -
> pRB->subbufferSizeInBytes;
58446 |             newWriteOffsetLoopFlag ^= 0x80000000; /* <-- Looped. */
58447 |         } else {
58448 |             newWriteOffsetInBytes = (ma_uint32)(writeOffsetInBytes + offsetInBytes);
58449 |         }
58450 |     } else {
58451 |         if ((writeOffsetInBytes + offsetInBytes) > readOffsetInBytes) {
58452 |             newWriteOffsetInBytes = readOffsetInBytes;
58453 |         } else {
58454 |             newWriteOffsetInBytes = (ma_uint32)(writeOffsetInBytes + offsetInBytes);
58455 |         }
58456 |     }
58457 |
58458 |     ma_atomic_exchange_32(&pRB->encodedWriteOffset, ma_rb__construct_offset(newWriteOffsetInBytes,
> newWriteOffsetLoopFlag));
58459 |     return MA_SUCCESS;
58460 | }
58461 |
58462 | MA_API ma_int32 ma_rb_pointer_distance(ma_rb* pRB)
58463 | {
58464 |     ma_uint32 readOffset;
58465 |     ma_uint32 readOffsetInBytes;
58466 |     ma_uint32 readOffsetLoopFlag;
58467 |     ma_uint32 writeOffset;
58468 |     ma_uint32 writeOffsetInBytes;
58469 |     ma_uint32 writeOffsetLoopFlag;
58470 |
58471 |     if (pRB == NULL) {
58472 |         return 0;
58473 |     }
58474 |
58475 |     readOffset = ma_atomic_load_32(&pRB->encodedReadOffset);
58476 |     ma_rb__deconstruct_offset(readOffset, &readOffsetInBytes, &readOffsetLoopFlag);
58477 |
58478 |     writeOffset = ma_atomic_load_32(&pRB->encodedWriteOffset);
58479 |     ma_rb__deconstruct_offset(writeOffset, &writeOffsetInBytes, &writeOffsetLoopFlag);
58480 |
58481 |     if (readOffsetLoopFlag == writeOffsetLoopFlag) {
58482 |         return writeOffsetInBytes - readOffsetInBytes;
58483 |     } else {
58484 |         return writeOffsetInBytes + (pRB->subbufferSizeInBytes - readOffsetInBytes);
58485 |     }
58486 | }
58487 |
58488 | MA_API ma_uint32 ma_rb_available_read(ma_rb* pRB)
58489 | {
58490 |     ma_int32 dist;
58491 |
58492 |     if (pRB == NULL) {
58493 |         return 0;
58494 |     }
58495 | }

```

```

58496 |     dist = ma_rb_pointer_distance(pRB);
58497 |     if (dist < 0) {
58498 |         return 0;
58499 |     }
58500 |
58501 |     return dist;
58502 | }
58503 |
58504 | MA_API ma_uint32 ma_rb_available_write(ma_rb* pRB)
58505 | {
58506 |     if (pRB == NULL) {
58507 |         return 0;
58508 |     }
58509 |
58510 |     return (ma_uint32)(ma_rb_get_subbuffer_size(pRB) - ma_rb_pointer_distance(pRB));
58511 | }
58512 |
58513 | MA_API size_t ma_rb_get_subbuffer_size(ma_rb* pRB)
58514 | {
58515 |     if (pRB == NULL) {
58516 |         return 0;
58517 |     }
58518 |
58519 |     return pRB->subbufferSizeInBytes;
58520 | }
58521 |
58522 | MA_API size_t ma_rb_get_subbuffer_stride(ma_rb* pRB)
58523 | {
58524 |     if (pRB == NULL) {
58525 |         return 0;
58526 |     }
58527 |
58528 |     if (pRB->subbufferStrideInBytes == 0) {
58529 |         return (size_t)pRB->subbufferSizeInBytes;
58530 |     }
58531 |
58532 |     return (size_t)pRB->subbufferStrideInBytes;
58533 | }
58534 |
58535 | MA_API size_t ma_rb_get_subbuffer_offset(ma_rb* pRB, size_t subbufferIndex)
58536 | {
58537 |     if (pRB == NULL) {
58538 |         return 0;
58539 |     }
58540 |
58541 |     return subbufferIndex * ma_rb_get_subbuffer_stride(pRB);
58542 | }
58543 |
58544 | MA_API void* ma_rb_get_subbuffer_ptr(ma_rb* pRB, size_t subbufferIndex, void* pBuffer)
58545 | {
58546 |     if (pRB == NULL) {
58547 |         return NULL;
58548 |     }
58549 |
58550 |     return ma_offset_ptr(pBuffer, ma_rb_get_subbuffer_offset(pRB, subbufferIndex));
58551 | }
58552 |
58553 |
58554 |
58555 | static ma_result ma_pcm_rb_data_source__on_read(ma_data_source* pDataSource, void* pFramesOut,
> ma_uint64 frameCount, ma_uint64* pFramesRead)
58556 | {
58557 |     /* Since there's no notion of an end, we don't ever want to return MA_AT_END here. But it is
> possible to return 0. */
58558 |     ma_pcm_rb* pRB = (ma_pcm_rb*)pDataSource;
58559 |     ma_result result;
58560 |     ma_uint64 totalFramesRead;
58561 |
58562 |     MA_ASSERT(pRB != NULL);
58563 |
58564 |     /* We need to run this in a loop since the ring buffer itself may loop. */
58565 |     totalFramesRead = 0;
58566 |     while (totalFramesRead < frameCount) {
58567 |         void* pMappedBuffer;
58568 |         ma_uint32 mappedFrameCount;
58569 |         ma_uint64 framesToRead = frameCount - totalFramesRead;
58570 |         if (framesToRead > 0xFFFFFFFF) {
58571 |             framesToRead = 0xFFFFFFFF;
58572 |         }

```

```

58573 |
58574 |     mappedFrameCount = (ma_uint32)framesToRead;
58575 |     result = ma_pcm_rb_acquire_read(pRB, &mappedFrameCount, &pMappedBuffer);
58576 |     if (result != MA_SUCCESS) {
58577 |         break;
58578 |     }
58579 |
58580 |     if (mappedFrameCount == 0) {
58581 |         break; /* <-- End of ring buffer. */
58582 |     }
58583 |
58584 |     ma_copy_pcm_frames(ma_offset_pcm_frames_ptr(pFramesOut, totalFramesRead, pRB->format,
> pRB->channels), pMappedBuffer, mappedFrameCount, pRB->format, pRB->channels);
58585 |
58586 |     result = ma_pcm_rb_commit_read(pRB, mappedFrameCount);
58587 |     if (result != MA_SUCCESS) {
58588 |         break;
58589 |     }
58590 |
58591 |     totalFramesRead += mappedFrameCount;
58592 | }
58593 |
58594 | /*
58595 |     There is no notion of an "end" in a ring buffer. If we didn't have enough data to fill the
> requested frame
58596 |     count we'll need to pad with silence. If we don't do this, totalFramesRead might equal 0 which
> will result
58597 |     in the data source layer at a higher level translating this to MA_AT_END which is incorrect for
> a ring buffer.
58598 | */
58599 |     if (totalFramesRead < frameCount) {
58600 |         ma_silence_pcm_frames(ma_offset_pcm_frames_ptr(pFramesOut, totalFramesRead, pRB->format,
> pRB->channels), (frameCount - totalFramesRead), pRB->format, pRB->channels);
58601 |         totalFramesRead = frameCount;
58602 |     }
58603 |
58604 |     *pFramesRead = totalFramesRead;
58605 |     return MA_SUCCESS;
58606 | }
58607 |
58608 | static ma_result ma_pcm_rb_data_source__on_get_data_format(ma_data_source* pDataSource, ma_format*
> pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
58609 | {
58610 |     ma_pcm_rb* pRB = (ma_pcm_rb*)pDataSource;
58611 |     MA_ASSERT(pRB != NULL);
58612 |
58613 |     if (pFormat != NULL) {
58614 |         *pFormat = pRB->format;
58615 |     }
58616 |
58617 |     if (pChannels != NULL) {
58618 |         *pChannels = pRB->channels;
58619 |     }
58620 |
58621 |     if (pSampleRate != NULL) {
58622 |         *pSampleRate = pRB->sampleRate;
58623 |     }
58624 |
58625 |     /* Just assume the default channel map. */
58626 |     if (pChannelMap != NULL) {
58627 |         ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pRB->channels);
58628 |     }
58629 |
58630 |     return MA_SUCCESS;
58631 | }
58632 |
58633 | static ma_data_source_vtable ma_gRBDDataSourceVTable =
58634 | {
58635 |     ma_pcm_rb_data_source__on_read,
58636 |     NULL, /* onSeek */
58637 |     ma_pcm_rb_data_source__on_get_data_format,
58638 |     NULL, /* onGetCursor */
58639 |     NULL, /* onGetLength */
58640 |     NULL, /* onSetLooping */
58641 |     0
58642 | };
58643 |

```

```

58644 | static MA_INLINE ma_uint32 ma_pcm_rb_get_bpf(ma_pcm_rb* pRB)
58645 | {
58646 |     MA_ASSERT(pRB != NULL);
58647 |
58648 |     return ma_get_bytes_per_frame(pRB->format, pRB->channels);
58649 | }
58650 |
58651 | MA_API ma_result ma_pcm_rb_init_ex(ma_format format, ma_uint32 channels, ma_uint32
> subbufferSizeInFrames, ma_uint32 subbufferCount, ma_uint32 subbufferStrideInFrames, void*
> pOptionalPreallocatedBuffer, const ma_allocation_callbacks* pAllocationCallbacks, ma_pcm_rb* pRB)
58652 | {
58653 |     ma_uint32 bpf;
58654 |     ma_result result;
58655 |
58656 |     if (pRB == NULL) {
58657 |         return MA_INVALID_ARGS;
58658 |     }
58659 |
58660 |     MA_ZERO_OBJECT(pRB);
58661 |
58662 |     bpf = ma_get_bytes_per_frame(format, channels);
58663 |     if (bpf == 0) {
58664 |         return MA_INVALID_ARGS;
58665 |     }
58666 |
58667 |     result = ma_rb_init_ex(subbufferSizeInFrames*bpf, subbufferCount, subbufferStrideInFrames*bpf,
> pOptionalPreallocatedBuffer, pAllocationCallbacks, &pRB->rb);
58668 |     if (result != MA_SUCCESS) {
58669 |         return result;
58670 |     }
58671 |
58672 |     pRB->format      = format;
58673 |     pRB->channels     = channels;
58674 |     pRB->sampleRate = 0; /* The sample rate is not passed in as a parameter. */
58675 |
58676 |     /* The PCM ring buffer is a data source. We need to get that set up as well. */
58677 |     {
58678 |         ma_data_source_config dataSourceConfig = ma_data_source_config_init();
58679 |         dataSourceConfig.vtable = &ma_gRbDataSourceVTable;
58680 |
58681 |         result = ma_data_source_init(&dataSourceConfig, &pRB->ds);
58682 |         if (result != MA_SUCCESS) {
58683 |             ma_rb_uninit(&pRB->rb);
58684 |             return result;
58685 |         }
58686 |     }
58687 |
58688 |     return MA_SUCCESS;
58689 | }
58690 |
58691 | MA_API ma_result ma_pcm_rb_init(ma_format format, ma_uint32 channels, ma_uint32 bufferSizeInFrames,
> void* pOptionalPreallocatedBuffer, const ma_allocation_callbacks* pAllocationCallbacks, ma_pcm_rb*
> pRB)
58692 | {
58693 |     return ma_pcm_rb_init_ex(format, channels, bufferSizeInFrames, 1, 0,
> pOptionalPreallocatedBuffer, pAllocationCallbacks, pRB);
58694 | }
58695 |
58696 | MA_API void ma_pcm_rb_uninit(ma_pcm_rb* pRB)
58697 | {
58698 |     if (pRB == NULL) {
58699 |         return;
58700 |     }
58701 |
58702 |     ma_data_source_uninit(&pRB->ds);
58703 |     ma_rb_uninit(&pRB->rb);
58704 | }
58705 |
58706 | MA_API void ma_pcm_rb_reset(ma_pcm_rb* pRB)
58707 | {
58708 |     if (pRB == NULL) {
58709 |         return;
58710 |     }
58711 |
58712 |     ma_rb_reset(&pRB->rb);
58713 | }
58714 |
58715 | MA_API ma_result ma_pcm_rb_acquire_read(ma_pcm_rb* pRB, ma_uint32* pSizeInFrames, void**
> ppBufferOut)

```

```

58716 | {
58717 |     size_t sizeInBytes;
58718 |     ma_result result;
58719 |
58720 |     if (pRB == NULL || pSizeInFrames == NULL) {
58721 |         return MA_INVALID_ARGS;
58722 |     }
58723 |
58724 |     sizeInBytes = *pSizeInFrames * ma_pcm_rb_get_bpf(pRB);
58725 |
58726 |     result = ma_rb_acquire_read(&pRB->rb, &sizeInBytes, ppBufferOut);
58727 |     if (result != MA_SUCCESS) {
58728 |         return result;
58729 |     }
58730 |
58731 |     *pSizeInFrames = (ma_uint32)(sizeInBytes / (size_t)ma_pcm_rb_get_bpf(pRB));
58732 |     return MA_SUCCESS;
58733 | }
58734 |
58735 | MA_API ma_result ma_pcm_rb_commit_read(ma_pcm_rb* pRB, ma_uint32 sizeInFrames)
58736 | {
58737 |     if (pRB == NULL) {
58738 |         return MA_INVALID_ARGS;
58739 |     }
58740 |
58741 |     return ma_rb_commit_read(&pRB->rb, sizeInFrames * ma_pcm_rb_get_bpf(pRB));
58742 | }
58743 |
58744 | MA_API ma_result ma_pcm_rb_acquire_write(ma_pcm_rb* pRB, ma_uint32* pSizeInFrames, void**
> ppBufferOut)
58745 | {
58746 |     size_t sizeInBytes;
58747 |     ma_result result;
58748 |
58749 |     if (pRB == NULL) {
58750 |         return MA_INVALID_ARGS;
58751 |     }
58752 |
58753 |     sizeInBytes = *pSizeInFrames * ma_pcm_rb_get_bpf(pRB);
58754 |
58755 |     result = ma_rb_acquire_write(&pRB->rb, &sizeInBytes, ppBufferOut);
58756 |     if (result != MA_SUCCESS) {
58757 |         return result;
58758 |     }
58759 |
58760 |     *pSizeInFrames = (ma_uint32)(sizeInBytes / ma_pcm_rb_get_bpf(pRB));
58761 |     return MA_SUCCESS;
58762 | }
58763 |
58764 | MA_API ma_result ma_pcm_rb_commit_write(ma_pcm_rb* pRB, ma_uint32 sizeInFrames)
58765 | {
58766 |     if (pRB == NULL) {
58767 |         return MA_INVALID_ARGS;
58768 |     }
58769 |
58770 |     return ma_rb_commit_write(&pRB->rb, sizeInFrames * ma_pcm_rb_get_bpf(pRB));
58771 | }
58772 |
58773 | MA_API ma_result ma_pcm_rb_seek_read(ma_pcm_rb* pRB, ma_uint32 offsetInFrames)
58774 | {
58775 |     if (pRB == NULL) {
58776 |         return MA_INVALID_ARGS;
58777 |     }
58778 |
58779 |     return ma_rb_seek_read(&pRB->rb, offsetInFrames * ma_pcm_rb_get_bpf(pRB));
58780 | }
58781 |
58782 | MA_API ma_result ma_pcm_rb_seek_write(ma_pcm_rb* pRB, ma_uint32 offsetInFrames)
58783 | {
58784 |     if (pRB == NULL) {
58785 |         return MA_INVALID_ARGS;
58786 |     }
58787 |
58788 |     return ma_rb_seek_write(&pRB->rb, offsetInFrames * ma_pcm_rb_get_bpf(pRB));
58789 | }
58790 |
58791 | MA_API ma_int32 ma_pcm_rb_pointer_distance(ma_pcm_rb* pRB)
58792 | {
58793 |     if (pRB == NULL) {

```



```

58794 |         return 0;
58795 |     }
58796 |
58797 |     return ma_rb_pointer_distance(&pRB->rb) / ma_pcm_rb_get_bpf(pRB);
58798 | }
58799 |
58800 | MA_API ma_uint32 ma_pcm_rb_available_read(ma_pcm_rb* pRB)
58801 | {
58802 |     if (pRB == NULL) {
58803 |         return 0;
58804 |     }
58805 |
58806 |     return ma_rb_available_read(&pRB->rb) / ma_pcm_rb_get_bpf(pRB);
58807 | }
58808 |
58809 | MA_API ma_uint32 ma_pcm_rb_available_write(ma_pcm_rb* pRB)
58810 | {
58811 |     if (pRB == NULL) {
58812 |         return 0;
58813 |     }
58814 |
58815 |     return ma_rb_available_write(&pRB->rb) / ma_pcm_rb_get_bpf(pRB);
58816 | }
58817 |
58818 | MA_API ma_uint32 ma_pcm_rb_get_subbuffer_size(ma_pcm_rb* pRB)
58819 | {
58820 |     if (pRB == NULL) {
58821 |         return 0;
58822 |     }
58823 |
58824 |     return (ma_uint32)(ma_rb_get_subbuffer_size(&pRB->rb) / ma_pcm_rb_get_bpf(pRB));
58825 | }
58826 |
58827 | MA_API ma_uint32 ma_pcm_rb_get_subbuffer_stride(ma_pcm_rb* pRB)
58828 | {
58829 |     if (pRB == NULL) {
58830 |         return 0;
58831 |     }
58832 |
58833 |     return (ma_uint32)(ma_rb_get_subbuffer_stride(&pRB->rb) / ma_pcm_rb_get_bpf(pRB));
58834 | }
58835 |
58836 | MA_API ma_uint32 ma_pcm_rb_get_subbuffer_offset(ma_pcm_rb* pRB, ma_uint32 subbufferIndex)
58837 | {
58838 |     if (pRB == NULL) {
58839 |         return 0;
58840 |     }
58841 |
58842 |     return (ma_uint32)(ma_rb_get_subbuffer_offset(&pRB->rb, subbufferIndex) /
> ma_pcm_rb_get_bpf(pRB));
58843 | }
58844 |
58845 | MA_API void* ma_pcm_rb_get_subbuffer_ptr(ma_pcm_rb* pRB, ma_uint32 subbufferIndex, void* pBuffer)
58846 | {
58847 |     if (pRB == NULL) {
58848 |         return NULL;
58849 |     }
58850 |
58851 |     return ma_rb_get_subbuffer_ptr(&pRB->rb, subbufferIndex, pBuffer);
58852 | }
58853 |
58854 | MA_API ma_format ma_pcm_rb_get_format(const ma_pcm_rb* pRB)
58855 | {
58856 |     if (pRB == NULL) {
58857 |         return ma_format_unknown;
58858 |     }
58859 |
58860 |     return pRB->format;
58861 | }
58862 |
58863 | MA_API ma_uint32 ma_pcm_rb_get_channels(const ma_pcm_rb* pRB)
58864 | {
58865 |     if (pRB == NULL) {
58866 |         return 0;
58867 |     }
58868 |
58869 |     return pRB->channels;
58870 | }
58871 |

```

```

58872 | MA_API ma_uint32 ma_pcm_rb_get_sample_rate(const ma_pcm_rb* pRB)
58873 | {
58874 |     if (pRB == NULL) {
58875 |         return 0;
58876 |     }
58877 |
58878 |     return pRB->sampleRate;
58879 | }
58880 |
58881 | MA_API void ma_pcm_rb_set_sample_rate(ma_pcm_rb* pRB, ma_uint32 sampleRate)
58882 | {
58883 |     if (pRB == NULL) {
58884 |         return;
58885 |     }
58886 |
58887 |     pRB->sampleRate = sampleRate;
58888 | }
58889 |
58890 |
58891 |
58892 | MA_API ma_result ma_duplex_rb_init(ma_format captureFormat, ma_uint32 captureChannels, ma_uint32
> sampleRate, ma_uint32 captureInternalSampleRate, ma_uint32 captureInternalPeriodSizeInFrames, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_duplex_rb* pRB)
58893 | {
58894 |     ma_result result;
58895 |     ma_uint32 sizeInFrames;
58896 |
58897 |     sizeInFrames = (ma_uint32)ma_calculate_frame_count_after_resampling(sampleRate,
> captureInternalSampleRate, captureInternalPeriodSizeInFrames * 5);
58898 |     if (sizeInFrames == 0) {
58899 |         return MA_INVALID_ARGS;
58900 |     }
58901 |
58902 |     result = ma_pcm_rb_init(captureFormat, captureChannels, sizeInFrames, NULL,
> pAllocationCallbacks, &pRB->rb);
58903 |     if (result != MA_SUCCESS) {
58904 |         return result;
58905 |     }
58906 |
58907 |     /* Seek forward a bit so we have a bit of a buffer in case of desyncs. */
58908 |     ma_pcm_rb_seek_write((ma_pcm_rb*)pRB, captureInternalPeriodSizeInFrames * 2);
58909 |
58910 |     return MA_SUCCESS;
58911 | }
58912 |
58913 | MA_API ma_result ma_duplex_rb_uninit(ma_duplex_rb* pRB)
58914 | {
58915 |     ma_pcm_rb_uninit((ma_pcm_rb*)pRB);
58916 |     return MA_SUCCESS;
58917 | }
58918 |
58919 |
58920 |
58921 | /*****
> *****/
58922 |
58923 | Miscellaneous Helpers
58924 |
58925 | *****/
> *****/
58926 | MA_API const char* ma_result_description(ma_result result)
58927 | {
58928 |     switch (result)
58929 |     {
58930 |         case MA_SUCCESS:           return "No error";
58931 |         case MA_ERROR:             return "Unknown error";
58932 |         case MA_INVALID_ARGS:      return "Invalid argument";
58933 |         case MA_INVALID_OPERATION: return "Invalid operation";
58934 |         case MA_OUT_OF_MEMORY:     return "Out of memory";
58935 |         case MA_OUT_OF_RANGE:      return "Out of range";
58936 |         case MA_ACCESS_DENIED:     return "Permission denied";
58937 |         case MA_DOES_NOT_EXIST:    return "Resource does not exist";
58938 |         case MA_ALREADY_EXISTS:    return "Resource already exists";
58939 |         case MA_TOO_MANY_OPEN_FILES: return "Too many open files";
58940 |         case MA_INVALID_FILE:      return "Invalid file";
58941 |         case MA_TOO_BIG:           return "Too large";
58942 |         case MA_PATH_TOO_LONG:     return "Path too long";
58943 |         case MA_NAME_TOO_LONG:     return "Name too long";
58944 |         case MA_NOT_DIRECTORY:     return "Not a directory";

```

```

58945 |         case MA_IS_DIRECTORY:           return "Is a directory";
58946 |         case MA_DIRECTORY_NOT_EMPTY:    return "Directory not empty";
58947 |         case MA_AT_END:                 return "At end";
58948 |         case MA_NO_SPACE:               return "No space available";
58949 |         case MA_BUSY:                  return "Device or resource busy";
58950 |         case MA_IO_ERROR:              return "Input/output error";
58951 |         case MA_INTERRUPT:             return "Interrupted";
58952 |         case MA_UNAVAILABLE:           return "Resource unavailable";
58953 |         case MA_ALREADY_IN_USE:        return "Resource already in use";
58954 |         case MA_BAD_ADDRESS:           return "Bad address";
58955 |         case MA_BAD_SEEK:              return "Illegal seek";
58956 |         case MA_BAD_PIPE:              return "Broken pipe";
58957 |         case MA_DEADLOCK:              return "Deadlock";
58958 |         case MA_TOO_MANY_LINKS:        return "Too many links";
58959 |         case MA_NOT_IMPLEMENTED:       return "Not implemented";
58960 |         case MA_NO_MESSAGE:            return "No message of desired type";
58961 |         case MA_BAD_MESSAGE:          return "Invalid message";
58962 |         case MA_NO_DATA_AVAILABLE:     return "No data available";
58963 |         case MA_INVALID_DATA:         return "Invalid data";
58964 |         case MA_TIMEOUT:              return "Timeout";
58965 |         case MA_NO_NETWORK:            return "Network unavailable";
58966 |         case MA_NOT_UNIQUE:            return "Not unique";
58967 |         case MA_NOT_SOCKET:            return "Socket operation on non-socket";
58968 |         case MA_NO_ADDRESS:            return "Destination address required";
58969 |         case MA_BAD_PROTOCOL:         return "Protocol wrong type for socket";
58970 |         case MA_PROTOCOL_UNAVAILABLE:  return "Protocol not available";
58971 |         case MA_PROTOCOL_NOT_SUPPORTED: return "Protocol not supported";
58972 |         case MA_PROTOCOL_FAMILY_NOT_SUPPORTED: return "Protocol family not supported";
58973 |         case MA_ADDRESS_FAMILY_NOT_SUPPORTED: return "Address family not supported";
58974 |         case MA_SOCKET_NOT_SUPPORTED:  return "Socket type not supported";
58975 |         case MA_CONNECTION_RESET:     return "Connection reset";
58976 |         case MA_ALREADY_CONNECTED:     return "Already connected";
58977 |         case MA_NOT_CONNECTED:        return "Not connected";
58978 |         case MA_CONNECTION_REFUSED:    return "Connection refused";
58979 |         case MA_NO_HOST:              return "No host";
58980 |         case MA_IN_PROGRESS:          return "Operation in progress";
58981 |         case MA_CANCELLED:            return "Operation cancelled";
58982 |         case MA_MEMORY_ALREADY_MAPPED: return "Memory already mapped";
58983 |
58984 |         case MA_FORMAT_NOT_SUPPORTED:  return "Format not supported";
58985 |         case MA_DEVICE_TYPE_NOT_SUPPORTED: return "Device type not supported";
58986 |         case MA_SHARE_MODE_NOT_SUPPORTED: return "Share mode not supported";
58987 |         case MA_NO_BACKEND:            return "No backend";
58988 |         case MA_NO_DEVICE:            return "No device";
58989 |         case MA_API_NOT_FOUND:        return "API not found";
58990 |         case MA_INVALID_DEVICE_CONFIG: return "Invalid device config";
58991 |
58992 |         case MA_DEVICE_NOT_INITIALIZED: return "Device not initialized";
58993 |         case MA_DEVICE_NOT_STARTED:    return "Device not started";
58994 |
58995 |         case MA_FAILED_TO_INIT_BACKEND: return "Failed to initialize backend";
58996 |         case MA_FAILED_TO_OPEN_BACKEND_DEVICE: return "Failed to open backend device";
58997 |         case MA_FAILED_TO_START_BACKEND_DEVICE: return "Failed to start backend device";
58998 |         case MA_FAILED_TO_STOP_BACKEND_DEVICE: return "Failed to stop backend device";
58999 |
59000 |         default:                       return "Unknown error";
59001 |     }
59002 | }
59003 |
59004 | MA_API void* ma_malloc(size_t sz, const ma_allocation_callbacks* pAllocationCallbacks)
59005 | {
59006 |     if (pAllocationCallbacks != NULL) {
59007 |         if (pAllocationCallbacks->onMalloc != NULL) {
59008 |             return pAllocationCallbacks->onMalloc(sz, pAllocationCallbacks->pUserData);
59009 |         } else {
59010 |             return NULL; /* Do not fall back to the default implementation. */
59011 |         }
59012 |     } else {
59013 |         return ma__malloc_default(sz, NULL);
59014 |     }
59015 | }
59016 |
59017 | MA_API void* ma_calloc(size_t sz, const ma_allocation_callbacks* pAllocationCallbacks)
59018 | {
59019 |     void* p = ma_malloc(sz, pAllocationCallbacks);
59020 |     if (p != NULL) {
59021 |         MA_ZERO_MEMORY(p, sz);
59022 |     }
59023 | }

```

```

59024 |     return p;
59025 | }
59026 |
59027 | MA_API void* ma_realloc(void* p, size_t sz, const ma_allocation_callbacks* pAllocationCallbacks)
59028 | {
59029 |     if (pAllocationCallbacks != NULL) {
59030 |         if (pAllocationCallbacks->onRealloc != NULL) {
59031 |             return pAllocationCallbacks->onRealloc(p, sz, pAllocationCallbacks->pUserData);
59032 |         } else {
59033 |             return NULL; /* Do not fall back to the default implementation. */
59034 |         }
59035 |     } else {
59036 |         return ma__realloc_default(p, sz, NULL);
59037 |     }
59038 | }
59039 |
59040 | MA_API void ma_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks)
59041 | {
59042 |     if (p == NULL) {
59043 |         return;
59044 |     }
59045 |
59046 |     if (pAllocationCallbacks != NULL) {
59047 |         if (pAllocationCallbacks->onFree != NULL) {
59048 |             pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
59049 |         } else {
59050 |             return; /* Do not fall back to the default implementation. */
59051 |         }
59052 |     } else {
59053 |         ma__free_default(p, NULL);
59054 |     }
59055 | }
59056 |
59057 | MA_API void* ma_aligned_malloc(size_t sz, size_t alignment, const ma_allocation_callbacks*
> pAllocationCallbacks)
59058 | {
59059 |     size_t extraBytes;
59060 |     void* pUnaligned;
59061 |     void* pAligned;
59062 |
59063 |     if (alignment == 0) {
59064 |         return 0;
59065 |     }
59066 |
59067 |     extraBytes = alignment-1 + sizeof(void*);
59068 |
59069 |     pUnaligned = ma_malloc(sz + extraBytes, pAllocationCallbacks);
59070 |     if (pUnaligned == NULL) {
59071 |         return NULL;
59072 |     }
59073 |
59074 |     pAligned = (void*)((ma_uintptr)pUnaligned + extraBytes & ~(ma_uintptr)(alignment-1));
59075 |     ((void**)pAligned)[-1] = pUnaligned;
59076 |
59077 |     return pAligned;
59078 | }
59079 |
59080 | MA_API void ma_aligned_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks)
59081 | {
59082 |     ma_free(((void**)p)[-1], pAllocationCallbacks);
59083 | }
59084 |
59085 | MA_API const char* ma_get_format_name(ma_format format)
59086 | {
59087 |     switch (format)
59088 |     {
59089 |         case ma_format_unknown: return "Unknown";
59090 |         case ma_format_u8:      return "8-bit Unsigned Integer";
59091 |         case ma_format_s16:     return "16-bit Signed Integer";
59092 |         case ma_format_s24:     return "24-bit Signed Integer (Tightly Packed)";
59093 |         case ma_format_s32:     return "32-bit Signed Integer";
59094 |         case ma_format_f32:     return "32-bit IEEE Floating Point";
59095 |         default:                return "Invalid";
59096 |     }
59097 | }
59098 |
59099 | MA_API void ma_blend_f32(float* pOut, float* pInA, float* pInB, float factor, ma_uint32 channels)
59100 | {
59101 |     ma_uint32 i;

```

```

59102 |     for (i = 0; i < channels; ++i) {
59103 |         pOut[i] = ma_mix_f32(pInA[i], pInB[i], factor);
59104 |     }
59105 | }
59106 |
59107 |
59108 | MA_API ma_uint32 ma_get_bytes_per_sample(ma_format format)
59109 | {
59110 |     ma_uint32 sizes[] = {
59111 |         0, /* unknown */
59112 |         1, /* u8 */
59113 |         2, /* s16 */
59114 |         3, /* s24 */
59115 |         4, /* s32 */
59116 |         4, /* f32 */
59117 |     };
59118 |     return sizes[format];
59119 | }
59120 |
59121 |
59122 |
59123 | #define MA_DATA_SOURCE_DEFAULT_RANGE_BEG      0
59124 | #define MA_DATA_SOURCE_DEFAULT_RANGE_END      ~((ma_uint64)0)
59125 | #define MA_DATA_SOURCE_DEFAULT_LOOP_POINT_BEG 0
59126 | #define MA_DATA_SOURCE_DEFAULT_LOOP_POINT_END ~((ma_uint64)0)
59127 |
59128 | MA_API ma_data_source_config ma_data_source_config_init(void)
59129 | {
59130 |     ma_data_source_config config;
59131 |
59132 |     MA_ZERO_OBJECT(&config);
59133 |
59134 |     return config;
59135 | }
59136 |
59137 |
59138 | MA_API ma_result ma_data_source_init(const ma_data_source_config* pConfig, ma_data_source*
> pDataSource)
59139 | {
59140 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59141 |
59142 |     if (pDataSource == NULL) {
59143 |         return MA_INVALID_ARGS;
59144 |     }
59145 |
59146 |     MA_ZERO_OBJECT(pDataSourceBase);
59147 |
59148 |     if (pConfig == NULL) {
59149 |         return MA_INVALID_ARGS;
59150 |     }
59151 |
59152 |     if (pConfig->vtable == NULL) {
59153 |         return MA_INVALID_ARGS;
59154 |     }
59155 |
59156 |     pDataSourceBase->vtable      = pConfig->vtable;
59157 |     pDataSourceBase->rangeBegInFrames = MA_DATA_SOURCE_DEFAULT_RANGE_BEG;
59158 |     pDataSourceBase->rangeEndInFrames = MA_DATA_SOURCE_DEFAULT_RANGE_END;
59159 |     pDataSourceBase->loopBegInFrames  = MA_DATA_SOURCE_DEFAULT_LOOP_POINT_BEG;
59160 |     pDataSourceBase->loopEndInFrames  = MA_DATA_SOURCE_DEFAULT_LOOP_POINT_END;
59161 |     pDataSourceBase->pCurrent          = pDataSource; /* Always read from ourself by default. */
59162 |     pDataSourceBase->pNext             = NULL;
59163 |     pDataSourceBase->onGetNext         = NULL;
59164 |
59165 |     return MA_SUCCESS;
59166 | }
59167 |
59168 | MA_API void ma_data_source_uninit(ma_data_source* pDataSource)
59169 | {
59170 |     if (pDataSource == NULL) {
59171 |         return;
59172 |     }
59173 |
59174 |     /*
59175 |     This is placeholder in case we need this later. Data sources need to call this in their
59176 |     uninitialization routine to ensure things work later on if something is added here.
59177 |     */
59178 | }
59179 |

```

```

59180 | static ma_result ma_data_source_resolve_current(ma_data_source* pDataSource, ma_data_source**
> ppCurrentDataSource)
59181 | {
59182 |     ma_data_source_base* pCurrentDataSource = (ma_data_source_base*)pDataSource;
59183 |
59184 |     MA_ASSERT(pDataSource != NULL);
59185 |     MA_ASSERT(ppCurrentDataSource != NULL);
59186 |
59187 |     if (pCurrentDataSource->pCurrent == NULL) {
59188 |         /*
59189 |         The current data source is NULL. If we're using this in the context of a chain we need to
> return NULL
59190 |         here so that we don't end up looping. Otherwise we just return the data source itself.
59191 |         */
59192 |         if (pCurrentDataSource->pNext != NULL || pCurrentDataSource->onGetNext != NULL) {
59193 |             pCurrentDataSource = NULL;
59194 |         } else {
59195 |             pCurrentDataSource = (ma_data_source_base*)pDataSource; /* Not being used in a chain.
> Make sure we just always read from the data source itself at all times. */
59196 |         }
59197 |     } else {
59198 |         pCurrentDataSource = (ma_data_source_base*)pCurrentDataSource->pCurrent;
59199 |     }
59200 |
59201 |     *ppCurrentDataSource = pCurrentDataSource;
59202 |
59203 |     return MA_SUCCESS;
59204 | }
59205 |
59206 | static ma_result ma_data_source_read_pcm_frames_from_backend(ma_data_source* pDataSource, void*
> pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
59207 | {
59208 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59209 |
59210 |     MA_ASSERT(pDataSourceBase != NULL);
59211 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59212 |     MA_ASSERT(pDataSourceBase->vtable->onRead != NULL);
59213 |     MA_ASSERT(pFramesRead != NULL);
59214 |
59215 |     if (pFramesOut != NULL) {
59216 |         return pDataSourceBase->vtable->onRead(pDataSourceBase, pFramesOut, frameCount,
> pFramesRead);
59217 |     } else {
59218 |         /*
59219 |         No output buffer. Probably seeking forward. Read and discard. Can probably optimize this in
> terms of
59220 |         onSeek and onGetCursor, but need to keep in mind that the data source may not implement
> these functions.
59221 |         */
59222 |         ma_result result;
59223 |         ma_uint64 framesRead;
59224 |         ma_format format;
59225 |         ma_uint32 channels;
59226 |         ma_uint64 discardBufferCapInFrames;
59227 |         ma_uint8 pDiscardBuffer[4096];
59228 |
59229 |         result = ma_data_source_get_data_format(pDataSource, &format, &channels, NULL, NULL, 0);
59230 |         if (result != MA_SUCCESS) {
59231 |             return result;
59232 |         }
59233 |
59234 |         discardBufferCapInFrames = sizeof(pDiscardBuffer) / ma_get_bytes_per_frame(format,
> channels);
59235 |
59236 |         framesRead = 0;
59237 |         while (framesRead < frameCount) {
59238 |             ma_uint64 framesReadThisIteration = 0;
59239 |             ma_uint64 framesToRead = frameCount - framesRead;
59240 |             if (framesToRead > discardBufferCapInFrames) {
59241 |                 framesToRead = discardBufferCapInFrames;
59242 |             }
59243 |
59244 |             result = pDataSourceBase->vtable->onRead(pDataSourceBase, pDiscardBuffer, framesToRead,
> &framesReadThisIteration);
59245 |             if (result != MA_SUCCESS) {
59246 |                 return result;
59247 |             }
59248 |
59249 |             framesRead += framesReadThisIteration;

```

```

59250 |     }
59251 |
59252 |     *pFramesRead = framesRead;
59253 |
59254 |     return MA_SUCCESS;
59255 | }
59256 | }
59257 |
59258 | static ma_result ma_data_source_read_pcm_frames_within_range(ma_data_source* pDataSource, void*
> pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
59259 | {
59260 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59261 |     ma_result result;
59262 |     ma_uint64 framesRead = 0;
59263 |     ma_bool32 loop = ma_data_source_is_looping(pDataSource);
59264 |
59265 |     if (pFramesRead != NULL) {
59266 |         *pFramesRead = 0;
59267 |     }
59268 |
59269 |     if (pDataSourceBase == NULL) {
59270 |         return MA_AT_END;
59271 |     }
59272 |
59273 |     if (frameCount == 0) {
59274 |         return MA_INVALID_ARGS;
59275 |     }
59276 |
59277 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59278 |
59279 |     if ((pDataSourceBase->vtable->flags & MA_DATA_SOURCE_SELF_MANAGED_RANGE_AND_LOOP_POINT) != 0 ||
> (pDataSourceBase->rangeEndInFrames == ~((ma_uint64)0) && (pDataSourceBase->loopEndInFrames ==
> ~((ma_uint64)0) || loop == MA_FALSE))) {
59280 |         /* Either the data source is self-managing the range, or no range is set - just read like
> normal. The data source itself will tell us when the end is reached. */
59281 |         result = ma_data_source_read_pcm_frames_from_backend(pDataSource, pFramesOut, frameCount,
> &framesRead);
59282 |     } else {
59283 |         /* Need to clamp to within the range. */
59284 |         ma_uint64 relativeCursor;
59285 |         ma_uint64 absoluteCursor;
59286 |
59287 |         result = ma_data_source_get_cursor_in_pcm_frames(pDataSourceBase, &relativeCursor);
59288 |         if (result != MA_SUCCESS) {
59289 |             /* Failed to retrieve the cursor. Cannot read within a range or loop points. Just read
> like normal - this may happen for things like noise data sources where it doesn't really matter. */
59290 |             result = ma_data_source_read_pcm_frames_from_backend(pDataSource, pFramesOut,
> frameCount, &framesRead);
59291 |         } else {
59292 |             ma_uint64 rangeBeg;
59293 |             ma_uint64 rangeEnd;
59294 |
59295 |             /* We have the cursor. We need to make sure we don't read beyond our range. */
59296 |             rangeBeg = pDataSourceBase->rangeBegInFrames;
59297 |             rangeEnd = pDataSourceBase->rangeEndInFrames;
59298 |
59299 |             absoluteCursor = rangeBeg + relativeCursor;
59300 |
59301 |             /* If looping, make sure we're within range. */
59302 |             if (loop) {
59303 |                 if (pDataSourceBase->loopEndInFrames != ~((ma_uint64)0)) {
59304 |                     rangeEnd = ma_min(rangeEnd, pDataSourceBase->rangeBegInFrames +
> pDataSourceBase->loopEndInFrames);
59305 |                 }
59306 |             }
59307 |
59308 |             if (frameCount > (rangeEnd - absoluteCursor) && rangeEnd != ~((ma_uint64)0)) {
59309 |                 frameCount = (rangeEnd - absoluteCursor);
59310 |             }
59311 |
59312 |             /*
59313 |             If the cursor is sitting on the end of the range the frame count will be set to 0 which
> can
59314 |             result in MA_INVALID_ARGS. In this case, we don't want to try reading, but instead
> return
59315 |             MA_AT_END so the higher level function can know about it.
59316 |             */
59317 |             if (frameCount > 0) {
59318 |                 result = ma_data_source_read_pcm_frames_from_backend(pDataSource, pFramesOut,

```



```

    > frameCount, &framesRead);
59319 |         } else {
59320 |             result = MA_AT_END; /* The cursor is sitting on the end of the range which means
    > we're at the end. */
59321 |         }
59322 |     }
59323 | }
59324 |
59325 | if (pFramesRead != NULL) {
59326 |     *pFramesRead = framesRead;
59327 | }
59328 |
59329 | /* We need to make sure MA_AT_END is returned if we hit the end of the range. */
59330 | if (result == MA_SUCCESS && framesRead == 0) {
59331 |     result = MA_AT_END;
59332 | }
59333 |
59334 | return result;
59335 | }
59336 |
59337 | MA_API ma_result ma_data_source_read_pcm_frames(ma_data_source* pDataSource, void* pFramesOut,
    > ma_uint64 frameCount, ma_uint64* pFramesRead)
59338 | {
59339 |     ma_result result = MA_SUCCESS;
59340 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59341 |     ma_data_source_base* pCurrentDataSource;
59342 |     void* pRunningFramesOut = pFramesOut;
59343 |     ma_uint64 totalFramesProcessed = 0;
59344 |     ma_format format;
59345 |     ma_uint32 channels;
59346 |     ma_uint32 emptyLoopCounter = 0; /* Keeps track of how many times 0 frames have been read. For
    > infinite loop detection of sounds with no audio data. */
59347 |     ma_bool32 loop;
59348 |
59349 |     if (pFramesRead != NULL) {
59350 |         *pFramesRead = 0;
59351 |     }
59352 |
59353 |     if (frameCount == 0) {
59354 |         return MA_INVALID_ARGS;
59355 |     }
59356 |
59357 |     if (pDataSourceBase == NULL) {
59358 |         return MA_INVALID_ARGS;
59359 |     }
59360 |
59361 |     loop = ma_data_source_is_looping(pDataSource);
59362 |
59363 |     /*
59364 |     We need to know the data format so we can advance the output buffer as we read frames. If this
59365 |     fails, chaining will not work and we'll just read as much as we can from the current source.
59366 |     */
59367 |     if (ma_data_source_get_data_format(pDataSource, &format, &channels, NULL, NULL, 0) !=
    > MA_SUCCESS) {
59368 |         result = ma_data_source_resolve_current(pDataSource, (ma_data_source**)&pCurrentDataSource);
59369 |         if (result != MA_SUCCESS) {
59370 |             return result;
59371 |         }
59372 |
59373 |         return ma_data_source_read_pcm_frames_within_range(pCurrentDataSource, pFramesOut,
    > frameCount, pFramesRead);
59374 |     }
59375 |
59376 |     /*
59377 |     Looping is a bit of a special case. When the `loop` argument is true, chaining will not work and
59378 |     only the current data source will be read from.
59379 |     */
59380 |
59381 |     /* Keep reading until we've read as many frames as possible. */
59382 |     while (totalFramesProcessed < frameCount) {
59383 |         ma_uint64 framesProcessed;
59384 |         ma_uint64 framesRemaining = frameCount - totalFramesProcessed;
59385 |
59386 |         /* We need to resolve the data source that we'll actually be reading from. */
59387 |         result = ma_data_source_resolve_current(pDataSource, (ma_data_source**)&pCurrentDataSource);
59388 |         if (result != MA_SUCCESS) {
59389 |             break;
59390 |         }
59391 |     }

```

```

59392 |         if (pCurrentDataSource == NULL) {
59393 |             break;
59394 |         }
59395 |
59396 |         result = ma_data_source_read_pcm_frames_within_range(pCurrentDataSource, pRunningFramesOut,
> framesRemaining, &framesProcessed);
59397 |         totalFramesProcessed += framesProcessed;
59398 |
59399 |         /*
59400 |         If we encountered an error from the read callback, make sure it's propagated to the caller.
> The caller may need to know whether or not MA_BUSY is returned which is
59401 |         not necessarily considered an error.
59402 |         */
59403 |         if (result != MA_SUCCESS && result != MA_AT_END) {
59404 |             break;
59405 |         }
59406 |
59407 |         /*
59408 |         We can determine if we've reached the end by checking if
> ma_data_source_read_pcm_frames_within_range() returned
59409 |         MA_AT_END. To loop back to the start, all we need to do is seek back to the first frame.
59410 |         */
59411 |         if (result == MA_AT_END) {
59412 |             /*
59413 |             The result needs to be reset back to MA_SUCCESS (from MA_AT_END) so that we don't
59414 |             accidentally return MA_AT_END when data has been read in prior loop iterations. at the
59415 |             end of this function, the result will be checked for MA_SUCCESS, and if the total
59416 |             number of frames processed is 0, will be explicitly set to MA_AT_END.
59417 |             */
59418 |             result = MA_SUCCESS;
59419 |
59420 |             /*
59421 |             We reached the end. If we're looping, we just loop back to the start of the current
59422 |             data source. If we're not looping we need to check if we have another in the chain, and
59423 |             if so, switch to it.
59424 |             */
59425 |             if (loop) {
59426 |                 if (framesProcessed == 0) {
59427 |                     emptyLoopCounter += 1;
59428 |                     if (emptyLoopCounter > 1) {
59429 |                         break; /* Infinite loop detected. Get out. */
59430 |                     }
59431 |                 } else {
59432 |                     emptyLoopCounter = 0;
59433 |                 }
59434 |
59435 |                 result = ma_data_source_seek_to_pcm_frame(pCurrentDataSource,
> pCurrentDataSource->loopBeginFrames);
59436 |                 if (result != MA_SUCCESS) {
59437 |                     break; /* Failed to loop. Abort. */
59438 |                 }
59439 |
59440 |                 /* Don't return MA_AT_END for looping sounds. */
59441 |                 result = MA_SUCCESS;
59442 |             } else {
59443 |                 if (pCurrentDataSource->pNext != NULL) {
59444 |                     pDataSourceBase->pCurrent = pCurrentDataSource->pNext;
59445 |                 } else if (pCurrentDataSource->onGetNext != NULL) {
59446 |                     pDataSourceBase->pCurrent = pCurrentDataSource->onGetNext(pCurrentDataSource);
59447 |                     if (pDataSourceBase->pCurrent == NULL) {
59448 |                         break; /* Our callback did not return a next data source. We're done. */
59449 |                     }
59450 |                 } else {
59451 |                     /* Reached the end of the chain. We're done. */
59452 |                     break;
59453 |                 }
59454 |
59455 |                 /* The next data source needs to be rewound to ensure data is read in looping
> scenarios. */
59456 |                 result = ma_data_source_seek_to_pcm_frame(pDataSourceBase->pCurrent, 0);
59457 |                 if (result != MA_SUCCESS) {
59458 |                     break;
59459 |                 }
59460 |             }
59461 |         }
59462 |
59463 |         if (pRunningFramesOut != NULL) {
59464 |             pRunningFramesOut = ma_offset_ptr(pRunningFramesOut, framesProcessed *
> ma_get_bytes_per_frame(format, channels));

```

```

59465 |     }
59466 | }
59467 |
59468 | if (pFramesRead != NULL) {
59469 |     *pFramesRead = totalFramesProcessed;
59470 | }
59471 |
59472 | MA_ASSERT(!(result == MA_AT_END && totalFramesProcessed > 0)); /* We should never be returning
> MA_AT_END if we read some data. */
59473 |
59474 | if (result == MA_SUCCESS && totalFramesProcessed == 0) {
59475 |     result = MA_AT_END;
59476 | }
59477 |
59478 | return result;
59479 | }
59480 |
59481 | MA_API ma_result ma_data_source_seek_pcm_frames(ma_data_source* pDataSource, ma_uint64 frameCount,
> ma_uint64* pFramesSearched)
59482 | {
59483 |     return ma_data_source_read_pcm_frames(pDataSource, NULL, frameCount, pFramesSearched);
59484 | }
59485 |
59486 | MA_API ma_result ma_data_source_seek_to_pcm_frame(ma_data_source* pDataSource, ma_uint64 frameIndex)
59487 | {
59488 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59489 |
59490 |     if (pDataSourceBase == NULL) {
59491 |         return MA_INVALID_ARGS;
59492 |     }
59493 |
59494 |     if (pDataSourceBase->vtable->onSeek == NULL) {
59495 |         return MA_NOT_IMPLEMENTED;
59496 |     }
59497 |
59498 |     if (frameIndex > pDataSourceBase->rangeEndInFrames) {
59499 |         return MA_INVALID_OPERATION; /* Trying to seek too far forward. */
59500 |     }
59501 |
59502 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59503 |
59504 |     return pDataSourceBase->vtable->onSeek(pDataSource, pDataSourceBase->rangeBegInFrames +
> frameIndex);
59505 | }
59506 |
59507 | MA_API ma_result ma_data_source_seek_seconds(ma_data_source* pDataSource, float secondCount, float*
> pSecondsSearched)
59508 | {
59509 |     ma_uint64 frameCount;
59510 |     ma_uint64 framesSearched = 0;
59511 |     ma_uint32 sampleRate;
59512 |     ma_result result;
59513 |
59514 |     if (pDataSource == NULL) {
59515 |         return MA_INVALID_ARGS;
59516 |     }
59517 |
59518 |     result = ma_data_source_get_data_format(pDataSource, NULL, NULL, &sampleRate, NULL, 0);
59519 |     if (result != MA_SUCCESS) {
59520 |         return result;
59521 |     }
59522 |
59523 |     /* We need PCM frames instead of seconds */
59524 |     frameCount = (ma_uint64)(secondCount * sampleRate);
59525 |
59526 |     result = ma_data_source_seek_pcm_frames(pDataSource, frameCount, &framesSearched);
59527 |
59528 |     /* VC6 doesn't support division between unsigned 64-bit integer and floating point number.
> Signed integer needed. This shouldn't affect anything in practice */
59529 |     *pSecondsSearched = (ma_int64)framesSearched / (float)sampleRate;
59530 |     return result;
59531 | }
59532 |
59533 | MA_API ma_result ma_data_source_seek_to_second(ma_data_source* pDataSource, float
> seekPointInSeconds)
59534 | {
59535 |     ma_uint64 frameIndex;
59536 |     ma_uint32 sampleRate;
59537 |     ma_result result;

```

```

59538 |
59539 |     if (pDataSource == NULL) {
59540 |         return MA_INVALID_ARGS;
59541 |     }
59542 |
59543 |     result = ma_data_source_get_data_format(pDataSource, NULL, NULL, &sampleRate, NULL, 0);
59544 |     if (result != MA_SUCCESS) {
59545 |         return result;
59546 |     }
59547 |
59548 |     /* We need PCM frames instead of seconds */
59549 |     frameIndex = (ma_uint64)(seekPointInSeconds * sampleRate);
59550 |
59551 |     return ma_data_source_seek_to_pcm_frame(pDataSource, frameIndex);
59552 | }
59553 |
59554 | MA_API ma_result ma_data_source_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
59555 | {
59556 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59557 |     ma_result result;
59558 |     ma_format format;
59559 |     ma_uint32 channels;
59560 |     ma_uint32 sampleRate;
59561 |
59562 |     /* Initialize to defaults for safety just in case the data source does not implement this
> callback. */
59563 |     if (pFormat != NULL) {
59564 |         *pFormat = ma_format_unknown;
59565 |     }
59566 |     if (pChannels != NULL) {
59567 |         *pChannels = 0;
59568 |     }
59569 |     if (pSampleRate != NULL) {
59570 |         *pSampleRate = 0;
59571 |     }
59572 |     if (pChannelMap != NULL) {
59573 |         MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channelMapCap);
59574 |     }
59575 |
59576 |     if (pDataSourceBase == NULL) {
59577 |         return MA_INVALID_ARGS;
59578 |     }
59579 |
59580 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59581 |
59582 |     if (pDataSourceBase->vtable->onGetDataFormat == NULL) {
59583 |         return MA_NOT_IMPLEMENTED;
59584 |     }
59585 |
59586 |     result = pDataSourceBase->vtable->onGetDataFormat(pDataSource, &format, &channels, &sampleRate,
> pChannelMap, channelMapCap);
59587 |     if (result != MA_SUCCESS) {
59588 |         return result;
59589 |     }
59590 |
59591 |     if (pFormat != NULL) {
59592 |         *pFormat = format;
59593 |     }
59594 |     if (pChannels != NULL) {
59595 |         *pChannels = channels;
59596 |     }
59597 |     if (pSampleRate != NULL) {
59598 |         *pSampleRate = sampleRate;
59599 |     }
59600 |
59601 |     /* Channel map was passed in directly to the callback. This is safe due to the channelMapCap
> parameter. */
59602 |
59603 |     return MA_SUCCESS;
59604 | }
59605 |
59606 | MA_API ma_result ma_data_source_get_cursor_in_pcm_frames(ma_data_source* pDataSource, ma_uint64*
> pCursor)
59607 | {
59608 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59609 |     ma_result result;
59610 |     ma_uint64 cursor;
59611 |

```

```

59612 |     if (pCursor == NULL) {
59613 |         return MA_INVALID_ARGS;
59614 |     }
59615 |
59616 |     *pCursor = 0;
59617 |
59618 |     if (pDataSourceBase == NULL) {
59619 |         return MA_SUCCESS;
59620 |     }
59621 |
59622 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59623 |
59624 |     if (pDataSourceBase->vtable->onGetCursor == NULL) {
59625 |         return MA_NOT_IMPLEMENTED;
59626 |     }
59627 |
59628 |     result = pDataSourceBase->vtable->onGetCursor(pDataSourceBase, &cursor);
59629 |     if (result != MA_SUCCESS) {
59630 |         return result;
59631 |     }
59632 |
59633 |     /* The cursor needs to be made relative to the start of the range. */
59634 |     if (cursor < pDataSourceBase->rangeBegInFrames) { /* Safety check so we don't return some huge
> number. */
59635 |         *pCursor = 0;
59636 |     } else {
59637 |         *pCursor = cursor - pDataSourceBase->rangeBegInFrames;
59638 |     }
59639 |
59640 |     return MA_SUCCESS;
59641 | }
59642 |
59643 | MA_API ma_result ma_data_source_get_length_in_pcm_frames(ma_data_source* pDataSource, ma_uint64*
> pLength)
59644 | {
59645 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59646 |
59647 |     if (pLength == NULL) {
59648 |         return MA_INVALID_ARGS;
59649 |     }
59650 |
59651 |     *pLength = 0;
59652 |
59653 |     if (pDataSourceBase == NULL) {
59654 |         return MA_INVALID_ARGS;
59655 |     }
59656 |
59657 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59658 |
59659 |     /*
59660 |     If we have a range defined we'll use that to determine the length. This is one of rare times
59661 |     where we'll actually trust the caller. If they've set the range, I think it's mostly safe to
59662 |     assume they've set it based on some higher level knowledge of the structure of the sound bank.
59663 |     */
59664 |     if (pDataSourceBase->rangeEndInFrames != ~((ma_uint64)0)) {
59665 |         *pLength = pDataSourceBase->rangeEndInFrames - pDataSourceBase->rangeBegInFrames;
59666 |         return MA_SUCCESS;
59667 |     }
59668 |
59669 |     /*
59670 |     Getting here means a range is not defined so we'll need to get the data source itself to tell
59671 |     us the length.
59672 |     */
59673 |     if (pDataSourceBase->vtable->onGetLength == NULL) {
59674 |         return MA_NOT_IMPLEMENTED;
59675 |     }
59676 |
59677 |     return pDataSourceBase->vtable->onGetLength(pDataSource, pLength);
59678 | }
59679 |
59680 | MA_API ma_result ma_data_source_get_cursor_in_seconds(ma_data_source* pDataSource, float* pCursor)
59681 | {
59682 |     ma_result result;
59683 |     ma_uint64 cursorInPCMFrames;
59684 |     ma_uint32 sampleRate;
59685 |
59686 |     if (pCursor == NULL) {
59687 |         return MA_INVALID_ARGS;
59688 |     }

```

```

59689 |
59690 |     *pCursor = 0;
59691 |
59692 |     result = ma_data_source_get_cursor_in_pcm_frames(pDataSource, &cursorInPCMFrames);
59693 |     if (result != MA_SUCCESS) {
59694 |         return result;
59695 |     }
59696 |
59697 |     result = ma_data_source_get_data_format(pDataSource, NULL, NULL, &sampleRate, NULL, 0);
59698 |     if (result != MA_SUCCESS) {
59699 |         return result;
59700 |     }
59701 |
59702 |     /* VC6 does not support division of unsigned 64-bit integers with floating point numbers. Need
> to use a signed number. This shouldn't effect anything in practice. */
59703 |     *pCursor = (ma_int64)cursorInPCMFrames / (float)sampleRate;
59704 |
59705 |     return MA_SUCCESS;
59706 | }
59707 |
59708 | MA_API ma_result ma_data_source_get_length_in_seconds(ma_data_source* pDataSource, float* pLength)
59709 | {
59710 |     ma_result result;
59711 |     ma_uint64 lengthInPCMFrames;
59712 |     ma_uint32 sampleRate;
59713 |
59714 |     if (pLength == NULL) {
59715 |         return MA_INVALID_ARGS;
59716 |     }
59717 |
59718 |     *pLength = 0;
59719 |
59720 |     result = ma_data_source_get_length_in_pcm_frames(pDataSource, &lengthInPCMFrames);
59721 |     if (result != MA_SUCCESS) {
59722 |         return result;
59723 |     }
59724 |
59725 |     result = ma_data_source_get_data_format(pDataSource, NULL, NULL, &sampleRate, NULL, 0);
59726 |     if (result != MA_SUCCESS) {
59727 |         return result;
59728 |     }
59729 |
59730 |     /* VC6 does not support division of unsigned 64-bit integers with floating point numbers. Need
> to use a signed number. This shouldn't effect anything in practice. */
59731 |     *pLength = (ma_int64)lengthInPCMFrames / (float)sampleRate;
59732 |
59733 |     return MA_SUCCESS;
59734 | }
59735 |
59736 | MA_API ma_result ma_data_source_set_looping(ma_data_source* pDataSource, ma_bool32 isLooping)
59737 | {
59738 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59739 |
59740 |     if (pDataSource == NULL) {
59741 |         return MA_INVALID_ARGS;
59742 |     }
59743 |
59744 |     ma_atomic_exchange_32(&pDataSourceBase->isLooping, isLooping);
59745 |
59746 |     MA_ASSERT(pDataSourceBase->vtable != NULL);
59747 |
59748 |     /* If there's no callback for this just treat it as a successful no-op. */
59749 |     if (pDataSourceBase->vtable->onSetLooping == NULL) {
59750 |         return MA_SUCCESS;
59751 |     }
59752 |
59753 |     return pDataSourceBase->vtable->onSetLooping(pDataSource, isLooping);
59754 | }
59755 |
59756 | MA_API ma_bool32 ma_data_source_is_looping(const ma_data_source* pDataSource)
59757 | {
59758 |     const ma_data_source_base* pDataSourceBase = (const ma_data_source_base*)pDataSource;
59759 |
59760 |     if (pDataSource == NULL) {
59761 |         return MA_FALSE;
59762 |     }
59763 |
59764 |     return ma_atomic_load_32(&pDataSourceBase->isLooping);
59765 | }

```

```

59766 |
59767 | MA_API ma_result ma_data_source_set_range_in_pcm_frames(ma_data_source* pDataSource, ma_uint64
> rangeBegInFrames, ma_uint64 rangeEndInFrames)
59768 | {
59769 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59770 |     ma_result result;
59771 |     ma_uint64 relativeCursor;
59772 |     ma_uint64 absoluteCursor;
59773 |     ma_bool32 doSeekAdjustment = MA_FALSE;
59774 |
59775 |     if (pDataSource == NULL) {
59776 |         return MA_INVALID_ARGS;
59777 |     }
59778 |
59779 |     if (rangeEndInFrames < rangeBegInFrames) {
59780 |         return MA_INVALID_ARGS; /* The end of the range must come after the beginning. */
59781 |     }
59782 |
59783 |     /*
59784 |     We may need to adjust the position of the cursor to ensure it's clamped to the range. Grab it
> now
59785 |     so we can calculate its absolute position before we change the range.
59786 |     */
59787 |     result = ma_data_source_get_cursor_in_pcm_frames(pDataSource, &relativeCursor);
59788 |     if (result == MA_SUCCESS) {
59789 |         doSeekAdjustment = MA_TRUE;
59790 |         absoluteCursor = relativeCursor + pDataSourceBase->rangeBegInFrames;
59791 |     } else {
59792 |         /*
59793 |         We couldn't get the position of the cursor. It probably means the data source has no notion
59794 |         of a cursor. We'll just leave it at position 0. Don't treat this as an error.
59795 |         */
59796 |         doSeekAdjustment = MA_FALSE;
59797 |         relativeCursor = 0;
59798 |         absoluteCursor = 0;
59799 |     }
59800 |
59801 |     pDataSourceBase->rangeBegInFrames = rangeBegInFrames;
59802 |     pDataSourceBase->rangeEndInFrames = rangeEndInFrames;
59803 |
59804 |     /*
59805 |     The commented out logic below was intended to maintain loop points in response to a change in
> the
59806 |     range. However, this is not useful because it results in the sound breaking when you move the
> range
59807 |     outside of the old loop points. I'm simplifying this by simply resetting the loop points. The
59808 |     caller is expected to update their loop points if they change the range.
59809 |
59810 |     In practice this should be mostly a non-issue because the majority of the time the range will be
59811 |     set once right after initialization.
59812 |     */
59813 |     pDataSourceBase->loopBegInFrames = 0;
59814 |     pDataSourceBase->loopEndInFrames = ~((ma_uint64)0);
59815 |
59816 |     /*
59817 |     Seek to within range. Note that our seek positions here are relative to the new range. We don't
> want
59818 |     to do this if we failed to retrieve the cursor earlier on because it probably means the data
> source
59819 |     has no notion of a cursor. In practice the seek would probably fail (which we silently ignore),
> but
59820 |     I'm just not even going to attempt it.
59821 |     */
59822 |     if (doSeekAdjustment) {
59823 |         if (absoluteCursor < rangeBegInFrames) {
59824 |             ma_data_source_seek_to_pcm_frame(pDataSource, 0);
59825 |         } else if (absoluteCursor > rangeEndInFrames) {
59826 |             ma_data_source_seek_to_pcm_frame(pDataSource, rangeEndInFrames - rangeBegInFrames);
59827 |         }
59828 |     }
59829 | }
59830 |
59831 | return MA_SUCCESS;
59832 | }
59833 |
59834 | MA_API void ma_data_source_get_range_in_pcm_frames(const ma_data_source* pDataSource, ma_uint64*
> pRangeBegInFrames, ma_uint64* pRangeEndInFrames)
59835 | {
59836 |     const ma_data_source_base* pDataSourceBase = (const ma_data_source_base*)pDataSource;

```



```

59837 |
59838 |     if (pRangeBegInFrames != NULL) {
59839 |         *pRangeBegInFrames = 0;
59840 |     }
59841 |     if (pRangeEndInFrames != NULL) {
59842 |         *pRangeEndInFrames = 0;
59843 |     }
59844 |
59845 |     if (pDataSource == NULL) {
59846 |         return;
59847 |     }
59848 |
59849 |     if (pRangeBegInFrames != NULL) {
59850 |         *pRangeBegInFrames = pDataSourceBase->rangeBegInFrames;
59851 |     }
59852 |
59853 |     if (pRangeEndInFrames != NULL) {
59854 |         *pRangeEndInFrames = pDataSourceBase->rangeEndInFrames;
59855 |     }
59856 | }
59857 |
59858 | MA_API ma_result ma_data_source_set_loop_point_in_pcm_frames(ma_data_source* pDataSource, ma_uint64
> loopBegInFrames, ma_uint64 loopEndInFrames)
59859 | {
59860 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59861 |
59862 |     if (pDataSource == NULL) {
59863 |         return MA_INVALID_ARGS;
59864 |     }
59865 |
59866 |     if (loopEndInFrames < loopBegInFrames) {
59867 |         return MA_INVALID_ARGS; /* The end of the loop point must come after the beginning. */
59868 |     }
59869 |
59870 |     if (loopEndInFrames > pDataSourceBase->rangeEndInFrames && loopEndInFrames != ~((ma_uint64)0)) {
59871 |         return MA_INVALID_ARGS; /* The end of the loop point must not go beyond the range. */
59872 |     }
59873 |
59874 |     pDataSourceBase->loopBegInFrames = loopBegInFrames;
59875 |     pDataSourceBase->loopEndInFrames = loopEndInFrames;
59876 |
59877 |     /* The end cannot exceed the range. */
59878 |     if (pDataSourceBase->loopEndInFrames > (pDataSourceBase->rangeEndInFrames -
> pDataSourceBase->rangeBegInFrames) && pDataSourceBase->loopEndInFrames != ~((ma_uint64)0)) {
59879 |         pDataSourceBase->loopEndInFrames = (pDataSourceBase->rangeEndInFrames -
> pDataSourceBase->rangeBegInFrames);
59880 |     }
59881 |
59882 |     return MA_SUCCESS;
59883 | }
59884 |
59885 | MA_API void ma_data_source_get_loop_point_in_pcm_frames(const ma_data_source* pDataSource,
> ma_uint64* pLoopBegInFrames, ma_uint64* pLoopEndInFrames)
59886 | {
59887 |     const ma_data_source_base* pDataSourceBase = (const ma_data_source_base*)pDataSource;
59888 |
59889 |     if (pLoopBegInFrames != NULL) {
59890 |         *pLoopBegInFrames = 0;
59891 |     }
59892 |     if (pLoopEndInFrames != NULL) {
59893 |         *pLoopEndInFrames = 0;
59894 |     }
59895 |
59896 |     if (pDataSource == NULL) {
59897 |         return;
59898 |     }
59899 |
59900 |     if (pLoopBegInFrames != NULL) {
59901 |         *pLoopBegInFrames = pDataSourceBase->loopBegInFrames;
59902 |     }
59903 |
59904 |     if (pLoopEndInFrames != NULL) {
59905 |         *pLoopEndInFrames = pDataSourceBase->loopEndInFrames;
59906 |     }
59907 | }
59908 |
59909 | MA_API ma_result ma_data_source_set_current(ma_data_source* pDataSource, ma_data_source*
> pCurrentDataSource)
59910 | {

```

```

59911 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59912 |
59913 |     if (pDataSource == NULL) {
59914 |         return MA_INVALID_ARGS;
59915 |     }
59916 |
59917 |     pDataSourceBase->pCurrent = pCurrentDataSource;
59918 |
59919 |     return MA_SUCCESS;
59920 | }
59921 |
59922 | MA_API ma_data_source* ma_data_source_get_current(const ma_data_source* pDataSource)
59923 | {
59924 |     const ma_data_source_base* pDataSourceBase = (const ma_data_source_base*)pDataSource;
59925 |
59926 |     if (pDataSource == NULL) {
59927 |         return NULL;
59928 |     }
59929 |
59930 |     return pDataSourceBase->pCurrent;
59931 | }
59932 |
59933 | MA_API ma_result ma_data_source_set_next(ma_data_source* pDataSource, ma_data_source*
> pNextDataSource)
59934 | {
59935 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59936 |
59937 |     if (pDataSource == NULL) {
59938 |         return MA_INVALID_ARGS;
59939 |     }
59940 |
59941 |     pDataSourceBase->pNext = pNextDataSource;
59942 |
59943 |     return MA_SUCCESS;
59944 | }
59945 |
59946 | MA_API ma_data_source* ma_data_source_get_next(const ma_data_source* pDataSource)
59947 | {
59948 |     const ma_data_source_base* pDataSourceBase = (const ma_data_source_base*)pDataSource;
59949 |
59950 |     if (pDataSource == NULL) {
59951 |         return NULL;
59952 |     }
59953 |
59954 |     return pDataSourceBase->pNext;
59955 | }
59956 |
59957 | MA_API ma_result ma_data_source_set_next_callback(ma_data_source* pDataSource,
> ma_data_source_get_next_proc onGetNext)
59958 | {
59959 |     ma_data_source_base* pDataSourceBase = (ma_data_source_base*)pDataSource;
59960 |
59961 |     if (pDataSource == NULL) {
59962 |         return MA_INVALID_ARGS;
59963 |     }
59964 |
59965 |     pDataSourceBase->onGetNext = onGetNext;
59966 |
59967 |     return MA_SUCCESS;
59968 | }
59969 |
59970 | MA_API ma_data_source_get_next_proc ma_data_source_get_next_callback(const ma_data_source*
> pDataSource)
59971 | {
59972 |     const ma_data_source_base* pDataSourceBase = (const ma_data_source_base*)pDataSource;
59973 |
59974 |     if (pDataSource == NULL) {
59975 |         return NULL;
59976 |     }
59977 |
59978 |     return pDataSourceBase->onGetNext;
59979 | }
59980 |
59981 |
59982 | static ma_result ma_audio_buffer_ref_data_source_on_read(ma_data_source* pDataSource, void*
> pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
59983 | {
59984 |     ma_audio_buffer_ref* pAudioBufferRef = (ma_audio_buffer_ref*)pDataSource;
59985 |     ma_uint64 framesRead = ma_audio_buffer_ref_read_pcm_frames(pAudioBufferRef, pFramesOut,

```

```

    > frameCount, MA_FALSE);
59986 |
59987 |     if (pFramesRead != NULL) {
59988 |         *pFramesRead = framesRead;
59989 |     }
59990 |
59991 |     if (framesRead < frameCount || framesRead == 0) {
59992 |         return MA_AT_END;
59993 |     }
59994 |
59995 |     return MA_SUCCESS;
59996 | }
59997 |
59998 | static ma_result ma_audio_buffer_ref__data_source_on_seek(ma_data_source* pDataSource, ma_uint64
> frameIndex)
59999 | {
60000 |     return ma_audio_buffer_ref_seek_to_pcm_frame((ma_audio_buffer_ref*)pDataSource, frameIndex);
60001 | }
60002 |
60003 | static ma_result ma_audio_buffer_ref__data_source_on_get_data_format(ma_data_source* pDataSource,
> ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
60004 | {
60005 |     ma_audio_buffer_ref* pAudioBufferRef = (ma_audio_buffer_ref*)pDataSource;
60006 |
60007 |     *pFormat = pAudioBufferRef->format;
60008 |     *pChannels = pAudioBufferRef->channels;
60009 |     *pSampleRate = pAudioBufferRef->sampleRate;
60010 |     ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pAudioBufferRef->channels);
60011 |
60012 |     return MA_SUCCESS;
60013 | }
60014 |
60015 | static ma_result ma_audio_buffer_ref__data_source_on_get_cursor(ma_data_source* pDataSource,
> ma_uint64* pCursor)
60016 | {
60017 |     ma_audio_buffer_ref* pAudioBufferRef = (ma_audio_buffer_ref*)pDataSource;
60018 |
60019 |     *pCursor = pAudioBufferRef->cursor;
60020 |
60021 |     return MA_SUCCESS;
60022 | }
60023 |
60024 | static ma_result ma_audio_buffer_ref__data_source_on_get_length(ma_data_source* pDataSource,
> ma_uint64* pLength)
60025 | {
60026 |     ma_audio_buffer_ref* pAudioBufferRef = (ma_audio_buffer_ref*)pDataSource;
60027 |
60028 |     *pLength = pAudioBufferRef->sizeInFrames;
60029 |
60030 |     return MA_SUCCESS;
60031 | }
60032 |
60033 | static ma_data_source_vtable g_ma_audio_buffer_ref_data_source_vtable =
60034 | {
60035 |     ma_audio_buffer_ref__data_source_on_read,
60036 |     ma_audio_buffer_ref__data_source_on_seek,
60037 |     ma_audio_buffer_ref__data_source_on_get_data_format,
60038 |     ma_audio_buffer_ref__data_source_on_get_cursor,
60039 |     ma_audio_buffer_ref__data_source_on_get_length,
60040 |     NULL, /* onSetLooping */
60041 |     0
60042 | };
60043 |
60044 | MA_API ma_result ma_audio_buffer_ref_init(ma_format format, ma_uint32 channels, const void* pData,
> ma_uint64 sizeInFrames, ma_audio_buffer_ref* pAudioBufferRef)
60045 | {
60046 |     ma_result result;
60047 |     ma_data_source_config dataSourceConfig;
60048 |
60049 |     if (pAudioBufferRef == NULL) {
60050 |         return MA_INVALID_ARGS;
60051 |     }
60052 |
60053 |     MA_ZERO_OBJECT(pAudioBufferRef);
60054 |
60055 |     dataSourceConfig = ma_data_source_config_init();
60056 |     dataSourceConfig.vtable = &g_ma_audio_buffer_ref_data_source_vtable;

```

```

60057 |
60058 |     result = ma_data_source_init(&dataSourceConfig, &pAudioBufferRef->ds);
60059 |     if (result != MA_SUCCESS) {
60060 |         return result;
60061 |     }
60062 |
60063 |     pAudioBufferRef->format      = format;
60064 |     pAudioBufferRef->channels    = channels;
60065 |     pAudioBufferRef->sampleRate = 0; /* TODO: Version 0.12. Set this to sampleRate. */
60066 |     pAudioBufferRef->cursor      = 0;
60067 |     pAudioBufferRef->sizeInFrames = sizeInFrames;
60068 |     pAudioBufferRef->pData       = pData;
60069 |
60070 |     return MA_SUCCESS;
60071 | }
60072 |
60073 | MA_API void ma_audio_buffer_ref_uninit(ma_audio_buffer_ref* pAudioBufferRef)
60074 | {
60075 |     if (pAudioBufferRef == NULL) {
60076 |         return;
60077 |     }
60078 |
60079 |     ma_data_source_uninit(&pAudioBufferRef->ds);
60080 | }
60081 |
60082 | MA_API ma_result ma_audio_buffer_ref_set_data(ma_audio_buffer_ref* pAudioBufferRef, const void*
> pData, ma_uint64 sizeInFrames)
60083 | {
60084 |     if (pAudioBufferRef == NULL) {
60085 |         return MA_INVALID_ARGS;
60086 |     }
60087 |
60088 |     pAudioBufferRef->cursor      = 0;
60089 |     pAudioBufferRef->sizeInFrames = sizeInFrames;
60090 |     pAudioBufferRef->pData       = pData;
60091 |
60092 |     return MA_SUCCESS;
60093 | }
60094 |
60095 | MA_API ma_uint64 ma_audio_buffer_ref_read_pcm_frames(ma_audio_buffer_ref* pAudioBufferRef, void*
> pFramesOut, ma_uint64 frameCount, ma_bool32 loop)
60096 | {
60097 |     ma_uint64 totalFramesRead = 0;
60098 |
60099 |     if (pAudioBufferRef == NULL) {
60100 |         return 0;
60101 |     }
60102 |
60103 |     if (frameCount == 0) {
60104 |         return 0;
60105 |     }
60106 |
60107 |     while (totalFramesRead < frameCount) {
60108 |         ma_uint64 framesAvailable = pAudioBufferRef->sizeInFrames - pAudioBufferRef->cursor;
60109 |         ma_uint64 framesRemaining = frameCount - totalFramesRead;
60110 |         ma_uint64 framesToRead;
60111 |
60112 |         framesToRead = framesRemaining;
60113 |         if (framesToRead > framesAvailable) {
60114 |             framesToRead = framesAvailable;
60115 |         }
60116 |
60117 |         if (pFramesOut != NULL) {
60118 |             ma_copy_pcm_frames(ma_offset_ptr(pFramesOut, totalFramesRead *
> ma_get_bytes_per_frame(pAudioBufferRef->format, pAudioBufferRef->channels)),
> ma_offset_ptr(pAudioBufferRef->pData, pAudioBufferRef->cursor *
> ma_get_bytes_per_frame(pAudioBufferRef->format, pAudioBufferRef->channels)), framesToRead,
> pAudioBufferRef->format, pAudioBufferRef->channels);
60119 |         }
60120 |
60121 |         totalFramesRead += framesToRead;
60122 |
60123 |         pAudioBufferRef->cursor += framesToRead;
60124 |         if (pAudioBufferRef->cursor == pAudioBufferRef->sizeInFrames) {
60125 |             if (loop) {
60126 |                 pAudioBufferRef->cursor = 0;
60127 |             } else {
60128 |                 break; /* We've reached the end and we're not looping. Done. */
60129 |             }

```

```

60130 |     }
60131 |
60132 |     MA_ASSERT(pAudioBufferRef->cursor < pAudioBufferRef->sizeInFrames);
60133 | }
60134 |
60135 |     return totalFramesRead;
60136 | }
60137 |
60138 | MA_API ma_result ma_audio_buffer_ref_seek_to_pcm_frame(ma_audio_buffer_ref* pAudioBufferRef,
> ma_uint64 frameIndex)
60139 | {
60140 |     if (pAudioBufferRef == NULL) {
60141 |         return MA_INVALID_ARGS;
60142 |     }
60143 |
60144 |     if (frameIndex > pAudioBufferRef->sizeInFrames) {
60145 |         return MA_INVALID_ARGS;
60146 |     }
60147 |
60148 |     pAudioBufferRef->cursor = (size_t)frameIndex;
60149 |
60150 |     return MA_SUCCESS;
60151 | }
60152 |
60153 | MA_API ma_result ma_audio_buffer_ref_map(ma_audio_buffer_ref* pAudioBufferRef, void** ppFramesOut,
> ma_uint64* pFrameCount)
60154 | {
60155 |     ma_uint64 framesAvailable;
60156 |     ma_uint64 frameCount = 0;
60157 |
60158 |     if (ppFramesOut != NULL) {
60159 |         *ppFramesOut = NULL; /* Safety. */
60160 |     }
60161 |
60162 |     if (pFrameCount != NULL) {
60163 |         frameCount = *pFrameCount;
60164 |         *pFrameCount = 0; /* Safety. */
60165 |     }
60166 |
60167 |     if (pAudioBufferRef == NULL || ppFramesOut == NULL || pFrameCount == NULL) {
60168 |         return MA_INVALID_ARGS;
60169 |     }
60170 |
60171 |     framesAvailable = pAudioBufferRef->sizeInFrames - pAudioBufferRef->cursor;
60172 |     if (frameCount > framesAvailable) {
60173 |         frameCount = framesAvailable;
60174 |     }
60175 |
60176 |     *ppFramesOut = ma_offset_ptr(pAudioBufferRef->pData, pAudioBufferRef->cursor *
> ma_get_bytes_per_frame(pAudioBufferRef->format, pAudioBufferRef->channels));
60177 |     *pFrameCount = frameCount;
60178 |
60179 |     return MA_SUCCESS;
60180 | }
60181 |
60182 | MA_API ma_result ma_audio_buffer_ref_unmap(ma_audio_buffer_ref* pAudioBufferRef, ma_uint64
> frameCount)
60183 | {
60184 |     ma_uint64 framesAvailable;
60185 |
60186 |     if (pAudioBufferRef == NULL) {
60187 |         return MA_INVALID_ARGS;
60188 |     }
60189 |
60190 |     framesAvailable = pAudioBufferRef->sizeInFrames - pAudioBufferRef->cursor;
60191 |     if (frameCount > framesAvailable) {
60192 |         return MA_INVALID_ARGS; /* The frame count was too big. This should never happen in an
> unmapping. Need to make sure the caller is aware of this. */
60193 |     }
60194 |
60195 |     pAudioBufferRef->cursor += frameCount;
60196 |
60197 |     if (pAudioBufferRef->cursor == pAudioBufferRef->sizeInFrames) {
60198 |         return MA_AT_END; /* Successful. Need to tell the caller that the end has been reached so
> that it can loop if desired. */
60199 |     } else {
60200 |         return MA_SUCCESS;
60201 |     }
60202 | }

```

```
60203 |
60204 | MA_API ma_bool32 ma_audio_buffer_ref_at_end(const ma_audio_buffer_ref* pAudioBufferRef)
60205 | {
60206 |     if (pAudioBufferRef == NULL) {
60207 |         return MA_FALSE;
60208 |     }
60209 |
60210 |     return pAudioBufferRef->cursor == pAudioBufferRef->sizeInFrames;
60211 | }
60212 |
60213 | MA_API ma_result ma_audio_buffer_ref_get_cursor_in_pcm_frames(const ma_audio_buffer_ref*
> pAudioBufferRef, ma_uint64* pCursor)
60214 | {
60215 |     if (pCursor == NULL) {
60216 |         return MA_INVALID_ARGS;
60217 |     }
60218 |
60219 |     *pCursor = 0;
60220 |
60221 |     if (pAudioBufferRef == NULL) {
60222 |         return MA_INVALID_ARGS;
60223 |     }
60224 |
60225 |     *pCursor = pAudioBufferRef->cursor;
60226 |
60227 |     return MA_SUCCESS;
60228 | }
60229 |
60230 | MA_API ma_result ma_audio_buffer_ref_get_length_in_pcm_frames(const ma_audio_buffer_ref*
> pAudioBufferRef, ma_uint64* pLength)
60231 | {
60232 |     if (pLength == NULL) {
60233 |         return MA_INVALID_ARGS;
60234 |     }
60235 |
60236 |     *pLength = 0;
60237 |
60238 |     if (pAudioBufferRef == NULL) {
60239 |         return MA_INVALID_ARGS;
60240 |     }
60241 |
60242 |     *pLength = pAudioBufferRef->sizeInFrames;
60243 |
60244 |     return MA_SUCCESS;
60245 | }
60246 |
60247 | MA_API ma_result ma_audio_buffer_ref_get_available_frames(const ma_audio_buffer_ref*
> pAudioBufferRef, ma_uint64* pAvailableFrames)
60248 | {
60249 |     if (pAvailableFrames == NULL) {
60250 |         return MA_INVALID_ARGS;
60251 |     }
60252 |
60253 |     *pAvailableFrames = 0;
60254 |
60255 |     if (pAudioBufferRef == NULL) {
60256 |         return MA_INVALID_ARGS;
60257 |     }
60258 |
60259 |     if (pAudioBufferRef->sizeInFrames <= pAudioBufferRef->cursor) {
60260 |         *pAvailableFrames = 0;
60261 |     } else {
60262 |         *pAvailableFrames = pAudioBufferRef->sizeInFrames - pAudioBufferRef->cursor;
60263 |     }
60264 |
60265 |     return MA_SUCCESS;
60266 | }
60267 |
60268 |
60269 |
60270 |
60271 | MA_API ma_audio_buffer_config ma_audio_buffer_config_init(ma_format format, ma_uint32 channels,
> ma_uint64 sizeInFrames, const void* pData, const ma_allocation_callbacks* pAllocationCallbacks)
60272 | {
60273 |     ma_audio_buffer_config config;
60274 |
60275 |     MA_ZERO_OBJECT(&config);
60276 |     config.format = format;
60277 |     config.channels = channels;
```

```

60278 |     config.sampleRate = 0;    /* TODO: Version 0.12. Set this to sampleRate. */
60279 |     config.sizeInFrames = sizeInFrames;
60280 |     config.pData = pData;
60281 |     ma_allocation_callbacks_init_copy(&config.allocationCallbacks, pAllocationCallbacks);
60282 |
60283 |     return config;
60284 | }
60285 |
60286 | static ma_result ma_audio_buffer_init_ex(const ma_audio_buffer_config* pConfig, ma_bool32 doCopy,
> ma_audio_buffer* pAudioBuffer)
60287 | {
60288 |     ma_result result;
60289 |
60290 |     if (pAudioBuffer == NULL) {
60291 |         return MA_INVALID_ARGS;
60292 |     }
60293 |
60294 |     MA_ZERO_MEMORY(pAudioBuffer, sizeof(*pAudioBuffer) - sizeof(pAudioBuffer->pExtraData)); /*
> Safety. Don't overwrite the extra data. */
60295 |
60296 |     if (pConfig == NULL) {
60297 |         return MA_INVALID_ARGS;
60298 |     }
60299 |
60300 |     if (pConfig->sizeInFrames == 0) {
60301 |         return MA_INVALID_ARGS; /* Not allowing buffer sizes of 0 frames. */
60302 |     }
60303 |
60304 |     result = ma_audio_buffer_ref_init(pConfig->format, pConfig->channels, NULL, 0,
> &pAudioBuffer->ref);
60305 |     if (result != MA_SUCCESS) {
60306 |         return result;
60307 |     }
60308 |
60309 |     /* TODO: Version 0.12. Set this in ma_audio_buffer_ref_init() instead of here. */
60310 |     pAudioBuffer->ref.sampleRate = pConfig->sampleRate;
60311 |
60312 |     ma_allocation_callbacks_init_copy(&pAudioBuffer->allocationCallbacks,
> &pConfig->allocationCallbacks);
60313 |
60314 |     if (doCopy) {
60315 |         ma_uint64 allocationSizeInBytes;
60316 |         void* pData;
60317 |
60318 |         allocationSizeInBytes = pConfig->sizeInFrames * ma_get_bytes_per_frame(pConfig->format,
> pConfig->channels);
60319 |         if (allocationSizeInBytes > MA_SIZE_MAX) {
60320 |             return MA_OUT_OF_MEMORY; /* Too big. */
60321 |         }
60322 |
60323 |         pData = ma_malloc((size_t)allocationSizeInBytes, &pAudioBuffer->allocationCallbacks); /*
> Safe cast to size_t. */
60324 |         if (pData == NULL) {
60325 |             return MA_OUT_OF_MEMORY;
60326 |         }
60327 |
60328 |         if (pConfig->pData != NULL) {
60329 |             ma_copy_pcm_frames(pData, pConfig->pData, pConfig->sizeInFrames, pConfig->format,
> pConfig->channels);
60330 |         } else {
60331 |             ma_silence_pcm_frames(pData, pConfig->sizeInFrames, pConfig->format, pConfig->channels);
60332 |         }
60333 |
60334 |         ma_audio_buffer_ref_set_data(&pAudioBuffer->ref, pData, pConfig->sizeInFrames);
60335 |         pAudioBuffer->ownsData = MA_TRUE;
60336 |     } else {
60337 |         ma_audio_buffer_ref_set_data(&pAudioBuffer->ref, pConfig->pData, pConfig->sizeInFrames);
60338 |         pAudioBuffer->ownsData = MA_FALSE;
60339 |     }
60340 |
60341 |     return MA_SUCCESS;
60342 | }
60343 |
60344 | static void ma_audio_buffer_uninit_ex(ma_audio_buffer* pAudioBuffer, ma_bool32 doFree)
60345 | {
60346 |     if (pAudioBuffer == NULL) {
60347 |         return;
60348 |     }
60349 |

```



```

60350 |     if (pAudioBuffer->ownsData && pAudioBuffer->ref.pData != &pAudioBuffer->_pExtraData[0]) {
60351 |         ma_free((void*)pAudioBuffer->ref.pData, &pAudioBuffer->allocationCallbacks);    /* Naughty
> const cast, but OK in this case since we've guarded it with the ownsData check. */
60352 |     }
60353 |
60354 |     if (doFree) {
60355 |         ma_free(pAudioBuffer, &pAudioBuffer->allocationCallbacks);
60356 |     }
60357 |
60358 |     ma_audio_buffer_ref_uninit(&pAudioBuffer->ref);
60359 | }
60360 |
60361 | MA_API ma_result ma_audio_buffer_init(const ma_audio_buffer_config* pConfig, ma_audio_buffer*
> pAudioBuffer)
60362 | {
60363 |     return ma_audio_buffer_init_ex(pConfig, MA_FALSE, pAudioBuffer);
60364 | }
60365 |
60366 | MA_API ma_result ma_audio_buffer_init_copy(const ma_audio_buffer_config* pConfig, ma_audio_buffer*
> pAudioBuffer)
60367 | {
60368 |     return ma_audio_buffer_init_ex(pConfig, MA_TRUE, pAudioBuffer);
60369 | }
60370 |
60371 | MA_API ma_result ma_audio_buffer_alloc_and_init(const ma_audio_buffer_config* pConfig,
> ma_audio_buffer** ppAudioBuffer)
60372 | {
60373 |     ma_result result;
60374 |     ma_audio_buffer* pAudioBuffer;
60375 |     ma_audio_buffer_config innerConfig; /* We'll be making some changes to the config, so need to
> make a copy. */
60376 |     ma_uint64 allocationSizeInBytes;
60377 |
60378 |     if (ppAudioBuffer == NULL) {
60379 |         return MA_INVALID_ARGS;
60380 |     }
60381 |
60382 |     *ppAudioBuffer = NULL; /* Safety. */
60383 |
60384 |     if (pConfig == NULL) {
60385 |         return MA_INVALID_ARGS;
60386 |     }
60387 |
60388 |     innerConfig = *pConfig;
60389 |     ma_allocation_callbacks_init_copy(&innerConfig.allocationCallbacks,
> &pConfig->allocationCallbacks);
60390 |
60391 |     allocationSizeInBytes = sizeof(*pAudioBuffer) - sizeof(pAudioBuffer->_pExtraData) +
> (pConfig->sizeInFrames * ma_get_bytes_per_frame(pConfig->format, pConfig->channels));
60392 |     if (allocationSizeInBytes > MA_SIZE_MAX) {
60393 |         return MA_OUT_OF_MEMORY;    /* Too big. */
60394 |     }
60395 |
60396 |     pAudioBuffer = (ma_audio_buffer*)ma_malloc((size_t)allocationSizeInBytes,
> &innerConfig.allocationCallbacks); /* Safe cast to size_t. */
60397 |     if (pAudioBuffer == NULL) {
60398 |         return MA_OUT_OF_MEMORY;
60399 |     }
60400 |
60401 |     if (pConfig->pData != NULL) {
60402 |         ma_copy_pcm_frames(&pAudioBuffer->_pExtraData[0], pConfig->pData, pConfig->sizeInFrames,
> pConfig->format, pConfig->channels);
60403 |     } else {
60404 |         ma_silence_pcm_frames(&pAudioBuffer->_pExtraData[0], pConfig->sizeInFrames, pConfig->format,
> pConfig->channels);
60405 |     }
60406 |
60407 |     innerConfig.pData = &pAudioBuffer->_pExtraData[0];
60408 |
60409 |     result = ma_audio_buffer_init_ex(&innerConfig, MA_FALSE, pAudioBuffer);
60410 |     if (result != MA_SUCCESS) {
60411 |         ma_free(pAudioBuffer, &innerConfig.allocationCallbacks);
60412 |         return result;
60413 |     }
60414 |
60415 |     *ppAudioBuffer = pAudioBuffer;
60416 |
60417 |     return MA_SUCCESS;
60418 | }

```

```

60419 |
60420 | MA_API void ma_audio_buffer_uninit(ma_audio_buffer* pAudioBuffer)
60421 | {
60422 |     ma_audio_buffer_uninit_ex(pAudioBuffer, MA_FALSE);
60423 | }
60424 |
60425 | MA_API void ma_audio_buffer_uninit_and_free(ma_audio_buffer* pAudioBuffer)
60426 | {
60427 |     ma_audio_buffer_uninit_ex(pAudioBuffer, MA_TRUE);
60428 | }
60429 |
60430 | MA_API ma_uint64 ma_audio_buffer_read_pcm_frames(ma_audio_buffer* pAudioBuffer, void* pFramesOut,
> ma_uint64 frameCount, ma_bool32 loop)
60431 | {
60432 |     if (pAudioBuffer == NULL) {
60433 |         return 0;
60434 |     }
60435 |
60436 |     return ma_audio_buffer_ref_read_pcm_frames(&pAudioBuffer->ref, pFramesOut, frameCount, loop);
60437 | }
60438 |
60439 | MA_API ma_result ma_audio_buffer_seek_to_pcm_frame(ma_audio_buffer* pAudioBuffer, ma_uint64
> frameIndex)
60440 | {
60441 |     if (pAudioBuffer == NULL) {
60442 |         return MA_INVALID_ARGS;
60443 |     }
60444 |
60445 |     return ma_audio_buffer_ref_seek_to_pcm_frame(&pAudioBuffer->ref, frameIndex);
60446 | }
60447 |
60448 | MA_API ma_result ma_audio_buffer_map(ma_audio_buffer* pAudioBuffer, void** ppFramesOut, ma_uint64*
> pFrameCount)
60449 | {
60450 |     if (ppFramesOut != NULL) {
60451 |         *ppFramesOut = NULL;    /* Safety. */
60452 |     }
60453 |
60454 |     if (pAudioBuffer == NULL) {
60455 |         if (pFrameCount != NULL) {
60456 |             *pFrameCount = 0;
60457 |         }
60458 |
60459 |         return MA_INVALID_ARGS;
60460 |     }
60461 |
60462 |     return ma_audio_buffer_ref_map(&pAudioBuffer->ref, ppFramesOut, pFrameCount);
60463 | }
60464 |
60465 | MA_API ma_result ma_audio_buffer_unmap(ma_audio_buffer* pAudioBuffer, ma_uint64 frameCount)
60466 | {
60467 |     if (pAudioBuffer == NULL) {
60468 |         return MA_INVALID_ARGS;
60469 |     }
60470 |
60471 |     return ma_audio_buffer_ref_unmap(&pAudioBuffer->ref, frameCount);
60472 | }
60473 |
60474 | MA_API ma_bool32 ma_audio_buffer_at_end(const ma_audio_buffer* pAudioBuffer)
60475 | {
60476 |     if (pAudioBuffer == NULL) {
60477 |         return MA_FALSE;
60478 |     }
60479 |
60480 |     return ma_audio_buffer_ref_at_end(&pAudioBuffer->ref);
60481 | }
60482 |
60483 | MA_API ma_result ma_audio_buffer_get_cursor_in_pcm_frames(const ma_audio_buffer* pAudioBuffer,
> ma_uint64* pCursor)
60484 | {
60485 |     if (pAudioBuffer == NULL) {
60486 |         return MA_INVALID_ARGS;
60487 |     }
60488 |
60489 |     return ma_audio_buffer_ref_get_cursor_in_pcm_frames(&pAudioBuffer->ref, pCursor);
60490 | }
60491 |
60492 | MA_API ma_result ma_audio_buffer_get_length_in_pcm_frames(const ma_audio_buffer* pAudioBuffer,
> ma_uint64* pLength)

```

```
60493 | {
60494 |     if (pAudioBuffer == NULL) {
60495 |         return MA_INVALID_ARGS;
60496 |     }
60497 |
60498 |     return ma_audio_buffer_ref_get_length_in_pcm_frames(&pAudioBuffer->ref, pLength);
60499 | }
60500 |
60501 | MA_API ma_result ma_audio_buffer_get_available_frames(const ma_audio_buffer* pAudioBuffer,
60502 | > ma_uint64* pAvailableFrames)
60503 | {
60504 |     if (pAvailableFrames == NULL) {
60505 |         return MA_INVALID_ARGS;
60506 |     }
60507 |
60508 |     *pAvailableFrames = 0;
60509 |
60510 |     if (pAudioBuffer == NULL) {
60511 |         return MA_INVALID_ARGS;
60512 |     }
60513 |
60514 |     return ma_audio_buffer_ref_get_available_frames(&pAudioBuffer->ref, pAvailableFrames);
60515 | }
60516 |
60517 |
60518 |
60519 |
60520 | MA_API ma_result ma_paged_audio_buffer_data_init(ma_format format, ma_uint32 channels,
60521 | > ma_paged_audio_buffer_data* pData)
60522 | {
60523 |     if (pData == NULL) {
60524 |         return MA_INVALID_ARGS;
60525 |     }
60526 |
60527 |     MA_ZERO_OBJECT(pData);
60528 |
60529 |     pData->format = format;
60530 |     pData->channels = channels;
60531 |     pData->pTail = &pData->head;
60532 |
60533 |     return MA_SUCCESS;
60534 | }
60535 |
60536 | MA_API void ma_paged_audio_buffer_data_uninit(ma_paged_audio_buffer_data* pData, const
60537 | > ma_allocation_callbacks* pAllocationCallbacks)
60538 | {
60539 |     ma_paged_audio_buffer_page* pPage;
60540 |
60541 |     if (pData == NULL) {
60542 |         return;
60543 |     }
60544 |
60545 |     /* All pages need to be freed. */
60546 |     pPage = (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pData->head.pNext);
60547 |     while (pPage != NULL) {
60548 |         ma_paged_audio_buffer_page* pNext =
60549 | > (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pPage->pNext);
60550 |
60551 |         ma_free(pPage, pAllocationCallbacks);
60552 |         pPage = pNext;
60553 |     }
60554 | }
60555 |
60556 | MA_API ma_paged_audio_buffer_page* ma_paged_audio_buffer_data_get_head(ma_paged_audio_buffer_data*
60557 | > pData)
60558 | {
60559 |     if (pData == NULL) {
60560 |         return NULL;
60561 |     }
60562 |
60563 |     return &pData->head;
60564 | }
60565 |
60566 | MA_API ma_paged_audio_buffer_page* ma_paged_audio_buffer_data_get_tail(ma_paged_audio_buffer_data*
60567 | > pData)
60568 | {
60569 |     if (pData == NULL) {
60570 |         return NULL;
60571 |     }
60572 |
60573 |     return &pData->tail;
```

```

60566 |     }
60567 |
60568 |     return pData->pTail;
60569 | }
60570 |
60571 | MA_API ma_result ma_paged_audio_buffer_data_get_length_in_pcm_frames(ma_paged_audio_buffer_data*
> pData, ma_uint64* pLength)
60572 | {
60573 |     ma_paged_audio_buffer_page* pPage;
60574 |
60575 |     if (pLength == NULL) {
60576 |         return MA_INVALID_ARGS;
60577 |     }
60578 |
60579 |     *pLength = 0;
60580 |
60581 |     if (pData == NULL) {
60582 |         return MA_INVALID_ARGS;
60583 |     }
60584 |
60585 |     /* Calculate the length from the linked list. */
60586 |     for (pPage = (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pData->head.pNext); pPage != NULL;
> pPage = (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pPage->pNext)) {
60587 |         *pLength += pPage->sizeInFrames;
60588 |     }
60589 |
60590 |     return MA_SUCCESS;
60591 | }
60592 |
60593 | MA_API ma_result ma_paged_audio_buffer_data_allocate_page(ma_paged_audio_buffer_data* pData,
> ma_uint64 pageSizeInFrames, const void* pInitialData, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_paged_audio_buffer_page** ppPage)
60594 | {
60595 |     ma_paged_audio_buffer_page* pPage;
60596 |     ma_uint64 allocationSize;
60597 |
60598 |     if (ppPage == NULL) {
60599 |         return MA_INVALID_ARGS;
60600 |     }
60601 |
60602 |     *ppPage = NULL;
60603 |
60604 |     if (pData == NULL) {
60605 |         return MA_INVALID_ARGS;
60606 |     }
60607 |
60608 |     allocationSize = sizeof(*pPage) + (pageSizeInFrames * ma_get_bytes_per_frame(pData->format,
> pData->channels));
60609 |     if (allocationSize > MA_SIZE_MAX) {
60610 |         return MA_OUT_OF_MEMORY;    /* Too big. */
60611 |     }
60612 |
60613 |     pPage = (ma_paged_audio_buffer_page*)ma_malloc((size_t)allocationSize, pAllocationCallbacks);
> /* Safe cast to size_t. */
60614 |     if (pPage == NULL) {
60615 |         return MA_OUT_OF_MEMORY;
60616 |     }
60617 |
60618 |     pPage->pNext = NULL;
60619 |     pPage->sizeInFrames = pageSizeInFrames;
60620 |
60621 |     if (pInitialData != NULL) {
60622 |         ma_copy_pcm_frames(pPage->pAudioData, pInitialData, pageSizeInFrames, pData->format,
> pData->channels);
60623 |     }
60624 |
60625 |     *ppPage = pPage;
60626 |
60627 |     return MA_SUCCESS;
60628 | }
60629 |
60630 | MA_API ma_result ma_paged_audio_buffer_data_free_page(ma_paged_audio_buffer_data* pData,
> ma_paged_audio_buffer_page* pPage, const ma_allocation_callbacks* pAllocationCallbacks)
60631 | {
60632 |     if (pData == NULL || pPage == NULL) {
60633 |         return MA_INVALID_ARGS;
60634 |     }
60635 |
60636 |     /* It's assumed the page is not attached to the list. */

```

```

60637 |     ma_free(pPage, pAllocationCallbacks);
60638 |
60639 |     return MA_SUCCESS;
60640 | }
60641 |
60642 | MA_API ma_result ma_paged_audio_buffer_data_append_page(ma_paged_audio_buffer_data* pData,
> ma_paged_audio_buffer_page* pPage)
60643 | {
60644 |     if (pData == NULL || pPage == NULL) {
60645 |         return MA_INVALID_ARGS;
60646 |     }
60647 |
60648 |     /* This function assumes the page has been filled with audio data by this point. As soon as we
> append, the page will be available for reading. */
60649 |
60650 |     /* First thing to do is update the tail. */
60651 |     for (;;) {
60652 |         ma_paged_audio_buffer_page* pOldTail =
> (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pData->pTail);
60653 |         ma_paged_audio_buffer_page* pNewTail = pPage;
60654 |
60655 |         if (ma_atomic_compare_exchange_weak_ptr((volatile void**)&pData->pTail, (void**)&pOldTail,
> pNewTail)) {
60656 |             /* Here is where we append the page to the list. After this, the page is attached to the
> list and ready to be read from. */
60657 |             ma_atomic_exchange_ptr(&pOldTail->pNext, pPage);
60658 |             break; /* Done. */
60659 |         }
60660 |     }
60661 |
60662 |     return MA_SUCCESS;
60663 | }
60664 |
60665 | MA_API ma_result ma_paged_audio_buffer_data_allocate_and_append_page(ma_paged_audio_buffer_data*
> pData, ma_uint32 pageSizeInFrames, const void* pInitialData, const ma_allocation_callbacks*
> pAllocationCallbacks)
60666 | {
60667 |     ma_result result;
60668 |     ma_paged_audio_buffer_page* pPage;
60669 |
60670 |     result = ma_paged_audio_buffer_data_allocate_page(pData, pageSizeInFrames, pInitialData,
> pAllocationCallbacks, &pPage);
60671 |     if (result != MA_SUCCESS) {
60672 |         return result;
60673 |     }
60674 |
60675 |     return ma_paged_audio_buffer_data_append_page(pData, pPage); /* <-- Should never fail. */
60676 | }
60677 |
60678 |
60679 | MA_API ma_paged_audio_buffer_config ma_paged_audio_buffer_config_init(ma_paged_audio_buffer_data*
> pData)
60680 | {
60681 |     ma_paged_audio_buffer_config config;
60682 |
60683 |     MA_ZERO_OBJECT(&config);
60684 |     config.pData = pData;
60685 |
60686 |     return config;
60687 | }
60688 |
60689 |
60690 | static ma_result ma_paged_audio_buffer__data_source_on_read(ma_data_source* pDataSource, void*
> pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
60691 | {
60692 |     return ma_paged_audio_buffer_read_pcm_frames((ma_paged_audio_buffer*)pDataSource, pFramesOut,
> frameCount, pFramesRead);
60693 | }
60694 |
60695 | static ma_result ma_paged_audio_buffer__data_source_on_seek(ma_data_source* pDataSource, ma_uint64
> frameIndex)
60696 | {
60697 |     return ma_paged_audio_buffer_seek_to_pcm_frame((ma_paged_audio_buffer*)pDataSource, frameIndex);
60698 | }
60699 |
60700 | static ma_result ma_paged_audio_buffer__data_source_on_get_data_format(ma_data_source* pDataSource,
> ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
60701 | {

```

```

60702 |     ma_paged_audio_buffer* pPagedAudioBuffer = (ma_paged_audio_buffer*)pDataSource;
60703 |
60704 |     *pFormat      = pPagedAudioBuffer->pData->format;
60705 |     *pChannels     = pPagedAudioBuffer->pData->channels;
60706 |     *pSampleRate = 0; /* There is no notion of a sample rate with audio buffers. */
60707 |     ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pPagedAudioBuffer->pData->channels);
60708 |
60709 |     return MA_SUCCESS;
60710 | }
60711 |
60712 | static ma_result ma_paged_audio_buffer__data_source_on_get_cursor(ma_data_source* pDataSource,
> ma_uint64* pCursor)
60713 | {
60714 |     return ma_paged_audio_buffer_get_cursor_in_pcm_frames((ma_paged_audio_buffer*)pDataSource,
> pCursor);
60715 | }
60716 |
60717 | static ma_result ma_paged_audio_buffer__data_source_on_get_length(ma_data_source* pDataSource,
> ma_uint64* pLength)
60718 | {
60719 |     return ma_paged_audio_buffer_get_length_in_pcm_frames((ma_paged_audio_buffer*)pDataSource,
> pLength);
60720 | }
60721 |
60722 | static ma_data_source_vtable g_ma_paged_audio_buffer_data_source_vtable =
60723 | {
60724 |     ma_paged_audio_buffer__data_source_on_read,
60725 |     ma_paged_audio_buffer__data_source_on_seek,
60726 |     ma_paged_audio_buffer__data_source_on_get_data_format,
60727 |     ma_paged_audio_buffer__data_source_on_get_cursor,
60728 |     ma_paged_audio_buffer__data_source_on_get_length,
60729 |     NULL, /* onSetLooping */
60730 |     0
60731 | };
60732 |
60733 | MA_API ma_result ma_paged_audio_buffer_init(const ma_paged_audio_buffer_config* pConfig,
> ma_paged_audio_buffer* pPagedAudioBuffer)
60734 | {
60735 |     ma_result result;
60736 |     ma_data_source_config dataSourceConfig;
60737 |
60738 |     if (pPagedAudioBuffer == NULL) {
60739 |         return MA_INVALID_ARGS;
60740 |     }
60741 |
60742 |     MA_ZERO_OBJECT(pPagedAudioBuffer);
60743 |
60744 |     /* A config is required for the format and channel count. */
60745 |     if (pConfig == NULL) {
60746 |         return MA_INVALID_ARGS;
60747 |     }
60748 |
60749 |     if (pConfig->pData == NULL) {
60750 |         return MA_INVALID_ARGS; /* No underlying data specified. */
60751 |     }
60752 |
60753 |     dataSourceConfig = ma_data_source_config_init();
60754 |     dataSourceConfig.vtable = &g_ma_paged_audio_buffer_data_source_vtable;
60755 |
60756 |     result = ma_data_source_init(&dataSourceConfig, &pPagedAudioBuffer->ds);
60757 |     if (result != MA_SUCCESS) {
60758 |         return result;
60759 |     }
60760 |
60761 |     pPagedAudioBuffer->pData      = pConfig->pData;
60762 |     pPagedAudioBuffer->pCurrent    = ma_paged_audio_buffer_data_get_head(pConfig->pData);
60763 |     pPagedAudioBuffer->relativeCursor = 0;
60764 |     pPagedAudioBuffer->absoluteCursor = 0;
60765 |
60766 |     return MA_SUCCESS;
60767 | }
60768 |
60769 | MA_API void ma_paged_audio_buffer_uninit(ma_paged_audio_buffer* pPagedAudioBuffer)
60770 | {
60771 |     if (pPagedAudioBuffer == NULL) {
60772 |         return;
60773 |     }
60774 | }

```

```

60775 |     /* Nothing to do. The data needs to be deleted separately. */
60776 | }
60777 |
60778 | MA_API ma_result ma_paged_audio_buffer_read_pcm_frames(ma_paged_audio_buffer* pPagedAudioBuffer,
> void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
60779 | {
60780 |     ma_result result = MA_SUCCESS;
60781 |     ma_uint64 totalFramesRead = 0;
60782 |     ma_format format;
60783 |     ma_uint32 channels;
60784 |
60785 |     if (pPagedAudioBuffer == NULL) {
60786 |         return MA_INVALID_ARGS;
60787 |     }
60788 |
60789 |     format = pPagedAudioBuffer->pData->format;
60790 |     channels = pPagedAudioBuffer->pData->channels;
60791 |
60792 |     while (totalFramesRead < frameCount) {
60793 |         /* Read from the current page. The buffer should never be in a state where this is NULL. */
60794 |         ma_uint64 framesRemainingInCurrentPage;
60795 |         ma_uint64 framesRemainingToRead = frameCount - totalFramesRead;
60796 |         ma_uint64 framesToReadThisIteration;
60797 |
60798 |         MA_ASSERT(pPagedAudioBuffer->pCurrent != NULL);
60799 |
60800 |         framesRemainingInCurrentPage = pPagedAudioBuffer->pCurrent->sizeInFrames -
> pPagedAudioBuffer->relativeCursor;
60801 |
60802 |         framesToReadThisIteration = ma_min(framesRemainingInCurrentPage, framesRemainingToRead);
60803 |         ma_copy_pcm_frames(ma_offset_pcm_frames_ptr(pFramesOut, totalFramesRead, format, channels),
> ma_offset_pcm_frames_ptr(pPagedAudioBuffer->pCurrent->pAudioData, pPagedAudioBuffer->relativeCursor,
> format, channels), framesToReadThisIteration, format, channels);
60804 |         totalFramesRead += framesToReadThisIteration;
60805 |
60806 |         pPagedAudioBuffer->absoluteCursor += framesToReadThisIteration;
60807 |         pPagedAudioBuffer->relativeCursor += framesToReadThisIteration;
60808 |
60809 |         /* Move to the next page if necessary. If there's no more pages, we need to return
> MA_AT_END. */
60810 |         MA_ASSERT(pPagedAudioBuffer->relativeCursor <= pPagedAudioBuffer->pCurrent->sizeInFrames);
60811 |
60812 |         if (pPagedAudioBuffer->relativeCursor == pPagedAudioBuffer->pCurrent->sizeInFrames) {
60813 |             /* We reached the end of the page. Need to move to the next. If there's no more pages,
> we're done. */
60814 |             ma_paged_audio_buffer_page* pNext =
> (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pPagedAudioBuffer->pCurrent->pNext);
60815 |             if (pNext == NULL) {
60816 |                 result = MA_AT_END;
60817 |                 break; /* We've reached the end. */
60818 |             } else {
60819 |                 pPagedAudioBuffer->pCurrent = pNext;
60820 |                 pPagedAudioBuffer->relativeCursor = 0;
60821 |             }
60822 |         }
60823 |     }
60824 |
60825 |     if (pFramesRead != NULL) {
60826 |         *pFramesRead = totalFramesRead;
60827 |     }
60828 |
60829 |     return result;
60830 | }
60831 |
60832 | MA_API ma_result ma_paged_audio_buffer_seek_to_pcm_frame(ma_paged_audio_buffer* pPagedAudioBuffer,
> ma_uint64 frameIndex)
60833 | {
60834 |     if (pPagedAudioBuffer == NULL) {
60835 |         return MA_INVALID_ARGS;
60836 |     }
60837 |
60838 |     if (frameIndex == pPagedAudioBuffer->absoluteCursor) {
60839 |         return MA_SUCCESS; /* Nothing to do. */
60840 |     }
60841 |
60842 |     if (frameIndex < pPagedAudioBuffer->absoluteCursor) {
60843 |         /* Moving backwards. Need to move the cursor back to the start, and then move forward. */
60844 |         pPagedAudioBuffer->pCurrent =
> ma_paged_audio_buffer_data_get_head(pPagedAudioBuffer->pData);

```



```

60845 |         pPagedAudioBuffer->absoluteCursor = 0;
60846 |         pPagedAudioBuffer->relativeCursor = 0;
60847 |
60848 |         /* Fall through to the forward seeking section below. */
60849 |     }
60850 |
60851 |     if (frameIndex > pPagedAudioBuffer->absoluteCursor) {
60852 |         /* Moving forward. */
60853 |         ma_paged_audio_buffer_page* pPage;
60854 |         ma_uint64 runningCursor = 0;
60855 |
60856 |         for (pPage = (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&ma_paged_audio_buffer_data_get
> _head(pPagedAudioBuffer->pData)->pNext); pPage != NULL; pPage =
> (ma_paged_audio_buffer_page*)ma_atomic_load_ptr(&pPage->pNext)) {
60857 |             ma_uint64 pageRangeBeg = runningCursor;
60858 |             ma_uint64 pageRangeEnd = pageRangeBeg + pPage->sizeInFrames;
60859 |
60860 |             if (frameIndex >= pageRangeBeg) {
60861 |                 if (frameIndex < pageRangeEnd || (frameIndex == pageRangeEnd && pPage == (ma_paged_a
> udio_buffer_page*)ma_atomic_load_ptr(ma_paged_audio_buffer_data_get_tail(pPagedAudioBuffer->pData)))
> ) { /* A small edge case - allow seeking to the very end of the buffer. */
60862 |                     /* We found the page. */
60863 |                     pPagedAudioBuffer->pCurrent = pPage;
60864 |                     pPagedAudioBuffer->absoluteCursor = frameIndex;
60865 |                     pPagedAudioBuffer->relativeCursor = frameIndex - pageRangeBeg;
60866 |                     return MA_SUCCESS;
60867 |                 }
60868 |             }
60869 |
60870 |             runningCursor = pageRangeEnd;
60871 |         }
60872 |
60873 |         /* Getting here means we tried seeking too far forward. Don't change any state. */
60874 |         return MA_BAD_SEEK;
60875 |     }
60876 |
60877 |     return MA_SUCCESS;
60878 | }
60879 |
60880 | MA_API ma_result ma_paged_audio_buffer_get_cursor_in_pcm_frames(ma_paged_audio_buffer*
> pPagedAudioBuffer, ma_uint64* pCursor)
60881 | {
60882 |     if (pCursor == NULL) {
60883 |         return MA_INVALID_ARGS;
60884 |     }
60885 |
60886 |     *pCursor = 0; /* Safety. */
60887 |
60888 |     if (pPagedAudioBuffer == NULL) {
60889 |         return MA_INVALID_ARGS;
60890 |     }
60891 |
60892 |     *pCursor = pPagedAudioBuffer->absoluteCursor;
60893 |
60894 |     return MA_SUCCESS;
60895 | }
60896 |
60897 | MA_API ma_result ma_paged_audio_buffer_get_length_in_pcm_frames(ma_paged_audio_buffer*
> pPagedAudioBuffer, ma_uint64* pLength)
60898 | {
60899 |     return ma_paged_audio_buffer_data_get_length_in_pcm_frames(pPagedAudioBuffer->pData, pLength);
60900 | }
60901 |
60902 |
60903 |
60904 | /*****
> *****/
60905 |
60906 | VFS
60907 |
60908 | *****/
> *****/
60909 | MA_API ma_result ma_vfs_open(ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode, ma_vfs_file*
> pFile)
60910 | {
60911 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
60912 |
60913 |     if (pFile == NULL) {
60914 |         return MA_INVALID_ARGS;

```

```

60915 |     }
60916 |
60917 |     *pFile = NULL;
60918 |
60919 |     if (pVFS == NULL || pFilePath == NULL || openMode == 0) {
60920 |         return MA_INVALID_ARGS;
60921 |     }
60922 |
60923 |     if (pCallbacks->onOpen == NULL) {
60924 |         return MA_NOT_IMPLEMENTED;
60925 |     }
60926 |
60927 |     return pCallbacks->onOpen(pVFS, pFilePath, openMode, pFile);
60928 | }
60929 |
60930 | MA_API ma_result ma_vfs_open_w(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile)
60931 | {
60932 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
60933 |
60934 |     if (pFile == NULL) {
60935 |         return MA_INVALID_ARGS;
60936 |     }
60937 |
60938 |     *pFile = NULL;
60939 |
60940 |     if (pVFS == NULL || pFilePath == NULL || openMode == 0) {
60941 |         return MA_INVALID_ARGS;
60942 |     }
60943 |
60944 |     if (pCallbacks->onOpenW == NULL) {
60945 |         return MA_NOT_IMPLEMENTED;
60946 |     }
60947 |
60948 |     return pCallbacks->onOpenW(pVFS, pFilePath, openMode, pFile);
60949 | }
60950 |
60951 | MA_API ma_result ma_vfs_close(ma_vfs* pVFS, ma_vfs_file file)
60952 | {
60953 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
60954 |
60955 |     if (pVFS == NULL || file == NULL) {
60956 |         return MA_INVALID_ARGS;
60957 |     }
60958 |
60959 |     if (pCallbacks->onClose == NULL) {
60960 |         return MA_NOT_IMPLEMENTED;
60961 |     }
60962 |
60963 |     return pCallbacks->onClose(pVFS, file);
60964 | }
60965 |
60966 | MA_API ma_result ma_vfs_read(ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t sizeInBytes, size_t*
> pBytesRead)
60967 | {
60968 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
60969 |     ma_result result;
60970 |     size_t bytesRead = 0;
60971 |
60972 |     if (pBytesRead != NULL) {
60973 |         *pBytesRead = 0;
60974 |     }
60975 |
60976 |     if (pVFS == NULL || file == NULL || pDst == NULL) {
60977 |         return MA_INVALID_ARGS;
60978 |     }
60979 |
60980 |     if (pCallbacks->onRead == NULL) {
60981 |         return MA_NOT_IMPLEMENTED;
60982 |     }
60983 |
60984 |     result = pCallbacks->onRead(pVFS, file, pDst, sizeInBytes, &bytesRead);
60985 |
60986 |     if (pBytesRead != NULL) {
60987 |         *pBytesRead = bytesRead;
60988 |     }
60989 |
60990 |     if (result == MA_SUCCESS && bytesRead == 0 && sizeInBytes > 0) {
60991 |         result = MA_AT_END;

```

```
60992 |     }
60993 |
60994 |     return result;
60995 | }
60996 |
60997 | MA_API ma_result ma_vfs_write(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc, size_t sizeInBytes,
> size_t* pBytesWritten)
60998 | {
60999 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
61000 |
61001 |     if (pBytesWritten != NULL) {
61002 |         *pBytesWritten = 0;
61003 |     }
61004 |
61005 |     if (pVFS == NULL || file == NULL || pSrc == NULL) {
61006 |         return MA_INVALID_ARGS;
61007 |     }
61008 |
61009 |     if (pCallbacks->onWrite == NULL) {
61010 |         return MA_NOT_IMPLEMENTED;
61011 |     }
61012 |
61013 |     return pCallbacks->onWrite(pVFS, file, pSrc, sizeInBytes, pBytesWritten);
61014 | }
61015 |
61016 | MA_API ma_result ma_vfs_seek(ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset, ma_seek_origin origin)
61017 | {
61018 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
61019 |
61020 |     if (pVFS == NULL || file == NULL) {
61021 |         return MA_INVALID_ARGS;
61022 |     }
61023 |
61024 |     if (pCallbacks->onSeek == NULL) {
61025 |         return MA_NOT_IMPLEMENTED;
61026 |     }
61027 |
61028 |     return pCallbacks->onSeek(pVFS, file, offset, origin);
61029 | }
61030 |
61031 | MA_API ma_result ma_vfs_tell(ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor)
61032 | {
61033 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
61034 |
61035 |     if (pCursor == NULL) {
61036 |         return MA_INVALID_ARGS;
61037 |     }
61038 |
61039 |     *pCursor = 0;
61040 |
61041 |     if (pVFS == NULL || file == NULL) {
61042 |         return MA_INVALID_ARGS;
61043 |     }
61044 |
61045 |     if (pCallbacks->onTell == NULL) {
61046 |         return MA_NOT_IMPLEMENTED;
61047 |     }
61048 |
61049 |     return pCallbacks->onTell(pVFS, file, pCursor);
61050 | }
61051 |
61052 | MA_API ma_result ma_vfs_info(ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo)
61053 | {
61054 |     ma_vfs_callbacks* pCallbacks = (ma_vfs_callbacks*)pVFS;
61055 |
61056 |     if (pInfo == NULL) {
61057 |         return MA_INVALID_ARGS;
61058 |     }
61059 |
61060 |     MA_ZERO_OBJECT(pInfo);
61061 |
61062 |     if (pVFS == NULL || file == NULL) {
61063 |         return MA_INVALID_ARGS;
61064 |     }
61065 |
61066 |     if (pCallbacks->onInfo == NULL) {
61067 |         return MA_NOT_IMPLEMENTED;
61068 |     }
61069 | }
```

```

61070 |     return pCallbacks->onInfo(pVFS, file, pInfo);
61071 | }
61072 |
61073 |
61074 | #if !defined(MA_USE_WIN32_FILEIO) && (defined(MA_WIN32) && (defined(MA_WIN32_DESKTOP) ||
> defined(MA_WIN32_NXDK)) && !defined(MA_NO_WIN32_FILEIO) && !defined(MA_POSIX))
61075 |     #define MA_USE_WIN32_FILEIO
61076 | #endif
61077 |
61078 | #if defined(MA_USE_WIN32_FILEIO)
61079 | /*
61080 | We need to dynamically load SetFilePointer or SetFilePointerEx because older versions of Windows do
61081 | not have the Ex version. We therefore need to do some dynamic branching depending on what's
> available.
61082 |
61083 | We load these when we load our first file from the default VFS. It's left open for the life of the
61084 | program and is left to the OS to uninitialized when the program terminates.
61085 | */
61086 | typedef DWORD (__stdcall * ma_SetFilePointer_proc)(HANDLE hFile, LONG lDistanceToMove, LONG*
> lpDistanceToMoveHigh, DWORD dwMoveMethod);
61087 | typedef BOOL (__stdcall * ma_SetFilePointerEx_proc)(HANDLE hFile, LARGE_INTEGER liDistanceToMove,
> LARGE_INTEGER* lpNewFilePointer, DWORD dwMoveMethod);
61088 |
61089 | static ma_handle hKernel32DLL = NULL;
61090 | static ma_SetFilePointer_proc ma_SetFilePointer = NULL;
61091 | static ma_SetFilePointerEx_proc ma_SetFilePointerEx = NULL;
61092 |
61093 | static void ma_win32_fileio_init(void)
61094 | {
61095 |     if (hKernel32DLL == NULL) {
61096 |         hKernel32DLL = ma_dlopen(NULL, "kernel32.dll");
61097 |         if (hKernel32DLL != NULL) {
61098 |             ma_SetFilePointer = (ma_SetFilePointer_proc) ma_dlsym(NULL, hKernel32DLL,
> "SetFilePointer");
61099 |             ma_SetFilePointerEx = (ma_SetFilePointerEx_proc)ma_dlsym(NULL, hKernel32DLL,
> "SetFilePointerEx");
61100 |         }
61101 |     }
61102 | }
61103 |
61104 | static void ma_default_vfs__get_open_settings_win32(ma_uint32 openMode, DWORD* pDesiredAccess,
> DWORD* pShareMode, DWORD* pCreationDisposition)
61105 | {
61106 |     *pDesiredAccess = 0;
61107 |     if ((openMode & MA_OPEN_MODE_READ) != 0) {
61108 |         *pDesiredAccess |= GENERIC_READ;
61109 |     }
61110 |     if ((openMode & MA_OPEN_MODE_WRITE) != 0) {
61111 |         *pDesiredAccess |= GENERIC_WRITE;
61112 |     }
61113 |
61114 |     *pShareMode = 0;
61115 |     if ((openMode & MA_OPEN_MODE_READ) != 0) {
61116 |         *pShareMode |= FILE_SHARE_READ;
61117 |     }
61118 |
61119 |     if ((openMode & MA_OPEN_MODE_WRITE) != 0) {
61120 |         *pCreationDisposition = CREATE_ALWAYS; /* Opening in write mode. Truncate. */
61121 |     } else {
61122 |         *pCreationDisposition = OPEN_EXISTING; /* Opening in read mode. File must exist. */
61123 |     }
61124 | }
61125 |
61126 | static ma_result ma_default_vfs_open__win32(ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile)
61127 | {
61128 |     HANDLE hFile;
61129 |     DWORD dwDesiredAccess;
61130 |     DWORD dwShareMode;
61131 |     DWORD dwCreationDisposition;
61132 |
61133 |     (void)pVFS;
61134 |
61135 |     /* Load some Win32 symbols dynamically so we can dynamically check for the existence of
> SetFilePointerEx. */
61136 |     ma_win32_fileio_init();
61137 |
61138 |     ma_default_vfs__get_open_settings_win32(openMode, &dwDesiredAccess, &dwShareMode,
> &dwCreationDisposition);

```

```

61139 |
61140 |     hFile = CreateFileA(pFilePath, dwDesiredAccess, dwShareMode, NULL, dwCreationDisposition,
> FILE_ATTRIBUTE_NORMAL, NULL);
61141 |     if (hFile == INVALID_HANDLE_VALUE) {
61142 |         return ma_result_from_GetLastError(GetLastError());
61143 |     }
61144 |
61145 |     *pFile = hFile;
61146 |     return MA_SUCCESS;
61147 | }
61148 |
61149 | static ma_result ma_default_vfs_open_w__win32(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32
> openMode, ma_vfs_file* pFile)
61150 | {
61151 |     #if !defined(MA_XBOX_NXDK)
61152 |     {
61153 |         HANDLE hFile;
61154 |         DWORD dwDesiredAccess;
61155 |         DWORD dwShareMode;
61156 |         DWORD dwCreationDisposition;
61157 |
61158 |         (void)pVFS;
61159 |
61160 |         /* Load some Win32 symbols dynamically so we can dynamically check for the existence of
> SetFilePointerEx. */
61161 |         ma_win32_fileio_init();
61162 |
61163 |         ma_default_vfs__get_open_settings_win32(openMode, &dwDesiredAccess, &dwShareMode,
> &dwCreationDisposition);
61164 |
61165 |         hFile = CreateFileW(pFilePath, dwDesiredAccess, dwShareMode, NULL, dwCreationDisposition,
> FILE_ATTRIBUTE_NORMAL, NULL);
61166 |         if (hFile == INVALID_HANDLE_VALUE) {
61167 |             return ma_result_from_GetLastError(GetLastError());
61168 |         }
61169 |
61170 |         *pFile = hFile;
61171 |         return MA_SUCCESS;
61172 |     }
61173 |     #else
61174 |     {
61175 |         /* No CreateFileW() available. */
61176 |         return MA_NOT_IMPLEMENTED;
61177 |     }
61178 |     #endif
61179 | }
61180 |
61181 | static ma_result ma_default_vfs_close__win32(ma_vfs* pVFS, ma_vfs_file file)
61182 | {
61183 |     (void)pVFS;
61184 |
61185 |     if (CloseHandle((HANDLE)file) == 0) {
61186 |         return ma_result_from_GetLastError(GetLastError());
61187 |     }
61188 |
61189 |     return MA_SUCCESS;
61190 | }
61191 |
61192 |
61193 | static ma_result ma_default_vfs_read__win32(ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t
> sizeInBytes, size_t* pBytesRead)
61194 | {
61195 |     ma_result result = MA_SUCCESS;
61196 |     size_t totalBytesRead;
61197 |
61198 |     (void)pVFS;
61199 |
61200 |     totalBytesRead = 0;
61201 |     while (totalBytesRead < sizeInBytes) {
61202 |         size_t bytesRemaining;
61203 |         DWORD bytesToRead;
61204 |         DWORD bytesRead;
61205 |         BOOL readResult;
61206 |
61207 |         bytesRemaining = sizeInBytes - totalBytesRead;
61208 |         if (bytesRemaining >= 0xFFFFFFFF) {
61209 |             bytesToRead = 0xFFFFFFFF;
61210 |         } else {
61211 |             bytesToRead = (DWORD)bytesRemaining;

```

```

61212 |     }
61213 |
61214 |     readResult = ReadFile((HANDLE)file, ma_offset_ptr(pDst, totalBytesRead), bytesToRead,
> &bytesRead, NULL);
61215 |     if (readResult == 1 && bytesRead == 0) {
61216 |         result = MA_AT_END;
61217 |         break; /* EOF */
61218 |     }
61219 |
61220 |     totalBytesRead += bytesRead;
61221 |
61222 |     if (bytesRead < bytesToRead) {
61223 |         break; /* EOF */
61224 |     }
61225 |
61226 |     if (readResult == 0) {
61227 |         result = ma_result_from_GetLastError(GetLastError());
61228 |         break;
61229 |     }
61230 | }
61231 |
61232 | if (pBytesRead != NULL) {
61233 |     *pBytesRead = totalBytesRead;
61234 | }
61235 |
61236 | return result;
61237 | }
61238 |
61239 | static ma_result ma_default_vfs_write__win32(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc,
> size_t sizeInBytes, size_t* pBytesWritten)
61240 | {
61241 |     ma_result result = MA_SUCCESS;
61242 |     size_t totalBytesWritten;
61243 |
61244 |     (void)pVFS;
61245 |
61246 |     totalBytesWritten = 0;
61247 |     while (totalBytesWritten < sizeInBytes) {
61248 |         size_t bytesRemaining;
61249 |         DWORD bytesToWrite;
61250 |         DWORD bytesWritten;
61251 |         BOOL writeResult;
61252 |
61253 |         bytesRemaining = sizeInBytes - totalBytesWritten;
61254 |         if (bytesRemaining >= 0xFFFFFFFF) {
61255 |             bytesToWrite = 0xFFFFFFFF;
61256 |         } else {
61257 |             bytesToWrite = (DWORD)bytesRemaining;
61258 |         }
61259 |
61260 |         writeResult = WriteFile((HANDLE)file, ma_offset_ptr(pSrc, totalBytesWritten), bytesToWrite,
> &bytesWritten, NULL);
61261 |         totalBytesWritten += bytesWritten;
61262 |
61263 |         if (writeResult == 0) {
61264 |             result = ma_result_from_GetLastError(GetLastError());
61265 |             break;
61266 |         }
61267 |     }
61268 |
61269 |     if (pBytesWritten != NULL) {
61270 |         *pBytesWritten = totalBytesWritten;
61271 |     }
61272 |
61273 |     return result;
61274 | }
61275 |
61276 |
61277 | static ma_result ma_default_vfs_seek__win32(ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset,
> ma_seek_origin origin)
61278 | {
61279 |     LARGE_INTEGER liDistanceToMove;
61280 |     DWORD dwMoveMethod;
61281 |     BOOL result;
61282 |
61283 |     (void)pVFS;
61284 |
61285 |     liDistanceToMove.QuadPart = offset;
61286 |

```

```

61287 |     /* */ if (origin == ma_seek_origin_current) {
61288 |         dwMoveMethod = FILE_CURRENT;
61289 |     } else if (origin == ma_seek_origin_end) {
61290 |         dwMoveMethod = FILE_END;
61291 |     } else {
61292 |         dwMoveMethod = FILE_BEGIN;
61293 |     }
61294 |
61295 |     if (ma_SetFilePointerEx != NULL) {
61296 |         result = ma_SetFilePointerEx((HANDLE)file, liDistanceToMove, NULL, dwMoveMethod);
61297 |     } else if (ma_SetFilePointer != NULL) {
61298 |         /* No SetFilePointerEx() so restrict to 31 bits. */
61299 |         if (offset > 0x7FFFFFFF) {
61300 |             return MA_OUT_OF_RANGE;
61301 |         }
61302 |
61303 |         result = ma_SetFilePointer((HANDLE)file, (LONG)liDistanceToMove.QuadPart, NULL,
> dwMoveMethod);
61304 |     } else {
61305 |         return MA_NOT_IMPLEMENTED;
61306 |     }
61307 |
61308 |     if (result == 0) {
61309 |         return ma_result_from_GetLastError(GetLastError());
61310 |     }
61311 |
61312 |     return MA_SUCCESS;
61313 | }
61314 |
61315 | static ma_result ma_default_vfs_tell__win32(ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor)
61316 | {
61317 |     LARGE_INTEGER liZero;
61318 |     LARGE_INTEGER liTell;
61319 |     BOOL result;
61320 |
61321 |     (void)pVFS;
61322 |
61323 |     liZero.QuadPart = 0;
61324 |
61325 |     if (ma_SetFilePointerEx != NULL) {
61326 |         result = ma_SetFilePointerEx((HANDLE)file, liZero, &liTell, FILE_CURRENT);
61327 |     } else if (ma_SetFilePointer != NULL) {
61328 |         LONG tell;
61329 |
61330 |         result = ma_SetFilePointer((HANDLE)file, (LONG)liZero.QuadPart, &tell, FILE_CURRENT);
61331 |         liTell.QuadPart = tell;
61332 |     } else {
61333 |         return MA_NOT_IMPLEMENTED;
61334 |     }
61335 |
61336 |     if (result == 0) {
61337 |         return ma_result_from_GetLastError(GetLastError());
61338 |     }
61339 |
61340 |     if (pCursor != NULL) {
61341 |         *pCursor = liTell.QuadPart;
61342 |     }
61343 |
61344 |     return MA_SUCCESS;
61345 | }
61346 |
61347 | static ma_result ma_default_vfs_info__win32(ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo)
61348 | {
61349 |     (void)pVFS;
61350 |
61351 |     #if !defined(MA_XBOX_NXDK)
61352 |     {
61353 |         BY_HANDLE_FILE_INFORMATION fi;
61354 |         BOOL result;
61355 |
61356 |         result = GetFileInformationByHandle((HANDLE)file, &fi);
61357 |         if (result == 0) {
61358 |             return ma_result_from_GetLastError(GetLastError());
61359 |         }
61360 |
61361 |         pInfo->sizeInBytes = ((ma_uint64)fi.nFileSizeHigh << 32) | ((ma_uint64)fi.nFileSizeLow);
61362 |
61363 |         return MA_SUCCESS;
61364 |     }

```



```

61365 |     #else
61366 |     {
61367 |         /* GetFileInformationByHandle() is unavailable. */
61368 |         return MA_NOT_IMPLEMENTED;
61369 |     }
61370 |     #endif
61371 | }
61372 | #else
61373 | static ma_result ma_default_vfs_open__stdio(ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile)
61374 | {
61375 |     ma_result result;
61376 |     FILE* pFileStd;
61377 |     const char* pOpenModeStr;
61378 |
61379 |     MA_ASSERT(pFilePath != NULL);
61380 |     MA_ASSERT(openMode != 0);
61381 |     MA_ASSERT(pFile != NULL);
61382 |
61383 |     (void)pVFS;
61384 |
61385 |     if ((openMode & MA_OPEN_MODE_READ) != 0) {
61386 |         if ((openMode & MA_OPEN_MODE_WRITE) != 0) {
61387 |             pOpenModeStr = "r+";
61388 |         } else {
61389 |             pOpenModeStr = "rb";
61390 |         }
61391 |     } else {
61392 |         pOpenModeStr = "wb";
61393 |     }
61394 |
61395 |     result = ma_fopen(&pFileStd, pFilePath, pOpenModeStr);
61396 |     if (result != MA_SUCCESS) {
61397 |         return result;
61398 |     }
61399 |
61400 |     *pFile = pFileStd;
61401 |
61402 |     return MA_SUCCESS;
61403 | }
61404 |
61405 | static ma_result ma_default_vfs_open_w__stdio(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32
> openMode, ma_vfs_file* pFile)
61406 | {
61407 |     ma_result result;
61408 |     FILE* pFileStd;
61409 |     const wchar_t* pOpenModeStr;
61410 |
61411 |     MA_ASSERT(pFilePath != NULL);
61412 |     MA_ASSERT(openMode != 0);
61413 |     MA_ASSERT(pFile != NULL);
61414 |
61415 |     (void)pVFS;
61416 |
61417 |     if ((openMode & MA_OPEN_MODE_READ) != 0) {
61418 |         if ((openMode & MA_OPEN_MODE_WRITE) != 0) {
61419 |             pOpenModeStr = L"r+";
61420 |         } else {
61421 |             pOpenModeStr = L"rb";
61422 |         }
61423 |     } else {
61424 |         pOpenModeStr = L"wb";
61425 |     }
61426 |
61427 |     result = ma_wfopen(&pFileStd, pFilePath, pOpenModeStr, (pVFS != NULL) ?
> &((ma_default_vfs*)pVFS)->allocationCallbacks : NULL);
61428 |     if (result != MA_SUCCESS) {
61429 |         return result;
61430 |     }
61431 |
61432 |     *pFile = pFileStd;
61433 |
61434 |     return MA_SUCCESS;
61435 | }
61436 |
61437 | static ma_result ma_default_vfs_close__stdio(ma_vfs* pVFS, ma_vfs_file file)
61438 | {
61439 |     MA_ASSERT(file != NULL);
61440 |

```

```
61441 |     (void)pVFS;
61442 |
61443 |     fclose((FILE*)file);
61444 |
61445 |     return MA_SUCCESS;
61446 | }
61447 |
61448 | static ma_result ma_default_vfs_read__stdio(ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t
> sizeInBytes, size_t* pBytesRead)
61449 | {
61450 |     size_t result;
61451 |
61452 |     MA_ASSERT(file != NULL);
61453 |     MA_ASSERT(pDst != NULL);
61454 |
61455 |     (void)pVFS;
61456 |
61457 |     result = fread(pDst, 1, sizeInBytes, (FILE*)file);
61458 |
61459 |     if (pBytesRead != NULL) {
61460 |         *pBytesRead = result;
61461 |     }
61462 |
61463 |     if (result != sizeInBytes) {
61464 |         if (result == 0 && feof((FILE*)file)) {
61465 |             return MA_AT_END;
61466 |         } else {
61467 |             return ma_result_from_errno(ferror((FILE*)file));
61468 |         }
61469 |     }
61470 |
61471 |     return MA_SUCCESS;
61472 | }
61473 |
61474 | static ma_result ma_default_vfs_write__stdio(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc,
> size_t sizeInBytes, size_t* pBytesWritten)
61475 | {
61476 |     size_t result;
61477 |
61478 |     MA_ASSERT(file != NULL);
61479 |     MA_ASSERT(pSrc != NULL);
61480 |
61481 |     (void)pVFS;
61482 |
61483 |     result = fwrite(pSrc, 1, sizeInBytes, (FILE*)file);
61484 |
61485 |     if (pBytesWritten != NULL) {
61486 |         *pBytesWritten = result;
61487 |     }
61488 |
61489 |     if (result != sizeInBytes) {
61490 |         return ma_result_from_errno(ferror((FILE*)file));
61491 |     }
61492 |
61493 |     return MA_SUCCESS;
61494 | }
61495 |
61496 | static ma_result ma_default_vfs_seek__stdio(ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset,
> ma_seek_origin origin)
61497 | {
61498 |     int result;
61499 |     int whence;
61500 |
61501 |     MA_ASSERT(file != NULL);
61502 |
61503 |     (void)pVFS;
61504 |
61505 |     if (origin == ma_seek_origin_start) {
61506 |         whence = SEEK_SET;
61507 |     } else if (origin == ma_seek_origin_end) {
61508 |         whence = SEEK_END;
61509 |     } else {
61510 |         whence = SEEK_CUR;
61511 |     }
61512 |
61513 |     #if defined(_WIN32)
61514 |         #if defined(_MSC_VER) && _MSC_VER > 1200
61515 |             result = _fseeki64((FILE*)file, offset, whence);
61516 |         #else
```

```

61517 |         /* No _fseeki64() so restrict to 31 bits. */
61518 |         if (offset > 0x7FFFFFFF) {
61519 |             return MA_OUT_OF_RANGE;
61520 |         }
61521 |
61522 |         result = fseek((FILE*)file, (int)offset, whence);
61523 |     #endif
61524 | #else
61525 |     result = fseek((FILE*)file, (long int)offset, whence);
61526 | #endif
61527 |     if (result != 0) {
61528 |         return MA_ERROR;
61529 |     }
61530 |
61531 |     return MA_SUCCESS;
61532 | }
61533 |
61534 | static ma_result ma_default_vfs_tell__stdio(ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor)
61535 | {
61536 |     ma_int64 result;
61537 |
61538 |     MA_ASSERT(file != NULL);
61539 |     MA_ASSERT(pCursor != NULL);
61540 |
61541 |     (void)pVFS;
61542 |
61543 |     #if defined(_WIN32)
61544 |         #if defined(_MSC_VER) && _MSC_VER > 1200
61545 |             result = _ftelli64((FILE*)file);
61546 |         #else
61547 |             result = ftell((FILE*)file);
61548 |         #endif
61549 |     #else
61550 |         result = ftell((FILE*)file);
61551 |     #endif
61552 |
61553 |     *pCursor = result;
61554 |
61555 |     return MA_SUCCESS;
61556 | }
61557 |
61558 | #if !defined(_MSC_VER) && !((defined(_POSIX_C_SOURCE) && _POSIX_C_SOURCE >= 1) ||
> defined(_XOPEN_SOURCE) || defined(_POSIX_SOURCE)) && !defined(MA_BSD)
61559 | int fileno(FILE *stream);
61560 | #endif
61561 |
61562 | static ma_result ma_default_vfs_info__stdio(ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo)
61563 | {
61564 |     int fd;
61565 |     struct stat info;
61566 |
61567 |     MA_ASSERT(file != NULL);
61568 |     MA_ASSERT(pInfo != NULL);
61569 |
61570 |     (void)pVFS;
61571 |
61572 |     #if defined(_MSC_VER)
61573 |         fd = _fileno((FILE*)file);
61574 |     #else
61575 |         fd = fileno((FILE*)file);
61576 |     #endif
61577 |
61578 |     if (fstat(fd, &info) != 0) {
61579 |         return ma_result_from_errno(errno);
61580 |     }
61581 |
61582 |     pInfo->sizeInBytes = info.st_size;
61583 |
61584 |     return MA_SUCCESS;
61585 | }
61586 | #endif
61587 |
61588 |
61589 | static ma_result ma_default_vfs_open(ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile)
61590 | {
61591 |     if (pFile == NULL) {
61592 |         return MA_INVALID_ARGS;
61593 |     }

```

```

61594 |
61595 |     *pFile = NULL;
61596 |
61597 |     if (pFilePath == NULL || openMode == 0) {
61598 |         return MA_INVALID_ARGS;
61599 |     }
61600 |
61601 |     #if defined(MA_USE_WIN32_FILEIO)
61602 |         return ma_default_vfs_open__win32(pVFS, pFilePath, openMode, pFile);
61603 |     #else
61604 |         return ma_default_vfs_open__stdio(pVFS, pFilePath, openMode, pFile);
61605 |     #endif
61606 | }
61607 |
61608 | static ma_result ma_default_vfs_open_w(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile)
61609 | {
61610 |     if (pFile == NULL) {
61611 |         return MA_INVALID_ARGS;
61612 |     }
61613 |
61614 |     *pFile = NULL;
61615 |
61616 |     if (pFilePath == NULL || openMode == 0) {
61617 |         return MA_INVALID_ARGS;
61618 |     }
61619 |
61620 |     #if defined(MA_USE_WIN32_FILEIO)
61621 |         return ma_default_vfs_open_w__win32(pVFS, pFilePath, openMode, pFile);
61622 |     #else
61623 |         return ma_default_vfs_open_w__stdio(pVFS, pFilePath, openMode, pFile);
61624 |     #endif
61625 | }
61626 |
61627 | static ma_result ma_default_vfs_close(ma_vfs* pVFS, ma_vfs_file file)
61628 | {
61629 |     if (file == NULL) {
61630 |         return MA_INVALID_ARGS;
61631 |     }
61632 |
61633 |     #if defined(MA_USE_WIN32_FILEIO)
61634 |         return ma_default_vfs_close__win32(pVFS, file);
61635 |     #else
61636 |         return ma_default_vfs_close__stdio(pVFS, file);
61637 |     #endif
61638 | }
61639 |
61640 | static ma_result ma_default_vfs_read(ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t sizeInBytes,
> size_t* pBytesRead)
61641 | {
61642 |     if (pBytesRead != NULL) {
61643 |         *pBytesRead = 0;
61644 |     }
61645 |
61646 |     if (file == NULL || pDst == NULL) {
61647 |         return MA_INVALID_ARGS;
61648 |     }
61649 |
61650 |     #if defined(MA_USE_WIN32_FILEIO)
61651 |         return ma_default_vfs_read__win32(pVFS, file, pDst, sizeInBytes, pBytesRead);
61652 |     #else
61653 |         return ma_default_vfs_read__stdio(pVFS, file, pDst, sizeInBytes, pBytesRead);
61654 |     #endif
61655 | }
61656 |
61657 | static ma_result ma_default_vfs_write(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc, size_t
> sizeInBytes, size_t* pBytesWritten)
61658 | {
61659 |     if (pBytesWritten != NULL) {
61660 |         *pBytesWritten = 0;
61661 |     }
61662 |
61663 |     if (file == NULL || pSrc == NULL) {
61664 |         return MA_INVALID_ARGS;
61665 |     }
61666 |
61667 |     #if defined(MA_USE_WIN32_FILEIO)
61668 |         return ma_default_vfs_write__win32(pVFS, file, pSrc, sizeInBytes, pBytesWritten);
61669 |     #else

```

```
61670 |     return ma_default_vfs_write_stdio(pVFS, file, pSrc, sizeInBytes, pBytesWritten);
61671 | #endif
61672 | }
61673 |
61674 | static ma_result ma_default_vfs_seek(ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset, ma_seek_origin
> origin)
61675 | {
61676 |     if (file == NULL) {
61677 |         return MA_INVALID_ARGS;
61678 |     }
61679 |
61680 |     #if defined(MA_USE_WIN32_FILEIO)
61681 |         return ma_default_vfs_seek_win32(pVFS, file, offset, origin);
61682 |     #else
61683 |         return ma_default_vfs_seek_stdio(pVFS, file, offset, origin);
61684 |     #endif
61685 | }
61686 |
61687 | static ma_result ma_default_vfs_tell(ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor)
61688 | {
61689 |     if (pCursor == NULL) {
61690 |         return MA_INVALID_ARGS;
61691 |     }
61692 |
61693 |     *pCursor = 0;
61694 |
61695 |     if (file == NULL) {
61696 |         return MA_INVALID_ARGS;
61697 |     }
61698 |
61699 |     #if defined(MA_USE_WIN32_FILEIO)
61700 |         return ma_default_vfs_tell_win32(pVFS, file, pCursor);
61701 |     #else
61702 |         return ma_default_vfs_tell_stdio(pVFS, file, pCursor);
61703 |     #endif
61704 | }
61705 |
61706 | static ma_result ma_default_vfs_info(ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo)
61707 | {
61708 |     ma_result result;
61709 |
61710 |     if (pInfo == NULL) {
61711 |         return MA_INVALID_ARGS;
61712 |     }
61713 |
61714 |     MA_ZERO_OBJECT(pInfo);
61715 |
61716 |     if (file == NULL) {
61717 |         return MA_INVALID_ARGS;
61718 |     }
61719 |
61720 |     #if defined(MA_USE_WIN32_FILEIO)
61721 |         result = ma_default_vfs_info_win32(pVFS, file, pInfo);
61722 |     #else
61723 |         result = ma_default_vfs_info_stdio(pVFS, file, pInfo);
61724 |     #endif
61725 |
61726 |     if (result == MA_NOT_IMPLEMENTED) {
61727 |         /* Not implemented. Fall back to seek/tell/seek. */
61728 |         ma_int64 cursor;
61729 |         ma_int64 sizeInBytes;
61730 |
61731 |         result = ma_default_vfs_tell(pVFS, file, &cursor);
61732 |         if (result != MA_SUCCESS) {
61733 |             return result;
61734 |         }
61735 |
61736 |         result = ma_default_vfs_seek(pVFS, file, 0, ma_seek_origin_end);
61737 |         if (result != MA_SUCCESS) {
61738 |             return result;
61739 |         }
61740 |
61741 |         result = ma_default_vfs_tell(pVFS, file, &sizeInBytes);
61742 |         if (result != MA_SUCCESS) {
61743 |             return result;
61744 |         }
61745 |
61746 |         pInfo->sizeInBytes = sizeInBytes;
61747 |     }
```

```

61748 |         result = ma_default_vfs_seek(pVFS, file, cursor, ma_seek_origin_start);
61749 |         if (result != MA_SUCCESS) {
61750 |             return result;
61751 |         }
61752 |
61753 |         MA_ASSERT(result == MA_SUCCESS);
61754 |     }
61755 |
61756 |     return result;
61757 | }
61758 |
61759 |
61760 | MA_API ma_result ma_default_vfs_init(ma_default_vfs* pVFS, const ma_allocation_callbacks*
> pAllocationCallbacks)
61761 | {
61762 |     if (pVFS == NULL) {
61763 |         return MA_INVALID_ARGS;
61764 |     }
61765 |
61766 |     pVFS->cb.onOpen = ma_default_vfs_open;
61767 |     pVFS->cb.onOpenW = ma_default_vfs_open_w;
61768 |     pVFS->cb.onClose = ma_default_vfs_close;
61769 |     pVFS->cb.onRead = ma_default_vfs_read;
61770 |     pVFS->cb.onWrite = ma_default_vfs_write;
61771 |     pVFS->cb.onSeek = ma_default_vfs_seek;
61772 |     pVFS->cb.onTell = ma_default_vfs_tell;
61773 |     pVFS->cb.onInfo = ma_default_vfs_info;
61774 |     ma_allocation_callbacks_init_copy(&pVFS->allocationCallbacks, pAllocationCallbacks);
61775 |
61776 |     return MA_SUCCESS;
61777 | }
61778 |
61779 |
61780 | MA_API ma_result ma_vfs_or_default_open(ma_vfs* pVFS, const char* pFilePath, ma_uint32 openMode,
> ma_vfs_file* pFile)
61781 | {
61782 |     if (pVFS != NULL) {
61783 |         return ma_vfs_open(pVFS, pFilePath, openMode, pFile);
61784 |     } else {
61785 |         return ma_default_vfs_open(pVFS, pFilePath, openMode, pFile);
61786 |     }
61787 | }
61788 |
61789 | MA_API ma_result ma_vfs_or_default_open_w(ma_vfs* pVFS, const wchar_t* pFilePath, ma_uint32
> openMode, ma_vfs_file* pFile)
61790 | {
61791 |     if (pVFS != NULL) {
61792 |         return ma_vfs_open_w(pVFS, pFilePath, openMode, pFile);
61793 |     } else {
61794 |         return ma_default_vfs_open_w(pVFS, pFilePath, openMode, pFile);
61795 |     }
61796 | }
61797 |
61798 | MA_API ma_result ma_vfs_or_default_close(ma_vfs* pVFS, ma_vfs_file file)
61799 | {
61800 |     if (pVFS != NULL) {
61801 |         return ma_vfs_close(pVFS, file);
61802 |     } else {
61803 |         return ma_default_vfs_close(pVFS, file);
61804 |     }
61805 | }
61806 |
61807 | MA_API ma_result ma_vfs_or_default_read(ma_vfs* pVFS, ma_vfs_file file, void* pDst, size_t
> sizeInBytes, size_t* pBytesRead)
61808 | {
61809 |     if (pVFS != NULL) {
61810 |         return ma_vfs_read(pVFS, file, pDst, sizeInBytes, pBytesRead);
61811 |     } else {
61812 |         return ma_default_vfs_read(pVFS, file, pDst, sizeInBytes, pBytesRead);
61813 |     }
61814 | }
61815 |
61816 | MA_API ma_result ma_vfs_or_default_write(ma_vfs* pVFS, ma_vfs_file file, const void* pSrc, size_t
> sizeInBytes, size_t* pBytesWritten)
61817 | {
61818 |     if (pVFS != NULL) {
61819 |         return ma_vfs_write(pVFS, file, pSrc, sizeInBytes, pBytesWritten);
61820 |     } else {
61821 |         return ma_default_vfs_write(pVFS, file, pSrc, sizeInBytes, pBytesWritten);

```

```

61822 |     }
61823 | }
61824 |
61825 | MA_API ma_result ma_vfs_or_default_seek(ma_vfs* pVFS, ma_vfs_file file, ma_int64 offset,
> ma_seek_origin origin)
61826 | {
61827 |     if (pVFS != NULL) {
61828 |         return ma_vfs_seek(pVFS, file, offset, origin);
61829 |     } else {
61830 |         return ma_default_vfs_seek(pVFS, file, offset, origin);
61831 |     }
61832 | }
61833 |
61834 | MA_API ma_result ma_vfs_or_default_tell(ma_vfs* pVFS, ma_vfs_file file, ma_int64* pCursor)
61835 | {
61836 |     if (pVFS != NULL) {
61837 |         return ma_vfs_tell(pVFS, file, pCursor);
61838 |     } else {
61839 |         return ma_default_vfs_tell(pVFS, file, pCursor);
61840 |     }
61841 | }
61842 |
61843 | MA_API ma_result ma_vfs_or_default_info(ma_vfs* pVFS, ma_vfs_file file, ma_file_info* pInfo)
61844 | {
61845 |     if (pVFS != NULL) {
61846 |         return ma_vfs_info(pVFS, file, pInfo);
61847 |     } else {
61848 |         return ma_default_vfs_info(pVFS, file, pInfo);
61849 |     }
61850 | }
61851 |
61852 |
61853 |
61854 | static ma_result ma_vfs_open_and_read_file_ex(ma_vfs* pVFS, const char* pFilePath, const wchar_t*
> pFilePathW, void** ppData, size_t* pSize, const ma_allocation_callbacks* pAllocationCallbacks)
61855 | {
61856 |     ma_result result;
61857 |     ma_vfs_file file;
61858 |     ma_file_info info;
61859 |     void* pData;
61860 |     size_t bytesRead;
61861 |
61862 |     if (ppData != NULL) {
61863 |         *ppData = NULL;
61864 |     }
61865 |     if (pSize != NULL) {
61866 |         *pSize = 0;
61867 |     }
61868 |
61869 |     if (ppData == NULL) {
61870 |         return MA_INVALID_ARGS;
61871 |     }
61872 |
61873 |     if (pFilePath != NULL) {
61874 |         result = ma_vfs_or_default_open(pVFS, pFilePath, MA_OPEN_MODE_READ, &file);
61875 |     } else {
61876 |         result = ma_vfs_or_default_open_w(pVFS, pFilePathW, MA_OPEN_MODE_READ, &file);
61877 |     }
61878 |     if (result != MA_SUCCESS) {
61879 |         return result;
61880 |     }
61881 |
61882 |     result = ma_vfs_or_default_info(pVFS, file, &info);
61883 |     if (result != MA_SUCCESS) {
61884 |         ma_vfs_or_default_close(pVFS, file);
61885 |         return result;
61886 |     }
61887 |
61888 |     if (info.sizeInBytes > MA_SIZE_MAX) {
61889 |         ma_vfs_or_default_close(pVFS, file);
61890 |         return MA_TOO_BIG;
61891 |     }
61892 |
61893 |     pData = ma_malloc((size_t)info.sizeInBytes, pAllocationCallbacks); /* Safe cast. */
61894 |     if (pData == NULL) {
61895 |         ma_vfs_or_default_close(pVFS, file);
61896 |         return result;
61897 |     }
61898 | }

```



```

61899 |     result = ma_vfs_or_default_read(pVFS, file, pData, (size_t)info.sizeInBytes, &bytesRead); /*
> Safe cast. */
61900 |     ma_vfs_or_default_close(pVFS, file);
61901 |
61902 |     if (result != MA_SUCCESS) {
61903 |         ma_free(pData, pAllocationCallbacks);
61904 |         return result;
61905 |     }
61906 |
61907 |     if (pSize != NULL) {
61908 |         *pSize = bytesRead;
61909 |     }
61910 |
61911 |     MA_ASSERT(ppData != NULL);
61912 |     *ppData = pData;
61913 |
61914 |     return MA_SUCCESS;
61915 | }
61916 |
61917 | MA_API ma_result ma_vfs_open_and_read_file(ma_vfs* pVFS, const char* pFilePath, void** ppData,
> size_t* pSize, const ma_allocation_callbacks* pAllocationCallbacks)
61918 | {
61919 |     return ma_vfs_open_and_read_file_ex(pVFS, pFilePath, NULL, ppData, pSize, pAllocationCallbacks);
61920 | }
61921 |
61922 | MA_API ma_result ma_vfs_open_and_read_file_w(ma_vfs* pVFS, const wchar_t* pFilePath, void** ppData,
> size_t* pSize, const ma_allocation_callbacks* pAllocationCallbacks)
61923 | {
61924 |     return ma_vfs_open_and_read_file_ex(pVFS, NULL, pFilePath, ppData, pSize, pAllocationCallbacks);
61925 | }
61926 |
61927 |
61928 |
61929 | /*****
> *****/
61930 |
61931 | Decoding and Encoding Headers. These are auto-generated from a tool.
61932 |
61933 | *****/
> *****/
61934 | #if !defined(MA_NO_WAV) && (!defined(MA_NO_DECODING) || !defined(MA_NO_ENCODING))
61935 | #define MA_HAS_WAV
61936 |
61937 | /* dr_wav_h begin */
61938 | #ifndef ma_dr_wav_h
61939 | #define ma_dr_wav_h
61940 | #ifdef __cplusplus
61941 | extern "C" {
61942 | #endif
61943 | #define MA_DR_WAV_STRINGIFY(x)      #x
61944 | #define MA_DR_WAV_XSTRINGIFY(x)    MA_DR_WAV_STRINGIFY(x)
61945 | #define MA_DR_WAV_VERSION_MAJOR    0
61946 | #define MA_DR_WAV_VERSION_MINOR    14
61947 | #define MA_DR_WAV_VERSION_REVISION 5
61948 | #define MA_DR_WAV_VERSION_STRING    MA_DR_WAV_XSTRINGIFY(MA_DR_WAV_VERSION_MAJOR) "."
> MA_DR_WAV_XSTRINGIFY(MA_DR_WAV_VERSION_MINOR) "." MA_DR_WAV_XSTRINGIFY(MA_DR_WAV_VERSION_REVISION)
61949 | #include <stddef.h>
61950 | #define MA_DR_WAVE_FORMAT_PCM      0x1
61951 | #define MA_DR_WAVE_FORMAT_ADPCM    0x2
61952 | #define MA_DR_WAVE_FORMAT_IEEE_FLOAT 0x3
61953 | #define MA_DR_WAVE_FORMAT_ALAW     0x6
61954 | #define MA_DR_WAVE_FORMAT_MULAW    0x7
61955 | #define MA_DR_WAVE_FORMAT_DVI_ADPCM 0x11
61956 | #define MA_DR_WAVE_FORMAT_EXTENSIBLE 0xFFFE
61957 | #define MA_DR_WAV_SEQUENTIAL        0x00000001
61958 | #define MA_DR_WAV_WITH_METADATA     0x00000002
61959 | MA_API void ma_dr_wav_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision);
61960 | MA_API const char* ma_dr_wav_version_string(void);
61961 | typedef enum
61962 | {
61963 |     MA_DR_WAV_SEEK_SET,
61964 |     MA_DR_WAV_SEEK_CUR,
61965 |     MA_DR_WAV_SEEK_END
61966 | } ma_dr_wav_seek_origin;
61967 | typedef enum
61968 | {
61969 |     ma_dr_wav_container_riff,
61970 |     ma_dr_wav_container_rifx,
61971 |     ma_dr_wav_container_w64,

```

```

61972 |     ma_dr_wav_container_rf64,
61973 |     ma_dr_wav_container_aiff
61974 | } ma_dr_wav_container;
61975 | typedef struct
61976 | {
61977 |     union
61978 |     {
61979 |         ma_uint8 fourcc[4];
61980 |         ma_uint8 guid[16];
61981 |     } id;
61982 |     ma_uint64 sizeInBytes;
61983 |     unsigned int paddingSize;
61984 | } ma_dr_wav_chunk_header;
61985 | typedef struct
61986 | {
61987 |     ma_uint16 formatTag;
61988 |     ma_uint16 channels;
61989 |     ma_uint32 sampleRate;
61990 |     ma_uint32 avgBytesPerSec;
61991 |     ma_uint16 blockAlign;
61992 |     ma_uint16 bitsPerSample;
61993 |     ma_uint16 extendedSize;
61994 |     ma_uint16 validBitsPerSample;
61995 |     ma_uint32 channelMask;
61996 |     ma_uint8 subFormat[16];
61997 | } ma_dr_wav_fmt;
61998 | MA_API ma_uint16 ma_dr_wav_fmt_get_format(const ma_dr_wav_fmt* pFMT);
61999 | typedef size_t (* ma_dr_wav_read_proc)(void* pUserData, void* pBufferOut, size_t bytesToRead);
62000 | typedef size_t (* ma_dr_wav_write_proc)(void* pUserData, const void* pData, size_t bytesToWrite);
62001 | typedef ma_bool32 (* ma_dr_wav_seek_proc)(void* pUserData, int offset, ma_dr_wav_seek_origin
> origin);
62002 | typedef ma_bool32 (* ma_dr_wav_tell_proc)(void* pUserData, ma_int64* pCursor);
62003 | typedef ma_uint64 (* ma_dr_wav_chunk_proc)(void* pChunkUserData, ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, void* pReadSeekUserData, const ma_dr_wav_chunk_header* pChunkHeader,
> ma_dr_wav_container container, const ma_dr_wav_fmt* pFMT);
62004 | typedef struct
62005 | {
62006 |     const ma_uint8* data;
62007 |     size_t dataSize;
62008 |     size_t currentReadPos;
62009 | } ma_dr_wav__memory_stream;
62010 | typedef struct
62011 | {
62012 |     void** ppData;
62013 |     size_t* pDataSize;
62014 |     size_t dataSize;
62015 |     size_t dataCapacity;
62016 |     size_t currentWritePos;
62017 | } ma_dr_wav__memory_stream_write;
62018 | typedef struct
62019 | {
62020 |     ma_dr_wav_container container;
62021 |     ma_uint32 format;
62022 |     ma_uint32 channels;
62023 |     ma_uint32 sampleRate;
62024 |     ma_uint32 bitsPerSample;
62025 | } ma_dr_wav_data_format;
62026 | typedef enum
62027 | {
62028 |     ma_dr_wav_metadata_type_none = 0,
62029 |     ma_dr_wav_metadata_type_unknown = 1 << 0,
62030 |     ma_dr_wav_metadata_type_smpl = 1 << 1,
62031 |     ma_dr_wav_metadata_type_inst = 1 << 2,
62032 |     ma_dr_wav_metadata_type_cue = 1 << 3,
62033 |     ma_dr_wav_metadata_type_acid = 1 << 4,
62034 |     ma_dr_wav_metadata_type_bext = 1 << 5,
62035 |     ma_dr_wav_metadata_type_list_label = 1 << 6,
62036 |     ma_dr_wav_metadata_type_list_note = 1 << 7,
62037 |     ma_dr_wav_metadata_type_list_labelled_cue_region = 1 << 8,
62038 |     ma_dr_wav_metadata_type_list_info_software = 1 << 9,
62039 |     ma_dr_wav_metadata_type_list_info_copyright = 1 << 10,
62040 |     ma_dr_wav_metadata_type_list_info_title = 1 << 11,
62041 |     ma_dr_wav_metadata_type_list_info_artist = 1 << 12,
62042 |     ma_dr_wav_metadata_type_list_info_comment = 1 << 13,
62043 |     ma_dr_wav_metadata_type_list_info_date = 1 << 14,
62044 |     ma_dr_wav_metadata_type_list_info_genre = 1 << 15,
62045 |     ma_dr_wav_metadata_type_list_info_album = 1 << 16,
62046 |     ma_dr_wav_metadata_type_list_info_tracknumber = 1 << 17,
62047 |     ma_dr_wav_metadata_type_list_info_location = 1 << 18,

```

```

62048 |     ma_dr_wav_metadata_type_list_info_organization      = 1 << 19,
62049 |     ma_dr_wav_metadata_type_list_info_keywords        = 1 << 20,
62050 |     ma_dr_wav_metadata_type_list_info_medium          = 1 << 21,
62051 |     ma_dr_wav_metadata_type_list_info_description     = 1 << 22,
62052 |     ma_dr_wav_metadata_type_list_all_info_strings     = ma_dr_wav_metadata_type_list_info_software
62053 | | ma_dr_wav_metadata_type_list_info_copyright
62054 | | ma_dr_wav_metadata_type_list_info_title
62055 | | ma_dr_wav_metadata_type_list_info_artist
62056 | | ma_dr_wav_metadata_type_list_info_comment
62057 | | ma_dr_wav_metadata_type_list_info_date
62058 | | ma_dr_wav_metadata_type_list_info_genre
62059 | | ma_dr_wav_metadata_type_list_info_album
62060 | | ma_dr_wav_metadata_type_list_info_tracknumber
62061 | | ma_dr_wav_metadata_type_list_info_location
62062 | | ma_dr_wav_metadata_type_list_info_organization
62063 | | ma_dr_wav_metadata_type_list_info_keywords
62064 | | ma_dr_wav_metadata_type_list_info_medium
62065 | | ma_dr_wav_metadata_type_list_info_description,
62066 |     ma_dr_wav_metadata_type_list_all_adtl             = ma_dr_wav_metadata_type_list_label
62067 | | ma_dr_wav_metadata_type_list_note
62068 | |
> ma_dr_wav_metadata_type_list_labelled_cue_region,
62069 |     ma_dr_wav_metadata_type_all                       = -2,
62070 |     ma_dr_wav_metadata_type_all_including_unknown     = -1
62071 | } ma_dr_wav_metadata_type;
62072 | typedef enum
62073 | {
62074 |     ma_dr_wav_smpl_loop_type_forward = 0,
62075 |     ma_dr_wav_smpl_loop_type_pingpong = 1,
62076 |     ma_dr_wav_smpl_loop_type_backward = 2
62077 | } ma_dr_wav_smpl_loop_type;
62078 | typedef struct
62079 | {
62080 |     ma_uint32 cuePointId;
62081 |     ma_uint32 type;
62082 |     ma_uint32 firstSampleOffset;
62083 |     ma_uint32 lastSampleOffset;
62084 |     ma_uint32 sampleFraction;
62085 |     ma_uint32 playCount;
62086 | } ma_dr_wav_smpl_loop;
62087 | typedef struct
62088 | {
62089 |     ma_uint32 manufacturerId;
62090 |     ma_uint32 productId;
62091 |     ma_uint32 samplePeriodNanoseconds;
62092 |     ma_uint32 midiUnityNote;
62093 |     ma_uint32 midiPitchFraction;
62094 |     ma_uint32 smpteFormat;
62095 |     ma_uint32 smpteOffset;
62096 |     ma_uint32 sampleLoopCount;
62097 |     ma_uint32 samplerSpecificDataSizeInBytes;
62098 |     ma_dr_wav_smpl_loop* pLoops;
62099 |     ma_uint8* pSamplerSpecificData;
62100 | } ma_dr_wav_smpl;
62101 | typedef struct
62102 | {
62103 |     ma_int8 midiUnityNote;
62104 |     ma_int8 fineTuneCents;
62105 |     ma_int8 gainDecibels;
62106 |     ma_int8 lowNote;
62107 |     ma_int8 highNote;
62108 |     ma_int8 lowVelocity;
62109 |     ma_int8 highVelocity;
62110 | } ma_dr_wav_inst;
62111 | typedef struct
62112 | {
62113 |     ma_uint32 id;
62114 |     ma_uint32 playOrderPosition;
62115 |     ma_uint8 dataChunkId[4];
62116 |     ma_uint32 chunkStart;
62117 |     ma_uint32 blockStart;
62118 |     ma_uint32 sampleOffset;
62119 | } ma_dr_wav_cue_point;
62120 | typedef struct
62121 | {
62122 |     ma_uint32 cuePointCount;
62123 |     ma_dr_wav_cue_point *pCuePoints;
62124 | } ma_dr_wav_cue;
62125 | typedef enum

```

```

62126 | {
62127 |     ma_dr_wav_acid_flag_one_shot      = 1,
62128 |     ma_dr_wav_acid_flag_root_note_set = 2,
62129 |     ma_dr_wav_acid_flag_stretch       = 4,
62130 |     ma_dr_wav_acid_flag_disk_based    = 8,
62131 |     ma_dr_wav_acid_flag_acidizer      = 16
62132 | } ma_dr_wav_acid_flag;
62133 | typedef struct
62134 | {
62135 |     ma_uint32 flags;
62136 |     ma_uint16 midiUnityNote;
62137 |     ma_uint16 reserved1;
62138 |     float reserved2;
62139 |     ma_uint32 numBeats;
62140 |     ma_uint16 meterDenominator;
62141 |     ma_uint16 meterNumerator;
62142 |     float tempo;
62143 | } ma_dr_wav_acid;
62144 | typedef struct
62145 | {
62146 |     ma_uint32 cuePointId;
62147 |     ma_uint32 stringLength;
62148 |     char* pString;
62149 | } ma_dr_wav_list_label_or_note;
62150 | typedef struct
62151 | {
62152 |     char* pDescription;
62153 |     char* pOriginatorName;
62154 |     char* pOriginatorReference;
62155 |     char  pOriginationDate[10];
62156 |     char  pOriginationTime[8];
62157 |     ma_uint64 timeReference;
62158 |     ma_uint16 version;
62159 |     char* pCodingHistory;
62160 |     ma_uint32 codingHistorySize;
62161 |     ma_uint8* pUMID;
62162 |     ma_uint16 loudnessValue;
62163 |     ma_uint16 loudnessRange;
62164 |     ma_uint16 maxTruePeakLevel;
62165 |     ma_uint16 maxMomentaryLoudness;
62166 |     ma_uint16 maxShortTermLoudness;
62167 | } ma_dr_wav_bext;
62168 | typedef struct
62169 | {
62170 |     ma_uint32 stringLength;
62171 |     char* pString;
62172 | } ma_dr_wav_list_info_text;
62173 | typedef struct
62174 | {
62175 |     ma_uint32 cuePointId;
62176 |     ma_uint32 sampleLength;
62177 |     ma_uint8 purposeId[4];
62178 |     ma_uint16 country;
62179 |     ma_uint16 language;
62180 |     ma_uint16 dialect;
62181 |     ma_uint16 codePage;
62182 |     ma_uint32 stringLength;
62183 |     char* pString;
62184 | } ma_dr_wav_list_labelled_cue_region;
62185 | typedef enum
62186 | {
62187 |     ma_dr_wav_metadata_location_invalid,
62188 |     ma_dr_wav_metadata_location_top_level,
62189 |     ma_dr_wav_metadata_location_inside_info_list,
62190 |     ma_dr_wav_metadata_location_inside_adtl_list
62191 | } ma_dr_wav_metadata_location;
62192 | typedef struct
62193 | {
62194 |     ma_uint8 id[4];
62195 |     ma_dr_wav_metadata_location chunkLocation;
62196 |     ma_uint32 dataSizeInBytes;
62197 |     ma_uint8* pData;
62198 | } ma_dr_wav_unknown_metadata;
62199 | typedef struct
62200 | {
62201 |     ma_dr_wav_metadata_type type;
62202 |     union
62203 |     {
62204 |         ma_dr_wav_cue cue;

```

```

62205 |         ma_dr_wav_smpl smpl;
62206 |         ma_dr_wav_acid acid;
62207 |         ma_dr_wav_inst inst;
62208 |         ma_dr_wav_bext bext;
62209 |         ma_dr_wav_list_label_or_note labelOrNote;
62210 |         ma_dr_wav_list_labelled_cue_region labelledCueRegion;
62211 |         ma_dr_wav_list_info_text infoText;
62212 |         ma_dr_wav_unknown_metadata unknown;
62213 |     } data;
62214 | } ma_dr_wav_metadata;
62215 | typedef struct
62216 | {
62217 |     ma_dr_wav_read_proc onRead;
62218 |     ma_dr_wav_write_proc onWrite;
62219 |     ma_dr_wav_seek_proc onSeek;
62220 |     ma_dr_wav_tell_proc onTell;
62221 |     void* pUserData;
62222 |     ma_allocation_callbacks allocationCallbacks;
62223 |     ma_dr_wav_container container;
62224 |     ma_dr_wav_fmt fmt;
62225 |     ma_uint32 sampleRate;
62226 |     ma_uint16 channels;
62227 |     ma_uint16 bitsPerSample;
62228 |     ma_uint16 translatedFormatTag;
62229 |     ma_uint64 totalPCMFrameCount;
62230 |     ma_uint64 dataChunkDataSize;
62231 |     ma_uint64 dataChunkDataPos;
62232 |     ma_uint64 bytesRemaining;
62233 |     ma_uint64 readCursorInPCMFrames;
62234 |     ma_uint64 dataChunkDataSizeTargetWrite;
62235 |     ma_bool32 isSequentialWrite;
62236 |     ma_dr_wav_metadata* pMetadata;
62237 |     ma_uint32 metadataCount;
62238 |     ma_dr_wav__memory_stream memoryStream;
62239 |     ma_dr_wav__memory_stream_write memoryStreamWrite;
62240 |     struct
62241 |     {
62242 |         ma_uint32 bytesRemainingInBlock;
62243 |         ma_uint16 predictor[2];
62244 |         ma_int32 delta[2];
62245 |         ma_int32 cachedFrames[4];
62246 |         ma_uint32 cachedFrameCount;
62247 |         ma_int32 prevFrames[2][2];
62248 |     } msadpcm;
62249 |     struct
62250 |     {
62251 |         ma_uint32 bytesRemainingInBlock;
62252 |         ma_int32 predictor[2];
62253 |         ma_int32 stepIndex[2];
62254 |         ma_int32 cachedFrames[16];
62255 |         ma_uint32 cachedFrameCount;
62256 |     } ima;
62257 |     struct
62258 |     {
62259 |         ma_bool8 isLE;
62260 |         ma_bool8 isUnsigned;
62261 |     } aiff;
62262 | } ma_dr_wav;
62263 | MA_API ma_bool32 ma_dr_wav_init(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead, ma_dr_wav_seek_proc
> onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, const ma_allocation_callbacks*
> pAllocationCallbacks);
62264 | MA_API ma_bool32 ma_dr_wav_init_ex(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead, ma_dr_wav_seek_proc
> onSeek, ma_dr_wav_tell_proc onTell, ma_dr_wav_chunk_proc onChunk, void* pReadSeekTellUserData, void*
> pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks);
62265 | MA_API ma_bool32 ma_dr_wav_init_with_metadata(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, ma_uint32 flags, const
> ma_allocation_callbacks* pAllocationCallbacks);
62266 | MA_API ma_bool32 ma_dr_wav_init_write(ma_dr_wav* pWav, const ma_dr_wav_data_format* pFormat,
> ma_dr_wav_write_proc onWrite, ma_dr_wav_seek_proc onSeek, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks);
62267 | MA_API ma_bool32 ma_dr_wav_init_write_sequential(ma_dr_wav* pWav, const ma_dr_wav_data_format*
> pFormat, ma_uint64 totalSampleCount, ma_dr_wav_write_proc onWrite, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks);
62268 | MA_API ma_bool32 ma_dr_wav_init_write_sequential_pcm_frames(ma_dr_wav* pWav, const
> ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, ma_dr_wav_write_proc onWrite, void*
> pUserData, const ma_allocation_callbacks* pAllocationCallbacks);
62269 | MA_API ma_bool32 ma_dr_wav_init_write_with_metadata(ma_dr_wav* pWav, const ma_dr_wav_data_format*
> pFormat, ma_dr_wav_write_proc onWrite, ma_dr_wav_seek_proc onSeek, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_dr_wav_metadata* pMetadata, ma_uint32

```

```

> metadataCount);
62270 | MA_API ma_uint64 ma_dr_wav_target_write_size_bytes(const ma_dr_wav_data_format* pFormat, ma_uint64
> totalFrameCount, ma_dr_wav_metadata* pMetadata, ma_uint32 metadataCount);
62271 | MA_API ma_dr_wav_metadata* ma_dr_wav_take_ownership_of_metadata(ma_dr_wav* pWav);
62272 | MA_API ma_result ma_dr_wav_uninit(ma_dr_wav* pWav);
62273 | MA_API size_t ma_dr_wav_read_raw(ma_dr_wav* pWav, size_t bytesToRead, void* pBufferOut);
62274 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames(ma_dr_wav* pWav, ma_uint64 framesToRead, void*
> pBufferOut);
62275 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_le(ma_dr_wav* pWav, ma_uint64 framesToRead, void*
> pBufferOut);
62276 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_be(ma_dr_wav* pWav, ma_uint64 framesToRead, void*
> pBufferOut);
62277 | MA_API ma_bool32 ma_dr_wav_seek_to_pcm_frame(ma_dr_wav* pWav, ma_uint64 targetFrameIndex);
62278 | MA_API ma_result ma_dr_wav_get_cursor_in_pcm_frames(ma_dr_wav* pWav, ma_uint64* pCursor);
62279 | MA_API ma_result ma_dr_wav_get_length_in_pcm_frames(ma_dr_wav* pWav, ma_uint64* pLength);
62280 | MA_API size_t ma_dr_wav_write_raw(ma_dr_wav* pWav, size_t bytesToWrite, const void* pData);
62281 | MA_API ma_uint64 ma_dr_wav_write_pcm_frames(ma_dr_wav* pWav, ma_uint64 framesToWrite, const void*
> pData);
62282 | MA_API ma_uint64 ma_dr_wav_write_pcm_frames_le(ma_dr_wav* pWav, ma_uint64 framesToWrite, const void*
> pData);
62283 | MA_API ma_uint64 ma_dr_wav_write_pcm_frames_be(ma_dr_wav* pWav, ma_uint64 framesToWrite, const void*
> pData);
62284 | #ifndef MA_DR_WAV_NO_CONVERSION_API
62285 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s16(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int16*
> pBufferOut);
62286 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s16le(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int16*
> pBufferOut);
62287 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s16be(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int16*
> pBufferOut);
62288 | MA_API void ma_dr_wav_u8_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount);
62289 | MA_API void ma_dr_wav_s24_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount);
62290 | MA_API void ma_dr_wav_s32_to_s16(ma_int16* pOut, const ma_int32* pIn, size_t sampleCount);
62291 | MA_API void ma_dr_wav_f32_to_s16(ma_int16* pOut, const float* pIn, size_t sampleCount);
62292 | MA_API void ma_dr_wav_f64_to_s16(ma_int16* pOut, const double* pIn, size_t sampleCount);
62293 | MA_API void ma_dr_wav_alaw_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount);
62294 | MA_API void ma_dr_wav_mulaw_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount);
62295 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_f32(ma_dr_wav* pWav, ma_uint64 framesToRead, float*
> pBufferOut);
62296 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_f32le(ma_dr_wav* pWav, ma_uint64 framesToRead, float*
> pBufferOut);
62297 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_f32be(ma_dr_wav* pWav, ma_uint64 framesToRead, float*
> pBufferOut);
62298 | MA_API void ma_dr_wav_u8_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount);
62299 | MA_API void ma_dr_wav_s16_to_f32(float* pOut, const ma_int16* pIn, size_t sampleCount);
62300 | MA_API void ma_dr_wav_s24_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount);
62301 | MA_API void ma_dr_wav_s32_to_f32(float* pOut, const ma_int32* pIn, size_t sampleCount);
62302 | MA_API void ma_dr_wav_f64_to_f32(float* pOut, const double* pIn, size_t sampleCount);
62303 | MA_API void ma_dr_wav_alaw_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount);
62304 | MA_API void ma_dr_wav_mulaw_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount);
62305 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s32(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int32*
> pBufferOut);
62306 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s32le(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int32*
> pBufferOut);
62307 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s32be(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int32*
> pBufferOut);
62308 | MA_API void ma_dr_wav_u8_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount);
62309 | MA_API void ma_dr_wav_s16_to_s32(ma_int32* pOut, const ma_int16* pIn, size_t sampleCount);
62310 | MA_API void ma_dr_wav_s24_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount);
62311 | MA_API void ma_dr_wav_f32_to_s32(ma_int32* pOut, const float* pIn, size_t sampleCount);
62312 | MA_API void ma_dr_wav_f64_to_s32(ma_int32* pOut, const double* pIn, size_t sampleCount);
62313 | MA_API void ma_dr_wav_alaw_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount);
62314 | MA_API void ma_dr_wav_mulaw_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount);
62315 | #endif
62316 | #ifndef MA_DR_WAV_NO_STDIO
62317 | MA_API ma_bool32 ma_dr_wav_init_file(ma_dr_wav* pWav, const char* filename, const
> ma_allocation_callbacks* pAllocationCallbacks);
62318 | MA_API ma_bool32 ma_dr_wav_init_file_ex(ma_dr_wav* pWav, const char* filename, ma_dr_wav_chunk_proc
> onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks*
> pAllocationCallbacks);
62319 | MA_API ma_bool32 ma_dr_wav_init_file_w(ma_dr_wav* pWav, const wchar_t* filename, const
> ma_allocation_callbacks* pAllocationCallbacks);
62320 | MA_API ma_bool32 ma_dr_wav_init_file_ex_w(ma_dr_wav* pWav, const wchar_t* filename,
> ma_dr_wav_chunk_proc onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks*
> pAllocationCallbacks);
62321 | MA_API ma_bool32 ma_dr_wav_init_file_with_metadata(ma_dr_wav* pWav, const char* filename, ma_uint32
> flags, const ma_allocation_callbacks* pAllocationCallbacks);
62322 | MA_API ma_bool32 ma_dr_wav_init_file_with_metadata_w(ma_dr_wav* pWav, const wchar_t* filename,
> ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks);
62323 | MA_API ma_bool32 ma_dr_wav_init_file_write(ma_dr_wav* pWav, const char* filename, const

```



```

> ma_dr_wav_data_format* pFormat, const ma_allocation_callbacks* pAllocationCallbacks);
62324 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential(ma_dr_wav* pWav, const char* filename, const
> ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62325 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential_pcm_frames(ma_dr_wav* pWav, const char*
> filename, const ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62326 | MA_API ma_bool32 ma_dr_wav_init_file_write_w(ma_dr_wav* pWav, const wchar_t* filename, const
> ma_dr_wav_data_format* pFormat, const ma_allocation_callbacks* pAllocationCallbacks);
62327 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential_w(ma_dr_wav* pWav, const wchar_t* filename,
> const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62328 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential_pcm_frames_w(ma_dr_wav* pWav, const wchar_t*
> filename, const ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62329 | #endif
62330 | MA_API ma_bool32 ma_dr_wav_init_memory(ma_dr_wav* pWav, const void* data, size_t dataSize, const
> ma_allocation_callbacks* pAllocationCallbacks);
62331 | MA_API ma_bool32 ma_dr_wav_init_memory_ex(ma_dr_wav* pWav, const void* data, size_t dataSize,
> ma_dr_wav_chunk_proc onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks*
> pAllocationCallbacks);
62332 | MA_API ma_bool32 ma_dr_wav_init_memory_with_metadata(ma_dr_wav* pWav, const void* data, size_t
> dataSize, ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks);
62333 | MA_API ma_bool32 ma_dr_wav_init_memory_write(ma_dr_wav* pWav, void** ppData, size_t* pDataSize,
> const ma_dr_wav_data_format* pFormat, const ma_allocation_callbacks* pAllocationCallbacks);
62334 | MA_API ma_bool32 ma_dr_wav_init_memory_write_sequential(ma_dr_wav* pWav, void** ppData, size_t*
> pDataSize, const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62335 | MA_API ma_bool32 ma_dr_wav_init_memory_write_sequential_pcm_frames(ma_dr_wav* pWav, void** ppData,
> size_t* pDataSize, const ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62336 | #ifndef MA_DR_WAV_NO_CONVERSION_API
62337 | MA_API ma_int16* ma_dr_wav_open_and_read_pcm_frames_s16(ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, unsigned int* channelsOut,
> unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const ma_allocation_callbacks*
> pAllocationCallbacks);
62338 | MA_API float* ma_dr_wav_open_and_read_pcm_frames_f32(ma_dr_wav_read_proc onRead, ma_dr_wav_seek_proc
> onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, unsigned int* channelsOut, unsigned int*
> sampleRateOut, ma_uint64* totalFrameCountOut, const ma_allocation_callbacks* pAllocationCallbacks);
62339 | MA_API ma_int32* ma_dr_wav_open_and_read_pcm_frames_s32(ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, unsigned int* channelsOut,
> unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const ma_allocation_callbacks*
> pAllocationCallbacks);
62340 | #ifndef MA_DR_WAV_NO_STDIO
62341 | MA_API ma_int16* ma_dr_wav_open_file_and_read_pcm_frames_s16(const char* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62342 | MA_API float* ma_dr_wav_open_file_and_read_pcm_frames_f32(const char* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62343 | MA_API ma_int32* ma_dr_wav_open_file_and_read_pcm_frames_s32(const char* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62344 | MA_API ma_int16* ma_dr_wav_open_file_and_read_pcm_frames_s16_w(const wchar_t* filename, unsigned
> int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62345 | MA_API float* ma_dr_wav_open_file_and_read_pcm_frames_f32_w(const wchar_t* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62346 | MA_API ma_int32* ma_dr_wav_open_file_and_read_pcm_frames_s32_w(const wchar_t* filename, unsigned
> int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62347 | #endif
62348 | MA_API ma_int16* ma_dr_wav_open_memory_and_read_pcm_frames_s16(const void* data, size_t dataSize,
> unsigned int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62349 | MA_API float* ma_dr_wav_open_memory_and_read_pcm_frames_f32(const void* data, size_t dataSize,
> unsigned int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62350 | MA_API ma_int32* ma_dr_wav_open_memory_and_read_pcm_frames_s32(const void* data, size_t dataSize,
> unsigned int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks);
62351 | #endif
62352 | MA_API void ma_dr_wav_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks);
62353 | MA_API ma_uint16 ma_dr_wav_bytes_to_u16(const ma_uint8* data);
62354 | MA_API ma_int16 ma_dr_wav_bytes_to_s16(const ma_uint8* data);
62355 | MA_API ma_uint32 ma_dr_wav_bytes_to_u32(const ma_uint8* data);
62356 | MA_API ma_int32 ma_dr_wav_bytes_to_s32(const ma_uint8* data);
62357 | MA_API ma_uint64 ma_dr_wav_bytes_to_u64(const ma_uint8* data);

```



```

62358 | MA_API ma_int64 ma_dr_wav_bytes_to_s64(const ma_uint8* data);
62359 | MA_API float ma_dr_wav_bytes_to_f32(const ma_uint8* data);
62360 | MA_API ma_bool32 ma_dr_wav_guid_equal(const ma_uint8 a[16], const ma_uint8 b[16]);
62361 | MA_API ma_bool32 ma_dr_wav_fourcc_equal(const ma_uint8* a, const char* b);
62362 | #ifdef __cplusplus
62363 | }
62364 | #endif
62365 | #endif
62366 | /* dr_wav_h end */
62367 | #endif /* MA_NO_WAV */
62368 |
62369 | #if !defined(MA_NO_FLAC) && !defined(MA_NO_DECODING)
62370 | #define MA_HAS_FLAC
62371 |
62372 | /* dr_flac_h begin */
62373 | #ifndef ma_dr_flac_h
62374 | #define ma_dr_flac_h
62375 | #ifdef __cplusplus
62376 | extern "C" {
62377 | #endif
62378 | #define MA_DR_FLAC_STRINGIFY(x)      #x
62379 | #define MA_DR_FLAC_XSTRINGIFY(x)    MA_DR_FLAC_STRINGIFY(x)
62380 | #define MA_DR_FLAC_VERSION_MAJOR    0
62381 | #define MA_DR_FLAC_VERSION_MINOR    13
62382 | #define MA_DR_FLAC_VERSION_REVISION 3
62383 | #define MA_DR_FLAC_VERSION_STRING    MA_DR_FLAC_XSTRINGIFY(MA_DR_FLAC_VERSION_MAJOR) "."
> MA_DR_FLAC_XSTRINGIFY(MA_DR_FLAC_VERSION_MINOR) "."
> MA_DR_FLAC_XSTRINGIFY(MA_DR_FLAC_VERSION_REVISION)
62384 | #include <stddef.h>
62385 | #if defined(_MSC_VER) && _MSC_VER >= 1700
62386 |     #define MA_DR_FLAC_DEPRECATED    __declspec(deprecated)
62387 | #elif (defined(__GNUC__) && __GNUC__ >= 4)
62388 |     #define MA_DR_FLAC_DEPRECATED    __attribute__((deprecated))
62389 | #elif defined(__has_feature)
62390 |     #if __has_feature(attribute_deprecated)
62391 |         #define MA_DR_FLAC_DEPRECATED __attribute__((deprecated))
62392 |     #else
62393 |         #define MA_DR_FLAC_DEPRECATED
62394 |     #endif
62395 | #else
62396 |     #define MA_DR_FLAC_DEPRECATED
62397 | #endif
62398 | MA_API void ma_dr_flac_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision);
62399 | MA_API const char* ma_dr_flac_version_string(void);
62400 | #ifndef MA_DR_FLAC_BUFFER_SIZE
62401 | #define MA_DR_FLAC_BUFFER_SIZE      4096
62402 | #endif
62403 | #ifdef MA_64BIT
62404 | typedef ma_uint64 ma_dr_flac_cache_t;
62405 | #else
62406 | typedef ma_uint32 ma_dr_flac_cache_t;
62407 | #endif
62408 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_STREAMINFO    0
62409 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_PADDING      1
62410 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_APPLICATION  2
62411 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_SEEKTABLE    3
62412 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_VORBIS_COMMENT 4
62413 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_CUESHEET     5
62414 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_PICTURE      6
62415 | #define MA_DR_FLAC_METADATA_BLOCK_TYPE_INVALID      127
62416 | #define MA_DR_FLAC_PICTURE_TYPE_OTHER              0
62417 | #define MA_DR_FLAC_PICTURE_TYPE_FILE_ICON          1
62418 | #define MA_DR_FLAC_PICTURE_TYPE_OTHER_FILE_ICON    2
62419 | #define MA_DR_FLAC_PICTURE_TYPE_COVER_FRONT        3
62420 | #define MA_DR_FLAC_PICTURE_TYPE_COVER_BACK         4
62421 | #define MA_DR_FLAC_PICTURE_TYPE_LEAFLET_PAGE       5
62422 | #define MA_DR_FLAC_PICTURE_TYPE_MEDIA              6
62423 | #define MA_DR_FLAC_PICTURE_TYPE_LEAD_ARTIST        7
62424 | #define MA_DR_FLAC_PICTURE_TYPE_ARTIST             8
62425 | #define MA_DR_FLAC_PICTURE_TYPE_CONDUCTOR          9
62426 | #define MA_DR_FLAC_PICTURE_TYPE_BAND              10
62427 | #define MA_DR_FLAC_PICTURE_TYPE_COMPOSER           11
62428 | #define MA_DR_FLAC_PICTURE_TYPE_LYRICIST           12
62429 | #define MA_DR_FLAC_PICTURE_TYPE_RECORDING_LOCATION 13
62430 | #define MA_DR_FLAC_PICTURE_TYPE_DURING_RECORDING   14
62431 | #define MA_DR_FLAC_PICTURE_TYPE_DURING_PERFORMANCE 15
62432 | #define MA_DR_FLAC_PICTURE_TYPE_SCREEN_CAPTURE     16
62433 | #define MA_DR_FLAC_PICTURE_TYPE_BRIGHT_COLORED_FISH 17
62434 | #define MA_DR_FLAC_PICTURE_TYPE_ILLUSTRATION       18

```

```

62435 | #define MA_DR_FLAC_PICTURE_TYPE_BAND_LOGOTYPE          19
62436 | #define MA_DR_FLAC_PICTURE_TYPE_PUBLISHER_LOGOTYPE     20
62437 | typedef enum
62438 | {
62439 |     ma_dr_flac_container_native,
62440 |     ma_dr_flac_container_ogg,
62441 |     ma_dr_flac_container_unknown
62442 | } ma_dr_flac_container;
62443 | typedef enum
62444 | {
62445 |     MA_DR_FLAC_SEEK_SET,
62446 |     MA_DR_FLAC_SEEK_CUR,
62447 |     MA_DR_FLAC_SEEK_END
62448 | } ma_dr_flac_seek_origin;
62449 | typedef struct
62450 | {
62451 |     ma_uint64 firstPCFrame;
62452 |     ma_uint64 flacFrameOffset;
62453 |     ma_uint16 pcmFrameCount;
62454 | } ma_dr_flac_seekpoint;
62455 | typedef struct
62456 | {
62457 |     ma_uint16 minBlockSizeInPCMframes;
62458 |     ma_uint16 maxBlockSizeInPCMframes;
62459 |     ma_uint32 minFrameSizeInPCMframes;
62460 |     ma_uint32 maxFrameSizeInPCMframes;
62461 |     ma_uint32 sampleRate;
62462 |     ma_uint8  channels;
62463 |     ma_uint8  bitsPerSample;
62464 |     ma_uint64 totalPCFrameCount;
62465 |     ma_uint8  md5[16];
62466 | } ma_dr_flac_streaminfo;
62467 | typedef struct
62468 | {
62469 |     ma_uint32 type;
62470 |     ma_uint32 rawDataSize;
62471 |     ma_uint64 rawDataOffset;
62472 |     const void* pRawData;
62473 |     union
62474 |     {
62475 |         ma_dr_flac_streaminfo streaminfo;
62476 |         struct
62477 |         {
62478 |             int unused;
62479 |         } padding;
62480 |         struct
62481 |         {
62482 |             ma_uint32 id;
62483 |             const void* pData;
62484 |             ma_uint32 dataSize;
62485 |         } application;
62486 |         struct
62487 |         {
62488 |             ma_uint32 seekpointCount;
62489 |             const ma_dr_flac_seekpoint* pSeekpoints;
62490 |         } seektable;
62491 |         struct
62492 |         {
62493 |             ma_uint32 vendorLength;
62494 |             const char* vendor;
62495 |             ma_uint32 commentCount;
62496 |             const void* pComments;
62497 |         } vorbis_comment;
62498 |         struct
62499 |         {
62500 |             char catalog[128];
62501 |             ma_uint64 leadInSampleCount;
62502 |             ma_bool32 isCD;
62503 |             ma_uint8 trackCount;
62504 |             const void* pTrackData;
62505 |         } cuesheet;
62506 |         struct
62507 |         {
62508 |             ma_uint32 type;
62509 |             ma_uint32 mimeTypeLength;
62510 |             const char* mimeType;
62511 |             ma_uint32 descriptionLength;
62512 |             const char* description;
62513 |             ma_uint32 width;

```

```

62514 |         ma_uint32 height;
62515 |         ma_uint32 colorDepth;
62516 |         ma_uint32 indexColorCount;
62517 |         ma_uint32 pictureDataSize;
62518 |         ma_uint64 pictureDataOffset;
62519 |         const ma_uint8* pPictureData;
62520 |     } picture;
62521 | } data;
62522 | } ma_dr_flac_metadata;
62523 | typedef size_t (* ma_dr_flac_read_proc)(void* pUserData, void* pBufferOut, size_t bytesToRead);
62524 | typedef ma_bool32 (* ma_dr_flac_seek_proc)(void* pUserData, int offset, ma_dr_flac_seek_origin
> origin);
62525 | typedef ma_bool32 (* ma_dr_flac_tell_proc)(void* pUserData, ma_int64* pCursor);
62526 | typedef void (* ma_dr_flac_meta_proc)(void* pUserData, ma_dr_flac_metadata* pMetadata);
62527 | typedef struct
62528 | {
62529 |     const ma_uint8* data;
62530 |     size_t dataSize;
62531 |     size_t currentReadPos;
62532 | } ma_dr_flac__memory_stream;
62533 | typedef struct
62534 | {
62535 |     ma_dr_flac_read_proc onRead;
62536 |     ma_dr_flac_seek_proc onSeek;
62537 |     ma_dr_flac_tell_proc onTell;
62538 |     void* pUserData;
62539 |     size_t unalignedByteCount;
62540 |     ma_dr_flac_cache_t unalignedCache;
62541 |     ma_uint32 nextL2Line;
62542 |     ma_uint32 consumedBits;
62543 |     ma_dr_flac_cache_t cacheL2[MA_DR_FLAC_BUFFER_SIZE/sizeof(ma_dr_flac_cache_t)];
62544 |     ma_dr_flac_cache_t cache;
62545 |     ma_uint16 crc16;
62546 |     ma_dr_flac_cache_t crc16Cache;
62547 |     ma_uint32 crc16CacheIgnoredBytes;
62548 | } ma_dr_flac_bs;
62549 | typedef struct
62550 | {
62551 |     ma_uint8 subframeType;
62552 |     ma_uint8 wastedBitsPerSample;
62553 |     ma_uint8 lpcOrder;
62554 |     ma_int32* pSamplesS32;
62555 | } ma_dr_flac_subframe;
62556 | typedef struct
62557 | {
62558 |     ma_uint64 pcmFrameNumber;
62559 |     ma_uint32 flacFrameNumber;
62560 |     ma_uint32 sampleRate;
62561 |     ma_uint16 blockSizeInPCMFrames;
62562 |     ma_uint8 channelAssignment;
62563 |     ma_uint8 bitsPerSample;
62564 |     ma_uint8 crc8;
62565 | } ma_dr_flac_frame_header;
62566 | typedef struct
62567 | {
62568 |     ma_dr_flac_frame_header header;
62569 |     ma_uint32 pcmFramesRemaining;
62570 |     ma_dr_flac_subframe subframes[8];
62571 | } ma_dr_flac_frame;
62572 | typedef struct
62573 | {
62574 |     ma_dr_flac_meta_proc onMeta;
62575 |     void* pUserDataMD;
62576 |     ma_allocation_callbacks allocationCallbacks;
62577 |     ma_uint32 sampleRate;
62578 |     ma_uint8 channels;
62579 |     ma_uint8 bitsPerSample;
62580 |     ma_uint16 maxBlockSizeInPCMFrames;
62581 |     ma_uint64 totalPCMFrameCount;
62582 |     ma_dr_flac_container container;
62583 |     ma_uint32 seekpointCount;
62584 |     ma_dr_flac_frame currentFLACFrame;
62585 |     ma_uint64 currentPCMFrame;
62586 |     ma_uint64 firstFLACFramePosInBytes;
62587 |     ma_dr_flac__memory_stream memoryStream;
62588 |     ma_int32* pDecodedSamples;
62589 |     ma_dr_flac_seekpoint* pSeekpoints;
62590 |     void* _oggbs;
62591 |     ma_bool32 _noSeekTableSeek : 1;

```

```

62592 |     ma_bool32_noBinarySearchSeek : 1;
62593 |     ma_bool32_noBruteForceSeek  : 1;
62594 |     ma_dr_flac_bs bs;
62595 |     ma_uint8 pExtraData[1];
62596 | } ma_dr_flac;
62597 | MA_API ma_dr_flac* ma_dr_flac_open(ma_dr_flac_read_proc onRead, ma_dr_flac_seek_proc onSeek,
> ma_dr_flac_tell_proc onTell, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks);
62598 | MA_API ma_dr_flac* ma_dr_flac_open_relaxed(ma_dr_flac_read_proc onRead, ma_dr_flac_seek_proc onSeek,
> ma_dr_flac_tell_proc onTell, ma_dr_flac_container container, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks);
62599 | MA_API ma_dr_flac* ma_dr_flac_open_with_metadata(ma_dr_flac_read_proc onRead, ma_dr_flac_seek_proc
> onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks);
62600 | MA_API ma_dr_flac* ma_dr_flac_open_with_metadata_relaxed(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta,
> ma_dr_flac_container container, void* pUserData, const ma_allocation_callbacks*
> pAllocationCallbacks);
62601 | MA_API void ma_dr_flac_close(ma_dr_flac* pFlac);
62602 | MA_API ma_uint64 ma_dr_flac_read_pcm_frames_s32(ma_dr_flac* pFlac, ma_uint64 framesToRead, ma_int32*
> pBufferOut);
62603 | MA_API ma_uint64 ma_dr_flac_read_pcm_frames_s16(ma_dr_flac* pFlac, ma_uint64 framesToRead, ma_int16*
> pBufferOut);
62604 | MA_API ma_uint64 ma_dr_flac_read_pcm_frames_f32(ma_dr_flac* pFlac, ma_uint64 framesToRead, float*
> pBufferOut);
62605 | MA_API ma_bool32 ma_dr_flac_seek_to_pcm_frame(ma_dr_flac* pFlac, ma_uint64 pcmFrameIndex);
62606 | #ifndef MA_DR_FLAC_NO_STDIO
62607 | MA_API ma_dr_flac* ma_dr_flac_open_file(const char* pFileName, const ma_allocation_callbacks*
> pAllocationCallbacks);
62608 | MA_API ma_dr_flac* ma_dr_flac_open_file_w(const wchar_t* pFileName, const ma_allocation_callbacks*
> pAllocationCallbacks);
62609 | MA_API ma_dr_flac* ma_dr_flac_open_file_with_metadata(const char* pFileName, ma_dr_flac_meta_proc
> onMeta, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks);
62610 | MA_API ma_dr_flac* ma_dr_flac_open_file_with_metadata_w(const wchar_t* pFileName,
> ma_dr_flac_meta_proc onMeta, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks);
62611 | #endif
62612 | MA_API ma_dr_flac* ma_dr_flac_open_memory(const void* pData, size_t dataSize, const
> ma_allocation_callbacks* pAllocationCallbacks);
62613 | MA_API ma_dr_flac* ma_dr_flac_open_memory_with_metadata(const void* pData, size_t dataSize,
> ma_dr_flac_meta_proc onMeta, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks);
62614 | MA_API ma_int32* ma_dr_flac_open_and_read_pcm_frames_s32(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, void* pUserData, unsigned int* channels,
> unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62615 | MA_API ma_int16* ma_dr_flac_open_and_read_pcm_frames_s16(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, void* pUserData, unsigned int* channels,
> unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62616 | MA_API float* ma_dr_flac_open_and_read_pcm_frames_f32(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, void* pUserData, unsigned int* channels,
> unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62617 | #ifndef MA_DR_FLAC_NO_STDIO
62618 | MA_API ma_int32* ma_dr_flac_open_file_and_read_pcm_frames_s32(const char* filename, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62619 | MA_API ma_int16* ma_dr_flac_open_file_and_read_pcm_frames_s16(const char* filename, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62620 | MA_API float* ma_dr_flac_open_file_and_read_pcm_frames_f32(const char* filename, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62621 | #endif
62622 | MA_API ma_int32* ma_dr_flac_open_memory_and_read_pcm_frames_s32(const void* data, size_t dataSize,
> unsigned int* channels, unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62623 | MA_API ma_int16* ma_dr_flac_open_memory_and_read_pcm_frames_s16(const void* data, size_t dataSize,
> unsigned int* channels, unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62624 | MA_API float* ma_dr_flac_open_memory_and_read_pcm_frames_f32(const void* data, size_t dataSize,
> unsigned int* channels, unsigned int* sampleRate, ma_uint64* totalPCMFramCount, const
> ma_allocation_callbacks* pAllocationCallbacks);
62625 | MA_API void ma_dr_flac_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks);
62626 | typedef struct
62627 | {
62628 |     ma_uint32 countRemaining;
62629 |     const char* pRunningData;
62630 | } ma_dr_flac_vorbis_comment_iterator;
62631 | MA_API void ma_dr_flac_init_vorbis_comment_iterator(ma_dr_flac_vorbis_comment_iterator* pIter,
> ma_uint32 commentCount, const void* pComments);

```

```

62632 | MA_API const char* ma_dr_flac_next_vorbis_comment(ma_dr_flac_vorbis_comment_iterator* pIter,
> ma_uint32* pCommentLengthOut);
62633 | typedef struct
62634 | {
62635 |     ma_uint32 countRemaining;
62636 |     const char* pRunningData;
62637 | } ma_dr_flac_cuesheet_track_iterator;
62638 | typedef struct
62639 | {
62640 |     ma_uint64 offset;
62641 |     ma_uint8 index;
62642 |     ma_uint8 reserved[3];
62643 | } ma_dr_flac_cuesheet_track_index;
62644 | typedef struct
62645 | {
62646 |     ma_uint64 offset;
62647 |     ma_uint8 trackNumber;
62648 |     char ISRC[12];
62649 |     ma_bool8 isAudio;
62650 |     ma_bool8 preEmphasis;
62651 |     ma_uint8 indexCount;
62652 |     const ma_dr_flac_cuesheet_track_index* pIndexPoints;
62653 | } ma_dr_flac_cuesheet_track;
62654 | MA_API void ma_dr_flac_init_cuesheet_track_iterator(ma_dr_flac_cuesheet_track_iterator* pIter,
> ma_uint32 trackCount, const void* pTrackData);
62655 | MA_API ma_bool32 ma_dr_flac_next_cuesheet_track(ma_dr_flac_cuesheet_track_iterator* pIter,
> ma_dr_flac_cuesheet_track* pCuesheetTrack);
62656 | #ifdef __cplusplus
62657 | }
62658 | #endif
62659 | #endif
62660 | /* dr_flac_h end */
62661 | #endif /* MA_NO_FLAC */
62662 |
62663 | #if !defined(MA_NO_MP3) && !defined(MA_NO_DECODING)
62664 | #define MA_HAS_MP3
62665 |
62666 | #ifndef MA_DR_MP3_NO_SIMD
62667 |     #if (defined(MA_NO_NEON) && defined(MA_ARM)) || (defined(MA_NO_SSE2) && (defined(MA_X86) ||
> defined(MA_X64)))
62668 |         #define MA_DR_MP3_NO_SIMD
62669 |     #endif
62670 | #endif
62671 |
62672 | /* dr_mp3_h begin */
62673 | #ifndef ma_dr_mp3_h
62674 | #define ma_dr_mp3_h
62675 | #ifdef __cplusplus
62676 | extern "C" {
62677 | #endif
62678 | #define MA_DR_MP3_STRINGIFY(x)      #x
62679 | #define MA_DR_MP3_XSTRINGIFY(x)    MA_DR_MP3_STRINGIFY(x)
62680 | #define MA_DR_MP3_VERSION_MAJOR    0
62681 | #define MA_DR_MP3_VERSION_MINOR    7
62682 | #define MA_DR_MP3_VERSION_REVISION 3
62683 | #define MA_DR_MP3_VERSION_STRING    MA_DR_MP3_XSTRINGIFY(MA_DR_MP3_VERSION_MAJOR) "."
> MA_DR_MP3_XSTRINGIFY(MA_DR_MP3_VERSION_MINOR) "." MA_DR_MP3_XSTRINGIFY(MA_DR_MP3_VERSION_REVISION)
62684 | #include <stddef.h>
62685 | #define MA_DR_MP3_MAX_PCM_FRAMES_PER_MP3_FRAME 1152
62686 | #define MA_DR_MP3_MAX_SAMPLES_PER_FRAME        (MA_DR_MP3_MAX_PCM_FRAMES_PER_MP3_FRAME*2)
62687 | MA_API void ma_dr_mp3_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision);
62688 | MA_API const char* ma_dr_mp3_version_string(void);
62689 | #define MA_DR_MP3_MAX_BITRESERVOIR_BYTES      511
62690 | #define MA_DR_MP3_MAX_FREE_FORMAT_FRAME_SIZE  2304
62691 | #define MA_DR_MP3_MAX_L3_FRAME_PAYLOAD_BYTES  MA_DR_MP3_MAX_FREE_FORMAT_FRAME_SIZE
62692 | typedef struct
62693 | {
62694 |     int frame_bytes, channels, sample_rate, layer, bitrate_kbps;
62695 | } ma_dr_mp3dec_frame_info;
62696 | typedef struct
62697 | {
62698 |     const ma_uint8 *buf;
62699 |     int pos, limit;
62700 | } ma_dr_mp3_bs;
62701 | typedef struct
62702 | {
62703 |     const ma_uint8 *sfbtabs;
62704 |     ma_uint16 part_23_length, big_values, scalefac_compress;
62705 |     ma_uint8 global_gain, block_type, mixed_block_flag, n_long_sfb, n_short_sfb;

```

```

62706 |     ma_uint8 table_select[3], region_count[3], subblock_gain[3];
62707 |     ma_uint8 preflag, scalefac_scale, count1_table, scfsi;
62708 | } ma_dr_mp3_L3_gr_info;
62709 | typedef struct
62710 | {
62711 |     ma_dr_mp3_bs bs;
62712 |     ma_uint8 maindata[MA_DR_MP3_MAX_BITRESERVOIR_BYTES + MA_DR_MP3_MAX_L3_FRAME_PAYLOAD_BYTES];
62713 |     ma_dr_mp3_L3_gr_info gr_info[4];
62714 |     float grbuf[2][576], scf[40], syn[18 + 15][2*32];
62715 |     ma_uint8 ist_pos[2][39];
62716 | } ma_dr_mp3dec_scratch;
62717 | typedef struct
62718 | {
62719 |     float mdct_overlap[2][9*32], qmf_state[15*2*32];
62720 |     int reserv, free_format_bytes;
62721 |     ma_uint8 header[4], reserv_buf[511];
62722 |     ma_dr_mp3dec_scratch scratch;
62723 | } ma_dr_mp3dec;
62724 | MA_API void ma_dr_mp3dec_init(ma_dr_mp3dec *dec);
62725 | MA_API int ma_dr_mp3dec_decode_frame(ma_dr_mp3dec *dec, const ma_uint8 *mp3, int mp3_bytes, void
> *pcm, ma_dr_mp3dec_frame_info *info);
62726 | MA_API void ma_dr_mp3dec_f32_to_s16(const float *in, ma_int16 *out, size_t num_samples);
62727 | typedef enum
62728 | {
62729 |     MA_DR_MP3_SEEK_SET,
62730 |     MA_DR_MP3_SEEK_CUR,
62731 |     MA_DR_MP3_SEEK_END
62732 | } ma_dr_mp3_seek_origin;
62733 | typedef struct
62734 | {
62735 |     ma_uint64 seekPosInBytes;
62736 |     ma_uint64 pcmFrameIndex;
62737 |     ma_uint16 mp3FramesToDiscard;
62738 |     ma_uint16 pcmFramesToDiscard;
62739 | } ma_dr_mp3_seek_point;
62740 | typedef enum
62741 | {
62742 |     MA_DR_MP3_METADATA_TYPE_ID3V1,
62743 |     MA_DR_MP3_METADATA_TYPE_ID3V2,
62744 |     MA_DR_MP3_METADATA_TYPE_APE,
62745 |     MA_DR_MP3_METADATA_TYPE_XING,
62746 |     MA_DR_MP3_METADATA_TYPE_VBRI
62747 | } ma_dr_mp3_metadata_type;
62748 | typedef struct
62749 | {
62750 |     ma_dr_mp3_metadata_type type;
62751 |     const void* pRawData;
62752 |     size_t rawDataSize;
62753 | } ma_dr_mp3_metadata;
62754 | typedef size_t (* ma_dr_mp3_read_proc)(void* pUserData, void* pBufferOut, size_t bytesToRead);
62755 | typedef ma_bool32 (* ma_dr_mp3_seek_proc)(void* pUserData, int offset, ma_dr_mp3_seek_origin
> origin);
62756 | typedef ma_bool32 (* ma_dr_mp3_tell_proc)(void* pUserData, ma_int64* pCursor);
62757 | typedef void (* ma_dr_mp3_meta_proc)(void* pUserData, const ma_dr_mp3_metadata* pMetadata);
62758 | typedef struct
62759 | {
62760 |     ma_uint32 channels;
62761 |     ma_uint32 sampleRate;
62762 | } ma_dr_mp3_config;
62763 | typedef struct
62764 | {
62765 |     ma_dr_mp3dec decoder;
62766 |     ma_uint32 channels;
62767 |     ma_uint32 sampleRate;
62768 |     ma_dr_mp3_read_proc onRead;
62769 |     ma_dr_mp3_seek_proc onSeek;
62770 |     ma_dr_mp3_meta_proc onMeta;
62771 |     void* pUserData;
62772 |     void* pUserDataMeta;
62773 |     ma_allocation_callbacks allocationCallbacks;
62774 |     ma_uint32 mp3FrameChannels;
62775 |     ma_uint32 mp3FrameSampleRate;
62776 |     ma_uint32 pcmFramesConsumedInMP3Frame;
62777 |     ma_uint32 pcmFramesRemainingInMP3Frame;
62778 |     ma_uint8 pcmFrames[sizeof(float)*MA_DR_MP3_MAX_SAMPLES_PER_FRAME];
62779 |     ma_uint64 currentPCMFrame;
62780 |     ma_uint64 streamCursor;
62781 |     ma_uint64 streamLength;
62782 |     ma_uint64 streamStartOffset;

```



```

62783 |     ma_dr_mp3_seek_point* pSeekPoints;
62784 |     ma_uint32 seekPointCount;
62785 |     ma_uint32 delayInPCMFrames;
62786 |     ma_uint32 paddingInPCMFrames;
62787 |     ma_uint64 totalPCMFrameCount;
62788 |     ma_bool32 isVBR;
62789 |     ma_bool32 isCBR;
62790 |     size_t dataSize;
62791 |     size_t dataCapacity;
62792 |     size_t dataConsumed;
62793 |     ma_uint8* pData;
62794 |     ma_bool32 atEnd;
62795 |     struct
62796 |     {
62797 |         const ma_uint8* pData;
62798 |         size_t dataSize;
62799 |         size_t currentReadPos;
62800 |     } memory;
62801 | } ma_dr_mp3;
62802 | MA_API ma_bool32 ma_dr_mp3_init(ma_dr_mp3* pMP3, ma_dr_mp3_read_proc onRead, ma_dr_mp3_seek_proc
> onSeek, ma_dr_mp3_tell_proc onTell, ma_dr_mp3_meta_proc onMeta, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks);
62803 | MA_API ma_bool32 ma_dr_mp3_init_memory_with_metadata(ma_dr_mp3* pMP3, const void* pData, size_t
> dataSize, ma_dr_mp3_meta_proc onMeta, void* pUserDataMeta, const ma_allocation_callbacks*
> pAllocationCallbacks);
62804 | MA_API ma_bool32 ma_dr_mp3_init_memory(ma_dr_mp3* pMP3, const void* pData, size_t dataSize, const
> ma_allocation_callbacks* pAllocationCallbacks);
62805 | #ifndef MA_DR_MP3_NO_STDIO
62806 | MA_API ma_bool32 ma_dr_mp3_init_file_with_metadata(ma_dr_mp3* pMP3, const char* filePath,
> ma_dr_mp3_meta_proc onMeta, void* pUserDataMeta, const ma_allocation_callbacks*
> pAllocationCallbacks);
62807 | MA_API ma_bool32 ma_dr_mp3_init_file_with_metadata_w(ma_dr_mp3* pMP3, const wchar_t* filePath,
> ma_dr_mp3_meta_proc onMeta, void* pUserDataMeta, const ma_allocation_callbacks*
> pAllocationCallbacks);
62808 | MA_API ma_bool32 ma_dr_mp3_init_file(ma_dr_mp3* pMP3, const char* filePath, const
> ma_allocation_callbacks* pAllocationCallbacks);
62809 | MA_API ma_bool32 ma_dr_mp3_init_file_w(ma_dr_mp3* pMP3, const wchar_t* filePath, const
> ma_allocation_callbacks* pAllocationCallbacks);
62810 | #endif
62811 | MA_API void ma_dr_mp3_uninit(ma_dr_mp3* pMP3);
62812 | MA_API ma_uint64 ma_dr_mp3_read_pcm_frames_f32(ma_dr_mp3* pMP3, ma_uint64 framesToRead, float*
> pBufferOut);
62813 | MA_API ma_uint64 ma_dr_mp3_read_pcm_frames_s16(ma_dr_mp3* pMP3, ma_uint64 framesToRead, ma_int16*
> pBufferOut);
62814 | MA_API ma_bool32 ma_dr_mp3_seek_to_pcm_frame(ma_dr_mp3* pMP3, ma_uint64 frameIndex);
62815 | MA_API ma_uint64 ma_dr_mp3_get_pcm_frame_count(ma_dr_mp3* pMP3);
62816 | MA_API ma_uint64 ma_dr_mp3_get_mp3_frame_count(ma_dr_mp3* pMP3);
62817 | MA_API ma_bool32 ma_dr_mp3_get_mp3_and_pcm_frame_count(ma_dr_mp3* pMP3, ma_uint64* pMP3FrameCount,
> ma_uint64* pPCMFrameCount);
62818 | MA_API ma_bool32 ma_dr_mp3_calculate_seek_points(ma_dr_mp3* pMP3, ma_uint32* pSeekPointCount,
> ma_dr_mp3_seek_point* pSeekPoints);
62819 | MA_API ma_bool32 ma_dr_mp3_bind_seek_table(ma_dr_mp3* pMP3, ma_uint32 seekPointCount,
> ma_dr_mp3_seek_point* pSeekPoints);
62820 | MA_API float* ma_dr_mp3_open_and_read_pcm_frames_f32(ma_dr_mp3_read_proc onRead, ma_dr_mp3_seek_proc
> onSeek, ma_dr_mp3_tell_proc onTell, void* pUserData, ma_dr_mp3_config* pConfig, ma_uint64*
> pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks);
62821 | MA_API ma_int16* ma_dr_mp3_open_and_read_pcm_frames_s16(ma_dr_mp3_read_proc onRead,
> ma_dr_mp3_seek_proc onSeek, ma_dr_mp3_tell_proc onTell, void* pUserData, ma_dr_mp3_config* pConfig,
> ma_uint64* pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks);
62822 | MA_API float* ma_dr_mp3_open_memory_and_read_pcm_frames_f32(const void* pData, size_t dataSize,
> ma_dr_mp3_config* pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62823 | MA_API ma_int16* ma_dr_mp3_open_memory_and_read_pcm_frames_s16(const void* pData, size_t dataSize,
> ma_dr_mp3_config* pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks);
62824 | #ifndef MA_DR_MP3_NO_STDIO
62825 | MA_API float* ma_dr_mp3_open_file_and_read_pcm_frames_f32(const char* filePath, ma_dr_mp3_config*
> pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks);
62826 | MA_API ma_int16* ma_dr_mp3_open_file_and_read_pcm_frames_s16(const char* filePath, ma_dr_mp3_config*
> pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks);
62827 | #endif
62828 | MA_API void* ma_dr_mp3_malloc(size_t sz, const ma_allocation_callbacks* pAllocationCallbacks);
62829 | MA_API void ma_dr_mp3_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks);
62830 | #ifdef __cplusplus
62831 | }
62832 | #endif
62833 | #endif
62834 | /* dr_mp3_h end */
62835 | #endif /* MA_NO_MP3 */

```



```

62836 |
62837 |
62838 | /*****
> *****/
62839 |
62840 | Decoding
62841 |
62842 | *****/
> *****/
62843 | #ifndef MA_NO_DECODING
62844 |
62845 | static ma_result ma_decoder_read_bytes(ma_decoder* pDecoder, void* pBufferOut, size_t bytesToRead,
> size_t* pBytesRead)
62846 | {
62847 |     MA_ASSERT(pDecoder != NULL);
62848 |
62849 |     return pDecoder->onRead(pDecoder, pBufferOut, bytesToRead, pBytesRead);
62850 | }
62851 |
62852 | static ma_result ma_decoder_seek_bytes(ma_decoder* pDecoder, ma_int64 byteOffset, ma_seek_origin
> origin)
62853 | {
62854 |     MA_ASSERT(pDecoder != NULL);
62855 |
62856 |     return pDecoder->onSeek(pDecoder, byteOffset, origin);
62857 | }
62858 |
62859 | static ma_result ma_decoder_tell_bytes(ma_decoder* pDecoder, ma_int64* pCursor)
62860 | {
62861 |     MA_ASSERT(pDecoder != NULL);
62862 |
62863 |     if (pDecoder->onTell == NULL) {
62864 |         return MA_NOT_IMPLEMENTED;
62865 |     }
62866 |
62867 |     return pDecoder->onTell(pDecoder, pCursor);
62868 | }
62869 |
62870 |
62871 | MA_API ma_decoding_backend_config ma_decoding_backend_config_init(ma_format preferredFormat,
> ma_uint32 seekPointCount)
62872 | {
62873 |     ma_decoding_backend_config config;
62874 |
62875 |     MA_ZERO_OBJECT(&config);
62876 |     config.preferredFormat = preferredFormat;
62877 |     config.seekPointCount = seekPointCount;
62878 |
62879 |     return config;
62880 | }
62881 |
62882 |
62883 | MA_API ma_decoder_config ma_decoder_config_init(ma_format outputFormat, ma_uint32 outputChannels,
> ma_uint32 outputSampleRate)
62884 | {
62885 |     ma_decoder_config config;
62886 |     MA_ZERO_OBJECT(&config);
62887 |     config.format = outputFormat;
62888 |     config.channels = outputChannels;
62889 |     config.sampleRate = outputSampleRate;
62890 |     config.resampling = ma_resampler_config_init(ma_format_unknown, 0, 0, 0,
> ma_resample_algorithm_linear); /* Format/channels/rate doesn't matter here. */
62891 |     config.encodingFormat = ma_encoding_format_unknown;
62892 |
62893 |     /* Note that we are intentionally leaving the channel map empty here which will cause the
> default channel map to be used. */
62894 |
62895 |     return config;
62896 | }
62897 |
62898 | MA_API ma_decoder_config ma_decoder_config_init_default(void)
62899 | {
62900 |     return ma_decoder_config_init(ma_format_unknown, 0, 0);
62901 | }
62902 |
62903 | MA_API ma_decoder_config ma_decoder_config_init_copy(const ma_decoder_config* pConfig)
62904 | {
62905 |     ma_decoder_config config;
62906 |     if (pConfig != NULL) {

```

```

62907 |         config = *pConfig;
62908 |     } else {
62909 |         MA_ZERO_OBJECT(&config);
62910 |     }
62911 |
62912 |     return config;
62913 | }
62914 |
62915 | static ma_result ma_decoder__init_data_converter(ma_decoder* pDecoder, const ma_decoder_config*
> pConfig)
62916 | {
62917 |     ma_result result;
62918 |     ma_data_converter_config converterConfig;
62919 |     ma_format internalFormat;
62920 |     ma_uint32 internalChannels;
62921 |     ma_uint32 internalSampleRate;
62922 |     ma_channel internalChannelMap[MA_MAX_CHANNELS];
62923 |
62924 |     MA_ASSERT(pDecoder != NULL);
62925 |     MA_ASSERT(pConfig != NULL);
62926 |
62927 |     result = ma_data_source_get_data_format(pDecoder->pBackend, &internalFormat, &internalChannels,
> &internalSampleRate, internalChannelMap, ma_countof(internalChannelMap));
62928 |     if (result != MA_SUCCESS) {
62929 |         return result; /* Failed to retrieve the internal data format. */
62930 |     }
62931 |
62932 |
62933 |     /* Make sure we're not asking for too many channels. */
62934 |     if (pConfig->channels > MA_MAX_CHANNELS) {
62935 |         return MA_INVALID_ARGS;
62936 |     }
62937 |
62938 |     /* The internal channels should have already been validated at a higher level, but we'll do it
> again explicitly here for safety. */
62939 |     if (internalChannels > MA_MAX_CHANNELS) {
62940 |         return MA_INVALID_ARGS;
62941 |     }
62942 |
62943 |
62944 |     /* Output format. */
62945 |     if (pConfig->format == ma_format_unknown) {
62946 |         pDecoder->outputFormat = internalFormat;
62947 |     } else {
62948 |         pDecoder->outputFormat = pConfig->format;
62949 |     }
62950 |
62951 |     if (pConfig->channels == 0) {
62952 |         pDecoder->outputChannels = internalChannels;
62953 |     } else {
62954 |         pDecoder->outputChannels = pConfig->channels;
62955 |     }
62956 |
62957 |     if (pConfig->sampleRate == 0) {
62958 |         pDecoder->outputSampleRate = internalSampleRate;
62959 |     } else {
62960 |         pDecoder->outputSampleRate = pConfig->sampleRate;
62961 |     }
62962 |
62963 |     converterConfig = ma_data_converter_config_init(
62964 |         internalFormat, pDecoder->outputFormat,
62965 |         internalChannels, pDecoder->outputChannels,
62966 |         internalSampleRate, pDecoder->outputSampleRate
62967 |     );
62968 |     converterConfig.pChannelMapIn = internalChannelMap;
62969 |     converterConfig.pChannelMapOut = pConfig->pChannelMap;
62970 |     converterConfig.channelMixMode = pConfig->channelMixMode;
62971 |     converterConfig.ditherMode = pConfig->ditherMode;
62972 |     converterConfig.allowDynamicSampleRate = MA_FALSE; /* Never allow dynamic sample rate
> conversion. Setting this to true will disable passthrough optimizations. */
62973 |     converterConfig.resampling = pConfig->resampling;
62974 |
62975 |     result = ma_data_converter_init(&converterConfig, &pDecoder->allocationCallbacks,
> &pDecoder->converter);
62976 |     if (result != MA_SUCCESS) {
62977 |         return result;
62978 |     }
62979 |
62980 |     /*

```

```

62981 |     Now that we have the decoder we need to determine whether or not we need a heap-allocated cache.
> | We'll
62982 |     need this if the data converter does not support calculation of the required input frame count.
> | To
62983 |     determine support for this we'll just run a test.
62984 |     */
62985 |     {
62986 |         ma_uint64 unused;
62987 |
62988 |         result = ma_data_converter_get_required_input_frame_count(&pDecoder->converter, 1, &unused);
62989 |         if (result != MA_SUCCESS) {
62990 |             /*
62991 |              We were unable to calculate the required input frame count which means we'll need to use
62992 |              a heap-allocated cache.
62993 |              */
62994 |             ma_uint64 inputCacheCapSizeInBytes;
62995 |
62996 |             pDecoder->inputCacheCap = MA_DATA_CONVERTER_STACK_BUFFER_SIZE /
> | ma_get_bytes_per_frame(internalFormat, internalChannels);
62997 |
62998 |             /* Not strictly necessary, but keeping here for safety in case we change the default
> | value of pDecoder->inputCacheCap. */
62999 |             inputCacheCapSizeInBytes = pDecoder->inputCacheCap *
> | ma_get_bytes_per_frame(internalFormat, internalChannels);
63000 |             if (inputCacheCapSizeInBytes > MA_SIZE_MAX) {
63001 |                 ma_data_converter_uninit(&pDecoder->converter, &pDecoder->allocationCallbacks);
63002 |                 return MA_OUT_OF_MEMORY;
63003 |             }
63004 |
63005 |             pDecoder->pInputCache = ma_malloc((size_t)inputCacheCapSizeInBytes,
> | &pDecoder->allocationCallbacks); /* Safe cast to size_t. */
63006 |             if (pDecoder->pInputCache == NULL) {
63007 |                 ma_data_converter_uninit(&pDecoder->converter, &pDecoder->allocationCallbacks);
63008 |                 return MA_OUT_OF_MEMORY;
63009 |             }
63010 |         }
63011 |     }
63012 |
63013 |     return MA_SUCCESS;
63014 | }
63015 |
63016 |
63017 |
63018 | static ma_result ma_decoder_internal_on_read__custom(void* pUserData, void* pBufferOut, size_t
> | bytesToRead, size_t* pBytesRead)
63019 | {
63020 |     ma_decoder* pDecoder = (ma_decoder*)pUserData;
63021 |     MA_ASSERT(pDecoder != NULL);
63022 |
63023 |     return ma_decoder_read_bytes(pDecoder, pBufferOut, bytesToRead, pBytesRead);
63024 | }
63025 |
63026 | static ma_result ma_decoder_internal_on_seek__custom(void* pUserData, ma_int64 offset,
> | ma_seek_origin origin)
63027 | {
63028 |     ma_decoder* pDecoder = (ma_decoder*)pUserData;
63029 |     MA_ASSERT(pDecoder != NULL);
63030 |
63031 |     return ma_decoder_seek_bytes(pDecoder, offset, origin);
63032 | }
63033 |
63034 | static ma_result ma_decoder_internal_on_tell__custom(void* pUserData, ma_int64* pCursor)
63035 | {
63036 |     ma_decoder* pDecoder = (ma_decoder*)pUserData;
63037 |     MA_ASSERT(pDecoder != NULL);
63038 |
63039 |     return ma_decoder_tell_bytes(pDecoder, pCursor);
63040 | }
63041 |
63042 |
63043 | static ma_result ma_decoder_init_from_vtable__internal(const ma_decoding_backend_vtable* pVTable,
> | void* pVTableUserData, const ma_decoder_config* pConfig, ma_decoder* pDecoder)
63044 | {
63045 |     ma_result result;
63046 |     ma_decoding_backend_config backendConfig;
63047 |     ma_data_source* pBackend;
63048 |
63049 |     MA_ASSERT(pVTable != NULL);
63050 |     MA_ASSERT(pConfig != NULL);

```

```

63051 |     MA_ASSERT(pDecoder != NULL);
63052 |
63053 |     if (pVTable->onInit == NULL) {
63054 |         return MA_NOT_IMPLEMENTED;
63055 |     }
63056 |
63057 |     backendConfig = ma_decoding_backend_config_init(pConfig->format, pConfig->seekPointCount);
63058 |
63059 |     result = pVTable->onInit(pVTableUserData, ma_decoder_internal_on_read_custom,
> ma_decoder_internal_on_seek_custom, ma_decoder_internal_on_tell_custom, pDecoder, &backendConfig,
> &pDecoder->allocationCallbacks, &pBackend);
63060 |     if (result != MA_SUCCESS) {
63061 |         return result; /* Failed to initialize the backend from this vtable. */
63062 |     }
63063 |
63064 |     /* Getting here means we were able to initialize the backend so we can now initialize the
> decoder. */
63065 |     pDecoder->pBackend = pBackend;
63066 |     pDecoder->pBackendVTable = pVTable;
63067 |     pDecoder->pBackendUserData = pConfig->pCustomBackendUserData;
63068 |
63069 |     return MA_SUCCESS;
63070 | }
63071 |
63072 | static ma_result ma_decoder_init_from_file__internal(const ma_decoding_backend_vtable* pVTable,
> void* pVTableUserData, const char* pFilePath, const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
63073 | {
63074 |     ma_result result;
63075 |     ma_decoding_backend_config backendConfig;
63076 |     ma_data_source* pBackend;
63077 |
63078 |     MA_ASSERT(pVTable != NULL);
63079 |     MA_ASSERT(pConfig != NULL);
63080 |     MA_ASSERT(pDecoder != NULL);
63081 |
63082 |     if (pVTable->onInitFile == NULL) {
63083 |         return MA_NOT_IMPLEMENTED;
63084 |     }
63085 |
63086 |     backendConfig = ma_decoding_backend_config_init(pConfig->format, pConfig->seekPointCount);
63087 |
63088 |     result = pVTable->onInitFile(pVTableUserData, pFilePath, &backendConfig,
> &pDecoder->allocationCallbacks, &pBackend);
63089 |     if (result != MA_SUCCESS) {
63090 |         return result; /* Failed to initialize the backend from this vtable. */
63091 |     }
63092 |
63093 |     /* Getting here means we were able to initialize the backend so we can now initialize the
> decoder. */
63094 |     pDecoder->pBackend = pBackend;
63095 |     pDecoder->pBackendVTable = pVTable;
63096 |     pDecoder->pBackendUserData = pConfig->pCustomBackendUserData;
63097 |
63098 |     return MA_SUCCESS;
63099 | }
63100 |
63101 | static ma_result ma_decoder_init_from_file_w__internal(const ma_decoding_backend_vtable* pVTable,
> void* pVTableUserData, const wchar_t* pFilePath, const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
63102 | {
63103 |     ma_result result;
63104 |     ma_decoding_backend_config backendConfig;
63105 |     ma_data_source* pBackend;
63106 |
63107 |     MA_ASSERT(pVTable != NULL);
63108 |     MA_ASSERT(pConfig != NULL);
63109 |     MA_ASSERT(pDecoder != NULL);
63110 |
63111 |     if (pVTable->onInitFileW == NULL) {
63112 |         return MA_NOT_IMPLEMENTED;
63113 |     }
63114 |
63115 |     backendConfig = ma_decoding_backend_config_init(pConfig->format, pConfig->seekPointCount);
63116 |
63117 |     result = pVTable->onInitFileW(pVTableUserData, pFilePath, &backendConfig,
> &pDecoder->allocationCallbacks, &pBackend);
63118 |     if (result != MA_SUCCESS) {
63119 |         return result; /* Failed to initialize the backend from this vtable. */

```

```

63120 |     }
63121 |
63122 |     /* Getting here means we were able to initialize the backend so we can now initialize the
> decoder. */
63123 |     pDecoder->pBackend      = pBackend;
63124 |     pDecoder->pBackendVTable = pVTable;
63125 |     pDecoder->pBackendUserData = pConfig->pCustomBackendUserData;
63126 |
63127 |     return MA_SUCCESS;
63128 | }
63129 |
63130 | static ma_result ma_decoder_init_from_memory__internal(const ma_decoding_backend_vtable* pVTable,
> void* pVTableUserData, const void* pData, size_t dataSize, const ma_decoder_config* pConfig,
> ma_decoder* pDecoder)
63131 | {
63132 |     ma_result result;
63133 |     ma_decoding_backend_config backendConfig;
63134 |     ma_data_source* pBackend;
63135 |
63136 |     MA_ASSERT(pVTable != NULL);
63137 |     MA_ASSERT(pConfig != NULL);
63138 |     MA_ASSERT(pDecoder != NULL);
63139 |
63140 |     if (pVTable->onInitMemory == NULL) {
63141 |         return MA_NOT_IMPLEMENTED;
63142 |     }
63143 |
63144 |     backendConfig = ma_decoding_backend_config_init(pConfig->format, pConfig->seekPointCount);
63145 |
63146 |     result = pVTable->onInitMemory(pVTableUserData, pData, dataSize, &backendConfig,
> &pDecoder->allocationCallbacks, &pBackend);
63147 |     if (result != MA_SUCCESS) {
63148 |         return result; /* Failed to initialize the backend from this vtable. */
63149 |     }
63150 |
63151 |     /* Getting here means we were able to initialize the backend so we can now initialize the
> decoder. */
63152 |     pDecoder->pBackend      = pBackend;
63153 |     pDecoder->pBackendVTable = pVTable;
63154 |     pDecoder->pBackendUserData = pConfig->pCustomBackendUserData;
63155 |
63156 |     return MA_SUCCESS;
63157 | }
63158 |
63159 |
63160 |
63161 | static ma_result ma_decoder_init_custom__internal(const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
63162 | {
63163 |     ma_result result = MA_NO_BACKEND;
63164 |     size_t ivtable;
63165 |
63166 |     MA_ASSERT(pConfig != NULL);
63167 |     MA_ASSERT(pDecoder != NULL);
63168 |
63169 |     if (pConfig->ppCustomBackendVTables == NULL) {
63170 |         return MA_NO_BACKEND;
63171 |     }
63172 |
63173 |     /* The order each backend is listed is what defines the priority. */
63174 |     for (ivtable = 0; ivtable < pConfig->customBackendCount; ivtable += 1) {
63175 |         const ma_decoding_backend_vtable* pVTable = pConfig->ppCustomBackendVTables[ivtable];
63176 |         if (pVTable != NULL) {
63177 |             result = ma_decoder_init_from_vtable__internal(pVTable, pConfig->pCustomBackendUserData,
> pConfig, pDecoder);
63178 |             if (result == MA_SUCCESS) {
63179 |                 return MA_SUCCESS;
63180 |             } else {
63181 |                 /* Initialization failed. Move on to the next one, but seek back to the start first
> so the next vtable starts from the first byte of the file. */
63182 |                 result = ma_decoder_seek_bytes(pDecoder, 0, ma_seek_origin_start);
63183 |                 if (result != MA_SUCCESS) {
63184 |                     return result; /* Failed to seek back to the start. */
63185 |                 }
63186 |             }
63187 |         } else {
63188 |             /* No vtable. */
63189 |         }
63190 |     }

```

```

63191 |
63192 |     /* Getting here means we couldn't find a backend. */
63193 |     return MA_NO_BACKEND;
63194 | }
63195 |
63196 | static ma_result ma_decoder_init_custom_from_file__internal(const char* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
63197 | {
63198 |     ma_result result = MA_NO_BACKEND;
63199 |     size_t ivtable;
63200 |
63201 |     MA_ASSERT(pConfig != NULL);
63202 |     MA_ASSERT(pDecoder != NULL);
63203 |
63204 |     if (pConfig->ppCustomBackendVTables == NULL) {
63205 |         return MA_NO_BACKEND;
63206 |     }
63207 |
63208 |     /* The order each backend is listed is what defines the priority. */
63209 |     for (ivtable = 0; ivtable < pConfig->customBackendCount; ivtable += 1) {
63210 |         const ma_decoding_backend_vtable* pVTable = pConfig->ppCustomBackendVTables[ivtable];
63211 |         if (pVTable != NULL) {
63212 |             result = ma_decoder_init_from_file__internal(pVTable, pConfig->pCustomBackendUserData,
> pFilePath, pConfig, pDecoder);
63213 |             if (result == MA_SUCCESS) {
63214 |                 return MA_SUCCESS;
63215 |             }
63216 |             } else {
63217 |                 /* No vtable. */
63218 |             }
63219 |         }
63220 |
63221 |     /* Getting here means we couldn't find a backend. */
63222 |     return MA_NO_BACKEND;
63223 | }
63224 |
63225 | static ma_result ma_decoder_init_custom_from_file_w__internal(const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
63226 | {
63227 |     ma_result result = MA_NO_BACKEND;
63228 |     size_t ivtable;
63229 |
63230 |     MA_ASSERT(pConfig != NULL);
63231 |     MA_ASSERT(pDecoder != NULL);
63232 |
63233 |     if (pConfig->ppCustomBackendVTables == NULL) {
63234 |         return MA_NO_BACKEND;
63235 |     }
63236 |
63237 |     /* The order each backend is listed is what defines the priority. */
63238 |     for (ivtable = 0; ivtable < pConfig->customBackendCount; ivtable += 1) {
63239 |         const ma_decoding_backend_vtable* pVTable = pConfig->ppCustomBackendVTables[ivtable];
63240 |         if (pVTable != NULL) {
63241 |             result = ma_decoder_init_from_file_w__internal(pVTable, pConfig->pCustomBackendUserData,
> pFilePath, pConfig, pDecoder);
63242 |             if (result == MA_SUCCESS) {
63243 |                 return MA_SUCCESS;
63244 |             }
63245 |             } else {
63246 |                 /* No vtable. */
63247 |             }
63248 |         }
63249 |
63250 |     /* Getting here means we couldn't find a backend. */
63251 |     return MA_NO_BACKEND;
63252 | }
63253 |
63254 | static ma_result ma_decoder_init_custom_from_memory__internal(const void* pData, size_t dataSize,
> const ma_decoder_config* pConfig, ma_decoder* pDecoder)
63255 | {
63256 |     ma_result result = MA_NO_BACKEND;
63257 |     size_t ivtable;
63258 |
63259 |     MA_ASSERT(pConfig != NULL);
63260 |     MA_ASSERT(pDecoder != NULL);
63261 |
63262 |     if (pConfig->ppCustomBackendVTables == NULL) {
63263 |         return MA_NO_BACKEND;
63264 |     }

```

```

63265 |
63266 |     /* The order each backend is listed is what defines the priority. */
63267 |     for (ivtable = 0; ivtable < pConfig->customBackendCount; ivtable += 1) {
63268 |         const ma_decoding_backend_vtable* pVTable = pConfig->ppCustomBackendVTables[ivtable];
63269 |         if (pVTable != NULL) {
63270 |             result = ma_decoder_init_from_memory__internal(pVTable, pConfig->pCustomBackendUserData,
> pData, dataSize, pConfig, pDecoder);
63271 |             if (result == MA_SUCCESS) {
63272 |                 return MA_SUCCESS;
63273 |             }
63274 |         } else {
63275 |             /* No vtable. */
63276 |         }
63277 |     }
63278 |
63279 |     /* Getting here means we couldn't find a backend. */
63280 |     return MA_NO_BACKEND;
63281 | }
63282 |
63283 |
63284 | /* WAV */
63285 | #ifdef ma_dr_wav_h
63286 |
63287 | typedef struct
63288 | {
63289 |     ma_data_source_base ds;
63290 |     ma_read_proc onRead;
63291 |     ma_seek_proc onSeek;
63292 |     ma_tell_proc onTell;
63293 |     void* pReadSeekTellUserData;
63294 |     ma_format format; /* Can be f32, s16 or s32. */
63295 | #if !defined(MA_NO_WAV)
63296 |     ma_dr_wav dr;
63297 | #endif
63298 | } ma_wav;
63299 |
63300 | MA_API ma_result ma_wav_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell, void*
> pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_wav* pwav);
63301 | MA_API ma_result ma_wav_init_file(const char* pFilePath, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_wav* pwav);
63302 | MA_API ma_result ma_wav_init_file_w(const wchar_t* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_wav* pwav);
63303 | MA_API ma_result ma_wav_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_wav*
> pwav);
63304 | MA_API void ma_wav_uninit(ma_wav* pwav, const ma_allocation_callbacks* pAllocationCallbacks);
63305 | MA_API ma_result ma_wav_read_pcm_frames(ma_wav* pwav, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead);
63306 | MA_API ma_result ma_wav_seek_to_pcm_frame(ma_wav* pwav, ma_uint64 frameIndex);
63307 | MA_API ma_result ma_wav_get_data_format(ma_wav* pwav, ma_format* pFormat, ma_uint32* pChannels,
> ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
63308 | MA_API ma_result ma_wav_get_cursor_in_pcm_frames(ma_wav* pwav, ma_uint64* pCursor);
63309 | MA_API ma_result ma_wav_get_length_in_pcm_frames(ma_wav* pwav, ma_uint64* pLength);
63310 |
63311 |
63312 | static ma_result ma_wav_ds_read(ma_data_source* pDataSource, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead)
63313 | {
63314 |     return ma_wav_read_pcm_frames((ma_wav*)pDataSource, pFramesOut, frameCount, pFramesRead);
63315 | }
63316 |
63317 | static ma_result ma_wav_ds_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
63318 | {
63319 |     return ma_wav_seek_to_pcm_frame((ma_wav*)pDataSource, frameIndex);
63320 | }
63321 |
63322 | static ma_result ma_wav_ds_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
63323 | {
63324 |     return ma_wav_get_data_format((ma_wav*)pDataSource, pFormat, pChannels, pSampleRate,
> pChannelMap, channelMapCap);
63325 | }
63326 |
63327 | static ma_result ma_wav_ds_get_cursor(ma_data_source* pDataSource, ma_uint64* pCursor)
63328 | {
63329 |     return ma_wav_get_cursor_in_pcm_frames((ma_wav*)pDataSource, pCursor);
63330 | }
63331 |

```



```

63332 | static ma_result ma_wav_ds_get_length(ma_data_source* pDataSource, ma_uint64* pLength)
63333 | {
63334 |     return ma_wav_get_length_in_pcm_frames((ma_wav*)pDataSource, pLength);
63335 | }
63336 |
63337 | static ma_data_source_vtable g_ma_wav_ds_vtable =
63338 | {
63339 |     ma_wav_ds_read,
63340 |     ma_wav_ds_seek,
63341 |     ma_wav_ds_get_data_format,
63342 |     ma_wav_ds_get_cursor,
63343 |     ma_wav_ds_get_length,
63344 |     NULL, /* onSetLooping */
63345 |     0
63346 | };
63347 |
63348 |
63349 | #if !defined(MA_NO_WAV)
63350 | static size_t ma_wav_dr_callback__read(void* pUserData, void* pBufferOut, size_t bytesToRead)
63351 | {
63352 |     ma_wav* pWav = (ma_wav*)pUserData;
63353 |     ma_result result;
63354 |     size_t bytesRead;
63355 |
63356 |     MA_ASSERT(pWav != NULL);
63357 |
63358 |     result = pWav->onRead(pWav->pReadSeekTellUserData, pBufferOut, bytesToRead, &bytesRead);
63359 |     (void)result;
63360 |
63361 |     return bytesRead;
63362 | }
63363 |
63364 | static ma_bool32 ma_wav_dr_callback__seek(void* pUserData, int offset, ma_dr_wav_seek_origin origin)
63365 | {
63366 |     ma_wav* pWav = (ma_wav*)pUserData;
63367 |     ma_result result;
63368 |     ma_seek_origin maSeekOrigin;
63369 |
63370 |     MA_ASSERT(pWav != NULL);
63371 |
63372 |     maSeekOrigin = ma_seek_origin_start;
63373 |     if (origin == MA_DR_WAV_SEEK_CUR) {
63374 |         maSeekOrigin = ma_seek_origin_current;
63375 |     } else if (origin == MA_DR_WAV_SEEK_END) {
63376 |         maSeekOrigin = ma_seek_origin_end;
63377 |     }
63378 |
63379 |     result = pWav->onSeek(pWav->pReadSeekTellUserData, offset, maSeekOrigin);
63380 |     if (result != MA_SUCCESS) {
63381 |         return MA_FALSE;
63382 |     }
63383 |
63384 |     return MA_TRUE;
63385 | }
63386 |
63387 | static ma_bool32 ma_wav_dr_callback__tell(void* pUserData, ma_int64* pCursor)
63388 | {
63389 |     ma_wav* pWav = (ma_wav*)pUserData;
63390 |     ma_result result;
63391 |
63392 |     MA_ASSERT(pWav != NULL);
63393 |     MA_ASSERT(pCursor != NULL);
63394 |
63395 |     if (pWav->onTell == NULL) {
63396 |         return MA_FALSE; /* Not implemented. */
63397 |     }
63398 |
63399 |     result = pWav->onTell(pWav->pReadSeekTellUserData, pCursor);
63400 |     if (result != MA_SUCCESS) {
63401 |         return MA_FALSE; /* Failed to tell. */
63402 |     }
63403 |
63404 |     return MA_TRUE;
63405 | }
63406 | #endif
63407 |
63408 | static ma_result ma_wav_init_internal(const ma_decoding_backend_config* pConfig, ma_wav* pWav)
63409 | {
63410 |     ma_result result;

```

```

63411 |     ma_data_source_config dataSourceConfig;
63412 |
63413 |     if (pWav == NULL) {
63414 |         return MA_INVALID_ARGS;
63415 |     }
63416 |
63417 |     MA_ZERO_OBJECT(pWav);
63418 |     pWav->format = ma_format_unknown;    /* Use closest match to source file by default. */
63419 |
63420 |     if (pConfig != NULL && (pConfig->preferredFormat == ma_format_f32 || pConfig->preferredFormat ==
> ma_format_s16 || pConfig->preferredFormat == ma_format_s32)) {
63421 |         pWav->format = pConfig->preferredFormat;
63422 |     } else {
63423 |         /* Getting here means something other than f32 and s16 was specified. Just leave this unset
> to use the default format. */
63424 |     }
63425 |
63426 |     dataSourceConfig = ma_data_source_config_init();
63427 |     dataSourceConfig.vtable = &g_ma_wav_ds_vtable;
63428 |
63429 |     result = ma_data_source_init(&dataSourceConfig, &pWav->ds);
63430 |     if (result != MA_SUCCESS) {
63431 |         return result; /* Failed to initialize the base data source. */
63432 |     }
63433 |
63434 |     return MA_SUCCESS;
63435 | }
63436 |
63437 | static ma_result ma_wav_post_init(ma_wav* pWav)
63438 | {
63439 |     /*
63440 |     If an explicit format was not specified, try picking the closest match based on the internal
63441 |     format. The format needs to be supported by miniaudio.
63442 |     */
63443 |     if (pWav->format == ma_format_unknown) {
63444 |         switch (pWav->dr.translatedFormatTag)
63445 |         {
63446 |             case MA_DR_WAVE_FORMAT_PCM:
63447 |             {
63448 |                 if (pWav->dr.bitsPerSample == 8) {
63449 |                     pWav->format = ma_format_u8;
63450 |                 } else if (pWav->dr.bitsPerSample == 16) {
63451 |                     pWav->format = ma_format_s16;
63452 |                 } else if (pWav->dr.bitsPerSample == 24) {
63453 |                     pWav->format = ma_format_s24;
63454 |                 } else if (pWav->dr.bitsPerSample == 32) {
63455 |                     pWav->format = ma_format_s32;
63456 |                 }
63457 |             } break;
63458 |
63459 |             case MA_DR_WAVE_FORMAT_IEEE_FLOAT:
63460 |             {
63461 |                 if (pWav->dr.bitsPerSample == 32) {
63462 |                     pWav->format = ma_format_f32;
63463 |                 }
63464 |             } break;
63465 |
63466 |             default: break;
63467 |         }
63468 |
63469 |         /* Fall back to f32 if we couldn't find anything. */
63470 |         if (pWav->format == ma_format_unknown) {
63471 |             pWav->format = ma_format_f32;
63472 |         }
63473 |     }
63474 |
63475 |     return MA_SUCCESS;
63476 | }
63477 |
63478 | MA_API ma_result ma_wav_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell, void*
> pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_wav* pWav)
63479 | {
63480 |     ma_result result;
63481 |
63482 |     result = ma_wav_init_internal(pConfig, pWav);
63483 |     if (result != MA_SUCCESS) {
63484 |         return result;
63485 |     }

```

```

63486 |
63487 |     if (onRead == NULL || onSeek == NULL) {
63488 |         return MA_INVALID_ARGS; /* onRead and onSeek are mandatory. */
63489 |     }
63490 |
63491 |     pWav->onRead = onRead;
63492 |     pWav->onSeek = onSeek;
63493 |     pWav->onTell = onTell;
63494 |     pWav->pReadSeekTellUserData = pReadSeekTellUserData;
63495 |
63496 |     #if !defined(MA_NO_WAV)
63497 |     {
63498 |         ma_bool32 wavResult;
63499 |
63500 |         wavResult = ma_dr_wav_init(&pWav->dr, ma_wav_dr_callback__read, ma_wav_dr_callback__seek,
> ma_wav_dr_callback__tell, pWav, pAllocationCallbacks);
63501 |         if (wavResult != MA_TRUE) {
63502 |             return MA_INVALID_FILE;
63503 |         }
63504 |
63505 |         ma_wav_post_init(pWav);
63506 |
63507 |         return MA_SUCCESS;
63508 |     }
63509 |     #else
63510 |     {
63511 |         /* wav is disabled. */
63512 |         (void)pAllocationCallbacks;
63513 |         return MA_NOT_IMPLEMENTED;
63514 |     }
63515 |     #endif
63516 | }
63517 |
63518 | MA_API ma_result ma_wav_init_file(const char* pFilePath, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_wav* pWav)
63519 | {
63520 |     ma_result result;
63521 |
63522 |     result = ma_wav_init_internal(pConfig, pWav);
63523 |     if (result != MA_SUCCESS) {
63524 |         return result;
63525 |     }
63526 |
63527 |     #if !defined(MA_NO_WAV)
63528 |     {
63529 |         ma_bool32 wavResult;
63530 |
63531 |         wavResult = ma_dr_wav_init_file(&pWav->dr, pFilePath, pAllocationCallbacks);
63532 |         if (wavResult != MA_TRUE) {
63533 |             return MA_INVALID_FILE;
63534 |         }
63535 |
63536 |         ma_wav_post_init(pWav);
63537 |
63538 |         return MA_SUCCESS;
63539 |     }
63540 |     #else
63541 |     {
63542 |         /* wav is disabled. */
63543 |         (void)pFilePath;
63544 |         (void)pAllocationCallbacks;
63545 |         return MA_NOT_IMPLEMENTED;
63546 |     }
63547 |     #endif
63548 | }
63549 |
63550 | MA_API ma_result ma_wav_init_file_w(const wchar_t* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_wav* pWav)
63551 | {
63552 |     ma_result result;
63553 |
63554 |     result = ma_wav_init_internal(pConfig, pWav);
63555 |     if (result != MA_SUCCESS) {
63556 |         return result;
63557 |     }
63558 |
63559 |     #if !defined(MA_NO_WAV)
63560 |     {
63561 |         ma_bool32 wavResult;

```

```

63562 |
63563 |     wavResult = ma_dr_wav_init_file_w(&pwav->dr, pFilePath, pAllocationCallbacks);
63564 |     if (wavResult != MA_TRUE) {
63565 |         return MA_INVALID_FILE;
63566 |     }
63567 |
63568 |     ma_wav_post_init(pWav);
63569 |
63570 |     return MA_SUCCESS;
63571 | }
63572 | #else
63573 | {
63574 |     /* wav is disabled. */
63575 |     (void)pFilePath;
63576 |     (void)pAllocationCallbacks;
63577 |     return MA_NOT_IMPLEMENTED;
63578 | }
63579 | #endif
63580 | }
63581 |
63582 | MA_API ma_result ma_wav_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_wav*
> pwav)
63583 | {
63584 |     ma_result result;
63585 |
63586 |     result = ma_wav_init_internal(pConfig, pwav);
63587 |     if (result != MA_SUCCESS) {
63588 |         return result;
63589 |     }
63590 |
63591 |     #if !defined(MA_NO_WAV)
63592 |     {
63593 |         ma_bool32 wavResult;
63594 |
63595 |         wavResult = ma_dr_wav_init_memory(&pwav->dr, pData, dataSize, pAllocationCallbacks);
63596 |         if (wavResult != MA_TRUE) {
63597 |             return MA_INVALID_FILE;
63598 |         }
63599 |
63600 |         ma_wav_post_init(pWav);
63601 |
63602 |         return MA_SUCCESS;
63603 |     }
63604 |     #else
63605 |     {
63606 |         /* wav is disabled. */
63607 |         (void)pData;
63608 |         (void)dataSize;
63609 |         (void)pAllocationCallbacks;
63610 |         return MA_NOT_IMPLEMENTED;
63611 |     }
63612 |     #endif
63613 | }
63614 |
63615 | MA_API void ma_wav_uninit(ma_wav* pwav, const ma_allocation_callbacks* pAllocationCallbacks)
63616 | {
63617 |     if (pwav == NULL) {
63618 |         return;
63619 |     }
63620 |
63621 |     (void)pAllocationCallbacks;
63622 |
63623 |     #if !defined(MA_NO_WAV)
63624 |     {
63625 |         ma_dr_wav_uninit(&pwav->dr);
63626 |     }
63627 |     #else
63628 |     {
63629 |         /* wav is disabled. Should never hit this since initialization would have failed. */
63630 |         MA_ASSERT(MA_FALSE);
63631 |     }
63632 |     #endif
63633 |
63634 |     ma_data_source_uninit(&pwav->ds);
63635 | }
63636 |
63637 | MA_API ma_result ma_wav_read_pcm_frames(ma_wav* pwav, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead)

```

```

63638 | {
63639 |     if (pFramesRead != NULL) {
63640 |         *pFramesRead = 0;
63641 |     }
63642 |
63643 |     if (frameCount == 0) {
63644 |         return MA_INVALID_ARGS;
63645 |     }
63646 |
63647 |     if (pWav == NULL) {
63648 |         return MA_INVALID_ARGS;
63649 |     }
63650 |
63651 |     #if !defined(MA_NO_WAV)
63652 |     {
63653 |         /* We always use floating point format. */
63654 |         ma_result result = MA_SUCCESS; /* Must be initialized to MA_SUCCESS. */
63655 |         ma_uint64 totalFramesRead = 0;
63656 |         ma_format format;
63657 |
63658 |         ma_wav_get_data_format(pWav, &format, NULL, NULL, NULL, 0);
63659 |
63660 |         switch (format)
63661 |         {
63662 |             case ma_format_f32:
63663 |             {
63664 |                 totalFramesRead = ma_dr_wav_read_pcm_frames_f32(&pWav->dr, frameCount,
> (float*)pFramesOut);
63665 |             } break;
63666 |
63667 |             case ma_format_s16:
63668 |             {
63669 |                 totalFramesRead = ma_dr_wav_read_pcm_frames_s16(&pWav->dr, frameCount,
> (ma_int16*)pFramesOut);
63670 |             } break;
63671 |
63672 |             case ma_format_s32:
63673 |             {
63674 |                 totalFramesRead = ma_dr_wav_read_pcm_frames_s32(&pWav->dr, frameCount,
> (ma_int32*)pFramesOut);
63675 |             } break;
63676 |
63677 |             /* Fallback to a raw read. */
63678 |             case ma_format_unknown: return MA_INVALID_OPERATION; /* <-- this should never be hit
> because initialization would just fall back to a supported format. */
63679 |             default:
63680 |             {
63681 |                 totalFramesRead = ma_dr_wav_read_pcm_frames(&pWav->dr, frameCount, pFramesOut);
63682 |             } break;
63683 |         }
63684 |
63685 |         /* In the future we'll update ma_dr_wav to return MA_AT_END for us. */
63686 |         if (totalFramesRead == 0) {
63687 |             result = MA_AT_END;
63688 |         }
63689 |
63690 |         if (pFramesRead != NULL) {
63691 |             *pFramesRead = totalFramesRead;
63692 |         }
63693 |
63694 |         if (result == MA_SUCCESS && totalFramesRead == 0) {
63695 |             result = MA_AT_END;
63696 |         }
63697 |
63698 |         return result;
63699 |     }
63700 |     #else
63701 |     {
63702 |         /* wav is disabled. Should never hit this since initialization would have failed. */
63703 |         MA_ASSERT(MA_FALSE);
63704 |
63705 |         (void)pFramesOut;
63706 |         (void)frameCount;
63707 |         (void)pFramesRead;
63708 |
63709 |         return MA_NOT_IMPLEMENTED;
63710 |     }
63711 |     #endif
63712 | }

```

```

63713 |
63714 | MA_API ma_result ma_wav_seek_to_pcm_frame(ma_wav* pWav, ma_uint64 frameIndex)
63715 | {
63716 |     if (pWav == NULL) {
63717 |         return MA_INVALID_ARGS;
63718 |     }
63719 |
63720 |     #if !defined(MA_NO_WAV)
63721 |     {
63722 |         ma_bool32 wavResult;
63723 |
63724 |         wavResult = ma_dr_wav_seek_to_pcm_frame(&pWav->dr, frameIndex);
63725 |         if (wavResult != MA_TRUE) {
63726 |             return MA_ERROR;
63727 |         }
63728 |
63729 |         return MA_SUCCESS;
63730 |     }
63731 |     #else
63732 |     {
63733 |         /* wav is disabled. Should never hit this since initialization would have failed. */
63734 |         MA_ASSERT(MA_FALSE);
63735 |
63736 |         (void)frameIndex;
63737 |
63738 |         return MA_NOT_IMPLEMENTED;
63739 |     }
63740 |     #endif
63741 | }
63742 |
63743 | MA_API ma_result ma_wav_get_data_format(ma_wav* pWav, ma_format* pFormat, ma_uint32* pChannels,
> ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
63744 | {
63745 |     /* Defaults for safety. */
63746 |     if (pFormat != NULL) {
63747 |         *pFormat = ma_format_unknown;
63748 |     }
63749 |     if (pChannels != NULL) {
63750 |         *pChannels = 0;
63751 |     }
63752 |     if (pSampleRate != NULL) {
63753 |         *pSampleRate = 0;
63754 |     }
63755 |     if (pChannelMap != NULL) {
63756 |         MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channelMapCap);
63757 |     }
63758 |
63759 |     if (pWav == NULL) {
63760 |         return MA_INVALID_OPERATION;
63761 |     }
63762 |
63763 |     if (pFormat != NULL) {
63764 |         *pFormat = pWav->format;
63765 |     }
63766 |
63767 |     #if !defined(MA_NO_WAV)
63768 |     {
63769 |         if (pChannels != NULL) {
63770 |             *pChannels = pWav->dr.channels;
63771 |         }
63772 |
63773 |         if (pSampleRate != NULL) {
63774 |             *pSampleRate = pWav->dr.sampleRate;
63775 |         }
63776 |
63777 |         if (pChannelMap != NULL) {
63778 |             ma_channel_map_init_standard(ma_standard_channel_map_microsoft, pChannelMap,
> channelMapCap, pWav->dr.channels);
63779 |         }
63780 |
63781 |         return MA_SUCCESS;
63782 |     }
63783 |     #else
63784 |     {
63785 |         /* wav is disabled. Should never hit this since initialization would have failed. */
63786 |         MA_ASSERT(MA_FALSE);
63787 |         return MA_NOT_IMPLEMENTED;
63788 |     }
63789 |     #endif

```

```

63790 | }
63791 |
63792 | MA_API ma_result ma_wav_get_cursor_in_pcm_frames(ma_wav* pWav, ma_uint64* pCursor)
63793 | {
63794 |     if (pCursor == NULL) {
63795 |         return MA_INVALID_ARGS;
63796 |     }
63797 |
63798 |     *pCursor = 0;    /* Safety. */
63799 |
63800 |     if (pWav == NULL) {
63801 |         return MA_INVALID_ARGS;
63802 |     }
63803 |
63804 |     #if !defined(MA_NO_WAV)
63805 |     {
63806 |         ma_result wavResult = ma_dr_wav_get_cursor_in_pcm_frames(&pWav->dr, pCursor);
63807 |         if (wavResult != MA_SUCCESS) {
63808 |             return (ma_result)wavResult;    /* ma_dr_wav result codes map to miniaudio's. */
63809 |         }
63810 |
63811 |         return MA_SUCCESS;
63812 |     }
63813 |     #else
63814 |     {
63815 |         /* wav is disabled. Should never hit this since initialization would have failed. */
63816 |         MA_ASSERT(MA_FALSE);
63817 |         return MA_NOT_IMPLEMENTED;
63818 |     }
63819 |     #endif
63820 | }
63821 |
63822 | MA_API ma_result ma_wav_get_length_in_pcm_frames(ma_wav* pWav, ma_uint64* pLength)
63823 | {
63824 |     if (pLength == NULL) {
63825 |         return MA_INVALID_ARGS;
63826 |     }
63827 |
63828 |     *pLength = 0;    /* Safety. */
63829 |
63830 |     if (pWav == NULL) {
63831 |         return MA_INVALID_ARGS;
63832 |     }
63833 |
63834 |     #if !defined(MA_NO_WAV)
63835 |     {
63836 |         ma_result wavResult = ma_dr_wav_get_length_in_pcm_frames(&pWav->dr, pLength);
63837 |         if (wavResult != MA_SUCCESS) {
63838 |             return (ma_result)wavResult;    /* ma_dr_wav result codes map to miniaudio's. */
63839 |         }
63840 |
63841 |         return MA_SUCCESS;
63842 |     }
63843 |     #else
63844 |     {
63845 |         /* wav is disabled. Should never hit this since initialization would have failed. */
63846 |         MA_ASSERT(MA_FALSE);
63847 |         return MA_NOT_IMPLEMENTED;
63848 |     }
63849 |     #endif
63850 | }
63851 |
63852 |
63853 | static ma_result ma_decoding_backend_init__wav(void* pUserData, ma_read_proc onRead, ma_seek_proc
> onSeek, ma_tell_proc onTell, void* pReadSeekTellUserData, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_data_source** ppBackend)
63854 | {
63855 |     ma_result result;
63856 |     ma_wav* pWav;
63857 |
63858 |     (void)pUserData;    /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
63859 |
63860 |     /* For now we're just allocating the decoder backend on the heap. */
63861 |     pWav = (ma_wav*)ma_malloc(sizeof(*pWav), pAllocationCallbacks);
63862 |     if (pWav == NULL) {
63863 |         return MA_OUT_OF_MEMORY;
63864 |     }
63865 |

```



```

63866 |     result = ma_wav_init(onRead, onSeek, onTell, pReadSeekTellUserData, pConfig,
> pAllocationCallbacks, pwav);
63867 |     if (result != MA_SUCCESS) {
63868 |         ma_free(pwav, pAllocationCallbacks);
63869 |         return result;
63870 |     }
63871 |
63872 |     *ppBackend = pwav;
63873 |
63874 |     return MA_SUCCESS;
63875 | }
63876 |
63877 | static ma_result ma_decoding_backend_init_file__wav(void* pUserData, const char* pFilePath, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
63878 | {
63879 |     ma_result result;
63880 |     ma_wav* pwav;
63881 |
63882 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
63883 |
63884 |     /* For now we're just allocating the decoder backend on the heap. */
63885 |     pwav = (ma_wav*)ma_malloc(sizeof(*pwav), pAllocationCallbacks);
63886 |     if (pwav == NULL) {
63887 |         return MA_OUT_OF_MEMORY;
63888 |     }
63889 |
63890 |     result = ma_wav_init_file(pFilePath, pConfig, pAllocationCallbacks, pwav);
63891 |     if (result != MA_SUCCESS) {
63892 |         ma_free(pwav, pAllocationCallbacks);
63893 |         return result;
63894 |     }
63895 |
63896 |     *ppBackend = pwav;
63897 |
63898 |     return MA_SUCCESS;
63899 | }
63900 |
63901 | static ma_result ma_decoding_backend_init_file_w__wav(void* pUserData, const wchar_t* pFilePath,
> const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
63902 | {
63903 |     ma_result result;
63904 |     ma_wav* pwav;
63905 |
63906 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
63907 |
63908 |     /* For now we're just allocating the decoder backend on the heap. */
63909 |     pwav = (ma_wav*)ma_malloc(sizeof(*pwav), pAllocationCallbacks);
63910 |     if (pwav == NULL) {
63911 |         return MA_OUT_OF_MEMORY;
63912 |     }
63913 |
63914 |     result = ma_wav_init_file_w(pFilePath, pConfig, pAllocationCallbacks, pwav);
63915 |     if (result != MA_SUCCESS) {
63916 |         ma_free(pwav, pAllocationCallbacks);
63917 |         return result;
63918 |     }
63919 |
63920 |     *ppBackend = pwav;
63921 |
63922 |     return MA_SUCCESS;
63923 | }
63924 |
63925 | static ma_result ma_decoding_backend_init_memory__wav(void* pUserData, const void* pData, size_t
> dataSize, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_data_source** ppBackend)
63926 | {
63927 |     ma_result result;
63928 |     ma_wav* pwav;
63929 |
63930 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
63931 |
63932 |     /* For now we're just allocating the decoder backend on the heap. */
63933 |     pwav = (ma_wav*)ma_malloc(sizeof(*pwav), pAllocationCallbacks);
63934 |     if (pwav == NULL) {

```

```

63935 |         return MA_OUT_OF_MEMORY;
63936 |     }
63937 |
63938 |     result = ma_wav_init_memory(pData, dataSize, pConfig, pAllocationCallbacks, pWav);
63939 |     if (result != MA_SUCCESS) {
63940 |         ma_free(pWav, pAllocationCallbacks);
63941 |         return result;
63942 |     }
63943 |
63944 |     *ppBackend = pWav;
63945 |
63946 |     return MA_SUCCESS;
63947 | }
63948 |
63949 | static void ma_decoding_backend_uninit__wav(void* pUserData, ma_data_source* pBackend, const
> ma_allocation_callbacks* pAllocationCallbacks)
63950 | {
63951 |     ma_wav* pWav = (ma_wav*)pBackend;
63952 |
63953 |     (void)pUserData;
63954 |
63955 |     ma_wav_uninit(pWav, pAllocationCallbacks);
63956 |     ma_free(pWav, pAllocationCallbacks);
63957 | }
63958 |
63959 | static ma_decoding_backend_vtable g_ma_decoding_backend_vtable_wav =
63960 | {
63961 |     ma_decoding_backend_init__wav,
63962 |     ma_decoding_backend_init_file__wav,
63963 |     ma_decoding_backend_init_file_w__wav,
63964 |     ma_decoding_backend_init_memory__wav,
63965 |     ma_decoding_backend_uninit__wav
63966 | };
63967 |
63968 | static ma_result ma_decoder_init_wav__internal(const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
63969 | {
63970 |     return ma_decoder_init_from_vtable__internal(&g_ma_decoding_backend_vtable_wav, NULL, pConfig,
> pDecoder);
63971 | }
63972 |
63973 | static ma_result ma_decoder_init_wav_from_file__internal(const char* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
63974 | {
63975 |     return ma_decoder_init_from_file__internal(&g_ma_decoding_backend_vtable_wav, NULL, pFilePath,
> pConfig, pDecoder);
63976 | }
63977 |
63978 | static ma_result ma_decoder_init_wav_from_file_w__internal(const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
63979 | {
63980 |     return ma_decoder_init_from_file_w__internal(&g_ma_decoding_backend_vtable_wav, NULL, pFilePath,
> pConfig, pDecoder);
63981 | }
63982 |
63983 | static ma_result ma_decoder_init_wav_from_memory__internal(const void* pData, size_t dataSize, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
63984 | {
63985 |     return ma_decoder_init_from_memory__internal(&g_ma_decoding_backend_vtable_wav, NULL, pData,
> dataSize, pConfig, pDecoder);
63986 | }
63987 | #endif /* ma_dr_wav_h */
63988 |
63989 | /* FLAC */
63990 | #ifdef ma_dr_flac_h
63991 |
63992 | typedef struct
63993 | {
63994 |     ma_data_source_base ds;
63995 |     ma_read_proc onRead;
63996 |     ma_seek_proc onSeek;
63997 |     ma_tell_proc onTell;
63998 |     void* pReadSeekTellUserData;
63999 |     ma_format format; /* Can be f32, s16 or s32. */
64000 | #if !defined(MA_NO_FLAC)
64001 |     ma_dr_flac* dr;
64002 | #endif
64003 | } ma_flac;
64004 |

```

```

64005 | MA_API ma_result ma_flac_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell, void*
> pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_flac* pFlac);
64006 | MA_API ma_result ma_flac_init_file(const char* pFilePath, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_flac* pFlac);
64007 | MA_API ma_result ma_flac_init_file_w(const wchar_t* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_flac* pFlac);
64008 | MA_API ma_result ma_flac_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_flac*
> pFlac);
64009 | MA_API void ma_flac_uninit(ma_flac* pFlac, const ma_allocation_callbacks* pAllocationCallbacks);
64010 | MA_API ma_result ma_flac_read_pcm_frames(ma_flac* pFlac, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead);
64011 | MA_API ma_result ma_flac_seek_to_pcm_frame(ma_flac* pFlac, ma_uint64 frameIndex);
64012 | MA_API ma_result ma_flac_get_data_format(ma_flac* pFlac, ma_format* pFormat, ma_uint32* pChannels,
> ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
64013 | MA_API ma_result ma_flac_get_cursor_in_pcm_frames(ma_flac* pFlac, ma_uint64* pCursor);
64014 | MA_API ma_result ma_flac_get_length_in_pcm_frames(ma_flac* pFlac, ma_uint64* pLength);
64015 |
64016 |
64017 | static ma_result ma_flac_ds_read(ma_data_source* pDataSource, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
64018 | {
64019 |     return ma_flac_read_pcm_frames((ma_flac*)pDataSource, pFramesOut, frameCount, pFramesRead);
64020 | }
64021 |
64022 | static ma_result ma_flac_ds_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
64023 | {
64024 |     return ma_flac_seek_to_pcm_frame((ma_flac*)pDataSource, frameIndex);
64025 | }
64026 |
64027 | static ma_result ma_flac_ds_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
64028 | {
64029 |     return ma_flac_get_data_format((ma_flac*)pDataSource, pFormat, pChannels, pSampleRate,
> pChannelMap, channelMapCap);
64030 | }
64031 |
64032 | static ma_result ma_flac_ds_get_cursor(ma_data_source* pDataSource, ma_uint64* pCursor)
64033 | {
64034 |     return ma_flac_get_cursor_in_pcm_frames((ma_flac*)pDataSource, pCursor);
64035 | }
64036 |
64037 | static ma_result ma_flac_ds_get_length(ma_data_source* pDataSource, ma_uint64* pLength)
64038 | {
64039 |     return ma_flac_get_length_in_pcm_frames((ma_flac*)pDataSource, pLength);
64040 | }
64041 |
64042 | static ma_data_source_vtable g_ma_flac_ds_vtable =
64043 | {
64044 |     ma_flac_ds_read,
64045 |     ma_flac_ds_seek,
64046 |     ma_flac_ds_get_data_format,
64047 |     ma_flac_ds_get_cursor,
64048 |     ma_flac_ds_get_length,
64049 |     NULL, /* onSetLooping */
64050 |     0
64051 | };
64052 |
64053 |
64054 | #if !defined(MA_NO_FLAC)
64055 | static size_t ma_flac_dr_callback__read(void* pUserData, void* pBufferOut, size_t bytesToRead)
64056 | {
64057 |     ma_flac* pFlac = (ma_flac*)pUserData;
64058 |     ma_result result;
64059 |     size_t bytesRead;
64060 |
64061 |     MA_ASSERT(pFlac != NULL);
64062 |
64063 |     result = pFlac->onRead(pFlac->pReadSeekTellUserData, pBufferOut, bytesToRead, &bytesRead);
64064 |     (void)result;
64065 |
64066 |     return bytesRead;
64067 | }
64068 |
64069 | static ma_bool32 ma_flac_dr_callback__seek(void* pUserData, int offset, ma_dr_flac_seek_origin
> origin)
64070 | {
64071 |     ma_flac* pFlac = (ma_flac*)pUserData;

```

```

64072 |     ma_result result;
64073 |     ma_seek_origin maSeekOrigin;
64074 |
64075 |     MA_ASSERT(pFlac != NULL);
64076 |
64077 |     maSeekOrigin = ma_seek_origin_start;
64078 |     if (origin == MA_DR_FLAC_SEEK_CUR) {
64079 |         maSeekOrigin = ma_seek_origin_current;
64080 |     } else if (origin == MA_DR_FLAC_SEEK_END) {
64081 |         maSeekOrigin = ma_seek_origin_end;
64082 |     }
64083 |
64084 |     result = pFlac->onSeek(pFlac->pReadSeekTellUserData, offset, maSeekOrigin);
64085 |     if (result != MA_SUCCESS) {
64086 |         return MA_FALSE;
64087 |     }
64088 |
64089 |     return MA_TRUE;
64090 | }
64091 |
64092 | static ma_bool32 ma_flac_dr_callback__tell(void* pUserData, ma_int64* pCursor)
64093 | {
64094 |     ma_flac* pFlac = (ma_flac*)pUserData;
64095 |     ma_result result;
64096 |
64097 |     MA_ASSERT(pFlac != NULL);
64098 |     MA_ASSERT(pCursor != NULL);
64099 |
64100 |     if (pFlac->onTell == NULL) {
64101 |         return MA_FALSE; /* Not implemented. */
64102 |     }
64103 |
64104 |     result = pFlac->onTell(pFlac->pReadSeekTellUserData, pCursor);
64105 |     if (result != MA_SUCCESS) {
64106 |         return MA_FALSE; /* Failed to tell. */
64107 |     }
64108 |
64109 |     return MA_TRUE;
64110 | }
64111 | #endif
64112 |
64113 | static ma_result ma_flac_init_internal(const ma_decoding_backend_config* pConfig, ma_flac* pFlac)
64114 | {
64115 |     ma_result result;
64116 |     ma_data_source_config dataSourceConfig;
64117 |
64118 |     if (pFlac == NULL) {
64119 |         return MA_INVALID_ARGS;
64120 |     }
64121 |
64122 |     MA_ZERO_OBJECT(pFlac);
64123 |     pFlac->format = ma_format_f32; /* f32 by default. */
64124 |
64125 |     if (pConfig != NULL && (pConfig->preferredFormat == ma_format_f32 || pConfig->preferredFormat ==
> ma_format_s16 || pConfig->preferredFormat == ma_format_s32)) {
64126 |         pFlac->format = pConfig->preferredFormat;
64127 |     } else {
64128 |         /* Getting here means something other than f32 and s16 was specified. Just leave this unset
> to use the default format. */
64129 |     }
64130 |
64131 |     dataSourceConfig = ma_data_source_config_init();
64132 |     dataSourceConfig.vtable = &g_ma_flac_ds_vtable;
64133 |
64134 |     result = ma_data_source_init(&dataSourceConfig, &pFlac->ds);
64135 |     if (result != MA_SUCCESS) {
64136 |         return result; /* Failed to initialize the base data source. */
64137 |     }
64138 |
64139 |     return MA_SUCCESS;
64140 | }
64141 |
64142 | MA_API ma_result ma_flac_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell, void*
> pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_flac* pFlac)
64143 | {
64144 |     ma_result result;
64145 |
64146 |     result = ma_flac_init_internal(pConfig, pFlac);

```

```

64147 |     if (result != MA_SUCCESS) {
64148 |         return result;
64149 |     }
64150 |
64151 |     if (onRead == NULL || onSeek == NULL) {
64152 |         return MA_INVALID_ARGS; /* onRead and onSeek are mandatory. */
64153 |     }
64154 |
64155 |     pFlac->onRead = onRead;
64156 |     pFlac->onSeek = onSeek;
64157 |     pFlac->onTell = onTell;
64158 |     pFlac->pReadSeekTellUserData = pReadSeekTellUserData;
64159 |
64160 |     #if !defined(MA_NO_FLAC)
64161 |     {
64162 |         pFlac->dr = ma_dr_flac_open(ma_flac_dr_callback_read, ma_flac_dr_callback_seek,
> ma_flac_dr_callback_tell, pFlac, pAllocationCallbacks);
64163 |         if (pFlac->dr == NULL) {
64164 |             return MA_INVALID_FILE;
64165 |         }
64166 |
64167 |         return MA_SUCCESS;
64168 |     }
64169 |     #else
64170 |     {
64171 |         /* flac is disabled. */
64172 |         (void)pAllocationCallbacks;
64173 |         return MA_NOT_IMPLEMENTED;
64174 |     }
64175 |     #endif
64176 | }
64177 |
64178 | MA_API ma_result ma_flac_init_file(const char* pFilePath, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_flac* pFlac)
64179 | {
64180 |     ma_result result;
64181 |
64182 |     result = ma_flac_init_internal(pConfig, pFlac);
64183 |     if (result != MA_SUCCESS) {
64184 |         return result;
64185 |     }
64186 |
64187 |     #if !defined(MA_NO_FLAC)
64188 |     {
64189 |         pFlac->dr = ma_dr_flac_open_file(pFilePath, pAllocationCallbacks);
64190 |         if (pFlac->dr == NULL) {
64191 |             return MA_INVALID_FILE;
64192 |         }
64193 |
64194 |         return MA_SUCCESS;
64195 |     }
64196 |     #else
64197 |     {
64198 |         /* flac is disabled. */
64199 |         (void)pFilePath;
64200 |         (void)pAllocationCallbacks;
64201 |         return MA_NOT_IMPLEMENTED;
64202 |     }
64203 |     #endif
64204 | }
64205 |
64206 | MA_API ma_result ma_flac_init_file_w(const wchar_t* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_flac* pFlac)
64207 | {
64208 |     ma_result result;
64209 |
64210 |     result = ma_flac_init_internal(pConfig, pFlac);
64211 |     if (result != MA_SUCCESS) {
64212 |         return result;
64213 |     }
64214 |
64215 |     #if !defined(MA_NO_FLAC)
64216 |     {
64217 |         pFlac->dr = ma_dr_flac_open_file_w(pFilePath, pAllocationCallbacks);
64218 |         if (pFlac->dr == NULL) {
64219 |             return MA_INVALID_FILE;
64220 |         }
64221 |
64222 |         return MA_SUCCESS;

```

```

64223 |     }
64224 |     #else
64225 |     {
64226 |         /* flac is disabled. */
64227 |         (void)pFilePath;
64228 |         (void)pAllocationCallbacks;
64229 |         return MA_NOT_IMPLEMENTED;
64230 |     }
64231 |     #endif
64232 | }
64233 |
64234 | MA_API ma_result ma_flac_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_flac*
> pFlac)
64235 | {
64236 |     ma_result result;
64237 |
64238 |     result = ma_flac_init_internal(pConfig, pFlac);
64239 |     if (result != MA_SUCCESS) {
64240 |         return result;
64241 |     }
64242 |
64243 |     #if !defined(MA_NO_FLAC)
64244 |     {
64245 |         pFlac->dr = ma_dr_flac_open_memory(pData, dataSize, pAllocationCallbacks);
64246 |         if (pFlac->dr == NULL) {
64247 |             return MA_INVALID_FILE;
64248 |         }
64249 |
64250 |         return MA_SUCCESS;
64251 |     }
64252 |     #else
64253 |     {
64254 |         /* flac is disabled. */
64255 |         (void)pData;
64256 |         (void)dataSize;
64257 |         (void)pAllocationCallbacks;
64258 |         return MA_NOT_IMPLEMENTED;
64259 |     }
64260 |     #endif
64261 | }
64262 |
64263 | MA_API void ma_flac_uninit(ma_flac* pFlac, const ma_allocation_callbacks* pAllocationCallbacks)
64264 | {
64265 |     if (pFlac == NULL) {
64266 |         return;
64267 |     }
64268 |
64269 |     (void)pAllocationCallbacks;
64270 |
64271 |     #if !defined(MA_NO_FLAC)
64272 |     {
64273 |         ma_dr_flac_close(pFlac->dr);
64274 |     }
64275 |     #else
64276 |     {
64277 |         /* flac is disabled. Should never hit this since initialization would have failed. */
64278 |         MA_ASSERT(MA_FALSE);
64279 |     }
64280 |     #endif
64281 |
64282 |     ma_data_source_uninit(&pFlac->ds);
64283 | }
64284 |
64285 | MA_API ma_result ma_flac_read_pcm_frames(ma_flac* pFlac, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead)
64286 | {
64287 |     if (pFramesRead != NULL) {
64288 |         *pFramesRead = 0;
64289 |     }
64290 |
64291 |     if (frameCount == 0) {
64292 |         return MA_INVALID_ARGS;
64293 |     }
64294 |
64295 |     if (pFlac == NULL) {
64296 |         return MA_INVALID_ARGS;
64297 |     }
64298 | }

```

```

64299 |     #if !defined(MA_NO_FLAC)
64300 |     {
64301 |         /* We always use floating point format. */
64302 |         ma_result result = MA_SUCCESS; /* Must be initialized to MA_SUCCESS. */
64303 |         ma_uint64 totalFramesRead = 0;
64304 |         ma_format format;
64305 |
64306 |         ma_flac_get_data_format(pFlac, &format, NULL, NULL, NULL, 0);
64307 |
64308 |         switch (format)
64309 |         {
64310 |             case ma_format_f32:
64311 |             {
64312 |                 totalFramesRead = ma_dr_flac_read_pcm_frames_f32(pFlac->dr, frameCount,
> (float*)pFramesOut);
64313 |             } break;
64314 |
64315 |             case ma_format_s16:
64316 |             {
64317 |                 totalFramesRead = ma_dr_flac_read_pcm_frames_s16(pFlac->dr, frameCount,
> (ma_int16*)pFramesOut);
64318 |             } break;
64319 |
64320 |             case ma_format_s32:
64321 |             {
64322 |                 totalFramesRead = ma_dr_flac_read_pcm_frames_s32(pFlac->dr, frameCount,
> (ma_int32*)pFramesOut);
64323 |             } break;
64324 |
64325 |             case ma_format_u8:
64326 |             case ma_format_s24:
64327 |             case ma_format_unknown:
64328 |             default:
64329 |             {
64330 |                 return MA_INVALID_OPERATION;
64331 |             };
64332 |         }
64333 |
64334 |         /* In the future we'll update ma_dr_flac to return MA_AT_END for us. */
64335 |         if (totalFramesRead == 0) {
64336 |             result = MA_AT_END;
64337 |         }
64338 |
64339 |         if (pFramesRead != NULL) {
64340 |             *pFramesRead = totalFramesRead;
64341 |         }
64342 |
64343 |         if (result == MA_SUCCESS && totalFramesRead == 0) {
64344 |             result = MA_AT_END;
64345 |         }
64346 |
64347 |         return result;
64348 |     }
64349 |     #else
64350 |     {
64351 |         /* flac is disabled. Should never hit this since initialization would have failed. */
64352 |         MA_ASSERT(MA_FALSE);
64353 |
64354 |         (void)pFramesOut;
64355 |         (void)frameCount;
64356 |         (void)pFramesRead;
64357 |
64358 |         return MA_NOT_IMPLEMENTED;
64359 |     }
64360 |     #endif
64361 | }
64362 |
64363 | MA_API ma_result ma_flac_seek_to_pcm_frame(ma_flac* pFlac, ma_uint64 frameIndex)
64364 | {
64365 |     if (pFlac == NULL) {
64366 |         return MA_INVALID_ARGS;
64367 |     }
64368 |
64369 |     #if !defined(MA_NO_FLAC)
64370 |     {
64371 |         ma_bool32 flacResult;
64372 |
64373 |         flacResult = ma_dr_flac_seek_to_pcm_frame(pFlac->dr, frameIndex);
64374 |         if (flacResult != MA_TRUE) {

```



```
64375 |         return MA_ERROR;
64376 |     }
64377 |
64378 |     return MA_SUCCESS;
64379 | }
64380 | #else
64381 | {
64382 |     /* flac is disabled. Should never hit this since initialization would have failed. */
64383 |     MA_ASSERT(MA_FALSE);
64384 |
64385 |     (void)frameIndex;
64386 |
64387 |     return MA_NOT_IMPLEMENTED;
64388 | }
64389 | #endif
64390 | }
64391 |
64392 | MA_API ma_result ma_flac_get_data_format(ma_flac* pFlac, ma_format* pFormat, ma_uint32* pChannels,
> ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
64393 | {
64394 |     /* Defaults for safety. */
64395 |     if (pFormat != NULL) {
64396 |         *pFormat = ma_format_unknown;
64397 |     }
64398 |     if (pChannels != NULL) {
64399 |         *pChannels = 0;
64400 |     }
64401 |     if (pSampleRate != NULL) {
64402 |         *pSampleRate = 0;
64403 |     }
64404 |     if (pChannelMap != NULL) {
64405 |         MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channelMapCap);
64406 |     }
64407 |
64408 |     if (pFlac == NULL) {
64409 |         return MA_INVALID_OPERATION;
64410 |     }
64411 |
64412 |     if (pFormat != NULL) {
64413 |         *pFormat = pFlac->format;
64414 |     }
64415 |
64416 |     #if !defined(MA_NO_FLAC)
64417 |     {
64418 |         if (pChannels != NULL) {
64419 |             *pChannels = pFlac->dr->channels;
64420 |         }
64421 |
64422 |         if (pSampleRate != NULL) {
64423 |             *pSampleRate = pFlac->dr->sampleRate;
64424 |         }
64425 |
64426 |         if (pChannelMap != NULL) {
64427 |             ma_channel_map_init_standard(ma_standard_channel_map_microsoft, pChannelMap,
> channelMapCap, pFlac->dr->channels);
64428 |         }
64429 |
64430 |         return MA_SUCCESS;
64431 |     }
64432 | #else
64433 | {
64434 |     /* flac is disabled. Should never hit this since initialization would have failed. */
64435 |     MA_ASSERT(MA_FALSE);
64436 |     return MA_NOT_IMPLEMENTED;
64437 | }
64438 | #endif
64439 | }
64440 |
64441 | MA_API ma_result ma_flac_get_cursor_in_pcm_frames(ma_flac* pFlac, ma_uint64* pCursor)
64442 | {
64443 |     if (pCursor == NULL) {
64444 |         return MA_INVALID_ARGS;
64445 |     }
64446 |
64447 |     *pCursor = 0;    /* Safety. */
64448 |
64449 |     if (pFlac == NULL) {
64450 |         return MA_INVALID_ARGS;
64451 |     }
```

```

64452 |
64453 |     #if !defined(MA_NO_FLAC)
64454 |     {
64455 |         *pCursor = pFlac->dr->currentPCMFrame;
64456 |
64457 |         return MA_SUCCESS;
64458 |     }
64459 |     #else
64460 |     {
64461 |         /* flac is disabled. Should never hit this since initialization would have failed. */
64462 |         MA_ASSERT(MA_FALSE);
64463 |         return MA_NOT_IMPLEMENTED;
64464 |     }
64465 |     #endif
64466 | }
64467 |
64468 | MA_API ma_result ma_flac_get_length_in_pcm_frames(ma_flac* pFlac, ma_uint64* pLength)
64469 | {
64470 |     if (pLength == NULL) {
64471 |         return MA_INVALID_ARGS;
64472 |     }
64473 |
64474 |     *pLength = 0;    /* Safety. */
64475 |
64476 |     if (pFlac == NULL) {
64477 |         return MA_INVALID_ARGS;
64478 |     }
64479 |
64480 |     #if !defined(MA_NO_FLAC)
64481 |     {
64482 |         *pLength = pFlac->dr->totalPCMFrameCount;
64483 |
64484 |         return MA_SUCCESS;
64485 |     }
64486 |     #else
64487 |     {
64488 |         /* flac is disabled. Should never hit this since initialization would have failed. */
64489 |         MA_ASSERT(MA_FALSE);
64490 |         return MA_NOT_IMPLEMENTED;
64491 |     }
64492 |     #endif
64493 | }
64494 |
64495 |
64496 | static ma_result ma_decoding_backend_init__flac(void* pUserData, ma_read_proc onRead, ma_seek_proc
> onSeek, ma_tell_proc onTell, void* pReadSeekTellUserData, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_data_source** ppBackend)
64497 | {
64498 |     ma_result result;
64499 |     ma_flac* pFlac;
64500 |
64501 |     (void)pUserData;    /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
64502 |
64503 |     /* For now we're just allocating the decoder backend on the heap. */
64504 |     pFlac = (ma_flac*)ma_malloc(sizeof(*pFlac), pAllocationCallbacks);
64505 |     if (pFlac == NULL) {
64506 |         return MA_OUT_OF_MEMORY;
64507 |     }
64508 |
64509 |     result = ma_flac_init(onRead, onSeek, onTell, pReadSeekTellUserData, pConfig,
> pAllocationCallbacks, pFlac);
64510 |     if (result != MA_SUCCESS) {
64511 |         ma_free(pFlac, pAllocationCallbacks);
64512 |         return result;
64513 |     }
64514 |
64515 |     *ppBackend = pFlac;
64516 |
64517 |     return MA_SUCCESS;
64518 | }
64519 |
64520 | static ma_result ma_decoding_backend_init_file__flac(void* pUserData, const char* pFilePath, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
64521 | {
64522 |     ma_result result;
64523 |     ma_flac* pFlac;
64524 |

```

```

64525 |     (void)pUserData;    /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
64526 |
64527 |     /* For now we're just allocating the decoder backend on the heap. */
64528 |     pFlac = (ma_flac*)ma_malloc(sizeof(*pFlac), pAllocationCallbacks);
64529 |     if (pFlac == NULL) {
64530 |         return MA_OUT_OF_MEMORY;
64531 |     }
64532 |
64533 |     result = ma_flac_init_file(pFilePath, pConfig, pAllocationCallbacks, pFlac);
64534 |     if (result != MA_SUCCESS) {
64535 |         ma_free(pFlac, pAllocationCallbacks);
64536 |         return result;
64537 |     }
64538 |
64539 |     *ppBackend = pFlac;
64540 |
64541 |     return MA_SUCCESS;
64542 | }
64543 |
64544 | static ma_result ma_decoding_backend_init_file_w__flac(void* pUserData, const wchar_t* pFilePath,
> const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
64545 | {
64546 |     ma_result result;
64547 |     ma_flac* pFlac;
64548 |
64549 |     (void)pUserData;    /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
64550 |
64551 |     /* For now we're just allocating the decoder backend on the heap. */
64552 |     pFlac = (ma_flac*)ma_malloc(sizeof(*pFlac), pAllocationCallbacks);
64553 |     if (pFlac == NULL) {
64554 |         return MA_OUT_OF_MEMORY;
64555 |     }
64556 |
64557 |     result = ma_flac_init_file_w(pFilePath, pConfig, pAllocationCallbacks, pFlac);
64558 |     if (result != MA_SUCCESS) {
64559 |         ma_free(pFlac, pAllocationCallbacks);
64560 |         return result;
64561 |     }
64562 |
64563 |     *ppBackend = pFlac;
64564 |
64565 |     return MA_SUCCESS;
64566 | }
64567 |
64568 | static ma_result ma_decoding_backend_init_memory__flac(void* pUserData, const void* pData, size_t
> dataSize, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_data_source** ppBackend)
64569 | {
64570 |     ma_result result;
64571 |     ma_flac* pFlac;
64572 |
64573 |     (void)pUserData;    /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
64574 |
64575 |     /* For now we're just allocating the decoder backend on the heap. */
64576 |     pFlac = (ma_flac*)ma_malloc(sizeof(*pFlac), pAllocationCallbacks);
64577 |     if (pFlac == NULL) {
64578 |         return MA_OUT_OF_MEMORY;
64579 |     }
64580 |
64581 |     result = ma_flac_init_memory(pData, dataSize, pConfig, pAllocationCallbacks, pFlac);
64582 |     if (result != MA_SUCCESS) {
64583 |         ma_free(pFlac, pAllocationCallbacks);
64584 |         return result;
64585 |     }
64586 |
64587 |     *ppBackend = pFlac;
64588 |
64589 |     return MA_SUCCESS;
64590 | }
64591 |
64592 | static void ma_decoding_backend_uninit__flac(void* pUserData, ma_data_source* pBackend, const
> ma_allocation_callbacks* pAllocationCallbacks)
64593 | {
64594 |     ma_flac* pFlac = (ma_flac*)pBackend;
64595 |

```

```

64596 |     (void)pUserData;
64597 |
64598 |     ma_flac_uninit(pFlac, pAllocationCallbacks);
64599 |     ma_free(pFlac, pAllocationCallbacks);
64600 | }
64601 |
64602 | static ma_decoding_backend_vtable g_ma_decoding_backend_vtable_flac =
64603 | {
64604 |     ma_decoding_backend_init__flac,
64605 |     ma_decoding_backend_init_file__flac,
64606 |     ma_decoding_backend_init_file_w__flac,
64607 |     ma_decoding_backend_init_memory__flac,
64608 |     ma_decoding_backend_uninit__flac
64609 | };
64610 |
64611 | static ma_result ma_decoder_init_flac__internal(const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
64612 | {
64613 |     return ma_decoder_init_from_vtable__internal(&g_ma_decoding_backend_vtable_flac, NULL, pConfig,
> pDecoder);
64614 | }
64615 |
64616 | static ma_result ma_decoder_init_flac_from_file__internal(const char* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
64617 | {
64618 |     return ma_decoder_init_from_file__internal(&g_ma_decoding_backend_vtable_flac, NULL, pFilePath,
> pConfig, pDecoder);
64619 | }
64620 |
64621 | static ma_result ma_decoder_init_flac_from_file_w__internal(const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
64622 | {
64623 |     return ma_decoder_init_from_file_w__internal(&g_ma_decoding_backend_vtable_flac, NULL,
> pFilePath, pConfig, pDecoder);
64624 | }
64625 |
64626 | static ma_result ma_decoder_init_flac_from_memory__internal(const void* pData, size_t dataSize,
> const ma_decoder_config* pConfig, ma_decoder* pDecoder)
64627 | {
64628 |     return ma_decoder_init_from_memory__internal(&g_ma_decoding_backend_vtable_flac, NULL, pData,
> dataSize, pConfig, pDecoder);
64629 | }
64630 | #endif /* ma_dr_flac_h */
64631 |
64632 | /* MP3 */
64633 | #ifdef ma_dr_mp3_h
64634 |
64635 | typedef struct
64636 | {
64637 |     ma_data_source_base ds;
64638 |     ma_read_proc onRead;
64639 |     ma_seek_proc onSeek;
64640 |     ma_tell_proc onTell;
64641 |     void* pReadSeekTellUserData;
64642 |     ma_format format; /* Can be f32 or s16. */
64643 | #if !defined(MA_NO_MP3)
64644 |     ma_dr_mp3 dr;
64645 |     ma_uint32 seekPointCount;
64646 |     ma_dr_mp3_seek_point* pSeekPoints; /* Only used if seek table generation is used. */
64647 | #endif
64648 | } ma_mp3;
64649 |
64650 | MA_API ma_result ma_mp3_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell, void*
> pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_mp3* pMP3);
64651 | MA_API ma_result ma_mp3_init_file(const char* pFilePath, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_mp3* pMP3);
64652 | MA_API ma_result ma_mp3_init_file_w(const wchar_t* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_mp3* pMP3);
64653 | MA_API ma_result ma_mp3_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_mp3*
> pMP3);
64654 | MA_API void ma_mp3_uninit(ma_mp3* pMP3, const ma_allocation_callbacks* pAllocationCallbacks);
64655 | MA_API ma_result ma_mp3_read_pcm_frames(ma_mp3* pMP3, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead);
64656 | MA_API ma_result ma_mp3_seek_to_pcm_frame(ma_mp3* pMP3, ma_uint64 frameIndex);
64657 | MA_API ma_result ma_mp3_get_data_format(ma_mp3* pMP3, ma_format* pFormat, ma_uint32* pChannels,
> ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
64658 | MA_API ma_result ma_mp3_get_cursor_in_pcm_frames(ma_mp3* pMP3, ma_uint64* pCursor);

```

```

64659 | MA_API ma_result ma_mp3_get_length_in_pcm_frames(ma_mp3* pMP3, ma_uint64* pLength);
64660 |
64661 |
64662 | static ma_result ma_mp3_ds_read(ma_data_source* pDataSource, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead)
64663 | {
64664 |     return ma_mp3_read_pcm_frames((ma_mp3*)pDataSource, pFramesOut, frameCount, pFramesRead);
64665 | }
64666 |
64667 | static ma_result ma_mp3_ds_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
64668 | {
64669 |     return ma_mp3_seek_to_pcm_frame((ma_mp3*)pDataSource, frameIndex);
64670 | }
64671 |
64672 | static ma_result ma_mp3_ds_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
64673 | {
64674 |     return ma_mp3_get_data_format((ma_mp3*)pDataSource, pFormat, pChannels, pSampleRate,
> pChannelMap, channelMapCap);
64675 | }
64676 |
64677 | static ma_result ma_mp3_ds_get_cursor(ma_data_source* pDataSource, ma_uint64* pCursor)
64678 | {
64679 |     return ma_mp3_get_cursor_in_pcm_frames((ma_mp3*)pDataSource, pCursor);
64680 | }
64681 |
64682 | static ma_result ma_mp3_ds_get_length(ma_data_source* pDataSource, ma_uint64* pLength)
64683 | {
64684 |     return ma_mp3_get_length_in_pcm_frames((ma_mp3*)pDataSource, pLength);
64685 | }
64686 |
64687 | static ma_data_source_vtable g_ma_mp3_ds_vtable =
64688 | {
64689 |     ma_mp3_ds_read,
64690 |     ma_mp3_ds_seek,
64691 |     ma_mp3_ds_get_data_format,
64692 |     ma_mp3_ds_get_cursor,
64693 |     ma_mp3_ds_get_length,
64694 |     NULL, /* onSetLooping */
64695 |     0
64696 | };
64697 |
64698 |
64699 | #if !defined(MA_NO_MP3)
64700 | static size_t ma_mp3_dr_callback__read(void* pUserData, void* pBufferOut, size_t bytesToRead)
64701 | {
64702 |     ma_mp3* pMP3 = (ma_mp3*)pUserData;
64703 |     ma_result result;
64704 |     size_t bytesRead;
64705 |
64706 |     MA_ASSERT(pMP3 != NULL);
64707 |
64708 |     result = pMP3->onRead(pMP3->pReadSeekTellUserData, pBufferOut, bytesToRead, &bytesRead);
64709 |     (void)result;
64710 |
64711 |     return bytesRead;
64712 | }
64713 |
64714 | static ma_bool32 ma_mp3_dr_callback__seek(void* pUserData, int offset, ma_dr_mp3_seek_origin origin)
64715 | {
64716 |     ma_mp3* pMP3 = (ma_mp3*)pUserData;
64717 |     ma_result result;
64718 |     ma_seek_origin maSeekOrigin;
64719 |
64720 |     MA_ASSERT(pMP3 != NULL);
64721 |
64722 |     if (origin == MA_DR_MP3_SEEK_SET) {
64723 |         maSeekOrigin = ma_seek_origin_start;
64724 |     } else if (origin == MA_DR_MP3_SEEK_END) {
64725 |         maSeekOrigin = ma_seek_origin_end;
64726 |     } else {
64727 |         maSeekOrigin = ma_seek_origin_current;
64728 |     }
64729 |
64730 |     result = pMP3->onSeek(pMP3->pReadSeekTellUserData, offset, maSeekOrigin);
64731 |     if (result != MA_SUCCESS) {
64732 |         return MA_FALSE;
64733 |     }
64734 | }

```

```

64735 |     return MA_TRUE;
64736 | }
64737 |
64738 | static ma_bool32 ma_mp3_dr_callback_tell(void* pUserData, ma_int64* pCursor)
64739 | {
64740 |     ma_mp3* pMP3 = (ma_mp3*)pUserData;
64741 |     ma_result result;
64742 |
64743 |     MA_ASSERT(pMP3 != NULL);
64744 |
64745 |     result = pMP3->onTell(pMP3->pReadSeekTellUserData, pCursor);
64746 |     if (result != MA_SUCCESS) {
64747 |         return MA_FALSE;
64748 |     }
64749 |
64750 |     return MA_TRUE;
64751 | }
64752 | #endif
64753 |
64754 | static ma_result ma_mp3_init_internal(const ma_decoding_backend_config* pConfig, ma_mp3* pMP3)
64755 | {
64756 |     ma_result result;
64757 |     ma_data_source_config dataSourceConfig;
64758 |
64759 |     if (pMP3 == NULL) {
64760 |         return MA_INVALID_ARGS;
64761 |     }
64762 |
64763 |     MA_ZERO_OBJECT(pMP3);
64764 |     pMP3->format = ma_format_f32; /* f32 by default. */
64765 |
64766 |     if (pConfig != NULL && (pConfig->preferredFormat == ma_format_f32 || pConfig->preferredFormat ==
> ma_format_s16)) {
64767 |         pMP3->format = pConfig->preferredFormat;
64768 |     } else {
64769 |         /* Getting here means something other than f32 and s16 was specified. Just leave this unset
> to use the default format. */
64770 |     }
64771 |
64772 |     dataSourceConfig = ma_data_source_config_init();
64773 |     dataSourceConfig.vtable = &g_ma_mp3_ds_vtable;
64774 |
64775 |     result = ma_data_source_init(&dataSourceConfig, &pMP3->ds);
64776 |     if (result != MA_SUCCESS) {
64777 |         return result; /* Failed to initialize the base data source. */
64778 |     }
64779 |
64780 |     return MA_SUCCESS;
64781 | }
64782 |
64783 | static ma_result ma_mp3_generate_seek_table(ma_mp3* pMP3, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks)
64784 | {
64785 |     ma_bool32 mp3Result;
64786 |     ma_uint32 seekPointCount = 0;
64787 |     ma_dr_mp3_seek_point* pSeekPoints = NULL;
64788 |
64789 |     MA_ASSERT(pMP3 != NULL);
64790 |     MA_ASSERT(pConfig != NULL);
64791 |
64792 |     seekPointCount = pConfig->seekPointCount;
64793 |     if (seekPointCount > 0) {
64794 |         pSeekPoints = (ma_dr_mp3_seek_point*)ma_malloc(sizeof(*pMP3->pSeekPoints) * seekPointCount,
> pAllocationCallbacks);
64795 |         if (pSeekPoints == NULL) {
64796 |             return MA_OUT_OF_MEMORY;
64797 |         }
64798 |     }
64799 |
64800 |     mp3Result = ma_dr_mp3_calculate_seek_points(&pMP3->dr, &seekPointCount, pSeekPoints);
64801 |     if (mp3Result != MA_TRUE) {
64802 |         ma_free(pSeekPoints, pAllocationCallbacks);
64803 |         return MA_ERROR;
64804 |     }
64805 |
64806 |     mp3Result = ma_dr_mp3_bind_seek_table(&pMP3->dr, seekPointCount, pSeekPoints);
64807 |     if (mp3Result != MA_TRUE) {
64808 |         ma_free(pSeekPoints, pAllocationCallbacks);
64809 |         return MA_ERROR;

```

```

64810 |     }
64811 |
64812 |     pMP3->seekPointCount = seekPointCount;
64813 |     pMP3->pSeekPoints    = pSeekPoints;
64814 |
64815 |     return MA_SUCCESS;
64816 | }
64817 |
64818 | static ma_result ma_mp3_post_init(ma_mp3* pMP3, const ma_decoding_backend_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks)
64819 | {
64820 |     ma_result result;
64821 |
64822 |     result = ma_mp3_generate_seek_table(pMP3, pConfig, pAllocationCallbacks);
64823 |     if (result != MA_SUCCESS) {
64824 |         return result;
64825 |     }
64826 |
64827 |     return MA_SUCCESS;
64828 | }
64829 |
64830 | MA_API ma_result ma_mp3_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell, void*
> pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_mp3* pMP3)
64831 | {
64832 |     ma_result result;
64833 |
64834 |     result = ma_mp3_init_internal(pConfig, pMP3);
64835 |     if (result != MA_SUCCESS) {
64836 |         return result;
64837 |     }
64838 |
64839 |     if (onRead == NULL || onSeek == NULL) {
64840 |         return MA_INVALID_ARGS; /* onRead and onSeek are mandatory. */
64841 |     }
64842 |
64843 |     pMP3->onRead = onRead;
64844 |     pMP3->onSeek = onSeek;
64845 |     pMP3->onTell = onTell;
64846 |     pMP3->pReadSeekTellUserData = pReadSeekTellUserData;
64847 |
64848 |     #if !defined(MA_NO_MP3)
64849 |     {
64850 |         ma_bool32 mp3Result;
64851 |
64852 |         mp3Result = ma_dr_mp3_init(&pMP3->dr, ma_mp3_dr_callback__read, ma_mp3_dr_callback__seek,
> ma_mp3_dr_callback__tell, NULL, pMP3, pAllocationCallbacks);
64853 |         if (mp3Result != MA_TRUE) {
64854 |             return MA_INVALID_FILE;
64855 |         }
64856 |
64857 |         ma_mp3_post_init(pMP3, pConfig, pAllocationCallbacks);
64858 |
64859 |         return MA_SUCCESS;
64860 |     }
64861 |     #else
64862 |     {
64863 |         /* mp3 is disabled. */
64864 |         (void)pAllocationCallbacks;
64865 |         return MA_NOT_IMPLEMENTED;
64866 |     }
64867 |     #endif
64868 | }
64869 |
64870 | MA_API ma_result ma_mp3_init_file(const char* pFilePath, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_mp3* pMP3)
64871 | {
64872 |     ma_result result;
64873 |
64874 |     result = ma_mp3_init_internal(pConfig, pMP3);
64875 |     if (result != MA_SUCCESS) {
64876 |         return result;
64877 |     }
64878 |
64879 |     #if !defined(MA_NO_MP3)
64880 |     {
64881 |         ma_bool32 mp3Result;
64882 |
64883 |         mp3Result = ma_dr_mp3_init_file(&pMP3->dr, pFilePath, pAllocationCallbacks);

```



```

64884 |         if (mp3Result != MA_TRUE) {
64885 |             return MA_INVALID_FILE;
64886 |         }
64887 |
64888 |         ma_mp3_post_init(pMP3, pConfig, pAllocationCallbacks);
64889 |
64890 |         return MA_SUCCESS;
64891 |     }
64892 |     #else
64893 |     {
64894 |         /* mp3 is disabled. */
64895 |         (void)pFilePath;
64896 |         (void)pAllocationCallbacks;
64897 |         return MA_NOT_IMPLEMENTED;
64898 |     }
64899 |     #endif
64900 | }
64901 |
64902 | MA_API ma_result ma_mp3_init_file_w(const wchar_t* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_mp3* pMP3)
64903 | {
64904 |     ma_result result;
64905 |
64906 |     result = ma_mp3_init_internal(pConfig, pMP3);
64907 |     if (result != MA_SUCCESS) {
64908 |         return result;
64909 |     }
64910 |
64911 |     #if !defined(MA_NO_MP3)
64912 |     {
64913 |         ma_bool32 mp3Result;
64914 |
64915 |         mp3Result = ma_dr_mp3_init_file_w(&pMP3->dr, pFilePath, pAllocationCallbacks);
64916 |         if (mp3Result != MA_TRUE) {
64917 |             return MA_INVALID_FILE;
64918 |         }
64919 |
64920 |         ma_mp3_post_init(pMP3, pConfig, pAllocationCallbacks);
64921 |
64922 |         return MA_SUCCESS;
64923 |     }
64924 |     #else
64925 |     {
64926 |         /* mp3 is disabled. */
64927 |         (void)pFilePath;
64928 |         (void)pAllocationCallbacks;
64929 |         return MA_NOT_IMPLEMENTED;
64930 |     }
64931 |     #endif
64932 | }
64933 |
64934 | MA_API ma_result ma_mp3_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_mp3*
> pMP3)
64935 | {
64936 |     ma_result result;
64937 |
64938 |     result = ma_mp3_init_internal(pConfig, pMP3);
64939 |     if (result != MA_SUCCESS) {
64940 |         return result;
64941 |     }
64942 |
64943 |     #if !defined(MA_NO_MP3)
64944 |     {
64945 |         ma_bool32 mp3Result;
64946 |
64947 |         mp3Result = ma_dr_mp3_init_memory(&pMP3->dr, pData, dataSize, pAllocationCallbacks);
64948 |         if (mp3Result != MA_TRUE) {
64949 |             return MA_INVALID_FILE;
64950 |         }
64951 |
64952 |         ma_mp3_post_init(pMP3, pConfig, pAllocationCallbacks);
64953 |
64954 |         return MA_SUCCESS;
64955 |     }
64956 |     #else
64957 |     {
64958 |         /* mp3 is disabled. */
64959 |         (void)pData;

```

```

64960 |         (void)dataSize;
64961 |         (void)pAllocationCallbacks;
64962 |         return MA_NOT_IMPLEMENTED;
64963 |     }
64964 | #endif
64965 | }
64966 |
64967 | MA_API void ma_mp3_uninit(ma_mp3* pMP3, const ma_allocation_callbacks* pAllocationCallbacks)
64968 | {
64969 |     if (pMP3 == NULL) {
64970 |         return;
64971 |     }
64972 |
64973 |     #if !defined(MA_NO_MP3)
64974 |     {
64975 |         ma_dr_mp3_uninit(&pMP3->dr);
64976 |     }
64977 |     #else
64978 |     {
64979 |         /* mp3 is disabled. Should never hit this since initialization would have failed. */
64980 |         MA_ASSERT(MA_FALSE);
64981 |     }
64982 |     #endif
64983 |
64984 |     /* Seek points need to be freed after the MP3 decoder has been uninitialized to ensure they're
> no longer being referenced. */
64985 |     ma_free(pMP3->pSeekPoints, pAllocationCallbacks);
64986 |
64987 |     ma_data_source_uninit(&pMP3->ds);
64988 | }
64989 |
64990 | MA_API ma_result ma_mp3_read_pcm_frames(ma_mp3* pMP3, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead)
64991 | {
64992 |     if (pFramesRead != NULL) {
64993 |         *pFramesRead = 0;
64994 |     }
64995 |
64996 |     if (frameCount == 0) {
64997 |         return MA_INVALID_ARGS;
64998 |     }
64999 |
65000 |     if (pMP3 == NULL) {
65001 |         return MA_INVALID_ARGS;
65002 |     }
65003 |
65004 |     #if !defined(MA_NO_MP3)
65005 |     {
65006 |         /* We always use floating point format. */
65007 |         ma_result result = MA_SUCCESS; /* Must be initialized to MA_SUCCESS. */
65008 |         ma_uint64 totalFramesRead = 0;
65009 |         ma_format format;
65010 |
65011 |         ma_mp3_get_data_format(pMP3, &format, NULL, NULL, NULL, 0);
65012 |
65013 |         switch (format)
65014 |         {
65015 |             case ma_format_f32:
65016 |             {
65017 |                 totalFramesRead = ma_dr_mp3_read_pcm_frames_f32(&pMP3->dr, frameCount,
> (float*)pFramesOut);
65018 |                 } break;
65019 |
65020 |             case ma_format_s16:
65021 |             {
65022 |                 totalFramesRead = ma_dr_mp3_read_pcm_frames_s16(&pMP3->dr, frameCount,
> (ma_int16*)pFramesOut);
65023 |                 } break;
65024 |
65025 |             case ma_format_u8:
65026 |             case ma_format_s24:
65027 |             case ma_format_s32:
65028 |             case ma_format_unknown:
65029 |             default:
65030 |             {
65031 |                 return MA_INVALID_OPERATION;
65032 |             };
65033 |         }
65034 |     }

```

```

65035 |         /* In the future we'll update ma_dr_mp3 to return MA_AT_END for us. */
65036 |         if (totalFramesRead == 0) {
65037 |             result = MA_AT_END;
65038 |         }
65039 |
65040 |         if (pFramesRead != NULL) {
65041 |             *pFramesRead = totalFramesRead;
65042 |         }
65043 |
65044 |         return result;
65045 |     }
65046 |     #else
65047 |     {
65048 |         /* mp3 is disabled. Should never hit this since initialization would have failed. */
65049 |         MA_ASSERT(MA_FALSE);
65050 |
65051 |         (void)pFramesOut;
65052 |         (void)frameCount;
65053 |         (void)pFramesRead;
65054 |
65055 |         return MA_NOT_IMPLEMENTED;
65056 |     }
65057 |     #endif
65058 | }
65059 |
65060 | MA_API ma_result ma_mp3_seek_to_pcm_frame(ma_mp3* pMP3, ma_uint64 frameIndex)
65061 | {
65062 |     if (pMP3 == NULL) {
65063 |         return MA_INVALID_ARGS;
65064 |     }
65065 |
65066 |     #if !defined(MA_NO_MP3)
65067 |     {
65068 |         ma_bool32 mp3Result;
65069 |
65070 |         mp3Result = ma_dr_mp3_seek_to_pcm_frame(&pMP3->dr, frameIndex);
65071 |         if (mp3Result != MA_TRUE) {
65072 |             return MA_ERROR;
65073 |         }
65074 |
65075 |         return MA_SUCCESS;
65076 |     }
65077 |     #else
65078 |     {
65079 |         /* mp3 is disabled. Should never hit this since initialization would have failed. */
65080 |         MA_ASSERT(MA_FALSE);
65081 |
65082 |         (void)frameIndex;
65083 |
65084 |         return MA_NOT_IMPLEMENTED;
65085 |     }
65086 |     #endif
65087 | }
65088 |
65089 | MA_API ma_result ma_mp3_get_data_format(ma_mp3* pMP3, ma_format* pFormat, ma_uint32* pChannels,
> ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
65090 | {
65091 |     /* Defaults for safety. */
65092 |     if (pFormat != NULL) {
65093 |         *pFormat = ma_format_unknown;
65094 |     }
65095 |     if (pChannels != NULL) {
65096 |         *pChannels = 0;
65097 |     }
65098 |     if (pSampleRate != NULL) {
65099 |         *pSampleRate = 0;
65100 |     }
65101 |     if (pChannelMap != NULL) {
65102 |         MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channelMapCap);
65103 |     }
65104 |
65105 |     if (pMP3 == NULL) {
65106 |         return MA_INVALID_OPERATION;
65107 |     }
65108 |
65109 |     if (pFormat != NULL) {
65110 |         *pFormat = pMP3->format;
65111 |     }
65112 | }

```

```

65113 |     #if !defined(MA_NO_MP3)
65114 |     {
65115 |         if (pChannels != NULL) {
65116 |             *pChannels = pMP3->dr.channels;
65117 |         }
65118 |
65119 |         if (pSampleRate != NULL) {
65120 |             *pSampleRate = pMP3->dr.sampleRate;
65121 |         }
65122 |
65123 |         if (pChannelMap != NULL) {
65124 |             ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap,
> channelMapCap, pMP3->dr.channels);
65125 |         }
65126 |
65127 |         return MA_SUCCESS;
65128 |     }
65129 |     #else
65130 |     {
65131 |         /* mp3 is disabled. Should never hit this since initialization would have failed. */
65132 |         MA_ASSERT(MA_FALSE);
65133 |         return MA_NOT_IMPLEMENTED;
65134 |     }
65135 |     #endif
65136 | }
65137 |
65138 | MA_API ma_result ma_mp3_get_cursor_in_pcm_frames(ma_mp3* pMP3, ma_uint64* pCursor)
65139 | {
65140 |     if (pCursor == NULL) {
65141 |         return MA_INVALID_ARGS;
65142 |     }
65143 |
65144 |     *pCursor = 0;    /* Safety. */
65145 |
65146 |     if (pMP3 == NULL) {
65147 |         return MA_INVALID_ARGS;
65148 |     }
65149 |
65150 |     #if !defined(MA_NO_MP3)
65151 |     {
65152 |         *pCursor = pMP3->dr.currentPCMFrame;
65153 |
65154 |         return MA_SUCCESS;
65155 |     }
65156 |     #else
65157 |     {
65158 |         /* mp3 is disabled. Should never hit this since initialization would have failed. */
65159 |         MA_ASSERT(MA_FALSE);
65160 |         return MA_NOT_IMPLEMENTED;
65161 |     }
65162 |     #endif
65163 | }
65164 |
65165 | MA_API ma_result ma_mp3_get_length_in_pcm_frames(ma_mp3* pMP3, ma_uint64* pLength)
65166 | {
65167 |     if (pLength == NULL) {
65168 |         return MA_INVALID_ARGS;
65169 |     }
65170 |
65171 |     *pLength = 0;    /* Safety. */
65172 |
65173 |     if (pMP3 == NULL) {
65174 |         return MA_INVALID_ARGS;
65175 |     }
65176 |
65177 |     #if !defined(MA_NO_MP3)
65178 |     {
65179 |         *pLength = ma_dr_mp3_get_pcm_frame_count(&pMP3->dr);
65180 |
65181 |         return MA_SUCCESS;
65182 |     }
65183 |     #else
65184 |     {
65185 |         /* mp3 is disabled. Should never hit this since initialization would have failed. */
65186 |         MA_ASSERT(MA_FALSE);
65187 |         return MA_NOT_IMPLEMENTED;
65188 |     }
65189 |     #endif
65190 | }

```

```
65191 |
65192 |
65193 | static ma_result ma_decoding_backend_init_mp3(void* pUserData, ma_read_proc onRead, ma_seek_proc
> onSeek, ma_tell_proc onTell, void* pReadSeekTellUserData, const ma_decoding_backend_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_data_source** ppBackend)
65194 | {
65195 |     ma_result result;
65196 |     ma_mp3* pMP3;
65197 |
65198 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
65199 |
65200 |     /* For now we're just allocating the decoder backend on the heap. */
65201 |     pMP3 = (ma_mp3*)ma_malloc(sizeof(*pMP3), pAllocationCallbacks);
65202 |     if (pMP3 == NULL) {
65203 |         return MA_OUT_OF_MEMORY;
65204 |     }
65205 |
65206 |     result = ma_mp3_init(onRead, onSeek, onTell, pReadSeekTellUserData, pConfig,
> pAllocationCallbacks, pMP3);
65207 |     if (result != MA_SUCCESS) {
65208 |         ma_free(pMP3, pAllocationCallbacks);
65209 |         return result;
65210 |     }
65211 |
65212 |     *ppBackend = pMP3;
65213 |
65214 |     return MA_SUCCESS;
65215 | }
65216 |
65217 | static ma_result ma_decoding_backend_init_file_mp3(void* pUserData, const char* pFilePath, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
65218 | {
65219 |     ma_result result;
65220 |     ma_mp3* pMP3;
65221 |
65222 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
65223 |
65224 |     /* For now we're just allocating the decoder backend on the heap. */
65225 |     pMP3 = (ma_mp3*)ma_malloc(sizeof(*pMP3), pAllocationCallbacks);
65226 |     if (pMP3 == NULL) {
65227 |         return MA_OUT_OF_MEMORY;
65228 |     }
65229 |
65230 |     result = ma_mp3_init_file(pFilePath, pConfig, pAllocationCallbacks, pMP3);
65231 |     if (result != MA_SUCCESS) {
65232 |         ma_free(pMP3, pAllocationCallbacks);
65233 |         return result;
65234 |     }
65235 |
65236 |     *ppBackend = pMP3;
65237 |
65238 |     return MA_SUCCESS;
65239 | }
65240 |
65241 | static ma_result ma_decoding_backend_init_file_w_mp3(void* pUserData, const wchar_t* pFilePath,
> const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
65242 | {
65243 |     ma_result result;
65244 |     ma_mp3* pMP3;
65245 |
65246 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
65247 |
65248 |     /* For now we're just allocating the decoder backend on the heap. */
65249 |     pMP3 = (ma_mp3*)ma_malloc(sizeof(*pMP3), pAllocationCallbacks);
65250 |     if (pMP3 == NULL) {
65251 |         return MA_OUT_OF_MEMORY;
65252 |     }
65253 |
65254 |     result = ma_mp3_init_file_w(pFilePath, pConfig, pAllocationCallbacks, pMP3);
65255 |     if (result != MA_SUCCESS) {
65256 |         ma_free(pMP3, pAllocationCallbacks);
65257 |         return result;
65258 |     }
65259 | }
```

```

65260 |     *ppBackend = pMP3;
65261 |
65262 |     return MA_SUCCESS;
65263 | }
65264 |
65265 | static ma_result ma_decoding_backend_init_memory__mp3(void* pUserData, const void* pData, size_t
> dataSize, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_data_source** ppBackend)
65266 | {
65267 |     ma_result result;
65268 |     ma_mp3* pMP3;
65269 |
65270 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
65271 |
65272 |     /* For now we're just allocating the decoder backend on the heap. */
65273 |     pMP3 = (ma_mp3*)ma_malloc(sizeof(*pMP3), pAllocationCallbacks);
65274 |     if (pMP3 == NULL) {
65275 |         return MA_OUT_OF_MEMORY;
65276 |     }
65277 |
65278 |     result = ma_mp3_init_memory(pData, dataSize, pConfig, pAllocationCallbacks, pMP3);
65279 |     if (result != MA_SUCCESS) {
65280 |         ma_free(pMP3, pAllocationCallbacks);
65281 |         return result;
65282 |     }
65283 |
65284 |     *ppBackend = pMP3;
65285 |
65286 |     return MA_SUCCESS;
65287 | }
65288 |
65289 | static void ma_decoding_backend_uninit__mp3(void* pUserData, ma_data_source* pBackend, const
> ma_allocation_callbacks* pAllocationCallbacks)
65290 | {
65291 |     ma_mp3* pMP3 = (ma_mp3*)pBackend;
65292 |
65293 |     (void)pUserData;
65294 |
65295 |     ma_mp3_uninit(pMP3, pAllocationCallbacks);
65296 |     ma_free(pMP3, pAllocationCallbacks);
65297 | }
65298 |
65299 | static ma_decoding_backend_vtable g_ma_decoding_backend_vtable_mp3 =
65300 | {
65301 |     ma_decoding_backend_init__mp3,
65302 |     ma_decoding_backend_init_file__mp3,
65303 |     ma_decoding_backend_init_file_w__mp3,
65304 |     ma_decoding_backend_init_memory__mp3,
65305 |     ma_decoding_backend_uninit__mp3
65306 | };
65307 |
65308 | static ma_result ma_decoder_init_mp3__internal(const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
65309 | {
65310 |     return ma_decoder_init_from_vtable__internal(&g_ma_decoding_backend_vtable_mp3, NULL, pConfig,
> pDecoder);
65311 | }
65312 |
65313 | static ma_result ma_decoder_init_mp3_from_file__internal(const char* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
65314 | {
65315 |     return ma_decoder_init_from_file__internal(&g_ma_decoding_backend_vtable_mp3, NULL, pFilePath,
> pConfig, pDecoder);
65316 | }
65317 |
65318 | static ma_result ma_decoder_init_mp3_from_file_w__internal(const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
65319 | {
65320 |     return ma_decoder_init_from_file_w__internal(&g_ma_decoding_backend_vtable_mp3, NULL, pFilePath,
> pConfig, pDecoder);
65321 | }
65322 |
65323 | static ma_result ma_decoder_init_mp3_from_memory__internal(const void* pData, size_t dataSize, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
65324 | {
65325 |     return ma_decoder_init_from_memory__internal(&g_ma_decoding_backend_vtable_mp3, NULL, pData,
> dataSize, pConfig, pDecoder);
65326 | }

```

```

65327 | #endif /* ma_dr_mp3_h */
65328 |
65329 | /* Vorbis */
65330 | #ifdef STB_VORBIS_INCLUDE_STB_VORBIS_H
65331 | #define MA_HAS_VORBIS
65332 |
65333 | /* The size in bytes of each chunk of data to read from the Vorbis stream. */
65334 | #define MA_VORBIS_DATA_CHUNK_SIZE 4096
65335 |
65336 | typedef struct
65337 | {
65338 |     ma_data_source_base ds;
65339 |     ma_read_proc onRead;
65340 |     ma_seek_proc onSeek;
65341 |     ma_tell_proc onTell;
65342 |     void* pReadSeekTellUserData;
65343 |     ma_allocation_callbacks allocationCallbacks; /* Store the allocation callbacks within the
> structure because we may need to dynamically expand a buffer in ma_stbvorbis_read_pcm_frames() when
> using push mode. */
65344 |     ma_format format; /* Only f32 is allowed with stb_vorbis. */
65345 |     ma_uint32 channels;
65346 |     ma_uint32 sampleRate;
65347 |     ma_uint64 cursor;
65348 |     #if !defined(MA_NO_VORBIS)
65349 |     stb_vorbis* stb;
65350 |     ma_bool32 usingPushMode;
65351 |     struct
65352 |     {
65353 |         ma_uint8* pData;
65354 |         size_t dataSize;
65355 |         size_t dataCapacity;
65356 |         size_t audioStartOffsetInBytes;
65357 |         ma_uint32 framesConsumed; /* The number of frames consumed in ppPacketData. */
65358 |         ma_uint32 framesRemaining; /* The number of frames remaining in ppPacketData. */
65359 |         float** ppPacketData;
65360 |     } push;
65361 | #endif
65362 | } ma_stbvorbis;
65363 |
65364 | MA_API ma_result ma_stbvorbis_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell,
> void* pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_stbvorbis* pVorbis);
65365 | MA_API ma_result ma_stbvorbis_init_file(const char* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_stbvorbis* pVorbis);
65366 | MA_API ma_result ma_stbvorbis_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_stbvorbis* pVorbis);
65367 | MA_API void ma_stbvorbis_uninit(ma_stbvorbis* pVorbis, const ma_allocation_callbacks*
> pAllocationCallbacks);
65368 | MA_API ma_result ma_stbvorbis_read_pcm_frames(ma_stbvorbis* pVorbis, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead);
65369 | MA_API ma_result ma_stbvorbis_seek_to_pcm_frame(ma_stbvorbis* pVorbis, ma_uint64 frameIndex);
65370 | MA_API ma_result ma_stbvorbis_get_data_format(ma_stbvorbis* pVorbis, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap);
65371 | MA_API ma_result ma_stbvorbis_get_cursor_in_pcm_frames(ma_stbvorbis* pVorbis, ma_uint64* pCursor);
65372 | MA_API ma_result ma_stbvorbis_get_length_in_pcm_frames(ma_stbvorbis* pVorbis, ma_uint64* pLength);
65373 |
65374 |
65375 | static ma_result ma_stbvorbis_ds_read(ma_data_source* pDataSource, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
65376 | {
65377 |     return ma_stbvorbis_read_pcm_frames((ma_stbvorbis*)pDataSource, pFramesOut, frameCount,
> pFramesRead);
65378 | }
65379 |
65380 | static ma_result ma_stbvorbis_ds_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
65381 | {
65382 |     return ma_stbvorbis_seek_to_pcm_frame((ma_stbvorbis*)pDataSource, frameIndex);
65383 | }
65384 |
65385 | static ma_result ma_stbvorbis_ds_get_data_format(ma_data_source* pDataSource, ma_format* pFormat,
> ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
65386 | {
65387 |     return ma_stbvorbis_get_data_format((ma_stbvorbis*)pDataSource, pFormat, pChannels, pSampleRate,
> pChannelMap, channelMapCap);
65388 | }
65389 |
65390 | static ma_result ma_stbvorbis_ds_get_cursor(ma_data_source* pDataSource, ma_uint64* pCursor)
65391 | {

```



```

65392 |     return ma_stbvorbis_get_cursor_in_pcm_frames((ma_stbvorbis*)pDataSource, pCursor);
65393 | }
65394 |
65395 | static ma_result ma_stbvorbis_ds_get_length(ma_data_source* pDataSource, ma_uint64* pLength)
65396 | {
65397 |     return ma_stbvorbis_get_length_in_pcm_frames((ma_stbvorbis*)pDataSource, pLength);
65398 | }
65399 |
65400 | static ma_data_source_vtable g_ma_stbvorbis_ds_vtable =
65401 | {
65402 |     ma_stbvorbis_ds_read,
65403 |     ma_stbvorbis_ds_seek,
65404 |     ma_stbvorbis_ds_get_data_format,
65405 |     ma_stbvorbis_ds_get_cursor,
65406 |     ma_stbvorbis_ds_get_length,
65407 |     NULL, /* onSetLooping */
65408 |     0
65409 | };
65410 |
65411 |
65412 | static ma_result ma_stbvorbis_init_internal(const ma_decoding_backend_config* pConfig, ma_stbvorbis*
> pVorbis)
65413 | {
65414 |     ma_result result;
65415 |     ma_data_source_config dataSourceConfig;
65416 |
65417 |     (void)pConfig;
65418 |
65419 |     if (pVorbis == NULL) {
65420 |         return MA_INVALID_ARGS;
65421 |     }
65422 |
65423 |     MA_ZERO_OBJECT(pVorbis);
65424 |     pVorbis->format = ma_format_f32; /* Only supporting f32. */
65425 |
65426 |     dataSourceConfig = ma_data_source_config_init();
65427 |     dataSourceConfig.vtable = &g_ma_stbvorbis_ds_vtable;
65428 |
65429 |     result = ma_data_source_init(&dataSourceConfig, &pVorbis->ds);
65430 |     if (result != MA_SUCCESS) {
65431 |         return result; /* Failed to initialize the base data source. */
65432 |     }
65433 |
65434 |     return MA_SUCCESS;
65435 | }
65436 |
65437 | #if !defined(MA_NO_VORBIS)
65438 | static ma_result ma_stbvorbis_post_init(ma_stbvorbis* pVorbis)
65439 | {
65440 |     stb_vorbis_info info;
65441 |
65442 |     MA_ASSERT(pVorbis != NULL);
65443 |
65444 |     info = stb_vorbis_get_info(pVorbis->stb);
65445 |
65446 |     pVorbis->channels = info.channels;
65447 |     pVorbis->sampleRate = info.sample_rate;
65448 |
65449 |     return MA_SUCCESS;
65450 | }
65451 |
65452 | static ma_result ma_stbvorbis_init_internal_decoder_push(ma_stbvorbis* pVorbis)
65453 | {
65454 |     ma_result result;
65455 |     stb_vorbis* stb;
65456 |     size_t dataSize = 0;
65457 |     size_t dataCapacity = 0;
65458 |     ma_uint8* pData = NULL; /* <-- Must be initialized to NULL. */
65459 |
65460 |     for (;;) {
65461 |         int vorbisError;
65462 |         int consumedDataSize; /* <-- Fill by stb_vorbis_open_pushdata(). */
65463 |         size_t bytesRead;
65464 |         ma_uint8* pNewData;
65465 |
65466 |         /* Allocate memory for the new chunk. */
65467 |         dataCapacity += MA_VORBIS_DATA_CHUNK_SIZE;
65468 |         pNewData = (ma_uint8*)ma_realloc(pData, dataCapacity, &pVorbis->allocationCallbacks);
65469 |         if (pNewData == NULL) {

```

```

65470 |         ma_free(pData, &pVorbis->allocationCallbacks);
65471 |         return MA_OUT_OF_MEMORY;
65472 |     }
65473 |
65474 |     pData = pNewData;
65475 |
65476 |     /* Read in the next chunk. */
65477 |     result = pVorbis->onRead(pVorbis->pReadSeekTellUserData, ma_offset_ptr(pData, dataSize),
> (dataCapacity - dataSize), &bytesRead);
65478 |     dataSize += bytesRead;
65479 |
65480 |     if (result != MA_SUCCESS) {
65481 |         ma_free(pData, &pVorbis->allocationCallbacks);
65482 |         return result;
65483 |     }
65484 |
65485 |     /* We have a maximum of 31 bits with stb_vorbis. */
65486 |     if (dataSize > INT_MAX) {
65487 |         ma_free(pData, &pVorbis->allocationCallbacks);
65488 |         return MA_TOO_BIG;
65489 |     }
65490 |
65491 |     stb = stb_vorbis_open_pushdata(pData, (int)dataSize, &consumedDataSize, &vorbisError, NULL);
65492 |     if (stb != NULL) {
65493 |         /*
65494 |         Successfully opened the Vorbis decoder. We might have some leftover unprocessed
65495 |         data so we'll need to move that down to the front.
65496 |         */
65497 |         dataSize -= (size_t)consumedDataSize; /* Consume the data. */
65498 |         MA_MOVE_MEMORY(pData, ma_offset_ptr(pData, consumedDataSize), dataSize);
65499 |
65500 |         /*
65501 |         We need to track the start point so we can seek back to the start of the audio
65502 |         data when seeking.
65503 |         */
65504 |         pVorbis->push.audioStartOffsetInBytes = consumedDataSize;
65505 |
65506 |         break;
65507 |     } else {
65508 |         /* Failed to open the decoder. */
65509 |         if (vorbisError == VORBIS_need_more_data) {
65510 |             continue;
65511 |         } else {
65512 |             ma_free(pData, &pVorbis->allocationCallbacks);
65513 |             return MA_ERROR; /* Failed to open the stb_vorbis decoder. */
65514 |         }
65515 |     }
65516 | }
65517 |
65518 | MA_ASSERT(stb != NULL);
65519 | pVorbis->stb = stb;
65520 | pVorbis->push.pData = pData;
65521 | pVorbis->push.dataSize = dataSize;
65522 | pVorbis->push.dataCapacity = dataCapacity;
65523 |
65524 | return MA_SUCCESS;
65525 | }
65526 | #endif
65527 |
65528 | MA_API ma_result ma_stbvorbis_init(ma_read_proc onRead, ma_seek_proc onSeek, ma_tell_proc onTell,
> void* pReadSeekTellUserData, const ma_decoding_backend_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_stbvorbis* pVorbis)
65529 | {
65530 |     ma_result result;
65531 |
65532 |     result = ma_stbvorbis_init_internal(pConfig, pVorbis);
65533 |     if (result != MA_SUCCESS) {
65534 |         return result;
65535 |     }
65536 |
65537 |     if (onRead == NULL || onSeek == NULL) {
65538 |         return MA_INVALID_ARGS; /* onRead and onSeek are mandatory. */
65539 |     }
65540 |
65541 |     pVorbis->onRead = onRead;
65542 |     pVorbis->onSeek = onSeek;
65543 |     pVorbis->onTell = onTell;
65544 |     pVorbis->pReadSeekTellUserData = pReadSeekTellUserData;
65545 |     ma_allocation_callbacks_init_copy(&pVorbis->allocationCallbacks, pAllocationCallbacks);

```

```

65546 |
65547 |     #if !defined(MA_NO_VORBIS)
65548 |     {
65549 |         /*
65550 |         stb_vorbis lacks a callback based API for its pulling API which means we're stuck with the
65551 |         pushing API. In order for us to be able to successfully initialize the decoder we need to
65552 |         supply it with enough data. We need to keep loading data until we have enough.
65553 |         */
65554 |         result = ma_stbvorbis_init_internal_decoder_push(pVorbis);
65555 |         if (result != MA_SUCCESS) {
65556 |             return result;
65557 |         }
65558 |
65559 |         pVorbis->usingPushMode = MA_TRUE;
65560 |
65561 |         result = ma_stbvorbis_post_init(pVorbis);
65562 |         if (result != MA_SUCCESS) {
65563 |             stb_vorbis_close(pVorbis->stb);
65564 |             ma_free(pVorbis->push.pData, pAllocationCallbacks);
65565 |             return result;
65566 |         }
65567 |
65568 |         return MA_SUCCESS;
65569 |     }
65570 |     #else
65571 |     {
65572 |         /* vorbis is disabled. */
65573 |         (void)pAllocationCallbacks;
65574 |         return MA_NOT_IMPLEMENTED;
65575 |     }
65576 |     #endif
65577 | }
65578 |
65579 | MA_API ma_result ma_stbvorbis_init_file(const char* pFilePath, const ma_decoding_backend_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_stbvorbis* pVorbis)
65580 | {
65581 |     ma_result result;
65582 |
65583 |     result = ma_stbvorbis_init_internal(pConfig, pVorbis);
65584 |     if (result != MA_SUCCESS) {
65585 |         return result;
65586 |     }
65587 |
65588 |     #if !defined(MA_NO_VORBIS)
65589 |     {
65590 |         (void)pAllocationCallbacks; /* Don't know how to make use of this with stb_vorbis. */
65591 |
65592 |         /* We can use stb_vorbis' pull mode for file based streams. */
65593 |         pVorbis->stb = stb_vorbis_open_filename(pFilePath, NULL, NULL);
65594 |         if (pVorbis->stb == NULL) {
65595 |             return MA_INVALID_FILE;
65596 |         }
65597 |
65598 |         pVorbis->usingPushMode = MA_FALSE;
65599 |
65600 |         result = ma_stbvorbis_post_init(pVorbis);
65601 |         if (result != MA_SUCCESS) {
65602 |             stb_vorbis_close(pVorbis->stb);
65603 |             return result;
65604 |         }
65605 |
65606 |         return MA_SUCCESS;
65607 |     }
65608 |     #else
65609 |     {
65610 |         /* vorbis is disabled. */
65611 |         (void)pFilePath;
65612 |         (void)pAllocationCallbacks;
65613 |         return MA_NOT_IMPLEMENTED;
65614 |     }
65615 |     #endif
65616 | }
65617 |
65618 | MA_API ma_result ma_stbvorbis_init_memory(const void* pData, size_t dataSize, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_stbvorbis* pVorbis)
65619 | {
65620 |     ma_result result;
65621 |

```

```

65622 |     result = ma_stbvorbis_init_internal(pConfig, pVorbis);
65623 |     if (result != MA_SUCCESS) {
65624 |         return result;
65625 |     }
65626 |
65627 |     #if !defined(MA_NO_VORBIS)
65628 |     {
65629 |         (void)pAllocationCallbacks;
65630 |
65631 |         /* stb_vorbis uses an int as its size specifier, restricting it to 32-bit even on 64-bit
> systems. *sigh*. */
65632 |         if (dataSize > INT_MAX) {
65633 |             return MA_TOO_BIG;
65634 |         }
65635 |
65636 |         pVorbis->stb = stb_vorbis_open_memory((const unsigned char*)pData, (int)dataSize, NULL,
> NULL);
65637 |         if (pVorbis->stb == NULL) {
65638 |             return MA_INVALID_FILE;
65639 |         }
65640 |
65641 |         pVorbis->usingPushMode = MA_FALSE;
65642 |
65643 |         result = ma_stbvorbis_post_init(pVorbis);
65644 |         if (result != MA_SUCCESS) {
65645 |             stb_vorbis_close(pVorbis->stb);
65646 |             return result;
65647 |         }
65648 |
65649 |         return MA_SUCCESS;
65650 |     }
65651 |     #else
65652 |     {
65653 |         /* vorbis is disabled. */
65654 |         (void)pData;
65655 |         (void)dataSize;
65656 |         (void)pAllocationCallbacks;
65657 |         return MA_NOT_IMPLEMENTED;
65658 |     }
65659 |     #endif
65660 | }
65661 |
65662 | MA_API void ma_stbvorbis_uninit(ma_stbvorbis* pVorbis, const ma_allocation_callbacks*
> pAllocationCallbacks)
65663 | {
65664 |     if (pVorbis == NULL) {
65665 |         return;
65666 |     }
65667 |
65668 |     #if !defined(MA_NO_VORBIS)
65669 |     {
65670 |         stb_vorbis_close(pVorbis->stb);
65671 |
65672 |         /* We'll have to clear some memory if we're using push mode. */
65673 |         if (pVorbis->usingPushMode) {
65674 |             ma_free(pVorbis->push.pData, pAllocationCallbacks);
65675 |         }
65676 |     }
65677 |     #else
65678 |     {
65679 |         /* vorbis is disabled. Should never hit this since initialization would have failed. */
65680 |         MA_ASSERT(MA_FALSE);
65681 |     }
65682 |     #endif
65683 |
65684 |     ma_data_source_uninit(&pVorbis->ds);
65685 | }
65686 |
65687 | MA_API ma_result ma_stbvorbis_read_pcm_frames(ma_stbvorbis* pVorbis, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
65688 | {
65689 |     if (pFramesRead != NULL) {
65690 |         *pFramesRead = 0;
65691 |     }
65692 |
65693 |     if (frameCount == 0) {
65694 |         return MA_INVALID_ARGS;
65695 |     }
65696 |

```

```

65697 |     if (pVorbis == NULL) {
65698 |         return MA_INVALID_ARGS;
65699 |     }
65700 |
65701 |     #if !defined(MA_NO_VORBIS)
65702 |     {
65703 |         /* We always use floating point format. */
65704 |         ma_result result = MA_SUCCESS; /* Must be initialized to MA_SUCCESS. */
65705 |         ma_uint64 totalFramesRead = 0;
65706 |         ma_format format;
65707 |         ma_uint32 channels;
65708 |
65709 |         ma_stbvorbis_get_data_format(pVorbis, &format, &channels, NULL, NULL, 0);
65710 |
65711 |         if (format == ma_format_f32) {
65712 |             /* We read differently depending on whether or not we're using push mode. */
65713 |             if (pVorbis->usingPushMode) {
65714 |                 /* Push mode. This is the complex case. */
65715 |                 float* pFramesOutF32 = (float*)pFramesOut;
65716 |
65717 |                 while (totalFramesRead < frameCount) {
65718 |                     /* The first thing to do is read from any already-cached frames. */
65719 |                     ma_uint32 framesToReadFromCache =
> (ma_uint32)ma_min(pVorbis->push.framesRemaining, (frameCount - totalFramesRead)); /* Safe cast
> because pVorbis->framesRemaining is 32-bit. */
65720 |
65721 |                     /* The output pointer can be null in which case we just treat it as a seek. */
65722 |                     if (pFramesOut != NULL) {
65723 |                         ma_uint64 iFrame;
65724 |                         for (iFrame = 0; iFrame < framesToReadFromCache; iFrame += 1) {
65725 |                             ma_uint32 iChannel;
65726 |                             for (iChannel = 0; iChannel < pVorbis->channels; iChannel += 1) {
65727 |                                 pFramesOutF32[iChannel] =
> pVorbis->push.ppPacketData[iChannel][pVorbis->push.framesConsumed + iFrame];
65728 |                             }
65729 |
65730 |                             pFramesOutF32 += pVorbis->channels;
65731 |                         }
65732 |                     }
65733 |
65734 |                     /* Update pointers and counters. */
65735 |                     pVorbis->push.framesConsumed += framesToReadFromCache;
65736 |                     pVorbis->push.framesRemaining -= framesToReadFromCache;
65737 |                     totalFramesRead += framesToReadFromCache;
65738 |
65739 |                     /* Don't bother reading any more frames right now if we've just finished
> loading. */
65740 |                     if (totalFramesRead == frameCount) {
65741 |                         break;
65742 |                     }
65743 |
65744 |                     MA_ASSERT(pVorbis->push.framesRemaining == 0);
65745 |
65746 |                     /* Getting here means we've run out of cached frames. We'll need to load some
> more. */
65747 |                     for (;;) {
65748 |                         int samplesRead = 0;
65749 |                         int consumedDataSize;
65750 |
65751 |                         /* We need to case dataSize to an int, so make sure we can do it safely. */
65752 |                         if (pVorbis->push.dataSize > INT_MAX) {
65753 |                             break; /* Too big. */
65754 |                         }
65755 |
65756 |                         consumedDataSize = stb_vorbis_decode_frame_pushdata(pVorbis->stb,
> pVorbis->push.pData, (int)pVorbis->push.dataSize, NULL, &pVorbis->push.ppPacketData, &samplesRead);
65757 |                         if (consumedDataSize != 0) {
65758 |                             /* Successfully decoded a Vorbis frame. Consume the data. */
65759 |                             pVorbis->push.dataSize -= (size_t)consumedDataSize;
65760 |                             MA_MOVE_MEMORY(pVorbis->push.pData, ma_offset_ptr(pVorbis->push.pData,
> consumedDataSize), pVorbis->push.dataSize);
65761 |
65762 |                             pVorbis->push.framesConsumed = 0;
65763 |                             pVorbis->push.framesRemaining = samplesRead;
65764 |
65765 |                             break;
65766 |                         } else {
65767 |                             /* Not enough data. Read more. */
65768 |                             size_t bytesRead;

```

```

65769 |
65770 |             /* Expand the data buffer if necessary. */
65771 |             if (pVorbis->push.dataCapacity == pVorbis->push.dataSize) {
65772 |                 size_t newCap = pVorbis->push.dataCapacity +
> MA_VORBIS_DATA_CHUNK_SIZE;
65773 |                 ma_uint8* pNewData;
65774 |
65775 |                 pNewData = (ma_uint8*)ma_realloc(pVorbis->push.pData, newCap,
> &pVorbis->allocationCallbacks);
65776 |                 if (pNewData == NULL) {
65777 |                     result = MA_OUT_OF_MEMORY;
65778 |                     break;
65779 |                 }
65780 |
65781 |                 pVorbis->push.pData = pNewData;
65782 |                 pVorbis->push.dataCapacity = newCap;
65783 |             }
65784 |
65785 |             /* We should have enough room to load some data. */
65786 |             result = pVorbis->onRead(pVorbis->pReadSeekTellUserData,
> ma_offset_ptr(pVorbis->push.pData, pVorbis->push.dataSize), (pVorbis->push.dataCapacity -
> pVorbis->push.dataSize), &bytesRead);
65787 |             pVorbis->push.dataSize += bytesRead;
65788 |
65789 |             if (result != MA_SUCCESS) {
65790 |                 break; /* Failed to read any data. Get out. */
65791 |             }
65792 |         }
65793 |     }
65794 |
65795 |     /* If we don't have a success code at this point it means we've encountered an
> error or the end of the file has been reached (probably the latter). */
65796 |     if (result != MA_SUCCESS) {
65797 |         break;
65798 |     }
65799 | }
65800 | } else {
65801 |     /* Pull mode. This is the simple case, but we still need to run in a loop because
> stb_vorbis loves using 32-bit instead of 64-bit. */
65802 |     while (totalFramesRead < frameCount) {
65803 |         ma_uint64 framesRemaining = (frameCount - totalFramesRead);
65804 |         int framesRead;
65805 |
65806 |         if (framesRemaining > INT_MAX) {
65807 |             framesRemaining = INT_MAX;
65808 |         }
65809 |
65810 |         framesRead = stb_vorbis_get_samples_float_interleaved(pVorbis->stb, channels,
> (float*)ma_offset_pcm_frames_ptr(pFramesOut, totalFramesRead, format, channels),
> (int)framesRemaining * channels); /* Safe cast. */
65811 |         totalFramesRead += framesRead;
65812 |
65813 |         if (framesRead < (int)framesRemaining) {
65814 |             break; /* Nothing left to read. Get out. */
65815 |         }
65816 |     }
65817 | }
65818 | } else {
65819 |     result = MA_INVALID_ARGS;
65820 | }
65821 |
65822 | pVorbis->cursor += totalFramesRead;
65823 |
65824 | if (totalFramesRead == 0) {
65825 |     result = MA_AT_END;
65826 | }
65827 |
65828 | if (pFramesRead != NULL) {
65829 |     *pFramesRead = totalFramesRead;
65830 | }
65831 |
65832 | if (result == MA_SUCCESS && totalFramesRead == 0) {
65833 |     result = MA_AT_END;
65834 | }
65835 |
65836 | return result;
65837 | }
65838 | #else
65839 | {

```

```

65840 |         /* vorbis is disabled. Should never hit this since initialization would have failed. */
65841 |         MA_ASSERT(MA_FALSE);
65842 |
65843 |         (void)pFramesOut;
65844 |         (void)frameCount;
65845 |         (void)pFramesRead;
65846 |
65847 |         return MA_NOT_IMPLEMENTED;
65848 |     }
65849 | #endif
65850 | }
65851 |
65852 | MA_API ma_result ma_stbvorbis_seek_to_pcm_frame(ma_stbvorbis* pVorbis, ma_uint64 frameIndex)
65853 | {
65854 |     if (pVorbis == NULL) {
65855 |         return MA_INVALID_ARGS;
65856 |     }
65857 |
65858 |     #if !defined(MA_NO_VORBIS)
65859 |     {
65860 |         /* Different seeking methods depending on whether or not we're using push mode. */
65861 |         if (pVorbis->usingPushMode) {
65862 |             /* Push mode. This is the complex case. */
65863 |             ma_result result;
65864 |             float buffer[4096];
65865 |
65866 |             /* If we're seeking backwards, we need to seek back to the start and then brute-force
> forward. */
65867 |             if (frameIndex < pVorbis->cursor) {
65868 |                 if (frameIndex > 0x7FFFFFFF) {
65869 |                     return MA_INVALID_ARGS; /* Trying to seek beyond the 32-bit maximum of
> stb_vorbis. */
65870 |                 }
65871 |
65872 |                 /*
65873 |                 This is wildly inefficient due to me having trouble getting sample exact seeking
> working
65874 |                 robustly with stb_vorbis_flush_pushdata(). The only way I can think to make this
> work
65875 |                 perfectly is to reinitialize the decoder. Note that we only enter this path when
> seeking
65876 |                 backwards. This will hopefully be removed once we get our own Vorbis decoder
> implemented.
65877 |                 */
65878 |                 stb_vorbis_close(pVorbis->stb);
65879 |                 ma_free(pVorbis->push.pData, &pVorbis->allocationCallbacks);
65880 |
65881 |                 MA_ZERO_OBJECT(&pVorbis->push);
65882 |
65883 |                 /* Seek to the start of the file. */
65884 |                 result = pVorbis->onSeek(pVorbis->pReadSeekTellUserData, 0, ma_seek_origin_start);
65885 |                 if (result != MA_SUCCESS) {
65886 |                     return result;
65887 |                 }
65888 |
65889 |                 result = ma_stbvorbis_init_internal_decoder_push(pVorbis);
65890 |                 if (result != MA_SUCCESS) {
65891 |                     return result;
65892 |                 }
65893 |
65894 |                 /* At this point we should be sitting on the first frame. */
65895 |                 pVorbis->cursor = 0;
65896 |             }
65897 |
65898 |             /* We're just brute-forcing this for now. */
65899 |             while (pVorbis->cursor < frameIndex) {
65900 |                 ma_uint64 framesRead;
65901 |                 ma_uint64 framesToRead = ma_countof(buffer)/pVorbis->channels;
65902 |                 if (framesToRead > (frameIndex - pVorbis->cursor)) {
65903 |                     framesToRead = (frameIndex - pVorbis->cursor);
65904 |                 }
65905 |
65906 |                 result = ma_stbvorbis_read_pcm_frames(pVorbis, buffer, framesToRead, &framesRead);
65907 |                 if (result != MA_SUCCESS) {
65908 |                     return result;
65909 |                 }
65910 |             }
65911 |         } else {
65912 |             /* Pull mode. This is the simple case. */

```



```

65913 |         int vorbisResult;
65914 |
65915 |         if (frameIndex > UINT_MAX) {
65916 |             return MA_INVALID_ARGS; /* Trying to seek beyond the 32-bit maximum of stb_vorbis.
> */
65917 |         }
65918 |
65919 |         vorbisResult = stb_vorbis_seek(pVorbis->stb, (unsigned int)frameIndex); /* Safe cast.
> */
65920 |         if (vorbisResult == 0) {
65921 |             return MA_ERROR; /* See failed. */
65922 |         }
65923 |
65924 |         pVorbis->cursor = frameIndex;
65925 |     }
65926 |
65927 |     return MA_SUCCESS;
65928 | }
65929 | #else
65930 | {
65931 |     /* vorbis is disabled. Should never hit this since initialization would have failed. */
65932 |     MA_ASSERT(MA_FALSE);
65933 |
65934 |     (void)frameIndex;
65935 |
65936 |     return MA_NOT_IMPLEMENTED;
65937 | }
65938 | #endif
65939 | }
65940 |
65941 | MA_API ma_result ma_stbvorbis_get_data_format(ma_stbvorbis* pVorbis, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
65942 | {
65943 |     /* Defaults for safety. */
65944 |     if (pFormat != NULL) {
65945 |         *pFormat = ma_format_unknown;
65946 |     }
65947 |     if (pChannels != NULL) {
65948 |         *pChannels = 0;
65949 |     }
65950 |     if (pSampleRate != NULL) {
65951 |         *pSampleRate = 0;
65952 |     }
65953 |     if (pChannelMap != NULL) {
65954 |         MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channelMapCap);
65955 |     }
65956 |
65957 |     if (pVorbis == NULL) {
65958 |         return MA_INVALID_OPERATION;
65959 |     }
65960 |
65961 |     if (pFormat != NULL) {
65962 |         *pFormat = pVorbis->format;
65963 |     }
65964 |
65965 |     #if !defined(MA_NO_VORBIS)
65966 |     {
65967 |         if (pChannels != NULL) {
65968 |             *pChannels = pVorbis->channels;
65969 |         }
65970 |
65971 |         if (pSampleRate != NULL) {
65972 |             *pSampleRate = pVorbis->sampleRate;
65973 |         }
65974 |
65975 |         if (pChannelMap != NULL) {
65976 |             ma_channel_map_init_standard(ma_standard_channel_map_vorbis, pChannelMap, channelMapCap,
> pVorbis->channels);
65977 |         }
65978 |
65979 |         return MA_SUCCESS;
65980 |     }
65981 |     #else
65982 |     {
65983 |         /* vorbis is disabled. Should never hit this since initialization would have failed. */
65984 |         MA_ASSERT(MA_FALSE);
65985 |         return MA_NOT_IMPLEMENTED;
65986 |     }
65987 |     #endif

```

```

65988 | }
65989 |
65990 | MA_API ma_result ma_stbvorbis_get_cursor_in_pcm_frames(ma_stbvorbis* pVorbis, ma_uint64* pCursor)
65991 | {
65992 |     if (pCursor == NULL) {
65993 |         return MA_INVALID_ARGS;
65994 |     }
65995 |
65996 |     *pCursor = 0;    /* Safety. */
65997 |
65998 |     if (pVorbis == NULL) {
65999 |         return MA_INVALID_ARGS;
66000 |     }
66001 |
66002 |     #if !defined(MA_NO_VORBIS)
66003 |     {
66004 |         *pCursor = pVorbis->cursor;
66005 |
66006 |         return MA_SUCCESS;
66007 |     }
66008 |     #else
66009 |     {
66010 |         /* vorbis is disabled. Should never hit this since initialization would have failed. */
66011 |         MA_ASSERT(MA_FALSE);
66012 |         return MA_NOT_IMPLEMENTED;
66013 |     }
66014 |     #endif
66015 | }
66016 |
66017 | MA_API ma_result ma_stbvorbis_get_length_in_pcm_frames(ma_stbvorbis* pVorbis, ma_uint64* pLength)
66018 | {
66019 |     if (pLength == NULL) {
66020 |         return MA_INVALID_ARGS;
66021 |     }
66022 |
66023 |     *pLength = 0;    /* Safety. */
66024 |
66025 |     if (pVorbis == NULL) {
66026 |         return MA_INVALID_ARGS;
66027 |     }
66028 |
66029 |     #if !defined(MA_NO_VORBIS)
66030 |     {
66031 |         if (pVorbis->usingPushMode) {
66032 |             *pLength = 0;    /* I don't know of a good way to determine this reliably with stb_vorbis
> and push mode. */
66033 |         } else {
66034 |             *pLength = stb_vorbis_stream_length_in_samples(pVorbis->stb);
66035 |         }
66036 |
66037 |         return MA_SUCCESS;
66038 |     }
66039 |     #else
66040 |     {
66041 |         /* vorbis is disabled. Should never hit this since initialization would have failed. */
66042 |         MA_ASSERT(MA_FALSE);
66043 |         return MA_NOT_IMPLEMENTED;
66044 |     }
66045 |     #endif
66046 | }
66047 |
66048 |
66049 | static ma_result ma_decoding_backend_init__stbvorbis(void* pUserData, ma_read_proc onRead,
> ma_seek_proc onSeek, ma_tell_proc onTell, void* pReadSeekTellUserData, const
> ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
66050 | {
66051 |     ma_result result;
66052 |     ma_stbvorbis* pVorbis;
66053 |
66054 |     (void)pUserData;    /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
66055 |
66056 |     /* For now we're just allocating the decoder backend on the heap. */
66057 |     pVorbis = (ma_stbvorbis*)ma_malloc(sizeof(*pVorbis), pAllocationCallbacks);
66058 |     if (pVorbis == NULL) {
66059 |         return MA_OUT_OF_MEMORY;
66060 |     }
66061 |

```

```

66062 |     result = ma_stbvorbis_init(onRead, onSeek, onTell, pReadSeekTellUserData, pConfig,
> pAllocationCallbacks, pVorbis);
66063 |     if (result != MA_SUCCESS) {
66064 |         ma_free(pVorbis, pAllocationCallbacks);
66065 |         return result;
66066 |     }
66067 |
66068 |     *ppBackend = pVorbis;
66069 |
66070 |     return MA_SUCCESS;
66071 | }
66072 |
66073 | static ma_result ma_decoding_backend_init_file__stbvorbis(void* pUserData, const char* pFilePath,
> const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source** ppBackend)
66074 | {
66075 |     ma_result result;
66076 |     ma_stbvorbis* pVorbis;
66077 |
66078 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
66079 |
66080 |     /* For now we're just allocating the decoder backend on the heap. */
66081 |     pVorbis = (ma_stbvorbis*)ma_malloc(sizeof(*pVorbis), pAllocationCallbacks);
66082 |     if (pVorbis == NULL) {
66083 |         return MA_OUT_OF_MEMORY;
66084 |     }
66085 |
66086 |     result = ma_stbvorbis_init_file(pFilePath, pConfig, pAllocationCallbacks, pVorbis);
66087 |     if (result != MA_SUCCESS) {
66088 |         ma_free(pVorbis, pAllocationCallbacks);
66089 |         return result;
66090 |     }
66091 |
66092 |     *ppBackend = pVorbis;
66093 |
66094 |     return MA_SUCCESS;
66095 | }
66096 |
66097 | static ma_result ma_decoding_backend_init_memory__stbvorbis(void* pUserData, const void* pData,
> size_t dataSize, const ma_decoding_backend_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_data_source** ppBackend)
66098 | {
66099 |     ma_result result;
66100 |     ma_stbvorbis* pVorbis;
66101 |
66102 |     (void)pUserData; /* For now not using pUserData, but once we start storing the vorbis decoder
> state within the ma_decoder structure this will be set to the decoder so we can avoid a malloc. */
66103 |
66104 |     /* For now we're just allocating the decoder backend on the heap. */
66105 |     pVorbis = (ma_stbvorbis*)ma_malloc(sizeof(*pVorbis), pAllocationCallbacks);
66106 |     if (pVorbis == NULL) {
66107 |         return MA_OUT_OF_MEMORY;
66108 |     }
66109 |
66110 |     result = ma_stbvorbis_init_memory(pData, dataSize, pConfig, pAllocationCallbacks, pVorbis);
66111 |     if (result != MA_SUCCESS) {
66112 |         ma_free(pVorbis, pAllocationCallbacks);
66113 |         return result;
66114 |     }
66115 |
66116 |     *ppBackend = pVorbis;
66117 |
66118 |     return MA_SUCCESS;
66119 | }
66120 |
66121 | static void ma_decoding_backend_uninit__stbvorbis(void* pUserData, ma_data_source* pBackend, const
> ma_allocation_callbacks* pAllocationCallbacks)
66122 | {
66123 |     ma_stbvorbis* pVorbis = (ma_stbvorbis*)pBackend;
66124 |
66125 |     (void)pUserData;
66126 |
66127 |     ma_stbvorbis_uninit(pVorbis, pAllocationCallbacks);
66128 |     ma_free(pVorbis, pAllocationCallbacks);
66129 | }
66130 |
66131 | static ma_decoding_backend_vtable g_ma_decoding_backend_vtable_stbvorbis =
66132 | {

```

```

66133 |     ma_decoding_backend_init__stbvorbis,
66134 |     ma_decoding_backend_init_file__stbvorbis,
66135 |     NULL, /* onInitFileW() */
66136 |     ma_decoding_backend_init_memory__stbvorbis,
66137 |     ma_decoding_backend_uninit__stbvorbis
66138 | };
66139 |
66140 | static ma_result ma_decoder_init_vorbis__internal(const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
66141 | {
66142 |     return ma_decoder_init_from_vtable__internal(&g_ma_decoding_backend_vtable_stbvorbis, NULL,
> pConfig, pDecoder);
66143 | }
66144 |
66145 | static ma_result ma_decoder_init_vorbis_from_file__internal(const char* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
66146 | {
66147 |     return ma_decoder_init_from_file__internal(&g_ma_decoding_backend_vtable_stbvorbis, NULL,
> pFilePath, pConfig, pDecoder);
66148 | }
66149 |
66150 | static ma_result ma_decoder_init_vorbis_from_file_w__internal(const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
66151 | {
66152 |     return ma_decoder_init_from_file_w__internal(&g_ma_decoding_backend_vtable_stbvorbis, NULL,
> pFilePath, pConfig, pDecoder);
66153 | }
66154 |
66155 | static ma_result ma_decoder_init_vorbis_from_memory__internal(const void* pData, size_t dataSize,
> const ma_decoder_config* pConfig, ma_decoder* pDecoder)
66156 | {
66157 |     return ma_decoder_init_from_memory__internal(&g_ma_decoding_backend_vtable_stbvorbis, NULL,
> pData, dataSize, pConfig, pDecoder);
66158 | }
66159 | #endif /* STB_VORBIS_INCLUDE_STB_VORBIS_H */
66160 |
66161 |
66162 |
66163 | static ma_result ma_decoder__init_allocation_callbacks(const ma_decoder_config* pConfig, ma_decoder*
> pDecoder)
66164 | {
66165 |     MA_ASSERT(pDecoder != NULL);
66166 |
66167 |     if (pConfig != NULL) {
66168 |         return ma_allocation_callbacks_init_copy(&pDecoder->allocationCallbacks,
> &pConfig->allocationCallbacks);
66169 |     } else {
66170 |         pDecoder->allocationCallbacks = ma_allocation_callbacks_init_default();
66171 |         return MA_SUCCESS;
66172 |     }
66173 | }
66174 |
66175 | static ma_result ma_decoder__data_source_on_read(ma_data_source* pDataSource, void* pFramesOut,
> ma_uint64 frameCount, ma_uint64* pFramesRead)
66176 | {
66177 |     return ma_decoder_read_pcm_frames((ma_decoder*)pDataSource, pFramesOut, frameCount,
> pFramesRead);
66178 | }
66179 |
66180 | static ma_result ma_decoder__data_source_on_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
66181 | {
66182 |     return ma_decoder_seek_to_pcm_frame((ma_decoder*)pDataSource, frameIndex);
66183 | }
66184 |
66185 | static ma_result ma_decoder__data_source_on_get_data_format(ma_data_source* pDataSource, ma_format*
> pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
66186 | {
66187 |     return ma_decoder_get_data_format((ma_decoder*)pDataSource, pFormat, pChannels, pSampleRate,
> pChannelMap, channelMapCap);
66188 | }
66189 |
66190 | static ma_result ma_decoder__data_source_on_get_cursor(ma_data_source* pDataSource, ma_uint64*
> pCursor)
66191 | {
66192 |     return ma_decoder_get_cursor_in_pcm_frames((ma_decoder*)pDataSource, pCursor);
66193 | }
66194 |
66195 | static ma_result ma_decoder__data_source_on_get_length(ma_data_source* pDataSource, ma_uint64*

```

```

    > pLength)
66196 | {
66197 |     return ma_decoder_get_length_in_pcm_frames((ma_decoder*)pDataSource, pLength);
66198 | }
66199 |
66200 | static ma_data_source_vtable g_ma_decoder_data_source_vtable =
66201 | {
66202 |     ma_decoder__data_source_on_read,
66203 |     ma_decoder__data_source_on_seek,
66204 |     ma_decoder__data_source_on_get_data_format,
66205 |     ma_decoder__data_source_on_get_cursor,
66206 |     ma_decoder__data_source_on_get_length,
66207 |     NULL, /* onSetLooping */
66208 |     0
66209 | };
66210 |
66211 | static ma_result ma_decoder__preinit(ma_decoder_read_proc onRead, ma_decoder_seek_proc onSeek,
    > ma_decoder_tell_proc onTell, void* pUserData, const ma_decoder_config* pConfig, ma_decoder*
    > pDecoder)
66212 | {
66213 |     ma_result result;
66214 |     ma_data_source_config dataSourceConfig;
66215 |
66216 |     MA_ASSERT(pConfig != NULL);
66217 |
66218 |     if (pDecoder == NULL) {
66219 |         return MA_INVALID_ARGS;
66220 |     }
66221 |
66222 |     MA_ZERO_OBJECT(pDecoder);
66223 |
66224 |     dataSourceConfig = ma_data_source_config_init();
66225 |     dataSourceConfig.vtable = &g_ma_decoder_data_source_vtable;
66226 |
66227 |     result = ma_data_source_init(&dataSourceConfig, &pDecoder->ds);
66228 |     if (result != MA_SUCCESS) {
66229 |         return result;
66230 |     }
66231 |
66232 |     pDecoder->onRead    = onRead;
66233 |     pDecoder->onSeek    = onSeek;
66234 |     pDecoder->onTell    = onTell;
66235 |     pDecoder->pUserData = pUserData;
66236 |
66237 |     result = ma_decoder__init_allocation_callbacks(pConfig, pDecoder);
66238 |     if (result != MA_SUCCESS) {
66239 |         ma_data_source_uninit(&pDecoder->ds);
66240 |         return result;
66241 |     }
66242 |
66243 |     return MA_SUCCESS;
66244 | }
66245 |
66246 | static ma_result ma_decoder__postinit(const ma_decoder_config* pConfig, ma_decoder* pDecoder)
66247 | {
66248 |     ma_result result;
66249 |
66250 |     result = ma_decoder__init_data_converter(pDecoder, pConfig);
66251 |
66252 |     /* If we failed post initialization we need to uninitialized the decoder before returning to
    > prevent a memory leak. */
66253 |     if (result != MA_SUCCESS) {
66254 |         ma_decoder_uninit(pDecoder);
66255 |         return result;
66256 |     }
66257 |
66258 |     return result;
66259 | }
66260 |
66261 |
66262 | static ma_result ma_decoder_init__internal(ma_decoder_read_proc onRead, ma_decoder_seek_proc onSeek,
    > void* pUserData, const ma_decoder_config* pConfig, ma_decoder* pDecoder)
66263 | {
66264 |     ma_result result = MA_NO_BACKEND;
66265 |
66266 |     MA_ASSERT(pConfig != NULL);
66267 |     MA_ASSERT(pDecoder != NULL);
66268 |
66269 |     /* Silence some warnings in the case that we don't have any decoder backends enabled. */

```

```

66270 | (void)onRead;
66271 | (void)onSeek;
66272 | (void)pUserData;
66273 |
66274 |
66275 | /* If we've specified a specific encoding type, try that first. */
66276 | if (pConfig->encodingFormat != ma_encoding_format_unknown) {
66277 |     #ifdef MA_HAS_WAV
66278 |         if (pConfig->encodingFormat == ma_encoding_format_wav) {
66279 |             result = ma_decoder_init_wav__internal(pConfig, pDecoder);
66280 |         }
66281 |     #endif
66282 |     #ifdef MA_HAS_FLAC
66283 |         if (pConfig->encodingFormat == ma_encoding_format_flac) {
66284 |             result = ma_decoder_init_flac__internal(pConfig, pDecoder);
66285 |         }
66286 |     #endif
66287 |     #ifdef MA_HAS_MP3
66288 |         if (pConfig->encodingFormat == ma_encoding_format_mp3) {
66289 |             result = ma_decoder_init_mp3__internal(pConfig, pDecoder);
66290 |         }
66291 |     #endif
66292 |     #ifdef MA_HAS_VORBIS
66293 |         if (pConfig->encodingFormat == ma_encoding_format_vorbis) {
66294 |             result = ma_decoder_init_vorbis__internal(pConfig, pDecoder);
66295 |         }
66296 |     #endif
66297 |
66298 |     /* If we weren't able to initialize the decoder, seek back to the start to give the next
> attempts a clean start. */
66299 |     if (result != MA_SUCCESS) {
66300 |         onSeek(pDecoder, 0, ma_seek_origin_start);
66301 |     }
66302 | }
66303 |
66304 | if (result != MA_SUCCESS) {
66305 |     /* Getting here means we couldn't load a specific decoding backend based on the encoding
> format. */
66306 |
66307 |     /*
66308 |     We use trial and error to open a decoder. We prioritize custom decoders so that if they
66309 |     implement the same encoding format they take priority over the built-in decoders.
66310 |     */
66311 |     result = ma_decoder_init_custom__internal(pConfig, pDecoder);
66312 |     if (result != MA_SUCCESS) {
66313 |         onSeek(pDecoder, 0, ma_seek_origin_start);
66314 |     }
66315 |
66316 |     /*
66317 |     If we get to this point and we still haven't found a decoder, and the caller has requested a
66318 |     specific encoding format, there's no hope for it. Abort.
66319 |     */
66320 |     if (pConfig->encodingFormat != ma_encoding_format_unknown) {
66321 |         return MA_NO_BACKEND;
66322 |     }
66323 |
66324 |     #ifdef MA_HAS_WAV
66325 |         if (result != MA_SUCCESS) {
66326 |             result = ma_decoder_init_wav__internal(pConfig, pDecoder);
66327 |             if (result != MA_SUCCESS) {
66328 |                 onSeek(pDecoder, 0, ma_seek_origin_start);
66329 |             }
66330 |         }
66331 |     #endif
66332 |     #ifdef MA_HAS_FLAC
66333 |         if (result != MA_SUCCESS) {
66334 |             result = ma_decoder_init_flac__internal(pConfig, pDecoder);
66335 |             if (result != MA_SUCCESS) {
66336 |                 onSeek(pDecoder, 0, ma_seek_origin_start);
66337 |             }
66338 |         }
66339 |     #endif
66340 |     #ifdef MA_HAS_MP3
66341 |         if (result != MA_SUCCESS) {
66342 |             result = ma_decoder_init_mp3__internal(pConfig, pDecoder);
66343 |             if (result != MA_SUCCESS) {
66344 |                 onSeek(pDecoder, 0, ma_seek_origin_start);
66345 |             }
66346 |         }

```

```

66347 |     #endif
66348 |     #ifdef MA_HAS_VORBIS
66349 |         if (result != MA_SUCCESS) {
66350 |             result = ma_decoder_init_vorbis_internal(pConfig, pDecoder);
66351 |             if (result != MA_SUCCESS) {
66352 |                 onSeek(pDecoder, 0, ma_seek_origin_start);
66353 |             }
66354 |         }
66355 |     #endif
66356 | }
66357 |
66358 | if (result != MA_SUCCESS) {
66359 |     return result;
66360 | }
66361 |
66362 | return ma_decoder__postinit(pConfig, pDecoder);
66363 | }
66364 |
66365 | MA_API ma_result ma_decoder_init(ma_decoder_read_proc onRead, ma_decoder_seek_proc onSeek, void*
> pUserData, const ma_decoder_config* pConfig, ma_decoder* pDecoder)
66366 | {
66367 |     ma_decoder_config config;
66368 |     ma_result result;
66369 |
66370 |     config = ma_decoder_config_init_copy(pConfig);
66371 |
66372 |     result = ma_decoder__preinit(onRead, onSeek, NULL, pUserData, &config, pDecoder);
66373 |     if (result != MA_SUCCESS) {
66374 |         return result;
66375 |     }
66376 |
66377 |     return ma_decoder_init__internal(onRead, onSeek, pUserData, &config, pDecoder);
66378 | }
66379 |
66380 |
66381 | static ma_result ma_decoder__on_read_memory(ma_decoder* pDecoder, void* pBufferOut, size_t
> bytesToRead, size_t* pBytesRead)
66382 | {
66383 |     size_t bytesRemaining;
66384 |
66385 |     MA_ASSERT(pDecoder->data.memory.dataSize >= pDecoder->data.memory.currentReadPos);
66386 |
66387 |     if (pBytesRead != NULL) {
66388 |         *pBytesRead = 0;
66389 |     }
66390 |
66391 |     bytesRemaining = pDecoder->data.memory.dataSize - pDecoder->data.memory.currentReadPos;
66392 |     if (bytesToRead > bytesRemaining) {
66393 |         bytesToRead = bytesRemaining;
66394 |     }
66395 |
66396 |     if (bytesRemaining == 0) {
66397 |         return MA_AT_END;
66398 |     }
66399 |
66400 |     if (bytesToRead > 0) {
66401 |         MA_COPY_MEMORY(pBufferOut, pDecoder->data.memory.pData +
> pDecoder->data.memory.currentReadPos, bytesToRead);
66402 |         pDecoder->data.memory.currentReadPos += bytesToRead;
66403 |     }
66404 |
66405 |     if (pBytesRead != NULL) {
66406 |         *pBytesRead = bytesToRead;
66407 |     }
66408 |
66409 |     return MA_SUCCESS;
66410 | }
66411 |
66412 | static ma_result ma_decoder__on_seek_memory(ma_decoder* pDecoder, ma_int64 byteOffset,
> ma_seek_origin origin)
66413 | {
66414 |     if (byteOffset > 0 && (ma_uint64)byteOffset > MA_SIZE_MAX) {
66415 |         return MA_BAD_SEEK;
66416 |     }
66417 |
66418 |     if (origin == ma_seek_origin_current) {
66419 |         if (byteOffset > 0) {
66420 |             if (pDecoder->data.memory.currentReadPos + byteOffset > pDecoder->data.memory.dataSize)
> {

```



```

66421 |         byteOffset = (ma_int64)(pDecoder->data.memory.dataSize -
> pDecoder->data.memory.currentReadPos); /* Trying to seek too far forward. */
66422 |     }
66423 |
66424 |     pDecoder->data.memory.currentReadPos += (size_t)byteOffset;
66425 | } else {
66426 |     if (pDecoder->data.memory.currentReadPos < (size_t)-byteOffset) {
66427 |         byteOffset = -(ma_int64)pDecoder->data.memory.currentReadPos; /* Trying to seek too
> far backwards. */
66428 |     }
66429 |
66430 |     pDecoder->data.memory.currentReadPos -= (size_t)-byteOffset;
66431 | }
66432 | } else {
66433 |     if (origin == ma_seek_origin_end) {
66434 |         if (byteOffset < 0) {
66435 |             byteOffset = -byteOffset;
66436 |         }
66437 |
66438 |         if (byteOffset > (ma_int64)pDecoder->data.memory.dataSize) {
66439 |             pDecoder->data.memory.currentReadPos = 0; /* Trying to seek too far back. */
66440 |         } else {
66441 |             pDecoder->data.memory.currentReadPos = pDecoder->data.memory.dataSize -
> (size_t)byteOffset;
66442 |         }
66443 |     } else {
66444 |         if ((size_t)byteOffset <= pDecoder->data.memory.dataSize) {
66445 |             pDecoder->data.memory.currentReadPos = (size_t)byteOffset;
66446 |         } else {
66447 |             pDecoder->data.memory.currentReadPos = pDecoder->data.memory.dataSize; /* Trying to
> seek too far forward. */
66448 |         }
66449 |     }
66450 | }
66451 |
66452 |     return MA_SUCCESS;
66453 | }
66454 |
66455 | static ma_result ma_decoder__on_tell_memory(ma_decoder* pDecoder, ma_int64* pCursor)
66456 | {
66457 |     MA_ASSERT(pDecoder != NULL);
66458 |     MA_ASSERT(pCursor != NULL);
66459 |
66460 |     *pCursor = (ma_int64)pDecoder->data.memory.currentReadPos;
66461 |
66462 |     return MA_SUCCESS;
66463 | }
66464 |
66465 | static ma_result ma_decoder__preinit_memory_wrapper(const void* pData, size_t dataSize, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
66466 | {
66467 |     ma_result result = ma_decoder__preinit(ma_decoder__on_read_memory, ma_decoder__on_seek_memory,
> ma_decoder__on_tell_memory, NULL, pConfig, pDecoder);
66468 |     if (result != MA_SUCCESS) {
66469 |         return result;
66470 |     }
66471 |
66472 |     if (pData == NULL || dataSize == 0) {
66473 |         return MA_INVALID_ARGS;
66474 |     }
66475 |
66476 |     pDecoder->data.memory.pData = (const ma_uint8*)pData;
66477 |     pDecoder->data.memory.dataSize = dataSize;
66478 |     pDecoder->data.memory.currentReadPos = 0;
66479 |
66480 |     (void)pConfig;
66481 |     return MA_SUCCESS;
66482 | }
66483 |
66484 | MA_API ma_result ma_decoder_init_memory(const void* pData, size_t dataSize, const ma_decoder_config*
> pConfig, ma_decoder* pDecoder)
66485 | {
66486 |     ma_result result;
66487 |     ma_decoder_config config;
66488 |
66489 |     config = ma_decoder_config_init_copy(pConfig);
66490 |
66491 |     result = ma_decoder__preinit(NULL, NULL, NULL, NULL, &config, pDecoder);
66492 |     if (result != MA_SUCCESS) {

```

```

66493 |         return result;
66494 |     }
66495 |
66496 |     if (pData == NULL || dataSize == 0) {
66497 |         return MA_INVALID_ARGS;
66498 |     }
66499 |
66500 |     /* If the backend has support for loading from a file path we'll want to use that. If that all
> fails we'll fall back to the VFS path. */
66501 |     result = MA_NO_BACKEND;
66502 |
66503 |     if (config.encodingFormat != ma_encoding_format_unknown) {
66504 |         #ifdef MA_HAS_WAV
66505 |             if (config.encodingFormat == ma_encoding_format_wav) {
66506 |                 result = ma_decoder_init_wav_from_memory__internal(pData, dataSize, &config, pDecoder);
66507 |             }
66508 |         #endif
66509 |         #ifdef MA_HAS_FLAC
66510 |             if (config.encodingFormat == ma_encoding_format_flac) {
66511 |                 result = ma_decoder_init_flac_from_memory__internal(pData, dataSize, &config, pDecoder);
66512 |             }
66513 |         #endif
66514 |         #ifdef MA_HAS_MP3
66515 |             if (config.encodingFormat == ma_encoding_format_mp3) {
66516 |                 result = ma_decoder_init_mp3_from_memory__internal(pData, dataSize, &config, pDecoder);
66517 |             }
66518 |         #endif
66519 |         #ifdef MA_HAS_VORBIS
66520 |             if (config.encodingFormat == ma_encoding_format_vorbis) {
66521 |                 result = ma_decoder_init_vorbis_from_memory__internal(pData, dataSize, &config,
> pDecoder);
66522 |             }
66523 |         #endif
66524 |     }
66525 |
66526 |     if (result != MA_SUCCESS) {
66527 |         /* Getting here means we weren't able to initialize a decoder of a specific encoding format.
> */
66528 |
66529 |         /*
66530 |         We use trial and error to open a decoder. We prioritize custom decoders so that if they
66531 |         implement the same encoding format they take priority over the built-in decoders.
66532 |         */
66533 |         result = ma_decoder_init_custom_from_memory__internal(pData, dataSize, &config, pDecoder);
66534 |
66535 |         /*
66536 |         If we get to this point and we still haven't found a decoder, and the caller has requested a
66537 |         specific encoding format, there's no hope for it. Abort.
66538 |         */
66539 |         if (result != MA_SUCCESS && config.encodingFormat != ma_encoding_format_unknown) {
66540 |             return MA_NO_BACKEND;
66541 |         }
66542 |
66543 |         /* Use trial and error for stock decoders. */
66544 |         if (result != MA_SUCCESS) {
66545 |             #ifdef MA_HAS_WAV
66546 |                 if (result != MA_SUCCESS) {
66547 |                     result = ma_decoder_init_wav_from_memory__internal(pData, dataSize, &config,
> pDecoder);
66548 |                 }
66549 |             #endif
66550 |             #ifdef MA_HAS_FLAC
66551 |                 if (result != MA_SUCCESS) {
66552 |                     result = ma_decoder_init_flac_from_memory__internal(pData, dataSize, &config,
> pDecoder);
66553 |                 }
66554 |             #endif
66555 |             #ifdef MA_HAS_MP3
66556 |                 if (result != MA_SUCCESS) {
66557 |                     result = ma_decoder_init_mp3_from_memory__internal(pData, dataSize, &config,
> pDecoder);
66558 |                 }
66559 |             #endif
66560 |             #ifdef MA_HAS_VORBIS
66561 |                 if (result != MA_SUCCESS) {
66562 |                     result = ma_decoder_init_vorbis_from_memory__internal(pData, dataSize, &config,
> pDecoder);
66563 |                 }
66564 |             #endif

```

```

66565 |     }
66566 | }
66567 |
66568 | /*
66569 | If at this point we still haven't successfully initialized the decoder it most likely means
66570 | the backend doesn't have an implementation for loading from a file path. We'll try using
66571 | miniaudio's built-in file IO for loading file.
66572 | */
66573 | if (result == MA_SUCCESS) {
66574 |     /* Initialization was successful. Finish up. */
66575 |     result = ma_decoder__postinit(&config, pDecoder);
66576 |     if (result != MA_SUCCESS) {
66577 |         return result;
66578 |     }
66579 | } else {
66580 |     /* Probably no implementation for loading from a block of memory. Use miniaudio's
> abstraction instead. */
66581 |     result = ma_decoder__preinit_memory_wrapper(pData, dataSize, &config, pDecoder);
66582 |     if (result != MA_SUCCESS) {
66583 |         return result;
66584 |     }
66585 |
66586 |     result = ma_decoder_init__internal(ma_decoder__on_read_memory, ma_decoder__on_seek_memory,
> NULL, &config, pDecoder);
66587 |     if (result != MA_SUCCESS) {
66588 |         return result;
66589 |     }
66590 | }
66591 | return MA_SUCCESS;
66592 | }
66593 | }
66594 |
66595 | #if defined(MA_HAS_WAV) || \
66596 |     defined(MA_HAS_MP3) || \
66597 |     defined(MA_HAS_FLAC) || \
66598 |     defined(MA_HAS_VORBIS)
66599 | #define MA_HAS_PATH_API
66600 | #endif
66601 |
66602 | #if defined(MA_HAS_PATH_API)
66603 | static const char* ma_path_file_name(const char* path)
66604 | {
66605 |     const char* fileName;
66606 |
66607 |     if (path == NULL) {
66608 |         return NULL;
66609 |     }
66610 |
66611 |     fileName = path;
66612 |
66613 |     /* We just loop through the path until we find the last slash. */
66614 |     while (path[0] != '\0') {
66615 |         if (path[0] == '/' || path[0] == '\\') {
66616 |             fileName = path;
66617 |         }
66618 |
66619 |         path += 1;
66620 |     }
66621 |
66622 |     /* At this point the file name is sitting on a slash, so just move forward. */
66623 |     while (fileName[0] != '\0' && (fileName[0] == '/' || fileName[0] == '\\')) {
66624 |         fileName += 1;
66625 |     }
66626 |
66627 |     return fileName;
66628 | }
66629 | }
66630 |
66631 | static const wchar_t* ma_path_file_name_w(const wchar_t* path)
66632 | {
66633 |     const wchar_t* fileName;
66634 |
66635 |     if (path == NULL) {
66636 |         return NULL;
66637 |     }
66638 |
66639 |     fileName = path;
66640 |
66641 |     /* We just loop through the path until we find the last slash. */

```

```
66642 | while (path[0] != '\0') {
66643 |     if (path[0] == '/' || path[0] == '\\') {
66644 |         fileName = path;
66645 |     }
66646 |
66647 |     path += 1;
66648 | }
66649 |
66650 | /* At this point the file name is sitting on a slash, so just move forward. */
66651 | while (fileName[0] != '\0' && (fileName[0] == '/' || fileName[0] == '\\')) {
66652 |     fileName += 1;
66653 | }
66654 |
66655 | return fileName;
66656 | }
66657 |
66658 |
66659 | static const char* ma_path_extension(const char* path)
66660 | {
66661 |     const char* extension;
66662 |     const char* lastOccurance;
66663 |
66664 |     if (path == NULL) {
66665 |         path = "";
66666 |     }
66667 |
66668 |     extension = ma_path_file_name(path);
66669 |     lastOccurance = NULL;
66670 |
66671 |     /* Just find the last '.' and return. */
66672 |     while (extension[0] != '\0') {
66673 |         if (extension[0] == '.') {
66674 |             extension += 1;
66675 |             lastOccurance = extension;
66676 |         }
66677 |
66678 |         extension += 1;
66679 |     }
66680 |
66681 |     return (lastOccurance != NULL) ? lastOccurance : extension;
66682 | }
66683 |
66684 | static const wchar_t* ma_path_extension_w(const wchar_t* path)
66685 | {
66686 |     const wchar_t* extension;
66687 |     const wchar_t* lastOccurance;
66688 |
66689 |     if (path == NULL) {
66690 |         path = L"";
66691 |     }
66692 |
66693 |     extension = ma_path_file_name_w(path);
66694 |     lastOccurance = NULL;
66695 |
66696 |     /* Just find the last '.' and return. */
66697 |     while (extension[0] != '\0') {
66698 |         if (extension[0] == '.') {
66699 |             extension += 1;
66700 |             lastOccurance = extension;
66701 |         }
66702 |
66703 |         extension += 1;
66704 |     }
66705 |
66706 |     return (lastOccurance != NULL) ? lastOccurance : extension;
66707 | }
66708 |
66709 |
66710 | static ma_bool32 ma_path_extension_equal(const char* path, const char* extension)
66711 | {
66712 |     const char* ext1;
66713 |     const char* ext2;
66714 |
66715 |     if (path == NULL || extension == NULL) {
66716 |         return MA_FALSE;
66717 |     }
66718 |
66719 |     ext1 = extension;
66720 |     ext2 = ma_path_extension(path);
```

```

66721 |
66722 | #if defined(_MSC_VER) || defined(__DMC__)
66723 |     return _stricmp(ext1, ext2) == 0;
66724 | #else
66725 |     return strcasecmp(ext1, ext2) == 0;
66726 | #endif
66727 | }
66728 |
66729 | static ma_bool32 ma_path_extension_equal_w(const wchar_t* path, const wchar_t* extension)
66730 | {
66731 |     const wchar_t* ext1;
66732 |     const wchar_t* ext2;
66733 |
66734 |     if (path == NULL || extension == NULL) {
66735 |         return MA_FALSE;
66736 |     }
66737 |
66738 |     ext1 = extension;
66739 |     ext2 = ma_path_extension_w(path);
66740 |
66741 |     #if (defined(_MSC_VER) || defined(__WATCOMC__) || defined(__DMC__)) && !defined(MA_XBOX_NXDK)
66742 |     {
66743 |         return _wcsicmp(ext1, ext2) == 0;
66744 |     }
66745 |     #elif !defined(MA_XBOX_NXDK) && !defined(MA_DOS)
66746 |     {
66747 |         /*
66748 |         > I'm not aware of a wide character version of strcasecmp(). I'm therefore converting the
66749 |         > extensions to multibyte strings and comparing those. This
66750 |         > isn't the most efficient way to do it, but it should work OK.
66751 |         */
66752 |         char ext1MB[4096];
66753 |         char ext2MB[4096];
66754 |         const wchar_t* pext1 = ext1;
66755 |         const wchar_t* pext2 = ext2;
66756 |         mbstate_t mbs1;
66757 |         mbstate_t mbs2;
66758 |
66759 |         MA_ZERO_OBJECT(&mbs1);
66760 |         MA_ZERO_OBJECT(&mbs2);
66761 |
66762 |         if (wcsrtombs(ext1MB, &pext1, sizeof(ext1MB), &mbs1) == (size_t)-1) {
66763 |             return MA_FALSE;
66764 |         }
66765 |         if (wcsrtombs(ext2MB, &pext2, sizeof(ext2MB), &mbs2) == (size_t)-1) {
66766 |             return MA_FALSE;
66767 |         }
66768 |
66769 |         return strcasecmp(ext1MB, ext2MB) == 0;
66770 |     }
66771 |     #else
66772 |     {
66773 |         /* Getting here means we don't have a way to do a case-sensitive comparison for wide
66774 |         > strings. Fall back to a simple case-sensitive comparison. */
66775 |         /* TODO: Implement our own wchar_t-to-char conversion routine and then use the char* version
66776 |         > for comparing. */
66777 |         return ma_wcscmp(ext1, ext2) == 0;
66778 |     }
66779 |     #endif
66780 | }
66781 | #endif /* MA_HAS_PATH_API */
66782 |
66783 | static ma_result ma_decoder__on_read_vfs(ma_decoder* pDecoder, void* pBufferOut, size_t bytesToRead,
66784 | > size_t* pBytesRead)
66785 | {
66786 |     MA_ASSERT(pDecoder != NULL);
66787 |     MA_ASSERT(pBufferOut != NULL);
66788 |
66789 |     return ma_vfs_or_default_read(pDecoder->data.vfs.pVFS, pDecoder->data.vfs.file, pBufferOut,
66790 | > bytesToRead, pBytesRead);
66791 | }
66792 |
66793 | static ma_result ma_decoder__on_seek_vfs(ma_decoder* pDecoder, ma_int64 offset, ma_seek_origin
66794 | > origin)
66795 | {
66796 |     MA_ASSERT(pDecoder != NULL);
66797 | }

```

```

66794 |     return ma_vfs_or_default_seek(pDecoder->data.vfs.pVFS, pDecoder->data.vfs.file, offset, origin);
66795 | }
66796 |
66797 | static ma_result ma_decoder__on_tell_vfs(ma_decoder* pDecoder, ma_int64* pCursor)
66798 | {
66799 |     MA_ASSERT(pDecoder != NULL);
66800 |
66801 |     return ma_vfs_or_default_tell(pDecoder->data.vfs.pVFS, pDecoder->data.vfs.file, pCursor);
66802 | }
66803 |
66804 | static ma_result ma_decoder__preinit_vfs(ma_vfs* pVFS, const char* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
66805 | {
66806 |     ma_result result;
66807 |     ma_vfs_file file;
66808 |
66809 |     result = ma_decoder__preinit(ma_decoder__on_read_vfs, ma_decoder__on_seek_vfs,
> ma_decoder__on_tell_vfs, NULL, pConfig, pDecoder);
66810 |     if (result != MA_SUCCESS) {
66811 |         return result;
66812 |     }
66813 |
66814 |     if (pFilePath == NULL || pFilePath[0] == '\\0') {
66815 |         return MA_INVALID_ARGS;
66816 |     }
66817 |
66818 |     result = ma_vfs_or_default_open(pVFS, pFilePath, MA_OPEN_MODE_READ, &file);
66819 |     if (result != MA_SUCCESS) {
66820 |         return result;
66821 |     }
66822 |
66823 |     pDecoder->data.vfs.pVFS = pVFS;
66824 |     pDecoder->data.vfs.file = file;
66825 |
66826 |     return MA_SUCCESS;
66827 | }
66828 |
66829 | MA_API ma_result ma_decoder_init_vfs(ma_vfs* pVFS, const char* pFilePath, const ma_decoder_config*
> pConfig, ma_decoder* pDecoder)
66830 | {
66831 |     ma_result result;
66832 |     ma_decoder_config config;
66833 |
66834 |     config = ma_decoder_config_init_copy(pConfig);
66835 |     result = ma_decoder__preinit_vfs(pVFS, pFilePath, &config, pDecoder);
66836 |     if (result != MA_SUCCESS) {
66837 |         return result;
66838 |     }
66839 |
66840 |     result = MA_NO_BACKEND;
66841 |
66842 |     if (config.encodingFormat != ma_encoding_format_unknown) {
66843 | #ifdef MA_HAS_WAV
66844 |         if (config.encodingFormat == ma_encoding_format_wav) {
66845 |             result = ma_decoder_init_wav__internal(&config, pDecoder);
66846 |         }
66847 | #endif
66848 | #ifdef MA_HAS_FLAC
66849 |         if (config.encodingFormat == ma_encoding_format_flac) {
66850 |             result = ma_decoder_init_flac__internal(&config, pDecoder);
66851 |         }
66852 | #endif
66853 | #ifdef MA_HAS_MP3
66854 |         if (config.encodingFormat == ma_encoding_format_mp3) {
66855 |             result = ma_decoder_init_mp3__internal(&config, pDecoder);
66856 |         }
66857 | #endif
66858 | #ifdef MA_HAS_VORBIS
66859 |         if (config.encodingFormat == ma_encoding_format_vorbis) {
66860 |             result = ma_decoder_init_vorbis__internal(&config, pDecoder);
66861 |         }
66862 | #endif
66863 |
66864 |         /* Make sure we seek back to the start if we didn't initialize a decoder successfully so the
> next attempts have a fresh start. */
66865 |         if (result != MA_SUCCESS) {
66866 |             ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
66867 |         }
66868 |     }

```

```

66869 |
66870 |     if (result != MA_SUCCESS) {
66871 |         /* Getting here means we weren't able to initialize a decoder of a specific encoding format.
> */
66872 |
66873 |         /*
66874 |         We use trial and error to open a decoder. We prioritize custom decoders so that if they
66875 |         implement the same encoding format they take priority over the built-in decoders.
66876 |         */
66877 |         result = ma_decoder_init_custom__internal(&config, pDecoder);
66878 |         if (result != MA_SUCCESS) {
66879 |             ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
66880 |         }
66881 |
66882 |         /*
66883 |         If we get to this point and we still haven't found a decoder, and the caller has requested a
66884 |         specific encoding format, there's no hope for it. Abort.
66885 |         */
66886 |         if (config.encodingFormat != ma_encoding_format_unknown) {
66887 |             return MA_NO_BACKEND;
66888 |         }
66889 |
66890 |         #ifdef MA_HAS_WAV
66891 |             if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "wav")) {
66892 |                 result = ma_decoder_init_wav__internal(&config, pDecoder);
66893 |                 if (result != MA_SUCCESS) {
66894 |                     ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
66895 |                 }
66896 |             }
66897 |         #endif
66898 |         #ifdef MA_HAS_FLAC
66899 |             if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "flac")) {
66900 |                 result = ma_decoder_init_flac__internal(&config, pDecoder);
66901 |                 if (result != MA_SUCCESS) {
66902 |                     ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
66903 |                 }
66904 |             }
66905 |         #endif
66906 |         #ifdef MA_HAS_MP3
66907 |             if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "mp3")) {
66908 |                 result = ma_decoder_init_mp3__internal(&config, pDecoder);
66909 |                 if (result != MA_SUCCESS) {
66910 |                     ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
66911 |                 }
66912 |             }
66913 |         #endif
66914 |     }
66915 |
66916 |     /* If we still haven't got a result just use trial and error. Otherwise we can finish up. */
66917 |     if (result != MA_SUCCESS) {
66918 |         result = ma_decoder_init__internal(ma_decoder__on_read_vfs, ma_decoder__on_seek_vfs, NULL,
> &config, pDecoder);
66919 |     } else {
66920 |         result = ma_decoder__postinit(&config, pDecoder);
66921 |     }
66922 |
66923 |     if (result != MA_SUCCESS) {
66924 |         if (pDecoder->data.vfs.file != NULL) { /* <-- Will be reset to NULL if ma_decoder_uninit()
> is called in one of the steps above which allows us to avoid a double close of the file. */
66925 |             ma_vfs_or_default_close(pVFS, pDecoder->data.vfs.file);
66926 |         }
66927 |
66928 |         return result;
66929 |     }
66930 |
66931 |     return MA_SUCCESS;
66932 | }
66933 |
66934 |
66935 | static ma_result ma_decoder__preinit_vfs_w(ma_vfs* pVFS, const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
66936 | {
66937 |     ma_result result;
66938 |     ma_vfs_file file;
66939 |
66940 |     result = ma_decoder__preinit(ma_decoder__on_read_vfs, ma_decoder__on_seek_vfs,
> ma_decoder__on_tell_vfs, NULL, pConfig, pDecoder);
66941 |     if (result != MA_SUCCESS) {
66942 |         return result;

```



```

66943 |     }
66944 |
66945 |     if (pFilePath == NULL || pFilePath[0] == '\\0') {
66946 |         return MA_INVALID_ARGS;
66947 |     }
66948 |
66949 |     result = ma_vfs_or_default_open_w(pVFS, pFilePath, MA_OPEN_MODE_READ, &file);
66950 |     if (result != MA_SUCCESS) {
66951 |         return result;
66952 |     }
66953 |
66954 |     pDecoder->data.vfs.pVFS = pVFS;
66955 |     pDecoder->data.vfs.file = file;
66956 |
66957 |     return MA_SUCCESS;
66958 | }
66959 |
66960 | MA_API ma_result ma_decoder_init_vfs_w(ma_vfs* pVFS, const wchar_t* pFilePath, const
> ma_decoder_config* pConfig, ma_decoder* pDecoder)
66961 | {
66962 |     ma_result result;
66963 |     ma_decoder_config config;
66964 |
66965 |     config = ma_decoder_config_init_copy(pConfig);
66966 |     result = ma_decoder__preinit_vfs_w(pVFS, pFilePath, &config, pDecoder);
66967 |     if (result != MA_SUCCESS) {
66968 |         return result;
66969 |     }
66970 |
66971 |     result = MA_NO_BACKEND;
66972 |
66973 |     if (config.encodingFormat != ma_encoding_format_unknown) {
66974 | #ifdef MA_HAS_WAV
66975 |         if (config.encodingFormat == ma_encoding_format_wav) {
66976 |             result = ma_decoder_init_wav__internal(&config, pDecoder);
66977 |         }
66978 |     #endif
66979 | #ifdef MA_HAS_FLAC
66980 |         if (config.encodingFormat == ma_encoding_format_flac) {
66981 |             result = ma_decoder_init_flac__internal(&config, pDecoder);
66982 |         }
66983 |     #endif
66984 | #ifdef MA_HAS_MP3
66985 |         if (config.encodingFormat == ma_encoding_format_mp3) {
66986 |             result = ma_decoder_init_mp3__internal(&config, pDecoder);
66987 |         }
66988 |     #endif
66989 | #ifdef MA_HAS_VORBIS
66990 |         if (config.encodingFormat == ma_encoding_format_vorbis) {
66991 |             result = ma_decoder_init_vorbis__internal(&config, pDecoder);
66992 |         }
66993 |     #endif
66994 |
66995 |     /* Make sure we seek back to the start if we didn't initialize a decoder successfully so the
> next attempts have a fresh start. */
66996 |     if (result != MA_SUCCESS) {
66997 |         ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
66998 |     }
66999 | }
67000 |
67001 | if (result != MA_SUCCESS) {
67002 |     /* Getting here means we weren't able to initialize a decoder of a specific encoding format.
> */
67003 |
67004 |     /*
67005 |     We use trial and error to open a decoder. We prioritize custom decoders so that if they
67006 |     implement the same encoding format they take priority over the built-in decoders.
67007 |     */
67008 |     result = ma_decoder_init_custom__internal(&config, pDecoder);
67009 |     if (result != MA_SUCCESS) {
67010 |         ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
67011 |     }
67012 |
67013 |     /*
67014 |     If we get to this point and we still haven't found a decoder, and the caller has requested a
67015 |     specific encoding format, there's no hope for it. Abort.
67016 |     */
67017 |     if (config.encodingFormat != ma_encoding_format_unknown) {
67018 |         return MA_NO_BACKEND;

```

```

67019 |     }
67020 |
67021 |     #ifdef MA_HAS_WAV
67022 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"wav")) {
67023 |             result = ma_decoder_init_wav__internal(&config, pDecoder);
67024 |             if (result != MA_SUCCESS) {
67025 |                 ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
67026 |             }
67027 |         }
67028 |     #endif
67029 |     #ifdef MA_HAS_FLAC
67030 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"flac")) {
67031 |             result = ma_decoder_init_flac__internal(&config, pDecoder);
67032 |             if (result != MA_SUCCESS) {
67033 |                 ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
67034 |             }
67035 |         }
67036 |     #endif
67037 |     #ifdef MA_HAS_MP3
67038 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"mp3")) {
67039 |             result = ma_decoder_init_mp3__internal(&config, pDecoder);
67040 |             if (result != MA_SUCCESS) {
67041 |                 ma_decoder__on_seek_vfs(pDecoder, 0, ma_seek_origin_start);
67042 |             }
67043 |         }
67044 |     #endif
67045 | }
67046 |
67047 | /* If we still haven't got a result just use trial and error. Otherwise we can finish up. */
67048 | if (result != MA_SUCCESS) {
67049 |     result = ma_decoder_init__internal(ma_decoder__on_read_vfs, ma_decoder__on_seek_vfs, NULL,
> &config, pDecoder);
67050 | } else {
67051 |     result = ma_decoder__postinit(&config, pDecoder);
67052 | }
67053 |
67054 | if (result != MA_SUCCESS) {
67055 |     ma_vfs_or_default_close(pVFS, pDecoder->data.vfs.file);
67056 |     return result;
67057 | }
67058 |
67059 | return MA_SUCCESS;
67060 | }
67061 |
67062 |
67063 | static ma_result ma_decoder__preinit_file(const char* pFilePath, const ma_decoder_config* pConfig,
> ma_decoder* pDecoder)
67064 | {
67065 |     ma_result result;
67066 |
67067 |     result = ma_decoder__preinit(NULL, NULL, NULL, NULL, pConfig, pDecoder);
67068 |     if (result != MA_SUCCESS) {
67069 |         return result;
67070 |     }
67071 |
67072 |     if (pFilePath == NULL || pFilePath[0] == '\0') {
67073 |         return MA_INVALID_ARGS;
67074 |     }
67075 |
67076 |     return MA_SUCCESS;
67077 | }
67078 |
67079 | MA_API ma_result ma_decoder_init_file(const char* pFilePath, const ma_decoder_config* pConfig,
> ma_decoder* pDecoder)
67080 | {
67081 |     ma_result result;
67082 |     ma_decoder_config config;
67083 |
67084 |     config = ma_decoder_config_init_copy(pConfig);
67085 |     result = ma_decoder__preinit_file(pFilePath, &config, pDecoder);
67086 |     if (result != MA_SUCCESS) {
67087 |         return result;
67088 |     }
67089 |
67090 |     /* If the backend has support for loading from a file path we'll want to use that. If that all
> fails we'll fall back to the VFS path. */
67091 |     result = MA_NO_BACKEND;
67092 |
67093 |     if (config.encodingFormat != ma_encoding_format_unknown) {

```

```

67094 |     #ifdef MA_HAS_WAV
67095 |         if (config.encodingFormat == ma_encoding_format_wav) {
67096 |             result = ma_decoder_init_wav_from_file__internal(pFilePath, &config, pDecoder);
67097 |         }
67098 |     #endif
67099 |     #ifdef MA_HAS_FLAC
67100 |         if (config.encodingFormat == ma_encoding_format_flac) {
67101 |             result = ma_decoder_init_flac_from_file__internal(pFilePath, &config, pDecoder);
67102 |         }
67103 |     #endif
67104 |     #ifdef MA_HAS_MP3
67105 |         if (config.encodingFormat == ma_encoding_format_mp3) {
67106 |             result = ma_decoder_init_mp3_from_file__internal(pFilePath, &config, pDecoder);
67107 |         }
67108 |     #endif
67109 |     #ifdef MA_HAS_VORBIS
67110 |         if (config.encodingFormat == ma_encoding_format_vorbis) {
67111 |             result = ma_decoder_init_vorbis_from_file__internal(pFilePath, &config, pDecoder);
67112 |         }
67113 |     #endif
67114 | }
67115 |
67116 | if (result != MA_SUCCESS) {
67117 |     /* Getting here means we weren't able to initialize a decoder of a specific encoding format.
> */
67118 |
67119 |     /*
67120 |     We use trial and error to open a decoder. We prioritize custom decoders so that if they
67121 |     implement the same encoding format they take priority over the built-in decoders.
67122 |     */
67123 |     result = ma_decoder_init_custom_from_file__internal(pFilePath, &config, pDecoder);
67124 |
67125 |     /*
67126 |     If we get to this point and we still haven't found a decoder, and the caller has requested a
67127 |     specific encoding format, there's no hope for it. Abort.
67128 |     */
67129 |     if (result != MA_SUCCESS && config.encodingFormat != ma_encoding_format_unknown) {
67130 |         return MA_NO_BACKEND;
67131 |     }
67132 |
67133 |     /* First try loading based on the file extension so we don't waste time opening and closing
> files. */
67134 |     #ifdef MA_HAS_WAV
67135 |         if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "wav")) {
67136 |             result = ma_decoder_init_wav_from_file__internal(pFilePath, &config, pDecoder);
67137 |         }
67138 |     #endif
67139 |     #ifdef MA_HAS_FLAC
67140 |         if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "flac")) {
67141 |             result = ma_decoder_init_flac_from_file__internal(pFilePath, &config, pDecoder);
67142 |         }
67143 |     #endif
67144 |     #ifdef MA_HAS_MP3
67145 |         if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "mp3")) {
67146 |             result = ma_decoder_init_mp3_from_file__internal(pFilePath, &config, pDecoder);
67147 |         }
67148 |     #endif
67149 |     #ifdef MA_HAS_VORBIS
67150 |         if (result != MA_SUCCESS && ma_path_extension_equal(pFilePath, "ogg")) {
67151 |             result = ma_decoder_init_vorbis_from_file__internal(pFilePath, &config, pDecoder);
67152 |         }
67153 |     #endif
67154 |
67155 |     /*
67156 |     If we still haven't got a result just use trial and error. Custom decoders have already been
> attempted, so here we
67157 |     need only iterate over our stock decoders.
67158 |     */
67159 |     if (result != MA_SUCCESS) {
67160 |         #ifdef MA_HAS_WAV
67161 |             if (result != MA_SUCCESS) {
67162 |                 result = ma_decoder_init_wav_from_file__internal(pFilePath, &config, pDecoder);
67163 |             }
67164 |         #endif
67165 |         #ifdef MA_HAS_FLAC
67166 |             if (result != MA_SUCCESS) {
67167 |                 result = ma_decoder_init_flac_from_file__internal(pFilePath, &config, pDecoder);
67168 |             }
67169 |         #endif

```

```

67170 |         #ifdef MA_HAS_MP3
67171 |             if (result != MA_SUCCESS) {
67172 |                 result = ma_decoder_init_mp3_from_file__internal(pFilePath, &config, pDecoder);
67173 |             }
67174 |         #endif
67175 |         #ifdef MA_HAS_VORBIS
67176 |             if (result != MA_SUCCESS) {
67177 |                 result = ma_decoder_init_vorbis_from_file__internal(pFilePath, &config, pDecoder);
67178 |             }
67179 |         #endif
67180 |     }
67181 | }
67182 |
67183 | /*
67184 | If at this point we still haven't successfully initialized the decoder it most likely means
67185 | the backend doesn't have an implementation for loading from a file path. We'll try using
67186 | miniaudio's built-in file IO for loading file.
67187 | */
67188 | if (result == MA_SUCCESS) {
67189 |     /* Initialization was successful. Finish up. */
67190 |     result = ma_decoder__postinit(&config, pDecoder);
67191 |     if (result != MA_SUCCESS) {
67192 |         return result;
67193 |     }
67194 | } else {
67195 |     /* Probably no implementation for loading from a file path. Use miniaudio's file IO instead.
> */
67196 |     result = ma_decoder_init_vfs(NULL, pFilePath, pConfig, pDecoder);
67197 |     if (result != MA_SUCCESS) {
67198 |         return result;
67199 |     }
67200 | }
67201 |
67202 | return MA_SUCCESS;
67203 | }
67204 |
67205 | static ma_result ma_decoder__preinit_file_w(const wchar_t* pFilePath, const ma_decoder_config*
> pConfig, ma_decoder* pDecoder)
67206 | {
67207 |     ma_result result;
67208 |
67209 |     result = ma_decoder__preinit(NULL, NULL, NULL, NULL, pConfig, pDecoder);
67210 |     if (result != MA_SUCCESS) {
67211 |         return result;
67212 |     }
67213 |
67214 |     if (pFilePath == NULL || pFilePath[0] == '\\0') {
67215 |         return MA_INVALID_ARGS;
67216 |     }
67217 |
67218 |     return MA_SUCCESS;
67219 | }
67220 |
67221 | MA_API ma_result ma_decoder_init_file_w(const wchar_t* pFilePath, const ma_decoder_config* pConfig,
> ma_decoder* pDecoder)
67222 | {
67223 |     ma_result result;
67224 |     ma_decoder_config config;
67225 |
67226 |     config = ma_decoder_config_init_copy(pConfig);
67227 |     result = ma_decoder__preinit_file_w(pFilePath, &config, pDecoder);
67228 |     if (result != MA_SUCCESS) {
67229 |         return result;
67230 |     }
67231 |
67232 |     /* If the backend has support for loading from a file path we'll want to use that. If that all
> fails we'll fall back to the VFS path. */
67233 |     result = MA_NO_BACKEND;
67234 |
67235 |     if (config.encodingFormat != ma_encoding_format_unknown) {
67236 |         #ifdef MA_HAS_WAV
67237 |             if (config.encodingFormat == ma_encoding_format_wav) {
67238 |                 result = ma_decoder_init_wav_from_file_w__internal(pFilePath, &config, pDecoder);
67239 |             }
67240 |         #endif
67241 |         #ifdef MA_HAS_FLAC
67242 |             if (config.encodingFormat == ma_encoding_format_flac) {
67243 |                 result = ma_decoder_init_flac_from_file_w__internal(pFilePath, &config, pDecoder);
67244 |             }

```

```

67245 | #endif
67246 | #ifdef MA_HAS_MP3
67247 |     if (config.encodingFormat == ma_encoding_format_mp3) {
67248 |         result = ma_decoder_init_mp3_from_file_w__internal(pFilePath, &config, pDecoder);
67249 |     }
67250 | #endif
67251 | #ifdef MA_HAS_VORBIS
67252 |     if (config.encodingFormat == ma_encoding_format_vorbis) {
67253 |         result = ma_decoder_init_vorbis_from_file_w__internal(pFilePath, &config, pDecoder);
67254 |     }
67255 | #endif
67256 | }
67257 |
67258 | if (result != MA_SUCCESS) {
67259 |     /* Getting here means we weren't able to initialize a decoder of a specific encoding format.
> */
67260 |
67261 |     /*
67262 |     We use trial and error to open a decoder. We prioritize custom decoders so that if they
67263 |     implement the same encoding format they take priority over the built-in decoders.
67264 |     */
67265 |     result = ma_decoder_init_custom_from_file_w__internal(pFilePath, &config, pDecoder);
67266 |
67267 |     /*
67268 |     If we get to this point and we still haven't found a decoder, and the caller has requested a
67269 |     specific encoding format, there's no hope for it. Abort.
67270 |     */
67271 |     if (result != MA_SUCCESS && config.encodingFormat != ma_encoding_format_unknown) {
67272 |         return MA_NO_BACKEND;
67273 |     }
67274 |
67275 |     /* First try loading based on the file extension so we don't waste time opening and closing
> files. */
67276 |     #ifdef MA_HAS_WAV
67277 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"wav")) {
67278 |             result = ma_decoder_init_wav_from_file_w__internal(pFilePath, &config, pDecoder);
67279 |         }
67280 |     #endif
67281 |     #ifdef MA_HAS_FLAC
67282 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"flac")) {
67283 |             result = ma_decoder_init_flac_from_file_w__internal(pFilePath, &config, pDecoder);
67284 |         }
67285 |     #endif
67286 |     #ifdef MA_HAS_MP3
67287 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"mp3")) {
67288 |             result = ma_decoder_init_mp3_from_file_w__internal(pFilePath, &config, pDecoder);
67289 |         }
67290 |     #endif
67291 |     #ifdef MA_HAS_VORBIS
67292 |         if (result != MA_SUCCESS && ma_path_extension_equal_w(pFilePath, L"ogg")) {
67293 |             result = ma_decoder_init_vorbis_from_file_w__internal(pFilePath, &config, pDecoder);
67294 |         }
67295 |     #endif
67296 |
67297 |     /*
67298 |     If we still haven't got a result just use trial and error. Custom decoders have already been
> attempted, so here we
67299 |     need only iterate over our stock decoders.
67300 |     */
67301 |     if (result != MA_SUCCESS) {
67302 |         #ifdef MA_HAS_WAV
67303 |             if (result != MA_SUCCESS) {
67304 |                 result = ma_decoder_init_wav_from_file_w__internal(pFilePath, &config, pDecoder);
67305 |             }
67306 |         #endif
67307 |         #ifdef MA_HAS_FLAC
67308 |             if (result != MA_SUCCESS) {
67309 |                 result = ma_decoder_init_flac_from_file_w__internal(pFilePath, &config, pDecoder);
67310 |             }
67311 |         #endif
67312 |         #ifdef MA_HAS_MP3
67313 |             if (result != MA_SUCCESS) {
67314 |                 result = ma_decoder_init_mp3_from_file_w__internal(pFilePath, &config, pDecoder);
67315 |             }
67316 |         #endif
67317 |         #ifdef MA_HAS_VORBIS
67318 |             if (result != MA_SUCCESS) {
67319 |                 result = ma_decoder_init_vorbis_from_file_w__internal(pFilePath, &config, pDecoder);
67320 |             }

```

```

67321 |         #endif
67322 |     }
67323 | }
67324 |
67325 | /*
67326 | If at this point we still haven't successfully initialized the decoder it most likely means
67327 | the backend doesn't have an implementation for loading from a file path. We'll try using
67328 | miniaudio's built-in file IO for loading file.
67329 | */
67330 | if (result == MA_SUCCESS) {
67331 |     /* Initialization was successful. Finish up. */
67332 |     result = ma_decoder__postinit(&config, pDecoder);
67333 |     if (result != MA_SUCCESS) {
67334 |         return result;
67335 |     }
67336 | } else {
67337 |     /* Probably no implementation for loading from a file path. Use miniaudio's file IO instead.
> */
67338 |     result = ma_decoder_init_vfs_w(NULL, pFilePath, pConfig, pDecoder);
67339 |     if (result != MA_SUCCESS) {
67340 |         return result;
67341 |     }
67342 | }
67343 |
67344 | return MA_SUCCESS;
67345 | }
67346 |
67347 | MA_API ma_result ma_decoder_uninit(ma_decoder* pDecoder)
67348 | {
67349 |     if (pDecoder == NULL) {
67350 |         return MA_INVALID_ARGS;
67351 |     }
67352 |
67353 |     if (pDecoder->pBackend != NULL) {
67354 |         if (pDecoder->pBackendVTable != NULL && pDecoder->pBackendVTable->onUninit != NULL) {
67355 |             pDecoder->pBackendVTable->onUninit(pDecoder->pBackendUserData, pDecoder->pBackend,
> &pDecoder->allocationCallbacks);
67356 |         }
67357 |     }
67358 |
67359 |     if (pDecoder->onRead == ma_decoder__on_read_vfs) {
67360 |         ma_vfs_or_default_close(pDecoder->data.vfs.pVFS, pDecoder->data.vfs.file);
67361 |         pDecoder->data.vfs.file = NULL;
67362 |     }
67363 |
67364 |     ma_data_converter_uninit(&pDecoder->converter, &pDecoder->allocationCallbacks);
67365 |     ma_data_source_uninit(&pDecoder->ds);
67366 |
67367 |     if (pDecoder->pInputCache != NULL) {
67368 |         ma_free(pDecoder->pInputCache, &pDecoder->allocationCallbacks);
67369 |     }
67370 |
67371 |     return MA_SUCCESS;
67372 | }
67373 |
67374 | MA_API ma_result ma_decoder_read_pcm_frames(ma_decoder* pDecoder, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
67375 | {
67376 |     ma_result result = MA_SUCCESS;
67377 |     ma_uint64 totalFramesReadOut;
67378 |     void* pRunningFramesOut;
67379 |
67380 |     if (pFramesRead != NULL) {
67381 |         *pFramesRead = 0; /* Safety. */
67382 |     }
67383 |
67384 |     if (frameCount == 0) {
67385 |         return MA_INVALID_ARGS;
67386 |     }
67387 |
67388 |     if (pDecoder == NULL) {
67389 |         return MA_INVALID_ARGS;
67390 |     }
67391 |
67392 |     if (pDecoder->pBackend == NULL) {
67393 |         return MA_INVALID_OPERATION;
67394 |     }
67395 |
67396 |     /* Fast path. */

```

```

67397 |     if (pDecoder->converter.isPassthrough) {
67398 |         result = ma_data_source_read_pcm_frames(pDecoder->pBackend, pFramesOut, frameCount,
> &totalFramesReadOut);
67399 |     } else {
67400 |         /*
67401 |         Getting here means we need to do data conversion. If we're seeking forward and are _not_
> doing resampling we can run this in a fast path. If we're doing resampling we
67402 |         need to run through each sample because we need to ensure its internal cache is updated.
67403 |         */
67404 |         if (pFramesOut == NULL && pDecoder->converter.hasResampler == MA_FALSE) {
67405 |             result = ma_data_source_read_pcm_frames(pDecoder->pBackend, NULL, frameCount,
> &totalFramesReadOut);
67406 |         } else {
67407 |             /* Slow path. Need to run everything through the data converter. */
67408 |             ma_format internalFormat;
67409 |             ma_uint32 internalChannels;
67410 |
67411 |             totalFramesReadOut = 0;
67412 |             pRunningFramesOut = pFramesOut;
67413 |
67414 |             result = ma_data_source_get_data_format(pDecoder->pBackend, &internalFormat,
> &internalChannels, NULL, NULL, 0);
67415 |             if (result != MA_SUCCESS) {
67416 |                 return result; /* Failed to retrieve the internal format and channel count. */
67417 |             }
67418 |
67419 |             /*
67420 |             We run a different path depending on whether or not we are using a heap-allocated
67421 |             intermediary buffer or not. If the data converter does not support the calculation of
67422 |             the required number of input frames, we'll use the heap-allocated path. Otherwise we'll
67423 |             use the stack-allocated path.
67424 |             */
67425 |             if (pDecoder->pInputCache != NULL) {
67426 |                 /* We don't have a way of determining the required number of input frames, so need
> to persistently store input data in a cache. */
67427 |                 while (totalFramesReadOut < frameCount) {
67428 |                     ma_uint64 framesToReadThisIterationIn;
67429 |                     ma_uint64 framesToReadThisIterationOut;
67430 |
67431 |                     /* If there's any data available in the cache, that needs to get processed
> first. */
67432 |                     if (pDecoder->inputCacheRemaining > 0) {
67433 |                         framesToReadThisIterationOut = (frameCount - totalFramesReadOut);
67434 |                         framesToReadThisIterationIn = framesToReadThisIterationOut;
67435 |                         if (framesToReadThisIterationIn > pDecoder->inputCacheRemaining) {
67436 |                             framesToReadThisIterationIn = pDecoder->inputCacheRemaining;
67437 |                         }
67438 |
67439 |                         result = ma_data_converter_process_pcm_frames(&pDecoder->converter,
> ma_offset_pcm_frames_ptr(pDecoder->pInputCache, pDecoder->inputCacheConsumed, internalFormat,
> internalChannels), &framesToReadThisIterationIn, pRunningFramesOut, &framesToReadThisIterationOut);
67440 |                         if (result != MA_SUCCESS) {
67441 |                             break;
67442 |                         }
67443 |
67444 |                         pDecoder->inputCacheConsumed += framesToReadThisIterationIn;
67445 |                         pDecoder->inputCacheRemaining -= framesToReadThisIterationIn;
67446 |
67447 |                         totalFramesReadOut += framesToReadThisIterationOut;
67448 |
67449 |                         if (pRunningFramesOut != NULL) {
67450 |                             pRunningFramesOut = ma_offset_ptr(pRunningFramesOut,
> framesToReadThisIterationOut * ma_get_bytes_per_frame(pDecoder->outputFormat,
> pDecoder->outputChannels));
67451 |                         }
67452 |
67453 |                         if (framesToReadThisIterationIn == 0 && framesToReadThisIterationOut == 0) {
67454 |                             break; /* We're done. */
67455 |                         }
67456 |                     }
67457 |
67458 |                     /* Getting here means there's no data in the cache and we need to fill it up
> from the data source. */
67459 |                     if (pDecoder->inputCacheRemaining == 0) {
67460 |                         pDecoder->inputCacheConsumed = 0;
67461 |
67462 |                         result = ma_data_source_read_pcm_frames(pDecoder->pBackend,
> pDecoder->pInputCache, pDecoder->inputCacheCap, &pDecoder->inputCacheRemaining);
67463 |                         if (result != MA_SUCCESS) {

```



```

67464 |             break;
67465 |         }
67466 |     }
67467 | }
67468 | } else {
67469 |     /* We have a way of determining the required number of input frames so just use the
> stack. */
67470 |     while (totalFramesReadOut < frameCount) {
67471 |         ma_uint8 pIntermediaryBuffer[MA_DATA_CONVERTER_STACK_BUFFER_SIZE]; /* In
> internal format. */
67472 |         ma_uint64 intermediaryBufferCap = sizeof(pIntermediaryBuffer) /
> ma_get_bytes_per_frame(internalFormat, internalChannels);
67473 |         ma_uint64 framesToReadThisIterationIn;
67474 |         ma_uint64 framesReadThisIterationIn;
67475 |         ma_uint64 framesToReadThisIterationOut;
67476 |         ma_uint64 framesReadThisIterationOut;
67477 |         ma_uint64 requiredInputFrameCount;
67478 |
67479 |         framesToReadThisIterationOut = (frameCount - totalFramesReadOut);
67480 |         framesToReadThisIterationIn = framesToReadThisIterationOut;
67481 |         if (framesToReadThisIterationIn > intermediaryBufferCap) {
67482 |             framesToReadThisIterationIn = intermediaryBufferCap;
67483 |         }
67484 |
67485 |         ma_data_converter_get_required_input_frame_count(&pDecoder->converter,
> framesToReadThisIterationOut, &requiredInputFrameCount);
67486 |         if (framesToReadThisIterationIn > requiredInputFrameCount) {
67487 |             framesToReadThisIterationIn = requiredInputFrameCount;
67488 |         }
67489 |
67490 |         if (requiredInputFrameCount > 0) {
67491 |             result = ma_data_source_read_pcm_frames(pDecoder->pBackend,
> pIntermediaryBuffer, framesToReadThisIterationIn, &framesReadThisIterationIn);
67492 |
67493 |             /*
67494 |             Note here that even if we've reached the end, we don't want to abort because
> there might be more output frames needing to be
67495 |             generated from cached input data, which might happen if resampling is being
> performed.
67496 |             */
67497 |             if (result != MA_SUCCESS && result != MA_AT_END) {
67498 |                 break;
67499 |             }
67500 |         } else {
67501 |             framesReadThisIterationIn = 0;
67502 |             pIntermediaryBuffer[0] = 0; /* <-- This is just to silence a static analysis
> warning. */
67503 |         }
67504 |
67505 |         /*
67506 |         At this point we have our decoded data in input format and now we need to
> convert to output format. Note that even if we didn't read any
67507 |         input frames, we still want to try processing frames because there may some
> output frames generated from cached input data.
67508 |         */
67509 |         framesReadThisIterationOut = framesToReadThisIterationOut;
67510 |         result = ma_data_converter_process_pcm_frames(&pDecoder->converter,
> pIntermediaryBuffer, &framesReadThisIterationIn, pRunningFramesOut, &framesReadThisIterationOut);
67511 |         if (result != MA_SUCCESS) {
67512 |             break;
67513 |         }
67514 |
67515 |         totalFramesReadOut += framesReadThisIterationOut;
67516 |
67517 |         if (pRunningFramesOut != NULL) {
67518 |             pRunningFramesOut = ma_offset_ptr(pRunningFramesOut,
> framesReadThisIterationOut * ma_get_bytes_per_frame(pDecoder->outputFormat,
> pDecoder->outputChannels));
67519 |         }
67520 |
67521 |         if (framesReadThisIterationIn == 0 && framesReadThisIterationOut == 0) {
67522 |             break; /* We're done. */
67523 |         }
67524 |     }
67525 | }
67526 | }
67527 | }
67528 |
67529 | pDecoder->readPointerInPCMFrames += totalFramesReadOut;

```

```

67530 |
67531 |     if (pFramesRead != NULL) {
67532 |         *pFramesRead = totalFramesReadOut;
67533 |     }
67534 |
67535 |     if (result == MA_SUCCESS && totalFramesReadOut == 0) {
67536 |         result = MA_AT_END;
67537 |     }
67538 |
67539 |     return result;
67540 | }
67541 |
67542 | MA_API ma_result ma_decoder_seek_to_pcm_frame(ma_decoder* pDecoder, ma_uint64 frameIndex)
67543 | {
67544 |     if (pDecoder == NULL) {
67545 |         return MA_INVALID_ARGS;
67546 |     }
67547 |
67548 |     if (pDecoder->pBackend != NULL) {
67549 |         ma_result result;
67550 |         ma_uint64 internalFrameIndex;
67551 |         ma_uint32 internalSampleRate;
67552 |         ma_uint64 currentFrameIndex;
67553 |
67554 |         result = ma_data_source_get_data_format(pDecoder->pBackend, NULL, NULL, &internalSampleRate,
67555 | > NULL, 0);
67556 |         if (result != MA_SUCCESS) {
67557 |             return result; /* Failed to retrieve the internal sample rate. */
67558 |         }
67559 |
67560 |         if (internalSampleRate == pDecoder->outputSampleRate) {
67561 |             internalFrameIndex = frameIndex;
67562 |         } else {
67563 |             internalFrameIndex = ma_calculate_frame_count_after_resampling(internalSampleRate,
67564 | > pDecoder->outputSampleRate, frameIndex);
67565 |         }
67566 |
67567 |         /* Only seek if we're requesting a different frame to what we're currently sitting on. */
67568 |         ma_data_source_get_cursor_in_pcm_frames(pDecoder->pBackend, &currentFrameIndex);
67569 |         if (currentFrameIndex != internalFrameIndex) {
67570 |             result = ma_data_source_seek_to_pcm_frame(pDecoder->pBackend, internalFrameIndex);
67571 |             if (result == MA_SUCCESS) {
67572 |                 pDecoder->readPointerInPCMFrames = frameIndex;
67573 |             }
67574 |
67575 |             /* Reset the data converter so that any cached data in the resampler is cleared. */
67576 |             ma_data_converter_reset(&pDecoder->converter);
67577 |         }
67578 |
67579 |         return result;
67580 |     }
67581 |
67582 |     /* Should never get here, but if we do it means onSeekToPCMFrame was not set by the backend. */
67583 |     return MA_INVALID_ARGS;
67584 | }
67585 |
67586 | MA_API ma_result ma_decoder_get_data_format(ma_decoder* pDecoder, ma_format* pFormat, ma_uint32*
67587 | > pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
67588 | {
67589 |     if (pDecoder == NULL) {
67590 |         return MA_INVALID_ARGS;
67591 |     }
67592 |
67593 |     if (pFormat != NULL) {
67594 |         *pFormat = pDecoder->outputFormat;
67595 |     }
67596 |
67597 |     if (pChannels != NULL) {
67598 |         *pChannels = pDecoder->outputChannels;
67599 |     }
67600 |
67601 |     if (pSampleRate != NULL) {
67602 |         *pSampleRate = pDecoder->outputSampleRate;
67603 |     }
67604 |
67605 |     if (pChannelMap != NULL) {
67606 |         ma_data_converter_get_output_channel_map(&pDecoder->converter, pChannelMap, channelMapCap);
67607 |     }
67608 | }

```

```

67606 |     return MA_SUCCESS;
67607 | }
67608 |
67609 | MA_API ma_result ma_decoder_get_cursor_in_pcm_frames(ma_decoder* pDecoder, ma_uint64* pCursor)
67610 | {
67611 |     if (pCursor == NULL) {
67612 |         return MA_INVALID_ARGS;
67613 |     }
67614 |
67615 |     *pCursor = 0;
67616 |
67617 |     if (pDecoder == NULL) {
67618 |         return MA_INVALID_ARGS;
67619 |     }
67620 |
67621 |     *pCursor = pDecoder->readPointerInPCMFrames;
67622 |
67623 |     return MA_SUCCESS;
67624 | }
67625 |
67626 | MA_API ma_result ma_decoder_get_length_in_pcm_frames(ma_decoder* pDecoder, ma_uint64* pLength)
67627 | {
67628 |     if (pLength == NULL) {
67629 |         return MA_INVALID_ARGS;
67630 |     }
67631 |
67632 |     *pLength = 0;
67633 |
67634 |     if (pDecoder == NULL) {
67635 |         return MA_INVALID_ARGS;
67636 |     }
67637 |
67638 |     if (pDecoder->pBackend != NULL) {
67639 |         ma_result result;
67640 |         ma_uint64 internalLengthInPCMFrames;
67641 |         ma_uint32 internalSampleRate;
67642 |
67643 |         result = ma_data_source_get_length_in_pcm_frames(pDecoder->pBackend,
67644 | > &internalLengthInPCMFrames);
67645 |         if (result != MA_SUCCESS) {
67646 |             return result; /* Failed to retrieve the internal length. */
67647 |         }
67648 |
67649 |         result = ma_data_source_get_data_format(pDecoder->pBackend, NULL, NULL, &internalSampleRate,
67650 | > NULL, 0);
67651 |         if (result != MA_SUCCESS) {
67652 |             return result; /* Failed to retrieve the internal sample rate. */
67653 |         }
67654 |
67655 |         if (internalSampleRate == pDecoder->outputSampleRate) {
67656 |             *pLength = internalLengthInPCMFrames;
67657 |         } else {
67658 |             *pLength = ma_calculate_frame_count_after_resampling(pDecoder->outputSampleRate,
67659 | > internalSampleRate, internalLengthInPCMFrames);
67660 |         }
67661 |
67662 |         return MA_SUCCESS;
67663 |     } else {
67664 |         return MA_NO_BACKEND;
67665 |     }
67666 | }
67667 |
67668 | MA_API ma_result ma_decoder_get_available_frames(ma_decoder* pDecoder, ma_uint64* pAvailableFrames)
67669 | {
67670 |     ma_result result;
67671 |     ma_uint64 totalFrameCount;
67672 |
67673 |     if (pAvailableFrames == NULL) {
67674 |         return MA_INVALID_ARGS;
67675 |     }
67676 |
67677 |     *pAvailableFrames = 0;
67678 |
67679 |     if (pDecoder == NULL) {
67680 |         return MA_INVALID_ARGS;
67681 |     }
67682 |
67683 |     result = ma_decoder_get_length_in_pcm_frames(pDecoder, &totalFrameCount);
67684 |     if (result != MA_SUCCESS) {

```

```

67682 |         return result;
67683 |     }
67684 |
67685 |     if (totalFrameCount <= pDecoder->readPointerInPCMFrames) {
67686 |         *pAvailableFrames = 0;
67687 |     } else {
67688 |         *pAvailableFrames = totalFrameCount - pDecoder->readPointerInPCMFrames;
67689 |     }
67690 |
67691 |     return MA_SUCCESS;
67692 | }
67693 |
67694 |
67695 | static ma_result ma_decoder_full_decode_and_uninit(ma_decoder* pDecoder, ma_decoder_config*
> pConfigOut, ma_uint64* pFrameCountOut, void** ppPCMFramesOut)
67696 | {
67697 |     ma_result result;
67698 |     ma_uint64 totalFrameCount;
67699 |     ma_uint64 bpf;
67700 |     ma_uint64 dataCapInFrames;
67701 |     void* pPCMFramesOut;
67702 |
67703 |     MA_ASSERT(pDecoder != NULL);
67704 |
67705 |     totalFrameCount = 0;
67706 |     bpf = ma_get_bytes_per_frame(pDecoder->outputFormat, pDecoder->outputChannels);
67707 |
67708 |     /* The frame count is unknown until we try reading. Thus, we just run in a loop. */
67709 |     dataCapInFrames = 0;
67710 |     pPCMFramesOut = NULL;
67711 |     for (;;) {
67712 |         ma_uint64 frameCountToTryReading;
67713 |         ma_uint64 framesJustRead;
67714 |
67715 |         /* Make room if there's not enough. */
67716 |         if (totalFrameCount == dataCapInFrames) {
67717 |             void* pNewPCMFramesOut;
67718 |             ma_uint64 newDataCapInFrames = dataCapInFrames*2;
67719 |             if (newDataCapInFrames == 0) {
67720 |                 newDataCapInFrames = 4096;
67721 |             }
67722 |
67723 |             if ((newDataCapInFrames * bpf) > MA_SIZE_MAX) {
67724 |                 ma_free(pPCMFramesOut, &pDecoder->allocationCallbacks);
67725 |                 return MA_TOO_BIG;
67726 |             }
67727 |
67728 |             pNewPCMFramesOut = (void*)ma_realloc(pPCMFramesOut, (size_t)(newDataCapInFrames * bpf),
> &pDecoder->allocationCallbacks);
67729 |             if (pNewPCMFramesOut == NULL) {
67730 |                 ma_free(pPCMFramesOut, &pDecoder->allocationCallbacks);
67731 |                 return MA_OUT_OF_MEMORY;
67732 |             }
67733 |
67734 |             dataCapInFrames = newDataCapInFrames;
67735 |             pPCMFramesOut = pNewPCMFramesOut;
67736 |         }
67737 |
67738 |         frameCountToTryReading = dataCapInFrames - totalFrameCount;
67739 |         MA_ASSERT(frameCountToTryReading > 0);
67740 |
67741 |         result = ma_decoder_read_pcm_frames(pDecoder, (ma_uint8*)pPCMFramesOut + (totalFrameCount *
> bpf), frameCountToTryReading, &framesJustRead);
67742 |         totalFrameCount += framesJustRead;
67743 |
67744 |         if (result != MA_SUCCESS) {
67745 |             break;
67746 |         }
67747 |
67748 |         if (framesJustRead < frameCountToTryReading) {
67749 |             break;
67750 |         }
67751 |     }
67752 |
67753 |
67754 |     if (pConfigOut != NULL) {
67755 |         pConfigOut->format = pDecoder->outputFormat;
67756 |         pConfigOut->channels = pDecoder->outputChannels;
67757 |         pConfigOut->sampleRate = pDecoder->outputSampleRate;

```

```

67758 | }
67759 |
67760 | if (ppPCMFramesOut != NULL) {
67761 |     *ppPCMFramesOut = pPCMFramesOut;
67762 | } else {
67763 |     ma_free(pPCMFramesOut, &pDecoder->allocationCallbacks);
67764 | }
67765 |
67766 | if (pFrameCountOut != NULL) {
67767 |     *pFrameCountOut = totalFrameCount;
67768 | }
67769 |
67770 | ma_decoder_uninit(pDecoder);
67771 | return MA_SUCCESS;
67772 | }
67773 |
67774 | MA_API ma_result ma_decode_from_vfs(ma_vfs* pVFS, const char* pFilePath, ma_decoder_config* pConfig,
> ma_uint64* pFrameCountOut, void** ppPCMFramesOut)
67775 | {
67776 |     ma_result result;
67777 |     ma_decoder_config config;
67778 |     ma_decoder decoder;
67779 |
67780 |     if (pFrameCountOut != NULL) {
67781 |         *pFrameCountOut = 0;
67782 |     }
67783 |     if (ppPCMFramesOut != NULL) {
67784 |         *ppPCMFramesOut = NULL;
67785 |     }
67786 |
67787 |     config = ma_decoder_config_init_copy(pConfig);
67788 |
67789 |     result = ma_decoder_init_vfs(pVFS, pFilePath, &config, &decoder);
67790 |     if (result != MA_SUCCESS) {
67791 |         return result;
67792 |     }
67793 |
67794 |     result = ma_decoder__full_decode_and_uninit(&decoder, pConfig, pFrameCountOut, ppPCMFramesOut);
67795 |
67796 |     return result;
67797 | }
67798 |
67799 | MA_API ma_result ma_decode_file(const char* pFilePath, ma_decoder_config* pConfig, ma_uint64*
> pFrameCountOut, void** ppPCMFramesOut)
67800 | {
67801 |     return ma_decode_from_vfs(NULL, pFilePath, pConfig, pFrameCountOut, ppPCMFramesOut);
67802 | }
67803 |
67804 | MA_API ma_result ma_decode_memory(const void* pData, size_t dataSize, ma_decoder_config* pConfig,
> ma_uint64* pFrameCountOut, void** ppPCMFramesOut)
67805 | {
67806 |     ma_decoder_config config;
67807 |     ma_decoder decoder;
67808 |     ma_result result;
67809 |
67810 |     if (pFrameCountOut != NULL) {
67811 |         *pFrameCountOut = 0;
67812 |     }
67813 |     if (ppPCMFramesOut != NULL) {
67814 |         *ppPCMFramesOut = NULL;
67815 |     }
67816 |
67817 |     if (pData == NULL || dataSize == 0) {
67818 |         return MA_INVALID_ARGS;
67819 |     }
67820 |
67821 |     config = ma_decoder_config_init_copy(pConfig);
67822 |
67823 |     result = ma_decoder_init_memory(pData, dataSize, &config, &decoder);
67824 |     if (result != MA_SUCCESS) {
67825 |         return result;
67826 |     }
67827 |
67828 |     return ma_decoder__full_decode_and_uninit(&decoder, pConfig, pFrameCountOut, ppPCMFramesOut);
67829 | }
67830 | #endif /* MA_NO_DECODING */
67831 |
67832 |
67833 | #ifndef MA_NO_ENCODING

```

```

67834 |
67835 | #if defined(MA_HAS_WAV)
67836 | static size_t ma_encoder__internal_on_write_wav(void* pUserData, const void* pData, size_t
> bytesToWrite)
67837 | {
67838 |     ma_encoder* pEncoder = (ma_encoder*)pUserData;
67839 |     size_t bytesWritten = 0;
67840 |
67841 |     MA_ASSERT(pEncoder != NULL);
67842 |
67843 |     pEncoder->onWrite(pEncoder, pData, bytesToWrite, &bytesWritten);
67844 |     return bytesWritten;
67845 | }
67846 |
67847 | static ma_bool32 ma_encoder__internal_on_seek_wav(void* pUserData, int offset, ma_dr_wav_seek_origin
> origin)
67848 | {
67849 |     ma_encoder* pEncoder = (ma_encoder*)pUserData;
67850 |     ma_result result;
67851 |     ma_seek_origin maSeekOrigin;
67852 |
67853 |     MA_ASSERT(pEncoder != NULL);
67854 |
67855 |     maSeekOrigin = ma_seek_origin_start;
67856 |     if (origin == MA_DR_WAV_SEEK_CUR) {
67857 |         maSeekOrigin = ma_seek_origin_current;
67858 |     } else if (origin == MA_DR_WAV_SEEK_END) {
67859 |         maSeekOrigin = ma_seek_origin_end;
67860 |     }
67861 |
67862 |     result = pEncoder->onSeek(pEncoder, offset, maSeekOrigin);
67863 |     if (result != MA_SUCCESS) {
67864 |         return MA_FALSE;
67865 |     } else {
67866 |         return MA_TRUE;
67867 |     }
67868 | }
67869 |
67870 | static ma_result ma_encoder__on_init_wav(ma_encoder* pEncoder)
67871 | {
67872 |     ma_dr_wav_data_format wavFormat;
67873 |     ma_allocation_callbacks allocationCallbacks;
67874 |     ma_dr_wav* pWav;
67875 |
67876 |     MA_ASSERT(pEncoder != NULL);
67877 |
67878 |     pWav = (ma_dr_wav*)ma_malloc(sizeof(*pWav), &pEncoder->config.allocationCallbacks);
67879 |     if (pWav == NULL) {
67880 |         return MA_OUT_OF_MEMORY;
67881 |     }
67882 |
67883 |     wavFormat.container = ma_dr_wav_container_riff;
67884 |     wavFormat.channels = pEncoder->config.channels;
67885 |     wavFormat.sampleRate = pEncoder->config.sampleRate;
67886 |     wavFormat.bitsPerSample = ma_get_bytes_per_sample(pEncoder->config.format) * 8;
67887 |     if (pEncoder->config.format == ma_format_f32) {
67888 |         wavFormat.format = MA_DR_WAVE_FORMAT_IEEE_FLOAT;
67889 |     } else {
67890 |         wavFormat.format = MA_DR_WAVE_FORMAT_PCM;
67891 |     }
67892 |
67893 |     allocationCallbacks.pUserData = pEncoder->config.allocationCallbacks.pUserData;
67894 |     allocationCallbacks.onMalloc = pEncoder->config.allocationCallbacks.onMalloc;
67895 |     allocationCallbacks.onRealloc = pEncoder->config.allocationCallbacks.onRealloc;
67896 |     allocationCallbacks.onFree = pEncoder->config.allocationCallbacks.onFree;
67897 |
67898 |     if (!ma_dr_wav_init_write(pWav, &wavFormat, ma_encoder__internal_on_write_wav,
> ma_encoder__internal_on_seek_wav, pEncoder, &allocationCallbacks)) {
67899 |         return MA_ERROR;
67900 |     }
67901 |
67902 |     pEncoder->pInternalEncoder = pWav;
67903 |
67904 |     return MA_SUCCESS;
67905 | }
67906 |
67907 | static void ma_encoder__on_uninit_wav(ma_encoder* pEncoder)
67908 | {
67909 |     ma_dr_wav* pWav;

```

```

67910 |
67911 |     MA_ASSERT(pEncoder != NULL);
67912 |
67913 |     pWav = (ma_dr_wav*)pEncoder->pInternalEncoder;
67914 |     MA_ASSERT(pWav != NULL);
67915 |
67916 |     ma_dr_wav_uninit(pWav);
67917 |     ma_free(pWav, &pEncoder->config.allocationCallbacks);
67918 | }
67919 |
67920 | static ma_result ma_encoder__on_write_pcm_frames_wav(ma_encoder* pEncoder, const void* pFramesIn,
> ma_uint64 frameCount, ma_uint64* pFramesWritten)
67921 | {
67922 |     ma_dr_wav* pWav;
67923 |     ma_uint64 framesWritten;
67924 |
67925 |     MA_ASSERT(pEncoder != NULL);
67926 |
67927 |     pWav = (ma_dr_wav*)pEncoder->pInternalEncoder;
67928 |     MA_ASSERT(pWav != NULL);
67929 |
67930 |     framesWritten = ma_dr_wav_write_pcm_frames(pWav, frameCount, pFramesIn);
67931 |
67932 |     if (pFramesWritten != NULL) {
67933 |         *pFramesWritten = framesWritten;
67934 |     }
67935 |
67936 |     return MA_SUCCESS;
67937 | }
67938 | #endif
67939 |
67940 | MA_API ma_encoder_config ma_encoder_config_init(ma_encoding_format encodingFormat, ma_format format,
> ma_uint32 channels, ma_uint32 sampleRate)
67941 | {
67942 |     ma_encoder_config config;
67943 |
67944 |     MA_ZERO_OBJECT(&config);
67945 |     config.encodingFormat = encodingFormat;
67946 |     config.format = format;
67947 |     config.channels = channels;
67948 |     config.sampleRate = sampleRate;
67949 |
67950 |     return config;
67951 | }
67952 |
67953 | MA_API ma_result ma_encoder_preinit(const ma_encoder_config* pConfig, ma_encoder* pEncoder)
67954 | {
67955 |     ma_result result;
67956 |
67957 |     if (pEncoder == NULL) {
67958 |         return MA_INVALID_ARGS;
67959 |     }
67960 |
67961 |     MA_ZERO_OBJECT(pEncoder);
67962 |
67963 |     if (pConfig == NULL) {
67964 |         return MA_INVALID_ARGS;
67965 |     }
67966 |
67967 |     if (pConfig->format == ma_format_unknown || pConfig->channels == 0 || pConfig->sampleRate == 0)
> {
67968 |         return MA_INVALID_ARGS;
67969 |     }
67970 |
67971 |     pEncoder->config = *pConfig;
67972 |
67973 |     result = ma_allocation_callbacks_init_copy(&pEncoder->config.allocationCallbacks,
> &pConfig->allocationCallbacks);
67974 |     if (result != MA_SUCCESS) {
67975 |         return result;
67976 |     }
67977 |
67978 |     return MA_SUCCESS;
67979 | }
67980 |
67981 | MA_API ma_result ma_encoder_init__internal(ma_encoder_write_proc onWrite, ma_encoder_seek_proc
> onSeek, void* pUserData, ma_encoder* pEncoder)
67982 | {
67983 |     ma_result result = MA_SUCCESS;

```



```

67984 |
67985 |     /* This assumes ma_encoder_preinit() has been called prior. */
67986 |     MA_ASSERT(pEncoder != NULL);
67987 |
67988 |     if (onWrite == NULL || onSeek == NULL) {
67989 |         return MA_INVALID_ARGS;
67990 |     }
67991 |
67992 |     pEncoder->onWrite    = onWrite;
67993 |     pEncoder->onSeek     = onSeek;
67994 |     pEncoder->pUserData  = pUserData;
67995 |
67996 |     switch (pEncoder->config.encodingFormat)
67997 |     {
67998 |         case ma_encoding_format_wav:
67999 |         {
68000 |             #if defined(MA_HAS_WAV)
68001 |                 pEncoder->onInit          = ma_encoder__on_init_wav;
68002 |                 pEncoder->onUninit        = ma_encoder__on_uninit_wav;
68003 |                 pEncoder->onWritePCMframes = ma_encoder__on_write_pcm_frames_wav;
68004 |             #else
68005 |                 result = MA_NO_BACKEND;
68006 |             #endif
68007 |             } break;
68008 |
68009 |         default:
68010 |         {
68011 |             result = MA_INVALID_ARGS;
68012 |             } break;
68013 |     }
68014 |
68015 |     /* Getting here means we should have our backend callbacks set up. */
68016 |     if (result == MA_SUCCESS) {
68017 |         result = pEncoder->onInit(pEncoder);
68018 |     }
68019 |
68020 |     return result;
68021 | }
68022 |
68023 | static ma_result ma_encoder__on_write_vfs(ma_encoder* pEncoder, const void* pBufferIn, size_t
> bytesToWrite, size_t* pBytesWritten)
68024 | {
68025 |     return ma_vfs_or_default_write(pEncoder->data.vfs.pVFS, pEncoder->data.vfs.file, pBufferIn,
> bytesToWrite, pBytesWritten);
68026 | }
68027 |
68028 | static ma_result ma_encoder__on_seek_vfs(ma_encoder* pEncoder, ma_int64 offset, ma_seek_origin
> origin)
68029 | {
68030 |     return ma_vfs_or_default_seek(pEncoder->data.vfs.pVFS, pEncoder->data.vfs.file, offset, origin);
68031 | }
68032 |
68033 | MA_API ma_result ma_encoder_init_vfs(ma_vfs* pVFS, const char* pFilePath, const ma_encoder_config*
> pConfig, ma_encoder* pEncoder)
68034 | {
68035 |     ma_result result;
68036 |     ma_vfs_file file;
68037 |
68038 |     result = ma_encoder_preinit(pConfig, pEncoder);
68039 |     if (result != MA_SUCCESS) {
68040 |         return result;
68041 |     }
68042 |
68043 |     /* Now open the file. If this fails we don't need to uninitialized the encoder. */
68044 |     result = ma_vfs_or_default_open(pVFS, pFilePath, MA_OPEN_MODE_WRITE, &file);
68045 |     if (result != MA_SUCCESS) {
68046 |         return result;
68047 |     }
68048 |
68049 |     pEncoder->data.vfs.pVFS = pVFS;
68050 |     pEncoder->data.vfs.file = file;
68051 |
68052 |     result = ma_encoder_init__internal(ma_encoder__on_write_vfs, ma_encoder__on_seek_vfs, NULL,
> pEncoder);
68053 |     if (result != MA_SUCCESS) {
68054 |         ma_vfs_or_default_close(pVFS, file);
68055 |         return result;
68056 |     }
68057 |

```

```

68058 |     return MA_SUCCESS;
68059 | }
68060 |
68061 | MA_API ma_result ma_encoder_init_vfs_w(ma_vfs* pVFS, const wchar_t* pFilePath, const
> ma_encoder_config* pConfig, ma_encoder* pEncoder)
68062 | {
68063 |     ma_result result;
68064 |     ma_vfs_file file;
68065 |
68066 |     result = ma_encoder_preinit(pConfig, pEncoder);
68067 |     if (result != MA_SUCCESS) {
68068 |         return result;
68069 |     }
68070 |
68071 |     /* Now open the file. If this fails we don't need to uninitialized the encoder. */
68072 |     result = ma_vfs_or_default_open_w(pVFS, pFilePath, MA_OPEN_MODE_WRITE, &file);
68073 |     if (result != MA_SUCCESS) {
68074 |         return result;
68075 |     }
68076 |
68077 |     pEncoder->data.vfs.pVFS = pVFS;
68078 |     pEncoder->data.vfs.file = file;
68079 |
68080 |     result = ma_encoder_init__internal(ma_encoder__on_write_vfs, ma_encoder__on_seek_vfs, NULL,
> pEncoder);
68081 |     if (result != MA_SUCCESS) {
68082 |         ma_vfs_or_default_close(pVFS, file);
68083 |         return result;
68084 |     }
68085 |
68086 |     return MA_SUCCESS;
68087 | }
68088 |
68089 | MA_API ma_result ma_encoder_init_file(const char* pFilePath, const ma_encoder_config* pConfig,
> ma_encoder* pEncoder)
68090 | {
68091 |     return ma_encoder_init_vfs(NULL, pFilePath, pConfig, pEncoder);
68092 | }
68093 |
68094 | MA_API ma_result ma_encoder_init_file_w(const wchar_t* pFilePath, const ma_encoder_config* pConfig,
> ma_encoder* pEncoder)
68095 | {
68096 |     return ma_encoder_init_vfs_w(NULL, pFilePath, pConfig, pEncoder);
68097 | }
68098 |
68099 | MA_API ma_result ma_encoder_init(ma_encoder_write_proc onWrite, ma_encoder_seek_proc onSeek, void*
> pUserData, const ma_encoder_config* pConfig, ma_encoder* pEncoder)
68100 | {
68101 |     ma_result result;
68102 |
68103 |     result = ma_encoder_preinit(pConfig, pEncoder);
68104 |     if (result != MA_SUCCESS) {
68105 |         return result;
68106 |     }
68107 |
68108 |     return ma_encoder_init__internal(onWrite, onSeek, pUserData, pEncoder);
68109 | }
68110 |
68111 |
68112 | MA_API void ma_encoder_uninit(ma_encoder* pEncoder)
68113 | {
68114 |     if (pEncoder == NULL) {
68115 |         return;
68116 |     }
68117 |
68118 |     if (pEncoder->onUninit) {
68119 |         pEncoder->onUninit(pEncoder);
68120 |     }
68121 |
68122 |     /* If we have a file handle, close it. */
68123 |     if (pEncoder->onWrite == ma_encoder__on_write_vfs) {
68124 |         ma_vfs_or_default_close(pEncoder->data.vfs.pVFS, pEncoder->data.vfs.file);
68125 |         pEncoder->data.vfs.file = NULL;
68126 |     }
68127 | }
68128 |
68129 |
68130 | MA_API ma_result ma_encoder_write_pcm_frames(ma_encoder* pEncoder, const void* pFramesIn, ma_uint64
> frameCount, ma_uint64* pFramesWritten)

```

```

68131 | {
68132 |     if (pFramesWritten != NULL) {
68133 |         *pFramesWritten = 0;
68134 |     }
68135 |
68136 |     if (pEncoder == NULL || pFramesIn == NULL) {
68137 |         return MA_INVALID_ARGS;
68138 |     }
68139 |
68140 |     return pEncoder->onWritePCMFrames(pEncoder, pFramesIn, frameCount, pFramesWritten);
68141 | }
68142 | #endif /* MA_NO_ENCODING */
68143 |
68144 |
68145 |
68146 | /*****
> *****/
68147 |
68148 | Generation
68149 |
68150 | *****/
> *****/
68151 | #ifndef MA_NO_GENERATION
68152 | MA_API ma_waveform_config ma_waveform_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, ma_waveform_type type, double amplitude, double frequency)
68153 | {
68154 |     ma_waveform_config config;
68155 |
68156 |     MA_ZERO_OBJECT(&config);
68157 |     config.format      = format;
68158 |     config.channels    = channels;
68159 |     config.sampleRate  = sampleRate;
68160 |     config.type        = type;
68161 |     config.amplitude   = amplitude;
68162 |     config.frequency   = frequency;
68163 |
68164 |     return config;
68165 | }
68166 |
68167 | static ma_result ma_waveform__data_source_on_read(ma_data_source* pDataSource, void* pFramesOut,
> ma_uint64 frameCount, ma_uint64* pFramesRead)
68168 | {
68169 |     return ma_waveform_read_pcm_frames((ma_waveform*)pDataSource, pFramesOut, frameCount,
> pFramesRead);
68170 | }
68171 |
68172 | static ma_result ma_waveform__data_source_on_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
68173 | {
68174 |     return ma_waveform_seek_to_pcm_frame((ma_waveform*)pDataSource, frameIndex);
68175 | }
68176 |
68177 | static ma_result ma_waveform__data_source_on_get_data_format(ma_data_source* pDataSource, ma_format*
> pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
68178 | {
68179 |     ma_waveform* pWaveform = (ma_waveform*)pDataSource;
68180 |
68181 |     *pFormat      = pWaveform->config.format;
68182 |     *pChannels    = pWaveform->config.channels;
68183 |     *pSampleRate  = pWaveform->config.sampleRate;
68184 |     ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pWaveform->config.channels);
68185 |
68186 |     return MA_SUCCESS;
68187 | }
68188 |
68189 | static ma_result ma_waveform__data_source_on_get_cursor(ma_data_source* pDataSource, ma_uint64*
> pCursor)
68190 | {
68191 |     ma_waveform* pWaveform = (ma_waveform*)pDataSource;
68192 |
68193 |     *pCursor = (ma_uint64)(pWaveform->time / pWaveform->advance);
68194 |
68195 |     return MA_SUCCESS;
68196 | }
68197 |
68198 | static double ma_waveform__calculate_advance(ma_uint32 sampleRate, double frequency)
68199 | {
68200 |     return (1.0 / (sampleRate / frequency));

```

```

68201 | }
68202 |
68203 | static void ma_waveform__update_advance(ma_waveform* pWaveform)
68204 | {
68205 |     pWaveform->advance = ma_waveform__calculate_advance(pWaveform->config.sampleRate,
> pWaveform->config.frequency);
68206 | }
68207 |
68208 | static ma_data_source_vtable g_ma_waveform_data_source_vtable =
68209 | {
68210 |     ma_waveform__data_source_on_read,
68211 |     ma_waveform__data_source_on_seek,
68212 |     ma_waveform__data_source_on_get_data_format,
68213 |     ma_waveform__data_source_on_get_cursor,
68214 |     NULL, /* onGetLength. There's no notion of a length in waveforms. */
68215 |     NULL, /* onSetLooping */
68216 |     0
68217 | };
68218 |
68219 | MA_API ma_result ma_waveform_init(const ma_waveform_config* pConfig, ma_waveform* pWaveform)
68220 | {
68221 |     ma_result result;
68222 |     ma_data_source_config dataSourceConfig;
68223 |
68224 |     if (pWaveform == NULL) {
68225 |         return MA_INVALID_ARGS;
68226 |     }
68227 |
68228 |     MA_ZERO_OBJECT(pWaveform);
68229 |
68230 |     dataSourceConfig = ma_data_source_config_init();
68231 |     dataSourceConfig.vtable = &g_ma_waveform_data_source_vtable;
68232 |
68233 |     result = ma_data_source_init(&dataSourceConfig, &pWaveform->ds);
68234 |     if (result != MA_SUCCESS) {
68235 |         return result;
68236 |     }
68237 |
68238 |     pWaveform->config = *pConfig;
68239 |     pWaveform->advance = ma_waveform__calculate_advance(pWaveform->config.sampleRate,
> pWaveform->config.frequency);
68240 |     pWaveform->time = 0;
68241 |
68242 |     return MA_SUCCESS;
68243 | }
68244 |
68245 | MA_API void ma_waveform_uninit(ma_waveform* pWaveform)
68246 | {
68247 |     if (pWaveform == NULL) {
68248 |         return;
68249 |     }
68250 |
68251 |     ma_data_source_uninit(&pWaveform->ds);
68252 | }
68253 |
68254 | MA_API ma_result ma_waveform_set_amplitude(ma_waveform* pWaveform, double amplitude)
68255 | {
68256 |     if (pWaveform == NULL) {
68257 |         return MA_INVALID_ARGS;
68258 |     }
68259 |
68260 |     pWaveform->config.amplitude = amplitude;
68261 |     return MA_SUCCESS;
68262 | }
68263 |
68264 | MA_API ma_result ma_waveform_set_frequency(ma_waveform* pWaveform, double frequency)
68265 | {
68266 |     if (pWaveform == NULL) {
68267 |         return MA_INVALID_ARGS;
68268 |     }
68269 |
68270 |     pWaveform->config.frequency = frequency;
68271 |     ma_waveform__update_advance(pWaveform);
68272 |
68273 |     return MA_SUCCESS;
68274 | }
68275 |
68276 | MA_API ma_result ma_waveform_set_type(ma_waveform* pWaveform, ma_waveform_type type)
68277 | {

```

```

68278 |     if (pWaveform == NULL) {
68279 |         return MA_INVALID_ARGS;
68280 |     }
68281 |
68282 |     pWaveform->config.type = type;
68283 |     return MA_SUCCESS;
68284 | }
68285 |
68286 | MA_API ma_result ma_waveform_set_sample_rate(ma_waveform* pWaveform, ma_uint32 sampleRate)
68287 | {
68288 |     if (pWaveform == NULL) {
68289 |         return MA_INVALID_ARGS;
68290 |     }
68291 |
68292 |     pWaveform->config.sampleRate = sampleRate;
68293 |     ma_waveform__update_advance(pWaveform);
68294 |
68295 |     return MA_SUCCESS;
68296 | }
68297 |
68298 | static float ma_waveform_sine_f32(double time, double amplitude)
68299 | {
68300 |     return (float)(ma_sind(MA_TAU_D * time) * amplitude);
68301 | }
68302 |
68303 | static ma_int16 ma_waveform_sine_s16(double time, double amplitude)
68304 | {
68305 |     return ma_pcm_sample_f32_to_s16(ma_waveform_sine_f32(time, amplitude));
68306 | }
68307 |
68308 | static float ma_waveform_square_f32(double time, double dutyCycle, double amplitude)
68309 | {
68310 |     double f = time - (ma_int64)time;
68311 |     double r;
68312 |
68313 |     if (f < dutyCycle) {
68314 |         r = amplitude;
68315 |     } else {
68316 |         r = -amplitude;
68317 |     }
68318 |
68319 |     return (float)r;
68320 | }
68321 |
68322 | static ma_int16 ma_waveform_square_s16(double time, double dutyCycle, double amplitude)
68323 | {
68324 |     return ma_pcm_sample_f32_to_s16(ma_waveform_square_f32(time, dutyCycle, amplitude));
68325 | }
68326 |
68327 | static float ma_waveform_triangle_f32(double time, double amplitude)
68328 | {
68329 |     double f = time - (ma_int64)time;
68330 |     double r;
68331 |
68332 |     r = 2 * ma_abs(2 * (f - 0.5)) - 1;
68333 |
68334 |     return (float)(r * amplitude);
68335 | }
68336 |
68337 | static ma_int16 ma_waveform_triangle_s16(double time, double amplitude)
68338 | {
68339 |     return ma_pcm_sample_f32_to_s16(ma_waveform_triangle_f32(time, amplitude));
68340 | }
68341 |
68342 | static float ma_waveform_sawtooth_f32(double time, double amplitude)
68343 | {
68344 |     double f = time - (ma_int64)time;
68345 |     double r;
68346 |
68347 |     r = 2 * (f - 0.5);
68348 |
68349 |     return (float)(r * amplitude);
68350 | }
68351 |
68352 | static ma_int16 ma_waveform_sawtooth_s16(double time, double amplitude)
68353 | {
68354 |     return ma_pcm_sample_f32_to_s16(ma_waveform_sawtooth_f32(time, amplitude));
68355 | }
68356 |

```

```

68357 | static void ma_waveform_read_pcm_frames__sine(ma_waveform* pWaveform, void* pFramesOut, ma_uint64
> frameCount)
68358 | {
68359 |     ma_uint64 iFrame;
68360 |     ma_uint64 iChannel;
68361 |     ma_uint32 bps = ma_get_bytes_per_sample(pWaveform->config.format);
68362 |     ma_uint32 bpf = bps * pWaveform->config.channels;
68363 |
68364 |     MA_ASSERT(pWaveform != NULL);
68365 |     MA_ASSERT(pFramesOut != NULL);
68366 |
68367 |     if (pWaveform->config.format == ma_format_f32) {
68368 |         float* pFramesOutF32 = (float*)pFramesOut;
68369 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68370 |             float s = ma_waveform_sine_f32(pWaveform->time, pWaveform->config.amplitude);
68371 |             pWaveform->time += pWaveform->advance;
68372 |
68373 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68374 |                 pFramesOutF32[iFrame*pWaveform->config.channels + iChannel] = s;
68375 |             }
68376 |         }
68377 |     } else if (pWaveform->config.format == ma_format_s16) {
68378 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
68379 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68380 |             ma_int16 s = ma_waveform_sine_s16(pWaveform->time, pWaveform->config.amplitude);
68381 |             pWaveform->time += pWaveform->advance;
68382 |
68383 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68384 |                 pFramesOutS16[iFrame*pWaveform->config.channels + iChannel] = s;
68385 |             }
68386 |         }
68387 |     } else {
68388 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68389 |             float s = ma_waveform_sine_f32(pWaveform->time, pWaveform->config.amplitude);
68390 |             pWaveform->time += pWaveform->advance;
68391 |
68392 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68393 |                 ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pWaveform->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
68394 |             }
68395 |         }
68396 |     }
68397 | }
68398 |
68399 | static void ma_waveform_read_pcm_frames__square(ma_waveform* pWaveform, double dutyCycle, void*
> pFramesOut, ma_uint64 frameCount)
68400 | {
68401 |     ma_uint64 iFrame;
68402 |     ma_uint64 iChannel;
68403 |     ma_uint32 bps = ma_get_bytes_per_sample(pWaveform->config.format);
68404 |     ma_uint32 bpf = bps * pWaveform->config.channels;
68405 |
68406 |     MA_ASSERT(pWaveform != NULL);
68407 |     MA_ASSERT(pFramesOut != NULL);
68408 |
68409 |     if (pWaveform->config.format == ma_format_f32) {
68410 |         float* pFramesOutF32 = (float*)pFramesOut;
68411 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68412 |             float s = ma_waveform_square_f32(pWaveform->time, dutyCycle,
> pWaveform->config.amplitude);
68413 |             pWaveform->time += pWaveform->advance;
68414 |
68415 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68416 |                 pFramesOutF32[iFrame*pWaveform->config.channels + iChannel] = s;
68417 |             }
68418 |         }
68419 |     } else if (pWaveform->config.format == ma_format_s16) {
68420 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
68421 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68422 |             ma_int16 s = ma_waveform_square_s16(pWaveform->time, dutyCycle,
> pWaveform->config.amplitude);
68423 |             pWaveform->time += pWaveform->advance;
68424 |
68425 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68426 |                 pFramesOutS16[iFrame*pWaveform->config.channels + iChannel] = s;
68427 |             }
68428 |         }
68429 |     } else {
68430 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {

```

```

68431 |         float s = ma_waveform_square_f32(pWaveform->time, dutyCycle,
> pWaveform->config.amplitude);
68432 |         pWaveform->time += pWaveform->advance;
68433 |
68434 |         for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68435 |             ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pWaveform->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
68436 |         }
68437 |     }
68438 | }
68439 | }
68440 |
68441 | static void ma_waveform_read_pcm_frames__triangle(ma_waveform* pWaveform, void* pFramesOut,
> ma_uint64 frameCount)
68442 | {
68443 |     ma_uint64 iFrame;
68444 |     ma_uint64 iChannel;
68445 |     ma_uint32 bps = ma_get_bytes_per_sample(pWaveform->config.format);
68446 |     ma_uint32 bpf = bps * pWaveform->config.channels;
68447 |
68448 |     MA_ASSERT(pWaveform != NULL);
68449 |     MA_ASSERT(pFramesOut != NULL);
68450 |
68451 |     if (pWaveform->config.format == ma_format_f32) {
68452 |         float* pFramesOutF32 = (float*)pFramesOut;
68453 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68454 |             float s = ma_waveform_triangle_f32(pWaveform->time, pWaveform->config.amplitude);
68455 |             pWaveform->time += pWaveform->advance;
68456 |
68457 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68458 |                 pFramesOutF32[iFrame*pWaveform->config.channels + iChannel] = s;
68459 |             }
68460 |         }
68461 |     } else if (pWaveform->config.format == ma_format_s16) {
68462 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
68463 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68464 |             ma_int16 s = ma_waveform_triangle_s16(pWaveform->time, pWaveform->config.amplitude);
68465 |             pWaveform->time += pWaveform->advance;
68466 |
68467 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68468 |                 pFramesOutS16[iFrame*pWaveform->config.channels + iChannel] = s;
68469 |             }
68470 |         }
68471 |     } else {
68472 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68473 |             float s = ma_waveform_triangle_f32(pWaveform->time, pWaveform->config.amplitude);
68474 |             pWaveform->time += pWaveform->advance;
68475 |
68476 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68477 |                 ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pWaveform->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
68478 |             }
68479 |         }
68480 |     }
68481 | }
68482 |
68483 | static void ma_waveform_read_pcm_frames__sawtooth(ma_waveform* pWaveform, void* pFramesOut,
> ma_uint64 frameCount)
68484 | {
68485 |     ma_uint64 iFrame;
68486 |     ma_uint64 iChannel;
68487 |     ma_uint32 bps = ma_get_bytes_per_sample(pWaveform->config.format);
68488 |     ma_uint32 bpf = bps * pWaveform->config.channels;
68489 |
68490 |     MA_ASSERT(pWaveform != NULL);
68491 |     MA_ASSERT(pFramesOut != NULL);
68492 |
68493 |     if (pWaveform->config.format == ma_format_f32) {
68494 |         float* pFramesOutF32 = (float*)pFramesOut;
68495 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68496 |             float s = ma_waveform_sawtooth_f32(pWaveform->time, pWaveform->config.amplitude);
68497 |             pWaveform->time += pWaveform->advance;
68498 |
68499 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68500 |                 pFramesOutF32[iFrame*pWaveform->config.channels + iChannel] = s;
68501 |             }
68502 |         }
68503 |     } else if (pWaveform->config.format == ma_format_s16) {
68504 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;

```



```

68505 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68506 |             ma_int16 s = ma_waveform_sawtooth_s16(pWaveform->time, pWaveform->config.amplitude);
68507 |             pWaveform->time += pWaveform->advance;
68508 |
68509 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68510 |                 pFramesOutS16[iFrame*pWaveform->config.channels + iChannel] = s;
68511 |             }
68512 |         }
68513 |     } else {
68514 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
68515 |             float s = ma_waveform_sawtooth_f32(pWaveform->time, pWaveform->config.amplitude);
68516 |             pWaveform->time += pWaveform->advance;
68517 |
68518 |             for (iChannel = 0; iChannel < pWaveform->config.channels; iChannel += 1) {
68519 |                 ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pWaveform->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
68520 |             }
68521 |         }
68522 |     }
68523 | }
68524 |
68525 | MA_API ma_result ma_waveform_read_pcm_frames(ma_waveform* pWaveform, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
68526 | {
68527 |     if (pFramesRead != NULL) {
68528 |         *pFramesRead = 0;
68529 |     }
68530 |
68531 |     if (frameCount == 0) {
68532 |         return MA_INVALID_ARGS;
68533 |     }
68534 |
68535 |     if (pWaveform == NULL) {
68536 |         return MA_INVALID_ARGS;
68537 |     }
68538 |
68539 |     if (pFramesOut != NULL) {
68540 |         switch (pWaveform->config.type)
68541 |         {
68542 |             case ma_waveform_type_sine:
68543 |             {
68544 |                 ma_waveform_read_pcm_frames__sine(pWaveform, pFramesOut, frameCount);
68545 |             } break;
68546 |
68547 |             case ma_waveform_type_square:
68548 |             {
68549 |                 ma_waveform_read_pcm_frames__square(pWaveform, 0.5, pFramesOut, frameCount);
68550 |             } break;
68551 |
68552 |             case ma_waveform_type_triangle:
68553 |             {
68554 |                 ma_waveform_read_pcm_frames__triangle(pWaveform, pFramesOut, frameCount);
68555 |             } break;
68556 |
68557 |             case ma_waveform_type_sawtooth:
68558 |             {
68559 |                 ma_waveform_read_pcm_frames__sawtooth(pWaveform, pFramesOut, frameCount);
68560 |             } break;
68561 |
68562 |             default: return MA_INVALID_OPERATION; /* Unknown waveform type. */
68563 |         }
68564 |     } else {
68565 |         pWaveform->time += pWaveform->advance * (ma_int64)frameCount; /* Cast to int64 required for
> VC6. Won't affect anything in practice. */
68566 |     }
68567 |
68568 |     if (pFramesRead != NULL) {
68569 |         *pFramesRead = frameCount;
68570 |     }
68571 |
68572 |     return MA_SUCCESS;
68573 | }
68574 |
68575 | MA_API ma_result ma_waveform_seek_to_pcm_frame(ma_waveform* pWaveform, ma_uint64 frameIndex)
68576 | {
68577 |     if (pWaveform == NULL) {
68578 |         return MA_INVALID_ARGS;
68579 |     }
68580 |

```

```

68581 |     pWaveform->time = pWaveform->advance * (ma_int64)frameIndex;    /* Casting for VC6. Won't be an
> issue in practice. */
68582 |
68583 |     return MA_SUCCESS;
68584 | }
68585 |
68586 | MA_API ma_pulsewave_config ma_pulsewave_config_init(ma_format format, ma_uint32 channels, ma_uint32
> sampleRate, double dutyCycle, double amplitude, double frequency)
68587 | {
68588 |     ma_pulsewave_config config;
68589 |
68590 |     MA_ZERO_OBJECT(&config);
68591 |     config.format      = format;
68592 |     config.channels    = channels;
68593 |     config.sampleRate  = sampleRate;
68594 |     config.dutyCycle   = dutyCycle;
68595 |     config.amplitude   = amplitude;
68596 |     config.frequency   = frequency;
68597 |
68598 |     return config;
68599 | }
68600 |
68601 | MA_API ma_result ma_pulsewave_init(const ma_pulsewave_config* pConfig, ma_pulsewave* pWaveform)
68602 | {
68603 |     ma_result result;
68604 |     ma_waveform_config config;
68605 |
68606 |     if (pWaveform == NULL) {
68607 |         return MA_INVALID_ARGS;
68608 |     }
68609 |
68610 |     MA_ZERO_OBJECT(pWaveform);
68611 |
68612 |     config = ma_waveform_config_init(
68613 |         pConfig->format,
68614 |         pConfig->channels,
68615 |         pConfig->sampleRate,
68616 |         ma_waveform_type_square,
68617 |         pConfig->amplitude,
68618 |         pConfig->frequency
68619 |     );
68620 |
68621 |     result = ma_waveform_init(&config, &pWaveform->waveform);
68622 |     ma_pulsewave_set_duty_cycle(pWaveform, pConfig->dutyCycle);
68623 |
68624 |     return result;
68625 | }
68626 |
68627 | MA_API void ma_pulsewave_uninit(ma_pulsewave* pWaveform)
68628 | {
68629 |     if (pWaveform == NULL) {
68630 |         return;
68631 |     }
68632 |
68633 |     ma_waveform_uninit(&pWaveform->waveform);
68634 | }
68635 |
68636 | MA_API ma_result ma_pulsewave_read_pcm_frames(ma_pulsewave* pWaveform, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
68637 | {
68638 |     if (pFramesRead != NULL) {
68639 |         *pFramesRead = 0;
68640 |     }
68641 |
68642 |     if (frameCount == 0) {
68643 |         return MA_INVALID_ARGS;
68644 |     }
68645 |
68646 |     if (pWaveform == NULL) {
68647 |         return MA_INVALID_ARGS;
68648 |     }
68649 |
68650 |     if (pFramesOut != NULL) {
68651 |         ma_waveform_read_pcm_frames__square(&pWaveform->waveform, pWaveform->config.dutyCycle,
> pFramesOut, frameCount);
68652 |     } else {
68653 |         pWaveform->waveform.time += pWaveform->waveform.advance * (ma_int64)frameCount; /* Cast to
> int64 required for VC6. Won't affect anything in practice. */
68654 |     }

```

```
68655 |
68656 |     if (pFramesRead != NULL) {
68657 |         *pFramesRead = frameCount;
68658 |     }
68659 |
68660 |     return MA_SUCCESS;
68661 | }
68662 |
68663 | MA_API ma_result ma_pulsewave_seek_to_pcm_frame(ma_pulsewave* pWaveform, ma_uint64 frameIndex)
68664 | {
68665 |     if (pWaveform == NULL) {
68666 |         return MA_INVALID_ARGS;
68667 |     }
68668 |
68669 |     ma_waveform_seek_to_pcm_frame(&pWaveform->waveform, frameIndex);
68670 |
68671 |     return MA_SUCCESS;
68672 | }
68673 |
68674 | MA_API ma_result ma_pulsewave_set_amplitude(ma_pulsewave* pWaveform, double amplitude)
68675 | {
68676 |     if (pWaveform == NULL) {
68677 |         return MA_INVALID_ARGS;
68678 |     }
68679 |
68680 |     pWaveform->config.amplitude = amplitude;
68681 |     ma_waveform_set_amplitude(&pWaveform->waveform, amplitude);
68682 |
68683 |     return MA_SUCCESS;
68684 | }
68685 |
68686 | MA_API ma_result ma_pulsewave_set_frequency(ma_pulsewave* pWaveform, double frequency)
68687 | {
68688 |     if (pWaveform == NULL) {
68689 |         return MA_INVALID_ARGS;
68690 |     }
68691 |
68692 |     pWaveform->config.frequency = frequency;
68693 |     ma_waveform_set_frequency(&pWaveform->waveform, frequency);
68694 |
68695 |     return MA_SUCCESS;
68696 | }
68697 |
68698 | MA_API ma_result ma_pulsewave_set_sample_rate(ma_pulsewave* pWaveform, ma_uint32 sampleRate)
68699 | {
68700 |     if (pWaveform == NULL) {
68701 |         return MA_INVALID_ARGS;
68702 |     }
68703 |
68704 |     pWaveform->config.sampleRate = sampleRate;
68705 |     ma_waveform_set_sample_rate(&pWaveform->waveform, sampleRate);
68706 |
68707 |     return MA_SUCCESS;
68708 | }
68709 |
68710 | MA_API ma_result ma_pulsewave_set_duty_cycle(ma_pulsewave* pWaveform, double dutyCycle)
68711 | {
68712 |     if (pWaveform == NULL) {
68713 |         return MA_INVALID_ARGS;
68714 |     }
68715 |
68716 |     pWaveform->config.dutyCycle = dutyCycle;
68717 |
68718 |     return MA_SUCCESS;
68719 | }
68720 |
68721 |
68722 |
68723 | MA_API ma_noise_config ma_noise_config_init(ma_format format, ma_uint32 channels, ma_noise_type
> type, ma_int32 seed, double amplitude)
68724 | {
68725 |     ma_noise_config config;
68726 |     MA_ZERO_OBJECT(&config);
68727 |
68728 |     config.format    = format;
68729 |     config.channels  = channels;
68730 |     config.type      = type;
68731 |     config.seed      = seed;
68732 |     config.amplitude = amplitude;
```

```

68733 |
68734 |     if (config.seed == 0) {
68735 |         config.seed = MA_DEFAULT_LCG_SEED;
68736 |     }
68737 |
68738 |     return config;
68739 | }
68740 |
68741 |
68742 | static ma_result ma_noise__data_source_on_read(ma_data_source* pDataSource, void* pFramesOut,
> ma_uint64 frameCount, ma_uint64* pFramesRead)
68743 | {
68744 |     return ma_noise_read_pcm_frames((ma_noise*)pDataSource, pFramesOut, frameCount, pFramesRead);
68745 | }
68746 |
68747 | static ma_result ma_noise__data_source_on_seek(ma_data_source* pDataSource, ma_uint64 frameIndex)
68748 | {
68749 |     /* No-op. Just pretend to be successful. */
68750 |     (void)pDataSource;
68751 |     (void)frameIndex;
68752 |     return MA_SUCCESS;
68753 | }
68754 |
68755 | static ma_result ma_noise__data_source_on_get_data_format(ma_data_source* pDataSource, ma_format*
> pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
68756 | {
68757 |     ma_noise* pNoise = (ma_noise*)pDataSource;
68758 |
68759 |     *pFormat      = pNoise->config.format;
68760 |     *pChannels     = pNoise->config.channels;
68761 |     *pSampleRate = 0; /* There is no notion of sample rate with noise generation. */
68762 |     ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap, channelMapCap,
> pNoise->config.channels);
68763 |
68764 |     return MA_SUCCESS;
68765 | }
68766 |
68767 | static ma_data_source_vtable g_ma_noise_data_source_vtable =
68768 | {
68769 |     ma_noise__data_source_on_read,
68770 |     ma_noise__data_source_on_seek, /* No-op for noise. */
68771 |     ma_noise__data_source_on_get_data_format,
68772 |     NULL, /* onGetCursor. No notion of a cursor for noise. */
68773 |     NULL, /* onGetLength. No notion of a length for noise. */
68774 |     NULL, /* onSetLooping */
68775 |     0
68776 | };
68777 |
68778 |
68779 | #ifndef MA_PINK_NOISE_BIN_SIZE
68780 | #define MA_PINK_NOISE_BIN_SIZE 16
68781 | #endif
68782 |
68783 | typedef struct
68784 | {
68785 |     size_t sizeInBytes;
68786 |     struct
68787 |     {
68788 |         size_t binOffset;
68789 |         size_t accumulationOffset;
68790 |         size_t counterOffset;
68791 |     } pink;
68792 |     struct
68793 |     {
68794 |         size_t accumulationOffset;
68795 |     } brownian;
68796 | } ma_noise_heap_layout;
68797 |
68798 | static ma_result ma_noise_get_heap_layout(const ma_noise_config* pConfig, ma_noise_heap_layout*
> pHeapLayout)
68799 | {
68800 |     MA_ASSERT(pHeapLayout != NULL);
68801 |
68802 |     MA_ZERO_OBJECT(pHeapLayout);
68803 |
68804 |     if (pConfig == NULL) {
68805 |         return MA_INVALID_ARGS;
68806 |     }

```

```

68807 |
68808 |     if (pConfig->channels == 0) {
68809 |         return MA_INVALID_ARGS;
68810 |     }
68811 |
68812 |     pHeapLayout->sizeInBytes = 0;
68813 |
68814 |     /* Pink. */
68815 |     if (pConfig->type == ma_noise_type_pink) {
68816 |         /* bin */
68817 |         pHeapLayout->pink.binOffset = pHeapLayout->sizeInBytes;
68818 |         pHeapLayout->sizeInBytes += sizeof(double) * pConfig->channels;
68819 |         pHeapLayout->sizeInBytes += sizeof(double) * pConfig->channels * MA_PINK_NOISE_BIN_SIZE;
68820 |
68821 |         /* accumulation */
68822 |         pHeapLayout->pink.accumulationOffset = pHeapLayout->sizeInBytes;
68823 |         pHeapLayout->sizeInBytes += sizeof(double) * pConfig->channels;
68824 |
68825 |         /* counter */
68826 |         pHeapLayout->pink.counterOffset = pHeapLayout->sizeInBytes;
68827 |         pHeapLayout->sizeInBytes += sizeof(ma_uint32) * pConfig->channels;
68828 |     }
68829 |
68830 |     /* Brownian. */
68831 |     if (pConfig->type == ma_noise_type_brownian) {
68832 |         /* accumulation */
68833 |         pHeapLayout->brownian.accumulationOffset = pHeapLayout->sizeInBytes;
68834 |         pHeapLayout->sizeInBytes += sizeof(double) * pConfig->channels;
68835 |     }
68836 |
68837 |     /* Make sure allocation size is aligned. */
68838 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
68839 |
68840 |     return MA_SUCCESS;
68841 | }
68842 |
68843 | MA_API ma_result ma_noise_get_heap_size(const ma_noise_config* pConfig, size_t* pHeapSizeInBytes)
68844 | {
68845 |     ma_result result;
68846 |     ma_noise_heap_layout heapLayout;
68847 |
68848 |     if (pHeapSizeInBytes == NULL) {
68849 |         return MA_INVALID_ARGS;
68850 |     }
68851 |
68852 |     *pHeapSizeInBytes = 0;
68853 |
68854 |     result = ma_noise_get_heap_layout(pConfig, &heapLayout);
68855 |     if (result != MA_SUCCESS) {
68856 |         return result;
68857 |     }
68858 |
68859 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
68860 |
68861 |     return MA_SUCCESS;
68862 | }
68863 |
68864 | MA_API ma_result ma_noise_init_preallocated(const ma_noise_config* pConfig, void* pHeap, ma_noise*
68865 | > pNoise)
68866 | {
68867 |     ma_result result;
68868 |     ma_noise_heap_layout heapLayout;
68869 |     ma_data_source_config dataSourceConfig;
68870 |     ma_uint32 iChannel;
68871 |
68872 |     if (pNoise == NULL) {
68873 |         return MA_INVALID_ARGS;
68874 |     }
68875 |
68876 |     MA_ZERO_OBJECT(pNoise);
68877 |
68878 |     result = ma_noise_get_heap_layout(pConfig, &heapLayout);
68879 |     if (result != MA_SUCCESS) {
68880 |         return result;
68881 |     }
68882 |
68883 |     pNoise->pHeap = pHeap;
68884 |     MA_ZERO_MEMORY(pNoise->pHeap, heapLayout.sizeInBytes);

```

```

68885 |     dataSourceConfig = ma_data_source_config_init();
68886 |     dataSourceConfig.vtable = &g_ma_noise_data_source_vtable;
68887 |
68888 |     result = ma_data_source_init(&dataSourceConfig, &pNoise->ds);
68889 |     if (result != MA_SUCCESS) {
68890 |         return result;
68891 |     }
68892 |
68893 |     pNoise->config = *pConfig;
68894 |     ma_lcg_seed(&pNoise->lcg, pConfig->seed);
68895 |
68896 |     if (pNoise->config.type == ma_noise_type_pink) {
68897 |         pNoise->state.pink.bin = (double**)ma_offset_ptr(pHeap,
> heapLayout.pink.binOffset);
68898 |         pNoise->state.pink.accumulation = (double*)ma_offset_ptr(pHeap,
> heapLayout.pink.accumulationOffset);
68899 |         pNoise->state.pink.counter = (ma_uint32*)ma_offset_ptr(pHeap,
> heapLayout.pink.counterOffset);
68900 |
68901 |         for (iChannel = 0; iChannel < pConfig->channels; iChannel += 1) {
68902 |             pNoise->state.pink.bin[iChannel] = (double*)ma_offset_ptr(pHeap,
> heapLayout.pink.binOffset + (sizeof(double*) * pConfig->channels) + (sizeof(double) *
> MA_PINK_NOISE_BIN_SIZE * iChannel));
68903 |             pNoise->state.pink.accumulation[iChannel] = 0;
68904 |             pNoise->state.pink.counter[iChannel] = 1;
68905 |         }
68906 |     }
68907 |
68908 |     if (pNoise->config.type == ma_noise_type_brownian) {
68909 |         pNoise->state.brownian.accumulation = (double*)ma_offset_ptr(pHeap,
> heapLayout.brownian.accumulationOffset);
68910 |
68911 |         for (iChannel = 0; iChannel < pConfig->channels; iChannel += 1) {
68912 |             pNoise->state.brownian.accumulation[iChannel] = 0;
68913 |         }
68914 |     }
68915 |
68916 |     return MA_SUCCESS;
68917 | }
68918 |
68919 | MA_API ma_result ma_noise_init(const ma_noise_config* pConfig, const ma_allocation_callbacks*
> pAllocationCallbacks, ma_noise* pNoise)
68920 | {
68921 |     ma_result result;
68922 |     size_t heapSizeInBytes;
68923 |     void* pHeap;
68924 |
68925 |     result = ma_noise_get_heap_size(pConfig, &heapSizeInBytes);
68926 |     if (result != MA_SUCCESS) {
68927 |         return result;
68928 |     }
68929 |
68930 |     if (heapSizeInBytes > 0) {
68931 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
68932 |         if (pHeap == NULL) {
68933 |             return MA_OUT_OF_MEMORY;
68934 |         }
68935 |     } else {
68936 |         pHeap = NULL;
68937 |     }
68938 |
68939 |     result = ma_noise_init_preallocated(pConfig, pHeap, pNoise);
68940 |     if (result != MA_SUCCESS) {
68941 |         ma_free(pHeap, pAllocationCallbacks);
68942 |         return result;
68943 |     }
68944 |
68945 |     pNoise->ownsHeap = MA_TRUE;
68946 |     return MA_SUCCESS;
68947 | }
68948 |
68949 | MA_API void ma_noise_uninit(ma_noise* pNoise, const ma_allocation_callbacks* pAllocationCallbacks)
68950 | {
68951 |     if (pNoise == NULL) {
68952 |         return;
68953 |     }
68954 |
68955 |     ma_data_source_uninit(&pNoise->ds);
68956 |

```

```

68957 |     if (pNoise->_ownsHeap) {
68958 |         ma_free(pNoise->_pHeap, pAllocationCallbacks);
68959 |     }
68960 | }
68961 |
68962 | MA_API ma_result ma_noise_set_amplitude(ma_noise* pNoise, double amplitude)
68963 | {
68964 |     if (pNoise == NULL) {
68965 |         return MA_INVALID_ARGS;
68966 |     }
68967 |
68968 |     pNoise->config.amplitude = amplitude;
68969 |     return MA_SUCCESS;
68970 | }
68971 |
68972 | MA_API ma_result ma_noise_set_seed(ma_noise* pNoise, ma_int32 seed)
68973 | {
68974 |     if (pNoise == NULL) {
68975 |         return MA_INVALID_ARGS;
68976 |     }
68977 |
68978 |     pNoise->lcg.state = seed;
68979 |     return MA_SUCCESS;
68980 | }
68981 |
68982 |
68983 | MA_API ma_result ma_noise_set_type(ma_noise* pNoise, ma_noise_type type)
68984 | {
68985 |     if (pNoise == NULL) {
68986 |         return MA_INVALID_ARGS;
68987 |     }
68988 |
68989 |     /*
68990 |     This function should never have been implemented in the first place. Changing the type
68991 |     > dynamically is not
68992 |     supported. Instead you need to uninitialized and reinitialize a fresh `ma_noise` object. This
68993 |     > function
68994 |     will be removed in version 0.12.
68995 |     */
68996 |     MA_ASSERT(MA_FALSE);
68997 |     (void)type;
68998 |
68999 |     return MA_INVALID_OPERATION;
69000 | }
69001 |
69002 | static MA_INLINE float ma_noise_f32_white(ma_noise* pNoise)
69003 | {
69004 |     return (float)(ma_lcg_rand_f64(&pNoise->lcg) * pNoise->config.amplitude);
69005 | }
69006 |
69007 | static MA_INLINE ma_int16 ma_noise_s16_white(ma_noise* pNoise)
69008 | {
69009 |     return ma_pcm_sample_f32_to_s16(ma_noise_f32_white(pNoise));
69010 | }
69011 |
69012 | static MA_INLINE ma_uint64 ma_noise_read_pcm_frames__white(ma_noise* pNoise, void* pFramesOut,
69013 | > ma_uint64 frameCount)
69014 | {
69015 |     ma_uint64 iFrame;
69016 |     ma_uint32 iChannel;
69017 |     const ma_uint32 channels = pNoise->config.channels;
69018 |     MA_ASSUME(channels > 0);
69019 |
69020 |     if (pNoise->config.format == ma_format_f32) {
69021 |         float* pFramesOutF32 = (float*)pFramesOut;
69022 |         if (pNoise->config.duplicateChannels) {
69023 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69024 |                 float s = ma_noise_f32_white(pNoise);
69025 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69026 |                     pFramesOutF32[iFrame*channels + iChannel] = s;
69027 |                 }
69028 |             }
69029 |         } else {
69030 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69031 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69032 |                     pFramesOutF32[iFrame*channels + iChannel] = ma_noise_f32_white(pNoise);
69033 |                 }
69034 |             }
69035 |         }
69036 |     }

```



```

69033 |     } else if (pNoise->config.format == ma_format_s16) {
69034 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
69035 |         if (pNoise->config.duplicateChannels) {
69036 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69037 |                 ma_int16 s = ma_noise_s16_white(pNoise);
69038 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69039 |                     pFramesOutS16[iFrame*channels + iChannel] = s;
69040 |                 }
69041 |             }
69042 |         } else {
69043 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69044 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69045 |                     pFramesOutS16[iFrame*channels + iChannel] = ma_noise_s16_white(pNoise);
69046 |                 }
69047 |             }
69048 |         }
69049 |     } else {
69050 |         const ma_uint32 bps = ma_get_bytes_per_sample(pNoise->config.format);
69051 |         const ma_uint32 bpf = bps * channels;
69052 |
69053 |         if (pNoise->config.duplicateChannels) {
69054 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69055 |                 float s = ma_noise_f32_white(pNoise);
69056 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69057 |                     ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pNoise->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
69058 |                 }
69059 |             }
69060 |         } else {
69061 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69062 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69063 |                     float s = ma_noise_f32_white(pNoise);
69064 |                     ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pNoise->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
69065 |                 }
69066 |             }
69067 |         }
69068 |     }
69069 |
69070 |     return frameCount;
69071 | }
69072 |
69073 |
69074 | static MA_INLINE unsigned int ma_tzcnt32(unsigned int x)
69075 | {
69076 |     unsigned int n;
69077 |
69078 |     /* Special case for odd numbers since they should happen about half the time. */
69079 |     if (x & 0x1) {
69080 |         return 0;
69081 |     }
69082 |
69083 |     if (x == 0) {
69084 |         return sizeof(x) << 3;
69085 |     }
69086 |
69087 |     n = 1;
69088 |     if ((x & 0x0000FFFF) == 0) { x >>= 16; n += 16; }
69089 |     if ((x & 0x000000FF) == 0) { x >>= 8; n += 8; }
69090 |     if ((x & 0x0000000F) == 0) { x >>= 4; n += 4; }
69091 |     if ((x & 0x00000003) == 0) { x >>= 2; n += 2; }
69092 |     n -= x & 0x00000001;
69093 |
69094 |     return n;
69095 | }
69096 |
69097 | /*
69098 | Pink noise generation based on Tonic (public domain) with modifications.
> https://github.com/TonicAudio/Tonic/blob/master/src/Tonic/Noise.h
69099 |
69100 | This is basically _the_ reference for pink noise from what I've found:
> http://www.firstpr.com.au/dsp/pink-noise/
69101 | */
69102 | static MA_INLINE float ma_noise_f32_pink(ma_noise* pNoise, ma_uint32 iChannel)
69103 | {
69104 |     double result;
69105 |     double binPrev;
69106 |     double binNext;
69107 |     unsigned int ibin;

```

```

69108 |
69109 |     ibin = ma_tzcnt32(pNoise->state.pink.counter[iChannel]) & (MA_PINK_NOISE_BIN_SIZE - 1);
69110 |
69111 |     binPrev = pNoise->state.pink.bin[iChannel][ibin];
69112 |     binNext = ma_lcg_rand_f64(&pNoise->lcg);
69113 |     pNoise->state.pink.bin[iChannel][ibin] = binNext;
69114 |
69115 |     pNoise->state.pink.accumulation[iChannel] += (binNext - binPrev);
69116 |     pNoise->state.pink.counter[iChannel]     += 1;
69117 |
69118 |     result = (ma_lcg_rand_f64(&pNoise->lcg) + pNoise->state.pink.accumulation[iChannel]);
69119 |     result /= 10;
69120 |
69121 |     return (float)(result * pNoise->config.amplitude);
69122 | }
69123 |
69124 | static MA_INLINE ma_int16 ma_noise_s16_pink(ma_noise* pNoise, ma_uint32 iChannel)
69125 | {
69126 |     return ma_pcm_sample_f32_to_s16(ma_noise_f32_pink(pNoise, iChannel));
69127 | }
69128 |
69129 | static MA_INLINE ma_uint64 ma_noise_read_pcm_frames__pink(ma_noise* pNoise, void* pFramesOut,
> ma_uint64 frameCount)
69130 | {
69131 |     ma_uint64 iFrame;
69132 |     ma_uint32 iChannel;
69133 |     const ma_uint32 channels = pNoise->config.channels;
69134 |     MA_ASSUME(channels > 0);
69135 |
69136 |     if (pNoise->config.format == ma_format_f32) {
69137 |         float* pFramesOutF32 = (float*)pFramesOut;
69138 |         if (pNoise->config.duplicateChannels) {
69139 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69140 |                 float s = ma_noise_f32_pink(pNoise, 0);
69141 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69142 |                     pFramesOutF32[iFrame*channels + iChannel] = s;
69143 |                 }
69144 |             }
69145 |         } else {
69146 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69147 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69148 |                     pFramesOutF32[iFrame*channels + iChannel] = ma_noise_f32_pink(pNoise, iChannel);
69149 |                 }
69150 |             }
69151 |         }
69152 |     } else if (pNoise->config.format == ma_format_s16) {
69153 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
69154 |         if (pNoise->config.duplicateChannels) {
69155 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69156 |                 ma_int16 s = ma_noise_s16_pink(pNoise, 0);
69157 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69158 |                     pFramesOutS16[iFrame*channels + iChannel] = s;
69159 |                 }
69160 |             }
69161 |         } else {
69162 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69163 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69164 |                     pFramesOutS16[iFrame*channels + iChannel] = ma_noise_s16_pink(pNoise, iChannel);
69165 |                 }
69166 |             }
69167 |         }
69168 |     } else {
69169 |         const ma_uint32 bps = ma_get_bytes_per_sample(pNoise->config.format);
69170 |         const ma_uint32 bpf = bps * channels;
69171 |
69172 |         if (pNoise->config.duplicateChannels) {
69173 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69174 |                 float s = ma_noise_f32_pink(pNoise, 0);
69175 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69176 |                     ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pNoise->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
69177 |                 }
69178 |             }
69179 |         } else {
69180 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69181 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69182 |                     float s = ma_noise_f32_pink(pNoise, iChannel);
69183 |                     ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pNoise->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);

```

```

69184 |     }
69185 | }
69186 | }
69187 | }
69188 |
69189 |     return frameCount;
69190 | }
69191 |
69192 |
69193 | static MA_INLINE float ma_noise_f32_brownian(ma_noise* pNoise, ma_uint32 iChannel)
69194 | {
69195 |     double result;
69196 |
69197 |     result = (ma_lcg_rand_f64(&pNoise->lcg) + pNoise->state.brownian.accumulation[iChannel]);
69198 |     result /= 1.005; /* Don't escape the -1..1 range on average. */
69199 |
69200 |     pNoise->state.brownian.accumulation[iChannel] = result;
69201 |     result /= 20;
69202 |
69203 |     return (float)(result * pNoise->config.amplitude);
69204 | }
69205 |
69206 | static MA_INLINE ma_int16 ma_noise_s16_brownian(ma_noise* pNoise, ma_uint32 iChannel)
69207 | {
69208 |     return ma_pcm_sample_f32_to_s16(ma_noise_f32_brownian(pNoise, iChannel));
69209 | }
69210 |
69211 | static MA_INLINE ma_uint64 ma_noise_read_pcm_frames__brownian(ma_noise* pNoise, void* pFramesOut,
> ma_uint64 frameCount)
69212 | {
69213 |     ma_uint64 iFrame;
69214 |     ma_uint32 iChannel;
69215 |     const ma_uint32 channels = pNoise->config.channels;
69216 |     MA_ASSUME(channels > 0);
69217 |
69218 |     if (pNoise->config.format == ma_format_f32) {
69219 |         float* pFramesOutF32 = (float*)pFramesOut;
69220 |         if (pNoise->config.duplicateChannels) {
69221 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69222 |                 float s = ma_noise_f32_brownian(pNoise, 0);
69223 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69224 |                     pFramesOutF32[iFrame*channels + iChannel] = s;
69225 |                 }
69226 |             }
69227 |         } else {
69228 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69229 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69230 |                     pFramesOutF32[iFrame*channels + iChannel] = ma_noise_f32_brownian(pNoise,
> iChannel);
69231 |                 }
69232 |             }
69233 |         }
69234 |     } else if (pNoise->config.format == ma_format_s16) {
69235 |         ma_int16* pFramesOutS16 = (ma_int16*)pFramesOut;
69236 |         if (pNoise->config.duplicateChannels) {
69237 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69238 |                 ma_int16 s = ma_noise_s16_brownian(pNoise, 0);
69239 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69240 |                     pFramesOutS16[iFrame*channels + iChannel] = s;
69241 |                 }
69242 |             }
69243 |         } else {
69244 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69245 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69246 |                     pFramesOutS16[iFrame*channels + iChannel] = ma_noise_s16_brownian(pNoise,
> iChannel);
69247 |                 }
69248 |             }
69249 |         }
69250 |     } else {
69251 |         const ma_uint32 bps = ma_get_bytes_per_sample(pNoise->config.format);
69252 |         const ma_uint32 bpf = bps * channels;
69253 |
69254 |         if (pNoise->config.duplicateChannels) {
69255 |             for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69256 |                 float s = ma_noise_f32_brownian(pNoise, 0);
69257 |                 for (iChannel = 0; iChannel < channels; iChannel += 1) {
69258 |                     ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pNoise->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);

```

```

69259 |         }
69260 |     }
69261 |     } else {
69262 |         for (iFrame = 0; iFrame < frameCount; iFrame += 1) {
69263 |             for (iChannel = 0; iChannel < channels; iChannel += 1) {
69264 |                 float s = ma_noise_f32_brownian(pNoise, iChannel);
69265 |                 ma_pcm_convert(ma_offset_ptr(pFramesOut, iFrame*bpf + iChannel*bps),
> pNoise->config.format, &s, ma_format_f32, 1, ma_dither_mode_none);
69266 |             }
69267 |         }
69268 |     }
69269 | }
69270 |
69271 |     return frameCount;
69272 | }
69273 |
69274 | MA_API ma_result ma_noise_read_pcm_frames(ma_noise* pNoise, void* pFramesOut, ma_uint64 frameCount,
> ma_uint64* pFramesRead)
69275 | {
69276 |     ma_uint64 framesRead = 0;
69277 |
69278 |     if (pFramesRead != NULL) {
69279 |         *pFramesRead = 0;
69280 |     }
69281 |
69282 |     if (frameCount == 0) {
69283 |         return MA_INVALID_ARGS;
69284 |     }
69285 |
69286 |     if (pNoise == NULL) {
69287 |         return MA_INVALID_ARGS;
69288 |     }
69289 |
69290 |     /* The output buffer is allowed to be NULL. Since we aren't tracking cursors or anything we can
> just do nothing and pretend to be successful. */
69291 |     if (pFramesOut == NULL) {
69292 |         framesRead = frameCount;
69293 |     } else {
69294 |         switch (pNoise->config.type) {
69295 |             case ma_noise_type_white:     framesRead = ma_noise_read_pcm_frames__white    (pNoise,
> pFramesOut, frameCount); break;
69296 |             case ma_noise_type_pink:      framesRead = ma_noise_read_pcm_frames__pink    (pNoise,
> pFramesOut, frameCount); break;
69297 |             case ma_noise_type_brownian:  framesRead = ma_noise_read_pcm_frames__brownian(pNoise,
> pFramesOut, frameCount); break;
69298 |             default: return MA_INVALID_OPERATION; /* Unknown noise type. */
69299 |         }
69300 |     }
69301 |
69302 |     if (pFramesRead != NULL) {
69303 |         *pFramesRead = framesRead;
69304 |     }
69305 |
69306 |     return MA_SUCCESS;
69307 | }
69308 | #endif /* MA_NO_GENERATION */
69309 |
69310 |
69311 |
69312 | #ifndef MA_NO_RESOURCE_MANAGER
69313 | #ifndef MA_RESOURCE_MANAGER_PAGE_SIZE_IN_MILLISECONDS
69314 | #define MA_RESOURCE_MANAGER_PAGE_SIZE_IN_MILLISECONDS    1000
69315 | #endif
69316 |
69317 | #ifndef MA_JOB_TYPE_RESOURCE_MANAGER_QUEUE_CAPACITY
69318 | #define MA_JOB_TYPE_RESOURCE_MANAGER_QUEUE_CAPACITY      1024
69319 | #endif
69320 |
69321 | MA_API ma_resource_manager_pipeline_notifications
> ma_resource_manager_pipeline_notifications_init(void)
69322 | {
69323 |     ma_resource_manager_pipeline_notifications notifications;
69324 |
69325 |     MA_ZERO_OBJECT(&notifications);
69326 |
69327 |     return notifications;
69328 | }
69329 |
69330 | static void ma_resource_manager_pipeline_notifications_signal_all_notifications(const

```

```

    > ma_resource_manager_pipeline_notifications* pPipelineNotifications)
69331 | {
69332 |     if (pPipelineNotifications == NULL) {
69333 |         return;
69334 |     }
69335 |
69336 |     if (pPipelineNotifications->init.pNotification) {
    > ma_async_notification_signal(pPipelineNotifications->init.pNotification); }
69337 |     if (pPipelineNotifications->done.pNotification) {
    > ma_async_notification_signal(pPipelineNotifications->done.pNotification); }
69338 | }
69339 |
69340 | static void ma_resource_manager_pipeline_notifications_acquire_all_fences(const
    > ma_resource_manager_pipeline_notifications* pPipelineNotifications)
69341 | {
69342 |     if (pPipelineNotifications == NULL) {
69343 |         return;
69344 |     }
69345 |
69346 |     if (pPipelineNotifications->init.pFence != NULL) {
    > ma_fence_acquire(pPipelineNotifications->init.pFence); }
69347 |     if (pPipelineNotifications->done.pFence != NULL) {
    > ma_fence_acquire(pPipelineNotifications->done.pFence); }
69348 | }
69349 |
69350 | static void ma_resource_manager_pipeline_notifications_release_all_fences(const
    > ma_resource_manager_pipeline_notifications* pPipelineNotifications)
69351 | {
69352 |     if (pPipelineNotifications == NULL) {
69353 |         return;
69354 |     }
69355 |
69356 |     if (pPipelineNotifications->init.pFence != NULL) {
    > ma_fence_release(pPipelineNotifications->init.pFence); }
69357 |     if (pPipelineNotifications->done.pFence != NULL) {
    > ma_fence_release(pPipelineNotifications->done.pFence); }
69358 | }
69359 |
69360 |
69361 |
69362 | #ifndef MA_DEFAULT_HASH_SEED
69363 | #define MA_DEFAULT_HASH_SEED    42
69364 | #endif
69365 |
69366 | /* MurmurHash3. Based on code from https://github.com/PeterScott/murmur3/blob/master/murmur3.c
    > (public domain). */
69367 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
    > 6)))
69368 |     #pragma GCC diagnostic push
69369 |     #if __GNUC__ >= 7
69370 |     #pragma GCC diagnostic ignored "-Wimplicit-fallthrough"
69371 |     #endif
69372 | #endif
69373 |
69374 | static MA_INLINE ma_uint32 ma_rotl32(ma_uint32 x, ma_int8 r)
69375 | {
69376 |     return (x << r) | (x >> (32 - r));
69377 | }
69378 |
69379 | static MA_INLINE ma_uint32 ma_hash_getblock(const ma_uint32* blocks, int i)
69380 | {
69381 |     ma_uint32 block;
69382 |
69383 |     /* Try silencing a sanitization warning about unaligned access by doing a memcpy() instead of
    > assignment. */
69384 |     MA_COPY_MEMORY(&block, ma_offset_ptr(blocks, i * (int) sizeof(block)), sizeof(block));
69385 |
69386 |     if (ma_is_little_endian()) {
69387 |         return block;
69388 |     } else {
69389 |         return ma_swap_endian_uint32(block);
69390 |     }
69391 | }
69392 |
69393 | static MA_INLINE ma_uint32 ma_hash_fm32(ma_uint32 h)
69394 | {
69395 |     h ^= h >> 16;
69396 |     h *= 0x85ebca6b;
69397 |     h ^= h >> 13;

```

```

69398 |     h *= 0xc2b2ae35;
69399 |     h ^= h >> 16;
69400 |
69401 |     return h;
69402 | }
69403 |
69404 | static ma_uint32 ma_hash_32(const void* key, int len, ma_uint32 seed)
69405 | {
69406 |     const ma_uint8* data = (const ma_uint8*)key;
69407 |     const ma_uint32* blocks;
69408 |     const ma_uint8* tail;
69409 |     const int nblocks = len / 4;
69410 |     ma_uint32 h1 = seed;
69411 |     ma_uint32 c1 = 0xcc9e2d51;
69412 |     ma_uint32 c2 = 0x1b873593;
69413 |     ma_uint32 k1;
69414 |     int i;
69415 |
69416 |     blocks = (const ma_uint32*)(data + nblocks*4);
69417 |
69418 |     for(i = -nblocks; i; i++) {
69419 |         k1 = ma_hash_getblock(blocks,i);
69420 |
69421 |         k1 *= c1;
69422 |         k1 = ma_rotl32(k1, 15);
69423 |         k1 *= c2;
69424 |
69425 |         h1 ^= k1;
69426 |         h1 = ma_rotl32(h1, 13);
69427 |         h1 = h1*5 + 0xe6546b64;
69428 |     }
69429 |
69430 |
69431 |     tail = (const ma_uint8*)(data + nblocks*4);
69432 |
69433 |     k1 = 0;
69434 |     switch(len & 3) {
69435 |         case 3: k1 ^= tail[2] << 16;
69436 |         case 2: k1 ^= tail[1] << 8;
69437 |         case 1: k1 ^= tail[0];
69438 |             k1 *= c1; k1 = ma_rotl32(k1, 15); k1 *= c2; h1 ^= k1;
69439 |     };
69440 |
69441 |
69442 |     h1 ^= len;
69443 |     h1 = ma_hash_fm32(h1);
69444 |
69445 |     return h1;
69446 | }
69447 |
69448 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
> 6)))
69449 |     #pragma GCC diagnostic push
69450 | #endif
69451 | /* End MurmurHash3 */
69452 |
69453 | static ma_uint32 ma_hash_string_32(const char* str)
69454 | {
69455 |     return ma_hash_32(str, (int)strlen(str), MA_DEFAULT_HASH_SEED);
69456 | }
69457 |
69458 | static ma_uint32 ma_hash_string_w_32(const wchar_t* str)
69459 | {
69460 |     return ma_hash_32(str, (int)wcslen(str) * sizeof(*str), MA_DEFAULT_HASH_SEED);
69461 | }
69462 |
69463 |
69464 |
69465 | /*
69466 | Basic BST Functions
69467 | */
69468 |
69469 | static ma_result ma_resource_manager_data_buffer_node_search(ma_resource_manager* pResourceManager,
> ma_uint32 hashedName32, ma_resource_manager_data_buffer_node** ppDataBufferNode)
69470 | {
69471 |     ma_resource_manager_data_buffer_node* pCurrentNode;
69472 |
69473 |     MA_ASSERT(pResourceManager != NULL);
69474 |     MA_ASSERT(ppDataBufferNode != NULL);

```

```

69475 |
69476 |     pCurrentNode = pResourceManager->pRootDataBufferNode;
69477 |     while (pCurrentNode != NULL) {
69478 |         if (hashedName32 == pCurrentNode->hashedName32) {
69479 |             break; /* Found. */
69480 |         } else if (hashedName32 < pCurrentNode->hashedName32) {
69481 |             pCurrentNode = pCurrentNode->pChildLo;
69482 |         } else {
69483 |             pCurrentNode = pCurrentNode->pChildHi;
69484 |         }
69485 |     }
69486 |
69487 |     *ppDataBufferNode = pCurrentNode;
69488 |
69489 |     if (pCurrentNode == NULL) {
69490 |         return MA_DOES_NOT_EXIST;
69491 |     } else {
69492 |         return MA_SUCCESS;
69493 |     }
69494 | }
69495 |
69496 | static ma_result ma_resource_manager_data_buffer_node_insert_point(ma_resource_manager*
> pResourceManager, ma_uint32 hashedName32, ma_resource_manager_data_buffer_node** ppInsertPoint)
69497 | {
69498 |     ma_result result = MA_SUCCESS;
69499 |     ma_resource_manager_data_buffer_node* pCurrentNode;
69500 |
69501 |     MA_ASSERT(pResourceManager != NULL);
69502 |     MA_ASSERT(ppInsertPoint != NULL);
69503 |
69504 |     *ppInsertPoint = NULL;
69505 |
69506 |     if (pResourceManager->pRootDataBufferNode == NULL) {
69507 |         return MA_SUCCESS; /* No items. */
69508 |     }
69509 |
69510 |     /* We need to find the node that will become the parent of the new node. If a node is found that
> already has the same hashed name we need to return MA_ALREADY_EXISTS. */
69511 |     pCurrentNode = pResourceManager->pRootDataBufferNode;
69512 |     while (pCurrentNode != NULL) {
69513 |         if (hashedName32 == pCurrentNode->hashedName32) {
69514 |             result = MA_ALREADY_EXISTS;
69515 |             break;
69516 |         } else {
69517 |             if (hashedName32 < pCurrentNode->hashedName32) {
69518 |                 if (pCurrentNode->pChildLo == NULL) {
69519 |                     result = MA_SUCCESS;
69520 |                     break;
69521 |                 } else {
69522 |                     pCurrentNode = pCurrentNode->pChildLo;
69523 |                 }
69524 |             } else {
69525 |                 if (pCurrentNode->pChildHi == NULL) {
69526 |                     result = MA_SUCCESS;
69527 |                     break;
69528 |                 } else {
69529 |                     pCurrentNode = pCurrentNode->pChildHi;
69530 |                 }
69531 |             }
69532 |         }
69533 |     }
69534 |
69535 |     *ppInsertPoint = pCurrentNode;
69536 |     return result;
69537 | }
69538 |
69539 | static ma_result ma_resource_manager_data_buffer_node_insert_at(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode,
> ma_resource_manager_data_buffer_node* pInsertPoint)
69540 | {
69541 |     MA_ASSERT(pResourceManager != NULL);
69542 |     MA_ASSERT(pDataBufferNode != NULL);
69543 |
69544 |     /* The key must have been set before calling this function. */
69545 |     MA_ASSERT(pDataBufferNode->hashedName32 != 0);
69546 |
69547 |     if (pInsertPoint == NULL) {
69548 |         /* It's the first node. */
69549 |         pResourceManager->pRootDataBufferNode = pDataBufferNode;

```



```

69550 |     } else {
69551 |         /* It's not the first node. It needs to be inserted. */
69552 |         if (pDataBufferNode->hashedName32 < pInsertPoint->hashedName32) {
69553 |             MA_ASSERT(pInsertPoint->pChildLo == NULL);
69554 |             pInsertPoint->pChildLo = pDataBufferNode;
69555 |         } else {
69556 |             MA_ASSERT(pInsertPoint->pChildHi == NULL);
69557 |             pInsertPoint->pChildHi = pDataBufferNode;
69558 |         }
69559 |     }
69560 |
69561 |     pDataBufferNode->pParent = pInsertPoint;
69562 |
69563 |     return MA_SUCCESS;
69564 | }
69565 |
69566 | #if 0 /* Unused for now. */
69567 | static ma_result ma_resource_manager_data_buffer_node_insert(ma_resource_manager* pResourceManager,
69568 | > ma_resource_manager_data_buffer_node* pDataBufferNode)
69569 | {
69570 |     ma_result result;
69571 |     ma_resource_manager_data_buffer_node* pInsertPoint;
69572 |
69573 |     MA_ASSERT(pResourceManager != NULL);
69574 |     MA_ASSERT(pDataBufferNode != NULL);
69575 |
69576 |     result = ma_resource_manager_data_buffer_node_insert_point(pResourceManager,
69577 | > pDataBufferNode->hashedName32, &pInsertPoint);
69578 |     if (result != MA_SUCCESS) {
69579 |         return MA_INVALID_ARGS;
69580 |     }
69581 |
69582 |     return ma_resource_manager_data_buffer_node_insert_at(pResourceManager, pDataBufferNode,
69583 | > pInsertPoint);
69584 | }
69585 | #endif
69586 |
69587 | static MA_INLINE ma_resource_manager_data_buffer_node*
69588 | > ma_resource_manager_data_buffer_node_find_min(ma_resource_manager_data_buffer_node* pDataBufferNode)
69589 | {
69590 |     ma_resource_manager_data_buffer_node* pCurrentNode;
69591 |
69592 |     MA_ASSERT(pDataBufferNode != NULL);
69593 |
69594 |     pCurrentNode = pDataBufferNode;
69595 |     while (pCurrentNode->pChildLo != NULL) {
69596 |         pCurrentNode = pCurrentNode->pChildLo;
69597 |     }
69598 |
69599 |     return pCurrentNode;
69600 | }
69601 |
69602 | static MA_INLINE ma_resource_manager_data_buffer_node*
69603 | > ma_resource_manager_data_buffer_node_find_max(ma_resource_manager_data_buffer_node* pDataBufferNode)
69604 | {
69605 |     ma_resource_manager_data_buffer_node* pCurrentNode;
69606 |
69607 |     MA_ASSERT(pDataBufferNode != NULL);
69608 |
69609 |     pCurrentNode = pDataBufferNode;
69610 |     while (pCurrentNode->pChildHi != NULL) {
69611 |         pCurrentNode = pCurrentNode->pChildHi;
69612 |     }
69613 |
69614 |     return pCurrentNode;
69615 | }
69616 |
69617 | static MA_INLINE ma_resource_manager_data_buffer_node*
69618 | > ma_resource_manager_data_buffer_node_find_inorder_successor(ma_resource_manager_data_buffer_node*
69619 | > pDataBufferNode)
69620 | {
69621 |     MA_ASSERT(pDataBufferNode != NULL);
69622 |     MA_ASSERT(pDataBufferNode->pChildHi != NULL);
69623 |
69624 |     return ma_resource_manager_data_buffer_node_find_min(pDataBufferNode->pChildHi);
69625 | }
69626 |
69627 | #if 0 /* Currently unused, but might make use of this later. */
69628 | static MA_INLINE ma_resource_manager_data_buffer_node*

```

```

> ma_resource_manager_data_buffer_node_find_inorder_predecessor(ma_resource_manager_data_buffer_node*
> pDataBufferNode)
69622 | {
69623 |     MA_ASSERT(pDataBufferNode != NULL);
69624 |     MA_ASSERT(pDataBufferNode->pChildLo != NULL);
69625 |
69626 |     return ma_resource_manager_data_buffer_node_find_max(pDataBufferNode->pChildLo);
69627 | }
69628 | #endif
69629 |
69630 | static ma_result ma_resource_manager_data_buffer_node_remove(ma_resource_manager* pResourceManager,
> ma_resource_manager_data_buffer_node* pDataBufferNode)
69631 | {
69632 |     MA_ASSERT(pResourceManager != NULL);
69633 |     MA_ASSERT(pDataBufferNode != NULL);
69634 |
69635 |     if (pDataBufferNode->pChildLo == NULL) {
69636 |         if (pDataBufferNode->pChildHi == NULL) {
69637 |             /* Simple case - deleting a buffer with no children. */
69638 |             if (pDataBufferNode->pParent == NULL) {
69639 |                 MA_ASSERT(pResourceManager->pRootDataBufferNode == pDataBufferNode); /* There is
> only a single buffer in the tree which should be equal to the root node. */
69640 |                 pResourceManager->pRootDataBufferNode = NULL;
69641 |             } else {
69642 |                 if (pDataBufferNode->pParent->pChildLo == pDataBufferNode) {
69643 |                     pDataBufferNode->pParent->pChildLo = NULL;
69644 |                 } else {
69645 |                     pDataBufferNode->pParent->pChildHi = NULL;
69646 |                 }
69647 |             }
69648 |         } else {
69649 |             /* Node has one child - pChildHi != NULL. */
69650 |             pDataBufferNode->pChildHi->pParent = pDataBufferNode->pParent;
69651 |
69652 |             if (pDataBufferNode->pParent == NULL) {
69653 |                 MA_ASSERT(pResourceManager->pRootDataBufferNode == pDataBufferNode);
69654 |                 pResourceManager->pRootDataBufferNode = pDataBufferNode->pChildHi;
69655 |             } else {
69656 |                 if (pDataBufferNode->pParent->pChildLo == pDataBufferNode) {
69657 |                     pDataBufferNode->pParent->pChildLo = pDataBufferNode->pChildHi;
69658 |                 } else {
69659 |                     pDataBufferNode->pParent->pChildHi = pDataBufferNode->pChildHi;
69660 |                 }
69661 |             }
69662 |         }
69663 |     } else {
69664 |         if (pDataBufferNode->pChildHi == NULL) {
69665 |             /* Node has one child - pChildLo != NULL. */
69666 |             pDataBufferNode->pChildLo->pParent = pDataBufferNode->pParent;
69667 |
69668 |             if (pDataBufferNode->pParent == NULL) {
69669 |                 MA_ASSERT(pResourceManager->pRootDataBufferNode == pDataBufferNode);
69670 |                 pResourceManager->pRootDataBufferNode = pDataBufferNode->pChildLo;
69671 |             } else {
69672 |                 if (pDataBufferNode->pParent->pChildLo == pDataBufferNode) {
69673 |                     pDataBufferNode->pParent->pChildLo = pDataBufferNode->pChildLo;
69674 |                 } else {
69675 |                     pDataBufferNode->pParent->pChildHi = pDataBufferNode->pChildLo;
69676 |                 }
69677 |             }
69678 |         } else {
69679 |             /* Complex case - deleting a node with two children. */
69680 |             ma_resource_manager_data_buffer_node* pReplacementDataBufferNode;
69681 |
69682 |             /* For now we are just going to use the in-order successor as the replacement, but we
> may want to try to keep this balanced by switching between the two. */
69683 |             pReplacementDataBufferNode =
> ma_resource_manager_data_buffer_node_find_inorder_successor(pDataBufferNode);
69684 |             MA_ASSERT(pReplacementDataBufferNode != NULL);
69685 |
69686 |             /*
69687 |             Now that we have our replacement node we can make the change. The simple way to do this
> would be to just exchange the values, and then remove the replacement
69688 |             node, however we track specific nodes via pointers which means we can't just swap out
> the values. We need to instead just change the pointers around. The
69689 |             replacement node should have at most 1 child. Therefore, we can detach it in terms of
> our simpler cases above. What we're essentially doing is detaching the
69690 |             replacement node and reinserting it into the same position as the deleted node.
69691 |             */

```

```

69692 |         MA_ASSERT(pReplacementDataBufferNode->pParent != NULL); /* The replacement node should
> never be the root which means it should always have a parent. */
69693 |         MA_ASSERT(pReplacementDataBufferNode->pChildLo == NULL); /* Because we used in-order
> successor. This would be pChildHi == NULL if we used in-order predecessor. */
69694 |
69695 |         if (pReplacementDataBufferNode->pChildHi == NULL) {
69696 |             if (pReplacementDataBufferNode->pParent->pChildLo == pReplacementDataBufferNode) {
69697 |                 pReplacementDataBufferNode->pParent->pChildLo = NULL;
69698 |             } else {
69699 |                 pReplacementDataBufferNode->pParent->pChildHi = NULL;
69700 |             }
69701 |         } else {
69702 |             pReplacementDataBufferNode->pChildHi->pParent = pReplacementDataBufferNode->pParent;
69703 |             if (pReplacementDataBufferNode->pParent->pChildLo == pReplacementDataBufferNode) {
69704 |                 pReplacementDataBufferNode->pParent->pChildLo =
> pReplacementDataBufferNode->pChildHi;
69705 |             } else {
69706 |                 pReplacementDataBufferNode->pParent->pChildHi =
> pReplacementDataBufferNode->pChildHi;
69707 |             }
69708 |         }
69709 |
69710 |
69711 |         /* The replacement node has essentially been detached from the binary tree, so now we
> need to replace the old data buffer with it. The first thing to update is the parent */
69712 |         if (pDataBufferNode->pParent != NULL) {
69713 |             if (pDataBufferNode->pParent->pChildLo == pDataBufferNode) {
69714 |                 pDataBufferNode->pParent->pChildLo = pReplacementDataBufferNode;
69715 |             } else {
69716 |                 pDataBufferNode->pParent->pChildHi = pReplacementDataBufferNode;
69717 |             }
69718 |         }
69719 |
69720 |         /* Now need to update the replacement node's pointers. */
69721 |         pReplacementDataBufferNode->pParent = pDataBufferNode->pParent;
69722 |         pReplacementDataBufferNode->pChildLo = pDataBufferNode->pChildLo;
69723 |         pReplacementDataBufferNode->pChildHi = pDataBufferNode->pChildHi;
69724 |
69725 |         /* Now the children of the replacement node need to have their parent pointers updated.
> */
69726 |         if (pReplacementDataBufferNode->pChildLo != NULL) {
69727 |             pReplacementDataBufferNode->pChildLo->pParent = pReplacementDataBufferNode;
69728 |         }
69729 |         if (pReplacementDataBufferNode->pChildHi != NULL) {
69730 |             pReplacementDataBufferNode->pChildHi->pParent = pReplacementDataBufferNode;
69731 |         }
69732 |
69733 |         /* Now the root node needs to be updated. */
69734 |         if (pResourceManager->pRootDataBufferNode == pDataBufferNode) {
69735 |             pResourceManager->pRootDataBufferNode = pReplacementDataBufferNode;
69736 |         }
69737 |     }
69738 | }
69739 |
69740 |     return MA_SUCCESS;
69741 | }
69742 |
69743 | #if 0 /* Unused for now. */
69744 | static ma_result ma_resource_manager_data_buffer_node_remove_by_key(ma_resource_manager*
> pResourceManager, ma_uint32 hashedName32)
69745 | {
69746 |     ma_result result;
69747 |     ma_resource_manager_data_buffer_node* pDataBufferNode;
69748 |
69749 |     result = ma_resource_manager_data_buffer_search(pResourceManager, hashedName32,
> &pDataBufferNode);
69750 |     if (result != MA_SUCCESS) {
69751 |         return result; /* Could not find the data buffer. */
69752 |     }
69753 |
69754 |     return ma_resource_manager_data_buffer_remove(pResourceManager, pDataBufferNode);
69755 | }
69756 | #endif
69757 |
69758 | static ma_resource_manager_data_supply_type
> ma_resource_manager_data_buffer_node_get_data_supply_type(ma_resource_manager_data_buffer_node*
> pDataBufferNode)
69759 | {
69760 |     return (ma_resource_manager_data_supply_type)ma_atomic_load_i32(&pDataBufferNode->data.type);

```

```

69761 | }
69762 |
69763 | static void
> ma_resource_manager_data_buffer_node_set_data_supply_type(ma_resource_manager_data_buffer_node*
> pDataBufferNode, ma_resource_manager_data_supply_type supplyType)
69764 | {
69765 |     ma_atomic_exchange_i32(&pDataBufferNode->data.type, supplyType);
69766 | }
69767 |
69768 | static ma_result ma_resource_manager_data_buffer_node_increment_ref(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode, ma_uint32* pNewRefCount)
69769 | {
69770 |     ma_uint32 refCount;
69771 |
69772 |     MA_ASSERT(pResourceManager != NULL);
69773 |     MA_ASSERT(pDataBufferNode != NULL);
69774 |
69775 |     (void)pResourceManager;
69776 |
69777 |     refCount = ma_atomic_fetch_add_32(&pDataBufferNode->refCount, 1) + 1;
69778 |
69779 |     if (pNewRefCount != NULL) {
69780 |         *pNewRefCount = refCount;
69781 |     }
69782 |
69783 |     return MA_SUCCESS;
69784 | }
69785 |
69786 | static ma_result ma_resource_manager_data_buffer_node_decrement_ref(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode, ma_uint32* pNewRefCount)
69787 | {
69788 |     ma_uint32 refCount;
69789 |
69790 |     MA_ASSERT(pResourceManager != NULL);
69791 |     MA_ASSERT(pDataBufferNode != NULL);
69792 |
69793 |     (void)pResourceManager;
69794 |
69795 |     refCount = ma_atomic_fetch_sub_32(&pDataBufferNode->refCount, 1) - 1;
69796 |
69797 |     if (pNewRefCount != NULL) {
69798 |         *pNewRefCount = refCount;
69799 |     }
69800 |
69801 |     return MA_SUCCESS;
69802 | }
69803 |
69804 | static void ma_resource_manager_data_buffer_node_free(ma_resource_manager* pResourceManager,
> ma_resource_manager_data_buffer_node* pDataBufferNode)
69805 | {
69806 |     MA_ASSERT(pResourceManager != NULL);
69807 |     MA_ASSERT(pDataBufferNode != NULL);
69808 |
69809 |     if (pDataBufferNode->isDataOwnedByResourceManager) {
69810 |         if (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBufferNode) ==
> ma_resource_manager_data_supply_type_encoded) {
69811 |             ma_free((void*)pDataBufferNode->data.backend.encoded.pData,
> &pResourceManager->config.allocationCallbacks);
69812 |             pDataBufferNode->data.backend.encoded.pData = NULL;
69813 |             pDataBufferNode->data.backend.encoded.sizeInBytes = 0;
69814 |         } else if (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBufferNode) ==
> ma_resource_manager_data_supply_type_decoded) {
69815 |             ma_free((void*)pDataBufferNode->data.backend.decoded.pData,
> &pResourceManager->config.allocationCallbacks);
69816 |             pDataBufferNode->data.backend.decoded.pData = NULL;
69817 |             pDataBufferNode->data.backend.decoded.totalFrameCount = 0;
69818 |         } else if (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBufferNode) ==
> ma_resource_manager_data_supply_type_decoded_paged) {
69819 |             ma_paged_audio_buffer_data_uninit(&pDataBufferNode->data.backend.decodedPaged.data,
> &pResourceManager->config.allocationCallbacks);
69820 |         } else {
69821 |             /* Should never hit this if the node was successfully initialized. */
69822 |             MA_ASSERT(pDataBufferNode->result != MA_SUCCESS);
69823 |         }
69824 |     }
69825 |
69826 |     /* The data buffer itself needs to be freed. */
69827 |     ma_free(pDataBufferNode, &pResourceManager->config.allocationCallbacks);
69828 | }

```

```

69829 |
69830 | static ma_result ma_resource_manager_data_buffer_node_result(const
> ma_resource_manager_data_buffer_node* pDataBufferNode)
69831 | {
69832 |     MA_ASSERT(pDataBufferNode != NULL);
69833 |
69834 |     return (ma_result)ma_atomic_load_i32((ma_result*)&pDataBufferNode->result);    /* Need a naughty
> const-cast here. */
69835 | }
69836 |
69837 |
69838 | static ma_bool32 ma_resource_manager_is_threading_enabled(const ma_resource_manager*
> pResourceManager)
69839 | {
69840 |     MA_ASSERT(pResourceManager != NULL);
69841 |
69842 |     return (pResourceManager->config.flags & MA_RESOURCE_MANAGER_FLAG_NO_THREADING) == 0;
69843 | }
69844 |
69845 |
69846 | typedef struct
69847 | {
69848 |     union
69849 |     {
69850 |         ma_async_notification_event e;
69851 |         ma_async_notification_poll p;
69852 |     } backend; /* Must be the first member. */
69853 |     ma_resource_manager* pResourceManager;
69854 | } ma_resource_manager_inline_notification;
69855 |
69856 | static ma_result ma_resource_manager_inline_notification_init(ma_resource_manager* pResourceManager,
> ma_resource_manager_inline_notification* pNotification)
69857 | {
69858 |     MA_ASSERT(pResourceManager != NULL);
69859 |     MA_ASSERT(pNotification != NULL);
69860 |
69861 |     pNotification->pResourceManager = pResourceManager;
69862 |
69863 |     if (ma_resource_manager_is_threading_enabled(pResourceManager)) {
69864 |         return ma_async_notification_event_init(&pNotification->backend.e);
69865 |     } else {
69866 |         return ma_async_notification_poll_init(&pNotification->backend.p);
69867 |     }
69868 | }
69869 |
69870 | static void ma_resource_manager_inline_notification_uninit(ma_resource_manager_inline_notification*
> pNotification)
69871 | {
69872 |     MA_ASSERT(pNotification != NULL);
69873 |
69874 |     if (ma_resource_manager_is_threading_enabled(pNotification->pResourceManager)) {
69875 |         ma_async_notification_event_uninit(&pNotification->backend.e);
69876 |     } else {
69877 |         /* No need to uninitialized a polling notification. */
69878 |     }
69879 | }
69880 |
69881 | static void ma_resource_manager_inline_notification_wait(ma_resource_manager_inline_notification*
> pNotification)
69882 | {
69883 |     MA_ASSERT(pNotification != NULL);
69884 |
69885 |     if (ma_resource_manager_is_threading_enabled(pNotification->pResourceManager)) {
69886 |         ma_async_notification_event_wait(&pNotification->backend.e);
69887 |     } else {
69888 |         while (ma_async_notification_poll_is_signalled(&pNotification->backend.p) == MA_FALSE) {
69889 |             ma_result result =
> ma_resource_manager_process_next_job(pNotification->pResourceManager);
69890 |             if (result == MA_NO_DATA_AVAILABLE || result == MA_CANCELLED) {
69891 |                 break;
69892 |             }
69893 |         }
69894 |     }
69895 | }
69896 |
69897 | static void
> ma_resource_manager_inline_notification_wait_and_uninit(ma_resource_manager_inline_notification*
> pNotification)
69898 | {

```

```

69899 |     ma_resource_manager_inline_notification_wait(pNotification);
69900 |     ma_resource_manager_inline_notification_uninit(pNotification);
69901 | }
69902 |
69903 |
69904 | static void ma_resource_manager_data_buffer_bst_lock(ma_resource_manager* pResourceManager)
69905 | {
69906 |     MA_ASSERT(pResourceManager != NULL);
69907 |
69908 |     if (ma_resource_manager_is_threading_enabled(pResourceManager)) {
69909 |         #ifndef MA_NO_THREADING
69910 |         {
69911 |             ma_mutex_lock(&pResourceManager->dataBufferBSTLock);
69912 |         }
69913 |         #else
69914 |         {
69915 |             MA_ASSERT(MA_FALSE);    /* Should never hit this. */
69916 |         }
69917 |         #endif
69918 |     } else {
69919 |         /* Threading not enabled. Do nothing. */
69920 |     }
69921 | }
69922 |
69923 | static void ma_resource_manager_data_buffer_bst_unlock(ma_resource_manager* pResourceManager)
69924 | {
69925 |     MA_ASSERT(pResourceManager != NULL);
69926 |
69927 |     if (ma_resource_manager_is_threading_enabled(pResourceManager)) {
69928 |         #ifndef MA_NO_THREADING
69929 |         {
69930 |             ma_mutex_unlock(&pResourceManager->dataBufferBSTLock);
69931 |         }
69932 |         #else
69933 |         {
69934 |             MA_ASSERT(MA_FALSE);    /* Should never hit this. */
69935 |         }
69936 |         #endif
69937 |     } else {
69938 |         /* Threading not enabled. Do nothing. */
69939 |     }
69940 | }
69941 |
69942 | #ifndef MA_NO_THREADING
69943 | static ma_thread_result MA_THREADCALL ma_resource_manager_job_thread(void* pUserData)
69944 | {
69945 |     ma_resource_manager* pResourceManager = (ma_resource_manager*)pUserData;
69946 |     MA_ASSERT(pResourceManager != NULL);
69947 |
69948 |     for (;;) {
69949 |         ma_result result;
69950 |         ma_job job;
69951 |
69952 |         result = ma_resource_manager_next_job(pResourceManager, &job);
69953 |         if (result != MA_SUCCESS) {
69954 |             break;
69955 |         }
69956 |
69957 |         /* Terminate if we got a quit message. */
69958 |         if (job.toc.breakup.code == MA_JOB_TYPE_QUIT) {
69959 |             break;
69960 |         }
69961 |
69962 |         ma_job_process(&job);
69963 |     }
69964 |
69965 |     return (ma_thread_result)0;
69966 | }
69967 | #endif
69968 |
69969 | MA_API ma_resource_manager_config ma_resource_manager_config_init(void)
69970 | {
69971 |     ma_resource_manager_config config;
69972 |
69973 |     MA_ZERO_OBJECT(&config);
69974 |     config.decodedFormat = ma_format_unknown;
69975 |     config.decodedChannels = 0;
69976 |     config.decodedSampleRate = 0;
69977 |     config.jobThreadCount = 1;    /* A single miniaudio-managed job thread by default. */

```

```

69978 |     config.jobQueueCapacity = MA_JOB_TYPE_RESOURCE_MANAGER_QUEUE_CAPACITY;
69979 |     config.resampling       = ma_resampler_config_init(ma_format_unknown, 0, 0, 0,
> ma_resample_algorithm_linear); /* Format/channels/rate doesn't matter here. */
69980 |
69981 |     /* Flags. */
69982 |     config.flags = 0;
69983 |     #ifdef MA_NO_THREADING
69984 |     {
69985 |         /* Threading is disabled at compile time so disable threading at runtime as well by default.
> */
69986 |         config.flags |= MA_RESOURCE_MANAGER_FLAG_NO_THREADING;
69987 |         config.jobThreadCount = 0;
69988 |     }
69989 |     #endif
69990 |
69991 |     return config;
69992 | }
69993 |
69994 |
69995 | MA_API ma_result ma_resource_manager_init(const ma_resource_manager_config* pConfig,
> ma_resource_manager* pResourceManager)
69996 | {
69997 |     ma_result result;
69998 |     ma_job_queue_config jobQueueConfig;
69999 |
70000 |     if (pResourceManager == NULL) {
70001 |         return MA_INVALID_ARGS;
70002 |     }
70003 |
70004 |     MA_ZERO_OBJECT(pResourceManager);
70005 |
70006 |     if (pConfig == NULL) {
70007 |         return MA_INVALID_ARGS;
70008 |     }
70009 |
70010 |     #ifndef MA_NO_THREADING
70011 |     {
70012 |         if (pConfig->jobThreadCount > ma_countof(pResourceManager->jobThreads)) {
70013 |             return MA_INVALID_ARGS; /* Requesting too many job threads. */
70014 |         }
70015 |     }
70016 |     #endif
70017 |
70018 |     pResourceManager->config = *pConfig;
70019 |     ma_allocation_callbacks_init_copy(&pResourceManager->config.allocationCallbacks,
> &pConfig->allocationCallbacks);
70020 |
70021 |     /* Get the log set up early so we can start using it as soon as possible. */
70022 |     if (pResourceManager->config.pLog == NULL) {
70023 |         result = ma_log_init(&pResourceManager->config.allocationCallbacks, &pResourceManager->log);
70024 |         if (result == MA_SUCCESS) {
70025 |             pResourceManager->config.pLog = &pResourceManager->log;
70026 |         } else {
70027 |             pResourceManager->config.pLog = NULL; /* Logging is unavailable. */
70028 |         }
70029 |     }
70030 |
70031 |     if (pResourceManager->config.pVFS == NULL) {
70032 |         result = ma_default_vfs_init(&pResourceManager->defaultVFS,
> &pResourceManager->config.allocationCallbacks);
70033 |         if (result != MA_SUCCESS) {
70034 |             return result; /* Failed to initialize the default file system. */
70035 |         }
70036 |
70037 |         pResourceManager->config.pVFS = &pResourceManager->defaultVFS;
70038 |     }
70039 |
70040 |     /* If threading has been disabled at compile time, enforce it at run time as well. */
70041 |     #ifdef MA_NO_THREADING
70042 |     {
70043 |         pResourceManager->config.flags |= MA_RESOURCE_MANAGER_FLAG_NO_THREADING;
70044 |     }
70045 |     #endif
70046 |
70047 |     /* We need to force MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING if
> MA_RESOURCE_MANAGER_FLAG_NO_THREADING is set. */
70048 |     if ((pResourceManager->config.flags & MA_RESOURCE_MANAGER_FLAG_NO_THREADING) != 0) {
70049 |         pResourceManager->config.flags |= MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING;
70050 |     }

```



```

70051 |         /* We cannot allow job threads when MA_RESOURCE_MANAGER_FLAG_NO_THREADING has been set. This
> | is an invalid use case. */
70052 |         if (pResourceManager->config.jobThreadCount > 0) {
70053 |             return MA_INVALID_ARGS;
70054 |         }
70055 |     }
70056 |
70057 |     /* Job queue. */
70058 |     jobQueueConfig.capacity = pResourceManager->config.jobQueueCapacity;
70059 |     jobQueueConfig.flags    = 0;
70060 |     if ((pResourceManager->config.flags & MA_RESOURCE_MANAGER_FLAG_NON_BLOCKING) != 0) {
70061 |         if (pResourceManager->config.jobThreadCount > 0) {
70062 |             return MA_INVALID_ARGS; /* Non-blocking mode is only valid for self-managed job threads.
> | */
70063 |         }
70064 |
70065 |         jobQueueConfig.flags |= MA_JOB_QUEUE_FLAG_NON_BLOCKING;
70066 |     }
70067 |
70068 |     result = ma_job_queue_init(&jobQueueConfig, &pResourceManager->config.allocationCallbacks,
> | &pResourceManager->jobQueue);
70069 |     if (result != MA_SUCCESS) {
70070 |         return result;
70071 |     }
70072 |
70073 |
70074 |     /* Custom decoding backends. */
70075 |     if (pConfig->ppCustomDecodingBackendVTables != NULL && pConfig->customDecodingBackendCount > 0)
> | {
70076 |         size_t sizeInBytes = sizeof(*pResourceManager->config.ppCustomDecodingBackendVTables) *
> | pConfig->customDecodingBackendCount;
70077 |         ma_decoding_backend_vtable** ppCustomDecodingBackendVTables;
70078 |
70079 |         ppCustomDecodingBackendVTables = (ma_decoding_backend_vtable**)ma_malloc(sizeInBytes,
> | &pResourceManager->config.allocationCallbacks);
70080 |         if (pResourceManager->config.ppCustomDecodingBackendVTables == NULL) {
70081 |             ma_job_queue_uninit(&pResourceManager->jobQueue,
> | &pResourceManager->config.allocationCallbacks);
70082 |             return MA_OUT_OF_MEMORY;
70083 |         }
70084 |
70085 |         MA_COPY_MEMORY(ppCustomDecodingBackendVTables, pConfig->ppCustomDecodingBackendVTables,
> | sizeInBytes);
70086 |
70087 |         pResourceManager->config.ppCustomDecodingBackendVTables = ppCustomDecodingBackendVTables;
70088 |         pResourceManager->config.customDecodingBackendCount      =
> | pConfig->customDecodingBackendCount;
70089 |         pResourceManager->config.pCustomDecodingBackendUserData =
> | pConfig->pCustomDecodingBackendUserData;
70090 |     }
70091 |
70092 |
70093 |
70094 |     /* Here is where we initialize our threading stuff. We don't do this if we don't support
> | threading. */
70095 |     if (ma_resource_manager_is_threading_enabled(pResourceManager)) {
70096 |         #ifndef MA_NO_THREADING
70097 |         {
70098 |             ma_uint32 iJobThread;
70099 |
70100 |             /* Data buffer lock. */
70101 |             result = ma_mutex_init(&pResourceManager->dataBufferBSTLock);
70102 |             if (result != MA_SUCCESS) {
70103 |                 ma_job_queue_uninit(&pResourceManager->jobQueue,
> | &pResourceManager->config.allocationCallbacks);
70104 |                 return result;
70105 |             }
70106 |
70107 |             /* Create the job threads last to ensure the threads has access to valid data. */
70108 |             for (iJobThread = 0; iJobThread < pResourceManager->config.jobThreadCount; iJobThread +=
> | 1) {
70109 |                 result = ma_thread_create(&pResourceManager->jobThreads[iJobThread],
> | ma_thread_priority_normal, pResourceManager->config.jobThreadStackSize,
> | ma_resource_manager_job_thread, pResourceManager, &pResourceManager->config.allocationCallbacks);
70110 |                 if (result != MA_SUCCESS) {
70111 |                     ma_mutex_uninit(&pResourceManager->dataBufferBSTLock);
70112 |                     ma_job_queue_uninit(&pResourceManager->jobQueue,
> | &pResourceManager->config.allocationCallbacks);
70113 |                     return result;

```

```

70114 |         }
70115 |     }
70116 | }
70117 | #else
70118 | {
70119 |     /* Threading is disabled at compile time. We should never get here because validation
> checks should have already been performed. */
70120 |     MA_ASSERT(MA_FALSE);
70121 | }
70122 | #endif
70123 | }
70124 |
70125 | return MA_SUCCESS;
70126 | }
70127 |
70128 |
70129 | static void ma_resource_manager_delete_all_data_buffer_nodes(ma_resource_manager* pResourceManager)
70130 | {
70131 |     MA_ASSERT(pResourceManager);
70132 |
70133 |     /* If everything was done properly, there shouldn't be any active data buffers. */
70134 |     while (pResourceManager->pRootDataBufferNode != NULL) {
70135 |         ma_resource_manager_data_buffer_node* pDataBufferNode =
> pResourceManager->pRootDataBufferNode;
70136 |         ma_resource_manager_data_buffer_node_remove(pResourceManager, pDataBufferNode);
70137 |
70138 |         /* The data buffer has been removed from the BST, so now we need to free its data. */
70139 |         ma_resource_manager_data_buffer_node_free(pResourceManager, pDataBufferNode);
70140 |     }
70141 | }
70142 |
70143 | MA_API void ma_resource_manager_uninit(ma_resource_manager* pResourceManager)
70144 | {
70145 |     if (pResourceManager == NULL) {
70146 |         return;
70147 |     }
70148 |
70149 |     /*
70150 |     Job threads need to be killed first. To do this we need to post a quit message to the message
> queue and then wait for the thread. The quit message will never be removed from the
70151 |     queue which means it will never not be returned after being encountered for the first time which
> means all threads will eventually receive it.
70152 |     */
70153 |     ma_resource_manager_post_job_quit(pResourceManager);
70154 |
70155 |     /* Wait for every job to finish before continuing to ensure nothing is still trying to access any
> of our objects below. */
70156 |     if (ma_resource_manager_is_threading_enabled(pResourceManager)) {
70157 |         #ifndef MA_NO_THREADING
70158 |         {
70159 |             ma_uint32 iJobThread;
70160 |
70161 |             for (iJobThread = 0; iJobThread < pResourceManager->config.jobThreadCount; iJobThread +=
> 1) {
70162 |                 ma_thread_wait(&pResourceManager->jobThreads[iJobThread]);
70163 |             }
70164 |         }
70165 |         #else
70166 |         {
70167 |             MA_ASSERT(MA_FALSE); /* Should never hit this. */
70168 |         }
70169 |         #endif
70170 |     }
70171 |
70172 |     /* At this point the thread should have returned and no other thread should be accessing our
> data. We can now delete all data buffers. */
70173 |     ma_resource_manager_delete_all_data_buffer_nodes(pResourceManager);
70174 |
70175 |     /* The job queue is no longer needed. */
70176 |     ma_job_queue_uninit(&pResourceManager->jobQueue, &pResourceManager->config.allocationCallbacks);
70177 |
70178 |     /* We're no longer doing anything with data buffers so the lock can now be uninitialized. */
70179 |     if (ma_resource_manager_is_threading_enabled(pResourceManager)) {
70180 |         #ifndef MA_NO_THREADING
70181 |         {
70182 |             ma_mutex_uninit(&pResourceManager->dataBufferBSTLock);
70183 |         }
70184 |         #else
70185 |         {

```

```

70186 |         MA_ASSERT(MA_FALSE);    /* Should never hit this. */
70187 |     }
70188 |     #endif
70189 | }
70190 |
70191 |     ma_free((ma_decoding_backend_vtable**)pResourceManager->config.ppCustomDecodingBackendVTables,
> &pResourceManager->config.allocationCallbacks); /* <-- Naughty const-cast, but this is safe. */
70192 |
70193 |     if (pResourceManager->config.pLog == &pResourceManager->log) {
70194 |         ma_log_uninit(&pResourceManager->log);
70195 |     }
70196 | }
70197 |
70198 | MA_API ma_log* ma_resource_manager_get_log(ma_resource_manager* pResourceManager)
70199 | {
70200 |     if (pResourceManager == NULL) {
70201 |         return NULL;
70202 |     }
70203 |
70204 |     return pResourceManager->config.pLog;
70205 | }
70206 |
70207 |
70208 |
70209 | MA_API ma_resource_manager_data_source_config ma_resource_manager_data_source_config_init(void)
70210 | {
70211 |     ma_resource_manager_data_source_config config;
70212 |
70213 |     MA_ZERO_OBJECT(&config);
70214 |     config.rangeBegInPCMFrames = MA_DATA_SOURCE_DEFAULT_RANGE_BEG;
70215 |     config.rangeEndInPCMFrames = MA_DATA_SOURCE_DEFAULT_RANGE_END;
70216 |     config.loopPointBegInPCMFrames = MA_DATA_SOURCE_DEFAULT_LOOP_POINT_BEG;
70217 |     config.loopPointEndInPCMFrames = MA_DATA_SOURCE_DEFAULT_LOOP_POINT_END;
70218 |     config.isLooping = MA_FALSE;
70219 |
70220 |     return config;
70221 | }
70222 |
70223 |
70224 | static ma_decoder_config ma_resource_manager__init_decoder_config(ma_resource_manager*
> pResourceManager)
70225 | {
70226 |     ma_decoder_config config;
70227 |
70228 |     config = ma_decoder_config_init(pResourceManager->config.decodedFormat,
> pResourceManager->config.decodedChannels, pResourceManager->config.decodedSampleRate);
70229 |     config.allocationCallbacks = pResourceManager->config.allocationCallbacks;
70230 |     config.ppCustomBackendVTables = pResourceManager->config.ppCustomDecodingBackendVTables;
70231 |     config.customBackendCount = pResourceManager->config.customDecodingBackendCount;
70232 |     config.pCustomBackendUserData = pResourceManager->config.pCustomDecodingBackendUserData;
70233 |     config.resampling = pResourceManager->config.resampling;
70234 |
70235 |     return config;
70236 | }
70237 |
70238 | static ma_result ma_resource_manager__init_decoder(ma_resource_manager* pResourceManager, const
> char* pFilePath, const wchar_t* pFilePathW, ma_decoder* pDecoder)
70239 | {
70240 |     ma_result result;
70241 |     ma_decoder_config config;
70242 |
70243 |     MA_ASSERT(pResourceManager != NULL);
70244 |     MA_ASSERT(pFilePath != NULL || pFilePathW != NULL);
70245 |     MA_ASSERT(pDecoder != NULL);
70246 |
70247 |     config = ma_resource_manager__init_decoder_config(pResourceManager);
70248 |
70249 |     if (pFilePath != NULL) {
70250 |         result = ma_decoder_init_vfs(pResourceManager->config.pVFS, pFilePath, &config, pDecoder);
70251 |         if (result != MA_SUCCESS) {
70252 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_WARNING,
> "Failed to load file \"%s\". %s.\n", pFilePath, ma_result_description(result));
70253 |             return result;
70254 |         }
70255 |     } else {
70256 |         result = ma_decoder_init_vfs_w(pResourceManager->config.pVFS, pFilePathW, &config,
> pDecoder);
70257 |         if (result != MA_SUCCESS) {
70258 |             #if (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L) || defined(_MSC_VER)

```

```

70259 |         ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_WARNING,
> "Failed to load file \"%ls\\\". %s.\\n", pFilePathW, ma_result_description(result));
70260 |         #endif
70261 |         return result;
70262 |     }
70263 | }
70264 |
70265 |     return MA_SUCCESS;
70266 | }
70267 |
70268 | static ma_bool32 ma_resource_manager_data_buffer_has_connector(ma_resource_manager_data_buffer*
> pDataBuffer)
70269 | {
70270 |     return ma_atomic_bool32_get(&pDataBuffer->isConnectorInitialized);
70271 | }
70272 |
70273 | static ma_data_source*
> ma_resource_manager_data_buffer_get_connector(ma_resource_manager_data_buffer* pDataBuffer)
70274 | {
70275 |     if (ma_resource_manager_data_buffer_has_connector(pDataBuffer) == MA_FALSE) {
70276 |         return NULL; /* Connector not yet initialized. */
70277 |     }
70278 |
70279 |     switch (pDataBuffer->pNode->data.type)
70280 |     {
70281 |         case ma_resource_manager_data_supply_type_encoded: return
> &pDataBuffer->connector.decoder;
70282 |         case ma_resource_manager_data_supply_type_decoded: return
> &pDataBuffer->connector.buffer;
70283 |         case ma_resource_manager_data_supply_type_decoded_paged: return
> &pDataBuffer->connector.pagedBuffer;
70284 |
70285 |         case ma_resource_manager_data_supply_type_unknown:
70286 |         default:
70287 |         {
70288 |             ma_log_postf(ma_resource_manager_get_log(pDataBuffer->pResourceManager),
> MA_LOG_LEVEL_ERROR, "Failed to retrieve data buffer connector. Unknown data supply type.\\n");
70289 |             return NULL;
70290 |         };
70291 |     };
70292 | }
70293 |
70294 | static ma_result ma_resource_manager_data_buffer_init_connector(ma_resource_manager_data_buffer*
> pDataBuffer, const ma_resource_manager_data_source_config* pConfig, ma_async_notification*
> pInitNotification, ma_fence* pInitFence)
70295 | {
70296 |     ma_result result;
70297 |
70298 |     MA_ASSERT(pDataBuffer != NULL);
70299 |     MA_ASSERT(pConfig != NULL);
70300 |     MA_ASSERT(ma_resource_manager_data_buffer_has_connector(pDataBuffer) == MA_FALSE);
70301 |
70302 |     /* The underlying data buffer must be initialized before we'll be able to know how to initialize
> the backend. */
70303 |     result = ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode);
70304 |     if (result != MA_SUCCESS && result != MA_BUSY) {
70305 |         return result; /* The data buffer is in an erroneous state. */
70306 |     }
70307 |
70308 |     /*
70309 |     We need to initialize either a ma_decoder or an ma_audio_buffer depending on whether or not the
> backing data is encoded or decoded. These act as the
70310 |     "instance" to the data and are used to form the connection between underlying data buffer and
> the data source. If the data buffer is decoded, we can use
70311 |     an ma_audio_buffer. This enables us to use memory mapping when mixing which saves us a bit of
> data movement overhead.
70312 |     */
70313 |     switch (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode))
70314 |     {
70315 |         case ma_resource_manager_data_supply_type_encoded: /* Connector is a decoder. */
70316 |         {
70317 |             ma_decoder_config config;
70318 |             config = ma_resource_manager__init_decoder_config(pDataBuffer->pResourceManager);
70319 |             result = ma_decoder_init_memory(pDataBuffer->pNode->data.backend.encoded.pData,
> pDataBuffer->pNode->data.backend.encoded.sizeInBytes, &config, &pDataBuffer->connector.decoder);
70320 |             } break;
70321 |
70322 |         case ma_resource_manager_data_supply_type_decoded: /* Connector is an audio buffer.
> */

```

```

70323 |         {
70324 |             ma_audio_buffer_config config;
70325 |             config = ma_audio_buffer_config_init(pDataBuffer->pNode->data.backend.decoded.format,
> pDataBuffer->pNode->data.backend.decoded.channels,
> pDataBuffer->pNode->data.backend.decoded.totalFrameCount,
> pDataBuffer->pNode->data.backend.decoded.pData, NULL);
70326 |             result = ma_audio_buffer_init(&config, &pDataBuffer->connector.buffer);
70327 |         } break;
70328 |
70329 |         case ma_resource_manager_data_supply_type_decoded_paged: /* Connector is a paged audio
> buffer. */
70330 |         {
70331 |             ma_paged_audio_buffer_config config;
70332 |             config =
> ma_paged_audio_buffer_config_init(&pDataBuffer->pNode->data.backend.decodedPaged.data);
70333 |             result = ma_paged_audio_buffer_init(&config, &pDataBuffer->connector.pagedBuffer);
70334 |         } break;
70335 |
70336 |         case ma_resource_manager_data_supply_type_unknown:
70337 |         default:
70338 |         {
70339 |             /* Unknown data supply type. Should never happen. Need to post an error here. */
70340 |             return MA_INVALID_ARGS;
70341 |         };
70342 |     }
70343 |
70344 |     /*
70345 |     Initialization of the connector is when we can fire the init notification. This will give the
> application access to
70346 |     the format/channels/rate of the data source.
70347 |     */
70348 |     if (result == MA_SUCCESS) {
70349 |         /*
70350 |         The resource manager supports the ability to set the range and loop settings via a config at
70351 |         initialization time. This results in an case where the ranges could be set explicitly via
70352 |         ma_data_source_set_*() before we get to this point here. If this happens, we'll end up
70353 |         hitting a case where we just override those settings which results in what feels like a bug.
70354 |
70355 |         To address this we only change the relevant properties if they're not equal to defaults. If
70356 |         they're equal to defaults there's no need to change them anyway. If they're *not* set to the
70357 |         default values, we can assume the user has set the range and loop settings via the config.
> If
70358 |         they're doing their own calls to ma_data_source_set_*() in addition to setting them via the
70359 |         config, that's entirely on the caller and any synchronization issue becomes their problem.
70360 |         */
70361 |         if (pConfig->rangeBegInPCMframes != MA_DATA_SOURCE_DEFAULT_RANGE_BEG ||
> pConfig->rangeEndInPCMframes != MA_DATA_SOURCE_DEFAULT_RANGE_END) {
70362 |             ma_data_source_set_range_in_pcm_frames(pDataBuffer, pConfig->rangeBegInPCMframes,
> pConfig->rangeEndInPCMframes);
70363 |         }
70364 |
70365 |         if (pConfig->loopPointBegInPCMframes != MA_DATA_SOURCE_DEFAULT_LOOP_POINT_BEG ||
> pConfig->loopPointEndInPCMframes != MA_DATA_SOURCE_DEFAULT_LOOP_POINT_END) {
70366 |             ma_data_source_set_loop_point_in_pcm_frames(pDataBuffer,
> pConfig->loopPointBegInPCMframes, pConfig->loopPointEndInPCMframes);
70367 |         }
70368 |
70369 |         if (pConfig->isLooping != MA_FALSE) {
70370 |             ma_data_source_set_looping(pDataBuffer, pConfig->isLooping);
70371 |         }
70372 |
70373 |         ma_atomic_bool32_set(&pDataBuffer->isConnectorInitialized, MA_TRUE);
70374 |
70375 |         if (pInitNotification != NULL) {
70376 |             ma_async_notification_signal(pInitNotification);
70377 |         }
70378 |
70379 |         if (pInitFence != NULL) {
70380 |             ma_fence_release(pInitFence);
70381 |         }
70382 |     }
70383 |
70384 |     /* At this point the backend should be initialized. We do *not* want to set pDataSource->result
> here - that needs to be done at a higher level to ensure it's done as the last step. */
70385 |     return result;
70386 | }
70387 |
70388 | static ma_result ma_resource_manager_data_buffer_uninit_connector(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer* pDataBuffer)

```

```

70389 | {
70390 |     MA_ASSERT(pResourceManager != NULL);
70391 |     MA_ASSERT(pDataBuffer      != NULL);
70392 |
70393 |     (void)pResourceManager;
70394 |
70395 |     switch (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode))
70396 |     {
70397 |         case ma_resource_manager_data_supply_type_encoded:          /* Connector is a decoder. */
70398 |         {
70399 |             ma_decoder_uninit(&pDataBuffer->connector.decoder);
70400 |         } break;
70401 |
70402 |         case ma_resource_manager_data_supply_type_decoded:          /* Connector is an audio buffer.
> */
70403 |         {
70404 |             ma_audio_buffer_uninit(&pDataBuffer->connector.buffer);
70405 |         } break;
70406 |
70407 |         case ma_resource_manager_data_supply_type_decoded_paged:    /* Connector is a paged audio
> buffer. */
70408 |         {
70409 |             ma_paged_audio_buffer_uninit(&pDataBuffer->connector.pagedBuffer);
70410 |         } break;
70411 |
70412 |         case ma_resource_manager_data_supply_type_unknown:
70413 |         default:
70414 |         {
70415 |             /* Unknown data supply type. Should never happen. Need to post an error here. */
70416 |             return MA_INVALID_ARGS;
70417 |         };
70418 |     }
70419 |
70420 |     return MA_SUCCESS;
70421 | }
70422 |
70423 | static ma_uint32
> ma_resource_manager_data_buffer_node_next_execution_order(ma_resource_manager_data_buffer_node*
> pDataBufferNode)
70424 | {
70425 |     MA_ASSERT(pDataBufferNode != NULL);
70426 |     return ma_atomic_fetch_add_32(&pDataBufferNode->executionCounter, 1);
70427 | }
70428 |
70429 | static ma_result ma_resource_manager_data_buffer_node_init_supply_encoded(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode, const char* pFilePath,
> const wchar_t* pFilePathW)
70430 | {
70431 |     ma_result result;
70432 |     size_t dataSizeInBytes;
70433 |     void* pData;
70434 |
70435 |     MA_ASSERT(pResourceManager != NULL);
70436 |     MA_ASSERT(pDataBufferNode != NULL);
70437 |     MA_ASSERT(pFilePath != NULL || pFilePathW != NULL);
70438 |
70439 |     result = ma_vfs_open_and_read_file_ex(pResourceManager->config.pVFS, pFilePath, pFilePathW,
> &pData, &dataSizeInBytes, &pResourceManager->config.allocationCallbacks);
70440 |     if (result != MA_SUCCESS) {
70441 |         if (pFilePath != NULL) {
70442 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_WARNING,
> "Failed to load file \"%s\". %s.\n", pFilePath, ma_result_description(result));
70443 |         } else {
70444 |             #if (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L) || defined(_MSC_VER)
70445 |                 ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_WARNING,
> "Failed to load file \"%ls\". %s.\n", pFilePathW, ma_result_description(result));
70446 |             #endif
70447 |         }
70448 |
70449 |         return result;
70450 |     }
70451 |
70452 |     pDataBufferNode->data.backend.encoded.pData      = pData;
70453 |     pDataBufferNode->data.backend.encoded.sizeInBytes = dataSizeInBytes;
70454 |     ma_resource_manager_data_buffer_node_set_data_supply_type(pDataBufferNode,
> ma_resource_manager_data_supply_type_encoded); /* <-- Must be set last. */
70455 |
70456 |     return MA_SUCCESS;
70457 | }

```

```

70458 |
70459 | static ma_result ma_resource_manager_data_buffer_node_init_supply_decoded(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode, const char* pFilePath,
> const wchar_t* pFilePathW, ma_uint32 flags, ma_decoder** ppDecoder)
70460 | {
70461 |     ma_result result = MA_SUCCESS;
70462 |     ma_decoder* pDecoder;
70463 |     ma_uint64 totalFrameCount;
70464 |
70465 |     MA_ASSERT(pResourceManager != NULL);
70466 |     MA_ASSERT(pDataBufferNode != NULL);
70467 |     MA_ASSERT(ppDecoder != NULL);
70468 |     MA_ASSERT(pFilePath != NULL || pFilePathW != NULL);
70469 |
70470 |     *ppDecoder = NULL; /* For safety. */
70471 |
70472 |     pDecoder = (ma_decoder*)ma_malloc(sizeof(*pDecoder),
> &pResourceManager->config.allocationCallbacks);
70473 |     if (pDecoder == NULL) {
70474 |         return MA_OUT_OF_MEMORY;
70475 |     }
70476 |
70477 |     result = ma_resource_manager__init_decoder(pResourceManager, pFilePath, pFilePathW, pDecoder);
70478 |     if (result != MA_SUCCESS) {
70479 |         ma_free(pDecoder, &pResourceManager->config.allocationCallbacks);
70480 |         return result;
70481 |     }
70482 |
70483 |     /*
70484 |     At this point we have the decoder and we now need to initialize the data supply. This will
70485 |     be either a decoded buffer, or a decoded paged buffer. A regular buffer is just one big heap
70486 |     allocated buffer, whereas a paged buffer is a linked list of paged-sized buffers. The latter
70487 |     is used when the length of a sound is unknown until a full decode has been performed.
70488 |     */
70489 |     if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_UNKNOWN_LENGTH) == 0) {
70490 |         result = ma_decoder_get_length_in_pcm_frames(pDecoder, &totalFrameCount);
70491 |         if (result != MA_SUCCESS) {
70492 |             return result;
70493 |         }
70494 |     } else {
70495 |         totalFrameCount = 0;
70496 |     }
70497 |
70498 |     if (totalFrameCount > 0) {
70499 |         /* It's a known length. The data supply is a regular decoded buffer. */
70500 |         ma_uint64 dataSizeInBytes;
70501 |         void* pData;
70502 |
70503 |         dataSizeInBytes = totalFrameCount * ma_get_bytes_per_frame(pDecoder->outputFormat,
> pDecoder->outputChannels);
70504 |         if (dataSizeInBytes > MA_SIZE_MAX) {
70505 |             ma_decoder_uninit(pDecoder);
70506 |             ma_free(pDecoder, &pResourceManager->config.allocationCallbacks);
70507 |             return MA_TOO_BIG;
70508 |         }
70509 |
70510 |         pData = ma_malloc((size_t)dataSizeInBytes, &pResourceManager->config.allocationCallbacks);
70511 |         if (pData == NULL) {
70512 |             ma_decoder_uninit(pDecoder);
70513 |             ma_free(pDecoder, &pResourceManager->config.allocationCallbacks);
70514 |             return MA_OUT_OF_MEMORY;
70515 |         }
70516 |
70517 |         /* The buffer needs to be initialized to silence in case the caller reads from it. */
70518 |         ma_silence_pcm_frames(pData, totalFrameCount, pDecoder->outputFormat,
> pDecoder->outputChannels);
70519 |
70520 |         /* Data has been allocated and the data supply can now be initialized. */
70521 |         pDataBufferNode->data.backend.decoded.pData = pData;
70522 |         pDataBufferNode->data.backend.decoded.totalFrameCount = totalFrameCount;
70523 |         pDataBufferNode->data.backend.decoded.format = pDecoder->outputFormat;
70524 |         pDataBufferNode->data.backend.decoded.channels = pDecoder->outputChannels;
70525 |         pDataBufferNode->data.backend.decoded.sampleRate = pDecoder->outputSampleRate;
70526 |         pDataBufferNode->data.backend.decoded.decodedFrameCount = 0;
70527 |         ma_resource_manager_data_buffer_node_set_data_supply_type(pDataBufferNode,
> ma_resource_manager_data_supply_type_decoded); /* <-- Must be set last. */
70528 |     } else {
70529 |         /*
70530 |         It's an unknown length. The data supply is a paged decoded buffer. Setting this up is

```



```

70531 |         actually easier than the non-paged decoded buffer because we just need to initialize
70532 |         a ma_paged_audio_buffer object.
70533 |         */
70534 |         result = ma_paged_audio_buffer_data_init(pDecoder->outputFormat, pDecoder->outputChannels,
> &pDataBufferNode->data.backend.decodedPaged.data);
70535 |         if (result != MA_SUCCESS) {
70536 |             ma_decoder_uninit(pDecoder);
70537 |             ma_free(pDecoder, &pResourceManager->config.allocationCallbacks);
70538 |             return result;
70539 |         }
70540 |
70541 |         pDataBufferNode->data.backend.decodedPaged.sampleRate = pDecoder->outputSampleRate;
70542 |         pDataBufferNode->data.backend.decodedPaged.decodedFrameCount = 0;
70543 |         ma_resource_manager_data_buffer_node_set_data_supply_type(pDataBufferNode,
> ma_resource_manager_data_supply_type_decoded_paged); /* <-- Must be set last. */
70544 |     }
70545 |
70546 |     *ppDecoder = pDecoder;
70547 |
70548 |     return MA_SUCCESS;
70549 | }
70550 |
70551 | static ma_result ma_resource_manager_data_buffer_node_decode_next_page(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode, ma_decoder* pDecoder)
70552 | {
70553 |     ma_result result = MA_SUCCESS;
70554 |     ma_uint64 pageSizeInFrames;
70555 |     ma_uint64 framesToTryReading;
70556 |     ma_uint64 framesRead;
70557 |
70558 |     MA_ASSERT(pResourceManager != NULL);
70559 |     MA_ASSERT(pDataBufferNode != NULL);
70560 |     MA_ASSERT(pDecoder != NULL);
70561 |
70562 |     /* We need to know the size of a page in frames to know how many frames to decode. */
70563 |     pageSizeInFrames = MA_RESOURCE_MANAGER_PAGE_SIZE_IN_MILLISECONDS *
> (pDecoder->outputSampleRate/1000);
70564 |     framesToTryReading = pageSizeInFrames;
70565 |
70566 |     /*
70567 |     Here is where we do the decoding of the next page. We'll run a slightly different path depending
70568 |     on whether or not we're using a flat or paged buffer because the allocation of the page differs
70569 |     between the two. For a flat buffer it's an offset to an already-allocated buffer. For a paged
70570 |     buffer, we need to allocate a new page and attach it to the linked list.
70571 |     */
70572 |     switch (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBufferNode))
70573 |     {
70574 |         case ma_resource_manager_data_supply_type_decoded:
70575 |         {
70576 |             /* The destination buffer is an offset to the existing buffer. Don't read more than we
> originally retrieved when we first initialized the decoder. */
70577 |             void* pDst;
70578 |             ma_uint64 framesRemaining = pDataBufferNode->data.backend.decoded.totalFrameCount -
> pDataBufferNode->data.backend.decoded.decodedFrameCount;
70579 |             if (framesToTryReading > framesRemaining) {
70580 |                 framesToTryReading = framesRemaining;
70581 |             }
70582 |
70583 |             if (framesToTryReading > 0) {
70584 |                 pDst = ma_offset_ptr(
70585 |                     pDataBufferNode->data.backend.decoded.pData,
70586 |                     pDataBufferNode->data.backend.decoded.decodedFrameCount *
> ma_get_bytes_per_frame(pDataBufferNode->data.backend.decoded.format,
> pDataBufferNode->data.backend.decoded.channels)
70587 |                 );
70588 |                 MA_ASSERT(pDst != NULL);
70589 |
70590 |                 result = ma_decoder_read_pcm_frames(pDecoder, pDst, framesToTryReading,
> &framesRead);
70591 |                 if (framesRead > 0) {
70592 |                     pDataBufferNode->data.backend.decoded.decodedFrameCount += framesRead;
70593 |                 }
70594 |             } else {
70595 |                 framesRead = 0;
70596 |             }
70597 |             break;
70598 |
70599 |             case ma_resource_manager_data_supply_type_decoded_paged:
70600 |             {

```

```

70601 |         /* The destination buffer is a freshly allocated page. */
70602 |         ma_paged_audio_buffer_page* pPage;
70603 |
70604 |         result =
70605 | > ma_paged_audio_buffer_data_allocate_page(&pDataBufferNode->data.backend.decodedPaged.data,
70606 | > framesToTryReading, NULL, &pResourceManager->config.allocationCallbacks, &pPage);
70607 |         if (result != MA_SUCCESS) {
70608 |             return result;
70609 |         }
70610 |         result = ma_decoder_read_pcm_frames(pDecoder, pPage->pAudioData, framesToTryReading,
70611 | > &framesRead);
70612 |         if (result == MA_SUCCESS && framesRead > 0) {
70613 |             pPage->sizeInFrames = framesRead;
70614 |             result =
70615 | > ma_paged_audio_buffer_data_append_page(&pDataBufferNode->data.backend.decodedPaged.data, pPage);
70616 |             if (result == MA_SUCCESS) {
70617 |                 pDataBufferNode->data.backend.decodedPaged.decodedFrameCount += framesRead;
70618 |             } else {
70619 |                 /* Failed to append the page. Just abort and set the status to MA_AT_END. */
70620 |                 > ma_paged_audio_buffer_data_free_page(&pDataBufferNode->data.backend.decodedPaged.data, pPage,
70621 | > &pResourceManager->config.allocationCallbacks);
70622 |                 result = MA_AT_END;
70623 |             }
70624 |         } else {
70625 |             /* No frames were read. Free the page and just set the status to MA_AT_END. */
70626 |             > ma_paged_audio_buffer_data_free_page(&pDataBufferNode->data.backend.decodedPaged.data, pPage,
70627 | > &pResourceManager->config.allocationCallbacks);
70628 |             result = MA_AT_END;
70629 |         }
70630 |     } break;
70631 |
70632 |     case ma_resource_manager_data_supply_type_encoded:
70633 |     case ma_resource_manager_data_supply_type_unknown:
70634 |     default:
70635 |     {
70636 |         /* Unexpected data supply type. */
70637 |         ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_ERROR,
70638 | > "Unexpected data supply type (%d) when decoding page.",
70639 | > ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBufferNode));
70640 |         return MA_ERROR;
70641 |     };
70642 | }
70643 |
70644 | if (result == MA_SUCCESS && framesRead == 0) {
70645 |     result = MA_AT_END;
70646 | }
70647 | return result;
70648 | }
70649 |
70650 | static ma_result ma_resource_manager_data_buffer_node_acquire_critical_section(ma_resource_manager*
70651 | > pResourceManager, const char* pFilePath, const wchar_t* pFilePathW, ma_uint32 hashedName32,
70652 | > ma_uint32 flags, const ma_resource_manager_data_supply* pExistingData, ma_fence* pInitFence,
70653 | > ma_fence* pDoneFence, ma_resource_manager_inline_notification* pInitNotification,
70654 | > ma_resource_manager_data_buffer_node** ppDataBufferNode)
70655 | {
70656 |     ma_result result = MA_SUCCESS;
70657 |     ma_resource_manager_data_buffer_node* pDataBufferNode = NULL;
70658 |     ma_resource_manager_data_buffer_node* pInsertPoint;
70659 |
70660 |     if (ppDataBufferNode != NULL) {
70661 |         *ppDataBufferNode = NULL;
70662 |     }
70663 |
70664 |     result = ma_resource_manager_data_buffer_node_insert_point(pResourceManager, hashedName32,
70665 | > &pInsertPoint);
70666 |     if (result == MA_ALREADY_EXISTS) {
70667 |         /* The node already exists. We just need to increment the reference count. */
70668 |         pDataBufferNode = pInsertPoint;
70669 |
70670 |         result = ma_resource_manager_data_buffer_node_increment_ref(pResourceManager,
70671 | > pDataBufferNode, NULL);
70672 |         if (result != MA_SUCCESS) {
70673 |             return result; /* Should never happen. Failed to increment the reference count. */
70674 |         }
70675 |     }

```

```

70664 |
70665 |         result = MA_ALREADY_EXISTS;
70666 |         goto done;
70667 |     } else {
70668 |         /*
70669 |         The node does not already exist. We need to post a LOAD_DATA_BUFFER_NODE job here. This
70670 |         needs to be done inside the critical section to ensure an uninitialization of the node
70671 |         does not occur before initialization on another thread.
70672 |         */
70673 |         pDataBufferNode = (ma_resource_manager_data_buffer_node*)ma_malloc(sizeof(*pDataBufferNode),
> &pResourceManager->config.allocationCallbacks);
70674 |         if (pDataBufferNode == NULL) {
70675 |             return MA_OUT_OF_MEMORY;
70676 |         }
70677 |
70678 |         MA_ZERO_OBJECT(pDataBufferNode);
70679 |         pDataBufferNode->hashedName32 = hashedName32;
70680 |         pDataBufferNode->refCount = 1; /* Always set to 1 by default (this is our first
> reference). */
70681 |
70682 |         if (pExistingData == NULL) {
70683 |             pDataBufferNode->data.type = ma_resource_manager_data_supply_type_unknown; /* <--
> We won't know this until we start decoding. */
70684 |             pDataBufferNode->result = MA_BUSY; /* Must be set to MA_BUSY before we leave the
> critical section, so might as well do it now. */
70685 |             pDataBufferNode->isDataOwnedByResourceManager = MA_TRUE;
70686 |         } else {
70687 |             pDataBufferNode->data = *pExistingData;
70688 |             pDataBufferNode->result = MA_SUCCESS; /* Not loading asynchronously, so just set
> the status */
70689 |             pDataBufferNode->isDataOwnedByResourceManager = MA_FALSE;
70690 |         }
70691 |
70692 |         result = ma_resource_manager_data_buffer_node_insert_at(pResourceManager, pDataBufferNode,
> pInsertPoint);
70693 |         if (result != MA_SUCCESS) {
70694 |             ma_free(pDataBufferNode, &pResourceManager->config.allocationCallbacks);
70695 |             return result; /* Should never happen. Failed to insert the data buffer into the BST.
> */
70696 |         }
70697 |
70698 |         /*
70699 |         Here is where we'll post the job, but only if we're loading asynchronously. If we're
70700 |         loading synchronously we'll defer loading to a later stage, outside of the critical
70701 |         section.
70702 |         */
70703 |         if (pDataBufferNode->isDataOwnedByResourceManager && (flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC) != 0) {
70704 |             /* Loading asynchronously. Post the job. */
70705 |             ma_job job;
70706 |             char* pFilePathCopy = NULL;
70707 |             wchar_t* pFilePathWCopy = NULL;
70708 |
70709 |             /* We need a copy of the file path. We should probably make this more efficient, but for
> now we'll do a transient memory allocation. */
70710 |             if (pFilePath != NULL) {
70711 |                 pFilePathCopy = ma_copy_string(pFilePath,
> &pResourceManager->config.allocationCallbacks);
70712 |             } else {
70713 |                 pFilePathWCopy = ma_copy_string_w(pFilePathW,
> &pResourceManager->config.allocationCallbacks);
70714 |             }
70715 |
70716 |             if (pFilePathCopy == NULL && pFilePathWCopy == NULL) {
70717 |                 ma_resource_manager_data_buffer_node_remove(pResourceManager, pDataBufferNode);
70718 |                 ma_free(pDataBufferNode, &pResourceManager->config.allocationCallbacks);
70719 |                 return MA_OUT_OF_MEMORY;
70720 |             }
70721 |
70722 |             if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
70723 |                 ma_resource_manager_inline_notification_init(pResourceManager, pInitNotification);
70724 |             }
70725 |
70726 |             /* Acquire init and done fences before posting the job. These will be unacquired by the
> job thread. */
70727 |             if (pInitFence != NULL) { ma_fence_acquire(pInitFence); }
70728 |             if (pDoneFence != NULL) { ma_fence_acquire(pDoneFence); }
70729 |
70730 |             /* We now have everything we need to post the job to the job thread. */

```

```

70731 |         job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER_NODE);
70732 |         job.order = ma_resource_manager_data_buffer_node_next_execution_order(pDataBufferNode);
70733 |         job.data.resourceManager.loadDataBufferNode.pResourceManager = pResourceManager;
70734 |         job.data.resourceManager.loadDataBufferNode.pDataBufferNode = pDataBufferNode;
70735 |         job.data.resourceManager.loadDataBufferNode.pFilePath = pFilePathCopy;
70736 |         job.data.resourceManager.loadDataBufferNode.pFilePathW = pFilePathWCopy;
70737 |         job.data.resourceManager.loadDataBufferNode.flags = flags;
70738 |         job.data.resourceManager.loadDataBufferNode.pInitNotification = ((flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) ? pInitNotification : NULL;
70739 |         job.data.resourceManager.loadDataBufferNode.pDoneNotification = NULL;
70740 |         job.data.resourceManager.loadDataBufferNode.pInitFence = pInitFence;
70741 |         job.data.resourceManager.loadDataBufferNode.pDoneFence = pDoneFence;
70742 |
70743 |         if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
70744 |             result = ma_job_process(&job);
70745 |         } else {
70746 |             result = ma_resource_manager_post_job(pResourceManager, &job);
70747 |         }
70748 |
70749 |         if (result != MA_SUCCESS) {
70750 |             /* Failed to post job. Probably ran out of memory. */
70751 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_ERROR,
> "Failed to post MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER_NODE job. %s.\n",
> ma_result_description(result));
70752 |
70753 |             if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
70754 |                 ma_resource_manager_inline_notification_uninit(pInitNotification);
70755 |             } else {
70756 |                 /*
70757 |                  Fences were acquired before posting the job, but since the job was not able to
70758 |                  be posted, we need to make sure we release them so nothing gets stuck waiting.
70759 |
70760 |                  In the WAIT_INIT case, these will have already been released in ma_job_process()
70761 |                  so we should only release fences in this branch.
70762 |                  */
70763 |                 if (pInitFence != NULL) { ma_fence_release(pInitFence); }
70764 |                 if (pDoneFence != NULL) { ma_fence_release(pDoneFence); }
70765 |
70766 |                 /* These will have been freed by the job thread, but with WAIT_INIT they will
> already have happened since the job has already been handled. */
70767 |                 ma_free(pFilePathCopy, &pResourceManager->config.allocationCallbacks);
70768 |                 ma_free(pFilePathWCopy, &pResourceManager->config.allocationCallbacks);
70769 |             }
70770 |
70771 |             ma_resource_manager_data_buffer_node_remove(pResourceManager, pDataBufferNode);
70772 |             ma_free(pDataBufferNode, &pResourceManager->config.allocationCallbacks);
70773 |
70774 |             return result;
70775 |         }
70776 |     }
70777 | }
70778 |
70779 | done:
70780 |     if (ppDataBufferNode != NULL) {
70781 |         *ppDataBufferNode = pDataBufferNode;
70782 |     }
70783 |
70784 |     return result;
70785 | }
70786 |
70787 | static ma_result ma_resource_manager_data_buffer_node_acquire(ma_resource_manager* pResourceManager,
> const char* pFilePath, const wchar_t* pFilePathW, ma_uint32 hashedName32, ma_uint32 flags, const
> ma_resource_manager_data_supply* pExistingData, ma_fence* pInitFence, ma_fence* pDoneFence,
> ma_resource_manager_data_buffer_node** ppDataBufferNode)
70788 | {
70789 |     ma_result result = MA_SUCCESS;
70790 |     ma_bool32 nodeAlreadyExists = MA_FALSE;
70791 |     ma_resource_manager_data_buffer_node* pDataBufferNode = NULL;
70792 |     ma_resource_manager_inline_notification initNotification; /* Used when the WAIT_INIT flag is
> set. */
70793 |
70794 |     if (ppDataBufferNode != NULL) {
70795 |         *ppDataBufferNode = NULL; /* Safety. */
70796 |     }
70797 |
70798 |     if (pResourceManager == NULL || (pFilePath == NULL && pFilePathW == NULL && hashedName32 == 0))
> {
70799 |         return MA_INVALID_ARGS;
70800 |     }

```

```

70801 |
70802 |     /* If we're specifying existing data, it must be valid. */
70803 |     if (pExistingData != NULL && pExistingData->type ==
> ma_resource_manager_data_supply_type_unknown) {
70804 |         return MA_INVALID_ARGS;
70805 |     }
70806 |
70807 |     /* If we don't support threading, remove the ASYNC flag to make the rest of this a bit simpler.
> */
70808 |     if (ma_resource_manager_is_threading_enabled(pResourceManager) == MA_FALSE) {
70809 |         flags &= ~MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC;
70810 |     }
70811 |
70812 |     if (hashedName32 == 0) {
70813 |         if (pFilePath != NULL) {
70814 |             hashedName32 = ma_hash_string_32(pFilePath);
70815 |         } else {
70816 |             hashedName32 = ma_hash_string_w_32(pFilePathW);
70817 |         }
70818 |     }
70819 |
70820 |     /*
70821 |     Here is where we either increment the node's reference count or allocate a new one and add it
70822 |     to the BST. When allocating a new node, we need to make sure the LOAD_DATA_BUFFER_NODE job is
70823 |     posted inside the critical section just in case the caller immediately uninitializes the node
70824 |     as this will ensure the FREE_DATA_BUFFER_NODE job is given an execution order such that the
70825 |     node is not uninitialized before initialization.
70826 |     */
70827 |     ma_resource_manager_data_buffer_bst_lock(pResourceManager);
70828 |     {
70829 |         result = ma_resource_manager_data_buffer_node_acquire_critical_section(pResourceManager,
> pFilePath, pFilePathW, hashedName32, flags, pExistingData, pInitFence, pDoneFence,
> &initNotification, &pDataBufferNode);
70830 |     }
70831 |     ma_resource_manager_data_buffer_bst_unlock(pResourceManager);
70832 |
70833 |     if (result == MA_ALREADY_EXISTS) {
70834 |         nodeAlreadyExists = MA_TRUE;
70835 |         result = MA_SUCCESS;
70836 |     } else {
70837 |         if (result != MA_SUCCESS) {
70838 |             return result;
70839 |         }
70840 |     }
70841 |
70842 |     /*
70843 |     If we're loading synchronously, we'll need to load everything now. When loading asynchronously,
70844 |     a job will have been posted inside the BST critical section so that an uninitialization can be
70845 |     allocated an appropriate execution order thereby preventing it from being uninitialized before
70846 |     the node is initialized by the decoding thread(s).
70847 |     */
70848 |     if (nodeAlreadyExists == MA_FALSE) { /* Don't need to try loading anything if the node
> already exists. */
70849 |         if (pFilePath == NULL && pFilePathW == NULL) {
70850 |             /*
70851 |             If this path is hit, it means a buffer is being copied (i.e. initialized from only the
70852 |             hashed name), but that node has been freed in the meantime, probably from some other
70853 |             thread. This is an invalid operation.
70854 |             */
70855 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_WARNING,
> "Cloning data buffer node failed because the source node was released. The source node must remain
> valid until the cloning has completed.\n");
70856 |             result = MA_INVALID_OPERATION;
70857 |             goto done;
70858 |         }
70859 |
70860 |         if (pDataBufferNode->isDataOwnedByResourceManager) {
70861 |             if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC) == 0) {
70862 |                 /* Loading synchronously. Load the sound in it's entirety here. */
70863 |                 if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE) == 0) {
70864 |                     /* No decoding. This is the simple case - just store the file contents in
> memory. */
70865 |                     result =
> ma_resource_manager_data_buffer_node_init_supply_encoded(pResourceManager, pDataBufferNode,
> pFilePath, pFilePathW);
70866 |                     if (result != MA_SUCCESS) {
70867 |                         goto done;
70868 |                     }
70869 |                 } else {

```

```

70870 |             /* Decoding. We do this the same way as we do when loading asynchronously. */
70871 |             ma_decoder* pDecoder;
70872 |             result =
> ma_resource_manager_data_buffer_node_init_supply_decoded(pResourceManager, pDataBufferNode,
> pFilePath, pFilePathW, flags, &pDecoder);
70873 |             if (result != MA_SUCCESS) {
70874 |                 goto done;
70875 |             }
70876 |
70877 |             /* We have the decoder, now decode page by page just like we do when loading
> asynchronously. */
70878 |             for (;;) {
70879 |                 /* Decode next page. */
70880 |                 result =
> ma_resource_manager_data_buffer_node_decode_next_page(pResourceManager, pDataBufferNode, pDecoder);
70881 |                 if (result != MA_SUCCESS) {
70882 |                     break; /* Will return MA_AT_END when the last page has been decoded. */
70883 |                 }
70884 |             }
70885 |
70886 |             /* Reaching the end needs to be considered successful. */
70887 |             if (result == MA_AT_END) {
70888 |                 result = MA_SUCCESS;
70889 |             }
70890 |
70891 |             /*
70892 |             At this point the data buffer is either fully decoded or some error occurred.
> Either
70893 |             way, the decoder is no longer necessary.
70894 |             */
70895 |             ma_decoder_uninit(pDecoder);
70896 |             ma_free(pDecoder, &pResourceManager->config.allocationCallbacks);
70897 |         }
70898 |
70899 |         /* Getting here means we were successful. Make sure the status of the node is
> updated accordingly. */
70900 |         ma_atomic_exchange_i32(&pDataBufferNode->result, result);
70901 |     } else {
70902 |         /* Loading asynchronously. We may need to wait for initialization. */
70903 |         if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
70904 |             ma_resource_manager_inline_notification_wait(&initNotification);
70905 |         }
70906 |     }
70907 | } else {
70908 |     /* The data is not managed by the resource manager so there's nothing else to do. */
70909 |     MA_ASSERT(pExistingData != NULL);
70910 | }
70911 | }
70912 |
70913 | done:
70914 |     /* If we failed to initialize the data buffer we need to free it. */
70915 |     if (result != MA_SUCCESS) {
70916 |         if (nodeAlreadyExists == MA_FALSE) {
70917 |             ma_resource_manager_data_buffer_node_remove(pResourceManager, pDataBufferNode);
70918 |             ma_free(pDataBufferNode, &pResourceManager->config.allocationCallbacks);
70919 |         }
70920 |     }
70921 |
70922 |     /*
70923 |     The init notification needs to be uninitialized. This will be used if the node does not already
70924 |     exist, and we've specified ASYNC | WAIT_INIT.
70925 |     */
70926 |     if (nodeAlreadyExists == MA_FALSE && pDataBufferNode->isDataOwnedByResourceManager && (flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC) != 0) {
70927 |         if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
70928 |             ma_resource_manager_inline_notification_uninit(&initNotification);
70929 |         }
70930 |     }
70931 |
70932 |     if (ppDataBufferNode != NULL) {
70933 |         *ppDataBufferNode = pDataBufferNode;
70934 |     }
70935 |
70936 |     return result;
70937 | }
70938 |
70939 | static ma_result ma_resource_manager_data_buffer_node_unacquire(ma_resource_manager*
> pResourceManager, ma_resource_manager_data_buffer_node* pDataBufferNode, const char* pName, const
> wchar_t* pNameW)

```

```

70940 | {
70941 |     ma_result result = MA_SUCCESS;
70942 |     ma_uint32 refCount = 0xFFFFFFFF; /* The new reference count of the node after decrementing.
> Initialize to non-0 to be safe we don't fall into the freeing path. */
70943 |     ma_uint32 hashedName32 = 0;
70944 |
70945 |     if (pResourceManager == NULL) {
70946 |         return MA_INVALID_ARGS;
70947 |     }
70948 |
70949 |     if (pDataBufferNode == NULL) {
70950 |         if (pName == NULL && pNameW == NULL) {
70951 |             return MA_INVALID_ARGS;
70952 |         }
70953 |
70954 |         if (pName != NULL) {
70955 |             hashedName32 = ma_hash_string_32(pName);
70956 |         } else {
70957 |             hashedName32 = ma_hash_string_w_32(pNameW);
70958 |         }
70959 |     }
70960 |
70961 |     /*
70962 |     The first thing to do is decrement the reference counter of the node. Then, if the reference
70963 |     count is zero, we need to free the node. If the node is still in the process of loading, we'll
70964 |     need to post a job to the job queue to free the node. Otherwise we'll just do it here.
70965 |     */
70966 |     ma_resource_manager_data_buffer_bst_lock(pResourceManager);
70967 |     {
70968 |         /* Might need to find the node. Must be done inside the critical section. */
70969 |         if (pDataBufferNode == NULL) {
70970 |             result = ma_resource_manager_data_buffer_node_search(pResourceManager, hashedName32,
> &pDataBufferNode);
70971 |             if (result != MA_SUCCESS) {
70972 |                 goto stage2; /* Couldn't find the node. */
70973 |             }
70974 |         }
70975 |
70976 |         result = ma_resource_manager_data_buffer_node_decrement_ref(pResourceManager,
> pDataBufferNode, &refCount);
70977 |         if (result != MA_SUCCESS) {
70978 |             goto stage2; /* Should never happen. */
70979 |         }
70980 |
70981 |         if (refCount == 0) {
70982 |             result = ma_resource_manager_data_buffer_node_remove(pResourceManager, pDataBufferNode);
70983 |             if (result != MA_SUCCESS) {
70984 |                 goto stage2; /* An error occurred when trying to remove the data buffer. This
> should never happen. */
70985 |             }
70986 |         }
70987 |     }
70988 |     ma_resource_manager_data_buffer_bst_unlock(pResourceManager);
70989 |
70990 | stage2:
70991 |     if (result != MA_SUCCESS) {
70992 |         return result;
70993 |     }
70994 |
70995 |     /*
70996 |     Here is where we need to free the node. We don't want to do this inside the critical section
70997 |     above because we want to keep that as small as possible for multi-threaded efficiency.
70998 |     */
70999 |     if (refCount == 0) {
71000 |         if (ma_resource_manager_data_buffer_node_result(pDataBufferNode) == MA_BUSY) {
71001 |             /* The sound is still loading. We need to delay the freeing of the node to a safe time.
> */
71002 |             ma_job job;
71003 |
71004 |             /* We need to mark the node as unavailable for the sake of the resource manager worker
> threads. */
71005 |             ma_atomic_exchange_i32(&pDataBufferNode->result, MA_UNAVAILABLE);
71006 |
71007 |             job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER_NODE);
71008 |             job.order = ma_resource_manager_data_buffer_node_next_execution_order(pDataBufferNode);
71009 |             job.data.resourceManager.freeDataBufferNode.pResourceManager = pResourceManager;
71010 |             job.data.resourceManager.freeDataBufferNode.pDataBufferNode = pDataBufferNode;
71011 |
71012 |             result = ma_resource_manager_post_job(pResourceManager, &job);

```



```

71013 |         if (result != MA_SUCCESS) {
71014 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_ERROR,
> "Failed to post MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER_NODE job. %s.\n",
> ma_result_description(result));
71015 |             return result;
71016 |         }
71017 |
71018 |         /* If we don't support threading, process the job queue here. */
71019 |         if (ma_resource_manager_is_threading_enabled(pResourceManager) == MA_FALSE) {
71020 |             while (ma_resource_manager_data_buffer_node_result(pDataBufferNode) == MA_BUSY) {
71021 |                 result = ma_resource_manager_process_next_job(pResourceManager);
71022 |                 if (result == MA_NO_DATA_AVAILABLE || result == MA_CANCELLED) {
71023 |                     result = MA_SUCCESS;
71024 |                     break;
71025 |                 }
71026 |             }
71027 |         } else {
71028 |             /* Threading is enabled. The job queue will deal with the rest of the cleanup from
> here. */
71029 |         }
71030 |     } else {
71031 |         /* The sound isn't loading so we can just free the node here. */
71032 |         ma_resource_manager_data_buffer_node_free(pResourceManager, pDataBufferNode);
71033 |     }
71034 | }
71035 |
71036 | return result;
71037 | }
71038 |
71039 |
71040 |
71041 | static ma_uint32
> ma_resource_manager_data_buffer_next_execution_order(ma_resource_manager_data_buffer* pDataBuffer)
71042 | {
71043 |     MA_ASSERT(pDataBuffer != NULL);
71044 |     return ma_atomic_fetch_add_32(&pDataBuffer->executionCounter, 1);
71045 | }
71046 |
71047 | static ma_result ma_resource_manager_data_buffer_cb__read_pcm_frames(ma_data_source* pDataSource,
> void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
71048 | {
71049 |     return
> ma_resource_manager_data_buffer_read_pcm_frames((ma_resource_manager_data_buffer*)pDataSource,
> pFramesOut, frameCount, pFramesRead);
71050 | }
71051 |
71052 | static ma_result ma_resource_manager_data_buffer_cb__seek_to_pcm_frame(ma_data_source* pDataSource,
> ma_uint64 frameIndex)
71053 | {
71054 |     return
> ma_resource_manager_data_buffer_seek_to_pcm_frame((ma_resource_manager_data_buffer*)pDataSource,
> frameIndex);
71055 | }
71056 |
71057 | static ma_result ma_resource_manager_data_buffer_cb__get_data_format(ma_data_source* pDataSource,
> ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
71058 | {
71059 |     return
> ma_resource_manager_data_buffer_get_data_format((ma_resource_manager_data_buffer*)pDataSource,
> pFormat, pChannels, pSampleRate, pChannelMap, channelMapCap);
71060 | }
71061 |
71062 | static ma_result ma_resource_manager_data_buffer_cb__get_cursor_in_pcm_frames(ma_data_source*
> pDataSource, ma_uint64* pCursor)
71063 | {
71064 |     return ma_resource_manager_data_buffer_get_cursor_in_pcm_frames((ma_resource_manager_data_buffer
> *)pDataSource, pCursor);
71065 | }
71066 |
71067 | static ma_result ma_resource_manager_data_buffer_cb__get_length_in_pcm_frames(ma_data_source*
> pDataSource, ma_uint64* pLength)
71068 | {
71069 |     return ma_resource_manager_data_buffer_get_length_in_pcm_frames((ma_resource_manager_data_buffer
> *)pDataSource, pLength);
71070 | }
71071 |
71072 | static ma_result ma_resource_manager_data_buffer_cb__set_looping(ma_data_source* pDataSource,
> ma_bool32 isLooping)

```

```

71073 | {
71074 |     ma_resource_manager_data_buffer* pDataBuffer = (ma_resource_manager_data_buffer*)pDataSource;
71075 |     MA_ASSERT(pDataBuffer != NULL);
71076 |
71077 |     ma_atomic_exchange_32(&pDataBuffer->isLooping, isLooping);
71078 |
71079 |     /* The looping state needs to be set on the connector as well or else looping won't work when we
> read audio data. */
71080 |     ma_data_source_set_looping(ma_resource_manager_data_buffer_get_connector(pDataBuffer),
> isLooping);
71081 |
71082 |     return MA_SUCCESS;
71083 | }
71084 |
71085 | static ma_data_source_vtable g_ma_resource_manager_data_buffer_vtable =
71086 | {
71087 |     ma_resource_manager_data_buffer_cb__read_pcm_frames,
71088 |     ma_resource_manager_data_buffer_cb__seek_to_pcm_frame,
71089 |     ma_resource_manager_data_buffer_cb__get_data_format,
71090 |     ma_resource_manager_data_buffer_cb__get_cursor_in_pcm_frames,
71091 |     ma_resource_manager_data_buffer_cb__get_length_in_pcm_frames,
71092 |     ma_resource_manager_data_buffer_cb__set_looping,
71093 |     0
71094 | };
71095 |
71096 | static ma_result ma_resource_manager_data_buffer_init_ex_internal(ma_resource_manager*
> pResourceManager, const ma_resource_manager_data_source_config* pConfig, ma_uint32 hashedName32,
> ma_resource_manager_data_buffer* pDataBuffer)
71097 | {
71098 |     ma_result result = MA_SUCCESS;
71099 |     ma_resource_manager_data_buffer_node* pDataBufferNode;
71100 |     ma_data_source_config dataSourceConfig;
71101 |     ma_bool32 async;
71102 |     ma_uint32 flags;
71103 |     ma_resource_manager_pipeline_notifications notifications;
71104 |
71105 |     if (pDataBuffer == NULL) {
71106 |         if (pConfig != NULL && pConfig->pNotifications != NULL) {
71107 |
> ma_resource_manager_pipeline_notifications_signal_all_notifications(pConfig->pNotifications);
71108 |         }
71109 |
71110 |         return MA_INVALID_ARGS;
71111 |     }
71112 |
71113 |     MA_ZERO_OBJECT(pDataBuffer);
71114 |
71115 |     if (pConfig == NULL) {
71116 |         return MA_INVALID_ARGS;
71117 |     }
71118 |
71119 |     if (pConfig->pNotifications != NULL) {
71120 |         notifications = *pConfig->pNotifications; /* From here on out we should be referencing
> `notifications` instead of `pNotifications`. Set this to NULL to catch errors at testing time. */
71121 |     } else {
71122 |         MA_ZERO_OBJECT(&notifications);
71123 |     }
71124 |
71125 |     /* For safety, always remove the ASYNC flag if threading is disabled on the resource manager. */
71126 |     flags = pConfig->flags;
71127 |     if (ma_resource_manager_is_threading_enabled(pResourceManager) == MA_FALSE) {
71128 |         flags &= ~MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC;
71129 |     }
71130 |
71131 |     if (pConfig->isLooping) {
71132 |         flags |= MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING;
71133 |     }
71134 |
71135 |     async = (flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC) != 0;
71136 |
71137 |     /*
71138 |     Fences need to be acquired before doing anything. These must be acquired and released outside of
71139 |     the node to ensure there's no holes where ma_fence_wait() could prematurely return before the
71140 |     data buffer has completed initialization.
71141 |
71142 |     When loading asynchronously, the node acquisition routine below will acquire the fences on this
71143 |     thread and then release them on the async thread when the operation is complete.
71144 |
71145 |     These fences are always released at the "done" tag at the end of this function. They'll be

```

```

71146 |     acquired a second if loading asynchronously. This double acquisition system is just done to
71147 |     simplify code maintenance.
71148 |     */
71149 |     ma_resource_manager_pipeline_notifications_acquire_all_fences(&notifications);
71150 |     {
71151 |         /* We first need to acquire a node. If ASYNC is not set, this will not return until the
> entire sound has been loaded. */
71152 |         result = ma_resource_manager_data_buffer_node_acquire(pResourceManager, pConfig->pFilePath,
> pConfig->pFilePathW, hashedName32, flags, NULL, notifications.init.pFence,
> notifications.done.pFence, &pDataBufferNode);
71153 |         if (result != MA_SUCCESS) {
71154 |             ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71155 |             goto done;
71156 |         }
71157 |
71158 |         dataSourceConfig = ma_data_source_config_init();
71159 |         dataSourceConfig.vtable = &g_ma_resource_manager_data_buffer_vtable;
71160 |
71161 |         result = ma_data_source_init(&dataSourceConfig, &pDataBuffer->ds);
71162 |         if (result != MA_SUCCESS) {
71163 |             ma_resource_manager_data_buffer_node_unacquire(pResourceManager, pDataBufferNode, NULL,
> NULL);
71164 |             ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71165 |             goto done;
71166 |         }
71167 |
71168 |         pDataBuffer->pResourceManager = pResourceManager;
71169 |         pDataBuffer->pNode = pDataBufferNode;
71170 |         pDataBuffer->flags = flags;
71171 |         pDataBuffer->result = MA_BUSY; /* Always default to MA_BUSY for safety. It'll be
> overwritten when loading completes or an error occurs. */
71172 |
71173 |         /* If we're loading asynchronously we need to post a job to the job queue to initialize the
> connector. */
71174 |         if (async == MA_FALSE || ma_resource_manager_data_buffer_node_result(pDataBufferNode) ==
> MA_SUCCESS) {
71175 |             /* Loading synchronously or the data has already been fully loaded. We can just
> initialize the connector from here without a job. */
71176 |             result = ma_resource_manager_data_buffer_init_connector(pDataBuffer, pConfig, NULL,
> NULL);
71177 |             ma_atomic_exchange_i32(&pDataBuffer->result, result);
71178 |
71179 |             ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71180 |             goto done;
71181 |         } else {
71182 |             /* The node's data supply isn't initialized yet. The caller has requested that we load
> asynchronously so we need to post a job to do this. */
71183 |             ma_job job;
71184 |             ma_resource_manager_inline_notification initNotification; /* Used when the WAIT_INIT
> flag is set. */
71185 |
71186 |             if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
71187 |                 ma_resource_manager_inline_notification_init(pResourceManager, &initNotification);
71188 |             }
71189 |
71190 |             /*
71191 |             The status of the data buffer needs to be set to MA_BUSY before posting the job so that
> the
71192 |             worker thread is aware of its busy state. If the LOAD_DATA_BUFFER job sees a status
> other
71193 |             than MA_BUSY, it'll assume an error and fall through to an early exit.
71194 |             */
71195 |             ma_atomic_exchange_i32(&pDataBuffer->result, MA_BUSY);
71196 |
71197 |             /* Acquire fences a second time. These will be released by the async thread. */
71198 |             ma_resource_manager_pipeline_notifications_acquire_all_fences(&notifications);
71199 |
71200 |             job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER);
71201 |             job.order = ma_resource_manager_data_buffer_next_execution_order(pDataBuffer);
71202 |             job.data.resourceManager.loadDataBuffer.pDataBuffer = pDataBuffer;
71203 |             job.data.resourceManager.loadDataBuffer.pInitNotification = ((flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) ? &initNotification :
> notifications.init.pNotification;
71204 |             job.data.resourceManager.loadDataBuffer.pDoneNotification =
> notifications.done.pNotification;
71205 |             job.data.resourceManager.loadDataBuffer.pInitFence =
> notifications.init.pFence;
71206 |             job.data.resourceManager.loadDataBuffer.pDoneFence =
> notifications.done.pFence;

```

```

71207 |         job.data.resourceManager.loadDataBuffer.rangeBegInPCMFrames    =
> pConfig->rangeBegInPCMFrames;
71208 |         job.data.resourceManager.loadDataBuffer.rangeEndInPCMFrames    =
> pConfig->rangeEndInPCMFrames;
71209 |         job.data.resourceManager.loadDataBuffer.loopPointBegInPCMFrames =
> pConfig->loopPointBegInPCMFrames;
71210 |         job.data.resourceManager.loadDataBuffer.loopPointEndInPCMFrames =
> pConfig->loopPointEndInPCMFrames;
71211 |         job.data.resourceManager.loadDataBuffer.isLooping              = (flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING) != 0;
71212 |
71213 |         /* If we need to wait for initialization to complete we can just process the job in
> place. */
71214 |         if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
71215 |             result = ma_job_process(&job);
71216 |         } else {
71217 |             result = ma_resource_manager_post_job(pResourceManager, &job);
71218 |         }
71219 |
71220 |         if (result != MA_SUCCESS) {
71221 |             /* We failed to post the job. Most likely there isn't enough room in the queue's
> buffer. */
71222 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_ERROR,
> "Failed to post MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_BUFFER job. %s.\n",
> ma_result_description(result));
71223 |             ma_atomic_exchange_i32(&pDataBuffer->result, result);
71224 |
71225 |             /* Release the fences after the result has been set on the data buffer. */
71226 |             ma_resource_manager_pipeline_notifications_release_all_fences(&notifications);
71227 |         } else {
71228 |             if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
71229 |                 ma_resource_manager_inline_notification_wait(&initNotification);
71230 |
71231 |                 if (notifications.init.pNotification != NULL) {
71232 |                     ma_async_notification_signal(notifications.init.pNotification);
71233 |                 }
71234 |
71235 |                 /* NOTE: Do not release the init fence here. It will have been done by the job.
> */
71236 |
71237 |                 /* Make sure we return an error if initialization failed on the async thread. */
71238 |                 result = ma_resource_manager_data_buffer_result(pDataBuffer);
71239 |                 if (result == MA_BUSY) {
71240 |                     result = MA_SUCCESS;
71241 |                 }
71242 |             }
71243 |         }
71244 |
71245 |         if ((flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
71246 |             ma_resource_manager_inline_notification_uninit(&initNotification);
71247 |         }
71248 |     }
71249 |
71250 |     if (result != MA_SUCCESS) {
71251 |         ma_resource_manager_data_buffer_node_unacquire(pResourceManager, pDataBufferNode, NULL,
> NULL);
71252 |         goto done;
71253 |     }
71254 | }
71255 | done:
71256 |     if (result == MA_SUCCESS) {
71257 |         if (pConfig->initialSeekPointInPCMFrames > 0) {
71258 |             ma_resource_manager_data_buffer_seek_to_pcm_frame(pDataBuffer,
> pConfig->initialSeekPointInPCMFrames);
71259 |         }
71260 |     }
71261 |
71262 |     ma_resource_manager_pipeline_notifications_release_all_fences(&notifications);
71263 |
71264 |     return result;
71265 | }
71266 |
71267 | MA_API ma_result ma_resource_manager_data_buffer_init_ex(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_buffer* pDataBuffer)
71268 | {
71269 |     return ma_resource_manager_data_buffer_init_ex_internal(pResourceManager, pConfig, 0,
> pDataBuffer);
71270 | }
71271 |

```

```

71272 | MA_API ma_result ma_resource_manager_data_buffer_init(ma_resource_manager* pResourceManager, const
> char* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
> ma_resource_manager_data_buffer* pDataBuffer)
71273 | {
71274 |     ma_resource_manager_data_source_config config;
71275 |
71276 |     config = ma_resource_manager_data_source_config_init();
71277 |     config.pFilePath      = pFilePath;
71278 |     config.flags          = flags;
71279 |     config.pNotifications = pNotifications;
71280 |
71281 |     return ma_resource_manager_data_buffer_init_ex(pResourceManager, &config, pDataBuffer);
71282 | }
71283 |
71284 | MA_API ma_result ma_resource_manager_data_buffer_init_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications*
> pNotifications, ma_resource_manager_data_buffer* pDataBuffer)
71285 | {
71286 |     ma_resource_manager_data_source_config config;
71287 |
71288 |     config = ma_resource_manager_data_source_config_init();
71289 |     config.pFilePathW    = pFilePath;
71290 |     config.flags         = flags;
71291 |     config.pNotifications = pNotifications;
71292 |
71293 |     return ma_resource_manager_data_buffer_init_ex(pResourceManager, &config, pDataBuffer);
71294 | }
71295 |
71296 | MA_API ma_result ma_resource_manager_data_buffer_init_copy(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_buffer* pExistingDataBuffer, ma_resource_manager_data_buffer*
> pDataBuffer)
71297 | {
71298 |     ma_resource_manager_data_source_config config;
71299 |
71300 |     if (pExistingDataBuffer == NULL) {
71301 |         return MA_INVALID_ARGS;
71302 |     }
71303 |
71304 |     MA_ASSERT(pExistingDataBuffer->pNode != NULL); /* <-- If you've triggered this, you've passed
> in an invalid existing data buffer. */
71305 |
71306 |     config = ma_resource_manager_data_source_config_init();
71307 |     config.flags = pExistingDataBuffer->flags;
71308 |
71309 |     return ma_resource_manager_data_buffer_init_ex_internal(pResourceManager, &config,
> pExistingDataBuffer->pNode->hashedName32, pDataBuffer);
71310 | }
71311 |
71312 | static ma_result ma_resource_manager_data_buffer_uninit_internal(ma_resource_manager_data_buffer*
> pDataBuffer)
71313 | {
71314 |     MA_ASSERT(pDataBuffer != NULL);
71315 |
71316 |     /* The connector should be uninitialized first. */
71317 |     ma_resource_manager_data_buffer_uninit_connector(pDataBuffer->pResourceManager, pDataBuffer);
71318 |
71319 |     /* With the connector uninitialized we can unacquire the node. */
71320 |     ma_resource_manager_data_buffer_node_unacquire(pDataBuffer->pResourceManager,
> pDataBuffer->pNode, NULL, NULL);
71321 |
71322 |     /* The base data source needs to be uninitialized as well. */
71323 |     ma_data_source_uninit(&pDataBuffer->ds);
71324 |
71325 |     return MA_SUCCESS;
71326 | }
71327 |
71328 | MA_API ma_result ma_resource_manager_data_buffer_uninit(ma_resource_manager_data_buffer*
> pDataBuffer)
71329 | {
71330 |     ma_result result;
71331 |
71332 |     if (pDataBuffer == NULL) {
71333 |         return MA_INVALID_ARGS;
71334 |     }
71335 |
71336 |     if (ma_resource_manager_data_buffer_result(pDataBuffer) == MA_SUCCESS) {
71337 |         /* The data buffer can be deleted synchronously. */
71338 |         return ma_resource_manager_data_buffer_uninit_internal(pDataBuffer);
71339 |     } else {

```

```

71340 |      /*
71341 |      The data buffer needs to be deleted asynchronously because it's still loading. With the
> status set to MA_UNAVAILABLE, no more pages will
71342 |      be loaded and the uninitialization should happen fairly quickly. Since the caller owns the
> data buffer, we need to wait for this event
71343 |      to get processed before returning.
71344 |      */
71345 |      ma_resource_manager_inline_notification notification;
71346 |      ma_job job;
71347 |
71348 |      /*
71349 |      We need to mark the node as unavailable so we don't try reading from it anymore, but also to
71350 |      let the loading thread know that it needs to abort it's loading procedure.
71351 |      */
71352 |      ma_atomic_exchange_i32(&pDataBuffer->result, MA_UNAVAILABLE);
71353 |
71354 |      result = ma_resource_manager_inline_notification_init(pDataBuffer->pResourceManager,
> &notification);
71355 |      if (result != MA_SUCCESS) {
71356 |          return result; /* Failed to create the notification. This should rarely, if ever,
> happen. */
71357 |      }
71358 |
71359 |      job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER);
71360 |      job.order = ma_resource_manager_data_buffer_next_execution_order(pDataBuffer);
71361 |      job.data.resourceManager.freeDataBuffer.pDataBuffer = pDataBuffer;
71362 |      job.data.resourceManager.freeDataBuffer.pDoneNotification = &notification;
71363 |      job.data.resourceManager.freeDataBuffer.pDoneFence = NULL;
71364 |
71365 |      result = ma_resource_manager_post_job(pDataBuffer->pResourceManager, &job);
71366 |      if (result != MA_SUCCESS) {
71367 |          ma_resource_manager_inline_notification_uninit(&notification);
71368 |          return result;
71369 |      }
71370 |
71371 |      ma_resource_manager_inline_notification_wait_and_uninit(&notification);
71372 |  }
71373 |
71374 |  return result;
71375 | }
71376 |
71377 | MA_API ma_result ma_resource_manager_data_buffer_read_pcm_frames(ma_resource_manager_data_buffer*
> pDataBuffer, void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
71378 | {
71379 |     ma_result result = MA_SUCCESS;
71380 |     ma_uint64 framesRead = 0;
71381 |     ma_bool32 isDecodedBufferBusy = MA_FALSE;
71382 |
71383 |     /* Safety. */
71384 |     if (pFramesRead != NULL) {
71385 |         *pFramesRead = 0;
71386 |     }
71387 |
71388 |     if (frameCount == 0) {
71389 |         return MA_INVALID_ARGS;
71390 |     }
71391 |
71392 |     /*
71393 |     We cannot be using the data buffer after it's been uninitialized. If you trigger this assert it
> means you're trying to read from the data buffer after
71394 |     it's been uninitialized or is in the process of uninitializing.
71395 |     */
71396 |     MA_ASSERT(ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) != MA_UNAVAILABLE);
71397 |
71398 |     /* If the node is not initialized we need to abort with a busy code. */
71399 |     if (ma_resource_manager_data_buffer_has_connector(pDataBuffer) == MA_FALSE) {
71400 |         return MA_BUSY; /* Still loading. */
71401 |     }
71402 |
71403 |     /*
71404 |     If we've got a seek scheduled we'll want to do that before reading. However, for paged buffers,
> there's
71405 |     a chance that the sound hasn't yet been decoded up to the seek point will result in the seek
> failing. If
71406 |     this happens, we need to keep the seek scheduled and return MA_BUSY.
71407 |     */
71408 |     if (pDataBuffer->seekToCursorOnNextRead) {
71409 |         pDataBuffer->seekToCursorOnNextRead = MA_FALSE;
71410 |     }

```

```

71411 |         result =
> ma_data_source_seek_to_pcm_frame(ma_resource_manager_data_buffer_get_connector(pDataBuffer),
> pDataBuffer->seekTargetInPCMFrames);
71412 |         if (result != MA_SUCCESS) {
71413 |             if (result == MA_BAD_SEEK &&
> ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode) ==
> ma_resource_manager_data_supply_type_decoded_paged) {
71414 |                 pDataBuffer->seekToCursorOnNextRead = MA_TRUE; /* Keep the seek scheduled. We just
> haven't loaded enough data yet to do the seek properly. */
71415 |                 return MA_BUSY;
71416 |             }
71417 |
71418 |             return result;
71419 |         }
71420 |     }
71421 |
71422 |     /*
71423 |     For decoded buffers (not paged) we need to check beforehand how many frames we have available.
> We cannot
71424 |     exceed this amount. We'll read as much as we can, and then return MA_BUSY.
71425 |     */
71426 |     if (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode) ==
> ma_resource_manager_data_supply_type_decoded) {
71427 |         ma_uint64 availableFrames;
71428 |
71429 |         isDecodedBufferBusy = (ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) ==
> MA_BUSY);
71430 |
71431 |         if (ma_resource_manager_data_buffer_get_available_frames(pDataBuffer, &availableFrames) ==
> MA_SUCCESS) {
71432 |             /* Don't try reading more than the available frame count if the data buffer node is
> still loading. */
71433 |             if (isDecodedBufferBusy) {
71434 |                 if (frameCount > availableFrames) {
71435 |                     frameCount = availableFrames;
71436 |                 }
71437 |                 /*
71438 |                 If there's no frames available we want to set the status to MA_AT_END. The logic
> below
71439 |                 will check if the node is busy, and if so, change it to MA_BUSY. The reason we
> do this
71440 |                 is because we don't want to call `ma_data_source_read_pcm_frames()` if the frame
> count
71441 |                 is 0 because that'll result in a situation where it's possible MA_AT_END won't
> get
71442 |                 returned.
71443 |                 */
71444 |                 if (frameCount == 0) {
71445 |                     result = MA_AT_END;
71446 |                 }
71447 |             } else {
71448 |                 isDecodedBufferBusy = MA_FALSE; /* We have enough frames available in the buffer
> to avoid a MA_BUSY status. */
71449 |             }
71450 |             } else {
71451 |                 /*
71452 |                 Getting here means the buffer has been fully loaded. We can just pass the frame
> count straight
71453 |                 into ma_data_source_read_pcm_frames() below and let ma_data_source handle it.
71454 |                 */
71455 |             }
71456 |         }
71457 |     }
71458 |
71459 |     /* Don't attempt to read anything if we've got no frames available. */
71460 |     if (frameCount > 0) {
71461 |         result =
> ma_data_source_read_pcm_frames(ma_resource_manager_data_buffer_get_connector(pDataBuffer),
> pFramesOut, frameCount, &framesRead);
71462 |     }
71463 |
71464 |     /*
71465 |     If we returned MA_AT_END, but the node is still loading, we don't want to return that code or
> else the caller will interpret the sound
71466 |     as at the end and terminate decoding.
71467 |     */
71468 |     if (result == MA_AT_END) {
71469 |         if (ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) == MA_BUSY) {
71470 |             result = MA_BUSY;

```



```

71471 |     }
71472 | }
71473 |
71474 |     if (isDecodedBufferBusy) {
71475 |         result = MA_BUSY;
71476 |     }
71477 |
71478 |     if (pFramesRead != NULL) {
71479 |         *pFramesRead = framesRead;
71480 |     }
71481 |
71482 |     if (result == MA_SUCCESS && framesRead == 0) {
71483 |         result = MA_AT_END;
71484 |     }
71485 |
71486 |     return result;
71487 | }
71488 |
71489 | MA_API ma_result ma_resource_manager_data_buffer_seek_to_pcm_frame(ma_resource_manager_data_buffer*
> pDataBuffer, ma_uint64 frameIndex)
71490 | {
71491 |     ma_result result;
71492 |
71493 |     /* We cannot be using the data source after it's been uninitialized. */
71494 |     MA_ASSERT(ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) != MA_UNAVAILABLE);
71495 |
71496 |     /* If we haven't yet got a connector we need to abort. */
71497 |     if (ma_resource_manager_data_buffer_has_connector(pDataBuffer) == MA_FALSE) {
71498 |         pDataBuffer->seekTargetInPCMFrames = frameIndex;
71499 |         pDataBuffer->seekToCursorOnNextRead = MA_TRUE;
71500 |         return MA_BUSY; /* Still loading. */
71501 |     }
71502 |
71503 |     result =
> ma_data_source_seek_to_pcm_frame(ma_resource_manager_data_buffer_get_connector(pDataBuffer),
> frameIndex);
71504 |     if (result != MA_SUCCESS) {
71505 |         return result;
71506 |     }
71507 |
71508 |     pDataBuffer->seekTargetInPCMFrames = ~(ma_uint64)0; /* <-- For identification purposes. */
71509 |     pDataBuffer->seekToCursorOnNextRead = MA_FALSE;
71510 |
71511 |     return MA_SUCCESS;
71512 | }
71513 |
71514 | MA_API ma_result ma_resource_manager_data_buffer_get_data_format(ma_resource_manager_data_buffer*
> pDataBuffer, ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel*
> pChannelMap, size_t channelMapCap)
71515 | {
71516 |     /* We cannot be using the data source after it's been uninitialized. */
71517 |     MA_ASSERT(ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) != MA_UNAVAILABLE);
71518 |
71519 |     switch (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode))
71520 |     {
71521 |         case ma_resource_manager_data_supply_type_encoded:
71522 |         {
71523 |             return ma_data_source_get_data_format(&pDataBuffer->connector.decoder, pFormat,
> pChannels, pSampleRate, pChannelMap, channelMapCap);
71524 |         };
71525 |
71526 |         case ma_resource_manager_data_supply_type_decoded:
71527 |         {
71528 |             *pFormat = pDataBuffer->pNode->data.backend.decoded.format;
71529 |             *pChannels = pDataBuffer->pNode->data.backend.decoded.channels;
71530 |             *pSampleRate = pDataBuffer->pNode->data.backend.decoded.sampleRate;
71531 |             ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap,
> channelMapCap, pDataBuffer->pNode->data.backend.decoded.channels);
71532 |             return MA_SUCCESS;
71533 |         };
71534 |
71535 |         case ma_resource_manager_data_supply_type_decoded_paged:
71536 |         {
71537 |             *pFormat = pDataBuffer->pNode->data.backend.decodedPaged.data.format;
71538 |             *pChannels = pDataBuffer->pNode->data.backend.decodedPaged.data.channels;
71539 |             *pSampleRate = pDataBuffer->pNode->data.backend.decodedPaged.sampleRate;
71540 |             ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap,
> channelMapCap, pDataBuffer->pNode->data.backend.decoded.channels);
71541 |             return MA_SUCCESS;

```

```

71542 |     };
71543 |
71544 |     case ma_resource_manager_data_supply_type_unknown:
71545 |     {
71546 |         return MA_BUSY; /* Still loading. */
71547 |     };
71548 |
71549 |     default:
71550 |     {
71551 |         /* Unknown supply type. Should never hit this. */
71552 |         return MA_INVALID_ARGS;
71553 |     }
71554 | }
71555 | }
71556 |
71557 | MA_API ma_result
> ma_resource_manager_data_buffer_get_cursor_in_pcm_frames(ma_resource_manager_data_buffer*
> pDataBuffer, ma_uint64* pCursor)
71558 | {
71559 |     if (pDataBuffer == NULL || pCursor == NULL) {
71560 |         return MA_INVALID_ARGS;
71561 |     }
71562 |
71563 |     /* We cannot be using the data source after it's been uninitialized. */
71564 |     MA_ASSERT(ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) != MA_UNAVAILABLE);
71565 |
71566 |     *pCursor = 0;
71567 |
71568 |     switch (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode))
71569 |     {
71570 |         case ma_resource_manager_data_supply_type_encoded:
71571 |         {
71572 |             return ma_decoder_get_cursor_in_pcm_frames(&pDataBuffer->connector.decoder, pCursor);
71573 |         };
71574 |
71575 |         case ma_resource_manager_data_supply_type_decoded:
71576 |         {
71577 |             return ma_audio_buffer_get_cursor_in_pcm_frames(&pDataBuffer->connector.buffer,
> pCursor);
71578 |         };
71579 |
71580 |         case ma_resource_manager_data_supply_type_decoded_paged:
71581 |         {
71582 |             return
> ma_paged_audio_buffer_get_cursor_in_pcm_frames(&pDataBuffer->connector.pagedBuffer, pCursor);
71583 |         };
71584 |
71585 |         case ma_resource_manager_data_supply_type_unknown:
71586 |         {
71587 |             return MA_BUSY;
71588 |         };
71589 |
71590 |         default:
71591 |         {
71592 |             return MA_INVALID_ARGS;
71593 |         }
71594 |     }
71595 | }
71596 |
71597 | MA_API ma_result
> ma_resource_manager_data_buffer_get_length_in_pcm_frames(ma_resource_manager_data_buffer*
> pDataBuffer, ma_uint64* pLength)
71598 | {
71599 |     if (pDataBuffer == NULL || pLength == NULL) {
71600 |         return MA_INVALID_ARGS;
71601 |     }
71602 |
71603 |     /* We cannot be using the data source after it's been uninitialized. */
71604 |     MA_ASSERT(ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode) != MA_UNAVAILABLE);
71605 |
71606 |     if (ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode) ==
> ma_resource_manager_data_supply_type_unknown) {
71607 |         return MA_BUSY; /* Still loading. */
71608 |     }
71609 |
71610 |     return
> ma_data_source_get_length_in_pcm_frames(ma_resource_manager_data_buffer_get_connector(pDataBuffer),
> pLength);
71611 | }

```

```

71612 |
71613 | MA_API ma_result ma_resource_manager_data_buffer_result(const ma_resource_manager_data_buffer*
> pBuffer)
71614 | {
71615 |     if (pPBuffer == NULL) {
71616 |         return MA_INVALID_ARGS;
71617 |     }
71618 |
71619 |     return (ma_result)ma_atomic_load_i32((ma_result*)&pPBuffer->result);    /* Need a naughty
> const-cast here. */
71620 | }
71621 |
71622 | MA_API ma_result ma_resource_manager_data_buffer_set_looping(ma_resource_manager_data_buffer*
> pBuffer, ma_bool32 isLooping)
71623 | {
71624 |     return ma_data_source_set_looping(pPBuffer, isLooping);
71625 | }
71626 |
71627 | MA_API ma_bool32 ma_resource_manager_data_buffer_is_looping(const ma_resource_manager_data_buffer*
> pBuffer)
71628 | {
71629 |     return ma_data_source_is_looping(pPBuffer);
71630 | }
71631 |
71632 | MA_API ma_result
> ma_resource_manager_data_buffer_get_available_frames(ma_resource_manager_data_buffer* pBuffer,
> ma_uint64* pAvailableFrames)
71633 | {
71634 |     if (pAvailableFrames == NULL) {
71635 |         return MA_INVALID_ARGS;
71636 |     }
71637 |
71638 |     *pAvailableFrames = 0;
71639 |
71640 |     if (pPBuffer == NULL) {
71641 |         return MA_INVALID_ARGS;
71642 |     }
71643 |
71644 |     if (ma_resource_manager_data_buffer_node_get_data_supply_type(pPBuffer->pNode) ==
> ma_resource_manager_data_supply_type_unknown) {
71645 |         if (ma_resource_manager_data_buffer_node_result(pPBuffer->pNode) == MA_BUSY) {
71646 |             return MA_BUSY;
71647 |         } else {
71648 |             return MA_INVALID_OPERATION;    /* No connector. */
71649 |         }
71650 |     }
71651 |
71652 |     switch (ma_resource_manager_data_buffer_node_get_data_supply_type(pPBuffer->pNode))
71653 |     {
71654 |         case ma_resource_manager_data_supply_type_encoded:
71655 |         {
71656 |             return ma_decoder_get_available_frames(&pPBuffer->connector.decoder,
> pAvailableFrames);
71657 |         };
71658 |
71659 |         case ma_resource_manager_data_supply_type_decoded:
71660 |         {
71661 |             return ma_audio_buffer_get_available_frames(&pPBuffer->connector.buffer,
> pAvailableFrames);
71662 |         };
71663 |
71664 |         case ma_resource_manager_data_supply_type_decoded_paged:
71665 |         {
71666 |             ma_uint64 cursor;
71667 |             ma_paged_audio_buffer_get_cursor_in_pcm_frames(&pPBuffer->connector.pagedBuffer,
> &cursor);
71668 |
71669 |             if (pPBuffer->pNode->data.backend.decodedPaged.decodedFrameCount > cursor) {
71670 |                 *pAvailableFrames = pPBuffer->pNode->data.backend.decodedPaged.decodedFrameCount
> - cursor;
71671 |             } else {
71672 |                 *pAvailableFrames = 0;
71673 |             }
71674 |
71675 |             return MA_SUCCESS;
71676 |         };
71677 |
71678 |         case ma_resource_manager_data_supply_type_unknown:
71679 |         default:

```

```

71680 |         {
71681 |             /* Unknown supply type. Should never hit this. */
71682 |             return MA_INVALID_ARGS;
71683 |         }
71684 |     }
71685 | }
71686 |
71687 | MA_API ma_result ma_resource_manager_register_file(ma_resource_manager* pResourceManager, const
> char* pFilePath, ma_uint32 flags)
71688 | {
71689 |     return ma_resource_manager_data_buffer_node_acquire(pResourceManager, pFilePath, NULL, 0, flags,
> NULL, NULL, NULL, NULL);
71690 | }
71691 |
71692 | MA_API ma_result ma_resource_manager_register_file_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath, ma_uint32 flags)
71693 | {
71694 |     return ma_resource_manager_data_buffer_node_acquire(pResourceManager, NULL, pFilePath, 0, flags,
> NULL, NULL, NULL, NULL);
71695 | }
71696 |
71697 |
71698 | static ma_result ma_resource_manager_register_data(ma_resource_manager* pResourceManager, const
> char* pName, const wchar_t* pNameW, ma_resource_manager_data_supply* pExistingData)
71699 | {
71700 |     return ma_resource_manager_data_buffer_node_acquire(pResourceManager, pName, pNameW, 0, 0,
> pExistingData, NULL, NULL, NULL);
71701 | }
71702 |
71703 | static ma_result ma_resource_manager_register_decoded_data_internal(ma_resource_manager*
> pResourceManager, const char* pName, const wchar_t* pNameW, const void* pData, ma_uint64 frameCount,
> ma_format format, ma_uint32 channels, ma_uint32 sampleRate)
71704 | {
71705 |     ma_resource_manager_data_supply data;
71706 |     data.type = ma_resource_manager_data_supply_type_decoded;
71707 |     data.backend.decoded.pData = pData;
71708 |     data.backend.decoded.totalFrameCount = frameCount;
71709 |     data.backend.decoded.format = format;
71710 |     data.backend.decoded.channels = channels;
71711 |     data.backend.decoded.sampleRate = sampleRate;
71712 |
71713 |     return ma_resource_manager_register_data(pResourceManager, pName, pNameW, &data);
71714 | }
71715 |
71716 | MA_API ma_result ma_resource_manager_register_decoded_data(ma_resource_manager* pResourceManager,
> const char* pName, const void* pData, ma_uint64 frameCount, ma_format format, ma_uint32 channels,
> ma_uint32 sampleRate)
71717 | {
71718 |     return ma_resource_manager_register_decoded_data_internal(pResourceManager, pName, NULL, pData,
> frameCount, format, channels, sampleRate);
71719 | }
71720 |
71721 | MA_API ma_result ma_resource_manager_register_decoded_data_w(ma_resource_manager* pResourceManager,
> const wchar_t* pName, const void* pData, ma_uint64 frameCount, ma_format format, ma_uint32 channels,
> ma_uint32 sampleRate)
71722 | {
71723 |     return ma_resource_manager_register_decoded_data_internal(pResourceManager, NULL, pName, pData,
> frameCount, format, channels, sampleRate);
71724 | }
71725 |
71726 |
71727 | static ma_result ma_resource_manager_register_encoded_data_internal(ma_resource_manager*
> pResourceManager, const char* pName, const wchar_t* pNameW, const void* pData, size_t sizeInBytes)
71728 | {
71729 |     ma_resource_manager_data_supply data;
71730 |     data.type = ma_resource_manager_data_supply_type_encoded;
71731 |     data.backend.encoded.pData = pData;
71732 |     data.backend.encoded.sizeInBytes = sizeInBytes;
71733 |
71734 |     return ma_resource_manager_register_data(pResourceManager, pName, pNameW, &data);
71735 | }
71736 |
71737 | MA_API ma_result ma_resource_manager_register_encoded_data(ma_resource_manager* pResourceManager,
> const char* pName, const void* pData, size_t sizeInBytes)
71738 | {
71739 |     return ma_resource_manager_register_encoded_data_internal(pResourceManager, pName, NULL, pData,
> sizeInBytes);
71740 | }
71741 |

```

```

71742 | MA_API ma_result ma_resource_manager_register_encoded_data_w(ma_resource_manager* pResourceManager,
> const wchar_t* pName, const void* pData, size_t sizeInBytes)
71743 | {
71744 |     return ma_resource_manager_register_encoded_data_internal(pResourceManager, NULL, pName, pData,
> sizeInBytes);
71745 | }
71746 |
71747 |
71748 | MA_API ma_result ma_resource_manager_unregister_file(ma_resource_manager* pResourceManager, const
> char* pFilePath)
71749 | {
71750 |     return ma_resource_manager_unregister_data(pResourceManager, pFilePath);
71751 | }
71752 |
71753 | MA_API ma_result ma_resource_manager_unregister_file_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath)
71754 | {
71755 |     return ma_resource_manager_unregister_data_w(pResourceManager, pFilePath);
71756 | }
71757 |
71758 | MA_API ma_result ma_resource_manager_unregister_data(ma_resource_manager* pResourceManager, const
> char* pName)
71759 | {
71760 |     return ma_resource_manager_data_buffer_node_unacquire(pResourceManager, NULL, pName, NULL);
71761 | }
71762 |
71763 | MA_API ma_result ma_resource_manager_unregister_data_w(ma_resource_manager* pResourceManager, const
> wchar_t* pName)
71764 | {
71765 |     return ma_resource_manager_data_buffer_node_unacquire(pResourceManager, NULL, NULL, pName);
71766 | }
71767 |
71768 |
71769 | static ma_uint32
> ma_resource_manager_data_stream_next_execution_order(ma_resource_manager_data_stream* pDataStream)
71770 | {
71771 |     MA_ASSERT(pDataStream != NULL);
71772 |     return ma_atomic_fetch_add_32(&pDataStream->executionCounter, 1);
71773 | }
71774 |
71775 | static ma_bool32 ma_resource_manager_data_stream_is_decoder_at_end(const
> ma_resource_manager_data_stream* pDataStream)
71776 | {
71777 |     MA_ASSERT(pDataStream != NULL);
71778 |     return ma_atomic_load_32((ma_bool32*)&pDataStream->isDecoderAtEnd);
71779 | }
71780 |
71781 | static ma_uint32 ma_resource_manager_data_stream_seek_counter(const ma_resource_manager_data_stream*
> pDataStream)
71782 | {
71783 |     MA_ASSERT(pDataStream != NULL);
71784 |     return ma_atomic_load_32((ma_uint32*)&pDataStream->seekCounter);
71785 | }
71786 |
71787 |
71788 | static ma_result ma_resource_manager_data_stream_cb__read_pcm_frames(ma_data_source* pDataSource,
> void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
71789 | {
71790 |     return
> ma_resource_manager_data_stream_read_pcm_frames((ma_resource_manager_data_stream*)pDataSource,
> pFramesOut, frameCount, pFramesRead);
71791 | }
71792 |
71793 | static ma_result ma_resource_manager_data_stream_cb__seek_to_pcm_frame(ma_data_source* pDataSource,
> ma_uint64 frameIndex)
71794 | {
71795 |     return
> ma_resource_manager_data_stream_seek_to_pcm_frame((ma_resource_manager_data_stream*)pDataSource,
> frameIndex);
71796 | }
71797 |
71798 | static ma_result ma_resource_manager_data_stream_cb__get_data_format(ma_data_source* pDataSource,
> ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t
> channelMapCap)
71799 | {
71800 |     return
> ma_resource_manager_data_stream_get_data_format((ma_resource_manager_data_stream*)pDataSource,
> pFormat, pChannels, pSampleRate, pChannelMap, channelMapCap);
71801 | }

```

```

71802 |
71803 | static ma_result ma_resource_manager_data_stream_cb__get_cursor_in_pcm_frames(ma_data_source*
> pDataSource, ma_uint64* pCursor)
71804 | {
71805 |     return ma_resource_manager_data_stream_get_cursor_in_pcm_frames((ma_resource_manager_data_stream
> *)pDataSource, pCursor);
71806 | }
71807 |
71808 | static ma_result ma_resource_manager_data_stream_cb__get_length_in_pcm_frames(ma_data_source*
> pDataSource, ma_uint64* pLength)
71809 | {
71810 |     return ma_resource_manager_data_stream_get_length_in_pcm_frames((ma_resource_manager_data_stream
> *)pDataSource, pLength);
71811 | }
71812 |
71813 | static ma_result ma_resource_manager_data_stream_cb__set_looping(ma_data_source* pDataSource,
> ma_bool32 isLooping)
71814 | {
71815 |     ma_resource_manager_data_stream* pDataStream = (ma_resource_manager_data_stream*)pDataSource;
71816 |     MA_ASSERT(pDataStream != NULL);
71817 |
71818 |     ma_atomic_exchange_32(&pDataStream->isLooping, isLooping);
71819 |
71820 |     return MA_SUCCESS;
71821 | }
71822 |
71823 | static ma_data_source_vtable g_ma_resource_manager_data_stream_vtable =
71824 | {
71825 |     ma_resource_manager_data_stream_cb__read_pcm_frames,
71826 |     ma_resource_manager_data_stream_cb__seek_to_pcm_frame,
71827 |     ma_resource_manager_data_stream_cb__get_data_format,
71828 |     ma_resource_manager_data_stream_cb__get_cursor_in_pcm_frames,
71829 |     ma_resource_manager_data_stream_cb__get_length_in_pcm_frames,
71830 |     ma_resource_manager_data_stream_cb__set_looping,
71831 |     0 /*MA_DATA_SOURCE_SELF_MANAGED_RANGE_AND_LOOP_POINT*/
71832 | };
71833 |
71834 | static void ma_resource_manager_data_stream_set_absolute_cursor(ma_resource_manager_data_stream*
> pDataStream, ma_uint64 absoluteCursor)
71835 | {
71836 |     /* Loop if possible. */
71837 |     if (absoluteCursor > pDataStream->totalLengthInPCMFrames && pDataStream->totalLengthInPCMFrames
> > 0) {
71838 |         absoluteCursor = absoluteCursor % pDataStream->totalLengthInPCMFrames;
71839 |     }
71840 |
71841 |     ma_atomic_exchange_64(&pDataStream->absoluteCursor, absoluteCursor);
71842 | }
71843 |
71844 | MA_API ma_result ma_resource_manager_data_stream_init_ex(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_stream* pDataStream)
71845 | {
71846 |     ma_result result;
71847 |     ma_data_source_config dataSourceConfig;
71848 |     char* pFilePathCopy = NULL;
71849 |     wchar_t* pFilePathWCopy = NULL;
71850 |     ma_job job;
71851 |     ma_bool32 waitBeforeReturning = MA_FALSE;
71852 |     ma_resource_manager_inline_notifications waitNotification;
71853 |     ma_resource_manager_pipeline_notifications notifications;
71854 |     ma_uint32 flags;
71855 |
71856 |     if (pDataStream == NULL) {
71857 |         if (pConfig != NULL && pConfig->pNotifications != NULL) {
71858 |
> ma_resource_manager_pipeline_notifications_signal_all_notifications(pConfig->pNotifications);
71859 |         }
71860 |
71861 |         return MA_INVALID_ARGS;
71862 |     }
71863 |
71864 |     MA_ZERO_OBJECT(pDataStream);
71865 |
71866 |     if (pConfig == NULL) {
71867 |         return MA_INVALID_ARGS;
71868 |     }
71869 |
71870 |     if (pConfig->pNotifications != NULL) {
71871 |         notifications = *pConfig->pNotifications; /* From here on out, `notifications` should be

```

```

> used instead of `pNotifications`. Setting this to NULL to catch any errors at testing time. */
71872 |     } else {
71873 |         MA_ZERO_OBJECT(&notifications);
71874 |     }
71875 |
71876 |     dataSourceConfig = ma_data_source_config_init();
71877 |     dataSourceConfig.vtable = &g_ma_resource_manager_data_stream_vtable;
71878 |
71879 |     result = ma_data_source_init(&dataSourceConfig, &pDataStream->ds);
71880 |     if (result != MA_SUCCESS) {
71881 |         ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71882 |         return result;
71883 |     }
71884 |
71885 |     flags = pConfig->flags;
71886 |     if (pConfig->isLooping) {
71887 |         flags |= MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING;
71888 |     }
71889 |
71890 |     pDataStream->pResourceManager = pResourceManager;
71891 |     pDataStream->flags             = pConfig->flags;
71892 |     pDataStream->result            = MA_BUSY;
71893 |
71894 |     ma_data_source_set_range_in_pcm_frames(pDataStream, pConfig->rangeBegInPCMFrames,
> pConfig->rangeEndInPCMFrames);
71895 |     ma_data_source_set_loop_point_in_pcm_frames(pDataStream, pConfig->loopPointBegInPCMFrames,
> pConfig->loopPointEndInPCMFrames);
71896 |     ma_data_source_set_looping(pDataStream, (flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING)
> != 0);
71897 |
71898 |     if (pResourceManager == NULL || (pConfig->pFilePath == NULL && pConfig->pFilePathW == NULL)) {
71899 |         ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71900 |         return MA_INVALID_ARGS;
71901 |     }
71902 |
71903 |     /* We want all access to the VFS and the internal decoder to happen on the job thread just to
> keep things easier to manage for the VFS. */
71904 |
71905 |     /* We need a copy of the file path. We should probably make this more efficient, but for now
> we'll do a transient memory allocation. */
71906 |     if (pConfig->pFilePath != NULL) {
71907 |         pFilePathCopy = ma_copy_string(pConfig->pFilePath,
> &pResourceManager->config.allocationCallbacks);
71908 |     } else {
71909 |         pFilePathWCopy = ma_copy_string_w(pConfig->pFilePathW,
> &pResourceManager->config.allocationCallbacks);
71910 |     }
71911 |
71912 |     if (pFilePathCopy == NULL && pFilePathWCopy == NULL) {
71913 |         ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71914 |         return MA_OUT_OF_MEMORY;
71915 |     }
71916 |
71917 |     /*
71918 |     We need to check for the presence of MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC. If it's not
> set, we need to wait before returning. Otherwise we
71919 |     can return immediately. Likewise, we'll also check for
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT and do the same.
71920 |     */
71921 |     if ((pConfig->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_ASYNC) == 0 || (pConfig->flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT) != 0) {
71922 |         waitBeforeReturning = MA_TRUE;
71923 |         ma_resource_manager_inline_notification_init(pResourceManager, &waitNotification);
71924 |     }
71925 |
71926 |     ma_resource_manager_pipeline_notifications_acquire_all_fences(&notifications);
71927 |
71928 |     /* Set the absolute cursor to our initial seek position so retrieval of the cursor returns a
> good value. */
71929 |     ma_resource_manager_data_stream_set_absolute_cursor(pDataStream,
> pConfig->initialSeekPointInPCMFrames);
71930 |
71931 |     /* We now have everything we need to post the job. This is the last thing we need to do from
> here. The rest will be done by the job thread. */
71932 |     job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_LOAD_DATA_STREAM);
71933 |     job.order = ma_resource_manager_data_stream_next_execution_order(pDataStream);
71934 |     job.data.resourceManager.loadDataStream.pDataStream = pDataStream;
71935 |     job.data.resourceManager.loadDataStream.pFilePath   = pFilePathCopy;
71936 |     job.data.resourceManager.loadDataStream.pFilePathW  = pFilePathWCopy;

```



```

71937 |     job.data.resourceManager.loadDataStream.initialSeekPoint =
> pConfig->initialSeekPointInPCMFrames;
71938 |     job.data.resourceManager.loadDataStream.pInitNotification = (waitBeforeReturning == MA_TRUE) ?
> &waitNotification : notifications.init.pNotification;
71939 |     job.data.resourceManager.loadDataStream.pInitFence      = notifications.init.pFence;
71940 |     result = ma_resource_manager_post_job(pResourceManager, &job);
71941 |     if (result != MA_SUCCESS) {
71942 |         ma_resource_manager_pipeline_notifications_signal_all_notifications(&notifications);
71943 |         ma_resource_manager_pipeline_notifications_release_all_fences(&notifications);
71944 |
71945 |         if (waitBeforeReturning) {
71946 |             ma_resource_manager_inline_notification_uninit(&waitNotification);
71947 |         }
71948 |
71949 |         ma_free(pFilePathCopy, &pResourceManager->config.allocationCallbacks);
71950 |         ma_free(pFilePathWCopy, &pResourceManager->config.allocationCallbacks);
71951 |         return result;
71952 |     }
71953 |
71954 |     /* Wait if needed. */
71955 |     if (waitBeforeReturning) {
71956 |         ma_resource_manager_inline_notification_wait_and_uninit(&waitNotification);
71957 |
71958 |         if (notifications.init.pNotification != NULL) {
71959 |             ma_async_notification_signal(notifications.init.pNotification);
71960 |         }
71961 |
71962 |         /*
71963 |         If there was an error during initialization make sure we return that result here. We don't
> want to do this
71964 |         if we're not waiting because it will most likely be in a busy state.
71965 |         */
71966 |         if (pDataStream->result != MA_SUCCESS) {
71967 |             return pDataStream->result;
71968 |         }
71969 |
71970 |         /* NOTE: Do not release pInitFence here. That will be done by the job. */
71971 |     }
71972 |
71973 |     return MA_SUCCESS;
71974 | }
71975 |
71976 | MA_API ma_result ma_resource_manager_data_stream_init(ma_resource_manager* pResourceManager, const
> char* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
> ma_resource_manager_data_stream* pDataStream)
71977 | {
71978 |     ma_resource_manager_data_source_config config;
71979 |
71980 |     config = ma_resource_manager_data_source_config_init();
71981 |     config.pFilePath      = pFilePath;
71982 |     config.flags          = flags;
71983 |     config.pNotifications = pNotifications;
71984 |
71985 |     return ma_resource_manager_data_stream_init_ex(pResourceManager, &config, pDataStream);
71986 | }
71987 |
71988 | MA_API ma_result ma_resource_manager_data_stream_init_w(ma_resource_manager* pResourceManager, const
> wchar_t* pFilePath, ma_uint32 flags, const ma_resource_manager_pipeline_notifications*
> pNotifications, ma_resource_manager_data_stream* pDataStream)
71989 | {
71990 |     ma_resource_manager_data_source_config config;
71991 |
71992 |     config = ma_resource_manager_data_source_config_init();
71993 |     config.pFilePathW      = pFilePath;
71994 |     config.flags           = flags;
71995 |     config.pNotifications = pNotifications;
71996 |
71997 |     return ma_resource_manager_data_stream_init_ex(pResourceManager, &config, pDataStream);
71998 | }
71999 |
72000 | MA_API ma_result ma_resource_manager_data_stream_uninit(ma_resource_manager_data_stream*
> pDataStream)
72001 | {
72002 |     ma_resource_manager_inline_notification freeEvent;
72003 |     ma_job job;
72004 |
72005 |     if (pDataStream == NULL) {
72006 |         return MA_INVALID_ARGS;
72007 |     }

```

```

72008 |
72009 |     /* The first thing to do is set the result to unavailable. This will prevent future page
> decoding. */
72010 |     ma_atomic_exchange_i32(&pDataStream->result, MA_UNAVAILABLE);
72011 |
72012 |     /*
72013 |     We need to post a job to ensure we're not in the middle of decoding or anything. Because the
> object is owned by the caller, we'll need
72014 |     to wait for it to complete before returning which means we need an event.
72015 |     */
72016 |     ma_resource_manager_inline_notification_init(pDataStream->pResourceManager, &freeEvent);
72017 |
72018 |     job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_STREAM);
72019 |     job.order = ma_resource_manager_data_stream_next_execution_order(pDataStream);
72020 |     job.data.resourceManager.freeDataStream.pDataStream = pDataStream;
72021 |     job.data.resourceManager.freeDataStream.pDoneNotification = &freeEvent;
72022 |     job.data.resourceManager.freeDataStream.pDoneFence = NULL;
72023 |     ma_resource_manager_post_job(pDataStream->pResourceManager, &job);
72024 |
72025 |     /* We need to wait for the job to finish processing before we return. */
72026 |     ma_resource_manager_inline_notification_wait_and_uninit(&freeEvent);
72027 |
72028 |     return MA_SUCCESS;
72029 | }
72030 |
72031 |
72032 | static ma_uint32
> ma_resource_manager_data_stream_get_page_size_in_frames(ma_resource_manager_data_stream*
> pDataStream)
72033 | {
72034 |     MA_ASSERT(pDataStream != NULL);
72035 |     MA_ASSERT(pDataStream->isDecoderInitialized == MA_TRUE);
72036 |
72037 |     return MA_RESOURCE_MANAGER_PAGE_SIZE_IN_MILLISECONDS *
> (pDataStream->decoder.outputSampleRate/1000);
72038 | }
72039 |
72040 | static void* ma_resource_manager_data_stream_get_page_data_pointer(ma_resource_manager_data_stream*
> pDataStream, ma_uint32 pageIndex, ma_uint32 relativeCursor)
72041 | {
72042 |     MA_ASSERT(pDataStream != NULL);
72043 |     MA_ASSERT(pDataStream->isDecoderInitialized == MA_TRUE);
72044 |     MA_ASSERT(pageIndex == 0 || pageIndex == 1);
72045 |
72046 |     return ma_offset_ptr(pDataStream->pPageData,
> ((ma_resource_manager_data_stream_get_page_size_in_frames(pDataStream) * pageIndex) +
> relativeCursor) * ma_get_bytes_per_frame(pDataStream->decoder.outputFormat,
> pDataStream->decoder.outputChannels));
72047 | }
72048 |
72049 | static void ma_resource_manager_data_stream_fill_page(ma_resource_manager_data_stream* pDataStream,
> ma_uint32 pageIndex)
72050 | {
72051 |     ma_result result = MA_SUCCESS;
72052 |     ma_uint64 pageSizeInFrames;
72053 |     ma_uint64 totalFramesReadForThisPage = 0;
72054 |     void* pPageData = ma_resource_manager_data_stream_get_page_data_pointer(pDataStream, pageIndex,
> 0);
72055 |
72056 |     pageSizeInFrames = ma_resource_manager_data_stream_get_page_size_in_frames(pDataStream);
72057 |
72058 |     /* The decoder needs to inherit the stream's looping and range state. */
72059 |     {
72060 |         ma_uint64 rangeBeg;
72061 |         ma_uint64 rangeEnd;
72062 |         ma_uint64 loopPointBeg;
72063 |         ma_uint64 loopPointEnd;
72064 |
72065 |         ma_data_source_set_looping(&pDataStream->decoder,
> ma_resource_manager_data_stream_is_looping(pDataStream));
72066 |
72067 |         ma_data_source_get_range_in_pcm_frames(pDataStream, &rangeBeg, &rangeEnd);
72068 |         ma_data_source_set_range_in_pcm_frames(&pDataStream->decoder, rangeBeg, rangeEnd);
72069 |
72070 |         ma_data_source_get_loop_point_in_pcm_frames(pDataStream, &loopPointBeg, &loopPointEnd);
72071 |         ma_data_source_set_loop_point_in_pcm_frames(&pDataStream->decoder, loopPointBeg,
> loopPointEnd);
72072 |     }
72073 |

```

```

72074 | /* Just read straight from the decoder. It will deal with ranges and looping for us. */
72075 | result = ma_data_source_read_pcm_frames(&pDataStream->decoder, pPageData, pageSizeInFrames,
> &totalFramesReadForThisPage);
72076 | if (result == MA_AT_END || totalFramesReadForThisPage < pageSizeInFrames) {
72077 |     ma_atomic_exchange_32(&pDataStream->isDecoderAtEnd, MA_TRUE);
72078 | }
72079 |
72080 | ma_atomic_exchange_32(&pDataStream->pageFrameCount[pPageIndex],
> (ma_uint32)totalFramesReadForThisPage);
72081 | ma_atomic_exchange_32(&pDataStream->isPageValid[pPageIndex], MA_TRUE);
72082 | }
72083 |
72084 | static void ma_resource_manager_data_stream_fill_pages(ma_resource_manager_data_stream* pDataStream)
72085 | {
72086 |     ma_uint32 iPage;
72087 |
72088 |     MA_ASSERT(pDataStream != NULL);
72089 |
72090 |     for (iPage = 0; iPage < 2; iPage += 1) {
72091 |         ma_resource_manager_data_stream_fill_page(pDataStream, iPage);
72092 |     }
72093 | }
72094 |
72095 |
72096 | static ma_result ma_resource_manager_data_stream_map(ma_resource_manager_data_stream* pDataStream,
> void** ppFramesOut, ma_uint64* pFrameCount)
72097 | {
72098 |     ma_uint64 framesAvailable;
72099 |     ma_uint64 frameCount = 0;
72100 |
72101 |     /* We cannot be using the data source after it's been uninitialized. */
72102 |     MA_ASSERT(ma_resource_manager_data_stream_result(pDataStream) != MA_UNAVAILABLE);
72103 |
72104 |     if (pFrameCount != NULL) {
72105 |         frameCount = *pFrameCount;
72106 |         *pFrameCount = 0;
72107 |     }
72108 |     if (ppFramesOut != NULL) {
72109 |         *ppFramesOut = NULL;
72110 |     }
72111 |
72112 |     if (pDataStream == NULL || ppFramesOut == NULL || pFrameCount == NULL) {
72113 |         return MA_INVALID_ARGS;
72114 |     }
72115 |
72116 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_SUCCESS) {
72117 |         return MA_INVALID_OPERATION;
72118 |     }
72119 |
72120 |     /* Don't attempt to read while we're in the middle of seeking. Tell the caller that we're busy.
> */
72121 |     if (ma_resource_manager_data_stream_seek_counter(pDataStream) > 0) {
72122 |         return MA_BUSY;
72123 |     }
72124 |
72125 |     /* If the page we're on is invalid it means we've caught up to the job thread. */
72126 |     if (ma_atomic_load_32(&pDataStream->isPageValid[pDataStream->currentPageIndex]) == MA_FALSE) {
72127 |         framesAvailable = 0;
72128 |     } else {
72129 |         /*
72130 |         The page we're on is valid so we must have some frames available. We need to make sure that
> we don't overflow into the next page, even if it's valid. The reason is
72131 |         that the unmap process will only post an update for one page at a time. Keeping mapping tied
> to page boundaries makes this simpler.
72132 |         */
72133 |         ma_uint32 currentPageFrameCount =
> ma_atomic_load_32(&pDataStream->pageFrameCount[pDataStream->currentPageIndex]);
72134 |         MA_ASSERT(currentPageFrameCount >= pDataStream->relativeCursor);
72135 |
72136 |         framesAvailable = currentPageFrameCount - pDataStream->relativeCursor;
72137 |     }
72138 |
72139 |     /* If there's no frames available and the result is set to MA_AT_END we need to return
> MA_AT_END. */
72140 |     if (framesAvailable == 0) {
72141 |         if (ma_resource_manager_data_stream_is_decoder_at_end(pDataStream)) {
72142 |             return MA_AT_END;
72143 |         } else {
72144 |             return MA_BUSY; /* There are no frames available, but we're not marked as EOF so we

```

```

> might have caught up to the job thread. Need to return MA_BUSY and wait for more data. */
72145 |     }
72146 | }
72147 |
72148 |     MA_ASSERT(framesAvailable > 0);
72149 |
72150 |     if (frameCount > framesAvailable) {
72151 |         frameCount = framesAvailable;
72152 |     }
72153 |
72154 |     *ppFramesOut = ma_resource_manager_data_stream_get_page_data_pointer(pDataStream,
> pDataStream->currentPageIndex, pDataStream->relativeCursor);
72155 |     *pFrameCount = frameCount;
72156 |
72157 |     return MA_SUCCESS;
72158 | }
72159 |
72160 | static ma_result ma_resource_manager_data_stream_unmap(ma_resource_manager_data_stream* pDataStream,
> ma_uint64 frameCount)
72161 | {
72162 |     ma_uint32 newRelativeCursor;
72163 |     ma_uint32 pageSizeInFrames;
72164 |     ma_job job;
72165 |
72166 |     /* We cannot be using the data source after it's been uninitialized. */
72167 |     MA_ASSERT(ma_resource_manager_data_stream_result(pDataStream) != MA_UNAVAILABLE);
72168 |
72169 |     if (pDataStream == NULL) {
72170 |         return MA_INVALID_ARGS;
72171 |     }
72172 |
72173 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_SUCCESS) {
72174 |         return MA_INVALID_OPERATION;
72175 |     }
72176 |
72177 |     /* The frame count should always fit inside a 32-bit integer. */
72178 |     if (frameCount > 0xFFFFFFFF) {
72179 |         return MA_INVALID_ARGS;
72180 |     }
72181 |
72182 |     pageSizeInFrames = ma_resource_manager_data_stream_get_page_size_in_frames(pDataStream);
72183 |
72184 |     /* The absolute cursor needs to be updated for
> ma_resource_manager_data_stream_get_cursor_in_pcm_frames(). */
72185 |     ma_resource_manager_data_stream_set_absolute_cursor(pDataStream,
> ma_atomic_load_64(&pDataStream->absoluteCursor) + frameCount);
72186 |
72187 |     /* Here is where we need to check if we need to load a new page, and if so, post a job to load
> it. */
72188 |     newRelativeCursor = pDataStream->relativeCursor + (ma_uint32)frameCount;
72189 |
72190 |     /* If the new cursor has flowed over to the next page we need to mark the old one as invalid and
> post an event for it. */
72191 |     if (newRelativeCursor >= pageSizeInFrames) {
72192 |         newRelativeCursor -= pageSizeInFrames;
72193 |
72194 |         /* Here is where we post the job start decoding. */
72195 |         job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_STREAM);
72196 |         job.order = ma_resource_manager_data_stream_next_execution_order(pDataStream);
72197 |         job.data.resourceManager.pageDataStream.pDataStream = pDataStream;
72198 |         job.data.resourceManager.pageDataStream.pageIndex = pDataStream->currentPageIndex;
72199 |
72200 |         /* The page needs to be marked as invalid so that the public API doesn't try reading from
> it. */
72201 |         ma_atomic_exchange_32(&pDataStream->isPageValid[pDataStream->currentPageIndex], MA_FALSE);
72202 |
72203 |         /* Before posting the job we need to make sure we set some state. */
72204 |         pDataStream->relativeCursor = newRelativeCursor;
72205 |         pDataStream->currentPageIndex = (pDataStream->currentPageIndex + 1) & 0x01;
72206 |         return ma_resource_manager_post_job(pDataStream->pResourceManager, &job);
72207 |     } else {
72208 |         /* We haven't moved into a new page so we can just move the cursor forward. */
72209 |         pDataStream->relativeCursor = newRelativeCursor;
72210 |         return MA_SUCCESS;
72211 |     }
72212 | }
72213 |
72214 |
72215 | MA_API ma_result ma_resource_manager_data_stream_read_pcm_frames(ma_resource_manager_data_stream*

```

```

> pDataStream, void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
72216 | {
72217 |     ma_result result = MA_SUCCESS;
72218 |     ma_uint64 totalFramesProcessed;
72219 |     ma_format format;
72220 |     ma_uint32 channels;
72221 |
72222 |     /* Safety. */
72223 |     if (pFramesRead != NULL) {
72224 |         *pFramesRead = 0;
72225 |     }
72226 |
72227 |     if (frameCount == 0) {
72228 |         return MA_INVALID_ARGS;
72229 |     }
72230 |
72231 |     /* We cannot be using the data source after it's been uninitialized. */
72232 |     MA_ASSERT(ma_resource_manager_data_stream_result(pDataStream) != MA_UNAVAILABLE);
72233 |
72234 |     if (pDataStream == NULL) {
72235 |         return MA_INVALID_ARGS;
72236 |     }
72237 |
72238 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_SUCCESS) {
72239 |         return MA_INVALID_OPERATION;
72240 |     }
72241 |
72242 |     /* Don't attempt to read while we're in the middle of seeking. Tell the caller that we're busy.
> */
72243 |     if (ma_resource_manager_data_stream_seek_counter(pDataStream) > 0) {
72244 |         return MA_BUSY;
72245 |     }
72246 |
72247 |     ma_resource_manager_data_stream_get_data_format(pDataStream, &format, &channels, NULL, NULL, 0);
72248 |
72249 |     /* Reading is implemented in terms of map/unmap. We need to run this in a loop because mapping
> is clamped against page boundaries. */
72250 |     totalFramesProcessed = 0;
72251 |     while (totalFramesProcessed < frameCount) {
72252 |         void* pMappedFrames;
72253 |         ma_uint64 mappedFrameCount;
72254 |
72255 |         mappedFrameCount = frameCount - totalFramesProcessed;
72256 |         result = ma_resource_manager_data_stream_map(pDataStream, &pMappedFrames,
> &mappedFrameCount);
72257 |         if (result != MA_SUCCESS) {
72258 |             break;
72259 |         }
72260 |
72261 |         /* Copy the mapped data to the output buffer if we have one. It's allowed for pFramesOut to
> be NULL in which case a relative forward seek is performed. */
72262 |         if (pFramesOut != NULL) {
72263 |             ma_copy_pcm_frames(ma_offset_pcm_frames_ptr(pFramesOut, totalFramesProcessed, format,
> channels), pMappedFrames, mappedFrameCount, format, channels);
72264 |         }
72265 |
72266 |         totalFramesProcessed += mappedFrameCount;
72267 |
72268 |         result = ma_resource_manager_data_stream_unmap(pDataStream, mappedFrameCount);
72269 |         if (result != MA_SUCCESS) {
72270 |             break; /* This is really bad - will only get an error here if we failed to post a job
> to the queue for loading the next page. */
72271 |         }
72272 |     }
72273 |
72274 |     if (pFramesRead != NULL) {
72275 |         *pFramesRead = totalFramesProcessed;
72276 |     }
72277 |
72278 |     if (result == MA_SUCCESS && totalFramesProcessed == 0) {
72279 |         result = MA_AT_END;
72280 |     }
72281 |
72282 |     return result;
72283 | }
72284 |
72285 | MA_API ma_result ma_resource_manager_data_stream_seek_to_pcm_frame(ma_resource_manager_data_stream*
> pDataStream, ma_uint64 frameIndex)
72286 | {

```

```

72287 |     ma_job job;
72288 |     ma_result streamResult;
72289 |
72290 |     streamResult = ma_resource_manager_data_stream_result(pDataStream);
72291 |
72292 |     /* We cannot be using the data source after it's been uninitialized. */
72293 |     MA_ASSERT(streamResult != MA_UNAVAILABLE);
72294 |
72295 |     if (pDataStream == NULL) {
72296 |         return MA_INVALID_ARGS;
72297 |     }
72298 |
72299 |     if (streamResult != MA_SUCCESS && streamResult != MA_BUSY) {
72300 |         return MA_INVALID_OPERATION;
72301 |     }
72302 |
72303 |     /* If we're not already seeking and we're sitting on the same frame, just make this a no-op. */
72304 |     if (ma_atomic_load_32(&pDataStream->seekCounter) == 0) {
72305 |         if (ma_atomic_load_64(&pDataStream->absoluteCursor) == frameIndex) {
72306 |             return MA_SUCCESS;
72307 |         }
72308 |     }
72309 |
72310 |
72311 |     /* Increment the seek counter first to indicate to read_paged_pcm_frames() and
> map_paged_pcm_frames() that we are in the middle of a seek and MA_BUSY should be returned. */
72312 |     ma_atomic_fetch_add_32(&pDataStream->seekCounter, 1);
72313 |
72314 |     /* Update the absolute cursor so that ma_resource_manager_data_stream_get_cursor_in_pcm_frames()
> returns the new position. */
72315 |     ma_resource_manager_data_stream_set_absolute_cursor(pDataStream, frameIndex);
72316 |
72317 |     /*
72318 |     We need to clear our currently loaded pages so that the stream starts playback from the new seek
> point as soon as possible. These are for the purpose of the public
72319 |     API and will be ignored by the seek job. The seek job will operate on the assumption that both
> pages have been marked as invalid and the cursor is at the start of
72320 |     the first page.
72321 |     */
72322 |     pDataStream->relativeCursor = 0;
72323 |     pDataStream->currentPageIndex = 0;
72324 |     ma_atomic_exchange_32(&pDataStream->isPageValid[0], MA_FALSE);
72325 |     ma_atomic_exchange_32(&pDataStream->isPageValid[1], MA_FALSE);
72326 |
72327 |     /* Make sure the data stream is not marked as at the end or else if we seek in response to
> hitting the end, we won't be able to read any more data. */
72328 |     ma_atomic_exchange_32(&pDataStream->isDecoderAtEnd, MA_FALSE);
72329 |
72330 |     /*
72331 |     The public API is not allowed to touch the internal decoder so we need to use a job to perform
> the seek. When seeking, the job thread will assume both pages
72332 |     are invalid and any content contained within them will be discarded and replaced with newly
> decoded data.
72333 |     */
72334 |     job = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_SEEK_DATA_STREAM);
72335 |     job.order = ma_resource_manager_data_stream_next_execution_order(pDataStream);
72336 |     job.data.resourceManager.seekDataStream.pDataStream = pDataStream;
72337 |     job.data.resourceManager.seekDataStream.frameIndex = frameIndex;
72338 |     return ma_resource_manager_post_job(pDataStream->pResourceManager, &job);
72339 | }
72340 |
72341 | MA_API ma_result ma_resource_manager_data_stream_get_data_format(ma_resource_manager_data_stream*
> pDataStream, ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel*
> pChannelMap, size_t channelMapCap)
72342 | {
72343 |     /* We cannot be using the data source after it's been uninitialized. */
72344 |     MA_ASSERT(ma_resource_manager_data_stream_result(pDataStream) != MA_UNAVAILABLE);
72345 |
72346 |     if (pFormat != NULL) {
72347 |         *pFormat = ma_format_unknown;
72348 |     }
72349 |
72350 |     if (pChannels != NULL) {
72351 |         *pChannels = 0;
72352 |     }
72353 |
72354 |     if (pSampleRate != NULL) {
72355 |         *pSampleRate = 0;
72356 |     }

```

```

72357 |
72358 |     if (pChannelMap != NULL) {
72359 |         MA_ZERO_MEMORY(pChannelMap, sizeof(*pChannelMap) * channelMapCap);
72360 |     }
72361 |
72362 |     if (pDataStream == NULL) {
72363 |         return MA_INVALID_ARGS;
72364 |     }
72365 |
72366 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_SUCCESS) {
72367 |         return MA_INVALID_OPERATION;
72368 |     }
72369 |
72370 |     /*
72371 |     We're being a little bit naughty here and accessing the internal decoder from the public API.
72372 |     > The output data format is constant, and we've defined this function
72373 |     such that the application is responsible for ensuring it's not called while uninitializing so it
72374 |     > should be safe.
72375 |     */
72376 |     return ma_data_source_get_data_format(&pDataStream->decoder, pFormat, pChannels, pSampleRate,
72377 |     > pChannelMap, channelMapCap);
72378 | }
72379 |
72380 | MA_API ma_result
72381 | > ma_resource_manager_data_stream_get_cursor_in_pcm_frames(ma_resource_manager_data_stream*
72382 | > pDataStream, ma_uint64* pCursor)
72383 | {
72384 |     ma_result result;
72385 |
72386 |     if (pCursor == NULL) {
72387 |         return MA_INVALID_ARGS;
72388 |     }
72389 |
72390 |     *pCursor = 0;
72391 |
72392 |     /* We cannot be using the data source after it's been uninitialized. */
72393 |     MA_ASSERT(ma_resource_manager_data_stream_result(pDataStream) != MA_UNAVAILABLE);
72394 |
72395 |     if (pDataStream == NULL) {
72396 |         return MA_INVALID_ARGS;
72397 |     }
72398 |
72399 |     /*
72400 |     If the stream is in an erroneous state we need to return an invalid operation. We can allow
72401 |     this to be called when the data stream is in a busy state because the caller may have asked
72402 |     for an initial seek position and it's convenient to return that as the cursor position.
72403 |     */
72404 |     result = ma_resource_manager_data_stream_result(pDataStream);
72405 |     if (result != MA_SUCCESS && result != MA_BUSY) {
72406 |         return MA_INVALID_OPERATION;
72407 |     }
72408 |
72409 |     *pCursor = ma_atomic_load_64(&pDataStream->absoluteCursor);
72410 |
72411 |     return MA_SUCCESS;
72412 | }
72413 |
72414 | MA_API ma_result
72415 | > ma_resource_manager_data_stream_get_length_in_pcm_frames(ma_resource_manager_data_stream*
72416 | > pDataStream, ma_uint64* pLength)
72417 | {
72418 |     ma_result streamResult;
72419 |
72420 |     if (pLength == NULL) {
72421 |         return MA_INVALID_ARGS;
72422 |     }
72423 |
72424 |     *pLength = 0;
72425 |
72426 |     streamResult = ma_resource_manager_data_stream_result(pDataStream);
72427 |
72428 |     /* We cannot be using the data source after it's been uninitialized. */
72429 |     MA_ASSERT(streamResult != MA_UNAVAILABLE);
72430 |
72431 |     if (pDataStream == NULL) {
72432 |         return MA_INVALID_ARGS;
72433 |     }
72434 |
72435 |     if (streamResult != MA_SUCCESS) {

```



```

72429 |         return streamResult;
72430 |     }
72431 |
72432 |     /*
72433 |     We most definitely do not want to be calling ma_decoder_get_length_in_pcm_frames() directly.
> Instead we want to use a cached value that we
72434 |     calculated when we initialized it on the job thread.
72435 |     */
72436 |     *pLength = pDataStream->totalLengthInPCMFrames;
72437 |     if (*pLength == 0) {
72438 |         return MA_NOT_IMPLEMENTED; /* Some decoders may not have a known length. */
72439 |     }
72440 |
72441 |     return MA_SUCCESS;
72442 | }
72443 |
72444 | MA_API ma_result ma_resource_manager_data_stream_result(const ma_resource_manager_data_stream*
> pDataStream)
72445 | {
72446 |     if (pDataStream == NULL) {
72447 |         return MA_INVALID_ARGS;
72448 |     }
72449 |
72450 |     return (ma_result)ma_atomic_load_i32(&pDataStream->result);
72451 | }
72452 |
72453 | MA_API ma_result ma_resource_manager_data_stream_set_looping(ma_resource_manager_data_stream*
> pDataStream, ma_bool32 isLooping)
72454 | {
72455 |     return ma_data_source_set_looping(pDataStream, isLooping);
72456 | }
72457 |
72458 | MA_API ma_bool32 ma_resource_manager_data_stream_is_looping(const ma_resource_manager_data_stream*
> pDataStream)
72459 | {
72460 |     if (pDataStream == NULL) {
72461 |         return MA_FALSE;
72462 |     }
72463 |
72464 |     return ma_atomic_load_32((ma_bool32*)&pDataStream->isLooping); /* Naughty const-cast. Value
> won't change from here in practice (maybe from another thread). */
72465 | }
72466 |
72467 | MA_API ma_result
> ma_resource_manager_data_stream_get_available_frames(ma_resource_manager_data_stream* pDataStream,
> ma_uint64* pAvailableFrames)
72468 | {
72469 |     ma_uint32 pageIndex0;
72470 |     ma_uint32 pageIndex1;
72471 |     ma_uint32 relativeCursor;
72472 |     ma_uint64 availableFrames;
72473 |
72474 |     if (pAvailableFrames == NULL) {
72475 |         return MA_INVALID_ARGS;
72476 |     }
72477 |
72478 |     *pAvailableFrames = 0;
72479 |
72480 |     if (pDataStream == NULL) {
72481 |         return MA_INVALID_ARGS;
72482 |     }
72483 |
72484 |     pageIndex0 = pDataStream->currentPageIndex;
72485 |     pageIndex1 = (pDataStream->currentPageIndex + 1) & 0x01;
72486 |     relativeCursor = pDataStream->relativeCursor;
72487 |
72488 |     availableFrames = 0;
72489 |     if (ma_atomic_load_32(&pDataStream->isPageValid[pageIndex0])) {
72490 |         availableFrames += ma_atomic_load_32(&pDataStream->pageFrameCount[pageIndex0]) -
> relativeCursor;
72491 |         if (ma_atomic_load_32(&pDataStream->isPageValid[pageIndex1])) {
72492 |             availableFrames += ma_atomic_load_32(&pDataStream->pageFrameCount[pageIndex1]);
72493 |         }
72494 |     }
72495 |
72496 |     *pAvailableFrames = availableFrames;
72497 |     return MA_SUCCESS;
72498 | }
72499 |

```

```

72500 |
72501 | static ma_result ma_resource_manager_data_source_preinit(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_source* pDataSource)
72502 | {
72503 |     if (pDataSource == NULL) {
72504 |         return MA_INVALID_ARGS;
72505 |     }
72506 |
72507 |     MA_ZERO_OBJECT(pDataSource);
72508 |
72509 |     if (pConfig == NULL) {
72510 |         return MA_INVALID_ARGS;
72511 |     }
72512 |
72513 |     if (pResourceManager == NULL) {
72514 |         return MA_INVALID_ARGS;
72515 |     }
72516 |
72517 |     pDataSource->flags = pConfig->flags;
72518 |     if (pConfig->isLooping) {
72519 |         pDataSource->flags |= MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING;
72520 |     }
72521 |
72522 |     return MA_SUCCESS;
72523 | }
72524 |
72525 | MA_API ma_result ma_resource_manager_data_source_init_ex(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_source_config* pConfig, ma_resource_manager_data_source* pDataSource)
72526 | {
72527 |     ma_result result;
72528 |
72529 |     result = ma_resource_manager_data_source_preinit(pResourceManager, pConfig, pDataSource);
72530 |     if (result != MA_SUCCESS) {
72531 |         return result;
72532 |     }
72533 |
72534 |     /* The data source itself is just a data stream or a data buffer. */
72535 |     if ((pConfig->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72536 |         return ma_resource_manager_data_stream_init_ex(pResourceManager, pConfig,
> &pDataSource->backend.stream);
72537 |     } else {
72538 |         return ma_resource_manager_data_buffer_init_ex(pResourceManager, pConfig,
> &pDataSource->backend.buffer);
72539 |     }
72540 | }
72541 |
72542 | MA_API ma_result ma_resource_manager_data_source_init(ma_resource_manager* pResourceManager, const
> char* pName, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
> ma_resource_manager_data_source* pDataSource)
72543 | {
72544 |     ma_resource_manager_data_source_config config;
72545 |
72546 |     config = ma_resource_manager_data_source_config_init();
72547 |     config.pFilePath = pName;
72548 |     config.flags = flags;
72549 |     config.pNotifications = pNotifications;
72550 |
72551 |     return ma_resource_manager_data_source_init_ex(pResourceManager, &config, pDataSource);
72552 | }
72553 |
72554 | MA_API ma_result ma_resource_manager_data_source_init_w(ma_resource_manager* pResourceManager, const
> wchar_t* pName, ma_uint32 flags, const ma_resource_manager_pipeline_notifications* pNotifications,
> ma_resource_manager_data_source* pDataSource)
72555 | {
72556 |     ma_resource_manager_data_source_config config;
72557 |
72558 |     config = ma_resource_manager_data_source_config_init();
72559 |     config.pFilePathW = pName;
72560 |     config.flags = flags;
72561 |     config.pNotifications = pNotifications;
72562 |
72563 |     return ma_resource_manager_data_source_init_ex(pResourceManager, &config, pDataSource);
72564 | }
72565 |
72566 | MA_API ma_result ma_resource_manager_data_source_init_copy(ma_resource_manager* pResourceManager,
> const ma_resource_manager_data_source* pExistingDataSource, ma_resource_manager_data_source*
> pDataSource)
72567 | {
72568 |     ma_result result;

```

```

72569 |     ma_resource_manager_data_source_config config;
72570 |
72571 |     if (pExistingDataSource == NULL) {
72572 |         return MA_INVALID_ARGS;
72573 |     }
72574 |
72575 |     config = ma_resource_manager_data_source_config_init();
72576 |     config.flags = pExistingDataSource->flags;
72577 |
72578 |     result = ma_resource_manager_data_source_preinit(pResourceManager, &config, pDataSource);
72579 |     if (result != MA_SUCCESS) {
72580 |         return result;
72581 |     }
72582 |
72583 |     /* Copying can only be done from data buffers. Streams cannot be copied. */
72584 |     if ((pExistingDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72585 |         return MA_INVALID_OPERATION;
72586 |     }
72587 |
72588 |     return ma_resource_manager_data_buffer_init_copy(pResourceManager,
> &pExistingDataSource->backend.buffer, &pDataSource->backend.buffer);
72589 | }
72590 |
72591 | MA_API ma_result ma_resource_manager_data_source_uninit(ma_resource_manager_data_source*
> pDataSource)
72592 | {
72593 |     if (pDataSource == NULL) {
72594 |         return MA_INVALID_ARGS;
72595 |     }
72596 |
72597 |     /* All we need to is uninitialized the underlying data buffer or data stream. */
72598 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72599 |         return ma_resource_manager_data_stream_uninit(&pDataSource->backend.stream);
72600 |     } else {
72601 |         return ma_resource_manager_data_buffer_uninit(&pDataSource->backend.buffer);
72602 |     }
72603 | }
72604 |
72605 | MA_API ma_result ma_resource_manager_data_source_read_pcm_frames(ma_resource_manager_data_source*
> pDataSource, void* pFramesOut, ma_uint64 frameCount, ma_uint64* pFramesRead)
72606 | {
72607 |     /* Safety. */
72608 |     if (pFramesRead != NULL) {
72609 |         *pFramesRead = 0;
72610 |     }
72611 |
72612 |     if (pDataSource == NULL) {
72613 |         return MA_INVALID_ARGS;
72614 |     }
72615 |
72616 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72617 |         return ma_resource_manager_data_stream_read_pcm_frames(&pDataSource->backend.stream,
> pFramesOut, frameCount, pFramesRead);
72618 |     } else {
72619 |         return ma_resource_manager_data_buffer_read_pcm_frames(&pDataSource->backend.buffer,
> pFramesOut, frameCount, pFramesRead);
72620 |     }
72621 | }
72622 |
72623 | MA_API ma_result ma_resource_manager_data_source_seek_to_pcm_frame(ma_resource_manager_data_source*
> pDataSource, ma_uint64 frameIndex)
72624 | {
72625 |     if (pDataSource == NULL) {
72626 |         return MA_INVALID_ARGS;
72627 |     }
72628 |
72629 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72630 |         return ma_resource_manager_data_stream_seek_to_pcm_frame(&pDataSource->backend.stream,
> frameIndex);
72631 |     } else {
72632 |         return ma_resource_manager_data_buffer_seek_to_pcm_frame(&pDataSource->backend.buffer,
> frameIndex);
72633 |     }
72634 | }
72635 |
72636 | MA_API ma_result ma_resource_manager_data_source_map(ma_resource_manager_data_source* pDataSource,
> void** ppFramesOut, ma_uint64* pFrameCount)
72637 | {
72638 |     if (pDataSource == NULL) {

```

```

72639 |         return MA_INVALID_ARGS;
72640 |     }
72641 |
72642 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72643 |         return ma_resource_manager_data_stream_map(&pDataSource->backend.stream, ppFramesOut,
> pFrameCount);
72644 |     } else {
72645 |         return MA_NOT_IMPLEMENTED; /* Mapping not supported with data buffers. */
72646 |     }
72647 | }
72648 |
72649 | MA_API ma_result ma_resource_manager_data_source_unmap(ma_resource_manager_data_source* pDataSource,
> ma_uint64 frameCount)
72650 | {
72651 |     if (pDataSource == NULL) {
72652 |         return MA_INVALID_ARGS;
72653 |     }
72654 |
72655 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72656 |         return ma_resource_manager_data_stream_unmap(&pDataSource->backend.stream, frameCount);
72657 |     } else {
72658 |         return MA_NOT_IMPLEMENTED; /* Mapping not supported with data buffers. */
72659 |     }
72660 | }
72661 |
72662 | MA_API ma_result ma_resource_manager_data_source_get_data_format(ma_resource_manager_data_source*
> pDataSource, ma_format* pFormat, ma_uint32* pChannels, ma_uint32* pSampleRate, ma_channel*
> pChannelMap, size_t channelMapCap)
72663 | {
72664 |     if (pDataSource == NULL) {
72665 |         return MA_INVALID_ARGS;
72666 |     }
72667 |
72668 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72669 |         return ma_resource_manager_data_stream_get_data_format(&pDataSource->backend.stream,
> pFormat, pChannels, pSampleRate, pChannelMap, channelMapCap);
72670 |     } else {
72671 |         return ma_resource_manager_data_buffer_get_data_format(&pDataSource->backend.buffer,
> pFormat, pChannels, pSampleRate, pChannelMap, channelMapCap);
72672 |     }
72673 | }
72674 |
72675 | MA_API ma_result
> ma_resource_manager_data_source_get_cursor_in_pcm_frames(ma_resource_manager_data_source*
> pDataSource, ma_uint64* pCursor)
72676 | {
72677 |     if (pDataSource == NULL) {
72678 |         return MA_INVALID_ARGS;
72679 |     }
72680 |
72681 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72682 |         return
> ma_resource_manager_data_stream_get_cursor_in_pcm_frames(&pDataSource->backend.stream, pCursor);
72683 |     } else {
72684 |         return
> ma_resource_manager_data_buffer_get_cursor_in_pcm_frames(&pDataSource->backend.buffer, pCursor);
72685 |     }
72686 | }
72687 |
72688 | MA_API ma_result
> ma_resource_manager_data_source_get_length_in_pcm_frames(ma_resource_manager_data_source*
> pDataSource, ma_uint64* pLength)
72689 | {
72690 |     if (pDataSource == NULL) {
72691 |         return MA_INVALID_ARGS;
72692 |     }
72693 |
72694 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72695 |         return
> ma_resource_manager_data_stream_get_length_in_pcm_frames(&pDataSource->backend.stream, pLength);
72696 |     } else {
72697 |         return
> ma_resource_manager_data_buffer_get_length_in_pcm_frames(&pDataSource->backend.buffer, pLength);
72698 |     }
72699 | }
72700 |
72701 | MA_API ma_result ma_resource_manager_data_source_result(const ma_resource_manager_data_source*
> pDataSource)
72702 | {

```

```

72703 |     if (pDataSource == NULL) {
72704 |         return MA_INVALID_ARGS;
72705 |     }
72706 |
72707 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72708 |         return ma_resource_manager_data_stream_result(&pDataSource->backend.stream);
72709 |     } else {
72710 |         return ma_resource_manager_data_buffer_result(&pDataSource->backend.buffer);
72711 |     }
72712 | }
72713 |
72714 | MA_API ma_result ma_resource_manager_data_source_set_looping(ma_resource_manager_data_source*
> pDataSource, ma_bool32 isLooping)
72715 | {
72716 |     if (pDataSource == NULL) {
72717 |         return MA_INVALID_ARGS;
72718 |     }
72719 |
72720 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72721 |         return ma_resource_manager_data_stream_set_looping(&pDataSource->backend.stream, isLooping);
72722 |     } else {
72723 |         return ma_resource_manager_data_buffer_set_looping(&pDataSource->backend.buffer, isLooping);
72724 |     }
72725 | }
72726 |
72727 | MA_API ma_bool32 ma_resource_manager_data_source_is_looping(const ma_resource_manager_data_source*
> pDataSource)
72728 | {
72729 |     if (pDataSource == NULL) {
72730 |         return MA_FALSE;
72731 |     }
72732 |
72733 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72734 |         return ma_resource_manager_data_stream_is_looping(&pDataSource->backend.stream);
72735 |     } else {
72736 |         return ma_resource_manager_data_buffer_is_looping(&pDataSource->backend.buffer);
72737 |     }
72738 | }
72739 |
72740 | MA_API ma_result
> ma_resource_manager_data_source_get_available_frames(ma_resource_manager_data_source* pDataSource,
> ma_uint64* pAvailableFrames)
72741 | {
72742 |     if (pAvailableFrames == NULL) {
72743 |         return MA_INVALID_ARGS;
72744 |     }
72745 |
72746 |     *pAvailableFrames = 0;
72747 |
72748 |     if (pDataSource == NULL) {
72749 |         return MA_INVALID_ARGS;
72750 |     }
72751 |
72752 |     if ((pDataSource->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_STREAM) != 0) {
72753 |         return ma_resource_manager_data_stream_get_available_frames(&pDataSource->backend.stream,
> pAvailableFrames);
72754 |     } else {
72755 |         return ma_resource_manager_data_buffer_get_available_frames(&pDataSource->backend.buffer,
> pAvailableFrames);
72756 |     }
72757 | }
72758 |
72759 |
72760 | MA_API ma_result ma_resource_manager_post_job(ma_resource_manager* pResourceManager, const ma_job*
> pJob)
72761 | {
72762 |     if (pResourceManager == NULL) {
72763 |         return MA_INVALID_ARGS;
72764 |     }
72765 |
72766 |     return ma_job_queue_post(&pResourceManager->jobQueue, pJob);
72767 | }
72768 |
72769 | MA_API ma_result ma_resource_manager_post_job_quit(ma_resource_manager* pResourceManager)
72770 | {
72771 |     ma_job job = ma_job_init(MA_JOB_TYPE_QUIT);
72772 |     return ma_resource_manager_post_job(pResourceManager, &job);
72773 | }
72774 |

```

```

72775 | MA_API ma_result ma_resource_manager_next_job(ma_resource_manager* pResourceManager, ma_job* pJob)
72776 | {
72777 |     if (pResourceManager == NULL) {
72778 |         return MA_INVALID_ARGS;
72779 |     }
72780 |
72781 |     return ma_job_queue_next(&pResourceManager->jobQueue, pJob);
72782 | }
72783 |
72784 |
72785 | static ma_result ma_job_process__resource_manager__load_data_buffer_node(ma_job* pJob)
72786 | {
72787 |     ma_result result = MA_SUCCESS;
72788 |     ma_resource_manager* pResourceManager;
72789 |     ma_resource_manager_data_buffer_node* pDataBufferNode;
72790 |
72791 |     MA_ASSERT(pJob != NULL);
72792 |
72793 |     pResourceManager =
72794 | > (ma_resource_manager*)pJob->data.resourceManager.loadDataBufferNode.pResourceManager;
72795 |     MA_ASSERT(pResourceManager != NULL);
72796 |
72797 |     pDataBufferNode = (ma_resource_manager_data_buffer_node*)pJob->data.resourceManager.loadDataBuff
72798 | > erNode.pDataBufferNode;
72799 |     MA_ASSERT(pDataBufferNode != NULL);
72799 |     MA_ASSERT(pDataBufferNode->isDataOwnedByResourceManager == MA_TRUE); /* The data should always
72800 | > be owned by the resource manager. */
72801 |
72802 |     /* The data buffer is not getting deleted, but we may be getting executed out of order. If so,
72803 | > we need to push the job back onto the queue and return. */
72804 |     if (pJob->order != ma_atomic_load_32(&pDataBufferNode->executionPointer)) {
72805 |         return ma_resource_manager_post_job(pResourceManager, pJob); /* Attempting to execute out
72806 | > of order. Probably interleaved with a MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER job. */
72807 |     }
72808 |
72809 |     /* First thing we need to do is check whether or not the data buffer is getting deleted. If so
72810 | > we just abort. */
72811 |     if (ma_resource_manager_data_buffer_node_result(pDataBufferNode) != MA_BUSY) {
72812 |         result = ma_resource_manager_data_buffer_node_result(pDataBufferNode); /* The data buffer
72813 | > may be getting deleted before it's even been loaded. */
72814 |         goto done;
72815 |     }
72816 |
72817 |     /*
72818 |     We're ready to start loading. Essentially what we're doing here is initializing the data supply
72819 |     of the node. Once this is complete, data buffers can have their connectors initialized which
72820 |     will allow then to have audio data read from them.
72821 |
72822 |     Note that when the data supply type has been moved away from "unknown", that is when other
72823 | > threads
72824 |     will determine that the node is available for data delivery and the data buffer connectors can
72825 | > be
72826 |     initialized. Therefore, it's important that it is set after the data supply has been
72827 | > initialized.
72828 |     */
72829 |     if ((pJob->data.resourceManager.loadDataBufferNode.flags &
72830 | > MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_DECODE) != 0) {
72831 |         /*
72832 |         Decoding. This is the complex case because we're not going to be doing the entire decoding
72833 |         process here. Instead it's going to be split of multiple jobs and loaded in pages. The
72834 |         reason for this is to evenly distribute decoding time across multiple sounds, rather than
72835 |         having one huge sound hog all the available processing resources.
72836 |
72837 |         The first thing we do is initialize a decoder. This is allocated on the heap and is passed
72838 |         around to the paging jobs. When the last paging job has completed it's processing, it'll
72839 |         free the decoder for us.
72840 |
72841 |         This job does not do any actual decoding. It instead just posts a PAGE_DATA_BUFFER_NODE job
72842 |         which is where the actual decoding work will be done. However, once this job is complete,
72843 |         the node will be in a state where data buffer connectors can be initialized.
72844 |         */
72845 |         ma_decoder* pDecoder; /* <-- Free'd on the last page decode. */
72846 |         ma_job pageDataBufferNodeJob;
72847 |
72848 |         /* Allocate the decoder by initializing a decoded data supply. */
72849 |         result = ma_resource_manager_data_buffer_node_init_supply_decoded(pResourceManager,
72850 | > pDataBufferNode, pJob->data.resourceManager.loadDataBufferNode.pFilePath,
72851 | > pJob->data.resourceManager.loadDataBufferNode.pFilePathW,
72852 | > pJob->data.resourceManager.loadDataBufferNode.flags, &pDecoder);

```

```

72840 |
72841 |     /*
72842 |     Don't ever propagate an MA_BUSY result code or else the resource manager will think the
72843 |     node is just busy decoding rather than in an error state. This should never happen, but
72844 |     including this logic for safety just in case.
72845 |     */
72846 |     if (result == MA_BUSY) {
72847 |         result = MA_ERROR;
72848 |     }
72849 |
72850 |     if (result != MA_SUCCESS) {
72851 |         if (pJob->data.resourceManager.loadDataBufferNode.pFilePath != NULL) {
72852 |             ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_WARNING,
> "Failed to initialize data supply for \"%s\". %s.\n",
> pJob->data.resourceManager.loadDataBufferNode.pFilePath, ma_result_description(result));
72853 |         } else {
72854 |             #if (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L) || defined(_MSC_VER)
72855 |                 ma_log_postf(ma_resource_manager_get_log(pResourceManager),
> MA_LOG_LEVEL_WARNING, "Failed to initialize data supply for \"%ls\". %s.\n",
> pJob->data.resourceManager.loadDataBufferNode.pFilePathW, ma_result_description(result));
72856 |             #endif
72857 |         }
72858 |
72859 |         goto done;
72860 |     }
72861 |
72862 |     /*
72863 |     At this point the node's data supply is initialized and other threads can start initializing
72864 |     their data buffer connectors. However, no data will actually be available until we start to
72865 |     actually decode it. To do this, we need to post a paging job which is where the decoding
72866 |     work is done.
72867 |
72868 |     Note that if an error occurred at an earlier point, this section will have been skipped.
72869 |     */
72870 |     pageDataBufferNodeJob = ma_job_init(MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE);
72871 |     pageDataBufferNodeJob.order =
> ma_resource_manager_data_buffer_node_next_execution_order(pDataBufferNode);
72872 |     pageDataBufferNodeJob.data.resourceManager.pageDataBufferNode.pResourceManager =
> pResourceManager;
72873 |     pageDataBufferNodeJob.data.resourceManager.pageDataBufferNode.pDataBufferNode =
> pDataBufferNode;
72874 |     pageDataBufferNodeJob.data.resourceManager.pageDataBufferNode.pDecoder = pDecoder;
72875 |     pageDataBufferNodeJob.data.resourceManager.pageDataBufferNode.pDoneNotification =
> pJob->data.resourceManager.loadDataBufferNode.pDoneNotification;
72876 |     pageDataBufferNodeJob.data.resourceManager.pageDataBufferNode.pDoneFence =
> pJob->data.resourceManager.loadDataBufferNode.pDoneFence;
72877 |
72878 |     /* The job has been set up so it can now be posted. */
72879 |     result = ma_resource_manager_post_job(pResourceManager, &pageDataBufferNodeJob);
72880 |
72881 |     /*
72882 |     When we get here, we want to make sure the result code is set to MA_BUSY. The reason for
72883 |     this is that the result will be copied over to the node's internal result variable. In
72884 |     this case, since the decoding is still in-progress, we need to make sure the result code
72885 |     is set to MA_BUSY.
72886 |     */
72887 |     if (result != MA_SUCCESS) {
72888 |         ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_ERROR, "Failed
> to post MA_JOB_TYPE_RESOURCE_MANAGER_PAGE_DATA_BUFFER_NODE job. %s\n",
> ma_result_description(result));
72889 |         ma_decoder_uninit(pDecoder);
72890 |         ma_free(pDecoder, &pResourceManager->config.allocationCallbacks);
72891 |     } else {
72892 |         result = MA_BUSY;
72893 |     }
72894 | } else {
72895 |     /* No decoding. This is the simple case. We need only read the file content into memory and
> we're done. */
72896 |     result = ma_resource_manager_data_buffer_node_init_supply_encoded(pResourceManager,
> pDataBufferNode, pJob->data.resourceManager.loadDataBufferNode.pFilePath,
> pJob->data.resourceManager.loadDataBufferNode.pFilePathW);
72897 | }
72898 |
72899 |
72900 | done:
72901 |     /* File paths are no longer needed. */
72902 |     ma_free(pJob->data.resourceManager.loadDataBufferNode.pFilePath,
> &pResourceManager->config.allocationCallbacks);
72903 |     ma_free(pJob->data.resourceManager.loadDataBufferNode.pFilePathW,

```



```

> &pResourceManager->config.allocationCallbacks);
72904 |
72905 |     /*
72906 |         We need to set the result to at the very end to ensure no other threads try reading the data
> before we've fully initialized the object. Other threads
72907 |         are going to be inspecting this variable to determine whether or not they're ready to read data.
> We can only change the result if it's set to MA_BUSY
72908 |         because otherwise we may be changing away from an error code which would be bad. An example is
> if the application creates a data buffer, but then
72909 |         immediately deletes it before we've got to this point. In this case, pDataBuffer->result will be
> MA_UNAVAILABLE, and setting it to MA_SUCCESS or any
72910 |         other error code would cause the buffer to look like it's in a state that it's not.
72911 |     */
72912 |     ma_atomic_compare_and_swap_i32(&pDataBufferNode->result, MA_BUSY, result);
72913 |
72914 |     /* At this point initialization is complete and we can signal the notification if any. */
72915 |     if (pJob->data.resourceManager.loadDataBufferNode.pInitNotification != NULL) {
72916 |
> ma_async_notification_signal(pJob->data.resourceManager.loadDataBufferNode.pInitNotification);
72917 |     }
72918 |     if (pJob->data.resourceManager.loadDataBufferNode.pInitFence != NULL) {
72919 |         ma_fence_release(pJob->data.resourceManager.loadDataBufferNode.pInitFence);
72920 |     }
72921 |
72922 |     /* If we have a success result it means we've fully loaded the buffer. This will happen in the
> non-decoding case. */
72923 |     if (result != MA_BUSY) {
72924 |         if (pJob->data.resourceManager.loadDataBufferNode.pDoneNotification != NULL) {
72925 |
> ma_async_notification_signal(pJob->data.resourceManager.loadDataBufferNode.pDoneNotification);
72926 |         }
72927 |         if (pJob->data.resourceManager.loadDataBufferNode.pDoneFence != NULL) {
72928 |             ma_fence_release(pJob->data.resourceManager.loadDataBufferNode.pDoneFence);
72929 |         }
72930 |     }
72931 |
72932 |     /* Increment the node's execution pointer so that the next jobs can be processed. This is how we
> keep decoding of pages in-order. */
72933 |     ma_atomic_fetch_add_32(&pDataBufferNode->executionPointer, 1);
72934 |
72935 |     /* A busy result should be considered successful from the point of view of the job system. */
72936 |     if (result == MA_BUSY) {
72937 |         result = MA_SUCCESS;
72938 |     }
72939 |
72940 |     return result;
72941 | }
72942 |
72943 | static ma_result ma_job_process__resource_manager__free_data_buffer_node(ma_job* pJob)
72944 | {
72945 |     ma_resource_manager* pResourceManager;
72946 |     ma_resource_manager_data_buffer_node* pDataBufferNode;
72947 |
72948 |     MA_ASSERT(pJob != NULL);
72949 |
72950 |     pResourceManager =
> (ma_resource_manager*)pJob->data.resourceManager.freeDataBufferNode.pResourceManager;
72951 |     MA_ASSERT(pResourceManager != NULL);
72952 |
72953 |     pDataBufferNode = (ma_resource_manager_data_buffer_node*)pJob->data.resourceManager.freeDataBuff
> erNode.pDataBufferNode;
72954 |     MA_ASSERT(pDataBufferNode != NULL);
72955 |
72956 |     if (pJob->order != ma_atomic_load_32(&pDataBufferNode->executionPointer)) {
72957 |         return ma_resource_manager_post_job(pResourceManager, pJob);    /* Out of order. */
72958 |     }
72959 |
72960 |     /* The event needs to be signalled last. */
72961 |     if (pJob->data.resourceManager.freeDataBufferNode.pDoneNotification != NULL) {
72962 |
> ma_async_notification_signal(pJob->data.resourceManager.freeDataBufferNode.pDoneNotification);
72963 |     }
72964 |
72965 |     if (pJob->data.resourceManager.freeDataBufferNode.pDoneFence != NULL) {
72966 |         ma_fence_release(pJob->data.resourceManager.freeDataBufferNode.pDoneFence);
72967 |     }
72968 |
72969 |     ma_atomic_fetch_add_32(&pDataBufferNode->executionPointer, 1);
72970 |

```

```

72971 |     ma_resource_manager_data_buffer_node_free(pResourceManager, pDataBufferNode);
72972 |
72973 |     return MA_SUCCESS;
72974 | }
72975 |
72976 | static ma_result ma_job_process__resource_manager__page_data_buffer_node(ma_job* pJob)
72977 | {
72978 |     ma_result result = MA_SUCCESS;
72979 |     ma_resource_manager* pResourceManager;
72980 |     ma_resource_manager_data_buffer_node* pDataBufferNode;
72981 |
72982 |     MA_ASSERT(pJob != NULL);
72983 |
72984 |     pResourceManager =
> (ma_resource_manager*)pJob->data.resourceManager.pageDataBufferNode.pResourceManager;
72985 |     MA_ASSERT(pResourceManager != NULL);
72986 |
72987 |     pDataBufferNode = (ma_resource_manager_data_buffer_node*)pJob->data.resourceManager.pageDataBuff
> erNode.pDataBufferNode;
72988 |     MA_ASSERT(pDataBufferNode != NULL);
72989 |
72990 |     if (pJob->order != ma_atomic_load_32(&pDataBufferNode->executionPointer)) {
72991 |         return ma_resource_manager_post_job(pResourceManager, pJob);    /* Out of order. */
72992 |     }
72993 |
72994 |     /* Don't do any more decoding if the data buffer has started the uninitialization process. */
72995 |     result = ma_resource_manager_data_buffer_node_result(pDataBufferNode);
72996 |     if (result != MA_BUSY) {
72997 |         goto done;
72998 |     }
72999 |
73000 |     /* We're ready to decode the next page. */
73001 |     result = ma_resource_manager_data_buffer_node_decode_next_page(pResourceManager,
> pDataBufferNode, (ma_decoder*)pJob->data.resourceManager.pageDataBufferNode.pDecoder);
73002 |
73003 |     /*
73004 |     If we have a success code by this point, we want to post another job. We're going to set the
73005 |     result back to MA_BUSY to make it clear that there's still more to load.
73006 |     */
73007 |     if (result == MA_SUCCESS) {
73008 |         ma_job newJob;
73009 |         newJob = *pJob; /* Everything is the same as the input job, except the execution order. */
73010 |         newJob.order = ma_resource_manager_data_buffer_node_next_execution_order(pDataBufferNode);
> /* We need a fresh execution order. */
73011 |
73012 |         result = ma_resource_manager_post_job(pResourceManager, &newJob);
73013 |
73014 |         /* Since the sound isn't yet fully decoded we want the status to be set to busy. */
73015 |         if (result == MA_SUCCESS) {
73016 |             result = MA_BUSY;
73017 |         }
73018 |     }
73019 |
73020 | done:
73021 |     /* If there's still more to decode the result will be set to MA_BUSY. Otherwise we can free the
> decoder. */
73022 |     if (result != MA_BUSY) {
73023 |         ma_decoder_uninit((ma_decoder*)pJob->data.resourceManager.pageDataBufferNode.pDecoder);
73024 |         ma_free(pJob->data.resourceManager.pageDataBufferNode.pDecoder,
> &pResourceManager->config.allocationCallbacks);
73025 |     }
73026 |
73027 |     /* If we reached the end we need to treat it as successful. */
73028 |     if (result == MA_AT_END) {
73029 |         result = MA_SUCCESS;
73030 |     }
73031 |
73032 |     /* Make sure we set the result of node in case some error occurred. */
73033 |     ma_atomic_compare_and_swap_i32(&pDataBufferNode->result, MA_BUSY, result);
73034 |
73035 |     /* Signal the notification after setting the result in case the notification callback wants to
> inspect the result code. */
73036 |     if (result != MA_BUSY) {
73037 |         if (pJob->data.resourceManager.pageDataBufferNode.pDoneNotification != NULL) {
73038 |
> ma_async_notification_signal(pJob->data.resourceManager.pageDataBufferNode.pDoneNotification);
73039 |         }
73040 |
73041 |         if (pJob->data.resourceManager.pageDataBufferNode.pDoneFence != NULL) {

```

```

73042 |         ma_fence_release(pJob->data.resourceManager.pageDataBufferNode.pDoneFence));
73043 |     }
73044 | }
73045 |
73046 |     ma_atomic_fetch_add_32(&pDataBufferNode->executionPointer, 1);
73047 |     return result;
73048 | }
73049 |
73050 |
73051 | static ma_result ma_job_process__resource_manager__load_data_buffer(ma_job* pJob)
73052 | {
73053 |     ma_result result = MA_SUCCESS;
73054 |     ma_resource_manager* pResourceManager;
73055 |     ma_resource_manager_data_buffer* pDataBuffer;
73056 |     ma_resource_manager_data_supply_type dataSupplyType =
> ma_resource_manager_data_supply_type_unknown;
73057 |     ma_bool32 isConnectorInitialized = MA_FALSE;
73058 |
73059 |     /*
73060 |     All we're doing here is checking if the node has finished loading. If not, we just re-post the
> job
73061 |     and keep waiting. Otherwise we increment the execution counter and set the buffer's result code.
73062 |     */
73063 |     MA_ASSERT(pJob != NULL);
73064 |
73065 |     pDataBuffer =
> (ma_resource_manager_data_buffer*)pJob->data.resourceManager.loadDataBuffer.pDataBuffer;
73066 |     MA_ASSERT(pDataBuffer != NULL);
73067 |
73068 |     pResourceManager = pDataBuffer->pResourceManager;
73069 |
73070 |     if (pJob->order != ma_atomic_load_32(&pDataBuffer->executionPointer)) {
73071 |         return ma_resource_manager_post_job(pResourceManager, pJob); /* Attempting to execute out
> of order. Probably interleaved with a MA_JOB_TYPE_RESOURCE_MANAGER_FREE_DATA_BUFFER job. */
73072 |     }
73073 |
73074 |     /*
73075 |     First thing we need to do is check whether or not the data buffer is getting deleted. If so we
73076 |     just abort, but making sure we increment the execution pointer.
73077 |     */
73078 |     result = ma_resource_manager_data_buffer_result(pDataBuffer);
73079 |     if (result != MA_BUSY) {
73080 |         goto done; /* <-- This will ensure the execution pointer is incremented. */
73081 |     } else {
73082 |         result = MA_SUCCESS; /* <-- Make sure this is reset. */
73083 |         (void)result; /* <-- This is to suppress a static analysis diagnostic about
> "result" not being used. But for safety when I do future maintenance I don't want to delete that
> assignment. */
73084 |     }
73085 |
73086 |     /* Try initializing the connector if we haven't already. */
73087 |     isConnectorInitialized = ma_resource_manager_data_buffer_has_connector(pDataBuffer);
73088 |     if (isConnectorInitialized == MA_FALSE) {
73089 |         dataSupplyType =
> ma_resource_manager_data_buffer_node_get_data_supply_type(pDataBuffer->pNode);
73090 |
73091 |         if (dataSupplyType != ma_resource_manager_data_supply_type_unknown) {
73092 |             /* We can now initialize the connector. If this fails, we need to abort. It's very rare
> for this to fail. */
73093 |             ma_resource_manager_data_source_config dataSourceConfig; /* For setting initial
> looping state and range. */
73094 |             dataSourceConfig = ma_resource_manager_data_source_config_init();
73095 |             dataSourceConfig.rangeBegInPCMFrames =
> pJob->data.resourceManager.loadDataBuffer.rangeBegInPCMFrames;
73096 |             dataSourceConfig.rangeEndInPCMFrames =
> pJob->data.resourceManager.loadDataBuffer.rangeEndInPCMFrames;
73097 |             dataSourceConfig.loopPointBegInPCMFrames =
> pJob->data.resourceManager.loadDataBuffer.loopPointBegInPCMFrames;
73098 |             dataSourceConfig.loopPointEndInPCMFrames =
> pJob->data.resourceManager.loadDataBuffer.loopPointEndInPCMFrames;
73099 |             dataSourceConfig.isLooping =
> pJob->data.resourceManager.loadDataBuffer.isLooping;
73100 |
73101 |             result = ma_resource_manager_data_buffer_init_connector(pDataBuffer, &dataSourceConfig,
> pJob->data.resourceManager.loadDataBuffer.pInitNotification,
> pJob->data.resourceManager.loadDataBuffer.pInitFence);
73102 |             if (result != MA_SUCCESS) {
73103 |                 ma_log_postf(ma_resource_manager_get_log(pResourceManager), MA_LOG_LEVEL_ERROR,
> "Failed to initialize connector for data buffer. %s.\n", ma_result_description(result));

```

```

73104 |         goto done;
73105 |     }
73106 |     } else {
73107 |         /* Don't have a known data supply type. Most likely the data buffer node is still
> loading, but it could be that an error occurred. */
73108 |     }
73109 |     } else {
73110 |         /* The connector is already initialized. Nothing to do here. */
73111 |     }
73112 |
73113 |     /*
73114 |     If the data node is still loading, we need to repost the job and *not* increment the execution
73115 |     pointer (i.e. we need to not fall through to the "done" label).
73116 |
73117 |     There is a hole between here and the where the data connector is initialized where the data
73118 |     buffer node may have finished initializing. We need to check for this by checking the result of
73119 |     the data buffer node and whether or not we had an unknown data supply type at the time of
73120 |     trying to initialize the data connector.
73121 |     */
73122 |     result = ma_resource_manager_data_buffer_node_result(pDataBuffer->pNode);
73123 |     if (result == MA_BUSY || (result == MA_SUCCESS && isConnectorInitialized == MA_FALSE &&
> dataSupplyType == ma_resource_manager_data_supply_type_unknown)) {
73124 |         return ma_resource_manager_post_job(pResourceManager, pJob);
73125 |     }
73126 |
73127 | done:
73128 |     /* Only move away from a busy code so that we don't trash any existing error codes. */
73129 |     ma_atomic_compare_and_swap_i32(&pDataBuffer->result, MA_BUSY, result);
73130 |
73131 |     /* Only signal the other threads after the result has been set just for cleanliness sake. */
73132 |     if (pJob->data.resourceManager.loadDataBuffer.pDoneNotification != NULL) {
73133 |         ma_async_notification_signal(pJob->data.resourceManager.loadDataBuffer.pDoneNotification);
73134 |     }
73135 |     if (pJob->data.resourceManager.loadDataBuffer.pDoneFence != NULL) {
73136 |         ma_fence_release(pJob->data.resourceManager.loadDataBuffer.pDoneFence);
73137 |     }
73138 |
73139 |     /*
73140 |     If at this point the data buffer has not had it's connector initialized, it means the
73141 |     notification event was never signalled which means we need to signal it here.
73142 |     */
73143 |     if (ma_resource_manager_data_buffer_has_connector(pDataBuffer) == MA_FALSE && result !=
> MA_SUCCESS) {
73144 |         if (pJob->data.resourceManager.loadDataBuffer.pInitNotification != NULL) {
73145 |             > ma_async_notification_signal(pJob->data.resourceManager.loadDataBuffer.pInitNotification);
73146 |         }
73147 |         if (pJob->data.resourceManager.loadDataBuffer.pInitFence != NULL) {
73148 |             ma_fence_release(pJob->data.resourceManager.loadDataBuffer.pInitFence);
73149 |         }
73150 |     }
73151 |
73152 |     ma_atomic_fetch_add_32(&pDataBuffer->executionPointer, 1);
73153 |     return result;
73154 | }
73155 |
73156 | static ma_result ma_job_process__resource_manager__free_data_buffer(ma_job* pJob)
73157 | {
73158 |     ma_resource_manager* pResourceManager;
73159 |     ma_resource_manager_data_buffer* pDataBuffer;
73160 |
73161 |     MA_ASSERT(pJob != NULL);
73162 |
73163 |     pDataBuffer =
> (ma_resource_manager_data_buffer*)pJob->data.resourceManager.freeDataBuffer.pDataBuffer;
73164 |     MA_ASSERT(pDataBuffer != NULL);
73165 |
73166 |     pResourceManager = pDataBuffer->pResourceManager;
73167 |
73168 |     if (pJob->order != ma_atomic_load_32(&pDataBuffer->executionPointer)) {
73169 |         return ma_resource_manager_post_job(pResourceManager, pJob); /* Out of order. */
73170 |     }
73171 |
73172 |     ma_resource_manager_data_buffer_uninit_internal(pDataBuffer);
73173 |
73174 |     /* The event needs to be signalled last. */
73175 |     if (pJob->data.resourceManager.freeDataBuffer.pDoneNotification != NULL) {
73176 |         ma_async_notification_signal(pJob->data.resourceManager.freeDataBuffer.pDoneNotification);
73177 |     }

```

```

73178 |
73179 |     if (pJob->data.resourceManager.freeDataBuffer.pDoneFence != NULL) {
73180 |         ma_fence_release(pJob->data.resourceManager.freeDataBuffer.pDoneFence);
73181 |     }
73182 |
73183 |     ma_atomic_fetch_add_32(&pDataBuffer->executionPointer, 1);
73184 |     return MA_SUCCESS;
73185 | }
73186 |
73187 | static ma_result ma_job_process__resource_manager__load_data_stream(ma_job* pJob)
73188 | {
73189 |     ma_result result = MA_SUCCESS;
73190 |     ma_decoder_config decoderConfig;
73191 |     ma_uint32 pageBufferSizeInBytes;
73192 |     ma_resource_manager* pResourceManager;
73193 |     ma_resource_manager_data_stream* pDataStream;
73194 |
73195 |     MA_ASSERT(pJob != NULL);
73196 |
73197 |     pDataStream =
> (ma_resource_manager_data_stream*)pJob->data.resourceManager.loadDataStream.pDataStream;
73198 |     MA_ASSERT(pDataStream != NULL);
73199 |
73200 |     pResourceManager = pDataStream->pResourceManager;
73201 |
73202 |     if (pJob->order != ma_atomic_load_32(&pDataStream->executionPointer)) {
73203 |         return ma_resource_manager_post_job(pResourceManager, pJob);    /* Out of order. */
73204 |     }
73205 |
73206 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_BUSY) {
73207 |         result = MA_INVALID_OPERATION; /* Most likely the data stream is being uninitialized. */
73208 |         goto done;
73209 |     }
73210 |
73211 |     /* We need to initialize the decoder first so we can determine the size of the pages. */
73212 |     decoderConfig = ma_resource_manager__init_decoder_config(pResourceManager);
73213 |
73214 |     if (pJob->data.resourceManager.loadDataStream.pFilePath != NULL) {
73215 |         result = ma_decoder_init_vfs(pResourceManager->config.pVFS,
> pJob->data.resourceManager.loadDataStream.pFilePath, &decoderConfig, &pDataStream->decoder);
73216 |     } else {
73217 |         result = ma_decoder_init_vfs_w(pResourceManager->config.pVFS,
> pJob->data.resourceManager.loadDataStream.pFilePathW, &decoderConfig, &pDataStream->decoder);
73218 |     }
73219 |     if (result != MA_SUCCESS) {
73220 |         goto done;
73221 |     }
73222 |
73223 |     /* Retrieve the total length of the file before marking the decoder as loaded. */
73224 |     if ((pDataStream->flags & MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_UNKNOWN_LENGTH) == 0) {
73225 |         result = ma_decoder_get_length_in_pcm_frames(&pDataStream->decoder,
> &pDataStream->totalLengthInPCMFrames);
73226 |         if (result != MA_SUCCESS) {
73227 |             goto done; /* Failed to retrieve the length. */
73228 |         }
73229 |     } else {
73230 |         pDataStream->totalLengthInPCMFrames = 0;
73231 |     }
73232 |
73233 |     /*
73234 |     Only mark the decoder as initialized when the length of the decoder has been retrieved because
> that can possibly require a scan over the whole file
73235 |     and we don't want to have another thread trying to access the decoder while it's scanning.
73236 |     */
73237 |     pDataStream->isDecoderInitialized = MA_TRUE;
73238 |
73239 |     /* We have the decoder so we can now initialize our page buffer. */
73240 |     pageBufferSizeInBytes = ma_resource_manager_data_stream_get_page_size_in_frames(pDataStream) * 2
> * ma_get_bytes_per_frame(pDataStream->decoder.outputFormat, pDataStream->decoder.outputChannels);
73241 |
73242 |     pDataStream->pPageData = ma_malloc(pageBufferSizeInBytes,
> &pResourceManager->config.allocationCallbacks);
73243 |     if (pDataStream->pPageData == NULL) {
73244 |         ma_decoder_uninit(&pDataStream->decoder);
73245 |         result = MA_OUT_OF_MEMORY;
73246 |         goto done;
73247 |     }
73248 |
73249 |     /* Seek to our initial seek point before filling the initial pages. */

```

```

73250 |     ma_decoder_seek_to_pcm_frame(&pDataStream->decoder,
> pJob->data.resourceManager.loadDataStream.initialSeekPoint);
73251 |
73252 |     /* We have our decoder and our page buffer, so now we need to fill our pages. */
73253 |     ma_resource_manager_data_stream_fill_pages(pDataStream);
73254 |
73255 |     /* And now we're done. We want to make sure the result is MA_SUCCESS. */
73256 |     result = MA_SUCCESS;
73257 |
73258 | done:
73259 |     ma_free(pJob->data.resourceManager.loadDataStream.pFilePath,
> &pResourceManager->config.allocationCallbacks);
73260 |     ma_free(pJob->data.resourceManager.loadDataStream.pFilePathW,
> &pResourceManager->config.allocationCallbacks);
73261 |
73262 |     /* We can only change the status away from MA_BUSY. If it's set to anything else it means an
> error has occurred somewhere or the uninitialization process has started (most likely). */
73263 |     ma_atomic_compare_and_swap_i32(&pDataStream->result, MA_BUSY, result);
73264 |
73265 |     /* Only signal the other threads after the result has been set just for cleanliness sake. */
73266 |     if (pJob->data.resourceManager.loadDataStream.pInitNotification != NULL) {
73267 |         ma_async_notification_signal(pJob->data.resourceManager.loadDataStream.pInitNotification);
73268 |     }
73269 |     if (pJob->data.resourceManager.loadDataStream.pInitFence != NULL) {
73270 |         ma_fence_release(pJob->data.resourceManager.loadDataStream.pInitFence);
73271 |     }
73272 |
73273 |     ma_atomic_fetch_add_32(&pDataStream->executionPointer, 1);
73274 |     return result;
73275 | }
73276 |
73277 | static ma_result ma_job_process__resource_manager__free_data_stream(ma_job* pJob)
73278 | {
73279 |     ma_resource_manager* pResourceManager;
73280 |     ma_resource_manager_data_stream* pDataStream;
73281 |
73282 |     MA_ASSERT(pJob != NULL);
73283 |
73284 |     pDataStream =
> (ma_resource_manager_data_stream*)pJob->data.resourceManager.freeDataStream.pDataStream;
73285 |     MA_ASSERT(pDataStream != NULL);
73286 |
73287 |     pResourceManager = pDataStream->pResourceManager;
73288 |
73289 |     if (pJob->order != ma_atomic_load_32(&pDataStream->executionPointer)) {
73290 |         return ma_resource_manager_post_job(pResourceManager, pJob);    /* Out of order. */
73291 |     }
73292 |
73293 |     /* If our status is not MA_UNAVAILABLE we have a bug somewhere. */
73294 |     MA_ASSERT(ma_resource_manager_data_stream_result(pDataStream) == MA_UNAVAILABLE);
73295 |
73296 |     if (pDataStream->isDecoderInitialized) {
73297 |         ma_decoder_uninit(&pDataStream->decoder);
73298 |     }
73299 |
73300 |     if (pDataStream->pPageData != NULL) {
73301 |         ma_free(pDataStream->pPageData, &pResourceManager->config.allocationCallbacks);
73302 |         pDataStream->pPageData = NULL;    /* Just in case... */
73303 |     }
73304 |
73305 |     ma_data_source_uninit(&pDataStream->ds);
73306 |
73307 |     /* The event needs to be signalled last. */
73308 |     if (pJob->data.resourceManager.freeDataStream.pDoneNotification != NULL) {
73309 |         ma_async_notification_signal(pJob->data.resourceManager.freeDataStream.pDoneNotification);
73310 |     }
73311 |     if (pJob->data.resourceManager.freeDataStream.pDoneFence != NULL) {
73312 |         ma_fence_release(pJob->data.resourceManager.freeDataStream.pDoneFence);
73313 |     }
73314 |
73315 |     /*ma_atomic_fetch_add_32(&pDataStream->executionPointer, 1);*/
73316 |     return MA_SUCCESS;
73317 | }
73318 |
73319 | static ma_result ma_job_process__resource_manager__page_data_stream(ma_job* pJob)
73320 | {
73321 |     ma_result result = MA_SUCCESS;
73322 |     ma_resource_manager* pResourceManager;
73323 |     ma_resource_manager_data_stream* pDataStream;

```

```

73324 |
73325 |     MA_ASSERT(pJob != NULL);
73326 |
73327 |     pDataStream =
> (ma_resource_manager_data_stream*)pJob->data.resourceManager.pageDataStream.pDataStream;
73328 |     MA_ASSERT(pDataStream != NULL);
73329 |
73330 |     pResourceManager = pDataStream->pResourceManager;
73331 |
73332 |     if (pJob->order != ma_atomic_load_32(&pDataStream->executionPointer)) {
73333 |         return ma_resource_manager_post_job(pResourceManager, pJob);    /* Out of order. */
73334 |     }
73335 |
73336 |     /* For streams, the status should be MA_SUCCESS. */
73337 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_SUCCESS) {
73338 |         result = MA_INVALID_OPERATION;
73339 |         goto done;
73340 |     }
73341 |
73342 |     ma_resource_manager_data_stream_fill_page(pDataStream,
> pJob->data.resourceManager.pageDataStream.pageIndex);
73343 |
73344 | done:
73345 |     ma_atomic_fetch_add_32(&pDataStream->executionPointer, 1);
73346 |     return result;
73347 | }
73348 |
73349 | static ma_result ma_job_process__resource_manager__seek_data_stream(ma_job* pJob)
73350 | {
73351 |     ma_result result = MA_SUCCESS;
73352 |     ma_resource_manager* pResourceManager;
73353 |     ma_resource_manager_data_stream* pDataStream;
73354 |
73355 |     MA_ASSERT(pJob != NULL);
73356 |
73357 |     pDataStream =
> (ma_resource_manager_data_stream*)pJob->data.resourceManager.seekDataStream.pDataStream;
73358 |     MA_ASSERT(pDataStream != NULL);
73359 |
73360 |     pResourceManager = pDataStream->pResourceManager;
73361 |
73362 |     if (pJob->order != ma_atomic_load_32(&pDataStream->executionPointer)) {
73363 |         return ma_resource_manager_post_job(pResourceManager, pJob);    /* Out of order. */
73364 |     }
73365 |
73366 |     /* For streams the status should be MA_SUCCESS for this to do anything. */
73367 |     if (ma_resource_manager_data_stream_result(pDataStream) != MA_SUCCESS ||
> pDataStream->isDecoderInitialized == MA_FALSE) {
73368 |         result = MA_INVALID_OPERATION;
73369 |         goto done;
73370 |     }
73371 |
73372 |     /*
73373 |     With seeking we just assume both pages are invalid and the relative frame cursor at position 0.
> This is basically exactly the same as loading, except
73374 |     instead of initializing the decoder, we seek to a frame.
73375 |     */
73376 |     ma_decoder_seek_to_pcm_frame(&pDataStream->decoder,
> pJob->data.resourceManager.seekDataStream.frameIndex);
73377 |
73378 |     /* After seeking we'll need to reload the pages. */
73379 |     ma_resource_manager_data_stream_fill_pages(pDataStream);
73380 |
73381 |     /* We need to let the public API know that we're done seeking. */
73382 |     ma_atomic_fetch_sub_32(&pDataStream->seekCounter, 1);
73383 |
73384 | done:
73385 |     ma_atomic_fetch_add_32(&pDataStream->executionPointer, 1);
73386 |     return result;
73387 | }
73388 |
73389 | MA_API ma_result ma_resource_manager_process_job(ma_resource_manager* pResourceManager, ma_job*
> pJob)
73390 | {
73391 |     if (pResourceManager == NULL || pJob == NULL) {
73392 |         return MA_INVALID_ARGS;
73393 |     }
73394 |
73395 |     return ma_job_process(pJob);

```



```

73396 | }
73397 |
73398 | MA_API ma_result ma_resource_manager_process_next_job(ma_resource_manager* pResourceManager)
73399 | {
73400 |     ma_result result;
73401 |     ma_job job;
73402 |
73403 |     if (pResourceManager == NULL) {
73404 |         return MA_INVALID_ARGS;
73405 |     }
73406 |
73407 |     /* This will return MA_CANCELLED if the next job is a quit job. */
73408 |     result = ma_resource_manager_next_job(pResourceManager, &job);
73409 |     if (result != MA_SUCCESS) {
73410 |         return result;
73411 |     }
73412 |
73413 |     return ma_job_process(&job);
73414 | }
73415 | #else
73416 | /* We'll get here if the resource manager is being excluded from the build. We need to define the
> job processing callbacks as no-ops. */
73417 | static ma_result ma_job_process__resource_manager__load_data_buffer_node(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73418 | static ma_result ma_job_process__resource_manager__free_data_buffer_node(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73419 | static ma_result ma_job_process__resource_manager__page_data_buffer_node(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73420 | static ma_result ma_job_process__resource_manager__load_data_buffer(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73421 | static ma_result ma_job_process__resource_manager__free_data_buffer(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73422 | static ma_result ma_job_process__resource_manager__load_data_stream(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73423 | static ma_result ma_job_process__resource_manager__free_data_stream(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73424 | static ma_result ma_job_process__resource_manager__page_data_stream(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73425 | static ma_result ma_job_process__resource_manager__seek_data_stream(ma_job* pJob) { return
> ma_job_process__noop(pJob); }
73426 | #endif /* MA_NO_RESOURCE_MANAGER */
73427 |
73428 |
73429 | #ifndef MA_NO_NODE_GRAPH
73430 |
73431 | static ma_stack* ma_stack_init(size_t sizeInBytes, const ma_allocation_callbacks*
> pAllocationCallbacks)
73432 | {
73433 |     ma_stack* pStack;
73434 |
73435 |     if (sizeInBytes == 0) {
73436 |         return NULL;
73437 |     }
73438 |
73439 |     pStack = (ma_stack*)ma_malloc(sizeof(*pStack) - sizeof(pStack->_data) + sizeInBytes,
> pAllocationCallbacks);
73440 |     if (pStack == NULL) {
73441 |         return NULL;
73442 |     }
73443 |
73444 |     pStack->offset = 0;
73445 |     pStack->sizeInBytes = sizeInBytes;
73446 |
73447 |     return pStack;
73448 | }
73449 |
73450 | static void ma_stack_uninit(ma_stack* pStack, const ma_allocation_callbacks* pAllocationCallbacks)
73451 | {
73452 |     if (pStack == NULL) {
73453 |         return;
73454 |     }
73455 |
73456 |     ma_free(pStack, pAllocationCallbacks);
73457 | }
73458 |
73459 | static void* ma_stack_alloc(ma_stack* pStack, size_t sz)
73460 | {
73461 |     /* The size of the allocation is stored in the memory directly before the pointer. This needs to
> include padding to keep it aligned to ma_uintptr */

```

```

73462 | void* p = (void*)((char*)pStack->_data + pStack->offset);
73463 | size_t* pSize = (size_t*)p;
73464 |
73465 | sz = (sz + (sizeof(ma_uintptr) - 1)) & ~(sizeof(ma_uintptr) - 1); /* Padding. */
73466 | if (pStack->offset + sz + sizeof(size_t) > pStack->sizeInBytes) {
73467 |     return NULL; /* Out of memory. */
73468 | }
73469 |
73470 | pStack->offset += sz + sizeof(size_t);
73471 |
73472 | *pSize = sz;
73473 | return (void*)((char*)p + sizeof(size_t));
73474 | }
73475 |
73476 | static void ma_stack_free(ma_stack* pStack, void* p)
73477 | {
73478 |     size_t* pSize;
73479 |
73480 |     if (p == NULL) {
73481 |         return;
73482 |     }
73483 |
73484 |     pSize = (size_t*)p - 1;
73485 |     pStack->offset -= *pSize + sizeof(size_t);
73486 | }
73487 |
73488 |
73489 |
73490 | /* 10ms @ 48K = 480. Must never exceed 65535. */
73491 | #ifndef MA_DEFAULT_NODE_CACHE_CAP_IN_FRAMES_PER_BUS
73492 | #define MA_DEFAULT_NODE_CACHE_CAP_IN_FRAMES_PER_BUS 480
73493 | #endif
73494 |
73495 | #ifndef MA_DEFAULT_PREMIX_STACK_SIZE_PER_CHANNEL
73496 | #define MA_DEFAULT_PREMIX_STACK_SIZE_PER_CHANNEL 524288
73497 | #endif
73498 |
73499 | static ma_result ma_node_read_pcm_frames(ma_node* pNode, ma_uint32 outputBusIndex, float*
> pFramesOut, ma_uint32 frameCount, ma_uint32* pFramesRead, ma_uint64 globalTime);
73500 |
73501 | MA_API void ma_debug_fill_pcm_frames_with_sine_wave(float* pFramesOut, ma_uint32 frameCount,
> ma_format format, ma_uint32 channels, ma_uint32 sampleRate)
73502 | {
73503 |     #ifndef MA_NO_GENERATION
73504 |     {
73505 |         ma_waveform_config waveformConfig;
73506 |         ma_waveform waveform;
73507 |
73508 |         waveformConfig = ma_waveform_config_init(format, channels, sampleRate,
> ma_waveform_type_sine, 1.0, 400);
73509 |         ma_waveform_init(&waveformConfig, &waveform);
73510 |         ma_waveform_read_pcm_frames(&waveform, pFramesOut, frameCount, NULL);
73511 |     }
73512 |     #else
73513 |     {
73514 |         (void)pFramesOut;
73515 |         (void)frameCount;
73516 |         (void)format;
73517 |         (void)channels;
73518 |         (void)sampleRate;
73519 |         #if defined(MA_DEBUG_OUTPUT)
73520 |         {
73521 |             #if _MSC_VER
73522 |             #pragma message ("ma_debug_fill_pcm_frames_with_sine_wave() will do nothing because
> MA_NO_GENERATION is enabled.")
73523 |             #endif
73524 |         }
73525 |     }
73526 |     #endif
73527 | }
73528 |
73529 |
73530 |
73531 |
73532 | MA_API ma_node_graph_config ma_node_graph_config_init(ma_uint32 channels)
73533 | {
73534 |     ma_node_graph_config config;
73535 |
73536 |     MA_ZERO_OBJECT(&config);

```

```

73537 |     config.channels                = channels;
73538 |     config.processingSizeInFrames = 0;
73539 |
73540 |     return config;
73541 | }
73542 |
73543 |
73544 | static void ma_node_graph_set_is_reading(ma_node_graph* pNodeGraph, ma_bool32 isReading)
73545 | {
73546 |     MA_ASSERT(pNodeGraph != NULL);
73547 |     ma_atomic_exchange_32(&pNodeGraph->isReading, isReading);
73548 | }
73549 |
73550 | #if 0
73551 | static ma_bool32 ma_node_graph_is_reading(ma_node_graph* pNodeGraph)
73552 | {
73553 |     MA_ASSERT(pNodeGraph != NULL);
73554 |     return ma_atomic_load_32(&pNodeGraph->isReading);
73555 | }
73556 | #endif
73557 |
73558 |
73559 | static void ma_node_graph_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn,
> ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
73560 | {
73561 |     ma_node_graph* pNodeGraph = (ma_node_graph*)pNode;
73562 |     ma_uint64 framesRead;
73563 |
73564 |     ma_node_graph_read_pcm_frames(pNodeGraph, ppFramesOut[0], *pFrameCountOut, &framesRead);
73565 |
73566 |     *pFrameCountOut = (ma_uint32)framesRead;    /* Safe cast. */
73567 |
73568 |     (void)ppFramesIn;
73569 |     (void)pFrameCountIn;
73570 | }
73571 |
73572 | static ma_node_vtable g_node_graph_node_vtable =
73573 | {
73574 |     ma_node_graph_node_process_pcm_frames,
73575 |     NULL,    /* onGetRequiredInputFrameCount */
73576 |     0,    /* 0 input buses. */
73577 |     1,    /* 1 output bus. */
73578 |     0    /* Flags. */
73579 | };
73580 |
73581 | static void ma_node_graph_endpoint_process_pcm_frames(ma_node* pNode, const float** ppFramesIn,
> ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
73582 | {
73583 |     MA_ASSERT(pNode != NULL);
73584 |     MA_ASSERT(ma_node_get_input_bus_count(pNode) == 1);
73585 |     MA_ASSERT(ma_node_get_output_bus_count(pNode) == 1);
73586 |
73587 |     /* Input channel count needs to be the same as the output channel count. */
73588 |     MA_ASSERT(ma_node_get_input_channels(pNode, 0) == ma_node_get_output_channels(pNode, 0));
73589 |
73590 |     /* We don't need to do anything here because it's a passthrough. */
73591 |     (void)pNode;
73592 |     (void)ppFramesIn;
73593 |     (void)pFrameCountIn;
73594 |     (void)ppFramesOut;
73595 |     (void)pFrameCountOut;
73596 |
73597 |     #if 0
73598 |     /* The data has already been mixed. We just need to move it to the output buffer. */
73599 |     if (ppFramesIn != NULL) {
73600 |         ma_copy_pcm_frames(ppFramesOut[0], ppFramesIn[0], *pFrameCountOut, ma_format_f32,
> ma_node_get_output_channels(pNode, 0));
73601 |     }
73602 |     #endif
73603 | }
73604 |
73605 | static ma_node_vtable g_node_graph_endpoint_vtable =
73606 | {
73607 |     ma_node_graph_endpoint_process_pcm_frames,
73608 |     NULL,    /* onGetRequiredInputFrameCount */
73609 |     1,    /* 1 input bus. */
73610 |     1,    /* 1 output bus. */
73611 |     MA_NODE_FLAG_PASSTHROUGH    /* Flags. The endpoint is a passthrough. */
73612 | };

```

```

73613 |
73614 | MA_API ma_result ma_node_graph_init(const ma_node_graph_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_node_graph* pNodeGraph)
73615 | {
73616 |     ma_result result;
73617 |     ma_node_config baseConfig;
73618 |     ma_node_config endpointConfig;
73619 |
73620 |     if (pNodeGraph == NULL) {
73621 |         return MA_INVALID_ARGS;
73622 |     }
73623 |
73624 |     MA_ZERO_OBJECT(pNodeGraph);
73625 |     pNodeGraph->processingSizeInFrames = pConfig->processingSizeInFrames;
73626 |
73627 |     /* Base node so we can use the node graph as a node into another graph. */
73628 |     baseConfig = ma_node_config_init();
73629 |     baseConfig.vtable = &g_node_graph_node_vtable;
73630 |     baseConfig.pOutputChannels = &pConfig->channels;
73631 |
73632 |     result = ma_node_init(pNodeGraph, &baseConfig, pAllocationCallbacks, &pNodeGraph->base);
73633 |     if (result != MA_SUCCESS) {
73634 |         return result;
73635 |     }
73636 |
73637 |
73638 |     /* Endpoint. */
73639 |     endpointConfig = ma_node_config_init();
73640 |     endpointConfig.vtable = &g_node_graph_endpoint_vtable;
73641 |     endpointConfig.pInputChannels = &pConfig->channels;
73642 |     endpointConfig.pOutputChannels = &pConfig->channels;
73643 |
73644 |     result = ma_node_init(pNodeGraph, &endpointConfig, pAllocationCallbacks, &pNodeGraph->endpoint);
73645 |     if (result != MA_SUCCESS) {
73646 |         ma_node_uninit(&pNodeGraph->base, pAllocationCallbacks);
73647 |         return result;
73648 |     }
73649 |
73650 |
73651 |     /* Processing cache. */
73652 |     if (pConfig->processingSizeInFrames > 0) {
73653 |         pNodeGraph->pProcessingCache = (float*)ma_malloc(pConfig->processingSizeInFrames *
> pConfig->channels * sizeof(float), pAllocationCallbacks);
73654 |         if (pNodeGraph->pProcessingCache == NULL) {
73655 |             ma_node_uninit(&pNodeGraph->endpoint, pAllocationCallbacks);
73656 |             ma_node_uninit(&pNodeGraph->base, pAllocationCallbacks);
73657 |             return MA_OUT_OF_MEMORY;
73658 |         }
73659 |     }
73660 |
73661 |
73662 |     /*
73663 |     We need a pre-mix stack. The size of this stack is configurable via the config. The default
> value depends on the channel count.
73664 |     */
73665 |     {
73666 |         size_t preMixStackSizeInBytes = pConfig->preMixStackSizeInBytes;
73667 |         if (preMixStackSizeInBytes == 0) {
73668 |             preMixStackSizeInBytes = pConfig->channels * MA_DEFAULT_PREMIX_STACK_SIZE_PER_CHANNEL;
73669 |         }
73670 |
73671 |         pNodeGraph->pPreMixStack = ma_stack_init(preMixStackSizeInBytes, pAllocationCallbacks);
73672 |         if (pNodeGraph->pPreMixStack == NULL) {
73673 |             ma_node_uninit(&pNodeGraph->endpoint, pAllocationCallbacks);
73674 |             ma_node_uninit(&pNodeGraph->base, pAllocationCallbacks);
73675 |             if (pNodeGraph->pProcessingCache != NULL) {
73676 |                 ma_free(pNodeGraph->pProcessingCache, pAllocationCallbacks);
73677 |             }
73678 |
73679 |             return MA_OUT_OF_MEMORY;
73680 |         }
73681 |     }
73682 |
73683 |
73684 |     return MA_SUCCESS;
73685 | }
73686 |
73687 | MA_API void ma_node_graph_uninit(ma_node_graph* pNodeGraph, const ma_allocation_callbacks*
> pAllocationCallbacks)

```

```

73688 | {
73689 |     if (pNodeGraph == NULL) {
73690 |         return;
73691 |     }
73692 |
73693 |     ma_node_uninit(&pNodeGraph->endpoint, pAllocationCallbacks);
73694 |     ma_node_uninit(&pNodeGraph->base, pAllocationCallbacks);
73695 |
73696 |     if (pNodeGraph->pProcessingCache != NULL) {
73697 |         ma_free(pNodeGraph->pProcessingCache, pAllocationCallbacks);
73698 |         pNodeGraph->pProcessingCache = NULL;
73699 |     }
73700 |
73701 |     if (pNodeGraph->pPreMixStack != NULL) {
73702 |         ma_stack_uninit(pNodeGraph->pPreMixStack, pAllocationCallbacks);
73703 |         pNodeGraph->pPreMixStack = NULL;
73704 |     }
73705 | }
73706 |
73707 | MA_API ma_node* ma_node_graph_get_endpoint(ma_node_graph* pNodeGraph)
73708 | {
73709 |     if (pNodeGraph == NULL) {
73710 |         return NULL;
73711 |     }
73712 |
73713 |     return &pNodeGraph->endpoint;
73714 | }
73715 |
73716 | MA_API ma_result ma_node_graph_read_pcm_frames(ma_node_graph* pNodeGraph, void* pFramesOut,
73717 | > ma_uint64 frameCount, ma_uint64* pFramesRead)
73718 | {
73719 |     ma_result result = MA_SUCCESS;
73720 |     ma_uint64 totalFramesRead;
73721 |     ma_uint32 channels;
73722 |
73723 |     if (pFramesRead != NULL) {
73724 |         *pFramesRead = 0; /* Safety. */
73725 |     }
73726 |
73727 |     if (pNodeGraph == NULL) {
73728 |         return MA_INVALID_ARGS;
73729 |     }
73730 |
73731 |     channels = ma_node_get_output_channels(&pNodeGraph->endpoint, 0);
73732 |
73733 |     /* We'll be nice and try to do a full read of all frameCount frames. */
73734 |     totalFramesRead = 0;
73735 |     while (totalFramesRead < frameCount) {
73736 |         ma_uint32 framesJustRead;
73737 |         ma_uint64 framesToRead;
73738 |         float* pRunningFramesOut;
73739 |
73740 |         framesToRead = frameCount - totalFramesRead;
73741 |         if (framesToRead > 0xFFFFFFFF) {
73742 |             framesToRead = 0xFFFFFFFF;
73743 |         }
73744 |
73745 |         pRunningFramesOut = (float*)ma_offset_pcm_frames_ptr(pFramesOut, totalFramesRead,
73746 | > ma_format_f32, channels);
73747 |
73748 |         /* If there's anything in the cache, consume that first. */
73749 |         if (pNodeGraph->processingCacheFramesRemaining > 0) {
73750 |             ma_uint32 framesToReadFromCache;
73751 |
73752 |             framesToReadFromCache = (ma_uint32)framesToRead;
73753 |             if (framesToReadFromCache > pNodeGraph->processingCacheFramesRemaining) {
73754 |                 framesToReadFromCache = pNodeGraph->processingCacheFramesRemaining;
73755 |             }
73756 |
73757 |             MA_COPY_MEMORY(pRunningFramesOut, pNodeGraph->pProcessingCache, framesToReadFromCache *
73758 | > channels * sizeof(float));
73759 |             MA_MOVE_MEMORY(pNodeGraph->pProcessingCache, pNodeGraph->pProcessingCache +
73760 | > (framesToReadFromCache * channels), (pNodeGraph->processingCacheFramesRemaining -
73761 | > framesToReadFromCache) * channels * sizeof(float));
73762 |             pNodeGraph->processingCacheFramesRemaining -= framesToReadFromCache;
73763 |
73764 |             totalFramesRead += framesToReadFromCache;
73765 |             continue;

```

```

73762 |         } else {
73763 |             /*
73764 |             If processingSizeInFrames is non-zero, we need to make sure we always read in chunks of
> that size. If the frame count is less than
73765 |             that, we need to read into the cache and then continue on.
73766 |             */
73767 |             float* pReadDst = pRunningFramesOut;
73768 |
73769 |             if (pNodeGraph->processingSizeInFrames > 0) {
73770 |                 if (framesToRead < pNodeGraph->processingSizeInFrames) {
73771 |                     pReadDst = pNodeGraph->pProcessingCache; /* We need to read into the cache
> because otherwise we'll overflow the output buffer. */
73772 |                 }
73773 |
73774 |                 framesToRead = pNodeGraph->processingSizeInFrames;
73775 |             }
73776 |
73777 |             ma_node_graph_set_is_reading(pNodeGraph, MA_TRUE);
73778 |             {
73779 |                 result = ma_node_read_pcm_frames(&pNodeGraph->endpoint, 0, pReadDst,
> (ma_uint32)framesToRead, &framesJustRead, ma_node_get_time(&pNodeGraph->endpoint));
73780 |             }
73781 |             ma_node_graph_set_is_reading(pNodeGraph, MA_FALSE);
73782 |
73783 |             /*
73784 |             Do not increment the total frames read counter if we read into the cache. We use this to
> determine how many frames have
73785 |             been written to the final output buffer.
73786 |             */
73787 |             if (pReadDst == pNodeGraph->pProcessingCache) {
73788 |                 /* We read into the cache. */
73789 |                 pNodeGraph->processingCacheFramesRemaining = framesJustRead;
73790 |             } else {
73791 |                 /* We read straight into the output buffer. */
73792 |                 totalFramesRead += framesJustRead;
73793 |             }
73794 |
73795 |             if (result != MA_SUCCESS) {
73796 |                 break;
73797 |             }
73798 |
73799 |             /* Abort if we weren't able to read any frames or else we risk getting stuck in a loop.
> */
73800 |             if (framesJustRead == 0) {
73801 |                 break;
73802 |             }
73803 |         }
73804 |     }
73805 |
73806 |     /* Let's go ahead and silence any leftover frames just for some added safety to ensure the
> caller doesn't try emitting garbage out of the speakers. */
73807 |     if (totalFramesRead < frameCount) {
73808 |         ma_silence_pcm_frames(ma_offset_pcm_frames_ptr(pFramesOut, totalFramesRead, ma_format_f32,
> channels), (frameCount - totalFramesRead), ma_format_f32, channels);
73809 |     }
73810 |
73811 |     if (pFramesRead != NULL) {
73812 |         *pFramesRead = totalFramesRead;
73813 |     }
73814 |
73815 |     return result;
73816 | }
73817 |
73818 | MA_API ma_uint32 ma_node_graph_get_channels(const ma_node_graph* pNodeGraph)
73819 | {
73820 |     if (pNodeGraph == NULL) {
73821 |         return 0;
73822 |     }
73823 |
73824 |     return ma_node_get_output_channels(&pNodeGraph->endpoint, 0);
73825 | }
73826 |
73827 | MA_API ma_uint64 ma_node_graph_get_time(const ma_node_graph* pNodeGraph)
73828 | {
73829 |     if (pNodeGraph == NULL) {
73830 |         return 0;
73831 |     }
73832 |
73833 |     return ma_node_get_time(&pNodeGraph->endpoint); /* Global time is just the local time of the

```

```

> endpoint. */
73834 | }
73835 |
73836 | MA_API ma_result ma_node_graph_set_time(ma_node_graph* pNodeGraph, ma_uint64 globalTime)
73837 | {
73838 |     if (pNodeGraph == NULL) {
73839 |         return MA_INVALID_ARGS;
73840 |     }
73841 |
73842 |     return ma_node_set_time(&pNodeGraph->endpoint, globalTime); /* Global time is just the local
> time of the endpoint. */
73843 | }
73844 |
73845 | MA_API ma_uint32 ma_node_graph_get_processing_size_in_frames(const ma_node_graph* pNodeGraph)
73846 | {
73847 |     if (pNodeGraph == NULL) {
73848 |         return 0;
73849 |     }
73850 |
73851 |     return pNodeGraph->processingSizeInFrames;
73852 | }
73853 |
73854 |
73855 | #define MA_NODE_OUTPUT_BUS_FLAG_HAS_READ    0x01    /* Whether or not this bus ready to read more
> data. Only used on nodes with multiple output buses. */
73856 |
73857 | static ma_result ma_node_output_bus_init(ma_node* pNode, ma_uint32 outputBusIndex, ma_uint32
> channels, ma_node_output_bus* pOutputBus)
73858 | {
73859 |     MA_ASSERT(pOutputBus != NULL);
73860 |     MA_ASSERT(outputBusIndex < MA_MAX_NODE_BUS_COUNT);
73861 |     MA_ASSERT(outputBusIndex < ma_node_get_output_bus_count(pNode));
73862 |     MA_ASSERT(channels < 256);
73863 |
73864 |     MA_ZERO_OBJECT(pOutputBus);
73865 |
73866 |     if (channels == 0) {
73867 |         return MA_INVALID_ARGS;
73868 |     }
73869 |
73870 |     pOutputBus->pNode          = pNode;
73871 |     pOutputBus->outputBusIndex = (ma_uint8)outputBusIndex;
73872 |     pOutputBus->channels       = (ma_uint8)channels;
73873 |     pOutputBus->flags          = MA_NODE_OUTPUT_BUS_FLAG_HAS_READ; /* <-- Important that this flag
> is set by default. */
73874 |     pOutputBus->volume         = 1;
73875 |
73876 |     return MA_SUCCESS;
73877 | }
73878 |
73879 | static void ma_node_output_bus_lock(ma_node_output_bus* pOutputBus)
73880 | {
73881 |     ma_spinlock_lock(&pOutputBus->lock);
73882 | }
73883 |
73884 | static void ma_node_output_bus_unlock(ma_node_output_bus* pOutputBus)
73885 | {
73886 |     ma_spinlock_unlock(&pOutputBus->lock);
73887 | }
73888 |
73889 |
73890 | static ma_uint32 ma_node_output_bus_get_channels(const ma_node_output_bus* pOutputBus)
73891 | {
73892 |     return pOutputBus->channels;
73893 | }
73894 |
73895 |
73896 | static void ma_node_output_bus_set_has_read(ma_node_output_bus* pOutputBus, ma_bool32 hasRead)
73897 | {
73898 |     if (hasRead) {
73899 |         ma_atomic_fetch_or_32(&pOutputBus->flags, MA_NODE_OUTPUT_BUS_FLAG_HAS_READ);
73900 |     } else {
73901 |         ma_atomic_fetch_and_32(&pOutputBus->flags, (ma_uint32)~MA_NODE_OUTPUT_BUS_FLAG_HAS_READ);
73902 |     }
73903 | }
73904 |
73905 | static ma_bool32 ma_node_output_bus_has_read(ma_node_output_bus* pOutputBus)
73906 | {
73907 |     return (ma_atomic_load_32(&pOutputBus->flags) & MA_NODE_OUTPUT_BUS_FLAG_HAS_READ) != 0;

```



```
73908 | }
73909 |
73910 |
73911 | static void ma_node_output_bus_set_is_attached(ma_node_output_bus* pOutputBus, ma_bool32 isAttached)
73912 | {
73913 |     ma_atomic_exchange_32(&pOutputBus->isAttached, isAttached);
73914 | }
73915 |
73916 | static ma_bool32 ma_node_output_bus_is_attached(ma_node_output_bus* pOutputBus)
73917 | {
73918 |     return ma_atomic_load_32(&pOutputBus->isAttached);
73919 | }
73920 |
73921 |
73922 | static ma_result ma_node_output_bus_set_volume(ma_node_output_bus* pOutputBus, float volume)
73923 | {
73924 |     MA_ASSERT(pOutputBus != NULL);
73925 |
73926 |     if (volume < 0.0f) {
73927 |         volume = 0.0f;
73928 |     }
73929 |
73930 |     ma_atomic_exchange_f32(&pOutputBus->volume, volume);
73931 |
73932 |     return MA_SUCCESS;
73933 | }
73934 |
73935 | static float ma_node_output_bus_get_volume(const ma_node_output_bus* pOutputBus)
73936 | {
73937 |     return ma_atomic_load_f32((float*)&pOutputBus->volume);
73938 | }
73939 |
73940 |
73941 | static ma_result ma_node_input_bus_init(ma_uint32 channels, ma_node_input_bus* pInputBus)
73942 | {
73943 |     MA_ASSERT(pInputBus != NULL);
73944 |     MA_ASSERT(channels < 256);
73945 |
73946 |     MA_ZERO_OBJECT(pInputBus);
73947 |
73948 |     if (channels == 0) {
73949 |         return MA_INVALID_ARGS;
73950 |     }
73951 |
73952 |     pInputBus->channels = (ma_uint8)channels;
73953 |
73954 |     return MA_SUCCESS;
73955 | }
73956 |
73957 | static void ma_node_input_bus_lock(ma_node_input_bus* pInputBus)
73958 | {
73959 |     MA_ASSERT(pInputBus != NULL);
73960 |
73961 |     ma_spinlock_lock(&pInputBus->lock);
73962 | }
73963 |
73964 | static void ma_node_input_bus_unlock(ma_node_input_bus* pInputBus)
73965 | {
73966 |     MA_ASSERT(pInputBus != NULL);
73967 |
73968 |     ma_spinlock_unlock(&pInputBus->lock);
73969 | }
73970 |
73971 |
73972 | static void ma_node_input_bus_next_begin(ma_node_input_bus* pInputBus)
73973 | {
73974 |     ma_atomic_fetch_add_32(&pInputBus->nextCounter, 1);
73975 | }
73976 |
73977 | static void ma_node_input_bus_next_end(ma_node_input_bus* pInputBus)
73978 | {
73979 |     ma_atomic_fetch_sub_32(&pInputBus->nextCounter, 1);
73980 | }
73981 |
73982 | static ma_uint32 ma_node_input_bus_get_next_counter(ma_node_input_bus* pInputBus)
73983 | {
73984 |     return ma_atomic_load_32(&pInputBus->nextCounter);
73985 | }
73986 |
```

```

73987 |
73988 | static ma_uint32 ma_node_input_bus_get_channels(const ma_node_input_bus* pInputBus)
73989 | {
73990 |     return pInputBus->channels;
73991 | }
73992 |
73993 |
73994 | static void ma_node_input_bus_detach__no_output_bus_lock(ma_node_input_bus* pInputBus,
> ma_node_output_bus* pOutputBus)
73995 | {
73996 |     MA_ASSERT(pInputBus != NULL);
73997 |     MA_ASSERT(pOutputBus != NULL);
73998 |
73999 |     /*
74000 |     Mark the output bus as detached first. This will prevent future iterations on the audio thread
74001 |     from iterating this output bus.
74002 |     */
74003 |     ma_node_output_bus_set_is_attached(pOutputBus, MA_FALSE);
74004 |
74005 |     /*
74006 |     We cannot use the output bus lock here since it'll be getting used at a higher level, but we do
74007 |     still need to use the input bus lock since we'll be updating pointers on two different output
74008 |     buses. The same rules apply here as the attaching case. Although we're using a lock here, we're
74009 |     *not* using a lock when iterating over the list in the audio thread. We therefore need to craft
74010 |     this in a way such that the iteration on the audio thread doesn't break.
74011 |
74012 |     The first thing to do is swap out the "next" pointer of the previous output bus with the
74013 |     new "next" output bus. This is the operation that matters for iteration on the audio thread.
74014 |     After that, the previous pointer on the new "next" pointer needs to be updated, after which
74015 |     point the linked list will be in a good state.
74016 |     */
74017 |     ma_node_input_bus_lock(pInputBus);
74018 |     {
74019 |         ma_node_output_bus* pOldPrev = (ma_node_output_bus*)ma_atomic_load_ptr(&pOutputBus->pPrev);
74020 |         ma_node_output_bus* pOldNext = (ma_node_output_bus*)ma_atomic_load_ptr(&pOutputBus->pNext);
74021 |
74022 |         if (pOldPrev != NULL) {
74023 |             ma_atomic_exchange_ptr(&pOldPrev->pNext, pOldNext); /* <-- This is where the output bus
> is detached from the list. */
74024 |         }
74025 |         if (pOldNext != NULL) {
74026 |             ma_atomic_exchange_ptr(&pOldNext->pPrev, pOldPrev); /* <-- This is required for
> detachment. */
74027 |         }
74028 |     }
74029 |     ma_node_input_bus_unlock(pInputBus);
74030 |
74031 |     /* At this point the output bus is detached and the linked list is completely unaware of it.
> Reset some data for safety. */
74032 |     ma_atomic_exchange_ptr(&pOutputBus->pNext, NULL); /* Using atomic exchanges here, mainly for
> the benefit of analysis tools which don't always recognize spinlocks. */
74033 |     ma_atomic_exchange_ptr(&pOutputBus->pPrev, NULL); /* As above. */
74034 |     pOutputBus->pInputNode = NULL;
74035 |     pOutputBus->inputNodeInputBusIndex = 0;
74036 |
74037 |
74038 |     /*
74039 |     For thread-safety reasons, we don't want to be returning from this straight away. We need to
74040 |     wait for the audio thread to finish with the output bus. There's two things we need to wait
74041 |     for. The first is the part that selects the next output bus in the list, and the other is the
74042 |     part that reads from the output bus. Basically all we're doing is waiting for the input bus
74043 |     to stop referencing the output bus.
74044 |
74045 |     We're doing this part last because we want the section above to run while the audio thread
74046 |     is finishing up with the output bus, just for efficiency reasons. We marked the output bus as
74047 |     detached right at the top of this function which is going to prevent the audio thread from
74048 |     iterating the output bus again.
74049 |     */
74050 |
74051 |     /* Part 1: Wait for the current iteration to complete. */
74052 |     while (ma_node_input_bus_get_next_counter(pInputBus) > 0) {
74053 |         ma_yield();
74054 |     }
74055 |
74056 |     /* Part 2: Wait for any reads to complete. */
74057 |     while (ma_atomic_load_32(&pOutputBus->refCount) > 0) {
74058 |         ma_yield();
74059 |     }
74060 |

```

```

74061 | /*
74062 | At this point we're done detaching and we can be guaranteed that the audio thread is not going
74063 | to attempt to reference this output bus again (until attached again).
74064 | */
74065 | }
74066 |
74067 | #if 0 /* Not used at the moment, but leaving here in case I need it later. */
74068 | static void ma_node_input_bus_detach(ma_node_input_bus* pInputBus, ma_node_output_bus* pOutputBus)
74069 | {
74070 |     MA_ASSERT(pInputBus != NULL);
74071 |     MA_ASSERT(pOutputBus != NULL);
74072 |
74073 |     ma_node_output_bus_lock(pOutputBus);
74074 |     {
74075 |         ma_node_input_bus_detach__no_output_bus_lock(pInputBus, pOutputBus);
74076 |     }
74077 |     ma_node_output_bus_unlock(pOutputBus);
74078 | }
74079 | #endif
74080 |
74081 | static void ma_node_input_bus_attach(ma_node_input_bus* pInputBus, ma_node_output_bus* pOutputBus,
> ma_node* pNewInputNode, ma_uint32 inputNodeInputBusIndex)
74082 | {
74083 |     MA_ASSERT(pInputBus != NULL);
74084 |     MA_ASSERT(pOutputBus != NULL);
74085 |
74086 |     ma_node_output_bus_lock(pOutputBus);
74087 |     {
74088 |         ma_node_output_bus* pOldInputNode =
> (ma_node_output_bus*)ma_atomic_load_ptr(&pOutputBus->pInputNode);
74089 |
74090 |         /* Detach from any existing attachment first if necessary. */
74091 |         if (pOldInputNode != NULL) {
74092 |             ma_node_input_bus_detach__no_output_bus_lock(pInputBus, pOutputBus);
74093 |         }
74094 |
74095 |         /*
74096 |         At this point we can be sure the output bus is not attached to anything. The linked list in
> the
74097 |         old input bus has been updated so that pOutputBus will not get iterated again.
74098 |         */
74099 |         pOutputBus->pInputNode = pNewInputNode; /* No need for an
> atomic assignment here because modification of this variable always happens within a lock. */
74100 |         pOutputBus->inputNodeInputBusIndex = (ma_uint8)inputNodeInputBusIndex;
74101 |
74102 |         /*
74103 |         Now we need to attach the output bus to the linked list. This involves updating two pointers
> on
74104 |         two different output buses so I'm going to go ahead and keep this simple and just use a
> lock.
74105 |         There are ways to do this without a lock, but it's just too hard to maintain for its value.
74106 |
74107 |         Although we're locking here, it's important to remember that we're *not* locking when
> iterating
74108 |         and reading audio data since that'll be running on the audio thread. As a result we need to
> be
74109 |         careful how we craft this so that we don't break iteration. What we're going to do is always
74110 |         attach the new item so that it becomes the first item in the list. That way, as we're
> iterating
74111 |         we won't break any links in the list and iteration will continue safely. The detaching case
> will
74112 |         also be crafted in a way as to not break list iteration. It's important to remember to use
74113 |         atomic exchanges here since no locking is happening on the audio thread during iteration.
74114 |         */
74115 |         ma_node_input_bus_lock(pInputBus);
74116 |         {
74117 |             ma_node_output_bus* pNewPrev = &pInputBus->head;
74118 |             ma_node_output_bus* pNewNext =
> (ma_node_output_bus*)ma_atomic_load_ptr(&pInputBus->head.pNext);
74119 |
74120 |             /* Update the local output bus. */
74121 |             ma_atomic_exchange_ptr(&pOutputBus->pPrev, pNewPrev);
74122 |             ma_atomic_exchange_ptr(&pOutputBus->pNext, pNewNext);
74123 |
74124 |             /* Update the other output buses to point back to the local output bus. */
74125 |             ma_atomic_exchange_ptr(&pInputBus->head.pNext, pOutputBus); /* <-- This is where the
> output bus is actually attached to the input bus. */
74126 |
74127 |             /* Do the previous pointer last. This is only used for detachment. */

```

```

74128 |         if (pNewNext != NULL) {
74129 |             ma_atomic_exchange_ptr(&pNewNext->pPrev, pOutputBus);
74130 |         }
74131 |     }
74132 |     ma_node_input_bus_unlock(pInputBus);
74133 |
74134 |     /*
74135 |     Mark the node as attached last. This is used to controlling whether or the output bus will
> be
74136 |     iterated on the audio thread. Mainly required for detachment purposes.
74137 |     */
74138 |     ma_node_output_bus_set_is_attached(pOutputBus, MA_TRUE);
74139 | }
74140 | ma_node_output_bus_unlock(pOutputBus);
74141 | }
74142 |
74143 | static ma_node_output_bus* ma_node_input_bus_next(ma_node_input_bus* pInputBus, ma_node_output_bus*
> pOutputBus)
74144 | {
74145 |     ma_node_output_bus* pNext;
74146 |
74147 |     MA_ASSERT(pInputBus != NULL);
74148 |
74149 |     if (pOutputBus == NULL) {
74150 |         return NULL;
74151 |     }
74152 |
74153 |     ma_node_input_bus_next_begin(pInputBus);
74154 |     {
74155 |         pNext = pOutputBus;
74156 |         for (;;) {
74157 |             pNext = (ma_node_output_bus*)ma_atomic_load_ptr(&pNext->pNext);
74158 |             if (pNext == NULL) {
74159 |                 break; /* Reached the end. */
74160 |             }
74161 |
74162 |             if (ma_node_output_bus_is_attached(pNext) == MA_FALSE) {
74163 |                 continue; /* The node is not attached. Keep checking. */
74164 |             }
74165 |
74166 |             /* The next node has been selected. */
74167 |             break;
74168 |         }
74169 |
74170 |         /* We need to increment the reference count of the selected node. */
74171 |         if (pNext != NULL) {
74172 |             ma_atomic_fetch_add_32(&pNext->refCount, 1);
74173 |         }
74174 |
74175 |         /* The previous node is no longer being referenced. */
74176 |         ma_atomic_fetch_sub_32(&pOutputBus->refCount, 1);
74177 |     }
74178 |     ma_node_input_bus_next_end(pInputBus);
74179 |
74180 |     return pNext;
74181 | }
74182 |
74183 | static ma_node_output_bus* ma_node_input_bus_first(ma_node_input_bus* pInputBus)
74184 | {
74185 |     return ma_node_input_bus_next(pInputBus, &pInputBus->head);
74186 | }
74187 |
74188 |
74189 |
74190 | static ma_result ma_node_input_bus_read_pcm_frames(ma_node* pInputNode, ma_node_input_bus*
> pInputBus, float* pFramesOut, ma_uint32 frameCount, ma_uint32* pFramesRead, ma_uint64 globalTime)
74191 | {
74192 |     ma_result result = MA_SUCCESS;
74193 |     ma_node_output_bus* pOutputBus;
74194 |     ma_node_output_bus* pFirst;
74195 |     ma_uint32 inputChannels;
74196 |     ma_bool32 doesOutputBufferHaveContent = MA_FALSE;
74197 |
74198 |     /*
74199 |     This will be called from the audio thread which means we can't be doing any locking. Basically,
74200 |     this function will not perform any locking, whereas attaching and detaching will, but crafted in
74201 |     such a way that we don't need to perform any locking here. The important thing to remember is
74202 |     to always iterate in a forward direction.
74203 |

```

```

74204 | In order to process any data we need to first read from all input buses. That's where this
74205 | function comes in. This iterates over each of the attachments and accumulates/mixes them. We
74206 | also convert the channels to the nodes output channel count before mixing. We want to do this
74207 | channel conversion so that the caller of this function can invoke the processing callback
74208 | without having to do it themselves.
74209 |
74210 | When we iterate over each of the attachments on the input bus, we need to read as much data as
74211 | we can from each of them so that we don't end up with holes between each of the attachments. To
74212 | do this, we need to read from each attachment in a loop and read as many frames as we can, up
74213 | to `frameCount`.
74214 | */
74215 | MA_ASSERT(pInputNode != NULL);
74216 | MA_ASSERT(pFramesRead != NULL); /* pFramesRead is critical and must always be specified. On
> input it's undefined and on output it'll be set to the number of frames actually read. */
74217 |
74218 | *pFramesRead = 0; /* Safety. */
74219 |
74220 | inputChannels = ma_node_input_bus_get_channels(pInputBus);
74221 |
74222 | /*
74223 | We need to be careful with how we call ma_node_input_bus_first() and ma_node_input_bus_next().
> They
74224 | are both critical to our lock-free thread-safety system. We can only call
> ma_node_input_bus_first()
74225 | once per iteration, however we have an optimization to checks whether or not it's the first item
> in
74226 | the list. We therefore need to store a pointer to the first item rather than repeatedly calling
74227 | ma_node_input_bus_first(). It's safe to keep hold of this pointer, so long as we don't
> dereference it
74228 | after calling ma_node_input_bus_next(), which we won't be.
74229 | */
74230 | pFirst = ma_node_input_bus_first(pInputBus);
74231 | if (pFirst == NULL) {
74232 |     return MA_SUCCESS; /* No attachments. Read nothing. */
74233 | }
74234 |
74235 | for (pOutputBus = pFirst; pOutputBus != NULL; pOutputBus = ma_node_input_bus_next(pInputBus,
> pOutputBus)) {
74236 |     ma_uint32 framesProcessed = 0;
74237 |     ma_bool32 isSilentOutput = MA_FALSE;
74238 |
74239 |     MA_ASSERT(pOutputBus->pNode != NULL);
74240 |     MA_ASSERT(((ma_node_base*)pOutputBus->pNode)->vtable != NULL);
74241 |
74242 |     isSilentOutput = (((ma_node_base*)pOutputBus->pNode)->vtable->flags &
> MA_NODE_FLAG_SILENT_OUTPUT) != 0;
74243 |
74244 |     if (pFramesOut != NULL) {
74245 |         /* Read. */
74246 |         while (framesProcessed < frameCount) {
74247 |             float* pRunningFramesOut;
74248 |             ma_uint32 framesToRead;
74249 |             ma_uint32 framesJustRead = 0;
74250 |
74251 |             framesToRead = frameCount - framesProcessed;
74252 |             pRunningFramesOut = ma_offset_pcm_frames_ptr_f32(pFramesOut, framesProcessed,
> inputChannels);
74253 |
74254 |             if (doesOutputBufferHaveContent == MA_FALSE) {
74255 |                 /* Fast path. First attachment. We just read straight into the output buffer (no
> mixing required). */
74256 |                 result = ma_node_read_pcm_frames(pOutputBus->pNode, pOutputBus->outputBusIndex,
> pRunningFramesOut, framesToRead, &framesJustRead, globalTime + framesProcessed);
74257 |             } else {
74258 |                 /* Slow path. Not the first attachment. Mixing required. */
74259 |                 ma_uint32 preMixBufferCapInFrames =
> ((ma_node_base*)pInputNode)->cachedDataCapInFramesPerBus;
74260 |                 float* pPreMixBuffer =
> (float*)ma_stack_alloc(((ma_node_base*)pInputNode)->pNodeGraph->pPreMixStack,
> preMixBufferCapInFrames * inputChannels * sizeof(float));
74261 |
74262 |                 if (pPreMixBuffer == NULL) {
74263 |                     /*
74264 |                     If you're hitting this assert it means you've got an unusually deep chain of
> nodes, you've got an excessively large processing
74265 |                     size, or you have a combination of both, and as a result have run out of
> stack space. You can increase this using the
74266 |                     preMixStackSizeInBytes variable in ma_node_graph_config. If you're using
> ma_engine, you can do it via the preMixStackSizeInBytes

```

```

74267 |         variable in ma_engine_config. It defaults to 512KB per output channel.
74268 |     */
74269 |     MA_ASSERT(MA_FALSE);
74270 | } else {
74271 |     if (framesToRead > preMixBufferCapInFrames) {
74272 |         framesToRead = preMixBufferCapInFrames;
74273 |     }
74274 |
74275 |     result = ma_node_read_pcm_frames(pOutputBus->pNode,
> pOutputBus->outputBusIndex, pPreMixBuffer, framesToRead, &framesJustRead, globalTime +
> framesProcessed);
74276 |     if (result == MA_SUCCESS || result == MA_AT_END) {
74277 |         if (isSilentOutput == MA_FALSE) { /* Don't mix if the node outputs
> silence. */
74278 |             ma_mix_pcm_frames_f32(pRunningFramesOut, pPreMixBuffer,
> framesJustRead, inputChannels, /*volume*/1);
74279 |         }
74280 |     }
74281 |
74282 |     /* The pre-mix buffer is no longer required. */
74283 |     ma_stack_free(((ma_node_base*)pInputNode)->pNodeGraph->pPreMixStack,
> pPreMixBuffer);
74284 |     pPreMixBuffer = NULL;
74285 | }
74286 | }
74287 |
74288 |     framesProcessed += framesJustRead;
74289 |
74290 |     /* If we reached the end or otherwise failed to read any data we need to finish up
> with this output node. */
74291 |     if (result != MA_SUCCESS) {
74292 |         break;
74293 |     }
74294 |
74295 |     /* If we didn't read anything, abort so we don't get stuck in a loop. */
74296 |     if (framesJustRead == 0) {
74297 |         break;
74298 |     }
74299 | }
74300 |
74301 |     /* If it's the first attachment we didn't do any mixing. Any leftover samples need to be
> silenced. */
74302 |     if (pOutputBus == pFirst && framesProcessed < frameCount) {
74303 |         ma_silence_pcm_frames(ma_offset_pcm_frames_ptr(pFramesOut, framesProcessed,
> ma_format_f32, inputChannels), (frameCount - framesProcessed), ma_format_f32, inputChannels);
74304 |     }
74305 |
74306 |     if (isSilentOutput == MA_FALSE) {
74307 |         doesOutputBufferHaveContent = MA_TRUE;
74308 |     }
74309 | } else {
74310 |     /* Seek. */
74311 |     ma_node_read_pcm_frames(pOutputBus->pNode, pOutputBus->outputBusIndex, NULL, frameCount,
> &framesProcessed, globalTime);
74312 | }
74313 | }
74314 |
74315 |     /* If we didn't output anything, output silence. */
74316 |     if (doesOutputBufferHaveContent == MA_FALSE && pFramesOut != NULL) {
74317 |         ma_silence_pcm_frames(pFramesOut, frameCount, ma_format_f32, inputChannels);
74318 |     }
74319 |
74320 |     /* In this path we always "process" the entire amount. */
74321 |     *pFramesRead = frameCount;
74322 |
74323 |     return result;
74324 | }
74325 |
74326 |
74327 | MA_API ma_node_config ma_node_config_init(void)
74328 | {
74329 |     ma_node_config config;
74330 |
74331 |     MA_ZERO_OBJECT(&config);
74332 |     config.initialState = ma_node_state_started; /* Nodes are started by default. */
74333 |     config.inputBusCount = MA_NODE_BUS_COUNT_UNKNOWN;
74334 |     config.outputBusCount = MA_NODE_BUS_COUNT_UNKNOWN;
74335 |
74336 |     return config;

```

```

74337 | }
74338 |
74339 | static ma_uint16 ma_node_config_get_cache_size_in_frames(const ma_node_config* pConfig, const
> ma_node_graph* pNodeGraph)
74340 | {
74341 |     ma_uint32 cacheSizeInFrames;
74342 |
74343 |     (void)pConfig;
74344 |
74345 |     if (pNodeGraph->processingSizeInFrames > 0) {
74346 |         cacheSizeInFrames = pNodeGraph->processingSizeInFrames;
74347 |     } else {
74348 |         cacheSizeInFrames = MA_DEFAULT_NODE_CACHE_CAP_IN_FRAMES_PER_BUS;
74349 |     }
74350 |
74351 |     if (cacheSizeInFrames > 0xFFFF) {
74352 |         cacheSizeInFrames = 0xFFFF;
74353 |     }
74354 |
74355 |     return (ma_uint16)cacheSizeInFrames;
74356 | }
74357 |
74358 |
74359 |
74360 | static ma_result ma_node_detach_full(ma_node* pNode);
74361 |
74362 | static float* ma_node_get_cached_input_ptr(ma_node* pNode, ma_uint32 inputBusIndex)
74363 | {
74364 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74365 |     ma_uint32 iInputBus;
74366 |     float* pBasePtr;
74367 |
74368 |     MA_ASSERT(pNodeBase != NULL);
74369 |
74370 |     /* Input data is stored at the front of the buffer. */
74371 |     pBasePtr = pNodeBase->pCachedData;
74372 |     for (iInputBus = 0; iInputBus < inputBusIndex; iInputBus += 1) {
74373 |         pBasePtr += pNodeBase->cachedDataCapInFramesPerBus *
> ma_node_input_bus_get_channels(&pNodeBase->pInputBuses[iInputBus]);
74374 |     }
74375 |
74376 |     return pBasePtr;
74377 | }
74378 |
74379 | static float* ma_node_get_cached_output_ptr(ma_node* pNode, ma_uint32 outputBusIndex)
74380 | {
74381 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74382 |     ma_uint32 iInputBus;
74383 |     ma_uint32 iOutputBus;
74384 |     float* pBasePtr;
74385 |
74386 |     MA_ASSERT(pNodeBase != NULL);
74387 |
74388 |     /* Cached output data starts after the input data. */
74389 |     pBasePtr = pNodeBase->pCachedData;
74390 |     for (iInputBus = 0; iInputBus < ma_node_get_input_bus_count(pNodeBase); iInputBus += 1) {
74391 |         pBasePtr += pNodeBase->cachedDataCapInFramesPerBus *
> ma_node_input_bus_get_channels(&pNodeBase->pInputBuses[iInputBus]);
74392 |     }
74393 |
74394 |     for (iOutputBus = 0; iOutputBus < outputBusIndex; iOutputBus += 1) {
74395 |         pBasePtr += pNodeBase->cachedDataCapInFramesPerBus *
> ma_node_output_bus_get_channels(&pNodeBase->pOutputBuses[iOutputBus]);
74396 |     }
74397 |
74398 |     return pBasePtr;
74399 | }
74400 |
74401 |
74402 | typedef struct
74403 | {
74404 |     size_t sizeInBytes;
74405 |     size_t inputBusOffset;
74406 |     size_t outputBusOffset;
74407 |     size_t cachedDataOffset;
74408 |     ma_uint32 inputBusCount; /* So it doesn't have to be calculated twice. */
74409 |     ma_uint32 outputBusCount; /* So it doesn't have to be calculated twice. */
74410 | } ma_node_heap_layout;
74411 |

```



```

74412 | static ma_result ma_node_translate_bus_counts(const ma_node_config* pConfig, ma_uint32*
> pInputBusCount, ma_uint32* pOutputBusCount)
74413 | {
74414 |     ma_uint32 inputBusCount;
74415 |     ma_uint32 outputBusCount;
74416 |
74417 |     MA_ASSERT(pConfig != NULL);
74418 |     MA_ASSERT(pInputBusCount != NULL);
74419 |     MA_ASSERT(pOutputBusCount != NULL);
74420 |
74421 |     /* Bus counts are determined by the vtable, unless they're set to `MA_NODE_BUS_COUNT_UNKNWON`,
> in which case they're taken from the config. */
74422 |     if (pConfig->vtable->inputBusCount == MA_NODE_BUS_COUNT_UNKNOWN) {
74423 |         inputBusCount = pConfig->inputBusCount;
74424 |     } else {
74425 |         inputBusCount = pConfig->vtable->inputBusCount;
74426 |
74427 |         if (pConfig->inputBusCount != MA_NODE_BUS_COUNT_UNKNOWN && pConfig->inputBusCount !=
> pConfig->vtable->inputBusCount) {
74428 |             return MA_INVALID_ARGS; /* Invalid configuration. You must not specify a conflicting bus
> count between the node's config and the vtable. */
74429 |         }
74430 |     }
74431 |
74432 |     if (pConfig->vtable->outputBusCount == MA_NODE_BUS_COUNT_UNKNOWN) {
74433 |         outputBusCount = pConfig->outputBusCount;
74434 |     } else {
74435 |         outputBusCount = pConfig->vtable->outputBusCount;
74436 |
74437 |         if (pConfig->outputBusCount != MA_NODE_BUS_COUNT_UNKNOWN && pConfig->outputBusCount !=
> pConfig->vtable->outputBusCount) {
74438 |             return MA_INVALID_ARGS; /* Invalid configuration. You must not specify a conflicting bus
> count between the node's config and the vtable. */
74439 |         }
74440 |     }
74441 |
74442 |     /* Bus counts must be within limits. */
74443 |     if (inputBusCount > MA_MAX_NODE_BUS_COUNT || outputBusCount > MA_MAX_NODE_BUS_COUNT) {
74444 |         return MA_INVALID_ARGS;
74445 |     }
74446 |
74447 |
74448 |     /* We must have channel counts for each bus. */
74449 |     if ((inputBusCount > 0 && pConfig->pInputChannels == NULL) || (outputBusCount > 0 &&
> pConfig->pOutputChannels == NULL)) {
74450 |         return MA_INVALID_ARGS; /* You must specify channel counts for each input and output bus. */
74451 |     }
74452 |
74453 |
74454 |     /* Some special rules for passthrough nodes. */
74455 |     if ((pConfig->vtable->flags & MA_NODE_FLAG_PASSTHROUGH) != 0) {
74456 |         if ((pConfig->vtable->inputBusCount != 0 && pConfig->vtable->inputBusCount != 1) ||
> pConfig->vtable->outputBusCount != 1) {
74457 |             return MA_INVALID_ARGS; /* Passthrough nodes must have exactly 1 output bus and either 0
> or 1 input bus. */
74458 |         }
74459 |
74460 |         if (pConfig->pInputChannels[0] != pConfig->pOutputChannels[0]) {
74461 |             return MA_INVALID_ARGS; /* Passthrough nodes must have the same number of channels
> between input and output nodes. */
74462 |         }
74463 |     }
74464 |
74465 |
74466 |     *pInputBusCount = inputBusCount;
74467 |     *pOutputBusCount = outputBusCount;
74468 |
74469 |     return MA_SUCCESS;
74470 | }
74471 |
74472 | static ma_result ma_node_get_heap_layout(ma_node_graph* pNodeGraph, const ma_node_config* pConfig,
> ma_node_heap_layout* pHeapLayout)
74473 | {
74474 |     ma_result result;
74475 |     ma_uint32 inputBusCount;
74476 |     ma_uint32 outputBusCount;
74477 |
74478 |     MA_ASSERT(pHeapLayout != NULL);
74479 |

```

```

74480 |     MA_ZERO_OBJECT(pHeapLayout);
74481 |
74482 |     if (pConfig == NULL || pConfig->vtable == NULL || pConfig->vtable->onProcess == NULL) {
74483 |         return MA_INVALID_ARGS;
74484 |     }
74485 |
74486 |     result = ma_node_translate_bus_counts(pConfig, &inputBusCount, &outputBusCount);
74487 |     if (result != MA_SUCCESS) {
74488 |         return result;
74489 |     }
74490 |
74491 |     pHeapLayout->sizeInBytes = 0;
74492 |
74493 |     /* Input buses. */
74494 |     if (inputBusCount > MA_MAX_NODE_LOCAL_BUS_COUNT) {
74495 |         pHeapLayout->inputBusOffset = pHeapLayout->sizeInBytes;
74496 |         pHeapLayout->sizeInBytes += ma_align_64(sizeof(ma_node_input_bus) * inputBusCount);
74497 |     } else {
74498 |         pHeapLayout->inputBusOffset = MA_SIZE_MAX; /* MA_SIZE_MAX indicates that no heap allocation
> is required for the input bus. */
74499 |     }
74500 |
74501 |     /* Output buses. */
74502 |     if (outputBusCount > MA_MAX_NODE_LOCAL_BUS_COUNT) {
74503 |         pHeapLayout->outputBusOffset = pHeapLayout->sizeInBytes;
74504 |         pHeapLayout->sizeInBytes += ma_align_64(sizeof(ma_node_output_bus) * outputBusCount);
74505 |     } else {
74506 |         pHeapLayout->outputBusOffset = MA_SIZE_MAX;
74507 |     }
74508 |
74509 |     /*
74510 |     Cached audio data.
74511 |
74512 |     We need to allocate memory for caching both input and output data. We have an optimization
74513 |     where no caching is necessary for specific conditions:
74514 |
74515 |         - The node has 0 inputs and 1 output.
74516 |
74517 |     When a node meets the above conditions, no cache is allocated.
74518 |
74519 |     The size choice for this buffer is a little bit finicky. We don't want to be too wasteful by
74520 |     allocating too much, but at the same time we want it be large enough so that enough frames can
74521 |     be processed for each call to ma_node_read_pcm_frames() so that it keeps things efficient. For
74522 |     now I'm going with 10ms @ 48K which is 480 frames per bus. This is configurable at compile
74523 |     time. It might also be worth investigating whether or not this can be configured at run time.
74524 |     */
74525 |     if (inputBusCount == 0 && outputBusCount == 1) {
74526 |         /* Fast path. No cache needed. */
74527 |         pHeapLayout->cachedDataOffset = MA_SIZE_MAX;
74528 |     } else {
74529 |         /* Slow path. Cache needed. */
74530 |         size_t cachedDataSizeInBytes = 0;
74531 |         ma_uint32 cacheCapInFrames;
74532 |         ma_uint32 iBus;
74533 |
74534 |         /* The capacity of the cache is based on our callback processing size. */
74535 |         cacheCapInFrames = ma_node_config_get_cache_size_in_frames(pConfig, pNodeGraph);
74536 |
74537 |         for (iBus = 0; iBus < inputBusCount; iBus += 1) {
74538 |             cachedDataSizeInBytes += cacheCapInFrames * ma_get_bytes_per_frame(ma_format_f32,
> pConfig->pInputChannels[iBus]);
74539 |         }
74540 |
74541 |         for (iBus = 0; iBus < outputBusCount; iBus += 1) {
74542 |             cachedDataSizeInBytes += cacheCapInFrames * ma_get_bytes_per_frame(ma_format_f32,
> pConfig->pOutputChannels[iBus]);
74543 |         }
74544 |
74545 |         pHeapLayout->cachedDataOffset = pHeapLayout->sizeInBytes;
74546 |         pHeapLayout->sizeInBytes += ma_align_64(cachedDataSizeInBytes);
74547 |     }
74548 |
74549 |     /*
74550 |     Not technically part of the heap, but we can output the input and output bus counts so we can
74551 |     avoid a redundant call to ma_node_translate_bus_counts().
74552 |     */
74553 |     pHeapLayout->inputBusCount = inputBusCount;
74554 |     pHeapLayout->outputBusCount = outputBusCount;

```

```

74556 |
74557 |     /* Make sure allocation size is aligned. */
74558 |     pHeapLayout->sizeInBytes = ma_align_64(pHeapLayout->sizeInBytes);
74559 |
74560 |     return MA_SUCCESS;
74561 | }
74562 |
74563 | MA_API ma_result ma_node_get_heap_size(ma_node_graph* pNodeGraph, const ma_node_config* pConfig,
> size_t* pHeapSizeInBytes)
74564 | {
74565 |     ma_result result;
74566 |     ma_node_heap_layout heapLayout;
74567 |
74568 |     if (pHeapSizeInBytes == NULL) {
74569 |         return MA_INVALID_ARGS;
74570 |     }
74571 |
74572 |     *pHeapSizeInBytes = 0;
74573 |
74574 |     result = ma_node_get_heap_layout(pNodeGraph, pConfig, &heapLayout);
74575 |     if (result != MA_SUCCESS) {
74576 |         return result;
74577 |     }
74578 |
74579 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
74580 |
74581 |     return MA_SUCCESS;
74582 | }
74583 |
74584 | MA_API ma_result ma_node_init_preallocated(ma_node_graph* pNodeGraph, const ma_node_config* pConfig,
> void* pHeap, ma_node* pNode)
74585 | {
74586 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74587 |     ma_result result;
74588 |     ma_node_heap_layout heapLayout;
74589 |     ma_uint32 iInputBus;
74590 |     ma_uint32 iOutputBus;
74591 |
74592 |     if (pNodeBase == NULL) {
74593 |         return MA_INVALID_ARGS;
74594 |     }
74595 |
74596 |     MA_ZERO_OBJECT(pNodeBase);
74597 |
74598 |     result = ma_node_get_heap_layout(pNodeGraph, pConfig, &heapLayout);
74599 |     if (result != MA_SUCCESS) {
74600 |         return result;
74601 |     }
74602 |
74603 |     pNodeBase->pHeap = pHeap;
74604 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
74605 |
74606 |     pNodeBase->pNodeGraph = pNodeGraph;
74607 |     pNodeBase->vtable = pConfig->vtable;
74608 |     pNodeBase->state = pConfig->initialState;
74609 |     pNodeBase->stateTimes[ma_node_state_started] = 0;
74610 |     pNodeBase->stateTimes[ma_node_state_stopped] = (ma_uint64)(ma_int64)-1; /* Weird casting for VC6
> compatibility. */
74611 |     pNodeBase->inputBusCount = heapLayout.inputBusCount;
74612 |     pNodeBase->outputBusCount = heapLayout.outputBusCount;
74613 |
74614 |     if (heapLayout.inputBusOffset != MA_SIZE_MAX) {
74615 |         pNodeBase->pInputBuses = (ma_node_input_bus*)ma_offset_ptr(pHeap,
> heapLayout.inputBusOffset);
74616 |     } else {
74617 |         pNodeBase->pInputBuses = pNodeBase->_inputBuses;
74618 |     }
74619 |
74620 |     if (heapLayout.outputBusOffset != MA_SIZE_MAX) {
74621 |         pNodeBase->pOutputBuses = (ma_node_output_bus*)ma_offset_ptr(pHeap,
> heapLayout.outputBusOffset);
74622 |     } else {
74623 |         pNodeBase->pOutputBuses = pNodeBase->_outputBuses;
74624 |     }
74625 |
74626 |     if (heapLayout.cachedDataOffset != MA_SIZE_MAX) {
74627 |         pNodeBase->pCachedData = (float*)ma_offset_ptr(pHeap, heapLayout.cachedDataOffset);
74628 |         pNodeBase->cachedDataCapInFramesPerBus = ma_node_config_get_cache_size_in_frames(pConfig,
> pNodeGraph);

```

```

74629 |     } else {
74630 |         pNodeBase->pCachedData = NULL;
74631 |     }
74632 |
74633 |
74634 |     /* We need to run an initialization step for each input and output bus. */
74635 |     for (iInputBus = 0; iInputBus < ma_node_get_input_bus_count(pNodeBase); iInputBus += 1) {
74636 |         result = ma_node_input_bus_init(pNodeBase, iOutputBus,
> &pNodeBase->pInputBuses[iInputBus]);
74637 |         if (result != MA_SUCCESS) {
74638 |             return result;
74639 |         }
74640 |     }
74641 |
74642 |     for (iOutputBus = 0; iOutputBus < ma_node_get_output_bus_count(pNodeBase); iOutputBus += 1) {
74643 |         result = ma_node_output_bus_init(pNodeBase, iOutputBus,
> pConfig->pOutputChannels[iOutputBus], &pNodeBase->pOutputBuses[iOutputBus]);
74644 |         if (result != MA_SUCCESS) {
74645 |             return result;
74646 |         }
74647 |     }
74648 |
74649 |
74650 |     /* The cached data needs to be initialized to silence (or a sine wave tone if we're debugging).
> */
74651 |     if (pNodeBase->pCachedData != NULL) {
74652 |         ma_uint32 iBus;
74653 |
74654 |         #if 1 /* Toggle this between 0 and 1 to turn debugging on or off. 1 = fill with a sine wave
> for debugging; 0 = fill with silence. */
74655 |             /* For safety we'll go ahead and default the buffer to silence. */
74656 |             for (iBus = 0; iBus < ma_node_get_input_bus_count(pNodeBase); iBus += 1) {
74657 |                 ma_silence_pcm_frames(ma_node_get_cached_input_ptr(pNode, iBus),
> pNodeBase->cachedDataCapInFramesPerBus, ma_format_f32,
> ma_node_input_bus_get_channels(&pNodeBase->pInputBuses[iBus]));
74658 |             }
74659 |             for (iBus = 0; iBus < ma_node_get_output_bus_count(pNodeBase); iBus += 1) {
74660 |                 ma_silence_pcm_frames(ma_node_get_cached_output_ptr(pNode, iBus),
> pNodeBase->cachedDataCapInFramesPerBus, ma_format_f32,
> ma_node_output_bus_get_channels(&pNodeBase->pOutputBuses[iBus]));
74661 |             }
74662 |         #else
74663 |             /* For debugging. Default to a sine wave. */
74664 |             for (iBus = 0; iBus < ma_node_get_input_bus_count(pNodeBase); iBus += 1) {
74665 |                 ma_debug_fill_pcm_frames_with_sine_wave(ma_node_get_cached_input_ptr(pNode, iBus),
> pNodeBase->cachedDataCapInFramesPerBus, ma_format_f32,
> ma_node_input_bus_get_channels(&pNodeBase->pInputBuses[iBus]), 48000);
74666 |             }
74667 |             for (iBus = 0; iBus < ma_node_get_output_bus_count(pNodeBase); iBus += 1) {
74668 |                 ma_debug_fill_pcm_frames_with_sine_wave(ma_node_get_cached_output_ptr(pNode, iBus),
> pNodeBase->cachedDataCapInFramesPerBus, ma_format_f32,
> ma_node_output_bus_get_channels(&pNodeBase->pOutputBuses[iBus]), 48000);
74669 |             }
74670 |         #endif
74671 |     }
74672 |
74673 |     return MA_SUCCESS;
74674 | }
74675 |
74676 | MA_API ma_result ma_node_init(ma_node_graph* pNodeGraph, const ma_node_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_node* pNode)
74677 | {
74678 |     ma_result result;
74679 |     size_t heapSizeInBytes;
74680 |     void* pHeap;
74681 |
74682 |     result = ma_node_get_heap_size(pNodeGraph, pConfig, &heapSizeInBytes);
74683 |     if (result != MA_SUCCESS) {
74684 |         return result;
74685 |     }
74686 |
74687 |     if (heapSizeInBytes > 0) {
74688 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
74689 |         if (pHeap == NULL) {
74690 |             return MA_OUT_OF_MEMORY;
74691 |         }
74692 |     } else {
74693 |         pHeap = NULL;
74694 |     }

```

```

74695 |
74696 |     result = ma_node_init_preallocated(pNodeGraph, pConfig, pHeap, pNode);
74697 |     if (result != MA_SUCCESS) {
74698 |         ma_free(pHeap, pAllocationCallbacks);
74699 |         return result;
74700 |     }
74701 |
74702 |     ((ma_node_base*)pNode)->_ownsHeap = MA_TRUE;
74703 |     return MA_SUCCESS;
74704 | }
74705 |
74706 | MA_API void ma_node_uninit(ma_node* pNode, const ma_allocation_callbacks* pAllocationCallbacks)
74707 | {
74708 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74709 |
74710 |     if (pNodeBase == NULL) {
74711 |         return;
74712 |     }
74713 |
74714 |     /*
74715 |     The first thing we need to do is fully detach the node. This will detach all inputs and
74716 |     outputs. We need to do this first because it will sever the connection with the node graph and
74717 |     allow us to complete uninitialization without needing to worry about thread-safety with the
74718 |     audio thread. The detachment process will wait for any local processing of the node to finish.
74719 |     */
74720 |     ma_node_detach_full(pNode);
74721 |
74722 |     /*
74723 |     At this point the node should be completely unreferenced by the node graph and we can finish up
74724 |     the uninitialization process without needing to worry about thread-safety.
74725 |     */
74726 |     if (pNodeBase->_ownsHeap) {
74727 |         ma_free(pNodeBase->_pHeap, pAllocationCallbacks);
74728 |     }
74729 | }
74730 |
74731 | MA_API ma_node_graph* ma_node_get_node_graph(const ma_node* pNode)
74732 | {
74733 |     if (pNode == NULL) {
74734 |         return NULL;
74735 |     }
74736 |
74737 |     return ((const ma_node_base*)pNode)->pNodeGraph;
74738 | }
74739 |
74740 | MA_API ma_uint32 ma_node_get_input_bus_count(const ma_node* pNode)
74741 | {
74742 |     if (pNode == NULL) {
74743 |         return 0;
74744 |     }
74745 |
74746 |     return ((ma_node_base*)pNode)->inputBusCount;
74747 | }
74748 |
74749 | MA_API ma_uint32 ma_node_get_output_bus_count(const ma_node* pNode)
74750 | {
74751 |     if (pNode == NULL) {
74752 |         return 0;
74753 |     }
74754 |
74755 |     return ((ma_node_base*)pNode)->outputBusCount;
74756 | }
74757 |
74758 |
74759 | MA_API ma_uint32 ma_node_get_input_channels(const ma_node* pNode, ma_uint32 inputBusIndex)
74760 | {
74761 |     const ma_node_base* pNodeBase = (const ma_node_base*)pNode;
74762 |
74763 |     if (pNode == NULL) {
74764 |         return 0;
74765 |     }
74766 |
74767 |     if (inputBusIndex >= ma_node_get_input_bus_count(pNode)) {
74768 |         return 0; /* Invalid bus index. */
74769 |     }
74770 |
74771 |     return ma_node_input_bus_get_channels(&pNodeBase->pInputBuses[inputBusIndex]);
74772 | }
74773 |

```

```

74774 | MA_API ma_uint32 ma_node_get_output_channels(const ma_node* pNode, ma_uint32 outputBusIndex)
74775 | {
74776 |     const ma_node_base* pNodeBase = (const ma_node_base*)pNode;
74777 |
74778 |     if (pNode == NULL) {
74779 |         return 0;
74780 |     }
74781 |
74782 |     if (outputBusIndex >= ma_node_get_output_bus_count(pNode)) {
74783 |         return 0; /* Invalid bus index. */
74784 |     }
74785 |
74786 |     return ma_node_output_bus_get_channels(&pNodeBase->pOutputBuses[outputBusIndex]);
74787 | }
74788 |
74789 |
74790 | static ma_result ma_node_detach_full(ma_node* pNode)
74791 | {
74792 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74793 |     ma_uint32 iInputBus;
74794 |
74795 |     if (pNodeBase == NULL) {
74796 |         return MA_INVALID_ARGS;
74797 |     }
74798 |
74799 |     /*
74800 |     Make sure the node is completely detached first. This will not return until the output bus is
74801 |     guaranteed to no longer be referenced by the audio thread.
74802 |     */
74803 |     ma_node_detach_all_output_buses(pNode);
74804 |
74805 |     /*
74806 |     At this point all output buses will have been detached from the graph and we can be guaranteed
74807 |     that none of its input nodes will be getting processed by the graph. We can detach these
74808 |     without needing to worry about the audio thread touching them.
74809 |     */
74810 |     for (iInputBus = 0; iInputBus < ma_node_get_input_bus_count(pNode); iInputBus += 1) {
74811 |         ma_node_input_bus* pInputBus;
74812 |         ma_node_output_bus* pOutputBus;
74813 |
74814 |         pInputBus = &pNodeBase->pInputBuses[iInputBus];
74815 |
74816 |         /*
74817 |         This is important. We cannot be using ma_node_input_bus_first() or ma_node_input_bus_next().
74818 |         > Those
74819 |         functions are specifically for the audio thread. We'll instead just manually iterate using
74820 |         > standard
74821 |         linked list logic. We don't need to worry about the audio thread referencing these because
74822 |         > the step
74823 |         above severed the connection to the graph.
74824 |         */
74825 |         for (pOutputBus = (ma_node_output_bus*)ma_atomic_load_ptr(&pInputBus->head.pNext);
74826 |              > pOutputBus != NULL; pOutputBus = (ma_node_output_bus*)ma_atomic_load_ptr(&pInputBus->head.pNext)) {
74827 |             ma_node_detach_output_bus(pOutputBus->pNode, pOutputBus->outputBusIndex); /* This
74828 |             > won't do any waiting in practice and should be efficient. */
74829 |         }
74830 |     }
74831 |
74832 |     return MA_SUCCESS;
74833 | }
74834 |
74835 | MA_API ma_result ma_node_detach_output_bus(ma_node* pNode, ma_uint32 outputBusIndex)
74836 | {
74837 |     ma_result result = MA_SUCCESS;
74838 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74839 |     ma_node_base* pInputNodeBase;
74840 |
74841 |     if (pNode == NULL) {
74842 |         return MA_INVALID_ARGS;
74843 |     }
74844 |
74845 |     if (outputBusIndex >= ma_node_get_output_bus_count(pNode)) {
74846 |         return MA_INVALID_ARGS; /* Invalid output bus index. */
74847 |     }
74848 |
74849 |     /* We need to lock the output bus because we need to inspect the input node and grab its input
74850 |     > bus. */
74851 |     ma_node_output_bus_lock(&pNodeBase->pOutputBuses[outputBusIndex]);
74852 |     {

```

```

74847 |         pInputNodeBase = (ma_node_base*)pNodeBase->pOutputBuses[outputBusIndex].pInputNode;
74848 |         if (pInputNodeBase != NULL) {
74849 |             ma_node_input_bus_detach__no_output_bus_lock(&pInputNodeBase->pInputBuses[pNodeBase->pOu
> tputBuses[outputBusIndex].inputNodeInputBusIndex], &pNodeBase->pOutputBuses[outputBusIndex]);
74850 |         }
74851 |     }
74852 |     ma_node_output_bus_unlock(&pNodeBase->pOutputBuses[outputBusIndex]);
74853 |
74854 |     return result;
74855 | }
74856 |
74857 | MA_API ma_result ma_node_detach_all_output_buses(ma_node* pNode)
74858 | {
74859 |     ma_uint32 iOutputBus;
74860 |
74861 |     if (pNode == NULL) {
74862 |         return MA_INVALID_ARGS;
74863 |     }
74864 |
74865 |     for (iOutputBus = 0; iOutputBus < ma_node_get_output_bus_count(pNode); iOutputBus += 1) {
74866 |         ma_node_detach_output_bus(pNode, iOutputBus);
74867 |     }
74868 |
74869 |     return MA_SUCCESS;
74870 | }
74871 |
74872 | MA_API ma_result ma_node_attach_output_bus(ma_node* pNode, ma_uint32 outputBusIndex, ma_node*
> pOtherNode, ma_uint32 otherNodeInputBusIndex)
74873 | {
74874 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74875 |     ma_node_base* pOtherNodeBase = (ma_node_base*)pOtherNode;
74876 |
74877 |     if (pNodeBase == NULL || pOtherNodeBase == NULL) {
74878 |         return MA_INVALID_ARGS;
74879 |     }
74880 |
74881 |     if (pNodeBase == pOtherNodeBase) {
74882 |         return MA_INVALID_OPERATION; /* Cannot attach a node to itself. */
74883 |     }
74884 |
74885 |     if (outputBusIndex >= ma_node_get_output_bus_count(pNode) || otherNodeInputBusIndex >=
> ma_node_get_input_bus_count(pOtherNode)) {
74886 |         return MA_INVALID_OPERATION; /* Invalid bus index. */
74887 |     }
74888 |
74889 |     /* The output channel count of the output node must be the same as the input channel count of
> the input node. */
74890 |     if (ma_node_get_output_channels(pNode, outputBusIndex) != ma_node_get_input_channels(pOtherNode,
> otherNodeInputBusIndex)) {
74891 |         return MA_INVALID_OPERATION; /* Channel count is incompatible. */
74892 |     }
74893 |
74894 |     /* This will deal with detaching if the output bus is already attached to something. */
74895 |     ma_node_input_bus_attach(&pOtherNodeBase->pInputBuses[otherNodeInputBusIndex],
> &pNodeBase->pOutputBuses[outputBusIndex], pOtherNode, otherNodeInputBusIndex);
74896 |
74897 |     return MA_SUCCESS;
74898 | }
74899 |
74900 | MA_API ma_result ma_node_set_output_bus_volume(ma_node* pNode, ma_uint32 outputBusIndex, float
> volume)
74901 | {
74902 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74903 |
74904 |     if (pNodeBase == NULL) {
74905 |         return MA_INVALID_ARGS;
74906 |     }
74907 |
74908 |     if (outputBusIndex >= ma_node_get_output_bus_count(pNode)) {
74909 |         return MA_INVALID_ARGS; /* Invalid bus index. */
74910 |     }
74911 |
74912 |     return ma_node_output_bus_set_volume(&pNodeBase->pOutputBuses[outputBusIndex], volume);
74913 | }
74914 |
74915 | MA_API float ma_node_get_output_bus_volume(const ma_node* pNode, ma_uint32 outputBusIndex)
74916 | {
74917 |     const ma_node_base* pNodeBase = (const ma_node_base*)pNode;
74918 |

```



```

74919 |     if (pNodeBase == NULL) {
74920 |         return 0;
74921 |     }
74922 |
74923 |     if (outputBusIndex >= ma_node_get_output_bus_count(pNode)) {
74924 |         return 0; /* Invalid bus index. */
74925 |     }
74926 |
74927 |     return ma_node_output_bus_get_volume(&pNodeBase->pOutputBuses[outputBusIndex]);
74928 | }
74929 |
74930 | MA_API ma_result ma_node_set_state(ma_node* pNode, ma_node_state state)
74931 | {
74932 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
74933 |
74934 |     if (pNodeBase == NULL) {
74935 |         return MA_INVALID_ARGS;
74936 |     }
74937 |
74938 |     ma_atomic_exchange_i32(&pNodeBase->state, state);
74939 |
74940 |     return MA_SUCCESS;
74941 | }
74942 |
74943 | MA_API ma_node_state ma_node_get_state(const ma_node* pNode)
74944 | {
74945 |     const ma_node_base* pNodeBase = (const ma_node_base*)pNode;
74946 |
74947 |     if (pNodeBase == NULL) {
74948 |         return ma_node_state_stopped;
74949 |     }
74950 |
74951 |     return (ma_node_state)ma_atomic_load_i32(&pNodeBase->state);
74952 | }
74953 |
74954 | MA_API ma_result ma_node_set_state_time(ma_node* pNode, ma_node_state state, ma_uint64 globalTime)
74955 | {
74956 |     if (pNode == NULL) {
74957 |         return MA_INVALID_ARGS;
74958 |     }
74959 |
74960 |     /* Validation check for safety since we'll be using this as an index into stateTimes[]. */
74961 |     if (state != ma_node_state_started && state != ma_node_state_stopped) {
74962 |         return MA_INVALID_ARGS;
74963 |     }
74964 |
74965 |     ma_atomic_exchange_64(&((ma_node_base*)pNode)->stateTimes[state], globalTime);
74966 |
74967 |     return MA_SUCCESS;
74968 | }
74969 |
74970 | MA_API ma_uint64 ma_node_get_state_time(const ma_node* pNode, ma_node_state state)
74971 | {
74972 |     if (pNode == NULL) {
74973 |         return 0;
74974 |     }
74975 |
74976 |     /* Validation check for safety since we'll be using this as an index into stateTimes[]. */
74977 |     if (state != ma_node_state_started && state != ma_node_state_stopped) {
74978 |         return 0;
74979 |     }
74980 |
74981 |     return ma_atomic_load_64(&((ma_node_base*)pNode)->stateTimes[state]);
74982 | }
74983 |
74984 | MA_API ma_node_state ma_node_get_state_by_time(const ma_node* pNode, ma_uint64 globalTime)
74985 | {
74986 |     if (pNode == NULL) {
74987 |         return ma_node_state_stopped;
74988 |     }
74989 |
74990 |     return ma_node_get_state_by_time_range(pNode, globalTime, globalTime);
74991 | }
74992 |
74993 | MA_API ma_node_state ma_node_get_state_by_time_range(const ma_node* pNode, ma_uint64 globalTimeBeg,
74994 | > ma_uint64 globalTimeEnd)
74995 | {
74996 |     ma_node_state state;

```

```

74997 |     if (pNode == NULL) {
74998 |         return ma_node_state_stopped;
74999 |     }
75000 |
75001 |     state = ma_node_get_state(pNode);
75002 |
75003 |     /* An explicitly stopped node is always stopped. */
75004 |     if (state == ma_node_state_stopped) {
75005 |         return ma_node_state_stopped;
75006 |     }
75007 |
75008 |     /*
75009 |     Getting here means the node is marked as started, but it may still not be truly started due to
75010 |     its start time not having been reached yet. Also, the stop time may have also been reached in
75011 |     which case it'll be considered stopped.
75012 |     */
75013 |     if (ma_node_get_state_time(pNode, ma_node_state_stopped) < globalTimeBeg) {
75014 |         return ma_node_state_stopped; /* End time is before the start of the range. */
75015 |     }
75016 |
75017 |     if (ma_node_get_state_time(pNode, ma_node_state_started) > globalTimeEnd) {
75018 |         return ma_node_state_stopped; /* Start time is after the end of the range. */
75019 |     }
75020 |
75021 |     /* Getting here means the node is marked as started and is within its start/stop times. */
75022 |     return ma_node_state_started;
75023 | }
75024 |
75025 | MA_API ma_uint64 ma_node_get_time(const ma_node* pNode)
75026 | {
75027 |     if (pNode == NULL) {
75028 |         return 0;
75029 |     }
75030 |
75031 |     return ma_atomic_load_64(&((ma_node_base*)pNode)->localTime);
75032 | }
75033 |
75034 | MA_API ma_result ma_node_set_time(ma_node* pNode, ma_uint64 localTime)
75035 | {
75036 |     if (pNode == NULL) {
75037 |         return MA_INVALID_ARGS;
75038 |     }
75039 |
75040 |     ma_atomic_exchange_64(&((ma_node_base*)pNode)->localTime, localTime);
75041 |
75042 |     return MA_SUCCESS;
75043 | }
75044 |
75045 |
75046 |
75047 | static void ma_node_process_pcm_frames_internal(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75048 | {
75049 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
75050 |
75051 |     MA_ASSERT(pNode != NULL);
75052 |
75053 |     if (pNodeBase->vtable->onProcess) {
75054 |         pNodeBase->vtable->onProcess(pNode, ppFramesIn, pFrameCountIn, ppFramesOut, pFrameCountOut);
75055 |     }
75056 | }
75057 |
75058 | static ma_result ma_node_read_pcm_frames(ma_node* pNode, ma_uint32 outputBusIndex, float*
> pFramesOut, ma_uint32 frameCount, ma_uint32* pFramesRead, ma_uint64 globalTime)
75059 | {
75060 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
75061 |     ma_result result = MA_SUCCESS;
75062 |     ma_uint32 iInputBus;
75063 |     ma_uint32 iOutputBus;
75064 |     ma_uint32 inputBusCount;
75065 |     ma_uint32 outputBusCount;
75066 |     ma_uint32 totalFramesRead = 0;
75067 |     float* ppFramesIn[MA_MAX_NODE_BUS_COUNT];
75068 |     float* ppFramesOut[MA_MAX_NODE_BUS_COUNT];
75069 |     ma_uint64 globalTimeBeg;
75070 |     ma_uint64 globalTimeEnd;
75071 |     ma_uint64 startTime;
75072 |     ma_uint64 stopTime;
75073 |     ma_uint32 timeOffsetBeg;

```

```

75074 |     ma_uint32 timeOffsetEnd;
75075 |     ma_uint32 frameCountIn;
75076 |     ma_uint32 frameCountOut;
75077 |
75078 |     /*
75079 |     pFramesRead is mandatory. It must be used to determine how many frames were read. It's normal
> and
75080 |     expected that the number of frames read may be different to that requested. Therefore, the
> caller
75081 |     must look at this value to correctly determine how many frames were read.
75082 |     */
75083 |     MA_ASSERT(pFramesRead != NULL); /* <-- If you've triggered this assert, you're using this
> function wrong. You *must* use this variable and inspect it after the call returns. */
75084 |     if (pFramesRead == NULL) {
75085 |         return MA_INVALID_ARGS;
75086 |     }
75087 |
75088 |     *pFramesRead = 0; /* Safety. */
75089 |
75090 |     if (pNodeBase == NULL) {
75091 |         return MA_INVALID_ARGS;
75092 |     }
75093 |
75094 |     if (outputBusIndex >= ma_node_get_output_bus_count(pNodeBase)) {
75095 |         return MA_INVALID_ARGS; /* Invalid output bus index. */
75096 |     }
75097 |
75098 |     globalTimeBeg = globalTime;
75099 |     globalTimeEnd = globalTime + frameCount;
75100 |
75101 |     /* Don't do anything if we're in a stopped state. */
75102 |     if (ma_node_get_state_by_time_range(pNode, globalTimeBeg, globalTimeEnd) !=
> ma_node_state_started) {
75103 |         return MA_SUCCESS; /* We're in a stopped state. This is not an error - we just need to not
> read anything. */
75104 |     }
75105 |
75106 |     startTime = ma_node_get_state_time(pNode, ma_node_state_started);
75107 |     stopTime = ma_node_get_state_time(pNode, ma_node_state_stopped);
75108 |
75109 |     /*
75110 |     At this point we know that we are inside our start/stop times. However, we may need to adjust
75111 |     our frame count and output pointer to accommodate since we could be straddling the time period
75112 |     that this function is getting called for.
75113 |
75114 |     It's possible (and likely) that the start time does not line up with the output buffer. We
75115 |     therefore need to offset it by a number of frames to accommodate. The same thing applies for
75116 |     the stop time.
75117 |     */
75118 |     timeOffsetBeg = (globalTimeBeg < startTime) ? (ma_uint32)(startTime - globalTimeBeg) : 0;
75119 |     timeOffsetEnd = (globalTimeEnd > stopTime) ? (ma_uint32)(globalTimeEnd - stopTime) : 0;
75120 |
75121 |     /* Trim based on the start offset. We need to silence the start of the buffer. */
75122 |     if (timeOffsetBeg > 0) {
75123 |         MA_ASSERT(timeOffsetBeg <= frameCount);
75124 |         if (timeOffsetBeg > frameCount) {
75125 |             timeOffsetBeg = frameCount;
75126 |         }
75127 |
75128 |         ma_silence_pcm_frames(pFramesOut, timeOffsetBeg, ma_format_f32,
> ma_node_get_output_channels(pNode, outputBusIndex));
75129 |         pFramesOut += timeOffsetBeg * ma_node_get_output_channels(pNode, outputBusIndex);
75130 |         frameCount -= timeOffsetBeg;
75131 |     }
75132 |
75133 |     /* Trim based on the end offset. We don't need to silence the tail section because we'll just
> have a reduced value written to pFramesRead. */
75134 |     if (timeOffsetEnd > 0) {
75135 |         MA_ASSERT(timeOffsetEnd <= frameCount);
75136 |         if (timeOffsetEnd > frameCount) {
75137 |             timeOffsetEnd = frameCount;
75138 |         }
75139 |
75140 |         frameCount -= timeOffsetEnd;
75141 |     }
75142 |
75143 |     /* We run on different paths depending on the bus counts. */
75144 |     inputBusCount = ma_node_get_input_bus_count(pNode);
75145 |

```

```

75146 |     outputBusCount = ma_node_get_output_bus_count(pNode);
75147 |
75148 |     /*
75149 |     Run a simplified path when there are no inputs and one output. In this case there's nothing to
75150 |     actually read and we can go straight to output. This is a very common scenario because the vast
75151 |     majority of data source nodes will use this setup so this optimization I think is worthwhile.
75152 |     */
75153 |     if (inputBusCount == 0 && outputBusCount == 1) {
75154 |         /* Fast path. No need to read from input and no need for any caching. */
75155 |         frameCountIn = 0;
75156 |         frameCountOut = frameCount; /* Just read as much as we can. The callback will return what
> was actually read. */
75157 |
75158 |         ppFramesOut[0] = pFramesOut;
75159 |
75160 |         /*
75161 |         If it's a passthrough we won't be expecting the callback to output anything, so we'll
75162 |         need to pre-silence the output buffer.
75163 |         */
75164 |         if ((pNodeBase->vtable->flags & MA_NODE_FLAG_PASSTHROUGH) != 0) {
75165 |             ma_silence_pcm_frames(pFramesOut, frameCount, ma_format_f32,
> ma_node_get_output_channels(pNode, outputBusIndex));
75166 |         }
75167 |
75168 |         ma_node_process_pcm_frames_internal(pNode, NULL, &frameCountIn, ppFramesOut,
> &frameCountOut);
75169 |         totalFramesRead = frameCountOut;
75170 |     } else {
75171 |         /* Slow path. Need to read input data. */
75172 |         if ((pNodeBase->vtable->flags & MA_NODE_FLAG_PASSTHROUGH) != 0) {
75173 |             /*
75174 |             Fast path. We're running a passthrough. We need to read directly into the output buffer,
> but
75175 |             still fire the callback so that event handling and trigger nodes can do their thing.
> Since
75176 |             it's a passthrough there's no need for any kind of caching logic.
75177 |             */
75178 |             MA_ASSERT(outputBusCount == inputBusCount);
75179 |             MA_ASSERT(outputBusCount == 1);
75180 |             MA_ASSERT(outputBusIndex == 0);
75181 |
75182 |             /* We just read directly from input bus to output buffer, and then afterwards fire the
> callback. */
75183 |             ppFramesOut[0] = pFramesOut;
75184 |             ppFramesIn[0] = ppFramesOut[0];
75185 |
75186 |             result = ma_node_input_bus_read_pcm_frames(pNodeBase, &pNodeBase->pInputBuses[0],
> ppFramesIn[0], frameCount, &totalFramesRead, globalTime);
75187 |             if (result == MA_SUCCESS) {
75188 |                 /* Even though it's a passthrough, we still need to fire the callback. */
75189 |                 frameCountIn = totalFramesRead;
75190 |                 frameCountOut = totalFramesRead;
75191 |
75192 |                 if (totalFramesRead > 0) {
75193 |                     ma_node_process_pcm_frames_internal(pNode, (const float**)ppFramesIn,
> &frameCountIn, ppFramesOut, &frameCountOut); /* From GCC: expected 'const float **' but argument is
> of type 'float **'. Shouldn't this be implicit? Explicit cast to silence the warning. */
75194 |                 }
75195 |
75196 |                 /*
75197 |                 A passthrough should never have modified the input and output frame counts. If
> you're
75198 |                 triggering these asserts you need to fix your processing callback.
75199 |                 */
75200 |                 MA_ASSERT(frameCountIn == totalFramesRead);
75201 |                 MA_ASSERT(frameCountOut == totalFramesRead);
75202 |             }
75203 |         } else {
75204 |             /* Slow path. Need to do caching. */
75205 |             ma_uint32 framesToProcessIn;
75206 |             ma_uint32 framesToProcessOut;
75207 |             ma_bool32 consumeNullInput = MA_FALSE;
75208 |
75209 |             /*
75210 |             We use frameCount as a basis for the number of frames to read since that's what's being
75211 |             requested, however we still need to clamp it to whatever can fit in the cache.
75212 |
75213 |             This will also be used as the basis for determining how many input frames to read. This
> is

```

```

75214 |         not ideal because it can result in too many input frames being read which introduces
> latency.
75215 |         To solve this, nodes can implement an optional callback called
> onGetRequiredInputFrameCount
75216 |         which is used as hint to miniaudio as to how many input frames it needs to read at a
> time. This
75217 |         callback is completely optional, and if it's not set, miniaudio will assume
> `frameCount`.
75218 |
75219 |         This function will be called multiple times for each period of time, once for each
> output node.
75220 |         We cannot read from each input node each time this function is called. Instead we need
> to check
75221 |         whether or not this is first output bus to be read from for this time period, and if so,
> read
75222 |         from our input data.
75223 |
75224 |         To determine whether or not we're ready to read data, we check a flag. There will be one
> flag
75225 |         for each output. When the flag is set, it means data has been read previously and that
> we're
75226 |         ready to advance time forward for our input nodes by reading fresh data.
75227 |         */
75228 |         framesToProcessOut = frameCount;
75229 |         if (framesToProcessOut > pNodeBase->cachedDataCapInFramesPerBus) {
75230 |             framesToProcessOut = pNodeBase->cachedDataCapInFramesPerBus;
75231 |         }
75232 |
75233 |         framesToProcessIn = frameCount;
75234 |         if (pNodeBase->vtable->onGetRequiredInputFrameCount) {
75235 |             pNodeBase->vtable->onGetRequiredInputFrameCount(pNode, framesToProcessOut,
> &framesToProcessIn); /* <-- It does not matter if this fails. */
75236 |         }
75237 |         if (framesToProcessIn > pNodeBase->cachedDataCapInFramesPerBus) {
75238 |             framesToProcessIn = pNodeBase->cachedDataCapInFramesPerBus;
75239 |         }
75240 |
75241 |
75242 |         MA_ASSERT(framesToProcessIn <= 0xFFFF);
75243 |         MA_ASSERT(framesToProcessOut <= 0xFFFF);
75244 |
75245 |         if (ma_node_output_bus_has_read(&pNodeBase->pOutputBuses[outputBusIndex])) {
75246 |             /* Getting here means we need to do another round of processing. */
75247 |             pNodeBase->cachedFrameCountOut = 0;
75248 |
75249 |             for (;;) {
75250 |                 frameCountOut = 0;
75251 |
75252 |                 /*
75253 |                 We need to prepare our output frame pointers for processing. In the same
> iteration we need
75254 |                 to mark every output bus as unread so that future calls to this function for
> different buses
75255 |                 for the current time period don't pull in data when they should instead be
> reading from cache.
75256 |                 */
75257 |                 for (iOutputBus = 0; iOutputBus < outputBusCount; iOutputBus += 1) {
75258 |                     ma_node_output_bus_set_has_read(&pNodeBase->pOutputBuses[iOutputBus],
> MA_FALSE); /* <-- This is what tells the next calls to this function for other output buses for this
> time period to read from cache instead of pulling in more data. */
75259 |                     ppFramesOut[iOutputBus] = ma_node_get_cached_output_ptr(pNode, iOutputBus);
75260 |                 }
75261 |
75262 |                 /* We only need to read from input buses if there isn't already some data in the
> cache. */
75263 |                 if (pNodeBase->cachedFrameCountIn == 0) {
75264 |                     ma_uint32 maxFramesReadIn = 0;
75265 |
75266 |                     /* Here is where we pull in data from the input buses. This is what will
> trigger an advance in time. */
75267 |                     for (iInputBus = 0; iInputBus < inputBusCount; iInputBus += 1) {
75268 |                         ma_uint32 framesRead;
75269 |
75270 |                         /* The first thing to do is get the offset within our bulk allocation to
> store this input data. */
75271 |                         ppFramesIn[iInputBus] = ma_node_get_cached_input_ptr(pNode, iInputBus);
75272 |
75273 |                         /* Once we've determined our destination pointer we can read. Note that
> we must inspect the number of frames read and fill any leftovers with silence for safety. */

```

```

75274 |                                     result = ma_node_input_bus_read_pcm_frames(pNodeBase,
> &pNodeBase->pInputBuses[iInputBus], ppFramesIn[iInputBus], framesToProcessIn, &framesRead,
> globalTime);
75275 |                                     if (result != MA_SUCCESS) {
75276 |                                         /* It doesn't really matter if we fail because we'll just fill with
> silence. */
75277 |                                         framesRead = 0; /* Just for safety, but I don't think it's really
> needed. */
75278 |                                     }
75279 |
75280 |                                     /* TODO: Minor optimization opportunity here. If no frames were read and
> the buffer is already filled with silence, no need to re-silence it. */
75281 |                                     /* Any leftover frames need to be silenced for safety. */
75282 |                                     if (framesRead < framesToProcessIn) {
75283 |                                         ma_silence_pcm_frames(ppFramesIn[iInputBus] + (framesRead *
> ma_node_get_input_channels(pNodeBase, iInputBus)), (framesToProcessIn - framesRead), ma_format_f32,
> ma_node_get_input_channels(pNodeBase, iInputBus));
75284 |                                     }
75285 |
75286 |                                     maxFramesReadIn = ma_max(maxFramesReadIn, framesRead);
75287 |                                 }
75288 |
75289 |                                     /* This was a fresh load of input data so reset our consumption counter. */
75290 |                                     pNodeBase->consumedFrameCountIn = 0;
75291 |
75292 |                                     /*
75293 |                                     We don't want to keep processing if there's nothing to process, so set the
> number of cached
75294 |                                     input frames to the maximum number we read from each attachment (the lesser
> will be padded
75295 |                                     with silence). If we didn't read anything, this will be set to 0 and the
> entire buffer will
75296 |                                     have been assigned to silence. This being equal to 0 is an important
> property for us because
75297 |                                     it allows us to detect when NULL can be passed into the processing callback
> for the input
75298 |                                     buffer for the purpose of continuous processing.
75299 |                                     */
75300 |                                     pNodeBase->cachedFrameCountIn = (ma_uint16)maxFramesReadIn;
75301 |                                 } else {
75302 |                                     /* We don't need to read anything, but we do need to prepare our input frame
> pointers. */
75303 |                                     for (iInputBus = 0; iInputBus < inputBusCount; iInputBus += 1) {
75304 |                                         ppFramesIn[iInputBus] = ma_node_get_cached_input_ptr(pNode, iInputBus) +
> (pNodeBase->consumedFrameCountIn * ma_node_get_input_channels(pNodeBase, iInputBus));
75305 |                                     }
75306 |                                 }
75307 |
75308 |                                     /*
75309 |                                     At this point we have our input data so now we need to do some processing.
> Sneaky little
75310 |                                     optimization here - we can set the pointer to the output buffer for this output
> bus so
75311 |                                     that the final copy into the output buffer is done directly by onProcess().
75312 |                                     */
75313 |                                     if (pFramesOut != NULL) {
75314 |                                         ppFramesOut[outputBusIndex] = ma_offset_pcm_frames_ptr_f32(pFramesOut,
> pNodeBase->cachedFrameCountOut, ma_node_get_output_channels(pNode, outputBusIndex));
75315 |                                     }
75316 |
75317 |
75318 |                                     /* Give the processing function the entire capacity of the output buffer. */
75319 |                                     frameCountOut = (framesToProcessOut - pNodeBase->cachedFrameCountOut);
75320 |
75321 |                                     /*
75322 |                                     We need to treat nodes with continuous processing a little differently. For
> these ones,
75323 |                                     we always want to fire the callback with the requested number of frames,
> regardless of
75324 |                                     pNodeBase->cachedFrameCountIn, which could be 0. Also, we want to check if we
> can pass
75325 |                                     in NULL for the input buffer to the callback.
75326 |                                     */
75327 |                                     if ((pNodeBase->vtable->flags & MA_NODE_FLAG_CONTINUOUS_PROCESSING) != 0) {
75328 |                                         /* We're using continuous processing. Make sure we specify the whole frame
> count at all times. */
75329 |                                         frameCountIn = framesToProcessIn; /* Give the processing function as much
> input data as we've got in the buffer, including any silenced padding from short reads. */
75330 |                                     }

```

```

75331 |             if ((pNodeBase->vtable->flags & MA_NODE_FLAG_ALLOW_NULL_INPUT) != 0 &&
> pNodeBase->consumedFrameCountIn == 0 && pNodeBase->cachedFrameCountIn == 0) {
75332 |                 consumeNullInput = MA_TRUE;
75333 |             } else {
75334 |                 consumeNullInput = MA_FALSE;
75335 |             }
75336 |
75337 |             /*
75338 |             Since we're using continuous processing we're always passing in a full frame
> count
75339 |             regardless of how much input data was read. If this is greater than what we
> read as
75340 |             input, we'll end up with an underflow. We instead need to make sure our
> cached frame
75341 |             count is set to the number of frames we'll be passing to the data callback.
> Not
75342 |             doing this will result in an underflow when we "consume" the cached data
> later on.
75343 |
75344 |             Note that this check needs to be done after the "consumeNullInput" check
> above because
75345 |             we use the property of cachedFrameCountIn being 0 to determine whether or
> not we
75346 |             should be passing in a null pointer to the processing callback for when the
> node is
75347 |             configured with MA_NODE_FLAG_ALLOW_NULL_INPUT.
75348 |             */
75349 |             if (pNodeBase->cachedFrameCountIn < (ma_uint16)frameCountIn) {
75350 |                 pNodeBase->cachedFrameCountIn = (ma_uint16)frameCountIn;
75351 |             }
75352 |             } else {
75353 |                 frameCountIn = pNodeBase->cachedFrameCountIn; /* Give the processing
> function as much valid input data as we've got. */
75354 |                 consumeNullInput = MA_FALSE;
75355 |             }
75356 |
75357 |             /*
75358 |             Process data slightly differently depending on whether or not we're consuming
> NULL
75359 |             input (checked just above).
75360 |             */
75361 |             if (consumeNullInput) {
75362 |                 ma_node_process_pcm_frames_internal(pNode, NULL, &frameCountIn, ppFramesOut,
> &frameCountOut);
75363 |             } else {
75364 |                 /*
75365 |                 We want to skip processing if there's no input data, but we can only do that
> safely if
75366 |                 we know that there is no chance of any output frames being produced. If
> continuous
75367 |                 processing is being used, this won't be a problem because the input frame
> count will
75368 |                 always be non-0. However, if continuous processing is *not* enabled and
> input and output
75369 |                 data is processed at different rates, we still need to process that last
> input frame
75370 |                 because there could be a few excess output frames needing to be produced
> from cached
75371 |                 data. The `MA_NODE_FLAG_DIFFERENT_PROCESSING_RATES` flag is used as the
> indicator for
75372 |                 determining whether or not we need to process the node even when there are
> no input
75373 |                 frames available right now.
75374 |                 */
75375 |                 if (frameCountIn > 0 || (pNodeBase->vtable->flags &
> MA_NODE_FLAG_DIFFERENT_PROCESSING_RATES) != 0) {
75376 |                     ma_node_process_pcm_frames_internal(pNode, (const float**)ppFramesIn,
> &frameCountIn, ppFramesOut, &frameCountOut); /* From GCC: expected 'const float **' but argument
> is of type 'float ***'. Shouldn't this be implicit? Explicit cast to silence the warning. */
75377 |                 } else {
75378 |                     frameCountOut = 0; /* No data was processed. */
75379 |                 }
75380 |             }
75381 |
75382 |             /*
75383 |             Thanks to our sneaky optimization above we don't need to do any data copying
> directly into
75384 |             the output buffer - the onProcess() callback just did that for us. We do,
> however, need to

```



```

75385 |         apply the number of input and output frames that were processed. Note that due
> to continuous
75386 |         processing above, we need to do explicit checks here. If we just consumed a NULL
> input
75387 |         buffer it means that no actual input data was processed from the internal
> buffers and we
75388 |         don't want to be modifying any counters.
75389 |         */
75390 |         if (consumeNullInput == MA_FALSE) {
75391 |             pNodeBase->consumedFrameCountIn += (ma_uint16)frameCountIn;
75392 |             pNodeBase->cachedFrameCountIn -= (ma_uint16)frameCountIn;
75393 |         }
75394 |
75395 |         /* The cached output frame count is always equal to what we just read. */
75396 |         pNodeBase->cachedFrameCountOut += (ma_uint16)frameCountOut;
75397 |
75398 |         /* If we couldn't process any data, we're done. The loop needs to be terminated
> here or else we'll get stuck in a loop. */
75399 |         if (pNodeBase->cachedFrameCountOut == framesToProcessOut || (frameCountOut == 0
> && frameCountIn == 0)) {
75400 |             break;
75401 |         }
75402 |     }
75403 | } else {
75404 |     /*
75405 |         We're not needing to read anything from the input buffer so just read directly from
> our
75406 |         already-processed data.
75407 |         */
75408 |         if (pFramesOut != NULL) {
75409 |             ma_copy_pcm_frames(pFramesOut, ma_node_get_cached_output_ptr(pNodeBase,
> outputBusIndex), pNodeBase->cachedFrameCountOut, ma_format_f32,
> ma_node_get_output_channels(pNodeBase, outputBusIndex));
75410 |         }
75411 |     }
75412 |
75413 |     /* The number of frames read is always equal to the number of cached output frames. */
75414 |     totalFramesRead = pNodeBase->cachedFrameCountOut;
75415 |
75416 |     /* Now that we've read the data, make sure our read flag is set. */
75417 |     ma_node_output_bus_set_has_read(&pNodeBase->pOutputBuses[outputBusIndex], MA_TRUE);
75418 | }
75419 | }
75420 |
75421 | /* Apply volume, if necessary. */
75422 | ma_apply_volume_factor_f32(pFramesOut, totalFramesRead * ma_node_get_output_channels(pNodeBase,
> outputBusIndex), ma_node_output_bus_get_volume(&pNodeBase->pOutputBuses[outputBusIndex]));
75423 |
75424 | /* Advance our local time forward. */
75425 | ma_atomic_fetch_add_64(&pNodeBase->localTime, (ma_uint64)totalFramesRead);
75426 |
75427 | *pFramesRead = totalFramesRead + timeOffsetBeg; /* Must include the silenced section at the
> start of the buffer. */
75428 | return result;
75429 | }
75430 |
75431 |
75432 |
75433 |
75434 | /* Data source node. */
75435 | MA_API ma_data_source_node_config ma_data_source_node_config_init(ma_data_source* pDataSource)
75436 | {
75437 |     ma_data_source_node_config config;
75438 |
75439 |     MA_ZERO_OBJECT(&config);
75440 |     config.nodeConfig = ma_node_config_init();
75441 |     config.pDataSource = pDataSource;
75442 |
75443 |     return config;
75444 | }
75445 |
75446 |
75447 | static void ma_data_source_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn,
> ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75448 | {
75449 |     ma_data_source_node* pDataSourceNode = (ma_data_source_node*)pNode;
75450 |     ma_format format;
75451 |     ma_uint32 channels;
75452 |     ma_uint32 frameCount;

```

```

75453 |     ma_uint64 framesRead = 0;
75454 |
75455 |     MA_ASSERT(pDataSourceNode != NULL);
75456 |     MA_ASSERT(pDataSourceNode->pDataSource != NULL);
75457 |     MA_ASSERT(ma_node_get_input_bus_count(pDataSourceNode) == 0);
75458 |     MA_ASSERT(ma_node_get_output_bus_count(pDataSourceNode) == 1);
75459 |
75460 |     /* We don't want to read from ppFramesIn at all. Instead we read from the data source. */
75461 |     (void)ppFramesIn;
75462 |     (void)pFrameCountIn;
75463 |
75464 |     frameCount = *pFrameCountOut;
75465 |
75466 |     /* miniaudio should never be calling this with a frame count of zero. */
75467 |     MA_ASSERT(frameCount > 0);
75468 |
75469 |     if (ma_data_source_get_data_format(pDataSourceNode->pDataSource, &format, &channels, NULL, NULL,
> 0) == MA_SUCCESS) { /* <-- Don't care about sample rate here. */
75470 |         /* The node graph system requires samples be in floating point format. This is checked in
> ma_data_source_node_init(). */
75471 |         MA_ASSERT(format == ma_format_f32);
75472 |         (void)format; /* Just to silence some static analysis tools. */
75473 |
75474 |         ma_data_source_read_pcm_frames(pDataSourceNode->pDataSource, ppFramesOut[0], frameCount,
> &framesRead);
75475 |     }
75476 |
75477 |     *pFrameCountOut = (ma_uint32)framesRead;
75478 | }
75479 |
75480 | static ma_node_vtable g_ma_data_source_node_vtable =
75481 | {
75482 |     ma_data_source_node_process_pcm_frames,
75483 |     NULL, /* onGetRequiredInputFrameCount */
75484 |     0, /* 0 input buses. */
75485 |     1, /* 1 output bus. */
75486 |     0
75487 | };
75488 |
75489 | MA_API ma_result ma_data_source_node_init(ma_node_graph* pNodeGraph, const
> ma_data_source_node_config* pConfig, const ma_allocation_callbacks* pAllocationCallbacks,
> ma_data_source_node* pDataSourceNode)
75490 | {
75491 |     ma_result result;
75492 |     ma_format format; /* For validating the format, which must be ma_format_f32. */
75493 |     ma_uint32 channels; /* For specifying the channel count of the output bus. */
75494 |     ma_node_config baseConfig;
75495 |
75496 |     if (pDataSourceNode == NULL) {
75497 |         return MA_INVALID_ARGS;
75498 |     }
75499 |
75500 |     MA_ZERO_OBJECT(pDataSourceNode);
75501 |
75502 |     if (pConfig == NULL) {
75503 |         return MA_INVALID_ARGS;
75504 |     }
75505 |
75506 |     result = ma_data_source_get_data_format(pConfig->pDataSource, &format, &channels, NULL, NULL,
> 0); /* Don't care about sample rate. This will check pDataSource for NULL. */
75507 |     if (result != MA_SUCCESS) {
75508 |         return result;
75509 |     }
75510 |
75511 |     MA_ASSERT(format == ma_format_f32); /* <-- If you've triggered this it means your data source is
> not outputting floating-point samples. You must configure your data source to use ma_format_f32. */
75512 |     if (format != ma_format_f32) {
75513 |         return MA_INVALID_ARGS; /* Invalid format. */
75514 |     }
75515 |
75516 |     /* The channel count is defined by the data source. If the caller has manually changed the
> channels we just ignore it. */
75517 |     baseConfig = pConfig->nodeConfig;
75518 |     baseConfig.vtable = &g_ma_data_source_node_vtable; /* Explicitly set the vtable here to prevent
> callers from setting it incorrectly. */
75519 |
75520 |     /*
75521 |     The channel count is defined by the data source. It is invalid for the caller to manually set
75522 |     the channel counts in the config. `ma_data_source_node_config_init()` will have defaulted the

```

```

75523 |     channel count pointer to NULL which is how it must remain. If you trigger any of these asserts
75524 |     it means you're explicitly setting the channel count. Instead, configure the output channel
75525 |     count of your data source to be the necessary channel count.
75526 |     */
75527 |     if (baseConfig.pOutputChannels != NULL) {
75528 |         return MA_INVALID_ARGS;
75529 |     }
75530 |
75531 |     baseConfig.pOutputChannels = &channels;
75532 |
75533 |     result = ma_node_init(pNodeGraph, &baseConfig, pAllocationCallbacks, &pDataSourceNode->base);
75534 |     if (result != MA_SUCCESS) {
75535 |         return result;
75536 |     }
75537 |
75538 |     pDataSourceNode->pDataSource = pConfig->pDataSource;
75539 |
75540 |     return MA_SUCCESS;
75541 | }
75542 |
75543 | MA_API void ma_data_source_node_uninit(ma_data_source_node* pDataSourceNode, const
> ma_allocation_callbacks* pAllocationCallbacks)
75544 | {
75545 |     ma_node_uninit(&pDataSourceNode->base, pAllocationCallbacks);
75546 | }
75547 |
75548 | MA_API ma_result ma_data_source_node_set_looping(ma_data_source_node* pDataSourceNode, ma_bool32
> isLooping)
75549 | {
75550 |     if (pDataSourceNode == NULL) {
75551 |         return MA_INVALID_ARGS;
75552 |     }
75553 |
75554 |     return ma_data_source_set_looping(pDataSourceNode->pDataSource, isLooping);
75555 | }
75556 |
75557 | MA_API ma_bool32 ma_data_source_node_is_looping(ma_data_source_node* pDataSourceNode)
75558 | {
75559 |     if (pDataSourceNode == NULL) {
75560 |         return MA_FALSE;
75561 |     }
75562 |
75563 |     return ma_data_source_is_looping(pDataSourceNode->pDataSource);
75564 | }
75565 |
75566 |
75567 |
75568 | /* Splitter Node. */
75569 | MA_API ma_splitter_node_config ma_splitter_node_config_init(ma_uint32 channels)
75570 | {
75571 |     ma_splitter_node_config config;
75572 |
75573 |     MA_ZERO_OBJECT(&config);
75574 |     config.nodeConfig = ma_node_config_init();
75575 |     config.channels = channels;
75576 |     config.outputBusCount = 2;
75577 |
75578 |     return config;
75579 | }
75580 |
75581 |
75582 | static void ma_splitter_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75583 | {
75584 |     ma_node_base* pNodeBase = (ma_node_base*)pNode;
75585 |     ma_uint32 iOutputBus;
75586 |     ma_uint32 channels;
75587 |
75588 |     MA_ASSERT(pNodeBase != NULL);
75589 |     MA_ASSERT(ma_node_get_input_bus_count(pNodeBase) == 1);
75590 |
75591 |     /* We don't need to consider the input frame count - it'll be the same as the output frame count
> and we process everything. */
75592 |     (void)pFrameCountIn;
75593 |
75594 |     /* NOTE: This assumes the same number of channels for all inputs and outputs. This was checked
> in ma_splitter_node_init(). */
75595 |     channels = ma_node_get_input_channels(pNodeBase, 0);
75596 |

```

```

75597 |     /* Splitting is just copying the first input bus and copying it over to each output bus. */
75598 |     for (iOutputBus = 0; iOutputBus < ma_node_get_output_bus_count(pNodeBase); iOutputBus += 1) {
75599 |         ma_copy_pcm_frames(ppFramesOut[iOutputBus], ppFramesIn[0], *pFrameCountOut, ma_format_f32,
> channels);
75600 |     }
75601 | }
75602 |
75603 | static ma_node_vtable g_ma_splitter_node_vtable =
75604 | {
75605 |     ma_splitter_node_process_pcm_frames,
75606 |     NULL, /* onGetRequiredInputFrameCount */
75607 |     1, /* 1 input bus. */
75608 |     MA_NODE_BUS_COUNT_UNKNOWN, /* The output bus count is specified on a per-node basis. */
75609 |     0
75610 | };
75611 |
75612 | MA_API ma_result ma_splitter_node_init(ma_node_graph* pNodeGraph, const ma_splitter_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_splitter_node* pSplitterNode)
75613 | {
75614 |     ma_result result;
75615 |     ma_node_config baseConfig;
75616 |     ma_uint32 pInputChannels[1];
75617 |     ma_uint32 pOutputChannels[MA_MAX_NODE_BUS_COUNT];
75618 |     ma_uint32 iOutputBus;
75619 |
75620 |     if (pSplitterNode == NULL) {
75621 |         return MA_INVALID_ARGS;
75622 |     }
75623 |
75624 |     MA_ZERO_OBJECT(pSplitterNode);
75625 |
75626 |     if (pConfig == NULL) {
75627 |         return MA_INVALID_ARGS;
75628 |     }
75629 |
75630 |     if (pConfig->outputBusCount > MA_MAX_NODE_BUS_COUNT) {
75631 |         return MA_INVALID_ARGS; /* Too many output buses. */
75632 |     }
75633 |
75634 |     /* Splitters require the same number of channels between inputs and outputs. */
75635 |     pInputChannels[0] = pConfig->channels;
75636 |     for (iOutputBus = 0; iOutputBus < pConfig->outputBusCount; iOutputBus += 1) {
75637 |         pOutputChannels[iOutputBus] = pConfig->channels;
75638 |     }
75639 |
75640 |     baseConfig = pConfig->nodeConfig;
75641 |     baseConfig.vtable = &g_ma_splitter_node_vtable;
75642 |     baseConfig.pInputChannels = pInputChannels;
75643 |     baseConfig.pOutputChannels = pOutputChannels;
75644 |     baseConfig.outputBusCount = pConfig->outputBusCount;
75645 |
75646 |     result = ma_node_init(pNodeGraph, &baseConfig, pAllocationCallbacks, &pSplitterNode->base);
75647 |     if (result != MA_SUCCESS) {
75648 |         return result; /* Failed to initialize the base node. */
75649 |     }
75650 |
75651 |     return MA_SUCCESS;
75652 | }
75653 |
75654 | MA_API void ma_splitter_node_uninit(ma_splitter_node* pSplitterNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
75655 | {
75656 |     ma_node_uninit(pSplitterNode, pAllocationCallbacks);
75657 | }
75658 |
75659 |
75660 | /*
75661 | Biquad Node
75662 | */
75663 | MA_API ma_biquad_node_config ma_biquad_node_config_init(ma_uint32 channels, float b0, float b1,
> float b2, float a0, float a1, float a2)
75664 | {
75665 |     ma_biquad_node_config config;
75666 |
75667 |     config.nodeConfig = ma_node_config_init();
75668 |     config.biquad = ma_biquad_config_init(ma_format_f32, channels, b0, b1, b2, a0, a1, a2);
75669 |
75670 |     return config;
75671 | }

```

```

75672 |
75673 | static void ma_biquad_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75674 | {
75675 |     ma_biquad_node* pLPFNode = (ma_biquad_node*)pNode;
75676 |
75677 |     MA_ASSERT(pNode != NULL);
75678 |     (void)pFrameCountIn;
75679 |
75680 |     ma_biquad_process_pcm_frames(&pLPFNode->biquad, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
75681 | }
75682 |
75683 | static ma_node_vtable g_ma_biquad_node_vtable =
75684 | {
75685 |     ma_biquad_node_process_pcm_frames,
75686 |     NULL, /* onGetRequiredInputFrameCount */
75687 |     1, /* One input. */
75688 |     1, /* One output. */
75689 |     0 /* Default flags. */
75690 | };
75691 |
75692 | MA_API ma_result ma_biquad_node_init(ma_node_graph* pNodeGraph, const ma_biquad_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_biquad_node* pNode)
75693 | {
75694 |     ma_result result;
75695 |     ma_node_config baseNodeConfig;
75696 |
75697 |     if (pNode == NULL) {
75698 |         return MA_INVALID_ARGS;
75699 |     }
75700 |
75701 |     MA_ZERO_OBJECT(pNode);
75702 |
75703 |     if (pConfig == NULL) {
75704 |         return MA_INVALID_ARGS;
75705 |     }
75706 |
75707 |     if (pConfig->biquad.format != ma_format_f32) {
75708 |         return MA_INVALID_ARGS; /* The format must be f32. */
75709 |     }
75710 |
75711 |     result = ma_biquad_init(&pConfig->biquad, pAllocationCallbacks, &pNode->biquad);
75712 |     if (result != MA_SUCCESS) {
75713 |         return result;
75714 |     }
75715 |
75716 |     baseNodeConfig = ma_node_config_init();
75717 |     baseNodeConfig.vtable = &g_ma_biquad_node_vtable;
75718 |     baseNodeConfig.pInputChannels = &pConfig->biquad.channels;
75719 |     baseNodeConfig.pOutputChannels = &pConfig->biquad.channels;
75720 |
75721 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
75722 |     if (result != MA_SUCCESS) {
75723 |         return result;
75724 |     }
75725 |
75726 |     return result;
75727 | }
75728 |
75729 | MA_API ma_result ma_biquad_node_reinit(const ma_biquad_config* pConfig, ma_biquad_node* pNode)
75730 | {
75731 |     ma_biquad_node* pLPFNode = (ma_biquad_node*)pNode;
75732 |
75733 |     MA_ASSERT(pNode != NULL);
75734 |
75735 |     return ma_biquad_reinit(pConfig, &pLPFNode->biquad);
75736 | }
75737 |
75738 | MA_API void ma_biquad_node_uninit(ma_biquad_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
75739 | {
75740 |     ma_biquad_node* pLPFNode = (ma_biquad_node*)pNode;
75741 |
75742 |     if (pNode == NULL) {
75743 |         return;
75744 |     }
75745 |
75746 |     ma_node_uninit(pNode, pAllocationCallbacks);
75747 |     ma_biquad_uninit(&pLPFNode->biquad, pAllocationCallbacks);

```

```

75748 | }
75749 |
75750 |
75751 |
75752 | /*
75753 | Low Pass Filter Node
75754 | */
75755 | MA_API ma_lpf_node_config ma_lpf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> cutoffFrequency, ma_uint32 order)
75756 | {
75757 |     ma_lpf_node_config config;
75758 |
75759 |     config.nodeConfig = ma_node_config_init();
75760 |     config.lpf = ma_lpf_config_init(ma_format_f32, channels, sampleRate, cutoffFrequency, order);
75761 |
75762 |     return config;
75763 | }
75764 |
75765 | static void ma_lpf_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75766 | {
75767 |     ma_lpf_node* pLPFNode = (ma_lpf_node*)pNode;
75768 |
75769 |     MA_ASSERT(pNode != NULL);
75770 |     (void)pFrameCountIn;
75771 |
75772 |     ma_lpf_process_pcm_frames(&pLPFNode->lpf, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
75773 | }
75774 |
75775 | static ma_node_vtable g_ma_lpf_node_vtable =
75776 | {
75777 |     ma_lpf_node_process_pcm_frames,
75778 |     NULL, /* onGetRequiredInputFrameCount */
75779 |     1, /* One input. */
75780 |     1, /* One output. */
75781 |     0 /* Default flags. */
75782 | };
75783 |
75784 | MA_API ma_result ma_lpf_node_init(ma_node_graph* pNodeGraph, const ma_lpf_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_lpf_node* pNode)
75785 | {
75786 |     ma_result result;
75787 |     ma_node_config baseNodeConfig;
75788 |
75789 |     if (pNode == NULL) {
75790 |         return MA_INVALID_ARGS;
75791 |     }
75792 |
75793 |     MA_ZERO_OBJECT(pNode);
75794 |
75795 |     if (pConfig == NULL) {
75796 |         return MA_INVALID_ARGS;
75797 |     }
75798 |
75799 |     if (pConfig->lpf.format != ma_format_f32) {
75800 |         return MA_INVALID_ARGS; /* The format must be f32. */
75801 |     }
75802 |
75803 |     result = ma_lpf_init(&pConfig->lpf, pAllocationCallbacks, &pNode->lpf);
75804 |     if (result != MA_SUCCESS) {
75805 |         return result;
75806 |     }
75807 |
75808 |     baseNodeConfig = ma_node_config_init();
75809 |     baseNodeConfig.vtable = &g_ma_lpf_node_vtable;
75810 |     baseNodeConfig.pInputChannels = &pConfig->lpf.channels;
75811 |     baseNodeConfig.pOutputChannels = &pConfig->lpf.channels;
75812 |
75813 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
75814 |     if (result != MA_SUCCESS) {
75815 |         return result;
75816 |     }
75817 |
75818 |     return result;
75819 | }
75820 |
75821 | MA_API ma_result ma_lpf_node_reinit(const ma_lpf_config* pConfig, ma_lpf_node* pNode)
75822 | {
75823 |     ma_lpf_node* pLPFNode = (ma_lpf_node*)pNode;

```

```

75824 |
75825 |     if (pNode == NULL) {
75826 |         return MA_INVALID_ARGS;
75827 |     }
75828 |
75829 |     return ma_lpf_reinit(pConfig, &pLPFNode->lpf);
75830 | }
75831 |
75832 | MA_API void ma_lpf_node_uninit(ma_lpf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
75833 | {
75834 |     ma_lpf_node* pLPFNode = (ma_lpf_node*)pNode;
75835 |
75836 |     if (pNode == NULL) {
75837 |         return;
75838 |     }
75839 |
75840 |     ma_node_uninit(pNode, pAllocationCallbacks);
75841 |     ma_lpf_uninit(&pLPFNode->lpf, pAllocationCallbacks);
75842 | }
75843 |
75844 |
75845 |
75846 | /*
75847 | High Pass Filter Node
75848 | */
75849 | MA_API ma_hpf_node_config ma_hpf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> cutoffFrequency, ma_uint32 order)
75850 | {
75851 |     ma_hpf_node_config config;
75852 |
75853 |     config.nodeConfig = ma_node_config_init();
75854 |     config.hpf = ma_hpf_config_init(ma_format_f32, channels, sampleRate, cutoffFrequency, order);
75855 |
75856 |     return config;
75857 | }
75858 |
75859 | static void ma_hpf_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75860 | {
75861 |     ma_hpf_node* pHPFNode = (ma_hpf_node*)pNode;
75862 |
75863 |     MA_ASSERT(pNode != NULL);
75864 |     (void)pFrameCountIn;
75865 |
75866 |     ma_hpf_process_pcm_frames(&pHPFNode->hpf, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
75867 | }
75868 |
75869 | static ma_node_vtable g_ma_hpf_node_vtable =
75870 | {
75871 |     ma_hpf_node_process_pcm_frames,
75872 |     NULL, /* onGetRequiredInputFrameCount */
75873 |     1, /* One input. */
75874 |     1, /* One output. */
75875 |     0 /* Default flags. */
75876 | };
75877 |
75878 | MA_API ma_result ma_hpf_node_init(ma_node_graph* pNodeGraph, const ma_hpf_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_hpf_node* pNode)
75879 | {
75880 |     ma_result result;
75881 |     ma_node_config baseNodeConfig;
75882 |
75883 |     if (pNode == NULL) {
75884 |         return MA_INVALID_ARGS;
75885 |     }
75886 |
75887 |     MA_ZERO_OBJECT(pNode);
75888 |
75889 |     if (pConfig == NULL) {
75890 |         return MA_INVALID_ARGS;
75891 |     }
75892 |
75893 |     if (pConfig->hpf.format != ma_format_f32) {
75894 |         return MA_INVALID_ARGS; /* The format must be f32. */
75895 |     }
75896 |
75897 |     result = ma_hpf_init(&pConfig->hpf, pAllocationCallbacks, &pNode->hpf);
75898 |     if (result != MA_SUCCESS) {

```



```

75899 |         return result;
75900 |     }
75901 |
75902 |     baseNodeConfig = ma_node_config_init();
75903 |     baseNodeConfig.vtable = &g_ma_hpf_node_vtable;
75904 |     baseNodeConfig.pInputChannels = &pConfig->hpf.channels;
75905 |     baseNodeConfig.pOutputChannels = &pConfig->hpf.channels;
75906 |
75907 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
75908 |     if (result != MA_SUCCESS) {
75909 |         return result;
75910 |     }
75911 |
75912 |     return result;
75913 | }
75914 |
75915 | MA_API ma_result ma_hpf_node_reinit(const ma_hpf_config* pConfig, ma_hpf_node* pNode)
75916 | {
75917 |     ma_hpf_node* pHPFNode = (ma_hpf_node*)pNode;
75918 |
75919 |     if (pNode == NULL) {
75920 |         return MA_INVALID_ARGS;
75921 |     }
75922 |
75923 |     return ma_hpf_reinit(pConfig, &pHPFNode->hpf);
75924 | }
75925 |
75926 | MA_API void ma_hpf_node_uninit(ma_hpf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
75927 | {
75928 |     ma_hpf_node* pHPFNode = (ma_hpf_node*)pNode;
75929 |
75930 |     if (pNode == NULL) {
75931 |         return;
75932 |     }
75933 |
75934 |     ma_node_uninit(pNode, pAllocationCallbacks);
75935 |     ma_hpf_uninit(&pHPFNode->hpf, pAllocationCallbacks);
75936 | }
75937 |
75938 |
75939 |
75940 |
75941 | /*
75942 | Band Pass Filter Node
75943 | */
75944 | MA_API ma_bpf_node_config ma_bpf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> cutoffFrequency, ma_uint32 order)
75945 | {
75946 |     ma_bpf_node_config config;
75947 |
75948 |     config.nodeConfig = ma_node_config_init();
75949 |     config.bpf = ma_bpf_config_init(ma_format_f32, channels, sampleRate, cutoffFrequency, order);
75950 |
75951 |     return config;
75952 | }
75953 |
75954 | static void ma_bpf_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
75955 | {
75956 |     ma_bpf_node* pBPFNode = (ma_bpf_node*)pNode;
75957 |
75958 |     MA_ASSERT(pNode != NULL);
75959 |     (void)pFrameCountIn;
75960 |
75961 |     ma_bpf_process_pcm_frames(&pBPFNode->bpf, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
75962 | }
75963 |
75964 | static ma_node_vtable g_ma_bpf_node_vtable =
75965 | {
75966 |     ma_bpf_node_process_pcm_frames,
75967 |     NULL, /* onGetRequiredInputFrameCount */
75968 |     1, /* One input. */
75969 |     1, /* One output. */
75970 |     0 /* Default flags. */
75971 | };
75972 |
75973 | MA_API ma_result ma_bpf_node_init(ma_node_graph* pNodeGraph, const ma_bpf_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_bpf_node* pNode)

```

```

75974 | {
75975 |     ma_result result;
75976 |     ma_node_config baseNodeConfig;
75977 |
75978 |     if (pNode == NULL) {
75979 |         return MA_INVALID_ARGS;
75980 |     }
75981 |
75982 |     MA_ZERO_OBJECT(pNode);
75983 |
75984 |     if (pConfig == NULL) {
75985 |         return MA_INVALID_ARGS;
75986 |     }
75987 |
75988 |     if (pConfig->bpf.format != ma_format_f32) {
75989 |         return MA_INVALID_ARGS; /* The format must be f32. */
75990 |     }
75991 |
75992 |     result = ma_bpf_init(&pConfig->bpf, pAllocationCallbacks, &pNode->bpf);
75993 |     if (result != MA_SUCCESS) {
75994 |         return result;
75995 |     }
75996 |
75997 |     baseNodeConfig = ma_node_config_init();
75998 |     baseNodeConfig.vtable = &g_ma_bpf_node_vtable;
75999 |     baseNodeConfig.pInputChannels = &pConfig->bpf.channels;
76000 |     baseNodeConfig.pOutputChannels = &pConfig->bpf.channels;
76001 |
76002 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
76003 |     if (result != MA_SUCCESS) {
76004 |         return result;
76005 |     }
76006 |
76007 |     return result;
76008 | }
76009 |
76010 | MA_API ma_result ma_bpf_node_reinit(const ma_bpf_config* pConfig, ma_bpf_node* pNode)
76011 | {
76012 |     ma_bpf_node* pBPFNode = (ma_bpf_node*)pNode;
76013 |
76014 |     if (pNode == NULL) {
76015 |         return MA_INVALID_ARGS;
76016 |     }
76017 |
76018 |     return ma_bpf_reinit(pConfig, &pBPFNode->bpf);
76019 | }
76020 |
76021 | MA_API void ma_bpf_node_uninit(ma_bpf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
76022 | {
76023 |     ma_bpf_node* pBPFNode = (ma_bpf_node*)pNode;
76024 |
76025 |     if (pNode == NULL) {
76026 |         return;
76027 |     }
76028 |
76029 |     ma_node_uninit(pNode, pAllocationCallbacks);
76030 |     ma_bpf_uninit(&pBPFNode->bpf, pAllocationCallbacks);
76031 | }
76032 |
76033 |
76034 |
76035 | /*
76036 | Notching Filter Node
76037 | */
76038 | MA_API ma_notch_node_config ma_notch_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
> double q, double frequency)
76039 | {
76040 |     ma_notch_node_config config;
76041 |
76042 |     config.nodeConfig = ma_node_config_init();
76043 |     config.notch = ma_notch2_config_init(ma_format_f32, channels, sampleRate, q, frequency);
76044 |
76045 |     return config;
76046 | }
76047 |
76048 | static void ma_notch_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
76049 | {

```

```

76050 |     ma_notch_node* pBPFNode = (ma_notch_node*)pNode;
76051 |
76052 |     MA_ASSERT(pNode != NULL);
76053 |     (void)pFrameCountIn;
76054 |
76055 |     ma_notch2_process_pcm_frames(&pBPFNode->notch, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
76056 | }
76057 |
76058 | static ma_node_vtable g_ma_notch_node_vtable =
76059 | {
76060 |     ma_notch_node_process_pcm_frames,
76061 |     NULL, /* onGetRequiredInputFrameCount */
76062 |     1, /* One input. */
76063 |     1, /* One output. */
76064 |     0 /* Default flags. */
76065 | };
76066 |
76067 | MA_API ma_result ma_notch_node_init(ma_node_graph* pNodeGraph, const ma_notch_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_notch_node* pNode)
76068 | {
76069 |     ma_result result;
76070 |     ma_node_config baseNodeConfig;
76071 |
76072 |     if (pNode == NULL) {
76073 |         return MA_INVALID_ARGS;
76074 |     }
76075 |
76076 |     MA_ZERO_OBJECT(pNode);
76077 |
76078 |     if (pConfig == NULL) {
76079 |         return MA_INVALID_ARGS;
76080 |     }
76081 |
76082 |     if (pConfig->notch.format != ma_format_f32) {
76083 |         return MA_INVALID_ARGS; /* The format must be f32. */
76084 |     }
76085 |
76086 |     result = ma_notch2_init(&pConfig->notch, pAllocationCallbacks, &pNode->notch);
76087 |     if (result != MA_SUCCESS) {
76088 |         return result;
76089 |     }
76090 |
76091 |     baseNodeConfig = ma_node_config_init();
76092 |     baseNodeConfig.vtable = &g_ma_notch_node_vtable;
76093 |     baseNodeConfig.pInputChannels = &pConfig->notch.channels;
76094 |     baseNodeConfig.pOutputChannels = &pConfig->notch.channels;
76095 |
76096 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
76097 |     if (result != MA_SUCCESS) {
76098 |         return result;
76099 |     }
76100 |
76101 |     return result;
76102 | }
76103 |
76104 | MA_API ma_result ma_notch_node_reinit(const ma_notch_config* pConfig, ma_notch_node* pNode)
76105 | {
76106 |     ma_notch_node* pNotchNode = (ma_notch_node*)pNode;
76107 |
76108 |     if (pNode == NULL) {
76109 |         return MA_INVALID_ARGS;
76110 |     }
76111 |
76112 |     return ma_notch2_reinit(pConfig, &pNotchNode->notch);
76113 | }
76114 |
76115 | MA_API void ma_notch_node_uninit(ma_notch_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
76116 | {
76117 |     ma_notch_node* pNotchNode = (ma_notch_node*)pNode;
76118 |
76119 |     if (pNode == NULL) {
76120 |         return;
76121 |     }
76122 |
76123 |     ma_node_uninit(pNode, pAllocationCallbacks);
76124 |     ma_notch2_uninit(&pNotchNode->notch, pAllocationCallbacks);
76125 | }
76126 |

```

```

76127 |
76128 |
76129 | /*
76130 | Peaking Filter Node
76131 | */
76132 | MA_API ma_peak_node_config ma_peak_node_config_init(ma_uint32 channels, ma_uint32 sampleRate, double
> gainDB, double q, double frequency)
76133 | {
76134 |     ma_peak_node_config config;
76135 |
76136 |     config.nodeConfig = ma_node_config_init();
76137 |     config.peak = ma_peak2_config_init(ma_format_f32, channels, sampleRate, gainDB, q, frequency);
76138 |
76139 |     return config;
76140 | }
76141 |
76142 | static void ma_peak_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
76143 | {
76144 |     ma_peak_node* pBPFNode = (ma_peak_node*)pNode;
76145 |
76146 |     MA_ASSERT(pNode != NULL);
76147 |     (void)pFrameCountIn;
76148 |
76149 |     ma_peak2_process_pcm_frames(&pBPFNode->peak, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
76150 | }
76151 |
76152 | static ma_node_vtable g_ma_peak_node_vtable =
76153 | {
76154 |     ma_peak_node_process_pcm_frames,
76155 |     NULL, /* onGetRequiredInputFrameCount */
76156 |     1, /* One input. */
76157 |     1, /* One output. */
76158 |     0 /* Default flags. */
76159 | };
76160 |
76161 | MA_API ma_result ma_peak_node_init(ma_node_graph* pNodeGraph, const ma_peak_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_peak_node* pNode)
76162 | {
76163 |     ma_result result;
76164 |     ma_node_config baseNodeConfig;
76165 |
76166 |     if (pNode == NULL) {
76167 |         return MA_INVALID_ARGS;
76168 |     }
76169 |
76170 |     MA_ZERO_OBJECT(pNode);
76171 |
76172 |     if (pConfig == NULL) {
76173 |         return MA_INVALID_ARGS;
76174 |     }
76175 |
76176 |     if (pConfig->peak.format != ma_format_f32) {
76177 |         return MA_INVALID_ARGS; /* The format must be f32. */
76178 |     }
76179 |
76180 |     result = ma_peak2_init(&pConfig->peak, pAllocationCallbacks, &pNode->peak);
76181 |     if (result != MA_SUCCESS) {
76182 |         ma_node_uninit(pNode, pAllocationCallbacks);
76183 |         return result;
76184 |     }
76185 |
76186 |     baseNodeConfig = ma_node_config_init();
76187 |     baseNodeConfig.vtable = &g_ma_peak_node_vtable;
76188 |     baseNodeConfig.pInputChannels = &pConfig->peak.channels;
76189 |     baseNodeConfig.pOutputChannels = &pConfig->peak.channels;
76190 |
76191 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
76192 |     if (result != MA_SUCCESS) {
76193 |         return result;
76194 |     }
76195 |
76196 |     return result;
76197 | }
76198 |
76199 | MA_API ma_result ma_peak_node_reinit(const ma_peak_config* pConfig, ma_peak_node* pNode)
76200 | {
76201 |     ma_peak_node* pPeakNode = (ma_peak_node*)pNode;
76202 |

```

```

76203 |     if (pNode == NULL) {
76204 |         return MA_INVALID_ARGS;
76205 |     }
76206 |
76207 |     return ma_peak2_reinit(pConfig, &pPeakNode->peak);
76208 | }
76209 |
76210 | MA_API void ma_peak_node_uninit(ma_peak_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
76211 | {
76212 |     ma_peak_node* pPeakNode = (ma_peak_node*)pNode;
76213 |
76214 |     if (pNode == NULL) {
76215 |         return;
76216 |     }
76217 |
76218 |     ma_node_uninit(pNode, pAllocationCallbacks);
76219 |     ma_peak2_uninit(&pPeakNode->peak, pAllocationCallbacks);
76220 | }
76221 |
76222 |
76223 |
76224 | /*
76225 | Low Shelf Filter Node
76226 | */
76227 | MA_API ma_loshelf_node_config ma_loshelf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
> double gainDB, double q, double frequency)
76228 | {
76229 |     ma_loshelf_node_config config;
76230 |
76231 |     config.nodeConfig = ma_node_config_init();
76232 |     config.loshelf = ma_loshelf2_config_init(ma_format_f32, channels, sampleRate, gainDB, q,
> frequency);
76233 |
76234 |     return config;
76235 | }
76236 |
76237 | static void ma_loshelf_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
76238 | {
76239 |     ma_loshelf_node* pBPFNode = (ma_loshelf_node*)pNode;
76240 |
76241 |     MA_ASSERT(pNode != NULL);
76242 |     (void)pFrameCountIn;
76243 |
76244 |     ma_loshelf2_process_pcm_frames(&pBPFNode->loshelf, ppFramesOut[0], ppFramesIn[0],
> *pFrameCountOut);
76245 | }
76246 |
76247 | static ma_node_vtable g_ma_loshelf_node_vtable =
76248 | {
76249 |     ma_loshelf_node_process_pcm_frames,
76250 |     NULL, /* onGetRequiredInputFrameCount */
76251 |     1, /* One input. */
76252 |     1, /* One output. */
76253 |     0 /* Default flags. */
76254 | };
76255 |
76256 | MA_API ma_result ma_loshelf_node_init(ma_node_graph* pNodeGraph, const ma_loshelf_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_loshelf_node* pNode)
76257 | {
76258 |     ma_result result;
76259 |     ma_node_config baseNodeConfig;
76260 |
76261 |     if (pNode == NULL) {
76262 |         return MA_INVALID_ARGS;
76263 |     }
76264 |
76265 |     MA_ZERO_OBJECT(pNode);
76266 |
76267 |     if (pConfig == NULL) {
76268 |         return MA_INVALID_ARGS;
76269 |     }
76270 |
76271 |     if (pConfig->loshelf.format != ma_format_f32) {
76272 |         return MA_INVALID_ARGS; /* The format must be f32. */
76273 |     }
76274 |
76275 |     result = ma_loshelf2_init(&pConfig->loshelf, pAllocationCallbacks, &pNode->loshelf);

```

```

76276 |     if (result != MA_SUCCESS) {
76277 |         return result;
76278 |     }
76279 |
76280 |     baseNodeConfig = ma_node_config_init();
76281 |     baseNodeConfig.vtable = &g_ma_loshelf_node_vtable;
76282 |     baseNodeConfig.pInputChannels = &pConfig->loshelf.channels;
76283 |     baseNodeConfig.pOutputChannels = &pConfig->loshelf.channels;
76284 |
76285 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
76286 |     if (result != MA_SUCCESS) {
76287 |         return result;
76288 |     }
76289 |
76290 |     return result;
76291 | }
76292 |
76293 | MA_API ma_result ma_loshelf_node_reinit(const ma_loshelf_config* pConfig, ma_loshelf_node* pNode)
76294 | {
76295 |     ma_loshelf_node* pLoshelfNode = (ma_loshelf_node*)pNode;
76296 |
76297 |     if (pNode == NULL) {
76298 |         return MA_INVALID_ARGS;
76299 |     }
76300 |
76301 |     return ma_loshelf2_reinit(pConfig, &pLoshelfNode->loshelf);
76302 | }
76303 |
76304 | MA_API void ma_loshelf_node_uninit(ma_loshelf_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
76305 | {
76306 |     ma_loshelf_node* pLoshelfNode = (ma_loshelf_node*)pNode;
76307 |
76308 |     if (pNode == NULL) {
76309 |         return;
76310 |     }
76311 |
76312 |     ma_node_uninit(pNode, pAllocationCallbacks);
76313 |     ma_loshelf2_uninit(&pLoshelfNode->loshelf, pAllocationCallbacks);
76314 | }
76315 |
76316 |
76317 |
76318 | /*
76319 | High Shelf Filter Node
76320 | */
76321 | MA_API ma_hishelf_node_config ma_hishelf_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
> double gainDB, double q, double frequency)
76322 | {
76323 |     ma_hishelf_node_config config;
76324 |
76325 |     config.nodeConfig = ma_node_config_init();
76326 |     config.hishelf = ma_hishelf2_config_init(ma_format_f32, channels, sampleRate, gainDB, q,
> frequency);
76327 |
76328 |     return config;
76329 | }
76330 |
76331 | static void ma_hishelf_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
76332 | {
76333 |     ma_hishelf_node* pBPFNode = (ma_hishelf_node*)pNode;
76334 |
76335 |     MA_ASSERT(pNode != NULL);
76336 |     (void)pFrameCountIn;
76337 |
76338 |     ma_hishelf2_process_pcm_frames(&pBPFNode->hishelf, ppFramesOut[0], ppFramesIn[0],
> *pFrameCountOut);
76339 | }
76340 |
76341 | static ma_node_vtable g_ma_hishelf_node_vtable =
76342 | {
76343 |     ma_hishelf_node_process_pcm_frames,
76344 |     NULL, /* onGetRequiredInputFrameCount */
76345 |     1, /* One input. */
76346 |     1, /* One output. */
76347 |     0 /* Default flags. */
76348 | };
76349 |

```

```

76350 | MA_API ma_result ma_hishef_node_init(ma_node_graph* pNodeGraph, const ma_hishef_node_config*
> pConfig, const ma_allocation_callbacks* pAllocationCallbacks, ma_hishef_node* pNode)
76351 | {
76352 |     ma_result result;
76353 |     ma_node_config baseNodeConfig;
76354 |
76355 |     if (pNode == NULL) {
76356 |         return MA_INVALID_ARGS;
76357 |     }
76358 |
76359 |     MA_ZERO_OBJECT(pNode);
76360 |
76361 |     if (pConfig == NULL) {
76362 |         return MA_INVALID_ARGS;
76363 |     }
76364 |
76365 |     if (pConfig->hishef.format != ma_format_f32) {
76366 |         return MA_INVALID_ARGS; /* The format must be f32. */
76367 |     }
76368 |
76369 |     result = ma_hishef2_init(&pConfig->hishef, pAllocationCallbacks, &pNode->hishef);
76370 |     if (result != MA_SUCCESS) {
76371 |         return result;
76372 |     }
76373 |
76374 |     baseNodeConfig = ma_node_config_init();
76375 |     baseNodeConfig.vtable = &g_ma_hishef_node_vtable;
76376 |     baseNodeConfig.pInputChannels = &pConfig->hishef.channels;
76377 |     baseNodeConfig.pOutputChannels = &pConfig->hishef.channels;
76378 |
76379 |     result = ma_node_init(pNodeGraph, &baseNodeConfig, pAllocationCallbacks, pNode);
76380 |     if (result != MA_SUCCESS) {
76381 |         return result;
76382 |     }
76383 |
76384 |     return result;
76385 | }
76386 |
76387 | MA_API ma_result ma_hishef_node_reinit(const ma_hishef_config* pConfig, ma_hishef_node* pNode)
76388 | {
76389 |     ma_hishef_node* pHishefNode = (ma_hishef_node*)pNode;
76390 |
76391 |     if (pNode == NULL) {
76392 |         return MA_INVALID_ARGS;
76393 |     }
76394 |
76395 |     return ma_hishef2_reinit(pConfig, &pHishefNode->hishef);
76396 | }
76397 |
76398 | MA_API void ma_hishef_node_uninit(ma_hishef_node* pNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
76399 | {
76400 |     ma_hishef_node* pHishefNode = (ma_hishef_node*)pNode;
76401 |
76402 |     if (pNode == NULL) {
76403 |         return;
76404 |     }
76405 |
76406 |     ma_node_uninit(pNode, pAllocationCallbacks);
76407 |     ma_hishef2_uninit(&pHishefNode->hishef, pAllocationCallbacks);
76408 | }
76409 |
76410 |
76411 |
76412 |
76413 | MA_API ma_delay_node_config ma_delay_node_config_init(ma_uint32 channels, ma_uint32 sampleRate,
> ma_uint32 delayInFrames, float decay)
76414 | {
76415 |     ma_delay_node_config config;
76416 |
76417 |     config.nodeConfig = ma_node_config_init();
76418 |     config.delay = ma_delay_config_init(channels, sampleRate, delayInFrames, decay);
76419 |
76420 |     return config;
76421 | }
76422 |
76423 |
76424 | static void ma_delay_node_process_pcm_frames(ma_node* pNode, const float** ppFramesIn, ma_uint32*
> pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)

```



```

76425 | {
76426 |     ma_delay_node* pDelayNode = (ma_delay_node*)pNode;
76427 |
76428 |     (void)pFrameCountIn;
76429 |
76430 |     ma_delay_process_pcm_frames(&pDelayNode->delay, ppFramesOut[0], ppFramesIn[0], *pFrameCountOut);
76431 | }
76432 |
76433 | static ma_node_vtable g_ma_delay_node_vtable =
76434 | {
76435 |     ma_delay_node_process_pcm_frames,
76436 |     NULL,
76437 |     1, /* 1 input channels. */
76438 |     1, /* 1 output channel. */
76439 |     MA_NODE_FLAG_CONTINUOUS_PROCESSING /* Delay requires continuous processing to ensure the tail
> get's processed. */
76440 | };
76441 |
76442 | MA_API ma_result ma_delay_node_init(ma_node_graph* pNodeGraph, const ma_delay_node_config* pConfig,
> const ma_allocation_callbacks* pAllocationCallbacks, ma_delay_node* pDelayNode)
76443 | {
76444 |     ma_result result;
76445 |     ma_node_config baseConfig;
76446 |
76447 |     if (pDelayNode == NULL) {
76448 |         return MA_INVALID_ARGS;
76449 |     }
76450 |
76451 |     MA_ZERO_OBJECT(pDelayNode);
76452 |
76453 |     result = ma_delay_init(&pConfig->delay, pAllocationCallbacks, &pDelayNode->delay);
76454 |     if (result != MA_SUCCESS) {
76455 |         return result;
76456 |     }
76457 |
76458 |     baseConfig = pConfig->nodeConfig;
76459 |     baseConfig.vtable = &g_ma_delay_node_vtable;
76460 |     baseConfig.pInputChannels = &pConfig->delay.channels;
76461 |     baseConfig.pOutputChannels = &pConfig->delay.channels;
76462 |
76463 |     result = ma_node_init(pNodeGraph, &baseConfig, pAllocationCallbacks, &pDelayNode->baseNode);
76464 |     if (result != MA_SUCCESS) {
76465 |         ma_delay_uninit(&pDelayNode->delay, pAllocationCallbacks);
76466 |         return result;
76467 |     }
76468 |
76469 |     return result;
76470 | }
76471 |
76472 | MA_API void ma_delay_node_uninit(ma_delay_node* pDelayNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
76473 | {
76474 |     if (pDelayNode == NULL) {
76475 |         return;
76476 |     }
76477 |
76478 |     /* The base node is always uninitialized first. */
76479 |     ma_node_uninit(pDelayNode, pAllocationCallbacks);
76480 |     ma_delay_uninit(&pDelayNode->delay, pAllocationCallbacks);
76481 | }
76482 |
76483 | MA_API void ma_delay_node_set_wet(ma_delay_node* pDelayNode, float value)
76484 | {
76485 |     if (pDelayNode == NULL) {
76486 |         return;
76487 |     }
76488 |
76489 |     ma_delay_set_wet(&pDelayNode->delay, value);
76490 | }
76491 |
76492 | MA_API float ma_delay_node_get_wet(const ma_delay_node* pDelayNode)
76493 | {
76494 |     if (pDelayNode == NULL) {
76495 |         return 0;
76496 |     }
76497 |
76498 |     return ma_delay_get_wet(&pDelayNode->delay);
76499 | }
76500 |

```

```

76501 | MA_API void ma_delay_node_set_dry(ma_delay_node* pDelayNode, float value)
76502 | {
76503 |     if (pDelayNode == NULL) {
76504 |         return;
76505 |     }
76506 |
76507 |     ma_delay_set_dry(&pDelayNode->delay, value);
76508 | }
76509 |
76510 | MA_API float ma_delay_node_get_dry(const ma_delay_node* pDelayNode)
76511 | {
76512 |     if (pDelayNode == NULL) {
76513 |         return 0;
76514 |     }
76515 |
76516 |     return ma_delay_get_dry(&pDelayNode->delay);
76517 | }
76518 |
76519 | MA_API void ma_delay_node_set_decay(ma_delay_node* pDelayNode, float value)
76520 | {
76521 |     if (pDelayNode == NULL) {
76522 |         return;
76523 |     }
76524 |
76525 |     ma_delay_set_decay(&pDelayNode->delay, value);
76526 | }
76527 |
76528 | MA_API float ma_delay_node_get_decay(const ma_delay_node* pDelayNode)
76529 | {
76530 |     if (pDelayNode == NULL) {
76531 |         return 0;
76532 |     }
76533 |
76534 |     return ma_delay_get_decay(&pDelayNode->delay);
76535 | }
76536 | #endif /* MA_NO_NODE_GRAPH */
76537 |
76538 |
76539 | /* SECTION: miniaudio_engine.c */
76540 | #if !defined(MA_NO_ENGINE) && !defined(MA_NO_NODE_GRAPH)
76541 | /*****
76542 | > *****/
76543 | Engine
76544 |
76545 | *****/
76546 | #define MA_SEEK_TARGET_NONE        (~(ma_uint64)0)
76547 |
76548 |
76549 | static void ma_sound_set_at_end(ma_sound* pSound, ma_bool32 atEnd)
76550 | {
76551 |     MA_ASSERT(pSound != NULL);
76552 |     ma_atomic_exchange_32(&pSound->atEnd, atEnd);
76553 |
76554 |     /*
76555 | > When this function is called the state of the sound will not yet be in a stopped state. This
76556 | > makes it confusing
76557 | > because an end callback will intuitively expect ma_sound_is_playing() to return false from
76558 | > inside the callback.
76559 | > I'm therefore no longer firing the callback here and will instead fire it manually in the *next*
76560 | > processing step
76561 | > when the state should be set to stopped as expected.
76562 | */
76563 | #if 0
76564 | /* Fire any callbacks or events. */
76565 | if (atEnd) {
76566 |     if (pSound->endCallback != NULL) {
76567 |         pSound->endCallback(pSound->pEndCallbackUserData, pSound);
76568 |     }
76569 | }
76570 | #endif
76571 |
76572 | static ma_bool32 ma_sound_get_at_end(const ma_sound* pSound)
76573 | {
76574 |     MA_ASSERT(pSound != NULL);
76575 |     return ma_atomic_load_32(&pSound->atEnd);
76576 | }

```

```

76575 |
76576 |
76577 | MA_API ma_engine_node_config ma_engine_node_config_init(ma_engine* pEngine, ma_engine_node_type
> type, ma_uint32 flags)
76578 | {
76579 |     ma_engine_node_config config;
76580 |
76581 |     MA_ZERO_OBJECT(&config);
76582 |     config.pEngine          = pEngine;
76583 |     config.type             = type;
76584 |     config.isPitchDisabled  = (flags & MA_SOUND_FLAG_NO_PITCH) != 0;
76585 |     config.isSpatializationDisabled = (flags & MA_SOUND_FLAG_NO_SPATIALIZATION) != 0;
76586 |     config.monoExpansionMode = pEngine->monoExpansionMode;
76587 |     config.resampling        = pEngine->pitchResamplingConfig;
76588 |
76589 |     return config;
76590 | }
76591 |
76592 |
76593 | static void ma_engine_node_update_pitch_if_required(ma_engine_node* pEngineNode)
76594 | {
76595 |     ma_bool32 isUpdateRequired = MA_FALSE;
76596 |     float newPitch;
76597 |
76598 |     MA_ASSERT(pEngineNode != NULL);
76599 |
76600 |     newPitch = ma_atomic_load_explicit_f32(&pEngineNode->pitch, ma_atomic_memory_order_acquire);
76601 |
76602 |     if (pEngineNode->oldPitch != newPitch) {
76603 |         pEngineNode->oldPitch = newPitch;
76604 |         isUpdateRequired = MA_TRUE;
76605 |     }
76606 |
76607 |     if (pEngineNode->oldDopplerPitch != pEngineNode->spatializer.dopplerPitch) {
76608 |         pEngineNode->oldDopplerPitch = pEngineNode->spatializer.dopplerPitch;
76609 |         isUpdateRequired = MA_TRUE;
76610 |     }
76611 |
76612 |     if (isUpdateRequired) {
76613 |         float basePitch = (float)pEngineNode->sampleRate /
> ma_engine_get_sample_rate(pEngineNode->pEngine);
76614 |         ma_resampler_set_rate_ratio(&pEngineNode->resampler, basePitch * pEngineNode->oldPitch *
> pEngineNode->oldDopplerPitch);
76615 |     }
76616 | }
76617 |
76618 | static ma_bool32 ma_engine_node_is_pitching_enabled(const ma_engine_node* pEngineNode)
76619 | {
76620 |     MA_ASSERT(pEngineNode != NULL);
76621 |
76622 |     /* Don't try to be clever by skipping resampling in the pitch=1 case or else you'll glitch when
> moving away from 1. */
76623 |     return !ma_atomic_load_explicit_32(&pEngineNode->isPitchDisabled,
> ma_atomic_memory_order_acquire);
76624 | }
76625 |
76626 | static ma_bool32 ma_engine_node_is_spatialization_enabled(const ma_engine_node* pEngineNode)
76627 | {
76628 |     MA_ASSERT(pEngineNode != NULL);
76629 |
76630 |     return !ma_atomic_load_explicit_32(&pEngineNode->isSpatializationDisabled,
> ma_atomic_memory_order_acquire);
76631 | }
76632 |
76633 | static ma_result ma_engine_node_set_volume(ma_engine_node* pEngineNode, float volume)
76634 | {
76635 |     if (pEngineNode == NULL) {
76636 |         return MA_INVALID_ARGS;
76637 |     }
76638 |
76639 |     ma_atomic_float_set(&pEngineNode->volume, volume);
76640 |
76641 |     /* If we're not smoothing we should bypass the volume gainer entirely. */
76642 |     if (pEngineNode->volumeSmoothTimeInPCMFrames == 0) {
76643 |         /* We should always have an active spatializer because it can be enabled and disabled
> dynamically. We can just use that for holding our volume. */
76644 |         ma_spatializer_set_master_volume(&pEngineNode->spatializer, volume);
76645 |     } else {
76646 |         /* We're using volume smoothing, so apply the master volume to the gainer. */

```

```

76647 |         ma_gainer_set_gain(&pEngineNode->volumeGainer, volume);
76648 |     }
76649 |
76650 |     return MA_SUCCESS;
76651 | }
76652 |
76653 | static ma_result ma_engine_node_get_volume(const ma_engine_node* pEngineNode, float* pVolume)
76654 | {
76655 |     if (pVolume == NULL) {
76656 |         return MA_INVALID_ARGS;
76657 |     }
76658 |
76659 |     *pVolume = 0.0f;
76660 |
76661 |     if (pEngineNode == NULL) {
76662 |         return MA_INVALID_ARGS;
76663 |     }
76664 |
76665 |     *pVolume = ma_atomic_float_get((ma_atomic_float*)&pEngineNode->volume);
76666 |
76667 |     return MA_SUCCESS;
76668 | }
76669 |
76670 |
76671 | static void ma_engine_node_process_pcm_frames_general(ma_engine_node* pEngineNode, const float**
> ppFramesIn, ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
76672 | {
76673 |     ma_uint32 frameCountIn;
76674 |     ma_uint32 frameCountOut;
76675 |     ma_uint32 totalFramesProcessedIn;
76676 |     ma_uint32 totalFramesProcessedOut;
76677 |     ma_uint32 channelsIn;
76678 |     ma_uint32 channelsOut;
76679 |     ma_bool32 isPitchingEnabled;
76680 |     ma_bool32 isFadingEnabled;
76681 |     ma_bool32 isSpatializationEnabled;
76682 |     ma_bool32 isPanningEnabled;
76683 |     ma_bool32 isVolumeSmoothingEnabled;
76684 |
76685 |     frameCountIn = *pFrameCountIn;
76686 |     frameCountOut = *pFrameCountOut;
76687 |
76688 |     channelsIn = ma_spatializer_get_input_channels(&pEngineNode->spatializer);
76689 |     channelsOut = ma_spatializer_get_output_channels(&pEngineNode->spatializer);
76690 |
76691 |     totalFramesProcessedIn = 0;
76692 |     totalFramesProcessedOut = 0;
76693 |
76694 |     /* Update the fader if applicable. */
76695 |     {
76696 |         ma_uint64 fadeLengthInFrames =
> ma_atomic_uint64_get(&pEngineNode->fadeSettings.fadeLengthInFrames);
76697 |         if (fadeLengthInFrames != ~(ma_uint64)0) {
76698 |             float fadeVolumeBeg = ma_atomic_float_get(&pEngineNode->fadeSettings.volumeBeg);
76699 |             float fadeVolumeEnd = ma_atomic_float_get(&pEngineNode->fadeSettings.volumeEnd);
76700 |             ma_int64 fadeStartOffsetInFrames =
> (ma_int64)ma_atomic_uint64_get(&pEngineNode->fadeSettings.absoluteGlobalTimeInFrames);
76701 |             if (fadeStartOffsetInFrames == (ma_int64)(~(ma_uint64)0)) {
76702 |                 fadeStartOffsetInFrames = 0;
76703 |             } else {
76704 |                 fadeStartOffsetInFrames -= ma_engine_get_time_in_pcm_frames(pEngineNode->pEngine);
76705 |             }
76706 |
76707 |             ma_fader_set_fade_ex(&pEngineNode->fader, fadeVolumeBeg, fadeVolumeEnd,
> fadeLengthInFrames, fadeStartOffsetInFrames);
76708 |
76709 |             /* Reset the fade length so we don't erroneously apply it again. */
76710 |             ma_atomic_uint64_set(&pEngineNode->fadeSettings.fadeLengthInFrames, ~(ma_uint64)0);
76711 |         }
76712 |     }
76713 |
76714 |     isPitchingEnabled = ma_engine_node_is_pitching_enabled(pEngineNode);
76715 |     isFadingEnabled = pEngineNode->fader.volumeBeg != 1 || pEngineNode->fader.volumeEnd !=
> 1;
76716 |     isSpatializationEnabled = ma_engine_node_is_spatialization_enabled(pEngineNode);
76717 |     isPanningEnabled = pEngineNode->panner.pan != 0 && channelsOut != 1;
76718 |     isVolumeSmoothingEnabled = pEngineNode->volumeSmoothTimeInPCMFrames > 0;
76719 |
76720 |     /* Keep going while we've still got data available for processing. */

```

```

76721 |     while (totalFramesProcessedOut < frameCountOut) {
76722 |         /*
76723 |         We need to process in a specific order. We always do resampling first because it's likely
76724 |         we're going to be increasing the channel count after spatialization. Also, I want to do
76725 |         fading based on the output sample rate.
76726 |
76727 |         We'll first read into a buffer from the resampler. Then we'll do all processing that
76728 |         operates on the on the input channel count. We'll then get the spatializer to output to
76729 |         the output buffer and then do all effects from that point directly in the output buffer
76730 |         in-place.
76731 |
76732 |         Note that we're always running the resampler if pitching is enabled, even when the pitch
76733 |         is 1. If we try to be clever and skip resampling when the pitch is 1, we'll get a glitch
76734 |         when we move away from 1, back to 1, and then away from 1 again. We'll want to implement
76735 |         any pitch=1 optimizations in the resampler itself.
76736 |
76737 |         There's a small optimization here that we'll utilize since it might be a fairly common
76738 |         case. When the input and output channel counts are the same, we'll read straight into the
76739 |         output buffer from the resampler and do everything in-place.
76740 |         */
76741 |         const float* pRunningFramesIn;
76742 |         float* pRunningFramesOut;
76743 |         float* pWorkingBuffer; /* This is the buffer that we'll be processing frames in. This is
> in input channels. */
76744 |         float temp[MA_DATA_CONVERTER_STACK_BUFFER_SIZE / sizeof(float)];
76745 |         ma_uint32 tempCapInFrames = ma_countof(temp) / channelsIn;
76746 |         ma_uint32 framesAvailableIn;
76747 |         ma_uint32 framesAvailableOut;
76748 |         ma_uint32 framesJustProcessedIn;
76749 |         ma_uint32 framesJustProcessedOut;
76750 |         ma_bool32 isWorkingBufferValid = MA_FALSE;
76751 |
76752 |         framesAvailableIn = frameCountIn - totalFramesProcessedIn;
76753 |         framesAvailableOut = frameCountOut - totalFramesProcessedOut;
76754 |
76755 |         pRunningFramesIn = ma_offset_pcm_frames_const_ptr_f32(ppFramesIn[0],
> totalFramesProcessedIn, channelsIn);
76756 |         pRunningFramesOut = ma_offset_pcm_frames_ptr_f32(ppFramesOut[0], totalFramesProcessedOut,
> channelsOut);
76757 |
76758 |         if (channelsIn == channelsOut) {
76759 |             /* Fast path. Channel counts are the same. No need for an intermediary input buffer. */
76760 |             pWorkingBuffer = pRunningFramesOut;
76761 |         } else {
76762 |             /* Slow path. Channel counts are different. Need to use an intermediary input buffer. */
76763 |             pWorkingBuffer = temp;
76764 |             if (framesAvailableOut > tempCapInFrames) {
76765 |                 framesAvailableOut = tempCapInFrames;
76766 |             }
76767 |         }
76768 |
76769 |         /* First is resampler. */
76770 |         if (isPitchingEnabled) {
76771 |             ma_uint64 resampleFrameCountIn = framesAvailableIn;
76772 |             ma_uint64 resampleFrameCountOut = framesAvailableOut;
76773 |
76774 |             ma_resampler_process_pcm_frames(&EngineNode->resampler, pRunningFramesIn,
> &resampleFrameCountIn, pWorkingBuffer, &resampleFrameCountOut);
76775 |             isWorkingBufferValid = MA_TRUE;
76776 |
76777 |             framesJustProcessedIn = (ma_uint32)resampleFrameCountIn;
76778 |             framesJustProcessedOut = (ma_uint32)resampleFrameCountOut;
76779 |         } else {
76780 |             framesJustProcessedIn = ma_min(framesAvailableIn, framesAvailableOut);
76781 |             framesJustProcessedOut = framesJustProcessedIn; /* When no resampling is being
> performed, the number of output frames is the same as input frames. */
76782 |         }
76783 |
76784 |         /* Fading. */
76785 |         if (isFadingEnabled) {
76786 |             if (isWorkingBufferValid) {
76787 |                 ma_fader_process_pcm_frames(&EngineNode->fader, pWorkingBuffer, pWorkingBuffer,
> framesJustProcessedOut); /* In-place processing. */
76788 |             } else {
76789 |                 ma_fader_process_pcm_frames(&EngineNode->fader, pWorkingBuffer, pRunningFramesIn,
> framesJustProcessedOut);
76790 |                 isWorkingBufferValid = MA_TRUE;
76791 |             }
76792 |         }

```

```

76793 |
76794 |     /*
76795 |     If we're using smoothing, we won't be applying volume via the spatializer, but instead from
> a ma_gainer. In this case
76796 |     we'll want to apply our volume now.
76797 |     */
76798 |     if (isVolumeSmoothingEnabled) {
76799 |         if (isWorkingBufferValid) {
76800 |             ma_gainer_process_pcm_frames(&pEngineNode->volumeGainer, pWorkingBuffer,
> pWorkingBuffer, framesJustProcessedOut);
76801 |         } else {
76802 |             ma_gainer_process_pcm_frames(&pEngineNode->volumeGainer, pWorkingBuffer,
> pRunningFramesIn, framesJustProcessedOut);
76803 |             isWorkingBufferValid = MA_TRUE;
76804 |         }
76805 |     }
76806 |
76807 |     /*
76808 |     If at this point we still haven't actually done anything with the working buffer we need
76809 |     to just read straight from the input buffer.
76810 |     */
76811 |     if (isWorkingBufferValid == MA_FALSE) {
76812 |         pWorkingBuffer = (float*)pRunningFramesIn; /* Naughty const cast, but it's safe at this
> point because we won't ever be writing to it from this point out. */
76813 |     }
76814 |
76815 |     /* Spatialization. */
76816 |     if (isSpatializationEnabled) {
76817 |         ma_uint32 iListener;
76818 |
76819 |         /*
76820 |         When determining the listener to use, we first check to see if the sound is pinned to a
76821 |         specific listener. If so, we use that. Otherwise we just use the closest listener.
76822 |         */
76823 |         if (pEngineNode->pinnedListenerIndex != MA_LISTENER_INDEX_CLOSEST &&
> pEngineNode->pinnedListenerIndex < ma_engine_get_listener_count(pEngineNode->pEngine)) {
76824 |             iListener = pEngineNode->pinnedListenerIndex;
76825 |         } else {
76826 |             ma_vec3f spatializerPosition =
> ma_spatializer_get_position(&pEngineNode->spatializer);
76827 |             iListener = ma_engine_find_closest_listener(pEngineNode->pEngine,
> spatializerPosition.x, spatializerPosition.y, spatializerPosition.z);
76828 |         }
76829 |
76830 |         ma_spatializer_process_pcm_frames(&pEngineNode->spatializer,
> &pEngineNode->pEngine->listeners[iListener], pRunningFramesOut, pWorkingBuffer,
> framesJustProcessedOut);
76831 |     } else {
76832 |         /* No spatialization, but we still need to do channel conversion and master volume. */
76833 |         float volume;
76834 |         ma_engine_node_get_volume(pEngineNode, &volume); /* Should never fail. */
76835 |
76836 |         if (channelsIn == channelsOut) {
76837 |             /* No channel conversion required. Just copy straight to the output buffer. */
76838 |             if (isVolumeSmoothingEnabled) {
76839 |                 /* Volume has already been applied. Just copy straight to the output buffer. */
76840 |                 ma_copy_pcm_frames(pRunningFramesOut, pWorkingBuffer, framesJustProcessedOut *
> channelsOut, ma_format_f32, channelsOut);
76841 |             } else {
76842 |                 /* Volume has not been applied yet. Copy and apply volume in the same pass. */
76843 |                 ma_copy_and_apply_volume_factor_f32(pRunningFramesOut, pWorkingBuffer,
> framesJustProcessedOut * channelsOut, volume);
76844 |             }
76845 |         } else {
76846 |             /* Channel conversion required. TODO: Add support for channel maps here. */
76847 |             ma_channel_map_apply_f32(pRunningFramesOut, NULL, channelsOut, pWorkingBuffer, NULL,
> channelsIn, framesJustProcessedOut, ma_channel_mix_mode_simple, pEngineNode->monoExpansionMode);
76848 |
76849 |             /* If we're using smoothing, the volume will have already been applied. */
76850 |             if (!isVolumeSmoothingEnabled) {
76851 |                 ma_apply_volume_factor_f32(pRunningFramesOut, framesJustProcessedOut *
> channelsOut, volume);
76852 |             }
76853 |         }
76854 |     }
76855 |
76856 |     /* At this point we can guarantee that the output buffer contains valid data. We can process
> everything in place now. */
76857 |

```

```

76858 |         /* Panning. */
76859 |         if (isPanningEnabled) {
76860 |             ma_panner_process_pcm_frames(&pEngineNode->panner, pRunningFramesOut, pRunningFramesOut,
> framesJustProcessedOut); /* In-place processing. */
76861 |         }
76862 |
76863 |         /* We're done for this chunk. */
76864 |         totalFramesProcessedIn += framesJustProcessedIn;
76865 |         totalFramesProcessedOut += framesJustProcessedOut;
76866 |
76867 |         /* If we didn't process any output frames this iteration it means we've either run out of
> input data, or run out of room in the output buffer. */
76868 |         if (framesJustProcessedOut == 0) {
76869 |             break;
76870 |         }
76871 |     }
76872 |
76873 |     /* At this point we're done processing. */
76874 |     *pFrameCountIn = totalFramesProcessedIn;
76875 |     *pFrameCountOut = totalFramesProcessedOut;
76876 | }
76877 |
76878 | static void ma_engine_node_process_pcm_frames__sound(ma_node* pNode, const float** ppFramesIn,
> ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
76879 | {
76880 |     /* For sounds, we need to first read from the data source. Then we need to apply the engine
> effects (pan, pitch, fades, etc.). */
76881 |     ma_result result = MA_SUCCESS;
76882 |     ma_sound* pSound = (ma_sound*)pNode;
76883 |     ma_uint32 frameCount = *pFrameCountOut;
76884 |     ma_uint32 totalFramesRead = 0;
76885 |     ma_format dataSourceFormat;
76886 |     ma_uint32 dataSourceChannels;
76887 |     ma_uint8 temp[MA_DATA_CONVERTER_STACK_BUFFER_SIZE];
76888 |     ma_uint32 tempCapInFrames;
76889 |     ma_uint64 seekTarget;
76890 |
76891 |     /* This is a data source node which means no input buses. */
76892 |     (void)ppFramesIn;
76893 |     (void)pFrameCountIn;
76894 |
76895 |     /* If we're marked at the end we need to stop the sound and do nothing. */
76896 |     if (ma_sound_at_end(pSound)) {
76897 |         ma_sound_stop(pSound);
76898 |
76899 |         if (pSound->endCallback != NULL) {
76900 |             pSound->endCallback(pSound->pEndCallbackUserData, pSound);
76901 |         }
76902 |
76903 |         *pFrameCountOut = 0;
76904 |         return;
76905 |     }
76906 |
76907 |     /* If we're seeking, do so now before reading. */
76908 |     seekTarget = ma_atomic_load_64(&pSound->seekTarget);
76909 |     if (seekTarget != MA_SEEK_TARGET_NONE) {
76910 |         ma_data_source_seek_to_pcm_frame(pSound->pDataSource, seekTarget);
76911 |
76912 |         /* Any time-dependant effects need to have their times updated. */
76913 |         ma_node_set_time(pSound, seekTarget);
76914 |
76915 |         ma_atomic_exchange_64(&pSound->seekTarget, MA_SEEK_TARGET_NONE);
76916 |     }
76917 |
76918 |     /*
76919 |     We want to update the pitch once. For sounds, this can be either at the start or at the end. If
76920 |     we don't force this to only ever be updating once, we could end up in a situation where
76921 |     retrieving the required input frame count ends up being different to what we actually retrieve.
76922 |     What could happen is that the required input frame count is calculated, the pitch is update,
76923 |     and then this processing function is called resulting in a different number of input frames
76924 |     being processed. Do not call this in ma_engine_node_process_pcm_frames__general() or else
76925 |     you'll hit the aforementioned bug.
76926 |     */
76927 |     ma_engine_node_update_pitch_if_required(&pSound->engineNode);
76928 |
76929 |     /*
76930 |     For the convenience of the caller, we're doing to allow data sources to use non-floating-point
> formats and channel counts that differ
76931 |     from the main engine.

```



```

76932 |     */
76933 |     result = ma_data_source_get_data_format(pSound->pDataSource, &dataSourceFormat,
> &dataSourceChannels, NULL, NULL, 0);
76934 |     if (result == MA_SUCCESS) {
76935 |         tempCapInFrames = sizeof(temp) / ma_get_bytes_per_frame(dataSourceFormat,
> dataSourceChannels);
76936 |
76937 |         /* Keep reading until we've read as much as was requested or we reach the end of the data
> source. */
76938 |         while (totalFramesRead < frameCount) {
76939 |             ma_uint32 framesRemaining = frameCount - totalFramesRead;
76940 |             ma_uint64 framesJustRead;
76941 |             ma_uint32 frameCountIn;
76942 |             ma_uint32 frameCountOut;
76943 |             const float* pRunningFramesIn;
76944 |             float* pRunningFramesOut;
76945 |
76946 |             /* If there's any input frames sitting in the cache get those processed first. */
76947 |             if (pSound->processingCacheFramesRemaining > 0) {
76948 |                 pRunningFramesIn = pSound->pProcessingCache;
76949 |                 frameCountIn = pSound->processingCacheFramesRemaining;
76950 |
76951 |                 pRunningFramesOut = ma_offset_pcm_frames_ptr_f32(ppFramesOut[0], totalFramesRead,
> ma_node_get_output_channels(pNode, 0));
76952 |                 frameCountOut = framesRemaining;
76953 |
76954 |                 ma_engine_node_process_pcm_frames_general(&pSound->engineNode, &pRunningFramesIn,
> &frameCountIn, &pRunningFramesOut, &frameCountOut);
76955 |
76956 |                 MA_ASSERT(frameCountIn <= pSound->processingCacheFramesRemaining);
76957 |                 pSound->processingCacheFramesRemaining -= frameCountIn;
76958 |
76959 |                 /* Move any remaining data in the cache down. */
76960 |                 if (pSound->processingCacheFramesRemaining > 0) {
76961 |                     MA_MOVE_MEMORY(pSound->pProcessingCache,
> ma_offset_pcm_frames_ptr_f32(pSound->pProcessingCache, frameCountIn, dataSourceChannels),
> pSound->processingCacheFramesRemaining * ma_get_bytes_per_frame(ma_format_f32, dataSourceChannels));
76962 |                 }
76963 |
76964 |                 totalFramesRead += (ma_uint32)frameCountOut; /* Safe cast. */
76965 |
76966 |                 if (result != MA_SUCCESS || ma_sound_at_end(pSound)) {
76967 |                     break; /* Might have reached the end. */
76968 |                 }
76969 |             } else {
76970 |                 /* Getting here means there's nothing in the cache. Read more data from the data
> source. */
76971 |                 if (dataSourceFormat == ma_format_f32) {
76972 |                     /* Fast path. No conversion to f32 necessary. */
76973 |                     result = ma_data_source_read_pcm_frames(pSound->pDataSource,
> pSound->pProcessingCache, pSound->processingCacheCap, &framesJustRead);
76974 |                 } else {
76975 |                     /* Slow path. Need to convert to f32. */
76976 |                     ma_uint64 totalFramesConverted = 0;
76977 |
76978 |                     while (totalFramesConverted < pSound->processingCacheCap) {
76979 |                         ma_uint64 framesConverted;
76980 |                         ma_uint32 framesToConvertThisIteration = pSound->processingCacheCap -
> (ma_uint32)totalFramesConverted;
76981 |                         if (framesToConvertThisIteration > tempCapInFrames) {
76982 |                             framesToConvertThisIteration = tempCapInFrames;
76983 |                         }
76984 |
76985 |                         result = ma_data_source_read_pcm_frames(pSound->pDataSource, temp,
> framesToConvertThisIteration, &framesConverted);
76986 |                         if (result != MA_SUCCESS) {
76987 |                             break;
76988 |                         }
76989 |
76990 |                         > ma_convert_pcm_frames_format(ma_offset_pcm_frames_ptr_f32(pSound->pProcessingCache,
> totalFramesConverted, dataSourceChannels), ma_format_f32, temp, dataSourceFormat, framesConverted,
> dataSourceChannels, ma_dither_mode_none);
76991 |                         totalFramesConverted += framesConverted;
76992 |                     }
76993 |
76994 |                     framesJustRead = totalFramesConverted;
76995 |                 }
76996 |             }

```

```

76997 |             MA_ASSERT(framesJustRead <= pSound->processingCacheCap);
76998 |             pSound->processingCacheFramesRemaining = (ma_uint32)framesJustRead;
76999 |
77000 |             /* If we reached the end of the sound we'll want to mark it as at the end and stop
> it. This should never be returned for looping sounds. */
77001 |             if (result == MA_AT_END) {
77002 |                 ma_sound_set_at_end(pSound, MA_TRUE); /* This will be set to false in
> ma_sound_start(). */
77003 |             }
77004 |
77005 |             if (result != MA_SUCCESS || ma_sound_at_end(pSound)) {
77006 |                 break;
77007 |             }
77008 |         }
77009 |     }
77010 | }
77011 |
77012 | *pFrameCountOut = totalFramesRead;
77013 | }
77014 |
77015 | static void ma_engine_node_process_pcm_frames_group(ma_node* pNode, const float** ppFramesIn,
> ma_uint32* pFrameCountIn, float** ppFramesOut, ma_uint32* pFrameCountOut)
77016 | {
77017 |     /*
77018 |     Make sure the pitch is updated before trying to read anything. It's important that this is done
77019 |     only once and not in ma_engine_node_process_pcm_frames_general(). The reason for this is that
77020 |     ma_engine_node_process_pcm_frames_general() will call
> ma_engine_node_get_required_input_frame_count(),
77021 |     and if another thread modifies the pitch just after that call it can result in a glitch due to
77022 |     the input rate changing.
77023 |     */
77024 |     ma_engine_node_update_pitch_if_required((ma_engine_node*)pNode);
77025 |
77026 |     /* For groups, the input data has already been read and we just need to apply the effect. */
77027 |     ma_engine_node_process_pcm_frames_general((ma_engine_node*)pNode, ppFramesIn, pFrameCountIn,
> ppFramesOut, pFrameCountOut);
77028 | }
77029 |
77030 |
77031 | static ma_node_vtable g_ma_engine_node_vtable__sound =
77032 | {
77033 |     ma_engine_node_process_pcm_frames_sound,
77034 |     NULL, /* onGetRequiredInputFrameCount */
77035 |     0, /* Sounds are data source nodes which means they have zero inputs (their input is drawn
> from the data source itself). */
77036 |     1, /* Sounds have one output bus. */
77037 |     0 /* Default flags. */
77038 | };
77039 |
77040 | static ma_node_vtable g_ma_engine_node_vtable__group =
77041 | {
77042 |     ma_engine_node_process_pcm_frames_group,
77043 |     NULL, /* onGetRequiredInputFrameCount */
77044 |     1, /* Groups have one input bus. */
77045 |     1, /* Groups have one output bus. */
77046 |     MA_NODE_FLAG_DIFFERENT_PROCESSING_RATES /* The engine node does resampling so should let
> miniaudio know about it. */
77047 | };
77048 |
77049 |
77050 |
77051 | static ma_node_config ma_engine_node_base_node_config_init(const ma_engine_node_config* pConfig)
77052 | {
77053 |     ma_node_config baseNodeConfig;
77054 |
77055 |     if (pConfig->type == ma_engine_node_type_sound) {
77056 |         /* Sound. */
77057 |         baseNodeConfig = ma_node_config_init();
77058 |         baseNodeConfig.vtable = &g_ma_engine_node_vtable__sound;
77059 |         baseNodeConfig.initialState = ma_node_state_stopped; /* Sounds are stopped by default. */
77060 |     } else {
77061 |         /* Group. */
77062 |         baseNodeConfig = ma_node_config_init();
77063 |         baseNodeConfig.vtable = &g_ma_engine_node_vtable__group;
77064 |         baseNodeConfig.initialState = ma_node_state_started; /* Groups are started by default. */
77065 |     }
77066 |
77067 |     return baseNodeConfig;
77068 | }

```

```

77069 |
77070 | static ma_spatializer_config ma_engine_node_spatializer_config_init(const ma_node_config*
> pBaseNodeConfig)
77071 | {
77072 |     return ma_spatializer_config_init(pBaseNodeConfig->pInputChannels[0],
> pBaseNodeConfig->pOutputChannels[0]);
77073 | }
77074 |
77075 | typedef struct
77076 | {
77077 |     size_t sizeInBytes;
77078 |     size_t baseNodeOffset;
77079 |     size_t resamplerOffset;
77080 |     size_t spatializerOffset;
77081 |     size_t gainerOffset;
77082 | } ma_engine_node_heap_layout;
77083 |
77084 | static ma_result ma_engine_node_get_heap_layout(const ma_engine_node_config* pConfig,
> ma_engine_node_heap_layout* pHeapLayout)
77085 | {
77086 |     ma_result result;
77087 |     size_t tempHeapSize;
77088 |     ma_node_config baseNodeConfig;
77089 |     ma_resampler_config resamplerConfig;
77090 |     ma_spatializer_config spatializerConfig;
77091 |     ma_gainer_config gainerConfig;
77092 |     ma_uint32 sampleRate;
77093 |     ma_uint32 channelsIn;
77094 |     ma_uint32 channelsOut;
77095 |     ma_channel defaultStereoChannelMap[2] = {MA_CHANNEL_SIDE_LEFT, MA_CHANNEL_SIDE_RIGHT}; /* <--
> Consistent with the default channel map of a stereo listener. Means channel conversion can run on a
> fast path. */
77096 |
77097 |     MA_ASSERT(pHeapLayout);
77098 |
77099 |     MA_ZERO_OBJECT(pHeapLayout);
77100 |
77101 |     if (pConfig == NULL) {
77102 |         return MA_INVALID_ARGS;
77103 |     }
77104 |
77105 |     if (pConfig->pEngine == NULL) {
77106 |         return MA_INVALID_ARGS; /* An engine must be specified. */
77107 |     }
77108 |
77109 |     pHeapLayout->sizeInBytes = 0;
77110 |
77111 |     sampleRate = (pConfig->sampleRate > 0) ? pConfig->sampleRate :
> ma_engine_get_sample_rate(pConfig->pEngine);
77112 |     channelsIn = (pConfig->channelsIn != 0) ? pConfig->channelsIn :
> ma_engine_get_channels(pConfig->pEngine);
77113 |     channelsOut = (pConfig->channelsOut != 0) ? pConfig->channelsOut :
> ma_engine_get_channels(pConfig->pEngine);
77114 |
77115 |
77116 |     /* Base node. */
77117 |     baseNodeConfig = ma_engine_node_base_node_config_init(pConfig);
77118 |     baseNodeConfig.pInputChannels = &channelsIn;
77119 |     baseNodeConfig.pOutputChannels = &channelsOut;
77120 |
77121 |     result = ma_node_get_heap_size(ma_engine_get_node_graph(pConfig->pEngine), &baseNodeConfig,
> &tempHeapSize);
77122 |     if (result != MA_SUCCESS) {
77123 |         return result; /* Failed to retrieve the size of the heap for the base node. */
77124 |     }
77125 |
77126 |     pHeapLayout->baseNodeOffset = pHeapLayout->sizeInBytes;
77127 |     pHeapLayout->sizeInBytes += ma_align_64(tempHeapSize);
77128 |
77129 |
77130 |     /* Resampler. */
77131 |     resamplerConfig = pConfig->resampling;
77132 |     resamplerConfig.format = ma_format_f32;
77133 |     resamplerConfig.channels = channelsIn;
77134 |     resamplerConfig.sampleRateIn = sampleRate;
77135 |     resamplerConfig.sampleRateOut = ma_engine_get_sample_rate(pConfig->pEngine);
77136 |
77137 |     result = ma_resampler_get_heap_size(&resamplerConfig, &tempHeapSize);
77138 |     if (result != MA_SUCCESS) {

```

```

77139 |         return result; /* Failed to retrieve the size of the heap for the resampler. */
77140 |     }
77141 |
77142 |     pHeapLayout->resamplerOffset = pHeapLayout->sizeInBytes;
77143 |     pHeapLayout->sizeInBytes += ma_align_64(tempHeapSize);
77144 |
77145 |
77146 |     /* Spatializer. */
77147 |     spatializerConfig = ma_engine_node_spatializer_config_init(&baseNodeConfig);
77148 |
77149 |     if (spatializerConfig.channelsIn == 2) {
77150 |         spatializerConfig.pChannelMapIn = defaultStereoChannelMap;
77151 |     }
77152 |
77153 |     result = ma_spatializer_get_heap_size(&spatializerConfig, &tempHeapSize);
77154 |     if (result != MA_SUCCESS) {
77155 |         return result; /* Failed to retrieve the size of the heap for the spatializer. */
77156 |     }
77157 |
77158 |     pHeapLayout->spatializerOffset = pHeapLayout->sizeInBytes;
77159 |     pHeapLayout->sizeInBytes += ma_align_64(tempHeapSize);
77160 |
77161 |
77162 |     /* Gainer. Will not be used if we are not using smoothing. */
77163 |     if (pConfig->volumeSmoothTimeInPCMFrames > 0) {
77164 |         gainerConfig = ma_gainer_config_init(channelsIn, pConfig->volumeSmoothTimeInPCMFrames);
77165 |
77166 |         result = ma_gainer_get_heap_size(&gainerConfig, &tempHeapSize);
77167 |         if (result != MA_SUCCESS) {
77168 |             return result;
77169 |         }
77170 |
77171 |         pHeapLayout->gainerOffset = pHeapLayout->sizeInBytes;
77172 |         pHeapLayout->sizeInBytes += ma_align_64(tempHeapSize);
77173 |     }
77174 |
77175 |
77176 |     return MA_SUCCESS;
77177 | }
77178 |
77179 | MA_API ma_result ma_engine_node_get_heap_size(const ma_engine_node_config* pConfig, size_t*
> pHeapSizeInBytes)
77180 | {
77181 |     ma_result result;
77182 |     ma_engine_node_heap_layout heapLayout;
77183 |
77184 |     if (pHeapSizeInBytes == NULL) {
77185 |         return MA_INVALID_ARGS;
77186 |     }
77187 |
77188 |     *pHeapSizeInBytes = 0;
77189 |
77190 |     result = ma_engine_node_get_heap_layout(pConfig, &heapLayout);
77191 |     if (result != MA_SUCCESS) {
77192 |         return result;
77193 |     }
77194 |
77195 |     *pHeapSizeInBytes = heapLayout.sizeInBytes;
77196 |
77197 |     return MA_SUCCESS;
77198 | }
77199 |
77200 | MA_API ma_result ma_engine_node_init_preallocated(const ma_engine_node_config* pConfig, void* pHeap,
> ma_engine_node* pEngineNode)
77201 | {
77202 |     ma_result result;
77203 |     ma_engine_node_heap_layout heapLayout;
77204 |     ma_node_config baseNodeConfig;
77205 |     ma_resampler_config resamplerConfig;
77206 |     ma_fader_config faderConfig;
77207 |     ma_spatializer_config spatializerConfig;
77208 |     ma_panner_config pannerConfig;
77209 |     ma_gainer_config gainerConfig;
77210 |     ma_uint32 channelsIn;
77211 |     ma_uint32 channelsOut;
77212 |     ma_channel defaultStereoChannelMap[2] = {MA_CHANNEL_SIDE_LEFT, MA_CHANNEL_SIDE_RIGHT}; /* <--
> Consistent with the default channel map of a stereo listener. Means channel conversion can run on a
> fast path. */
77213 |

```

```

77214 |     if (pEngineNode == NULL) {
77215 |         return MA_INVALID_ARGS;
77216 |     }
77217 |
77218 |     MA_ZERO_OBJECT(pEngineNode);
77219 |
77220 |     result = ma_engine_node_get_heap_layout(pConfig, &heapLayout);
77221 |     if (result != MA_SUCCESS) {
77222 |         return result;
77223 |     }
77224 |
77225 |     if (pConfig->pinnedListenerIndex != MA_LISTENER_INDEX_CLOSEST && pConfig->pinnedListenerIndex >=
> ma_engine_get_listener_count(pConfig->pEngine)) {
77226 |         return MA_INVALID_ARGS; /* Invalid listener. */
77227 |     }
77228 |
77229 |     pEngineNode->pHeap = pHeap;
77230 |     MA_ZERO_MEMORY(pHeap, heapLayout.sizeInBytes);
77231 |
77232 |     pEngineNode->pEngine = pConfig->pEngine;
77233 |     pEngineNode->sampleRate = (pConfig->sampleRate > 0) ? pConfig->sampleRate :
> ma_engine_get_sample_rate(pEngineNode->pEngine);
77234 |     pEngineNode->volumeSmoothTimeInPCMFrames = pConfig->volumeSmoothTimeInPCMFrames;
77235 |     pEngineNode->monoExpansionMode = pConfig->monoExpansionMode;
77236 |     ma_atomic_float_set(&pEngineNode->volume, 1);
77237 |     pEngineNode->pitch = 1;
77238 |     pEngineNode->oldPitch = 1;
77239 |     pEngineNode->oldDopplerPitch = 1;
77240 |     pEngineNode->isPitchDisabled = pConfig->isPitchDisabled;
77241 |     pEngineNode->isSpatializationDisabled = pConfig->isSpatializationDisabled;
77242 |     pEngineNode->pinnedListenerIndex = pConfig->pinnedListenerIndex;
77243 |     ma_atomic_float_set(&pEngineNode->fadeSettings.volumeBeg, 1);
77244 |     ma_atomic_float_set(&pEngineNode->fadeSettings.volumeEnd, 1);
77245 |     ma_atomic_uint64_set(&pEngineNode->fadeSettings.fadeLengthInFrames, ~(ma_uint64)0);
77246 |     ma_atomic_uint64_set(&pEngineNode->fadeSettings.absoluteGlobalTimeInFrames, ~(ma_uint64)0);
> /* <-- Indicates that the fade should start immediately. */
77247 |
77248 |     channelsIn = (pConfig->channelsIn != 0) ? pConfig->channelsIn :
> ma_engine_get_channels(pConfig->pEngine);
77249 |     channelsOut = (pConfig->channelsOut != 0) ? pConfig->channelsOut :
> ma_engine_get_channels(pConfig->pEngine);
77250 |
77251 |     /*
77252 |     If the sample rate of the sound is different to the engine, make sure pitching is enabled so
> that the resampler
77253 |     is activated. Not doing this will result in the sound not being resampled if
> MA_SOUND_FLAG_NO_PITCH is used.
77254 |     */
77255 |     if (pEngineNode->sampleRate != ma_engine_get_sample_rate(pEngineNode->pEngine)) {
77256 |         pEngineNode->isPitchDisabled = MA_FALSE;
77257 |     }
77258 |
77259 |
77260 |     /* Base node. */
77261 |     baseNodeConfig = ma_engine_node_base_node_config_init(pConfig);
77262 |     baseNodeConfig.pInputChannels = &channelsIn;
77263 |     baseNodeConfig.pOutputChannels = &channelsOut;
77264 |
77265 |     result = ma_node_init_preallocated(&pConfig->pEngine->nodeGraph, &baseNodeConfig,
> ma_offset_ptr(pHeap, heapLayout.baseNodeOffset), &pEngineNode->baseNode);
77266 |     if (result != MA_SUCCESS) {
77267 |         goto error0;
77268 |     }
77269 |
77270 |
77271 |     /*
77272 |     We can now initialize the effects we need in order to implement the engine node. There's a
77273 |     defined order of operations here, mainly centered around when we convert our channels from the
77274 |     data source's native channel count to the engine's channel count. As a rule, we want to do as
77275 |     much computation as possible before spatialization because there's a chance that will increase
77276 |     the channel count, thereby increasing the amount of work needing to be done to process.
77277 |     */
77278 |
77279 |     /* We'll always do resampling first. */
77280 |     resamplerConfig = pConfig->resampling;
77281 |     resamplerConfig.format = ma_format_f32;
77282 |     resamplerConfig.channels = baseNodeConfig.pInputChannels[0];
77283 |     resamplerConfig.sampleRateIn = pEngineNode->sampleRate;
77284 |     resamplerConfig.sampleRateOut = ma_engine_get_sample_rate(pEngineNode->pEngine);

```

```

77285 |
77286 |     result = ma_resampler_init_preallocated(&resamplerConfig, ma_offset_ptr(pHeap,
> heapLayout.resamplerOffset), &pEngineNode->resampler);
77287 |     if (result != MA_SUCCESS) {
77288 |         goto error1;
77289 |     }
77290 |
77291 |
77292 |     /* After resampling will come the fader. */
77293 |     faderConfig = ma_fader_config_init(ma_format_f32, baseNodeConfig.pInputChannels[0],
> ma_engine_get_sample_rate(pEngineNode->pEngine));
77294 |
77295 |     result = ma_fader_init(&faderConfig, &pEngineNode->fader);
77296 |     if (result != MA_SUCCESS) {
77297 |         goto error2;
77298 |     }
77299 |
77300 |
77301 |     /*
77302 |     Spatialization comes next. We spatialize based on the node's output channel count. It's up the
> caller to
77303 |     ensure channels counts link up correctly in the node graph.
77304 |     */
77305 |     spatializerConfig = ma_engine_node_spatializer_config_init(&baseNodeConfig);
77306 |     spatializerConfig.gainSmoothTimeInFrames = pEngineNode->pEngine->gainSmoothTimeInFrames;
77307 |
77308 |     if (spatializerConfig.channelsIn == 2) {
77309 |         spatializerConfig.pChannelMapIn = defaultStereoChannelMap;
77310 |     }
77311 |
77312 |     result = ma_spatializer_init_preallocated(&spatializerConfig, ma_offset_ptr(pHeap,
> heapLayout.spatializerOffset), &pEngineNode->spatializer);
77313 |     if (result != MA_SUCCESS) {
77314 |         goto error2;
77315 |     }
77316 |
77317 |
77318 |     /*
77319 |     After spatialization comes panning. We need to do this after spatialization because otherwise we
> wouldn't
77320 |     be able to pan mono sounds.
77321 |     */
77322 |     pannerConfig = ma_panner_config_init(ma_format_f32, baseNodeConfig.pOutputChannels[0]);
77323 |
77324 |     result = ma_panner_init(&pannerConfig, &pEngineNode->panner);
77325 |     if (result != MA_SUCCESS) {
77326 |         goto error3;
77327 |     }
77328 |
77329 |
77330 |     /* We'll need a gainer for smoothing out volume changes if we have a non-zero smooth time. We
> apply this before converting to the output channel count. */
77331 |     if (pConfig->volumeSmoothTimeInPCMFrames > 0) {
77332 |         gainerConfig = ma_gainer_config_init(channelsIn, pConfig->volumeSmoothTimeInPCMFrames);
77333 |
77334 |         result = ma_gainer_init_preallocated(&gainerConfig, ma_offset_ptr(pHeap,
> heapLayout.gainerOffset), &pEngineNode->volumeGainer);
77335 |         if (result != MA_SUCCESS) {
77336 |             goto error3;
77337 |         }
77338 |     }
77339 |
77340 |
77341 |     return MA_SUCCESS;
77342 |
77343 |     /* No need for allocation callbacks here because we use a preallocated heap. */
77344 | error3: ma_spatializer_uninit(&pEngineNode->spatializer, NULL);
77345 | error2: ma_resampler_uninit(&pEngineNode->resampler, NULL);
77346 | error1: ma_node_uninit(&pEngineNode->baseNode, NULL);
77347 | error0: return result;
77348 | }
77349 |
77350 | MA_API ma_result ma_engine_node_init(const ma_engine_node_config* pConfig, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_engine_node* pEngineNode)
77351 | {
77352 |     ma_result result;
77353 |     size_t heapSizeInBytes;
77354 |     void* pHeap;
77355 |

```

```

77356 |     result = ma_engine_node_get_heap_size(pConfig, &heapSizeInBytes);
77357 |     if (result != MA_SUCCESS) {
77358 |         return result;
77359 |     }
77360 |
77361 |     if (heapSizeInBytes > 0) {
77362 |         pHeap = ma_malloc(heapSizeInBytes, pAllocationCallbacks);
77363 |         if (pHeap == NULL) {
77364 |             return MA_OUT_OF_MEMORY;
77365 |         }
77366 |     } else {
77367 |         pHeap = NULL;
77368 |     }
77369 |
77370 |     result = ma_engine_node_init_preallocated(pConfig, pHeap, pEngineNode);
77371 |     if (result != MA_SUCCESS) {
77372 |         ma_free(pHeap, pAllocationCallbacks);
77373 |         return result;
77374 |     }
77375 |
77376 |     pEngineNode->_ownsHeap = MA_TRUE;
77377 |     return MA_SUCCESS;
77378 | }
77379 |
77380 | MA_API void ma_engine_node_uninit(ma_engine_node* pEngineNode, const ma_allocation_callbacks*
> pAllocationCallbacks)
77381 | {
77382 |     /*
77383 |     The base node always needs to be uninitialized first to ensure it's detached from the graph
> completely before we
77384 |     destroy anything that might be in the middle of being used by the processing function.
77385 |     */
77386 |     ma_node_uninit(&pEngineNode->baseNode, pAllocationCallbacks);
77387 |
77388 |     /* Now that the node has been uninitialized we can safely uninitialized the rest. */
77389 |     if (pEngineNode->volumeSmoothTimeInPCMFrames > 0) {
77390 |         ma_gainer_uninit(&pEngineNode->volumeGainer, pAllocationCallbacks);
77391 |     }
77392 |
77393 |     ma_spatializer_uninit(&pEngineNode->spatializer, pAllocationCallbacks);
77394 |     ma_resampler_uninit(&pEngineNode->resampler, pAllocationCallbacks);
77395 |
77396 |     /* Free the heap last. */
77397 |     if (pEngineNode->_ownsHeap) {
77398 |         ma_free(pEngineNode->pHeap, pAllocationCallbacks);
77399 |     }
77400 | }
77401 |
77402 |
77403 | MA_API ma_sound_config ma_sound_config_init(void)
77404 | {
77405 |     return ma_sound_config_init_2(NULL);
77406 | }
77407 |
77408 | MA_API ma_sound_config ma_sound_config_init_2(ma_engine* pEngine)
77409 | {
77410 |     ma_sound_config config;
77411 |
77412 |     MA_ZERO_OBJECT(&config);
77413 |
77414 |     if (pEngine != NULL) {
77415 |         config.monoExpansionMode = pEngine->monoExpansionMode;
77416 |         config.pitchResampling = pEngine->pitchResamplingConfig;
77417 |     } else {
77418 |         config.monoExpansionMode = ma_mono_expansion_mode_default;
77419 |
77420 |         config.pitchResampling = ma_resampler_config_init(ma_format_f32, 0, 0, 0,
> ma_resample_algorithm_linear);
77421 |         config.pitchResampling.linear.lpfOrder = 0; /* <-- Need to disable low-pass filtering for
> pitch shifting for now because there's cases where the biquads are becoming unstable. Need to figure
> out a better fix for this. */
77422 |     }
77423 |
77424 |     config.rangeEndInPCMFrames = ~((ma_uint64)0);
77425 |     config.loopPointEndInPCMFrames = ~((ma_uint64)0);
77426 |
77427 |     return config;
77428 | }
77429 |

```



```

77430 | MA_API ma_sound_group_config ma_sound_group_config_init(void)
77431 | {
77432 |     return ma_sound_group_config_init_2(NULL);
77433 | }
77434 |
77435 | MA_API ma_sound_group_config ma_sound_group_config_init_2(ma_engine* pEngine)
77436 | {
77437 |     ma_sound_group_config config;
77438 |
77439 |     MA_ZERO_OBJECT(&config);
77440 |
77441 |     if (pEngine != NULL) {
77442 |         config.monoExpansionMode = pEngine->monoExpansionMode;
77443 |         config.pitchResampling    = pEngine->pitchResamplingConfig;
77444 |     } else {
77445 |         config.monoExpansionMode = ma_mono_expansion_mode_default;
77446 |
77447 |         config.pitchResampling = ma_resampler_config_init(ma_format_f32, 0, 0, 0,
> ma_resample_algorithm_linear);
77448 |         config.pitchResampling.linear.lpfOrder = 0; /* <-- Need to disable low-pass filtering for
> pitch shifting for now because there's cases where the biquads are becoming unstable. Need to figure
> out a better fix for this. */
77449 |     }
77450 |
77451 |     return config;
77452 | }
77453 |
77454 |
77455 | MA_API ma_engine_config ma_engine_config_init(void)
77456 | {
77457 |     ma_engine_config config;
77458 |
77459 |     MA_ZERO_OBJECT(&config);
77460 |     config.listenerCount      = 1; /* Always want at least one listener. */
77461 |     config.monoExpansionMode  = ma_mono_expansion_mode_default;
77462 |     config.resourceManagerResampling = ma_resampler_config_init(ma_format_unknown, 0, 0, 0,
> ma_resample_algorithm_linear);
77463 |
77464 |     config.pitchResampling = ma_resampler_config_init(ma_format_f32, 0, 0, 0,
> ma_resample_algorithm_linear);
77465 |     config.pitchResampling.linear.lpfOrder = 0; /* <-- Need to disable low-pass filtering for pitch
> shifting for now because there's cases where the biquads are becoming unstable. Need to figure out a
> better fix for this. */
77466 |
77467 |     return config;
77468 | }
77469 |
77470 |
77471 | #if !defined(MA_NO_DEVICE_IO)
77472 | static void ma_engine_data_callback_internal(ma_device* pDevice, void* pFramesOut, const void*
> pFramesIn, ma_uint32 frameCount)
77473 | {
77474 |     ma_engine* pEngine = (ma_engine*)pDevice->pUserData;
77475 |
77476 |     (void)pFramesIn;
77477 |
77478 |     /*
77479 |     Experiment: Try processing a resource manager job if we're on the Emscripten build.
77480 |
77481 |     This serves two purposes:
77482 |
77483 |     1) It ensures jobs are actually processed at some point since we cannot guarantee that the
77484 |        caller is doing the right thing and calling ma_resource_manager_process_next_job(); and
77485 |
77486 |     2) It's an attempt at working around an issue where processing jobs on the Emscripten main
77487 |        loop doesn't work as well as it should. When trying to load sounds without the `DECODE`
77488 |        flag or with the `ASYNC` flag, the sound data is just not able to be loaded in time
77489 |        before the callback is processed. I think it's got something to do with the single-
77490 |        threaded nature of Web, but I'm not entirely sure.
77491 |     */
77492 |     #if !defined(MA_NO_RESOURCE_MANAGER) && defined(MA_EMSCRIPTEEN)
77493 |     {
77494 |         if (pEngine->pResourceManager != NULL) {
77495 |             if ((pEngine->pResourceManager->config.flags & MA_RESOURCE_MANAGER_FLAG_NO_THREADING) !=
> 0) {
77496 |                 ma_resource_manager_process_next_job(pEngine->pResourceManager);
77497 |             }
77498 |         }
77499 |     }

```

```

77500 |     #endif
77501 |
77502 |     ma_engine_read_pcm_frames(pEngine, pFramesOut, frameCount, NULL);
77503 | }
77504 |
77505 | static ma_uint32 ma_device__get_processing_size_in_frames(ma_device* pDevice)
77506 | {
77507 |     /*
77508 |     The processing size is the period size. The device can have a fixed sized processing size, or
77509 |     it can be decided by the backend in which case it can be variable.
77510 |     */
77511 |     if (pDevice->playback.intermediaryBufferCap > 0) {
77512 |         /* Using a fixed sized processing callback. */
77513 |         return pDevice->playback.intermediaryBufferCap;
77514 |     } else {
77515 |         /* Not using a fixed sized processing callback. Need to estimate the processing size based
> on the backend. */
77516 |         return pDevice->playback.internalPeriodSizeInFrames;
77517 |     }
77518 | }
77519 | #endif
77520 |
77521 | MA_API ma_result ma_engine_init(const ma_engine_config* pConfig, ma_engine* pEngine)
77522 | {
77523 |     ma_result result;
77524 |     ma_node_graph_config nodeGraphConfig;
77525 |     ma_engine_config engineConfig;
77526 |     ma_spatializer_listener_config listenerConfig;
77527 |     ma_uint32 iListener;
77528 |
77529 |     if (pEngine == NULL) {
77530 |         return MA_INVALID_ARGS;
77531 |     }
77532 |
77533 |     MA_ZERO_OBJECT(pEngine);
77534 |
77535 |     /* The config is allowed to be NULL in which case we use defaults for everything. */
77536 |     if (pConfig != NULL) {
77537 |         engineConfig = *pConfig;
77538 |     } else {
77539 |         engineConfig = ma_engine_config_init();
77540 |     }
77541 |
77542 |     pEngine->monoExpansionMode = engineConfig.monoExpansionMode;
77543 |     pEngine->defaultVolumeSmoothTimeInPCMFrames = engineConfig.defaultVolumeSmoothTimeInPCMFrames;
77544 |     pEngine->onProcess = engineConfig.onProcess;
77545 |     pEngine->pProcessUserData = engineConfig.pProcessUserData;
77546 |     pEngine->pitchResamplingConfig = engineConfig.pitchResampling;
77547 |     ma_allocation_callbacks_init_copy(&pEngine->allocationCallbacks,
> &engineConfig.allocationCallbacks);
77548 |
77549 |     #if !defined(MA_NO_RESOURCE_MANAGER)
77550 |     {
77551 |         pEngine->pResourceManager = engineConfig.pResourceManager;
77552 |     }
77553 |     #endif
77554 |
77555 |     #if !defined(MA_NO_DEVICE_IO)
77556 |     {
77557 |         pEngine->pDevice = engineConfig.pDevice;
77558 |
77559 |         /* If we don't have a device, we need one. */
77560 |         if (pEngine->pDevice == NULL && engineConfig.noDevice == MA_FALSE) {
77561 |             ma_device_config deviceConfig;
77562 |
77563 |             pEngine->pDevice = (ma_device*)ma_malloc(sizeof(*pEngine->pDevice),
> &pEngine->allocationCallbacks);
77564 |             if (pEngine->pDevice == NULL) {
77565 |                 return MA_OUT_OF_MEMORY;
77566 |             }
77567 |
77568 |             deviceConfig = ma_device_config_init(ma_device_type_playback);
77569 |             deviceConfig.playback.pDeviceID = engineConfig.pPlaybackDeviceID;
77570 |             deviceConfig.playback.format = ma_format_f32;
77571 |             deviceConfig.playback.channels = engineConfig.channels;
77572 |             deviceConfig.sampleRate = engineConfig.sampleRate;
77573 |             deviceConfig.dataCallback = (engineConfig.dataCallback != NULL) ?
> engineConfig.dataCallback : ma_engine_data_callback_internal;
77574 |             deviceConfig.pUserData = pEngine;

```

```

77575 |         deviceConfig.notificationCallback    = engineConfig.notificationCallback;
77576 |         deviceConfig.periodSizeInFrames     = engineConfig.periodSizeInFrames;
77577 |         deviceConfig.periodSizeInMilliseconds = engineConfig.periodSizeInMilliseconds;
77578 |         deviceConfig.noPreSilencedOutputBuffer = MA_TRUE;    /* We'll always be outputting to
> every frame in the callback so there's no need for a pre-silenced buffer. */
77579 |         deviceConfig.noClip                 = MA_TRUE;    /* The engine will do clipping
> itself. */
77580 |
77581 |         if (engineConfig.pContext == NULL) {
77582 |             ma_context_config contextConfig = ma_context_config_init();
77583 |             contextConfig.allocationCallbacks = pEngine->allocationCallbacks;
77584 |             contextConfig.pLog = engineConfig.pLog;
77585 |
77586 |             /* If the engine config does not specify a log, use the resource manager's if we
> have one. */
77587 |             #ifndef MA_NO_RESOURCE_MANAGER
77588 |             {
77589 |                 if (contextConfig.pLog == NULL && engineConfig.pResourceManager != NULL) {
77590 |                     contextConfig.pLog =
> ma_resource_manager_get_log(engineConfig.pResourceManager);
77591 |                 }
77592 |             }
77593 |             #endif
77594 |
77595 |             result = ma_device_init_ex(NULL, 0, &contextConfig, &deviceConfig,
> pEngine->pDevice);
77596 |         } else {
77597 |             result = ma_device_init(engineConfig.pContext, &deviceConfig, pEngine->pDevice);
77598 |         }
77599 |
77600 |         if (result != MA_SUCCESS) {
77601 |             ma_free(pEngine->pDevice, &pEngine->allocationCallbacks);
77602 |             pEngine->pDevice = NULL;
77603 |             return result;
77604 |         }
77605 |
77606 |         pEngine->ownsDevice = MA_TRUE;
77607 |     }
77608 |
77609 |     /* Update the channel count and sample rate of the engine config so we can reference it
> below. */
77610 |     if (pEngine->pDevice != NULL) {
77611 |         engineConfig.channels = pEngine->pDevice->playback.channels;
77612 |         engineConfig.sampleRate = pEngine->pDevice->sampleRate;
77613 |
77614 |         /*
77615 |         The processing size used by the engine is determined by engineConfig.periodSizeInFrames.
> We want
77616 |         to make this equal to what the device is using for it's period size. If we don't do
> that, it's
77617 |         possible that the node graph will split it's processing into multiple passes which can
> introduce
77618 |         glitching.
77619 |         */
77620 |         engineConfig.periodSizeInFrames =
> ma_device_get_processing_size_in_frames(pEngine->pDevice);
77621 |     }
77622 | }
77623 | #endif
77624 |
77625 | if (engineConfig.channels == 0 || engineConfig.sampleRate == 0) {
77626 |     return MA_INVALID_ARGS;
77627 | }
77628 |
77629 | pEngine->sampleRate = engineConfig.sampleRate;
77630 |
77631 | /* The engine always uses either the log that was passed into the config, or the context's log
> is available. */
77632 | if (engineConfig.pLog != NULL) {
77633 |     pEngine->pLog = engineConfig.pLog;
77634 | } else {
77635 |     #if !defined(MA_NO_DEVICE_IO)
77636 |     {
77637 |         pEngine->pLog = ma_device_get_log(pEngine->pDevice);
77638 |     }
77639 |     #else
77640 |     {
77641 |         pEngine->pLog = NULL;
77642 |     }

```

```

77643 |         #endif
77644 |     }
77645 |
77646 |
77647 |     /* The engine is a node graph. This needs to be initialized after we have the device so we can
> determine the channel count. */
77648 |     nodeGraphConfig = ma_node_graph_config_init(engineConfig.channels);
77649 |     nodeGraphConfig.processingSizeInFrames = engineConfig.periodSizeInFrames;
77650 |     nodeGraphConfig.preMixStackSizeInBytes = engineConfig.preMixStackSizeInBytes;
77651 |
77652 |     result = ma_node_graph_init(&nodeGraphConfig, &pEngine->allocationCallbacks,
> &pEngine->nodeGraph);
77653 |     if (result != MA_SUCCESS) {
77654 |         goto on_error_1;
77655 |     }
77656 |
77657 |
77658 |     /* We need at least one listener. */
77659 |     if (engineConfig.listenerCount == 0) {
77660 |         engineConfig.listenerCount = 1;
77661 |     }
77662 |
77663 |     if (engineConfig.listenerCount > MA_ENGINE_MAX_LISTENERS) {
77664 |         result = MA_INVALID_ARGS; /* Too many listeners. */
77665 |         goto on_error_1;
77666 |     }
77667 |
77668 |     for (iListener = 0; iListener < engineConfig.listenerCount; iListener += 1) {
77669 |         listenerConfig =
> ma_spatializer_listener_config_init(ma_node_graph_get_channels(&pEngine->nodeGraph));
77670 |
77671 |         /*
77672 |         If we're using a device, use the device's channel map for the listener. Otherwise just use
77673 |         miniaudio's default channel map.
77674 |         */
77675 |         #if !defined(MA_NO_DEVICE_IO)
77676 |         {
77677 |             if (pEngine->pDevice != NULL) {
77678 |                 /*
77679 |                 Temporarily disabled. There is a subtle bug here where front-left and front-right
77680 |                 will be used by the device's channel map, but this is not what we want to use for
77681 |                 spatialization. Instead we want to use side-left and side-right. I need to figure
77682 |                 out a better solution for this. For now, disabling the use of device channel maps.
77683 |                 */
77684 |                 /*listenerConfig.pChannelMapOut = pEngine->pDevice->playback.channelMap;*/
77685 |             }
77686 |         }
77687 |         #endif
77688 |
77689 |         result = ma_spatializer_listener_init(&listenerConfig, &pEngine->allocationCallbacks,
> &pEngine->listeners[iListener]); /* TODO: Change this to a pre-allocated heap. */
77690 |         if (result != MA_SUCCESS) {
77691 |             goto on_error_2;
77692 |         }
77693 |
77694 |         pEngine->listenerCount += 1;
77695 |     }
77696 |
77697 |
77698 |     /* Gain smoothing for spatialized sounds. */
77699 |     pEngine->gainSmoothTimeInFrames = engineConfig.gainSmoothTimeInFrames;
77700 |     if (pEngine->gainSmoothTimeInFrames == 0) {
77701 |         ma_uint32 gainSmoothTimeInMilliseconds = engineConfig.gainSmoothTimeInMilliseconds;
77702 |         if (gainSmoothTimeInMilliseconds == 0) {
77703 |             gainSmoothTimeInMilliseconds = 8;
77704 |         }
77705 |
77706 |         pEngine->gainSmoothTimeInFrames = (gainSmoothTimeInMilliseconds *
> ma_engine_get_sample_rate(pEngine)) / 1000; /* 8ms by default. */
77707 |     }
77708 |
77709 |
77710 |     /* We need a resource manager. */
77711 |     #ifndef MA_NO_RESOURCE_MANAGER
77712 |     {
77713 |         if (pEngine->pResourceManager == NULL) {
77714 |             ma_resource_manager_config resourceManagerConfig;
77715 |
77716 |             pEngine->pResourceManager =

```

```

> (ma_resource_manager*)ma_malloc(sizeof(*pEngine->pResourceManager), &pEngine->allocationCallbacks);
77717 |         if (pEngine->pResourceManager == NULL) {
77718 |             result = MA_OUT_OF_MEMORY;
77719 |             goto on_error_2;
77720 |         }
77721 |
77722 |         resourceManagerConfig = ma_resource_manager_config_init();
77723 |         resourceManagerConfig.pLog = pEngine->pLog; /* Always use the engine's
> log for internally-managed resource managers. */
77724 |         resourceManagerConfig.decodedFormat = ma_format_f32;
77725 |         resourceManagerConfig.decodedChannels = 0; /* Leave the decoded channel count as 0 so
> we can get good spatialization. */
77726 |         resourceManagerConfig.decodedSampleRate = ma_engine_get_sample_rate(pEngine);
77727 |         ma_allocation_callbacks_init_copy(&resourceManagerConfig.allocationCallbacks,
> &pEngine->allocationCallbacks);
77728 |         resourceManagerConfig.pVFS = engineConfig.pResourceManagerVFS;
77729 |         resourceManagerConfig.resampling = engineConfig.resourceManagerResampling;
77730 |
77731 |         /* The Emscripten build cannot use threads unless it's targeting pthreads. */
77732 |         #if defined(MA_EMSCRIPTEN) && !defined(__EMSCRIPTEN_PTHREADS__)
77733 |         {
77734 |             resourceManagerConfig.jobThreadCount = 0;
77735 |             resourceManagerConfig.flags |= MA_RESOURCE_MANAGER_FLAG_NO_THREADING;
77736 |         }
77737 |         #endif
77738 |
77739 |         result = ma_resource_manager_init(&resourceManagerConfig, pEngine->pResourceManager);
77740 |         if (result != MA_SUCCESS) {
77741 |             goto on_error_3;
77742 |         }
77743 |
77744 |         pEngine->ownsResourceManager = MA_TRUE;
77745 |     }
77746 | }
77747 | #endif
77748 |
77749 | /* Setup some stuff for inlined sounds. That is sounds played with ma_engine_play_sound(). */
77750 | pEngine->inlinedSoundLock = 0;
77751 | pEngine->pInlinedSoundHead = NULL;
77752 |
77753 | /* Start the engine if required. This should always be the last step. */
77754 | #if !defined(MA_NO_DEVICE_IO)
77755 | {
77756 |     if (engineConfig.noAutoStart == MA_FALSE && pEngine->pDevice != NULL) {
77757 |         result = ma_engine_start(pEngine);
77758 |         if (result != MA_SUCCESS) {
77759 |             goto on_error_4; /* Failed to start the engine. */
77760 |         }
77761 |     }
77762 | }
77763 | #endif
77764 |
77765 | return MA_SUCCESS;
77766 |
77767 | #if !defined(MA_NO_DEVICE_IO)
77768 | on_error_4:
77769 | #endif
77770 | #if !defined(MA_NO_RESOURCE_MANAGER)
77771 | on_error_3:
77772 |     if (pEngine->ownsResourceManager) {
77773 |         ma_free(pEngine->pResourceManager, &pEngine->allocationCallbacks);
77774 |     }
77775 | #endif /* MA_NO_RESOURCE_MANAGER */
77776 | on_error_2:
77777 |     for (iListener = 0; iListener < pEngine->listenerCount; iListener += 1) {
77778 |         ma_spatializer_listener_uninit(&pEngine->listeners[iListener],
> &pEngine->allocationCallbacks);
77779 |     }
77780 |
77781 |     ma_node_graph_uninit(&pEngine->nodeGraph, &pEngine->allocationCallbacks);
77782 | on_error_1:
77783 |     #if !defined(MA_NO_DEVICE_IO)
77784 |     {
77785 |         if (pEngine->ownsDevice) {
77786 |             ma_device_uninit(pEngine->pDevice);
77787 |             ma_free(pEngine->pDevice, &pEngine->allocationCallbacks);
77788 |         }
77789 |     }
77790 |     #endif

```

```

77791 |
77792 |     return result;
77793 | }
77794 |
77795 | MA_API void ma_engine_uninit(ma_engine* pEngine)
77796 | {
77797 |     ma_uint32 iListener;
77798 |
77799 |     if (pEngine == NULL) {
77800 |         return;
77801 |     }
77802 |
77803 |     /* The device must be uninitialized before the node graph to ensure the audio thread doesn't try
> accessing it. */
77804 |     #if !defined(MA_NO_DEVICE_IO)
77805 |     {
77806 |         if (pEngine->ownsDevice) {
77807 |             ma_device_uninit(pEngine->pDevice);
77808 |             ma_free(pEngine->pDevice, &pEngine->allocationCallbacks);
77809 |         } else {
77810 |             if (pEngine->pDevice != NULL) {
77811 |                 ma_device_stop(pEngine->pDevice);
77812 |             }
77813 |         }
77814 |     }
77815 |     #endif
77816 |
77817 |     /*
77818 |     All inlined sounds need to be deleted. I'm going to use a lock here just to future proof in case
77819 |     I want to do some kind of garbage collection later on.
77820 |     */
77821 |     ma_spinlock_lock(&pEngine->inlinedSoundLock);
77822 |     {
77823 |         for (;;) {
77824 |             ma_sound_inlined* pSoundToDelete = pEngine->pInlinedSoundHead;
77825 |             if (pSoundToDelete == NULL) {
77826 |                 break; /* Done. */
77827 |             }
77828 |
77829 |             pEngine->pInlinedSoundHead = pSoundToDelete->pNext;
77830 |
77831 |             ma_sound_uninit(&pSoundToDelete->sound);
77832 |             ma_free(pSoundToDelete, &pEngine->allocationCallbacks);
77833 |         }
77834 |     }
77835 |     ma_spinlock_unlock(&pEngine->inlinedSoundLock);
77836 |
77837 |     for (iListener = 0; iListener < pEngine->listenerCount; iListener += 1) {
77838 |         ma_spatializer_listener_uninit(&pEngine->listeners[iListener],
> &pEngine->allocationCallbacks);
77839 |     }
77840 |
77841 |     /* Make sure the node graph is uninitialized after the audio thread has been shutdown to prevent
> accessing of the node graph after being uninitialized. */
77842 |     ma_node_graph_uninit(&pEngine->nodeGraph, &pEngine->allocationCallbacks);
77843 |
77844 |     /* Uninitialize the resource manager last to ensure we don't have a thread still trying to
> access it. */
77845 |     #ifndef MA_NO_RESOURCE_MANAGER
77846 |     if (pEngine->ownsResourceManager) {
77847 |         ma_resource_manager_uninit(pEngine->pResourceManager);
77848 |         ma_free(pEngine->pResourceManager, &pEngine->allocationCallbacks);
77849 |     }
77850 |     #endif
77851 | }
77852 |
77853 | MA_API ma_result ma_engine_read_pcm_frames(ma_engine* pEngine, void* pFramesOut, ma_uint64
> frameCount, ma_uint64* pFramesRead)
77854 | {
77855 |     ma_result result;
77856 |     ma_uint64 framesRead = 0;
77857 |
77858 |     if (pFramesRead != NULL) {
77859 |         *pFramesRead = 0;
77860 |     }
77861 |
77862 |     result = ma_node_graph_read_pcm_frames(&pEngine->nodeGraph, pFramesOut, frameCount,
> &framesRead);
77863 |     if (result != MA_SUCCESS) {

```

```

77864 |         return result;
77865 |     }
77866 |
77867 |     if (pFramesRead != NULL) {
77868 |         *pFramesRead = framesRead;
77869 |     }
77870 |
77871 |     if (pEngine->onProcess) {
77872 |         pEngine->onProcess(pEngine->pProcessUserData, (float*)pFramesOut, framesRead); /* Safe cast
> to float* because the engine always works on floating point samples. */
77873 |     }
77874 |
77875 |     return MA_SUCCESS;
77876 | }
77877 |
77878 | MA_API ma_node_graph* ma_engine_get_node_graph(ma_engine* pEngine)
77879 | {
77880 |     if (pEngine == NULL) {
77881 |         return NULL;
77882 |     }
77883 |
77884 |     return &pEngine->nodeGraph;
77885 | }
77886 |
77887 | #if !defined(MA_NO_RESOURCE_MANAGER)
77888 | MA_API ma_resource_manager* ma_engine_get_resource_manager(ma_engine* pEngine)
77889 | {
77890 |     if (pEngine == NULL) {
77891 |         return NULL;
77892 |     }
77893 |
77894 |     #if !defined(MA_NO_RESOURCE_MANAGER)
77895 |     {
77896 |         return pEngine->pResourceManager;
77897 |     }
77898 |     #else
77899 |     {
77900 |         return NULL;
77901 |     }
77902 |     #endif
77903 | }
77904 | #endif
77905 |
77906 | MA_API ma_device* ma_engine_get_device(ma_engine* pEngine)
77907 | {
77908 |     if (pEngine == NULL) {
77909 |         return NULL;
77910 |     }
77911 |
77912 |     #if !defined(MA_NO_DEVICE_IO)
77913 |     {
77914 |         return pEngine->pDevice;
77915 |     }
77916 |     #else
77917 |     {
77918 |         return NULL;
77919 |     }
77920 |     #endif
77921 | }
77922 |
77923 | MA_API ma_log* ma_engine_get_log(ma_engine* pEngine)
77924 | {
77925 |     if (pEngine == NULL) {
77926 |         return NULL;
77927 |     }
77928 |
77929 |     if (pEngine->pLog != NULL) {
77930 |         return pEngine->pLog;
77931 |     } else {
77932 |         #if !defined(MA_NO_DEVICE_IO)
77933 |         {
77934 |             return ma_device_get_log(ma_engine_get_device(pEngine));
77935 |         }
77936 |         #else
77937 |         {
77938 |             return NULL;
77939 |         }
77940 |         #endif
77941 |     }

```



```

77942 | }
77943 |
77944 | MA_API ma_node* ma_engine_get_endpoint(ma_engine* pEngine)
77945 | {
77946 |     return ma_node_graph_get_endpoint(&pEngine->nodeGraph);
77947 | }
77948 |
77949 | MA_API ma_uint64 ma_engine_get_time_in_pcm_frames(const ma_engine* pEngine)
77950 | {
77951 |     return ma_node_graph_get_time(&pEngine->nodeGraph);
77952 | }
77953 |
77954 | MA_API ma_uint64 ma_engine_get_time_in_milliseconds(const ma_engine* pEngine)
77955 | {
77956 |     return ma_engine_get_time_in_pcm_frames(pEngine) * 1000 / ma_engine_get_sample_rate(pEngine);
77957 | }
77958 |
77959 | MA_API ma_result ma_engine_set_time_in_pcm_frames(ma_engine* pEngine, ma_uint64 globalTime)
77960 | {
77961 |     return ma_node_graph_set_time(&pEngine->nodeGraph, globalTime);
77962 | }
77963 |
77964 | MA_API ma_result ma_engine_set_time_in_milliseconds(ma_engine* pEngine, ma_uint64 globalTime)
77965 | {
77966 |     return ma_engine_set_time_in_pcm_frames(pEngine, globalTime * ma_engine_get_sample_rate(pEngine)
> / 1000);
77967 | }
77968 |
77969 | MA_API ma_uint64 ma_engine_get_time(const ma_engine* pEngine)
77970 | {
77971 |     return ma_engine_get_time_in_pcm_frames(pEngine);
77972 | }
77973 |
77974 | MA_API ma_result ma_engine_set_time(ma_engine* pEngine, ma_uint64 globalTime)
77975 | {
77976 |     return ma_engine_set_time_in_pcm_frames(pEngine, globalTime);
77977 | }
77978 |
77979 | MA_API ma_uint32 ma_engine_get_channels(const ma_engine* pEngine)
77980 | {
77981 |     return ma_node_graph_get_channels(&pEngine->nodeGraph);
77982 | }
77983 |
77984 | MA_API ma_uint32 ma_engine_get_sample_rate(const ma_engine* pEngine)
77985 | {
77986 |     if (pEngine == NULL) {
77987 |         return 0;
77988 |     }
77989 |
77990 |     return pEngine->sampleRate;
77991 | }
77992 |
77993 |
77994 | MA_API ma_result ma_engine_start(ma_engine* pEngine)
77995 | {
77996 |     ma_result result;
77997 |
77998 |     if (pEngine == NULL) {
77999 |         return MA_INVALID_ARGS;
78000 |     }
78001 |
78002 |     #if !defined(MA_NO_DEVICE_IO)
78003 |     {
78004 |         if (pEngine->pDevice != NULL) {
78005 |             result = ma_device_start(pEngine->pDevice);
78006 |         } else {
78007 |             result = MA_INVALID_OPERATION; /* The engine is running without a device which means
> there's no real notion of "starting" the engine. */
78008 |         }
78009 |     }
78010 |     #else
78011 |     {
78012 |         result = MA_INVALID_OPERATION; /* Device IO is disabled, so there's no real notion of
> "starting" the engine. */
78013 |     }
78014 |     #endif
78015 |
78016 |     if (result != MA_SUCCESS) {
78017 |         return result;

```

```

78018 |     }
78019 |
78020 |     return MA_SUCCESS;
78021 | }
78022 |
78023 | MA_API ma_result ma_engine_stop(ma_engine* pEngine)
78024 | {
78025 |     ma_result result;
78026 |
78027 |     if (pEngine == NULL) {
78028 |         return MA_INVALID_ARGS;
78029 |     }
78030 |
78031 |     #if !defined(MA_NO_DEVICE_IO)
78032 |     {
78033 |         if (pEngine->pDevice != NULL) {
78034 |             result = ma_device_stop(pEngine->pDevice);
78035 |         } else {
78036 |             result = MA_INVALID_OPERATION; /* The engine is running without a device which means
> there's no real notion of "stopping" the engine. */
78037 |         }
78038 |     }
78039 |     #else
78040 |     {
78041 |         result = MA_INVALID_OPERATION; /* Device IO is disabled, so there's no real notion of
> "stopping" the engine. */
78042 |     }
78043 |     #endif
78044 |
78045 |     if (result != MA_SUCCESS) {
78046 |         return result;
78047 |     }
78048 |
78049 |     return MA_SUCCESS;
78050 | }
78051 |
78052 | MA_API ma_result ma_engine_set_volume(ma_engine* pEngine, float volume)
78053 | {
78054 |     if (pEngine == NULL) {
78055 |         return MA_INVALID_ARGS;
78056 |     }
78057 |
78058 |     return ma_node_set_output_bus_volume(ma_node_graph_get_endpoint(&pEngine->nodeGraph), 0,
> volume);
78059 | }
78060 |
78061 | MA_API float ma_engine_get_volume(ma_engine* pEngine)
78062 | {
78063 |     if (pEngine == NULL) {
78064 |         return 0;
78065 |     }
78066 |
78067 |     return ma_node_get_output_bus_volume(ma_node_graph_get_endpoint(&pEngine->nodeGraph), 0);
78068 | }
78069 |
78070 | MA_API ma_result ma_engine_set_gain_db(ma_engine* pEngine, float gainDB)
78071 | {
78072 |     return ma_engine_set_volume(pEngine, ma_volume_db_to_linear(gainDB));
78073 | }
78074 |
78075 | MA_API float ma_engine_get_gain_db(ma_engine* pEngine)
78076 | {
78077 |     return ma_volume_linear_to_db(ma_engine_get_volume(pEngine));
78078 | }
78079 |
78080 |
78081 | MA_API ma_uint32 ma_engine_get_listener_count(const ma_engine* pEngine)
78082 | {
78083 |     if (pEngine == NULL) {
78084 |         return 0;
78085 |     }
78086 |
78087 |     return pEngine->listenerCount;
78088 | }
78089 |
78090 | MA_API ma_uint32 ma_engine_find_closest_listener(const ma_engine* pEngine, float absolutePosX, float
> absolutePosY, float absolutePosZ)
78091 | {
78092 |     ma_uint32 iListener;

```

```

78093 |     ma_uint32 iListenerClosest;
78094 |     float closestLen2 = MA_FLT_MAX;
78095 |
78096 |     if (pEngine == NULL || pEngine->listenerCount == 1) {
78097 |         return 0;
78098 |     }
78099 |
78100 |     iListenerClosest = 0;
78101 |     for (iListener = 0; iListener < pEngine->listenerCount; iListener += 1) {
78102 |         if (ma_engine_listener_is_enabled(pEngine, iListener)) {
78103 |             float len2 =
> ma_vec3f_len2(ma_vec3f_sub(ma_spatializer_listener_get_position(&pEngine->listeners[iListener]),
> ma_vec3f_init_3f(absolutePosX, absolutePosY, absolutePosZ)));
78104 |             if (closestLen2 > len2) {
78105 |                 closestLen2 = len2;
78106 |                 iListenerClosest = iListener;
78107 |             }
78108 |         }
78109 |     }
78110 |
78111 |     MA_ASSERT(iListenerClosest < 255);
78112 |     return iListenerClosest;
78113 | }
78114 |
78115 | MA_API void ma_engine_listener_set_position(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z)
78116 | {
78117 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78118 |         return;
78119 |     }
78120 |
78121 |     ma_spatializer_listener_set_position(&pEngine->listeners[listenerIndex], x, y, z);
78122 | }
78123 |
78124 | MA_API ma_vec3f ma_engine_listener_get_position(const ma_engine* pEngine, ma_uint32 listenerIndex)
78125 | {
78126 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78127 |         return ma_vec3f_init_3f(0, 0, 0);
78128 |     }
78129 |
78130 |     return ma_spatializer_listener_get_position(&pEngine->listeners[listenerIndex]);
78131 | }
78132 |
78133 | MA_API void ma_engine_listener_set_direction(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z)
78134 | {
78135 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78136 |         return;
78137 |     }
78138 |
78139 |     ma_spatializer_listener_set_direction(&pEngine->listeners[listenerIndex], x, y, z);
78140 | }
78141 |
78142 | MA_API ma_vec3f ma_engine_listener_get_direction(const ma_engine* pEngine, ma_uint32 listenerIndex)
78143 | {
78144 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78145 |         return ma_vec3f_init_3f(0, 0, -1);
78146 |     }
78147 |
78148 |     return ma_spatializer_listener_get_direction(&pEngine->listeners[listenerIndex]);
78149 | }
78150 |
78151 | MA_API void ma_engine_listener_set_velocity(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z)
78152 | {
78153 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78154 |         return;
78155 |     }
78156 |
78157 |     ma_spatializer_listener_set_velocity(&pEngine->listeners[listenerIndex], x, y, z);
78158 | }
78159 |
78160 | MA_API ma_vec3f ma_engine_listener_get_velocity(const ma_engine* pEngine, ma_uint32 listenerIndex)
78161 | {
78162 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78163 |         return ma_vec3f_init_3f(0, 0, 0);
78164 |     }
78165 |
78166 |     return ma_spatializer_listener_get_velocity(&pEngine->listeners[listenerIndex]);

```

```

78167 | }
78168 |
78169 | MA_API void ma_engine_listener_set_cone(ma_engine* pEngine, ma_uint32 listenerIndex, float
> innerAngleInRadians, float outerAngleInRadians, float outerGain)
78170 | {
78171 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78172 |         return;
78173 |     }
78174 |
78175 |     ma_spatializer_listener_set_cone(&pEngine->listeners[listenerIndex], innerAngleInRadians,
> outerAngleInRadians, outerGain);
78176 | }
78177 |
78178 | MA_API void ma_engine_listener_get_cone(const ma_engine* pEngine, ma_uint32 listenerIndex, float*
> pInnerAngleInRadians, float* pOuterAngleInRadians, float* pOuterGain)
78179 | {
78180 |     if (pInnerAngleInRadians != NULL) {
78181 |         *pInnerAngleInRadians = 0;
78182 |     }
78183 |
78184 |     if (pOuterAngleInRadians != NULL) {
78185 |         *pOuterAngleInRadians = 0;
78186 |     }
78187 |
78188 |     if (pOuterGain != NULL) {
78189 |         *pOuterGain = 0;
78190 |     }
78191 |
78192 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78193 |         return;
78194 |     }
78195 |
78196 |     ma_spatializer_listener_get_cone(&pEngine->listeners[listenerIndex], pInnerAngleInRadians,
> pOuterAngleInRadians, pOuterGain);
78197 | }
78198 |
78199 | MA_API void ma_engine_listener_set_world_up(ma_engine* pEngine, ma_uint32 listenerIndex, float x,
> float y, float z)
78200 | {
78201 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78202 |         return;
78203 |     }
78204 |
78205 |     ma_spatializer_listener_set_world_up(&pEngine->listeners[listenerIndex], x, y, z);
78206 | }
78207 |
78208 | MA_API ma_vec3f ma_engine_listener_get_world_up(const ma_engine* pEngine, ma_uint32 listenerIndex)
78209 | {
78210 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78211 |         return ma_vec3f_init_3f(0, 1, 0);
78212 |     }
78213 |
78214 |     return ma_spatializer_listener_get_world_up(&pEngine->listeners[listenerIndex]);
78215 | }
78216 |
78217 | MA_API void ma_engine_listener_set_enabled(ma_engine* pEngine, ma_uint32 listenerIndex, ma_bool32
> isEnabled)
78218 | {
78219 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78220 |         return;
78221 |     }
78222 |
78223 |     ma_spatializer_listener_set_enabled(&pEngine->listeners[listenerIndex], isEnabled);
78224 | }
78225 |
78226 | MA_API ma_bool32 ma_engine_listener_is_enabled(const ma_engine* pEngine, ma_uint32 listenerIndex)
78227 | {
78228 |     if (pEngine == NULL || listenerIndex >= pEngine->listenerCount) {
78229 |         return MA_FALSE;
78230 |     }
78231 |
78232 |     return ma_spatializer_listener_is_enabled(&pEngine->listeners[listenerIndex]);
78233 | }
78234 |
78235 |
78236 | #ifndef MA_NO_RESOURCE_MANAGER
78237 | MA_API ma_result ma_engine_play_sound_ex(ma_engine* pEngine, const char* pFilePath, ma_node* pNode,
> ma_uint32 nodeInputBusIndex)
78238 | {

```

```

78239 |     ma_result result = MA_SUCCESS;
78240 |     ma_sound_inlined* pSound = NULL;
78241 |     ma_sound_inlined* pNextSound = NULL;
78242 |
78243 |     if (pEngine == NULL || pFilePath == NULL) {
78244 |         return MA_INVALID_ARGS;
78245 |     }
78246 |
78247 |     /* Attach to the endpoint node if nothing is specified. */
78248 |     if (pNode == NULL) {
78249 |         pNode = ma_node_graph_get_endpoint(&pEngine->nodeGraph);
78250 |         nodeInputBusIndex = 0;
78251 |     }
78252 |
78253 |     /*
78254 |     We want to check if we can recycle an already-allocated inlined sound. Since this is just a
78255 |     helper I'm not *too* concerned about performance here and I'm happy to use a lock to keep
78256 |     the implementation simple. Maybe this can be optimized later if there's enough demand, but
78257 |     if this function is being used it probably means the caller doesn't really care too much.
78258 |
78259 |     What we do is check the atEnd flag. When this is true, we can recycle the sound. Otherwise
78260 |     we just keep iterating. If we reach the end without finding a sound to recycle we just
78261 |     allocate a new one. This doesn't scale well for a massive number of sounds being played
78262 |     simultaneously as we don't ever actually free the sound objects. Some kind of garbage
78263 |     collection routine might be valuable for this which I'll think about.
78264 |     */
78265 |     ma_spinlock_lock(&pEngine->inlinedSoundLock);
78266 |     {
78267 |         ma_uint32 soundFlags = 0;
78268 |
78269 |         for (pNextSound = pEngine->pInlinedSoundHead; pNextSound != NULL; pNextSound =
> pNextSound->pNext) {
78270 |             if (ma_sound_at_end(&pNextSound->sound)) {
78271 |                 /*
78272 |                 The sound is at the end which means it's available for recycling. All we need to do
78273 |                 is uninitialized it and reinitialize it. All we're doing is recycling memory.
78274 |                 */
78275 |                 pSound = pNextSound;
78276 |                 ma_atomic_fetch_sub_32(&pEngine->inlinedSoundCount, 1);
78277 |                 break;
78278 |             }
78279 |         }
78280 |
78281 |         if (pSound != NULL) {
78282 |             /*
78283 |             We actually want to detach the sound from the list here. The reason is because we want
> the sound
78284 |             to be in a consistent state at the non-recycled case to simplify the logic below.
78285 |             */
78286 |             if (pEngine->pInlinedSoundHead == pSound) {
78287 |                 pEngine->pInlinedSoundHead = pSound->pNext;
78288 |             }
78289 |
78290 |             if (pSound->pPrev != NULL) {
78291 |                 pSound->pPrev->pNext = pSound->pNext;
78292 |             }
78293 |             if (pSound->pNext != NULL) {
78294 |                 pSound->pNext->pPrev = pSound->pPrev;
78295 |             }
78296 |
78297 |             /* Now the previous sound needs to be uninitialized. */
78298 |             ma_sound_uninit(&pNextSound->sound);
78299 |         } else {
78300 |             /* No sound available for recycling. Allocate one now. */
78301 |             pSound = (ma_sound_inlined*)ma_malloc(sizeof(*pSound), &pEngine->allocationCallbacks);
78302 |         }
78303 |
78304 |         if (pSound != NULL) { /* Safety check for the allocation above. */
78305 |             /*
78306 |             At this point we should have memory allocated for the inlined sound. We just need
78307 |             to initialize it like a normal sound now.
78308 |             */
78309 |             soundFlags |= MA_SOUND_FLAG_ASYNC; /* For inlined sounds we don't want
> to be sitting around waiting for stuff to load so force an async load. */
78310 |             soundFlags |= MA_SOUND_FLAG_NO_DEFAULT_ATTACHMENT; /* We want specific control over
> where the sound is attached in the graph. We'll attach it manually just before playing the sound. */
78311 |             soundFlags |= MA_SOUND_FLAG_NO_PITCH; /* Pitching isn't usable with inlined
> sounds, so disable it to save on speed. */
78312 |             soundFlags |= MA_SOUND_FLAG_NO_SPATIALIZATION; /* Not currently doing spatialization

```

```

> with inlined sounds, but this might actually change later. For now disable spatialization. Will be
> removed if we ever add support for spatialization here. */
78313 |
78314 |         result = ma_sound_init_from_file(pEngine, pFilePath, soundFlags, NULL, NULL,
> &pSound->sound);
78315 |         if (result == MA_SUCCESS) {
78316 |             /* Now attach the sound to the graph. */
78317 |             result = ma_node_attach_output_bus(pSound, 0, pNode, nodeInputBusIndex);
78318 |             if (result == MA_SUCCESS) {
78319 |                 /* At this point the sound should be loaded and we can go ahead and add it to
> the list. The new item becomes the new head. */
78320 |                 pSound->pNext = pEngine->pInlinedSoundHead;
78321 |                 pSound->pPrev = NULL;
78322 |
78323 |                 pEngine->pInlinedSoundHead = pSound;    /* <-- This is what attaches the sound
> to the list. */
78324 |                 if (pSound->pNext != NULL) {
78325 |                     pSound->pNext->pPrev = pSound;
78326 |                 }
78327 |             } else {
78328 |                 ma_free(pSound, &pEngine->allocationCallbacks);
78329 |             }
78330 |         } else {
78331 |             ma_free(pSound, &pEngine->allocationCallbacks);
78332 |         }
78333 |     } else {
78334 |         result = MA_OUT_OF_MEMORY;
78335 |     }
78336 | }
78337 | ma_spinlock_unlock(&pEngine->inlinedSoundLock);
78338 |
78339 | if (result != MA_SUCCESS) {
78340 |     return result;
78341 | }
78342 |
78343 | /* Finally we can start playing the sound. */
78344 | result = ma_sound_start(&pSound->sound);
78345 | if (result != MA_SUCCESS) {
78346 |     /* Failed to start the sound. We need to mark it for recycling and return an error. */
78347 |     ma_atomic_exchange_32(&pSound->sound.atEnd, MA_TRUE);
78348 |     return result;
78349 | }
78350 |
78351 | ma_atomic_fetch_add_32(&pEngine->inlinedSoundCount, 1);
78352 | return result;
78353 | }
78354 |
78355 | MA_API ma_result ma_engine_play_sound(ma_engine* pEngine, const char* pFilePath, ma_sound_group*
> pGroup)
78356 | {
78357 |     return ma_engine_play_sound_ex(pEngine, pFilePath, pGroup, 0);
78358 | }
78359 | #endif
78360 |
78361 |
78362 | static ma_result ma_sound_preinit(ma_engine* pEngine, ma_sound* pSound)
78363 | {
78364 |     if (pSound == NULL) {
78365 |         return MA_INVALID_ARGS;
78366 |     }
78367 |
78368 |     MA_ZERO_OBJECT(pSound);
78369 |     pSound->seekTarget = MA_SEEK_TARGET_NONE;
78370 |
78371 |     if (pEngine == NULL) {
78372 |         return MA_INVALID_ARGS;
78373 |     }
78374 |
78375 |     return MA_SUCCESS;
78376 | }
78377 |
78378 | static ma_result ma_sound_init_from_data_source_internal(ma_engine* pEngine, const ma_sound_config*
> pConfig, ma_sound* pSound)
78379 | {
78380 |     ma_result result;
78381 |     ma_engine_node_config engineNodeConfig;
78382 |     ma_engine_node_type type;    /* Will be set to ma_engine_node_type_group if no data source is
> specified. */
78383 |

```

```

78384 | /* Do not clear pSound to zero here - that's done at a higher level with ma_sound_preinit(). */
78385 | MA_ASSERT(pEngine != NULL);
78386 | MA_ASSERT(pSound != NULL);
78387 |
78388 | if (pConfig == NULL) {
78389 |     return MA_INVALID_ARGS;
78390 | }
78391 |
78392 | pSound->pDataSource = pConfig->pDataSource;
78393 |
78394 | if (pConfig->pDataSource != NULL) {
78395 |     type = ma_engine_node_type_sound;
78396 | } else {
78397 |     type = ma_engine_node_type_group;
78398 | }
78399 |
78400 | /*
78401 | Sounds are engine nodes. Before we can initialize this we need to determine the channel count.
78402 | If we can't do this we need to abort. It's up to the caller to ensure they're using a data
78403 | source that provides this information upfront.
78404 | */
78405 | engineNodeConfig = ma_engine_node_config_init(pEngine, type, pConfig->flags);
78406 | engineNodeConfig.channelsIn = pConfig->channelsIn;
78407 | engineNodeConfig.channelsOut = pConfig->channelsOut;
78408 | engineNodeConfig.volumeSmoothTimeInPCMFrames = pConfig->volumeSmoothTimeInPCMFrames;
78409 | engineNodeConfig.monoExpansionMode = pConfig->monoExpansionMode;
78410 |
78411 | if (engineNodeConfig.volumeSmoothTimeInPCMFrames == 0) {
78412 |     engineNodeConfig.volumeSmoothTimeInPCMFrames = pEngine->defaultVolumeSmoothTimeInPCMFrames;
78413 | }
78414 |
78415 | /* If we're loading from a data source the input channel count needs to be the data source's
> native channel count. */
78416 | if (pConfig->pDataSource != NULL) {
78417 |     result = ma_data_source_get_data_format(pConfig->pDataSource, NULL,
> &engineNodeConfig.channelsIn, &engineNodeConfig.sampleRate, NULL, 0);
78418 |     if (result != MA_SUCCESS) {
78419 |         return result; /* Failed to retrieve the channel count. */
78420 |     }
78421 |
78422 |     if (engineNodeConfig.channelsIn == 0) {
78423 |         return MA_INVALID_OPERATION; /* Invalid channel count. */
78424 |     }
78425 |
78426 |     if (engineNodeConfig.channelsOut == MA_SOUND_SOURCE_CHANNEL_COUNT) {
78427 |         engineNodeConfig.channelsOut = engineNodeConfig.channelsIn;
78428 |     }
78429 | }
78430 |
78431 |
78432 | /* Getting here means we should have a valid channel count and we can initialize the engine
> node. */
78433 | result = ma_engine_node_init(&engineNodeConfig, &pEngine->allocationCallbacks,
> &pSound->engineNode);
78434 | if (result != MA_SUCCESS) {
78435 |     return result;
78436 | }
78437 |
78438 | /* If no attachment is specified, attach the sound straight to the endpoint. */
78439 | if (pConfig->pInitialAttachment == NULL) {
78440 |     /* No group. Attach straight to the endpoint by default, unless the caller has requested
> that it not. */
78441 |     if ((pConfig->flags & MA_SOUND_FLAG_NO_DEFAULT_ATTACHMENT) == 0) {
78442 |         result = ma_node_attach_output_bus(pSound, 0,
> ma_node_graph_get_endpoint(&pEngine->nodeGraph), 0);
78443 |     }
78444 |     } else {
78445 |         /* An attachment is specified. Attach to it by default. The sound has only a single output
> bus, and the config will specify which input bus to attach to. */
78446 |         result = ma_node_attach_output_bus(pSound, 0, pConfig->pInitialAttachment,
> pConfig->initialAttachmentInputBusIndex);
78447 |     }
78448 |
78449 |     if (result != MA_SUCCESS) {
78450 |         ma_engine_node_uninit(&pSound->engineNode, &pEngine->allocationCallbacks);
78451 |         return result;
78452 |     }
78453 |
78454 |

```



```

78455 |     /*
78456 |     When pulling data from a data source we need a processing cache to hold onto unprocessed input
> data from the data source
78457 |     after doing resampling.
78458 |     */
78459 |     if (pSound->pDataSource != NULL) {
78460 |         pSound->processingCacheFramesRemaining = 0;
78461 |         pSound->processingCacheCap =
> ma_node_graph_get_processing_size_in_frames(&pEngine->nodeGraph);
78462 |         if (pSound->processingCacheCap == 0) {
78463 |             pSound->processingCacheCap = 512;
78464 |         }
78465 |
78466 |         pSound->pProcessingCache = (float*)ma_malloc(pSound->processingCacheCap *
> ma_get_bytes_per_frame(ma_format_f32, engineNodeConfig.channelsIn), &pEngine->allocationCallbacks);
78467 |         if (pSound->pProcessingCache == NULL) {
78468 |             ma_engine_node_uninit(&pSound->engineNode, &pEngine->allocationCallbacks);
78469 |             return MA_OUT_OF_MEMORY;
78470 |         }
78471 |     }
78472 |
78473 |
78474 |     /* Apply initial range and looping state to the data source if applicable. */
78475 |     if (pConfig->rangeBegInPCMFrames != 0 || pConfig->rangeEndInPCMFrames != ~((ma_uint64)0)) {
78476 |         ma_data_source_set_range_in_pcm_frames(ma_sound_get_data_source(pSound),
> pConfig->rangeBegInPCMFrames, pConfig->rangeEndInPCMFrames);
78477 |     }
78478 |
78479 |     if (pConfig->loopPointBegInPCMFrames != 0 || pConfig->loopPointEndInPCMFrames !=
> ~((ma_uint64)0)) {
78480 |         ma_data_source_set_loop_point_in_pcm_frames(ma_sound_get_data_source(pSound),
> pConfig->loopPointBegInPCMFrames, pConfig->loopPointEndInPCMFrames);
78481 |     }
78482 |
78483 |     ma_sound_set_looping(pSound, pConfig->isLooping || ((pConfig->flags & MA_SOUND_FLAG_LOOPING) !=
> 0));
78484 |
78485 |     return MA_SUCCESS;
78486 | }
78487 |
78488 | #ifndef MA_NO_RESOURCE_MANAGER
78489 | MA_API ma_result ma_sound_init_from_file_internal(ma_engine* pEngine, const ma_sound_config*
> pConfig, ma_sound* pSound)
78490 | {
78491 |     ma_result result = MA_SUCCESS;
78492 |     ma_uint32 flags;
78493 |     ma_sound_config config;
78494 |     ma_resource_manager_pipeline_notifications notifications;
78495 |
78496 |     /*
78497 |     The engine requires knowledge of the channel count of the underlying data source before it can
78498 |     initialize the sound. Therefore, we need to make the resource manager wait until initialization
78499 |     of the underlying data source to be initialized so we can get access to the channel count. To
78500 |     do this, the MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT is forced.
78501 |
78502 |     Because we're initializing the data source before the sound, there's a chance the notification
78503 |     will get triggered before this function returns. This is OK, so long as the caller is aware of
78504 |     it and can avoid accessing the sound from within the notification.
78505 |     */
78506 |     flags = pConfig->flags | MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_WAIT_INIT;
78507 |     if (pConfig->isLooping) {
78508 |         flags |= MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING;
78509 |     }
78510 |
78511 |     pSound->pResourceManagerDataSource =
> (ma_resource_manager_data_source*)ma_malloc(sizeof(*pSound->pResourceManagerDataSource),
> &pEngine->allocationCallbacks);
78512 |     if (pSound->pResourceManagerDataSource == NULL) {
78513 |         return MA_OUT_OF_MEMORY;
78514 |     }
78515 |
78516 |     /* Removed in 0.12. Set pDoneFence on the notifications. */
78517 |     notifications = pConfig->initNotifications;
78518 |     if (pConfig->pDoneFence != NULL && notifications.done.pFence == NULL) {
78519 |         notifications.done.pFence = pConfig->pDoneFence;
78520 |     }
78521 |
78522 |     /*
78523 |     We must wrap everything around the fence if one was specified. This ensures ma_fence_wait() does

```

```

78524 |     not return prematurely before the sound has finished initializing.
78525 |     */
78526 |     if (notifications.done.pFence) { ma_fence_acquire(notifications.done.pFence); }
78527 |     {
78528 |         ma_resource_manager_data_source_config resourceManagerDataSourceConfig =
> ma_resource_manager_data_source_config_init();
78529 |         resourceManagerDataSourceConfig.pFilePath          = pConfig->pFilePath;
78530 |         resourceManagerDataSourceConfig.pFilePathW         = pConfig->pFilePathW;
78531 |         resourceManagerDataSourceConfig.flags              = flags;
78532 |         resourceManagerDataSourceConfig.pNotifications     = &notifications;
78533 |         resourceManagerDataSourceConfig.initialSeekPointInPCMFrames =
> pConfig->initialSeekPointInPCMFrames;
78534 |         resourceManagerDataSourceConfig.rangeBegInPCMFrames = pConfig->rangeBegInPCMFrames;
78535 |         resourceManagerDataSourceConfig.rangeEndInPCMFrames = pConfig->rangeEndInPCMFrames;
78536 |         resourceManagerDataSourceConfig.loopPointBegInPCMFrames =
> pConfig->loopPointBegInPCMFrames;
78537 |         resourceManagerDataSourceConfig.loopPointEndInPCMFrames =
> pConfig->loopPointEndInPCMFrames;
78538 |         resourceManagerDataSourceConfig.isLooping           = (flags &
> MA_RESOURCE_MANAGER_DATA_SOURCE_FLAG_LOOPING) != 0;
78539 |
78540 |         result = ma_resource_manager_data_source_init_ex(pEngine->pResourceManager,
> &resourceManagerDataSourceConfig, pSound->pResourceManagerDataSource);
78541 |         if (result != MA_SUCCESS) {
78542 |             ma_free(pSound->pResourceManagerDataSource, &pEngine->allocationCallbacks);
78543 |             goto done;
78544 |         }
78545 |
78546 |         pSound->ownsDataSource = MA_TRUE; /* <-- Important. Not setting this will result in the
> resource manager data source never getting uninitialized. */
78547 |
78548 |         /* We need to use a slightly customized version of the config so we'll need to make a copy.
> */
78549 |         config = *pConfig;
78550 |         config.pFilePath = NULL;
78551 |         config.pFilePathW = NULL;
78552 |         config.pDataSource = pSound->pResourceManagerDataSource;
78553 |
78554 |         result = ma_sound_init_from_data_source_internal(pEngine, &config, pSound);
78555 |         if (result != MA_SUCCESS) {
78556 |             ma_resource_manager_data_source_uninit(pSound->pResourceManagerDataSource);
78557 |             ma_free(pSound->pResourceManagerDataSource, &pEngine->allocationCallbacks);
78558 |             MA_ZERO_OBJECT(pSound);
78559 |             goto done;
78560 |         }
78561 |     }
78562 | done:
78563 |     if (notifications.done.pFence) { ma_fence_release(notifications.done.pFence); }
78564 |     return result;
78565 | }
78566 |
78567 | MA_API ma_result ma_sound_init_from_file(ma_engine* pEngine, const char* pFilePath, ma_uint32 flags,
> ma_sound_group* pGroup, ma_fence* pDoneFence, ma_sound* pSound)
78568 | {
78569 |     ma_sound_config config;
78570 |
78571 |     if (pFilePath == NULL) {
78572 |         return MA_INVALID_ARGS;
78573 |     }
78574 |
78575 |     config = ma_sound_config_init_2(pEngine);
78576 |     config.pFilePath = pFilePath;
78577 |     config.flags = flags;
78578 |     config.pInitialAttachment = pGroup;
78579 |     config.pDoneFence = pDoneFence;
78580 |
78581 |     return ma_sound_init_ex(pEngine, &config, pSound);
78582 | }
78583 |
78584 | MA_API ma_result ma_sound_init_from_file_w(ma_engine* pEngine, const wchar_t* pFilePath, ma_uint32
> flags, ma_sound_group* pGroup, ma_fence* pDoneFence, ma_sound* pSound)
78585 | {
78586 |     ma_sound_config config;
78587 |
78588 |     if (pFilePath == NULL) {
78589 |         return MA_INVALID_ARGS;
78590 |     }
78591 |
78592 |     config = ma_sound_config_init_2(pEngine);

```

```

78593 |     config.pFilePathW      = pFilePath;
78594 |     config.flags           = flags;
78595 |     config.pInitialAttachment = pGroup;
78596 |     config.pDoneFence      = pDoneFence;
78597 |
78598 |     return ma_sound_init_ex(pEngine, &config, pSound);
78599 | }
78600 |
78601 | MA_API ma_result ma_sound_init_copy(ma_engine* pEngine, const ma_sound* pExistingSound, ma_uint32
> flags, ma_sound_group* pGroup, ma_sound* pSound)
78602 | {
78603 |     ma_result result;
78604 |     ma_sound_config config;
78605 |
78606 |     result = ma_sound_preinit(pEngine, pSound);
78607 |     if (result != MA_SUCCESS) {
78608 |         return result;
78609 |     }
78610 |
78611 |     if (pExistingSound == NULL) {
78612 |         return MA_INVALID_ARGS;
78613 |     }
78614 |
78615 |     /* Cloning only works for data buffers (not streams) that are loaded from the resource manager.
> */
78616 |     if (pExistingSound->pResourceManagerDataSource == NULL) {
78617 |         return MA_INVALID_OPERATION;
78618 |     }
78619 |
78620 |     /*
78621 |     We need to make a clone of the data source. If the data source is not a data buffer (i.e. a
> stream)
78622 |     this will fail.
78623 |     */
78624 |     pSound->pResourceManagerDataSource =
> (ma_resource_manager_data_source*)ma_malloc(sizeof(*pSound->pResourceManagerDataSource),
> &pEngine->allocationCallbacks);
78625 |     if (pSound->pResourceManagerDataSource == NULL) {
78626 |         return MA_OUT_OF_MEMORY;
78627 |     }
78628 |
78629 |     result = ma_resource_manager_data_source_init_copy(pEngine->pResourceManager,
> pExistingSound->pResourceManagerDataSource, pSound->pResourceManagerDataSource);
78630 |     if (result != MA_SUCCESS) {
78631 |         ma_free(pSound->pResourceManagerDataSource, &pEngine->allocationCallbacks);
78632 |         return result;
78633 |     }
78634 |
78635 |     config = ma_sound_config_init_2(pEngine);
78636 |     config.pDataSource      = pSound->pResourceManagerDataSource;
78637 |     config.flags            = flags;
78638 |     config.pInitialAttachment = pGroup;
78639 |     config.monoExpansionMode = pExistingSound->engineNode.monoExpansionMode;
78640 |     config.volumeSmoothTimeInPCMFrames = pExistingSound->engineNode.volumeSmoothTimeInPCMFrames;
78641 |
78642 |     result = ma_sound_init_from_data_source_internal(pEngine, &config, pSound);
78643 |     if (result != MA_SUCCESS) {
78644 |         ma_resource_manager_data_source_uninit(pSound->pResourceManagerDataSource);
78645 |         ma_free(pSound->pResourceManagerDataSource, &pEngine->allocationCallbacks);
78646 |         MA_ZERO_OBJECT(pSound);
78647 |         return result;
78648 |     }
78649 |
78650 |     /* Make sure the sound is marked as the owner of the data source or else it will never get
> uninitialized. */
78651 |     pSound->ownsDataSource = MA_TRUE;
78652 |
78653 |     return MA_SUCCESS;
78654 | }
78655 | #endif
78656 |
78657 | MA_API ma_result ma_sound_init_from_data_source(ma_engine* pEngine, ma_data_source* pDataSource,
> ma_uint32 flags, ma_sound_group* pGroup, ma_sound* pSound)
78658 | {
78659 |     ma_sound_config config = ma_sound_config_init_2(pEngine);
78660 |     config.pDataSource      = pDataSource;
78661 |     config.flags            = flags;
78662 |     config.pInitialAttachment = pGroup;
78663 |     return ma_sound_init_ex(pEngine, &config, pSound);

```

```

78664 | }
78665 |
78666 | MA_API ma_result ma_sound_init_ex(ma_engine* pEngine, const ma_sound_config* pConfig, ma_sound*
> pSound)
78667 | {
78668 |     ma_result result;
78669 |
78670 |     result = ma_sound_preinit(pEngine, pSound);
78671 |     if (result != MA_SUCCESS) {
78672 |         return result;
78673 |     }
78674 |
78675 |     if (pConfig == NULL) {
78676 |         return MA_INVALID_ARGS;
78677 |     }
78678 |
78679 |     pSound->endCallback = pConfig->endCallback;
78680 |     pSound->pEndCallbackUserData = pConfig->pEndCallbackUserData;
78681 |
78682 |     /* We need to load the sound differently depending on whether or not we're loading from a file.
> */
78683 | #ifndef MA_NO_RESOURCE_MANAGER
78684 |     if (pConfig->pFilePath != NULL || pConfig->pFilePathW != NULL) {
78685 |         return ma_sound_init_from_file_internal(pEngine, pConfig, pSound);
78686 |     } else
78687 | #endif
78688 |     {
78689 |         /*
78690 |         Getting here means we're not loading from a file. We may be loading from an
> already-initialized
78691 |         data source, or none at all. If we aren't specifying any data source, we'll be initializing
78692 |         the equivalent to a group. ma_data_source_init_from_data_source_internal() will deal with
> this
78693 |         for us, so no special treatment required here.
78694 |         */
78695 |         return ma_sound_init_from_data_source_internal(pEngine, pConfig, pSound);
78696 |     }
78697 | }
78698 |
78699 | MA_API void ma_sound_uninit(ma_sound* pSound)
78700 | {
78701 |     if (pSound == NULL) {
78702 |         return;
78703 |     }
78704 |
78705 |     /*
78706 |     Always uninitialize the node first. This ensures it's detached from the graph and does not
> return until it has done
78707 |     so which makes thread safety beyond this point trivial.
78708 |     */
78709 |     ma_engine_node_uninit(&pSound->engineNode, &pSound->engineNode.pEngine->allocationCallbacks);
78710 |
78711 |     if (pSound->pProcessingCache != NULL) {
78712 |         ma_free(pSound->pProcessingCache, &pSound->engineNode.pEngine->allocationCallbacks);
78713 |         pSound->pProcessingCache = NULL;
78714 |     }
78715 |
78716 |     /* Once the sound is detached from the group we can guarantee that it won't be referenced by the
> mixer thread which means it's safe for us to destroy the data source. */
78717 | #ifndef MA_NO_RESOURCE_MANAGER
78718 |     if (pSound->ownsDataSource) {
78719 |         ma_resource_manager_data_source_uninit(pSound->pResourceManagerDataSource);
78720 |         ma_free(pSound->pResourceManagerDataSource,
> &pSound->engineNode.pEngine->allocationCallbacks);
78721 |         pSound->pDataSource = NULL;
78722 |     }
78723 | #else
78724 |     MA_ASSERT(pSound->ownsDataSource == MA_FALSE);
78725 | #endif
78726 | }
78727 |
78728 | MA_API ma_engine* ma_sound_get_engine(const ma_sound* pSound)
78729 | {
78730 |     if (pSound == NULL) {
78731 |         return NULL;
78732 |     }
78733 |
78734 |     return pSound->engineNode.pEngine;
78735 | }

```

```

78736 |
78737 | MA_API ma_data_source* ma_sound_get_data_source(const ma_sound* pSound)
78738 | {
78739 |     if (pSound == NULL) {
78740 |         return NULL;
78741 |     }
78742 |
78743 |     return pSound->pDataSource;
78744 | }
78745 |
78746 | MA_API ma_result ma_sound_start(ma_sound* pSound)
78747 | {
78748 |     if (pSound == NULL) {
78749 |         return MA_INVALID_ARGS;
78750 |     }
78751 |
78752 |     /* If the sound is already playing, do nothing. */
78753 |     if (ma_sound_is_playing(pSound)) {
78754 |         return MA_SUCCESS;
78755 |     }
78756 |
78757 |     /* If the sound is at the end it means we want to start from the start again. */
78758 |     if (ma_sound_at_end(pSound)) {
78759 |         ma_result result = ma_data_source_seek_to_pcm_frame(pSound->pDataSource, 0);
78760 |         if (result != MA_SUCCESS && result != MA_NOT_IMPLEMENTED) {
78761 |             return result; /* Failed to seek back to the start. */
78762 |         }
78763 |
78764 |         /* Make sure we clear the end indicator. */
78765 |         ma_atomic_exchange_32(&pSound->atEnd, MA_FALSE);
78766 |     }
78767 |
78768 |     /* Make sure the sound is started. If there's a start delay, the sound won't actually start
> until the start time is reached. */
78769 |     ma_node_set_state(pSound, ma_node_state_started);
78770 |
78771 |     return MA_SUCCESS;
78772 | }
78773 |
78774 | MA_API ma_result ma_sound_stop(ma_sound* pSound)
78775 | {
78776 |     if (pSound == NULL) {
78777 |         return MA_INVALID_ARGS;
78778 |     }
78779 |
78780 |     /* This will stop the sound immediately. Use ma_sound_set_stop_time() to stop the sound at a
> specific time. */
78781 |     ma_node_set_state(pSound, ma_node_state_stopped);
78782 |
78783 |     return MA_SUCCESS;
78784 | }
78785 |
78786 | MA_API ma_result ma_sound_stop_with_fade_in_pcm_frames(ma_sound* pSound, ma_uint64
> fadeLengthInFrames)
78787 | {
78788 |     if (pSound == NULL) {
78789 |         return MA_INVALID_ARGS;
78790 |     }
78791 |
78792 |     /* Stopping with a fade out requires us to schedule the stop into the future by the fade length.
> */
78793 |     ma_sound_set_stop_time_with_fade_in_pcm_frames(pSound,
> ma_engine_get_time_in_pcm_frames(ma_sound_get_engine(pSound)) + fadeLengthInFrames,
> fadeLengthInFrames);
78794 |
78795 |     return MA_SUCCESS;
78796 | }
78797 |
78798 | MA_API ma_result ma_sound_stop_with_fade_in_milliseconds(ma_sound* pSound, ma_uint64
> fadeLengthInMilliseconds)
78799 | {
78800 |     ma_uint64 sampleRate;
78801 |
78802 |     if (pSound == NULL) {
78803 |         return MA_INVALID_ARGS;
78804 |     }
78805 |
78806 |     sampleRate = ma_engine_get_sample_rate(ma_sound_get_engine(pSound));
78807 |

```

```

78808 |     return ma_sound_stop_with_fade_in_pcm_frames(pSound, (fadeLengthInMilliseconds * sampleRate) /
> 1000);
78809 | }
78810 |
78811 | MA_API void ma_sound_reset_start_time(ma_sound* pSound)
78812 | {
78813 |     ma_sound_set_start_time_in_pcm_frames(pSound, 0);
78814 | }
78815 |
78816 | MA_API void ma_sound_reset_stop_time(ma_sound* pSound)
78817 | {
78818 |     ma_sound_set_stop_time_in_pcm_frames(pSound, ~(ma_uint64)0);
78819 | }
78820 |
78821 | MA_API void ma_sound_reset_fade(ma_sound* pSound)
78822 | {
78823 |     ma_sound_set_fade_in_pcm_frames(pSound, 0, 1, 0);
78824 | }
78825 |
78826 | MA_API void ma_sound_reset_stop_time_and_fade(ma_sound* pSound)
78827 | {
78828 |     ma_sound_reset_stop_time(pSound);
78829 |     ma_sound_reset_fade(pSound);
78830 | }
78831 |
78832 | MA_API void ma_sound_set_volume(ma_sound* pSound, float volume)
78833 | {
78834 |     if (pSound == NULL) {
78835 |         return;
78836 |     }
78837 |
78838 |     ma_engine_node_set_volume(&pSound->engineNode, volume);
78839 | }
78840 |
78841 | MA_API float ma_sound_get_volume(const ma_sound* pSound)
78842 | {
78843 |     float volume = 0;
78844 |
78845 |     if (pSound == NULL) {
78846 |         return 0;
78847 |     }
78848 |
78849 |     ma_engine_node_get_volume(&pSound->engineNode, &volume);
78850 |
78851 |     return volume;
78852 | }
78853 |
78854 | MA_API void ma_sound_set_pan(ma_sound* pSound, float pan)
78855 | {
78856 |     if (pSound == NULL) {
78857 |         return;
78858 |     }
78859 |
78860 |     ma_panner_set_pan(&pSound->engineNode.panner, pan);
78861 | }
78862 |
78863 | MA_API float ma_sound_get_pan(const ma_sound* pSound)
78864 | {
78865 |     if (pSound == NULL) {
78866 |         return 0;
78867 |     }
78868 |
78869 |     return ma_panner_get_pan(&pSound->engineNode.panner);
78870 | }
78871 |
78872 | MA_API void ma_sound_set_pan_mode(ma_sound* pSound, ma_pan_mode panMode)
78873 | {
78874 |     if (pSound == NULL) {
78875 |         return;
78876 |     }
78877 |
78878 |     ma_panner_set_mode(&pSound->engineNode.panner, panMode);
78879 | }
78880 |
78881 | MA_API ma_pan_mode ma_sound_get_pan_mode(const ma_sound* pSound)
78882 | {
78883 |     if (pSound == NULL) {
78884 |         return ma_pan_mode_balance;
78885 |     }

```

```

78886 |
78887 |     return ma_panner_get_mode(&pSound->engineNode.panner);
78888 | }
78889 |
78890 | MA_API void ma_sound_set_pitch(ma_sound* pSound, float pitch)
78891 | {
78892 |     if (pSound == NULL) {
78893 |         return;
78894 |     }
78895 |
78896 |     if (pitch <= 0) {
78897 |         return;
78898 |     }
78899 |
78900 |     ma_atomic_exchange_explicit_f32(&pSound->engineNode.pitch, pitch,
> ma_atomic_memory_order_release);
78901 | }
78902 |
78903 | MA_API float ma_sound_get_pitch(const ma_sound* pSound)
78904 | {
78905 |     if (pSound == NULL) {
78906 |         return 0;
78907 |     }
78908 |
78909 |     return ma_atomic_load_f32(&pSound->engineNode.pitch);    /* Naughty const-cast for this. */
78910 | }
78911 |
78912 | MA_API void ma_sound_set_spatialization_enabled(ma_sound* pSound, ma_bool32 enabled)
78913 | {
78914 |     if (pSound == NULL) {
78915 |         return;
78916 |     }
78917 |
78918 |     ma_atomic_exchange_explicit_32(&pSound->engineNode.isSpatializationDisabled, !enabled,
> ma_atomic_memory_order_release);
78919 | }
78920 |
78921 | MA_API ma_bool32 ma_sound_is_spatialization_enabled(const ma_sound* pSound)
78922 | {
78923 |     if (pSound == NULL) {
78924 |         return MA_FALSE;
78925 |     }
78926 |
78927 |     return ma_engine_node_is_spatialization_enabled(&pSound->engineNode);
78928 | }
78929 |
78930 | MA_API void ma_sound_set_pinned_listener_index(ma_sound* pSound, ma_uint32 listenerIndex)
78931 | {
78932 |     if (pSound == NULL || listenerIndex >=
> ma_engine_get_listener_count(ma_sound_get_engine(pSound))) {
78933 |         return;
78934 |     }
78935 |
78936 |     ma_atomic_exchange_explicit_32(&pSound->engineNode.pinnedListenerIndex, listenerIndex,
> ma_atomic_memory_order_release);
78937 | }
78938 |
78939 | MA_API ma_uint32 ma_sound_get_pinned_listener_index(const ma_sound* pSound)
78940 | {
78941 |     if (pSound == NULL) {
78942 |         return MA_LISTENER_INDEX_CLOSEST;
78943 |     }
78944 |
78945 |     return ma_atomic_load_explicit_32(&pSound->engineNode.pinnedListenerIndex,
> ma_atomic_memory_order_acquire);
78946 | }
78947 |
78948 | MA_API ma_uint32 ma_sound_get_listener_index(const ma_sound* pSound)
78949 | {
78950 |     ma_uint32 listenerIndex;
78951 |
78952 |     if (pSound == NULL) {
78953 |         return 0;
78954 |     }
78955 |
78956 |     listenerIndex = ma_sound_get_pinned_listener_index(pSound);
78957 |     if (listenerIndex == MA_LISTENER_INDEX_CLOSEST) {
78958 |         ma_vec3f position = ma_sound_get_position(pSound);
78959 |         return ma_engine_find_closest_listener(ma_sound_get_engine(pSound), position.x, position.y,

```



```
> position.z);
78960 |     }
78961 |
78962 |     return listenerIndex;
78963 | }
78964 |
78965 | MA_API ma_vec3f ma_sound_get_direction_to_listener(const ma_sound* pSound)
78966 | {
78967 |     ma_vec3f relativePos;
78968 |     ma_engine* pEngine;
78969 |
78970 |     if (pSound == NULL) {
78971 |         return ma_vec3f_init_3f(0, 0, -1);
78972 |     }
78973 |
78974 |     pEngine = ma_sound_get_engine(pSound);
78975 |     if (pEngine == NULL) {
78976 |         return ma_vec3f_init_3f(0, 0, -1);
78977 |     }
78978 |
78979 |     ma_spatializer_get_relative_position_and_direction(&pSound->engineNode.spatializer,
> &pEngine->listeners[ma_sound_get_listener_index(pSound)], &relativePos, NULL);
78980 |
78981 |     return ma_vec3f_normalize(ma_vec3f_neg(relativePos));
78982 | }
78983 |
78984 | MA_API void ma_sound_set_position(ma_sound* pSound, float x, float y, float z)
78985 | {
78986 |     if (pSound == NULL) {
78987 |         return;
78988 |     }
78989 |
78990 |     ma_spatializer_set_position(&pSound->engineNode.spatializer, x, y, z);
78991 | }
78992 |
78993 | MA_API ma_vec3f ma_sound_get_position(const ma_sound* pSound)
78994 | {
78995 |     if (pSound == NULL) {
78996 |         return ma_vec3f_init_3f(0, 0, 0);
78997 |     }
78998 |
78999 |     return ma_spatializer_get_position(&pSound->engineNode.spatializer);
79000 | }
79001 |
79002 | MA_API void ma_sound_set_direction(ma_sound* pSound, float x, float y, float z)
79003 | {
79004 |     if (pSound == NULL) {
79005 |         return;
79006 |     }
79007 |
79008 |     ma_spatializer_set_direction(&pSound->engineNode.spatializer, x, y, z);
79009 | }
79010 |
79011 | MA_API ma_vec3f ma_sound_get_direction(const ma_sound* pSound)
79012 | {
79013 |     if (pSound == NULL) {
79014 |         return ma_vec3f_init_3f(0, 0, 0);
79015 |     }
79016 |
79017 |     return ma_spatializer_get_direction(&pSound->engineNode.spatializer);
79018 | }
79019 |
79020 | MA_API void ma_sound_set_velocity(ma_sound* pSound, float x, float y, float z)
79021 | {
79022 |     if (pSound == NULL) {
79023 |         return;
79024 |     }
79025 |
79026 |     ma_spatializer_set_velocity(&pSound->engineNode.spatializer, x, y, z);
79027 | }
79028 |
79029 | MA_API ma_vec3f ma_sound_get_velocity(const ma_sound* pSound)
79030 | {
79031 |     if (pSound == NULL) {
79032 |         return ma_vec3f_init_3f(0, 0, 0);
79033 |     }
79034 |
79035 |     return ma_spatializer_get_velocity(&pSound->engineNode.spatializer);
79036 | }
```

```
79037 |
79038 | MA_API void ma_sound_set_attenuation_model(ma_sound* pSound, ma_attenuation_model attenuationModel)
79039 | {
79040 |     if (pSound == NULL) {
79041 |         return;
79042 |     }
79043 |
79044 |     ma_spatializer_set_attenuation_model(&pSound->engineNode.spatializer, attenuationModel);
79045 | }
79046 |
79047 | MA_API ma_attenuation_model ma_sound_get_attenuation_model(const ma_sound* pSound)
79048 | {
79049 |     if (pSound == NULL) {
79050 |         return ma_attenuation_model_none;
79051 |     }
79052 |
79053 |     return ma_spatializer_get_attenuation_model(&pSound->engineNode.spatializer);
79054 | }
79055 |
79056 | MA_API void ma_sound_set_positioning(ma_sound* pSound, ma_positioning positioning)
79057 | {
79058 |     if (pSound == NULL) {
79059 |         return;
79060 |     }
79061 |
79062 |     ma_spatializer_set_positioning(&pSound->engineNode.spatializer, positioning);
79063 | }
79064 |
79065 | MA_API ma_positioning ma_sound_get_positioning(const ma_sound* pSound)
79066 | {
79067 |     if (pSound == NULL) {
79068 |         return ma_positioning_absolute;
79069 |     }
79070 |
79071 |     return ma_spatializer_get_positioning(&pSound->engineNode.spatializer);
79072 | }
79073 |
79074 | MA_API void ma_sound_set_rolloff(ma_sound* pSound, float rolloff)
79075 | {
79076 |     if (pSound == NULL) {
79077 |         return;
79078 |     }
79079 |
79080 |     ma_spatializer_set_rolloff(&pSound->engineNode.spatializer, rolloff);
79081 | }
79082 |
79083 | MA_API float ma_sound_get_rolloff(const ma_sound* pSound)
79084 | {
79085 |     if (pSound == NULL) {
79086 |         return 0;
79087 |     }
79088 |
79089 |     return ma_spatializer_get_rolloff(&pSound->engineNode.spatializer);
79090 | }
79091 |
79092 | MA_API void ma_sound_set_min_gain(ma_sound* pSound, float minGain)
79093 | {
79094 |     if (pSound == NULL) {
79095 |         return;
79096 |     }
79097 |
79098 |     ma_spatializer_set_min_gain(&pSound->engineNode.spatializer, minGain);
79099 | }
79100 |
79101 | MA_API float ma_sound_get_min_gain(const ma_sound* pSound)
79102 | {
79103 |     if (pSound == NULL) {
79104 |         return 0;
79105 |     }
79106 |
79107 |     return ma_spatializer_get_min_gain(&pSound->engineNode.spatializer);
79108 | }
79109 |
79110 | MA_API void ma_sound_set_max_gain(ma_sound* pSound, float maxGain)
79111 | {
79112 |     if (pSound == NULL) {
79113 |         return;
79114 |     }
79115 | }
```

```
79116 |     ma_spatializer_set_max_gain(&pSound->engineNode.spatializer, maxGain);
79117 | }
79118 |
79119 | MA_API float ma_sound_get_max_gain(const ma_sound* pSound)
79120 | {
79121 |     if (pSound == NULL) {
79122 |         return 0;
79123 |     }
79124 |
79125 |     return ma_spatializer_get_max_gain(&pSound->engineNode.spatializer);
79126 | }
79127 |
79128 | MA_API void ma_sound_set_min_distance(ma_sound* pSound, float minDistance)
79129 | {
79130 |     if (pSound == NULL) {
79131 |         return;
79132 |     }
79133 |
79134 |     ma_spatializer_set_min_distance(&pSound->engineNode.spatializer, minDistance);
79135 | }
79136 |
79137 | MA_API float ma_sound_get_min_distance(const ma_sound* pSound)
79138 | {
79139 |     if (pSound == NULL) {
79140 |         return 0;
79141 |     }
79142 |
79143 |     return ma_spatializer_get_min_distance(&pSound->engineNode.spatializer);
79144 | }
79145 |
79146 | MA_API void ma_sound_set_max_distance(ma_sound* pSound, float maxDistance)
79147 | {
79148 |     if (pSound == NULL) {
79149 |         return;
79150 |     }
79151 |
79152 |     ma_spatializer_set_max_distance(&pSound->engineNode.spatializer, maxDistance);
79153 | }
79154 |
79155 | MA_API float ma_sound_get_max_distance(const ma_sound* pSound)
79156 | {
79157 |     if (pSound == NULL) {
79158 |         return 0;
79159 |     }
79160 |
79161 |     return ma_spatializer_get_max_distance(&pSound->engineNode.spatializer);
79162 | }
79163 |
79164 | MA_API void ma_sound_set_cone(ma_sound* pSound, float innerAngleInRadians, float
> outerAngleInRadians, float outerGain)
79165 | {
79166 |     if (pSound == NULL) {
79167 |         return;
79168 |     }
79169 |
79170 |     ma_spatializer_set_cone(&pSound->engineNode.spatializer, innerAngleInRadians,
> outerAngleInRadians, outerGain);
79171 | }
79172 |
79173 | MA_API void ma_sound_get_cone(const ma_sound* pSound, float* pInnerAngleInRadians, float*
> pOuterAngleInRadians, float* pOuterGain)
79174 | {
79175 |     if (pInnerAngleInRadians != NULL) {
79176 |         *pInnerAngleInRadians = 0;
79177 |     }
79178 |
79179 |     if (pOuterAngleInRadians != NULL) {
79180 |         *pOuterAngleInRadians = 0;
79181 |     }
79182 |
79183 |     if (pOuterGain != NULL) {
79184 |         *pOuterGain = 0;
79185 |     }
79186 |
79187 |     if (pSound == NULL) {
79188 |         return;
79189 |     }
79190 |
79191 |     ma_spatializer_get_cone(&pSound->engineNode.spatializer, pInnerAngleInRadians,
```

```

    > pOuterAngleInRadians, pOuterGain);
79192 | }
79193 |
79194 | MA_API void ma_sound_set_doppler_factor(ma_sound* pSound, float dopplerFactor)
79195 | {
79196 |     if (pSound == NULL) {
79197 |         return;
79198 |     }
79199 |
79200 |     ma_spatializer_set_doppler_factor(&pSound->engineNode.spatializer, dopplerFactor);
79201 | }
79202 |
79203 | MA_API float ma_sound_get_doppler_factor(const ma_sound* pSound)
79204 | {
79205 |     if (pSound == NULL) {
79206 |         return 0;
79207 |     }
79208 |
79209 |     return ma_spatializer_get_doppler_factor(&pSound->engineNode.spatializer);
79210 | }
79211 |
79212 | MA_API void ma_sound_set_directional_attenuation_factor(ma_sound* pSound, float
> directionalAttenuationFactor)
79213 | {
79214 |     if (pSound == NULL) {
79215 |         return;
79216 |     }
79217 |
79218 |     ma_spatializer_set_directional_attenuation_factor(&pSound->engineNode.spatializer,
> directionalAttenuationFactor);
79219 | }
79220 |
79221 | MA_API float ma_sound_get_directional_attenuation_factor(const ma_sound* pSound)
79222 | {
79223 |     if (pSound == NULL) {
79224 |         return 1;
79225 |     }
79226 |
79227 |     return ma_spatializer_get_directional_attenuation_factor(&pSound->engineNode.spatializer);
79228 | }
79229 |
79230 |
79231 | MA_API void ma_sound_set_fade_in_pcm_frames(ma_sound* pSound, float volumeBeg, float volumeEnd,
> ma_uint64 fadeLengthInFrames)
79232 | {
79233 |     if (pSound == NULL) {
79234 |         return;
79235 |     }
79236 |
79237 |     ma_sound_set_fade_start_in_pcm_frames(pSound, volumeBeg, volumeEnd, fadeLengthInFrames,
> ~(ma_uint64)0));
79238 | }
79239 |
79240 | MA_API void ma_sound_set_fade_in_milliseconds(ma_sound* pSound, float volumeBeg, float volumeEnd,
> ma_uint64 fadeLengthInMilliseconds)
79241 | {
79242 |     if (pSound == NULL) {
79243 |         return;
79244 |     }
79245 |
79246 |     ma_sound_set_fade_in_pcm_frames(pSound, volumeBeg, volumeEnd, (fadeLengthInMilliseconds *
> pSound->engineNode.fader.config.sampleRate) / 1000);
79247 | }
79248 |
79249 | MA_API void ma_sound_set_fade_start_in_pcm_frames(ma_sound* pSound, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInFrames, ma_uint64 absoluteGlobalTimeInFrames)
79250 | {
79251 |     if (pSound == NULL) {
79252 |         return;
79253 |     }
79254 |
79255 |     /*
79256 |     We don't want to update the fader at this point because we need to use the engine's current time
79257 |     to derive the fader's start offset. The timer is being updated on the audio thread so in order
> to
79258 |     do this as accurately as possible we'll need to defer this to the audio thread.
79259 |     */
79260 |     ma_atomic_float_set(&pSound->engineNode.fadeSettings.volumeBeg, volumeBeg);
79261 |     ma_atomic_float_set(&pSound->engineNode.fadeSettings.volumeEnd, volumeEnd);

```

```
79262 |     ma_atomic_uint64_set(&pSound->engineNode.fadeSettings.fadeLengthInFrames, fadeLengthInFrames);
79263 |     ma_atomic_uint64_set(&pSound->engineNode.fadeSettings.absoluteGlobalTimeInFrames,
> absoluteGlobalTimeInFrames);
79264 | }
79265 |
79266 | MA_API void ma_sound_set_fade_start_in_milliseconds(ma_sound* pSound, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInMilliseconds, ma_uint64 absoluteGlobalTimeInMilliseconds)
79267 | {
79268 |     ma_uint32 sampleRate;
79269 |
79270 |     if (pSound == NULL) {
79271 |         return;
79272 |     }
79273 |
79274 |     sampleRate = ma_engine_get_sample_rate(ma_sound_get_engine(pSound));
79275 |
79276 |     ma_sound_set_fade_start_in_pcm_frames(pSound, volumeBeg, volumeEnd, (fadeLengthInMilliseconds *
> sampleRate) / 1000, (absoluteGlobalTimeInMilliseconds * sampleRate) / 1000);
79277 | }
79278 |
79279 | MA_API float ma_sound_get_current_fade_volume(const ma_sound* pSound)
79280 | {
79281 |     if (pSound == NULL) {
79282 |         return MA_INVALID_ARGS;
79283 |     }
79284 |
79285 |     return ma_fader_get_current_volume(&pSound->engineNode.fader);
79286 | }
79287 |
79288 | MA_API void ma_sound_set_start_time_in_pcm_frames(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInFrames)
79289 | {
79290 |     if (pSound == NULL) {
79291 |         return;
79292 |     }
79293 |
79294 |     ma_node_set_state_time(pSound, ma_node_state_started, absoluteGlobalTimeInFrames);
79295 | }
79296 |
79297 | MA_API void ma_sound_set_start_time_in_milliseconds(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInMilliseconds)
79298 | {
79299 |     if (pSound == NULL) {
79300 |         return;
79301 |     }
79302 |
79303 |     ma_sound_set_start_time_in_pcm_frames(pSound, absoluteGlobalTimeInMilliseconds *
> ma_engine_get_sample_rate(ma_sound_get_engine(pSound)) / 1000);
79304 | }
79305 |
79306 | MA_API void ma_sound_set_stop_time_in_pcm_frames(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInFrames)
79307 | {
79308 |     if (pSound == NULL) {
79309 |         return;
79310 |     }
79311 |
79312 |     ma_sound_set_stop_time_with_fade_in_pcm_frames(pSound, absoluteGlobalTimeInFrames, 0);
79313 | }
79314 |
79315 | MA_API void ma_sound_set_stop_time_in_milliseconds(ma_sound* pSound, ma_uint64
> absoluteGlobalTimeInMilliseconds)
79316 | {
79317 |     if (pSound == NULL) {
79318 |         return;
79319 |     }
79320 |
79321 |     ma_sound_set_stop_time_in_pcm_frames(pSound, absoluteGlobalTimeInMilliseconds *
> ma_engine_get_sample_rate(ma_sound_get_engine(pSound)) / 1000);
79322 | }
79323 |
79324 | MA_API void ma_sound_set_stop_time_with_fade_in_pcm_frames(ma_sound* pSound, ma_uint64
> stopAbsoluteGlobalTimeInFrames, ma_uint64 fadeLengthInFrames)
79325 | {
79326 |     if (pSound == NULL) {
79327 |         return;
79328 |     }
79329 |
79330 |     if (fadeLengthInFrames > 0) {
```

```

79331 |         if (fadeLengthInFrames > stopAbsoluteGlobalTimeInFrames) {
79332 |             fadeLengthInFrames = stopAbsoluteGlobalTimeInFrames;
79333 |         }
79334 |
79335 |         ma_sound_set_fade_start_in_pcm_frames(pSound, -1, 0, fadeLengthInFrames,
> stopAbsoluteGlobalTimeInFrames - fadeLengthInFrames);
79336 |     }
79337 |
79338 |     ma_node_set_state_time(pSound, ma_node_state_stopped, stopAbsoluteGlobalTimeInFrames);
79339 | }
79340 |
79341 | MA_API void ma_sound_set_stop_time_with_fade_in_milliseconds(ma_sound* pSound, ma_uint64
> stopAbsoluteGlobalTimeInMilliseconds, ma_uint64 fadeLengthInMilliseconds)
79342 | {
79343 |     ma_uint32 sampleRate;
79344 |
79345 |     if (pSound == NULL) {
79346 |         return;
79347 |     }
79348 |
79349 |     sampleRate = ma_engine_get_sample_rate(ma_sound_get_engine(pSound));
79350 |
79351 |     ma_sound_set_stop_time_with_fade_in_pcm_frames(pSound, (stopAbsoluteGlobalTimeInMilliseconds *
> sampleRate) / 1000, (fadeLengthInMilliseconds * sampleRate) / 1000);
79352 | }
79353 |
79354 | MA_API ma_bool32 ma_sound_is_playing(const ma_sound* pSound)
79355 | {
79356 |     if (pSound == NULL) {
79357 |         return MA_FALSE;
79358 |     }
79359 |
79360 |     return ma_node_get_state_by_time(pSound,
> ma_engine_get_time_in_pcm_frames(ma_sound_get_engine(pSound))) == ma_node_state_started;
79361 | }
79362 |
79363 | MA_API ma_uint64 ma_sound_get_time_in_pcm_frames(const ma_sound* pSound)
79364 | {
79365 |     if (pSound == NULL) {
79366 |         return 0;
79367 |     }
79368 |
79369 |     return ma_node_get_time(pSound);
79370 | }
79371 |
79372 | MA_API ma_uint64 ma_sound_get_time_in_milliseconds(const ma_sound* pSound)
79373 | {
79374 |     ma_uint32 sampleRate = ma_engine_get_sample_rate(ma_sound_get_engine(pSound));
79375 |     if (sampleRate == 0) {
79376 |         return 0; /* Prevent a division by zero. */
79377 |     }
79378 |
79379 |     return ma_sound_get_time_in_pcm_frames(pSound) * 1000 / sampleRate;
79380 | }
79381 |
79382 | MA_API void ma_sound_set_looping(ma_sound* pSound, ma_bool32 isLooping)
79383 | {
79384 |     if (pSound == NULL) {
79385 |         return;
79386 |     }
79387 |
79388 |     /* Looping is only a valid concept if the sound is backed by a data source. */
79389 |     if (pSound->pDataSource == NULL) {
79390 |         return;
79391 |     }
79392 |
79393 |     /* The looping state needs to be applied to the data source in order for any looping to actually
> happen. */
79394 |     ma_data_source_set_looping(pSound->pDataSource, isLooping);
79395 | }
79396 |
79397 | MA_API ma_bool32 ma_sound_is_looping(const ma_sound* pSound)
79398 | {
79399 |     if (pSound == NULL) {
79400 |         return MA_FALSE;
79401 |     }
79402 |
79403 |     /* There is no notion of looping for sounds that are not backed by a data source. */
79404 |     if (pSound->pDataSource == NULL) {

```

```

79405 |         return MA_FALSE;
79406 |     }
79407 |
79408 |     return ma_data_source_is_looping(pSound->pDataSource);
79409 | }
79410 |
79411 | MA_API ma_bool32 ma_sound_at_end(const ma_sound* pSound)
79412 | {
79413 |     if (pSound == NULL) {
79414 |         return MA_FALSE;
79415 |     }
79416 |
79417 |     /* There is no notion of an end of a sound if it's not backed by a data source. */
79418 |     if (pSound->pDataSource == NULL) {
79419 |         return MA_FALSE;
79420 |     }
79421 |
79422 |     return ma_sound_get_at_end(pSound);
79423 | }
79424 |
79425 | MA_API ma_result ma_sound_seek_to_pcm_frame(ma_sound* pSound, ma_uint64 frameIndex)
79426 | {
79427 |     if (pSound == NULL) {
79428 |         return MA_INVALID_ARGS;
79429 |     }
79430 |
79431 |     /* Seeking is only valid for sounds that are backed by a data source. */
79432 |     if (pSound->pDataSource == NULL) {
79433 |         return MA_INVALID_OPERATION;
79434 |     }
79435 |
79436 |     /* We can't be seeking while reading at the same time. We just set the seek target and get the
> mixing thread to do the actual seek. */
79437 |     ma_atomic_exchange_64(&pSound->seekTarget, frameIndex);
79438 |
79439 |     return MA_SUCCESS;
79440 | }
79441 |
79442 | MA_API ma_result ma_sound_seek_to_second(ma_sound* pSound, float seekPointInSeconds)
79443 | {
79444 |     ma_uint64 frameIndex;
79445 |     ma_uint32 sampleRate;
79446 |     ma_result result;
79447 |
79448 |     if (pSound == NULL) {
79449 |         return MA_INVALID_ARGS;
79450 |     }
79451 |
79452 |     result = ma_sound_get_data_format(pSound, NULL, NULL, &sampleRate, NULL, 0);
79453 |     if (result != MA_SUCCESS) {
79454 |         return result;
79455 |     }
79456 |
79457 |     /* We need PCM frames. We need to convert first */
79458 |     frameIndex = (ma_uint64)(seekPointInSeconds * sampleRate);
79459 |
79460 |     return ma_sound_seek_to_pcm_frame(pSound, frameIndex);
79461 | }
79462 |
79463 | MA_API ma_result ma_sound_get_data_format(const ma_sound* pSound, ma_format* pFormat, ma_uint32*
> pChannels, ma_uint32* pSampleRate, ma_channel* pChannelMap, size_t channelMapCap)
79464 | {
79465 |     if (pSound == NULL) {
79466 |         return MA_INVALID_ARGS;
79467 |     }
79468 |
79469 |     /* The data format is retrieved directly from the data source if the sound is backed by one.
> Otherwise we pull it from the node. */
79470 |     if (pSound->pDataSource == NULL) {
79471 |         ma_uint32 channels;
79472 |
79473 |         if (pFormat != NULL) {
79474 |             *pFormat = ma_format_f32;
79475 |         }
79476 |
79477 |         channels = ma_node_get_input_channels(&pSound->engineNode, 0);
79478 |         if (pChannels != NULL) {
79479 |             *pChannels = channels;
79480 |         }

```



```

79481 |
79482 |         if (pSampleRate != NULL) {
79483 |             *pSampleRate = pSound->engineNode.resampler.sampleRateIn;
79484 |         }
79485 |
79486 |         if (pChannelMap != NULL) {
79487 |             ma_channel_map_init_standard(ma_standard_channel_map_default, pChannelMap,
> channelMapCap, channels);
79488 |         }
79489 |
79490 |         return MA_SUCCESS;
79491 |     } else {
79492 |         return ma_data_source_get_data_format(pSound->pDataSource, pFormat, pChannels, pSampleRate,
> pChannelMap, channelMapCap);
79493 |     }
79494 | }
79495 |
79496 | MA_API ma_result ma_sound_get_cursor_in_pcm_frames(const ma_sound* pSound, ma_uint64* pCursor)
79497 | {
79498 |     ma_uint64 seekTarget;
79499 |
79500 |     if (pSound == NULL) {
79501 |         return MA_INVALID_ARGS;
79502 |     }
79503 |
79504 |     /* The notion of a cursor is only valid for sounds that are backed by a data source. */
79505 |     if (pSound->pDataSource == NULL) {
79506 |         return MA_INVALID_OPERATION;
79507 |     }
79508 |
79509 |     seekTarget = ma_atomic_load_64(&pSound->seekTarget);
79510 |     if (seekTarget != MA_SEEK_TARGET_NONE) {
79511 |         *pCursor = seekTarget;
79512 |         return MA_SUCCESS;
79513 |     } else {
79514 |         return ma_data_source_get_cursor_in_pcm_frames(pSound->pDataSource, pCursor);
79515 |     }
79516 | }
79517 |
79518 | MA_API ma_result ma_sound_get_length_in_pcm_frames(const ma_sound* pSound, ma_uint64* pLength)
79519 | {
79520 |     if (pSound == NULL) {
79521 |         return MA_INVALID_ARGS;
79522 |     }
79523 |
79524 |     /* The notion of a sound length is only valid for sounds that are backed by a data source. */
79525 |     if (pSound->pDataSource == NULL) {
79526 |         return MA_INVALID_OPERATION;
79527 |     }
79528 |
79529 |     return ma_data_source_get_length_in_pcm_frames(pSound->pDataSource, pLength);
79530 | }
79531 |
79532 | MA_API ma_result ma_sound_get_cursor_in_seconds(const ma_sound* pSound, float* pCursor)
79533 | {
79534 |     ma_result result;
79535 |     ma_uint64 cursorInPCMFrames;
79536 |     ma_uint32 sampleRate;
79537 |
79538 |     if (pCursor != NULL) {
79539 |         *pCursor = 0;
79540 |     }
79541 |
79542 |     result = ma_sound_get_cursor_in_pcm_frames(pSound, &cursorInPCMFrames);
79543 |     if (result != MA_SUCCESS) {
79544 |         return result;
79545 |     }
79546 |
79547 |     result = ma_sound_get_data_format(pSound, NULL, NULL, &sampleRate, NULL, 0);
79548 |     if (result != MA_SUCCESS) {
79549 |         return result;
79550 |     }
79551 |
79552 |     /* VC6 does not support division of unsigned 64-bit integers with floating point numbers. Need
> to use a signed number. This shouldn't effect anything in practice. */
79553 |     *pCursor = (ma_int64)cursorInPCMFrames / (float)sampleRate;
79554 |
79555 |     return MA_SUCCESS;
79556 | }

```

```

79557 |
79558 | MA_API ma_result ma_sound_get_length_in_seconds(const ma_sound* pSound, float* pLength)
79559 | {
79560 |     if (pSound == NULL) {
79561 |         return MA_INVALID_ARGS;
79562 |     }
79563 |
79564 |     /* The notion of a sound length is only valid for sounds that are backed by a data source. */
79565 |     if (pSound->pDataSource == NULL) {
79566 |         return MA_INVALID_OPERATION;
79567 |     }
79568 |
79569 |     return ma_data_source_get_length_in_seconds(pSound->pDataSource, pLength);
79570 | }
79571 |
79572 | MA_API ma_result ma_sound_set_end_callback(ma_sound* pSound, ma_sound_end_proc callback, void*
> pUserData)
79573 | {
79574 |     if (pSound == NULL) {
79575 |         return MA_INVALID_ARGS;
79576 |     }
79577 |
79578 |     /* The notion of an end is only valid for sounds that are backed by a data source. */
79579 |     if (pSound->pDataSource == NULL) {
79580 |         return MA_INVALID_OPERATION;
79581 |     }
79582 |
79583 |     pSound->endCallback = callback;
79584 |     pSound->pEndCallbackUserData = pUserData;
79585 |
79586 |     return MA_SUCCESS;
79587 | }
79588 |
79589 |
79590 | MA_API ma_result ma_sound_group_init(ma_engine* pEngine, ma_uint32 flags, ma_sound_group*
> pParentGroup, ma_sound_group* pGroup)
79591 | {
79592 |     ma_sound_group_config config = ma_sound_group_config_init_2(pEngine);
79593 |     config.flags = flags;
79594 |     config.pInitialAttachment = pParentGroup;
79595 |     return ma_sound_group_init_ex(pEngine, &config, pGroup);
79596 | }
79597 |
79598 | MA_API ma_result ma_sound_group_init_ex(ma_engine* pEngine, const ma_sound_group_config* pConfig,
> ma_sound_group* pGroup)
79599 | {
79600 |     ma_sound_config soundConfig;
79601 |
79602 |     if (pGroup == NULL) {
79603 |         return MA_INVALID_ARGS;
79604 |     }
79605 |
79606 |     MA_ZERO_OBJECT(pGroup);
79607 |
79608 |     if (pConfig == NULL) {
79609 |         return MA_INVALID_ARGS;
79610 |     }
79611 |
79612 |     /* A sound group is just a sound without a data source. */
79613 |     soundConfig = *pConfig;
79614 |     soundConfig.pFilePath = NULL;
79615 |     soundConfig.pFilePathW = NULL;
79616 |     soundConfig.pDataSource = NULL;
79617 |
79618 |     /*
79619 |     Groups need to have spatialization disabled by default because I think it'll be pretty rare
79620 |     that programs will want to spatialize groups (but not unheard of). Certainly it feels like
79621 |     disabling this by default feels like the right option. Spatialization can be enabled with a
79622 |     call to ma_sound_group_set_spatialization_enabled().
79623 |     */
79624 |     soundConfig.flags |= MA_SOUND_FLAG_NO_SPATIALIZATION;
79625 |
79626 |     return ma_sound_init_ex(pEngine, &soundConfig, pGroup);
79627 | }
79628 |
79629 | MA_API void ma_sound_group_uninit(ma_sound_group* pGroup)
79630 | {
79631 |     ma_sound_uninit(pGroup);
79632 | }

```

```
79633 |
79634 | MA_API ma_engine* ma_sound_group_get_engine(const ma_sound_group* pGroup)
79635 | {
79636 |     return ma_sound_get_engine(pGroup);
79637 | }
79638 |
79639 | MA_API ma_result ma_sound_group_start(ma_sound_group* pGroup)
79640 | {
79641 |     return ma_sound_start(pGroup);
79642 | }
79643 |
79644 | MA_API ma_result ma_sound_group_stop(ma_sound_group* pGroup)
79645 | {
79646 |     return ma_sound_stop(pGroup);
79647 | }
79648 |
79649 | MA_API void ma_sound_group_set_volume(ma_sound_group* pGroup, float volume)
79650 | {
79651 |     ma_sound_set_volume(pGroup, volume);
79652 | }
79653 |
79654 | MA_API float ma_sound_group_get_volume(const ma_sound_group* pGroup)
79655 | {
79656 |     return ma_sound_get_volume(pGroup);
79657 | }
79658 |
79659 | MA_API void ma_sound_group_set_pan(ma_sound_group* pGroup, float pan)
79660 | {
79661 |     ma_sound_set_pan(pGroup, pan);
79662 | }
79663 |
79664 | MA_API float ma_sound_group_get_pan(const ma_sound_group* pGroup)
79665 | {
79666 |     return ma_sound_get_pan(pGroup);
79667 | }
79668 |
79669 | MA_API void ma_sound_group_set_pan_mode(ma_sound_group* pGroup, ma_pan_mode panMode)
79670 | {
79671 |     ma_sound_set_pan_mode(pGroup, panMode);
79672 | }
79673 |
79674 | MA_API ma_pan_mode ma_sound_group_get_pan_mode(const ma_sound_group* pGroup)
79675 | {
79676 |     return ma_sound_get_pan_mode(pGroup);
79677 | }
79678 |
79679 | MA_API void ma_sound_group_set_pitch(ma_sound_group* pGroup, float pitch)
79680 | {
79681 |     ma_sound_set_pitch(pGroup, pitch);
79682 | }
79683 |
79684 | MA_API float ma_sound_group_get_pitch(const ma_sound_group* pGroup)
79685 | {
79686 |     return ma_sound_get_pitch(pGroup);
79687 | }
79688 |
79689 | MA_API void ma_sound_group_set_spatialization_enabled(ma_sound_group* pGroup, ma_bool32 enabled)
79690 | {
79691 |     ma_sound_set_spatialization_enabled(pGroup, enabled);
79692 | }
79693 |
79694 | MA_API ma_bool32 ma_sound_group_is_spatialization_enabled(const ma_sound_group* pGroup)
79695 | {
79696 |     return ma_sound_is_spatialization_enabled(pGroup);
79697 | }
79698 |
79699 | MA_API void ma_sound_group_set_pinned_listener_index(ma_sound_group* pGroup, ma_uint32
> listenerIndex)
79700 | {
79701 |     ma_sound_set_pinned_listener_index(pGroup, listenerIndex);
79702 | }
79703 |
79704 | MA_API ma_uint32 ma_sound_group_get_pinned_listener_index(const ma_sound_group* pGroup)
79705 | {
79706 |     return ma_sound_get_pinned_listener_index(pGroup);
79707 | }
79708 |
79709 | MA_API ma_uint32 ma_sound_group_get_listener_index(const ma_sound_group* pGroup)
79710 | {
```

```
79711 |     return ma_sound_get_listener_index(pGroup);
79712 | }
79713 |
79714 | MA_API ma_vec3f ma_sound_group_get_direction_to_listener(const ma_sound_group* pGroup)
79715 | {
79716 |     return ma_sound_get_direction_to_listener(pGroup);
79717 | }
79718 |
79719 | MA_API void ma_sound_group_set_position(ma_sound_group* pGroup, float x, float y, float z)
79720 | {
79721 |     ma_sound_set_position(pGroup, x, y, z);
79722 | }
79723 |
79724 | MA_API ma_vec3f ma_sound_group_get_position(const ma_sound_group* pGroup)
79725 | {
79726 |     return ma_sound_get_position(pGroup);
79727 | }
79728 |
79729 | MA_API void ma_sound_group_set_direction(ma_sound_group* pGroup, float x, float y, float z)
79730 | {
79731 |     ma_sound_set_direction(pGroup, x, y, z);
79732 | }
79733 |
79734 | MA_API ma_vec3f ma_sound_group_get_direction(const ma_sound_group* pGroup)
79735 | {
79736 |     return ma_sound_get_direction(pGroup);
79737 | }
79738 |
79739 | MA_API void ma_sound_group_set_velocity(ma_sound_group* pGroup, float x, float y, float z)
79740 | {
79741 |     ma_sound_set_velocity(pGroup, x, y, z);
79742 | }
79743 |
79744 | MA_API ma_vec3f ma_sound_group_get_velocity(const ma_sound_group* pGroup)
79745 | {
79746 |     return ma_sound_get_velocity(pGroup);
79747 | }
79748 |
79749 | MA_API void ma_sound_group_set_attenuation_model(ma_sound_group* pGroup, ma_attenuation_model
> attenuationModel)
79750 | {
79751 |     ma_sound_set_attenuation_model(pGroup, attenuationModel);
79752 | }
79753 |
79754 | MA_API ma_attenuation_model ma_sound_group_get_attenuation_model(const ma_sound_group* pGroup)
79755 | {
79756 |     return ma_sound_get_attenuation_model(pGroup);
79757 | }
79758 |
79759 | MA_API void ma_sound_group_set_positioning(ma_sound_group* pGroup, ma_positioning positioning)
79760 | {
79761 |     ma_sound_set_positioning(pGroup, positioning);
79762 | }
79763 |
79764 | MA_API ma_positioning ma_sound_group_get_positioning(const ma_sound_group* pGroup)
79765 | {
79766 |     return ma_sound_get_positioning(pGroup);
79767 | }
79768 |
79769 | MA_API void ma_sound_group_set_rolloff(ma_sound_group* pGroup, float rolloff)
79770 | {
79771 |     ma_sound_set_rolloff(pGroup, rolloff);
79772 | }
79773 |
79774 | MA_API float ma_sound_group_get_rolloff(const ma_sound_group* pGroup)
79775 | {
79776 |     return ma_sound_get_rolloff(pGroup);
79777 | }
79778 |
79779 | MA_API void ma_sound_group_set_min_gain(ma_sound_group* pGroup, float minGain)
79780 | {
79781 |     ma_sound_set_min_gain(pGroup, minGain);
79782 | }
79783 |
79784 | MA_API float ma_sound_group_get_min_gain(const ma_sound_group* pGroup)
79785 | {
79786 |     return ma_sound_get_min_gain(pGroup);
79787 | }
79788 |
```

```
79789 | MA_API void ma_sound_group_set_max_gain(ma_sound_group* pGroup, float maxGain)
79790 | {
79791 |     ma_sound_set_max_gain(pGroup, maxGain);
79792 | }
79793 |
79794 | MA_API float ma_sound_group_get_max_gain(const ma_sound_group* pGroup)
79795 | {
79796 |     return ma_sound_get_max_gain(pGroup);
79797 | }
79798 |
79799 | MA_API void ma_sound_group_set_min_distance(ma_sound_group* pGroup, float minDistance)
79800 | {
79801 |     ma_sound_set_min_distance(pGroup, minDistance);
79802 | }
79803 |
79804 | MA_API float ma_sound_group_get_min_distance(const ma_sound_group* pGroup)
79805 | {
79806 |     return ma_sound_get_min_distance(pGroup);
79807 | }
79808 |
79809 | MA_API void ma_sound_group_set_max_distance(ma_sound_group* pGroup, float maxDistance)
79810 | {
79811 |     ma_sound_set_max_distance(pGroup, maxDistance);
79812 | }
79813 |
79814 | MA_API float ma_sound_group_get_max_distance(const ma_sound_group* pGroup)
79815 | {
79816 |     return ma_sound_get_max_distance(pGroup);
79817 | }
79818 |
79819 | MA_API void ma_sound_group_set_cone(ma_sound_group* pGroup, float innerAngleInRadians, float
> outerAngleInRadians, float outerGain)
79820 | {
79821 |     ma_sound_set_cone(pGroup, innerAngleInRadians, outerAngleInRadians, outerGain);
79822 | }
79823 |
79824 | MA_API void ma_sound_group_get_cone(const ma_sound_group* pGroup, float* pInnerAngleInRadians,
> float* pOuterAngleInRadians, float* pOuterGain)
79825 | {
79826 |     ma_sound_get_cone(pGroup, pInnerAngleInRadians, pOuterAngleInRadians, pOuterGain);
79827 | }
79828 |
79829 | MA_API void ma_sound_group_set_doppler_factor(ma_sound_group* pGroup, float dopplerFactor)
79830 | {
79831 |     ma_sound_set_doppler_factor(pGroup, dopplerFactor);
79832 | }
79833 |
79834 | MA_API float ma_sound_group_get_doppler_factor(const ma_sound_group* pGroup)
79835 | {
79836 |     return ma_sound_get_doppler_factor(pGroup);
79837 | }
79838 |
79839 | MA_API void ma_sound_group_set_directional_attenuation_factor(ma_sound_group* pGroup, float
> directionalAttenuationFactor)
79840 | {
79841 |     ma_sound_set_directional_attenuation_factor(pGroup, directionalAttenuationFactor);
79842 | }
79843 |
79844 | MA_API float ma_sound_group_get_directional_attenuation_factor(const ma_sound_group* pGroup)
79845 | {
79846 |     return ma_sound_get_directional_attenuation_factor(pGroup);
79847 | }
79848 |
79849 | MA_API void ma_sound_group_set_fade_in_pcm_frames(ma_sound_group* pGroup, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInFrames)
79850 | {
79851 |     ma_sound_set_fade_in_pcm_frames(pGroup, volumeBeg, volumeEnd, fadeLengthInFrames);
79852 | }
79853 |
79854 | MA_API void ma_sound_group_set_fade_in_milliseconds(ma_sound_group* pGroup, float volumeBeg, float
> volumeEnd, ma_uint64 fadeLengthInMilliseconds)
79855 | {
79856 |     ma_sound_set_fade_in_milliseconds(pGroup, volumeBeg, volumeEnd, fadeLengthInMilliseconds);
79857 | }
79858 |
79859 | MA_API float ma_sound_group_get_current_fade_volume(ma_sound_group* pGroup)
79860 | {
79861 |     return ma_sound_get_current_fade_volume(pGroup);
79862 | }
```

```

79863 |
79864 | MA_API void ma_sound_group_set_start_time_in_pcm_frames(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInFrames)
79865 | {
79866 |     ma_sound_set_start_time_in_pcm_frames(pGroup, absoluteGlobalTimeInFrames);
79867 | }
79868 |
79869 | MA_API void ma_sound_group_set_start_time_in_milliseconds(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInMilliseconds)
79870 | {
79871 |     ma_sound_set_start_time_in_milliseconds(pGroup, absoluteGlobalTimeInMilliseconds);
79872 | }
79873 |
79874 | MA_API void ma_sound_group_set_stop_time_in_pcm_frames(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInFrames)
79875 | {
79876 |     ma_sound_set_stop_time_in_pcm_frames(pGroup, absoluteGlobalTimeInFrames);
79877 | }
79878 |
79879 | MA_API void ma_sound_group_set_stop_time_in_milliseconds(ma_sound_group* pGroup, ma_uint64
> absoluteGlobalTimeInMilliseconds)
79880 | {
79881 |     ma_sound_set_stop_time_in_milliseconds(pGroup, absoluteGlobalTimeInMilliseconds);
79882 | }
79883 |
79884 | MA_API ma_bool32 ma_sound_group_is_playing(const ma_sound_group* pGroup)
79885 | {
79886 |     return ma_sound_is_playing(pGroup);
79887 | }
79888 |
79889 | MA_API ma_uint64 ma_sound_group_get_time_in_pcm_frames(const ma_sound_group* pGroup)
79890 | {
79891 |     return ma_sound_get_time_in_pcm_frames(pGroup);
79892 | }
79893 | #endif /* MA_NO_ENGINE */
79894 | /* END SECTION: miniaudio_engine.c */
79895 |
79896 |
79897 |
79898 | /*****
> *****/
79899 | *****/
> *****/
79900 |
79901 | Auto Generated
79902 | =====
79903 | All code below is auto-generated from a tool. This mostly consists of decoding backend
> implementations such as ma_dr_wav, ma_dr_flac, etc. If you find a bug in the
79904 | code below please report the bug to the respective repository for the relevant project (probably
> dr_libs).
79905 |
79906 | *****/
> *****/
79907 | *****/
> *****/
79908 | #if !defined(MA_NO_WAV) && (!defined(MA_NO_DECODING) || !defined(MA_NO_ENCODING))
79909 | #if !defined(MA_DR_WAV_IMPLEMENTATION)
79910 | /* dr_wav_c begin */
79911 | #ifndef ma_dr_wav_c
79912 | #define ma_dr_wav_c
79913 | #ifdef __MRC__
79914 | #pragma options opt off
79915 | #endif
79916 | #include <stdlib.h>
79917 | #include <string.h>
79918 | #include <limits.h>
79919 | #ifndef MA_DR_WAV_NO_STDIO
79920 | #include <stdio.h>
79921 | #ifndef MA_DR_WAV_NO_WCHAR
79922 | #include <wchar.h>
79923 | #endif
79924 | #endif
79925 | #ifndef MA_DR_WAV_ASSERT
79926 | #include <assert.h>
79927 | #define MA_DR_WAV_ASSERT(expression)          assert(expression)
79928 | #endif
79929 | #ifndef MA_DR_WAV_MALLOC
79930 | #define MA_DR_WAV_MALLOC(sz)                  malloc((sz))
79931 | #endif

```

```

79932 | #ifndef MA_DR_WAV_REALLOC
79933 | #define MA_DR_WAV_REALLOC(p, sz)          realloc((p), (sz))
79934 | #endif
79935 | #ifndef MA_DR_WAV_FREE
79936 | #define MA_DR_WAV_FREE(p)                free((p))
79937 | #endif
79938 | #ifndef MA_DR_WAV_COPY_MEMORY
79939 | #define MA_DR_WAV_COPY_MEMORY(dst, src, sz) memcpy((dst), (src), (sz))
79940 | #endif
79941 | #ifndef MA_DR_WAV_ZERO_MEMORY
79942 | #define MA_DR_WAV_ZERO_MEMORY(p, sz)      memset((p), 0, (sz))
79943 | #endif
79944 | #ifndef MA_DR_WAV_ZERO_OBJECT
79945 | #define MA_DR_WAV_ZERO_OBJECT(p)          MA_DR_WAV_ZERO_MEMORY((p), sizeof(*p))
79946 | #endif
79947 | #define ma_dr_wav_countof(x)              (sizeof(x) / sizeof(x[0]))
79948 | #define ma_dr_wav_align(x, a)             (((x) + (a) - 1) / (a)) * (a)
79949 | #define ma_dr_wav_min(a, b)               (((a) < (b)) ? (a) : (b))
79950 | #define ma_dr_wav_max(a, b)               (((a) > (b)) ? (a) : (b))
79951 | #define ma_dr_wav_clamp(x, lo, hi)        (ma_dr_wav_max((lo), ma_dr_wav_min((hi), (x))))
79952 | #define ma_dr_wav_offset_ptr(p, offset)   (((ma_uint8*)(p)) + (offset))
79953 | #define MA_DR_WAV_MAX_SIMD_VECTOR_SIZE   32
79954 | #define MA_DR_WAV_INT64_MIN ((ma_int64) ((ma_uint64)0x80000000 << 32))
79955 | #define MA_DR_WAV_INT64_MAX ((ma_int64) (((ma_uint64)0x7FFFFFFF << 32) | 0xFFFFFFFF))
79956 | #if defined(_MSC_VER) && _MSC_VER >= 1400
79957 |     #define MA_DR_WAV_HAS_BYTESWAP16_INTRINSIC
79958 |     #define MA_DR_WAV_HAS_BYTESWAP32_INTRINSIC
79959 |     #define MA_DR_WAV_HAS_BYTESWAP64_INTRINSIC
79960 | #elif defined(__clang__)
79961 |     #if defined(__has_builtin)
79962 |         #if __has_builtin(__builtin_bswap16)
79963 |             #define MA_DR_WAV_HAS_BYTESWAP16_INTRINSIC
79964 |         #endif
79965 |         #if __has_builtin(__builtin_bswap32)
79966 |             #define MA_DR_WAV_HAS_BYTESWAP32_INTRINSIC
79967 |         #endif
79968 |         #if __has_builtin(__builtin_bswap64)
79969 |             #define MA_DR_WAV_HAS_BYTESWAP64_INTRINSIC
79970 |         #endif
79971 |     #endif
79972 | #elif defined(__GNUC__)
79973 |     #if ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 3))
79974 |         #define MA_DR_WAV_HAS_BYTESWAP32_INTRINSIC
79975 |         #define MA_DR_WAV_HAS_BYTESWAP64_INTRINSIC
79976 |     #endif
79977 |     #if ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 8))
79978 |         #define MA_DR_WAV_HAS_BYTESWAP16_INTRINSIC
79979 |     #endif
79980 | #endif
79981 | MA_API void ma_dr_wav_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision)
79982 | {
79983 |     if (pMajor) {
79984 |         *pMajor = MA_DR_WAV_VERSION_MAJOR;
79985 |     }
79986 |     if (pMinor) {
79987 |         *pMinor = MA_DR_WAV_VERSION_MINOR;
79988 |     }
79989 |     if (pRevision) {
79990 |         *pRevision = MA_DR_WAV_VERSION_REVISION;
79991 |     }
79992 | }
79993 | MA_API const char* ma_dr_wav_version_string(void)
79994 | {
79995 |     return MA_DR_WAV_VERSION_STRING;
79996 | }
79997 | #ifndef MA_DR_WAV_MAX_SAMPLE_RATE
79998 | #define MA_DR_WAV_MAX_SAMPLE_RATE         384000
79999 | #endif
80000 | #ifndef MA_DR_WAV_MAX_CHANNELS
80001 | #define MA_DR_WAV_MAX_CHANNELS             256
80002 | #endif
80003 | #ifndef MA_DR_WAV_MAX_BITS_PER_SAMPLE
80004 | #define MA_DR_WAV_MAX_BITS_PER_SAMPLE     64
80005 | #endif
80006 | static const ma_uint8 ma_dr_wavGUID_W64_RIFF[16] = {0x72, 0x69, 0x66, 0x66, 0x2E, 0x91, 0xCF, 0x11,
80007 | > 0xA5, 0xD6, 0x28, 0xDB, 0x04, 0xC1, 0x00, 0x00};
80007 | static const ma_uint8 ma_dr_wavGUID_W64_WAVE[16] = {0x77, 0x61, 0x76, 0x65, 0xF3, 0xAC, 0xD3, 0x11,
80008 | > 0x8C, 0xD1, 0x00, 0xC0, 0x4F, 0x8E, 0xDB, 0x8A};
80008 | static const ma_uint8 ma_dr_wavGUID_W64_FMT [16] = {0x66, 0x6D, 0x74, 0x20, 0xF3, 0xAC, 0xD3, 0x11,

```



```

> 0x8C,0xD1, 0x00,0xC0,0x4F,0x8E,0xDB,0x8A};
80009 | static const ma_uint8 ma_dr_wavGUID_W64_FACT[16] = {0x66,0x61,0x63,0x74, 0xF3,0xAC, 0xD3,0x11,
> 0x8C,0xD1, 0x00,0xC0,0x4F,0x8E,0xDB,0x8A};
80010 | static const ma_uint8 ma_dr_wavGUID_W64_DATA[16] = {0x64,0x61,0x74,0x61, 0xF3,0xAC, 0xD3,0x11,
> 0x8C,0xD1, 0x00,0xC0,0x4F,0x8E,0xDB,0x8A};
80011 | static MA_INLINE int ma_dr_wav__is_little_endian(void)
80012 | {
80013 | #if defined(MA_X86) || defined(MA_X64)
80014 |     return MA_TRUE;
80015 | #elif defined(__BYTE_ORDER) && defined(__LITTLE_ENDIAN) && __BYTE_ORDER == __LITTLE_ENDIAN
80016 |     return MA_TRUE;
80017 | #else
80018 |     int n = 1;
80019 |     return (*(char*)&n) == 1;
80020 | #endif
80021 | }
80022 | static MA_INLINE void ma_dr_wav_bytes_to_guid(const ma_uint8* data, ma_uint8* guid)
80023 | {
80024 |     int i;
80025 |     for (i = 0; i < 16; ++i) {
80026 |         guid[i] = data[i];
80027 |     }
80028 | }
80029 | static MA_INLINE ma_uint16 ma_dr_wav__bswap16(ma_uint16 n)
80030 | {
80031 | #ifdef MA_DR_WAV_HAS_BYTESWAP16_INTRINSIC
80032 |     #if defined(_MSC_VER)
80033 |         return _byteswap_ushort(n);
80034 |     #elif defined(__GNUC__) || defined(__clang__)
80035 |         return __builtin_bswap16(n);
80036 |     #else
80037 |         #error "This compiler does not support the byte swap intrinsic."
80038 |     #endif
80039 | #else
80040 |     return ((n & 0xFF00) >> 8) |
80041 |           ((n & 0x00FF) << 8);
80042 | #endif
80043 | }
80044 | static MA_INLINE ma_uint32 ma_dr_wav__bswap32(ma_uint32 n)
80045 | {
80046 | #ifdef MA_DR_WAV_HAS_BYTESWAP32_INTRINSIC
80047 |     #if defined(_MSC_VER)
80048 |         return _byteswap_ulong(n);
80049 |     #elif defined(__GNUC__) || defined(__clang__)
80050 |         #if defined(MA_ARM) && (defined(__ARM_ARCH) && __ARM_ARCH >= 6) && !defined(MA_64BIT)
80051 |             ma_uint32 r;
80052 |             __asm__ __volatile__ (
80053 |                 #if defined(MA_64BIT)
80054 |                     "rev %w[out], %w[in]" : [out]"=r"(r) : [in]"r"(n)
80055 |                 #else
80056 |                     "rev %[out], %[in]" : [out]"=r"(r) : [in]"r"(n)
80057 |                 #endif
80058 |                 );
80059 |             return r;
80060 |         #else
80061 |             return __builtin_bswap32(n);
80062 |         #endif
80063 |     #else
80064 |         #error "This compiler does not support the byte swap intrinsic."
80065 |     #endif
80066 | #else
80067 |     return ((n & 0xFF000000) >> 24) |
80068 |           ((n & 0x00FF0000) >> 8) |
80069 |           ((n & 0x0000FF00) << 8) |
80070 |           ((n & 0x000000FF) << 24);
80071 | #endif
80072 | }
80073 | static MA_INLINE ma_uint64 ma_dr_wav__bswap64(ma_uint64 n)
80074 | {
80075 | #ifdef MA_DR_WAV_HAS_BYTESWAP64_INTRINSIC
80076 |     #if defined(_MSC_VER)
80077 |         return _byteswap_uint64(n);
80078 |     #elif defined(__GNUC__) || defined(__clang__)
80079 |         return __builtin_bswap64(n);
80080 |     #else
80081 |         #error "This compiler does not support the byte swap intrinsic."
80082 |     #endif
80083 | #else
80084 |     return ((n & ((ma_uint64)0xFF000000 << 32)) >> 56) |

```

```

80085 |         ((n & ((ma_uint64)0x00FF0000 << 32)) >> 40) |
80086 |         ((n & ((ma_uint64)0x0000FF00 << 32)) >> 24) |
80087 |         ((n & ((ma_uint64)0x000000FF << 32)) >> 8) |
80088 |         ((n & ((ma_uint64)0xFF000000 )) << 8) |
80089 |         ((n & ((ma_uint64)0x00FF0000 )) << 24) |
80090 |         ((n & ((ma_uint64)0x0000FF00 )) << 40) |
80091 |         ((n & ((ma_uint64)0x000000FF )) << 56);
80092 | #endif
80093 | }
80094 | static MA_INLINE ma_int16 ma_dr_wav__bswap_s16(ma_int16 n)
80095 | {
80096 |     return (ma_int16)ma_dr_wav__bswap16((ma_uint16)n);
80097 | }
80098 | static MA_INLINE void ma_dr_wav__bswap_samples_s16(ma_int16* pSamples, ma_uint64 sampleCount)
80099 | {
80100 |     ma_uint64 iSample;
80101 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
80102 |         pSamples[iSample] = ma_dr_wav__bswap_s16(pSamples[iSample]);
80103 |     }
80104 | }
80105 | static MA_INLINE void ma_dr_wav__bswap_s24(ma_uint8* p)
80106 | {
80107 |     ma_uint8 t;
80108 |     t = p[0];
80109 |     p[0] = p[2];
80110 |     p[2] = t;
80111 | }
80112 | static MA_INLINE void ma_dr_wav__bswap_samples_s24(ma_uint8* pSamples, ma_uint64 sampleCount)
80113 | {
80114 |     ma_uint64 iSample;
80115 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
80116 |         ma_uint8* pSample = pSamples + (iSample*3);
80117 |         ma_dr_wav__bswap_s24(pSample);
80118 |     }
80119 | }
80120 | static MA_INLINE ma_int32 ma_dr_wav__bswap_s32(ma_int32 n)
80121 | {
80122 |     return (ma_int32)ma_dr_wav__bswap32((ma_uint32)n);
80123 | }
80124 | static MA_INLINE void ma_dr_wav__bswap_samples_s32(ma_int32* pSamples, ma_uint64 sampleCount)
80125 | {
80126 |     ma_uint64 iSample;
80127 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
80128 |         pSamples[iSample] = ma_dr_wav__bswap_s32(pSamples[iSample]);
80129 |     }
80130 | }
80131 | static MA_INLINE ma_int64 ma_dr_wav__bswap_s64(ma_int64 n)
80132 | {
80133 |     return (ma_int64)ma_dr_wav__bswap64((ma_uint64)n);
80134 | }
80135 | static MA_INLINE void ma_dr_wav__bswap_samples_s64(ma_int64* pSamples, ma_uint64 sampleCount)
80136 | {
80137 |     ma_uint64 iSample;
80138 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
80139 |         pSamples[iSample] = ma_dr_wav__bswap_s64(pSamples[iSample]);
80140 |     }
80141 | }
80142 | static MA_INLINE float ma_dr_wav__bswap_f32(float n)
80143 | {
80144 |     union {
80145 |         ma_uint32 i;
80146 |         float f;
80147 |     } x;
80148 |     x.f = n;
80149 |     x.i = ma_dr_wav__bswap32(x.i);
80150 |     return x.f;
80151 | }
80152 | static MA_INLINE void ma_dr_wav__bswap_samples_f32(float* pSamples, ma_uint64 sampleCount)
80153 | {
80154 |     ma_uint64 iSample;
80155 |     for (iSample = 0; iSample < sampleCount; iSample += 1) {
80156 |         pSamples[iSample] = ma_dr_wav__bswap_f32(pSamples[iSample]);
80157 |     }
80158 | }
80159 | static MA_INLINE void ma_dr_wav__bswap_samples(void* pSamples, ma_uint64 sampleCount, ma_uint32
> bytesPerSample)
80160 | {
80161 |     switch (bytesPerSample)
80162 |     {

```

```

80163 |         case 1:
80164 |         {
80165 |         } break;
80166 |         case 2:
80167 |         {
80168 |             ma_dr_wav__bswap_samples_s16((ma_int16*)pSamples, sampleCount);
80169 |         } break;
80170 |         case 3:
80171 |         {
80172 |             ma_dr_wav__bswap_samples_s24((ma_uint8*)pSamples, sampleCount);
80173 |         } break;
80174 |         case 4:
80175 |         {
80176 |             ma_dr_wav__bswap_samples_s32((ma_int32*)pSamples, sampleCount);
80177 |         } break;
80178 |         case 8:
80179 |         {
80180 |             ma_dr_wav__bswap_samples_s64((ma_int64*)pSamples, sampleCount);
80181 |         } break;
80182 |         default:
80183 |         {
80184 |             MA_DR_WAV_ASSERT(MA_FALSE);
80185 |         } break;
80186 |     }
80187 | }
80188 | MA_PRIVATE MA_INLINE ma_bool32 ma_dr_wav_is_container_be(ma_dr_wav_container container)
80189 | {
80190 |     if (container == ma_dr_wav_container_rifx || container == ma_dr_wav_container_aiff) {
80191 |         return MA_TRUE;
80192 |     } else {
80193 |         return MA_FALSE;
80194 |     }
80195 | }
80196 | MA_PRIVATE MA_INLINE ma_uint16 ma_dr_wav_bytes_to_u16_le(const ma_uint8* data)
80197 | {
80198 |     return ((ma_uint16)data[0] << 0) | ((ma_uint16)data[1] << 8);
80199 | }
80200 | MA_PRIVATE MA_INLINE ma_uint16 ma_dr_wav_bytes_to_u16_be(const ma_uint8* data)
80201 | {
80202 |     return ((ma_uint16)data[1] << 0) | ((ma_uint16)data[0] << 8);
80203 | }
80204 | MA_PRIVATE MA_INLINE ma_uint16 ma_dr_wav_bytes_to_u16_ex(const ma_uint8* data, ma_dr_wav_container
> container)
80205 | {
80206 |     if (ma_dr_wav_is_container_be(container)) {
80207 |         return ma_dr_wav_bytes_to_u16_be(data);
80208 |     } else {
80209 |         return ma_dr_wav_bytes_to_u16_le(data);
80210 |     }
80211 | }
80212 | MA_PRIVATE MA_INLINE ma_uint32 ma_dr_wav_bytes_to_u32_le(const ma_uint8* data)
80213 | {
80214 |     return ((ma_uint32)data[0] << 0) | ((ma_uint32)data[1] << 8) | ((ma_uint32)data[2] << 16) |
> ((ma_uint32)data[3] << 24);
80215 | }
80216 | MA_PRIVATE MA_INLINE ma_uint32 ma_dr_wav_bytes_to_u32_be(const ma_uint8* data)
80217 | {
80218 |     return ((ma_uint32)data[3] << 0) | ((ma_uint32)data[2] << 8) | ((ma_uint32)data[1] << 16) |
> ((ma_uint32)data[0] << 24);
80219 | }
80220 | MA_PRIVATE MA_INLINE ma_uint32 ma_dr_wav_bytes_to_u32_ex(const ma_uint8* data, ma_dr_wav_container
> container)
80221 | {
80222 |     if (ma_dr_wav_is_container_be(container)) {
80223 |         return ma_dr_wav_bytes_to_u32_be(data);
80224 |     } else {
80225 |         return ma_dr_wav_bytes_to_u32_le(data);
80226 |     }
80227 | }
80228 | MA_PRIVATE ma_int64 ma_dr_wav_aiff_extented_to_s64(const ma_uint8* data)
80229 | {
80230 |     ma_uint32 exponent = ((ma_uint32)data[0] << 8) | data[1];
80231 |     ma_uint64 hi = ((ma_uint64)data[2] << 24) | ((ma_uint64)data[3] << 16) | ((ma_uint64)data[4] <<
> 8) | ((ma_uint64)data[5] << 0);
80232 |     ma_uint64 lo = ((ma_uint64)data[6] << 24) | ((ma_uint64)data[7] << 16) | ((ma_uint64)data[8] <<
> 8) | ((ma_uint64)data[9] << 0);
80233 |     ma_uint64 significand = (hi << 32) | lo;
80234 |     int sign = exponent >> 15;
80235 |     exponent &= 0x7FFF;

```

```

80236 |     if (exponent == 0 && significand == 0) {
80237 |         return 0;
80238 |     } else if (exponent == 0x7FFF) {
80239 |         return sign ? MA_DR_WAV_INT64_MIN : MA_DR_WAV_INT64_MAX;
80240 |     }
80241 |     exponent -= 16383;
80242 |     if (exponent > 63) {
80243 |         return sign ? MA_DR_WAV_INT64_MIN : MA_DR_WAV_INT64_MAX;
80244 |     } else if (exponent < 1) {
80245 |         return 0;
80246 |     }
80247 |     significand >= (63 - exponent);
80248 |     if (sign) {
80249 |         return -(ma_int64)significand;
80250 |     } else {
80251 |         return (ma_int64)significand;
80252 |     }
80253 | }
80254 | MA_PRIVATE void* ma_dr_wav__malloc_default(size_t sz, void* pUserData)
80255 | {
80256 |     (void)pUserData;
80257 |     return MA_DR_WAV_MALLOC(sz);
80258 | }
80259 | MA_PRIVATE void* ma_dr_wav__realloc_default(void* p, size_t sz, void* pUserData)
80260 | {
80261 |     (void)pUserData;
80262 |     return MA_DR_WAV_REALLOC(p, sz);
80263 | }
80264 | MA_PRIVATE void ma_dr_wav__free_default(void* p, void* pUserData)
80265 | {
80266 |     (void)pUserData;
80267 |     MA_DR_WAV_FREE(p);
80268 | }
80269 | MA_PRIVATE void* ma_dr_wav__malloc_from_callbacks(size_t sz, const ma_allocation_callbacks*
> pAllocationCallbacks)
80270 | {
80271 |     if (pAllocationCallbacks == NULL) {
80272 |         return NULL;
80273 |     }
80274 |     if (pAllocationCallbacks->onMalloc != NULL) {
80275 |         return pAllocationCallbacks->onMalloc(sz, pAllocationCallbacks->pUserData);
80276 |     }
80277 |     if (pAllocationCallbacks->onRealloc != NULL) {
80278 |         return pAllocationCallbacks->onRealloc(NULL, sz, pAllocationCallbacks->pUserData);
80279 |     }
80280 |     return NULL;
80281 | }
80282 | MA_PRIVATE void* ma_dr_wav__realloc_from_callbacks(void* p, size_t szNew, size_t szOld, const
> ma_allocation_callbacks* pAllocationCallbacks)
80283 | {
80284 |     if (pAllocationCallbacks == NULL) {
80285 |         return NULL;
80286 |     }
80287 |     if (pAllocationCallbacks->onRealloc != NULL) {
80288 |         return pAllocationCallbacks->onRealloc(p, szNew, pAllocationCallbacks->pUserData);
80289 |     }
80290 |     if (pAllocationCallbacks->onMalloc != NULL && pAllocationCallbacks->onFree != NULL) {
80291 |         void* p2;
80292 |         p2 = pAllocationCallbacks->onMalloc(szNew, pAllocationCallbacks->pUserData);
80293 |         if (p2 == NULL) {
80294 |             return NULL;
80295 |         }
80296 |         if (p != NULL) {
80297 |             MA_DR_WAV_COPY_MEMORY(p2, p, szOld);
80298 |             pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
80299 |         }
80300 |         return p2;
80301 |     }
80302 |     return NULL;
80303 | }
80304 | MA_PRIVATE void ma_dr_wav__free_from_callbacks(void* p, const ma_allocation_callbacks*
> pAllocationCallbacks)
80305 | {
80306 |     if (p == NULL || pAllocationCallbacks == NULL) {
80307 |         return;
80308 |     }
80309 |     if (pAllocationCallbacks->onFree != NULL) {
80310 |         pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
80311 |     }

```

```

80312 | }
80313 | MA_PRIVATE ma_allocation_callbacks ma_dr_wav_copy_allocation_callbacks_or_defaults(const
> ma_allocation_callbacks* pAllocationCallbacks)
80314 | {
80315 |     if (pAllocationCallbacks != NULL) {
80316 |         return *pAllocationCallbacks;
80317 |     } else {
80318 |         ma_allocation_callbacks allocationCallbacks;
80319 |         allocationCallbacks.pUserData = NULL;
80320 |         allocationCallbacks.onMalloc = ma_dr_wav__malloc_default;
80321 |         allocationCallbacks.onRealloc = ma_dr_wav__realloc_default;
80322 |         allocationCallbacks.onFree = ma_dr_wav__free_default;
80323 |         return allocationCallbacks;
80324 |     }
80325 | }
80326 | static MA_INLINE ma_bool32 ma_dr_wav__is_compressed_format_tag(ma_uint16 formatTag)
80327 | {
80328 |     return
80329 |         formatTag == MA_DR_WAVE_FORMAT_ADPCM ||
80330 |         formatTag == MA_DR_WAVE_FORMAT_DVI_ADPCM;
80331 | }
80332 | MA_PRIVATE unsigned int ma_dr_wav__chunk_padding_size_riff(ma_uint64 chunkSize)
80333 | {
80334 |     return (unsigned int)(chunkSize % 2);
80335 | }
80336 | MA_PRIVATE unsigned int ma_dr_wav__chunk_padding_size_w64(ma_uint64 chunkSize)
80337 | {
80338 |     return (unsigned int)(chunkSize % 8);
80339 | }
80340 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16_msadpcm(ma_dr_wav* pWav, ma_uint64
> samplesToRead, ma_int16* pBufferOut);
80341 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16_ima(ma_dr_wav* pWav, ma_uint64 samplesToRead,
> ma_int16* pBufferOut);
80342 | MA_PRIVATE ma_bool32 ma_dr_wav_init_write_internal(ma_dr_wav* pWav, const ma_dr_wav_data_format*
> pFormat, ma_uint64 totalSampleCount);
80343 | MA_PRIVATE ma_result ma_dr_wav__read_chunk_header(ma_dr_wav_read_proc onRead, void* pUserData,
> ma_dr_wav_container container, ma_uint64* pRunningBytesReadOut, ma_dr_wav_chunk_header* pHeaderOut)
80344 | {
80345 |     if (container == ma_dr_wav_container_riff || container == ma_dr_wav_container_rifx || container
> == ma_dr_wav_container_rf64 || container == ma_dr_wav_container_aiff) {
80346 |         ma_uint8 sizeInBytes[4];
80347 |         if (onRead(pUserData, pHeaderOut->id.fourcc, 4) != 4) {
80348 |             return MA_AT_END;
80349 |         }
80350 |         if (onRead(pUserData, sizeInBytes, 4) != 4) {
80351 |             return MA_INVALID_FILE;
80352 |         }
80353 |         pHeaderOut->sizeInBytes = ma_dr_wav_bytes_to_u32_ex(sizeInBytes, container);
80354 |         pHeaderOut->paddingSize = ma_dr_wav__chunk_padding_size_riff(pHeaderOut->sizeInBytes);
80355 |         *pRunningBytesReadOut += 8;
80356 |     } else if (container == ma_dr_wav_container_w64) {
80357 |         ma_uint8 sizeInBytes[8];
80358 |         if (onRead(pUserData, pHeaderOut->id.guid, 16) != 16) {
80359 |             return MA_AT_END;
80360 |         }
80361 |         if (onRead(pUserData, sizeInBytes, 8) != 8) {
80362 |             return MA_INVALID_FILE;
80363 |         }
80364 |         pHeaderOut->sizeInBytes = ma_dr_wav_bytes_to_u64(sizeInBytes) - 24;
80365 |         pHeaderOut->paddingSize = ma_dr_wav__chunk_padding_size_w64(pHeaderOut->sizeInBytes);
80366 |         *pRunningBytesReadOut += 24;
80367 |     } else {
80368 |         return MA_INVALID_FILE;
80369 |     }
80370 |     return MA_SUCCESS;
80371 | }
80372 | MA_PRIVATE ma_bool32 ma_dr_wav__seek_forward(ma_dr_wav_seek_proc onSeek, ma_uint64 offset, void*
> pUserData)
80373 | {
80374 |     ma_uint64 bytesRemainingToSeek = offset;
80375 |     while (bytesRemainingToSeek > 0) {
80376 |         if (bytesRemainingToSeek > 0x7FFFFFFF) {
80377 |             if (!onSeek(pUserData, 0x7FFFFFFF, MA_DR_WAV_SEEK_CUR)) {
80378 |                 return MA_FALSE;
80379 |             }
80380 |             bytesRemainingToSeek -= 0x7FFFFFFF;
80381 |         } else {
80382 |             if (!onSeek(pUserData, (int)bytesRemainingToSeek, MA_DR_WAV_SEEK_CUR)) {
80383 |                 return MA_FALSE;

```

```

80384 |         }
80385 |         bytesRemainingToSeek = 0;
80386 |     }
80387 | }
80388 |     return MA_TRUE;
80389 | }
80390 | MA_PRIVATE ma_bool32 ma_dr_wav__seek_from_start(ma_dr_wav_seek_proc onSeek, ma_uint64 offset, void*
> pUserData)
80391 | {
80392 |     if (offset <= 0x7FFFFFFF) {
80393 |         return onSeek(pUserData, (int)offset, MA_DR_WAV_SEEK_SET);
80394 |     }
80395 |     if (!onSeek(pUserData, 0x7FFFFFFF, MA_DR_WAV_SEEK_SET)) {
80396 |         return MA_FALSE;
80397 |     }
80398 |     offset -= 0x7FFFFFFF;
80399 |     for (;;) {
80400 |         if (offset <= 0x7FFFFFFF) {
80401 |             return onSeek(pUserData, (int)offset, MA_DR_WAV_SEEK_CUR);
80402 |         }
80403 |         if (!onSeek(pUserData, 0x7FFFFFFF, MA_DR_WAV_SEEK_CUR)) {
80404 |             return MA_FALSE;
80405 |         }
80406 |         offset -= 0x7FFFFFFF;
80407 |     }
80408 | }
80409 | MA_PRIVATE size_t ma_dr_wav__on_read(ma_dr_wav_read_proc onRead, void* pUserData, void* pBufferOut,
> size_t bytesToRead, ma_uint64* pCursor)
80410 | {
80411 |     size_t bytesRead;
80412 |     MA_DR_WAV_ASSERT(onRead != NULL);
80413 |     MA_DR_WAV_ASSERT(pCursor != NULL);
80414 |     bytesRead = onRead(pUserData, pBufferOut, bytesToRead);
80415 |     *pCursor += bytesRead;
80416 |     return bytesRead;
80417 | }
80418 | #if 0
80419 | MA_PRIVATE ma_bool32 ma_dr_wav__on_seek(ma_dr_wav_seek_proc onSeek, void* pUserData, int offset,
> ma_dr_wav_seek_origin origin, ma_uint64* pCursor)
80420 | {
80421 |     MA_DR_WAV_ASSERT(onSeek != NULL);
80422 |     MA_DR_WAV_ASSERT(pCursor != NULL);
80423 |     if (!onSeek(pUserData, offset, origin)) {
80424 |         return MA_FALSE;
80425 |     }
80426 |     if (origin == MA_DR_WAV_SEEK_SET) {
80427 |         *pCursor = offset;
80428 |     } else {
80429 |         *pCursor += offset;
80430 |     }
80431 |     return MA_TRUE;
80432 | }
80433 | #endif
80434 | #define MA_DR_WAV_SMPL_BYTES 36
80435 | #define MA_DR_WAV_SMPL_LOOP_BYTES 24
80436 | #define MA_DR_WAV_INST_BYTES 7
80437 | #define MA_DR_WAV_ACID_BYTES 24
80438 | #define MA_DR_WAV_CUE_BYTES 4
80439 | #define MA_DR_WAV_BEXT_BYTES 602
80440 | #define MA_DR_WAV_BEXT_DESCRIPTION_BYTES 256
80441 | #define MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES 32
80442 | #define MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES 32
80443 | #define MA_DR_WAV_BEXT_RESERVED_BYTES 180
80444 | #define MA_DR_WAV_BEXT_UMID_BYTES 64
80445 | #define MA_DR_WAV_CUE_POINT_BYTES 24
80446 | #define MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES 4
80447 | #define MA_DR_WAV_LIST_LABELLED_TEXT_BYTES 20
80448 | #define MA_DR_WAV_METADATA_ALIGNMENT 8
80449 | typedef enum
80450 | {
80451 |     ma_dr_wav__metadata_parser_stage_count,
80452 |     ma_dr_wav__metadata_parser_stage_read
80453 | } ma_dr_wav__metadata_parser_stage;
80454 | typedef struct
80455 | {
80456 |     ma_dr_wav_read_proc onRead;
80457 |     ma_dr_wav_seek_proc onSeek;
80458 |     void *pReadSeekUserData;
80459 |     ma_dr_wav__metadata_parser_stage stage;

```

```

80460 |     ma_dr_wav_metadata *pMetadata;
80461 |     ma_uint32 metadataCount;
80462 |     ma_uint8 *pData;
80463 |     ma_uint8 *pDataCursor;
80464 |     ma_uint64 metadataCursor;
80465 |     ma_uint64 extraCapacity;
80466 | } ma_dr_wav__metadata_parser;
80467 | MA_PRIVATE size_t ma_dr_wav__metadata_memory_capacity(ma_dr_wav__metadata_parser* pParser)
80468 | {
80469 |     ma_uint64 cap = sizeof(ma_dr_wav_metadata) * (ma_uint64)pParser->metadataCount +
> pParser->extraCapacity;
80470 |     if (cap > MA_SIZE_MAX) {
80471 |         return 0;
80472 |     }
80473 |     return (size_t)cap;
80474 | }
80475 | MA_PRIVATE ma_uint8* ma_dr_wav__metadata_get_memory(ma_dr_wav__metadata_parser* pParser, size_t
> size, size_t align)
80476 | {
80477 |     ma_uint8* pResult;
80478 |     if (align) {
80479 |         ma_uintptr modulo = (ma_uintptr)pParser->pDataCursor % align;
80480 |         if (modulo != 0) {
80481 |             pParser->pDataCursor += align - modulo;
80482 |         }
80483 |     }
80484 |     pResult = pParser->pDataCursor;
80485 |     MA_DR_WAV_ASSERT((pResult + size) <= (pParser->pData +
> ma_dr_wav__metadata_memory_capacity(pParser)));
80486 |     pParser->pDataCursor += size;
80487 |     return pResult;
80488 | }
80489 | MA_PRIVATE void ma_dr_wav__metadata_request_extra_memory_for_stage_2(ma_dr_wav__metadata_parser*
> pParser, size_t bytes, size_t align)
80490 | {
80491 |     size_t extra = bytes + (align ? (align - 1) : 0);
80492 |     pParser->extraCapacity += extra;
80493 | }
80494 | MA_PRIVATE ma_result ma_dr_wav__metadata_alloc(ma_dr_wav__metadata_parser* pParser,
> ma_allocation_callbacks* pAllocationCallbacks)
80495 | {
80496 |     if (pParser->extraCapacity != 0 || pParser->metadataCount != 0) {
80497 |         pAllocationCallbacks->onFree(pParser->pData, pAllocationCallbacks->pUserData);
80498 |         pParser->pData =
> (ma_uint8*)pAllocationCallbacks->onMalloc(ma_dr_wav__metadata_memory_capacity(pParser),
> pAllocationCallbacks->pUserData);
80499 |         pParser->pDataCursor = pParser->pData;
80500 |         if (pParser->pData == NULL) {
80501 |             return MA_OUT_OF_MEMORY;
80502 |         }
80503 |         pParser->pMetadata = (ma_dr_wav_metadata*)ma_dr_wav__metadata_get_memory(pParser,
> sizeof(ma_dr_wav_metadata) * pParser->metadataCount, 1);
80504 |         pParser->metadataCursor = 0;
80505 |     }
80506 |     return MA_SUCCESS;
80507 | }
80508 | MA_PRIVATE size_t ma_dr_wav__metadata_parser_read(ma_dr_wav__metadata_parser* pParser, void*
> pBufferOut, size_t bytesToRead, ma_uint64* pCursor)
80509 | {
80510 |     if (pCursor != NULL) {
80511 |         return ma_dr_wav__on_read(pParser->onRead, pParser->pReadSeekUserData, pBufferOut,
> bytesToRead, pCursor);
80512 |     } else {
80513 |         return pParser->onRead(pParser->pReadSeekUserData, pBufferOut, bytesToRead);
80514 |     }
80515 | }
80516 | MA_PRIVATE ma_uint64 ma_dr_wav__read_smpl_to_metadata_obj(ma_dr_wav__metadata_parser* pParser, const
> ma_dr_wav_chunk_header* pChunkHeader, ma_dr_wav_metadata* pMetadata)
80517 | {
80518 |     ma_uint8 smplHeaderData[MA_DR_WAV_SMPL_BYTES];
80519 |     ma_uint64 totalBytesRead = 0;
80520 |     size_t bytesJustRead;
80521 |     if (pMetadata == NULL) {
80522 |         return 0;
80523 |     }
80524 |     bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, smplHeaderData, sizeof(smplHeaderData),
> &totalBytesRead);
80525 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80526 |     MA_DR_WAV_ASSERT(pChunkHeader != NULL);

```



```

80527 |     if (pMetadata != NULL && bytesJustRead == sizeof(smplHeaderData)) {
80528 |         ma_uint32 iSampleLoop;
80529 |         ma_uint32 loopCount;
80530 |         ma_uint32 calculatedLoopCount;
80531 |         loopCount = ma_dr_wav_bytes_to_u32(smplHeaderData + 28);
80532 |         calculatedLoopCount = (pChunkHeader->sizeInBytes - MA_DR_WAV_SMPL_BYTES) /
> MA_DR_WAV_SMPL_LOOP_BYTES;
80533 |         if (loopCount != calculatedLoopCount) {
80534 |             return totalBytesRead;
80535 |         }
80536 |         pMetadata->type = ma_dr_wav_metadata_type_smpl;
80537 |         pMetadata->data.smpl.manufacturerId = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 0);
80538 |         pMetadata->data.smpl.productId = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 4);
80539 |         pMetadata->data.smpl.samplePeriodNanoseconds = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 8);
80540 |         pMetadata->data.smpl.midiUnityNote = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 12);
80541 |         pMetadata->data.smpl.midiPitchFraction = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 16);
80542 |         pMetadata->data.smpl.smpteFormat = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 20);
80543 |         pMetadata->data.smpl.smpteOffset = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 24);
80544 |         pMetadata->data.smpl.sampleLoopCount = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 28);
80545 |         pMetadata->data.smpl.samplerSpecificDataSizeInBytes = ma_dr_wav_bytes_to_u32(smplHeaderData
> + 32);
80546 |         if (pMetadata->data.smpl.sampleLoopCount == calculatedLoopCount) {
80547 |             pMetadata->data.smpl.pLoops =
> (ma_dr_wav_smpl_loop*)ma_dr_wav__metadata_get_memory(pParser, sizeof(ma_dr_wav_smpl_loop) *
> pMetadata->data.smpl.sampleLoopCount, MA_DR_WAV_METADATA_ALIGNMENT);
80548 |             for (iSampleLoop = 0; iSampleLoop < pMetadata->data.smpl.sampleLoopCount; ++iSampleLoop)
> {
80549 |                 ma_uint8 smplLoopData[MA_DR_WAV_SMPL_LOOP_BYTES];
80550 |                 bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, smplLoopData,
> sizeof(smplLoopData), &totalBytesRead);
80551 |                 if (bytesJustRead == sizeof(smplLoopData)) {
80552 |                     pMetadata->data.smpl.pLoops[iSampleLoop].cuePointId =
> ma_dr_wav_bytes_to_u32(smplLoopData + 0);
80553 |                     pMetadata->data.smpl.pLoops[iSampleLoop].type =
> ma_dr_wav_bytes_to_u32(smplLoopData + 4);
80554 |                     pMetadata->data.smpl.pLoops[iSampleLoop].firstSampleOffset =
> ma_dr_wav_bytes_to_u32(smplLoopData + 8);
80555 |                     pMetadata->data.smpl.pLoops[iSampleLoop].lastSampleOffset =
> ma_dr_wav_bytes_to_u32(smplLoopData + 12);
80556 |                     pMetadata->data.smpl.pLoops[iSampleLoop].sampleFraction =
> ma_dr_wav_bytes_to_u32(smplLoopData + 16);
80557 |                     pMetadata->data.smpl.pLoops[iSampleLoop].playCount =
> ma_dr_wav_bytes_to_u32(smplLoopData + 20);
80558 |                 } else {
80559 |                     break;
80560 |                 }
80561 |             }
80562 |             if (pMetadata->data.smpl.samplerSpecificDataSizeInBytes > 0) {
80563 |                 pMetadata->data.smpl.pSamplerSpecificData = ma_dr_wav__metadata_get_memory(pParser,
> pMetadata->data.smpl.samplerSpecificDataSizeInBytes, 1);
80564 |                 MA_DR_WAV_ASSERT(pMetadata->data.smpl.pSamplerSpecificData != NULL);
80565 |                 ma_dr_wav__metadata_parser_read(pParser, pMetadata->data.smpl.pSamplerSpecificData,
> pMetadata->data.smpl.samplerSpecificDataSizeInBytes, &totalBytesRead);
80566 |             }
80567 |             } else {
80568 |                 MA_DR_WAV_ZERO_OBJECT(&pMetadata->data.smpl);
80569 |             }
80570 |         }
80571 |         return totalBytesRead;
80572 |     }
80573 | MA_PRIVATE ma_uint64 ma_dr_wav__read_cue_to_metadata_obj(ma_dr_wav__metadata_parser* pParser, const
> ma_dr_wav_chunk_header* pChunkHeader, ma_dr_wav_metadata* pMetadata)
80574 | {
80575 |     ma_uint8 cueHeaderSectionData[MA_DR_WAV_CUE_BYTES];
80576 |     ma_uint64 totalBytesRead = 0;
80577 |     size_t bytesJustRead;
80578 |     if (pMetadata == NULL) {
80579 |         return 0;
80580 |     }
80581 |     bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, cueHeaderSectionData,
> sizeof(cueHeaderSectionData), &totalBytesRead);

```

```

80582 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80583 |     if (bytesJustRead == sizeof(cueHeaderSectionData)) {
80584 |         pMetadata->type = ma_dr_wav_metadata_type_cue;
80585 |         pMetadata->data.cue.cuePointCount = ma_dr_wav_bytes_to_u32(cueHeaderSectionData);
80586 |         if (pMetadata->data.cue.cuePointCount == (pChunkHeader->sizeInBytes - MA_DR_WAV_CUE_BYTES) /
> MA_DR_WAV_CUE_POINT_BYTES) {
80587 |             pMetadata->data.cue.pCuePoints =
> (ma_dr_wav_cue_point*)ma_dr_wav__metadata_get_memory(pParser, sizeof(ma_dr_wav_cue_point) *
> pMetadata->data.cue.cuePointCount, MA_DR_WAV_METADATA_ALIGNMENT);
80588 |             MA_DR_WAV_ASSERT(pMetadata->data.cue.pCuePoints != NULL);
80589 |             if (pMetadata->data.cue.cuePointCount > 0) {
80590 |                 ma_uint32 iCuePoint;
80591 |                 for (iCuePoint = 0; iCuePoint < pMetadata->data.cue.cuePointCount; ++iCuePoint) {
80592 |                     ma_uint8 cuePointData[MA_DR_WAV_CUE_POINT_BYTES];
80593 |                     bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, cuePointData,
> sizeof(cuePointData), &totalBytesRead);
80594 |                     if (bytesJustRead == sizeof(cuePointData)) {
80595 |                         pMetadata->data.cue.pCuePoints[iCuePoint].id =
> ma_dr_wav_bytes_to_u32(cuePointData + 0);
80596 |                         pMetadata->data.cue.pCuePoints[iCuePoint].playOrderPosition =
> ma_dr_wav_bytes_to_u32(cuePointData + 4);
80597 |                         pMetadata->data.cue.pCuePoints[iCuePoint].dataChunkId[0] =
> cuePointData[8];
80598 |                         pMetadata->data.cue.pCuePoints[iCuePoint].dataChunkId[1] =
> cuePointData[9];
80599 |                         pMetadata->data.cue.pCuePoints[iCuePoint].dataChunkId[2] =
> cuePointData[10];
80600 |                         pMetadata->data.cue.pCuePoints[iCuePoint].dataChunkId[3] =
> cuePointData[11];
80601 |                         pMetadata->data.cue.pCuePoints[iCuePoint].chunkStart =
> ma_dr_wav_bytes_to_u32(cuePointData + 12);
80602 |                         pMetadata->data.cue.pCuePoints[iCuePoint].blockStart =
> ma_dr_wav_bytes_to_u32(cuePointData + 16);
80603 |                         pMetadata->data.cue.pCuePoints[iCuePoint].sampleOffset =
> ma_dr_wav_bytes_to_u32(cuePointData + 20);
80604 |                     } else {
80605 |                         break;
80606 |                     }
80607 |                 }
80608 |             }
80609 |         }
80610 |     }
80611 |     return totalBytesRead;
80612 | }
80613 | MA_PRIVATE ma_uint64 ma_dr_wav__read_inst_to_metadata_obj(ma_dr_wav__metadata_parser* pParser,
> ma_dr_wav_metadata* pMetadata)
80614 | {
80615 |     ma_uint8 instData[MA_DR_WAV_INST_BYTES];
80616 |     ma_uint64 bytesRead;
80617 |     if (pMetadata == NULL) {
80618 |         return 0;
80619 |     }
80620 |     bytesRead = ma_dr_wav__metadata_parser_read(pParser, instData, sizeof(instData), NULL);
80621 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80622 |     if (bytesRead == sizeof(instData)) {
80623 |         pMetadata->type = ma_dr_wav_metadata_type_inst;
80624 |         pMetadata->data.inst.midiUnityNote = (ma_int8)instData[0];
80625 |         pMetadata->data.inst.fineTuneCents = (ma_int8)instData[1];
80626 |         pMetadata->data.inst.gainDecibels = (ma_int8)instData[2];
80627 |         pMetadata->data.inst.lowNote = (ma_int8)instData[3];
80628 |         pMetadata->data.inst.highNote = (ma_int8)instData[4];
80629 |         pMetadata->data.inst.lowVelocity = (ma_int8)instData[5];
80630 |         pMetadata->data.inst.highVelocity = (ma_int8)instData[6];
80631 |     }
80632 |     return bytesRead;
80633 | }
80634 | MA_PRIVATE ma_uint64 ma_dr_wav__read_acid_to_metadata_obj(ma_dr_wav__metadata_parser* pParser,
> ma_dr_wav_metadata* pMetadata)
80635 | {
80636 |     ma_uint8 acidData[MA_DR_WAV_ACID_BYTES];
80637 |     ma_uint64 bytesRead;
80638 |     if (pMetadata == NULL) {
80639 |         return 0;
80640 |     }
80641 |     bytesRead = ma_dr_wav__metadata_parser_read(pParser, acidData, sizeof(acidData), NULL);
80642 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80643 |     if (bytesRead == sizeof(acidData)) {
80644 |         pMetadata->type = ma_dr_wav_metadata_type_acid;
80645 |         pMetadata->data.acid.flags = ma_dr_wav_bytes_to_u32(acidData + 0);

```

```

80646 |         pMetadata->data.acid.midiUnityNote    = ma_dr_wav_bytes_to_u16(acidData + 4);
80647 |         pMetadata->data.acid.reserved1        = ma_dr_wav_bytes_to_u16(acidData + 6);
80648 |         pMetadata->data.acid.reserved2        = ma_dr_wav_bytes_to_f32(acidData + 8);
80649 |         pMetadata->data.acid.numBeats         = ma_dr_wav_bytes_to_u32(acidData + 12);
80650 |         pMetadata->data.acid.meterDenominator = ma_dr_wav_bytes_to_u16(acidData + 16);
80651 |         pMetadata->data.acid.meterNumerator   = ma_dr_wav_bytes_to_u16(acidData + 18);
80652 |         pMetadata->data.acid.tempo            = ma_dr_wav_bytes_to_f32(acidData + 20);
80653 |     }
80654 |     return bytesRead;
80655 | }
80656 | MA_PRIVATE size_t ma_dr_wav__strlen(const char* str)
80657 | {
80658 |     size_t result = 0;
80659 |     while (*str++) {
80660 |         result += 1;
80661 |     }
80662 |     return result;
80663 | }
80664 | MA_PRIVATE size_t ma_dr_wav__strlen_clamped(const char* str, size_t maxToRead)
80665 | {
80666 |     size_t result = 0;
80667 |     while (*str++ && result < maxToRead) {
80668 |         result += 1;
80669 |     }
80670 |     return result;
80671 | }
80672 | MA_PRIVATE char* ma_dr_wav__metadata_copy_string(ma_dr_wav__metadata_parser* pParser, const char*
> str, size_t maxToRead)
80673 | {
80674 |     size_t len = ma_dr_wav__strlen_clamped(str, maxToRead);
80675 |     if (len) {
80676 |         char* result = (char*)ma_dr_wav__metadata_get_memory(pParser, len + 1, 1);
80677 |         MA_DR_WAV_ASSERT(result != NULL);
80678 |         MA_DR_WAV_COPY_MEMORY(result, str, len);
80679 |         result[len] = '\0';
80680 |         return result;
80681 |     } else {
80682 |         return NULL;
80683 |     }
80684 | }
80685 | typedef struct
80686 | {
80687 |     const void* pBuffer;
80688 |     size_t sizeInBytes;
80689 |     size_t cursor;
80690 | } ma_dr_wav_buffer_reader;
80691 | MA_PRIVATE ma_result ma_dr_wav_buffer_reader_init(const void* pBuffer, size_t sizeInBytes,
> ma_dr_wav_buffer_reader* pReader)
80692 | {
80693 |     MA_DR_WAV_ASSERT(pBuffer != NULL);
80694 |     MA_DR_WAV_ASSERT(pReader != NULL);
80695 |     MA_DR_WAV_ZERO_OBJECT(pReader);
80696 |     pReader->pBuffer    = pBuffer;
80697 |     pReader->sizeInBytes = sizeInBytes;
80698 |     pReader->cursor      = 0;
80699 |     return MA_SUCCESS;
80700 | }
80701 | MA_PRIVATE const void* ma_dr_wav_buffer_reader_ptr(const ma_dr_wav_buffer_reader* pReader)
80702 | {
80703 |     MA_DR_WAV_ASSERT(pReader != NULL);
80704 |     return ma_dr_wav_offset_ptr(pReader->pBuffer, pReader->cursor);
80705 | }
80706 | MA_PRIVATE ma_result ma_dr_wav_buffer_reader_seek(ma_dr_wav_buffer_reader* pReader, size_t
> bytesToSeek)
80707 | {
80708 |     MA_DR_WAV_ASSERT(pReader != NULL);
80709 |     if (pReader->cursor + bytesToSeek > pReader->sizeInBytes) {
80710 |         return MA_BAD_SEEK;
80711 |     }
80712 |     pReader->cursor += bytesToSeek;
80713 |     return MA_SUCCESS;
80714 | }
80715 | MA_PRIVATE ma_result ma_dr_wav_buffer_reader_read(ma_dr_wav_buffer_reader* pReader, void* pDst,
> size_t bytesToRead, size_t* pBytesRead)
80716 | {
80717 |     ma_result result = MA_SUCCESS;
80718 |     size_t bytesRemaining;
80719 |     MA_DR_WAV_ASSERT(pReader != NULL);
80720 |     if (pBytesRead != NULL) {

```

```

80721 |         *pBytesRead = 0;
80722 |     }
80723 |     bytesRemaining = (pReader->sizeInBytes - pReader->cursor);
80724 |     if (bytesToRead > bytesRemaining) {
80725 |         bytesToRead = bytesRemaining;
80726 |     }
80727 |     if (pDst == NULL) {
80728 |         result = ma_dr_wav_buffer_reader_seek(pReader, bytesToRead);
80729 |     } else {
80730 |         MA_DR_WAV_COPY_MEMORY(pDst, ma_dr_wav_buffer_reader_ptr(pReader), bytesToRead);
80731 |         pReader->cursor += bytesToRead;
80732 |     }
80733 |     MA_DR_WAV_ASSERT(pReader->cursor <= pReader->sizeInBytes);
80734 |     if (result == MA_SUCCESS) {
80735 |         if (pBytesRead != NULL) {
80736 |             *pBytesRead = bytesToRead;
80737 |         }
80738 |     }
80739 |     return MA_SUCCESS;
80740 | }
80741 | MA_PRIVATE ma_result ma_dr_wav_buffer_reader_read_u16(ma_dr_wav_buffer_reader* pReader, ma_uint16*
> pDst)
80742 | {
80743 |     ma_result result;
80744 |     size_t bytesRead;
80745 |     ma_uint8 data[2];
80746 |     MA_DR_WAV_ASSERT(pReader != NULL);
80747 |     MA_DR_WAV_ASSERT(pDst != NULL);
80748 |     *pDst = 0;
80749 |     result = ma_dr_wav_buffer_reader_read(pReader, data, sizeof(*pDst), &bytesRead);
80750 |     if (result != MA_SUCCESS || bytesRead != sizeof(*pDst)) {
80751 |         return result;
80752 |     }
80753 |     *pDst = ma_dr_wav_bytes_to_u16(data);
80754 |     return MA_SUCCESS;
80755 | }
80756 | MA_PRIVATE ma_result ma_dr_wav_buffer_reader_read_u32(ma_dr_wav_buffer_reader* pReader, ma_uint32*
> pDst)
80757 | {
80758 |     ma_result result;
80759 |     size_t bytesRead;
80760 |     ma_uint8 data[4];
80761 |     MA_DR_WAV_ASSERT(pReader != NULL);
80762 |     MA_DR_WAV_ASSERT(pDst != NULL);
80763 |     *pDst = 0;
80764 |     result = ma_dr_wav_buffer_reader_read(pReader, data, sizeof(*pDst), &bytesRead);
80765 |     if (result != MA_SUCCESS || bytesRead != sizeof(*pDst)) {
80766 |         return result;
80767 |     }
80768 |     *pDst = ma_dr_wav_bytes_to_u32(data);
80769 |     return MA_SUCCESS;
80770 | }
80771 | MA_PRIVATE ma_uint64 ma_dr_wav__read_bext_to_metadata_obj(ma_dr_wav__metadata_parser* pParser,
> ma_dr_wav_metadata* pMetadata, ma_uint64 chunkSize)
80772 | {
80773 |     ma_uint8 bextData[MA_DR_WAV_BEXT_BYTES];
80774 |     size_t bytesRead = ma_dr_wav__metadata_parser_read(pParser, bextData, sizeof(bextData), NULL);
80775 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80776 |     if (bytesRead == sizeof(bextData)) {
80777 |         ma_dr_wav_buffer_reader reader;
80778 |         ma_uint32 timeReferenceLow;
80779 |         ma_uint32 timeReferenceHigh;
80780 |         size_t extraBytes;
80781 |         pMetadata->type = ma_dr_wav_metadata_type_bext;
80782 |         if (ma_dr_wav_buffer_reader_init(bextData, bytesRead, &reader) == MA_SUCCESS) {
80783 |             pMetadata->data.bext.pDescription = ma_dr_wav__metadata_copy_string(pParser, (const
> char*)ma_dr_wav_buffer_reader_ptr(&reader), MA_DR_WAV_BEXT_DESCRIPTION_BYTES);
80784 |             ma_dr_wav_buffer_reader_seek(&reader, MA_DR_WAV_BEXT_DESCRIPTION_BYTES);
80785 |             pMetadata->data.bext.pOriginatorName = ma_dr_wav__metadata_copy_string(pParser, (const
> char*)ma_dr_wav_buffer_reader_ptr(&reader), MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES);
80786 |             ma_dr_wav_buffer_reader_seek(&reader, MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES);
80787 |             pMetadata->data.bext.pOriginatorReference = ma_dr_wav__metadata_copy_string(pParser,
> (const char*)ma_dr_wav_buffer_reader_ptr(&reader), MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES);
80788 |             ma_dr_wav_buffer_reader_seek(&reader, MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES);
80789 |             ma_dr_wav_buffer_reader_read(&reader, pMetadata->data.bext.pOriginationDate,
> sizeof(pMetadata->data.bext.pOriginationDate), NULL);
80790 |             ma_dr_wav_buffer_reader_read(&reader, pMetadata->data.bext.pOriginationTime,
> sizeof(pMetadata->data.bext.pOriginationTime), NULL);
80791 |             ma_dr_wav_buffer_reader_read_u32(&reader, &timeReferenceLow);

```

```

80792 |         ma_dr_wav_buffer_reader_read_u32(&reader, &timeReferenceHigh);
80793 |         pMetadata->data.bext.timeReference = ((ma_uint64)timeReferenceHigh << 32) +
> timeReferenceLow;
80794 |         ma_dr_wav_buffer_reader_read_u16(&reader, &pMetadata->data.bext.version);
80795 |         pMetadata->data.bext.pUMID = ma_dr_wav__metadata_get_memory(pParser,
> MA_DR_WAV_BEXT_UMID_BYTES, 1);
80796 |         ma_dr_wav_buffer_reader_read(&reader, pMetadata->data.bext.pUMID,
> MA_DR_WAV_BEXT_UMID_BYTES, NULL);
80797 |         ma_dr_wav_buffer_reader_read_u16(&reader, &pMetadata->data.bext.loudnessValue);
80798 |         ma_dr_wav_buffer_reader_read_u16(&reader, &pMetadata->data.bext.loudnessRange);
80799 |         ma_dr_wav_buffer_reader_read_u16(&reader, &pMetadata->data.bext.maxTruePeakLevel);
80800 |         ma_dr_wav_buffer_reader_read_u16(&reader, &pMetadata->data.bext.maxMomentaryLoudness);
80801 |         ma_dr_wav_buffer_reader_read_u16(&reader, &pMetadata->data.bext.maxShortTermLoudness);
80802 |         MA_DR_WAV_ASSERT((ma_dr_wav_offset_ptr(ma_dr_wav_buffer_reader_ptr(&reader),
> MA_DR_WAV_BEXT_RESERVED_BYTES)) == (bextData + MA_DR_WAV_BEXT_BYTES));
80803 |         extraBytes = (size_t)(chunkSize - MA_DR_WAV_BEXT_BYTES);
80804 |         if (extraBytes > 0) {
80805 |             pMetadata->data.bext.pCodingHistory = (char*)ma_dr_wav__metadata_get_memory(pParser,
> extraBytes + 1, 1);
80806 |             MA_DR_WAV_ASSERT(pMetadata->data.bext.pCodingHistory != NULL);
80807 |             bytesRead += ma_dr_wav__metadata_parser_read(pParser,
> pMetadata->data.bext.pCodingHistory, extraBytes, NULL);
80808 |             pMetadata->data.bext.codingHistorySize =
> (ma_uint32)ma_dr_wav__strlen(pMetadata->data.bext.pCodingHistory);
80809 |             } else {
80810 |                 pMetadata->data.bext.pCodingHistory = NULL;
80811 |                 pMetadata->data.bext.codingHistorySize = 0;
80812 |             }
80813 |         }
80814 |     }
80815 |     return bytesRead;
80816 | }
80817 | MA_PRIVATE ma_uint64 ma_dr_wav__read_list_label_or_note_to_metadata_obj(ma_dr_wav__metadata_parser*
> pParser, ma_dr_wav_metadata* pMetadata, ma_uint64 chunkSize, ma_dr_wav_metadata_type type)
80818 | {
80819 |     ma_uint8 cueIDBuffer[MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES];
80820 |     ma_uint64 totalBytesRead = 0;
80821 |     size_t bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, cueIDBuffer,
> sizeof(cueIDBuffer), &totalBytesRead);
80822 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80823 |     if (bytesJustRead == sizeof(cueIDBuffer)) {
80824 |         ma_uint32 sizeIncludingNullTerminator;
80825 |         pMetadata->type = type;
80826 |         pMetadata->data.labelOrNote.cuePointId = ma_dr_wav_bytes_to_u32(cueIDBuffer);
80827 |         sizeIncludingNullTerminator = (ma_uint32)chunkSize - MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES;
80828 |         if (sizeIncludingNullTerminator > 0) {
80829 |             pMetadata->data.labelOrNote.stringLength = sizeIncludingNullTerminator - 1;
80830 |             pMetadata->data.labelOrNote.pString =
> (char*)ma_dr_wav__metadata_get_memory(pParser, sizeIncludingNullTerminator, 1);
80831 |             MA_DR_WAV_ASSERT(pMetadata->data.labelOrNote.pString != NULL);
80832 |             ma_dr_wav__metadata_parser_read(pParser, pMetadata->data.labelOrNote.pString,
> sizeIncludingNullTerminator, &totalBytesRead);
80833 |         } else {
80834 |             pMetadata->data.labelOrNote.stringLength = 0;
80835 |             pMetadata->data.labelOrNote.pString = NULL;
80836 |         }
80837 |     }
80838 |     return totalBytesRead;
80839 | }
80840 | MA_PRIVATE ma_uint64
> ma_dr_wav__read_list_labelled_cue_region_to_metadata_obj(ma_dr_wav__metadata_parser* pParser,
> ma_dr_wav_metadata* pMetadata, ma_uint64 chunkSize)
80841 | {
80842 |     ma_uint8 buffer[MA_DR_WAV_LIST_LABELLED_TEXT_BYTES];
80843 |     ma_uint64 totalBytesRead = 0;
80844 |     size_t bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, buffer, sizeof(buffer),
> &totalBytesRead);
80845 |     MA_DR_WAV_ASSERT(pParser->stage == ma_dr_wav__metadata_parser_stage_read);
80846 |     if (bytesJustRead == sizeof(buffer)) {
80847 |         ma_uint32 sizeIncludingNullTerminator;
80848 |         pMetadata->type =
> ma_dr_wav_metadata_type_list_labelled_cue_region;
80849 |         pMetadata->data.labelledCueRegion.cuePointId = ma_dr_wav_bytes_to_u32(buffer + 0);
80850 |         pMetadata->data.labelledCueRegion.sampleLength = ma_dr_wav_bytes_to_u32(buffer + 4);
80851 |         pMetadata->data.labelledCueRegion.purposeId[0] = buffer[8];
80852 |         pMetadata->data.labelledCueRegion.purposeId[1] = buffer[9];
80853 |         pMetadata->data.labelledCueRegion.purposeId[2] = buffer[10];
80854 |         pMetadata->data.labelledCueRegion.purposeId[3] = buffer[11];
80855 |         pMetadata->data.labelledCueRegion.country = ma_dr_wav_bytes_to_u16(buffer + 12);

```

```

80856 |         pMetadata->data.labelledCueRegion.language      = ma_dr_wav_bytes_to_u16(buffer + 14);
80857 |         pMetadata->data.labelledCueRegion.dialect       = ma_dr_wav_bytes_to_u16(buffer + 16);
80858 |         pMetadata->data.labelledCueRegion.codePage      = ma_dr_wav_bytes_to_u16(buffer + 18);
80859 |         sizeIncludingNullTerminator = (ma_uint32)chunkSize - MA_DR_WAV_LIST_LABELLED_TEXT_BYTES;
80860 |         if (sizeIncludingNullTerminator > 0) {
80861 |             pMetadata->data.labelledCueRegion.stringLength = sizeIncludingNullTerminator - 1;
80862 |             pMetadata->data.labelledCueRegion.pString      =
> (char*)ma_dr_wav__metadata_get_memory(pParser, sizeIncludingNullTerminator, 1);
80863 |             MA_DR_WAV_ASSERT(pMetadata->data.labelledCueRegion.pString != NULL);
80864 |             ma_dr_wav__metadata_parser_read(pParser, pMetadata->data.labelledCueRegion.pString,
> sizeIncludingNullTerminator, &totalBytesRead);
80865 |         } else {
80866 |             pMetadata->data.labelledCueRegion.stringLength = 0;
80867 |             pMetadata->data.labelledCueRegion.pString      = NULL;
80868 |         }
80869 |     }
80870 |     return totalBytesRead;
80871 | }
80872 | MA_PRIVATE ma_uint64 ma_dr_wav__metadata_process_info_text_chunk(ma_dr_wav__metadata_parser*
> pParser, ma_uint64 chunkSize, ma_dr_wav_metadata_type type)
80873 | {
80874 |     ma_uint64 bytesRead = 0;
80875 |     ma_uint32 stringSizeWithNullTerminator = (ma_uint32)chunkSize;
80876 |     if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
80877 |         pParser->metadataCount += 1;
80878 |         ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser, stringSizeWithNullTerminator,
> 1);
80879 |     } else {
80880 |         ma_dr_wav_metadata* pMetadata = &pParser->pMetadata[pParser->metadataCursor];
80881 |         pMetadata->type = type;
80882 |         if (stringSizeWithNullTerminator > 0) {
80883 |             pMetadata->data.infoText.stringLength = stringSizeWithNullTerminator - 1;
80884 |             pMetadata->data.infoText.pString = (char*)ma_dr_wav__metadata_get_memory(pParser,
> stringSizeWithNullTerminator, 1);
80885 |             MA_DR_WAV_ASSERT(pMetadata->data.infoText.pString != NULL);
80886 |             bytesRead = ma_dr_wav__metadata_parser_read(pParser, pMetadata->data.infoText.pString,
> (size_t)stringSizeWithNullTerminator, NULL);
80887 |             if (bytesRead == chunkSize) {
80888 |                 pParser->metadataCursor += 1;
80889 |             } else {
80890 |             }
80891 |         } else {
80892 |             pMetadata->data.infoText.stringLength = 0;
80893 |             pMetadata->data.infoText.pString      = NULL;
80894 |             pParser->metadataCursor += 1;
80895 |         }
80896 |     }
80897 |     return bytesRead;
80898 | }
80899 | MA_PRIVATE ma_uint64 ma_dr_wav__metadata_process_unknown_chunk(ma_dr_wav__metadata_parser* pParser,
> const ma_uint8* pChunkId, ma_uint64 chunkSize, ma_dr_wav_metadata_location location)
80900 | {
80901 |     ma_uint64 bytesRead = 0;
80902 |     if (location == ma_dr_wav_metadata_location_invalid) {
80903 |         return 0;
80904 |     }
80905 |     if (ma_dr_wav_fourcc_equal(pChunkId, "data") || ma_dr_wav_fourcc_equal(pChunkId, "fmt ") ||
> ma_dr_wav_fourcc_equal(pChunkId, "fact")) {
80906 |         return 0;
80907 |     }
80908 |     if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
80909 |         pParser->metadataCount += 1;
80910 |         ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser, (size_t)chunkSize, 1);
80911 |     } else {
80912 |         ma_dr_wav_metadata* pMetadata = &pParser->pMetadata[pParser->metadataCursor];
80913 |         pMetadata->type
= ma_dr_wav_metadata_type_unknown;
80914 |         pMetadata->data.unknown.chunkLocation = location;
80915 |         pMetadata->data.unknown.id[0]
= pChunkId[0];
80916 |         pMetadata->data.unknown.id[1]
= pChunkId[1];
80917 |         pMetadata->data.unknown.id[2]
= pChunkId[2];
80918 |         pMetadata->data.unknown.id[3]
= pChunkId[3];
80919 |         pMetadata->data.unknown.dataSizeInBytes = (ma_uint32)chunkSize;
80920 |         pMetadata->data.unknown.pData
= (ma_uint8
> *)ma_dr_wav__metadata_get_memory(pParser, (size_t)chunkSize, 1);
80921 |         MA_DR_WAV_ASSERT(pMetadata->data.unknown.pData != NULL);
80922 |         bytesRead = ma_dr_wav__metadata_parser_read(pParser, pMetadata->data.unknown.pData,
> pMetadata->data.unknown.dataSizeInBytes, NULL);
80923 |         if (bytesRead == pMetadata->data.unknown.dataSizeInBytes) {
80924 |             pParser->metadataCursor += 1;

```



```

80925 |         } else {
80926 |         }
80927 |     }
80928 |     return bytesRead;
80929 | }
80930 | MA_PRIVATE ma_bool32 ma_dr_wav__chunk_matches(ma_dr_wav_metadata_type allowedMetadataTypes, const
> ma_uint8* pChunkID, ma_dr_wav_metadata_type type, const char* pID)
80931 | {
80932 |     return (allowedMetadataTypes & type) && ma_dr_wav_fourcc_equal(pChunkID, pID);
80933 | }
80934 | MA_PRIVATE ma_uint64 ma_dr_wav__metadata_process_chunk(ma_dr_wav__metadata_parser* pParser, const
> ma_dr_wav_chunk_header* pChunkHeader, ma_dr_wav_metadata_type allowedMetadataTypes)
80935 | {
80936 |     const ma_uint8 *pChunkID = pChunkHeader->id.fourcc;
80937 |     ma_uint64 bytesRead = 0;
80938 |     if (ma_dr_wav__chunk_matches(allowedMetadataTypes, pChunkID, ma_dr_wav_metadata_type_smpl,
> "smpl")) {
80939 |         if (pChunkHeader->sizeInBytes >= MA_DR_WAV_SMPL_BYTES) {
80940 |             if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
80941 |                 ma_uint8 buffer[4];
80942 |                 size_t bytesJustRead;
80943 |                 if (!pParser->onSeek(pParser->pReadSeekUserData, 28, MA_DR_WAV_SEEK_CUR)) {
80944 |                     return bytesRead;
80945 |                 }
80946 |                 bytesRead += 28;
80947 |                 bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, buffer, sizeof(buffer),
> &bytesRead);
80948 |                 if (bytesJustRead == sizeof(buffer)) {
80949 |                     ma_uint32 loopCount = ma_dr_wav_bytes_to_u32(buffer);
80950 |                     ma_uint64 calculatedLoopCount;
80951 |                     calculatedLoopCount = (pChunkHeader->sizeInBytes - MA_DR_WAV_SMPL_BYTES) /
> MA_DR_WAV_SMPL_LOOP_BYTES;
80952 |                     if (calculatedLoopCount == loopCount) {
80953 |                         bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, buffer,
> sizeof(buffer), &bytesRead);
80954 |                         if (bytesJustRead == sizeof(buffer)) {
80955 |                             ma_uint32 samplerSpecificDataSizeInBytes =
> ma_dr_wav_bytes_to_u32(buffer);
80956 |                             pParser->metadataCount += 1;
80957 |                             ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser,
> sizeof(ma_dr_wav_smpl_loop) * loopCount, MA_DR_WAV_METADATA_ALIGNMENT);
80958 |                             ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser,
> samplerSpecificDataSizeInBytes, 1);
80959 |                         }
80960 |                     } else {
80961 |                     }
80962 |                 }
80963 |             } else {
80964 |                 bytesRead = ma_dr_wav__read_smpl_to_metadata_obj(pParser, pChunkHeader,
> &pParser->pMetadata[pParser->metadataCursor]);
80965 |                 if (bytesRead == pChunkHeader->sizeInBytes) {
80966 |                     pParser->metadataCursor += 1;
80967 |                 } else {
80968 |                 }
80969 |             }
80970 |         } else {
80971 |         }
80972 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, pChunkID,
> ma_dr_wav_metadata_type_inst, "inst")) {
80973 |         if (pChunkHeader->sizeInBytes == MA_DR_WAV_INST_BYTES) {
80974 |             if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
80975 |                 pParser->metadataCount += 1;
80976 |             } else {
80977 |                 bytesRead = ma_dr_wav__read_inst_to_metadata_obj(pParser,
> &pParser->pMetadata[pParser->metadataCursor]);
80978 |                 if (bytesRead == pChunkHeader->sizeInBytes) {
80979 |                     pParser->metadataCursor += 1;
80980 |                 } else {
80981 |                 }
80982 |             }
80983 |         } else {
80984 |         }
80985 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, pChunkID,
> ma_dr_wav_metadata_type_acid, "acid")) {
80986 |         if (pChunkHeader->sizeInBytes == MA_DR_WAV_ACID_BYTES) {
80987 |             if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
80988 |                 pParser->metadataCount += 1;
80989 |             } else {
80990 |                 bytesRead = ma_dr_wav__read_acid_to_metadata_obj(pParser,

```



```

> &pParser->pMetadata[pParser->metadataCursor]);
80991 |         if (bytesRead == pChunkHeader->sizeInBytes) {
80992 |             pParser->metadataCursor += 1;
80993 |         } else {
80994 |             }
80995 |         }
80996 |     } else {
80997 |     }
80998 | } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, pChunkID, ma_dr_wav_metadata_type_cue,
> "cue ")) {
80999 |     if (pChunkHeader->sizeInBytes >= MA_DR_WAV_CUE_BYTES) {
81000 |         if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
81001 |             size_t cueCount;
81002 |             pParser->metadataCount += 1;
81003 |             cueCount = (size_t)(pChunkHeader->sizeInBytes - MA_DR_WAV_CUE_BYTES) /
> MA_DR_WAV_CUE_POINT_BYTES;
81004 |             ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser,
> sizeof(ma_dr_wav_cue_point) * cueCount, MA_DR_WAV_METADATA_ALIGNMENT);
81005 |         } else {
81006 |             bytesRead = ma_dr_wav__read_cue_to_metadata_obj(pParser, pChunkHeader,
> &pParser->pMetadata[pParser->metadataCursor]);
81007 |             if (bytesRead == pChunkHeader->sizeInBytes) {
81008 |                 pParser->metadataCursor += 1;
81009 |             } else {
81010 |                 }
81011 |             }
81012 |         } else {
81013 |         }
81014 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, pChunkID,
> ma_dr_wav_metadata_type_bext, "bext")) {
81015 |         if (pChunkHeader->sizeInBytes >= MA_DR_WAV_BEXT_BYTES) {
81016 |             if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
81017 |                 char buffer[MA_DR_WAV_BEXT_DESCRIPTION_BYTES + 1];
81018 |                 size_t allocSizeNeeded = MA_DR_WAV_BEXT_UMID_BYTES;
81019 |                 size_t bytesJustRead;
81020 |                 buffer[MA_DR_WAV_BEXT_DESCRIPTION_BYTES] = '\0';
81021 |                 bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, buffer,
> MA_DR_WAV_BEXT_DESCRIPTION_BYTES, &bytesRead);
81022 |                 if (bytesJustRead != MA_DR_WAV_BEXT_DESCRIPTION_BYTES) {
81023 |                     return bytesRead;
81024 |                 }
81025 |                 allocSizeNeeded += ma_dr_wav__strlen(buffer) + 1;
81026 |                 buffer[MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES] = '\0';
81027 |                 bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, buffer,
> MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES, &bytesRead);
81028 |                 if (bytesJustRead != MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES) {
81029 |                     return bytesRead;
81030 |                 }
81031 |                 allocSizeNeeded += ma_dr_wav__strlen(buffer) + 1;
81032 |                 buffer[MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES] = '\0';
81033 |                 bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, buffer,
> MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES, &bytesRead);
81034 |                 if (bytesJustRead != MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES) {
81035 |                     return bytesRead;
81036 |                 }
81037 |                 allocSizeNeeded += ma_dr_wav__strlen(buffer) + 1;
81038 |                 allocSizeNeeded += (size_t)pChunkHeader->sizeInBytes - MA_DR_WAV_BEXT_BYTES + 1;
81039 |                 ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser, allocSizeNeeded, 1);
81040 |                 pParser->metadataCount += 1;
81041 |             } else {
81042 |                 bytesRead = ma_dr_wav__read_bext_to_metadata_obj(pParser,
> &pParser->pMetadata[pParser->metadataCursor], pChunkHeader->sizeInBytes);
81043 |                 if (bytesRead == pChunkHeader->sizeInBytes) {
81044 |                     pParser->metadataCursor += 1;
81045 |                 } else {
81046 |                     }
81047 |                 }
81048 |             } else {
81049 |             }
81050 |         } else if (ma_dr_wav_fourcc_equal(pChunkID, "LIST") || ma_dr_wav_fourcc_equal(pChunkID, "list"))
> {
81051 |             ma_dr_wav_metadata_location listType = ma_dr_wav_metadata_location_invalid;
81052 |             while (bytesRead < pChunkHeader->sizeInBytes) {
81053 |                 ma_uint8 subchunkId[4];
81054 |                 ma_uint8 subchunkSizeBuffer[4];
81055 |                 ma_uint64 subchunkDataSize;
81056 |                 ma_uint64 subchunkBytesRead = 0;
81057 |                 ma_uint64 bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, subchunkId,
> sizeof(subchunkId), &bytesRead);

```

```

81058 |         if (bytesJustRead != sizeof(subchunkId)) {
81059 |             break;
81060 |         }
81061 |         if (ma_dr_wav_fourcc_equal(subchunkId, "adtl")) {
81062 |             listType = ma_dr_wav_metadata_location_inside_adtl_list;
81063 |             continue;
81064 |         } else if (ma_dr_wav_fourcc_equal(subchunkId, "INFO")) {
81065 |             listType = ma_dr_wav_metadata_location_inside_info_list;
81066 |             continue;
81067 |         }
81068 |         bytesJustRead = ma_dr_wav__metadata_parser_read(pParser, subchunkSizeBuffer,
> sizeof(subchunkSizeBuffer), &bytesRead);
81069 |         if (bytesJustRead != sizeof(subchunkSizeBuffer)) {
81070 |             break;
81071 |         }
81072 |         subchunkDataSize = ma_dr_wav_bytes_to_u32(subchunkSizeBuffer);
81073 |         if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_label, "label") || ma_dr_wav__chunk_matches(allowedMetadataTypes,
> subchunkId, ma_dr_wav_metadata_type_list_note, "note")) {
81074 |             if (subchunkDataSize >= MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES) {
81075 |                 ma_uint64 stringSizeWithNullTerm = subchunkDataSize -
> MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES;
81076 |                 if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
81077 |                     pParser->metadataCount += 1;
81078 |                     ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser,
> (size_t)stringSizeWithNullTerm, 1);
81079 |                 } else {
81080 |                     subchunkBytesRead =
> ma_dr_wav__read_list_label_or_note_to_metadata_obj(pParser,
> &pParser->pMetadata[pParser->metadataCursor], subchunkDataSize, ma_dr_wav_fourcc_equal(subchunkId,
> "label") ? ma_dr_wav_metadata_type_list_label : ma_dr_wav_metadata_type_list_note);
81081 |                     if (subchunkBytesRead == subchunkDataSize) {
81082 |                         pParser->metadataCursor += 1;
81083 |                     } else {
81084 |                         }
81085 |                     }
81086 |                 } else {
81087 |                 }
81088 |             } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_labelled_cue_region, "ltxr")) {
81089 |                 if (subchunkDataSize >= MA_DR_WAV_LIST_LABELLED_TEXT_BYTES) {
81090 |                     ma_uint64 stringSizeWithNullTerminator = subchunkDataSize -
> MA_DR_WAV_LIST_LABELLED_TEXT_BYTES;
81091 |                     if (pParser->stage == ma_dr_wav__metadata_parser_stage_count) {
81092 |                         pParser->metadataCount += 1;
81093 |                         ma_dr_wav__metadata_request_extra_memory_for_stage_2(pParser,
> (size_t)stringSizeWithNullTerminator, 1);
81094 |                     } else {
81095 |                         subchunkBytesRead =
> ma_dr_wav__read_list_labelled_cue_region_to_metadata_obj(pParser,
> &pParser->pMetadata[pParser->metadataCursor], subchunkDataSize);
81096 |                         if (subchunkBytesRead == subchunkDataSize) {
81097 |                             pParser->metadataCursor += 1;
81098 |                         } else {
81099 |                             }
81100 |                         }
81101 |                     } else {
81102 |                     }
81103 |                 } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_software, "ISFT")) {
81104 |                     subchunkBytesRead = ma_dr_wav__metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_software);
81105 |                 } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_copyright, "ICOP")) {
81106 |                     subchunkBytesRead = ma_dr_wav__metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_copyright);
81107 |                 } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_title, "INAM")) {
81108 |                     subchunkBytesRead = ma_dr_wav__metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_title);
81109 |                 } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_artist, "IART")) {
81110 |                     subchunkBytesRead = ma_dr_wav__metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_artist);
81111 |                 } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_comment, "ICMT")) {
81112 |                     subchunkBytesRead = ma_dr_wav__metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_comment);
81113 |                 } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,

```

```

> ma_dr_wav_metadata_type_list_info_date, "ICRD")) {
81114 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_date);
81115 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_genre, "IGNR")) {
81116 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_genre);
81117 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_album, "IPRD")) {
81118 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_album);
81119 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_tracknumber, "ITRK")) {
81120 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_tracknumber);
81121 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_location, "IARL")) {
81122 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_location);
81123 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_organization, "ICMS")) {
81124 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_organization);
81125 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_keywords, "IKEY")) {
81126 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_keywords);
81127 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_medium, "IMED")) {
81128 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_medium);
81129 |     } else if (ma_dr_wav__chunk_matches(allowedMetadataTypes, subchunkId,
> ma_dr_wav_metadata_type_list_info_description, "ISBJ")) {
81130 |         subchunkBytesRead = ma_dr_wav_metadata_process_info_text_chunk(pParser,
> subchunkDataSize, ma_dr_wav_metadata_type_list_info_description);
81131 |     } else if ((allowedMetadataTypes & ma_dr_wav_metadata_type_unknown) != 0) {
81132 |         subchunkBytesRead = ma_dr_wav_metadata_process_unknown_chunk(pParser, subchunkId,
> subchunkDataSize, listType);
81133 |     }
81134 |     bytesRead += subchunkBytesRead;
81135 |     MA_DR_WAV_ASSERT(subchunkBytesRead <= subchunkDataSize);
81136 |     if (subchunkBytesRead < subchunkDataSize) {
81137 |         ma_uint64 bytesToSeek = subchunkDataSize - subchunkBytesRead;
81138 |         if (!pParser->onSeek(pParser->pReadSeekUserData, (int)bytesToSeek,
> MA_DR_WAV_SEEK_CUR)) {
81139 |             break;
81140 |         }
81141 |         bytesRead += bytesToSeek;
81142 |     }
81143 |     if ((subchunkDataSize % 2) == 1) {
81144 |         if (!pParser->onSeek(pParser->pReadSeekUserData, 1, MA_DR_WAV_SEEK_CUR)) {
81145 |             break;
81146 |         }
81147 |         bytesRead += 1;
81148 |     }
81149 | }
81150 | } else if ((allowedMetadataTypes & ma_dr_wav_metadata_type_unknown) != 0) {
81151 |     bytesRead = ma_dr_wav_metadata_process_unknown_chunk(pParser, pChunkID,
> pChunkHeader->sizeInBytes, ma_dr_wav_metadata_location_top_level);
81152 | }
81153 | return bytesRead;
81154 | }
81155 | MA_PRIVATE ma_uint32 ma_dr_wav_get_bytes_per_pcm_frame(ma_dr_wav* pWav)
81156 | {
81157 |     ma_uint32 bytesPerFrame;
81158 |     if ((pWav->bitsPerSample & 0x7) == 0) {
81159 |         bytesPerFrame = (pWav->bitsPerSample * pWav->fmt.channels) >> 3;
81160 |     } else {
81161 |         bytesPerFrame = pWav->fmt.blockAlign;
81162 |     }
81163 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ALAW || pWav->translatedFormatTag ==
> MA_DR_WAVE_FORMAT_MULAW) {
81164 |         if (bytesPerFrame != pWav->fmt.channels) {
81165 |             return 0;
81166 |         }
81167 |     }
81168 |     return bytesPerFrame;
81169 | }
81170 | MA_API ma_uint16 ma_dr_wav_fmt_get_format(const ma_dr_wav_fmt* pFMT)

```

```

81171 | {
81172 |     if (pFMT == NULL) {
81173 |         return 0;
81174 |     }
81175 |     if (pFMT->formatTag != MA_DR_WAVE_FORMAT_EXTENSIBLE) {
81176 |         return pFMT->formatTag;
81177 |     } else {
81178 |         return ma_dr_wav_bytes_to_u16(pFMT->subFormat);
81179 |     }
81180 | }
81181 | MA_PRIVATE ma_bool32 ma_dr_wav_preinit(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pReadSeekTellUserData, const
> ma_allocation_callbacks* pAllocationCallbacks)
81182 | {
81183 |     if (pWav == NULL || onRead == NULL || onSeek == NULL) {
81184 |         return MA_FALSE;
81185 |     }
81186 |     MA_DR_WAV_ZERO_MEMORY(pWav, sizeof(*pWav));
81187 |     pWav->onRead = onRead;
81188 |     pWav->onSeek = onSeek;
81189 |     pWav->onTell = onTell;
81190 |     pWav->pUserData = pReadSeekTellUserData;
81191 |     pWav->allocationCallbacks =
> ma_dr_wav_copy_allocation_callbacks_or_defaults(pAllocationCallbacks);
81192 |     if (pWav->allocationCallbacks.onFree == NULL || (pWav->allocationCallbacks.onMalloc == NULL &&
> pWav->allocationCallbacks.onRealloc == NULL)) {
81193 |         return MA_FALSE;
81194 |     }
81195 |     return MA_TRUE;
81196 | }
81197 | MA_PRIVATE ma_bool32 ma_dr_wav_init__internal(ma_dr_wav* pWav, ma_dr_wav_chunk_proc onChunk, void*
> pChunkUserData, ma_uint32 flags)
81198 | {
81199 |     ma_result result;
81200 |     ma_uint64 cursor;
81201 |     ma_bool32 sequential;
81202 |     ma_uint8 riff[4];
81203 |     ma_dr_wav_fmt fmt;
81204 |     unsigned short translatedFormatTag;
81205 |     ma_uint64 dataChunkSize = 0;
81206 |     ma_uint64 sampleCountFromFactChunk = 0;
81207 |     ma_uint64 metadataStartPos;
81208 |     ma_dr_wav__metadata_parser metadataParser;
81209 |     ma_bool8 isProcessingMetadata = MA_FALSE;
81210 |     ma_bool8 foundChunk_fmt = MA_FALSE;
81211 |     ma_bool8 foundChunk_data = MA_FALSE;
81212 |     ma_bool8 isAIFCFormType = MA_FALSE;
81213 |     ma_uint64 aiffFrameCount = 0;
81214 |     cursor = 0;
81215 |     sequential = (flags & MA_DR_WAV_SEQUENTIAL) != 0;
81216 |     MA_DR_WAV_ZERO_OBJECT(&fmt);
81217 |     if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, riff, sizeof(riff), &cursor) !=
> sizeof(riff)) {
81218 |         return MA_FALSE;
81219 |     }
81220 |     if (ma_dr_wav_fourcc_equal(riff, "RIFF")) {
81221 |         pWav->container = ma_dr_wav_container_riff;
81222 |     } else if (ma_dr_wav_fourcc_equal(riff, "RIFX")) {
81223 |         pWav->container = ma_dr_wav_container_rifx;
81224 |     } else if (ma_dr_wav_fourcc_equal(riff, "riff")) {
81225 |         int i;
81226 |         ma_uint8 riff2[12];
81227 |         pWav->container = ma_dr_wav_container_w64;
81228 |         if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, riff2, sizeof(riff2), &cursor) !=
> sizeof(riff2)) {
81229 |             return MA_FALSE;
81230 |         }
81231 |         for (i = 0; i < 12; ++i) {
81232 |             if (riff2[i] != ma_dr_wavGUID_W64_RIFF[i+4]) {
81233 |                 return MA_FALSE;
81234 |             }
81235 |         }
81236 |     } else if (ma_dr_wav_fourcc_equal(riff, "RF64")) {
81237 |         pWav->container = ma_dr_wav_container_rf64;
81238 |     } else if (ma_dr_wav_fourcc_equal(riff, "FORM")) {
81239 |         pWav->container = ma_dr_wav_container_aiff;
81240 |     } else {
81241 |         return MA_FALSE;
81242 |     }

```

```

81243 |     if (pWav->container == ma_dr_wav_container_riff || pWav->container == ma_dr_wav_container_rifx
> || pWav->container == ma_dr_wav_container_rf64) {
81244 |         ma_uint8 chunkSizeBytes[4];
81245 |         ma_uint8 wave[4];
81246 |         if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, chunkSizeBytes,
> sizeof(chunkSizeBytes), &cursor) != sizeof(chunkSizeBytes)) {
81247 |             return MA_FALSE;
81248 |         }
81249 |         if (pWav->container == ma_dr_wav_container_riff || pWav->container ==
> ma_dr_wav_container_rifx) {
81250 |             if (ma_dr_wav_bytes_to_u32_ex(chunkSizeBytes, pWav->container) < 36) {
81251 |                 }
81252 |             } else if (pWav->container == ma_dr_wav_container_rf64) {
81253 |                 if (ma_dr_wav_bytes_to_u32_le(chunkSizeBytes) != 0xFFFFFFFF) {
81254 |                     return MA_FALSE;
81255 |                 }
81256 |             } else {
81257 |                 return MA_FALSE;
81258 |             }
81259 |             if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, wave, sizeof(wave), &cursor) !=
> sizeof(wave)) {
81260 |                 return MA_FALSE;
81261 |             }
81262 |             if (!ma_dr_wav_fourcc_equal(wave, "WAVE")) {
81263 |                 return MA_FALSE;
81264 |             }
81265 |         } else if (pWav->container == ma_dr_wav_container_w64) {
81266 |             ma_uint8 chunkSizeBytes[8];
81267 |             ma_uint8 wave[16];
81268 |             if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, chunkSizeBytes,
> sizeof(chunkSizeBytes), &cursor) != sizeof(chunkSizeBytes)) {
81269 |                 return MA_FALSE;
81270 |             }
81271 |             if (ma_dr_wav_bytes_to_u64(chunkSizeBytes) < 80) {
81272 |                 return MA_FALSE;
81273 |             }
81274 |             if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, wave, sizeof(wave), &cursor) !=
> sizeof(wave)) {
81275 |                 return MA_FALSE;
81276 |             }
81277 |             if (!ma_dr_wav_guid_equal(wave, ma_dr_wavGUID_W64_WAVE)) {
81278 |                 return MA_FALSE;
81279 |             }
81280 |         } else if (pWav->container == ma_dr_wav_container_aiff) {
81281 |             ma_uint8 chunkSizeBytes[4];
81282 |             ma_uint8 aiff[4];
81283 |             if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, chunkSizeBytes,
> sizeof(chunkSizeBytes), &cursor) != sizeof(chunkSizeBytes)) {
81284 |                 return MA_FALSE;
81285 |             }
81286 |             if (ma_dr_wav_bytes_to_u32_be(chunkSizeBytes) < 18) {
81287 |                 return MA_FALSE;
81288 |             }
81289 |             if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, aiff, sizeof(aiff), &cursor) !=
> sizeof(aiff)) {
81290 |                 return MA_FALSE;
81291 |             }
81292 |             if (ma_dr_wav_fourcc_equal(aiff, "AIFF")) {
81293 |                 isAIFCFormType = MA_FALSE;
81294 |             } else if (ma_dr_wav_fourcc_equal(aiff, "AIFC")) {
81295 |                 isAIFCFormType = MA_TRUE;
81296 |             } else {
81297 |                 return MA_FALSE;
81298 |             }
81299 |         } else {
81300 |             return MA_FALSE;
81301 |         }
81302 |         if (pWav->container == ma_dr_wav_container_rf64) {
81303 |             ma_uint8 sizeBytes[8];
81304 |             ma_uint64 bytesRemainingInChunk;
81305 |             ma_dr_wav_chunk_header header;
81306 |             result = ma_dr_wav__read_chunk_header(pWav->onRead, pWav->pUserData, pWav->container,
> &cursor, &header);
81307 |             if (result != MA_SUCCESS) {
81308 |                 return MA_FALSE;
81309 |             }
81310 |             if (!ma_dr_wav_fourcc_equal(header.id.fourcc, "ds64")) {
81311 |                 return MA_FALSE;
81312 |             }

```

```

81313 |         bytesRemainingInChunk = header.sizeInBytes + header.paddingSize;
81314 |         if (!ma_dr_wav__seek_forward(pWav->onSeek, 8, pWav->pUserData)) {
81315 |             return MA_FALSE;
81316 |         }
81317 |         bytesRemainingInChunk -= 8;
81318 |         cursor += 8;
81319 |         if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, sizeBytes, sizeof(sizeBytes), &cursor)
> != sizeof(sizeBytes)) {
81320 |             return MA_FALSE;
81321 |         }
81322 |         bytesRemainingInChunk -= 8;
81323 |         dataChunkSize = ma_dr_wav_bytes_to_u64(sizeBytes);
81324 |         if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, sizeBytes, sizeof(sizeBytes), &cursor)
> != sizeof(sizeBytes)) {
81325 |             return MA_FALSE;
81326 |         }
81327 |         bytesRemainingInChunk -= 8;
81328 |         sampleCountFromFactChunk = ma_dr_wav_bytes_to_u64(sizeBytes);
81329 |         if (!ma_dr_wav__seek_forward(pWav->onSeek, bytesRemainingInChunk, pWav->pUserData)) {
81330 |             return MA_FALSE;
81331 |         }
81332 |         cursor += bytesRemainingInChunk;
81333 |     }
81334 |     metadataStartPos = cursor;
81335 |     isProcessingMetadata = !sequential && ((flags & MA_DR_WAV_WITH_METADATA) != 0);
81336 |     if (pWav->container != ma_dr_wav_container_riff && pWav->container != ma_dr_wav_container_rf64)
> {
81337 |         isProcessingMetadata = MA_FALSE;
81338 |     }
81339 |     MA_DR_WAV_ZERO_MEMORY(&metadataParser, sizeof(metadataParser));
81340 |     if (isProcessingMetadata) {
81341 |         metadataParser.onRead = pWav->onRead;
81342 |         metadataParser.onSeek = pWav->onSeek;
81343 |         metadataParser.pReadSeekUserData = pWav->pUserData;
81344 |         metadataParser.stage = ma_dr_wav__metadata_parser_stage_count;
81345 |     }
81346 |     for (;;) {
81347 |         ma_dr_wav_chunk_header header;
81348 |         ma_uint64 chunkSize;
81349 |         result = ma_dr_wav__read_chunk_header(pWav->onRead, pWav->pUserData, pWav->container,
> &cursor, &header);
81350 |         if (result != MA_SUCCESS) {
81351 |             break;
81352 |         }
81353 |         chunkSize = header.sizeInBytes;
81354 |         if (!sequential && onChunk != NULL) {
81355 |             ma_uint64 callbackBytesRead = onChunk(pChunkUserData, pWav->onRead, pWav->onSeek,
> pWav->pUserData, &header, pWav->container, &fmt);
81356 |             if (callbackBytesRead > 0) {
81357 |                 if (ma_dr_wav__seek_from_start(pWav->onSeek, cursor, pWav->pUserData) == MA_FALSE) {
81358 |                     return MA_FALSE;
81359 |                 }
81360 |             }
81361 |         }
81362 |         if (((pWav->container == ma_dr_wav_container_riff || pWav->container ==
> ma_dr_wav_container_rifx || pWav->container == ma_dr_wav_container_rf64) &&
> ma_dr_wav_fourcc_equal(header.id.fourcc, "fmt ") ||
81363 |             ((pWav->container == ma_dr_wav_container_w64) && ma_dr_wav_guid_equal(header.id.guid,
> ma_dr_wavGUID_W64_FMT))) {
81364 |             ma_uint8 fmtData[16];
81365 |             foundChunk_fmt = MA_TRUE;
81366 |             if (pWav->onRead(pWav->pUserData, fmtData, sizeof(fmtData)) != sizeof(fmtData)) {
81367 |                 return MA_FALSE;
81368 |             }
81369 |             cursor += sizeof(fmtData);
81370 |             fmt.formatTag = ma_dr_wav_bytes_to_u16_ex(fmtData + 0, pWav->container);
81371 |             fmt.channels = ma_dr_wav_bytes_to_u16_ex(fmtData + 2, pWav->container);
81372 |             fmt.sampleRate = ma_dr_wav_bytes_to_u32_ex(fmtData + 4, pWav->container);
81373 |             fmt.avgBytesPerSec = ma_dr_wav_bytes_to_u32_ex(fmtData + 8, pWav->container);
81374 |             fmt.blockAlign = ma_dr_wav_bytes_to_u16_ex(fmtData + 12, pWav->container);
81375 |             fmt.bitsPerSample = ma_dr_wav_bytes_to_u16_ex(fmtData + 14, pWav->container);
81376 |             fmt.extendedSize = 0;
81377 |             fmt.validBitsPerSample = 0;
81378 |             fmt.channelMask = 0;
81379 |             MA_DR_WAV_ZERO_MEMORY(fmt.subFormat, sizeof(fmt.subFormat));
81380 |             if (header.sizeInBytes > 16) {
81381 |                 ma_uint8 fmt_cbSize[2];
81382 |                 int bytesReadSoFar = 0;
81383 |                 if (pWav->onRead(pWav->pUserData, fmt_cbSize, sizeof(fmt_cbSize)) !=

```



```

> sizeof(fmt_cbSize)) {
81384 |         return MA_FALSE;
81385 |     }
81386 |     cursor += sizeof(fmt_cbSize);
81387 |     bytesReadSoFar = 18;
81388 |     fmt.extendedSize = ma_dr_wav_bytes_to_u16_ex(fmt_cbSize, pWav->container);
81389 |     if (fmt.extendedSize > 0) {
81390 |         if (fmt.formatTag == MA_DR_WAVE_FORMAT_EXTENSIBLE) {
81391 |             if (fmt.extendedSize != 22) {
81392 |                 return MA_FALSE;
81393 |             }
81394 |         }
81395 |         if (fmt.formatTag == MA_DR_WAVE_FORMAT_EXTENSIBLE) {
81396 |             ma_uint8 fmttext[22];
81397 |             if (pWav->onRead(pWav->pUserData, fmttext, fmt.extendedSize) !=
> fmt.extendedSize) {
81398 |                 return MA_FALSE;
81399 |             }
81400 |             fmt.validBitsPerSample = ma_dr_wav_bytes_to_u16_ex(fmttext + 0,
> pWav->container);
81401 |             fmt.channelMask = ma_dr_wav_bytes_to_u32_ex(fmttext + 2,
> pWav->container);
81402 |             ma_dr_wav_bytes_to_guid(fmttext + 6, fmt.subFormat);
81403 |         } else {
81404 |             if (pWav->onSeek(pWav->pUserData, fmt.extendedSize, MA_DR_WAV_SEEK_CUR) ==
> MA_FALSE) {
81405 |                 return MA_FALSE;
81406 |             }
81407 |         }
81408 |         cursor += fmt.extendedSize;
81409 |         bytesReadSoFar += fmt.extendedSize;
81410 |     }
81411 |     if (pWav->onSeek(pWav->pUserData, (int)(header.sizeInBytes - bytesReadSoFar),
> MA_DR_WAV_SEEK_CUR) == MA_FALSE) {
81412 |         return MA_FALSE;
81413 |     }
81414 |     cursor += (header.sizeInBytes - bytesReadSoFar);
81415 | }
81416 | if (header.paddingSize > 0) {
81417 |     if (ma_dr_wav__seek_forward(pWav->onSeek, header.paddingSize, pWav->pUserData) ==
> MA_FALSE) {
81418 |         break;
81419 |     }
81420 |     cursor += header.paddingSize;
81421 | }
81422 | continue;
81423 | }
81424 | if (((pWav->container == ma_dr_wav_container_riff || pWav->container ==
> ma_dr_wav_container_rifx || pWav->container == ma_dr_wav_container_rf64) &&
> ma_dr_wav_fourcc_equal(header.id.fourcc, "data")) ||
81425 |     ((pWav->container == ma_dr_wav_container_w64) && ma_dr_wav_guid_equal(header.id.guid,
> ma_dr_wavGUID_W64_DATA))) {
81426 |     foundChunk_data = MA_TRUE;
81427 |     pWav->dataChunkDataPos = cursor;
81428 |     if (pWav->container != ma_dr_wav_container_rf64) {
81429 |         dataChunkSize = chunkSize;
81430 |     }
81431 |     if (sequential || !isProcessingMetadata) {
81432 |         break;
81433 |     } else {
81434 |         chunkSize += header.paddingSize;
81435 |         if (ma_dr_wav__seek_forward(pWav->onSeek, chunkSize, pWav->pUserData) == MA_FALSE) {
81436 |             break;
81437 |         }
81438 |         cursor += chunkSize;
81439 |         continue;
81440 |     }
81441 | }
81442 | if (((pWav->container == ma_dr_wav_container_riff || pWav->container ==
> ma_dr_wav_container_rifx || pWav->container == ma_dr_wav_container_rf64) &&
> ma_dr_wav_fourcc_equal(header.id.fourcc, "fact")) ||
81443 |     ((pWav->container == ma_dr_wav_container_w64) && ma_dr_wav_guid_equal(header.id.guid,
> ma_dr_wavGUID_W64_FACT))) {
81444 |     if (pWav->container == ma_dr_wav_container_riff || pWav->container ==
> ma_dr_wav_container_rifx) {
81445 |         ma_uint8 sampleCount[4];
81446 |         if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, &sampleCount, 4, &cursor) !=
> 4) {
81447 |             return MA_FALSE;

```



```

81448 |         }
81449 |         chunkSize -= 4;
81450 |         if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM) {
81451 |             sampleCountFromFactChunk = ma_dr_wav_bytes_to_u32_ex(sampleCount,
> pWav->container);
81452 |         } else {
81453 |             sampleCountFromFactChunk = 0;
81454 |         }
81455 |     } else if (pWav->container == ma_dr_wav_container_w64) {
81456 |         if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, &sampleCountFromFactChunk, 8,
> &cursor) != 8) {
81457 |             return MA_FALSE;
81458 |         }
81459 |         chunkSize -= 8;
81460 |     } else if (pWav->container == ma_dr_wav_container_rf64) {
81461 |     }
81462 |     chunkSize += header.paddingSize;
81463 |     if (ma_dr_wav__seek_forward(pWav->onSeek, chunkSize, pWav->pUserData) == MA_FALSE) {
81464 |         break;
81465 |     }
81466 |     cursor += chunkSize;
81467 |     continue;
81468 | }
81469 | if (pWav->container == ma_dr_wav_container_aiff && ma_dr_wav_fourcc_equal(header.id.fourcc,
> "COMM")) {
81470 |     ma_uint8 commData[24];
81471 |     ma_uint32 commDataBytesToRead;
81472 |     ma_uint16 channels;
81473 |     ma_uint32 frameCount;
81474 |     ma_uint16 sampleSizeInBits;
81475 |     ma_int64 sampleRate;
81476 |     ma_uint16 compressionFormat;
81477 |     foundChunk_fmt = MA_TRUE;
81478 |     if (isAIFCFormType) {
81479 |         commDataBytesToRead = 24;
81480 |         if (header.sizeInBytes < commDataBytesToRead) {
81481 |             return MA_FALSE;
81482 |         }
81483 |     } else {
81484 |         commDataBytesToRead = 18;
81485 |         if (header.sizeInBytes != commDataBytesToRead) {
81486 |             return MA_FALSE;
81487 |         }
81488 |     }
81489 |     if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, commData, commDataBytesToRead,
> &cursor) != commDataBytesToRead) {
81490 |         return MA_FALSE;
81491 |     }
81492 |     channels = ma_dr_wav_bytes_to_u16_ex(commData + 0, pWav->container);
81493 |     frameCount = ma_dr_wav_bytes_to_u32_ex(commData + 2, pWav->container);
81494 |     sampleSizeInBits = ma_dr_wav_bytes_to_u16_ex(commData + 6, pWav->container);
81495 |     sampleRate = ma_dr_wav_aiff_extented_to_s64(commData + 8);
81496 |     if (sampleRate < 0 || sampleRate > 0xFFFFFFFF) {
81497 |         return MA_FALSE;
81498 |     }
81499 |     if (isAIFCFormType) {
81500 |         const ma_uint8* type = commData + 18;
81501 |         if (ma_dr_wav_fourcc_equal(type, "NONE")) {
81502 |             compressionFormat = MA_DR_WAVE_FORMAT_PCM;
81503 |         } else if (ma_dr_wav_fourcc_equal(type, "raw")) {
81504 |             compressionFormat = MA_DR_WAVE_FORMAT_PCM;
81505 |             if (sampleSizeInBits == 8) {
81506 |                 pWav->aiff.isUnsigned = MA_TRUE;
81507 |             }
81508 |         } else if (ma_dr_wav_fourcc_equal(type, "sowt")) {
81509 |             compressionFormat = MA_DR_WAVE_FORMAT_PCM;
81510 |             pWav->aiff.isLE = MA_TRUE;
81511 |         } else if (ma_dr_wav_fourcc_equal(type, "fl32") || ma_dr_wav_fourcc_equal(type,
> "fl64") || ma_dr_wav_fourcc_equal(type, "FL32") || ma_dr_wav_fourcc_equal(type, "FL64")) {
81512 |             compressionFormat = MA_DR_WAVE_FORMAT_IEEE_FLOAT;
81513 |         } else if (ma_dr_wav_fourcc_equal(type, "alaw") || ma_dr_wav_fourcc_equal(type,
> "ALAW")) {
81514 |             compressionFormat = MA_DR_WAVE_FORMAT_ALAW;
81515 |         } else if (ma_dr_wav_fourcc_equal(type, "ulaw") || ma_dr_wav_fourcc_equal(type,
> "ULAW")) {
81516 |             compressionFormat = MA_DR_WAVE_FORMAT_MULAW;
81517 |         } else if (ma_dr_wav_fourcc_equal(type, "ima4")) {
81518 |             compressionFormat = MA_DR_WAVE_FORMAT_DVI_ADPCM;
81519 |             sampleSizeInBits = 4;

```

```

81520 |         (void)compressionFormat;
81521 |         (void)sampleSizeInBits;
81522 |         return MA_FALSE;
81523 |     } else {
81524 |         return MA_FALSE;
81525 |     }
81526 | } else {
81527 |     compressionFormat = MA_DR_WAVE_FORMAT_PCM;
81528 | }
81529 | aiffFrameCount = frameCount;
81530 | fmt.formatTag      = compressionFormat;
81531 | fmt.channels       = channels;
81532 | fmt.sampleRate     = (ma_uint32)sampleRate;
81533 | fmt.bitsPerSample  = sampleSizeInBits;
81534 | fmt.blockAlign     = (ma_uint16)(fmt.channels * fmt.bitsPerSample / 8);
81535 | fmt.avgBytesPerSec = fmt.blockAlign * fmt.sampleRate;
81536 | if (fmt.blockAlign == 0 && compressionFormat == MA_DR_WAVE_FORMAT_DVI_ADPCM) {
81537 |     fmt.blockAlign = 34 * fmt.channels;
81538 | }
81539 | if (compressionFormat == MA_DR_WAVE_FORMAT_ALAW || compressionFormat ==
> MA_DR_WAVE_FORMAT_MULAW) {
81540 |     if (fmt.bitsPerSample > 8) {
81541 |         fmt.bitsPerSample = 8;
81542 |         fmt.blockAlign = fmt.channels;
81543 |     }
81544 | }
81545 | fmt.bitsPerSample += (fmt.bitsPerSample & 7);
81546 | if (isAIFCFormType) {
81547 |     if (ma_dr_wav__seek_forward(pWav->onSeek, (chunkSize - commDataBytesToRead),
> pWav->pUserData) == MA_FALSE) {
81548 |         return MA_FALSE;
81549 |     }
81550 |     cursor += (chunkSize - commDataBytesToRead);
81551 | }
81552 | continue;
81553 | }
81554 | if (pWav->container == ma_dr_wav_container_aiff && ma_dr_wav_fourcc_equal(header.id.fourcc,
> "SSND")) {
81555 |     ma_uint8 offsetAndBlockSizeData[8];
81556 |     ma_uint32 offset;
81557 |     foundChunk_data = MA_TRUE;
81558 |     if (ma_dr_wav__on_read(pWav->onRead, pWav->pUserData, offsetAndBlockSizeData,
> sizeof(offsetAndBlockSizeData), &cursor) != sizeof(offsetAndBlockSizeData)) {
81559 |         return MA_FALSE;
81560 |     }
81561 |     offset = ma_dr_wav_bytes_to_u32_ex(offsetAndBlockSizeData + 0, pWav->container);
81562 |     pWav->dataChunkDataPos = cursor + offset;
81563 |     dataChunkSize = chunkSize;
81564 |     if (dataChunkSize > offset) {
81565 |         dataChunkSize -= offset;
81566 |     } else {
81567 |         dataChunkSize = 0;
81568 |     }
81569 |     if (sequential) {
81570 |         if (foundChunk_fmt) {
81571 |             if (ma_dr_wav__seek_forward(pWav->onSeek, offset, pWav->pUserData) == MA_FALSE)
> {
81572 |                 return MA_FALSE;
81573 |             }
81574 |             cursor += offset;
81575 |             break;
81576 |         } else {
81577 |             return MA_FALSE;
81578 |         }
81579 |     } else {
81580 |         chunkSize += header.paddingSize;
81581 |         chunkSize -= sizeof(offsetAndBlockSizeData);
81582 |         if (ma_dr_wav__seek_forward(pWav->onSeek, chunkSize, pWav->pUserData) == MA_FALSE) {
81583 |             break;
81584 |         }
81585 |         cursor += chunkSize;
81586 |         continue;
81587 |     }
81588 | }
81589 | if (isProcessingMetadata) {
81590 |     ma_dr_wav__metadata_process_chunk(&metadataParser, &header,
> ma_dr_wav_metadata_type_all_including_unknown);
81591 |     if (ma_dr_wav__seek_from_start(pWav->onSeek, cursor, pWav->pUserData) == MA_FALSE) {
81592 |         break;

```

```

81593 |     }
81594 | }
81595 | chunkSize += header.paddingSize;
81596 | if (ma_dr_wav__seek_forward(pWav->onSeek, chunkSize, pWav->pUserData) == MA_FALSE) {
81597 |     break;
81598 | }
81599 | cursor += chunkSize;
81600 | }
81601 | if (!foundChunk_fmt || !foundChunk_data) {
81602 |     return MA_FALSE;
81603 | }
81604 | if ((fmt.sampleRate == 0 || fmt.sampleRate > MA_DR_WAV_MAX_SAMPLE_RATE) ||
81605 |     (fmt.channels == 0 || fmt.channels > MA_DR_WAV_MAX_CHANNELS) ||
81606 |     (fmt.bitsPerSample == 0 || fmt.bitsPerSample > MA_DR_WAV_MAX_BITS_PER_SAMPLE) ||
81607 |     fmt.blockAlign == 0) {
81608 |     return MA_FALSE;
81609 | }
81610 | translatedFormatTag = fmt.formatTag;
81611 | if (translatedFormatTag == MA_DR_WAVE_FORMAT_EXTENSIBLE) {
81612 |     translatedFormatTag = ma_dr_wav_bytes_to_u16_ex(fmt.subFormat + 0, pWav->container);
81613 | }
81614 | if (!sequential) {
81615 |     if (!ma_dr_wav__seek_from_start(pWav->onSeek, pWav->dataChunkDataPos, pWav->pUserData)) {
81616 |         return MA_FALSE;
81617 |     }
81618 |     cursor = pWav->dataChunkDataPos;
81619 | }
81620 | if (isProcessingMetadata && metadataParser.metadataCount > 0) {
81621 |     if (ma_dr_wav__seek_from_start(pWav->onSeek, metadataStartPos, pWav->pUserData) == MA_FALSE)
81622 | > {
81623 |         return MA_FALSE;
81624 |     }
81625 |     result = ma_dr_wav__metadata_alloc(&metadataParser, &pWav->allocationCallbacks);
81626 |     if (result != MA_SUCCESS) {
81627 |         return MA_FALSE;
81628 |     }
81629 |     metadataParser.stage = ma_dr_wav__metadata_parser_stage_read;
81630 |     for (;;) {
81631 |         ma_dr_wav_chunk_header header;
81632 |         ma_uint64 metadataBytesRead;
81633 |         result = ma_dr_wav__read_chunk_header(pWav->onRead, pWav->pUserData, pWav->container,
81634 | > &cursor, &header);
81635 |         if (result != MA_SUCCESS) {
81636 |             break;
81637 |         }
81638 |         metadataBytesRead = ma_dr_wav__metadata_process_chunk(&metadataParser, &header,
81639 | > ma_dr_wav_metadata_type_all_including_unknown);
81640 |         if (ma_dr_wav__seek_forward(pWav->onSeek, (header.sizeInBytes + header.paddingSize) -
81641 | > metadataBytesRead, pWav->pUserData) == MA_FALSE) {
81642 |             ma_dr_wav_free(metadataParser.pMetadata, &pWav->allocationCallbacks);
81643 |             return MA_FALSE;
81644 |         }
81645 |     }
81646 |     pWav->pMetadata = metadataParser.pMetadata;
81647 |     pWav->metadataCount = metadataParser.metadataCount;
81648 | }
81649 | if (pWav->onTell != NULL && pWav->onSeek != NULL) {
81650 |     if (pWav->onSeek(pWav->pUserData, 0, MA_DR_WAV_SEEK_END) == MA_TRUE) {
81651 |         ma_int64 fileSize;
81652 |         if (pWav->onTell(pWav->pUserData, &fileSize)) {
81653 |             if (dataChunkSize + pWav->dataChunkDataPos > (ma_uint64)fileSize) {
81654 |                 dataChunkSize = (ma_uint64)fileSize - pWav->dataChunkDataPos;
81655 |             }
81656 |         }
81657 |     } else {
81658 |     }
81659 | }
81660 | if (dataChunkSize == 0xFFFFFFFF && (pWav->container == ma_dr_wav_container_riff ||
81661 | > pWav->container == ma_dr_wav_container_rifx) && pWav->isSequentialWrite == MA_FALSE) {
81662 |     dataChunkSize = 0;
81663 |     for (;;) {
81664 |         ma_uint8 temp[4096];
81665 |         size_t bytesRead = pWav->onRead(pWav->pUserData, temp, sizeof(temp));
81666 |         dataChunkSize += bytesRead;
81667 |         if (bytesRead < sizeof(temp)) {
81668 |             break;
81669 |         }
81670 |     }
81671 | }

```

```

81667 |     if (ma_dr_wav__seek_from_start(pWav->onSeek, pWav->dataChunkDataPos, pWav->pUserData) ==
> MA_FALSE) {
81668 |         ma_dr_wav_free(pWav->pMetadata, &pWav->allocationCallbacks);
81669 |         return MA_FALSE;
81670 |     }
81671 |     pWav->fmt                = fmt;
81672 |     pWav->sampleRate         = fmt.sampleRate;
81673 |     pWav->channels            = fmt.channels;
81674 |     pWav->bitsPerSample      = fmt.bitsPerSample;
81675 |     pWav->translatedFormatTag = translatedFormatTag;
81676 |     if (!ma_dr_wav__is_compressed_format_tag(translatedFormatTag)) {
81677 |         ma_uint32 bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
81678 |         if (bytesPerFrame > 0) {
81679 |             dataChunkSize -= (dataChunkSize % bytesPerFrame);
81680 |         }
81681 |     }
81682 |     pWav->bytesRemaining      = dataChunkSize;
81683 |     pWav->dataChunkDataSize   = dataChunkSize;
81684 |     if (sampleCountFromFactChunk != 0) {
81685 |         pWav->totalPCMFrameCount = sampleCountFromFactChunk;
81686 |     } else if (aiffFrameCount != 0) {
81687 |         pWav->totalPCMFrameCount = aiffFrameCount;
81688 |     } else {
81689 |         ma_uint32 bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
81690 |         if (bytesPerFrame == 0) {
81691 |             ma_dr_wav_free(pWav->pMetadata, &pWav->allocationCallbacks);
81692 |             return MA_FALSE;
81693 |         }
81694 |         pWav->totalPCMFrameCount = dataChunkSize / bytesPerFrame;
81695 |         if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM) {
81696 |             ma_uint64 totalBlockHeaderSizeInBytes;
81697 |             ma_uint64 blockCount = dataChunkSize / fmt.blockAlign;
81698 |             if ((blockCount * fmt.blockAlign) < dataChunkSize) {
81699 |                 blockCount += 1;
81700 |             }
81701 |             totalBlockHeaderSizeInBytes = blockCount * (6*fmt.channels);
81702 |             pWav->totalPCMFrameCount = ((dataChunkSize - totalBlockHeaderSizeInBytes) * 2) /
> fmt.channels;
81703 |         }
81704 |         if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_DVI_ADPCM) {
81705 |             ma_uint64 totalBlockHeaderSizeInBytes;
81706 |             ma_uint64 blockCount = dataChunkSize / fmt.blockAlign;
81707 |             if ((blockCount * fmt.blockAlign) < dataChunkSize) {
81708 |                 blockCount += 1;
81709 |             }
81710 |             totalBlockHeaderSizeInBytes = blockCount * (4*fmt.channels);
81711 |             pWav->totalPCMFrameCount = ((dataChunkSize - totalBlockHeaderSizeInBytes) * 2) /
> fmt.channels;
81712 |             pWav->totalPCMFrameCount += blockCount;
81713 |         }
81714 |     }
81715 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM || pWav->translatedFormatTag ==
> MA_DR_WAVE_FORMAT_DVI_ADPCM) {
81716 |         if (pWav->channels > 2) {
81717 |             ma_dr_wav_free(pWav->pMetadata, &pWav->allocationCallbacks);
81718 |             return MA_FALSE;
81719 |         }
81720 |     }
81721 |     if (ma_dr_wav_get_bytes_per_pcm_frame(pWav) == 0) {
81722 |         ma_dr_wav_free(pWav->pMetadata, &pWav->allocationCallbacks);
81723 |         return MA_FALSE;
81724 |     }
81725 | #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
81726 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM) {
81727 |         ma_uint64 blockCount = dataChunkSize / fmt.blockAlign;
81728 |         pWav->totalPCMFrameCount = (((blockCount * (fmt.blockAlign - (6*pWav->channels)))) * 2) /
> fmt.channels;
81729 |     }
81730 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_DVI_ADPCM) {
81731 |         ma_uint64 blockCount = dataChunkSize / fmt.blockAlign;
81732 |         pWav->totalPCMFrameCount = (((blockCount * (fmt.blockAlign - (4*pWav->channels)))) * 2) +
> (blockCount * pWav->channels)) / fmt.channels;
81733 |     }
81734 | #endif
81735 |     return MA_TRUE;
81736 | }
81737 | MA_API ma_bool32 ma_dr_wav_init(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead, ma_dr_wav_seek_proc
> onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, const ma_allocation_callbacks*
> pAllocationCallbacks)

```

```

81738 | {
81739 |     return ma_dr_wav_init_ex(pWav, onRead, onSeek, onTell, NULL, pUserData, NULL, 0,
> pAllocationCallbacks);
81740 | }
81741 | MA_API ma_bool32 ma_dr_wav_init_ex(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead, ma_dr_wav_seek_proc
> onSeek, ma_dr_wav_tell_proc onTell, ma_dr_wav_chunk_proc onChunk, void* pReadSeekTellUserData, void*
> pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks)
81742 | {
81743 |     if (!ma_dr_wav_preinit(pWav, onRead, onSeek, onTell, pReadSeekTellUserData,
> pAllocationCallbacks)) {
81744 |         return MA_FALSE;
81745 |     }
81746 |     return ma_dr_wav_init__internal(pWav, onChunk, pChunkUserData, flags);
81747 | }
81748 | MA_API ma_bool32 ma_dr_wav_init_with_metadata(ma_dr_wav* pWav, ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, ma_uint32 flags, const
> ma_allocation_callbacks* pAllocationCallbacks)
81749 | {
81750 |     if (!ma_dr_wav_preinit(pWav, onRead, onSeek, onTell, pUserData, pAllocationCallbacks)) {
81751 |         return MA_FALSE;
81752 |     }
81753 |     return ma_dr_wav_init__internal(pWav, NULL, NULL, flags | MA_DR_WAV_WITH_METADATA);
81754 | }
81755 | MA_API ma_dr_wav_metadata* ma_dr_wav_take_ownership_of_metadata(ma_dr_wav* pWav)
81756 | {
81757 |     ma_dr_wav_metadata *result = pWav->pMetadata;
81758 |     pWav->pMetadata = NULL;
81759 |     pWav->metadataCount = 0;
81760 |     return result;
81761 | }
81762 | MA_PRIVATE size_t ma_dr_wav__write(ma_dr_wav* pWav, const void* pData, size_t dataSize)
81763 | {
81764 |     MA_DR_WAV_ASSERT(pWav != NULL);
81765 |     MA_DR_WAV_ASSERT(pWav->onWrite != NULL);
81766 |     return pWav->onWrite(pWav->pUserData, pData, dataSize);
81767 | }
81768 | MA_PRIVATE size_t ma_dr_wav__write_byte(ma_dr_wav* pWav, ma_uint8 byte)
81769 | {
81770 |     MA_DR_WAV_ASSERT(pWav != NULL);
81771 |     MA_DR_WAV_ASSERT(pWav->onWrite != NULL);
81772 |     return pWav->onWrite(pWav->pUserData, &byte, 1);
81773 | }
81774 | MA_PRIVATE size_t ma_dr_wav__write_u16ne_to_le(ma_dr_wav* pWav, ma_uint16 value)
81775 | {
81776 |     MA_DR_WAV_ASSERT(pWav != NULL);
81777 |     MA_DR_WAV_ASSERT(pWav->onWrite != NULL);
81778 |     if (!ma_dr_wav__is_little_endian()) {
81779 |         value = ma_dr_wav__bswap16(value);
81780 |     }
81781 |     return ma_dr_wav__write(pWav, &value, 2);
81782 | }
81783 | MA_PRIVATE size_t ma_dr_wav__write_u32ne_to_le(ma_dr_wav* pWav, ma_uint32 value)
81784 | {
81785 |     MA_DR_WAV_ASSERT(pWav != NULL);
81786 |     MA_DR_WAV_ASSERT(pWav->onWrite != NULL);
81787 |     if (!ma_dr_wav__is_little_endian()) {
81788 |         value = ma_dr_wav__bswap32(value);
81789 |     }
81790 |     return ma_dr_wav__write(pWav, &value, 4);
81791 | }
81792 | MA_PRIVATE size_t ma_dr_wav__write_u64ne_to_le(ma_dr_wav* pWav, ma_uint64 value)
81793 | {
81794 |     MA_DR_WAV_ASSERT(pWav != NULL);
81795 |     MA_DR_WAV_ASSERT(pWav->onWrite != NULL);
81796 |     if (!ma_dr_wav__is_little_endian()) {
81797 |         value = ma_dr_wav__bswap64(value);
81798 |     }
81799 |     return ma_dr_wav__write(pWav, &value, 8);
81800 | }
81801 | MA_PRIVATE size_t ma_dr_wav__write_f32ne_to_le(ma_dr_wav* pWav, float value)
81802 | {
81803 |     union {
81804 |         ma_uint32 u32;
81805 |         float f32;
81806 |     } u;
81807 |     MA_DR_WAV_ASSERT(pWav != NULL);
81808 |     MA_DR_WAV_ASSERT(pWav->onWrite != NULL);
81809 |     u.f32 = value;
81810 |     if (!ma_dr_wav__is_little_endian()) {

```

```

81811 |         u.u32 = ma_dr_wav__bswap32(u.u32);
81812 |     }
81813 |     return ma_dr_wav__write(pWav, &u.u32, 4);
81814 | }
81815 | MA_PRIVATE size_t ma_dr_wav__write_or_count(ma_dr_wav* pWav, const void* pData, size_t dataSize)
81816 | {
81817 |     if (pWav == NULL) {
81818 |         return dataSize;
81819 |     }
81820 |     return ma_dr_wav__write(pWav, pData, dataSize);
81821 | }
81822 | MA_PRIVATE size_t ma_dr_wav__write_or_count_byte(ma_dr_wav* pWav, ma_uint8 byte)
81823 | {
81824 |     if (pWav == NULL) {
81825 |         return 1;
81826 |     }
81827 |     return ma_dr_wav__write_byte(pWav, byte);
81828 | }
81829 | MA_PRIVATE size_t ma_dr_wav__write_or_count_u16ne_to_le(ma_dr_wav* pWav, ma_uint16 value)
81830 | {
81831 |     if (pWav == NULL) {
81832 |         return 2;
81833 |     }
81834 |     return ma_dr_wav__write_u16ne_to_le(pWav, value);
81835 | }
81836 | MA_PRIVATE size_t ma_dr_wav__write_or_count_u32ne_to_le(ma_dr_wav* pWav, ma_uint32 value)
81837 | {
81838 |     if (pWav == NULL) {
81839 |         return 4;
81840 |     }
81841 |     return ma_dr_wav__write_u32ne_to_le(pWav, value);
81842 | }
81843 | #if 0
81844 | MA_PRIVATE size_t ma_dr_wav__write_or_count_u64ne_to_le(ma_dr_wav* pWav, ma_uint64 value)
81845 | {
81846 |     if (pWav == NULL) {
81847 |         return 8;
81848 |     }
81849 |     return ma_dr_wav__write_u64ne_to_le(pWav, value);
81850 | }
81851 | #endif
81852 | MA_PRIVATE size_t ma_dr_wav__write_or_count_f32ne_to_le(ma_dr_wav* pWav, float value)
81853 | {
81854 |     if (pWav == NULL) {
81855 |         return 4;
81856 |     }
81857 |     return ma_dr_wav__write_f32ne_to_le(pWav, value);
81858 | }
81859 | MA_PRIVATE size_t ma_dr_wav__write_or_count_string_to_fixed_size_buf(ma_dr_wav* pWav, char* str,
> size_t bufFixedSize)
81860 | {
81861 |     size_t len;
81862 |     if (pWav == NULL) {
81863 |         return bufFixedSize;
81864 |     }
81865 |     len = ma_dr_wav__strlen_clamped(str, bufFixedSize);
81866 |     ma_dr_wav__write_or_count(pWav, str, len);
81867 |     if (len < bufFixedSize) {
81868 |         size_t i;
81869 |         for (i = 0; i < bufFixedSize - len; ++i) {
81870 |             ma_dr_wav__write_byte(pWav, 0);
81871 |         }
81872 |     }
81873 |     return bufFixedSize;
81874 | }
81875 | MA_PRIVATE size_t ma_dr_wav__write_or_count_metadata(ma_dr_wav* pWav, ma_dr_wav_metadata*
> pMetadatas, ma_uint32 metadataCount)
81876 | {
81877 |     size_t bytesWritten = 0;
81878 |     ma_bool32 hasListAdtl = MA_FALSE;
81879 |     ma_bool32 hasListInfo = MA_FALSE;
81880 |     ma_uint32 iMetadata;
81881 |     if (pMetadatas == NULL || metadataCount == 0) {
81882 |         return 0;
81883 |     }
81884 |     for (iMetadata = 0; iMetadata < metadataCount; ++iMetadata) {
81885 |         ma_dr_wav_metadata* pMetadata = &pMetadatas[iMetadata];
81886 |         ma_uint32 chunkSize = 0;
81887 |         if ((pMetadata->type & ma_dr_wav_metadata_type_list_all_info_strings) || (pMetadata->type ==

```

```

> ma_dr_wav_metadata_type_unknown && pMetadata->data.unknown.chunkLocation ==
> ma_dr_wav_metadata_location_inside_info_list)) {
81888 |     hasListInfo = MA_TRUE;
81889 | }
81890 |     if ((pMetadata->type & ma_dr_wav_metadata_type_list_all_adtl) || (pMetadata->type ==
> ma_dr_wav_metadata_type_unknown && pMetadata->data.unknown.chunkLocation ==
> ma_dr_wav_metadata_location_inside_adtl_list)) {
81891 |         hasListAdtl = MA_TRUE;
81892 |     }
81893 |     switch (pMetadata->type) {
81894 |     case ma_dr_wav_metadata_type_smpl:
81895 |     {
81896 |         ma_uint32 iLoop;
81897 |         chunkSize = MA_DR_WAV_SMPL_BYTES + MA_DR_WAV_SMPL_LOOP_BYTES *
> pMetadata->data.smpl.sampleLoopCount + pMetadata->data.smpl.samplerSpecificDataSizeInBytes;
81898 |         bytesWritten += ma_dr_wav__write_or_count(pWav, "smpl", 4);
81899 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, chunkSize);
81900 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.manufacturerId);
81901 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.productId);
81902 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.samplePeriodNanoseconds);
81903 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.midiUnityNote);
81904 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.midiPitchFraction);
81905 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.smpteFormat);
81906 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.smpteOffset);
81907 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.sampleLoopCount);
81908 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.samplerSpecificDataSizeInBytes);
81909 |         for (iLoop = 0; iLoop < pMetadata->data.smpl.sampleLoopCount; ++iLoop) {
81910 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.pLoops[iLoop].cuePointId);
81911 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.pLoops[iLoop].type);
81912 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.pLoops[iLoop].firstSampleOffset);
81913 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.pLoops[iLoop].lastSampleOffset);
81914 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.pLoops[iLoop].sampleFraction);
81915 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.smpl.pLoops[iLoop].playCount);
81916 |         }
81917 |         if (pMetadata->data.smpl.samplerSpecificDataSizeInBytes > 0) {
81918 |             bytesWritten += ma_dr_wav__write_or_count(pWav,
> pMetadata->data.smpl.pSamplerSpecificData, pMetadata->data.smpl.samplerSpecificDataSizeInBytes);
81919 |         }
81920 |     } break;
81921 |     case ma_dr_wav_metadata_type_inst:
81922 |     {
81923 |         chunkSize = MA_DR_WAV_INST_BYTES;
81924 |         bytesWritten += ma_dr_wav__write_or_count(pWav, "inst", 4);
81925 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, chunkSize);
81926 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.midiUnityNote,
> 1);
81927 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.fineTuneCents,
> 1);
81928 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.gainDecibels,
> 1);
81929 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.lowNote, 1);
81930 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.highNote, 1);
81931 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.lowVelocity,
> 1);
81932 |         bytesWritten += ma_dr_wav__write_or_count(pWav, &pMetadata->data.inst.highVelocity,
> 1);
81933 |     } break;
81934 |     case ma_dr_wav_metadata_type_cue:
81935 |     {
81936 |         ma_uint32 iCuePoint;
81937 |         chunkSize = MA_DR_WAV_CUE_BYTES + MA_DR_WAV_CUE_POINT_BYTES *
> pMetadata->data.cue.cuePointCount;
81938 |         bytesWritten += ma_dr_wav__write_or_count(pWav, "cue ", 4);
81939 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, chunkSize);

```



```

81940 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.cue.cuePointCount);
81941 |         for (iCuePoint = 0; iCuePoint < pMetadata->data.cue.cuePointCount; ++iCuePoint) {
81942 |             bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.cue.pCuePoints[iCuePoint].id);
81943 |             bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.cue.pCuePoints[iCuePoint].playOrderPosition);
81944 |             bytesWritten += ma_dr_wav_write_or_count(pWav,
> pMetadata->data.cue.pCuePoints[iCuePoint].dataChunkId, 4);
81945 |             bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.cue.pCuePoints[iCuePoint].chunkStart);
81946 |             bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.cue.pCuePoints[iCuePoint].blockStart);
81947 |             bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.cue.pCuePoints[iCuePoint].sampleOffset);
81948 |         }
81949 |     } break;
81950 |     case ma_dr_wav_metadata_type_acid:
81951 |     {
81952 |         chunkSize = MA_DR_WAV_ACID_BYTES;
81953 |         bytesWritten += ma_dr_wav_write_or_count(pWav, "acid", 4);
81954 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav, chunkSize);
81955 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.acid.flags);
81956 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.acid.midiUnityNote);
81957 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.acid.reserved1);
81958 |         bytesWritten += ma_dr_wav_write_or_count_f32ne_to_le(pWav,
> pMetadata->data.acid.reserved2);
81959 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav,
> pMetadata->data.acid.numBeats);
81960 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.acid.meterDenominator);
81961 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.acid.meterNumerator);
81962 |         bytesWritten += ma_dr_wav_write_or_count_f32ne_to_le(pWav,
> pMetadata->data.acid.tempo);
81963 |     } break;
81964 |     case ma_dr_wav_metadata_type_bext:
81965 |     {
81966 |         char reservedBuf[MA_DR_WAV_BEXT_RESERVED_BYTES];
81967 |         ma_uint32 timeReferenceLow;
81968 |         ma_uint32 timeReferenceHigh;
81969 |         chunkSize = MA_DR_WAV_BEXT_BYTES + pMetadata->data.bext.codingHistorySize;
81970 |         bytesWritten += ma_dr_wav_write_or_count(pWav, "bext", 4);
81971 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav, chunkSize);
81972 |         bytesWritten += ma_dr_wav_write_or_count_string_to_fixed_size_buf(pWav,
> pMetadata->data.bext.pDescription, MA_DR_WAV_BEXT_DESCRIPTION_BYTES);
81973 |         bytesWritten += ma_dr_wav_write_or_count_string_to_fixed_size_buf(pWav,
> pMetadata->data.bext.pOriginatorName, MA_DR_WAV_BEXT_ORIGINATOR_NAME_BYTES);
81974 |         bytesWritten += ma_dr_wav_write_or_count_string_to_fixed_size_buf(pWav,
> pMetadata->data.bext.pOriginatorReference, MA_DR_WAV_BEXT_ORIGINATOR_REF_BYTES);
81975 |         bytesWritten += ma_dr_wav_write_or_count(pWav,
> pMetadata->data.bext.pOriginationDate, sizeof(pMetadata->data.bext.pOriginationDate));
81976 |         bytesWritten += ma_dr_wav_write_or_count(pWav,
> pMetadata->data.bext.pOriginationTime, sizeof(pMetadata->data.bext.pOriginationTime));
81977 |         timeReferenceLow = (ma_uint32)(pMetadata->data.bext.timeReference & 0xFFFFFFFF);
81978 |         timeReferenceHigh = (ma_uint32)(pMetadata->data.bext.timeReference >> 32);
81979 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav, timeReferenceLow);
81980 |         bytesWritten += ma_dr_wav_write_or_count_u32ne_to_le(pWav, timeReferenceHigh);
81981 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.bext.version);
81982 |         bytesWritten += ma_dr_wav_write_or_count(pWav, pMetadata->data.bext.pUMID,
> MA_DR_WAV_BEXT_UMID_BYTES);
81983 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.bext.loudnessValue);
81984 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.bext.loudnessRange);
81985 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.bext.maxTruePeakLevel);
81986 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.bext.maxMomentaryLoudness);
81987 |         bytesWritten += ma_dr_wav_write_or_count_u16ne_to_le(pWav,
> pMetadata->data.bext.maxShortTermLoudness);
81988 |         MA_DR_WAV_ZERO_MEMORY(reservedBuf, sizeof(reservedBuf));
81989 |         bytesWritten += ma_dr_wav_write_or_count(pWav, reservedBuf, sizeof(reservedBuf));
81990 |         if (pMetadata->data.bext.codingHistorySize > 0) {
81991 |             bytesWritten += ma_dr_wav_write_or_count(pWav,

```

```

> pMetadata->data.bext.pCodingHistory, pMetadata->data.bext.codingHistorySize);
81992 |     }
81993 |     } break;
81994 |     case ma_dr_wav_metadata_type_unknown:
81995 |     {
81996 |         if (pMetadata->data.unknown.chunkLocation == ma_dr_wav_metadata_location_top_level)
> {
81997 |             chunkSize = pMetadata->data.unknown.dataSizeInBytes;
81998 |             bytesWritten += ma_dr_wav__write_or_count(pWav, pMetadata->data.unknown.id, 4);
81999 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, chunkSize);
82000 |             bytesWritten += ma_dr_wav__write_or_count(pWav, pMetadata->data.unknown.pData,
> pMetadata->data.unknown.dataSizeInBytes);
82001 |         }
82002 |     } break;
82003 |     default: break;
82004 | }
82005 | if ((chunkSize % 2) != 0) {
82006 |     bytesWritten += ma_dr_wav__write_or_count_byte(pWav, 0);
82007 | }
82008 | }
82009 | if (hasListInfo) {
82010 |     ma_uint32 chunkSize = 4;
82011 |     for (iMetadata = 0; iMetadata < metadataCount; ++iMetadata) {
82012 |         ma_dr_wav_metadata* pMetadata = &pMetadatas[iMetadata];
82013 |         if ((pMetadata->type & ma_dr_wav_metadata_type_list_all_info_strings)) {
82014 |             chunkSize += 8;
82015 |             chunkSize += pMetadata->data.infoText.stringLength + 1;
82016 |         } else if (pMetadata->type == ma_dr_wav_metadata_type_unknown &&
> pMetadata->data.unknown.chunkLocation == ma_dr_wav_metadata_location_inside_info_list) {
82017 |             chunkSize += 8;
82018 |             chunkSize += pMetadata->data.unknown.dataSizeInBytes;
82019 |         }
82020 |         if ((chunkSize % 2) != 0) {
82021 |             chunkSize += 1;
82022 |         }
82023 |     }
82024 |     bytesWritten += ma_dr_wav__write_or_count(pWav, "LIST", 4);
82025 |     bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, chunkSize);
82026 |     bytesWritten += ma_dr_wav__write_or_count(pWav, "INFO", 4);
82027 |     for (iMetadata = 0; iMetadata < metadataCount; ++iMetadata) {
82028 |         ma_dr_wav_metadata* pMetadata = &pMetadatas[iMetadata];
82029 |         ma_uint32 subchunkSize = 0;
82030 |         if (pMetadata->type & ma_dr_wav_metadata_type_list_all_info_strings) {
82031 |             const char* pID = NULL;
82032 |             switch (pMetadata->type) {
82033 |                 case ma_dr_wav_metadata_type_list_info_software:    pID = "ISFT"; break;
82034 |                 case ma_dr_wav_metadata_type_list_info_copyright:  pID = "ICOP"; break;
82035 |                 case ma_dr_wav_metadata_type_list_info_title:       pID = "INAM"; break;
82036 |                 case ma_dr_wav_metadata_type_list_info_artist:      pID = "IART"; break;
82037 |                 case ma_dr_wav_metadata_type_list_info_comment:     pID = "ICMT"; break;
82038 |                 case ma_dr_wav_metadata_type_list_info_date:        pID = "ICRD"; break;
82039 |                 case ma_dr_wav_metadata_type_list_info_genre:       pID = "IGNR"; break;
82040 |                 case ma_dr_wav_metadata_type_list_info_album:       pID = "IPRD"; break;
82041 |                 case ma_dr_wav_metadata_type_list_info_tracknumber: pID = "ITRK"; break;
82042 |                 case ma_dr_wav_metadata_type_list_info_location:    pID = "IARL"; break;
82043 |                 case ma_dr_wav_metadata_type_list_info_organization: pID = "ICMS"; break;
82044 |                 case ma_dr_wav_metadata_type_list_info_keywords:    pID = "IKEY"; break;
82045 |                 case ma_dr_wav_metadata_type_list_info_medium:      pID = "IMED"; break;
82046 |                 case ma_dr_wav_metadata_type_list_info_description: pID = "ISBJ"; break;
82047 |                 default: break;
82048 |             }
82049 |             MA_DR_WAV_ASSERT(pID != NULL);
82050 |             if (pMetadata->data.infoText.stringLength) {
82051 |                 subchunkSize = pMetadata->data.infoText.stringLength + 1;
82052 |                 bytesWritten += ma_dr_wav__write_or_count(pWav, pID, 4);
82053 |                 bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, subchunkSize);
82054 |                 bytesWritten += ma_dr_wav__write_or_count(pWav,
> pMetadata->data.infoText.pString, pMetadata->data.infoText.stringLength);
82055 |                 bytesWritten += ma_dr_wav__write_or_count_byte(pWav, '\0');
82056 |             }
82057 |         } else if (pMetadata->type == ma_dr_wav_metadata_type_unknown &&
> pMetadata->data.unknown.chunkLocation == ma_dr_wav_metadata_location_inside_info_list) {
82058 |             if (pMetadata->data.unknown.dataSizeInBytes) {
82059 |                 subchunkSize = pMetadata->data.unknown.dataSizeInBytes;
82060 |                 bytesWritten += ma_dr_wav__write_or_count(pWav, pMetadata->data.unknown.id, 4);
82061 |                 bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.unknown.dataSizeInBytes);
82062 |                 bytesWritten += ma_dr_wav__write_or_count(pWav, pMetadata->data.unknown.pData,
> subchunkSize);

```

```

82063 |     }
82064 | }
82065 | if ((subchunkSize % 2) != 0) {
82066 |     bytesWritten += ma_dr_wav__write_or_count_byte(pWav, 0);
82067 | }
82068 | }
82069 | }
82070 | if (hasListAdtl) {
82071 |     ma_uint32 chunkSize = 4;
82072 |     for (iMetadata = 0; iMetadata < metadataCount; ++iMetadata) {
82073 |         ma_dr_wav_metadata* pMetadata = &pMetadatas[iMetadata];
82074 |         switch (pMetadata->type)
82075 |         {
82076 |             case ma_dr_wav_metadata_type_list_label:
82077 |             case ma_dr_wav_metadata_type_list_note:
82078 |             {
82079 |                 chunkSize += 8;
82080 |                 chunkSize += MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES;
82081 |                 if (pMetadata->data.labelOrNote.stringLength > 0) {
82082 |                     chunkSize += pMetadata->data.labelOrNote.stringLength + 1;
82083 |                 }
82084 |             } break;
82085 |             case ma_dr_wav_metadata_type_list_labelled_cue_region:
82086 |             {
82087 |                 chunkSize += 8;
82088 |                 chunkSize += MA_DR_WAV_LIST_LABELLED_TEXT_BYTES;
82089 |                 if (pMetadata->data.labelledCueRegion.stringLength > 0) {
82090 |                     chunkSize += pMetadata->data.labelledCueRegion.stringLength + 1;
82091 |                 }
82092 |             } break;
82093 |             case ma_dr_wav_metadata_type_unknown:
82094 |             {
82095 |                 if (pMetadata->data.unknown.chunkLocation ==
> ma_dr_wav_metadata_location_inside_adtl_list) {
82096 |                     chunkSize += 8;
82097 |                     chunkSize += pMetadata->data.unknown.dataSizeInBytes;
82098 |                 }
82099 |             } break;
82100 |             default: break;
82101 |         }
82102 |         if ((chunkSize % 2) != 0) {
82103 |             chunkSize += 1;
82104 |         }
82105 |     }
82106 |     bytesWritten += ma_dr_wav__write_or_count(pWav, "LIST", 4);
82107 |     bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, chunkSize);
82108 |     bytesWritten += ma_dr_wav__write_or_count(pWav, "adtl", 4);
82109 |     for (iMetadata = 0; iMetadata < metadataCount; ++iMetadata) {
82110 |         ma_dr_wav_metadata* pMetadata = &pMetadatas[iMetadata];
82111 |         ma_uint32 subchunkSize = 0;
82112 |         switch (pMetadata->type)
82113 |         {
82114 |             case ma_dr_wav_metadata_type_list_label:
82115 |             case ma_dr_wav_metadata_type_list_note:
82116 |             {
82117 |                 if (pMetadata->data.labelOrNote.stringLength > 0) {
82118 |                     const char *pID = NULL;
82119 |                     if (pMetadata->type == ma_dr_wav_metadata_type_list_label) {
82120 |                         pID = "labl";
82121 |                     }
82122 |                     else if (pMetadata->type == ma_dr_wav_metadata_type_list_note) {
82123 |                         pID = "note";
82124 |                     }
82125 |                     MA_DR_WAV_ASSERT(pID != NULL);
82126 |                     MA_DR_WAV_ASSERT(pMetadata->data.labelOrNote.pString != NULL);
82127 |                     subchunkSize = MA_DR_WAV_LIST_LABEL_OR_NOTE_BYTES;
82128 |                     bytesWritten += ma_dr_wav__write_or_count(pWav, pID, 4);
82129 |                     subchunkSize += pMetadata->data.labelOrNote.stringLength + 1;
82130 |                     bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, subchunkSize);
82131 |                     bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.labelOrNote.cuePointId);
82132 |                     bytesWritten += ma_dr_wav__write_or_count(pWav,
> pMetadata->data.labelOrNote.pString, pMetadata->data.labelOrNote.stringLength);
82133 |                     bytesWritten += ma_dr_wav__write_or_count_byte(pWav, '\0');
82134 |                 }
82135 |             } break;
82136 |             case ma_dr_wav_metadata_type_list_labelled_cue_region:
82137 |             {
82138 |                 subchunkSize = MA_DR_WAV_LIST_LABELLED_TEXT_BYTES;

```

```

82139 |         bytesWritten += ma_dr_wav__write_or_count(pWav, "ltxt", 4);
82140 |         if (pMetadata->data.labelledCueRegion.stringLength > 0) {
82141 |             subchunkSize += pMetadata->data.labelledCueRegion.stringLength + 1;
82142 |         }
82143 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, subchunkSize);
82144 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.labelledCueRegion.cuePointId);
82145 |         bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav,
> pMetadata->data.labelledCueRegion.sampleLength);
82146 |         bytesWritten += ma_dr_wav__write_or_count(pWav,
> pMetadata->data.labelledCueRegion.purposeId, 4);
82147 |         bytesWritten += ma_dr_wav__write_or_count_u16ne_to_le(pWav,
> pMetadata->data.labelledCueRegion.country);
82148 |         bytesWritten += ma_dr_wav__write_or_count_u16ne_to_le(pWav,
> pMetadata->data.labelledCueRegion.language);
82149 |         bytesWritten += ma_dr_wav__write_or_count_u16ne_to_le(pWav,
> pMetadata->data.labelledCueRegion.dialect);
82150 |         bytesWritten += ma_dr_wav__write_or_count_u16ne_to_le(pWav,
> pMetadata->data.labelledCueRegion.codePage);
82151 |         if (pMetadata->data.labelledCueRegion.stringLength > 0) {
82152 |             MA_DR_WAV_ASSERT(pMetadata->data.labelledCueRegion.pString != NULL);
82153 |             bytesWritten += ma_dr_wav__write_or_count(pWav,
> pMetadata->data.labelledCueRegion.pString, pMetadata->data.labelledCueRegion.stringLength);
82154 |             bytesWritten += ma_dr_wav__write_or_count_byte(pWav, '\0');
82155 |         }
82156 |     } break;
82157 |     case ma_dr_wav_metadata_type_unknown:
82158 |     {
82159 |         if (pMetadata->data.unknown.chunkLocation ==
> ma_dr_wav_metadata_location_inside_adtl_list) {
82160 |             subchunkSize = pMetadata->data.unknown.dataSizeInBytes;
82161 |             MA_DR_WAV_ASSERT(pMetadata->data.unknown.pData != NULL);
82162 |             bytesWritten += ma_dr_wav__write_or_count(pWav, pMetadata->data.unknown.id,
> 4);
82163 |             bytesWritten += ma_dr_wav__write_or_count_u32ne_to_le(pWav, subchunkSize);
82164 |             bytesWritten += ma_dr_wav__write_or_count(pWav,
> pMetadata->data.unknown.pData, subchunkSize);
82165 |         }
82166 |     } break;
82167 |     default: break;
82168 | }
82169 | if ((subchunkSize % 2) != 0) {
82170 |     bytesWritten += ma_dr_wav__write_or_count_byte(pWav, 0);
82171 | }
82172 | }
82173 | }
82174 | MA_DR_WAV_ASSERT((bytesWritten % 2) == 0);
82175 | return bytesWritten;
82176 | }
82177 | MA_PRIVATE ma_uint32 ma_dr_wav__riff_chunk_size_riff(ma_uint64 dataChunkSize, ma_dr_wav_metadata*
> pMetadata, ma_uint32 metadataCount)
82178 | {
82179 |     ma_uint64 chunkSize = 4 + 24 + (ma_uint64)ma_dr_wav__write_or_count_metadata(NULL, pMetadata,
> metadataCount) + 8 + dataChunkSize + ma_dr_wav__chunk_padding_size_riff(dataChunkSize);
82180 |     if (chunkSize > 0xFFFFFFFFFUL) {
82181 |         chunkSize = 0xFFFFFFFFFUL;
82182 |     }
82183 |     return (ma_uint32)chunkSize;
82184 | }
82185 | MA_PRIVATE ma_uint32 ma_dr_wav__data_chunk_size_riff(ma_uint64 dataChunkSize)
82186 | {
82187 |     if (dataChunkSize <= 0xFFFFFFFFFUL) {
82188 |         return (ma_uint32)dataChunkSize;
82189 |     } else {
82190 |         return 0xFFFFFFFFFUL;
82191 |     }
82192 | }
82193 | MA_PRIVATE ma_uint64 ma_dr_wav__riff_chunk_size_w64(ma_uint64 dataChunkSize)
82194 | {
82195 |     ma_uint64 dataSubchunkPaddingSize = ma_dr_wav__chunk_padding_size_w64(dataChunkSize);
82196 |     return 80 + 24 + dataChunkSize + dataSubchunkPaddingSize;
82197 | }
82198 | MA_PRIVATE ma_uint64 ma_dr_wav__data_chunk_size_w64(ma_uint64 dataChunkSize)
82199 | {
82200 |     return 24 + dataChunkSize;
82201 | }
82202 | MA_PRIVATE ma_uint64 ma_dr_wav__riff_chunk_size_rf64(ma_uint64 dataChunkSize, ma_dr_wav_metadata
> *metadata, ma_uint32 numMetadata)
82203 | {

```

```

82204 |     ma_uint64 chunkSize = 4 + 36 + 24 + (ma_uint64)ma_dr_wav__write_or_count_metadata(NULL,
> metadata, numMetadata) + 8 + dataChunkSize + ma_dr_wav__chunk_padding_size_riff(dataChunkSize);
82205 |     if (chunkSize > 0xFFFFFFFFUL) {
82206 |         chunkSize = 0xFFFFFFFFUL;
82207 |     }
82208 |     return chunkSize;
82209 | }
82210 | MA_PRIVATE ma_uint64 ma_dr_wav__data_chunk_size_rf64(ma_uint64 dataChunkSize)
82211 | {
82212 |     return dataChunkSize;
82213 | }
82214 | MA_PRIVATE ma_bool32 ma_dr_wav_preinit_write(ma_dr_wav* pWav, const ma_dr_wav_data_format* pFormat,
> ma_bool32 isSequential, ma_dr_wav_write_proc onWrite, ma_dr_wav_seek_proc onSeek, void* pUserData,
> const ma_allocation_callbacks* pAllocationCallbacks)
82215 | {
82216 |     if (pWav == NULL || onWrite == NULL) {
82217 |         return MA_FALSE;
82218 |     }
82219 |     if (!isSequential && onSeek == NULL) {
82220 |         return MA_FALSE;
82221 |     }
82222 |     if (pFormat->format == MA_DR_WAVE_FORMAT_EXTENSIBLE) {
82223 |         return MA_FALSE;
82224 |     }
82225 |     if (pFormat->format == MA_DR_WAVE_FORMAT_ADPCM || pFormat->format ==
> MA_DR_WAVE_FORMAT_DVI_ADPCM) {
82226 |         return MA_FALSE;
82227 |     }
82228 |     MA_DR_WAV_ZERO_MEMORY(pWav, sizeof(*pWav));
82229 |     pWav->onWrite = onWrite;
82230 |     pWav->onSeek = onSeek;
82231 |     pWav->pUserData = pUserData;
82232 |     pWav->allocationCallbacks =
> ma_dr_wav_copy_allocation_callbacks_or_defaults(pAllocationCallbacks);
82233 |     if (pWav->allocationCallbacks.onFree == NULL || (pWav->allocationCallbacks.onMalloc == NULL &&
> pWav->allocationCallbacks.onRealloc == NULL)) {
82234 |         return MA_FALSE;
82235 |     }
82236 |     pWav->fmt.formatTag = (ma_uint16)pFormat->format;
82237 |     pWav->fmt.channels = (ma_uint16)pFormat->channels;
82238 |     pWav->fmt.sampleRate = pFormat->sampleRate;
82239 |     pWav->fmt.avgBytesPerSec = (ma_uint32)((pFormat->bitsPerSample * pFormat->sampleRate *
> pFormat->channels) / 8);
82240 |     pWav->fmt.blockAlign = (ma_uint16)((pFormat->channels * pFormat->bitsPerSample) / 8);
82241 |     pWav->fmt.bitsPerSample = (ma_uint16)pFormat->bitsPerSample;
82242 |     pWav->fmt.extendedSize = 0;
82243 |     pWav->isSequentialWrite = isSequential;
82244 |     return MA_TRUE;
82245 | }
82246 | MA_PRIVATE ma_bool32 ma_dr_wav_init_write__internal(ma_dr_wav* pWav, const ma_dr_wav_data_format*
> pFormat, ma_uint64 totalSampleCount)
82247 | {
82248 |     size_t runningPos = 0;
82249 |     ma_uint64 initialDataChunkSize = 0;
82250 |     ma_uint64 chunkSizeFMT;
82251 |     if (pWav->isSequentialWrite) {
82252 |         initialDataChunkSize = (totalSampleCount * pWav->fmt.bitsPerSample) / 8;
82253 |         if (pFormat->container == ma_dr_wav_container_riff) {
82254 |             if (initialDataChunkSize > (0xFFFFFFFFUL - 36)) {
82255 |                 return MA_FALSE;
82256 |             }
82257 |         }
82258 |     }
82259 |     pWav->dataChunkDataSizeTargetWrite = initialDataChunkSize;
82260 |     if (pFormat->container == ma_dr_wav_container_riff) {
82261 |         ma_uint32 chunkSizeRIFF = 36 + (ma_uint32)initialDataChunkSize;
82262 |         runningPos += ma_dr_wav__write(pWav, "RIFF", 4);
82263 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, chunkSizeRIFF);
82264 |         runningPos += ma_dr_wav__write(pWav, "WAVE", 4);
82265 |     } else if (pFormat->container == ma_dr_wav_container_w64) {
82266 |         ma_uint64 chunkSizeRIFF = 80 + 24 + initialDataChunkSize;
82267 |         runningPos += ma_dr_wav__write(pWav, ma_dr_wavGUID_W64_RIFF, 16);
82268 |         runningPos += ma_dr_wav__write_u64ne_to_le(pWav, chunkSizeRIFF);
82269 |         runningPos += ma_dr_wav__write(pWav, ma_dr_wavGUID_W64_WAVE, 16);
82270 |     } else if (pFormat->container == ma_dr_wav_container_rf64) {
82271 |         runningPos += ma_dr_wav__write(pWav, "RF64", 4);
82272 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, 0xFFFFFFFF);
82273 |         runningPos += ma_dr_wav__write(pWav, "WAVE", 4);
82274 |     } else {

```

```

82275 |         return MA_FALSE;
82276 |     }
82277 |     if (pFormat->container == ma_dr_wav_container_rf64) {
82278 |         ma_uint32 initialds64ChunkSize = 28;
82279 |         ma_uint64 initialRiffChunkSize = 8 + initialds64ChunkSize + initialDataChunkSize;
82280 |         runningPos += ma_dr_wav__write(pWav, "ds64", 4);
82281 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, initialds64ChunkSize);
82282 |         runningPos += ma_dr_wav__write_u64ne_to_le(pWav, initialRiffChunkSize);
82283 |         runningPos += ma_dr_wav__write_u64ne_to_le(pWav, initialDataChunkSize);
82284 |         runningPos += ma_dr_wav__write_u64ne_to_le(pWav, totalSampleCount);
82285 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, 0);
82286 |     }
82287 |     if (pFormat->container == ma_dr_wav_container_riff || pFormat->container ==
> ma_dr_wav_container_rf64) {
82288 |         chunkSizeFMT = 16;
82289 |         runningPos += ma_dr_wav__write(pWav, "fmt ", 4);
82290 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, (ma_uint32)chunkSizeFMT);
82291 |     } else if (pFormat->container == ma_dr_wav_container_w64) {
82292 |         chunkSizeFMT = 40;
82293 |         runningPos += ma_dr_wav__write(pWav, ma_dr_wavGUID_W64_FMT, 16);
82294 |         runningPos += ma_dr_wav__write_u64ne_to_le(pWav, chunkSizeFMT);
82295 |     }
82296 |     runningPos += ma_dr_wav__write_u16ne_to_le(pWav, pWav->fmt.formatTag);
82297 |     runningPos += ma_dr_wav__write_u16ne_to_le(pWav, pWav->fmt.channels);
82298 |     runningPos += ma_dr_wav__write_u32ne_to_le(pWav, pWav->fmt.sampleRate);
82299 |     runningPos += ma_dr_wav__write_u32ne_to_le(pWav, pWav->fmt.avgBytesPerSec);
82300 |     runningPos += ma_dr_wav__write_u16ne_to_le(pWav, pWav->fmt.blockAlign);
82301 |     runningPos += ma_dr_wav__write_u16ne_to_le(pWav, pWav->fmt.bitsPerSample);
82302 |     if (!pWav->isSequentialWrite && pWav->pMetadata != NULL && pWav->metadataCount > 0 &&
> (pFormat->container == ma_dr_wav_container_riff || pFormat->container == ma_dr_wav_container_rf64))
> {
82303 |         runningPos += ma_dr_wav__write_or_count_metadata(pWav, pWav->pMetadata,
> pWav->metadataCount);
82304 |     }
82305 |     pWav->dataChunkDataPos = runningPos;
82306 |     if (pFormat->container == ma_dr_wav_container_riff) {
82307 |         ma_uint32 chunkSizeDATA = (ma_uint32)initialDataChunkSize;
82308 |         runningPos += ma_dr_wav__write(pWav, "data", 4);
82309 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, chunkSizeDATA);
82310 |     } else if (pFormat->container == ma_dr_wav_container_w64) {
82311 |         ma_uint64 chunkSizeDATA = 24 + initialDataChunkSize;
82312 |         runningPos += ma_dr_wav__write(pWav, ma_dr_wavGUID_W64_DATA, 16);
82313 |         runningPos += ma_dr_wav__write_u64ne_to_le(pWav, chunkSizeDATA);
82314 |     } else if (pFormat->container == ma_dr_wav_container_rf64) {
82315 |         runningPos += ma_dr_wav__write(pWav, "data", 4);
82316 |         runningPos += ma_dr_wav__write_u32ne_to_le(pWav, 0xFFFFFFFF);
82317 |     }
82318 |     pWav->container = pFormat->container;
82319 |     pWav->channels = (ma_uint16)pFormat->channels;
82320 |     pWav->sampleRate = pFormat->sampleRate;
82321 |     pWav->bitsPerSample = (ma_uint16)pFormat->bitsPerSample;
82322 |     pWav->translatedFormatTag = (ma_uint16)pFormat->format;
82323 |     pWav->dataChunkDataPos = runningPos;
82324 |     return MA_TRUE;
82325 | }
82326 | MA_API ma_bool32 ma_dr_wav_init_write(ma_dr_wav* pWav, const ma_dr_wav_data_format* pFormat,
> ma_dr_wav_write_proc onWrite, ma_dr_wav_seek_proc onSeek, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks)
82327 | {
82328 |     if (!ma_dr_wav_preinit_write(pWav, pFormat, MA_FALSE, onWrite, onSeek, pUserData,
> pAllocationCallbacks)) {
82329 |         return MA_FALSE;
82330 |     }
82331 |     return ma_dr_wav_init_write_internal(pWav, pFormat, 0);
82332 | }
82333 | MA_API ma_bool32 ma_dr_wav_init_write_sequential(ma_dr_wav* pWav, const ma_dr_wav_data_format*
> pFormat, ma_uint64 totalSampleCount, ma_dr_wav_write_proc onWrite, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks)
82334 | {
82335 |     if (!ma_dr_wav_preinit_write(pWav, pFormat, MA_TRUE, onWrite, NULL, pUserData,
> pAllocationCallbacks)) {
82336 |         return MA_FALSE;
82337 |     }
82338 |     return ma_dr_wav_init_write_internal(pWav, pFormat, totalSampleCount);
82339 | }
82340 | MA_API ma_bool32 ma_dr_wav_init_write_sequential_pcm_frames(ma_dr_wav* pWav, const
> ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, ma_dr_wav_write_proc onWrite, void*
> pUserData, const ma_allocation_callbacks* pAllocationCallbacks)
82341 | {

```



```

82342 |     if (pFormat == NULL) {
82343 |         return MA_FALSE;
82344 |     }
82345 |     return ma_dr_wav_init_write_sequential(pWav, pFormat, totalPCMFrameCount*pFormat->channels,
> onWrite, pUserData, pAllocationCallbacks);
82346 | }
82347 | MA_API ma_bool32 ma_dr_wav_init_write_with_metadata(ma_dr_wav* pWav, const ma_dr_wav_data_format*
> pFormat, ma_dr_wav_write_proc onWrite, ma_dr_wav_seek_proc onSeek, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks, ma_dr_wav_metadata* pMetadata, ma_uint32
> metadataCount)
82348 | {
82349 |     if (!ma_dr_wav_preinit_write(pWav, pFormat, MA_FALSE, onWrite, onSeek, pUserData,
> pAllocationCallbacks)) {
82350 |         return MA_FALSE;
82351 |     }
82352 |     pWav->pMetadata = pMetadata;
82353 |     pWav->metadataCount = metadataCount;
82354 |     return ma_dr_wav_init_write__internal(pWav, pFormat, 0);
82355 | }
82356 | MA_API ma_uint64 ma_dr_wav_target_write_size_bytes(const ma_dr_wav_data_format* pFormat, ma_uint64
> totalFrameCount, ma_dr_wav_metadata* pMetadata, ma_uint32 metadataCount)
82357 | {
82358 |     ma_uint64 targetDataSizeBytes = (ma_uint64)((ma_int64)totalFrameCount * pFormat->channels *
> pFormat->bitsPerSample/8.0);
82359 |     ma_uint64 riffChunkSizeBytes;
82360 |     ma_uint64 fileSizeBytes = 0;
82361 |     if (pFormat->container == ma_dr_wav_container_riff) {
82362 |         riffChunkSizeBytes = ma_dr_wav__riff_chunk_size_riff(targetDataSizeBytes, pMetadata,
> metadataCount);
82363 |         fileSizeBytes = (8 + riffChunkSizeBytes);
82364 |     } else if (pFormat->container == ma_dr_wav_container_w64) {
82365 |         riffChunkSizeBytes = ma_dr_wav__riff_chunk_size_w64(targetDataSizeBytes);
82366 |         fileSizeBytes = riffChunkSizeBytes;
82367 |     } else if (pFormat->container == ma_dr_wav_container_rf64) {
82368 |         riffChunkSizeBytes = ma_dr_wav__riff_chunk_size_rf64(targetDataSizeBytes, pMetadata,
> metadataCount);
82369 |         fileSizeBytes = (8 + riffChunkSizeBytes);
82370 |     }
82371 |     return fileSizeBytes;
82372 | }
82373 | #ifndef MA_DR_WAV_NO_STDIO
82374 | MA_PRIVATE size_t ma_dr_wav__on_read_stdio(void* pUserData, void* pBufferOut, size_t bytesToRead)
82375 | {
82376 |     return fread(pBufferOut, 1, bytesToRead, (FILE*)pUserData);
82377 | }
82378 | MA_PRIVATE size_t ma_dr_wav__on_write_stdio(void* pUserData, const void* pData, size_t bytesToWrite)
82379 | {
82380 |     return fwrite(pData, 1, bytesToWrite, (FILE*)pUserData);
82381 | }
82382 | MA_PRIVATE ma_bool32 ma_dr_wav__on_seek_stdio(void* pUserData, int offset, ma_dr_wav_seek_origin
> origin)
82383 | {
82384 |     int whence = SEEK_SET;
82385 |     if (origin == MA_DR_WAV_SEEK_CUR) {
82386 |         whence = SEEK_CUR;
82387 |     } else if (origin == MA_DR_WAV_SEEK_END) {
82388 |         whence = SEEK_END;
82389 |     }
82390 |     return fseek((FILE*)pUserData, offset, whence) == 0;
82391 | }
82392 | MA_PRIVATE ma_bool32 ma_dr_wav__on_tell_stdio(void* pUserData, ma_int64* pCursor)
82393 | {
82394 |     FILE* pFileStdio = (FILE*)pUserData;
82395 |     ma_int64 result;
82396 |     MA_DR_WAV_ASSERT(pFileStdio != NULL);
82397 |     MA_DR_WAV_ASSERT(pCursor != NULL);
82398 |     #if defined(_WIN32) && !defined(NXDK)
82399 |     #if defined(_MSC_VER) && _MSC_VER > 1200
82400 |         result = _ftelli64(pFileStdio);
82401 |     #else
82402 |         result = ftell(pFileStdio);
82403 |     #endif
82404 | #else
82405 |     result = ftell(pFileStdio);
82406 | #endif
82407 |     *pCursor = result;
82408 |     return MA_TRUE;
82409 | }
82410 | MA_API ma_bool32 ma_dr_wav_init_file(ma_dr_wav* pWav, const char* filename, const

```



```

> ma_allocation_callbacks* pAllocationCallbacks)
82411 | {
82412 |     return ma_dr_wav_init_file_ex(pWav, filename, NULL, NULL, 0, pAllocationCallbacks);
82413 | }
82414 | MA_PRIVATE ma_bool32 ma_dr_wav_init_file__internal_FILE(ma_dr_wav* pWav, FILE* pFile,
> ma_dr_wav_chunk_proc onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks*
> pAllocationCallbacks)
82415 | {
82416 |     ma_bool32 result;
82417 |     result = ma_dr_wav_preinit(pWav, ma_dr_wav__on_read_stdio, ma_dr_wav__on_seek_stdio,
> ma_dr_wav__on_tell_stdio, (void*)pFile, pAllocationCallbacks);
82418 |     if (result != MA_TRUE) {
82419 |         fclose(pFile);
82420 |         return result;
82421 |     }
82422 |     result = ma_dr_wav_init__internal(pWav, onChunk, pChunkUserData, flags);
82423 |     if (result != MA_TRUE) {
82424 |         fclose(pFile);
82425 |         return result;
82426 |     }
82427 |     return MA_TRUE;
82428 | }
82429 | MA_API ma_bool32 ma_dr_wav_init_file_ex(ma_dr_wav* pWav, const char* filename, ma_dr_wav_chunk_proc
> onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks)
82430 | {
82431 |     FILE* pFile;
82432 |     if (ma_fopen(&pFile, filename, "rb") != MA_SUCCESS) {
82433 |         return MA_FALSE;
82434 |     }
82435 |     return ma_dr_wav_init_file__internal_FILE(pWav, pFile, onChunk, pChunkUserData, flags,
> pAllocationCallbacks);
82436 | }
82437 | #ifndef MA_DR_WAV_NO_WCHAR
82438 | MA_API ma_bool32 ma_dr_wav_init_file_w(ma_dr_wav* pWav, const wchar_t* filename, const
> ma_allocation_callbacks* pAllocationCallbacks)
82439 | {
82440 |     return ma_dr_wav_init_file_ex_w(pWav, filename, NULL, NULL, 0, pAllocationCallbacks);
82441 | }
82442 | MA_API ma_bool32 ma_dr_wav_init_file_ex_w(ma_dr_wav* pWav, const wchar_t* filename,
> ma_dr_wav_chunk_proc onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks*
> pAllocationCallbacks)
82443 | {
82444 |     FILE* pFile;
82445 |     if (ma_wfopen(&pFile, filename, L"rb", pAllocationCallbacks) != MA_SUCCESS) {
82446 |         return MA_FALSE;
82447 |     }
82448 |     return ma_dr_wav_init_file__internal_FILE(pWav, pFile, onChunk, pChunkUserData, flags,
> pAllocationCallbacks);
82449 | }
82450 | #endif
82451 | MA_API ma_bool32 ma_dr_wav_init_file_with_metadata(ma_dr_wav* pWav, const char* filename, ma_uint32
> flags, const ma_allocation_callbacks* pAllocationCallbacks)
82452 | {
82453 |     FILE* pFile;
82454 |     if (ma_fopen(&pFile, filename, "rb") != MA_SUCCESS) {
82455 |         return MA_FALSE;
82456 |     }
82457 |     return ma_dr_wav_init_file__internal_FILE(pWav, pFile, NULL, NULL, flags |
> MA_DR_WAV_WITH_METADATA, pAllocationCallbacks);
82458 | }
82459 | #ifndef MA_DR_WAV_NO_WCHAR
82460 | MA_API ma_bool32 ma_dr_wav_init_file_with_metadata_w(ma_dr_wav* pWav, const wchar_t* filename,
> ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks)
82461 | {
82462 |     FILE* pFile;
82463 |     if (ma_wfopen(&pFile, filename, L"rb", pAllocationCallbacks) != MA_SUCCESS) {
82464 |         return MA_FALSE;
82465 |     }
82466 |     return ma_dr_wav_init_file__internal_FILE(pWav, pFile, NULL, NULL, flags |
> MA_DR_WAV_WITH_METADATA, pAllocationCallbacks);
82467 | }
82468 | #endif
82469 | MA_PRIVATE ma_bool32 ma_dr_wav_init_file_write__internal_FILE(ma_dr_wav* pWav, FILE* pFile, const
> ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, ma_bool32 isSequential, const
> ma_allocation_callbacks* pAllocationCallbacks)
82470 | {
82471 |     ma_bool32 result;
82472 |     result = ma_dr_wav_preinit_write(pWav, pFormat, isSequential, ma_dr_wav__on_write_stdio,
> ma_dr_wav__on_seek_stdio, (void*)pFile, pAllocationCallbacks);

```

```

82473 |     if (result != MA_TRUE) {
82474 |         fclose(pFile);
82475 |         return result;
82476 |     }
82477 |     result = ma_dr_wav_init_write__internal(pWav, pFormat, totalSampleCount);
82478 |     if (result != MA_TRUE) {
82479 |         fclose(pFile);
82480 |         return result;
82481 |     }
82482 |     return MA_TRUE;
82483 | }
82484 | MA_PRIVATE ma_bool32 ma_dr_wav_init_file_write__internal(ma_dr_wav* pWav, const char* filename,
> const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, ma_bool32 isSequential, const
> ma_allocation_callbacks* pAllocationCallbacks)
82485 | {
82486 |     FILE* pFile;
82487 |     if (ma_fopen(&pFile, filename, "wb") != MA_SUCCESS) {
82488 |         return MA_FALSE;
82489 |     }
82490 |     return ma_dr_wav_init_file_write__internal_FILE(pWav, pFile, pFormat, totalSampleCount,
> isSequential, pAllocationCallbacks);
82491 | }
82492 | #ifndef MA_DR_WAV_NO_WCHAR
82493 | MA_PRIVATE ma_bool32 ma_dr_wav_init_file_write_w__internal(ma_dr_wav* pWav, const wchar_t* filename,
> const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, ma_bool32 isSequential, const
> ma_allocation_callbacks* pAllocationCallbacks)
82494 | {
82495 |     FILE* pFile;
82496 |     if (ma_wfopen(&pFile, filename, L"wb", pAllocationCallbacks) != MA_SUCCESS) {
82497 |         return MA_FALSE;
82498 |     }
82499 |     return ma_dr_wav_init_file_write__internal_FILE(pWav, pFile, pFormat, totalSampleCount,
> isSequential, pAllocationCallbacks);
82500 | }
82501 | #endif
82502 | MA_API ma_bool32 ma_dr_wav_init_file_write(ma_dr_wav* pWav, const char* filename, const
> ma_dr_wav_data_format* pFormat, const ma_allocation_callbacks* pAllocationCallbacks)
82503 | {
82504 |     return ma_dr_wav_init_file_write__internal(pWav, filename, pFormat, 0, MA_FALSE,
> pAllocationCallbacks);
82505 | }
82506 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential(ma_dr_wav* pWav, const char* filename, const
> ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
82507 | {
82508 |     return ma_dr_wav_init_file_write__internal(pWav, filename, pFormat, totalSampleCount, MA_TRUE,
> pAllocationCallbacks);
82509 | }
82510 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential_pcm_frames(ma_dr_wav* pWav, const char*
> filename, const ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
82511 | {
82512 |     if (pFormat == NULL) {
82513 |         return MA_FALSE;
82514 |     }
82515 |     return ma_dr_wav_init_file_write_sequential(pWav, filename, pFormat,
> totalPCMFrameCount*pFormat->channels, pAllocationCallbacks);
82516 | }
82517 | #ifndef MA_DR_WAV_NO_WCHAR
82518 | MA_API ma_bool32 ma_dr_wav_init_file_write_w(ma_dr_wav* pWav, const wchar_t* filename, const
> ma_dr_wav_data_format* pFormat, const ma_allocation_callbacks* pAllocationCallbacks)
82519 | {
82520 |     return ma_dr_wav_init_file_write_w__internal(pWav, filename, pFormat, 0, MA_FALSE,
> pAllocationCallbacks);
82521 | }
82522 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential_w(ma_dr_wav* pWav, const wchar_t* filename,
> const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
82523 | {
82524 |     return ma_dr_wav_init_file_write_w__internal(pWav, filename, pFormat, totalSampleCount, MA_TRUE,
> pAllocationCallbacks);
82525 | }
82526 | MA_API ma_bool32 ma_dr_wav_init_file_write_sequential_pcm_frames_w(ma_dr_wav* pWav, const wchar_t*
> filename, const ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
82527 | {
82528 |     if (pFormat == NULL) {
82529 |         return MA_FALSE;
82530 |     }

```

```

82531 |     return ma_dr_wav_init_file_write_sequential_w(pWav, filename, pFormat,
> totalPCMFrameCount*pFormat->channels, pAllocationCallbacks);
82532 | }
82533 | #endif
82534 | #endif
82535 | MA_PRIVATE size_t ma_dr_wav__on_read_memory(void* pUserData, void* pBufferOut, size_t bytesToRead)
82536 | {
82537 |     ma_dr_wav* pWav = (ma_dr_wav*)pUserData;
82538 |     size_t bytesRemaining;
82539 |     MA_DR_WAV_ASSERT(pWav != NULL);
82540 |     MA_DR_WAV_ASSERT(pWav->memoryStream.dataSize >= pWav->memoryStream.currentReadPos);
82541 |     bytesRemaining = pWav->memoryStream.dataSize - pWav->memoryStream.currentReadPos;
82542 |     if (bytesToRead > bytesRemaining) {
82543 |         bytesToRead = bytesRemaining;
82544 |     }
82545 |     if (bytesToRead > 0) {
82546 |         MA_DR_WAV_COPY_MEMORY(pBufferOut, pWav->memoryStream.data +
> pWav->memoryStream.currentReadPos, bytesToRead);
82547 |         pWav->memoryStream.currentReadPos += bytesToRead;
82548 |     }
82549 |     return bytesToRead;
82550 | }
82551 | MA_PRIVATE ma_bool32 ma_dr_wav__on_seek_memory(void* pUserData, int offset, ma_dr_wav_seek_origin
> origin)
82552 | {
82553 |     ma_dr_wav* pWav = (ma_dr_wav*)pUserData;
82554 |     ma_int64 newCursor;
82555 |     MA_DR_WAV_ASSERT(pWav != NULL);
82556 |     if (origin == MA_DR_WAV_SEEK_SET) {
82557 |         newCursor = 0;
82558 |     } else if (origin == MA_DR_WAV_SEEK_CUR) {
82559 |         newCursor = (ma_int64)pWav->memoryStream.currentReadPos;
82560 |     } else if (origin == MA_DR_WAV_SEEK_END) {
82561 |         newCursor = (ma_int64)pWav->memoryStream.dataSize;
82562 |     } else {
82563 |         MA_DR_WAV_ASSERT(!"Invalid seek origin");
82564 |         return MA_FALSE;
82565 |     }
82566 |     newCursor += offset;
82567 |     if (newCursor < 0) {
82568 |         return MA_FALSE;
82569 |     }
82570 |     if ((size_t)newCursor > pWav->memoryStream.dataSize) {
82571 |         return MA_FALSE;
82572 |     }
82573 |     pWav->memoryStream.currentReadPos = (size_t)newCursor;
82574 |     return MA_TRUE;
82575 | }
82576 | MA_PRIVATE size_t ma_dr_wav__on_write_memory(void* pUserData, const void* pDataIn, size_t
> bytesToWrite)
82577 | {
82578 |     ma_dr_wav* pWav = (ma_dr_wav*)pUserData;
82579 |     size_t bytesRemaining;
82580 |     MA_DR_WAV_ASSERT(pWav != NULL);
82581 |     MA_DR_WAV_ASSERT(pWav->memoryStreamWrite.dataCapacity >=
> pWav->memoryStreamWrite.currentWritePos);
82582 |     bytesRemaining = pWav->memoryStreamWrite.dataCapacity - pWav->memoryStreamWrite.currentWritePos;
82583 |     if (bytesRemaining < bytesToWrite) {
82584 |         void* pNewData;
82585 |         size_t newDataCapacity = (pWav->memoryStreamWrite.dataCapacity == 0) ? 256 :
> pWav->memoryStreamWrite.dataCapacity * 2;
82586 |         if ((newDataCapacity - pWav->memoryStreamWrite.currentWritePos) < bytesToWrite) {
82587 |             newDataCapacity = pWav->memoryStreamWrite.currentWritePos + bytesToWrite;
82588 |         }
82589 |         pNewData = ma_dr_wav__realloc_from_callbacks(*pWav->memoryStreamWrite.ppData,
> newDataCapacity, pWav->memoryStreamWrite.dataCapacity, &pWav->allocationCallbacks);
82590 |         if (pNewData == NULL) {
82591 |             return 0;
82592 |         }
82593 |         *pWav->memoryStreamWrite.ppData = pNewData;
82594 |         pWav->memoryStreamWrite.dataCapacity = newDataCapacity;
82595 |     }
82596 |     MA_DR_WAV_COPY_MEMORY(((ma_uint8*)(*pWav->memoryStreamWrite.ppData)) +
> pWav->memoryStreamWrite.currentWritePos, pDataIn, bytesToWrite);
82597 |     pWav->memoryStreamWrite.currentWritePos += bytesToWrite;
82598 |     if (pWav->memoryStreamWrite.dataSize < pWav->memoryStreamWrite.currentWritePos) {
82599 |         pWav->memoryStreamWrite.dataSize = pWav->memoryStreamWrite.currentWritePos;
82600 |     }
82601 |     *pWav->memoryStreamWrite.pDataSize = pWav->memoryStreamWrite.dataSize;

```

```

82602 |     return bytesToWrite;
82603 | }
82604 | MA_PRIVATE ma_bool32 ma_dr_wav__on_seek_memory_write(void* pUserData, int offset,
> ma_dr_wav_seek_origin origin)
82605 | {
82606 |     ma_dr_wav* pWav = (ma_dr_wav*)pUserData;
82607 |     ma_int64 newCursor;
82608 |     MA_DR_WAV_ASSERT(pWav != NULL);
82609 |     if (origin == MA_DR_WAV_SEEK_SET) {
82610 |         newCursor = 0;
82611 |     } else if (origin == MA_DR_WAV_SEEK_CUR) {
82612 |         newCursor = (ma_int64)pWav->memoryStreamWrite.currentWritePos;
82613 |     } else if (origin == MA_DR_WAV_SEEK_END) {
82614 |         newCursor = (ma_int64)pWav->memoryStreamWrite.dataSize;
82615 |     } else {
82616 |         MA_DR_WAV_ASSERT(!"Invalid seek origin");
82617 |         return MA_FALSE;
82618 |     }
82619 |     newCursor += offset;
82620 |     if (newCursor < 0) {
82621 |         return MA_FALSE;
82622 |     }
82623 |     if ((size_t)newCursor > pWav->memoryStreamWrite.dataSize) {
82624 |         return MA_FALSE;
82625 |     }
82626 |     pWav->memoryStreamWrite.currentWritePos = (size_t)newCursor;
82627 |     return MA_TRUE;
82628 | }
82629 | MA_PRIVATE ma_bool32 ma_dr_wav__on_tell_memory(void* pUserData, ma_int64* pCursor)
82630 | {
82631 |     ma_dr_wav* pWav = (ma_dr_wav*)pUserData;
82632 |     MA_DR_WAV_ASSERT(pWav != NULL);
82633 |     MA_DR_WAV_ASSERT(pCursor != NULL);
82634 |     *pCursor = (ma_int64)pWav->memoryStream.currentReadPos;
82635 |     return MA_TRUE;
82636 | }
82637 | MA_API ma_bool32 ma_dr_wav_init_memory(ma_dr_wav* pWav, const void* data, size_t dataSize, const
> ma_allocation_callbacks* pAllocationCallbacks)
82638 | {
82639 |     return ma_dr_wav_init_memory_ex(pWav, data, dataSize, NULL, NULL, 0, pAllocationCallbacks);
82640 | }
82641 | MA_API ma_bool32 ma_dr_wav_init_memory_ex(ma_dr_wav* pWav, const void* data, size_t dataSize,
> ma_dr_wav_chunk_proc onChunk, void* pChunkUserData, ma_uint32 flags, const ma_allocation_callbacks*
> pAllocationCallbacks)
82642 | {
82643 |     if (data == NULL || dataSize == 0) {
82644 |         return MA_FALSE;
82645 |     }
82646 |     if (!ma_dr_wav_preinit(pWav, ma_dr_wav__on_read_memory, ma_dr_wav__on_seek_memory,
> ma_dr_wav__on_tell_memory, pWav, pAllocationCallbacks)) {
82647 |         return MA_FALSE;
82648 |     }
82649 |     pWav->memoryStream.data = (const ma_uint8*)data;
82650 |     pWav->memoryStream.dataSize = dataSize;
82651 |     pWav->memoryStream.currentReadPos = 0;
82652 |     return ma_dr_wav_init__internal(pWav, onChunk, pChunkUserData, flags);
82653 | }
82654 | MA_API ma_bool32 ma_dr_wav_init_memory_with_metadata(ma_dr_wav* pWav, const void* data, size_t
> dataSize, ma_uint32 flags, const ma_allocation_callbacks* pAllocationCallbacks)
82655 | {
82656 |     if (data == NULL || dataSize == 0) {
82657 |         return MA_FALSE;
82658 |     }
82659 |     if (!ma_dr_wav_preinit(pWav, ma_dr_wav__on_read_memory, ma_dr_wav__on_seek_memory,
> ma_dr_wav__on_tell_memory, pWav, pAllocationCallbacks)) {
82660 |         return MA_FALSE;
82661 |     }
82662 |     pWav->memoryStream.data = (const ma_uint8*)data;
82663 |     pWav->memoryStream.dataSize = dataSize;
82664 |     pWav->memoryStream.currentReadPos = 0;
82665 |     return ma_dr_wav_init__internal(pWav, NULL, NULL, flags | MA_DR_WAV_WITH_METADATA);
82666 | }
82667 | MA_PRIVATE ma_bool32 ma_dr_wav_init_memory_write__internal(ma_dr_wav* pWav, void** ppData, size_t*
> pDataSize, const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, ma_bool32 isSequential,
> const ma_allocation_callbacks* pAllocationCallbacks)
82668 | {
82669 |     if (ppData == NULL || pDataSize == NULL) {
82670 |         return MA_FALSE;
82671 |     }

```

```

82672 |     *ppData = NULL;
82673 |     *pDataSize = 0;
82674 |     if (!ma_dr_wav_preinit_write(pWav, pFormat, isSequential, ma_dr_wav__on_write_memory,
> ma_dr_wav__on_seek_memory_write, pWav, pAllocationCallbacks)) {
82675 |         return MA_FALSE;
82676 |     }
82677 |     pWav->memoryStreamWrite.ppData = ppData;
82678 |     pWav->memoryStreamWrite.pDataSize = pDataSize;
82679 |     pWav->memoryStreamWrite.dataSize = 0;
82680 |     pWav->memoryStreamWrite.dataCapacity = 0;
82681 |     pWav->memoryStreamWrite.currentWritePos = 0;
82682 |     return ma_dr_wav_init_write__internal(pWav, pFormat, totalSampleCount);
82683 | }
82684 | MA_API ma_bool32 ma_dr_wav_init_memory_write(ma_dr_wav* pWav, void** ppData, size_t* pDataSize,
> const ma_dr_wav_data_format* pFormat, const ma_allocation_callbacks* pAllocationCallbacks)
82685 | {
82686 |     return ma_dr_wav_init_memory_write__internal(pWav, ppData, pDataSize, pFormat, 0, MA_FALSE,
> pAllocationCallbacks);
82687 | }
82688 | MA_API ma_bool32 ma_dr_wav_init_memory_write_sequential(ma_dr_wav* pWav, void** ppData, size_t*
> pDataSize, const ma_dr_wav_data_format* pFormat, ma_uint64 totalSampleCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
82689 | {
82690 |     return ma_dr_wav_init_memory_write__internal(pWav, ppData, pDataSize, pFormat, totalSampleCount,
> MA_TRUE, pAllocationCallbacks);
82691 | }
82692 | MA_API ma_bool32 ma_dr_wav_init_memory_write_sequential_pcm_frames(ma_dr_wav* pWav, void** ppData,
> size_t* pDataSize, const ma_dr_wav_data_format* pFormat, ma_uint64 totalPCMFramCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
82693 | {
82694 |     if (pFormat == NULL) {
82695 |         return MA_FALSE;
82696 |     }
82697 |     return ma_dr_wav_init_memory_write_sequential(pWav, ppData, pDataSize, pFormat,
> totalPCMFramCount*pFormat->channels, pAllocationCallbacks);
82698 | }
82699 | MA_API ma_result ma_dr_wav_uninit(ma_dr_wav* pWav)
82700 | {
82701 |     ma_result result = MA_SUCCESS;
82702 |     if (pWav == NULL) {
82703 |         return MA_INVALID_ARGS;
82704 |     }
82705 |     if (pWav->onWrite != NULL) {
82706 |         ma_uint32 paddingSize = 0;
82707 |         if (pWav->container == ma_dr_wav_container_riff || pWav->container ==
> ma_dr_wav_container_rf64) {
82708 |             paddingSize = ma_dr_wav__chunk_padding_size_riff(pWav->dataChunkDataSize);
82709 |         } else {
82710 |             paddingSize = ma_dr_wav__chunk_padding_size_w64(pWav->dataChunkDataSize);
82711 |         }
82712 |         if (paddingSize > 0) {
82713 |             ma_uint64 paddingData = 0;
82714 |             ma_dr_wav__write(pWav, &paddingData, paddingSize);
82715 |         }
82716 |         if (pWav->onSeek && !pWav->isSequentialWrite) {
82717 |             if (pWav->container == ma_dr_wav_container_riff) {
82718 |                 if (pWav->onSeek(pWav->pUserData, 4, MA_DR_WAV_SEEK_SET)) {
82719 |                     ma_uint32 riffChunkSize =
> ma_dr_wav__riff_chunk_size_riff(pWav->dataChunkDataSize, pWav->pMetadata, pWav->metadataCount);
82720 |                     ma_dr_wav__write_u32ne_to_le(pWav, riffChunkSize);
82721 |                 }
82722 |                 if (pWav->onSeek(pWav->pUserData, (int)pWav->dataChunkDataPos - 4,
> MA_DR_WAV_SEEK_SET)) {
82723 |                     ma_uint32 dataChunkSize =
> ma_dr_wav__data_chunk_size_riff(pWav->dataChunkDataSize);
82724 |                     ma_dr_wav__write_u32ne_to_le(pWav, dataChunkSize);
82725 |                 }
82726 |             } else if (pWav->container == ma_dr_wav_container_w64) {
82727 |                 if (pWav->onSeek(pWav->pUserData, 16, MA_DR_WAV_SEEK_SET)) {
82728 |                     ma_uint64 riffChunkSize =
> ma_dr_wav__riff_chunk_size_w64(pWav->dataChunkDataSize);
82729 |                     ma_dr_wav__write_u64ne_to_le(pWav, riffChunkSize);
82730 |                 }
82731 |                 if (pWav->onSeek(pWav->pUserData, (int)pWav->dataChunkDataPos - 8,
> MA_DR_WAV_SEEK_SET)) {
82732 |                     ma_uint64 dataChunkSize =
> ma_dr_wav__data_chunk_size_w64(pWav->dataChunkDataSize);
82733 |                     ma_dr_wav__write_u64ne_to_le(pWav, dataChunkSize);
82734 |                 }

```

```

82735 |         } else if (pWav->container == ma_dr_wav_container_rf64) {
82736 |             int ds64BodyPos = 12 + 8;
82737 |             if (pWav->onSeek(pWav->pUserData, ds64BodyPos + 0, MA_DR_WAV_SEEK_SET)) {
82738 |                 ma_uint64 riffChunkSize =
> ma_dr_wav__riff_chunk_size_rf64(pWav->dataChunkDataSize, pWav->pMetadata, pWav->metadataCount);
82739 |                 ma_dr_wav__write_u64ne_to_le(pWav, riffChunkSize);
82740 |             }
82741 |             if (pWav->onSeek(pWav->pUserData, ds64BodyPos + 8, MA_DR_WAV_SEEK_SET)) {
82742 |                 ma_uint64 dataChunkSize =
> ma_dr_wav__data_chunk_size_rf64(pWav->dataChunkDataSize);
82743 |                 ma_dr_wav__write_u64ne_to_le(pWav, dataChunkSize);
82744 |             }
82745 |         }
82746 |     }
82747 |     if (pWav->isSequentialWrite) {
82748 |         if (pWav->dataChunkDataSize != pWav->dataChunkDataSizeTargetWrite) {
82749 |             result = MA_INVALID_FILE;
82750 |         }
82751 |     }
82752 | } else {
82753 |     ma_dr_wav_free(pWav->pMetadata, &pWav->allocationCallbacks);
82754 | }
82755 | #ifndef MA_DR_WAV_NO_STDIO
82756 |     if (pWav->onRead == ma_dr_wav__on_read_stdio || pWav->onWrite == ma_dr_wav__on_write_stdio) {
82757 |         fclose((FILE*)pWav->pUserData);
82758 |     }
82759 | #endif
82760 |     return result;
82761 | }
82762 | MA_API size_t ma_dr_wav_read_raw(ma_dr_wav* pWav, size_t bytesToRead, void* pBufferOut)
82763 | {
82764 |     size_t bytesRead;
82765 |     ma_uint32 bytesPerFrame;
82766 |     if (pWav == NULL || bytesToRead == 0) {
82767 |         return 0;
82768 |     }
82769 |     if (bytesToRead > pWav->bytesRemaining) {
82770 |         bytesToRead = (size_t)pWav->bytesRemaining;
82771 |     }
82772 |     if (bytesToRead == 0) {
82773 |         return 0;
82774 |     }
82775 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
82776 |     if (bytesPerFrame == 0) {
82777 |         return 0;
82778 |     }
82779 |     if (pBufferOut != NULL) {
82780 |         bytesRead = pWav->onRead(pWav->pUserData, pBufferOut, bytesToRead);
82781 |     } else {
82782 |         bytesRead = 0;
82783 |         while (bytesRead < bytesToRead) {
82784 |             size_t bytesToSeek = (bytesToRead - bytesRead);
82785 |             if (bytesToSeek > 0x7FFFFFFF) {
82786 |                 bytesToSeek = 0x7FFFFFFF;
82787 |             }
82788 |             if (pWav->onSeek(pWav->pUserData, (int)bytesToSeek, MA_DR_WAV_SEEK_CUR) == MA_FALSE) {
82789 |                 break;
82790 |             }
82791 |             bytesRead += bytesToSeek;
82792 |         }
82793 |         while (bytesRead < bytesToRead) {
82794 |             ma_uint8 buffer[4096];
82795 |             size_t bytesSearched;
82796 |             size_t bytesToSeek = (bytesToRead - bytesRead);
82797 |             if (bytesToSeek > sizeof(buffer)) {
82798 |                 bytesToSeek = sizeof(buffer);
82799 |             }
82800 |             bytesSearched = pWav->onRead(pWav->pUserData, buffer, bytesToSeek);
82801 |             bytesRead += bytesSearched;
82802 |             if (bytesSearched < bytesToSeek) {
82803 |                 break;
82804 |             }
82805 |         }
82806 |     }
82807 |     pWav->readCursorInPCMFrames += bytesRead / bytesPerFrame;
82808 |     pWav->bytesRemaining -= bytesRead;
82809 |     return bytesRead;
82810 | }
82811 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_le(ma_dr_wav* pWav, ma_uint64 framesToRead, void*

```

```

    > pBufferOut)
82812 | {
82813 |     ma_uint32 bytesPerFrame;
82814 |     ma_uint64 bytesToRead;
82815 |     ma_uint64 framesRemainingInFile;
82816 |     if (pWav == NULL || framesToRead == 0) {
82817 |         return 0;
82818 |     }
82819 |     if (ma_dr_wav__is_compressed_format_tag(pWav->translatedFormatTag)) {
82820 |         return 0;
82821 |     }
82822 |     framesRemainingInFile = pWav->totalPCMFrameCount - pWav->readCursorInPCMFrames;
82823 |     if (framesToRead > framesRemainingInFile) {
82824 |         framesToRead = framesRemainingInFile;
82825 |     }
82826 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
82827 |     if (bytesPerFrame == 0) {
82828 |         return 0;
82829 |     }
82830 |     bytesToRead = framesToRead * bytesPerFrame;
82831 |     if (bytesToRead > MA_SIZE_MAX) {
82832 |         bytesToRead = (MA_SIZE_MAX / bytesPerFrame) * bytesPerFrame;
82833 |     }
82834 |     if (bytesToRead == 0) {
82835 |         return 0;
82836 |     }
82837 |     return ma_dr_wav_read_raw(pWav, (size_t)bytesToRead, pBufferOut) / bytesPerFrame;
82838 | }
82839 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_be(ma_dr_wav* pWav, ma_uint64 framesToRead, void*
> pBufferOut)
82840 | {
82841 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_le(pWav, framesToRead, pBufferOut);
82842 |     if (pBufferOut != NULL) {
82843 |         ma_uint32 bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
82844 |         if (bytesPerFrame == 0) {
82845 |             return 0;
82846 |         }
82847 |         ma_dr_wav__bswap_samples(pBufferOut, framesRead*pWav->channels,
> bytesPerFrame/pWav->channels);
82848 |     }
82849 |     return framesRead;
82850 | }
82851 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames(ma_dr_wav* pWav, ma_uint64 framesToRead, void*
> pBufferOut)
82852 | {
82853 |     ma_uint64 framesRead = 0;
82854 |     if (ma_dr_wav_is_container_be(pWav->container)) {
82855 |         if (pWav->container != ma_dr_wav_container_aiff || pWav->aiff.isLE == MA_FALSE) {
82856 |             if (ma_dr_wav__is_little_endian()) {
82857 |                 framesRead = ma_dr_wav_read_pcm_frames_be(pWav, framesToRead, pBufferOut);
82858 |             } else {
82859 |                 framesRead = ma_dr_wav_read_pcm_frames_le(pWav, framesToRead, pBufferOut);
82860 |             }
82861 |             goto post_process;
82862 |         }
82863 |     }
82864 |     if (ma_dr_wav__is_little_endian()) {
82865 |         framesRead = ma_dr_wav_read_pcm_frames_le(pWav, framesToRead, pBufferOut);
82866 |     } else {
82867 |         framesRead = ma_dr_wav_read_pcm_frames_be(pWav, framesToRead, pBufferOut);
82868 |     }
82869 |     post_process:
82870 |     {
82871 |         if (pWav->container == ma_dr_wav_container_aiff && pWav->bitsPerSample == 8 &&
> pWav->aiff.isUnsigned == MA_FALSE) {
82872 |             if (pBufferOut != NULL) {
82873 |                 ma_uint64 iSample;
82874 |                 for (iSample = 0; iSample < framesRead * pWav->channels; iSample += 1) {
82875 |                     ((ma_uint8*)pBufferOut)[iSample] += 128;
82876 |                 }
82877 |             }
82878 |         }
82879 |     }
82880 |     return framesRead;
82881 | }
82882 | MA_PRIVATE ma_bool32 ma_dr_wav_seek_to_first_pcm_frame(ma_dr_wav* pWav)
82883 | {
82884 |     if (pWav->onWrite != NULL) {
82885 |         return MA_FALSE;

```



```

82886 |     }
82887 |     if (!pWav->onSeek(pWav->pUserData, (int)pWav->dataChunkDataPos, MA_DR_WAV_SEEK_SET)) {
82888 |         return MA_FALSE;
82889 |     }
82890 |     if (ma_dr_wav__is_compressed_format_tag(pWav->translatedFormatTag)) {
82891 |         if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM) {
82892 |             MA_DR_WAV_ZERO_OBJECT(&pWav->msadpcm);
82893 |         } else if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_DVI_ADPCM) {
82894 |             MA_DR_WAV_ZERO_OBJECT(&pWav->ima);
82895 |         } else {
82896 |             MA_DR_WAV_ASSERT(MA_FALSE);
82897 |         }
82898 |     }
82899 |     pWav->readCursorInPCMFrames = 0;
82900 |     pWav->bytesRemaining = pWav->dataChunkDataSize;
82901 |     return MA_TRUE;
82902 | }
82903 | MA_API ma_bool32 ma_dr_wav_seek_to_pcm_frame(ma_dr_wav* pWav, ma_uint64 targetFrameIndex)
82904 | {
82905 |     if (pWav == NULL || pWav->onSeek == NULL) {
82906 |         return MA_FALSE;
82907 |     }
82908 |     if (pWav->onWrite != NULL) {
82909 |         return MA_FALSE;
82910 |     }
82911 |     if (pWav->totalPCMFrameCount == 0) {
82912 |         return MA_TRUE;
82913 |     }
82914 |     if (targetFrameIndex > pWav->totalPCMFrameCount) {
82915 |         targetFrameIndex = pWav->totalPCMFrameCount;
82916 |     }
82917 |     if (ma_dr_wav__is_compressed_format_tag(pWav->translatedFormatTag)) {
82918 |         if (targetFrameIndex < pWav->readCursorInPCMFrames) {
82919 |             if (!ma_dr_wav_seek_to_first_pcm_frame(pWav)) {
82920 |                 return MA_FALSE;
82921 |             }
82922 |         }
82923 |         if (targetFrameIndex > pWav->readCursorInPCMFrames) {
82924 |             ma_uint64 offsetInFrames = targetFrameIndex - pWav->readCursorInPCMFrames;
82925 |             ma_int16 devnull[2048];
82926 |             while (offsetInFrames > 0) {
82927 |                 ma_uint64 framesRead = 0;
82928 |                 ma_uint64 framesToRead = offsetInFrames;
82929 |                 if (framesToRead > ma_dr_wav_countof(devnull)/pWav->channels) {
82930 |                     framesToRead = ma_dr_wav_countof(devnull)/pWav->channels;
82931 |                 }
82932 |                 if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM) {
82933 |                     framesRead = ma_dr_wav_read_pcm_frames_s16__msadpcm(pWav, framesToRead,
82934 | > devnull);
82935 |                 } else if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_DVI_ADPCM) {
82936 |                     framesRead = ma_dr_wav_read_pcm_frames_s16__ima(pWav, framesToRead, devnull);
82937 |                 } else {
82938 |                     MA_DR_WAV_ASSERT(MA_FALSE);
82939 |                 }
82940 |                 if (framesRead != framesToRead) {
82941 |                     return MA_FALSE;
82942 |                 }
82943 |                 offsetInFrames -= framesRead;
82944 |             }
82945 |         } else {
82946 |             ma_uint64 totalSizeInBytes;
82947 |             ma_uint64 currentBytePos;
82948 |             ma_uint64 targetBytePos;
82949 |             ma_uint64 offset;
82950 |             ma_uint32 bytesPerFrame;
82951 |             bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
82952 |             if (bytesPerFrame == 0) {
82953 |                 return MA_FALSE;
82954 |             }
82955 |             totalSizeInBytes = pWav->totalPCMFrameCount * bytesPerFrame;
82956 |             currentBytePos = totalSizeInBytes - pWav->bytesRemaining;
82957 |             targetBytePos = targetFrameIndex * bytesPerFrame;
82958 |             if (currentBytePos < targetBytePos) {
82959 |                 offset = (targetBytePos - currentBytePos);
82960 |             } else {
82961 |                 if (!ma_dr_wav_seek_to_first_pcm_frame(pWav)) {
82962 |                     return MA_FALSE;
82963 |                 }

```

```

82964 |         offset = targetBytePos;
82965 |     }
82966 |     while (offset > 0) {
82967 |         int offset32 = ((offset > INT_MAX) ? INT_MAX : (int)offset);
82968 |         if (!pWav->onSeek(pWav->pUserData, offset32, MA_DR_WAV_SEEK_CUR)) {
82969 |             return MA_FALSE;
82970 |         }
82971 |         pWav->readCursorInPCMFrames += offset32 / bytesPerFrame;
82972 |         pWav->bytesRemaining        -= offset32;
82973 |         offset                      -= offset32;
82974 |     }
82975 | }
82976 | return MA_TRUE;
82977 | }
82978 | MA_API ma_result ma_dr_wav_get_cursor_in_pcm_frames(ma_dr_wav* pWav, ma_uint64* pCursor)
82979 | {
82980 |     if (pCursor == NULL) {
82981 |         return MA_INVALID_ARGS;
82982 |     }
82983 |     *pCursor = 0;
82984 |     if (pWav == NULL) {
82985 |         return MA_INVALID_ARGS;
82986 |     }
82987 |     *pCursor = pWav->readCursorInPCMFrames;
82988 |     return MA_SUCCESS;
82989 | }
82990 | MA_API ma_result ma_dr_wav_get_length_in_pcm_frames(ma_dr_wav* pWav, ma_uint64* pLength)
82991 | {
82992 |     if (pLength == NULL) {
82993 |         return MA_INVALID_ARGS;
82994 |     }
82995 |     *pLength = 0;
82996 |     if (pWav == NULL) {
82997 |         return MA_INVALID_ARGS;
82998 |     }
82999 |     *pLength = pWav->totalPCMFrameCount;
83000 |     return MA_SUCCESS;
83001 | }
83002 | MA_API size_t ma_dr_wav_write_raw(ma_dr_wav* pWav, size_t bytesToWrite, const void* pData)
83003 | {
83004 |     size_t bytesWritten;
83005 |     if (pWav == NULL || bytesToWrite == 0 || pData == NULL) {
83006 |         return 0;
83007 |     }
83008 |     bytesWritten = pWav->onWrite(pWav->pUserData, pData, bytesToWrite);
83009 |     pWav->dataChunkDataSize += bytesWritten;
83010 |     return bytesWritten;
83011 | }
83012 | MA_API ma_uint64 ma_dr_wav_write_pcm_frames_le(ma_dr_wav* pWav, ma_uint64 framesToWrite, const void*
> pData)
83013 | {
83014 |     ma_uint64 bytesToWrite;
83015 |     ma_uint64 bytesWritten;
83016 |     const ma_uint8* pRunningData;
83017 |     if (pWav == NULL || framesToWrite == 0 || pData == NULL) {
83018 |         return 0;
83019 |     }
83020 |     bytesToWrite = ((framesToWrite * pWav->channels * pWav->bitsPerSample) / 8);
83021 |     if (bytesToWrite > MA_SIZE_MAX) {
83022 |         return 0;
83023 |     }
83024 |     bytesWritten = 0;
83025 |     pRunningData = (const ma_uint8*)pData;
83026 |     while (bytesToWrite > 0) {
83027 |         size_t bytesJustWritten;
83028 |         ma_uint64 bytesToWriteThisIteration;
83029 |         bytesToWriteThisIteration = bytesToWrite;
83030 |         MA_DR_WAV_ASSERT(bytesToWriteThisIteration <= MA_SIZE_MAX);
83031 |         bytesJustWritten = ma_dr_wav_write_raw(pWav, (size_t)bytesToWriteThisIteration,
> pRunningData);
83032 |         if (bytesJustWritten == 0) {
83033 |             break;
83034 |         }
83035 |         bytesToWrite -= bytesJustWritten;
83036 |         bytesWritten += bytesJustWritten;
83037 |         pRunningData += bytesJustWritten;
83038 |     }
83039 |     return (bytesWritten * 8) / pWav->bitsPerSample / pWav->channels;
83040 | }

```

```

83041 | MA_API ma_uint64 ma_dr_wav_write_pcm_frames_be(ma_dr_wav* pWav, ma_uint64 framesToWrite, const void*
> pData)
83042 | {
83043 |     ma_uint64 bytesToWrite;
83044 |     ma_uint64 bytesWritten;
83045 |     ma_uint32 bytesPerSample;
83046 |     const ma_uint8* pRunningData;
83047 |     if (pWav == NULL || framesToWrite == 0 || pData == NULL) {
83048 |         return 0;
83049 |     }
83050 |     bytesToWrite = ((framesToWrite * pWav->channels * pWav->bitsPerSample) / 8);
83051 |     if (bytesToWrite > MA_SIZE_MAX) {
83052 |         return 0;
83053 |     }
83054 |     bytesWritten = 0;
83055 |     pRunningData = (const ma_uint8*)pData;
83056 |     bytesPerSample = ma_dr_wav_get_bytes_per_pcm_frame(pWav) / pWav->channels;
83057 |     if (bytesPerSample == 0) {
83058 |         return 0;
83059 |     }
83060 |     while (bytesToWrite > 0) {
83061 |         ma_uint8 temp[4096];
83062 |         ma_uint32 sampleCount;
83063 |         size_t bytesJustWritten;
83064 |         ma_uint64 bytesToWriteThisIteration;
83065 |         bytesToWriteThisIteration = bytesToWrite;
83066 |         MA_DR_WAV_ASSERT(bytesToWriteThisIteration <= MA_SIZE_MAX);
83067 |         sampleCount = sizeof(temp)/bytesPerSample;
83068 |         if (bytesToWriteThisIteration > ((ma_uint64)sampleCount)*bytesPerSample) {
83069 |             bytesToWriteThisIteration = ((ma_uint64)sampleCount)*bytesPerSample;
83070 |         }
83071 |         MA_DR_WAV_COPY_MEMORY(temp, pRunningData, (size_t)bytesToWriteThisIteration);
83072 |         ma_dr_wav__bswap_samples(temp, sampleCount, bytesPerSample);
83073 |         bytesJustWritten = ma_dr_wav_write_raw(pWav, (size_t)bytesToWriteThisIteration, temp);
83074 |         if (bytesJustWritten == 0) {
83075 |             break;
83076 |         }
83077 |         bytesToWrite -= bytesJustWritten;
83078 |         bytesWritten += bytesJustWritten;
83079 |         pRunningData += bytesJustWritten;
83080 |     }
83081 |     return (bytesWritten * 8) / pWav->bitsPerSample / pWav->channels;
83082 | }
83083 | MA_API ma_uint64 ma_dr_wav_write_pcm_frames(ma_dr_wav* pWav, ma_uint64 framesToWrite, const void*
> pData)
83084 | {
83085 |     if (ma_dr_wav__is_little_endian()) {
83086 |         return ma_dr_wav_write_pcm_frames_le(pWav, framesToWrite, pData);
83087 |     } else {
83088 |         return ma_dr_wav_write_pcm_frames_be(pWav, framesToWrite, pData);
83089 |     }
83090 | }
83091 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16__msadpcm(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int16* pBufferOut)
83092 | {
83093 |     ma_uint64 totalFramesRead = 0;
83094 |     static const ma_int32 adaptationTable[] = {
83095 |         230, 230, 230, 230, 307, 409, 512, 614,
83096 |         768, 614, 512, 409, 307, 230, 230, 230
83097 |     };
83098 |     static const ma_int32 coeff1Table[] = { 256, 512, 0, 192, 240, 460, 392 };
83099 |     static const ma_int32 coeff2Table[] = { 0, -256, 0, 64, 0, -208, -232 };
83100 |     MA_DR_WAV_ASSERT(pWav != NULL);
83101 |     MA_DR_WAV_ASSERT(framesToRead > 0);
83102 |     while (pWav->readCursorInPCMFrames < pWav->totalPCMFrameCount) {
83103 |         MA_DR_WAV_ASSERT(framesToRead > 0);
83104 |         if (pWav->msadpcm.cachedFrameCount == 0 && pWav->msadpcm.bytesRemainingInBlock == 0) {
83105 |             if (pWav->channels == 1) {
83106 |                 ma_uint8 header[7];
83107 |                 if (pWav->onRead(pWav->pUserData, header, sizeof(header)) != sizeof(header)) {
83108 |                     return totalFramesRead;
83109 |                 }
83110 |                 pWav->msadpcm.bytesRemainingInBlock = pWav->fmt.blockAlign - sizeof(header);
83111 |                 pWav->msadpcm.predictor[0] = header[0];
83112 |                 pWav->msadpcm.delta[0] = ma_dr_wav_bytes_to_s16(header + 1);
83113 |                 pWav->msadpcm.prevFrames[0][1] = (ma_int32)ma_dr_wav_bytes_to_s16(header + 3);
83114 |                 pWav->msadpcm.prevFrames[0][0] = (ma_int32)ma_dr_wav_bytes_to_s16(header + 5);
83115 |                 pWav->msadpcm.cachedFrames[2] = pWav->msadpcm.prevFrames[0][0];
83116 |                 pWav->msadpcm.cachedFrames[3] = pWav->msadpcm.prevFrames[0][1];

```

```

83117 |         pWav->msadpcm.cachedFrameCount = 2;
83118 |         if (pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff1Table) ||
> pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff2Table)) {
83119 |             return totalFramesRead;
83120 |         }
83121 |     } else {
83122 |         ma_uint8 header[14];
83123 |         if (pWav->onRead(pWav->pUserData, header, sizeof(header)) != sizeof(header)) {
83124 |             return totalFramesRead;
83125 |         }
83126 |         pWav->msadpcm.bytesRemainingInBlock = pWav->fmt.blockAlign - sizeof(header);
83127 |         pWav->msadpcm.predictor[0] = header[0];
83128 |         pWav->msadpcm.predictor[1] = header[1];
83129 |         pWav->msadpcm.delta[0] = ma_dr_wav_bytes_to_s16(header + 2);
83130 |         pWav->msadpcm.delta[1] = ma_dr_wav_bytes_to_s16(header + 4);
83131 |         pWav->msadpcm.prevFrames[0][1] = (ma_int32)ma_dr_wav_bytes_to_s16(header + 6);
83132 |         pWav->msadpcm.prevFrames[1][1] = (ma_int32)ma_dr_wav_bytes_to_s16(header + 8);
83133 |         pWav->msadpcm.prevFrames[0][0] = (ma_int32)ma_dr_wav_bytes_to_s16(header + 10);
83134 |         pWav->msadpcm.prevFrames[1][0] = (ma_int32)ma_dr_wav_bytes_to_s16(header + 12);
83135 |         pWav->msadpcm.cachedFrames[0] = pWav->msadpcm.prevFrames[0][0];
83136 |         pWav->msadpcm.cachedFrames[1] = pWav->msadpcm.prevFrames[1][0];
83137 |         pWav->msadpcm.cachedFrames[2] = pWav->msadpcm.prevFrames[0][1];
83138 |         pWav->msadpcm.cachedFrames[3] = pWav->msadpcm.prevFrames[1][1];
83139 |         pWav->msadpcm.cachedFrameCount = 2;
83140 |         if (pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff1Table) ||
> pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff2Table) ||
83141 |         pWav->msadpcm.predictor[1] >= ma_dr_wav_countof(coeff1Table) ||
> pWav->msadpcm.predictor[1] >= ma_dr_wav_countof(coeff2Table)) {
83142 |             return totalFramesRead;
83143 |         }
83144 |     }
83145 | }
83146 | while (framesToRead > 0 && pWav->msadpcm.cachedFrameCount > 0 && pWav->readCursorInPCMFrames
> < pWav->totalPCMFrameCount) {
83147 |     if (pBufferOut != NULL) {
83148 |         ma_uint32 iSample = 0;
83149 |         for (iSample = 0; iSample < pWav->channels; iSample += 1) {
83150 |             pBufferOut[iSample] =
> (ma_int16)pWav->msadpcm.cachedFrames[(ma_dr_wav_countof(pWav->msadpcm.cachedFrames) -
> (pWav->msadpcm.cachedFrameCount*pWav->channels)) + iSample];
83151 |         }
83152 |         pBufferOut += pWav->channels;
83153 |     }
83154 |     framesToRead -= 1;
83155 |     totalFramesRead += 1;
83156 |     pWav->readCursorInPCMFrames += 1;
83157 |     pWav->msadpcm.cachedFrameCount -= 1;
83158 | }
83159 | if (framesToRead == 0) {
83160 |     break;
83161 | }
83162 | if (pWav->msadpcm.cachedFrameCount == 0) {
83163 |     if (pWav->msadpcm.bytesRemainingInBlock == 0) {
83164 |         continue;
83165 |     } else {
83166 |         ma_uint8 nibbles;
83167 |         ma_int32 nibble0;
83168 |         ma_int32 nibble1;
83169 |         if (pWav->onRead(pWav->pUserData, &nibbles, 1) != 1) {
83170 |             return totalFramesRead;
83171 |         }
83172 |         pWav->msadpcm.bytesRemainingInBlock -= 1;
83173 |         nibble0 = ((nibbles & 0xF0) >> 4); if ((nibbles & 0x80)) { nibble0 |= 0xFFFFFFFF0UL;
> }
83174 |         nibble1 = ((nibbles & 0x0F) >> 0); if ((nibbles & 0x08)) { nibble1 |= 0xFFFFFFFF0UL;
> }
83175 |         if (pWav->channels == 1) {
83176 |             ma_int32 newSample0;
83177 |             ma_int32 newSample1;
83178 |             if (pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff1Table) ||
> pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff2Table)) {
83179 |                 return totalFramesRead;
83180 |             }
83181 |             newSample0 = ((pWav->msadpcm.prevFrames[0][1] *
> coeff1Table[pWav->msadpcm.predictor[0]]) + (pWav->msadpcm.prevFrames[0][0] *
> coeff2Table[pWav->msadpcm.predictor[0]])) >> 8;
83182 |             newSample0 += nibble0 * pWav->msadpcm.delta[0];
83183 |             newSample0 = ma_dr_wav_clamp(newSample0, -32768, 32767);
83184 |             pWav->msadpcm.delta[0] =

```

```

> (ma_int32)ma_dr_wav_clamp(((ma_int64)adaptationTable[((nibbles & 0xF0) >> 4)] *
> pWav->msadpcm.delta[0]) >> 8, 16, 0x7FFFFFFF);
83185 |         pWav->msadpcm.prevFrames[0][0] = pWav->msadpcm.prevFrames[0][1];
83186 |         pWav->msadpcm.prevFrames[0][1] = newSample0;
83187 |         newSample1 = ((pWav->msadpcm.prevFrames[0][1] *
> coeff1Table[pWav->msadpcm.predictor[0]]) + (pWav->msadpcm.prevFrames[0][0] *
> coeff2Table[pWav->msadpcm.predictor[0]])) >> 8;
83188 |         newSample1 += nibble1 * pWav->msadpcm.delta[0];
83189 |         newSample1 = ma_dr_wav_clamp(newSample1, -32768, 32767);
83190 |         pWav->msadpcm.delta[0] =
> (ma_int32)ma_dr_wav_clamp(((ma_int64)adaptationTable[((nibbles & 0x0F) >> 0)] *
> pWav->msadpcm.delta[0]) >> 8, 16, 0x7FFFFFFF);
83191 |         pWav->msadpcm.prevFrames[0][0] = pWav->msadpcm.prevFrames[0][1];
83192 |         pWav->msadpcm.prevFrames[0][1] = newSample1;
83193 |         pWav->msadpcm.cachedFrames[2] = newSample0;
83194 |         pWav->msadpcm.cachedFrames[3] = newSample1;
83195 |         pWav->msadpcm.cachedFrameCount = 2;
83196 |     } else {
83197 |         ma_int32 newSample0;
83198 |         ma_int32 newSample1;
83199 |         if (pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff1Table) ||
> pWav->msadpcm.predictor[0] >= ma_dr_wav_countof(coeff2Table)) {
83200 |             return totalFramesRead;
83201 |         }
83202 |         newSample0 = ((pWav->msadpcm.prevFrames[0][1] *
> coeff1Table[pWav->msadpcm.predictor[0]]) + (pWav->msadpcm.prevFrames[0][0] *
> coeff2Table[pWav->msadpcm.predictor[0]])) >> 8;
83203 |         newSample0 += nibble0 * pWav->msadpcm.delta[0];
83204 |         newSample0 = ma_dr_wav_clamp(newSample0, -32768, 32767);
83205 |         pWav->msadpcm.delta[0] =
> (ma_int32)ma_dr_wav_clamp(((ma_int64)adaptationTable[((nibbles & 0xF0) >> 4)] *
> pWav->msadpcm.delta[0]) >> 8, 16, 0x7FFFFFFF);
83206 |         pWav->msadpcm.prevFrames[0][0] = pWav->msadpcm.prevFrames[0][1];
83207 |         pWav->msadpcm.prevFrames[0][1] = newSample0;
83208 |         if (pWav->msadpcm.predictor[1] >= ma_dr_wav_countof(coeff1Table) ||
> pWav->msadpcm.predictor[1] >= ma_dr_wav_countof(coeff2Table)) {
83209 |             return totalFramesRead;
83210 |         }
83211 |         newSample1 = ((pWav->msadpcm.prevFrames[1][1] *
> coeff1Table[pWav->msadpcm.predictor[1]]) + (pWav->msadpcm.prevFrames[1][0] *
> coeff2Table[pWav->msadpcm.predictor[1]])) >> 8;
83212 |         newSample1 += nibble1 * pWav->msadpcm.delta[1];
83213 |         newSample1 = ma_dr_wav_clamp(newSample1, -32768, 32767);
83214 |         pWav->msadpcm.delta[1] =
> (ma_int32)ma_dr_wav_clamp(((ma_int64)adaptationTable[((nibbles & 0x0F) >> 0)] *
> pWav->msadpcm.delta[1]) >> 8, 16, 0x7FFFFFFF);
83215 |         pWav->msadpcm.prevFrames[1][0] = pWav->msadpcm.prevFrames[1][1];
83216 |         pWav->msadpcm.prevFrames[1][1] = newSample1;
83217 |         pWav->msadpcm.cachedFrames[2] = newSample0;
83218 |         pWav->msadpcm.cachedFrames[3] = newSample1;
83219 |         pWav->msadpcm.cachedFrameCount = 1;
83220 |     }
83221 | }
83222 | }
83223 | }
83224 | return totalFramesRead;
83225 | }
83226 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16__ima(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int16* pBufferOut)
83227 | {
83228 |     ma_uint64 totalFramesRead = 0;
83229 |     ma_uint32 iChannel;
83230 |     static const ma_int32 indexTable[16] = {
83231 |         -1, -1, -1, -1, 2, 4, 6, 8,
83232 |         -1, -1, -1, -1, 2, 4, 6, 8
83233 |     };
83234 |     static const ma_int32 stepTable[89] = {
83235 |         7, 8, 9, 10, 11, 12, 13, 14, 16, 17,
83236 |         19, 21, 23, 25, 28, 31, 34, 37, 41, 45,
83237 |         50, 55, 60, 66, 73, 80, 88, 97, 107, 118,
83238 |         130, 143, 157, 173, 190, 209, 230, 253, 279, 307,
83239 |         337, 371, 408, 449, 494, 544, 598, 658, 724, 796,
83240 |         876, 963, 1060, 1166, 1282, 1411, 1552, 1707, 1878, 2066,
83241 |         2272, 2499, 2749, 3024, 3327, 3660, 4026, 4428, 4871, 5358,
83242 |         5894, 6484, 7132, 7845, 8630, 9493, 10442, 11487, 12635, 13899,
83243 |         15289, 16818, 18500, 20350, 22385, 24623, 27086, 29794, 32767
83244 |     };
83245 |     MA_DR_WAV_ASSERT(pWav != NULL);
83246 |     MA_DR_WAV_ASSERT(framesToRead > 0);

```

```

83247 |     while (pWav->readCursorInPCMFrames < pWav->totalPCMFrameCount) {
83248 |         MA_DR_WAV_ASSERT(framesToRead > 0);
83249 |         if (pWav->ima.cachedFrameCount == 0 && pWav->ima.bytesRemainingInBlock == 0) {
83250 |             if (pWav->channels == 1) {
83251 |                 ma_uint8 header[4];
83252 |                 if (pWav->onRead(pWav->pUserData, header, sizeof(header)) != sizeof(header)) {
83253 |                     return totalFramesRead;
83254 |                 }
83255 |                 pWav->ima.bytesRemainingInBlock = pWav->fmt.blockAlign - sizeof(header);
83256 |                 if (header[2] >= ma_dr_wav_countof(stepTable)) {
83257 |                     pWav->onSeek(pWav->pUserData, pWav->ima.bytesRemainingInBlock,
> MA_DR_WAV_SEEK_CUR);
83258 |                     pWav->ima.bytesRemainingInBlock = 0;
83259 |                     return totalFramesRead;
83260 |                 }
83261 |                 pWav->ima.predictor[0] = (ma_int16)ma_dr_wav_bytes_to_u16(header + 0);
83262 |                 pWav->ima.stepIndex[0] = ma_dr_wav_clamp(header[2], 0,
> (ma_int32)ma_dr_wav_countof(stepTable)-1);
83263 |                 pWav->ima.cachedFrames[ma_dr_wav_countof(pWav->ima.cachedFrames) - 1] =
> pWav->ima.predictor[0];
83264 |                 pWav->ima.cachedFrameCount = 1;
83265 |             } else {
83266 |                 ma_uint8 header[8];
83267 |                 if (pWav->onRead(pWav->pUserData, header, sizeof(header)) != sizeof(header)) {
83268 |                     return totalFramesRead;
83269 |                 }
83270 |                 pWav->ima.bytesRemainingInBlock = pWav->fmt.blockAlign - sizeof(header);
83271 |                 if (header[2] >= ma_dr_wav_countof(stepTable) || header[6] >=
> ma_dr_wav_countof(stepTable)) {
83272 |                     pWav->onSeek(pWav->pUserData, pWav->ima.bytesRemainingInBlock,
> MA_DR_WAV_SEEK_CUR);
83273 |                     pWav->ima.bytesRemainingInBlock = 0;
83274 |                     return totalFramesRead;
83275 |                 }
83276 |                 pWav->ima.predictor[0] = ma_dr_wav_bytes_to_s16(header + 0);
83277 |                 pWav->ima.stepIndex[0] = ma_dr_wav_clamp(header[2], 0,
> (ma_int32)ma_dr_wav_countof(stepTable)-1);
83278 |                 pWav->ima.predictor[1] = ma_dr_wav_bytes_to_s16(header + 4);
83279 |                 pWav->ima.stepIndex[1] = ma_dr_wav_clamp(header[6], 0,
> (ma_int32)ma_dr_wav_countof(stepTable)-1);
83280 |                 pWav->ima.cachedFrames[ma_dr_wav_countof(pWav->ima.cachedFrames) - 2] =
> pWav->ima.predictor[0];
83281 |                 pWav->ima.cachedFrames[ma_dr_wav_countof(pWav->ima.cachedFrames) - 1] =
> pWav->ima.predictor[1];
83282 |                 pWav->ima.cachedFrameCount = 1;
83283 |             }
83284 |         }
83285 |         while (framesToRead > 0 && pWav->ima.cachedFrameCount > 0 && pWav->readCursorInPCMFrames <
> pWav->totalPCMFrameCount) {
83286 |             if (pBufferOut != NULL) {
83287 |                 ma_uint32 iSample;
83288 |                 for (iSample = 0; iSample < pWav->channels; iSample += 1) {
83289 |                     pBufferOut[iSample] =
> (ma_int16)pWav->ima.cachedFrames[(ma_dr_wav_countof(pWav->ima.cachedFrames) -
> (pWav->ima.cachedFrameCount*pWav->channels)) + iSample];
83290 |                 }
83291 |                 pBufferOut += pWav->channels;
83292 |             }
83293 |             framesToRead -= 1;
83294 |             totalFramesRead += 1;
83295 |             pWav->readCursorInPCMFrames += 1;
83296 |             pWav->ima.cachedFrameCount -= 1;
83297 |         }
83298 |         if (framesToRead == 0) {
83299 |             break;
83300 |         }
83301 |         if (pWav->ima.cachedFrameCount == 0) {
83302 |             if (pWav->ima.bytesRemainingInBlock == 0) {
83303 |                 continue;
83304 |             } else {
83305 |                 pWav->ima.cachedFrameCount = 8;
83306 |                 for (iChannel = 0; iChannel < pWav->channels; ++iChannel) {
83307 |                     ma_uint32 iByte;
83308 |                     ma_uint8 nibbles[4];
83309 |                     if (pWav->onRead(pWav->pUserData, &nibbles, 4) != 4) {
83310 |                         pWav->ima.cachedFrameCount = 0;
83311 |                         return totalFramesRead;
83312 |                     }
83313 |                     pWav->ima.bytesRemainingInBlock -= 4;

```



```

83314 |         for (iByte = 0; iByte < 4; ++iByte) {
83315 |             ma_uint8 nibble0 = ((nibbles[iByte] & 0x0F) >> 0);
83316 |             ma_uint8 nibble1 = ((nibbles[iByte] & 0xF0) >> 4);
83317 |             ma_int32 step      = stepTable[pWav->ima.stepIndex[iChannel]];
83318 |             ma_int32 predictor = pWav->ima.predictor[iChannel];
83319 |             ma_int32 diff      = step >> 3;
83320 |             if (nibble0 & 1) diff += step >> 2;
83321 |             if (nibble0 & 2) diff += step >> 1;
83322 |             if (nibble0 & 4) diff += step;
83323 |             if (nibble0 & 8) diff = -diff;
83324 |             predictor = ma_dr_wav_clamp(predictor + diff, -32768, 32767);
83325 |             pWav->ima.predictor[iChannel] = predictor;
83326 |             pWav->ima.stepIndex[iChannel] =
> ma_dr_wav_clamp(pWav->ima.stepIndex[iChannel] + indexTable[nibble0], 0,
> (ma_int32)ma_dr_wav_countof(stepTable)-1);
83327 |             pWav->ima.cachedFrames[(ma_dr_wav_countof(pWav->ima.cachedFrames) -
> (pWav->ima.cachedFrameCount*pWav->channels)) + (iByte*2+0)*pWav->channels + iChannel] = predictor;
83328 |             step      = stepTable[pWav->ima.stepIndex[iChannel]];
83329 |             predictor = pWav->ima.predictor[iChannel];
83330 |             diff      = step >> 3;
83331 |             if (nibble1 & 1) diff += step >> 2;
83332 |             if (nibble1 & 2) diff += step >> 1;
83333 |             if (nibble1 & 4) diff += step;
83334 |             if (nibble1 & 8) diff = -diff;
83335 |             predictor = ma_dr_wav_clamp(predictor + diff, -32768, 32767);
83336 |             pWav->ima.predictor[iChannel] = predictor;
83337 |             pWav->ima.stepIndex[iChannel] =
> ma_dr_wav_clamp(pWav->ima.stepIndex[iChannel] + indexTable[nibble1], 0,
> (ma_int32)ma_dr_wav_countof(stepTable)-1);
83338 |             pWav->ima.cachedFrames[(ma_dr_wav_countof(pWav->ima.cachedFrames) -
> (pWav->ima.cachedFrameCount*pWav->channels)) + (iByte*2+1)*pWav->channels + iChannel] = predictor;
83339 |         }
83340 |     }
83341 | }
83342 | }
83343 | }
83344 |     return totalFramesRead;
83345 | }
83346 | #ifndef MA_DR_WAV_NO_CONVERSION_API
83347 | static const unsigned short ma_dr_wav_gAlawTable[256] = {
83348 |     0xEA80, 0xEB80, 0xE880, 0xE980, 0xEE80, 0xEF80, 0xEC80, 0xED80, 0xE280, 0xE380, 0xE080, 0xE180,
> 0xE680, 0xE780, 0xE480, 0xE580,
83349 |     0xF540, 0xF5C0, 0xF440, 0xF4C0, 0xF740, 0xF7C0, 0xF640, 0xF6C0, 0xF140, 0xF1C0, 0xF040, 0xF0C0,
> 0xF340, 0xF3C0, 0xF240, 0xF2C0,
83350 |     0xAA00, 0xAE00, 0xA200, 0xA600, 0xBA00, 0xBE00, 0xB200, 0xB600, 0x8A00, 0x8E00, 0x8200, 0x8600,
> 0x9A00, 0x9E00, 0x9200, 0x9600,
83351 |     0xD500, 0xD700, 0xD100, 0xD300, 0xDD00, 0xDF00, 0xD900, 0xDB00, 0xC500, 0xC700, 0xC100, 0xC300,
> 0xCD00, 0xCF00, 0xC900, 0xCB00,
83352 |     0xFEA8, 0xFEB8, 0xFE88, 0xFE98, 0xFEE8, 0xFEf8, 0xFEC8, 0xFED8, 0xFE28, 0xFE38, 0xFE08, 0xFE18,
> 0xFE68, 0xFE78, 0xFE48, 0xFE58,
83353 |     0xFFA8, 0xFFB8, 0xFF88, 0xFF98, 0xFFE8, 0xFFFF8, 0xFFC8, 0xFFD8, 0xFF28, 0xFF38, 0xFF08, 0xFF18,
> 0xFF68, 0xFF78, 0xFF48, 0xFF58,
83354 |     0xFAA0, 0xFAE0, 0xFA20, 0xFA60, 0xFBA0, 0xFBE0, 0xFB20, 0xFB60, 0xF8A0, 0xF8E0, 0xF820, 0xF860,
> 0xF9A0, 0xF9E0, 0xF920, 0xF960,
83355 |     0xFD50, 0xFD70, 0xFD10, 0xFD30, 0xFDD0, 0xFDF0, 0xFD90, 0xFDB0, 0xFC50, 0xFC70, 0xFC10, 0xFC30,
> 0xFCD0, 0xFCF0, 0xFC90, 0xFCB0,
83356 |     0x1580, 0x1480, 0x1780, 0x1680, 0x1180, 0x1080, 0x1380, 0x1280, 0x1D80, 0x1C80, 0x1F80, 0x1E80,
> 0x1980, 0x1880, 0x1B80, 0x1A80,
83357 |     0x0AC0, 0x0A40, 0x0ABC0, 0x0B40, 0x08C0, 0x0840, 0x09C0, 0x0940, 0x0EC0, 0x0E40, 0x0FC0, 0x0F40,
> 0x0CC0, 0x0C40, 0x0DC0, 0x0D40,
83358 |     0x5600, 0x5200, 0x5E00, 0x5A00, 0x4600, 0x4200, 0x4E00, 0x4A00, 0x7600, 0x7200, 0x7E00, 0x7A00,
> 0x6600, 0x6200, 0x6E00, 0x6A00,
83359 |     0x2B00, 0x2900, 0x2F00, 0x2D00, 0x2300, 0x2100, 0x2700, 0x2500, 0x3B00, 0x3900, 0x3F00, 0x3D00,
> 0x3300, 0x3100, 0x3700, 0x3500,
83360 |     0x0158, 0x0148, 0x0178, 0x0168, 0x0118, 0x0108, 0x0138, 0x0128, 0x01D8, 0x01C8, 0x01F8, 0x01E8,
> 0x0198, 0x0188, 0x01B8, 0x01A8,
83361 |     0x0058, 0x0048, 0x0078, 0x0068, 0x0018, 0x0008, 0x0038, 0x0028, 0x00D8, 0x00C8, 0x00F8, 0x00E8,
> 0x0098, 0x0088, 0x00B8, 0x00A8,
83362 |     0x0560, 0x0520, 0x05E0, 0x05A0, 0x0460, 0x0420, 0x04E0, 0x04A0, 0x0760, 0x0720, 0x07E0, 0x07A0,
> 0x0660, 0x0620, 0x06E0, 0x06A0,
83363 |     0x02B0, 0x0290, 0x02F0, 0x02D0, 0x0230, 0x0210, 0x0270, 0x0250, 0x03B0, 0x0390, 0x03F0, 0x03D0,
> 0x0330, 0x0310, 0x0370, 0x0350
83364 | };
83365 | static const unsigned short ma_dr_wav_gMulawTable[256] = {
83366 |     0x8284, 0x8684, 0x8A84, 0x8E84, 0x9284, 0x9684, 0x9A84, 0x9E84, 0xA284, 0xA684, 0xAA84, 0xAE84,
> 0xB284, 0xB684, 0xBA84, 0xBE84,
83367 |     0xC184, 0xC384, 0xC584, 0xC784, 0xC984, 0xCB84, 0xCD84, 0xCF84, 0xD184, 0xD384, 0xD584, 0xD784,
> 0xD984, 0xDB84, 0xDD84, 0xDF84,
83368 |     0xE104, 0xE204, 0xE304, 0xE404, 0xE504, 0xE604, 0xE704, 0xE804, 0xE904, 0xEA04, 0xEB04, 0xEC04,

```



```

> 0xED04, 0xEE04, 0xEF04, 0xF004,
83369 |     0xF0C4, 0xF144, 0xF1C4, 0xF244, 0xF2C4, 0xF344, 0xF3C4, 0xF444, 0xF4C4, 0xF544, 0xF5C4, 0xF644,
> 0xF6C4, 0xF744, 0xF7C4, 0xF844,
83370 |     0xF8A4, 0xF8E4, 0xF924, 0xF964, 0xF9A4, 0xF9E4, 0xFA24, 0xFA64, 0FAA4, 0FAE4, 0xFB24, 0xFB64,
> 0xFBA4, 0xFBE4, 0xFC24, 0xFC64,
83371 |     0xFC94, 0xFCB4, 0xFCD4, 0xFCF4, 0xFD14, 0xFD34, 0xFD54, 0xFD74, 0xFD94, 0FDB4, 0FDD4, 0FDF4,
> 0xFE14, 0xFE34, 0xFE54, 0xFE74,
83372 |     0xFE8C, 0xFE9C, 0xFEAC, 0FEBC, 0FECC, 0FEDC, 0FEEC, 0FEFC, 0FF0C, 0FF1C, 0FF2C, 0FF3C,
> 0xFF4C, 0xFF5C, 0xFF6C, 0xFF7C,
83373 |     0xFF88, 0xFF90, 0xFF98, 0FFA0, 0FFA8, 0FFB0, 0FFB8, 0FFC0, 0FFC8, 0FFD0, 0FFD8, 0FFE0,
> 0FFE8, 0FFF0, 0FFF8, 0x0000,
83374 |     0x7D7C, 0x797C, 0x757C, 0x717C, 0x6D7C, 0x697C, 0x657C, 0x617C, 0x5D7C, 0x597C, 0x557C, 0x517C,
> 0x4D7C, 0x497C, 0x457C, 0x417C,
83375 |     0x3E7C, 0x3C7C, 0x3A7C, 0x387C, 0x367C, 0x347C, 0x327C, 0x307C, 0x2E7C, 0x2C7C, 0x2A7C, 0x287C,
> 0x267C, 0x247C, 0x227C, 0x207C,
83376 |     0x1EFC, 0x1DFC, 0x1CFC, 0x1BFC, 0x1AFC, 0x19FC, 0x18FC, 0x17FC, 0x16FC, 0x15FC, 0x14FC, 0x13FC,
> 0x12FC, 0x11FC, 0x10FC, 0x0FFC,
83377 |     0x0F3C, 0x0EBC, 0x0E3C, 0x0DBC, 0x0D3C, 0x0CBC, 0x0C3C, 0x0BBC, 0x0B3C, 0x0ABC, 0x0A3C, 0x09BC,
> 0x093C, 0x08BC, 0x083C, 0x07BC,
83378 |     0x075C, 0x071C, 0x06DC, 0x069C, 0x065C, 0x061C, 0x05DC, 0x059C, 0x055C, 0x051C, 0x04DC, 0x049C,
> 0x045C, 0x041C, 0x03DC, 0x039C,
83379 |     0x036C, 0x034C, 0x032C, 0x030C, 0x02EC, 0x02CC, 0x02AC, 0x028C, 0x026C, 0x024C, 0x022C, 0x020C,
> 0x01EC, 0x01CC, 0x01AC, 0x018C,
83380 |     0x0174, 0x0164, 0x0154, 0x0144, 0x0134, 0x0124, 0x0114, 0x0104, 0x00F4, 0x00E4, 0x00D4, 0x00C4,
> 0x00B4, 0x00A4, 0x0094, 0x0084,
83381 |     0x0078, 0x0070, 0x0068, 0x0060, 0x0058, 0x0050, 0x0048, 0x0040, 0x0038, 0x0030, 0x0028, 0x0020,
> 0x0018, 0x0010, 0x0008, 0x0000
83382 | };
83383 | static MA_INLINE ma_int16 ma_dr_wav__alaw_to_s16(ma_uint8 sampleIn)
83384 | {
83385 |     return (short)ma_dr_wav_gAlawTable[sampleIn];
83386 | }
83387 | static MA_INLINE ma_int16 ma_dr_wav__mulaw_to_s16(ma_uint8 sampleIn)
83388 | {
83389 |     return (short)ma_dr_wav_gMulawTable[sampleIn];
83390 | }
83391 | MA_PRIVATE void ma_dr_wav__pcm_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t totalSampleCount,
> unsigned int bytesPerSample)
83392 | {
83393 |     size_t i;
83394 |     if (bytesPerSample == 1) {
83395 |         ma_dr_wav_u8_to_s16(pOut, pIn, totalSampleCount);
83396 |         return;
83397 |     }
83398 |     if (bytesPerSample == 2) {
83399 |         for (i = 0; i < totalSampleCount; ++i) {
83400 |             *pOut++ = ((const ma_int16*)pIn)[i];
83401 |         }
83402 |         return;
83403 |     }
83404 |     if (bytesPerSample == 3) {
83405 |         ma_dr_wav_s24_to_s16(pOut, pIn, totalSampleCount);
83406 |         return;
83407 |     }
83408 |     if (bytesPerSample == 4) {
83409 |         ma_dr_wav_s32_to_s16(pOut, (const ma_int32*)pIn, totalSampleCount);
83410 |         return;
83411 |     }
83412 |     if (bytesPerSample > 8) {
83413 |         MA_DR_WAV_ZERO_MEMORY(pOut, totalSampleCount * sizeof(*pOut));
83414 |         return;
83415 |     }
83416 |     for (i = 0; i < totalSampleCount; ++i) {
83417 |         ma_uint64 sample = 0;
83418 |         unsigned int shift = (8 - bytesPerSample) * 8;
83419 |         unsigned int j;
83420 |         for (j = 0; j < bytesPerSample; j += 1) {
83421 |             MA_DR_WAV_ASSERT(j < 8);
83422 |             sample |= (ma_uint64)(pIn[j]) << shift;
83423 |             shift += 8;
83424 |         }
83425 |         pIn += j;
83426 |         *pOut++ = (ma_int16)((ma_int64)sample >> 48);
83427 |     }
83428 | }
83429 | MA_PRIVATE void ma_dr_wav__ieee_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t totalSampleCount,
> unsigned int bytesPerSample)
83430 | {
83431 |     if (bytesPerSample == 4) {

```

```

83432 |         ma_dr_wav_f32_to_s16(pOut, (const float*)pIn, totalSampleCount);
83433 |         return;
83434 |     } else if (bytesPerSample == 8) {
83435 |         ma_dr_wav_f64_to_s16(pOut, (const double*)pIn, totalSampleCount);
83436 |         return;
83437 |     } else {
83438 |         MA_DR_WAV_ZERO_MEMORY(pOut, totalSampleCount * sizeof(*pOut));
83439 |         return;
83440 |     }
83441 | }
83442 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16__pcm(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int16* pBufferOut)
83443 | {
83444 |     ma_uint64 totalFramesRead;
83445 |     ma_uint8 sampleData[4096] = {0};
83446 |     ma_uint32 bytesPerFrame;
83447 |     ma_uint32 bytesPerSample;
83448 |     ma_uint64 samplesRead;
83449 |     if ((pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_PCM && pWav->bitsPerSample == 16) ||
> pBufferOut == NULL) {
83450 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, pBufferOut);
83451 |     }
83452 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83453 |     if (bytesPerFrame == 0) {
83454 |         return 0;
83455 |     }
83456 |     bytesPerSample = bytesPerFrame / pWav->channels;
83457 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83458 |         return 0;
83459 |     }
83460 |     totalFramesRead = 0;
83461 |     while (framesToRead > 0) {
83462 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83463 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83464 |         if (framesRead == 0) {
83465 |             break;
83466 |         }
83467 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83468 |         samplesRead = framesRead * pWav->channels;
83469 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83470 |             MA_DR_WAV_ASSERT(MA_FALSE);
83471 |             break;
83472 |         }
83473 |         ma_dr_wav__pcm_to_s16(pBufferOut, sampleData, (size_t)samplesRead, bytesPerSample);
83474 |         pBufferOut += samplesRead;
83475 |         framesToRead -= framesRead;
83476 |         totalFramesRead += framesRead;
83477 |     }
83478 |     return totalFramesRead;
83479 | }
83480 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16__ieee(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int16* pBufferOut)
83481 | {
83482 |     ma_uint64 totalFramesRead;
83483 |     ma_uint8 sampleData[4096] = {0};
83484 |     ma_uint32 bytesPerFrame;
83485 |     ma_uint32 bytesPerSample;
83486 |     ma_uint64 samplesRead;
83487 |     if (pBufferOut == NULL) {
83488 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, NULL);
83489 |     }
83490 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83491 |     if (bytesPerFrame == 0) {
83492 |         return 0;
83493 |     }
83494 |     bytesPerSample = bytesPerFrame / pWav->channels;
83495 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83496 |         return 0;
83497 |     }
83498 |     totalFramesRead = 0;
83499 |     while (framesToRead > 0) {
83500 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83501 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83502 |         if (framesRead == 0) {
83503 |             break;

```

```

83504 |     }
83505 |     MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83506 |     samplesRead = framesRead * pWav->channels;
83507 |     if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83508 |         MA_DR_WAV_ASSERT(MA_FALSE);
83509 |         break;
83510 |     }
83511 |     ma_dr_wav_ieee_to_s16(pBufferOut, sampleData, (size_t)samplesRead, bytesPerSample);
83512 |     pBufferOut      += samplesRead;
83513 |     framesToRead    -= framesRead;
83514 |     totalFramesRead += framesRead;
83515 | }
83516 | return totalFramesRead;
83517 | }
83518 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16_alaw(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int16* pBufferOut)
83519 | {
83520 |     ma_uint64 totalFramesRead;
83521 |     ma_uint8 sampleData[4096] = {0};
83522 |     ma_uint32 bytesPerFrame;
83523 |     ma_uint32 bytesPerSample;
83524 |     ma_uint64 samplesRead;
83525 |     if (pBufferOut == NULL) {
83526 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, NULL);
83527 |     }
83528 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83529 |     if (bytesPerFrame == 0) {
83530 |         return 0;
83531 |     }
83532 |     bytesPerSample = bytesPerFrame / pWav->channels;
83533 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83534 |         return 0;
83535 |     }
83536 |     totalFramesRead = 0;
83537 |     while (framesToRead > 0) {
83538 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83539 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83540 |         if (framesRead == 0) {
83541 |             break;
83542 |         }
83543 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83544 |         samplesRead = framesRead * pWav->channels;
83545 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83546 |             MA_DR_WAV_ASSERT(MA_FALSE);
83547 |             break;
83548 |         }
83549 |         ma_dr_wav_alaw_to_s16(pBufferOut, sampleData, (size_t)samplesRead);
83550 | #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
83551 |     {
83552 |         if (pWav->container == ma_dr_wav_container_aiff) {
83553 |             ma_uint64 iSample;
83554 |             for (iSample = 0; iSample < samplesRead; iSample += 1) {
83555 |                 pBufferOut[iSample] = -pBufferOut[iSample];
83556 |             }
83557 |         }
83558 |     }
83559 | #endif
83560 |     pBufferOut      += samplesRead;
83561 |     framesToRead    -= framesRead;
83562 |     totalFramesRead += framesRead;
83563 | }
83564 | return totalFramesRead;
83565 | }
83566 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s16_mulaw(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int16* pBufferOut)
83567 | {
83568 |     ma_uint64 totalFramesRead;
83569 |     ma_uint8 sampleData[4096] = {0};
83570 |     ma_uint32 bytesPerFrame;
83571 |     ma_uint32 bytesPerSample;
83572 |     ma_uint64 samplesRead;
83573 |     if (pBufferOut == NULL) {
83574 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, NULL);
83575 |     }
83576 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83577 |     if (bytesPerFrame == 0) {
83578 |         return 0;

```

```

83579 |     }
83580 |     bytesPerSample = bytesPerFrame / pWav->channels;
83581 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83582 |         return 0;
83583 |     }
83584 |     totalFramesRead = 0;
83585 |     while (framesToRead > 0) {
83586 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83587 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83588 |         if (framesRead == 0) {
83589 |             break;
83590 |         }
83591 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83592 |         samplesRead = framesRead * pWav->channels;
83593 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83594 |             MA_DR_WAV_ASSERT(MA_FALSE);
83595 |             break;
83596 |         }
83597 |         ma_dr_wav_mulaw_to_s16(pBufferOut, sampleData, (size_t)samplesRead);
83598 |         #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
83599 |         {
83600 |             if (pWav->container == ma_dr_wav_container_aiff) {
83601 |                 ma_uint64 iSample;
83602 |                 for (iSample = 0; iSample < samplesRead; iSample += 1) {
83603 |                     pBufferOut[iSample] = -pBufferOut[iSample];
83604 |                 }
83605 |             }
83606 |         }
83607 |         #endif
83608 |         pBufferOut      += samplesRead;
83609 |         framesToRead    -= framesRead;
83610 |         totalFramesRead += framesRead;
83611 |     }
83612 |     return totalFramesRead;
83613 | }
83614 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s16(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int16*
> pBufferOut)
83615 | {
83616 |     if (pWav == NULL || framesToRead == 0) {
83617 |         return 0;
83618 |     }
83619 |     if (pBufferOut == NULL) {
83620 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, NULL);
83621 |     }
83622 |     if (framesToRead * pWav->channels * sizeof(ma_int16) > MA_SIZE_MAX) {
83623 |         framesToRead = MA_SIZE_MAX / sizeof(ma_int16) / pWav->channels;
83624 |     }
83625 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_PCM) {
83626 |         return ma_dr_wav_read_pcm_frames_s16__pcm(pWav, framesToRead, pBufferOut);
83627 |     }
83628 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_IEEE_FLOAT) {
83629 |         return ma_dr_wav_read_pcm_frames_s16__ieee(pWav, framesToRead, pBufferOut);
83630 |     }
83631 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ALAW) {
83632 |         return ma_dr_wav_read_pcm_frames_s16__alaw(pWav, framesToRead, pBufferOut);
83633 |     }
83634 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_MULAW) {
83635 |         return ma_dr_wav_read_pcm_frames_s16__mulaw(pWav, framesToRead, pBufferOut);
83636 |     }
83637 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM) {
83638 |         return ma_dr_wav_read_pcm_frames_s16__msadpcm(pWav, framesToRead, pBufferOut);
83639 |     }
83640 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_DVI_ADPCM) {
83641 |         return ma_dr_wav_read_pcm_frames_s16__ima(pWav, framesToRead, pBufferOut);
83642 |     }
83643 |     return 0;
83644 | }
83645 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s16le(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int16*
> pBufferOut)
83646 | {
83647 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_s16(pWav, framesToRead, pBufferOut);
83648 |     if (pBufferOut != NULL && ma_dr_wav__is_little_endian() == MA_FALSE) {
83649 |         ma_dr_wav__bswap_samples_s16(pBufferOut, framesRead*pWav->channels);
83650 |     }
83651 |     return framesRead;
83652 | }
83653 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s16be(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int16*

```

```

    > pBufferOut)
83654 | {
83655 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_s16(pWav, framesToRead, pBufferOut);
83656 |     if (pBufferOut != NULL && ma_dr_wav__is_little_endian() == MA_TRUE) {
83657 |         ma_dr_wav__bswap_samples_s16(pBufferOut, framesRead*pWav->channels);
83658 |     }
83659 |     return framesRead;
83660 | }
83661 | MA_API void ma_dr_wav_u8_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount)
83662 | {
83663 |     int r;
83664 |     size_t i;
83665 |     for (i = 0; i < sampleCount; ++i) {
83666 |         int x = pIn[i];
83667 |         r = x << 8;
83668 |         r = r - 32768;
83669 |         pOut[i] = (short)r;
83670 |     }
83671 | }
83672 | MA_API void ma_dr_wav_s24_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount)
83673 | {
83674 |     int r;
83675 |     size_t i;
83676 |     for (i = 0; i < sampleCount; ++i) {
83677 |         int x = ((int)((((unsigned int)((const ma_uint8*)pIn)[i*3+0]) << 8) | ((unsigned
> int)((((const ma_uint8*)pIn)[i*3+1]) << 16) | ((unsigned int)((((const ma_uint8*)pIn)[i*3+2])) << 24))
> >> 8;
83678 |         r = x >> 8;
83679 |         pOut[i] = (short)r;
83680 |     }
83681 | }
83682 | MA_API void ma_dr_wav_s32_to_s16(ma_int16* pOut, const ma_int32* pIn, size_t sampleCount)
83683 | {
83684 |     int r;
83685 |     size_t i;
83686 |     for (i = 0; i < sampleCount; ++i) {
83687 |         int x = pIn[i];
83688 |         r = x >> 16;
83689 |         pOut[i] = (short)r;
83690 |     }
83691 | }
83692 | MA_API void ma_dr_wav_f32_to_s16(ma_int16* pOut, const float* pIn, size_t sampleCount)
83693 | {
83694 |     int r;
83695 |     size_t i;
83696 |     for (i = 0; i < sampleCount; ++i) {
83697 |         float x = pIn[i];
83698 |         float c;
83699 |         c = ((x < -1) ? -1 : ((x > 1) ? 1 : x));
83700 |         c = c + 1;
83701 |         r = (int)(c * 32767.5f);
83702 |         r = r - 32768;
83703 |         pOut[i] = (short)r;
83704 |     }
83705 | }
83706 | MA_API void ma_dr_wav_f64_to_s16(ma_int16* pOut, const double* pIn, size_t sampleCount)
83707 | {
83708 |     int r;
83709 |     size_t i;
83710 |     for (i = 0; i < sampleCount; ++i) {
83711 |         double x = pIn[i];
83712 |         double c;
83713 |         c = ((x < -1) ? -1 : ((x > 1) ? 1 : x));
83714 |         c = c + 1;
83715 |         r = (int)(c * 32767.5);
83716 |         r = r - 32768;
83717 |         pOut[i] = (short)r;
83718 |     }
83719 | }
83720 | MA_API void ma_dr_wav_alaw_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount)
83721 | {
83722 |     size_t i;
83723 |     for (i = 0; i < sampleCount; ++i) {
83724 |         pOut[i] = ma_dr_wav__alaw_to_s16(pIn[i]);
83725 |     }
83726 | }
83727 | MA_API void ma_dr_wav_mulaw_to_s16(ma_int16* pOut, const ma_uint8* pIn, size_t sampleCount)
83728 | {
83729 |     size_t i;

```

```

83730 |     for (i = 0; i < sampleCount; ++i) {
83731 |         pOut[i] = ma_dr_wav__mulaw_to_s16(pIn[i]);
83732 |     }
83733 | }
83734 | MA_PRIVATE void ma_dr_wav__pcm_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount, unsigned
> int bytesPerSample)
83735 | {
83736 |     unsigned int i;
83737 |     if (bytesPerSample == 1) {
83738 |         ma_dr_wav_u8_to_f32(pOut, pIn, sampleCount);
83739 |         return;
83740 |     }
83741 |     if (bytesPerSample == 2) {
83742 |         ma_dr_wav_s16_to_f32(pOut, (const ma_int16*)pIn, sampleCount);
83743 |         return;
83744 |     }
83745 |     if (bytesPerSample == 3) {
83746 |         ma_dr_wav_s24_to_f32(pOut, pIn, sampleCount);
83747 |         return;
83748 |     }
83749 |     if (bytesPerSample == 4) {
83750 |         ma_dr_wav_s32_to_f32(pOut, (const ma_int32*)pIn, sampleCount);
83751 |         return;
83752 |     }
83753 |     if (bytesPerSample > 8) {
83754 |         MA_DR_WAV_ZERO_MEMORY(pOut, sampleCount * sizeof(*pOut));
83755 |         return;
83756 |     }
83757 |     for (i = 0; i < sampleCount; ++i) {
83758 |         ma_uint64 sample = 0;
83759 |         unsigned int shift = (8 - bytesPerSample) * 8;
83760 |         unsigned int j;
83761 |         for (j = 0; j < bytesPerSample; j += 1) {
83762 |             MA_DR_WAV_ASSERT(j < 8);
83763 |             sample |= (ma_uint64)(pIn[j]) << shift;
83764 |             shift += 8;
83765 |         }
83766 |         pIn += j;
83767 |         *pOut++ = (float)((ma_int64)sample / 9223372036854775807.0);
83768 |     }
83769 | }
83770 | MA_PRIVATE void ma_dr_wav__ieee_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount,
> unsigned int bytesPerSample)
83771 | {
83772 |     if (bytesPerSample == 4) {
83773 |         unsigned int i;
83774 |         for (i = 0; i < sampleCount; ++i) {
83775 |             *pOut++ = ((const float*)pIn)[i];
83776 |         }
83777 |         return;
83778 |     } else if (bytesPerSample == 8) {
83779 |         ma_dr_wav_f64_to_f32(pOut, (const double*)pIn, sampleCount);
83780 |         return;
83781 |     } else {
83782 |         MA_DR_WAV_ZERO_MEMORY(pOut, sampleCount * sizeof(*pOut));
83783 |         return;
83784 |     }
83785 | }
83786 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_f32_pcm(ma_dr_wav* pWav, ma_uint64 framesToRead,
> float* pBufferOut)
83787 | {
83788 |     ma_uint64 totalFramesRead;
83789 |     ma_uint8 sampleData[4096] = {0};
83790 |     ma_uint32 bytesPerFrame;
83791 |     ma_uint32 bytesPerSample;
83792 |     ma_uint64 samplesRead;
83793 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83794 |     if (bytesPerFrame == 0) {
83795 |         return 0;
83796 |     }
83797 |     bytesPerSample = bytesPerFrame / pWav->channels;
83798 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83799 |         return 0;
83800 |     }
83801 |     totalFramesRead = 0;
83802 |     while (framesToRead > 0) {
83803 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83804 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,

```

```

> sampleData);
83805 |         if (framesRead == 0) {
83806 |             break;
83807 |         }
83808 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83809 |         samplesRead = framesRead * pWav->channels;
83810 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83811 |             MA_DR_WAV_ASSERT(MA_FALSE);
83812 |             break;
83813 |         }
83814 |         ma_dr_wav__pcm_to_f32(pBufferOut, sampleData, (size_t)samplesRead, bytesPerSample);
83815 |         pBufferOut      += samplesRead;
83816 |         framesToRead    -= framesRead;
83817 |         totalFramesRead += framesRead;
83818 |     }
83819 |     return totalFramesRead;
83820 | }
83821 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_f32__msadpcm_ima(ma_dr_wav* pWav, ma_uint64
> framesToRead, float* pBufferOut)
83822 | {
83823 |     ma_uint64 totalFramesRead;
83824 |     ma_int16 samples16[2048];
83825 |     totalFramesRead = 0;
83826 |     while (framesToRead > 0) {
83827 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> ma_dr_wav_countof(samples16)/pWav->channels);
83828 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_s16(pWav, framesToReadThisIteration,
> samples16);
83829 |         if (framesRead == 0) {
83830 |             break;
83831 |         }
83832 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83833 |         ma_dr_wav_s16_to_f32(pBufferOut, samples16, (size_t)(framesRead*pWav->channels));
83834 |         pBufferOut      += framesRead*pWav->channels;
83835 |         framesToRead    -= framesRead;
83836 |         totalFramesRead += framesRead;
83837 |     }
83838 |     return totalFramesRead;
83839 | }
83840 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_f32__ieee(ma_dr_wav* pWav, ma_uint64 framesToRead,
> float* pBufferOut)
83841 | {
83842 |     ma_uint64 totalFramesRead;
83843 |     ma_uint8 sampleData[4096] = {0};
83844 |     ma_uint32 bytesPerFrame;
83845 |     ma_uint32 bytesPerSample;
83846 |     ma_uint64 samplesRead;
83847 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_IEEE_FLOAT && pWav->bitsPerSample == 32) {
83848 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, pBufferOut);
83849 |     }
83850 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83851 |     if (bytesPerFrame == 0) {
83852 |         return 0;
83853 |     }
83854 |     bytesPerSample = bytesPerFrame / pWav->channels;
83855 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83856 |         return 0;
83857 |     }
83858 |     totalFramesRead = 0;
83859 |     while (framesToRead > 0) {
83860 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83861 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83862 |         if (framesRead == 0) {
83863 |             break;
83864 |         }
83865 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83866 |         samplesRead = framesRead * pWav->channels;
83867 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83868 |             MA_DR_WAV_ASSERT(MA_FALSE);
83869 |             break;
83870 |         }
83871 |         ma_dr_wav__ieee_to_f32(pBufferOut, sampleData, (size_t)samplesRead, bytesPerSample);
83872 |         pBufferOut      += samplesRead;
83873 |         framesToRead    -= framesRead;
83874 |         totalFramesRead += framesRead;
83875 |     }
83876 |     return totalFramesRead;

```



```

83877 | }
83878 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_f32_alaw(ma_dr_wav* pWav, ma_uint64 framesToRead,
> float* pBufferOut)
83879 | {
83880 |     ma_uint64 totalFramesRead;
83881 |     ma_uint8 sampleData[4096] = {0};
83882 |     ma_uint32 bytesPerFrame;
83883 |     ma_uint32 bytesPerSample;
83884 |     ma_uint64 samplesRead;
83885 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83886 |     if (bytesPerFrame == 0) {
83887 |         return 0;
83888 |     }
83889 |     bytesPerSample = bytesPerFrame / pWav->channels;
83890 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83891 |         return 0;
83892 |     }
83893 |     totalFramesRead = 0;
83894 |     while (framesToRead > 0) {
83895 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83896 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83897 |         if (framesRead == 0) {
83898 |             break;
83899 |         }
83900 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83901 |         samplesRead = framesRead * pWav->channels;
83902 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83903 |             MA_DR_WAV_ASSERT(MA_FALSE);
83904 |             break;
83905 |         }
83906 |         ma_dr_wav_alaw_to_f32(pBufferOut, sampleData, (size_t)samplesRead);
83907 |         #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
83908 |         {
83909 |             if (pWav->container == ma_dr_wav_container_aiff) {
83910 |                 ma_uint64 iSample;
83911 |                 for (iSample = 0; iSample < samplesRead; iSample += 1) {
83912 |                     pBufferOut[iSample] = -pBufferOut[iSample];
83913 |                 }
83914 |             }
83915 |         }
83916 |         #endif
83917 |         pBufferOut += samplesRead;
83918 |         framesToRead -= framesRead;
83919 |         totalFramesRead += framesRead;
83920 |     }
83921 |     return totalFramesRead;
83922 | }
83923 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_f32_mulaw(ma_dr_wav* pWav, ma_uint64 framesToRead,
> float* pBufferOut)
83924 | {
83925 |     ma_uint64 totalFramesRead;
83926 |     ma_uint8 sampleData[4096] = {0};
83927 |     ma_uint32 bytesPerFrame;
83928 |     ma_uint32 bytesPerSample;
83929 |     ma_uint64 samplesRead;
83930 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
83931 |     if (bytesPerFrame == 0) {
83932 |         return 0;
83933 |     }
83934 |     bytesPerSample = bytesPerFrame / pWav->channels;
83935 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
83936 |         return 0;
83937 |     }
83938 |     totalFramesRead = 0;
83939 |     while (framesToRead > 0) {
83940 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
83941 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
83942 |         if (framesRead == 0) {
83943 |             break;
83944 |         }
83945 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
83946 |         samplesRead = framesRead * pWav->channels;
83947 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
83948 |             MA_DR_WAV_ASSERT(MA_FALSE);
83949 |             break;

```

```

83950 |     }
83951 |     ma_dr_wav_mulaw_to_f32(pBufferOut, sampleData, (size_t)samplesRead);
83952 |     #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
83953 |     {
83954 |         if (pWav->container == ma_dr_wav_container_aiff) {
83955 |             ma_uint64 iSample;
83956 |             for (iSample = 0; iSample < samplesRead; iSample += 1) {
83957 |                 pBufferOut[iSample] = -pBufferOut[iSample];
83958 |             }
83959 |         }
83960 |     }
83961 |     #endif
83962 |     pBufferOut      += samplesRead;
83963 |     framesToRead    -= framesRead;
83964 |     totalFramesRead += framesRead;
83965 | }
83966 | return totalFramesRead;
83967 | }
83968 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_f32(ma_dr_wav* pWav, ma_uint64 framesToRead, float*
> pBufferOut)
83969 | {
83970 |     if (pWav == NULL || framesToRead == 0) {
83971 |         return 0;
83972 |     }
83973 |     if (pBufferOut == NULL) {
83974 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, NULL);
83975 |     }
83976 |     if (framesToRead * pWav->channels * sizeof(float) > MA_SIZE_MAX) {
83977 |         framesToRead = MA_SIZE_MAX / sizeof(float) / pWav->channels;
83978 |     }
83979 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_PCM) {
83980 |         return ma_dr_wav_read_pcm_frames_f32__pcm(pWav, framesToRead, pBufferOut);
83981 |     }
83982 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM || pWav->translatedFormatTag ==
> MA_DR_WAVE_FORMAT_DVI_ADPCM) {
83983 |         return ma_dr_wav_read_pcm_frames_f32__msadpcm_ima(pWav, framesToRead, pBufferOut);
83984 |     }
83985 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_IEEE_FLOAT) {
83986 |         return ma_dr_wav_read_pcm_frames_f32__ieee(pWav, framesToRead, pBufferOut);
83987 |     }
83988 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ALAW) {
83989 |         return ma_dr_wav_read_pcm_frames_f32__alaw(pWav, framesToRead, pBufferOut);
83990 |     }
83991 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_MULAW) {
83992 |         return ma_dr_wav_read_pcm_frames_f32__mulaw(pWav, framesToRead, pBufferOut);
83993 |     }
83994 |     return 0;
83995 | }
83996 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_f32le(ma_dr_wav* pWav, ma_uint64 framesToRead, float*
> pBufferOut)
83997 | {
83998 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_f32(pWav, framesToRead, pBufferOut);
83999 |     if (pBufferOut != NULL && ma_dr_wav__is_little_endian() == MA_FALSE) {
84000 |         ma_dr_wav__bswap_samples_f32(pBufferOut, framesRead*pWav->channels);
84001 |     }
84002 |     return framesRead;
84003 | }
84004 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_f32be(ma_dr_wav* pWav, ma_uint64 framesToRead, float*
> pBufferOut)
84005 | {
84006 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_f32(pWav, framesToRead, pBufferOut);
84007 |     if (pBufferOut != NULL && ma_dr_wav__is_little_endian() == MA_TRUE) {
84008 |         ma_dr_wav__bswap_samples_f32(pBufferOut, framesRead*pWav->channels);
84009 |     }
84010 |     return framesRead;
84011 | }
84012 | MA_API void ma_dr_wav_u8_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount)
84013 | {
84014 |     size_t i;
84015 |     if (pOut == NULL || pIn == NULL) {
84016 |         return;
84017 |     }
84018 |     #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
84019 |     for (i = 0; i < sampleCount; ++i) {
84020 |         *pOut++ = (pIn[i] / 256.0f) * 2 - 1;
84021 |     }
84022 |     #else
84023 |     for (i = 0; i < sampleCount; ++i) {
84024 |         float x = pIn[i];

```

```

84025 |         x = x * 0.00784313725490196078f;
84026 |         x = x - 1;
84027 |         *pOut++ = x;
84028 |     }
84029 | #endif
84030 | }
84031 | MA_API void ma_dr_wav_s16_to_f32(float* pOut, const ma_int16* pIn, size_t sampleCount)
84032 | {
84033 |     size_t i;
84034 |     if (pOut == NULL || pIn == NULL) {
84035 |         return;
84036 |     }
84037 |     for (i = 0; i < sampleCount; ++i) {
84038 |         *pOut++ = pIn[i] * 0.000030517578125f;
84039 |     }
84040 | }
84041 | MA_API void ma_dr_wav_s24_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount)
84042 | {
84043 |     size_t i;
84044 |     if (pOut == NULL || pIn == NULL) {
84045 |         return;
84046 |     }
84047 |     for (i = 0; i < sampleCount; ++i) {
84048 |         double x;
84049 |         ma_uint32 a = ((ma_uint32)(pIn[i*3+0]) << 8);
84050 |         ma_uint32 b = ((ma_uint32)(pIn[i*3+1]) << 16);
84051 |         ma_uint32 c = ((ma_uint32)(pIn[i*3+2]) << 24);
84052 |         x = (double)((ma_int32)(a | b | c) >> 8);
84053 |         *pOut++ = (float)(x * 0.00000011920928955078125);
84054 |     }
84055 | }
84056 | MA_API void ma_dr_wav_s32_to_f32(float* pOut, const ma_int32* pIn, size_t sampleCount)
84057 | {
84058 |     size_t i;
84059 |     if (pOut == NULL || pIn == NULL) {
84060 |         return;
84061 |     }
84062 |     for (i = 0; i < sampleCount; ++i) {
84063 |         *pOut++ = (float)(pIn[i] / 2147483648.0);
84064 |     }
84065 | }
84066 | MA_API void ma_dr_wav_f64_to_f32(float* pOut, const double* pIn, size_t sampleCount)
84067 | {
84068 |     size_t i;
84069 |     if (pOut == NULL || pIn == NULL) {
84070 |         return;
84071 |     }
84072 |     for (i = 0; i < sampleCount; ++i) {
84073 |         *pOut++ = (float)pIn[i];
84074 |     }
84075 | }
84076 | MA_API void ma_dr_wav_alaw_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount)
84077 | {
84078 |     size_t i;
84079 |     if (pOut == NULL || pIn == NULL) {
84080 |         return;
84081 |     }
84082 |     for (i = 0; i < sampleCount; ++i) {
84083 |         *pOut++ = ma_dr_wav__alaw_to_s16(pIn[i]) / 32768.0f;
84084 |     }
84085 | }
84086 | MA_API void ma_dr_wav_mulaw_to_f32(float* pOut, const ma_uint8* pIn, size_t sampleCount)
84087 | {
84088 |     size_t i;
84089 |     if (pOut == NULL || pIn == NULL) {
84090 |         return;
84091 |     }
84092 |     for (i = 0; i < sampleCount; ++i) {
84093 |         *pOut++ = ma_dr_wav__mulaw_to_s16(pIn[i]) / 32768.0f;
84094 |     }
84095 | }
84096 | MA_PRIVATE void ma_dr_wav__pcm_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t totalSampleCount,
> unsigned int bytesPerSample)
84097 | {
84098 |     unsigned int i;
84099 |     if (bytesPerSample == 1) {
84100 |         ma_dr_wav_u8_to_s32(pOut, pIn, totalSampleCount);
84101 |         return;
84102 |     }

```

```

84103 |     if (bytesPerSample == 2) {
84104 |         ma_dr_wav_s16_to_s32(pOut, (const ma_int16*)pIn, totalSampleCount);
84105 |         return;
84106 |     }
84107 |     if (bytesPerSample == 3) {
84108 |         ma_dr_wav_s24_to_s32(pOut, pIn, totalSampleCount);
84109 |         return;
84110 |     }
84111 |     if (bytesPerSample == 4) {
84112 |         for (i = 0; i < totalSampleCount; ++i) {
84113 |             *pOut++ = ((const ma_int32*)pIn)[i];
84114 |         }
84115 |         return;
84116 |     }
84117 |     if (bytesPerSample > 8) {
84118 |         MA_DR_WAV_ZERO_MEMORY(pOut, totalSampleCount * sizeof(*pOut));
84119 |         return;
84120 |     }
84121 |     for (i = 0; i < totalSampleCount; ++i) {
84122 |         ma_uint64 sample = 0;
84123 |         unsigned int shift = (8 - bytesPerSample) * 8;
84124 |         unsigned int j;
84125 |         for (j = 0; j < bytesPerSample; j += 1) {
84126 |             MA_DR_WAV_ASSERT(j < 8);
84127 |             sample |= (ma_uint64)(pIn[j]) << shift;
84128 |             shift += 8;
84129 |         }
84130 |         pIn += j;
84131 |         *pOut++ = (ma_int32)((ma_int64)sample >> 32);
84132 |     }
84133 | }
84134 | MA_PRIVATE void ma_dr_wav_ieee_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t totalSampleCount,
> unsigned int bytesPerSample)
84135 | {
84136 |     if (bytesPerSample == 4) {
84137 |         ma_dr_wav_f32_to_s32(pOut, (const float*)pIn, totalSampleCount);
84138 |         return;
84139 |     } else if (bytesPerSample == 8) {
84140 |         ma_dr_wav_f64_to_s32(pOut, (const double*)pIn, totalSampleCount);
84141 |         return;
84142 |     } else {
84143 |         MA_DR_WAV_ZERO_MEMORY(pOut, totalSampleCount * sizeof(*pOut));
84144 |         return;
84145 |     }
84146 | }
84147 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s32_pcm(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int32* pBufferOut)
84148 | {
84149 |     ma_uint64 totalFramesRead;
84150 |     ma_uint8 sampleData[4096] = {0};
84151 |     ma_uint32 bytesPerFrame;
84152 |     ma_uint32 bytesPerSample;
84153 |     ma_uint64 samplesRead;
84154 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_PCM && pWav->bitsPerSample == 32) {
84155 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, pBufferOut);
84156 |     }
84157 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
84158 |     if (bytesPerFrame == 0) {
84159 |         return 0;
84160 |     }
84161 |     bytesPerSample = bytesPerFrame / pWav->channels;
84162 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
84163 |         return 0;
84164 |     }
84165 |     totalFramesRead = 0;
84166 |     while (framesToRead > 0) {
84167 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
84168 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
84169 |         if (framesRead == 0) {
84170 |             break;
84171 |         }
84172 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
84173 |         samplesRead = framesRead * pWav->channels;
84174 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
84175 |             MA_DR_WAV_ASSERT(MA_FALSE);
84176 |             break;
84177 |         }

```

```

84178 |         ma_dr_wav__pcm_to_s32(pBufferOut, sampleData, (size_t)samplesRead, bytesPerSample);
84179 |         pBufferOut      += samplesRead;
84180 |         framesToRead    -= framesRead;
84181 |         totalFramesRead += framesRead;
84182 |     }
84183 |     return totalFramesRead;
84184 | }
84185 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s32__msadpcm_ima(ma_dr_wav* pWav, ma_uint64
> framesToRead, ma_int32* pBufferOut)
84186 | {
84187 |     ma_uint64 totalFramesRead = 0;
84188 |     ma_int16 samples16[2048];
84189 |     while (framesToRead > 0) {
84190 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> ma_dr_wav_countof(samples16)/pWav->channels);
84191 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_s16(pWav, framesToReadThisIteration,
> samples16);
84192 |         if (framesRead == 0) {
84193 |             break;
84194 |         }
84195 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
84196 |         ma_dr_wav_s16_to_s32(pBufferOut, samples16, (size_t)(framesRead*pWav->channels));
84197 |         pBufferOut      += framesRead*pWav->channels;
84198 |         framesToRead    -= framesRead;
84199 |         totalFramesRead += framesRead;
84200 |     }
84201 |     return totalFramesRead;
84202 | }
84203 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s32__ieee(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int32* pBufferOut)
84204 | {
84205 |     ma_uint64 totalFramesRead;
84206 |     ma_uint8 sampleData[4096] = {0};
84207 |     ma_uint32 bytesPerFrame;
84208 |     ma_uint32 bytesPerSample;
84209 |     ma_uint64 samplesRead;
84210 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
84211 |     if (bytesPerFrame == 0) {
84212 |         return 0;
84213 |     }
84214 |     bytesPerSample = bytesPerFrame / pWav->channels;
84215 |     if (bytesPerSample == 0 || (bytesPerFrame % pWav->channels) != 0) {
84216 |         return 0;
84217 |     }
84218 |     totalFramesRead = 0;
84219 |     while (framesToRead > 0) {
84220 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
84221 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
84222 |         if (framesRead == 0) {
84223 |             break;
84224 |         }
84225 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
84226 |         samplesRead = framesRead * pWav->channels;
84227 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
84228 |             MA_DR_WAV_ASSERT(MA_FALSE);
84229 |             break;
84230 |         }
84231 |         ma_dr_wav__ieee_to_s32(pBufferOut, sampleData, (size_t)samplesRead, bytesPerSample);
84232 |         pBufferOut      += samplesRead;
84233 |         framesToRead    -= framesRead;
84234 |         totalFramesRead += framesRead;
84235 |     }
84236 |     return totalFramesRead;
84237 | }
84238 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s32__alaw(ma_dr_wav* pWav, ma_uint64 framesToRead,
> ma_int32* pBufferOut)
84239 | {
84240 |     ma_uint64 totalFramesRead;
84241 |     ma_uint8 sampleData[4096] = {0};
84242 |     ma_uint32 bytesPerFrame;
84243 |     ma_uint32 bytesPerSample;
84244 |     ma_uint64 samplesRead;
84245 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pWav);
84246 |     if (bytesPerFrame == 0) {
84247 |         return 0;
84248 |     }
84249 |     bytesPerSample = bytesPerFrame / pWav->channels;

```

```

84250 |     if (bytesPerSample == 0 || (bytesPerFrame % pwav->channels) != 0) {
84251 |         return 0;
84252 |     }
84253 |     totalFramesRead = 0;
84254 |     while (framesToRead > 0) {
84255 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
84256 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pWav, framesToReadThisIteration,
> sampleData);
84257 |         if (framesRead == 0) {
84258 |             break;
84259 |         }
84260 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
84261 |         samplesRead = framesRead * pwav->channels;
84262 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
84263 |             MA_DR_WAV_ASSERT(MA_FALSE);
84264 |             break;
84265 |         }
84266 |         ma_dr_wav_alaw_to_s32(pBufferOut, sampleData, (size_t)samplesRead);
84267 |         #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
84268 |         {
84269 |             if (pwav->container == ma_dr_wav_container_aiff) {
84270 |                 ma_uint64 iSample;
84271 |                 for (iSample = 0; iSample < samplesRead; iSample += 1) {
84272 |                     pBufferOut[iSample] = -pBufferOut[iSample];
84273 |                 }
84274 |             }
84275 |         }
84276 |         #endif
84277 |         pBufferOut      += samplesRead;
84278 |         framesToRead    -= framesRead;
84279 |         totalFramesRead += framesRead;
84280 |     }
84281 |     return totalFramesRead;
84282 | }
84283 | MA_PRIVATE ma_uint64 ma_dr_wav_read_pcm_frames_s32__mulaw(ma_dr_wav* pwav, ma_uint64 framesToRead,
> ma_int32* pBufferOut)
84284 | {
84285 |     ma_uint64 totalFramesRead;
84286 |     ma_uint8 sampleData[4096] = {0};
84287 |     ma_uint32 bytesPerFrame;
84288 |     ma_uint32 bytesPerSample;
84289 |     ma_uint64 samplesRead;
84290 |     bytesPerFrame = ma_dr_wav_get_bytes_per_pcm_frame(pwav);
84291 |     if (bytesPerFrame == 0) {
84292 |         return 0;
84293 |     }
84294 |     bytesPerSample = bytesPerFrame / pwav->channels;
84295 |     if (bytesPerSample == 0 || (bytesPerFrame % pwav->channels) != 0) {
84296 |         return 0;
84297 |     }
84298 |     totalFramesRead = 0;
84299 |     while (framesToRead > 0) {
84300 |         ma_uint64 framesToReadThisIteration = ma_dr_wav_min(framesToRead,
> sizeof(sampleData)/bytesPerFrame);
84301 |         ma_uint64 framesRead = ma_dr_wav_read_pcm_frames(pwav, framesToReadThisIteration,
> sampleData);
84302 |         if (framesRead == 0) {
84303 |             break;
84304 |         }
84305 |         MA_DR_WAV_ASSERT(framesRead <= framesToReadThisIteration);
84306 |         samplesRead = framesRead * pwav->channels;
84307 |         if ((samplesRead * bytesPerSample) > sizeof(sampleData)) {
84308 |             MA_DR_WAV_ASSERT(MA_FALSE);
84309 |             break;
84310 |         }
84311 |         ma_dr_wav_mulaw_to_s32(pBufferOut, sampleData, (size_t)samplesRead);
84312 |         #ifdef MA_DR_WAV_LIBSNDFILE_COMPAT
84313 |         {
84314 |             if (pwav->container == ma_dr_wav_container_aiff) {
84315 |                 ma_uint64 iSample;
84316 |                 for (iSample = 0; iSample < samplesRead; iSample += 1) {
84317 |                     pBufferOut[iSample] = -pBufferOut[iSample];
84318 |                 }
84319 |             }
84320 |         }
84321 |         #endif
84322 |         pBufferOut      += samplesRead;
84323 |         framesToRead    -= framesRead;

```

```

84324 |         totalFramesRead += framesRead;
84325 |     }
84326 |     return totalFramesRead;
84327 | }
84328 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s32(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int32*
> pBufferOut)
84329 | {
84330 |     if (pWav == NULL || framesToRead == 0) {
84331 |         return 0;
84332 |     }
84333 |     if (pBufferOut == NULL) {
84334 |         return ma_dr_wav_read_pcm_frames(pWav, framesToRead, NULL);
84335 |     }
84336 |     if (framesToRead * pWav->channels * sizeof(ma_int32) > MA_SIZE_MAX) {
84337 |         framesToRead = MA_SIZE_MAX / sizeof(ma_int32) / pWav->channels;
84338 |     }
84339 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_PCM) {
84340 |         return ma_dr_wav_read_pcm_frames_s32__pcm(pWav, framesToRead, pBufferOut);
84341 |     }
84342 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ADPCM || pWav->translatedFormatTag ==
> MA_DR_WAVE_FORMAT_DVI_ADPCM) {
84343 |         return ma_dr_wav_read_pcm_frames_s32__msadpcm_ima(pWav, framesToRead, pBufferOut);
84344 |     }
84345 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_IEEE_FLOAT) {
84346 |         return ma_dr_wav_read_pcm_frames_s32__ieee(pWav, framesToRead, pBufferOut);
84347 |     }
84348 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_ALAW) {
84349 |         return ma_dr_wav_read_pcm_frames_s32__alaw(pWav, framesToRead, pBufferOut);
84350 |     }
84351 |     if (pWav->translatedFormatTag == MA_DR_WAVE_FORMAT_MULAW) {
84352 |         return ma_dr_wav_read_pcm_frames_s32__mulaw(pWav, framesToRead, pBufferOut);
84353 |     }
84354 |     return 0;
84355 | }
84356 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s32le(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int32*
> pBufferOut)
84357 | {
84358 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_s32(pWav, framesToRead, pBufferOut);
84359 |     if (pBufferOut != NULL && ma_dr_wav__is_little_endian() == MA_FALSE) {
84360 |         ma_dr_wav__bswap_samples_s32(pBufferOut, framesRead*pWav->channels);
84361 |     }
84362 |     return framesRead;
84363 | }
84364 | MA_API ma_uint64 ma_dr_wav_read_pcm_frames_s32be(ma_dr_wav* pWav, ma_uint64 framesToRead, ma_int32*
> pBufferOut)
84365 | {
84366 |     ma_uint64 framesRead = ma_dr_wav_read_pcm_frames_s32(pWav, framesToRead, pBufferOut);
84367 |     if (pBufferOut != NULL && ma_dr_wav__is_little_endian() == MA_TRUE) {
84368 |         ma_dr_wav__bswap_samples_s32(pBufferOut, framesRead*pWav->channels);
84369 |     }
84370 |     return framesRead;
84371 | }
84372 | MA_API void ma_dr_wav_u8_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount)
84373 | {
84374 |     size_t i;
84375 |     if (pOut == NULL || pIn == NULL) {
84376 |         return;
84377 |     }
84378 |     for (i = 0; i < sampleCount; ++i) {
84379 |         *pOut++ = ((int)pIn[i] - 128) << 24;
84380 |     }
84381 | }
84382 | MA_API void ma_dr_wav_s16_to_s32(ma_int32* pOut, const ma_int16* pIn, size_t sampleCount)
84383 | {
84384 |     size_t i;
84385 |     if (pOut == NULL || pIn == NULL) {
84386 |         return;
84387 |     }
84388 |     for (i = 0; i < sampleCount; ++i) {
84389 |         *pOut++ = pIn[i] << 16;
84390 |     }
84391 | }
84392 | MA_API void ma_dr_wav_s24_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount)
84393 | {
84394 |     size_t i;
84395 |     if (pOut == NULL || pIn == NULL) {
84396 |         return;
84397 |     }
84398 |     for (i = 0; i < sampleCount; ++i) {

```



```

84399 |         unsigned int s0 = pIn[i*3 + 0];
84400 |         unsigned int s1 = pIn[i*3 + 1];
84401 |         unsigned int s2 = pIn[i*3 + 2];
84402 |         ma_int32 sample32 = (ma_int32)((s0 << 8) | (s1 << 16) | (s2 << 24));
84403 |         *pOut++ = sample32;
84404 |     }
84405 | }
84406 | MA_API void ma_dr_wav_f32_to_s32(ma_int32* pOut, const float* pIn, size_t sampleCount)
84407 | {
84408 |     size_t i;
84409 |     if (pOut == NULL || pIn == NULL) {
84410 |         return;
84411 |     }
84412 |     for (i = 0; i < sampleCount; ++i) {
84413 |         *pOut++ = (ma_int32)(2147483648.0f * pIn[i]);
84414 |     }
84415 | }
84416 | MA_API void ma_dr_wav_f64_to_s32(ma_int32* pOut, const double* pIn, size_t sampleCount)
84417 | {
84418 |     size_t i;
84419 |     if (pOut == NULL || pIn == NULL) {
84420 |         return;
84421 |     }
84422 |     for (i = 0; i < sampleCount; ++i) {
84423 |         *pOut++ = (ma_int32)(2147483648.0 * pIn[i]);
84424 |     }
84425 | }
84426 | MA_API void ma_dr_wav_alaw_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount)
84427 | {
84428 |     size_t i;
84429 |     if (pOut == NULL || pIn == NULL) {
84430 |         return;
84431 |     }
84432 |     for (i = 0; i < sampleCount; ++i) {
84433 |         *pOut++ = ((ma_int32)ma_dr_wav__alaw_to_s16(pIn[i])) << 16;
84434 |     }
84435 | }
84436 | MA_API void ma_dr_wav_mulaw_to_s32(ma_int32* pOut, const ma_uint8* pIn, size_t sampleCount)
84437 | {
84438 |     size_t i;
84439 |     if (pOut == NULL || pIn == NULL) {
84440 |         return;
84441 |     }
84442 |     for (i = 0; i < sampleCount; ++i) {
84443 |         *pOut++ = ((ma_int32)ma_dr_wav__mulaw_to_s16(pIn[i])) << 16;
84444 |     }
84445 | }
84446 | MA_PRIVATE ma_int16* ma_dr_wav__read_pcm_frames_and_close_s16(ma_dr_wav* pWav, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalFrameCount)
84447 | {
84448 |     ma_uint64 sampleDataSize;
84449 |     ma_int16* pSampleData;
84450 |     ma_uint64 framesRead;
84451 |     MA_DR_WAV_ASSERT(pWav != NULL);
84452 |     if (pWav->channels == 0 || pWav->totalPCMFrameCount > MA_SIZE_MAX / pWav->channels /
> sizeof(ma_int16)) {
84453 |         ma_dr_wav_uninit(pWav);
84454 |         return NULL;
84455 |     }
84456 |     sampleDataSize = pWav->totalPCMFrameCount * pWav->channels * sizeof(ma_int16);
84457 |     if (sampleDataSize > MA_SIZE_MAX) {
84458 |         ma_dr_wav_uninit(pWav);
84459 |         return NULL;
84460 |     }
84461 |     pSampleData = (ma_int16*)ma_dr_wav__malloc_from_callbacks((size_t)sampleDataSize,
> &pWav->allocationCallbacks);
84462 |     if (pSampleData == NULL) {
84463 |         ma_dr_wav_uninit(pWav);
84464 |         return NULL;
84465 |     }
84466 |     framesRead = ma_dr_wav_read_pcm_frames_s16(pWav, (size_t)pWav->totalPCMFrameCount, pSampleData);
84467 |     if (framesRead != pWav->totalPCMFrameCount) {
84468 |         ma_dr_wav__free_from_callbacks(pSampleData, &pWav->allocationCallbacks);
84469 |         ma_dr_wav_uninit(pWav);
84470 |         return NULL;
84471 |     }
84472 |     ma_dr_wav_uninit(pWav);
84473 |     if (sampleRate) {
84474 |         *sampleRate = pWav->sampleRate;

```

```

84475 |     }
84476 |     if (channels) {
84477 |         *channels = pWav->channels;
84478 |     }
84479 |     if (totalFrameCount) {
84480 |         *totalFrameCount = pWav->totalPCMFrameCount;
84481 |     }
84482 |     return pSampleData;
84483 | }
84484 | MA_PRIVATE float* ma_dr_wav__read_pcm_frames_and_close_f32(ma_dr_wav* pWav, unsigned int* channels,
> unsigned int* sampleRate, ma_uint64* totalFrameCount)
84485 | {
84486 |     ma_uint64 sampleDataSize;
84487 |     float* pSampleData;
84488 |     ma_uint64 framesRead;
84489 |     MA_DR_WAV_ASSERT(pWav != NULL);
84490 |     if (pWav->channels == 0 || pWav->totalPCMFrameCount > MA_SIZE_MAX / pWav->channels /
> sizeof(float)) {
84491 |         ma_dr_wav__uninit(pWav);
84492 |         return NULL;
84493 |     }
84494 |     sampleDataSize = pWav->totalPCMFrameCount * pWav->channels * sizeof(float);
84495 |     if (sampleDataSize > MA_SIZE_MAX) {
84496 |         ma_dr_wav__uninit(pWav);
84497 |         return NULL;
84498 |     }
84499 |     pSampleData = (float*)ma_dr_wav__malloc_from_callbacks((size_t)sampleDataSize,
> &pWav->allocationCallbacks);
84500 |     if (pSampleData == NULL) {
84501 |         ma_dr_wav__uninit(pWav);
84502 |         return NULL;
84503 |     }
84504 |     framesRead = ma_dr_wav__read_pcm_frames_f32(pWav, (size_t)pWav->totalPCMFrameCount, pSampleData);
84505 |     if (framesRead != pWav->totalPCMFrameCount) {
84506 |         ma_dr_wav__free_from_callbacks(pSampleData, &pWav->allocationCallbacks);
84507 |         ma_dr_wav__uninit(pWav);
84508 |         return NULL;
84509 |     }
84510 |     ma_dr_wav__uninit(pWav);
84511 |     if (sampleRate) {
84512 |         *sampleRate = pWav->sampleRate;
84513 |     }
84514 |     if (channels) {
84515 |         *channels = pWav->channels;
84516 |     }
84517 |     if (totalFrameCount) {
84518 |         *totalFrameCount = pWav->totalPCMFrameCount;
84519 |     }
84520 |     return pSampleData;
84521 | }
84522 | MA_PRIVATE ma_int32* ma_dr_wav__read_pcm_frames_and_close_s32(ma_dr_wav* pWav, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalFrameCount)
84523 | {
84524 |     ma_uint64 sampleDataSize;
84525 |     ma_int32* pSampleData;
84526 |     ma_uint64 framesRead;
84527 |     MA_DR_WAV_ASSERT(pWav != NULL);
84528 |     if (pWav->channels == 0 || pWav->totalPCMFrameCount > MA_SIZE_MAX / pWav->channels /
> sizeof(ma_int32)) {
84529 |         ma_dr_wav__uninit(pWav);
84530 |         return NULL;
84531 |     }
84532 |     sampleDataSize = pWav->totalPCMFrameCount * pWav->channels * sizeof(ma_int32);
84533 |     if (sampleDataSize > MA_SIZE_MAX) {
84534 |         ma_dr_wav__uninit(pWav);
84535 |         return NULL;
84536 |     }
84537 |     pSampleData = (ma_int32*)ma_dr_wav__malloc_from_callbacks((size_t)sampleDataSize,
> &pWav->allocationCallbacks);
84538 |     if (pSampleData == NULL) {
84539 |         ma_dr_wav__uninit(pWav);
84540 |         return NULL;
84541 |     }
84542 |     framesRead = ma_dr_wav__read_pcm_frames_s32(pWav, (size_t)pWav->totalPCMFrameCount, pSampleData);
84543 |     if (framesRead != pWav->totalPCMFrameCount) {
84544 |         ma_dr_wav__free_from_callbacks(pSampleData, &pWav->allocationCallbacks);
84545 |         ma_dr_wav__uninit(pWav);
84546 |         return NULL;
84547 |     }

```

```

84548 |     ma_dr_wav_uninit(pWav);
84549 |     if (sampleRate) {
84550 |         *sampleRate = pWav->sampleRate;
84551 |     }
84552 |     if (channels) {
84553 |         *channels = pWav->channels;
84554 |     }
84555 |     if (totalFrameCount) {
84556 |         *totalFrameCount = pWav->totalPCMFrameCount;
84557 |     }
84558 |     return pSampleData;
84559 | }
84560 | MA_API ma_int16* ma_dr_wav_open_and_read_pcm_frames_s16(ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, unsigned int* channelsOut,
> unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const ma_allocation_callbacks*
> pAllocationCallbacks)
84561 | {
84562 |     ma_dr_wav wav;
84563 |     if (channelsOut) {
84564 |         *channelsOut = 0;
84565 |     }
84566 |     if (sampleRateOut) {
84567 |         *sampleRateOut = 0;
84568 |     }
84569 |     if (totalFrameCountOut) {
84570 |         *totalFrameCountOut = 0;
84571 |     }
84572 |     if (!ma_dr_wav_init(&wav, onRead, onSeek, onTell, pUserData, pAllocationCallbacks)) {
84573 |         return NULL;
84574 |     }
84575 |     return ma_dr_wav__read_pcm_frames_and_close_s16(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84576 | }
84577 | MA_API float* ma_dr_wav_open_and_read_pcm_frames_f32(ma_dr_wav_read_proc onRead, ma_dr_wav_seek_proc
> onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, unsigned int* channelsOut, unsigned int*
> sampleRateOut, ma_uint64* totalFrameCountOut, const ma_allocation_callbacks* pAllocationCallbacks)
84578 | {
84579 |     ma_dr_wav wav;
84580 |     if (channelsOut) {
84581 |         *channelsOut = 0;
84582 |     }
84583 |     if (sampleRateOut) {
84584 |         *sampleRateOut = 0;
84585 |     }
84586 |     if (totalFrameCountOut) {
84587 |         *totalFrameCountOut = 0;
84588 |     }
84589 |     if (!ma_dr_wav_init(&wav, onRead, onSeek, onTell, pUserData, pAllocationCallbacks)) {
84590 |         return NULL;
84591 |     }
84592 |     return ma_dr_wav__read_pcm_frames_and_close_f32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84593 | }
84594 | MA_API ma_int32* ma_dr_wav_open_and_read_pcm_frames_s32(ma_dr_wav_read_proc onRead,
> ma_dr_wav_seek_proc onSeek, ma_dr_wav_tell_proc onTell, void* pUserData, unsigned int* channelsOut,
> unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const ma_allocation_callbacks*
> pAllocationCallbacks)
84595 | {
84596 |     ma_dr_wav wav;
84597 |     if (channelsOut) {
84598 |         *channelsOut = 0;
84599 |     }
84600 |     if (sampleRateOut) {
84601 |         *sampleRateOut = 0;
84602 |     }
84603 |     if (totalFrameCountOut) {
84604 |         *totalFrameCountOut = 0;
84605 |     }
84606 |     if (!ma_dr_wav_init(&wav, onRead, onSeek, onTell, pUserData, pAllocationCallbacks)) {
84607 |         return NULL;
84608 |     }
84609 |     return ma_dr_wav__read_pcm_frames_and_close_s32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84610 | }
84611 | #ifndef MA_DR_WAV_NO_STDIO
84612 | MA_API ma_int16* ma_dr_wav_open_file_and_read_pcm_frames_s16(const char* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84613 | {

```

```

84614 |     ma_dr_wav wav;
84615 |     if (channelsOut) {
84616 |         *channelsOut = 0;
84617 |     }
84618 |     if (sampleRateOut) {
84619 |         *sampleRateOut = 0;
84620 |     }
84621 |     if (totalFrameCountOut) {
84622 |         *totalFrameCountOut = 0;
84623 |     }
84624 |     if (!ma_dr_wav_init_file(&wav, filename, pAllocationCallbacks)) {
84625 |         return NULL;
84626 |     }
84627 |     return ma_dr_wav__read_pcm_frames_and_close_s16(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84628 | }
84629 | MA_API float* ma_dr_wav_open_file_and_read_pcm_frames_f32(const char* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84630 | {
84631 |     ma_dr_wav wav;
84632 |     if (channelsOut) {
84633 |         *channelsOut = 0;
84634 |     }
84635 |     if (sampleRateOut) {
84636 |         *sampleRateOut = 0;
84637 |     }
84638 |     if (totalFrameCountOut) {
84639 |         *totalFrameCountOut = 0;
84640 |     }
84641 |     if (!ma_dr_wav_init_file(&wav, filename, pAllocationCallbacks)) {
84642 |         return NULL;
84643 |     }
84644 |     return ma_dr_wav__read_pcm_frames_and_close_f32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84645 | }
84646 | MA_API ma_int32* ma_dr_wav_open_file_and_read_pcm_frames_s32(const char* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84647 | {
84648 |     ma_dr_wav wav;
84649 |     if (channelsOut) {
84650 |         *channelsOut = 0;
84651 |     }
84652 |     if (sampleRateOut) {
84653 |         *sampleRateOut = 0;
84654 |     }
84655 |     if (totalFrameCountOut) {
84656 |         *totalFrameCountOut = 0;
84657 |     }
84658 |     if (!ma_dr_wav_init_file(&wav, filename, pAllocationCallbacks)) {
84659 |         return NULL;
84660 |     }
84661 |     return ma_dr_wav__read_pcm_frames_and_close_s32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84662 | }
84663 | #ifndef MA_DR_WAV_NO_WCHAR
84664 | MA_API ma_int16* ma_dr_wav_open_file_and_read_pcm_frames_s16_w(const wchar_t* filename, unsigned
> int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84665 | {
84666 |     ma_dr_wav wav;
84667 |     if (sampleRateOut) {
84668 |         *sampleRateOut = 0;
84669 |     }
84670 |     if (channelsOut) {
84671 |         *channelsOut = 0;
84672 |     }
84673 |     if (totalFrameCountOut) {
84674 |         *totalFrameCountOut = 0;
84675 |     }
84676 |     if (!ma_dr_wav_init_file_w(&wav, filename, pAllocationCallbacks)) {
84677 |         return NULL;
84678 |     }
84679 |     return ma_dr_wav__read_pcm_frames_and_close_s16(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84680 | }
84681 | MA_API float* ma_dr_wav_open_file_and_read_pcm_frames_f32_w(const wchar_t* filename, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const

```

```

> ma_allocation_callbacks* pAllocationCallbacks)
84682 | {
84683 |     ma_dr_wav wav;
84684 |     if (sampleRateOut) {
84685 |         *sampleRateOut = 0;
84686 |     }
84687 |     if (channelsOut) {
84688 |         *channelsOut = 0;
84689 |     }
84690 |     if (totalFrameCountOut) {
84691 |         *totalFrameCountOut = 0;
84692 |     }
84693 |     if (!ma_dr_wav_init_file_w(&wav, filename, pAllocationCallbacks)) {
84694 |         return NULL;
84695 |     }
84696 |     return ma_dr_wav__read_pcm_frames_and_close_f32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84697 | }
84698 | MA_API ma_int32* ma_dr_wav_open_file_and_read_pcm_frames_s32_w(const wchar_t* filename, unsigned
> int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84699 | {
84700 |     ma_dr_wav wav;
84701 |     if (sampleRateOut) {
84702 |         *sampleRateOut = 0;
84703 |     }
84704 |     if (channelsOut) {
84705 |         *channelsOut = 0;
84706 |     }
84707 |     if (totalFrameCountOut) {
84708 |         *totalFrameCountOut = 0;
84709 |     }
84710 |     if (!ma_dr_wav_init_file_w(&wav, filename, pAllocationCallbacks)) {
84711 |         return NULL;
84712 |     }
84713 |     return ma_dr_wav__read_pcm_frames_and_close_s32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84714 | }
84715 | #endif
84716 | #endif
84717 | MA_API ma_int16* ma_dr_wav_open_memory_and_read_pcm_frames_s16(const void* data, size_t dataSize,
> unsigned int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84718 | {
84719 |     ma_dr_wav wav;
84720 |     if (channelsOut) {
84721 |         *channelsOut = 0;
84722 |     }
84723 |     if (sampleRateOut) {
84724 |         *sampleRateOut = 0;
84725 |     }
84726 |     if (totalFrameCountOut) {
84727 |         *totalFrameCountOut = 0;
84728 |     }
84729 |     if (!ma_dr_wav_init_memory(&wav, data, dataSize, pAllocationCallbacks)) {
84730 |         return NULL;
84731 |     }
84732 |     return ma_dr_wav__read_pcm_frames_and_close_s16(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84733 | }
84734 | MA_API float* ma_dr_wav_open_memory_and_read_pcm_frames_f32(const void* data, size_t dataSize,
> unsigned int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84735 | {
84736 |     ma_dr_wav wav;
84737 |     if (channelsOut) {
84738 |         *channelsOut = 0;
84739 |     }
84740 |     if (sampleRateOut) {
84741 |         *sampleRateOut = 0;
84742 |     }
84743 |     if (totalFrameCountOut) {
84744 |         *totalFrameCountOut = 0;
84745 |     }
84746 |     if (!ma_dr_wav_init_memory(&wav, data, dataSize, pAllocationCallbacks)) {
84747 |         return NULL;
84748 |     }
84749 |     return ma_dr_wav__read_pcm_frames_and_close_f32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);

```

```

84750 | }
84751 | MA_API ma_int32* ma_dr_wav_open_memory_and_read_pcm_frames_s32(const void* data, size_t dataSize,
> unsigned int* channelsOut, unsigned int* sampleRateOut, ma_uint64* totalFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
84752 | {
84753 |     ma_dr_wav wav;
84754 |     if (channelsOut) {
84755 |         *channelsOut = 0;
84756 |     }
84757 |     if (sampleRateOut) {
84758 |         *sampleRateOut = 0;
84759 |     }
84760 |     if (totalFrameCountOut) {
84761 |         *totalFrameCountOut = 0;
84762 |     }
84763 |     if (!ma_dr_wav_init_memory(&wav, data, dataSize, pAllocationCallbacks)) {
84764 |         return NULL;
84765 |     }
84766 |     return ma_dr_wav__read_pcm_frames_and_close_s32(&wav, channelsOut, sampleRateOut,
> totalFrameCountOut);
84767 | }
84768 | #endif
84769 | MA_API void ma_dr_wav_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks)
84770 | {
84771 |     if (pAllocationCallbacks != NULL) {
84772 |         ma_dr_wav__free_from_callbacks(p, pAllocationCallbacks);
84773 |     } else {
84774 |         ma_dr_wav__free_default(p, NULL);
84775 |     }
84776 | }
84777 | MA_API ma_uint16 ma_dr_wav_bytes_to_u16(const ma_uint8* data)
84778 | {
84779 |     return ((ma_uint16)data[0] << 0) | ((ma_uint16)data[1] << 8);
84780 | }
84781 | MA_API ma_int16 ma_dr_wav_bytes_to_s16(const ma_uint8* data)
84782 | {
84783 |     return (ma_int16)ma_dr_wav_bytes_to_u16(data);
84784 | }
84785 | MA_API ma_uint32 ma_dr_wav_bytes_to_u32(const ma_uint8* data)
84786 | {
84787 |     return ma_dr_wav_bytes_to_u32_le(data);
84788 | }
84789 | MA_API float ma_dr_wav_bytes_to_f32(const ma_uint8* data)
84790 | {
84791 |     union {
84792 |         ma_uint32 u32;
84793 |         float f32;
84794 |     } value;
84795 |     value.u32 = ma_dr_wav_bytes_to_u32(data);
84796 |     return value.f32;
84797 | }
84798 | MA_API ma_int32 ma_dr_wav_bytes_to_s32(const ma_uint8* data)
84799 | {
84800 |     return (ma_int32)ma_dr_wav_bytes_to_u32(data);
84801 | }
84802 | MA_API ma_uint64 ma_dr_wav_bytes_to_u64(const ma_uint8* data)
84803 | {
84804 |     return
84805 |         ((ma_uint64)data[0] << 0) | ((ma_uint64)data[1] << 8) | ((ma_uint64)data[2] << 16) |
> ((ma_uint64)data[3] << 24) |
84806 |         ((ma_uint64)data[4] << 32) | ((ma_uint64)data[5] << 40) | ((ma_uint64)data[6] << 48) |
> ((ma_uint64)data[7] << 56);
84807 | }
84808 | MA_API ma_int64 ma_dr_wav_bytes_to_s64(const ma_uint8* data)
84809 | {
84810 |     return (ma_int64)ma_dr_wav_bytes_to_u64(data);
84811 | }
84812 | MA_API ma_bool32 ma_dr_wav_guid_equal(const ma_uint8 a[16], const ma_uint8 b[16])
84813 | {
84814 |     int i;
84815 |     for (i = 0; i < 16; i += 1) {
84816 |         if (a[i] != b[i]) {
84817 |             return MA_FALSE;
84818 |         }
84819 |     }
84820 |     return MA_TRUE;
84821 | }
84822 | MA_API ma_bool32 ma_dr_wav_fourcc_equal(const ma_uint8* a, const char* b)
84823 | {

```

```

84824 |     return
84825 |         a[0] == b[0] &&
84826 |         a[1] == b[1] &&
84827 |         a[2] == b[2] &&
84828 |         a[3] == b[3];
84829 | }
84830 | #ifdef __MRC__
84831 | #pragma options opt reset
84832 | #endif
84833 | #endif
84834 | /* dr_wav_c end */
84835 | #endif /* MA_DR_WAV_IMPLEMENTATION */
84836 | #endif /* MA_NO_WAV */
84837 |
84838 | #if !defined(MA_NO_FLAC) && !defined(MA_NO_DECODING)
84839 | #if !defined(MA_DR_FLAC_IMPLEMENTATION)
84840 | /* dr_flac_c begin */
84841 | #ifndef ma_dr_flac_c
84842 | #define ma_dr_flac_c
84843 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
> 6)))
84844 |     #pragma GCC diagnostic push
84845 |     #if __GNUC__ >= 7
84846 |     #pragma GCC diagnostic ignored "-Wimplicit-fallthrough"
84847 |     #endif
84848 | #endif
84849 | #ifdef __linux__
84850 |     #ifndef _BSD_SOURCE
84851 |     #define _BSD_SOURCE
84852 |     #endif
84853 |     #ifndef _DEFAULT_SOURCE
84854 |     #define _DEFAULT_SOURCE
84855 |     #endif
84856 |     #ifndef __USE_BSD
84857 |     #define __USE_BSD
84858 |     #endif
84859 |     #include <endian.h>
84860 | #endif
84861 | #include <stdlib.h>
84862 | #include <string.h>
84863 | #if !defined(MA_DR_FLAC_NO_SIMD)
84864 |     #if defined(MA_X64) || defined(MA_X86)
84865 |         #if defined(_MSC_VER) && !defined(__clang__)
84866 |             #if _MSC_VER >= 1400 && !defined(MA_DR_FLAC_NO_SSE2)
84867 |             #define MA_DR_FLAC_SUPPORT_SSE2
84868 |             #endif
84869 |             #if _MSC_VER >= 1600 && !defined(MA_DR_FLAC_NO_SSE41)
84870 |             #define MA_DR_FLAC_SUPPORT_SSE41
84871 |             #endif
84872 |         #elif defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 &&
> __GNUC_MINOR__ >= 3)))
84873 |             #if defined(__SSE2__) && !defined(MA_DR_FLAC_NO_SSE2)
84874 |             #define MA_DR_FLAC_SUPPORT_SSE2
84875 |             #endif
84876 |             #if defined(__SSE4_1__) && !defined(MA_DR_FLAC_NO_SSE41)
84877 |             #define MA_DR_FLAC_SUPPORT_SSE41
84878 |             #endif
84879 |         #endif
84880 |         #if !defined(__GNUC__) && !defined(__clang__) && defined(__has_include)
84881 |         #if !defined(MA_DR_FLAC_SUPPORT_SSE2) && !defined(MA_DR_FLAC_NO_SSE2) &&
> __has_include(<emmintrin.h>)
84882 |         #define MA_DR_FLAC_SUPPORT_SSE2
84883 |         #endif
84884 |         #if !defined(MA_DR_FLAC_SUPPORT_SSE41) && !defined(MA_DR_FLAC_NO_SSE41) &&
> __has_include(<smmintrin.h>)
84885 |         #define MA_DR_FLAC_SUPPORT_SSE41
84886 |         #endif
84887 |         #endif
84888 |         #if defined(MA_DR_FLAC_SUPPORT_SSE41)
84889 |         #include <smmintrin.h>
84890 |         #elif defined(MA_DR_FLAC_SUPPORT_SSE2)
84891 |         #include <emmintrin.h>
84892 |         #endif
84893 |     #endif
84894 |     #if defined(MA_ARM)
84895 |     #if !defined(MA_DR_FLAC_NO_NEON) && (defined(__ARM_NEON) || defined(__aarch64__) ||
> defined(_M_ARM64))
84896 |     #define MA_DR_FLAC_SUPPORT_NEON
84897 |     #include <arm_neon.h>

```



```

84898 |         #endif
84899 |     #endif
84900 | #endif
84901 | #if !defined(MA_DR_FLAC_NO_SIMD) && (defined(MA_X86) || defined(MA_X64))
84902 |     #if defined(_MSC_VER) && !defined(__clang__)
84903 |         #if _MSC_VER >= 1400
84904 |             #include <intrin.h>
84905 |             static void ma_dr_flac_cpuid(int info[4], int fid)
84906 |             {
84907 |                 __cpuid(info, fid);
84908 |             }
84909 |         #else
84910 |             #define MA_DR_FLAC_NO_CPUID
84911 |         #endif
84912 |     #else
84913 |         #if defined(__GNUC__) || defined(__clang__)
84914 |             static void ma_dr_flac_cpuid(int info[4], int fid)
84915 |             {
84916 |                 #if defined(MA_X86) && defined(__PIC__)
84917 |                     __asm__ __volatile__ (
84918 |                         "xchg{l} {%%}ebx, %k1;"
84919 |                         "cpuid;"
84920 |                         "xchg{l} {%%}ebx, %k1;"
84921 |                         : "a"(info[0]), "=r"(info[1]), "=c"(info[2]), "=d"(info[3]) : "a"(fid),
> "c"(0)
84922 |                     );
84923 |                 #else
84924 |                     __asm__ __volatile__ (
84925 |                         "cpuid" : "a"(info[0]), "=b"(info[1]), "=c"(info[2]), "=d"(info[3]) :
> "a"(fid), "c"(0)
84926 |                     );
84927 |                 #endif
84928 |             }
84929 |         #else
84930 |             #define MA_DR_FLAC_NO_CPUID
84931 |         #endif
84932 |     #endif
84933 | #else
84934 |     #define MA_DR_FLAC_NO_CPUID
84935 | #endif
84936 | static MA_INLINE ma_bool32 ma_dr_flac_has_sse2(void)
84937 | {
84938 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
84939 |         #if (defined(MA_X64) || defined(MA_X86)) && !defined(MA_DR_FLAC_NO_SSE2)
84940 |             #if defined(MA_X64)
84941 |                 return MA_TRUE;
84942 |             #elif (defined(_M_IX86_FP) && _M_IX86_FP == 2) || defined(__SSE2__)
84943 |                 return MA_TRUE;
84944 |             #else
84945 |                 #if defined(MA_DR_FLAC_NO_CPUID)
84946 |                     return MA_FALSE;
84947 |                 #else
84948 |                     int info[4];
84949 |                     ma_dr_flac_cpuid(info, 1);
84950 |                     return (info[3] & (1 << 26)) != 0;
84951 |                 #endif
84952 |             #endif
84953 |         #else
84954 |             return MA_FALSE;
84955 |         #endif
84956 |     #else
84957 |         return MA_FALSE;
84958 |     #endif
84959 | }
84960 | static MA_INLINE ma_bool32 ma_dr_flac_has_sse41(void)
84961 | {
84962 |     #if defined(MA_DR_FLAC_SUPPORT_SSE41)
84963 |         #if (defined(MA_X64) || defined(MA_X86)) && !defined(MA_DR_FLAC_NO_SSE41)
84964 |             #if defined(__SSE4_1__) || defined(__AVX__)
84965 |                 return MA_TRUE;
84966 |             #else
84967 |                 #if defined(MA_DR_FLAC_NO_CPUID)
84968 |                     return MA_FALSE;
84969 |                 #else
84970 |                     int info[4];
84971 |                     ma_dr_flac_cpuid(info, 1);
84972 |                     return (info[2] & (1 << 19)) != 0;
84973 |                 #endif
84974 |             #endif
84975 |         #endif
84976 |     #endif

```

```

84975 |     #else
84976 |         return MA_FALSE;
84977 |     #endif
84978 | #else
84979 |     return MA_FALSE;
84980 | #endif
84981 | }
84982 | #if defined(_MSC_VER) && _MSC_VER >= 1500 && (defined(MA_X86) || defined(MA_X64)) &&
> !defined(__clang__)
84983 |     #define MA_DR_FLAC_HAS_LZCNT_INTRINSIC
84984 | #elif (defined(__GNUC__) && ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 7)))
84985 |     #define MA_DR_FLAC_HAS_LZCNT_INTRINSIC
84986 | #elif defined(__clang__)
84987 |     #if defined(__has_builtin)
84988 |         #if __has_builtin(__builtin_clzll) || __has_builtin(__builtin_clzl)
84989 |             #define MA_DR_FLAC_HAS_LZCNT_INTRINSIC
84990 |         #endif
84991 |     #endif
84992 | #endif
84993 | #if defined(_MSC_VER) && _MSC_VER >= 1400 && !defined(__clang__)
84994 |     #define MA_DR_FLAC_HAS_BYTESWAP16_INTRINSIC
84995 |     #define MA_DR_FLAC_HAS_BYTESWAP32_INTRINSIC
84996 |     #define MA_DR_FLAC_HAS_BYTESWAP64_INTRINSIC
84997 | #elif defined(__clang__)
84998 |     #if defined(__has_builtin)
84999 |         #if __has_builtin(__builtin_bswap16)
85000 |             #define MA_DR_FLAC_HAS_BYTESWAP16_INTRINSIC
85001 |         #endif
85002 |         #if __has_builtin(__builtin_bswap32)
85003 |             #define MA_DR_FLAC_HAS_BYTESWAP32_INTRINSIC
85004 |         #endif
85005 |         #if __has_builtin(__builtin_bswap64)
85006 |             #define MA_DR_FLAC_HAS_BYTESWAP64_INTRINSIC
85007 |         #endif
85008 |     #endif
85009 | #elif defined(__GNUC__)
85010 |     #if ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 3))
85011 |         #define MA_DR_FLAC_HAS_BYTESWAP32_INTRINSIC
85012 |         #define MA_DR_FLAC_HAS_BYTESWAP64_INTRINSIC
85013 |     #endif
85014 |     #if ((__GNUC__ > 4) || (__GNUC__ == 4 && __GNUC_MINOR__ >= 8))
85015 |         #define MA_DR_FLAC_HAS_BYTESWAP16_INTRINSIC
85016 |     #endif
85017 | #elif defined(__WATCOMC__) && defined(__386__)
85018 |     #define MA_DR_FLAC_HAS_BYTESWAP16_INTRINSIC
85019 |     #define MA_DR_FLAC_HAS_BYTESWAP32_INTRINSIC
85020 |     #define MA_DR_FLAC_HAS_BYTESWAP64_INTRINSIC
85021 |     extern __inline ma_uint16 _watcom_bswap16(ma_uint16);
85022 |     extern __inline ma_uint32 _watcom_bswap32(ma_uint32);
85023 |     extern __inline ma_uint64 _watcom_bswap64(ma_uint64);
85024 | #pragma aux _watcom_bswap16 = \
85025 |     "xchg al, ah" \
85026 |     parm [ax] \
85027 |     value [ax] \
85028 |     modify nomemory;
85029 | #pragma aux _watcom_bswap32 = \
85030 |     "bswap eax" \
85031 |     parm [eax] \
85032 |     value [eax] \
85033 |     modify nomemory;
85034 | #pragma aux _watcom_bswap64 = \
85035 |     "bswap eax" \
85036 |     "bswap edx" \
85037 |     "xchg eax,edx" \
85038 |     parm [eax edx] \
85039 |     value [eax edx] \
85040 |     modify nomemory;
85041 | #endif
85042 | #ifndef MA_DR_FLAC_ASSERT
85043 | #include <assert.h>
85044 | #define MA_DR_FLAC_ASSERT(expression)          assert(expression)
85045 | #endif
85046 | #ifndef MA_DR_FLAC_MALLOC
85047 | #define MA_DR_FLAC_MALLOC(sz)                  malloc((sz))
85048 | #endif
85049 | #ifndef MA_DR_FLAC_REALLOC
85050 | #define MA_DR_FLAC_REALLOC(p, sz)              realloc((p), (sz))
85051 | #endif
85052 | #ifndef MA_DR_FLAC_FREE

```

```

85053 | #define MA_DR_FLAC_FREE(p)                free((p))
85054 | #endif
85055 | #ifndef MA_DR_FLAC_COPY_MEMORY
85056 | #define MA_DR_FLAC_COPY_MEMORY(dst, src, sz)  memcpy((dst), (src), (sz))
85057 | #endif
85058 | #ifndef MA_DR_FLAC_ZERO_MEMORY
85059 | #define MA_DR_FLAC_ZERO_MEMORY(p, sz)        memset((p), 0, (sz))
85060 | #endif
85061 | #ifndef MA_DR_FLAC_ZERO_OBJECT
85062 | #define MA_DR_FLAC_ZERO_OBJECT(p)            MA_DR_FLAC_ZERO_MEMORY((p), sizeof(*(p)))
85063 | #endif
85064 | #define MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE      64
85065 | #define MA_DR_FLAC_SUBFRAME_CONSTANT        0
85066 | #define MA_DR_FLAC_SUBFRAME_VERBATIM        1
85067 | #define MA_DR_FLAC_SUBFRAME_FIXED           8
85068 | #define MA_DR_FLAC_SUBFRAME_LPC             32
85069 | #define MA_DR_FLAC_SUBFRAME_RESERVED        255
85070 | #define MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE 0
85071 | #define MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE2 1
85072 | #define MA_DR_FLAC_CHANNEL_ASSIGNMENT_INDEPENDENT 0
85073 | #define MA_DR_FLAC_CHANNEL_ASSIGNMENT_LEFT_SIDE 8
85074 | #define MA_DR_FLAC_CHANNEL_ASSIGNMENT_RIGHT_SIDE 9
85075 | #define MA_DR_FLAC_CHANNEL_ASSIGNMENT_MID_SIDE 10
85076 | #define MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES 18
85077 | #define MA_DR_FLAC_CUESHEET_TRACK_SIZE_IN_BYTES 36
85078 | #define MA_DR_FLAC_CUESHEET_TRACK_INDEX_SIZE_IN_BYTES 12
85079 | #define ma_dr_flac_align(x, a)               (((x) + (a) - 1) / (a)) * (a))
85080 | MA_API void ma_dr_flac_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision)
85081 | {
85082 |     if (pMajor) {
85083 |         *pMajor = MA_DR_FLAC_VERSION_MAJOR;
85084 |     }
85085 |     if (pMinor) {
85086 |         *pMinor = MA_DR_FLAC_VERSION_MINOR;
85087 |     }
85088 |     if (pRevision) {
85089 |         *pRevision = MA_DR_FLAC_VERSION_REVISION;
85090 |     }
85091 | }
85092 | MA_API const char* ma_dr_flac_version_string(void)
85093 | {
85094 |     return MA_DR_FLAC_VERSION_STRING;
85095 | }
85096 | #if defined(__has_feature)
85097 |     #if __has_feature(thread_sanitizer)
85098 |         #define MA_DR_FLAC_NO_THREAD_SANITIZE __attribute__((no_sanitize("thread")))
85099 |     #else
85100 |         #define MA_DR_FLAC_NO_THREAD_SANITIZE
85101 |     #endif
85102 | #else
85103 |     #define MA_DR_FLAC_NO_THREAD_SANITIZE
85104 | #endif
85105 | #if defined(MA_DR_FLAC_HAS_LZCNT_INTRINSIC)
85106 | static ma_bool32 ma_dr_flac_gIsLZCNTSupported = MA_FALSE;
85107 | #endif
85108 | #ifndef MA_DR_FLAC_NO_CPUID
85109 | static ma_bool32 ma_dr_flac_gIsSSE2Supported = MA_FALSE;
85110 | static ma_bool32 ma_dr_flac_gIsSSE41Supported = MA_FALSE;
85111 | MA_DR_FLAC_NO_THREAD_SANITIZE static void ma_dr_flac__init_cpu_caps(void)
85112 | {
85113 |     static ma_bool32 isCPUCapsInitialized = MA_FALSE;
85114 |     if (!isCPUCapsInitialized) {
85115 |         #if defined(MA_DR_FLAC_HAS_LZCNT_INTRINSIC)
85116 |             int info[4] = {0};
85117 |             ma_dr_flac_cpuid(info, 0x80000001);
85118 |             ma_dr_flac_gIsLZCNTSupported = (info[2] & (1 << 5)) != 0;
85119 |         #endif
85120 |             ma_dr_flac_gIsSSE2Supported = ma_dr_flac_has_sse2();
85121 |             ma_dr_flac_gIsSSE41Supported = ma_dr_flac_has_sse41();
85122 |             isCPUCapsInitialized = MA_TRUE;
85123 |         }
85124 |     }
85125 | #else
85126 | static ma_bool32 ma_dr_flac_gIsNEONSupported = MA_FALSE;
85127 | static MA_INLINE ma_bool32 ma_dr_flac__has_neon(void)
85128 | {
85129 |     #if defined(MA_DR_FLAC_SUPPORT_NEON)
85130 |         #if defined(MA_ARM) && !defined(MA_DR_FLAC_NO_NEON)
85131 |             #if (defined(__ARM_NEON) || defined(__aarch64__) || defined(_M_ARM64))

```

```

85132 |         return MA_TRUE;
85133 |     #else
85134 |         return MA_FALSE;
85135 |     #endif
85136 | #else
85137 |     return MA_FALSE;
85138 | #endif
85139 | #else
85140 |     return MA_FALSE;
85141 | #endif
85142 | }
85143 | MA_DR_FLAC_NO_THREAD_SANITIZE static void ma_dr_flac__init_cpu_caps(void)
85144 | {
85145 |     ma_dr_flac__gIsNEONSupported = ma_dr_flac__has_neon();
85146 |     #if defined(MA_DR_FLAC_HAS_LZCNT_INTRINSIC) && defined(MA_ARM) && (defined(__ARM_ARCH) && __ARM_ARCH
>     >= 5)
85147 |         ma_dr_flac__gIsLZCNTSupported = MA_TRUE;
85148 |     #endif
85149 | }
85150 | #endif
85151 | static MA_INLINE ma_bool32 ma_dr_flac__is_little_endian(void)
85152 | {
85153 |     #if defined(MA_X86) || defined(MA_X64)
85154 |         return MA_TRUE;
85155 |     #elif defined(__BYTE_ORDER) && defined(__LITTLE_ENDIAN) && __BYTE_ORDER == __LITTLE_ENDIAN
85156 |         return MA_TRUE;
85157 |     #else
85158 |         int n = 1;
85159 |         return (*(char*)&n) == 1;
85160 |     #endif
85161 | }
85162 | static MA_INLINE ma_uint16 ma_dr_flac__swap_endian_uint16(ma_uint16 n)
85163 | {
85164 |     #ifdef MA_DR_FLAC_HAS_BYTESWAP16_INTRINSIC
85165 |         #if defined(_MSC_VER) && !defined(__clang__)
85166 |             return _byteswap_ushort(n);
85167 |         #elif defined(__GNUC__) || defined(__clang__)
85168 |             return __builtin_bswap16(n);
85169 |         #elif defined(__WATCOMC__) && defined(__386__)
85170 |             return _watcom_bswap16(n);
85171 |         #else
85172 |             #error "This compiler does not support the byte swap intrinsic."
85173 |         #endif
85174 |     #else
85175 |         return ((n & 0xFF00) >> 8) |
85176 |             ((n & 0x00FF) << 8);
85177 |     #endif
85178 | }
85179 | static MA_INLINE ma_uint32 ma_dr_flac__swap_endian_uint32(ma_uint32 n)
85180 | {
85181 |     #ifdef MA_DR_FLAC_HAS_BYTESWAP32_INTRINSIC
85182 |         #if defined(_MSC_VER) && !defined(__clang__)
85183 |             return _byteswap_ulong(n);
85184 |         #elif defined(__GNUC__) || defined(__clang__)
85185 |             #if defined(MA_ARM) && (defined(__ARM_ARCH) && __ARM_ARCH >= 6) && !defined(__ARM_ARCH_6M__)
>             && !defined(MA_64BIT)
85186 |                 ma_uint32 r;
85187 |                 __asm__ __volatile__ (
85188 |                     #if defined(MA_64BIT)
85189 |                         "rev %w[out], %w[in]" : [out]"=r"(r) : [in]"r"(n)
85190 |                     #else
85191 |                         "rev %[out], %[in]" : [out]"=r"(r) : [in]"r"(n)
85192 |                     #endif
85193 |                 );
85194 |                 return r;
85195 |             #else
85196 |                 return __builtin_bswap32(n);
85197 |             #endif
85198 |         #elif defined(__WATCOMC__) && defined(__386__)
85199 |             return _watcom_bswap32(n);
85200 |         #else
85201 |             #error "This compiler does not support the byte swap intrinsic."
85202 |         #endif
85203 |     #else
85204 |         return ((n & 0xFF000000) >> 24) |
85205 |             ((n & 0x00FF0000) >> 8) |
85206 |             ((n & 0x0000FF00) << 8) |
85207 |             ((n & 0x000000FF) << 24);
85208 |     #endif

```

```

85209 | }
85210 | static MA_INLINE ma_uint64 ma_dr_flac__swap_endian_uint64(ma_uint64 n)
85211 | {
85212 |     #ifdef MA_DR_FLAC_HAS_BYTESWAP64_INTRINSIC
85213 |         #if defined(_MSC_VER) && !defined(__clang__)
85214 |             return _byteswap_uint64(n);
85215 |         #elif defined(__GNUC__) || defined(__clang__)
85216 |             return __builtin_bswap64(n);
85217 |         #elif defined(__WATCOMC__) && defined(__386__)
85218 |             return _watcom_bswap64(n);
85219 |         #else
85220 |             #error "This compiler does not support the byte swap intrinsic."
85221 |         #endif
85222 |     #else
85223 |         return ((n & ((ma_uint64)0xFF000000 << 32)) >> 56) |
85224 |             ((n & ((ma_uint64)0x00FF0000 << 32)) >> 40) |
85225 |             ((n & ((ma_uint64)0x0000FF00 << 32)) >> 24) |
85226 |             ((n & ((ma_uint64)0x000000FF << 32)) >> 8) |
85227 |             ((n & ((ma_uint64)0xFF000000)) << 8) |
85228 |             ((n & ((ma_uint64)0x00FF0000)) << 24) |
85229 |             ((n & ((ma_uint64)0x0000FF00)) << 40) |
85230 |             ((n & ((ma_uint64)0x000000FF)) << 56);
85231 |     #endif
85232 | }
85233 | static MA_INLINE ma_uint16 ma_dr_flac__be2host_16(ma_uint16 n)
85234 | {
85235 |     if (ma_dr_flac__is_little_endian()) {
85236 |         return ma_dr_flac__swap_endian_uint16(n);
85237 |     }
85238 |     return n;
85239 | }
85240 | static MA_INLINE ma_uint32 ma_dr_flac__be2host_32(ma_uint32 n)
85241 | {
85242 |     if (ma_dr_flac__is_little_endian()) {
85243 |         return ma_dr_flac__swap_endian_uint32(n);
85244 |     }
85245 |     return n;
85246 | }
85247 | static MA_INLINE ma_uint32 ma_dr_flac__be2host_32_ptr_unaligned(const void* pData)
85248 | {
85249 |     const ma_uint8* pNum = (ma_uint8*)pData;
85250 |     return *(pNum) << 24 | *(pNum+1) << 16 | *(pNum+2) << 8 | *(pNum+3);
85251 | }
85252 | static MA_INLINE ma_uint64 ma_dr_flac__be2host_64(ma_uint64 n)
85253 | {
85254 |     if (ma_dr_flac__is_little_endian()) {
85255 |         return ma_dr_flac__swap_endian_uint64(n);
85256 |     }
85257 |     return n;
85258 | }
85259 | static MA_INLINE ma_uint32 ma_dr_flac__le2host_32(ma_uint32 n)
85260 | {
85261 |     if (!ma_dr_flac__is_little_endian()) {
85262 |         return ma_dr_flac__swap_endian_uint32(n);
85263 |     }
85264 |     return n;
85265 | }
85266 | static MA_INLINE ma_uint32 ma_dr_flac__le2host_32_ptr_unaligned(const void* pData)
85267 | {
85268 |     const ma_uint8* pNum = (ma_uint8*)pData;
85269 |     return *pNum | *(pNum+1) << 8 | *(pNum+2) << 16 | *(pNum+3) << 24;
85270 | }
85271 | static MA_INLINE ma_uint32 ma_dr_flac__unsynchsafe_32(ma_uint32 n)
85272 | {
85273 |     ma_uint32 result = 0;
85274 |     result |= (n & 0x7F000000) >> 3;
85275 |     result |= (n & 0x007F0000) >> 2;
85276 |     result |= (n & 0x00007F00) >> 1;
85277 |     result |= (n & 0x0000007F) >> 0;
85278 |     return result;
85279 | }
85280 | static ma_uint8 ma_dr_flac__crc8_table[] = {
85281 |     0x00, 0x07, 0x0E, 0x09, 0x1C, 0x1B, 0x12, 0x15, 0x38, 0x3F, 0x36, 0x31, 0x24, 0x23, 0x2A, 0x2D,
85282 |     0x70, 0x77, 0x7E, 0x79, 0x6C, 0x6B, 0x62, 0x65, 0x48, 0x4F, 0x46, 0x41, 0x54, 0x53, 0x5A, 0x5D,
85283 |     0xE0, 0xE7, 0xEE, 0xE9, 0xFC, 0xFB, 0xF2, 0xF5, 0xD8, 0xDF, 0xD6, 0xD1, 0xC4, 0xC3, 0xCA, 0xCD,
85284 |     0x90, 0x97, 0x9E, 0x99, 0x8C, 0x8B, 0x82, 0x85, 0xA8, 0xAF, 0xA6, 0xA1, 0xB4, 0xB3, 0xBA, 0xBD,
85285 |     0xC7, 0xC0, 0xC9, 0xCE, 0xDB, 0xDC, 0xD5, 0xD2, 0xFF, 0xF8, 0xF1, 0xF6, 0xE3, 0xE4, 0xED, 0xEA,
85286 |     0xB7, 0xB0, 0xB9, 0xBE, 0xAB, 0xAC, 0xA5, 0xA2, 0x8F, 0x88, 0x81, 0x86, 0x93, 0x94, 0x9D, 0x9A,
85287 |     0x27, 0x20, 0x29, 0x2E, 0x3B, 0x3C, 0x35, 0x32, 0x1F, 0x18, 0x11, 0x16, 0x03, 0x04, 0x0D, 0x0A,

```

```

85288 | 0x57, 0x50, 0x59, 0x5E, 0x4B, 0x4C, 0x45, 0x42, 0x6F, 0x68, 0x61, 0x66, 0x73, 0x74, 0x7D, 0x7A,
85289 | 0x89, 0x8E, 0x87, 0x80, 0x95, 0x92, 0x9B, 0x9C, 0xB1, 0xB6, 0xBF, 0xB8, 0xAD, 0xAA, 0xA3, 0xA4,
85290 | 0xF9, 0xFE, 0xF7, 0xF0, 0xE5, 0xE2, 0xEB, 0xEC, 0xC1, 0xC6, 0xCF, 0xC8, 0xDD, 0xDA, 0xD3, 0xD4,
85291 | 0x69, 0x6E, 0x67, 0x60, 0x75, 0x72, 0x7B, 0x7C, 0x51, 0x56, 0x5F, 0x58, 0x4D, 0x4A, 0x43, 0x44,
85292 | 0x19, 0x1E, 0x17, 0x10, 0x05, 0x02, 0x0B, 0x0C, 0x21, 0x26, 0x2F, 0x28, 0x3D, 0x3A, 0x33, 0x34,
85293 | 0x4E, 0x49, 0x40, 0x47, 0x52, 0x55, 0x5C, 0x5B, 0x76, 0x71, 0x78, 0x7F, 0x6A, 0x6D, 0x64, 0x63,
85294 | 0x3E, 0x39, 0x30, 0x37, 0x22, 0x25, 0x2C, 0x2B, 0x06, 0x01, 0x08, 0x0F, 0x1A, 0x1D, 0x14, 0x13,
85295 | 0xAE, 0xA9, 0xA0, 0xA7, 0xB2, 0xB5, 0xBC, 0xBB, 0x96, 0x91, 0x98, 0x9F, 0x8A, 0x8D, 0x84, 0x83,
85296 | 0xDE, 0xD9, 0xD0, 0xD7, 0xC2, 0xC5, 0xCC, 0xCB, 0xE6, 0xE1, 0xE8, 0xEF, 0xFA, 0xFD, 0xF4, 0xF3
85297 | };
85298 | static ma_uint16 ma_dr_flac_crc16_table[] = {
85299 |     0x0000, 0x8005, 0x800F, 0x000A, 0x801B, 0x001E, 0x0014, 0x8011,
85300 |     0x8033, 0x0036, 0x003C, 0x8039, 0x0028, 0x802D, 0x8027, 0x0022,
85301 |     0x8063, 0x0066, 0x006C, 0x8069, 0x0078, 0x807D, 0x8077, 0x0072,
85302 |     0x0050, 0x8055, 0x805F, 0x005A, 0x804B, 0x004E, 0x0044, 0x8041,
85303 |     0x80C3, 0x00C6, 0x00CC, 0x80C9, 0x00D8, 0x80DD, 0x80D7, 0x00D2,
85304 |     0x00F0, 0x80F5, 0x80FF, 0x00FA, 0x80EB, 0x00EE, 0x00E4, 0x80E1,
85305 |     0x00A0, 0x80A5, 0x80AF, 0x00AA, 0x80BB, 0x00BE, 0x00B4, 0x80B1,
85306 |     0x8093, 0x0096, 0x009C, 0x8099, 0x0088, 0x808D, 0x8087, 0x0082,
85307 |     0x8183, 0x0186, 0x018C, 0x8189, 0x0198, 0x819D, 0x8197, 0x0192,
85308 |     0x01B0, 0x81B5, 0x81BF, 0x01BA, 0x81AB, 0x01AE, 0x01A4, 0x81A1,
85309 |     0x01E0, 0x81E5, 0x81EF, 0x01EA, 0x81FB, 0x01FE, 0x01F4, 0x81F1,
85310 |     0x81D3, 0x01D6, 0x01DC, 0x81D9, 0x01C8, 0x81CD, 0x81C7, 0x01C2,
85311 |     0x0140, 0x8145, 0x814F, 0x014A, 0x815B, 0x015E, 0x0154, 0x8151,
85312 |     0x8173, 0x0176, 0x017C, 0x8179, 0x0168, 0x816D, 0x8167, 0x0162,
85313 |     0x8123, 0x0126, 0x012C, 0x8129, 0x0138, 0x813D, 0x8137, 0x0132,
85314 |     0x0110, 0x8115, 0x811F, 0x011A, 0x810B, 0x010E, 0x0104, 0x8101,
85315 |     0x8303, 0x0306, 0x030C, 0x8309, 0x0318, 0x831D, 0x8317, 0x0312,
85316 |     0x0330, 0x8335, 0x833F, 0x033A, 0x832B, 0x032E, 0x0324, 0x8321,
85317 |     0x0360, 0x8365, 0x836F, 0x036A, 0x837B, 0x037E, 0x0374, 0x8371,
85318 |     0x8353, 0x0356, 0x035C, 0x8359, 0x0348, 0x834D, 0x8347, 0x0342,
85319 |     0x03C0, 0x83C5, 0x83CF, 0x03CA, 0x83DB, 0x03DE, 0x03D4, 0x83D1,
85320 |     0x83F3, 0x03F6, 0x03FC, 0x83F9, 0x03E8, 0x83ED, 0x83E7, 0x03E2,
85321 |     0x83A3, 0x03A6, 0x03AC, 0x83A9, 0x03B8, 0x83BD, 0x83B7, 0x03B2,
85322 |     0x0390, 0x8395, 0x839F, 0x039A, 0x838B, 0x038E, 0x0384, 0x8381,
85323 |     0x0280, 0x8285, 0x828F, 0x028A, 0x829B, 0x029E, 0x0294, 0x8291,
85324 |     0x82B3, 0x02B6, 0x02BC, 0x82B9, 0x02A8, 0x82AD, 0x82A7, 0x02A2,
85325 |     0x82E3, 0x02E6, 0x02EC, 0x82E9, 0x02F8, 0x82FD, 0x82F7, 0x02F2,
85326 |     0x02D0, 0x82D5, 0x82DF, 0x02DA, 0x82CB, 0x02CE, 0x02C4, 0x82C1,
85327 |     0x8243, 0x0246, 0x024C, 0x8249, 0x0258, 0x825D, 0x8257, 0x0252,
85328 |     0x0270, 0x8275, 0x827F, 0x027A, 0x826B, 0x026E, 0x0264, 0x8261,
85329 |     0x0220, 0x8225, 0x822F, 0x022A, 0x822B, 0x022E, 0x0234, 0x8231,
85330 |     0x8213, 0x0216, 0x021C, 0x8219, 0x0208, 0x820D, 0x8207, 0x0202
85331 | };
85332 | static MA_INLINE ma_uint8 ma_dr_flac_crc8_byte(ma_uint8 crc, ma_uint8 data)
85333 | {
85334 |     return ma_dr_flac_crc8_table[crc ^ data];
85335 | }
85336 | static MA_INLINE ma_uint8 ma_dr_flac_crc8(ma_uint8 crc, ma_uint32 data, ma_uint32 count)
85337 | {
85338 |     #ifdef MA_DR_FLAC_NO_CRC
85339 |         (void)crc;
85340 |         (void)data;
85341 |         (void)count;
85342 |         return 0;
85343 |     #else
85344 |     #if 0
85345 |         ma_uint8 p = 0x07;
85346 |         for (int i = count-1; i >= 0; --i) {
85347 |             ma_uint8 bit = (data & (1 << i)) >> i;
85348 |             if (crc & 0x80) {
85349 |                 crc = ((crc << 1) | bit) ^ p;
85350 |             } else {
85351 |                 crc = ((crc << 1) | bit);
85352 |             }
85353 |         }
85354 |         return crc;
85355 |     #else
85356 |         ma_uint32 wholeBytes;
85357 |         ma_uint32 leftoverBits;
85358 |         ma_uint64 leftoverDataMask;
85359 |         static ma_uint64 leftoverDataMaskTable[8] = {
85360 |             0x00, 0x01, 0x03, 0x07, 0x0F, 0x1F, 0x3F, 0x7F
85361 |         };
85362 |         MA_DR_FLAC_ASSERT(count <= 32);
85363 |         wholeBytes = count >> 3;
85364 |         leftoverBits = count - (wholeBytes*8);
85365 |         leftoverDataMask = leftoverDataMaskTable[leftoverBits];
85366 |         switch (wholeBytes) {

```

```

85367 |         case 4: crc = ma_dr_flac_crc8_byte(crc, (ma_uint8)((data & (0xFF000000UL << leftoverBits))
> >> (24 + leftoverBits)));
85368 |         case 3: crc = ma_dr_flac_crc8_byte(crc, (ma_uint8)((data & (0x00FF0000UL << leftoverBits))
> >> (16 + leftoverBits)));
85369 |         case 2: crc = ma_dr_flac_crc8_byte(crc, (ma_uint8)((data & (0x0000FF00UL << leftoverBits))
> >> ( 8 + leftoverBits)));
85370 |         case 1: crc = ma_dr_flac_crc8_byte(crc, (ma_uint8)((data & (0x000000FFUL << leftoverBits))
> >> ( 0 + leftoverBits)));
85371 |         case 0: if (leftoverBits > 0) crc = (ma_uint8)((crc << leftoverBits) ^
> ma_dr_flac_crc8_table[(crc >> (8 - leftoverBits)) ^ (data & leftoverDataMask)]);
85372 |     }
85373 |     return crc;
85374 | #endif
85375 | #endif
85376 | }
85377 | static MA_INLINE ma_uint16 ma_dr_flac_crc16_byte(ma_uint16 crc, ma_uint8 data)
85378 | {
85379 |     return (crc << 8) ^ ma_dr_flac_crc16_table[(ma_uint8)(crc >> 8) ^ data];
85380 | }
85381 | static MA_INLINE ma_uint16 ma_dr_flac_crc16_cache(ma_uint16 crc, ma_dr_flac_cache_t data)
85382 | {
85383 | #ifdef MA_64BIT
85384 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 56) & 0xFF));
85385 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 48) & 0xFF));
85386 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 40) & 0xFF));
85387 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 32) & 0xFF));
85388 | #endif
85389 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 24) & 0xFF));
85390 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 16) & 0xFF));
85391 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >>  8) & 0xFF));
85392 |     crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >>  0) & 0xFF));
85393 |     return crc;
85394 | }
85395 | static MA_INLINE ma_uint16 ma_dr_flac_crc16_bytes(ma_uint16 crc, ma_dr_flac_cache_t data, ma_uint32
> byteCount)
85396 | {
85397 |     switch (byteCount)
85398 |     {
85399 | #ifdef MA_64BIT
85400 |         case 8: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 56) & 0xFF));
85401 |         case 7: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 48) & 0xFF));
85402 |         case 6: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 40) & 0xFF));
85403 |         case 5: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 32) & 0xFF));
85404 | #endif
85405 |         case 4: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 24) & 0xFF));
85406 |         case 3: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >> 16) & 0xFF));
85407 |         case 2: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >>  8) & 0xFF));
85408 |         case 1: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data >>  0) & 0xFF));
85409 |     }
85410 |     return crc;
85411 | }
85412 | #if 0
85413 | static MA_INLINE ma_uint16 ma_dr_flac_crc16_32bit(ma_uint16 crc, ma_uint32 data, ma_uint32 count)
85414 | {
85415 | #ifdef MA_DR_FLAC_NO_CRC
85416 |     (void)crc;
85417 |     (void)data;
85418 |     (void)count;
85419 |     return 0;
85420 | #else
85421 | #if 0
85422 |     ma_uint16 p = 0x8005;
85423 |     for (int i = count-1; i >= 0; --i) {
85424 |         ma_uint16 bit = (data & (1ULL << i)) >> i;
85425 |         if (r & 0x8000) {
85426 |             r = ((r << 1) | bit) ^ p;
85427 |         } else {
85428 |             r = ((r << 1) | bit);
85429 |         }
85430 |     }
85431 |     return crc;
85432 | #else
85433 |     ma_uint32 wholeBytes;
85434 |     ma_uint32 leftoverBits;
85435 |     ma_uint64 leftoverDataMask;
85436 |     static ma_uint64 leftoverDataMaskTable[8] = {
85437 |         0x00, 0x01, 0x03, 0x07, 0x0F, 0x1F, 0x3F, 0x7F
85438 |     };
85439 |     MA_DR_FLAC_ASSERT(count <= 64);

```



```

85440 |     wholeBytes = count >> 3;
85441 |     leftoverBits = count & 7;
85442 |     leftoverDataMask = leftoverDataMaskTable[leftoverBits];
85443 |     switch (wholeBytes) {
85444 |         default:
85445 |             case 4: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (0xFF000000UL << leftoverBits))
> >> (24 + leftoverBits)));
85446 |             case 3: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (0x00FF0000UL << leftoverBits))
> >> (16 + leftoverBits)));
85447 |             case 2: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (0x0000FF00UL << leftoverBits))
> >> ( 8 + leftoverBits)));
85448 |             case 1: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (0x000000FFUL << leftoverBits))
> >> ( 0 + leftoverBits)));
85449 |             case 0: if (leftoverBits > 0) crc = (crc << leftoverBits) ^ ma_dr_flac_crc16_table[(crc >>
> (16 - leftoverBits)) ^ (data & leftoverDataMask)];
85450 |         }
85451 |     return crc;
85452 | #endif
85453 | #endif
85454 | }
85455 | static MA_INLINE ma_uint16 ma_dr_flac_crc16__64bit(ma_uint16 crc, ma_uint64 data, ma_uint32 count)
85456 | {
85457 |     #ifdef MA_DR_FLAC_NO_CRC
85458 |         (void)crc;
85459 |         (void)data;
85460 |         (void)count;
85461 |         return 0;
85462 |     #else
85463 |         ma_uint32 wholeBytes;
85464 |         ma_uint32 leftoverBits;
85465 |         ma_uint64 leftoverDataMask;
85466 |         static ma_uint64 leftoverDataMaskTable[8] = {
85467 |             0x00, 0x01, 0x03, 0x07, 0x0F, 0x1F, 0x3F, 0x7F
85468 |         };
85469 |         MA_DR_FLAC_ASSERT(count <= 64);
85470 |         wholeBytes = count >> 3;
85471 |         leftoverBits = count & 7;
85472 |         leftoverDataMask = leftoverDataMaskTable[leftoverBits];
85473 |         switch (wholeBytes) {
85474 |             default:
85475 |                 case 8: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0xFF000000 << 32)
> << leftoverBits)) >> (56 + leftoverBits)));
85476 |                 case 7: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0x00FF0000 << 32)
> << leftoverBits)) >> (48 + leftoverBits)));
85477 |                 case 6: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0x0000FF00 << 32)
> << leftoverBits)) >> (40 + leftoverBits)));
85478 |                 case 5: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0x000000FF << 32)
> << leftoverBits)) >> (32 + leftoverBits)));
85479 |                 case 4: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0xFF000000
> << leftoverBits)) >> (24 + leftoverBits)));
85480 |                 case 3: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0x00FF0000
> << leftoverBits)) >> (16 + leftoverBits)));
85481 |                 case 2: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0x0000FF00
> << leftoverBits)) >> ( 8 + leftoverBits)));
85482 |                 case 1: crc = ma_dr_flac_crc16_byte(crc, (ma_uint8)((data & (((ma_uint64)0x000000FF
> << leftoverBits)) >> ( 0 + leftoverBits)));
85483 |                 case 0: if (leftoverBits > 0) crc = (crc << leftoverBits) ^ ma_dr_flac_crc16_table[(crc >>
> (16 - leftoverBits)) ^ (data & leftoverDataMask)];
85484 |             }
85485 |         return crc;
85486 |     #endif
85487 | }
85488 | static MA_INLINE ma_uint16 ma_dr_flac_crc16(ma_uint16 crc, ma_dr_flac_cache_t data, ma_uint32 count)
85489 | {
85490 |     #ifdef MA_64BIT
85491 |         return ma_dr_flac_crc16__64bit(crc, data, count);
85492 |     #else
85493 |         return ma_dr_flac_crc16__32bit(crc, data, count);
85494 |     #endif
85495 | }
85496 | #endif
85497 | #ifdef MA_64BIT
85498 | #define ma_dr_flac__be2host__cache_line ma_dr_flac__be2host_64
85499 | #else
85500 | #define ma_dr_flac__be2host__cache_line ma_dr_flac__be2host_32
85501 | #endif
85502 | #define MA_DR_FLAC_CACHE_L1_SIZE_BYTES(bs) (sizeof((bs)->cache))
85503 | #define MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs) (sizeof((bs)->cache)*8)
85504 | #define MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs) (MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs) -

```

```

> (bs)->consumedBits)
85505 | #define MA_DR_FLAC_CACHE_L1_SELECTION_MASK(_bitCount)      (~( ~(ma_dr_flac_cache_t)0) >>
> (_bitCount)))
85506 | #define MA_DR_FLAC_CACHE_L1_SELECTION_SHIFT(bs, _bitCount) (MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs) -
> (_bitCount))
85507 | #define MA_DR_FLAC_CACHE_L1_SELECT(bs, _bitCount)          (((bs)->cache) &
> MA_DR_FLAC_CACHE_L1_SELECTION_MASK(_bitCount))
85508 | #define MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT(bs, _bitCount) (MA_DR_FLAC_CACHE_L1_SELECT((bs),
> (_bitCount)) >> MA_DR_FLAC_CACHE_L1_SELECTION_SHIFT((bs), (_bitCount)))
85509 | #define MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT_SAFE(bs, _bitCount) (MA_DR_FLAC_CACHE_L1_SELECT((bs),
> (_bitCount)) >> (MA_DR_FLAC_CACHE_L1_SELECTION_SHIFT((bs), (_bitCount)) &
> (MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs)-1)))
85510 | #define MA_DR_FLAC_CACHE_L2_SIZE_BYTES(bs)                  (sizeof((bs)->cacheL2))
85511 | #define MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)                   (MA_DR_FLAC_CACHE_L2_SIZE_BYTES(bs)
> / sizeof((bs)->cacheL2[0]))
85512 | #define MA_DR_FLAC_CACHE_L2_LINES_REMAINING(bs)              (MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)
> - (bs)->nextL2Line)
85513 | #ifndef MA_DR_FLAC_NO_CRC
85514 | static MA_INLINE void ma_dr_flac__reset_crc16(ma_dr_flac_bs* bs)
85515 | {
85516 |     bs->crc16 = 0;
85517 |     bs->crc16CacheIgnoredBytes = bs->consumedBits >> 3;
85518 | }
85519 | static MA_INLINE void ma_dr_flac__update_crc16(ma_dr_flac_bs* bs)
85520 | {
85521 |     if (bs->crc16CacheIgnoredBytes == 0) {
85522 |         bs->crc16 = ma_dr_flac_crc16_cache(bs->crc16, bs->crc16Cache);
85523 |     } else {
85524 |         bs->crc16 = ma_dr_flac_crc16_bytes(bs->crc16, bs->crc16Cache,
> MA_DR_FLAC_CACHE_L1_SIZE_BYTES(bs) - bs->crc16CacheIgnoredBytes);
85525 |         bs->crc16CacheIgnoredBytes = 0;
85526 |     }
85527 | }
85528 | static MA_INLINE ma_uint16 ma_dr_flac__flush_crc16(ma_dr_flac_bs* bs)
85529 | {
85530 |     MA_DR_FLAC_ASSERT((MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs) & 7) == 0);
85531 |     if (MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs) == 0) {
85532 |         ma_dr_flac__update_crc16(bs);
85533 |     } else {
85534 |         bs->crc16 = ma_dr_flac_crc16_bytes(bs->crc16, bs->crc16Cache >>
> MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs), (bs->consumedBits >> 3) - bs->crc16CacheIgnoredBytes);
85535 |         bs->crc16CacheIgnoredBytes = bs->consumedBits >> 3;
85536 |     }
85537 |     return bs->crc16;
85538 | }
85539 | #endif
85540 | static MA_INLINE ma_bool32 ma_dr_flac__reload_l1_cache_from_l2(ma_dr_flac_bs* bs)
85541 | {
85542 |     size_t bytesRead;
85543 |     size_t alignedL1LineCount;
85544 |     if (bs->nextL2Line < MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)) {
85545 |         bs->cache = bs->cacheL2[bs->nextL2Line++];
85546 |         return MA_TRUE;
85547 |     }
85548 |     if (bs->unalignedByteCount > 0) {
85549 |         return MA_FALSE;
85550 |     }
85551 |     bytesRead = bs->onRead(bs->pUserData, bs->cacheL2, MA_DR_FLAC_CACHE_L2_SIZE_BYTES(bs));
85552 |     bs->nextL2Line = 0;
85553 |     if (bytesRead == MA_DR_FLAC_CACHE_L2_SIZE_BYTES(bs)) {
85554 |         bs->cache = bs->cacheL2[bs->nextL2Line++];
85555 |         return MA_TRUE;
85556 |     }
85557 |     alignedL1LineCount = bytesRead / MA_DR_FLAC_CACHE_L1_SIZE_BYTES(bs);
85558 |     bs->unalignedByteCount = bytesRead - (alignedL1LineCount * MA_DR_FLAC_CACHE_L1_SIZE_BYTES(bs));
85559 |     if (bs->unalignedByteCount > 0) {
85560 |         bs->unalignedCache = bs->cacheL2[alignedL1LineCount];
85561 |     }
85562 |     if (alignedL1LineCount > 0) {
85563 |         size_t offset = MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs) - alignedL1LineCount;
85564 |         size_t i;
85565 |         for (i = alignedL1LineCount; i > 0; --i) {
85566 |             bs->cacheL2[i-1 + offset] = bs->cacheL2[i-1];
85567 |         }
85568 |         bs->nextL2Line = (ma_uint32)offset;
85569 |         bs->cache = bs->cacheL2[bs->nextL2Line++];
85570 |         return MA_TRUE;
85571 |     } else {
85572 |         bs->nextL2Line = MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs);

```

```

85573 |         return MA_FALSE;
85574 |     }
85575 | }
85576 | static ma_bool32 ma_dr_flac__reload_cache(ma_dr_flac_bs* bs)
85577 | {
85578 |     size_t bytesRead;
85579 | #ifndef MA_DR_FLAC_NO_CRC
85580 |     ma_dr_flac__update_crc16(bs);
85581 | #endif
85582 |     if (ma_dr_flac__reload_l1_cache_from_l2(bs)) {
85583 |         bs->cache = ma_dr_flac__be2host__cache_line(bs->cache);
85584 |         bs->consumedBits = 0;
85585 | #ifndef MA_DR_FLAC_NO_CRC
85586 |         bs->crc16Cache = bs->cache;
85587 | #endif
85588 |         return MA_TRUE;
85589 |     }
85590 |     bytesRead = bs->unalignedByteCount;
85591 |     if (bytesRead == 0) {
85592 |         bs->consumedBits = MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs);
85593 |         return MA_FALSE;
85594 |     }
85595 |     MA_DR_FLAC_ASSERT(bytesRead < MA_DR_FLAC_CACHE_L1_SIZE_BYTES(bs));
85596 |     bs->consumedBits = (ma_uint32)(MA_DR_FLAC_CACHE_L1_SIZE_BYTES(bs) - bytesRead) * 8;
85597 |     bs->cache = ma_dr_flac__be2host__cache_line(bs->unalignedCache);
85598 |     bs->cache &= MA_DR_FLAC_CACHE_L1_SELECTION_MASK(MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs));
85599 |     bs->unalignedByteCount = 0;
85600 | #ifndef MA_DR_FLAC_NO_CRC
85601 |     bs->crc16Cache = bs->cache >> bs->consumedBits;
85602 |     bs->crc16CacheIgnoredBytes = bs->consumedBits >> 3;
85603 | #endif
85604 |     return MA_TRUE;
85605 | }
85606 | static void ma_dr_flac__reset_cache(ma_dr_flac_bs* bs)
85607 | {
85608 |     bs->nextL2Line = MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs);
85609 |     bs->consumedBits = MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs);
85610 |     bs->cache = 0;
85611 |     bs->unalignedByteCount = 0;
85612 |     bs->unalignedCache = 0;
85613 | #ifndef MA_DR_FLAC_NO_CRC
85614 |     bs->crc16Cache = 0;
85615 |     bs->crc16CacheIgnoredBytes = 0;
85616 | #endif
85617 | }
85618 | static MA_INLINE ma_bool32 ma_dr_flac__read_uint32(ma_dr_flac_bs* bs, unsigned int bitCount,
> ma_uint32* pResultOut)
85619 | {
85620 |     MA_DR_FLAC_ASSERT(bs != NULL);
85621 |     MA_DR_FLAC_ASSERT(pResultOut != NULL);
85622 |     MA_DR_FLAC_ASSERT(bitCount > 0);
85623 |     MA_DR_FLAC_ASSERT(bitCount <= 32);
85624 |     if (bs->consumedBits == MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs)) {
85625 |         if (!ma_dr_flac__reload_cache(bs)) {
85626 |             return MA_FALSE;
85627 |         }
85628 |     }
85629 |     if (bitCount <= MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
85630 | #ifdef MA_64BIT
85631 |         *pResultOut = (ma_uint32)MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT(bs, bitCount);
85632 |         bs->consumedBits += bitCount;
85633 |         bs->cache <<= bitCount;
85634 | #else
85635 |         if (bitCount < MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs)) {
85636 |             *pResultOut = (ma_uint32)MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT(bs, bitCount);
85637 |             bs->consumedBits += bitCount;
85638 |             bs->cache <<= bitCount;
85639 |         } else {
85640 |             *pResultOut = (ma_uint32)bs->cache;
85641 |             bs->consumedBits = MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs);
85642 |             bs->cache = 0;
85643 |         }
85644 | #endif
85645 |         return MA_TRUE;
85646 |     } else {
85647 |         ma_uint32 bitCountHi = MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs);
85648 |         ma_uint32 bitCountLo = bitCount - bitCountHi;
85649 |         ma_uint32 resultHi;
85650 |         MA_DR_FLAC_ASSERT(bitCountHi > 0);

```

```

85651 |         MA_DR_FLAC_ASSERT(bitCountHi < 32);
85652 |         resultHi = (ma_uint32)MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT(bs, bitCountHi);
85653 |         if (!ma_dr_flac__reload_cache(bs)) {
85654 |             return MA_FALSE;
85655 |         }
85656 |         if (bitCountLo > MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
85657 |             return MA_FALSE;
85658 |         }
85659 |         *pResultOut = (resultHi << bitCountLo) | (ma_uint32)MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT(bs,
> bitCountLo);
85660 |         bs->consumedBits += bitCountLo;
85661 |         bs->cache <=< bitCountLo;
85662 |         return MA_TRUE;
85663 |     }
85664 | }
85665 | static ma_bool32 ma_dr_flac__read_int32(ma_dr_flac_bs* bs, unsigned int bitCount, ma_int32* pResult)
85666 | {
85667 |     ma_uint32 result;
85668 |     MA_DR_FLAC_ASSERT(bs != NULL);
85669 |     MA_DR_FLAC_ASSERT(pResult != NULL);
85670 |     MA_DR_FLAC_ASSERT(bitCount > 0);
85671 |     MA_DR_FLAC_ASSERT(bitCount <= 32);
85672 |     if (!ma_dr_flac__read_uint32(bs, bitCount, &result)) {
85673 |         return MA_FALSE;
85674 |     }
85675 |     if (bitCount < 32) {
85676 |         ma_uint32 signbit;
85677 |         signbit = ((result >> (bitCount-1)) & 0x01);
85678 |         result |= (~signbit + 1) << bitCount;
85679 |     }
85680 |     *pResult = (ma_int32)result;
85681 |     return MA_TRUE;
85682 | }
85683 | #ifdef MA_64BIT
85684 | static ma_bool32 ma_dr_flac__read_uint64(ma_dr_flac_bs* bs, unsigned int bitCount, ma_uint64*
> pResultOut)
85685 | {
85686 |     ma_uint32 resultHi;
85687 |     ma_uint32 resultLo;
85688 |     MA_DR_FLAC_ASSERT(bitCount <= 64);
85689 |     MA_DR_FLAC_ASSERT(bitCount > 32);
85690 |     if (!ma_dr_flac__read_uint32(bs, bitCount - 32, &resultHi)) {
85691 |         return MA_FALSE;
85692 |     }
85693 |     if (!ma_dr_flac__read_uint32(bs, 32, &resultLo)) {
85694 |         return MA_FALSE;
85695 |     }
85696 |     *pResultOut = (((ma_uint64)resultHi) << 32) | ((ma_uint64)resultLo);
85697 |     return MA_TRUE;
85698 | }
85699 | #endif
85700 | #if 0
85701 | static ma_bool32 ma_dr_flac__read_int64(ma_dr_flac_bs* bs, unsigned int bitCount, ma_int64*
> pResultOut)
85702 | {
85703 |     ma_uint64 result;
85704 |     ma_uint64 signbit;
85705 |     MA_DR_FLAC_ASSERT(bitCount <= 64);
85706 |     if (!ma_dr_flac__read_uint64(bs, bitCount, &result)) {
85707 |         return MA_FALSE;
85708 |     }
85709 |     signbit = ((result >> (bitCount-1)) & 0x01);
85710 |     result |= (~signbit + 1) << bitCount;
85711 |     *pResultOut = (ma_int64)result;
85712 |     return MA_TRUE;
85713 | }
85714 | #endif
85715 | static ma_bool32 ma_dr_flac__read_uint16(ma_dr_flac_bs* bs, unsigned int bitCount, ma_uint16*
> pResult)
85716 | {
85717 |     ma_uint32 result;
85718 |     MA_DR_FLAC_ASSERT(bs != NULL);
85719 |     MA_DR_FLAC_ASSERT(pResult != NULL);
85720 |     MA_DR_FLAC_ASSERT(bitCount > 0);
85721 |     MA_DR_FLAC_ASSERT(bitCount <= 16);
85722 |     if (!ma_dr_flac__read_uint32(bs, bitCount, &result)) {
85723 |         return MA_FALSE;
85724 |     }
85725 |     *pResult = (ma_uint16)result;

```

```

85726 |     return MA_TRUE;
85727 | }
85728 | #if 0
85729 | static ma_bool32 ma_dr_flac__read_int16(ma_dr_flac_bs* bs, unsigned int bitCount, ma_int16* pResult)
85730 | {
85731 |     ma_int32 result;
85732 |     MA_DR_FLAC_ASSERT(bs != NULL);
85733 |     MA_DR_FLAC_ASSERT(pResult != NULL);
85734 |     MA_DR_FLAC_ASSERT(bitCount > 0);
85735 |     MA_DR_FLAC_ASSERT(bitCount <= 16);
85736 |     if (!ma_dr_flac__read_int32(bs, bitCount, &result)) {
85737 |         return MA_FALSE;
85738 |     }
85739 |     *pResult = (ma_int16)result;
85740 |     return MA_TRUE;
85741 | }
85742 | #endif
85743 | static ma_bool32 ma_dr_flac__read_uint8(ma_dr_flac_bs* bs, unsigned int bitCount, ma_uint8* pResult)
85744 | {
85745 |     ma_uint32 result;
85746 |     MA_DR_FLAC_ASSERT(bs != NULL);
85747 |     MA_DR_FLAC_ASSERT(pResult != NULL);
85748 |     MA_DR_FLAC_ASSERT(bitCount > 0);
85749 |     MA_DR_FLAC_ASSERT(bitCount <= 8);
85750 |     if (!ma_dr_flac__read_uint32(bs, bitCount, &result)) {
85751 |         return MA_FALSE;
85752 |     }
85753 |     *pResult = (ma_uint8)result;
85754 |     return MA_TRUE;
85755 | }
85756 | static ma_bool32 ma_dr_flac__read_int8(ma_dr_flac_bs* bs, unsigned int bitCount, ma_int8* pResult)
85757 | {
85758 |     ma_int32 result;
85759 |     MA_DR_FLAC_ASSERT(bs != NULL);
85760 |     MA_DR_FLAC_ASSERT(pResult != NULL);
85761 |     MA_DR_FLAC_ASSERT(bitCount > 0);
85762 |     MA_DR_FLAC_ASSERT(bitCount <= 8);
85763 |     if (!ma_dr_flac__read_int32(bs, bitCount, &result)) {
85764 |         return MA_FALSE;
85765 |     }
85766 |     *pResult = (ma_int8)result;
85767 |     return MA_TRUE;
85768 | }
85769 | static ma_bool32 ma_dr_flac__seek_bits(ma_dr_flac_bs* bs, size_t bitsToSeek)
85770 | {
85771 |     if (bitsToSeek <= MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
85772 |         bs->consumedBits += (ma_uint32)bitsToSeek;
85773 |         bs->cache <= bitsToSeek;
85774 |         return MA_TRUE;
85775 |     } else {
85776 |         bitsToSeek -= MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs);
85777 |         bs->consumedBits += MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs);
85778 |         bs->cache = 0;
85779 | #ifdef MA_64BIT
85780 |         while (bitsToSeek >= MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs)) {
85781 |             ma_uint64 bin;
85782 |             if (!ma_dr_flac__read_uint64(bs, MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs), &bin)) {
85783 |                 return MA_FALSE;
85784 |             }
85785 |             bitsToSeek -= MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs);
85786 |         }
85787 | #else
85788 |         while (bitsToSeek >= MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs)) {
85789 |             ma_uint32 bin;
85790 |             if (!ma_dr_flac__read_uint32(bs, MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs), &bin)) {
85791 |                 return MA_FALSE;
85792 |             }
85793 |             bitsToSeek -= MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs);
85794 |         }
85795 | #endif
85796 |         while (bitsToSeek >= 8) {
85797 |             ma_uint8 bin;
85798 |             if (!ma_dr_flac__read_uint8(bs, 8, &bin)) {
85799 |                 return MA_FALSE;
85800 |             }
85801 |             bitsToSeek -= 8;
85802 |         }
85803 |         if (bitsToSeek > 0) {
85804 |             ma_uint8 bin;

```

```

85805 |         if (!ma_dr_flac__read_uint8(bs, (ma_uint32)bitsToSeek, &bin)) {
85806 |             return MA_FALSE;
85807 |         }
85808 |         bitsToSeek = 0;
85809 |     }
85810 |     MA_DR_FLAC_ASSERT(bitsToSeek == 0);
85811 |     return MA_TRUE;
85812 | }
85813 | }
85814 | static ma_bool32 ma_dr_flac__find_and_seek_to_next_sync_code(ma_dr_flac_bs* bs)
85815 | {
85816 |     MA_DR_FLAC_ASSERT(bs != NULL);
85817 |     if (!ma_dr_flac__seek_bits(bs, MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs) & 7)) {
85818 |         return MA_FALSE;
85819 |     }
85820 |     for (;;) {
85821 |         ma_uint8 hi;
85822 |         #ifndef MA_DR_FLAC_NO_CRC
85823 |         ma_dr_flac__reset_crc16(bs);
85824 |         #endif
85825 |         if (!ma_dr_flac__read_uint8(bs, 8, &hi)) {
85826 |             return MA_FALSE;
85827 |         }
85828 |         if (hi == 0xFF) {
85829 |             ma_uint8 lo;
85830 |             if (!ma_dr_flac__read_uint8(bs, 6, &lo)) {
85831 |                 return MA_FALSE;
85832 |             }
85833 |             if (lo == 0x3E) {
85834 |                 return MA_TRUE;
85835 |             } else {
85836 |                 if (!ma_dr_flac__seek_bits(bs, MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs) & 7)) {
85837 |                     return MA_FALSE;
85838 |                 }
85839 |             }
85840 |         }
85841 |     }
85842 | }
85843 | #if defined(MA_DR_FLAC_HAS_LZCNT_INTRINSIC)
85844 | #define MA_DR_FLAC_IMPLEMENT_CLZ_LZCNT
85845 | #endif
85846 | #if defined(_MSC_VER) && _MSC_VER >= 1400 && (defined(MA_X64) || defined(MA_X86)) &&
> !defined(__clang__)
85847 | #define MA_DR_FLAC_IMPLEMENT_CLZ_MSVC
85848 | #endif
85849 | #if defined(__WATCOMC__) && defined(__386__)
85850 | #define MA_DR_FLAC_IMPLEMENT_CLZ_WATCOM
85851 | #endif
85852 | #ifdef __MRC__
85853 | #include <intrinsics.h>
85854 | #define MA_DR_FLAC_IMPLEMENT_CLZ_MRC
85855 | #endif
85856 | static MA_INLINE ma_uint32 ma_dr_flac__clz_software(ma_dr_flac_cache_t x)
85857 | {
85858 |     ma_uint32 n;
85859 |     static ma_uint32 clz_table_4[] = {
85860 |         0,
85861 |         4,
85862 |         3, 3,
85863 |         2, 2, 2, 2,
85864 |         1, 1, 1, 1, 1, 1, 1, 1
85865 |     };
85866 |     if (x == 0) {
85867 |         return sizeof(x)*8;
85868 |     }
85869 |     n = clz_table_4[x >> (sizeof(x)*8 - 4)];
85870 |     if (n == 0) {
85871 |         #ifdef MA_64BIT
85872 |             if ((x & ((ma_uint64)0xFFFFFFFF << 32)) == 0) { n = 32; x <<= 32; }
85873 |             if ((x & ((ma_uint64)0xFFFF0000 << 32)) == 0) { n += 16; x <<= 16; }
85874 |             if ((x & ((ma_uint64)0xFF000000 << 32)) == 0) { n += 8; x <<= 8; }
85875 |             if ((x & ((ma_uint64)0xF0000000 << 32)) == 0) { n += 4; x <<= 4; }
85876 |         #else
85877 |             if ((x & 0xFFFF0000) == 0) { n = 16; x <<= 16; }
85878 |             if ((x & 0xFF000000) == 0) { n += 8; x <<= 8; }
85879 |             if ((x & 0xF0000000) == 0) { n += 4; x <<= 4; }
85880 |         #endif
85881 |         n += clz_table_4[x >> (sizeof(x)*8 - 4)];
85882 |     }

```

```

85883 |     return n - 1;
85884 | }
85885 | #ifdef MA_DR_FLAC_IMPLEMENT_CLZ_LZCNT
85886 | static MA_INLINE ma_bool32 ma_dr_flac__is_lzcnt_supported(void)
85887 | {
85888 | #if defined(MA_DR_FLAC_HAS_LZCNT_INTRINSIC) && defined(MA_ARM) && (defined(__ARM_ARCH) && __ARM_ARCH
> >= 5)
85889 |     return MA_TRUE;
85890 | #elif defined(__MRC__)
85891 |     return MA_TRUE;
85892 | #else
85893 |     #ifdef MA_DR_FLAC_HAS_LZCNT_INTRINSIC
85894 |         return ma_dr_flac__gIsLZCNTSupported;
85895 |     #else
85896 |         return MA_FALSE;
85897 |     #endif
85898 | #endif
85899 | }
85900 | static MA_INLINE ma_uint32 ma_dr_flac__clz_lzcnt(ma_dr_flac_cache_t x)
85901 | {
85902 | #if defined(_MSC_VER)
85903 |     #ifdef MA_64BIT
85904 |         return (ma_uint32)__lzcnt64(x);
85905 |     #else
85906 |         return (ma_uint32)__lzcnt(x);
85907 |     #endif
85908 | #else
85909 |     #if defined(__GNUC__) || defined(__clang__)
85910 |         #if defined(MA_X64)
85911 |             {
85912 |                 ma_uint64 r;
85913 |                 __asm__ __volatile__ (
85914 |                     "rep; bsr{q %1, %0| %0, %1}" : "=r"(r) : "r"(x) : "cc"
85915 |                 );
85916 |                 return (ma_uint32)r;
85917 |             }
85918 |         #elif defined(MA_X86)
85919 |             {
85920 |                 ma_uint32 r;
85921 |                 __asm__ __volatile__ (
85922 |                     "rep; bsr{l %1, %0| %0, %1}" : "=r"(r) : "r"(x) : "cc"
85923 |                 );
85924 |                 return r;
85925 |             }
85926 |         #elif defined(MA_ARM) && (defined(__ARM_ARCH) && __ARM_ARCH >= 5) &&
> !defined(__ARM_ARCH_6M__) && !(defined(__thumb__) && !defined(__thumb2__)) && !defined(MA_64BIT)
85927 |             {
85928 |                 unsigned int r;
85929 |                 __asm__ __volatile__ (
85930 |                     #if defined(MA_64BIT)
85931 |                         "clz %w[out], %w[in]" : [out]"=r"(r) : [in]"r"(x)
85932 |                     #else
85933 |                         "clz %[out], %[in]" : [out]"=r"(r) : [in]"r"(x)
85934 |                     #endif
85935 |                 );
85936 |                 return r;
85937 |             }
85938 |         #else
85939 |             if (x == 0) {
85940 |                 return sizeof(x)*8;
85941 |             }
85942 |             #ifdef MA_64BIT
85943 |                 return (ma_uint32)__builtin_clzll((ma_uint64)x);
85944 |             #else
85945 |                 return (ma_uint32)__builtin_clzl((ma_uint32)x);
85946 |             #endif
85947 |         #endif
85948 |     #else
85949 |         #error "This compiler does not support the lzcnt intrinsic."
85950 |     #endif
85951 | #endif
85952 | }
85953 | #endif
85954 | #ifdef MA_DR_FLAC_IMPLEMENT_CLZ_MSVC
85955 | #include <intrin.h>
85956 | static MA_INLINE ma_uint32 ma_dr_flac__clz_msvc(ma_dr_flac_cache_t x)
85957 | {
85958 |     ma_uint32 n;
85959 |     if (x == 0) {

```



```

85960 |         return sizeof(x)*8;
85961 |     }
85962 | #ifdef MA_64BIT
85963 |     _BitScanReverse64((unsigned long*)&n, x);
85964 | #else
85965 |     _BitScanReverse((unsigned long*)&n, x);
85966 | #endif
85967 |     return sizeof(x)*8 - n - 1;
85968 | }
85969 | #endif
85970 | #ifdef MA_DR_FLAC_IMPLEMENT_CLZ_WATCOM
85971 | static __inline ma_uint32 ma_dr_flac__clz_watcom (ma_uint32);
85972 | #ifdef MA_DR_FLAC_IMPLEMENT_CLZ_WATCOM_LZCNT
85973 | #pragma aux ma_dr_flac__clz_watcom_lzcnt = \
85974 |     "db 0F3h, 0Fh, 0BDh, 0C0h" \
85975 |     parm [eax] \
85976 |     value [eax] \
85977 |     modify nomemory;
85978 | #else
85979 | #pragma aux ma_dr_flac__clz_watcom = \
85980 |     "bsr eax, eax" \
85981 |     "xor eax, 31" \
85982 |     parm [eax] nomemory \
85983 |     value [eax] \
85984 |     modify exact [eax] nomemory;
85985 | #endif
85986 | #endif
85987 | static MA_INLINE ma_uint32 ma_dr_flac__clz(ma_dr_flac_cache_t x)
85988 | {
85989 | #ifdef MA_DR_FLAC_IMPLEMENT_CLZ_LZCNT
85990 |     if (ma_dr_flac__is_lzcnt_supported()) {
85991 |         return ma_dr_flac__clz_lzcnt(x);
85992 |     } else
85993 | #endif
85994 |     {
85995 | #ifdef MA_DR_FLAC_IMPLEMENT_CLZ_MSVC
85996 |         return ma_dr_flac__clz_msvc(x);
85997 | #elif defined(MA_DR_FLAC_IMPLEMENT_CLZ_WATCOM_LZCNT)
85998 |         return ma_dr_flac__clz_watcom_lzcnt(x);
85999 | #elif defined(MA_DR_FLAC_IMPLEMENT_CLZ_WATCOM)
86000 |         return (x == 0) ? sizeof(x)*8 : ma_dr_flac__clz_watcom(x);
86001 | #elif defined(__MRC__)
86002 |         return __cntlzw(x);
86003 | #else
86004 |         return ma_dr_flac__clz_software(x);
86005 | #endif
86006 |     }
86007 | }
86008 | static MA_INLINE ma_bool32 ma_dr_flac__seek_past_next_set_bit(ma_dr_flac_bs* bs, unsigned int*
> pOffsetOut)
86009 | {
86010 |     ma_uint32 zeroCounter = 0;
86011 |     ma_uint32 setBitOffsetPlus1;
86012 |     while (bs->cache == 0) {
86013 |         zeroCounter += (ma_uint32)MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs);
86014 |         if (!ma_dr_flac__reload_cache(bs)) {
86015 |             return MA_FALSE;
86016 |         }
86017 |     }
86018 |     if (bs->cache == 1) {
86019 |         *pOffsetOut = zeroCounter + (ma_uint32)MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs) - 1;
86020 |         if (!ma_dr_flac__reload_cache(bs)) {
86021 |             return MA_FALSE;
86022 |         }
86023 |         return MA_TRUE;
86024 |     }
86025 |     setBitOffsetPlus1 = ma_dr_flac__clz(bs->cache);
86026 |     setBitOffsetPlus1 += 1;
86027 |     if (setBitOffsetPlus1 > MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
86028 |         return MA_FALSE;
86029 |     }
86030 |     bs->consumedBits += setBitOffsetPlus1;
86031 |     bs->cache <= setBitOffsetPlus1;
86032 |     *pOffsetOut = zeroCounter + setBitOffsetPlus1 - 1;
86033 |     return MA_TRUE;
86034 | }
86035 | static ma_bool32 ma_dr_flac__seek_to_byte(ma_dr_flac_bs* bs, ma_uint64 offsetFromStart)
86036 | {
86037 |     MA_DR_FLAC_ASSERT(bs != NULL);

```

```

86038 |     MA_DR_FLAC_ASSERT(offsetFromStart > 0);
86039 |     if (offsetFromStart > 0x7FFFFFFF) {
86040 |         ma_uint64 bytesRemaining = offsetFromStart;
86041 |         if (!bs->onSeek(bs->pUserData, 0x7FFFFFFF, MA_DR_FLAC_SEEK_SET)) {
86042 |             return MA_FALSE;
86043 |         }
86044 |         bytesRemaining -= 0x7FFFFFFF;
86045 |         while (bytesRemaining > 0x7FFFFFFF) {
86046 |             if (!bs->onSeek(bs->pUserData, 0x7FFFFFFF, MA_DR_FLAC_SEEK_CUR)) {
86047 |                 return MA_FALSE;
86048 |             }
86049 |             bytesRemaining -= 0x7FFFFFFF;
86050 |         }
86051 |         if (bytesRemaining > 0) {
86052 |             if (!bs->onSeek(bs->pUserData, (int)bytesRemaining, MA_DR_FLAC_SEEK_CUR)) {
86053 |                 return MA_FALSE;
86054 |             }
86055 |         }
86056 |     } else {
86057 |         if (!bs->onSeek(bs->pUserData, (int)offsetFromStart, MA_DR_FLAC_SEEK_SET)) {
86058 |             return MA_FALSE;
86059 |         }
86060 |     }
86061 |     ma_dr_flac__reset_cache(bs);
86062 |     return MA_TRUE;
86063 | }
86064 | static ma_result ma_dr_flac__read_utf8_coded_number(ma_dr_flac_bs* bs, ma_uint64* pNumberOut,
> ma_uint8* pCRCOut)
86065 | {
86066 |     ma_uint8 crc;
86067 |     ma_uint64 result;
86068 |     ma_uint8 utf8[7] = {0};
86069 |     int byteCount;
86070 |     int i;
86071 |     MA_DR_FLAC_ASSERT(bs != NULL);
86072 |     MA_DR_FLAC_ASSERT(pNumberOut != NULL);
86073 |     MA_DR_FLAC_ASSERT(pCRCOut != NULL);
86074 |     crc = *pCRCOut;
86075 |     if (!ma_dr_flac__read_uint8(bs, 8, utf8)) {
86076 |         *pNumberOut = 0;
86077 |         return MA_AT_END;
86078 |     }
86079 |     crc = ma_dr_flac_crc8(crc, utf8[0], 8);
86080 |     if ((utf8[0] & 0x80) == 0) {
86081 |         *pNumberOut = utf8[0];
86082 |         *pCRCOut = crc;
86083 |         return MA_SUCCESS;
86084 |     }
86085 |     if ((utf8[0] & 0xE0) == 0xC0) {
86086 |         byteCount = 2;
86087 |     } else if ((utf8[0] & 0xF0) == 0xE0) {
86088 |         byteCount = 3;
86089 |     } else if ((utf8[0] & 0xF8) == 0xF0) {
86090 |         byteCount = 4;
86091 |     } else if ((utf8[0] & 0xFC) == 0xF8) {
86092 |         byteCount = 5;
86093 |     } else if ((utf8[0] & 0xFE) == 0xFC) {
86094 |         byteCount = 6;
86095 |     } else if ((utf8[0] & 0xFF) == 0xFE) {
86096 |         byteCount = 7;
86097 |     } else {
86098 |         *pNumberOut = 0;
86099 |         return MA_CRC_MISMATCH;
86100 |     }
86101 |     MA_DR_FLAC_ASSERT(byteCount > 1);
86102 |     result = (ma_uint64)(utf8[0] & (0xFF >> (byteCount + 1)));
86103 |     for (i = 1; i < byteCount; ++i) {
86104 |         if (!ma_dr_flac__read_uint8(bs, 8, utf8 + i)) {
86105 |             *pNumberOut = 0;
86106 |             return MA_AT_END;
86107 |         }
86108 |         crc = ma_dr_flac_crc8(crc, utf8[i], 8);
86109 |         result = (result << 6) | (utf8[i] & 0x3F);
86110 |     }
86111 |     *pNumberOut = result;
86112 |     *pCRCOut = crc;
86113 |     return MA_SUCCESS;
86114 | }
86115 | static MA_INLINE ma_uint32 ma_dr_flac__ilog2_u32(ma_uint32 x)

```

```

86116 | {
86117 | #if 1
86118 |     ma_uint32 result = 0;
86119 |     while (x > 0) {
86120 |         result += 1;
86121 |         x >>= 1;
86122 |     }
86123 |     return result;
86124 | #endif
86125 | }
86126 | static MA_INLINE ma_bool32 ma_dr_flac__use_64_bit_prediction(ma_uint32 bitsPerSample, ma_uint32
> order, ma_uint32 precision)
86127 | {
86128 |     return bitsPerSample + precision + ma_dr_flac__ilog2_u32(order) > 32;
86129 | }
86130 | #if defined(__clang__)
86131 | __attribute__((no_sanitize("signed-integer-overflow")))
86132 | #endif
86133 | static MA_INLINE ma_int32 ma_dr_flac__calculate_prediction_32(ma_uint32 order, ma_int32 shift, const
> ma_int32* coefficients, ma_int32* pDecodedSamples)
86134 | {
86135 |     ma_int32 prediction = 0;
86136 |     MA_DR_FLAC_ASSERT(order <= 32);
86137 |     switch (order)
86138 |     {
86139 |     case 32: prediction += coefficients[31] * pDecodedSamples[-32];
86140 |     case 31: prediction += coefficients[30] * pDecodedSamples[-31];
86141 |     case 30: prediction += coefficients[29] * pDecodedSamples[-30];
86142 |     case 29: prediction += coefficients[28] * pDecodedSamples[-29];
86143 |     case 28: prediction += coefficients[27] * pDecodedSamples[-28];
86144 |     case 27: prediction += coefficients[26] * pDecodedSamples[-27];
86145 |     case 26: prediction += coefficients[25] * pDecodedSamples[-26];
86146 |     case 25: prediction += coefficients[24] * pDecodedSamples[-25];
86147 |     case 24: prediction += coefficients[23] * pDecodedSamples[-24];
86148 |     case 23: prediction += coefficients[22] * pDecodedSamples[-23];
86149 |     case 22: prediction += coefficients[21] * pDecodedSamples[-22];
86150 |     case 21: prediction += coefficients[20] * pDecodedSamples[-21];
86151 |     case 20: prediction += coefficients[19] * pDecodedSamples[-20];
86152 |     case 19: prediction += coefficients[18] * pDecodedSamples[-19];
86153 |     case 18: prediction += coefficients[17] * pDecodedSamples[-18];
86154 |     case 17: prediction += coefficients[16] * pDecodedSamples[-17];
86155 |     case 16: prediction += coefficients[15] * pDecodedSamples[-16];
86156 |     case 15: prediction += coefficients[14] * pDecodedSamples[-15];
86157 |     case 14: prediction += coefficients[13] * pDecodedSamples[-14];
86158 |     case 13: prediction += coefficients[12] * pDecodedSamples[-13];
86159 |     case 12: prediction += coefficients[11] * pDecodedSamples[-12];
86160 |     case 11: prediction += coefficients[10] * pDecodedSamples[-11];
86161 |     case 10: prediction += coefficients[ 9] * pDecodedSamples[-10];
86162 |     case  9: prediction += coefficients[ 8] * pDecodedSamples[- 9];
86163 |     case  8: prediction += coefficients[ 7] * pDecodedSamples[- 8];
86164 |     case  7: prediction += coefficients[ 6] * pDecodedSamples[- 7];
86165 |     case  6: prediction += coefficients[ 5] * pDecodedSamples[- 6];
86166 |     case  5: prediction += coefficients[ 4] * pDecodedSamples[- 5];
86167 |     case  4: prediction += coefficients[ 3] * pDecodedSamples[- 4];
86168 |     case  3: prediction += coefficients[ 2] * pDecodedSamples[- 3];
86169 |     case  2: prediction += coefficients[ 1] * pDecodedSamples[- 2];
86170 |     case  1: prediction += coefficients[ 0] * pDecodedSamples[- 1];
86171 |     }
86172 |     return (ma_int32)(prediction >> shift);
86173 | }
86174 | static MA_INLINE ma_int32 ma_dr_flac__calculate_prediction_64(ma_uint32 order, ma_int32 shift, const
> ma_int32* coefficients, ma_int32* pDecodedSamples)
86175 | {
86176 |     ma_int64 prediction;
86177 |     MA_DR_FLAC_ASSERT(order <= 32);
86178 |     #ifndef MA_64BIT
86179 |     if (order == 8)
86180 |     {
86181 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86182 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86183 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86184 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86185 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86186 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86187 |         prediction += coefficients[6] * (ma_int64)pDecodedSamples[-7];
86188 |         prediction += coefficients[7] * (ma_int64)pDecodedSamples[-8];
86189 |     }
86190 |     else if (order == 7)
86191 |     {

```

```
86192 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86193 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86194 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86195 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86196 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86197 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86198 |         prediction += coefficients[6] * (ma_int64)pDecodedSamples[-7];
86199 |     }
86200 |     else if (order == 3)
86201 |     {
86202 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86203 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86204 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86205 |     }
86206 |     else if (order == 6)
86207 |     {
86208 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86209 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86210 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86211 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86212 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86213 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86214 |     }
86215 |     else if (order == 5)
86216 |     {
86217 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86218 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86219 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86220 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86221 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86222 |     }
86223 |     else if (order == 4)
86224 |     {
86225 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86226 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86227 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86228 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86229 |     }
86230 |     else if (order == 12)
86231 |     {
86232 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86233 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86234 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86235 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86236 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86237 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86238 |         prediction += coefficients[6] * (ma_int64)pDecodedSamples[-7];
86239 |         prediction += coefficients[7] * (ma_int64)pDecodedSamples[-8];
86240 |         prediction += coefficients[8] * (ma_int64)pDecodedSamples[-9];
86241 |         prediction += coefficients[9] * (ma_int64)pDecodedSamples[-10];
86242 |         prediction += coefficients[10] * (ma_int64)pDecodedSamples[-11];
86243 |         prediction += coefficients[11] * (ma_int64)pDecodedSamples[-12];
86244 |     }
86245 |     else if (order == 2)
86246 |     {
86247 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86248 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86249 |     }
86250 |     else if (order == 1)
86251 |     {
86252 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86253 |     }
86254 |     else if (order == 10)
86255 |     {
86256 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86257 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86258 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86259 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86260 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86261 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86262 |         prediction += coefficients[6] * (ma_int64)pDecodedSamples[-7];
86263 |         prediction += coefficients[7] * (ma_int64)pDecodedSamples[-8];
86264 |         prediction += coefficients[8] * (ma_int64)pDecodedSamples[-9];
86265 |         prediction += coefficients[9] * (ma_int64)pDecodedSamples[-10];
86266 |     }
86267 |     else if (order == 9)
86268 |     {
86269 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86270 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
```

```

86271 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86272 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86273 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86274 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86275 |         prediction += coefficients[6] * (ma_int64)pDecodedSamples[-7];
86276 |         prediction += coefficients[7] * (ma_int64)pDecodedSamples[-8];
86277 |         prediction += coefficients[8] * (ma_int64)pDecodedSamples[-9];
86278 |     }
86279 |     else if (order == 11)
86280 |     {
86281 |         prediction = coefficients[0] * (ma_int64)pDecodedSamples[-1];
86282 |         prediction += coefficients[1] * (ma_int64)pDecodedSamples[-2];
86283 |         prediction += coefficients[2] * (ma_int64)pDecodedSamples[-3];
86284 |         prediction += coefficients[3] * (ma_int64)pDecodedSamples[-4];
86285 |         prediction += coefficients[4] * (ma_int64)pDecodedSamples[-5];
86286 |         prediction += coefficients[5] * (ma_int64)pDecodedSamples[-6];
86287 |         prediction += coefficients[6] * (ma_int64)pDecodedSamples[-7];
86288 |         prediction += coefficients[7] * (ma_int64)pDecodedSamples[-8];
86289 |         prediction += coefficients[8] * (ma_int64)pDecodedSamples[-9];
86290 |         prediction += coefficients[9] * (ma_int64)pDecodedSamples[-10];
86291 |         prediction += coefficients[10] * (ma_int64)pDecodedSamples[-11];
86292 |     }
86293 |     else
86294 |     {
86295 |         int j;
86296 |         prediction = 0;
86297 |         for (j = 0; j < (int)order; ++j) {
86298 |             prediction += coefficients[j] * (ma_int64)pDecodedSamples[-j-1];
86299 |         }
86300 |     }
86301 | #endif
86302 | #ifdef MA_64BIT
86303 |     prediction = 0;
86304 |     switch (order)
86305 |     {
86306 |         case 32: prediction += coefficients[31] * (ma_int64)pDecodedSamples[-32];
86307 |         case 31: prediction += coefficients[30] * (ma_int64)pDecodedSamples[-31];
86308 |         case 30: prediction += coefficients[29] * (ma_int64)pDecodedSamples[-30];
86309 |         case 29: prediction += coefficients[28] * (ma_int64)pDecodedSamples[-29];
86310 |         case 28: prediction += coefficients[27] * (ma_int64)pDecodedSamples[-28];
86311 |         case 27: prediction += coefficients[26] * (ma_int64)pDecodedSamples[-27];
86312 |         case 26: prediction += coefficients[25] * (ma_int64)pDecodedSamples[-26];
86313 |         case 25: prediction += coefficients[24] * (ma_int64)pDecodedSamples[-25];
86314 |         case 24: prediction += coefficients[23] * (ma_int64)pDecodedSamples[-24];
86315 |         case 23: prediction += coefficients[22] * (ma_int64)pDecodedSamples[-23];
86316 |         case 22: prediction += coefficients[21] * (ma_int64)pDecodedSamples[-22];
86317 |         case 21: prediction += coefficients[20] * (ma_int64)pDecodedSamples[-21];
86318 |         case 20: prediction += coefficients[19] * (ma_int64)pDecodedSamples[-20];
86319 |         case 19: prediction += coefficients[18] * (ma_int64)pDecodedSamples[-19];
86320 |         case 18: prediction += coefficients[17] * (ma_int64)pDecodedSamples[-18];
86321 |         case 17: prediction += coefficients[16] * (ma_int64)pDecodedSamples[-17];
86322 |         case 16: prediction += coefficients[15] * (ma_int64)pDecodedSamples[-16];
86323 |         case 15: prediction += coefficients[14] * (ma_int64)pDecodedSamples[-15];
86324 |         case 14: prediction += coefficients[13] * (ma_int64)pDecodedSamples[-14];
86325 |         case 13: prediction += coefficients[12] * (ma_int64)pDecodedSamples[-13];
86326 |         case 12: prediction += coefficients[11] * (ma_int64)pDecodedSamples[-12];
86327 |         case 11: prediction += coefficients[10] * (ma_int64)pDecodedSamples[-11];
86328 |         case 10: prediction += coefficients[ 9] * (ma_int64)pDecodedSamples[-10];
86329 |         case  9: prediction += coefficients[ 8] * (ma_int64)pDecodedSamples[- 9];
86330 |         case  8: prediction += coefficients[ 7] * (ma_int64)pDecodedSamples[- 8];
86331 |         case  7: prediction += coefficients[ 6] * (ma_int64)pDecodedSamples[- 7];
86332 |         case  6: prediction += coefficients[ 5] * (ma_int64)pDecodedSamples[- 6];
86333 |         case  5: prediction += coefficients[ 4] * (ma_int64)pDecodedSamples[- 5];
86334 |         case  4: prediction += coefficients[ 3] * (ma_int64)pDecodedSamples[- 4];
86335 |         case  3: prediction += coefficients[ 2] * (ma_int64)pDecodedSamples[- 3];
86336 |         case  2: prediction += coefficients[ 1] * (ma_int64)pDecodedSamples[- 2];
86337 |         case  1: prediction += coefficients[ 0] * (ma_int64)pDecodedSamples[- 1];
86338 |     }
86339 | #endif
86340 |     return (ma_int32)(prediction >> shift);
86341 | }
86342 | #if 0
86343 | static ma_bool32 ma_dr_flac_decode_samples_with_residual_rice_reference(ma_dr_flac_bs* bs,
> ma_uint32 bitsPerSample, ma_uint32 count, ma_uint8 riceParam, ma_uint32 lpcOrder, ma_int32 lpcShift,
> ma_uint32 lpcPrecision, const ma_int32* coefficients, ma_int32* pSamplesOut)
86344 | {
86345 |     ma_uint32 i;
86346 |     MA_DR_FLAC_ASSERT(bs != NULL);
86347 |     MA_DR_FLAC_ASSERT(pSamplesOut != NULL);

```

```

86348 |     for (i = 0; i < count; ++i) {
86349 |         ma_uint32 zeroCounter = 0;
86350 |         for (;;) {
86351 |             ma_uint8 bit;
86352 |             if (!ma_dr_flac__read_uint8(bs, 1, &bit)) {
86353 |                 return MA_FALSE;
86354 |             }
86355 |             if (bit == 0) {
86356 |                 zeroCounter += 1;
86357 |             } else {
86358 |                 break;
86359 |             }
86360 |         }
86361 |         ma_uint32 decodedRice;
86362 |         if (riceParam > 0) {
86363 |             if (!ma_dr_flac__read_uint32(bs, riceParam, &decodedRice)) {
86364 |                 return MA_FALSE;
86365 |             }
86366 |         } else {
86367 |             decodedRice = 0;
86368 |         }
86369 |         decodedRice |= (zeroCounter << riceParam);
86370 |         if ((decodedRice & 0x01)) {
86371 |             decodedRice = ~(decodedRice >> 1);
86372 |         } else {
86373 |             decodedRice = (decodedRice >> 1);
86374 |         }
86375 |         if (ma_dr_flac__use_64_bit_prediction(bitsPerSample, lpcOrder, lpcPrecision)) {
86376 |             pSamplesOut[i] = decodedRice + ma_dr_flac__calculate_prediction_64(lpcOrder, lpcShift,
> coefficients, pSamplesOut + i);
86377 |         } else {
86378 |             pSamplesOut[i] = decodedRice + ma_dr_flac__calculate_prediction_32(lpcOrder, lpcShift,
> coefficients, pSamplesOut + i);
86379 |         }
86380 |     }
86381 |     return MA_TRUE;
86382 | }
86383 | #endif
86384 | #if 0
86385 | static ma_bool32 ma_dr_flac__read_rice_parts__reference(ma_dr_flac_bs* bs, ma_uint8 riceParam,
> ma_uint32* pZeroCounterOut, ma_uint32* pRiceParamPartOut)
86386 | {
86387 |     ma_uint32 zeroCounter = 0;
86388 |     ma_uint32 decodedRice;
86389 |     for (;;) {
86390 |         ma_uint8 bit;
86391 |         if (!ma_dr_flac__read_uint8(bs, 1, &bit)) {
86392 |             return MA_FALSE;
86393 |         }
86394 |         if (bit == 0) {
86395 |             zeroCounter += 1;
86396 |         } else {
86397 |             break;
86398 |         }
86399 |     }
86400 |     if (riceParam > 0) {
86401 |         if (!ma_dr_flac__read_uint32(bs, riceParam, &decodedRice)) {
86402 |             return MA_FALSE;
86403 |         }
86404 |     } else {
86405 |         decodedRice = 0;
86406 |     }
86407 |     *pZeroCounterOut = zeroCounter;
86408 |     *pRiceParamPartOut = decodedRice;
86409 |     return MA_TRUE;
86410 | }
86411 | #endif
86412 | #if 0
86413 | static MA_INLINE ma_bool32 ma_dr_flac__read_rice_parts(ma_dr_flac_bs* bs, ma_uint8 riceParam,
> ma_uint32* pZeroCounterOut, ma_uint32* pRiceParamPartOut)
86414 | {
86415 |     ma_dr_flac_cache_t riceParamMask;
86416 |     ma_uint32 zeroCounter;
86417 |     ma_uint32 setBitOffsetPlus1;
86418 |     ma_uint32 riceParamPart;
86419 |     ma_uint32 riceLength;
86420 |     MA_DR_FLAC_ASSERT(riceParam > 0);
86421 |     riceParamMask = MA_DR_FLAC_CACHE_L1_SELECTION_MASK(riceParam);
86422 |     zeroCounter = 0;

```



```

86423 | while (bs->cache == 0) {
86424 |     zeroCounter += (ma_uint32)MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs);
86425 |     if (!ma_dr_flac__reload_cache(bs)) {
86426 |         return MA_FALSE;
86427 |     }
86428 | }
86429 | setBitOffsetPlus1 = ma_dr_flac__clz(bs->cache);
86430 | zeroCounter += setBitOffsetPlus1;
86431 | setBitOffsetPlus1 += 1;
86432 | riceLength = setBitOffsetPlus1 + riceParam;
86433 | if (riceLength < MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
86434 |     riceParamPart = (ma_uint32)((bs->cache & (riceParamMask >> setBitOffsetPlus1)) >>
> MA_DR_FLAC_CACHE_L1_SELECTION_SHIFT(bs, riceLength));
86435 |     bs->consumedBits += riceLength;
86436 |     bs->cache <= riceLength;
86437 | } else {
86438 |     ma_uint32 bitCountLo;
86439 |     ma_dr_flac_cache_t resultHi;
86440 |     bs->consumedBits += riceLength;
86441 |     bs->cache <= setBitOffsetPlus1 & (MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs)-1);
86442 |     bitCountLo = bs->consumedBits - MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs);
86443 |     resultHi = MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT(bs, riceParam);
86444 |     if (bs->nextL2Line < MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)) {
86445 | #ifndef MA_DR_FLAC_NO_CRC
86446 |         ma_dr_flac__update_crc16(bs);
86447 | #endif
86448 |         bs->cache = ma_dr_flac__be2host__cache_line(bs->cacheL2[bs->nextL2Line++]);
86449 |         bs->consumedBits = 0;
86450 | #ifndef MA_DR_FLAC_NO_CRC
86451 |         bs->crc16Cache = bs->cache;
86452 | #endif
86453 |     } else {
86454 |         if (!ma_dr_flac__reload_cache(bs)) {
86455 |             return MA_FALSE;
86456 |         }
86457 |         if (bitCountLo > MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
86458 |             return MA_FALSE;
86459 |         }
86460 |     }
86461 |     riceParamPart = (ma_uint32)(resultHi | MA_DR_FLAC_CACHE_L1_SELECT_AND_SHIFT_SAFE(bs,
> bitCountLo));
86462 |     bs->consumedBits += bitCountLo;
86463 |     bs->cache <= bitCountLo;
86464 | }
86465 | pZeroCounterOut[0] = zeroCounter;
86466 | pRiceParamPartOut[0] = riceParamPart;
86467 | return MA_TRUE;
86468 | }
86469 | #endif
86470 | static MA_INLINE ma_bool32 ma_dr_flac__read_rice_parts_x1(ma_dr_flac_bs* bs, ma_uint8 riceParam,
> ma_uint32* pZeroCounterOut, ma_uint32* pRiceParamPartOut)
86471 | {
86472 |     ma_uint32 riceParamPlus1 = riceParam + 1;
86473 |     ma_uint32 riceParamPlus1Shift = MA_DR_FLAC_CACHE_L1_SELECTION_SHIFT(bs, riceParamPlus1);
86474 |     ma_uint32 riceParamPlus1MaxConsumedBits = MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs) - riceParamPlus1;
86475 |     ma_dr_flac_cache_t bs_cache = bs->cache;
86476 |     ma_uint32 bs_consumedBits = bs->consumedBits;
86477 |     ma_uint32 lzcount = ma_dr_flac__clz(bs_cache);
86478 |     if (lzcount < sizeof(bs_cache)*8) {
86479 |         pZeroCounterOut[0] = lzcount;
86480 |         extract_rice_param_part:
86481 |         bs_cache <= lzcount;
86482 |         bs_consumedBits += lzcount;
86483 |         if (bs_consumedBits <= riceParamPlus1MaxConsumedBits) {
86484 |             pRiceParamPartOut[0] = (ma_uint32)(bs_cache >> riceParamPlus1Shift);
86485 |             bs_cache <= riceParamPlus1;
86486 |             bs_consumedBits += riceParamPlus1;
86487 |         } else {
86488 |             ma_uint32 riceParamPartHi;
86489 |             ma_uint32 riceParamPartLo;
86490 |             ma_uint32 riceParamPartLoBitCount;
86491 |             riceParamPartHi = (ma_uint32)(bs_cache >> riceParamPlus1Shift);
86492 |             riceParamPartLoBitCount = bs_consumedBits - riceParamPlus1MaxConsumedBits;
86493 |             MA_DR_FLAC_ASSERT(riceParamPartLoBitCount > 0 && riceParamPartLoBitCount < 32);
86494 |             if (bs->nextL2Line < MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)) {
86495 | #ifndef MA_DR_FLAC_NO_CRC
86496 |                 ma_dr_flac__update_crc16(bs);
86497 | #endif
86498 |                 bs_cache = ma_dr_flac__be2host__cache_line(bs->cacheL2[bs->nextL2Line++]);

```



```

86499 |         bs_consumedBits = riceParamPartLoBitCount;
86500 |     #ifndef MA_DR_FLAC_NO_CRC
86501 |         bs->crc16Cache = bs_cache;
86502 |     #endif
86503 |     } else {
86504 |         if (!ma_dr_flac__reload_cache(bs)) {
86505 |             return MA_FALSE;
86506 |         }
86507 |         if (riceParamPartLoBitCount > MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
86508 |             return MA_FALSE;
86509 |         }
86510 |         bs_cache = bs->cache;
86511 |         bs_consumedBits = bs->consumedBits + riceParamPartLoBitCount;
86512 |     }
86513 |     riceParamPartLo = (ma_uint32)(bs_cache >> (MA_DR_FLAC_CACHE_L1_SELECTION_SHIFT(bs,
> riceParamPartLoBitCount)));
86514 |     pRiceParamPartOut[0] = riceParamPartHi | riceParamPartLo;
86515 |     bs_cache <<= riceParamPartLoBitCount;
86516 | }
86517 | } else {
86518 |     ma_uint32 zeroCounter = (ma_uint32)(MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs) - bs_consumedBits);
86519 |     for (;;) {
86520 |         if (bs->nextL2Line < MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)) {
86521 |             #ifndef MA_DR_FLAC_NO_CRC
86522 |                 ma_dr_flac__update_crc16(bs);
86523 |             #endif
86524 |             bs_cache = ma_dr_flac__be2host__cache_line(bs->cacheL2[bs->nextL2Line++]);
86525 |             bs_consumedBits = 0;
86526 |             #ifndef MA_DR_FLAC_NO_CRC
86527 |                 bs->crc16Cache = bs_cache;
86528 |             #endif
86529 |             } else {
86530 |                 if (!ma_dr_flac__reload_cache(bs)) {
86531 |                     return MA_FALSE;
86532 |                 }
86533 |                 bs_cache = bs->cache;
86534 |                 bs_consumedBits = bs->consumedBits;
86535 |             }
86536 |             lzcount = ma_dr_flac__clz(bs_cache);
86537 |             zeroCounter += lzcount;
86538 |             if (lzcount < sizeof(bs_cache)*8) {
86539 |                 break;
86540 |             }
86541 |         }
86542 |         pZeroCounterOut[0] = zeroCounter;
86543 |         goto extract_rice_param_part;
86544 |     }
86545 |     bs->cache = bs_cache;
86546 |     bs->consumedBits = bs_consumedBits;
86547 |     return MA_TRUE;
86548 | }
86549 | static MA_INLINE ma_bool32 ma_dr_flac__seek_rice_parts(ma_dr_flac_bs* bs, ma_uint8 riceParam)
86550 | {
86551 |     ma_uint32 riceParamPlus1 = riceParam + 1;
86552 |     ma_uint32 riceParamPlus1MaxConsumedBits = MA_DR_FLAC_CACHE_L1_SIZE_BITS(bs) - riceParamPlus1;
86553 |     ma_dr_flac_cache_t bs_cache = bs->cache;
86554 |     ma_uint32 bs_consumedBits = bs->consumedBits;
86555 |     ma_uint32 lzcount = ma_dr_flac__clz(bs_cache);
86556 |     if (lzcount < sizeof(bs_cache)*8) {
86557 |         extract_rice_param_part:
86558 |         bs_cache <<= lzcount;
86559 |         bs_consumedBits += lzcount;
86560 |         if (bs_consumedBits <= riceParamPlus1MaxConsumedBits) {
86561 |             bs_cache <<= riceParamPlus1;
86562 |             bs_consumedBits += riceParamPlus1;
86563 |         } else {
86564 |             ma_uint32 riceParamPartLoBitCount = bs_consumedBits - riceParamPlus1MaxConsumedBits;
86565 |             MA_DR_FLAC_ASSERT(riceParamPartLoBitCount > 0 && riceParamPartLoBitCount < 32);
86566 |             if (bs->nextL2Line < MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)) {
86567 |                 #ifndef MA_DR_FLAC_NO_CRC
86568 |                     ma_dr_flac__update_crc16(bs);
86569 |                 #endif
86570 |                 bs_cache = ma_dr_flac__be2host__cache_line(bs->cacheL2[bs->nextL2Line++]);
86571 |                 bs_consumedBits = riceParamPartLoBitCount;
86572 |                 #ifndef MA_DR_FLAC_NO_CRC
86573 |                     bs->crc16Cache = bs_cache;
86574 |                 #endif
86575 |             } else {
86576 |                 if (!ma_dr_flac__reload_cache(bs)) {

```

```

86577 |         return MA_FALSE;
86578 |     }
86579 |     if (riceParamPartLoBitCount > MA_DR_FLAC_CACHE_L1_BITS_REMAINING(bs)) {
86580 |         return MA_FALSE;
86581 |     }
86582 |     bs_cache = bs->cache;
86583 |     bs_consumedBits = bs->consumedBits + riceParamPartLoBitCount;
86584 | }
86585 | bs_cache <=< riceParamPartLoBitCount;
86586 | }
86587 | } else {
86588 |     for (;;) {
86589 |         if (bs->nextL2Line < MA_DR_FLAC_CACHE_L2_LINE_COUNT(bs)) {
86590 |             #ifndef MA_DR_FLAC_NO_CRC
86591 |                 ma_dr_flac__update_crc16(bs);
86592 |             #endif
86593 |             bs_cache = ma_dr_flac__be2host__cache_line(bs->cacheL2[bs->nextL2Line++]);
86594 |             bs_consumedBits = 0;
86595 |             #ifndef MA_DR_FLAC_NO_CRC
86596 |                 bs->crc16Cache = bs_cache;
86597 |             #endif
86598 |         } else {
86599 |             if (!ma_dr_flac__reload_cache(bs)) {
86600 |                 return MA_FALSE;
86601 |             }
86602 |             bs_cache = bs->cache;
86603 |             bs_consumedBits = bs->consumedBits;
86604 |         }
86605 |         lzcount = ma_dr_flac__clz(bs_cache);
86606 |         if (lzcount < sizeof(bs_cache)*8) {
86607 |             break;
86608 |         }
86609 |     }
86610 |     goto extract_rice_param_part;
86611 | }
86612 | bs->cache = bs_cache;
86613 | bs->consumedBits = bs_consumedBits;
86614 | return MA_TRUE;
86615 | }
86616 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_rice_scalar_zeroorder(ma_dr_flac_bs* bs,
> ma_uint32 bitsPerSample, ma_uint32 count, ma_uint8 riceParam, ma_uint32 order, ma_int32 shift,
> const ma_int32* coefficients, ma_int32* pSamplesOut)
86617 | {
86618 |     ma_uint32 t[2] = {0x00000000, 0xFFFFFFFF};
86619 |     ma_uint32 zeroCountPart0;
86620 |     ma_uint32 riceParamPart0;
86621 |     ma_uint32 riceParamMask;
86622 |     ma_uint32 i;
86623 |     MA_DR_FLAC_ASSERT(bs != NULL);
86624 |     MA_DR_FLAC_ASSERT(pSamplesOut != NULL);
86625 |     (void)bitsPerSample;
86626 |     (void)order;
86627 |     (void)shift;
86628 |     (void)coefficients;
86629 |     riceParamMask = (ma_uint32)~((~0UL) << riceParam);
86630 |     i = 0;
86631 |     while (i < count) {
86632 |         if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart0, &riceParamPart0)) {
86633 |             return MA_FALSE;
86634 |         }
86635 |         riceParamPart0 &= riceParamMask;
86636 |         riceParamPart0 |= (zeroCountPart0 << riceParam);
86637 |         riceParamPart0 = (riceParamPart0 >> 1) ^ t[riceParamPart0 & 0x01];
86638 |         pSamplesOut[i] = riceParamPart0;
86639 |         i += 1;
86640 |     }
86641 |     return MA_TRUE;
86642 | }
86643 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_rice_scalar(ma_dr_flac_bs* bs, ma_uint32
> bitsPerSample, ma_uint32 count, ma_uint8 riceParam, ma_uint32 lpcOrder, ma_int32 lpcShift,
> ma_uint32 lpcPrecision, const ma_int32* coefficients, ma_int32* pSamplesOut)
86644 | {
86645 |     ma_uint32 t[2] = {0x00000000, 0xFFFFFFFF};
86646 |     ma_uint32 zeroCountPart0 = 0;
86647 |     ma_uint32 zeroCountPart1 = 0;
86648 |     ma_uint32 zeroCountPart2 = 0;
86649 |     ma_uint32 zeroCountPart3 = 0;
86650 |     ma_uint32 riceParamPart0 = 0;
86651 |     ma_uint32 riceParamPart1 = 0;

```

```

86652 |     ma_uint32 riceParamPart2 = 0;
86653 |     ma_uint32 riceParamPart3 = 0;
86654 |     ma_uint32 riceParamMask;
86655 |     const ma_int32* pSamplesOutEnd;
86656 |     ma_uint32 i;
86657 |     MA_DR_FLAC_ASSERT(bs != NULL);
86658 |     MA_DR_FLAC_ASSERT(pSamplesOut != NULL);
86659 |     if (lpcOrder == 0) {
86660 |         return ma_dr_flac__decode_samples_with_residual_rice_scalar_zeroorder(bs, bitsPerSample,
> count, riceParam, lpcOrder, lpcShift, coefficients, pSamplesOut);
86661 |     }
86662 |     riceParamMask = (ma_uint32)~((~0UL) << riceParam);
86663 |     pSamplesOutEnd = pSamplesOut + (count & ~3);
86664 |     if (ma_dr_flac__use_64_bit_prediction(bitsPerSample, lpcOrder, lpcPrecision)) {
86665 |         while (pSamplesOut < pSamplesOutEnd) {
86666 |             if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart0, &riceParamPart0) ||
86667 |                 !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart1, &riceParamPart1) ||
86668 |                 !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart2, &riceParamPart2) ||
86669 |                 !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart3, &riceParamPart3)) {
86670 |                 return MA_FALSE;
86671 |             }
86672 |             riceParamPart0 &= riceParamMask;
86673 |             riceParamPart1 &= riceParamMask;
86674 |             riceParamPart2 &= riceParamMask;
86675 |             riceParamPart3 &= riceParamMask;
86676 |             riceParamPart0 |= (zeroCountPart0 << riceParam);
86677 |             riceParamPart1 |= (zeroCountPart1 << riceParam);
86678 |             riceParamPart2 |= (zeroCountPart2 << riceParam);
86679 |             riceParamPart3 |= (zeroCountPart3 << riceParam);
86680 |             riceParamPart0 = (riceParamPart0 >> 1) ^ t[riceParamPart0 & 0x01];
86681 |             riceParamPart1 = (riceParamPart1 >> 1) ^ t[riceParamPart1 & 0x01];
86682 |             riceParamPart2 = (riceParamPart2 >> 1) ^ t[riceParamPart2 & 0x01];
86683 |             riceParamPart3 = (riceParamPart3 >> 1) ^ t[riceParamPart3 & 0x01];
86684 |             pSamplesOut[0] = riceParamPart0 + ma_dr_flac__calculate_prediction_64(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 0);
86685 |             pSamplesOut[1] = riceParamPart1 + ma_dr_flac__calculate_prediction_64(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 1);
86686 |             pSamplesOut[2] = riceParamPart2 + ma_dr_flac__calculate_prediction_64(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 2);
86687 |             pSamplesOut[3] = riceParamPart3 + ma_dr_flac__calculate_prediction_64(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 3);
86688 |             pSamplesOut += 4;
86689 |         }
86690 |     } else {
86691 |         while (pSamplesOut < pSamplesOutEnd) {
86692 |             if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart0, &riceParamPart0) ||
86693 |                 !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart1, &riceParamPart1) ||
86694 |                 !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart2, &riceParamPart2) ||
86695 |                 !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart3, &riceParamPart3)) {
86696 |                 return MA_FALSE;
86697 |             }
86698 |             riceParamPart0 &= riceParamMask;
86699 |             riceParamPart1 &= riceParamMask;
86700 |             riceParamPart2 &= riceParamMask;
86701 |             riceParamPart3 &= riceParamMask;
86702 |             riceParamPart0 |= (zeroCountPart0 << riceParam);
86703 |             riceParamPart1 |= (zeroCountPart1 << riceParam);
86704 |             riceParamPart2 |= (zeroCountPart2 << riceParam);
86705 |             riceParamPart3 |= (zeroCountPart3 << riceParam);
86706 |             riceParamPart0 = (riceParamPart0 >> 1) ^ t[riceParamPart0 & 0x01];
86707 |             riceParamPart1 = (riceParamPart1 >> 1) ^ t[riceParamPart1 & 0x01];
86708 |             riceParamPart2 = (riceParamPart2 >> 1) ^ t[riceParamPart2 & 0x01];
86709 |             riceParamPart3 = (riceParamPart3 >> 1) ^ t[riceParamPart3 & 0x01];
86710 |             pSamplesOut[0] = riceParamPart0 + ma_dr_flac__calculate_prediction_32(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 0);
86711 |             pSamplesOut[1] = riceParamPart1 + ma_dr_flac__calculate_prediction_32(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 1);
86712 |             pSamplesOut[2] = riceParamPart2 + ma_dr_flac__calculate_prediction_32(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 2);
86713 |             pSamplesOut[3] = riceParamPart3 + ma_dr_flac__calculate_prediction_32(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 3);
86714 |             pSamplesOut += 4;
86715 |         }
86716 |     }
86717 |     i = (count & ~3);
86718 |     while (i < count) {
86719 |         if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountPart0, &riceParamPart0)) {
86720 |             return MA_FALSE;
86721 |         }

```

```

86722 |         riceParamPart0 &= riceParamMask;
86723 |         riceParamPart0 |= (zeroCountPart0 << riceParam);
86724 |         riceParamPart0 = (riceParamPart0 >> 1) ^ t[riceParamPart0 & 0x01];
86725 |         if (ma_dr_flac__use_64_bit_prediction(bitsPerSample, lpcOrder, lpcPrecision)) {
86726 |             pSamplesOut[0] = riceParamPart0 + ma_dr_flac__calculate_prediction_64(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 0);
86727 |         } else {
86728 |             pSamplesOut[0] = riceParamPart0 + ma_dr_flac__calculate_prediction_32(lpcOrder,
> lpcShift, coefficients, pSamplesOut + 0);
86729 |         }
86730 |         i += 1;
86731 |         pSamplesOut += 1;
86732 |     }
86733 |     return MA_TRUE;
86734 | }
86735 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
86736 | static MA_INLINE __m128i ma_dr_flac__mm_packs_interleaved_epi32(__m128i a, __m128i b)
86737 | {
86738 |     __m128i r;
86739 |     r = _mm_packs_epi32(a, b);
86740 |     r = _mm_shuffle_epi32(r, _MM_SHUFFLE(3, 1, 2, 0));
86741 |     r = _mm_shufflehi_epi16(r, _MM_SHUFFLE(3, 1, 2, 0));
86742 |     r = _mm_shufflelo_epi16(r, _MM_SHUFFLE(3, 1, 2, 0));
86743 |     return r;
86744 | }
86745 | #endif
86746 | #if defined(MA_DR_FLAC_SUPPORT_SSE41)
86747 | static MA_INLINE __m128i ma_dr_flac__mm_not_si128(__m128i a)
86748 | {
86749 |     return _mm_xor_si128(a, _mm_cmpeq_epi32(_mm_setzero_si128(), _mm_setzero_si128()));
86750 | }
86751 | static MA_INLINE __m128i ma_dr_flac__mm_hadd_epi32(__m128i x)
86752 | {
86753 |     __m128i x64 = _mm_add_epi32(x, _mm_shuffle_epi32(x, _MM_SHUFFLE(1, 0, 3, 2)));
86754 |     __m128i x32 = _mm_shufflelo_epi16(x64, _MM_SHUFFLE(1, 0, 3, 2));
86755 |     return _mm_add_epi32(x64, x32);
86756 | }
86757 | static MA_INLINE __m128i ma_dr_flac__mm_hadd_epi64(__m128i x)
86758 | {
86759 |     return _mm_add_epi64(x, _mm_shuffle_epi32(x, _MM_SHUFFLE(1, 0, 3, 2)));
86760 | }
86761 | static MA_INLINE __m128i ma_dr_flac__mm_srai_epi64(__m128i x, int count)
86762 | {
86763 |     __m128i lo = _mm_srli_epi64(x, count);
86764 |     __m128i hi = _mm_srai_epi32(x, count);
86765 |     hi = _mm_and_si128(hi, _mm_set_epi32(0xFFFFFFFF, 0, 0xFFFFFFFF, 0));
86766 |     return _mm_or_si128(lo, hi);
86767 | }
86768 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_rice_sse41_32(ma_dr_flac_bs* bs,
> ma_uint32 count, ma_uint8 riceParam, ma_uint32 order, ma_int32 shift, const ma_int32* coefficients,
> ma_int32* pSamplesOut)
86769 | {
86770 |     int i;
86771 |     ma_uint32 riceParamMask;
86772 |     ma_int32* pDecodedSamples = pSamplesOut;
86773 |     ma_int32* pDecodedSamplesEnd = pSamplesOut + (count & ~3);
86774 |     ma_uint32 zeroCountParts0 = 0;
86775 |     ma_uint32 zeroCountParts1 = 0;
86776 |     ma_uint32 zeroCountParts2 = 0;
86777 |     ma_uint32 zeroCountParts3 = 0;
86778 |     ma_uint32 riceParamParts0 = 0;
86779 |     ma_uint32 riceParamParts1 = 0;
86780 |     ma_uint32 riceParamParts2 = 0;
86781 |     ma_uint32 riceParamParts3 = 0;
86782 |     __m128i coefficients128_0;
86783 |     __m128i coefficients128_4;
86784 |     __m128i coefficients128_8;
86785 |     __m128i samples128_0;
86786 |     __m128i samples128_4;
86787 |     __m128i samples128_8;
86788 |     __m128i riceParamMask128;
86789 |     const ma_uint32 t[2] = {0x00000000, 0xFFFFFFFF};
86790 |     riceParamMask = (ma_uint32)~((~0UL) << riceParam);
86791 |     riceParamMask128 = _mm_set1_epi32(riceParamMask);
86792 |     coefficients128_0 = _mm_setzero_si128();
86793 |     coefficients128_4 = _mm_setzero_si128();
86794 |     coefficients128_8 = _mm_setzero_si128();
86795 |     samples128_0 = _mm_setzero_si128();
86796 |     samples128_4 = _mm_setzero_si128();

```

```

86797 |     samples128_8 = _mm_setzero_si128();
86798 | #if 1
86799 | {
86800 |     int runningOrder = order;
86801 |     if (runningOrder >= 4) {
86802 |         coefficients128_0 = _mm_loadu_si128((const __m128i*)(coefficients + 0));
86803 |         samples128_0 = _mm_loadu_si128((const __m128i*)(pSamplesOut - 4));
86804 |         runningOrder -= 4;
86805 |     } else {
86806 |         switch (runningOrder) {
86807 |             case 3: coefficients128_0 = _mm_set_epi32(0, coefficients[2], coefficients[1],
> coefficients[0]); samples128_0 = _mm_set_epi32(pSamplesOut[-1], pSamplesOut[-2], pSamplesOut[-3],
> 0); break;
86808 |             case 2: coefficients128_0 = _mm_set_epi32(0, 0, coefficients[1],
> coefficients[0]); samples128_0 = _mm_set_epi32(pSamplesOut[-1], pSamplesOut[-2], 0,
> 0); break;
86809 |             case 1: coefficients128_0 = _mm_set_epi32(0, 0, 0,
> coefficients[0]); samples128_0 = _mm_set_epi32(pSamplesOut[-1], 0, 0,
> 0); break;
86810 |         }
86811 |         runningOrder = 0;
86812 |     }
86813 |     if (runningOrder >= 4) {
86814 |         coefficients128_4 = _mm_loadu_si128((const __m128i*)(coefficients + 4));
86815 |         samples128_4 = _mm_loadu_si128((const __m128i*)(pSamplesOut - 8));
86816 |         runningOrder -= 4;
86817 |     } else {
86818 |         switch (runningOrder) {
86819 |             case 3: coefficients128_4 = _mm_set_epi32(0, coefficients[6], coefficients[5],
> coefficients[4]); samples128_4 = _mm_set_epi32(pSamplesOut[-5], pSamplesOut[-6], pSamplesOut[-7],
> 0); break;
86820 |             case 2: coefficients128_4 = _mm_set_epi32(0, 0, coefficients[5],
> coefficients[4]); samples128_4 = _mm_set_epi32(pSamplesOut[-5], pSamplesOut[-6], 0,
> 0); break;
86821 |             case 1: coefficients128_4 = _mm_set_epi32(0, 0, 0,
> coefficients[4]); samples128_4 = _mm_set_epi32(pSamplesOut[-5], 0, 0,
> 0); break;
86822 |         }
86823 |         runningOrder = 0;
86824 |     }
86825 |     if (runningOrder == 4) {
86826 |         coefficients128_8 = _mm_loadu_si128((const __m128i*)(coefficients + 8));
86827 |         samples128_8 = _mm_loadu_si128((const __m128i*)(pSamplesOut - 12));
86828 |         runningOrder -= 4;
86829 |     } else {
86830 |         switch (runningOrder) {
86831 |             case 3: coefficients128_8 = _mm_set_epi32(0, coefficients[10], coefficients[9],
> coefficients[8]); samples128_8 = _mm_set_epi32(pSamplesOut[-9], pSamplesOut[-10], pSamplesOut[-11],
> 0); break;
86832 |             case 2: coefficients128_8 = _mm_set_epi32(0, 0, coefficients[9],
> coefficients[8]); samples128_8 = _mm_set_epi32(pSamplesOut[-9], pSamplesOut[-10], 0,
> 0); break;
86833 |             case 1: coefficients128_8 = _mm_set_epi32(0, 0, 0,
> coefficients[8]); samples128_8 = _mm_set_epi32(pSamplesOut[-9], 0, 0,
> 0); break;
86834 |         }
86835 |         runningOrder = 0;
86836 |     }
86837 |     coefficients128_0 = _mm_shuffle_epi32(coefficients128_0, _MM_SHUFFLE(0, 1, 2, 3));
86838 |     coefficients128_4 = _mm_shuffle_epi32(coefficients128_4, _MM_SHUFFLE(0, 1, 2, 3));
86839 |     coefficients128_8 = _mm_shuffle_epi32(coefficients128_8, _MM_SHUFFLE(0, 1, 2, 3));
86840 | }
86841 | #else
86842 |     switch (order)
86843 |     {
86844 |         case 12: ((ma_int32*)&coefficients128_8)[0] = coefficients[11]; ((ma_int32*)&samples128_8)[0] =
> pDecodedSamples[-12];
86845 |         case 11: ((ma_int32*)&coefficients128_8)[1] = coefficients[10]; ((ma_int32*)&samples128_8)[1] =
> pDecodedSamples[-11];
86846 |         case 10: ((ma_int32*)&coefficients128_8)[2] = coefficients[ 9]; ((ma_int32*)&samples128_8)[2] =
> pDecodedSamples[-10];
86847 |         case 9: ((ma_int32*)&coefficients128_8)[3] = coefficients[ 8]; ((ma_int32*)&samples128_8)[3] =
> pDecodedSamples[- 9];
86848 |         case 8: ((ma_int32*)&coefficients128_4)[0] = coefficients[ 7]; ((ma_int32*)&samples128_4)[0] =
> pDecodedSamples[- 8];
86849 |         case 7: ((ma_int32*)&coefficients128_4)[1] = coefficients[ 6]; ((ma_int32*)&samples128_4)[1] =
> pDecodedSamples[- 7];
86850 |         case 6: ((ma_int32*)&coefficients128_4)[2] = coefficients[ 5]; ((ma_int32*)&samples128_4)[2] =
> pDecodedSamples[- 6];

```

```

86851 |     case 5: ((ma_int32*)&coefficients128_4)[3] = coefficients[ 4]; ((ma_int32*)&samples128_4)[3] =
> pDecodedSamples[- 5];
86852 |     case 4: ((ma_int32*)&coefficients128_0)[0] = coefficients[ 3]; ((ma_int32*)&samples128_0)[0] =
> pDecodedSamples[- 4];
86853 |     case 3: ((ma_int32*)&coefficients128_0)[1] = coefficients[ 2]; ((ma_int32*)&samples128_0)[1] =
> pDecodedSamples[- 3];
86854 |     case 2: ((ma_int32*)&coefficients128_0)[2] = coefficients[ 1]; ((ma_int32*)&samples128_0)[2] =
> pDecodedSamples[- 2];
86855 |     case 1: ((ma_int32*)&coefficients128_0)[3] = coefficients[ 0]; ((ma_int32*)&samples128_0)[3] =
> pDecodedSamples[- 1];
86856 |     }
86857 | #endif
86858 |     while (pDecodedSamples < pDecodedSamplesEnd) {
86859 |         __m128i prediction128;
86860 |         __m128i zeroCountPart128;
86861 |         __m128i riceParamPart128;
86862 |         if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts0, &riceParamParts0) ||
86863 |             !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts1, &riceParamParts1) ||
86864 |             !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts2, &riceParamParts2) ||
86865 |             !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts3, &riceParamParts3)) {
86866 |             return MA_FALSE;
86867 |         }
86868 |         zeroCountPart128 = __mm_set_epi32(zeroCountParts3, zeroCountParts2, zeroCountParts1,
> zeroCountParts0);
86869 |         riceParamPart128 = __mm_set_epi32(riceParamParts3, riceParamParts2, riceParamParts1,
> riceParamParts0);
86870 |         riceParamPart128 = __mm_and_si128(riceParamPart128, riceParamMask128);
86871 |         riceParamPart128 = __mm_or_si128(riceParamPart128, __mm_slli_epi32(zeroCountPart128,
> riceParam));
86872 |         riceParamPart128 = __mm_xor_si128(__mm_srli_epi32(riceParamPart128, 1),
> __mm_add_epi32(ma_dr_flac__mm_not_si128(__mm_and_si128(riceParamPart128, __mm_set1_epi32(0x01))),
> __mm_set1_epi32(0x01)));
86873 |         if (order <= 4) {
86874 |             for (i = 0; i < 4; i += 1) {
86875 |                 prediction128 = __mm_mullo_epi32(coefficients128_0, samples128_0);
86876 |                 prediction128 = ma_dr_flac__mm_hadd_epi32(prediction128);
86877 |                 prediction128 = __mm_srai_epi32(prediction128, shift);
86878 |                 prediction128 = __mm_add_epi32(riceParamPart128, prediction128);
86879 |                 samples128_0 = __mm_alignr_epi8(prediction128, samples128_0, 4);
86880 |                 riceParamPart128 = __mm_alignr_epi8(__mm_setzero_si128(), riceParamPart128, 4);
86881 |             }
86882 |         } else if (order <= 8) {
86883 |             for (i = 0; i < 4; i += 1) {
86884 |                 prediction128 =
> __mm_mullo_epi32(coefficients128_4,
> samples128_4);
86885 |                 prediction128 = __mm_add_epi32(prediction128, __mm_mullo_epi32(coefficients128_0,
> samples128_0));
86886 |                 prediction128 = ma_dr_flac__mm_hadd_epi32(prediction128);
86887 |                 prediction128 = __mm_srai_epi32(prediction128, shift);
86888 |                 prediction128 = __mm_add_epi32(riceParamPart128, prediction128);
86889 |                 samples128_4 = __mm_alignr_epi8(samples128_0, samples128_4, 4);
86890 |                 samples128_0 = __mm_alignr_epi8(prediction128, samples128_0, 4);
86891 |                 riceParamPart128 = __mm_alignr_epi8(__mm_setzero_si128(), riceParamPart128, 4);
86892 |             }
86893 |         } else {
86894 |             for (i = 0; i < 4; i += 1) {
86895 |                 prediction128 =
> __mm_mullo_epi32(coefficients128_8,
> samples128_8);
86896 |                 prediction128 = __mm_add_epi32(prediction128, __mm_mullo_epi32(coefficients128_4,
> samples128_4));
86897 |                 prediction128 = __mm_add_epi32(prediction128, __mm_mullo_epi32(coefficients128_0,
> samples128_0));
86898 |                 prediction128 = ma_dr_flac__mm_hadd_epi32(prediction128);
86899 |                 prediction128 = __mm_srai_epi32(prediction128, shift);
86900 |                 prediction128 = __mm_add_epi32(riceParamPart128, prediction128);
86901 |                 samples128_8 = __mm_alignr_epi8(samples128_4, samples128_8, 4);
86902 |                 samples128_4 = __mm_alignr_epi8(samples128_0, samples128_4, 4);
86903 |                 samples128_0 = __mm_alignr_epi8(prediction128, samples128_0, 4);
86904 |                 riceParamPart128 = __mm_alignr_epi8(__mm_setzero_si128(), riceParamPart128, 4);
86905 |             }
86906 |         }
86907 |         __mm_storeu_si128((__m128i*)pDecodedSamples, samples128_0);
86908 |         pDecodedSamples += 4;
86909 |     }
86910 |     i = (count & ~3);
86911 |     while (i < (int)count) {
86912 |         if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts0, &riceParamParts0)) {
86913 |             return MA_FALSE;
86914 |         }

```



```

86915 |         riceParamParts0 &= riceParamMask;
86916 |         riceParamParts0 |= (zeroCountParts0 << riceParam);
86917 |         riceParamParts0 = (riceParamParts0 >> 1) ^ t[riceParamParts0 & 0x01];
86918 |         pDecodedSamples[0] = riceParamParts0 + ma_dr_flac__calculate_prediction_32(order, shift,
> coefficients, pDecodedSamples);
86919 |         i += 1;
86920 |         pDecodedSamples += 1;
86921 |     }
86922 |     return MA_TRUE;
86923 | }
86924 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_rice_sse41_64(ma_dr_flac_bs* bs,
> ma_uint32 count, ma_uint8 riceParam, ma_uint32 order, ma_int32 shift, const ma_int32* coefficients,
> ma_int32* pSamplesOut)
86925 | {
86926 |     int i;
86927 |     ma_uint32 riceParamMask;
86928 |     ma_int32* pDecodedSamples = pSamplesOut;
86929 |     ma_int32* pDecodedSamplesEnd = pSamplesOut + (count & ~3);
86930 |     ma_uint32 zeroCountParts0 = 0;
86931 |     ma_uint32 zeroCountParts1 = 0;
86932 |     ma_uint32 zeroCountParts2 = 0;
86933 |     ma_uint32 zeroCountParts3 = 0;
86934 |     ma_uint32 riceParamParts0 = 0;
86935 |     ma_uint32 riceParamParts1 = 0;
86936 |     ma_uint32 riceParamParts2 = 0;
86937 |     ma_uint32 riceParamParts3 = 0;
86938 |     __m128i coefficients128_0;
86939 |     __m128i coefficients128_4;
86940 |     __m128i coefficients128_8;
86941 |     __m128i samples128_0;
86942 |     __m128i samples128_4;
86943 |     __m128i samples128_8;
86944 |     __m128i prediction128;
86945 |     __m128i riceParamMask128;
86946 |     const ma_uint32 t[2] = {0x00000000, 0xFFFFFFFF};
86947 |     MA_DR_FLAC_ASSERT(order <= 12);
86948 |     riceParamMask = (ma_uint32)~((~0UL) << riceParam);
86949 |     riceParamMask128 = _mm_set1_epi32(riceParamMask);
86950 |     prediction128 = _mm_setzero_si128();
86951 |     coefficients128_0 = _mm_setzero_si128();
86952 |     coefficients128_4 = _mm_setzero_si128();
86953 |     coefficients128_8 = _mm_setzero_si128();
86954 |     samples128_0 = _mm_setzero_si128();
86955 |     samples128_4 = _mm_setzero_si128();
86956 |     samples128_8 = _mm_setzero_si128();
86957 |     #if 1
86958 |     {
86959 |         int runningOrder = order;
86960 |         if (runningOrder >= 4) {
86961 |             coefficients128_0 = _mm_loadu_si128((const __m128i*)(coefficients + 0));
86962 |             samples128_0 = _mm_loadu_si128((const __m128i*)(pSamplesOut - 4));
86963 |             runningOrder -= 4;
86964 |         } else {
86965 |             switch (runningOrder) {
86966 |                 case 3: coefficients128_0 = _mm_set_epi32(0, coefficients[2], coefficients[1],
> coefficients[0]); samples128_0 = _mm_set_epi32(pSamplesOut[-1], pSamplesOut[-2], pSamplesOut[-3],
> 0); break;
86967 |                 case 2: coefficients128_0 = _mm_set_epi32(0, 0, coefficients[1],
> coefficients[0]); samples128_0 = _mm_set_epi32(pSamplesOut[-1], pSamplesOut[-2], 0,
> 0); break;
86968 |                 case 1: coefficients128_0 = _mm_set_epi32(0, 0, 0,
> coefficients[0]); samples128_0 = _mm_set_epi32(pSamplesOut[-1], 0, 0,
> 0); break;
86969 |             }
86970 |             runningOrder = 0;
86971 |         }
86972 |         if (runningOrder >= 4) {
86973 |             coefficients128_4 = _mm_loadu_si128((const __m128i*)(coefficients + 4));
86974 |             samples128_4 = _mm_loadu_si128((const __m128i*)(pSamplesOut - 8));
86975 |             runningOrder -= 4;
86976 |         } else {
86977 |             switch (runningOrder) {
86978 |                 case 3: coefficients128_4 = _mm_set_epi32(0, coefficients[6], coefficients[5],
> coefficients[4]); samples128_4 = _mm_set_epi32(pSamplesOut[-5], pSamplesOut[-6], pSamplesOut[-7],
> 0); break;
86979 |                 case 2: coefficients128_4 = _mm_set_epi32(0, 0, coefficients[5],
> coefficients[4]); samples128_4 = _mm_set_epi32(pSamplesOut[-5], pSamplesOut[-6], 0,
> 0); break;
86980 |                 case 1: coefficients128_4 = _mm_set_epi32(0, 0, 0,

```



```

    > coefficients[4]); samples128_4 = _mm_set_epi32(pSamplesOut[-5], 0,
    > 0); break;
86981 |     }
86982 |     runningOrder = 0;
86983 | }
86984 | if (runningOrder == 4) {
86985 |     coefficients128_8 = _mm_loadu_si128((const __m128i*)(coefficients + 8));
86986 |     samples128_8 = _mm_loadu_si128((const __m128i*)(pSamplesOut - 12));
86987 |     runningOrder -= 4;
86988 | } else {
86989 |     switch (runningOrder) {
86990 |     case 3: coefficients128_8 = _mm_set_epi32(0, coefficients[10], coefficients[9],
    > coefficients[8]); samples128_8 = _mm_set_epi32(pSamplesOut[-9], pSamplesOut[-10], pSamplesOut[-11],
    > 0); break;
86991 |     case 2: coefficients128_8 = _mm_set_epi32(0, 0, coefficients[9],
    > coefficients[8]); samples128_8 = _mm_set_epi32(pSamplesOut[-9], pSamplesOut[-10], 0,
    > 0); break;
86992 |     case 1: coefficients128_8 = _mm_set_epi32(0, 0, 0,
    > coefficients[8]); samples128_8 = _mm_set_epi32(pSamplesOut[-9], 0, 0,
    > 0); break;
86993 |     }
86994 |     runningOrder = 0;
86995 | }
86996 | coefficients128_0 = _mm_shuffle_epi32(coefficients128_0, _MM_SHUFFLE(0, 1, 2, 3));
86997 | coefficients128_4 = _mm_shuffle_epi32(coefficients128_4, _MM_SHUFFLE(0, 1, 2, 3));
86998 | coefficients128_8 = _mm_shuffle_epi32(coefficients128_8, _MM_SHUFFLE(0, 1, 2, 3));
86999 | }
87000 | #else
87001 |     switch (order)
87002 |     {
87003 |     case 12: ((ma_int32*)&coefficients128_8)[0] = coefficients[11]; ((ma_int32*)&samples128_8)[0] =
    > pDecodedSamples[-12];
87004 |     case 11: ((ma_int32*)&coefficients128_8)[1] = coefficients[10]; ((ma_int32*)&samples128_8)[1] =
    > pDecodedSamples[-11];
87005 |     case 10: ((ma_int32*)&coefficients128_8)[2] = coefficients[ 9]; ((ma_int32*)&samples128_8)[2] =
    > pDecodedSamples[-10];
87006 |     case 9: ((ma_int32*)&coefficients128_8)[3] = coefficients[ 8]; ((ma_int32*)&samples128_8)[3] =
    > pDecodedSamples[- 9];
87007 |     case 8: ((ma_int32*)&coefficients128_4)[0] = coefficients[ 7]; ((ma_int32*)&samples128_4)[0] =
    > pDecodedSamples[- 8];
87008 |     case 7: ((ma_int32*)&coefficients128_4)[1] = coefficients[ 6]; ((ma_int32*)&samples128_4)[1] =
    > pDecodedSamples[- 7];
87009 |     case 6: ((ma_int32*)&coefficients128_4)[2] = coefficients[ 5]; ((ma_int32*)&samples128_4)[2] =
    > pDecodedSamples[- 6];
87010 |     case 5: ((ma_int32*)&coefficients128_4)[3] = coefficients[ 4]; ((ma_int32*)&samples128_4)[3] =
    > pDecodedSamples[- 5];
87011 |     case 4: ((ma_int32*)&coefficients128_0)[0] = coefficients[ 3]; ((ma_int32*)&samples128_0)[0] =
    > pDecodedSamples[- 4];
87012 |     case 3: ((ma_int32*)&coefficients128_0)[1] = coefficients[ 2]; ((ma_int32*)&samples128_0)[1] =
    > pDecodedSamples[- 3];
87013 |     case 2: ((ma_int32*)&coefficients128_0)[2] = coefficients[ 1]; ((ma_int32*)&samples128_0)[2] =
    > pDecodedSamples[- 2];
87014 |     case 1: ((ma_int32*)&coefficients128_0)[3] = coefficients[ 0]; ((ma_int32*)&samples128_0)[3] =
    > pDecodedSamples[- 1];
87015 |     }
87016 | #endif
87017 | while (pDecodedSamples < pDecodedSamplesEnd) {
87018 |     __m128i zeroCountPart128;
87019 |     __m128i riceParamPart128;
87020 |     if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts0, &riceParamParts0) ||
87021 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts1, &riceParamParts1) ||
87022 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts2, &riceParamParts2) ||
87023 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts3, &riceParamParts3)) {
87024 |         return MA_FALSE;
87025 |     }
87026 |     zeroCountPart128 = _mm_set_epi32(zeroCountParts3, zeroCountParts2, zeroCountParts1,
    > zeroCountParts0);
87027 |     riceParamPart128 = _mm_set_epi32(riceParamParts3, riceParamParts2, riceParamParts1,
    > riceParamParts0);
87028 |     riceParamPart128 = _mm_and_si128(riceParamPart128, riceParamMask128);
87029 |     riceParamPart128 = _mm_or_si128(riceParamPart128, _mm_slli_epi32(zeroCountPart128,
    > riceParam));
87030 |     riceParamPart128 = _mm_xor_si128(_mm_srli_epi32(riceParamPart128, 1),
    > _mm_add_epi32(ma_dr_flac__mm_not_si128(_mm_and_si128(riceParamPart128, _mm_set1_epi32(1))),
    > _mm_set1_epi32(1)));
87031 |     for (i = 0; i < 4; i += 1) {
87032 |         prediction128 = _mm_xor_si128(prediction128, prediction128);
87033 |         switch (order)
87034 |         {

```

```

87035 |         case 12:
87036 |         case 11: prediction128 = _mm_add_epi64(prediction128,
> _mm_mul_epi32(_mm_shuffle_epi32(coefficients128_8, _MM_SHUFFLE(1, 1, 0, 0)),
> _mm_shuffle_epi32(samples128_8, _MM_SHUFFLE(1, 1, 0, 0))));
87037 |         case 10:
87038 |         case 9: prediction128 = _mm_add_epi64(prediction128,
> _mm_mul_epi32(_mm_shuffle_epi32(coefficients128_8, _MM_SHUFFLE(3, 3, 2, 2)),
> _mm_shuffle_epi32(samples128_8, _MM_SHUFFLE(3, 3, 2, 2))));
87039 |         case 8:
87040 |         case 7: prediction128 = _mm_add_epi64(prediction128,
> _mm_mul_epi32(_mm_shuffle_epi32(coefficients128_4, _MM_SHUFFLE(1, 1, 0, 0)),
> _mm_shuffle_epi32(samples128_4, _MM_SHUFFLE(1, 1, 0, 0))));
87041 |         case 6:
87042 |         case 5: prediction128 = _mm_add_epi64(prediction128,
> _mm_mul_epi32(_mm_shuffle_epi32(coefficients128_4, _MM_SHUFFLE(3, 3, 2, 2)),
> _mm_shuffle_epi32(samples128_4, _MM_SHUFFLE(3, 3, 2, 2))));
87043 |         case 4:
87044 |         case 3: prediction128 = _mm_add_epi64(prediction128,
> _mm_mul_epi32(_mm_shuffle_epi32(coefficients128_0, _MM_SHUFFLE(1, 1, 0, 0)),
> _mm_shuffle_epi32(samples128_0, _MM_SHUFFLE(1, 1, 0, 0))));
87045 |         case 2:
87046 |         case 1: prediction128 = _mm_add_epi64(prediction128,
> _mm_mul_epi32(_mm_shuffle_epi32(coefficients128_0, _MM_SHUFFLE(3, 3, 2, 2)),
> _mm_shuffle_epi32(samples128_0, _MM_SHUFFLE(3, 3, 2, 2))));
87047 |     }
87048 |     prediction128 = ma_dr_flac__mm_hadd_epi64(prediction128);
87049 |     prediction128 = ma_dr_flac__mm_srai_epi64(prediction128, shift);
87050 |     prediction128 = _mm_add_epi32(riceParamPart128, prediction128);
87051 |     samples128_8 = _mm_alignr_epi8(samples128_4, samples128_8, 4);
87052 |     samples128_4 = _mm_alignr_epi8(samples128_0, samples128_4, 4);
87053 |     samples128_0 = _mm_alignr_epi8(prediction128, samples128_0, 4);
87054 |     riceParamPart128 = _mm_alignr_epi8(_mm_setzero_si128(), riceParamPart128, 4);
87055 | }
87056 | _mm_storeu_si128((__m128i*)pDecodedSamples, samples128_0);
87057 | pDecodedSamples += 4;
87058 | }
87059 | i = (count & ~3);
87060 | while (i < (int)count) {
87061 |     if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts0, &riceParamParts0)) {
87062 |         return MA_FALSE;
87063 |     }
87064 |     riceParamParts0 &= riceParamMask;
87065 |     riceParamParts0 |= (zeroCountParts0 << riceParam);
87066 |     riceParamParts0 = (riceParamParts0 >> 1) ^ t[riceParamParts0 & 0x01];
87067 |     pDecodedSamples[0] = riceParamParts0 + ma_dr_flac__calculate_prediction_64(order, shift,
> coefficients, pDecodedSamples);
87068 |     i += 1;
87069 |     pDecodedSamples += 1;
87070 | }
87071 | return MA_TRUE;
87072 | }
87073 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_rice_sse41(ma_dr_flac_bs* bs, ma_uint32
> bitsPerSample, ma_uint32 count, ma_uint8 riceParam, ma_uint32 lpcOrder, ma_int32 lpcShift, ma_uint32
> lpcPrecision, const ma_int32* coefficients, ma_int32* pSamplesOut)
87074 | {
87075 |     MA_DR_FLAC_ASSERT(bs != NULL);
87076 |     MA_DR_FLAC_ASSERT(pSamplesOut != NULL);
87077 |     if (lpcOrder > 0 && lpcOrder <= 12) {
87078 |         if (ma_dr_flac__use_64_bit_prediction(bitsPerSample, lpcOrder, lpcPrecision)) {
87079 |             return ma_dr_flac__decode_samples_with_residual_rice_sse41_64(bs, count, riceParam,
> lpcOrder, lpcShift, coefficients, pSamplesOut);
87080 |         } else {
87081 |             return ma_dr_flac__decode_samples_with_residual_rice_sse41_32(bs, count, riceParam,
> lpcOrder, lpcShift, coefficients, pSamplesOut);
87082 |         }
87083 |     } else {
87084 |         return ma_dr_flac__decode_samples_with_residual_rice_scalar(bs, bitsPerSample, count,
> riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pSamplesOut);
87085 |     }
87086 | }
87087 | #endif
87088 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
87089 | static MA_INLINE void ma_dr_flac__vst2q_s32(ma_int32* p, int32x4x2_t x)
87090 | {
87091 |     vst1q_s32(p+0, x.val[0]);
87092 |     vst1q_s32(p+4, x.val[1]);
87093 | }
87094 | static MA_INLINE void ma_dr_flac__vst2q_u32(ma_uint32* p, uint32x4x2_t x)
87095 | {

```

```

87096 |     vst1q_u32(p+0, x.val[0]);
87097 |     vst1q_u32(p+4, x.val[1]);
87098 | }
87099 | static MA_INLINE void ma_dr_flac__vst2q_f32(float* p, float32x4x2_t x)
87100 | {
87101 |     vst1q_f32(p+0, x.val[0]);
87102 |     vst1q_f32(p+4, x.val[1]);
87103 | }
87104 | static MA_INLINE void ma_dr_flac__vst2q_s16(ma_int16* p, int16x4x2_t x)
87105 | {
87106 |     vst1q_s16(p, vcombine_s16(x.val[0], x.val[1]));
87107 | }
87108 | static MA_INLINE void ma_dr_flac__vst2q_u16(ma_uint16* p, uint16x4x2_t x)
87109 | {
87110 |     vst1q_u16(p, vcombine_u16(x.val[0], x.val[1]));
87111 | }
87112 | static MA_INLINE int32x4_t ma_dr_flac__vdupq_n_s32x4(ma_int32 x3, ma_int32 x2, ma_int32 x1, ma_int32
> x0)
87113 | {
87114 |     ma_int32 x[4];
87115 |     x[3] = x3;
87116 |     x[2] = x2;
87117 |     x[1] = x1;
87118 |     x[0] = x0;
87119 |     return vld1q_s32(x);
87120 | }
87121 | static MA_INLINE int32x4_t ma_dr_flac__valignrq_s32_1(int32x4_t a, int32x4_t b)
87122 | {
87123 |     return vextq_s32(b, a, 1);
87124 | }
87125 | static MA_INLINE uint32x4_t ma_dr_flac__valignrq_u32_1(uint32x4_t a, uint32x4_t b)
87126 | {
87127 |     return vextq_u32(b, a, 1);
87128 | }
87129 | static MA_INLINE int32x2_t ma_dr_flac__vhaddq_s32(int32x4_t x)
87130 | {
87131 |     int32x2_t r = vadd_s32(vget_high_s32(x), vget_low_s32(x));
87132 |     return vpadd_s32(r, r);
87133 | }
87134 | static MA_INLINE int64x1_t ma_dr_flac__vhaddq_s64(int64x2_t x)
87135 | {
87136 |     return vadd_s64(vget_high_s64(x), vget_low_s64(x));
87137 | }
87138 | static MA_INLINE int32x4_t ma_dr_flac__vrevq_s32(int32x4_t x)
87139 | {
87140 |     return vrev64q_s32(vcombine_s32(vget_high_s32(x), vget_low_s32(x)));
87141 | }
87142 | static MA_INLINE int32x4_t ma_dr_flac__vnotq_s32(int32x4_t x)
87143 | {
87144 |     return veorq_s32(x, vdupq_n_s32(0xFFFFFFFF));
87145 | }
87146 | static MA_INLINE uint32x4_t ma_dr_flac__vnotq_u32(uint32x4_t x)
87147 | {
87148 |     return veorq_u32(x, vdupq_n_u32(0xFFFFFFFF));
87149 | }
87150 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_rice_neon_32(ma_dr_flac_bs* bs,
> ma_uint32 count, ma_uint8 riceParam, ma_uint32 order, ma_int32 shift, const ma_int32* coefficients,
> ma_int32* pSamplesOut)
87151 | {
87152 |     int i;
87153 |     ma_uint32 riceParamMask;
87154 |     ma_int32* pDecodedSamples = pSamplesOut;
87155 |     ma_int32* pDecodedSamplesEnd = pSamplesOut + (count & ~3);
87156 |     ma_uint32 zeroCountParts[4];
87157 |     ma_uint32 riceParamParts[4];
87158 |     int32x4_t coefficients128_0;
87159 |     int32x4_t coefficients128_4;
87160 |     int32x4_t coefficients128_8;
87161 |     int32x4_t samples128_0;
87162 |     int32x4_t samples128_4;
87163 |     int32x4_t samples128_8;
87164 |     uint32x4_t riceParamMask128;
87165 |     int32x4_t riceParam128;
87166 |     int32x2_t shift64;
87167 |     uint32x4_t one128;
87168 |     const ma_uint32 t[2] = {0x00000000, 0xFFFFFFFF};
87169 |     riceParamMask = (ma_uint32)~((~0UL) << riceParam);
87170 |     riceParamMask128 = vdupq_n_u32(riceParamMask);
87171 |     riceParam128 = vdupq_n_s32(riceParam);

```

```

87172 | shift64 = vdup_n_s32(-shift);
87173 | one128 = vdupq_n_u32(1);
87174 | {
87175 |     int runningOrder = order;
87176 |     ma_int32 tempC[4] = {0, 0, 0, 0};
87177 |     ma_int32 tempS[4] = {0, 0, 0, 0};
87178 |     if (runningOrder >= 4) {
87179 |         coefficients128_0 = vld1q_s32(coefficients + 0);
87180 |         samples128_0      = vld1q_s32(pSamplesOut - 4);
87181 |         runningOrder -= 4;
87182 |     } else {
87183 |         switch (runningOrder) {
87184 |             case 3: tempC[2] = coefficients[2]; tempS[1] = pSamplesOut[-3];
87185 |             case 2: tempC[1] = coefficients[1]; tempS[2] = pSamplesOut[-2];
87186 |             case 1: tempC[0] = coefficients[0]; tempS[3] = pSamplesOut[-1];
87187 |         }
87188 |         coefficients128_0 = vld1q_s32(tempC);
87189 |         samples128_0      = vld1q_s32(tempS);
87190 |         runningOrder = 0;
87191 |     }
87192 |     if (runningOrder >= 4) {
87193 |         coefficients128_4 = vld1q_s32(coefficients + 4);
87194 |         samples128_4      = vld1q_s32(pSamplesOut - 8);
87195 |         runningOrder -= 4;
87196 |     } else {
87197 |         switch (runningOrder) {
87198 |             case 3: tempC[2] = coefficients[6]; tempS[1] = pSamplesOut[-7];
87199 |             case 2: tempC[1] = coefficients[5]; tempS[2] = pSamplesOut[-6];
87200 |             case 1: tempC[0] = coefficients[4]; tempS[3] = pSamplesOut[-5];
87201 |         }
87202 |         coefficients128_4 = vld1q_s32(tempC);
87203 |         samples128_4      = vld1q_s32(tempS);
87204 |         runningOrder = 0;
87205 |     }
87206 |     if (runningOrder == 4) {
87207 |         coefficients128_8 = vld1q_s32(coefficients + 8);
87208 |         samples128_8      = vld1q_s32(pSamplesOut - 12);
87209 |         runningOrder -= 4;
87210 |     } else {
87211 |         switch (runningOrder) {
87212 |             case 3: tempC[2] = coefficients[10]; tempS[1] = pSamplesOut[-11];
87213 |             case 2: tempC[1] = coefficients[ 9]; tempS[2] = pSamplesOut[-10];
87214 |             case 1: tempC[0] = coefficients[ 8]; tempS[3] = pSamplesOut[- 9];
87215 |         }
87216 |         coefficients128_8 = vld1q_s32(tempC);
87217 |         samples128_8      = vld1q_s32(tempS);
87218 |         runningOrder = 0;
87219 |     }
87220 |     coefficients128_0 = ma_dr_flac__vrevq_s32(coefficients128_0);
87221 |     coefficients128_4 = ma_dr_flac__vrevq_s32(coefficients128_4);
87222 |     coefficients128_8 = ma_dr_flac__vrevq_s32(coefficients128_8);
87223 | }
87224 | while (pDecodedSamples < pDecodedSamplesEnd) {
87225 |     int32x4_t prediction128;
87226 |     int32x2_t prediction64;
87227 |     uint32x4_t zeroCountPart128;
87228 |     uint32x4_t riceParamPart128;
87229 |     if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[0], &riceParamParts[0])
> ||
87230 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[1], &riceParamParts[1])
> ||
87231 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[2], &riceParamParts[2])
> ||
87232 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[3], &riceParamParts[3]))
> {
87233 |         return MA_FALSE;
87234 |     }
87235 |     zeroCountPart128 = vld1q_u32(zeroCountParts);
87236 |     riceParamPart128 = vld1q_u32(riceParamParts);
87237 |     riceParamPart128 = vandq_u32(riceParamPart128, riceParamMask128);
87238 |     riceParamPart128 = vorrq_u32(riceParamPart128, vshlq_u32(zeroCountPart128, riceParam128));
87239 |     riceParamPart128 = veorq_u32(vshrq_n_u32(riceParamPart128, 1),
> vaddq_u32(ma_dr_flac__vnotq_u32(vandq_u32(riceParamPart128, one128)), one128));
87240 |     if (order <= 4) {
87241 |         for (i = 0; i < 4; i += 1) {
87242 |             prediction128 = vmulq_s32(coefficients128_0, samples128_0);
87243 |             prediction64 = ma_dr_flac__vhaddq_s32(prediction128);
87244 |             prediction64 = vshl_s32(prediction64, shift64);
87245 |             prediction64 = vadd_s32(prediction64,

```

```

> vget_low_s32(vreinterpretq_s32_u32(riceParamPart128)));
87246 |         samples128_0 = ma_dr_flac_valignrq_s32_1(vcombine_s32(prediction64, vdup_n_s32(0)),
> samples128_0);
87247 |         riceParamPart128 = ma_dr_flac_valignrq_u32_1(vdupq_n_u32(0), riceParamPart128);
87248 |     }
87249 |     } else if (order <= 8) {
87250 |         for (i = 0; i < 4; i += 1) {
87251 |             prediction128 = vmulq_s32(coefficients128_4, samples128_4);
87252 |             prediction128 = vmlaq_s32(prediction128, coefficients128_0, samples128_0);
87253 |             prediction64 = ma_dr_flac_vhaddq_s32(prediction128);
87254 |             prediction64 = vshl_s32(prediction64, shift64);
87255 |             prediction64 = vadd_s32(prediction64,
> vget_low_s32(vreinterpretq_s32_u32(riceParamPart128)));
87256 |             samples128_4 = ma_dr_flac_valignrq_s32_1(samples128_0, samples128_4);
87257 |             samples128_0 = ma_dr_flac_valignrq_s32_1(vcombine_s32(prediction64, vdup_n_s32(0)),
> samples128_0);
87258 |             riceParamPart128 = ma_dr_flac_valignrq_u32_1(vdupq_n_u32(0), riceParamPart128);
87259 |         }
87260 |     } else {
87261 |         for (i = 0; i < 4; i += 1) {
87262 |             prediction128 = vmulq_s32(coefficients128_8, samples128_8);
87263 |             prediction128 = vmlaq_s32(prediction128, coefficients128_4, samples128_4);
87264 |             prediction128 = vmlaq_s32(prediction128, coefficients128_0, samples128_0);
87265 |             prediction64 = ma_dr_flac_vhaddq_s32(prediction128);
87266 |             prediction64 = vshl_s32(prediction64, shift64);
87267 |             prediction64 = vadd_s32(prediction64,
> vget_low_s32(vreinterpretq_s32_u32(riceParamPart128)));
87268 |             samples128_8 = ma_dr_flac_valignrq_s32_1(samples128_4, samples128_8);
87269 |             samples128_4 = ma_dr_flac_valignrq_s32_1(samples128_0, samples128_4);
87270 |             samples128_0 = ma_dr_flac_valignrq_s32_1(vcombine_s32(prediction64, vdup_n_s32(0)),
> samples128_0);
87271 |             riceParamPart128 = ma_dr_flac_valignrq_u32_1(vdupq_n_u32(0), riceParamPart128);
87272 |         }
87273 |     }
87274 |     vst1q_s32(pDecodedSamples, samples128_0);
87275 |     pDecodedSamples += 4;
87276 | }
87277 | i = (count & ~3);
87278 | while (i < (int)count) {
87279 |     if (!ma_dr_flac_read_rice_parts_x1(bs, riceParam, &zeroCountParts[0], &riceParamParts[0]))
> {
87280 |         return MA_FALSE;
87281 |     }
87282 |     riceParamParts[0] &= riceParamMask;
87283 |     riceParamParts[0] |= (zeroCountParts[0] << riceParam);
87284 |     riceParamParts[0] = (riceParamParts[0] >> 1) ^ t[riceParamParts[0] & 0x01];
87285 |     pDecodedSamples[0] = riceParamParts[0] + ma_dr_flac_calculate_prediction_32(order, shift,
> coefficients, pDecodedSamples);
87286 |     i += 1;
87287 |     pDecodedSamples += 1;
87288 | }
87289 | return MA_TRUE;
87290 | }
87291 | static ma_bool32 ma_dr_flac_decode_samples_with_residual_rice_neon_64(ma_dr_flac_bs* bs,
> ma_uint32 count, ma_uint8 riceParam, ma_uint32 order, ma_int32 shift, const ma_int32* coefficients,
> ma_int32* pSamplesOut)
87292 | {
87293 |     int i;
87294 |     ma_uint32 riceParamMask;
87295 |     ma_int32* pDecodedSamples = pSamplesOut;
87296 |     ma_int32* pDecodedSamplesEnd = pSamplesOut + (count & ~3);
87297 |     ma_uint32 zeroCountParts[4];
87298 |     ma_uint32 riceParamParts[4];
87299 |     int32x4_t coefficients128_0;
87300 |     int32x4_t coefficients128_4;
87301 |     int32x4_t coefficients128_8;
87302 |     int32x4_t samples128_0;
87303 |     int32x4_t samples128_4;
87304 |     int32x4_t samples128_8;
87305 |     uint32x4_t riceParamMask128;
87306 |     int32x4_t riceParam128;
87307 |     int64x1_t shift64;
87308 |     uint32x4_t one128;
87309 |     int64x2_t prediction128 = { 0 };
87310 |     uint32x4_t zeroCountPart128;
87311 |     uint32x4_t riceParamPart128;
87312 |     const ma_uint32 t[2] = {0x00000000, 0xFFFFFFFF};
87313 |     riceParamMask = (ma_uint32)~((~0UL) << riceParam);
87314 |     riceParamMask128 = vdupq_n_u32(riceParamMask);

```

```

87315 | riceParam128 = vdupq_n_s32(riceParam);
87316 | shift64 = vdup_n_s64(-shift);
87317 | one128 = vdupq_n_u32(1);
87318 | {
87319 |     int runningOrder = order;
87320 |     ma_int32 tempC[4] = {0, 0, 0, 0};
87321 |     ma_int32 tempS[4] = {0, 0, 0, 0};
87322 |     if (runningOrder >= 4) {
87323 |         coefficients128_0 = vld1q_s32(coefficients + 0);
87324 |         samples128_0 = vld1q_s32(pSamplesOut - 4);
87325 |         runningOrder -= 4;
87326 |     } else {
87327 |         switch (runningOrder) {
87328 |             case 3: tempC[2] = coefficients[2]; tempS[1] = pSamplesOut[-3];
87329 |             case 2: tempC[1] = coefficients[1]; tempS[2] = pSamplesOut[-2];
87330 |             case 1: tempC[0] = coefficients[0]; tempS[3] = pSamplesOut[-1];
87331 |         }
87332 |         coefficients128_0 = vld1q_s32(tempC);
87333 |         samples128_0 = vld1q_s32(tempS);
87334 |         runningOrder = 0;
87335 |     }
87336 |     if (runningOrder >= 4) {
87337 |         coefficients128_4 = vld1q_s32(coefficients + 4);
87338 |         samples128_4 = vld1q_s32(pSamplesOut - 8);
87339 |         runningOrder -= 4;
87340 |     } else {
87341 |         switch (runningOrder) {
87342 |             case 3: tempC[2] = coefficients[6]; tempS[1] = pSamplesOut[-7];
87343 |             case 2: tempC[1] = coefficients[5]; tempS[2] = pSamplesOut[-6];
87344 |             case 1: tempC[0] = coefficients[4]; tempS[3] = pSamplesOut[-5];
87345 |         }
87346 |         coefficients128_4 = vld1q_s32(tempC);
87347 |         samples128_4 = vld1q_s32(tempS);
87348 |         runningOrder = 0;
87349 |     }
87350 |     if (runningOrder == 4) {
87351 |         coefficients128_8 = vld1q_s32(coefficients + 8);
87352 |         samples128_8 = vld1q_s32(pSamplesOut - 12);
87353 |         runningOrder -= 4;
87354 |     } else {
87355 |         switch (runningOrder) {
87356 |             case 3: tempC[2] = coefficients[10]; tempS[1] = pSamplesOut[-11];
87357 |             case 2: tempC[1] = coefficients[ 9]; tempS[2] = pSamplesOut[-10];
87358 |             case 1: tempC[0] = coefficients[ 8]; tempS[3] = pSamplesOut[- 9];
87359 |         }
87360 |         coefficients128_8 = vld1q_s32(tempC);
87361 |         samples128_8 = vld1q_s32(tempS);
87362 |         runningOrder = 0;
87363 |     }
87364 |     coefficients128_0 = ma_dr_flac_vrevq_s32(coefficients128_0);
87365 |     coefficients128_4 = ma_dr_flac_vrevq_s32(coefficients128_4);
87366 |     coefficients128_8 = ma_dr_flac_vrevq_s32(coefficients128_8);
87367 | }
87368 | while (pDecodedSamples < pDecodedSamplesEnd) {
87369 |     if (!ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[0], &riceParamParts[0])
> ||
87370 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[1], &riceParamParts[1])
> ||
87371 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[2], &riceParamParts[2])
> ||
87372 |         !ma_dr_flac__read_rice_parts_x1(bs, riceParam, &zeroCountParts[3], &riceParamParts[3]))
> {
87373 |         return MA_FALSE;
87374 |     }
87375 |     zeroCountPart128 = vld1q_u32(zeroCountParts);
87376 |     riceParamPart128 = vld1q_u32(riceParamParts);
87377 |     riceParamPart128 = vandq_u32(riceParamPart128, riceParamMask128);
87378 |     riceParamPart128 = vorrq_u32(riceParamPart128, vshlq_u32(zeroCountPart128, riceParam128));
87379 |     riceParamPart128 = veorq_u32(vshrq_n_u32(riceParamPart128, 1),
> vaddq_u32(ma_dr_flac__vnotq_u32(vandq_u32(riceParamPart128, one128)), one128));
87380 |     for (i = 0; i < 4; i += 1) {
87381 |         int64x1_t prediction64;
87382 |         prediction128 = veorq_s64(prediction128, prediction128);
87383 |         switch (order)
87384 |         {
87385 |             case 12:
87386 |             case 11: prediction128 = vaddq_s64(prediction128,
> vmull_s32(vget_low_s32(coefficients128_8), vget_low_s32(samples128_8)));
87387 |             case 10:

```



```

87388 |         case 9: prediction128 = vaddq_s64(prediction128,
> vmull_s32(vget_high_s32(coefficients128_8), vget_high_s32(samples128_8)));
87389 |         case 8:
87390 |         case 7: prediction128 = vaddq_s64(prediction128,
> vmull_s32(vget_low_s32(coefficients128_4), vget_low_s32(samples128_4)));
87391 |         case 6:
87392 |         case 5: prediction128 = vaddq_s64(prediction128,
> vmull_s32(vget_high_s32(coefficients128_4), vget_high_s32(samples128_4)));
87393 |         case 4:
87394 |         case 3: prediction128 = vaddq_s64(prediction128,
> vmull_s32(vget_low_s32(coefficients128_0), vget_low_s32(samples128_0)));
87395 |         case 2:
87396 |         case 1: prediction128 = vaddq_s64(prediction128,
> vmull_s32(vget_high_s32(coefficients128_0), vget_high_s32(samples128_0)));
87397 |     }
87398 |     prediction64 = ma_dr_flac_vhaddq_s64(prediction128);
87399 |     prediction64 = vshl_s64(prediction64, shift64);
87400 |     prediction64 = vadd_s64(prediction64, vdup_n_s64(vgetq_lane_u32(riceParamPart128, 0)));
87401 |     samples128_8 = ma_dr_flac_valignrq_s32_1(samples128_4, samples128_8);
87402 |     samples128_4 = ma_dr_flac_valignrq_s32_1(samples128_0, samples128_4);
87403 |     samples128_0 =
> ma_dr_flac_valignrq_s32_1(vcombine_s32(vreinterpret_s32_s64(prediction64), vdup_n_s32(0)),
> samples128_0);
87404 |     riceParamPart128 = ma_dr_flac_valignrq_u32_1(vdupq_n_u32(0), riceParamPart128);
87405 | }
87406 | vst1q_s32(pDecodedSamples, samples128_0);
87407 | pDecodedSamples += 4;
87408 | }
87409 | i = (count & ~3);
87410 | while (i < (int)count) {
87411 |     if (!ma_dr_flac_read_rice_parts_x1(bs, riceParam, &zeroCountParts[0], &riceParamParts[0]))
> {
87412 |         return MA_FALSE;
87413 |     }
87414 |     riceParamParts[0] &= riceParamMask;
87415 |     riceParamParts[0] |= (zeroCountParts[0] << riceParam);
87416 |     riceParamParts[0] = (riceParamParts[0] >> 1) ^ t[riceParamParts[0] & 0x01];
87417 |     pDecodedSamples[0] = riceParamParts[0] + ma_dr_flac_calculate_prediction_64(order, shift,
> coefficients, pDecodedSamples);
87418 |     i += 1;
87419 |     pDecodedSamples += 1;
87420 | }
87421 | return MA_TRUE;
87422 | }
87423 | static ma_bool32 ma_dr_flac_decode_samples_with_residual_rice_neon(ma_dr_flac_bs* bs, ma_uint32
> bitsPerSample, ma_uint32 count, ma_uint8 riceParam, ma_uint32 lpcOrder, ma_int32 lpcShift, ma_uint32
> lpcPrecision, const ma_int32* coefficients, ma_int32* pSamplesOut)
87424 | {
87425 |     MA_DR_FLAC_ASSERT(bs != NULL);
87426 |     MA_DR_FLAC_ASSERT(pSamplesOut != NULL);
87427 |     if (lpcOrder > 0 && lpcOrder <= 12) {
87428 |         if (ma_dr_flac_use_64_bit_prediction(bitsPerSample, lpcOrder, lpcPrecision)) {
87429 |             return ma_dr_flac_decode_samples_with_residual_rice_neon_64(bs, count, riceParam,
> lpcOrder, lpcShift, coefficients, pSamplesOut);
87430 |         } else {
87431 |             return ma_dr_flac_decode_samples_with_residual_rice_neon_32(bs, count, riceParam,
> lpcOrder, lpcShift, coefficients, pSamplesOut);
87432 |         }
87433 |     } else {
87434 |         return ma_dr_flac_decode_samples_with_residual_rice_scalar(bs, bitsPerSample, count,
> riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pSamplesOut);
87435 |     }
87436 | }
87437 | #endif
87438 | static ma_bool32 ma_dr_flac_decode_samples_with_residual_rice(ma_dr_flac_bs* bs, ma_uint32
> bitsPerSample, ma_uint32 count, ma_uint8 riceParam, ma_uint32 lpcOrder, ma_int32 lpcShift, ma_uint32
> lpcPrecision, const ma_int32* coefficients, ma_int32* pSamplesOut)
87439 | {
87440 |     #if defined(MA_DR_FLAC_SUPPORT_SSE41)
87441 |         if (ma_dr_flac_gIsSSE41Supported) {
87442 |             return ma_dr_flac_decode_samples_with_residual_rice_sse41(bs, bitsPerSample, count,
> riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pSamplesOut);
87443 |         } else
87444 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
87445 |             if (ma_dr_flac_gIsNEONSupported) {
87446 |                 return ma_dr_flac_decode_samples_with_residual_rice_neon(bs, bitsPerSample, count,
> riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pSamplesOut);
87447 |             } else
87448 |             #endif

```



```

87449 |     {
87450 |         #if 0
87451 |             return ma_dr_flac__decode_samples_with_residual_rice_reference(bs, bitsPerSample, count,
> riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pSamplesOut);
87452 |         #else
87453 |             return ma_dr_flac__decode_samples_with_residual_rice_scalar(bs, bitsPerSample, count,
> riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pSamplesOut);
87454 |         #endif
87455 |     }
87456 | }
87457 | static ma_bool32 ma_dr_flac__read_and_seek_residual_rice(ma_dr_flac_bs* bs, ma_uint32 count,
> ma_uint8 riceParam)
87458 | {
87459 |     ma_uint32 i;
87460 |     MA_DR_FLAC_ASSERT(bs != NULL);
87461 |     for (i = 0; i < count; ++i) {
87462 |         if (!ma_dr_flac__seek_rice_parts(bs, riceParam)) {
87463 |             return MA_FALSE;
87464 |         }
87465 |     }
87466 |     return MA_TRUE;
87467 | }
87468 | #if defined(__clang__)
87469 | __attribute__((no_sanitize("signed-integer-overflow")))
87470 | #endif
87471 | static ma_bool32 ma_dr_flac__decode_samples_with_residual_unencoded(ma_dr_flac_bs* bs, ma_uint32
> bitsPerSample, ma_uint32 count, ma_uint8 unencodedBitsPerSample, ma_uint32 lpcOrder, ma_int32
> lpcShift, ma_uint32 lpcPrecision, const ma_int32* coefficients, ma_int32* pSamplesOut)
87472 | {
87473 |     ma_uint32 i;
87474 |     MA_DR_FLAC_ASSERT(bs != NULL);
87475 |     MA_DR_FLAC_ASSERT(unencodedBitsPerSample <= 31);
87476 |     MA_DR_FLAC_ASSERT(pSamplesOut != NULL);
87477 |     for (i = 0; i < count; ++i) {
87478 |         if (unencodedBitsPerSample > 0) {
87479 |             if (!ma_dr_flac__read_int32(bs, unencodedBitsPerSample, pSamplesOut + i)) {
87480 |                 return MA_FALSE;
87481 |             }
87482 |         } else {
87483 |             pSamplesOut[i] = 0;
87484 |         }
87485 |         if (ma_dr_flac__use_64_bit_prediction(bitsPerSample, lpcOrder, lpcPrecision)) {
87486 |             pSamplesOut[i] += ma_dr_flac__calculate_prediction_64(lpcOrder, lpcShift, coefficients,
> pSamplesOut + i);
87487 |         } else {
87488 |             pSamplesOut[i] += ma_dr_flac__calculate_prediction_32(lpcOrder, lpcShift, coefficients,
> pSamplesOut + i);
87489 |         }
87490 |     }
87491 |     return MA_TRUE;
87492 | }
87493 | static ma_bool32 ma_dr_flac__decode_samples_with_residual(ma_dr_flac_bs* bs, ma_uint32
> bitsPerSample, ma_uint32 blockSize, ma_uint32 lpcOrder, ma_int32 lpcShift, ma_uint32 lpcPrecision,
> const ma_int32* coefficients, ma_int32* pDecodedSamples)
87494 | {
87495 |     ma_uint8 residualMethod;
87496 |     ma_uint8 partitionOrder;
87497 |     ma_uint32 samplesInPartition;
87498 |     ma_uint32 partitionsRemaining;
87499 |     MA_DR_FLAC_ASSERT(bs != NULL);
87500 |     MA_DR_FLAC_ASSERT(blockSize != 0);
87501 |     MA_DR_FLAC_ASSERT(pDecodedSamples != NULL);
87502 |     if (!ma_dr_flac__read_uint8(bs, 2, &residualMethod)) {
87503 |         return MA_FALSE;
87504 |     }
87505 |     if (residualMethod != MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE && residualMethod !=
> MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE2) {
87506 |         return MA_FALSE;
87507 |     }
87508 |     pDecodedSamples += lpcOrder;
87509 |     if (!ma_dr_flac__read_uint8(bs, 4, &partitionOrder)) {
87510 |         return MA_FALSE;
87511 |     }
87512 |     if (partitionOrder > 8) {
87513 |         return MA_FALSE;
87514 |     }
87515 |     if ((blockSize / (1 << partitionOrder)) < lpcOrder) {
87516 |         return MA_FALSE;
87517 |     }

```

```

87518 |     samplesInPartition = (blockSize / (1 << partitionOrder)) - lpcOrder;
87519 |     partitionsRemaining = (1 << partitionOrder);
87520 |     for (;;) {
87521 |         ma_uint8 riceParam = 0;
87522 |         if (residualMethod == MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE) {
87523 |             if (!ma_dr_flac__read_uint8(bs, 4, &riceParam)) {
87524 |                 return MA_FALSE;
87525 |             }
87526 |             if (riceParam == 15) {
87527 |                 riceParam = 0xFF;
87528 |             }
87529 |         } else if (residualMethod == MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE2) {
87530 |             if (!ma_dr_flac__read_uint8(bs, 5, &riceParam)) {
87531 |                 return MA_FALSE;
87532 |             }
87533 |             if (riceParam == 31) {
87534 |                 riceParam = 0xFF;
87535 |             }
87536 |         }
87537 |         if (riceParam != 0xFF) {
87538 |             if (!ma_dr_flac__decode_samples_with_residual_rice(bs, bitsPerSample,
> samplesInPartition, riceParam, lpcOrder, lpcShift, lpcPrecision, coefficients, pDecodedSamples)) {
87539 |                 return MA_FALSE;
87540 |             }
87541 |         } else {
87542 |             ma_uint8 unencodedBitsPerSample = 0;
87543 |             if (!ma_dr_flac__read_uint8(bs, 5, &unencodedBitsPerSample)) {
87544 |                 return MA_FALSE;
87545 |             }
87546 |             if (!ma_dr_flac__decode_samples_with_residual_unencoded(bs, bitsPerSample,
> samplesInPartition, unencodedBitsPerSample, lpcOrder, lpcShift, lpcPrecision, coefficients,
> pDecodedSamples)) {
87547 |                 return MA_FALSE;
87548 |             }
87549 |         }
87550 |         pDecodedSamples += samplesInPartition;
87551 |         if (partitionsRemaining == 1) {
87552 |             break;
87553 |         }
87554 |         partitionsRemaining -= 1;
87555 |         if (partitionOrder != 0) {
87556 |             samplesInPartition = blockSize / (1 << partitionOrder);
87557 |         }
87558 |     }
87559 |     return MA_TRUE;
87560 | }
87561 | static ma_bool32 ma_dr_flac__read_and_seek_residual(ma_dr_flac_bs* bs, ma_uint32 blockSize,
> ma_uint32 order)
87562 | {
87563 |     ma_uint8 residualMethod;
87564 |     ma_uint8 partitionOrder;
87565 |     ma_uint32 samplesInPartition;
87566 |     ma_uint32 partitionsRemaining;
87567 |     MA_DR_FLAC_ASSERT(bs != NULL);
87568 |     MA_DR_FLAC_ASSERT(blockSize != 0);
87569 |     if (!ma_dr_flac__read_uint8(bs, 2, &residualMethod)) {
87570 |         return MA_FALSE;
87571 |     }
87572 |     if (residualMethod != MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE && residualMethod !=
> MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE2) {
87573 |         return MA_FALSE;
87574 |     }
87575 |     if (!ma_dr_flac__read_uint8(bs, 4, &partitionOrder)) {
87576 |         return MA_FALSE;
87577 |     }
87578 |     if (partitionOrder > 8) {
87579 |         return MA_FALSE;
87580 |     }
87581 |     if ((blockSize / (1 << partitionOrder)) <= order) {
87582 |         return MA_FALSE;
87583 |     }
87584 |     samplesInPartition = (blockSize / (1 << partitionOrder)) - order;
87585 |     partitionsRemaining = (1 << partitionOrder);
87586 |     for (;;) {
87587 |     {
87588 |         ma_uint8 riceParam = 0;
87589 |         if (residualMethod == MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE) {
87590 |             if (!ma_dr_flac__read_uint8(bs, 4, &riceParam)) {
87591 |                 return MA_FALSE;

```

```

87592 |     }
87593 |     if (riceParam == 15) {
87594 |         riceParam = 0xFF;
87595 |     }
87596 | } else if (residualMethod == MA_DR_FLAC_RESIDUAL_CODING_METHOD_PARTITIONED_RICE2) {
87597 |     if (!ma_dr_flac__read_uint8(bs, 5, &riceParam)) {
87598 |         return MA_FALSE;
87599 |     }
87600 |     if (riceParam == 31) {
87601 |         riceParam = 0xFF;
87602 |     }
87603 | }
87604 | if (riceParam != 0xFF) {
87605 |     if (!ma_dr_flac__read_and_seek_residual__rice(bs, samplesInPartition, riceParam)) {
87606 |         return MA_FALSE;
87607 |     }
87608 | } else {
87609 |     ma_uint8 unencodedBitsPerSample = 0;
87610 |     if (!ma_dr_flac__read_uint8(bs, 5, &unencodedBitsPerSample)) {
87611 |         return MA_FALSE;
87612 |     }
87613 |     if (!ma_dr_flac__seek_bits(bs, unencodedBitsPerSample * samplesInPartition)) {
87614 |         return MA_FALSE;
87615 |     }
87616 | }
87617 | if (partitionsRemaining == 1) {
87618 |     break;
87619 | }
87620 | partitionsRemaining -= 1;
87621 | samplesInPartition = blockSize / (1 << partitionOrder);
87622 | }
87623 | return MA_TRUE;
87624 | }
87625 | static ma_bool32 ma_dr_flac__decode_samples__constant(ma_dr_flac_bs* bs, ma_uint32 blockSize,
> ma_uint32 subframeBitsPerSample, ma_int32* pDecodedSamples)
87626 | {
87627 |     ma_uint32 i;
87628 |     ma_int32 sample;
87629 |     if (!ma_dr_flac__read_int32(bs, subframeBitsPerSample, &sample)) {
87630 |         return MA_FALSE;
87631 |     }
87632 |     for (i = 0; i < blockSize; ++i) {
87633 |         pDecodedSamples[i] = sample;
87634 |     }
87635 |     return MA_TRUE;
87636 | }
87637 | static ma_bool32 ma_dr_flac__decode_samples__verbatim(ma_dr_flac_bs* bs, ma_uint32 blockSize,
> ma_uint32 subframeBitsPerSample, ma_int32* pDecodedSamples)
87638 | {
87639 |     ma_uint32 i;
87640 |     for (i = 0; i < blockSize; ++i) {
87641 |         ma_int32 sample;
87642 |         if (!ma_dr_flac__read_int32(bs, subframeBitsPerSample, &sample)) {
87643 |             return MA_FALSE;
87644 |         }
87645 |         pDecodedSamples[i] = sample;
87646 |     }
87647 |     return MA_TRUE;
87648 | }
87649 | static ma_bool32 ma_dr_flac__decode_samples__fixed(ma_dr_flac_bs* bs, ma_uint32 blockSize, ma_uint32
> subframeBitsPerSample, ma_uint8 lpcOrder, ma_int32* pDecodedSamples)
87650 | {
87651 |     ma_uint32 i;
87652 |     static ma_int32 lpcCoefficientsTable[5][4] = {
87653 |         {0, 0, 0, 0},
87654 |         {1, 0, 0, 0},
87655 |         {2, -1, 0, 0},
87656 |         {3, -3, 1, 0},
87657 |         {4, -6, 4, -1}
87658 |     };
87659 |     for (i = 0; i < lpcOrder; ++i) {
87660 |         ma_int32 sample;
87661 |         if (!ma_dr_flac__read_int32(bs, subframeBitsPerSample, &sample)) {
87662 |             return MA_FALSE;
87663 |         }
87664 |         pDecodedSamples[i] = sample;
87665 |     }
87666 |     if (!ma_dr_flac__decode_samples_with_residual(bs, subframeBitsPerSample, blockSize, lpcOrder, 0,
> 4, lpcCoefficientsTable[lpcOrder], pDecodedSamples)) {

```

```

87667 |         return MA_FALSE;
87668 |     }
87669 |     return MA_TRUE;
87670 | }
87671 | static ma_bool32 ma_dr_flac_decode_samples_lpc(ma_dr_flac_bs* bs, ma_uint32 blockSize, ma_uint32
> bitsPerSample, ma_uint8 lpcOrder, ma_int32* pDecodedSamples)
87672 | {
87673 |     ma_uint8 i;
87674 |     ma_uint8 lpcPrecision;
87675 |     ma_int8 lpcShift;
87676 |     ma_int32 coefficients[32];
87677 |     for (i = 0; i < lpcOrder; ++i) {
87678 |         ma_int32 sample;
87679 |         if (!ma_dr_flac_read_int32(bs, bitsPerSample, &sample)) {
87680 |             return MA_FALSE;
87681 |         }
87682 |         pDecodedSamples[i] = sample;
87683 |     }
87684 |     if (!ma_dr_flac_read_uint8(bs, 4, &lpcPrecision)) {
87685 |         return MA_FALSE;
87686 |     }
87687 |     if (lpcPrecision == 15) {
87688 |         return MA_FALSE;
87689 |     }
87690 |     lpcPrecision += 1;
87691 |     if (!ma_dr_flac_read_int8(bs, 5, &lpcShift)) {
87692 |         return MA_FALSE;
87693 |     }
87694 |     if (lpcShift < 0) {
87695 |         return MA_FALSE;
87696 |     }
87697 |     MA_DR_FLAC_ZERO_MEMORY(coefficients, sizeof(coefficients));
87698 |     for (i = 0; i < lpcOrder; ++i) {
87699 |         if (!ma_dr_flac_read_int32(bs, lpcPrecision, coefficients + i)) {
87700 |             return MA_FALSE;
87701 |         }
87702 |     }
87703 |     if (!ma_dr_flac_decode_samples_with_residual(bs, bitsPerSample, blockSize, lpcOrder, lpcShift,
> lpcPrecision, coefficients, pDecodedSamples)) {
87704 |         return MA_FALSE;
87705 |     }
87706 |     return MA_TRUE;
87707 | }
87708 | static ma_bool32 ma_dr_flac_read_next_flac_frame_header(ma_dr_flac_bs* bs, ma_uint8
> streaminfoBitsPerSample, ma_dr_flac_frame_header* header)
87709 | {
87710 |     const ma_uint32 sampleRateTable[12] = {0, 88200, 176400, 192000, 8000, 16000, 22050, 24000,
> 32000, 44100, 48000, 96000};
87711 |     const ma_uint8 bitsPerSampleTable[8] = {0, 8, 12, (ma_uint8)-1, 16, 20, 24, (ma_uint8)-1};
87712 |     MA_DR_FLAC_ASSERT(bs != NULL);
87713 |     MA_DR_FLAC_ASSERT(header != NULL);
87714 |     for (;;) {
87715 |         ma_uint8 crc8 = 0xCE;
87716 |         ma_uint8 reserved = 0;
87717 |         ma_uint8 blockingStrategy = 0;
87718 |         ma_uint8 blockSize = 0;
87719 |         ma_uint8 sampleRate = 0;
87720 |         ma_uint8 channelAssignment = 0;
87721 |         ma_uint8 bitsPerSample = 0;
87722 |         ma_bool32 isVariableBlockSize;
87723 |         if (!ma_dr_flac_find_and_seek_to_next_sync_code(bs)) {
87724 |             return MA_FALSE;
87725 |         }
87726 |         if (!ma_dr_flac_read_uint8(bs, 1, &reserved)) {
87727 |             return MA_FALSE;
87728 |         }
87729 |         if (reserved == 1) {
87730 |             continue;
87731 |         }
87732 |         crc8 = ma_dr_flac_crc8(crc8, reserved, 1);
87733 |         if (!ma_dr_flac_read_uint8(bs, 1, &blockingStrategy)) {
87734 |             return MA_FALSE;
87735 |         }
87736 |         crc8 = ma_dr_flac_crc8(crc8, blockingStrategy, 1);
87737 |         if (!ma_dr_flac_read_uint8(bs, 4, &blockSize)) {
87738 |             return MA_FALSE;
87739 |         }
87740 |         if (blockSize == 0) {
87741 |             continue;

```

```

87742 |     }
87743 |     crc8 = ma_dr_flac_crc8(crc8, blockSize, 4);
87744 |     if (!ma_dr_flac__read_uint8(bs, 4, &sampleRate)) {
87745 |         return MA_FALSE;
87746 |     }
87747 |     crc8 = ma_dr_flac_crc8(crc8, sampleRate, 4);
87748 |     if (!ma_dr_flac__read_uint8(bs, 4, &channelAssignment)) {
87749 |         return MA_FALSE;
87750 |     }
87751 |     if (channelAssignment > 10) {
87752 |         continue;
87753 |     }
87754 |     crc8 = ma_dr_flac_crc8(crc8, channelAssignment, 4);
87755 |     if (!ma_dr_flac__read_uint8(bs, 3, &bitsPerSample)) {
87756 |         return MA_FALSE;
87757 |     }
87758 |     if (bitsPerSample == 3 || bitsPerSample == 7) {
87759 |         continue;
87760 |     }
87761 |     crc8 = ma_dr_flac_crc8(crc8, bitsPerSample, 3);
87762 |     if (!ma_dr_flac__read_uint8(bs, 1, &reserved)) {
87763 |         return MA_FALSE;
87764 |     }
87765 |     if (reserved == 1) {
87766 |         continue;
87767 |     }
87768 |     crc8 = ma_dr_flac_crc8(crc8, reserved, 1);
87769 |     isVariableBlockSize = blockingStrategy == 1;
87770 |     if (isVariableBlockSize) {
87771 |         ma_uint64 pcmFrameNumber;
87772 |         ma_result result = ma_dr_flac__read_utf8_coded_number(bs, &pcmFrameNumber, &crc8);
87773 |         if (result != MA_SUCCESS) {
87774 |             if (result == MA_AT_END) {
87775 |                 return MA_FALSE;
87776 |             } else {
87777 |                 continue;
87778 |             }
87779 |         }
87780 |         header->flacFrameNumber = 0;
87781 |         header->pcmFrameNumber = pcmFrameNumber;
87782 |     } else {
87783 |         ma_uint64 flacFrameNumber = 0;
87784 |         ma_result result = ma_dr_flac__read_utf8_coded_number(bs, &flacFrameNumber, &crc8);
87785 |         if (result != MA_SUCCESS) {
87786 |             if (result == MA_AT_END) {
87787 |                 return MA_FALSE;
87788 |             } else {
87789 |                 continue;
87790 |             }
87791 |         }
87792 |         header->flacFrameNumber = (ma_uint32)flacFrameNumber;
87793 |         header->pcmFrameNumber = 0;
87794 |     }
87795 |     MA_DR_FLAC_ASSERT(blockSize > 0);
87796 |     if (blockSize == 1) {
87797 |         header->blockSizeInPCMFrames = 192;
87798 |     } else if (blockSize <= 5) {
87799 |         MA_DR_FLAC_ASSERT(blockSize >= 2);
87800 |         header->blockSizeInPCMFrames = 576 * (1 << (blockSize - 2));
87801 |     } else if (blockSize == 6) {
87802 |         if (!ma_dr_flac__read_uint16(bs, 8, &header->blockSizeInPCMFrames)) {
87803 |             return MA_FALSE;
87804 |         }
87805 |         crc8 = ma_dr_flac_crc8(crc8, header->blockSizeInPCMFrames, 8);
87806 |         header->blockSizeInPCMFrames += 1;
87807 |     } else if (blockSize == 7) {
87808 |         if (!ma_dr_flac__read_uint16(bs, 16, &header->blockSizeInPCMFrames)) {
87809 |             return MA_FALSE;
87810 |         }
87811 |         crc8 = ma_dr_flac_crc8(crc8, header->blockSizeInPCMFrames, 16);
87812 |         if (header->blockSizeInPCMFrames == 0xFFFF) {
87813 |             return MA_FALSE;
87814 |         }
87815 |         header->blockSizeInPCMFrames += 1;
87816 |     } else {
87817 |         MA_DR_FLAC_ASSERT(blockSize >= 8);
87818 |         header->blockSizeInPCMFrames = 256 * (1 << (blockSize - 8));
87819 |     }
87820 |     if (sampleRate <= 11) {

```

```

87821 |         header->sampleRate = sampleRateTable[sampleRate];
87822 |     } else if (sampleRate == 12) {
87823 |         if (!ma_dr_flac__read_uint32(bs, 8, &header->sampleRate)) {
87824 |             return MA_FALSE;
87825 |         }
87826 |         crc8 = ma_dr_flac_crc8(crc8, header->sampleRate, 8);
87827 |         header->sampleRate *= 1000;
87828 |     } else if (sampleRate == 13) {
87829 |         if (!ma_dr_flac__read_uint32(bs, 16, &header->sampleRate)) {
87830 |             return MA_FALSE;
87831 |         }
87832 |         crc8 = ma_dr_flac_crc8(crc8, header->sampleRate, 16);
87833 |     } else if (sampleRate == 14) {
87834 |         if (!ma_dr_flac__read_uint32(bs, 16, &header->sampleRate)) {
87835 |             return MA_FALSE;
87836 |         }
87837 |         crc8 = ma_dr_flac_crc8(crc8, header->sampleRate, 16);
87838 |         header->sampleRate *= 10;
87839 |     } else {
87840 |         continue;
87841 |     }
87842 |     header->channelAssignment = channelAssignment;
87843 |     header->bitsPerSample = bitsPerSampleTable[bitsPerSample];
87844 |     if (header->bitsPerSample == 0) {
87845 |         header->bitsPerSample = streaminfoBitsPerSample;
87846 |     }
87847 |     if (header->bitsPerSample != streaminfoBitsPerSample) {
87848 |         return MA_FALSE;
87849 |     }
87850 |     if (!ma_dr_flac__read_uint8(bs, 8, &header->crc8)) {
87851 |         return MA_FALSE;
87852 |     }
87853 | #ifndef MA_DR_FLAC_NO_CRC
87854 |     if (header->crc8 != crc8) {
87855 |         continue;
87856 |     }
87857 | #endif
87858 |     return MA_TRUE;
87859 | }
87860 | }
87861 | static ma_bool32 ma_dr_flac__read_subframe_header(ma_dr_flac_bs* bs, ma_dr_flac_subframe* pSubframe)
87862 | {
87863 |     ma_uint8 header;
87864 |     int type;
87865 |     if (!ma_dr_flac__read_uint8(bs, 8, &header)) {
87866 |         return MA_FALSE;
87867 |     }
87868 |     if ((header & 0x80) != 0) {
87869 |         return MA_FALSE;
87870 |     }
87871 |     pSubframe->lpcOrder = 0;
87872 |     type = (header & 0x7E) >> 1;
87873 |     if (type == 0) {
87874 |         pSubframe->subframeType = MA_DR_FLAC_SUBFRAME_CONSTANT;
87875 |     } else if (type == 1) {
87876 |         pSubframe->subframeType = MA_DR_FLAC_SUBFRAME_VERBATIM;
87877 |     } else {
87878 |         if ((type & 0x20) != 0) {
87879 |             pSubframe->subframeType = MA_DR_FLAC_SUBFRAME_LPC;
87880 |             pSubframe->lpcOrder = (ma_uint8)(type & 0x1F) + 1;
87881 |         } else if ((type & 0x08) != 0) {
87882 |             pSubframe->subframeType = MA_DR_FLAC_SUBFRAME_FIXED;
87883 |             pSubframe->lpcOrder = (ma_uint8)(type & 0x07);
87884 |             if (pSubframe->lpcOrder > 4) {
87885 |                 pSubframe->subframeType = MA_DR_FLAC_SUBFRAME_RESERVED;
87886 |                 pSubframe->lpcOrder = 0;
87887 |             }
87888 |         } else {
87889 |             pSubframe->subframeType = MA_DR_FLAC_SUBFRAME_RESERVED;
87890 |         }
87891 |     }
87892 |     if (pSubframe->subframeType == MA_DR_FLAC_SUBFRAME_RESERVED) {
87893 |         return MA_FALSE;
87894 |     }
87895 |     pSubframe->wastedBitsPerSample = 0;
87896 |     if ((header & 0x01) == 1) {
87897 |         unsigned int wastedBitsPerSample;
87898 |         if (!ma_dr_flac__seek_past_next_set_bit(bs, &wastedBitsPerSample)) {
87899 |             return MA_FALSE;

```

```

87900 |     }
87901 |     pSubframe->wastedBitsPerSample = (ma_uint8)wastedBitsPerSample + 1;
87902 | }
87903 | return MA_TRUE;
87904 | }
87905 | static ma_bool32 ma_dr_flac_decode_subframe(ma_dr_flac_bs* bs, ma_dr_flac_frame* frame, int
> subframeIndex, ma_int32* pDecodedSamplesOut)
87906 | {
87907 |     ma_dr_flac_subframe* pSubframe;
87908 |     ma_uint32 subframeBitsPerSample;
87909 |     MA_DR_FLAC_ASSERT(bs != NULL);
87910 |     MA_DR_FLAC_ASSERT(frame != NULL);
87911 |     pSubframe = frame->subframes + subframeIndex;
87912 |     if (!ma_dr_flac__read_subframe_header(bs, pSubframe)) {
87913 |         return MA_FALSE;
87914 |     }
87915 |     subframeBitsPerSample = frame->header.bitsPerSample;
87916 |     if ((frame->header.channelAssignment == MA_DR_FLAC_CHANNEL_ASSIGNMENT_LEFT_SIDE ||
> frame->header.channelAssignment == MA_DR_FLAC_CHANNEL_ASSIGNMENT_MID_SIDE) && subframeIndex == 1) {
87917 |         subframeBitsPerSample += 1;
87918 |     } else if (frame->header.channelAssignment == MA_DR_FLAC_CHANNEL_ASSIGNMENT_RIGHT_SIDE &&
> subframeIndex == 0) {
87919 |         subframeBitsPerSample += 1;
87920 |     }
87921 |     if (subframeBitsPerSample > 32) {
87922 |         return MA_FALSE;
87923 |     }
87924 |     if (pSubframe->wastedBitsPerSample >= subframeBitsPerSample) {
87925 |         return MA_FALSE;
87926 |     }
87927 |     subframeBitsPerSample -= pSubframe->wastedBitsPerSample;
87928 |     pSubframe->pSamplesS32 = pDecodedSamplesOut;
87929 |     if (frame->header.blockSizeInPCMFrames < pSubframe->lpcOrder) {
87930 |         return MA_FALSE;
87931 |     }
87932 |     switch (pSubframe->subframeType)
87933 |     {
87934 |         case MA_DR_FLAC_SUBFRAME_CONSTANT:
87935 |         {
87936 |             ma_dr_flac__decode_samples__constant(bs, frame->header.blockSizeInPCMFrames,
> subframeBitsPerSample, pSubframe->pSamplesS32);
87937 |         } break;
87938 |         case MA_DR_FLAC_SUBFRAME_VERBATIM:
87939 |         {
87940 |             ma_dr_flac__decode_samples__verbatim(bs, frame->header.blockSizeInPCMFrames,
> subframeBitsPerSample, pSubframe->pSamplesS32);
87941 |         } break;
87942 |         case MA_DR_FLAC_SUBFRAME_FIXED:
87943 |         {
87944 |             ma_dr_flac__decode_samples__fixed(bs, frame->header.blockSizeInPCMFrames,
> subframeBitsPerSample, pSubframe->lpcOrder, pSubframe->pSamplesS32);
87945 |         } break;
87946 |         case MA_DR_FLAC_SUBFRAME_LPC:
87947 |         {
87948 |             ma_dr_flac__decode_samples__lpc(bs, frame->header.blockSizeInPCMFrames,
> subframeBitsPerSample, pSubframe->lpcOrder, pSubframe->pSamplesS32);
87949 |         } break;
87950 |         default: return MA_FALSE;
87951 |     }
87952 |     return MA_TRUE;
87953 | }
87954 | static ma_bool32 ma_dr_flac_seek_subframe(ma_dr_flac_bs* bs, ma_dr_flac_frame* frame, int
> subframeIndex)
87955 | {
87956 |     ma_dr_flac_subframe* pSubframe;
87957 |     ma_uint32 subframeBitsPerSample;
87958 |     MA_DR_FLAC_ASSERT(bs != NULL);
87959 |     MA_DR_FLAC_ASSERT(frame != NULL);
87960 |     pSubframe = frame->subframes + subframeIndex;
87961 |     if (!ma_dr_flac__read_subframe_header(bs, pSubframe)) {
87962 |         return MA_FALSE;
87963 |     }
87964 |     subframeBitsPerSample = frame->header.bitsPerSample;
87965 |     if ((frame->header.channelAssignment == MA_DR_FLAC_CHANNEL_ASSIGNMENT_LEFT_SIDE ||
> frame->header.channelAssignment == MA_DR_FLAC_CHANNEL_ASSIGNMENT_MID_SIDE) && subframeIndex == 1) {
87966 |         subframeBitsPerSample += 1;
87967 |     } else if (frame->header.channelAssignment == MA_DR_FLAC_CHANNEL_ASSIGNMENT_RIGHT_SIDE &&
> subframeIndex == 0) {
87968 |         subframeBitsPerSample += 1;

```



```

87969 |     }
87970 |     if (pSubframe->wastedBitsPerSample >= subframeBitsPerSample) {
87971 |         return MA_FALSE;
87972 |     }
87973 |     subframeBitsPerSample -= pSubframe->wastedBitsPerSample;
87974 |     pSubframe->pSamplesS32 = NULL;
87975 |     switch (pSubframe->subframeType)
87976 |     {
87977 |         case MA_DR_FLAC_SUBFRAME_CONSTANT:
87978 |         {
87979 |             if (!ma_dr_flac__seek_bits(bs, subframeBitsPerSample)) {
87980 |                 return MA_FALSE;
87981 |             }
87982 |         } break;
87983 |         case MA_DR_FLAC_SUBFRAME_VERBATIM:
87984 |         {
87985 |             unsigned int bitsToSeek = frame->header.blockSizeInPCMFrames * subframeBitsPerSample;
87986 |             if (!ma_dr_flac__seek_bits(bs, bitsToSeek)) {
87987 |                 return MA_FALSE;
87988 |             }
87989 |         } break;
87990 |         case MA_DR_FLAC_SUBFRAME_FIXED:
87991 |         {
87992 |             unsigned int bitsToSeek = pSubframe->lpcOrder * subframeBitsPerSample;
87993 |             if (!ma_dr_flac__seek_bits(bs, bitsToSeek)) {
87994 |                 return MA_FALSE;
87995 |             }
87996 |             if (!ma_dr_flac__read_and_seek_residual(bs, frame->header.blockSizeInPCMFrames,
> pSubframe->lpcOrder)) {
87997 |                 return MA_FALSE;
87998 |             }
87999 |         } break;
88000 |         case MA_DR_FLAC_SUBFRAME_LPC:
88001 |         {
88002 |             ma_uint8 lpcPrecision;
88003 |             unsigned int bitsToSeek = pSubframe->lpcOrder * subframeBitsPerSample;
88004 |             if (!ma_dr_flac__seek_bits(bs, bitsToSeek)) {
88005 |                 return MA_FALSE;
88006 |             }
88007 |             if (!ma_dr_flac__read_uint8(bs, 4, &lpcPrecision)) {
88008 |                 return MA_FALSE;
88009 |             }
88010 |             if (lpcPrecision == 15) {
88011 |                 return MA_FALSE;
88012 |             }
88013 |             lpcPrecision += 1;
88014 |             bitsToSeek = (pSubframe->lpcOrder * lpcPrecision) + 5;
88015 |             if (!ma_dr_flac__seek_bits(bs, bitsToSeek)) {
88016 |                 return MA_FALSE;
88017 |             }
88018 |             if (!ma_dr_flac__read_and_seek_residual(bs, frame->header.blockSizeInPCMFrames,
> pSubframe->lpcOrder)) {
88019 |                 return MA_FALSE;
88020 |             }
88021 |         } break;
88022 |         default: return MA_FALSE;
88023 |     }
88024 |     return MA_TRUE;
88025 | }
88026 | static MA_INLINE ma_uint8 ma_dr_flac__get_channel_count_from_channel_assignment(ma_int8
> channelAssignment)
88027 | {
88028 |     ma_uint8 lookup[] = {1, 2, 3, 4, 5, 6, 7, 8, 2, 2, 2};
88029 |     MA_DR_FLAC_ASSERT(channelAssignment <= 10);
88030 |     return lookup[channelAssignment];
88031 | }
88032 | static ma_result ma_dr_flac__decode_flac_frame(ma_dr_flac* pFlac)
88033 | {
88034 |     int channelCount;
88035 |     int i;
88036 |     ma_uint8 paddingSizeInBits;
88037 |     ma_uint16 desiredCRC16;
88038 |     #ifndef MA_DR_FLAC_NO_CRC
88039 |     ma_uint16 actualCRC16;
88040 |     #endif
88041 |     MA_DR_FLAC_ZERO_MEMORY(pFlac->currentFLACFrame.subframes,
> sizeof(pFlac->currentFLACFrame.subframes));
88042 |     if (pFlac->currentFLACFrame.header.blockSizeInPCMFrames > pFlac->maxBlockSizeInPCMFrames) {
88043 |         return MA_ERROR;

```

```

88044 |     }
88045 |     channelCount = ma_dr_flac__get_channel_count_from_channel_assignment(pFlac->currentFLACFrame.hea
> der.channelAssignment);
88046 |     if (channelCount != (int)pFlac->channels) {
88047 |         return MA_ERROR;
88048 |     }
88049 |     for (i = 0; i < channelCount; ++i) {
88050 |         if (!ma_dr_flac__decode_subframe(&pFlac->bs, &pFlac->currentFLACFrame, i,
> pFlac->pDecodedSamples + (pFlac->currentFLACFrame.header.blockSizeInPCMFrames * i))) {
88051 |             return MA_ERROR;
88052 |         }
88053 |     }
88054 |     paddingSizeInBits = (ma_uint8)(MA_DR_FLAC_CACHE_L1_BITS_REMAINING(&pFlac->bs) & 7);
88055 |     if (paddingSizeInBits > 0) {
88056 |         ma_uint8 padding = 0;
88057 |         if (!ma_dr_flac__read_uint8(&pFlac->bs, paddingSizeInBits, &padding)) {
88058 |             return MA_AT_END;
88059 |         }
88060 |     }
88061 | #ifndef MA_DR_FLAC_NO_CRC
88062 |     actualCRC16 = ma_dr_flac__flush_crc16(&pFlac->bs);
88063 | #endif
88064 |     if (!ma_dr_flac__read_uint16(&pFlac->bs, 16, &desiredCRC16)) {
88065 |         return MA_AT_END;
88066 |     }
88067 | #ifndef MA_DR_FLAC_NO_CRC
88068 |     if (actualCRC16 != desiredCRC16) {
88069 |         return MA_CRC_MISMATCH;
88070 |     }
88071 | #endif
88072 |     pFlac->currentFLACFrame.pcmFramesRemaining =
> pFlac->currentFLACFrame.header.blockSizeInPCMFrames;
88073 |     return MA_SUCCESS;
88074 | }
88075 | static ma_result ma_dr_flac__seek_flac_frame(ma_dr_flac* pFlac)
88076 | {
88077 |     int channelCount;
88078 |     int i;
88079 |     ma_uint16 desiredCRC16;
88080 | #ifndef MA_DR_FLAC_NO_CRC
88081 |     ma_uint16 actualCRC16;
88082 | #endif
88083 |     channelCount = ma_dr_flac__get_channel_count_from_channel_assignment(pFlac->currentFLACFrame.hea
> der.channelAssignment);
88084 |     for (i = 0; i < channelCount; ++i) {
88085 |         if (!ma_dr_flac__seek_subframe(&pFlac->bs, &pFlac->currentFLACFrame, i)) {
88086 |             return MA_ERROR;
88087 |         }
88088 |     }
88089 |     if (!ma_dr_flac__seek_bits(&pFlac->bs, MA_DR_FLAC_CACHE_L1_BITS_REMAINING(&pFlac->bs) & 7)) {
88090 |         return MA_ERROR;
88091 |     }
88092 | #ifndef MA_DR_FLAC_NO_CRC
88093 |     actualCRC16 = ma_dr_flac__flush_crc16(&pFlac->bs);
88094 | #endif
88095 |     if (!ma_dr_flac__read_uint16(&pFlac->bs, 16, &desiredCRC16)) {
88096 |         return MA_AT_END;
88097 |     }
88098 | #ifndef MA_DR_FLAC_NO_CRC
88099 |     if (actualCRC16 != desiredCRC16) {
88100 |         return MA_CRC_MISMATCH;
88101 |     }
88102 | #endif
88103 |     return MA_SUCCESS;
88104 | }
88105 | static ma_bool32 ma_dr_flac__read_and_decode_next_flac_frame(ma_dr_flac* pFlac)
88106 | {
88107 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88108 |     for (;;) {
88109 |         ma_result result;
88110 |         if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88111 |             return MA_FALSE;
88112 |         }
88113 |         result = ma_dr_flac__decode_flac_frame(pFlac);
88114 |         if (result != MA_SUCCESS) {
88115 |             if (result == MA_CRC_MISMATCH) {
88116 |                 continue;
88117 |             } else {

```

```

88118 |         return MA_FALSE;
88119 |     }
88120 | }
88121 |     return MA_TRUE;
88122 | }
88123 | }
88124 | static void ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(ma_dr_flac* pFlac, ma_uint64*
> pFirstPCMFrame, ma_uint64* pLastPCMFrame)
88125 | {
88126 |     ma_uint64 firstPCMFrame;
88127 |     ma_uint64 lastPCMFrame;
88128 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88129 |     firstPCMFrame = pFlac->currentFLACFrame.header.pcmFrameNumber;
88130 |     if (firstPCMFrame == 0) {
88131 |         firstPCMFrame = ((ma_uint64)pFlac->currentFLACFrame.header.flacFrameNumber) *
> pFlac->maxBlockSizeInPCMFrames;
88132 |     }
88133 |     lastPCMFrame = firstPCMFrame + pFlac->currentFLACFrame.header.blockSizeInPCMFrames;
88134 |     if (lastPCMFrame > 0) {
88135 |         lastPCMFrame -= 1;
88136 |     }
88137 |     if (pFirstPCMFrame) {
88138 |         *pFirstPCMFrame = firstPCMFrame;
88139 |     }
88140 |     if (pLastPCMFrame) {
88141 |         *pLastPCMFrame = lastPCMFrame;
88142 |     }
88143 | }
88144 | static ma_bool32 ma_dr_flac__seek_to_first_frame(ma_dr_flac* pFlac)
88145 | {
88146 |     ma_bool32 result;
88147 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88148 |     result = ma_dr_flac__seek_to_byte(&pFlac->bs, pFlac->firstFLACFramePosInBytes);
88149 |     MA_DR_FLAC_ZERO_MEMORY(&pFlac->currentFLACFrame, sizeof(pFlac->currentFLACFrame));
88150 |     pFlac->currentPCMFrame = 0;
88151 |     return result;
88152 | }
88153 | static MA_INLINE ma_result ma_dr_flac__seek_to_next_flac_frame(ma_dr_flac* pFlac)
88154 | {
88155 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88156 |     return ma_dr_flac__seek_flac_frame(pFlac);
88157 | }
88158 | static ma_uint64 ma_dr_flac__seek_forward_by_pcm_frames(ma_dr_flac* pFlac, ma_uint64
> pcmFramesToSeek)
88159 | {
88160 |     ma_uint64 pcmFramesRead = 0;
88161 |     while (pcmFramesToSeek > 0) {
88162 |         if (pFlac->currentFLACFrame.pcmFramesRemaining == 0) {
88163 |             if (!ma_dr_flac__read_and_decode_next_flac_frame(pFlac)) {
88164 |                 break;
88165 |             }
88166 |         } else {
88167 |             if (pFlac->currentFLACFrame.pcmFramesRemaining > pcmFramesToSeek) {
88168 |                 pcmFramesRead += pcmFramesToSeek;
88169 |                 pFlac->currentFLACFrame.pcmFramesRemaining -= (ma_uint32)pcmFramesToSeek;
88170 |                 pcmFramesToSeek = 0;
88171 |             } else {
88172 |                 pcmFramesRead += pFlac->currentFLACFrame.pcmFramesRemaining;
88173 |                 pcmFramesToSeek -= pFlac->currentFLACFrame.pcmFramesRemaining;
88174 |                 pFlac->currentFLACFrame.pcmFramesRemaining = 0;
88175 |             }
88176 |         }
88177 |     }
88178 |     pFlac->currentPCMFrame += pcmFramesRead;
88179 |     return pcmFramesRead;
88180 | }
88181 | static ma_bool32 ma_dr_flac__seek_to_pcm_frame_brute_force(ma_dr_flac* pFlac, ma_uint64
> pcmFrameIndex)
88182 | {
88183 |     ma_bool32 isMidFrame = MA_FALSE;
88184 |     ma_uint64 runningPCMFrameCount;
88185 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88186 |     if (pcmFrameIndex >= pFlac->currentPCMFrame) {
88187 |         runningPCMFrameCount = pFlac->currentPCMFrame;
88188 |         if (pFlac->currentPCMFrame == 0 && pFlac->currentFLACFrame.pcmFramesRemaining == 0) {
88189 |             if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88190 |                 return MA_FALSE;
88191 |             }

```

```

88192 |         } else {
88193 |             isMidFrame = MA_TRUE;
88194 |         }
88195 |     } else {
88196 |         runningPCMFrameCount = 0;
88197 |         if (!ma_dr_flac__seek_to_first_frame(pFlac)) {
88198 |             return MA_FALSE;
88199 |         }
88200 |         if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88201 |             return MA_FALSE;
88202 |         }
88203 |     }
88204 |     for (;;) {
88205 |         ma_uint64 pcmFrameCountInThisFLACFrame;
88206 |         ma_uint64 firstPCMFrameInFLACFrame = 0;
88207 |         ma_uint64 lastPCMFrameInFLACFrame = 0;
88208 |         ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(pFlac, &firstPCMFrameInFLACFrame,
> &lastPCMFrameInFLACFrame);
88209 |         pcmFrameCountInThisFLACFrame = (lastPCMFrameInFLACFrame - firstPCMFrameInFLACFrame) + 1;
88210 |         if (pcmFrameIndex < (runningPCMFrameCount + pcmFrameCountInThisFLACFrame)) {
88211 |             ma_uint64 pcmFramesToDecode = pcmFrameIndex - runningPCMFrameCount;
88212 |             if (!isMidFrame) {
88213 |                 ma_result result = ma_dr_flac__decode_flac_frame(pFlac);
88214 |                 if (result == MA_SUCCESS) {
88215 |                     return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, pcmFramesToDecode) ==
> pcmFramesToDecode;
88216 |                 } else {
88217 |                     if (result == MA_CRC_MISMATCH) {
88218 |                         goto next_iteration;
88219 |                     } else {
88220 |                         return MA_FALSE;
88221 |                     }
88222 |                 }
88223 |             } else {
88224 |                 return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, pcmFramesToDecode) ==
> pcmFramesToDecode;
88225 |             }
88226 |         } else {
88227 |             if (!isMidFrame) {
88228 |                 ma_result result = ma_dr_flac__seek_to_next_flac_frame(pFlac);
88229 |                 if (result == MA_SUCCESS) {
88230 |                     runningPCMFrameCount += pcmFrameCountInThisFLACFrame;
88231 |                 } else {
88232 |                     if (result == MA_CRC_MISMATCH) {
88233 |                         goto next_iteration;
88234 |                     } else {
88235 |                         return MA_FALSE;
88236 |                     }
88237 |                 }
88238 |             } else {
88239 |                 runningPCMFrameCount += pFlac->currentFLACFrame.pcmFramesRemaining;
88240 |                 pFlac->currentFLACFrame.pcmFramesRemaining = 0;
88241 |                 isMidFrame = MA_FALSE;
88242 |             }
88243 |             if (pcmFrameIndex == pFlac->totalLPCMFrameCount && runningPCMFrameCount ==
> pFlac->totalLPCMFrameCount) {
88244 |                 return MA_TRUE;
88245 |             }
88246 |         }
88247 |         next_iteration:
88248 |         if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88249 |             return MA_FALSE;
88250 |         }
88251 |     }
88252 | }
88253 | #if !defined(MA_DR_FLAC_NO_CRC)
88254 | #define MA_DR_FLAC_BINARY_SEARCH_APPROX_COMPRESSION_RATIO 0.6f
88255 | static ma_bool32 ma_dr_flac__seek_to_approximate_flac_frame_to_byte(ma_dr_flac* pFlac, ma_uint64
> targetByte, ma_uint64 rangeLo, ma_uint64 rangeHi, ma_uint64* pLastSuccessfulSeekOffset)
88256 | {
88257 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88258 |     MA_DR_FLAC_ASSERT(pLastSuccessfulSeekOffset != NULL);
88259 |     MA_DR_FLAC_ASSERT(targetByte >= rangeLo);
88260 |     MA_DR_FLAC_ASSERT(targetByte <= rangeHi);
88261 |     *pLastSuccessfulSeekOffset = pFlac->firstFLACFramePosInBytes;
88262 |     for (;;) {
88263 |         ma_uint64 lastTargetByte = targetByte;

```

```

88264 |         if (!ma_dr_flac__seek_to_byte(&pFlac->bs, targetByte)) {
88265 |             if (targetByte == 0) {
88266 |                 ma_dr_flac__seek_to_first_frame(pFlac);
88267 |                 return MA_FALSE;
88268 |             }
88269 |             targetByte = rangeLo + ((rangeHi - rangeLo)/2);
88270 |             rangeHi = targetByte;
88271 |         } else {
88272 |             MA_DR_FLAC_ZERO_MEMORY(&pFlac->currentFLACFrame, sizeof(pFlac->currentFLACFrame));
88273 | #if 1
88274 |             if (!ma_dr_flac__read_and_decode_next_flac_frame(pFlac)) {
88275 |                 targetByte = rangeLo + ((rangeHi - rangeLo)/2);
88276 |                 rangeHi = targetByte;
88277 |             } else {
88278 |                 break;
88279 |             }
88280 | #else
88281 |             if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
88282 | > &pFlac->currentFLACFrame.header)) {
88283 |                 targetByte = rangeLo + ((rangeHi - rangeLo)/2);
88284 |                 rangeHi = targetByte;
88285 |             } else {
88286 |                 break;
88287 |             }
88288 | #endif
88289 |             if(targetByte == lastTargetByte) {
88290 |                 return MA_FALSE;
88291 |             }
88292 |         }
88293 |         ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(pFlac, &pFlac->currentPCMFrame, NULL);
88294 |         MA_DR_FLAC_ASSERT(targetByte <= rangeHi);
88295 |         *pLastSuccessfulSeekOffset = targetByte;
88296 |         return MA_TRUE;
88297 |     }
88298 | static ma_bool32 ma_dr_flac__decode_flac_frame_and_seek_forward_by_pcm_frames(ma_dr_flac* pFlac,
88299 | > ma_uint64 offset)
88300 | {
88301 |     #if 0
88302 |         if (ma_dr_flac__decode_flac_frame(pFlac) != MA_SUCCESS) {
88303 |             if (ma_dr_flac__read_and_decode_next_flac_frame(pFlac) == MA_FALSE) {
88304 |                 return MA_FALSE;
88305 |             }
88306 |         }
88307 |     #endif
88308 |     return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, offset) == offset;
88309 | }
88310 | static ma_bool32 ma_dr_flac__seek_to_pcm_frame_binary_search_internal(ma_dr_flac* pFlac, ma_uint64
88311 | > pcmFrameIndex, ma_uint64 byteRangeLo, ma_uint64 byteRangeHi)
88312 | {
88313 |     ma_uint64 targetByte;
88314 |     ma_uint64 pcmRangeLo = pFlac->totalPCMFrameCount;
88315 |     ma_uint64 pcmRangeHi = 0;
88316 |     ma_uint64 lastSuccessfulSeekOffset = (ma_uint64)-1;
88317 |     ma_uint64 closestSeekOffsetBeforeTargetPCMFrame = byteRangeLo;
88318 |     ma_uint32 seekForwardThreshold = (pFlac->maxBlockSizeInPCMFrames != 0) ?
88319 | > pFlac->maxBlockSizeInPCMFrames*2 : 4096;
88320 |     targetByte = byteRangeLo + (ma_uint64)((ma_int64)((pcmFrameIndex - pFlac->currentPCMFrame) *
88321 | > pFlac->channels * pFlac->bitsPerSample)/8.0f) * MA_DR_FLAC_BINARY_SEARCH_APPROX_COMPRESSION_RATIO);
88322 |     if (targetByte > byteRangeHi) {
88323 |         targetByte = byteRangeHi;
88324 |     }
88325 |     for (;;) {
88326 |         if (ma_dr_flac__seek_to_approximate_flac_frame_to_byte(pFlac, targetByte, byteRangeLo,
88327 | > byteRangeHi, &lastSuccessfulSeekOffset)) {
88328 |             ma_uint64 newPCMRangeLo;
88329 |             ma_uint64 newPCMRangeHi;
88330 |             ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(pFlac, &newPCMRangeLo,
88331 | > &newPCMRangeHi);
88332 |             if (pcmRangeLo == newPCMRangeLo) {
88333 |                 if (!ma_dr_flac__seek_to_approximate_flac_frame_to_byte(pFlac,
88334 | > closestSeekOffsetBeforeTargetPCMFrame, closestSeekOffsetBeforeTargetPCMFrame, byteRangeHi,
88335 | > &lastSuccessfulSeekOffset)) {
88336 |                     break;
88337 |                 }
88338 |                 if (ma_dr_flac__decode_flac_frame_and_seek_forward_by_pcm_frames(pFlac,
88339 | > pcmFrameIndex - pFlac->currentPCMFrame)) {
88340 |                     return MA_TRUE;
88341 |                 } else {

```

```

88333 |         break;
88334 |     }
88335 | }
88336 |     pcmRangeLo = newPCMRangeLo;
88337 |     pcmRangeHi = newPCMRangeHi;
88338 |     if (pcmRangeLo <= pcmFrameIndex && pcmRangeHi >= pcmFrameIndex) {
88339 |         if (ma_dr_flac__decode_flac_frame_and_seek_forward_by_pcm_frames(pFlac,
> pcmFrameIndex - pFlac->currentPCMFrame) ) {
88340 |             return MA_TRUE;
88341 |         } else {
88342 |             break;
88343 |         }
88344 |     } else {
88345 |         const float approxCompressionRatio = (ma_int64)(lastSuccessfulSeekOffset -
> pFlac->firstFLACFramePosInBytes) / ((ma_int64)(pcmRangeLo * pFlac->channels *
> pFlac->bitsPerSample)/8.0f);
88346 |         if (pcmRangeLo > pcmFrameIndex) {
88347 |             byteRangeHi = lastSuccessfulSeekOffset;
88348 |             if (byteRangeLo > byteRangeHi) {
88349 |                 byteRangeLo = byteRangeHi;
88350 |             }
88351 |             targetByte = byteRangeLo + ((byteRangeHi - byteRangeLo) / 2);
88352 |             if (targetByte < byteRangeLo) {
88353 |                 targetByte = byteRangeLo;
88354 |             }
88355 |         } else {
88356 |             if ((pcmFrameIndex - pcmRangeLo) < seekForwardThreshold) {
88357 |                 if (ma_dr_flac__decode_flac_frame_and_seek_forward_by_pcm_frames(pFlac,
> pcmFrameIndex - pFlac->currentPCMFrame)) {
88358 |                     return MA_TRUE;
88359 |                 } else {
88360 |                     break;
88361 |                 }
88362 |             } else {
88363 |                 byteRangeLo = lastSuccessfulSeekOffset;
88364 |                 if (byteRangeHi < byteRangeLo) {
88365 |                     byteRangeHi = byteRangeLo;
88366 |                 }
88367 |                 targetByte = lastSuccessfulSeekOffset +
> (ma_uint64)(((ma_int64)((pcmFrameIndex-pcmRangeLo) * pFlac->channels * pFlac->bitsPerSample)/8.0f) *
> approxCompressionRatio);
88368 |                 if (targetByte > byteRangeHi) {
88369 |                     targetByte = byteRangeHi;
88370 |                 }
88371 |                 if (closestSeekOffsetBeforeTargetPCMFrame < lastSuccessfulSeekOffset) {
88372 |                     closestSeekOffsetBeforeTargetPCMFrame = lastSuccessfulSeekOffset;
88373 |                 }
88374 |             }
88375 |         }
88376 |     }
88377 | } else {
88378 |     break;
88379 | }
88380 | }
88381 | ma_dr_flac__seek_to_first_frame(pFlac);
88382 | return MA_FALSE;
88383 | }
88384 | static ma_bool32 ma_dr_flac__seek_to_pcm_frame__binary_search(ma_dr_flac* pFlac, ma_uint64
> pcmFrameIndex)
88385 | {
88386 |     ma_uint64 byteRangeLo;
88387 |     ma_uint64 byteRangeHi;
88388 |     ma_uint32 seekForwardThreshold = (pFlac->maxBlockSizeInPCMFrames != 0) ?
> pFlac->maxBlockSizeInPCMFrames*2 : 4096;
88389 |     if (ma_dr_flac__seek_to_first_frame(pFlac) == MA_FALSE) {
88390 |         return MA_FALSE;
88391 |     }
88392 |     if (pcmFrameIndex < seekForwardThreshold) {
88393 |         return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, pcmFrameIndex) == pcmFrameIndex;
88394 |     }
88395 |     byteRangeLo = pFlac->firstFLACFramePosInBytes;
88396 |     byteRangeHi = pFlac->firstFLACFramePosInBytes + (ma_uint64)((ma_int64)(pFlac->totalPCMFrameCount
> * pFlac->channels * pFlac->bitsPerSample)/8.0f);
88397 |     return ma_dr_flac__seek_to_pcm_frame__binary_search_internal(pFlac, pcmFrameIndex, byteRangeLo,
> byteRangeHi);
88398 | }
88399 | #endif
88400 | static ma_bool32 ma_dr_flac__seek_to_pcm_frame__seek_table(ma_dr_flac* pFlac, ma_uint64
> pcmFrameIndex)

```

```

88401 | {
88402 |     ma_uint32 iClosestSeekpoint = 0;
88403 |     ma_bool32 isMidFrame = MA_FALSE;
88404 |     ma_uint64 runningPCMFrameCount;
88405 |     ma_uint32 iSeekpoint;
88406 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
88407 |     if (pFlac->pSeekpoints == NULL || pFlac->seekpointCount == 0) {
88408 |         return MA_FALSE;
88409 |     }
88410 |     if (pFlac->pSeekpoints[0].firstPCMFrame > pcmFrameIndex) {
88411 |         return MA_FALSE;
88412 |     }
88413 |     for (iSeekpoint = 0; iSeekpoint < pFlac->seekpointCount; ++iSeekpoint) {
88414 |         if (pFlac->pSeekpoints[iSeekpoint].firstPCMFrame >= pcmFrameIndex) {
88415 |             break;
88416 |         }
88417 |         iClosestSeekpoint = iSeekpoint;
88418 |     }
88419 |     if (pFlac->pSeekpoints[iClosestSeekpoint].pcmFrameCount == 0 ||
> pFlac->pSeekpoints[iClosestSeekpoint].pcmFrameCount > pFlac->maxBlockSizeInPCMFrames) {
88420 |         return MA_FALSE;
88421 |     }
88422 |     if (pFlac->pSeekpoints[iClosestSeekpoint].firstPCMFrame > pFlac->totalPCMFrameCount &&
> pFlac->totalPCMFrameCount > 0) {
88423 |         return MA_FALSE;
88424 |     }
88425 |     #if !defined(MA_DR_FLAC_NO_CRC)
88426 |     if (pFlac->totalPCMFrameCount > 0) {
88427 |         ma_uint64 byteRangeLo;
88428 |         ma_uint64 byteRangeHi;
88429 |         byteRangeHi = pFlac->firstFLACFramePosInBytes +
> (ma_uint64)((ma_int64)(pFlac->totalPCMFrameCount * pFlac->channels * pFlac->bitsPerSample)/8.0f);
88430 |         byteRangeLo = pFlac->firstFLACFramePosInBytes +
> pFlac->pSeekpoints[iClosestSeekpoint].flacFrameOffset;
88431 |         if (iClosestSeekpoint < pFlac->seekpointCount-1) {
88432 |             ma_uint32 iNextSeekpoint = iClosestSeekpoint + 1;
88433 |             if (pFlac->pSeekpoints[iClosestSeekpoint].flacFrameOffset >=
> pFlac->pSeekpoints[iNextSeekpoint].flacFrameOffset ||
> pFlac->pSeekpoints[iNextSeekpoint].pcmFrameCount == 0) {
88434 |                 return MA_FALSE;
88435 |             }
88436 |             if (pFlac->pSeekpoints[iNextSeekpoint].firstPCMFrame != (((ma_uint64)0xFFFFFFFF << 32) |
> 0xFFFFFFFF)) {
88437 |                 byteRangeHi = pFlac->firstFLACFramePosInBytes +
> pFlac->pSeekpoints[iNextSeekpoint].flacFrameOffset - 1;
88438 |             }
88439 |         }
88440 |         if (ma_dr_flac__seek_to_byte(&pFlac->bs, pFlac->firstFLACFramePosInBytes +
> pFlac->pSeekpoints[iClosestSeekpoint].flacFrameOffset)) {
88441 |             if (ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88442 |                 ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(pFlac,
> &pFlac->currentPCMFrame, NULL);
88443 |                 if (ma_dr_flac__seek_to_pcm_frame_binary_search_internal(pFlac, pcmFrameIndex,
> byteRangeLo, byteRangeHi)) {
88444 |                     return MA_TRUE;
88445 |                 }
88446 |             }
88447 |         }
88448 |     }
88449 | #endif
88450 |     if (pcmFrameIndex >= pFlac->currentPCMFrame &&
> pFlac->pSeekpoints[iClosestSeekpoint].firstPCMFrame <= pFlac->currentPCMFrame) {
88451 |         runningPCMFrameCount = pFlac->currentPCMFrame;
88452 |         if (pFlac->currentPCMFrame == 0 && pFlac->currentFLACFrame.pcmFramesRemaining == 0) {
88453 |             if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88454 |                 return MA_FALSE;
88455 |             }
88456 |         } else {
88457 |             isMidFrame = MA_TRUE;
88458 |         }
88459 |     } else {
88460 |         runningPCMFrameCount = pFlac->pSeekpoints[iClosestSeekpoint].firstPCMFrame;
88461 |         if (!ma_dr_flac__seek_to_byte(&pFlac->bs, pFlac->firstFLACFramePosInBytes +
> pFlac->pSeekpoints[iClosestSeekpoint].flacFrameOffset)) {
88462 |             return MA_FALSE;
88463 |         }
88464 |         if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,

```



```

> &pFlac->currentFLACFrame.header)) {
88465 |         return MA_FALSE;
88466 |     }
88467 | }
88468 | for (;;) {
88469 |     ma_uint64 pcmFrameCountInThisFLACFrame;
88470 |     ma_uint64 firstPCMFrameInFLACFrame = 0;
88471 |     ma_uint64 lastPCMFrameInFLACFrame = 0;
88472 |     ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(pFlac, &firstPCMFrameInFLACFrame,
> &lastPCMFrameInFLACFrame);
88473 |     pcmFrameCountInThisFLACFrame = (lastPCMFrameInFLACFrame - firstPCMFrameInFLACFrame) + 1;
88474 |     if (pcmFrameIndex < (runningPCMFrameCount + pcmFrameCountInThisFLACFrame)) {
88475 |         ma_uint64 pcmFramesToDecode = pcmFrameIndex - runningPCMFrameCount;
88476 |         if (!isMidFrame) {
88477 |             ma_result result = ma_dr_flac__decode_flac_frame(pFlac);
88478 |             if (result == MA_SUCCESS) {
88479 |                 return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, pcmFramesToDecode) ==
> pcmFramesToDecode;
88480 |             } else {
88481 |                 if (result == MA_CRC_MISMATCH) {
88482 |                     goto next_iteration;
88483 |                 } else {
88484 |                     return MA_FALSE;
88485 |                 }
88486 |             }
88487 |         } else {
88488 |             return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, pcmFramesToDecode) ==
> pcmFramesToDecode;
88489 |         }
88490 |     } else {
88491 |         if (!isMidFrame) {
88492 |             ma_result result = ma_dr_flac__seek_to_next_flac_frame(pFlac);
88493 |             if (result == MA_SUCCESS) {
88494 |                 runningPCMFrameCount += pcmFrameCountInThisFLACFrame;
88495 |             } else {
88496 |                 if (result == MA_CRC_MISMATCH) {
88497 |                     goto next_iteration;
88498 |                 } else {
88499 |                     return MA_FALSE;
88500 |                 }
88501 |             }
88502 |         } else {
88503 |             runningPCMFrameCount += pFlac->currentFLACFrame.pcmFramesRemaining;
88504 |             pFlac->currentFLACFrame.pcmFramesRemaining = 0;
88505 |             isMidFrame = MA_FALSE;
88506 |         }
88507 |         if (pcmFrameIndex == pFlac->totalPCMFrameCount && runningPCMFrameCount ==
> pFlac->totalPCMFrameCount) {
88508 |             return MA_TRUE;
88509 |         }
88510 |     }
88511 |     next_iteration:
88512 |     if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
88513 |         return MA_FALSE;
88514 |     }
88515 | }
88516 | }
88517 | #ifndef MA_DR_FLAC_NO_OGG
88518 | typedef struct
88519 | {
88520 |     ma_uint8 capturePattern[4];
88521 |     ma_uint8 structureVersion;
88522 |     ma_uint8 headerType;
88523 |     ma_uint64 granulePosition;
88524 |     ma_uint32 serialNumber;
88525 |     ma_uint32 sequenceNumber;
88526 |     ma_uint32 checksum;
88527 |     ma_uint8 segmentCount;
88528 |     ma_uint8 segmentTable[255];
88529 | } ma_dr_flac_ogg_page_header;
88530 | #endif
88531 | typedef struct
88532 | {
88533 |     ma_dr_flac_read_proc onRead;
88534 |     ma_dr_flac_seek_proc onSeek;
88535 |     ma_dr_flac_tell_proc onTell;
88536 |     ma_dr_flac_meta_proc onMeta;
88537 |     ma_dr_flac_container container;

```

```

88538 | void* pUserData;
88539 | void* pUserDataMD;
88540 | ma_uint32 sampleRate;
88541 | ma_uint8 channels;
88542 | ma_uint8 bitsPerSample;
88543 | ma_uint64 totalPCMFrameCount;
88544 | ma_uint16 maxBlockSizeInPCMframes;
88545 | ma_uint64 runningFilePos;
88546 | ma_bool32 hasStreamInfoBlock;
88547 | ma_bool32 hasMetadataBlocks;
88548 | ma_dr_flac_bs bs;
88549 | ma_dr_flac_frame_header firstFrameHeader;
88550 | #ifndef MA_DR_FLAC_NO_OGG
88551 | ma_uint32 oggSerial;
88552 | ma_uint64 oggFirstBytePos;
88553 | ma_dr_flac_ogg_page_header oggBosHeader;
88554 | #endif
88555 | } ma_dr_flac_init_info;
88556 | static MA_INLINE void ma_dr_flac__decode_block_header(ma_uint32 blockHeader, ma_uint8* isLastBlock,
> ma_uint8* blockType, ma_uint32* blockSize)
88557 | {
88558 |     blockHeader = ma_dr_flac__be2host_32(blockHeader);
88559 |     *isLastBlock = (ma_uint8)((blockHeader & 0x80000000UL) >> 31);
88560 |     *blockType = (ma_uint8)((blockHeader & 0x7F000000UL) >> 24);
88561 |     *blockSize = (blockHeader & 0x00FFFFFFUL);
88562 | }
88563 | static MA_INLINE ma_bool32 ma_dr_flac__read_and_decode_block_header(ma_dr_flac_read_proc onRead,
> void* pUserData, ma_uint8* isLastBlock, ma_uint8* blockType, ma_uint32* blockSize)
88564 | {
88565 |     ma_uint32 blockHeader;
88566 |     *blockSize = 0;
88567 |     if (onRead(pUserData, &blockHeader, 4) != 4) {
88568 |         return MA_FALSE;
88569 |     }
88570 |     ma_dr_flac__decode_block_header(blockHeader, isLastBlock, blockType, blockSize);
88571 |     return MA_TRUE;
88572 | }
88573 | static ma_bool32 ma_dr_flac__read_streaminfo(ma_dr_flac_read_proc onRead, void* pUserData,
> ma_dr_flac_streaminfo* pStreamInfo)
88574 | {
88575 |     ma_uint32 blockSizes;
88576 |     ma_uint64 frameSizes = 0;
88577 |     ma_uint64 importantProps;
88578 |     ma_uint8 md5[16];
88579 |     if (onRead(pUserData, &blockSizes, 4) != 4) {
88580 |         return MA_FALSE;
88581 |     }
88582 |     if (onRead(pUserData, &frameSizes, 6) != 6) {
88583 |         return MA_FALSE;
88584 |     }
88585 |     if (onRead(pUserData, &importantProps, 8) != 8) {
88586 |         return MA_FALSE;
88587 |     }
88588 |     if (onRead(pUserData, md5, sizeof(md5)) != sizeof(md5)) {
88589 |         return MA_FALSE;
88590 |     }
88591 |     blockSizes = ma_dr_flac__be2host_32(blockSizes);
88592 |     frameSizes = ma_dr_flac__be2host_64(frameSizes);
88593 |     importantProps = ma_dr_flac__be2host_64(importantProps);
88594 |     pStreamInfo->minBlockSizeInPCMframes = (ma_uint16)((blockSizes & 0xFFFF0000) >> 16);
88595 |     pStreamInfo->maxBlockSizeInPCMframes = (ma_uint16)(blockSizes & 0x0000FFFF);
88596 |     pStreamInfo->minFrameSizeInPCMframes = (ma_uint32)((frameSizes & (((ma_uint64)0x00FFFFFF <<
> 16) << 24)) >> 40);
88597 |     pStreamInfo->maxFrameSizeInPCMframes = (ma_uint32)((frameSizes & (((ma_uint64)0x00FFFFFF <<
> 16) << 0)) >> 16);
88598 |     pStreamInfo->sampleRate = (ma_uint32)((importantProps & (((ma_uint64)0x00FFFFFF <<
> 16) << 28)) >> 44);
88599 |     pStreamInfo->channels = (ma_uint8)((importantProps & (((ma_uint64)0x0000000E <<
> 16) << 24)) >> 41) + 1;
88600 |     pStreamInfo->bitsPerSample = (ma_uint8)((importantProps & (((ma_uint64)0x0000001F <<
> 16) << 20)) >> 36) + 1;
88601 |     pStreamInfo->totalPCMFrameCount = ((importantProps &
> (((ma_uint64)0x0000000F << 16) << 16) | 0xFFFFFFFF));
88602 |     MA_DR_FLAC_COPY_MEMORY(pStreamInfo->md5, md5, sizeof(md5));
88603 |     return MA_TRUE;
88604 | }
88605 | static void* ma_dr_flac__malloc_default(size_t sz, void* pUserData)
88606 | {
88607 |     (void)pUserData;

```

```

88608 |     return MA_DR_FLAC_MALLOC(sz);
88609 | }
88610 | static void* ma_dr_flac__realloc_default(void* p, size_t sz, void* pUserData)
88611 | {
88612 |     (void)pUserData;
88613 |     return MA_DR_FLAC_REALLOC(p, sz);
88614 | }
88615 | static void ma_dr_flac__free_default(void* p, void* pUserData)
88616 | {
88617 |     (void)pUserData;
88618 |     MA_DR_FLAC_FREE(p);
88619 | }
88620 | static void* ma_dr_flac__malloc_from_callbacks(size_t sz, const ma_allocation_callbacks*
> pAllocationCallbacks)
88621 | {
88622 |     if (pAllocationCallbacks == NULL) {
88623 |         return NULL;
88624 |     }
88625 |     if (pAllocationCallbacks->onMalloc != NULL) {
88626 |         return pAllocationCallbacks->onMalloc(sz, pAllocationCallbacks->pUserData);
88627 |     }
88628 |     if (pAllocationCallbacks->onRealloc != NULL) {
88629 |         return pAllocationCallbacks->onRealloc(NULL, sz, pAllocationCallbacks->pUserData);
88630 |     }
88631 |     return NULL;
88632 | }
88633 | static void* ma_dr_flac__realloc_from_callbacks(void* p, size_t szNew, size_t szOld, const
> ma_allocation_callbacks* pAllocationCallbacks)
88634 | {
88635 |     if (pAllocationCallbacks == NULL) {
88636 |         return NULL;
88637 |     }
88638 |     if (pAllocationCallbacks->onRealloc != NULL) {
88639 |         return pAllocationCallbacks->onRealloc(p, szNew, pAllocationCallbacks->pUserData);
88640 |     }
88641 |     if (pAllocationCallbacks->onMalloc != NULL && pAllocationCallbacks->onFree != NULL) {
88642 |         void* p2;
88643 |         p2 = pAllocationCallbacks->onMalloc(szNew, pAllocationCallbacks->pUserData);
88644 |         if (p2 == NULL) {
88645 |             return NULL;
88646 |         }
88647 |         if (p != NULL) {
88648 |             MA_DR_FLAC_COPY_MEMORY(p2, p, szOld);
88649 |             pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
88650 |         }
88651 |         return p2;
88652 |     }
88653 |     return NULL;
88654 | }
88655 | static void ma_dr_flac__free_from_callbacks(void* p, const ma_allocation_callbacks*
> pAllocationCallbacks)
88656 | {
88657 |     if (p == NULL || pAllocationCallbacks == NULL) {
88658 |         return;
88659 |     }
88660 |     if (pAllocationCallbacks->onFree != NULL) {
88661 |         pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
88662 |     }
88663 | }
88664 | static ma_bool32 ma_dr_flac__read_and_decode_metadata(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta, void*
> pUserData, void* pUserDataMD, ma_uint64* pFirstFramePos, ma_uint64* pSeektablePos, ma_uint32*
> pSeekpointCount, ma_allocation_callbacks* pAllocationCallbacks)
88665 | {
88666 |     ma_uint64 runningFilePos = 42;
88667 |     ma_uint64 seektablePos = 0;
88668 |     ma_uint32 seektableSize = 0;
88669 |     (void)onTell;
88670 |     for (;;) {
88671 |         ma_dr_flac_metadata metadata;
88672 |         ma_uint8 isLastBlock = 0;
88673 |         ma_uint8 blockType = 0;
88674 |         ma_uint32 blockSize;
88675 |         if (ma_dr_flac__read_and_decode_block_header(onRead, pUserData, &isLastBlock, &blockType,
> &blockSize) == MA_FALSE) {
88676 |             return MA_FALSE;
88677 |         }
88678 |         runningFilePos += 4;
88679 |         metadata.type = blockType;

```

```

88680 |     metadata.rawDataSize = 0;
88681 |     metadata.rawDataOffset = runningFilePos;
88682 |     metadata.pRawData = NULL;
88683 |     switch (blockType)
88684 |     {
88685 |         case MA_DR_FLAC_METADATA_BLOCK_TYPE_APPLICATION:
88686 |         {
88687 |             if (blockSize < 4) {
88688 |                 return MA_FALSE;
88689 |             }
88690 |             if (onMeta) {
88691 |                 void* pRawData = ma_dr_flac__malloc_from_callbacks(blockSize,
> pAllocationCallbacks);
88692 |                 if (pRawData == NULL) {
88693 |                     return MA_FALSE;
88694 |                 }
88695 |                 if (onRead(pUserData, pRawData, blockSize) != blockSize) {
88696 |                     ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88697 |                     return MA_FALSE;
88698 |                 }
88699 |                 metadata.pRawData = pRawData;
88700 |                 metadata.rawDataSize = blockSize;
88701 |                 metadata.data.application.id =
> ma_dr_flac__be2host_32(*(ma_uint32*)pRawData);
88702 |                 metadata.data.application.pData = (const void*)((ma_uint8*)pRawData +
> sizeof(ma_uint32));
88703 |                 metadata.data.application.dataSize = blockSize - sizeof(ma_uint32);
88704 |                 onMeta(pUserDataMD, &metadata);
88705 |                 ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88706 |             }
88707 |             break;
88708 |             case MA_DR_FLAC_METADATA_BLOCK_TYPE_SEEKTABLE:
88709 |             {
88710 |                 seektablePos = runningFilePos;
88711 |                 seektableSize = blockSize;
88712 |                 if (onMeta) {
88713 |                     ma_uint32 seekpointCount;
88714 |                     ma_uint32 iSeekpoint;
88715 |                     void* pRawData;
88716 |                     seekpointCount = blockSize/MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES;
88717 |                     pRawData = ma_dr_flac__malloc_from_callbacks(seekpointCount *
> sizeof(ma_dr_flac_seekpoint), pAllocationCallbacks);
88718 |                     if (pRawData == NULL) {
88719 |                         return MA_FALSE;
88720 |                     }
88721 |                     for (iSeekpoint = 0; iSeekpoint < seekpointCount; ++iSeekpoint) {
88722 |                         ma_dr_flac_seekpoint* pSeekpoint = (ma_dr_flac_seekpoint*)pRawData +
> iSeekpoint;
88723 |                         if (onRead(pUserData, pSeekpoint, MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES) !=
> MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES) {
88724 |                             ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88725 |                             return MA_FALSE;
88726 |                         }
88727 |                         pSeekpoint->firstPCMFrame =
> ma_dr_flac__be2host_64(pSeekpoint->firstPCMFrame);
88728 |                         pSeekpoint->flacFrameOffset =
> ma_dr_flac__be2host_64(pSeekpoint->flacFrameOffset);
88729 |                         pSeekpoint->pcmFrameCount =
> ma_dr_flac__be2host_16(pSeekpoint->pcmFrameCount);
88730 |                     }
88731 |                     metadata.pRawData = pRawData;
88732 |                     metadata.rawDataSize = blockSize;
88733 |                     metadata.data.seektable.seekpointCount = seekpointCount;
88734 |                     metadata.data.seektable.pSeekpoints = (const ma_dr_flac_seekpoint*)pRawData;
88735 |                     onMeta(pUserDataMD, &metadata);
88736 |                     ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88737 |                 }
88738 |             } break;
88739 |             case MA_DR_FLAC_METADATA_BLOCK_TYPE_VORBIS_COMMENT:
88740 |             {
88741 |                 if (blockSize < 8) {
88742 |                     return MA_FALSE;
88743 |                 }
88744 |                 if (onMeta) {
88745 |                     void* pRawData;
88746 |                     const char* pRunningData;
88747 |                     const char* pRunningDataEnd;
88748 |                     ma_uint32 i;
88749 |                     pRawData = ma_dr_flac__malloc_from_callbacks(blockSize, pAllocationCallbacks);

```

```

88750 |         if (pRawData == NULL) {
88751 |             return MA_FALSE;
88752 |         }
88753 |         if (onRead(pUserData, pRawData, blockSize) != blockSize) {
88754 |             ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88755 |             return MA_FALSE;
88756 |         }
88757 |         metadata.pRawData = pRawData;
88758 |         metadata.rawDataSize = blockSize;
88759 |         pRunningData = (const char*)pRawData;
88760 |         pRunningDataEnd = (const char*)pRawData + blockSize;
88761 |         metadata.data.vorbis_comment.vendorLength =
> ma_dr_flac__le2host_32_ptr_unaligned(pRunningData); pRunningData += 4;
88762 |         if ((pRunningDataEnd - pRunningData) - 4 <
> (ma_int64)metadata.data.vorbis_comment.vendorLength) {
88763 |             ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88764 |             return MA_FALSE;
88765 |         }
88766 |         metadata.data.vorbis_comment.vendor = pRunningData;
> pRunningData +=
> metadata.data.vorbis_comment.vendorLength;
88767 |         metadata.data.vorbis_comment.commentCount =
> ma_dr_flac__le2host_32_ptr_unaligned(pRunningData); pRunningData += 4;
88768 |         if ((pRunningDataEnd - pRunningData) / sizeof(ma_uint32) <
> metadata.data.vorbis_comment.commentCount) {
88769 |             ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88770 |             return MA_FALSE;
88771 |         }
88772 |         metadata.data.vorbis_comment.pComments = pRunningData;
88773 |         for (i = 0; i < metadata.data.vorbis_comment.commentCount; ++i) {
88774 |             ma_uint32 commentLength;
88775 |             if (pRunningDataEnd - pRunningData < 4) {
88776 |                 ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88777 |                 return MA_FALSE;
88778 |             }
88779 |             commentLength = ma_dr_flac__le2host_32_ptr_unaligned(pRunningData);
> pRunningData += 4;
88780 |             if (pRunningDataEnd - pRunningData < (ma_int64)commentLength) {
88781 |                 ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88782 |                 return MA_FALSE;
88783 |             }
88784 |             pRunningData += commentLength;
88785 |         }
88786 |         onMeta(pUserDataMD, &metadata);
88787 |         ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88788 |     }
88789 | } break;
88790 | case MA_DR_FLAC_METADATA_BLOCK_TYPE_CUESHEET:
88791 | {
88792 |     if (blockSize < 396) {
88793 |         return MA_FALSE;
88794 |     }
88795 |     if (onMeta) {
88796 |         void* pRawData;
88797 |         const char* pRunningData;
88798 |         const char* pRunningDataEnd;
88799 |         size_t bufferSize;
88800 |         ma_uint8 iTrack;
88801 |         ma_uint8 iIndex;
88802 |         void* pTrackData;
88803 |         pRawData = ma_dr_flac__malloc_from_callbacks(blockSize, pAllocationCallbacks);
88804 |         if (pRawData == NULL) {
88805 |             return MA_FALSE;
88806 |         }
88807 |         if (onRead(pUserData, pRawData, blockSize) != blockSize) {
88808 |             ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88809 |             return MA_FALSE;
88810 |         }
88811 |         metadata.pRawData = pRawData;
88812 |         metadata.rawDataSize = blockSize;
88813 |         pRunningData = (const char*)pRawData;
88814 |         pRunningDataEnd = (const char*)pRawData + blockSize;
88815 |         MA_DR_FLAC_COPY_MEMORY(metadata.data.cuesheet.catalog, pRunningData, 128);
> pRunningData += 128;
88816 |         metadata.data.cuesheet.leadInSampleCount = ma_dr_flac__be2host_64(*(const
> ma_uint64*)pRunningData); pRunningData += 8;
88817 |         metadata.data.cuesheet.isCD = (pRunningData[0] & 0x80) != 0;
> pRunningData += 259;
88818 |         metadata.data.cuesheet.trackCount = pRunningData[0];

```

```

>                                pRunningData += 1;
88819 |                                metadata.data.cuesheet.pTrackData      = NULL;
88820 |                                {
88821 |                                    const char* pRunningDataSaved = pRunningData;
88822 |                                    bufferSize = metadata.data.cuesheet.trackCount *
> MA_DR_FLAC_CUESHEET_TRACK_SIZE_IN_BYTES;
88823 |                                    for (iTrack = 0; iTrack < metadata.data.cuesheet.trackCount; ++iTrack) {
88824 |                                        ma_uint8 indexCount;
88825 |                                        ma_uint32 indexPointSize;
88826 |                                        if (pRunningDataEnd - pRunningData <
> MA_DR_FLAC_CUESHEET_TRACK_SIZE_IN_BYTES) {
88827 |                                            ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88828 |                                            return MA_FALSE;
88829 |                                        }
88830 |                                        pRunningData += 35;
88831 |                                        indexCount = pRunningData[0];
88832 |                                        pRunningData += 1;
88833 |                                        bufferSize += indexCount * sizeof(ma_dr_flac_cuesheet_track_index);
88834 |                                        indexPointSize = indexCount *
> MA_DR_FLAC_CUESHEET_TRACK_INDEX_SIZE_IN_BYTES;
88835 |                                        if (pRunningDataEnd - pRunningData < (ma_int64)indexPointSize) {
88836 |                                            ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88837 |                                            return MA_FALSE;
88838 |                                        }
88839 |                                        pRunningData += indexPointSize;
88840 |                                    }
88841 |                                    pRunningData = pRunningDataSaved;
88842 |                                }
88843 |                                {
88844 |                                    char* pRunningTrackData;
88845 |                                    pTrackData = ma_dr_flac__malloc_from_callbacks(bufferSize,
> pAllocationCallbacks);
88846 |                                    if (pTrackData == NULL) {
88847 |                                        ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88848 |                                        return MA_FALSE;
88849 |                                    }
88850 |                                    pRunningTrackData = (char*)pTrackData;
88851 |                                    for (iTrack = 0; iTrack < metadata.data.cuesheet.trackCount; ++iTrack) {
88852 |                                        ma_uint8 indexCount;
88853 |                                        MA_DR_FLAC_COPY_MEMORY(pRunningTrackData, pRunningData,
> MA_DR_FLAC_CUESHEET_TRACK_SIZE_IN_BYTES);
88854 |                                        pRunningData      += MA_DR_FLAC_CUESHEET_TRACK_SIZE_IN_BYTES-1;
88855 |                                        pRunningTrackData += MA_DR_FLAC_CUESHEET_TRACK_SIZE_IN_BYTES-1;
88856 |                                        indexCount = pRunningData[0];
88857 |                                        pRunningData      += 1;
88858 |                                        pRunningTrackData += 1;
88859 |                                        for (iIndex = 0; iIndex < indexCount; ++iIndex) {
88860 |                                            ma_dr_flac_cuesheet_track_index* pTrackIndex =
> (ma_dr_flac_cuesheet_track_index*)pRunningTrackData;
88861 |                                            MA_DR_FLAC_COPY_MEMORY(pRunningTrackData, pRunningData,
> MA_DR_FLAC_CUESHEET_TRACK_INDEX_SIZE_IN_BYTES);
88862 |                                            pRunningData      += MA_DR_FLAC_CUESHEET_TRACK_INDEX_SIZE_IN_BYTES;
88863 |                                            pRunningTrackData += sizeof(ma_dr_flac_cuesheet_track_index);
88864 |                                            pTrackIndex->offset = ma_dr_flac__be2host_64(pTrackIndex->offset);
88865 |                                        }
88866 |                                    }
88867 |                                    metadata.data.cuesheet.pTrackData = pTrackData;
88868 |                                }
88869 |                                ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
88870 |                                pRawData = NULL;
88871 |                                onMeta(pUserDataMD, &metadata);
88872 |                                ma_dr_flac__free_from_callbacks(pTrackData, pAllocationCallbacks);
88873 |                                pTrackData = NULL;
88874 |                            }
88875 |                    } break;
88876 |                    case MA_DR_FLAC_METADATA_BLOCK_TYPE_PICTURE:
88877 |                    {
88878 |                        if (blockSize < 32) {
88879 |                            return MA_FALSE;
88880 |                        }
88881 |                        if (onMeta) {
88882 |                            ma_bool32 result = MA_TRUE;
88883 |                            ma_uint32 blockSizeRemaining = blockSize;
88884 |                            char* pMime = NULL;
88885 |                            char* pDescription = NULL;
88886 |                            void* pPictureData = NULL;
88887 |                            if (blockSizeRemaining < 4 || onRead(pUserData, &metadata.data.picture.type, 4)
> != 4) {
88888 |                                result = MA_FALSE;

```

```

88889 |         goto done_flac;
88890 |     }
88891 |     blockSizeRemaining -= 4;
88892 |     metadata.data.picture.type = ma_dr_flac__be2host_32(metadata.data.picture.type);
88893 |     if (blockSizeRemaining < 4 || onRead(pUserData,
> &metadata.data.picture.mimeTypeLength, 4) != 4) {
88894 |         result = MA_FALSE;
88895 |         goto done_flac;
88896 |     }
88897 |     blockSizeRemaining -= 4;
88898 |     metadata.data.picture.mimeTypeLength =
> ma_dr_flac__be2host_32(metadata.data.picture.mimeTypeLength);
88899 |     pMime =
> (char*)ma_dr_flac__malloc_from_callbacks(metadata.data.picture.mimeTypeLength + 1,
> pAllocationCallbacks);
88900 |     if (pMime == NULL) {
88901 |         result = MA_FALSE;
88902 |         goto done_flac;
88903 |     }
88904 |     if (blockSizeRemaining < metadata.data.picture.mimeTypeLength || onRead(pUserData,
> pMime, metadata.data.picture.mimeTypeLength) != metadata.data.picture.mimeTypeLength) {
88905 |         result = MA_FALSE;
88906 |         goto done_flac;
88907 |     }
88908 |     blockSizeRemaining -= metadata.data.picture.mimeTypeLength;
88909 |     pMime[metadata.data.picture.mimeTypeLength] = '\0';
88910 |     metadata.data.picture.mime = (const char*)pMime;
88911 |     if (blockSizeRemaining < 4 || onRead(pUserData,
> &metadata.data.picture.descriptionLength, 4) != 4) {
88912 |         result = MA_FALSE;
88913 |         goto done_flac;
88914 |     }
88915 |     blockSizeRemaining -= 4;
88916 |     metadata.data.picture.descriptionLength =
> ma_dr_flac__be2host_32(metadata.data.picture.descriptionLength);
88917 |     pDescription =
> (char*)ma_dr_flac__malloc_from_callbacks(metadata.data.picture.descriptionLength + 1,
> pAllocationCallbacks);
88918 |     if (pDescription == NULL) {
88919 |         result = MA_FALSE;
88920 |         goto done_flac;
88921 |     }
88922 |     if (blockSizeRemaining < metadata.data.picture.descriptionLength ||
> onRead(pUserData, pDescription, metadata.data.picture.descriptionLength) !=
> metadata.data.picture.descriptionLength) {
88923 |         result = MA_FALSE;
88924 |         goto done_flac;
88925 |     }
88926 |     blockSizeRemaining -= metadata.data.picture.descriptionLength;
88927 |     pDescription[metadata.data.picture.descriptionLength] = '\0';
88928 |     metadata.data.picture.description = (const char*)pDescription;
88929 |     if (blockSizeRemaining < 4 || onRead(pUserData, &metadata.data.picture.width, 4)
> != 4) {
88930 |         result = MA_FALSE;
88931 |         goto done_flac;
88932 |     }
88933 |     blockSizeRemaining -= 4;
88934 |     metadata.data.picture.width =
> ma_dr_flac__be2host_32(metadata.data.picture.width);
88935 |     if (blockSizeRemaining < 4 || onRead(pUserData, &metadata.data.picture.height,
> 4) != 4) {
88936 |         result = MA_FALSE;
88937 |         goto done_flac;
88938 |     }
88939 |     blockSizeRemaining -= 4;
88940 |     metadata.data.picture.height =
> ma_dr_flac__be2host_32(metadata.data.picture.height);
88941 |     if (blockSizeRemaining < 4 || onRead(pUserData,
> &metadata.data.picture.colorDepth, 4) != 4) {
88942 |         result = MA_FALSE;
88943 |         goto done_flac;
88944 |     }
88945 |     blockSizeRemaining -= 4;
88946 |     metadata.data.picture.colorDepth =
> ma_dr_flac__be2host_32(metadata.data.picture.colorDepth);
88947 |     if (blockSizeRemaining < 4 || onRead(pUserData,
> &metadata.data.picture.indexColorCount, 4) != 4) {
88948 |         result = MA_FALSE;
88949 |         goto done_flac;

```



```

88950 |         }
88951 |         blockSizeRemaining -= 4;
88952 |         metadata.data.picture.indexColorCount =
> ma_dr_flac__be2host_32(metadata.data.picture.indexColorCount);
88953 |         if (blockSizeRemaining < 4 || onRead(pUserData,
> &metadata.data.picture.pictureDataSize, 4) != 4) {
88954 |             result = MA_FALSE;
88955 |             goto done_flac;
88956 |         }
88957 |         blockSizeRemaining -= 4;
88958 |         metadata.data.picture.pictureDataSize =
> ma_dr_flac__be2host_32(metadata.data.picture.pictureDataSize);
88959 |         if (blockSizeRemaining < metadata.data.picture.pictureDataSize) {
88960 |             result = MA_FALSE;
88961 |             goto done_flac;
88962 |         }
88963 |         metadata.data.picture.pictureDataOffset = runningFilePos + (blockSize -
> blockSizeRemaining);
88964 |         #ifndef MA_DR_FLAC_NO_PICTURE_METADATA_MALLOC
88965 |         pPictureData =
> ma_dr_flac__malloc_from_callbacks(metadata.data.picture.pictureDataSize, pAllocationCallbacks);
88966 |         if (pPictureData != NULL) {
88967 |             if (onRead(pUserData, pPictureData, metadata.data.picture.pictureDataSize)
> != metadata.data.picture.pictureDataSize) {
88968 |                 result = MA_FALSE;
88969 |                 goto done_flac;
88970 |             }
88971 |         } else
88972 |         #endif
88973 |         {
88974 |             if (!onSeek(pUserData, metadata.data.picture.pictureDataSize,
> MA_DR_FLAC_SEEK_CUR)) {
88975 |                 result = MA_FALSE;
88976 |                 goto done_flac;
88977 |             }
88978 |         }
88979 |         blockSizeRemaining -= metadata.data.picture.pictureDataSize;
88980 |         (void)blockSizeRemaining;
88981 |         metadata.data.picture.pPictureData = (const ma_uint8*)pPictureData;
88982 |         if (metadata.data.picture.pictureDataOffset != 0 ||
> metadata.data.picture.pPictureData != NULL) {
88983 |             onMeta(pUserDataMD, &metadata);
88984 |         } else {
88985 |         }
88986 |         done_flac:
88987 |         ma_dr_flac__free_from_callbacks(pMime, pAllocationCallbacks);
88988 |         ma_dr_flac__free_from_callbacks(pDescription, pAllocationCallbacks);
88989 |         ma_dr_flac__free_from_callbacks(pPictureData, pAllocationCallbacks);
88990 |         if (result != MA_TRUE) {
88991 |             return MA_FALSE;
88992 |         }
88993 |     }
88994 | } break;
88995 | case MA_DR_FLAC_METADATA_BLOCK_TYPE_PADDING:
88996 | {
88997 |     if (onMeta) {
88998 |         metadata.data.padding.unused = 0;
88999 |         if (!onSeek(pUserData, blockSize, MA_DR_FLAC_SEEK_CUR)) {
90000 |             isLastBlock = MA_TRUE;
90001 |         } else {
90002 |             onMeta(pUserDataMD, &metadata);
90003 |         }
90004 |     }
90005 | } break;
90006 | case MA_DR_FLAC_METADATA_BLOCK_TYPE_INVALID:
90007 | {
90008 |     if (onMeta) {
90009 |         if (!onSeek(pUserData, blockSize, MA_DR_FLAC_SEEK_CUR)) {
90010 |             isLastBlock = MA_TRUE;
90011 |         }
90012 |     }
90013 | } break;
90014 | default:
90015 | {
90016 |     if (onMeta) {
90017 |         void* pRawData = ma_dr_flac__malloc_from_callbacks(blockSize,
> pAllocationCallbacks);
90018 |         if (pRawData != NULL) {
90019 |             if (onRead(pUserData, pRawData, blockSize) != blockSize) {

```

```

89020 |                 ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
89021 |                 return MA_FALSE;
89022 |             }
89023 |         } else {
89024 |             if (!onSeek(pUserData, blockSize, MA_DR_FLAC_SEEK_CUR)) {
89025 |                 return MA_FALSE;
89026 |             }
89027 |         }
89028 |         metadata.pRawData = pRawData;
89029 |         metadata.rawDataSize = blockSize;
89030 |         onMeta(pUserDataMD, &metadata);
89031 |         ma_dr_flac__free_from_callbacks(pRawData, pAllocationCallbacks);
89032 |     }
89033 | } break;
89034 | }
89035 | if (onMeta == NULL && blockSize > 0) {
89036 |     if (!onSeek(pUserData, blockSize, MA_DR_FLAC_SEEK_CUR)) {
89037 |         isLastBlock = MA_TRUE;
89038 |     }
89039 | }
89040 | runningFilePos += blockSize;
89041 | if (isLastBlock) {
89042 |     break;
89043 | }
89044 | }
89045 | *pSeektablePos = seektablePos;
89046 | *pSeekpointCount = seektableSize / MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES;
89047 | *pFirstFramePos = runningFilePos;
89048 | return MA_TRUE;
89049 | }
89050 | static ma_bool32 ma_dr_flac__init_private_native(ma_dr_flac_init_info* pInit, ma_dr_flac_read_proc
> onRead, ma_dr_flac_seek_proc onSeek, ma_dr_flac_meta_proc onMeta, void* pUserData, void*
> pUserDataMD, ma_bool32 relaxed)
89051 | {
89052 |     ma_uint8 isLastBlock;
89053 |     ma_uint8 blockType;
89054 |     ma_uint32 blockSize;
89055 |     (void)onSeek;
89056 |     pInit->container = ma_dr_flac_container_native;
89057 |     if (!ma_dr_flac__read_and_decode_block_header(onRead, pUserData, &isLastBlock, &blockType,
> &blockSize)) {
89058 |         return MA_FALSE;
89059 |     }
89060 |     if (blockType != MA_DR_FLAC_METADATA_BLOCK_TYPE_STREAMINFO || blockSize != 34) {
89061 |         if (!relaxed) {
89062 |             return MA_FALSE;
89063 |         } else {
89064 |             pInit->hasStreamInfoBlock = MA_FALSE;
89065 |             pInit->hasMetadataBlocks = MA_FALSE;
89066 |             if (!ma_dr_flac__read_next_flac_frame_header(&pInit->bs, 0, &pInit->firstFrameHeader)) {
89067 |                 return MA_FALSE;
89068 |             }
89069 |             if (pInit->firstFrameHeader.bitsPerSample == 0) {
89070 |                 return MA_FALSE;
89071 |             }
89072 |             pInit->sampleRate = pInit->firstFrameHeader.sampleRate;
89073 |             pInit->channels =
> ma_dr_flac__get_channel_count_from_channel_assignment(pInit->firstFrameHeader.channelAssignment);
89074 |             pInit->bitsPerSample = pInit->firstFrameHeader.bitsPerSample;
89075 |             pInit->maxBlockSizeInPCMFrames = 65535;
89076 |             return MA_TRUE;
89077 |         }
89078 |     } else {
89079 |         ma_dr_flac_streaminfo streaminfo;
89080 |         if (!ma_dr_flac__read_streaminfo(onRead, pUserData, &streaminfo)) {
89081 |             return MA_FALSE;
89082 |         }
89083 |         pInit->hasStreamInfoBlock = MA_TRUE;
89084 |         pInit->sampleRate = streaminfo.sampleRate;
89085 |         pInit->channels = streaminfo.channels;
89086 |         pInit->bitsPerSample = streaminfo.bitsPerSample;
89087 |         pInit->totalPCMFrameCount = streaminfo.totalPCMFrameCount;
89088 |         pInit->maxBlockSizeInPCMFrames = streaminfo.maxBlockSizeInPCMFrames;
89089 |         pInit->hasMetadataBlocks = !isLastBlock;
89090 |         if (onMeta) {
89091 |             ma_dr_flac_metadata metadata;
89092 |             metadata.type = MA_DR_FLAC_METADATA_BLOCK_TYPE_STREAMINFO;
89093 |             metadata.pRawData = NULL;
89094 |             metadata.rawDataSize = 0;

```

```
89095 |         metadata.data.streaminfo = streaminfo;
89096 |         onMeta(pUserDataMD, &metadata);
89097 |     }
89098 |     return MA_TRUE;
89099 | }
89100 | }
89101 | #ifndef MA_DR_FLAC_NO_OGG
89102 | #define MA_DR_FLAC_OGG_MAX_PAGE_SIZE        65307
89103 | #define MA_DR_FLAC_OGG_CAPTURE_PATTERN_CRC32 1605413199
89104 | typedef enum
89105 | {
89106 |     ma_dr_flac_ogg_recover_on_crc_mismatch,
89107 |     ma_dr_flac_ogg_fail_on_crc_mismatch
89108 | } ma_dr_flac_ogg_crc_mismatch_recovery;
89109 | #ifndef MA_DR_FLAC_NO_CRC
89110 | static ma_uint32 ma_dr_flac_crc32_table[] = {
89111 |     0x00000000L, 0x04C11DB7L, 0x09823B6EL, 0x0D4326D9L,
89112 |     0x130476DCL, 0x17C56B6BL, 0x1A864DB2L, 0x1E475005L,
89113 |     0x2608EDB8L, 0x22C9F00FL, 0x2F8AD6D6L, 0x2B4BCB61L,
89114 |     0x350C9B64L, 0x31CD86D3L, 0x3C8EA00AL, 0x384FBDDBL,
89115 |     0x4C11DB70L, 0x48D0C6C7L, 0x4593E01EL, 0x4152FDA9L,
89116 |     0x5F15ADACL, 0x5BD4B01BL, 0x569796C2L, 0x52568B75L,
89117 |     0x6A1936C8L, 0x6ED82B7FL, 0x639B0DA6L, 0x675A1011L,
89118 |     0x791D4014L, 0x7DDC5DA3L, 0x709F7B7AL, 0x745E66CDL,
89119 |     0x9823B6E0L, 0x9CE2AB57L, 0x91A18D8EL, 0x95609039L,
89120 |     0x8B27C03CL, 0x8FE6DD8BL, 0x82A5FB52L, 0x8664E6E5L,
89121 |     0xBE2B5B58L, 0xBAEA46EFL, 0xB7A96036L, 0xB3687D81L,
89122 |     0xAD2F2D84L, 0xA9EE3033L, 0xA4AD16EAL, 0xA06C0B5DL,
89123 |     0xD4326D90L, 0xD0F37027L, 0xDDB056FEL, 0xD9714B49L,
89124 |     0xC7361B4CL, 0xC3F706FBL, 0xCEB42022L, 0xCA753D95L,
89125 |     0xF23A8028L, 0xF6FB9D9FL, 0xFBB8BB46L, 0xFF79A6F1L,
89126 |     0xE13EF6F4L, 0xE5FFEB43L, 0xE8BCCD9AL, 0xEC7DD02DL,
89127 |     0x34867077L, 0x30476DC0L, 0x3D044B19L, 0x39C556AEL,
89128 |     0x278206ABL, 0x23431B1CL, 0x2E003DC5L, 0x2AC12072L,
89129 |     0x128E9DCFL, 0x164F8078L, 0x1B0CA6A1L, 0x1FCDDB16L,
89130 |     0x018AEB13L, 0x054BF6A4L, 0x0808D07DL, 0x0CC9CDCAL,
89131 |     0x7897AB07L, 0x7C56B6B0L, 0x71159069L, 0x75D48DDEL,
89132 |     0x6B93DDDBL, 0x6F52C06CL, 0x6211E6B5L, 0x66D0FB02L,
89133 |     0x5E9F46BFL, 0x5A5E5B08L, 0x571D7DD1L, 0x53DC6066L,
89134 |     0x4D9B3063L, 0x495A2DD4L, 0x44190B0DL, 0x40D816BAL,
89135 |     0xACA5C697L, 0xA864DB20L, 0xA527FDF9L, 0xA1E6E04EL,
89136 |     0xBFA1B04BL, 0xBB60ADFCL, 0xB6238B25L, 0xB2E29692L,
89137 |     0x8AAD2B2FL, 0x8E6C3698L, 0x832F1041L, 0x87EE0DF6L,
89138 |     0x99A95DF3L, 0x9D684044L, 0x902B669DL, 0x94EA7B2AL,
89139 |     0xE0B41DE7L, 0xE4750050L, 0xE9362689L, 0xEDF73B3EL,
89140 |     0xF3B06B3BL, 0xF771768CL, 0xFA325055L, 0xFE734DE2L,
89141 |     0xC6BCF05FL, 0xC27DEDE8L, 0xCF3ECB31L, 0xCBFFD686L,
89142 |     0xD5B88683L, 0xD1799B34L, 0xDC3ABDEDL, 0xD8FBA05AL,
89143 |     0x690CE0EEL, 0x6DCDFD59L, 0x608EDB80L, 0x644FC637L,
89144 |     0x7A089632L, 0x7EC98B85L, 0x738AAD5CL, 0x774BB0EBL,
89145 |     0x4F040D56L, 0x4BC510E1L, 0x46863638L, 0x42472B8FL,
89146 |     0x5D007B8AL, 0x58C1663DL, 0x558240E4L, 0x51435D53L,
89147 |     0x251D3B9EL, 0x21DC2629L, 0x2C9F00F0L, 0x285E1D47L,
89148 |     0x36194D42L, 0x32D850F5L, 0x3F9B762CL, 0x3B5A6B9BL,
89149 |     0x0315D626L, 0x07D4CB91L, 0x0A97ED48L, 0x0E56F0FFL,
89150 |     0x1011A0FAL, 0x14D0BD4DL, 0x19939B94L, 0x1D528623L,
89151 |     0xF12F560EL, 0xF5EE4BB9L, 0xF8AD6D60L, 0xFC6C70D7L,
89152 |     0xE22B20D2L, 0xE6EA3D65L, 0xEBA91BBCL, 0xEF68060BL,
89153 |     0xD727BBB6L, 0xD3E6A601L, 0xDEA580D8L, 0xDA649D6FL,
89154 |     0xC423CD6AL, 0xC0E2D0DDL, 0xCDA1F604L, 0xC960EBB3L,
89155 |     0xBD3E8D7EL, 0xB9FF90C9L, 0xB4BCB610L, 0xB07DABA7L,
89156 |     0xAE3AFBA2L, 0xAAFBE615L, 0xA7B8C0CCL, 0xA379DD7BL,
89157 |     0x9B3660C6L, 0x9FF77D71L, 0x92B45BA8L, 0x9675461FL,
89158 |     0x8832161AL, 0x8CF30BADL, 0x81B02D74L, 0x857130C3L,
89159 |     0x5D8A9099L, 0x594B8D2EL, 0x5408ABF7L, 0x50C9B640L,
89160 |     0x4E8EE645L, 0x4A4FFBF2L, 0x470CDD2BL, 0x43CDC09CL,
89161 |     0x7B827D21L, 0x7F436096L, 0x7200464FL, 0x76C15BF8L,
89162 |     0x68860BFDL, 0x6C47164AL, 0x61043093L, 0x65C52D24L,
89163 |     0x119B4BE9L, 0x155A565EL, 0x18197087L, 0x1CD86D30L,
89164 |     0x029F3D35L, 0x065E2082L, 0x0B1D065BL, 0x0FDC1BECCL,
89165 |     0x3793A651L, 0x3352BBE6L, 0x3E119D3FL, 0x3AD08088L,
89166 |     0x2497D08DL, 0x2056CD3AL, 0x2D15EBE3L, 0x29D4F654L,
89167 |     0xC5A92679L, 0xC1683BCEL, 0xCC2B1D17L, 0xC8EA00A0L,
89168 |     0xD6AD50A5L, 0xD26CD12L, 0xDF2F6BCBL, 0xDBEE767CL,
89169 |     0xE3A1CBC1L, 0xE760D676L, 0xEA23F0AFL, 0xEE2ED18L,
89170 |     0xFA5BD1DL, 0xFA4A0AAL, 0xF9278673L, 0xFDE69BC4L,
89171 |     0x89B8FD09L, 0x8D79E0BEL, 0x803AC667L, 0x84FBDDB0L,
89172 |     0x9ABC8BD5L, 0x9E7D9662L, 0x933EB0BBL, 0x97FFAD0CL,
89173 |     0xAFB010B1L, 0xAB710D06L, 0xA6322BDFL, 0xA2F33668L,
```

```

89174 |     0xBCB4666DL, 0xB8757BDAL, 0xB5365D03L, 0xB1F740B4L
89175 | };
89176 | #endif
89177 | static MA_INLINE ma_uint32 ma_dr_flac_crc32_byte(ma_uint32 crc32, ma_uint8 data)
89178 | {
89179 | #ifndef MA_DR_FLAC_NO_CRC
89180 |     return (crc32 << 8) ^ ma_dr_flac__crc32_table[(ma_uint8)((crc32 >> 24) & 0xFF) ^ data];
89181 | #else
89182 |     (void)data;
89183 |     return crc32;
89184 | #endif
89185 | }
89186 | #if 0
89187 | static MA_INLINE ma_uint32 ma_dr_flac_crc32_uint32(ma_uint32 crc32, ma_uint32 data)
89188 | {
89189 |     crc32 = ma_dr_flac_crc32_byte(crc32, (ma_uint8)((data >> 24) & 0xFF));
89190 |     crc32 = ma_dr_flac_crc32_byte(crc32, (ma_uint8)((data >> 16) & 0xFF));
89191 |     crc32 = ma_dr_flac_crc32_byte(crc32, (ma_uint8)((data >> 8) & 0xFF));
89192 |     crc32 = ma_dr_flac_crc32_byte(crc32, (ma_uint8)((data >> 0) & 0xFF));
89193 |     return crc32;
89194 | }
89195 | static MA_INLINE ma_uint32 ma_dr_flac_crc32_uint64(ma_uint32 crc32, ma_uint64 data)
89196 | {
89197 |     crc32 = ma_dr_flac_crc32_uint32(crc32, (ma_uint32)((data >> 32) & 0xFFFFFFFF));
89198 |     crc32 = ma_dr_flac_crc32_uint32(crc32, (ma_uint32)((data >> 0) & 0xFFFFFFFF));
89199 |     return crc32;
89200 | }
89201 | #endif
89202 | static MA_INLINE ma_uint32 ma_dr_flac_crc32_buffer(ma_uint32 crc32, ma_uint8* pData, ma_uint32
> dataSize)
89203 | {
89204 |     ma_uint32 i;
89205 |     for (i = 0; i < dataSize; ++i) {
89206 |         crc32 = ma_dr_flac_crc32_byte(crc32, pData[i]);
89207 |     }
89208 |     return crc32;
89209 | }
89210 | static MA_INLINE ma_bool32 ma_dr_flac_ogg__is_capture_pattern(ma_uint8 pattern[4])
89211 | {
89212 |     return pattern[0] == '0' && pattern[1] == 'g' && pattern[2] == 'g' && pattern[3] == 'S';
89213 | }
89214 | static MA_INLINE ma_uint32 ma_dr_flac_ogg__get_page_header_size(ma_dr_flac_ogg_page_header* pHeader)
89215 | {
89216 |     return 27 + pHeader->segmentCount;
89217 | }
89218 | static MA_INLINE ma_uint32 ma_dr_flac_ogg__get_page_body_size(ma_dr_flac_ogg_page_header* pHeader)
89219 | {
89220 |     ma_uint32 pageBodySize = 0;
89221 |     int i;
89222 |     for (i = 0; i < pHeader->segmentCount; ++i) {
89223 |         pageBodySize += pHeader->segmentTable[i];
89224 |     }
89225 |     return pageBodySize;
89226 | }
89227 | static ma_result ma_dr_flac_ogg__read_page_header_after_capture_pattern(ma_dr_flac_read_proc onRead,
> void* pUserData, ma_dr_flac_ogg_page_header* pHeader, ma_uint32* pBytesRead, ma_uint32* pCRC32)
89228 | {
89229 |     ma_uint8 data[23];
89230 |     ma_uint32 i;
89231 |     MA_DR_FLAC_ASSERT(*pCRC32 == MA_DR_FLAC_OGG_CAPTURE_PATTERN_CRC32);
89232 |     if (onRead(pUserData, data, 23) != 23) {
89233 |         return MA_AT_END;
89234 |     }
89235 |     *pBytesRead += 23;
89236 |     pHeader->capturePattern[0] = '0';
89237 |     pHeader->capturePattern[1] = 'g';
89238 |     pHeader->capturePattern[2] = 'g';
89239 |     pHeader->capturePattern[3] = 'S';
89240 |     pHeader->structureVersion = data[0];
89241 |     pHeader->headerType = data[1];
89242 |     MA_DR_FLAC_COPY_MEMORY(&pHeader->granulePosition, &data[2], 8);
89243 |     MA_DR_FLAC_COPY_MEMORY(&pHeader->serialNumber, &data[10], 4);
89244 |     MA_DR_FLAC_COPY_MEMORY(&pHeader->sequenceNumber, &data[14], 4);
89245 |     MA_DR_FLAC_COPY_MEMORY(&pHeader->checksum, &data[18], 4);
89246 |     pHeader->segmentCount = data[22];
89247 |     data[18] = 0;
89248 |     data[19] = 0;
89249 |     data[20] = 0;
89250 |     data[21] = 0;

```

```

89251 |     for (i = 0; i < 23; ++i) {
89252 |         *pCRC32 = ma_dr_flac_crc32_byte(*pCRC32, data[i]);
89253 |     }
89254 |     if (onRead(pUserData, pHeader->segmentTable, pHeader->segmentCount) != pHeader->segmentCount) {
89255 |         return MA_AT_END;
89256 |     }
89257 |     *pBytesRead += pHeader->segmentCount;
89258 |     for (i = 0; i < pHeader->segmentCount; ++i) {
89259 |         *pCRC32 = ma_dr_flac_crc32_byte(*pCRC32, pHeader->segmentTable[i]);
89260 |     }
89261 |     return MA_SUCCESS;
89262 | }
89263 | static ma_result ma_dr_flac_ogg__read_page_header(ma_dr_flac_read_proc onRead, void* pUserData,
> ma_dr_flac_ogg_page_header* pHeader, ma_uint32* pBytesRead, ma_uint32* pCRC32)
89264 | {
89265 |     ma_uint8 id[4];
89266 |     *pBytesRead = 0;
89267 |     if (onRead(pUserData, id, 4) != 4) {
89268 |         return MA_AT_END;
89269 |     }
89270 |     *pBytesRead += 4;
89271 |     for (;;) {
89272 |         if (ma_dr_flac_ogg__is_capture_pattern(id)) {
89273 |             ma_result result;
89274 |             *pCRC32 = MA_DR_FLAC_OGG_CAPTURE_PATTERN_CRC32;
89275 |             result = ma_dr_flac_ogg__read_page_header_after_capture_pattern(onRead, pUserData,
> pHeader, pBytesRead, pCRC32);
89276 |             if (result == MA_SUCCESS) {
89277 |                 return MA_SUCCESS;
89278 |             } else {
89279 |                 if (result == MA_CRC_MISMATCH) {
89280 |                     continue;
89281 |                 } else {
89282 |                     return result;
89283 |                 }
89284 |             }
89285 |         } else {
89286 |             id[0] = id[1];
89287 |             id[1] = id[2];
89288 |             id[2] = id[3];
89289 |             if (onRead(pUserData, &id[3], 1) != 1) {
89290 |                 return MA_AT_END;
89291 |             }
89292 |             *pBytesRead += 1;
89293 |         }
89294 |     }
89295 | }
89296 | typedef struct
89297 | {
89298 |     ma_dr_flac_read_proc onRead;
89299 |     ma_dr_flac_seek_proc onSeek;
89300 |     ma_dr_flac_tell_proc onTell;
89301 |     void* pUserData;
89302 |     ma_uint64 currentBytePos;
89303 |     ma_uint64 firstBytePos;
89304 |     ma_uint32 serialNumber;
89305 |     ma_dr_flac_ogg_page_header bosPageHeader;
89306 |     ma_dr_flac_ogg_page_header currentPageHeader;
89307 |     ma_uint32 bytesRemainingInPage;
89308 |     ma_uint32 pageDataSize;
89309 |     ma_uint8 pageData[MA_DR_FLAC_OGG_MAX_PAGE_SIZE];
89310 | } ma_dr_flac_oggbs;
89311 | static size_t ma_dr_flac_oggbs__read_physical(ma_dr_flac_oggbs* oggbs, void* bufferOut, size_t
> bytesToRead)
89312 | {
89313 |     size_t bytesActuallyRead = oggbs->onRead(oggbs->pUserData, bufferOut, bytesToRead);
89314 |     oggbs->currentBytePos += bytesActuallyRead;
89315 |     return bytesActuallyRead;
89316 | }
89317 | static ma_bool32 ma_dr_flac_oggbs__seek_physical(ma_dr_flac_oggbs* oggbs, ma_uint64 offset,
> ma_dr_flac_seek_origin origin)
89318 | {
89319 |     if (origin == MA_DR_FLAC_SEEK_SET) {
89320 |         if (offset <= 0x7FFFFFFF) {
89321 |             if (!oggbs->onSeek(oggbs->pUserData, (int)offset, MA_DR_FLAC_SEEK_SET)) {
89322 |                 return MA_FALSE;
89323 |             }
89324 |             oggbs->currentBytePos = offset;
89325 |             return MA_TRUE;

```

```

89326 |         } else {
89327 |             if (!oggbs->onSeek(oggbs->pUserData, 0x7FFFFFFF, MA_DR_FLAC_SEEK_SET)) {
89328 |                 return MA_FALSE;
89329 |             }
89330 |             oggbs->currentBytePos = offset;
89331 |             return ma_dr_flac_oggbs__seek_physical(oggbs, offset - 0x7FFFFFFF, MA_DR_FLAC_SEEK_CUR);
89332 |         }
89333 |     } else {
89334 |         while (offset > 0x7FFFFFFF) {
89335 |             if (!oggbs->onSeek(oggbs->pUserData, 0x7FFFFFFF, MA_DR_FLAC_SEEK_CUR)) {
89336 |                 return MA_FALSE;
89337 |             }
89338 |             oggbs->currentBytePos += 0x7FFFFFFF;
89339 |             offset -= 0x7FFFFFFF;
89340 |         }
89341 |         if (!oggbs->onSeek(oggbs->pUserData, (int)offset, MA_DR_FLAC_SEEK_CUR)) {
89342 |             return MA_FALSE;
89343 |         }
89344 |         oggbs->currentBytePos += offset;
89345 |         return MA_TRUE;
89346 |     }
89347 | }
89348 | static ma_bool32 ma_dr_flac_oggbs__goto_next_page(ma_dr_flac_oggbs* oggbs,
> ma_dr_flac_ogg_crc_mismatch_recovery recoveryMethod)
89349 | {
89350 |     ma_dr_flac_ogg_page_header header;
89351 |     for (;;) {
89352 |         ma_uint32 crc32 = 0;
89353 |         ma_uint32 bytesRead;
89354 |         ma_uint32 pageBodySize;
89355 | #ifndef MA_DR_FLAC_NO_CRC
89356 |         ma_uint32 actualCRC32;
89357 | #endif
89358 |         if (ma_dr_flac_ogg__read_page_header(oggbs->onRead, oggbs->pUserData, &header, &bytesRead,
> &crc32) != MA_SUCCESS) {
89359 |             return MA_FALSE;
89360 |         }
89361 |         oggbs->currentBytePos += bytesRead;
89362 |         pageBodySize = ma_dr_flac_ogg__get_page_body_size(&header);
89363 |         if (pageBodySize > MA_DR_FLAC_OGG_MAX_PAGE_SIZE) {
89364 |             continue;
89365 |         }
89366 |         if (header.serialNumber != oggbs->serialNumber) {
89367 |             if (pageBodySize > 0 && !ma_dr_flac_oggbs__seek_physical(oggbs, pageBodySize,
> MA_DR_FLAC_SEEK_CUR)) {
89368 |                 return MA_FALSE;
89369 |             }
89370 |             continue;
89371 |         }
89372 |         if (ma_dr_flac_oggbs__read_physical(oggbs, oggbs->pageData, pageBodySize) != pageBodySize) {
89373 |             return MA_FALSE;
89374 |         }
89375 |         oggbs->pageDataSize = pageBodySize;
89376 | #ifndef MA_DR_FLAC_NO_CRC
89377 |         actualCRC32 = ma_dr_flac_crc32_buffer(crc32, oggbs->pageData, oggbs->pageDataSize);
89378 |         if (actualCRC32 != header.checksum) {
89379 |             if (recoveryMethod == ma_dr_flac_ogg_recover_on_crc_mismatch) {
89380 |                 continue;
89381 |             } else {
89382 |                 ma_dr_flac_oggbs__goto_next_page(oggbs, ma_dr_flac_ogg_recover_on_crc_mismatch);
89383 |                 return MA_FALSE;
89384 |             }
89385 |         }
89386 |     } #else
89387 |     (void)recoveryMethod;
89388 | #endif
89389 |     oggbs->currentPageHeader = header;
89390 |     oggbs->bytesRemainingInPage = pageBodySize;
89391 |     return MA_TRUE;
89392 | }
89393 | }
89394 | #if 0
89395 | static ma_uint8 ma_dr_flac_oggbs__get_current_segment_index(ma_dr_flac_oggbs* oggbs, ma_uint8*
> pBytesRemainingInSeg)
89396 | {
89397 |     ma_uint32 bytesConsumedInPage = ma_dr_flac_ogg__get_page_body_size(&oggbs->currentPageHeader) -
> oggbs->bytesRemainingInPage;
89398 |     ma_uint8 iSeg = 0;
89399 |     ma_uint32 iByte = 0;

```

```

89400 |     while (iByte < bytesConsumedInPage) {
89401 |         ma_uint8 segmentSize = oggbs->currentPageHeader.segmentTable[iSeg];
89402 |         if (iByte + segmentSize > bytesConsumedInPage) {
89403 |             break;
89404 |         } else {
89405 |             iSeg += 1;
89406 |             iByte += segmentSize;
89407 |         }
89408 |     }
89409 |     *pBytesRemainingInSeg = oggbs->currentPageHeader.segmentTable[iSeg] -
> (ma_uint8)(bytesConsumedInPage - iByte);
89410 |     return iSeg;
89411 | }
89412 | static ma_bool32 ma_dr_flac_oggbs__seek_to_next_packet(ma_dr_flac_oggbs* oggbs)
89413 | {
89414 |     for (;;) {
89415 |         ma_bool32 atEndOfPage = MA_FALSE;
89416 |         ma_uint8 bytesRemainingInSeg;
89417 |         ma_uint8 iFirstSeg = ma_dr_flac_oggbs__get_current_segment_index(oggbs,
> &bytesRemainingInSeg);
89418 |         ma_uint32 bytesToEndOfPacketOrPage = bytesRemainingInSeg;
89419 |         for (ma_uint8 iSeg = iFirstSeg; iSeg < oggbs->currentPageHeader.segmentCount; ++iSeg) {
89420 |             ma_uint8 segmentSize = oggbs->currentPageHeader.segmentTable[iSeg];
89421 |             if (segmentSize < 255) {
89422 |                 if (iSeg == oggbs->currentPageHeader.segmentCount-1) {
89423 |                     atEndOfPage = MA_TRUE;
89424 |                 }
89425 |                 break;
89426 |             }
89427 |             bytesToEndOfPacketOrPage += segmentSize;
89428 |         }
89429 |         ma_dr_flac_oggbs__seek_physical(oggbs, bytesToEndOfPacketOrPage, MA_DR_FLAC_SEEK_CUR);
89430 |         oggbs->bytesRemainingInPage -= bytesToEndOfPacketOrPage;
89431 |         if (atEndOfPage) {
89432 |             if (!ma_dr_flac_oggbs__goto_next_page(oggbs)) {
89433 |                 return MA_FALSE;
89434 |             }
89435 |             if ((oggbs->currentPageHeader.headerType & 0x01) == 0) {
89436 |                 return MA_TRUE;
89437 |             }
89438 |         } else {
89439 |             return MA_TRUE;
89440 |         }
89441 |     }
89442 | }
89443 | static ma_bool32 ma_dr_flac_oggbs__seek_to_next_frame(ma_dr_flac_oggbs* oggbs)
89444 | {
89445 |     return ma_dr_flac_oggbs__seek_to_next_packet(oggbs);
89446 | }
89447 | #endif
89448 | static size_t ma_dr_flac__on_read_ogg(void* pUserData, void* bufferOut, size_t bytesToRead)
89449 | {
89450 |     ma_dr_flac_oggbs* oggbs = (ma_dr_flac_oggbs*)pUserData;
89451 |     ma_uint8* pRunningBufferOut = (ma_uint8*)bufferOut;
89452 |     size_t bytesRead = 0;
89453 |     MA_DR_FLAC_ASSERT(oggbs != NULL);
89454 |     MA_DR_FLAC_ASSERT(pRunningBufferOut != NULL);
89455 |     while (bytesRead < bytesToRead) {
89456 |         size_t bytesRemainingToRead = bytesToRead - bytesRead;
89457 |         if (oggbs->bytesRemainingInPage >= bytesRemainingToRead) {
89458 |             MA_DR_FLAC_COPY_MEMORY(pRunningBufferOut, oggbs->pageData + (oggbs->pageDataSize -
> oggbs->bytesRemainingInPage), bytesRemainingToRead);
89459 |             bytesRead += bytesRemainingToRead;
89460 |             oggbs->bytesRemainingInPage -= (ma_uint32)bytesRemainingToRead;
89461 |             break;
89462 |         }
89463 |         if (oggbs->bytesRemainingInPage > 0) {
89464 |             MA_DR_FLAC_COPY_MEMORY(pRunningBufferOut, oggbs->pageData + (oggbs->pageDataSize -
> oggbs->bytesRemainingInPage), oggbs->bytesRemainingInPage);
89465 |             bytesRead += oggbs->bytesRemainingInPage;
89466 |             pRunningBufferOut += oggbs->bytesRemainingInPage;
89467 |             oggbs->bytesRemainingInPage = 0;
89468 |         }
89469 |         MA_DR_FLAC_ASSERT(bytesRemainingToRead > 0);
89470 |         if (!ma_dr_flac_oggbs__goto_next_page(oggbs, ma_dr_flac_ogg_recover_on_crc_mismatch)) {
89471 |             break;
89472 |         }
89473 |     }
89474 |     return bytesRead;

```



```

89475 | }
89476 | static ma_bool32 ma_dr_flac__on_seek_ogg(void* pUserData, int offset, ma_dr_flac_seek_origin origin)
89477 | {
89478 |     ma_dr_flac_oggbs* oggbs = (ma_dr_flac_oggbs*)pUserData;
89479 |     int bytesSeeked = 0;
89480 |     MA_DR_FLAC_ASSERT(oggbs != NULL);
89481 |     MA_DR_FLAC_ASSERT(offset >= 0);
89482 |     if (origin == MA_DR_FLAC_SEEK_SET) {
89483 |         if (!ma_dr_flac_oggbs__seek_physical(oggbs, (int)oggbs->firstBytePos, MA_DR_FLAC_SEEK_SET))
> {
89484 |             return MA_FALSE;
89485 |         }
89486 |         if (!ma_dr_flac_oggbs__goto_next_page(oggbs, ma_dr_flac_ogg_fail_on_crc_mismatch)) {
89487 |             return MA_FALSE;
89488 |         }
89489 |         return ma_dr_flac__on_seek_ogg(pUserData, offset, MA_DR_FLAC_SEEK_CUR);
89490 |     } else if (origin == MA_DR_FLAC_SEEK_CUR) {
89491 |         while (bytesSeeked < offset) {
89492 |             int bytesRemainingToSeek = offset - bytesSeeked;
89493 |             MA_DR_FLAC_ASSERT(bytesRemainingToSeek >= 0);
89494 |             if (oggbs->bytesRemainingInPage >= (size_t)bytesRemainingToSeek) {
89495 |                 bytesSeeked += bytesRemainingToSeek;
89496 |                 (void)bytesSeeked;
89497 |                 oggbs->bytesRemainingInPage -= bytesRemainingToSeek;
89498 |                 break;
89499 |             }
89500 |             if (oggbs->bytesRemainingInPage > 0) {
89501 |                 bytesSeeked += (int)oggbs->bytesRemainingInPage;
89502 |                 oggbs->bytesRemainingInPage = 0;
89503 |             }
89504 |             MA_DR_FLAC_ASSERT(bytesRemainingToSeek > 0);
89505 |             if (!ma_dr_flac_oggbs__goto_next_page(oggbs, ma_dr_flac_ogg_fail_on_crc_mismatch)) {
89506 |                 return MA_FALSE;
89507 |             }
89508 |         }
89509 |     } else if (origin == MA_DR_FLAC_SEEK_END) {
89510 |         return MA_FALSE;
89511 |     }
89512 |     return MA_TRUE;
89513 | }
89514 | static ma_bool32 ma_dr_flac__on_tell_ogg(void* pUserData, ma_int64* pCursor)
89515 | {
89516 |     (void)pUserData;
89517 |     (void)pCursor;
89518 |     return MA_FALSE;
89519 | }
89520 | static ma_bool32 ma_dr_flac_ogg__seek_to_pcm_frame(ma_dr_flac* pFlac, ma_uint64 pcmFrameIndex)
89521 | {
89522 |     ma_dr_flac_oggbs* oggbs = (ma_dr_flac_oggbs*)pFlac->oggbs;
89523 |     ma_uint64 originalBytePos;
89524 |     ma_uint64 runningGranulePosition;
89525 |     ma_uint64 runningFrameBytePos;
89526 |     ma_uint64 runningPCMFrameCount;
89527 |     MA_DR_FLAC_ASSERT(oggbs != NULL);
89528 |     originalBytePos = oggbs->currentBytePos;
89529 |     if (!ma_dr_flac__seek_to_byte(&pFlac->bs, pFlac->firstFLACFramePosInBytes)) {
89530 |         return MA_FALSE;
89531 |     }
89532 |     oggbs->bytesRemainingInPage = 0;
89533 |     runningGranulePosition = 0;
89534 |     for (;;) {
89535 |         if (!ma_dr_flac_oggbs__goto_next_page(oggbs, ma_dr_flac_ogg_recover_on_crc_mismatch)) {
89536 |             ma_dr_flac_oggbs__seek_physical(oggbs, originalBytePos, MA_DR_FLAC_SEEK_SET);
89537 |             return MA_FALSE;
89538 |         }
89539 |         runningFrameBytePos = oggbs->currentBytePos -
> ma_dr_flac_ogg__get_page_header_size(&oggbs->currentPageHeader) - oggbs->pageDataSize;
89540 |         if (oggbs->currentPageHeader.granulePosition >= pcmFrameIndex) {
89541 |             break;
89542 |         }
89543 |         if ((oggbs->currentPageHeader.headerType & 0x01) == 0) {
89544 |             if (oggbs->currentPageHeader.segmentTable[0] >= 2) {
89545 |                 ma_uint8 firstBytesInPage[2];
89546 |                 firstBytesInPage[0] = oggbs->pageData[0];
89547 |                 firstBytesInPage[1] = oggbs->pageData[1];
89548 |                 if ((firstBytesInPage[0] == 0xFF) && (firstBytesInPage[1] & 0xFC) == 0xF8) {
89549 |                     runningGranulePosition = oggbs->currentPageHeader.granulePosition;
89550 |                 }
89551 |                 continue;

```

```

89552 |     }
89553 | }
89554 | }
89555 | if (!ma_dr_flac_oggbs__seek_physical(oggbs, runningFrameBytePos, MA_DR_FLAC_SEEK_SET)) {
89556 |     return MA_FALSE;
89557 | }
89558 | if (!ma_dr_flac_oggbs__goto_next_page(oggbs, ma_dr_flac_ogg_recover_on_crc_mismatch)) {
89559 |     return MA_FALSE;
89560 | }
89561 | runningPCMFrameCount = runningGranulePosition;
89562 | for (;;) {
89563 |     ma_uint64 firstPCMFrameInFLACFrame = 0;
89564 |     ma_uint64 lastPCMFrameInFLACFrame = 0;
89565 |     ma_uint64 pcmFrameCountInThisFrame;
89566 |     if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
89567 |         return MA_FALSE;
89568 |     }
89569 |     ma_dr_flac__get_pcm_frame_range_of_current_flac_frame(pFlac, &firstPCMFrameInFLACFrame,
> &lastPCMFrameInFLACFrame);
89570 |     pcmFrameCountInThisFrame = (lastPCMFrameInFLACFrame - firstPCMFrameInFLACFrame) + 1;
89571 |     if (pcmFrameIndex == pFlac->totalPCMFrameCount && (runningPCMFrameCount +
> pcmFrameCountInThisFrame) == pFlac->totalPCMFrameCount) {
89572 |         ma_result result = ma_dr_flac__decode_flac_frame(pFlac);
89573 |         if (result == MA_SUCCESS) {
89574 |             pFlac->currentPCMFrame = pcmFrameIndex;
89575 |             pFlac->currentFLACFrame.pcmFramesRemaining = 0;
89576 |             return MA_TRUE;
89577 |         } else {
89578 |             return MA_FALSE;
89579 |         }
89580 |     }
89581 |     if (pcmFrameIndex < (runningPCMFrameCount + pcmFrameCountInThisFrame)) {
89582 |         ma_result result = ma_dr_flac__decode_flac_frame(pFlac);
89583 |         if (result == MA_SUCCESS) {
89584 |             ma_uint64 pcmFramesToDecode = (size_t)(pcmFrameIndex - runningPCMFrameCount);
89585 |             if (pcmFramesToDecode == 0) {
89586 |                 return MA_TRUE;
89587 |             }
89588 |             pFlac->currentPCMFrame = runningPCMFrameCount;
89589 |             return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, pcmFramesToDecode) ==
> pcmFramesToDecode;
89590 |         } else {
89591 |             if (result == MA_CRC_MISMATCH) {
89592 |                 continue;
89593 |             } else {
89594 |                 return MA_FALSE;
89595 |             }
89596 |         }
89597 |     } else {
89598 |         ma_result result = ma_dr_flac__seek_to_next_flac_frame(pFlac);
89599 |         if (result == MA_SUCCESS) {
89600 |             runningPCMFrameCount += pcmFrameCountInThisFrame;
89601 |         } else {
89602 |             if (result == MA_CRC_MISMATCH) {
89603 |                 continue;
89604 |             } else {
89605 |                 return MA_FALSE;
89606 |             }
89607 |         }
89608 |     }
89609 | }
89610 | }
89611 | static ma_bool32 ma_dr_flac__init_private_ogg(ma_dr_flac_init_info* pInit, ma_dr_flac_read_proc
> onRead, ma_dr_flac_seek_proc onSeek, ma_dr_flac_meta_proc onMeta, void* pUserData, void*
> pUserDataMD, ma_bool32 relaxed)
89612 | {
89613 |     ma_dr_flac_ogg_page_header header;
89614 |     ma_uint32 crc32 = MA_DR_FLAC_OGG_CAPTURE_PATTERN_CRC32;
89615 |     ma_uint32 bytesRead = 0;
89616 |     (void)relaxed;
89617 |     pInit->container = ma_dr_flac_container_ogg;
89618 |     pInit->oggFirstBytePos = 0;
89619 |     if (ma_dr_flac_ogg__read_page_header_after_capture_pattern(onRead, pUserData, &header,
> &bytesRead, &crc32) != MA_SUCCESS) {
89620 |         return MA_FALSE;
89621 |     }
89622 |     pInit->runningFilePos += bytesRead;
89623 |     for (;;) {

```

```

89624 |         int pageBodySize;
89625 |         if ((header.headerType & 0x02) == 0) {
89626 |             return MA_FALSE;
89627 |         }
89628 |         pageBodySize = ma_dr_flac_ogg__get_page_body_size(&header);
89629 |         if (pageBodySize == 51) {
89630 |             ma_uint32 bytesRemainingInPage = pageBodySize;
89631 |             ma_uint8 packetType;
89632 |             if (onRead(pUserData, &packetType, 1) != 1) {
89633 |                 return MA_FALSE;
89634 |             }
89635 |             bytesRemainingInPage -= 1;
89636 |             if (packetType == 0x7F) {
89637 |                 ma_uint8 sig[4];
89638 |                 if (onRead(pUserData, sig, 4) != 4) {
89639 |                     return MA_FALSE;
89640 |                 }
89641 |                 bytesRemainingInPage -= 4;
89642 |                 if (sig[0] == 'F' && sig[1] == 'L' && sig[2] == 'A' && sig[3] == 'C') {
89643 |                     ma_uint8 mappingVersion[2];
89644 |                     if (onRead(pUserData, mappingVersion, 2) != 2) {
89645 |                         return MA_FALSE;
89646 |                     }
89647 |                     if (mappingVersion[0] != 1) {
89648 |                         return MA_FALSE;
89649 |                     }
89650 |                     if (!onSeek(pUserData, 2, MA_DR_FLAC_SEEK_CUR)) {
89651 |                         return MA_FALSE;
89652 |                     }
89653 |                     if (onRead(pUserData, sig, 4) != 4) {
89654 |                         return MA_FALSE;
89655 |                     }
89656 |                     if (sig[0] == 'f' && sig[1] == 'L' && sig[2] == 'a' && sig[3] == 'C') {
89657 |                         ma_dr_flac_streaminfo streaminfo;
89658 |                         ma_uint8 isLastBlock;
89659 |                         ma_uint8 blockType;
89660 |                         ma_uint32 blockSize;
89661 |                         if (!ma_dr_flac__read_and_decode_block_header(onRead, pUserData,
> &isLastBlock, &blockType, &blockSize)) {
89662 |                             return MA_FALSE;
89663 |                         }
89664 |                         if (blockType != MA_DR_FLAC_METADATA_BLOCK_TYPE_STREAMINFO || blockSize !=
> 34) {
89665 |                             return MA_FALSE;
89666 |                         }
89667 |                         if (ma_dr_flac__read_streaminfo(onRead, pUserData, &streaminfo)) {
89668 |                             pInit->hasStreamInfoBlock = MA_TRUE;
89669 |                             pInit->sampleRate = streaminfo.sampleRate;
89670 |                             pInit->channels = streaminfo.channels;
89671 |                             pInit->bitsPerSample = streaminfo.bitsPerSample;
89672 |                             pInit->totalPCMFrameCount = streaminfo.totalPCMFrameCount;
89673 |                             pInit->maxBlockSizeInPCMFrames = streaminfo.maxBlockSizeInPCMFrames;
89674 |                             pInit->hasMetadataBlocks = !isLastBlock;
89675 |                             if (onMeta) {
89676 |                                 ma_dr_flac_metadata metadata;
89677 |                                 metadata.type = MA_DR_FLAC_METADATA_BLOCK_TYPE_STREAMINFO;
89678 |                                 metadata.pRawData = NULL;
89679 |                                 metadata.rawDataSize = 0;
89680 |                                 metadata.data.streaminfo = streaminfo;
89681 |                                 onMeta(pUserDataMD, &metadata);
89682 |                             }
89683 |                             pInit->runningFilePos += pageBodySize;
89684 |                             pInit->oggFirstBytePos = pInit->runningFilePos - 79;
89685 |                             pInit->oggSerial = header.serialNumber;
89686 |                             pInit->oggBosHeader = header;
89687 |                             break;
89688 |                         } else {
89689 |                             return MA_FALSE;
89690 |                         }
89691 |                     } else {
89692 |                         return MA_FALSE;
89693 |                     }
89694 |                 } else {
89695 |                     if (!onSeek(pUserData, bytesRemainingInPage, MA_DR_FLAC_SEEK_CUR)) {
89696 |                         return MA_FALSE;
89697 |                     }
89698 |                 }
89699 |             } else {
89700 |                 if (!onSeek(pUserData, bytesRemainingInPage, MA_DR_FLAC_SEEK_CUR)) {

```

```

89701 |         return MA_FALSE;
89702 |     }
89703 | }
89704 | } else {
89705 |     if (!onSeek(pUserData, pageBodySize, MA_DR_FLAC_SEEK_CUR)) {
89706 |         return MA_FALSE;
89707 |     }
89708 | }
89709 | pInit->runningFilePos += pageBodySize;
89710 | if (ma_dr_flac_ogg__read_page_header(onRead, pUserData, &header, &bytesRead, &crc32) !=
> MA_SUCCESS) {
89711 |     return MA_FALSE;
89712 | }
89713 | pInit->runningFilePos += bytesRead;
89714 | }
89715 | pInit->hasMetadataBlocks = MA_TRUE;
89716 | return MA_TRUE;
89717 | }
89718 | #endif
89719 | static ma_bool32 ma_dr_flac__init_private(ma_dr_flac_init_info* pInit, ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta,
> ma_dr_flac_container container, void* pUserData, void* pUserDataMD)
89720 | {
89721 |     ma_bool32 relaxed;
89722 |     ma_uint8 id[4];
89723 |     if (pInit == NULL || onRead == NULL || onSeek == NULL) {
89724 |         return MA_FALSE;
89725 |     }
89726 |     MA_DR_FLAC_ZERO_MEMORY(pInit, sizeof(*pInit));
89727 |     pInit->onRead = onRead;
89728 |     pInit->onSeek = onSeek;
89729 |     pInit->onTell = onTell;
89730 |     pInit->onMeta = onMeta;
89731 |     pInit->container = container;
89732 |     pInit->pUserData = pUserData;
89733 |     pInit->pUserDataMD = pUserDataMD;
89734 |     pInit->bs.onRead = onRead;
89735 |     pInit->bs.onSeek = onSeek;
89736 |     pInit->bs.onTell = onTell;
89737 |     pInit->bs.pUserData = pUserData;
89738 |     ma_dr_flac__reset_cache(&pInit->bs);
89739 |     relaxed = container != ma_dr_flac_container_unknown;
89740 |     for (;;) {
89741 |         if (onRead(pUserData, id, 4) != 4) {
89742 |             return MA_FALSE;
89743 |         }
89744 |         pInit->runningFilePos += 4;
89745 |         if (id[0] == 'I' && id[1] == 'D' && id[2] == '3') {
89746 |             ma_uint8 header[6];
89747 |             ma_uint8 flags;
89748 |             ma_uint32 headerSize;
89749 |             if (onRead(pUserData, header, 6) != 6) {
89750 |                 return MA_FALSE;
89751 |             }
89752 |             pInit->runningFilePos += 6;
89753 |             flags = header[1];
89754 |             MA_DR_FLAC_COPY_MEMORY(&headerSize, header+2, 4);
89755 |             headerSize = ma_dr_flac__unsynsafe_32(ma_dr_flac__be2host_32(headerSize));
89756 |             if (flags & 0x10) {
89757 |                 headerSize += 10;
89758 |             }
89759 |             if (!onSeek(pUserData, headerSize, MA_DR_FLAC_SEEK_CUR)) {
89760 |                 return MA_FALSE;
89761 |             }
89762 |             pInit->runningFilePos += headerSize;
89763 |         } else {
89764 |             break;
89765 |         }
89766 |     }
89767 |     if (id[0] == 'f' && id[1] == 'L' && id[2] == 'a' && id[3] == 'C') {
89768 |         return ma_dr_flac__init_private__native(pInit, onRead, onSeek, onMeta, pUserData,
> pUserDataMD, relaxed);
89769 |     }
89770 | #ifndef MA_DR_FLAC_NO_OGG
89771 |     if (id[0] == 'O' && id[1] == 'g' && id[2] == 'g' && id[3] == 'S') {
89772 |         return ma_dr_flac__init_private__ogg(pInit, onRead, onSeek, onMeta, pUserData, pUserDataMD,
> relaxed);
89773 |     }
89774 | #endif

```

```

89775 |     if (relaxed) {
89776 |         if (container == ma_dr_flac_container_native) {
89777 |             return ma_dr_flac__init_private_native(pInit, onRead, onSeek, onMeta, pUserData,
> pUserDataMD, relaxed);
89778 |         }
89779 | #ifndef MA_DR_FLAC_NO_OGG
89780 |         if (container == ma_dr_flac_container_ogg) {
89781 |             return ma_dr_flac__init_private_ogg(pInit, onRead, onSeek, onMeta, pUserData,
> pUserDataMD, relaxed);
89782 |         }
89783 | #endif
89784 |     }
89785 |     return MA_FALSE;
89786 | }
89787 | static void ma_dr_flac__init_from_info(ma_dr_flac* pFlac, const ma_dr_flac_init_info* pInit)
89788 | {
89789 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
89790 |     MA_DR_FLAC_ASSERT(pInit != NULL);
89791 |     MA_DR_FLAC_ZERO_MEMORY(pFlac, sizeof(*pFlac));
89792 |     pFlac->bs = pInit->bs;
89793 |     pFlac->onMeta = pInit->onMeta;
89794 |     pFlac->pUserDataMD = pInit->pUserDataMD;
89795 |     pFlac->maxBlockSizeInPCMFrames = pInit->maxBlockSizeInPCMFrames;
89796 |     pFlac->sampleRate = pInit->sampleRate;
89797 |     pFlac->channels = (ma_uint8)pInit->channels;
89798 |     pFlac->bitsPerSample = (ma_uint8)pInit->bitsPerSample;
89799 |     pFlac->totalPCMFrameCount = pInit->totalPCMFrameCount;
89800 |     pFlac->container = pInit->container;
89801 | }
89802 | static ma_dr_flac* ma_dr_flac_open_with_metadata_private(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta,
> ma_dr_flac_container container, void* pUserData, void* pUserDataMD, const ma_allocation_callbacks*
> pAllocationCallbacks)
89803 | {
89804 |     ma_dr_flac_init_info init;
89805 |     ma_uint32 allocationSize;
89806 |     ma_uint32 wholeSIMDVectorCountPerChannel;
89807 |     ma_uint32 decodedSamplesAllocationSize;
89808 | #ifndef MA_DR_FLAC_NO_OGG
89809 |     ma_dr_flac_oggbs* pOggbs = NULL;
89810 | #endif
89811 |     ma_uint64 firstFramePos;
89812 |     ma_uint64 seektablePos;
89813 |     ma_uint32 seekpointCount;
89814 |     ma_allocation_callbacks allocationCallbacks;
89815 |     ma_dr_flac* pFlac;
89816 |     ma_dr_flac__init_cpu_caps();
89817 |     if (!ma_dr_flac__init_private(&init, onRead, onSeek, onTell, onMeta, container, pUserData,
> pUserDataMD)) {
89818 |         return NULL;
89819 |     }
89820 |     if (pAllocationCallbacks != NULL) {
89821 |         allocationCallbacks = *pAllocationCallbacks;
89822 |         if (allocationCallbacks.onFree == NULL || (allocationCallbacks.onMalloc == NULL &&
> allocationCallbacks.onRealloc == NULL)) {
89823 |             return NULL;
89824 |         }
89825 |     } else {
89826 |         allocationCallbacks.pUserData = NULL;
89827 |         allocationCallbacks.onMalloc = ma_dr_flac__malloc_default;
89828 |         allocationCallbacks.onRealloc = ma_dr_flac__realloc_default;
89829 |         allocationCallbacks.onFree = ma_dr_flac__free_default;
89830 |     }
89831 |     allocationSize = sizeof(ma_dr_flac);
89832 |     if ((init.maxBlockSizeInPCMFrames % (MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE / sizeof(ma_int32))) == 0)
> {
89833 |         wholeSIMDVectorCountPerChannel = (init.maxBlockSizeInPCMFrames /
> (MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE / sizeof(ma_int32)));
89834 |     } else {
89835 |         wholeSIMDVectorCountPerChannel = (init.maxBlockSizeInPCMFrames /
> (MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE / sizeof(ma_int32))) + 1;
89836 |     }
89837 |     decodedSamplesAllocationSize = wholeSIMDVectorCountPerChannel * MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE
> * init.channels;
89838 |     allocationSize += decodedSamplesAllocationSize;
89839 |     allocationSize += MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE;
89840 | #ifndef MA_DR_FLAC_NO_OGG
89841 |     if (init.container == ma_dr_flac_container_ogg) {
89842 |         allocationSize += sizeof(ma_dr_flac_oggbs);

```

```

89843 |         pOggbs = (ma_dr_flac_oggbs*)ma_dr_flac__malloc_from_callbacks(sizeof(*pOggbs),
> &allocationCallbacks);
89844 |         if (pOggbs == NULL) {
89845 |             return NULL;
89846 |         }
89847 |         MA_DR_FLAC_ZERO_MEMORY(pOggbs, sizeof(*pOggbs));
89848 |         pOggbs->onRead = onRead;
89849 |         pOggbs->onSeek = onSeek;
89850 |         pOggbs->onTell = onTell;
89851 |         pOggbs->pUserData = pUserData;
89852 |         pOggbs->currentBytePos = init.oggFirstBytePos;
89853 |         pOggbs->firstBytePos = init.oggFirstBytePos;
89854 |         pOggbs->serialNumber = init.oggSerial;
89855 |         pOggbs->bosPageHeader = init.oggBosHeader;
89856 |         pOggbs->bytesRemainingInPage = 0;
89857 |     }
89858 | #endif
89859 |     firstFramePos = 42;
89860 |     seektablePos = 0;
89861 |     seekpointCount = 0;
89862 |     if (init.hasMetadataBlocks) {
89863 |         ma_dr_flac_read_proc onReadOverride = onRead;
89864 |         ma_dr_flac_seek_proc onSeekOverride = onSeek;
89865 |         ma_dr_flac_tell_proc onTellOverride = onTell;
89866 |         void* pUserDataOverride = pUserData;
89867 | #ifndef MA_DR_FLAC_NO_OGG
89868 |         if (init.container == ma_dr_flac_container_ogg) {
89869 |             onReadOverride = ma_dr_flac__on_read_ogg;
89870 |             onSeekOverride = ma_dr_flac__on_seek_ogg;
89871 |             onTellOverride = ma_dr_flac__on_tell_ogg;
89872 |             pUserDataOverride = (void*)pOggbs;
89873 |         }
89874 | #endif
89875 |         if (!ma_dr_flac_read_and_decode_metadata(onReadOverride, onSeekOverride, onTellOverride,
> onMeta, pUserDataOverride, pUserDataMD, &firstFramePos, &seektablePos, &seekpointCount,
> &allocationCallbacks)) {
89876 |             #ifndef MA_DR_FLAC_NO_OGG
89877 |                 ma_dr_flac__free_from_callbacks(pOggbs, &allocationCallbacks);
89878 |             #endif
89879 |             return NULL;
89880 |         }
89881 |         allocationSize += seekpointCount * sizeof(ma_dr_flac_seekpoint);
89882 |     }
89883 |     pFlac = (ma_dr_flac*)ma_dr_flac__malloc_from_callbacks(allocationSize, &allocationCallbacks);
89884 |     if (pFlac == NULL) {
89885 |         #ifndef MA_DR_FLAC_NO_OGG
89886 |             ma_dr_flac__free_from_callbacks(pOggbs, &allocationCallbacks);
89887 |         #endif
89888 |         return NULL;
89889 |     }
89890 |     ma_dr_flac__init_from_info(pFlac, &init);
89891 |     pFlac->allocationCallbacks = allocationCallbacks;
89892 |     pFlac->pDecodedSamples = (ma_int32*)ma_dr_flac_align((size_t)pFlac->pExtraData,
> MA_DR_FLAC_MAX_SIMD_VECTOR_SIZE);
89893 | #ifndef MA_DR_FLAC_NO_OGG
89894 |     if (init.container == ma_dr_flac_container_ogg) {
89895 |         ma_dr_flac_oggbs* pInternalOggbs = (ma_dr_flac_oggbs*)((ma_uint8*)pFlac->pDecodedSamples +
> decodedSamplesAllocationSize + (seekpointCount * sizeof(ma_dr_flac_seekpoint)));
89896 |         MA_DR_FLAC_COPY_MEMORY(pInternalOggbs, pOggbs, sizeof(*pOggbs));
89897 |         ma_dr_flac__free_from_callbacks(pOggbs, &allocationCallbacks);
89898 |         pOggbs = NULL;
89899 |         pFlac->bs.onRead = ma_dr_flac__on_read_ogg;
89900 |         pFlac->bs.onSeek = ma_dr_flac__on_seek_ogg;
89901 |         pFlac->bs.onTell = ma_dr_flac__on_tell_ogg;
89902 |         pFlac->bs.pUserData = (void*)pInternalOggbs;
89903 |         pFlac->_oggbs = (void*)pInternalOggbs;
89904 |     }
89905 | #endif
89906 |     pFlac->firstFLACFramePosInBytes = firstFramePos;
89907 | #ifndef MA_DR_FLAC_NO_OGG
89908 |     if (init.container == ma_dr_flac_container_ogg)
89909 |     {
89910 |         pFlac->pSeekpoints = NULL;
89911 |         pFlac->seekpointCount = 0;
89912 |     }
89913 |     else
89914 |     #endif
89915 |     {
89916 |         if (seektablePos != 0) {

```

```

89917 |         pFlac->seekpointCount = seekpointCount;
89918 |         pFlac->pSeekpoints = (ma_dr_flac_seekpoint*)((ma_uint8*)pFlac->pDecodedSamples +
> decodedSamplesAllocationSize);
89919 |         MA_DR_FLAC_ASSERT(pFlac->bs.onSeek != NULL);
89920 |         MA_DR_FLAC_ASSERT(pFlac->bs.onRead != NULL);
89921 |         if (pFlac->bs.onSeek(pFlac->bs.pUserData, (int)seektablePos, MA_DR_FLAC_SEEK_SET)) {
89922 |             ma_uint32 iSeekpoint;
89923 |             for (iSeekpoint = 0; iSeekpoint < seekpointCount; iSeekpoint += 1) {
89924 |                 if (pFlac->bs.onRead(pFlac->bs.pUserData, pFlac->pSeekpoints + iSeekpoint,
> MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES) == MA_DR_FLAC_SEEKPOINT_SIZE_IN_BYTES) {
89925 |                     pFlac->pSeekpoints[iSeekpoint].firstPCMFrame =
> ma_dr_flac__be2host_64(pFlac->pSeekpoints[iSeekpoint].firstPCMFrame);
89926 |                     pFlac->pSeekpoints[iSeekpoint].flacFrameOffset =
> ma_dr_flac__be2host_64(pFlac->pSeekpoints[iSeekpoint].flacFrameOffset);
89927 |                     pFlac->pSeekpoints[iSeekpoint].pcmFrameCount =
> ma_dr_flac__be2host_16(pFlac->pSeekpoints[iSeekpoint].pcmFrameCount);
89928 |                 } else {
89929 |                     pFlac->pSeekpoints = NULL;
89930 |                     pFlac->seekpointCount = 0;
89931 |                     break;
89932 |                 }
89933 |             }
89934 |             if (!pFlac->bs.onSeek(pFlac->bs.pUserData, (int)pFlac->firstFLACFramePosInBytes,
> MA_DR_FLAC_SEEK_SET)) {
89935 |                 ma_dr_flac__free_from_callbacks(pFlac, &allocationCallbacks);
89936 |                 return NULL;
89937 |             }
89938 |         } else {
89939 |             pFlac->pSeekpoints = NULL;
89940 |             pFlac->seekpointCount = 0;
89941 |         }
89942 |     }
89943 | }
89944 | if (!init.hasStreamInfoBlock) {
89945 |     pFlac->currentFLACFrame.header = init.firstFrameHeader;
89946 |     for (;;) {
89947 |         ma_result result = ma_dr_flac__decode_flac_frame(pFlac);
89948 |         if (result == MA_SUCCESS) {
89949 |             break;
89950 |         } else {
89951 |             if (result == MA_CRC_MISMATCH) {
89952 |                 if (!ma_dr_flac__read_next_flac_frame_header(&pFlac->bs, pFlac->bitsPerSample,
> &pFlac->currentFLACFrame.header)) {
89953 |                     ma_dr_flac__free_from_callbacks(pFlac, &allocationCallbacks);
89954 |                     return NULL;
89955 |                 }
89956 |                 continue;
89957 |             } else {
89958 |                 ma_dr_flac__free_from_callbacks(pFlac, &allocationCallbacks);
89959 |                 return NULL;
89960 |             }
89961 |         }
89962 |     }
89963 | }
89964 | return pFlac;
89965 | }
89966 | #ifndef MA_DR_FLAC_NO_STDIO
89967 | #include <stdio.h>
89968 | #ifndef MA_DR_FLAC_NO_WCHAR
89969 | #include <wchar.h>
89970 | #endif
89971 | static size_t ma_dr_flac__on_read_stdio(void* pUserData, void* bufferOut, size_t bytesToRead)
89972 | {
89973 |     return fread(bufferOut, 1, bytesToRead, (FILE*)pUserData);
89974 | }
89975 | static ma_bool32 ma_dr_flac__on_seek_stdio(void* pUserData, int offset, ma_dr_flac_seek_origin
> origin)
89976 | {
89977 |     int whence = SEEK_SET;
89978 |     if (origin == MA_DR_FLAC_SEEK_CUR) {
89979 |         whence = SEEK_CUR;
89980 |     } else if (origin == MA_DR_FLAC_SEEK_END) {
89981 |         whence = SEEK_END;
89982 |     }
89983 |     return fseek((FILE*)pUserData, offset, whence) == 0;
89984 | }
89985 | static ma_bool32 ma_dr_flac__on_tell_stdio(void* pUserData, ma_int64* pCursor)
89986 | {
89987 |     FILE* pFileStdio = (FILE*)pUserData;

```



```

89988 |     ma_int64 result;
89989 |     MA_DR_FLAC_ASSERT(pFileStdio != NULL);
89990 |     MA_DR_FLAC_ASSERT(pCursor != NULL);
89991 |     #if defined(_WIN32) && !defined(NXDK)
89992 |         #if defined(_MSC_VER) && _MSC_VER > 1200
89993 |             result = _ftelli64(pFileStdio);
89994 |         #else
89995 |             result = ftell(pFileStdio);
89996 |         #endif
89997 |     #else
89998 |         result = ftell(pFileStdio);
89999 |     #endif
90000 |     *pCursor = result;
90001 |     return MA_TRUE;
90002 | }
90003 | MA_API ma_dr_flac* ma_dr_flac_open_file(const char* pFileName, const ma_allocation_callbacks*
> pAllocationCallbacks)
90004 | {
90005 |     ma_dr_flac* pFlac;
90006 |     FILE* pFile;
90007 |     if (ma_fopen(&pFile, pFileName, "rb") != MA_SUCCESS) {
90008 |         return NULL;
90009 |     }
90010 |     pFlac = ma_dr_flac_open(ma_dr_flac__on_read_stdio, ma_dr_flac__on_seek_stdio,
> ma_dr_flac__on_tell_stdio, (void*)pFile, pAllocationCallbacks);
90011 |     if (pFlac == NULL) {
90012 |         fclose(pFile);
90013 |         return NULL;
90014 |     }
90015 |     return pFlac;
90016 | }
90017 | #ifndef MA_DR_FLAC_NO_WCHAR
90018 | MA_API ma_dr_flac* ma_dr_flac_open_file_w(const wchar_t* pFileName, const ma_allocation_callbacks*
> pAllocationCallbacks)
90019 | {
90020 |     ma_dr_flac* pFlac;
90021 |     FILE* pFile;
90022 |     if (ma_wfopen(&pFile, pFileName, L"rb", pAllocationCallbacks) != MA_SUCCESS) {
90023 |         return NULL;
90024 |     }
90025 |     pFlac = ma_dr_flac_open(ma_dr_flac__on_read_stdio, ma_dr_flac__on_seek_stdio,
> ma_dr_flac__on_tell_stdio, (void*)pFile, pAllocationCallbacks);
90026 |     if (pFlac == NULL) {
90027 |         fclose(pFile);
90028 |         return NULL;
90029 |     }
90030 |     return pFlac;
90031 | }
90032 | #endif
90033 | MA_API ma_dr_flac* ma_dr_flac_open_file_with_metadata(const char* pFileName, ma_dr_flac_meta_proc
> onMeta, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks)
90034 | {
90035 |     ma_dr_flac* pFlac;
90036 |     FILE* pFile;
90037 |     if (ma_fopen(&pFile, pFileName, "rb") != MA_SUCCESS) {
90038 |         return NULL;
90039 |     }
90040 |     pFlac = ma_dr_flac_open_with_metadata_private(ma_dr_flac__on_read_stdio,
> ma_dr_flac__on_seek_stdio, ma_dr_flac__on_tell_stdio, onMeta, ma_dr_flac_container_unknown,
> (void*)pFile, pUserData, pAllocationCallbacks);
90041 |     if (pFlac == NULL) {
90042 |         fclose(pFile);
90043 |         return pFlac;
90044 |     }
90045 |     return pFlac;
90046 | }
90047 | #ifndef MA_DR_FLAC_NO_WCHAR
90048 | MA_API ma_dr_flac* ma_dr_flac_open_file_with_metadata_w(const wchar_t* pFileName,
> ma_dr_flac_meta_proc onMeta, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks)
90049 | {
90050 |     ma_dr_flac* pFlac;
90051 |     FILE* pFile;
90052 |     if (ma_wfopen(&pFile, pFileName, L"rb", pAllocationCallbacks) != MA_SUCCESS) {
90053 |         return NULL;
90054 |     }
90055 |     pFlac = ma_dr_flac_open_with_metadata_private(ma_dr_flac__on_read_stdio,
> ma_dr_flac__on_seek_stdio, ma_dr_flac__on_tell_stdio, onMeta, ma_dr_flac_container_unknown,
> (void*)pFile, pUserData, pAllocationCallbacks);
90056 |     if (pFlac == NULL) {

```

```

90057 |         fclose(pFile);
90058 |         return pFlac;
90059 |     }
90060 |     return pFlac;
90061 | }
90062 | #endif
90063 | #endif
90064 | static size_t ma_dr_flac__on_read_memory(void* pUserData, void* bufferOut, size_t bytesToRead)
90065 | {
90066 |     ma_dr_flac__memory_stream* memoryStream = (ma_dr_flac__memory_stream*)pUserData;
90067 |     size_t bytesRemaining;
90068 |     MA_DR_FLAC_ASSERT(memoryStream != NULL);
90069 |     MA_DR_FLAC_ASSERT(memoryStream->dataSize >= memoryStream->currentReadPos);
90070 |     bytesRemaining = memoryStream->dataSize - memoryStream->currentReadPos;
90071 |     if (bytesToRead > bytesRemaining) {
90072 |         bytesToRead = bytesRemaining;
90073 |     }
90074 |     if (bytesToRead > 0) {
90075 |         MA_DR_FLAC_COPY_MEMORY(bufferOut, memoryStream->data + memoryStream->currentReadPos,
> bytesToRead);
90076 |         memoryStream->currentReadPos += bytesToRead;
90077 |     }
90078 |     return bytesToRead;
90079 | }
90080 | static ma_bool32 ma_dr_flac__on_seek_memory(void* pUserData, int offset, ma_dr_flac_seek_origin
> origin)
90081 | {
90082 |     ma_dr_flac__memory_stream* memoryStream = (ma_dr_flac__memory_stream*)pUserData;
90083 |     ma_int64 newCursor;
90084 |     MA_DR_FLAC_ASSERT(memoryStream != NULL);
90085 |     if (origin == MA_DR_FLAC_SEEK_SET) {
90086 |         newCursor = 0;
90087 |     } else if (origin == MA_DR_FLAC_SEEK_CUR) {
90088 |         newCursor = (ma_int64)memoryStream->currentReadPos;
90089 |     } else if (origin == MA_DR_FLAC_SEEK_END) {
90090 |         newCursor = (ma_int64)memoryStream->dataSize;
90091 |     } else {
90092 |         MA_DR_FLAC_ASSERT(!"Invalid seek origin");
90093 |         return MA_FALSE;
90094 |     }
90095 |     newCursor += offset;
90096 |     if (newCursor < 0) {
90097 |         return MA_FALSE;
90098 |     }
90099 |     if ((size_t)newCursor > memoryStream->dataSize) {
90100 |         return MA_FALSE;
90101 |     }
90102 |     memoryStream->currentReadPos = (size_t)newCursor;
90103 |     return MA_TRUE;
90104 | }
90105 | static ma_bool32 ma_dr_flac__on_tell_memory(void* pUserData, ma_int64* pCursor)
90106 | {
90107 |     ma_dr_flac__memory_stream* memoryStream = (ma_dr_flac__memory_stream*)pUserData;
90108 |     MA_DR_FLAC_ASSERT(memoryStream != NULL);
90109 |     MA_DR_FLAC_ASSERT(pCursor != NULL);
90110 |     *pCursor = (ma_int64)memoryStream->currentReadPos;
90111 |     return MA_TRUE;
90112 | }
90113 | MA_API ma_dr_flac* ma_dr_flac_open_memory(const void* pData, size_t dataSize, const
> ma_allocation_callbacks* pAllocationCallbacks)
90114 | {
90115 |     ma_dr_flac__memory_stream memoryStream;
90116 |     ma_dr_flac* pFlac;
90117 |     memoryStream.data = (const ma_uint8*)pData;
90118 |     memoryStream.dataSize = dataSize;
90119 |     memoryStream.currentReadPos = 0;
90120 |     pFlac = ma_dr_flac_open(ma_dr_flac__on_read_memory, ma_dr_flac__on_seek_memory,
> ma_dr_flac__on_tell_memory, &memoryStream, pAllocationCallbacks);
90121 |     if (pFlac == NULL) {
90122 |         return NULL;
90123 |     }
90124 |     pFlac->memoryStream = memoryStream;
90125 | #ifndef MA_DR_FLAC_NO_OGG
90126 |     if (pFlac->container == ma_dr_flac_container_ogg)
90127 |     {
90128 |         ma_dr_flac_oggbs* oggbs = (ma_dr_flac_oggbs*)pFlac->oggbs;
90129 |         oggbs->pUserData = &pFlac->memoryStream;
90130 |     }
90131 |     else

```

```

90132 | #endif
90133 | {
90134 |     pFlac->bs.pUserData = &pFlac->memoryStream;
90135 | }
90136 | return pFlac;
90137 | }
90138 | MA_API ma_dr_flac* ma_dr_flac_open_memory_with_metadata(const void* pData, size_t dataSize,
> ma_dr_flac_meta_proc onMeta, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks)
90139 | {
90140 |     ma_dr_flac__memory_stream memoryStream;
90141 |     ma_dr_flac* pFlac;
90142 |     memoryStream.data = (const ma_uint8*)pData;
90143 |     memoryStream.dataSize = dataSize;
90144 |     memoryStream.currentReadPos = 0;
90145 |     pFlac = ma_dr_flac_open_with_metadata_private(ma_dr_flac__on_read_memory,
> ma_dr_flac__on_seek_memory, ma_dr_flac__on_tell_memory, onMeta, ma_dr_flac_container_unknown,
> &memoryStream, pUserData, pAllocationCallbacks);
90146 |     if (pFlac == NULL) {
90147 |         return NULL;
90148 |     }
90149 |     pFlac->memoryStream = memoryStream;
90150 | #ifndef MA_DR_FLAC_NO_OGG
90151 |     if (pFlac->container == ma_dr_flac_container_ogg)
90152 |     {
90153 |         ma_dr_flac_oggbs* oggbs = (ma_dr_flac_oggbs*)pFlac->_oggbs;
90154 |         oggbs->pUserData = &pFlac->memoryStream;
90155 |     }
90156 |     else
90157 | #endif
90158 |     {
90159 |         pFlac->bs.pUserData = &pFlac->memoryStream;
90160 |     }
90161 |     return pFlac;
90162 | }
90163 | MA_API ma_dr_flac* ma_dr_flac_open(ma_dr_flac_read_proc onRead, ma_dr_flac_seek_proc onSeek,
> ma_dr_flac_tell_proc onTell, void* pUserData, const ma_allocation_callbacks* pAllocationCallbacks)
90164 | {
90165 |     return ma_dr_flac_open_with_metadata_private(onRead, onSeek, onTell, NULL,
> ma_dr_flac_container_unknown, pUserData, pUserData, pAllocationCallbacks);
90166 | }
90167 | MA_API ma_dr_flac* ma_dr_flac_open_relaxed(ma_dr_flac_read_proc onRead, ma_dr_flac_seek_proc onSeek,
> ma_dr_flac_tell_proc onTell, ma_dr_flac_container container, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks)
90168 | {
90169 |     return ma_dr_flac_open_with_metadata_private(onRead, onSeek, onTell, NULL, container, pUserData,
> pUserData, pAllocationCallbacks);
90170 | }
90171 | MA_API ma_dr_flac* ma_dr_flac_open_with_metadata(ma_dr_flac_read_proc onRead, ma_dr_flac_seek_proc
> onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks)
90172 | {
90173 |     return ma_dr_flac_open_with_metadata_private(onRead, onSeek, onTell, onMeta,
> ma_dr_flac_container_unknown, pUserData, pUserData, pAllocationCallbacks);
90174 | }
90175 | MA_API ma_dr_flac* ma_dr_flac_open_with_metadata_relaxed(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, ma_dr_flac_meta_proc onMeta,
> ma_dr_flac_container container, void* pUserData, const ma_allocation_callbacks*
> pAllocationCallbacks)
90176 | {
90177 |     return ma_dr_flac_open_with_metadata_private(onRead, onSeek, onTell, onMeta, container,
> pUserData, pUserData, pAllocationCallbacks);
90178 | }
90179 | MA_API void ma_dr_flac_close(ma_dr_flac* pFlac)
90180 | {
90181 |     if (pFlac == NULL) {
90182 |         return;
90183 |     }
90184 | #ifndef MA_DR_FLAC_NO_STDIO
90185 |     if (pFlac->bs.onRead == ma_dr_flac__on_read_stdio) {
90186 |         fclose((FILE*)pFlac->bs.pUserData);
90187 |     }
90188 | #ifndef MA_DR_FLAC_NO_OGG
90189 |     if (pFlac->container == ma_dr_flac_container_ogg) {
90190 |         ma_dr_flac_oggbs* oggbs = (ma_dr_flac_oggbs*)pFlac->_oggbs;
90191 |         MA_DR_FLAC_ASSERT(pFlac->bs.onRead == ma_dr_flac__on_read_ogg);
90192 |         if (oggbs->onRead == ma_dr_flac__on_read_stdio) {
90193 |             fclose((FILE*)oggbs->pUserData);
90194 |         }
90195 |     }

```

```

90196 | #endif
90197 | #endif
90198 |     ma_dr_flac__free_from_callbacks(pFlac, &pFlac->allocationCallbacks);
90199 | }
90200 | #if 0
90201 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_left_side__reference(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int32* pOutputSamples)
90202 | {
90203 |     ma_uint64 i;
90204 |     for (i = 0; i < frameCount; ++i) {
90205 |         ma_uint32 left = (ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
90206 |         ma_uint32 side = (ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
90207 |         ma_uint32 right = left - side;
90208 |         pOutputSamples[i*2+0] = (ma_int32)left;
90209 |         pOutputSamples[i*2+1] = (ma_int32)right;
90210 |     }
90211 | }
90212 | #endif
90213 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_left_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90214 | {
90215 |     ma_uint64 i;
90216 |     ma_uint64 frameCount4 = frameCount >> 2;
90217 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90218 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90219 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90220 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90221 |     for (i = 0; i < frameCount4; ++i) {
90222 |         ma_uint32 left0 = pInputSamples0U32[i*4+0] << shift0;
90223 |         ma_uint32 left1 = pInputSamples0U32[i*4+1] << shift0;
90224 |         ma_uint32 left2 = pInputSamples0U32[i*4+2] << shift0;
90225 |         ma_uint32 left3 = pInputSamples0U32[i*4+3] << shift0;
90226 |         ma_uint32 side0 = pInputSamples1U32[i*4+0] << shift1;
90227 |         ma_uint32 side1 = pInputSamples1U32[i*4+1] << shift1;
90228 |         ma_uint32 side2 = pInputSamples1U32[i*4+2] << shift1;
90229 |         ma_uint32 side3 = pInputSamples1U32[i*4+3] << shift1;
90230 |         ma_uint32 right0 = left0 - side0;
90231 |         ma_uint32 right1 = left1 - side1;
90232 |         ma_uint32 right2 = left2 - side2;
90233 |         ma_uint32 right3 = left3 - side3;
90234 |         pOutputSamples[i*8+0] = (ma_int32)left0;
90235 |         pOutputSamples[i*8+1] = (ma_int32)right0;
90236 |         pOutputSamples[i*8+2] = (ma_int32)left1;
90237 |         pOutputSamples[i*8+3] = (ma_int32)right1;
90238 |         pOutputSamples[i*8+4] = (ma_int32)left2;
90239 |         pOutputSamples[i*8+5] = (ma_int32)right2;
90240 |         pOutputSamples[i*8+6] = (ma_int32)left3;
90241 |         pOutputSamples[i*8+7] = (ma_int32)right3;
90242 |     }
90243 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90244 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
90245 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
90246 |         ma_uint32 right = left - side;
90247 |         pOutputSamples[i*2+0] = (ma_int32)left;
90248 |         pOutputSamples[i*2+1] = (ma_int32)right;
90249 |     }
90250 | }
90251 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90252 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_left_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90253 | {
90254 |     ma_uint64 i;
90255 |     ma_uint64 frameCount4 = frameCount >> 2;
90256 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90257 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90258 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90259 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90260 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90261 |     for (i = 0; i < frameCount4; ++i) {
90262 |         __m128i left = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);

```

```

90263 |         __m128i side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
90264 |         __m128i right = _mm_sub_epi32(left, side);
90265 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 0), _mm_unpacklo_epi32(left, right));
90266 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 4), _mm_unpackhi_epi32(left, right));
90267 |     }
90268 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90269 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
90270 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
90271 |         ma_uint32 right = left - side;
90272 |         pOutputSamples[i*2+0] = (ma_int32)left;
90273 |         pOutputSamples[i*2+1] = (ma_int32)right;
90274 |     }
90275 | }
90276 | #endif
90277 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
90278 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_left_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90279 | {
90280 |     ma_uint64 i;
90281 |     ma_uint64 frameCount4 = frameCount >> 2;
90282 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90283 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90284 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90285 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90286 |     int32x4_t shift0_4;
90287 |     int32x4_t shift1_4;
90288 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90289 |     shift0_4 = vdupq_n_s32(shift0);
90290 |     shift1_4 = vdupq_n_s32(shift1);
90291 |     for (i = 0; i < frameCount4; ++i) {
90292 |         uint32x4_t left;
90293 |         uint32x4_t side;
90294 |         uint32x4_t right;
90295 |         left = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4);
90296 |         side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4);
90297 |         right = vsubq_u32(left, side);
90298 |         ma_dr_flac__vst2q_u32((ma_uint32*)pOutputSamples + i*8, vzipq_u32(left, right));
90299 |     }
90300 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90301 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
90302 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
90303 |         ma_uint32 right = left - side;
90304 |         pOutputSamples[i*2+0] = (ma_int32)left;
90305 |         pOutputSamples[i*2+1] = (ma_int32)right;
90306 |     }
90307 | }
90308 | #endif
90309 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_left_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90310 | {
90311 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90312 |         if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
90313 |             ma_dr_flac_read_pcm_frames_s32__decode_left_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90314 |         } else
90315 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
90316 |             if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
90317 |                 ma_dr_flac_read_pcm_frames_s32__decode_left_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90318 |             } else
90319 |             #endif
90320 |             {
90321 |                 #if 0
90322 |                     ma_dr_flac_read_pcm_frames_s32__decode_left_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90323 |                 #else
90324 |                     ma_dr_flac_read_pcm_frames_s32__decode_left_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90325 |                 #endif
90326 |             }
90327 |         }
90328 |         #if 0
90329 |         static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_right_side__reference(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int32* pOutputSamples)

```

```

90330 | {
90331 |     ma_uint64 i;
90332 |     for (i = 0; i < frameCount; ++i) {
90333 |         ma_uint32 side = (ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
90334 |         ma_uint32 right = (ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
90335 |         ma_uint32 left = right + side;
90336 |         pOutputSamples[i*2+0] = (ma_int32)left;
90337 |         pOutputSamples[i*2+1] = (ma_int32)right;
90338 |     }
90339 | }
90340 | #endif
90341 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_right_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90342 | {
90343 |     ma_uint64 i;
90344 |     ma_uint64 frameCount4 = frameCount >> 2;
90345 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90346 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90347 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90348 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90349 |     for (i = 0; i < frameCount4; ++i) {
90350 |         ma_uint32 side0 = pInputSamples0U32[i*4+0] << shift0;
90351 |         ma_uint32 side1 = pInputSamples0U32[i*4+1] << shift0;
90352 |         ma_uint32 side2 = pInputSamples0U32[i*4+2] << shift0;
90353 |         ma_uint32 side3 = pInputSamples0U32[i*4+3] << shift0;
90354 |         ma_uint32 right0 = pInputSamples1U32[i*4+0] << shift1;
90355 |         ma_uint32 right1 = pInputSamples1U32[i*4+1] << shift1;
90356 |         ma_uint32 right2 = pInputSamples1U32[i*4+2] << shift1;
90357 |         ma_uint32 right3 = pInputSamples1U32[i*4+3] << shift1;
90358 |         ma_uint32 left0 = right0 + side0;
90359 |         ma_uint32 left1 = right1 + side1;
90360 |         ma_uint32 left2 = right2 + side2;
90361 |         ma_uint32 left3 = right3 + side3;
90362 |         pOutputSamples[i*8+0] = (ma_int32)left0;
90363 |         pOutputSamples[i*8+1] = (ma_int32)right0;
90364 |         pOutputSamples[i*8+2] = (ma_int32)left1;
90365 |         pOutputSamples[i*8+3] = (ma_int32)right1;
90366 |         pOutputSamples[i*8+4] = (ma_int32)left2;
90367 |         pOutputSamples[i*8+5] = (ma_int32)right2;
90368 |         pOutputSamples[i*8+6] = (ma_int32)left3;
90369 |         pOutputSamples[i*8+7] = (ma_int32)right3;
90370 |     }
90371 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90372 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
90373 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
90374 |         ma_uint32 left = right + side;
90375 |         pOutputSamples[i*2+0] = (ma_int32)left;
90376 |         pOutputSamples[i*2+1] = (ma_int32)right;
90377 |     }
90378 | }
90379 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90380 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_right_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90381 | {
90382 |     ma_uint64 i;
90383 |     ma_uint64 frameCount4 = frameCount >> 2;
90384 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90385 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90386 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90387 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90388 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90389 |     for (i = 0; i < frameCount4; ++i) {
90390 |         __m128i side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
90391 |         __m128i right = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
90392 |         __m128i left = _mm_add_epi32(right, side);
90393 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 0), _mm_unpacklo_epi32(left, right));
90394 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 4), _mm_unpackhi_epi32(left, right));
90395 |     }
90396 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90397 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
90398 |         ma_uint32 right = pInputSamples1U32[i] << shift1;

```



```

90399 |         ma_uint32 left = right + side;
90400 |         pOutputSamples[i*2+0] = (ma_int32)left;
90401 |         pOutputSamples[i*2+1] = (ma_int32)right;
90402 |     }
90403 | }
90404 | #endif
90405 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
90406 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_right_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90407 | {
90408 |     ma_uint64 i;
90409 |     ma_uint64 frameCount4 = frameCount >> 2;
90410 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90411 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90412 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90413 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90414 |     int32x4_t shift0_4;
90415 |     int32x4_t shift1_4;
90416 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90417 |     shift0_4 = vdupq_n_s32(shift0);
90418 |     shift1_4 = vdupq_n_s32(shift1);
90419 |     for (i = 0; i < frameCount4; ++i) {
90420 |         uint32x4_t side;
90421 |         uint32x4_t right;
90422 |         uint32x4_t left;
90423 |         side = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4);
90424 |         right = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4);
90425 |         left = vaddq_u32(right, side);
90426 |         ma_dr_flac__vst2q_u32((ma_uint32*)pOutputSamples + i*8, vzipq_u32(left, right));
90427 |     }
90428 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90429 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
90430 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
90431 |         ma_uint32 left = right + side;
90432 |         pOutputSamples[i*2+0] = (ma_int32)left;
90433 |         pOutputSamples[i*2+1] = (ma_int32)right;
90434 |     }
90435 | }
90436 | #endif
90437 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_right_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90438 | {
90439 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90440 |         if (ma_dr_flac__gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
90441 |             ma_dr_flac_read_pcm_frames_s32__decode_right_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90442 |         } else
90443 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
90444 |             if (ma_dr_flac__gIsNEONSupported && pFlac->bitsPerSample <= 24) {
90445 |                 ma_dr_flac_read_pcm_frames_s32__decode_right_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90446 |             } else
90447 |             #endif
90448 |             {
90449 |                 #if 0
90450 |                     ma_dr_flac_read_pcm_frames_s32__decode_right_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90451 |                 #else
90452 |                     ma_dr_flac_read_pcm_frames_s32__decode_right_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90453 |                 #endif
90454 |             }
90455 |         }
90456 |         #if 0
90457 |         static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_mid_side__reference(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90458 |         {
90459 |             for (ma_uint64 i = 0; i < frameCount; ++i) {
90460 |                 ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90461 |                 ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90462 |                 mid = (mid << 1) | (side & 0x01);
90463 |                 pOutputSamples[i*2+0] = (ma_int32)((ma_uint32)((ma_int32)(mid + side) >> 1) <<

```



```

> unusedBitsPerSample);
90464 |     pOutputSamples[i*2+1] = (ma_int32)((ma_uint32)((ma_int32)(mid - side) >> 1) <<
> unusedBitsPerSample);
90465 |     }
90466 | }
90467 | #endif
90468 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_mid_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90469 | {
90470 |     ma_uint64 i;
90471 |     ma_uint64 frameCount4 = frameCount >> 2;
90472 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90473 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90474 |     ma_int32 shift = unusedBitsPerSample;
90475 |     if (shift > 0) {
90476 |         shift -= 1;
90477 |         for (i = 0; i < frameCount4; ++i) {
90478 |             ma_uint32 temp0L;
90479 |             ma_uint32 temp1L;
90480 |             ma_uint32 temp2L;
90481 |             ma_uint32 temp3L;
90482 |             ma_uint32 temp0R;
90483 |             ma_uint32 temp1R;
90484 |             ma_uint32 temp2R;
90485 |             ma_uint32 temp3R;
90486 |             ma_uint32 mid0 = pInputSamples0U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90487 |             ma_uint32 mid1 = pInputSamples0U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90488 |             ma_uint32 mid2 = pInputSamples0U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90489 |             ma_uint32 mid3 = pInputSamples0U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90490 |             ma_uint32 side0 = pInputSamples1U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90491 |             ma_uint32 side1 = pInputSamples1U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90492 |             ma_uint32 side2 = pInputSamples1U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90493 |             ma_uint32 side3 = pInputSamples1U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90494 |             mid0 = (mid0 << 1) | (side0 & 0x01);
90495 |             mid1 = (mid1 << 1) | (side1 & 0x01);
90496 |             mid2 = (mid2 << 1) | (side2 & 0x01);
90497 |             mid3 = (mid3 << 1) | (side3 & 0x01);
90498 |             temp0L = (mid0 + side0) << shift;
90499 |             temp1L = (mid1 + side1) << shift;
90500 |             temp2L = (mid2 + side2) << shift;
90501 |             temp3L = (mid3 + side3) << shift;
90502 |             temp0R = (mid0 - side0) << shift;
90503 |             temp1R = (mid1 - side1) << shift;
90504 |             temp2R = (mid2 - side2) << shift;
90505 |             temp3R = (mid3 - side3) << shift;
90506 |             pOutputSamples[i*8+0] = (ma_int32)temp0L;
90507 |             pOutputSamples[i*8+1] = (ma_int32)temp0R;
90508 |             pOutputSamples[i*8+2] = (ma_int32)temp1L;
90509 |             pOutputSamples[i*8+3] = (ma_int32)temp1R;
90510 |             pOutputSamples[i*8+4] = (ma_int32)temp2L;
90511 |             pOutputSamples[i*8+5] = (ma_int32)temp2R;
90512 |             pOutputSamples[i*8+6] = (ma_int32)temp3L;
90513 |             pOutputSamples[i*8+7] = (ma_int32)temp3R;
90514 |         }
90515 |     } else {
90516 |         for (i = 0; i < frameCount4; ++i) {
90517 |             ma_uint32 temp0L;
90518 |             ma_uint32 temp1L;
90519 |             ma_uint32 temp2L;
90520 |             ma_uint32 temp3L;
90521 |             ma_uint32 temp0R;
90522 |             ma_uint32 temp1R;
90523 |             ma_uint32 temp2R;
90524 |             ma_uint32 temp3R;
90525 |             ma_uint32 mid0 = pInputSamples0U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90526 |             ma_uint32 mid1 = pInputSamples0U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90527 |             ma_uint32 mid2 = pInputSamples0U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;

```

```

90528 |         ma_uint32 mid3 = pInputSamples0U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90529 |         ma_uint32 side0 = pInputSamples1U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90530 |         ma_uint32 side1 = pInputSamples1U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90531 |         ma_uint32 side2 = pInputSamples1U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90532 |         ma_uint32 side3 = pInputSamples1U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90533 |         mid0 = (mid0 << 1) | (side0 & 0x01);
90534 |         mid1 = (mid1 << 1) | (side1 & 0x01);
90535 |         mid2 = (mid2 << 1) | (side2 & 0x01);
90536 |         mid3 = (mid3 << 1) | (side3 & 0x01);
90537 |         temp0L = (ma_uint32)((ma_int32)(mid0 + side0) >> 1);
90538 |         temp1L = (ma_uint32)((ma_int32)(mid1 + side1) >> 1);
90539 |         temp2L = (ma_uint32)((ma_int32)(mid2 + side2) >> 1);
90540 |         temp3L = (ma_uint32)((ma_int32)(mid3 + side3) >> 1);
90541 |         temp0R = (ma_uint32)((ma_int32)(mid0 - side0) >> 1);
90542 |         temp1R = (ma_uint32)((ma_int32)(mid1 - side1) >> 1);
90543 |         temp2R = (ma_uint32)((ma_int32)(mid2 - side2) >> 1);
90544 |         temp3R = (ma_uint32)((ma_int32)(mid3 - side3) >> 1);
90545 |         pOutputSamples[i*8+0] = (ma_int32)temp0L;
90546 |         pOutputSamples[i*8+1] = (ma_int32)temp0R;
90547 |         pOutputSamples[i*8+2] = (ma_int32)temp1L;
90548 |         pOutputSamples[i*8+3] = (ma_int32)temp1R;
90549 |         pOutputSamples[i*8+4] = (ma_int32)temp2L;
90550 |         pOutputSamples[i*8+5] = (ma_int32)temp2R;
90551 |         pOutputSamples[i*8+6] = (ma_int32)temp3L;
90552 |         pOutputSamples[i*8+7] = (ma_int32)temp3R;
90553 |     }
90554 | }
90555 | for (i = (frameCount4 << 2); i < frameCount; ++i) {
90556 |     ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90557 |     ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90558 |     mid = (mid << 1) | (side & 0x01);
90559 |     pOutputSamples[i*2+0] = (ma_int32)((ma_uint32)((ma_int32)(mid + side) >> 1) <<
> unusedBitsPerSample);
90560 |     pOutputSamples[i*2+1] = (ma_int32)((ma_uint32)((ma_int32)(mid - side) >> 1) <<
> unusedBitsPerSample);
90561 | }
90562 | }
90563 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90564 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_mid_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90565 | {
90566 |     ma_uint64 i;
90567 |     ma_uint64 frameCount4 = frameCount >> 2;
90568 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90569 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90570 |     ma_int32 shift = unusedBitsPerSample;
90571 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90572 |     if (shift == 0) {
90573 |         for (i = 0; i < frameCount4; ++i) {
90574 |             __m128i mid;
90575 |             __m128i side;
90576 |             __m128i left;
90577 |             __m128i right;
90578 |             mid = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i),
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
90579 |             side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i),
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
90580 |             mid = _mm_or_si128(_mm_slli_epi32(mid, 1), _mm_and_si128(side, _mm_set1_epi32(0x01)));
90581 |             left = _mm_srai_epi32(_mm_add_epi32(mid, side), 1);
90582 |             right = _mm_srai_epi32(_mm_sub_epi32(mid, side), 1);
90583 |             _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 0), _mm_unpacklo_epi32(left, right));
90584 |             _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 4), _mm_unpackhi_epi32(left, right));
90585 |         }
90586 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
90587 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90588 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90589 |             mid = (mid << 1) | (side & 0x01);
90590 |             pOutputSamples[i*2+0] = (ma_int32)(mid + side) >> 1;
90591 |             pOutputSamples[i*2+1] = (ma_int32)(mid - side) >> 1;

```

```

90592 |     }
90593 |   } else {
90594 |     shift -= 1;
90595 |     for (i = 0; i < frameCount4; ++i) {
90596 |       __m128i mid;
90597 |       __m128i side;
90598 |       __m128i left;
90599 |       __m128i right;
90600 |       mid = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i),
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
90601 |       side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i),
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
90602 |       mid = _mm_or_si128(_mm_slli_epi32(mid, 1), _mm_and_si128(side, _mm_set1_epi32(0x01)));
90603 |       left = _mm_slli_epi32(_mm_add_epi32(mid, side), shift);
90604 |       right = _mm_slli_epi32(_mm_sub_epi32(mid, side), shift);
90605 |       _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 0), _mm_unpacklo_epi32(left, right));
90606 |       _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 4), _mm_unpackhi_epi32(left, right));
90607 |     }
90608 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90609 |       ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90610 |       ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90611 |       mid = (mid << 1) | (side & 0x01);
90612 |       pOutputSamples[i*2+0] = (ma_int32)((mid + side) << shift);
90613 |       pOutputSamples[i*2+1] = (ma_int32)((mid - side) << shift);
90614 |     }
90615 |   }
90616 | }
90617 | #endif
90618 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
90619 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_mid_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90620 | {
90621 |   ma_uint64 i;
90622 |   ma_uint64 frameCount4 = frameCount >> 2;
90623 |   const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90624 |   const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90625 |   ma_int32 shift = unusedBitsPerSample;
90626 |   int32x4_t wbpsShift0_4;
90627 |   int32x4_t wbpsShift1_4;
90628 |   uint32x4_t one4;
90629 |   MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90630 |   wbpsShift0_4 = vdupq_n_s32(pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
90631 |   wbpsShift1_4 = vdupq_n_s32(pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
90632 |   one4 = vdupq_n_u32(1);
90633 |   if (shift == 0) {
90634 |     for (i = 0; i < frameCount4; ++i) {
90635 |       uint32x4_t mid;
90636 |       uint32x4_t side;
90637 |       int32x4_t left;
90638 |       int32x4_t right;
90639 |       mid = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), wbpsShift0_4);
90640 |       side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), wbpsShift1_4);
90641 |       mid = vorrq_u32(vshlq_n_u32(mid, 1), vandq_u32(side, one4));
90642 |       left = vshrq_n_s32(vreinterpretq_s32_u32(vaddq_u32(mid, side)), 1);
90643 |       right = vshrq_n_s32(vreinterpretq_s32_u32(vsubq_u32(mid, side)), 1);
90644 |       ma_dr_flac__vst2q_s32(pOutputSamples + i*8, vzipq_s32(left, right));
90645 |     }
90646 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90647 |       ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90648 |       ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90649 |       mid = (mid << 1) | (side & 0x01);
90650 |       pOutputSamples[i*2+0] = (ma_int32)(mid + side) >> 1;
90651 |       pOutputSamples[i*2+1] = (ma_int32)(mid - side) >> 1;
90652 |     }
90653 |   } else {
90654 |     int32x4_t shift4;
90655 |     shift -= 1;
90656 |     shift4 = vdupq_n_s32(shift);
90657 |     for (i = 0; i < frameCount4; ++i) {
90658 |       uint32x4_t mid;
90659 |       uint32x4_t side;
90660 |       int32x4_t left;
90661 |       int32x4_t right;
90662 |       mid = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), wbpsShift0_4);

```

```

90663 |         side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), wbpsShift1_4);
90664 |         mid = vorrq_u32(vshlq_n_u32(mid, 1), vandq_u32(side, one4));
90665 |         left = vreinterpretq_s32_u32(vshlq_u32(vaddq_u32(mid, side), shift4));
90666 |         right = vreinterpretq_s32_u32(vshlq_u32(vsubq_u32(mid, side), shift4));
90667 |         ma_dr_flac__vst2q_s32(pOutputSamples + i*8, vzipq_s32(left, right));
90668 |     }
90669 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90670 |         ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90671 |         ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90672 |         mid = (mid << 1) | (side & 0x01);
90673 |         pOutputSamples[i*2+0] = (ma_int32)((mid + side) << shift);
90674 |         pOutputSamples[i*2+1] = (ma_int32)((mid - side) << shift);
90675 |     }
90676 | }
90677 | }
90678 | #endif
90679 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_mid_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90680 | {
90681 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90682 |         if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
90683 |             ma_dr_flac_read_pcm_frames_s32__decode_mid_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90684 |         } else
90685 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
90686 |             if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
90687 |                 ma_dr_flac_read_pcm_frames_s32__decode_mid_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90688 |             } else
90689 |             #endif
90690 |             {
90691 |                 #if 0
90692 |                 ma_dr_flac_read_pcm_frames_s32__decode_mid_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90693 |                 #else
90694 |                 ma_dr_flac_read_pcm_frames_s32__decode_mid_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90695 |                 #endif
90696 |             }
90697 |         }
90698 |         #if 0
90699 |         static MA_INLINE void
> ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__reference(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90700 |         {
90701 |             for (ma_uint64 i = 0; i < frameCount; ++i) {
90702 |                 pOutputSamples[i*2+0] = (ma_int32)((ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample));
90703 |                 pOutputSamples[i*2+1] = (ma_int32)((ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample));
90704 |             }
90705 |         }
90706 |         #endif
90707 |         static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__scalar(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int32* pOutputSamples)
90708 |         {
90709 |             ma_uint64 i;
90710 |             ma_uint64 frameCount4 = frameCount >> 2;
90711 |             const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90712 |             const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90713 |             ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90714 |             ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90715 |             for (i = 0; i < frameCount4; ++i) {
90716 |                 ma_uint32 tempL0 = pInputSamples0U32[i*4+0] << shift0;
90717 |                 ma_uint32 tempL1 = pInputSamples0U32[i*4+1] << shift0;
90718 |                 ma_uint32 tempL2 = pInputSamples0U32[i*4+2] << shift0;
90719 |                 ma_uint32 tempL3 = pInputSamples0U32[i*4+3] << shift0;
90720 |                 ma_uint32 tempR0 = pInputSamples1U32[i*4+0] << shift1;
90721 |                 ma_uint32 tempR1 = pInputSamples1U32[i*4+1] << shift1;
90722 |                 ma_uint32 tempR2 = pInputSamples1U32[i*4+2] << shift1;
90723 |                 ma_uint32 tempR3 = pInputSamples1U32[i*4+3] << shift1;
90724 |                 pOutputSamples[i*8+0] = (ma_int32)tempL0;

```

```

90725 |         pOutputSamples[i*8+1] = (ma_int32)tempR0;
90726 |         pOutputSamples[i*8+2] = (ma_int32)tempL1;
90727 |         pOutputSamples[i*8+3] = (ma_int32)tempR1;
90728 |         pOutputSamples[i*8+4] = (ma_int32)tempL2;
90729 |         pOutputSamples[i*8+5] = (ma_int32)tempR2;
90730 |         pOutputSamples[i*8+6] = (ma_int32)tempL3;
90731 |         pOutputSamples[i*8+7] = (ma_int32)tempR3;
90732 |     }
90733 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90734 |         pOutputSamples[i*2+0] = (ma_int32)(pInputSamples0U32[i] << shift0);
90735 |         pOutputSamples[i*2+1] = (ma_int32)(pInputSamples1U32[i] << shift1);
90736 |     }
90737 | }
90738 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90739 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__sse2(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int32* pOutputSamples)
90740 | {
90741 |     ma_uint64 i;
90742 |     ma_uint64 frameCount4 = frameCount >> 2;
90743 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90744 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90745 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90746 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90747 |     for (i = 0; i < frameCount4; ++i) {
90748 |         __m128i left = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
90749 |         __m128i right = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
90750 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 0), _mm_unpacklo_epi32(left, right));
90751 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8 + 4), _mm_unpackhi_epi32(left, right));
90752 |     }
90753 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90754 |         pOutputSamples[i*2+0] = (ma_int32)(pInputSamples0U32[i] << shift0);
90755 |         pOutputSamples[i*2+1] = (ma_int32)(pInputSamples1U32[i] << shift1);
90756 |     }
90757 | }
90758 | #endif
90759 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
90760 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__neon(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int32* pOutputSamples)
90761 | {
90762 |     ma_uint64 i;
90763 |     ma_uint64 frameCount4 = frameCount >> 2;
90764 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90765 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90766 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90767 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90768 |     int32x4_t shift4_0 = vdupq_n_s32(shift0);
90769 |     int32x4_t shift4_1 = vdupq_n_s32(shift1);
90770 |     for (i = 0; i < frameCount4; ++i) {
90771 |         int32x4_t left;
90772 |         int32x4_t right;
90773 |         left = vreinterpretq_s32_u32(vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift4_0));
90774 |         right = vreinterpretq_s32_u32(vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift4_1));
90775 |         ma_dr_flac__vst2q_s32(pOutputSamples + i*8, vzipq_s32(left, right));
90776 |     }
90777 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90778 |         pOutputSamples[i*2+0] = (ma_int32)(pInputSamples0U32[i] << shift0);
90779 |         pOutputSamples[i*2+1] = (ma_int32)(pInputSamples1U32[i] << shift1);
90780 |     }
90781 | }
90782 | #endif
90783 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int32* pOutputSamples)
90784 | {
90785 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90786 |         if (ma_dr_flac__gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
90787 |             ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90788 |         } else
90789 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
90790 |             if (ma_dr_flac__gIsNEONSupported && pFlac->bitsPerSample <= 24) {
90791 |                 ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);

```

```

90792 |     } else
90793 | #endif
90794 | {
90795 | #if 0
90796 |     ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90797 | #else
90798 |     ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
90799 | #endif
90800 | }
90801 | }
90802 | MA_API ma_uint64 ma_dr_flac_read_pcm_frames_s32(ma_dr_flac* pFlac, ma_uint64 framesToRead, ma_int32*
> pBufferOut)
90803 | {
90804 |     ma_uint64 framesRead;
90805 |     ma_uint32 unusedBitsPerSample;
90806 |     if (pFlac == NULL || framesToRead == 0) {
90807 |         return 0;
90808 |     }
90809 |     if (pBufferOut == NULL) {
90810 |         return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, framesToRead);
90811 |     }
90812 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 32);
90813 |     unusedBitsPerSample = 32 - pFlac->bitsPerSample;
90814 |     framesRead = 0;
90815 |     while (framesToRead > 0) {
90816 |         if (pFlac->currentFLACFrame.pcmFramesRemaining == 0) {
90817 |             if (!ma_dr_flac__read_and_decode_next_flac_frame(pFlac)) {
90818 |                 break;
90819 |             }
90820 |         } else {
90821 |             unsigned int channelCount = ma_dr_flac__get_channel_count_from_channel_assignment(pFlac-
> >currentFLACFrame.header.channelAssignment);
90822 |             ma_uint64 iFirstPCMFrame = pFlac->currentFLACFrame.header.blockSizeInPCMFrames -
> pFlac->currentFLACFrame.pcmFramesRemaining;
90823 |             ma_uint64 frameCountThisIteration = framesToRead;
90824 |             if (frameCountThisIteration > pFlac->currentFLACFrame.pcmFramesRemaining) {
90825 |                 frameCountThisIteration = pFlac->currentFLACFrame.pcmFramesRemaining;
90826 |             }
90827 |             if (channelCount == 2) {
90828 |                 const ma_int32* pDecodedSamples0 = pFlac->currentFLACFrame.subframes[0].pSamplesS32
> + iFirstPCMFrame;
90829 |                 const ma_int32* pDecodedSamples1 = pFlac->currentFLACFrame.subframes[1].pSamplesS32
> + iFirstPCMFrame;
90830 |                 switch (pFlac->currentFLACFrame.header.channelAssignment)
90831 |                 {
90832 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_LEFT_SIDE:
90833 |                     {
90834 |                         ma_dr_flac_read_pcm_frames_s32__decode_left_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
90835 |                     } break;
90836 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_RIGHT_SIDE:
90837 |                     {
90838 |                         ma_dr_flac_read_pcm_frames_s32__decode_right_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
90839 |                     } break;
90840 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_MID_SIDE:
90841 |                     {
90842 |                         ma_dr_flac_read_pcm_frames_s32__decode_mid_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
90843 |                     } break;
90844 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_INDEPENDENT:
90845 |                     default:
90846 |                     {
90847 |                         ma_dr_flac_read_pcm_frames_s32__decode_independent_stereo(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
90848 |                     } break;
90849 |                 }
90850 |             } else {
90851 |                 ma_uint64 i;
90852 |                 for (i = 0; i < frameCountThisIteration; ++i) {
90853 |                     unsigned int j;
90854 |                     for (j = 0; j < channelCount; ++j) {
90855 |                         pBufferOut[(i*channelCount)+j] =
> (ma_int32)((ma_uint32)(pFlac->currentFLACFrame.subframes[j].pSamplesS32[iFirstPCMFrame + i]) <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[j].wastedBitsPerSample));
90856 |                     }
90857 |                 }

```



```

90858 |     }
90859 |     framesRead          += frameCountThisIteration;
90860 |     pBufferOut          += frameCountThisIteration * channelCount;
90861 |     framesToRead        -= frameCountThisIteration;
90862 |     pFlac->currentPCMFrame += frameCountThisIteration;
90863 |     pFlac->currentFLACFrame.pcmFramesRemaining -= (ma_uint32)frameCountThisIteration;
90864 | }
90865 | }
90866 | return framesRead;
90867 | }
90868 | #if 0
90869 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_left_side__reference(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int16* pOutputSamples)
90870 | {
90871 |     ma_uint64 i;
90872 |     for (i = 0; i < frameCount; ++i) {
90873 |         ma_uint32 left = (ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
90874 |         ma_uint32 side = (ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
90875 |         ma_uint32 right = left - side;
90876 |         left >>= 16;
90877 |         right >>= 16;
90878 |         pOutputSamples[i*2+0] = (ma_int16)left;
90879 |         pOutputSamples[i*2+1] = (ma_int16)right;
90880 |     }
90881 | }
90882 | #endif
90883 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_left_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
90884 | {
90885 |     ma_uint64 i;
90886 |     ma_uint64 frameCount4 = frameCount >> 2;
90887 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90888 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90889 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90890 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90891 |     for (i = 0; i < frameCount4; ++i) {
90892 |         ma_uint32 left0 = pInputSamples0U32[i*4+0] << shift0;
90893 |         ma_uint32 left1 = pInputSamples0U32[i*4+1] << shift0;
90894 |         ma_uint32 left2 = pInputSamples0U32[i*4+2] << shift0;
90895 |         ma_uint32 left3 = pInputSamples0U32[i*4+3] << shift0;
90896 |         ma_uint32 side0 = pInputSamples1U32[i*4+0] << shift1;
90897 |         ma_uint32 side1 = pInputSamples1U32[i*4+1] << shift1;
90898 |         ma_uint32 side2 = pInputSamples1U32[i*4+2] << shift1;
90899 |         ma_uint32 side3 = pInputSamples1U32[i*4+3] << shift1;
90900 |         ma_uint32 right0 = left0 - side0;
90901 |         ma_uint32 right1 = left1 - side1;
90902 |         ma_uint32 right2 = left2 - side2;
90903 |         ma_uint32 right3 = left3 - side3;
90904 |         left0 >>= 16;
90905 |         left1 >>= 16;
90906 |         left2 >>= 16;
90907 |         left3 >>= 16;
90908 |         right0 >>= 16;
90909 |         right1 >>= 16;
90910 |         right2 >>= 16;
90911 |         right3 >>= 16;
90912 |         pOutputSamples[i*8+0] = (ma_int16)left0;
90913 |         pOutputSamples[i*8+1] = (ma_int16)right0;
90914 |         pOutputSamples[i*8+2] = (ma_int16)left1;
90915 |         pOutputSamples[i*8+3] = (ma_int16)right1;
90916 |         pOutputSamples[i*8+4] = (ma_int16)left2;
90917 |         pOutputSamples[i*8+5] = (ma_int16)right2;
90918 |         pOutputSamples[i*8+6] = (ma_int16)left3;
90919 |         pOutputSamples[i*8+7] = (ma_int16)right3;
90920 |     }
90921 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90922 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
90923 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
90924 |         ma_uint32 right = left - side;
90925 |         left >>= 16;
90926 |         right >>= 16;
90927 |         pOutputSamples[i*2+0] = (ma_int16)left;
90928 |         pOutputSamples[i*2+1] = (ma_int16)right;

```



```

90929 |     }
90930 | }
90931 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90932 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_left_side_sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
90933 | {
90934 |     ma_uint64 i;
90935 |     ma_uint64 frameCount4 = frameCount >> 2;
90936 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90937 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90938 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90939 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90940 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90941 |     for (i = 0; i < frameCount4; ++i) {
90942 |         __m128i left  = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
90943 |         __m128i side  = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
90944 |         __m128i right = _mm_sub_epi32(left, side);
90945 |         left = _mm_srai_epi32(left, 16);
90946 |         right = _mm_srai_epi32(right, 16);
90947 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8),
> ma_dr_flac__mm_packs_interleaved_epi32(left, right));
90948 |     }
90949 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90950 |         ma_uint32 left  = pInputSamples0U32[i] << shift0;
90951 |         ma_uint32 side  = pInputSamples1U32[i] << shift1;
90952 |         ma_uint32 right = left - side;
90953 |         left >>= 16;
90954 |         right >>= 16;
90955 |         pOutputSamples[i*2+0] = (ma_int16)left;
90956 |         pOutputSamples[i*2+1] = (ma_int16)right;
90957 |     }
90958 | }
90959 | #endif
90960 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
90961 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_left_side_neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
90962 | {
90963 |     ma_uint64 i;
90964 |     ma_uint64 frameCount4 = frameCount >> 2;
90965 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
90966 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
90967 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
90968 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
90969 |     int32x4_t shift0_4;
90970 |     int32x4_t shift1_4;
90971 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
90972 |     shift0_4 = vdupq_n_s32(shift0);
90973 |     shift1_4 = vdupq_n_s32(shift1);
90974 |     for (i = 0; i < frameCount4; ++i) {
90975 |         uint32x4_t left;
90976 |         uint32x4_t side;
90977 |         uint32x4_t right;
90978 |         left = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4);
90979 |         side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4);
90980 |         right = vsubq_u32(left, side);
90981 |         left = vshrq_n_u32(left, 16);
90982 |         right = vshrq_n_u32(right, 16);
90983 |         ma_dr_flac__vst2q_u16((ma_uint16*)pOutputSamples + i*8, vzip_u16(vmovn_u32(left),
> vmovn_u32(right)));
90984 |     }
90985 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
90986 |         ma_uint32 left  = pInputSamples0U32[i] << shift0;
90987 |         ma_uint32 side  = pInputSamples1U32[i] << shift1;
90988 |         ma_uint32 right = left - side;
90989 |         left >>= 16;
90990 |         right >>= 16;
90991 |         pOutputSamples[i*2+0] = (ma_int16)left;
90992 |         pOutputSamples[i*2+1] = (ma_int16)right;
90993 |     }
90994 | }
90995 | #endif
90996 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_left_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*

```

```

> pInputSamples1, ma_int16* pOutputSamples)
90997 | {
90998 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
90999 |     if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
91000 |         ma_dr_flac_read_pcm_frames_s16__decode_left_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91001 |     } else
91002 | #elif defined(MA_DR_FLAC_SUPPORT_NEON)
91003 |     if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
91004 |         ma_dr_flac_read_pcm_frames_s16__decode_left_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91005 |     } else
91006 | #endif
91007 |     {
91008 | #if 0
91009 |         ma_dr_flac_read_pcm_frames_s16__decode_left_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91010 | #else
91011 |         ma_dr_flac_read_pcm_frames_s16__decode_left_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91012 | #endif
91013 |     }
91014 | }
91015 | #if 0
91016 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_right_side__reference(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int16* pOutputSamples)
91017 | {
91018 |     ma_uint64 i;
91019 |     for (i = 0; i < frameCount; ++i) {
91020 |         ma_uint32 side = (ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
91021 |         ma_uint32 right = (ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
91022 |         ma_uint32 left = right + side;
91023 |         left >>= 16;
91024 |         right >>= 16;
91025 |         pOutputSamples[i*2+0] = (ma_int16)left;
91026 |         pOutputSamples[i*2+1] = (ma_int16)right;
91027 |     }
91028 | }
91029 | #endif
91030 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_right_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91031 | {
91032 |     ma_uint64 i;
91033 |     ma_uint64 frameCount4 = frameCount >> 2;
91034 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91035 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91036 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91037 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91038 |     for (i = 0; i < frameCount4; ++i) {
91039 |         ma_uint32 side0 = pInputSamples0U32[i*4+0] << shift0;
91040 |         ma_uint32 side1 = pInputSamples0U32[i*4+1] << shift0;
91041 |         ma_uint32 side2 = pInputSamples0U32[i*4+2] << shift0;
91042 |         ma_uint32 side3 = pInputSamples0U32[i*4+3] << shift0;
91043 |         ma_uint32 right0 = pInputSamples1U32[i*4+0] << shift1;
91044 |         ma_uint32 right1 = pInputSamples1U32[i*4+1] << shift1;
91045 |         ma_uint32 right2 = pInputSamples1U32[i*4+2] << shift1;
91046 |         ma_uint32 right3 = pInputSamples1U32[i*4+3] << shift1;
91047 |         ma_uint32 left0 = right0 + side0;
91048 |         ma_uint32 left1 = right1 + side1;
91049 |         ma_uint32 left2 = right2 + side2;
91050 |         ma_uint32 left3 = right3 + side3;
91051 |         left0 >>= 16;
91052 |         left1 >>= 16;
91053 |         left2 >>= 16;
91054 |         left3 >>= 16;
91055 |         right0 >>= 16;
91056 |         right1 >>= 16;
91057 |         right2 >>= 16;
91058 |         right3 >>= 16;
91059 |         pOutputSamples[i*8+0] = (ma_int16)left0;
91060 |         pOutputSamples[i*8+1] = (ma_int16)right0;
91061 |         pOutputSamples[i*8+2] = (ma_int16)left1;
91062 |         pOutputSamples[i*8+3] = (ma_int16)right1;

```

```

91063 |         pOutputSamples[i*8+4] = (ma_int16)left2;
91064 |         pOutputSamples[i*8+5] = (ma_int16)right2;
91065 |         pOutputSamples[i*8+6] = (ma_int16)left3;
91066 |         pOutputSamples[i*8+7] = (ma_int16)right3;
91067 |     }
91068 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91069 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
91070 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
91071 |         ma_uint32 left = right + side;
91072 |         left >>= 16;
91073 |         right >>= 16;
91074 |         pOutputSamples[i*2+0] = (ma_int16)left;
91075 |         pOutputSamples[i*2+1] = (ma_int16)right;
91076 |     }
91077 | }
91078 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91079 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_right_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91080 | {
91081 |     ma_uint64 i;
91082 |     ma_uint64 frameCount4 = frameCount >> 2;
91083 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91084 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91085 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91086 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91087 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91088 |     for (i = 0; i < frameCount4; ++i) {
91089 |         __m128i side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
91090 |         __m128i right = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
91091 |         __m128i left = _mm_add_epi32(right, side);
91092 |         left = _mm_srai_epi32(left, 16);
91093 |         right = _mm_srai_epi32(right, 16);
91094 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8),
> ma_dr_flac__mm_packs_interleaved_epi32(left, right));
91095 |     }
91096 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91097 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
91098 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
91099 |         ma_uint32 left = right + side;
91100 |         left >>= 16;
91101 |         right >>= 16;
91102 |         pOutputSamples[i*2+0] = (ma_int16)left;
91103 |         pOutputSamples[i*2+1] = (ma_int16)right;
91104 |     }
91105 | }
91106 | #endif
91107 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
91108 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_right_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91109 | {
91110 |     ma_uint64 i;
91111 |     ma_uint64 frameCount4 = frameCount >> 2;
91112 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91113 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91114 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91115 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91116 |     int32x4_t shift0_4;
91117 |     int32x4_t shift1_4;
91118 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91119 |     shift0_4 = vdupq_n_s32(shift0);
91120 |     shift1_4 = vdupq_n_s32(shift1);
91121 |     for (i = 0; i < frameCount4; ++i) {
91122 |         uint32x4_t side;
91123 |         uint32x4_t right;
91124 |         uint32x4_t left;
91125 |         side = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4);
91126 |         right = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4);
91127 |         left = vaddq_u32(right, side);
91128 |         left = vshrq_n_u32(left, 16);
91129 |         right = vshrq_n_u32(right, 16);
91130 |         ma_dr_flac__vst2q_u16((ma_uint16*)pOutputSamples + i*8, vzip_u16(vmovn_u32(left),
> vmovn_u32(right)));
91131 |     }

```

```

91132 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91133 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
91134 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
91135 |         ma_uint32 left = right + side;
91136 |         left >>= 16;
91137 |         right >>= 16;
91138 |         pOutputSamples[i*2+0] = (ma_int16)left;
91139 |         pOutputSamples[i*2+1] = (ma_int16)right;
91140 |     }
91141 | }
91142 | #endif
91143 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_right_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91144 | {
91145 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91146 |         if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
91147 |             ma_dr_flac_read_pcm_frames_s16__decode_right_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91148 |         } else
91149 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
91150 |             if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
91151 |                 ma_dr_flac_read_pcm_frames_s16__decode_right_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91152 |             } else
91153 |             #endif
91154 |             {
91155 |                 #if 0
91156 |                     ma_dr_flac_read_pcm_frames_s16__decode_right_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91157 |                 #else
91158 |                     ma_dr_flac_read_pcm_frames_s16__decode_right_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91159 |                 #endif
91160 |             }
91161 |         }
91162 |         #if 0
91163 |         static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_mid_side__reference(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91164 |         {
91165 |             for (ma_uint64 i = 0; i < frameCount; ++i) {
91166 |                 ma_uint32 mid = (ma_uint32)pInputSamples0[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91167 |                 ma_uint32 side = (ma_uint32)pInputSamples1[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91168 |                 mid = (mid << 1) | (side & 0x01);
91169 |                 pOutputSamples[i*2+0] = (ma_int16)((((ma_uint32)((ma_int32)(mid + side) >> 1) <<
> unusedBitsPerSample) >> 16);
91170 |                 pOutputSamples[i*2+1] = (ma_int16)((((ma_uint32)((ma_int32)(mid - side) >> 1) <<
> unusedBitsPerSample) >> 16);
91171 |             }
91172 |         }
91173 |     #endif
91174 |     static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_mid_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91175 |     {
91176 |         ma_uint64 i;
91177 |         ma_uint64 frameCount4 = frameCount >> 2;
91178 |         const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91179 |         const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91180 |         ma_uint32 shift = unusedBitsPerSample;
91181 |         if (shift > 0) {
91182 |             shift -= 1;
91183 |             for (i = 0; i < frameCount4; ++i) {
91184 |                 ma_uint32 temp0L;
91185 |                 ma_uint32 temp1L;
91186 |                 ma_uint32 temp2L;
91187 |                 ma_uint32 temp3L;
91188 |                 ma_uint32 temp0R;
91189 |                 ma_uint32 temp1R;
91190 |                 ma_uint32 temp2R;
91191 |                 ma_uint32 temp3R;
91192 |                 ma_uint32 mid0 = pInputSamples0U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91193 |                 ma_uint32 mid1 = pInputSamples0U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91194 |                 ma_uint32 mid2 = pInputSamples0U32[i*4+2] <<

```

```

> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91195 |     ma_uint32 mid3 = pInputSamples0U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91196 |     ma_uint32 side0 = pInputSamples1U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91197 |     ma_uint32 side1 = pInputSamples1U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91198 |     ma_uint32 side2 = pInputSamples1U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91199 |     ma_uint32 side3 = pInputSamples1U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91200 |     mid0 = (mid0 << 1) | (side0 & 0x01);
91201 |     mid1 = (mid1 << 1) | (side1 & 0x01);
91202 |     mid2 = (mid2 << 1) | (side2 & 0x01);
91203 |     mid3 = (mid3 << 1) | (side3 & 0x01);
91204 |     temp0L = (mid0 + side0) << shift;
91205 |     temp1L = (mid1 + side1) << shift;
91206 |     temp2L = (mid2 + side2) << shift;
91207 |     temp3L = (mid3 + side3) << shift;
91208 |     temp0R = (mid0 - side0) << shift;
91209 |     temp1R = (mid1 - side1) << shift;
91210 |     temp2R = (mid2 - side2) << shift;
91211 |     temp3R = (mid3 - side3) << shift;
91212 |     temp0L >>= 16;
91213 |     temp1L >>= 16;
91214 |     temp2L >>= 16;
91215 |     temp3L >>= 16;
91216 |     temp0R >>= 16;
91217 |     temp1R >>= 16;
91218 |     temp2R >>= 16;
91219 |     temp3R >>= 16;
91220 |     pOutputSamples[i*8+0] = (ma_int16)temp0L;
91221 |     pOutputSamples[i*8+1] = (ma_int16)temp0R;
91222 |     pOutputSamples[i*8+2] = (ma_int16)temp1L;
91223 |     pOutputSamples[i*8+3] = (ma_int16)temp1R;
91224 |     pOutputSamples[i*8+4] = (ma_int16)temp2L;
91225 |     pOutputSamples[i*8+5] = (ma_int16)temp2R;
91226 |     pOutputSamples[i*8+6] = (ma_int16)temp3L;
91227 |     pOutputSamples[i*8+7] = (ma_int16)temp3R;
91228 | }
91229 | } else {
91230 |     for (i = 0; i < frameCount4; ++i) {
91231 |         ma_uint32 temp0L;
91232 |         ma_uint32 temp1L;
91233 |         ma_uint32 temp2L;
91234 |         ma_uint32 temp3L;
91235 |         ma_uint32 temp0R;
91236 |         ma_uint32 temp1R;
91237 |         ma_uint32 temp2R;
91238 |         ma_uint32 temp3R;
91239 |         ma_uint32 mid0 = pInputSamples0U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91240 |         ma_uint32 mid1 = pInputSamples0U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91241 |         ma_uint32 mid2 = pInputSamples0U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91242 |         ma_uint32 mid3 = pInputSamples0U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91243 |         ma_uint32 side0 = pInputSamples1U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91244 |         ma_uint32 side1 = pInputSamples1U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91245 |         ma_uint32 side2 = pInputSamples1U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91246 |         ma_uint32 side3 = pInputSamples1U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91247 |         mid0 = (mid0 << 1) | (side0 & 0x01);
91248 |         mid1 = (mid1 << 1) | (side1 & 0x01);
91249 |         mid2 = (mid2 << 1) | (side2 & 0x01);
91250 |         mid3 = (mid3 << 1) | (side3 & 0x01);
91251 |         temp0L = ((ma_int32)(mid0 + side0) >> 1);
91252 |         temp1L = ((ma_int32)(mid1 + side1) >> 1);
91253 |         temp2L = ((ma_int32)(mid2 + side2) >> 1);
91254 |         temp3L = ((ma_int32)(mid3 + side3) >> 1);
91255 |         temp0R = ((ma_int32)(mid0 - side0) >> 1);
91256 |         temp1R = ((ma_int32)(mid1 - side1) >> 1);
91257 |         temp2R = ((ma_int32)(mid2 - side2) >> 1);
91258 |         temp3R = ((ma_int32)(mid3 - side3) >> 1);
91259 |         temp0L >>= 16;

```

```

91260 |         temp1L >>= 16;
91261 |         temp2L >>= 16;
91262 |         temp3L >>= 16;
91263 |         temp0R >>= 16;
91264 |         temp1R >>= 16;
91265 |         temp2R >>= 16;
91266 |         temp3R >>= 16;
91267 |         pOutputSamples[i*8+0] = (ma_int16)temp0L;
91268 |         pOutputSamples[i*8+1] = (ma_int16)temp0R;
91269 |         pOutputSamples[i*8+2] = (ma_int16)temp1L;
91270 |         pOutputSamples[i*8+3] = (ma_int16)temp1R;
91271 |         pOutputSamples[i*8+4] = (ma_int16)temp2L;
91272 |         pOutputSamples[i*8+5] = (ma_int16)temp2R;
91273 |         pOutputSamples[i*8+6] = (ma_int16)temp3L;
91274 |         pOutputSamples[i*8+7] = (ma_int16)temp3R;
91275 |     }
91276 | }
91277 | for (i = (frameCount4 << 2); i < frameCount; ++i) {
91278 |     ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91279 |     ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91280 |     mid = (mid << 1) | (side & 0x01);
91281 |     pOutputSamples[i*2+0] = (ma_int16)(((ma_uint32)((ma_int32)(mid + side) >> 1) <<
> unusedBitsPerSample) >> 16);
91282 |     pOutputSamples[i*2+1] = (ma_int16)(((ma_uint32)((ma_int32)(mid - side) >> 1) <<
> unusedBitsPerSample) >> 16);
91283 | }
91284 | }
91285 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91286 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_mid_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91287 | {
91288 |     ma_uint64 i;
91289 |     ma_uint64 frameCount4 = frameCount >> 2;
91290 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91291 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91292 |     ma_uint32 shift = unusedBitsPerSample;
91293 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91294 |     if (shift == 0) {
91295 |         for (i = 0; i < frameCount4; ++i) {
91296 |             __m128i mid;
91297 |             __m128i side;
91298 |             __m128i left;
91299 |             __m128i right;
91300 |             mid = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i),
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
91301 |             side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i),
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
91302 |             mid = _mm_or_si128(_mm_slli_epi32(mid, 1), _mm_and_si128(side, _mm_set1_epi32(0x01)));
91303 |             left = _mm_srai_epi32(_mm_add_epi32(mid, side), 1);
91304 |             right = _mm_srai_epi32(_mm_sub_epi32(mid, side), 1);
91305 |             left = _mm_srai_epi32(left, 16);
91306 |             right = _mm_srai_epi32(right, 16);
91307 |             _mm_storeu_si128((__m128i*)(pOutputSamples + i*8),
> ma_dr_flac__mm_packs_interleaved_epi32(left, right));
91308 |         }
91309 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
91310 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91311 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91312 |             mid = (mid << 1) | (side & 0x01);
91313 |             pOutputSamples[i*2+0] = (ma_int16)(((ma_int32)(mid + side) >> 1) >> 16);
91314 |             pOutputSamples[i*2+1] = (ma_int16)(((ma_int32)(mid - side) >> 1) >> 16);
91315 |         }
91316 |     } else {
91317 |         shift -= 1;
91318 |         for (i = 0; i < frameCount4; ++i) {
91319 |             __m128i mid;
91320 |             __m128i side;
91321 |             __m128i left;
91322 |             __m128i right;
91323 |             mid = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i),
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
91324 |             side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i),
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
91325 |             mid = _mm_or_si128(_mm_slli_epi32(mid, 1), _mm_and_si128(side, _mm_set1_epi32(0x01)));

```



```

91326 |         left = _mm_slli_epi32(_mm_add_epi32(mid, side), shift);
91327 |         right = _mm_slli_epi32(_mm_sub_epi32(mid, side), shift);
91328 |         left = _mm_srai_epi32(left, 16);
91329 |         right = _mm_srai_epi32(right, 16);
91330 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8),
> ma_dr_flac__mm_packs_interleaved_epi32(left, right));
91331 |     }
91332 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91333 |         ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91334 |         ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91335 |         mid = (mid << 1) | (side & 0x01);
91336 |         pOutputSamples[i*2+0] = (ma_int16)((((mid + side) << shift) >> 16);
91337 |         pOutputSamples[i*2+1] = (ma_int16)((((mid - side) << shift) >> 16);
91338 |     }
91339 | }
91340 | }
91341 | #endif
91342 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
91343 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_mid_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91344 | {
91345 |     ma_uint64 i;
91346 |     ma_uint64 frameCount4 = frameCount >> 2;
91347 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91348 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91349 |     ma_uint32 shift = unusedBitsPerSample;
91350 |     int32x4_t wbpsShift0_4;
91351 |     int32x4_t wbpsShift1_4;
91352 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91353 |     wbpsShift0_4 = vdupq_n_s32(pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
91354 |     wbpsShift1_4 = vdupq_n_s32(pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
91355 |     if (shift == 0) {
91356 |         for (i = 0; i < frameCount4; ++i) {
91357 |             uint32x4_t mid;
91358 |             uint32x4_t side;
91359 |             int32x4_t left;
91360 |             int32x4_t right;
91361 |             mid = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), wbpsShift0_4);
91362 |             side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), wbpsShift1_4);
91363 |             mid = vorrq_u32(vshlq_n_u32(mid, 1), vandq_u32(side, vdupq_n_u32(1)));
91364 |             left = vshrq_n_s32(vreinterpretq_s32_u32(vaddq_u32(mid, side)), 1);
91365 |             right = vshrq_n_s32(vreinterpretq_s32_u32(vsubq_u32(mid, side)), 1);
91366 |             left = vshrq_n_s32(left, 16);
91367 |             right = vshrq_n_s32(right, 16);
91368 |             ma_dr_flac__vst2q_s16(pOutputSamples + i*8, vzip_s16(vmovn_s32(left),
> vmovn_s32(right)));
91369 |         }
91370 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
91371 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91372 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91373 |             mid = (mid << 1) | (side & 0x01);
91374 |             pOutputSamples[i*2+0] = (ma_int16)((((ma_int32)(mid + side) >> 1) >> 16);
91375 |             pOutputSamples[i*2+1] = (ma_int16)((((ma_int32)(mid - side) >> 1) >> 16);
91376 |         }
91377 |     } else {
91378 |         int32x4_t shift4;
91379 |         shift -= 1;
91380 |         shift4 = vdupq_n_s32(shift);
91381 |         for (i = 0; i < frameCount4; ++i) {
91382 |             uint32x4_t mid;
91383 |             uint32x4_t side;
91384 |             int32x4_t left;
91385 |             int32x4_t right;
91386 |             mid = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), wbpsShift0_4);
91387 |             side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), wbpsShift1_4);
91388 |             mid = vorrq_u32(vshlq_n_u32(mid, 1), vandq_u32(side, vdupq_n_u32(1)));
91389 |             left = vreinterpretq_s32_u32(vshlq_u32(vaddq_u32(mid, side), shift4));
91390 |             right = vreinterpretq_s32_u32(vshlq_u32(vsubq_u32(mid, side), shift4));
91391 |             left = vshrq_n_s32(left, 16);
91392 |             right = vshrq_n_s32(right, 16);
91393 |             ma_dr_flac__vst2q_s16(pOutputSamples + i*8, vzip_s16(vmovn_s32(left),
> vmovn_s32(right)));
91394 |         }
91395 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {

```



```

91396 |         ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91397 |         ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91398 |         mid = (mid << 1) | (side & 0x01);
91399 |         pOutputSamples[i*2+0] = (ma_int16)(((mid + side) << shift) >> 16);
91400 |         pOutputSamples[i*2+1] = (ma_int16)(((mid - side) << shift) >> 16);
91401 |     }
91402 | }
91403 | }
91404 | #endif
91405 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_mid_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91406 | {
91407 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91408 |         if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
91409 |             ma_dr_flac_read_pcm_frames_s16__decode_mid_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91410 |         } else
91411 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
91412 |             if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
91413 |                 ma_dr_flac_read_pcm_frames_s16__decode_mid_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91414 |             } else
91415 |             #endif
91416 |             {
91417 |                 #if 0
91418 |                     ma_dr_flac_read_pcm_frames_s16__decode_mid_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91419 |                 #else
91420 |                     ma_dr_flac_read_pcm_frames_s16__decode_mid_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91421 |                 #endif
91422 |             }
91423 |         }
91424 |     #if 0
91425 |     static MA_INLINE void
> ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__reference(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91426 |     {
91427 |         for (ma_uint64 i = 0; i < frameCount; ++i) {
91428 |             pOutputSamples[i*2+0] = (ma_int16)((ma_int32)((ma_uint32)pInputSamples0[i] <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample)) >> 16);
91429 |             pOutputSamples[i*2+1] = (ma_int16)((ma_int32)((ma_uint32)pInputSamples1[i] <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample)) >> 16);
91430 |         }
91431 |     }
91432 | #endif
91433 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__scalar(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int16* pOutputSamples)
91434 | {
91435 |     ma_uint64 i;
91436 |     ma_uint64 frameCount4 = frameCount >> 2;
91437 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91438 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91439 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91440 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91441 |     for (i = 0; i < frameCount4; ++i) {
91442 |         ma_uint32 tempL0 = pInputSamples0U32[i*4+0] << shift0;
91443 |         ma_uint32 tempL1 = pInputSamples0U32[i*4+1] << shift0;
91444 |         ma_uint32 tempL2 = pInputSamples0U32[i*4+2] << shift0;
91445 |         ma_uint32 tempL3 = pInputSamples0U32[i*4+3] << shift0;
91446 |         ma_uint32 tempR0 = pInputSamples1U32[i*4+0] << shift1;
91447 |         ma_uint32 tempR1 = pInputSamples1U32[i*4+1] << shift1;
91448 |         ma_uint32 tempR2 = pInputSamples1U32[i*4+2] << shift1;
91449 |         ma_uint32 tempR3 = pInputSamples1U32[i*4+3] << shift1;
91450 |         tempL0 >>= 16;
91451 |         tempL1 >>= 16;
91452 |         tempL2 >>= 16;
91453 |         tempL3 >>= 16;
91454 |         tempR0 >>= 16;
91455 |         tempR1 >>= 16;
91456 |         tempR2 >>= 16;
91457 |         tempR3 >>= 16;

```

```

91458 |         pOutputSamples[i*8+0] = (ma_int16)tempL0;
91459 |         pOutputSamples[i*8+1] = (ma_int16)tempR0;
91460 |         pOutputSamples[i*8+2] = (ma_int16)tempL1;
91461 |         pOutputSamples[i*8+3] = (ma_int16)tempR1;
91462 |         pOutputSamples[i*8+4] = (ma_int16)tempL2;
91463 |         pOutputSamples[i*8+5] = (ma_int16)tempR2;
91464 |         pOutputSamples[i*8+6] = (ma_int16)tempL3;
91465 |         pOutputSamples[i*8+7] = (ma_int16)tempR3;
91466 |     }
91467 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91468 |         pOutputSamples[i*2+0] = (ma_int16)((pInputSamples0U32[i] << shift0) >> 16);
91469 |         pOutputSamples[i*2+1] = (ma_int16)((pInputSamples1U32[i] << shift1) >> 16);
91470 |     }
91471 | }
91472 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91473 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__sse2(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int16* pOutputSamples)
91474 | {
91475 |     ma_uint64 i;
91476 |     ma_uint64 frameCount4 = frameCount >> 2;
91477 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91478 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91479 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91480 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91481 |     for (i = 0; i < frameCount4; ++i) {
91482 |         __m128i left = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
91483 |         __m128i right = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
91484 |         left = _mm_srai_epi32(left, 16);
91485 |         right = _mm_srai_epi32(right, 16);
91486 |         _mm_storeu_si128((__m128i*)(pOutputSamples + i*8),
> ma_dr_flac__mm_packs_interleaved_epi32(left, right));
91487 |     }
91488 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91489 |         pOutputSamples[i*2+0] = (ma_int16)((pInputSamples0U32[i] << shift0) >> 16);
91490 |         pOutputSamples[i*2+1] = (ma_int16)((pInputSamples1U32[i] << shift1) >> 16);
91491 |     }
91492 | }
91493 | #endif
91494 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
91495 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__neon(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, ma_int16* pOutputSamples)
91496 | {
91497 |     ma_uint64 i;
91498 |     ma_uint64 frameCount4 = frameCount >> 2;
91499 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91500 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91501 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91502 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91503 |     int32x4_t shift0_4 = vdupq_n_s32(shift0);
91504 |     int32x4_t shift1_4 = vdupq_n_s32(shift1);
91505 |     for (i = 0; i < frameCount4; ++i) {
91506 |         int32x4_t left;
91507 |         int32x4_t right;
91508 |         left = vreinterpretq_s32_u32(vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4));
91509 |         right = vreinterpretq_s32_u32(vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4));
91510 |         left = vshrq_n_s32(left, 16);
91511 |         right = vshrq_n_s32(right, 16);
91512 |         ma_dr_flac__vst2q_s16(pOutputSamples + i*8, vzip_s16(vmovn_s32(left), vmovn_s32(right)));
91513 |     }
91514 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91515 |         pOutputSamples[i*2+0] = (ma_int16)((pInputSamples0U32[i] << shift0) >> 16);
91516 |         pOutputSamples[i*2+1] = (ma_int16)((pInputSamples1U32[i] << shift1) >> 16);
91517 |     }
91518 | }
91519 | #endif
91520 | static MA_INLINE void ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, ma_int16* pOutputSamples)
91521 | {
91522 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91523 |         if (ma_dr_flac__gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
91524 |             ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);

```

```

91525 |     } else
91526 | #elif defined(MA_DR_FLAC_SUPPORT_NEON)
91527 |     if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
91528 |         ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91529 |     } else
91530 | #endif
91531 |     {
91532 | #if 0
91533 |         ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91534 | #else
91535 |         ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91536 | #endif
91537 |     }
91538 | }
91539 | MA_API ma_uint64 ma_dr_flac_read_pcm_frames_s16(ma_dr_flac* pFlac, ma_uint64 framesToRead, ma_int16*
> pBufferOut)
91540 | {
91541 |     ma_uint64 framesRead;
91542 |     ma_uint32 unusedBitsPerSample;
91543 |     if (pFlac == NULL || framesToRead == 0) {
91544 |         return 0;
91545 |     }
91546 |     if (pBufferOut == NULL) {
91547 |         return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, framesToRead);
91548 |     }
91549 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 32);
91550 |     unusedBitsPerSample = 32 - pFlac->bitsPerSample;
91551 |     framesRead = 0;
91552 |     while (framesToRead > 0) {
91553 |         if (pFlac->currentFLACFrame.pcmFramesRemaining == 0) {
91554 |             if (!ma_dr_flac__read_and_decode_next_flac_frame(pFlac)) {
91555 |                 break;
91556 |             }
91557 |         } else {
91558 |             unsigned int channelCount = ma_dr_flac__get_channel_count_from_channel_assignment(pFlac-
> >currentFLACFrame.header.channelAssignment);
91559 |             ma_uint64 iFirstPCMFrame = pFlac->currentFLACFrame.header.blockSizeInPCMFrames -
> pFlac->currentFLACFrame.pcmFramesRemaining;
91560 |             ma_uint64 frameCountThisIteration = framesToRead;
91561 |             if (frameCountThisIteration > pFlac->currentFLACFrame.pcmFramesRemaining) {
91562 |                 frameCountThisIteration = pFlac->currentFLACFrame.pcmFramesRemaining;
91563 |             }
91564 |             if (channelCount == 2) {
91565 |                 const ma_int32* pDecodedSamples0 = pFlac->currentFLACFrame.subframes[0].pSamplesS32
> + iFirstPCMFrame;
91566 |                 const ma_int32* pDecodedSamples1 = pFlac->currentFLACFrame.subframes[1].pSamplesS32
> + iFirstPCMFrame;
91567 |                 switch (pFlac->currentFLACFrame.header.channelAssignment)
91568 |                 {
91569 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_LEFT_SIDE:
91570 |                     {
91571 |                         ma_dr_flac_read_pcm_frames_s16__decode_left_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
91572 |                     } break;
91573 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_RIGHT_SIDE:
91574 |                     {
91575 |                         ma_dr_flac_read_pcm_frames_s16__decode_right_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
91576 |                     } break;
91577 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_MID_SIDE:
91578 |                     {
91579 |                         ma_dr_flac_read_pcm_frames_s16__decode_mid_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
91580 |                     } break;
91581 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_INDEPENDENT:
91582 |                     default:
91583 |                     {
91584 |                         ma_dr_flac_read_pcm_frames_s16__decode_independent_stereo(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
91585 |                     } break;
91586 |                     }
91587 |                 } else {
91588 |                     ma_uint64 i;
91589 |                     for (i = 0; i < frameCountThisIteration; ++i) {
91590 |                         unsigned int j;
91591 |                         for (j = 0; j < channelCount; ++j) {

```

```

91592 |             ma_int32 samplesS32 =
> (ma_int32)((ma_uint32)(pFlac->currentFLACFrame.subframes[j].pSamplesS32[iFirstPCMFrame + i]) <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[j].wastedBitsPerSample));
91593 |             pBufferOut[(i*channelCount)+j] = (ma_int16)(samplesS32 >> 16);
91594 |         }
91595 |     }
91596 | }
91597 |     framesRead           += frameCountThisIteration;
91598 |     pBufferOut           += frameCountThisIteration * channelCount;
91599 |     framesToRead         -= frameCountThisIteration;
91600 |     pFlac->currentPCMFrame += frameCountThisIteration;
91601 |     pFlac->currentFLACFrame.pcmFramesRemaining -= (ma_uint32)frameCountThisIteration;
91602 | }
91603 | }
91604 | return framesRead;
91605 | }
91606 | #if 0
91607 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_left_side__reference(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, float* pOutputSamples)
91608 | {
91609 |     ma_uint64 i;
91610 |     for (i = 0; i < frameCount; ++i) {
91611 |         ma_uint32 left = (ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
91612 |         ma_uint32 side = (ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
91613 |         ma_uint32 right = left - side;
91614 |         pOutputSamples[i*2+0] = (float)((ma_int32)left / 2147483648.0);
91615 |         pOutputSamples[i*2+1] = (float)((ma_int32)right / 2147483648.0);
91616 |     }
91617 | }
91618 | #endif
91619 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_left_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91620 | {
91621 |     ma_uint64 i;
91622 |     ma_uint64 frameCount4 = frameCount >> 2;
91623 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91624 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91625 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91626 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91627 |     float factor = 1 / 2147483648.0;
91628 |     for (i = 0; i < frameCount4; ++i) {
91629 |         ma_uint32 left0 = pInputSamples0U32[i*4+0] << shift0;
91630 |         ma_uint32 left1 = pInputSamples0U32[i*4+1] << shift0;
91631 |         ma_uint32 left2 = pInputSamples0U32[i*4+2] << shift0;
91632 |         ma_uint32 left3 = pInputSamples0U32[i*4+3] << shift0;
91633 |         ma_uint32 side0 = pInputSamples1U32[i*4+0] << shift1;
91634 |         ma_uint32 side1 = pInputSamples1U32[i*4+1] << shift1;
91635 |         ma_uint32 side2 = pInputSamples1U32[i*4+2] << shift1;
91636 |         ma_uint32 side3 = pInputSamples1U32[i*4+3] << shift1;
91637 |         ma_uint32 right0 = left0 - side0;
91638 |         ma_uint32 right1 = left1 - side1;
91639 |         ma_uint32 right2 = left2 - side2;
91640 |         ma_uint32 right3 = left3 - side3;
91641 |         pOutputSamples[i*8+0] = (ma_int32)left0 * factor;
91642 |         pOutputSamples[i*8+1] = (ma_int32)right0 * factor;
91643 |         pOutputSamples[i*8+2] = (ma_int32)left1 * factor;
91644 |         pOutputSamples[i*8+3] = (ma_int32)right1 * factor;
91645 |         pOutputSamples[i*8+4] = (ma_int32)left2 * factor;
91646 |         pOutputSamples[i*8+5] = (ma_int32)right2 * factor;
91647 |         pOutputSamples[i*8+6] = (ma_int32)left3 * factor;
91648 |         pOutputSamples[i*8+7] = (ma_int32)right3 * factor;
91649 |     }
91650 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91651 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
91652 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
91653 |         ma_uint32 right = left - side;
91654 |         pOutputSamples[i*2+0] = (ma_int32)left * factor;
91655 |         pOutputSamples[i*2+1] = (ma_int32)right * factor;
91656 |     }
91657 | }
91658 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91659 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_left_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*

```

```

    > pInputSamples1, float* pOutputSamples)
91660 | {
91661 |     ma_uint64 i;
91662 |     ma_uint64 frameCount4 = frameCount >> 2;
91663 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91664 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91665 |     ma_uint32 shift0 = (unusedBitsPerSample +
    > pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample) - 8;
91666 |     ma_uint32 shift1 = (unusedBitsPerSample +
    > pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample) - 8;
91667 |     __m128 factor;
91668 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91669 |     factor = _mm_set1_ps(1.0f / 8388608.0f);
91670 |     for (i = 0; i < frameCount4; ++i) {
91671 |         __m128i left = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
91672 |         __m128i side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
91673 |         __m128i right = _mm_sub_epi32(left, side);
91674 |         __m128 leftf = _mm_mul_ps(_mm_cvtepi32_ps(left), factor);
91675 |         __m128 rightf = _mm_mul_ps(_mm_cvtepi32_ps(right), factor);
91676 |         _mm_storeu_ps(pOutputSamples + i*8 + 0, _mm_unpacklo_ps(leftf, rightf));
91677 |         _mm_storeu_ps(pOutputSamples + i*8 + 4, _mm_unpackhi_ps(leftf, rightf));
91678 |     }
91679 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91680 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
91681 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
91682 |         ma_uint32 right = left - side;
91683 |         pOutputSamples[i*2+0] = (ma_int32)left / 8388608.0f;
91684 |         pOutputSamples[i*2+1] = (ma_int32)right / 8388608.0f;
91685 |     }
91686 | }
91687 | #endif
91688 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
91689 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_left_side_neon(ma_dr_flac* pFlac,
    > ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
    > pInputSamples1, float* pOutputSamples)
91690 | {
91691 |     ma_uint64 i;
91692 |     ma_uint64 frameCount4 = frameCount >> 2;
91693 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91694 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91695 |     ma_uint32 shift0 = (unusedBitsPerSample +
    > pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample) - 8;
91696 |     ma_uint32 shift1 = (unusedBitsPerSample +
    > pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample) - 8;
91697 |     float32x4_t factor4;
91698 |     int32x4_t shift0_4;
91699 |     int32x4_t shift1_4;
91700 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91701 |     factor4 = vdupq_n_f32(1.0f / 8388608.0f);
91702 |     shift0_4 = vdupq_n_s32(shift0);
91703 |     shift1_4 = vdupq_n_s32(shift1);
91704 |     for (i = 0; i < frameCount4; ++i) {
91705 |         uint32x4_t left;
91706 |         uint32x4_t side;
91707 |         uint32x4_t right;
91708 |         float32x4_t leftf;
91709 |         float32x4_t rightf;
91710 |         left = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4);
91711 |         side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4);
91712 |         right = vsubq_u32(left, side);
91713 |         leftf = vmulq_f32(vcvttq_f32_s32(vreinterpretq_s32_u32(left)), factor4);
91714 |         rightf = vmulq_f32(vcvttq_f32_s32(vreinterpretq_s32_u32(right)), factor4);
91715 |         ma_dr_flac__vst2q_f32(pOutputSamples + i*8, vzipq_f32(leftf, rightf));
91716 |     }
91717 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91718 |         ma_uint32 left = pInputSamples0U32[i] << shift0;
91719 |         ma_uint32 side = pInputSamples1U32[i] << shift1;
91720 |         ma_uint32 right = left - side;
91721 |         pOutputSamples[i*2+0] = (ma_int32)left / 8388608.0f;
91722 |         pOutputSamples[i*2+1] = (ma_int32)right / 8388608.0f;
91723 |     }
91724 | }
91725 | #endif
91726 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_left_side(ma_dr_flac* pFlac, ma_uint64
    > frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
    > pInputSamples1, float* pOutputSamples)
91727 | {
91728 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91729 |         if (ma_dr_flac__gIsSSE2Supported && pFlac->bitsPerSample <= 24) {

```

```

91730 |         ma_dr_flac_read_pcm_frames_f32__decode_left_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91731 |     } else
91732 |     #elif defined(MA_DR_FLAC_SUPPORT_NEON)
91733 |     if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
91734 |         ma_dr_flac_read_pcm_frames_f32__decode_left_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91735 |     } else
91736 |     #endif
91737 |     {
91738 |     #if 0
91739 |         ma_dr_flac_read_pcm_frames_f32__decode_left_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91740 |     #else
91741 |         ma_dr_flac_read_pcm_frames_f32__decode_left_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91742 |     #endif
91743 |     }
91744 | }
91745 | #if 0
91746 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_right_side__reference(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, float* pOutputSamples)
91747 | {
91748 |     ma_uint64 i;
91749 |     for (i = 0; i < frameCount; ++i) {
91750 |         ma_uint32 side = (ma_uint32)pInputSamples0[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
91751 |         ma_uint32 right = (ma_uint32)pInputSamples1[i] << (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
91752 |         ma_uint32 left = right + side;
91753 |         pOutputSamples[i*2+0] = (float)((ma_int32)left / 2147483648.0);
91754 |         pOutputSamples[i*2+1] = (float)((ma_int32)right / 2147483648.0);
91755 |     }
91756 | }
91757 | #endif
91758 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_right_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91759 | {
91760 |     ma_uint64 i;
91761 |     ma_uint64 frameCount4 = frameCount >> 2;
91762 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91763 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91764 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91765 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91766 |     float factor = 1 / 2147483648.0;
91767 |     for (i = 0; i < frameCount4; ++i) {
91768 |         ma_uint32 side0 = pInputSamples0U32[i*4+0] << shift0;
91769 |         ma_uint32 side1 = pInputSamples0U32[i*4+1] << shift0;
91770 |         ma_uint32 side2 = pInputSamples0U32[i*4+2] << shift0;
91771 |         ma_uint32 side3 = pInputSamples0U32[i*4+3] << shift0;
91772 |         ma_uint32 right0 = pInputSamples1U32[i*4+0] << shift1;
91773 |         ma_uint32 right1 = pInputSamples1U32[i*4+1] << shift1;
91774 |         ma_uint32 right2 = pInputSamples1U32[i*4+2] << shift1;
91775 |         ma_uint32 right3 = pInputSamples1U32[i*4+3] << shift1;
91776 |         ma_uint32 left0 = right0 + side0;
91777 |         ma_uint32 left1 = right1 + side1;
91778 |         ma_uint32 left2 = right2 + side2;
91779 |         ma_uint32 left3 = right3 + side3;
91780 |         pOutputSamples[i*8+0] = (ma_int32)left0 * factor;
91781 |         pOutputSamples[i*8+1] = (ma_int32)right0 * factor;
91782 |         pOutputSamples[i*8+2] = (ma_int32)left1 * factor;
91783 |         pOutputSamples[i*8+3] = (ma_int32)right1 * factor;
91784 |         pOutputSamples[i*8+4] = (ma_int32)left2 * factor;
91785 |         pOutputSamples[i*8+5] = (ma_int32)right2 * factor;
91786 |         pOutputSamples[i*8+6] = (ma_int32)left3 * factor;
91787 |         pOutputSamples[i*8+7] = (ma_int32)right3 * factor;
91788 |     }
91789 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91790 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
91791 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
91792 |         ma_uint32 left = right + side;
91793 |         pOutputSamples[i*2+0] = (ma_int32)left * factor;
91794 |         pOutputSamples[i*2+1] = (ma_int32)right * factor;
91795 |     }
91796 | }

```



```

91797 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91798 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_right_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91799 | {
91800 |     ma_uint64 i;
91801 |     ma_uint64 frameCount4 = frameCount >> 2;
91802 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91803 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91804 |     ma_uint32 shift0 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample) - 8;
91805 |     ma_uint32 shift1 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample) - 8;
91806 |     __m128 factor;
91807 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91808 |     factor = _mm_set1_ps(1.0f / 8388608.0f);
91809 |     for (i = 0; i < frameCount4; ++i) {
91810 |         __m128i side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
91811 |         __m128i right = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
91812 |         __m128i left = _mm_add_epi32(right, side);
91813 |         __m128 leftf = _mm_mul_ps(_mm_cvtepi32_ps(left), factor);
91814 |         __m128 rightf = _mm_mul_ps(_mm_cvtepi32_ps(right), factor);
91815 |         _mm_storeu_ps(pOutputSamples + i*8 + 0, _mm_unpacklo_ps(leftf, rightf));
91816 |         _mm_storeu_ps(pOutputSamples + i*8 + 4, _mm_unpackhi_ps(leftf, rightf));
91817 |     }
91818 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91819 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
91820 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
91821 |         ma_uint32 left = right + side;
91822 |         pOutputSamples[i*2+0] = (ma_int32)left / 8388608.0f;
91823 |         pOutputSamples[i*2+1] = (ma_int32)right / 8388608.0f;
91824 |     }
91825 | }
91826 | #endif
91827 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
91828 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_right_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91829 | {
91830 |     ma_uint64 i;
91831 |     ma_uint64 frameCount4 = frameCount >> 2;
91832 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91833 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91834 |     ma_uint32 shift0 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample) - 8;
91835 |     ma_uint32 shift1 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample) - 8;
91836 |     float32x4_t factor4;
91837 |     int32x4_t shift0_4;
91838 |     int32x4_t shift1_4;
91839 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
91840 |     factor4 = vdupq_n_f32(1.0f / 8388608.0f);
91841 |     shift0_4 = vdupq_n_s32(shift0);
91842 |     shift1_4 = vdupq_n_s32(shift1);
91843 |     for (i = 0; i < frameCount4; ++i) {
91844 |         uint32x4_t side;
91845 |         uint32x4_t right;
91846 |         uint32x4_t left;
91847 |         float32x4_t leftf;
91848 |         float32x4_t rightf;
91849 |         side = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4);
91850 |         right = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4);
91851 |         left = vaddq_u32(right, side);
91852 |         leftf = vmulq_f32(vcvttq_f32_s32(vreinterpretq_s32_u32(left)), factor4);
91853 |         rightf = vmulq_f32(vcvttq_f32_s32(vreinterpretq_s32_u32(right)), factor4);
91854 |         ma_dr_flac__vst2q_f32(pOutputSamples + i*8, vzipq_f32(leftf, rightf));
91855 |     }
91856 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
91857 |         ma_uint32 side = pInputSamples0U32[i] << shift0;
91858 |         ma_uint32 right = pInputSamples1U32[i] << shift1;
91859 |         ma_uint32 left = right + side;
91860 |         pOutputSamples[i*2+0] = (ma_int32)left / 8388608.0f;
91861 |         pOutputSamples[i*2+1] = (ma_int32)right / 8388608.0f;
91862 |     }
91863 | }
91864 | #endif
91865 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_right_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)

```



```

91866 | {
91867 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91868 |     if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
91869 |         ma_dr_flac_read_pcm_frames_f32__decode_right_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91870 |     } else
91871 | #elif defined(MA_DR_FLAC_SUPPORT_NEON)
91872 |     if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
91873 |         ma_dr_flac_read_pcm_frames_f32__decode_right_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91874 |     } else
91875 | #endif
91876 |     {
91877 |         #if 0
91878 |             ma_dr_flac_read_pcm_frames_f32__decode_right_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91879 |         #else
91880 |             ma_dr_flac_read_pcm_frames_f32__decode_right_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
91881 |         #endif
91882 |     }
91883 | }
91884 | #if 0
91885 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_mid_side__reference(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91886 | {
91887 |     for (ma_uint64 i = 0; i < frameCount; ++i) {
91888 |         ma_uint32 mid = (ma_uint32)pInputSamples0[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91889 |         ma_uint32 side = (ma_uint32)pInputSamples1[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91890 |         mid = (mid << 1) | (side & 0x01);
91891 |         pOutputSamples[i*2+0] = (float)((((ma_int32)(mid + side) >> 1) << (unusedBitsPerSample)) /
> 2147483648.0);
91892 |         pOutputSamples[i*2+1] = (float)((((ma_int32)(mid - side) >> 1) << (unusedBitsPerSample)) /
> 2147483648.0);
91893 |     }
91894 | }
91895 | #endif
91896 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_mid_side__scalar(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91897 | {
91898 |     ma_uint64 i;
91899 |     ma_uint64 frameCount4 = frameCount >> 2;
91900 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91901 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91902 |     ma_uint32 shift = unusedBitsPerSample;
91903 |     float factor = 1 / 2147483648.0;
91904 |     if (shift > 0) {
91905 |         shift -= 1;
91906 |         for (i = 0; i < frameCount4; ++i) {
91907 |             ma_uint32 temp0L;
91908 |             ma_uint32 temp1L;
91909 |             ma_uint32 temp2L;
91910 |             ma_uint32 temp3L;
91911 |             ma_uint32 temp0R;
91912 |             ma_uint32 temp1R;
91913 |             ma_uint32 temp2R;
91914 |             ma_uint32 temp3R;
91915 |             ma_uint32 mid0 = pInputSamples0U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91916 |             ma_uint32 mid1 = pInputSamples0U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91917 |             ma_uint32 mid2 = pInputSamples0U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91918 |             ma_uint32 mid3 = pInputSamples0U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91919 |             ma_uint32 side0 = pInputSamples1U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91920 |             ma_uint32 side1 = pInputSamples1U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91921 |             ma_uint32 side2 = pInputSamples1U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91922 |             ma_uint32 side3 = pInputSamples1U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91923 |             mid0 = (mid0 << 1) | (side0 & 0x01);
91924 |             mid1 = (mid1 << 1) | (side1 & 0x01);

```

```

91925 |         mid2 = (mid2 << 1) | (side2 & 0x01);
91926 |         mid3 = (mid3 << 1) | (side3 & 0x01);
91927 |         temp0L = (mid0 + side0) << shift;
91928 |         temp1L = (mid1 + side1) << shift;
91929 |         temp2L = (mid2 + side2) << shift;
91930 |         temp3L = (mid3 + side3) << shift;
91931 |         temp0R = (mid0 - side0) << shift;
91932 |         temp1R = (mid1 - side1) << shift;
91933 |         temp2R = (mid2 - side2) << shift;
91934 |         temp3R = (mid3 - side3) << shift;
91935 |         pOutputSamples[i*8+0] = (ma_int32)temp0L * factor;
91936 |         pOutputSamples[i*8+1] = (ma_int32)temp0R * factor;
91937 |         pOutputSamples[i*8+2] = (ma_int32)temp1L * factor;
91938 |         pOutputSamples[i*8+3] = (ma_int32)temp1R * factor;
91939 |         pOutputSamples[i*8+4] = (ma_int32)temp2L * factor;
91940 |         pOutputSamples[i*8+5] = (ma_int32)temp2R * factor;
91941 |         pOutputSamples[i*8+6] = (ma_int32)temp3L * factor;
91942 |         pOutputSamples[i*8+7] = (ma_int32)temp3R * factor;
91943 |     }
91944 | } else {
91945 |     for (i = 0; i < frameCount4; ++i) {
91946 |         ma_uint32 temp0L;
91947 |         ma_uint32 temp1L;
91948 |         ma_uint32 temp2L;
91949 |         ma_uint32 temp3L;
91950 |         ma_uint32 temp0R;
91951 |         ma_uint32 temp1R;
91952 |         ma_uint32 temp2R;
91953 |         ma_uint32 temp3R;
91954 |         ma_uint32 mid0 = pInputSamples0U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91955 |         ma_uint32 mid1 = pInputSamples0U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91956 |         ma_uint32 mid2 = pInputSamples0U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91957 |         ma_uint32 mid3 = pInputSamples0U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91958 |         ma_uint32 side0 = pInputSamples1U32[i*4+0] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91959 |         ma_uint32 side1 = pInputSamples1U32[i*4+1] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91960 |         ma_uint32 side2 = pInputSamples1U32[i*4+2] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91961 |         ma_uint32 side3 = pInputSamples1U32[i*4+3] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91962 |         mid0 = (mid0 << 1) | (side0 & 0x01);
91963 |         mid1 = (mid1 << 1) | (side1 & 0x01);
91964 |         mid2 = (mid2 << 1) | (side2 & 0x01);
91965 |         mid3 = (mid3 << 1) | (side3 & 0x01);
91966 |         temp0L = (ma_uint32)((ma_int32)(mid0 + side0) >> 1);
91967 |         temp1L = (ma_uint32)((ma_int32)(mid1 + side1) >> 1);
91968 |         temp2L = (ma_uint32)((ma_int32)(mid2 + side2) >> 1);
91969 |         temp3L = (ma_uint32)((ma_int32)(mid3 + side3) >> 1);
91970 |         temp0R = (ma_uint32)((ma_int32)(mid0 - side0) >> 1);
91971 |         temp1R = (ma_uint32)((ma_int32)(mid1 - side1) >> 1);
91972 |         temp2R = (ma_uint32)((ma_int32)(mid2 - side2) >> 1);
91973 |         temp3R = (ma_uint32)((ma_int32)(mid3 - side3) >> 1);
91974 |         pOutputSamples[i*8+0] = (ma_int32)temp0L * factor;
91975 |         pOutputSamples[i*8+1] = (ma_int32)temp0R * factor;
91976 |         pOutputSamples[i*8+2] = (ma_int32)temp1L * factor;
91977 |         pOutputSamples[i*8+3] = (ma_int32)temp1R * factor;
91978 |         pOutputSamples[i*8+4] = (ma_int32)temp2L * factor;
91979 |         pOutputSamples[i*8+5] = (ma_int32)temp2R * factor;
91980 |         pOutputSamples[i*8+6] = (ma_int32)temp3L * factor;
91981 |         pOutputSamples[i*8+7] = (ma_int32)temp3R * factor;
91982 |     }
91983 | }
91984 | for (i = (frameCount4 << 2); i < frameCount; ++i) {
91985 |     ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
91986 |     ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
91987 |     mid = (mid << 1) | (side & 0x01);
91988 |     pOutputSamples[i*2+0] = (ma_int32)((ma_uint32)((ma_int32)(mid + side) >> 1) <<
> unusedBitsPerSample) * factor;
91989 |     pOutputSamples[i*2+1] = (ma_int32)((ma_uint32)((ma_int32)(mid - side) >> 1) <<
> unusedBitsPerSample) * factor;
91990 | }
91991 | }

```

```

91992 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
91993 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_mid_side__sse2(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
91994 | {
91995 |     ma_uint64 i;
91996 |     ma_uint64 frameCount4 = frameCount >> 2;
91997 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
91998 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
91999 |     ma_uint32 shift = unusedBitsPerSample - 8;
92000 |     float factor;
92001 |     __m128 factor128;
92002 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
92003 |     factor = 1.0f / 8388608.0f;
92004 |     factor128 = _mm_set1_ps(factor);
92005 |     if (shift == 0) {
92006 |         for (i = 0; i < frameCount4; ++i) {
92007 |             __m128i mid;
92008 |             __m128i side;
92009 |             __m128i tempL;
92010 |             __m128i tempR;
92011 |             __m128 lefttf;
92012 |             __m128 righttf;
92013 |             mid = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i),
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
92014 |             side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i),
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
92015 |             mid = _mm_or_si128(_mm_slli_epi32(mid, 1), _mm_and_si128(side,
> _mm_set1_epi32(0x01)));
92016 |             tempL = _mm_srai_epi32(_mm_add_epi32(mid, side), 1);
92017 |             tempR = _mm_srai_epi32(_mm_sub_epi32(mid, side), 1);
92018 |             lefttf = _mm_mul_ps(_mm_cvtepi32_ps(tempL), factor128);
92019 |             righttf = _mm_mul_ps(_mm_cvtepi32_ps(tempR), factor128);
92020 |             _mm_storeu_ps(pOutputSamples + i*8 + 0, _mm_unpacklo_ps(lefttf, righttf));
92021 |             _mm_storeu_ps(pOutputSamples + i*8 + 4, _mm_unpackhi_ps(lefttf, righttf));
92022 |         }
92023 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
92024 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
92025 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
92026 |             mid = (mid << 1) | (side & 0x01);
92027 |             pOutputSamples[i*2+0] = ((ma_int32)(mid + side) >> 1) * factor;
92028 |             pOutputSamples[i*2+1] = ((ma_int32)(mid - side) >> 1) * factor;
92029 |         }
92030 |     } else {
92031 |         shift -= 1;
92032 |         for (i = 0; i < frameCount4; ++i) {
92033 |             __m128i mid;
92034 |             __m128i side;
92035 |             __m128i tempL;
92036 |             __m128i tempR;
92037 |             __m128 lefttf;
92038 |             __m128 righttf;
92039 |             mid = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i),
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
92040 |             side = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i),
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
92041 |             mid = _mm_or_si128(_mm_slli_epi32(mid, 1), _mm_and_si128(side,
> _mm_set1_epi32(0x01)));
92042 |             tempL = _mm_slli_epi32(_mm_add_epi32(mid, side), shift);
92043 |             tempR = _mm_slli_epi32(_mm_sub_epi32(mid, side), shift);
92044 |             lefttf = _mm_mul_ps(_mm_cvtepi32_ps(tempL), factor128);
92045 |             righttf = _mm_mul_ps(_mm_cvtepi32_ps(tempR), factor128);
92046 |             _mm_storeu_ps(pOutputSamples + i*8 + 0, _mm_unpacklo_ps(lefttf, righttf));
92047 |             _mm_storeu_ps(pOutputSamples + i*8 + 4, _mm_unpackhi_ps(lefttf, righttf));
92048 |         }
92049 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
92050 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
92051 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
92052 |             mid = (mid << 1) | (side & 0x01);
92053 |             pOutputSamples[i*2+0] = (ma_int32)((mid + side) << shift) * factor;
92054 |             pOutputSamples[i*2+1] = (ma_int32)((mid - side) << shift) * factor;
92055 |         }
92056 |     }
92057 | }
92058 | #endif

```

```

92059 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
92060 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_mid_side__neon(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
92061 | {
92062 |     ma_uint64 i;
92063 |     ma_uint64 frameCount4 = frameCount >> 2;
92064 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
92065 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
92066 |     ma_uint32 shift = unusedBitsPerSample - 8;
92067 |     float factor;
92068 |     float32x4_t factor4;
92069 |     int32x4_t shift4;
92070 |     int32x4_t wbps0_4;
92071 |     int32x4_t wbps1_4;
92072 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 24);
92073 |     factor = 1.0f / 8388608.0f;
92074 |     factor4 = vdupq_n_f32(factor);
92075 |     wbps0_4 = vdupq_n_s32(pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample);
92076 |     wbps1_4 = vdupq_n_s32(pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample);
92077 |     if (shift == 0) {
92078 |         for (i = 0; i < frameCount4; ++i) {
92079 |             int32x4_t lefti;
92080 |             int32x4_t righti;
92081 |             float32x4_t leftf;
92082 |             float32x4_t rightf;
92083 |             uint32x4_t mid = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), wbps0_4);
92084 |             uint32x4_t side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), wbps1_4);
92085 |             mid = vorrq_u32(vshlq_n_u32(mid, 1), vandq_u32(side, vdupq_n_u32(1)));
92086 |             lefti = vshrq_n_s32(vreinterpretq_s32_u32(vaddq_u32(mid, side), 1));
92087 |             righti = vshrq_n_s32(vreinterpretq_s32_u32(vsubq_u32(mid, side), 1));
92088 |             leftf = vmulq_f32(vcvttq_f32_s32(lefti), factor4);
92089 |             rightf = vmulq_f32(vcvttq_f32_s32(righti), factor4);
92090 |             ma_dr_flac__vst2q_f32(pOutputSamples + i*8, vzipq_f32(leftf, rightf));
92091 |         }
92092 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
92093 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
92094 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
92095 |             mid = (mid << 1) | (side & 0x01);
92096 |             pOutputSamples[i*2+0] = ((ma_int32)(mid + side) >> 1) * factor;
92097 |             pOutputSamples[i*2+1] = ((ma_int32)(mid - side) >> 1) * factor;
92098 |         }
92099 |     } else {
92100 |         shift -= 1;
92101 |         shift4 = vdupq_n_s32(shift);
92102 |         for (i = 0; i < frameCount4; ++i) {
92103 |             uint32x4_t mid;
92104 |             uint32x4_t side;
92105 |             int32x4_t lefti;
92106 |             int32x4_t righti;
92107 |             float32x4_t leftf;
92108 |             float32x4_t rightf;
92109 |             mid = vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), wbps0_4);
92110 |             side = vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), wbps1_4);
92111 |             mid = vorrq_u32(vshlq_n_u32(mid, 1), vandq_u32(side, vdupq_n_u32(1)));
92112 |             lefti = vreinterpretq_s32_u32(vshlq_u32(vaddq_u32(mid, side), shift4));
92113 |             righti = vreinterpretq_s32_u32(vshlq_u32(vsubq_u32(mid, side), shift4));
92114 |             leftf = vmulq_f32(vcvttq_f32_s32(lefti), factor4);
92115 |             rightf = vmulq_f32(vcvttq_f32_s32(righti), factor4);
92116 |             ma_dr_flac__vst2q_f32(pOutputSamples + i*8, vzipq_f32(leftf, rightf));
92117 |         }
92118 |         for (i = (frameCount4 << 2); i < frameCount; ++i) {
92119 |             ma_uint32 mid = pInputSamples0U32[i] <<
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
92120 |             ma_uint32 side = pInputSamples1U32[i] <<
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
92121 |             mid = (mid << 1) | (side & 0x01);
92122 |             pOutputSamples[i*2+0] = (ma_int32)((mid + side) << shift) * factor;
92123 |             pOutputSamples[i*2+1] = (ma_int32)((mid - side) << shift) * factor;
92124 |         }
92125 |     }
92126 | }
92127 | #endif
92128 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_mid_side(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
92129 | {

```

```

92130 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
92131 |     if (ma_dr_flac_gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
92132 |         ma_dr_flac_read_pcm_frames_f32__decode_mid_side__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92133 |     } else
92134 | #elif defined(MA_DR_FLAC_SUPPORT_NEON)
92135 |     if (ma_dr_flac_gIsNEONSupported && pFlac->bitsPerSample <= 24) {
92136 |         ma_dr_flac_read_pcm_frames_f32__decode_mid_side__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92137 |     } else
92138 | #endif
92139 |     {
92140 |     #if 0
92141 |         ma_dr_flac_read_pcm_frames_f32__decode_mid_side__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92142 |     #else
92143 |         ma_dr_flac_read_pcm_frames_f32__decode_mid_side__scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92144 |     #endif
92145 |     }
92146 | }
92147 | #if 0
92148 | static MA_INLINE void
> ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__reference(ma_dr_flac* pFlac, ma_uint64
> frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
92149 | {
92150 |     for (ma_uint64 i = 0; i < frameCount; ++i) {
92151 |         pOutputSamples[i*2+0] = (float)((ma_int32)((ma_uint32)pInputSamples0[i] <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample)) / 2147483648.0);
92152 |         pOutputSamples[i*2+1] = (float)((ma_int32)((ma_uint32)pInputSamples1[i] <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample)) / 2147483648.0);
92153 |     }
92154 | }
92155 | #endif
92156 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__scalar(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, float* pOutputSamples)
92157 | {
92158 |     ma_uint64 i;
92159 |     ma_uint64 frameCount4 = frameCount >> 2;
92160 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
92161 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
92162 |     ma_uint32 shift0 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample;
92163 |     ma_uint32 shift1 = unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample;
92164 |     float factor = 1 / 2147483648.0;
92165 |     for (i = 0; i < frameCount4; ++i) {
92166 |         ma_uint32 tempL0 = pInputSamples0U32[i*4+0] << shift0;
92167 |         ma_uint32 tempL1 = pInputSamples0U32[i*4+1] << shift0;
92168 |         ma_uint32 tempL2 = pInputSamples0U32[i*4+2] << shift0;
92169 |         ma_uint32 tempL3 = pInputSamples0U32[i*4+3] << shift0;
92170 |         ma_uint32 tempR0 = pInputSamples1U32[i*4+0] << shift1;
92171 |         ma_uint32 tempR1 = pInputSamples1U32[i*4+1] << shift1;
92172 |         ma_uint32 tempR2 = pInputSamples1U32[i*4+2] << shift1;
92173 |         ma_uint32 tempR3 = pInputSamples1U32[i*4+3] << shift1;
92174 |         pOutputSamples[i*8+0] = (ma_int32)tempL0 * factor;
92175 |         pOutputSamples[i*8+1] = (ma_int32)tempR0 * factor;
92176 |         pOutputSamples[i*8+2] = (ma_int32)tempL1 * factor;
92177 |         pOutputSamples[i*8+3] = (ma_int32)tempR1 * factor;
92178 |         pOutputSamples[i*8+4] = (ma_int32)tempL2 * factor;
92179 |         pOutputSamples[i*8+5] = (ma_int32)tempR2 * factor;
92180 |         pOutputSamples[i*8+6] = (ma_int32)tempL3 * factor;
92181 |         pOutputSamples[i*8+7] = (ma_int32)tempR3 * factor;
92182 |     }
92183 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
92184 |         pOutputSamples[i*2+0] = (ma_int32)(pInputSamples0U32[i] << shift0) * factor;
92185 |         pOutputSamples[i*2+1] = (ma_int32)(pInputSamples1U32[i] << shift1) * factor;
92186 |     }
92187 | }
92188 | #if defined(MA_DR_FLAC_SUPPORT_SSE2)
92189 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__sse2(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, float* pOutputSamples)
92190 | {
92191 |     ma_uint64 i;
92192 |     ma_uint64 frameCount4 = frameCount >> 2;
92193 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;

```

```

92194 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
92195 |     ma_uint32 shift0 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample) - 8;
92196 |     ma_uint32 shift1 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample) - 8;
92197 |     float factor = 1.0f / 8388608.0f;
92198 |     __m128 factor128 = _mm_set1_ps(factor);
92199 |     for (i = 0; i < frameCount4; ++i) {
92200 |         __m128i lefti;
92201 |         __m128i righti;
92202 |         __m128 leftf;
92203 |         __m128 rightf;
92204 |         lefti = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples0 + i), shift0);
92205 |         righti = _mm_slli_epi32(_mm_loadu_si128((const __m128i*)pInputSamples1 + i), shift1);
92206 |         leftf = _mm_mul_ps(_mm_cvtepi32_ps(lefti), factor128);
92207 |         rightf = _mm_mul_ps(_mm_cvtepi32_ps(righti), factor128);
92208 |         _mm_storeu_ps(pOutputSamples + i*8 + 0, _mm_unpacklo_ps(leftf, rightf));
92209 |         _mm_storeu_ps(pOutputSamples + i*8 + 4, _mm_unpackhi_ps(leftf, rightf));
92210 |     }
92211 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
92212 |         pOutputSamples[i*2+0] = (ma_int32)(pInputSamples0U32[i] << shift0) * factor;
92213 |         pOutputSamples[i*2+1] = (ma_int32)(pInputSamples1U32[i] << shift1) * factor;
92214 |     }
92215 | }
92216 | #endif
92217 | #if defined(MA_DR_FLAC_SUPPORT_NEON)
92218 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__neon(ma_dr_flac*
> pFlac, ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const
> ma_int32* pInputSamples1, float* pOutputSamples)
92219 | {
92220 |     ma_uint64 i;
92221 |     ma_uint64 frameCount4 = frameCount >> 2;
92222 |     const ma_uint32* pInputSamples0U32 = (const ma_uint32*)pInputSamples0;
92223 |     const ma_uint32* pInputSamples1U32 = (const ma_uint32*)pInputSamples1;
92224 |     ma_uint32 shift0 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[0].wastedBitsPerSample) - 8;
92225 |     ma_uint32 shift1 = (unusedBitsPerSample +
> pFlac->currentFLACFrame.subframes[1].wastedBitsPerSample) - 8;
92226 |     float factor = 1.0f / 8388608.0f;
92227 |     float32x4_t factor4 = vdupq_n_f32(factor);
92228 |     int32x4_t shift0_4 = vdupq_n_s32(shift0);
92229 |     int32x4_t shift1_4 = vdupq_n_s32(shift1);
92230 |     for (i = 0; i < frameCount4; ++i) {
92231 |         int32x4_t lefti;
92232 |         int32x4_t righti;
92233 |         float32x4_t leftf;
92234 |         float32x4_t rightf;
92235 |         lefti = vreinterpretq_s32_u32(vshlq_u32(vld1q_u32(pInputSamples0U32 + i*4), shift0_4));
92236 |         righti = vreinterpretq_s32_u32(vshlq_u32(vld1q_u32(pInputSamples1U32 + i*4), shift1_4));
92237 |         leftf = vmulq_f32(vcvtq_f32_s32(lefti), factor4);
92238 |         rightf = vmulq_f32(vcvtq_f32_s32(righti), factor4);
92239 |         ma_dr_flac__vst2q_f32(pOutputSamples + i*8, vzipq_f32(leftf, rightf));
92240 |     }
92241 |     for (i = (frameCount4 << 2); i < frameCount; ++i) {
92242 |         pOutputSamples[i*2+0] = (ma_int32)(pInputSamples0U32[i] << shift0) * factor;
92243 |         pOutputSamples[i*2+1] = (ma_int32)(pInputSamples1U32[i] << shift1) * factor;
92244 |     }
92245 | }
92246 | #endif
92247 | static MA_INLINE void ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo(ma_dr_flac* pFlac,
> ma_uint64 frameCount, ma_uint32 unusedBitsPerSample, const ma_int32* pInputSamples0, const ma_int32*
> pInputSamples1, float* pOutputSamples)
92248 | {
92249 |     #if defined(MA_DR_FLAC_SUPPORT_SSE2)
92250 |         if (ma_dr_flac__gIsSSE2Supported && pFlac->bitsPerSample <= 24) {
92251 |             ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__sse2(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92252 |         } else
92253 |         #elif defined(MA_DR_FLAC_SUPPORT_NEON)
92254 |             if (ma_dr_flac__gIsNEONSupported && pFlac->bitsPerSample <= 24) {
92255 |                 ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__neon(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92256 |             } else
92257 |             #endif
92258 |             {
92259 |                 #if 0
92260 |                 ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo__reference(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92261 |                 #else

```



```

92262 |         ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo_scalar(pFlac, frameCount,
> unusedBitsPerSample, pInputSamples0, pInputSamples1, pOutputSamples);
92263 | #endif
92264 |     }
92265 | }
92266 | MA_API ma_uint64 ma_dr_flac_read_pcm_frames_f32(ma_dr_flac* pFlac, ma_uint64 framesToRead, float*
> pBufferOut)
92267 | {
92268 |     ma_uint64 framesRead;
92269 |     ma_uint32 unusedBitsPerSample;
92270 |     if (pFlac == NULL || framesToRead == 0) {
92271 |         return 0;
92272 |     }
92273 |     if (pBufferOut == NULL) {
92274 |         return ma_dr_flac__seek_forward_by_pcm_frames(pFlac, framesToRead);
92275 |     }
92276 |     MA_DR_FLAC_ASSERT(pFlac->bitsPerSample <= 32);
92277 |     unusedBitsPerSample = 32 - pFlac->bitsPerSample;
92278 |     framesRead = 0;
92279 |     while (framesToRead > 0) {
92280 |         if (pFlac->currentFLACFrame.pcmFramesRemaining == 0) {
92281 |             if (!ma_dr_flac__read_and_decode_next_flac_frame(pFlac)) {
92282 |                 break;
92283 |             }
92284 |         } else {
92285 |             unsigned int channelCount = ma_dr_flac__get_channel_count_from_channel_assignment(pFlac-
> >currentFLACFrame.header.channelAssignment);
92286 |             ma_uint64 iFirstPCMFrame = pFlac->currentFLACFrame.header.blockSizeInPCMFrames -
> pFlac->currentFLACFrame.pcmFramesRemaining;
92287 |             ma_uint64 frameCountThisIteration = framesToRead;
92288 |             if (frameCountThisIteration > pFlac->currentFLACFrame.pcmFramesRemaining) {
92289 |                 frameCountThisIteration = pFlac->currentFLACFrame.pcmFramesRemaining;
92290 |             }
92291 |             if (channelCount == 2) {
92292 |                 const ma_int32* pDecodedSamples0 = pFlac->currentFLACFrame.subframes[0].pSamplesS32
> + iFirstPCMFrame;
92293 |                 const ma_int32* pDecodedSamples1 = pFlac->currentFLACFrame.subframes[1].pSamplesS32
> + iFirstPCMFrame;
92294 |                 switch (pFlac->currentFLACFrame.header.channelAssignment)
92295 |                 {
92296 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_LEFT_SIDE:
92297 |                     {
92298 |                         ma_dr_flac_read_pcm_frames_f32__decode_left_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
92299 |                     } break;
92300 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_RIGHT_SIDE:
92301 |                     {
92302 |                         ma_dr_flac_read_pcm_frames_f32__decode_right_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
92303 |                     } break;
92304 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_MID_SIDE:
92305 |                     {
92306 |                         ma_dr_flac_read_pcm_frames_f32__decode_mid_side(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
92307 |                     } break;
92308 |                     case MA_DR_FLAC_CHANNEL_ASSIGNMENT_INDEPENDENT:
92309 |                     default:
92310 |                     {
92311 |                         ma_dr_flac_read_pcm_frames_f32__decode_independent_stereo(pFlac,
> frameCountThisIteration, unusedBitsPerSample, pDecodedSamples0, pDecodedSamples1, pBufferOut);
92312 |                     } break;
92313 |                 }
92314 |             } else {
92315 |                 ma_uint64 i;
92316 |                 for (i = 0; i < frameCountThisIteration; ++i) {
92317 |                     unsigned int j;
92318 |                     for (j = 0; j < channelCount; ++j) {
92319 |                         ma_int32 samplesS32 =
> (ma_int32)((ma_uint32)(pFlac->currentFLACFrame.subframes[j].pSamplesS32[iFirstPCMFrame + i]) <<
> (unusedBitsPerSample + pFlac->currentFLACFrame.subframes[j].wastedBitsPerSample));
92320 |                         pBufferOut[(i*channelCount)+j] = (float)(samplesS32 / 2147483648.0);
92321 |                     }
92322 |                 }
92323 |             }
92324 |             framesRead += frameCountThisIteration;
92325 |             pBufferOut += frameCountThisIteration * channelCount;
92326 |             framesToRead -= frameCountThisIteration;
92327 |             pFlac->currentPCMFrame += frameCountThisIteration;
92328 |             pFlac->currentFLACFrame.pcmFramesRemaining -= (unsigned int)frameCountThisIteration;

```



```

92329 |     }
92330 | }
92331 |     return framesRead;
92332 | }
92333 | MA_API ma_bool32 ma_dr_flac_seek_to_pcm_frame(ma_dr_flac* pFlac, ma_uint64 pcmFrameIndex)
92334 | {
92335 |     if (pFlac == NULL) {
92336 |         return MA_FALSE;
92337 |     }
92338 |     if (pFlac->currentPCMFrame == pcmFrameIndex) {
92339 |         return MA_TRUE;
92340 |     }
92341 |     if (pFlac->firstFLACFramePosInBytes == 0) {
92342 |         return MA_FALSE;
92343 |     }
92344 |     if (pcmFrameIndex == 0) {
92345 |         pFlac->currentPCMFrame = 0;
92346 |         return ma_dr_flac__seek_to_first_frame(pFlac);
92347 |     } else {
92348 |         ma_bool32 wasSuccessful = MA_FALSE;
92349 |         ma_uint64 originalPCMFrame = pFlac->currentPCMFrame;
92350 |         if (pcmFrameIndex > pFlac->totalPCMFrameCount) {
92351 |             pcmFrameIndex = pFlac->totalPCMFrameCount;
92352 |         }
92353 |         if (pcmFrameIndex > pFlac->currentPCMFrame) {
92354 |             ma_uint32 offset = (ma_uint32)(pcmFrameIndex - pFlac->currentPCMFrame);
92355 |             if (pFlac->currentFLACFrame.pcmFramesRemaining > offset) {
92356 |                 pFlac->currentFLACFrame.pcmFramesRemaining -= offset;
92357 |                 pFlac->currentPCMFrame = pcmFrameIndex;
92358 |                 return MA_TRUE;
92359 |             }
92360 |         } else {
92361 |             ma_uint32 offsetAbs = (ma_uint32)(pFlac->currentPCMFrame - pcmFrameIndex);
92362 |             ma_uint32 currentFLACFramePCMFrameCount =
92363 | > pFlac->currentFLACFrame.header.blockSizeInPCMFrames;
92364 |             ma_uint32 currentFLACFramePCMFramesConsumed = currentFLACFramePCMFrameCount -
92365 | > pFlac->currentFLACFrame.pcmFramesRemaining;
92366 |             if (currentFLACFramePCMFramesConsumed > offsetAbs) {
92367 |                 pFlac->currentFLACFrame.pcmFramesRemaining += offsetAbs;
92368 |                 pFlac->currentPCMFrame = pcmFrameIndex;
92369 |                 return MA_TRUE;
92370 |             }
92371 |         }
92372 |         #ifndef MA_DR_FLAC_NO_OGG
92373 |         if (pFlac->container == ma_dr_flac_container_ogg)
92374 |         {
92375 |             wasSuccessful = ma_dr_flac_ogg__seek_to_pcm_frame(pFlac, pcmFrameIndex);
92376 |         }
92377 |         #endif
92378 |         {
92379 |             if (!pFlac->noSeekTableSeek) {
92380 |                 wasSuccessful = ma_dr_flac__seek_to_pcm_frame__seek_table(pFlac, pcmFrameIndex);
92381 |             }
92382 |             #if !defined(MA_DR_FLAC_NO_CRC)
92383 |             if (!wasSuccessful && !pFlac->noBinarySearchSeek && pFlac->totalPCMFrameCount > 0) {
92384 |                 wasSuccessful = ma_dr_flac__seek_to_pcm_frame__binary_search(pFlac, pcmFrameIndex);
92385 |             }
92386 |             #endif
92387 |             if (!wasSuccessful && !pFlac->noBruteForceSeek) {
92388 |                 wasSuccessful = ma_dr_flac__seek_to_pcm_frame__brute_force(pFlac, pcmFrameIndex);
92389 |             }
92390 |             if (wasSuccessful) {
92391 |                 pFlac->currentPCMFrame = pcmFrameIndex;
92392 |             } else {
92393 |                 if (ma_dr_flac_seek_to_pcm_frame(pFlac, originalPCMFrame) == MA_FALSE) {
92394 |                     ma_dr_flac_seek_to_pcm_frame(pFlac, 0);
92395 |                 }
92396 |             }
92397 |             return wasSuccessful;
92398 |         }
92399 |     }
92400 | }
92401 | #define MA_DR_FLAC_DEFINE_FULL_READ_AND_CLOSE(extension, type) \
92402 | static type* ma_dr_flac__full_read_and_close_ ## extension (ma_dr_flac* pFlac, unsigned int* \
92403 | > channelsOut, unsigned int* sampleRateOut, ma_uint64* totalPCMFrameCountOut)\
92404 | {
92405 |     \
92406 |     type* pSampleData = NULL;

```

```

>
92404 |     ma_uint64 totalPCMFrameCount;
>
92405 |     type buffer[4096];
>
92406 |     ma_uint64 pcmFramesRead;
>
92407 |     size_t sampleDataBufferSize = sizeof(buffer);
>
92408 |
>
92409 |     MA_DR_FLAC_ASSERT(pFlac != NULL);
>
92410 |
>
92411 |     totalPCMFrameCount = 0;
>
92412 |
>
92413 |     pSampleData = (type*)ma_dr_flac__malloc_from_callbacks(sampleDataBufferSize,
> &pFlac->allocationCallbacks);
92414 |     if (pSampleData == NULL) {
>
92415 |         goto on_error;
>
92416 |     }
>
92417 |
>
92418 |     while ((pcmFramesRead = (ma_uint64)ma_dr_flac_read_pcm_frames_##extension(pFlac,
> sizeof(buffer)/sizeof(buffer[0])/pFlac->channels, buffer)) > 0) {
92419 |         if (((totalPCMFrameCount + pcmFramesRead) * pFlac->channels * sizeof(type)) >
> sampleDataBufferSize) {
92420 |             type* pNewSampleData;
>
92421 |             size_t newSampleDataBufferSize;
>
92422 |
>
92423 |             newSampleDataBufferSize = sampleDataBufferSize * 2;
>
92424 |             pNewSampleData = (type*)ma_dr_flac__realloc_from_callbacks(pSampleData,
> newSampleDataBufferSize, sampleDataBufferSize, &pFlac->allocationCallbacks);
92425 |             if (pNewSampleData == NULL) {
>
92426 |                 ma_dr_flac__free_from_callbacks(pSampleData, &pFlac->allocationCallbacks);
>
92427 |                 goto on_error;
>
92428 |             }
>
92429 |
>
92430 |             sampleDataBufferSize = newSampleDataBufferSize;
>
92431 |             pSampleData = pNewSampleData;
>
92432 |         }
>
92433 |
>
92434 |         MA_DR_FLAC_COPY_MEMORY(pSampleData + (totalPCMFrameCount*pFlac->channels), buffer,
> (size_t)(pcmFramesRead*pFlac->channels*sizeof(type)));
92435 |         totalPCMFrameCount += pcmFramesRead;
>
92436 |     }
>
92437 |
>
92438 |
>
92439 |     MA_DR_FLAC_ZERO_MEMORY(pSampleData + (totalPCMFrameCount*pFlac->channels),
> (size_t)(sampleDataBufferSize - totalPCMFrameCount*pFlac->channels*sizeof(type)));
92440 |
>
92441 |     if (sampleRateOut) *sampleRateOut = pFlac->sampleRate;
>
92442 |     if (channelsOut) *channelsOut = pFlac->channels;
>

```

```

92443 |     if (totalPCMFrameCountOut) *totalPCMFrameCountOut = totalPCMFrameCount;
92444 | >
92445 |     ma_dr_flac_close(pFlac);
92446 |     return pSampleData;
92447 | >
92448 | on_error:
92449 |     ma_dr_flac_close(pFlac);
92450 |     return NULL;
92451 | }
92452 | MA_DR_FLAC_DEFINE_FULL_READ_AND_CLOSE(s32, ma_int32)
92453 | MA_DR_FLAC_DEFINE_FULL_READ_AND_CLOSE(s16, ma_int16)
92454 | MA_DR_FLAC_DEFINE_FULL_READ_AND_CLOSE(f32, float)
92455 | MA_API ma_int32* ma_dr_flac_open_and_read_pcm_frames_s32(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, void* pUserData, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalPCMFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
92456 | {
92457 |     ma_dr_flac* pFlac;
92458 |     if (channelsOut) {
92459 |         *channelsOut = 0;
92460 |     }
92461 |     if (sampleRateOut) {
92462 |         *sampleRateOut = 0;
92463 |     }
92464 |     if (totalPCMFrameCountOut) {
92465 |         *totalPCMFrameCountOut = 0;
92466 |     }
92467 |     pFlac = ma_dr_flac_open(onRead, onSeek, onTell, pUserData, pAllocationCallbacks);
92468 |     if (pFlac == NULL) {
92469 |         return NULL;
92470 |     }
92471 |     return ma_dr_flac__full_read_and_close_s32(pFlac, channelsOut, sampleRateOut,
> totalPCMFrameCountOut);
92472 | }
92473 | MA_API ma_int16* ma_dr_flac_open_and_read_pcm_frames_s16(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, void* pUserData, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalPCMFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
92474 | {
92475 |     ma_dr_flac* pFlac;
92476 |     if (channelsOut) {
92477 |         *channelsOut = 0;
92478 |     }
92479 |     if (sampleRateOut) {
92480 |         *sampleRateOut = 0;
92481 |     }
92482 |     if (totalPCMFrameCountOut) {
92483 |         *totalPCMFrameCountOut = 0;
92484 |     }
92485 |     pFlac = ma_dr_flac_open(onRead, onSeek, onTell, pUserData, pAllocationCallbacks);
92486 |     if (pFlac == NULL) {
92487 |         return NULL;
92488 |     }
92489 |     return ma_dr_flac__full_read_and_close_s16(pFlac, channelsOut, sampleRateOut,
> totalPCMFrameCountOut);
92490 | }
92491 | MA_API float* ma_dr_flac_open_and_read_pcm_frames_f32(ma_dr_flac_read_proc onRead,
> ma_dr_flac_seek_proc onSeek, ma_dr_flac_tell_proc onTell, void* pUserData, unsigned int*
> channelsOut, unsigned int* sampleRateOut, ma_uint64* totalPCMFrameCountOut, const
> ma_allocation_callbacks* pAllocationCallbacks)
92492 | {
92493 |     ma_dr_flac* pFlac;
92494 |     if (channelsOut) {
92495 |         *channelsOut = 0;
92496 |     }
92497 |     if (sampleRateOut) {
92498 |         *sampleRateOut = 0;
92499 |     }
92500 |     if (totalPCMFrameCountOut) {
92501 |         *totalPCMFrameCountOut = 0;
92502 |     }

```

```

92503 |     pFlac = ma_dr_flac_open(onRead, onSeek, onTell, pUserData, pAllocationCallbacks);
92504 |     if (pFlac == NULL) {
92505 |         return NULL;
92506 |     }
92507 |     return ma_dr_flac__full_read_and_close_f32(pFlac, channelsOut, sampleRateOut,
> totalPCMFrameCountOut);
92508 | }
92509 | #ifndef MA_DR_FLAC_NO_STDIO
92510 | MA_API ma_int32* ma_dr_flac_open_file_and_read_pcm_frames_s32(const char* filename, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalPCMFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
92511 | {
92512 |     ma_dr_flac* pFlac;
92513 |     if (sampleRate) {
92514 |         *sampleRate = 0;
92515 |     }
92516 |     if (channels) {
92517 |         *channels = 0;
92518 |     }
92519 |     if (totalPCMFrameCount) {
92520 |         *totalPCMFrameCount = 0;
92521 |     }
92522 |     pFlac = ma_dr_flac_open_file(filename, pAllocationCallbacks);
92523 |     if (pFlac == NULL) {
92524 |         return NULL;
92525 |     }
92526 |     return ma_dr_flac__full_read_and_close_s32(pFlac, channels, sampleRate, totalPCMFrameCount);
92527 | }
92528 | MA_API ma_int16* ma_dr_flac_open_file_and_read_pcm_frames_s16(const char* filename, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalPCMFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
92529 | {
92530 |     ma_dr_flac* pFlac;
92531 |     if (sampleRate) {
92532 |         *sampleRate = 0;
92533 |     }
92534 |     if (channels) {
92535 |         *channels = 0;
92536 |     }
92537 |     if (totalPCMFrameCount) {
92538 |         *totalPCMFrameCount = 0;
92539 |     }
92540 |     pFlac = ma_dr_flac_open_file(filename, pAllocationCallbacks);
92541 |     if (pFlac == NULL) {
92542 |         return NULL;
92543 |     }
92544 |     return ma_dr_flac__full_read_and_close_s16(pFlac, channels, sampleRate, totalPCMFrameCount);
92545 | }
92546 | MA_API float* ma_dr_flac_open_file_and_read_pcm_frames_f32(const char* filename, unsigned int*
> channels, unsigned int* sampleRate, ma_uint64* totalPCMFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
92547 | {
92548 |     ma_dr_flac* pFlac;
92549 |     if (sampleRate) {
92550 |         *sampleRate = 0;
92551 |     }
92552 |     if (channels) {
92553 |         *channels = 0;
92554 |     }
92555 |     if (totalPCMFrameCount) {
92556 |         *totalPCMFrameCount = 0;
92557 |     }
92558 |     pFlac = ma_dr_flac_open_file(filename, pAllocationCallbacks);
92559 |     if (pFlac == NULL) {
92560 |         return NULL;
92561 |     }
92562 |     return ma_dr_flac__full_read_and_close_f32(pFlac, channels, sampleRate, totalPCMFrameCount);
92563 | }
92564 | #endif
92565 | MA_API ma_int32* ma_dr_flac_open_memory_and_read_pcm_frames_s32(const void* data, size_t dataSize,
> unsigned int* channels, unsigned int* sampleRate, ma_uint64* totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
92566 | {
92567 |     ma_dr_flac* pFlac;
92568 |     if (sampleRate) {
92569 |         *sampleRate = 0;
92570 |     }
92571 |     if (channels) {
92572 |         *channels = 0;

```

```

92573 |     }
92574 |     if (totalPCMFrameCount) {
92575 |         *totalPCMFrameCount = 0;
92576 |     }
92577 |     pFlac = ma_dr_flac_open_memory(data, dataSize, pAllocationCallbacks);
92578 |     if (pFlac == NULL) {
92579 |         return NULL;
92580 |     }
92581 |     return ma_dr_flac__full_read_and_close_s32(pFlac, channels, sampleRate, totalPCMFrameCount);
92582 | }
92583 | MA_API ma_int16* ma_dr_flac_open_memory_and_read_pcm_frames_s16(const void* data, size_t dataSize,
> unsigned int* channels, unsigned int* sampleRate, ma_uint64* totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
92584 | {
92585 |     ma_dr_flac* pFlac;
92586 |     if (sampleRate) {
92587 |         *sampleRate = 0;
92588 |     }
92589 |     if (channels) {
92590 |         *channels = 0;
92591 |     }
92592 |     if (totalPCMFrameCount) {
92593 |         *totalPCMFrameCount = 0;
92594 |     }
92595 |     pFlac = ma_dr_flac_open_memory(data, dataSize, pAllocationCallbacks);
92596 |     if (pFlac == NULL) {
92597 |         return NULL;
92598 |     }
92599 |     return ma_dr_flac__full_read_and_close_s16(pFlac, channels, sampleRate, totalPCMFrameCount);
92600 | }
92601 | MA_API float* ma_dr_flac_open_memory_and_read_pcm_frames_f32(const void* data, size_t dataSize,
> unsigned int* channels, unsigned int* sampleRate, ma_uint64* totalPCMFrameCount, const
> ma_allocation_callbacks* pAllocationCallbacks)
92602 | {
92603 |     ma_dr_flac* pFlac;
92604 |     if (sampleRate) {
92605 |         *sampleRate = 0;
92606 |     }
92607 |     if (channels) {
92608 |         *channels = 0;
92609 |     }
92610 |     if (totalPCMFrameCount) {
92611 |         *totalPCMFrameCount = 0;
92612 |     }
92613 |     pFlac = ma_dr_flac_open_memory(data, dataSize, pAllocationCallbacks);
92614 |     if (pFlac == NULL) {
92615 |         return NULL;
92616 |     }
92617 |     return ma_dr_flac__full_read_and_close_f32(pFlac, channels, sampleRate, totalPCMFrameCount);
92618 | }
92619 | MA_API void ma_dr_flac_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks)
92620 | {
92621 |     if (pAllocationCallbacks != NULL) {
92622 |         ma_dr_flac__free_from_callbacks(p, pAllocationCallbacks);
92623 |     } else {
92624 |         ma_dr_flac__free_default(p, NULL);
92625 |     }
92626 | }
92627 | MA_API void ma_dr_flac_init_vorbis_comment_iterator(ma_dr_flac_vorbis_comment_iterator* pIter,
> ma_uint32 commentCount, const void* pComments)
92628 | {
92629 |     if (pIter == NULL) {
92630 |         return;
92631 |     }
92632 |     pIter->countRemaining = commentCount;
92633 |     pIter->pRunningData = (const char*)pComments;
92634 | }
92635 | MA_API const char* ma_dr_flac_next_vorbis_comment(ma_dr_flac_vorbis_comment_iterator* pIter,
> ma_uint32* pCommentLengthOut)
92636 | {
92637 |     ma_int32 length;
92638 |     const char* pComment;
92639 |     if (pCommentLengthOut) {
92640 |         *pCommentLengthOut = 0;
92641 |     }
92642 |     if (pIter == NULL || pIter->countRemaining == 0 || pIter->pRunningData == NULL) {
92643 |         return NULL;
92644 |     }
92645 |     length = ma_dr_flac__le2host_32_ptr_unaligned(pIter->pRunningData);

```

```

92646 |     pIter->pRunningData += 4;
92647 |     pComment = pIter->pRunningData;
92648 |     pIter->pRunningData += length;
92649 |     pIter->countRemaining -= 1;
92650 |     if (pCommentLengthOut) {
92651 |         *pCommentLengthOut = length;
92652 |     }
92653 |     return pComment;
92654 | }
92655 | MA_API void ma_dr_flac_init_cuesheet_track_iterator(ma_dr_flac_cuesheet_track_iterator* pIter,
> ma_uint32 trackCount, const void* pTrackData)
92656 | {
92657 |     if (pIter == NULL) {
92658 |         return;
92659 |     }
92660 |     pIter->countRemaining = trackCount;
92661 |     pIter->pRunningData = (const char*)pTrackData;
92662 | }
92663 | MA_API ma_bool32 ma_dr_flac_next_cuesheet_track(ma_dr_flac_cuesheet_track_iterator* pIter,
> ma_dr_flac_cuesheet_track* pCuesheetTrack)
92664 | {
92665 |     ma_dr_flac_cuesheet_track cuesheetTrack;
92666 |     const char* pRunningData;
92667 |     ma_uint64 offsetHi;
92668 |     ma_uint64 offsetLo;
92669 |     if (pIter == NULL || pIter->countRemaining == 0 || pIter->pRunningData == NULL) {
92670 |         return MA_FALSE;
92671 |     }
92672 |     pRunningData = pIter->pRunningData;
92673 |     offsetHi = ma_dr_flac_be2host_32(*(const ma_uint32*)pRunningData);
> pRunningData += 4;
92674 |     offsetLo = ma_dr_flac_be2host_32(*(const ma_uint32*)pRunningData);
> pRunningData += 4;
92675 |     cuesheetTrack.offset = offsetLo | (offsetHi << 32);
92676 |     cuesheetTrack.trackNumber = pRunningData[0];
> pRunningData += 1;
92677 |     MA_DR_FLAC_COPY_MEMORY(cuesheetTrack.ISRC, pRunningData, sizeof(cuesheetTrack.ISRC));
> pRunningData += 12;
92678 |     cuesheetTrack.isAudio = (pRunningData[0] & 0x80) != 0;
92679 |     cuesheetTrack.preEmphasis = (pRunningData[0] & 0x40) != 0;
> pRunningData += 14;
92680 |     cuesheetTrack.indexCount = pRunningData[0];
> pRunningData += 1;
92681 |     cuesheetTrack.pIndexPoints = (const ma_dr_flac_cuesheet_track_index*)pRunningData;
> pRunningData += cuesheetTrack.indexCount * sizeof(ma_dr_flac_cuesheet_track_index);
92682 |     pIter->pRunningData = pRunningData;
92683 |     pIter->countRemaining -= 1;
92684 |     if (pCuesheetTrack) {
92685 |         *pCuesheetTrack = cuesheetTrack;
92686 |     }
92687 |     return MA_TRUE;
92688 | }
92689 | #if defined(__clang__) || (defined(__GNUC__) && (__GNUC__ > 4 || (__GNUC__ == 4 && __GNUC_MINOR__ >=
> 6)))
92690 |     #pragma GCC diagnostic pop
92691 | #endif
92692 | #endif
92693 | /* dr_flac_c end */
92694 | #endif /* MA_DR_FLAC_IMPLEMENTATION */
92695 | #endif /* MA_NO_FLAC */
92696 |
92697 | #if !defined(MA_NO_MP3) && !defined(MA_NO_DECODING)
92698 | #if !defined(MA_DR_MP3_IMPLEMENTATION)
92699 |     /* dr_mp3_c begin */
92700 |     #ifndef ma_dr_mp3_c
92701 |     #define ma_dr_mp3_c
92702 |     #include <stdlib.h>
92703 |     #include <string.h>
92704 |     #include <limits.h>
92705 |     MA_API void ma_dr_mp3_version(ma_uint32* pMajor, ma_uint32* pMinor, ma_uint32* pRevision)
92706 |     {
92707 |         if (pMajor) {
92708 |             *pMajor = MA_DR_MP3_VERSION_MAJOR;
92709 |         }
92710 |         if (pMinor) {
92711 |             *pMinor = MA_DR_MP3_VERSION_MINOR;
92712 |         }
92713 |         if (pRevision) {
92714 |             *pRevision = MA_DR_MP3_VERSION_REVISION;

```

```

92715 |     }
92716 | }
92717 | MA_API const char* ma_dr_mp3_version_string(void)
92718 | {
92719 |     return MA_DR_MP3_VERSION_STRING;
92720 | }
92721 | #if defined(__TINYC__)
92722 | #define MA_DR_MP3_NO_SIMD
92723 | #endif
92724 | #define MA_DR_MP3_OFFSET_PTR(p, offset) ((void*)((ma_uint8*)(p) + (offset)))
92725 | #ifndef MA_DR_MP3_MAX_FRAME_SYNC_MATCHES
92726 | #define MA_DR_MP3_MAX_FRAME_SYNC_MATCHES 10
92727 | #endif
92728 | #define MA_DR_MP3_SHORT_BLOCK_TYPE 2
92729 | #define MA_DR_MP3_STOP_BLOCK_TYPE 3
92730 | #define MA_DR_MP3_MODE_MONO 3
92731 | #define MA_DR_MP3_MODE_JOINT_STEREO 1
92732 | #define MA_DR_MP3_HDR_SIZE 4
92733 | #define MA_DR_MP3_HDR_IS_MONO(h) (((h[3]) & 0xC0) == 0xC0)
92734 | #define MA_DR_MP3_HDR_IS_MS_STEREO(h) (((h[3]) & 0xE0) == 0xE0)
92735 | #define MA_DR_MP3_HDR_IS_FREE_FORMAT(h) (((h[2]) & 0xF0) == 0)
92736 | #define MA_DR_MP3_HDR_IS_CRC(h) (!(h[1] & 1))
92737 | #define MA_DR_MP3_HDR_TEST_PADDING(h) ((h[2]) & 0x2)
92738 | #define MA_DR_MP3_HDR_TEST_MPEG1(h) ((h[1]) & 0x8)
92739 | #define MA_DR_MP3_HDR_TEST_NOT_MPEG25(h) ((h[1]) & 0x10)
92740 | #define MA_DR_MP3_HDR_TEST_I_STEREO(h) ((h[3]) & 0x10)
92741 | #define MA_DR_MP3_HDR_TEST_MS_STEREO(h) ((h[3]) & 0x20)
92742 | #define MA_DR_MP3_HDR_GET_STEREO_MODE(h) (((h[3]) >> 6) & 3)
92743 | #define MA_DR_MP3_HDR_GET_STEREO_MODE_EXT(h) (((h[3]) >> 4) & 3)
92744 | #define MA_DR_MP3_HDR_GET_LAYER(h) (((h[1]) >> 1) & 3)
92745 | #define MA_DR_MP3_HDR_GET_BITRATE(h) ((h[2]) >> 4)
92746 | #define MA_DR_MP3_HDR_GET_SAMPLE_RATE(h) (((h[2]) >> 2) & 3)
92747 | #define MA_DR_MP3_HDR_GET_MY_SAMPLE_RATE(h) (MA_DR_MP3_HDR_GET_SAMPLE_RATE(h) + (((h[1] >> 3) & 1)
> + ((h[1] >> 4) & 1))*3)
92748 | #define MA_DR_MP3_HDR_IS_FRAME_576(h) ((h[1] & 14) == 2)
92749 | #define MA_DR_MP3_HDR_IS_LAYER_1(h) ((h[1] & 6) == 6)
92750 | #define MA_DR_MP3_BITS_DEQUANTIZER_OUT -1
92751 | #define MA_DR_MP3_MAX_SCF (255 + MA_DR_MP3_BITS_DEQUANTIZER_OUT*4 - 210)
92752 | #define MA_DR_MP3_MAX_SCFI ((MA_DR_MP3_MAX_SCF + 3) & ~3)
92753 | #define MA_DR_MP3_MIN(a, b) ((a) > (b) ? (b) : (a))
92754 | #define MA_DR_MP3_MAX(a, b) ((a) < (b) ? (b) : (a))
92755 | #if !defined(MA_DR_MP3_NO_SIMD)
92756 | #if !defined(MA_DR_MP3_ONLY_SIMD) && (defined(_M_X64) || defined(__x86_64__) || defined(__aarch64__)
> || defined(_M_ARM64) || defined(_M_ARM64EC))
92757 | #define MA_DR_MP3_ONLY_SIMD
92758 | #endif
92759 | #if ((defined(_MSC_VER) && _MSC_VER >= 1400) && defined(_M_X64)) || ((defined(__i386) ||
> defined(_M_IX86) || defined(__i386__) || defined(__x86_64__) && ((defined(_M_IX86_FP) && _M_IX86_FP
> == 2) || defined(__SSE2__)))
92760 | #if defined(_MSC_VER)
92761 | #include <intrin.h>
92762 | #endif
92763 | #include <emmintrin.h>
92764 | #define MA_DR_MP3_HAVE_SSE 1
92765 | #define MA_DR_MP3_HAVE_SIMD 1
92766 | #define MA_DR_MP3_VSTORE _mm_storeu_ps
92767 | #define MA_DR_MP3_VLD _mm_loadu_ps
92768 | #define MA_DR_MP3_VSET _mm_set1_ps
92769 | #define MA_DR_MP3_VADD _mm_add_ps
92770 | #define MA_DR_MP3_VSUB _mm_sub_ps
92771 | #define MA_DR_MP3_VMUL _mm_mul_ps
92772 | #define MA_DR_MP3_VMAC(a, x, y) _mm_add_ps(a, _mm_mul_ps(x, y))
92773 | #define MA_DR_MP3_VMSB(a, x, y) _mm_sub_ps(a, _mm_mul_ps(x, y))
92774 | #define MA_DR_MP3_VMUL_S(x, s) _mm_mul_ps(x, _mm_set1_ps(s))
92775 | #define MA_DR_MP3_VREV(x) _mm_shuffle_ps(x, x, _MM_SHUFFLE(0, 1, 2, 3))
92776 | typedef __m128 ma_dr_mp3_f4;
92777 | #if (defined(_MSC_VER) || defined(MA_DR_MP3_ONLY_SIMD)) && !defined(__clang__)
92778 | #define ma_dr_mp3_cpuid _cpuid
92779 | #else
92780 | static __inline__ __attribute__((always_inline)) void ma_dr_mp3_cpuid(int CPUInfo[], const int
> InfoType)
92781 | {
92782 | #if defined(__PIC__)
92783 |     __asm__ __volatile__(
92784 | #if defined(__x86_64__)
92785 |         "push %%rbx\n"
92786 |         "cpuid\n"
92787 |         "xchgl %%ebx, %1\n"
92788 |         "pop %%rbx\n"

```



```

92789 | #else
92790 |     "xchgl %%ebx, %1\n"
92791 |     "cpuid\n"
92792 |     "xchgl %%ebx, %1\n"
92793 | #endif
92794 |     : "=a" (CPUInfo[0]), "=r" (CPUInfo[1]), "=c" (CPUInfo[2]), "=d" (CPUInfo[3])
92795 |     : "a" (InfoType));
92796 | #else
92797 |     __asm__ __volatile__(
92798 |         "cpuid"
92799 |         : "=a" (CPUInfo[0]), "=b" (CPUInfo[1]), "=c" (CPUInfo[2]), "=d" (CPUInfo[3])
92800 |         : "a" (InfoType));
92801 | #endif
92802 | }
92803 | #endif
92804 | static int ma_dr_mp3_have_simd(void)
92805 | {
92806 | #ifdef MA_DR_MP3_ONLY_SIMD
92807 |     return 1;
92808 | #else
92809 |     static int g_have_simd;
92810 |     int CPUInfo[4];
92811 | #ifdef MINIMP3_TEST
92812 |     static int g_counter;
92813 |     if (g_counter++ > 100)
92814 |         return 0;
92815 | #endif
92816 |     if (g_have_simd)
92817 |         goto end;
92818 |     ma_dr_mp3_cpuid(CPUInfo, 0);
92819 |     if (CPUInfo[0] > 0)
92820 |     {
92821 |         ma_dr_mp3_cpuid(CPUInfo, 1);
92822 |         g_have_simd = (CPUInfo[3] & (1 << 26)) + 1;
92823 |         return g_have_simd - 1;
92824 |     }
92825 | end:
92826 |     return g_have_simd - 1;
92827 | #endif
92828 | }
92829 | #elif defined(__ARM_NEON) || defined(__aarch64__) || defined(_M_ARM64) || defined(_M_ARM64EC)
92830 | #include <arm_neon.h>
92831 | #define MA_DR_MP3_HAVE_SSE 0
92832 | #define MA_DR_MP3_HAVE_SIMD 1
92833 | #define MA_DR_MP3_VSTORE vst1q_f32
92834 | #define MA_DR_MP3_VLD vld1q_f32
92835 | #define MA_DR_MP3_VSET vmovq_n_f32
92836 | #define MA_DR_MP3_VADD vaddq_f32
92837 | #define MA_DR_MP3_VSUB vsubq_f32
92838 | #define MA_DR_MP3_VMUL vmulq_f32
92839 | #define MA_DR_MP3_VMAC(a, x, y) vmlaq_f32(a, x, y)
92840 | #define MA_DR_MP3_VMSB(a, x, y) vmlsq_f32(a, x, y)
92841 | #define MA_DR_MP3_VMUL_S(x, s) vmulq_f32(x, vmovq_n_f32(s))
92842 | #define MA_DR_MP3_VREV(x) vcombine_f32(vget_high_f32(vrev64q_f32(x)), vget_low_f32(vrev64q_f32(x)))
92843 | typedef float32x4_t ma_dr_mp3_f4;
92844 | static int ma_dr_mp3_have_simd(void)
92845 | {
92846 |     return 1;
92847 | }
92848 | #else
92849 | #define MA_DR_MP3_HAVE_SSE 0
92850 | #define MA_DR_MP3_HAVE_SIMD 0
92851 | #ifdef MA_DR_MP3_ONLY_SIMD
92852 | #error MA_DR_MP3_ONLY_SIMD used, but SSE/NEON not enabled
92853 | #endif
92854 | #endif
92855 | #else
92856 | #define MA_DR_MP3_HAVE_SIMD 0
92857 | #endif
92858 | #if defined(__ARM_ARCH) && (__ARM_ARCH >= 6) && !defined(__aarch64__) && !defined(_M_ARM64) &&
> !defined(_M_ARM64EC) && !defined(__ARM_ARCH_6M__)
92859 | #define MA_DR_MP3_HAVE_ARMV6 1
92860 | static __inline__ __attribute__((always_inline)) ma_int32 ma_dr_mp3_clip_int16_arm(ma_int32 a)
92861 | {
92862 |     ma_int32 x = 0;
92863 |     __asm__ ("ssat %0, #16, %1 : "=r"(x) : "r"(a));
92864 |     return x;
92865 | }
92866 | #else

```

```

92867 | #define MA_DR_MP3_HAVE_ARMV6 0
92868 | #endif
92869 | #ifndef MA_DR_MP3_ASSERT
92870 | #include <assert.h>
92871 | #define MA_DR_MP3_ASSERT(expression) assert(expression)
92872 | #endif
92873 | #ifndef MA_DR_MP3_COPY_MEMORY
92874 | #define MA_DR_MP3_COPY_MEMORY(dst, src, sz) memcpy((dst), (src), (sz))
92875 | #endif
92876 | #ifndef MA_DR_MP3_MOVE_MEMORY
92877 | #define MA_DR_MP3_MOVE_MEMORY(dst, src, sz) memmove((dst), (src), (sz))
92878 | #endif
92879 | #ifndef MA_DR_MP3_ZERO_MEMORY
92880 | #define MA_DR_MP3_ZERO_MEMORY(p, sz) memset((p), 0, (sz))
92881 | #endif
92882 | #define MA_DR_MP3_ZERO_OBJECT(p) MA_DR_MP3_ZERO_MEMORY((p), sizeof(*(p)))
92883 | #ifndef MA_DR_MP3_MALLOC
92884 | #define MA_DR_MP3_MALLOC(sz) malloc((sz))
92885 | #endif
92886 | #ifndef MA_DR_MP3_REALLOC
92887 | #define MA_DR_MP3_REALLOC(p, sz) realloc((p), (sz))
92888 | #endif
92889 | #ifndef MA_DR_MP3_FREE
92890 | #define MA_DR_MP3_FREE(p) free((p))
92891 | #endif
92892 | typedef struct
92893 | {
92894 |     float scf[3*64];
92895 |     ma_uint8 total_bands, stereo_bands, bitalloc[64], scfcod[64];
92896 | } ma_dr_mp3_L12_scale_info;
92897 | typedef struct
92898 | {
92899 |     ma_uint8 tab_offset, code_tab_width, band_count;
92900 | } ma_dr_mp3_L12_subband_alloc;
92901 | static void ma_dr_mp3_bs_init(ma_dr_mp3_bs *bs, const ma_uint8 *data, int bytes)
92902 | {
92903 |     bs->buf = data;
92904 |     bs->pos = 0;
92905 |     bs->limit = bytes*8;
92906 | }
92907 | static ma_uint32 ma_dr_mp3_bs_get_bits(ma_dr_mp3_bs *bs, int n)
92908 | {
92909 |     ma_uint32 next, cache = 0, s = bs->pos & 7;
92910 |     int shl = n + s;
92911 |     const ma_uint8 *p = bs->buf + (bs->pos >> 3);
92912 |     if ((bs->pos += n) > bs->limit)
92913 |         return 0;
92914 |     next = *p++ & (255 >> s);
92915 |     while ((shl -= 8) > 0)
92916 |     {
92917 |         cache |= next << shl;
92918 |         next = *p++;
92919 |     }
92920 |     return cache | (next >> -shl);
92921 | }
92922 | static int ma_dr_mp3_hdr_valid(const ma_uint8 *h)
92923 | {
92924 |     return h[0] == 0xff &&
92925 |         ((h[1] & 0xF0) == 0xF0 || (h[1] & 0xFE) == 0xE2) &&
92926 |         (MA_DR_MP3_HDR_GET_LAYER(h) != 0) &&
92927 |         (MA_DR_MP3_HDR_GET_BITRATE(h) != 15) &&
92928 |         (MA_DR_MP3_HDR_GET_SAMPLE_RATE(h) != 3);
92929 | }
92930 | static int ma_dr_mp3_hdr_compare(const ma_uint8 *h1, const ma_uint8 *h2)
92931 | {
92932 |     return ma_dr_mp3_hdr_valid(h2) &&
92933 |         ((h1[1] ^ h2[1]) & 0xFE) == 0 &&
92934 |         ((h1[2] ^ h2[2]) & 0x0C) == 0 &&
92935 |         !((MA_DR_MP3_HDR_IS_FREE_FORMAT(h1) ^ MA_DR_MP3_HDR_IS_FREE_FORMAT(h2)));
92936 | }
92937 | static unsigned ma_dr_mp3_hdr_bitrate_kbps(const ma_uint8 *h)
92938 | {
92939 |     static const ma_uint8 halfrate[2][3][15] = {
92940 |         { { 0, 4, 8, 12, 16, 20, 24, 28, 32, 40, 48, 56, 64, 72, 80 }, { 0, 4, 8, 12, 16, 20, 24, 28, 32, 40, 48, 56, 64, 72, 80 } },
92941 |         { { 0, 16, 24, 28, 32, 40, 48, 56, 64, 72, 80, 88, 96, 112, 128 } }, { { 0, 16, 20, 24, 28, 32, 40, 48, 56, 64, 80, 96, 112, 128, 160 } }, { { 0, 16, 24, 28, 32, 40, 48, 56, 64, 80, 96, 112, 128, 160, 192 } }, { { 0, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192, 208, 224 } } },
92942 |     };

```

```

92943 |     return 2*halftrate[!MA_DR_MP3_HDR_TEST_MPEG1(h)][MA_DR_MP3_HDR_GET_LAYER(h) -
> 1][MA_DR_MP3_HDR_GET_BITRATE(h)];
92944 | }
92945 | static unsigned ma_dr_mp3_hdr_sample_rate_hz(const ma_uint8 *h)
92946 | {
92947 |     static const unsigned g_hz[3] = { 44100, 48000, 32000 };
92948 |     return g_hz[MA_DR_MP3_HDR_GET_SAMPLE_RATE(h)] >> (int)!MA_DR_MP3_HDR_TEST_MPEG1(h) >>
> (int)!MA_DR_MP3_HDR_TEST_NOT_MPEG25(h);
92949 | }
92950 | static unsigned ma_dr_mp3_hdr_frame_samples(const ma_uint8 *h)
92951 | {
92952 |     return MA_DR_MP3_HDR_IS_LAYER_1(h) ? 384 : (1152 >> (int)MA_DR_MP3_HDR_IS_FRAME_576(h));
92953 | }
92954 | static int ma_dr_mp3_hdr_frame_bytes(const ma_uint8 *h, int free_format_size)
92955 | {
92956 |     int frame_bytes =
> ma_dr_mp3_hdr_frame_samples(h)*ma_dr_mp3_hdr_bitrate_kbps(h)*125/ma_dr_mp3_hdr_sample_rate_hz(h);
92957 |     if (MA_DR_MP3_HDR_IS_LAYER_1(h))
92958 |     {
92959 |         frame_bytes &= ~3;
92960 |     }
92961 |     return frame_bytes ? frame_bytes : free_format_size;
92962 | }
92963 | static int ma_dr_mp3_hdr_padding(const ma_uint8 *h)
92964 | {
92965 |     return MA_DR_MP3_HDR_TEST_PADDING(h) ? (MA_DR_MP3_HDR_IS_LAYER_1(h) ? 4 : 1) : 0;
92966 | }
92967 | #ifndef MA_DR_MP3_ONLY_MP3
92968 | static const ma_dr_mp3_L12_subband_alloc *ma_dr_mp3_L12_subband_alloc_table(const ma_uint8 *hdr,
> ma_dr_mp3_L12_scale_info *sci)
92969 | {
92970 |     const ma_dr_mp3_L12_subband_alloc *alloc;
92971 |     int mode = MA_DR_MP3_HDR_GET_STEREO_MODE(hdr);
92972 |     int nbands, stereo_bands = (mode == MA_DR_MP3_MODE_MONO) ? 0 : (mode ==
> MA_DR_MP3_MODE_JOINT_STEREO) ? (MA_DR_MP3_HDR_GET_STEREO_MODE_EXT(hdr) <= 2) + 4 : 32;
92973 |     if (MA_DR_MP3_HDR_IS_LAYER_1(hdr))
92974 |     {
92975 |         static const ma_dr_mp3_L12_subband_alloc g_alloc_L1[] = { { 76, 4, 32 } };
92976 |         alloc = g_alloc_L1;
92977 |         nbands = 32;
92978 |     } else if (!MA_DR_MP3_HDR_TEST_MPEG1(hdr))
92979 |     {
92980 |         static const ma_dr_mp3_L12_subband_alloc g_alloc_L2M2[] = { { 60, 4, 4 }, { 44, 3, 7 }, {
> 44, 2, 19 } };
92981 |         alloc = g_alloc_L2M2;
92982 |         nbands = 30;
92983 |     } else
92984 |     {
92985 |         static const ma_dr_mp3_L12_subband_alloc g_alloc_L2M1[] = { { 0, 4, 3 }, { 16, 4, 8 }, { 32,
> 3, 12 }, { 40, 2, 7 } };
92986 |         int sample_rate_idx = MA_DR_MP3_HDR_GET_SAMPLE_RATE(hdr);
92987 |         unsigned kbps = ma_dr_mp3_hdr_bitrate_kbps(hdr) >> (int)(mode != MA_DR_MP3_MODE_MONO);
92988 |         if (!kbps)
92989 |         {
92990 |             kbps = 192;
92991 |         }
92992 |         alloc = g_alloc_L2M1;
92993 |         nbands = 27;
92994 |         if (kbps < 56)
92995 |         {
92996 |             static const ma_dr_mp3_L12_subband_alloc g_alloc_L2M1_lowrate[] = { { 44, 4, 2 }, { 44,
> 3, 10 } };
92997 |             alloc = g_alloc_L2M1_lowrate;
92998 |             nbands = sample_rate_idx == 2 ? 12 : 8;
92999 |         } else if (kbps >= 96 && sample_rate_idx != 1)
93000 |         {
93001 |             nbands = 30;
93002 |         }
93003 |     }
93004 |     sci->total_bands = (ma_uint8)nbands;
93005 |     sci->stereo_bands = (ma_uint8)MA_DR_MP3_MIN(stereo_bands, nbands);
93006 |     return alloc;
93007 | }
93008 | static void ma_dr_mp3_L12_read_scalefactors(ma_dr_mp3_bs *bs, ma_uint8 *pba, ma_uint8 *scfcod, int
> bands, float *scf)
93009 | {
93010 |     static const float g_deq_L12[18*3] = {
93011 | #define MA_DR_MP3_DQ(x) 9.53674316e-07f/x, 7.56931807e-07f/x, 6.00777173e-07f/x
93012 |     MA_DR_MP3_DQ(3),MA_DR_MP3_DQ(7),MA_DR_MP3_DQ(15),MA_DR_MP3_DQ(31),MA_DR_MP3_DQ(63),MA_DR_MP3

```

```

> _DQ(127),MA_DR_MP3_DQ(255),MA_DR_MP3_DQ(511),MA_DR_MP3_DQ(1023),MA_DR_MP3_DQ(2047),MA_DR_MP3_DQ(4095
> ),MA_DR_MP3_DQ(8191),MA_DR_MP3_DQ(16383),MA_DR_MP3_DQ(32767),MA_DR_MP3_DQ(65535),MA_DR_MP3_DQ(3),MA_
> DR_MP3_DQ(5),MA_DR_MP3_DQ(9)
93013 |     };
93014 |     int i, m;
93015 |     for (i = 0; i < bands; i++)
93016 |     {
93017 |         float s = 0;
93018 |         int ba = *pba++;
93019 |         int mask = ba ? 4 + ((19 >> scfcod[i]) & 3) : 0;
93020 |         for (m = 4; m; m >= 1)
93021 |         {
93022 |             if (mask & m)
93023 |             {
93024 |                 int b = ma_dr_mp3_bs_get_bits(bs, 6);
93025 |                 s = g_deq_L12[ba*3 - 6 + b % 3]*(int)(1 << 21 >> b/3);
93026 |             }
93027 |             *scf++ = s;
93028 |         }
93029 |     }
93030 | }
93031 | static void ma_dr_mp3_L12_read_scale_info(const ma_uint8 *hdr, ma_dr_mp3_bs *bs,
> ma_dr_mp3_L12_scale_info *sci)
93032 | {
93033 |     static const ma_uint8 g_bitalloc_code_tab[] = {
93034 |         0,17, 3, 4, 5,6,7, 8,9,10,11,12,13,14,15,16,
93035 |         0,17,18, 3,19,4,5, 6,7, 8, 9,10,11,12,13,16,
93036 |         0,17,18, 3,19,4,5,16,
93037 |         0,17,18,16,
93038 |         0,17,18,19, 4,5,6, 7,8, 9,10,11,12,13,14,15,
93039 |         0,17,18, 3,19,4,5, 6,7, 8, 9,10,11,12,13,14,
93040 |         0, 2, 3, 4, 5,6,7, 8,9,10,11,12,13,14,15,16
93041 |     };
93042 |     const ma_dr_mp3_L12_subband_alloc *subband_alloc = ma_dr_mp3_L12_subband_alloc_table(hdr, sci);
93043 |     int i, k = 0, ba_bits = 0;
93044 |     const ma_uint8 *ba_code_tab = g_bitalloc_code_tab;
93045 |     for (i = 0; i < sci->total_bands; i++)
93046 |     {
93047 |         ma_uint8 ba;
93048 |         if (i == k)
93049 |         {
93050 |             k += subband_alloc->band_count;
93051 |             ba_bits = subband_alloc->code_tab_width;
93052 |             ba_code_tab = g_bitalloc_code_tab + subband_alloc->tab_offset;
93053 |             subband_alloc++;
93054 |         }
93055 |         ba = ba_code_tab[ma_dr_mp3_bs_get_bits(bs, ba_bits)];
93056 |         sci->bitalloc[2*i] = ba;
93057 |         if (i < sci->stereo_bands)
93058 |         {
93059 |             ba = ba_code_tab[ma_dr_mp3_bs_get_bits(bs, ba_bits)];
93060 |         }
93061 |         sci->bitalloc[2*i + 1] = sci->stereo_bands ? ba : 0;
93062 |     }
93063 |     for (i = 0; i < 2*sci->total_bands; i++)
93064 |     {
93065 |         sci->scfcod[i] = (ma_uint8)(sci->bitalloc[i] ? MA_DR_MP3_HDR_IS_LAYER_1(hdr) ? 2 :
> ma_dr_mp3_bs_get_bits(bs, 2) : 6);
93066 |     }
93067 |     ma_dr_mp3_L12_read_scalefactors(bs, sci->bitalloc, sci->scfcod, sci->total_bands*2, sci->scf);
93068 |     for (i = sci->stereo_bands; i < sci->total_bands; i++)
93069 |     {
93070 |         sci->bitalloc[2*i + 1] = 0;
93071 |     }
93072 | }
93073 | static int ma_dr_mp3_L12_dequantize_granule(float *grbuf, ma_dr_mp3_bs *bs, ma_dr_mp3_L12_scale_info
> *sci, int group_size)
93074 | {
93075 |     int i, j, k, choff = 576;
93076 |     for (j = 0; j < 4; j++)
93077 |     {
93078 |         float *dst = grbuf + group_size*j;
93079 |         for (i = 0; i < 2*sci->total_bands; i++)
93080 |         {
93081 |             int ba = sci->bitalloc[i];
93082 |             if (ba != 0)
93083 |             {
93084 |                 if (ba < 17)
93085 |                 {

```

```

93086 |         int half = (1 << (ba - 1)) - 1;
93087 |         for (k = 0; k < group_size; k++)
93088 |         {
93089 |             dst[k] = (float)((int)ma_dr_mp3_bs_get_bits(bs, ba) - half);
93090 |         }
93091 |     } else
93092 |     {
93093 |         unsigned mod = (2 << (ba - 17)) + 1;
93094 |         unsigned code = ma_dr_mp3_bs_get_bits(bs, mod + 2 - (mod >> 3));
93095 |         for (k = 0; k < group_size; k++, code /= mod)
93096 |         {
93097 |             dst[k] = (float)((int)(code % mod - mod/2));
93098 |         }
93099 |     }
93100 | }
93101 | dst += choff;
93102 | choff = 18 - choff;
93103 | }
93104 | }
93105 | return group_size*4;
93106 | }
93107 | static void ma_dr_mp3_L12_apply_scf_384(ma_dr_mp3_L12_scale_info *sci, const float *scf, float *dst)
93108 | {
93109 |     int i, k;
93110 |     MA_DR_MP3_COPY_MEMORY(dst + 576 + sci->stereo_bands*18, dst + sci->stereo_bands*18,
> (sci->total_bands - sci->stereo_bands)*18*sizeof(float));
93111 |     for (i = 0; i < sci->total_bands; i++, dst += 18, scf += 6)
93112 |     {
93113 |         for (k = 0; k < 12; k++)
93114 |         {
93115 |             dst[k + 0] *= scf[0];
93116 |             dst[k + 576] *= scf[3];
93117 |         }
93118 |     }
93119 | }
93120 | #endif
93121 | static int ma_dr_mp3_L3_read_side_info(ma_dr_mp3_bs *bs, ma_dr_mp3_L3_gr_info *gr, const ma_uint8
> *hdr)
93122 | {
93123 |     static const ma_uint8 g_scf_long[8][23] = {
93124 |         { 6,6,6,6,6,6,8,10,12,14,16,20,24,28,32,38,46,52,60,68,58,54,0 },
93125 |         { 12,12,12,12,12,12,16,20,24,28,32,40,48,56,64,76,90,2,2,2,2,2,0 },
93126 |         { 6,6,6,6,6,6,8,10,12,14,16,20,24,28,32,38,46,52,60,68,58,54,0 },
93127 |         { 6,6,6,6,6,6,8,10,12,14,16,18,22,26,32,38,46,54,62,70,76,36,0 },
93128 |         { 6,6,6,6,6,6,8,10,12,14,16,20,24,28,32,38,46,52,60,68,58,54,0 },
93129 |         { 4,4,4,4,4,4,6,6,8,8,10,12,16,20,24,28,34,42,50,54,76,158,0 },
93130 |         { 4,4,4,4,4,4,6,6,8,10,12,16,18,22,28,34,40,46,54,54,192,0 },
93131 |         { 4,4,4,4,4,4,6,6,8,10,12,16,20,24,30,38,46,56,68,84,102,26,0 }
93132 |     };
93133 |     static const ma_uint8 g_scf_short[8][40] = {
93134 |         { 4,4,4,4,4,4,4,4,6,6,6,8,8,8,10,10,10,12,12,12,14,14,14,18,18,18,24,24,24,30,30,30,40,40,
> 40,18,18,18,0 },
93135 |         {
> 8,8,8,8,8,8,8,8,12,12,12,16,16,16,20,20,20,24,24,24,28,28,28,36,36,36,2,2,2,2,2,2,2,2,26,26,26,0
> },
93136 |         {
> 4,4,4,4,4,4,4,4,4,6,6,6,6,6,8,8,8,10,10,10,14,14,14,18,18,18,26,26,26,32,32,32,42,42,42,18,18,18,0
> },
93137 |         { 4,4,4,4,4,4,4,4,4,6,6,6,8,8,8,10,10,10,12,12,12,14,14,14,18,18,18,24,24,24,32,32,32,44,44,
> 44,12,12,12,0 },
93138 |         { 4,4,4,4,4,4,4,4,4,6,6,6,8,8,8,10,10,10,12,12,12,14,14,14,18,18,18,24,24,24,30,30,30,40,40,
> 40,18,18,18,0 },
93139 |         {
> 4,4,4,4,4,4,4,4,4,4,4,6,6,6,8,8,8,10,10,10,12,12,12,14,14,14,18,18,18,22,22,22,30,30,30,56,56,56,0
> },
93140 |         {
> 4,4,4,4,4,4,4,4,4,4,4,6,6,6,6,6,10,10,10,12,12,12,14,14,14,16,16,16,20,20,20,26,26,26,66,66,66,0
> },
93141 |         {
> 4,4,4,4,4,4,4,4,4,4,4,6,6,6,8,8,8,12,12,12,16,16,16,20,20,20,26,26,26,34,34,34,42,42,42,12,12,12,0
> }
93142 |     };
93143 |     static const ma_uint8 g_scf_mixed[8][40] = {
93144 |         {
> 6,6,6,6,6,6,6,6,8,8,8,10,10,10,12,12,12,14,14,14,18,18,18,24,24,24,30,30,30,40,40,40,18,18,18,0 },
93145 |         { 12,12,12,4,4,4,8,8,8,12,12,12,16,16,16,20,20,20,24,24,24,28,28,28,36,36,36,2,2,2,2,2,2,2,2,
> ,2,26,26,26,0 },
93146 |         {
> 6,6,6,6,6,6,6,6,6,6,6,8,8,8,10,10,10,14,14,14,18,18,18,26,26,26,32,32,32,42,42,42,18,18,18,0 },

```

```

93147 |         {
> 6,6,6,6,6,6,6,6,6,8,8,10,10,10,12,12,12,14,14,14,18,18,18,24,24,24,32,32,32,44,44,44,12,12,12,0 },
93148 |         {
> 6,6,6,6,6,6,6,6,6,8,8,10,10,10,12,12,12,14,14,14,18,18,18,24,24,24,30,30,30,40,40,40,18,18,18,0 },
93149 |         {
> 4,4,4,4,4,4,6,6,4,4,4,6,6,6,8,8,10,10,10,12,12,12,14,14,14,18,18,18,22,22,22,30,30,30,56,56,56,0
> },
93150 |         {
> 4,4,4,4,4,4,6,6,4,4,4,6,6,6,6,6,10,10,10,12,12,12,14,14,14,16,16,16,20,20,20,26,26,26,66,66,66,0
> },
93151 |         {
> 4,4,4,4,4,4,6,6,4,4,4,6,6,6,8,8,12,12,12,16,16,16,20,20,20,26,26,26,34,34,34,42,42,42,12,12,12,0 }
93152 |     };
93153 |     unsigned tables, scfsi = 0;
93154 |     int main_data_begin, part_23_sum = 0;
93155 |     int gr_count = MA_DR_MP3_HDR_IS_MONO(hdr) ? 1 : 2;
93156 |     int sr_idx = MA_DR_MP3_HDR_GET_MY_SAMPLE_RATE(hdr); sr_idx -= (sr_idx != 0);
93157 |     if (MA_DR_MP3_HDR_TEST_MPEG1(hdr))
93158 |     {
93159 |         gr_count *= 2;
93160 |         main_data_begin = ma_dr_mp3_bs_get_bits(bs, 9);
93161 |         scfsi = ma_dr_mp3_bs_get_bits(bs, 7 + gr_count);
93162 |     } else
93163 |     {
93164 |         main_data_begin = ma_dr_mp3_bs_get_bits(bs, 8 + gr_count) >> gr_count;
93165 |     }
93166 |     do
93167 |     {
93168 |         if (MA_DR_MP3_HDR_IS_MONO(hdr))
93169 |         {
93170 |             scfsi <= 4;
93171 |         }
93172 |         gr->part_23_length = (ma_uint16)ma_dr_mp3_bs_get_bits(bs, 12);
93173 |         part_23_sum += gr->part_23_length;
93174 |         gr->big_values = (ma_uint16)ma_dr_mp3_bs_get_bits(bs, 9);
93175 |         if (gr->big_values > 288)
93176 |         {
93177 |             return -1;
93178 |         }
93179 |         gr->global_gain = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 8);
93180 |         gr->scalefac_compress = (ma_uint16)ma_dr_mp3_bs_get_bits(bs, MA_DR_MP3_HDR_TEST_MPEG1(hdr) ?
> 4 : 9);
93181 |         gr->sfbtab = g_scf_long[sr_idx];
93182 |         gr->n_long_sfb = 22;
93183 |         gr->n_short_sfb = 0;
93184 |         if (ma_dr_mp3_bs_get_bits(bs, 1))
93185 |         {
93186 |             gr->block_type = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 2);
93187 |             if (!gr->block_type)
93188 |             {
93189 |                 return -1;
93190 |             }
93191 |             gr->mixed_block_flag = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 1);
93192 |             gr->region_count[0] = 7;
93193 |             gr->region_count[1] = 255;
93194 |             if (gr->block_type == MA_DR_MP3_SHORT_BLOCK_TYPE)
93195 |             {
93196 |                 scfsi &= 0x0F0F;
93197 |                 if (!gr->mixed_block_flag)
93198 |                 {
93199 |                     gr->region_count[0] = 8;
93200 |                     gr->sfbtab = g_scf_short[sr_idx];
93201 |                     gr->n_long_sfb = 0;
93202 |                     gr->n_short_sfb = 39;
93203 |                 } else
93204 |                 {
93205 |                     gr->sfbtab = g_scf_mixed[sr_idx];
93206 |                     gr->n_long_sfb = MA_DR_MP3_HDR_TEST_MPEG1(hdr) ? 8 : 6;
93207 |                     gr->n_short_sfb = 30;
93208 |                 }
93209 |             }
93210 |             tables = ma_dr_mp3_bs_get_bits(bs, 10);
93211 |             tables <= 5;
93212 |             gr->subblock_gain[0] = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 3);
93213 |             gr->subblock_gain[1] = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 3);
93214 |             gr->subblock_gain[2] = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 3);
93215 |         } else
93216 |         {
93217 |             gr->block_type = 0;

```

```

93218 |         gr->mixed_block_flag = 0;
93219 |         tables = ma_dr_mp3_bs_get_bits(bs, 15);
93220 |         gr->region_count[0] = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 4);
93221 |         gr->region_count[1] = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 3);
93222 |         gr->region_count[2] = 255;
93223 |     }
93224 |     gr->table_select[0] = (ma_uint8)(tables >> 10);
93225 |     gr->table_select[1] = (ma_uint8)((tables >> 5) & 31);
93226 |     gr->table_select[2] = (ma_uint8)((tables) & 31);
93227 |     gr->preflag = (ma_uint8)(MA_DR_MP3_HDR_TEST_MPEG1(hdr) ? ma_dr_mp3_bs_get_bits(bs, 1) :
> (gr->scalegfac_compress >= 500));
93228 |     gr->scalegfac_scale = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 1);
93229 |     gr->count1_table = (ma_uint8)ma_dr_mp3_bs_get_bits(bs, 1);
93230 |     gr->scfsi = (ma_uint8)((scfsi >> 12) & 15);
93231 |     scfsi <= 4;
93232 |     gr++;
93233 | } while(--gr_count);
93234 | if (part_23_sum + bs->pos > bs->limit + main_data_begin*8)
93235 | {
93236 |     return -1;
93237 | }
93238 | return main_data_begin;
93239 | }
93240 | static void ma_dr_mp3_L3_read_scalefactors(ma_uint8 *scf, ma_uint8 *ist_pos, const ma_uint8
> *scf_size, const ma_uint8 *scf_count, ma_dr_mp3_bs *bitbuf, int scfsi)
93241 | {
93242 |     int i, k;
93243 |     for (i = 0; i < 4 && scf_count[i]; i++, scfsi *= 2)
93244 |     {
93245 |         int cnt = scf_count[i];
93246 |         if (scfsi & 8)
93247 |         {
93248 |             MA_DR_MP3_COPY_MEMORY(scf, ist_pos, cnt);
93249 |         } else
93250 |         {
93251 |             int bits = scf_size[i];
93252 |             if (!bits)
93253 |             {
93254 |                 MA_DR_MP3_ZERO_MEMORY(scf, cnt);
93255 |                 MA_DR_MP3_ZERO_MEMORY(ist_pos, cnt);
93256 |             } else
93257 |             {
93258 |                 int max_scf = (scfsi < 0) ? (1 << bits) - 1 : -1;
93259 |                 for (k = 0; k < cnt; k++)
93260 |                 {
93261 |                     int s = ma_dr_mp3_bs_get_bits(bitbuf, bits);
93262 |                     ist_pos[k] = (ma_uint8)(s == max_scf ? -1 : s);
93263 |                     scf[k] = (ma_uint8)s;
93264 |                 }
93265 |             }
93266 |         }
93267 |         ist_pos += cnt;
93268 |         scf += cnt;
93269 |     }
93270 |     scf[0] = scf[1] = scf[2] = 0;
93271 | }
93272 | static float ma_dr_mp3_L3_ldexp_q2(float y, int exp_q2)
93273 | {
93274 |     static const float g_expfrac[4] = {
> 9.31322575e-10f, 7.83145814e-10f, 6.58544508e-10f, 5.53767716e-10f };
93275 |     int e;
93276 |     do
93277 |     {
93278 |         e = MA_DR_MP3_MIN(30*4, exp_q2);
93279 |         y *= g_expfrac[e & 3]*(1 << 30 >> (e >> 2));
93280 |     } while ((exp_q2 -= e) > 0);
93281 |     return y;
93282 | }
93283 | #if (defined(__GNUC__) && (__GNUC__ >= 13)) && !defined(__clang__)
93284 | #pragma GCC diagnostic push
93285 | #pragma GCC diagnostic ignored "-Wstringop-overflow"
93286 | #endif
93287 | static void ma_dr_mp3_L3_decode_scalefactors(const ma_uint8 *hdr, ma_uint8 *ist_pos, ma_dr_mp3_bs
> *bs, const ma_dr_mp3_L3_gr_info *gr, float *scf, int ch)
93288 | {
93289 |     static const ma_uint8 g_scf_partitions[3][28] = {
93290 |         { 6,5,5, 5,6,5,5,5,6,5, 7,3,11,10,0,0, 7, 7, 7,0, 6, 6,6,3, 8, 8,5,0 },
93291 |         { 8,9,6,12,6,9,9,9,6,9,12,6,15,18,0,0, 6,15,12,0, 6,12,9,6, 6,18,9,0 },
93292 |         { 9,9,6,12,9,9,9,9,9,9,12,6,18,18,0,0,12,12,12,0,12, 9,9,6,15,12,9,0 }

```



```

93293 | };
93294 | const ma_uint8 *scf_partition = g_scf_partitions[!!gr->n_short_sfb + !gr->n_long_sfb];
93295 | ma_uint8 scf_size[4], iscf[40];
93296 | int i, scf_shift = gr->scalefac_scale + 1, gain_exp, scfsi = gr->scfsi;
93297 | float gain;
93298 | if (MA_DR_MP3_HDR_TEST_MPEG1(hdr))
93299 | {
93300 |     static const ma_uint8 g_scf_decode[16] = { 0,1,2,3, 12,5,6,7, 9,10,11,13, 14,15,18,19 };
93301 |     int part = g_scf_decode[gr->scalefac_compress];
93302 |     scf_size[1] = scf_size[0] = (ma_uint8)(part >> 2);
93303 |     scf_size[3] = scf_size[2] = (ma_uint8)(part & 3);
93304 | } else
93305 | {
93306 |     static const ma_uint8 g_mod[6*4] = { 5,5,4,4,5,5,4,1,4,3,1,1,5,6,6,1,4,4,4,1,4,3,1,1 };
93307 |     int k, modprod, sfc, ist = MA_DR_MP3_HDR_TEST_I_STEREO(hdr) && ch;
93308 |     sfc = gr->scalefac_compress >> ist;
93309 |     for (k = ist*3*4; sfc >= 0; sfc -= modprod, k += 4)
93310 |     {
93311 |         for (modprod = 1, i = 3; i >= 0; i--)
93312 |         {
93313 |             scf_size[i] = (ma_uint8)(sfc / modprod % g_mod[k + i]);
93314 |             modprod *= g_mod[k + i];
93315 |         }
93316 |     }
93317 |     scf_partition += k;
93318 |     scfsi = -16;
93319 | }
93320 | ma_dr_mp3_L3_read_scalefactors(iscf, ist_pos, scf_size, scf_partition, bs, scfsi);
93321 | if (gr->n_short_sfb)
93322 | {
93323 |     int sh = 3 - scf_shift;
93324 |     for (i = 0; i < gr->n_short_sfb; i += 3)
93325 |     {
93326 |         iscf[gr->n_long_sfb + i + 0] = (ma_uint8)(iscf[gr->n_long_sfb + i + 0] +
> (gr->subblock_gain[0] << sh));
93327 |         iscf[gr->n_long_sfb + i + 1] = (ma_uint8)(iscf[gr->n_long_sfb + i + 1] +
> (gr->subblock_gain[1] << sh));
93328 |         iscf[gr->n_long_sfb + i + 2] = (ma_uint8)(iscf[gr->n_long_sfb + i + 2] +
> (gr->subblock_gain[2] << sh));
93329 |     }
93330 | } else if (gr->preflag)
93331 | {
93332 |     static const ma_uint8 g_preamp[10] = { 1,1,1,1,2,2,3,3,3,2 };
93333 |     for (i = 0; i < 10; i++)
93334 |     {
93335 |         iscf[11 + i] = (ma_uint8)(iscf[11 + i] + g_preamp[i]);
93336 |     }
93337 | }
93338 | gain_exp = gr->global_gain + MA_DR_MP3_BITS_DEQUANTIZER_OUT*4 - 210 -
> (MA_DR_MP3_HDR_IS_MS_STEREO(hdr) ? 2 : 0);
93339 | gain = ma_dr_mp3_L3_ldexp_q2(1 << (MA_DR_MP3_MAX_SCFI/4), MA_DR_MP3_MAX_SCFI - gain_exp);
93340 | for (i = 0; i < (int)(gr->n_long_sfb + gr->n_short_sfb); i++)
93341 | {
93342 |     scf[i] = ma_dr_mp3_L3_ldexp_q2(gain, iscf[i] << scf_shift);
93343 | }
93344 | }
93345 | #if (defined(__GNUC__) && (__GNUC__ >= 13)) && !defined(__clang__)
93346 | #pragma GCC diagnostic pop
93347 | #endif
93348 | static const float ma_dr_mp3_g_pow43[129 + 16] = {
93349 |     0, -1, -2.519842f, -4.326749f, -6.349604f, -8.549880f, -10.902724f, -13.390518f, -16.000000f, -18.720754f
> , -21.544347f, -24.463781f, -27.473142f, -30.567351f, -33.741992f, -36.993181f,
93350 |     0, 1, 2.519842f, 4.326749f, 6.349604f, 8.549880f, 10.902724f, 13.390518f, 16.000000f, 18.720754f, 21.54434
> 7f, 24.463781f, 27.473142f, 30.567351f, 33.741992f, 36.993181f, 40.317474f, 43.711787f, 47.173345f, 50.699631
> f, 54.288352f, 57.937408f, 61.644865f, 65.408941f, 69.227979f, 73.100443f, 77.024898f, 81.000000f, 85.024491f
> , 89.097188f, 93.216975f, 97.382800f, 101.593667f, 105.848633f, 110.146801f, 114.487321f, 118.869381f, 123.29
> 2209f, 127.755065f, 132.257246f, 136.798076f, 141.376907f, 145.993119f, 150.646117f, 155.335327f, 160.060199
> f, 164.820202f, 169.614826f, 174.443577f, 179.305980f, 184.201575f, 189.129918f, 194.090580f, 199.083145f, 20
> 4.107210f, 209.162385f, 214.248292f, 219.364564f, 224.510845f, 229.686789f, 234.892058f, 240.126328f, 245.38
> 9280f, 250.680604f, 256.000000f, 261.347174f, 266.721841f, 272.123723f, 277.552547f, 283.008049f, 288.489971
> f, 293.998060f, 299.532071f, 305.091761f, 310.676898f, 316.287249f, 321.922592f, 327.582707f, 333.267377f, 33
> 8.976394f, 344.709550f, 350.466646f, 356.247482f, 362.051866f, 367.879608f, 373.730522f, 379.604427f, 385.50
> 1143f, 391.420496f, 397.362314f, 403.326427f, 409.312672f, 415.320884f, 421.350905f, 427.402579f, 433.475750
> f, 439.570269f, 445.685987f, 451.822757f, 457.980436f, 464.158883f, 470.357960f, 476.577530f, 482.817459f, 48
> 9.077615f, 495.357868f, 501.658090f, 507.978156f, 514.317941f, 520.677324f, 527.056184f, 533.454404f, 539.87
> 1867f, 546.308458f, 552.764065f, 559.238575f, 565.731879f, 572.243870f, 578.774440f, 585.323483f, 591.890898
> f, 598.476581f, 605.080431f, 611.702349f, 618.342238f, 625.000000f, 631.675540f, 638.368763f, 645.079578f
93351 | };
93352 | static float ma_dr_mp3_L3_pow_43(int x)

```

```
93353 | {
93354 |     float frac;
93355 |     int sign, mult = 256;
93356 |     if (x < 129)
93357 |     {
93358 |         return ma_dr_mp3_g_pow43[16 + x];
93359 |     }
93360 |     if (x < 1024)
93361 |     {
93362 |         mult = 16;
93363 |         x <= 3;
93364 |     }
93365 |     sign = 2*x & 64;
93366 |     frac = (float)((x & 63) - sign) / ((x & ~63) + sign);
93367 |     return ma_dr_mp3_g_pow43[16 + ((x + sign) >> 6)]*(1.f + frac*((4.f/3) + frac*(2.f/9)))*mult;
93368 | }
93369 | static void ma_dr_mp3_L3_huffman(float *dst, ma_dr_mp3_bs *bs, const ma_dr_mp3_L3_gr_info *gr_info,
> const float *scf, int layer3gr_limit)
93370 | {
93371 |     static const ma_int16 tabs[] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
93372 |     785, 785, 785, 785, 784, 784, 784, 784, 513, 513, 513, 513, 513, 513, 513, 513, 256, 256, 256, 256, 256, 256, 256,
> 256, 256, 256, 256, 256, 256, 256, 256, 256,
93373 |     -255, 1313, 1298, 1282, 785, 785, 785, 785, 784, 784, 784, 784, 769, 769, 769, 769, 256, 256, 256, 256, 256, 256,
> 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256,
93374 |     -255, 1313, 1298, 1282, 769, 769, 769, 769, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529, 529,
> 528, 528, 512, 512, 512, 512, 512, 512, 512, 512, 290, 288,
93375 |     -253, -318, -351, -367, 785, 785, 785, 785, 784, 784, 784, 784, 769, 769, 769, 769, 256, 256, 256, 256, 256, 256,
> 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256,
> 258,
93376 |     -254, -287, 1329, 1299, 1314, 1312, 1057, 1057, 1042, 1042, 1026, 1026, 784, 784, 784, 784, 529, 529, 529, 529,
> 529, 529, 529, 529, 769, 769, 769, 769, 768, 768, 768, 768, 563, 560, 306, 306, 291, 259,
93377 |     -252, -413, -477, -542, 1298, -575, 1041, 1041, 784, 784, 784, 784, 769, 769, 769, 769, 256, 256, 256, 256, 256,
> 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256,
> 773, 773, 1076, 1075, 341, 340, 325, 309, 834, 804, 577, 577, 532, 532, 516, 516, 832, 818, 803, 816, 561, 561, 531, 531, 51
> 5, 546, 289, 289, 288, 258,
93378 |     -252, -429, -493, -559, 1057, 1057, 1042, 1042, 529, 529, 529, 529, 529, 529, 529, 529, 784, 784, 784, 784, 769,
> 769, 769, 769, 512, 512, 512, 512, 512, 512, 512, 512, -382, 1077, -415, 1106, 1061, 1104, 849, 849, 789, 789, 1091, 1076,
> 1029, 1075, 834, 834, 597, 581, 340, 340, 339, 324, 804, 833, 532, 532, 832, 772, 818, 803, 817, 787, 816, 771, 290, 290, 29
> 0, 290, 288, 258,
93379 |     -253, -349, -414, -447, -463, 1329, 1299, -479, 1314, 1312, 1057, 1057, 1042, 1042, 1026, 1026, 785, 785, 785,
> 785, 784, 784, 784, 784, 769, 769, 769, 769, 768, 768, 768, 768, -319, 851, 821, -335, 836, 850, 805, 849, 341, 340, 325, 33
> 6, 533, 533, 579, 579, 564, 564, 773, 832, 578, 548, 563, 516, 321, 276, 306, 291, 304, 259,
93380 |     -251, -572, -733, -830, -863, -879, 1041, 1041, 784, 784, 784, 784, 769, 769, 769, 769, 256, 256, 256, 256, 256,
> 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256,
> 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256,
> 334, 1138, 1138, 1063, 1063, 1350, 1392, 1031, 1031, 1062, 1062, 1364, 1363, 1120, 1120, 1333, 1348, 881, 881, 881, 881,
> 375, 374, 359, 373, 343, 358, 341, 325, 791, 791, 1123, 1122, -703, 1105, 1045, -719, 865, 865, 790, 790, 774, 774, 1104, 1
> 029, 338, 293, 323, 308, -799, -815, 833, 788, 772, 818, 803, 816, 322, 292, 307, 320, 561, 531, 515, 546, 289, 274, 288, 25
> 8,
93381 |     -251, -525, -605, -685, -765, -831, -846, 1298, 1057, 1057, 1312, 1282, 785, 785, 785, 785, 784, 784, 784, 784,
> 769, 769, 769, 769, 512, 512, 512, 512, 512, 512, 512, 512, 1399, 1398, 1383, 1367, 1382, 1396, 1351, -511, 1381, 1366, 11
> 39, 1139, 1079, 1079, 1124, 1124, 1364, 1349, 1363, 1333, 882, 882, 882, 807, 807, 807, 807, 1094, 1094, 1136, 1136,
> 373, 341, 535, 535, 881, 775, 867, 822, 774, -591, 324, 338, -671, 849, 550, 550, 866, 864, 609, 609, 293, 336, 534, 534, 78
> 9, 835, 773, -751, 834, 804, 308, 307, 833, 788, 832, 772, 562, 562, 547, 547, 305, 275, 560, 515, 290, 290,
93382 |     -252, -397, -477, -557, -622, -653, -719, -735, -750, 1329, 1299, 1314, 1057, 1057, 1042, 1042, 1312, 1282, 10
> 24, 1024, 785, 785, 785, 784, 784, 784, 784, 769, 769, 769, 769, -383, 1127, 1141, 1111, 1126, 1140, 1095, 1110, 869,
> 869, 883, 883, 1079, 1109, 882, 882, 375, 374, 807, 868, 838, 881, 791, -463, 867, 822, 368, 263, 852, 837, 836, -543, 610,
> 610, 550, 550, 352, 336, 534, 534, 865, 774, 851, 821, 850, 805, 593, 533, 579, 564, 773, 832, 578, 578, 548, 548, 577, 577,
> 307, 276, 306, 291, 516, 560, 259, 259,
93383 |     -250, -2107, -2507, -2764, -2909, -2974, -3007, -3023, 1041, 1041, 1040, 1040, 769, 769, 769, 769, 256, 256, 2
> 56, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256,
> 5, -1550, -1582, -1614, -1647, -1662, -1694, -1726, -1759, -1774, -1807, -1822, -1854, -1886, 1565, -1919, -1935, -19
> 51, -1967, 1731, 1730, 1580, 1717, -1983, 1729, 1564, -1999, 1548, -2015, -2031, 1715, 1595, -2047, 1714, -2063, 1610,
> -2079, 1609, -2095, 1323, 1323, 1457, 1457, 1307, 1307, 1712, 1547, 1641, 1700, 1699, 1594, 1685, 1625, 1442, 1442, 132
> 2, 1322, -780, -973, -910, 1279, 1278, 1277, 1262, 1276, 1261, 1275, 1215, 1260, 1229, -959, 974, 974, 989, 989, -943, 73
> 5, 478, 478, 495, 463, 506, 414, -1039, 1003, 958, 1017, 927, 942, 987, 957, 431, 476, 1272, 1167, 1228, -1183, 1256, -119
> 9, 895, 895, 941, 941, 1242, 1227, 1212, 1135, 1014, 1014, 490, 489, 503, 487, 910, 1013, 985, 925, 863, 894, 970, 955, 101
> 2, 847, -1343, 831, 755, 755, 984, 909, 428, 366, 754, 559, -1391, 752, 486, 457, 924, 997, 698, 698, 983, 893, 740, 740, 90
> 8, 877, 739, 739, 667, 667, 953, 938, 497, 287, 271, 271, 683, 606, 590, 712, 726, 574, 302, 302, 738, 736, 481, 286, 526, 72
> 5, 605, 711, 636, 724, 696, 651, 589, 681, 666, 710, 364, 467, 573, 695, 466, 466, 301, 465, 379, 379, 709, 604, 665, 679, 31
> 6, 316, 634, 633, 436, 436, 464, 269, 424, 394, 452, 332, 438, 363, 347, 408, 393, 448, 331, 422, 362, 407, 392, 421, 346, 40
> 6, 391, 376, 375, 359, 1441, 1306, -2367, 1290, -2383, 1337, -2399, -2415, 1426, 1321, -2431, 1411, 1336, -2447, -2463,
> -2479, 1169, 1169, 1049, 1049, 1424, 1289, 1412, 1352, 1319, -2495, 1154, 1154, 1064, 1064, 1153, 1153, 416, 390, 360, 4
> 04, 403, 389, 344, 374, 373, 343, 358, 372, 327, 357, 342, 311, 356, 326, 1395, 1394, 1137, 1137, 1047, 1047, 1365, 1392, 1
> 287, 1379, 1334, 1364, 1349, 1378, 1318, 1363, 792, 792, 792, 792, 1152, 1152, 1032, 1032, 1121, 1121, 1046, 1046, 1120,
> 1120, 1030, 1030, -2895, 1106, 1061, 1104, 849, 849, 789, 789, 1091, 1076, 1029, 1090, 1060, 1075, 833, 833, 309, 324, 53
> 2, 532, 832, 772, 818, 803, 561, 561, 531, 560, 515, 546, 289, 274, 288, 258,
93384 |     -250, -1179, -1579, -1836, -1996, -2124, -2253, -2333, -2413, -2477, -2542, -2574, -2607, -2622, -2655, 131
> 4, 1313, 1298, 1312, 1282, 785, 785, 785, 785, 1040, 1040, 1025, 1025, 768, 768, 768, 768, -766, -798, -830, -862, -895, -
```

```

> 911, -927, -943, -959, -975, -991, -1007, -1023, -1039, -1055, -1070, 1724, 1647, -1103, -1119, 1631, 1767, 1662, 1738
> , 1708, 1723, -1135, 1780, 1615, 1779, 1599, 1677, 1646, 1778, 1583, -1151, 1777, 1567, 1737, 1692, 1765, 1722, 1707, 16
> 30, 1751, 1661, 1764, 1614, 1736, 1676, 1763, 1750, 1645, 1598, 1721, 1691, 1762, 1706, 1582, 1761, 1566, -1167, 1749, 1
> 629, 767, 766, 751, 765, 494, 494, 735, 764, 719, 749, 734, 763, 447, 447, 748, 718, 477, 506, 431, 491, 446, 476, 461, 505,
> 415, 430, 475, 445, 504, 399, 460, 489, 414, 503, 383, 474, 429, 459, 502, 502, 746, 752, 488, 398, 501, 473, 413, 472, 486,
> 271, 480, 270, -1439, -1455, 1357, -1471, -1487, -1503, 1341, 1325, -1519, 1489, 1463, 1403, 1309, -1535, 1372, 1448, 1
> 418, 1476, 1356, 1462, 1387, -1551, 1475, 1340, 1447, 1402, 1386, -1567, 1068, 1068, 1474, 1461, 455, 380, 468, 440, 395
> , 425, 410, 454, 364, 467, 466, 464, 453, 269, 409, 448, 268, 432, 1371, 1473, 1432, 1417, 1308, 1460, 1355, 1446, 1459, 14
> 31, 1083, 1083, 1401, 1416, 1458, 1445, 1067, 1067, 1370, 1457, 1051, 1051, 1291, 1430, 1385, 1444, 1354, 1415, 1400, 14
> 43, 1082, 1082, 1173, 1113, 1186, 1066, 1185, 1050, -1967, 1158, 1128, 1172, 1097, 1171, 1081, -1983, 1157, 1112, 416, 2
> 66, 375, 400, 1170, 1142, 1127, 1065, 793, 793, 1169, 1033, 1156, 1096, 1141, 1111, 1155, 1080, 1126, 1140, 898, 898, 808
> , 808, 897, 897, 792, 792, 1095, 1152, 1032, 1125, 1110, 1139, 1079, 1124, 882, 807, 838, 881, 853, 791, -2319, 867, 368, 2
> 63, 822, 852, 837, 866, 806, 865, -2399, 851, 352, 262, 534, 534, 821, 836, 594, 594, 549, 549, 593, 593, 533, 533, 848, 773
> , 579, 579, 564, 578, 548, 563, 276, 276, 577, 576, 306, 291, 516, 560, 305, 305, 275, 259,
93385 | -251, -892, -2058, -2620, -2828, -2957, -3023, -3039, 1041, 1041, 1040, 1040, 769, 769, 769, 769, 256, 256, 25
> 6, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, 256, -511, -527, -543, -559, 1530, -575, -591, 1528, 1527, 1
> 407, 1526, 1391, 1023, 1023, 1023, 1023, 1525, 1375, 1268, 1268, 1103, 1103, 1087, 1087, 1039, 1039, 1523, -604, 815, 81
> 5, 815, 815, 510, 495, 509, 479, 508, 463, 507, 447, 431, 505, 415, 399, -734, -782, 1262, -815, 1259, 1244, -831, 1258, 12
> 28, -847, -863, 1196, -879, 1253, 987, 987, 748, -767, 493, 493, 462, 477, 414, 414, 686, 669, 478, 446, 461, 445, 474, 429
> , 487, 458, 412, 471, 1266, 1264, 1009, 1009, 799, 799, -1019, -1276, -1452, -1581, -1677, -1757, -1821, -1886, -1933, -
> 1997, 1257, 1257, 1483, 1468, 1512, 1422, 1497, 1406, 1467, 1496, 1421, 1510, 1134, 1134, 1225, 1225, 1466, 1451, 1374,
> 1405, 1252, 1252, 1358, 1480, 1164, 1164, 1251, 1251, 1238, 1238, 1389, 1465, -1407, 1054, 1101, -1423, 1207, -1439, 83
> 0, 830, 1248, 1038, 1237, 1117, 1223, 1148, 1236, 1208, 411, 426, 395, 410, 379, 269, 1193, 1222, 1132, 1235, 1221, 1116,
> 976, 976, 1192, 1162, 1177, 1220, 1131, 1191, 963, 963, -1647, 961, 780, -1663, 558, 558, 994, 993, 437, 408, 393, 407, 82
> 9, 978, 813, 797, 947, -1743, 721, 721, 377, 392, 844, 950, 828, 890, 706, 706, 812, 859, 796, 960, 948, 843, 934, 874, 571,
> 571, -1919, 690, 555, 689, 421, 346, 539, 539, 944, 779, 918, 873, 932, 842, 903, 888, 570, 570, 931, 917, 674, 674, -2575,
> 1562, -2591, 1609, -2607, 1654, 1322, 1322, 1441, 1441, 1696, 1546, 1683, 1593, 1669, 1624, 1426, 1426, 1321, 1321, 163
> 9, 1680, 1425, 1425, 1305, 1305, 1545, 1668, 1608, 1623, 1667, 1592, 1638, 1666, 1320, 1320, 1652, 1607, 1409, 1409, 130
> 4, 1304, 1288, 1288, 1664, 1637, 1395, 1395, 1335, 1335, 1622, 1636, 1394, 1394, 1319, 1319, 1606, 1621, 1392, 1392, 113
> 7, 1137, 1137, 1137, 345, 390, 360, 375, 404, 373, 1047, -2751, -2767, -2783, 1062, 1121, 1046, -2799, 1077, -2815, 1106
> , 1061, 789, 789, 1105, 1104, 263, 355, 310, 340, 325, 354, 352, 262, 339, 324, 1091, 1076, 1029, 1090, 1060, 1075, 833, 83
> 3, 788, 788, 1088, 1028, 818, 818, 803, 803, 561, 561, 531, 531, 816, 771, 546, 546, 289, 274, 288, 258,
93386 | -253, -317, -381, -446, -478, -509, 1279, 1279, -811, -1179, -1451, -1756, -1900, -2028, -2189, -2253, -2333
> , -2414, -2445, -2511, -2526, 1313, 1298, -2559, 1041, 1041, 1040, 1040, 1025, 1025, 1024, 1024, 1022, 1007, 1021, 991,
> 1020, 975, 1019, 959, 687, 687, 1018, 1017, 671, 671, 655, 655, 1016, 1015, 639, 639, 758, 758, 623, 623, 757, 607, 756, 59
> 1, 755, 575, 754, 559, 543, 543, 1009, 783, -575, -621, -685, -749, 496, -590, 750, 749, 734, 748, 974, 989, 1003, 958, 988
> , 973, 1002, 942, 987, 957, 972, 1001, 926, 986, 941, 971, 956, 1000, 910, 985, 925, 999, 894, 970, -1071, -1087, -1102, 13
> 90, -1135, 1436, 1509, 1451, 1374, -1151, 1405, 1358, 1480, 1420, -1167, 1507, 1494, 1389, 1342, 1465, 1435, 1450, 1326
> , 1505, 1310, 1493, 1373, 1479, 1404, 1492, 1464, 1419, 428, 443, 472, 397, 736, 526, 464, 464, 486, 457, 442, 471, 484, 48
> 2, 1357, 1449, 1434, 1478, 1388, 1491, 1341, 1490, 1325, 1489, 1463, 1403, 1309, 1477, 1372, 1448, 1418, 1433, 1476, 135
> 6, 1462, 1387, -1439, 1475, 1340, 1447, 1402, 1474, 1324, 1461, 1371, 1473, 269, 448, 1432, 1417, 1308, 1460, -1711, 145
> 9, -1727, 1441, 1099, 1099, 1446, 1386, 1431, 1401, -1743, 1289, 1083, 1083, 1160, 1160, 1458, 1445, 1067, 1067, 1370, 1
> 457, 1307, 1430, 1129, 1129, 1098, 1098, 268, 432, 267, 416, 266, 400, -1887, 1144, 1187, 1082, 1173, 1113, 1186, 1066, 1
> 050, 1158, 1128, 1143, 1172, 1097, 1171, 1081, 420, 391, 1157, 1112, 1170, 1142, 1127, 1065, 1169, 1049, 1156, 1096, 114
> 1, 1111, 1155, 1080, 1126, 1154, 1064, 1153, 1140, 1095, 1048, -2159, 1125, 1110, 1137, -2175, 823, 823, 1139, 1138, 807
> , 807, 384, 264, 368, 263, 868, 838, 853, 791, 867, 822, 852, 837, 866, 806, 865, 790, -2319, 851, 821, 836, 352, 262, 850, 8
> 05, 849, -2399, 533, 533, 835, 820, 336, 261, 578, 548, 563, 577, 532, 532, 832, 772, 562, 562, 547, 547, 305, 275, 560, 515
> , 290, 290, 288, 258 };
93387 | static const ma_uint8 tab32[] = {
> 130, 162, 193, 209, 44, 28, 76, 140, 9, 9, 9, 9, 9, 9, 9, 190, 254, 222, 238, 126, 94, 157, 157, 109, 61, 173, 205};
93388 | static const ma_uint8 tab33[] = { 252, 236, 220, 204, 188, 172, 156, 140, 124, 108, 92, 76, 60, 44, 28, 12 };
93389 | static const ma_int16 tabindex[2*16] = { 0, 32, 64, 98, 0, 132, 180, 218, 292, 364, 426, 538, 648, 746, 0, 1126
> , 1460, 1460, 1460, 1460, 1460, 1460, 1460, 1842, 1842, 1842, 1842, 1842, 1842, 1842, 1842 };
93390 | static const ma_uint8 g_linbits[] = {
> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 6, 8, 10, 13, 4, 5, 6, 7, 8, 9, 11, 13 };
93391 | #define MA_DR_MP3_PEEK_BITS(n) (bs_cache >> (32 - (n)))
93392 | #define MA_DR_MP3_FLUSH_BITS(n) { bs_cache <= (n); bs_sh += (n); }
93393 | #define MA_DR_MP3_CHECK_BITS while (bs_sh >= 0) { bs_cache |= (ma_uint32)*bs_next_ptr++ <<
> bs_sh; bs_sh -= 8; }
93394 | #define MA_DR_MP3_BSPOS ((bs_next_ptr - bs->buf)*8 - 24 + bs_sh)
93395 | float one = 0.0f;
93396 | int ireg = 0, big_val_cnt = gr_info->big_values;
93397 | const ma_uint8 *sfb = gr_info->sfbtab;
93398 | const ma_uint8 *bs_next_ptr = bs->buf + bs->pos/8;
93399 | ma_uint32 bs_cache = (((bs_next_ptr[0]*256u + bs_next_ptr[1])*256u + bs_next_ptr[2])*256u +
> bs_next_ptr[3]) << (bs->pos & 7);
93400 | int pairs_to_decode, np, bs_sh = (bs->pos & 7) - 8;
93401 | bs_next_ptr += 4;
93402 | while (big_val_cnt > 0)
93403 | {
93404 | int tab_num = gr_info->table_select[ireg];
93405 | int sfb_cnt = gr_info->region_count[ireg++];
93406 | const ma_int16 *codebook = tabs + tabindex[tab_num];
93407 | int linbits = g_linbits[tab_num];
93408 | if (linbits)
93409 | {
93410 | do
93411 | {
93412 | np = *sfb++ / 2;

```

```

93413 |         pairs_to_decode = MA_DR_MP3_MIN(big_val_cnt, np);
93414 |         one = *scf++;
93415 |         do
93416 |         {
93417 |             int j, w = 5;
93418 |             int leaf = codebook[MA_DR_MP3_PEEK_BITS(w)];
93419 |             while (leaf < 0)
93420 |             {
93421 |                 MA_DR_MP3_FLUSH_BITS(w);
93422 |                 w = leaf & 7;
93423 |                 leaf = codebook[MA_DR_MP3_PEEK_BITS(w) - (leaf >> 3)];
93424 |             }
93425 |             MA_DR_MP3_FLUSH_BITS(leaf >> 8);
93426 |             for (j = 0; j < 2; j++, dst++, leaf >>= 4)
93427 |             {
93428 |                 int lsb = leaf & 0x0F;
93429 |                 if (lsb == 15)
93430 |                 {
93431 |                     lsb += MA_DR_MP3_PEEK_BITS(linbits);
93432 |                     MA_DR_MP3_FLUSH_BITS(linbits);
93433 |                     MA_DR_MP3_CHECK_BITS;
93434 |                     *dst = one*ma_dr_mp3_L3_pow43(lsb)*((ma_int32)bs_cache < 0 ? -1: 1);
93435 |                 } else
93436 |                 {
93437 |                     *dst = ma_dr_mp3_g_pow43[16 + lsb - 16*(bs_cache >> 31)]*one;
93438 |                 }
93439 |                 MA_DR_MP3_FLUSH_BITS(lsb ? 1 : 0);
93440 |             }
93441 |             MA_DR_MP3_CHECK_BITS;
93442 |         } while (--pairs_to_decode);
93443 |     } while ((big_val_cnt -= np) > 0 && --sfb_cnt >= 0);
93444 | } else
93445 | {
93446 |     do
93447 |     {
93448 |         np = *sfb++ / 2;
93449 |         pairs_to_decode = MA_DR_MP3_MIN(big_val_cnt, np);
93450 |         one = *scf++;
93451 |         do
93452 |         {
93453 |             int j, w = 5;
93454 |             int leaf = codebook[MA_DR_MP3_PEEK_BITS(w)];
93455 |             while (leaf < 0)
93456 |             {
93457 |                 MA_DR_MP3_FLUSH_BITS(w);
93458 |                 w = leaf & 7;
93459 |                 leaf = codebook[MA_DR_MP3_PEEK_BITS(w) - (leaf >> 3)];
93460 |             }
93461 |             MA_DR_MP3_FLUSH_BITS(leaf >> 8);
93462 |             for (j = 0; j < 2; j++, dst++, leaf >>= 4)
93463 |             {
93464 |                 int lsb = leaf & 0x0F;
93465 |                 *dst = ma_dr_mp3_g_pow43[16 + lsb - 16*(bs_cache >> 31)]*one;
93466 |                 MA_DR_MP3_FLUSH_BITS(lsb ? 1 : 0);
93467 |             }
93468 |             MA_DR_MP3_CHECK_BITS;
93469 |         } while (--pairs_to_decode);
93470 |     } while ((big_val_cnt -= np) > 0 && --sfb_cnt >= 0);
93471 | }
93472 | }
93473 | for (np = 1 - big_val_cnt;; dst += 4)
93474 | {
93475 |     const ma_uint8 *codebook_count1 = (gr_info->count1_table) ? tab33 : tab32;
93476 |     int leaf = codebook_count1[MA_DR_MP3_PEEK_BITS(4)];
93477 |     if (!(leaf & 8))
93478 |     {
93479 |         leaf = codebook_count1[(leaf >> 3) + (bs_cache << 4 >> (32 - (leaf & 3)))]];
93480 |     }
93481 |     MA_DR_MP3_FLUSH_BITS(leaf & 7);
93482 |     if (MA_DR_MP3_BSPOS > layer3gr_limit)
93483 |     {
93484 |         break;
93485 |     }
93486 | #define MA_DR_MP3_RELOAD_SCALEFACTOR if (!--np) { np = *sfb++/2; if (!np) break; one = *scf++; }
93487 | #define MA_DR_MP3_DEQ_COUNT1(s) if (leaf & (128 >> s)) { dst[s] = ((ma_int32)bs_cache < 0) ? -one :
> one; MA_DR_MP3_FLUSH_BITS(1) }
93488 |     MA_DR_MP3_RELOAD_SCALEFACTOR;
93489 |     MA_DR_MP3_DEQ_COUNT1(0);
93490 |     MA_DR_MP3_DEQ_COUNT1(1);

```

```

93491 |         MA_DR_MP3_RELOAD_SCALEFACTOR;
93492 |         MA_DR_MP3_DEQ_COUNT1(2);
93493 |         MA_DR_MP3_DEQ_COUNT1(3);
93494 |         MA_DR_MP3_CHECK_BITS;
93495 |     }
93496 |     bs->pos = layer3gr_limit;
93497 | }
93498 | static void ma_dr_mp3_L3_midside_stereo(float *left, int n)
93499 | {
93500 |     int i = 0;
93501 |     float *right = left + 576;
93502 |     #if MA_DR_MP3_HAVE_SIMD
93503 |     if (ma_dr_mp3_have_simd())
93504 |     {
93505 |         for (; i < n - 3; i += 4)
93506 |         {
93507 |             ma_dr_mp3_f4 vl = MA_DR_MP3_VLD(left + i);
93508 |             ma_dr_mp3_f4 vr = MA_DR_MP3_VLD(right + i);
93509 |             MA_DR_MP3_VSTORE(left + i, MA_DR_MP3_VADD(vl, vr));
93510 |             MA_DR_MP3_VSTORE(right + i, MA_DR_MP3_VSUB(vl, vr));
93511 |         }
93512 |     #ifdef __GNUC__
93513 |     if (__builtin_constant_p(n % 4 == 0) && n % 4 == 0)
93514 |         return;
93515 |     #endif
93516 |     }
93517 |     #endif
93518 |     for (; i < n; i++)
93519 |     {
93520 |         float a = left[i];
93521 |         float b = right[i];
93522 |         left[i] = a + b;
93523 |         right[i] = a - b;
93524 |     }
93525 | }
93526 | static void ma_dr_mp3_L3_intensity_stereo_band(float *left, int n, float kl, float kr)
93527 | {
93528 |     int i;
93529 |     for (i = 0; i < n; i++)
93530 |     {
93531 |         left[i + 576] = left[i]*kr;
93532 |         left[i] = left[i]*kl;
93533 |     }
93534 | }
93535 | static void ma_dr_mp3_L3_stereo_top_band(const float *right, const ma_uint8 *sfb, int nbands, int
> max_band[3])
93536 | {
93537 |     int i, k;
93538 |     max_band[0] = max_band[1] = max_band[2] = -1;
93539 |     for (i = 0; i < nbands; i++)
93540 |     {
93541 |         for (k = 0; k < sfb[i]; k += 2)
93542 |         {
93543 |             if (right[k] != 0 || right[k + 1] != 0)
93544 |             {
93545 |                 max_band[i % 3] = i;
93546 |                 break;
93547 |             }
93548 |         }
93549 |         right += sfb[i];
93550 |     }
93551 | }
93552 | static void ma_dr_mp3_L3_stereo_process(float *left, const ma_uint8 *ist_pos, const ma_uint8 *sfb,
> const ma_uint8 *hdr, int max_band[3], int mpeg2_sh)
93553 | {
93554 |     static const float g_pan[7*2] = { 0,1,0.21132487f,0.78867513f,0.36602540f,0.63397460f,0.5f,0.5f,
> 0.63397460f,0.36602540f,0.78867513f,0.21132487f,1,0 };
93555 |     unsigned i, max_pos = MA_DR_MP3_HDR_TEST_MPEG1(hdr) ? 7 : 64;
93556 |     for (i = 0; sfb[i]; i++)
93557 |     {
93558 |         unsigned ipos = ist_pos[i];
93559 |         if ((int)i > max_band[i % 3] && ipos < max_pos)
93560 |         {
93561 |             float kl, kr, s = MA_DR_MP3_HDR_TEST_MS_STEREO(hdr) ? 1.41421356f : 1;
93562 |             if (MA_DR_MP3_HDR_TEST_MPEG1(hdr))
93563 |             {
93564 |                 kl = g_pan[2*ipos];
93565 |                 kr = g_pan[2*ipos + 1];
93566 |             } else

```

```

93567 |         {
93568 |             kl = 1;
93569 |             kr = ma_dr_mp3_L3_ldexp_q2(1, (ipos + 1) >> 1 << mpeg2_sh);
93570 |             if (ipos & 1)
93571 |             {
93572 |                 kl = kr;
93573 |                 kr = 1;
93574 |             }
93575 |         }
93576 |         ma_dr_mp3_L3_intensity_stereo_band(left, sfb[i], kl*s, kr*s);
93577 |     } else if (MA_DR_MP3_HDR_TEST_MS_STEREO(hdr))
93578 |     {
93579 |         ma_dr_mp3_L3_midside_stereo(left, sfb[i]);
93580 |     }
93581 |     left += sfb[i];
93582 | }
93583 | }
93584 | static void ma_dr_mp3_L3_intensity_stereo(float *left, ma_uint8 *ist_pos, const ma_dr_mp3_L3_gr_info
> *gr, const ma_uint8 *hdr)
93585 | {
93586 |     int max_band[3], n_sfb = gr->n_long_sfb + gr->n_short_sfb;
93587 |     int i, max_blocks = gr->n_short_sfb ? 3 : 1;
93588 |     ma_dr_mp3_L3_stereo_top_band(left + 576, gr->sfbtab, n_sfb, max_band);
93589 |     if (gr->n_long_sfb)
93590 |     {
93591 |         max_band[0] = max_band[1] = max_band[2] = MA_DR_MP3_MAX(MA_DR_MP3_MAX(max_band[0],
> max_band[1]), max_band[2]);
93592 |     }
93593 |     for (i = 0; i < max_blocks; i++)
93594 |     {
93595 |         int default_pos = MA_DR_MP3_HDR_TEST_MPEG1(hdr) ? 3 : 0;
93596 |         int itop = n_sfb - max_blocks + i;
93597 |         int prev = itop - max_blocks;
93598 |         ist_pos[itop] = (ma_uint8)(max_band[i] >= prev ? default_pos : ist_pos[prev]);
93599 |     }
93600 |     ma_dr_mp3_L3_stereo_process(left, ist_pos, gr->sfbtab, hdr, max_band, gr[1].scalefac_compress &
> 1);
93601 | }
93602 | static void ma_dr_mp3_L3_reorder(float *grbuf, float *scratch, const ma_uint8 *sfb)
93603 | {
93604 |     int i, len;
93605 |     float *src = grbuf, *dst = scratch;
93606 |     for (; 0 != (len = *sfb); sfb += 3, src += 2*len)
93607 |     {
93608 |         for (i = 0; i < len; i++, src++)
93609 |         {
93610 |             *dst++ = src[0*len];
93611 |             *dst++ = src[1*len];
93612 |             *dst++ = src[2*len];
93613 |         }
93614 |     }
93615 |     MA_DR_MP3_COPY_MEMORY(grbuf, scratch, (dst - scratch)*sizeof(float));
93616 | }
93617 | static void ma_dr_mp3_L3_antialias(float *grbuf, int nbands)
93618 | {
93619 |     static const float g_aa[2][8] = {
93620 |
> {0.85749293f,0.88174200f,0.94962865f,0.98331459f,0.99551782f,0.99916056f,0.99989920f,0.99999316f},
93621 |
> {0.51449576f,0.47173197f,0.31337745f,0.18191320f,0.09457419f,0.04096558f,0.01419856f,0.00369997f}
93622 |     };
93623 |     for (; nbands > 0; nbands--, grbuf += 18)
93624 |     {
93625 |         int i = 0;
93626 |         #if MA_DR_MP3_HAVE_SIMD
93627 |             if (ma_dr_mp3_have_simd()) for (; i < 8; i += 4)
93628 |             {
93629 |                 ma_dr_mp3_f4 vu = MA_DR_MP3_VLD(grbuf + 18 + i);
93630 |                 ma_dr_mp3_f4 vd = MA_DR_MP3_VLD(grbuf + 14 - i);
93631 |                 ma_dr_mp3_f4 vc0 = MA_DR_MP3_VLD(g_aa[0] + i);
93632 |                 ma_dr_mp3_f4 vc1 = MA_DR_MP3_VLD(g_aa[1] + i);
93633 |                 vd = MA_DR_MP3_VREV(vd);
93634 |                 MA_DR_MP3_VSTORE(grbuf + 18 + i, MA_DR_MP3_VSUB(MA_DR_MP3_VMUL(vu, vc0),
> MA_DR_MP3_VMUL(vd, vc1)));
93635 |                 vd = MA_DR_MP3_VADD(MA_DR_MP3_VMUL(vu, vc1), MA_DR_MP3_VMUL(vd, vc0));
93636 |                 MA_DR_MP3_VSTORE(grbuf + 14 - i, MA_DR_MP3_VREV(vd));
93637 |             }
93638 |         #endif
93639 |         #ifndef MA_DR_MP3_ONLY_SIMD

```

```

93640 |         for(; i < 8; i++)
93641 |         {
93642 |             float u = grbuf[18 + i];
93643 |             float d = grbuf[17 - i];
93644 |             grbuf[18 + i] = u*g_aa[0][i] - d*g_aa[1][i];
93645 |             grbuf[17 - i] = u*g_aa[1][i] + d*g_aa[0][i];
93646 |         }
93647 |     #endif
93648 | }
93649 | }
93650 | static void ma_dr_mp3_L3_dct3_9(float *y)
93651 | {
93652 |     float s0, s1, s2, s3, s4, s5, s6, s7, s8, t0, t2, t4;
93653 |     s0 = y[0]; s2 = y[2]; s4 = y[4]; s6 = y[6]; s8 = y[8];
93654 |     t0 = s0 + s6*0.5f;
93655 |     s0 -= s6;
93656 |     t4 = (s4 + s2)*0.93969262f;
93657 |     t2 = (s8 + s2)*0.76604444f;
93658 |     s6 = (s4 - s8)*0.17364818f;
93659 |     s4 += s8 - s2;
93660 |     s2 = s0 - s4*0.5f;
93661 |     y[4] = s4 + s0;
93662 |     s8 = t0 - t2 + s6;
93663 |     s0 = t0 - t4 + t2;
93664 |     s4 = t0 + t4 - s6;
93665 |     s1 = y[1]; s3 = y[3]; s5 = y[5]; s7 = y[7];
93666 |     s3 *= 0.86602540f;
93667 |     t0 = (s5 + s1)*0.98480775f;
93668 |     t4 = (s5 - s7)*0.34202014f;
93669 |     t2 = (s1 + s7)*0.64278761f;
93670 |     s1 = (s1 - s5 - s7)*0.86602540f;
93671 |     s5 = t0 - s3 - t2;
93672 |     s7 = t4 - s3 - t0;
93673 |     s3 = t4 + s3 - t2;
93674 |     y[0] = s4 - s7;
93675 |     y[1] = s2 + s1;
93676 |     y[2] = s0 - s3;
93677 |     y[3] = s8 + s5;
93678 |     y[5] = s8 - s5;
93679 |     y[6] = s0 + s3;
93680 |     y[7] = s2 - s1;
93681 |     y[8] = s4 + s7;
93682 | }
93683 | static void ma_dr_mp3_L3_imdct36(float *grbuf, float *overlap, const float *window, int nbands)
93684 | {
93685 |     int i, j;
93686 |     static const float g_twid9[18] = {
93687 |         0.73727734f, 0.79335334f, 0.84339145f, 0.88701083f, 0.92387953f, 0.95371695f, 0.97629601f, 0.991444
> 86f, 0.99904822f, 0.67559021f, 0.60876143f, 0.53729961f, 0.46174861f, 0.38268343f, 0.30070580f, 0.21643961f,
> 0.13052619f, 0.04361938f
93688 |     };
93689 |     for (j = 0; j < nbands; j++, grbuf += 18, overlap += 9)
93690 |     {
93691 |         float co[9], si[9];
93692 |         co[0] = -grbuf[0];
93693 |         si[0] = grbuf[17];
93694 |         for (i = 0; i < 4; i++)
93695 |         {
93696 |             si[8 - 2*i] = grbuf[4*i + 1] - grbuf[4*i + 2];
93697 |             co[1 + 2*i] = grbuf[4*i + 1] + grbuf[4*i + 2];
93698 |             si[7 - 2*i] = grbuf[4*i + 4] - grbuf[4*i + 3];
93699 |             co[2 + 2*i] = -(grbuf[4*i + 3] + grbuf[4*i + 4]);
93700 |         }
93701 |         ma_dr_mp3_L3_dct3_9(co);
93702 |         ma_dr_mp3_L3_dct3_9(si);
93703 |         si[1] = -si[1];
93704 |         si[3] = -si[3];
93705 |         si[5] = -si[5];
93706 |         si[7] = -si[7];
93707 |         i = 0;
93708 |     #if MA_DR_MP3_HAVE_SIMD
93709 |         if (ma_dr_mp3_have_simd()) for (; i < 8; i += 4)
93710 |         {
93711 |             ma_dr_mp3_f4 vovl = MA_DR_MP3_VLD(overlap + i);
93712 |             ma_dr_mp3_f4 vc = MA_DR_MP3_VLD(co + i);
93713 |             ma_dr_mp3_f4 vs = MA_DR_MP3_VLD(si + i);
93714 |             ma_dr_mp3_f4 vr0 = MA_DR_MP3_VLD(g_twid9 + i);
93715 |             ma_dr_mp3_f4 vr1 = MA_DR_MP3_VLD(g_twid9 + 9 + i);
93716 |             ma_dr_mp3_f4 vw0 = MA_DR_MP3_VLD(window + i);

```



```

93717 |         ma_dr_mp3_f4 vw1 = MA_DR_MP3_VLD(window + 9 + i);
93718 |         ma_dr_mp3_f4 vsum = MA_DR_MP3_VADD(MA_DR_MP3_VMUL(vc, vr1), MA_DR_MP3_VMUL(vs, vr0));
93719 |         MA_DR_MP3_VSTORE(overlap + i, MA_DR_MP3_VSUB(MA_DR_MP3_VMUL(vc, vr0), MA_DR_MP3_VMUL(vs,
> vr1)));
93720 |         MA_DR_MP3_VSTORE(grbuf + i, MA_DR_MP3_VSUB(MA_DR_MP3_VMUL(vovl, vw0),
> MA_DR_MP3_VMUL(vsum, vw1)));
93721 |         vsum = MA_DR_MP3_VADD(MA_DR_MP3_VMUL(vovl, vw1), MA_DR_MP3_VMUL(vsum, vw0));
93722 |         MA_DR_MP3_VSTORE(grbuf + 14 - i, MA_DR_MP3_VREV(vsum));
93723 |     }
93724 | #endif
93725 |     for (; i < 9; i++)
93726 |     {
93727 |         float ovl = overlap[i];
93728 |         float sum = co[i]*g_twid9[9 + i] + si[i]*g_twid9[0 + i];
93729 |         overlap[i] = co[i]*g_twid9[0 + i] - si[i]*g_twid9[9 + i];
93730 |         grbuf[i] = ovl*window[0 + i] - sum*window[9 + i];
93731 |         grbuf[17 - i] = ovl*window[9 + i] + sum*window[0 + i];
93732 |     }
93733 | }
93734 | }
93735 | static void ma_dr_mp3_L3_idct3(float x0, float x1, float x2, float *dst)
93736 | {
93737 |     float m1 = x1*0.86602540f;
93738 |     float a1 = x0 - x2*0.5f;
93739 |     dst[1] = x0 + x2;
93740 |     dst[0] = a1 + m1;
93741 |     dst[2] = a1 - m1;
93742 | }
93743 | static void ma_dr_mp3_L3_imdct12(float *x, float *dst, float *overlap)
93744 | {
93745 |     static const float g_twid3[6] = { 0.79335334f, 0.92387953f, 0.99144486f,
> 0.60876143f, 0.38268343f, 0.13052619f };
93746 |     float co[3], si[3];
93747 |     int i;
93748 |     ma_dr_mp3_L3_idct3(-x[0], x[6] + x[3], x[12] + x[9], co);
93749 |     ma_dr_mp3_L3_idct3(x[15], x[12] - x[9], x[6] - x[3], si);
93750 |     si[1] = -si[1];
93751 |     for (i = 0; i < 3; i++)
93752 |     {
93753 |         float ovl = overlap[i];
93754 |         float sum = co[i]*g_twid3[3 + i] + si[i]*g_twid3[0 + i];
93755 |         overlap[i] = co[i]*g_twid3[0 + i] - si[i]*g_twid3[3 + i];
93756 |         dst[i] = ovl*g_twid3[2 - i] - sum*g_twid3[5 - i];
93757 |         dst[5 - i] = ovl*g_twid3[5 - i] + sum*g_twid3[2 - i];
93758 |     }
93759 | }
93760 | static void ma_dr_mp3_L3_imdct_short(float *grbuf, float *overlap, int nbands)
93761 | {
93762 |     for (; nbands > 0; nbands--, overlap += 9, grbuf += 18)
93763 |     {
93764 |         float tmp[18];
93765 |         MA_DR_MP3_COPY_MEMORY(tmp, grbuf, sizeof(tmp));
93766 |         MA_DR_MP3_COPY_MEMORY(grbuf, overlap, 6*sizeof(float));
93767 |         ma_dr_mp3_L3_imdct12(tmp, grbuf + 6, overlap + 6);
93768 |         ma_dr_mp3_L3_imdct12(tmp + 1, grbuf + 12, overlap + 6);
93769 |         ma_dr_mp3_L3_imdct12(tmp + 2, overlap, overlap + 6);
93770 |     }
93771 | }
93772 | static void ma_dr_mp3_L3_change_sign(float *grbuf)
93773 | {
93774 |     int b, i;
93775 |     for (b = 0, grbuf += 18; b < 32; b += 2, grbuf += 36)
93776 |         for (i = 1; i < 18; i += 2)
93777 |             grbuf[i] = -grbuf[i];
93778 | }
93779 | static void ma_dr_mp3_L3_imdct_gr(float *grbuf, float *overlap, unsigned block_type, unsigned
> n_long_bands)
93780 | {
93781 |     static const float g_mdct_window[2][18] = {
93782 |         { 0.99904822f, 0.99144486f, 0.97629601f, 0.95371695f, 0.92387953f, 0.88701083f, 0.84339145f, 0.7933
> 5334f, 0.73727734f, 0.04361938f, 0.13052619f, 0.21643961f, 0.30070580f, 0.38268343f, 0.46174861f, 0.53729961
> f, 0.60876143f, 0.67559021f },
93783 |         {
> 1, 1, 1, 1, 1, 1, 0.99144486f, 0.92387953f, 0.79335334f, 0, 0, 0, 0, 0, 0.13052619f, 0.38268343f, 0.60876143f }
93784 |     };
93785 |     if (n_long_bands)
93786 |     {
93787 |         ma_dr_mp3_L3_imdct36(grbuf, overlap, g_mdct_window[0], n_long_bands);
93788 |         grbuf += 18*n_long_bands;

```

```

93789 |         overlap += 9*n_long_bands;
93790 |     }
93791 |     if (block_type == MA_DR_MP3_SHORT_BLOCK_TYPE)
93792 |         ma_dr_mp3_L3_imdct_short(grbuf, overlap, 32 - n_long_bands);
93793 |     else
93794 |         ma_dr_mp3_L3_imdct36(grbuf, overlap, g_mdct_window[block_type == MA_DR_MP3_STOP_BLOCK_TYPE],
> 32 - n_long_bands);
93795 | }
93796 | static void ma_dr_mp3_L3_save_reservoir(ma_dr_mp3dec *h, ma_dr_mp3dec_scratch *s)
93797 | {
93798 |     int pos = (s->bs.pos + 7)/8u;
93799 |     int remains = s->bs.limit/8u - pos;
93800 |     if (remains > MA_DR_MP3_MAX_BITRESERVOIR_BYTES)
93801 |     {
93802 |         pos += remains - MA_DR_MP3_MAX_BITRESERVOIR_BYTES;
93803 |         remains = MA_DR_MP3_MAX_BITRESERVOIR_BYTES;
93804 |     }
93805 |     if (remains > 0)
93806 |     {
93807 |         MA_DR_MP3_MOVE_MEMORY(h->reserv_buf, s->maindata + pos, remains);
93808 |     }
93809 |     h->reserv = remains;
93810 | }
93811 | static int ma_dr_mp3_L3_restore_reservoir(ma_dr_mp3dec *h, ma_dr_mp3_bs *bs, ma_dr_mp3dec_scratch
> *s, int main_data_begin)
93812 | {
93813 |     int frame_bytes = (bs->limit - bs->pos)/8;
93814 |     int bytes_have = MA_DR_MP3_MIN(h->reserv, main_data_begin);
93815 |     MA_DR_MP3_COPY_MEMORY(s->maindata, h->reserv_buf + MA_DR_MP3_MAX(0, h->reserv -
> main_data_begin), MA_DR_MP3_MIN(h->reserv, main_data_begin));
93816 |     MA_DR_MP3_COPY_MEMORY(s->maindata + bytes_have, bs->buf + bs->pos/8, frame_bytes);
93817 |     ma_dr_mp3_bs_init(&s->bs, s->maindata, bytes_have + frame_bytes);
93818 |     return h->reserv >= main_data_begin;
93819 | }
93820 | static void ma_dr_mp3_L3_decode(ma_dr_mp3dec *h, ma_dr_mp3dec_scratch *s, ma_dr_mp3_L3_gr_info
> *gr_info, int nch)
93821 | {
93822 |     int ch;
93823 |     for (ch = 0; ch < nch; ch++)
93824 |     {
93825 |         int layer3gr_limit = s->bs.pos + gr_info[ch].part_23_length;
93826 |         ma_dr_mp3_L3_decode_scalefactors(h->header, s->ist_pos[ch], &s->bs, gr_info + ch, s->scf,
> ch);
93827 |         ma_dr_mp3_L3_huffman(s->grbuf[ch], &s->bs, gr_info + ch, s->scf, layer3gr_limit);
93828 |     }
93829 |     if (MA_DR_MP3_HDR_TEST_I_STEREO(h->header))
93830 |     {
93831 |         ma_dr_mp3_L3_intensity_stereo(s->grbuf[0], s->ist_pos[1], gr_info, h->header);
93832 |     } else if (MA_DR_MP3_HDR_IS_MS_STEREO(h->header))
93833 |     {
93834 |         ma_dr_mp3_L3_midside_stereo(s->grbuf[0], 576);
93835 |     }
93836 |     for (ch = 0; ch < nch; ch++, gr_info++)
93837 |     {
93838 |         int aa_bands = 31;
93839 |         int n_long_bands = (gr_info->mixed_block_flag ? 2 : 0) <<
> (int)(MA_DR_MP3_HDR_GET_MY_SAMPLE_RATE(h->header) == 2);
93840 |         if (gr_info->n_short_sfb)
93841 |         {
93842 |             aa_bands = n_long_bands - 1;
93843 |             ma_dr_mp3_L3_reorder(s->grbuf[ch] + n_long_bands*18, s->syn[0], gr_info->sfbtab +
> gr_info->n_long_sfb);
93844 |         }
93845 |         ma_dr_mp3_L3_antialias(s->grbuf[ch], aa_bands);
93846 |         ma_dr_mp3_L3_imdct_gr(s->grbuf[ch], h->mdct_overlap[ch], gr_info->block_type, n_long_bands);
93847 |         ma_dr_mp3_L3_change_sign(s->grbuf[ch]);
93848 |     }
93849 | }
93850 | static void ma_dr_mp3d_DCT_II(float *grbuf, int n)
93851 | {
93852 |     static const float g_sec[24] = {
93853 |         10.19000816f, 0.50060302f, 0.50241929f, 3.40760851f, 0.50547093f, 0.52249861f, 2.05778098f, 0.51544
> 732f, 0.56694406f, 1.48416460f, 0.53104258f, 0.64682180f, 1.16943991f, 0.55310392f, 0.78815460f, 0.97256821f
> , 0.58293498f, 1.06067765f, 0.83934963f, 0.62250412f, 1.72244716f, 0.74453628f, 0.67480832f, 5.10114861f
93854 |     };
93855 |     int i, k = 0;
93856 |     #if MA_DR_MP3_HAVE_SIMD
93857 |     if (ma_dr_mp3_have_simd()) for (; k < n; k += 4)
93858 |     {

```

```

93859 |     ma_dr_mp3_f4 t[4][8], *x;
93860 |     float *y = grbuf + k;
93861 |     for (x = t[0], i = 0; i < 8; i++, x++)
93862 |     {
93863 |         ma_dr_mp3_f4 x0 = MA_DR_MP3_VLD(&y[i*18]);
93864 |         ma_dr_mp3_f4 x1 = MA_DR_MP3_VLD(&y[(15 - i)*18]);
93865 |         ma_dr_mp3_f4 x2 = MA_DR_MP3_VLD(&y[(16 + i)*18]);
93866 |         ma_dr_mp3_f4 x3 = MA_DR_MP3_VLD(&y[(31 - i)*18]);
93867 |         ma_dr_mp3_f4 t0 = MA_DR_MP3_VADD(x0, x3);
93868 |         ma_dr_mp3_f4 t1 = MA_DR_MP3_VADD(x1, x2);
93869 |         ma_dr_mp3_f4 t2 = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(x1, x2), g_sec[3*i + 0]);
93870 |         ma_dr_mp3_f4 t3 = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(x0, x3), g_sec[3*i + 1]);
93871 |         x[0] = MA_DR_MP3_VADD(t0, t1);
93872 |         x[8] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(t0, t1), g_sec[3*i + 2]);
93873 |         x[16] = MA_DR_MP3_VADD(t3, t2);
93874 |         x[24] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(t3, t2), g_sec[3*i + 2]);
93875 |     }
93876 |     for (x = t[0], i = 0; i < 4; i++, x += 8)
93877 |     {
93878 |         ma_dr_mp3_f4 x0 = x[0], x1 = x[1], x2 = x[2], x3 = x[3], x4 = x[4], x5 = x[5], x6 =
> x[6], x7 = x[7], xt;
93879 |         xt = MA_DR_MP3_VSUB(x0, x7); x0 = MA_DR_MP3_VADD(x0, x7);
93880 |         x7 = MA_DR_MP3_VSUB(x1, x6); x1 = MA_DR_MP3_VADD(x1, x6);
93881 |         x6 = MA_DR_MP3_VSUB(x2, x5); x2 = MA_DR_MP3_VADD(x2, x5);
93882 |         x5 = MA_DR_MP3_VSUB(x3, x4); x3 = MA_DR_MP3_VADD(x3, x4);
93883 |         x4 = MA_DR_MP3_VSUB(x0, x3); x0 = MA_DR_MP3_VADD(x0, x3);
93884 |         x3 = MA_DR_MP3_VSUB(x1, x2); x1 = MA_DR_MP3_VADD(x1, x2);
93885 |         x[0] = MA_DR_MP3_VADD(x0, x1);
93886 |         x[4] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(x0, x1), 0.70710677f);
93887 |         x5 = MA_DR_MP3_VADD(x5, x6);
93888 |         x6 = MA_DR_MP3_VMUL_S(MA_DR_MP3_VADD(x6, x7), 0.70710677f);
93889 |         x7 = MA_DR_MP3_VADD(x7, xt);
93890 |         x3 = MA_DR_MP3_VMUL_S(MA_DR_MP3_VADD(x3, x4), 0.70710677f);
93891 |         x5 = MA_DR_MP3_VSUB(x5, MA_DR_MP3_VMUL_S(x7, 0.198912367f));
93892 |         x7 = MA_DR_MP3_VADD(x7, MA_DR_MP3_VMUL_S(x5, 0.382683432f));
93893 |         x5 = MA_DR_MP3_VSUB(x5, MA_DR_MP3_VMUL_S(x7, 0.198912367f));
93894 |         x0 = MA_DR_MP3_VSUB(xt, x6); xt = MA_DR_MP3_VADD(xt, x6);
93895 |         x[1] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VADD(xt, x7), 0.50979561f);
93896 |         x[2] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VADD(x4, x3), 0.54119611f);
93897 |         x[3] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(x0, x5), 0.60134488f);
93898 |         x[5] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VADD(x0, x5), 0.89997619f);
93899 |         x[6] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(x4, x3), 1.30656302f);
93900 |         x[7] = MA_DR_MP3_VMUL_S(MA_DR_MP3_VSUB(xt, x7), 2.56291556f);
93901 |     }
93902 |     if (k > n - 3)
93903 |     {
93904 |         #if MA_DR_MP3_HAVE_SSE
93905 |         #define MA_DR_MP3_VSAVE2(i, v) _mm_storel_pi((__m64 *) (void *) &y[i*18], v)
93906 |         #else
93907 |         #define MA_DR_MP3_VSAVE2(i, v) vst1_f32((float32_t *) &y[(i)*18], vget_low_f32(v))
93908 |         #endif
93909 |         for (i = 0; i < 7; i++, y += 4*18)
93910 |         {
93911 |             ma_dr_mp3_f4 s = MA_DR_MP3_VADD(t[3][i], t[3][i + 1]);
93912 |             MA_DR_MP3_VSAVE2(0, t[0][i]);
93913 |             MA_DR_MP3_VSAVE2(1, MA_DR_MP3_VADD(t[2][i], s));
93914 |             MA_DR_MP3_VSAVE2(2, MA_DR_MP3_VADD(t[1][i], t[1][i + 1]));
93915 |             MA_DR_MP3_VSAVE2(3, MA_DR_MP3_VADD(t[2][1 + i], s));
93916 |         }
93917 |         MA_DR_MP3_VSAVE2(0, t[0][7]);
93918 |         MA_DR_MP3_VSAVE2(1, MA_DR_MP3_VADD(t[2][7], t[3][7]));
93919 |         MA_DR_MP3_VSAVE2(2, t[1][7]);
93920 |         MA_DR_MP3_VSAVE2(3, t[3][7]);
93921 |     } else
93922 |     {
93923 |         #define MA_DR_MP3_VSAVE4(i, v) MA_DR_MP3_VSTORE(&y[(i)*18], v)
93924 |         for (i = 0; i < 7; i++, y += 4*18)
93925 |         {
93926 |             ma_dr_mp3_f4 s = MA_DR_MP3_VADD(t[3][i], t[3][i + 1]);
93927 |             MA_DR_MP3_VSAVE4(0, t[0][i]);
93928 |             MA_DR_MP3_VSAVE4(1, MA_DR_MP3_VADD(t[2][i], s));
93929 |             MA_DR_MP3_VSAVE4(2, MA_DR_MP3_VADD(t[1][i], t[1][i + 1]));
93930 |             MA_DR_MP3_VSAVE4(3, MA_DR_MP3_VADD(t[2][1 + i], s));
93931 |         }
93932 |         MA_DR_MP3_VSAVE4(0, t[0][7]);
93933 |         MA_DR_MP3_VSAVE4(1, MA_DR_MP3_VADD(t[2][7], t[3][7]));
93934 |         MA_DR_MP3_VSAVE4(2, t[1][7]);
93935 |         MA_DR_MP3_VSAVE4(3, t[3][7]);
93936 |     }

```

```

93937 |     } else
93938 | #endif
93939 | #ifdef MA_DR_MP3_ONLY_SIMD
93940 | {}
93941 | #else
93942 |     for (; k < n; k++)
93943 |     {
93944 |         float t[4][8], *x, *y = grbuf + k;
93945 |         for (x = t[0], i = 0; i < 8; i++, x++)
93946 |         {
93947 |             float x0 = y[i*18];
93948 |             float x1 = y[(15 - i)*18];
93949 |             float x2 = y[(16 + i)*18];
93950 |             float x3 = y[(31 - i)*18];
93951 |             float t0 = x0 + x3;
93952 |             float t1 = x1 + x2;
93953 |             float t2 = (x1 - x2)*g_sec[3*i + 0];
93954 |             float t3 = (x0 - x3)*g_sec[3*i + 1];
93955 |             x[0] = t0 + t1;
93956 |             x[8] = (t0 - t1)*g_sec[3*i + 2];
93957 |             x[16] = t3 + t2;
93958 |             x[24] = (t3 - t2)*g_sec[3*i + 2];
93959 |         }
93960 |         for (x = t[0], i = 0; i < 4; i++, x += 8)
93961 |         {
93962 |             float x0 = x[0], x1 = x[1], x2 = x[2], x3 = x[3], x4 = x[4], x5 = x[5], x6 = x[6], x7 =
> x[7], xt;
93963 |             xt = x0 - x7; x0 += x7;
93964 |             x7 = x1 - x6; x1 += x6;
93965 |             x6 = x2 - x5; x2 += x5;
93966 |             x5 = x3 - x4; x3 += x4;
93967 |             x4 = x0 - x3; x0 += x3;
93968 |             x3 = x1 - x2; x1 += x2;
93969 |             x[0] = x0 + x1;
93970 |             x[4] = (x0 - x1)*0.70710677f;
93971 |             x5 = x5 + x6;
93972 |             x6 = (x6 + x7)*0.70710677f;
93973 |             x7 = x7 + xt;
93974 |             x3 = (x3 + x4)*0.70710677f;
93975 |             x5 -= x7*0.198912367f;
93976 |             x7 += x5*0.382683432f;
93977 |             x5 -= x7*0.198912367f;
93978 |             x0 = xt - x6; xt += x6;
93979 |             x[1] = (xt + x7)*0.50979561f;
93980 |             x[2] = (x4 + x3)*0.54119611f;
93981 |             x[3] = (x0 - x5)*0.60134488f;
93982 |             x[5] = (x0 + x5)*0.89997619f;
93983 |             x[6] = (x4 - x3)*1.30656302f;
93984 |             x[7] = (xt - x7)*2.56291556f;
93985 |         }
93986 |         for (i = 0; i < 7; i++, y += 4*18)
93987 |         {
93988 |             y[0*18] = t[0][i];
93989 |             y[1*18] = t[2][i] + t[3][i] + t[3][i + 1];
93990 |             y[2*18] = t[1][i] + t[1][i + 1];
93991 |             y[3*18] = t[2][i + 1] + t[3][i] + t[3][i + 1];
93992 |         }
93993 |         y[0*18] = t[0][7];
93994 |         y[1*18] = t[2][7] + t[3][7];
93995 |         y[2*18] = t[1][7];
93996 |         y[3*18] = t[3][7];
93997 |     }
93998 | #endif
93999 | }
94000 | #ifndef MA_DR_MP3_FLOAT_OUTPUT
94001 | typedef ma_int16 ma_dr_mp3d_sample_t;
94002 | static ma_int16 ma_dr_mp3d_scale_pcm(float sample)
94003 | {
94004 |     ma_int16 s;
94005 |     #if MA_DR_MP3_HAVE_ARMV6
94006 |         ma_int32 s32 = (ma_int32)(sample + .5f);
94007 |         s32 -= (s32 < 0);
94008 |         s = (ma_int16)ma_dr_mp3_clip_int16_arm(s32);
94009 |     #else
94010 |         if (sample >= 32766.5f) return (ma_int16) 32767;
94011 |         if (sample <= -32767.5f) return (ma_int16)-32768;
94012 |         s = (ma_int16)(sample + .5f);
94013 |         s -= (s < 0);
94014 |     #endif

```

```

94015 |     return s;
94016 | }
94017 | #else
94018 | typedef float ma_dr_mp3d_sample_t;
94019 | static float ma_dr_mp3d_scale_pcm(float sample)
94020 | {
94021 |     return sample*(1.f/32768.f);
94022 | }
94023 | #endif
94024 | static void ma_dr_mp3d_synth_pair(ma_dr_mp3d_sample_t *pcm, int nch, const float *z)
94025 | {
94026 |     float a;
94027 |     a = (z[14*64] - z[0]) * 29;
94028 |     a += (z[1*64] + z[13*64]) * 213;
94029 |     a += (z[12*64] - z[2*64]) * 459;
94030 |     a += (z[3*64] + z[11*64]) * 2037;
94031 |     a += (z[10*64] - z[4*64]) * 5153;
94032 |     a += (z[5*64] + z[9*64]) * 6574;
94033 |     a += (z[8*64] - z[6*64]) * 37489;
94034 |     a += z[7*64] * 75038;
94035 |     pcm[0] = ma_dr_mp3d_scale_pcm(a);
94036 |     z += 2;
94037 |     a = z[14*64] * 104;
94038 |     a += z[12*64] * 1567;
94039 |     a += z[10*64] * 9727;
94040 |     a += z[8*64] * 64019;
94041 |     a += z[6*64] * -9975;
94042 |     a += z[4*64] * -45;
94043 |     a += z[2*64] * 146;
94044 |     a += z[0*64] * -5;
94045 |     pcm[16*nch] = ma_dr_mp3d_scale_pcm(a);
94046 | }
94047 | static void ma_dr_mp3d_synth(float *xl, ma_dr_mp3d_sample_t *dstl, int nch, float *lins)
94048 | {
94049 |     int i;
94050 |     float *xr = xl + 576*(nch - 1);
94051 |     ma_dr_mp3d_sample_t *dstr = dstl + (nch - 1);
94052 |     static const float g_win[] = {
94053 |         -1,26,-31,208,218,401,-519,2063,2000,4788,-5517,7134,5959,35640,-39336,74992,
94054 |         -1,24,-35,202,222,347,-581,2080,1952,4425,-5879,7640,5288,33791,-41176,74856,
94055 |         -1,21,-38,196,225,294,-645,2087,1893,4063,-6237,8092,4561,31947,-43006,74630,
94056 |         -1,19,-41,190,227,244,-711,2085,1822,3705,-6589,8492,3776,30112,-44821,74313,
94057 |         -1,17,-45,183,228,197,-779,2075,1739,3351,-6935,8840,2935,28289,-46617,73908,
94058 |         -1,16,-49,176,228,153,-848,2057,1644,3004,-7271,9139,2037,26482,-48390,73415,
94059 |         -2,14,-53,169,227,111,-919,2032,1535,2663,-7597,9389,1082,24694,-50137,72835,
94060 |         -2,13,-58,161,224,72,-991,2001,1414,2330,-7910,9592,70,22929,-51853,72169,
94061 |         -2,11,-63,154,221,36,-1064,1962,1280,2006,-8209,9750,-998,21189,-53534,71420,
94062 |         -2,10,-68,147,215,2,-1137,1919,1131,1692,-8491,9863,-2122,19478,-55178,70590,
94063 |         -3,9,-73,139,208,-29,-1210,1870,970,1388,-8755,9935,-3300,17799,-56778,69679,
94064 |         -3,8,-79,132,200,-57,-1283,1817,794,1095,-8998,9966,-4533,16155,-58333,68692,
94065 |         -4,7,-85,125,189,-83,-1356,1759,605,814,-9219,9959,-5818,14548,-59838,67629,
94066 |         -4,7,-91,117,177,-106,-1428,1698,402,545,-9416,9916,-7154,12980,-61289,66494,
94067 |         -5,6,-97,111,163,-127,-1498,1634,185,288,-9585,9838,-8540,11455,-62684,65290
94068 |     };
94069 |     float *zlin = lins + 15*64;
94070 |     const float *w = g_win;
94071 |     zlin[4*15] = xl[18*16];
94072 |     zlin[4*15 + 1] = xr[18*16];
94073 |     zlin[4*15 + 2] = xl[0];
94074 |     zlin[4*15 + 3] = xr[0];
94075 |     zlin[4*31] = xl[1 + 18*16];
94076 |     zlin[4*31 + 1] = xr[1 + 18*16];
94077 |     zlin[4*31 + 2] = xl[1];
94078 |     zlin[4*31 + 3] = xr[1];
94079 |     ma_dr_mp3d_synth_pair(dstr, nch, lins + 4*15 + 1);
94080 |     ma_dr_mp3d_synth_pair(dstr + 32*nch, nch, lins + 4*15 + 64 + 1);
94081 |     ma_dr_mp3d_synth_pair(dstl, nch, lins + 4*15);
94082 |     ma_dr_mp3d_synth_pair(dstl + 32*nch, nch, lins + 4*15 + 64);
94083 | #if MA_DR_MP3_HAVE_SIMD
94084 |     if (ma_dr_mp3_have_simd()) for (i = 14; i >= 0; i--)
94085 |     {
94086 | #define MA_DR_MP3_VLOAD(k) ma_dr_mp3_f4 w0 = MA_DR_MP3_VSET(*w++); ma_dr_mp3_f4 w1 =
> MA_DR_MP3_VSET(*w++); ma_dr_mp3_f4 vz = MA_DR_MP3_VLD(&zlin[4*i - 64*k]); ma_dr_mp3_f4 vy =
> MA_DR_MP3_VLD(&zlin[4*i - 64*(15 - k)]);
94087 | #define MA_DR_MP3_V0(k) { MA_DR_MP3_VLOAD(k) b = MA_DR_MP3_VADD(MA_DR_MP3_VMUL(vz,
> w1), MA_DR_MP3_VMUL(vy, w0)); a = MA_DR_MP3_VSUB(MA_DR_MP3_VMUL(vz, w0),
> MA_DR_MP3_VMUL(vy, w1)); }
94088 | #define MA_DR_MP3_V1(k) { MA_DR_MP3_VLOAD(k) b = MA_DR_MP3_VADD(b, MA_DR_MP3_VADD(MA_DR_MP3_VMUL(vz,
> w1), MA_DR_MP3_VMUL(vy, w0))); a = MA_DR_MP3_VADD(a, MA_DR_MP3_VSUB(MA_DR_MP3_VMUL(vz, w0),

```

```

> MA_DR_MP3_VMUL(vy, w1)); }
94089 | #define MA_DR_MP3_V2(k) { MA_DR_MP3_VLOAD(k) b = MA_DR_MP3_VADD(b, MA_DR_MP3_VADD(MA_DR_MP3_VMUL(vz,
> w1), MA_DR_MP3_VMUL(vy, w0)); a = MA_DR_MP3_VADD(a, MA_DR_MP3_VSUB(MA_DR_MP3_VMUL(vy, w1),
> MA_DR_MP3_VMUL(vz, w0)); }
94090 |     ma_dr_mp3_f4 a, b;
94091 |     zlin[4*i] = xl[18*(31 - i)];
94092 |     zlin[4*i + 1] = xr[18*(31 - i)];
94093 |     zlin[4*i + 2] = xl[1 + 18*(31 - i)];
94094 |     zlin[4*i + 3] = xr[1 + 18*(31 - i)];
94095 |     zlin[4*i + 64] = xl[1 + 18*(1 + i)];
94096 |     zlin[4*i + 64 + 1] = xr[1 + 18*(1 + i)];
94097 |     zlin[4*i - 64 + 2] = xl[18*(1 + i)];
94098 |     zlin[4*i - 64 + 3] = xr[18*(1 + i)];
94099 |     MA_DR_MP3_V0(0) MA_DR_MP3_V2(1) MA_DR_MP3_V1(2) MA_DR_MP3_V2(3) MA_DR_MP3_V1(4)
> MA_DR_MP3_V2(5) MA_DR_MP3_V1(6) MA_DR_MP3_V2(7)
94100 |     {
94101 | #ifndef MA_DR_MP3_FLOAT_OUTPUT
94102 | #if MA_DR_MP3_HAVE_SSE
94103 |     static const ma_dr_mp3_f4 g_max = { 32767.0f, 32767.0f, 32767.0f, 32767.0f };
94104 |     static const ma_dr_mp3_f4 g_min = { -32768.0f, -32768.0f, -32768.0f, -32768.0f };
94105 |     __m128i pcm8 = _mm_packs_epi32(_mm_cvtps_epi32(_mm_max_ps(_mm_min_ps(a, g_max), g_min)),
94106 |     _mm_cvtps_epi32(_mm_max_ps(_mm_min_ps(b, g_max),
> g_min)));
94107 |     dstr[(15 - i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 1);
94108 |     dstr[(17 + i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 5);
94109 |     dstl[(15 - i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 0);
94110 |     dstl[(17 + i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 4);
94111 |     dstr[(47 - i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 3);
94112 |     dstr[(49 + i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 7);
94113 |     dstl[(47 - i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 2);
94114 |     dstl[(49 + i)*nch] = (ma_int16)_mm_extract_epi16(pcm8, 6);
94115 | #else
94116 |     int16x4_t pcma, pcmb;
94117 |     a = MA_DR_MP3_VADD(a, MA_DR_MP3_VSET(0.5f));
94118 |     b = MA_DR_MP3_VADD(b, MA_DR_MP3_VSET(0.5f));
94119 |     pcma = vqmovn_s32(vqaddq_s32(vcvttq_s32_f32(a), vreinterpretq_s32_u32(vcltq_f32(a,
> MA_DR_MP3_VSET(0)))));
94120 |     pcmb = vqmovn_s32(vqaddq_s32(vcvttq_s32_f32(b), vreinterpretq_s32_u32(vcltq_f32(b,
> MA_DR_MP3_VSET(0)))));
94121 |     vst1_lane_s16(dstr + (15 - i)*nch, pcma, 1);
94122 |     vst1_lane_s16(dstr + (17 + i)*nch, pcmb, 1);
94123 |     vst1_lane_s16(dstl + (15 - i)*nch, pcma, 0);
94124 |     vst1_lane_s16(dstl + (17 + i)*nch, pcmb, 0);
94125 |     vst1_lane_s16(dstr + (47 - i)*nch, pcma, 3);
94126 |     vst1_lane_s16(dstr + (49 + i)*nch, pcmb, 3);
94127 |     vst1_lane_s16(dstl + (47 - i)*nch, pcma, 2);
94128 |     vst1_lane_s16(dstl + (49 + i)*nch, pcmb, 2);
94129 | #endif
94130 | #else
94131 |     #if MA_DR_MP3_HAVE_SSE
94132 |     static const ma_dr_mp3_f4 g_scale = { 1.0f/32768.0f, 1.0f/32768.0f, 1.0f/32768.0f,
> 1.0f/32768.0f };
94133 |     #else
94134 |     const ma_dr_mp3_f4 g_scale = vdupq_n_f32(1.0f/32768.0f);
94135 |     #endif
94136 |     a = MA_DR_MP3_VMUL(a, g_scale);
94137 |     b = MA_DR_MP3_VMUL(b, g_scale);
94138 | #if MA_DR_MP3_HAVE_SSE
94139 |     _mm_store_ss(dstr + (15 - i)*nch, _mm_shuffle_ps(a, a, _MM_SHUFFLE(1, 1, 1, 1)));
94140 |     _mm_store_ss(dstr + (17 + i)*nch, _mm_shuffle_ps(b, b, _MM_SHUFFLE(1, 1, 1, 1)));
94141 |     _mm_store_ss(dstl + (15 - i)*nch, _mm_shuffle_ps(a, a, _MM_SHUFFLE(0, 0, 0, 0)));
94142 |     _mm_store_ss(dstl + (17 + i)*nch, _mm_shuffle_ps(b, b, _MM_SHUFFLE(0, 0, 0, 0)));
94143 |     _mm_store_ss(dstr + (47 - i)*nch, _mm_shuffle_ps(a, a, _MM_SHUFFLE(3, 3, 3, 3)));
94144 |     _mm_store_ss(dstr + (49 + i)*nch, _mm_shuffle_ps(b, b, _MM_SHUFFLE(3, 3, 3, 3)));
94145 |     _mm_store_ss(dstl + (47 - i)*nch, _mm_shuffle_ps(a, a, _MM_SHUFFLE(2, 2, 2, 2)));
94146 |     _mm_store_ss(dstl + (49 + i)*nch, _mm_shuffle_ps(b, b, _MM_SHUFFLE(2, 2, 2, 2)));
94147 | #else
94148 |     vst1q_lane_f32(dstr + (15 - i)*nch, a, 1);
94149 |     vst1q_lane_f32(dstr + (17 + i)*nch, b, 1);
94150 |     vst1q_lane_f32(dstl + (15 - i)*nch, a, 0);
94151 |     vst1q_lane_f32(dstl + (17 + i)*nch, b, 0);
94152 |     vst1q_lane_f32(dstr + (47 - i)*nch, a, 3);
94153 |     vst1q_lane_f32(dstr + (49 + i)*nch, b, 3);
94154 |     vst1q_lane_f32(dstl + (47 - i)*nch, a, 2);
94155 |     vst1q_lane_f32(dstl + (49 + i)*nch, b, 2);
94156 | #endif
94157 | #endif
94158 |     }
94159 | } else

```

```

94160 | #endif
94161 | #ifdef MA_DR_MP3_ONLY_SIMD
94162 | {
94163 | #else
94164 |     for (i = 14; i >= 0; i--)
94165 |     {
94166 | #define MA_DR_MP3_LOAD(k) float w0 = *w++; float w1 = *w++; float *vz = &zlin[4*i - k*64]; float *vy
> = &zlin[4*i - (15 - k)*64];
94167 | #define MA_DR_MP3_S0(k) { int j; MA_DR_MP3_LOAD(k); for (j = 0; j < 4; j++) b[j] = vz[j]*w1 +
> vy[j]*w0, a[j] = vz[j]*w0 - vy[j]*w1; }
94168 | #define MA_DR_MP3_S1(k) { int j; MA_DR_MP3_LOAD(k); for (j = 0; j < 4; j++) b[j] += vz[j]*w1 +
> vy[j]*w0, a[j] += vz[j]*w0 - vy[j]*w1; }
94169 | #define MA_DR_MP3_S2(k) { int j; MA_DR_MP3_LOAD(k); for (j = 0; j < 4; j++) b[j] += vz[j]*w1 +
> vy[j]*w0, a[j] += vy[j]*w1 - vz[j]*w0; }
94170 |         float a[4], b[4];
94171 |         zlin[4*i] = xl[18*(31 - i)];
94172 |         zlin[4*i + 1] = xr[18*(31 - i)];
94173 |         zlin[4*i + 2] = xl[1 + 18*(31 - i)];
94174 |         zlin[4*i + 3] = xr[1 + 18*(31 - i)];
94175 |         zlin[4*(i + 16)] = xl[1 + 18*(1 + i)];
94176 |         zlin[4*(i + 16) + 1] = xr[1 + 18*(1 + i)];
94177 |         zlin[4*(i - 16) + 2] = xl[18*(1 + i)];
94178 |         zlin[4*(i - 16) + 3] = xr[18*(1 + i)];
94179 |         MA_DR_MP3_S0(0) MA_DR_MP3_S2(1) MA_DR_MP3_S1(2) MA_DR_MP3_S2(3) MA_DR_MP3_S1(4)
> MA_DR_MP3_S2(5) MA_DR_MP3_S1(6) MA_DR_MP3_S2(7)
94180 |         dstr[(15 - i)*nch] = ma_dr_mp3d_scale_pcm(a[1]);
94181 |         dstr[(17 + i)*nch] = ma_dr_mp3d_scale_pcm(b[1]);
94182 |         dstl[(15 - i)*nch] = ma_dr_mp3d_scale_pcm(a[0]);
94183 |         dstl[(17 + i)*nch] = ma_dr_mp3d_scale_pcm(b[0]);
94184 |         dstr[(47 - i)*nch] = ma_dr_mp3d_scale_pcm(a[3]);
94185 |         dstr[(49 + i)*nch] = ma_dr_mp3d_scale_pcm(b[3]);
94186 |         dstl[(47 - i)*nch] = ma_dr_mp3d_scale_pcm(a[2]);
94187 |         dstl[(49 + i)*nch] = ma_dr_mp3d_scale_pcm(b[2]);
94188 |     }
94189 | #endif
94190 | }
94191 | static void ma_dr_mp3d_synth_granule(float *qmf_state, float *grbuf, int nbands, int nch,
> ma_dr_mp3d_sample_t *pcm, float *lins)
94192 | {
94193 |     int i;
94194 |     for (i = 0; i < nch; i++)
94195 |     {
94196 |         ma_dr_mp3d_DCT_II(grbuf + 576*i, nbands);
94197 |     }
94198 |     MA_DR_MP3_COPY_MEMORY(lins, qmf_state, sizeof(float)*15*64);
94199 |     for (i = 0; i < nbands; i += 2)
94200 |     {
94201 |         ma_dr_mp3d_synth(grbuf + i, pcm + 32*nch*i, nch, lins + i*64);
94202 |     }
94203 | #ifndef MA_DR_MP3_NONSTANDARD_BUT_LOGICAL
94204 |     if (nch == 1)
94205 |     {
94206 |         for (i = 0; i < 15*64; i += 2)
94207 |         {
94208 |             qmf_state[i] = lins[nbands*64 + i];
94209 |         }
94210 |     } else
94211 | #endif
94212 |     {
94213 |         MA_DR_MP3_COPY_MEMORY(qmf_state, lins + nbands*64, sizeof(float)*15*64);
94214 |     }
94215 | }
94216 | static int ma_dr_mp3d_match_frame(const ma_uint8 *hdr, int mp3_bytes, int frame_bytes)
94217 | {
94218 |     int i, nmatch;
94219 |     for (i = 0, nmatch = 0; nmatch < MA_DR_MP3_MAX_FRAME_SYNC_MATCHES; nmatch++)
94220 |     {
94221 |         i += ma_dr_mp3_hdr_frame_bytes(hdr + i, frame_bytes) + ma_dr_mp3_hdr_padding(hdr + i);
94222 |         if (i + MA_DR_MP3_HDR_SIZE > mp3_bytes)
94223 |             return nmatch > 0;
94224 |         if (!ma_dr_mp3_hdr_compare(hdr, hdr + i))
94225 |             return 0;
94226 |     }
94227 |     return 1;
94228 | }
94229 | static int ma_dr_mp3d_find_frame(const ma_uint8 *mp3, int mp3_bytes, int *free_format_bytes, int
> *ptr_frame_bytes)
94230 | {
94231 |     int i, k;

```



```

94232 |     for (i = 0; i < mp3_bytes - MA_DR_MP3_HDR_SIZE; i++, mp3++)
94233 |     {
94234 |         if (ma_dr_mp3_hdr_valid(mp3))
94235 |         {
94236 |             int frame_bytes = ma_dr_mp3_hdr_frame_bytes(mp3, *free_format_bytes);
94237 |             int frame_and_padding = frame_bytes + ma_dr_mp3_hdr_padding(mp3);
94238 |             for (k = MA_DR_MP3_HDR_SIZE; !frame_bytes && k < MA_DR_MP3_MAX_FREE_FORMAT_FRAME_SIZE &&
> i + 2*k < mp3_bytes - MA_DR_MP3_HDR_SIZE; k++)
94239 |             {
94240 |                 if (ma_dr_mp3_hdr_compare(mp3, mp3 + k))
94241 |                 {
94242 |                     int fb = k - ma_dr_mp3_hdr_padding(mp3);
94243 |                     int nextfb = fb + ma_dr_mp3_hdr_padding(mp3 + k);
94244 |                     if (i + k + nextfb + MA_DR_MP3_HDR_SIZE > mp3_bytes ||
> !ma_dr_mp3_hdr_compare(mp3, mp3 + k + nextfb))
94245 |                         continue;
94246 |                     frame_and_padding = k;
94247 |                     frame_bytes = fb;
94248 |                     *free_format_bytes = fb;
94249 |                 }
94250 |             }
94251 |             if ((frame_bytes && i + frame_and_padding <= mp3_bytes &&
94252 |                 ma_dr_mp3d_match_frame(mp3, mp3_bytes - i, frame_bytes)) ||
94253 |                 (!i && frame_and_padding == mp3_bytes))
94254 |             {
94255 |                 *ptr_frame_bytes = frame_and_padding;
94256 |                 return i;
94257 |             }
94258 |             *free_format_bytes = 0;
94259 |         }
94260 |     }
94261 |     *ptr_frame_bytes = 0;
94262 |     return mp3_bytes;
94263 | }
94264 | MA_API void ma_dr_mp3dec_init(ma_dr_mp3dec *dec)
94265 | {
94266 |     dec->header[0] = 0;
94267 | }
94268 | MA_API int ma_dr_mp3dec_decode_frame(ma_dr_mp3dec *dec, const ma_uint8 *mp3, int mp3_bytes, void
> *pcm, ma_dr_mp3dec_frame_info *info)
94269 | {
94270 |     int i = 0, igr, frame_size = 0, success = 1;
94271 |     const ma_uint8 *hdr;
94272 |     ma_dr_mp3_bs bs_frame[1];
94273 |     if (mp3_bytes > 4 && dec->header[0] == 0xff && ma_dr_mp3_hdr_compare(dec->header, mp3))
94274 |     {
94275 |         frame_size = ma_dr_mp3_hdr_frame_bytes(mp3, dec->free_format_bytes) +
> ma_dr_mp3_hdr_padding(mp3);
94276 |         if (frame_size != mp3_bytes && (frame_size + MA_DR_MP3_HDR_SIZE > mp3_bytes ||
> !ma_dr_mp3_hdr_compare(mp3, mp3 + frame_size)))
94277 |         {
94278 |             frame_size = 0;
94279 |         }
94280 |     }
94281 |     if (!frame_size)
94282 |     {
94283 |         MA_DR_MP3_ZERO_MEMORY(dec, sizeof(ma_dr_mp3dec));
94284 |         i = ma_dr_mp3d_find_frame(mp3, mp3_bytes, &dec->free_format_bytes, &frame_size);
94285 |         if (!frame_size || i + frame_size > mp3_bytes)
94286 |         {
94287 |             info->frame_bytes = i;
94288 |             return 0;
94289 |         }
94290 |     }
94291 |     hdr = mp3 + i;
94292 |     MA_DR_MP3_COPY_MEMORY(dec->header, hdr, MA_DR_MP3_HDR_SIZE);
94293 |     info->frame_bytes = i + frame_size;
94294 |     info->channels = MA_DR_MP3_HDR_IS_MONO(hdr) ? 1 : 2;
94295 |     info->sample_rate = ma_dr_mp3_hdr_sample_rate_hz(hdr);
94296 |     info->layer = 4 - MA_DR_MP3_HDR_GET_LAYER(hdr);
94297 |     info->bitrate_kbps = ma_dr_mp3_hdr_bitrate_kbps(hdr);
94298 |     ma_dr_mp3_bs_init(bs_frame, hdr + MA_DR_MP3_HDR_SIZE, frame_size - MA_DR_MP3_HDR_SIZE);
94299 |     if (MA_DR_MP3_HDR_IS_CRC(hdr))
94300 |     {
94301 |         ma_dr_mp3_bs_get_bits(bs_frame, 16);
94302 |     }
94303 |     if (info->layer == 3)
94304 |     {
94305 |         int main_data_begin = ma_dr_mp3_L3_read_side_info(bs_frame, dec->scratch.gr_info, hdr);

```

```

94306 |         if (main_data_begin < 0 || bs_frame->pos > bs_frame->limit)
94307 |         {
94308 |             ma_dr_mp3dec_init(dec);
94309 |             return 0;
94310 |         }
94311 |         success = ma_dr_mp3_L3_restore_reservoir(dec, bs_frame, &dec->scratch, main_data_begin);
94312 |         if (success && pcm != NULL)
94313 |         {
94314 |             for (igr = 0; igr < (MA_DR_MP3_HDR_TEST_MPEG1(hdr) ? 2 : 1); igr++, pcm =
> MA_DR_MP3_OFFSET_PTR(pcm, sizeof(ma_dr_mp3d_sample_t)*576*info->channels))
94315 |             {
94316 |                 MA_DR_MP3_ZERO_MEMORY(dec->scratch.grbuf[0], 576*2*sizeof(float));
94317 |                 ma_dr_mp3_L3_decode(dec, &dec->scratch, dec->scratch.gr_info + igr*info->channels,
> info->channels);
94318 |                 ma_dr_mp3d_synth_granule(dec->qmf_state, dec->scratch.grbuf[0], 18, info->channels,
> (ma_dr_mp3d_sample_t*)pcm, dec->scratch.syn[0]);
94319 |             }
94320 |         }
94321 |         ma_dr_mp3_L3_save_reservoir(dec, &dec->scratch);
94322 |     } else
94323 |     {
94324 | #ifdef MA_DR_MP3_ONLY_MP3
94325 |         return 0;
94326 | #else
94327 |         ma_dr_mp3_L12_scale_info sci[1];
94328 |         if (pcm == NULL) {
94329 |             return ma_dr_mp3_hdr_frame_samples(hdr);
94330 |         }
94331 |         ma_dr_mp3_L12_read_scale_info(hdr, bs_frame, sci);
94332 |         MA_DR_MP3_ZERO_MEMORY(dec->scratch.grbuf[0], 576*2*sizeof(float));
94333 |         for (i = 0, igr = 0; igr < 3; igr++)
94334 |         {
94335 |             if (12 == (i += ma_dr_mp3_L12_dequantize_granule(dec->scratch.grbuf[0] + i, bs_frame,
> sci, info->layer | 1)))
94336 |             {
94337 |                 i = 0;
94338 |                 ma_dr_mp3_L12_apply_scf_384(sci, sci->scf + igr, dec->scratch.grbuf[0]);
94339 |                 ma_dr_mp3d_synth_granule(dec->qmf_state, dec->scratch.grbuf[0], 12, info->channels,
> (ma_dr_mp3d_sample_t*)pcm, dec->scratch.syn[0]);
94340 |                 MA_DR_MP3_ZERO_MEMORY(dec->scratch.grbuf[0], 576*2*sizeof(float));
94341 |                 pcm = MA_DR_MP3_OFFSET_PTR(pcm, sizeof(ma_dr_mp3d_sample_t)*384*info->channels);
94342 |             }
94343 |             if (bs_frame->pos > bs_frame->limit)
94344 |             {
94345 |                 ma_dr_mp3dec_init(dec);
94346 |                 return 0;
94347 |             }
94348 |         }
94349 | #endif
94350 |     }
94351 |     return success*ma_dr_mp3_hdr_frame_samples(dec->header);
94352 | }
94353 | MA_API void ma_dr_mp3dec_f32_to_s16(const float *in, ma_int16 *out, size_t num_samples)
94354 | {
94355 |     size_t i = 0;
94356 | #if MA_DR_MP3_HAVE_SIMD
94357 |     size_t aligned_count = num_samples & ~7;
94358 |     for(; i < aligned_count; i+=8)
94359 |     {
94360 |         ma_dr_mp3_f4 scale = MA_DR_MP3_VSET(32768.0f);
94361 |         ma_dr_mp3_f4 a = MA_DR_MP3_VMUL(MA_DR_MP3_VLD(&in[i ]), scale);
94362 |         ma_dr_mp3_f4 b = MA_DR_MP3_VMUL(MA_DR_MP3_VLD(&in[i+4]), scale);
94363 | #if MA_DR_MP3_HAVE_SSE
94364 |         ma_dr_mp3_f4 s16max = MA_DR_MP3_VSET( 32767.0f);
94365 |         ma_dr_mp3_f4 s16min = MA_DR_MP3_VSET(-32768.0f);
94366 |         __m128i pcm8 = _mm_packs_epi32(_mm_cvtps_epi32(_mm_max_ps(_mm_min_ps(a, s16max), s16min)),
> _mm_cvtps_epi32(_mm_max_ps(_mm_min_ps(b, s16max), s16min)));
94367 |         out[i ] = (ma_int16)_mm_extract_epi16(pcm8, 0);
94368 |         out[i+1] = (ma_int16)_mm_extract_epi16(pcm8, 1);
94369 |         out[i+2] = (ma_int16)_mm_extract_epi16(pcm8, 2);
94370 |         out[i+3] = (ma_int16)_mm_extract_epi16(pcm8, 3);
94371 |         out[i+4] = (ma_int16)_mm_extract_epi16(pcm8, 4);
94372 |         out[i+5] = (ma_int16)_mm_extract_epi16(pcm8, 5);
94373 |         out[i+6] = (ma_int16)_mm_extract_epi16(pcm8, 6);
94374 |         out[i+7] = (ma_int16)_mm_extract_epi16(pcm8, 7);
94375 |     }
94376 | #else
94377 |         int16x4_t pcma, pcmb;
94378 |         a = MA_DR_MP3_VADD(a, MA_DR_MP3_VSET(0.5f));
94379 |         b = MA_DR_MP3_VADD(b, MA_DR_MP3_VSET(0.5f));

```

```

94380 |         pcma = vqmovn_s32(vqaddq_s32(vcvttq_s32_f32(a), vreinterpretq_s32_u32(vclttq_f32(a,
> MA_DR_MP3_VSET(0)))));
94381 |         pcmb = vqmovn_s32(vqaddq_s32(vcvttq_s32_f32(b), vreinterpretq_s32_u32(vclttq_f32(b,
> MA_DR_MP3_VSET(0)))));
94382 |         vst1_lane_s16(out+i, pcma, 0);
94383 |         vst1_lane_s16(out+i+1, pcma, 1);
94384 |         vst1_lane_s16(out+i+2, pcma, 2);
94385 |         vst1_lane_s16(out+i+3, pcma, 3);
94386 |         vst1_lane_s16(out+i+4, pcmb, 0);
94387 |         vst1_lane_s16(out+i+5, pcmb, 1);
94388 |         vst1_lane_s16(out+i+6, pcmb, 2);
94389 |         vst1_lane_s16(out+i+7, pcmb, 3);
94390 |     #endif
94391 | }
94392 | #endif
94393 |     for(; i < num_samples; i++)
94394 |     {
94395 |         float sample = in[i] * 32768.0f;
94396 |         if (sample >= 32766.5f)
94397 |             out[i] = (ma_int16) 32767;
94398 |         else if (sample <= -32767.5f)
94399 |             out[i] = (ma_int16)-32768;
94400 |         else
94401 |         {
94402 |             short s = (ma_int16)(sample + .5f);
94403 |             s -= (s < 0);
94404 |             out[i] = s;
94405 |         }
94406 |     }
94407 | }
94408 | #ifndef MA_DR_MP3_SEEK_LEADING_MP3_FRAMES
94409 | #define MA_DR_MP3_SEEK_LEADING_MP3_FRAMES 2
94410 | #endif
94411 | #define MA_DR_MP3_MIN_DATA_CHUNK_SIZE 16384
94412 | #ifndef MA_DR_MP3_DATA_CHUNK_SIZE
94413 | #define MA_DR_MP3_DATA_CHUNK_SIZE (MA_DR_MP3_MIN_DATA_CHUNK_SIZE*4)
94414 | #endif
94415 | #define MA_DR_MP3_COUNTOF(x) (sizeof(x) / sizeof(x[0]))
94416 | #define MA_DR_MP3_CLAMP(x, lo, hi) (MA_DR_MP3_MAX(lo, MA_DR_MP3_MIN(x, hi)))
94417 | #ifndef MA_DR_MP3_PI_D
94418 | #define MA_DR_MP3_PI_D 3.14159265358979323846264
94419 | #endif
94420 | #define MA_DR_MP3_DEFAULT_RESAMPLER_LPF_ORDER 2
94421 | static MA_INLINE float ma_dr_mp3_mix_f32(float x, float y, float a)
94422 | {
94423 |     return x*(1-a) + y*a;
94424 | }
94425 | static MA_INLINE float ma_dr_mp3_mix_f32_fast(float x, float y, float a)
94426 | {
94427 |     float r0 = (y - x);
94428 |     float r1 = r0*a;
94429 |     return x + r1;
94430 | }
94431 | static MA_INLINE ma_uint32 ma_dr_mp3_gcf_u32(ma_uint32 a, ma_uint32 b)
94432 | {
94433 |     for (;;) {
94434 |         if (b == 0) {
94435 |             break;
94436 |         } else {
94437 |             ma_uint32 t = a;
94438 |             a = b;
94439 |             b = t % a;
94440 |         }
94441 |     }
94442 |     return a;
94443 | }
94444 | static void* ma_dr_mp3__malloc_default(size_t sz, void* pUserData)
94445 | {
94446 |     (void)pUserData;
94447 |     return MA_DR_MP3_MALLOC(sz);
94448 | }
94449 | static void* ma_dr_mp3__realloc_default(void* p, size_t sz, void* pUserData)
94450 | {
94451 |     (void)pUserData;
94452 |     return MA_DR_MP3_REALLOC(p, sz);
94453 | }
94454 | static void ma_dr_mp3__free_default(void* p, void* pUserData)
94455 | {
94456 |     (void)pUserData;

```

```

94457 |     MA_DR_MP3_FREE(p);
94458 | }
94459 | static void* ma_dr_mp3__malloc_from_callbacks(size_t sz, const ma_allocation_callbacks*
> pAllocationCallbacks)
94460 | {
94461 |     if (pAllocationCallbacks == NULL) {
94462 |         return NULL;
94463 |     }
94464 |     if (pAllocationCallbacks->onMalloc != NULL) {
94465 |         return pAllocationCallbacks->onMalloc(sz, pAllocationCallbacks->pUserData);
94466 |     }
94467 |     if (pAllocationCallbacks->onRealloc != NULL) {
94468 |         return pAllocationCallbacks->onRealloc(NULL, sz, pAllocationCallbacks->pUserData);
94469 |     }
94470 |     return NULL;
94471 | }
94472 | static void* ma_dr_mp3__realloc_from_callbacks(void* p, size_t szNew, size_t szOld, const
> ma_allocation_callbacks* pAllocationCallbacks)
94473 | {
94474 |     if (pAllocationCallbacks == NULL) {
94475 |         return NULL;
94476 |     }
94477 |     if (pAllocationCallbacks->onRealloc != NULL) {
94478 |         return pAllocationCallbacks->onRealloc(p, szNew, pAllocationCallbacks->pUserData);
94479 |     }
94480 |     if (pAllocationCallbacks->onMalloc != NULL && pAllocationCallbacks->onFree != NULL) {
94481 |         void* p2;
94482 |         p2 = pAllocationCallbacks->onMalloc(szNew, pAllocationCallbacks->pUserData);
94483 |         if (p2 == NULL) {
94484 |             return NULL;
94485 |         }
94486 |         if (p != NULL) {
94487 |             MA_DR_MP3_COPY_MEMORY(p2, p, szOld);
94488 |             pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
94489 |         }
94490 |         return p2;
94491 |     }
94492 |     return NULL;
94493 | }
94494 | static void ma_dr_mp3__free_from_callbacks(void* p, const ma_allocation_callbacks*
> pAllocationCallbacks)
94495 | {
94496 |     if (p == NULL || pAllocationCallbacks == NULL) {
94497 |         return;
94498 |     }
94499 |     if (pAllocationCallbacks->onFree != NULL) {
94500 |         pAllocationCallbacks->onFree(p, pAllocationCallbacks->pUserData);
94501 |     }
94502 | }
94503 | static ma_allocation_callbacks ma_dr_mp3_copy_allocation_callbacks_or_defaults(const
> ma_allocation_callbacks* pAllocationCallbacks)
94504 | {
94505 |     if (pAllocationCallbacks != NULL) {
94506 |         return *pAllocationCallbacks;
94507 |     } else {
94508 |         ma_allocation_callbacks allocationCallbacks;
94509 |         allocationCallbacks.pUserData = NULL;
94510 |         allocationCallbacks.onMalloc = ma_dr_mp3__malloc_default;
94511 |         allocationCallbacks.onRealloc = ma_dr_mp3__realloc_default;
94512 |         allocationCallbacks.onFree = ma_dr_mp3__free_default;
94513 |         return allocationCallbacks;
94514 |     }
94515 | }
94516 | static size_t ma_dr_mp3__on_read(ma_dr_mp3* pMP3, void* pBufferOut, size_t bytesToRead)
94517 | {
94518 |     size_t bytesRead;
94519 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94520 |     MA_DR_MP3_ASSERT(pMP3->onRead != NULL);
94521 |     if (bytesToRead == 0) {
94522 |         return 0;
94523 |     }
94524 |     bytesRead = pMP3->onRead(pMP3->pUserData, pBufferOut, bytesToRead);
94525 |     pMP3->streamCursor += bytesRead;
94526 |     return bytesRead;
94527 | }
94528 | static size_t ma_dr_mp3__on_read_clamped(ma_dr_mp3* pMP3, void* pBufferOut, size_t bytesToRead)
94529 | {
94530 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94531 |     MA_DR_MP3_ASSERT(pMP3->onRead != NULL);

```

```

94532 |     if (pMP3->streamLength == MA_UINT64_MAX) {
94533 |         return ma_dr_mp3__on_read(pMP3, pBufferOut, bytesToRead);
94534 |     } else {
94535 |         ma_uint64 bytesRemaining;
94536 |         bytesRemaining = (pMP3->streamLength - pMP3->streamCursor);
94537 |         if (bytesToRead > bytesRemaining) {
94538 |             bytesToRead = (size_t)bytesRemaining;
94539 |         }
94540 |         return ma_dr_mp3__on_read(pMP3, pBufferOut, bytesToRead);
94541 |     }
94542 | }
94543 | static ma_bool32 ma_dr_mp3__on_seek(ma_dr_mp3* pMP3, int offset, ma_dr_mp3_seek_origin origin)
94544 | {
94545 |     MA_DR_MP3_ASSERT(offset >= 0);
94546 |     MA_DR_MP3_ASSERT(origin == MA_DR_MP3_SEEK_SET || origin == MA_DR_MP3_SEEK_CUR);
94547 |     if (!pMP3->onSeek(pMP3->pUserData, offset, origin)) {
94548 |         return MA_FALSE;
94549 |     }
94550 |     if (origin == MA_DR_MP3_SEEK_SET) {
94551 |         pMP3->streamCursor = (ma_uint64)offset;
94552 |     } else {
94553 |         pMP3->streamCursor += offset;
94554 |     }
94555 |     return MA_TRUE;
94556 | }
94557 | static ma_bool32 ma_dr_mp3__on_seek_64(ma_dr_mp3* pMP3, ma_uint64 offset, ma_dr_mp3_seek_origin
> origin)
94558 | {
94559 |     if (offset <= 0x7FFFFFFF) {
94560 |         return ma_dr_mp3__on_seek(pMP3, (int)offset, origin);
94561 |     }
94562 |     if (!ma_dr_mp3__on_seek(pMP3, 0x7FFFFFFF, MA_DR_MP3_SEEK_SET)) {
94563 |         return MA_FALSE;
94564 |     }
94565 |     offset -= 0x7FFFFFFF;
94566 |     while (offset > 0) {
94567 |         if (offset <= 0x7FFFFFFF) {
94568 |             if (!ma_dr_mp3__on_seek(pMP3, (int)offset, MA_DR_MP3_SEEK_CUR)) {
94569 |                 return MA_FALSE;
94570 |             }
94571 |             offset = 0;
94572 |         } else {
94573 |             if (!ma_dr_mp3__on_seek(pMP3, 0x7FFFFFFF, MA_DR_MP3_SEEK_CUR)) {
94574 |                 return MA_FALSE;
94575 |             }
94576 |             offset -= 0x7FFFFFFF;
94577 |         }
94578 |     }
94579 |     return MA_TRUE;
94580 | }
94581 | static void ma_dr_mp3__on_meta(ma_dr_mp3* pMP3, ma_dr_mp3_metadata_type type, const void* pRawData,
> size_t rawDataSize)
94582 | {
94583 |     if (pMP3->onMeta) {
94584 |         ma_dr_mp3_metadata metadata;
94585 |         MA_DR_MP3_ZERO_OBJECT(&metadata);
94586 |         metadata.type = type;
94587 |         metadata.pRawData = pRawData;
94588 |         metadata.rawDataSize = rawDataSize;
94589 |         pMP3->onMeta(pMP3->pUserDataMeta, &metadata);
94590 |     }
94591 | }
94592 | static ma_uint32 ma_dr_mp3_decode_next_frame_ex_callbacks(ma_dr_mp3* pMP3, ma_dr_mp3d_sample_t*
> pPCMFrames, ma_dr_mp3dec_frame_info* pMP3FrameInfo, const ma_uint8** ppMP3FrameData)
94593 | {
94594 |     ma_uint32 pcmFramesRead = 0;
94595 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94596 |     MA_DR_MP3_ASSERT(pMP3->onRead != NULL);
94597 |     if (pMP3->atEnd) {
94598 |         return 0;
94599 |     }
94600 |     for (;;) {
94601 |         ma_dr_mp3dec_frame_info info;
94602 |         if (pMP3->dataSize < MA_DR_MP3_MIN_DATA_CHUNK_SIZE) {
94603 |             size_t bytesRead;
94604 |             if (pMP3->pData != NULL) {
94605 |                 MA_DR_MP3_MOVE_MEMORY(pMP3->pData, pMP3->pData + pMP3->dataConsumed,
> pMP3->dataSize);
94606 |             }

```

```

94607 |         pMP3->dataConsumed = 0;
94608 |         if (pMP3->dataCapacity < MA_DR_MP3_DATA_CHUNK_SIZE) {
94609 |             ma_uint8* pNewData;
94610 |             size_t newDataCap;
94611 |             newDataCap = MA_DR_MP3_DATA_CHUNK_SIZE;
94612 |             pNewData = (ma_uint8*)ma_dr_mp3__realloc_from_callbacks(pMP3->pData, newDataCap,
> pMP3->dataCapacity, &pMP3->allocationCallbacks);
94613 |             if (pNewData == NULL) {
94614 |                 return 0;
94615 |             }
94616 |             pMP3->pData = pNewData;
94617 |             pMP3->dataCapacity = newDataCap;
94618 |         }
94619 |         bytesRead = ma_dr_mp3__on_read_clamped(pMP3, pMP3->pData + pMP3->dataSize,
> (pMP3->dataCapacity - pMP3->dataSize));
94620 |         if (bytesRead == 0) {
94621 |             if (pMP3->dataSize == 0) {
94622 |                 pMP3->atEnd = MA_TRUE;
94623 |                 return 0;
94624 |             }
94625 |         }
94626 |         pMP3->dataSize += bytesRead;
94627 |     }
94628 |     if (pMP3->dataSize > INT_MAX) {
94629 |         pMP3->atEnd = MA_TRUE;
94630 |         return 0;
94631 |     }
94632 |     MA_DR_MP3_ASSERT(pMP3->pData != NULL);
94633 |     MA_DR_MP3_ASSERT(pMP3->dataCapacity > 0);
94634 |     if (pMP3->pData == NULL) {
94635 |         return 0;
94636 |     }
94637 |     pcmFramesRead = ma_dr_mp3dec_decode_frame(&pMP3->decoder, pMP3->pData + pMP3->dataConsumed,
> (int)pMP3->dataSize, pPCMFrames, &info);
94638 |     pMP3->dataConsumed += (size_t)info.frame_bytes;
94639 |     pMP3->dataSize -= (size_t)info.frame_bytes;
94640 |     if (pcmFramesRead > 0) {
94641 |         pcmFramesRead = ma_dr_mp3_hdr_frame_samples(pMP3->decoder.header);
94642 |         pMP3->pcmFramesConsumedInMP3Frame = 0;
94643 |         pMP3->pcmFramesRemainingInMP3Frame = pcmFramesRead;
94644 |         pMP3->mp3FrameChannels = info.channels;
94645 |         pMP3->mp3FrameSampleRate = info.sample_rate;
94646 |         if (pMP3FrameInfo != NULL) {
94647 |             *pMP3FrameInfo = info;
94648 |         }
94649 |         if (ppMP3FrameData != NULL) {
94650 |             *ppMP3FrameData = pMP3->pData + pMP3->dataConsumed - (size_t)info.frame_bytes;
94651 |         }
94652 |         break;
94653 |     } else if (info.frame_bytes == 0) {
94654 |         size_t bytesRead;
94655 |         MA_DR_MP3_MOVE_MEMORY(pMP3->pData, pMP3->pData + pMP3->dataConsumed, pMP3->dataSize);
94656 |         pMP3->dataConsumed = 0;
94657 |         if (pMP3->dataCapacity == pMP3->dataSize) {
94658 |             ma_uint8* pNewData;
94659 |             size_t newDataCap;
94660 |             newDataCap = pMP3->dataCapacity + MA_DR_MP3_DATA_CHUNK_SIZE;
94661 |             pNewData = (ma_uint8*)ma_dr_mp3__realloc_from_callbacks(pMP3->pData, newDataCap,
> pMP3->dataCapacity, &pMP3->allocationCallbacks);
94662 |             if (pNewData == NULL) {
94663 |                 return 0;
94664 |             }
94665 |             pMP3->pData = pNewData;
94666 |             pMP3->dataCapacity = newDataCap;
94667 |         }
94668 |         bytesRead = ma_dr_mp3__on_read_clamped(pMP3, pMP3->pData + pMP3->dataSize,
> (pMP3->dataCapacity - pMP3->dataSize));
94669 |         if (bytesRead == 0) {
94670 |             pMP3->atEnd = MA_TRUE;
94671 |             return 0;
94672 |         }
94673 |         pMP3->dataSize += bytesRead;
94674 |     }
94675 | };
94676 | return pcmFramesRead;
94677 | }
94678 | static ma_uint32 ma_dr_mp3_decode_next_frame_ex__memory(ma_dr_mp3* pMP3, ma_dr_mp3d_sample_t*
> pPCMFrames, ma_dr_mp3dec_frame_info* pMP3FrameInfo, const ma_uint8** ppMP3FrameData)
94679 | {

```

```

94680 |     ma_uint32 pcmFramesRead = 0;
94681 |     ma_dr_mp3dec_frame_info info;
94682 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94683 |     MA_DR_MP3_ASSERT(pMP3->memory.pData != NULL);
94684 |     if (pMP3->atEnd) {
94685 |         return 0;
94686 |     }
94687 |     for (;;) {
94688 |         pcmFramesRead = ma_dr_mp3dec_decode_frame(&pMP3->decoder, pMP3->memory.pData +
> pMP3->memory.currentReadPos, (int)(pMP3->memory.dataSize - pMP3->memory.currentReadPos), pPCMFrames,
> &info);
94689 |         if (pcmFramesRead > 0) {
94690 |             pcmFramesRead = ma_dr_mp3_hdr_frame_samples(pMP3->decoder.header);
94691 |             pMP3->pcmFramesConsumedInMP3Frame = 0;
94692 |             pMP3->pcmFramesRemainingInMP3Frame = pcmFramesRead;
94693 |             pMP3->mp3FrameChannels = info.channels;
94694 |             pMP3->mp3FrameSampleRate = info.sample_rate;
94695 |             if (pMP3FrameInfo != NULL) {
94696 |                 *pMP3FrameInfo = info;
94697 |             }
94698 |             if (ppMP3FrameData != NULL) {
94699 |                 *ppMP3FrameData = pMP3->memory.pData + pMP3->memory.currentReadPos;
94700 |             }
94701 |             break;
94702 |         } else if (info.frame_bytes > 0) {
94703 |             pMP3->memory.currentReadPos += (size_t)info.frame_bytes;
94704 |             pMP3->streamCursor += (size_t)info.frame_bytes;
94705 |         } else {
94706 |             break;
94707 |         }
94708 |     }
94709 |     pMP3->memory.currentReadPos += (size_t)info.frame_bytes;
94710 |     pMP3->streamCursor += (size_t)info.frame_bytes;
94711 |     return pcmFramesRead;
94712 | }
94713 | static ma_uint32 ma_dr_mp3_decode_next_frame_ex(ma_dr_mp3* pMP3, ma_dr_mp3d_sample_t* pPCMFrames,
> ma_dr_mp3dec_frame_info* pMP3FrameInfo, const ma_uint8** ppMP3FrameData)
94714 | {
94715 |     if (pMP3->memory.pData != NULL && pMP3->memory.dataSize > 0) {
94716 |         return ma_dr_mp3_decode_next_frame_ex__memory(pMP3, pPCMFrames, pMP3FrameInfo,
> ppMP3FrameData);
94717 |     } else {
94718 |         return ma_dr_mp3_decode_next_frame_ex__callbacks(pMP3, pPCMFrames, pMP3FrameInfo,
> ppMP3FrameData);
94719 |     }
94720 | }
94721 | static ma_uint32 ma_dr_mp3_decode_next_frame(ma_dr_mp3* pMP3)
94722 | {
94723 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94724 |     return ma_dr_mp3_decode_next_frame_ex(pMP3, (ma_dr_mp3d_sample_t*)pMP3->pcmFrames, NULL, NULL);
94725 | }
94726 | #if 0
94727 | static ma_uint32 ma_dr_mp3_seek_next_frame(ma_dr_mp3* pMP3)
94728 | {
94729 |     ma_uint32 pcmFrameCount;
94730 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94731 |     pcmFrameCount = ma_dr_mp3_decode_next_frame_ex(pMP3, NULL, NULL, NULL);
94732 |     if (pcmFrameCount == 0) {
94733 |         return 0;
94734 |     }
94735 |     pMP3->currentPCMFrame += pcmFrameCount;
94736 |     pMP3->pcmFramesConsumedInMP3Frame = pcmFrameCount;
94737 |     pMP3->pcmFramesRemainingInMP3Frame = 0;
94738 |     return pcmFrameCount;
94739 | }
94740 | #endif
94741 | static ma_bool32 ma_dr_mp3_init_internal(ma_dr_mp3* pMP3, ma_dr_mp3_read_proc onRead,
> ma_dr_mp3_seek_proc onSeek, ma_dr_mp3_tell_proc onTell, ma_dr_mp3_meta_proc onMeta, void* pUserData,
> void* pUserDataMeta, const ma_allocation_callbacks* pAllocationCallbacks)
94742 | {
94743 |     ma_dr_mp3dec_frame_info firstFrameInfo;
94744 |     const ma_uint8* pFirstFrameData;
94745 |     ma_uint32 firstFramePCMFrameCount;
94746 |     ma_uint32 detectedMP3FrameCount = 0xFFFFFFFF;
94747 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94748 |     MA_DR_MP3_ASSERT(onRead != NULL);
94749 |     ma_dr_mp3dec_init(&pMP3->decoder);
94750 |     pMP3->onRead = onRead;
94751 |     pMP3->onSeek = onSeek;

```



```

94752 |     pMP3->onMeta = onMeta;
94753 |     pMP3->pUserData = pUserData;
94754 |     pMP3->pUserDataMeta = pUserDataMeta;
94755 |     pMP3->allocationCallbacks =
> ma_dr_mp3_copy_allocation_callbacks_or_defaults(pAllocationCallbacks);
94756 |     if (pMP3->allocationCallbacks.onFree == NULL || (pMP3->allocationCallbacks.onMalloc == NULL &&
> pMP3->allocationCallbacks.onRealloc == NULL)) {
94757 |         return MA_FALSE;
94758 |     }
94759 |     pMP3->streamCursor = 0;
94760 |     pMP3->streamLength = MA_UINT64_MAX;
94761 |     pMP3->streamStartOffset = 0;
94762 |     pMP3->delayInPCMFrames = 0;
94763 |     pMP3->paddingInPCMFrames = 0;
94764 |     pMP3->totalPCMFrameCount = MA_UINT64_MAX;
94765 |     #if 1
94766 |     if (onSeek != NULL && onTell != NULL) {
94767 |         if (onSeek(pUserData, 0, MA_DR_MP3_SEEK_END)) {
94768 |             ma_int64 streamLen;
94769 |             int streamEndOffset = 0;
94770 |             if (onTell(pUserData, &streamLen)) {
94771 |                 if (streamLen > 128) {
94772 |                     char id3[3];
94773 |                     if (onSeek(pUserData, streamEndOffset - 128, MA_DR_MP3_SEEK_END)) {
94774 |                         if (onRead(pUserData, id3, 3) == 3 && id3[0] == 'T' && id3[1] == 'A' &&
> id3[2] == 'G') {
94775 |                             streamEndOffset -= 128;
94776 |                             streamLen -= 128;
94777 |                             if (onMeta != NULL) {
94778 |                                 ma_uint8 tag[128];
94779 |                                 tag[0] = 'T'; tag[1] = 'A'; tag[2] = 'G';
94780 |                                 if (onRead(pUserData, tag + 3, 125) == 125) {
94781 |                                     ma_dr_mp3__on_meta(pMP3, MA_DR_MP3_METADATA_TYPE_ID3V1, tag,
> 128);
94782 |                                 }
94783 |                             }
94784 |                         } else {
94785 |                             }
94786 |                         } else {
94787 |                             }
94788 |                         } else {
94789 |                             }
94790 |                             if (streamLen > 32) {
94791 |                                 char ape[32];
94792 |                                 if (onSeek(pUserData, streamEndOffset - 32, MA_DR_MP3_SEEK_END)) {
94793 |                                     if (onRead(pUserData, ape, 32) == 32 && ape[0] == 'A' && ape[1] == 'P' &&
> ape[2] == 'E' && ape[3] == 'T' && ape[4] == 'A' && ape[5] == 'G' && ape[6] == 'E' && ape[7] == 'X')
> {
94794 |                                         ma_uint32 tagSize =
94795 |                                             ((ma_uint32)ape[24] << 0) |
94796 |                                             ((ma_uint32)ape[25] << 8) |
94797 |                                             ((ma_uint32)ape[26] << 16) |
94798 |                                             ((ma_uint32)ape[27] << 24);
94799 |                                         if (32 + tagSize < streamLen) {
94800 |                                             streamEndOffset -= 32 + tagSize;
94801 |                                             streamLen -= 32 + tagSize;
94802 |                                             if (onMeta != NULL) {
94803 |                                                 if (onSeek(pUserData, streamEndOffset, MA_DR_MP3_SEEK_END)) {
94804 |                                                     size_t apeTagSize = (size_t)tagSize + 32;
94805 |                                                     ma_uint8* pTagData = (ma_uint8*)ma_dr_mp3_malloc(apeTagSize,
> pAllocationCallbacks);
94806 |                                                     if (pTagData != NULL) {
94807 |                                                         if (onRead(pUserData, pTagData, apeTagSize) ==
> apeTagSize) {
94808 |                                                             ma_dr_mp3__on_meta(pMP3,
> MA_DR_MP3_METADATA_TYPE_APE, pTagData, apeTagSize);
94809 |                                                         }
94810 |                                                         ma_dr_mp3_free(pTagData, pAllocationCallbacks);
94811 |                                                         }
94812 |                                                     }
94813 |                                                 }
94814 |                                             } else {
94815 |                                                 }
94816 |                                             }
94817 |                                         }
94818 |                                     } else {
94819 |                                     }
94820 |                                     if (!onSeek(pUserData, 0, MA_DR_MP3_SEEK_SET)) {
94821 |                                         return MA_FALSE;

```

```

94822 |         }
94823 |         pMP3->streamLength = (ma_uint64)streamLen;
94824 |         if (pMP3->memory.pData != NULL) {
94825 |             pMP3->memory.dataSize = (size_t)pMP3->streamLength;
94826 |         }
94827 |     } else {
94828 |         if (!onSeek(pUserData, 0, MA_DR_MP3_SEEK_SET)) {
94829 |             return MA_FALSE;
94830 |         }
94831 |     }
94832 | } else {
94833 | }
94834 | } else {
94835 | }
94836 | #endif
94837 | #if 1
94838 | {
94839 |     char header[10];
94840 |     if (onRead(pUserData, header, 10) == 10) {
94841 |         if (header[0] == 'I' && header[1] == 'D' && header[2] == '3') {
94842 |             ma_uint32 tagSize =
94843 |                 (((ma_uint32)header[6] & 0x7F) << 21) |
94844 |                 (((ma_uint32)header[7] & 0x7F) << 14) |
94845 |                 (((ma_uint32)header[8] & 0x7F) << 7) |
94846 |                 (((ma_uint32)header[9] & 0x7F) << 0);
94847 |             if (header[5] & 0x10) {
94848 |                 tagSize += 10;
94849 |             }
94850 |             if (onMeta != NULL) {
94851 |                 size_t tagSizeWithHeader = 10 + tagSize;
94852 |                 ma_uint8* pTagData = (ma_uint8*)ma_dr_mp3_malloc(tagSizeWithHeader,
> pAllocationCallbacks);
94853 |                 if (pTagData != NULL) {
94854 |                     MA_DR_MP3_COPY_MEMORY(pTagData, header, 10);
94855 |                     if (onRead(pUserData, pTagData + 10, tagSize) == tagSize) {
94856 |                         ma_dr_mp3_on_meta(pMP3, MA_DR_MP3_METADATA_TYPE_ID3V2, pTagData,
> tagSizeWithHeader);
94857 |                     }
94858 |                     ma_dr_mp3_free(pTagData, pAllocationCallbacks);
94859 |                 }
94860 |             } else {
94861 |                 if (onSeek != NULL) {
94862 |                     if (!onSeek(pUserData, tagSize, MA_DR_MP3_SEEK_CUR)) {
94863 |                         return MA_FALSE;
94864 |                     }
94865 |                 } else {
94866 |                     char discard[1024];
94867 |                     while (tagSize > 0) {
94868 |                         size_t bytesToRead = tagSize;
94869 |                         if (bytesToRead > sizeof(discard)) {
94870 |                             bytesToRead = sizeof(discard);
94871 |                         }
94872 |                         if (onRead(pUserData, discard, bytesToRead) != bytesToRead) {
94873 |                             return MA_FALSE;
94874 |                         }
94875 |                         tagSize -= (ma_uint32)bytesToRead;
94876 |                     }
94877 |                 }
94878 |             }
94879 |             pMP3->streamStartOffset += 10 + tagSize;
94880 |             pMP3->streamCursor = pMP3->streamStartOffset;
94881 |         } else {
94882 |             if (onSeek != NULL) {
94883 |                 if (!onSeek(pUserData, 0, MA_DR_MP3_SEEK_SET)) {
94884 |                     return MA_FALSE;
94885 |                 }
94886 |             } else {
94887 |             }
94888 |         }
94889 |     } else {
94890 |         return MA_FALSE;
94891 |     }
94892 | }
94893 | #endif
94894 | firstFramePCMFrameCount = ma_dr_mp3_decode_next_frame_ex(pMP3,
> (ma_dr_mp3d_sample_t*)pMP3->pcmFrames, &firstFrameInfo, &pFirstFrameData);
94895 | if (firstFramePCMFrameCount > 0) {
94896 |     MA_DR_MP3_ASSERT(pFirstFrameData != NULL);
94897 |     #if 1

```

```

94898 |         MA_DR_MP3_ASSERT(firstFrameInfo.frame_bytes > 0);
94899 |     {
94900 |         ma_dr_mp3_bs bs;
94901 |         ma_dr_mp3_L3_gr_info grInfo[4];
94902 |         ma_dr_mp3_bs_init(&bs, pFirstFrameData + MA_DR_MP3_HDR_SIZE, firstFrameInfo.frame_bytes
> - MA_DR_MP3_HDR_SIZE);
94903 |         if (MA_DR_MP3_HDR_IS_CRC(pFirstFrameData)) {
94904 |             ma_dr_mp3_bs_get_bits(&bs, 16);
94905 |         }
94906 |         if (ma_dr_mp3_L3_read_side_info(&bs, grInfo, pFirstFrameData) >= 0) {
94907 |             ma_bool32 isXing = MA_FALSE;
94908 |             ma_bool32 isInfo = MA_FALSE;
94909 |             const ma_uint8* pTagData;
94910 |             const ma_uint8* pTagDataBeg;
94911 |             pTagDataBeg = pFirstFrameData + MA_DR_MP3_HDR_SIZE + (bs.pos/8);
94912 |             pTagData = pTagDataBeg;
94913 |             isXing = (pTagData[0] == 'X' && pTagData[1] == 'i' && pTagData[2] == 'n' &&
> pTagData[3] == 'g');
94914 |             isInfo = (pTagData[0] == 'I' && pTagData[1] == 'n' && pTagData[2] == 'f' &&
> pTagData[3] == 'o');
94915 |             if (isXing || isInfo) {
94916 |                 ma_uint32 bytes = 0;
94917 |                 ma_uint32 flags = pTagData[7];
94918 |                 pTagData += 8;
94919 |                 if (flags & 0x01) {
94920 |                     detectedMP3FrameCount = (ma_uint32)pTagData[0] << 24 |
> (ma_uint32)pTagData[1] << 16 | (ma_uint32)pTagData[2] << 8 | (ma_uint32)pTagData[3];
94921 |                     pTagData += 4;
94922 |                 }
94923 |                 if (flags & 0x02) {
94924 |                     bytes = (ma_uint32)pTagData[0] << 24 | (ma_uint32)pTagData[1] << 16 |
> (ma_uint32)pTagData[2] << 8 | (ma_uint32)pTagData[3];
94925 |                     (void)bytes;
94926 |                     pTagData += 4;
94927 |                 }
94928 |                 if (flags & 0x04) {
94929 |                     pTagData += 100;
94930 |                 }
94931 |                 if (flags & 0x08) {
94932 |                     pTagData += 4;
94933 |                 }
94934 |                 if (pTagData[0]) {
94935 |                     pTagData += 21;
94936 |                     if (pTagData - pFirstFrameData + 14 < firstFrameInfo.frame_bytes) {
94937 |                         int delayInPCMFrames;
94938 |                         int paddingInPCMFrames;
94939 |                         delayInPCMFrames = (( (ma_uint32)pTagData[0] << 4) |
> ((ma_uint32)pTagData[1] >> 4)) + (528 + 1);
94940 |                         paddingInPCMFrames = (((ma_uint32)pTagData[1] & 0xF) << 8) |
> ((ma_uint32)pTagData[2] >> 4)) - (528 + 1);
94941 |                         if (paddingInPCMFrames < 0) {
94942 |                             paddingInPCMFrames = 0;
94943 |                         }
94944 |                         pMP3->delayInPCMFrames = (ma_uint32)delayInPCMFrames;
94945 |                         pMP3->paddingInPCMFrames = (ma_uint32)paddingInPCMFrames;
94946 |                     }
94947 |                 }
94948 |                 if (isXing) {
94949 |                     pMP3->isVBR = MA_TRUE;
94950 |                 } else if (isInfo) {
94951 |                     pMP3->isCBR = MA_TRUE;
94952 |                 }
94953 |                 if (onMeta != NULL) {
94954 |                     ma_dr_mp3_metadata_type metadataType = isXing ? MA_DR_MP3_METADATA_TYPE_XING
> : MA_DR_MP3_METADATA_TYPE_VBRI;
94955 |                     size_t tagDataSize;
94956 |                     tagDataSize = (size_t)firstFrameInfo.frame_bytes;
94957 |                     tagDataSize -= (size_t)(pTagDataBeg - pFirstFrameData);
94958 |                     ma_dr_mp3_on_meta(pMP3, metadataType, pTagDataBeg, tagDataSize);
94959 |                 }
94960 |                 pMP3->pcmFramesRemainingInMP3Frame = 0;
94961 |                 pMP3->streamStartOffset += (ma_uint32)(firstFrameInfo.frame_bytes);
94962 |                 pMP3->streamCursor = pMP3->streamStartOffset;
94963 |                 ma_dr_mp3dec_init(&pMP3->decoder);
94964 |             }
94965 |         } else {
94966 |         }
94967 |     }
94968 | #endif

```

```

94969 |     } else {
94970 |         ma_dr_mp3__free_from_callbacks(pMP3->pData, &pMP3->allocationCallbacks);
94971 |         return MA_FALSE;
94972 |     }
94973 |     if (detectedMP3FrameCount != 0xFFFFFFFF) {
94974 |         pMP3->totalPCMFrameCount = detectedMP3FrameCount * firstFramePCMFrameCount;
94975 |     }
94976 |     pMP3->channels = pMP3->mp3FrameChannels;
94977 |     pMP3->sampleRate = pMP3->mp3FrameSampleRate;
94978 |     return MA_TRUE;
94979 | }
94980 | MA_API ma_bool32 ma_dr_mp3_init(ma_dr_mp3* pMP3, ma_dr_mp3_read_proc onRead, ma_dr_mp3_seek_proc
> onSeek, ma_dr_mp3_tell_proc onTell, ma_dr_mp3_meta_proc onMeta, void* pUserData, const
> ma_allocation_callbacks* pAllocationCallbacks)
94981 | {
94982 |     if (pMP3 == NULL || onRead == NULL) {
94983 |         return MA_FALSE;
94984 |     }
94985 |     MA_DR_MP3_ZERO_OBJECT(pMP3);
94986 |     return ma_dr_mp3_init_internal(pMP3, onRead, onSeek, onTell, onMeta, pUserData, pUserData,
> pAllocationCallbacks);
94987 | }
94988 | static size_t ma_dr_mp3__on_read_memory(void* pUserData, void* pBufferOut, size_t bytesToRead)
94989 | {
94990 |     ma_dr_mp3* pMP3 = (ma_dr_mp3*)pUserData;
94991 |     size_t bytesRemaining;
94992 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
94993 |     MA_DR_MP3_ASSERT(pMP3->memory.dataSize >= pMP3->memory.currentReadPos);
94994 |     bytesRemaining = pMP3->memory.dataSize - pMP3->memory.currentReadPos;
94995 |     if (bytesToRead > bytesRemaining) {
94996 |         bytesToRead = bytesRemaining;
94997 |     }
94998 |     if (bytesToRead > 0) {
94999 |         MA_DR_MP3_COPY_MEMORY(pBufferOut, pMP3->memory.pData + pMP3->memory.currentReadPos,
> bytesToRead);
95000 |         pMP3->memory.currentReadPos += bytesToRead;
95001 |     }
95002 |     return bytesToRead;
95003 | }
95004 | static ma_bool32 ma_dr_mp3__on_seek_memory(void* pUserData, int byteOffset, ma_dr_mp3_seek_origin
> origin)
95005 | {
95006 |     ma_dr_mp3* pMP3 = (ma_dr_mp3*)pUserData;
95007 |     ma_int64 newCursor;
95008 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95009 |     if (origin == MA_DR_MP3_SEEK_SET) {
95010 |         newCursor = 0;
95011 |     } else if (origin == MA_DR_MP3_SEEK_CUR) {
95012 |         newCursor = (ma_int64)pMP3->memory.currentReadPos;
95013 |     } else if (origin == MA_DR_MP3_SEEK_END) {
95014 |         newCursor = (ma_int64)pMP3->memory.dataSize;
95015 |     } else {
95016 |         MA_DR_MP3_ASSERT(!"Invalid seek origin");
95017 |         return MA_FALSE;
95018 |     }
95019 |     newCursor += byteOffset;
95020 |     if (newCursor < 0) {
95021 |         return MA_FALSE;
95022 |     }
95023 |     if ((size_t)newCursor > pMP3->memory.dataSize) {
95024 |         return MA_FALSE;
95025 |     }
95026 |     pMP3->memory.currentReadPos = (size_t)newCursor;
95027 |     return MA_TRUE;
95028 | }
95029 | static ma_bool32 ma_dr_mp3__on_tell_memory(void* pUserData, ma_int64* pCursor)
95030 | {
95031 |     ma_dr_mp3* pMP3 = (ma_dr_mp3*)pUserData;
95032 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95033 |     MA_DR_MP3_ASSERT(pCursor != NULL);
95034 |     *pCursor = (ma_int64)pMP3->memory.currentReadPos;
95035 |     return MA_TRUE;
95036 | }
95037 | MA_API ma_bool32 ma_dr_mp3_init_memory_with_metadata(ma_dr_mp3* pMP3, const void* pData, size_t
> dataSize, ma_dr_mp3_meta_proc onMeta, void* pUserDataMeta, const ma_allocation_callbacks*
> pAllocationCallbacks)
95038 | {
95039 |     ma_bool32 result;
95040 |     if (pMP3 == NULL) {

```

```

95041 |         return MA_FALSE;
95042 |     }
95043 |     MA_DR_MP3_ZERO_OBJECT(pMP3);
95044 |     if (pData == NULL || dataSize == 0) {
95045 |         return MA_FALSE;
95046 |     }
95047 |     pMP3->memory.pData = (const ma_uint8*)pData;
95048 |     pMP3->memory.dataSize = dataSize;
95049 |     pMP3->memory.currentReadPos = 0;
95050 |     result = ma_dr_mp3_init_internal(pMP3, ma_dr_mp3__on_read_memory, ma_dr_mp3__on_seek_memory,
> ma_dr_mp3__on_tell_memory, onMeta, pMP3, pUserDataMeta, pAllocationCallbacks);
95051 |     if (result == MA_FALSE) {
95052 |         return MA_FALSE;
95053 |     }
95054 |     if (pMP3->streamLength <= (ma_uint64)MA_SIZE_MAX) {
95055 |         pMP3->memory.dataSize = (size_t)pMP3->streamLength;
95056 |     }
95057 |     if (pMP3->streamStartOffset > (ma_uint64)MA_SIZE_MAX) {
95058 |         return MA_FALSE;
95059 |     }
95060 |     return MA_TRUE;
95061 | }
95062 | MA_API ma_bool32 ma_dr_mp3_init_memory(ma_dr_mp3* pMP3, const void* pData, size_t dataSize, const
> ma_allocation_callbacks* pAllocationCallbacks)
95063 | {
95064 |     return ma_dr_mp3_init_memory_with_metadata(pMP3, pData, dataSize, NULL, NULL,
> pAllocationCallbacks);
95065 | }
95066 | #ifndef MA_DR_MP3_NO_STDIO
95067 | #include <stdio.h>
95068 | #include <wchar.h>
95069 | static size_t ma_dr_mp3__on_read_stdio(void* pUserData, void* pBufferOut, size_t bytesToRead)
95070 | {
95071 |     return fread(pBufferOut, 1, bytesToRead, (FILE*)pUserData);
95072 | }
95073 | static ma_bool32 ma_dr_mp3__on_seek_stdio(void* pUserData, int offset, ma_dr_mp3_seek_origin origin)
95074 | {
95075 |     int whence = SEEK_SET;
95076 |     if (origin == MA_DR_MP3_SEEK_CUR) {
95077 |         whence = SEEK_CUR;
95078 |     } else if (origin == MA_DR_MP3_SEEK_END) {
95079 |         whence = SEEK_END;
95080 |     }
95081 |     return fseek((FILE*)pUserData, offset, whence) == 0;
95082 | }
95083 | static ma_bool32 ma_dr_mp3__on_tell_stdio(void* pUserData, ma_int64* pCursor)
95084 | {
95085 |     FILE* pFileStdio = (FILE*)pUserData;
95086 |     ma_int64 result;
95087 |     MA_DR_MP3_ASSERT(pFileStdio != NULL);
95088 |     MA_DR_MP3_ASSERT(pCursor != NULL);
95089 |     #if defined(_WIN32) && !defined(NXDK)
95090 |         #if defined(_MSC_VER) && _MSC_VER > 1200
95091 |             result = _ftelli64(pFileStdio);
95092 |         #else
95093 |             result = ftell(pFileStdio);
95094 |         #endif
95095 |     #else
95096 |         result = ftell(pFileStdio);
95097 |     #endif
95098 |     *pCursor = result;
95099 |     return MA_TRUE;
95100 | }
95101 | MA_API ma_bool32 ma_dr_mp3_init_file_with_metadata(ma_dr_mp3* pMP3, const char* pFilePath,
> ma_dr_mp3_meta_proc onMeta, void* pUserDataMeta, const ma_allocation_callbacks*
> pAllocationCallbacks)
95102 | {
95103 |     ma_bool32 result;
95104 |     FILE* pFile;
95105 |     if (pMP3 == NULL) {
95106 |         return MA_FALSE;
95107 |     }
95108 |     MA_DR_MP3_ZERO_OBJECT(pMP3);
95109 |     if (ma_fopen(&pFile, pFilePath, "rb") != MA_SUCCESS) {
95110 |         return MA_FALSE;
95111 |     }
95112 |     result = ma_dr_mp3_init_internal(pMP3, ma_dr_mp3__on_read_stdio, ma_dr_mp3__on_seek_stdio,
> ma_dr_mp3__on_tell_stdio, onMeta, (void*)pFile, pUserDataMeta, pAllocationCallbacks);
95113 |     if (result != MA_TRUE) {

```

```

95114 |         fclose(pFile);
95115 |         return result;
95116 |     }
95117 |     return MA_TRUE;
95118 | }
95119 | MA_API ma_bool32 ma_dr_mp3_init_file_with_metadata_w(ma_dr_mp3* pMP3, const wchar_t* pFilePath,
> ma_dr_mp3_meta_proc onMeta, void* pUserDataMeta, const ma_allocation_callbacks*
> pAllocationCallbacks)
95120 | {
95121 |     ma_bool32 result;
95122 |     FILE* pFile;
95123 |     if (pMP3 == NULL) {
95124 |         return MA_FALSE;
95125 |     }
95126 |     MA_DR_MP3_ZERO_OBJECT(pMP3);
95127 |     if (ma_wfopen(&pFile, pFilePath, L"rb", pAllocationCallbacks) != MA_SUCCESS) {
95128 |         return MA_FALSE;
95129 |     }
95130 |     result = ma_dr_mp3_init_internal(pMP3, ma_dr_mp3__on_read_stdio, ma_dr_mp3__on_seek_stdio,
> ma_dr_mp3__on_tell_stdio, onMeta, (void*)pFile, pUserDataMeta, pAllocationCallbacks);
95131 |     if (result != MA_TRUE) {
95132 |         fclose(pFile);
95133 |         return result;
95134 |     }
95135 |     return MA_TRUE;
95136 | }
95137 | MA_API ma_bool32 ma_dr_mp3_init_file(ma_dr_mp3* pMP3, const char* pFilePath, const
> ma_allocation_callbacks* pAllocationCallbacks)
95138 | {
95139 |     return ma_dr_mp3_init_file_with_metadata(pMP3, pFilePath, NULL, NULL, pAllocationCallbacks);
95140 | }
95141 | MA_API ma_bool32 ma_dr_mp3_init_file_w(ma_dr_mp3* pMP3, const wchar_t* pFilePath, const
> ma_allocation_callbacks* pAllocationCallbacks)
95142 | {
95143 |     return ma_dr_mp3_init_file_with_metadata_w(pMP3, pFilePath, NULL, NULL, pAllocationCallbacks);
95144 | }
95145 | #endif
95146 | MA_API void ma_dr_mp3_uninit(ma_dr_mp3* pMP3)
95147 | {
95148 |     if (pMP3 == NULL) {
95149 |         return;
95150 |     }
95151 | #ifndef MA_DR_MP3_NO_STDIO
95152 |     if (pMP3->onRead == ma_dr_mp3__on_read_stdio) {
95153 |         FILE* pFile = (FILE*)pMP3->pUserData;
95154 |         if (pFile != NULL) {
95155 |             fclose(pFile);
95156 |             pMP3->pUserData = NULL;
95157 |         }
95158 |     }
95159 | #endif
95160 |     ma_dr_mp3__free_from_callbacks(pMP3->pData, &pMP3->allocationCallbacks);
95161 | }
95162 | #if defined(MA_DR_MP3_FLOAT_OUTPUT)
95163 | static void ma_dr_mp3_f32_to_s16(ma_int16* dst, const float* src, ma_uint64 sampleCount)
95164 | {
95165 |     ma_uint64 i;
95166 |     ma_uint64 i4;
95167 |     ma_uint64 sampleCount4;
95168 |     i = 0;
95169 |     sampleCount4 = sampleCount >> 2;
95170 |     for (i4 = 0; i4 < sampleCount4; i4 += 1) {
95171 |         float x0 = src[i+0];
95172 |         float x1 = src[i+1];
95173 |         float x2 = src[i+2];
95174 |         float x3 = src[i+3];
95175 |         x0 = ((x0 < -1) ? -1 : ((x0 > 1) ? 1 : x0));
95176 |         x1 = ((x1 < -1) ? -1 : ((x1 > 1) ? 1 : x1));
95177 |         x2 = ((x2 < -1) ? -1 : ((x2 > 1) ? 1 : x2));
95178 |         x3 = ((x3 < -1) ? -1 : ((x3 > 1) ? 1 : x3));
95179 |         x0 = x0 * 32767.0f;
95180 |         x1 = x1 * 32767.0f;
95181 |         x2 = x2 * 32767.0f;
95182 |         x3 = x3 * 32767.0f;
95183 |         dst[i+0] = (ma_int16)x0;
95184 |         dst[i+1] = (ma_int16)x1;
95185 |         dst[i+2] = (ma_int16)x2;
95186 |         dst[i+3] = (ma_int16)x3;
95187 |         i += 4;

```

```

95188 |     }
95189 |     for (; i < sampleCount; i += 1) {
95190 |         float x = src[i];
95191 |         x = ((x < -1) ? -1 : ((x > 1) ? 1 : x));
95192 |         x = x * 32767.0f;
95193 |         dst[i] = (ma_int16)x;
95194 |     }
95195 | }
95196 | #endif
95197 | #if !defined(MA_DR_MP3_FLOAT_OUTPUT)
95198 | static void ma_dr_mp3_s16_to_f32(float* dst, const ma_int16* src, ma_uint64 sampleCount)
95199 | {
95200 |     ma_uint64 i;
95201 |     for (i = 0; i < sampleCount; i += 1) {
95202 |         float x = (float)src[i];
95203 |         x = x * 0.000030517578125f;
95204 |         dst[i] = x;
95205 |     }
95206 | }
95207 | #endif
95208 | static ma_uint64 ma_dr_mp3_read_pcm_frames_raw(ma_dr_mp3* pMP3, ma_uint64 framesToRead, void*
> pBufferOut)
95209 | {
95210 |     ma_uint64 totalFramesRead = 0;
95211 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95212 |     MA_DR_MP3_ASSERT(pMP3->onRead != NULL);
95213 |     while (framesToRead > 0) {
95214 |         ma_uint32 framesToConsume;
95215 |         if (pMP3->currentPCMFrame < pMP3->delayInPCMFrames) {
95216 |             ma_uint32 framesToSkip = (ma_uint32)MA_DR_MP3_MIN(pMP3->pcmFramesRemainingInMP3Frame,
> pMP3->delayInPCMFrames - pMP3->currentPCMFrame);
95217 |             pMP3->currentPCMFrame += framesToSkip;
95218 |             pMP3->pcmFramesConsumedInMP3Frame += framesToSkip;
95219 |             pMP3->pcmFramesRemainingInMP3Frame -= framesToSkip;
95220 |         }
95221 |         framesToConsume = (ma_uint32)MA_DR_MP3_MIN(pMP3->pcmFramesRemainingInMP3Frame,
> framesToRead);
95222 |         if (pMP3->totalPCMFrameCount != MA_UINT64_MAX && pMP3->totalPCMFrameCount >
> pMP3->paddingInPCMFrames) {
95223 |             if (pMP3->currentPCMFrame < (pMP3->totalPCMFrameCount - pMP3->paddingInPCMFrames)) {
95224 |                 ma_uint64 framesRemainigToPadding = (pMP3->totalPCMFrameCount -
> pMP3->paddingInPCMFrames) - pMP3->currentPCMFrame;
95225 |                 if (framesToConsume > framesRemainigToPadding) {
95226 |                     framesToConsume = (ma_uint32)framesRemainigToPadding;
95227 |                 }
95228 |             } else {
95229 |                 break;
95230 |             }
95231 |         }
95232 |         if (pBufferOut != NULL) {
95233 |             #if defined(MA_DR_MP3_FLOAT_OUTPUT)
95234 |             {
95235 |                 float* pFramesOutF32 = (float*)MA_DR_MP3_OFFSET_PTR(pBufferOut,
> sizeof(float) * totalFramesRead * pMP3->channels);
95236 |                 float* pFramesInF32 = (float*)MA_DR_MP3_OFFSET_PTR(&pMP3->pcmFrames[0],
> sizeof(float) * pMP3->pcmFramesConsumedInMP3Frame * pMP3->mp3FrameChannels);
95237 |                 MA_DR_MP3_COPY_MEMORY(pFramesOutF32, pFramesInF32, sizeof(float) * framesToConsume *
> pMP3->channels);
95238 |             }
95239 |             #else
95240 |             {
95241 |                 ma_int16* pFramesOutS16 = (ma_int16*)MA_DR_MP3_OFFSET_PTR(pBufferOut,
> sizeof(ma_int16) * totalFramesRead * pMP3->channels);
95242 |                 ma_int16* pFramesInS16 = (ma_int16*)MA_DR_MP3_OFFSET_PTR(&pMP3->pcmFrames[0],
> sizeof(ma_int16) * pMP3->pcmFramesConsumedInMP3Frame * pMP3->mp3FrameChannels);
95243 |                 MA_DR_MP3_COPY_MEMORY(pFramesOutS16, pFramesInS16, sizeof(ma_int16) *
> framesToConsume * pMP3->channels);
95244 |             }
95245 |             #endif
95246 |         }
95247 |         pMP3->currentPCMFrame += framesToConsume;
95248 |         pMP3->pcmFramesConsumedInMP3Frame += framesToConsume;
95249 |         pMP3->pcmFramesRemainingInMP3Frame -= framesToConsume;
95250 |         totalFramesRead += framesToConsume;
95251 |         framesToRead -= framesToConsume;
95252 |         if (framesToRead == 0) {
95253 |             break;
95254 |         }
95255 |         if (pMP3->totalPCMFrameCount != MA_UINT64_MAX && pMP3->totalPCMFrameCount >

```



```

> pMP3->paddingInPCMFrames && pMP3->currentPCMFrame >= (pMP3->totalPCMFrameCount -
> pMP3->paddingInPCMFrames)) {
95256 |         break;
95257 |     }
95258 |     MA_DR_MP3_ASSERT(pMP3->pcmFramesRemainingInMP3Frame == 0);
95259 |     if (ma_dr_mp3_decode_next_frame(pMP3) == 0) {
95260 |         break;
95261 |     }
95262 | }
95263 | return totalFramesRead;
95264 | }
95265 | MA_API ma_uint64 ma_dr_mp3_read_pcm_frames_f32(ma_dr_mp3* pMP3, ma_uint64 framesToRead, float*
> pBufferOut)
95266 | {
95267 |     if (pMP3 == NULL || pMP3->onRead == NULL) {
95268 |         return 0;
95269 |     }
95270 | #if defined(MA_DR_MP3_FLOAT_OUTPUT)
95271 |     return ma_dr_mp3_read_pcm_frames_raw(pMP3, framesToRead, pBufferOut);
95272 | #else
95273 |     {
95274 |         ma_int16 pTempS16[8192];
95275 |         ma_uint64 totalPCMFramesRead = 0;
95276 |         while (totalPCMFramesRead < framesToRead) {
95277 |             ma_uint64 framesJustRead;
95278 |             ma_uint64 framesRemaining = framesToRead - totalPCMFramesRead;
95279 |             ma_uint64 framesToReadNow = MA_DR_MP3_COUNTOF(pTempS16) / pMP3->channels;
95280 |             if (framesToReadNow > framesRemaining) {
95281 |                 framesToReadNow = framesRemaining;
95282 |             }
95283 |             framesJustRead = ma_dr_mp3_read_pcm_frames_raw(pMP3, framesToReadNow, pTempS16);
95284 |             if (framesJustRead == 0) {
95285 |                 break;
95286 |             }
95287 |             ma_dr_mp3_s16_to_f32((float*)MA_DR_MP3_OFFSET_PTR(pBufferOut, sizeof(float) *
> totalPCMFramesRead * pMP3->channels), pTempS16, framesJustRead * pMP3->channels);
95288 |             totalPCMFramesRead += framesJustRead;
95289 |         }
95290 |         return totalPCMFramesRead;
95291 |     }
95292 | #endif
95293 | }
95294 | MA_API ma_uint64 ma_dr_mp3_read_pcm_frames_s16(ma_dr_mp3* pMP3, ma_uint64 framesToRead, ma_int16*
> pBufferOut)
95295 | {
95296 |     if (pMP3 == NULL || pMP3->onRead == NULL) {
95297 |         return 0;
95298 |     }
95299 | #if !defined(MA_DR_MP3_FLOAT_OUTPUT)
95300 |     return ma_dr_mp3_read_pcm_frames_raw(pMP3, framesToRead, pBufferOut);
95301 | #else
95302 |     {
95303 |         float pTempF32[4096];
95304 |         ma_uint64 totalPCMFramesRead = 0;
95305 |         while (totalPCMFramesRead < framesToRead) {
95306 |             ma_uint64 framesJustRead;
95307 |             ma_uint64 framesRemaining = framesToRead - totalPCMFramesRead;
95308 |             ma_uint64 framesToReadNow = MA_DR_MP3_COUNTOF(pTempF32) / pMP3->channels;
95309 |             if (framesToReadNow > framesRemaining) {
95310 |                 framesToReadNow = framesRemaining;
95311 |             }
95312 |             framesJustRead = ma_dr_mp3_read_pcm_frames_raw(pMP3, framesToReadNow, pTempF32);
95313 |             if (framesJustRead == 0) {
95314 |                 break;
95315 |             }
95316 |             ma_dr_mp3_f32_to_s16((ma_int16*)MA_DR_MP3_OFFSET_PTR(pBufferOut, sizeof(ma_int16) *
> totalPCMFramesRead * pMP3->channels), pTempF32, framesJustRead * pMP3->channels);
95317 |             totalPCMFramesRead += framesJustRead;
95318 |         }
95319 |         return totalPCMFramesRead;
95320 |     }
95321 | #endif
95322 | }
95323 | static void ma_dr_mp3_reset(ma_dr_mp3* pMP3)
95324 | {
95325 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95326 |     pMP3->pcmFramesConsumedInMP3Frame = 0;
95327 |     pMP3->pcmFramesRemainingInMP3Frame = 0;
95328 |     pMP3->currentPCMFrame = 0;

```

```

95329 |     pMP3->dataSize = 0;
95330 |     pMP3->atEnd = MA_FALSE;
95331 |     ma_dr_mp3dec_init(&pMP3->decoder);
95332 | }
95333 | static ma_bool32 ma_dr_mp3_seek_to_start_of_stream(ma_dr_mp3* pMP3)
95334 | {
95335 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95336 |     MA_DR_MP3_ASSERT(pMP3->onSeek != NULL);
95337 |     if (!ma_dr_mp3__on_seek_64(pMP3, pMP3->streamStartOffset, MA_DR_MP3_SEEK_SET)) {
95338 |         return MA_FALSE;
95339 |     }
95340 |     ma_dr_mp3_reset(pMP3);
95341 |     return MA_TRUE;
95342 | }
95343 | static ma_bool32 ma_dr_mp3_seek_forward_by_pcm_frames__brute_force(ma_dr_mp3* pMP3, ma_uint64
> frameOffset)
95344 | {
95345 |     ma_uint64 framesRead;
95346 |     #if defined(MA_DR_MP3_FLOAT_OUTPUT)
95347 |         framesRead = ma_dr_mp3_read_pcm_frames_f32(pMP3, frameOffset, NULL);
95348 |     #else
95349 |         framesRead = ma_dr_mp3_read_pcm_frames_s16(pMP3, frameOffset, NULL);
95350 |     #endif
95351 |     if (framesRead != frameOffset) {
95352 |         return MA_FALSE;
95353 |     }
95354 |     return MA_TRUE;
95355 | }
95356 | static ma_bool32 ma_dr_mp3_seek_to_pcm_frame__brute_force(ma_dr_mp3* pMP3, ma_uint64 frameIndex)
95357 | {
95358 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95359 |     if (frameIndex == pMP3->currentPCMFrame) {
95360 |         return MA_TRUE;
95361 |     }
95362 |     if (frameIndex < pMP3->currentPCMFrame) {
95363 |         if (!ma_dr_mp3_seek_to_start_of_stream(pMP3)) {
95364 |             return MA_FALSE;
95365 |         }
95366 |     }
95367 |     MA_DR_MP3_ASSERT(frameIndex >= pMP3->currentPCMFrame);
95368 |     return ma_dr_mp3_seek_forward_by_pcm_frames__brute_force(pMP3, (frameIndex -
> pMP3->currentPCMFrame));
95369 | }
95370 | static ma_bool32 ma_dr_mp3_find_closest_seek_point(ma_dr_mp3* pMP3, ma_uint64 frameIndex, ma_uint32*
> pSeekPointIndex)
95371 | {
95372 |     ma_uint32 iSeekPoint;
95373 |     MA_DR_MP3_ASSERT(pSeekPointIndex != NULL);
95374 |     *pSeekPointIndex = 0;
95375 |     if (frameIndex < pMP3->pSeekPoints[0].pcmFrameIndex) {
95376 |         return MA_FALSE;
95377 |     }
95378 |     for (iSeekPoint = 0; iSeekPoint < pMP3->seekPointCount; ++iSeekPoint) {
95379 |         if (pMP3->pSeekPoints[iSeekPoint].pcmFrameIndex > frameIndex) {
95380 |             break;
95381 |         }
95382 |         *pSeekPointIndex = iSeekPoint;
95383 |     }
95384 |     return MA_TRUE;
95385 | }
95386 | static ma_bool32 ma_dr_mp3_seek_to_pcm_frame__seek_table(ma_dr_mp3* pMP3, ma_uint64 frameIndex)
95387 | {
95388 |     ma_dr_mp3_seek_point seekPoint;
95389 |     ma_uint32 priorSeekPointIndex;
95390 |     ma_uint16 iMP3Frame;
95391 |     ma_uint64 leftoverFrames;
95392 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95393 |     MA_DR_MP3_ASSERT(pMP3->pSeekPoints != NULL);
95394 |     MA_DR_MP3_ASSERT(pMP3->seekPointCount > 0);
95395 |     if (ma_dr_mp3_find_closest_seek_point(pMP3, frameIndex, &priorSeekPointIndex)) {
95396 |         seekPoint = pMP3->pSeekPoints[priorSeekPointIndex];
95397 |     } else {
95398 |         seekPoint.seekPosInBytes = 0;
95399 |         seekPoint.pcmFrameIndex = 0;
95400 |         seekPoint.mp3FramesToDiscard = 0;
95401 |         seekPoint.pcmFramesToDiscard = 0;
95402 |     }
95403 |     if (!ma_dr_mp3__on_seek_64(pMP3, seekPoint.seekPosInBytes, MA_DR_MP3_SEEK_SET)) {
95404 |         return MA_FALSE;

```

```

95405 |     }
95406 |     ma_dr_mp3_reset(pMP3);
95407 |     for (iMP3Frame = 0; iMP3Frame < seekPoint.mp3FramesToDiscard; ++iMP3Frame) {
95408 |         ma_uint32 pcmFramesRead;
95409 |         ma_dr_mp3d_sample_t* pPCMFrames;
95410 |         pPCMFrames = NULL;
95411 |         if (iMP3Frame == seekPoint.mp3FramesToDiscard-1) {
95412 |             pPCMFrames = (ma_dr_mp3d_sample_t*)pMP3->pcmFrames;
95413 |         }
95414 |         pcmFramesRead = ma_dr_mp3_decode_next_frame_ex(pMP3, pPCMFrames, NULL, NULL);
95415 |         if (pcmFramesRead == 0) {
95416 |             return MA_FALSE;
95417 |         }
95418 |     }
95419 |     pMP3->currentPCMFrame = seekPoint.pcmFrameIndex - seekPoint.pcmFramesToDiscard;
95420 |     leftoverFrames = frameIndex - pMP3->currentPCMFrame;
95421 |     return ma_dr_mp3_seek_forward_by_pcm_frames__brute_force(pMP3, leftoverFrames);
95422 | }
95423 | MA_API ma_bool32 ma_dr_mp3_seek_to_pcm_frame(ma_dr_mp3* pMP3, ma_uint64 frameIndex)
95424 | {
95425 |     if (pMP3 == NULL || pMP3->onSeek == NULL) {
95426 |         return MA_FALSE;
95427 |     }
95428 |     if (frameIndex == 0) {
95429 |         return ma_dr_mp3_seek_to_start_of_stream(pMP3);
95430 |     }
95431 |     if (pMP3->pSeekPoints != NULL && pMP3->seekPointCount > 0) {
95432 |         return ma_dr_mp3_seek_to_pcm_frame__seek_table(pMP3, frameIndex);
95433 |     } else {
95434 |         return ma_dr_mp3_seek_to_pcm_frame__brute_force(pMP3, frameIndex);
95435 |     }
95436 | }
95437 | MA_API ma_bool32 ma_dr_mp3_get_mp3_and_pcm_frame_count(ma_dr_mp3* pMP3, ma_uint64* pMP3FrameCount,
95438 | > ma_uint64* pPCMFrameCount)
95439 | {
95440 |     ma_uint64 currentPCMFrame;
95441 |     ma_uint64 totalPCMFrameCount;
95442 |     ma_uint64 totalMP3FrameCount;
95443 |     if (pMP3 == NULL) {
95444 |         return MA_FALSE;
95445 |     }
95446 |     if (pMP3->onSeek == NULL) {
95447 |         return MA_FALSE;
95448 |     }
95449 |     currentPCMFrame = pMP3->currentPCMFrame;
95450 |     if (!ma_dr_mp3_seek_to_start_of_stream(pMP3)) {
95451 |         return MA_FALSE;
95452 |     }
95453 |     totalPCMFrameCount = 0;
95454 |     totalMP3FrameCount = 0;
95455 |     for (;;) {
95456 |         ma_uint32 pcmFramesInCurrentMP3Frame;
95457 |         pcmFramesInCurrentMP3Frame = ma_dr_mp3_decode_next_frame_ex(pMP3, NULL, NULL, NULL);
95458 |         if (pcmFramesInCurrentMP3Frame == 0) {
95459 |             break;
95460 |         }
95461 |         totalPCMFrameCount += pcmFramesInCurrentMP3Frame;
95462 |         totalMP3FrameCount += 1;
95463 |     }
95464 |     if (!ma_dr_mp3_seek_to_start_of_stream(pMP3)) {
95465 |         return MA_FALSE;
95466 |     }
95467 |     if (!ma_dr_mp3_seek_to_pcm_frame(pMP3, currentPCMFrame)) {
95468 |         return MA_FALSE;
95469 |     }
95470 |     if (pMP3FrameCount != NULL) {
95471 |         *pMP3FrameCount = totalMP3FrameCount;
95472 |     }
95473 |     if (pPCMFrameCount != NULL) {
95474 |         *pPCMFrameCount = totalPCMFrameCount;
95475 |     }
95476 |     return MA_TRUE;
95477 | }
95478 | MA_API ma_uint64 ma_dr_mp3_get_pcm_frame_count(ma_dr_mp3* pMP3)
95479 | {
95480 |     ma_uint64 totalPCMFrameCount;
95481 |     if (pMP3 == NULL) {
95482 |         return 0;

```

```

95483 |     if (pMP3->totalPCMFrameCount != MA_UINT64_MAX) {
95484 |         totalPCMFrameCount = pMP3->totalPCMFrameCount;
95485 |         if (totalPCMFrameCount >= pMP3->delayInPCMFrames) {
95486 |             totalPCMFrameCount -= pMP3->delayInPCMFrames;
95487 |         } else {
95488 |             }
95489 |         if (totalPCMFrameCount >= pMP3->paddingInPCMFrames) {
95490 |             totalPCMFrameCount -= pMP3->paddingInPCMFrames;
95491 |         } else {
95492 |             }
95493 |         return totalPCMFrameCount;
95494 |     } else {
95495 |         if (!ma_dr_mp3_get_mp3_and_pcm_frame_count(pMP3, NULL, &totalPCMFrameCount)) {
95496 |             return 0;
95497 |         }
95498 |         return totalPCMFrameCount;
95499 |     }
95500 | }
95501 | MA_API ma_uint64 ma_dr_mp3_get_mp3_frame_count(ma_dr_mp3* pMP3)
95502 | {
95503 |     ma_uint64 totalMP3FrameCount;
95504 |     if (!ma_dr_mp3_get_mp3_and_pcm_frame_count(pMP3, &totalMP3FrameCount, NULL)) {
95505 |         return 0;
95506 |     }
95507 |     return totalMP3FrameCount;
95508 | }
95509 | static void ma_dr_mp3_accumulate_running_pcm_frame_count(ma_dr_mp3* pMP3, ma_uint32
> pcmFrameCountIn, ma_uint64* pRunningPCMFrameCount, float* pRunningPCMFrameCountFractionalPart)
95510 | {
95511 |     float srcRatio;
95512 |     float pcmFrameCountOutF;
95513 |     ma_uint32 pcmFrameCountOut;
95514 |     srcRatio = (float)pMP3->mp3FrameSampleRate / (float)pMP3->sampleRate;
95515 |     MA_DR_MP3_ASSERT(srcRatio > 0);
95516 |     pcmFrameCountOutF = *pRunningPCMFrameCountFractionalPart + (pcmFrameCountIn / srcRatio);
95517 |     pcmFrameCountOut = (ma_uint32)pcmFrameCountOutF;
95518 |     *pRunningPCMFrameCountFractionalPart = pcmFrameCountOutF - pcmFrameCountOut;
95519 |     *pRunningPCMFrameCount += pcmFrameCountOut;
95520 | }
95521 | typedef struct
95522 | {
95523 |     ma_uint64 bytePos;
95524 |     ma_uint64 pcmFrameIndex;
95525 | } ma_dr_mp3__seeking_mp3_frame_info;
95526 | MA_API ma_bool32 ma_dr_mp3_calculate_seek_points(ma_dr_mp3* pMP3, ma_uint32* pSeekPointCount,
> ma_dr_mp3_seek_point* pSeekPoints)
95527 | {
95528 |     ma_uint32 seekPointCount;
95529 |     ma_uint64 currentPCMFrame;
95530 |     ma_uint64 totalMP3FrameCount;
95531 |     ma_uint64 totalPCMFrameCount;
95532 |     if (pMP3 == NULL || pSeekPointCount == NULL || pSeekPoints == NULL) {
95533 |         return MA_FALSE;
95534 |     }
95535 |     seekPointCount = *pSeekPointCount;
95536 |     if (seekPointCount == 0) {
95537 |         return MA_FALSE;
95538 |     }
95539 |     currentPCMFrame = pMP3->currentPCMFrame;
95540 |     if (!ma_dr_mp3_get_mp3_and_pcm_frame_count(pMP3, &totalMP3FrameCount, &totalPCMFrameCount)) {
95541 |         return MA_FALSE;
95542 |     }
95543 |     if (totalMP3FrameCount < MA_DR_MP3_SEEK_LEADING_MP3_FRAMES+1) {
95544 |         seekPointCount = 1;
95545 |         pSeekPoints[0].seekPosInBytes = 0;
95546 |         pSeekPoints[0].pcmFrameIndex = 0;
95547 |         pSeekPoints[0].mp3FramesToDiscard = 0;
95548 |         pSeekPoints[0].pcmFramesToDiscard = 0;
95549 |     } else {
95550 |         ma_uint64 pcmFramesBetweenSeekPoints;
95551 |         ma_dr_mp3__seeking_mp3_frame_info mp3FrameInfo[MA_DR_MP3_SEEK_LEADING_MP3_FRAMES+1];
95552 |         ma_uint64 runningPCMFrameCount = 0;
95553 |         float runningPCMFrameCountFractionalPart = 0;
95554 |         ma_uint64 nextTargetPCMFrame;
95555 |         ma_uint32 iMP3Frame;
95556 |         ma_uint32 iSeekPoint;
95557 |         if (seekPointCount > totalMP3FrameCount-1) {
95558 |             seekPointCount = (ma_uint32)totalMP3FrameCount-1;
95559 |         }

```

```

95560 |         pcmFramesBetweenSeekPoints = totalPCMFrameCount / (seekPointCount+1);
95561 |         if (!ma_dr_mp3_seek_to_start_of_stream(pMP3)) {
95562 |             return MA_FALSE;
95563 |         }
95564 |         for (iMP3Frame = 0; iMP3Frame < MA_DR_MP3_SEEK_LEADING_MP3_FRAMES+1; ++iMP3Frame) {
95565 |             ma_uint32 pcmFramesInCurrentMP3FrameIn;
95566 |             MA_DR_MP3_ASSERT(pMP3->streamCursor >= pMP3->dataSize);
95567 |             mp3FrameInfo[iMP3Frame].bytePos = pMP3->streamCursor - pMP3->dataSize;
95568 |             mp3FrameInfo[iMP3Frame].pcmFrameIndex = runningPCMFrameCount;
95569 |             pcmFramesInCurrentMP3FrameIn = ma_dr_mp3_decode_next_frame_ex(pMP3, NULL, NULL, NULL);
95570 |             if (pcmFramesInCurrentMP3FrameIn == 0) {
95571 |                 return MA_FALSE;
95572 |             }
95573 |             ma_dr_mp3__accumulate_running_pcm_frame_count(pMP3, pcmFramesInCurrentMP3FrameIn,
> &runningPCMFrameCount, &runningPCMFrameCountFractionalPart);
95574 |         }
95575 |         nextTargetPCMFrame = 0;
95576 |         for (iSeekPoint = 0; iSeekPoint < seekPointCount; ++iSeekPoint) {
95577 |             nextTargetPCMFrame += pcmFramesBetweenSeekPoints;
95578 |             for (;;) {
95579 |                 if (nextTargetPCMFrame < runningPCMFrameCount) {
95580 |                     pSeekPoints[iSeekPoint].seekPosInBytes = mp3FrameInfo[0].bytePos;
95581 |                     pSeekPoints[iSeekPoint].pcmFrameIndex = nextTargetPCMFrame;
95582 |                     pSeekPoints[iSeekPoint].mp3FramesToDiscard = MA_DR_MP3_SEEK_LEADING_MP3_FRAMES;
95583 |                     pSeekPoints[iSeekPoint].pcmFramesToDiscard = (ma_uint16)(nextTargetPCMFrame -
> mp3FrameInfo[MA_DR_MP3_SEEK_LEADING_MP3_FRAMES-1].pcmFrameIndex);
95584 |                     break;
95585 |                 } else {
95586 |                     size_t i;
95587 |                     ma_uint32 pcmFramesInCurrentMP3FrameIn;
95588 |                     for (i = 0; i < MA_DR_MP3_COUNTOF(mp3FrameInfo)-1; ++i) {
95589 |                         mp3FrameInfo[i] = mp3FrameInfo[i+1];
95590 |                     }
95591 |                     mp3FrameInfo[MA_DR_MP3_COUNTOF(mp3FrameInfo)-1].bytePos =
> pMP3->streamCursor - pMP3->dataSize;
95592 |                     mp3FrameInfo[MA_DR_MP3_COUNTOF(mp3FrameInfo)-1].pcmFrameIndex =
> runningPCMFrameCount;
95593 |                     pcmFramesInCurrentMP3FrameIn = ma_dr_mp3_decode_next_frame_ex(pMP3, NULL, NULL,
> NULL);
95594 |                     if (pcmFramesInCurrentMP3FrameIn == 0) {
95595 |                         pSeekPoints[iSeekPoint].seekPosInBytes = mp3FrameInfo[0].bytePos;
95596 |                         pSeekPoints[iSeekPoint].pcmFrameIndex = nextTargetPCMFrame;
95597 |                         pSeekPoints[iSeekPoint].mp3FramesToDiscard =
> MA_DR_MP3_SEEK_LEADING_MP3_FRAMES;
95598 |                         pSeekPoints[iSeekPoint].pcmFramesToDiscard = (ma_uint16)(nextTargetPCMFrame
> - mp3FrameInfo[MA_DR_MP3_SEEK_LEADING_MP3_FRAMES-1].pcmFrameIndex);
95599 |                         break;
95600 |                     }
95601 |                     ma_dr_mp3__accumulate_running_pcm_frame_count(pMP3,
> pcmFramesInCurrentMP3FrameIn, &runningPCMFrameCount, &runningPCMFrameCountFractionalPart);
95602 |                 }
95603 |             }
95604 |         }
95605 |         if (!ma_dr_mp3_seek_to_start_of_stream(pMP3)) {
95606 |             return MA_FALSE;
95607 |         }
95608 |         if (!ma_dr_mp3_seek_to_pcm_frame(pMP3, currentPCMFrame)) {
95609 |             return MA_FALSE;
95610 |         }
95611 |     }
95612 |     *pSeekPointCount = seekPointCount;
95613 |     return MA_TRUE;
95614 | }
95615 | MA_API ma_bool32 ma_dr_mp3_bind_seek_table(ma_dr_mp3* pMP3, ma_uint32 seekPointCount,
> ma_dr_mp3_seek_point* pSeekPoints)
95616 | {
95617 |     if (pMP3 == NULL) {
95618 |         return MA_FALSE;
95619 |     }
95620 |     if (seekPointCount == 0 || pSeekPoints == NULL) {
95621 |         pMP3->seekPointCount = 0;
95622 |         pMP3->pSeekPoints = NULL;
95623 |     } else {
95624 |         pMP3->seekPointCount = seekPointCount;
95625 |         pMP3->pSeekPoints = pSeekPoints;
95626 |     }
95627 |     return MA_TRUE;
95628 | }
95629 | static float* ma_dr_mp3__full_read_and_close_f32(ma_dr_mp3* pMP3, ma_dr_mp3_config* pConfig,

```

```

> ma_uint64* pTotalFrameCount)
95630 | {
95631 |     ma_uint64 totalFramesRead = 0;
95632 |     ma_uint64 framesCapacity = 0;
95633 |     float* pFrames = NULL;
95634 |     float temp[4096];
95635 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95636 |     for (;;) {
95637 |         ma_uint64 framesToReadRightNow = MA_DR_MP3_COUNTOF(temp) / pMP3->channels;
95638 |         ma_uint64 framesJustRead = ma_dr_mp3_read_pcm_frames_f32(pMP3, framesToReadRightNow, temp);
95639 |         if (framesJustRead == 0) {
95640 |             break;
95641 |         }
95642 |         if (framesCapacity < totalFramesRead + framesJustRead) {
95643 |             ma_uint64 oldFramesBufferSize;
95644 |             ma_uint64 newFramesBufferSize;
95645 |             ma_uint64 newFramesCap;
95646 |             float* pNewFrames;
95647 |             newFramesCap = framesCapacity * 2;
95648 |             if (newFramesCap < totalFramesRead + framesJustRead) {
95649 |                 newFramesCap = totalFramesRead + framesJustRead;
95650 |             }
95651 |             oldFramesBufferSize = framesCapacity * pMP3->channels * sizeof(float);
95652 |             newFramesBufferSize = newFramesCap * pMP3->channels * sizeof(float);
95653 |             if (newFramesBufferSize > (ma_uint64)MA_SIZE_MAX) {
95654 |                 break;
95655 |             }
95656 |             pNewFrames = (float*)ma_dr_mp3__realloc_from_callbacks(pFrames,
> (size_t)newFramesBufferSize, (size_t)oldFramesBufferSize, &pMP3->allocationCallbacks);
95657 |             if (pNewFrames == NULL) {
95658 |                 ma_dr_mp3__free_from_callbacks(pFrames, &pMP3->allocationCallbacks);
95659 |                 pFrames = NULL;
95660 |                 totalFramesRead = 0;
95661 |                 break;
95662 |             }
95663 |             pFrames = pNewFrames;
95664 |             framesCapacity = newFramesCap;
95665 |         }
95666 |         MA_DR_MP3_COPY_MEMORY(pFrames + totalFramesRead*pMP3->channels, temp,
> (size_t)(framesJustRead*pMP3->channels*sizeof(float)));
95667 |         totalFramesRead += framesJustRead;
95668 |         if (framesJustRead != framesToReadRightNow) {
95669 |             break;
95670 |         }
95671 |     }
95672 |     if (pConfig != NULL) {
95673 |         pConfig->channels = pMP3->channels;
95674 |         pConfig->sampleRate = pMP3->sampleRate;
95675 |     }
95676 |     ma_dr_mp3_uninit(pMP3);
95677 |     if (pTotalFrameCount) {
95678 |         *pTotalFrameCount = totalFramesRead;
95679 |     }
95680 |     return pFrames;
95681 | }
95682 | static ma_int16* ma_dr_mp3__full_read_and_close_s16(ma_dr_mp3* pMP3, ma_dr_mp3_config* pConfig,
> ma_uint64* pTotalFrameCount)
95683 | {
95684 |     ma_uint64 totalFramesRead = 0;
95685 |     ma_uint64 framesCapacity = 0;
95686 |     ma_int16* pFrames = NULL;
95687 |     ma_int16 temp[4096];
95688 |     MA_DR_MP3_ASSERT(pMP3 != NULL);
95689 |     for (;;) {
95690 |         ma_uint64 framesToReadRightNow = MA_DR_MP3_COUNTOF(temp) / pMP3->channels;
95691 |         ma_uint64 framesJustRead = ma_dr_mp3_read_pcm_frames_s16(pMP3, framesToReadRightNow, temp);
95692 |         if (framesJustRead == 0) {
95693 |             break;
95694 |         }
95695 |         if (framesCapacity < totalFramesRead + framesJustRead) {
95696 |             ma_uint64 newFramesBufferSize;
95697 |             ma_uint64 oldFramesBufferSize;
95698 |             ma_uint64 newFramesCap;
95699 |             ma_int16* pNewFrames;
95700 |             newFramesCap = framesCapacity * 2;
95701 |             if (newFramesCap < totalFramesRead + framesJustRead) {
95702 |                 newFramesCap = totalFramesRead + framesJustRead;
95703 |             }
95704 |             oldFramesBufferSize = framesCapacity * pMP3->channels * sizeof(ma_int16);

```

```

95705 |         newFramesBufferSize = newFramesCap * pMP3->channels * sizeof(ma_int16);
95706 |         if (newFramesBufferSize > (ma_uint64)MA_SIZE_MAX) {
95707 |             break;
95708 |         }
95709 |         pNewFrames = (ma_int16*)ma_dr_mp3__realloc_from_callbacks(pFrames,
> (size_t)newFramesBufferSize, (size_t)oldFramesBufferSize, &pMP3->allocationCallbacks);
95710 |         if (pNewFrames == NULL) {
95711 |             ma_dr_mp3__free_from_callbacks(pFrames, &pMP3->allocationCallbacks);
95712 |             pFrames = NULL;
95713 |             totalFramesRead = 0;
95714 |             break;
95715 |         }
95716 |         pFrames = pNewFrames;
95717 |         framesCapacity = newFramesCap;
95718 |     }
95719 |     MA_DR_MP3_COPY_MEMORY(pFrames + totalFramesRead*pMP3->channels, temp,
> (size_t)(framesJustRead*pMP3->channels*sizeof(ma_int16)));
95720 |     totalFramesRead += framesJustRead;
95721 |     if (framesJustRead != framesToReadRightNow) {
95722 |         break;
95723 |     }
95724 | }
95725 | if (pConfig != NULL) {
95726 |     pConfig->channels = pMP3->channels;
95727 |     pConfig->sampleRate = pMP3->sampleRate;
95728 | }
95729 | ma_dr_mp3__uninit(pMP3);
95730 | if (pTotalFrameCount) {
95731 |     *pTotalFrameCount = totalFramesRead;
95732 | }
95733 | return pFrames;
95734 | }
95735 | MA_API float* ma_dr_mp3_open_and_read_pcm_frames_f32(ma_dr_mp3_read_proc onRead, ma_dr_mp3_seek_proc
> onSeek, ma_dr_mp3_tell_proc onTell, void* pUserData, ma_dr_mp3_config* pConfig, ma_uint64*
> pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks)
95736 | {
95737 |     ma_dr_mp3 mp3;
95738 |     if (!ma_dr_mp3_init(&mp3, onRead, onSeek, onTell, NULL, pUserData, pAllocationCallbacks)) {
95739 |         return NULL;
95740 |     }
95741 |     return ma_dr_mp3__full_read_and_close_f32(&mp3, pConfig, pTotalFrameCount);
95742 | }
95743 | MA_API ma_int16* ma_dr_mp3_open_and_read_pcm_frames_s16(ma_dr_mp3_read_proc onRead,
> ma_dr_mp3_seek_proc onSeek, ma_dr_mp3_tell_proc onTell, void* pUserData, ma_dr_mp3_config* pConfig,
> ma_uint64* pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks)
95744 | {
95745 |     ma_dr_mp3 mp3;
95746 |     if (!ma_dr_mp3_init(&mp3, onRead, onSeek, onTell, NULL, pUserData, pAllocationCallbacks)) {
95747 |         return NULL;
95748 |     }
95749 |     return ma_dr_mp3__full_read_and_close_s16(&mp3, pConfig, pTotalFrameCount);
95750 | }
95751 | MA_API float* ma_dr_mp3_open_memory_and_read_pcm_frames_f32(const void* pData, size_t dataSize,
> ma_dr_mp3_config* pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
95752 | {
95753 |     ma_dr_mp3 mp3;
95754 |     if (!ma_dr_mp3_init_memory(&mp3, pData, dataSize, pAllocationCallbacks)) {
95755 |         return NULL;
95756 |     }
95757 |     return ma_dr_mp3__full_read_and_close_f32(&mp3, pConfig, pTotalFrameCount);
95758 | }
95759 | MA_API ma_int16* ma_dr_mp3_open_memory_and_read_pcm_frames_s16(const void* pData, size_t dataSize,
> ma_dr_mp3_config* pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks*
> pAllocationCallbacks)
95760 | {
95761 |     ma_dr_mp3 mp3;
95762 |     if (!ma_dr_mp3_init_memory(&mp3, pData, dataSize, pAllocationCallbacks)) {
95763 |         return NULL;
95764 |     }
95765 |     return ma_dr_mp3__full_read_and_close_s16(&mp3, pConfig, pTotalFrameCount);
95766 | }
95767 | #ifndef MA_DR_MP3_NO_STDIO
95768 | MA_API float* ma_dr_mp3_open_file_and_read_pcm_frames_f32(const char* filePath, ma_dr_mp3_config*
> pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks)
95769 | {
95770 |     ma_dr_mp3 mp3;
95771 |     if (!ma_dr_mp3_init_file(&mp3, filePath, pAllocationCallbacks)) {
95772 |         return NULL;

```



```

95773 |     }
95774 |     return ma_dr_mp3__full_read_and_close_f32(&mp3, pConfig, pTotalFrameCount);
95775 | }
95776 | MA_API ma_int16* ma_dr_mp3_open_file_and_read_pcm_frames_s16(const char* filePath, ma_dr_mp3_config*
> pConfig, ma_uint64* pTotalFrameCount, const ma_allocation_callbacks* pAllocationCallbacks)
95777 | {
95778 |     ma_dr_mp3 mp3;
95779 |     if (!ma_dr_mp3_init_file(&mp3, filePath, pAllocationCallbacks)) {
95780 |         return NULL;
95781 |     }
95782 |     return ma_dr_mp3__full_read_and_close_s16(&mp3, pConfig, pTotalFrameCount);
95783 | }
95784 | #endif
95785 | MA_API void* ma_dr_mp3_malloc(size_t sz, const ma_allocation_callbacks* pAllocationCallbacks)
95786 | {
95787 |     if (pAllocationCallbacks != NULL) {
95788 |         return ma_dr_mp3__malloc_from_callbacks(sz, pAllocationCallbacks);
95789 |     } else {
95790 |         return ma_dr_mp3__malloc_default(sz, NULL);
95791 |     }
95792 | }
95793 | MA_API void ma_dr_mp3_free(void* p, const ma_allocation_callbacks* pAllocationCallbacks)
95794 | {
95795 |     if (pAllocationCallbacks != NULL) {
95796 |         ma_dr_mp3__free_from_callbacks(p, pAllocationCallbacks);
95797 |     } else {
95798 |         ma_dr_mp3__free_default(p, NULL);
95799 |     }
95800 | }
95801 | #endif
95802 | /* dr_mp3_c end */
95803 | #endif /* MA_DR_MP3_IMPLEMENTATION */
95804 | #endif /* MA_NO_MP3 */
95805 |
95806 |
95807 | /* End globally disabled warnings. */
95808 | #if defined(_MSC_VER)
95809 |     #pragma warning(pop)
95810 | #endif
95811 |
95812 | #endif /* miniaudio_c */
95813 | #endif /* MINIAUDIO_IMPLEMENTATION */
95814 |
95815 |
95816 | /*
95817 | This software is available as a choice of the following licenses. Choose
95818 | whichever you prefer.
95819 |
95820 | =====
95821 | ALTERNATIVE 1 - Public Domain (www.unlicense.org)
95822 | =====
95823 | This is free and unencumbered software released into the public domain.
95824 |
95825 | Anyone is free to copy, modify, publish, use, compile, sell, or distribute this
95826 | software, either in source code form or as a compiled binary, for any purpose,
95827 | commercial or non-commercial, and by any means.
95828 |
95829 | In jurisdictions that recognize copyright laws, the author or authors of this
95830 | software dedicate any and all copyright interest in the software to the public
95831 | domain. We make this dedication for the benefit of the public at large and to
95832 | the detriment of our heirs and successors. We intend this dedication to be an
95833 | overt act of relinquishment in perpetuity of all present and future rights to
95834 | this software under copyright law.
95835 |
95836 | THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
95837 | IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
95838 | FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
95839 | AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN
95840 | ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
95841 | WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
95842 |
95843 | For more information, please refer to <http://unlicense.org/>
95844 |
95845 | =====
95846 | ALTERNATIVE 2 - MIT No Attribution
95847 | =====
95848 | Copyright 2026 David Reid
95849 |
95850 | Permission is hereby granted, free of charge, to any person obtaining a copy of

```

```
95851 | this software and associated documentation files (the "Software"), to deal in
95852 | the Software without restriction, including without limitation the rights to
95853 | use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies
95854 | of the Software, and to permit persons to whom the Software is furnished to do
95855 | so.
95856 |
95857 | THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
95858 | IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
95859 | FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
95860 | AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
95861 | LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
95862 | OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
95863 | SOFTWARE.
95864 | */
```